

INTRODUCTION TO
JAVATM
PROGRAMMING

BRIEF VERSION



9TH EDITION



Y. Daniel Liang

ÇEVİRİMİÇİ ERİŞİM

Yeni bir kopya satın aldığınız için teşekkür ederiz *Java Programlamaya Giriş, Kısa Versiyon*, Dokuzuncu Baskı. Ders kitabınız, kitabın Eşlikçi Web Sitesine altı ay süreyle önden ödemeli erişim içermektedir. Bu önden ödemeli abonelik, aşağıdaki öğrenci destek alanlarına tam erişim sağlar:

VideoNotları, bu ders kitabında sunulan programlama kavramlarını geliştirmek için özel olarak tasarlanmış adım adım video eğitimleridir.

Kazınabilir kaplamayı kazımak için bir madeni para kullanın ve öğrenci erişim kodunuzu ortaya çıkarın.

Kodu zarar verebileceğinden dolayı bir bıçak veya başka bir keskin nesne kullanmayın.

İlk kez *Java Programlamaya Giriş, Kısa Versiyon*, Dokuzuncu Baskı, Eşlikçi Web Sitesine erişmek için, bir internet bağlantısına ve bir web tarayıcısına sahip bir bilgisayar kullanarak çevrimiçi kayıt olmanız gerekecektir. Bu işlem sadece birkaç dakika sürer ve yalnızca bir kez tamamlanması gerekir.

1. <http://www.pearsonhighered.com/liang/> adresine gidin
2. Eşlikçi Web Sitesi'ne tıklayın.
3. Kayıt butonuna tıklayın.
4. Kayıt sayfasında, kazınabilir panelin altında bulunan öğrenci erişim kodunuzu* girin . Tireleri yazmayın. Küçük veya büyük harf kullanabilirsiniz.
5. Ekrandaki talimatları izleyin. Çevrimiçi kayıt süreci sırasında herhangi bir zamanda yardıma ihtiyacınız olursa, sadece Yardıma mı ihtiyacınız var? simgesine tıklayın.
6. Kişisel Giriş Adınızı ve Parolanızı onaylandıktan sonra, kullanmaya başlayabilirsiniz *Java Programlamaya Giriş, Kısa Versiyon* Eşlikçi Web Sitesi!

Kayıt olduktan sonra giriş yapmak için:

Bu Eşlikçi Web Sitesi için yalnızca bir kez kayıt olmanız gerekir. Bunun ardından, her zaman time'da <http://www.pearsonhighered.com/liang/> Giriş Adınızı ve

Parolanızı sağladığınızda giriş yapabilirsiniz.

*Önemli: Erişim kodu yalnızca bir kez kullanılabilir. Bu abonelik, etkinleştirildikten sonra altı ay boyunca geçerlidir ve devredilemez. Bu erişim kodu zaten açığa çıkarıldıysa, artık geçerli olmayabilir. Bu durumda, bir abonelik satın alabilirsiniz <http://www.pearsonhighered.com/liang/> adresine giderek ve ekrandaki talimatları izleyerek.



programlamaya katıl

Uygulama gücü ve anında kişiselleştirilmiş
geri bildirim sayesinde, MyProgrammingLab performansınızı artırır.

MytProgrammingtLab

Daha fazla bilgi için www.myprogramminglab.com adresini ziyaret edin

Bu sayfa kasıtlı olarak boş bırakılmıştır

GİRİŞ

JAVAtTM

PROGRAMLAMA

KISA VERSİYON

Dokuzuncu Baskı

Y. Daniel Liang

Armstrong Atlantik Eyalet Üniversitesi

PEARSON

Boston Columbus Indianapolis New York San Francisco Upper Saddle River nAmst
erdam Cape Town Dubai Londra Madrid Milan Münih Paris Montreal Toronto nDelhi Mexico Cit
y Sao Paulo Sidney Hong Kong Seul Singapur Taipei Tokyo

Yazı İşleri Direktörü: Marcia Horton
Baş Editör: Michael Hirsch
İcra Editörü: Tracy Dunkelberger
Yardımcı Editör: Carole Snyder
Pazarlama Direktörü: Patrice Jones
Pazarlama Müdürü: Yez Alayan
Pazarlama Koordinatörü: Kathryn Ferranti
Pazarlama Asistanı: Emma Snider
Üretim Direktörü: Vince O'Brien
Yönetici Editör: Jeff Holcomb
Üretim Proje Yöneticisi: Kayla Smith-Tarbox
Operasyonlar Süpervizörü: Alan Fischer
Üretim Alımcısı: Lisa McDowell

Sanat Yönetmeni: Anthony Gemmellaro
Kapak Tasarımcısı: Anthony Gemmellaro
Görsel Araştırma Müdürü: Karen Sanatar
Haklar ve İzinler Müdürü: Mike Joyce
Metin İzin Koordinatörü: Danielle Simon
v ve Jenn Kennett
Kapak İllüstrasyonu: Jason Consalvo
Ana Medya Proje Yöneticisi: Daniel Sandin
Proje Yönetimi: Gillian Hall
Kompozisyon ve Sanat: Laserwords
Baskı/Bantlama: Edwards Brothers
Kapak Baskıcısı: Lehigh-Phoenix Color/Hagerstown
Metin Fontu: Times 10/12

Bu ders kitabında diğer kaynaklardan alınan ve izinle yeniden üretilen kredi ve teşekkürler, metin içinde uygun sayfada ve şu şekilde görünmektedir: Tablo 3.2 ve 10.1: IRS'ten alınan veriler. Şekil 8.1, 8.12, 12.3, 12.5, 12.7, 12.9, 12.10, 12.12–12.21, 12.26–12.30, 13.1, 13.4, 13.9, 13.11, 13.15, 13.17, 13.19, 13.21, 13.23, 13.25–13.35, 14.10, 14.14, 15.9–15.11, 16.1, 16.2, 16.8, 16.11, 16.14, 16.17, 16.19–16.35, 17.1, 17.3, 17.6, 17.9, 17.12, 17.13, 17.15, 17.17–17.32, 18.6–18.8, 18.10, 18.15–18.35, 19.19, 19.20, 19.22, 20.1, 20.9, 20.12–20.14, 20.16–20.20: Ekran görüntüleri © 2011 Oracle Corporation. İzinle yeniden basılmıştır.

Microsoft® ve Windows® Amerika Birleşik Devletleri'nde ve diğer ülkelerde Microsoft Corporation'ın tescilli ticari markalarıdır. Ekran görüntüleri ve simgeler Microsoft Corporation'dan izin alınarak yeniden basılmıştır. Bu kitap Microsoft Corporation tarafından desteklenme mekte, onaylanmamış veya onunla bağlantılı değildir.

Telif Hakkı © 2013, 2011, 2009, 2007, 2004 Pearson Education, Inc. tarafından, Prentice Hall olarak yayımlanmıştır. Tüm hakları saklıdır. Amerika Birleşik Devletleri'nde üretilmiştir. Bu yayın Telif Hakkı ile korunmaktadır ve e yasaklanmış çoğaltma, bir geri alma sisteminde depolama veya herhangi bir formda veya herhangi bir yöntemle, elektronik, mekanik, fotokopi, kayıt veya benzeri ile iletim öncesinde yayınevinden izin alınmalıdır. Bu çalışmadan materyal kullanmak için izin almak üzere, lütfen Pearson Education, Inc.'e yazılı bir talep gönderin, İzinler Departmanı, One Lake Street, Upper Saddle River, New Jersey 07458, veya talebinizi 201-236-3290 numaralı faksla gönderebilirsiniz.

Üreticiler ve satıcılar tarafından ürünlerini ayırt etmek için kullanılan birçok tanım ticari marka olarak talep edilmektedir. Bu kitapta bu tanımlar yer aldığı ve yayınevi bir ticari marka talebinden haberdar olduğunda, tanımlar büyük harfle ve eya tamamen büyük harfle basılmıştır.

Talep üzerine Kongre Kütüphanesi Yayın Kataloğu Verisi mevcuttur.

Prentice Hall
is an imprint of



www.pearsonhighered.com

10 9 8 7 6 5 4 3 2 1

ISBN 13: 978-0-13-292373-6 ISBN 10: 0-13-292373-4

Bu kitap Profesör Myers Foreman'a adanmıştır. Myers, Lamar Üniversitesi'nde CS1, CS2 ve CS3 derslerinde bu kitabın kapsamlı versiyonunu kullandı ve kitabın geliştirilmesi için paha biçilmez önerilerde bulundu. Ne yazık ki, Myers bu baskının incelemesini tamamladıktan sonra vefat etti.

Samantha, Michael ve Michelle'e

Bu sayfa kasıtlı olarak boş bırakılmıştır

ÖNSÖZ

Sevgili Okuyucu,

Birçoğunuz bu kitabın önceki baskıları hakkında geri bildirimde bulundunuz ve yorumlarınız ve önerileriniz kitabı büyük ölçüde geliştirdi. Bu baskı, sunum, organizasyon, örnekler, alıştırmalar ve ekler açısından önemli ölçüde geliştirilmiştir. Biz:

yeni olan nedir?

■Konu başlıklarını daha mantıklı bir sırayla sunmak için bölümleri ve bölümleri yeniden düzenledik

■İlgi uyandırmak için birçok yeni ilginç örnek ve alıştırma ekledik

■Java 7'ye güncellendi

■Kavramları görsel olarak göstermek için algoritmalar ve veri yapıları için animasyonlar oluşturduk

■Destek Web Sitesini daha kolay gezinilebilir hale getirmek için yenide

n tasarlandıBu kitap, sözdizimi yerine problem çözmeye odaklanan, problem odaklı bir şekilde programlama öğretmektedir.Giriş niteliğindeki programlamayı, geniş bir bağlamda düşündürücü problemler kullanarak ilginç hale getiriyoruz. İlk bölümlerin merkezi teması problem çözme üzerinedir. Okuyucuların sorunları çözmek için programlar yazabilmelerini sağlamak amacıyla uygun sözdizimi ve kütüphane tanıtılmaktadır. Problemlere dayalı bir şekilde programlama öğretimini desteklemek için kitap, öğrencileri motive etmek amacıyla çeşitli zorluk seviyelerinde geniş bir problem yelpazesi sunmaktadır. Tüm bölümlerdeki öğrencilere hitap etmek için problemler, matematik, bilim, iş, finans, oyun, animasyon ve multimedya gibi birçok uygulama alanını kapsamaktadır.

problem odaklı

Kitap, özel sınıflar tasarlamadan önce temel programlama kavramları ve tekniklerini tanıtarak öncelikle temellere odaklanmaktadır. Döngüler, yöntemler ve dizilerin temel kavramları ve teknikleri programlamanın temelini oluşturur. Bu sağlam temeli oluşturmak, öğrencilerin nesne yönelimli programlama ve ileri düzey Java programlamayı öğrenmelerine hazırlık yapar.

temel ilkeler öncelikli

Bu kitap, Introduction to Java Programming, Comprehensive Version, 9E'nin kısa versiyonudur. Bu versiyon, genellikle CS1 olarak bilinen bir giriş programlama dersi için tasarlanmıştır. Bu versiyon, kapsamlı versiyondaki ilk yirmi bölümü içermektedir. Kısa versiyon, programlamanın temellerini, nesne yönelimli programlamayı, GUI programlamayı, istisna yönetimini, G/Ç ve özyinelemeyi kapsamaktadır. Kapsamlı versiyon, veri yapıları ve algoritmalar, iş parçacıkları, paralel programlama, ağ oluşturma, uluslararasılaştırma, ileri düzey GUI, veritabanı ve Web programlamayı kapsayan ek otuz bölüm içermektedir.

kısa versiyon
kapsamlı versiyon

Programlama öğretmenin en iyi yolu *örneklerle* ve programlamayı öğrenmenin tek yolu pratik yapmaktır. Temel kavramlar örneklerle açıklanır ve öğrencilerin pratik yapması için çeşitli zorluk seviyelerinde çok sayıda alıştırma sağlanır. Programlama derslerimiz için, her dersten sonra programlama alıştırmaları veriyoruz.

örnekler ve alıştırmalar

Amacımız, problem çözmeyi ve programlamayı geniş bir

kontekste, ilginç örneklerin geniş bir yelpazesi kullanarak öğretmektir. Kitabı geliştirmek için herhangi bir yorumunuz ve öneriniz varsa, lütfen bana e-posta gönderin.

Saygılarımla,

Y. Daniel Liang

Evet.Daniel.two@Gmail.com

W.Su.anda.Armstrong.Quota/iki.yil

WWW.Pearson.daha.yüksek.Ed.com/two

Bu Baskıda Neler Yeni?

Bu baskı, *Java Programlamaya Giriş*, Sekizinci Baskı üzerinde önemli iyileştirmeler yapmaktadır. Ana iyileştirmeler şunlardır:

tam revizyon	■Bu baskı, açıklığı, sunumu, içeriği, örnekleri ve alıştırmaları artırmak için her detayda tamamen gözden geçirilmiştir.
yeni problemler	■Programlamaya öğrenci ilgisini artırmak ve teşvik etmek için yeni örnekler ve alıştırmalar sağlanmıştır.
ana nokta	■Her bölüm, bölümde ele alınan önemli kavramları vurgulayan bir Ana Nokta ile başlamaktadır.
kontrol noktası	■Kontrol Noktaları, öğrencilerin ilerlemelerini takip etmelerine ve önemli bir kavram veya örnek ele alındıktan sonra öğrenmelerini değerlendirmelerine yardımcı olmak için gözden geçirme soruları sağlar.
test soruları	■Her bölüm çevrimiçi test soruları sunar. Öğrencilerin kendi kendine test yapabilmeleri için bölümler halinde gruplandırılmıştır. Sorular çevrimiçi olarak notlandırılır.
VideoNotları	■Yeni VideoNotları, kodu pekiştirmek için tasarlanmış kısa video eğitimleri sağlar.
erken temel GUI ve grafikler	■Java GUI API, nesne yönelimli ilkenin nasıl uygulandığının mükemmel bir örneğidir. Öğrenciler somut ve görsel örneklerle daha iyi öğrenirler. Bu nedenle, temel GUI/Grafikler, soyut sınıflar ve arayüzler tanıtılmadan önce ele alınmıştır. Ancak, GUI'yi ele almayı atlayabilir veya soyut sınıflar ve arayüzleri GUI'den önce ele almayı seçebilirsiniz.
erken kapsanan sayısal sınıflar	■Sayısal sarmalayıcı sınıflar, BigInteger ve BigDecimal artık öğrencilerin bu sınıfları erken kullanarak kod yazmalarını sağlamak için Bölüm 10'da tanıtılmaktadır.
erken istisna yönetimi	■İstisna yönetimi, öğrencilerin erken sağlam programlar geliştirebilmeleri için soyut sınıflar ve arayüzlerden önce ele alınmaktadır. Eğitimci, istisna yönetimini daha sonra ele almayı da seçebilir. Metin G/Ç artık istisna yönetimi ile birleştirilerek yeni bir bölüm oluşturmuştur.
basit jenerikler erken	■Basit jenerik kullanımı, Bölüm 11'de ArrayList ile ve Bölüm 15'te Comparable ile tanıtılmakta, jeneriklerle ilgili karmaşık detaylar ise kitabın kapsamlı versiyonunda sunulmaktadır.

Lütfen www.cs.armstrong.edu/liang/intro9e/newfeatures.html adresini ziyaret edin, yeni özelliklerin tam listesini ve önceki baskıyla olan ilişkilerini bulabilirsiniz.

Eğitimsel Özellikler

Kitap, öğrencilerin materyalden en iyi şekilde yararlanmalarına yardımcı olmak için aşağıdaki unsurları kullanmaktadır:

- Her bölümün başındaki Hedefler, öğrencilerin bölümden ne öğrenmeleri gerektiğini listelemektedir. Bu, bölüm tamamlandıktan sonra hedeflere ulaşmış ve ulaşmadıklarını belirlemelerine yardımcı olacaktır.
- Giriş, okuyucuya bölümden ne beklemesi gerektiğine dair bir genel bakış sunmak için temsili problemlerle tartışmayı başlatmaktadır.
- Ana Noktalar, her bölümde ele alınan önemli kavramları vurgulamaktadır.
- Kontrol Noktaları, öğrencilerin bölüm boyunca okurken ilerlemelerini takip etmelerine ve öğrenmelerini değerlendirmelerine yardımcı olmak için gözden geçirme soruları sunmaktadır.
- Problemler ve Vaka Çalışmaları, dikkatlice seçilmiş ve takip edilmesi kolay bir tarzda sunulmuş, problem çözme ve programlama kavramlarını öğretmektedir. Kitap, önemli fikirleri göstermek için birçok küçük, basit ve teşvik edici örnek kullanmaktadır.

■ Bölüm Özeti, öğrencilerin anlaması ve hatırlaması gereken önemli konuları gözden geçirir.

Bu, onlara bölümde öğrendikleri ana kavramları pekiştirmelerine yardımcı olur.

■ Test Soruları, öğrencilerin programlama kavramları ve teknikleri üzerinde kendi kendine test yapabilmeleri için bölümler halinde çevrimiçi olarak erişilebilir.

■ Programlama Alıştırmaları, öğrencilerin öğrendikleri yeni becerileri kendi başlarına uygulama fırsatları sunmak için bölümler halinde gruplandırılmıştır. Zorluk seviyesi kolay (hiç yıldız yok), orta (*), zor (**) veya zorlayıcı (***) olarak derecelendirilmiştir. Programlama öğrenmenin püf noktası pratik, pratik ve pratik yapmaktır. Bu amaçla, kitap çok sayıda alıştırmayı sunmaktadır.

■ Notlar, İpuçları, Uyarılar ve Tasarım Rehberleri, program geliştirme ile ilgili önemli yönler hakkında değerli öneri ve içgörü sunmak için metin boyunca yerleştirilmiştir.



Not

Konu hakkında ek bilgi sağlar ve önemli kavramları pekiştirir.



İpucu

İyi programlama tarzı ve pratiği öğretir.



Dikkat

Öğrencilerin programlama hatalarının tuzaklarından uzak durmalarına yardımcı olur.



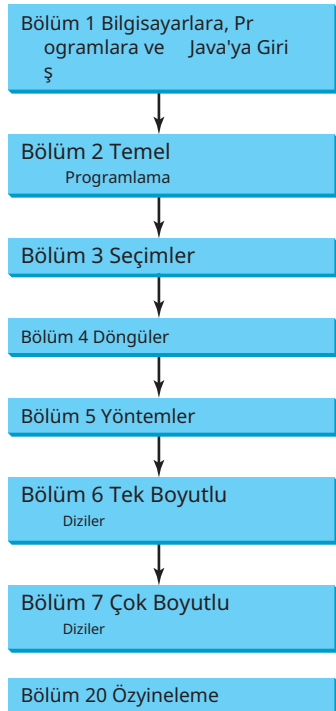
Tasarım Kılavuzu

Program tasarımı için yönergeler sağlar.

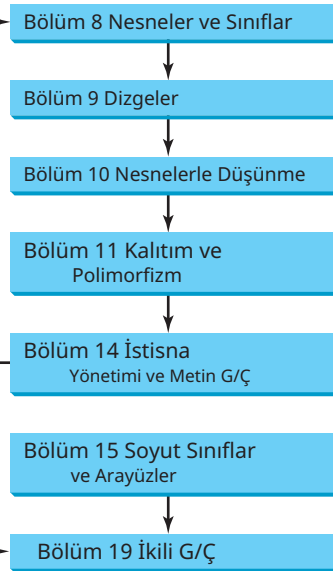
Esnek Bölüm Sıralamaları

Kitap, GUI, istisna işleme ve özyinelemeyi daha erken veya daha geç ele alabilmek için esnek bölüm sıralamaları sağlamak üzere tasarlanmıştır. Diyagram bölüm bağımlılıklarını göstermektedir.

Bölüm I: Programlamanın Temelleri



Bölüm II: Nesne Yönelimli Programlama



Bölüm III: GUI Programlama



Kitabın Organizasyonu

Kısa versiyondaki bölümler, birlikte alındığında Java programlamaya sağlam bir giriş oluşturan üç parçaya gruplandırılabilir. Bilgi birikimli olduğu için, erken bölümler programlamayı anlamak için kavramsal temeli sağlar ve öğrencileri basit örnekler ve alıştırma larla yönlendirir; sonraki bölümler Java programlamasını ayrıntılı olarak sunar ve kapsamlı Java uygulamalarının geliştirilmesiyle sonuçlanır. Ekler, sayı sistemleri ve bit düzeyindeki işlemler gibi konuları içeren karışık bir konu yelpazesini barındırmaktadır.

Bölüm I: Programlamanın Temelleri (Bölümler 1–7, 20)

Kitabın ilk kısmı, Java öğrenme yolculuğuna çıkmanız için bir basamak taşınıdır. Java hakkında (Bölüm 1) ve ilkel veri türleri, değişkenler, sabitler, atamalar, ifadeler ve operatörlerle (Bölüm 2) temel programlama tekniklerini öğrenmeye başlayacaksınız, kontrol ifadeleri (Bölümler 3–4), yöntemler (Bölüm 5) ve diziler (Bölümler 6–7). Bölüm 6'dan sonra, özünde özyinelemeli problemleri çözmek için özyinelemeli yöntemler yazmayı öğrenmek üzere Bölüm 20'ye geçebilirsiniz.

2. Kısım: Nesne Yönelimli Programlama (Bölümler 8–11, 14–15 ve 19)

Bu kısım nesne yönelimli programlamayı tanıtmaktadır. Java, yazılım geliştirmede büyük esneklik, modülerlik ve yeniden kullanılabilirlik sağlamak için soyutlama, kapsülleme, kalıtım ve çok biçimliliği kullanan bir nesne yönelimli programlama dilidir. Nesneler ve sınıflarla programlamayı (Bölümler 8–10), sınıf kalıtımını (Bölüm 11), çok biçimliliği (Bölüm 11), istisna yönetimini ve metin G/Ç'yi (Bölüm 14), soyut sınıfları (Bölüm 15) ve arayüzleri (Bölüm 15) öğreneceksiniz. Dize işleme Bölüm 9'da tanıtılmakta ve ikili G/Ç Bölüm 19'da tartışılmaktadır.

3. Kısım: GUI Programlama (Bölümler 12–13, 16–18)

Bu kısım, Bölümler 12–13 ve 16–18'de temel Java GUI programlamayı tanıtmaktadır. Ana konular arasında GUI temelleri (Bölüm 12), şekil çizimi (Bölüm 13), olay odaklı programlama (Bölüm 16), GUI bileşenlerini kullanma (Bölüm 17) ve applet yazma (Bölüm 18) bulunmaktadır. Java GUI programlamanın mimarisini öğrenecek ve bu temel GUI bölümlerinden uygulamalar ve appletler geliştirmek için GUI bileşenlerini kullanacaksınız.

Ekler

Bu kitabın bu kısmı çeşitli konuları kapsamaktadır. Ek A, Java anahtar kelimelerini listelemektedir. Ek B, ASCII karakterlerinin ve bunların ondalık ve onaltılık kodlarının tablolarını verir. Ek C, operatör önceliğini gösterir. Ek D, Java dönüştüricilerini ve bunların kullanımını özetler. Ek E, özel kayan nokta değerlerini tartışır. Ek F, sayı sistemlerini ve ikili, ondalık ve onaltılık sayılar arasındaki dönüşümleri tanıtır. Son olarak, Ek G bit düzeyinde işlemleri tanıtır.

Java Geliştirme Araçları

Java programları oluşturmak ve komut penceresinden programları derleyip çalıştırmak için Windows Notepad veya WordPad gibi bir metin düzenleyici kullanabilirsiniz. Ayrıca, TextPad, NetBeans veya Eclipse gibi bir Java geliştirme aracı da kullanabilirsiniz. Bu araçlar, Java programlarını hızlı bir şekilde geliştirmek için entegre bir geliştirme ortamı (IDE) destekler. Programları düzenleme, derleme, oluşturma, yürütme ve hata ayıklama, tek bir grafik kullanıcı arayüzünde entegre edilmiştir.

Bu araçları etkili bir şekilde kullanmak, programlama verimliliğinizi büyük ölçüde artırabilir. TextPad, ilkel bir IDE aracıdır. NetBeans ve Eclipse daha sofistike olup, eğer eğitimleri takip ederseniz kullanımı kolaydır. TextPad, NetBeans ve Eclipse ile ilgili eğitimler, Eşlikçi Web Sitesindeki eklerde bulunabilir www.cs.armstrong.edu/liang/intro9e.

MyProgrammingLab ile Çevrimiç i Pratik ve Değerlendirme

MyProgrammingLab™

MyProgrammingLab, öğrencilerin programlama mantığını, anlamını ve sözdizimini tam olarak kavramalarına yardımcı olur. Pratik alıştırmalar ve anında, kişiselleştirilmiş geri bildirim aracılığıyla, MyProgrammingLab, genellikle popüler yüksek seviyeli programlama dillerinin temel kavramları ve paradigmalarıyla mücadele eden başlangıç düzeyindeki öğrencilerin programlama yeterliliklerini artırır.

Kendi kendine çalışma ve ödev aracı olan bir MyProgrammingLab dersi, bu ders kitabının yapısına göre düzenlenmiş yüzlerce küçük pratik problemi içerir. Öğrenciler için sistem, kod gönderimlerinde mantık ve sözdizimindeki hataları otomatik olarak tespit eder ve öğrencilerin neyin yanlış gittiğini ve nedenini anlamalarına yardımcı olan hedeflenmiş ipuçları sunar. Öğitmenler için kapsamlı bir not defteri, doğru ve yanlış cevapları takip eder ve öğrenciler tarafından girilen kodları gözden geçirmek için saklar.

MyProgrammingLab, bu kitabın kullanıcılarına, CodeLab etkileşimli programlama alıştırma sisteminin yapımcıları Turing's Craft ile ortaklık içinde sunulmaktadır. Tam bir gösterim için, öğretmenlerden ve öğrencilerden geri bildirim görmek veya MyProgrammingLab'ı dersinizde kullanmaya başlamak için www.myprogramminglab.com adresini ziyaret edin.

VideoNotları



VideoNot

Bu yeni baskıda bulunan yeni VideoNotlar özelliği için heyecanlıyız. Bu videolar

anahtar konuların örneklerini sunarak ve sorunları tasarımdan kodlamaya kadar tamamen nasıl çözeceğimizi göstererek ek yardım sağlar.

VideoNotlar, ilk kez kullanıcılar için ücretsizdir ve bu kitabın önündeki erişim kodunu kullanarak www.pearsonhighered.com/liang adresinden erişilebilir.

Canlı Laboratuvar

Bu kitap, öğretmenler için tamamlayıcı bir Web tabanlı kurs değerlendirme ve yönetim sistemi ile birlikte gelir. Sistem dört ana bileşene sahiptir:

- Otomatik Notlandırma Sistemi programları otomatik olarak notlandırabilir.
- Sınav Oluşturma/Gönderme/Notlandırma Sistemi öğretmenlerin öğrencilerin alabileceği ve otomatik olarak notlandırılacak sınavlar oluşturup değiştirmesine olanak tanır.
- Ekran Değerlendirme Sistemi ekran değerlendirmelerine olanak tanır.
- Notları, devamsızlık vb. takip etme öğrencilerin notlarını takip etmelerini sağlar ve öğretmenlerin tüm öğrencilerin notlarını görmesine ve öğrencilerin devamsızlıklarını takip etmesine olanak tanır.

Otomatik Notlandırma Sisteminin ana özellikleri şunlardır:

- Öğrenciler alıştırmaları çalıştırabilir ve gönderebilir. (Sistem, programlarının doğru çalışıp çalışmadığını kontrol eder—öğrenciler son tarihten önce programı çalıştırmaya ve yeniden göndermeye devam edebilir.)
- Öğitmenler, gönderimleri gözden geçirebilir, öğretmen test durumları ile programları çalıştırabilir, bunları düzeltebilir, öğrencilere geri bildirim verebilir ve intihali kontrol edebilir.
- Öğitmenler, kendi alıştırmalarını oluşturabilir/değiştirebilir, kamuya açık ve gizli test durumları oluşturabilir, alıştırmaları atayabilir ve tüm sınıf veya bireyler için son tarihler belirleyebilir.
- Öğitmenler, metindeki tüm alıştırmaları öğrencilere atayabilir. Ayrıca, LiveLab metinde basılmamış ek alıştırmalar sunar.
- Öğitmenler, tüm alıştırmaları sıralayabilir ve filtreleyebilir ve notları kontrol edebilir (zaman dilimi, öğrenci, ve/veya alıştırmada).

■Eğitmenler, sistemden öğrencileri silebilir.

■Öğrenciler ve eğitmenler, alıştırmalardaki notları takip edebilir.

Quiz Sisteminin ana özellikleri şunlardır:

■Eğitmenler, test bankasından veya bir metin dosyasından quizler oluşturabilir/değiştirebilir veya tamamen yeni testler çevrimiçi oluşturabilir.

■Eğitmenler, sınavları öğrencilere atayabilir ve tüm sınıf veya bireyler için son tarih ve test süre sınırı belirleyebilir.

■Öğrenciler ve eğitmenler, gönderilen sınavları gözden geçirebilir.

■Eğitmenler, sınavları analiz edebilir ve öğrencilerin zayıf yönlerini belirleyebilir.

■Öğrenciler ve eğitmenler, sınavlardaki notları takip edebilir.

Peer Değerlendirme Sisteminin ana özellikleri şunlardır:

■Eğitmenler, programlama alıştırmaları için akran değerlendirmesi atayabilir.

■Eğitmenler, akran değerlendirme raporlarını görüntüleyebilir.

Öğrenci Kaynak Web Sitesi

Öğrenci Kaynak Web Sitesi (www.cs.armstrong.edu/liang/intro9e) aşağıdaki

kaynakları içermektedir:

■VideoNotları'na erişim (www.pearsonhighered.com/liang).

■Kontrol noktası sorularının cevapları

■Çift numaralı programlama alıştırmalarının çözümleri

■Kitaptaki örnekler için kaynak kodu

■Etkileşimli kendini test etme (her bölüm için düzenlenmiş)

■Veri yapıları ve algoritma animasyonları

■Hatalar

Öğretmen Kaynak Web Sitesi

Öğretmen Kaynak Web Sitesi, www.cs.armstrong.edu/liang/intro9e adresinden erişilebilir ve şu kaynakları içermektedir:

■Tam renkli, sözdizimi vurgulu kaynak kodunu görüntülemek ve slaytlardan çıkmadan programları çalıştırmak için etkileşimli düğmelere sahip Microsoft PowerPoint slaytları.

■Tüm programlama alıştırmalarının çözümleri. Öğrenciler, çift numaralı programlama alıştırmalarının çözümlerine erişim sağlayacaklardır.

■Web tabanlı sınav oluşturucu. (Eğitmenler, iki binden fazla sorudan oluşan büyük bir veritabanından sınav oluşturmak için bölümleri seçebilir.)

■Örnek sınavlar. Çoğu sınav dört bölümden oluşmaktadır:

■Çoktan seçmeli sorular veya kısa cevaplı sorular

■Programlama hatalarını düzeltin

■Programları izleyin

■Programlar yazın

■Projeler. Genel olarak, her proje bir tanım verir ve öğrencilerden projeyi analiz etmelerini, tasarımlarını ve uygulamalarını ister.

Bazı okuyucular, Eğitimci Kaynak Web Sitesi'ndeki materyalleri talep ettiler. Lütfen bunları yalnızca eğitimci için olduğunu anlayın. Bu tür talepler yanıtlanmayacaktır.

Algoritma Animasyonları

Algoritmalar için birçok animasyon sağladık. Bunlar, algoritmaların nasıl çalıştığını göstermek için değerli pedagojik araçlardır. Algoritma animasyonlarına Eşlikçi Web Sitesi'nden erişilebilir.

Teşekkürler

Yazdıklarımla öğretmeme olanak tanıdığı ve öğrettiklerimi yazmamda destek olduğu için Armstrong Atlantic State University'e teşekkür etmek istiyorum. Öğretim, kitabı sürekli geliştirmek için ilham kaynağıdır. Yorum, öneri, hata raporu ve övgü sunan eğitimci ve öğrencilere minnettarım.

Bu kitap, bu ve önceki baskılar için olağanüstü incelemeler sayesinde büyük ölçüde geliştirilmiştir. İncelemeleri yapanlar: Elizabeth Adams (James Madison Üniversitesi), Syed Ahmed (Kuzey Georgia Koleji ve Eyalet Üniversitesi), Omar Aldawud (Illinois Teknoloji Enstitüsü), Yang Ang (Wollongong Üniversitesi, Avustralya), Kevin Bierre (Rochester Teknoloji Enstitüsü), David Champion (DeVry Enstitüsü), James Chegwidan (Tarrant County College), Anup Dargar (Kuzey Dakota Üniversitesi), Charles Dierbach (Towson Üniversitesi), Frank Ducrest (Louisiana Lafayette Üniversitesi), Erica Eddy (Wisconsin Parkside Üniversitesi), Deena Engel (New York Üniversitesi), Henry A. Etlinger (Rochester Teknoloji Enstitüsü), James Ten Eyck (Marist Koleji), Myers Foreman (Lamar Üniversitesi), Olaf Fuentes (Texas Üniversitesi El Paso), Edward F.Gehringer (Kuzey Carolina Eyalet Üniversitesi), Harold Grossman (Clemson Üniversitesi), Barbara Guillot (Louisiana Eyalet Üniversitesi), Stuart Hansen (Wisconsin Parkside Üniversitesi), Dan Harvey (Güney Oregon Üniversitesi), Ron Hofman (Red River Koleji, Kanada), Stephen Hughes (Roanoke Koleji), Vladan Jovanovic (Georgia Southern Üniversitesi), Edwin Kay (Lehigh Üniversitesi), Larry King (Texas Üniversitesi Dallas), Nana Kofi (Langara Koleji, Kanada), George Koutsogiannakis (Illinois Teknoloji Enstitüsü), Roger Kraft (Purdue Üniversitesi Calumet), Norman Krumpke (Miami Üniversitesi), Hong Lin (DeVry Enstitüsü), Dan Lipsa (Armstrong Atlantic Eyalet Üniversitesi), James Madison (Rensselaer Politeknik Enstitüsü), Frank Malinowski (Dartmouth Koleji), Tim Margush (Akron Üniversitesi), Debbie Masada (Sun Microsystems), Blayne Mayfield (Oklahoma Eyalet Üniversitesi), John McGrath (J.P.McGrath Danışmanlığı), Hugh McGuire (Grand Valley Eyalet), Shyamal Mitra (Texas Üniversitesi Austin), Michel Mitri (James Madison Üniversitesi), Kenrick Mock (Alaska Anchorage Üniversitesi), Frank Morgolo (California Eyalet Üniversitesi, Long Beach), Jun Ni (Iowa Üniversitesi), Benjamin Nystruen (Colorado Üniversitesi Colorado Springs), Maureen Opkins (CA Eyalet Üniversitesi, Long Beach), Gavin Osborne (Saskatchewan Üniversitesi), Kevin Parker (Idaho Eyalet Üniversitesi), Dale Parson (Kutztown Üniversitesi), Mark Pendergast (Florida Gulf Coast Üniversitesi), Richard Povinelli (Marquette Üniversitesi), Roger Priebe (Texas Üniversitesi Austin), Mary Ann Pumphrey (De Anza Junior Koleji), Pat Roth (Güney Politeknik Eyalet Üniversitesi), Amr Sabry (Indiana Üniversitesi), Carolyn Schauble (Colorado Eyalet Üniversitesi), David Scuse (Manitoba Üniversitesi), Ashraf Shirani (San Jose Eyalet Üniversitesi), Daniel Spiegel (Kutztown Üniversitesi), Joslyn A. Smith (Florida Atlantic Üniversitesi), Lixin Tao (Pace Üniversitesi), Ronald F. Taylor (Wright Eyalet Üniversitesi), Russ

Tront (Simon Fraser Üniversitesi), Deborah Trytten (Oklahoma Üniversitesi), Kent Vidrine (George Washington Üniversitesi) ve Bahram Zartoshty (California Eyalet Üniversitesi, Northridge).

Pearson ile çalışmak büyük bir zevk, onur ve ayrıcalıktır. Bu projeyi organize eden, üreten ve tanıtan Tracy Dunkelberger ve meslektaşları Marcia Horton, Michael Hirsch, Matt Goldstein, Carole Snyder, Tim Huddleston, Yez Alayan, Jeff Holcomb, Kayla Smith-Tarbox, Gillian Hall, Rebecca Greenberg ve diğer meslektaşlarına teşekkür etmek isterim.

Her zamanki gibi, eşim Samantha'ya sevgisi, desteği ve cesaretlendirmesi için minnettarım.

KISACA İÇİNDEKİLER

1	Bilgisayarlara, Programlara ve Java'ya Giriş	1	16	Olay Tabanlı Programlama	599
2	Temel Programlama	33	17	GUI Bileşenleri	639
3	Seçimler	81	18	Appletler ve Multimedya	671
4	Döngüler	133	19	İkili G/Ç	709
5	Metotlar	177	20	Özyineleme	737
6	Tek Boyutlu Diziler	223		AEKLER	
7	Çok Boyutlu Diziler	263		AJava Anahtar Kelimeleri	771
8	Objeler ve Sınıflar	295		BASCI Karakter Seti	772
9	Dizeler	335		COperatör Öncelik Tablosu	774
10	Objelerde Düşünme	369		DJava Değiştiricileri	776
11	Miras Alma ve Polimorfizm	407		EÖzel Kaydırmalı Nokta Değerleri	778
12	GUI Temelleri	445		FSayı Sistemleri	779
13	Grafikler	479		GBit Düzeyinde İşlemler	783
14	İstisna Yönetimi ve Metin G/Ç	517		INDİS	785
15	Soyut Sınıflar ve Arayüzler	559			

İÇİNDEKİLER

Bölüm 1 Bilgisayarlara, Programlara,

ve Java'ya Giriş	1
1.1 Giriş	2
1.2 Bilgisayar Nedir?	2
1.3 Programlama Dilleri	9
1.4 İşletim Sistemleri	12
1.5 Java, Dünya Çapında Ağ ve Ötesi	13
1.6 Java Dili Spesifikasyonu, API, JDK ve IDE	16
1.7 Basit Bir Java Programı	16
1.8 Bir Java Programı Oluşturma, Derleme ve Çalıştırma	19
1.9 Bir Mesaj Diyalog Kutusunda Metin Gösterme	22
1.10 Programlama Tarzı ve Dokümantasyon	24
1.11 Programlama Hataları	26

Bölüm 2 Temel Programlama

2.1 Giriş	34
2.2 Basit Bir Program Yazma	34
2.3 Konsoldan Girdi Okuma	37
2.4 Tanımlayıcılar	40
2.5 Değişkenler	40
2.6 Atama İfadeleri ve Atama İfadeleri	42
2.7 Adlandırılmış Sabitler	43
2.8 İsimlendirme Kuralları	44
2.9 Sayısal Veri Türleri ve İşlemleri	44
2.10 Sayısal Literaller	48
2.11 İfadeleri Değerlendirme ve Operatör Önceliği	50
2.12 Vaka Çalışması: Mevcut Zamanı Gösterme	51
2.13 Artırılmış Atama Operatörleri	53
2.14 Artış ve Azalış Operatörleri	54
2.15 Sayısal Tür Dönüşümleri	56
2.16 Yazılım Geliştirme Süreci	58
2.17 Karakter Veri Türü ve İşlemleri	62
2.18 String Türü	68
2.19 Giriş Diyaloglarından Girdi Alma	70

Bölüm 3	Seçimler	81
3.1	Giriş	82
3.2	boolean Veri Türü	82
3.3	if İfadeleri	84
3.4	Vaka Çalışması: Doğum Günlerini Tahmin Etme	86
3.5	İki Yönlü if-else İfadeleri	89
3.6	İç İçe if ve Çok Yönlü if-else İfadeleri	91
3.7	Seçim İfadelerinde Yaygın Hatalar	93
3.8	Rastgele Sayılar Üretme	96
3.9	Vaka Çalışması: Vücut Kitle İndeksini Hesaplama	97
3.10	Vaka Çalışması: Vergi Hesaplama	99
3.11	Mantıksal Operatörler	101
3.12	Vaka Çalışması: Artık Yılı Belirleme	105
3.13	Vaka Çalışması: Piyango	106
3.14	switch İfadeleri	108
3.15	Koşullu İfadeler	111
3.16	Konsol Çıktısını Biçimlendirme	112
3.17	Operatör Önceliği ve Birliktelik	115
3.18	Onay Diyalogları	117
3.19	Hata Ayıklama	119
Bölüm 4	Döngüler	133
4.1	Giriş	134
4.2	while Döngüsü	134
4.3	do-while Döngüsü	144
4.4	for Döngüsü	146
4.5	Hangi Döngüyü Kullanmalıyım?	150
4.6	İç İçe Döngüler	152
4.7	Sayısal Hataları En Aza İndirme	154
4.8	Vaka Çalışmaları	155
4.9	Anahtar Kelimeler break ve continue	159
4.10	Vaka Çalışması: Asal Sayıları Gösterme	162
4.11	Onay Diyaloğu ile Bir Döngüyü Kontrol Etme	164
Bölüm 5	Yöntemler	177
5.1	Giriş	178
5.2	Bir Yöntemi Tanımlama	178
5.3	Bir Yöntemi Çağırma	180
5.4	void Yöntem Örneği	183
5.5	Parametreleri Değerlerle Geçirme	186

5.6	Kodu Modülerleştirme	189
5.7	Vaka Çalışması: Ondalık Sayıları Onaltılı Sayılara Dönüştürme	191
5.8	Yöntem Aşırı Yükleme	193
5.9	Değişkenlerin Kapsamı	196
5.10	Math Sınıfı	197
5.11	Vaka Çalışması: Rastgele Karakterler Üretme	201
5.12	Yöntem Soyutlaması ve Aşamalı İyileştirme	203

Bölüm 6 Tek Boyutlu Diziler 223

6.1	Giriş	224
6.2	Dizi Temelleri	224
6.3	Vaka Çalışması: Loto Numaraları	231
6.4	Vaka Çalışması: Deste Kartlar	234
6.5	Dizileri Kopyalama	236
6.6	Dizileri Yöntemlere Geçirme	237
6.7	Bir Yöntemden Dizi Döndürme	240
6.8	Vaka Çalışması: Her Harfin Görülme Sayısını Sayma	241
6.9	Değişken Uzunlukta Argüman Listeleri	244
6.10	Dizilerde Arama	245
6.11	Dizileri Sıralama	248
6.12	t Arrayst Sınıfı	252

Bölüm 7Çok Boyutlu Diziler 263

7.1	Giriş	264
7.2	İki Boyutlu Dizi Temelleri	264
7.3	İki Boyutlu Dizileri İşleme	267
7.4	İki Boyutlu Dizileri Yöntemlere Geçirme	269
7.5	Vaka Çalışması: Çoktan Seçmeli Bir Testin Notlandırılması	270
7.6	Vaka Çalışması: En Yakın Çifti Bulma	272
7.7	Vaka Çalışması: Sudoku	274
7.8	Çok Boyutlu Diziler	277

Bölüm 8Nesneler ve Sınıflar 295

8.1	Giriş	296
8.2	Nesneler için Sınıfların Tanımlanması	296
8.3	Örnek: Sınıfları Tanımlama ve Nesneler Oluşturma	298
8.4	Yapıcılar Kullanarak Nesnelerin Oluşturulması	303
8.5	Referans Değişkenleri Aracılığıyla Nesnelere Erişim	304
8.6	Java Kütüphanesinden Sınıfların Kullanılması	308
8.7	Statik Değişkenler, Sabitler ve Metotlar	312
8.8	Görünürlük Değiştiricileri	317

8.9	Veri Alanı Kapsülleme	319
8.10	Nesneleri Metotlara Geçirme	322
8.11	Nesneler Dizisi	326

Bölüm 9 Dizeler 335

9.1	Giriş	336
9.2	Dize Sınıfı	336
9.3	Vaka Çalışması: Palindromları Kontrol Etme	347
9.4	Vaka Çalışması: Onaltılıları Onluk Sayılara Dönüştürme	348
9.5	Karakter Sınıfı	350
9.6	StringBuilder ve StringBuffer Sınıfları	353
9.7	Komut Satırı Argümanları	358

Bölüm 10 Nesnelerde Düşünme 369

10.1	Giriş	370
10.2	Değiştirilemez Nesneler ve Sınıflar	370
10.3	Değişkenlerin Kapsamı	371
10.4	this Referansı	373
10.5	Sınıf Soyutlaması ve Kapsülleme	375
10.6	Nesne Yönelimli Düşünme	379
10.7	Nesne Bileşimi	382
10.8	Vaka Çalışması: Course Sınıfını Tasarlamak	384
10.9	Vaka Çalışması: Yığınlar için Bir Sınıf Tasarlamak	386
10.10	Vaka Çalışması: GuessDate Sınıfını Tasarlama	388
10.11	Sınıf Tasarım Kılavuzları	391
10.12	Temel Veri Tipi Değerlerini Nesneler Olarak İşleme	393
10.13	Temel Türler ile Sarıcı Sınıf Türleri Arasında Otomatik Dönüşüm	396
10.14	BigInteger ve BigDecimal Sınıfları	397

Bölüm 11 Kalıtım ve Polimorfizm 407

11.1	Giriş	408
11.2	Üst Sınıflar ve Alt Sınıflar	408
11.3	super Anahtar Kelimesini Kullanma	414
11.4	Yöntemleri Geçersiz Kılma	418
11.5	Geçersiz Kılma vs. Aşırı Yükleme	418
11.6	Nesne Sınıfı ve toString() Yöntemleri	420
11.7	Polimorfizm	421
11.8	Dinamik Bağlama	422
11.9	Nesneleri Dönüştürme ve instanceof Operatörü	425
11.10	Nesne'nin equals Metodu	429

11.1.1	ArrayList Sınıfı	430
11.1.2	Vaka Çalışması: Özel Bir Yığın Sınıfı	436
11.1.3	korumalı Veri ve Yöntemler	437
11.1.4	Uzatma ve Geçersiz Kılmayı Önleme	439

Bölüm 12 GUI Temelleri 445

12.1	Giriş	446
12.2	Swing ve AWT	446
12.3	Java GUI API'si	446
12.4	Çerçeveler	449
12.5	Yerleşim Yöneticileri	451
12.6	Alt Konteyner Olarak Panellerin Kullanımı	458
12.7	Color Sınıfı	460
12.8	Font Sınıfı	461
12.9	Swing GUI Bileşenlerinin Ortak Özellikleri	462
12.10	Görüntü İkonları	465
12.11	JDüğmesi	467
12.12	JCheckBox	471
12.13	JRadioButton	472
12.14	Etiketler	473
12.15	Metin Alanları	474

Bölüm 13 Grafikler 479

13.1	Giriş	480
13.2	Grafikler Sınıfı	480
13.3	String'leri, Çizgileri, Dikdörtgenleri ve Ovaları Çizme	483
13.4	Vaka Çalışması: FigurePanel Sınıfı	485
13.5	Yaylar Çizme	488
13.6	Çokgenler ve Polilinler Çizme	490
13.7	FontMetrics Sınıfını Kullanarak Bir String'i Ortalamak	493
13.8	Vaka Çalışması: MessagePanel Sınıfı	495
13.9	Vaka Çalışması: StillClock Sınıfı	500
13.10	Görüntüleri Gösterme	504
13.11	Vaka Çalışması: ImageViewer Sınıfı	506

Bölüm 14 İstisna Yönetimi ve Metin G/Ç 517

14.1	Giriş	518
14.2	İstisna Yönetimi Genel Görünümü	518
14.3	İstisna Türleri	523
14.4	İstisna Yönetimi Hakkında Daha Fazla	526
14.5	finally Klozu	534

14.6	İstisnaları Ne Zaman Kullanmalıyız	535
14.7	İstisnaları Yeniden Fırlatma	536
14.8	Zincirleme İstisnalar	537
14.9	Özel İstisna Sınıflarını Tanımlama	538
14.10	Dosya Sınıfı	541
14.11	Dosya Girişi ve Çıkışı	544
14.12	Dosya Diyalogları	549
14.13	Web'den Veri Okuma	551

Bölüm 15 Soyut Sınıflar ve Arayüzler 559

15.1	Giriş	560
15.2	Soyut Sınıflar	560
15.3	Vaka Çalışması: Soyut Sayı Sınıfı	565
15.4	Vaka Çalışması: Takvim ve Gregoryen Takvimi	567
15.5	Arayüzler	570
15.6	Comparable Arayüzü	573
15.7	Cloneable Arayüzü	577
15.8	Arayüzler vs. Soyut Sınıflar	581
15.9	Vaka Çalışması: Rational Sınıfı	584

Bölüm 16 Olay Tabanlı Programlama 599

16.1	Giriş	600
16.2	Olaylar ve Olay Kaynakları	602
16.3	Dinleyiciler, Kayıtlar ve Olayları Yönetme	603
16.4	İç Sınıflar	608
16.5	Anonim Sınıf Dinleyicileri	609
16.6	Dinleyici Sınıflarını Tanımlamanın Alternatif Yolları	612
16.7	Vaka Çalışması: Kredi Hesaplayıcı	615
16.8	Fare Olayları	617
16.9	Dinleyici Arayüz Adaptörleri	620
16.10	Tuşa Basma Olayları	621
16.11	Timer Sınıfını Kullanarak Animasyon	625

Bölüm 17 GUI Bileşenleri 639

17.1	Giriş	640
17.2	JCheckBox, JRadioButton ve JtextField için Olaylar	640
17.3	Metin Alanları	644
17.4	Combo Kutuları	647
17.5	Listeler	650
17.6	Kaydırma Çubukları	654

17.7	Kaydıncılar	657
17.8	Birden Fazla Pencere Oluşturma	660

Bölüm 18 Applet'ler ve Multimedya 671

18.1	Giriş	672
18.2	Applet'leri Geliştirme	672
18.3	HTML Dosyası ve <applet>Etiketi	673
18.4	Applet Güvenlik Kısıtlamaları	675
18.5	Applet'lerin Uygulama Olarak Çalışmasına İzin Verme	676
18.6	Applet Yaşam Döngüsü Yöntemleri	677
18.7	Dize Geçiş ile Applet'lere	679
18.8	Vaka Çalışması: Zıplayan Top	683
18.9	Vaka Çalışması: XOX Oyunu Geliştirme	686
18.10	Kaynakları URL Sınıfı Kullanarak Bulma	691
18.11	Herhangi Bir Java Programında Ses Oynatma	693
18.12	Vaka Çalışması: Ulusal Bayraklar ve Marşlar	695

Bölüm 19 İkili G/Ç 709

19.1	Giriş	710
19.2	Java'da Metin G/Ç Nasıl Yönetilir?	710
19.3	Metin G/Ç ile İkili G/Ç Arasındaki Fark	711
19.4	İkili G/Ç Sınıfları	712
19.5	Vaka Çalışması: Dosyaları Kopyalama	722
19.6	Nesne G/Ç	724
19.7	Rastgele Erişimli Dosyalar	729

Bölüm 20 Yeniden Çağırma 737

20.1	Giriş	738
20.2	Vaka Çalışması: Faktöriyel Hesaplama	738
20.3	Vaka Çalışması: Fibonacci Sayılarını Hesaplama	741
20.4	Yeniden Çağırma Kullanarak Problem Çözme	744
20.5	Yardımcı Yöntemler için Yeniden Çağırma	746
20.6	Vaka Çalışması: Dizin Boyutunu Bulma	749
20.7	Vaka Çalışması: Hanoi Kuleleri	750
20.8	Vaka Çalışması: Fraktallar	754
20.9	Özyineleme vs. İterasyon	757
20.10	Sonlu Özyineleme	758

AEKLER

AekA Java Anahtar Kelimeleri	771
AekB ASCII Karakter Seti	772
AekC Operatör Öncelik Tablosu	774
AekD Java Değiştiricileri	776
AekE Özel Kaydırmalı Değerler	778
AekF Sayı Sistemleri	779
AppendixG Bitwise İşlemler	783
 INDİS	 785



VideoNot

VideoNotları

VideoNotları'nın Yerleri

[HTTP://www.pearsonyukseled.com/](http://www.pearsonyukseled.com/)

Bölüm 1	Bilgisayarlara, Programlara ve Java'ya Giriş	
	İlk Java programınız	17
	Eclipse kısa eğitimi	19
	NetBeans kısa eğitimi	19
	Bir Java programını derleyin ve çalıştırın	21
Bölüm 2	Temel Programlama	
	Girdi al	37
	Operatörleri / ve % kullanın	51
	Yazılım geliştirme süreci	58
	Kredi ödemelerini hesaplayın	59
	BMI'yi hesaplayın	77
Bölüm 3	Seçimler	
	Toplama sınavı programı	83
	Çıkarma sınavı programı	96
	Çok yönlü if-else ifadelerini 99 kullanın	99
	Üç tam sayıyı sırala	123
	Nokta konumunu kontrol et	125
Bölüm 4	Döngüler	
	Bir sayı tahmin et	137
	Çoklu çıkarma sınavı	139
	Sayısal hataları en aza indir	154
	Kredi takvimini göster	170
	Bir seriyi topla	170
Bölüm 5	Metotlar	
	max metodunu tanımla/çağır	180
	void metodunu kullan	183
	Kodu modüler hale getir	189
	Adım adım iyileştirme	203
	Bir tam sayıyı tersine çevir	212
	215'i tahmin et	
Bölüm 6	Tek Boyutlu Diziler	
	Rastgele karıştırma	228
	Lotto numaraları	231
	Seçim sıralaması	249
	Kupon toplayıcısının problemi	260
	Arka arkaya dört	261
Bölüm 7	Çok Boyutlu Diziler	
	En büyük toplamı olan satırı bul	268
	Çoktan seçmeli testi notlandır	270
	Sudoku	274
	İki matris çarp	282
	Çift sayıda 1	289
Bölüm 8	Objeler ve Sınıflar	
	Sınıfları ve nesneleri tanımla	296
	Sınıfları kullan	311
	Statik vs. örnek	312
	Veri alanı kapsülleme	319
	The Fant sınıfı	331

Bölüm 9	Dizeler	
	Palindrom kontrolü	347
	Komut satırı argümanı	359
	Numara dönüştürme	364
	ISBN-10 kontrolü	367
Bölüm 10	Objelerde Düşünme	
	Değiştirilemez nesneler ve bu anahtar kelimesi	370
	Ödünç sınıfı	376
	BMI sınıfı	380
	StackOfIntegers sınıfı	386
	Büyük sayıları işleme	397
	MyPoint sınıfı	400
Bölüm 11	Miras Alma ve Polimorfizm	
	Geometrik sınıf hiyerarşisi	408
	Polimorfizm ve dinamik bağlama demosu	423
	ArrayList sınıfı	430
	MyStack sınıfı	436
	Yeni Hesap sınıfı	443
Bölüm 12	GUI Temelleri	
	KullanFlowLayout	452
	Pano'ları alt konteynerler olarak kullanın	458
	Swing ortak özelliklerini kullanın	462
	Bir dama tahtası gösterin	477
	Rastgele bir matris gösterin	478
Bölüm 13	Grafikler	
	FigurePanel sınıfı	485
	MessagePanel sınıfı	495
	StillClock sınıfı	500
	Bir fonksiyonu çizin	511
	Bir çubuk grafik çizin	512
Bölüm 14	İstisna Yönetimi ve Metin G/Ç	
	İstisna işleme avantajları	518
	Özel istisna sınıfları oluşturun	538
	Veri yazın ve okuyun	544
	HexFormatException	555
Bölüm 15	Soyut Sınıflar ve Arayüzler	
	AbstractGeometricObject sınıfı	560
	Takvim veGregorianCalendar sınıfları	567
	Arayüz kavramı	570
	Yeniden tasarlayın Dikdörtgen sınıfı	593
Bölüm 16	Olay Tabanlı Programlama	
	Dinleyici ve kaydı	607
	Anonim dinleyici	610
	Mesajı fare ile taşı	618
	Bir saati animasyonla göster	628
	Yükselen bir bayrağı animasyonla göster	632
	Fare noktasının konumunu kontrol et	632
Bölüm 17	GUI Bileşenleri	
	Metin alanlarını kullan	668
Bölüm 18	Appletler ve Multimedya	
	İlk applet	672
	Applet'leri bağımsız çalıştır	676

	TicTacToe	686
	Ses ve görüntü	695
	Bir grup saati kontrol et	701
Bölüm 19	İkili G/Ç	
	Dosyayı kopyala	722
	Nesne G/Ç	724
	Büyük bir dosyayı böl	734
Bölüm 20	Özyineleme	
	İkili arama	748
	Dizin boyutu	749
	Fraktal (Sierpinski üçgeni)	754
	Bir dizinde bir dize arama	764

GİRİŞBİLGİSAY ARLARA,PROGR AMLARA,V EJAVA'YA

Hedefler

- Bilgisayar temellerini, programları ve işletim sistemlerini anlamak (§§1.2–1.4).
- Java ile Dünya Çapında Ağ arasındaki ilişkiyi tanımlamak (§1.5).
- Java dil spesifikasyonu, API, JDK,
ve IDE'nin anlamını anlamak (§1.6).
- Basit bir Java programı yazmak (§1.7).
- Çıktıyı konsolda görüntülemek için (§1.7).
- Bir Java programının temel sözdizimini açıklamak için (§1.7).
- Java programları oluşturmak, derlemek ve çalıştırmak için (§1.8).
- Çıktıyı **JOptionPane** mesaj iletişim kutularını kullanarak görüntülemek için (§1.9).
- Java programlama tarzı ve belgelendirmesi ile tanışmak için (§1.10).
- Sözdizimi hataları, çalışma zamanı hataları ve mantık hataları arasındaki farkları açıklamak için (§1.11).



2 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

1.1 Giriş



Bu kitabın merkezi teması, bir program yazarak sorunları nasıl çözeceğinizi öğrenmektir.

programlama nedir?

programlama

program

Bu kitap programlama ile ilgilidir. Peki, programlama nedir? Programlama terimi, bir yazılım oluşturmak (veya geliştirmek) anlamına gelir; bu da bir program olarak adlandırılır. Temel terimlerle, yazılım, bir bilgisayar veya bilgisayarlı bir cihaza ne yapması gerektiğini söyleyen talimatları içerir.

Yazılım her yerde, hatta ihtiyaç duymayacağınızı düşündüğünüz cihazlarda bile mevcuttur. Elbette, bir kişisel bilgisayarda yazılım bulmayı ve kullanmayı beklersiniz, ancak yazılım aynı zamanda uçakların, arabaların, cep telefonlarının ve hatta tost makinelerinin çalışmasında da rol oynar. Kişisel bir bilgisayarda, belgeleri yazmak için kelime işlemcileri, İnterneti keşfetmek için web tarayıcıları ve mesaj göndermek için e-posta programları kullanırsınız. Bu programların hepsi yazılım örnekleridir. Yazılım geliştiricileri, programlama dilleri adı verilen güçlü araçların yardımıyla yazılım oluştururlar.

Bu kitap, Java programlama dilini kullanarak programlar oluşturmaya öğretir. Pek çok programlama dili vardır, bunlardan bazıları on yıllık geçmişe sahiptir. Her dil, belirli bir amaç için icat edilmiştir; örneğin, önceki bir dilin güçlü yönlerini geliştirmek ve ya programcıya yeni ve benzersiz bir araç seti sunmak için. Bu kadar çok programlama dili olduğuna bildiğinizde, hangisinin en iyi olduğunu merak etmeniz doğal olacaktır. Ancak gerçekte, "en iyi" bir dil yoktur. Her birinin kendi güçlü ve zayıf yönleri vardır. Deneyimli programcılar, bir dilin bazı durumlarda iyi çalışabileceğini, oysa farklı bir dilin diğer durumlarda daha uygun olabileceğini bilirler. Bu nedenle, deneyimli programcılar mümkün olduğunca çok farklı programlama dilini öğrenmeye çalışırlar ve bu da onlara geniş bir yazılım geliştirme araçları yelpazesine erişim sağlar.

Eğer bir dili kullanarak programlamayı öğrenirseniz, diğer dilleri öğrenmekte kolaylık çekmelisiniz. Anahtar, bir programlama yaklaşımı kullanarak sorunları nasıl çözeceğinizi öğrenmektir. Bu kitabın ana teması budur.

Heyecan verici bir yolculuğa başlamak üzeresiniz: programlamayı öğrenmek. Başlangıçta, bilgisayar temellerini, programları ve işletim sistemlerini gözden geçirmek faydalıdır. CPU, bellek, diskler, işletim sistemleri ve programlama dilleri gibi terimlere zaten aşina iseniz, Bölüm 1.2-1.4'teki gözden geçirmeyi atlayabilirsiniz.

1.2 Bilgisayar Nedir?



Bir bilgisayar, verileri depolayan ve işleyen elektronik bir cihazdır.

donanım

yazılım

Bir bilgisayar hem donanım hem de yazılım içerir. Genel olarak, donanım bilgisayarın görünür, fiziksel unsurlarını oluşturur ve yazılım, donanımı kontrol eden ve belirli görevleri yerine getirmesini sağlayan görünmez talimatları sağlar. Bilgisayar donanımını bilmek, bir programlama dilini öğrenmek için zorunlu değildir, ancak bir programın talimatlarının bilgisayar ve bileşenleri üzerindeki etkilerini daha iyi anlamana yardımcı olabilir. Bu bölüm, bilgisayar donanım bileşenlerini ve işlevlerini tanıtmaktadır.

Bir bilgisayar aşağıdaki ana donanım bileşenlerinden oluşur (Şekil 1.1):

- Merkezi işlem birimi (CPU)
 - Bellek (ana bellek)
 - Depolama aygıtları (diskler ve CD'ler gibi)
 - Giriş aygıtları (fare ve klavye gibi)
 - Çıkış aygıtları (monitörler ve yazıcılar gibi)
 - İletişim aygıtları (modemler ve ağ arayüz kartları gibi)
- Bir bilgisayarın b

ileşenleri, bir *veri yolu* adı verilen bir alt sistemle birbirine bağlıdır. Bir veri yolunu, bilgisayarın bileşenleri arasında uzanan bir yol sistemi olarak düşünebilirsiniz; veriler ve

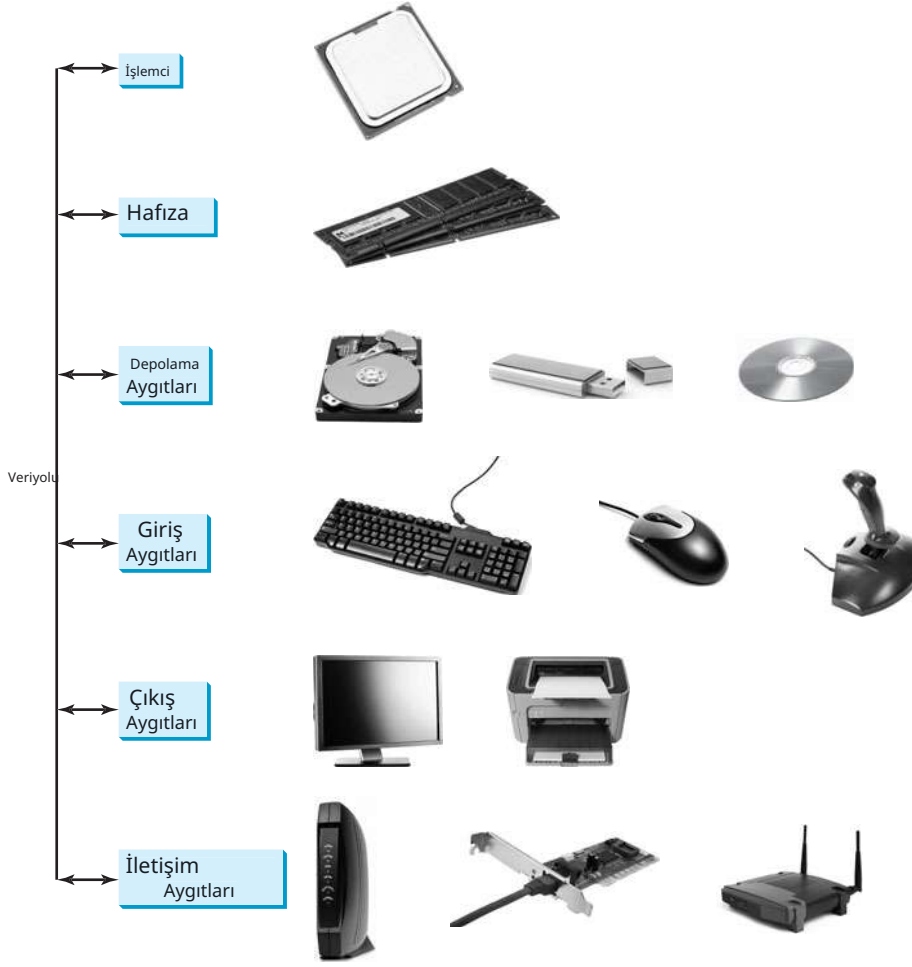


FIGURE 1.1 Bir bilgisayar, bir CPU, bellek, depolama aygıtları, giriş aygıtları, çıkış aygıtları ve iletişim aygıtlarından oluşur.

Güç, bilgisayarın bir parçasından diğerine otobüs boyunca hareket eder. Kişisel bilgisayarlarda, otobüs, bilgisayarın *anakartına* entegre edilmiştir; bu, bir bilgisayarın tüm parçalarını bir araya bağlayan bir devre kutusudur, Şekil 1.2'de gösterildiği gibi.

anakart

1.2.1 Merkezi İşlem Birimi

Merkezi işlem birimi (CPU), bilgisayarın beynidir. Bellekten talimatları alır ve bunları yerine getirir. CPU genellikle iki bileşenden oluşur: bir kontrol birimi ve bir aritmetik/lojik birim. Kontrol birimi, diğer bileşenlerin eylemlerini kontrol eder ve koordine eder. Aritmetik/lojik birim, sayısal işlemleri (toplama, çıkarma, çarpma, bölme) ve mantıksal işlemleri (karşılaştırmalar) gerçekleştirir.

İşlemci

Günümüz CPU'ları, bilgi işlemek için milyonlarca küçük elektrik anahtarı içeren küçük silikon yarı iletken yongalar üzerine inşa edilmiştir ve bu anahtarlara *transistör* denir.

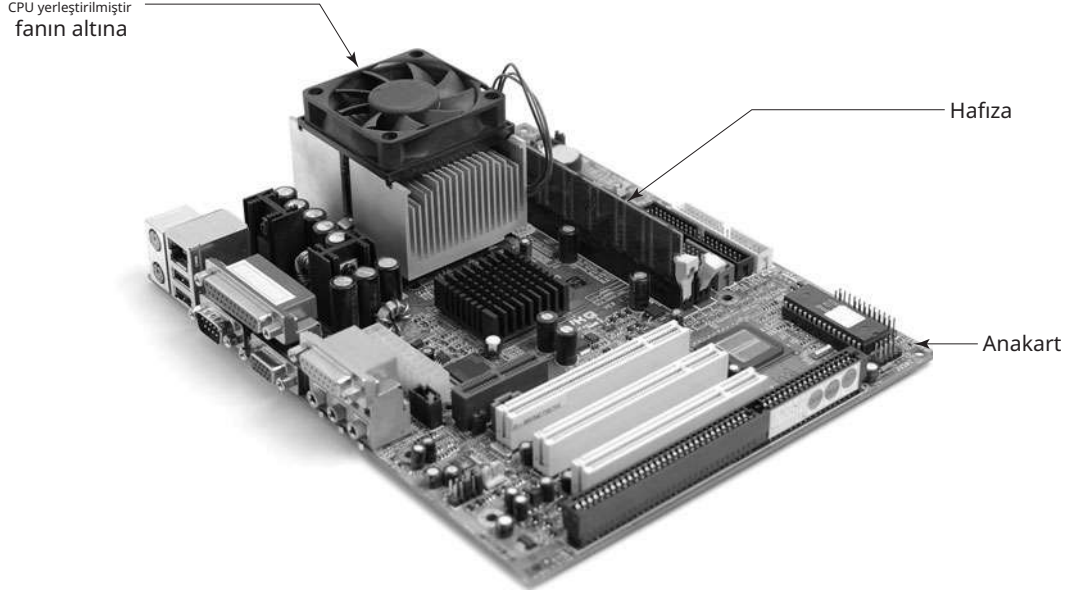
Her bilgisayarın, sabit bir hızda elektronik darbeler yayan bir iç saati vardır. Bu darbeler, işlemlerin hızını kontrol etmek ve senkronize etmek için kullanılır. Daha yüksek bir saat hızı, belirli bir zaman diliminde daha fazla talimatın yerine getirilmesini sağlar. Saat hızının ölçüm birimi *hertz (Hz)* olup, 1 hertz 1 darbe/saniye eşittir. 1990'larda bilgisayarlar saat hızını *megahertz (MHz)* cinsinden ölçüyordu, ancak CPU hızı sürekli olarak gelişmektedir,

hız

hertz

megahertz

4 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş



ŞEKİL 1.2 Anakart, bir bilgisayarın tüm parçalarını bir araya bağlar.

gigahertz

ve bir bilgisayarın saat hızı genellikle *gigahertz (GHz) cinsinden belirtilir*. Intel'in en yeni işlemcileri yaklaşık 3 GHz hızında çalışmaktadır.

çekirdek

CPU'lar başlangıçta yalnızca bir çekirdek ile geliştirilmiştir. *çekirdek*, işlemcinin komutları okuma ve yürütme işlevini yerine getiren kısımdır. CPU işleme gücünü artırmak için, çip üreticileri artık birden fazla çekirdek içeren CPU'lar üretmektedir. Çok çekirdekli bir CPU, iki veya daha fazla bağımsız işlemciye sahip tek bir bileşendir. Günümüz tüketici bilgisayarları genellikle iki, üç ve hatta dört ayrı çekirdeğe sahiptir. Yakında, onlarca veya hatta yüzlerce çekirdeğe sahip CPU'lar uygun fiyatlı hale gelecektir.

1.2.2 Bitler ve Baytlar

Hafızayı tartışmadan önce, bilgilerin (veri ve programlar) bir bilgisayarda nasıl saklandığını a bakalım.

Bir bilgisayar aslında bir dizi anahtardan başka bir şey değildir. Her anahtar iki durumda bulunur: açık veya kapalı. Bilgiyi bir bilgisayarda saklamak, bir dizi anahtarı açık veya kapalı konuma getirmekten ibarettir. Anahtar açıksa, değeri 1'dir. Anahtar kapalıysa, değeri 0'dır.

bit

Bu 0'lar ve 1'ler, ikili sayı sisteminde rakamlar olarak yorumlanır ve *bitler*

(ikili rakamlar) olarak adlandırılır.

bayt

Bir bilgisayardaki en küçük depolama birimi bayttır. Bir byte sekiz bitten oluşur. 3 bit küçük bir sayı tek bir byte olarak saklanabilir. Tek bir byte'a sığmayan bir sayıyı saklamak için, bilgisayar birkaç byte kullanır.

kodlama şeması

Çeşitli türdeki veriler, sayılar ve karakterler gibi, bir dizi bayt olarak kodlanır. Bir programcı olarak, verilerin kodlanması ve kod çözülmesi ile ilgili endişelenmenize gerek yoktur; bu, bilgisayar ayar sistemi tarafından otomatik olarak, kodlama şemasına dayanarak gerçekleştirilir. Bir kodlama şeması, bir bilgisayarın karakterleri, sayıları ve sembolleri, bilgisayarın gerçekten çalışabil eceği verilere nasıl çevirdiğini yöneten bir dizi kuraldır. Çoğu şema, her karakteri önceden belirlenmiş bir sayı dizisine çevirir. Örneğin, popüler ASCII kodlama şemasında, karakter C, bir baytta 01000011 olarak temsil edilir.

Bir bilgisayarın depolama kapasitesi bayt ve baytın katları cinsinden ölçülür, şu şekilde:

- Bir *kilobayt (KB)* yaklaşık 1.000 bayttır. kilobayt (KB)
- Bir *megabayt (MB)* yaklaşık 1 milyon bayttır. megabayt (MB)
- Bir *gigabayt (GB)* yaklaşık 1 milyar bayttır. gigabayt (GB)
- Bir *terabayt (TB)* yaklaşık 1 trilyon bayttır. terabayt (TB)

Tipik bir sayfalık kelime belgesi yaklaşık 20 KB yer kaplayabilir. Bu nedenle, 1 MB 50 sayfa belge depolayabilir ve 1 GB 50,000 sayfa belge depolayabilir. Tipik bir iki saatlik yüksek çözünürlüklü film yaklaşık 8 GB yer kaplayabilir, bu nedenle 20 filmi depolamak için 160 GB gereklidir.

1.2.3 Bellek

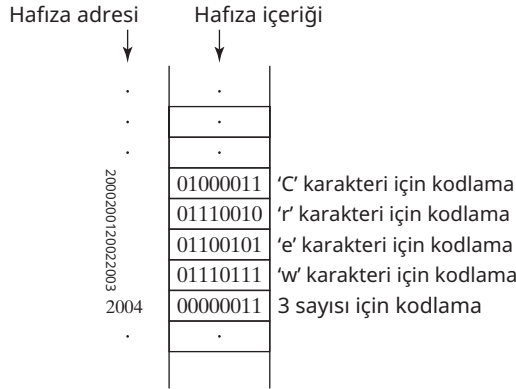
Bir bilgisayarın *belleği*, programları ve programın çalıştığı verileri depolamak için sıralı bir bayt dizisinden oluşur. Belleği, bir programı çalıştırmak için bilgisayarın çalışması alanı olarak düşünebilirsiniz. Bir program ve verileri, CPU tarafından çalıştırılmadan önce bilgisayarın belleğine taşınmalıdır.

Bellekteki her baytın benzersiz bir adresi vardır; bu, Şekil 1.3'te gösterilmiştir. Adres, verileri depolamak ve geri almak için baytı bulmak için kullanılır. Bellekteki baytlara herhangi bir sırayla erişilebildiğinden, bellek ayrıca rastgele erişim belleği (RAM) olarak da adlandırılır.

bellek

benzersiz adres

RAM



FIGÜRİN 1.3 Hafıza, verileri ve program talimatlarını benzersiz adreslenmiş hafıza konumlarında depolar. Her hafıza konumu bir bayt veri depolayabilir.

Günümüz kişisel bilgisayarları genellikle en az 1 gigabayt RAM'e sahiptir, ancak daha yaygın olarak 2 ila 4 GB kurulu bulunmaktadır. Genel olarak, bir bilgisayarın sahip olduğu RAM miktarı arttıkça, daha hızlı çalışabilir, ancak bu basit kuralın sınırlamaları vardır.

Bir bellek baytı asla boş değildir, ancak başlangıçtaki içeriği programınız için anlamsız olabilir. Bir bellek baytının mevcut içeriği, içine yeni bilgi yerleştirildiğinde kaybolur.

CPU gibi, bellek de yüzeyinde milyonlarca transistör bulunan silikon yarı iletken yongalar üzerine inşa edilmiştir. CPU yongalarına kıyasla, bellek yongaları daha az karmaşık, daha yavaş ve daha ucuzdur.

1.2.4 Depolama Cihazları

Bir bilgisayarın belleği (RAM) geçici bir veri depolama biçimidir: bellekte (yani, kaydedilmiş) depolanan herhangi bir bilgi, sistemin gücü kapatıldığında kaybolur. Programlar ve veriler kalıcı depolama cihazlarında kalıcı olarak saklanır ve bilgisayar bunları kullandığında, belleğe taşınır; bu, kalıcı depolama cihazlarının sağlayabileceğinden çok daha hızlı çalışır.

depolama cihazları

6 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

Üç ana depolama cihazı türü vardır:

- Manyetik disk sürücülere
- Optik disk sürücülere (CD ve DVD)
- USB bellekler

sürücü

Sürücüler, diskler ve CD'ler gibi bir ortamı işletmek için kullanılan cihazlardır. Bir depolama ortamı verileri ve program talimatlarını fiziksel olarak depolar. Sürücü, ortamdan verileri okur ve ortama verileri yazar.

Diskler

sabit disk

Bir bilgisayar genellikle en az bir sabit disk sürücüsüne sahiptir (Şekil 1.4). *Sabit diskler*, verileri ve programları kalıcı olarak depolamak için kullanılır. Daha yeni bilgisayarlar, 200 ile 800 gigabayt arasında veri depolayabilen sabit disklerle donatılmıştır. Sabit disk sürücülere genellikle bilgisayarın içinde yer alır, ancak çıkarılabilir sabit diskler de mevcuttur.



FIGURE 1.4 Bir sabit disk, programları ve verileri kalıcı olarak depolamak için bir cihazdır.

CD'ler ve DVD'ler

CD-R

CD, kompakt disk anlamına gelir. İki tür CD sürücüsü vardır: CD-R ve CD-RW. Bir *CD-R*, yalnızca okunabilir kalıcı depolama içindir; kullanıcı, kaydedildikten sonra içeriğini değiştiremez.

nCD-RW

Bir *CD-RW*, bir sabit disk gibi kullanılabilir; yani, diske veri yazabilir ve ardından bu veriyi yeni verilerle üzerine yazabilirsiniz. Tek bir CD, 700 MB'a kadar veri tutabilir. Çoğu yeni PC, hem CD-R hem de CD-RW disklerle çalışabilen bir CD-RW sürücüsü ile donatılmıştır.

DVD

DVD, dijital çok yönlü disk veya dijital video disk anlamına gelir. DVD'ler ve CD'ler benzer görünür ve verileri depolamak için her ikisini de kullanabilirsiniz. Bir DVD, bir CD'den daha fazla bilgi tutabilir; standart bir DVD'nin depolama kapasitesi 4.7 GB'dir. CD'ler gibi, iki tür DVD vardır: DVD-R (sadece okunabilir) ve DVD-RW (yeniden yazılabilir).

USB Flash Sürücüler

Evensel seri veri yolu (USB) konektörleri, kullanıcının bilgisayara birçok türde çevresel cihaz bağlamasına olanak tanır. Bir yazıcı, dijital kamera, fare, harici sabit disk sürücüsü ve diğer cihazları bilgisayara bağlamak için bir USB kullanabilirsiniz.

Bir USB *bellek sürücüsü*, verileri depolamak ve taşımak için bir cihazdır. Bir bellek sürücüsü küçüktür—bir sakız paketi boyutunda, Şekil 1.5'te gösterildiği gibi. Bilgisayarınızın USB portuna takılabilen taşınabilir bir sabit disk gibi çalışır. USB bellek sürücüler şu anda 256 GB'a kadar depolama kapasitesi ile mevcuttur.



FIGURE 1.5 USB bellekler çok taşınabilir ve çok fazla veri depolayabilir.

1.2.5 Giriş ve Çıkış Cihazları

Giriş ve çıkış cihazları, kullanıcının bilgisayarla iletişim kurmasını sağlar. En yaygın giriş cihazları *klavyeler* ve *farelerdir*. En yaygın çıkış cihazları ise *monitörler* ve *yazıcılardır*.

Klavyenin

Bir klavye, girişi girmek için bir cihazdır. Şekil 1.6, tipik bir klavyeyi göstermektedir. Sayısal tuş takımı olmayan kompakt klavyeler mevcuttur.



FIGURE 1.6 Bir bilgisayar klavyesi, bir bilgisayara girdi göndermek için tuşları içerir.

Fonksiyon tuşları klavyenin üst kısmında yer alır ve F harfi ile başlar. Fonksiyonları, şu anda kullanılan yazılıma bağlıdır.

fonksiyon tuşu

8 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

değiştirici tuş

A *değiştirici tuş*, iki tuşa aynı anda basıldığında diğer bir tuşun normal eylemini d eğiştiren özel bir tuştur (örneğin *Shift*, *Alt* ve *Ctrl* tuşları).

sayısal tuş takımı

sayısal tuş takımı, çoğu klavyenin sağ tarafında bulunan, sayıları hızlı bir şekilde girmek için hesap makinesi gibi tasarlanmış ayrı bir tuş setidir.

ok tuşları

Ok tuşları, ana tuş takımı ile sayısal tuş takımı arasında yer alır ve birçok programda ekr an üzerinde fare imlecini yukarı, aşağı, sola ve sağa hareket ettirmek için kullanılır.

Ekle tuşu

Ekle, Sil, Sayfa Yukarı ve Sayfa Aşağı tuşları, kelime işlem ve diğer programlarda metin ve nesneleri eklemek, metin ve nesneleri silmek ve bir belge içinde bir ekranı bir seferde yukarı v eya aşağı hareket ettirmek için kullanılır.

Sil tuşu

Sayfa Yukarı tuşu

Sayfa Aşağı tuşu

Fare

A *fare*, bir işaretleme cihazıdır. Ekranda bir grafik işaretçi (genellikle bir ok şeklinde) ola n *imleci* hareket ettirmek veya ekran nesnelerine (örneğin bir düğmeye) tıklayarak bir eylemi tetiklemek için kullanılır.

Monitör

monitör, bilgileri (metin ve grafikler) görüntüler. Ekran çözünürlüğü ve nokta aralı ığı, görüntü kalitesini belirler.

ekran çözünürlüğü
piksel

Ekran çözünürlüğü, görüntü cihazının yatay ve dikey boyutlarındaki piksel sayısını belirtir. Pikse ller ("resim elemanları" için kısaltma) ekranda bir görüntü oluşturan küçük noktalar dır. Örneğin, 1 7 inçlik bir ekran için yaygın bir çözünürlük 1.024 piksel genişliğinde ve 768 piksel yüksekliğindedir . Çözünürlük manuel olarak ayarlanabilir. Çözünürlük ne kadar yüksekse, görüntü o kadar keskin venet olur.

nokta aralığı

Nokta aralığı, pikseller arasındaki boşluk miktarıdır ve milimetre cinsinden ölçülür. Nokta aralığı ne ka dar küçükse, görüntü o kadar keskindir.

1.2.6 İletişim Cihazları

modem

Bilgisayarlar, bir telefon hattı üzerinden veri iletebilen bir dial-up modem (modülatör/demodülatör), DSL veya kablo modemi, kablolu bir ağ arayüz kartı veya kablosuz adaptör gibi iletişim cihazları aracılığıyla ağa bağlana bilir.

■ Dial-up modem, bir telefon hattı kullanır ve verileri saniyede 56.000 bps (bit/saniye) h ızında iletebilir.

dijital abone hattı (DSL)

■ Dijital abone hattı (DSL) bağlantısı da standart bir telefon hattı kullanır, ancak verileri standart bir dial-up modemden 20 kat daha hızlı iletebilir.

kablo modem

■ Kablo modemi, kablo şirketi tarafından sağlanan kablo TV hattını kullanır ve genellikle D SL'den daha hızlıdır.

ağ arayüz kartı (NIC)
yerel alan ağı (LAN)

■ Ağ arayüz kartı (NIC), bir bilgisayarı yerel alan ağına (LAN) bağlayan bir cihazdır ; Şekil 1.7'de gösterilmiştir. LAN'lar genellikle üniversitelerde, şirketlerde ve dev let dairelerinde kullanılır. 1000BaseT adı verilen yüksek hızlı bir NIC, verileri san iyede 1.000 milyon bit (mbps) hızında iletebilir.

milyon bit/saniye
(mbps)

■ Kablosuz ağ bağlantısı artık evlerde, işletmelerde ve okullarda son derece popülerdir. Bugün satılan her dizüstü bilgisayar, bilgisayarın yerel alan ağına ve İnternete bağlanmasını sağlayan bir kablosuz adaptörle donatılmıştır.



Not

Kontrol noktası sorularının cevapları Eşlikçi Web Sitesinde bulunmaktadır.

1.1 Donanım ve yazılım nedir?

1.2 Bir bilgisayarın beş ana donanım bileşenini listeleyin.

1.3 "CPU" kısaltması ne anlama geliyor?

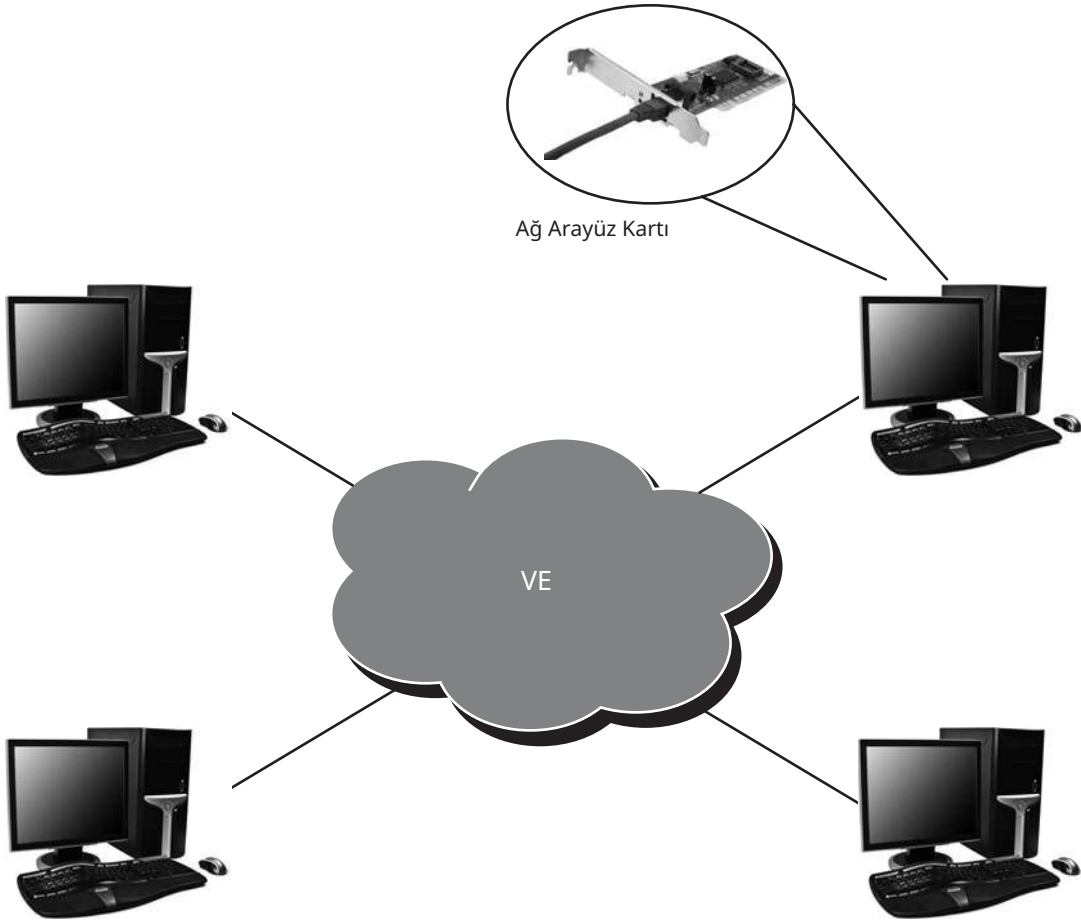


FIGURE 1.7 Yerel alan ağı, birbirine yakın bilgisayarları bağlar.

- 1.4** CPU hızını ölçmek için hangi birim kullanılır?
- 1.5** Bit nedir? Bayt nedir?
- 1.6** Hafıza ne için kullanılır? RAM neyin kısaltmasıdır? Hafıza neden RAM olarak adlandırılır?
- 1.7** Hafıza boyutunu ölçmek için hangi birim kullanılır?
- 1.8** Disk boyutunu ölçmek için hangi birim kullanılır?
- 1.9** Hafıza ile bir depolama cihazı arasındaki temel fark nedir?

1.3 Programlama Dilleri

Bilgisayar programları, yazılım olarak bilinir, bir bilgisayara ne yapması gerektiğini söyleyen talimatlardır.

Bilgisayarlar insan dillerini anlamaz, bu nedenle programlar bir bilgisayarın kullanabileceği bir dilde yazılmalıdır. Yüzlerce programlama dili vardır ve bunlar programlama sürecini insanların daha kolay hale getirmek için geliştirilmiştir. Ancak, tüm programlar bir bilgisayarı anlayabileceği bir dile dönüştürülmelidir.



1.3.1 Makine Dili

Bir bilgisayarın yerel dili, farklı bilgisayar türleri arasında farklılık gösterir, makine dilidir—gömülü ilkel talimatlar kümesidir. Bu talimatlar ikili kod şeklindedir, bu nedenle bir bilgisayara yerel dilinde bir talimat vermek istiyorsanız,

makine dili

10 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

talimatı ikili kod olarak girmek zorundasınız. Örneğin, iki sayıyı toplamak için, böyle bir talimatı ikili kodda yazmanız gerekebilir:

1101101010011010

1.3.2 Montaj Dili

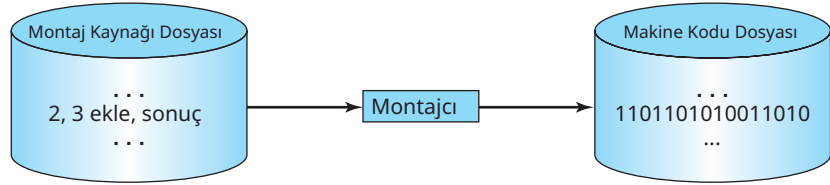
montaj dili

Makine dilinde programlama zahmetli bir süreçtir. Ayrıca, makine dilinde yazılan programlar çok zor okunur ve değiştirilir. Bu nedenle, montaj dili, makine dillerine alternatif olarak bilgisayarın ilk günlerinde oluşturulmuştur. Montaj dili, her bir makine dil talimatını temsil etmek için mnemonik olarak bilinen kısa bir tanımlayıcı kelime kullanır. Örneğin, **add** mnemonik kelimesi genellikle sayıları toplamak anlamına gelir ve **sub** sayıları çıkarmak anlamına gelir. 2 ve 3 sayılarının toplamını almak için, montaj kodunda şöyle bir talimat yazabilirsiniz:

2, 3, sonuç ekle

montajcı

Montaj dilleri, programlamayı daha kolay hale getirmek için geliştirilmiştir. Ancak, bilgisayar montaj dilini anlayamadığı için, montaj dili programlarını makine koduna çevirmek için başka bir program— *montajcı*—kullanılır; bu, Şekil 1.8'de gösterilmiştir.



ŞEKİL 1.8 Bir montajcı, montaj dili talimatlarını makine koduna çevirir.

düşük seviyeli dil

Montaj dilinde kod yazmak, makine dilinde yazmaktan daha kolaydır. Ancak, montaj dilinde kod yazmak yine de zahmetlidir. Montaj dilindeki bir talimat esasen makine kodundaki bir talimata karşılık gelir. Montaj dilinde yazmak, CPU'nun nasıl çalıştığını bilmenizi gerektirir. Montaj dili, makine diline doğası gereği yakın olduğu ve makineye bağımlı olduğu için *düşük seviyeli dil* olarak adlandırılır.

1.3.3 Yüksek Düzey Dil

yüksek seviyeli dil

1950'lerde, yüksek düzey diller olarak bilinen yeni bir programlama dilleri nesli ortaya çıktı. Platformdan bağımsızdılar, bu da yüksek düzey bir dilde bir program yazabileceğiniz ve bunu farklı türde makinelerde çalıştırabileceğiniz anlamına gelir. Yüksek düzey diller, İngilizce benzeri ve öğrenmesi ve kullanması kolaydır. Yüksek düzey bir programlama dilindeki talimatlara ifadeler denir. İşte, örneğin, bir yarıçapı 5 olan bir dairenin alanını hesaplayan yüksek düzey bir dil ifadesi:

alan = 5 * 5 * 3.1415

kaynak program

Birçok yüksek düzey programlama dili vardır ve her biri belirli bir amaç için tasarlanmıştır. Tablo 1.1, bazı popüler olanları listelemektedir.

kaynak kodu

Yüksek düzey bir dilde yazılan bir programa kaynak program veya kaynak kod denir. Bir bilgisayar bir kaynak programı anlayamadığı için, bir kaynak programın çalıştırılması için makine koduna çevrilmesi gerekir. Çeviri, bir yorumlayıcı veya bir derleyici olarak adlandırılan başka bir programlama aracı kullanılarak yapılabilir.

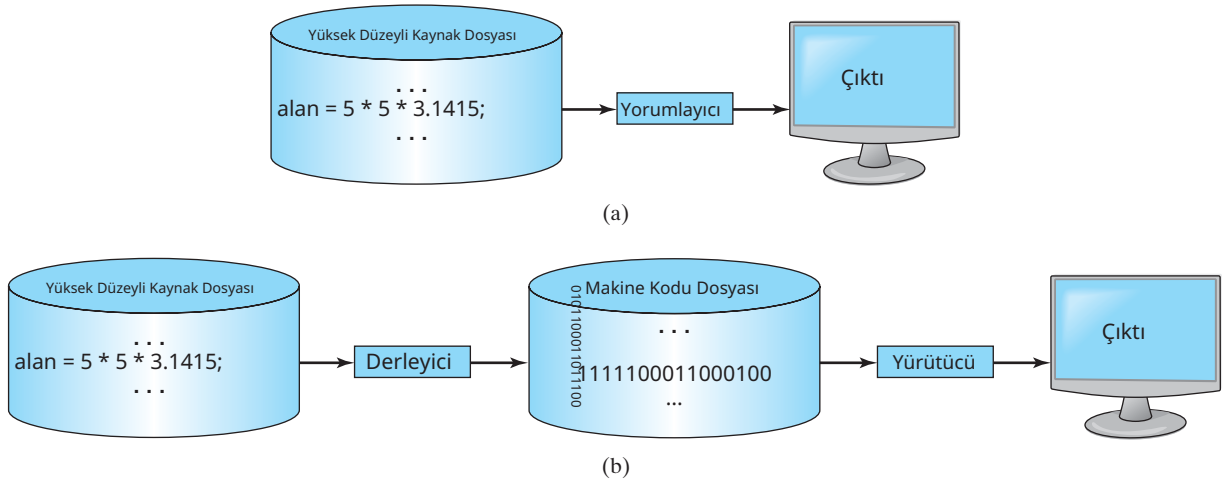
yorumlayıcı

derleyici

■ Bir yorumlayıcı, kaynak koddan bir ifadeyi okur, bunu makine koduna veya sanal makine koduna çevirir ve hemen ardından çalıştırır; bu, Şekil 1.9a'da gösterilmiştir.

TABLO 1.1 Popüler Yüksek Seviyeli Programlama Dilleri

Dil	Açıklama
Orada	Ada Lovelace için adlandırılmıştır, mekanik genel amaçlı bilgisayarlar üzerinde çalışmıştır. Ada dili, Savunma Bakanlığı için geliştirilmiş ve esasen savunma projelerinde kullanılmaktadır.
TEMEL	Başlangıç Seviyesi Çok Amaçlı Sembolik Talimat Kodu. Başlangıç seviyesindeki kullanıcılar tarafından kolayca öğrenilmesi ve kullanılması için tasarlanmıştır.
C	Bell Laboratuvarları'nda geliştirilmiştir. C, bir montaj dilinin gücünü yüksek seviyeli bir dilin kullanım kolaylığı ve taşınabilirliği ile birleştirir.
C++	C++, C tabanlı bir nesne yönelimli dildir.
C#	"C Sharp" olarak telaffuz edilir. Java ve C++'ın bir hibritidir ve Microsoft tarafından geliştirilmiştir.
COBOL	COMmon Business Oriented Language. İş uygulamaları için kullanılır.
FORTRAN	FORMül çevirisi. Bilimsel ve matematiksel uygulamalar için popülerdir.
Java	Sun Microsystems tarafından geliştirildi, şimdi Oracle'ın bir parçası. Platformdan bağımsız İnternet uygulamaları geliştirmek için yaygın olarak kullanılmaktadır.
Pascal'ın	Blaise Pascal'dan adını almıştır, on yedinci yüzyılda hesaplama makinelerinin öncüsüdür. Temel olarak programlama öğretimi için kullanılan basit, yapılandırılmış, genel amaçlı bir dildir.
Piton	Kısa programlar yazmak için iyi olan basit bir genel amaçlı betik dilidir.
Visual Basic	Visual Basic, Microsoft tarafından geliştirilmiştir ve programcıların grafiksel kullanıcı arayüzlerini hızlı bir şekilde geliştirmelerini sağlar.



FIGÖRÜNTÜ 1.9 (a) Bir yorumlayıcı, bir programı bir ifade bir seferde çevirir ve yürütür. (b) Bir derleyici, tüm kaynak kodu programı yürütmeye için bir makine dili dosyasına çevirir.

Kaynak koddan bir ifadenin birkaç makine talimatına çevrilebileceğini unutmayın.

- Bir derleyici, tüm kaynak kodunu bir makine kodu dosyasına çevirir ve ardından makine kodu dosyası çalıştırılır; bu, Şekil 1.9b'de gösterilmiştir.

1.10 CPU hangi dili anlar?

1.11 Assembly dili nedir?

12 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

MyProgrammingLab™

- 1.12 Assembler nedir?
- 1.13 Yüksek seviyeli programlama dili nedir?
- 1.14 Kaynak program nedir?
- 1.15 Yorumlayıcı nedir?
- 1.16 Derleyici nedir?
- 1.17 Yorumlanan bir dil ile derlenen bir dil arasındaki fark nedir?

1.4 İşletim Sistemleri



İşletim sistemi (OS), bir bilgisayarda çalışan en önemli programdır. OS, bir bilgisayarın aktivitelerini yönetir ve kontrol eder.

işletim sistemi (OS)

Genel amaçlı bilgisayarlar için popüler işletim sistemleri Microsoft Windows, MacOS ve Linux'tur. Bir Web tarayıcısı veya bir kelime işlemci gibi uygulama programları, bir işletim sistemi yüklenip çalışmadıkça çalışamaz. Şekil 1.10, donanım, işletim sistemi, uygulama yazılımı ve kullanıcı arasındaki karşılıklı ilişkiyi göstermektedir.

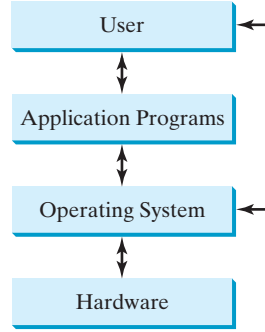


FIGURE 1.10 Kullanıcılar ve uygulamalar, bilgisayarın donanımına işletim sistemi aracılığıyla erişir.

Bir işletim sisteminin ana görevleri şunlardır:

- Sistem aktivitelerini kontrol etme ve izleme
- Sistem kaynaklarını tahsis etme ve atama
- İşlemleri planlama

1.4.1 Sistem Aktivitelerini Kontrol Etme ve İzleme

İşletim sistemleri, klavyeden gelen girişi tanıma, monitöre çıktı gönderme, depolama aygıtlarındaki dosya ve klasörleri takip etme ve disk sürücülerini ve yazıcılar gibi çevresel aygıtları kontrol etme gibi temel görevleri yerine getirir. Bir işletim sistemi ayrıca, aynı anda çalışan farklı programların ve kullanıcıların birbirine müdahale etmemesini sağlamalıdır. Ayrıca, OS güvenlikten de sorumludur; yetkisiz kullanıcıların ve programların sisteme erişmemesini sağlamalıdır.

1.4.2 Sistem Kaynaklarını Tahsis Etme ve Atama

İşletim sistemi, bir programın ihtiyaç duyduğu bilgisayar kaynaklarını (CPU zamanı, bellek alanı, diskler, giriş ve çıkış cihazları gibi) belirlemekten ve bunları programı çalıştırmak için tahsis etmekten ve atamaktan sorumludur.

1.4.3 İşlemleri Zamanlama

İşletim sistemi, sistem kaynaklarının verimli bir şekilde kullanılmasını sağlamak için programların aktivitelerini zamanlamaktan sorumludur. Günümüzün birçok işletim sistemi, sistem performansını artırmak için çoklu programlama, çoklu iş parçacığı ve çoklu işlem gibi teknikleri desteklemektedir.

Çoklu programlama, aynı CPU'yu paylaşılarak birden fazla programın aynı anda çalışmasına olanak tanır. CPU, bilgisayarın diğer bileşenlerinden çok daha hızlıdır. Sonuç olarak, çoğu zaman boşta kalır; örneğin, bir diskten veri aktarılmasını beklerken veya diğer sistem kaynaklarının yanıt vermesini beklerken. Çoklu programlama işletim sistemi, CPU'nun boşta kalacağı durumlarda birden fazla programın CPU'yu kullanmasına izin vererek bu durumdan faydalanır. Örneğin, çoklu programlama, bir kelime işlemci kullanarak bir dosyayı düzenlerken aynı anda web tarayıcınızın bir dosyayı indirmesine olanak tanır.

Çoklu iş parçacığı, tek bir programın aynı anda birden fazla görevi yerine getirmesine olanak tanır. Örneğin, bir kelime işleme programı, kullanıcıların metni aynı anda düzenlemesine ve bir diske kaydetmesine olanak tanır. Bu örnekte, düzenleme ve kaydetme, aynı uygulama içindeki iki görevdir. Bu iki görev eşzamanlı olarak çalışabilir.

Çoklu işlem veya paralel işlem, iki veya daha fazla işlemciyi bir arada kullanarak alt görevleri eşzamanlı olarak yerine getirir ve ardından alt görevlerin çözümlerini birleştirerek tüm görev için bir çözüm elde eder. Bu, birkaç doktorun bir hasta üzerinde birlikte çalıştığı cerrahi bir operasyona benzer.

çoklu programlama

çoklu iş parçacığı

çoklu işlem

1.18 Bir işletim sistemi nedir? Bazı popüler işletim sistemlerini listeleyin.

1.19 Bir işletim sisteminin ana sorumlulukları nelerdir?

1.20 Çoklu programlama, çoklu iş parçacığı ve çoklu işlem nedir?

Point Kontrol

MyProgrammingLab™

1.5 Java, Dünya Çapında Web ve Ötesi

Java, mobil cihazlar, masaüstü bilgisayarlar ve sunucular üzerinde çalışan yazılımlar geliştirmek için güçlü ve çok yönlü bir programlama dilidir.

Anahtar Nokta

Bu kitap Java programlamasını tanıtmaktadır. Java, Sun Microsystems'ta James Gosling liderliğindeki bir ekip tarafından geliştirilmiştir. Sun Microsystems, 2010 yılında Oracle tarafından satın alınmıştır. İlk olarak *Meşe*, Java, 1991 yılında tüketici elektroniği cihazlarındaki gömülü yongalar için kullanılmak üzere tasarlanmıştır. 1995'te, *Java* olarak yeniden adlandırılmış ve Web uygulamaları geliştirmek için yeniden tasarlanmıştır. Java'nın tarihi için, www.java.com/en/javahistory/index.jsp adresine bakın.

Java son derece popüler hale gelmiştir. Hızlı yükselişi ve geniş kabulü, tasarım özelliklerine, özellikle de bir programı bir kez yazıp her yerde çalıştırabileceğiniz vaadine dayandırılabilir. Tasarımcısının belirttiği gibi, Java sade, nesne yönelimli, dağıtık, yorumlanmış, sağlam, güvenli, mimari olarak tarafsız, taşınabilir, yüksek performanslı, çok iş parçacıklı ve dinamik'dir. Java özelliklerinin anatomisi için, www.cs.armstrong.edu/liang/JavaCharacteristics.pdf adresine bakın.

Java, sağlam misyon kritik uygulamalar geliştirmek için kullanılabilen, tam özellikli, genel amaçlı bir programlama dilidir. Bugün, yalnızca Web programlama için değil, aynı zamanda sunucular, masaüstü bilgisayarlar ve mobil cihazlar üzerinde bağımsız uygulamalar geliştirmek için de kullanılmaktadır. Mars'taki robotik keşif aracıyla iletişim kurmak ve onu kontrol etmek için kod geliştirmek için kullanıldı. Bir zamanlar Java'yı daha çok abartı olarak gören birçok şirket, şimdi onu müşteriler ve ortaklar tarafından İnternet üzerinden erişilen dağıtık uygulamalar oluşturmak için kullanıyor. Bugün geliştirilen her yeni proje için şirketler, işlerini kolaylaştırmak için Java'yı nasıl kullanabileceklerini soruyorlar.

Dünya Çapında Ağ, dünyanın herhangi bir yerinden İnternet üzerinden erişilebilen elektronik bir bilgi deposudur. İnternet, Web'in altyapısı, kırk yıldan fazla bir süredir var. Renkli Dünya Çapında Ağ ve sofistike Web tarayıcıları, İnternet'i n popüleritesinin başlıca nedenidir.

14 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

küçük uygulama

HTML

Java, Java programlarının bir Web tarayıcısından çalıştırılabilmesi nedeniyle başlangıçta çekici hale geldi. Bu tür programlara *applet* denir. Applet'ler, kullanıcılarla etkileşimde bulunmak ve taleplerini işlemek için Web'de düğmeler,metin alanları, metin kutuları, radyo düğmeleri vb. ile modern bir grafik arayüz kullanır. Applet'ler, Web'i duyarlı, etkileşimli ve kullanımı eğlenceli hale getirir . Applet'ler, bir HTML dosyasına gömülüdür. *HTML (Hiper Metin İşaretleme Dili)*, belgeleri düzenlemek, belgeleri İnternet'te bağlamak ve görüntüleri,sesleri ve videoları Web'de canlandırmak için basit bir betik dilidir. Şekil 1.11, bir Web tarayıcısından bir üçlü oyun oynamak için çalışan bir applet'i göstermektedir.

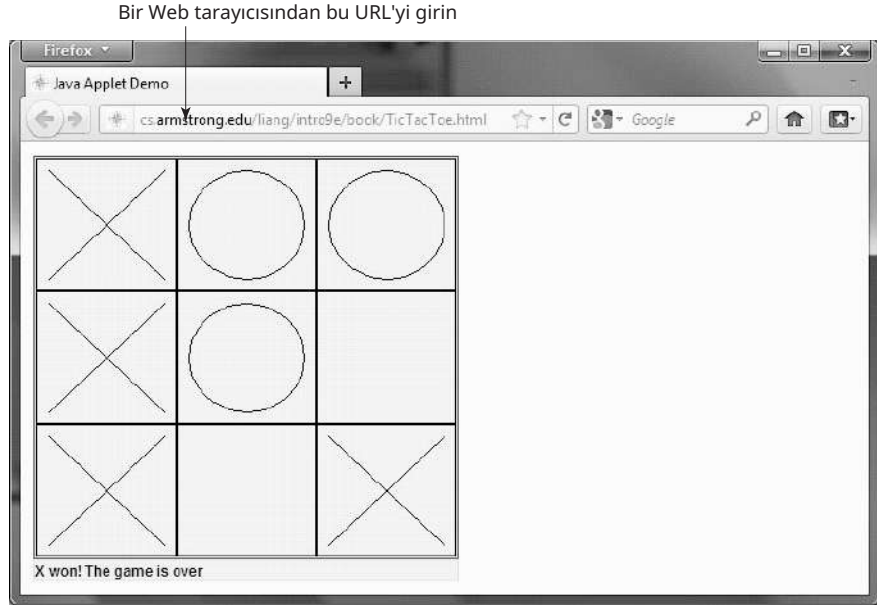


Fig.1.11 Tic-tac-toe oynamak için bir Java appleti bir Web tarayıcısından çalışır.



Ttftp

Java applet'lerinin bir gösterimi için java.sun.com/applets adresini ziyaret edin. Bu site, zengin bir Java kaynağı ve diğer harika applet demo sitelerine bağlantılar sağlar.

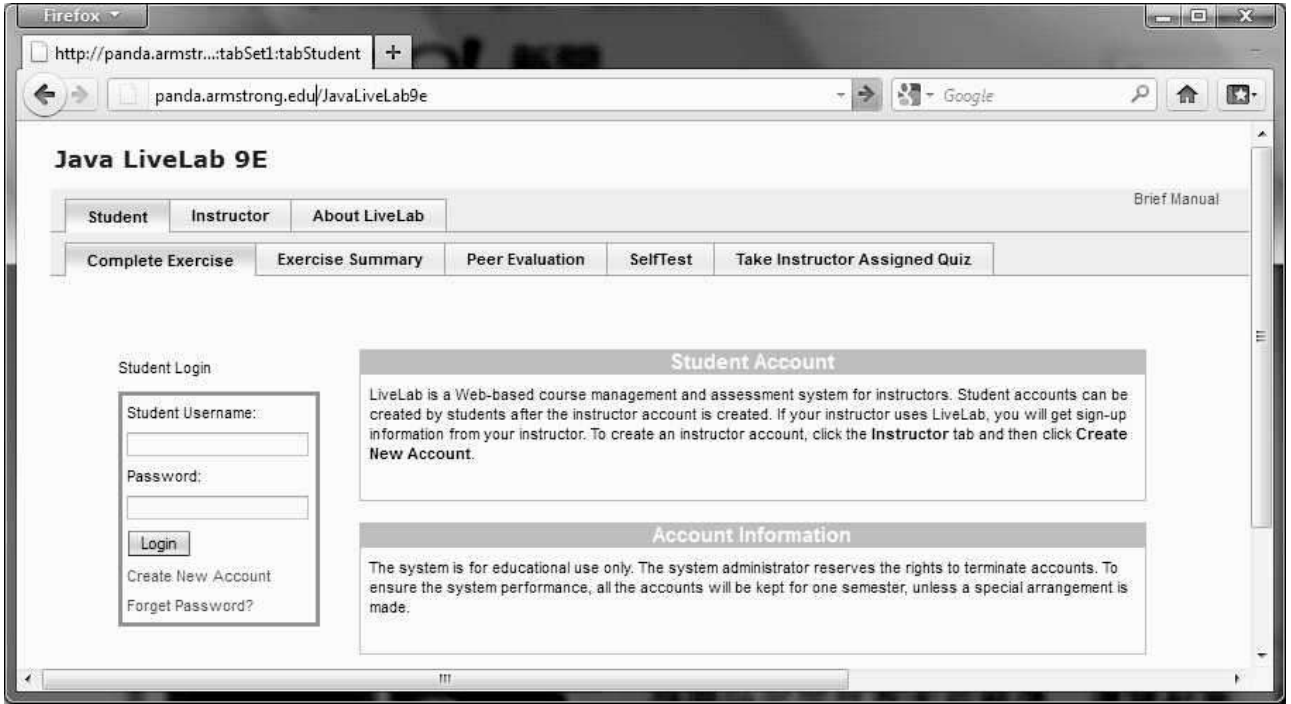
Java, artık Web sunucularında uygulama geliştirmek için çok popülerdir. Bu uygulamalar, d ata işler, hesaplamalar yapar ve dinamik Web sayfaları oluşturur. Şekil 1.12'de gösterilen ve b u kitapla birlikte kullanabileceğiniz LiveLab otomatiknotlandırma sistemi Java kullanılarak geliştirilmiştir.

Java, çok yönlü bir programlama dilidir: Masaüstü bilgisayarlar, sunucular ve küçük taşınabilir cihazlar için uygulamalar geliştirmek için kullanılabilir. Android cep telefonları için yazılım Java kull anılarak geliştirilmiştir. Şekil 1.13, Android telefon uygulamaları geliştirmek için bir emülatörün gös termektedir.

- 1.21** Java'yı kim icat etti? Şu anda Java'nın sahibi hangi şirket?
- 1.22** Java applet'i nedir?
- 1.23** Android hangi programlama dilini kullanıyor?

Point Kontrol

MyProgrammingLab™



Fikre 1.12 Java, bu kitabın eşlik eden otomatik notlandırma sistemi LiveLab'ı geliştirmek için kullanıldı.



Fikre 1.13 Java, Android telefonlarında kullanılır.

1.6 Java Dili Spesifikasyonu, API, JDK ve IDE



Java sözdizimi, Java dili spesifikasyonunda tanımlanmıştır ve Java kütüphanesi Java API'sinde tanımlanmıştır. JDK, Java programları geliştirmek ve çalıştırmak için kullanılan yazılımdır. IDE, programları hızlı bir şekilde geliştirmek için entegre bir geliştirme ortamıdır.

Bilgisayar dilleri katı kullanım kurallarına sahiptir. Bir program yazarken kurallara uymazsanız, bilgisayar bunu anlayamaz. Java dili spesifikasyonu ve Java API, Java standartlarını tanımlar.

Java dil spesifikasyonu

Java dili spesifikasyonu, Java programlama dilinin sözdizimi ve anlamsal yapısının teknik bir tanımıdır. Tam Java dili spesifikasyonunu

[Java.Sun.com/docs/books/Hatta bacaklar](http://Java.Sun.com/docs/books/Hatta%20bacaklar).

API

kütüphanesi

Uygulama programı arayüzü (API), kütüphane olarak da bilinir, Java programları geliştirmek için önceden tanımlanmış sınıflar ve arayüzler içerir. API hâlâ genişlemektedir. Java API'nin en son sürümünü www.oracle.com/technetwork/java/index.html adresinden görüntüleyebilir ve indirebilirsiniz.

Java, birçok şekilde kullanılabilen tam teşekküllü ve güçlü bir dildir. Üç sürüm de gelir:

Java SE, EE ve ME

- **Java Standart Sürüm (Java SE)** istemci tarafında bağımsız uygulamalar veya applet'ler geliştirmek için.
- **Java Kurumsal Sürüm (Java EE)** Java servlet'leri, JavaServer Pages (JSP) ve JavaServer Faces (JSF) gibi sunucu tarafı uygulamaları geliştirmek için.
- **Java Mikro Sürüm (Java ME)** cep telefonları gibi mobil cihazlar için uygulamalar geliştirmek için.

Java Geliştirme Araç Takımı (JDK)

JDK 1.7t = JDK 7

Bu kitap, Java programlamasını tanıtmak için Java SE'yi kullanmaktadır. Java SE, diğer tüm Java teknolojilerinin temelini oluşturur. Java SE'nin birçok sürümü vardır. Bu kitaptaki kullanılan en son sürüm, Java SE 7'dir. Oracle, her sürümü bir *Java Geliştirme Kiti* (JDK). Java SE 7 için Java Geliştirme Kiti *JDK 1.7* (aynı zamanda *Java 7* veya *JDK 7* olarak da bilinir) olarak adlandırılmaktadır.

Entegre geliştirme ağı

JDK, Java programları geliştirmek ve test etmek için komut satırından çağrılan ayrı programlar setinden oluşur. JDK yerine, Java geliştirme aracı (örneğin, NetBeans, Eclipse ve TextPad) kullanabilirsiniz; bu yazılım, Java programlarını hızlı bir şekilde geliştirmek için entegre bir geliştirme ortamı (IDE) sağlar. Düzenleme, derleme, oluşturma, hata ayıklama ve çevrimiçi yardım, tek bir grafik kullanıcı arayüzünde entegre edilmiştir. Kaynak kodunu bir pencerede girer veya mevcut bir dosyayı bir pencerede açar ve ardından programı derlemek ve çalıştırmak için bir düğmeye veya menü öğesine tıklarsınız veya bir fonksiyon tuşuna basarsınız.



MyProgrammingLab™

1.24 Java dil spesifikasyonu nedir?

1.25 JDK neyin kısaltmasıdır?

1.26 IDE neyin kısaltmasıdır?

1.27 NetBeans ve Eclipse gibi araçlar Java'dan farklı diller mi, yoksa Java'nın lehçeleri veya uzantıları mı?

1.7 Basit Bir Java Programı

*Bir Java programı, sınıftaki **main** metodundan çalıştırılır.*



konsol nedir?

konsol girişi

konsol çıkışı

Şimdi, konsolda **Java'ya Hoş Geldiniz!** mesajını gösteren basit bir Java programı ile başlayalım. (*konsol* kelimesi, bir bilgisayarın metin girişi ve görüntüleme cihazını ifade eden eski bir bilgisayar terimidir. *Konsol girişi*, klavyeden giriş almak anlamına gelir ve *konsol çıkışı*, monitörde çıktı göstermek anlamına gelir.) Program, Liste 1.1'de gösterilmiştir.

LİSTELERİ.1Welcome.java

```
1 genel sınıfHoşgeldiniz {
2     public static voidmain(String[] args) {
3         // Konsolda "Java'ya Hoş Geldiniz!" mesajını göster
4         System.out.println("Java'ya Hoş Geldiniz!");
5     }
6 }
```

sınıf
main metodu
mesajı göster



VideoNot
İlk Java programınız



Java'ya Hoş Geldiniz!

Satır numaralarının yalnızca referans amaçlı olduğunu unutmayın; bunlar programın bir parçası değildir. Bu nedenle, programınızda satır numaralarını yazmayın.

satır numaraları

Satır 1 bir sınıf tanımlar. Her Java programının en az bir sınıfı olmalıdır. Her sınıfın bir adı vardır. Gelenek olarak, sınıf adları büyük harfle başlar. Bu örnekte, sınıf adı **Welcome**.

sınıf adı

Satır 2 **main** metodunu tanımlar. Program **main** metodundan yürütülür. Bir sınıf birden fazla metod içerebilir. **main** metodu, programın yürütülmeye başlandığı giriş noktasıdır.

ana yöntem

Bir metod, ifadeleri içeren bir yapıdır. Bu programdaki **main** metodu, **System.out.println** ifadesini içerir. Bu ifade, dizeyi **Java'ya Hoş Geldiniz!** konsolda (satır 4) gösterir. **String**, bir karakter dizisini ifade eden bir programlama terimidir. Bir dize çift tırnak işaretleri içinde bulunmalıdır. Java'daki her ifade bir noktalı virgül (;) ile biter, bu da ifade sonlandırıcısı olarak bilinir.

dize

Rezerve edilmiş kelimeler veya *anahtar kelimeler*, derleyici için belirli bir anlama sahiptir ve programda başka amaçlar için kullanılamaz. Örneğin, derleyici **class** kelimesini gördüğünde, **class** kelimesinden sonra gelen kelimenin sınıf adı olduğunu anlar. Bu programdaki diğer rezerve edilmiş kelimeler **public**, **static** ve **void**'dir.

ifade sonlandırıcı
ayrılmış kelime
anahtar kelime

3. Satır, programın ne olduğunu ve nasıl inşa edildiğini belgeleyen bir *yorum* dir. Yorumlar, programcıların programı iletmesine ve anlamasına yardımcı olur. Bunlar programlama ifadeleri değildir ve bu nedenle derleyici tarafından göz ardı edilir. Java'da, yorumlar bir satırda iki eğik çizgi (//) ile başlar, buna *satır yorumu* denir veya bir veya birkaç satır arasında /* ve */ ile kaplanır, buna *blok yorumu* veya *paragraf yorumu* denir. Derleyici // gördüğünde, aynı satırdaki // sonrasındaki tüm metni göz ardı eder. /* gördüğünde, bir sonraki */ için tarar ve /* ile */ arasındaki metni göz ardı eder. İşte yorum örnekleri:

yorum

```
// Bu uygulama programı Java'ya Hoş Geldiniz! mesajını görüntüler!
/* Bu uygulama programı Java'ya Hoş Geldiniz! mesajını görüntüler! */
/* Bu uygulama programı
   Java'ya Hoş Geldiniz! mesajını görüntüler! */
```

satır yorumu
blok yorumu

Bir programdaki bir çift süslü parantez, programın bileşenlerini gruplandıran bir *blok* oluşturur. Java'da, her blok bir açılış parantezi ({) ile başlar ve bir kapanış parantezi (}) ile biter. Her sınıfın, sınıf verilerini ve yöntemlerini gruplandıran bir *sınıf bloğu* vardır. Benzer şekilde, her yöntemin, yöntemdeki ifadeleri gruplandıran bir *yöntem bloğu* vardır. Bloklar iç içe olabilir, yani bir blok başka bir bloğun içine yerleştirilebilir, aşağıdaki kodda gösterildiği gibi.

blok



Tt tip

Açılış süslü parantez bir kapanış süslü parantez ile eşleşmelidir. Bir açılış

süslü parantez yazdığınızda, kayıp parantez hatasını önlemek için hemen bir kapanış süslü parantez yazın. Çoğu Java IDE'si her açılış süslü parantez için kapanış süslü parantezi otomatik olarak ekler.

parantezleri eşleştir

18 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Java'ya Hoş Geldiniz!");  
    }  
}
```

Sınıf bloğu
Metot bloğu



Dikkat

Java kaynak programları büyük/küçük harf duyarlıdır. Örneğin, programda `main` ifadesini `Main` ile değiştirmek yanlıştır.

Programda birkaç özel karakter (örneğin, `{ } .//.;`) gördünüz. Bunlar neredeyse her programda kullanılır. Tablo 1.2, bunların kullanımını özetlemektedir.

TABLO 1.2Özel Karakterler

Karakter	İsim	Açıklama
{ }	Açılış ve kapanış süslü parantezler	İfadeleri kapsayan bir bloğu belirtir.
()	Açılış ve kapanış parantezleri	Metotlarla birlikte kullanılır.
[]	Açılış ve kapanış köşeli parantezler	Bir diziyi belirtir.
//	Çift eğik çizgiler	Bir yorum satırını önceler.
" "	Açılış ve kapanış tırnak işaretleri	Bir diziyi (yani, karakter dizisini) kapsayın.
;	Noktalı virgül	Bir ifadenin sonunu işaretler.

Programlamayı öğrenirken yapacağınız en yaygın hatalar sözdizimi hataları olacaktır. Her programlama dili gibi, Java'nın da kendi sözdizimi vardır ve kurallara uygun kod yazmalısınız. Eğer programınız bir kuralı ihlal ederse—örneğin, noktalı virgül eksikse, bir süslü parantez ek sikse, bir tırnak işareti eksikse veya bir kelime yanlış yazılmışsa—Java derleyicisi sözdizimi hatalarını raporlayacaktır. Bu hatalarla programı derlemeyi deneyin ve derleyicinin ne bildirdiğine bakın.



Not

Muhtemelen neden `main` metodunun bu şekilde tanımlandığını ve neden `System.out.println(...)` konsolda bir mesaj görüntülemek için kullanıldığını merak ediyorsunuz. Şimdilik, bunun böyle yapıldığını kabul edin. Sorularınız sonraki bölümlerde tamamen yanıtlanacaktır.

Liste 1.1'deki program bir mesaj görüntüler. Programı anladığınızda, daha fazla mesaj görüntülemek için kolayca genişletebilirsiniz. Örneğin, programı üç mesaj görüntüleyecek şekilde yeniden yazabilirsiniz; bu, Liste 1.2'de gösterilmiştir.

LISTING 1.2WelcomeWithThreeMessages.java

```
1 genel sınıfWelcomeWithThreeMessages {  
2     public static voidtmain(String[] args) {  
3         System.out.println("Programlama eğlencelidir!");  
4         System.out.println("Önce Temeller");  
5         System.out.println("Sorun Odaklı");  
6     }  
7 }
```



Programlama eğlencelidir!
Temel İlkeler Öncelikle
Probleme Dayalı

büyük/küçük harf duyarlı

özel karakterler

yaygın hatalar

sözdizimi kuralları

sınıf
main metodu
mesajı göster

Ayrıca, matematiksel hesaplamalar yapabilir ve sonucu konsolda görüntüleyebilirsiniz.

Liste 1.3, değerlendirme örneği vermektedir. $\frac{10.5+2 \times 3}{45-3.5}$.

LISTE 1.3 ComputeExpression.java

```
1 genel classtComputeExpression {
2     public static void main(String[] args) {
3         System.out.println((t10.5t+t2t*t3t) / (t45tt3.5t));
4     }n5 }
```

sınıf
main metodu
hesaplama ifadesi

0.39759036144578314



Java'daki çarpma operatörü `*`dır. Gördüğünüz gibi, bir aritmetik ifadeyi Java ifadesine çevirmek oldukça basit bir süreçtir. Java ifadelerini 2. Bölümde daha ayrıntılı olarak tartışacağız.

- 1.28** Anahtar kelime nedir? Bazı Java anahtar kelimelerini listeyin.
- 1.29** Java büyük/küçük harf duyarlı mı? Java anahtar kelimeleri için durum nedir?
- 1.30** Yorum nedir? Yorum derleyici tarafından yok sayılır mı? Bir yorum satırını ve bir yorum paragrafını nasıl belirtirsiniz?
- 1.31** Konsolda bir dize görüntülemek için hangi ifadeyi kullanmalıyız?
- 1.32** Aşağıdaki kodun çıktısını gösterin:

```
genel sınıfTest {
    public static void main(String[] args) {
        System.out.println("3.5 * 4 / 2 - 2.5 şudur ");
        System.out.println(3.5*4/2-2.5);
    }
}
```

Point Kontrol
MyProgrammingLab™

1.8 Java Programı Oluşturma, Derleme ve Çalıştırma

Bir Java programını .java dosyasında kaydedersiniz ve bunu .class dosyasına derlersiniz. .class dosyası Java Sanal Makinesi tarafından çalıştırılır.

Programınızı oluşturup derlemeden çalıştırılmaz. Bu süreçtekarlayıcıdır, Şekil 1.14'te gösterildiği gibi. Programınızda derleme hataları varsa, bunları düzeltmek içinprogramı değiştirmeniz ve ardından yeniden derlemeniz gerekir. Programınızda çalışma zamanı hataları varsa veyadoğru sonuç üretmiyorsa, programı değiştirmeniz, yeniden derlemeniz ve tekrar çalıştırmanız gerekir.

Bir Java kaynak kodu dosyası oluşturmak ve düzenlemek için herhangi bir metin düzenleyici veya IDE kullanabilirsiniz. Bu bölüm, Java programlarını bir komut penceresinden nasıl oluşturacağınızı, derleyeceğinizi ve çalıştıracağınızı gösterir. Eğer Eclipse, NetBeans veya TextPad gibi bir IDE kullanmak isterseniz, eğitimler için Ek II'ye başvurun. Komut penceresinden, Notepad gibi bir metin düzenleyici kullanarak Java kaynak kodu dosyasını oluşturabilirsiniz, Şekil 1.15'te gösterildiği gibi.



Not

Kaynak dosyası .java uzantısıyla bitmeli ve tam olarak aynı

isimde olmalıdır, yani genel sınıf adıyla aynı olmalıdır. Örneğin, Liste 1.1'deki kaynak kodu için dosya Welcome.java olarak adlandırılmalıdır, çünkü genel sınıf adı Welcome'dir.



VideoNot

Eclipse kısa eğitimi

komut penceresi
IDE Ekleri



VideoNot

NetBeans kısa eğitimi

dosya adı

20 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

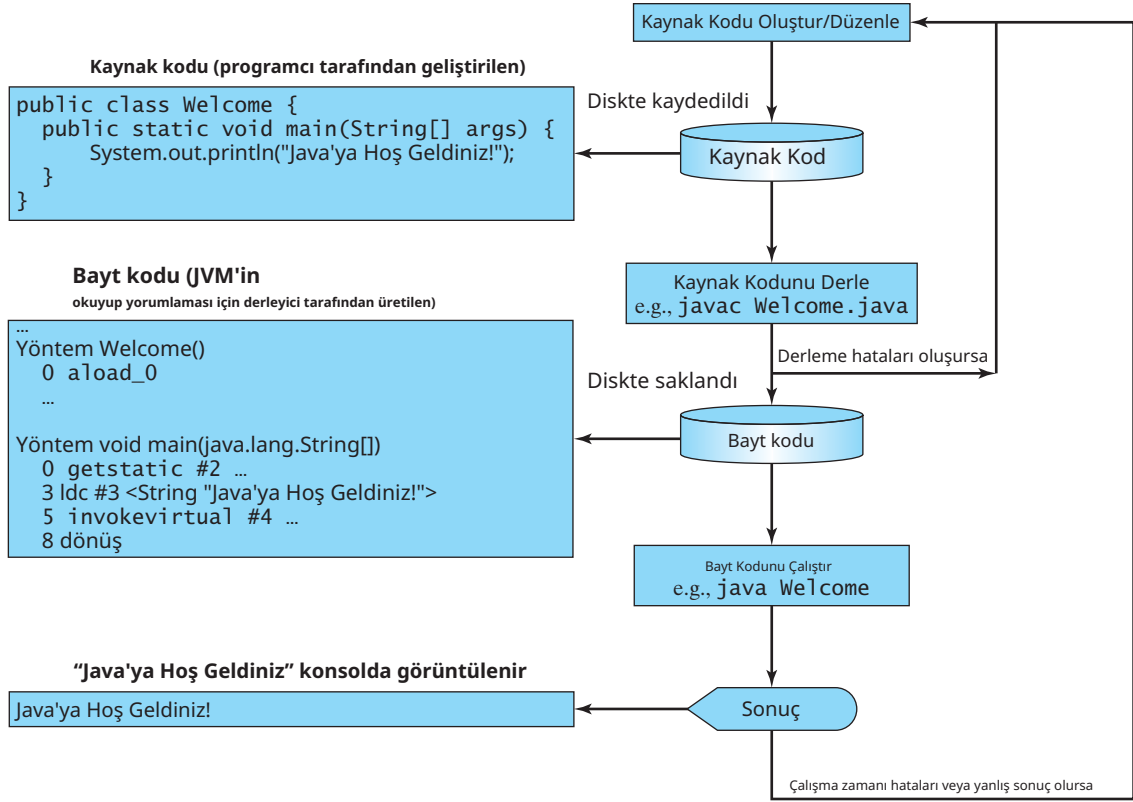


FIGURE 1.14 Java program geliştirme süreci, kaynak kodunu tekrar tekrar oluşturma/değiştirme, derleme ve programları çalıştırmaktan oluşur.

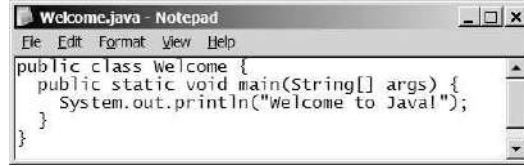


FIGURE 1.15 Windows Notepad kullanarak bir Java kaynak dosyası oluşturabilirsiniz.

derlemek

Bir Java derleyicisi, bir Java kaynak dosyasını bir Java bayt kodu dosyasına çevirir. Aşağıdaki komut, **Welcome.java** dosyasını derler:

javac hoş geldiniz.java



Not

Programları derleyip çalıştırmadan önce JDK'yi önce yükleyip yapılandırmanız. JDK'yi nasıl yükleyeceğinizi ve Java programlarınızı derleyip çalıştırmak için ortamı nasıl ayarlayacağınız hakkında bilgi için Ek I.B'ye, JDK 7'yi Yükleme ve Yapılandırma'ya bakın. Programları derleyip çalıştırmada sorun yaşıyorsanız, Komut Penceresinden Java'yı Derleme ve Çalıştırma hakkında bilgi için Ek I.C'ye bakın. Bu ek ayrıca temel DOS komutlarını nasıl kullanacağınızı ve dosya oluşturup düzenlemek için Windows Notepad ve WordPad'i nasıl kullanacağınızı açıklar. Tüm ekler Eşlikçi Web Sitesi'nden erişilebilir.

Ek I.B

Ek I.C

.class bayt kodu dosyası

Eğer herhangi bir sözdizimi hatası yoksa, **derleyici** bir **.class** uzantılı bayt kodu dosyası oluşturur. Böylece, yukarıdaki komut **Welcome.class** adında bir dosya oluşturur, bu da

1.8 Java Programı Oluşturma, Derleme ve Çalıştırma21

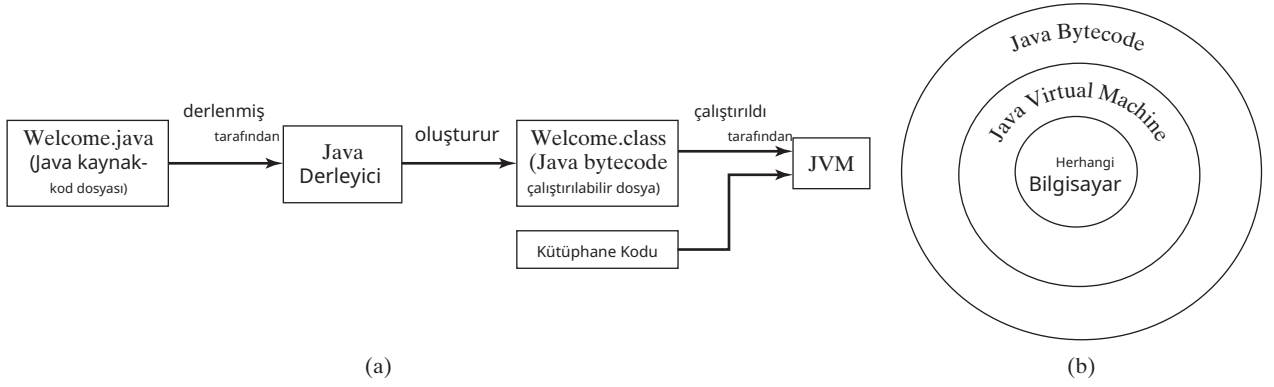


FIGURE 1.16(a) Java kaynak kodu bytecode'a dönüştürülür. (b) Java bytecode'u, herhangi bir bilgisayarda çalıştırılabilir Java Sanal Makinesi ile.

Şekil 1.16a. Java dili yüksek seviyeli bir dildir, ancak Java bayt kodu düşük seviyeli bir dildir. Bayt kodu, makine talimatlarına benzer, ancak mimariden bağımsızdır ve Şekil 1.16b'de gösterildiği gibi, bir Java Sanal Makinesi (JVM) olan herhangi bir platformda çalışabilir. Fiziksel bir makine yerine, sanal makine Java bayt kodunu yorumlayan bir programdır. Bu, Java'nın temel avantajlarından biridir: Java bayt kodu çeşitli donanım platformlarında ve işletim sistemlerinde çalışabilir. Java kaynak kodu Java bayt koduna derlenir ve Java bayt kodu JVM tarafından yorumlanır. Java kodunuz Java kütüphanesindeki kodu kullanabilir. JVM, kütüphane kodla birlikte kodunuzu çalıştırır.

Bir Java programını çalıştırmak, programın bayt kodunu çalıştırmak demektir. Bayt kodunu, bir yorumlayıcı olan JVM ile herhangi bir platformda çalıştırabilirsiniz. Bu, bayt kodundaki bireysel talimatları, tüm programı tek bir birim olarak değil, birer birer hedef makine dili koduna çevirir. Her adım, çevrildikten hemen sonra yürütülür.

Aşağıdaki komut, Liste 1.1 için bayt kodunu çalıştırır:

`java'ya hoş geldiniz`

Şekil 1.17, `javac` komutunu `Welcome.java` dosyasını derlemek için göstermektedir. Derleyici, `Welcome.class` dosyasını oluşturur ve bu dosya `java` komutu kullanılarak çalıştırılır.



Not

Basitlik ve tutarlılık açısından, bu kitapta kullanılan tüm kaynak kodu ve sınıf dosyaları cook altında yer almaktadır, aksi belirtilmedikçe.

```
Derle → c:\book>javac Welcome.java
Dosyaları göster → c:\book>dir Welcome.*
                    Volume in drive C has no label.
                    Volume Serial Number is 2EF7-CA93

                    Directory of c:\book

10/29/2011  03:43 PM                424 Welcome.class
10/29/2011  03:42 PM                176 Welcome.java
                2 File(s)                600 bytes
                0 Dir(s)  70,200,397,824 bytes free

Çalıştır → c:\book>java Welcome
            Welcome to Java!

c:\book>_
```

FIGURE 1.17Liste 1.1'in çıktısı "Java'ya Hoş Geldiniz!" mesajını gösterir.

bayt kodu
Java Sanal Makinesi (JVM)

bayt kodunu yorumla

çalıştır

javac komutu
java komutu

c:\book



VideoNot
Bir Java

programını derleyin ve çalıştırın

22 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

java SınıfAdı



Dikkat

Programı çalıştırırken komut satırında **.class** uzantısını kullanmayın. Programı çalıştırmak için **java ClassName** kullanın. Komut satırında **java ClassName e.class** kullanırsanız, sistem **ClassName.class.class** almaya çalışacaktır.

NoClassDefFoundError



Ttftp

Eğer var olmayan bir sınıf dosyasını çalıştırırsanız, bir **NoClassDefFoundError** oluşacaktır. Eğer bir sınıf dosyasını çalıştırırsanız ve bu dosyada bir **main** metodu yoksa veya **main** metodunu yanlış yazarsanız (örneğin, **Main** yerine **ma'in** yazarak), bir **NoSuchMethodError** oluşacaktır.

NoSuchMethodError

sınıf yükleyici



Not

Bir Java programı çalıştırıldığında, JVM önce sınıfın bytecode'unu belleğe yükler *class loader* adlı bir program kullanarak. Programınız diğer sınıfları kullanıyorsa, *class loader* bunları ihtiyaç duyulmadan hemen önce dinamik olarak yükler. Bir sınıf yüklendikten sonra, *JVMbytecode verifier* adlı bir program kullanarak bytecode'un geçerliliğini kontrol eder ve bytecode'unJava'nın güvenlik kısıtlamalarını ihlal etmediğinden emin olur. Java, Java sınıf dosyalarının değiştirilmediğinden ve bilgisayarınıza zarar vermediğinden emin olmak için sıkı güvenlik önlemleri uygular.

byte kodu doğrulayıcı

paket kullan



Petdagtotgtictal Ntote

Eğitmeniniz, programları düzenlemek için paketler kullanmanızı isteyebilir. Örneğin, bu bölümde ki tüm programları *chapter1* adlı bir pakette toplayabilirsiniz. Paketleri nasıl kullanacağınızla ilgili talimatlar için Ek I.F, Metindeki Sınıfları Düzenlemek için Paketleri Kullanma'ya bakın.

MyProgrammingLab™

Point Kontrol

- 1.33 Java kaynak dosya adı uzantısı nedir ve Java bytecode dosya adı uzantısı nedir?
- 1.34 Java derleyicisinin girdi ve çıktısı nedir?
- 1.35 Bir Java programını derlemek için komut nedir?
- 1.36 Bir Java programını çalıştırmak için komut nedir?
- 1.37 JVM nedir?
- 1.38 Java herhangi bir makinede çalışabilir mi? Bir bilgisayarda Java çalıştırmak için ne gereklidir?
- 1.39 Bir programı çalıştırdığınızda **NoClassDefFoundError** oluşursa, hatanın nedeni nedir?
- 1.40 Bir programı çalıştırdığınızda **NoSuchMethodError** oluşursa, hatanın nedeni nedir?

1.9 Mesaj Diyalog Kutusunda Metin Gösterme



Grafiksel bir diyalog kutusunda metin gösterebilirsiniz.

JOptionPane

showMessageDialog

Liste 1.1'deki program, metni konsolda gösterir; Şekil 1.17'de gösterildiği gibi. Metni bir mesaj diyalog kutusunda göstermek için programı yeniden yazabilirsiniz. Bunu yapmak için, *JOptionPane* sınıfındaki *showMessageDialog* yöntemi. *JOptionPane*, "tekerleği yeniden icat etmek" yerine yeniden kullanabileceğiniz Java kütüphanesindeki birçok önceden tanımlanmış sınıftan biridir. **showMessageDialog** yöntemini, Şekil 1.18'de gösterildiği gibi, bir mesaj iletişim kutusunda herhangi bir metni görüntülemek için kullanabilirsiniz. Yeni program, Liste 1.4'te verilmiştir.

LISTE 1.4WelcomeInMessageDialogBox.java

blok yorumu

```
1 /* Bu uygulama programı, mesaj iletişim kutusunda Java'ya Hoş Geldiniz! görüntüler.
2  * bir mesaj iletişim kutusunda.
3  */
```



FIGÜRİ.18 "Java'ya Hoş Geldiniz!" bir mesaj kutusunda görüntülenir.

```

4 import javax.swing.JOptionPane;
5
6 genel sınıf WelcomeInMessageDialogBox {
7     public static void main(String[] args) {
8         // Java'ya Hoş Geldiniz! mesaj iletişim kutusunda görüntüle
9         JOptionPane.showMessageDialog(null, "Java'ya Hoş Geldiniz!");
10    }

```

içe aktarmak
 ana metod
 mesajı göster

İlk üç satır blok yorumlarıdır. İlk satır `/*` ile başlar ve son satır `*/` ile biter. Geleneksel olarak, diğer tüm satırlar bir yıldız (*) ile başlar.

Bu program, Java sınıfı `JOptionPane` kullanır (satır 9). Java'nın önceden tanımlanmış sınıfları paketler halinde gruplanmıştır. `JOptionPane`, `javax.swing` paketindedir. `JOptionPane`, programda satır 4'teki `import` ifadesi kullanılarak içe aktarılır, böylece derleyici sınıfı tam adı olmadan `javax.swing.JOptionPane` ile bulabilir.



Not

Satır 9'daki `JOptionPane` ifadesini `javax.swing.JOptionPane` ile değiştirirseniz, line 4'te içe aktarmanıza gerek yoktur. `javax.swing.JOptionPane`, `JOptionPane` sınıfının tam adıdır.

`showMessageDialog` metodu bir *static* metodudur. Böyle bir metod, sınıf adının ardından bir nokta operatörü (.) ve argümanlarla birlikte metod adının kullanılmasıyla çağrılmalıdır. Yöntemlerin ayrıntıları Bölüm 5'te tartışılacaktır. `showMessageDialog` metodu iki argümanla çağrılabilir, aşağıda gösterildiği gibi.



`JOptionPane.showMessageDialog(null, "Java'ya Hoş Geldiniz!");`

İlk argüman her zaman `null` olabilir. `null`, Bölüm 8'de tamamen tartışılacak bir Java anahtar kelimesidir. İkinci argüman, gösterilecek metin için bir dizedir.

`showMessageDialog` metodunu kullanmanın birkaç yolu vardır. Şu anda, yalnızca iki yolu bilmeniz gerekiyor. Biri, örnekte gösterildiği gibi bir ifade kullanmaktır: iki versiyon
showMessageDialog

```
JOptionPane.showMessageDialog(null, x);
```

burada `x` görüntülenecek metin için bir dizedir.

Diğeri, bu gibi bir ifade kullanmaktır:

```
JOptionPane.showMessageDialog(null, x, ny, J
OptionPane.INFORMATION_MESSAGE);
```

24 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

burada **x** görüntülenecek metin için bir dizedir ve **y** mesaj kutusunun başlığı için bir dizedir. Dördüncü argüman **JOptionPane.INFORMATION_MESSAGE** olabilir, bu da görüntü simgesini () mesaj kutusunda gösterilecek, aşağıdaki gibi örnek.



```
JOptionPane.showMessageDialog(null,
    "Java'ya Hoş Geldiniz!",
    "Mesajı Göster",
    JOptionPane.INFORMATION_MESSAGE);
```



Not

İki tür **import** ifadesi vardır: *belirli import* ve *joker import*. *Belirli import*, **import** ifadesinde tek bir sınıfı belirtir. Örneğin, aşağıdaki ifade **JOptionPane**'i **javax.swing** paketinden içe aktarır.

```
import javax.swing.JOptionPane;
```

Joker import, joker olarak yıldız işaretini kullanarak bir paketdeki tüm sınıfları içe aktarır. Örneğin, aşağıdaki ifade paketten tüm sınıfları içe aktarır **javax.swing**.

```
import javax.swing.*;
```

İçe aktarılan bir paketdeki sınıflar hakkında bilgi, sınıf programda kullanılmadıkça derleme zamanında veya çalışma zamanında okunmaz. **Import** ifadesi, derleyiciye sınıfların nerede bulunacağını söyler. Belirli bir **import** ile joker **import** beyanı arasın da performans farkı yoktur.



Not

Listing 1.1'deki **System** sınıfını **System.out.println("Java'ya Hoş Geldiniz");** ifade sinde kullandığınızı hatırlayın. **System** sınıfı, **java.lang** paketinde olduğu için içe aktarılmamıştır. **java.lang** paketindeki tüm sınıflar her Java programında d olaylı olarak içe aktarılır.

belirli içe aktarma

joker içe aktarma

performans farkı yok

java.lang paketi

örtük olarak içe aktarılmıştır

Point Kontrol

MyProgrammingLab™

1.41 Bir mesaj diyalog kutusunda "Merhaba dünya" mesajını göstermek için ifade nedir?

1.42 Neden **System** sınıfının içe aktarılması gerekmez?

1.43 Aşağıdaki iki **import** ifadesi arasında herhangi bir performans farkı var mı?

```
import javax.swing.JOptionPane;
import javax.swing.*;
```

1.10 Programlama Stili ve Dokümantasyon

İyi programlama stili ve uygun dokümantasyon, bir programı okunabilir hale getirir ve programcıların hataları önlemesine yardımcı olur.



programlama stili

dokümantasyon

Programlama stili, programların nasıl görüldüğü ile ilgilidir. Bir program yalnızca bir satırda yazılmış olsa bile derlenip çalıştırılabilir, bununla birlikte hepsinin tek bir satırda yazılması kötü bir programlama stili olur çünkü okunması zor olur. *Dokümantasyon*, bir programa ilişkin açıklayıcı notlar ve yorumlar bütünüdür. Programlama stili ve dokümantasyon, kodlama kadar önemlidir. İyi programlama stili ve uygun dokümantasyon, hataların oluşma olasılığını azaltır ve programları okunması kolay hale getirir. Bu bölüm birkaç kılavuz sunmaktadır. İçin

daha ayrıntılı yönergeler için, Eşlikçi Web Sitesi'ndeki Ek I.D, Java Kodlama Stili Yönergelerine bakın.

1.10.1 Uygun Yorumlar ve Yorum Stilleri

Programın ne yaptığını, ana özelliklerini ve kullandığı herhangi bir benzersiz tekniği açıklayan bir özetle programın başında yer almalıdır. Uzun bir programda, her ana adımı tanıtan ve okunması zor olan her şeyi açıklayan yorumlar da eklemelisiniz. Yorumların, programı kalabalıklaştırmaması veya okunmasını zorlaştırmaması için özlü olması önemlidir.

Satır yorumlarına (başında `//` bulunan) ve blok yorumlarına (başında `/*` ek olarak, Java, *javadoc yorumları* olarak adlandırılan özel bir tür yorumları destekler. *javadoc* yorumları `/**` ile başlar ve `*/` ile biter. JDK'nın *javadoc* komutunu kullanarak bir HTML dosyasına çıkarılabilirler. Daha fazla bilgi için java.sun.com/j2se/javadoc adresine bakın.

javadoc yorumu

Bir sınıfın veya bir metodun tamamı hakkında yorum yapmak için *javadoc* yorumlarını (`/** ... */`) kullanın. Bu yorumlar, *javadoc* HTML dosyasına çıkarılabilmesi için sınıf veya metod başından önce gelmelidir. Bir metodun içindeki adımlar hakkında yorum yapmak için satır yorumlarını (`//`) kullanın. Bir *javadoc* HTML dosyasının örneğini görmek için www.cs.armstrong.edu/liang/javadoc/Exercise1.html adresine göz atın. İlgili Java kodu www.cs.armstrong.edu/liang/javadoc/

Exercise1.java adresinde gösterilmektedir.

1.10.2 Uygun Girinti ve Boşluk

Tutarlı bir girinti stili, programları net ve okunması, hata ayıklanması ve bakımı kolay hale getirir. *Girinti*, bir programın bileşenleri

kod girintisi

veya ifadeleri arasındaki yapısal ilişkileri göstermek için kullanılır. Java, tüm ifadeler aynı uzun satırda olsa bile programı okuyabilir, bununla birlikte insanlar, düzgün bir şekilde hizalanmış kodu okumayı ve bakımını yapmayı daha kolay bulurlar. Her alt-bileşeni veya ifadeyi, içinde bulunduğu yapının en az *iki* boşluk daha fazla olacak şekilde girintileyin.

Bir ikili operatörün her iki tarafına bir boşluk eklenmelidir, aşağıdaki ifadede gösterildiği gibi:

System.out.println(t3t+t4*t4t);	←	Kötü stil
System.out.println(t3t+t4*t4t);	←	İyi stil

1.10.3 Blok Stilleri

A *bloğu*, süslü parantezlerle çevrili bir dizi ifadedir. İki popüler stil vardır, *sonraki-satır* stili ve *satır-sonu* stili, aşağıda gösterildiği gibi.

```
genel sınıfTestn{
    public static voidtmain(String[] args){
        System.out.println("Blok Stilleri");
    }
}
```

Sonraki-satır stili

```
genel sınıfTest {
    public static voidmain(String[] args) {
        System.out.println("Blok Stilleri");
    }
}
```

Satır-sonu stiliSo

nraki-satır stili, süslü parantezleri dikey olarak hizalar ve programların okunmasını kolaylaştırır, oysasatır-sonu stili alan tasarrufu sağlar ve bazı ince programlama hatalarını önlemeye yardımcı olabilir. Her ikisi de kabul edilebilir blok stilleridir. Seçim, kişisel veya kurumsal tercihe bağlıdır. Bir blok stilini tutarlı bir şekilde kullanmalısınız—stilleri karıştırmak önerilmez. Bu kitap, Java API kaynak koduyla tutarlı olmak içinsatır-sonu stilini kullanmaktadır.

26 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

MyProgrammingLab™

Point Kontrol

1.44 Aşağıdaki programı programlama stili ve belgelendirme

kılavuzlarına göre yeniden formatlayın. Satır sonu süslemesi stilini kullanın.

```
genel sınıfTestn{

    // Ana yöntem
    public static void main(String[] args) {
        /** Çıktıyı göster */
        System.out.println("Java'ya Hoş Geldiniz");
    }
}
```

1.11 Programlama Hataları



Programlama hataları üç türe ayrılabilir: sözdizimi hataları, çalışma zamanı hataları, ve mantık hataları.

1.11.1 Sözdizimi Hataları

Derleyici tarafından tespit edilen hatalara sözdizimi hataları veya derleme hataları denir. Sözdizimi hataları, bir anahtar kelimeyi yanlış yazmak, bazı

gerekli noktalama işaretlerini atlamak veya bir açılış süslemesini karşılık gelen bir kapanış süslemesi olmadan kullanmak gibi kod yapısındaki hatalardan kaynaklanır.

Bu hatalar genellikle kolayca tespit edilir, çünkü derleyici size nerede olduklarını ve neyin neden olduğunu söyler. Örneğin, Liste 1.5'teki programda bir sözdizimi hatası vardır, bu

Şekil 1.19'da gösterilmiştir.

LISTE 1.5 ShowSyntaxErrors.java

```
1 genel classtShowSyntaxErrors {
2     public staticmain(String[] args) {
3         System.out.println("Java'ya Hoş Geldiniz");
4     }
5 }
```

Dört hata rapor edilmiştir, ancak programda aslında iki hata vardır:

- 2. satırda **main** önünde **void** anahtar kelimesi eksiktir.
- Dize **Java'ya Hoş Geldiniz**, 3. satırda bir kapanış tırnak işareti ile kapatılmalıdır.

Tek bir hata genellikle birçok derleme hatası göstereceğinden, hataları üst satırdan başlayarak düzeltmek iyi bir uygulamadır. Programda daha önce meydana gelen hataları düzeltmek, daha sonra meydana gelen ek hataları da düzeltebilir.

Derle

```
Administrator: Command Prompt
c:\book>javac ShowSyntaxErrors.java
ShowSyntaxErrors.java:2: error: invalid method declaration; return type required
    public staticmain(String[] args) {
                   ^
ShowSyntaxErrors.java:3: error: unclosed string literal
        System.out.println("Welcome to Java);
                           ^
ShowSyntaxErrors.java:3: error: ';' expected
        System.out.println("Welcome to Java);
                           ^
ShowSyntaxErrors.java:5: error: reached end of file while parsing
    }
    ^
4 errors
c:\book>
```

FIGÜRİ 1.19 Derleyici sözdizimi hatalarını bildirir.

**Ttitp**

Eğer nasıl düzelteceğinizi bilmiyorsanız, programınızı metindeki benzer örneklerle karakter karakter karşılaştırın. Bu dersin ilk birkaç haftasında, muhtemelen sözdizimi hatalarını düzeltmek için çok zaman harcayacaksınız. Yakında Java sözdizimine aşina olacaksınız ve sözdizimi hatalarını hızlı bir şekilde düzeltebileceksiniz.

sözdizimi hatalarını düzelt

1.11.2 Çalışma Zamanı Hataları

Çalışma zamanı hataları, bir programın anormal bir şekilde sona ermesine neden olan hatalardır. Bir program çalışırken, ortam gerçekleştirilemeyecek bir işlemi tespit ettiğinde meydana gelir.

çalışma zamanı hataları

Girdi hataları genellikle çalışma zamanı hatalarına neden olur. Bir *girdi hatası*, programın kullanıcıdan bir değer girmesini beklediği, ancak kullanıcının programın işleyemeyeceği bir değer girdiği durumlarda meydana gelir. Örneğin, program bir sayı okumayı bekliyorsa, ancak kullanıcı bir dize girse, bu programda veri türü hatalarının meydana gelmesine neden olur.

Çalışma zamanı hatalarına bir diğer örnek, sıfıra bölmedir. Bu, bölenin sıfır olduğu tam sayı bölmeleri için gerçekleşir. Örneğin, Liste 1.6'daki program bir çalışma zamanı

hatasına neden olur, Şekil 1.20'de gösterildiği gibi.

LİSTE 1.6 ShowRuntimeErrors.java

```
1 genel classtShowRuntimeErrors {
2     public static voidtmain(String[] args) {
3         System.out.println(t1t/t0t);
4     }n5 }
```

çalışma zamanı hatası



FIGİNİ.20 Çalışma zamanı hatası programın anormal bir şekilde sona ermesine neden olur.

1.11.3 Mantık Hataları

Mantık hataları, bir programın beklenildiği gibi çalışmadığında ortaya çıkar. Bu tür hatalar birçok farklı nedenle meydana gelir. Örneğin, Liste 1.7'deki programı Celsius 35 dereceyi Fahrenheit derecesine dönüştürmek için yazdığınızı varsayalım:

mantık hataları

LİSTE 1.7 ShowLogicErrors.java

```
1 genel classtShowLogicErrors {
2     public static voidtmain(String[] args) {
3         System.out.println("Celsius 35, Fahrenheit derecesidir ");
4         System.out.println((9 / 5) * 35 + 32);
5     }
```

Celsius 35, Fahrenheit derecesidir
67



28 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

Fahrenheit 67 derece alacaksınız, bu yanlıştır. Bu 95.0 olmalı. Java'da, tam sayılar için bölme bir tam sayıdır—kesirli kısım kesilir—bu nedenle Java'da $9 / 5$, 1'dir. Doğru sonucu almak için $9.0 / 5$ kullanmalısınız, bu da 1.8 sonucunu verir.

Genel olarak, sözdizimi hataları bulması ve düzeltmesi kolaydır, çünkü derleyici hataları nereden geldiği ve neden yanlış olduğu konusunda ipuçları verir. Çalışma zamanı hataları da zor değildir, çünkü hataların nedenleri ve yerleri programın sona erdiğinde konsolda görüntülenir. Öte yandan, mantık hatalarını bulmak çok zor olabilir. Önümüzdeki bölümlerde, programları izleme ve mantık hatalarını bulma tekniklerini öğreneceksiniz.



1.45 Sözdizimi hataları (derleme hataları), çalışma zamanı hataları ve mantık hataları nedir?

1.46 Sözdizimi hataları, çalışma zamanı hataları ve mantık hatalarına örnekler verin.

1.47 Bir dize üzerinde kapanış tırnak işareti koymayı unutursanız, hangi tür hata oluşur?

1.48 Programınızın tam sayıları okuması gerekiyorsa, ancak kullanıcı dizeleri girdiyse, bu programı çalıştırırken bir hata oluşur. Bu hangi tür bir hatadır?

1.49 Bir dikdörtgenin çevresini hesaplamak için bir program yazdığınızı varsayalım ve programınızı yanlışlıkla bir dikdörtgenin alanını hesaplayacak şekilde yazarsanız. Bu hangi tür bir hatadır?

1.50 Aşağıdaki koddaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfHoşgeldiniz {
2     public voidtMain(String[] args) {
3         System.out.println('Java'ya Hoş Geldiniz!);
4     }
5 }
```

1.51 Aşağıdaki program yanlıştır. Programın şu şekilde görüntülemesi için satırları yeniden sıralayın **sabah** ardından **öğleden sonra**.

```
1     public static voidtmain(String[] args) {
2     }
3     genel sınıfHoşgeldiniz {
4         System.out.println("öğleden sonra");
5         System.out.println(" Sabah ");
6     }
```

ANAHTAR TERİMLER

Uygulama Programı Arayüzü (API) 16

montajcı 10

montaj dili 10

bit 4

blok 17

blok yorumu 17

veriyolu 2

bayt 4

bayt kodu 21

bayt kodu doğrulayıcı 22

kablo modem 8

merkezi işlem birimi (CPU) 3

sınıf yükleyici 22

yorum 17

derleyici 10

konsol 16

nokta aralığı 8

DSL (dijital abone hattı) 8

kodlama şeması 4

donanım 2

yüksek seviyeli dil 10

entegre geliştirme ortamı
(IDE) 16

yorumlayıcı 10

java komutu 21

Java Geliştirme Kiti (JDK) 16

Java dil spesifikasyonu 16

Java Sanal Makinesi (JVM) 21

javac komutu 21

anahtar kelime (veya ayrılmış kelime) 17

kütüphane 16

satır yorumu 17

mantık hatası 27
 düşük seviyeli dil 10
 makine dili 9
 ana yöntem 17
 bellek 5
 modem 8
 anakart 3
 ağ arayüz kartı (NIC) 8
 işletim sistemi (OS) 12
 piksel 8
 program 2

programlama 2
 çalışma zamanı hatası 27
 ekran çözünürlüğü 8
 yazılım 2
 kaynak kodu 10
 kaynak program 10
 belirli ithalat 24
 ifade 10
 depolama aygıtları 5
 sözdizimi hatası 26
 joker ithalat 24



Not

Yukarıdaki terimler bu bölümde tanımlanmıştır. Ek I.A, Sözlük, kitapta yer alan tüm anahtar terimleri ve tanımları, bölümlerine göre düzenlenmiş olarak listeler.

Ek I.A

BİLGİSAYAR ÖZETİ

1. Bir bilgisayar, verileri depolayan ve işleyen elektronik bir cihazdır.
2. Bir bilgisayar hem *donanım* hem de *yazılımı* içerir.
3. Donanım, dokunulabilen bilgisayarın fiziksel yönüdür.
4. Bilgisayar programları, yazılım olarak bilinir, donanımı kontrol eden ve görevleri yerine getirmesini sağlayan görünmez talimatlardır.
5. Bilgisayar programlama, bilgisayarların gerçekleştirmesi için talimatların (yani, kodun) yazılmasıdır.
6. Merkezi işlem birimi (CPU), bir bilgisayarın beynidir. Bellekten talimatları alır ve bunları yürütür.
7. Bilgisayarlar, dijital cihazların iki kararlı durumu olduğu için sıfır ve bir kullanır; bunlar geleneksel olarak sıfır ve bir olarak adlandırılır.
8. Bir bit, 0 veya 1 olan bir ikili basamaktır.
9. Bir bayt, 8 bitlik bir dizidir.
10. Bir kilobayt yaklaşık 1.000 bayt, bir megabayt yaklaşık 1 milyon bayt, bir gigabayt yaklaşık 1 milyar bayt ve bir terabayt yaklaşık 1.000 gigabayttır.
11. Bellek, CPU'nun yürütmesi için veri ve program talimatlarını depolar.
12. Bir bellek birimi, sıralı bir bayt dizisidir.
13. Bellek, güç kapatıldığında bilgilerin kaybolması nedeniyle uçucudur.

30 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

14. Programlar ve veriler, *depolama aygıtlarında* kalıcı olarak saklanır ve bilgisayar bunları gerçekten kullandığında belleğe taşınır.
15. makine dili, her bilgisayara yerleştirilmiş bir dizi ilkel talimattır.
16. Montaj dili, her makine dili talimatını temsil etmek için bir mnemonik kullanılan düşük seviyeli programlama dilidir.
17. *Yüksek seviyeli diller*, İngilizce benzeri ve öğrenmesi ve programlaması kolaydır.
18. Yüksek seviyeli bir dilde yazılmış bir programa *kaynak programı* denir.
19. Bir *derleyici*, kaynak programı *makine-dili programına çeviren bir yazılım programıdır*.
20. İşletim sistemi (OS), bir bilgisayarın faaliyetlerini yöneten ve kontrol eden bir programdır.
21. Java platformdan bağımsızdır, bu da bir programı bir kez yazıp herhangi bir bilgisayarda çalıştırabileceğiniz anlamına gelir.
22. Java programları HTML sayfalarına gömülebilir ve Web tarayıcıları tarafından indirilerek Web istemcilerine canlı animasyon ve etkileşim getirebilir.
23. Java kaynak dosya adı, programdaki public sınıf adıyla eşleşmelidir. Java kaynak kodu dosyaları .java uzantısıyla bitmelidir.
24. Her sınıf, sınıfla aynı adı taşıyan ve **.class** uzantısıyla biten ayrı bir bytecode dosyasına derlenir.
25. Komut satırından bir Java kaynak kodu dosyasını derlemek için *javac* komutunu kullanın.
26. Komut satırından bir Java sınıfını çalıştırmak için *java* komutunu kullanın.
27. Her Java programı, sınıf tanımlarından oluşan bir settir. **class** anahtar kelimesi bir sınıf tanımını tanıtır. Sınıfın içeriği bir block içinde yer alır.
28. Bir blok, bir açılış süslü parantez (**{**) ile başlar ve bir kapanış süslü parantez (**}**) ile biter.
29. Yöntemler bir sınıfta bulunur. Bir Java programını çalıştırmak için programın bir **main** yöntemi olmalıdır. **main** yöntemi, programın çalıştırıldığında başladığı giriş noktasıdır.
30. Java'daki her *statement* bir noktalı virgül (**;**) ile biter, bu da *statement terminator* olarak bilinir.
31. *Ayrılmış kelimeler*, veya *anahtar kelimeler*, derleyiciye belirli bir anlam taşıyor ve programda başka amaçlar için kullanılamaz.
32. Java'da, yorumlar bir satırda iki eğik çizgi (**//**) ile başlar, bu bir *line comment* olarak adlandırılır, veya bir veya birkaç satır arasında **/*** ve ***/** ile yer alır, bu da *block comment* veya *paragraph comment* olarak adlandırılır. Yorumlar derleyici tarafından göz ardı edilir.
33. Java kaynak programları büyük/küçük harf duyarlıdır.

34. İki tür **import** ifadesi vardır: özel import ve joker import. Özel import, import ifadesinde tek bir sınıfı belirtir; joker import ise bir paketdeki tüm sınıfları içe aktarır.
35. Programlama hataları üç türe ayrılabilir: *sözdizimi hataları*, *çalışma zamanı hataları* ve *mantık hataları*. Derleme sırasında meydana gelen hatalara sözdizimi hataları veya derleme hataları denir. Çalışma zamanı hataları, bir programın anormal bir şekilde sonlanmasına neden olan hatalardır. Mantık hataları, bir programın beklenildiği gibi çalışmadığı durumlarda meydana gelir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinde yapın.

PROGRAMLAMA ALIŞTIRMALARI

MyProgrammingLab™



Not

Çift numaralı alıştırmaların çözümleri Eşlikçi Web Sitesinde bulunmaktadır. Tüm alıştırmaları n çözümleri Eğitimci Kaynak Web Sitesinde yer almaktadır. Zorluk seviyesi kolay (yıldız yok), orta (*), zor (**) veya zorlu (***) olarak derecelendirilmiştir.

zor seviyesinin

- 1.1 (Üç mesaj görüntüle) **Java'ya Hoş Geldiniz**,
Bilgisayar Bilimine Hoş Geldiniz ve Programlama eğlencelidir mesajlarını görüntüleyen bir program yazın.
- 1.2 (Beş mesaj görüntüle) **Java'ya Hoş Geldiniz** mesajını beş kez görüntüleyen bir program yazın.
- * 1.3 (Bir desen göster) Aşağıdaki deseni gösteren bir program yazın:

```
J A V V A n J A A   V V A A n J J A A
A A A V V A A A A A n J J A A   V A
A
```

- 1.4 (Bir tablo yazdır) Aşağıdaki tabloyu gösteren bir program yazın:

```
a a^2 a^3 n1 1 1 n2 4 8
n3 9 27 n4 16 64
```

- 1.5 (İfadeleri hesapla) Aşağıdaki sonucun gösterildiği bir program yazın:

$$\frac{9.5 \times 4.5 - 2.5 \times 3}{45.5 - 3.5}$$

- 1.6 (Bir serinin toplamı) Aşağıdaki sonucun gösterildiği bir program yazın:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9.$$

- 1.7 (Yaklaşık) kişi başı şu formül kullanılarak hesaplanabilir:

$$\pi = 4t - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots \leq$$

Sonucun gösterildiği bir program yazın $4t - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} \leq$

ve $4t - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \frac{1}{13}$ T. 1.0 yerine 1 kullanın programı.

32 Bölüm 1 Bilgisayarlara, Programlara ve Java'ya Giriş

- 1.8** (Bir dairenin alanı ve çevresi) Aşağıdaki formülü kullanarak 5.5 yarıçapa sahip bir dairenin alanını ve çevresini gösteren bir program yazın:

$$\text{çevre} = 2 \times \text{yarıçap} \times \pi$$

$$\text{alan} = \text{yarıçap} \times \text{yarıçap} \times \pi$$

- 1.9** (Bir dikdörtgenin alanı ve çevresi) Aşağıdaki formülü kullanarak 4.5 genişliğe ve 7.9 yüksekliğe sahip bir dikdörtgenin alanını ve çevresini gösteren bir program yazın:

$$\text{alan} = \text{genişlik} \times \text{yükseklik}$$

- 1.10** (Mil cinsinden ortalama hız) Bir koşucunun 14 kilometreyi 45 dakikada koştuğunu varsayın ve 30 saniye. Saatte mil cinsinden ortalama hızı gösteren bir program yazın. (Not edin ki 1 mil 1.6 kilometredir.)

- ***1.11** (Nüfus projeksiyonu) ABD Nüfus Bürosu, nüfusu aşağıdaki varsayımlara dayanarak projekte etmektedir:

- Her 7 saniyede bir doğum
- Her 13 saniyede bir ölüm
- Her 45 saniyede bir yeni göçmen

Önümüzdeki beş yıl için nüfusu görüntüleyen bir program yazın. Mevcut nüfusun 312,032,486 olduğunu ve bir yılın 365 gün olduğunu varsayın. İpucu: Java'da, iki tam sayı bölme işlemi yaptığında, sonuç bir tam sayıdır. Kesir kısmı kesilir. Örneğin, 5/41 (1.25 değil) ve 10/42 (2.5 değil) olarak sonuçlanır.

- 1.12** (Kilometre cinsinden ortalama hız) Bir koşucunun 24 mil koştuğunu varsayın 1 saat, 40 dakika ve 35 saniyede. Saatte kilometre cinsinden ortalama hızı görüntüleyen bir program yazın. (Not edin ki 1 mil 1.6 kilometredir.)

İLKÖĞRETİM PROGRAMLAMA

Hedefler

- Basit hesaplamalar yapmak için Java programları yazmak (§2.2).
- Konsoldan girdi almak için **Scanner** sınıfını kullanmak (§2.3).
- Değişkenler, sabitler, yöntemler ve sınıflar için tanımlayıcılar kullanmak (§2.4).
- Verileri depolamak için değişkenleri kullanmak (§§2.5–2.6).
- Atama ifadeleri ve atama ifadeleri ile program yazmak (§2.6).
- Sabitleri kalıcı verileri depolamak için kullanmak (§2.7).
- Sınıfları, yöntemleri, değişkenleri ve sabitleri adlandırma kurallarına uyararak adlandırmak (§2.8).
- Java sayısal ilkel veri türlerini keşfetmek: **byte**, **short**, **int**, **long**, **float** ve **double** (§2.9.1).
- **+**, **-**, *****, **/** ve **%** operatörlerini kullanarak işlemler yapmak (§2.9.2).
- **Math.pow(a, b)** kullanarak üslü işlemler yapmak (§2.9.3).
- Tam sayı literalleri, kayan nokta literalleri ve bilimsel notasyonda literaller yazmak (§2.10).
- Sayısal ifadeleri yazmak ve değerlendirmek (§2.11).
- Mevcut sistem zamanını elde etmek için **System.currentTimeMillis()** (§2.12).
- Artırılmış atama operatörlerini kullanmak (§2.13).
- Postincrement ve preincrement ile postdecrement ve predecrement arasındaki farkı ayırt etmek için (§2.14).
- Bir türün değerini başka bir türe dönüştürmek için (§2.15).
- Yazılım geliştirme sürecini tanımlamak ve bunu kredi ödeme programını geliştirmek için uygulamak (§2.16).
- Karakterleri **char** türü kullanarak temsil etmek için (§2.17).
- Bir dizeyi **String** türü kullanarak temsil etmek için (§2.18).
- Girdi almak için **JOptionPane** girdi iletişim kutularını kullanın (§2.19).



2.1 Giriş



Bu bölümün odak noktası, problemleri çözmek için temel programlama tekniklerini öğrenmektir.

Bölüm 1'de, çok temel Java programları oluşturmayı, derlemeyi ve çalıştırmayı öğrendiniz. Şimdi, program yazarak problemleri nasıl çözeceğinizi öğreneceksiniz. Bu problemler aracılığıyla, ilkel veri türleri, değişkenler, sabitler, operatörler, ifadeler ve girdi ve çıktı kullanarak temel programlama öğreneceksiniz.

Örneğin, bir öğrenci kredisi almanız gerektiğini varsayalım. Kredi miktarı, kredi süresi ve yıllık faiz oranı verildiğinde, aylık ödemeyi ve toplam ödemeyi hesaplamak için bir program yazabilir misiniz? Bu bölüm, bu tür programları nasıl yazacağınızı gösterir. Bu süreçte, bir problemi analiz etme, bir çözüm tasarlama ve bir program oluşturarak çözümü uygulama adımlarını öğreneceksiniz.

2.2 Basit Bir Program Yazma



Bir program yazmak, problemi çözmek için bir strateji tasarlamayı ve ardından bu stratejiyi uygulamak için bir programlama dili kullanmayı içerir.

sorun

Öncelikle, bir dairenin alanını hesaplama gibi basit bir problemi ele alalım. Bu problemi çözmek için bir programı nasıl yazıyoruz?

algoritma

Bir program yazmak, algoritmalar tasarlamayı ve algoritmaları programlama talimatlarına veya koda çevirmeyi içerir. Bir algoritma, bir problemin nasıl çözüleceğini, yapılması gereken eylemleri ve bunların yürütülme sırasını listeleyerek açıklar. Algoritmalar, programcıya bir programı yazmadan önce plan yapmasına yardımcı olabilir. Algoritmalar, doğal dillerde veya bazı programlama kodlarıyla karıştırılmış doğal dile (sözde kod) tanımlanabilir. Bir dairenin alanını hesaplamak için algoritma şu şekilde tanımlanabilir:

sahte kod

1. Dairenin yarıçapını okuyun.
2. Aşağıdaki formülü kullanarak alanı hesaplayın:

$$\text{alan} = \text{yarıçap} \times \text{yarıçap} \times \pi$$

3. Sonucu göster.



İpucu

Programınızı (veya onun temel problemini) kodlamaya başlamadan önce bir algoritma biçiminde taslak oluşturmak her zaman iyi bir uygulamadır.

Bir program yazdığınızda—yani bir kod yazdığınızda—bir algoritmayı bir programa çevirirsiniz. Her Java programının, anahtar kelime **class** ile sınıf adının takip edildiği bir sınıf tanımı ile başladığını zaten biliyorsunuz. Varsayalım ki siz **ComputeArea** sınıf adı olarak. Programın taslağı şöyle görünecektir:

```
kamusal sınıf HesaplaAlan {
    // Ayrıntılar daha sonra verilecektir
}
```

Bildiğiniz gibi, her Java programının program yürütmesinin başladığı bir **main** metodu olmalıdır. Program daha sonra şu şekilde genişletilir:

```
kamusal sınıf HesaplaAlan {
    kamusal statik boşana(String[] argümanlar) {
        // Adım 1: Yarıçapı oku

        // Adım 2: Alanı hesapla
```

```

    // Adım 3: Alanı göster
}
}

```

Program, kullanıcının klavyeden girdiği yarıçapı okumalıdır. Bu, iki önemli sorunu gündeme getirir:

- Yarıçapı okuma.
- Programda yarıçapı saklamak.

İkinci sorunu önce ele alalım. Yarıçapı saklamak için programın bir *değişken* adı verilen bir sembolbelirlemesi gerekir. Bir değişken, bilgisayarın belleğinde saklanan bir değeri temsil eder.

değişken

x veya yerine tanımlayıcı isimler seçin: bu durumda, yarıçap için *radius* ve alan için *area*. Derleyiciye *radius* ve *area*'nin ne olduğunu bildirmek için, data türlerini belirtin. Bu, bir değişkende saklanan verinin türüdür; tam sayı, ondalık sayı veya başka bir şey olabilir. Buna *değişkenleri tanımlama* denir. Java, tam sayıları, ondalık sayıları (yani, ondalık noktasına sahip sayılar), karakterlerle Boolean türlerini temsil etmek için basit veri türleri sağlar. Bu türler ilkel veri türleri veya temel türler olarak bilinir.

tanımlayıcı isimler

data türü

değişkenleri tanımla

ondalık sayı

ilkel veri türleri

Çift hassasiyetli kayan nokta sayıları olarak *radius* ve *area*'yi tanımlayın. Program şu şekilde genişletilebilir:

```

public class ComputeArea {
    public static void main(String[] args) {

```

ikiye katlama;

// Adım 1: radius'u oku

// Adım 2: alanı hesapla

// Adım 3: alanı göster

```

    }
}

```

Program, *radius* ve *area*'yi değişkenler olarak tanımlar. *double* anahtar kelimesi, *radius* ve *area*'nin bilgisayarda saklanan çift hassasiyetli kayan nokta değerleri olduğunu belirtir.

İlk adım, kullanıcıdan dairenin *radius*'unu belirtmesini istemektir. Kullanıcıdan bilgi istemeyi kısa süre içinde öğreneceksiniz. Şimdilik, değişkenlerin nasıl çalıştığını öğrenmek için, kodu yazarken programda *radius*'ye sabit bir değer atayabilirsiniz; daha sonra, bu değeri kullanıcıdan istemek için programı değiştireceksiniz.

İkinci adım, *area*'ya *radius* *

*radius * 3,14159 ifadesinin sonucunu atayarak alanı hesaplamaktır.*

Son adımda, program *area*'nin değerini konsolda gösterecektir.

System.out.println yöntemi kullanılarak.

Liste 2.1, tamamlanmış programı göstermektedir ve programın örnek çalışması Şekil 2.1'de gösterilmektedir.

LISTA 2.1 ComputeArea.java

```

1 genel class ComputeArea {
2     public static void main(String[] args) {
3         double radius; // Yarıçapı tanımla
4         double area; // Alanı tanımla
5
6         // Bir yarıçap atayın
7         radius = 20; // yarıçap artık 20

```

36 Bölüm 2 Temel Programlama

```
8
9      // Alanı hesapla
10 alan = yarıçap * yarıçap * t3.14159t;
11
12      // Sonuçları göster
13      System.out.println("Yarıçapı olan dairenin alanı "+
14                          radius + " dir "+ area);
15 }n16 }
```



FGÖRÜNTÜ 2.1 Program bir dairenin alanını gösterir.

değişken tanımla
değer ata

izleme programı

radius ve area gibi değişkenler bellek konumlarına karşılık gelir. Her değişkenin bir adı, bir türü, bir boyutu ve bir değeri vardır. 3. satır, radius değişkeninin bir double değeri saklayabileceğini belirtir. Değer, bir değeri atayana kadar tanımlanmaz. 7. satır, 20 değerini radius değişkenine atar. Benzer şekilde, 4. satır area değişkenini tanımlar ve 10. satır area değişkenine bir değeri atar. Aşağıdaki tablo, program çalıştırıldıkça area ve radius için bellek değerlerini gösterir. Tablo içindeki her satır, programdaki ilgili satırdaki ifade çalıştırıldığında değişkenlerin değerlerini gösterir. Bir programın nasıl çalıştığını inceleme yöntemi bir programı izlemek olarak adlandırılır. İzleme programları, programların nasıl çalıştığını anlamak için faydalıdır ve programlardaki hataları bulmak için yararlı araçlardır.



satır#	yarıçap	alan
3	değer yok	
4		değer yok
7	20	
10		1256.636

stringleri birleştir

stringleri
sayılara birleştir

Artı işareti (+) iki anlama gelir: biri toplama, diğeri ise (birleştirme) dizeleri birleştirme. 13-14. satırlardaki artı işareti (+) bir *dize birleştirme operatörü* olarak adlandırılır. İki dizeyi birleştirir. Eğer bir dize bir sayı ile birleştirilirse, sayı bir dizeye dönüştürülür ve diğer dize ile birleştirilir. Bu nedenle, 13-14. satırlardaki artı işaretleri (+) dizeleri daha uzun bir dizeye birleştirir ve bu daha sonra çıktıda görüntülenir. Dizeler ve dize birleştirme, Bölüm 2.18'de daha fazla tartışılacaktır.



Dikkat

Bir dize kaynak kodunda satırları geçemez. Bu nedenle, aşağıdaki ifade bir derleme hatasına yol açar:

```
System.out.println("Java Programlamaya Giriş,  
Y. Daniel Liang tarafından");
```

Hatanın düzeltilmesi için, dizeyi ayrı alt dizelere ayırın ve bunları birleştirmek için birleştirme operatörünü (+) kullanın:

```
System.out.println("Java Programlamaya Giriş, "+
    "Y. Daniel Liang tarafından");
```



İpucu

Bu örnek üç adımdan oluşmaktadır. Bu adımları birer birer ekleyerek geliştirmek ve test etmek iyi bir yaklaşımdır.

uzun bir dizeyi kır

artımlı geliştirme ve
test etme

Point Kontrol

MyProgrammingLab™

2.1 Aşağıdaki koddaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfTest {
2     public voidtmain(string[] args) {
3         int i;
4         inttk = t100.0t;
5         inttj = i + t1t;
6
7         System.out.println("j şu: "+ j + " ve
8             k şu: "+ k);
9     }n10
10 }
```

2.3 Konsoldan Girdi Okuma

Konsoldan girdi okumak, programın kullanıcıdan girdi almasını sağlar.

Liste 2.1'de, yarıçap kaynak kodunda sabitlenmıştır. Farklı bir yarıçap kullanmak için, kaynak kodunu değiştirmeniz ve yeniden derlemeniz gerekir. Açıkça, bu pratik değildir, bu yüzden bunun yerine **Scanner** sınıfını konsol girişi için kullanabilirsiniz.

Java, standart çıktı cihazına atıfta bulunmak için **System.out** ve standart girdi cihazına atıfta bulunmak için **System.in** kullanır. Varsayılan olarak, çıktı cihazı ekran monitörüdür ve girdi cihazı klavyedir. Konsol çıktısı gerçekleştirmek için, basit bir değeri veya bir dizeyi konsola görüntülemek için **println** yöntemini kullanırsınız. Java'da konsol girişi doğrudan desteklenmez, ancak **Scanner** sınıfını kullanarak **System.in**'den girdi okumak için bir nesne oluşturabilirsiniz, şu şekilde:

```
Tarayıcı girişi = tnewtScanner(System.in);
```

Sözdizimi **new Scanner(System.in)**, **Scanner** türünde bir nesne oluşturur. Sözdizimi **Scanner input**, **input**'ün türünün **Scanner** olduğunu belirler. Tüm satır **Scanner input = new Scanner(System.in)**, bir **Scanner** nesnesi oluşturur ve referansını **input** değişkenine atar. Bir nesne, yöntemlerini çağırabilir. Bir nesne üzerinde bir yöntemi çağırmak, nesnenin bir görevi yerine getirmesini istemektir. Çeşitli türde girdi okumak için Tablo 2.1'de listelenen yöntemleri çağırabilirsiniz.

Şu anda, bir ondalık nokta içeren bir sayıyı nasıl okuyacağımızı görmek için **nextDouble()** yöntemini çağıracağız. Diğer yöntemler kullanıldıkça ele alınacaktır. Liste 2.2

kullanıcıdan bir yarıçap girmesini istemek için Liste 2.1'i yeniden yazmaktadır.

LISTE 2.2 ComputeAreaWithConsoleInput.java

```
1 import java.util.Scanner; // Scanner java.util paketindedir
2
3 genel classtComputeAreaWithConsoleInput {
4     public static voidtmain(String[] args) {
5         // Bir Scanner nesnesi oluştur
6         Tarayıcı girişi = tnewtScanner(System.in);
```

import sınıfı

bir Scanner oluştur



VideoNot
Girdi al

38 Bölüm 2 Temel Programlama

TABLE 2.1 Scanner Nesneleri için Yöntemler

Yöntem	Açıklama
<code>nextByte()</code>	bir <code>byte</code> türünde tam sayı okur.
<code>nextKısa()</code>	bir <code>short</code> türünde tam sayı okur.
<code>nextInt()</code>	bir <code>int</code> türünde tam sayı okur.
<code>sonrakiUzun()</code>	bir <code>long</code> türünde tam sayı okur.
<code>nextFloat()</code>	<code>float</code> türünde bir sayı okur.
<code>nextDouble()</code>	<code>double</code> türünde bir sayı okur.
<code>Sonraki()</code>	bir boşluk karakterinden önce biten bir dize okur.
<code>nextLine()</code>	bir metin satırı okur (yani, <code>Enter</code> tuşuna basıldığında biten bir dize).

bir double oku

```
8 // Kullanıcıdan bir yarıçap girmesini isteyin
9 System.out.print("Yarıçap için bir sayı girin: ");
10 çift radius = input.nextDouble();
11
12 // Alanı hesapla
13 doubletarea = yarıçap * yarıçap * 3,14159;
14
15 // Sonuçları göster
16 System.out.println("Yarıçapı "+
17 radius + " olan dairenin alanı "+ area);
18} n19}
```



Yarıçap için bir sayı girin: 2.5 Enter
Yarıçapı 2.5 olan dairenin alanı 19.6349375



Yarıçap için bir sayı girin: 23 Enter
Yarıçapı 23.0 olan dairenin alanı 1661.90111

Scanner sınıfı `java.util` paketindedir. 1. satırda içe aktarılmıştır. 6. satırda bir

Scanner nesnesi oluşturulmaktadır.

9. satırdaki ifade, kullanıcıdan girdi istemek için bir mesaj görüntüler.

Sistem çıkışı. `baskı` ("Yarıçap için bir sayı girin: ");

printt vs.t println

`print` metodu, `println` metoduna benzer, tek farkı `println` dizgiyi gösterdikten sonra bir sonraki satırın başına geçerken, `print` tamamlandığında bir sonraki satıra geçmemesidir.

10. satırdaki ifade klavyeden girdi alır.

`doubletradius = input.nextDouble();`

Kullanıcı bir sayı girdikten ve `Enter` tuşuna bastıktan sonra, program sayıyı okur ve bunu `radius` değişkenine atar.

Objelerle ilgili daha fazla ayrıntı 8. Bölümde tanıtılacaktır. Şimdilik, konsoldan girdi almanın bu şekilde olduğunu kabul edin.

Liste 2.3, klavyeden birden fazla girdi okuma örneği vermektedir. Program üç sayı okur ve ortalamasını gösterir.

LİSTE 2.3 HesaplaOrtalama.java

```

1  import java.util.Scanner; // Scanner java.util paketindedir
2
3
4  public class HesaplaOrtalama {
5      public static void main(String[] args) {
6          // Bir Scanner nesnesi oluştur
7          Tarayıcı girişi = new Tarayıcı(System.in);
8
9          // Kullanıcıdan üç sayı girmesini iste
10         System.out.print("Üç sayı girin: ");
11         double sayi1 = nextDouble();
12         double sayi2 = ut.nextDouble();
13         double sayi3 = input.nextDouble();
14
15         // Ortalama hesapla
16         double ortalama = (sayi1 + sayi2 + sayi3) / 3;
17
18         // Sonuçları göster
19         System.out.println("Ortalama " + sayi1 + " " + sayi2
20             + " " + sayi3 + " şudur: " + ortalama);
21     }
22 }

```

import sınıfı

bir Scanner oluştur

bir double oku



girişi tek satırda girin



girişi birden fazla satırda girin

Üç sayı girin: 1 2 3 Enter
1.0 2.0 3.0 ortalaması 2.0'dır

Üç sayı girin: 10.5 Enter
11 Enter
11.5 Enter
10.5 11.0 11.5 ortalaması 11.0'dır

Scanner sınıfını içe aktarma kodu (satır 1) ve bir Scanner nesnesi oluşturma kodu (satır 6) önceki örnekte olduğu gibi ve klavyeden girdi okumak için yazacağınız tüm yeni programlarda aynıdır.

Satır 9, kullanıcıdan üç sayı girmesini ister. Sayılar satır 10-12'de okunur. Üç sayıyı aralarına boşluk koyarak girebilir, sonra Enter tuşuna basabilir veya her sayıyı girip ardından Enter tuşuna basabilirsiniz; bu programın örnek çalışmaları şeklinde gösterilmiştir.

Eğer sayısal bir değer dışında bir girdi girdiyerseniz, bir çalışma zamanı hatası oluşacaktır. Bölüm 14'te, programın çalışmaya devam etmesi için istisnayı nasıl ele alacağınızı öğreneceksiniz.

çalışma zamanı hatası



Not

Bu kitabın ilk bölümlerindeki programların çoğu üç adım gerçekleştirir: girdi, işlem ve çıktı, buna *IPO* denir. Girdi, kullanıcıdan girdi almak; işlem, girdi kullanılarak sonuçlar üretmek; ve çıktı, sonuçları görüntülemektir.

DURUM

40 Bölüm 2 Temel Programlama

MyProgrammingLab™

Point Kontrol

- 2.2 Kullanıcının klavyeden bir tam sayı veya bir ondalık değer girmesine izin veren bir ifade n asıl yazarsınız?
- 2.3 Bu kodu çalıştırdığınızda 5a girdiyseniz ne olur?
doubletradius = input.nextDouble();

2.4 Tanımlayıcılar



Tanımlayıcılar, bir programdaki sınıflar, yöntemler ve

değişkenler gibi öğeleri tanımlayan isimlerdir.

Liste 2.3'te gördüğünüz gibi, **ComputeAverage**, **main**, **input**, **number1**, **number2**, **number3**, ve benzeri, programda görünen şeylerin isimleridir. Programlama terminolojisinde, bu tür isimlere *tanımlayıcılar* denir. Tüm tanımlayıcılar aşağıdaki kurallara uymalıdır:

- Bir tanımlayıcı, harfler, rakamlar, alt çizgiler (`_`) ve dolar işaretlerinden (\$) oluşan bir karakter dizisidir.
- Bir tanımlayıcı bir harf, bir alt çizgi (`_`) veya bir dolar işareti (\$) ile başlamalıdır. Rakamla başlayamaz.
- Bir tanımlayıcı, ayrılmış bir kelime olamaz. (Ayrılmış kelimelerin listesi için Ek A'ya bakın.)
- Bir tanımlayıcı **true**, **false** veya **null** olamaz.
- Bir tanımlayıcı herhangi bir uzunlukta olabilir.

Örneğin, **\$2**, **ComputeArea**, **area**, **radius** ve **showMessageDialog** yasal tanımlayıcılardır, oysa **2A** ve **d+4** kurallara uymadıkları için yasal değildir. Java derleyicisi geçersiz tanımlayıcıları tespit eder ve söz dizimi hatalarını raporlar.

büyük/küçük harf duyarlı



Not

Java büyük/küçük harf duyarlı olduğundan, **area**, **Area** ve **AREA** hepsi farklı tanımlayıcılardır.

tanımlayıcı isimler



İpucu

Tanımlayıcılar, değişkenler, sabitler, yöntemler, sınıflar ve paketler için adlandırma amacıyla kullanılır. Açıklayıcı tanımlayıcılar, programların okunmasını kolaylaştırır. Tanımlayıcılar için kısaltmalar kullanmaktan kaçının.

Tam kelimeler kullanmak daha açıklayıcıdır. Örneğin, **numberOfStudents**, **numStuds**, **numOfStuds** veya **numOfStudents**'dan daha iyidir. Metinde tam programlar için açıklayıcı adlar kullanıyoruz. Ancak, kısalık için kod parçacıklarında **i**, **j**, **k**, **x** ve **y** gibi değişken adlarını zaman zaman kullanacağız. Bu adlar, kod parçacıklarına genel bir ton da sağlar.

karakter



İpucu

Tanımlayıcıları **\$** karakteri ile adlandırmayın. Geleneksel olarak, **\$** karakteri yalnızca mekanik olarak üretilen kaynak kodunda kullanılmalıdır.

MyProgrammingLab™

Point Kontrol

- 2.4 Aşağıdaki tanımlayıcılardan hangileri geçerlidir? Hangileri Java anahtar kelimeleridir?

milest, **t**, **Testt**, **t a++t**, **t at**, **t 4#Rt**, **t \$4t**, **t #44t**, **t appsn**
classt, **t publict**, **t intt**, **t xt**, **t yt**, **t radius**

2.5 Değişkenler



Değişkenler, programda değiştirilebilecek değerleri temsil etmek için kullanılır.

neden değişken denir?

Önceki bölümlerdeki programlardan gördüğünüz gibi, değişkenler, daha sonra bir programda kullanılmak üzere değerleri saklamak için kullanılır. Değerleri değiştirilebildiği için onlara değişken denir.

Liste 2.2'deki programda, **radius** ve **area** çift hassasiyetli, kayan-nokta türünde değişkenlerdir. **radius** ve **area**'ya herhangi bir sayısal değer atayabilirsiniz ve **radius** ve **area**'nın değerleri yeniden atanabilir. Örneğin, aşağıdaki kodda, **radius** başlangıçta **1.0** (satır 2) ve ardından **2.0** (satır 7) olarak değiştirilir, ve **area** **3.14159** (satır 3) olarak ayarlanır ve ardından **12.56636** (satır 8) olarak sıfırlanır.

```
1 // İlk alanı hesapla
2 yarıçap = t1.0t;
alan = yarıçap * yarıçap * 3.14159;
System.out.println("Alan " + alan + " için yarıçap " + yarıçap);

// İkinci alanı hesapla
yarıçap = 2.0;
alan = yarıçap * yarıçap * 3.14159;
System.out.println("Alan " + alan + " için yarıçap " + yarıçap);
```

yarıçap: 1.0
alan: 3.14159

yarıçap: 2.0
alan: 12.56636

Değişkenler, belirli bir türdeki verileri temsil etmek için kullanılır. Bir değişkeni kullanmak için, derleyiciye adını ve hangi tür veriyi saklayabileceğini belirterek onu tanımlarsınız. *Değişken tanımlama*, derleyiciye değişkenin veri türüne göre uygun bellek alanı ayırmasını söyler. Bir değişken tanımlamanın sözdizimi şudur:

veriTipi değişkenAdı;

İşte bazı değişken bildirimlerine örnekler:

değişkeni tanımla

```
int count; // count'u bir tamsayı değişkeni olarak tanımla
double radius; // radius'u bir double değişkeni olarak tanımla
double interestRate; // interestRate'i bir double değişkeni olarak tanımla
```

Bu örnekler **int** ve **double** veri türlerini kullanmaktadır. Daha sonra **byte**, **short**, **long**, **float**, **char** ve **boolean** gibi ek veri türleriyle tanışacaksınız.

Eğer değişkenler aynı türdeyse, aşağıdaki gibi birlikte tanımlanabilirler:

veriTipi değişken1, değişken2, ..., değişkenN;

Değişkenler virgüllerle ayrılır. Örneğin,

```
int i, j, k; // i, j ve k'yi int değişkenleri olarak tanımla
```

Değişkenlerin genellikle başlangıç değerleri vardır. Bir değişkeni tanımlayıp bir adımda başlatabilirsiniz.

değişkenleri başlat

Örneğin, aşağıdaki kodu düşünün:

```
int count = t1t;
```

Bu, sonraki iki ifadeye eşdeğerdir:

```
int count;
count = t1t;
```

Aynı türdeki değişkenleri birlikte tanımlayıp başlatmak için kısayol formunu da kullanabilirsiniz.

Örneğin,

```
int i = t1t, j = t2t;
```



İpucu

Bir değişken, bir değere atanabilmesi için tanımlanmalıdır. Bir metodda tanımlanan bir değişken, kullanılmadan önce bir değere atanmalıdır.

Mümkün olduğunda, bir değişken tanımlayıp ve başlangıç değerini tek adımda atayın. Bu, programın okunmasını kolaylaştıracak ve programlama hatalarını önleyecektir.

42 Bölüm 2 Temel Programlama

değişkenin kapsamı

Her değişkenin bir kapsamı vardır. Bir değişkenin *kapsamı*, değişkenin referans alınabil eceği programın kısmıdır. Bir değişkenin kapsamını tanımlayan kurallar, kitapta yavaş yavaş aş tanıtılacaktır. Şu anda bilmeniz gereken tek şey, bir değişkenin kullanılmadan önce tanımlanması ve başlatılması gerektiğidir. Aşağıdaki kodu düşünün:

```
int faizOranı = 0.05
int faiz = faizOranı * 45
```

Bu kod yanlıştır, çünkü `faizOranı` bir değere `0.05` atanmıştır, ancak `faizOranı` tanımlanmamış ve başlatılmamıştır. Java büyük/küçük harf duyarlıdır, bu nedenle `faizOranı` ve `faizOranı` iki farklı değişken olarak kabul edilir.

2.6 Atama İfadeleri ve Atama İfadeleri



Bir atama ifadesi, bir değişken için bir değer belirler. Bir atama ifadesi Java'da bir ifade olarak kullanılabilir.

atama ifadesi
atama operatörü

Bir değişken tanımlandıktan sonra, ona bir değer atamak için bir *atama ifadesi* kullanabilirsiniz. Java'da eşittir işareti (=) *atama operatörü* olarak kullanılır. Atama ifadelerinin sözdizimi şu şekildedir:

değişken = ifade;

ifade

Bir *ifade*, değerler, değişkenler ve operatörler içeren bir hesaplamayı temsil eder ve bunlar bir araya getirilerek bir değere değerlendirir. Örneğin, aşağıdaki kodu düşünün:

```
intty = t1t; // Değişken y'ye 1 atayın
doubletradius = t1.0t; // Değişken radius'a 1.0 atayın
intex = takht * (taat/taat); // İfadenin değerini x'e atayın
x = y + t1t; // y ve 1'in toplamını x'e atayın
area = radius * radius * 3.14159; // Alanı hesaplayın
```

Bir ifadede bir değişken kullanabilirsiniz. Bir değişken, operatörün her iki tarafında da kullanılabilir. Örneğin,

x = x + t1t;

Bu atama ifadesinde, `x + 1` ifadesinin sonucu `x` değişkenine atanır. Eğer `x` ifadenin ç alıştırılmasından önce `1` ise, ifade çalıştırıldıktan sonra `2` olur.

Bir değişkene değer atamak için, atama operatörünün soluna değişken adını yerleştirmelisiniz. Bu nedenle, aşağıdaki ifade yanlıştır:

1 = x; // Yanlış



Not

Matematikte, `x = 2 * x + 1` bir denklemin ifade eder. Ancak, Java'da `x = 2 * x + 1` bir atama ifadesidir ve `2 * x + 1` ifadesini değerlendirip sonucunu `x`'ye atar.

Java'da, bir atama ifadesi esasen atama operatörünün solundaki değişkene atanacak değeri değerlendiren bir ifadedir. Bu nedenle, bir atama ifadesi aynı zamanda *atama ifadesi* olarak da bilinir. Örneğin, aşağıdaki ifade doğrudur:

atama ifadesi

```
System.out.println(x = t1t);
```

eşdeğer olan

```
x = t1t;
System.out.println(x);
```

Bir değeri birden fazla değişkene atarsanız, bu sözdizimini kullanabilirsiniz:

```
ben = j = k = t1t;
```

eşdeğer olan

```
k = t1t;
nj = k;n
i = j;
```



Not

Bir atama ifadesinde, soldaki değişkenin veri türü sağdaki değerin veri türü ile uyumlu olmalıdır. Örneğin, `int x = 1.0` yasadışı olur, bu çünkü `x` 'in veri türü `int` 'dir. Bir `double` değerini (`1.0`) bir `int` değişkenine tür dönüştürme kullanmadan atayamazsınız. Tür dönüştürme Bölüm 2.15'te tanıtılmaktadır.

2.7 İsimlendirilmiş Sabitler

İsimlendirilmiş bir sabit, kalıcı bir değeri temsil eden bir tanımlayıcıdır.

Bir değişkenin değeri bir programın yürütülmesi sırasında değişebilir, ancak bir *isimlendirilmiş sabit*, yani kısaca *sabit*, asla değişmeyen kalıcı veriyi temsil eder. Bizim `ComputeArea` programımızda π bir sabittir. Eğer bunu sıkça kullanıyorsanız, `3.14159` yazmaya devam etmek istemezsiniz; bunun yerine,

pt. bir sabit tanımlamak için sözdizimi:

```
final datatype CONSTANTNAME = değer;
```

Bir sabit, aynı ifadede tanımlanmalı ve başlatılmalıdır. `final` kelimesi, bir sabiti tanımlamak için Java anahtar kelimesidir. Örneğin, şunu tanımlayabilirsiniz: π bir sabit olarak ve Liste 2.1'i aşağıdaki gibi yeniden yazın:

```
// ComputeArea.java: Bir dairenin alanını hesapla
genel classtComputeArea {
    public static void main(String[] args) {
        son ikiliPI = t3.14159t; // Bir sabit tanımlayın
```

```
// Bir yarıçap atayın
doubletradius = t20t;
```

```
// Alanı hesapla
doubletarea = yarıçap * yarıçap * PI;
```

```
// Sonuçları göster
System.out.println("Yarıçapı "+
    radius + " olan dairenin alanı "+ area);
```

```
}
```



sabit

final anahtar kelimesi

Sabitlerin kullanılmasının üç avantajı vardır: (1) Aynı değeri birden fazla kez kullanıyorsanız, tekrar tekrar yazmak zorunda kalmazsınız; (2) sabit değeri değiştirmek zorunda kalırsanız (örneğin, `PI` için `3.14` `3.14159` olarak), bunu yalnızca kaynak kodunda tek bir yerde değiştirmeniz gerekir; ve (3) bir sabit için açıklayıcı bir ad, programın okunmasını kolaylaştırır.

sabitlerin faydaları

2.8 İsimlendirme Kuralları



Java isimlendirme kurallarına uymak, programlarınızı okunabilir hale getirir ve hatalardan kaçınmanızı sağlar.

Programınızdaki değişkenler, sabitler, sınıflar ve yöntemler için anlamı açık olan tanımlayıcı isimler seçtiğinizden emin olun. Daha önce belirtildiği gibi, isimler büyük/küçük harf duyarlıdır. Aşağıda değişkenler, yöntemler ve sınıflar için isimlendirme kuralları listelenmiştir.

değişkenler ve yöntemler adlandır

- Değişkenler ve yöntemler için küçük harf kullanın. Eğer bir isim birden fazla kelimeyi oluştuyorsa, bunları birleştirerek ilk kelimeyi küçük harfle başlatın ve sonraki her kelimenin ilk harfini büyük yapın—örneğin, değişkenler **radius** ve **area** ile yöntem **showMessageDialog**.

sınıfları adlandır

- Bir sınıf adındaki her kelimenin ilk harfini büyük yapın—örneğin, sınıf adları **ComputeArea**, **System**, ve **JOptionPane**.

isim sabitleri

- Bir sabitin her harfini büyük yapın ve kelimeler arasında alt çizgi kullanın—örneğin, sabitler **PI** ve **MAX_VALUE**.

Programlarınızı okunması kolay hale getirmek için adlandırma kurallarına uymak önemlidir.

sınıfları adlandır



Dikkat

Java kütüphanesinde zaten kullanılan sınıf adlarını seçmeyin. Örneğin, **System** sınıfı Java'da tanımlandığı için, sınıfınızı **System** olarak adlandırmamalısınız.

2.5 Sabitlerin kullanmanın faydaları nelerdir? 20 değerine sahip bir **int** sabiti **SIZE** tanımlayın.

2.6 Sınıf adları, yöntem adları, sabitler ve

değişkenler için adlandırma kuralları nelerdir? Aşağıdaki öğelerden hangisi Java adlandırma kurallarına göre bir sabit, bir yöntem, bir değişken veya bir sınıf olabilir?

MAX_VALUE, Test, okuma, okuma

2.7 Aşağıdaki algoritmayı Java koduna çevirin:

Adım 1: Başlangıç değeri 100 olan **miles** adında bir **double** değişkeni tanımlayın.

Adım 2: **double** sabitini **KILOMETERS_PER_MILE** adıyla tanımlayın ve değeri 1.609 olarak ayarlayın.

Adım 3: **double** değişkenini **kilometers** adıyla tanımlayın, **miles** ile **KILOMETERS_PER_MILE** çarpın ve sonucu **kilometers**'ye atayın.

Adım 4: Konsola **kilometre** göster.

Adım 4'ten sonra kilometre nedir?



MyProgrammingLab™

2.9 Sayısal Veri Türleri ve İşlemleri



*Java, tamsayılar ve ondalık sayılar için +, -, *, / ve % operatörleri ile birlikte altı sayısal tür sunar. *, /, ve %.*

2.9.1 Sayısal Türler

Her veri türünün bir değer aralığı vardır. Derleyici, her değişken veya sabit için bellek alanı ayırır. Java, sayısal değerler,

karakterler ve Boolean değerler için sekiz ilkel veri türü sağlar. Bu bölüm, sayısal veri türlerini ve operatörleri tanıtmaktadır.

Tablo 2.2, altı sayısal veri türünü, aralıklarını ve depolama boyutlarını listelemektedir.

TABLE 2.2 Sayısal Veri Türleri

İsim	Aralık	Depolama Boyutu	
bayt	— Sen (t-128'den 127'ye)2t7t-t1'e	8-bit işaretli	byte türü
kısa	— t2t15 to (—32768 ile 32767)2 ¹⁵ —1	16-bit imzalı	short türü
int	—1 2147483647'ye kadar)(- 2147483648- 1	32-bit imzalı	int türü
uzun	— Düşükün2 ⁶³ —1 (yani, -9223372036854775807'ye kadar)	64-bit imzalı	long türü
batmadan yüzme	Negatif aralık: —T.4028235it+bit ile -t1.4Et-t45 Pozitif aralık: 1.4Et-t45 3,4028235Et+t38'e kadar	32 bit IEEE 754	float türü
çift	Negatif aralık: —T1.7976931348623157IT+T08 ile —tch.yaet-t324 Pozitif aralık: 4.9Henüz-Ta'a4 1.7976931348623157E+308	64 bit IEEE 754	çift tür



Not

IEEE 754, Elektrik ve Elektronik Mühendisleri Enstitüsü tarafından onaylanan bir standarttır. Bilgisayarlarda kayan nokta sayılarını temsil etmek için. Standart yaygın olarak benimsenmiştir. Java, **float** türü için 32-bit **IEEE 754** ve **double** türü için 64-bit **IEEE 754** kullanır. **IEEE 754** standardı ayrıca özel kayan nokta değerlerini tanımlar, bunlar Ek E'de listelenmiştir.

Java, tam sayılar için dört tür kullanır: **byte**, **short**, **int** ve **long**. Değişkeniniz için en uygun türü seçin. Örneğin, bir değişkende saklanan bir tam sayının byte aralığında olduğunu biliyorsanız, değişkeni **byte** olarak tanımlayın. Basitlik ve tutarlılık için, bu kitapta çoğu zaman **int** kullanacağız.

tam sayı türleri

Java, kayan nokta sayıları için iki tür kullanır: **float** ve **double**. **double** türü, **float** türünden iki kat daha büyüktür, bu nedenle **double**, çift hassasiyet olarak bilinir ve **float** ise tek hassasiyet olarak bilinir. Normalde **double** türünü kullanmalısınız, çünkü bu, float türünden daha doğrudur.

kayan nokta türleri



Dikkat

Bir değişkene saklanması için çok büyük (*büyük*) bir değer atandığında, taşma meydana gelir. Örneğin, aşağıdaki ifadeyi yürütmek taşmaya neden olur, çünkü **int** türündeki bir değişkende saklanabilecek en büyük değer **2147483647**'dir. **2147483648** bir **int** değeri için çok büyük olacaktır.

taşma nedir?

```
int value = 2147483647 + 1;
// değer aslında -2147483648 olacaktır
```

Aynı şekilde, aşağıdaki ifadenin yürütülmesi taşmayı tetikler, çünkü **int** türündeki bir değerde saklanabilecek en küçük değer **-2147483648**'dir. **-2147483649** bir **int** değişkeninde saklanmak için çok büyük olacaktır.

```
int value = -2147483648 - 1;
// değer aslında 2147483647 olacaktır
```

Java, taşma durumlarında uyarı veya hata bildirmez, bu nedenle belirli bir türün maksimum veya minimum aralığına yakın sayılarla çalışırken dikkatli olun.

46 Bölüm 2 Temel Programlama

taşma nedir?

Bir kayan nokta sayısı çok küçük (yani, sıfıra çok yakın) olduğunda, depolanamaz ve bualt akışa neden olur. Java bunu sıfıra yakınsatar, bu nedenle genellikle alt akış hakkında endişelenmenize gerek yoktur.

operatörler+, -, *, /, %

operatörler

2.9.2 Sayısal Operatörler

Sayısal veri türleri için operatörler, standart aritmetik operatörleri içerir: toplama (+), çıkarma (-), çarpma (*), bölme (/) ve kalan (%), Tablo 2.3'te gösterildiği gibi. Operanlar, bir operatör tarafından işlenen değerlerdir.

TABLO 2.3 Sayısal Operatörler

İsim	Anlam	Örnek	Sonuç
+	Toplama	34 + 1	35
-	Çıkarma	34.0 - 0.1	33.9
*	Çarpma	300 * 30	9000
/	Bölme	1.0 / 2.0	0.5
%	Kalan	20 % 3	2

tam sayı bölmesi

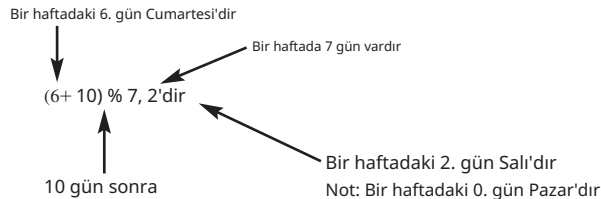
Bölme işleminin her iki operatörü de tam sayılar olduğunda, bölmenin sonucu bir tam sayı olur ve kesirli kısım kesilir. Örneğin, $5 / 2$ sonucu 2 verir, 2.5 değil, ve $-5 / 2$ sonucu -2 verir, -2.5 değil. Normal matematiksel bölme yapmak için, operatörlerden biri bir kayan-nokta sayısı olmalıdır. Örneğin, $5.0 / 2$ sonucu 2.5 verir.

% operatörü, kalan veya mod operatörü olarak bilinir, bölme işleminden sonra kalan değeri verir. Soldaki operatör bölünen, sağdaki operatör ise bölen olarak adlandırılır. Bu nedenle, $7 \% 3$ 1 verir, $3 \% 7$ 3 verir, $12 \% 4$ 0 verir, $26 \% 8$ 2 verir ve $20 \% 13$ 7 verir.

$$\begin{array}{r} 2 \\ 3 \overline{) 7} \\ \underline{6} \\ 1 \end{array} \quad \begin{array}{r} 0 \\ 7 \overline{) 3} \\ \underline{0} \\ 3 \end{array} \quad \begin{array}{r} 3 \\ 4 \overline{) 12} \\ \underline{12} \\ 0 \end{array} \quad \begin{array}{r} 3 \\ 8 \overline{) 26} \\ \underline{24} \\ 2 \end{array} \quad \text{Bölen} \rightarrow \begin{array}{r} 1 \\ 13 \overline{) 2013} \\ \underline{26} \\ 13 \\ \underline{13} \\ 0 \end{array} \begin{array}{l} \leftarrow \text{Sonuç} \\ \leftarrow \text{Bölünen} \\ \leftarrow \text{Kalan} \end{array}$$

% operatörü genellikle pozitif tam sayılar için kullanılır, ancak negatif tam sayılar ve kayan nokta değerleri ile de kullanılabilir. Kalan yalnızca pay negatifse negatiftir. Örneğin, $-7 \% 3$ sonucu -1, $-12 \% 4$ sonucu 0, $-26 \% -8$ sonucu -2 ve $20 \% -13$ sonucu -7 verir.

Kalan, programlamada çok faydalıdır. Örneğin, bir çift sayı % 2 her zaman 0 ve bir tek sayı % 2 her zaman 1'dir. Bu nedenle, bir sayının çift mi yoksa tek mi olduğunu belirlemek için bu özelliği kullanabilirsiniz. Eğer bugün Cumartesi ise, 7 gün sonra yine Cumartesi olacaktır. Diyelim ki siz ve arkadaşlarınız 10 gün içinde buluşacaksınız. 10 gün sonra hangi gün? Aşağıdaki ifadeyi kullanarak günün Salı olduğunu bulabilirsiniz:



Liste 2.4'teki program, bir zaman miktarından

saniye cinsinden dakikaları ve kalan saniyeleri alır. Örneğin, 500 saniye, 8 dakika ve 20 saniye içerir.

LİSTE 2.4 DisplayTime.java

```

13 import java.util.Scanner;
14
15 genel classtDisplayTime {
16     public static voidtmain(String[] args) {
17         Tarayıcı girişi = tnewt Tarayıcı(System.in);
18         // Kullanıcıdan girdi isteme
19         System.out.print("Saniye için bir tam sayı girin: ");
20         int seconds = input.nextInt();
21
22         int minutes = saniye / 60; // Saniyelerde dakikaları bulma
23         intremainingSaniye = saniye % 60; // Kalan saniyeler
24         System.out.println(seconds + " saniye" + minutes +
25             " dakika ve " + remainingSeconds + " saniye");
26     }
27 }

```

Tarayıcıyı içe aktar

bir Scanner oluştur

bir tam sayı oku

divide kalan

Saniye için bir tam sayı girin:
500 saniye 8 dakika ve 20 saniyedir

500 Enter



satır#	saniyeler	dakikalar	kalanSaniyeler
8	500		
10		8	
11			20



nextInt() metodu (satır 8) saniye için bir tam sayı okur. Satır 10, saniye / 60 kullanarak dakikaları elde eder. Satır 11 (saniye % 60) dakikaları çıkardıktan sonra kalan saniyeleri elde eder.

+ ve - operatörleri hem unary hem de binary olabilir. Bir *unary* operatörün yalnızca bir operandı vardır; bir *binary* operatörün ise iki operandı vardır. Örneğin, - operatörü -5 içinde bir unary operatördür ve 5 sayısını negatif hale getirirken, - operatörü 4 - 5 içinde 4'ten 5'i çıkarmak için bir binary operatördür.

unary operatör
ikili operatör



Not

Kayan nokta sayılarıyla ilgili hesaplamalar, bu sayıların tam doğrulukla saklanmadığı için yaklaşık olarak yapılır. Örneğin,

```
System.out.println(1.0t-t0.1t-t0.1t-t0.1t-t0.1t-t0.1t);
```

0.5000000000000001 gösterir, 0.5 değil, ve

```
System.out.println(1.0t-t0.9t);
```

0.09999999999999998 gösterir, 0.1 değil. Tam sayılar kesin olarak saklanır. Bu nedenle, tam sayılarla yapılan hesaplamalar kesin bir tam sayı sonucu verir.

kayan nokta yaklaşık değeri

2.9.3 Üs İşlemleri

Math.pow(a, b) metodu

Math.pow(a, b) yöntemi, The'yi hesaplamak için kullanılabilir. *özniye* **pow** metodu, içinde tanımlanmıştır Java API'sindeki Math sınıfı. Yöntemi **Math.pow(a, b)** sözdizimiyle çağırırsınız (yani, **Math.pow(2, 3)**), bu da *Eşik (parça).* Burada **a** ve **b** parametrelerdir **pow** metodu ve sayılar **2** ve **3** yöntemi çağırarak için kullanılan gerçek değerlerdir. Örneğin,

```
System.out.println(Math.pow(2, 3)); // 8.0'ı gösterir
System.out.println(Math.pow(4, 0.5)); // 2.0'ı gösterir
System.out.println(Math.pow(2.5, 2)); // 6.25'i gösterir
System.out.println(Math.pow(2.5, -2)); // 0.16'ı gösterir
```

Bölüm 5, yöntemler hakkında daha fazla ayrıntı sunmaktadır. Şu anda bilmeniz gereken tek şey, üstel işlemi gerçekleştirmek için **pow** yöntemini nasıl çağıracağınızdır.

Point Kontrol

MyProgrammingLab™

2.8 En büyük ve en küçük **byte**, **short**, **int**, **long**, **float** ve **double**'i bulun. Bu veri türlerinden hangisi en az bellek gerektirir?

2.9 Aşağıdaki kalıntıların sonucunu gösterin.

```
56 %6
78 %-4
-34%5
-34%-5
5 %1
1 %5
```

2.10 Bugün Salı ise, 100 gün sonra hangi gün olacak?

2.11 25 / 4'ün sonucu nedir? Sonucun bir ondalık sayı olmasını istiyorsanız, ifadeyi nasıl yeniden yazarsınız?

2.12 Aşağıdaki ifadeler doğru mu? Eğer öyleyse, çıktıyı gösterin.

```
System.out.println("25 / 4, "+25/4);
System.out.println("25 / 4.0, "+25/4.0);
System.out.println("3 * 2 / 4, "+3*2/4);
System.out.println("3.0 * 2 / 4, "+3.0*2/4);
```

2.13 Sonucu göstermek için bir ifade yazın $2^{3.5}$.

2.14 Varsayalım **kin** ve **ve** tam sayılardır. Bir ondalık sonuç elde etmek için bir Java ifadesi yazın Martha bir ondalık sonuç elde etmek için ondalık sonuç.

2.10 Sayısal Literaller

Anahtar Nokta

Aliteral, bir programda doğrudan görünen sabit bir değerdir.

Örneğin, **34** ve **0.305**, aşağıdaki ifadelerde literallerdir:

```
int numberOfYears = t34;
double weight = t0.305;
```

2.10.1 Tam Sayı Literalleri

Bir tam sayı literal'i, değişkenin içine sığabildiği sürece bir tam sayı değişkenine atanabilir. Eğer literal, değişkenin tutabileceğinden çok büyükse bir derleme hatası oluşur. Örneğin, **byteb = 128** ifadesi bir derleme hatasına neden olacaktır, çünkü **128** bir **byte** türündeki değişkende saklanamaz. (Bir byte değeri için aralığın **-128** ile **127** arasında olduğunu unutmayın.)

gerçek

2.11 İfadeleri Değerlendirme ve Operatör Önceliği



Java ifadeleri, aritmetik ifadelerle aynı şekilde değerlendirilir.

Java'da bir sayısal ifade yazmak, Java operatörlerini kullanarak bir aritmetik ifadenin basit bir çevirisini içerir. Örneğin, aritmetik ifade

$$\frac{3 + 4tx}{5} - \frac{10(tyt - t5)(tat + tbt + tct)}{x} + 9tt4txt + \frac{9t + tx}{y}$$

şu şekilde bir Java ifadesine çevrilebilir:

`(t3t+t4t* x) / t5tt10t* (y - t5t) * (a + b + c) / x +
9 * (t4t/ x + (t9t+ x) / y)`

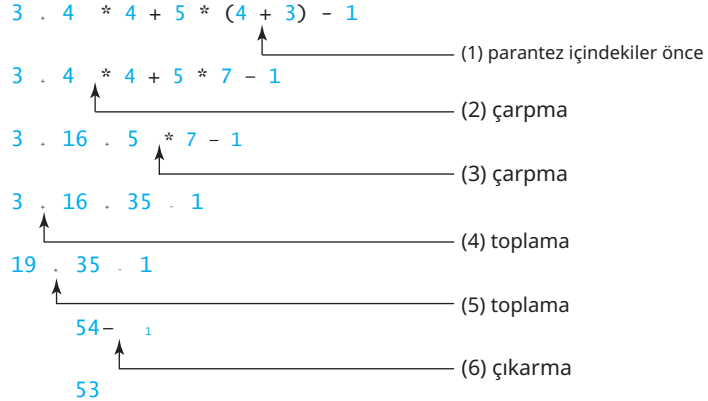
bir ifadeyi değerlendirme

operatör öncelik kuralı

Java'nın sahne arkasında bir ifadeyi değerlendirmek için kendi yolu olsa da, bir Javaifadesinin sonucu ve karşılık gelen aritmetik ifadesi aynıdır. Bu nedenle, bir Java ifadesini değerlendirmek için aritmetik kuralı güvenle uygulayabilirsiniz. Parantez çiftleri içindeki operatörler öncelikle değerlendirilir. Parantezler iç içe geçebilir; bu durumda içteki parantezlerdeki ifade öncelikle değerlendirilir. Bir ifadede birden fazla operatör kullanıldığında, değerlendirme sırasını belirlemek için aşağıdaki operatör öncelik kuralı kullanılır.

- Çarpma, bölme ve kalan operatörleri öncelikle uygulanır. Eğer bir ifadebirden fazla çarpma, bölme ve kalan operatörü içeriyorsa, soldan sağa doğru uygulanır.
- Toplama ve çıkarma operatörleri en son uygulanır. Eğer bir ifade birden fazla toplama ve çıkarma operatörü içeriyorsa, soldan sağa doğru uygulanır.

İşte bir ifadenin nasıl değerlendirildiğine dair bir örnek:



Liste 2.5, bir Fahrenheit derecesini Celsius'a dönüştüren bir programı formül kullanarak vermektedir

*santigrat = 5/9 * (t Fahrenheit - 32).*

LISTE 2.5 FahrenheitToCelsius.java

```
1 import java.util.Scanner;
2
3 genel sınıf FahrenheitToCelsius {
4     public static void main(String[] args) {
5         Tarayıcı girişi = new Scanner(System.in);
6
7         System.out.print("Fahrenheit'te bir derece girin: ");
```

```

8      double fahrenheit = input.nextDouble();
9
10     // Fahrenheit'ı Celsius'a dönüştür
11     çift santigrat = (5.0/9) * (fahrenheit - t32t);
12     System.out.println("Fahrenheit "+ fahrenheit + " şu kadardır "+
13         celsius + " Celsius'ta");
14 }n15 }

```

böl

Fahrenheit cinsinden bir derece girin:
 Fahrenheit 100.0, Celsius'ta 37.77777777777778'dir



satır#	fahrenheit	santigrat
8	100	
11		37.77777777777778



Bölme uygularken dikkatli olun. İki tam sayının bölümü Java'da bir tam sayı verir⁵.
 11. satırda 5.0 / 9 yerine 5 / 9 olarak çevrilmiştir, çünkü 5 / 9 Java'da 0 verir.

tam sayı vs. ondalık bölme

2.17 Aşağıdaki aritmetik ifadeyi Java'da nasıl yazarsınız?

$$\frac{4}{3(trt + t34)t - t9(tat + t bct)t} + \frac{3 + dt(2t + t en)}{a + bd}$$

Point Kontrol
MyProgrammingLab™

2.12 Vaka Çalışması: Mevcut Zamanı Gösterme

Mevcut zamanı döndürmek için `System.currentTimeMillis()` çağırabilirsiniz.

Problem, mevcut zamanı GMT (Greenwich Ortalama

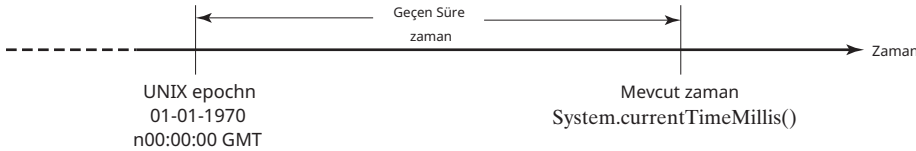
Zamanı) saat:dakika:saniye formatında, örneğin 13:19:8 olarak gösteren bir program geliştirmektir.

System sınıfındaki `currentTimeMillis` metodu, 1 Ocak 1970 GMT'de 00:00:00 zamanından itibaren geçen milisaniye cinsinden mevcut zamanı döndürür; bu, Şekil 2.2'de gösterilmiştir. Bu zaman *UNIX epoch* olarak bilinir. Epoch, zamanın başladığı noktadır ve **1970**, UNIX işletim sisteminin resmi olarak tanıtıldığı yıldır.



Operatörleri / ve % kullanın

`currentTimeMills`
 UNIX dönemi



FiÇERİK 2.2 `System.currentTimeMillis()`, UNIX epoch'undan bu yana geçen milisaniye sayısını döndürür.

Bu yöntemi mevcut zamanı elde etmek için kullanabilir ve ardından mevcut saniyeyi, dakika ve saati aşağıdaki gibi hesaplayabilirsiniz.

1. `System.currentTimeMillis()` çağırısını yaparak gece yarısından, 1 Ocak 1970'ten itibaren geçen toplam milisaniyeyi `totalMilliseconds` olarak elde edin (örneğin, **1203183068328** milisaniye).

52 Bölüm 2 Temel Programlama

2. `totalMilliseconds`'i 1000 ile bölerek toplam saniyeyi `totalSeconds` elde edin (örneğin, `1203183068328` milisaniye / 1000 = `1203183068` saniye).
3. `totalSeconds % 60` kullanarak mevcut saniyeyi hesaplayın (örneğin, `1203183068` saniye % 60 = 8, bu mevcut saniyedir).
4. `totalSeconds`'i 60 ile bölerek toplam dakikayı `totalMinutes` elde edin (örneğin, `1203183068` saniye / 60 = `20053051` dakika).
5. Mevcut dakikayı hesaplayın `totalMinutes % 60` (örneğin, `20053051` dakikalar % 60 = 31, şu anki dakikadır).
6. Toplam saatleri `totalHours` elde etmek için `totalMinutes`'i 60 ile bölün (örneğin, `20053051` dakikalar / 60 = `334217` saatler).
7. Mevcut saati `totalHours % 24` ile hesaplayın (örneğin, `334217` saat % 24 = 17, bu mevcut saattir).

Liste 2.6, tam programı vermektedir.

LISTE 2.6 ShowCurrentTime.java

```
1 genel classtShowCurrentTime {
2     public static void main(String[] args) {
3         // Gece yarısından itibaren geçen toplam milisaniyeyi alın, 1 Ocak 1970
4         long totalMilliseconds = System.currentTimeMillis();
5
6         // Gece yarısından, 1 Ocak 1970'ten itibaren toplam saniyeleri al
7         long totalSeconds = totalMilliseconds / 1000;
8
9         // Dakikadaki mevcut saniyeyi hesapla
10        long currentSecond = totalSeconds % 60;
11
12        // Toplam dakikaları al
13        long totalMinutes = totalSeconds / 60;
14
15        // Saatteki mevcut dakikayı hesapla
16        long currentMinute = totalMinutes % 60;
17
18        // Toplam saatleri al
19        long totalHours = totalMinutes / 60;
20
21        // Mevcut saati hesapla
22        long currentHour = totalHours % 24;
23
24        // Sonuçları göster
25        System.out.println("Mevcut zaman " + currentHour + ":"
26                           + currentMinute + ":" + currentSecond + " GMT");
27    }
28 }
```



Mevcut zaman 17:31:8 GMT

4. Satır, mevcut zamanı milisaniye cinsinden almak için `System.currentTimeMillis()` çağrısını yapar ve bu değeri long türünde alır. Bu nedenle, tüm değişkenler bu programda long türü olarak tanımlanmıştır. Saniyeler, dakikalar ve saatler, mevcut zamandan / ve% operatörleri kullanılarak çıkarılır (satırlar 6–22).

değişkenler	satır#	4	7	10	13	16	19	22
toplamMilisaniye		1203183068328						
toplamSaniye			1203183068					
mevcutSaniye				8				
toplamDakika					20053051			
mevcutDakika						31		
toplamSaat							334217	
mevcutSaat								17



Örnek çalışmada, ikinci için tek bir rakam 8 görüntülenir. İstenen çıktı

08 şeklinde olmalıdır. Bu, tek bir rakamı bir ön ek ile biçimlendiren bir fonksiyon kullanılarak düzeltilebilir.

0 (bkz. Alıştırma 5.37).

2.13 Artırılmış Atama Operatörleri

Operatörler +, -, *, / ve %, atama operatörü ile birleştirilerek

artırılmış operatörler oluşturulabilir.



Çoğu zaman bir değişkenin mevcut değeri kullanılır, değiştirilir ve ardından aynı değeri geri atanır. Örneğin, aşağıdaki ifade **count** değişkenini 1 artırır:

```
sayim = sayim + 1;
```

Java, artırılmış (veya bileşik) atama operatörü kullanarak atama ve toplama operatörlerini birleştirmenize olanak tanır. Örneğin, yukarıdaki ifade şu şekilde yazılabilir:

```
sayim += 1;
```

+=, toplama atama operatörü olarak adlandırılır. Tablo 2.4, diğer artırılmış atama operatörlerini göstermektedir.

TABLO 2.4 Artırılmış Atama Operatörleri

Operatör	İsim	Örnek	Eşdeğer
+=	Ek görev	ben += 8	ve = ve + 8n
-=	Çıkarma ataması	ben -= 8	ve = ve - 8n
=	Çarpma ataması	ben *= 8	ve = ve * 8n
/=	Bölme ataması	ben /= 8	ve = ve / 8n
%	Kalan ataması	ben %= 8	ben = ben % 8

**Dikkat**

Artırılmış atama operatörlerinde boşluk yoktur. Örneğin, `+=` olmalıdır.

**Not**

Atama operatörü (`=`) gibi, operatörler (`+=`, `-=`, `*=`, `/=`, `%=`) bir atama ifadesi ve bir ifade oluşturmak için kullanılabilir. Örneğin, aşağıdaki kodda, `x += 2` birinci satırda bir ifadedir ve ikinci satırda bir ifadedir.

```
x += t2t; // İfade
System.out.println(x += t2n); // İfade
```

2.14 Artırma ve Azaltma Operatörleri



Anahtar
Nokta

Artırma (`++`) ve azaltma (`--`) operatörleri, bir değişkeni 1 artırmak ve azaltmak için kullanılır.

arttırma operatörü (`++`)
azaltma operatörü (`--`)

`++` ve `--`, bir değişkeni 1 artırmak ve azaltmak için iki kısayol operatörüdür. Bunlar kullanışlıdır, çünkü birçok programlama görevinde değerin ne kadar değiştiği lmesi gerektiği genellikle budur. Örneğin, aşağıdaki kod `i`'yi 1 artırır ve `j`'yi 1 azaltır.

```
int i = 3, j = 3;
i++; // i 4 olur
j--; // j 2 olur
```

artma ve az
alma sonrası

`i++` olarak `i` artı artı ve `i--` olarak `i` eksi eksi şeklinde telaffuz edilir. Bu operatörler *postfix artırma* (veya postincrement) ve *postfix azaltma* (veya postdecrement) olarak bilinir, çünkü operatörler `++` ve `--` değişkenin ardından gelir. Bu operatörler ayrıca

değişkenin önünde de yer alabilir. Örneğin,

```
int i = 3, j = 3;
++i; // i 4 olur
--j; // j 2 olur
```

ön artış ve
ön azalma

`++i` 'yi 1 artırır ve `--j` 'yi 1 azaltır. Bu operatörler prefix *increment* (veya preincrement) ve *prefix decrement* (veya predecrement) olarak bilinir.

Gördüğünüz gibi, `++` ve `++i` veya `i--` ve `--i` etkileri önceki örneklerde aynıdır.

Ancak, bunların etkileri, yalnızca artırma ve azaltmanın ötesinde daha fazlasını yapan ifadelerde kullanıldıklarında farklıdır.

Tablo 2.5, bunların farklarını açıklar ve örnekler verir.

TABLO 2.5 Artırma ve Azaltma Operatörleri

Operatör	İsim	Açıklama	Örnek (<i>i</i> = 1 varsayalım)
<code>++var</code>	öncelikle artırma	<code>var</code> 'ı 1 artır ve ifadede yeni <code>var</code> değerini kullan	<code>int j = ++i;</code> <code>// j 2, i 2'dir</code>
<code>++ idi</code>	sonradan artırma	Artır <code>var</code> 1 ile, ancak orijinal <code>var</code> değerini ifadede kullan	<code>int j = i++;</code> <code>// j 1, i 2'dir</code>
<code>-- öyleydi</code>	önceden azaltma	Aşağı <code>var</code> 1 ile, ve ifadede yeni <code>var</code> değerini kullan	<code>int j = --i;</code> <code>// j 0, i 0'dir</code>
<code>öyleydi --</code>	sonradan azaltma	Decrement <code>var</code> 1 ile azaltın ve ifadede orijinal <code>var</code> değerini kullanın	<code>int j = i--;</code> <code>// j 1, i 0'dir</code>

İşte ++ (veya

--) ön ek biçimi ile ++ (veya--) son ek biçimi arasındaki farkları göstermek için ek örnekler:

```
ints = t 10;
```

```
intt yeniNum = t10t* i++;
```

Aynı etki

```
NTnNm = T10T*E; ne = e  
+ 1;
```

```
System.out.print("i şu anda "+ i  
+ ", newNum "+ newNum);
```

i 11, newNum 100'dür



Bu durumda, i 1 ile artırılır, ardından i'nin eski değeri çarpımda kullanılır. Böylece newNum 100 olur. Eğer i++ yerine ++i aşağıdaki gibi değiştirilirse,

```
ints = t 10;
```

```
intt yeniNum = t 10t * (++i);
```

Aynı etki

```
ben = ben + 1;  
inttnewNum = t10t* i;
```

```
System.out.print("i şu anda "+ i  
+ ", newNum "+ newNum);
```

i 11'dir, newNum 110'dur



i 1 ile artırılır ve i'nin yeni değeri çarpımda kullanılır. Böylece newNum 110 olur.

İşte başka bir örnek:

```
doubletx = t1.0t;nd  
oublety = t5.0t;ndo  
ubletz = x + (++y);
```

Üç satır çalıştırıldıktan sonra, y6.0 olur, z7.0 olur ve x0.0 olur.



İpucu

Artırma ve azaltma operatörlerini kullanmak ifadeleri kısaltır, ancak aynı zamanda bunları karmaşık ve okunması zor hale getirir. Birden fazla değişkeni veya aynı değişkeni birden fazla kez değiştiren ifadelerde bu operatörleri kullanmaktan kaçının, örneğin: `int k = ++i + i`.

2.18 Bu ifadelerden hangileri doğrudur?

- Herhangi bir ifade bir ifade olarak kullanılabilir.
- İfade `x++` bir ifade olarak kullanılabilir.
- İfade `x = x + 5` aynı zamanda bir ifadedir.
- İfade `x = y = x = 0` yasadışı.

2.19 Varsayalım ki `int a = 1` ve `double d = 1.0` ve her ifade bağımsızdır. Aşağıdaki ifadelerin sonuçları nedir?

```
a = t46t/t9t;  
A = T46T%Taqt+T4T*T4T-Taqt;Na  
= T45T+T43T%Taqt* (itaat*Taqt%Taqt);Na %  
= Taat/A + Taat;  
d = t4t+ d * d + t4t;nd  
+= t1.5t*t3t+ (++a);nd -= t1.  
5t*t3t+ a++;
```

Point Kontrol

MyProgrammingLab™

2.20 System.currentTimeMillis()

metodunu kullanarak mevcut dakikayı nasıl elde edersiniz?

2.15 Sayısal Tür Dönüşümleri



Kesirli sayılar açık casting kullanılarak tam sayılara dönüştürülebilir.

Farklı türde iki operand ile ikili işlemler gerçekleştirebilir misiniz? Evet. Eğer bir tam sayı ve bir kesirli sayı bir ikili işlemde yer alıyorsa, Java otomatik olarak tam sayıyı kesirli bir değere dönüştürür. Yani, `3 * 4.5`, `3.0 * 4.5` ile aynıdır.

Her zaman daha geniş bir değer aralığını destekleyen bir sayısal değişkene bir değer atayabilirsiniz; bu nedenle, örneğin, bir `long` değerini bir `float` değişkenine atayabilirsiniz. Ancak, daha küçük bir aralığa sahip bir türdeki bir değere bir değer atayamazsınız, aksi takdirde *tür casting'i kullanmalısınız*.

Tip dönüştürme, bir veri türünün değerini başka bir veri türünün değerine dönüştüren bir işlemdir. Küçük bir aralığa sahip bir türü daha büyük bir aralığa sahip bir türe dönüştürmeye genişletme denir. Büyük bir aralığa sahip bir türü daha küçük bir aralığa sahip bir türe dönüştürmeye daraltma denir. Java, bir türü otomatik olarak genişletecektir, ancak bir türü açıkça daraltmalısınız.

Bir türü dönüştürmenin sözdizimi, hedef türü parantez içinde belirtmek ve ardından değişkenin adını veya dönüştürülecek değeri yazmaktır. Örneğin, aşağıdaki ifade

```
System.out.println((tint)t1.7t);
```

1'yi görüntüler. Bir `double` değeri `int` değerine dönüştürüldüğünde, kesirli kısım kesilir. Aşağıdaki ifade

```
System.out.println((tdouble)t1t/t2t);
```

0.5'yi görüntüler, çünkü 1 önce 1.0'a dönüştürülür, ardından 1.0 2'ye bölünür. Ancak, ifade

```
System.out.println(t1t/t2t);
```

0'yi görüntüler, çünkü 1 ve 2 her ikisi de tam sayılardır ve sonuç değeri de bir tam sayı olmalıdır.

döküm
bir türü genişletme
bir türü daraltma

kesinlik kaybı olasılığı



Dikkat

Dönüştürme, daha küçük bir tür aralığına sahip bir değişkene değer atıyorsanız gereklidir, örneğin bir `double` değerini bir `int` değişkenine atamak gibi. Bu tür durumlarda dönüştürme yapılmazsa bir derleme hatası oluşur. Ancak, dönüştürme kullanırken dikkatli olun, çünkü bilgi kayıphatalı sonuçlara yol açabilir.



Not

Dönüştürme, dönüştürülen değişkeni değiştirmez. Örneğin, aşağıdaki kodda `d` dönüştürüldükten sonra değişmez:

```
double d = t4.line; d;  
oksan = (tint)d; // i 4 olur, ama d hala 4.5'tir
```



Not

Java'da, `x1 op= x2` biçimindeki bir artırılmış ifade, `x1 = (T)(x1 op x2)` olarak uygulanır; burada `T`, `x1` için türdür. Bu nedenle, aşağıdaki kod doğrudur.

```
int sum = t0t;  
sum += 4.5; // bu ifadeden sonra sum 4 olur  
  
sum += 4.5, sum = (int)(sum + 4.5) ile eşdeğerdir.
```

artırılmış
ifade içinde casting



Not

int türündeki bir değişkeni short veya byte türündeki bir değişkene atamak için, açık casting kullanılmalıdır. Örneğin, aşağıdaki ifadeler derleme hatası verir:

```
int = t1t;
byteb = i; // Hata çünkü açık tür dönüşümü gereklidir
```

Ancak, tamsayı literalı hedef değişkenin izin verilen aralığında olduğu sürece, bir tamsayı literalı bir **short** türündeki bir değişkene atamak için açık tür dönüşümü gerekli değildir (bkz. Bölüm 2.10, Sayısal Literaller).

Liste 2.7'deki program, ondalık noktadan sonra iki basamaklı satış vergisini görüntüler.

LISTE2.7SalesTax.java

```
1  import java.util.Scanner;
2
3  genel sınıfSatışVergisi {
4      public static voidmain(String[] args) {
5          Tarayıcı girişi = tnewScanner(System.in);
6
7          System.out.print("Satın alma tutarını girin: ");
8          doubletpurchaseTutar = input.nextDouble();
9
10         doublettax = satın alma Tutarı * t0,06t;
11         System.out.println("Satış vergisi $" + (int)(vergi * 100) / 100.0);
12     }
13 }
```

casting

Satın alma tutarını girin: 197.55

Satış vergisi \$11.85'tir



satır#	satınalmaTutarı	vergi	çıktı
8	197.55		
10		11.853	
11			11.85



Değişken purchaseAmount 197.55 (satır 8)'dir. Satış vergisi %6'dır, bu nedenle tax 11.853 (satır 10) olarak değerlendirilir. Not edin ki

sayıları biçimlendirme

```
tax * 100 1185.3'tür
(int)(tax * 100) 1185'tir
(int)(tax * 100) / 100.0 11.85'tir
```

Bu nedenle, satır 11'deki ifade vergi 11.85'i ondalık noktadan sonra iki basamakla gösterir.

2.21 Farklı türdeki sayısal değerler bir hesaplamada birlikte kullanılabilir mi?

2.22 double'dan int'ye açık bir tür dönüşümü,

double değerinin kesirli kısmıyla ne yapar? Dönüşüm, dönüştürülen değişkeni değiştirir mi?

2.23 Aşağıdaki çıktıyı göster:

```
vltf = t12.kht;ninti = (
renk tonu)f;
```

Point Kontrol

MyProgrammingLab™

```
System.out.println("f şudur "+ f);
System.out.println("i şudur "+ i);
```

- 2.24** Eğer 11. satırda `(int)(tax * 100) / 100.0` ifadesini `(int)(tax * 100) / 100` olarak değiştirirseniz, **197.556** giriş satın alma tutarı için çıktı ne olacaktır?

2.16 Yazılım Geliştirme Süreci



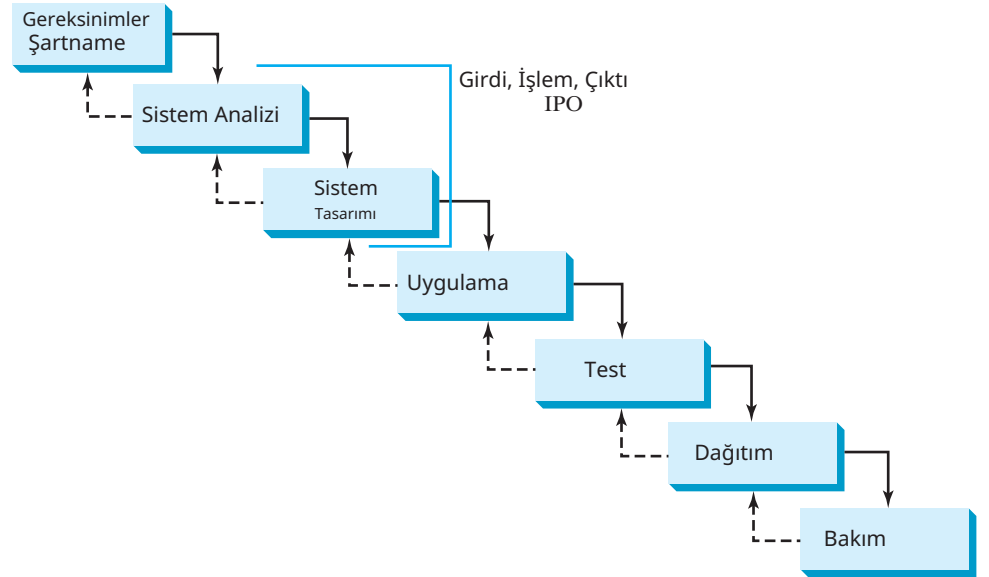
Yazılım geliştirme yaşam döngüsü, gereksinimlerin belirlenmesi, analizi, tasarımı, uygulanması, test edilmesi, dağıtımı ve bakımı içeren çok aşamalı bir süreçtir.

Bir yazılım ürünü geliştirmek mühendislik sürecidir. Yazılım ürünleri, ne kadar büyük veya küçük olursa olsun, aynı yaşam döngüsüne sahiptir: gereksinimlerin belirlenmesi, analizi, tasarımı, uygulanması, test edilmesi, dağıtımı ve bakımı, Şekil 2.3'te gösterildiği gibi.



VideoNot

Yazılım geliştirme süreci



FIGİN 2.3 Yazılım geliştirme yaşam döngüsünün herhangi bir aşamasında, yazılımın beklendiği gibi çalışmasını engelleyebilecek hataları düzeltmek veya diğer sorunlarla

baş etmek için önceki bir aşamaya geri dönmek gerekebilir.

gereksinimlerin belirlenmesi

Gereksinimlerin belirlenmesi, yazılımın ele alacağı problemi anlamayı ve yazılım sisteminin ne yapması gerektiğini ayrıntılı olarak belgelemeyi amaçlayan resmi bir süreçtir. Bu aşama, kullanıcılar ve geliştiriciler arasında yakın etkileşim gerektirir. Bu kitapta yer alan çoğu örnek basittir ve gereksinimleri açıkça belirtilmiştir. Ancak gerçek dünyada, problemler her zaman iyi tanımlanmış değildir. Geliştiricilerin, yazılımı kullanacak bireyler veya organizasyonlar ile yakın çalışmaları ve yazılımın ne yapması gerektiğini belirlemek için problemi dikkatlice incelemeleri gerekir.

sistem analizi

Sistem analizi, veri akışını analiz etmeyi ve sistemin girdi ve çıktısını tanımlamayı amaçlar. Analiz yaparken, önce çıktının ne olduğunu belirlemek ve ardından çıktıyı üretmek için hangi girdi verilerine ihtiyacınız olduğunu anlamak faydalıdır.

sistem tasarımı

Sistem tasarımı, girdiden çıktıyı elde etmek için bir süreç tasarlamaktır. Bu aşama, problemi yönetilebilir bileşenlere ayırmak ve her bileşeni uygulamak için tasarım stratejileri geliştirmek amacıyla birçok soyutlama seviyesinin kullanılmasını içerir. Her bir bileşeni, sistemin belirli bir işlevini yerine getiren bir alt sistem olarak görebilirsiniz. Sistem analizi ve tasarımının özü girdi, süreç ve çıktıdır (IPO).

DURUM

Uygulama, sistem tasarımını programlara dönüştürmeyi içerir. Her bileşen için ayrı programlar yazılır ve ardından birlikte çalışacak şekilde entegre edilir. Bu aşama, Java gibi bir programlama dilinin kullanılması gerektirir. Uygulama, kodlama, kendi kendine test etme ve hata ayıklama (yani, kodda bulunan hataları, *hatalar* olarak adlandırma) işlemlerini içerir.

uygulama

Test, kodun gereksinimlerin belirlenmesine uygun olup olmadığını sağlar ve hataları ayıklar. Ürünün tasarım ve uygulamasıyla ilgili olmayan bağımsız bir yazılım mühendisleri ekibi genellikle bu tür testleri gerçekleştirir.

test

Dağıtım yazılımı kullanılabilir hale getirir. Yazılımın türüne bağlı olarak, her kullanıcının makinesine kurulabilir veya İnternet üzerinden erişilebilen bir sunucuya kurulabilir.

dağıtım

Bakım ürünün güncellenmesi ve iyileştirilmesi ile ilgilidir. Bir yazılım ürünü, sürekli gelişen bir ortamda performans göstermeye ve iyileşmeye devam etmelidir. Bu, yeni keşfedilen hataları düzeltmek ve değişiklikleri entegre etmek için ürünün periyodik olarak güncellenmesini gerektirir.

bakım

Yazılım geliştirme sürecini eylemde görmek için, şimdi bir kredi ödemelerini hesaplayan bir program oluşturacağız. Kredi, bir araç kredisi, bir öğrenci kredisi veya bir konut ipoteği kredisi olabilir. Bir tanıtım programlama kursu için, gereksinimlerin belirlenmesi, analizi, tasarımı, uygulanması ve test edilmesine odaklanıyoruz.



VideoNot

Kredi ödemelerini hesaplayın

Aşama 1: Gereksinimlerin Belirlenmesi

Program aşağıdaki gereksinimleri karşılamalıdır:

- Kullanıcının faiz oranını, kredi miktarını ve ödemelerin yapılacağı yıl sayısını girmesine izin vermelidir.
- Aylık ödeme ve toplam ödeme miktarını hesaplayıp görüntülemelidir.

Aşama 2: Sistem Analizi

Çıktı, aylık ödeme ve toplam ödemedir; bu, aşağıdaki formüller kullanılarak elde edilebilir:

$$\text{aylıkÖdeme} = \frac{\text{krediMiktarı} \times \text{aylıkFaizOranı}}{1 - \frac{1}{(1 + \text{aylıkFaizOranı})^{\text{yılSayısı} \times 12}}}$$

$$\text{toplamÖdeme} = \text{aylıkÖdeme} \times \text{yılSayısı} \times 12$$

Yani, program için gereken giriş, aylık faiz oranı, kredinin süresi ve kredi miktarıdır.



Not

Gereksinim spesifikasyonu, kullanıcının yıllık faiz oranını, kredi miktarını ve ödemelerin yapılacağı yıl sayısını girmesi gerektiğini belirtmektedir. Ancak analiz sırasında, girişin yetersiz olduğunu veya bazı değerlerin çıktı için gereksiz olduğunu keşfedebilirsiniz. Bu durumda, gereksinim spesifikasyonunu geri dönüştürebilirsiniz.



Not

Gerçek dünyada, hayatın her kesiminden müşterilerle çalışacaksınız. Kimyagerler, fizikçiler, mühendisler, ekonomistler ve psikologlar için yazılım geliştirebilirsiniz ve elbette bu alanların hepsi hakkında (veya ihtiyaç duyduğunuz) tam bilgiye sahip olmanız gerekmiyor. Bu nedenle, formüllerin nasıl türetildiğini bilmenize gerek yok, ancak aylık faiz oranı, yıl sayısı ve kredi miktarı verildiğinde, bu programda aylık ödemeyi hesaplayabilirsiniz. Ancak, müşterilerle iletişim kurmanız ve bir matematiksel modelin sistem için nasıl çalıştığını anlamanız gerekecektir.

60 Bölüm 2 Temel Programlama

Aşama 3: Sistem Tasarımı

Sistem tasarımı sırasında, programdaki adımları belirlersiniz.

- Adım 1. Kullanıcıdan yıllık faiz oranını, yıl sayısını ve kredi miktarını girmesini isteyin.
- Adım 2. Yıllık faiz oranı girişi, % formatında bir sayı olmalıdır, örneğin 4.5%. Program, bunu 100 ile bölerek ondalık formata dönüştürmelidir. Yıllık faiz oranından aylık faiz oranını elde etmek için, bunu 12 ile bölün, çünkü bir yılda 12 ay vardır. Yani, aylık faiz oranını ondalık formatta elde etmek için, yıllık faiz oranını yüzde olarak 1200 ile bölmeniz gerekir. Örneğin, yıllık faiz oranı %4.5 ise, o zaman aylık faiz oranı $4.5/1200=0.00375$.
- Adım 3. Önceki formülü kullanarak aylık ödemeyi hesaplayın.
- Adım 4. Toplam ödemeyi hesaplayın, bu aylık ödemenin 12 ile çarpılması ve yıl sayısı ile çarpılmasıdır.
- Adım 5. Aylık ödemeyi ve toplam ödemeyi görüntüleyin.

Aşama 4: Uygulama

Uygulama, kodlama (kod yazma) olarak da bilinir. Formülde, hesaplamamız gereken $(1 + \text{aylıkFaizOranı})^{\text{kaçYıl} \times 12}$, şu şekilde elde edilebilir: `Math.pow(1 + aylıkFaizOranı, yılSayısı * 12)`.

Liste 2.8, tam programı vermektedir.

LİSTE 2.8 ComputeLoan.java

```
import sınıfı
123 import java.util.Scanner;
124
125 genel classtComputeLoan {
126     public static voidtmain(String[] args) {
127         // Bir Scanner oluşturun
128         Tarayıcı girişi = tnewtScanner(System.in);
129
130         // Yıllık faiz oranını yüzde olarak girin, örneğin, 7.25%
131         System.out.print("Yıllık faiz oranını girin, örneğin, 7.25%: ");
132         double yıllıkFaizOranı = input.nextDouble();
133
134         // Aylık faiz oranını elde et
135         double aylıkFaizOranı = yıllıkFaizOranı / 1200;
136
137         // Yıl sayısını girin
138         System.out.print(
139             "Bir tam sayı olarak yıl sayısını girin, örneğin, 5: ");
140         int kaçYıl = input.nextInt();
141
142         // Kredi miktarını girin
143         System.out.print("Kredi miktarını girin, örneğin, 120000.95: ");
144         double krediMiktarı = input.nextDouble();
145
146         // Ödemeyi hesapla
147         çift aylıkÖdeme = krediMiktarı * aylıkFaizOranı / (1
148             - 1/ Math.pow(1+ aylıkFaizOranı, yılSayısı * 12));
149         çift toplamÖdeme = aylıkÖdeme * yılSayısı * 12;
150
151         // Sonuçları göster
```

```

30      System.out.println("Aylık ödeme $" +
31          (int)(aylıkÖdeme * 100) / 100.0);
32      System.out.println("Toplam ödeme $" +
33          (int)(totalPayment * 100) / 100.0);
34 {Nah}

```

döküm

döküm

Yıllık faiz oranını girin, örneğin, 5.75%:

5.75 Enter

Yıl sayısını tam sayı olarak girin, örneğin, 5:

15 Enter

Kredi miktarını girin, örneğin, 120000.95:

250000 Enter

Aylık ödeme \$2076.02

Toplam ödeme \$373684.53



	satır#	10	13	18	22	25	27
değişkenler							
yıllıkFaizOranı		5.75					
aylıkFaizOranı			0.004791666666				
yılSayısı				15			
krediMiktarı					250000		
aylıkÖdeme						2076.0252175	
toplamÖdeme							373684.539



10. satır yıllık faiz oranını okur, bu oran 13. satırda aylık faiz oranına dönüştürülür.

Değişken için en uygun veri türünü seçin. Örneğin, numberOfYearsen iyi şekilde **int** (18. satır) olarak tanımlanır, ancak **long**, **float** veya **double** olarak da tanımlanabilir. **byte**, **numberOfYears** için en uygun olanı olabilir. Ancak basitlik açısından, bu kitapta verilen örnekler tam sayılar için **int** ve ondalık değerler için **double** kullanacaktır.

Aylık ödemenin hesaplanması için formül, 25-27. satırlarda Java koduna dönüştürülmüştür.

31 ve 33. satırlarda, ondalık noktadan sonra iki basamakla yeni bir **monthlyPayment** ve **totalPayment** almak için casting kullanılır.

Program, 1. satırda içe aktarılan **Scanner** sınıfını kullanır. Program ayrıca **Math** sınıfını kullanır ve bu sınıfın neden programa dahil edilmediğini merak ediyordunuz olabilir. **Math** sınıfı **java.lang** paketindedir ve **java.lang** paketindeki tüm sınıflar otomatik olarak **java.lang** paketi ile ithal edilir. Bu nedenle, **Math** sınıfını açıkça içe aktarmanıza gerek yoktur.

Aşama 5: Test Etme

Program uygulandıktan sonra, bazı örnek girdi verileri ile test edin ve çıktının doğru olup olmadığını doğrulayın. Bazı problemler birçok durumu içerebilir, bunu sonraki bölümlerde göreceksiniz. Bu tür problemler için, tüm durumları kapsayan test verileri tasarlamamız gerekir.



İpucu

Bu örnekte sistem tasarım aşaması birkaç adım belirledi. Bu adımları birer birer ekleyerek geliştirmek ve test etmek iyi bir yaklaşımdır. Bu yaklaşım, sorunları tespit etmeyi ve programı hata ayıklamayı çok daha kolay hale getirir.

artımlı geliştirme ve
test etme

2.25 Aşağıdaki aritmetik ifadesini nasıl yazarsınız?

$$\frac{-tbt + t \cdot 2tbt \cdot 2t - t \cdot 4tac}{\text{gelmek}}$$

2.17 Karakter Veri Türü ve İşlemleri

Bir karakter veri türü tek bir karakteri temsil eder.



char tür

Java'da sayısal değerleri işlemekle birlikte, karakterleri de işleyebilirsiniz. Karakter data türü, **char**, tek bir karakteri temsil etmek için kullanılır. Bir karakter literalı tekli alıntı işaretleri içinde yer alır. Aşağıdaki kodu düşünün:

```
harita mektubu = t'A't;
chartnumChar = t'4't;
```

İlk ifade karakter **A**'yi **char** değişkeni **letter**'e atar. İkinci ifade rakamsal karakter **4**'ü **char** değişkeni **numChar**'e atar.

**Dikkat**

Bir dize literalı, tırnak işaretleri (" ") içinde yer almalıdır. Bir karakter literalı, tek bir karakterin tek tırnak işaretleri (' ') içinde yer almasıdır. Bu nedenle, **"A"** bir dizedir, ancak **'A'** bir karakterdir.

grafik değişmezi

2.17.1 Unicode ve ASCII kodu

kodlama

Unicode

orijinal Unicode

tamamlayıcı Unicode

Bilgisayarlar içsel olarak ikili sayılar kullanır. Bir karakter, bir bilgisayarda 0 ve 1'lerin bir dizisi olarak saklanır. Bir karakterin ikili temsili eşlemek kodlama olarak adlandırılır. Bir karakteri kodlamanın farklı yolları vardır. Karakterlerin nasıl kodlandığı, bir kodlama şeması tarafından tanımlanır.

Java, dünyanın çeşitli dillerinde yazılı metinlerin değişimi, işlenmesi ve görüntülenmesini desteklemek için Unicode Konsorsiyumu tarafından oluşturulan bir kodlama şeması olan Unicode'i destekler. Unicode, başlangıçta 16 bitlik bir karakter kodlaması olarak tasarlanmıştır. Temel veri türü **char**, herhangi bir karakteri tutabilen basit bir veri türü sağlayarak bu tasarımda yararlanmak için tasarlanmıştır. Ancak, 16 bitlik bir kodlamada mümkün olan 65,536 karakterin, dünyadaki tüm karakterleri temsil etmek için yeterli olmadığı ortaya çıkmıştır. Bu nedenle, Unicode standardı 1,112,064 karaktere kadar izin verecek şekilde genişletilmiştir. Orijinal 16 bitlik sınırı aşan karakterler tamamlayıcı karakterler olarak adlandırılır. Java, tamamlayıcı karakterleri destekler. Tamamlayıcı karakterlerin işlenmesi ve temsil edilmesi, bu kitabın kapsamının dışındadır. Basitlik açısından, bu kitap yalnızca orijinal 16 bitlik Unicode karakterlerini dikkate alır. Bu karakterler, **char** türünde bir değişkende saklanabilir.

A 16-bit Unicode iki bayt alır, **\u** ile önceden belirtilmiş, dört onaltılık basamakla ifade edilir. **\u** arasında değişir. Onaltılık sayılar Ek F'de tanıtılmıştır, Number Sistemleri. Örneğin, İngilizce kelime **welcome**, Çince'de iki karakterle çevrilir, 欢迎. Bu iki karakterin Unicode'ları **\u0048\u0042\u0042\u0048\u0046\u0045\u0045**'dir.

Liste 2.9, iki Çince karakter ve üç Yunan harfi gösteren bir program vermektedir.

Listing 2.9: DisplayUnicode.java

```
1 javax.swing.JOptionPane'i içe aktarın;
2
3 genel sınıf Unicode'u görüntüle {
4     public static void main(String[] args) {
5         JOptionPane.showMessageDialog(null,
6             "\u0048\u0042\u0042\u0048\u0046\u0045\u0045",
7             "\u0048\u0042\u0042\u0048\u0046\u0045\u0045 Hoşgeldiniz",
```



```
8 JOptionPane.INFORMATION_MESSAGE);n 9 }
10 }
```

Sisteminizde Çince yazı tipi yüklü değilse, Çince karakterleri göremezsiniz. Yabancı harflerinin Unicode'ları Bay şunlardır \u03b1 \u03b2 \u03b3.

Çoğu bilgisayar ASCII (Amerikan Standart Bilgi Değişim Kodu) kullanır, bu, tüm büyük ve küçük harfleri, rakamları, noktalama işaretlerini ve kontrol karakterlerini temsil etmek için 7 bitlik bir kodlama şemasını ifade eder. Unicode, \u0000 ile \u007F arasında 128 ASCII karakterine karşılık gelen ASCII kodunu içerir. (ASCII karakterlerinin ve ondalık ile onaltılık kodlarının bir listesini görmek için Ek B'ye bakın.) Java programında 'X', '1' ve '\$' gibi ASCII karakterlerini ve Unicode'ları kullanabilirsiniz. Dolayısıyla, örneğin, aşağıdaki ifadeler eşdeğerdir:

```
harita mektubu = t'A't;
char letter = '\u0041'; // Karakter A'nın Unicode'u 0041'dir
```

Her iki ifade de karakter A'yı char değişkeni letter'a atar.



Not

Artırma ve azaltma operatörleri, bir sonraki veya önceki Unicode karakterini almak için char değişkenlerinde de kullanılabilir. Örneğin, aşağıdaki ifadeler karakter b'yı gosterir.

```
char ch = t'a't;nSystem.out.println(++ch);
```

karakter artırma ve düşürme

2.17.2 Kaçış Karakterleri

Çıktıda tırnak işaretleri ile bir mesaj yazdırmak istiyorsanız. Böyle bir ifade yazabilir misiniz?

```
System.out.println(t""Java eğlencelidir" dedi "t);
```

Hayır, bu ifade bir derleme hatası içeriyor. Derleyici, ikinci tırnak karakterinin stringin sonu olduğunu düşünür ve geri kalan karakterlerle ne yapacağını bilemez.

Bu sorunu aşmak için, Java özel karakterleri temsil etmek için özel bir notasyon kullanır; bu, Tablo 2.6'da gösterilmiştir. Bu özel notasyon, kaçış karakteri olarak adlandırılır ve bir ters eğik çizgi() ile ardından bir karakter veya karakter dizisi gelir. Örneğin, , Tab karakteri için bir kaçış karakteridir ve \u03b1 gibi bir kaçış karakteri, bir Unicode'u temsil etmek için kullanılır. Kaçış karakterindeki semboller, bireysel olarak değil, bir bütün olarak yorumlanır.

Bu nedenle, alıntılanan mesajı aşağıdaki ifade ile yazdırabilirsiniz:

```
System.out.println(t""Java eğlencelidir" dedi""t);
```

Çıktı şudur

O, "Java eğlencelidir" dedi

Sembollerin ve" birlikte bir karakteri temsil ettiğini unutmayın.

2.17.3 char ve Sayısal Türler Arasında Dönüştürme

Bir char herhangi bir sayısal türe dönüştürülebilir ve tersine. Bir tam sayı bir char'ye dönüştürüldüğünde, yalnızca alt 16 bit verisi kullanılır; diğer kısım göz ardı edilir. Örneğin:

```
char ch = (char)0XAB0041; // Alt 16 bit hex kodu 0041
// ch'ye atanır
System.out.println(ch); // ch karakter A'dır
```

TABLE 2.6 Kaçış Karakterleri

Kaçış Karakteri	İsim	Unicode Kodu	Ondalık Değer
B	Geri Alma	u0008	8
	Sekme	u0009	9
	Satır Sonu	u000A	10
F	Form Sonu	u000C	12
\r	Taşıma Dönüşü	u000D	13
\\	Tersten Eğik Çizgi	u005C	92
"	Çift Tırnak	u0022	34

Bir kayan nokta değeri bir **char** türüne dönüştürüldüğünde, kayan nokta değeri önce bir **int** türüne dönüştürülür, ardından bir **char** türüne dönüştürülür.

```
grafik = (çizelge)t65.25t; // Ondalık 65 ch'ye atanır
System.out.println(ch); // ch karakter A'dır
```

Bir **char** türü sayısal bir türe dönüştürüldüğünde, karakterin Unicode'u belirtilen sayısal türe dönüştürülür.

```
inti = (int)'A'; // A karakterinin Unicode'u i'ye atanır
System.out.println(i); // i 65'tir
```

Sonuç bir hedef değişkenine sığıyorsa, örtük dönüştürme kullanılabilir. Aksi takdirde, açık dönüştürme kullanılmalıdır. Örneğin, **'a'** karakterinin Unicode'u **97**'dir, bu da bir byte'ın aralığında olduğu için bu örtük dönüştürmeler uygundur:

```
bytetb = t'a't;nin
tti = t'a't;
```

Ancak aşağıdaki tür dönüştürmesi yanlıştır, çünkü Unicode **\uFFFF** bir bayta sığamaz:

```
bytetb = t'\uFFFF't;
```

Bu atamayı zorlamak için, aşağıdaki gibi açık tür dönüştürmesi kullanın:

```
bytetb = (tbytet)t'\uFFFF't;
```

0 ile FFFF arasında herhangi bir pozitif tam sayı, karaktere

örtük olarak dönüştürülebilir. Bu aralıkta olmayan herhangi bir sayı, **char** olarak açıkça dönüştürülmelidir.

karakterler üzerindeki sayısal
operatörler



Not

Tüm sayısal operatörler **char** operatörlerine uygulanabilir. Bir **char** operatörü, diğer operatör bir sayı veya karakter olduğunda otomatik olarak bir sayıya dönüştürülür. Diğer operatör bir dize ise, karakter dize ile birleştirilir. Örneğin, aşağıdaki ifadele

```
inti = '2' + '3'; // (int)'2' 50 ve (int)'3' 51'dir
System.out.println("İşu: " + i); // i 101'dir
```



```
int j = 2 + 'a'; // (int)'a' 97'dir
System.out.println("j şu: " + j); // j 99'dur
System.out.println(j + " karakterin Unicode'u "
    + (char)j); // j karakter c'nin Unicode'udur
System.out.println("Bölüm " + t + t);
```

görüntüle

```
i 101'dir
j 99'dur
99, karakter c için Unicode'dur
Bölüm 2
```



Not

Küçük harflerin Unicode'ları, Unicode'un başlangıcından itibaren ardışık tam sayılardır 'a', ardından 'b', 'c', . . . , ve 'z' için. Büyük harfler için de aynı şey geçerlidir. Ayrıca, 'a' için Unicode, 'A' için Unicode'dan büyüktür, bu nedenle 'a' - 'A', 'b' - 'B' ile aynıdır. Küçük bir harf *ch*, karşılık gelen büyük harfi (char)('A' + (ch - 'a')) olarak ifade eder.

2.17.4 Vaka Çalışması: Para Birimlerini Sayma

Diyelim ki, belirli bir miktar parayı daha küçük para birimlerine dönüştüren bir program geliştirmek istiyorsunuz. Program, kullanıcının toplamı dolar ve sent olarak temsil eden bir **double** d eğerini girmesine izin verir ve minimum sayıda madeni para elde etmek için bu sırayla maksimum sayıda dolar, çeyrek, onluk, beşlik ve sent cinsinden para eşdeğerini listeleyen bir rapor çıktısı verir; örnek çalışmada gösterildiği gibi.

Programı geliştirme adımları şunlardır:

1. Kullanıcıdan, **11.56** gibi bir ondalık sayı olarak miktarı girmesini isteyin.
2. Miktarı (örneğin, **11.56**) sent cinsine (**1156**) dönüştürün.
3. Dolar sayısını bulmak için sentleri **100**'e bölün. Kalan sentleri **100** ile kalan sentleri elde edin.
4. Kalan sentleri **25**'e bölerek çeyrek sayısını bulun. Kalan sentleri **25** ile kalan sentleri elde edin.
5. Kalan sentleri **10** ile bölerek onlukların sayısını bulun. Kalan sentleri kalan sentler mod **10** kullanarak elde edin.
6. Kalan sentleri **5** ile bölerek beşliklerin sayısını bulun. Kalan sentleri kalan sentler mod **5** kullanarak elde edin.
7. Kalan sentler, kuruşlardır.
8. Sonucu göster.

Tam program, Liste 2.10'da verilmiştir.

LISTE 2.10 ComputeChange.java

```
1 import java.util.Scanner;
23
24 genel classtComputeChange {
25     public static voidtmain(String[] args) {
26         // Bir Scanner oluşturun
27         Tarayıcı girişi = tnewtScanner(System.in);
28
29         // Miktarı al
30         System.out.print(
```

import sınıfı

66 Bölüm 2 Temel Programlama

```

1 // Bir miktar girin, örneğin, 11.56: ");
2 doubletamount = input.nextDouble();
3
4 intkalanMiktar = (int)(miktar * 100);
5
6 // Bir doların sayısını bul
7 intnumberOfOneDollars = kalanTutar / t100t;
8 kalanMiktar = kalanMiktar % 100;
9
10 // Kalan miktardaki çeyreklerin sayısını bul
11 intnumberOfQuarters = kalanTutar / t25t;
12 kalanMiktar = kalanMiktar % 25;
13
14 // Kalan miktardaki onlukların sayısını bul
15 intnumberOfDimes = kalanTutar / t10t;
16 kalanMiktar = kalanMiktar % 10;
17
18 // Kalan miktardaki beşliklerin sayısını bul
19 intnumberOfNickels = kalanTutar / t5t;
20 kalanMiktar = kalanMiktar % 5;
21
22 // Kalan miktardaki sentlerin sayısını bul
23 intnumberOfPennies = kalanTutar;
24
25 // Sonuçları göster
26 System.out.println("Toplaminiz "+ amount + " şunlardan oluşmaktadır: "+
27     ""+ numberOfOneDollars + " dolar"+
28     ""+ numberOfQuarters + " çeyrek"+
29     ""+ numberOfDimes + " onluk"+
30     ""+ numberOfNickels + " beşlik"+
31     ""+ numberOfPennies + " kuruş");
32 }

```



Bir miktar girin, örneğin, 11.56:

11.56 miktarınız

11 dolardan oluşmaktadır

2 çeyrek

0 onluk

1 beşlik

1 kuruş

11.56 Enter

[illegible]

Değişken `miktar`, konsoldan girilen miktarı saklar (satır 11). Bu değışkendağı ştirilmez, çünkü miktar programın sonunda sonuçları göstermek için kullanılm alıdır. Program, kalanmiktarı saklamak için `kalanMiktar` (satır 13) değışkenini t anıtır.

Değişken `miktar`, dolar ve sentleri temsil eden bir `double` ondalık sayıdır. Bu, tüm sentleri temsil eden bir `int` değışkeni `kalanMiktar`'ye dönüştürölür. Örneğın, eğer `miktar` `11.56` ise, o zaman başlangıç `kalanMiktar` `1156`'dır. Bölme operatöröl, bölmenin tam sayı kısmını verir, bu nedenle `1156 / 100` `11`'dir. Kalan operatöröl, bölmenin kalanını alır, bu nedenle `1156 % 100` `56`'dır.

Program, kalan miktardan maksimum sayıda tekleri çıkarır ve `remainingAmount` değışkeninde yeni bir kalan miktar elde eder (satırlar 16–17). Daha sonra `remainingAmount` değışkeninden maksimum sayıda çeyrek çıkarır ve yeni bir `remainingAmount` elde eder (satırlar 20–21). Aynı işlemi devam ettirerek, program kalan miktardaki maksimum sayıda onluk, beşlik ve kuruşu bulur.

Bu örnekteki ciddi bir sorun, bir `double` miktarını `int remainingAmount`'ye dönüştürölürken hassasiyet kaybı olasılığıdır. Bu, yanlış bir sonuca yol açabilir. Miktarı `10.03` olarak girmeye çalıştığınızda, `10.03 * 100`, `1002.9999999999999` olur. Programın `10` dolar ve `2` kuruş gösterdiğini göreceksiniz. Sorunu düzeltmek için, miktarı kuruşları temsil eden bir tam sayı değeri olarak girin (bkz. Alıştırma 2.24).

Örnek çalışmada gösterildiğı gibi, sonuçta `0` dime, `1` nickel ve `1` kuruş görüntölünmektedir. `0` dime göstermemek ve `1` nickel ile `1` kuruşu kelimelerin tekil formlarını kullanarak göstermek daha iyi olurdu. Bu programı değıştirmek için seçim ifadelerini nasıl kullanacağınızı bir sonraki bölümde öğreneceksiniz (bkz. Alıştırma 3.7).

kesinlik kaybı

2.26 `'1'`, `'A'`, `'B'`, `'a'` ve `'b'` için ASCII kodunu bulmak için print ifadelerini kullanın. Onluk kodlar `40,59,79,85` ve `90` için karakteri bulmak için print ifadelerini kullanın. Onaltılık kod `40,5A,71,72` ve `7A` için karakteri bulmak için print ifadelerini kullanın.

2.27 Aşağıdakilerden hangileri karakterler için doğru literalardır?

`'1'`, `'t'`, `'u345dE'`, `'t'`, `'u3fFa'`, `'t'`, `'b'`, `'t'`, `'t'`

2.28 Karakterleri nasıl görüntölörsünüz ve ""?

2.29 Aşağıdakileri değeriendirin:

```
int = t'1't;
inttj = t'1't+t'2't*(t'4't-t'3't)+t'b't/t'a't;ninttk = t'a't;
```

`grafikc = t90t;`

2.30 Aşağıdaki tür dönüştömleri izin verilebilir mi? Eğer öyleyse, dönüştörlölmiş sonucunu bulun.

`Durum = enerji`

2.31 Aşağıdaki programın çıktısını gösterin:

```

genel sınıfTest {
    public static void main(String[] args) {
        nchartx = t'a't
        ;
        charty = t'c't;nSy

        stem.out.println(++x);nSystem.
        out.println(y++);nSystem.out.p
        rintln(x - y);
    }
}

```

2.18 String Türü



Bir dize, karakterlerin bir dizisidir.

char türü yalnızca bir karakteri temsil eder. Karakterler dizisini temsil etmek için, String adı verilen veri türünü kullanın. Örneğin, aşağıdaki kod mesajı "Java'ya Hoş Geldiniz" değeriyle bir dize olarak tanımlar.

```
String message = "Java'ya Hoş Geldiniz";
```

String, Java kütüphanesinde önceden tanımlanmış bir sınıftır, tıpkı **System**, **JOptionPane**, ve **Scanner** sınıfları gibi. **String** türü, ilkel bir tür değildir. Bir referans türü olarak bilinir. Herhangi bir Java sınıfı, bir değişken için referans türü olarak kullanılabilir. Referans veri türleri, Bölüm 8, Nesneler ve Sınıflar'da kapsamlı bir şekilde ele alınacaktır. Şu anda, yalnızca bir **String** değişkenini nasıl tanımlayacağınızı, değişkene bir dize atamayı ve dizeleri nasıl birleştireceğinizi bilmeniz gerekiyor.

İlk olarak Liste 2.1'de gösterildiği gibi, iki dize birleştirilebilir. Artı işareti (+), eğer operandlardan biri bir dize ise birleştirme operatörüdür. Eğer operandlardan biri bir dize değilse (örneğin, bir sayı), dize olmayan değer bir dizeye dönüştürülür ve diğer dize ile birleştirilir. İşte bazı örnekler:

```

// Üç dize birleştiriliyor
String message = "Hoş geldiniz "+"Java'ya "+"hoş geldiniz";

// Dize Bölüm, sayı 2 ile birleştiriliyor
String s = "Bölüm"+2; // s Bölüm2 oluyor

// Dize Ek, karakter B ile birleştiriliyor
String s1 = "Ek"+"B"; // s1 EkB oluyor

```

Eğer operandlardan hiçbiri bir dize değilse, artı işareti (+) iki sayıyı toplayan toplama operatörüdür.

Artırılmış += operatörü de dize birleştirme için kullanılabilir. Örneğin, aşağıdaki kod, dize "ve Java eğlencelidir" ile dize "Java'ya hoş geldiniz"yi message'ye ekler.

```
message += " ve Java eğlencelidir";
```

Yani yeni **message** "Java'ya hoş geldiniz ve Java eğlencelidir" oluyor.

Eğer **i = 1** ve **j = 2** ise, aşağıdaki ifadenin çıktısı nedir?

```
System.out.println("i + j = " + i + j);
```

Çıktı "i + j = 12"dir çünkü "i + j = " ilk olarak i'nin değeri ile birleştirilir. **i + j**'nin önce çalıştırılmasını sağlamak için, **i + j**'yi parantez içine alın, şu şekilde:

```
System.out.println("i + j toplamı " + (i + j));
```

string'leri ve
sayılara birleştir

Konsoldan bir string okumak için, bir `Scanner` nesnesinde `next()` metodunu çağırın. Örneğin, aşağıdaki kod klavyeden üç string okur:

string'leri oku

```
Scanner input = new Scanner(System.in);
System.out.println("Üç kelime girin, aralarında boşluk bırakın: ");
String s1 = input.next();
String s2 = input.next();
String s3 = input.next();
System.out.println("s1 şu: " + s1);
System.out.println("s2 şu: " + s2);
System.out.println("s3 şu: " + s3);
```

Üç kelime girin, aralarında boşluk bırakın:
s1 şu: Welcome
s2 şu: to
s3 Java'dır

Java'ya hoş geldiniz

↵ Enter



`next()` metodu, bir boşluk karakteri ile biten bir dize okur. Karakterler ' ', ',', '\r', veya boşluk karakterleri olarak bilinir.

boşluk karakteri

Bir metin satırını okumak için `nextLine()` metodunu kullanabilirsiniz. `nextLine()` metodu, Enter tuşuna basıldığında biten bir dize okur. Örneğin, aşağıdaki ifadeler bir metin satırını okur.

```
Scanner input = new Scanner(System.in);
System.out.println("Bir satır girin: ");
String s = input.nextLine();
System.out.println("Girilen satır " + s);
```

Bir satır girin: Java'ya hoş geldiniz
Girilen satır: Java'ya Hoş Geldiniz

↵ Enter



Önemli Uyarı

Giriş hatalarını önlemek için, `nextByte()`, `nextShort()`,

`nextInt()`, `nextLong()`, `nextFloat()`, `nextDouble()` veya `next()` sonrasında `nextLine()` kullanmayın. Sebepler

Bölüm 14.11.3, "Scanner Nasıl Çalışır?"da açıklanacaktır.

girdi hatalarını önleyin

2.32 Aşağıdaki ifadelerin çıktısını gösterin (sonuçlarınızı doğrulamak için bir program yazın):

```
System.out.println(t"1"t+t1t);nSyste
m.out.println(t"1"t+t1t);nSystem.out.
println(t"1"t+t1t+t1t);nSystem.out.println
(t"1"t+ (t1t+t1t));nSystem.out.println(t"1"t+t1
t+t1t);
```

Point Kontrol

MyProgrammingLab™

2.33 Aşağıdaki ifadeleri değerlendirin (sonuçlarınızı doğrulamak için bir program yazın):

```
1 + "Hoş Geldin " + 1 + 1
1 + "Hoş Geldin " + (1 + 1)
1 + "Hoş geldiniz " + ('\u0001' + 1)
1 + "Hoş geldiniz " + 'a' + 1
```

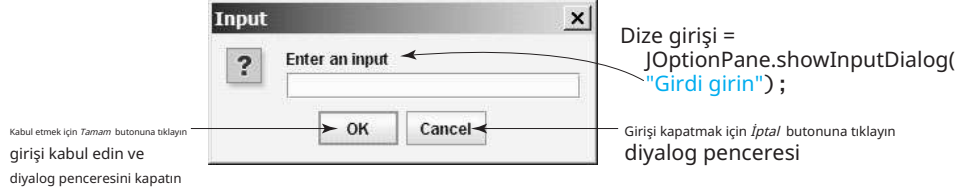
2.19 Giriş Diyaloglarından Girdi Alma



JOptionPane sınıfı

Bir giriş iletişim kutusu, kullanıcının girişi grafiksel olarak girmesini istemektedir.

Konsoldan giriş alabilirsiniz. Alternatif olarak, JOptionPane.showInputDialog yöntemini çağırarak bir giriş diyalog kutusundan giriş alabilirsiniz; bu, Şekil 2.4'te gösterilmiştir.



FIGİN2.4 Girdi diyalog kutusu kullanıcının bir dize girmesine olanak tanır.

Bu yöntem çalıştırıldığında, bir iletişim kutusu görüntülenir ve bir giriş değeri girmenizi sağlar. Bir dize girdikten sonra, girişi kabul etmek ve iletişim kutusunu kapatmak için *OK* düğmesine tıklayın. Giriş, yöntemden bir dize olarak döndürülür.

showInputDialog yöntemi

showInputDialog yöntemini kullanmanın birkaç yolu vardır. Şu anda, onu çağırmanın yalnızca iki yolunu bilmeniz gerekiyor.

Bunlardan biri, şu şekilde bir ifade kullanmaktır:

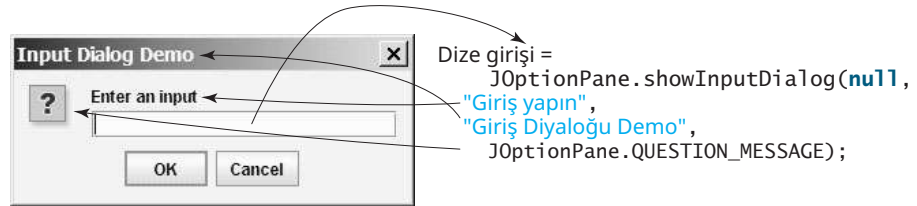
```
JOptionPane.showInputDialog(x);
```

burada *x*, istem mesajı için bir dizedir.

Diğeri ise aşağıdaki gibi bir ifade kullanmaktır:

```
Dize dize = JOptionPane.showInputDialog(tnullt, x, ny, JOptionPane.QUESTION_MESSAGE);
```

x, istemci mesajı için bir dize ve *y*, giriş iletişim kutusunun başlığı için bir dizedir örnekte gösterildiği gibi.



2.19.1 Dizeyi Sayılara Dönüştürme

Giriş iletişim kutusundan dönen değer bir dizedir. **123** gibi bir sayısal değer girerseniz, **"123"** olarak döner. Girişi bir sayı olarak elde etmek için bir dizeyi sayıya dönüştürmeniz gerekir.

Integer.parseInt yöntemi

Bir dizeyi bir *int* değerine dönüştürmek için *Integer.parseInt* yöntemini kullanın, aşağıdaki gibi:

```
intValue = Integer.parseInt(intString);
```

burada *intString* **123** gibi bir sayısal dizedir.

Double.parseDouble yöntem

Bir dizeyi bir *double* değerine dönüştürmek için *Double.parseDouble* yöntemini kullanın, aşağıdaki gibi:

```
doubletdoubleValue = Double.parseDouble(doubleString);
```

burada *doubleString* **123.45** gibi bir sayısal dizedir.

Integer ve *Double* sınıfları *java.lang* paketinde yer almaktadır ve bu nedenle otomatik olarak içe aktarılırlar.

2.19.2 Giriş Diyalog Kutuları Kullanma

Bir giriş diyalogu kutusundan giriş okumayı öğrendikten sonra, Listing 2.8, ComputeLoan.java'da ki programı, konsoldan değil, giriş diyalogu kutularından okumak için yeniden yazabilirsiniz. Listing 2.11 tam programı verir. Şekil 2.5 programın örnek çalışmasını gösterir.

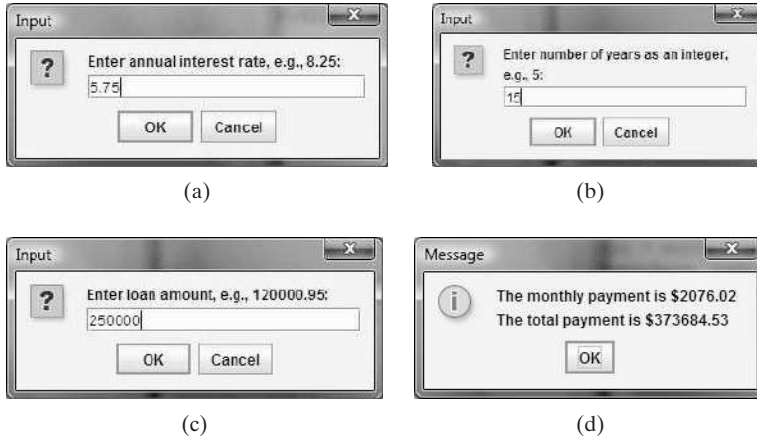


FIGURE 2.5 Program yıllık faiz oranını (a), yıl sayısını (b) ve kredi miktarını (c) alır, ardından aylık ödemeyi ve toplam ödemeyi (d) gösterir.

LİSTELER 2.1 | KrediHesaplaGirdiDiyalogu.java

```

1  import javax.swing.JOptionPane;
2
3  public class KrediHesaplaGirdiDiyalogu {
4      public static void tmain(String[] args) {
5          // Yıllık faiz oranını girin
6          String yıllıkFaizOranıString = JOptionPane.showInputDialog(
7              "Yıllık faiz oranını girin, örneğin, 8.25:");
8
9          // String'i double'a çevirin
10         double yıllıkFaizOranı =
11             Double.parseDouble(yıllıkFaizOranıString);
12
13         // Aylık faiz oranını elde et
14         double aylıkFaizOranı = yıllıkFaizOranı / 1200;
15
16         // Yıl sayısını girin
17         String yılSayısıString = JOptionPane.showInputDialog(
18             "Yılları bir tamsayı olarak girin, örneğin, 5:");
19
20         // Dizeyi int'e dönüştür
21         int numberOfYears = Integer.parseInt(numberOfYearsString);
22
23         // Kredi miktarını girin
24         String krediString = JOptionPane.showInputDialog(
25             "Kredi miktarını girin, örneğin, 120000.95:");
26
27         // String'i double'a çevirin
28         double loanAmount = Double.parseDouble(loanString);
29

```

72 Bölüm 2 Temel Programlama

```
aylıkÖdeme      30 // Ödemeyi hesapla
                  doubletmonthlyPayment = krediTutarı * aylıkFaizOranı / (t1
toplamÖdeme      31 32333435 - 1 / Math.pow(t1+ aylıkFaizOranı,Yıl Sayısı * t12t));
                  doublettotalPayment = aylıkÖdeme *Yıl Sayısı * t12t;

çıkıyı hazırlama 36373839 4041 // Ondalık noktadan sonra iki basamağı korumak için biçimlendir
                  Ödeme = (tintt)(aylık Ödeme * t100t) / t100.0t;
                  toplam Ödeme = (ton)(toplam Ödeme * t100t) / t100.0t; naylık

                  // Sonuçları göster
                  İlk ödeme $" + month1yPayment +
                  "Toplam ödeme $" + totalPayment;String output = "Ay
42 JOptionPane.showMessageDialog(tnullt,); çıktı
43 }n44
}
```

showInputDialog yöntemi 6-7. satırlarda bir giriş iletişim kutusu görüntüler. Faiz oranını bir double değeri olarak girin ve girişi kabul etmek için Tamam'a tıklayın. Değer, annualInterestRateString adlı String değişkenine atanan bir dize olarak döndürülür. Double.parseDouble(annualInterestRateString) (11. satır) dizeyi bir double değerine dönüştürmek için kullanılır.

JOptionPane veya Scanner?



Pedagogik Not

Girdi almak için ya **JOptionPane** ya da **Scanner** kullanabilirsiniz—hangisi daha uygun olursa. Tutarlılık ve basitlik açısından, bu kitapta girdi almak için **Scanner** kullanılarak örnekler verilmiştir. Örnekleri **JOptionPane**

kullanarak girdi almak için kolayca revize edebilirsiniz.

2.34 Neden **JOptionPane**'i içe aktarmanız gerekiyor da **Math** sınıfını değil?

2.35 Kullanıcıdan bir girdi girmesini nasıl bir diyalog kutusu ile istersiniz?

2.36 Bir dizeyi tam sayıya nasıl dönüştürsünüz? Bir dizeyi bir

çift sayıya nasıl dönüştürsünüz?

Point Kontrol

MyProgrammingLab™

ANAHTAR TERİMLER

algoritma 34	arttırma operatörü (++) 54
atama operatörü (=) 42	arttırmalı geliştirme
atama ifadesi 42	ve test 37
byte türü 45	int tür 45
casting 56	IPO 39
char türü 62	gerçek 48
sabit 43	long tür 45
veri türü 35	narrowing (türlerin daraltılması) 56
değişkenleri tanımla 35	operatörler 46
azaltma operatörü (- -) 54	operatör 46
double türü 45	aşım 45
kodlama 62	sonradan azaltma 54
kaçış karakteri 63	sonradan arttırma 54
ifade 42	önceden azaltma 54
final anahtar kelime 43	önceden arttırma 54
float türü 45	ilkel veri türü 35
kayan noktalı sayı 35	pseudocode 34
kimlik 40	gereksinim spesifikasyonu 58

değişkenin kapsamı 42
 short tür 45
 tamamlayıcı Unicode 62
 sistem analizi 58
 sistem tasarımı 58
 taşıma 46

Unicode 62
 UNIX epoch 51
 değişken 35
 boşluk karakteri 69
 tiplerin genişletilmesi 56

BÖLÜM ÖZETİ

1. *Tanımlayıcılar*, bir programda değişkenler, sabitler, metotlar, sınıflar, paketler gibi öğeleri adlandırmak için kullanılan isimlerdir.
2. Bir tanımlayıcı, harfler, rakamlar, alt çizgiler (`_`) ve dolar işaretleri (`$`) içeren bir karakter dizisidir. Bir tanımlayıcı bir harf veya alt çizgi ile başlamalıdır. Rakamla başlayamaz. Bir tanımlayıcı, rezerve edilmiş bir kelime olamaz. Bir tanımlayıcı herhangi bir uzunlukta olabilir.
3. *Değişkenler*, bir programda verileri saklamak için kullanılır.
4. Bir değişkeni bildirmek, derleyiciye bir değişkenin hangi tür veriyi tutabileceğini söylemektir.
5. Java'da, eşittir işareti (`=`) *atama operatörü olarak kullanılır*.
6. Bir metotta tanımlanan bir değişken, kullanılmadan önce bir değere atanmalıdır.
7. Bir *isimlendirilmiş sabit* (veya basitçe bir *sabit*) asla değişmeyen kalıcı veriyi temsil eder.
8. Bir isimlendirilmiş sabit, `final` anahtar kelimesi kullanılarak bildirilir.
9. Java, dört farklı boyutta tam sayıları temsil eden dört tamsayı türü (`byte`, `short`, `int` ve `long`) sağlar.
10. Java, iki farklı hassasiyette ondalıklı sayıları temsil eden iki *ondalıklı sayı türü* (`float` ve `double`) sağlar.
11. Java, sayısal işlemler gerçekleştiren *opERATOR*lar sağlar: `+` (toplama), `-` (çıkarma), `*` (çarpma), `/` (bölme) ve `%` (kalan).
12. Tam sayı aritmetiği (`/`) tam sayı sonucu verir.
13. Java ifadesindeki sayısal operatörler, aritmetik ifadelerdeki gibi uygulanır.
14. Java, artırılmış atama operatörlerini `+=` (toplama ataması), `-=` (çıkarma ataması), `*=` (çarpma ataması), `/=` (bölme ataması) ve `%=` (kalan ataması) sağlar.
15. Artırma operatörü (`++`) ve azaltma operatörü (`--`) bir değişkeni 1 kadar artırır veya azaltır.
16. Karışık türlerdeki değerlerle bir ifadeyi değerlendirirken, Java otomatik olarak operandları uygun türlere dönüştürür.
17. Bir değeri bir türden diğerine açıkça dönüştürmek için *(type)value* notasyonunu kullanabilirsiniz.

74 Bölüm 2 Temel Programlama

18. Bir aralığı küçük olan bir tipin değişkenini, aralığı daha büyük olan bir tipin değişkenine dönüştürmeye *tipin genişletilmesi* denir.
19. Bir aralığı büyük olan bir tipin değişkenini, aralığı daha küçük olan bir tipin değişkenine dönüştürmeye *tipin daraltılması* denir.
20. Tipin genişletilmesi, açık bir dönüşüm olmadan otomatik olarak gerçekleştirilebilir. Tipin daraltılması açıkça yapılmalıdır.
21. Karakter türü **char** tek bir karakteri temsil eder.
22. Bir *kaçış karakteri* özel bir karakteri temsil etmek için bir notasyondur. Bir kaçış karakteri, bir ters eğik çizgi () ve ardından bir karakter veya bir karakter sırası ile oluşur.
23. Karakterler ' ', , , \r ve boşluk karakterleri olarak bilinir.
24. Bilgisayar bilimlerinde, 1 Ocak 1970'in gece yarısı UNIX epoch olarak bilinir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinden yapın.

MyProgrammingLab™

örnek çalışmaları

örneklerden öğren

belge analizi ve tasarımı

PROGRAMLAMA EGZERSİZLERİ



Not

Tüm egzersizleri **exercise9e.zip** indirerek çalıştırabilirsiniz www.cs.armstrong.edu/liang/intro9e/exercise9e.zip adresinden ve `java -cp exercise9e.zip Exercise_i_j` k omutunu kullanarak Exercise_i_j'yi çalıştırabilirsiniz. Örneğin, 2.1. Egzersizi çalıştırmak için,

```
java -cp egzersiz9e.zip Egzersiz02_01
```

Bu, programın nasıl çalıştığına dair bir fikir verecektir.



Hata Ayıklama İPUCU

Derleyici genellikle bir sözdizimi hatası için bir neden verir. Eğer bunu nasıl düzelteceğinizi bilmiyorsanız, programınızı metindeki benzer örneklerle karakter karakter karşılaştırmak.



Pedagojik Not

Eğitmenler, seçilen alıştırmalar için analiz ve tasarınızı belgelemenizi isteyebilir. Problemi analiz etmek için kendi kelimelerinizi kullanın; bu, girdi, çıktı ve neyin hesaplanması gerektiğini içermelidir ve problemi nasıl çözeceğinizi pseudocode ile tanımlayın.

Bölümler 2.2-2.12

- 2.1 (Celsius'i Fahrenheit'a dönüştür) Bir program yazın, bu program bir Celsius derecesini okur **konsoldan double** değer alır, ardından bunu Fahrenheit'a dönüştürür ve sonucu görüntüler. Dönüşüm formülü aşağıdaki gibidir:

$$\text{Fahrenheit} = (9/5) * \text{Santigrat} + 32$$

İpucu: Java'da, 9 / 51'dir, ancak 9.0 / 51.8'dir.

İşte bir örnek çalışma:

Santigrat cinsinden bir derece girin: 43

43 Santigrat 109.4 Fahrenheit'tir



2.2 (*Bir silindirin hacmini hesapla*) Bir program yazın, silindirin yarıçapını ve uzunluğunu okuyun ve aşağıdaki formülleri kullanarak alanı ve hacmi hesaplayın:

alan = yarıçap * yarıçap * π
hacim = alan * uzunluk

İşte bir örnek çalışma:

Bir silindirin yarıçapını ve uzunluğunu girin: 5.5 12

Alan 95.0331'dir
Hacim 1140.4'tür



2.3 (*Ayakları metreye çevir*) Bir program yazın, ayak cinsinden bir sayı okuyun, bunu metreye çevirin ve sonucu gösterin. Bir ayak 0.305 metredir. İşte bir örnek çalışma:

Ayak için bir değer girin: 16.5

16.5 ayak 5.0325 metredir



2.4 (*Pound'ları kilogram'a çevir*) Pound'ları kilogram'a çeviren bir program yazın. Program, kullanıcının pound cinsinden bir sayı girmesini ister, bunu kilogram'a çevirir ve sonucu gösterir. Bir pound, 0.454 kilogramdır. İşte bir örnek çalışma:

Bir sayı girin (pound cinsinden): 55.5

55.5 pound, 25.197 kilogramdır



***2.5** (*Finans uygulaması: bahşiş hesapla*) Alt toplamı ve bahşiş oranını okuyan bir program yazın, ardından bahşişi ve toplamı hesaplayın. Örneğin, kullanıcı 10 alt toplamı ve 15% bahşiş oranını girerse, program \$1.5 bahşiş olarak ve \$11.5 toplam olarak gösterir. İşte bir örnek çalışma:

Alt toplamı ve bir bahşiş oranı girin: 10 15

Bahşiş \$1.5 ve toplam \$11.5'tir



****2.6** (*Bir tam sayının basamaklarını topla*) 0 ile arasında bir tam sayı okuyan bir program yazın 1000 ve tam sayıdaki tüm basamakları toplayın. Örneğin, bir tam sayı 932 ise, tüm basamaklarının toplamı 14'tür.

İpucu: Basamakları çıkarmak için % operatörünü kullanın ve çıkarılan basamağı kaldırmak için / operatörünü kullanın. Örneğin, 932 % 10 = 2 ve 932 / 10 = 93.

İşte bir örnek çalışma:

0 ile 1000 arasında bir sayı girin: 999

Rakamların toplamı 27'dir



76 Bölüm 2 Temel Programlama

- *2.7** (*Yıl sayısını bul*) Kullanıcıdan dakika girmesini isteyen bir program yazın (örneğin, 1 milyar) ve dakikalar için yıl ve gün sayısını gösterin. Basitlik açısından, bir yılın 365 gün olduğunu varsayın. İşte bir örnek çalışma:



Dakika sayısını girin: 1000000000 Enter
1000000000 dakika yaklaşık 1902 yıl ve 214 gündür

- *2.8** (*Mevcut zaman*) Liste 2.6, ShowCurrentTime.java, mevcut zamanı GMT'de gösteren bir programı verir. Programı, kullanıcının GMT'ye göre zaman dilimi farkını girmesini istemesi ve belirtilen zaman diliminde zamanı göstermesi için revize edin. İşte bir örnek çalışma:



GMT'ye olan saat dilimi farkını girin: -5 Enter
Mevcut zaman 4:50:34

- 2.9** (*Fizik: ivme*) Ortalama ivme, hız değişiminin

değişimi yapmak için geçen zamana bölünmesi olarak tanımlanır, aşağıdaki formülde gösterildiği gibi:

$$a = \frac{vt_1 - vt_0}{t}$$

Kullanıcıdan başlangıç hızını girmesini isteyen bir program yazın v_0 içinde metre/saniye, son hız v_1 metre/saniye cinsinden ve zaman aralığı t içinde saniye, ve ortalama ivmeyi gösterir. İşte bir örnek çalışma:



v_0 , v_1 ve t 'yi girin: 5.5 50.9 4.5 Enter
Ortalama ivme 10.0889'dur

- 2.10** (*Bilim: enerji hesaplama*) Suyu başlangıç sıcaklığından son sıcaklığa ısıtmak için gereken enerjiyi hesaplayan bir program yazın. Programınız, kullanıcının su miktarını kilogram cinsinden ve suyun başlangıç ve son sıcaklıklarını girmesini istemelidir. Enerjiyi hesaplamak için formül

$$Q = M * (\text{sonSıcaklık} - \text{başlangıçSıcaklığı}) * 4184$$

burada M suyun kilogram cinsinden ağırlığıdır, sıcaklıklar santigrat derecede olup, ve enerji Q joule cinsinden ölçülmektedir. İşte bir örnek çalışma:



Su miktarını kilogram cinsinden girin: 55.5 Enter
Başlangıç sıcaklığını girin: 3.5 Enter
Son sıcaklığı girin: 10.5 Enter
Gerekli enerji 1625484.0

- 2.11** (*Nüfus projeksiyonu*) Kullanıcıdan yıl sayısını girmesini isteyen ve yıl sayısından sonra nüfusu gösteren 1.11 numaralı alıştırmaı yeniden yazın. İşte programın bir örnek çalışması:

Yıl sayısını girin:
5 yıl sonra nüfus 325932970'dir

5

↵ Enter



- 2.12** (Fizik: kalkış pisti uzunluğunu bulma) Bir uçağın ivmesi a ve kalkış hızı v verildiğinde, bir uçağın kalkış yapabilmesi için gereken minimum pist uzunluğunu aşağıdaki formülü kullanarak hesaplayabilirsiniz:

$$\text{uzunluk} = \frac{v^2}{2a}$$

Kullanıcıdan v 'yi metre/saniye (m/s) cinsinden ve ivme a 'yı metre/saniye kare cinsinden girmesini isteyen ve minimum pisti gösteren bir program yazın. (m/st2t), ve minimum pist uzunluğunu gösterir burada bir örnek çalışma:

Hız ve ivmeyi girin:
Bu uçağın minimum pist uzunluğu 514.286'dır

60 3.5

↵ Enter



- **2.13** (Finansal uygulama: bileşik değer) Her ay \$100 tasarruf hesabına %5 yıllık faiz oranıyla yatırdığınızı varsayalım. Böylece, aylık faiz oranı $0.05/12=0.00417$. İlk ayın ardından, hesapta bulunan değer olur

$$100 * (1 + 0.00417) = 100.417$$

İkinci aydan sonra, hesapta bulunan değer

$$(100 + 100.417) * (1 + 0.00417) = 201.252$$

Üçüncü aydan sonra, hesapta bulunan değer

$$(100 + 201.252) * (1 + 0.00417) = 302.507$$

ve devam eder.

Kullanıcıdan aylık tasarruf miktarını girmesini isteyen ve altıncı aydan sonraki hesap değerini gösteren bir program yazın. (Egzersiz 4.30'da, kodu basitleştirmek ve herhangi bir ay için hesap değerini göstermek için bir döngü kullanacaksınız.)

Aylık tasarruf miktarını girin:
Altıncı aydan sonra, hesap değeri \$608.81'dir

100

↵ Enter



- *2.14** (Sağlık uygulaması: BMI hesaplama) Vücut Kitle İndeksi (BMI), ağırlık üzerindeki sağlık ölçüsüdür. Kilogram cinsinden ağırlığınızı alarak ve boyunuzun metre cinsinden karesi ile bölerek hesaplanabilir. Kullanıcıdan pound cinsinden bir ağırlık ve inç cinsinden bir boy girmesini isteyen ve BMI'yi gösteren bir program yazın. Bir pound'un **0.45359237** kilogram ve bir inç'in **0.0254** metre olduğunu unutmayın. İşte bir örnek çalışma:



VideoNotu
BMI Hesapla

78 Bölüm 2 Temel Programlama



Ağırlığı pound cinsinden girin: 95.5
Boyu inç cinsinden girin: 50
BMI 26.8573'tür

***2.15** (*Geometri: bir üçgenin alanı*) Kullanıcıdan üç nokta girmesini isteyen bir program yazın. Üçgenin (x1, y1), (x2, y2), (x3, y3) alanını gösterir. Bir üçgenin alanını hesaplamak için formül

$$s = (taraf1 + taraf2 + taraf3) / 2;$$

$$alan = \sqrt{s(s - kenar1)(s - kenar2)(s - kenar3)}$$

İşte bir örnek çalışma:



Bir üçgen için üç nokta girin: 1.5 -3.4 4.6 5 9.5 -3.4
Üçgenin alanı 33.6'dır

2.16 (*Geometri: bir altıgenin alanı*) Kullanıcıdan bir altıgenin kenarını girmesini isteyen ve alanını gösteren bir program yazın. Bir altıgenin alanını hesaplama formülü

$$Alan = \frac{3\sqrt{3}}{2} s^2$$

burada s bir kenarın uzunluğudur. İşte bir örnek çalışma:



Kenar uzunluğunu girin: 5.5'tir
Altıgenin alanı 78.5895

***2.17** (*Bilim: rüzgar-soğuk sıcaklığı*) Dışarıda ne kadar soğuk? Sadece sıcaklık cevap vermek için yeterli değildir. Rüzgar hızı, bağıl nem ve güneş ışığı gibi diğer faktörler dışarıdaki soğukluğu belirlemede önemli rol oynar. 2001 yılında, Ulusal Hava Durumu Servisi (NWS) sıcaklık ve rüzgar hızını kullanarak soğukluğu ölçmek için yeni rüzgar-soğuk sıcaklığını uygulamaya koydu. Formül şudur:

$$ttwct = 35.74 + t + 0.6215ttat - t35.75ttvt0.16t + t0.4275ttttvt0.16$$

dış sıcaklık Fahrenheit cinsinden ölçülür ve v hız saatte mil cinsinden ölçülür. $ttwc$ rüzgar soğutma sıcaklığıdır. Formül olamaz 2 mph'nin altındaki rüzgar hızları veya sıcaklıkların altındaki kullanılır — t58 F veya üzeri nt41 F. Bir program yazın, kullanıcıdan bir sıcaklık girmesini isteyin — t58 F ve t41 F ve rüzgar hızı 2 veya daha büyük olduğunda ve rüzgar soğutma sıcaklığını gösterir- Sıcaklığı hesaplamak için **Math.pow(a, b)** kullanın. İşte bir örnek çalıştırma: vt0.16t. İşte bir örnek çalışma:



Fahrenheit cinsinden sıcaklığı girin: 5.3
Saate mil cinsinden rüzgar hızını girin: 6
Rüzgar soğukluk indeksi -5.56707

2.18 (Bir tablo yazdır) Aşağıdaki tabloyu görüntüleyen bir program yazın:

a b pow(a, b)n1 2 1

2	3	8
3	4	81
4	5	1024
5	6	15625

2.19 (Geometri: iki noktanın mesafesi) Kullanıcıdan iki nokta girmesini isteyen bir program yazın (x1, y1) ve (x2, y2) ve aralarındaki mesafeyi görüntüleyin.

Mesafeyi hesaplamak için formül

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$
 Dikkat edin

Math.pow(a, 0.5) kullanarak hesaplayabilirsiniz. İşte bir örnek çalışma: $\sqrt{}$ İşte bir örnek çalışma: a.

X1 ve Y1 girin: 1.5 -3.4
 X2 ve Y2'yi girin: 4 5
 İki nokta arasındaki mesafe 8.764131445842194



Bölümler 2.13-2.16

***2.20** (Finansal uygulama: faiz hesaplama) Eğer bakiyeyi ve yıllık yüzde faiz oranını biliyorsanız, bir sonraki aylık ödeme için faizi aşağıdaki formülü kullanarak hesaplayabilirsiniz:

$$\text{faiz} = \text{bakiye} \times (\text{yillikFaizOranı} / 1200)$$

Bakiyeyi ve yıllık yüzde faiz oranını okuyup, bir sonraki ay için faizi gösteren bir program yazın. İşte bir örnek çalışma:

Bakiye ve faiz oranını girin (örneğin, %3 için 3):
 Faiz 2.91667'dir

1000 3.5



***2.21** (Finansal uygulama: gelecekteki yatırım değerini hesaplama) Yatırım miktarını, yıllık faiz oranını ve yıl sayısını okuyup, aşağıdaki formülü kullanarak gelecekteki yatırım değerini gösteren bir program yazın:

$$\text{gelecektekiYatırımDeğeri} = \text{yatırımMiktarı} \times (1 + \text{aylıkFaizOranı})^{\text{numarasıYıllar} \times 12}$$

Örneğin, eğer miktar 1000, yıllık faiz oranı 3.25% ve yıl sayısını 1 girerseniz, gelecekteki yatırım değeri 1032.98 olacaktır.

İşte bir örnek çalışma:

Yatırım miktarını girin: 1000
 Yıllık faiz oranını yüzde olarak girin: 4.25
 Yıl sayısını girin: 1
 Biriken değer \$1043.34



Bölümler 2.17-2.18

- 2.22** (*Rastgele karakter*) `System.currentTimeMillis()` metodunu kullanarak rastgele bir büyük harf gösteren bir program yazın.
- 2.23** (*ASCII kodunun karakterini bul*) Bir ASCII kodu (0 ile 127 arasında bir tamsayı) alın ve karakterini gösteren bir program yazın. Örneğin, eğer kullanıcı **97** girerse, program **a** karakterini gösterir. İşte bir örnek çalıştırma:



Bir ASCII kodu girin: **69**
Karakter E'dir

- *2.24** (*Finansal uygulama: para birimleri*) Olası doğruluk kaybını düzeltmek için Listing 2.10, `ComputeChange.java` dosyasını yeniden yazın, bir `double` değerini `int` değerine dönüştürürken. Girişi, son iki basamağı sentleri temsil eden bir tamsayı olarak girin. Örneğin, girdi **1156, 11** dolar ve **56** senti temsil eder.

- *2.25** (*Finansal uygulama: maaş bordrosu*) Aşağıdaki bilgileri okuyan ve bir maaş bordan basan bir program yazın:

Çalışanın adı (örneğin, Smith)
Bir haftada çalışılan saat sayısı (örneğin, 10)
Saatlik ücret (örneğin, 6.75)
Federal vergi kesinti oranı (örneğin, %20)
Eyalet vergi kesinti oranı (örneğin, %9)



Çalışanın adını girin: **Smith**
Bir haftada çalışılan saat sayısını girin: **10**
Saatlik ücret oranını girin: **6.75**
Federal vergi kesinti oranını girin: **0.20**
Eyalet vergi kesinti oranını girin: **0.09**

Çalışan Adı: Smith
Çalışılan Saat: 10.0
Ücret Oranı: \$6.75
Brüt Ücret: \$67.5
Kesintiler:
Federal Kesinti (20.0%): \$13.5
Eyalet Kesintisi (9.0%): \$6.07
Toplam Kesinti: \$19.57
Net Ücret: \$47.92

Bölüm 2.19

- *2.26** (*Girdi iletişim kutusunu kullan*) Listing 2.10, `ComputeChange.java`'yı girdi ve çıktı iletişim kutuları kullanarak yeniden yazın.
- *2.27** (*Finansal uygulama: bordro*) Egzersiz 2.25'i GUI girdi ve çıktı iletişim kutuları kullanarak yeniden yazın.

BÖLÜM

3

SEÇİMLER

Hedefler

- **boolean** değişkenleri bildirmek ve karşılaştırma operatörlerini kullanarak Boolean ifadeleri yazmak (§3.2).
- Tek yönlü **if** ifadeleri kullanarak seçim kontrolü uygulamak (§3.3).
- Tek yönlü **if** ifadeleri kullanarak program yazmak (**GuessBirthday**) (§3.4).
- İki yönlü **if-else** ifadelerini kullanarak seçim kontrolü uygulamak (§3.5).
- İç içe **if** ve çok yönlü **if** ifadelerini kullanarak seçim kontrolü uygulamak (§3.6).
- **if** ifadelerinde yaygın hatalardan kaçınmak (§3.7).
- **Math.random()** metodunu kullanarak rastgele sayılar üretmek (§3.8).
- Çeşitli örnekler için seçim ifadeleri kullanarak programlama yapmak (**SubtractionQuiz**, **BMI**, **ComputeTax**) (§§3.8–3.10).
- Mantıksal operatörleri kullanarak koşulları birleştirmek (**&&**, **||** ve **!**) (§3.11).
- Birleşik koşullarla seçim ifadeleri kullanarak programlama yapmak (**LeapYear**, **Lottery**) (§§3.12–3.13).
- **switch** ifadelerini kullanarak seçim kontrolü uygulamak (§3.14).
- Koşullu operatörü kullanarak ifadeler yazmak (§3.15).
- Çıktıyı **System.out.printf** metodu ile biçimlendirmek (§3.16).
- Operatör önceliği ve bağlayıcılığı yöneten kuralları incelemek (§3.17).
- Onay pencereleri kullanarak kullanıcı onayı almak (§3.18).
- Hataları ayıklamak için yaygın teknikleri uygulamak (§3.19).



3.1 Giriş



sorun

Program, bir koşula dayanarak hangi ifadelerin yürütüleceğine karar verebilir.

Eğer Listing 2.2, ComputeAreaWithConsoleInput.java'da **radius** için negatif bir değer girerseniz, program geçersiz bir sonuç gösterir. Eğer yarıçap negatifse, programın alanı hesaplamasını istemezsiniz. Bu durumla nasıl başa çıkabilirsiniz?

seçim ifadeleri

Tüm yüksek seviyeli programlama dilleri gibi, Java *seçim ifadeleri* sağlar: ifadeler size iki veya daha fazla alternatif eylem seçme imkanı tanır. Aşağıdaki

seçim ifadesini Listing 2.2'deki 12-17. satırları değiştirmek için kullanabilirsiniz:

```
ift(yarıçap < t0t) {
    System.out.println("Geçersiz girdi");
}
başka{
    area = radius * radius * 3.14159;
    System.out.println("Alan "+ area);
}
```

Boolean ifadesi

Boolean değeri

Seçim ifadeleri, Boolean ifadeleri olan koşulları kullanır. Bir *Boolean ifadesi*, *Boolean değeri*: **true** veya **false** olan bir ifadedir. Şimdi Boolean

tiplerini ve karşılaştırma operatörlerini tanıtıyoruz.

3.2 boolean Veri Türü



boolean veri türü

karşılaştırma operatörleri

Aboolean veri türü, bir değişkeni true veya false değeri ile tanımlar.

İki değeri, örneğin bir yarıçapın 0'dan büyük, 0'a eşit veya 0'dan küçük olup olmadığını nasıl karşılaştırırsınız? Java, i ki değeri karşılaştırmak için kullanılacak altı *karşılaştırma operatörü* (aynı zamanda *ilişkisel operatörler* olarak da bilinir) sağlar; bunlar Tablo 3.1'de gösterilmiştir (tablodaki yarıçapın 5 olduğunu varsayın).

TABLO3.1 Karşılaştırma Operatörleri

Java Operatörü	Matematik Sembolü	İsim	Örnek (yarıçap 5'tir) Sonuç
<	<	küçüktür	yarıçap < 0 yanlış
<=	<=	küçük veya eşittir	yarıçap <= 0 yanlış
>	>	büyüktür	yarıçap > 0 doğru
>=	>=	büyüktür veya eşittir	yarıçap >= 0 doğru
==	=	eşittir	yarıçap == 0 yanlış
!=	≠	eşit değildir	yarıçap != 0 doğru

karakterleri karşılaştı



Not

Karakterleri de karşılaştırabilirsiniz. Karakterleri karşılaştırmak, onların Unicode'larını karşılaştırmakla aynıdır. Örneğin, **a**, **A**'dan daha büyüktür çünkü **a**'nın Unicode'u **A**'nın Unicode'undan daha büyüktür. Karakterlerin sırasını bulmak için Ek B'ye, ASCII Karakter Seti'ne bakın.

== VS. =



Dikkat

Eşitlik karşılaştırma operatörü iki eşit işareti (==) olup, tek bir eşit işareti (=) değildir. Sonuncu sembol atama içindir.

Karşılaştırmamanın sonucu bir Boolean değerdir: **doğru** veya **yanlış**. Örneğin, aşağıdaki ifade **doğru**'yu gösterir:

```
çift yarıçap = t1t;Syste
m.out.println(yarıçap > t0t);
```

Boolean değeri tutan bir değişkene *Boolean değişkeni* denir. **boolean** veri türü, Boolean n değişkenlerini tanımlamak için kullanılır. **boolean** değişkeni iki değerden birini tutabilir: **doğru** veya **yanlış**. Örneğin, aşağıdaki ifade **lightsOn** değişkenine **doğru** atar:

boolean ışıklarAçık = **true**;

true ve **false**, **10** gibi bir sayı gibi, literallerdir. Bunlar rezerve edilmiş kelimelerdir ve programınızda tanımlayıcı olarak kullanılamazlar.

Diyelim ki bir birinci sınıf öğrencisinin toplama işlemi yapmasını sağlayacak bir program geliştirmek istiyorsunuz. Program rastgele iki tek haneli tam sayı, **number1** ve **number2**, oluşturur ve öğrenciye

“1 + 7 ne kadardır?” gibi bir soru gösterir, örnek çalışmada gösterildiği gibi. Öğrenci cevabı yazdıktan sonra, program doğru mu yanlış mı olduğunu belirtmek için bir mesaj gösterir.

Rastgele sayılar üretmenin birkaç yolu vardır. Şimdilik, ilktam sayıyı **System.currentTimeMillis()** % 10 **System.currentTimeMillis()** % 10 **System.currentTimeMillis()** % 10 **System.currentTimeMillis()** / 7 % 10 kullanarak ve ikincisini **System.currentTimeMillis()** / 7 % 10 kullanarak oluşturun. Liste 3.1 programı verir. Satırlar 5–6d iki

sayı, **number1** ve **number2** oluşturur. Satır 14, kullanıcının bir cevap alır. Cevap, satır 18’de bir Boolean ifadesi **number1 + number2 == cevap** kullanılarak değerlendirilir.

LISTE 3.1 ToplamaSınavı.java

```
1  import java.util.Scanner;
23
4  public class ToplamaSınavı {
5678      public static void main(String[] args) {
          1 = (int)(System.currentTimeMillis() % 10);
          int number2 = (int)(System.currentTimeMillis() / 7 % 10);
          // Bir Scanner oluştur
          9 Tarayıcı girişi = new Scanner(System.in);
          10
          11 Sistem.çıkışı.baskı(
          12     "Ne kadar " + number1 + " + " + number2 + "? ";
          13
          14     int answer = input.nextInt();
          15
          16     System.out.println(
          17     sayı1 + " + " + sayı2 + " = " + cevap + " eşittir " + "t+n19 }
          18     (sayı1 + sayı2 == cevap));
          20 }
```

Boolean değişken

Boolean sabitleri



VideoNotu

Program toplama sınavı

number1 üret
generate number2

soru göster

sonucu göster



1 + 7 nedir? Enter

1 + 7 = 8 doğrudur

4 + 8 nedir? Enter

4 + 8 = 9 yanlıştır

satır#	numara1	numara2	cevap	çıktı
5	4			
6		8		
14			9	
16				4 + 8 = 9 yanlıştır

- 3.1 Altı karşılaştırma operatörünü listeleyin.
3.2 Aşağıdaki ifadelerin çıktısını gösterin:

```
System.out.println(t'a't<t'b't');nSystem.out.println(t'a't<=t'A't');nSystem.out.println(t'a't>t'b't');nSystem.out.println(t'a't>=t'A't');nSystem.out.println(t'a't==t'a't');nSystem.out.println(t'a't!=t'b't');
```

- 3.3 Aşağıdaki casting içeren dönüşümlere izin verilebilir mi? Eğer öyleyse, dönüştürülmüş sonucu bulun.

```
booleanb = ttrue;if = (tintt)b;

int = t1t;
boolean = (boolean)in;
```

3.3 if İfadeleri



Biri if ifadesi, koşul doğruysa ifadeleri yürütür.

Önceki program, “ $6 + 2 = 7$ yanlıştır.” İsterseniz mesajın “ $6 + 2 = 7$ yanlış,” olması için bir seçim ifadesi kullanmalısınız.

bu küçük değişikliği yapmak için.

Java'nın birkaç tür seçim ifadesi vardır: tek yönlü **if** ifadeleri, iki yönlü **if-else** ifadeleri, iç içe **if** ifadeleri, **switch** ifadeleri ve koşullu ifadeler.

Tek yönlü **if** ifadesi, koşul **doğru** olduğu sürece bir eylemi gerçekleştirir. Tek yönlü **if** ifadesinin sözdizimi:

```
ift(boolean-ifadesi) {n ifade(ler)
;
}
```

Şekil 3.1'deki akış diyagramı, Java'nın bir **if** ifadesinin sözdizimini nasıl yürüttüğünü göstermektedir. Bir akış diyagramı, bir algoritmayı veya süreci tanımlayan, adımları çeşitli türlerde kutular olarak gösteren ve bunları oklarla bağlayarak sıralarını gösteren bir diyagramdır. Süreç işlemleri bu kutularda temsil edilir ve bunları bağlayan oklar kontrol akışını temsil eder. Bir elmas kutusu, bir Boolean koşulunu belirtmek için kullanılır ve bir dikdörtgen kutu ise ifadeleri temsil etmek için kullanılır.

Eğer **boolean-expression true** olarak değerlendirilirse, bloktaki ifadeler yürütülür. Bir örnek olarak, aşağıdaki kodu inceleyin:

```
ift(yarıçap >= t0t) {
    alan = radius * radius * PI;
    System.out.println("Yarıçapı "+
        radius + " olan dairenin alanı "+ alan);
}
```

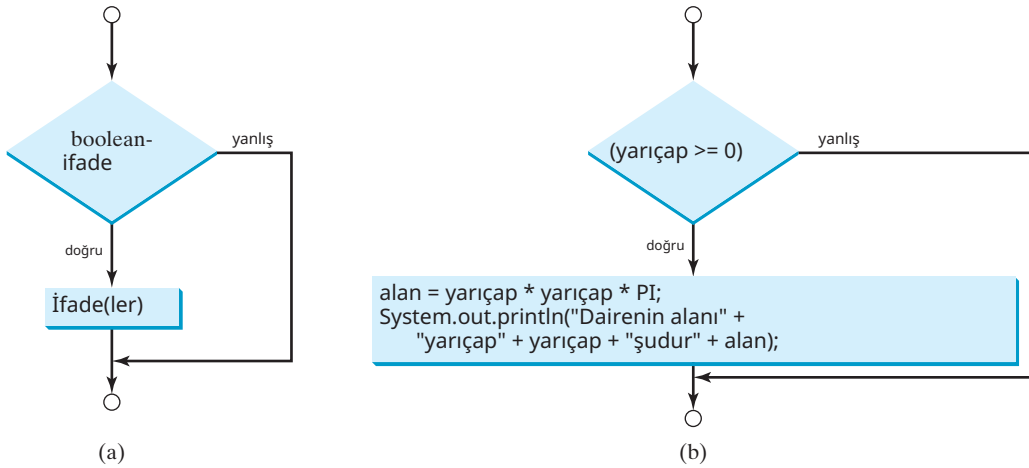
Önceki ifadenin akış diyagramı Şekil 3.1b'de gösterilmiştir. Eğer **radius** değeri **0** veya daha büyükse, **alan** hesaplanır ve sonuç görüntülenir; aksi takdirde, bloktaki iki ifade yürütülmeyecektir.

boolean-expression parantez içinde yer alır. Örneğin, aşağıdaki (a) kodu yanlıştır. (b) kısmında gösterildiği gibi düzeltilmelidir.

nedeni f ifadesi?

if ifadesi

akış diyagramı



Fiç3.1 Bir **if** ifadesi, boolean ifadesi doğru olduğunda ifadeleri çalıştırır.

```
eğer i > 0 {
    System.out.println("i pozitif");
}
```

(a) Yanlış

```
eğer ( i > t0 ) {
    System.out.println("i pozitif");
}
```

(b) Doğru

Blok süslü parantezleri, tek bir ifadeyi kapsıyorsa atlanabilir. Örneğin, aşağıdaki ifadeler eşdeğerdir.

```
eğer(i > 0) {
    System.out.println("i pozitif");
}
```

(a)

Eşdeğer

```
eğer(i > 0)
    System.out.println("i pozitif");
```

(b)



Not

Parantezleri atlamak kodu kısaltır, ancak hatalara açıktır. Parantezleri atladığınızda kodu değiştirmek için geri döndüğünüzde parantezleri unutmak yaygın bir hatadır.

Süslü parantezleri atlamak veya atlamamak

Liste 3.2, kullanıcıdan bir tam sayı girmesini isteyen bir programı vermektedir. Eğer sayı 5'in katıysa, program **MerhabaBeş**'i gösterir. Eğer sayı 2'ye tam bölünüyorsa, **MerhabaÇift**'i gösterir.

LİSTE3.2 SimpleIfDemo.java

```
1  import java.util.Scanner;
2
3  genel classt SimpleIfDemo {
4      public static voidtmain(String[] args) {
5          Scanner input = newScanner(System.in);
6          System.out.println("Bir tam sayı girin: ");
7          inttnumber = input.nextInt();
8
9          ıft(sayı % t5t==t0t)
10         System.out.println(t"HiFive"t);
11
12         ıft(sayı % t2t==t0t)
13         System.out.println(t"HiEven"t);
14     }n15 }
```

giriş yapın

5'i kontrol et

çift kontrol et

86 Bölüm 3 Seçimler



Bir tamsayı girin:

4

↵ Enter

MerhabaEven



Bir tamsayı girin:

30

↵ Enter

MerhabaBeş

MerhabaÇift

Program, kullanıcıdan bir tamsayı girmesini ister (satırlar 6-7) ve **MerhabaBeş** eğer 5 ile tam bölünebiliyor sa (satırlar 9-10) ve **MerhabaÇift** eğer 2 ile tam bölünebiliyorsa (satırlar 12-13) gösterir.

3.4 **y0**'dan büyükse x'e **1** atayan bir **if** ifadesi yazın.

3.5 skor**90**'dan büyükse ücreti %3 artıran bir **if** ifadesi yazın.

Nokta Kontrol

MyProgrammingLab™

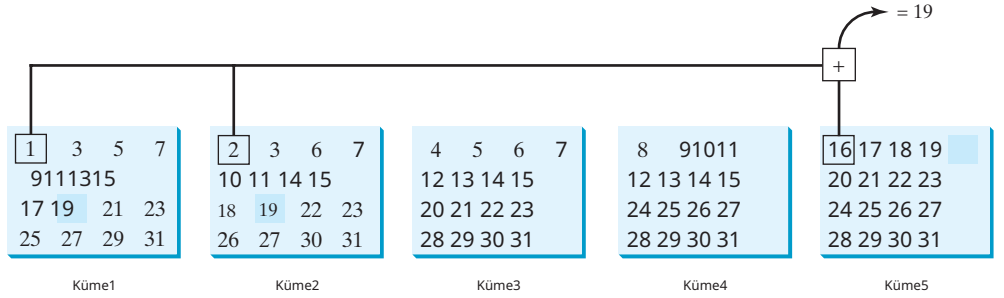
3.4 Vaka Çalışması: Doğum Günlerini Tahmin Etme



Anahtar
Nokta

Doğum günlerini tahmin etmek, basit bir programlama çözümü ile ilginç bir problemdir.

Arkadaşınızın doğduğu ayın tarihini öğrenmek için beş soru sorabilirsiniz. Her soru, günün beş sayı kümesinden birinde olup olmadığını sorar.



Doğum günü, günün görüldüğü kümelerdeki ilk sayıların toplamıdır. Örneğin, doğum günü **19** ise, Küme1, Küme2 ve Küme5'te görünür. Bu üç kümedeki ilk sayılar **1**, **2** ve **16**'dır. Topamları **19**'dur.

Liste 3.3, kullanıcının günün Set1'de olup olmadığını yanıtlaması için bir program verir (41-44. satırlar), Set2'de (50-53. satırlar), Set3'te (59-62. satırlar), Set4'te (68-71. satırlar) ve Set5'te (77-80. satırlar). Eğer sayı setteyse, program setteki ilk sayıyı **gün** (47, 56, 65, 74, 83. satırlar) ekler.

LISTE3.3 GuessBirthday.java

```
1 import java.util.Scanner;  
2  
3 genel sınıfTahminDogumGunu {  
4     public static void main(String[] args) {  
5         Dizi set1 =  
6             " 1 3 5 7n"t+  
7  
8     }  
9 }
```

```

7      " 9 11 13 15n"t+
8      "17 19 21 23n"t+
9      "25 27 29 31"t;
10
11 Dizi seti2 =
12      " 2 3 6 7n"t+
13      "10 11 14 15n"t+
14      "18 19 22 23n"t+
15      "26 27 30 31"t;
16
17 Dizi seti3 =
18      " 4 5 6 7n"t+
19      "12 13 14 15n"t+
20      "20 21 22 23n"t+
21      "28 29 30 31"t;
22
23 Dizi seti4 =
24      " 8 9 10 11n"t+
25      "12 13 14 15n"t+
26      "24 25 26 27n"t+
27      "28 29 30 31"t;
28
29 Dizi seti5 =
30      "16 17 18 19n"t+
31      "20 21 22 23n"t+
32      "24 25 26 27n"t+
33      "28 29 30 31"t;
34
35      intgün = 0;
36
37      // Bir Scanner oluştur
38      Tarayıcı girişi = tnewScanner(System.in);
39
40      // Kullanıcıdan soruları yanıtlamasını isteyin41
41      System.out.print("Doğum gününüz Set1'de mi?");42      System.out.print(
42      t(set1);
43      System.out.print("Hayır için 0, Evet için 1 girin: ");
44      inttanswer = input.nextInt();
45
46      eğer(cevap == 1)
47      gün += t1t;
48
49      // Kullanıcıdan soruları yanıtlamasını isteyin50
50      System.out.print("Doğum gününüz Set2'de mi?");51      System.out.print(s
51      et2);
52      System.out.print("Hayır için 0, Evet için 1 girin: ");53      answer
53      = input.nextInt();
54
55      eğer(cevap == 1)
56      gün += 2;
57
58      // Kullanıcıdan soruları yanıtlamasını isteyin59
59      System.out.print("Doğum gününüz Set3'te mi?");60      System.out.print
60      (set3);
61      System.out.print("Hayır için 0 ve Evet için 1 girin: ");62      cevap
62      = input.nextInt();
63
64      eğer(cevap == 1)
65      gün += 4;
66

```

gün belirlenecek

Set1'de mi?

Set2'de mi?

Set3'te mi?

88 Bölüm 3 Seçimler

Set4'te mi?

Set5'te mi?

```
67 // Kullanıcıyı soruları yanıtlamaya teşvik et
68 System.out.print("Doğum gününüz Set4'te mi?");69   Syste
m.out.print(set4);
70 System.out.print("Hayır için 0 ve Evet için 1 girin: ");71   answe
r = input.nextInt();
72
73 eğer(cevap == 1)
74 gün += t8t;
75
76 // Kullanıcıdan soruları yanıtlamasını isteyin77
System.out.print("Doğum gününüz Set5'te mi?");78   System.out.print(se
t5);
79 System.out.print("Hayır için 0 ve Evet için 1 girin: ");80   answe
r = input.nextInt();
81
82 eğer(cevap == 1)
83 gün += t16t;
84
85 System.out.println("Doğum gününüz "+ day + "!");86 }
87 }
```



Doğum gününüz Set1'de mi?
1 3 5 7
9 11 13 15
17 19 21 23
25 27 29 31
Hayır için 0, Evet için 1 girin: 1 Enter

Doğum gününüz Set2'de mi?
2 3 6 7
10 11 14 15
18 19 22 23
26 27 30 31
Hayır için 0, Evet için 1 girin: 1 Enter

Doğum gününüz Set3'de mi?
4 5 6 7
12 13 14 15
20 21 22 23
28 29 30 31
Hayır için 0, Evet için 1 girin: 0 Enter

Doğum gününüz Set4'de mi?
8 9 10 11
12 13 14 15
24 25 26 27
28 29 30 31
Hayır için 0, Evet için 1 girin: 0 Enter

Doğum gününüz Set5'de mi?
16 17 18 19
20 21 22 23
24 25 26 27
28 29 30 31
Hayır için 0, Evet için 1 girin: 1 Enter
Doğum gününüz 19!

İki yönlü **if-else** ifadesinin sözdizimi:

```
if(boolean-ifadesi) {
    true-durumu-için-ifade(ler);
}
başka{
    false-durumu-için-ifade(ler);
}
```

İfadenin akış diyagramı Şekil 3.3'te gösterilmiştir.

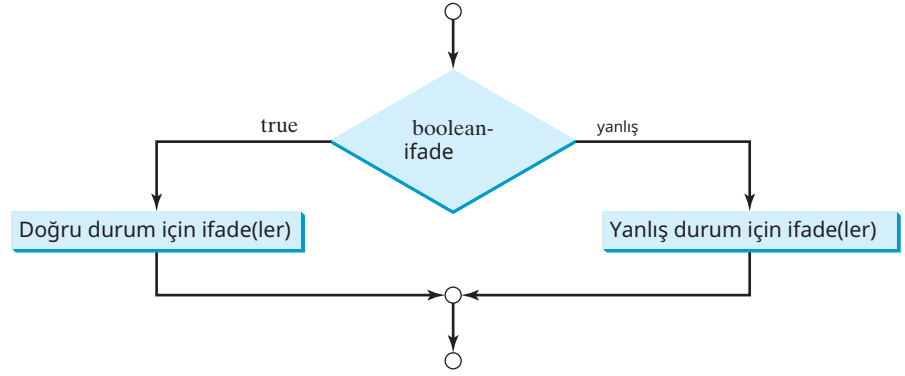


FIGURE 3.3 Bir if-else ifadesi, Boolean-

ifade true olarak değerlendirildiğinde doğru durum için ifadeleri çalıştırır; aksi takdirde, yanlış durum için ifadeler çalıştırılır.

Eğer **boolean-ifadesi true** olarak değerlendirilirse, true durumu için ifade(ler) yürütülür; aksi takdirde, false durumu için ifade(ler) yürütülür. Örneğin, aşağıdaki kodu düşünün:

iki yönlü if-else ifadesi

```
ift(yarıçap >= t0t) {
    alan = radius * radius * PI;
    System.out.println("Yarıçapı "+
        radius + " olan dairenin alanı "+ alan);
}
başka{
    System.out.println("Negatif girdi");
}
```

Eğer **radius >= 0 true** ise, **alan** hesaplanır ve görüntülenir; eğer **false** ise, "Negatif girdi" mesajı görüntülenir.

Her zamanki gibi, içinde yalnızca bir ifade varsa süslü parantezler atlanabilir. Bu nedenle, `System.out.println("Negatif girdi")` ifadesini kapsayan süslü parantezle önceki örnekte atlanabilir.

İşte **if-else** ifadesinin kullanımına dair bir başka örnek. Örnek, bir sayının çift mi yoksa tek mi olduğunu kontrol eder:

```
ift(sayı % 2 == 0) {
    System.out.println(number + " çift.");
}
else
    System.out.println(number + " tek.");
```

3.6 İç içe if ve Çok Yönlü if-else İfadeleri

3.6 Skor 90'dan büyükse maaşı %3 artıran bir **if** ifadesi yazın, aksi takdirde maaşı %1 artırın.

3.7 (a) ve (b) kodunun çıktısı number 30 ise nedir? number 35 ise ne olur?

Nokta Kontrol

MyProgrammingLab™

```
ift(sayı % 2==0){
    System.out.println(number + " çift.");
}
System.out.println(number + " tek.");
```

(a)

```
ift(sayı % 2==0){
    System.out.println(number + " çift.");
}
else
    System.out.println(number + " tek.");
```

(b)

3.6 İç içe if ve Çok Yönlü if-else İfadeleri

Bir **if** ifadesi, iç içe bir **if** ifadesi oluşturmak için başka bir **if** ifadesinin içinde olabilir.

Bir **if** veya **if-else** ifadesindeki ifade, başka bir **if** veya **if-else** ifadesi de dahil olmak üzere herhangi bir geçerli Java ifadesi olabilir. İçteki **if** ifadesinin dıştaki **if** ifadesinin içinde iç içe olduğu söylenir. İçteki **if** ifadesi başka bir **if** ifadesi içerebilir; aslında, iç içe geçmenin derinliği için bir sınır yoktur. Örneğin, aşağıdaki iç içe geçmiş bir **if** ifadesidir:

```
eğer(i > k) {
    eğer(j > k)
        System.out.println("i ve j k'dan büyüktür");
}
aksi takdirde
    System.out.println("i k'dan küçük veya eşittir");
```

eğer (j > k) ifadesi, if (i > k) ifadesinin içine yerleştirilmiştir.

İç içe geçmiş if ifadesi, birden fazla alternatif uygulamak için kullanılabilir. Örneğin, Şekil 3.4a'da verilen ifade, puana göre grade değişkenine bir harf notu atar ve birden fazla alternatif içerir.



iç içe if ifadesi

```
eğer(puan >= 90.0)
    grade = 'A';
aksi takdirde
    eğer(puan >= 80.0)
        grade = 'B';
aksi takdirde
    eğer(score >= 70.0)
        grade = 'C';
aksi takdirde
    eğer(score >= 60.0)
        grade = 'D';
aksi takdirde
    not = 'F';
```

(a)

Eşdeğer

Bu daha iyi

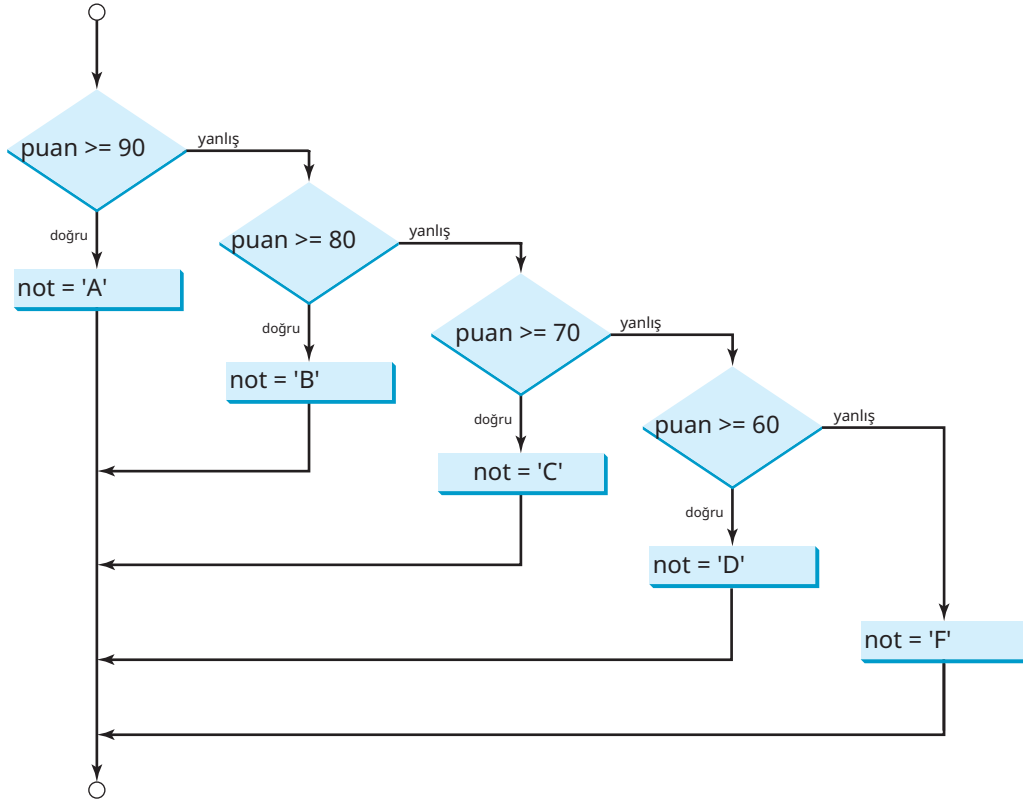
```
eğer(score >= 90.0)
    grade = 'A';
aksi takdirde eğer(score >= 80.0)
    grade = 'B';
aksi takdirde eğer(score >= 70.0)
    grade = 'C';
aksi takdirde eğer(score >= 60.0)
    grade = 'D';
aksi takdirde
    not = 'F';
```

(b)

ŞEKİL 3.4 Birden fazla alternatif için tercih edilen bir format (b) bölümünde çok yönlü bir

if-else ifadesi kullanılarak gösterilmiştir.

Bu **if** ifadesinin yürütülmesi Şekil 3.5'te gösterildiği gibi ilerler. İlk koşul (skor >= 90.0) test edilir. Eğer **doğru** ise, not **A** olur. Eğer **yanlış** ise, ikinci koşul (skor >= 80.0) test edilir. Eğer ikinci koşul **doğru** ise, not **B** olur. Eğer o koşul **yanlış** ise, üçüncü koşul ve geri kalan koşullar (gerekirse) bir koşul sağlanana kadar veya tüm koşullar **yanlış** çıkana kadar test edilir. Eğer tüm koşullar **yanlış** ise, not **F** olur. Bir koşul, yalnızca öncesindeki tüm koşullar **yanlış** olduğunda test edilir.



ŞEKİL 3.5 Bir not atamak için çok yönlü **if-else** ifadesi kullanabilirsiniz.

çoklu yol eğer ifadesi

Şekil 3.4a'daki **if** ifadesi, Şekil 3.4b'deki **if** ifadesine eşdeğerdir. Aslında, Şekil 3.4b, birden fazla alternatif **if** ifadesi için tercih edilen kodlama stildir. Bu stil, *çok yönlü if-else ifadeleri*, derin girintileri önler ve programı okunması kolay hale getirir.

- 3.8** Diyelim ki $x = 3$ veya $y = 2$; aşağıdaki kodun çıktısını gösterin, varsa. Eğer $x = 3$ veya $y = 4$ ise, çıktı nedir? $x = 2$ veya $y = 2$ ise, çıktı nedir? Kodun bir akış diyagramını çizin.

```

ift(x > t2t) {
    ift(y > t2t) {
        z = x + y;
        System.out.println("z şudur "+ z);
    }
}

```

aksi takdirde

```
System.out.println("x şudur "+ x);
```

- 3.9** Diyelim ki $x = 2$ veya $y = 3$. Aşağıdaki kodun çıktısını gösterin, varsa. Eğer $x = 3$ veya $y = 2$ ise, çıktı nedir? Eğer $x = 3$ veya $y = 3$ ise, çıktı nedir? (*İpucu*: Öncelikle ifadeyi doğru bir şekilde girintileyin.)

```

eğer(x > 2)
    ift(y > t2t) {
        intz = x + y;
        System.out.println("z şudur "+ z);
    }

```

aksi takdirde

```
System.out.println("x şudur "+ x);
```

Nokta Kontrol

MyProgrammingLab™

3.10 Aşağıdaki kodda ne yanlış?

```
eğer(puan >=60.0)
    not = 'D';
aksi takdirde eğer(puan >= 70.0)
    not = 'C';
aksi takdirde eğer(puan >= 80.0)
    not = 'B';
aksi takdirde eğer(puan >= 90.0)
    not = 'A';
aksi takdirde
    not ='F';
```

3.7 Seçim İfadelerinde Yaygın Hatalar

Gerekli süslü parantezleri unutmamak, bir **if** ifadesini yanlış bir yerde bitirmek, **==** yerine **=** kullanmak ve sarkan **else** kısımları, seçim ifadelerinde yaygın hatalardır.



Aşağıdaki hatalar yeni programcılar arasında yaygındır.

Common Hataor 1: Forgetting Necessary BracesKüme par

antezleri, blok tek bir ifade içeriyorsa atlanabilir. Ancak, birden fazla ifadeyi gruplamak için gerekli olduğunda küme parantezlerini unutmamak yaygın bir programlama hatasıdır. Eğer bir **if** ifadesine küme parantezleri olmadan yeni ifadeler ekleyerek kodu değiştirirseniz, küme parantezlerini eklemek zorunda kalacaksınız. Örneğin, (a) kısmındaki aşağıdaki kod yanlıştır. Birden fazla ifadeyi gruplamak için küme parantezleri ile yazılmalıdır, (b) kısmında gösterildiği gibi.

```
if(radius >= 0)
    area = radius * radius * PI;
    System.out.println("Alan "
        + "dır "+ area);
```

(a) Yanlış

```
if(radius >= 0) {
    area = radius * radius * PI;
    System.out.println("Alan "
        + "dır "+ area);
}
```

(b) Doğru

Common Hataor 2: Yanlış Semicolon at the if Line

Bir **if** satırının sonuna noktalı virgül eklemek, aşağıdaki (a) kısmında gösterildiği gibi, yaygın bir hatadır.

Mantık hatası

```
ift(yarıçap >= t0t)n{ ;
    area = radius * radius * PI;
    System.out.println("Alan "
        + "dır "+ area);
}
```

(a)

Eşdeğer

Boş blok

```
ift(yarıçap >= t0t) { };
{
    area = radius * radius * PI;
    System.out.println("Alan "
        + "dır "+ area);
}
```

(b)

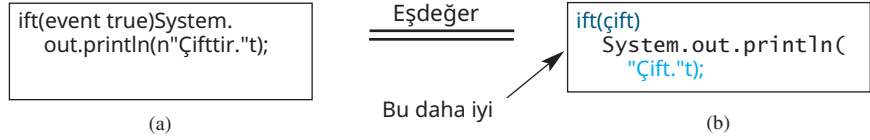
Bu hata bulması zor, çünkü ne bir derleme hatası ne de bir çalışma zamanı hatasıdır; bu bir mantık hatasıdır. (a) içindeki kod, boş bir blok ile (b) içindeki koda eşdeğerdir.

Bu hata genellikle bir sonraki satır blok stilini kullandığınızda ortaya çıkar. Satır sonu blok

stilini kullanmak bu hatayı önlemeye yardımcı olabilir.

Common Error 3: Redundant Testing of Boolean Değerler Bir bo

olean değişkeninin **true** veya **false** olup olmadığını test etmek için, eşitlik karşılaştırma operatörünü kullanmak gereksizdir, (a) kodundaki gibi:



Bunun yerine, boolean değişkenini doğrudan test etmek daha iyidir, (b)'de gösterildiği gibi. Bunu yapmanın bir diğer iyi nedeni, tespit edilmesi zor hatalardan kaçınmaktır. Eşitlik i test koşulunda iki öğeyi karşılaştırmak için **=** operatörünü kullanmak, yaygın bir hatadır. Aşağıdaki hatalı ifadeye yol açabilir:

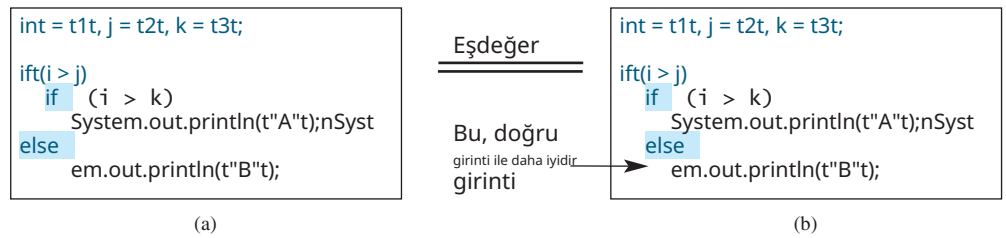
```
eğer(even=true)
    System.out.println(t'Çifttir.'t);
```

Bu ifade derleme hatası içermez. **true** değerini **even** değişkenine atar, böylece **even** her zaman **true** olur.

Common Error 4: Dangling else Ambiguity

Aşağıdaki (a) kodu iki **if** koşulu ve bir **else** koşulu içerir. Hangi **if** koşulu **else** k oşulu ile eşleşir? Girinti, **else** koşulunun ilk **if** koşuluyla eşleştiğini gösterir. Ancak, **else** durumu aslında ikinci **if** durumuyla eşleşir. Bu durum, the *dangling else ambiguity* olarak bilinir. **else** durumu her zaman aynı bloktaki en son eşleşmemiş **if** durumuyla eşleşir. Bu nedenle, (a) içindeki ifade (b) içindeki koda eşdeğerdir.

askıda kalan else belirsizliği



Çünkü **(i > j)** yanlıştır, (a) ve (b) içindeki ifadelerden hiçbir şey görüntülenmez. **else** durumunun ilk **if** durumu ile eşleşmesini sağlamak için bir çift süslü parantez eklemelisiniz:

```
intti = t1t, j = t2t, k = t3t; nift(i
> j) {
    if(i > k)
        System.out.println(t'A't);nels
}
e
    System.out.println(t'B't);
```

Bu ifade **B**'yi görüntüler.



TtitiP

Sıklıkla yeni programcılar, (a) kodundaki gibi bir test koşulunu **boolean** değişkenine atayan kod yazarlar:

boolean değişkeni atamak

```
ift(sayı % t2t==t0t)neven
    = ttrue;
aksi takdirde
    çift = tfalset;
```

(a)

Eşdeğer

Bu daha kısadır

```
booleantevenn
    = sayı % t2t==t0t;
```

(b)

Kod, test değerini doğrudan değişkene atayarak basitleştirilebilir, (b)'de gösterildiği gibi.

3.11 Aşağıdaki ifadelerden hangileri eşdeğerdir? Hangileri doğru şekilde girintilenmiştir?

Nokta Kontrol

MyProgrammingLab™

```
ift(i > t0t)tifn(j >
t0t)
x = 0;aksi
if(k > 0) y = 0;
aksiz = 0;
```

(a)

```
eğer(i > 0) {
    if(j > 0)
        x = 0;
    aksihalde (k > 0)
        y = 0;
}
aksi takdirde
    z = t0t;
```

(b)

```
eğer(i > 0)
    if(j > 0)
        x = 0;
    aksihalde eğer(k > 0)
        y = 0;
aksi takdirde
    z = t0t;
```

(c)

```
eğer(i > 0)
    if(j > 0)
        x = 0;
    aksihalde eğer(k > 0)
        y = 0;
aksi takdirde
    z = t0t;
```

(d)

3.12 Aşağıdaki ifadeyi bir Boolean ifadesi kullanarak yeniden yazın:

```
eğer(count % 10==0)
    yeniSatır = true;
aksihalde
    yeniSatır = false;
```

3.13 Aşağıdaki ifadeler doğru mu? Hangisi daha iyi?

```
eğer(yaş < 16)
    Sistem.out.println
        ("Ehliyet alınamaz");
eğer(yaş >= 16)
    Sistem.out.println
        ("Ehliyet alınabilir");
```

(a)

```
eğer(yaş < 16)
    Sistem.out.println
        ("Ehliyet alınamaz");
aksi takdirde
    Sistem.out.println
        ("Ehliyet alınabilir");
```

(b)

3.14 Aşağıdaki kodun çıktısı **number14,15** ve **30** olduğunda nedir?

```
eğer(number % 2 == 0)
    System.out.println
        (number + " çift");
eğer(number % 5 == 0)
    System.out.println
        (number + " 5'in katı");
```

(a)

```
eğer(number % 2 == 0)
    System.out.println
        (number + " çift");
aksi takdirde eğer(number % 5 == 0)
    System.out.println
        (number + " 5'in katı");
```

(b)

3.8 Rastgele Sayılar Üretme



Math.random() kullanarak 0.0 ile 1.0 arasında, 1.0 hariç, rastgele bir double değeri elde edebilirsiniz.



VideoNotu

Program çıkarma sınavı

rastgele() metodu

Diyeim ki birinci sınıf öğrencisinin çıkarma işlemi pratiği yapması için bir program geliştirmek istiyorsunuz. Program, **number1** ve **number2** olarak adlandırılan iki tek haneli tam sayıyı rastgele üretir, **number1 >= number2** koşulunu sağlar ve öğrenciyi "9 - 2 nedir?" gibi bir soru gösterir. Öğrenci cevabı girdikten sonra, program doğru olup olmadığını belirten bir mesaj gösterir.

Önceki programlar, rastgele sayılar üretmek için **System.currentTimeMillis()** kullanır. Daha iyi bir yaklaşım, **Math** sınıfındaki **random()** metodunu kullanmaktır. Bu metodu çağırmak, **d** gibi rastgele bir double değeri döndürür. Böylece, 0,0t ...t dt 6t 1,0. Böylece, **(int)(Math.random() * 10)** rastgele bir tek haneli tam sayı döndürür (yani, 0 ile 9 arasında bir sayı).

Program şu şekilde çalışabilir:

1. **number1** ve **number2**'ya iki tek haneli tam sayı üretin.
2. Eğer **number1 < number2** ise, **number1** ile **number2**'yi değiştir.
3. Öğrenciyi, "number1 - number2 nedir?" diye cevap vermesi için yönlendir.
4. Öğrencinin cevabını kontrol edin ve cevabın doğru olup olmadığını gösterin.

Tam program Liste 3.4'te gösterilmiştir.

LISTE 3.4 ÇıkarmaQuiz.java

rastgele sayı

cevabı al

cevabı kontrol et

```

1  import java.util.Scanner;
2
3  public class ÇıkarmaQuiz {
4      public static void main(String[] args) {
5          // 1. İki rastgele tek haneli tam sayı oluşturun
6          int number1 = (int)(Math.random() * 10);
7          int number2 = (int)(Math.random() * 10);
8
9          // 2. Eğer number1 < number2 ise, number1'i number2 ile değiştirin
10         if(number1 < number2) {
11             int temp = number1;
12             number1 = number2;
13             number2 = temp;
14         }
15
16         // 3. Öğrenciyi "number1 - number2 nedir?" diye cevap vermesi için teşvik et
17         System.out.print("number1 - number2 nedir? ");
18         ("Ne demek " + number1 + " - " + number2 + "? ");
19         Scanner input = new Scanner(System.in);
20         int answer = input.nextInt();
21
22         // 4. Cevabı değerlendir ve sonucu göster
23         if(number1 - number2 == answer) {
24             System.out.println("Doğru cevap verdiniz!");
25             // 5. Doğru cevap verildiğinde
26             System.out.println("Cevabınız yanlıştır. " + number1 + " - " + number2 + " = " + answer);
27         } else {
28             System.out.println("Cevabınız yanlıştır. " + number1 + " - " + number2 + " = " + answer);
29         }
30     }
31 }
```



6 - 6 nedir?
Doğru!

0

→ Enter

9 - 2 nedir?
Cevabınız yanlış
9 - 2, 7'dir

5

↵ Enter



satır#	numara1	numara2	geçici	cevap	çıktı
6	2				
7		9			
11			2		
12	9				
13		2			
20				5	
26					Cevabınız yanlış 9 - 2 7 olmalıdır

İki değişkeni **number1** ve **number2** değiştirmek için, geçici bir değişken **temp** (satır 11) kullanılır temel olarak **number1** içindeki değeri tutmak için. **number2** içindeki değer **number1**'ye atanır (satır 12), ve **temp** içindeki değer **number2**'ye atanır (satır 13).

3.15 Aşağıdakilerden hangisi **Math.random()** çağrıldığında olası bir çıktıdır?

323.4, 0.5, 34, 1.0, 0.0, 0.234

3.16 a. Rastgele bir tam sayı **i** nasıl üretilir b. Rastgele bir tamsayı **i** nasıl üretilir c. Rastgele bir tamsayı **i** nasıl üretilir

Nokta Kontrol

MyProgrammingLab™

3.9 Vaka Çalışması: Vücut Kitle İndeksi Hesaplama

Vücut kitle indeksini yorumlayan bir program yazmak için iç içe **if** ifadeleri kullanabilirsiniz.

Vücut Kitle İndeksi (VKİ), boy ve kiloya dayalı bir sağlık ölçüsüdür. Kilogram cinsinden kilonuzu alarak ve bunu boyunuzun metre cinsinden karesi ile bölerek hesaplanabilir. 20 yaş ve üzeri insanlar için VKİ'nin yorumu şu şekildedir:



VKİ	Yorumlama
18.5'in altında	Zayıf
18.5–24.9	Normal
25.0–29.9	Fazla kilolu
30.0'in üzerinde	Obez

Kullanıcıdan pound cinsinden bir ağırlık ve inç cinsinden bir yükseklik girmesini isteyen ve BMI'yi gösteren bir program yazın. Bir pound'un **0.45359237** kilogram ve bir inçin **0.0254**

metre olduğunu unutmayın. Liste 3.5 programı vermektedir.

LISTE 3.5 ComputeAndInterpretBMI.java

```

1  import java.util.Scanner;
2
3  genel sınıf ComputeAndInterpretBMI {
4      public static void main(String[] args) {
5          Tarayıcı girişi = new Scanner(System.in);
6
7          // Kullanıcıdan pound cinsinden ağırlığı girmesini isteyin
8          System.out.print("Ağırlığı pound cinsinden girin: ");
9          double weight = input.nextDouble();
10
11         // Kullanıcıdan inç cinsinden boyu girmesini isteyin
12         System.out.print("Boyu inç cinsinden girin: ");
13         double height = input.nextDouble();
14
15         final double KILOGRAMS_PER_POUND = 0.45359237; // Sabit
16         final double METERS_PER_INCH = 0.0254; // Sabit
17
18         // BMI'yi hesaplayın
19         double weightInKilograms = ağırlık * KILOGRAMS_PER_POUND;
20         double heightInMeters = yükseklik * METERS_PER_INCH;
21         double bmi = ağırlıkInKilograms /
22             (yükseklikMetreCinsinden * yükseklikMetreCinsinden);
23
24         // Sonucu göster
25         System.out.println("BMI " + bmi);
26         if (bmi < 18.5)
27             System.out.println("Aşırı zayıf");
28         else if (bmi < 25)
29             System.out.println("Normal");
30         else if (bmi < 30)
31             System.out.println("Aşırı kilolu");
32         aksi takdirde
33             System.out.println("Obez");
34     }
35 }

```

girdi ağırlığı

girdi boyu

hesapla bmi

çıktıyı göster



Ağırlığı pound cinsinden girin: 146

Boyu inç cinsinden girin: 70

BMI 20.948603801493316

Normal



satır#	ağırlık	boy ağırlıkKilogram	boyMetre bmi	çıktı
9	146			
13		70		
19		66.22448602		
20			1.778	
21				20.9486
25				BMI 20.95
31				Normal

Sabitler **KILOGRAMS_PER_POUND** ve **METERS_PER_INCH** satır

15-16'da tanımlanmıştır. Burada sabitlerin kullanılması programları okunması kolay hale getirir.

3.10 Vaka Çalışması: Vergi Hesaplama

Vergi hesaplamak için iç içe geçmiş **if** ifadelerini kullanarak bir program yazabilirsiniz.

Amerika Birleşik Devletleri federal kişisel gelir vergisi, beyan durumu ve vergilendirilebilir gelire dayalı olarak hesaplanır. Dört beyan durumu vardır: tek beyan verenler, evli olarak ortak beyan verenler veya nitelikli dul(erkek), evli olarak ayrı beyan verenler ve hane reisi. Vergi oranları her yıl değişir. Tablo 3.2, 2009 yılı için oranları göstermektedir. Diyelim ki, vergilendirilebilir geliri 10.000 \$ olan bir bekar iseniz, ilk 8.350 \$ %10 oranında vergilendirilir ve diğer 1.650 \$ %15 oranında vergilendirilir, bu nedenle toplam verginiz 1.082,50 \$'dır.



VideoNotu

Çok yönlü eğer-değilse ifadelerini kullanın

TABLO 3.2 2009 ABD Federal Kişisel Vergi Oranları

Marjinal Vergi Oranı	Bekar	Evli Ortak Beyan or Nitelikli Dul(erkek)	Evli Ayrı Beyan Verenler	Hane Reisi
10%	\$0 – \$8,350	\$0 – \$16,700	\$0 – \$8,350	\$0 – \$11,950
15%	\$8,351 – \$33,950	\$16,701 – \$67,900	\$8,351 – \$33,950	\$11,951 – \$45,500
25%	\$33,951 – \$82,250	\$67,901 – \$137,050	\$33,951 – \$68,525	\$45,501 – \$117,450
28%	\$82,251 – \$171,550	\$137,051 – \$208,850	\$68,526 – \$104,425	\$117,451 – \$190,200
33%	\$171,551 – \$372,950	\$208,851 – \$372,950	\$104,426 – \$186,475	\$190,201 – \$372,950
35%	372.951\$ +	372.951\$ +	186,476\$ +	372.951\$ +

Bir kişisel gelir vergisi hesaplamak için bir program yazmalısınız. Programınız, kullanıcının dosyala ma durumunu ve vergilendirilebilir geliri girmesini istemeli ve vergiyi hesaplamalıdır. Tek dosyalayıcı lar için **0**, evli ortak dosyalama veya nitelikli dul/dul için **1**, evli ayrı dosyalama için **2** ve hane reisi için **3** girin.

Programınız, vergilendirilebilir gelir için vergiyi dosyalama durumuna göre hesaplar. Dosyala ma durumu, aşağıda belirtilen **if** ifadeleri kullanılarak belirlenebilir:

```
eğer(durum == 0) {
    // Tek başına beyan edenler için vergi hespla
}
aksi takdirde eğer(durum == 1) {
    // Eşli beyan edenler veya nitelikli dul/dul için vergi hespla
}
aksi takdirde eğer(durum == 2) {
    // Ayrı beyan eden evli çiftler için vergi hespla
}
aksi takdirde eğer(durum == 3) {
    // Hanehalkı başı için vergi hespla
}
başka{
    // Yanlış durumu göster
}
```

Her beyan durumu için altı vergi oranı vardır. Her oran, belirli bir vergilendirilebilir gelir miktarına uygulanır. Örneğin, tek başına beyan edenler için 400.000 \$'lık vergilendirilebilir gelirden, 8.350 \$ %10, (33.950 – 8.350) %15, (82.250 – 33.950) %25, (171.550 – 82.250) %28, (372.950 – 171.550) %33 ve (400.000 – 372.950) %35 oranında vergilendirilir.

Liste 3.6, tek başına beyan edenler için vergileri hesaplama çözümünü vermektedir. Tam çözüm bir alıştırmaya bırakılmıştır.

Listing 3.6tComputeTax.java

```

1  import java.util.Scanner;
2
3  genel sınıfComputeTax {
4      public static voidtmain(String[] args) {
5          // Bir Scanner oluştur
6          Tarayıcı girişi = tnewtScanner(System.in);
7
8          // Kullanıcıdan dosyalama durumunu girmesini isteyin
9          System.out.print(
10             "(0-bekar, 1-evli birlikte veya nitelikli dul/dul,
11             + "2-evli ayrı, 3-hanehalkı başı)" +
12             "Dosyalama durumunu girin: ";
13             girdi durumu
14             inttstatus = input.nextInt();
15
16             // Kullanıcıdan vergilendirilebilir geliri girmesini isteyin
17             System.out.print("Vergilendirilebilir geliri girin: ");
18             girdi geliri
19             doublegelir = input.nextDouble();
20
21             // Vergiyi hesapla
22             doublevergi = 0;
23
24             vergi hesapla
25             if(durum == 0) { // Bekar beyannameler için vergiyi hesapla
26                 if(gelir <= 8350)
27                     vergi = gelir * 0.10;
28                 else if(gelir <= 33950)
29                     vergi = 8350*0.10+ (gelir - 8350) * 0.15;
30                 else if(gelir <= 82250)
31                     vergi = 8350*0.10+ (33950-8350) * 0.15+
32                     (gelir - 33950) * 0.25;
33                 diğer eğer(gelir <= 171550)
34                     vergi = 8350*0.10+ (33950-8350) * 0.15+
35                     (82250-33950) * 0.25+ (gelir - 82250) * 0.28;
36                 diğer eğer(gelir <= 372950)
37                     vergi = 8350*0.10+ (33950-8350) * 0.15+
38                     (82250-33950) * 0.25+ (171550-82250) * 0.28+
39                     (gelir - 171550) * 0.33;
40                 aksi takdirde
41                 vergi = 8350*0.10+ (33950-8350) * 0.15+
42                     (82250-33950) * 0.25+ (171550-82250) * 0.28+
43                     (372950-171550) * 0.33+ (gelir - 372950) * 0.35;
44             }
45             aksi takdirde eğer(durum == 1) { // Egzersiz olarak bırakıldı
46                 // Eşli dosya olarak veya nitelikli dul(erkek) için vergi hesapla
47             }
48             aksi takdirde eğer(durum == 2) { // Ayrı evli için vergi hesapla
49                 // Egzersiz olarak bırakıldı
50             }
51             else if(durum == 3) { // Hanehalkı için vergi hesapla
52                 // Egzersiz olarak bırakıldı
53             }
54             başka{
55                 System.out.println("Hata: geçersiz durum");
56                 System.exit(1);
57                 programdan çık
58             }
59             // Sonucu göster
60             System.out.println("Vergi " + (int)(tax * 100) / 100.0);
61         }
62     }

```

(0-bireysel vergi mükellefi, 1-evli ortak veya nitelikli dul/dul,
2-evli ayrı, 3-hane reisi)
Başvuru durumunu girin: 0
Vergiye tabi geliri girin: 400000
Vergi 117683.5'tir



satır#	durum	gelir	vergi	çıkıtı
13	0			
17		400000		
20			0	
38			117683.5	
57				Vergi 117683.5'tir

Program, başvuru durumunu ve vergiyi hesaplamak için vergiye tabi geliri alır. Çok yönlü **if-else** durumları (satırlar 22, 42, 45, 48, 51) başvuru durumunu kontrol eder ve vergi hesaplar.

System.exit(durum) (satır 53) **System** sınıfında tanımlanmıştır. Bu metodu çağırmak programı sonlandırır. Durum **0** programın normal bir şekilde sonlandığını gösterir. Bir sıfırdan farklı durum kodu anormal sonlanmayı gösterir.

System.exit(durum)

Başlangıç değeri olarak **0**, **tax** değişkenine (satır 20) atanmıştır. Eğer başlangıç değeri yoksa, derleme hatası meydana gelecektir, çünkü **tax** değişkenine değer atayan diğer tüm ifadeler ifadesinin içindedir. Derleyici, bu ifadelerin çalıştırılmayabileceğini düşündüğü için bir derleme hatası bildirir.

Bir programı test etmek için, tüm durumları kapsayan girişi sağlamalısınız. Bu program için, **girisiniz tüm durumları kapsamalıdır (0,1,2,3)**. Her durum için, altı

tüm durumları test et

vergi dilimi için vergiyi test edin. Yani, toplamda 24 durum vardır.



Tt tip

Tüm programlar için, küçük bir miktar kod yazmalı ve daha fazla kod eklemeyen önce bunu test etmelisiniz.

Bu, *artımlı geliştirme ve test etme* olarak adlandırılır. Bu yaklaşım, test etmeyi kolaylaştırır, çünkü hatalar muhtemelen yeni eklediğiniz koddadır.

artımlı geliştirme ve
test etme

3.17 Aşağıdaki iki ifade eşdeğer mi?

Nokta Control

MyProgrammingLab™

```
eğer(gelir <= 10000)
    vergi = gelir * 0.1;
aksi takdirde eğer(gelir <= 20000)
    vergi = 1000+
        (gelir - 10000) * 0.15;
```

```
eğer(gelir <= 10000)
    vergi = gelir * 0.1;
aksi takdirde eğer(gelir > 10000&&
    gelir <= 20000)
    vergi = 1000+
        (gelir - 10000) * 0.15;
```

3.11 Mantıksal Operatörler

Mantıksal operatörler !, &&, || ve ^, bileşik bir Boolean

ifadesi oluşturmak için kullanılabilir.



Bazen, bir ifadenin yürütülüp yürütülmeyeceği, birkaç koşulun bir kombinasyonu ile belirlenir. Bu koşulları birleştirerek bileşik bir Boolean ifadesi oluşturmak için mantıksal operatörleri kullanabilirsiniz. *Mantıksal operatörler*, ayrıca *Boolean operatörleri* olarak da bilinir, Boolean değerleri üzerinde çalışır.

yeni bir Boolean değeri oluşturmak için. Tablo 3.3 Boolean operatörlerini listeler. Tablo 3.4 not(!) operatörünü tanımlar, bu operatör **true**'yu **false**'ya ve **false**'yu **true**'ya çevirir. Tablo 3.5 ve(&&) operatörünü tanımlar. İki Boolean operandının ve (&&) durumu **true**'dur yalnızca her iki operand da **true** olduğunda. Tablo 3.6 veya (||) operatörünü tanımlar. İki Boolean operandının veya (||) durumu **true**'dur eğer en az bir operand **true** ise. Tablo 3.7 özel veya (^) operatörünü tanımlar. İki Boolean operandının özel veya (^) durumu **true**'dur yalnızca iki operand farklı Boolean değerlerine sahipse. Not edin ki **p1 ^ p2, p1 != p2** ile aynıdır.

TABLE 3.3 Boolean Operatörleri

Operatör	İsim	Açıklama
!	değil	mantıksal olumsuzlama
&&	ve	mantıksal kesişim
	veya	mantıksal ayırım
^	özel veya	mantıksal dışlama

TABLO 3.4 Operatör için Doğruluk Tablosu !

p	!p	Örnek (varsayalım ki yaş = 24, cinsiyet = 'K')
doğru	yanlış	!(yaş > 18) yanlıştır, çünkü (yaş > 18) doğrudur.
yanlış	doğru	!(cinsiyet == 'M') doğrudur, çünkü (cinsiyet == 'M') yanlıştır.

TABLO 3.5 Operatör için Doğruluk Tablosu &&

p ₁	p ₂	p ₁ && p ₂	Örnek (varsayalım ki yaş = 24, cinsiyet = 'K')
yanlış	yanlış	yanlış	(yaş > 18) && (cinsiyet == 'F') doğrudur, b çünkü (yaş > 18) ve (cinsiyet == 'F') her ikisi de doğrudur.
yanlış	doğru	yanlış	
doğru	yanlış	yanlış	(yaş > 18) && (cinsiyet != 'F') yanlıştır, b çünkü (cinsiyet != 'F') yanlıştır.
doğru	doğru	doğru	

TABLE 3.6 Operatör için Doğruluk Tablosu ||

p ₁	p ₂	p ₁ p ₂	Örnek (varsayalım ki yaş = 24, cinsiyet = 'K')
yanlış	yanlış	yanlış	(yaş > 34) (cinsiyet == 'K') doğrudur, çünkü (cinsiyet == 'K') doğrudur.
yanlış	doğru	doğru	
doğru	yanlış	doğru	(yaş > 34) (cinsiyet == 'E') yanlıştır, çünkü (yaş > 34) ve (cinsiyet == 'E') her ikisi de yanlıştır.
doğru	doğru	doğru	

TABL 03.7 Operatör için Doğruluk Tablosu ^

$\neg p$	$\neg q$	$\neg p \wedge \neg q$	Örnek (varsayalım yaş = 24, cinsiyet = 'F')
yanlış	yanlış	yanlış	$(yaş > 34) \wedge (cinsiyet == 'K')$ doğru, çünkü $(yaş > 34)$ yanlıştır ama $(cinsiyet == 'K')$ doğrudur.
yanlış	doğru	doğru	
doğru	yanlış	doğru	$(yaş > 34) \wedge (cinsiyet == 'E')$ yanlıştır, çünkü $(yaş > 34)$ ve $(cinsiyet == 'E')$ her ikisi de yanlıştır.
doğru	doğru	yanlış	

Liste 3.7, bir sayının 2 ve 3 ile, 2 veya

3 ile, ve 2 veya 3 ile ama her ikisiyle değil olup olmadığını kontrol eden bir program verir:

LISTE 3.7 TestBooleanOperators.java

```

1  import java.util.Scanner;
2
3  genel sınıf TestBooleanOperators {
4      public static void main(String[] args) {
5          // Bir Scanner oluştur
6          Tarayıcı girişi = new Scanner(System.in);
7
8          // Bir girdi al
9          System.out.print("Bir tam sayı girin: ");
10         int number = input.nextInt();
11
12         eğer(sayı % 2 == 0 && sayı % 3 != 0)
13             System.out.println(number + " 2 ve 3'e tam bölünebilir.");
14
15         if(sayı % 2 != 0 || sayı % 3 != 0)
16             System.out.println(number + " 2 veya 3'e tam bölünebilir.");
17
18         if(sayı % 2 != 0 ^ sayı % 3 != 0)
19             System.out.println(sayı +
20                 " 2 veya 3'e tam bölünebilir, ancak her ikisi de değil.");
21     }
22 }
```

import sınıf

girdi

ve

veya

özel veya

Bir tamsayı girin: 4

4, 2 veya 3'e tam bölünebilir.

4, 2 veya 3'e tam bölünebilir, ancak her ikisi de değil.



Bir tamsayı girin: 18

18, 2 ve 3'e tam bölünebilir.

18, 2 veya 3'e tam bölünebilir.



$(number \% 2 == 0 \ \&\& \ number \% 3 == 0)$ (satır 12) sayının hem 2 hem de 3'e tam bölünüp bölünmediğini kontrol eder. $(number \% 2 == 0 \ || \ number \% 3 == 0)$ (satır 15) sayının 2'ye ve/veya 3'e tam bölünüp bölünmediğini kontrol eder. $(number \% 2 == 0 \ \wedge \ number \% 3 == 0)$ (satır 18) sayının 2 veya 3'e tam bölünüp bölünmediğini, ancak her ikisine birden değil, kontrol eder.



Dikkat

Matematikte, ifade

1 <= AyınGün Sayısı <= t31

uyumsuz operandlar

doğrudur. Ancak, Java'da yanlıştır, çünkü `1 <= numberOfDaysInAMonth` bir `boolean` d egeri olarak değerlendirilir, bu da `31` ile karşılaştırılmaz. Burada, iki operand (bir `boolean` değeri ve bir sayısal değer) *uyumsuz*'dur. Java'daki doğru ifade

```
(t1 <= NumberOfDaysInAMonth) && (numberOfDaysInAMonth <= t 31t)
```

boolean'a dönüştürülemez



Not

Önceki bölümde gösterildiği gibi, bir `char` değeri bir `int` değerine dönüştürülebilir ve tersine. Ancak, bir `boolean` değeri başka bir türdeki bir değere dönüştürülemez, ayrıca başka bir türdeki bir değer de bir `boolean` değerine dönüştürülemez.

De Morgan'ın yasası



Not

De Morgan yasası, Hindistan doğumlu Britanyalı matematikçi ve mantıkçı Augustus De Morgan (1806-1871) adını taşıyor ve Boole ifadelerini basitleştirmek için kullanılabilir. Yasa şunu belirtir:

```
!(condition1 && condition2) şudur
!condition1 || !condition2
!(condition1 || condition2) şudur
!condition1 && !condition2
```

Örneğin,

```
!(sayı % t2t==t0 && sayı % t3t==t0t)
```

eşdeğer bir ifade kullanılarak basitleştirilebilir:

```
(sayı % t2t!=t0t || sayı % t3t!=t0t)
```

Başka bir örnek olarak,

```
!(sayı == t2t || sayı == t3t)
```

daha iyi şöyle yazılır

```
sayı != t2t&& sayı != t3
```

şartlı operatör
kısa devre operatörü

Bir `&&` operatörünün operandlarından biri `false` ise, ifade `false` olur; bir `||` operatörünün operandlarından biri `true` ise, ifade `true` olur. Java, bu özellikleri bu operatörlerin performansını artırmak için kullanır. `p1 && p2` değerlendirildiğinde, Java önce `p1`'i değerlendirir ve ardından, eğer `p1 true` ise, `p2`'yi değerlendirir; eğer `p1 false` ise, `p2`'yi değerlendirmez. `p1 || p2` değerlendirildiğinde, Java önce `p1`'i değerlendirir ve ardından, eğer `p1 false` ise, `p2`'yi değerlendirir; eğer `p1 true` ise, `p2`'yi değerlendirmez. Bu nedenle, `&&` koşullu veya kısa devre AND operatörü olarak adlandırılır ve `||` dir şartlı veya kısa devre OR operatörü olarak adlandırılır. Java ayrıca, ileri düzey okuyucular için Ek III.C ve III.D'de ele alınan şartlı AND (`&`) ve OR (`|`) operatörlerini de sağlar.

3.18 x'in 1 olduğunu varsayarak, aşağıdaki Boolean ifadelerinin sonucunu gösterin.

```
(ttrue) && (t3t>t4t)n(
x > t0t) && (x > t0t)n(x >
t0t) || (x < t0t)n(x != t0
```

```
t) || (x == t0t)n(x >= t0t) |
| (x < t0t)n(x != t1t) == !(
x == t1t)
```

3.19 num değişkeninde saklanan bir sayının 1 ile 100 arasında olduğunu değerlendirip true döndüren bir Boolean ifadesi yazın.

3.20 num değişkeninde saklanan bir sayının 1 ile 100 arasında olduğunu veya sayının negatif olduğunu değerlendirip true döndüren bir Boolean ifadesi yazın.

Nokta Kontrol

MyProgrammingLab™

3.21 `x` veyint türünde olduğunu varsayalım. Aşağıdakilerden hangileri geçerli Java ifadeleridir?

```
x > y > t0
x = y && ynx
/= y
x veya y
x ve y
(x != t0t) || (x = t0t)
```

3.22 `x1` olduğunu varsayalım. Aşağıdaki ifadenin değerlendirilmesinden sonra `x` nedir?

```
A. (x >= t1t) && (x++ > t1t)nb. (
x > t1t) && (x++ > t1t)
```

3.23 İfadenin değeri nedir `ch >= 'A' && ch <= 'Z'` eğer `ch 'A','p', 'E' veya '5'` ise?

3.24 Varsayalım ki, programı çalıştırdığınızda, konsoldan **2 3 6** girişi yapıyorsunu z. Çıktı nedir?

```
genel sınıfTest {
    public static voidtmain(String[] args) {njava.util.Sca
        nner input = tnewtjava.util.Scanner(System.in);ndoubletx = input.nextDoubl
        e();
        çiftlik = input.nextDouble();
        doubleletz = input.nextDouble();

        System.out.println("(x < y && y < z) ifadesi "+ (x < y && y < z));
        System.out.println("(x < y || y < z) ifadesi "+ (x < y || y < z));
        System.out.println("(!(x < y) ifadesi "+ !(x < y));
        System.out.println("(x + y < z) ifadesi "+ (x + y < z));
        System.out.println("(x + y < z) ifadesi "+ (x + y < z));
    }
}
```

3.25 Yaş**13**'ten büyük ve**18**'den küçükse**true** değerini döndüren bir Boolean ifadesi yazın.

3.26 Ağırlık**50** pound'dan büyükse veya boy**60** inçten büyükse**true** değerini döndüren bir Boolean ifadesi yazın.

3.27 Bir Boolean ifadesi yazın, **true** olarak değerlendirir eğer ağırlık**50** pounddan büyükse ve yükseklik **60** inçten büyükse.

3.28 Bir Boolean ifadesi yazın, true olarak değerlendirir eğer ya ağırlık 50 pounddan büyükse ya da yükseklik **60** inçten büyükse, ancak her ikisi de değil.

3.12 Vaka Çalışması: Artık Yılı Belirleme

Bir yıl, 4'e tam bölünebiliyorsa artık yıl olarak kabul edilir, ancak 100'e tam bölünemiyorsa veya 400'e tam bölünebiliyorsa.

Bir yılın artık yıl olup olmadığını kontrol etmek için aşağıdaki Boolean ifadelerini kullanabilirsiniz:

```
// Bir artık yıl 4'e tam bölünebilir
booleanisLeapYear = (year % 4==0);

// Bir artık yıl 4'e tam bölünebilir ancak 100'e tam bölünemez
isLeapYear = isLeapYear && (year % 100!=0);

// Bir artık yıl 4'e tam bölünebilir ancak 100'e tam bölünemez veya 400'e tam bölünebilir
isLeapYear = isLeapYear || (year % 400==0);
```

Ya da bu ifadeleri tek bir ifadede birleştirebilirsiniz:

```
isLeapYear = (yıl % 4t ==t 0t && yıl % 100t !=t 0t) || (yıl % 400t ==t 0t);
```



Anahtar
Nokta

Liste 3.8, kullanıcının bir yıl girmesine ve bunun artık yıl olup olmadığını kontrol etmesine olanak tanıyan programı vermektedir.

LISTE 3.8 LeapYear.java

```

1  import java.util.Scanner;
2
3  genel sınıfArtıkYıl {
4      public static void main(String[] args) {
5          // Bir Scanner oluştur
6          Scanner input = new Scanner(System.in);
7          System.out.print("Bir yıl girin: ");
8          int yıl = input.nextInt();
9
10         // Yılın artık yıl olup olmadığını kontrol et
11         boolean artıkYıl =
12             (yıl % 4 == 0 && yıl % 100 != 0) || (yıl % 400 == 0);
13
14         // Sonucu göster
15         System.out.println(yıl + " artık yıl mı? " + artıkYıl);
16     }
17 }

```

giriş

artık yıl mı?

sonucu göster



Bir yıl girin: 2012
2008 bir artık yıl mı? doğru



Bir yıl girin: 1900
1900 bir artık yıl mı? yanlış



Bir yıl girin: 2002
2002 bir artık yıl mı? yanlış

3.13 Vaka Çalışması: Piyango



Anahtar
Nokta

Piyango programı rastgele sayılar üretmeyi, rakamları karşılaştırmayı ve

Boolean operatörlerini kullanmayı içerir.

Diyeelim ki bir piyango oyunu oynamak için bir program geliştirmek istiyorsunuz. Program rastgele bir iki basamaklı piyango numarası üretir, kullanıcıdan iki basamaklı bir numara girmesini ister ve kullanıcının kazanıp kazanmadığını aşağıdaki kurallara göre belirler:

1. Eğer kullanıcı girişi piyango numarasıyla tam olarak eşleşiyorsa, ödül \$10,000'dır.
2. Eğer kullanıcı girişindeki tüm rakamlar piyango numarasındaki tüm rakamlarla eşleşiyorsa, ödül \$3,000'dır.
3. Eğer kullanıcı girişindeki bir rakam piyango numarasındaki bir rakamla eşleşiyorsa, ödül \$1,000'dır.

Tam program Liste 3.9'da gösterilmiştir.

LISTE 3.9 Lottery.java

```

1  import java.util.Scanner;
2
3  halka açık sınıfPiyango {
4      public static void main(String[] args) {
5          // Bir piyango numarası oluştur

```

```

678  int piyango = (int) (Math.random() * 100);
// Kullanıcıdan bir tahmin girmesini iste
9   Scanner input = new Scanner(System.in);
10  System.out.print("İki basamaklı piyango seçiminizi girin: ");
11  int tahmin = input.nextInt();
12
13  // Piyangodan basamakları al
14  int lotteryDigit1 = piyango / 10;
15  int lotteryDigit2 = piyango % 10;
16
17  // Tahminden basamakları al
18  int guessDigit1 = tahmin / 10;
19  int guessDigit2 = tahmin % 10;
20
21  System.out.println("Çekiliş numarası " + lottery);
22
23  // Tahmini kontrol et
24  go)
25  System.out.println("Tam eşleşme: 10,000$ kazandınız");
26  min Hane2 == lotteryDigit1 nift(tahmin == piyan
27  min Hane1 == piyangoDigit2) else if(tah
28  System.out.println("Tüm rakamlar eşleşti: 3,000$ kazandınız");
29  else if(tahmin Hane1 == piyangoDigit1 n&& tah
30  git1 == lotteryDigit2
31  minDigit2 == lotteryDigit1 n || tahminDi
32  || tahminDigit2 == lotteryDigit2) n || tah
33  System.out.println("Bir rakam eşleşti: 1,000$ kazandınız");
34  aksi takdirde
35  System.out.println("Üzgünüm, eşleşme yok");
36  }
37  }

```

bir piyango numarası oluştur

bir tahmin girin

tam eşleşme?

tüm rakamları eşleştir?

bir rakamı eşleştir?

Piyango seçiminizi girin (iki rakam):
Piyango numarası 12
Üzgünüm, eşleşme yok

45 Enter



Piyango seçiminizi girin:
Piyango numarası 34
Bir rakamı eşleştir: 1.000 \$ kazandınız

23 Enter



satır#	6	11	14	15	18	19	33
değişken							
piyango	34						
tahmin et		23					
piyangoRakam1			3				
piyangoRakam2				4			
tahminRakam1					2		
tahminRakam2						3	
Çıktı							Bir rakam eşleşti: 1.000 \$ kazandınız



Program, `random()` metodunu (satır 6) kullanarak bir piyango oluşturur ve kullanıcıdan tahmin girmesini ister (satır 11). `tahmin % 10`, `tahmin`'den son rakamı alır ve `tahmin / 10`, `tahmin`'den ilk rakamı alır, çünkü `tahmin` iki basamaklı bir sayıdır (satırlar 18–19).

Program, tahminin lotarya numarasıyla bu sırayla karşılaştırılmasını kontrol eder:

1. Öncelikle, tahminin lotarya ile tam olarak eşleşip eşleşmediğini kontrol edin (satır 24).
2. Eğer eşleşmiyorsa, tahminin tersinin lotarya ile eşleşip eşleşmediğini kontrol edin (satırlar 26–27).
3. Eğer eşleşmiyorsa, bir rakamın lotaryada olup olmadığını kontrol edin (satırlar 29–32).
4. Eğer eşleşmiyorsa, hiçbir şey eşleşmiyor ve "Üzgünüm, eşleşme yok" mesajını gösterin (satırlar 34–35).

3.14 switch İfadeleri



A switch ifadesi, bir değişkenin veya bir ifadenin değerine dayalı olarak ifadeleri yürütür.

Listing 3.6, `ComputeTax.java`'daki `if` ifadesi, tek bir `true` veya `false` koşuluna dayalı seçimler yapar. Vergi hesaplamak için durumun değerine bağlı dört durum vardır: `status`. Tüm durumları tam olarak hesaba katmak için iç içe `if` ifadeleri kullanılmıştır. İç içe `if` ifadelerinin aşırı kullanımı, bir programın okunmasını zorlaştırır. Java, birden fazla koşul için kodlamayı basitleştirmek amacıyla bir `switch` ifadesi sağlar. Listing 3.6'daki iç içe `if` ifadesini değiştirmek için aşağıdaki `switch` ifadesini yazabilirsiniz:

```
switch(durum) {
    case 0: tek başına beyan edenler için vergi hesapla;
            break;
    case 1: evli olarak ortak beyan edenler veya nitelikli dul(erkek) için vergi hesapla;
            kırılma;
    case 2: ayrı beyan eden evliler için vergi hesapla;
            break;
    case 3: hane reisi için vergi hesapla;
            break;
    default: System.out.println("Hata: geçersiz durum");
             System.exit(1);
}
```

Önceki `switch` ifadesinin akış diyagramı Şekil 3.6'da gösterilmiştir.

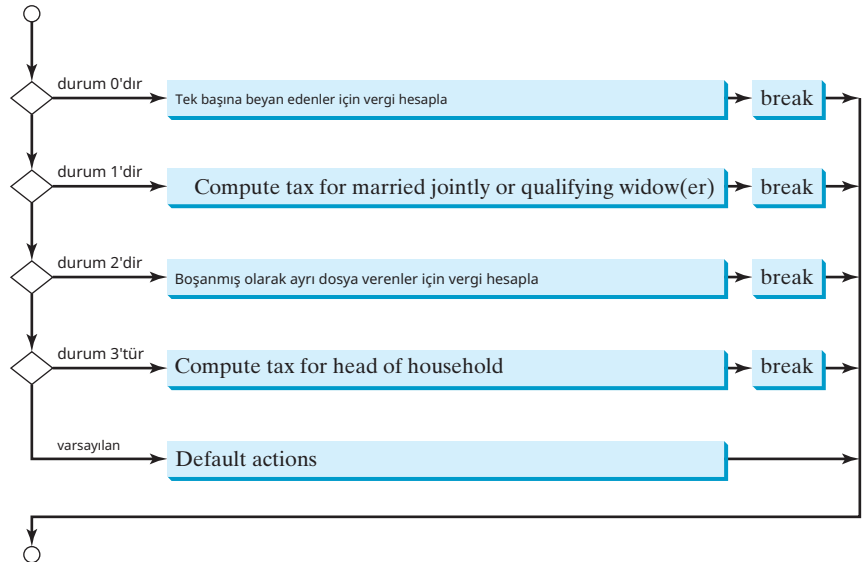


FIGURE 3.6 Anahtar ifadesi tüm durumları kontrol eder ve eşleşen durumda ifadeleri yürütür.

Bu ifade, durumun **0,1,2** veya **3** değerine eşit olup olmadığını kontrol eder, bu sırayla. Eşleşirse, ilgili vergi hesaplanır; eşleşmezse, bir mesaj görüntülenir. İşte **anahtar** ifadesinin tam sözdizimi:

```
anahtar(anahtar-ifadesi) {
    case deęer1: ifade(ler)1;
        break;
    case deęer2: ifade(ler)2;
        break;
    ...
    durumvalueN: ifade(ler)N;
        break;
    varsayılan: varsayılan-için-ifade(ler);
}
```

switch ifadesi

switch ifadesi aşağıdaki kurallara uyar:

- switch-ifadesi **char, byte, short, int** veya **String** türünde bir değer döndürmelidir ve her zaman parantez içinde olmalıdır. (switch ifadesinde **String** türünün kullanılması JDK 7'de yenidir.)
- **value1, . . . ,** ve **valueN**, switch-ifadesinin değerinin aynı veri türüne sahip olmalıdır. **value1, . . . ,** ve **valueN** sabit ifadeler olup, değişkenler içermemelidir, örneğin **1 + x**.
- Bir case ifadesindeki değer, switch-ifadesinin değeriyle eşleştğinde, bu case'ten itibaren başlayan ifadeler, ya bir **break** ifadesine ya da tamamen switch ifadesinin sonuna kadar yürütülür.
- İsteğe bağlı olan **default** case, belirtilen hiçbir case'in switch-ifadesiyle eşleşmediği durumlarda hareketler gerçekleştirmek için kullanılabilir.
- **break** anahtar kelimesi isteğe bağlıdır. **break** ifadesi hemen switch ifadesini sonlandırır.



Dikkat

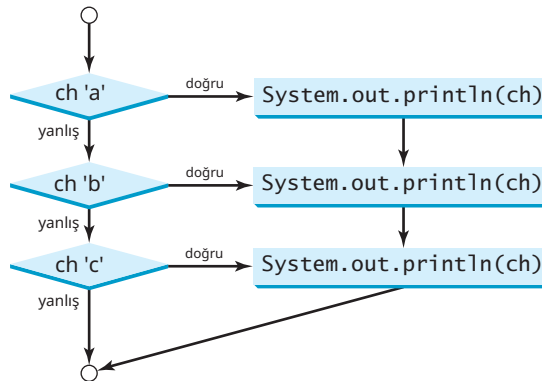
Bir **break** ifadesinin gerektiğinde kullanılmasını unutmayın. Bir case eşleştğinde, eşleşen case'ten itibaren başlayan ifadeler, ya bir **break** ifadesine ya da tamamen switch ifadesinin sonuna kadar yürütülür. Bu duruma **fall-through** davranışı denir. Örneğin, aşağıdaki kod **ch 'a'** ise karakteri **a** üç kez gösterir:

break olmadan

devam eden davranış

anahtarlar (kanal) {

```
vaka'a't: System.out.println(ch);n vaka'b't:
System.out.println(ch);n vaka'c't: System.o
ut.println(ch);n}
```

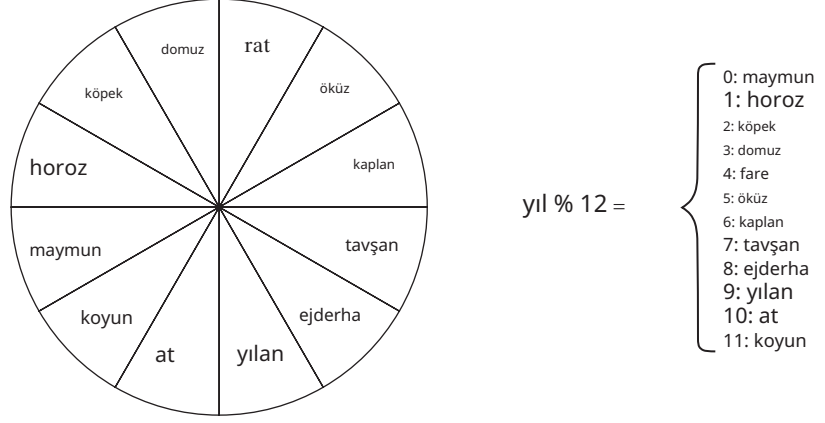


Tipp

Programlama hatalarını önlemek ve kodun bakımını iyileştirmek için, **break** kasıtlı olarak atlandığında bir yorum eklemek iyi bir fikirdir.

110 Bölüm 3 Seçimler

Şimdi, verilen bir yıl için Çin Zodyak burcunu bulmak üzere bir program yazalım. Çin Zodyak'ı, her yılın bir hayvanla temsil edildiği on iki yıllık bir döngüye dayanır—bu döngüde maymun, horoz, köpek, domuz, fare, öküz, kaplan, tavşan, ejderha, yılan, at veya koyun yer alır; bu, Şekil 3.7'de gösterilmiştir.



ŞEKİL3.7Çin Zodyakı on iki yıllık bir döngüye dayanmaktadır.

Dikkat edin ki yıl % 12 Zodyak burcunu belirler. 1900 yılı, 1900% 124 olduğu için fare yılındadır. Liste 3.10, kullanıcıdan bir yıl girmesini isteyen ve yıl için hayvanı gösteren bir programı vermektedir.

LİSTE 3.10 ÇinZodyak.java

```
1  import java.util.Scanner;  
23  
   public class ÇinZodyak {  
4       public static voidtmain(String[] args) {  
5   Tarayıcı girişi = tnewtScanner(System.in);  
6  
7       System.out.print("Bir yıl girin: ");  
8       intyıl = input.nextInt();  
9  
10      switch(yıl % 12) {  
11          case 0: System.out.println("maymun");break;  
12          case 1: System.out.println("horoz");break;  
13          case 2: System.out.println("köpek");break;  
14          case 3: System.out.println("domuz");break;  
15          case 4: System.out.println("sıçan");break;  
16          case 5: System.out.println("öküz");break;  
17          case 6: System.out.println("kapan");break;  
18          case 7: System.out.println("tavşan");break;  
19          case 8: System.out.println("ejderha");break;  
20          case 9: System.out.println("yılan");break;  
21          case 10: System.out.println("at");break;  
22          case 11: System.out.println("koyun");  
23 {Na 4}  
Nakh}
```

yılı girin

Zodyak burcunu belirleyin



Bir yıl girin:
tavşan

1963

Enter



Nokta C kontrol

MyProgrammingLab™

Bir yıl girin: 1877

ököz

- 3.29** Bir **switch** değişkeni için hangi veri türleri gereklidir? Eğer **break** anahtar kelimesi bir durum işlendiğinde kullanılmazsa, bir sonraki yürütülecek ifade nedir? Bir **switch** ifadesini eşdeğer bir **if** ifadesine dönüştürebilir misiniz, ya da tersine? Bir **switch** ifadesi kullanmanın avantajları nelerdir?
- 3.30** Aşağıdaki **switch** ifadesi yürütüldükten sonra **y** nedir? Kodu **if-else** ifadesi kullanarak yeniden yazın.

```
x = ver; y = tt; na
southa(x + tt) {nxt: y
  = t1t; ndflet: y += t
  1t;n}
```

- 3.31** Aşağıdaki **if-else** ifadesi yürütüldükten sonra **x** nedir? Bunu bir **switch** ifadesi kullanarak yeniden yazın ve yeni **switch** ifadesi için akış diyagramını çizin.

```
inttx = t1t, a = t3t;nift(
a == t1t)
  x += t5t;
else ift(a == t2t)nx +
  = t10t;
aksi takdirde eğer(a == 3)
  x += 16;
aksi takdirde eğer(a == 4)
  x += 34;
```

- 3.32** Bir **switch** ifadesi yazın, bu ifade bir **String** değişkeni **dayName**'ye Pazar, Pazartesi, Salı, Çarşamba, Perşembe, Cuma, Cumartesi atar, eğer **day** 0, 1, 2, 3, 4, 5, 6, buna göre.

3.15 Koşullu İfadeler

Bir koşullu ifade, bir koşula dayalı olarak bir ifadeyi değerlendirir.

Belirli koşullarla kısıtlanmış bir değişkene bir değer atamak isteyebilirsiniz. Örneğin, aşağıdaki ifade **1**'i **y**'ye atar eğer **x0**'dan büyükse, ve **-1**'i **y**'ye atar eğer **x0**'dan küçük veya eşitse.

```
eğer(x > 0)
  y = 1;
aksi takdirde
  y = t-1t;
```

Alternatif olarak, aşağıdaki örnekte olduğu gibi, aynı sonuca ulaşmak için bir koşullu ifade kullanabilirsiniz.

```
y = (x > t0t) ? 1 : -1 ;
```

Koşullu ifadeler tamamen farklı bir tarzda olup, durum bildiriminde açık bir **if** yoktur. Söz dizimi:

```
boolean-ifade ? ifade1 : ifade2;
```



koşullu ifade

112 Bölüm 3 Seçimler

Bu koşullu ifadenin sonucu **ifade1** dir **boolean-ifade** doğruysa; aksi takdirde sonuç **ifade2** dir.

Diyelim ki **num1** ve **num2** değişkenlerinin daha büyük sayısını **max** 'a atamak istiyorsunuz. Basi tçe koşullu ifadeyi kullanarak bir ifade yazabilirsiniz:

```
max = (sayı1 > sayı2) sayı1 : say2;
```

Başka bir örnek olarak, aşağıdaki ifade **num** çift ise "num çift" mesajını gösterir ve aksi takdirde "num tek" mesajını gösterir.

```
System.out.println((num % 2==0) ? "num çift":"num tek");
```

Bu örneklerden görebileceğiniz gibi, koşullu ifadeler kısa ve

özlü kod yazmanıza olanak tanır.



Not

Semboller **?** ve **:** bir koşullu ifadede birlikte görünür. Üç operand kullandığı için *ternary operator* olarak adlandırılan bir koşullu operatör oluştururlar. Java'daki tek ternary operatördür.

3.33 Diyelim ki, aşağıdaki programı çalıştırdığınızda, konsoldan **2 3 6** girişi y apıyorsunuz. Çıktı nedir?

```
genel sınıfTest {  
    public static voidtmain(String[] args) {njava.util.Sca  
        nner input = tnewtjava.util.Scanner(System.in);ndoubletx = input.nextDoubl  
        e();  
        çiftlik = input.nextDouble();  
        doubletz = input.nextDouble();  
  
        System.out.println((x < y && y < z) ? "sıralı": "sıralı değil");  
    }
```

3.34 Aşağıdaki **if** ifadelerini koşullu operatörü kullanarak yeniden yazın.

```
ift(yaş >= t16t)ntick  
    etPrice = t20t;nelse  
  
    biletFiyatı = t10t;
```

```
ift(count % t10t==t0t)Syste  
    m.out.print(count + t"n"t);nelse  
  
    System.out.print(sayım);
```

3.35 Aşağıdaki koşullu ifadeleri **if-else** ifadeleri kullanarak yeniden yazın.

- score = (x > 10) ? 3* ölçek : 4* ölçek;
- tax = (gelir > 10000) ? gelir * 0.2: gelir * 0.17+1000;
- System.out.println((sayı % t3t==t0t) i : j);

3.16 Konsol Çıktısını Biçimlendirme



*Konsolda biçimlendirilmiş çıktı göstermek için **System.out.printf** yöntemini kullanabilirsiniz.*

Sıklıkla, sayıları belirli bir formatta göstermek istenir. Örneğin, aşağıdaki kod miktar ve yıllık faiz oranı verildiğinde faizi hesaplar.

```
doublemiktar=12618.98;  
doublefaizOranı=0.0013;
```



```
double faiz = miktar * faizOrani;
System.out.println("Faiz " + faiz);
```

Faiz 16.404674



Faiz miktarı para birimi olduğundan, ondalık noktadan sonra yalnızca iki basamak göstermek istenir. Bunu yapmak için, kodu aşağıdaki gibi yazabilirsiniz:

```
double miktar = 12618.98;
double faizOrani = 0.0013;
double faiz = miktar * faizOrani;
System.out.println("Faiz "
    + (int)(faiz * 100) / 100.0);
```

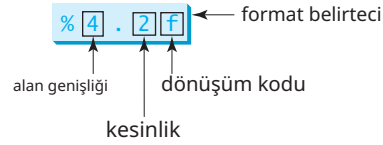
Faiz 16.4



Ancak, format hala doğru değil. Ondalık noktadan sonra iki basamak olmalıdır: **16.40** yerine **16.4**. Bunu **printf** metodu kullanarak düzeltebilirsiniz, şöyle:

baskı

```
double miktar = 12618.98;
double faizOrani = 0.0013;
double faiz = miktar * faizOrani;
System.out.printf("Faiz %4.2f", faiz);
```



Faiz 16.40



Bu metodu çağırmanın sözdizimi

```
System.out.printf(biçim, öge1, öge2, ..., ögek)
```

burada **format**, alt dizeler ve format belirteçlerinden oluşabilen bir dizedir.

A **format belirteci** bir ögenin nasıl görüntüleneceğini belirtir. Bir öge sayısal bir değer, bir karakter, bir Boolean değeri veya bir dize olabilir. Basit bir format belirteci, bir yüzde işareti (%) ve ardından bir dönüşüm kodu ile oluşur. Tablo 3.8, sıkça kullanılan

format belirteci

basit format belirteçlerinden bazılarını listelemektedir.

TABLO 3.8 Sık Kullanılan Format Belirteçleri

Format Belirteci	Çıktı	Örnek
%b	bir Boolean değeri	doğru veya yanlış
%c	bir karakter	A
%d	bir ondalık tam sayı	200
%f	bir ondalıklı sayı	45.460000
%Ve	standart bilimsel notasyonda bir sayı	4.556000e+tt01
%s	bir dize	"Java harika"

İşte bir örnek:

```
int sayısı = t5t;
ikili miktar = t45.56t;
System.out.printf(t"sayı %d ve miktar %f", sayım, miktar);
```

göster

count 5 ve amount 45.560000

öğeler

Öğeler, format belirleyicileriyle sırasına, sayısına ve tam türüne göre eşleşmelidir. Örneğin, count için format belirleyicisi %d ve amount için %f dir. Varsayılan olarak, bir ondalıklı değer ondalık noktadan sonra altı basamakla görüntülenir. Genişlik ve hassasiyeti bir format belirleyicisinde belirtebilirsiniz; bu, Tablo 3.9'daki örneklerde gösterilmiştir.

TABLO3.9Genişlik ve Kesinlik Belirleme Örnekleri

Örnek	Çıktı
%5c	Karakteri yazdır ve karakter öğesinin önüne dört boşluk ekle, çünkü genişlik 5'tir.
%ilaç	Boolean değerini yazdır ve false değerinin önüne bir boşluk, true değerinin önüne iki boşluk ekle.
%5d	En az 5 genişliğinde tam sayı öğesini yazdırın. Eğer öğedeki rakam sayısı <5, sayının önüne boşluk ekleyin. Eğer öğedeki rakam sayısı >5, genişlik otomatik olarak artırılır.
%10.F	Ondalık nokta ve noktadan sonra iki basamak dahil olmak üzere en az 10 genişliğinde ka yan nokta öğesini yazdırın.Böylece ondalık noktadan önce 7 rakam ayrılmıştır. Eğer öğ edeki ondalık noktadan önceki rakam sayısı <7, önüne boşluk ekleyin sayısı. Eğer öğedeki ondalık noktadan önceki rakam sayısı >7, genişlik otomatik olarak artırılır.
%10 A	Ondalık noktasını, noktadan sonraki iki basamağı ve üstel kısmı içeren en az 10 genişliğind e ondalıklı öğeyi çıktılaysın. Eğer bilimsel notasyonda görüntülenen sayının genişliği 10'd an küçükse, sayının önüne boşluk ekleyin.
%12'ler	Dizeyi en az 12 karakter genişliğinde yazdırın. Eğer dize öğesi 12 karakterden daha azsa, dize önüne boşluk ekleyin. Eğer dize öğesi 12 karakterden fazlaysa, genişlik otomatik olarak artırılır.

Eğer bir öğe belirtilen genişlikten daha fazla alana ihtiyaç duyuyorsa, genişlik otomatik olarak artırılır. Örneğin, aşağıdaki kod

```
System.out.printf(t"%3d#%2s#%3.2fn"t,t1234t,t"Java", 51.6653t);
```

gösterir

1234#Java#51.67

int öğesi 1234 için belirtilen genişlik 3'tür, bu da gerçek boyutundan 4'tür. Genişlik otomatik olarak 4'earttırılır. String öğesi Java için belirtilen genişlik 2'dir, bu da gerçek boyutu ndan 4'tür. Genişlik otomatik olarak 4'earttırılır. double öğesi 51.6653 için belirtilen genişli k 3'tür, ancak 51.67'yi göstermek için 5 genişliğe ihtiyaç vardır,bu nedenle genişlik otomati k olarak 5'e artırılır.

3.17 Operatör Önceliği ve Birliktelik 115

Varsayılan olarak, çıktı sağa yaslanmıştır. Çıktının belirtilen alanında ögenin sola yaslanması belirtmek için format belirleyicisinde eksi işaretini (-) koyabilirsiniz. Örneğin, aşağıdaki ifadeler

sağa yasla
sola yasla

```
System.out.printf("%8d%8s%8.1f\n",1234,t"Java", 5.63t);\nSystem.out.printf(t"%-8d%-8s%-8.1f n",t"1234,t"Java", 5.63t);
```

göster

```
|←∞→|←8→|←8→|
□□□1234□□□Java□□□5.6
1234□□□Java□□□5.6□□□
```

kare kutu () boş bir alanı belirtir.



Dikkat

Öğeler, format belirleyicileriyle tam olarak eşleşmelidir. Format belirleyicisi %f veya %e için öge, 40.0 gibi bir ondalıklı sayı türü değeri olmalıdır, 40 değil. Bu nedenle, bir int değeri ni %f veya %e ile eşleştiremez.



Tipp

% işareti bir format belirleyicisini belirtir. Format dizisinde bir literal % çıkarmak için %% kullanın.

3.36 Bir Boolean değeri, bir karakter, bir ondalık tam sayı, bir ondalıklı sayı ve bir dize çıkarmak için format belirleyicileri nelerdir?

3.37 Aşağıdaki ifadelere ne yanlıştır?

- System.out.printf(t"%5d %dt",t 1t,t 2t,t 3t);
- System.out.printf(t"%5d %ft",t 1t);
- System.out.printf(t"%5d %ft",t 1t,t 2t);

3.38 Aşağıdaki ifadelerin çıktısını gösterin.

- System.out.printf(" miktar %f %e",32.32,32.32);
- System.out.printf(" miktar %5.4f %5.4e",32.32,32.32);
- System.out.printf(t"%6bnt",t (t1t > t2t));
- System.out.printf(t"%6snt",t "tjavat");
- ve. System.out.printf(t"%-6b%nsnt",t (t1t > t2t)t, "tjavat");
- System.out.printf(t"%6b%-8snt",t (t1t > t2t)t, "tjavat");

Nokta Kontrol

MyProgrammingLab™

3.17 Operatör Önceliği ve Birliktelik

Operatör önceliği ve bağlanabilirlik, operatörlerin hangi sırayla değerlendirileceğini belirler.



Bölüm 2.11, aritmetik operatörlerle ilgili operatör önceliğini tanıttı. Bu bölüm, operatör önceliğini daha ayrıntılı olarak tartışmaktadır. Farz edelim ki bu ifadeye sahipsiniz:

```
3 + 4 * 4 > 5 * (4 + 3) - 1 && (4 - 3 > 5)
```

Değeri nedir? Operatörlerin yürütme sırası nedir?

Parantez içindeki ifade önce değerlendirilir. (Parantezler iç içe geçebilir, bu durumda içteki parantezdeki ifade önce yürütülür.) Bir ifadeyi değerlendirirken

parantezler, operatörler öncelik kuralı ve bağlanabilirlik kuralına göre uygulanır.

Öncelik kuralı, Tablo 3.10'da gösterildiği gibi, operatörler için öncelik tanımlar. Bu tablo, şimdiye kadar öğrendiğiniz operatörleri içermektedir. Operatörler, öncelik sırasına göre yukarıdan aşağıya doğru azalan sırada listelenmiştir. Mantıksal operatörler, ilişkisel operatörlerden daha düşük önceliğe sahiptir ve ilişkisel operatörler, aritmetik operatörlerden daha düşük önceliğe sahiptir. Aynı önceliğe sahip operatörler aynı grupta yer alır. (Bkz. Ek C, *Operatör Önceliği Tablosu*, Java operatörlerinin ve önceliklerinin tam listesi için.)

operatör önceliği

TABLO 3.10 Operatör Önceliği Tablosu

Öncelik	Operatör
	var++ ve var-- (Postfix)
	+, - (Unary artı ve eksi), ++var ve --var (Prefix)
	(tip) (Casting)
	!t (Değil)
	*, /, % (Çarpma, bölme ve kalan)
	+, - (İkili toplama ve çıkarma)
	<, <=, >, >= (Karşılaştırma)
	==, != (Eşitlik)
	^ (Özel VEYA)
	&& (VE)
	 (VEYA)
	=, +=, -=, *=, /=, %= (Atama operatörü)

operatör birleşimi

Aynı önceliğe sahip operatörler yan yana olduğunda, *birleşimleri* değerlendirme sırasını belirler. Atama operatörleri hariç tüm ikili operatörler *soldan birleşimlidir*. Örneğin, **+** ve **-** aynı önceliğe sahip oldukları ve soldan birleşim oldukları için, ifade

$$a - b + cd \quad \text{şuna eşdeğerdir} \quad ((a - b) + c) - d$$

Atama operatörleri *sağdan birleşik*. Bu nedenle, ifade

$$a = b += c = 5 \quad \text{şuna eşdeğerdir} \quad \text{bir} = (b += (c = 5))$$

Varsayalım **a**, **b** ve **c** atamadan önce 1'dir; tüm ifade değerlendirildikten sonra, **a** 6 olur, **b** 6 olur ve **c** 5 olur. Atama

operatörü için soldan birleşik olmanın mantıklı olmadığını unutmayın.



Not

Java'nın bir ifadeyi dahili olarak değerlendirmek için kendi yolu vardır. Java değerlendirmesinin sonucukarşılık gelen aritmetik değerlendirmenin sonucu ile aynıdır. İleri düzey okuyucular, Java'da bir ifadenin nasıl değerlendirildiği hakkında daha fazla tartışma için Ek III.B'ye başvurabilirler.

sahne arkasında

3.39 Boolean operatörlerinin öncelik sırasını listeleyin. Aşağıdaki ifadeleri değerlendirin:

tehdit edildi | | tehdit edildi & & yanlış t
ehdit edildi & & tehdit edildi | | yanlış

Nokta Kontrol

MyProgrammingLab™

3.40 Doğru mu yanlış mı? = hariç tüm ikili operatörler soldan sağa ilişkilidir.

3.41 Aşağıdaki ifadeleri değerlendirin:

2 * 2 - 3 > 2 && 4 - 2 > 5
2 * 2 - 3 > 2 || 4 - 2 > 5

3.42 Is (x > 0 && x < 10) aynı mı ((x > 0) && (x < 10))? Is (x > 0 || x < 10) aynı mı ((x > 0) || (x < 10))? Is (x > 0 || x < 10 && y < 0) aynı mı (x > 0 || (x < 10 && y < 0))?

3.18 Onay Diyalogları

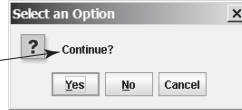
Kullanıcıdan onay almak için bir onay iletişim kutusu kullanabilirsiniz.

Bir mesaj iletişim kutusu görüntülemek için `showMessageDialog` ve bir giriş iletişim kutusu görüntülemek için `showInputDialog` kullandınız. Bazen bir soruya onay iletişim kutusu ile cevap vermek faydalı olabilir. Aşağıdaki

ifade kullanılarak bir onay iletişim kutusu oluşturulabilir:

int seçeneği =

`JOptionPane.showConfirmDialog`
(null, "Devam et");



Bir düğmeye tıklandığında, metod bir seçenek değeri döndürür. Değer `JOptionPane.YES_OPTION(0)` Evet düğmesi için, `JOptionPane.NO_OPTION(1)` Hayır düğmesi için ve `JOptionPane.CANCEL_OPTION(2)` İptal düğmesi için.

3.3 numaralı listedeki tahmin-birthday programını onay iletişim kutuları kullanarak yeniden yazabilirsiniz, 3.11 numaralı listedeki gibi. Şekil 3.8, programın gün için bir örnek çalışmasını göstermektedir.

19.

LİSTE 3.11 TahminDoğumGünüOnayİletişimKutusu.java

```
1 import javax.swing.JOptionPane;
2
3 genel sınıfTGuessDoğumGünüKullanımOnayDialog {
4     public static voidtmain(String[] args) {
5         Dize kümesi1 =
6             " 1 3 5 7n"t+
7             " 9 11 13 15n"t+
8             "17 19 21 23n"t+
9             "25 27 29 31"t;
10
11         Dize kümesi2 =
12             " 2 3 6 7n"t+
13             "10 11 14 15n"t+
14             "18 19 22 23n"t+
15             "26 27 30 31"t;
16
```

import sınıf

set1

set2



118 Bölüm 3 Seçimler

```
set3      1718      Dize kümesi3 =
           19      " 4 5 6 7n"t+
           20      "12 13 14 15n"t+
           21      "20 21 22 23n"t+
           22      "28 29 30 31"t;

set4      23      Dize kümesi4 =
           24      " 8 9 10 11n"t+
           25      "12 13 14 15n"t+
           26      "24 25 26 27n"t+
           27      "28 29 30 31"t;

set5      28      Dize seti5 =
           29      "16 17 18 19n"t+
           30      "20 21 22 23n"t+
           31      "24 25 26 27n"t+
           32      "28 29 30 31"t;
           33

           34      intgün = 0;

onay iletişim kutusu 35      // Kullanıcıyı soruları yanıtlamaya teşvik et
           36      nPane.showConfirmDialog(null,
           37      "Doğum gününüz bu sayılarda mı?" + set1);intcevap = JOptionPane
           38

set1'de mi? 39      if(cevap == JOptionPane.YES_OPTION)
           40
           41      42 gün += t1t;
           43
           44      cevap = JOptionPane.showConfirmDialog(nullt,
           45      "Doğum gününüz bu numaralar içinde mi?" + set2);
           46

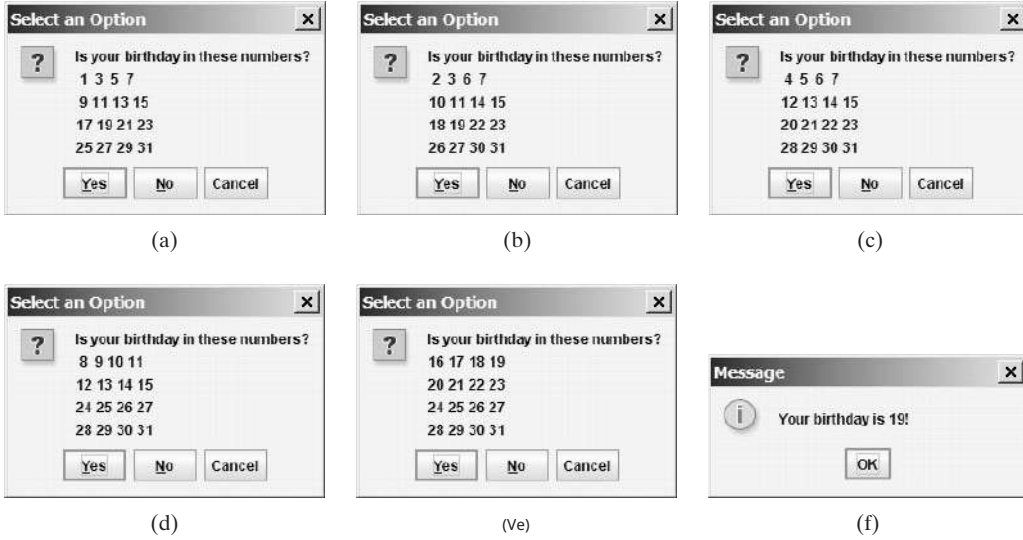
set2'de mi? 47      ift(answer == JOptionPane.YES_OPTION) n48 gü
           49      n += t2t;
           50
           51      50 cevap = JOptionPane.showConfirmDialog(nullt,
           52      "Doğum gününüz bu numaralarda mı?" + set3);
           53

set3'te mi? 54      eğer(answer == JOptionPane.YES_OPTION)
           55      gün += 4;
           56
           57      56 cevap = JOptionPane.showConfirmDialog(null,
           58      "Doğum gününüz bu numaralarda mı?" + set4);
           59

set4 içinde mi? 60      eğer(answer == JOptionPane.YES_OPTION)
           61      gün += 8;
           62
           63      62 cevap = JOptionPane.showConfirmDialog(null,
           64      "Doğum gününüz bu numaralarda mı?" + set5);
           65

set5 içinde mi? 66      eğer(answer == JOptionPane.YES_OPTION)
           67      gün += 16;
           68      JOptionPane.showMessageDialog(null, "Doğum gününüz " +
           69      gün + "!");
           70 }n71
           }
```

Program, kullanıcının bir sayının Set1'de (satır 38), Set2'de (satır 44), Set3'te (satır 50), Set4'te (satır 56) ve Set5'te (satır 62) olup olmadığını yanıtlaması için onay iletişim kutuları gösterir. Cevap Evet ise, set içindeki ilk sayı **gün** (satırlar 42, 48, 54, 60 ve 66) eklenir.



FIGÜ3.8(a)'da Evet, (b)'de Evet, (c)'de Hayır, (d)'de Hayır ve (e)'de Evet'e tıklayın.

3.43 Onaylama iletişim kutusunu nasıl görüntölüyorsunuz? Çağrıldığında hangi değer döndürülür
[JOptionPane.showConfirmDialogt](#)

Nokta kontrol
 MyProgrammingLab™

3.19 Hata Ayıklama

Hata ayıklama, bir programdaki hataları bulma ve düzeltme sürecidir.

Anahtar
 Nokta

Bölüm 1.11,1'de belirtildiği gibi, sözdizimi hataları bulması ve düzeltmesi kolaydır çünkü derleyici hataların nereden geldiğine ve neden orada olduğuna dair ipuçları verir. Çalışma zamanı hataları bulmak da zor değildir, çünkü Java yorumlayıcısı program sona erdiğinde bunları konsolda gösterir. Öte yandan, mantık hatalarını bulmak oldukça zorlayıcı olabilir.

Mantık hatalarına *hata* denir. Hataları bulma ve düzeltme sürecine debugging denir. Hata ayıklamanın yaygın bir yaklaşımı, hatanın bulunduğu programın kısmını belirlemeye yardımcı olmak için bir dizi yöntem kullanmaktır. Programı *el ile izleyebilir* (yani, programı okuyarak hataları yakalayabilirsiniz) veya değişkenlerin değerlerini veya programın yürütme akışını göstermek için print ifadeleri ekleyebilirsiniz. Bu yaklaşımlar kısa, basit bir programı hata ayıklamak için işe yarayabilir, ancak büyük, karmaşık bir program için en etkili yaklaşım bir hata ayıklayıcı yardımcı programı kullanmaktır.

hatalar
 hata ayıklama
 elle izleme

JDK, bir sınıf adı ile çağrılan bir komut satırı hata ayıklayıcısı olan jdb'yi içerir. jdb, kendi Java yorumlayıcısının bir kopyasını çalıştıran bir Java programıdır. Eclipse ve NetBeans gibi tüm Java IDE araçları, entegre hata ayıklayıcılar içerir. Hata ayıklayıcı yardımcı programları, bir programın yürütülmesini takip etmenizi sağlar. Sistemden sisteme değişiklik gösterirler, ancak hepsi aşağıdaki yararlı özelliklerin çoğunu destekler.

- Tek bir ifadeyi bir seferde yürütmek:Hata ayıklayıcı, her ifadenin etkisini görebilmeniz için bir ifadeyi bir seferde yürütmenize olanak tanır.
- Bir metoda adım atmak:Eğer bir metod yürütülüyorsa, hata ayıklayıcıdan metoda girmesini ve metod içinde bir ifadeyi bir seferde yürütmesini isteyebilir veya tüm metodu atlayarak geçmesini isteyebilirsiniz. Metodun çalıştığını biliyorsanız, tüm metodu atlamalısınız. Örneğin, her zaman sistem tarafından sağlanan metodları, örneğin `System.out.println`, atlayın.

- Kesme noktası ayarlama: Belirli bir ifadede de bir kesme noktası ayarlayabilirsiniz. Programınız bir kesme noktasına ulaştığında durur. İstediğiniz kadar kesme noktası ayarlayabilirsiniz. Kesme noktaları, programlama hatanızın nerede başladığını bildiğinizde özellikle yararlıdır. O ifadede bir kesme noktası ayarlayabilir ve programın kesme noktasına ulaşana kadar çalışmasını sağlayabilirsiniz.
- Değişkenleri görüntüleme:Hata ayıklayıcı, birkaç değişkeni seçmenize ve değerlerini görüntülenize olanak tanır. Bir programı izlerken, bir değişkenin içeriği sürekli olarak güncellenir.
- Yöntem çağrılarını izleme:Hata ayıklayıcı, tüm yöntem çağrılarını izlemenize olanak tanır. Bu özellik, program yürütme akışının büyük bir resmini görmek istediğinizde faydalıdır.
- Değişkenleri değiştirme:Bazı hata ayıklayıcılar, hata ayıklama sırasında bir değişkenin değerini değiştirmenize olanak tanır. Bu, farklı örneklerle bir programı test etmek istediğinizde ama hata ayıklayıcıdan çıkmak istemediğinizde kullanışlıdır.

IDE'de hata ayıklama



Ttıtıp

Eclipse veya NetBeans gibi bir IDE kullanıyorsanız, lütfen *Learning Java Effectivelywith Eclipse/NetBeans'* ye Companion Web Sitesi'ndeki Supplements II.C ve II.E bölümünde bakın. Ek, bir hata ayıklayıcı kullanarak programları nasıl izleyebileceğinizi ve hata ayıklamanın Java'yı etkili bir şekilde öğrenmenize nasıl yardımcı olabileceğini gösterir.

Anahtar Terimler

Boolean ifadesi 82	akış diyagramı 84
boolean veri türü 82	format belirtici 113
Boolean değeri 82	operatör bağlayıcılığı 116
koşullu operatör 104	operatör önceliği 116
belirsiz else durumu 94	seçim ifadesi 82
hata ayıklama 119	kısa devre operatörü 104
devam etme davranışı 109	

BÖLÜM ÖZETİ

1. Bir **boolean** türü değişken, **true** veya **false** değerini saklayabilir.
2. İlişkisel operatörler (<, <=, ==, !=, >, >=) sayılar ve karakterlerle çalışır ve Boolean değeri üretir.
3. Boolean operatörleri **&&**, **|**, **!** ve **^** Boolean değerleri ve değişkenleri ile çalışır.
4. p1 && p2 değerlendirildiğinde, Java önce p1'i değerlendirir ve ardından p2'yi değerlendirir eğer p1 true ise; eğer p1 false ise, p2'yi değerlendirmez. p1 || p2 değerlendirildiğinde, Java önce p1'i değerlendirir ve ardından p2'yi değerlendirir eğer p1 false ise; eğer p1 true ise, p2'yi değerlendirmez. Bu nedenle, && koşullu veya kısa devre AND operatörü olarak adlandırılır ve || koşullu veya kısa devre OR operatörü olarak adlandırılır.
5. *Seçim ifadeleri* alternatif eylem yolları ile programlama için kullanılır. Birçok türde seçim ifadesi vardır: **if** ifadeleri, iki yönlü **if-else** ifadeleri, iç içe **if** ifadeleri, çok yönlü **if-else** ifadeleri, **switch** durum ifadeleri ve koşullu ifadeler.

- Çeşitli **if** ifadeleri, bir *Boolean ifadesine* dayalı kontrol kararları alır. İfadenin **true** veya **false** değerlendirmesine dayanarak, bu ifadeler iki olası yoldan birini alır.
- switch** ifadesi,
`char, byte, short, int` veya `String` türündeki bir switch ifadesine dayalı kontrol kararları alır.
- break** anahtar kelimesi, bir switch ifadesinde isteğe bağlıdır, ancak genellikle her durumun sonunda switch ifadesinin geri kalanını atlamak için kullanılır. Eğer break ifadesi yoksa, bir sonraki case ifadesi çalıştırılacaktır.
- İfadelerdeki operatörler, parantezler, *operatör önceliği* ve *operatör bağlayıcılığı* kurallarına göre belirlenen sırada değerlendirilir.
- Parantezler, değerlendirme sırasını herhangi bir sırada zorlamak için kullanılabilir.
- Daha yüksek önceliğe sahip operatörler daha önce değerlendirilir. Aynı önceliğe sahip operatörler için, bağlayıcılıkları değerlendirme sırasını belirler.
- Tüm ikili operatörler, atama operatörleri hariç, soldan sağa ilişkilidir; atama operatörleri sağdan sola ilişkilidir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinden yapın.

PROGRAMLAMA EGZERSİZLERİ



Petdagtotgtictal Ntote

Her bir araştırma için, kodlamadan önce problem gereksinimlerini dikkatlice analiz edin ve problemleri çözmek için stratejiler tasarlayın.

MyProgrammingLab™

Kodlamadan önce düşün



Hata Ayıklama İpuçları

Yardıma istemeden önce, programı kendinize okuyun ve açıklayın, ardından birkaç temel silci girdi ile el ile veya bir IDE hata ayıklayıcı kullanarak izleyin. Kendi hatalarınızı hata ayıklayarak programlamayı öğrenirsiniz.

hatalardan öğren

Bölüm 3.2

- *3.1** (Cebir: ikinci dereceden denklemleri çöz) İkinci dereceden bir denklemin iki kökü aş
 $ax^2 + bx + c = 0$ ağıdaki formül kullanılarak elde edilebilir:

$$r_1 = \frac{t - t_1 t_2 + t_1^2 t_2 t_3 - t_1^3 t_2^2 t_3}{\text{gelmek}} \quad \text{ver}_2 = \frac{-t_1 t_2 - t_1^2 t_2 t_3 - t_1^3 t_2^2 t_3}{\text{gelmek}}$$

bt $2t - t - 4tac$ İkinci dereceden denklemin ayırmıcısı olarak adlandırılır. Pozitif ise, denklemin iki gerçek kökü vardır. Sıfır ise, denklemin bir kökü vardır. Negatif ise, denklemin gerçek kökü yoktur.

Kullanıcıdan a, b ve c için değerler girmesini isteyen ve ayrımcıya göre sonucu gösteren bir program yazın. Ayrımcı pozitifse, iki kökü gösterin. Ayrımcı 0 ise , bir kökü gösterin. Aksi takdirde, “Denklemin gerçek kökü yoktur” mesajını gösterin.

Burada `Math.pow(x, 0.5)` kullanarak hesaplayabileceğinizi unutmayın. İşte bazı örnek-
çalışmalar.

İşte bazı örnek-



a, b, c girin:
Kökler -0.381966 ve -2.61803'tür



a, b, c girin:
Kök -1'dir



a, b, c girin:
Denklemin gerçek kökleri yoktur

3.2 (Oyun: üç sayıyı ekle) Liste 3.1'deki program iki tamsayı üretir ve kullanıcıdan bu iki tamsayının toplamını girmesini ister. Programı gözden geçirerek aklınızda kalmayan üç tamsayı üretin ve kullanıcıdan bu üç tamsayının toplamını girmesini isteyin.

Bölümler 3.3-3.8

***3.3** (Cebir: çöz 2×2 doğrusal denklemler) Cramer kuralını kullanarak çözebilirsiniz sonraki 2×2 doğrusal denklem sistemi:

$$\begin{array}{rcl} axt + t byt = t & \text{edt - t dostum} & ft - t ec \\ encxt + t dyt = t fxt = & adt - t bc & y = na \end{array}$$

Bir program yazın, kullanıcıdan a, b, c, d, e ve f girmesini isteyin ve sonucu gösterin. Eğer $adt - t bc$ 0 ise, "Denklemin çözümü yoktur" rapor edin.



a, b, c, d, e, f girin:
x -2.0 ve y 3.0'dır



a, b, c, d, e, f girin:
Denklemin çözümü yok

****3.4** (Oyun: toplama öğren) 100'ün altında iki tamsayı üreten ve kullanıcıdan bu iki tamsayının toplamını girmesini isteyen bir program yazın. Program daha sonraki cevap doğruysa doğru, aksi takdirde yanlış olarak bildirir. Program, Liste 3.1'e benzer.

***3.5** (Gelecek tarihleri bul) Kullanıcının, haftanın gününü belirtmek için bir tam sayı girmesini isteyen bir program yazın (bugün Pazar 0, Pazartesi 1, . . . , ve Cumartesi 6'dır). Ayrıca kullanıcıdan bugünden itibaren kaç gün sonra bir gelecek gün girmesini isteyin ve gelecek haftanın gününü gösterin. İşte bir örnek çalışma:



Bugünün gününü girin:
Bugünden itibaren geçen gün sayısını girin:
Bugün Pazartesi ve gelecek gün Perşembe

Bugünün gününü girin: 0

Bugünden itibaren geçen gün sayısını girin: 31

Bugün Pazar ve gelecek gün Çarşamba



- *3.6** (*Sağlık uygulaması: BMI*) Liste 3.5'i, ComputeAndInterpretBMI.java, kullanıcının ağırlığı, ayakları ve inçleri girmesine izin verecek şekilde gözden geçirin. Örneğin, bir kişi 5 ayak ve 10 inçse, ayak için 5 ve inç için 10 gireceksiniz. İşte bir örnek çalışma:

Ağırlığı pound cinsinden girin: 140

Ayakları girin: 5

İnçleri girin: 10

BMI 20.087702275404553

Normal



- 3.7** (*Finans uygulaması: para birimleri*) Liste 2.10, ComputeChange.java'yı, yalnızca sıfır olmayan nominal değerleri gösterecek şekilde değiştirin, 1 dolar ve 1 kuruş gibi tekil kelimeler kullanarak ve 2 dolar ve 3 kuruş gibi birden fazla birim için çoğul kelimeler kullanarak.



VideoNotu

Üç tamsayıyı sırala

- *3.8** (*Üç tam sayıyı sırala*) Üç tam sayıyı sıralayan bir program yazın. Tam sayılar, girdi diyaloglarından girilir ve sırasıyla num1, num2 ve num3 değişkenlerinde saklanır. Program, sayıları şu şekilde sıralar: *sayı1t ...t sayı2t ...t sayı3.*

- **3.9** (*İş: ISBN-10 kontrolü*) Bir ISBN-10 (Uluslararası Standart Kitap Numarası) 10 rakamdan oluşur: $dt1dt2dt3dt4dt5dt6dt7dt8dt9dt10$. Son rakam, $dt10t$, diğer dokuz rakamdan aşağıdaki formülü kullanarak hesaplanan bir kontrol toplamıdır:

$$(dt1t * 1t + dt2t * 2t + dt3t * 3t + dt4t * 4t + dt5t * 5t + dt6t * 6t + dt7t * 7t + dt8t * 8t + dt9t * 9t) \% 11n$$

Checksum 10 ise, son rakam ISBN-10 kuralına göre X ile gösterilir. Kullanıcıdan ilk 9 rakamı girmesini isteyen ve 10 haneli ISBN'yi (ön sıfırlar dahil) gösteren bir program yazın. Programınız girişi bir tamsayı olarak okumalıdır. İşte örnek çalıştırmalar:

Bir ISBN'nin ilk 9 rakamını tamsayı olarak girin: 013601267

ISBN-10 numarası 0136012671



Bir ISBN'nin ilk 9 rakamını tamsayı olarak girin: 013031997

ISBN-10 numarası 013031997X



- 3.10** (*Oyun: toplama testi*) Liste 3.4, SubtractionQuiz.java, rastgele bir çıkarma sorusu oluşturur. Programı, 100'den küçük iki tamsayı ile rastgele bir toplama sorusu oluşturacak şekilde revize edin.

Bölümler 3.9–3.19

- *3.11** (*Bir ay içindeki gün sayısını bul*) Kullanıcıdan ay ve yılı girmesini isteyen ve ay içindeki gün sayısını gösteren bir program yazın. İçin

örneğin, kullanıcı ay 2 ve yıl 2012 girdiğinde, program

Şubat 2012'nin 29 gün sürdüğünü göstermelidir. Eğer kullanıcı ay 3 ve yıl 2015 girdiğinde, program Mart 2015'in 31 gün sürdüğünü göstermelidir.

3.12 (*Bir sayıyı kontrol et*) Kullanıcıdan bir tam sayı girmesini isteyen ve sayının hem 5'e hem de 6'ya, ya da ikisinden hiçbirisiyle, ya da sadece

birisiyle bölünüp bölünmediğini kontrol eden bir program yazın. İşte bazı örnek çalışmaları 10.30 ve 23 için.

10, 5 veya 6'ya bölünebilir, ancak her ikisine de bölünemez
30, hem 5'e hem de 6'ya bölünebilir
23, ne 5'e ne de 6'ya bölünemez

***3.13** (*Finansal uygulama: vergileri hesaplama*) Liste 3.6, ComputeTax.java, tek dosya için vergileri hesaplamak üzere kaynak kodunu verir. Liste 3.6'yı tamamlayarak tam kaynak kodunu verin.

3.14 (*Oyun: yazı mı tura mı*) Kullanıcının bir paranın atılışının yazı mı tura mı olduğunu tahmin etmesine izin veren bir program yazın. Program rastgele bir tam sayı 0 veya 1 üretir, bu yazı veya tura anlamına gelir. Program, kullanıcının bir tahmin girmesini ister ve bu tahminin doğru mu yanlış mı olduğunu bildirir.

****3.15** (*Oyun: piyango*) Liste 3.9, Lottery.java'yı, üç basamaklı bir piyango oluşturacak şekilde gözden geçirin. Program, kullanıcının üç basamaklı bir sayı girmesini ister ve kullanıcının aşağıdaki kurallara göre kazanıp kazanmadığını belirler:

1. Kullanıcı girişi, piyango numarasıyla tam olarak eşleşirse, ödül 10,000\$'dır.
2. Kullanıcı girişindeki tüm rakamlar, piyango numarasındaki tüm rakamlarla eşleşirse, ödül 3,000\$'dır.
3. Kullanıcı girişi içindeki bir rakam, piyango numarasındaki bir rakamla eşleşirse, ödül 1.000 \$'dır.

3.16 (*Rastgele karakter*) `Math.random()` metodunu kullanarak rastgele bir büyük harf gösteren bir program yazın.

***3.17** (*Oyun: makas, taş, kağıt*) Popüler makas-taş-kağıt oyununu oynayan bir program yazın. (Bir makas kağıdı kesebilir, bir taş makası devirebilir ve bir kağıt taşı sarabilir.) Program rastgele bir sayı 0,1 veya 2 üretir; bu sayılar makas, taş ve kağıdı temsil eder. Program, kullanıcının 0,1 veya 2 ve kullanıcının mı yoksa bilgisayarın mı kazandığını, kaybettiğini belirten bir mesaj görüntüler, veya çizer. İşte örnek çalıştırmalar:



makas (0), taş (1), kağıt (2):
Bilgisayar makas. Sen taşsın. Kazandın

1 Enter



makas (0), taş (1), kağıt (2):
Bilgisayar kağıttır. Sen de kağıtsın. Beraberlik.

2 Enter

***3.18** (*Girdi iletişim kutusunu kullanın*) Girdi iletişim kutusunu kullanarak Liste 3.8, LeapYear.java'yı yeniden yazın.

****3.19** (*Bir üçgenin çevresini hesaplama*) Üç kenarı okuyan ve girdi geçerliyse çevreyi hesaplayan bir program yazın. Aksi takdirde, girdinin geçersiz olduğunu gösterin. Girdi, her iki kenarın toplamı kalan kenardan büyüğe geçerlidir.

***3.20** (Bilim: rüzgar-soğutma sıcaklığı) Programlama Alıştırması 2.17, rüzgar-soğutma sıcaklığını hesaplamak için bir formül verir. Formül,

— t58°F ile 41°F arasındaki sıcaklıklar ve rüzgar hızı 2 veya daha büyük olduğunda geçerlidir.

Kullanıcıdan bir sıcaklık ve bir rüzgar hızı girmesini isteyen bir program yazın. Program, girdi geçerliyse rüzgar-soğutma sıcaklığını gösterir; aksi takdirde, girdi geçersizse sıcaklık ve/veya rüzgar hızının geçersiz olduğunu belirten bir mesaj gösterir.

Kapsamlı

****3.21** (Bilim: haftanın günü) Zeller'in uyumu, haftanın gününü hesaplamak için Christian Zeller tarafından geliştirilen bir algoritmadır. Formül

$$h = t \text{ tq}t + \frac{26(t \text{ mt} + t \text{ 1})}{10} + kt + t \text{ k} \overline{4}t + \overline{5}t \overline{4}t + t \text{ 5}t \overline{t}t \% 7$$

şu şekildedir

■ **h** haftanın günü (0: Cumartesi, 1: Pazar, 2: Pazartesi, 3: Salı, 4: Çarşamba, 5: Perşembe, 6: Cuma).

■ **q** ayın günüdür.

■ **m** aydır (3: Mart, 4: Nisan, . . . , 12: Aralık). Ocak ve Şubat önceki yılın 13 ve 14. ayları olarak sayılır.

■ **j** yüzyıldır (yani,). $\frac{y \text{ l}}{100}$

■ **k** yüzyılın yılını ifade eder (yani, $y \text{ l} \% 100$).

Formüldeki bölme işleminin tam sayı bölmesi yaptığını unutmayın. Kullanıcıdan bir yıl, ay ve ayın gününü girmesini isteyen bir program yazın ve haftanın gününün adını gösterin. İşte bazı örnek çalıştırmalar:

Yılı girin: (örneğin, 2012):
 Ayı girin: 1-12:
 Ayın gününü girin: 1-31:
 Haftanın günü Pazar



Yılı girin: (örneğin, 2012):
 Ayı girin: 1-12:
 Ayın gününü girin: 1-31:
 Haftanın günü Cumartesi



(İpucu: Ocak ve Şubat formülde 13 ve 14 olarak sayıldığından, kullanıcı girişi 1'i 13'e ve 2'yi 14'e dönüştürmeniz ve yılı bir önceki yıla değiştirmanız gerekiyor.)

****3.22** (Geometri: bir dairedaki nokta mı?) Kullanıcıdan bir

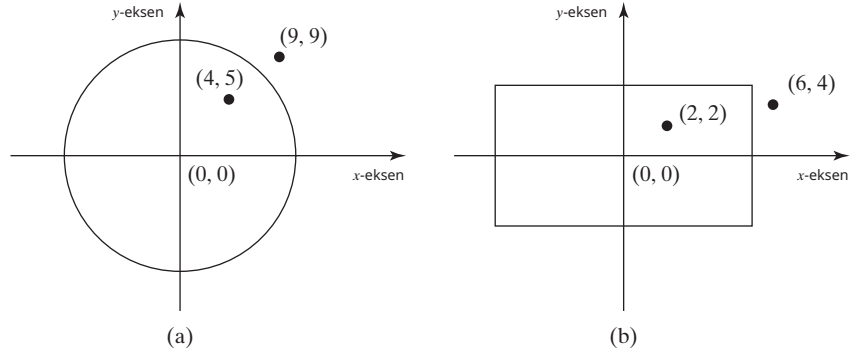
nokta (x,y) girmesini isteyen ve noktanın (0,0) merkezli dairenin içinde olup olmadığını kontrol eden bir program yazın.

Yarıçap 10. Örneğin, (4,5) dairenin içindedir ve (9,9) dairenin dışındadır, bu Şekil 3.9a'da gösterilmiştir.



VideoNotu

Kontrol noktası konumu

**Fiçin3.9**

dikdörtgen.

(a) Dairenin içindeki ve dışındaki noktalar. (b) Dairenin içindeki ve dışındaki

(İpucu: Bir nokta, (0,0) noktasına olan uzaklığı 10'dan küçük veya eşitse çemberin içinde dir. Uzaklığı hesaplamak için formül $2t(txt-2t-txt1t)2t+t(tyt2t-tyt1t)t2t$. Test etme pro- tüm durumları kapsamak için gram.) Aşağıda iki örnek çalıştırma gösterilmektedir.



İki koordinatla bir nokta girin:
Nokta (4.0, 5.0) dairenin içindedir

4 5 Enter



İki koordinatla bir nokta girin:
Nokta (9.0, 9.0) dairenin içinde değil

9 9 Enter

****3.23**

(Geometri: bir dikdörtgende nokta mı?) Kullanıcıdan bir

nokta (x, y) girmesini isteyen ve noktanın (0, 0) genişliği 10 ve yüksekliği 5 olan. Örneğin, (2, 2) dikdörtgenin içindedir ve (6, 4) dikdörtgenin dışındadır, Şekil 3.9b'de gösterildiği gibi. (İpucu: Bir nokta, (0,0) noktasına olan yatay mesafesi 10 / 2'ye eşit veya daha küçük ve dikey mesafesi 5.0 / 2'ye eşit ve ya daha küçükse dikdörtgenin içindedir. Programınızı tüm durumları kapsayacak şekilde test edin.) İşte iki örnek çalıştırma.



İki koordinatla bir nokta girin:
Nokta (2.0, 2.0) dikdörtgenin içindedir

2 2 Enter



İki koordinatla bir nokta girin:
Nokta (6.0, 4.0) dikdörtgenin içinde değil

6 4 Enter

****3.24**

(Oyun: bir kart seç) Bir deste karttan bir kart seçmeyi simüle eden bir program yazın

52 kartlar. Programınız, rütbeyi göstermelidir (As, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Kız, Kral) ve kartın takımı (Karo, Sinek, Kupa, Maça)

İşte programın bir örnek çalışması:



Seçtiğiniz kart Kupa Vale'si.

***3.25**

(Geometri: kesişim noktası) 1. doğruadaki iki nokta (x1,y1) ve (x2,

y2) olarak verilmiştir ve 2. doğruadaki noktalar (x3,y3) ve (x4,y4) olarak verilmiştir, Şekil 3.10a-b'de gösterildiği gibi.

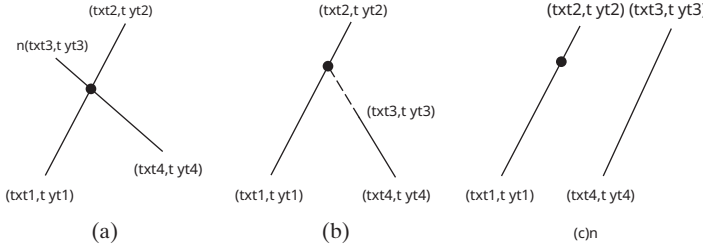


FIGURE 3.10 İki çizgi (a ve b) kesişiyor ve iki çizgi (c) paraleldir.

İki doğrunun kesişim noktası aşağıdaki doğrusal denklemi çözerek bu lunabilir:

$$yt1t - t yt2t)txt - t (txt 1t - t xt 2t)tyt = t (tyt1t - t yt2t)txt 1t - t (txt 1t - t xt 2t)tyt1$$

$$(tyt3t - t yt4t)txt - t (txt 3t - t xt 4t)tyt = t (tyt3t - t yt4t)txt 3t - t (txt 3t - t xt 4t)tyt3n(t$$

Bu doğrusal denklem Cramer kuralı kullanılarak çözülebilir (bkz. Alıştırma 3.3). Eğer denklem çözüm bulamıyorsa, iki doğru paraleldir (Şekil 3.10c). Kullanıcının dört nokta girmesini isteyen ve kesişim noktasını gösteren bir program yazın. İşte örnek çalıştırmalar:

X1, y1, x2, y2, x3, y3, x4, y4 girin: 2 2 5 -1.0 4.0 2.0 -1.0 -2.0 ↵ Enter
Kesişen nokta (2.88889, 1.1111) konumundadır.



X1, y1, x2, y2, x3, y3, x4, y4 girin: 2 2 7 6.0 4.0 2.0 -1.0 -2.0 ↵ Enter
İki çizgi paraleldir.

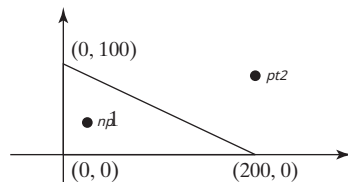


3.26 (&&, // ve ^ operatörlerini kullanın) Kullanıcının bir tam sayı girmesini isteyen ve bunun 5 ve 6'ya bölünüp bölünmediğini, 5 veya 6'ya bölünüp bölünmediğini ve 5 veya 6'ya bölünüp bölünmediğini belirleyen bir program yazın, fakat her ikisi de değil. İşte bu programın bir örnek çalışması:

Bir tamsayı girin: 10 ↵ Enter
10, 5 ve 6'ya bölünebilir mi? false
10, 5 veya 6'ya bölünebilir mi? true
10, 5 veya 6'ya bölünebilir, ama her ikisi de değil mi? true



****3.27** (Geometri: üçgendeki noktalar?) Aşağıda gösterildiği gibi bir düzlemde bir dik üçgen yerleştirildiğini varsayalım. Dik açı noktası (0, 0) konumuna yerleştirilmiştir ve diğer iki nokta (200, 0) ve (0, 100) konumlarına yerleştirilmiştir. Kullanıcıdan x- ve y-koordinatları ile bir nokta girmesini isteyen ve noktanın üçgenin içinde olup olmadığını belirleyen bir program yazın. İşte örnek çalıştırmalar:





Bir noktanın x- ve y-koordinatlarını girin:
Üçgenin içindedir.

100.5 25.5 Enter



Bir noktanın x- ve y-koordinatlarını girin:
Üçgenin içinde değildir.

100.5 50.5 Enter

****3.28**(Geometri: iki dikdörtgen) Kullanıcıdan iki dikdörtgenin merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girmesini isteyen bir program yazın ve ikinci dikdörtgenin birincinin içinde olup olmadığını veya birinci ile örtüşüp örtüşmediğini belirleyin, Şekil 3.11'de gösterildiği gibi. Programınızı tüm durumları kapsayacak şekilde test edin.

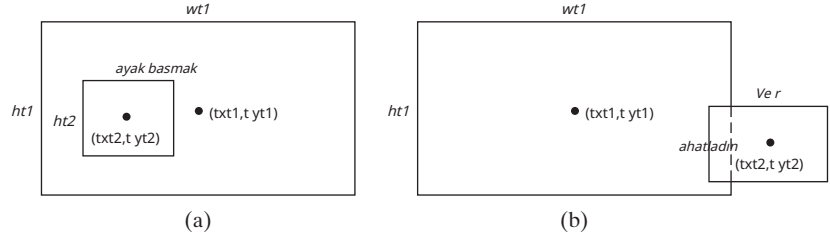


FIGURE 3.11 (a) Bir dikdörtgen diğ erinin içindedir. (b) Bir dikdörtgen diğ erinin üzerine gelmektedir.

İşte örnek çalıştırmalar:



r1'in merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2'nin merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2, r1'in içindedir

2.5 4 2.5 43 Enter

1.5 5 0.5 3 Enter



r1'in merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2'nin merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2, r1 ile örtüşmektedir

1 2 3 5.5 Enter

3 4 4.5 5 Enter

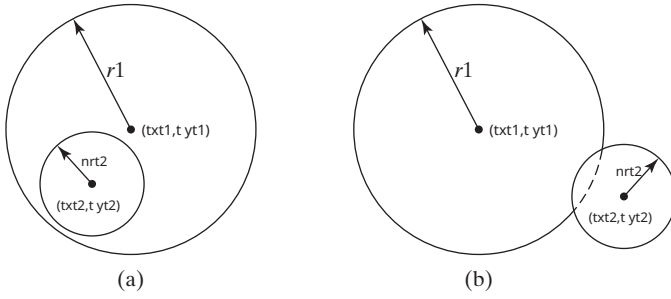


r1'in merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2'nin merkez x-, y-koordinatlarını, genişliğini ve yüksekliğini girin:
r2, r1 ile örtüşmemektedir

1 2 3 3 Enter

40 45 3 2 Enter

****3.29**(Geometri: iki daire) Kullanıcıdan iki dairenin merkez koordinatlarını ve yarıçaplarını girmesini isteyen ve ikinci dairenin birincinin içinde olup olmadığını veya birincisiyle örtüşüp örtüşmediğini belirleyen bir program yazın, Şekil 3.12'de gösterildiği gibi. (İpucu: daire2, daire1'in içindedir eğer iki merkez arasındaki mesafe $\leq |r1 - r2|$ ve daire2, daire1 ile örtüşmektedir eğer iki merkez arasındaki mesafe $\leq r1 + r2$. Programınızı tüm durumları kapsayacak şekilde test edin.)



ŞEKİL 3.12 (a) Bir çember başka bir çemberin içindedir. (b) Bir çember başka bir çemberle örtüşmektedir.

İşte örnek çalıştırmalar:

circle1'in merkez x-, y-koordinatlarını ve yarıçapını girin:
circle2'nin merkez x-, y-koordinatlarını ve yarıçapını girin:
çember2 çember1'in içindedir

0.5 5.1 13 ↵ Enter

1 1.7 4.5 ↵ Enter



circle1'in merkez x-, y-koordinatlarını ve yarıçapını girin:
circle2'nin merkez x-, y-koordinatlarını ve yarıçapını girin:
çember2 çember1 ile örtüşmektedir

3.4 5.7 5.5 ↵ Enter

6.7 3.5 3 ↵ Enter



circle1'in merkez x-, y-koordinatlarını ve yarıçapını girin:
circle2'nin merkez x-, y-koordinatlarını ve yarıçapını girin:
çember2 çember1 ile örtüşmemektedir

3.4 5.5 1 ↵ Enter

5.5 7.2 1 ↵ Enter



***3.30** (Mevcut zaman) Programlama Egzersizi 2.8'i 12 saatlik saat kullanarak saati gösterecek şekil de revize edin. İşte bir örnek çalışma:

GMT'ye olan saat dilimi farkını girin:
Mevcut zaman 4:50:34 AM

-5 ↵ Enter



***3.31** (Finans: döviz kuru) Kullanıcıdan ABD doları ile Çin RMB'si arasındaki döviz kurunu gir mesini isteyen bir program yazın. Kullanıcıdan ABD doları ile Çin RMB'si arasında dönüşüm yapmak için 0 girmesini ve Çin RMB'sinden ABD doları'na dönüşüm yapmak için 1 girmesini isteyin. Kullanıcıdan, sırasıyla Çin RMB'sine veya ABD doları'na dönüştürmek için ABD doları veya Çin RMB'si cinsinden miktarı girmesini isteyin. İşte örnek çalışmaları:

Dolar ile RMB arasındaki döviz kurunu girin:

6.81 ↵ Enter

Doları RMB'ye dönüştürmek için 0 girin ve tersine 1 girin:

0 ↵ Enter

Dolar miktarını girin:

100 ↵ Enter

100.0 \$ 681.0 yuan'dır





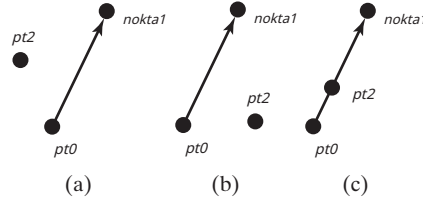
Dolar ile RMB arasındaki döviz kurunu girin: 6.81 ↵ Enter
 Doları RMB'ye dönüştürmek için 0 girin ve tersine 1 girin: 1 ↵ Enter
 RMB tutarını girin: 10000 ↵ Enter
 10000.0 yuan 1468.43 \$'dır



Dolar ile RMB arasındaki döviz kurunu girin: 6.81 ↵ Enter
 Doları RMB'ye dönüştürmek için 0 girin ve tersine 1 girin: 5 ↵ Enter
 Yanlış giriş

***3.32** (*Geometri: nokta konumu*) $p_0(x_0, y_0)$ noktasından $p_1(x_1, y_1)$ noktasına doğru bir yönlendirilmiş çizgi verildiğinde, $p_2(x_2, y_2)$ noktasının takimin solunda, sağında mı yoksa aynı çizgide mi olduğunu belirlemek için aşağıdaki koşulu kullanabilirsiniz (bkz. Şekil 3.13):

$$(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) \begin{cases} > 0 & p_2 \text{ çizginin sol tarafındadır} \\ = 0 & p_2 \text{ aynı çizgidedir} \\ < 0 & p_2 \text{ çizginin sağ tarafındadır} \end{cases}$$



FGÖRÜNTÜ 3.13

(a) p_2 , çizginin solundadır. (b) p_2 , çizginin sağındadır. (c) p_2 , üzerindedir.
 aynı satır.

Kullanıcıdan p_0 , p_1 ve p_2 için üç nokta girmesini isteyen ve p_2 'nin p_0 'dan p_1 'e gide n çizginin solunda, sağında mı yoksa aynı çizgide mi olduğunu gösteren bir program yazın. İşte bazı örnek çalıştırmalar:



p_0 , p_1 ve p_2 için üç nokta girin: 4.4 2 6.5 9.5 -5 4 ↵ Enter
 p_2 çizginin sol tarafındadır



p_0 , p_1 ve p_2 için üç nokta girin: 1 1 5 5 2 2 ↵ Enter
 p_2 aynı çizgidedir



p_0 , p_1 ve p_2 için üç nokta girin: 3.4 2 6.5 9.5 5 2.5 ↵ Enter
 p_2 , çizginin sağ tarafındadır

***3.33** (*Finans: maliyetleri karşılaştır*) Diyelim ki pirinç almak için iki farklı pakette alışveriş yapıyorsunuz. Bir maliyeti karşılaştırmak için bir program yazmak istersiniz. Program, kullanıcıdan her paketin ağırlığını ve fiyatını girmesini ister ve daha iyi fiyatı olanı gösterir. İşte bir örnek çalışma:

Paket 1 için ağırlık ve fiyat girin:
 Paket 2 için ağırlık ve fiyat girin:
 Paket 1 daha iyi bir fiyata sahip.

50 24.59 ↵ Enter
 25 11.99 ↵ Enter



*3.34

(Geometri: doğru parçasındaki nokta) Alıştırma 3.32, bir noktanın sınırsız bir doğru üzerinde olup olmadığını test etmenin yolunu gösterir. Alıştırma 3.32'yi, bir noktanın bir doğru parçasında olup olmadığını test edecek şekilde gözden geçirin. Kullanıcıdan p0, p1 ve p2 için üç noktayı girmesini isteyen ve p2'nin p0 ile p1 arasındaki doğru parçasında olup olmadığını gösteren bir program yazın. İşte bazı örnek çalıştırmalar:

p0, p1 ve p2 için üç nokta girin:
 (1.5, 1.5) (1.0, 1.0) ile (2.5, 2.5) arasındaki doğru parçasındadır

1 1 2.5 2.5 1.5 1.5 ↵ Enter



p0, p1 ve p2 için üç nokta girin:
 (3.5, 3.5) (1.0, 1.0) ile (2.0, 2.0) arasındaki doğru parçasında değildir

1 1 2 2 3.5 3.5 ↵ Enter



*3.35

(Onaltılıya dönüştür) Kullanıcıdan 0 ile 15 arasında bir tam sayı girmesini isteyen ve karşılık gelen onaltılı sayısını gösteren bir program yazın. İşte bazı örnek çalıştırmalar:

Bir ondalık değer girin (0 ile 15 arasında): 11 ↵ Enter
 Hex değeri B



Bir ondalık değer girin (0 ile 15 arasında): 5 ↵ Enter
 Hex değeri 5



Bir ondalık değer girin (0 ile 15 arasında): 31 ↵ Enter
 Geçersiz giriş



Bu sayfa kasıtlı olarak boş bırakılmıştır

BÖLÜM

4

LHATA

Hedefler

■ Bir while

döngüsü kullanarak ifadeleri tekrar tekrar çalıştırmak için programlar yazmak (§4.2).

■ Döngü tasarım stratejisini takip ederek döngüler geliştirmek (§§4.2.1–4.2.3).

■ Bir döngüyü bir sentinel değeri ile kontrol etmek (§4.2.4).

■ Klavye yerine girdi yönlendirmesi kullanarak bir dosyadan büyük girdi almak (§4.2.5).

■ Döngüleri **do-while** ifadeleri kullanarak yazmak (§4.3).

■ Döngüleri **for** ifadeleri kullanarak yazmak (§4.4).

■ Üç tür döngü durumunun benzerliklerini ve farklılıklarını keşfetmek (§4.5).

■ İç içe döngüler yazmak (§4.6).

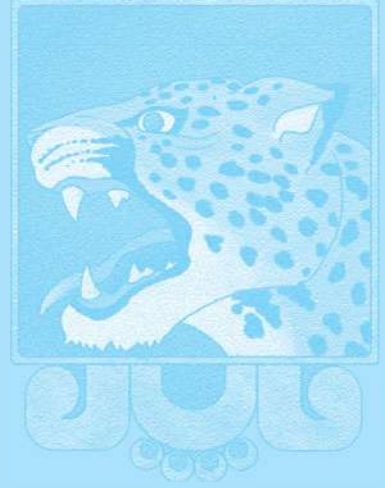
■ Sayısal hataları en aza indirme tekniklerini öğrenmek (§4.7).

■ Çeşitli örneklerden döngüleri öğrenmek (**GCD**, **FutureTuition**, **MonteCarloSimulation**) (§4.8).

■ Program kontrolünü **break** ve **continue** ile uygulamak (§4.9).

■ Asal sayıları görüntüleyen bir program yazmak (§4.10).

■ Bir onay penceresi ile döngüyü kontrol etmek (§4.11).



4.1 Giriş



Bir döngü, bir programın ifadeleri tekrar tekrar yürütmesini sağlamak için kullanılabilir.

sorun

Bir dizeyi (örneğin, **Java'ya hoş geldiniz!**) yüz kez görüntülemeniz gerektiğini ve r sayalım. Aşağıdaki ifadeyi yüz kez yazmak sıkıcı olurdu:

```
100 kez {
    System.out.println("Java'ya Hoş Geldiniz!");
    System.out.println("Java'ya Hoş Geldiniz!");
    ...
    System.out.println("Java'ya Hoş Geldiniz!");
}
```

döngü

Peki, bu sorunu nasıl çözersiniz?

Java, bir işlemin veya bir dizi işlemin ardışık olarak ne kadar süreyle gerçekleştirileceğini kontrol eden güçlü bir yapı olan *döngü* sağlar. Bir döngü ifadesi kullanarak, bilgisayara bir dizeyi yüz kez görüntülemesini söylersiniz, böylece print ifadesini yüz kez kodlamak zorunda kalmazsınız, şu şekilde:

```
inttcount = t0t;
while(count < 100) {
    System.out.println("Java'ya Hoş Geldiniz!");
    count++;
}
```

Değişken **count** başlangıçta **0**'dir. Döngü, **count < 100**'ün doğru olup olmadığını kontrol eder. Eğer öyleyse, döngü gövdesini çalıştırarak **Java'ya Hoş Geldiniz!** mesajını gösterir ve **count**'u **1** artırır. Bu, count < 100 yanlış olana kadar döngü gövdesini tekrar tekrar çalıştırır. count < 100 yanlış olduğunda (yani, count 100'e ulaştığında), döngü sona erer ve döngü ifadesinden sonraki ifade çalıştırılır.

Döngüler, bir dizi ifadenin tekrar eden yürütmelerini kontrol eden yapılar olarak tanımlanır. Döngü kavramı, programlamada temeldir. Java, üç tür döngü ifadesi sağlar:

while döngüleri, **do-while** döngüleri ve **for** döngüleri.

4.2 while Döngüsü



Bir while döngüsü, koşul doğru olduğu sürece ifadeleri tekrar tekrar yürütür.

while döngüsünün sözdizimi şudur:

```
while(döngü-devam-koşulu) {
    // Döngü gövdesi
    ifade(ler);
}
```

while döngüsü

döngü gövdesi

tekrarlama

döngü-devam-etme-koşulu

Şekil 4.1a, **while**-döngü akış diyagramını göstermektedir. Tekrar edilmesi gereken durumları içeren döngü kısmına döngü gövdesi denir. Bir döngü gövdesinin bir kez çalıştırılmasına yineleme (or tekrar) denir. Her döngü, gövdenin çalıştırılmasını kontrol eden bir Boolean ifade olan döngü-devam-etme-koşulu içerir. Bu, döngü gövdesinin çalıştırılıp çalıştırılmayacağını belirlemek için her seferinde değerlendirilir. Değerlendirmesi doğru ise, döngü gövdesi çalıştırılır; yanlış ise, tüm döngü sona erer ve program kontrolü **while** döngüsünü takip eden ifadeye geçer.

Döngü, **Java'ya Hoş Geldiniz!** ifadesini yüz kez görüntülemek için, önceki bölümde tanıtilen bir **while** döngüsüdür. Akış diyagramı Şekil 4.1b'de gösterilmiştir. Bu

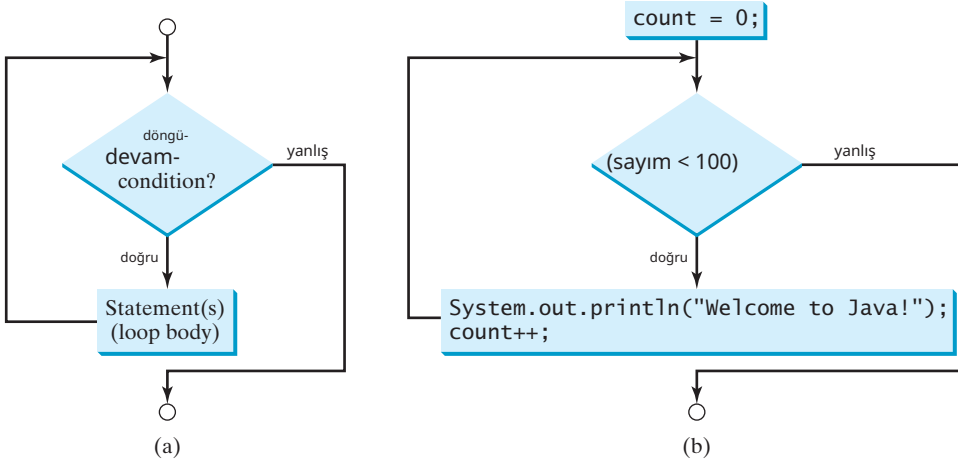


FIGURE 4.1 While döngüsü, döngü devam koşulu **true** olarak değerlendirildiğinde döngü gövdesindeki ifadeleri tekrar tekrar yürütür.

döngü-devam-etme-kosulu **count < 100** ve döngü gövdesi aşağıdaki iki ifadeyi içerir:

```

int count = 0;
while(count < 100) {
    System.out.println("Java'ya Hoş Geldiniz!");
    sayim++;
}
  
```

döngü-devam-etme-kosulu

döngü gövdesi

Bu örnekte, döngü gövdesinin kaç kez çalıştırılması gerektiğini tam olarak biliyorsunuz çünkü kontrol değişkeni **count**, çalıştırma sayısını saymak için kullanılıyor. Bu tür bir döngü, sayaç kontrollü döngü olarak bilinir.

sayı kontrollü döngü



Not

Döngü devam etme koşulu her zaman parantezlerin içinde yer almalıdır. Döngü gövdesini kapsayan süslü parantezler, döngü gövdesi bir

veya hiç ifade içermiyorsa atlanabilir.

Bir döngünün nasıl çalıştığını anlamaya yardımcı olacak başka bir örnek.

```

Intisam = T0T, E = T1T;N
ohaila(E < T10T) {Na
    sum = oruç + E;ni
    ++;
}
System.out.println("toplam "+ sum); // toplam 45
  
```

Eğer **i < 10** doğruysa, program **i**'yi **sum**'a ekler. Değişken **i** başlangıçta **1** olarak ayarlanır, sonra **2,3** ve **10**'a kadar artırılır. **i** 10 olduğunda, **i < 10** yanlış olur, böylece döngü sona erer.

Bu nedenle, toplam **1 + 2 + 3 + ... + 9 = 45**'dir.

Döngü yanlışlıkla aşağıdaki gibi yazılırsa ne olur?

```

Intisam = T0T, E = T1T;N
uhila(E < T10T) {Nas
    um = oruç + E;N}
  
```

Bu döngü sonsuzdur, çünkü `i` her zaman `1` ve `i < 10` her zaman `true` olacaktır.



Not

sonsuz döngü

Döngü devam koşulunun sonunda `false` olmasını sağlayın, böylece döngü sona erecektir. Yaygın bir programlama hatası, sonsuz döngüler ile ilgilidir (yani, döngü sonsuza kadar çalışır). Programınız alışılmadık derecede uzun bir süre çalışıyorsa ve durmuyorsa, sonsuz bir döngüye sahip olabilir. Programı komut penceresinden çalıştırıyorsanız, durdurmak için `CTRL+C` tuşlayın.



Dikkat

bir eksik hata

Programcılar genellikle bir döngüyü bir kez fazla veya az çalıştırma hatası yaparlar. Bu genellikle *bir fazladan hata* olarak bilinir. Örneğin, aşağıdaki döngü şunu gösterir: `Java'ya hoş geldiniz` 101 kez, 100 kez yerine. Hata koşuldadır, bu `count < 100` olmalıdır, `count <= 100` değil.

```
inttcount = t0t;nw
hile (sayım <= t100 ) {
    System.out.println("Java'ya hoş geldiniz!");
    count++;
}
```

Liste 3.1, AdditionQuiz.java, iki tek haneli sayının toplama sorusu için kullanıcıdan bir cevap girmeni isteyen bir programı verir. Bir döngü kullanarak, programı kullanıcıya doğru cevabı verene kadar yeni bir cevap girmesine izin verecek şekilde yeniden yazabilirsiniz; bu, Liste 4.1'de gösterilmiştir.

LİSTE 4.1 TekrarToplamaSınavı.java

oluşturnumber1
oluşturnumber2

soruyu göster

ilk cevabı al

cevabı kontrol et

bir cevabı oku

```
1  import java.util.Scanner;
2
3  public class TekrarToplamaSınavı {
4      public static voidtmain(String[] args) {
5          int sayı1 = (int)(Math.random() % 10);
6          int sayı2 = (int)(Math.random() % 10);
7
8          // Bir Scanner oluştur
9          Tarayıcı girişi = tnewScanner(System.in);
10
11         Sistem.çıkışı.baskı(
12             "Ne kadar" + number1 + " + " + number2 + "? ";
13         inttanswer = input.nextInt();
14
15         while () { tumbSystem.out.println("Cevap: " + sayı1 + " + " + sa
16             yi2 + "? ");
17
18             cevap = input.nextInt();
19         }
20
21         Sistem.out.println("Başardınız!");
22     }
23 }
```



5 + 9 nedir?

12 Enter

Yanlış cevap. Tekrar dene. 5 + 9 nedir?

34 Enter

Yanlış cevap. Tekrar dene. 5 + 9 nedir?

14 Enter

Başardın!

Döngü, 15-19. satırlarda kullanıcıdan bir `cevap` girmesini sürekli olarak isterken, `number1 + number2 != cevap doğru olduğunda çalışır`. `number1 + number2 != cevap yanlış olduğunda`, döngü çıkış yapar.

4.2.1 Vaka Çalışması: Sayıları Tahmin Etme

Sorun, bir bilgisayarın aklında hangi sayıyı tuttuğunu tahmin etmektir. 0 ile 100 arasında, dahil olmak üzere rastgele bir tam sayı üreten bir program yazacaksınız. Program, kullanıcıdan sayıyı sür ekli olarak girmesini ister, ta ki sayı rastgele üretilen sayı ile eşleşene kadar. Her kullanıcı girişi içi n, program kullanıcıya girişin çok düşük mü yoksa çok yüksek mi olduğunu söyler, böylece kullanıcı bir sonraki tahmini akıllıca yapabilir. İşte bir örnek çalışma:



VideoNotu

Bir sayı tahmin edin



```
0 ile 100 arasında bir sihirli sayı tahmin edin
Tahmininizi girin: 50 Enter
Tahmininiz çok yüksek
Tahmininizi girin: 25 Enter
Tahmininiz çok düşük
Tahmininizi girin: 42 Enter
Tahmininiz çok yüksek
Tahmininizi girin: 39 Enter
Evet, sayı 39
```

Sihirli sayı 0 ile 100 arasındadır. Tahmin sayısını en aza indirmek için, önce 50 giriniz. Eğer tahmininiz çok yüksekse, sihirli sayı 0 ile 49 arasındadır. Eğer tahmininiz çok düşükse, sihirli sayı 51 ile 100 arasındadır. Böylece, bir tahminden sonra sayıların yarısını kaldırabilirsiniz.

Bu programı nasıl yazarsınız? Hemen kodlamaya mı başlarsınız? Hayır. Kodlamadan önce düşünmek önemlidir. Problemi bir program yazmadan nasıl çözeceğinizi düşünün. Öncelikle 0 ile 100 arasında, dahil, rastgele bir sayı üretmeniz gerekir, ardından kullanıcıdan bir tahmin girmesini istemeniz ve tahmini rastgele sayı ile karşılaştırmanız gerekir.

Adım adım, kademeli olarak kod yazmak iyi bir uygulamadır. Döngüler içeren programlar için, döngüyü hemen nasıl yazacağınızı bilmiyorsanız, önce döngüyü bir kez çalıştıracak kodu yazabilir ve ardından kodu döngü içinde tekrar tekrar nasıl çalıştıracığınızı bulabilirsiniz. Bu program için, Şekil 4.2'de gösterildiği gibi bir ilk taslak oluşturabilirsiniz.

zekice tahmin

Kodlamadan önce düşün

kodları kademeli olarak yaz

LİSTE 4.2 TahminSayısıBirDefa.java

```
1 import java.util.Scanner;
2
3 public class TahminSayısıBirDefa {
4     public static void main(String[] args) {
5         // Tahmin edilecek rastgele bir sayı üret
6         int number = (int)(Math.random() * 101);
7
8         Tarayıcı girişi = new Scanner(System.in);
9         System.out.println("0 ile 100 arasında bir sihirli sayı tahmin edin");
10
11         // Kullanıcıdan sayıyı tahmin etmesini iste
12         System.out.print("Tahmininizi girin: ");
13         int tahmin = input.nextInt();
14
15         ve eğer(tahmin == sayı)
16         System.out.println("Evet, sayı " + number);
17         aksi takdirde eğer(tahmin > sayı)
18         System.out.println("Tahmininiz çok yüksek");
19         aksi takdirde
```

bir sayı oluştur

bir tahmin girin

doğru tahmin mi?

çok yüksek mi?

138 Bölüm 4 Döngüler

çok düşük mü?

```
20         System.out.println("Tahmininiz çok düşük");
21     }
    yirmi iki }
```

Bu programı çalıştırdığınızda, kullanıcıdan yalnızca bir kez tahmin girmesini ister. Kullanıcının tekrar tekrar tahmin girmesine izin vermek için, 11-20. satırlardaki kodu aşağıdaki gibi bir döngüye alabilirsiniz:

```
iken(true) {
    // Kullanıcıdan sayıyı tahmin etmesini isteSystem
    .out.print("Tahmininizi girin: ");guess = input.nextInt
    ();

    eğer(tahmin == sayı)
        System.out.println("Evet, sayı "+ sayı);
    aksi takdirde eğer(tahmin > sayı)
        System.out.println("Tahmininiz çok yüksek");
    aksi takdirde
        System.out.println("Tahmininiz çok düşük");
} // Döngünün sonu
```

Bu döngü, kullanıcıdan sürekli olarak bir tahmin girmesini ister. Ancak, bu döngü doğru değil, b çünkü asla sona ermez. tahmin sayı ile eşleştiğinde, döngü sona ermeli. Bu nedenle, döngü şu şekilde revize edilebilir:

```
iken(guess != number) {
    // Kullanıcıdan sayıyı tahmin etmesini isteSystem
    .out.print("Tahmininizi girin: ");guess = input.nextInt
    ();

    eğer(tahmin == sayı)
        System.out.println("Evet, sayı "+ sayı);
    aksi takdirde eğer(tahmin > sayı)
        System.out.println("Tahmininiz çok yüksek");
    aksi takdirde
        System.out.println("Tahmininiz çok düşük");
} // Döngünün sonu
```

Tam kod, Liste 4.3'te verilmiştir.

LISTE4.3TahminSayı.java

bir sayı oluştur

bir tahmin girin

```
1  import java.util.Scanner;
2
3
4  public classTahminSayı {
5      public static voidtmain(String[] args) {
6          // Tahmin edilecek rastgele bir sayı üret
7          inttnumber = (tintt)(Math.random() * t101t);
8
9          Scanner girdi = newScanner(System.in);
10         System.out.println("0 ile 100 arasında bir sihirli sayı tahmin edin");
11
12         intguess = -1;
13         iken(guess != number) {
14             // Kullanıcıdan sayıyı tahmin etmesini isteyin14
15             System.out.print("Tahmininizi girin: ");15         tahmin = girdi.ne
16             xtInt();
17
18             eğer(tahmin == sayı)
19                 System.out.println(" Evet, sayı "+ number);
20             aksi takdirde eğer(guess > number)
```

```

20      System.out.println("Tahmininiz çok yüksek");
21      aksi takdirde
22      System.out.println(" Tahmininiz çok düşük");
23      // Döngünün sonu
24 } n25
}n}

```

çok yüksek mi?
çok düşük mü?

	satır#	numara	tahmin et	çıktı
	6	39		
iterasyon 1	11		-1	
	15		50	
	20			Tahmininiz çok yüksek
	22			
iterasyon 2	15		25	
	22			Tahmininiz çok düşük
iterasyon 3	15		42	
	20			Tahmininiz çok yüksek
tekrarlama 4	15		39	
	18			Evet, sayı 39

Program, 6. satırda sihirli sayıyı üretir ve kullanıcıdan sürekli olarak bir tahmin girmesini ister (12-23. satırlar). Her tahminde, program tahminin doğru, çok yüksek veya çok düşük olup olmadığını kontrol eder (17-22. satırlar). Tahmin doğru olduğunda, program döngüden çıkar (12. satır). **tahmin -1 olarak başlatıldığını unutmayın. Bunu 0 ile 100 arasında bir değere başlatmak yanlış olur, çünkü bu tahmin edilmesi gereken sayı olabilir.**

4.2.2 Döngü Tasarım Stratejileri

Doğru bir döngü yazmak, acemi programcılar için kolay bir iş değildir. Bir döngü yazarken üç adımı göz önünde bulundurun.

Adım 1: Tekrar edilmesi gereken ifadeleri belirleyin.

Adım 2: Bu ifadeleri şu şekilde bir döngü içine alın:

```

while(true) {
    İfadeler;
}

```

Adım 3: **döngü-devam-etme-koşulunu** kodlayın ve döngüyü kontrol etmek için uygun ifadeleri ekleyin.

```

while(döngü-devam-etme-koşulu) {
    İfadeler;
    Döngüyü kontrol etmek için ek ifadeler;
}

```

4.2.3 Vaka Çalışması: Çoklu Çıkarma Sınavı

Liste 3.4'teki Matematik çıkarma öğrenme aracı programı, her çalıştırma için sadece bir soru üretir. Soruları tekrar tekrar üretmek için bir döngü kullanabilirsiniz. Beş soru üretmek için kodu nasıl yazarsınız? Döngü tasarım stratejisini izleyin. Öncelikle tekrarlanmasını gereken ifadeleri belirleyin. Bunlar, iki rastgele sayı elde etmek için ifadeler, isteme



VideoNotu

Birden fazla çıkarma sınavı

Kullanıcıya bir çıkarma sorusu sormak ve soruyu değerlendirmek. İkincisi, ifadeleri bir döngü içinde sarmak. Üçüncüsü, döngü kontrol değişkeni ve [döngü-devam-etme-koşulu](#) eklemek

döngüyü beş kez çalıştırmak için.

Liste 4.4, beş soru üreten bir programı verir ve bir öğrenci tümbeş soruyu yanıtladıktan sonra doğru cevap sayısını raporlar. Program ayrıca sınavda harcanan süreyi gösterir ve tüm soruları listeler.

LISTE4.4SubtractionQuizLoop.java

```

1  import java.util.Scanner;
2
3  genel sınıfSubtractionQuizLoop {
4      public static voidtmain(String[] args) {
5          final intNUMBER_OF_QUESTIONS = 5; // Soru sayısı
6          intdoğruSayısı = 0; // Doğru cevapların sayısını say
7          intcount = 0; // Soru sayısını say
8          longstartTime = System.currentTimeMillis();
9          String çıktı = " "; // çıktı dizesi başlangıçta boştur
10         Scanner girdi = newScanner(System.in);
11
12         // 1. İki rastgele tek haneli tam sayı oluşturun
13         intnumber1 = (int)(Math.random() * t10t);
14         intnumber2 = (int)(Math.random() * t10t);
15
16         // 2. Eğer number1 < number2 ise, number1'i number2 ile değiştirin
17         if(sayı1 < sayı2) {
18             inttemp = sayı1;
19             sayı1 = sayı2;
20             sayı2 = geçici;
21         }
22
23         // 3. Öğrenciyi "number1 - number2 nedir?" diye cevap vermesi için teşvik et
24         Sistem.çıkışı.baskı(
25             "Ne kadar " + sayı1 + " - " + sayı2 + "? ";
26             intanswer = input.nextInt();
27
28         // 4. Cevabı değerlendir ve sonucu göster
29         eğer(sayı1 - sayı2 == cevap) {
30             System.out.println("Doğru cevap verdiniz!");
31             doğruSayısı++; // Doğru cevap sayısını artır
32
33         aksi takdirde
34             System.out.println("Cevabınız yanlış." + sayı1 36 + " - " +
35             sayı2 + " şu olmalı " + (sayı1 - sayı2));
36
37         // Soru sayısını artır
38         sayım++;
39
40         çıktı += "" + sayı1 + "-" + sayı2 + "=" + cevap + 42 ((sayı1 - sayı2 == cevap) ?
41         " doğru" : " yanlış");
42     }
43
44     longbitişZamanı = System.currentTimeMillis();
45     longtestSüresi = bitişZamanı - başlangıçZamanı;
46
47     System.out.println("Doğru sayısı " + doğruSayım +
48     "Test süresi " + testSüresi / 1000+ " saniye" + çıktı);
49 }n51
50 }

```

başlangıç zamanını al

döngü

bir soru göster

bir cevabı değerlendir

doğru sayısını artır

kontrol değişkenini artır

çıktıyı hazırla

döngüyü sonlandır

bitiş zamanını al

test süresi

sonucu göster



```

9 - 2 nedir? 7 ↵ Enter
Doğru!

3 - 0 nedir? 3 ↵ Enter
Doğru!

3 - 2 nedir? 1 ↵ Enter
Doğru!

7 - 4 nedir? 4 ↵ Enter
Cevabınız yanlış.

7 - 4 3 etmelidir
7 - 5 nedir? 4 ↵ Enter

Cevabınız yanlış.
7 - 5 2 etmelidir

Doğru sayı 3'tür
Test süresi 1021 saniyedir

9-2=7 doğrudur
3-0=3 doğrudur
3-2=1 doğrudur
7-4=4 yanlıştır
7-5=4 yanlıştır

```

Program, döngünün yürütülmesini kontrol etmek için kontrol değişkenini **count** kullanır. **count** ilk olarak **0** (satır 7) olarak başlar ve her yinelemede **1** artırılır (satır 39). Her yinelemede bir çıkarma sorusu gösterilir ve işlenir. Program, test başlamadan önceki zamanı satır 8'de ve test bittikten sonraki zamanı satır 45'te alır ve test süresini satır 46'da hesaplar. Test süresi milisaniye cinsindendir ve satır 49'da saniyeye dönüştürülür.

4.2.4 Bir Gözetmen Değeri ile Döngüyü Kontrol Etme

Döngüyü kontrol etmenin bir diğer yaygın tekniği, bir dizi değeri okurken ve işlerken özel bir değer belirlemektir. Bu özel girdi değeri, *gözetmen değeri* olarak bilinir ve *girişi sonlandırır*. *Yürütmesini kontrol etmek için bir gözetmen değeri kullanan bir döngü, gözetmen kontrollü döngü olarak adlandırılır*.

sentinel değeri

sentinel kontrollü döngü

Liste 4.5, belirsiz sayıda tam sayıyı okuyan ve toplamını hesaplayan bir program yazar. Girdi **0**, girdinin sonunu belirtir. Her girdi değeri için yeni bir değişken bildirmeye ihtiyaç var mı? Hayır. Girdi değerini saklamak için sadece **data** (satır 12) adında bir değişken kullanın ve toplamı saklamak için **sum** (satır 15) adında bir değişken kullanın. Bir değer okunduğunda, onu **data**'ya atayın ve eğer sıfır değilse, **sum**'ya ekleyin (satır 17).

LISTE 4.5 SentinelValue.java

```

1  import java.util.Scanner;
2
3  genel sınıf SentinelValue {
4      /** Ana metod */
5      public static void main(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = new Scanner(System.in);
8
9          // Başlangıç verisini oku
10         System.out.print(
11             "Bir tam sayı girin (girdi 0 ise sona erer: ");
12         int data = input.nextInt();

```

giriş

```

1314 // Girdi 0 olana kadar verileri okumaya devam et
15 inttsum = t0t;
16 while(veri != t0t) {
17 toplam += veri;
18
19 // Sonraki veriyi oku
20 Sistem.çıkışı.baskı(
21 "Bir tam sayı girin (girdi 0 ise sona erer): ");
22 veri = input.nextInt();
2324 }
25 System.out.println("Toplam "+ sum);
asla {revize e
dilmedi}

```

döngü

döngü sonu

yazdır sonuç



Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Toplam 9'dur

2 Enter
 3 Enter
 4 Enter
 0 Enter



	satır#	veri	toplam	çıkıktı
	12	2		
	15		0	
iterasyon 1 {	17		2	
	22	3		
iterasyon 2 {	17		5	
	22	4		
iterasyon 3 {	17		9	
	22	0		
	25			Toplam 9'dur

Eğer **data0** değilse, **sum** 'a eklenir (satır 17) ve bir sonraki girdi verisi okunur (satırlar20–22). Eğer **data0** ise, döngü gövdesi artık çalıştırılmaz ve **while** döngüsü sona erer. Girdi değeri **0** bu döngü için sentinel değeridir. İlk okunan girdi **0** ise, döngü gövdesi asla çalışmaz ve sonuçta elde edilen toplam **0** olur.



Dikkat

Döngü kontrolünde eşitlik kontrolü için ondalıklı değerler kullanmayın. Çünkü ondalıklı değerler bazı değerler için yaklaşık değerlerdir, bunları kullanmak kesin olmayan sayaç değerlerine ve hatalı sonuçlara yol açabilir.

Aşağıdaki kodu $1 + 0.9 + 0.8 + \dots + 0.1$ hesaplamak için düşünün:

```

doubleitem = t1t;doubletsum = t0t;nwh
ile (öge != t0 ) { // item'in 0 olacağına dair bir garanti yok
    sum += item;
    öge -= t0.1t;
}
System.out.println(toplam);

```

Değişken **item** 1 ile başlar ve döngü gövdesi her çalıştırıldığında 0.1 azaltılır. Döngü, item 0 olduğunda sona ermeli. Ancak, item tam olarak 0 olacağına dair bir garanti yoktur, çünkü ondalıklı sayı aritmetiği yaklaşık olarak hesaplanmaktadır. Bu döngü yüzeyde iyi görünüyor, ancak aslında sonsuz bir döngüdür.

sayısal hata

4.2.5 Girdi ve Çıktı Yönlendirmeleri

Önceki örnekte, girmek için büyük miktarda veriniz varsa, klavyeden yazmak zahmetli olacaktır. Verileri boşluklarla ayrılmış olarak bir metin dosyasında saklayabilirsiniz, örneğin **input.txt**, ve programı aşağıdaki komutla çalıştırabilirsiniz:

```
java SentinelValue < input.txt
```

Bu komuta girdi yönlendirmesi denir. Program, kullanıcıdan verileri çalışma zamanında klavyeden yazdırmak yerine dosyadan girdi alır. Dosyanın içeriği şöyledir:

girdi yönlendirmesi

```
2 3 4 5 6 7 8 9 12 23 32
23 45 67 89 92 12 34 35 3 1 2 4 0
```

Program toplam değerini 518 olarak almalıdır.

Benzer şekilde, *çıkıı yönlendirmesi* vardır, bu da çıktıyı konsolda görüntülemek yerine bir dosyaya gönderir. Çıkıı yönlendirmesi için komut:

çıkıı yönlendirmesi

```
Java SınıfAdı > çıktı.txt
```

Girdi ve çıkıı yönlendirmesi aynı komutta kullanılabilir. Örneğin, aşağıdaki komut **input.txt** dosyasından girdi alır ve çıkııyı **output.txt** dosyasına gönderir:

```
java SentinelValue < input.txt > çıktı.txt
```

Programı çalıştırmayı deneyin ve **output.txt** dosyasında hangi içeriklerin olduğunu görün.

- 4.1** Aşağıdaki kodu analiz edin. **count < 100** her zaman doğru mu, her zaman yanlış mı, yoksa bazen doğru mu yoksa bazen yanlış mı Point A, Point B ve Point C'de?

```
int count = 0;
while(count < 100) {
    // Nokta A
    System.out.println("Java'ya hoş geldiniz!"); count++;
    // Nokta B
}
// Nokta C
```

Nokta kontrol
MyProgrammingLab™

- 4.2** Listing 4.3'te 11. satırda **guess0** olarak başlatıldığında ne yanlış?
- 4.3** Aşağıdaki döngü gövdesi kaç kez tekrarlanır? Her bir döngünün çıkıısı nedir?

```
int t1t;
while(i < t10t)nift(
    i % t2t==t0t)Syste
    m.out.println(i);
```

(a)

```
int t1t;
while(i < t10t)nift(
    i % t2t==t0t)Syste
    m.out.println(i++);
```

(b)

```
int t1t;
while(i < t10t)nift(
    (i++) % t2t==t0t)System.
    out.println(i);
```

(c)

4.4 Girdi 2 3 4 5 0 ise. Aşağıdaki kodun çıktısı nedir?

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner inp = new Scanner(System.in);
        int number, max;

        sayı = input.nextInt();
        max = sayı;

        while (sayı != 0) {
            number = input.nextInt();
            if (sayı > max)
                max = sayı;
        }

        System.out.println("max değeri " + max);
        System.out.println("number " + number);
    }
}
```

4.5 Aşağıdaki kodun çıktısı nedir? Sebebinizi açıklayın.

```
int x = 1;
while (x > 0) {
    x++;
}

System.out.println("x şudur " + x);
```

4.3 do-while Döngüsü



Bir **do-while** döngüsü, döngü gövdesini ilk olarak çalıştırıp ardından döngü devam etme koşulunu kontrol etmesi dışında bir **while** döngüsüne benzer.

do-while döngüsü, while döngüsünün bir varyasyonudur. Söz dizimi:

```
nokta{
    // Döngü gövdesi;
    ifade(ler);
}while(döngü-devam-etme-koşulu);
```

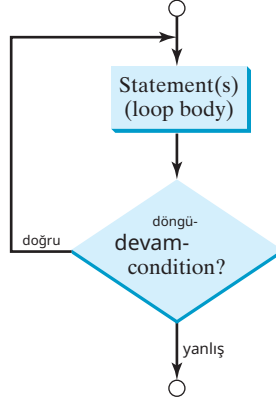
Döngünün akış diyagramı Şekil 4.2'de gösterilmiştir.

Döngü gövdesi ilk olarak çalıştırılır ve ardından **döngü-devam-etme-koşulu** değerlendirilir. Eğer değerlendirme **doğru** ise, döngü gövdesi tekrar çalıştırılır; eğer **yanlış** ise, **do-while** döngüsü sona erer. **while** döngüsü ile **do-while** döngüsü arasındaki fark, **döngü-devam-etme-koşulu** nun değerlendirildiği ve döngü gövdesinin çalıştırıldığı sıradır. İsterseniz döngüyü **while** döngüsü veya **do-while** döngüsü kullanarak yazabilirsiniz. Bazen biri diğerine göre daha uygun bir seçimdir. Örneğin, Liste 4.5'teki **while** döngüsünü, Liste 4.6'da gösterildiği gibi bir **do-while** döngüsü kullanarak yeniden yazabilirsiniz.

Liste 4.6 TestDoWhile.java

```
1 import java.util.Scanner;
2
3 genel sınıf TestDoWhile {
```

do-while döngüsü



ŞEKİL 4.2 do-while döngüsü önce döngü gövdesini çalıştırır, ardından döngü-devam-etme-koşulunu kontrol ederek döngünün devam edip etmeyeceğine karar verir.

```

4  /** Ana metod */
5  public static void main(String[] args) {
6      int veri;
7      int toplam = 0;
8
9      // Bir Scanner oluştur
10     Scanner girdi = new Scanner(System.in);
11
12     // Girdi 0 olana kadar verileri okumaya devam et
13     nokta:
14     // Sonraki veriyi oku
15     System.out.print(
16         "Bir tam sayı girin (girdi 0 ise sona erer): ");
17     veri = girdi.nextInt();
18
19     toplam += veri;
20     } while (veri != 0);
21
22     System.out.println("Toplam " + toplam);
23 }
24

```

döngü

döngüyü bitir

Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Bir tamsayı girin (giriş 0 ise sona erer):
 Toplam 14

3 Enter
 5 Enter
 6 Enter
 0 Enter



Ttipp

Döngü içinde en az bir kez çalıştırılması gereken ifadeler varsa, **do-while** döngüsünü kullanın, *önceki TestDoWhile* programındaki do-while döngüsünde olduğu gibi. Bu ifadeler, bir **while** döngüsü kullanıyorsanız, döngüden önce ve içinde görünmelidir.

2 3 4 5 0
Varsayalım ki girdi `import java.util.Scanner;` . Aşağıdaki kodun çıktısı nedir?

```
genel sınıfTest {
    public static void main(String[] args) {
        Scanner inp
        ut = tnewtScanner(System.in);
        ninttnumber, max;

        sayı = input.nextInt();
        nmax = sa
        yı;

        nokta{
            numara = input.nextInt();
            eğer(numara > max)
                maksimum = sayı;
        } while(numara != 0);

        System.out.println("max şudur: " + max);
        System.out.println("numara " + numara);
    }
}
```

4.7 **while** döngüsü ile do-while döngüsü arasındaki farklar nelerdir? Aşağıdaki **while** dön-
güsünü bir **do-while** döngüsüne dönüştürün.

```
Tarayıcı girişi = tnewtScanner(System.in);
ninttsum = t0t;
System.out.println("Bir tam sayı girin " +
    "(giriş 0 ise sona erer)");
int numara = input.nextInt();
while(numara != 0) {
    toplam += sayı;
    System.out.println("Bir tam sayı girin " +
        "(giriş 0 ise sona erer)");
    sayı = input.nextInt();
}
```

4.4 for Döngüsü



Bir **for** döngüsü, döngü yazmak için özlü bir sözdizimine sahiptir.

Sıklıkla bir döngüyü aşağıdaki yaygın biçimde yazarsınız:

```
i = başlangıçDeğeri; // Döngü kontrol değişkenini başlat
while(i < bitişDeğeri) {
    // Döngü gövdesi
    ...
    i++; // Döngü kontrol değişkenini ayarla
}
```

Bir **for** döngüsü, önceki döngüyü şu şekilde basitleştirmek için kullanılabilir:

```
for(i = başlangıçDeğeri; i < bitişDeğeri; i++) {
    // Döngü gövdesi
    ...
}
```

Genel olarak, bir **for** döngüsünün sözdizimi şudur:

```
for(başlangıç-eylemi; döngü-devam-etme-kşulu;  
    yinelemeden-sonra-eylem) {  
    // Döngü gövdesi;  
    İfade(ler);  
}
```

for döngüsü

for döngüsünün akış diyagramı Şekil 4.3a'da gösterilmiştir.

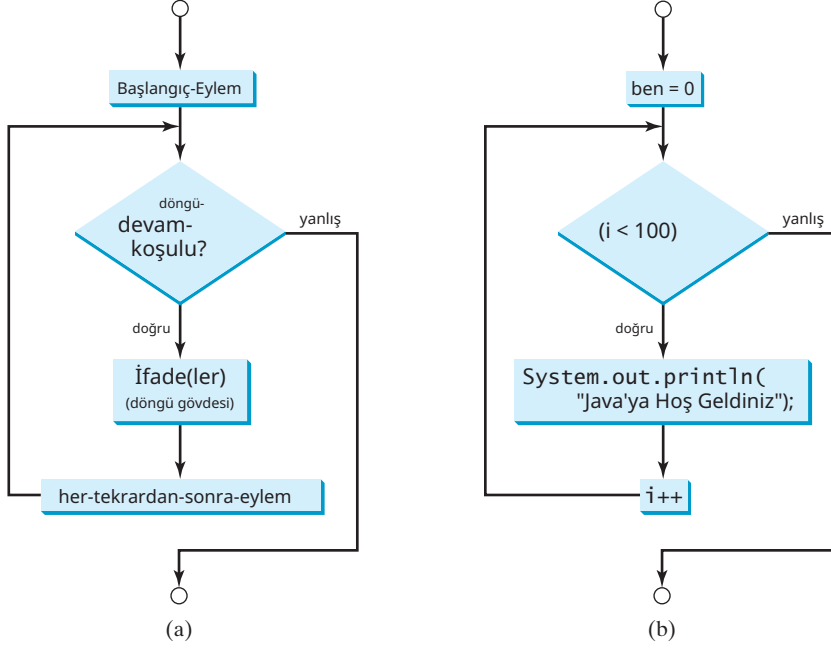


FIGURE 4.3 Bir **for** döngüsü, bir başlangıç eylemini bir kez gerçekleştirir, ardından döngü gövdesindeki ifadeleri tekrar tekrar yürütür ve bir yinelemeden sonra bir eylem gerçekleştirir, bu **döngü-devam-etme-kşulu** doğru olarak değerlendirildiğinde.

for döngüsü ifadesi, **for** anahtar kelimesi ile başlar, ardından döngünün kontrol ya pısını kapsayan bir çift parantez gelir. Bu yapı, başlangıç-eylemi, döngü-devam-etme-kşulu ve yinelemeden-sonra-eylem içerir. Kontrol yapısı, süslü parantezler içinde yer alan döngü gövdesi ile takip edilir. başlangıç-eylemi, döngü-devam-etme-kşulu ve yineleme den-sonra-eylem noktalı virgüllerle ayrılır.

Bir **for** döngüsü genellikle döngü gövdesinin ne kadar süreyle yürütüleceğini ve döngünün ne zaman sona ereceğini kontrol etmek için bir değişken kullanır. Bu değişkene kontrol değ i şkeni denir. başlangıç-eylemi genellikle bir kontrol değişkenini başlatır, yinelemeden-sonra-e ylem genellikle kontrol değişkenini artırır veya azaltır ve döngü-devam-etme-kşulu kontrol d eğişkeninin bir son değerine ulaşp ulaşmadığını test eder. Örneğin, aşağıdaki **for** döngüsü Ja va'ya Hoş Geldiniz! ifadesini yüz kez yazdırır:

kontrol değ i şkeni

```
int i;  
for(i = 0; i < 100; i++) {  
    System.out.println("Java'ya hoş geldiniz!");  
}
```

İfadenin akış diyagramı Şekil 4.3b'de gösterilmiştir. **for** döngüsü **i**'yi **0** olarak başlatır, ardından **println** ifadesini tekrar tekrar çalıştırır ve **i++**'yi değerlendirir, bu arada **i 100**'den küçüktür.

ilk-eylem

Başlangıç eylemi **i = 0**, kontrol değişkeni **i**'yi başlatır. Döngü-devam-etme-koşulu **i < 100**, bir Boolean ifadedir. İfade, başlatmadan hemen sonra ve her yinelemenin başında değerlendirilir. Eğer bu koşul doğru ise, döngü gövdesi çalıştırılır. Eğer yanlış ise, döngü sona erer ve program kontrolü döngünün ardından gelen satıra geçer.

her-tekrardan-sonra-eylem

Her yinelemeden sonraki eylem **i++**, kontrol değişkenini ayarlayan bir ifadedir. Bu ifade her yinelemeden sonra çalıştırılır ve kontrol değişkenini artırır. Sonunda, kontrol değişkeninin değeri döngü-devam-etme-koşulunu yanlış hale getirmelidir; aksi takdirde, döngü sonsuz olur.

Döngü kontrol değişkeni **for** döngüsünde bildirilebilir ve başlatılabilir. İşte bir örnek:

```
 için (int = t0 ; i < 100 ton; i++) {
    System.out.println("Java'ya Hoş Geldiniz!");
}
```

parantezleri atlama

Döngü gövdesinde yalnızca bir ifade varsa, bu örnekte olduğu gibi, süslü parantezler atlanabilir.

kontrol değişkeni bildir



Ttipp

Kontrol değişkeni, döngünün kontrol yapısı içinde veya döngüden önce bildirilmelidir. Eğer döngü kontrol değişkeni yalnızca döngü içinde kullanılıyorsa ve başka bir yerde değilse, onu **for** döngüsünün **initial-action** kısmında bildirmek iyi bir programlama pratiğidir. Eğer değişken döngü kontrol yapısı içinde bildirilirse, döngü dışında referans alınamaz. Önceki kodda, örneğin, **i**'yi **for** döngüsünün dışında referans alamazsınız, çünkü o **for** döngüsü içinde bildirilmiştir.

for döngü varyasyonları



Not

For döngüsündeki **initial-action**, sıfır veya daha fazla virgülle ayrılmış değişken bildirimi ifadeleri veya atama ifadeleri listesi olabilir. Örneğin:

```
 için (int = t0t, j = t0 ; (i + j < 10); i++, j++) {
    // Bir şey yap
}
```

For döngüsündeki **action-after-each-iteration**, sıfır veya daha fazla virgülle ayrılmış ifadeler listesi olabilir. Örneğin:

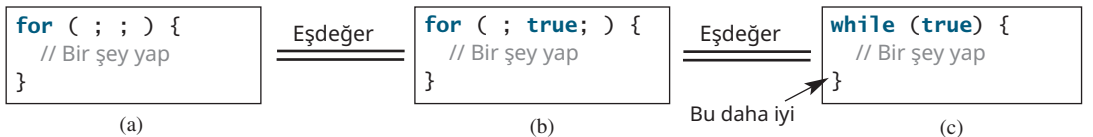
```
fort(tint = t1t; i < t100t); System.out.println(i), i++
```

Bu örnek doğrudur, ancak kötü bir örnektir, çünkü kodu okumayı zorlaştırır. Normalde, bir kontrol değişkenini başlangıç eylemi olarak bildirir ve başlatır ve her yinelemeden sonra kontrol değişkenini artırır veya azaltırsınız.



Not

Eğer **for** döngüsündeki **loop-continuation-condition** atlanırsa, bu durum dolaylı olarak doğrudur. Dolayısıyla, (a) kısmında verilen aşağıdaki ifade, sonsuz bir döngüdür ve (b) kısmındaki ile aynıdır. Ancak karışıklığı önlemek için, (c) kısmındaki eşdeğer döngüyü kullanmak daha iyidir.



4.8 Aşağıdaki iki döngü **sum** değerinin aynı sonucunu mu veriyor?

```
için(int i = 0; i < 10; ++ben) {
    nsum += i;
}
```

(a)

```
için(int i = 0; i < 10; i++) {
    nsum += i;
}
```

(b)

4.9 Bir **for** döngüsü kontrolünün üç parçası nedir? 1 ile 100 arasındaki sayıları yazdıran bir **for** döngüsü yazın.

4.10 Girdi **2 3 4 5 0** ise. Aşağıdaki kodun çıktısı nedir?

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner giri = new Scanner(System.in);
        int number, sum = 0, t0t, tcount;

        for(count = t0t; count < t5t; count++) {
            number = input.nextInt();
            toplam += sayi;
        }

        System.out.println("toplam " + sum);
        System.out.println("sayım " + count);
    }
}
```

4.11 Aşağıdaki ifade ne yapar?

```
for( ; ; ) {
    // Bir şey yap
}
```

4.12 Eğer bir değişken **for** döngüsünde tanımlanmışsa, döngü sona erdikten sonra kullanılabilir mi?

4.13 Aşağıdaki **for** döngüsünü ifadesini bir **while** döngüsüne ve bir **do-while** döngüsüne dönüştürün:

```
uzun toplam = t0t;
için(int i = 0; i <= 1000; i++)
    sum = sum + i;
```

4.14 Aşağıdaki döngülerdeki yinleme sayısını sayın.

```
inttcount = t0t;
while(count < n) {
    ncoun
    t++;
}
```

(a)

```
için(int count = 0;
    count <= n; count++) {
}
```

(b)

```
inttcount = t5t;
while(count < n) {
    ncoun
    t++;
}
```

(c)

```
inttcount = t5t;
while(count < n) {
    ncoun
    t = count + t 3t;
}
```

(d)



ön test döngüsü
son test döngüsü

4.5 Hangi Döngüyü Kullanmalıyım?

Bir **for** döngüsü, bir **while** döngüsü veya bir **do-while** döngüsü kullanabilirsiniz, hangisi uygunsa.

while döngüsü ve **for** döngüsü pretest loops olarak adlandırılır çünkü devam koşulu döngü gövdesi çalıştırılmadan önce kontrol edilir. **do-while** döngüsü ise posttest loop olarak adlandırılır çünkü koşul döngü gövdesi çalıştırıldıktan sonra kontrol edilir. Üç döngü ifadesi—**while**, **do-while** ve **for**—ifade olarak eşdeğerdir; yani, bu üç formdan herhangi birinde bir döngü yazabilirsiniz. Örneğin, aşağıdaki şeklin (a) kısmındaki bir **while** döngüsü her zaman (b) kısmındaki **for** döngüsüne dönüştürülebilir.

```
while(döngü-devam-koşulu) {  
    // Döngü gövdesi  
}
```

(a)

Eşdeğer

```
için( ; döngü-devam-etme-koşulu; ) {  
    // Döngü gövdesi  
}
```

(b)

Aşağıdaki şeklin (a) kısmındaki bir **for** döngüsü genellikle (b) kısmındaki **while** döngüsüne dönüştürülebilir, ancak belirli özel durumlar dışında (böyle bir durum için Kontrol Noktası Sorusu 4.23'e bakın).

```
için(başlangıç-eylemi;  
    döngü-devam-etme-koşulu;  
    each-iteration-sonrası-eylem) {  
    // Döngü gövdesi;  
}
```

(a)

Eşdeğer

```
başlangıç-eylemi;  
iken(döngü-devam-etme-koşulu) {  
    // Döngü gövdesi;  
    each-iteration-sonrası-eylem;  
}
```

(b)

Sizin için en sezgisel ve rahat olan döngü ifadesini kullanın. Genel olarak, bir **için** döngüsü, tekrarlama sayısı önceden biliniyorsa kullanılabilir; örneğin, bir mesajı yüz kez görüntülemeniz gerektiğinde. Bir **iken** döngüsü, tekrarlama sayısı sabit değilse kullanılabilir; örneğin, giriş **0** olana kadar sayıları okumak gibi. Bir **do-while** döngüsü, döngü gövdesinin devam etme koşulu test edilmeden önce çalıştırılması gerektiğinde bir **iken** döngüsünün yerini alabilir.



Dikkat

Döngü gövdesinden önceki **için** ifadesinin sonuna bir noktalı virgül eklemek yaygın bir hatadır, aşağıda (a) da gösterildiği gibi. (a) da, noktalı virgül döngünün sonunu erken belirtir. Döngü gövdesi aslında boştur, (b) de gösterildiği gibi. (a) ve (b) eşdeğerdir. Her ikisi de yanlıştır.

Hata

```
için(int i = 0; i < 10; i++) ;  
{  
    System.out.println("i şu anda "+ i);  
}
```

(a)

Boş gövde

```
for(tint = t0t; i < t10t; i++) n{  
    { };  
    System.out.println("i şu anda "+ i);  
}
```

(b)

Aynı şekilde, (c) içindeki döngü de yanlıştır. (c), (d) ile eşdeğerdir. Her ikisi de yanlıştır.

Hata

```
intti = t0t;nw  
hile(i < t10t)n{  
    ;  
    System.out.println("i şu anda "+ i);  
    i++;  
}
```

(c)

Boş gövde

```
intti = t0t;nw  
hile(i < t10t)n{  
    { };  
    System.out.println("i şu anda "+ i);  
    i++;  
}
```

(d)

Bu hatalar genellikle bir sonraki satır blok stilini kullandığınızda ortaya çıkar. Satır sonu blok stilini kullanmak bu tür hataları önleyebilir.

do-while döngüsü durumunda, döngüyü sonlandırmak için noktalı virgül gereklidir.

```
intti = t0t;nd
ot{
    System.out.println("i şu anda "+ i);
    i++;
}twhile(i < t10t)t;
```

Doğru

4.15 for döngüsünü while döngüsüne dönüştürebilir misiniz? for döngülerinin avantajlarını listeleyin.

4.16 Bir **while** döngüsünü her zaman bir **for** döngüsüne dönüştürebilir misiniz? Aşağıdaki **while**

döngüsünü bir **for** döngüsüne dönüştürün.

```
int = t1t;
inttsum = t0t;n
while(sum < t10000t) {nsu
    m = toplam + i;
    i++;n
}
```

4.17 Aşağıdaki koddaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfTest {
2     public voidtmain(String[] args) {
3         fort(tint = t0t; i < t10t; i++);
4 toplam += i;
5
6         ift(i < j);
7         System.out.println(i)
8         aksi takdirde
9         System.out.println(j);
10
11         whilet(j < t10t);n12
12 }
13 j++;n14 }
15
16
17 j++;n18 } } twhil
19 et(j < t10t)n19 }
20 }
```

4.18 Aşağıdaki programlarda ne yanlış?

```
1 genel classtShowErrors {
2     public static voidtmain(String[] args) {
3         int i;
4         intj = t5t;
5
6         ift(j > t3t)
7 System.out.println(i + t4t);n8 }
9 }
```

(a)

```
1 genel classtShowErrors {
2     public static voidtmain(String[] args) {
3         fort(tint = t0t; i < t10t; i++);
4 System.out.println(i + t4t);n5 }
6 }
```

(b)

Nokta Kontrol

MyProgrammingLab™

4.6 İç İç Döngüler



Bir döngü, başka bir döngünün içine yerleştirilebilir.

İç içe döngüler bir dış döngü ve bir veya daha fazla iç döngüden oluşur. Dış döngü her tekrarlandığında, iç döngüler yeniden girilir ve yeniden başlatılır.

Liste 4.7, bir çarpan tablosunu görüntülemek için iç içe **for** döngüleri kullanan bir programı sunmaktadır.

LİSTELER 4.7 ÇarpanTablosu.java

```

1  public class ÇarpanTablosu {
2      /** Ana metod */
3      public static void main(String[] args) {
4          // Tablo başlığını göster
5          System.out.println("        Çarpan Tablosu");
6
7          // Sayı başlığını göster
8          System.out.print("        ");
9          for(tinttj = t1t;tj <= t9t;tj++)n10 Syst
em.out.print(t" "t+ j);
11
12 System.out.println(t"n-----"t);
13
14          // Tablo gövdesini göster
15          ;ti <= t9t;ti++) {
16 System.out.print(i + t" | "t);
17          for(tint = t1t;tj <= t9t;tj) { for(tint = t1t
18              // Ürünü göster ve düzgün hizala
19 System.out.printf(t"%4d"t,ti * j);n20 }
21
22 System.out.println();n22 }
23 }n24
24 }

```



Çarpan Tablosu										
	1	2	3	4	5	6	7	8	9	
1		1	2	3	4	5	6	7	8	9
2		2	4	6	8	10	12	14	16	18
3		3	6	9	12	15	18	21	24	27
4		4	8	12	16	20	24	28	32	36
5		5	10	15	20	25	30	35	40	45
6		6	12	18	24	30	36	42	48	54
7		7	14	21	28	35	42	49	56	63
8		8	16	24	32	40	48	56	64	72
9		9	18	27	36	45	54	63	72	81

Program, çıktının ilk satırında bir başlık (satır 5) gösterir. İlk **for** döngüsü (satırlar 9–10) ikinci satırda 1 ile 9 arasındaki sayıları gösterir. Üçüncü satırda (satır 12) kesik bir (–) çizgi gösterilir.

Sonraki döngü (satırlar 15–22), dış döngüde kontrol değişkeni **i** ve iç döngüde **j** olan iç içe bir **for** döngüsüdür. Her **i** için, iç döngüde **i * j** çarpımı bir satırda gösterilir, **j** ise 1, 2, 3, ..., 9 olur.



Not

Dikkat edin ki, iç içe bir döngü çalıştırmak uzun sürebilir. Aşağıdaki döngüyü

üç seviyede iç içe düşünün:

```
for(int i = 0; i < 10000; i++)
    for(int j = 0; j < 10000; j++)
        for(int k = 0; k < 10000; k++)
            Bir işlem gerçekleştirin
```

İşlem bir trilyon kez gerçekleştirilir. Eğer işlemi gerçekleştirmek 1 mikro saniye alıyorsa, döngüyü çalıştırmak toplamda 277 saatten fazla sürecektir. 1 mikro saniyenin

bir saniyenin bir milyonuncu (10^{-6}) kısmı olduğunu unutmayın.

4.19 println ifadesi kaç kez çalıştırılıyor?

```
for(tint = t0t; i < t10t; i++) nfort(tinttj
    = t0t; j < i; j++) nSystem.out.println
    n(i * j)
```

Nokta Kontrol

MyProgrammingLab™

4.20 Aşağıdaki programların çıktısını gösterin. (İpucu: Bir tablo çizin ve değişkenleri sütunlarda listeleyin bu programları izlemek için.)

```
genel sınıfTest {
    /** Ana metod */
    public static void main(String[] args) {nfort(tintti = t
        1t; i < t5t; i++) {
        inttj = t0t;
        whilet(j < i) {nSyste
            m.out.print(j + t" "t);nj++;
        }
    }
}
```

(a)

```
genel sınıfTest {
    /** Ana metod */
    public static void main(String[] args) {nintti = t0t;

    while(i < t5t) {
        fort(tinttj = i; j > t1t;j——)nSystem.o
            ut.print(j + t" "t);nSystem.out.prin
            tln(t"*****t");ni++;
    }
}
```

(b)

```
genel sınıfTest {
    public static void main(String[] args) {nintti = t5t;

    while(i >= t1t) {
        inttnum = t1t;
        fort(tinttj = t1t; j <= i; j++) {nSystem.ou
            t.print(num + t"xxx"t);nnum *= t2t;
        }

        System.out.println();ni—
        —;
    }
}
```

(c)

```
genel sınıfTest {
    public static void main(String[] args) {nintti = t1t;

    nokta{
        inttnum = t1t;
        fort(tinttj = t1t; j <= i; j++) {nSystem.ou
            t.print(num + t"G"t);nnum += t2t;
        }

        System.out.println();ni++;
    }while(i <= t5t);n}
}
```

(d)

4.7 Sayısal Hataları En Aza İndirme

Döngü devam koşulunda ondalık sayılar kullanmak sayısal hatalara neden olabilir.

Ondalık sayılarla ilgili sayısal hatalar kaçınılmazdır. Bu bölüm, böyle hataları minimize etmeni n bir yolunu bir örnekle tartışmaktadır.

Liste 4.8, **0.01** ile başlayıp **1.0** ile biten bir seriyi toplama örneğini sunmaktadır. Serideki sayılar **0.01** kadar artacak, şu şekilde: **0.01 + 0.02 + 0.03** ve devamı.

LİSTELER4.8TestSum.java

```
1 genel sınıfTestSum {
2     public static voidtmain(String[] args) {
3         // Toplamı başlat
4         kayan toplam = t0t;
5
6         // 0.01, 0.02, ..., 0.99, 1'i toplamaya ekle
7         fort(tfloatti = t0,01ft; i <= t1,0ft; i = i + t0,01ft)
8 toplam += i;
9
10        // Sonucu göster
11        System.out.println("Toplam "+ sum);
12    }
13 }
```

Toplam 50.499985'tir.

for döngüsü (satırlar 7–8) kontrol değişkeni i'yi sürekli olarak sum'a ekler. Bu değişken, 0.01 ile başlar ve her yinelemeden sonra 0.01 kadar artırılır. Döngü, i'nin 1.0'ı aşmasıyla sona erer.

for döngüsünün başlangıç eylemi herhangi bir ifade olabilir, ancak genellikle bir kontrol değişkenini başlatmak için kullanılır. Bu örnekten, bir kontrol değişkeninin float türünde olabileceğini görebilirsiniz. Aslında, herhangi bir veri türü olabilir.

Kesin sum 50.50 olmalıdır, ancak cevap 50.499985'tir. Sonuç, bilgisayarların ondalıklı sayıları temsil etmek için sabit bir bit sayısı kullanmasından dolayı kesin değildir ve bu nedenle bazı ondalıklı sayıları tam olarak temsil edemezler. Programda float'ı double ile değiştirirseniz, aşağıdaki gibi, kesinlikte hafif bir iyileşme görmelisiniz, çünkü bir double değişkeni 64 bit tutarken, bir float değişkeni 32 bit tutar.

```
// Toplamı başlat
çift toplam = t0t;

// 0.01, 0.02, ..., 0.99, 1'i toplamaya ekle
için (çift i = 0,01 ton; i <= t1,0t; i = i + t0,01t)nsum
    += i;
```

Ancak, sonucun aslında 49.50000000000003 olduğunu görmek sizi şaşırtacak. Ne yanlış gitti? Döngüde her yinelemede i'yi görüntülerseniz, son i'nin 1'den (tam olarak 1 değil) biraz daha büyük olduğunu göreceksiniz. Bu, son i'nin sum'a eklenmemesine neden olur. Temel sorun, ondalıklı sayıların yaklaşık olarak temsil edilmesidir. Sorunu çözmek için, tüm sayıların sum'a eklenmesini sağlamak için bir tam sayı sayacı kullanın. İşte yeni döngü:

```
doublecurrentValue = t0.01t;nfor
    (tinttcount = t0t; count < t100t; count++) {
```



VideoNotu

Sayısal hataları en aza indir

döngü

double hassasiyeti

sayısal hata

```
toplam += güncelDeğer;n
geçerliDeğer += t0.01t;n}
```

Bu döngüden sonra, **sum50.50000000000003**. Bu döngü, en küçükten en büyüğe sayıları ekler. Eğer en büyükten en küçüğe sayıları (yani, **1.0,0.99,0.98,...**, **0.02,0.01** bu sırayla) eklerseniz, ne olur:

```
double currentValue = t1.0t;nfor
```

```
    (tinttcount = t0t; count < t100t; count++) {
        nsum += geçerliDeğer;
        mevcutDeğer -= t0.01t;
    }
```

Bu döngüden sonra, toplam **50.49999999999995**. En büyükten en küçüğe eklemek, en küçükten en büyüğe eklemekten daha az doğrudur. Bu fenomen, sonlu-kesirli aritmetiğin bir artefaktıdır. Çok küçük bir sayıyı çok büyük bir sayıya eklemek, eğer sonuç daha fazla hassasiyet gerektiriyorsa, hiçbir etki yaratmayabilir ve değişkenin depolayabileceğinden daha fazla hassasiyet gerektirir. Örneğin, **100000000.0 + 0.000000001** işleminin yanlış sonucu **100000000.0**'dir. Daha doğru sonuçlar elde etmek için, hesaplama sırasını dikkatlice seçin. Küçük sayıları büyük sayılardan önce eklemek, hataları en aza indirmenin bir yoludur.

sayısal hatalardan kaçınma

4.8 Vaka Çalışmaları

Döngüler programlamada temeldir. Döngü yazma yeteneği, Java programlamayı öğrenmek için gereklidir.



Eğer döngüler kullanarak programlar yazabiliyorsanız, programlamayı biliyorsunuzdur! Bu nedenle, bu bölüm döngüler kullanarak sorunları çözmenin üç ek örneğini sunmaktadır.

4.8.1 Vaka Çalışması: En Büyük Ortak Bölgenin Bulunması

İki tam sayının en büyük ortak böleni (gcd) **4** ve **2**, **2**'dir. İki tam sayının en büyük ortak böleni **16** ve **24**, **8**'dir. En büyük ortak böleni nasıl bulursunuz? İki girdi tam sayısını **n1** ve **n2** olarak alalım. Sayı **1** bir ortak bölen olduğunu biliyorsunuz, ancak bu en büyük ortak bölen olmayabilir. Bu nedenle, **k** (için **k2 kontrol edebilirsiniz**). **= 2, 3, 4**, ve devamı) bir ortak bölen **n1** ve **n2** için, **k** **n1** veya **n2**'den büyük olana kadar. Ortak böleni **gcd** adında bir değişkende saklayın. Başlangıçta, **gcd 1**'dir. Yeni bir ortak bölen bulunduğunda, bu yeni gcd olur. **2**'den başlayarak tüm olası ortak bölenleri kontrol ettiğinizde **n1** veya **n2**, değişken **gcd** içindeki değer en büyük ortak bölenidir. Bu fikir aşağıdaki döngüye çevrilebilir:

gcd

```
int gcd = 1; // Başlangıç gcd 1'dir
int k = 2; // Olası gcd

while(k <= n1 && k <= n2) {
    if(n1 % k == 0 && n2 % k == 0)
        gcd = k; // gcd'yi güncelle
    k++; // Son mümkün gcd
}
```

// Döngüden sonra, gcd n1 ve n2 için en büyük ortak bölenidir

Liste 4.9, kullanıcının iki pozitif tam sayı girmesini isteyen ve

bunların en büyük ortak bölenini bulan programı sunmaktadır.

LİSTE4.9GreatestCommonDivisor.java

```

1  import java.util.Scanner;
2
3  public class GreatestCommonDivisor {
4      /** Ana metod */
5      public static void main(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = new Scanner(System.in);
8
9          // Kullanıcıdan iki tam sayı girmesini iste
10         System.out.print("İlk tam sayıyı girin: ");
11         = input.nextInt();
12         System.out.print("İkinci tam sayıyı girin: ");
13         int n2 = input.nextInt(); int n1
14
15         int gcd = 1; // Başlangıç gcd 1'dir
16         int k = 2; // Olası gcd
17         while(k <= n1 && k <= n2) {
18             if(n1 % k == 0 && n2 % k == 0)
19                 gcd = k; // gcd'yi güncelle
20             k++;
21         }
22
23         System.out.println("En büyük ortak bölen " + n1 +
24             " ve " + n2 + " dir " + gcd);
25     }
26 }

```

girdi

girdi

gcd

bölgenin kontrolü

çıkış



İlk tam sayıyı girin: 125 Enter
 İkinci tam sayıyı girin: 2525 Enter
 125 ve 2525 için en büyük ortak bölen 25'tir

yazmadan önce düşün

Bu programı nasıl yazdınız? Hemen kod yazmaya mı başladınız? Hayır. Yazmadan önce düşünmek önemlidir. *Düşünmek, soruna mantıklı bir çözüm üretmenizi sağlar* kodu nasıl yazacağınız konusunda endişelenmeden. Mantıklı bir çözüm bulduğunuzda, çözümü bir Java programına çevirmek için kodu yazın. Çeviri benzersiz değildir. Örneğin, şu şekilde kodu yeniden yazmak için bir *için* döngüsü kullanabilirsiniz:

```

için(int k = 2; k <= n1 && k <= n2; k++) {
    if(n1 % k == 0 && n2 % k == 0)
        gcd = k;
}

```

birden fazla çözüm

Bir problemin genellikle birden fazla çözümü vardır ve gcd problemi birçok şekilde çözülebilir. Programlama Egzersizi 4.14 başka bir çözüm öneriyor. Daha verimli bir çözüm, klasik Öklid algoritmasını kullanmaktır (daha fazla bilgi için www.cut-the-knot.org/blue/Euclid.shtml bakın).

hatalı çözümler

Bir sayı için *n1* bir bölenin *n1 / 2*'den büyük olamayacağını düşünebilirsiniz ve aşağıdaki döngüyü kullanarak programı geliştirmeye çalışabilirsiniz:

```

for(int t = 2; t <= n1/2 && t <= n2/2; t++) {
    if(n1 % t == 0 && n2 % t == 0) gcd = t;
}

```

Bu revizyon yanıltır. Sebebini bulabilir misin? Cevap için Kontrol Noktası Sorusu 4.21'e bak.

4.8.2 Vaka Çalışması: Gelecek Öğrenim Ücretini Tahmin Etme

Varsayalım ki bir üniversitenin öğrenim ücreti bu yıl **\$10,000** ve öğrenim ücreti her yıl **%7** artıyor. Öğrenim ücreti kaç yılda iki katına çıkacak?

Bu problemi çözmek için bir program yazmadan önce, önce elle nasıl çözüleceğini düşü nün. İkinci yılın öğrenim ücreti birinci yılın öğrenim ücreti * **1.07**'dir. Gelecek yılın öğrenim ücreti, bir önceki yılın öğrenim ücreti * **1.07**'dir. Böylece, her yılın öğrenim ücreti aşağıdaki gibi hesaplanabilir:

```
çift eğitim = t10000t;      int yıl = 0;          // Yıl 0
tuition = tuition * 1.07; yıl++;          // Yıl 1
tuition = tuition * 1.07; yıl++;          // Yıl 2
tuition = tuition * 1.07; yıl++;          // Yıl 3
...
```

Yeni bir yıl için öğrenim ücretini hesaplamaya devam edin, ta ki en az **20000** olana kadar. O zaman öğ renim ücretinin iki katına çıkması için kaç yıl gerektiğini bileceksiniz. Şimdi mantığı aşağıdaki döngüye çevirebilirsiniz:

```
çift eğitim = t10000t;          // Yıl 0
int yıl = 0;
while(öğrenim < 20000) {
    öğrenim = öğrenim * 1.07;
    yıl++;
}
```

Tam program Liste 4.10'da gösterilmiştir.

LISTE 4.10 Gelecek Öğrenim Ücreti.java

```
1 public class GelecekÖğrenimÜcreti {
2     public static void tmain(String[] args) {
3         double öğrenimÜcreti = 10000;    // Yıl 0
4         int yıl = 0;
5         while(öğrenimÜcreti < 20000) {
6             öğrenimÜcreti = öğrenimÜcreti * 1.07;
7             yıl++;
8         }
9
10        System.out.println("Öğrenim ücreti "
11        + yıl + " yıl içinde iki katına çıkacak");
12        System.out.printf("Öğrenim ücreti $%.2f olacak %1d yıl içinde",
13        öğrenimÜcreti, yıl);
14    }n15 }
```

döngü
gelecek yılın öğrenim ücreti

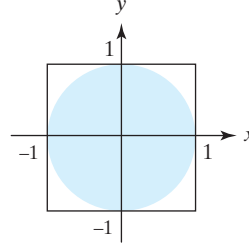
Öğrenim ücreti 11 yıl içinde iki katına çıkacak
Öğrenim ücreti 11 yıl içinde 21048,52 \$ olacak



while döngüsü (satırlar 5–8) yeni bir yıl için öğrenim ücretini tekrar tekrar hesaplamak için kullanılır. D öngü, öğrenim ücreti 20000'e eşit veya daha büyük olduğunda sona erer.

4.8.3 Vaka Çalışması: Monte Carlo Simülasyonu

Monte Carlo simülasyonu, problemleri çözmek için rastgele sayılar ve olasılık kullanır. Bu yöntem, hesaplamalı matematik, fizik, kimya ve finans gibi geniş bir uygulama yelpazesine sahiptir. Bu bölüm, Monte Carlo simülasyonunu kullanarak tahmin yapma örneği vermektedir. Monte Carlo yöntemini kullanarak tahmin yapmak için, aşağıda gösterildiği gibi bir daire çizin.



Dairenin yarıçapının 1 olduğunu varsayalım. Bu nedenle, dairenin alanı π ve kare alanı 4. Kare içinde rastgele bir nokta oluşturun. Noktanın dairenin içine düşme olasılığı $\text{daireninAlanı} / \text{kareAlanı} = \pi / 4$.

Kare içinde rastgele 1.000.000 nokta oluşturan bir program yazın ve `numberOfHits` dairenin içine düşen nokta sayısını belirtir. Böylece, `numberOfHits` yaklaşık olarak $1000000 * (\pi / 4) \cdot \pi$ yaklaşık olarak $4 * \text{numberOfHits} / 1000000$. Tam program Liste 4.11'de gösterilmiştir.

LISTE 4.11 MonteCarloSimulation.java

```

1  genel sınıf MonteCarloSimülasyon {
2      public static void main(String[] args) {
3          final int NUMBER_OF_TRIALS = 100000000;
4          int Hit Sayısı = 0;
5
6          for (int i = 0; i < NUMBER_OF_TRIALS; i++) {
7              double x = Math.random() * 2.0 - 1.0;
8              double y = Math.random() * 2.0 - 1.0;
9              if (x * x + y * y <= 1.0)
10                 Hit Sayısı++;
11          }
12
13          double pi = 4.0 * numberOfHits / NUMBER_OF_TRIALS;
14          System.out.println("PI şudur " + pi);
15      }
16  }
```

rastgele noktalar oluştur

çemberin içinde kontrol et

pi'yi tahmin et



PI 3.14124'tür

Program, satırlar 7-8'de kare içinde rastgele bir nokta (x, y) üretmeye devam eder:

```
double x = Math.random() * 2.0 - 1.0;
double y = Math.random() * 2.0 - 1.0;
```

Eğer `est kat + ta tataat...t 1,` nokta dairenin içindeyse ve `numberOfHits` 1 ile artırılıyorsa. π yaklaşık olarak $4 * \text{numberOfHits} / \text{NUMBER_OF_TRIALS}$ (satır 13).

4.21 Program, 17. satırda n1 ve n2'nin n1 / 2 ve n2 / 2 ile değiştirilmesi durumunda ç alışacak mı?

Nokta  kontrol

MyProgrammingLab™

4.9 Anahtar Kelimeler: break ve continue

break ve continue anahtar kelimeleri, bir döngüde ek kontrol sağlar.



Petdagtotgtictal Ntote

İki anahtar kelime, break ve continue, döngü ifadelerinde ek kontroller sağlamak için kullanılabilir. break ve continue kullanmak bazı durumlarda programlamayı basitleştirebilir. Ancak, bunların aşırı kullanımı veya yanlış kullanımı, programların okunmasını ve hata ayıklanmasını zorlaştırabilir. (Eğitmenler için not: Bu bölümü atlayabilirsiniz, bu öğrencilerin kitabın geri kalanını anlamasını etkilemez.)

switch ifadesinde break anahtar kelimesini kullandınız. Ayrıca, bir döngüyü hemen sonlandırmak için break k'i bir döngüde de kullanabilirsiniz. 4.12. liste, bir döngüde break kullanmanın etkisini göstermek için bir program sunmaktadır.

break ifadesi

LİSTE 4.12 TestBreak.java

```

1  genel sınıfTestBreak {
2      public static voidtmain(String[] args) {
3          inttsum = t0t;
4          inttnumber = t0t;
5
6          whilet(sayı < t20t) {n7 sayı
7              ++;n8 toplam += sayı;n
8              if (toplam >= t100t)
9                  kırmak ;
10             }
11         }
12
13         System.out.println("Sayı "+ number);
14         System.out.println("Toplam "+ sum);
15 }n16 }
```

kırmak

Sayı 14
Toplam 105



Liste 4.12'deki program, toplam 100'e eşit veya daha büyük olana kadar bu sırayla 1' den 20'ye kadar olan tam sayıları **sum**'a ekler. **if** ifadesi (satır 9) olmadan, program 1'de n 20'ye kadar olan sayıların toplamını hesaplar. Ancak **if** ifadesi ile döngü, **sum** 100'e eşit veya daha büyük olduğunda sona erer. **if** ifadesi olmadan, çıktı şöyle olur:

Sayı 20
Toplam 210



Bir döngüde **continue** anahtar kelimesini de kullanabilirsiniz. Karşılaşıldığında, mevcut yinelemeyi sonlandırır ve program kontrolü döngü gövdesinin sonuna gider. Diğer bir deyişle, continue bir yinelenen çıkarken, **break** anahtar kelimesi bir döngüden çıkar. Liste 4.13, bir döngüde **continue** kullanımının etkisini göstermek için bir program sunmaktadır.

continue ifadesi

LISTE 4.13 TestContinue.java

```

1  genel sınıfTestContinue {
2      public static void main(String[] args) {
3          int tsum = 0;
4          int tnumber = 0;
5
6          while(sayı < 20) {
7              sayı++;
8              eğer(sayı == 10 || sayı == 11)
9                  devam ;
10             toplam += sayı;
11         }
12
13         System.out.println("Toplam "+ toplam);
14     }n15
15 }

```

devam



Toplam 189

Liste 4.13'teki program, 1'den 20'ye kadar olan tam sayıları toplar, ancak 10 ve 11 toplam'a eklenmez. Programdaki **if** ifadesi (satır 8) ile **devam** ifadesi, **sayı** 10 veya 11 olduğunda çalıştırılır. **Devam** ifadesi, mevcut yinelemeyi sonlandırır, böylece döngü gövdesindeki diğer ifadeler çalıştırılmaz; bu nedenle, **sayı** toplam'a eklenmez. **10** veya **11**. Programda **if** ifadesi olmasaydı, çıktı aşağıdaki gibi olurdu:



Toplam 210'dir.

Bu durumda, tüm sayılar toplam'a eklenir, **sayı** 10 veya 11 olduğunda bile. Bu nedenle, sonuç 210'dir, bu da 21 daha fazladır, eğer **if** ifadesi ile karşılaştırıldığında.

**Not**

continue ifadesi her zaman bir döngü içinde bulunur. **while** ve **do-while** döngülerinde, döngü-devam-etme-koşulu hemen **continue** ifadesinden sonra değerlendirilir. **for** döngüsünde, **her-iterasyondan-sonra-yapılacak-eylem** gerçekleştirilir, sonra **döngü-devam-etme-koşulu** değerlendirilir, hemen **continue** ifadesinden sonra.

Bir döngüde **break** veya **continue** kullanmadan her zaman bir program yazabilirsiniz (bkz. Kontrol-noktası Soru 4.24). Ancak genel olarak, **break** ve **continue** kullanmak, kodlamayı basitleştiriyorsa ve programları okumayı kolaylaştırıyorsa uygundur.

Bir tam **sayı** **n** için **1** dışında en küçük çarpanı bulmak üzere bir program yazmanız gerektiğini varsayalım (**n** >= 2 varsayalım). Aşağıdaki gibi **break** ifadesini kullanarak basit ve sezgisel bir kod yazabilirsiniz:

```

inttfaktör = 2;nw
hile(faktör <= n) {nift(n % f
    aktör == 0t)nfaktör++;
    kırılma;
}

```



```
System.out.println("1 dışında en küçük çarpan "
    + n + " dir "+ factor);
```

Kodu **break** kullanmadan aşağıdaki gibi yeniden yazabilirsiniz:

```
boolean bulundu = false;
int faktör = 2;
while(faktör <= n && !bulundu) {
    if(n % faktör == 0) {
        bulundu = true;
        break;
    }
    faktör++;
}
System.out.println("1 dışında en küçük çarpan "
    + n + " dir "+ factor);
```

Ayrıca, **break** ifadesi bu programı bu durumda daha basit ve okunması daha kolay hale getiriyor. Ancak, **break** ve **continue** ifadelerini dikkatli kullanmalısınız. Çok fazla **break** ve **continue** ifadesi, birçok çıkış noktası olan bir döngü oluşturur ve programı okunması zor hale getirir.



Not

Bazı programlama dilleri **goto** ifadesine sahiptir. **goto** ifadesi, programdaki herhangi bir ifadeye kontrolü ayırt etmeden aktarır ve onu yürütür. Bu, programınızı hatalara karşı savunmasız hale getirir. Java'daki **break** ve **continue** ifadeleri, **goto** ifadelerinden farklıdır. Sadece bir döngü veya **switch** ifadesinde çalışırlar. **break** ifadesi döngüden çıkmayı sağlar ve **continue** ifadesi mevcut yinlemeyi atlar.

git

- 4.22** **break** anahtar kelimesi ne için kullanılır? **continue** anahtar kelimesi ne için kullanılır? Aşağıdaki programlar sonlanacak mı? Eğer öyleyse, çıktıyı verin.

Nokta kontrol

MyProgrammingLab™

```
int bakiye = 10;
while (true) {
    if (bakiye < 9)
        break;
    bakiye = bakiye - 9;
}

System.out.println("Bakiye "
    + bakiye);
```

(a)

```
int bakiye = 10;
while (true) {
    if (bakiye < 9)
        continue;
    bakiye = bakiye - 9;
}

System.out.println("Bakiye "
    + bakiye);
```

(b)

- 4.23** Soldaki **for** döngüsü, sağdaki **while** döngüsüne dönüştürülmüştür. Ne yanlış? Düzeltin.

```
for(tinti = t0t; i < t4t; i++) {nift(i % t3t
    ==t0t)tcontinue;tsum += i;
}
```

Dönüştürüldü

Yanlış dönüşüm

```
int = t0t;
while(i < t4t) {nift(it
    3t==t0t)tcontinue;ttoplam += i;
    i++;n
}
```

- 4.24** 4.12 ve 4.13 numaralı Listelerdeki **TestBreak** ve **TestContinue** programlarını **break** ve **continue** kullanmadan yeniden yazın.
- 4.25** (a) bölümündeki **break** ifadesi aşağıdaki döngüde çalıştırıldığında, hangi ifade çalıştırılır? Çıktıyı gösterin. (b) bölümündeki **continue** ifadesi aşağıdaki döngüde çalıştırıldığında, hangi ifade çalıştırılır? Çıktıyı gösterin.

```
fort(tint = t1t; i < t4t; i++) { nfort(tinttj
= t1t; j < t4t; j++) { nift(i * j > t2t)
```

kırılma;

System.out.println(i * j);n}

System.out.println(i);n}

(a)

```
fort(tint = t1t; i < t4t; i++) { nfort(tinttj
= t1t; j < t4t; j++) { nift(i * j > t2t)
```

devam;

System.out.println(i * j);n}

System.out.println(i);n}

(b)

4.10 Vaka Çalışması: Asal Sayıları Gösterme



Bu bölüm, ilk elli asal sayıyı beş satırda, her birinde on sayı olacak şekilde gösteren bir program sunmaktadır.

1'den büyük bir tam sayı, yalnızca pozitif böleni 1 veya kendisi ise asal sayıdır. Örneğin, 2, 3, 5 ve 7 asal sayılardır, ancak 4, 6, 8 ve 9 asal değildir.

Problem, ilk 50 asal sayıyı beş satırda göstermek, her birinin on sayı içermesi dir. Problem aşağıdaki görevlere ayrılabilir:

- Verilen bir sayının asal olup olmadığını belirleyin.
- `number = 2, 3, 4, 5, 6, ...,` asal olup olmadığını test edin.
- Asal sayıların sayısını sayın.
- Her asal sayıyı gösterin ve her satırda on sayı gösterin.

Belirgin bir şekilde, bir döngü yazmanız ve yeni bir sayının asal olup olmadığını sürekli test etmeniz gerekiyor. Eğer sayı asal ise, sayımı 1 artırın. Sayım başlangıçta 0'dır. 50'ye ulaştığında a, döngü sona erer.

İşte problemin algoritması:

Yazdırılacak asal sayıların sayısını

`SABİT NUMBER_OF_PRIMES` olarak ayarlayın;

Asal sayıların sayısını takip etmek için `count` kullanın ve

giriş sayısını 0 olarak ayarlayın;

Başlangıç sayısını 2 olarak ayarlayın;

```
while (sayacı < ASAL_SAYILARIN_SAYISI) {
```

Sayının asal olup olmadığını test et;

Eğer sayı asal ise {

Asal sayıyı göster ve sayacı artır;

}

Sayıyı 1 artır;

```
}
```

Bir sayının asal olup olmadığını test etmek için, sayının 2,3,4 gibi sayılara veya 2'ye kadar bölünüp bölünmediğini kontrol et. Eğer bir bölen bulunursa, sayı asal değildir. Algoritma şu şekilde tanımlanabilir:

Sayının asal olup olmadığını belirtmek için bir boolean değişkeni isPrime kullan; isPrime'i başlangıçta true olarak ayarla; for (int

bölen = 2; bölen <= sayı / 2; bölen++) {if (sayı % bölen == 0) {

isPrime'i false olarak ayarla
Döngüden çık;

}N}

Tam program, Liste 4.14'te verilmiştir.

LİSTE 4.14 PrimeNumber.java

```

1  genel classtPrimeNumber {
2      public static voidtmain(String[] args) {
3          final int NUMBER_OF_PRIMES = 50; // Gösterilecek asal sayıların sayısı
4          final int NUMBER_OF_PRIMES_PER_LINE = 10; // Satır başına 10 göster
5          int count = 0; // Asal sayıların sayısını say
6          int number = 2; // Asallık testi yapılacak bir sayı
7
8          System.out.println("İlk 50 asal sayı şunlardır: ");
9
10         // Tekrar tekrar asal sayıları bul
11         iken(count < NUMBER_OF_PRIMES) {
12             // Sayının asal olduğunu varsay
13             booleanisPrime = true; // Mevcut sayı asal mı?
14
15             // Sayının asal olup olmadığını test et
16             için(intbölgen = 2; bölgen <= number / 2; bölgen++) {
17                 if(number % divisor == 0) { // Eğer doğruysa, sayı asal değildir
18                     isPrime = false; // isPrime'i false olarak ayarla
19                     break; // for döngüsünden çık
20                 }
21             }
22
23             // Asal sayıyı göster ve sayacı artır
24             ift(isPrime) {
25                 count++; // Sayacı artır
26
27                 eğer(count % NUMBER_OF_PRIMES_PER_LINE == 0) {
28                     // Sayıyı göster ve yeni satıra geç
29                     System.out.println(sayı);n30 }
30
31                     aksi takdirde
32                     System.out.print(sayı + t" "t);n33 }
33
34         // Bir sonraki sayının asal olup olmadığını kontrol et
35         number++;
36     }
37 }
38
39 ısırık {na
40  q}n}

```

asal sayıları say

asal olma durumunu kontrol et

döngüden çık

asal ise göster



İlk 50 asal sayı şunlardır:
 2 3 5 7 11 13 17 19 23 29
 31 37 41 43 47 53 59 61 67 71
 73 79 83 89 97 101 103 107 109 113
 127 131 137 139 149 151 157 163 167 173
 179 181 191 193 197 199 211 223 227 229

alt problem

Bu, acemi programcılar için karmaşık bir programdır. Bu problem için programatik bir çözüm geliştirmekteki anahtar, ve diğer birçok problem için, onu alt problemlere ayırmak ve her biri için sırayla çözümler geliştirmektir. İlk denemede tam bir çözüm geliştirmeye çalışmayın. Bunun yerine, verilen bir sayının asal olup olmadığını belirlemek için kod yazmaya başlayın, ardından programı bir döngüde diğer sayıların asal olup olmadığını test edecek şekilde genişletin.

Bir sayının asal olup olmadığını belirlemek için, sayının 2 ile number/2 arasında (16-21. satırlar) bir sayıya tam bölünüp bölünmediğini kontrol edin. Eğer bölünüyorsa, asal bir sayı değildir (18. satır); aksi takdirde, asal bir sayıdır. Asal bir sayı için, onu gösterin. Eğer sayı 10'a tam bölünüyorsa (27-30. satırlar), yeni bir satıra geçin. Program, sayı 50'ye ulaştığında sona erer.

Program, 19. satırda sayının asal olmayan bir sayı olduğu bulunur bulunmaz döngüden çıkma k için **break** ifadesini kullanır. Döngüyü (16-21. satırlar) **break** ifadesini kullanmadan şu şekilde yeniden yazabilirsiniz:

```
for( int bölgen = 2; bölücü <= sayı / 2 && asal; bölgen++) {
    // Eğer doğruysa, sayı asal değildir
    if(sayı % bölgen == 0) {
        // isPrime'i false olarak ayarla, eğer sayı asal değilse
        isPrime = false;
    }
}
```

Ancak, **break** ifadesini kullanmak bu durumda programı daha basit ve okunması daha kolay hale getirir.

4.11 Onay Diyalogu ile Bir Döngüyü Kontrol Etme



Anahtar
Nokta

Kullanıcıdan bir döngüyü devam ettirmek veya çıkmak için onay almak üzere bir onay diyalogu kullanılabilir.

onay diyalogu

Bir sentinel kontrollü döngü, bir onay diyalogu kullanılarak uygulanabilir. Cevaplar *Evet* veya *Hayır* döngüyü devam ettirir veya sonlandırır. Döngünün şablonu aşağıdaki gibi görünebilir:

```
int option = JOptionPane.YES_OPTION;
while(option == JOptionPane.YES_OPTION) {
    System.out.println("devam et döngüsü");
    option = JOptionPane.showConfirmDialog(null, "Devam edilsin mi?");
}
```

Liste 4.15, bir onay diyalogu kutusu kullanarak Liste 4.5, SentinelValue.java'yı yeniden yazmaktadır. Bir örnek çalışma şekil 4.4'te gösterilmiştir.

LISTE 4.15 SentinelValueUsingConfirmationDialog.java

```
1 import javax.swing.JOptionPane;
2
3 genel sınıf SentinelValueUsingConfirmationDialog {
4     public static void main(String[] args) {
5         int sum = 0;
```

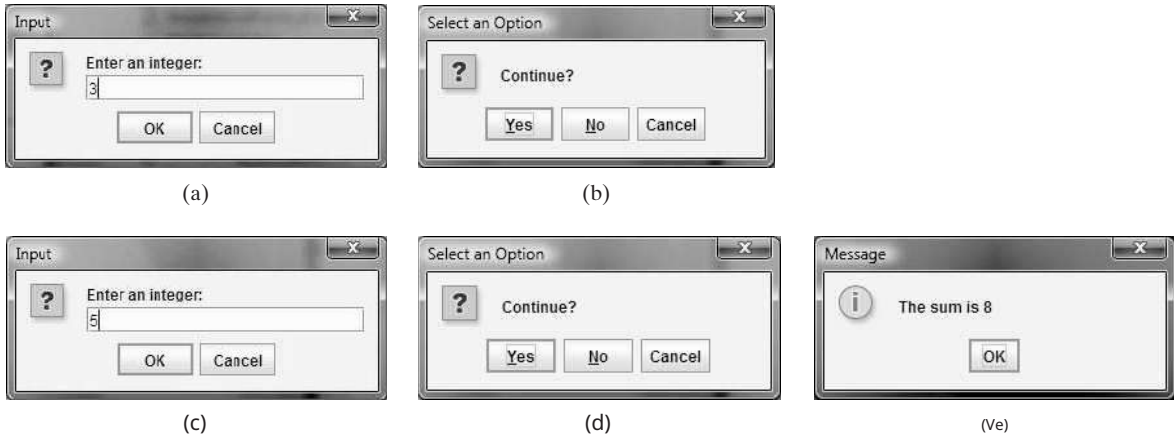
```

6
7 // Kullanıcı Hayır yanıt verene kadar verileri okumaya devam et
8 intseçenek = JOptionPane.YES_OPTION;
9 while(seçenek == JOptionPane.YES_OPTION) {
10 // Sonraki veriyi oku
11 String veriDizesi = JOptionPane.showInputDialog(
12     "Bir tam sayı girin: ");
13     intveri = Integer.parseInt(dataString);
14
15 toplam += veri;
16
17 seçenek = JOptionPane.showConfirmDialog(null, "Devam et?");
18 }
19
20 JOptionPane.showMessageDialog(null, "Toplam "+ toplam);
21 }

```

yirmi iki }

onay seçeneği
kontrol seçeneği
giriş iletişim kutusu
onay iletişim kutusu
mesaj iletişim kutusu



Fiçin4.4 Kullanıcı (a)'da 3 giriyor, (b)'de Evet'e tıklıyor, (c)'de 5 giriyor, (d)'de Hayır'a tıklıyor ve sonuç (e)'de gösteriliyor.

Program, kullanıcıdan bir tam sayı girmesini istemek için bir giriş iletişim kutusu gösterir (satır 11) ve bunu **toplam** (satır 15) ile toplar. Satır 17, kullanıcının girişi devam ettirip ettirmeyeceğine karar vermesi için bir onay iletişim kutusu gösterir. Eğer kullanıcı *Evet*'e tıklarsa, döngü devam eder; aksi takdirde, döngü sona erer. Son olarak, program sonucu bir mesaj iletişim kutusu (satır 20) içinde gösterir.

showConfirmDialog metodu (satır 17) bir tamsayı döndürür **JOptionPane.YES_OPTION**, **JOptionPane.NO_OPTION** veya **JOptionPane.CANCEL_OPTION**, eğer kullanıcı *Evet*, *Hayır* veya *İptal* butonuna tıklarsa. Dönen değer **option** değişkenine atanır (satır 17). Eğer bu değer **JOptionPane.YES_OPTION** ise, döngü devam eder (satır 9).

Anahtar Terimler

break ifadesi 159
continue ifadesi 159
do-while döngüsü 144
for döngüsü 147
sonsuz döngü 136
girdi yönlendirmesi 143
iterasyon 134
döngü 134

döngü gövdesi 134
iç içe döngü 152
bir fazlalık hatası 136
çıkış yönlendirmesi 143
son test döngüsü 150
ön test döngüsü 150
sentinel değeri 141
while döngüsü 134

BÖLÜM ÖZETİ

1. Üç tür tekrarlama ifadesi vardır: **while** döngüsü, **do-while** döngüsü, ve **for** döngüsü.
2. Döngünün tekrarlanacak ifadeleri içeren kısmına *döngü gövdesi* denir.
3. Döngü gövdesinin bir kez çalıştırılmasına *döngünün yinelemesi* denir.
4. Bir *sonsuz döngü*, sonsuz bir şekilde çalışan bir döngü ifadesidir.
5. Döngü tasarlarırken, hem *döngü kontrol yapısını* hem de döngü gövdesini dikkate almanız gerekir.
6. **while** döngüsü, önce *döngü-devam-etme-kosulunu* kontrol eder. Eğer koşul *doğru* ise, döngü gövdesi çalıştırılır; eğer *yanlış* ise, döngü sona erer.
7. **do-while** döngüsü, **while** döngüsüne benzer, ancak **do-while** döngüsü önce döngü gövdesini çalıştırır ve ardından *döngü-devam-etme-kosulunu* kontrol eder devam edip etmeyeceğine veya sona ereceğine karar vermek için.
8. **while** döngüsü ve **do-while** döngüsü genellikle tekrar sayısı önceden belirlenmediğinde kullanılır.
9. Bir *sentinel değeri* döngünün sonunu belirten özel bir değerdir.
10. Döngü gövdesinin öngörülebilir bir sayıda çalıştırılması için genellikle **for** döngüsü kullanılır; bu sayı döngü gövdesi tarafından belirlenmez.
11. **for** döngü kontrolü üç bölümden oluşur. İlk bölüm genellikle bir kontrol değişkenini başlatan bir eylemdir. İkinci bölüm, *döngü-devam-etme-kosulu*, döngü gövdesinin çalıştırılıp çalıştırılmayacağını belirler. Üçüncü bölüm ise her yinelemeden sonra çalıştırılır ve genellikle kontrol değişkenini ayarlamak için kullanılır. Genellikle, döngü kontrol değişkenleri kontrol yapısında başlatılır ve değiştirilir.
12. **while** döngüsü ve **for** döngüsü *ön-test döngüleri* olarak adlandırılır çünkü devam koşulu döngü gövdesi çalıştırılmadan önce kontrol edilir.
13. **do-while** döngüsü *son-test döngüsü* olarak adlandırılır çünkü koşul döngü gövdesi çalıştırdıktan sonra kontrol edilir.
14. İki anahtar kelime, **break** ve **continue**, bir döngüde kullanılabilir.
15. **break** anahtar kelimesi, iç içe geçmiş en içteki döngüyü hemen sonlandırır, bu döngü **break** içerir.
16. **continue** anahtar kelimesi yalnızca mevcut yinelemeyi sonlandırır.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinden yapın.

PROGRAMLAMA EGZERSİZLERİ



Petdagtotgtictal Ntote

Her problemi birkaç kez okuyun, ta ki onu anlayana kadar. Kodu yazmaya başlamadan önce problemi nasıl çözeceğinizi düşünün. Mantiğinizi bir programa çevirin.

Bir problem genellikle birçok farklı şekilde çözülebilir. Öğrencilerin çeşitli çözümleri keşfetmeleri teşvik edilir.

Bölümler 4.2-4.7

***4.1** (*Pozitif ve negatif sayıları say ve sayıların ortalamasını hesapla*) Bir program yazın, belirsiz sayıda tam sayıyı okuyan, okunan pozitif ve negatif değerlerin sayısını belirleyen ve giriş değerlerinin toplamını ve ortalamasını hesaplayan (sıfırları saymadan). Programınız, giriş **0** ile sona erer. Ortalamayı ondalıklı bir sayı olarak gösterin. İşte bir örnek çalışma:

Bir tam sayı girin, giriş 0 ise sona erer:
Pozitif sayıların sayısı 3
Negatif sayıların sayısı 1
Toplam 5
Ortalama 1.25

1 2 -1 3 0 ↵ Enter



Bir tam sayı girin, giriş 0 ise sona erer:
0 dışında hiçbir sayı girilmedi

0 ↵ Enter



4.2 (*Tekrar toplama işlemleri*) Liste 4.4, SubtractionQuizLoop.java, beş rastgele çıkarma sorusu üretir. Programı, **1** ile **15** arasında iki tam sayı için on rastgele toplama sorusu üretecek şekilde gözden geçirin. Doğru sayısını ve test süresini gösterin.

4.3 (*Kilogramdan pounda dönüşüm*) Aşağıdaki tabloyu gösteren bir program yazın (not: **1** kilogram **2.2** pounda eşittir):

Kilogramlar	Poundlar
1	2.2
3	6.6
...	
197	433.4
199	437.8

4.4 (*Mil'den kilometreye dönüşüm*) Aşağıdaki tabloyu gösteren bir program yazın (not: 1 mil 1.609 kilometredir):

Miller	Kilometre
1	1.609
2	3.218
...	
9	14.481
10	16.090

4.5 (*Kilogramdan pounda ve poundan kilograma dönüşüm*) Aşağıdaki iki tabloyu yan yana gösteren bir program yazın (1 kilogramın 2.2 pound olduğunu ve 1 poundun 0.453 kilogram olduğunu unutmayın):

Kilogram	Pound	Pound	Kilogram
1	2.2		20
3	6.6		25
...			
197	433.4		510
199	437.8		515

9.09
11.36
231.82
234.09

- 4.6** (*Milesten kilometreye dönüşüm*) Aşağıdaki iki tabloyu yan yana gösteren bir program yazın (1 milin 1.609 kilometre olduğunu ve 1 kilometrenin 0.621 mil olduğunu u nutmayın):

Miller	Kilometre	Kilometre	Miller
1	1.609	20	12.430
2	3.218	25	15.538
...			
9	14.481	60	37.290
10	16.090	65	40.398

- **4.7** (*Finansal uygulama: gelecekteki öğrenim ücretini hesapla*) Bu yıl bir üniversitenin öğrenim ücretinin 10,000 \$ olduğunu ve her yıl %5 arttığını varsayalım. On yıl içinde öğrenim ücretini ve on yıl sonra başlayacak dört yıllık öğrenim ücretinin toplam maliyetini hesaplayan bir program yazın.
- 4.8** (*En yüksek puanı bul*) Kullanıcıdan öğrenci sayısını ve her öğrencinin adını ve puanını girmesini isteyen bir program yazın ve sonunda en yüksek puana sahip öğrencinin adını gösterin.
- *4.9** (*İki en yüksek puanı bul*) Kullanıcıdan öğrenci sayısını ve her öğrencinin adını ve puanını girmesini isteyen bir program yazın ve sonunda en yüksek puana sahip öğrenciyi ve ikinci en yüksek puana sahip öğrenciyi gösterin.
- 4.10** (*5 ve 6 ile tam bölünebilen sayıları bul*) 100 ile 1.000 arasında, her satırda on tane, 5 ve 6 ile tam bölünebilen tüm sayıları gösteren bir program yazın. Sayılar tam olarak bir boşlukla ayrılmalıdır.
- 4.11** (*5 veya 6 ile, ancak her ikisiyle değil, tam bölünebilen sayıları bul*) 100 ile 200 arasında, her satırda on tane, 5 veya 6 ile tam bölünebilen ancak her ikisiyle değil olan tüm sayıları gösteren bir program yazın. Sayılar tam olarak bir boşlukla ayrılmalıdır.
- 4.12** (*En küçük n 'yi bul ki $n^2 > 12.000$*) En küçük
tam sayı n 'yi bulmak için bir `while` döngüsü kullanın ki n^2 12.000'den büyük olsun.
- 4.13** (*En büyük n 'yi bul ki $n^3 < 12.000$*) En büyük
tam sayı n 'yi bulmak için bir `while` döngüsü kullanın ki n^3 12.000'den küçük olsun.

Bölümler 4.8–4.10

- *4.14** (*En büyük ortak böleni hesapla*) Liste 4.9 için iki tam sayının en büyük ortak bölenini bulmanın bir diğer çözümünü $n1$ ve $n2$ şu şekildedir: Öncelikle d 'yi $n1$ ve $n2$ 'nin minimumu olarak bul, ardından $d, d-1, d-2, \dots, 2$ veya 1 'in bu sırayla hem $n1$ hem de $n2$ için bir bölen olup olmadığını kontrol et. Bu şekilde bulunan ilk ortak bölen, $n1$ ve $n2$ için en büyük ortak böldür. Kullanıcıdan iki pozitif tam sayı girmesini isteyen ve gcd'yi gösteren bir program yazın.
- *4.15** (*ASCII karakter tablosunu göster*) ASCII karakter tablosundaki karakterleri yazdıran bir program yazın tablosu ! ile ~ arasında. Her satırda on karakter gösterin. ASCII tablosu Ek B'de gösterilmektedir. Karakterler tam olarak bir boşlukla ayrılmalıdır.
- *4.16** (*Bir tam sayının çarpanlarını bul*) Bir tam sayı okuyan ve bütün en küçük çarpanlarını artan sırayla gösteren bir program yazın. Örneğin, eğer girdi tam sayısı 120 ise, çıktı şu şekilde olmalıdır: 2,2,2,3,5.
- **4.17** (*Piramidi göster*) Kullanıcıdan bir tam sayı girmesini isteyen bir program yazın 1 15 ve aşağıdaki örnek çalışmada gösterildiği gibi bir piramit gösterin:

Satır sayısını girin:

7

Enter

```

      1
    2 1 2
  3 2 1 2 3
4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
6 5 4 3 2 1 2 3 4 5 6
7 6 5 4 3 2 1 2 3 4 5 6 7

```



*4.18 (Döngüler kullanarak dört desen göster) Aşağıdaki

desenleri dört ayrı programda gösteren iç içe döngüler kullanın:

Desen A	Desen B	Desen C	Desen D
1		1 2 3 4 5 6	1 1 2 3 4 5 6
1 2		1 2 3 4 5	2 1 1 2 3 4 5
1 2 3		1 2 3 4	3 2 1 1 2 3 4
1 2 3 4		1 2 3	4 3 2 1 1 2 3
1 2 3 4 5		1 2	5 4 3 2 1 1 2
1 2 3 4 5 6	1		6 5 4 3 2 1 1

4.19 (Sayıları bir piramit deseninde göster) Aşağıdaki çıktıyı yazdıran iç içe bir **for döngüsü yazın:

```

      1
    1 2 1
  1 2 4 2 1
1 2 4 8 4 2 1
1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
1 2 4 8 16 32 64 32 16 8 4 2 1
1 2 4 8 16 32 64 128 64 32 16 8 4 2 1

```

*4.20 (2 ile 1.000 arasındaki asal sayıları göster) Liste 4.14'ü, 2 ile 1.000 arasındaki tüm asal sayıları, dahil olmak üzere, gösterecek şekilde değiştirin. Her satırda sekiz asal sayı gösterin. Sayılar tam olarak bir boşlukla ayrılmalıdır.

Kapsamlı

**4.21 (Finansal uygulama: çeşitli faiz oranlarıyla kredileri karşılaştır) Kullanıcının kredi miktarını ve kredi süresini yıl cinsinden girmesine olanak tanıyan bir program yazın ve 5%'ten 8%'e kadar, 1/8 artışla her faiz oranı için aylık ve toplam ödemeleri gösterin. İşte bir örnek çalışma:

Kredi Miktarı:	10000	Enter
Yıl Sayısı:	5	Enter
Faiz Oranı	Aylık Ödeme	Toplam Ödeme
5.000%	188.71	11322.74
5.125%	189.28	11357.13
5.250%	189.85	11391.59
...		
7.875%	202.17	12129.97
8.000%	202.76	12165.83



Aylık ödemeyi hesaplamak için formül için Liste 2.8'e bakın, ComputeLoan.java.



VideoNotu
Kredi takvimini göster

****4.22** (Finansal uygulama: kredi amortisman planı) Verilen bir kredi için aylık ödeme, anapara ve faizi öder. Aylık faiz, aylık faiz oranı ile bakiye (kalan anapara) çarpılarak hesaplanır. Aylık ödenen anapara, bu nedenle aylık ödeme ile aylık faiz arasındaki farktır. Kullanıcının kredi miktarını, yıl sayısını ve faiz oranını girmesine izin veren ve kredinin amortisman planını gösteren bir program yazın. İşte bir örnek çalışma:



Kredi Miktarı: 10000

↵ Enter

Yıl Sayısı: 1

↵ Enter

Yıllık Faiz Oranı: 7

↵ Enter

Aylık Ödeme: 865.26

Toplam Ödeme: 10383.21

Ödeme#	Faiz	Anapara	Bakiye
1	58.33	806.93	9193.07
2	53.62	811.64	8381.43
...			
11	10.0	855.26	860.27
12	5.01	860.25	0.01



Not

Son ödemenin ardından bakiye sıfır olmayabilir. Eğer öyleyse, son ödeme normal aylık ödemenin yanı sıra son bakiyeyi de içermelidir.

İpucu: Tabloyu görüntülemek için bir döngü yazın. Aylık ödeme her ay için aynı olduğundan, döngüden önce hesaplanmalıdır. Bakiye başlangıçta kredi miktarıdır. Döngüde her yineleme için, faizi ve anaparayı hesaplayın ve bakiyeyi güncelleyin. Döngü şu şekilde görünebilir:

```
for(i = 1; i <= numberOfYears * 12; i++) {
    faiz = aylık
    Faiz Oranı * bakiye;
    nprincipal = aylıkÖdeme - faiz;
    nbakiye = bakiye - anapara;
}
```

```
System.out.println(i + "t" + faiz
    + "t" + anapara + "t" + bakiye);
}
```

***4.23** (Daha doğru sonuçlar elde etmek için) Aşağıdaki seriyi hesaplarken, soldan sağa değil, sağdan sola hesaplayarak daha doğru sonuçlar elde edersiniz:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Soldan sağa ve sağdan sola **n = 50000** ile önceki serinin toplamını hesaplayan bir program yazın.

***4.24** (Bir seriyi topla) Aşağıdaki seriyi toplamak için bir program yazın:

$$\frac{3}{5} + \frac{5}{7} + \frac{7}{9} + \frac{9}{11} + \frac{11}{13} + \dots + \frac{95}{97} + \frac{97}{99}$$



VideoNotu
Bir seriyi topla

****4.25** (Hesapla) Yaklaşık olarak hesaplayabilirsiniz π aşağıdaki seriyi kullanarak:

$$\pi = \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots + \frac{4(-1)^{t+1}}{2t-1}$$

Değeri görüntüleyen bir program yazın π $i = 10000, 20000, \dots$, ve **100000**.

****4.26** (e'yi hesapla) Aşağıdaki seriyi kullanarak e'yi yaklaşık olarak hesaplayabilirsiniz:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots + \frac{1}{i!}$$

$i = 10000, 20000, \dots$, ve

100000. (İpucu: Çünkü $\frac{1}{i!} = \frac{1}{i \cdot (i-1)!}$)

$$\frac{1}{i!} = \frac{1}{i \cdot (i-1)!}$$

e'yi ve item'ı olarak başlatın ve yeni bir item'ı e'ye eklemeye devam edin. Yeni item, $i = 2, 3, 4, \dots$ için önceki item'ın i'ye bölünmesidir.

****4.27** (Artık yılları görüntüle) Tüm artık yılları, satır başına on tane olacak şekilde, 21. yüzyılda (2001'den 2100'e kadar) görüntüleyen bir program yazın, tam olarak bir boşlukla ayrılmıştır.

****4.28** (Her ayın ilk günlerini görüntüle) Kullanıcıdan yılı ve yılın ilk gününü girmesini isteyen bir program yazın, yıldaki her ayın ilk gününü konsolda görüntülesin. Örneğin, kullanıcı yılı **2013** ve **2** olarak girdiğinde Salı, 1 Ocak 2013, programınız aşağıdaki çıktıyı göstermelidir:

1 Ocak 2013 Salı'dır

...

1 Aralık 2013 Pazar günüdür

****4.29** (Takvimleri görüntüle) Kullanıcıdan yılı ve yılın ilk gününü girmesini isteyen bir program yazın ve konsolda yıl için takvim tablosunu görüntüleyin. Örneğin, kullanıcı yılı **2013** ve **2** olarak girdiğinde, 1 Ocak 2013 Salı, programınız yılın her ayı için takvimi aşağıdaki gibi görüntülemelidir:

Ocak 2013						
Güneş	Paz	Sal	Çar	Per	Cum	Cmt
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		
.						
.						
.						
Aralık 2013						
Güneş	Paz	Sal	Çar	Per	Cum	Cmt
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

***4.30** (Finansal uygulama: bileşik değer) Her ay tasarruf hesabınıza %5 yıllık faiz oranıyla \$100 yatırdığınızı varsayalım. Böylece, aylık faiz oranı $0.05 / 12 = 0.00417$. İlk aydan sonra, hesapta bulunan değer

$$100 * (1 + 0.00417) = 100.417$$

İkinci aydan sonra, hesapta bulunan değer

$$(100 + 100.417) * (1 + 0.00417) = 201.252$$

Üçüncü aydan sonra, hesapta bulunan değer

$$(100 + 201.252) * (1 + 0.00417) = 302.507$$

ve devam eder.

Kullanıcıdan bir miktar (örneğin, 100), yıllık faiz oranı (örneğin, 5) ve ay sayısını (örneğin, 6) girmesini isteyen ve belirtilen aydan sonra tasarruf hesabında ki miktarı gösteren bir program yazın.

***4.31** (Finans uygulaması: CD değerini hesapla) \$10,000'ı %5.75 yıllık yüzdelik getirisi ile bir CD'ye koyduğunuzu varsayalım. Bir ay sonra, CD'nin değeri

$$10000 + 10000 * 5.75 / 1200 = 10047.91$$

İki ay sonra, CD'nin değeri

$$10047.91 + 10047.91 * 5.75 / 1200 = 10096.06$$

Üç ay sonra, CD'nin değeri

$$10096.06 + 10096.06 * 5.75 / 1200 = 10144.43$$

ve devam eder.

Kullanıcıdan bir miktar (örneğin, 10000), yıllık yüzde getirisi (örneğin, 5.75) ve ay sayısını (örneğin, 18) girmesini isteyen bir program yazın ve örnek çalışım adaki gibi bir tablo gösterin.



Başlangıç mevduat miktarını girin: 10000

Yıllık yüzde getirisi girin: 5.75

Vade süresini girin (ay sayısı): 18

Ay CD Değeri

1	10047.91
2	10096.06
⋮	
17	10846.56
18	10898.54

****4.32** (Oyun: piyango) Listing 3.9, Lottery.java dosyasını, iki basamaklı bir piyango oluşturacak şekilde revize edin. Sayıdaki iki basamak farklıdır. (İpucu: İlk basamağı oluşturun. İlk basamaktan farklı olana kadar ikinci basamağı sürekli olarak oluşturmak için bir döngü kullanın.)

****4.33** (*Mükemmel sayı*) Pozitif bir tam sayı, kendisi hariç tüm pozitif bölenlerinin toplamına eşit ise *mükemmel sayı* olarak adlandırılır. Örneğin, 6, $6 = 3 + 2 + 1$ olduğu için ilk mükemmel sayıdır. Sonraki ise $28 = 14 + 7 + 4 + 2 + 1$. 10.000'den küçük dört mükemmel sayı vardır. Bu dört sayıyı bulmak için bir program yazın.

****4.34** (*Oyun: makas, taş, kağıt*) Alıştırma 3.17, makas-taş-kağıt oyununu oynayan bir program verir. Programı, kullanıcının ya da bilgisayarın iki kezden fazla kazanana kadar sürekli oynamasına izin verecek şekilde revize edin.

***4.35** (*Toplama*) Aşağıdaki toplamı hesaplamak için bir program yazın.

$$\frac{1}{1 + \sqrt{2}} + \frac{1}{\sqrt{2} + \sqrt{3}} + \frac{1}{\sqrt{3} + \sqrt{4}} + \dots + \frac{1}{\sqrt{624} + \sqrt{625}}$$

****4.36** (*Finans uygulaması: ISBN kontrolü*) Döngüleri kullanarak Egzersiz 3.9'u basitleştirin.

****4.37** (*Onluk tam sayıdan ikiliye*) Kullanıcıdan bir onluk tam sayı girmesini isteyen ve karşılık gelen ikili değerini gösteren bir program yazın. Java'nın `Integer.toBinaryString(int)` bu programda.

****4.38** (*Onluk tam sayıyı onaltılıya çevir*) Kullanıcıdan bir onluk tam sayı girmesini isteyen ve karşılık gelen onaltılı değerini gösteren bir program yazın. Java'nın `Integer.toHexString(int)` bu programda.

***4.39** (*Finans uygulaması: satış tutarını bulma*) Bir mağazada satış işine yeni başladınız. Maaşınız, bir taban maaşı ve bir komisyon içerir. Taban maaşı 5,000 \$'dır. Aşağıda gösterilen şema, komisyon oranını belirlemek için kullanılır.

Satış Tutarı	Komisyon Oranı
\$0.01–\$5,000	%8
\$5,000.01–\$10,000	%10
\$10,000.01 ve üzeri	%12

Amacınız yılda \$30,000 kazanmak. \$30,000 kazanmak için üretmeniz gereken minimum satış sayısını bulan bir program yazın.

****4.40** (*Simülasyon: yazı-tura*) Bir madeni parayı bir milyon kez atmayı simüle eden ve yazı ve tura sayısını gösteren bir program yazın.

****4.41** (*Maksimum sayıların tekrar sayısı*) Tam sayıları okuyan, en büyüğünü bulan ve tekrar sayısını sayan bir program yazın. Girişin 0 sayısı ile sona erdiğini varsayın.

Diyelim ki 3 5 2 5 5 0 girdiniz; program en büyük sayının 5 olduğunu ve 5 için tekrar sayısının 4 olduğunu bulur.

(İpucu: İki değişkeni, `max` ve `count` olarak tutun. `max` mevcut maksimum sayıyı, `count` ise tekrar sayısını saklar. Başlangıçta, ilk sayıyı `max`'ye ve 1 `count`'ye atayın. Her sonraki sayıyı `max` ile karşılaştırın. Eğer sayı `max`'den büyükse, onu `max`'ye atayın ve `count`'yi 1 olarak sıfırlayın. Eğer sayı `max` ile eşitse, `count`'yi 1 artırın.)

Sayıları girin: 3 5 2 5 5 0 → Enter
En büyük sayı 5'tir
En büyük sayının tekrar sayısı 4'tür



***4.42** (Finans uygulaması: satış tutarını bulma) Alıştırma 4.39'u aşağıdaki gibi yeniden yazın:

- Bir **for** döngüsü kullanın, **do-while** döngüsü yerine.
- Kullanıcının **COMMISSION_SOUGHT** girmesine izin verin, sabit olarak ayarlamak yerine.

***4.43** (Simülasyon: geri sayım saatli simülasyon) Kullanıcıdan saniye sayısını girmesini isteyen bir program yazın, saniyede bir mesaj gösterin ve süre dolduğunda sonlandırın. İşte bir örnek çalışma:



Saniye sayısını girin: 3

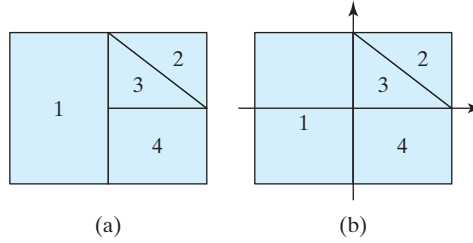
2 saniye kaldı

1 saniye kaldı

Durduruldu

****4.44** (Monte Carlo simülasyonu) Bir kare, aşağıda (a) gösterildiği gibi dört daha küçük bölgeye bölünmüştür. Kareye 1.000.000 kez bir ok atarsanız, bir okun tek numaralı bir bölgeye düşme olasılığı nedir? Süreci simüle eden bir program yazın ve sonucu gösterin.

(İpucu: Karenin merkezini bir koordinat sisteminin merkezine yerleştirin, aşağıda (b) gösterildiği gibi. Kare içinde rastgele bir nokta oluşturun ve bir noktanın tek numaralı bir bölgeye düşme sayısını sayın.)



***4.45** (Matematik: kombinasyonlar) 1 ile 7 arasındaki tam sayılardan iki sayı seçmek için mümkün olan tüm kombinasyonları gösteren bir program yazın. Ayrıca tüm kombinasyonların toplam sayısını gösterin.



1 2
1 3
...
...

Tüm kombinasyonların toplam sayısı 21'dir.

***4.46** (Bilgisayar mimarisi: bit düzeyinde işlemler) Kısa bir değer 16 bitte saklanır. Bir kullanıcıdan kısa bir tam sayı girmesini isteyen ve bu tam sayının 16 bitini gösteren bir program yazın. İşte örnek çalıştırmalar:



Bir tam sayı girin: 5

Bitler 0000000000000101

Bir tam sayı girin:
 Bitler 111111111111011



(*İpucu:* Bit düzeyinde sağa kaydırma operatörünü (>>) ve bit düzeyinde VE operatörünü (&) kullanmanız gerekiyor; bunlar Ek G'de, Bit Düzeyinde İşlemler bölümünde ele alınmıştır.)

****4.47** (*İstatistik: ortalama ve standart sapmayı hesapla*) İş uygulamalarında, verilerin ortalamasını ve standart sapmasını hesaplamamız sıkça istenir. Ortalama, sayıların basitçe ortalamasıdır. Standart sapma, verilerin bir veri setindeki ortalama etrafında ne kadar sıkı bir şekilde kümelendiğini gösteren bir istatistiktir. Örneğin, bir sınıftaki öğrencilerin ortalama yaşı nedir? Yaşlar ne kadar yakın? Tüm öğrenciler aynı yaştaysa, sapma 0'dır.

Kullanıcıdan on sayı girmesini isteyen ve bu sayıların ortalamasını ve standart sapmalarını aşağıdaki formülü kullanarak gösteren bir program yazın:

$$\text{ortalama} = \frac{\sum_{i=1}^n x_i}{n} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

$$\text{sapma} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

İşte bir örnek çalışma:

On sayı girin:
 Ortalama 5.61'dir
 Standart sapma 2.99794'tür



Bu sayfa kasıtlı olarak boş bırakılmıştır

BÖLÜM

5

Yöntemler

Hedefler

- Resmi parametrelerle yöntemleri tanımlamak (§5.2).
- Gerçek parametrelerle (yani, argümanlarla) yöntemleri çağırmak (§5.2).
- Bir dönüş değeri olan yöntemleri tanımlamak (§5.3).
- Bir dönüş değeri olmayan yöntemleri tanımlamak (§5.4).
- Argümanları değerle geçirmek (§5.5).
- Modüler, okunması kolay, hata ayıklaması kolay ve bakımı kolay yeniden kullanılabilir kod geliştirmek (§5.6).
- Ondalık sayıları onaltılı sayılara dönüştüren bir yöntem yazmak (§5.7).
- Yöntem aşırı yüklemesini kullanmak ve belirsiz aşırı yüklemeyi anlamak (§5.8).
- Değişkenlerin kapsamını belirlemek (§5.9).
- Matematik problemlerini **Math** sınıfındaki yöntemleri kullanarak çözmek (§§5.10–5.11).
- Yazılım geliştirmede yöntem soyutlama kavramını uygulamak (§5.12).
- Adım adım iyileştirme kullanarak yöntemler tasarlamak ve uygulamak (§5.12).



5.1 Giriş



Yeniden kullanılabilir kod tanımlamak ve kodu düzenlemek ve basitleştirmek için yöntemler kullanılabilir.

sorun

1'den 10'a, 20'den 37'ye ve 35'ten 49'a kadar olan tam sayıların toplamını bulmanız gerektiğini varsayalım. Kodu aşağıdaki gibi yazabilirsiniz:

```
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 37; i++)
    sum += i;
System.out.println("Sum from 20 to 37 is " + sum);

sum = 0;
for (int i = 35; i <= 49; i++)
    sum += i;
System.out.println("Sum from 35 to 49 is " + sum);
```

Bu toplamları 1 ile 10, 20 ile 37 arasında ve 35 49 çok benzer, tek fark başlangıç ve bitiş tam sayılarının farklı olmasıdır. Olmaz mıydı Ortak kodu bir kez yazıp yeniden kullanabilirsek harika olurdu, değil mi? Bunu bir

neden yöntemler?

metod tanımlayarak ve onu çağırarak yapabiliriz.

Önceki kod aşağıdaki gibi basitleştirilebilir:

tanımla toplam yöntemi

```
1 genel statik int toplam(renk tonu 1, renk tonu 2) {
2     int sonuç = 0;
3     for(tint = i1; i <= i2; i++)
4         sonuç += i;
5     dönüş sonucu;
6 }
7
8
9 public static void main(String[] args) {
10     System.out.println("1'den 10'a kadar toplam " + nsum(t1,t10t) );
11     System.out.println("20'den 37'ye kadar toplam " + )nsum(t20t,t37t) );
12     System.out.println("35'ten 49'a kadar toplam " + toplam(t35t,t49t) );
13 }
```

ana yöntem
çağır toplam

yöntem

1–7. satırlar, **sum** adında iki parametreye sahip bir yöntemi tanımlar **i1** ve **i2**.

main yöntemindeki ifadeler, **sum(1, 10)** çağrısını yaparak 1 ile 10 arasındaki toplamı, **sum(20, 37)** çağrısını yaparak 20 ile 37 arasındaki toplamı ve **sum(35, 49)** çağrısını yaparak 35 ile 49 arasındaki toplamı hesaplar.

Bir *yöntem*, bir işlemi gerçekleştirmek için bir araya getirilmiş ifadeler topluluğudur. Önceki bölümlerde **System.out.println**,

JOptionPane.showMessageDialog, **System.exit**, **Math.pow** ve **Math.random** gibi önceden tanımlanmış yöntemleri kullandınız.

Bu yöntemler Java kütüphanesinde tanımlanmıştır. Bu bölümde, kendi yöntemlerinizi nasıl tanımlayacağınızı ve karmaşık problemleri çözmek için yöntem soyutlamasını nasıl uygulayacağınızı öğreneceksiniz.

5.2 Bir Yöntem Tanımlama

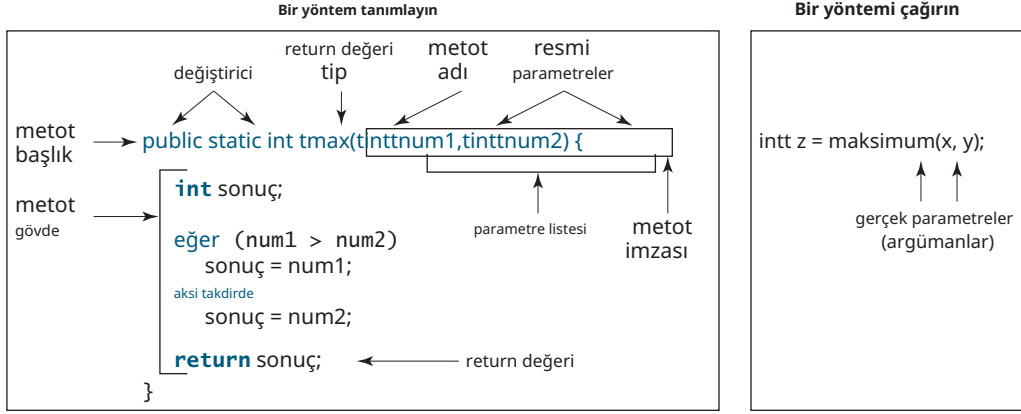


Bir yöntem tanımı, yöntem adı, parametreler, dönüş değeri türü ve gövdesinden oluşur.

Bir yöntemi tanımlamak için sözdizimi:

```
modifier returnType methodName(parametre listesi) {
    // Yöntem gövdesi;
}
```

İki tam sayı arasında daha büyük olanı bulmak için tanımlanmış bir yönteme bakalım. Bu yöntem, adlandırılmıştır **max**, iki **int** parametreye **num1** ve **num2** sahiptir, bunlardan daha büyüğü yöntem tarafından döndürülür. Şekil 5.1 bu yöntemin bileşenlerini göstermektedir.



ŞEKİL 5.1 Bir metod tanımı, bir metod başlığı ve bir metod gövdesinden oluşur.

Yöntem başlığı, değiştiricileri, geri dönüş değeri türü, yöntem adı ve parametreleri belirtir. Bu bölümdeki tüm yöntemler için **static** değiştiricisi kullanılmaktadır. Bunu kullanmanın nedeni Bölüm 8, Nesneler ve Sınıflar'da tartışılacaktır.

Bir yöntem bir değer döndürebilir. **returnValueType**, yöntemin döndürdüğü değer için veri türüdür. Bazı yöntemler, bir değer döndürmeden istenen işlemleri gerçekleştirir. Bu durumda, **returnValueType** anahtar kelimesi **void**'dir. Örneğin, **returnValueType void, main** yönteminde, ayrıca **System.exit, System.out.println** ve **JOptionPane.showMessageDialog**'da geçerlidir. Eğer bir yöntem bir değer döndürüyorsa,

bu, value-returning method olarak adlandırılır, aksi takdirde void method olarak adlandırılır.

Yöntem başlığında tanımlanan değişkenler, *biçimsel parametreler* veya kısaca *parametreler* olarak bilinir. Bir parametre, bir yer tutucu gibidir: Bir yöntem çağrıldığında, bir değeri parametreye geçirirsiniz. Bu değer, *gerçek parametre* veya *argüman* olarak adlandırılır. *Parametre listesi*, yöntemin türünü, sırasını ve parametre sayısını ifade eder. Yöntem adı ve parametre listesi birlikte yöntem imzasını oluşturur. Parametreler isteğe bağlıdır; yani, bir yöntemin herhangi bir parametre içermesi gerekmez. Örneğin, **Math.random()** yöntemi parametre içermez.

Yöntem gövdesi, yöntemi uygulayan bir dizi ifadeyi içerir. **max** yönteminin metod gövdesi, hangi sayının daha büyük olduğunu belirlemek ve o sayının değerini döndürmek için bir **if** ifadesi kullanır. Değer döndüren bir yöntemin bir sonuç döndürmesi için, **return** anahtar kelimesini kullanan bir return ifadesi *gereklidir*. Yöntem, bir return ifadesi yürütüldüğünde sona erer.



Not

Bazı programlama dilleri, yöntemlere prosedürler ve fonksiyonlar olarak atıfta bulunur. Budiller de, değer döndüren bir yöntem fonksiyon olarak adlandırılır ve void yöntemi prosedür olarak adlandırılır.



Dikkat

Yöntem başlığında, her bir parametreyi ayrı ayrı tanımlamanız gerekir. Örneğin, **max(int num1, int num2)** doğrudur, ancak **max(int num1, num2)** yanlıştır.

yöntem başlığı
değiştirici

değer döndüren yöntem
void yöntem
resmi parametre
parametre
gerçek parametre
argüman
parametre listesi
yöntem imzası

tanımla vs. beyan et

**Not**

"anımla bir metot" ve "bildir bir değişken" diyoruz. Burada ince bir ayırım yapıyoruz. Bir tanım, tanımlanan öğenin ne olduğunu tanımlar, ancak bir bildirim genellikle bildirilen öğe için veri depolamak üzere bellek ayırmayı içerir.

**Ana
Nokta**

5.3 Bir Metodu Çağırma

Bir metodu çağırmak, metot içindeki kodu çalıştırır.

Bir metot tanımında, metodun ne yapacağını tanımlarsınız. Metodu çalıştırmak için onu çağırmanız ve eya tetiklemeniz gerekir. Bir metodu çağırmanın iki yolu vardır; bu, metodun bir değer döndürüp döndürmediğine bağlıdır.

Eğer bir metot bir değer döndürüyorsa, metoda yapılan bir çağrı genellikle bir değer olarak değerlendirilir. Örneğin,

```
intbüyüktür = max(3, 4);
```

çağrılar `max(3, 4)` ve metodun sonucunu `larger` değişkenine atar. Değer olarak ele alınan bir çağrının başka bir örneği

```
System.out.println(max(t3t,t4t));
```

metod çağrısının döndürdüğü değeri `max(3, 4)` yazdırır.

Eğer bir metot `void` döndürüyorsa, metod çağrısı bir ifade olmalıdır. Örneğin, metod `println void` döndürür. Aşağıdaki çağrı bir ifadedir:

```
System.out.println("Java'ya hoş geldiniz!");
```

**Not**

Bir değer döndüren yöntem, Java'da bir ifade olarak da çağrılabilir. Bu durumda, çağırılan basitçe dönen değeri göz ardı eder. Bu sık yapılmaz, ancak çağırılan dönen değere ilgi duymuyorsa mümkündür.

Bir program bir yöntemi çağırdığında, program kontrolü çağrılan yönteme aktarılır. Bir çağrılan yöntem, dönüş ifadesi çalıştırıldığında veya yöntem sonlandırma kapama süresi ulaşıldığında kontrolü çağırana geri döner.

Liste 5.1, `max` yöntemini test etmek için kullanılan tam bir programı göstermektedir.

LISTE 5.1 TestMax.java

```
1 genel sınıf TestMax {
2     /** Ana yöntem */
3     public static void tmain(String[] args) {
4         int t5t;
5         int t2t;
6         int k = maksimum(i, j);
7         System.out.println("En büyük "+ i +
8             " ve "+ j + " şudur: " + k);
9     }
10
11     /** İki sayının maksimumunu döndür */
12     genel statik int tmax(int tnum1, int tnum2) {
13         int sonuç;
14
15         if (num1 > num2)
16             sonuç = num1;
17         aksi takdirde
18             sonuç = num2;
19
20         dönüş sonucu;
21     }
22 }
```

**VideoNotu**

Tanımla/çağır max yöntemi

ana yöntemi

çağırmax

yöntemi tanımla

5 ve 2'nin maksimumu 5'tir



	satır#	i	j	k	num1	num2	sonuç
	4	5					
	5		2				
maksimum çağrılıyor	12				5	2	
	13						tanımsız
	16						5
	6			5			

Bu program **ana** yöntemi ve **max** yöntemini içerir. **ana** yöntemi, programı başlatmak için JVM tarafından çağrılan diğer yöntemler gibi bir yöntemdir.

ana yöntemi

ana yönteminin başlığı her zaman aynıdır. Bu örnekteki gibi, değiştiricileri public static, dönüş değeri türü void, yöntem adı main ve bir parametre String[] türündedir. String[], parametrenin bir String dizisi olduğunu gösterir, bu konu Bölüm 6'da ele alınmıştır.

Açıklamalar **main** içinde, **main** yöntemini içeren sınıfta veya diğer sınıflarda tanımlanan diğer yöntemleri çağırabilir. Bu örnekte, **main** yöntemi **max(i,j)**'yi çağırır; bu, **main** yöntemi ile aynı sınıfta tanımlanmıştır.

max yöntemi çağrıldığında (satır 6), değişken **i**'nin değeri 5, **num1**'ye, ve değişken **j**'nin değeri 2, **num2**'ye **max** yönteminde geçerli. Kontrol akışı **max** yöntemine aktarılır ve **max** yöntemi yürütülür. **max** yöntemindeki **return** ifadesi yürütüldüğünde, **max** yöntemi kontrolü çağırana geri döner (bu durumda çağırana **main** yöntemidir). Bu süreç Şekil 5.2'de gösterilmektedir.

maksimum yöntemi

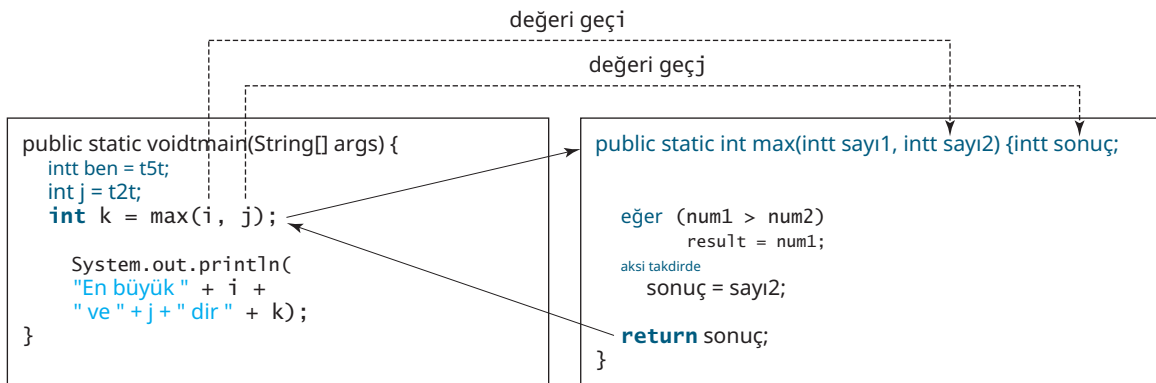


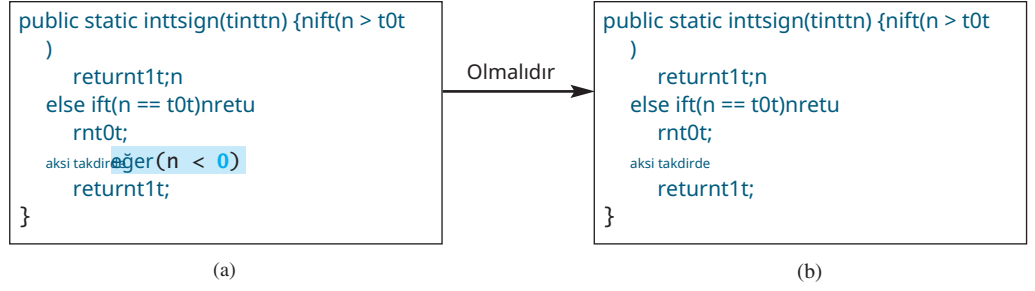
FIGURE 5.2 **max** yöntemi çağrıldığında, kontrol akışı ona aktarılır. **max** yöntemi tamamlandığında, çağırıcıya kontrol geri döner.



Dikkat

Bir **return** ifadesi, bir değer döndüren yöntem için gereklidir. Aşağıda gösterilen yöntem (a) mantıksal olarak doğrudur, ancak Java derleyicisi

bu yöntemin bir değer döndürmeyeceğini düşündüğü için bir derleme hatası vardır.



Bu sorunu düzeltmek için, derleyicinin `if` ifadesinin nasıl değerlendirildiğinden bağımsız olarak bir `return` durumuna ulaşacağını görmesi için (a) içindeki `if (n < 0)` ifadesini silin.



Not

Yöntemler kod paylaşımını ve yeniden kullanımını sağlar. `max` yöntemi, yalnızca `TestMax` değil, herhangi bir sınıftan çağrılabilir. Yeni bir sınıf oluşturursanız, `ClassName.methodName` (yani, `TestMax.max`) kullanarak `max` yöntemini çağırabilirsiniz.

yöntemi yeniden kullanma

aktivasyon kaydı

çağrı yığını

Her seferinde bir yöntem çağrıldığında, sistem, yöntemin parametrelerini ve değişkenlerini depolayan bir *activation record* (aynı zamanda *activation frame* olarak da adlandırılır) oluşturur ve bu aktivasyon kaydını c all stack olarak bilinen bir bellek alanına yerleştirir. Bir çağrı yığını, *execution stack*, *runtime stack* veya *machine stack* olarak da bilinir ve genellikle sadece “yığın” olarak kısaltılır. Bir yöntem başka bir yöntemi çağırıldığında, çağırının aktivasyon kaydı sağlam kalır ve çağrılan yeni yöntem için yeni bir aktivasyon kaydı oluşturulur. Bir yöntem işini bitirdiğinde ve çağırana döndüğünde, aktivasyon kaydı çağrı yığından kaldırılır.

Bir çağrı yığını, aktivasyon kayıtlarını son giren ilk çıkar (LIFO) şeklinde depolar: En son çağrılan yöntemin aktivasyon kaydı, yığından ilk olarak kaldırılır. Örneğin, `m1` yöntemi `m2` yöntemini ve ardından `m3` yöntemini çağırırsa. Çalışma zamanı sistemi, önce `m1`'in aktivasyon kaydını yığına iter, sonra `m2`'yi, ardından `m3`'yi. `m3` tamamlandığında, aktivasyon kaydı yığından kaldırılır. `m2` tamamlandığında, aktivasyon kaydı yığından kaldırılır. `m1` tamamlandığında, aktivasyon kaydı yığından kaldırılır.

Çağrı yığınlarını anlamak, yöntemlerin nasıl çağrıldığını kavramanıza yardımcı olur. Listing 5.1'deki `main` yönteminde tanımlanan değişkenler `i`, `j` ve `k`'dir. `max` yönteminde tanımlanan değişkenler `num1`, `num2` ve `result`'dir. `num1` ve `num2` değişkenleri yöntem imzasında tanımlanmış ve `max` yönteminin parametreleridir. Değerleri, yöntem çağrısı aracılığıyla geçerilir. Şekil 5.3, yığındaki yöntem çağrıları için aktivasyon kayıtlarını göstermektedir.

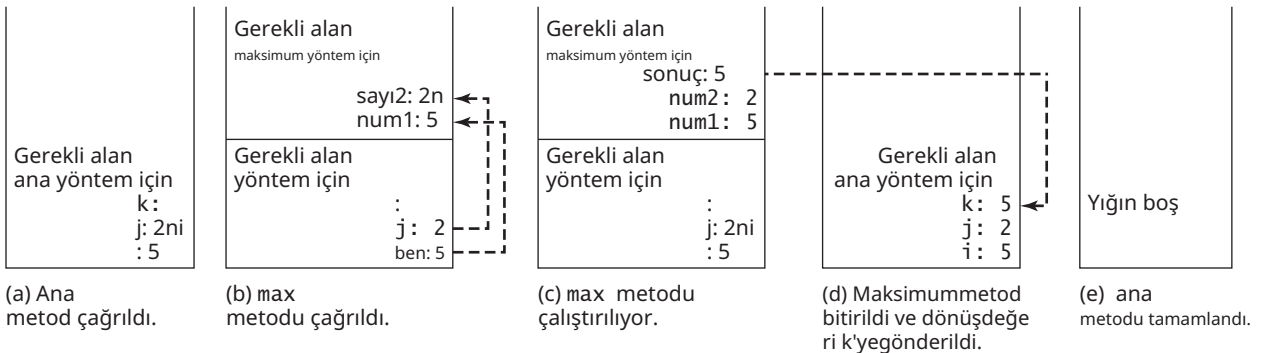


FIGURE 5.3 `max` metodu çağrıldığında, kontrol akışı `max` metoduna aktarılır. `max` metodu tamamlandığında, kontrol geri çağırana döner.

5.4void Yöntem Örneği

Avoid yöntemi bir değer döndürmez.

Önceki bölüm, bir değer döndüren bir yöntemin örneğini vermektedir. Bu bölüm, bir **void** yönteminin nasıl tanımlanacağını ve çağrılacağını göstermektedir. Listing 5.2, printGrade adında bir yöntem tanımlayan ve belirli bir puan için notu yazdırmak üzere onu çağıran bir program vermektedir.

Listing 5.2 testVoidMethod.java

```

1  genel classtestVoidMethod {
2      public static voidmain(String[] args) {
3          System.out.print("Not: ");
4          printGrade(t78.5t);

6      System.out.print("Not: ");
7      printGrade(t59.5t);
8  }
9
10     public static voidtprintGrade(tdoubletscore) {
11         eğer(score >= 90.0) {
12             System.out.println('A');
13         }
14         aksi takdirde eğer(score >= 80.0) {
15             System.out.println(t'B't);n16 }

17         aksi takdirde eğer(score >= 70.0) {
18             System.out.println(t'C't);n19 }

20         aksi takdirde eğer(score >= 60.0) {
21             System.out.println(t'D't);n22 }

23         aksi takdirde{
24             System.out.println(t'F't);n25 }

        asla {revize e
        dilmedi}

```



VideoNotu

Kullanvoid metodu

ana yöntem

çağır printGrade

printGrade yöntemi



Not C
Not F

printGrade yöntemi bir **void** yöntemdir çünkü herhangi bir değer döndürmez. Bir **void** yöntemine yapılan çağrı bir ifade olmalıdır. Bu nedenle, **main**

yönteminde 4. satırda bir ifade olarak çağrılır. Herhangi bir Java ifadesi gibi, bir noktalı virgül ile sonlandırılır.

Bir void ve değer döndüren yöntem arasındaki farkları görmek için,

printGrade yöntemini bir değer döndürecek şekilde yeniden tasarlayalım. Yeni yöntem, **getGrade** olarak adlandırdığımız, notu döndürür

5.3. Listesinde gösterildiği gibi.

LISTE5.3TestReturnGradeMethod.java

```

1  genel classtestReturnGradeMethod {
2      public static voidtmain(String[] args) {
3          System.out.print("Not "4      System. + etGrade(t78.5t) );
        out.print("Not "5      }          + getGrade(t59.5t)g );

```

ana yöntem

çağır getGrade

getGrade yöntemi

```

87 genel statik chartgetGrade(tdoubletscore) {
9     eğer(score >= 90.0)
10        dönüş 'A';
11     aksi takdirde eğer(score >= 80.0)
12        dönüş 'B';
13     aksi takdirde eğer(score >= 70.0)
14        dönüş 'C';
15     aksi takdirde eğer(score >= 60.0)
16        dönüş 'D';
17     aksi takdirde
18        dönüş 'F';
19 }

```



Not C
Not F

7–18. satırlarda tanımlanan **getGrade** yöntemi, sayısal

puan değerine dayalı bir karakter notu döndürür. Çağırın, bu yöntemi 3–4. satırlarda çağırır.

getGrade yöntemi, bir karakterin görünebileceği her yerde çağırın tarafından çağırılabilir.

printGrade yöntemi herhangi bir değer döndürmez, bu nedenle bir ifade olarak çağırılmalıdır.

void yönteminde dönüş



Not

void yöntemi için bir **return** ifadesine ihtiyaç yoktur, ancak yöntemi sonlandırmak ve yöntemin çağırıcısına geri dönmek için kullanılabilir. Söz dizimi basitçe

geri dönüş;

Bu sık yapılmaz, ancak bazen bir **void** yönteminde normal kontrol akışını aşmak için yararlıdır. Örneğin, aşağıdaki kod, puan geçersiz olduğunda yöntemi sonlandırmak için bir **return** ifadesine sahiptir.

```

public static void printGrade(tdoubletscore) {
    if (score < 0 || puan > 100) {
        System.out.println("Geçersiz puan");
        return;
    }

    eğer(score >= 90.0) {
        System.out.println('A');
    }
    aksi takdirde eğer(score >= 80.0) {
        System.out.println('B');
    }
    aksi takdirde eğer(score >= 70.0) {
        System.out.println('C');
    }
    aksi takdirde eğer(score >= 60.0) {
        System.out.println('D');
    }
    aksi takdirde {
        System.out.println('F');
    }
}

```


- 5.1** Bir yöntemi kullanmanın faydaları nelerdir?
- 5.2** Bir yöntemi nasıl tanımlarsınız? Bir yöntemi nasıl çağırırsınız?
- 5.3** Koşullu operatörü kullanarak 5.1 numaralı listedeki **max** yöntemini nasıl basitleştirirsiniz?
- 5.4** Doğru mu yanlış mı? **void** dönüş türüne sahip bir yömeme yapılan bir çağrı her zaman bir ifadedir, bir değer döndüren yömeme yapılan bir çağrı ise kendi başına bir ifade olamaz.
- 5.5** Bir **main** yönteminin **return** türü nedir?
- 5.6** Bir değer döndüren yöntemde **return** ifadesi yazmamanın ne gibi bir sakıncası olurdu? **void** yönteminde **return** ifadesine sahip olabilir misiniz? Aşağıdaki yöntemde ki **return** ifadesi sözdizimi hatalarına neden olur mu?

```
public static void Yöntem(double,tdoublety) {System.out.print
    ln(x + y);
    returnx + y;
}
```

- 5.7** Parametre, argüman ve yöntem imzası terimlerini tanımlayın.
- 5.8** Şu yöntemler için yöntem başlıklarını (gövde değil) yazın:
- Satış tutarı ve komisyon oranı verildiğinde bir satış komisyonu hesaplayın.
 - Ay ve yıl verildiğinde bir ayın takvimini görüntüleyin.
 - Bir sayının karekökünü hesaplayın.
 - Bir sayının çift olup olmadığını test edin ve eğer öyleyse **true** döndürün.
 - Belirtilen sayıda bir mesaj görüntüleyin.
 - Kredi tutarı, yıl sayısı ve yıllık faiz oranı verildiğinde aylık ödemeyi hesaplayın.
 - Küçük harf verildiğinde karşılık gelen büyük harfi bulun.

- 5.9** Aşağıdaki programdaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfTest {
2     public staticmethod1(tinttn, m) {
3 n += m;n4 yömte
4 m2(t3.4t);n5 }
5
6
7     public static inttmethod2(tinttn) {
8         ift(n > t0t)trturnt1t;
9         else ift(n == t0t)trturnt0t;
10        else ift(n < t0t)trturnt1t;
11 }n12 }
```

- 5.10** Aşağıdaki programı Bölüm 1.10, Programlama Stili ve Dokümantasyon'da önerilen programlama stili ve dokümantasyon yönergelerine göre yeniden formatlayın.

Sonraki satır süslemesi stilini kullanın.

```
genel sınıfTest {
    public static doubletmethod1(tdouletj,tdouletj)n{

        whilet(i < j) {nj- -;

        }

        geri dönüş;n
    }
}
```

5.5 Değerlerle Parametre Geçişi



Argümanlar, bir yöntemi çağırırken parametrelere değer olarak geçilir.

parametre sırası ilişkisi

Bir yöntemin gücü, parametrelerle çalışma yeteneğidir. Herhangi bir dizeyi yazdırmak için `println` kullanabilir ve iki `int` değerinin maksimumunu bulmak için `max` kullanabilirsiniz. Bir yöntemi çağırırken, argümanları sağlamanız gerekir; bunlar, yöntem imzasındaki ilgili parametrelerle aynı sırada verilmelidir. Bu, *parametre sırası ilişkisi* olarak bilinir. Örneğin, aşağıdaki yöntem bir mesaj `n` kez yazdırır:

```
public static void tntPrintln(Dize mesajı, tinttn) {
    for (tinti = t0; i <
        n; i++)
        System.out.println(mesaj);
}
```

`nPrintln("Merhaba", 3)` kullanarak Merhaba üç kez yazdırabilirsiniz. `nPrintln("Merhaba", 3)` ifadesi, gerçek dize parametresi `Merhaba` yı `message` parametresine, `3` ü ise `n` ye geçirir, ve `Merhaba` üç kez yazdırır. Ancak, `nPrintln(3, "Merhaba")` ifadesi yanlıştır. `3`'ün veri türü, ilk parametre olan `mesaj` ile eşleşmez, ayrıca ikinci argüman olan `Merhaba`, ikinci parametre olan `n` ile de eşleşmez.



Dikkat

Argümanlar, yöntem imzasında tanımlandığı gibi sıra, sayı ve uyumlu tür açısından parametrelerle eşleşmelidir. Uyumlu tür, bir argümanı açık bir tür dönüşümü olmadan bir parametreye geçirebileceğiniz anlamına gelir; örneğin, bir `int` değer argümanını bir `double` değer parametresine geçirebilirsiniz.

değer ile geçiş

Bir yöntemi bir argüman ile çağırdığınızda, argümanın değeri parametreye iletilir. Bu, değer ile geçiş (pass-by-value) olarak adlandırılır. Eğer argüman bir değişken ise, değişkenin değeri parametreye iletilir. Değişken, yöntemin içindeki parametreye yapılan değişikliklerden etkilenmez. Liste 5.4'te gösterildiği gibi, `x`'in (`1`) değeri, `increment` yöntemini çağırmak için `n` parametresine iletilir (satır 5). Parametre `n`, yöntemde `1` ile artırılır (satır 10), ancak `x`, yöntemin ne yaptığına bakılmaksızın değişmez.

LISTE 5.4 Increment.java

arttır çağırısı

arttırın

```
1  genel sınıf Artış {
2      public static void tmain(String[] args) {
3          int x = 1;
4          System.out.println("Çağrıdan önce, x "+ x);
5          arttır(x);
6          System.out.println("Çağrıdan sonra, x "+ x);
7      }
8  }
9
10     public static void tincrement(tinttn) {
11         n++;
12         System.out.println("Yöntem içindeki n "+ n);
13     }
```



Çağrıdan önce, `x` 1'dir
metodun içindeki `n` 2 mi?
Çağrıdan sonra, `x` 1'dir

Liste 5.5, değer ile geçişin etkisini gösteren başka bir program vermektedir. Program, i ki değişkenin yer değiştirmesi için bir metod oluşturur. **swap** metodu, iki argüman geçil erek çağrılır. İlginç bir şekilde, metod çağrıldıktan sonra argümanların değerleri değişm ez.

LİSTE 5.5 TestPassByValue.java

```

1  genel sınıfTestPassByValue {
2      /** Ana yöntem */
3      public static voidtmain(String[] args) {
4          // Değişkenleri tanımlayın ve başlatın
5          int sayı1 = t1t;
6          inttnum2 = t2t;
7
8          System.out.println("Swap yöntemini çağtırmadan önce, num1 "+
9              num1 + " ve num2 "+ num2);
10
11          // İki değişkeni değiştirmeye çalışmak için swap yöntemini çağırın
12          takas(sayı1, sayı2);
13
14          System.out.println("Swap yöntemini çağtırdıktan sonra, num1 "+
15              num1 + " ve num2 "+ num2);
16      }
17
18      /** İki değişkeni değiştir */
19      genel statik voidtswap(tinttn1,tinttn2) {
20          System.out.println("Değiştirme yönteminin içinde");
21          System.out.println("Değiştirmeden önce, n1 "+ n1
22              + " ve n2 "+ n2);
23
24          // n1'i n2 ile değiştir
25          int temp = n1;
26          n1 = n2;n27 n2 =
27          sıcaklık;
28
29          System.out.println("Değiştirdikten sonra, n1 "+ n1
30              + " ve n2 "+ n2);
31      }naak
32  }

```

yanlış değişim

Değiştirme yöntemini çağtırmadan önce, num1 1 ve num2 2'dir
 Değiştirme yönteminin içinde
 Değiştirmeden önce, n1 1 ve n2 2'dir
 Değiştirdikten sonra, n1 2 ve n2 1'dir
 Değiştirme yöntemini çağtırdıktan sonra, num1 1 ve num2 2'dir



Yöntem çağrılmadan önce (**swap** metodu, satır 12), **num1** 1'dir ve **num2** 2'dir. **swap** metodu çağırıldıktan sonra, **num1** hala 1'dir ve **num2** hala 2'dir. Değerleri değiştirilmemiştir. Şekil 5.4'te gösterildiği gibi, argümanların değerleri **num1** ve **num2**, **n1** ve **n2**'ye aktarılır, ancak **n1** ve **n2**, **num1** ve **num2**'den bağımsız kendi bellek alanlarına sahiptir. Bu nedenle, **n1** ve **n2**'deki değişiklikler **num1** ve **num2**'nin içeriğini etkilemez.

Başka bir değişiklik, **swap** içindeki parametre adını **n1**'den **num1**'e değiştirmektir. Bunun ne etkisi var? Hiçbir değişiklik olmaz, çünkü parametre ve argümanın aynı adı taşımasının bir önemi yoktur. Parametre, kendi bellek alanına sahip bir değişkendir. Değişken, yöntem çağrıldığında tahsis edilir ve yöntem çağrıldığı yere döndüğünde kaybolur.

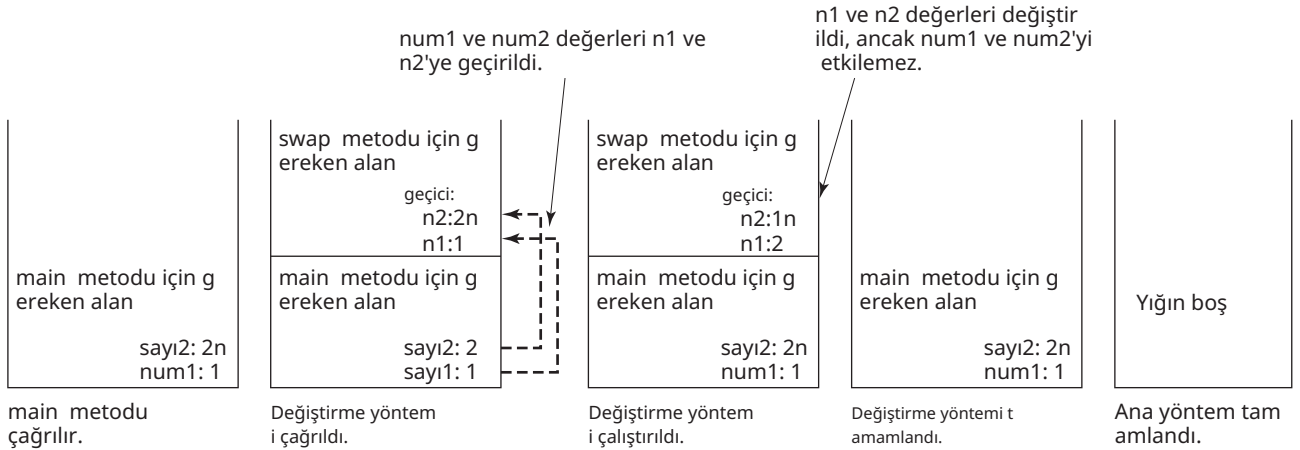


FIGURE 5.4 Değişkenlerin değerleri metodun parametrelerine aktarılır.



Not

Basitlik açısından, Java programcıları genellikle *x*'i *y*'ye aktarmak olarak söylerler, bu aslında argüman *x*'in değerin *i* parametre *y*'ye aktarmak anlamına gelir.

5.11 Bir argüman bir yonteme nasıl aktarılır? Argümanın, parametre ile aynı adı taşıması mümkün mü?

5.12 Aşağıdaki programdaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfTest {
2     public static voidtmain(String[] args) {
3         nPrintln(5, "Java'ya Hoş Geldiniz!");
4     }
5
6     public static voidtnPrintln(Dize mesajı, tinttn) {
7         inttn = t1t;
8         fort(tint = t0t; i < n; i++)
9 System.out.println(mesaj);n10 }
11 }
```

5.13 Değer geçişi nedir? Aşağıdaki programların sonucunu gösterin.

```
genel sınıfTest {
    public static voidtmain(String[] args) {ninttmax = t
    0t;
    max(t1t,t2t,tmax);
    nSystem.out.println(max);n}

    public static voidtmax(ninttva
    lue1,tinttvalue2,tinttmax) {nift(value1 > değ
    2)
    maksimum = değ
    1;
    aksi takdirde
    maksimum = değ
    2;n}
}
```

(a)

```
genel sınıfTest {
    public static voidtmain(String[] args) {nintti = t1t;

    while(i <= t6t) {met
        hod(in,t2t);i++;

    }

    public static voidtmethod1(nintti,ti
    nttnum) {
        fort(tinttj = t1t;tj <= i; j++) {nSystem.o
        ut.print(num + t" "t);nnum *= t2t;

    }

    System.out.println();n}
}
```

(b)

```

genel sınıfTest {
    public static void main(String[] args) {
        // Zamanları başlat
        int times = t3t;
        System.out.println("Çağrıdan önce,"
            + " değişken times "+ times);

        // nPrintln'ı çağır ve times'ı göster
        nPrintln("Java'ya hoş geldiniz!", times);
        System.out.println("Çağrıdan sonra,"
            + " değişken times "+ times);
    }

    // Mesajı n kez yazdır
    public static void nPrintln(
        Dize mesajı, tnttn) {nwhile(n >
        t0t) {
        System.out.println(t"n = "t n);nn Syste
        m.out.println(mesaj);nn--;
    }
}
}

```

(c)

```

genel sınıfTest {
    public static void tmain(String[] args) {nntti = t0t;

    while(i <= t4t) { nm
        etod1(i); hayır
        ++;
    }

    System.out.println("i şu anda "+ i);
}

    public static void tmethod1(int i) {ndot{

        ift(i % t3t!=t0t)nSys
            tem.out.print(i + t" "t);ni--;
        }
        while(i >= t1t);Syst
            em.out.println();n}
    }
}

```

(d)

5.14 Önceki sorudaki (a) için, **max** yöntemi çağırılmadan hemen önce, **max** yöntemine girildiğin de, **max** döndürülmeden hemen önce ve **max** döndürüldükten hemen sonra çağrı yığının da ki aktivasyon kayıtlarının içeriğini gösterin.

5.6 Modülerleştirme Kodu

Modülerleştirme, kodun bakımını ve hata ayıklamasını kolaylaştırır ve kodun yeniden kullanılmasını sağlar.

Yöntemler, gereksiz kodu azaltmak ve kodun yeniden kullanılmasını sağlamak için kullanılabilir. Yöntemler ayrıca kodu modülerleştirmek ve programın kalitesini artırmak için de kullanılabilir.

Liste 4.9, kullanıcının iki tam sayı girmesini isteyen ve bunların en büyük ortak bölenini gösteren bir program verir. Programı, Liste 5.6'da gösterildiği gibi bir yöntem kullanarak yeniden yazabilirsiniz.



VideoNotu

Kodu modüler hale getir

LİSTATİK 5.6 EnBüyükOrtakBölgeninYöntemi.java

```

1  import java.util.Scanner;
2
3  public class EnBüyükOrtakBölgeninYöntemi {
4      /** Ana yöntem */
5      public static void tmain(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = tnewScanner(System.in);
8
9          // Kullanıcıdan iki tam sayı girmesini iste
10         System.out.print("İlk tam sayıyı girin: ");
11         intn1 = input.nextInt();
12         System.out.print("İkinci tam sayıyı girin: ");
13         intn2 = input.nextInt();
14

```

```

15      System.out.println("En büyük ortak bölen "+ n1 +
16      " ve "+ n2 + " dir "      + gcd(n1; n2) );
17  }
18
19      /** İki tam sayının gcd'sini döndür */
20      genel statik inttgcd(tinttn1,tinttn2) {
21      intgcd = 1; // Başlangıç gcd'si 1
22      inttk = t2t; // Olası gcd
23
24      while(k <= n1 && k <= n2) {
25          ift(n1 %k == t0t&& n2 %k == t0t)
26          gcd = k; // gcd'yi güncelle
27          k++;
28      }
29
30      returntgcd; // gcd'yi döndür
31  }
32  }

```

gcd'yi çağır

gcd'yi hesapla

gcd'yi döndür



İlk tam sayıyı girin: 45

İkinci tam sayıyı girin: 75

45 ve 75 için en büyük ortak bölen 15'tir

gcd'yi elde etmek için kodu bir yöntemde kapsülleyerek, bu programın birkaç avantajı vardır:

1. gcd'yi hesaplama problemini ana metodun geri kalanından izole eder. Böylece, mantık netleşir ve program daha okunabilir hale gelir.
2. Gcd hesaplamasındaki hatalar, hata ayıklama kapsamını daraltan gcd yönteminde sınırlıdır.
3. gcd yöntemi artık diğer programlar tarafından yeniden kullanılabilir.

Liste 5.7, Liste 4.14, PrimeNumber.java'yı geliştirmek için kod modülerleştirme kavramını uygular.

LISTE5.7PrimeNumberMethod.java

```

1  genel classtPrimeNumberMethod {
2      public static voidtmain(String[] args) {
3          System.out.println("İlk 50 asal sayı ");4      printPrimeNumbers(50
4      );
5      }
6
7      public static voidtprintPrimeNumbers(tinttnnumberOfPrimes) {
8          final intNUMBER_OF_PRIMES_PER_LINE = 10; // Satır başına 10 göster
9          intcount = 0; // Asal sayıların sayısını say
10         intnumber = 2; // Asal olup olmadığını test etmek için bir sayı
11
12         // Tekrar tekrar asal sayıları bul
13         while(sayı < asalSayılar) {
14             // Asal sayıyı yazdır ve sayımı artır
15             eğer (asalMi(sayı) ) {
16                 sayım++; // Sayımı artır
17

```

çağır asalSayılarıYazdır

asalSayılarıYazdır yöntem

çağır asalMi

```

18     eğer(count % NUMBER_OF_PRIMES_PER_LINE == 0) {
19         // Sayıyı yazdır ve yeni bir satıra geç
20 System.out.printf(t"%-5s"t, sayı);n21 }

22     aksı takdirde
23 System.out.printf(t"%-5s"t, sayı);n24 }

25
26         // Bir sonraki sayının asal olup olmadığını kontrol et
27     number++;
28 }
29 }

30
31 /** Sayının asal olup olmadığını kontrol et */
32 public static boolean asal(int number) {
33     için (int bölün = 2; bölün <= number / 2; bölün++) {
34         eğer(number % bölün == 0) { // Eğer doğruysa, sayı asal değildir
35             return false; // Sayı asal değildir
36 } Hayır}

37
38     doğru döndür; // Sayı asal
39 }
40 }
41 }

```

isPrime yöntemi

İlk 50 asal sayı şunlardır:

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	53	59	61	67	71
73	79	83	89	97	101	103	107	109	113
127	131	137	139	149	151	157	163	167	173
179	181	191	193	197	199	211	223	227	229



Büyük bir problemi iki alt probleme böldük: bir sayının asal olup olmadığını belirlemek ve asal sayı ları yazdırmak. Sonuç olarak, yeni program okunması daha kolay ve hata ayıklaması daha kolaydı r. Ayrıca, yöntemler `printPrimeNumbers` ve `isPrime` diğer programlar tarafından yeniden kullanılabil ir.

5.7 Vaka Çalışması: Onluk Sayıları Onaltılı Sayılara Dönüştürme

Bu bölüm, bir onluk sayıyı onaltılı bir sayıya dönüştüren bir program sunmak tadır.



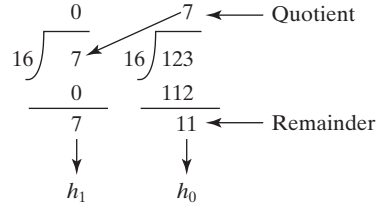
Onaltılı sayılar genellikle bilgisayar sistemleri programlamasında kullanılır (sayı sistemlerin e giriş için Ek F'ye bakın). Bir onluk sayıyı d onaltılı bir sayıya dönüştürmek, onaltılı basamak ları bulmak demektir.

htnt, t htnt - t 1t, t htnt - t 2t, t ct, t ht2t, t ht1t, ve ht0 öyle ki

$$d = h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \dots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

Bu onaltılı rakamlar, d'yi 16'ya bölerek ve bölüm 0 olana kadar devam ederek bul u nabilir. Kalanlar ht0t, t ht1t, t ht2t, t ct, t htnt - t 2t, t htnt - t 1t, ve htnt. Onaltılı rakamlar, onluk raka mlar 0, 1, 2, 3, 4, 5, 6, 7, 8 ve 9'un yanı sıra, onluk değeri 10 olan A; onluk değeri 11 olan B; 12 olan C; 13 olan D; 14 olan E; ve 15 olan F'yi içerir.

Örneğin, onluk sayı 123 onaltılıda 7B'ye eşittir. Dönüşüm şu şekilde yapılır: 123'ü 16'ya bölün. Kalan 11'dir (B onaltılıda) ve bölüm 7'dir. 7'yi 16'ya bölmeye devam edin. Kalan 7 ve bölüm 0'dır. Bu nedenle 7B, 123 için onaltılı sayıdır.



Liste 5.8, kullanıcıdan bir onluk sayı girmesini isteyen ve bunu bir hex sayısına dize olarak dönüştüren bir programı vermektedir.

Listing 5.8: Decimal2HexConversion.java

```

1  import java.util.Scanner;
23
   genel classtDecimal2HexConversion {
4      /** Ana yöntem */
5      public static voidtmain(String[] args) {
6          // Bir Scanner oluştur
7      Tarayıcı girişi = tnewScanner(System.in);
89
9          // Kullanıcıdan bir onluk tam sayı girmesini iste
10         System.out.print("Bir onluk sayı girin: ");
11         inttdecimal = input.nextInt();
12
13         System.out.println("Onluk için onaltılı sayı "+
14             decimal + " şudur: " + decimalToHex(decimal));
15     }
1617
18     /** Bir onluğu bir dize olarak onaltılıya dönüştür */
19     public staticString onlukTamSayııHexeDönüştür(intonlukTamSayı) {
20         Dize hex = t""t;
21
22         while(ondalık != t0t) {
23             intthexValue = ondalık % t16t;
24             altıgen = hexDeğeriToHexChar(hexValue) + altıgen;
25             onluk = onluk / 16;
26         }
27
28         returnthex;
29
30     }
3132
33     /** Bir tam sayıyı bir karakterde tek bir onaltılı basamağa dönüştür */
34     genel statik charttoHexChar(intthexValue) {
35         if(hexDeğeri <= t9t&& hexDeğeri >= t0t)
36             geri dönmek hart(hexValue + t'0't) ;
37         aksi takdirde// hexDeğeri <= 15 && hexDeğeri >= 10
38             geri dönmek (tchart)(hexValue - t10t+t'A't)n(tc) ;
39     }

```

girdi onluk tam sayı

onluktan hexa

bir hex karakteri al

bir harf al

Bir onluk sayı girin: 1234

Enter

Onluk 1234 için hex sayısı 4D2'dir



	satır#	onluk	onaltılı	hexDeğeri	hexDeğeriToHexChar(hexValue)
	19	1234	""		
tekrarlama 1	22			2	
	23		"2"		2
	24	77			
tekrarlama 2	22			13	
	23		"kayıp vur"		D
	24	4			
tekrarlama 3	22			4	
	23		"4D2"		4
	24	0			

Program, bir onluk tam sayıyı bir hex sayısına dize olarak dönüştürmek için **decimalToHex** yöntemini (satırlar 18–28) kullanır. Yöntem, onluk tam sayının 16 ile bölümünde kalanı alır (satır 22). Kalan, **toHexChar** yöntemini çağırarak bir karaktere dönüştürülür (satır 23). Karakter daha sonra hex dizisine eklenir (satır 23). Hex dizisi başlangıçta boş (satır 19). Hex basamağını sayının içinden çıkarmak için onluk sayıyı 16 ile böler (satır 24). **decimalToHex** yöntemi, bölüm 0 olana kadar bu işlemleri bir döngü içinde tekrarlar (satırlar 21–25).

toHexChar yöntemi (satırlar 31–36), 0 ile 15 arasında bir **hexValue**'yi bir hex karakteri ne dönüştürür. Eğer **hexValue** 0 ile 9 arasında ise, **(char)(hexValue + '0')**'ya dönüştürülür (satır 33). Bir karakter ile bir tam sayı eklerken, karakterin Unicode'u değerlendirilmez kullanılır. Örneğin, eğer **hexValue** 5 ise, **(char)(hexValue + '0')** 5 döner. Benzer şekilde, eğer **hexValue** 10 ile 15 arasında ise, **(char)(hexValue - 10 + 'A')**'ya dönüştürülür (satır 35). Örneğin, eğer **hexValue** 11 ise, **(char)(hexValue - 10 + 'A')** B döner.

5.15 **toHexChar(5)** çağrısından dönen değer nedir? **toHexChar(15)** çağrısından dönen değer nedir?

5.16 **decimalToHex(245)** çağrısından dönen değer nedir? **decimalToHex(3245)** çağrısından dönen değer nedir?

Point Control

MyProgrammingLab™

5.8 Yöntem Aşırı Yükleme

Yöntem aşırı yüklemesi, imzaları farklı olduğu sürece aynı adı taşıyan yöntemleri tanımlamanıza olanak tanır.



Daha önce kullanılan **max** yöntemi yalnızca **int** veri türü ile çalışır. Peki, iki ondalık sayının hangisinin maksimum değere sahip olduğunu belirlemeniz gerektiğinde ne olur? Çözüm, aşağıdaki kodla gösterildiği gibi aynı adı taşıyan ancak farklı parametreleri olan başka bir yöntem oluşturmaktır:

```
public static double max(double num1, double num2) {
    if (num1 > num2)
        return num1;
    else
        return num2;
}
```

yöntem aşırı yüklemesi

Eğer **max** metodunu **int** parametreleri ile çağırırsanız, **int** parametreleri bekleyen **max** metodu çağırılacaktır; eğer **max** metodunu **double** parametreleri ile çağırırsanız, **double** parametreleri bekleyen **max** metodu çağırılacaktır. Bu duruma *metod aşırı yüklemesi* denir; yani, iki metod aynı isme sahip ancak bir sınıf içinde farklı parametre listelerine sahiptir. Java derleyicisi hangi metodun kullanılacağını metod imzasına göre belirler.

Liste 5.9, üç yöntem oluşturan bir programdır. İlk yöntem maksimum tam sayıyı bulur, ikinci yöntem maksimum double'ı bulur ve üçüncü yöntem üç double

değeri arasında maksimumu bulur. Üç yöntemin adı da **max**'dir.

LISTE 5.9 TestMethodOverloading.java

```

1 public class TestMethodAşırıYükleme {
2     /** Ana yöntem */
3     public static void main(String[] args) {
4         // int parametreleri ile max yöntemini çağır
5         System.out.println("3 ve 4'ün maksimumu "
6             + maksimum(3,4) );
7
8         // double parametreleri ile max yöntemini çağır
9         System.out.println("3.0 ve 5.4'ün maksimumu "
10            + maksimum(3.0,5.4) );
11
12        // üç double parametre ile max yöntemini çağır
13        System.out.println("3.0, 5.4 ve 10.14'ün maksimumu "
14            + maksimum(3.0,5.4,10.14) );
15    }
16
17    /** İki int değerinin maksimumunu döndür */
18    genel statik int maksimum(int num1, int num2) {
19        if(sayı1 > sayı2)
20            return num1;
21        aksi takdirde
22            return num2;
23    }
24
25    /** İki double değerinin maksimumunu bul */
26    genel statik double maksimum(double num1, double num2) {
27        if(sayı1 > sayı2)
28            return num1;
29        aksi takdirde
30            return num2;
31    }
32
33    /** Üç double değerinin maksimumunu döndür */
34    genel statik double maksimum(double num1, double num2, double num3) {
35        return maksimum(maksimum(num1, num2), num3);
36    }
37 }

```

aşırı yüklenmiş max

aşırı yüklenmiş max

aşırı yüklenmiş max



3 ve 4'ün maksimumu 4'tür
 3.0 ve 5.4'ün maksimumu 5.4'tür
 3.0, 5.4 ve 10.14'ün maksimumu 10.14'tür

3, 4 için **max(3, 4)** (satır 6) çağırıldığında, iki tam sayının maksimumunu bulmak için **max** metodu çağırılır. 3.0, 5.4 için **max(3.0, 5.4)** (satır 10) çağırıldığında, iki double'ın maksimumunu bulmak için **max** metodu çağırılır. 3.0, 5.4, 10.14 için **max(3.0, 5.4, 10.14)** (satır 14) çağırıldığında, üç double değerinin maksimumunu bulmak için **max** metodu çağırılır.

max metodunu int değeri ve bir double değeri ile, örneğin `max(2,2.5)` çağırabilir misiniz? Eğer öyleyse, hangi max metodunun çağrıldığını biliyor musunuz? İlk sorunun cevabı evet. İkinci sorunun cevabı ise, iki double değerinin maksimumunu bulmak için max metodunu çağırıldığıdır. Argüman değeri 2 otomatik olarak bir double değerine dönüştürülür ve bu metoda iletilir.

Metodun `max(double, double)` çağrılmadığını merak ediyor olabilirsiniz `max(3, 4)` için. Hem `max(double, double)` hem de `max(int, int)` `max(3, 4)` için olası eşleşmelerdir. Java derleyicisi, bir metod çağrısı için en spesifik metodu bulur.

`max(int, int)` metodu, `max(double, double)` metodundan daha spesifik olduğundan, `max(3, 4)` çağrısını yapmak için `max(int, int)` kullanılır.



Ttip

Yükleme yöntemleri programları daha net ve okunabilir hale getirebilir. Farklı türde parametrelerle aynı işlevi yerine getiren yöntemlere aynı ad verilmelidir.



Not

Yüklenmiş yöntemlerin farklı parametre listelerine sahip olması gerekir. Yöntemleri farklı değiştiriciler ve ya dönüşüm türlerine göre yükleyemezsiniz.



Not

Bazen bir yöntemin çağrısı için iki veya daha fazla olası eşleşme vardır, ancak derleyici en spesifik eşleşmeyi belirleyemez. Bu duruma belirsiz çağrı denir. Belirsiz çağrı bir derleme hatasına neden olur. Aşağıdaki kodu düşünün:

belirsiz çağrı

```
public class AmbiguousOverloading {
    public static void main(String[] args) {
        System.out.println(
            maksimum(t1, t2));
    }

    genel statik çift        maksimum(tinttnum1, tdouletnum2) {
        if(sayı1 > sayı2) dö
            nüşsayısı1; nels
        e
        return tnum2; n
    }

    genel statik çift        maksimum(tdouletnum1, tinttnum2) {
        if(sayı1 > sayı2) dö
            nüşsayısı1; nels
        e
        return tnum2; n
    }
}
```

Hem `max(int, double)` hem de `max(double, int)` `max(1, 2)` ile eşleşmek için olası adaylardır. Hiçbiri diğerinden daha spesifik olmadığından, çağrı belirsizdir ve bu da bir derleme hatasına yol açar.

5.17 Metot aşırı yüklemesi nedir? Aynı adı taşıyan ancak farklı parametre türlerine sahip iki metod tanımlamak izin verilir mi? Aynı sınıfta, aynı metod adlarına ve parametre listelerine sahip ancak farklı dönüş değerleri türlerine veya farklı değiştiricilere sahip iki metod tanımlamak izin verilir mi?

5.18 Bu programda ne yanlış?

```
genel sınıf Testi {
    genel statik geçersiz yöntem(tint) {n}
```

```

        public static inttmethot(tintty) {nreturnty
        ;
    }
}

```

5.19 İki yöntem tanımı verildiğinde,

```

public static doubletm(tdoubletx,tdoublety)npublic st
atic doubletm(tinttx,tdoublety)

```

şunlar için hangi iki yöntem çağrıldığını belirtin:

- `doubletz = m(t4t,t5t);`
- `doubletz = m(t4t,t5.4t);`
- `doubletz = m(t4.5t,t5.4t);`

5.9 Değişkenlerin Kapsamı



Ana
Nokta

Bir değişkenin kapsamı, değişkenin

referans alınabileceği programın kısmıdır.

değişkenin kapsamı
yerel değişken

Bölüm 2.5, bir değişkenin kapsamını tanıttı. Bu bölüm, değişkenlerin kapsamını daha ayrıntılı olarak tartışmaktadır. Bir yöntem içinde tanımlanan bir değişken, yerel değişken olarak adlandırılır. Bir yerel değişkenin kapsamı, tanımından başlar ve değişkeni içeren bloğun sonuna kadar devam eder. Bir yerel değişken, kullanılmadan önce tanımlanmalı ve bir değer atanmalıdır.

Bir parametre aslında bir yerel değişkendir. Bir yöntem parametresinin kapsamı, tüm yöntemi kapsar. Bir **for**-döngüsü başlığının başlangıç eylem kısmında tanımlanan bir değişkenin kapsamı, tüm döngüde geçerlidir. Ancak, bir **for**-döngüsü gövdesinde tanımlanan bir değişkenin kapsamı, tanımından itibaren değişkeni içeren bloğun sonuna kadar döngü gövdesi ile sınırlıdır; bu, Şekil 5.5'te gösterilmiştir.

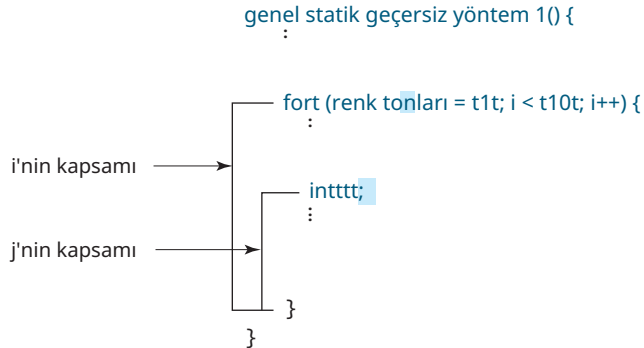


FIGURE 5.5 Bir **for** döngüsü başlığının başlangıç eylem kısmında tanımlanan bir değişkenin kapsamı döngünün tamamındadır.

Bir yöntem içinde farklı bloklarda aynı isimle bir yerel değişken tanımlayabilirsiniz, ancak aynı blokta veya iç içe geçmiş bloklarda bir yerel değişkeni iki kez tanımlayamazsınız; bu, Şekil 5.6'da gösterilmiştir.

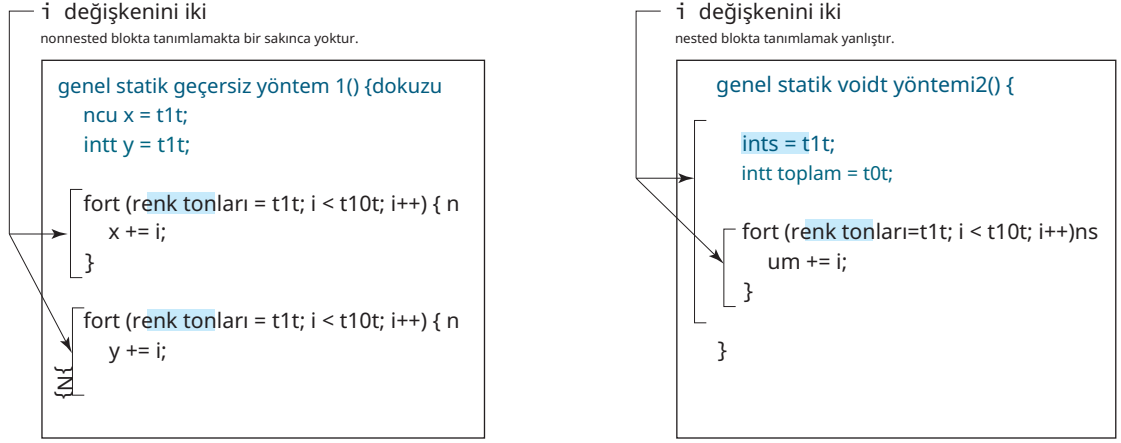


FIGURE 5.6 Bir değişken, iç içe olmayan bloklarda birden fazla kez tanımlanabilir, ancak iç içe bloklarda yalnızca bir kez tanımlanabilir.



Dikkat

Bir değişkeni bir blok içinde tanımlamayın ve ardından onu blok dışında kullanmaya çalışmayın. İşte yaygın bir hatanın örneği:

```
for(tint = t0t; i < t10t; i++) {n}
```

```
System.out.println(i);
```

Son ifade bir sözdizimi hatasına neden olacaktır, çünkü değişken `i` `for` döngüsü dışında tanımlanmamıştır.

5.20 Yerel bir değişken nedir?

5.21 Yerel bir değişkenin kapsamı nedir?

Point Control

MyProgrammingLab™

5.10 Math Sınıfı

Math sınıfı, temel matematiksel fonksiyonları gerçekleştirmek için gereken yöntemleri içerir.

Daha önce 2.9.3 Bölümünde `pow(a, b)` yöntemini hesaplamak için kullandınız. *atb* Bölüm 2.9.3'te, Üslü Oper- Bu bölüm, 3.8. Bölümdeki `Math.random()` yöntemini ve rastgele sayıların üretilmesini tanıtmaktadır. Bu bölüm, `Math` sınıfındaki diğer yararlı yöntemleri tanıtmaktadır. Bu yöntemler trigonometrik yöntemler, *üstel yöntemler* ve *hizmet yöntemleri* olarak kategorize edilebilir. Hizmet yöntemleri, yuvarlama, min, max, mutlak ve rastgele yöntemleri içerir. Yöntemlerin yanı sıra, `Math` sınıfı iki yararlı `double` sabiti, `PI` ve `E` (doğal logaritmaların tabanı) sağlar. Bu sabitleri herhangi bir programda `Math.PI` ve `Math.E` olarak kullanabilirsiniz.



5.10.1 Trigonometri Yöntemleri

Matematik sınıfı aşağıdaki trigonometrik yöntemleri içerir:

```
/** Bir açının radyan cinsinden trigonometrik sinüsünü döndür */
public static double sin(double radyan)
```

```
/** Bir açının radyan cinsinden trigonometrik kosinüsünü döndür */
public static double cos(double radyan)
```

```
/** Bir açının radyan cinsinden trigonometrik tanjantını döndür */
public static double tan(double radyan)
```

```

/** Açığı derece cinsinden radyan cinsine dönüştür */
public static double toRadians(double degree)

/** Açığı radyan cinsinden derece cinsine dönüştür */
public static double toDegrees(double radians)

/** Sinüsün tersine karşılık gelen açığı radyan cinsinden döndür */
public static double asin(double a)

/** Kosinüsün tersine karşılık gelen açığı radyan cinsinden döndür */
public static double acos(double a)

/** Tanjantın tersinin radyan cinsinden açısını döndür */
genel statik double atan(double a)

```

sin, cos ve tan için parametre, radyan cinsinden bir açıdır. asin, acos ve

atan için dönen değer, aralıkta radyan cinsinden bir dereceyi temsil eder ve Bir derece, $\pi/2$ ve $\pi/2$. Bir derece, $\pi/180$ radyan cinsinden eşittir, 90 derece ise eşittir $\pi/2$ radyan cinsinden ve 30 derece ise eşittir $\pi/6$ radyan cinsinden.

Örneğin,

```

Math.toDegrees(Math.PI / 2) 90.0 döndürür
Math.toRadians(30) 0.5236 döndürür(  $\pi/6$  ile aynı)
Math.sin(0) 0.0 döndürür
Math.sin(Math.toRadians(270)) -1.0 döndürür
Math.sin(Math.PI / 6) 0.5 döndürür
Math.sin(Math.PI / 2) 1.0 döndürür
Math.cos(0) 1.0 döndürür
Math.cos(Math.PI / 6) 0.866 döndürür
Math.cos(Math.PI / 2) 0 döndürür
Math.asin(0.5) 0.523598333 ( $\pi/6$  ile aynı)

```

5.10.2 Üs Yöntemleri

Math sınıfında üslerle ilgili beş yöntem bulunmaktadır:

```

/** e'nin x üssü ( $e^x$ ) olarak döndürülmesi */
public static double exp(double x)

/** x'in doğal logaritmasını döndür ( $\ln(x) = \log_e(x)$ ) */
public static double log(double x)

/** x'in 10 tabanındaki logaritmasını döndür ( $\log_{10}(x)$ ) */
kamusal statik çift log10(double x)

/** b üssü a döndür */
kamusal statik çift pow(double a, double b)

/** x'in karekökünü döndür  $\sqrt{x}$  için  $x \geq 0$  */
kamusal statik çift sqrt(double x)

```

Örneğin,

```

Math.exp(1) döner 2.71828
Math.log(Math.E) döner 1.0
Math.log10(10) döner 1.0
Math.pow(2,3) döner 8.0
Math.pow(3,2) döner 9.0
Math.pow(3.5,2.5) döner 22.91765

```

Math.sqrt(4) 2.0 döner
 Math.sqrt(10.5) 3.24 döner

5.10.3 Yuvarlama Yöntemleri

Matematik sınıfı beş yuvarlama yöntemini içerir:

```
/** x, en yakın tam sayıya yukarı yuvarlanır. Bu tam sayı
 * double değeri olarak döner. */
public static double ceil(double x)
```

```
/** x, en yakın tam sayıya aşağı yuvarlanır. Bu tam sayı
 * double değeri olarak döner. */
public static double floor(double x)/** x, en ya
```

kın tam sayıya yuvarlanır. Eğer x, iki tam sayıya eşit uzaklıkta ise, çift olanı double olarak döner. */public static double rint(double x)

```
/** (int)Math.floor(x + 0.5) döner. */
public static int round(float x)
```

```
/** (long)Math.floor(x + 0.5) döner. */
public static long round(double x)
```

Örneğin,

```
Math.ceil(2.1) 3.0 döner
Math.ceil(2.0) 2.0 döner
Math.ceil(-2.0) -2.0 döner
Math.ceil(-2.1) -2.0 döner
Math.floor(2.1) 2.0 döner
Math.floor(2.0) 2.0 döner
Math.floor(-2.0) -2.0 döner
Math.floor(-2.1) -3.0 döner
Math.rint(2.1) 2.0 döner
Math.rint(-2.0) -2.0 döner
Math.rint(-2.1) -2.0 döner
Math.rint(2.5) 2.0 döner
Math.rint(3.5) 4.0 döner
Math.rint(-2.5) -2.0 döner
Math.round(2.6f) 3// int döner
Math.round(2.0) 2// long döner
Math.round(-2.0f) -2// int döner
Math.round(-2.6) -3// long döner
Math.round(-2.4) -2// long döner
```

5.10.4 min,max ve abs Yöntemleri

min ve max yöntemleri, iki sayının (int,long,float veya double) minimum ve maksimum sayılarını döndürmek için aşırı yüklenmiştir. Örneğin, max(3.4, 5.0) 5.0 döner, min(3, 2) 2 döner.

abs yöntemi, sayının (int,long,

float veya double) mutlak değerini döndürmek için aşırı yüklenmiştir. Örneğin,

```
Math.max(2,3) 3 döndürür
Math.max(2.5,3) 3.0 döndürür
Math.min(2.5,3.6) 2.5 döndürür
```

Math.abs(-2) 2 döndürür
 Math.abs(-2.1) 2.1 döndürür

5.10.5 Rastgele Yöntem

Rastgele bir `double` değeri oluşturmak için `random()` yöntemini kullandınız. Bu değer `0.0` veya daha büyük ve `1.0`'den küçük olmalıdır (`0 <= Math.random() < 1.0`). Bu yöntem çok kullanışlıdır. Herhangi bir aralıkta rastgele sayılar oluşturmak için basit bir ifade yazmak için bunu kullanabilirsiniz. Örneğin,

`(int) (Math.random() * 10)` → 0 ile 9 arasında rastgele bir tam sayı döndürür

`50 + (int) (Math.random() * 50)` → 50 ile 99 arasında rastgele bir tam sayı döndürür

Genel olarak,

`a + t Math.random() * b` → Arasında rastgele bir sayı döndürür `a` ve `b`, `a + b`'yi hariç tutarak



Tip

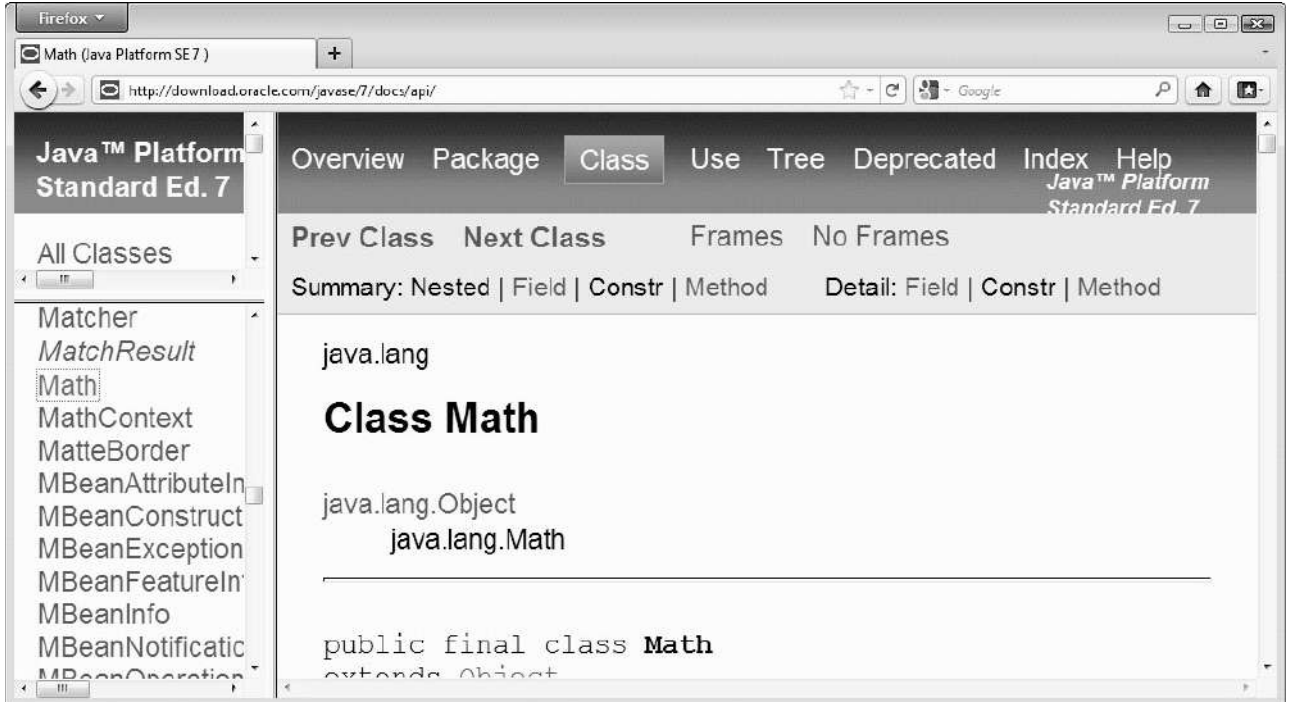
Math sınıfının tam belgelerini çevrimiçi olarak download

[.oracle.com/javase/7/docs/api/](http://download.oracle.com/javase/7/docs/api/) adresinde görüntüleyebilirsiniz; Şekil 5.7'de gösterildiği gibi.



Not

Tüm sınıfların `main` yöntemine ihtiyacı yoktur. `Math` sınıfı ve `JOptionPane` sınıfı `main` yöntemine sahip değildir. Bu sınıflar, diğer sınıfların kullanması için yöntemler içerir.



ŞEKİL 5.7 Java API'sinin belgelerini çevrimiçi olarak görüntüleyebilirsiniz.

- 5.22** Doğru mu yanlış mı? Trigonometrik yöntemlerin argümanı, radyan cinsinden bir açıdır.
- 5.23** 34 ile 55 arasında rastgele bir tam sayı elde eden bir ifade yazın. 0 ile 999 arasında rastgele bir tam sayı elde eden bir ifade yazın. 5.5 ile 55.5 arasında rastgele bir sayı elde eden bir ifade yazın. Rastgele bir küçük harf elde eden bir ifade yazın.
- 5.24** Aşağıdaki yöntem çağrılarını değerlendirin:

- | | |
|--------------------------------------|---------------------------------|
| a. Matematik.sqrt(t4t) | j. Math.floor(t-2.5t) |
| b. Math.Sin(:2t*Math.Pi) | k. Math.round(t-2,5ft) |
| c. Math.cos(t2t* Math.PI) | l. Math.round(t-2.5t) |
| d. Math.pow(t2t,t 2t) | m. Math rint(t2.5t) |
| ve. Math.log(Matematik.E) | n. Math.ceil(t2.5t) |
| f. Math.exp(t1t) | . Matematik.kat(t2.5t) |
| g. Math.max(t2t, Math.min(t3t,t 4t)) | p. Math.round(t2.5ft) |
| h. Math.rint(t-2.5t) | q. Math.round(t2.5t) |
| Ben. Math.ceil(t-2.5t) | r. Math.round(Math.abs(t-2.5t)) |

5.11 Vaka Çalışması: Rastgele Karakterler Üretme

Bir karakter, bir tam sayı kullanılarak kodlanır. Rastgele bir karakter üretmek, bir tam sayı üretmektir.



Bilgisayar programları sayısal verileri ve karakterleri işler. Sayısal verileri içeren birçok örnek gördünüz. Karakterleri ve bunları nasıl işleyeceğinizi anlamak da önemlidir. Bu bölüm, rastgele karakterler üretme örneğini sunmaktadır.

2.17. Bölümde tanıtıldığı gibi, her karakterin onaltılı (65535 onluk) arasında 0 ve FFFF arasında benzersiz bir Unicode'u vardır. Rastgele bir karakter üretmek, aşağıdaki ifadeyi kullanarak 0 ve 65535 arasında rastgele bir tam sayı üretmektir (not edin ki `0 <= Math.random() < 1.0`, 65535 eklemeniz gerekir):

```
(tintt)(Math.random()) * (t65535t+t1t))
```

Şimdi rastgele bir küçük harf nasıl oluşturulacağını düşünelim. Küçük harfler için Unicode değerleri, **a**, ardından **b**, **c**, ... ve **z** için Unicode değerinden başlayarak ardışık tam sayılardır. **a** için Unicode değeri

```
(int)'a'
```

Bu nedenle, `(int)'a'` ile `(int)'z'` arasında rastgele bir tam sayı

```
(int)((int)'a' + Math.random() * ((int)'z' - (int)'a' + 1))
```

2.17.3 Bölümünde tartışıldığı gibi, tüm sayısal operatörler **char** operatörlerine uygulanabilir. **char** operatörü, diğer operatör bir sayı veya bir karakter olduğunda bir sayıya dönüştürülür. Bu nedenle, önceki ifade şu şekilde basitleştirilebilir:

```
'a' + Math.random() * ('z' - 'a' + 1)
```

ve rastgele bir küçük harf

```
(char)('a' + Math.random() * ('z' - 'a' + 1))
```

202 Bölüm 5 Yöntemler

Bu nedenle, **ch1** ve **ch2** arasında, **ch1 < ch2** olan herhangi iki karakter arasında rastgele bir karakter oluşturulabilir:

```
(char)(ch1 + Math.random() * (ch2 - ch1 + 1))
```

Bu basit ama faydalı bir keşif. Liste 5.10, belirli bir tür karakteri rastgele almak için beş aşırı yüklenmiş yönteme sahip **RandomCharacter** adında bir sınıf oluşturur. Bu yöntemleri gelecekteki projelerinizde kullanabilirsiniz.

Liste 5.10 RandomCharacter.java

```
1 genel sınıf RandomCharacter {
2     /** ch1 ve ch2 arasında rastgele bir karakter oluştur */
rastgeleKarakterAl 3     public static char rastgeleKarakterAl(char ch1, char ch2) {
4         return (char)(ch1 + Math.random() * (ch2 - ch1 + 1));
5     }
6
7     /** Rastgele bir küçük harf oluştur */
rastgeleKüçükHarfAl() 8     public static char rastgeleKüçükHarfAl() {
9         return getRandomCharacter('a', 'z');
10
11
12     /** Rastgele bir büyük harf oluştur */
rastgeleBüyükHarfAl() 13     public static char rastgeleBüyükHarfAl() {
14         return getRandomCharacter('A', 'Z');
15     }
16
17     /** Rastgele bir rakam karakteri oluştur */
rastgeleRakamKarakterAl() 18     public static char rastgeleRakamKarakterAl() {
19         return getRandomCharacter('0', '9');
20     }
21
22     /** Rastgele bir karakter oluştur */
rastgeleKarakterAl() 23     public static char rastgeleKarakterAl() {
24         return getRandomCharacter(' ', 'z');
25 }
26 }
```

Liste 5.11, 175 rastgele küçük harfi görüntüleyen bir test programı verir.

Liste 5.11 TestRandomCharacter.java

```
1 public class TestRandomCharacter {
2     /** Ana yöntem */
sabitler 3     public static void main(String[] args) {
4         final int NUMBER_OF_CHARS = 175;
5         final int CHARS_PER_LINE = 25;
6
7         // 'a' ile 'z' arasında rastgele karakterler yazdır, satır başına 25 karakter
küçük harf 8         for (int i = 0; i < NUMBER_OF_CHARS; i++) {
9             char ch = RandomCharacter.getRandomLowerCaseLetter();
10            if ((i + 1) % CHARS_PER_LINE == 0)
11                System.out.println(ch);
12            // aksi takdirde
13            System.out.print(ch);
14
15 }
16 }
```

```
gmjssohezfkg tazqgmswfc lraonp
nrunulnwmaztlfjedmpchcfnlal
qdgivxkxpbzulrmqmbhikrnlbnr
jlsopfxa hssqhwuuljvbenxbhdo
tzhpehbqmuwsfktwsolincbuwk
zgxpmtzihgatdslvbwbnbfesokl
wbhnooygiigzdxuqni
```



9. Satır, `getRandomLowerCaseLetter()` yöntemini `RandomCharacter` sınıfında tanımlar.

Dikkat edin ki `getRandomLowerCaseLetter()` herhangi bir parametreye sahip değildir, ancak yöntemi tanımlarken ve çağırırken parantezleri kullanmanız gerekir.

parantez gereklidir

5.12 Yöntem Soyutlaması ve Aşamalı İyileştirme

Yazılım geliştirmede anahtar, soyutlama kavramını uygulamaktır.

Bu kitaptan birçok soyutlama seviyesini öğreneceksiniz. *Yöntem soyutlaması*, bir yöntemin kullanımını uygulamasından ayırarak elde edilir. Müşteri, bir yöntemi nasıl uygulandığını bilmeden kullanabilir. Uygulamanın detayları, yöntemi çağırarak müşteriye gizlenmiş ve yöntemde kapsüllenmiştir. Bu, *bilgi gizleme* veya *kapsülleme* olarak da bilinir. Uygulamayı değiştirmeye karar verirsiniz, müşteri programı etkilenmeyecektir, yeter ki yöntem imzasını değiştirmeyin. Yöntemin uygulaması, müşteriye “kara kutu” içinde gizlenmiştir; bu, Şekil 5.8’de gösterilmiştir.



VideoNotu

Adım adım iyileştirme

yöntem soyutlama

bilgi gizleme

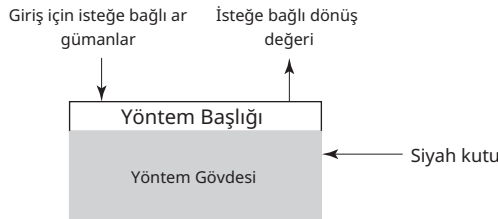


FIGURE 5.8 Yöntem gövdesi, yöntemin ayrıntılı

uygulamasını içeren bir kara kutu olarak düşünülebilir.

Artık bir dizeyi görüntülemek için `System.out.print` yöntemini ve maksimum sayıyı bulmak için `max` yöntemini kullandınız. Bu yöntemleri programınızda çağırarak kod yazmayı biliyorsunuz, ancak bu yöntemlerin nasıl uygulandığını bilmeniz gerekmiyor.

Yöntem soyutlama kavramı, program geliştirme sürecine uygulanabilir. Büyük bir program yazarken, onu alt problemlere ayırmak için *böl ve fethet* stratejisini, ayrıca adım adım iyileştirme olarak da bilinen yöntemi kullanabilirsiniz. Alt problemler daha küçük, daha yönetilebilir problemlere ayrılabilir.

Diyelim ki, yılın belirli bir ayının takvimini görüntüleyen bir program yazıyorsunuz. Program, kullanıcıdan yılı ve ayı girmesini ister, ardından ay için tüm takvimi görüntüler; aşağıdaki örnek çalışmada gösterildiği gibi.

böl ve fethet
adım adım iyileştirme



Tam yılı girin (örneğin, 2012):

Ayı 1 ile 12 arasında bir sayı olarak girin:

Mart 2012

Pazartesi	Salı	Çarşamba	Perşembe	Cuma	Cumartesi
			1	2	3
4	5	6	7	8	9 10
11	12	13	14	15	16 17
18	19	20	21	22	23 24
25	26	27	28	29	30 31

Bu örneği, böl ve fethet yaklaşımını göstermek için kullanalım.

5.12.1 Yukarıdan Aşağıya Tasarım

Böyle bir programa nasıl başlayacaksınız? Hemen kodlamaya mı başlayacaksınız? Yeni başlayan programcılar genellikle her detayın çözümünü bulmaya çalışarak başlarlar. Detaylar son programda önemli olsa da, erken aşamalarda detay kaygısı problem çözme sürecini engelleyebilir. Problem çözmenin mümkün olduğunca akıcı bir şekilde ilerlemesi için, bu örnek, tasarımdan detayları izletmek için yöntem soyutlaması kullanarak başlar ve yalnızca daha sonra detayları uygular.

Bu örnekte, problem önce iki alt probleme ayrılır: kullanıcıdan girdi almak ve ayın takvimini yazdırmak. Bu aşamada, alt problemlerin neyi başaracağı ile ilgilenmelisiniz, ayın takvimini yazdırmak ve girdi almakla değil. Problemin parçalanmasını görselleştirmeye yardımcı olmak için bir yapı şeması çizebilirsiniz (bkz. Şekil 5.9a).

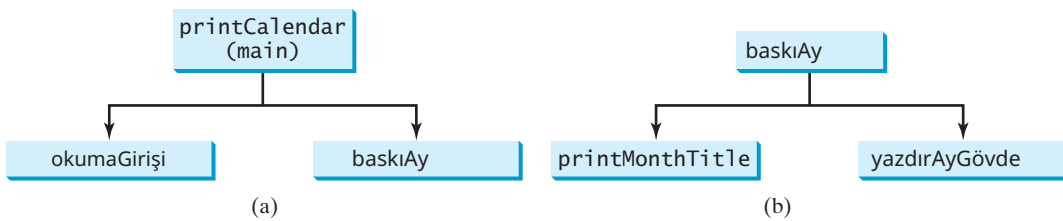


FIGURE 5.9Yapı grafiği, **printCalendar** probleminin iki alt probleme, **readInput** ve **printMonth** (a) olarak bölündüğünü ve **printMonth** probleminin iki daha küçük alt probleme, **printMonthTitle** ve **printMonthBody** (b) olarak bölündüğünü göstermektedir.

Yıl ve ay için girdi okumak için **Scanner** kullanabilirsiniz. Verilen bir ayın takvimini yazdırma problemi iki alt probleme ayrılabilir: ay başlığını yazdırmak ve ay gövdesini yazdırmak, Şekil 5.9b'de gösterildiği gibi. Ay başlığı üç satırdan oluşur: ay ve yıl, kesik bir çizgi ve haftanın yedi gününün adları. Sayısal aydan (örneğin, 1) ay adını (örneğin, Ocak) almanız gerekir. Bu, **getMonthName** içinde gerçekleştirilir (bkz. Şekil 5.10a).

Ay gövdesini yazdırmak için, ayın ilk gününün hangi gün olduğunu bilmeniz gerekir (**getStartDay**) ve ayın kaç gün sürdüğünü bilmeniz gerekir (**getNumberOfDaysInMonth**),

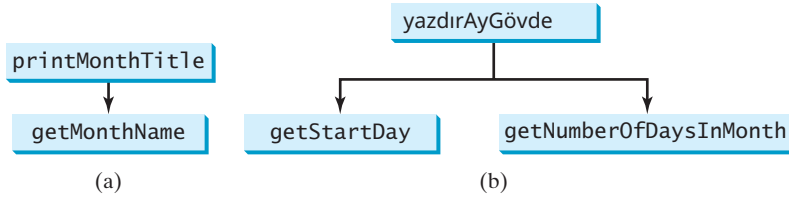


FIGURE 5.10(a) printMonthTitle yazdırmak için getMonthName gerekir. (b) printMonthBody problemi birkaç daha küçük probleme ayırmıştır.

Şekil 5.10b'de gösterildiği gibi. Örneğin, Aralık 2013'te 31 gün vardır ve 1 Aralık 2013, bir Pazar günüdür.

Bir ayın ilk tarihinin başlangıç gününü nasıl alırsınız? Bunu yapmanın birkaç yolu vardır. Şimdilik alternatif bir yaklaşım kullanacağız. 1 Ocak 1800'ün bir Çarşamba olduğunu biliyorsanız ($START_DAY_FOR_JAN_1_1800 = 3$) varsayalım. 1 Ocak 1800 ile takvim ayının ilk tarihi arasındaki toplam gün sayısını (`totalNumberOfDays`) hesaplayabilirsiniz. Takvim ayının başlangıç günü $(totalNumberOfDays + startDay1800) \% 7$ 'dir, çünkü her hafta yedi gündür. Böylece, `getStartDay` problemi daha da ayrıntılı hale getirebiliriz `getTotalNumberOfDays` olarak, Şekil 5.11a'da gösterildiği gibi.

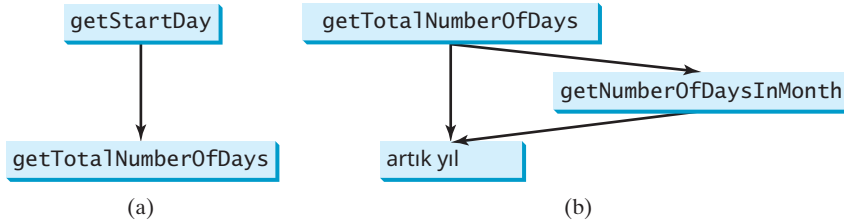


FIGURE 5.11 (a) getStartDay almak için getTotalNumberOfDays gerekir. (b) getTotalNumberOfDays problemi iki daha küçük probleme ayırmıştır.

Toplam gün sayısını elde etmek için, yılın artık yıl olup olmadığını ve her ayın gün sayısını bilmeniz gerekir. Böylece, `getTotalNumberOfDays` iki alt probleme daha ayrılabilir: `isLeapYear` ve `getNumberOfDaysInMonth`, Şekil 5.11b'de gösterildiği gibi. Tam yapı şeması Şekil 5.12'de gösterilmektedir.

5.12.2 Yukarıdan Aşağıya ve/veya Aşağıdan Yukarıya Uygulama

Şimdi uygulamaya dikkatimizi veriyoruz. Genel olarak, bir alt problem uygulamadaki bir metoda karşılık gelir, ancak bazıları o kadar basittir ki bu gereksizdir. Hangi modülleri metodlar olarak uygulayacağınıza ve hangilerini diğer metodlarda birleştireceğinize karar vermeniz gerekir. Bu tür kararlar, seçiminiz sonucunda genel programın okunmasının daha kolay olup olmayacağına dayanmalıdır. Bu örnekte, alt problem `readInput` basitçe `main` metodunda uygulanabilir.

Ya “yukarıdan aşağıya” ya da “aşağıdan yukarıya” bir yaklaşım kullanabilirsiniz. Yukarıdan aşağıya yaklaşım, yapı şemasındaki bir metodu bir seferde üstten alta doğru uygular. Uygulanmayı bekleyen metodlar için *Stubs* - basit ama eksik bir metod versiyonu - kullanılabilir. Stub'ların kullanımı, programın çerçevesini hızlı bir şekilde oluşturmanıza olanak tanır. Önce `main` metodunu uygulayın, ardından `printMonth` metodu için bir stub kullanın. Örneğin,

üstten aşağıya yaklaşım
stub

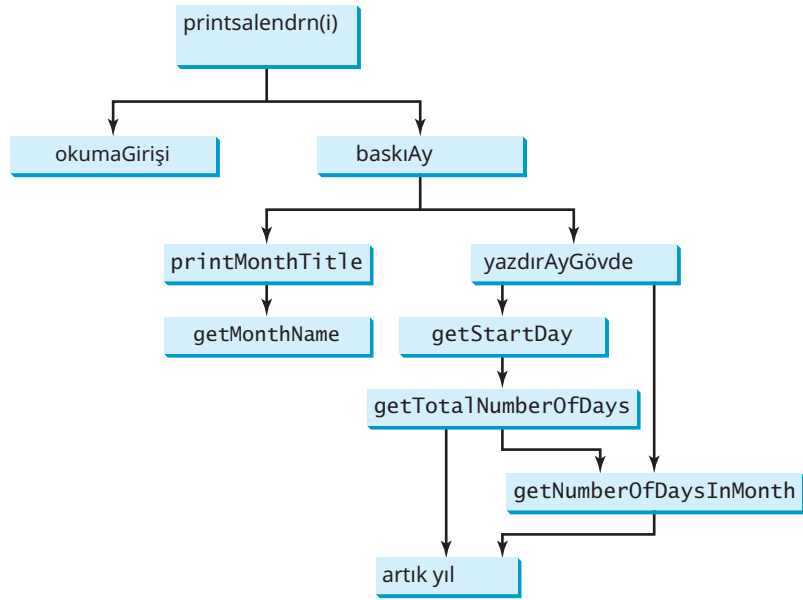


FIGURE 5.12Yapı grafiği, programdaki alt problemler arasındaki hiyerarşik ilişkiyi göstermektedir.

İlet **printMonth** yıl ve ayı stub'da görüntülesin. Böylece, programınız şu şekilde başlayabilir:

```

public class PrintCalendar {
    /** Ana yöntem */
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Kullanıcıdan yılı girmesini isteyin
        System.out.print("Tam yılı girin (örneğin, 2012): ");
        int year = input.nextInt();

        // Kullanıcıdan ayı girmesini isteyin
        System.out.print("Ayı 1 ile 12 arasında bir sayı olarak girin: ");
        int month = input.nextInt();

        // Yılın ayı için takvimi yazdırın
        printMonth(year, month);
    }

    /** printMonth için bir stub şu şekilde görünebilir */
    public static void printMonth(int year, int month) {
        System.out.print(month + " " + year);
    }

    /** printMonthTitle için bir stub şöyle görünebilir */
    public static void printMonthTitle(int year, int month) {
    }

    /** getMonthBody için bir stub şöyle görünebilir */
    public static void printMonthBody(int year, int month) {
    }
}

```

```

/** getMonthName için bir stub şöyle görünebilir */
public static String ayAdınıAl(int ay) {
    geri dönmek "Ocak"; // Bir dummy değer
}

/** getStartDay için bir stub şöyle görünebilir */
public static int ayBaşlangıçGünüAl(int yıl, int ay) {
    return 1; // Sahte bir değer
}

/** getTotalNumberOfDays için bir stub şöyle görünebilir */
public static int toplamGünSayısınıAl(int yıl, int ay) {
    return 10000; // Sahte bir değer
}

/** getNumberOfDaysInMonth için bir stub şöyle görünebilir */
public static int aynıAyınGünSayısınıAl(int yıl, int ay) {
    return 31; // Sahte bir değer
}

/** isLeapYear için bir stub şöyle görünebilir */
public static Boolean isLeapYear(int yıl) {
    return true; // Sahte bir değer
}
}

```

Programı derleyin ve test edin, ve herhangi bir hatayı düzeltin. Artık `printMonth` metodunu uygulayabilirsiniz. `printMonth` metodundan çağrılan metodlar için, yine stub'lar kullanabilirsiniz.

Aşağıdan yukarıya yaklaşım, yapı şemasındaki bir metodu bir seferde aşağıdan yukarıya doğru uygular. Uygulanan her metod için, onu test etmek üzere bir test programı yazın, buna *driver* denir. Yukarıdan aşağıya ve aşağıdan yukarıya yaklaşımlar eşit derecede iyidir: Her iki yaklaşım da metodları kademeli olarak uygular, programlama hatalarını izole etmeye yardımcı olur ve hata ayıklamayı kolaylaştırır. Birlikte kullanılabilirler.

alt-yukarı yaklaşım
driver

5.12.3 Uygulama Detayları

the `isLeapYear(int year)` yöntemi, Aşağıdaki kod kullanılarak uygulanabilir
Bölüm 3.12:

```
return (yıl % 400 == 0 || (yıl % 4 == 0 && yıl % 100 != 0));
```

Aşağıdaki bilgileri kullanarak `getTotalNumberOfDaysInMonth(int year, int month)` uygulayın:

- Ocak, Mart, Mayıs, Temmuz, Ağustos, Ekim ve Aralık 31 gündür.
- Nisan, Haziran, Eylül ve Kasım 30 gündür.
- Şubat, normal bir yılda 28 gün ve artık yılda 29 gündür. Bu nedenle, normal bir yıl 365 gündür, bir artık yıl 366 gündür.

`getTotalNumberOfDays(int year, int month)` uygulamak için, 1 Ocak 1800 ile takvim ayının ilk günü arasındaki toplam gün sayısını (`totalNumberOfDays`) hesaplamamız gerekir. 1800 yılı ile takvim yılı arasındaki toplam gün sayısını bulabilir ve ardından takvim yılındaki takvim ayından önceki toplam gün sayısını hesaplayabilirsiniz. Bu iki toplamın toplamı `totalNumberOfDays`'dir.

Bir gövde yazdırmak için, önce başlangıç gününden önce biraz boşluk ekleyin ve ardından her hafta için satırları yazdırın.

Tam program, Liste 5.12'de verilmiştir.

LISTELEN 5.12 PrintCalendar.java

```

1  import java.util.Scanner;
23
3  genel classtPrintCalendar {
4      /** Ana yöntem */
5      public static voidtmain(String[] args) {
6          Tarayıcı girişi = tnewScanner(System.in);
78
8          // Kullanıcıdan yılı girmesini iste
9          System.out.print("Tam yılı girin (örneğin, 2012): ");
10         intyıl = input.nextInt();
1112
12         // Kullanıcıdan ayı girmesini isteyin
13         System.out.print("Ayı 1 ile 12 arasında bir sayı olarak girin: ");
14         intay = input.nextInt();
1516
17         // Yılın ayı için takvimi yazdırın
18         printMonth(yıl, ay);
19
20         /** Bir yıldaki bir ayın takvimini yazdır */
21         baskıAy public static voidprintMonth(intyıl,intay) {
22             // Takvimin başlıklarını yazdırın
23             printMonthTitle(yıl, ay);
24
25             // Takvimin gövdesini yazdırın
26             printMonthBody(yıl, ay);
27         }
28
29         /** Ay başlığını yazdır, örneğin, Mart 2012 */
30         printMonthTitle public static voidprintMonthTitle(intyıl,intay) {
31             System.out.println("t" + getMonthName(month)n32 + t" " t+ yıl);
32
33             System.out.println("-----");
34             System.out.println(" Paz Sal Çar Per Cum Cmt");
35         }
36
37         /** Ayın İngilizce adını al */
38         getMonthName public static String ayAdınıAl(intay) {
39             String ayAdi = t"";
40             geçiş (ay) {
41                 case1: monthName = "Ocak"; break;
42                 case2: monthName = "Şubat"; break;
43                 case3: monthName = "Mart"; break;
44                 case4: monthName = "Nisan"; break;
45                 case5: monthName = "Mayıs"; break;
46                 case6: monthName = "Haziran"; break;
47                 case7: monthName = "Temmuz"; break;
48                 case8: monthName = "Ağustos"; break;
49                 case9: monthName = "Eylül"; break;
50                 case10: monthName = "Ekim"; break;
51                 case11: monthName = "Kasım"; break;
52                 case12: monthName = "Aralık";
53             }
54
55             returnmonthName;n
56 }
57
58         /** Ay gövdesini yazdır */

```



```

59 public static void printMonthBody(int yıl, int ay) { yazdırAyGövde
60 // Ayın ilk tarihinin haftanın başlangıç gününü al
61 int başlangıçGünü = getStartDay(yıl, ay);
62
63 // Aydaki gün sayısını al
64 int numberOfDaysInMonth = getNumberOfDaysInMonth(yıl, ay);
65
66 // Ayın ilk gününden önce boşluk ekle
67 int t = t0t;
68 for (i = 0; i < başlangıçGünü; i++)
69 System.out.print(" ");
70
71 for (i = 1; i <= ayınGünSayısı; i++) {
72     System.out.printf("%4d", i);
73
74     if ((i + başlangıçGünü) % 7 == 0)
75         System.out.println();
76 }
77
78 System.out.println();
79 }

80
81 /** Ay/1/yılın başlangıç gününü al */
82 public static int getStartDay(int yıl, int ay) { getStartDay
83     final int START_DAY_FOR_JAN_1_1800 = t3t;
84     // 1/1/1800'den ay/1/yıla kadar toplam gün sayısını al
85     int totalNumberOfDays = getTotalNumberOfDays(yıl, ay);
86
87     // Ay/1/yıl için başlangıç gününü döndür
88     return (totalNumberOfDays + START_DAY_FOR_JAN_1_1800) % 7;
89 }

90
91 /** 1 Ocak 1800'den bu yana toplam gün sayısını al */
92 public static int toplamGünSayısınıAl(int yıl, int ay) { getTotalNumberOfDays
93     int toplam = t0t;
94
95     // 1800'den 1/1/yıla kadar toplam günleri al
96     for (int t = t1800; t < yıl; t++)
97         if (isLeapYear(t))
98             toplam = toplam + t366;
99     // aksi takdirde
100 toplam = toplam + t365;
101
102     // Takvim ayından önceki aydan günleri ekle
103     for (int t = t1t; t < ay; t++)
104         toplam = toplam + getNumberOfDaysInMonth(yıl, t);
105
106     return toplam;
107 }

108
109 /** Bir ay içindeki gün sayısını al */
110 public static int ayınGünSayısınıAl(int yıl, int ay) { getNumberOfDaysInMonth
111     if (ay == t1t || ay == t3t || ay == t5t || ay == t7t ||
112         ay == 8 || ay == 10 || ay == 12)
113         return t31;
114
115     if (ay == 4 || ay == 6 || ay == 9 || ay == 11)
116         return t30;
117
118     if (ay == 2) return isLeapYear(yıl) ? 29 : 28;

```

```

119120         return 0; // Eğer ay yanlışsa
121     }
122123     /** Artık yıl olup olmadığını belirleyin */
124     public static boolean isLeapYear(int year) {
125         return yıl % 400 == 0 || (yıl % 4 == 0 & yıl % 100 != 0);
126     }
127 }

```

artık yıl

Program kullanıcı girişini doğrulamıyor. Örneğin, kullanıcı 1 ile 12 arasında olmayan bir ay veya 1800'den önce bir yıl girerse, program hatalı bir takvim gösterir. Bu hatayı önlemek için, takvimi yazdırmadan önce girişi kontrol etmek için bir **if** ifadesi ekleyin.

Bu program bir ay için takvimler yazdırır, ancak kolayca bir yıl için takvimler yazdıracak şekilde değiştirilebilir. Ocak 1800'den sonra yalnızca ayları yazdırabilse de, 1800'den önceki ayları yazdıracak şekilde değiştirilebilir.

5.12.4 Aşamalı İyileştirmenin Faydaları

Aşamalı iyileştirme, büyük bir problemi daha küçük yönetilebilir alt problemlere böler. Her alt problem bir yöntem kullanılarak uygulanabilir. Bu yaklaşım, programın yazılmasını, yeniden kullanılmasını, hata ayıklanmasını, test edilmesini, değiştirilmesini ve bakımını kolaylaştırır.

Daha Basit Program

Takvim yazdırma programı uzundur. Tek bir yöntemde uzun bir ifade dizisi yazmak yerine, aşamalı iyileştirme bunu daha küçük yöntemlere böler. Bu, programı basitleştirir ve tüm programın okunmasını ve anlaşılmasını kolaylaştırır.

Yöntemleri Yeniden Kullanma

Aşamalı iyileştirme, bir program içinde kodun yeniden kullanılmasını teşvik eder. **isLeapYear** yöntemi bir kez tanımlanır ve **getTotalNumberOfDays** ve **getNumberOfDayInMonth** yöntemlerinden çağrılır. Bu, gereksiz kodu azaltır.

Daha Kolay Geliştirme, Hata Ayıklama ve Test Etme

Her alt problem bir yöntemde çözüldüğünden, bir yöntem bireysel olarak geliştirilebilir, hata ayıklanabilir ve test edilebilir. Bu, hataları izole eder ve geliştirmeyi, hata ayıklamayı ve test etmeyi kolaylaştırır.

Büyük bir programı uygularken, yukarıdan aşağıya ve/veya aşağıdan yukarıya yaklaşımını kullanın. Tüm programı bir anda yazmayın. Bu yaklaşımları kullanmak daha fazla geliştirme süresi alıyormuş gibi görünse de (çünkü programı tekrar tekrar derleyip çalıştırıyorsunuz), aslında zaman kazandırır ve hata ayıklamayı kolaylaştırır.

Takım Çalışmasını Daha İyi Kolaylaştırma

Büyük bir problem alt programlara bölündüğünde, alt problemler farklı programcılara atanabilir. Bu, programcılarının ekipler halinde çalışmasını kolaylaştırır.

Anahtar Terimler

gerçek parametre 179
belirsiz çağrı 195
argüman 179
böl ve fethet 203
resmi parametre (yani, parametre) 179
bilgi gizleme 203
metot 178
metot soyutlama 203

metot aşırı yükleme 194
metot imzası 179
değiştirici 179
parametre 179
değer ile geçiş 186
bir değişkenin kapsamı 196
adım adım iyileştirme 203
stub 205

artımlı geliştirme ve
test etme

BÖLÜM ÖZETİ

1. Programları modüler ve yeniden kullanılabilir hale getirmek, yazılım mühendisliğindeki temel hedeflerden biridir. Java, bu hedefe ulaşmaya yardımcı olan birçok güçlü yapı sağlar. *Yöntemler* böyle bir yapıdır.
2. Yöntem başlığı, *değiştiricileri, dönüş değer türünü, yöntem adını ve parametreleri* belirtir. Bu bölümdeki tüm yöntemler için **static** değiştiricisi kullanılır.
3. Bir yöntem bir değer döndürebilir. **returnValueType**, yöntemin döndürdüğü değerin veri türüdür. Eğer yöntem bir değer döndürmüyorsa, **returnValueType** anahtar kelimesi **void**'dir.
4. parametre listesi, bir yöntemin parametrelerinin türünü, sırasını ve sayısını ifade eder. Yöntem adı ve parametre listesi birlikte yöntem imzasını oluşturur. Parametreler isteğe bağlıdır; yani, bir yöntem herhangi bir parametre içermek zorunda değildir.
5. Bir dönüş ifadesi, bir **void** yöntemde yöntemi sonlandırmak ve yöntemin çağrıldığı yere geri dönmek için de kullanılabilir. Bu, bazen bir yöntemde normal kontrol akışını aşmak için faydalıdır.
6. Bir yöntemge geçirilen argümanlar, yöntem imzasındaki parametrelerle aynı sayıda, türde ve sırada olmalıdır.
7. Bir program bir yöntemi çağırdığında, program kontrolü çağrılan yöntemge aktarılır. Bir çağrılan yöntem, dönüş ifadesi çalıştırıldığında veya yöntem sonlandırıcı kapama süslü paranteze ulaşıldığında kontrolü çağırıcıya geri döndürür.
8. Java'da bir değer döndüren yöntem, bir ifade olarak da çağrılabilir. Bu durumda, çağırana basitçe dönen değeri göz ardı eder.
9. Bir yöntem aşırı yüklenebilir. Bu, iki yöntemin aynı adı taşıyabileceği anlamına gelir, yeter ki yöntem parametre listeleri farklı olsun.
10. Bir yöntemde tanımlanan bir değişkene yerel değişken denir. Yerel değişkenin kapsamı, tanımından başlar ve değişkenin bulunduğu bloğun sonuna kadar devam eder. Bir yerel değişken, kullanılmadan önce tanımlanmalı ve başlatılmalıdır.
11. Yöntem soyutlaması, bir yöntemin kullanımını uygulamasından ayırarak elde edilir. İstemci, bir yöntemin nasıl uygulandığını bilmeden bir yöntemi kullanabilir. Uygulamanın detayları, yöntemde kapsüllenmiş ve yöntemi çağırana istemciden gizlenmiştir. Bu, bilgi gizleme veya kapsülleme olarak bilinir.
12. Yöntem soyutlaması, programları düzenli, hiyerarşik bir şekilde modüler hale getirir. Kısa yöntemlerin koleksiyonları olarak yazılan programlar, aksi takdirde olacağı gibi yazılması, hata ayıklanması, bakımı ve değiştirilmesi daha kolaydır. Bu yazım tarzı ayrıca yöntem yeniden kullanılabilirliğini teşvik eder.
13. Büyük bir programı uygularken, yukarıdan aşağıya ve/veya aşağıdan yukarıya kodlama yaklaşımını kullanın. Tüm programı bir anda yazmayın. Bu yaklaşım, kodlama için daha fazla zaman alıyormuş gibi görünebilir (çünkü programı tekrar tekrar derleyip çalıştırıyorsunuz), ancak aslında zaman kazandırır ve hata ayıklamayı kolaylaştırır.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın www.cs.armstrong.edu/liang/intro9e/test.html.

Bölümler 5.2-5.9

- 5.1** (*Matematik: beşgen sayılar*) Beşgen bir sayı, $n(3n-1)/2$ olarak tanımlanır $n = 1, 2, \dots$, ve devam eder. Bu nedenle, ilk birkaç sayı 1, 5, 12, 22, ... Aşağıdaki başlığa sahip bir yöntem yazın, bu yöntem bir beşgen sayı döndürsün:

```
public static int getPentagonalNumber(int n)
```

Bu yöntemi kullanarak her satırda 10 sayı ile ilk 100 beşgen sayıyı görüntüleyen bir test programı yazın.

- *5.2** (*Bir tam sayının basamaklarını topla*) Bir tam sayının basamaklarının toplamını hesaplayan bir yöntem yazın. Aşağıdaki yöntem başlığını kullanın:

```
genel statik int sumDigits(long n)
```

Örneğin, `sumDigits(234)` 9 döner ($2+3+4$). (İpucu: Rakamları çıkarmak için % operatörünü kullanın ve çıkarılan rakamı kaldırmak için / operatörünü kullanın. Örneğin, 234'ten 4'ü çıkarmak için `234 % 10` (= 4). kullanın. 234'ten 4'ü kaldırmak için `234 / 10` (= 23). Bir döngü kullanarak tüm rakamlar çıkarılana kadar rakamı tekrar tekrar çıkarın ve kaldırın. Bir test programı yazın, kullanıcıdan bir tam sayı girmesini isteyin ve tüm rakamlarının toplamını gösterin.

- **5.3** (*Palindrom tam sayı*) Aşağıdaki başlıklarla yöntemleri yazın

```
// Bir tam sayının tersini döndür, yani, reverse(456) 654 döndürür
```

```
public static int reverse(int number)
```

```
// Eğer sayı bir palindrom ise true döndür
```

```
public static boolean isPalindrome(int number)
```

`reverse` metodunu `isPalindrome` uygulamak için kullanın. Bir sayı, tersinin kendisiyle aynı olması durumunda bir palindromdur. Kullanıcının bir tam sayı girmesini isteyen ve bu tam sayının palindrom olup olmadığını bildiren bir test programı yazın.

- *5.4** (*Bir tam sayıyı ters olarak göster*) Aşağıdaki başlığa sahip bir yöntem yazın

bir tam sayıyı ters sırayla göstermek için:

```
public static void reverse(int number)
```

Örneğin, `reverse(3456)6543` gösterir. Kullanıcının bir tam sayı girmesini isteyen ve tersini gösteren bir test programı yazın.

- *5.5** (*Üç sayıyı sırala*) Aşağıdaki başlığa sahip bir yöntem yazın

üç sayıyı artan sırayla göstermek için:

```
public static void displaySortedNumbers(double num1, double num2, double num3)
```

Kullanıcının üç sayı girmesini isteyen ve bunları artan sırayla göstermek için yöntemi çağıran bir test programı yazın.

- *5.6** (*Desenleri göster*) Aşağıdaki gibi bir deseni göstermek için bir yöntem yazın:

```

      1
     2 1
    3 2 1
...
n n-1 ... 3 2 1
```



VideoNotu

Bir tam sayıyı ters çevirin

Yöntem başlığı şudur

genel statik voidtdisplayPattern(tinttn)

- *5.7** (*Finans uygulaması: gelecekteki yatırım değerini hesapla*) Belirli bir faiz oranı için belirli sayıda yıl boyunca gelecekteki yatırım değerini hesaplayan bir yöntem yazın. Gelecekteki yatırım, Programlama Egzersizi 2.21'deki formül kullanılarak belirlenir.

Aşağıdaki yöntem başlığını kullanın:

```
public static double futureYatirimDeğeri(double yatirimTutarı, double aylıkFaizOranı, int yıl)
```

Örneğin, **futureInvestmentValue(10000, 0.05/12, 5)** 12833.59 döner.

Kullanıcıdan yatırım miktarını (örneğin, 1000) ve faiz oranını (örneğin, %9) girmesini isteyen ve 1'den 30'a kadar olan yıllar için gelecekteki değeri gösteren bir tablo yazdıran bir test programı yazın:

Yatırılan miktar:	1000	Enter
Yıllık faiz oranı:	9	Enter
Yıllar	Gelecek Değer	
1	1093.80	
2	1196.41	
:		
29	13467.25	
30	14730.57	



- 5.8** (*Celsius ve Fahrenheit arasındaki dönüşümler*) Aşağıdaki iki yöntemi içeren bir sınıf yazın:

```
/** Celsius'tan Fahrenheit'a dönüşüm */
```

```
public static double celsiusToFahrenheit(double celsius)
```

```
/** Fahrenheit'tan Celsius'a dönüşüm */
```

```
genel statik double fahrenheitToCelsius(double fahrenheit)
```

Dönüşüm formülü:

$$\text{fahrenheit} = (9.0 / 5) * \text{celsius} + 32$$

$$\text{celsius} = (5.0 / 9) * (\text{fahrenheit} - 32)$$

Bu yöntemleri çağırarak aşağıdaki tabloları görüntüleyen bir test programı yazın:

Santigrat	Fahrenheit	Fahrenheit	Santigrat
40.0	104.0	120.0	48.89
39.0	102.2	110.0	43.33
...			
32.0	89.6	40.0	4.44
31.0	87.8	30.0	-1.11

- 5.9** (*Ayak ve metre arasındaki dönüşümler*) Aşağıdaki iki yöntemi içeren bir sınıf yazın:

```
/** Ayakları metreye dönüştür */
```

```
genel statik double footToMeter(double foot)
```

*/** Metreleri ayaklara dönüştür */*
genel statik doubletmetreToFoot(tdoubletmetre)

Dönüşüm formülü:

metre = 0,305 * ayak
 = 3,279 * metre

Bu yöntemleri çağırarak aşağıdaki tabloları görüntüleyen bir test programı yazın:

Ayak	Metre	Metre	Ayak
1.0	0.305	20.0	65.574
2.0	0.610	25.0	81.967
...			
9.0	2.745	60.0	196.721
10.0	3.050	65.0	213.115

5.10 (*isPrime* Yöntemini Kullan) Liste 5.7, *PrimeNumberMethod.java*, bir sayının asal olup olmadığını test etmek için *isPrime(int number)* yöntemini sağlar. Bu yöntemi kullanarak **10000**'den küçük asal sayıların sayısını bulun.

5.11 (*Finans uygulaması: komisyonları hesaplama*) Programlama Egzersizi 4.39'daki şemayı kullanarak komisyonu hesaplayan bir yöntem yazın. Yöntemin başlığı aşağıdaki gibidir:

genel statik doubletcomputeCommission(tdoubletsalesAmount)

Aşağıdaki tabloyu görüntüleyen bir test programı yazın:

Satış Tutarı	Komisyon
10000	900.0
15000	1500.0
...	
95000	11100.0
100000	11700.0

5.12 (*Karakterleri görüntüle*) Aşağıdaki başlığı kullanarak karakterleri yazdıran bir yöntem yazın:

public static void tprintChars(tchartch1,tchartch2,tintnnumberPerLine)

Bu yöntem, belirtilen satır başına numaralarla **ch1** ve **ch2** arasındaki karakterleri yazdırır. **1** ile **2** arasında satır başına on karakter yazdıran bir test programı yazın. Karakterler tam olarak bir boşlukla ayrılmıştır.

***5.13** (*Toplam serisi*) Aşağıdaki seriyi hesaplamak için bir yöntem yazın:

$$m(i) = \frac{1}{2} + \frac{2}{3} + \dots + \frac{i}{i+1}$$

Aşağıdaki tabloyu görüntüleyen bir test programı yazın:

i	m(i)
	0.5000
	1.1667
...	
19	16.4023
20	17.3546

***5.14** (*Tahmin*)_{kişi başı} aşağıdaki seri kullanılarak hesaplanabilir:

$$m(i) = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots + \frac{(-1)^{i+1}}{2i-1} \right)$$

Verilen bir i için $m(i)$ döndüren bir yöntem yazın ve aşağıdaki tabloyu gösteren bir test programı yazın:

	$m(i)$
1	4.0000
101	3.1515
201	3.1466
301	3.1449
401	3.1441
501	3.1436
601	3.1433
701	3.1430
801	3.1428
901	3.1427



***5.15** (*Finans uygulaması: bir vergi tablosu yazdır*) Liste 3.6, vergi hesaplamak için bir program vermektedir.

Aşağıdaki başlığı kullanarak vergi hesaplamak için bir yöntem yazın:

public static double computeTax(**int** durum, **double** vergilendirilebilirGelir)

Bu yöntemi kullanarak, vergilendirilebilir geliri \$50,000 ile \$60,000 arasında \$50 aralıklarla yazdıran bir program yazın.

Aşağıdaki tüm durumlar için:

Vergilendirilebilir Başkani	Bekar	Evlilik Birleşik veya Nitelikli Ayrı bir Ev Ayrı Dul(erkek)	Evlilik Ayrı	Gelir bir Ev
50000	8688	6665	8688	7352
50050	8700	6673	8700	7365
...				
59950	11175	8158	11175	9840
60000	11188	8165	11188	9852

***5.16** (*Bir yıldaki gün sayısı*) Aşağıdaki başlığı kullanarak bir yıl içindeki gün sayısını döndüren bir yöntem yazın:

public static int yılİçindekiGünSayısı(**int** yıl)

2000'den 2020'ye kadar olan yıllardaki gün sayısını gösteren bir test programı yazın.

Bölümler 5.10–5.11

***5.17** (*0 ve 1'lerden oluşan matris göster*) Aşağıdaki başlığı kullanarak bir n çarpanına sahip n matrisini gösteren bir yöntem yazın:

public static void matrisYazdır(**int** n)

Her bir eleman 0 veya 1'dir ve rastgele üretilir. Kullanıcıdan n girmesini isteyen ve bir n çarpanına sahip n matrisini gösteren bir test programı yazın. İşte bir örnek çalışma:

```
n girin: 3
0 1 0
0 0 0
1 1 1
```



- 5.18** (*Math.sqrt* yöntemi kullanın) Aşağıdaki tabloyu yazdıran bir program yazın *sqrt* yöntemini *Math* sınıfında kullanarak.

Sayı	Karekök
0	0.0000
2	1.4142
...	
18	4.2426
20	4.4721

- *5.19** (*MyTriangle* sınıfı) Aşağıdaki iki yöntemi içeren *MyTriangle* adında bir sınıf oluşturun:

```
/** İki kenarın toplamı üçüncü kenardan
 * büyükse true döndür. */
public static boolean isValid(
    double side1, double side2, double side3)

/** Üçgenin alanını döndür. */
public static double area(
    çifttaraf1, double taraf2, double taraf3)
```

Üçgenin üç kenarını okuyan ve eğergirdi geçerliyse alanı hesaplayan bir test programı yazın. Aksi takdirde, girdinin geçersiz olduğunu gösterir. Üçgenin alanını hesaplama formülü Programlama Egzersizi 2.15'te verilmiştir.

- 5.20** (*Trigonometrik yöntemleri kullanın*) Aşağıdaki tabloyu, 0'dan 360'a kadar 10 derece aralıklarla *sin* ve *cos* değerlerini gösterecek şekilde yazdırın. Değeri, ondalık noktadan sonra dört basamak tutacak şekilde yuvarlayın.

Derece	Günah	Çünkü
0	0.0000	1.0000
10	0.1736	0.9848
...		
350	-0.1736	0.9848
360	0.0000	1.0000

- *5.21** (*Geometri: büyük daire mesafesi*) Büyük daire mesafesi, bir kürenin yüzeyindeki dua nokta arasındaki mesafedir. (x_1, y_1) ve (x_2, y_2) iki noktanın coğrafi enlem ve boylamını temsil etsin. İki nokta arasındaki büyük daire mesafesi, aşağıdaki formül kullanılarak hesaplanabilir:

$$d = radius \times \arccos(\sin(x_1) \times \sin(x_2) + \cos(x_1) \times \cos(x_2) \times \cos(y_1 - y_2))$$

Kullanıcıdan, yeryüzündeki iki noktanın enlem ve boylamını derece cinsinden girmesini isteyen ve büyük daire mesafesini gösteren bir program yazın. Ortalama yeryüzü yarıçapı 6,371.01 km'dir. Java'nın trigonometrik yöntemleri radyan kullanıldığı için, dereceleri *Math.toRadians* yöntemi ile radyana çevirmeniz gerektiğini unutmayın. Formüldeki enlem ve boylam dereceleri Kuzey ve Batı içindir.

Güney ve Doğu derecelerini belirtmek için negatif kullanın. İşte bir örnek çalışma:



1. noktayı (enlem ve boylam) derece cinsinden girin:

39.55 -116.25

2. noktayı (enlem ve boylam) derece cinsinden girin:

41.5 87.37

İki nokta arasındaki mesafe 10691.79183231593 km'dir

****5.22** (Matematik: karekökü yaklaşık hesaplama) Math sınıfında `sqrt` yöntemini uygulamak için birkaç teknik vardır. Bu tekniklerden biri Babil yöntemi olarak bilinir. Bir sayının, `n`, karekökünü yaklaşık olarak hesaplamak için aşağıdaki formülü kullanarak sürekli bir hesaplama yapar:

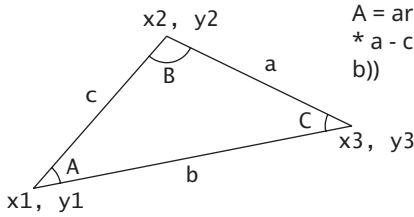
$$\text{nextGuess} = (\text{lastGuess} + n / \text{lastGuess}) / 2$$

`nextGuess` ve `lastGuess` neredeyse aynı olduğunda, `nextGuess` yaklaşık karekökü temsil eder. İlk tahmin herhangi bir pozitif değer olabilir (örneğin, 1). Bu değer, `lastGuess` için başlangıç değeri olacaktır. Eğer `nextGuess` ve `lastGuess` arasındaki fark çok küçük bir sayıdan, örneğin 0.0001, daha azsa, `nextGuess` sayısının yaklaşık karekökü olarak kabul edilebilir. Aksi takdirde, `nextGuess` `lastGuess` olur ve yaklaşık hesaplama süreci devam eder.

Aşağıdaki yöntemi uygulayın, bu yöntem `n` sayısının karekökünü döndürmelidir.

genel statik `doubletsqrt(tlongtn)`

***5.23** (Geometri: açıları göster) Kullanıcıdan bir üçgenin üç noktasını girmesini isteyen ve açıları derece cinsinden gösteren bir program yazın. Değeri ondalık noktadan sonra iki basamağa tutacak şekilde yuvarlayın. Açıları A, B ve C hesaplamak için kullanılan formüller aşağıdaki gibidir:



$$A = \arccos((a^2 + b^2 - c^2) / (-2 * b * c))$$

$$B = \arccos((b^2 + a^2 - c^2) / (-2 * a * c))$$

$$C = \arccos((c^2 - b^2 - a^2) / (-2 * a * b))$$

İşte programın bir örnek çalışması:

Üç nokta girin: 1 1 6.5 1 6.5 2.5 Enter
Üç açının değeri 15.26 90.0 74.74



Bölümler 5.10-5.12

****5.24** (Geçerli tarih ve saati göster) Liste 2.6, `ShowCurrentTime.java`, geçerli saati gösterir. Bu örneği, geçerli tarih ve saati gösterecek şekilde geliştirin. Liste 5.12'deki Takvim örneği, yıl, ay ve günü bulma konusunda size bazı fikirler vermelidir.

****5.25** (Milisaniyeleri saat, dakika ve saniyeye çevir) Aşağıdaki başlığı kullanarak milisaniyeleri saat, dakika ve saniyeye çeviren bir yöntem yazın:

`public static String ConvertMillis(tlongtmillis)`

Yöntem, `saat:dakika:saniye` biçiminde bir dize döndürür. Örneğin, `convertMillis(5500)` dizesi 0:0:5 döndürür, `convertMillis(100000)` dizesi 0:1:40 döndürür ve `convertMillis(555550000)` dizesi

154:19:10 döndürür.

Kapsamlı

****5.26**(*Palindromik asal*) Bir *palindromik asal*, asal bir sayıdır ve aynı zamanda palindromiktir. Örneğin, 131 bir asal ve aynı zamanda bir palindromik asaldır, 313 ve 757 de öyledir. İlk 100 palindromik asal sayıyı gösteren bir program yazın. Her satırda 10 sayıyı, tam olarak bir boşlukla ayrılmış olarak gösterin:

```
2 3 5 7 11 101 131 151 181 191
313 353 373 383 727 757 787 797 919 929
...
```

****5.27**(*Emirp*) Bir *emirp* (ters çevrildiğinde asal) palindromik olmayan bir asal sayıdır ve tersinin de asal olduğu bir sayıdır. Örneğin, 17 bir asal sayıdır ve 71 de bir asal sayıdır, bu nedenle 17 ve 71 emirplerdir. İlk 100 emirpi görüntüleyen bir program yazın. Her satırda 10 sayıyı, tam olarak bir boşlukla ayrılmış olarak aşağıdaki gibi görüntüleyin:

```
13 17 31 37 71 73 79 97 107 113
149 157 167 179 199 311 337 347 359 389
...
```

****5.28**(*Mersenne asal*) Bir asal sayıya *Mersenne asal* denir eğer $2^p - 1$ pozitif bir tam sayı p için. Tüm Mersenne asal sayılarını bulan bir program yazın ve çıktıyı aşağıdaki gibi görüntüleyin:

$p \ 2^p - 1$

```
3      7
5      31
...
```

****5.29**(*İkiz asal sayılar*) İkiz asal sayılar, 2 ile farklı olan bir asal sayı çiftidir. Örneğin, 3 ve 5 ikiz asal sayılardır, 5 ve 7 ikiz asal sayılardır ve 11 ve 13 ikiz asal sayılardır. 1.000'den küçük tüm ikiz asal sayıları bulan bir program yazın. Çıktıyı aşağıdaki gibi görüntüleyin:

```
(3, 5)
(5, 7)
...
```

****5.30**(*Oyun: zar*) Zar, kumarhanelerde oynanan popüler bir zar oyunudur. Oyunun bir varyasyonunu oynamak için bir program yazın, aşağıdaki gibi:

İki zar atın. Her zar, sırasıyla 1, 2, . . . , ve 6 değerlerini temsil eden altı yüzeye sahiptir. İki zarın toplamını kontrol edin. Eğer toplam 2, 3 veya 12 ise (bu duruma *craps* denir), kaybedersiniz; eğer toplam 7 veya 11 ise (bu duruma *doğal* denir), kazanırsınız; eğer toplam başka bir değere ise (yani, 4, 5, 6, 8, 9 veya 10), bir puan belirlenir. Zarları atmaya devam edin, ya 7 ya da aynı puan değeri atılana kadar. Eğer 7 atılırsa, kaybedersiniz. Aksi takdirde, kazanırsınız.

Programınız tek oyuncu olarak çalışır. İşte bazı örnek çalıştırmalar.



5 + 6 = 11 attınız
Kazandınız

1 + 2 = 3 attınız
Kaybettiniz



4 + 4 = 8 attınız
puan 8
6 + 2 = 8 attınız
Kazandınız



3 + 2 = 5 attınız
puan 5
2 + 5 = 7 attınız
Kaybettiniz

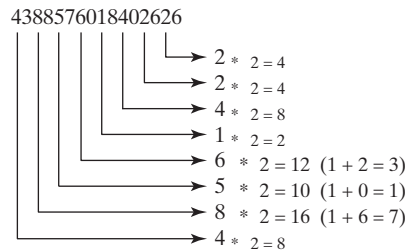


****5.3.1** (Finans: kredi kartı numarası doğrulama) Kredi kartı numaraları belirli kalıpları takip eder. Bir kredi kartı numarası 13 ile 16 rakam arasında olmalıdır. Şu rakamlarla başlamalıdır:

- 4 Visa kartları için
- 5 Master kartları için
- 37 American Express kartları için
- 6 Discover kartları için

1954 yılında, IBM'den Hans Luhn, kredi kartı numaralarını doğrulamak için bir algoritma önerdi. Bu algoritma, bir kart numarasının doğru girilip girilmediğini veya bir kredi kartının bir tarayıcı tarafından doğru bir şekilde taranıp taranmadığını belirlemek için faydalıdır. Kredi kartı numaraları, bu geçerlilik kontrolünü takip ederek üretilir ve genellikle *Luhn kontrolü* veya *Mod 10 kontrolü* olarak bilinir ve şu şekilde tanımlanabilir (örnek olarak, 4388576018402626 kart numarasını düşünün):

1. Sağdan sola her ikinci rakamı iki katına çıkarın. Bir rakamın iki katına çıkarılması iki basamaklı bir sayı ile sonuçlanırsa, iki basamağı toplayarak tek basamaklı bir sayı elde edin.



2. Şimdi Adım 1'den tüm tek basamaklı sayıları toplayın.

$$4 + 4 + 8 + 2 + 3 + 1 + 7 + 8 = 37$$

3. Kart numarasındaki sağdan sola tek haneli tüm rakamları toplayın.

$$6 + 6 + 0 + 8 + 0 + 7 + 8 + 3 = 38$$

4. Adım 2 ve Adım 3'ten elde edilen sonuçları toplayın.

$$37 + 38 = 75$$

5. Adım 4'ten elde edilen sonuç 10'a tam bölünebiliyorsa, kart numarası geçerlidir; aksi takdirde, geçersizdir. Örneğin, 4388576018402626 numarası geçersizdir, ancak 4388576018410707 numarası geçerlidir.

Kullanıcıdan bir kredi kartı numarasını uzun bir tamsayı olarak girmesini isteyen bir program yazın. Numaranın geçerli mi yoksa geçersiz mi olduğunu gösterin. Programınızı aşağıdaki yöntemleri kullanarak şekilde tasarlayın:

```
/** Kart numarası geçerliyse true döndür */
public static boolean isValid(long number)

/** Adım 2'den sonucu alın */
public static int sumOfDoubleEvenPlace(long number)

/** Bu rakam tek haneliyse döndür, aksi takdirde,
 * iki rakamın toplamını döndür */
public static int getDigit(int number)

/** Sayıdaki tek haneli rakamların toplamını döndür */
public static int sumOfOddPlace(long number)

/** d rakamı number için bir ön ekse true döndür */
public static boolean prefixMatched(long number, int d)

/** d'deki rakam sayısını döndür */
public static int getSize(long d)

/** number'dan ilk k rakamı döndür. Eğer
 * number'daki rakam sayısı k'dan azsa, number'ı döndür. */
public static long getPrefix(long number, int k)
```

İşte programın örnek çalışmaları:



Uzun bir tam sayı olarak bir kredi kartı numarası girin:
4388576018410707
4388576018410707 geçerlidir



Uzun bir tam sayı olarak bir kredi kartı numarası girin:
4388576018402626
4388576018402626 geçersizdir

****5.32** (Oyun: zar oyununda kazanma şansı) Alıştırma 5.30'u 10.000 kez çalışacak şekilde gözden geçirin ve kazanan oyunların sayısını gösterin.

****5.33** (Geçerli tarih ve saat) `System.currentTimeMillis()` çağırısı,

1 Ocak 1970 gece yarısından bu yana geçen süreyi milisaniye cinsinden döndürür. Tarih ve saati gösteren bir program yazın.

İşte bir örnek çalışma:



Geçerli tarih ve saat 16 Mayıs 2012 10:34:23

****5.34** (*Takvim yazdır*) Programlama Alıştırması 3.21, haftanın gününü hesaplamak için Zeller'in uyunu kullanır. Aylık başlangıç gününü almak için Zeller'in algoritmasını kullanarak Liste 5.12, PrintCalendar.java'yı basitleştirin.

5.35 (*Geometri: beşgenin alanı*) Bir beşgenin alanı aşağıdaki formül kullanılarak hesaplanabilir:

$$Area = \frac{5 \times s^2}{4 \times \tan\left(\frac{\pi}{5}\right)}$$

Bir kullanıcıdan bir beşgenin kenarını girmesini isteyen ve alanı gösteren bir program yazın. İşte bir örnek çalışma:

Kenar uzunluğunu girin: 5.5
Beşgenin alanı 52.04444136781625'tir



***5.36** (*Geometri: düzgün çokgenin alanı*) Düzgün bir çokgen, tüm kenarları aynı uzunlukta ve tüm açıları aynı dereceye sahip olan n kenarlı bir çokgendir (yani, çokgen hem eşkenar hem de eşaçıdadır). Düzgün bir çokgenin alanını hesaplamak için formül

$$Area = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$$

Bir düzenli çokgenin alanını döndüren bir yöntem yazın aşağıdaki başlık ile:

```
public static double area(int n, double side)
```

Kullanıcıdan bir düzenli çokgenin kenar sayısını ve kenar uzunluğunu girmesini isteyen ve alanını gösteren bir ana yöntem yazın. İşte bir örnek çalışma:

Kenar sayısını girin: 5
Kenar uzunluğunu girin: 6.5
Çokgenin alanı 72.69017017488385



5.37 (*Bir tam sayıyı biçimlendirin*) Belirtilen genişlikte tam sayıyı biçimlendirmek için aşağıdaki başlığa sahip bir yöntem yazın.

```
public static Stringtformat(tinttnumber,tinttwidth)
```

Yöntem, bir veya daha fazla ön ek 0 ile birlikte sayının bir dizesini döndürür. Dizenin boyutu genişliktir. Örneğin, format(34, 4) 0034 döndürür ve format(34, 5) 00034 döndürür. Eğer sayı genişlikten uzunsa, yöntem sayının dize temsilini döndürür. Örneğin, format(34, 1) 34 döndürür.

Kullanıcıdan bir sayı ve genişliğini girmesini isteyen ve `format(number, width)` çağrılarak döndürülen bir dizeyi gösteren bir test programı yazın.

***5.38** (*Rastgele karakterler oluşturun*) Listing 5.10'daki `RandomCharacter` yöntemlerini kullanarak 100 büyük harf ve ardından 100 tek haneli rakam yazdırın, her satıra on tane yazdırın.

5.39 (*Geometri: nokta konumu*) Programlama Egzersizi 3.32, bir noktanın yönlendirilmiş bir çizginin sol tarafında, sağ tarafında veya aynı çizgide olup olmadığını test etmenin nasıl yapılacağını gösterir. Aşağıdaki başlıklara sahip yöntemleri yazın:

```
/** Nokta (x2, y2)
```

```
* (x0, y0) ile (x1, y1) arasındaki yönlü doğrunun sol tarafındaysa true döndür */
```

```
public static boolean leftOfTheLine(double x0, double y0, double x1, double y1, double x2, double y2)
```

```
/** Nokta (x2, y2)
```

```
* (x0, y0) ile (x1, y1) arasındaki aynı doğrudaysa true döndür */
```

```
genel statik boolean onTheSameLine(double x0, double y0, double x1, double y1, double x2, double y2)
```

```
/** Nokta (x2, y2)
```

```
* (x0, y0) ile (x1, y1) arasındaki doğru parçasındaysa true döndür */
```

```
genel statik boolean onTheLineSegment(double x0, double y0, double x1, double y1, double x2, double y2)
```

Kullanıcıdan p_0 , p_1 ve p_2 için üç nokta girmesini isteyen bir program yazın ve p_2 'nin p_0 'dan p_1 'e giden doğrunun solunda, sağında, aynı

doğruda veya doğru parçasında olup olmadığını gösterin. İşte bazı örnek çalıştırmalar:



p_0 , p_1 ve p_2 için üç nokta girin:

(1.5, 1.5) (1.0, 1.0) ile (2.0, 2.0) arasındaki doğru parçasındadır

1 1 2 2 1.5 1.5

↵ Enter



p_0 , p_1 ve p_2 için üç nokta girin:

(3.0, 3.0) (1.0, 1.0) ile (2.0, 2.0) arasındaki aynı doğrudadır

1 1 2 2 3 3

↵ Enter



p_0 , p_1 ve p_2 için üç nokta girin:

(1.0, 1.5) çizginin sol tarafındadır

(1.0, 1.0) ile (2.0, 2.0) arasında

1 1 2 2 1 1.5

↵ Enter



p_0 , p_1 ve p_2 için üç nokta girin:

(1.0, -1.0) çizginin sağ tarafındadır

(1.0, 1.0) ile (2.0, 2.0) arasında

1 1 2 2 1 -1

↵ Enter

BÖLÜM

6

SİNGLE-DİMENSIYONAL ARRAYLER

Hedefler

- Dizilerin programlamada neden gerekli olduğunu açıklamak (§6.1).
- Dizi referans değişkenlerini tanımlamak ve diziler oluşturmak (§§6.2.1–6.2.2).
- Dizi boyutunu `arrayRefVar.length` kullanarak elde etmek ve bir dizideki varsayılan değerleri bilmek (§6.2.3).
- İndeksli değişkenler kullanarak dizi elemanlarına erişmek (§6.2.4).
- Bir dizi başlatıcısı kullanarak bir dizi tanımlamak, oluşturmak ve başlatmak (§6.2.5).
- Ortak dizi işlemlerini programlamak (dizileri görüntüleme, tüm elemanları toplama, minimum ve maksimum elemanları bulma, rastgele karıştırma ve elemanları kaydırma) (§6.2.6).
- For-each döngülerini kullanarak programlamayı basitleştirmek (§6.2.7).
- Uygulama geliştirmede dizileri uygulamak (`LottoNumbers`, `DeckOfCards`) (§§6.3–6.4).
- Bir dizinin içeriğini başka bir diziye kopyalamak (§6.5).
- Dizi argümanları ve dönüş değerleri ile yöntemler geliştirmek ve çağırmak (§§6.6–6.8).
- Değişken uzunlukta argüman listesi ile bir yöntem tanımlamak (§6.9).
- Öğeleri doğrusal (§6.10.1) veya ikili (§6.10.2) arama algoritması kullanarak aramak.
- Seçim sıralama yaklaşımını kullanarak bir diziyi sıralamak (§6.11.1).
- Ekleme sıralama yaklaşımını kullanarak bir diziyi sıralamak (§6.11.2).
- `java.util.Arrays` sınıfındaki yöntemleri kullanmak (§6.12).



6.1 Giriş



Tek bir dizi değişkeni, büyük bir veri koleksiyonuna referans verebilir.

problem
neden dizi?

Sıklıkla, bir programın yürütülmesi sırasında büyük sayıda değeri saklamanız gerekecektir. Örneğin, 100 sayıyı okumanız, ortalamasını hesaplamanız ve ortalamanın üzerinde kaç sayı olduğunu bulmanız gerektiğini varsayalım. Programınız önce sayıları okur ve ortalamasını hesaplar, ardından her sayıyı ortalama ile karşılaştırarak ortalamanın üzerinde olup olmadığını belirler. Bu görevi başarmak için, tüm sayılar değişkenlerde saklanmalıdır. 100 değişken tanımlamanız ve neredeyse aynı kodu 100 kez tekrar yazmanız gerekir. Bu şekilde bir program yazmak pratik olmayacaktır. Peki, bu sorunu nasıl çözersiniz?

dizi nedir?

Verimli, organize bir yaklaşım gereklidir. Java ve diğer çoğu yüksek seviyeli dil, aynı türdeki elemanların sabit boyutlu ardışık koleksiyonunu saklayan bir veri yapısı olan *dizi* sağlar. Mevcut durumda, tüm 100 sayıyı bir diziye saklayabilir ve bunlara tek bir dizi değişkeni aracılığıyla erişebilirsiniz. Çözüm şu şekilde görünebilir:

dizi oluştur

```
1 genel classtAnalyzeNumbers {
2     public static voidtmain(String[] args) {
3         final inttNUMBER_OF_ELEMENTS = t100t;
4         doublet[] sayılar = tnew doublet[NUMBER_OF_ELEMENTS];
5         çift toplam = t0t;
6
7     java.util.Scanner girişi = tnewt java.util.Scanner(System.in);
8     fort(tintti = t0t; i < NUMBER_OF_ELEMENTS; i++) {
9         System.out.print("Yeni bir sayı girin: ");
10        sayılar[i] = input.nextDouble();
11    toplam += sayılar[i];n12 }
```

sayılar	dizi
sayılar[0]:	
sayılar[1]:	
sayılar[2]:	
sayılar[i]	:
sayılar[97]:	.
sayılar[98]:	
sayılar[99]:	

sayıyı dizide sakla

ortalama al

```
13
14     doubletaverage = toplam / NUMBER_OF_ELEMENTS;
15
16     intcount = 0; // Ortalama üzerindeki eleman sayısı
17     fort(tintti = t0t; i < NUMBER_OF_ELEMENTS; i++)
18         ift(sayılar[i] > ortalama)
19             sayım++;
20
21     System.out.println("Ortalama "+ average);
22     System.out.println("Ortalama üzerindeki eleman sayısı "
23         + count);
24 }Nah}
```

ortalamanın üstünde mi?

Program, 4. satırda **100** elemanlı bir dizi oluşturur, 10. satırda dizinin içine sayıları depolar, 11. satırda her sayıyı **toplam**'a ekler ve 14. satırda ortalamayı elde eder. Daha sonra, dizideki her sayıyı ortalama ile karşılaştırarak ortalamanın üzerindeki değerlerin sayısını sayar (16-19. satırlar).

Bu bölüm, tek boyutlu dizileri tanıtmaktadır. Bir sonraki bölüm, iki boyutlu ve çok boyutlu dizileri tanıttacaktır.

6.2 Dizi Temelleri



Bir dizi oluşturulduğunda, boyutu sabittir. Bir dizi referans değişkeni, bir indeks kullanarak dizideki elemanlara erişmek için kullanılır.

indeks

Bir dizi, bir veri koleksiyonunu depolamak için kullanılır, ancak genellikle bir diziyi aynı türdeki değişkenlerin bir koleksiyonu olarak düşünmenin daha faydalı olduğunu buluruz. Bireysel değişkenler tanımlamak yerine, örneğin **number0, number1, ...**, ve **number99**, bir dizi değişkeni olarak **numbers** tanımlarsınız ve **numbers[0], numbers[1], ...**, ve **numbers[99]**'yi bireysel değişkenleri temsil etmek için kullanırsınız.

Bu bölüm, dizi değişkenlerini nasıl tanımlayacağınızı, dizileri nasıl oluşturacağınızı ve dizileri indeksli değişkenler kullanarak nasıl işleyeceğinizi tanıtmaktadır.

6.2.1 Dizi Değişkenlerini Bildirme

Bir programda bir diziyi kullanmak için, diziyi referans alacak bir değişken bildirmeli ve dizinin *eleman türünü* belirtmelisiniz. İşte bir dizi değişkeni bildirmek için kullanılan sözdizimi:

eleman türü

```
elementType[] arrayRefVar;
```

elementType her türlü veri türü olabilir ve dizideki tüm elemanlar aynıdata türüne sahip olacaktır. Örneğin, aşağıdaki kod, bir dizi çift elemanını referans alan myList adında bir değişken bildirir.

```
double[] listem;
```



Not

Ayrıca `elementType arrayRefVar[]` kullanarak bir dizi değişkeni de bildirebilirsiniz. Bustil C dilinden gelmektedir ve C programcılarını accommodate etmek için Java'da benimsenmiştir. `elementType[] arrayRefVar` stili tercih edilmektedir.

tercih edilen sözdizimi

6.2.2 Diziler Oluşturma

Temel veri türü değişkenleri için bildirimlerin aksine, bir dizi değişkeninin bildirimi dizin bellekte herhangi bir alan ayırmaz. Sadece bir diziye referans için bir depolama yeri oluşturur. Eğer bir değişken bir diziye referans içermiyorsa, değişkenin değeri **null** olur. Bir diziye eleman atayamazsınız, eğer dizi daha önce oluşturulmamışsa. Bir dizi değişkeni bildirildikten sonra, aşağıdaki sözdizimi ile **new** operatörünü kullanarak bir dizi oluşturabilirsiniz:

boş

```
diziRefDeğişkeni = newelementType[diziBoyutu];
```

yeni operatörü

Bu ifade iki şey yapar: (1) `new elementType[dizi-`

`Boyutu]` kullanarak bir dizi oluşturur; (2) yeni oluşturulan dizinin referansını `diziRefDeğişkeni` değişkenine atar.

Bir dizi değişkeni tanımlamak, bir dizi oluşturmak ve dizinin referansını değişkeni ne atamak tek bir ifadede birleştirilebilir:

```
elementType diziRefDeğişkeni = newelementType[diziBoyutu];
```

veya

```
elementType diziRefDeğişkeni[] = newelementType[diziBoyutu];
```

İşte böyle bir ifadenin bir örneği:

```
double[] benimListem = new double[10];
```

Bu ifade bir dizi değişkeni, `benimListem`, tanımlar, `double`

tipinde on elemanlı bir dizi oluşturur ve referansını `benimListem` değişkenine atar. Elemanlara değer atamak için şu sözdizimini kullanın:

```
arrayRefVar[index] = value;
```

Örneğin, aşağıdaki kod diziyi başlatır.

```
mlistt0t = t5.sh;nmlt  
stt1t = tch.5t;nmlistt  
2t = tz.zt;nmlistshtzt  
sh = t13.2t;
```

```
Melissa[t4t] = T4.0t
;
```

Bu dizi Şekil 6.1'de gösterilmektedir.

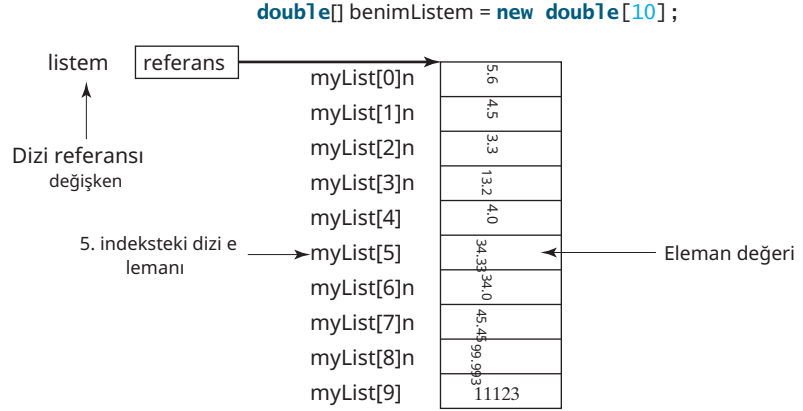


FIGURE 6.1 Dizi `myList` on adet `double` türünde eleman ve 0 ile 9 arasında `int` indeksleri vardır.



Not

Bir dizi değişkeni, bir diziyi tutuyormuş gibi görünse de aslında o diziyi bir referans içerir. Kesin olarak konuşursak, bir dizi değişkeni ve bir dizi farklıdır, ancak çoğu zaman bu ayrılm göz ardı edilebilir. Bu nedenle, basitlik açısından, `myList`'in on çift eleman içeren bir diziye referans içeren bir değişken olduğunu daha uzun bir şekilde belirtmek yerine, bir dizi olduğunu söylemekte bir sakınca yoktur.

dizi vs. dizi değişkeni

6.2.3 Dizi Boyutu ve Varsayılan Değerler

Bir dizi için alan tahsis edildiğinde, dizinin boyutu belirtilmeli ve içinde saklanabilecek eleman sayısı tanımlanmalıdır. Bir dizi oluşturulduktan sonra boyutu değiştirilemez. Boyut, `arrayRefVar.length` kullanılarak elde edilebilir. Örneğin, `myList.length 10`'dur.

Bir dizi oluşturulduğunda, elemanları sayısal ilkel veri türleri için varsayılan değer olarak 0, `char` türleri için `\u0000` ve `boolean` türleri için `false` atanır.

dizi uzunluğu

varsayılan değerler

6.2.4 Dizi İndeksli Değişkenler

Dizi elemanlarına indeks aracılığıyla erişilir. Dizi indeksleri 0 tabanlıdır; yani, 0'dan `arrayRefVar.length-1`'a kadar uzanır. Şekil 6.1'deki örnekte, `myList` on çift değeri tutar ve indeksler 0'dan 9'a kadardır.

Dizideki her bir eleman, *indeksli değişken* olarak bilinen aşağıdaki sözdizimi kullanılarak temsil edilir:

```
arrayRefVar[dizin];
```

Örneğin, `myList[9]`, dizi `myList` içindeki son öğeyi temsil eder.

0 dayalı

indeksli değişken



Dikkat

Bazı programlama dilleri, bir dizi öğesine referans vermek için parantez kullanır, örneğin `myList(9)`, ancak Java köşeli parantez kullanır, örneğin `myList[9]`.

Bir dizi oluşturulduktan sonra, bir indeksli değişken, normal bir değişken gibi kullanılabilir. Örneğin, aşağıdaki kod `myList[0]` ve `myList[1]` değerlerini `myList[2]`'ye ekler.

```
listem[t2t] = listem[t0t] + listem[t1t];
```

Aşağıdaki döngü 0 değerini `myList[0]`, 1 değerini `myList[1]`, ..., ve 9 değerini `myList[9]` atar:

```
fort(tint = t0t; i < listem.uzunluk; i++) { nmyList[i] = i;
}
```

6.2.5 Dizi Başlatıcıları

Java, *dizi başlatıcısı* olarak bilinen bir kısayol notasyonu sunar; bu, bir dizinin beyanını, dizi başlatıcı oluşumunu ve başlatılmasını tek bir ifadede birleştirir ve aşağıdaki sözdizimini kullanır:

```
elementType[] arrayRefVar = {değer0, değer1, ..., değerk};
```

Örneğin, bu ifade

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

diziyi `myList` dört elemanla beyan eder, oluşturur ve başlatır; bu, aşağıdaki ifadelerle eşdeğerdir:

```
doublet[] myList = tnew doublet[t4t];nmy
List[t0t] = t1.9t;
Melissa[t1t] = Taq.qat;
Namilisa[tat] = Ta.4t;N
amilisa[tat] = Ta.khat;
```



Dikkat

`new` operatörü dizi başlatıcı sözdiziminde kullanılmaz. Bir dizi başlatıcısı kullanarak, diziyi tek bir ifadede beyan etmeli, oluşturmalı ve başlatmalısınız. Bunu ayırmak, sözdizimi hatasına neden olur. Bu nedenle, bir sonraki ifade yanlıştır:

```
doublet[] myList;nmy
yList = {t1.9t,t2.9t,t3.4t,t3.5t};
```

6.2.6 Dizi İşleme

Dizi elemanlarını işlerken, genellikle iki nedenle bir `for` döngüsü kullanacaksınız:

- Bir dizideki tüm elemanlar aynı türdendir. Bir döngü kullanarak tekrar tekrar aynı şekilde eşit olarak işlenirler.
- Dizi boyutu bilindiğinden, bir `for` döngüsü kullanmak doğaldır.

Dizinin aşağıdaki gibi oluşturulduğunu varsayalım:

```
doublet[] myList = tnew doublet[t10t];
```

Aşağıda dizilerin işlenmesine dair bazı örnekler bulunmaktadır.

1. Dizileri giriş değerleri ile başlatma: Aşağıdaki döngü, diziyi `myList`

kullanıcı giriş değerleri ile başlatır.

```
java.util.Scanner input = new java.util.Scanner(System.in);
System.out.print("Değerleri girin: "+ myList.length + " değer: ");
for(int i = 0; i < myList.length; i++)
    myList[i] = input.nextDouble();
```

2. *Dizileri rastgele değerlerle başlatma:* Aşağıdaki döngü diziyi `myList0.0` ile `100.0` arasında, ancak `100.0`'dan daha az rastgele değerlerle başlatır.

```
for(tintti = t0t; i < myList.length; i++) {nmyList[i] = M
    ath.random() * t100t;
}
```

3. *Dizileri görüntüleme:* Bir diziyi yazdırmak için, dizideki her bir öğeyi aşağıdaki gibi bir döngü kullanılarak yazdırmalısınız:

```
for(tintti = t0t; i < benimListem.uzunluk; i++) {nSyste
    m.out.print(myList[i] + t" "t);
}
```

karakter dizisini yazdır



Ttftp

`char[]` türünde bir dizi, tek bir yazdırma ifadesi kullanılarak yazdırılabilir. Örneğin, aşağıdaki kod `Da11as`'ı görüntüler:

```
chart[] şehir = {t'D't,t'a't,t'n't,t'n't,t'at,t's};System.out.printl
n(şehir);
```

4. *Tüm elemanları toplama:* Toplamı saklamak için `total` adında bir değişken kullanın. Başlangıçta `total` 0'dır. Dizi içindeki her elemanı `total`'a eklemek için aşağıdaki gibi bir döngü kullanın:

```
doublettoplam = t0t;
for(tintti = t0t; i < listem.uzunluk; i++) {ntotal += liste
    m[i];
}
```

5. *En büyük elemanı bulma:* En büyük elemanı saklamak için `max` adında bir değişken kullanın. Başlangıçta `maxmyList[0]`'dır. Dizi `myList` içindeki en büyük elemanı bulmak için, her elemanı `max` ile karşılaştırın ve eğer eleman `max`'tan büyükse `max`'ı güncelleyin.

```
doubletmax = listem[t0t];nfor
t(tintti = t1t; i < listem.uzunluk; i++) {nift(listem[i] > m
    ax) max = listem[i];n}
```

6. *En büyük elemanın en küçük indeksini bulma:* Genellikle bir dizideki en büyük elemanı bulmanız gerekir. Eğer bir dizide birden fazla en büyük eleman varsa, böyle bir elemanın en küçük indeksini bulun. Varsayalım ki dizi `myList` şudur: `{1,5,3,4,5,5}`. En büyük eleman `5` ve `5` için en küçük indeks `1`'dir. En büyük elemanı saklamak için `max` adında bir değişken ve en büyük elemanın indeksini belirtmek için `indexOfMax` adında bir değişken kullanın. Başlangıçta `maxmyList[0]`'dır ve `indexOfMax0`'dır. Dizi içindeki her elemanı `max` ile karşılaştırın ve eğer eleman `max`'tan büyükse `max` ve `indexOfMax`'ı güncelleyin.

```
doubletmax = listem[t0t];nin
tindexOfMax = t0t;
for(tintti = t1t; i < listem.uzunluk; i++) {nif
    (Listem[i] > maks) {
        nmax = myList[i];nin
        dexOfMax = i;
    }
}
```



VideoNotu

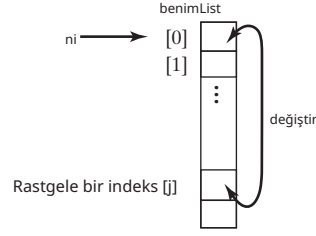
Rastgele karıştırma

7. *Rastgele karıştırma:* Birçok uygulamada, bir dizideki elemanları rastgele yeniden sıralamanız gerekir. Buna *karıştırma* denir. Bunu başarmak için, her bir eleman için

`myList[i]`, rastgele bir indeks `j` oluştur ve `myList[i]` ile `myList[j]`'yi değiştir, bunun gibi:

```
for (int i = 0; i < myList.length; i++) {
    // Rastgele bir indeks j oluştur
    int j = (int) (Math.random() * list
        em.uzunluk);

    // myList[i] ile myList[j]'yi değiştir
    double temp = listem[i]; n list
    em[i] = listem[j] n listem[j]
    = temp;
}
```

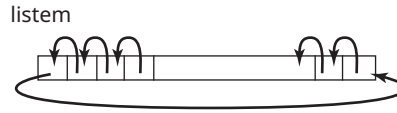


8. *Elemanları kaydırma*: Bazen elemanları sola veya sağa kaydırmanız gerekir. İşte elemanları bir pozisyon sola kaydırmanın ve son elemanı ilk elemanla doldurmanın bir örneği:

```
double temp = myList[0]; // İlk elemanı koru

// Elemanları sola kaydır
for (int i = 1; i < listem.uzunluk; i++) { n listem[i - t
    1t] = listem[i];
}

// İlk öğeyi son pozisyonu doldurmak için taşı
myList[myList.length - 1] = temp;
```



9. *Kodlamayı basitleştirme*: Diziler, belirli görevler için kodlamayı büyük ölçüde basitleştirmek için kullanılabilir. Örneğin, bir ayın numarasına göre İngilizce adını almak istediğinizi varsayalım. Ay adları bir dizide saklanıyorsa, belirli bir ay için ay adı basitçe indeks aracılığıyla erişilebilir. Aşağıdaki kod, kullanıcıdan bir ay numarası girmesini ister ve ay adını gösterir:

```
String[] months = {"Ocak", "Şubat", ..., "Aralık"};
System.out.print("Bir ay numarası girin (1 ile 12 arasında): ");
int monthNumber = input.nextInt();
System.out.println("Ay: " + months[monthNumber - 1]);
```

Eğer `months` dizisini kullanmasaydınız, ay adını aşağıdaki gibi uzun bir çoklu `if-else` ifadesi kullanarak belirlemeniz gerekirdi:

```
if (ayNumber == 1)
    System.out.println("Ay: Ocak");
else if (monthNumber == 2)
    System.out.println("Ay: Şubat");
...
// aksi takdirde
System.out.println("Ay: Aralık");
```

6.2.7 for-each Döngüleri

Java, bir diziyi sıralı olarak indeks değişkeni kullanmadan geçmenizi sağlayan, for-each döngüsü veya geliştirilmiş for döngüsü olarak bilinen kullanışlı bir `for` döngüsünü destekler. Örneğin, aşağıdaki kod, dizideki tüm elemanları `myList` olarak görüntüler:

```
for (tdoubletu: myList) { nSyste
    m.out.println(u); n}
```

230 Bölüm 6 Tek Boyutlu Diziler

Bu kodu “her bir eleman **umyList** içinde, aşağıdakileri yap.” şeklinde okuyabilirsiniz. Değişken **u**, **myList** içindeki elemanlarla aynı türde tanımlanmalıdır.

Genel olarak, bir for-each döngüsünün sözdizimi

```
for(elementType element: arrayRefVar) {  
    // Elemanı işleyin  
}
```

Diziyi farklı bir sırayla gezmek veya dizideki elemanları değiştirmek istiyorsanız, hala bir indeks değişkeni kullanmalısınız.



Dikkat

Dizinin sınırları dışında erişim, bir çalışma zamanı hatası olan yaygın bir programlama hatasıdır **ArrayIndexOutOfBoundsException**. Bunu önlemek için, **arrayRefVar.length - 1**'den daha büyük bir indeks kullanmadığınızdan emin olun.

Programcılar genellikle bir dizideki ilk elemanı **1** indeksi ile yanlışlıkla referans alırlar, ancak bu **0** olmalıdır. Bu duruma off-by-one hatası denir. Bir döngüde başka bir yaygın hata, **<** yerine **<=** kullanmaktır. Örneğin, aşağıdaki döngü yanlıştır.

```
for(renk tonu = t0t; i <= liste.uzunluk; i++)nSy  
    stem.out.print(list[i] + t" "t);
```

Değişkenin değerini **<** ile değiştirilmelidir.

DiziDışıHata-
İstisnası

bir eksik hata



MyProgrammingLab™

6.1 Bir dizi referans değişkeni nasıl tanımlanır ve bir dizi nasıl oluşturulur?

6.2 Bir dizi için bellek ne zaman ayrılır?

6.3 Aşağıdaki kodun çıktısı nedir?

```
inttx = t30t;  
intt[] sayılar = tnew intt[x];nx = t60  
t;  
System.out.println("x değeri "+ x);  
System.out.println("numbers dizisinin boyutu "+ numbers.length);
```

6.4 Aşağıdaki ifadeler için doğru **true** veya **false** belirtin:

- Bir dizideki her eleman aynı türdedir.
- Bir dizi referans değişkeni tanımlandıktan sonra dizi boyutu sabittir.
- Dizi oluşturulduktan sonra dizi boyutu sabittir.
- Bir dizideki elemanlar temel veri türü olmalıdır.

6.5 Aşağıdaki ifadelerden hangileri geçerlidir?

```
tnew intti grafiği();
```

6.6 Bir dizideki elemanlara nasıl erişirsiniz? Dizi indeksli bir değişken nedir?

6.7 Dizi indeks türü nedir? En düşük indeks nedir? **a** adında bir dizideki üçüncü elemanın temsili nedir?

6.8 Aşağıdakileri yapmak için ifadeler yazın:

- a. **10** çift değer tutacak bir dizi oluşturun.
- b. Dizideki son elemana **5.5** değerini atayın.
- c. İlk iki elemanın toplamını gösterin.
- d. Dizideki tüm elemanların toplamını hesaplayan bir döngü yazın.

- e. Dizi içindeki en küçük öğeyi bulan bir döngü yazın.
- f. Rastgele bir indeks oluşturun ve bu indeksin dizideki öğesini görüntüleyin.
- g. Başlangıç değerleri ile başka bir dizi oluşturmak için bir dizi başlatıcısı kullanın 3.5.5.5, 4.52 ve 5.6.

6.9 Programınız geçersiz bir indeks ile bir dizi öğesine erişmeye çalıştığında ne olur?

6.10 Aşağıdaki koddaki hataları tanımlayın ve düzeltin:

```
1 genel sınıfTest {
2     public static voidtmain(String[] args) {
3         ikili[t100t]r;
4
5         fort(tintti = t0t; i < r.length(); i++);n6 r(i) = Math.r
        andom * t100t;n7 }
8     }
```

6.11 Aşağıdaki kodun çıktısı nedir?

```
1 genel sınıfTest {
2     public static voidtmain(String[] args) {
3         inttlist[] = {t1t,t2t,t3t,t4t,t5t,t6t};
4         fort(tintti = t1t; i < liste.uzunluk; i++)
5 liste[i] = liste[i - t1t];
6
7         fort(tintti = t0t; i < liste.uzunluk; i++)n8 System.
        out.print(list[i] + t" "t);n9 }
10     }
```

6.3 Vaka Çalışması: Lotto Numaraları

Problem, tüm giriş sayıların 1 ile 99'u kapsayıp kapsamadığını kontrol eden bir program yazmaktır.

Pick-10 loto için her biletin 10 benzersiz numarası vardır ve bu numaralar 1 ile 99 arasında değişir. Farz edelim ki birçok bilet alıyorsunuz ve bunların 1 ile 99 arasındaki tüm numaraları kapsamasını istiyorsunuz. Bir program yazın ki bilet numaralarını bir dosyadan okur ve tüm numaraların kapsanıp kapsamadığını kontrol eder. Dosyadaki son numaranın 0 olduğunu varsayın. Farz edelim ki dosya şu numaraları içeriyor

```
80 3 87 62 30 90 10 21 46 27
12 40 83 9 39 88 95 59 20 37
80 40 87 67 31 90 11 24 56 77
11 48 51 42 8 74 1 41 36 53
52 82 16 72 19 70 44 56 29 33
54 64 99 14 23 22 94 79 55 2
60 86 34 4 31 63 84 89 7 78
43 93 97 45 25 38 28 26 85 49
47 65 57 67 73 69 32 71 24 66
92 98 96 77 6 75 17 61 58 13
35 81 18 15 5 68 91 50 76
0
```

Programınız şunu göstermelidir

Biletler tüm sayıları kapsar

Dosyanın sayıları içerdiğini varsayalım

```
11 48 51 42 8 74 1 41 36 53
52 82 16 72 19 70 44 56 29 33
0
```



VideoNotu

Lotto numaraları

232 Bölüm 6 Tek Boyutlu Diziler

Programınız şunu göstermelidir

Biletler tüm sayıları kapsamaz

Bir sayıyı kapalı olarak nasıl işaretlersiniz? 99 boolean elemanına sahip bir dizi oluşturabilirsiniz. Dizideki her eleman, bir sayının kapalı olup olmadığını işaretlemek için kullanılabilir. Diziyi **isCovered** olarak adlandıralım. Başlangıçta, her eleman **false** olarak ayarlanmıştır, Şekil 6.2a'da gösterildiği gibi. Bir sayı okunduğunda, karşılık gelen eleman **true** olarak ayarlanır. Girilen sayıların **1,2,3,99** olduğunu varsayalım, **0**. Sayı **1** okunduğunda, **isCovered[0]** **true** olarak ayarlanır (bkz. Şekil 6.2b). Sayı **2** okunduğunda, **isCovered[2 - 1]** **true** olarak ayarlanır (bkz. Şekil 6.2c). Sayı **3** okunduğunda, **isCovered[3 - 1]** **true** olarak ayarlanır (bkz. Şekil 6.2d). Sayı **99** okunduğunda, **isCovered[98]** **true** olarak ayarlanır (bkz. Şekil 6.2e).

kapalı mı	kapalı mı	kapalı mı	kapalı mı	kapalı mı
[0] yanlış	[0] doğru	[0] doğru	[0] doğru	[0] doğru
[1] yanlış	[1] yanlış	[1] doğru	[1] doğru	[1] doğru
[2] yanlış	[2] yanlış	[2] yanlış	[2] doğru	[2] doğru
[3] yanlış	[3] yanlış	[3] yanlış	[3] yanlış	[3] yanlış
⋮	⋮	⋮	⋮	⋮
[97] yanlış	[97] yanlış	[97] yanlış	[97] yanlış	[97] yanlış
[98] yanlış	[98] yanlış	[98] yanlış	[98] yanlış	[98] doğru
(a)	(b)	(c)	(d)	(e)

Fiç 6.2Eğer sayı **i** bir Lotto biletinde görünüyorsa, **isCovered[i-1]** **true** olarak ayarlanır.

Programın algoritması şu şekilde tanımlanabilir:

Dosyadan okunan her sayı için k,
numara k'yı isCovered[k-1] true olarak ayarlayarak kapsanmış olarak işaretleyin;
eğer her isCovered[i] true ise
Biletler tüm sayıları kapsar
aksi takdirde
Biletler tüm sayıları kapsamaz

Tam program Liste 6.1'de verilmiştir.

Liste 6.1 LottoNumbers.java

```
1  import java.util.Scanner;  
2  
3  genel sınıf LottoNumbers {  
4      public static void main(String[] args) {  
5          Tarayıcı girişi = new Scanner(System.in);  
6          boolean[] isCovered = new boolean[99]; // Varsayılan false  
7  
8          // Her sayıyı oku ve karşılık gelen elemanı kapalı olarak işaretle  
9          int number = input.nextInt();  
10         while (sayı != 0) {  
11             isCovered[sayı - 1] = true;   
12             sayı = input.nextInt();  
13         }  
14
```

dizi oluştur ve başlat

numarayı oku

numarayı işaretle

numarayı oku


```

15 // Tüm kapalı olup olmadığını kontrol et
16 boolean tümKaplı = tam; // Başlangıçta hepsinin kaplı olduğunu varsayalım
17 for(tint = t0t; i < isCovered.length; i++)
18     if(!isCovered[i]) {
19         tümKaplı = false; // Kaplı olmayan bir sayı bul
20         kırılma;
21     }
22 }
23 // Sonucu göster
24 if(tümKaplı)
25     System.out.println("Biletler tüm sayıları kapsıyor");
26 aksi takdirde
27     System.out.println("Biletler tüm sayıları kapsamıyor");
28 }
29 }

```

kontrol tümKaplı?

Varsayalım ki LottoNumbers.txt adında bir metin dosyası oluşturduunuz ve bu dosya aşağıdaki giriş verilerini içeriyor: 2 5

6 5 4 3 23 43 2 0. Programı aşağıdaki komutla çalıştırabilirsiniz:

```
java LottoNumbers < LottoNumbers.txt
```

Program şu şekilde izlenebilir:

Satır#	Dizideki temsilci elemanlar isCovered	numara	tümKaplı
	[1] [2] [3] [4] [5] [22] [42]		
6	yanlış yanlış yanlış yanlış yanlış yanlış yanlış		
9		2	
11	doğru		
12		5	
11			
12		6	
11			
12		5	
11			
12		4	
11			
12		3	
11	doğru		
12		23	
11			
12		43	
11			
12		2	
11	doğru		
12		0	
16			doğru
18(i=0)			yanlış



234 Bölüm 6 Tek Boyutlu Diziler

Program, her bir elemanı false olarak başlatılan **99 boolean** elemanlı bir dizi oluşturur (6. satır). İlk sayıyı dosyadan okur (9. satır). Program daha sonra aşağıdaki işlemleri bir döngü içinde tekrarlar:

- Sayı sıfır değilse, dizideki karşılık gelen değerini `isCovered` true olarak ayarla (11. satır);
- Sonraki sayıyı oku (12. satır).

Girdi **0** olduğunda, girdi sona erer. Program, tüm sayıların kapsanıp kapsanmadığını kontrol eder (16–21. satırlar) ve sonucu 24–27. satırlarda gösterir.

6.4 Vaka Çalışması: Deste Kartları

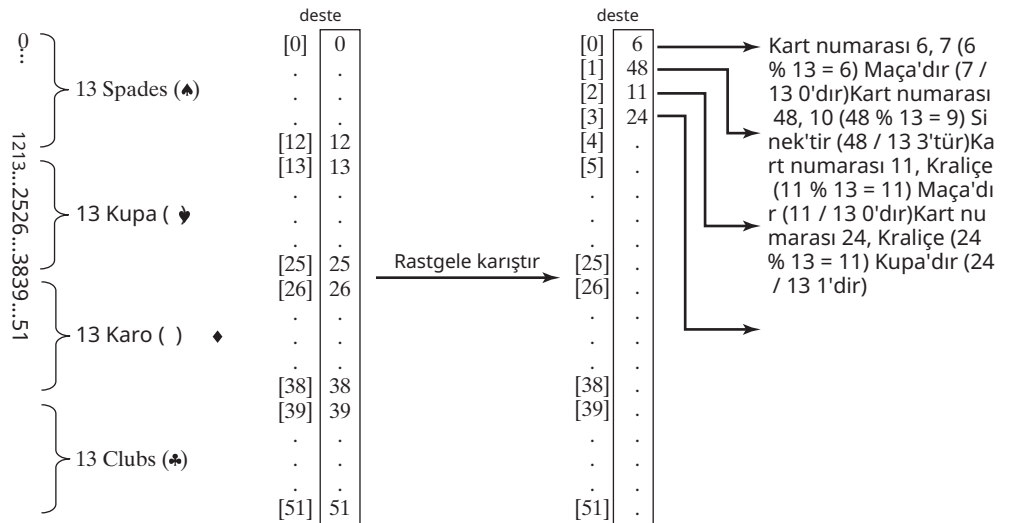


*Problem, bir deste karttan rastgele dört kart seçen bir program oluşturmak*tır.

Diyelim ki, 52 kartlık bir desteden rastgele dört kart seçecek bir program yazmak istiyorsunuz. Tüm kartlar, başlangıç değerleri **0** ile **51** arasında olan `deck` adında bir dizi kullanılarak temsil edilebilir, şu şekilde:

```
int[] deste = new int[52];  
  
// Kartları başlat  
for(int i = 0; i < deste.length; i++)  
    deste[i] = i;
```

Kart numaraları **0** ile **12,13** ile **25,26** ile **38**, ve **39** ile **51** sırasıyla 13 Karo, 13 Kupa, 13 Maç a ve 13 Sinek'i temsil eder, Şekil 6.3'te gösterildiği gibi. `kartNumarası / 13` kartın tür ünü belirler ve `kartNumarası % 13` kartın değerini belirler, Şekil 6.4'te gösterildiği gi bi. Diziyi `deste` karıştırdıktan sonra, `deste` den ilk dört kartı seçin. Program, bu dört kart numarasından kartları görüntüler.



FİKÜR 6.3 52 kart, `deste` adında bir dizide saklanmaktadır.

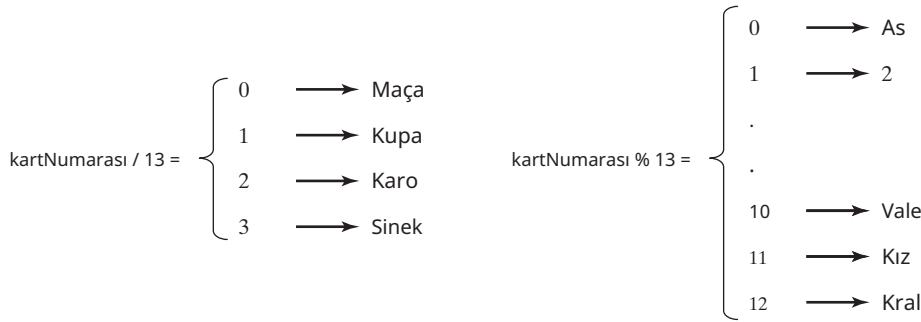


FIGURE 6.4 Kart numarasının bir kartın türünü ve rütbe numarasını nasıl tanımladığını gösterir.

Liste 6.2, sorunun çözümünü vermektedir.

LİSTE 6.2 DeckOfCards.java

```

1  genel sınıf DeckOfCards {
2      public static void main(String[] args) {
3          int[] destem = new int[52];
4          String[] takımlar = {"Maça", "Kupa", "karo", "Sinek"};
5          String[] rütbelere = {"As", "2", "3", "4", "5", "6", "7", "8", "9",
6              "10", "Jack", "Kraliçe", "Kral"};
7
8          // Kartları başlat
9          for (tinti = 0; i < destem.uzunluğu; i++) n10 dec
10         k[i] = i;
11
12         // Kartları karıştır
13         for (tinti = 0; i < güverte.uzunluğu; i++) {
14             // Rastgele bir indeks oluştur
15             int index = (tinti)(Math.random() * destem.length);
16             int temp = destem[i];
17         destem[i] = destem[index]; n18 güverte[in
18         deks] = sıcaklık; n19 }
19
20         // İlk dört kartı göster
21         for (tinti = 0; i < 4; i++) {
22             String suit = suits[destem[i] / 13];
23             String rank = ranks[destem[i] % 13];
24             System.out.println("Kart numarası " + destem[i] + ": "
25                 + rank + " of " + suit);
26         }
27     }
28 }
29 }

```

dizi oluştur destem
string dizisi
string dizisi

destem başlat

destem karıştır

bir kartın takımı
bir kartın rütbesi

Kart numarası 6: Karo 7
Kart numarası 48: Sinek 10
Kart numarası 11: Karo Kızı
Kart numarası 24: Kupa Kızı



Program, dört takım için bir dizi **suits** (satır 4) ve bir takımda 13 kart için bir dizi **rank**s tanımlar (satırlar 5–6). Bu dizilerdeki her bir eleman bir string'dir.

Program, **deck** değerini satır 9–10'da 0 ile 51 arasında başlatır. **deck** değeri 0, Karo As kartını temsil eder, 1, Karo 2 kartını temsil eder, 13, Kupa As kartını temsil eder ve 14, Kupa 2 kartını temsil eder.

Satır 13–19, deste karıştırır. Bir deste karıştırıldıktan sonra, **deck[i]** rastgele bir değer içerir. **deck[i] / 13, 0,1,2** veya 3 olur, bu da takımı belirler (satır 23). **deck[i]% 13, 0** ile 12 arasında bir değerdir, bu da rütbeyi belirler (satır 24). Eğer **suits** dizisi tanımlı değilse, takımı aşağıdaki gibi uzun bir çoklu **if-else** ifadesi kullanarak belirleme gerekir:

```
eğer(deck[i] / 13==0)
    System.out.print("takım Karo");
else if(deck[i] / 13==1)
    System.out.print("takım Kupa");
else if(deck[i] / 13==2)
    System.out.print("takım Sinek");
else
    System.out.print("takım elbise Kulüplerdir");
```

Bir dizi içinde oluşturulan **suits = {"Spades", "Hearts", "Diamonds", "Clubs"}**, **suits[deck / 13, deck]** için takımı verir. Dizilerin kullanımı, bu programın çözümünün büyük ölçüde basitleştirir.

6.5 Dizileri Kopyalama



Bir dizinin içeriğini başka bir diziyeye kopyalamak için, dizinin bireysel

elemanlarını diğer diziyeye kopyalamanız gerekir.

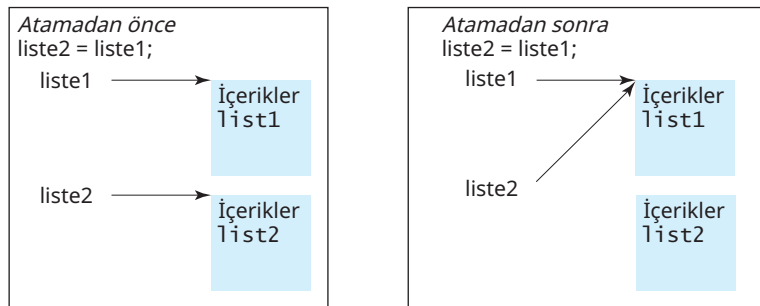
Sıklıkla, bir programda bir diziyi veya bir dizinin bir kısmını çoğaltmanız gerekir. Bu tür durumlarda, atama ifadesini (=) kullanmayı deneyebilirsiniz:

```
list2 = list1;
```

Ancak, bu ifade **list1** tarafından referans edilen dizinin içeriğini **list2**'ye kopyalamaz, bunun yerine yalnızca **list1**'den **list2**'ye referans değerini kopyalar. Bu ifadeden sonra, **list1** ve **list2** aynı diziyi referans alır; bu, Şekil 6.5'te gösterilmiştir. Daha önce **list2** tarafından referans alınan dizi artık referans alınmamaktadır; çöp haline gelir ve Java Sana I Makinesi tarafından otomatik olarak toplanır (bu işleme *çöp toplama* denir).

referansı kopyala

çöp toplama



ŞEKİL 6.5 Atama ifadesinden önce, liste1 ve liste2 ayrı bellek konumlarına işaret eder. Atamadan sonra, liste1 dizisinin referansı liste2'ye geçer.

Java'da, ilkel veri türü değişkenlerini kopyalamak için atama ifadelerini kullanabilirsiniz, ancak dizileri kopyalayamazsınız. Bir dizi değişkenini başka bir dizi değişkenine atamak, aslında bir referansı diğerine kopyalar ve her iki değişkenin de aynı bellek konumuna işaret etmesini sağlar.

Dizileri kopyalamanın üç yolu vardır:

- Bireysel elemanları birer birer kopyalamak için bir döngü kullanın.
- System sınıfındaki statik **arraycopy** yöntemini kullanın.
- Dizileri kopyalamak için **clone** yöntemini kullanın; bu, 15. Bölüm'de, Soyut Sınıflar ve Arayüzler'de tanıtılacaktır.

Kaynak diziden hedef dizideki karşılık gelen elemana her bir elemanı kopyalamak için bir döngü yazabilirsiniz. Aşağıdaki kod, örneğin, **sourceArray**'yi **targetArray**'ye kopyalar.

```
intt[] sourceArray = {t2t,t3t,t1t,t5t,t10t};nintt[] t
argetArray = tnew intt[sourceArray.length];nfort(tintti = t0t; i
< sourceArray.length; i++) {ntargetArray[i] = sourceArray[i]
;
}
```

Başka bir yaklaşım, döngü kullanmak yerine dizileri kopyalamak için **java.lang.System** sınıfındaki **arraycopy** yöntemini kullanmaktır. **arraycopy** için sözdizimi:

```
arraycopy(sourceArray, src_pos, targetArray, tar_pos, uzunluk);
```

Parametreler **src_pos** ve **tar_pos**, sırasıyla **sourceArray** ve **targetArray** içindeki başlangıç konumlarını belirtir. **sourceArray**'den **targetArray**'ye kopyalanan eleman sayısı **length** ile belirtilir. Örneğin, döngüyü aşağıdaki ifade ile yeniden yazabilirsiniz:

```
System.arraycopy(sourceArray,t 0t, targetArray, t 0t, sourceArray.length);
```

arraycopy yöntemi hedef dizi için bellek alanı tahsis etmez. Hedef dizi önceden bellek alanı tahsis edilerek oluşturulmuş olmalıdır. Kopyalama işlemi gerçekleştiğinde, **targetArray** ve **sourceArray** aynı içeriğe sahip ancak bağımsız bellek konumlarına sahiptir.



Not

arraycopy yöntemi Java adlandırma kuralını ihlal eder. Geleneksel olarak, bu yöntemin adı **arrayCopy** (yani, büyük C ile) olmalıdır.

6.12 **arraycopy()** yöntemini aşağıdaki diziyi hedef dizi t'ye kopyalamak için kullanın:

```
intt[] kaynak = {t3t,t4t,t5t};
```

6.13 Bir dizi oluşturulduğunda, boyutu değiştirilemez. Aşağıdaki kod diziyi yeniden boyutlandırıyor mu?

```
intt[]listem;
melissa = tanu int[t10t];
// Bir süre sonra myList'e yeni bir dizi atamak istersiniz
myList = new int[20];
```

Point Control
MyProgrammingLab™

6.6 Dizileri Yöntemlere Geçirme

Diziyi bir yönteme geçirirken, dizinin referansı yönteme iletilir.

Aynı şekilde, temel tür değerlerini yöntemlere geçirebildiğiniz gibi, dizileri de yöntemlere geçirebilirsiniz. Örneğin, aşağıdaki yöntem bir **int** dizisindeki elemanları görüntüler:

```
public static voidtprintArray(tintt[] array) {nfort(tintti = t
0t; i < array.length; i++) {
```



Ana
Nokta

238 Bölüm 6 Tek Boyutlu Diziler

```
System.out.print(dizi[i] + " " + t);n}

}
```

Bir dizi geçirerek çağırabilirsiniz. Örneğin, aşağıdaki ifade `printArray` yöntemini 3,1,2,6,4 ve 2'yi görüntülemek için çağırır.

```
printArray(tnew intt[] {t3t,t1t,t2t,t6t,t4t,t2t});
```



Not

Önceki ifade, aşağıdaki sözdizimini kullanarak bir dizi oluşturur:

```
newtelementType[]{value0, value1, ..., valuek};
```

anonim dizi

Array için açık bir referans değişkeni yoktur. Böyle bir diziye *anonim array* denir.

değer ile geçiş

Java, bir yömeme argümanları geçirmek için *pass-by-value* kullanır. Temel veri türlerinin değ işken değerlerini geçmek ile dizileri geçmek arasında önemli farklar vardır.

- Bir temel tür argümanı için, argümanın değeri geçirilir.
- Bir dizi türü argümanı için, argümanın değeri bir diziye referanstır; bu referans değeri yömeme geçirilir. Anlam olarak, en iyi şekilde *pass-by-sharing* olarak tanımlanabilir, yani yöntemdeki dizi, geçirilen dizi ile aynıdır. Bu nedenle, yöntemde diziye değ iştirirseniz, değ işikliğı yöntemin dışında göreceksiniz.

paylaşarak geçiş

Aşağıdaki kodu örnek olarak alın:

```
genel sınıfTest {
    public static voidmain(String[] args) {
        intx = 1; // x bir int değeri temsil eder
        int[] y = new int[10]; // y bir int değeri dizisini temsil eder

        m(x, y); // m'yi x ve y argümanlarıyla çağır

        System.out.println("x değeri " + x);
        System.out.println("y[0] değeri " + y[0]);
    }

    genel statik boşluk m(renk tonu numarası, renk tonu[] sayılar) {
        number = 1001; // number'a yeni bir değeri ata
        numbers[0] = 5555; // numbers[0]'a yeni bir değeri ata
    }
}
```



x 1'dir
y[0] 5555'tir

Çağrıldığında `m`, `x1` olarak kalırken, `y[0]5555` oluyor diye merak edebilirsiniz. Bu, `y` ve `numbers`, bağımsız değ işkenler olmalarına rağmen, aynı diziye referans alıyor olmalarındandır; bu, Şekil 6.6'da gösterilmiştir. `m(x, y)` çağrıldığında, `x` veya `y` değeri `number` ve `numbers`'ye iletilir. `y` dizinin referans değeri içerdiğinden, `numbers` artık aynı dizinin aynı referans değeri içerir.

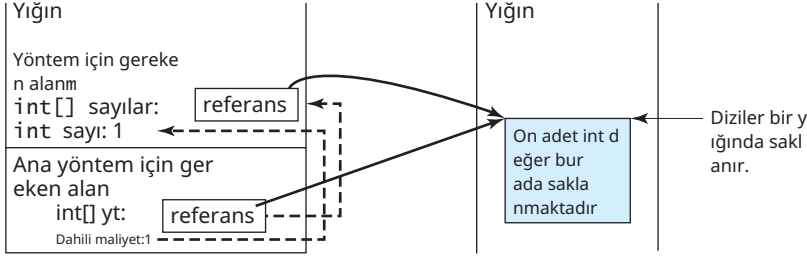


FIGURE 6.6 Temel tür değeri `x` içinde `number`'a, ve `y` içindeki referans değeri `numbers`'a aktarılır.



Not

Diziler Java'da nesnelerdir (nesneler Bölüm 8'de tanıtılmaktadır). JVM, nesneleri dinamik bellek tahsisi için kullanılan *heap* adı verilen bir bellek alanında saklar.

yığın

Liste 6.3, bir ilkel veri tipi değeri ile bir dizi referans değişkeninin bir metoda geçirilmesi arasındaki farkı gösteren başka bir programı vermektedir.

Program, bir dizideki elemanları değiştirmek için iki metoda sahiptir. İlk metod, `swap` olarak adlandırılan, iki `int` argümanı değiştirmeyi başaramaz. İkinci metod, `swapFirstTwoInArray` olarak adlandırılan, dizi argümanındaki ilk iki elemanı başarıyla değiştirir.

LİSTE 6.3 TestPassArray.java

```

1 genel classtestPassArray {
2     /** Ana yöntem */
3     public static void main(String[] args) {
4         int[] a = {t1t,t2t};
5
6         // Elemanları swap metodu kullanarak değiştir
7         System.out.println("swap çağrılmadan önce");
8         System.out.println("dizi {" + a[0] + ", " + a[1] + "}");
9         takas(a[t0t], a[t1t]);
10        System.out.println("swap çağrıldıktan sonra");
11        System.out.println("dizi {" + a[0] + ", " + a[1] + "}");
12
13        // Elemanları swapFirstTwoInArray metodu kullanarak değiştir
14        System.out.println("swapFirstTwoInArray çağrılmadan önce");
15        System.out.println("dizi {" + a[0] + ", " + a[1] + "}");
16        swapFirstTwoInArray(a);
17        System.out.println("swapFirstTwoInArray çağrıldıktan sonra");
18        System.out.println("dizi {" + a[0] + ", " + a[1] + "}");
19    }
20
21    /** İki değişkeni değiştir */
22    genel statik boşluk takas(tinttn1,tinttn2) {
23        int temp = n1;
24        n1 = n2;n25 n2 =
25        sıcaklık;n26 }
26
27
28    /** Dizideki ilk iki öğeyi değiştir */
29    genel statik boşluk swapFirstTwoInArray(tinttt[] dizisi) {
30        inttemp = dizi[t0t];
31        dizi[t0t] = dizi[t1t];n32 dizi[t1t] =
32        sıcaklık;n33 }
33
34 }
```

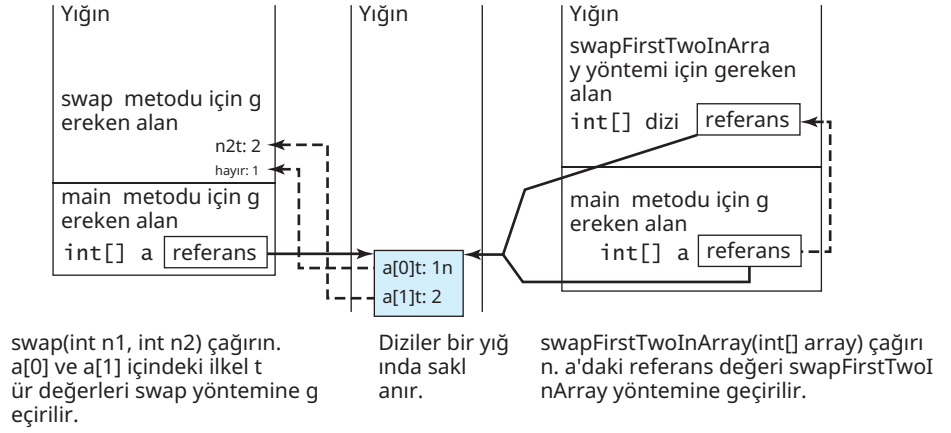
yanlış değişim

dizi elemanlarını değiştirme



Değiştirme çağrılmadan önce
dizi {1, 2}
Değiştirme çağrıldıktan sonra
dizi {1, 2}
swapFirstTwoInArray çağrılmadan önce
dizi {1, 2}
swapFirstTwoInArray çağrıldıktan sonra
dizi {2, 1}

Şekil 6.7'de gösterildiği gibi, iki eleman **swap** yöntemi kullanılarak değiştirilmez. Ancak, **swapFirstTwoInArray** yöntemi kullanılarak değiştirilir. **swap** yöntemindeki parametreler ilkel tür olduğundan, **a[0]** ve **a[1]** değerleri **n1** ve **n2**'ye metodun içinde **swap(a[0], a[1])** çağrıldığına geçilir. **n1** ve **n2** için bellek konumları **a[0]** ve **a[1]** için olanlardan bağımsızdır. Dizinin içeriği bu çağrıdan etkilenmez.



FGÖRÜNTÜ 6.7 Bir diziyi bir metoda geçirirken, dizinin referansı metoda geçirilir.

swapFirstTwoInArray yöntemindeki parametre bir dizidir. Şekil 6.7'de gösterildiği gibi, dizinin referansı metoda geçer. Böylece **a** (metodun dışında) ve **array** (metodun içinde) her ikisi de aynı diziyi aynı bellek konumunda referans alır. Bu nedenle, metodun içinde **swapFirstTwoInArray** **dizi[0]** ile **dizi[1]**'yi değiştirmek, metodun dışında **a[0]** ile **a[1]**'yi değiştirmekle aynıdır.

6.7 Bir Metottan Dizi Döndürme



Ana
Nokta

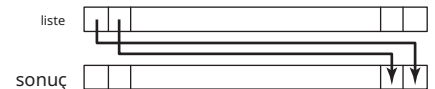
Bir metod bir dizi döndürdüğünde, dizinin referansı döndürülür.

Bir yöntemi çağırırken dizileri geçirebilirsiniz. Bir yöntem ayrıca bir dizi de döndürebilir. Örneğin, aşağıdaki yöntem başka bir dizinin tersini döndüren bir dizi döndürür.

dizi oluştur

```
1 public static int[] ters(int[] liste) {
2     int[] sonuç = new int[liste.length];
3
4     for (int i = 0, j = liste.length - 1;
5         i < liste.length; i++, j--) {
6         sonuç[j] = liste[i];
7     }
8
9     return sonuç;
10 }
```

dizi döndür



2. Satır, yeni bir dizi **result** oluşturur. 4-7. Satırlar, **list** dizisinden

result dizisine elemanları kopyalar. 9. Satır diziyi döndürür. Örneğin, aşağıdaki ifade, 6, 5, 4, 3, 2, 1. elemanları ile yeni bir dizi **list2** döndürür.

```
intt[] liste1 = {t1t,t2t,t3t,t4t,t5t,t6t};nintt
[] liste2 = ters(liste1);
```

6.8 Vaka Çalışması: Her Harfin Tekrar Sayısını Sayma

Bu bölüm, bir karakter dizisindeki her harfin tekrar sayısını sayan bir program sunmaktadır.



6.4 Listesinde verilen program şunları yapar:

- 100 küçük harfi rastgele üretir ve bunları, Şekil 6.8a'da gösterildiği gibi, bir karakter dizisine atar. Rastgele bir harf elde etmek için, 5.10 Listesindeki **RandomCharacter** sınıfındaki **getRandomLower-CaseLetter()** yöntemini kullanabilirsiniz.
- Dizideki her harfin tekrar sayısını sayın. Bunu yapmak için, her biri bir harfin tekrar sayısını sayan 26 int değerinden oluşan bir dizi, diyelim ki **counts** oluşturun, Şekil 6.8b'de gösterildiği gibi. Yani, **counts[0]** a'nın sayısını, **counts[1]** b'nin sayısını sayar ve bu şekilde devam eder.

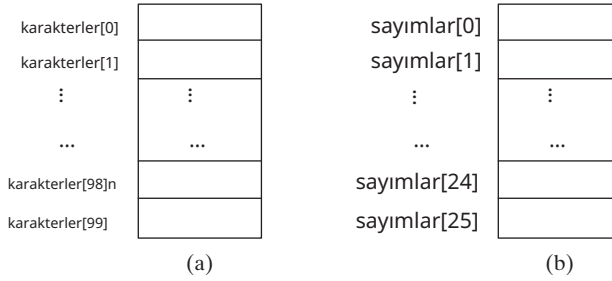


FIGURE 6.8 Karakterler dizisi 100 karakteri saklar ve sayılar dizisi 26 sayıyı saklar, hangi harfin kaç kez tekrarlandığını sayar.

LİSTELER6.4CountLettersInArray.java

```
1 genel classtCountLettersInArray {
2     /** Ana yöntem */
3     public static voidtmain(String[] args) {
4         // Bir dizi tanımlayın ve oluşturun
5         grafik[] karakterler = createArray();
6     }
7
8     // Diziyi görüntüleyin
9     System.out.println("Küçük harfler şunlardır:");
10    displayArray(karakterler);
11
12    // Her harfin tekrar sayısını sayın
13    int[] counts = countLetters(karakterler);
14
15    // Sayıları görüntüleyin
16    System.out.println();
17    System.out.println("Her harfin tekrar sayıları şunlardır:");
18    displayCounts(sayımlar);
19 }
20 /** Karakterlerden oluşan bir dizi oluşturun */
```

dizi oluştur

dizi geçir

dizi döndür

dizi geçir

242 Bölüm 6 Tek Boyutlu Diziler

```
21 genel statik grafik[] createArray() {
22     // Karakterlerden oluşan bir dizi tanımla ve oluştur
23     grafik[] grafik = yeni grafik[t100t];
24
25     // Küçük harfleri rastgele oluştur ve ata
26     // onları diziye
27     fort(tintti = t0t; i < karakter.uzunluk; i++)
28     karakter[i] = RandomCharacter.getRandomLowerCaseLetter();
29
30     // Dizi döndür
31     dönüş karakterleri;
32 }
33
34 /** Karakter dizisini göster */
35 genel statik voidtdisplayArray(tchart[] karakterler) {
36     // Dizideki karakterleri her satırda 20 tane göster
37     fort(tintti = t0t; i < karakter.uzunluk; i++) {
38         ift((i + t1t) % t20t==t0t)
39             System.out.println(chars[i]);
40         aksi takdirde
41     System.out.print(chars[i] + t" ";n42 }
42
43 }
44
45 /** Her harfin tekrar sayısını say */
46 genel statik int[] countLetters(tchart[] karakterler) {
47     // 26 int'lik bir dizi tanımla ve oluştur
48     int[] sayıları = tnew int[t26t];
49
50     // Dizideki her küçük harfi say
51     fort(tintti = t0t; i < karakter.uzunluk; i++)
52     sayım[karakter[i] - t'a't]++;
53
54     dönüş sayıları;n5
55 }
56
57 /** Sayıları göster */
58 genel statik voidtdisplayCounts(tintt[] sayıları) {
59     fort(tintti = t0t; i < sayım.uzunluk; i++) {
60         eğer((i + 1) % 10==0)
61     System.out.println(counts[i] + t" "+ (tchart)(i + t'a't));
62     aksi takdirde
63     System.out.print(counts[i] + t" "+ (tchart)(i + t'a't) + t" ";n64 }
64
65 }n66
66 }
```

sayımı artır



Küçük harfler şunlardır:

e y l s r i b k j v j h a b z n w b t v
s c c k r d w a m p w v u n q a m p l o
a z g d e g f i n d x m z o u l o z j v
h w i w n t g x w c d o t x h y v z y z
q e a m f w p g u q t r e n n w f c r f

Her harfin tekrar sayıları:

5 a 3 b 4 c 4 d 4 e 4 f 4 g 3 h 3 i 3 j n 2 k 3 l 4 m 6 n
4 o 3 p 3 q 4 r 2 s 4 t n 3 u 5 v 8 w 3 x 3 y 6 z

createArray metodu (satırlar 21–32) 100 rastgele küçük harften oluşan bir dizi oluşturur. 5. satır metodu çağırır ve diziyi **chars** değişkenine atar. Aşağıdaki gibi kodu yeniden yazarsanız ne yanlış olurdu?

```
chart[] chars = tnew chart[t100t];nch
ars = createArray();
```

İki dizi oluşturmuş olursunuz. İlk satır, **newchar[100]** kullanarak bir dizi oluşturur. İkinci satır, **createArray()** çağırarak bir dizi oluşturur ve dizinin referansını **chars** değişkenine atar. İlk satırda oluşturulan dizi artık referans edilmediği için çöp olacaktır ve daha önce belirtildiği gibi Java otomatik olarak arka planda çöp toplar. Programınız derlenecek ve doğru bir şekilde çalışacak, ancak gereksiz yere bir dizi oluşturacaktır.

getRandomLowerCaseLetter() çağırısı (satır 28) rastgele bir küçük harf döndürür. Bu yöntem, Liste 5.10'daki **RandomCharacter** sınıfında tanımlanmıştır.

countLetters metodu (satırlar 46–55) her biri bir harfin tekrar sayısını saklayan **26 int** değerinden oluşan bir dizi döndürür. Metod, dizideki her harfi işler ve e sayısını bir artırır. Her harfin tekrar sayısını saymak için bir kaba kuvvet yaklaşımı şu şekilde olabilir:

```
for (int i = 0; i < chars.length; i++)
    if (chars[i] == 'a')
        counts[0]++;
    else if (chars[i] == 'b')
        counts[1]++;
    ...
```

Ancak daha iyi bir çözüm 51-52. satırlarda verilmiştir.

```
for (int i = 0; i < chars.length; i++)
    counts[chars[i] - 'a']++;
```

Eğer harf (**chars[i]**) **a** ise, karşılık gelen sayı **counts['a' - 'a']** (yani, **counts[0]**) olur. Eğer harf **b** ise, karşılık gelen sayı **counts['b' - 'a']** (yani, **counts[1]**) olur, çünkü **b**'nin Unicode'u **a**'ninkinden bir fazladır. Eğer harf **z** ise, karşılık gelen sayı **counts['z' - 'a']** (yani, **counts[25]**) olur, çünkü **z**'nin Unicode'u **a**'ninkinden 25 fazladır.

Şekil 6.9, **createArray**'i çalıştırırken ve çalıştırdıktan sonra çağrı yığını ve yığın gösterir. Diğer yöntemler için çağrı yığını ve yığın belleği göstermek üzere Kontrol Noktası Soru 6.16'ya bakın.

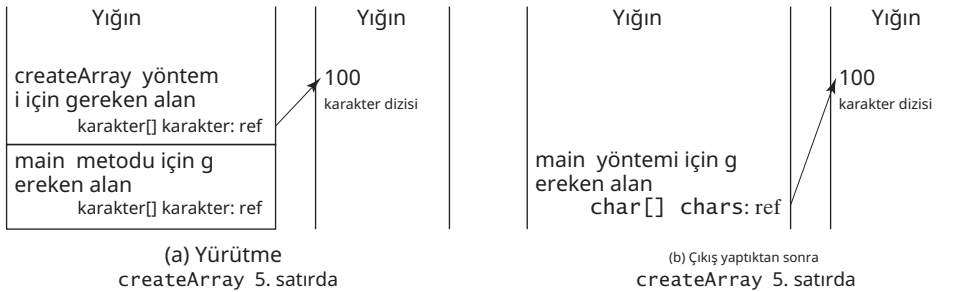


FIGURE 6.9(a) **createArray** yöntemini çalıştırdığınızda 100 karakterden oluşan bir dizi oluşturulur. (b) Bu dizi döndürülür ve **main** yöntemindeki **chars** değişkenine atanır.

244 Bölüm 6 Tek Boyutlu Diziler



MyProgrammingLab™

- 6.14** Doğru mu yanlış mı? Bir dizi bir yönteme geçirildiğinde, yeni bir dizi oluşturulur ve yönteme geçirilir.
- 6.15** Aşağıdaki iki programın çıktısını gösterin:

```
genel sınıfTest {
    public static void main(String[] args) {
        int n = Integer.parseInt(args[0]);
        int[] sayılar = new int[n];
        for (int i = 0; i < n; i++) {
            sayılar[i] = i;
        }

        System.out.println("sayı " + sayılar[0] + " ve sayılar[" + sayılar.length - 1 + "] = " + sayılar[sayılar.length - 1]);
    }
}
```

(a)

```
genel sınıfTest {
    public static void main(String[] args) {
        int[] liste = {1, 2, 3, 4, 5};
        ters(liste);
        for (int i = 0; i < liste.length; i++) {
            System.out.print(liste[i] + " ");
        }
    }

    public static void ters(int[] list) {
        int[] newList = new int[list.length];
        for (int i = 0; i < list.length; i++) {
            newList[i] = list[list.length - 1 - i];
        }
    }
}
```

(b)

- 6.16** Diziler yürütme sırasında nerede saklanır? displayArray, countLetters, displayCounts yöntemlerini çalıştırırken ve sonrasında yığın ve yığın içeriğini gösterin.



6.9 Değişken Uzunlukta Argüman Listeleri

Aynı türde değişken sayıda argüman bir yönteme geçirilebilir ve dizi olarak işlenebilir.

Bir metoda aynı türde değişken sayıda argüman geçirebilirsiniz. Metodun parametresi şu şekilde tanımlanır:

türAdı... parametreAdı

Metod tanımında, türü üç nokta ile (...) belirtirsiniz. Bir metodda yalnızca bir değişken uzunlukta parametre tanımlanabilir ve bu parametre son parametre olmalıdır.

Herhangi bir normal parametre bu parametreden önce gelmelidir.

Java, değişken uzunluktaki bir parametreyi bir dizi olarak ele alır. Bir dizi veya değişken sayıda argümanı değişken uzunluktaki bir parametreye geçirebilirsiniz. Değişken sayıda argümanla bir metod çağırıldığında, Java bir dizi oluşturur ve argümanları buna geçirir. Liste 6.5, belirtilmemiş sayıda değerin bulunduğu bir listede maksimum değeri yazdıran bir metodu içermektedir.

Listing 6.5tVarArgsDemo.java

değişken uzunlukta argüman listesi geçir
bir dizi argümanı geçir

değişken uzunlukta argüman
parametre

```
1 genel sınıfVarArgsDemo {
2     public static void main(String[] args) {
3         intMax(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
4         printMax(new double[]{1, 2, 3, 4, 5, 6, 7, 8, 9, 10});
5     }
6
7     genel statik boşluk printMax(double... numbers) {
8         if (numbers.length == 0) {
9             System.out.println("Hiçbir argüman geçirilmedi");
10        }
11    }
```

```

10         geri dönüş;
11     }
12
13     double sonuç = numbers[0];
14
15     for(tintti = t1t; i < sayılar.uzunluk; i++)
16         ift(sayılar[i] > sonuç)
17             sonuç = sayılar[i];
18
19     System.out.println("Maksimum değer: "t+ sonuç);n20 }

```

yirmi bir }

3. satır, **printMax** yöntemini, diziye geçirilen değişken uzunlukta bir argüman listesi ile çağırır **numbers**. Hiçbir argüman geçirilmezse, dizinin uzunluğu **0**'dır (8. satır).

4. satır, **printMax** yöntemini bir dizi ile çağırır.

6.17 Aşağıdaki yöntem başlığında ne yanlış?

```

public static void print(String... stringler, tdoublet... sayılar) {
    public static void print(tdoublet... sayılar, String adı)
    genel statik çift... print(tdoubletd1, tdoubletd2)
}

```



6.18 6.5'teki **printMax** yöntemini aşağıdaki ifadeleri kullanarak çağırabilir misiniz?

```

printMax(t1t, t2t, t2t, t1t, t4t); npr
intMax(tnew doublet[] {t1t, t2t, t3t}); nprint
Max(tnew intt[] {t1t, t2t, t3t});

```

6.10 Dizi Arama

Eğer bir dizi sıralıysa, ikili arama, dizide bir elemanı bulmak için doğrusal arama dan daha etkilidir.

Arama, bir dizide belirli bir elemanı bulma sürecidir—örneğin, belirli bir puanın bir puanlar listesinde yer alıp almadığını keşfetmek. Arama, bilgisayar programlamasında yaygın bir görevdir. Birçok algoritma ve veri yapısı aramaya adanmıştır. Bu bölüm, yaygın olarak kullanılan iki yaklaşımı, *doğrusal arama* ve *ikili arama* tartışmaktadır.



doğrusal arama
ikili arama

6.10.1 Doğrusal Arama Yaklaşımı

Doğrusal arama yaklaşımı, anahtar öğeyi **anahtar** dizideki her öğe ile sırasıyla karşılaştırır. Anahtar, dizideki bir öğe ile eşleşene kadar veya dizide eşleşme bulunmadan tükenene kadar devam eder. Eşleşme sağlanırsa, doğrusal arama anahtara karşılık gelen dizideki öğenin indeksini döndürür. Eşleşme bulunamazsa, arama **-1** döndürür. Liste 6.6'daki **LinearSearch** yöntemi çözümü verir.



doğrusal arama animasyonu
Eşlik Eden Web Sitesi'nde

LISTE 6.6 LinearSearch.java

```

1  genel sınıf Doğrusal Arama {
2      /** Listede bir anahtar bulma yöntemi */
3      public static intt linearSearch(tintt[] liste, renk tonu anahtarı) {
4          for (tintt i = t0t; i < liste.uzunluk; i++) {
5              ift (anahtar == liste[i])
6                  i'yi döndür;
7          }
8          dönüş-t1t;
9      }n10
}

```

[0][1][2] ...
liste

--	--	--	--	--	--

anahtar list[i] ile karşılaştır i = 0, 1, ...

Bu yöntemi daha iyi anlamak için, aşağıdaki ifadelerle izleyin:

```
1  int[] liste = {t1t,t4t,t4t,t2t,t5t,t-3t,t6t,t2t};
2  intti = doğrusalArama(liste, t4t);        // 1 döner
3  intj = linearSearch(list, -4); // -1 döner
4  intk = linearSearch(list, -3); // 5 döndürür
```

Doğrusal arama yöntemi, anahtarı dizideki her bir elemanla karşılaştırır. Elemanlar herhangi bir sırad a olabilir. Ortalama olarak, algoritmanın anahtarı bulmadan önce dizideki elemanların yarısını incele mesi gerekecektir, eğer anahtar mevcutsa. Doğrusal aramanın yürütme süresi, dizi elemanlarının say ısı arttıkça doğrusal olarak artar, bu nedenle büyük bir dizi için doğrusal arama verimsizdir.

6.10.2 İkili Arama Yaklaşımı

İkili arama, bir dizi değer için diğer yaygın arama yaklaşımıdır. İkili aramanın çalışabi lmesi için dizideki elemanların zaten sıralı olması gerekir. Dizinin artan sırada olduđu nu varsayalım. İkili arama önce anahtarı dizinin ortasındaki elemanla karşılaştırır. Aşağıdaki üç durumu düşünün:

- Eğer anahtar ortadaki elemandan küçükse, anahtarı yalnızca dizinin ilk yarısında aram aya devam etmeniz gerekir.
- Eğer anahtar ortadaki elemanla eşitse, arama bir eşleşme ile sona erer.
- Eğer anahtar ortadaki elemandan büyükse, anahtarı yalnızca dizinin ikinci yarısınd a aramaya devam etmeniz gerekir.

Açıkça, ikili arama yöntemi her karşılaştırmadan sonra dizinin en az yarısını ortadan kaldırır. Bazen yarıdan fazla elemanı ortadan kaldırabilirsiniz ve bazen yarıdan bir fazlasını ortadan kaldırabilirsiniz. Diyelim ki dizinin n elemanı var. Kolaylık olması açısından, n bir 2 kuvveti olsun. İlk karşılaştırmadan sonra, $n/2$ eleman kalır; ikinci karşılaştırmadan sonra, $(n/2)/2$ eleman kalır. k . karşılaştırmadan sonra, $n/2^k$ eleman kalır.

$= \log_2 n$, dizide yalnızca bir eleman kalır ve yalnızca bir karşılaştırma daha yapmanız gerekir. Bu nedenle, ikili arama yaklaşımını kullanırken en kötü durumda, sıralı dizide bir elemanı b ulmak için $\log_2 n + 1$ karşılaştırma gerekir. $1024(2^{10})$ elemanlı bir liste için en kötü durumda, i kili arama yalnızca 11 karşılaştırma gerektirirken, doğrusal arama en kötü durumda 1023k arşılaştırma gerektirir.

Aranan dizinin kısmı her karşılaştırmadan sonra yarıya iner. **low** ve **high** sırasıyla, şu a nda aranan dizinin ilk ve son indeksini belirtir. Başlangıçta, **low 0** ve **high list.length -1**. **mid** orta elemanın indeksini belirtir, bu nedenle **mid (low + high)/2** Şekil 6.10, iki li arama kullanarak listede 11 anahtarını nasıl bulacağınızı gösterir {2,4,7,10,11,45,50,59 ,60,66,69,70,79}.

Artık ikili aramanın nasıl çalıştığını biliyorsunuz. Bir sonraki görev, bunu Java'da uygulamaktır. Tam bir uygulama vermek için acele etmeyin. Bunu adım adım, kademeli olarak uygulayın. Aramanın ilk yinelemesi ile başlayabilirsiniz, Şekil 6.11a'da gösterildiği gibi. Bu, anahtarı, indeksinin 0 ve yüksek indeksinin

list.length - 1 olduğu listedeki orta elemanla karşılaştırır. Eğer **key < list[mid]** ise, yüksek indeksini **mid - 1** olarak ayarlayın; eğer **key = list[mid]** ise, bir eşleşme bulunur ve **mid** döndürülür; eğer **key > list[mid]** ise, düşük indeksini **mid + 1** olarak ayarlayın.

Sonraki adım, bir döngü ekleyerek aramayı tekrar tekrar gerçekleştiren yöntemi uygulamayı düşündürmektedir, Şekil 6.11b'de gösterildiği gibi. Arama, anahtar bulunduğu veya anahtar bulunmadığında sona erer **alçak > yükseklik**.

Anahtar bulunmadığında, **düşük**, bir anahtarın listenin sırasını korumak için eklenebileceği ekleme noktasıdır. Ekleme noktasını döndürmek, **-1** döndürmekten daha faydalıdır. Yöntem, anahtarın listede olmadığını belirtmek için negatif bir değer döndürmelidir. Basitçe **-low** dönd ürebilir mi? Hayır. Eğer anahtar **list[0]**'den küçükse, **düşük 0** olur. **-0, 0**'dir. Bu,



ikili arama animasyonu
Eşlik Eden Web Sitesi

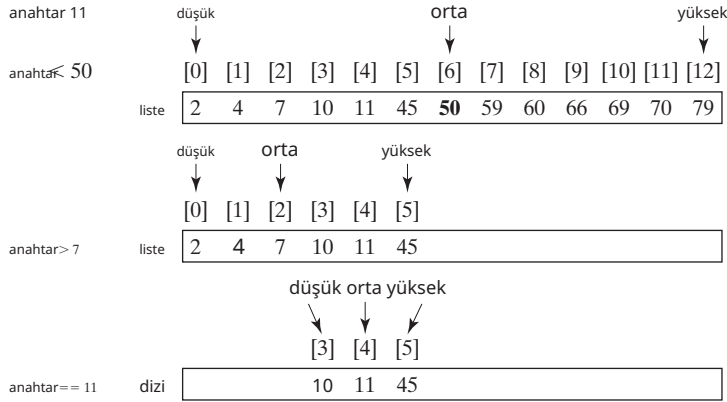


FIG.6.10 İkili arama, her karşılaştırmadan sonra listenin yarısını daha fazla değerlendirme den çıkarır.

```
public static int ikiliArama(
    int[] liste, int anahtar) {
    int düşük = 0;
    int yüksek = liste.length - 1;

    int mid = (düşük + yüksek) / 2;
    eğer(anahtar < liste[mid])
        yüksek = mid - 1;
    aksi takdirde eğer(anahtar == liste[mid])
        geri dönüş ortası;
    aksi
        düşük = mid + 1;
}
```

(a) Sürüm 1

```
public static int ikiliArama(
    int[] liste, int anahtar) {
    int düşük = 0;
    int yüksek = liste.length - 1;

    while(yüksek >= düşük) {
        int mid = (düşük + yüksek) / 2;
        if(anahtar < liste[mid])
            yüksek = mid - 1;
        else if(anahtar == liste[mid])
            return mid;
        aksi takdirde
            düşük = mid + 1;
    }
    dönüş -1; // Bulunamadı
}
```

(b) Sürüm 2

FIGURE 6.11 İkili arama kademeli olarak uygulanmaktadır.

anahtarın `liste[0]` ile eşleştiğini belirtir. İyi bir seçim, anahtar listede yoksa yöntemin `-low - 1` döndürmesine izin vermektir. `-low - 1` döndürmek, anahtarın listede olmadığını değil,

bunun yanı sıra anahtarın nereye yerleştirileceğini de gösterir.

Tam program, Liste 6.7'de verilmiştir.

LİSTELER6.7 BinarySearch.java

```
1 genel sınıf BinarySearch {
2     /** Anahtarın listede bulunması için ikili arama kullanın */
3     public static int binarySearch(int[] list, int key) {
4         int düşük = 0;
5         int yüksek = liste.length - 1;
6
7         while(yüksek >= düşük) {
8             int mid = (düşük + yüksek) / 2;
9             if(anahtar < liste[mid])
10                yüksek = mid - 1;
11             else if(anahtar == liste[mid])
```

ilk yarı

```

12         return mid;
13     else
14         low = mid + 1;
15     }
16
17     return -low - 1; // Now high < low, key not found
18 }
19 }

```

ikinci yarı

İkili arama, arama anahtarının listede bulunması durumunda indeksini döndürür (satır 12). Aksi takdirde, `-low - 1` döndürür (satır 17).

Satır 7'de `(high >= low)` yerine `(high > low)` koyarsak ne olur? Arama, olası bir eşleşen öğeyi kaçırmaz. Sadece bir öğe içeren bir listeyi düşünün. Arama, öğeyi kaçırmaz.

Liste içinde tekrar eden elemanlar varsa yöntem hala çalışır mı? Evet, elemanlar artan sırada sıralandığı sürece. Yöntem, eleman listede varsa eşleşen elemanlardan birinin indeksini döndürür.

Bu yöntemi daha iyi anlamak için, aşağıdaki ifadelerle izleyin ve yöntem döndüğünde **düşük** ve **yüksek** değerlerini belirleyin.

```

int[] liste = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};
inti = BinarySearch.binarySearch(liste, 2); // 0 döndürür
intj = BinarySearch.binarySearch(liste, 11); // 4 döndürür
intk = BinarySearch.binarySearch(liste, 12); // -6 döndürür
intl = BinarySearch.binarySearch(liste, 1); // -1 döndürür
intm = BinarySearch.binarySearch(liste, 3); // -2 döndürür

```

İşte yöntemin çıktığı zaman **düşük** ve **yüksek** değerlerini listeleyen tablo ve yöntem mi çağdırmaktan dönen değer.

Yöntem	Düşük	Yüksek	Geri Dönen Değer
ikiliArama(liste, 2)	0	1	0
nbinarySearch(liste, 11)	3	5	4
nbinarySearch(liste, 12)	5	4	-6
nbinaryArama(liste, 1)	0	-1	-1
nbinaryArama(liste, 3)	1	0	-2

ikili arama avantajları



Not

Doğrusal arama, küçük bir dizide veya sıralanmamış bir dizide bir öğeyi bulmak için yararlıdır, ancak büyük diziler için verimsizdir. İkili arama daha etkilidir, ancak dizinin önceden sıralanmış olması gerekir.



6.11 Dizileri Sıralama

Dizideki elemanları sıralamak için birçok strateji vardır. Seçim sıralaması ve ekleme sıralaması, iki yaygın yaklaşımdır.

Sıralama, arama gibi, bilgisayar programlamasında yaygın bir görevdir. Sıralama için birçok farklı algoritma geliştirilmiştir. Bu bölüm, iki basit, sezgisel sıralama algoritmasını tanıtmaktadır: seçim sıralaması ve ekleme sıralaması.

seçim sıralaması

ekleme sıralaması

6.11.1 Seçim Sıralaması

Bir listeyi artan sırada sıralamak istediğinizi varsayalım. Seçim sıralaması, listede en küçük sayıyı bulur ve bunu ilk elemanla değiştirir. Ardından, kalan en küçük sayıyı bulur ve bunu ikinci elemanla değiştirir ve bu şekilde devam eder, ta ki yalnızca bir sayı kalana kadar. Şekil 6.12, seçim sıralaması kullanılarak {2,9,5,4,8,1,6} listesini nasıl sıralayacağınızı göstermektedir.

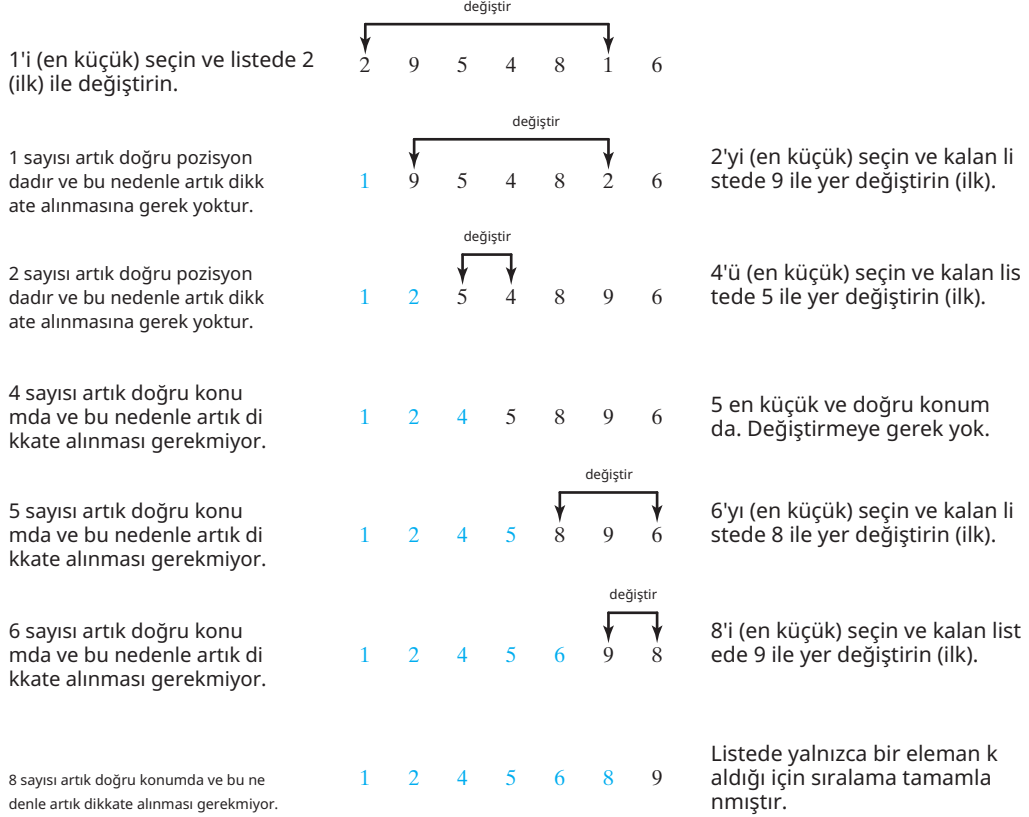


VideoNotu

Seçim sıralaması



seçim sıralama animasyonu
Ortak Web Sitesi



FIGİN6.12 Seçim sıralaması, en küçük sayıyı tekrar tekrar seçer ve bunu listede ilk sayı ile yer değiştirir.

Seçim sıralama yaklaşımının nasıl çalıştığını biliyorsunuz. Şimdi görev, bunu Java'da uygulamaktır. Acemiler, ilk denemede tam bir çözüm geliştirmekte zorlanıyorlar. Öncelikle, listedeki en küçük öğeyi bulmak ve bunu ilk öğe ile değiştirmek için ilk yineleme için kodu yazmaya başlayın, ardından ikinci yineleme, üçüncü ve devamı için neyin farklı olacağını gözlemleyin. Bu, tüm yinelemeleri genelleştiren bir döngü yazmanıza olanak tanıyacak bir içgörü sağlayacaktır.

Çözüm şu şekilde tanımlanabilir:

```
for (int i = 0; i < list.length - 1; i++) {
    list[i..list.length-1] içindeki en küçük öğeyi seç;
    en küçükü list[i] ile değiştir, gerekirse;
    // list[i] doğru konumundadır.
    // Bir sonraki yineleme list[i+1..list.length-1] üzerinde uygulanır.
}
```

Liste 6.8 çözümü uygular.

LISTE 6.8 SelectionSort.java

```

1 genel sınıfSelectionSort {
2     /** Sayıları sıralama yöntemi */
3     public static void selectionSort(double[] list) {
4         for (int i = 0; i < list.length - 1; i++) {
5             // List[i..list.length-1] içindeki en küçük değeri bul
6             double mevcutMin = list[i];
7             int mevcutMinIndeksi = i;
8
9             for (int j = i + 1; j < list.length; j++) {
10                 if (mevcutMin > list[j]) {
11                     mevcutMin = list[j];
12                     mevcutMinIndeksi = j;
13                 }
14             }
15
16             // Gerekirse list[i] ile list[mevcutMinIndeksi] yer değiştir
17             if (mevcutMinIndeksi != i) {
18                 list[mevcutMinIndeksi] = list[i];
19                 list[i] = mevcutMin;
20             }
21         }
22     }
23 }

```

seçin

değiştir

selectionSort(double[] list) metodu, herhangi bir double elemanlar dizisini sıralar. Metoda, iç içe geçmiş bir for döngüsü ile uygulanmıştır. Dış döngü (döngü kontrol değişkeni i) (satır 4), listede en küçük elemanı bulmak için list[i] ile list[list.length-1] arasında yinelemektir ve bunu list[i] ile değiştirmektedir.

Değişken **i** başlangıçta **0**'dır. Dış döngünün her yinelemesinden sonra, **list[i]** doğru yerdedir. Sonunda, tüm elemanlar doğru yere yerleştirilir; bu nedenle, tüm liste sıralanmıştır.

Bu yöntemi daha iyi anlamak için, aşağıdaki ifadelerle izleyin:

```
double[] liste = {1.1, 2.9, 4.5, 6.6, 5.7, 4.5};
Sort.selectionSort(liste);
```



ekleme sıralama animasyonu
Ortak Web Sitesi

6.11.2 Ekleme Sıralaması

Diyelim ki bir listeyi artan sırayla sıralamak istiyorsunuz. Ekleme sıralama algoritması, bir listeyi sıralı bir alt listeye sürekli olarak yeni bir eleman ekleyerek sıralar, ta ki tüm liste sıralanana kadar. Şekil 6.13, {2,9,5,4,8,1,6} listesini ekleme sıralaması kullanarak nasıl sıralayacağınızı göstermektedir.

Algoritma şu şekilde tanımlanabilir:

```
for (int i = 1; i < list.length; i++) {
    list[i] öğesini sıralı alt liste list[0..i-1]'ye ekleyin, böylece
    list[0..i] sıralı olur.
}
```

list[i]'yi **list[0..i-1]**'ye eklemek için, **list[i]**'yi geçici bir değişkene, diyelim ki **currentElement**'ye kaydedin. Eğer **list[i-1] > currentElement** ise, **list[i-1]**'yi **list[i]**'ye taşıyın, **list[i-2]**'yi **list[i-1]**'ye taşıyın eğer **list[i-2] > currentElement** ise, ve bu şekilde devam edin, ta ki **list[i-k] <= currentElement** veya **> i** (sıralı listenin ilk elemanını geçiyoruz). **list[i-k+1]** değerini **currentElement** olarak atayın. Örneğin, Şekil 6.14'teki Adım 4'te {2,5,9} içine 4 eklemek için, **list[2](9)** değerini **list[3]**'ye taşıyın çünkü **9 > 4**, ve **list[1](5)** değerini **list[2]**'ye taşıyın çünkü **5 > 4**. Son olarak, **currentElement(4)** değerini **list[1]**'ye taşıyın.

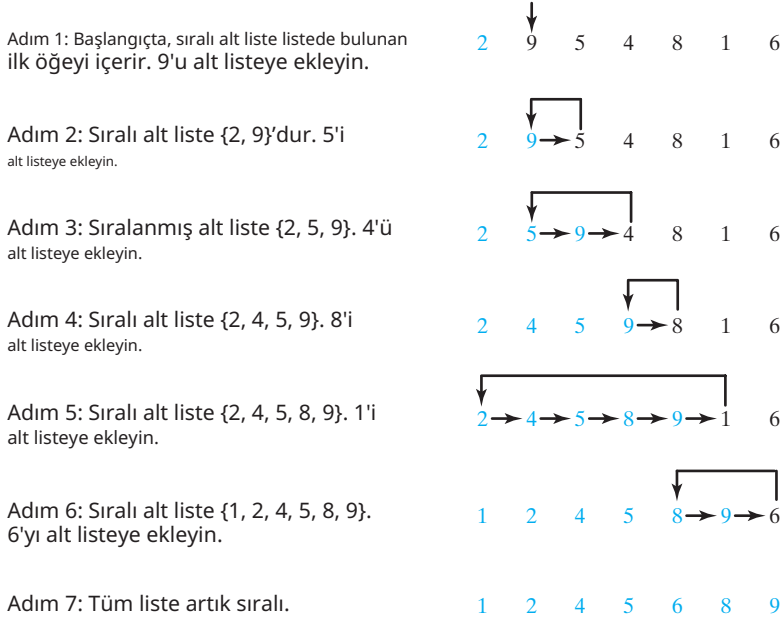
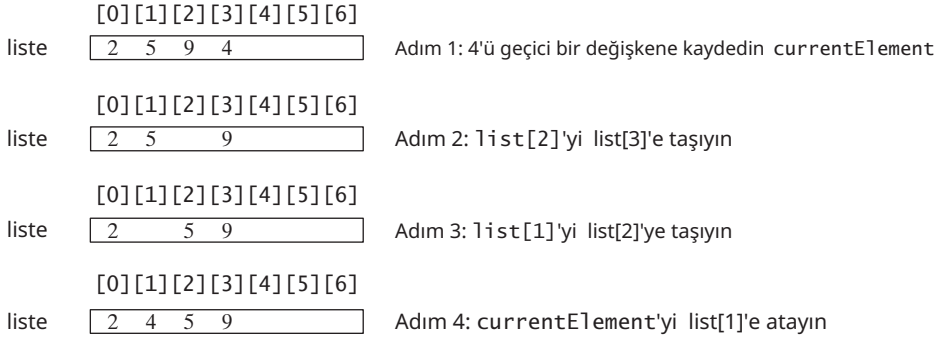


FIGURE 6.13 Ekleme sıralaması, sıralı bir alt listeye sürekli olarak yeni bir öge ekler.



FGÖRÜNTÜ 6.14 Sıralı bir alt listeye yeni bir eleman ekleniyor.

Algoritma genişletilebilir ve Liste 6.9'da olduğu gibi uygulanabilir.

LtISTINGt 6.9tInsertionSort.java

```

1  genel classtInsertionSort {
2      /** Sayıları sıralama yöntemi */
3      public static voidtinsertionSort(tdoublet[] list) {
4          fort(tinti = t1t; i < liste.uzunluk; i++) {
5              /** list[0..i-1] içinde sıralı bir alt listeye list[i] ekleyin, böylece
6                  list[0..i] sıralı olur. */
7                  doubletcurrentElement = liste[i];
8                  inttk;
9                  fort(k = i - t1t; k >= t0t&& liste[k] > currentElement; k— —) {
10                     liste[k + t1t] = liste[k];n11 }

```

kaydırma

252 Bölüm 6 Tek Boyutlu Diziler

ekle

```
14         liste[k + 1] = currentElement;
15     }n16 }n1
7 }
```

`double[]` list parametresi ile `insertionSort` metodu, herhangi bir `double` elemanlar dizisini sıralar. Metot, iç içe geçmiş bir `for` döngüsü ile uygulanmıştır. Dış döngü (döngü kontrol değişkeni `i`) (satır 4), `list[0]` ile `list[i]` arasında sıralı bir alt liste elde etmek için yinelemektedir. İç döngü (döngü kontrol değişkeni `k`) ise `list[i]`'yi `list[0]` ile `list[i-1]` arasındaki alt listeye ekler.

Bu yöntemi daha iyi anlamak için, aşağıdaki ifadelerle izleyin:

```
double[] liste = {1, 9, 4.5, 6.6, 5.7, -4.5};
InsertionSort.insertionSort(liste);
```

Point C
control

MyProgrammingLab™

- 6.19** Şekil 6.10'u, anahtar `10` ve anahtar `12` için `{2,4,7,10,11,45,50,59,60,66,69,70,79}` listesindeki arama için ikili arama yaklaşımının nasıl uygulanacağını göstermek için bir örnek olarak kullanın.
- 6.20** Eğer ikili arama yöntemi `-4` dönerse, anahtar listede mi? Anahtarı listeye eklemek isterseniz, anahtar nereye eklenmelidir?
- 6.21** Şekil 6.12'yi bir örnek olarak kullanarak, `{3.4,5,3,3.5,2.2,1.9,2}` dizisini sıralamak için seçim sıralama yaklaşımının nasıl uygulanacağını gösterin.
- 6.22** Şekil 6.13'ü bir örnek olarak kullanarak, `{3.4,5,3,3.5,2.2,1.9,2}` dizisini sıralamak için ekleme sıralama yaklaşımının nasıl uygulanacağını gösterin.
- 6.23** Liste 6.8'deki `selectionSort` yöntemini sayıları azalan sırada sıralamak için nasıl değiştirirsiniz?
- 6.24** Liste 6.9'daki `insertionSort` yöntemini sayıları azalan sırada sıralamak için nasıl değiştirirsiniz?

6.12 Arrays Sınıfı



java.util.Arrays sınıfı, sıralama ve arama gibi yaygın dizi işlemleri için yararlı yöntemler içerir.

`java.util.Arrays` sınıfı, dizileri sıralamak ve aramak, dizileri karşılaştırmak, dizi elemanlarını doldurmak ve dizinin string temsili döndürmek için çeşitli statik yöntemler içerir. Bu yöntemler tüm ilkel türler için aşırı yüklenmiştir.

`sort` yöntemini tüm bir diziyi veya kısmi bir diziyi sıralamak için kullanabilirsiniz. Örneğin, aşağıdaki kod bir sayı dizisini ve bir karakter dizisini sıralar.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
java.util.Arrays.sort(sayılar); // Tüm diziyi sırala
```

```
char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars, 1, 3); // Sort part of the array
```

Çağırma `sort(numbers)` tüm diziyi `numbers` sıralar. Çağırma `sort(chars, 1, 3)` dizinin bir kısmını `chars[1]` ile `chars[3-1]`.

Bir dizide bir anahtar aramak için `binarySearch` yöntemini kullanabilirsiniz. Dizi önceden sıralanmış olmalıdır. Anahtar dizide yoksa, yöntem `-(insertionindex + 1)` döner. Örneğin, aşağıdaki kod bir tamsayı dizisinde ve bir karakter dizisinde anahtarları arar.

```
int[] liste = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};
System.out.println("(1) İndeks "+
    java.util.Arrays.binarySearch(liste, 11));
```

sırala

ikili Arama

```
System.out.println("(2) İndeks "+
    java.util.Arrays.binarySearch(list, 12));

char[] chars = {'a', 'c', 'g', 'x', 'y', 'z'};
System.out.println("(3) İndeks "+
    java.util.Arrays.binarySearch(chars, 'a'));
System.out.println("(4) İndeks "+
    java.util.Arrays.binarySearch(chars, 't'));
```

Önceki kodun çıktısıdır

1. İndeks 4
2. İndeks -6
3. İndeks 0
4. İndeks -4

equals yöntemini kullanarak iki dizinin eşit olup olmadığını kontrol edebilirsiniz. İki dizi eşittir aynı içeriğe sahipse. Aşağıdaki kodda, **list1** ve **list2** eşittir, ancak **list2** ve **list3** eşit değildir.

```
int[] is not1 = {tat,t4t,under,t10t};nn
th[] ist = {tat,t4t,under,t10t};nth[] ist
= {t4t,tat,under,t10t};
System.out.println(ays.equals(list1, liste2) ); // doğru
System.out.println(java.util.Arrays.equals(list2, liste3)njava.util.Arr ); // yanlış
```

Array'in tamamını veya bir kısmını doldurmak için **fill** yöntemini kullanabilirsiniz. Örneğin, aşağıdaki kod **list1**'yi 5 ile doldurur ve **8**'yi **list2[1]** ve **list2[3-1]** elemanlarına doldurur.

```
int[] not1 = {tat,t4t,altında,t10t};
int[] list = {tat,t4t,altında,t10t};
ys.fill(list1,t5t) ;// Tüm diziye 5 doldur
java.util.Arrays.fill(list2,t1t,t3t,t8t)njava.util.Arra ;// Kısmi diziye 8 doldur
```

Array'deki tüm elemanları temsil eden bir dize döndürmek için **toString** yöntemini de kullanabilirsiniz. Bu, dizideki tüm elemanları görüntülemenin hızlı ve basit bir yoludur. Örneğin, aşağıdaki kod

```
intt[] list = {t2t,t4t,t7t,t10t};System.
out.println(Arrays.toString(list));
```

şu şekilde görüntüler: [2, 4, 7, 10].

6.25 Hangi tür diziler **java.util.Arrays.sort** yöntemi kullanılarak sıralanabilir?

Bu **sort** yöntemi yeni bir dizi mi oluşturuyor?

6.26 **java.util.Arrays.binarySearch(array, key)** uygulamak için, dizi artan sırada, azalan sırada mı yoksa hiçbirini mi sıralanmalı?

6.27 Aşağıdaki kodun çıktısını göster:

```
int[] not1 = {tat,t4t,under,t10t};chan
delier.athel.brides.feel(list1,under);text.o
ut.println(java.attil.brides.string(list1));
```

```
int[]list = {tat,t4t,altında,t10t};
System.out.println(java.util.Arrays.toString(list2));nSystem.out.print(
java.util.Arrays.equals(list1, list2));
```

Anahtar Terimler

anonim dizi 238	indeksli değişken 226
dizi 224	insertion sort 248
dizi başlatıcı 227	lineer arama 245
ikili arama 245	bir eksik hata 230
çöp toplama 236	seçim sıralaması 248
indeks 224	

BÖLÜM ÖZETİ

1. Bir değişken, `elementType[] arrayRefVar` veya `elementType arrayRefVar[]` sözdizimini kullanarak bir dizi türü olarak tanımlanır. `elementType[] arrayRefVar` stili terci h edilir, ancak `elementType arrayRefVar[]` yasal bir ifadedir.
2. Temel veri türü değişkenleri için yapılan bildirimlerin aksine, bir dizi değişkenini n bildirilmesi dizinin bellekte herhangi bir alan ayırmaz. Bir dizi değişkeni, temel veri türü değişkeni değildir. Bir dizi değişkeni, bir diziye referans içerir.
3. Bir diziye eleman atayamazsınız, eğer dizi daha önce oluşturulmamışsa. Bir diziyi, aşğdaki sözdizimini kullanarak `new` operatörü ile oluşturabilirsiniz: `new elementType[arraySize]`.
4. Dizideki her bir eleman, `arrayRefVar[index]` sözdizimi kullanılarak temsil edilir. Bir *indeks* bir tam sayı veya bir tam sayı ifadesi olmalıdır.
5. Bir dizi oluşturulduktan sonra, boyutu kalıcı hale gelir ve şu şekilde elde edilebilir `diziRefDeğişkeni.length`. Bir dizinin indeksi her zaman 0 ile başladığından, son indeks her zaman `diziRefDeğişkeni.length - 1`'dir. Bir dizinin sınırlarının ötesinde öğelere erişmeye çalışırsanız bir sınır dışı hatası oluşacaktır.
6. Programcılar genellikle bir dizideki ilk öğeye 1 indeksi ile erişmeye çalışır, bunun yerine 0 olmalıdır. Bu, *bir indeks hatası* olarak adlandırılır.
7. Bir dizi oluşturulduğunda, öğeleri sayısal ilkel veri türleri için varsayılan değer olan 0, char türleri için `\u0000` ve boolean türleri için `false` ile atanır.
8. Java, bir diziyi tanımlama, oluşturma ve başlatmayı tek bir ifadede birleştiren *dizi başlatıcısı* olarak bilinen bir kısayol notasyonu sunar; bu, şu sözdizimini kullanır: `elementType[] diziRefDeğişkeni = {değer0, değer1, ..., değerK}`.
9. Bir dizi argümanını bir metoda geçirdiğinizde, aslında dizinin referansını geçiriyor sunuz; yani, çağrılan metod, çağırının orijinal dizisindeki öğeleri değiştirebilir.
10. Eğer bir dizi sıralanmışsa, *ikili arama*, dizide bir öğe bulmak için *doğrusal arama*'dan daha etkilidir.
11. *Seçim sıralaması* listedeki en küçük sayıyı bulur ve onu ilk elemanla değiştirir. Daha sonra kalan en küçük sayıyı bulur ve onu kalan listedeki ilk elemanla değiştirir, v e bu şekilde devam eder, ta ki yalnızca bir sayı kalana kadar.

12. Ekleme sıralama algoritması değerler listesini, yeni bir elemanı sıralı bir alt listeye tekrar tekrar ekleyerek sıralar, ta ki tüm liste sıralanana kadar.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinde yapın.

PROGRAMLAMA EGZERSİZLERİ

MyProgrammingLab™

Bölmeler 6.2-6.5

- *6.1** (*Notları Ata*) Öğrenci notlarını okuyan, en iyi notu alan ve ardından aşağıdaki şemaya göre notları atan bir program yazın:

Not A'dır eğer not $\geq$ en iyi - 10

Not B'dir eğer not $\geq$ en iyi - 20 ;

Not ortalaması $\geq$ en iyi - 30 ;

Not ortalaması D $\geq$ en iyi - 40 ;

Aksi takdirde not ortalaması F.

Program, kullanıcıdan toplam öğrenci sayısını girmesini ister, ardından kullanıcıdan tüm notları girmesini ister ve notları göstererek sonuçlanır. İşte örnek bir çalışma:

```
Öğrenci sayısını girin: 4 Enter
4 not girin: 40 55 70 58 Enter
Öğrenci 0 notu 40 ve notu C
Öğrenci 1 notu 55 ve notu B
Öğrenci 2 notu 70 ve notu A
Öğrenci 3 notu 58 ve notu B
```



- 6.2** (*Girilen sayıları ters çevir*) On tam sayıyı okuyan ve okunduğu sıranın tersinde gösteren bir program yazın.

- **6.3** (*Sayıların tekrar sayısını say*) 1 ile 100 arasındaki tam sayıları okuyan ve her birinin tekrar sayısını sayan bir program yazın. Girişin 0 ile sona erdiğini varsayın. İşte programın örnek çalışması:

```
1 ile 100 arasındaki tam sayıları girin: 2 5 6 5 4 3 23 43 2 0 Enter
2, 2 kez geçiyor
3, 1 kez geçiyor
4, 1 kez geçiyor
5, 2 kez geçiyor
6, 1 kez geçiyor
23, 1 kez geçiyor
43, 1 kez geçiyor
```



Bir sayının birden fazla kez geçmesi durumunda, çıktıda "kez" kelimesinin çoğul hali kullanıldığını unutmayın.

6.4 (*Notları analiz et*) Belirsiz sayıda notu okuyan ve kaç notun ortalamasının üzerinde veya eşit olduğunu ve kaç notun ortalamasının altında olduğunu belirleyen bir program yazın. Girişi sonlandırmak için negatif bir sayı girin. Notların maksimum sayısının 100 olduğunu varsayın.

****6.5** (*Ayrı sayıları yazdır*) On sayıyı okuyan ve ayrı sayıları (yani, bir sayı birden fazla kez geçiyorsa, yalnızca bir kez gösterilir) görüntüleyen bir program yazın. (*İpucu:* Bir sayı okuyun ve eğer yeni ise bir diziye kaydedin. Sayı zaten dizide varsa, onu göz ardı edin.) Girişten sonra, dizi ayrı sayıları içerir. İşte programın örnek çalışması:



On sayı girin: 1 2 3 2 1 6 3 4 5 2
Ayrı sayılar: 1 2 3 6 4 5

***6.6** (*Liste 4.14'ü gözden geçirin, PrimeNumber.java*) Liste 4.14, bir sayının n asal olup olmadığını, 2, 3, 4, 5, 6, ..., $n/2$ 'nin bir bölen olup olmadığını kontrol ederek belirler. Bir bölen bulunursa, n asal değildir. Daha verimli bir yaklaşım,

pamuk n eşit olarak bölünebilir. Değilse, n asal sayıdır.

Bu yaklaşımı kullanarak ilk 50 asal sayıyı görüntülemek için Listeleme 4.14'ü yeniden yazın.

Asal sayıları depolamak için bir dizi kullanmanız ve daha sonra bunları n için olası bölenler olup olmadıklarını kontrol etmek için kullanmanız gerekiyor.

***6.7** (*Tek haneli sayıları say*) 0 ile 9 arasında 100 rastgele tam sayı üreten ve her sayı için sayımı görüntüleyen bir program yazın. (*İpucu:* 0 ile 9 arasında rastgele bir tam sayı üretmek için `(int)(Math.random()*10)` kullanın. 0, 1, ..., 9 sayılarının sayımlarını depolamak için on tam sayıdan oluşan bir dizi, diyelim ki `counts`, kullanın.)

Bölümler 6.6–6.8

6.8 (*Bir dizinin ortalamasını al*) Aşağıdaki başlıklarla bir dizinin ortalamasını döndüren iki aşırı yüklenmiş yöntem yazın:

```
public static int ortalama(int[] dizi)
public static double ortalama(double[] dizi)
```

Kullanıcıdan on tane double değeri girmesini isteyen, bu yöntemi çağıran ve ortalama değeri görüntüleyen bir test programı yazın.

6.9 (*En küçük elemanı bul*) Aşağıdaki başlıkla bir double değerler dizisinde en küçük elemanı bulan bir yöntem yazın:

```
public static double min(double[] dizi)
```

Kullanıcının on sayı girmesini isteyen, bu yöntemi çağırarak minimum değeri döndüren ve minimum değeri gösteren bir test programı yazın. İşte programın bir örnek çalışması:



On sayı girin: 1.9 2.5 3.7 2 1.5 6 3 4 5 2
Minimum sayı: 1.5

6.10 (*En küçük elemanın indeksini bul*) Bir tamsayı dizisindeki en küçük elemanın indeksini döndüren bir yöntem yazın. Eğer bu tür elemanların sayısı 1'den fazlaysa, en küçük indeksi döndürün. Aşağıdaki başlığı kullanın:

```
public static int indexOfSmallestElement(double[] dizisi)
```


Kullanıcıdan on sayı girmesini isteyen, bu yöntemi çağırarak en küçük elemanın indeksini döndüren ve indeksi gösteren bir test programı yazın.

- *6.11** (İstatistikler: sapmayı hesaplama) Programlama Egzersizi 5.37, sayıların standart sapmasını hesaplar. Bu egzersiz, n sayısının standart sapmasını hesaplamak için farklı ama eşdeğer bir formül kullanır.

$$mean = \frac{\sum_{i=1}^n x_i}{n} = \frac{x_1 + x_2 + \dots + x_n}{n} \quad deviation = \sqrt{\frac{\sum_{i=1}^n (x_i - mean)^2}{n - 1}}$$

Bu formülle standart sapmayı hesaplamak için, ortalama elde edildikten sonra kullanılabilmesi için bireysel sayıları bir dizi kullanarak saklamanız gerekir.

Programınız aşağıdaki yöntemleri içermelidir:

```
/** Çift değerlerin sapmasını hesaplama */
public static double deviation(double[] x)

/** Çift değerler dizisinin ortalamasını hesaplama */
public static double mean(double[] x)
```

Kullanıcıdan on sayı girmesini isteyen ve ortalama ile standart sapmayı gösteren bir test programı yazın; aşağıdaki örnek çalışmada gösterildiği gibi:

On sayı girin: 1.9 2.5 3.7 2 1 6 3 4 5 2 Enter
Ortalama 3.11'dir
Standart sapma 1.55738'dir



- *6.12** (Bir diziyi ters çevir) Bölüm 6.7'deki `ters` yöntemi, bir diziyi yeni bir diziye kopyalayıp ters çevirir. Argüman olarak geçirilen diziyi ters çeviren ve bu diziyi döndüren yöntemi yeniden yazın. Kullanıcıdan on sayı girmesini isteyen, sayıları ters çevirmek için yöntemi çağırarak ve sayıları görüntüleyen bir test programı yazın.

Bölüm 6.9

- *6.13** (Rastgele sayı seçici) Argümanda belirtilen sayılar hariç, 1 ile 54 arasında rastgele bir sayı döndüren bir yöntem yazın. Yöntem başlığı aşağıdaki gibi belirtilmiştir:

```
public static int getRandom(int... sayılar)
```

- 6.14** (En büyük ortak bölen (gcd) hesaplama) Belirsiz sayıda tam sayının gcd'sini döndüren bir yöntem yazın. Yöntem başlığı aşağıdaki gibi belirtilmiştir:

```
public static int gcd(int... sayılar)
```

Kullanıcıdan beş sayı girmesini isteyen, bu sayıların gcd'sini bulmak için yöntemi çağırarak ve gcd'yi görüntüleyen bir test programı yazın.

Bölümler 6.10–6.12

- 6.15** (Tekrarları ortadan kaldır) Aşağıdaki yöntem başlığını kullanarak dizideki tekrar eden değerleri ortadan kaldırarak yeni bir dizi döndüren bir yöntem yazın:

```
public static int[] Yinelenenleri ortadan kaldırır(int[] listesinde)
```

On tam sayı okuyan, yöntemi çağıran ve sonucu gösteren bir test programı yazın. İşte programın örnek çalışması:



On sayı girin: 1 2 3 2 1 6 3 4 5 2
Ayrı sayılar: 1 2 3 6 4 5

6.16 (*Çalışma süresi*) 100.000 tamsayıdan oluşan rastgele bir dizi ve bir anahtar üreten bir program yazın. `linearSearch` metodunun 6.6. Listesi çağırılmasının çalışma süresini tahmin edin. Diziyi sıralayın ve `binarySearch` metodunun 6.7. Listesi çağırılmasının çalışma süresini tahmin edin. Aşağıdaki kod şablonunu kullanarak çalışma süresini elde edebilirsiniz:

```
long startTime = System.currentTimeMillis();  
görevi yerine getir;  
long endTime = System.currentTimeMillis();  
long executionTime = endTime - startTime;
```

****6.17** (*Öğrencileri sırala*) Kullanıcıdan öğrenci sayısını, öğrencilerin isimlerini ve notlarını girmesini isteyen ve öğrencilerin isimlerini notlarının azalan sırasına göre yazdıran bir program yazın.

****6.18** (*Balon sıralama*) Balon sıralama algoritmasını kullanan bir sıralama metodu yazın. Balon sıralama algoritması dizinin üzerinden birkaç geçiş yapar. Her geçişte, ardışık komşu çiftler karşılaştırılır. Eğer bir çift sıralı değilse, değerleri değiştirilir; aksi takdirde, değerler değişmeden kalır. Bu teknik *balon sıralama* veya *batan sıralama* olarak adlandırılır çünkü daha küçük değerler yavaş yavaş "baloncuk" gibi yukarı çıkar ve daha büyük değerler "batar". On çift sayı okuyan, metodu çağıran ve sıralı sayıları gösteren bir test programı yazın.

****6.19** (*Sıralı mı?*) Liste artan sırada sıralıysa true döndüren aşağıdaki yöntemi yazın.

```
public static boolean isSorted(T[] list)
```

Kullanıcıdan bir liste girmesini isteyen ve listenin sıralı olup olmadığını gösteren bir test programı yazın. İşte bir örnek çalışma. Girdi içindeki ilk sayının, listedeki eleman sayısını gösterdiğini unutmayın.



Listeyi girin: 8 10 1 5 16 61 9 11 1
Liste sıralı değil



Listeyi girin: 10 1 1 3 4 4 5 7 9 11 21
Liste zaten sıralı

***6.20** (*Seçim sıralamasını gözden geçirin*) Bölüm 6.11.1'de, bir diziyi sıralamak için seçim sıralamasını kullandınız. Seçim sıralama yöntemi, mevcut dizideki en küçük sayıyı tekrar tekrar bulur ve onu ilk elemanla değiştirir. Bu programı, en büyük sayıyı bulup son elemanla değiştirerek yeniden yazın. On adet double sayı okuyan, yöntemi çağıran ve sıralı sayıları gösteren bir test programı yazın.

*****6.21** (*Oyun: fasulye makinesi*) Fasulye makinesi, aynı zamanda quincunx veya Galton kutusu olarak da bilinir, İngiliz bilim insanı Sir Francis Galton'un adını taşıyan istatistik deneyleri için bir cihazdır. Şekil 6.15'te gösterildiği gibi, üçgen formda eşit aralıklarla yerleştirilmiş çiviler (veya çubuklar) bulunan dik bir tahtadan oluşur.

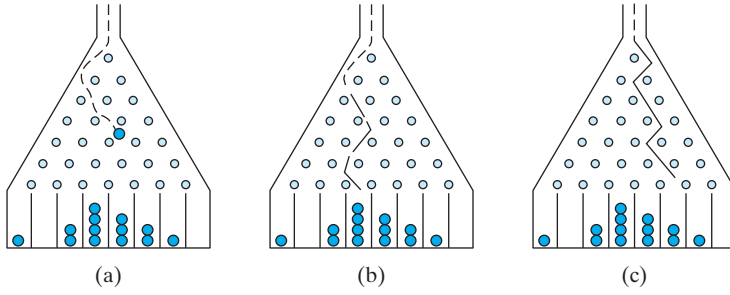


FIGURE 6.15 Her top rastgele bir yol alır ve bir slota düşer.

Ballar tahtanın açılışından düşürülür. Her seferinde bir top bir çiviye çarptığında, sola veya sağa düşme şansı %50'dir. Top yığınları tahtanın altındaki yuvalarda birikir.

Fasulye makinesini simüle eden bir program yazın. Programınız, kullanıcıdan topların sayısını ve makinedeki yuvaların sayısını girmesini istemelidir. Her topun düşüşünü simüle ederek yolunu yazdırın. Örneğin, Şekil 6.15b'deki topun yolu LLRRLLR'dir ve Şekil 6.15c'deki topun yolu RLRRLRR'dir. Yuvalardaki topların son birikimini bir histogramda gösterin. İşte programın bir örnek çalışması:

Atılacak top sayısını girin:

Fasulye makinesindeki yuva sayısını girin:

LRLRLRRnR
 RLLLLRRnL
 LRLLLRRnR
 RRLLLLLnL
 RLRLRLR

O
 O
 O
 O
 O



(İpucu: `slots` adında bir dizi oluşturun. `slots` içindeki her bir eleman, bir yuvada bulunan top sayısını saklar. Her top, bir yol aracılığıyla bir yuva ya da düşer. Bir yoldaki R sayısı, topun düştüğü yuvanın konumudur. Örneğin, LLRLRLR yolu için top `slots[4]`'e düşer ve RRLLLLL yolu için top `slots[2]`'e düşer.)

*****6.22**(Oyun: Sekiz Kraliçe) Klasik Sekiz Kraliçe bulmacası, sekiz kraliçeyi bir satranç tahtasına yerleştirmektir, böylece hiçbir iki kraliçe birbirini tehdit edemez (yani, hiçbir iki kraliçe aynı satırda, aynı sütunda veya aynı diyagonal üzerinde değildir). Birçok olasılık vardır. Böyle bir çözümü gösteren bir program yazın. Aşağıda bir örnek çıktı gösterilmektedir:

```

|Q| | | | | | |
| | | | |Q| | |
| | | | | | |Q|
| | | |Q| | | |
| | | | | |Q| |
| |Q| | | | | |
| | |Q| | | | |

```

****6.23** (Oyun: kilitli bulmaca) Bir okulda 100 kilit ve 100 öğrenci vardır. Tüm kilitler okulun ilk gününde kapalıdır. Öğrenciler içeri girdiğinde, ilk öğrenci, S1, her kilidi açar. Sonra ikinci öğrenci, S2, ikinci kilit olan L2 ile başlar ve her diğer kilidi kapatır. Öğrenci S3, üçüncü kilit ile başlar ve her üçüncü kilidi değiştirir (açık ise kapatır, kapalı ise açar). Öğrenci S4, L4 kilidi ile başlar ve her dördüncü kilidi değiştirir. Öğrenci S5, L5 ile başlar ve her beşinci kilidi değiştirir, ve bu şekilde devam eder, ta ki öğrenci S100 L100'ü değiştirene kadar.

Tüm öğrenciler binadan geçtikten ve kilitleri değiştirdikten sonra, hangi kilitler açıktır? Cevabınızı bulmak için bir program yazın.

(İpucu: 100 Boolean elemanından oluşan bir dizi kullanın, her biri bir kilidin açık olup olmadığını (**true**) veya kapalı olup olmadığını (**false**) gösterir. Başlangıçta, tüm kilitler kapalıdır.)



VideoNotu

Kupon toplayıcısının problemi

****6.24** (Simülasyon: kupon toplayıcısının problemi) Kupon toplayıcı, birçok pratik uygulama sı olanklasik bir istatistik problemidir. Problem, bir nesne setinden nesneleri tekrar tekrar seçmek ve bütün nesnelerin en az bir kez seçilmesi için kaç seçimin gerektiğini bulmaktır. Problemin bir varyasyonu, karıştırılmış 52 kartlık bir destede n kartları tekrar tekrar seçmek ve her bir suitten bir tane görmek için kaç seçimin gerektiğini bulmaktır. Seçilen bir kartın başka bir seçim yapmadan önce desteye geri konduğunu varsayın. Her suitten dört kart almak için kaç seçim gerektiğini simüle eden bir program yazın ve seçilen dört kartı gösterin (bir kartın iki kez seçilmesi mümkündür). İşte programın bir örnek çalışması:



Karo Kraliçesi
Sinek Beşi
Kalp Kraliçesi
Karo 4
Seçim sayısı: 12

6.25 (Cebir: ikinci dereceden denklemleri çöz) Aşağıdaki başlığı kullanarak bir ikinci dereceden denklemi çözme yöntemi yazın:

`public static int[] çözKuadratik(double[] denklem, double[] kökler)`

Bir ikinci dereceden denklemin katsayıları $ax^2 + bx + c = 0$ diziye aktarılır `denklem` ve karmaşık olmayan kökler köklerde saklanır. Yöntem kök sayısını döndürür. Kare denklemi nasıl çözeceğinizi görmek için Programlama Egzersizi 3.1'e bakın.

Kullanıcıdan a, b ve c değerlerini girmesini isteyen ve kök sayısını ve tüm karmaşık olmayan kökleri gösteren bir program yazın.

6.26 (Tamamen aynı diziler) Diziler `list1` ve `list2`, eşleşen elemanları eşitsiz ve tamamen aynıdır. Aşağıdaki başlığı kullanarak `true` döndüren bir yöntem yazın:

`public static boolean equals(int[] liste1, int[] liste2)`

Kullanıcıdan iki tam sayı listesi girmesini isteyen ve bu iki listenin tam olarak aynı olup olmadığını gösteren bir test programı yazın. İşte örnek ç alıştırmalar. Girişteki ilk sayı, listedeki eleman sayısını belirtir.



list1'i girin: 5 2 5 6 1 6 Enter
list2'yi girin: 5 2 5 6 1 6 Enter
İki liste kesinlikle özdeşdir

```
list1'i girin:  5 2 5 6 6 1  ↵ Enter
list2'yi girin: 5 2 5 6 1 6  ↵ Enter
İki liste kesinlikle özdeş değildir
```



6.27 (Aynı diziler) list1 ve list2 dizileri aynı içeriklere sahipse özdeşdir. list1 ve list2 özdeşse true döndüren bir yöntem yazın, aşağıdaki başlığı kullarak:

```
public static boolean equals(tintt[] liste1, tintt[] liste2)
```

Kullanıcıdan iki tam sayı listesi girmesini isteyen ve ikisinin özdeş olup olmadığını gösteren bir test programı yazın. İşte örnek çalıştırmalar. Girişteki ilksayı, listedeki eleman sayısını belirtir.

```
list1'i girin:  5 2 5 6 6 1  ↵ Enter
list2'yi girin: 5 5 2 6 1 6  ↵ Enter
İki liste özdeşdir
```



```
list1'i girin:  5 5 5 6 6 1  ↵ Enter
list2'yi girin: 5 2 5 6 1 6  ↵ Enter
İki liste özdeş değildir
```



***6.28** (Matematik: kombinasyonlar) Kullanıcıdan 10 tam sayı girmesini isteyen ve 10 sayıdan iki sayı seçmenin tüm kombinasyonlarını gösteren bir program yazın.

***6.29** (Oyun: dört kart seç) 52 kartlık bir desteden dört kart seçen ve bunların toplamını hesaplayan bir program yazın. Bir As, Kral, Kız ve Vale sırasıyla 1, 13, 12 ve 11'i temsil eder. Programınız, toplamı 24 olan seçimlerin sayısını göstermelidir.

***6.30** (Desen tanıma: ardışık dört eşit sayı) Dizinin aynı değere sahip dört ardışık sayıya sahip olup olmadığını test eden aşağıdaki yöntemi yazın.



VideoNotu
Ardışık dört

```
public static boolean ardışıkDört(int[] değerler)
```

Kullanıcıdan bir dizi tam sayı girmesini isteyen ve dizi dört ardışık aynı değere sahip ise true, aksi takdirde false gösteren bir test programı yazın. Programınız önce kullanıcıdan girdi boyutunu—yani dizideki değer sayısını—girmesini istemelidir.

****6.31** (İki sıralı listeyi birleştir) İki sıralı listeyi yeni bir sıralı listeye

birleştiren aşağıdaki yöntemi yazın.

```
public static int[] birleştir(int[] liste1, int[] liste2)
```

Yöntemi, liste1.length + liste2.length

karşılaştırması alacak şekilde uygulayın. Kullanıcıdan iki sıralı liste girmesini isteyen ve birleştirilmiş listeyi gösteren bir test programı yazın. İşte bir örnek çalışma. Girdi şeklinde ilk sayının listedeki eleman sayısını gösterdiğine dikkat edin.



```
list1'i girin:  5 1 5 16 61 111 ↵ Enter
list2'yi girin: 4 2 4 5 6 ↵ Enter
Birleştirilmiş liste 1 2 4 5 5 6 16 61 111
```

****6.32**(*Bir listeyi bölme*) Aşağıdaki yöntemi yazın, bu yöntem listeyi

ilk elemanı, *pivot* olarak adlandırarak böler.

public static intböl(**int**[] liste)

Bölmeden sonra, listedeki elemanlar, pivot'tan önceki tüm elemanların pivot'tan küçük veya eşit ve pivot'tan sonraki elemanların pivot'tan büyük olacak şekilde yeniden sıralanır. Yöntem, pivot'un yeni listedeki konumunu gösteren indksi döndürür. Örneğin, farz edelim ki liste {5, 2, 9, 3, 6, 8} . Parti-

sondan sonra, liste {3, 2, 5, 9, 6, 8} . Yöntemi,

List.length karşılaştırmalarını alacak şekilde uygulayın. Kullanıcıdan bir liste girme sini isteyen ve partiyon sonrası listeyi gösteren bir test programı yazın. İşte bir örne k çalışma. Girişteki ilk sayının listedeki eleman sayısını gösterdiğini unutmayın.



```
Listeyi girin:  8 10 1 5 16 61 9 11 1 ↵ Enter
Partisyonndan sonra, liste 9 1 5 1 10 61 11 16'dır.
```

***6.33**(*Kültür: Çin Zodyağı*) Hayvan isimlerini saklamak için bir dizi kullanarak Liste 3.10' u basitleştirin.

*****6.34**(*Oyun: çoklu Sekiz Kraliçe çözümleri*) Alıştırma 6.22, Sekiz Kraliçe problemi için bir çözüm bulur. Sekiz Kraliçe problemi için tüm olası çözümleri sayan ve tüm çözümleri gösteren bir program yazın.

MULTİDİMENSİYONAL ARRAYLER

Hedefler

- Veri temsilinin iki boyutlu diziler kullanarak örneklerini vermek (§7.1).
- İki boyutlu diziler için değişkenler tanımlamak, diziler oluşturmak ve iki boyutlu dizide satır ve sütun indekslerini kullanarak dizi elemanlarına erişmek (§7.2).
- İki boyutlu diziler için yaygın işlemleri programlamak (dizileri görüntüleme, tüm elemanları toplama, minimum ve maksimum elemanları bulma ve rastgele karıştırma) (§7.3).
- İki boyutlu dizileri yöntemlere geçirmek (§7.4).
- Çoktan seçmeli soruları notlandırmak için bir program yazmak (iki boyutlu diziler kullanarak (§7.5).
- İki boyutlu diziler kullanarak en yakın çift problemini çözmek (§7.6).
- İki boyutlu diziler kullanarak bir Sudoku çözümünü kontrol etmek (§7.7).
- Çok boyutlu dizileri kullanmak (§7.8).



7.1 Giriş



Bir tablo veya matris verileri, iki boyutlu bir dizi kullanılarak temsil edilebilir.

Önceki bölüm, bir boyutlu dizileri doğrusal eleman koleksiyonlarını depolamak için nasıl kullanacağınızı tanıttı. İki boyutlu bir dizi, bir matris veya tablo depolamak için kullanılabilir. Örneğin, şehirler arasındaki mesafeleri listeleyen aşağıdaki tablo, **distances** adında bir iki boyutlu dizi kullanılarak depolanabilir.

sorun

Distance Table (in miles)							
	Chicago	Boston	Detroit	Atlanta	Minneapolis	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
Detroit	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Minneapolis	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

```
iki boyutlu mesafeler = {n{t0,t983
t,t787,t714,t1375,t967,t1087},n{t983,t0
t,t214,t1102,t1763,t1723,t1842},n{t787,t2
14,t0,t888,t1549,t1548,t1627},n{t714,t1
102,t888,t0,t661,t781,t810},n{t1375,t
1763,t1549,t661,t0,t1426,t1187},n{t967,t1
723,t1548,t781,t1426,t0,t239},n{t1087,t
1842,t1627,t810,t1187,t239,t0},n};
```

7.2 İki Boyutlu Dizi Temelleri



İki boyutlu bir dizideki bir elemana satır ve sütun indeksi aracılığıyla erişilir.

İki boyutlu diziler için bir değişkeni nasıl tanımlarsınız? İki boyutlu bir dizi nasıl oluşturulur? İki boyutlu bir dizideki elemanlara nasıl erişilir? Bu bölüm bu konuları ele almaktadır.

7.2.1 İki Boyutlu Dizilerin Değişkenlerini Tanımlama ve İki Boyutlu Diziler Oluşturma

İki boyutlu bir diziyi tanımlamak için kullanılan sözdizimi:

```
elementType[] [] arrayRefVar;
```

veya

```
elementType arrayRefVar[] [] ; // İzin verilir, ancak tercih edilmez
```

Örnek olarak, int değerlerinden oluşan iki boyutlu bir dizi değişkeni `matrix`'i nasıl tanımlayacağınız:

```
int[] [] matrix;
```


veya

```
int matrix[][]; // Bu stil izinlidir, ancak tercih edilmez
```

5'e 5 **int** değerlerinden oluşan bir iki boyutlu dizi oluşturabilir ve bunu **matrix**

bu sözdizimi ile atayabilirsiniz:

```
matrix = new int[5][5];
```

İki boyutlu bir dizide iki alt indeks kullanılır, biri satır için diğeri ise sütun için.

Bir boyutlu dizide olduğu gibi, her alt indeksin indeksi **int** türündedir ve

0'dan başlar, Şekil 7.1a'da gösterildiği gibi.

	[0]	[1]	[2]	[3]	[4]
[0]	0				0
[1]	0				0
[2]	0				0
[3]	0				0
[4]	0				0

Matris = Tnu Ent[Takht][Takht]; matris[tat][t1t] = altında;

(a)

	[0]	[1]	[2]	[3]	[4]
[0]	0				0
[1]	0				0
[2]	0	7			0
[3]	0				0
[4]	0				0

(b)

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

intt[][] dizi = {n {t1t,t2
t,t3t},
{T4T,Takht,Tattat},
N {altında,Takht,Ta
qt},N {T10T,T11T,T12T}
N};

(c)

FIGURE 7.1 İki boyutlu bir dizinin her alt dizinin indeksi bir **int** değeridir ve 0'dan başlar.

Belirli bir öğeye 7 değerini atamak için, 2 satırında ve 1 sütununda, Şekil 7.1b'de gösterildiği gibi, aşağıdaki sözdizimini kullanabilirsiniz:

```
matris[tat][t1t] = altında;
```



Dikkat

2 satırındaki ve 1 sütunundaki öğeye erişmek için `matrix[2, 1]` kullanmak yaygın bir hatadır. Java'da, her alt dizin bir çift köşeli parantez içinde olmalıdır.

Ayrıca, iki boyutlu bir diziyi tanımlamak, oluşturmak ve başlatmak için bir dizi başlatıcısı kullanabilirsiniz. Örneğin, (a) bölümündeki aşağıdaki kod, belirtilen başlangıç değerleriyle bir dizi oluşturur, Şekil 7.1c'de gösterildiği gibi. Bu, (b) bölümündeki koda eşdeğerdir.

```
intt[][] dizi = {n {t1t,t2t  
,t3t},  
{T4T, Takht, Tatt}, N  
{under, Takht, Tatt},  
N {T10T, T11T, T12T}  
};
```

(a)

Eşdeğer

```
int[][] dizi = new int[4][3];  
dizi[0][0] = 1; dizi[0][1] = 2; dizi[0][2] = 3;  
dizi[1][0] = 4; dizi[1][1] = 5; dizi[1][2] = 6;  
dizi[2][0] = 7; dizi[2][1] = 8; dizi[2][2] = 9;  
dizi[3][0] = 10; dizi[3][1] = 11; dizi[3][2] = 12;
```

(b)

7.2.2 İki Boyutlu Dizilerin Uzunluklarını Elde Etme

İki boyutlu bir dizi, aslında her bir elemanının bir boyutlu dizi olduğu bir dizidir. Bir dizi **x**'in uzunluğu, dizideki eleman sayısıdır ve bu, **x.length** kullanılarak elde edilebilir. **x[0]**, **x[1]**, ..., ve **x[x.length-1]** dizileridir. Uzunlukları **x[0].length**, **x[1].length**, ..., ve **x[x.length-1].length** kullanılarak elde edilebilir.

Örneğin, `x = new int[3][4]` varsayalım, `x[0]`, `x[1]`, ve `x[2]` bir boyutlu dizilerdir ve her biri dört eleman içerir, Şekil 7.2'de gösterildiği gibi. `x.length` 'in değeri 3 'tür ve `x[0].length`, `x[1].length`, ve `x[2].length` 'in değeri 4 'tür.

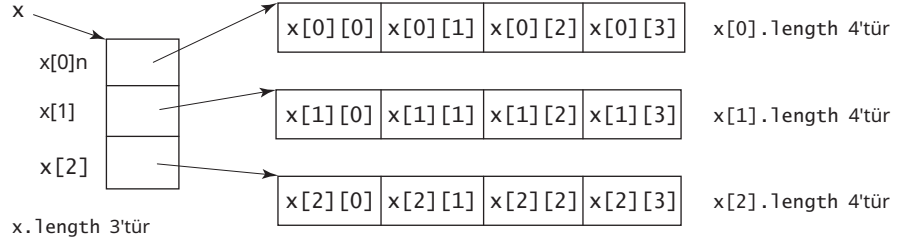
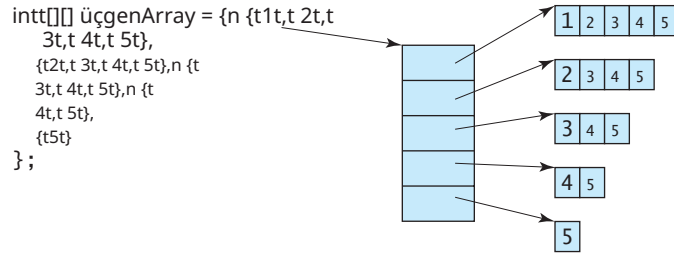


FIGURE 7.2 İki boyutlu bir dizi, her bir elemanın başka bir tek boyutlu dizi olduğu tek boyutlu bir dizidir.

7.2.3 Düzensiz Diziler

İki boyutlu bir dizideki her satır kendisi bir dizidir. Bu nedenle, satırlar farklı uzunlukta olabilir. Bu tür bir diziye *düzensiz dizi* denir. İşte bir düzensiz dizi oluşturma örneği:

dalgalı dizi



Gördüğünüz gibi, `triangleArray[0].length` 5'tir, `triangleArray[1].length` 4'tür, `triangleArray[2].length` 3'tür, `triangleArray[3].length` 2'dir ve `triangleArray[4].length` 1'dir.

Eğer bir ragged array'in değerlerini önceden bilmiyorsanız, ancak boyutlarını biliyorsanız—örneğin, da ha önceki gibi—aşağıdaki sözdizimini kullanarak bir ragged array oluşturabilirsiniz:

```
intt[][] üçgenArray = yeni intt[t5t][] ;
üçgenArray[t0t] = new intt[t5t]; ntriangleArray[t1t] = new intt[t4t]; ntriangleArray[t2t] = new intt[t3t]; ntriangleArray[t3t] = new intt[t2t]; ntriangleArray[t4t] = new intt[t1t];
```

Artık diziye değerler atayabilirsiniz. Örneğin,

```
triangleArray[0][3] = 50;
triangleArray[4][0] = 45;
```



Not

Bir dizi oluşturmak için `new int[5][]` sözdizimi, ilk indeksin belirtilmesini gerektirir. Sözdizimi `new int[][]` yanlış olurdu.

7.2 İki boyutlu bir dizideki satırlar farklı uzunluklara sahip olabilir mi?

7.3 Aşağıdaki kodun çıktısı nedir?

```
int[][] dizi = new int[5][6];
int[] x = {1, 2};
dizi[0] = x;
System.out.println("dizi[0][1] değeri " + dizi[0][1]);
```

7.4 Aşağıdaki ifadelerden hangileri geçerlidir?

```
intt[][] r = new intt[t2t];nint
t[] x = new intt[];nintt[][]
y = new intt[t3t[];nintt[][] z =
{{t1t,t2t}};nintt[][] m = {{t
1t,t2t}, {t2t,t3t}};nintt[][] n = {{t1t,t2
t}, {t2t,t3t}, };
```

7.3 İki Boyutlu Dizilerin İşlenmesi

İç içe for döngüleri genellikle iki boyutlu bir diziyi işlemek için kullanılır.

Diyelim ki bir dizi **matris** aşağıdaki gibi oluşturulmuştur:

```
int[][] matris = new int[10][10];
```

Aşağıda iki boyutlu dizilerin işlenmesine dair bazı örnekler bulunmaktadır.

1. *Kullanıcıdan alınan değerlerle dizileri başlatma.* Aşağıdaki döngü, diziyi kullanıcı

girişi değerleriyle başlatır:

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Satir sayısını girin: " + matrix.length + " ve " +
matrix[0].length + " sütun sayısını girin: ");
for(tinttrow = t0t; satır < matrix.length; satır++){
    fort(inttcolumn = t0 ; column < matrix[satır].length ; column++) {
        matris[satır][sütun] = input.nextInt(); N}
    }
```

2. *Rastgele değerlerle dizileri başlatma.* Aşağıdaki döngü, diziyi

0 ile 99 arasında rastgele değerlerle başlatır:

```
fort(inttrow = t0t; satır < matrix.length; satır++){
    fort(inttcolumn = t0 ; column < matrix[satır].length ; column++) {
        matris[satır][sütun] = (tintt)(Math.random() * t100t);n}
    }
```

3. *Dizileri yazdırma.* İki boyutlu bir diziyi yazdırmak için, dizideki her bir ögeyi aşağıdaki gibi bir döngü kullanarak yazdırmalısınız:

```
fort(inttrow = t0t; satır < matrix.length; satır++){
    için(int sütun = 0; sütun < matrix[satır].length ; sütun++){
        System.out.print(matrix[satır][sütun] + " ");
    }
    System.out.println();n}
```



4. *Tüm öğeleri toplama.* Toplamı saklamak için **total** adında bir değişken kullanın. Başlangıçta **total** 0. Dizi içindeki her bir öğeyi **total** 'a eklemek için aşağıdaki gibi bir döngü kullanın:

```
int toplam = 0;
için (int satir = 0; satir < matrix.length; satir++) {
    için (int sütun = 0; sütun < matrix[satir].length; sütun++) {
        total += matrix[satir][sütun];
    }
}
```

5. *Sütunlara göre öğeleri toplama.* Her sütun için, toplamını saklamak için **total** adında bir değişken kullanın. Sütundaki her bir öğeyi **total** 'a eklemek için aşağıdaki gibi bir döngü kullanın:

```
için (int sütun = 0; sütun < matrix[0].length; sütun++) {
    int total = 0;
    için (int satir = 0; satir < matrix.length; satir++)
        total += matrix[satir][sütun];
    System.out.println("Kolon için toplam " + column + " şudur "
        + total);
}
```

6. *Hangi satır en büyük toplamı var?* **maxRow** ve **indexOfMaxRow** değişkenlerini kullanarak büyük toplamı ve satırın indeksini takip et. Her satır için, toplamını hesapla ve yeni toplam daha büyükse **maxRow** ve **indexOfMaxRow** 'yi güncelle.

```
int maxRow = 0;
int indexOfMaxRow = 0;

// maxRow'daki ilk satırın toplamını al
for (int column = 0; column < matrix[0].length; column++) {
    maxRow += matrix[0][column];
}

for (int row = 1; row < matrix.uzunluk; row++) {
    int totalOfThisRow = 0;
    for (int column = 0; column < matrix[row].uzunluk; column++)
        totalOfThisRow += matrix[row][column];

    if (totalOfThisRow > maxRow) {
        maxRow = totalOfThisRow;
        indexOfMaxRow = row;
    }
}

System.out.println("Satır " + indexOfMaxRow
    + " maksimum toplamı " + maxRow);
```

7. *Rastgele karıştırma.* Tek boyutlu bir dizideki elemanları karıştırma, 6.2.6. Bölümde tanıtılmıştır. İki boyutlu bir dizideki tüm elemanları nasıl karıştırırsınız? Bunu başarmak için, her bir eleman **matrix[i][j]** için, rastgele indeksler **i1** ve **j1** oluşturun ve **matrix[i][j]** ile **matrix[i1][j1]** 'yi takas edin, aşağıdaki gibi:

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        int i1 = (int) (Math.random() * matrix.length);
        int j1 = (int) (Math.random() * matrix[i].length);

        // matrix[i][j] ile matrix[i1][j1] 'yi takas et
        int temp = matrix[i][j];
        matrix[i][j] = matrix[i1][j1];
        matrix[i1][j1] = temp;
    }
}
```



VideoNotu

En büyük toplamı olan satırı bulun

7.5 Aşağıdaki kodun çıktısını gösterin:

```
intt[][] dizi = {{t1t,t2t}, {t3t,t4t}, {t5t,t6t}};nfort(tint
ti = dizi.uzunluk - t1t; i >= t0t; i — ) {nfort(tinttj = dizi[i].
uzunluk - t1t; j >= t0t; j — )nSystem.out.print(array[i][j] +
t" ");
System.out.println();n}
```

PointttChtetck

MyProgrammingLab™

7.6 Aşağıdaki kodun çıktısını gösterin:

```
intt[][] dizi = {{t1t,t2t}, {t3t,t4t}, {t5t,t6t}};inttsum =
t0t;
fort(tintti = t0t; i < dizi.uzunluk; i++) nsum += diz
i[i][t0t];
System.out.println(toplam);
```

7.4 İki Boyutlu Dizileri Yöntemlere Geçirme

İki boyutlu bir diziyi bir yönteme geçirirken, dizinin referansı yönteme geçirilir.



İki boyutlu bir diziyi, bir boyutlu bir diziyi geçirdiğiniz gibi bir yönteme geçirebilirsiniz. Bir yöntemden bir dizi de döndürebilirsiniz. Liste 7.1, iki yöntemle bir örnek vermektedir. İlk yöntem, **getArray()**, iki boyutlu bir dizi döndürür ve ikinci yöntem, **sum(int[][] m)**, bir matrisin tüm elemanlarının toplamını döndürür.

LİSTE 7.1 PassTwoDimensionalArray.java

```
1  import java.util.Scanner;
2
3  public class PassTwoDimensionalArray {
4      public static void main(String[] args) {
5          int[][] m = getArray(); // Bir dizi al
6
7          // Elemanların toplamını göster
8          System.out.println("Tüm elemanların toplamı "9 } + toplam(m));
9
10
11     genel statik intt[][] getArray() {
12         // Bir Scanner oluştur
13         Tarayıcı girişi = tnewScanner(System.in);
14
15         // Dizi değerlerini girin
16         intt[][] m = tnew intt[t3t][t4t];
17         System.out.println(" " + m.length + " satır ve "
18             + m[0].length + " sütun girin: ");
19         fort(tint = t0t; i < m.uzunluk; i++)
20             fort(tinttj = t0t; j < m[i].uzunluk; j++)
21                 m[i][j] = input.nextInt();
22
23         geri dönüş;
24     }
25
26     public static inttsum(tintt[][] m) {
27         intttoplam = t0t;
28         fort(tinttrow = t0t; satır < m.uzunluk; satır++) {
29             fort(tinttcolumn = t0t; sütun < m[satır].uzunluk; sütun++) {
30                 toplam += m[satır][sütun];n31 }
```

dizi al

dizi geçir

getArray metodu

dizi döndür

sum metodu

270 Bölüm 7 Çok Boyutlu Diziler

```
32     }
33
34     dönüştoplam;n3
5 }
36 }
```



3 satır ve 4 sütun girin:

1	2	3	4	Enter
5	6	7	8	Enter
9	10	11	12	Enter

Tüm elemanların toplamı 78'dir

Yöntem **getArray** kullanıcıdan dizi için değerler girmesini ister (satırlar 11–24) ve diziye döndürür (satır 23).

Yöntem **sum** (satırlar 26–35) iki boyutlu bir dizi argümanına sahiptir. Satır sayısını **m.1 length** (satır 28) kullanarak ve belirli bir satırdaki sütun sayısını **m[row].1 length** (satır 29) kullanarak elde edebilirsiniz.

7.7 Aşağıdaki kodun çıktısını gösterin:

PointttChtetck

MyProgrammingLab™

```
genel sınıfTest {
    public static void tmain(String[] args) {nintt[][] dizi =
        {{t1t,t2t,t3t,t4t}, {t5t,t6t,t7t,t8t}};nSystem.out.println(m1(
        array)[t0t]);
        System.out.println(m1(array)[t1t]);n}

    public static int[]tm1(tintt[][] m) {nintt[] resul
        t = tnew intt[t2t];
        sonuç[t0t] = m.uzunluk;ns
        onuç[t1t] = m[t0t].uzunluk;
        dönüş sonucu;n}

}
```

7.5 Vaka Çalışması: Çoktan Seçmeli Bir Testin Notlandırılması



Ana
Nokta

Problem, çoktan seçmeli testleri notlandıracak bir program yazmaktır.

Diyelim ki çoktan seçmeli testleri notlandıracak bir program yazmanız gerekiyor. Sekiz öğrencinin ve on sorunun olduğunu varsayın ve cevaplar iki boyutlu bir dizide saklanmaktadır. Her satır, bir öğrencinin sorulara verdiği cevapları kaydeder; aşağıdaki dizide gösterildiği gibi.



VideoNotu

Çoktan seçmeli testi notlandırma

Öğrencilerin Sorulara Verdiği Cevaplar:

0 1 2 3 4 5 6 7 8 9

Öğrenci 0	A	B	A	C	C	D	E	E	A	D
Öğrenci 1	D	B	A	B	C	A	E	E	A	D
Öğrenci 2	E	D	D	A	C	B	E	E	A	D
Öğrenci 3	C	B	A	E	D	C	E	E	A	D
Öğrenci 4	A	B	D	C	C	D	E	E	A	D
Öğrenci 5	B	B	E	C	C	D	E	E	A	D
Öğrenci 6	B	B	A	C	C	D	E	E	A	D
Öğrenci 7	E	B	E	C	C	D	E	E	A	D

Anahtar, tek boyutlu bir dizide saklanır:

Sorulara Anahtar:

0 1 2 3 4 5 6 7 8 9

Anahtar D B D C C D A E A D

Programınız testi notlandırır ve sonucu gösterir. Her öğrencinin cevaplarını anahtar ile karşılaştırır, doğru cevapların sayısını sayar ve gösterir. Liste 7.2 programı vermektedir.

LISTE 7.2 GradeExam.java

```

1  genel sınıfSınıfSınavı {
2  /** Ana yöntem */
3  public static voidtmain(String[] args) {
4      // Öğrencilerin sorulara verdiği cevaplar
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19  // Soruların anahtarı
20  grafik[] tuşları = {t'D't,t'B't,t'D't,t'C't,t'C't,t'D't,t'E't,t'E't,t'A't,t'D't};
21
22  // Tüm cevapları notlandır
23  kale(renk tonu = t0; i < yanıtlar.uzunluk ; i++) {
24      // Bir öğrenciyi notlandır
25      inttcorrectCount = t0t;
26      kale(tinttj = t0 ; vaplar[i].uzunluk ; j++) {
27          ift() cevaplar[i][j] == tuşlar[j]nj < ce
28      }
29  }
30
31  doğruSayı++;n25 }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

2-D dizi

1-D dizi

anahtar ile karşılaştır

kesin



Öğrenci 0'ın doğru sayısı 7
 Öğrenci 1'in doğru sayısı 6
 Öğrenci 2'nin doğru sayısı 5
 Öğrenci 3'ün doğru sayısı 4
 Öğrenci 4'ün doğru sayısı 8
 Öğrenci 5'in doğru sayısı 7
 Öğrenci 6'nın doğru sayısı 7
 Öğrenci 7'nin doğru sayısı 7

5–13. satırlardaki ifade, karakterlerin iki boyutlu bir dizisini tanımlar, oluşturur ve başlatır ve `char[][]` türündeki `answers` referansını atar.

16. satırdaki ifade, bir dizi `char` değerini bildirir, oluşturur ve başlatır ve `char` türündeki `keys` referansını atar.

Dizideki her **answers** satırı, bir öğrencinin cevabını saklar ve bu cevap, dizideki **keys** ile karşılaştırılarak puanlanır. Sonuç, bir öğrencinin cevabı puanlandıktan hemen sonra görüntülenir.

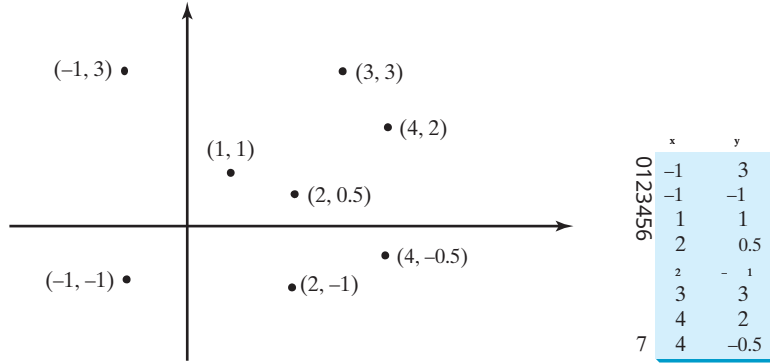
7.6 Vaka Çalışması: En Yakın Çifti Bulma

Bu bölüm, en yakın nokta çiftini bulma ile ilgili bir geometrik problemi sunmaktadır.

Bir nokta kümesi verildiğinde, en yakın çift problemi, birbirine en yakın iki noktayı bulmaktır. Örneğin, Şekil 7.3'te, **(1, 1)** ve **(2, 0.5)** noktaları birbirine en yakındır. Bu problemi çözmenin birkaç yolu vardır. Sezgisel bir yaklaşım, tüm nokta çiftleri arasında ki mesafeleri hesaplamak ve minimum mesafeye sahip olanı bulmaktır; bu, Liste 7.3'te uygulanmıştır.



en yakın çift animasyonu
Eşlik Eden Web Sitesi'nde



ŞEKİL 7.3 Noktalar iki boyutlu bir dizide temsil edilebilir.

LISTE 7.3 FindNearestPoints.java

```

1  import java.util.Scanner;
23
   genel classtFindNearestPoints {
4      public static voidtmain(String[] args) {
5          Scanner input = newScanner(System.in);
6          System.out.print("Kaç nokta gireceksiniz: ");
7          intnumberOfPoints = input.nextInt();
89
          // Noktaları saklamak için bir dizi oluştur
10         doublet[][] puan = tnew doublet[numberOfPoints][t2t];
11 System.out.print(t"Enter "t+ numberOfPoints + t" point: "t);
12         fort(tintti = t0t; i < puan.uzunluk; i++) {
13 puan[i][t0t] = input.nextDouble();n14 puan[i][t1t] = in
put.nextDouble();n15 }
16
17         // p1 ve p2, points dizisindeki indekslerdir
18         t, p2 = t1t; // İlk iki nokta
doubletshortestDistance = mesafe(noktalar[p1][t0t], noktalar[p1][t1t],ninttp1 = t0
puan[p2][t0t], puan[p2][t1t]); // shortestDistance'ı başlat
19
20         // Her iki nokta için mesafeyi hesapla
21
22         fort(tintti = t0t; i < puan.uzunluk; i++) {
23

```

nokta sayısı

2-B dizisi

nokta oku

iki noktayı takip et
kısaMesafe

her nokta için i


```

24     fort(inttj = i + 1; j < puan.uzunluk; j++) {
25         double distance = distance(points[i][0], points[i][1],
26             points[j][0], points[j][1]); // Uzaklığı bul
27
28         if(enKısaUzaklık > uzaklık) {
29             p1 = i; // p1'i güncelle
30             p2 = j; // p2'yi güncelle
31             shortestDistance = distance; // shortestDistance'i güncelle
32         }
33     }n34
34 }
35
36 // Sonucu göster
37 System.out.println("En yakın iki nokta "+
38     "(" + points[p1][0] + ", " + points[p1][1] + ") ve (" +
39     puan[p2][t0t] + ", " + puan[p2][t1t] + ")");n40 }
41
42 /** İki nokta (x1, y1) ve (x2, y2) arasındaki uzaklığı hesapla */
43 public static double uzaklık(
44     double x1, double y1, double x2, double y2) {
45     return Math.sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
46 }n47 }

```

her nokta için j

iki nokta arasındaki uzaklık i ve j

iki nokta arasındaki uzaklığı hesapla

güncelle enKısaUzaklık

```

Noktaların sayısını girin: 8 ↵ Enter
8 nokta girin: -1 3 -1 -1 1 1 2 0.5 2 -1 3 3 4 2 4 -0.5 ↵ Enter
En yakın iki nokta (1, 1) ve (2, 0.5)

```



Program, kullanıcının nokta sayısını girmesini ister (satırlar 6–7). Noktalar, komut satırından okunur ve **points** adlı iki boyutlu bir dizide saklanır (satırlar 12–15).

Program, en yakın iki nokta arasındaki uzaklığı saklamak için **shortestDistance** (satır 19) değişkenini kullanır ve bu iki noktanın **points** dizisindeki indeksleri **p1** ve **p2** (satır 18) içinde saklanır.

Her **i** indeksindeki nokta için, program **points[i]** ile **points[j]** arasındaki uzaklığı hesaplar ve tüm **j > i** için (satırlar 23–34). Daha kısa bir uzaklık bulunduğunda, **shortestDistance** ve **p1** ile **p2** güncellenir (satırlar 28–32).

İki nokta arasındaki uzaklık (x_1, y_1) ve (x_2, y_2) formülü (satırlar 43–46) kullanılarak hesaplanabilir.

Program, düzlemin en az iki noktaya sahip olduğunu varsayıyor. Düzlemin sıfır veya bir noktaya sahip olması durumunu ele almak için programı kolayca değiştirebilirsiniz.

Not edin ki, aynı minimum uzaklığa sahip birden fazla en yakın nokta çifti olabilir. Program, böyle bir çifti bulur. Tüm en yakın çiftleri bulmak için programı Programlama Egzersizi 7.8'de değiştirebilirsiniz.

birden fazla en yakın çift



Tipp

Klavyeden tüm noktaları girmek zahmetlidir. Girdiyi bir dosyada saklayabilirsiniz, örneğin **FindNearestPoints.txt**, ve programı aşağıdaki komutla derleyip çalıştırabilirsiniz:

girdi dosyası

```
java FindNearestPoints < FindNearestPoints.txt
```

7.7 Vaka Çalışması: Sudoku



Problem, verilen bir Sudoku çözümünün doğru olup olmadığını kontrol etmektir.



VideoNotu
Sudoku

sabit hücreler

serbest hücreler

Bu bölüm, her gün gazetelerde yer alan ilginç bir problemi sunmaktadır. Bu, genellikle *Sudoku* olarak bilinen bir sayı yerleştirme bulmacasıdır. Bu çok zor bir problemdir. Acemi için erişilebilir hale getirmek amacıyla, bu bölüm, bir çözümün doğru olup olmadığını doğrulamak için Sudoku probleminin basitleştirilmiş bir versiyonuna bir çözüm sunmaktadır. Sudoku problemini çözmek için tam çözüm Ek VI.A'da sunulmuştur.

Sudoku bir 9×9 ızgara daha küçük 3×3 kutulara (aynı zamanda *bölgeler* veya *bloklar* olarak da adlandırılır),

Şekil 7.4a'da gösterildiği gibi. Bazı hücreler, *sabit hücreler* olarak adlandırılır ve 1 ile 9 arasındaki sayılarla doldurulmuştur.

Amaç, 1 ile 9 arasındaki sayılarla doldurulmuş boş hücreleri, *serbest hücreler* olarak adlandırılan, doldurmaktır, böylece her satır, her sütun ve her 3×3 kutuda 1 ile 9 arasındaki sayılar bulunur, Şekil 7.4b'de gösterildiği gibi.

5	3			7				
6			1	9	5			
	9	8					6	
8				8				6
4			8		4			8
7				7				2
	6							
			4	1	9			5
				8			8	

(a) Bulmaca

Çözüm

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

(b) Çözüm

ŞEKİL 7.4 Sudoku bulmacası (a) kısmında çözülmüştür (b) kısmında.

bir ızgarayı temsil etmek

Kolaylık olması açısından, serbest bir hücreyi belirtmek için değer 0 kullanıyoruz, Şekil 7.5a'da gösterildiği gibi. ızgara doğal olarak iki boyutlu bir dizi kullanılarak temsil edilebilir, Şekil 7.5b'de gösterildiği gibi.

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8		0	0		6	
8	0	0	0	8	0	8		6
4	0	0	8	0	4	8		8
7	8		0	7	0	8		2
0	6	0	0	0	0	8		0
0	0	0	4	1	9	0	0	5
0	0	0	0	8	0		7	9

(a)

(b) Çözüm

```

intt[9][9] ızgara =
{
    {t5,t 3,t 0,t 0,t 7,t 0,t 0,t 0,t 0},n {t
    6,t 0,t 0,t 1,t 9,t 5,t 0,t 0,t 0},n {t
    0,t 9,t 8,t 0,t 0,t 0,t 0,t 6,t 0},n {t
    8,t 0,t 0,t 0,t 8,t 0,t 0,t 0,t 6},n {t
    4,t 0,t 0,t 8,t 0,t 4,t 8,t 0,t 8},n {t
    7,t 8,t 0,t 7,t 0,t 8,t 0,t 2},n {t
    0,t 6,t 0,t 0,t 0,t 0,t 8,t 0,t 0},n {t
    0,t 0,t 0,t 4,t 1,t 9,t 0,t 0,t 5},n {t
    0,t 0,t 0,t 0,t 8,t 0,t 7,t 9,t 0};

```

(b)

ŞEKİL 7.5 İki boyutlu bir dizi kullanılarak bir ızgara temsil edilebilir.

Bulmacanın çözümünü bulmak için, 0'ı 1 ile 9 arasındaki uygun bir sayı ile değiştiririz. Şekil 7.5'teki bulmacanın çözümü için, ızgara Şekil 7.6'da gösterildiği gibi olmalıdır.

Bir Sudoku bulmacasının çözümü bulunduğunda, bunun doğru olduğunu nasıl doğrularsınız? İşte iki yaklaşım:

- Her satırın 1 ile 9 arasındaki sayılara, her sütunun 1 ile 9 arasındaki sayılara ve her küçük kutunun 1 ile 9 arasındaki sayılara sahip olup olmadığını kontrol edin.

- Her hücreyi kontrol et. Her hücre 1 ile 9 arasında bir sayı olmalı ve hücre her satırda, her sütunda ve her küçük kutuda benzersiz olmalıdır.

Bir çözüm ızgarası

```
{ {5, 3, 4, 6, 7, 8, 9, 1, 2},
  {6, 7, 2, 1, 9, 5, 3, 4, 8},
  {1, 9, 8, 3, 4, 2, 5, 6, 7},
  {8, 5, 9, 7, 6, 1, 4, 2, 3},
  {4, 2, 6, 8, 5, 3, 7, 9, 1},
  {7, 1, 3, 9, 2, 4, 8, 5, 6},
  {9, 6, 1, 5, 3, 7, 2, 8, 4},
  {2, 8, 7, 4, 1, 9, 6, 3, 5},
  {3, 4, 5, 2, 8, 6, 1, 7, 9}
};
```

ŞEKİL 7.6 Bir çözüm **grid** içinde saklanır.

Liste 7.4'teki program, kullanıcıdan bir çözüm girmesini ister ve bunun geçerli olup olmadığını bildirir. Programda çözümün doğru olup olmadığını kontrol etmek için ikinci yaklaşımı kullanıyoruz.

LİSTE 7.4 CheckSudokuSolution.java

```
1  import java.util.Scanner;
2
3  genel classtCheckSudokuSolution {
4      public static voidtmain(String[] args) {
5          // Bir Sudoku çözümünü oku
6          intt[][] grid = readASolution();
7
8          System.out.println((geçerli(ızgara) ? "Geçerli çözüm":
9              "Geçersiz çözüm");
10     }
11
12     /** Konsoldan bir Sudoku çözümünü oku */
13     genel statik intt[][] readASolution() {
14         // Bir Scanner oluştur
15         Tarayıcı girişi = tnewtScanner(System.in);
16
17         System.out.println("Bir Sudoku bulmaca çözümü girin:");
18         intt[][] grid = tnew intt[t9t][t9t];
19         fort(tint = t0t; i < t9t; i++)
20             fort(tinttj = t0t; j < t9t; j++)
21                 grid[i][j] = input.nexttInt();
22
23         returntgrid;n2
24     }
25
26     /** Bir çözümün geçerli olup olmadığını kontrol et */
27     public static booleantisValid(tintt[][] grid) {
28         fort(tint = t0t; i < t9t; i++)
29             fort(tinttj = t0t; j < t9t; j++)
30                 ift(ızgara[i][j] < t1t | ızgara[i][j] > t9
31                     | !geçerliMi(i, j, grid))
32                     falset'i döndür;
33         return true; // Çözüm geçerli
34     }
35
36     /** grid[i][j]'nin grid içinde geçerli olup olmadığını kontrol et */
37     public static booleantisValid(tintti,tinttj,tintt[][] grid) {
38         // grid[i][j]'nin i'nin satırında geçerli olup olmadığını kontrol et
```

girdi oku

çözüm geçerli mi?

çözümü oku

çözümü kontrol et

276 Bölüm 7 Çok Boyutlu Diziler

```

39         0 için (int column = 0; column < 9; column++)
40             eğer(sütun != j && grid[i][sütun] == grid[i][j])
41                 falset'i döndür;
42
43 // grid[i][j]'nin j sütununda geçerli olup olmadığını kontrol et
44         0 için (int row = 0; row < 9; row++)
45             eğer(satır != i && grid[satır][j] == grid[i][j])
46                 falset'i döndür;
47
48 // grid[i][j]'nin 3'e 3 kutuda geçerli olup olmadığını kontrol et
49         i / 3 * 3 için (int row = (i / 3) * 3; row < (i / 3) * 3 + 3; row++)
50             için (int col = (j / 3) * 3; col < (j / 3) * 3 + 3; col++)
51                 eğer(satır != i && col != j && grid[satır][col] == grid[i][j])
52                     falset'i döndür;
53
54         doğru döndür; // grid[i][j]'deki mevcut değer geçerli
55     }
56 }

```



Bir Sudoku bulmaca çözümünü girin:

9	6	3	1	7	4	2	5	8
1	7	8	3	2	5	6	4	9
2	5	4	6	8	9	7	3	1
8	2	1	4	3	7	5	9	6
4	9	6	8	5	2	3	1	7
7	3	5	9	6	1	8	2	4
5	8	9	7	1	3	4	6	2
3	1	7	2	4	6	9	8	5
6	4	2	5	9	8	1	7	3

Geçerli çözüm

Program, bir Sudoku çözümünü okumak ve bir Sudoku ızgarasını temsil eden iki boyutlu bir dizi döndürmek için readASolution() yöntemini (satır 6) çağırır.

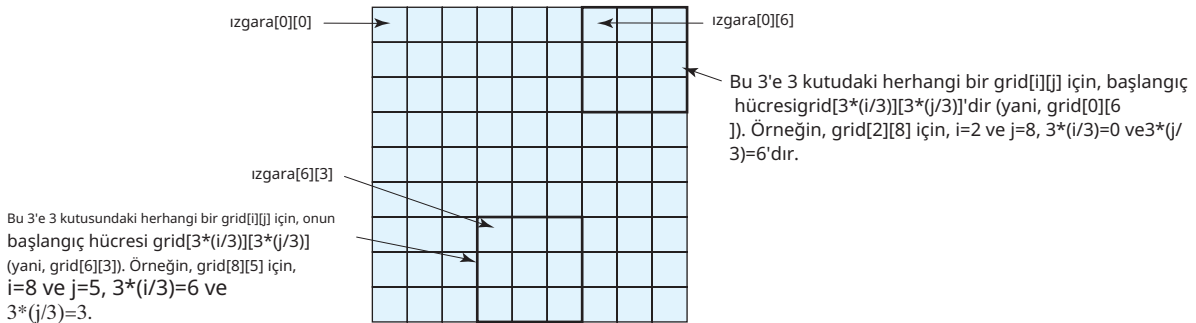
isValid yöntemi

isValid(grid) yöntemi, ızgaradaki değerlerin geçerli olup olmadığını kontrol eder; her değer 1 ile 9 arasında olduğunu ve her değer ızgarada geçerli olduğunu doğrular (satırlar 27-34).

overloaded isValid yöntemi

isValid(i, j, grid) yöntemi, grid[i][j] değerinin geçerli olup olmadığını kontrol eder. grid[i][j]'nin satır i'de (satırlar 39-41), sütun j'de (satırlar 44-46) ve 3 × 3 kutu (satırlar 49-52).

Aynı kutudaki tüm hücreleri nasıl bulursunuz? Herhangi bir grid[i][j] için, başlangıç hücre 3 × 3 içeren kutu grid[(i / 3) * 3][(j / 3) * 3], aşağıda gösterildiği gibi Şekil 7.7.



FGÖRÜNTÜ 7.7 İlk hücrenin konumu bir 3 × 3 kutucuk, kutudaki diğer hücrelerin konumlarını belirler.

Bu gözlemlerle, kutudaki tüm hücreleri kolayca tanımlayabilirsiniz. Örneğin, eğer

`grid[r][c]` bir kutunun başlangıç hücresidir, kutudaki hücreler a içinde geçilebilir 3×3 kutu, kutudaki hücreler bir iç içe döngü aşağıdaki gibi:

```
// grid[r][c] noktasından başlayarak 3'e 3'lük bir kutudaki tüm hücreleri al
for(int row = r; row < r + 3; row++)
    for(int col = c; col < c + 3; col++)
        // grid[row][col] kutunun içindedir
```

81 sayıyı konsoldan girmek zahmetlidir. Programı test ettiğinizde, girişi bir dosyada saklayabilirsiniz, örneğin **CheckSudokuSolution.txt** (bkz www.cs.armstrong.edu/liang/data/CheckSudokuSolution.txt), ve programı aşağıdaki komutla çalıştırabilirsiniz:

```
java CheckSudokuSolution < CheckSudokuSolution.txt
```

girdi dosyası

7.8 Çok Boyutlu Diziler

İki boyutlu bir dizi, bir boyutlu dizilerin bir dizisinden oluşur ve üç boyutlu bir dizi, iki boyutlu dizilerin bir dizisinden oluşur.



Önceki bölümde, bir matris veya tabloyu temsil etmek için iki boyutlu bir dizi kullandınız. Bazen, n -boyutlu veri yapıları temsil etmeniz gerekecektir. Java'da, herhangi bir tamsayı n için n -boyutlu diziler oluşturabilirsiniz.

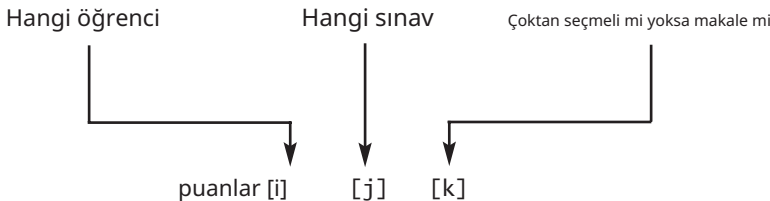
İki boyutlu dizi değişkenlerini tanımlama ve iki boyutlu diziler oluşturma yöntemi, n -boyutlu dizi değişkenlerini tanımlamak ve n -boyutlu diziler oluşturmak için genelleştirilebilir. Örneğin, altı öğrencinin beş sınav için sınav notlarını saklamak üzere üç boyutlu bir dizi kullanabilirsiniz ve her sınavın iki bölümü vardır (çoktan seçmeli ve kompozisyon). Aşağıdaki sözdizimi, bir üç boyutlu dizi değişkeni **scores** tanımlar, bir dizi oluşturur ve referansını **scores**'ye atar.

```
ikili[][] puanlar = yeni ikili[t6t][t5t][t2t];
```

Ayrıca, diziyi aşağıdaki gibi oluşturmak ve başlatmak için kısayol notasyonunu da kullanabilirsiniz:

```
ikili[][] puanlar = {
    {{t7.5t,t20.5t}, {t9.0t,t22.5t}, {t15t,t33.5t}, {t13t,t21.5t}, {t15t,t2.5t}}, n{{t4.5t,t21.5t},
    {t9.0t,t22.5t}, {t15t,t34.5t}, {t12t,t20.5t}, {t14t,t9.5t}}, n{{t6.5t,t30.5t}, {t9.4t,t10.5t},
    {t11t,t33.5t}, {t11t,t23.5t}, {t10t,t2.5t}}, n{{t6.5t,t23.5t}, {t9.4t,t32.5t}, {t13t,t34.5t},
    {t11t,t20.5t}, {t16t,t7.5t}}, n{{t8.5t,t26.5t}, {t9.4t,t52.5t}, {t13t,t36.5t}, {t13t,t24.5t},
    {t16t,t2.5t}}, n{{t9.5t,t20.5t}, {t9.4t,t42.5t}, {t13t,t31.5t}, {t12t,t20.5t}, {t16t,t6.5t}}};
```

scores[0][1][0], birinci öğrencinin ikinci sınavındaki çoktan seçmeli puanı ifade eder, ki bu **9.0**'dir. **scores[0][1][1]**, birinci öğrencinin ikinci sınavındaki makale puanını ifade eder, bu da **22.5**'dir. Bu, aşağıdaki şekilde gösterilmiştir:



Çok boyutlu bir dizi, aslında her bir elemanın başka bir dizi olduğu bir dizidir. Üç-dimensional bir dizi, iki boyutlu dizilerin bir dizisinden oluşur. İki boyutlu bir dizi, bir boyutlu dizilerin bir dizisinden oluşur. Örneğin, **x = new int[2][2][5]** varsayalım, ve **x[0]** ile **x[1]** iki boyutlu dizilerdir. **x[0][0]**, **x[0][1]**, **x[1][0]** ve **x[1][1]** bir boyutlu dizilerdir ve her biri beş eleman içerir.

`x.length2`'dir, `x[0].length` `vex[1].length2`'dir, ve `X[0][0].length`, `x[0][1].length`, `x[1][0].length` ve `x[1][1].length` 5'dir.

7.8.1 Vaka Çalışması: Günlük Sıcaklık ve Nem

Diyelim ki bir meteoroloji istasyonu her gün her saatte sıcaklık ve nemi kaydediyor ve verileri son on gün için **Weather.txt** adlı bir metin dosyasında saklıyor (bkz. www.cs.armstrong.edu/liang/data/Weather.txt). Dosyanın her satırı, günü, saati, sıcaklığı ve nemi belirt en dört sayıdan oluşmaktadır. Dosyanın içeriği (a) da olduğu gibi görünebilir.

Gün	Saat	Sıcaklık	Nem
1	1	76.4	0.92
1	2	77.7	0.93
...			
10	23	97.7	0.71
10	24	98.7	0.74

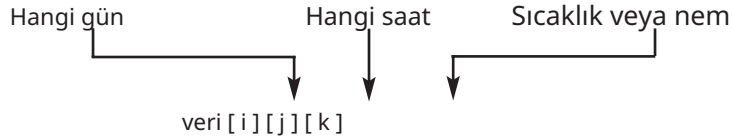
(a)

Gün	Saat	Sıcaklık	Nem
10	24	98.7	0.74
1	2	77.7	0.93
...			
10	23	97.7	0.71
1	1	76.4	0.92

(b)

Dosyadaki satırların gün ve saat açısından artan sırada olmayabileceğini unutmayın. Örneğin, dosya (b) şeklinde görünebilir.

Göreviniz, 10 gün için ortalama günlük sıcaklık ve nemi hesaplayan bir program yazmak tır. Girdi yönlendirmesini kullanarak dosyayı okuyabilir ve verileri `data` adlı üç boyutlu bir diziye saklayabilirsiniz. `data`'nın ilk indeksi 0'dan 9'a kadar uzanır ve 10 günü temsil eder, ikinci indeks 0'dan 23'e kadar uzanır ve 24 saati temsil eder, üçüncü indeks ise 0'dan 1'e kadar uzanır ve sıcaklık ile nemi temsil eder, aşağıdaki şekilde gösterildiği gibi:



Günlerin 1 ile 10 arasında ve saatlerin 1 ile 24 arasında numaralandırıldığını unutmayın. Dizi indeksi 0'dan başladığı için, `data[0][0][0]` gün 1'deki sıcaklığı saat 1'de saklar ve `data[9][23][1]` gün 10'daki nemi saat 24'te saklar.

Program, Liste 7.5'te verilmiştir.

LİSTE 7.5 Weather.java

```

1  import java.util.Scanner;
23
   genel sınıf HavaDurumu {
4      public static void main(String[] args) {
5          final int NUMBER_OF_DAYS = 10;
6          final int NUMBER_OF_HOURS = 24;
789         double[][][] data
           = new double[GÜN_SAYISI][SAAT_SAYISI][2];

10     Tarayıcı girişi = new Scanner(System.in);
11     // Bir dosyadan girdi yönlendirmesi kullanarak girişi okuyun
12     for (int t = 0; t < NUMBER_OF_DAYS * NUMBER_OF_HOURS; t++) {
13         int day = input.nextInt();
14         int saat = input.nextInt();
15         double ısı = input.nextDouble();
16         double nem = input.nextDouble();
17         data[gün - 1][saat - 1][0] = ısı;

```

üç boyutlu dizi

```

18     data[day - 1][hour - 1][1] = nem;
19 }
20
21 // Günlük ortalama sıcaklık ve nemi bul
22 for(int i = 0; i < GÜN_SAYISI; i++) {
23     doubletdailySıcaklıkToplam = t0t, günlükNemToplam = t0t;
24     for(int j = 0; j < SAAT_SAYISI; j++) {
25         dailyTemperatureTotal += data[i][j][t0t];
26         dailyHumidityTotal += data[i][j][t1t];
27     }
28
29 // Sonucu göster
30 System.out.println("Gün " + i + "in ortalama sıcaklığı, "
31 + dailyTemperatureTotal / SAAT_SAYISI);
32 System.out.println("Gün " + i + "in ortalama nemi "
33 + dailyHumidityTotal / SAAT_SAYISI);
34

```

```

Gün 0'in ortalama sıcaklığı 77.7708
Gün 0'in ortalama nemi 0.929583
Gün 1'in ortalama sıcaklığı 77.3125
Gün 1'in ortalama nemi 0.929583
. . .
Gün 9'un ortalama sıcaklığı 79.3542
Gün 9'un ortalama nemi 0.9125

```



Programı çalıştırmak için aşağıdaki komutu kullanabilirsiniz:

```
java Hava Durumu < Hava Durumu.txt
```

Sıcaklık ve nemi depolamak için üç boyutlu bir dizi, 8. satırda oluşturulmuştur. 12-19. satırlarda ki döngü, dizinin girdi verilerini okur. Girdiyi klavyeden girebilirsiniz, ancak bu rahatsız edici olacaktır. Kolaylık olması açısından verileri bir dosyada saklıyoruz ve dosyadan verileri okumak için gir di yönlendirmesi kullanıyoruz. 24-27. satırlardaki döngü, her gün için her saat için tüm sıcaklıkları **dailyTemperatureTotal**'ye ve her saat için tüm nemi **dailyHumidityTotal**'ye ekler. Ortalama günlük sıcaklık ve nem, 30-33. satırlarda görüntülenir.

7.8.2 Vaka Çalışması: Doğum Günlerini Tahmin Etme

Liste 3.3, `GuessBirthDay.java`, bir doğum gününü tahmin eden bir programı verir. Program, sa yıları üç boyutlu bir dizide beş kümede saklayarak basitleştirilebilir ve kullanıcıdan cevapları bi r döngü kullanarak istemektedir; bu, Liste 7.6'da gösterilmiştir. Programın örnek çalışması, Lis te 3.3'te gösterilenle aynı olabilir.

LtISTINGt 7.6tGuessDoğum GünüUsingArray.java

```

1  import java.util.Scanner;
2
3  public class GuessDoğum GünüUsingArray {
4      public static void tmain(String[] args) {
5          int day = 0; // Belirlenmesi gereken gün
6          int cevabi;
7
8          arihler = {
9              { t1t, 3, 5, 7t},
10             { t9t,t11t,t13t,t15t},nintt[] t

```

üç boyutlu dizi

```

11 {t17t,t19t,t21t,t23t},n12 {t25t,t
27t,t29t,t31t}},n13 {{ t2t,t3t,t6t,t7t}
,n14 {t10t,t11t,t14t,t15t},n15 {t18t
,t19t,t22t,t23t},n16 {t26t,t27t,t30t,
t31t}},n17 {{t4t,t5t,t6t,t7t}},n18 {t1
2t,t13t,t14t,t15t},n19 {t20t,t21t,t22
t,t23t},n20 {t28t,t29t,t30t,t31t}},n
21 {{t8t,t9t,t10t,t11t},n22 {t12t,t13
t,t14t,t15t},n23 {t24t,t25t,t26t,t27
t},n24 {t28t,t29t,t30t,t31t}},n25 {{t1
6t,t17t,t18t,t19t},n26 {t20t,t21t,t2
2t,t23t},n27 {t24t,t25t,t26t,t27t},n
28 {t28t,t29t,t30t,t31t}}};

```

Ben'i ayarla

```

29
30 // Bir Scanner oluştur
31 Tarayıcı girişi = tnewtScanner(System.in);
3233
34     fort(tint = t0t; i < t5t; i++) {
35         System.out.println("Doğum gününüz Set içinde mi" + (i + 1) + "?");
36         güçlü(tinttj = t0t; j < t4t; j++) {
37             için(intk = 0; k < 4; k++)
38 System.out.printf(t"%4d"n38 System.out.pr , tarihler[i][j][k] );
39 intln();n39 }
40
41 Sistem.çıkışı.baskı( "Hayır için 0 ve Evet için 1 girin: ");
42 cevap = input.nextInt();
43
44     eğer(answer == 1)
45     gün += tarihler[i][t0t][t0t];
46     }
47
48     System.out.println("Sizin doğum gününüz " + gün);
49 }
50 }

```

güne ekle

8–28. Satırlarda bir üç boyutlu dizi **dates** oluşturulmuştur. Bu dizi beş set sayı saklar. Her set 4'e 4 boyutunda bir iki boyutlu dizidir.

Döngü, 33. satırdan başlayarak her kümedeki sayıları gösterir ve kullanıcıdan doğum gününün kümede olup olmadığını yanıtlamasını ister (41-42. satırlar). Eğer gün kümede ise, kümedeki ilk sayı (**dates[i][0][0]**) değişkenine **day** (45. satır) eklenir.

7.8 Üç boyutlu bir dizi için bir dizi değişkeni tanımlayın, bir $4 \times 6 \times 5$ **int** dizisi oluşturun ve referansını değişkene atayın.

7.9 Varsayalım **int[][][] x = new char[12][5][2]** , dizide kaç eleman vardır? **x.length,x[2].length** ve **x[0][0].length** nedir?

7.10 Aşağıdaki kodun çıktısını gösterin:

```

intt[][][] dizi = {{{t1t,t2t}, {t3t,t4t}}, {{t5t,t6t},{t7t,t8t}}};System.out.p
rintln(array[t0t][t0t][t0t]);
System.out.println(array[t1t][t1t][t1t]);

```

PointttChtetck

MyProgrammingLab™

BÖLÜM ÖZETİ

1. İki boyutlu bir dizi, bir tabloyu saklamak için kullanılabilir.
2. İki boyutlu diziler için bir değişken şu sözdizimi ile tanımlanabilir:
`elementType[][] arrayVart`.
3. İki boyutlu bir dizi, aşağıdaki sözdizimi kullanılarak oluşturulabilir: `new elemanTipi [SATIR_BOYUTU][SÜTUN_BOYUTU]`.
4. İki boyutlu bir dizideki her eleman, aşağıdaki sözdizimi kullanılarak temsil edilir:
`diziDeğişkeni[satırIndeksi][sütunIndeksi]`.
5. Bir dizi başlatıcısı kullanarak iki boyutlu bir dizi oluşturabilir ve başlatabilirsiniz.
Sözdizimi: `elemanTipi[][] diziDeğişkeni = {{satır değerleri}, ... , {satır değerleri}}`.
6. Dizilerden diziler kullanarak çok boyutlu diziler oluşturabilirsiniz. Örneğin, üç boyutlu diziler için bir değişken `elemanTipi[][][] diziDeğişkeni` olarak tanımlanabilir ve `new elemanTipi[boyut1][boyut2][boyut3]` kullanılarak bir üç boyutlu dizi oluşturulabilir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinde yapın.

PROGRAMLAMA EGZERSİZLERİ

MyProgrammingLab™

- *7.1** (*Sütunları sütun sütun toplayın*) Aşağıdaki başlığı kullanarak bir matrisin belirli bir sütunundaki tüm elementlerin toplamını döndüren bir yöntem yazın:

```
public static double sütunTopla(double[][] m, int sütunIndeksi)
```

3'e 4'lük bir matris okuyan ve her sütunun toplamını gösteren bir test programı yazın. İşte bir örnek çalışma:

3'e 4'lük bir matris girin:

1	5	2	3	4	↵ Enter
5	5	6	7	8	↵ Enter
9	5	1	3	1	↵ Enter

Sütun 0'daki elemanların toplamı 16.5

Sütun 1'deki elemanların toplamı 9.0

Sütun 2'deki elemanların toplamı 13.0

Sütun 3'teki elemanların toplamı 13.0



- *7.2** (*Bir matrisin ana köşegenini toplayın*) Ana köşegendeki tüm sayıları toplayan bir yöntem yazın $n \times n$ aşağıdaki başlıkla tam sayılar matrisi:

```
public static double sumMajorDiagonal(double[][] m)
```

4x4 matris okuyan ve ana diyagonalindeki tüm elemanların toplamını gösteren bir test programı yazın. İşte bir örnek çalışma:



4x4 matrisini satır satır girin:

1 2 3 4.0 Enter
5 6.5 7 8 Enter
9 10 11 12 Enter
13 14 15 16 Enter

Ana diyagonal üzerindeki elemanların toplamı 34.5'tir.

***7.3** (Öğrencileri notlarına göre sırala) Liste 7.2'yi, GradeExam.java dosyasını, doğru cevap sayısının göre artan sırada öğrencileri gösterecek şekilde yeniden yazın.

****7.4** (Her çalışanın haftalık saatlerini hesapla) Tüm çalışanların haftalık saatlerinin iki boyutlu bir dizide saklandığını varsayalım. Her satır, bir çalışanın yedi günlük çalışma saatlerini yedi sütunla kaydeder. Örneğin, aşağıdaki dizi sekiz çalışanın çalışma saatlerini saklar. Çalışanları ve toplam saatlerini toplam saatlerin azalan sırasına göre gösteren bir program yazın.

	Su	P	T	Ç	C	C
Çalışan 0	2	4	3	4	5	8
Çalışan 1	7	3	4	3	3	4
Çalışan 2	3	3	4	3	3	2
Çalışan 3	9	3	4	7	3	4
Çalışan 4	3	5	4	3	6	3
Çalışan 5	3	4	4	6	3	4
Çalışan 6	3	7	4	8	3	8
Çalışan 7	6	3	5	9	2	7

7.5 (Cebir: iki matris ekle) İki matrisi eklemek için bir yöntem yazın. Yöntemin başlığı aşağıdaki gibidir:

public static double[][] matrisEkle(double[][] a, double[][] b)

İki matrisin eklenebilmesi için aynı boyutlara ve aynı veya uyumlu eleman türlerine sahip olmaları gerekir. Sonuç matrisini **c** olarak adlandıralım. Her bir eleman $c_{ij} = a_{ij} + b_{ij}$.

Örneğin, iki 3×3 matrisler **a** ve **b** dir

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} & a_{13} + b_{13} \\ a_{21} + b_{21} & a_{22} + b_{22} & a_{23} + b_{23} \\ a_{31} + b_{31} & a_{32} + b_{32} & a_{33} + b_{33} \end{pmatrix}$$

Kullanıcıdan iki tane girmesini isteyen bir test programı yazın 3×3 matrisler ve dis-toplamlarını oynar. İşte bir örnek çalışma:



VideoNotu

İki matris çarpın



Matris1'i girin: 1 2 3 4 5 6 7 8 9 Enter
Matris2'yi girin: 0 2 4 1 4.5 2.2 1.1 4.3 5.2 Enter
Matrisler aşağıdaki gibi toplanır
1.0 2.0 3.0 0.0 2.0 4.0 1.0 4.0 7.0
4.0 5.0 6.0 + 1.0 4.5 2.2 = 5.0 9.5 8.2
7.0 8.0 9.0 1.1 4.3 5.2 8.1 12.3 14.2

****7.6** (Cebir: iki matrisi çarp) İki matrisi çarpmak için bir yöntem yazın. Yöntemin başlığı şudur:

```
public static double[][] multi
    plyMatrix(double[][] a, double[][] b)
```

Matris **a**'yı matris **b** ile çarpmak için, **a**'daki sütun sayısı **b**'deki satır sayısı ile aynı olmalıdır ve iki matrisin de aynı veya uyumlu türde elemanları olmalıdır. **c** çarpımın sonucunu temsil etsin. Matris **a**'nın sütun boyutunun **n** olduğunu varsayalım. Her eleman $c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$

Örneğin, iki 3×3 matrisler **a** ve **b**, **c** dir

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{pmatrix}$$

nerede $c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$

Kullanıcıdan iki tane girmesini isteyen bir test programı yazın 3×3 matrisleri alırsınız ve ürünlerini gösterir. İşte bir örnek çalışma:

```
Matris1'i girin: 1 2 3 4 5 6 7 8 9 Enter
Matris2'yi girin: 0 2 4 1 4.5 2.2 1.1 4.3 5.2 Enter
Matrislerin çarpımı
1 2 3      0 2.0 4.0      5.3 23.9 24
4 5 6      * 1 4.5 2.2 = 11.6 56.3 58.2
7 8 9      1.1 4.3 5.2    17.9 88.7 92.4
```



***7.7** (Birbirine en yakın noktalar) Liste 7.3, iki boyutlu bir alanda birbirine en yakın iki noktayı bulan bir program verir. Programı, üç boyutlu bir alanda birbirine en yakın iki noktayı bulacak şekilde gözden geçirin. Noktaları temsil etmek için iki boyutlu bir dizi kullanın. Aşağıdaki noktaları kullanarak programı test edin:

```
double[][] points = {{-1, 0, 3}, {-1, -1, -1}, {4, 1, 1},
    {2, 0.5, 9}, {3.5, 2, -1}, {3, 1.5, 3}, {-1.5, 4, 2},
    {5.5, 4, -0.5}};
```

İki nokta arasındaki uzaklığı hesaplamak için formül (x_1, y_1, z_1) ve (x_2, y_2, z_2) is $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$.

****7.8** (Tüm en yakın nokta çiftleri) Tüm aynı minimum uzaklığa sahip en yakın nokta çiftlerini bulmak için 7.3. Listeyi, FindNearestPoints.java dosyasını gözden geçirin.

*****7.9** (Oyun: bir üçlü oyun oyna) Bir üçlü oyununda, iki oyuncu sırayla boş bir hücreyi işaretler. 3×3 her oyuncunun kendi jetonları (ya X ya da O) ile. Bir oyuncu, ızgarada yatay, dikey veya çapraz bir sırada üç jeton yerleştirdiğinde, oyun sona erer ve o oyuncu kazanır. Tüm hücreler jetonlarla doldurulduğunda ve hiçbir oyuncu kazanamadığında berabere (kazanan yok) durumu oluşur. Üçlü oynamak için bir program oluşturun.

Program, iki oyuncudan sırayla bir X jetonu ve bir O jetonu girmelerini ister. Herhangi bir jeton girildiğinde, program tahtayı konsolda yeniden gösterir ve oyun durumunu (kazanma, berabere veya devam etme) belirler. İşte bir örnek çalışma:



Oyuncu X için bir satır girin (0, 1 veya 2):

Oyuncu X için bir sütun girin (0, 1 veya 2):

1 ↵ Enter

1 ↵ Enter

	X		

O oyuncusu için bir satır girin (0, 1 veya 2):

O oyuncusu için bir sütun girin (0, 1 veya 2):

1 ↵ Enter

2 ↵ Enter

	X		

X oyuncusu için bir satır girin (0, 1 veya 2):

. . .

X			
0	X	0	
		X	

X oyuncusu kazandı

***7.10** (*En büyük satır ve sütun*) 0 ve 1'leri rastgele dolduran bir program yazın 4'e 4'lük bir matris, matrisi yazdırın ve en çok 1 olan ilk satır ve sütunu bulur. İşte programın bir örnek çalışması:

```
0011001111010101
```

En büyük satır indeksi: 2
En büyük sütun indeksi: 2

****7.11** (*Oyun: dokuz yazı ve tura*) Dokuz madeni para, bazıları

yüzü yukarı ve bazıları yüzü aşağı olacak şekilde 3'e 3'lük bir matrisle yerleştirilmiştir. Paraların durumunu 3'e 3'lük bir matrisle 0 (yazı) ve 1 (tura) değerleri kullanarak temsil edebilirsiniz. İşte bazı örnekler:

```
000 101 110 101 100
010 001 100 110 111
000 100 001 100 110
```

Her durum ayrıca bir ikili sayı kullanılarak temsil edilebilir. Örneğin, önceki matrisler şu sayılara karşılık gelir:

000010000 101001100 110100001 101110100 100111110

Toplam 512 olasılık vardır, bu nedenle matrisin tüm durumlarını temsil etmek için 0, 1, 2, 3, ... ve 511 arasında ondalık sayılar kullanabilirsiniz. Kullanıcıdan 0 ile 511 arasında bir sayı girmesini isteyen ve karşılık gelen matrisi **H** ve **T** karakterleriyle gösteren bir program yazın. İşte bir örnek çalışma:

0 ile 511 arasında bir sayı girin:

7

↵ Enter

H H H n
H H H H
n T T T



Kullanıcı **7** girdi, bu da **000000111**'ye karşılık geliyor. **0**, **H**'yi ve **1**

T'yi temsil ettiğinden, çıktı doğrudur.

**7.12

(Finansal uygulama: vergi hesaplama) Listing 3.6, ComputeTax.java dosyasını dizi kullanarak yeniden yazın. Her bir dosya durumu için altı vergi oranı vardır. Her oran, belirli bir miktar vergilendirilebilir gelire uygulanır. Örneğin, tek bir dosya için vergilendirilebilir gelir \$400,000 olduğunda, \$8,350 %10, (33,950 – 8,350) %15, (82,250 – 33,950) %25, (171,550 – 82,250) %28, (372,550 – 82,250) %33 ve (400,000 – 372,550) %36 oranında vergilendirilir. Altı oran, tüm dosya durumları için aynıdır ve aşağıdaki dizide temsil edilebilir:

double[] oranlar = {**0.10**, **0.15**, **0.25**, **0.28**, **0.33**, **0.35**};

Tüm dosyalama durumları için her oran için parantezler, aşağıdaki gibi iki boyutlu bir dizi olarak temsil edilebilir:

```
int[][] parantezler = {
    {8350, 33950, 82250, 171550, 372950}, // Bekar dosya veren
    {16700, 67900, 137050, 20885, 372950}, // Evlilikte ortak
                                     // veya nitelikli dul(erkek)
    {8350, 33950, 68525, 104425, 186475}, // Ayrı evli
    {11950, 45500, 117450, 190200, 372950} // Hanehalkı başı
};
```

Diyelim ki vergiye tabi gelir bekar dosya verenler için 400.000 \$'dır. Vergi aşağıdaki gibi hesaplanabilir:

```
vergi = parantezler[0][0] * oranlar[0] +
        (parantezler[0][1] - parantezler[0][0]) * oranlar[1] +
        (parantezler[0][2] - parantezler[0][1]) * oranlar[2] +
        (parantezler[0][3] - parantezler[0][2]) * oranlar[3] +
        (parantezler[0][4] - parantezler[0][3]) * oranlar[4] +
        (400000 - parantezler[0][4]) * oranlar[5]
```

*7.13

(En büyük öğeyi bul) Aşağıdaki yöntemi yazın, bu yöntem iki boyutlu bir dizideki en büyük öğenin konumunu döndürür.

```
public static int enBuyukBul(double[][] a)
```

Dönüş değeri, iki öğe içeren tek boyutlu bir dizidir. Bu dua öğe, iki boyutlu dizideki en büyük öğenin satır ve sütun indekslerini gösterir. Kullanıcının birini girmesini isteyen bir test programı yazın

iki boyutlu dizi ve dizideki en büyük elemanın konumunu gösterir. İşte bir örnek çalışma:



Dizinin satır ve sütun sayısını girin:

3 4 Enter

Diziyi girin:

23.5 35 2 10 Enter

4.5 3 45 3.5 Enter

35 44 5.5 9.6 Enter

En büyük elemanın konumu (1, 2) koordinatındadır.

****7.14** (*Matris keşfi*) Kullanıcıdan bir kare matrisin uzunluğunu girmesini isteyen, matrisi rastgele 0s ve 1s ile dolduran, matrisi yazdıran ve tüm 0s veya 1s olan satırları, sütunları ve köşegenleri bulan bir program yazın. İşte programın bir örnek çalışması:



Matrisin boyutunu girin:

4 Enter

011100001001111

1. satırdaki tüm 0'lar

3. satırdaki tüm 1'ler

Bir sütunda aynı numaralar yok

Ana köşegen üzerinde aynı numaralar yok

Alt köşegen üzerinde aynı numaralar yok

***7.15** (*Geometri: aynı çizgi mi?*) Programlama Egzersizi 5.39, üç noktanın aynı çizgide olup olmadığını test etmek için bir yöntem verir.

Aşağıdaki yöntemi yazın, böylece dizideki tüm noktaların **points** aynı çizgide olup olmadığını test edebilirsiniz.

```
public static boolean sameLine(tdoublet[][] puan)
```

Bir program yazın, kullanıcıdan beş nokta girmesini isteyin ve bu noktaların aynı çizgide olup olmadığını gösterin. İşte örnek çalıştırmalar:



Beş noktayı girin:

3.4 2 6.5 9.5 2.3 2.3 5.5 5 -5 4 Enter

Beş nokta aynı çizgide değildir



Beş noktayı girin:

1 1 2 2 3 3 4 4 5 5 Enter

Beş nokta aynı çizgide bulunmaktadır

***7.16** (*İki boyutlu diziyi sıralama*) Aşağıdaki başlığı kullanarak iki boyutlu bir diziyi sıralamak için bir yöntem yazın:

```
public static void sort(tinttm[][])
```

Yöntem, satırlarda birincil sıralama ve sütunlarda ikincil sıralama yapar. Örneğin, aşağıdaki dizi

$\{\{4, 2\}, \{1, 7\}, \{4, 5\}, \{1, 2\}, \{1, 1\}, \{4, 1\}\}$

şu şekilde sıralanacaktır

$\{\{1, 1\}, \{1, 2\}, \{1, 7\}, \{4, 1\}, \{4, 2\}, \{4, 5\}\}$.

***7.17

(Finansal tsunami) Bankalar birbirlerine para ödünç verir. Zor ekonomik zamanlarda, eğer bir banka iflas ederse, borcu geri ödeyemeyebilir. Bir bankanın toplam varlıkları, mevcut bakiyesi ile diğer bankalara olan kredilerinin toplamıdır. Şekil 7.8'deki diyagram beş bankayı göstermektedir. Bankaların mevcut bakiyeleri sırasıyla 25, 125, 175, 75 ve 181 milyon dolar dır. Düğüm 1'den düğüm 2'ye giden yönlü kenar, banka 1'in banka 2'ye 40 milyon dolar ödünç verdiğini göstermektedir.

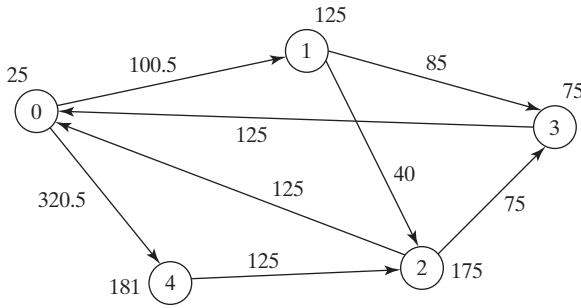


FIGURE 7.8 Bankalar birbirlerine para verir.

Bir bankanın toplam varlıkları belirli bir limitin altındaysa, banka güvensizdir. Borç aldığı parageri ödenemez ve alacaklı bu krediyi toplam varlıkları içinde sayamaz. Sonuç olarak, alacaklı dalimitin altındaysa güvensiz olabilir. Tüm güvensiz bankaları bulmak için bir program yazın. Programınız şu şekilde girdi alır. Öncelikle iki tam sayı n ve $limit$ okur; burada n bankaların sayısını belirtir ve $limit$ bir bankanın güvenli kılmak için gereken minimum toplam varlıkları ifade eder. Daha sonra n bankanın bilgilerini tanımlayan n satırı okur; bu bankaların kimlikleri 0 ile $n-1$ arasıdır.

Satırdaki ilk sayı bankanın bakiyesi, ikinci sayı bankadan borç alan banka sayısını belirtir ve geri kalanlar iki sayının çiftleridir. Her çift bir borç alanı tanımlar. Çiftteki ilk sayı borç alanın kimlik numarası ve ikincisi borç alınan miktardır. Örneğin, Şekil 7.8'deki beş banka için girdi aşağıdaki gibidir (limitin 201 olduğunu unutmayın):

```
5 201
25 2 1 100.5 4 320.5
125 2 2 40 3 85
175 2 0 125 3 75
75 1 0 125
181 1 2 125
```

Bankanın 3 toplam varlıkları $(75 + 125)$, 201'in altında, bu nedenle banka 3 güvensizdir. Banka 3 güvensiz hale geldikten sonra, banka 1'in toplam varlıkları $(125 + 40)$ şu seviyenin altına düşer. Böylece, banka 1 de güvensizdir. Programın çıktısı şöyle olmalıdır:

Güvensiz bankalar 3 1

(İpucu: Kredileri temsil etmek için iki boyutlu bir dizi **borç alanlar** kullanın. **borç alanlar[i][j]** banka **i**'nin banka **j**'ye verdiği krediyi belirtir. Bir kez banka **j** güvensiz hale geldiğinde, **borç alanlar[i][j]** 0 olarak ayarlanmalıdır.)

***7.18**(Satırları karıştır) Aşağıdaki başlığı kullanarak iki boyutlu bir **int**

dizisindeki satırları karıştıran bir yöntem yazın:

public static voidkarıştır(**int**[][] m)

Aşağıdaki matrisi karıştıran bir test programı yazın:

int[][] m = {{1, 2}, {3, 4}, {5, 6}, {7, 8}, {9, 10}};****7.19**

(Desen tanıma: dört ardışık eşit sayı) Aşağıdaki

metodu yazın, bu metodun iki boyutlu bir dizinin aynı değere sahip dört ardışık sayıya sahip olup olmadığını test etmesini sağlayın, yatay, dikey veya çapraz olarak.

public static booleanisConsecutiveFour(**int**[][] değerler)

Kullanıcıdan iki boyutlu bir dizinin satır ve sütun sayısını girmesini isteyen ve ardından dizideki değerleri alarak, dizinin aynı değere sahip dört ardışık sayıyı içerip içermediğini kontrol eden bir test programı yazın. Eğer içeriyorsa true, aksi takdirde false gösterin. İşte doğru durumların bazı örnekleri:

0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	6	2	1	8	2	9
6	5	6	1	1	9	1
1	3	6	1	4	0	7
3	3	3	3	4	0	7

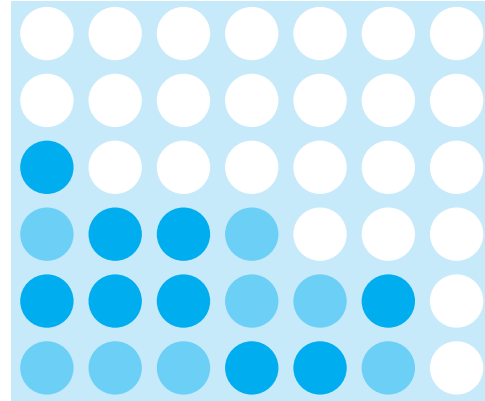
0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	5	2	1	8	2	9
6	5	6	1	1	9	1
1	5	6	1	4	0	7
3	5	3	3	4	0	7

0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	6	2	1	6	2	9
6	5	6	6	1	9	1
1	3	6	1	4	0	7
3	6	3	3	4	0	7

0	1	0	3	1	6	1
0	1	6	8	6	0	1
9	6	2	1	8	2	9
6	9	6	1	1	9	1
1	3	9	1	4	0	7
3	3	3	9	4	0	7

*****7.20**

(Oyun: dört bağla) Dört bağla, oyuncuların sırayla yedi sütunlu, altı sıralı dikey olarak askıya alınmış bir ızgaraya renkli diskler bıraktığı iki kişilik bir masa oyunudur, aşağıda gösterildiği gibi.



Oyunun amacı, rakibinizden önce bir sırada, bir sütunda veya bir diyagonal üzerinde dört aynı renkte diski bağlamaktır. Program, iki oyuncunun sırayla kırmızı veya sarı bir disk bırakmasını ister. Önceki şekilde, kırmızı disk koyu bir renkte ve sarı açık bir renkte gösterilmektedir. Bir disk bırakıldığında, program tahtayı konsolda yeniden gösterir ve oyunun durumunu (kazanma, berabere veya devam etme) belirler. İşte bir örnek çalışma:



```

| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

```

Kırmızı diski sütuna bırakın (0-6):

0 Enter

```

| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
|R | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

```

Sarı diski sütuna bırakın (0-6):

3 Enter

```

| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

```

```

: : :
: : :

```

Sarı diski sütuna bırakın (0-6):

6 Enter

```

| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
|R | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

```

Sarı oyuncu kazandı

- *7.21** (Merkez şehir) Bir şehirler kümesi verildiğinde, merkez nokta, diğer tüm şehirlere en kısatoplam uzaklığa sahip olan şehirdir. Kullanıcıdan şehir sayısını ve şehirlerin konumlarını(koordinatlar) girmesini isteyen bir program yazın ve merkez şehri bulun.

Şehir sayısını girin:

5 Enter

Şehirlerin koordinatlarını girin:

2.5 5 5.1 3 1 9 5.4 54 5.5 2.1 Enter

Merkez şehir (2.5, 5.0) konumundadır



- *7.22** (Çift sayıda 1) 0 ve 1'lerle doldurulmuş 6'ya 6'lık bir iki boyutlu matris oluşturan, matrisi gösteren ve her satır ve her sütunun çift sayıda 1'e sahip olup olmadığını kontrol eden bir program yazın.

- *7.23** (Oyun: ters çevrilen hücreyi bul) 0 ve 1'lerle doldurulmuş 6'ya 6'lık bir matris verildiğini varsayın. Tüm satırlar ve tüm sütunlar çift sayıda 1'e sahiptir. Kullanıcının birini çevirmesine izin verin



VideoNotu

Çift sayıda 1

hücre (yani, 1'den 0'a veya 0'dan 1'e çevirin) ve hangi hücrenin çevrildiğini bulmak için bir program yazın. Programınız, kullanıcıdan 0'lar ve 1'lerle 6'ya 6'lık bir dizi girmesini istemeli ve 1'lerin çift sayıda olduğu özelliğin ihlal edildiği ilk satır ve ilk sütun c'yi bulmalıdır (yani, 1'lerin sayısı çift değildir). Çevrilen hücre (r, c) konumundadır.

***7.24**

(*Sudoku çözümünü kontrol et*) Liste 7.4, her sayının tahtada geçerli olup olmadığını kontrol eder. Ek bir çözümün geçerli olup olmadığını kontrol eder. Programı, her satırın, her sütunun ve her küçük kutunun 1'den 9'a kadar sayılara sahip olup olmadığını kontrol ederek yeniden yazın.

***7.25**

(*Markov matrisi*) Bir $n \times n$ matrise *pozitif Markov matrisi* denir eğer her eleman pozitif ve her sütundaki elemanların toplamı 1'dir. Bir matrisin Markov matrisi olup olmadığını kontrol etmek için aşağıdaki yöntemi yazın.

```
public static boolean isMarkovMatrix(double[][] m)
```

Kullanıcıdan bir girdi almasını isteyen bir test programı yazın 3×3 çift değerlerden oluşan matris ve bunun bir Markov matris olup olmadığını test eder. İşte örnek çalıştırmalar:



```
3'e 3'lük bir matrisi satır satır girin:
0.15 0.875 0.375 ↵ Enter
0.55 0.005 0.225 ↵ Enter
0.30 0.12 0.4 ↵ Enter
Bu bir Markov matrisidir
```



```
3'e 3'lük bir matrisi satır satır girin:
0.95 -0.875 0.375 ↵ Enter
0.65 0.005 0.225 ↵ Enter
0.30 0.22 -0.4 ↵ Enter
Bu bir Markov matris değildir
```

***7.26**

(*Satır sıralama*) İki boyutlu bir dizideki satırları sıralamak için aşağıdaki yöntemi uygulayın. Yeni bir dizi döndürülür ve orijinal dizi bozulmaz.

```
public static double[][] sortRows(double[][] m)
```

Kullanıcıdan bir girdi almasını isteyen bir test programı yazın 3×3 çift değer matrisini ve yeni bir satır sıralı matris görüntüler. İşte bir örnek çalışma:



```
3'e 3'lük bir matrisi satır satır girin:
0.15 0.875 0.375 ↵ Enter
0.55 0.005 0.225 ↵ Enter
0.30 0.12 0.4 ↵ Enter

Satır sıralı dizi
0.15 0.375 0.875
0.005 0.225 0.55
0.12 0.30 0.4
```

***7.27**

(*Sütun sıralama*) İki boyutlu bir dizide sütunları sıralamak için aşağıdaki yöntemi uygulayın. Yeni bir dizi döndürülür ve orijinal dizi bozulmaz.

```
public static double[][] sortColumns(double[][] m)
```

Kullanıcıdan bir girdi almasını isteyen bir test programı yazın 3×3 çift değer matrisini ve yeni bir sütun sıralı matris görüntüler. İşte bir örnek çalıştırma:

3'e 4'lük bir matris girin:

0.15 0.875 0.375

0.55 0.005 0.225

0.30 0.12 0.4

Sütun sıralı dizi

0.15 0.0050 0.225

0.3 0.12 0.375

0.55 0.875 0.4



7.28 (Tamamen aynı diziler) İki boyutlu diziler **m1** ve **m2** tamamen aynı dizi eğer *k* arşılık gelen elemanları eşitse. Aşağıdaki başlığı kullanarak **m1** ve **m2**'nin *t* amamen aynı olduğunu döndüren bir yöntem yazın:

```
public static booleantequals(tintt[][] m1, tintt[][] m2)
```

Kullanıcıdan iki tane girmesini isteyen bir test programı yazın 3×3 tam sayılardan oluşan diziler ve ikisinin de tamamen aynı olup olmadığını görüntüler. İşte örnek çalıştırmalar.

list1'i girin: 51 22 25 6 1 4 24 54 6

Liste2'yi girin: 51 22 25 6 1 4 24 54 6

İki dizi kesinlikle aynıdır



list1'i girin: 51 25 22 6 1 4 24 54 6

Liste2'yi girin: 51 22 25 6 1 4 24 54 6

İki dizi kesinlikle aynı değildir



7.29 (Aynı diziler) İki boyutlu diziler **m1** ve **m2**, aynı içeriğe sahiplerse aynıdır. **m1** ve **m2**'nin aynı olduğunu döndüren bir yöntem yazın, aşağıdaki başlığı kullanarak:

```
public static booleantequals(tintt[][] m1, tintt[][] m2)
```

Kullanıcıdan iki tam sayı listesi girmesini isteyen ve ikisinin aynı olup olmadığını gösteren bir test programı yazın. İşte örnek çalıştırmalar.

list1'i girin: 51 25 22 6 1 4 24 54 6

Liste2'yi girin: 51 22 25 6 1 4 24 54 6

İki dizi aynıdır



list1'i girin: 51 5 22 6 1 4 24 54 6

Liste2'yi girin: 51 22 25 6 1 4 24 54 6

İki dizi aynı değildir



- *7.30** (Cebir: doğrusal denklemleri çöz) Aşağıdaki doğrusal denklemler sistemi 2×2 ni çözen bir yöntem yazın:

$$\begin{aligned} a_{00}x + a_{01}y &= b_0 \\ a_{10}x + a_{11}y &= b_1 \end{aligned} \quad x = \frac{b_0a_{11} - b_1a_{01}}{a_{00}a_{11} - a_{01}a_{10}} \quad y = \frac{b_1a_{00} - b_0a_{10}}{a_{00}a_{11} - a_{01}a_{10}}$$

Yöntem başlığı

`public static double[] linearEquation(double[][] a, double[] b)`

Yöntem `null` döner eğer `is 0 ise`.

ve `0` ise, "Denklemin çözümü yok." şeklinde rapor edin. Örnek bir çalışma Programlama Egzersizi 3.3'e benzer.

- *7.31** (Geometri: kesişim noktası) İki doğrunun kesişim noktasını döndüren bir yöntem yazın. İki doğrunun kesişim noktası, Programlama Egzersizi 3.25'te gösterilen formülü kullanarak bulunabilir. `(x1,y1)` ve `(x2,y2)` noktalarının 1. doğrudaki iki nokta olduğunu varsayın ve `(x3,y3)` ve `(x4,y4)` noktalarının 2. doğrudaki noktalar olduğunu varsayın. Yöntem başlığı

`public static double[] getIntersectingPoint(double[][] puan)`

Noktalar, `(points[0][0], points[0][1])` için `(x1, y1)` ile birlikte bir 4'e 2 boyutlu dizi `points` içinde saklanır. Yöntem, iki doğru paralel ise kesişim noktasını veya `null` döner. Kullanıcıdan dört nokta girmesini isteyen ve kesişim noktasını gösteren bir program yazın. Örnek bir çalışma için Programlama Egzersizi 3.25'e bakın.

- *7.32** (Geometri: bir üçgenin alanı) Aşağıdaki başlığı kullanarak bir üçgenin alanını döndüren bir yöntem yazın:

`public static double getTriangleArea(double[][] puan)`

Noktalar, `points` içinde 3'e 2 boyutlu bir dizi olarak saklanır noktalar[0][0] ve noktalar[0][1] için `(x1,y1)`. Üçgen alanı, Programlama Egzersizi 2.15'teki formül kullanılarak hesaplanabilir. Yöntem, üç nokta aynı çizgideyse `0` döner. Kullanıcıdan iki çizgi girmesini isteyen ve kesişim noktasını gösteren bir program yazın. İşte programın bir örnek çalışması:



x1, y1, x2, y2, x3, y3 girin: 2.5 2.5 -1.0 4.0 2.0 Enter
Üçgenin alanı 2.25



Enter x1, y1, x2, y2, x3, y3: 2 2 4.5 4.5 6 6 Enter
The three points are on the same line

- *7.33** (Geometri: çokgen alt alanları) Dört köşeli bir konveks çokgen, Şekil 7.9'da gösterildiği gibi dört üçgene ayrılmıştır.

Kullanıcıdan dört köşenin koordinatlarını girmesini isteyen ve dört üçgenin alanlarını artan sırayla gösteren bir program yazın. İşte bir örnek çalışma:



x1, y1, x2, y2, x3, y3, x4, y4 girin: -2.5 2 4 4 3 -2 -2 -3.5 Enter
Alanlar 6.17 7.96 8.08 10.42

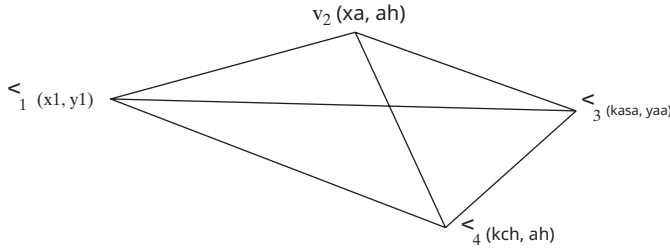


FIGURE 7.94 köşeli bir çokgen dört köşe ile tanımlanır.

***7.34** (*Geometri: en sağdaki en düşük nokta*) Hesaplamalı geometri alanında, genellikle bir dizi noktada en sağdaki en düşük noktayı bulmanız gerekir. Aşağıdaki yöntemi yazın ki bir dizi noktada en sağdaki en düşük noktayı döndürsün.

genel statik ikili[]

getRightmostLowestPoint(tdoublet[][] puan)

Kullanıcıdan altı noktanın koordinatlarını girmesini isteyen ve en sağdaki en alttaki noktayı gösteren bir test programı yazın. İşte bir örnek çalışma:

6 nokta girin: 1.5 2.5 -3 4.5 5.6 -7 6.5 -7 8 1 10 2.5 ↵ Enter
En sağdaki en alttaki nokta (6.5, -7.0)



****7.35** (*En büyük blok*) Elemanları 0 veya 1 olan bir kare matris verildiğinde, tüm elemanları 1 olan maksimum kare alt matrisini bulmak için bir program yazın. Programınız, kullanıcıdan matrisin satır sayısını girmesini istemelidir. Program daha sonra maksimum kare alt matrisindeki ilk elemanın konumunu ve alt matrisin satır sayısını gösterir. İşte bir örnek çalışma:

Matrisin satır sayısını girin: 5 ↵ Enter
Matrisin satırlarını satır satır girin:
1 0 1 0 1 ↵ Enter
1 1 1 0 1 ↵ Enter
1 0 1 1 1 ↵ Enter
1 0 1 1 1 ↵ Enter
1 0 1 1 1 ↵ Enter

Maksimum kare alt matris (2, 2) konumunda ve boyutu 3'tür



Programınız maksimum kare alt matrisini bulmak için aşağıdaki yöntemi uygulamalı ve kullanmalıdır:

public static int[] findLargestBlock(tintt[][] m)

Dönüş değeri, üç değerden oluşan bir dizi. İlk iki değer, alt matrisin ilk elemanının satır ve sütun indeksleridir ve üçüncü değer, alt matrisin satır sayısını belirtir.

****7.36** (*Latin kare*) Latin kare, n farklı Latin harfi ile doldurulmuş n -by- n dizisidir; h er harf, her satırda ve her sütunda tam olarak bir kez bulunur. Bir yazın.

294 Bölüm 7 Çok Boyutlu Diziler

kullanıcının n sayısını ve karakterler dizisini girmesini isteyen bir program, örnek çıktıda gösterildiği gibi, ve girdi dizisinin bir Latin kare olup olmadığını kontrol eder. Karakterler, A'dan başlayarak ilk n karakterdir.



Bir sayı girin n: 4
Boşluklarla ayrılmış 4 satır harf girin:
A B C D
B A D C
C D B A
D C A B
Girdi dizisi bir Latin karesidir



Bir sayı girin n: 3
Boşluklarla ayrılmış 3 satır harf girin:
A F D
Yanlış girdi: harfler A'dan C'ye kadar olmalıdır

BÖLÜM

8

NESNELER VE SINIFLAR

Hedefler

- Nesneleri ve sınıfları tanımlamak ve nesneleri modellemek için sınıfları kullanmak (§8.2).
- Sınıfları ve nesneleri tanımlamak için UML grafik notasyonunu kullanmak (§8.2).
- Sınıfları nasıl tanımlayacağınızı ve nesneleri nasıl oluşturacağınızı göstermek (§8.3).
- Yapıcılar kullanarak nesneler oluşturmak (§8.4).
- Nesnelere nesne referans değişkenleri aracılığıyla erişmek (§8.5).
- Bir referans türü kullanarak bir referans değişkeni tanımlamak (§8.5.1).
- Bir nesnenin verilerine ve yöntemlerine nesne üye erişim operatörü (.) kullanarak erişmek (§8.5.2).
- Referans türlerinin veri alanlarını tanımlamak ve bir nesnenin veri alanları için varsa yılan değerler atamak (§8.5.3).
- Nesne referans değişkenleri ile ilkel veri türü değişkenleri arasında ayırım yapmak (§8.5.4).
- Java kütüphane sınıflarını **Date**, **Random** ve **JFrame** kullanmak (§8.6).
- Örnek ve statik değişkenler ile yöntemler arasında ayırım yapmak (§8.7).
- Uygun **get** ve **set** yöntemleri ile özel veri alanları tanımlamak (§8.8).
- Veri alanlarını kapsülleyerek sınıfların bakımını kolaylaştırmak (§8.9).
- Nesne argümanları ile yöntemler geliştirmek ve ilkel tür argümanları ile nesne türü argümanları arasında ayırım yapmak (§8.10).
- Nesneleri dizilerde depolamak ve işlemek (§8.11).



8.1 Giriş



Nesne yönelimli programlama, büyük ölçekli yazılımlar ve GUI'ler geliştirmeyi

effektif bir şekilde sağlar.

Önceki bölümlerdeki materyali öğrendikten sonra, seçimler, döngüler, yöntemler ve diziler kullanılarak birçok programlama sorununu çözebilirsiniz. Ancak, bu Java özellikleri grafik kullanıcı arayüzleri ve büyük ölçekli yazılım sistemleri geliştirmek için yeterli değildir. Şekil 8.1'de gösterildiği gibi bir grafik kullanıcı arayüzü (GUI, "goo-ee" olarak telaffuz edilir) geliştirmek istiyorsanız, bunu nasıl programlarsınız? Bu bölümde bunu öğreneceksiniz.

Neden OOP?



ŞEKİL 8.1 GUI nesneleri sınıflardan oluşturulmuştur.

Bu bölüm, GUI ve büyük ölçekli yazılım sistemleri geliştirmek için kullanabileceğiniz nesne yönelimli programlamayı tanıtmaktadır.

8.2 Nesneler için Sınıfların Tanımlanması



Bir sınıf, nesnelerin özelliklerini ve davranışlarını tanımlar.

Nesne yönelimli programlama (OOP), nesneleri kullanarak programlama yapmayı içerir. Bir *nesne*, gerçek dünyada belirgin bir şekilde tanımlanabilen bir varlığı temsil eder. Örneğin, bir öğrenci, bir masa, bir daire, bir düğme ve hatta bir kredi, nesne olarak görülebilir. Bir nesnenin benzersiz bir kimliği, durumu ve davranışı vardır.

- Bir nesnenin *durumu* (aynı zamanda *özellikleri* veya *nitelikleri* olarak da bilinir) mevcut değerleri ile birlikte veri alanları ile temsil edilir. Örneğin, bir daire nesnesinin bir veri alanı vardır *yarıçap*, bu daireyi tanımlayan özelliktir. Bir dikdörtgen nesnesinin ise *genişlik* ve *yükseklik* veri alanları vardır, bu da dikdörtgeni tanımlayan özelliklerdir.
- Bir nesnenin *davranışı* (aynı zamanda *eylemleri* olarak da bilinir) yöntemler tarafından tanımlanır. Bir nesne üzerinde bir yöntemi çağırmak, nesneden bir eylem gerçekleştirmesini istemektir. Örneğin, daire nesneleri için `getArea()` ve `getPerimeter()` adında yöntemler tanımlayabilirsiniz. Bir daire nesnesi, alanını döndürmek için `getArea()` yöntemini çağırabilir ve çevresini döndürmek için `getPerimeter()` yöntemini çağırabilir. Ayrıca, `setRadius(radius)` yöntemini de tanımlayabilirsiniz. Bir daire nesnesi, yarıçapını değiştirmek için bu yöntemi çağırabilir.

Aynı türdeki nesneler, ortak bir sınıf kullanılarak tanımlanır. Bir *sınıf*, bir nesnenin veri alanları ve yöntemlerinin ne olacağını tanımlayan bir şablon, mavi-plan veya *sözleşme* dir. Bir nesne, bir sınıfın bir örneğidir. Bir sınıfın birçok örneğini oluşturabilirsiniz. Bir örnek oluşturma işlemine *örnekleme* denir. *nesne* ve *örnek* terimleri genellikle birbirinin yerine kullanılabilir. Sınıflar ve nesneler arasındaki ilişki, bir elmalı turta tarifi ile elmalı turtalar arasındaki ilişkiye benzer: Tek bir tarif ile istediğiniz kadar elmalı turta yapabilirsiniz. Şekil 8.2, *Daire* adında bir sınıfı ve onun üç nesnesini göstermektedir.

Bir Java sınıfı, veri alanlarını tanımlamak için değişkenler ve eylemleri tanımlamak için yöntemler kullanır. Ayrıca, bir sınıf, yeni bir nesne oluşturmak için çağrılan *yapıcılar* olarak bilinen özel bir türde yöntemler sağlar. Bir yapıcı herhangi bir eylemi gerçekleştirebilir, ancak yapıcılar, nesnelerin veri alanlarını başlatma gibi başlatma eylemlerini gerçekleştirmek için tasarlanmıştır. Şekil 8.3, daire nesneleri için sınıf tanımlamanın bir örneğini göstermektedir.



VideoNotu

Sınıfları ve nesneleri tanımlayın

nesne

bir nesnenin durumu

özellikler

nitelikler

data alanları

davranış

hareketler

sınıf

sözleşme

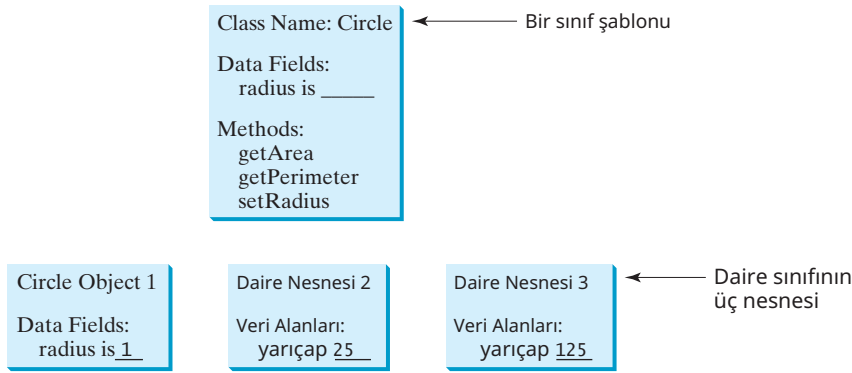
örnekleme

örnek

veri alanı

metot

kurucular



ŞEKİL8.2Bir sınıf, nesneler oluşturmak için bir şablondur.

```

class Daire {
    /** Bu dairenin yarıçapı */
    double yarıçap = 1;

    /** Bir daire nesnesi oluştur */
    Daire() {
    }

    /** Bir daire nesnesi oluştur */
    Daire(double yeniYarıçap) {
        yarıçap = yeniYarıçap;
    }

    /** Bu dairenin alanını döndür */
    double getAlan() {
        return yarıçap * yarıçap * Math.PI;
    }

    /** Bu dairenin çevresini döndür */
    double getPerimeter() {
        return 2 * radius * Math.PI;
    }

    /** Bu daire için yeni bir yarıçap ayarlayın */
    double setRadius(double newRadius) {
        radius = newRadius;
    }
}

```

Veri alanı

Yapıcılar

Metot

FIGURE 8.3 Bir sınıf, aynı türdeki nesneleri tanımlayan bir yapıdır.

Daire sınıfı, şimdiye kadar gördüğünüz diğer tüm sınıflardan farklıdır. Ana metodu yoktur ve bu nedenle çalıştırılmaz; bu sadece daire nesneleri için bir tanımdır. Ana metodu içeren sınıf, bu kitapta kolaylık olması açısından ana sınıf olarak adlandırılacaktır.

Şekil 8.2'deki sınıf şablonları ve nesnelerinin gösterimi, *Birleşik*

Modelleme Dili (UML) notasyonu kullanılarak standartlaştırılabilir. Bu notasyon, Şekil 8.4'te gösterildiği gibi, *UML sınıf diyagramı* veya kısaca *sınıf diyagramı* olarak adlandırılır. Sınıf diyagramında, veri alanı şu şekilde gösterilir

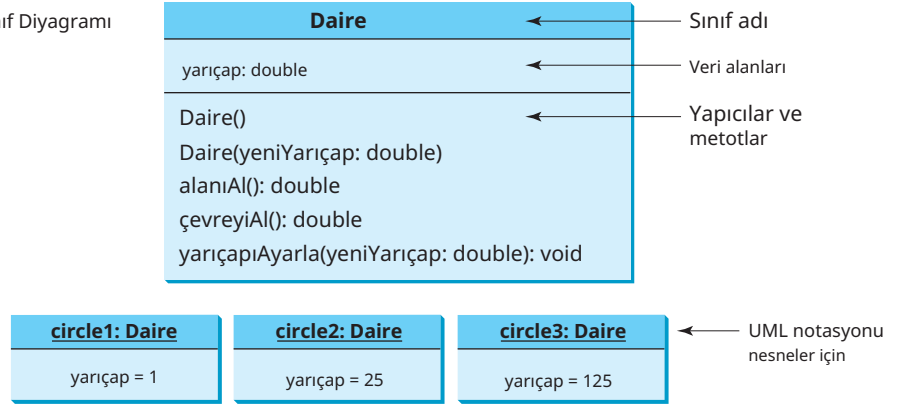
dataFieldName: dataType

Yapıcı şu şekilde gösterilir

SınıfAdı(parametreAdı: parametreTürü)

ana sınıf
Birleşik Modelleme Dili
(UML)
sınıf diyagramı

UML Sınıf Diyagramı

**FIGURE 8.4** Sınıflar ve nesneler UML notasyonu kullanılarak temsil edilebilir.

Yöntem şu şekilde gösterilir

methodName(parameterName: parametreType): returnType

8.3 Örnek: Sınıfları Tanımlama ve Nesneler Oluşturma

Sınıflar, nesnelerin tanımlarıdır ve nesneler sınıflardan oluşturulur.



Bu bölüm, sınıfları tanımlamanın iki örneğini verir ve nesneler oluşturmak için bu sınıfları kullanır. Liste 8.1, **Circle** sınıfını tanımlayan ve nesneler oluşturmak için kullanan bir programdır. Program, **1,25** ve **125** yarıçapına sahip üç daire nesnesi oluşturur ve her birinin yarıçapını ve alanını görüntüler. Daha sonra ikinci nesnenin yarıçapını **100** olarak değiştirir ve yeni yarıçapını ve alanını görüntüler.

isim çatışmalarından kaçının



Not

Bu bölümde daha sonra tanıtilan birkaç geliştirilmiş **Circle** sınıfı ile isim çakışmasının önlemek için, bu örnekteki **Circle** sınıfı **SimpleCircle** olarak adlandırılmıştır. Basitlik açısından, metinde hala bu sınıfa **Circle** olarak atıfta bulunacağız.

LISTE 8.1 TestSimpleCircle.java

ana sınıf

bakım yönteminin

esne oluşturma

nesne oluşturma

nesne oluşturma

```

1  genel sınıftestSimpleCircle {
2      /** Ana yöntem */
3      public static voidtmain(String[] args) {
4          // Yarıçapı 1 olan bir daire oluşturma
5          SimpleCircle Circle1 = tnewtSimpleCircle();
6          System.out.println("Yarıçapı "
7              + daire1.yarıçap + " olan dairenin alanı: " + circle1.getArea());
8
9          // Yarıçapı 25 olan bir daire oluşturma
10         SimpleCircle Circle2 = tnewtSimpleCircle(t25t);
11         System.out.println("Yarıçapı "
12             + circle2.radius + " olan dairenin alanı" + circle2.getArea());
13
14         // Yarıçapı 125 olan bir daire oluşturma
15         SimpleCircle Circle3 = tnewtSimpleCircle(t125t);
16         System.out.println("Yarıçapı "
17             + circle3.radius + " olan dairenin alanı" + circle3.getArea());
18
19         // daire yarıçapını değiştir
20         e 2.yarıçap = t100t; // veya circle2.setRadius(100)
21         System.out.println("Yarıçapı "
22             + daire.yarıçap dair + " olan dairenin alanı" + circle2.getArea());
23     }
24 }
  
```

```

yirmidört }
2526 // İki yapıcılıkla daire sınıfını tanımla
27
28 çift radius ;
29
30 /** Yarıçapı 1 olan bir daire oluştur */
31 BasitÇember() {
32     radius = 1;
33 }
34
35 /** Belirtilen yarıçapa sahip bir daire oluştur */
36 SimpleCircle(double newRadius) {
37     radius = newRadius;
38 }
39
40 /** Bu dairenin alanını döndür */
41 double getArea() {
42     return radius * yarıçap * Math.PI;
43 }
44
45 /** Bu dairenin çevresini döndür */
46 double getPerimeter() {
47     return 2 * yarıçap * Math.PI;
48 }
49
50 /** Bu daire için yeni bir yarıçap ayarla */
51 void setRadius(double newRadius) {
52     radius = newRadius;
53 }
54 }

```

şık Basit Ciri data alanı

parametresiz yapıcılı

ikinci yapıcılı

alanıAl

çevreyiAl

yarıçapıAyarla

Yarıçapı 1.0 olan dairenin alanı 3.141592653589793
 Yarıçapı 25.0 olan dairenin alanı 1963.4954084936207
 Yarıçapı 125.0 olan dairenin alanı 49087.385212340516
 Yarıçapı 100.0 olan dairenin alanı 31415.926535897932



Program iki sınıf içerir. Bunlardan ilki, **TestSimpleCircle**, ana sınıftır. Tek amacı, ikinci sınıfı test etmektir, **SimpleCircle**. Sınıfı kullanan böyle bir programa sıklıkla sınıfın *istemicisi* denir. Programı çalıştırdığınızda, Java çalışma zamanı sistemi ana sınıftaki **main** yöntemini çağırır.

istemicisi

İki sınıfı bir dosyaya koyabilirsiniz, ancak dosyada yalnızca bir sınıf *public class* olabilir. Ayrıca, *public* sınıfın dosya adıyla aynı adı taşıması gerekir. Bu nedenle, dosya adı **TestSimpleCircle.java**'dır, çünkü **TestSimpleCircle** *public*'tir. Kaynak kodundaki her sınıf bir *.class* dosyasına derlenir. **TestSimpleCircle.java** dosyasını derlediğinizde, iki sınıf dosyası **TestSimpleCircle.class** ve **SimpleCircle.class** oluşturulur, Şekil 8.5'te gösterildiği gibi.

genel sınıf

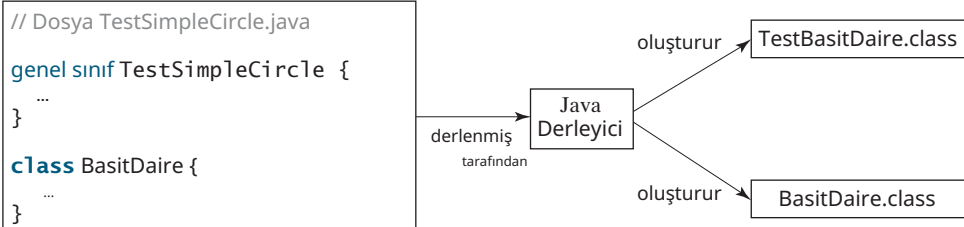


FIGURE 8.5 Kaynak kod dosyasındaki her sınıf bir *.class* dosyasına derlenir.

Ana sınıf, üç nesne oluşturan **main** yöntemini (satır 3) içerir. Bir dizi oluştururken olduğu gibi, **new** operatörü, yapıcından bir nesne oluşturmak için kullanılır. **new BasitDaire()**, **1** yarıçapa sahip bir nesne oluşturur (satır 5), **new BasitDaire(25)**, yarıçapa sahip bir nesne oluşturur **25** (satır 10) ve **new BasitDaire(125)**, **125** yarıçapa sahip bir nesne oluşturur (satır 15).

Bu üç nesne (sırasıyla **daire1**, **daire2** ve **daire3** ile referans alınır) farklı verilere sahiptir ancak aynı yöntemlere sahiptir. Bu nedenle, **getArea()** yöntemini kullanarak ilgili alanlarını hesaplayabilirsiniz. Veri alanlarına, nesnenin referansı aracılığıyla **circle1.radius**, **circle2.radius** ve **circle3.radius** kullanılarak erişilebilir. Nesne, nesnenin referansı aracılığıyla **circle1.getArea()**, **circle2.getArea()** ve **circle3.getArea()** yöntemini çağırabilir.

Bu üç nesne bağımsızdır. **daire2**'nin yarıçapı satır 20'de **100** olarak değiştirilmiştir. Objenin yeni yarıçapı ve alanı 21-22. satırlarda gösterilmektedir.

Java programları yazmanın birçok yolu vardır. Örneğin, örnekteki iki

sınıfı birleştirerek tek bir sınıf haline getirebilirsiniz; bu, Liste 8.2'de gösterilmektedir.

LISTE 8.2 SimpleCircle.java

```

1  SimpleCircle {
2  /** Ana yöntem */
3  public static void main(String[] args) {
4      // Yarıçapı 1 olan bir daire oluştur
5      SimpleCircle circle1 = new SimpleCircle();
6      System.out.println("Yarıçapı "
7          + circle1.yarıçap + " olan dairenin alanı: " + circle1.getArea());
8
9      // Yarıçapı 25 olan bir daire oluştur
10     SimpleCircle circle2 = new SimpleCircle(25);
11     System.out.println("Yarıçapı olan dairenin alanı "
12         + circle2.radius + " dir " + circle2.getArea());
13
14     // Yarıçapı 125 olan bir daire oluştur
15     SimpleCircle circle3 = new SimpleCircle(125);
16     System.out.println("Yarıçapı olan dairenin alanı "
17         + circle3.radius + " dir " + circle3.getArea());
18
19     // daire yarıçapını değiştir
20     circle2.yarıçap = 100;
21     System.out.println("Yarıçapı olan dairenin alanı "
22         + circle2.radius + " dir " + circle2.getArea());
23 }
24
25 double radius;
26
27 /** Yarıçapı 1 olan bir daire oluştur */
28 BasitÇember() {
29     radius = 1;
30 }
31
32 /** Belirtilen bir yarıçapa sahip bir daire oluştur */
33 SimpleCircle(double newRadius) {
34     yarıçap = yeniYarıçap;
35 }
36
37 /** Bu dairenin alanını döndür */
38 double getArea() {
39     return radius * yarıçap * Math.PI;
40 }
41

```

main metodu

veri alanı

parametresiz yapıcı

ikinci yapıcı

metot

```

42  /** Bu dairenin çevresini döndür */
43  doubletgetPerimeter() {
44      return 2*t* yarıçapı * Math.PI;
45  }
46
47  /** Bu daire için yeni bir yarıçap ayarlayın */
48  voidtsetRadius(tdoubletnewRadius) {
49      radius = newRadius;
50  }
51  }

```

Birleşik sınıfın **main** metodu olduğundan, Java yorumlayıcısı tarafından çalıştırılabilir. **main** metodu, Listeleme 8.1'dekiyle aynıdır. Bu, bir sınıfı test edebileceğinizi gösterir; sadece aynı sınıfa bir **main** metodu ekleyerek.

Başka bir örnek olarak, televizyon setlerini düşünün. Her TV, durumları (mevcut kanal, mevcut ses seviyesi, açık veya kapalı) ve davranışları (kanalları değiştirme, ses seviyesini ayarlama, açma/kapama) olan bir nesnedir. TV setlerini modellemek için bir sınıf kullanabilirsiniz. Sınıfın UML diyagramı Şekil 8.6'da gösterilmiştir.

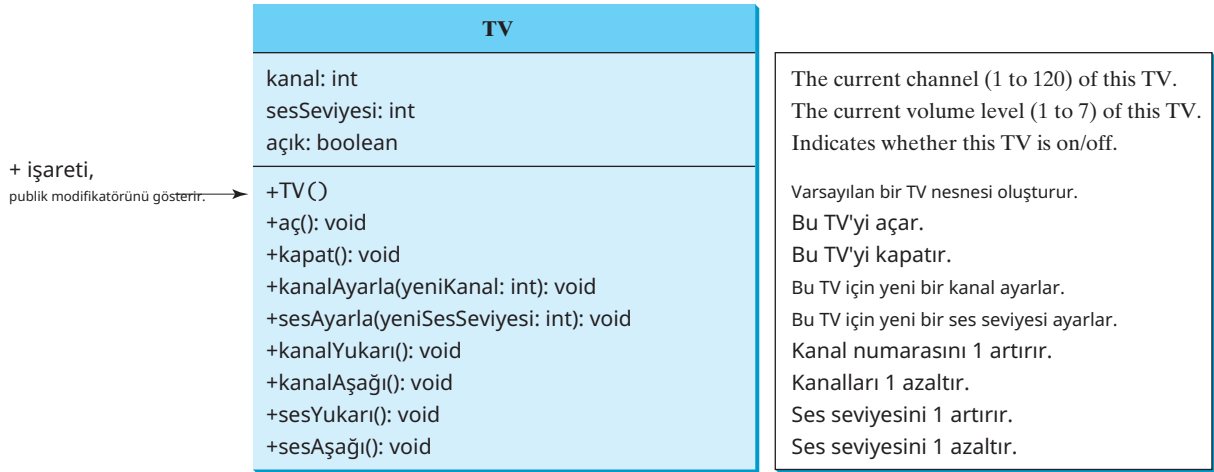


FIGURE 8.6 TV sınıfı TV setlerini modellemektedir.

Liste 8.3, **TV** sınıfını tanımlayan bir programı vermektedir.

Listing 8.3tTV.java

```

1  asstTV {
2      nnel = t1t;npublic cl // Varsayılan kanal 1'dir
3      olumeLevel = t1t;ninttcha // Varsayılan ses seviyesi 1'dir
4      oleanton = tfalset;ninttv // TV kapalı
5
6      publictTV() {nbo
7      }
8
9      genel voidtturnOn() {
10     açık = ttrue;t;n11 }
11
12
13     genel voidtturnOff() {

```

veri alanları

yapıcı

TV'yi aç

TV'yi kapat

302 Bölüm 8 Nesneler ve Sınıflar

```
14 açık = tfalset;n15 }

16
17
18     public void setKanal(int yeniKanal) {
19         eğer (on && newChannel >= 1 && newChannel <= 120)
20             kanal = newChannel;
21     }
22
23     public void setSes(int yeniSesSeviyesi) {
24         eğer (on && newVolumeLevel >= 1 && newVolumeLevel <= 7)
25             sesSeviyesi = newVolumeLevel;
26     }
27
28     public void kanalArttır() {
29         eğer (on && kanal < 120)
30             kanal++;
31     }
32
33     public void kanalAzalt() {
34         eğer (on && kanal > 1)
35             kanal--;
36     }
37
38     public void sesArttır() {
39         eğer (on && sesSeviyesi < 7)
40             sesSeviyesi++;
41     }
42
43     public void sesAzalt() {
44         eğer (on && sesSeviyesi > 1)
45             sesSeviyesi--;
46     }
```

Konstrüktör ve yöntemler, diğer sınıflardan erişilebilmeleri için public olarak tanımlanmıştır. TV açık değilse, kanal ve ses seviyesi değiştirilmez. Bu değerlerden herhangi biri değiştirilmeden önce, mevcut değeri kontrol edilir ve doğru aralıkta olup olmadığına bakılır.

Liste 8.4, TV sınıfını kullanarak iki nesne oluşturan bir programı vermektedir.

Listing 8.4 TestTV.java

```
1  genel sınıf TestTV {
2      public static void main(String[] args) {
3          TV tv1 = new TV();
4          tv1.turnOn(); n5 tv1.set
5          Channel(t30t); n6 tv1.setVolume(t
6          3t);
7
8          TV tv2 = new TV();
9          tv2.turnOn(); n10 tv2.ch
10         annelUp(); n11 tv2.channelUp()
11         ; n12 tv2.volumeUp();
12
13
14         System.out.println("tv1'in kanalı "+ tv1.kanal
15         + " ve ses seviyesi " + tv1.sesSeviyesi );
16         System.out.println("tv2'nin kanalı "+ tv2.channel
17         + " ve ses seviyesi " + tv2.volumeLevel);
18     }
19 }
```

```
tv1'in kanalı 30 ve ses seviyesi 3
tv2'nin kanalı 3 ve ses seviyesi 2
```



Program, 3. ve 8. satırlarda iki nesne oluşturur ve nesneler üzerinde kanalları ve ses seviyelerini ayarlamak ve kanalları ve sesleri artırmak için yöntemleri çağırır. Program, 14-17. satırlarda nesnelerin durumunu gösterir. Yöntemler, `tv1.turnOn()` (4. satır) gibi bir sözdizimi kullanılarak çağırılır. Veri alanlarına, `tv1.channel` (14. satır) gibi bir sözdizimi kullanılarak erişilir.

Bu örnekler, size sınıflar ve nesneler hakkında bir fikir verdi. Yapıcılar, nesneler, referans değişkenleri, veri alanlarına erişim ve nesne yöntemlerini çağırma ile ilgili birçok sorununuz olabilir. Aşağıdaki bölümler bu konuları ayrıntılı olarak tartışmaktadır.

- 8.1 Bir nesne ile onu tanımlayan sınıf arasındaki ilişkiyi tanımlayın.
- 8.2 Bir sınıfı nasıl tanımlarsınız?
- 8.3 Bir nesnenin referans değişkenini nasıl tanımlarsınız?
- 8.4 Bir nesne nasıl oluşturulur?

nokta ✓ Check

MyProgrammingLab™

8.4 Yapıcılar Kullanarak Nesnelerin Oluşturulması

*Bir nesne oluşturmak için **new** operatörü kullanılarak bir yapıcı çağrılır.*

Yapıcılar özel bir yöntem türüdür. Üç özelliği vardır:

- Bir yapıcının adı, sınıfın kendisiyle aynı olmalıdır.
- Yapıcıların bir dönüş türü yoktur—hatta **void** bile yoktur.
- Bir nesne oluşturulduğunda yapıcılar **new** operatörü kullanılarak çağrılır. Yapıcılar, nesneleri başlatma rolünü oynar.

Yapıcı, tanımlayıcı sınıfıyla tam olarak aynı adı taşır. Normal yöntemler gibi, yapıcılar da aşırı yüklenebilir (yani, birden fazla yapıcı aynı adı taşıyabilir ancak farklı imzalara sahip olabilir), bu da farklı başlangıç veri değerlerine sahip nesneleri oluşturmayı kolaylaştırır.

Bir yapıcının önüne **void** anahtar kelimesini koymak yaygın bir hatadır. Örneğin,

```
halk    geçersiz Daire() {
}
```



yapıcının adı

geri dönüş tipi yok

new operatörü

aşırı yüklenmiş yapıcılar

hayırvoid

Bu durumda, `Circle()` bir yöntemdir, yapıcı değil.

Yapıcılar, nesneleri oluşturmak için kullanılır. Bir sınıftan nesne oluşturmak için, sınıfın bir yapıcısını **new** operatörü ile çağırın, şu şekilde:

```
newClassName(argümanlar);
```

nesneleri oluşturma

Örneğin, `new Circle()`, `Circle` sınıfında tanımlanan ilk yapıcıyı kullanarak `Circle` sınıfının bir nesnesini oluşturur ve `new Circle(25)`, `Circle` sınıfında tanımlanan ikinci yapıcıyı kullanarak bir nesne oluşturur.

Bir sınıf genellikle argüman içermeyen bir yapıcı sağlar (örneğin, `Circle()`). Böyle bir yapıcıya *no-arg* veya *no-argument constructor* denir.

Bir sınıf yapıcılar olmadan tanımlanabilir. Bu durumda, sınıfta boş bir gövdeye sahip bir public no-arg yapıcı otomatik olarak tanımlanır. Bu yapıcıya *default constructor* denir ve *yalnızca sınıfta açıkça tanımlanmış yapıcı yoksa sağlanır*.

parametresiz yapıcı

varsayılan yapıcı

nokta ✓ Check

MyProgrammingLab™

8.5 Yapıcılar ile yöntemler arasındaki farklar nelerdir?

8.6 Bir sınıf ne zaman varsayılan bir yapıcıya sahip olur?



8.5 Nesnelere Referans Değişkenleri ile Erişim

Bir nesnenin verilerine ve yöntemlerine, nesnenin referans değişkeni aracılığıyla nokta (.) operatörü ile erişilebilir.

Yeni oluşturulan nesneler bellekte tahsis edilir. Referans değişkenleri aracılığıyla erişilebilirler.

8.5.1 Referans Değişkenleri ve Referans Türleri

referans değişkeni

Nesnelere, nesnenin *referans değişkenleri* aracılığıyla erişilir; bu değişkenler nesnelere referanslar içerir. Bu tür değişkenler aşağıdaki sözdizimi kullanılarak tanımlanır:

```
SınıfAdı nesneRefDeğişkeni;
```

referans türü

Bir sınıf esasen programcı tarafından tanımlanan bir türdür. Bir sınıf, *referans türü* olup, bu da sınıf türündeki bir değişkenin sınıfın bir örneğine referans verebileceği anlamına gelir. Aşağıdaki ifa de, **myCircle** değişkenini **Circle** türünde olarak tanımlar:

```
Daire myCircle;
```

Değişken **myCircle** bir **Daire** nesnesine referans verebilir. Bir sonraki ifade bir nesne oluşturur ve referansını **myCircle** değişkenine atar:

```
myCircle = new Circle();
```

Bir nesne referans değişkeninin bildirimini,

bir nesne oluşturmayı ve bir nesne referansını değişkene atamayı birleştiren tek bir ifade yazabilirsiniz şu sözdizimi ile:

```
ClassName objectRefVar = new ClassName();
```

İşte bir örnek:

```
Daire myCircle = newDaire();
```

Değişken **myCircle** bir **Daire** nesnesine referans tutar.

nesne vs. nesne referansı
değişkeni



Not

Bir nesne referans değişkeni, bir nesne tutuyormuş gibi görünsede aslında o nesneye referans içerir. Kesin olarak konuşursak, bir nesne referans değişkeni ve bir nesne farklıdır, bununla birlikte çoğu zaman bu ayrım göz ardı edilebilir. Bu nedenle, basitlik açısından, **myCircle** bir **Daire** nesnesidir demek uygundur, daha uzun açıklama kullanmak yerine **myCircle** bir **Daire** nesnesine referans içeren bir değişkendir demek yerine.

dizi nesnesi



Not

Diziler Java'da nesne olarak kabul edilir. Diziler **new** operatörü kullanılarak oluşturulur. Bir dizi değişkeni aslında bir diziye referans içeren bir değişkendir.

8.5.2 Bir Nesnenin Verilerine ve Yöntemlerine Erişim

nokta operatörü (.)

OOP terminolojisinde, bir nesnenin üyesi onun veri alanlarına ve yöntemlerine atıfta bulunur. Bir nesne oluşturulduktan sonra, verilerine erişilebilir ve yöntemleri *nokta operatörü* (.) kullanılarak çağırılabilir, aynı zamanda *nesne üye erişim operatörü* olarak da bilinir:

■ **objectRefVar.dataField**, nesnedeki bir veri alanını referans alır.

■ **objectRefVar.method(arguments)**, nesne üzerinde bir metodu çağırır.

Örneğin, `myCircle.radius` noktasını referans alır `myCircle`, `vemyCircle.getArea()`, `getArea` yöntemini `myCircle` üzerinde çağırır. Yöntemler, nesneler üzerinde işlemler olarak çağırılır.

Veri alanı `radius`, belirli bir örneğe bağımlı olduğu için *instance variable* olarak adlandırılır. Aynı nedenle, `getArea` metodu *instancemethod* olarak adlandırılır, çünkü yalnızca belirli bir örnek üzerinde çağırılabilir. Bir örnek metodunun çağırıldığı nesne *calling object* olarak adlandırılır.

örnek değişken
örnek yöntem
çağırılan nesne



Dikkat

Hatırlayın ki, `Math.methodName(arguments)` (örneğin, `Math.pow(3, 2.5)`) kullanarak `Math` sınıfında bir yöntemi çağırıyorsunuz. `getArea()`'yi `Circle.getArea()` kullanarak çağırabilir misiniz? Cevap hayır. `Math` sınıfındaki tüm yöntemler statik yöntemlerdir ve `static` anahtar kelimesi kullanılarak tanımlanmıştır. Ancak, `getArea()` bir örnek yöntemi değildir ve dolayısıyla statik değildir. Bir nesneden `objectRefVar.methodName(arguments)` (örneğin, `myCircle.getArea()`) kullanılarak çağırılmalıdır. Daha fazla açıklama Bölüm 8.7, Statik Değişkenler, Sabitler ve Yöntemler'de verilmiştir.

yöntemleri çağırma



Not

Genellikle bir nesne oluşturur ve onu bir değişkene atarsınız, ardından daha sonra bu değişkeni nesneye atıfta bulunmak için kullanabilirsiniz. Bazen bir nesne daha sonra atıfta bulunulmasına gerek duymaz. Bu durumda, bir nesneyi açıkça bir değişkene atmadan oluşturabilirsiniz:

```
newtCircle();
```

veya

```
System.out.println("Alan "+newtCircle(5).getArea());
```

İlk ifade bir `Circle` nesnesi oluşturur. İkincisi bir `Circle` nesnesi oluşturur ve alanını döndürmek için `getArea` yöntemini çağırır. Bu şekilde oluşturulan bir nesne *anonym nesne* olarak bilinir.

anonim nesne

8.5.3 Referans Veri Alanları ve `null` Değeri

Veri alanları referans türlerinde olabilir. Örneğin, aşağıdaki `Student` sınıfı bir `String` türünde `name` veri alanı içerir. `String`, önceden tanımlanmış bir Java sınıfıdır.

referans veri alanları

```
sınıfÖğrenci {
    String name; // name'in varsayılan değeri null
    int age; // age'in varsayılan değeri 0
    boolean isScienceMajor; // isScienceMajor varsayılan değeri false'dur
    char gender; // gender varsayılan değeri "
}
```

Eğer bir referans türünün veri alanı herhangi bir nesneye referans vermiyorsa, veri alanı özel bir Java değeri olan `null`'ı tutar. `null`, `true` ve `false` gibi bir literaldir. `true` ve `false`

`null` değeri

Boolean literalleri iken, `null` bir referans türü için bir literaldir.

Bir veri alanının varsayılan değeri, bir referans türü için `null`, bir sayısal tür için `0`, bir `boolean` türü için `false` ve bir `char` türü için `\u0000`'dir. Ancak, Java bir yöntem içindeki yerel bir değişkene varsayılan bir değer atamaz. Aşağıdaki kod, bir `Student` nesnesi için veri alanlarının varsayılan değerlerini `name`, `age`, `isScienceMajor` ve `gender` olarak gösterir:

```
sınıfTest {
    public static void main(String[] args) {
        Öğrenci öğrenci = newtStudent();
        println("name " + öğrenci.isim );
    }
}
```

```

System.out.println("yaş? " + öğrenci.yaş );
System.out.println("bilim dalı mı? " + öğrenci.bilimAnaDalında );
System.out.println("cinsiyet? " + öğrenci.cinsiyet );
}
}

```

Aşağıdaki kodda bir derleme hatası vardır, çünkü yerel değişkenler **x** ve **y** başlatılmamıştır:

```

sınıfTest {
    public static void main(String[] args) {
        ttx; // x'in varsayılan bir değeri yok
        String y;nin // y'nin varsayılan bir değeri yok
        System.out.println("x " + ); x
        System.out.println("y " + ); y
    }
}

```

NullPointerException



Dikkat

NullPointerException yaygın bir çalışma zamanı hatasıdır. Bu, bir referans değişkeninde **null** değeri olan bir metodu çağırdığınızda meydana gelir. Metodu referans değişkeni aracılığıyla çağırmadan önce değişkene bir nesne referansı atadığınızdan emin olun.

8.5.4 Temel Türlerin ve Referans Türlerinin Değişkenleri Arasındaki Farklar

Her değişken, bir değeri tutan bir bellek konumunu temsil eder. Bir değişken tanımladığınızda, değişkenin hangi tür değeri tutabileceğini derleyiciye söylüyorsunuz. Bir ilkel türdeki değişken için, değer ilkel türdedir. Bir referans türdeki değişken için, değer bir nesnenin nerede bulunduğu dair bir referanstır. Örneğin, Şekil 8.7'de gösterildiği gibi, **int** değişkeni **i**'nin değeri **int** değeri **1**'dir, yani **Circle** nesnesi **c**'nin değeri, **Circle** nesnesinin bellekte nerede saklandığına dair bir referans tutar.

Bir değişkeni diğerine atadığınızda, diğer değişken aynı değere ayarlanır. Bir ilkel türdeki değişken için, bir değişkenin gerçek değeri diğer değişkene atanır. Bir referans türündeki değişken için, bir değişkenin referansı diğer değişkene atanır. Şekil 8.8'de gösterildiği gibi, atama ifadesi **i = j**, **j**'nin içeriğini **i**'ye kopyalar

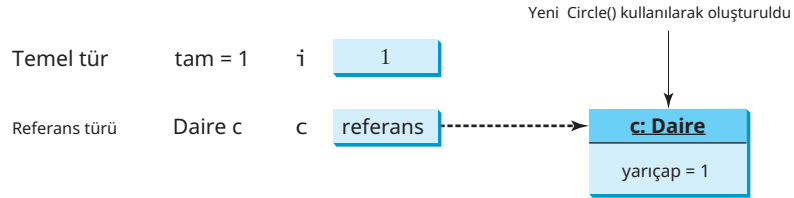


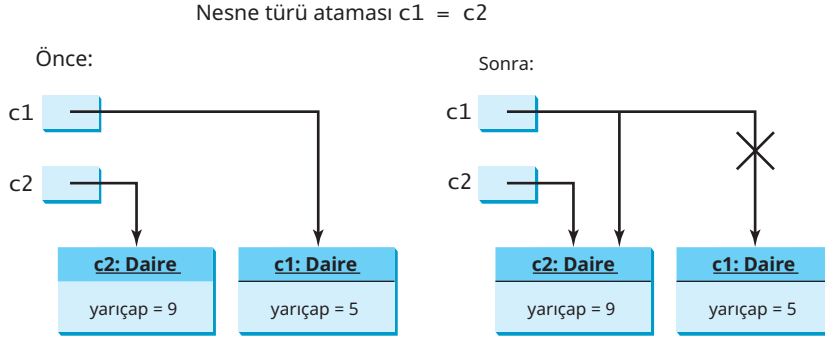
FIGURE 8.7 Bir temel türün değişkeni, temel türün bir değerini tutar ve bir referans türünün değişkeni bir nesnenin bellekte nerede saklandığına dair bir referans tutar.

Temel tür ataması **i = j**



FIGURE 8.8 Temel değişken **j** değişkenine kopyalanır **i**.

ilkel değişkenler. Şekil 8.9'da gösterildiği gibi, atama ifadesi `c1 = c2`, referans değişkenleri için `c2`'nin referansını `c1`'e kopyalar. Atamadan sonra, değişkenler `c1` ve `c2` aynı nesneye referans verir.



FGÖRÜNTÜ 8.9 Referans değişkeni `c2`, değişken `c1`'e kopyalanır.



Not

Şekil 8.9'da gösterildiği gibi, atama ifadesi `c1 = c2`, `c1`'in `c2` tarafından referans verilen aynı nesneye işaret ettiğini gösterir. Daha önce `c1` tarafından referans verilen nesne artık kullanışlı değildir ve bu nedenle *çöp* olarak bilinir. Çöp, bellek alanını kaplar, bu nedenle Java çalışma sistemi çöpü tespit eder ve kapladığı alanı otomatik olarak geri alır. Bu işleme *çöp toplama* denir.

çöp

çöp toplama



Titip

Eğer bir nesnenin artık gerekli olmadığını biliyorsanız, nesne için bir referans değişkenine `null` atayabilirsiniz. JVM, nesne herhangi bir referans değişkeni tarafından referans edilmediğinde otomatik olarak alanı toplayacaktır.

nokta ✓ Check

MyProgrammingLab™

- 8.7** Bir veri alanına erişmek veya bir nesneden bir yöntemi çağırmak için hangi operatör kullanılır?
- 8.8** Anonim bir nesne nedir?
- 8.9** `NullPointerException` nedir?
- 8.10** Bir dizi bir nesne mi yoksa bir ilkel tür değeri mi? Bir dizi, hem nesne türü hem de ilkel türdeki elemanları içerebilir mi? Bir dizinin elemanları için varsayılan değeri tanımlayın.
- 8.11** Aşağıdaki her bir programda ne yanlış?

```
1 genel classtShowErrors {
2     public static voidtmain(String[] args) {
3 ShowErrors t = tnewtShowErrors(t5t);n4 }
5 }
```

(a)

```
1 genel classtShowErrors {
2     public static voidtmain(String[] args) {
3 ShowErrors t = tnewtShowErrors();n4 t.x();
5 }
```

(b)

```

1 genel classtShowErrors {
2     genel voidtmethod1() {
3 Daire c;
4         System.out.println("Yarıçap nedir "
5             + c.getRadius());
6 c = tnewtCircle();n7 }
8     }

```

(c)

```

1 genel classtShowErrors {
2     public static voidtmain(String[] args) {
3 C c = tnewtC(t5.0t);n4 Syste
m.out.println(c.value);n5 }
6     }
7
8     sınıfC {
9         intdeğer = 2;
10    }

```

(d)

8.12 Aşağıdaki kodda ne yanlış?

```

1 sınıfTest {
2     public static voidtmain(String[] args) {
3 A a = tnewtA();n4 a.print(
);n5 }
6     }
7
8     classtA {n9
String s;
10
11 A(Dize yeniS) {n12 s = ye
niS;
13     }
14
15     public voidtprint() {n16 Sys
tem.out.print(s);n17 }
18     }

```

8.13 Aşağıdaki kodun çıktısı nedir?

```

genel sınıfA {
    özel booleanx;

    public static voidtmain(String[] args) {nA a = tnewt
A();
    System.out.println(a.x);n}
}

```

8.6 Java Kütüphanesinden Sınıflar Kullanma



Java API'si, Java programları geliştirmek için zengin bir sınıf seti içerir.

Liste 8.1, **SimpleCircle** sınıfını tanımladı ve sınıftan nesneler oluşturdu. Java kütüphanesinin deki sınıfları program geliştirmek için sıkça kullanacaksınız. Bu bölüm, Java kütüphanesindeki sınıflara bazı örnekler verir.

8.6.1 Date Sınıfı

Liste 2.6'da, ShowCurrentTime.java dosyasında, mevcut zamanı elde etmeyi öğrendiniz **System.currentTimeMillis()**. Çıkarma işlemi ve kalan operatörlerini kullanarak

mevcut saniye, dakika ve saat. Java, tarih ve saati **java.util.Date** sınıfında sistem bağımsız bir kapsülleme sağlar, Şekil 8.10'da gösterildiği gibi.

java.util.Date sınıfı

java.util.Date	
+Tarih() +Tarih(elapseTime: long)n+to String(): Stringn+getTim e(): long +setTime(elapseTime: uzun): geçersiz	Constructs a Date object for the current time. Constructs a Date object for a given time in milliseconds elapsed since January 1, 1970, GMT. Returns a string representing the date and time. Returns the number of milliseconds since January 1, 1970, GMT. Sets a new elapse time in the object.

FIGURE 8.10 Bir Date nesnesi belirli bir tarih ve saati temsil eder.

Argüman almayan yapıcıyı **Date** sınıfında kullanarak mevcut tarih ve saat için bir örnek oluşturabilirsiniz, **getTime()** yöntemi 1 Ocak 1970, GMT'den bu yana geçen süreyi döndürür ve **toString()** yöntemi tarihi ve saati bir dize olarak döndürür. Örneğin, aşağıdaki kod

```
java.util.Date tarih = new java.util.Date();  
System.out.println("1 Ocak 1970'den bu yana geçen süre "+  
    tarih.getTime() + " milisaniye");  
System.out.println(tarih.toString());
```

nesne oluştur

geçen süreyi al
toString

şu çıktıyı gösterir:

```
1 Ocak 1970'den bu yana geçen süre 1324903419651 milisaniye  
Paz Dec 26 07:43:39 EST 2011
```

Date sınıfının başka bir yapıcısı vardır, **Date(long elapseTime)**, bu yapıcı 1 Ocak 1970, GMT'den bu yana geçen milisaniye cinsinden belirli bir zaman için bir Date nesnesi oluşturmak için kullanılabilir.

8.6.2 Random Sınıfı

Rastgele bir **double** değeri elde etmek için **Math.random()** kullanınız **0.0** ile **1.0** (arasında **1.0** hariç). Rastgele sayılar üretmenin bir diğer yolu, Şekil 8.11'de gösterildiği gibi **java.util.Random** sınıfını kullanmaktır; bu sınıf rastgele bir **int**, **long**, **double**, **float** ve **boolean** değeri üretebilir.

boolean değeri üretebilir.

java.util.Random	
+rastgele() +Random(seed: long)n+ nextInt(): int n+next tInt(n: int): intn+nextLong() : longn+nextDouble(): double n+nextFloat(): float n+nextBoolean(): boolean an	Constructs a Random object with the current time as its seed. Constructs a Random object with a specified seed. Returns a random int value. Returns a random int value between 0 and n (excluding n). Returns a random long value. Returns a random double value between 0.0 and 1.0 (excluding 1.0). Returns a random float value between 0.0F and 1.0F (excluding 1.0F). Returns a random boolean value.

ŞEKİL 8.11 Rastgele bir nesne rastgele değerler üretmek için kullanılabilir.

310 Bölüm 8 Nesneler ve Sınıflar

Bir **Random** nesnesi oluşturduğunuzda, bir tohum belirtmeniz veya varsayılan tohumu kullanmanız gerekir. Birtohum, rastgele bir sayı üreticiyi başlatmak için kullanılan bir sayıdır. Parametresi z yapıcı, mevcut geçen zamanı tohum olarak kullanarak bir **Random** nesnesi oluşturur. Eğer iki **Random** nesnesi aynı tohuma sahipse, aynı sayı dizilerini üreteceklerdir. Örneğin, aşağıdakikod, aynı tohum olan iki **Random** nesnesi oluşturur, 3.

```
Random random1 = new Random(3);
System.out.print("random1'den: ");
for (int i = 0; i < 10; i++)
    System.out.print(random1.nextInt(t1000t) + t" ");nRando

m random2 = tnewtRandom(t3t);
System.out.print("random2'den: ");for (int
i = 0; i < 10; i++)
    System.out.print(random2.nextInt(t1000t) + t" ");
```

Bu kod, aynı rastgele **int** değerleri dizisini üretir:

```
random1'den: 734 660 210 581 128 202 549 564 459 961
random2'den: 734 660 210 581 128 202 549 564 459 961
```

aynı sıra



Not

Aynı rastgele değerler dizisini üretme yeteneği, yazılım testinde ve birçok diğer uygulamada faydalıdır. Yazılım testinde, genellikle sabit bir rastgele sayı dizisinden test vakalarını yeniden üretmeniz gerekir.

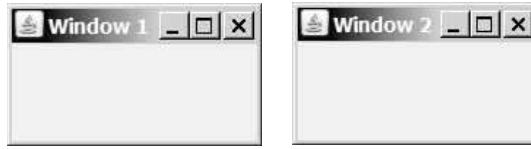
8.6.3 GUI Bileşenlerini Gösterme



Petdagtotgtitcal Ntote

Grafiksel kullanıcı arayüzü (GUI) bileşenleri, OOP öğretimi için iyi örneklerdir. Bu amaçla basitGUI örnekleri burada tanıtılmaktadır. GUI programlamaya tam giriş, Bölüm 12, GUI Temelleri ile başlar.

Grafiksel kullanıcı arayüzleri oluşturmak için programlar geliştirirken, çerçeveler, düğmeler, radyo düğmeleri, açılır kutular, listeler vb. oluşturmak için **JFrame**, **JButton**, **JRadioButton**, **JComboBox** ve **JList** gibi Java sınıflarını kullanacaksınız. Liste 8.5, **JFrame** sınıfını kullanarak iki pencere oluşturan bir örnektir. Programın çıktısı Şekil 8.12'de gösterilmektedir.



ŞEKİL 8.12Program, **JFrame** sınıfını kullanarak iki pencere oluşturur.

LISTELEN8.5TestFrame.java

```
1 import javax.swing.JFrame;
2
3 genel classtTestFrame {
4     public static voidtmain(String[] args) {
5         JFrame çerçeve1 = tnewtJFrame();
6 çerçeve1.setTitle(t"Pencere 1");n7 çerçeve
1.setSize(t200t,t150t);n8 çerçeve1.setLoc
ation(t200t,t100t);n9 çerçeve1.setDefaultClose
Operation(JFrame.EXIT_ON_CLOSE);n10 çerçeve1.setVisible(tttrue);
```

bir nesne oluştur
bir yöntemi çağır

```

11 JFrame çerçeve2 = tnewtJFrame();
12 çerçeve2.setTitle(t"Pencere 2");
13 çerçeve2.setSize(t200t,t150t);
14 çerçeve2.setLocation(t410t,t100t);
15 çerçeve2.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16 çerçeve2.setVisible(ttrue);
17
18 }
19

```

bir nesne oluştur
bir yöntemi çağır

Bu program, **JFrame** sınıfının iki nesnesini oluşturur (satırlar 5, 12) ve ardından **setTitle**, **setSize**, **setLocation**, **setDefaultCloseOperation** ve **setVisible** yöntemlerini kullanarak nesnelerin özelliklerini ayarlar. **setTitle** yöntemi, pencere için bir başlık ayarlar (satırlar 6,13). **setSize** yöntemi, pencerenin genişliğini ve yüksekliğini ayarlar (satırlar 7, 14). **setLocation** yöntemi, pencerenin sol üst köşesinin konumunu belirtir (satırlar 8, 15). **setDefaultCloseOperation** yöntemi, çerçeve kapatıldığında programı sonlandırır (satırlar 9, 16). **setVisible** yöntemi, pencereyi görüntüler.

Grafik kullanıcı arayüzü bileşenleri, düğmeler, etiketler, metin alanları, onay kutuları ve açılır kutular gibi pencereye ekleyebilirsiniz. Bileşenler sınıflar kullanılarak tanımlanır. Liste 8.6, Şekil 8.1'de gösterildiği gibi bir grafik kullanıcı arayüzü oluşturma örneği vermektedir.

LİSTE 8.6 GUIComponents.java

```

1  import javax.swing.*;
2
3  genel sınıf GUI Bileşenleri {
4      public static void main(String[] args) {
5          // Metni OK olan bir düğme oluştur
6          JButton jbtOK = new JButton("OK");
7
8          // Metni İptal olan bir düğme oluştur
9          JButton jbtCancel = new JButton("Cancel");
10
11         // Metni "Adınızı girin: " olan bir etiket oluştur
12         JLabel lblName = new JLabel("Enter your name: ");
13
14         // Metni "Buraya Adınızı Yazın" olan bir metin alanı oluştur
15         JTextField jtfName = new JTextField("Type Name Here");
16
17         // Metni Kalın olan bir onay kutusu oluştur
18         JCheckBox jchkBold = new JCheckBox("Bold");
19
20         // Metin İtalik olan bir onay kutusu oluştur
21         JCheckBox jchkItalic = new JCheckBox("Italic");
22
23         // Metin Kırmızı olan bir radyo düğmesi oluştur
24         JRadioButton jrbRed = new JRadioButton("Red");
25
26         // Metin Sarı olan bir radyo düğmesi oluştur
27         JRadioButton jrbYellow = new JRadioButton("Yellow");
28
29         // Birkaç seçenekle bir kombinasyon kutusu oluştur
30         JComboBox jcbColor = new JComboBox(new String[] {"Freshman",
31             "İkinci sınıf", "Üçüncü sınıf", "Dördüncü sınıf"});
32
33         // Bileşenleri gruplamak için bir panel oluştur
34         JPanel panel = new JPanel();
35         panel.add(jbtOK); // OK butonunu panele ekle
36         panel.add(jbtCancel); // İptal butonunu panele ekle

```

bir buton oluştur

bir buton oluştur

bir etiket oluştur

bir metin alanı oluştur

bir onay kutusu oluştur

bir onay kutusu oluştur

bir radyo butonu oluştur

bir radyo butonu oluştur

bir kombinasyon kutusu oluştur

bir panel oluştur
panele ekle



VideoNotu

Sınıfları kullanın

312 Bölüm 8 Nesneler ve Sınıflar

bir çerçeve oluştur
çerçeveye panel ekle

çerçeveyi göster

```
37 panel.add(jlblName); // Etiket paneli ekle
38 panel.add(jtfName); // Metin alanını paneli ekle
39 panel.add(jchkBold); // Onay kutusunu paneli ekle
40 panel.add(jchkItalic); // Onay kutusunu paneli ekle
41 panel.add(jrbRed); // Radyo düğmesini paneli ekle
42 panel.add(jrbYellow); // Radyo düğmesini paneli ekle
43 panel.add(jcboColor); // Kombinasyon kutusunu paneli ekle
44
45 Jafarmen Fermen = new JFrame(); // Bir çerçeve oluştur
46 frame.add(panel); // Paneli çerçeveye ekle
47 frame.setTitle("GUI Bileşenlerini Göster");
48 çerçeve.setSize(450t,100t);n49 çerçeve.setLocation(200t,100t);n50 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);n51 çerçeve.setVisible(true);
52 {naha
}
```

Bu program, **JButton**, **JLabel**, **JTextField**, **JCheckBox**, **JRadioButton** ve **JComboBox** sınıflarını kullanarak GUI nesneleri oluşturur (satırlar 6–31). Daha sonra, **JPanel** sınıfını kullanarak (satır 34) bir panel nesnesi oluşturur ve buton, etiket, metin alanı, onay kutusu, radyo butonu ve açılır kutuyu buna ekler (satırlar 35–43). Program daha sonra bir çerçeve oluşturur ve paneli çerçeveye ekler (satır 45). Çerçeve, satır 51'de görüntülenir.

MyProgrammingLab™

nokta Check

8.14 Mevcut zaman için bir **Date** nasıl oluşturursunuz? Mevcut zamanı nasıl görüntülersiniz?

8.15 Bir **JFrame** nasıl oluşturulur, bir çerçevede başlık nasıl ayarlanır ve bir çerçeve nasıl görüntülenir?

8.16 Hangi paketler **Date**, **JFrame**, **JOptionPane**, **System** ve

Math sınıflarını içerir?

8.7 Statik Değişkenler, Sabitler ve Metotlar

Statik bir değişken, sınıfın tüm nesneleri tarafından paylaşılır. Statik bir metot, sınıfın örnek üyelerine erişemez.

Daire sınıfındaki **radius** veri alanı, *instance variable* olarak bilinir. Bir örnek değişken, sınıfın belirli bir örneğine bağlıdır; aynı sınıfın nesneleri arasında paylaşılmaz. Örneğin, aşağıdaki nesneleri oluşturduğunuzu varsayalım:

```
Daire daire1 = tnewtCircle();nCircle dai
re2 = tnewtCircle(t5t);
```

Çember1'deki yarıçap, çember2'deki yarıçaptan bağımsızdır ve farklı bir bellek konumunda saklanır. Çember1'in yarıçapında yapılan değişiklikler, çember2'nin yarıçapını etkilemez ve tersine.

Bir sınıfın tüm örneklerinin verileri paylaşmasını istiyorsanız, sınıf değişkenleri olarak da bilinen statik değişkenler kullanın. Statik değişkenler, değişkenler için ortak bir bellek konumunda değerleri saklar. Bu ortak konum nedeniyle, bir nesne bir statik değişkenin değerini değiştirirse, aynı sınıfın tüm nesneleri etkilenir. Java, statik değişkenlerin yanı sıra statik yöntemleri de destekler. *Statik yöntemler, sınıfın bir örneğini oluşturmadan çağrılabilir.*

Çember nesnelerinin sayısını saymak için statik bir değişken olan **numberOfObjects**'u ekleyerek **Circle** sınıfını değiştirelim. Bu sınıfın ilk nesnesi oluşturulduğunda, **numberOfObjects** 1'dir. İkinci nesne oluşturulduğunda, **numberOfObjects** 2 olur. Yeni çember sınıfının UML'si Şekil 8.13'te gösterilmiştir. **Circle** sınıfı, *instance* değişkeni **radius** ve statik değişkeni **numberOfObjects**'u, *instance* yöntemleri **getRadius**, **setRadius** ve **getArea** ile statik yöntem **getNumberOfObjects**'u tanımlar. (Statik değişkenler ve yöntemler UML sınıf diyagramında altı çizili olarak gösterilmiştir.)



VideoNotu

Statik vs. örnek
örnek değişken

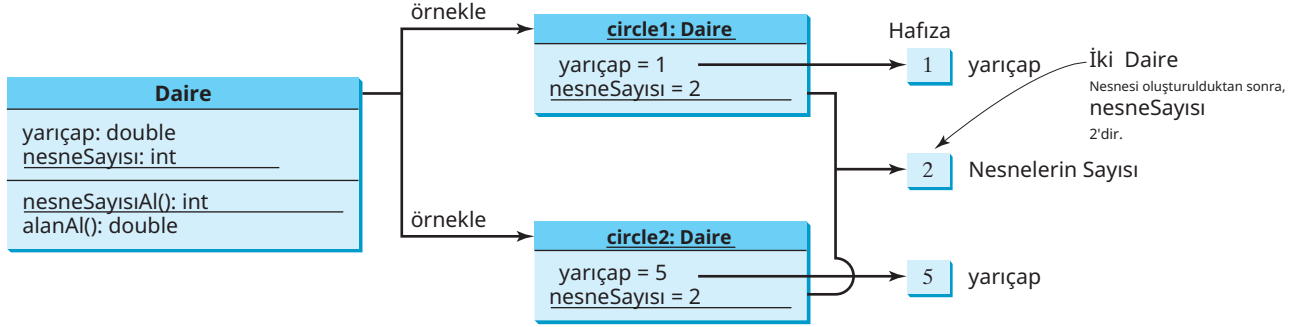
Ana
Nokta

statik değişken

statik yöntem

UML Notasyonu:

altını çizmek: statik değişkenler veya yöntemler



FİKON 8.13 Örnek değişkenler örneklere aittir ve birbirinden bağımsız bellek depolama alanına sahiptir. Statik değişkenler aynı sınıfın tüm örnekleri tarafından paylaşılır.

Statik bir değişkeni tanımlamak veya statik bir yöntemi tanımlamak için, değişken veya yöntem bildiriminde **static** değiştiricisini koyun. Statik değişken **numberOfObjects** ve statik yöntem **getNumberOfObjects()** aşağıdaki gibi tanımlanabilir:

statik `int numberOfObjects;`

statik değişkeni tanımla

Statik s `ttgetNumberOfObjects() {return
 numberOfObjects;
}`

statik yöntemi tanımla

Sınıftaki sabitler, sınıfın tüm nesneleri tarafından paylaşılır. Bu nedenle, sabitler

sabit tanımla

final static olarak tanımlanmalıdır. Örneğin, Math sınıfındaki sabit `PI` şu şekilde tanımlanmıştır:

`son statik ikiliPI = t3.14159265358979323846t;`

Yeni daire sınıfı, **CircleWithStaticMembers** adıyla, Liste 8.7'de tanımlanmıştır:

LISTING 8.7 CircleWithStaticMembers.java

```

1 public class CircleWithStaticMembers {
2     /** Dairenin yarıçapı */
3     double radius;
4
5     /** Oluşturulan nesne sayısı */
6     static int numberOfObjects = 0;
7
8     /** Yarıçapı 1 olan bir daire oluştur */
9     CircleWithStaticMembers() {
10         radius = 1;
11         nesnelerin sayısı++;
12     }
13
14     /** Belirtilen bir yarıçap ile bir daire oluştur */
15     CircleWithStaticMembers(double newRadius) {
16         radius = newRadius;
17         nesnelerin sayısı++;
18     }
19
20     /** numberOfObjects döndür */
21     static int getNumberOfObjects() {
22         return numberOfObjects;
23     }
24 }

```

statik değişken

1 artır

1 artır

statik yöntem

314 Bölüm 8 Nesneler ve Sınıflar

```
25         /** Bu dairenin alanını döndür */
26     çiftgetArea() {
27         returnradius * yarıçap * Math.PI;
28     }
29     {eksik
30     lik}
```

Yöntem `getNumberOfObjects()` `CircleWithStaticMembers` içinde statik bir yöntemdir. Statik yöntemlere diğer örnekler `showMessageDialog` ve `showInputDialog` dır `JOptionPane` sınıfında ve `Math` sınıfındaki tüm yöntemlerdir. `main` yöntemi de statiktir.

Örnek yöntemler (örneğin, `getArea()`) ve örnek veriler (örneğin, `radius`) örneklerle aittir ve yalnızca örnekler oluşturulduktan sonra kullanılabilir. Bir referans değişkeni aracılığıyla erişilir. Statik yöntemler (örneğin, `getNumberOfObjects()`) ve statik veriler (örneğin, `numberOfObjects`) bir referans değişkeni veya sınıf adı aracılığıyla erişilebilir.

Liste 8.8'deki program, örnek ve statik değişkenler ile

yöntemlerin nasıl kullanılacağını gösterir ve bunların kullanımının etkilerini açıklar.

LISTE 8.8 TestCircleWithStaticMembers.java

```
1     public class TestCircleWithStaticMembers {
2         /** Ana yöntem */
3         public static void main(String[] args) {
4             System.out.println("Objeleri oluşturmadan önce");
5             System.out.println("Daire nesnelerinin sayısı "+
6                 DaireStatikÜyeleri.nesneSayısı );
7
8             // c1 oluştur
9             DaireStatikÜyeleri c1 = yeni DaireStatikÜyeleri();
10
11             // c2 oluşturulmadan önce c1'i göster
12             System.out.println("c1 oluşturulduktan sonra");
13             System.out.println("c1: yarıçap (" + c1.radius +
14                 ") ve Daire nesnelerinin sayısı (" +
15                 c1.nesneSayısı + ")");
16
17             // c2 oluştur
18             CircleWithStaticMembers c2 = yeniCircleWithStaticMembers(5);
19
20             // c1'i değiştir
21             c1.yarıçap = 9;
22
23             // c2 oluşturulduktan SONRA c1 ve c2'yi göster
24             System.out.println("c2 oluşturulduktan ve c1 değiştirildikten sonra");
25             System.out.println("c1: yarıçap (" + c1.yarıçap +
26                 ") ve Daire nesnelerinin sayısı (" +
27                 c1.nesneSayısı + ")");
28             System.out.println("c2: yarıçap (" + c2.yarıçap +
29                 ") ve Daire nesnelerinin sayısı (" +
30                 c2.nesneSayısı + ")");
31         }
32     }
```

statik değişken

örnek değişken

statik değişken

örnek değişken

statik değişken

statik değişken



Objeler oluşturmadan önce
Daire nesnelerinin sayısı 0
c1 oluşturulduktan sonra
c1: yarıçap (1.0) ve Daire nesnelerinin sayısı (1)
c2 oluşturulduktan ve c1 değiştirildikten sonra
c1: yarıçap (9.0) ve Daire nesnelerinin sayısı (2)
c2: yarıçap (5.0) ve Daire nesnelerinin sayısı (2)

TestDaireStatikÜyeleri.java derlediğinizde, Java derleyicisi otomatik olarak DaireStatikÜyeleri.java dosyasını derler, eğer son değişiklikten bu yana derlenmemişse.

Statik değişkenler ve yöntemler, nesneler oluşturmadan erişilebilir. 6. satır, oluşturulmuş nesne olmadığı için nesne sayısını 0 olarak gösterir.

main yöntemi iki daire oluşturur, c1 ve c2 (satırlar 9, 18). c1 içindeki örnek değişken yarıçap 9 (satır 21) olacak şekilde değiştirilir. Bu değişiklik, c2 içindeki örnek değişken yarıçap üzerinde etkili olmaz, çünkü bu iki örnek değişken bağımsızdır. Statik değişken nesneSayısı c1 oluşturulduktan sonra 1 olur (satır 9), ve c2 oluşturulduktan sonra 2 olur (satır 18).

Not edin ki **PI**, **Math** içinde tanımlanmış bir sabittir ve **Math.PI** bu sabiti referans alır. **c1.numberOfObjects** (satır 27) ve **c2.numberOfObjects** (satır 30) yerine **CircleWithStaticMembers.numberOfObjects** kullanmak daha iyidir. Bu, okunabilirliği artırır, çünkü diğer programcılar statik değişkeni kolayca tanıyabilir. Ayrıca,

DaireStatikÜyeleri.nesneSayısı şunu **CircleWithStaticMembers.getNumberOfObjects()** ile değiştirebilirsiniz.

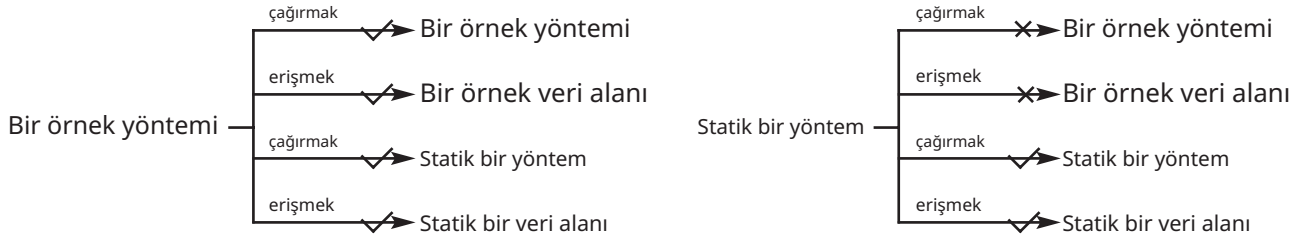


Tipp

Statik bir yöntemi çağırmak için **ClassName.methodName(arguments)** ve bir statik değişkene erişmek için **ClassName.staticVariable** kullanın. Bu, okunabilirliği artırır, çünkü diğer programcılar sınıftaki statik yöntem ve veriyi kolayca tanıyabilir.

sınıf adını kullan

Bir örnek yöntemi, bir örnek veya statik yöntemi çağırabilir ve bir örnek veya statik veri alanına erişebilir. Bir statik yöntem, bir statik yöntemi çağırabilir ve bir statik veri alanına erişebilir. Ancak, bir statik yöntem, bir örnek yöntemini çağıramaz veya bir örnek veri alanına erişemez, çünkü statik yöntemler ve statik veri alanları belirli bir nesneye ait değildir. Statik ve örnek üyeler arasındaki ilişki aşağıdaki diyagramda özetlenmiştir:



Örneğin, aşağıdaki kod yanlıştır.

```

1  genel sınıfA {
2      int = t5t;
3      statik intk = 2;
4
5
6      public static void tmain(String[] args) {
7          int j = i; // Yanlış çünkü i bir örnek değişkenidir
8          m1();n // Yanlış çünkü m1() bir örnek yöntemidir
9      }
10
11     genel void t1() {
12         // Doğru çünkü örnek ve statik değişkenler ve yöntemler
13         // bir örnek yönteminde kullanılabilir
14     }
15
16     genel Statik int t2(tint, tintj) {
17         return(int)(Math.pow(in, j));
18     }n19

```

316 Bölüm 8 Nesneler ve Sınıflar

Önceki kodu aşağıdaki yeni kodla değiştirirseniz, program iyi olacaktır, çünkü örnek veri alanı **i** ve yöntem **m1** artık bir nesne **a**'dan erişilmektedir (7-8. satırlar):

```
1 genel sınıfA {
2     int = t5t;
3     statik intk = 2;
4
5
6     public static voidtmain(String[] args) {
7         A a = tnewtA();
8         int = a.i; // TAMAM, a.i nesnenin örnek değişkenine erişir
9         a.m1();nj // TAMAM. a.m1() nesnenin örnek yöntemini çağırır
10    }
11
12    genel voidtm1() {
13        ben = i + k + m2(i, k);
14    }
15
16    genel Statik inttm2(tint,tinttj) {
17        return(int)(Math.pow(in, j));
18    }
```

örnek mi yoksa statik mi?



Design Guilde

Bir değişkenin veya metodun örnek değişkeni mi yoksa statik mi olacağına nasıl karar verirsiniz? Sınıfın belirli bir örneğine bağlı olan bir değişken veya metod, örnek değişkeni veya metod olmalıdır. Sınıfın belirli bir örneğine bağlı olmayan bir değişken veya metod, statik değişken veya metod olmalıdır. Örneğin, her dairenin kendi yarıçapı vardır, bu nedenle yarıçap belirli bir daireye bağlıdır. Bu nedenle, **radius**, **Circle** sınıfının bir örnek değişkenidir. **getArea** metodu belirli bir daireye bağlı olduğundan, bu bir örnek metodudur. **Math** sınıfındaki **random**, **pow**, **sin** ve **cos** gibi metodlar belirli bir örneğe bağlı değildir. Bu nedenle, bu metodlar statik metodlardır. **main** metodu statiktir ve bir sınıftan doğrudan çağırılabilir.

yaygın tasarım hatası



Dikkat

Bir örnek yöntemi, statik olarak tanımlanması gereken bir örnek yöntemi tanımlamak yaygın bir tasarım hatasıdır. Örneğin, **factorial(int n)** yöntemi, belirli bir örnekten bağımsız olduğu için aşağıda gösterildiği gibi statik olarak tanımlanmalıdır.

```
public class Test { public
    int faktöriyel(tintin) { ninttresult = t1t
        ;
        fort(tint = t1t; i <= n; i++)nresult *= i
        ;
        dönüş sonucu;n
    }
}
```

(a) Yanlış tasarım

```
genel sınıf Testi { genel
    statik inttfactorial(tinttn) { ninttre
        sult = t1t;
        fort(tint = t1t; i <= n; i++)nresult *= i
        ;
        dönüş sonucu;n
    }
}
```

(b) Doğru tasarım

nokta Check

```

genel sınıf F {
    nintti;

    statictString s; nvoid
    timethod() {n}

    statik voidtsmethod() {n}
}

```

(a)

```

System.out.println(f.i); nSystem
.out.println(f.s); nf.imethod();

f.smethod();
System.out.println(F.i); nSyste
m.out.println(F.s); nF.imethod()
;
F.smethod();

```

(b)

8.18 Uygun ise ? yerine **static** anahtar kelimesini ekleyin.

```

genel sınıf Testi { özel h
    esap; genel

    ? voidtmain(String[] args) { n...

    }

    halk ? inttgetCount() { nret
        urntcount;
    }

    halk ? inttfactorial(tinttn) { ninttres
        ult = t1t; nfort(tint =
            t1t; i <= n; i++) nresult *= i;

        dönüş sonucu; n}

    }

```

8.19 Bir statik metodundan bir örnek metodunu çağırabilir veya bir örnek değişkenine referans verebilir misiniz? Bir örnek metodundan bir statik metodunu çağırabilir veya bir statik değişkenine referans verebilir misiniz? Aşağıdaki kodda ne yanlış?

```

1 genel sınıf C {
2     public static voidtmain(String[] args) {
3 yöntem1(); n4 }

5
6     public voidtmethod1() { n7 m
7     ethod2(); n8 }

9
10    genel statik voidtmethod2() {
11        System.out.println("Yarıçap nedir "+ c.getRadius());
12    }
13
14 Çember c = tnewtCircle(); n15 }

```

8.8 Görünürlük Modifikatörleri

Görünürlük modifikatörleri, bir sınıfın ve üyelerinin görünürlüğünü belirtmek için kullanılabilir.

public görünürlük modifikatörünü sınıflar, yöntemler ve veri alanları için kullanabilirsiniz; bu, diğer sınıflardan erişilebileceğini belirtir. Eğer görünürlük modifikatörü kullanılmazsa, varsayılan olarak sınıflar, yöntemler ve veri alanları aynı paketdeki herhangi bir sınıf tarafından erişilebilir. Bu, paket-özel veya paket-erşimi olarak bilinir.



paket-özel (veya paket-erşimi)

paketleri kullanma



Not

Sınıfları düzenlemek için paketler kullanılabilir. Bunu yapmak için, programdaki ilk yorum olmayan ve boş olmayan ifade olarak aşağıdaki satırı eklemeniz gerekir:

```
paketpackageName;
```

Bir sınıf paket ifadesi olmadan tanımlanmışsa, varsayılan *paket* içinde yer alıyor demir.

Java, sınıfları varsayılan bir paket yerine paketlere yerleştirmenizi önerir. Ancak, basitlik açısından bu kitap varsayılan paketler kullanmaktadır. Paketler hakkında daha fazla bilgi için, Ek III.G, Paketler bölümüne bakın.

Java, **public** ve varsayılan görünürlük belirleyicilerine ek olarak, sınıf üyeleri için **private** ve **protected** belirleyicilerini sağlar. Bu bölüm **private** belirleyicisini tanıtmaktadır. **protected** belirleyicisi Bölüm 11.13, **protected** Veri ve Yöntemler bölümünde tanıtılacaktır.

private belirleyicisi, yöntemlerin ve veri alanlarının yalnızca kendi sınıfı içinden erişilebilir olmasını sağlar. Şekil 8.14, sınıf C1 içindeki bir public, varsayılan ve private veri alanı veya yönteminin, aynı paketteki bir sınıf C2 ve farklı bir paketteki bir sınıf C3 tarafından nasıl erişilebileceğini göstermektedir.

<pre> paketp1; public class C1 { public int x; int y; özel intttz;public voidtm1() {n} voidtm2() {n} özel voidtm3() {n} } </pre>	<pre> paketp1; public classC2 {nvoid taMethod() {ncan o. C1 o = new C1(); x'e erişebilir; o.y'ye erişemiyorum; no. z'ye erişilemiyor; n, o. m1()'i çağırabilir;n, o.m 2()'yi çağırabilir;n, o.m 3()'u çağıramaz;n} } </pre>	<pre> paketp2; public classtC3 {nvoid taMethod() {ncan o. C1 o = new C1(); x'e erişebilir; no.y'y e erişilemiyor; no.z'ye erişilemiyor; n, o.m1()'i çağıramaz;n, o.m2()'yi çağıramaz;n, o.m3()'u çağ ıramaz;n} } </pre>
---	--	--

FIGURE 8.14 Özel modifikatör, erişimi tanımlandığı sınıf ile kısıtlar, varsayılan modifikatör pakete erişimi kısıtlar ve genel modifikatör sınırsız erişimi sağlar.

Bir sınıf public olarak tanımlanmadıysa, yalnızca aynı paket içinde erişilebilir. Şekil 8.15'te gösterildiği gibi, C1, C2'den erişilebilir ancak C3'den erişilemez.

<pre> paketp1; classC1 { ... } </pre>	<pre> paketp1; public class C2 { can access C1 } </pre>	<pre> paketp2; public class C3 { cannot access C1; can access C2; } </pre>
--	--	---

FIGURE 8.15 Bir genel olmayan sınıf paket erişimine sahiptir.

iç erişim

Bir görünürlük modifikatörü, bir sınıftaki veri alanlarının ve yöntemlerin sınıf dışından nasıl erişilebileceğini belirtir. Sınıf içinden veri alanlarına ve yöntemlere erişim konusunda herhangi bir kısıtlama yoktur. Şekil 8.16b'de gösterildiği gibi, c sınıfı C olan bir nesne, özel üyelerine erişemez, çünkü c Test sınıfındadır. Şekil 8.16a'da gösterildiği gibi, c sınıfı C olan bir nesne, özel üyelerine erişebilir, çünkü c kendi sınıfı içinde tanımlanmıştır.

```

genel sınıf C {
    özel boolean

    public static voidtmain(String[] args) {nC c = tnewtC
    ();
    System.out.println( ); x
    System.out.println( c.convert()nc. ); N
    }

    özel inttconvert() {nretu
    rntx t1:t-1;t;n}
}

```

(a) Bu, nesne c sınıfı C içinde kullanıldığı için tamamdır.

```

genel sınıf Test {
    public static voidtmain(String[] args) {nC c = tnewtC
    ();
    System.out.println( ); x
    System.out.println( c.convert()nc. ); N
    }
}

```

(b) Bu yanlıştır çünkü x ve convert sınıf C içinde özel.

FIGURE 8.16 Nesne, kendi sınıfında tanımlanmışsa özel üyelerine erişebilir.



Dikkat

private belirleyici yalnızca bir sınıfın üyelerine uygulanır. public belirleyici bir sınıfa veya bir sınıfın üyelerine uygulanabilir. Yerel değişkenler üzerinde public ve private belirleyicilerini kullanmak bir derleme hatasına neden olur.



Not

Çoğu durumda, yapıcı public olmalıdır. Ancak, bir kullanıcının bir sınıfın örneğini oluşturmasını yasaklamak istiyorsanız, *private bir yapıcı* kullanın. Örneğin, **Math** sınıfından bir örnek oluşturmak için bir neden yoktur, çünkü tüm veri alanları ve yöntemleri statiktir. Kullanıcının **Math** sınıfından nesneler oluşturmasını önlemek için, **java.lang.Math** içindeki yapıcı aşağıdaki gibi tanımlanmıştır:

```
özelMath() { n}
```

özel yapıcı

8.9 Veri Alanı Kapsülleme

Veri alanlarını private yapmak, verileri korur ve sınıfın bakımını kolaylaştırır.

CircleWithStaticMembers sınıfındaki radius ve numberOfObjects veri alanları, Listing 8.7'de doğrudan değiştirilebilir (örneğin, c1.radius = 5c1.radius = 5c1.radius = 5 veya CircleWithStaticMembers.numberOfObjects = 1

(b) Bu iyi bir uygulama değildir—iki nedendir:

- İlk olarak, verilerle oynanabilir. Örneğin, **numberOfObjects** oluşturulan nesne sayısı saymak içindir, ancak yanlışlıkla keyfi bir değere ayarlanabilir (örneğin, **CircleWithStaticMembers.numberOfObjects = 10**).
- İkinci olarak, sınıfın bakımı zorlaşır ve hatalara karşı savunmasız hale gelir. Diyelim ki, **CircleWithStaticMembers** sınıfını, diğer programların sınıfı kullandıktan sonra radius'un negatif olmamasını sağlamak için değiştirmek istiyorsunuz. Sadece **CircleWithStaticMembers** sınıfını değil, aynı zamanda onu kullanan programları da değiştirmeniz gerekir, çünkü istemciler radius'u doğrudan değiştirmiş olabilir (örneğin, **c1.radius = -5**).

Veri alanlarının doğrudan değiştirilmesini önlemek için, veri alanlarını özel olarak tanımlamalısınız, **private** belirleyicisini kullanarak. Bu, *veri alanı kapsülleme* olarak bilinir.



VideoNotu

Veri alanı kapsülleme

veri alanı kapsülleme

320 Bölüm 8 Nesneler ve Sınıflar

Bir özel veri alanı, onu tanımlayan sınıfın dışındaki bir nesne tarafından erişilemez. Ancak, bir i stemci genellikle bir veri alanını almak ve değiştirmek ister. Bir özel veri alanını erişilebilir hale getirmek için, değerini döndüren bir *get* yöntemi sağlayın. Bir özel veri alanının güncellenebilmesi için, yeni bir değer ayarlamak üzere bir *set* yöntemi sağlayın.



Not

Gündelik dilde, bir *get* yöntemi *getter* (veya *accessor*) olarak adlandırılır ve bir *set* yöntemi *setter* (veya *mutator*) olarak adlandırılır.

getter (veya erişimci)
setter (veya değiştirici)

Bir *get* yönteminin aşağıdaki imzası vardır:

```
public returnType getPropertyNames()
```

boolean erişimci

Eğer *returnType* *boolean* ise, *get* metodu geleneksel olarak aşağıdaki gibi tanımlanmalıdır:

```
public boolean isPropertyNames()
```

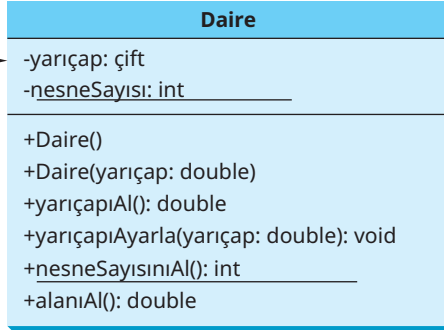
Bir *set* metodu aşağıdaki imzaya sahiptir:

```
public void setPropertyNames(dataType propertyValue)
```

Özel bir veri alanı olan *radius* ve ona bağlı erişim ve

değiştirme yöntemleri ile yeni bir daire sınıfı oluşturalım. Sınıf diyagramı Şekil 8.17'de gösterilmiştir. Yeni daire sınıfı, adıyla *CircleWithPrivateDataFields*, Liste 8.9'da tanımlanmıştır:

- işareti, bir özel belirleyici olduğunu gösterir



The radius of this circle (default: 1.0).
The number of circle objects created.

Constructs a default circle object.
Constructs a circle object with the specified radius.
Returns the radius of this circle.
Sets a new radius for this circle.
Returns the number of circle objects created.
Returns the area of this circle.

FiçERİK8.17 Daire sınıfı daire özelliklerini kapsar ve get/set ve diğer yöntemleri sağlar.

LISTELER8.9 ÖzelVeriAlanlarıylaDaire.java

kapsülle *radius*

kapsülle
Nesnelerin Sayısı

```
1 public class ÖzelVeriAlanlarıylaDaire {
2     /** Dairenin yarıçapı */
3     özel doubleradius = 1;
4
5     /** Oluşturulan nesne sayısı */
6     özel statik int nesneSayısı = 0;
7
8     /** Yarıçapı 1 olan bir daire oluştur */
9     public ÖzelVeriAlanlarıylaDaire() {
10         nesneSayısı++;
11     }
12
13     /** Belirtilen bir yarıçapa sahip bir daire oluştur */
14     public ÖzelVeriAlanlarıylaDaire(double yeniYarıçap) {
15         yarıçap = newRadius;
```



```

16 nesneSayısı++;
17 }
18
19 /** Radyusu döndür */
20 public double yarıçapıAl() {
21     return radius;
22 }
23
24 /** Yeni bir yarıçap ayarla */
25 public void yarıçapıAyarla(double yeniYarıçap) {
26     yarıçap = (yeniYarıçap >= 0 ? yeniYarıçap : 0);
27 }
28
29 /** numberOfObjects döndür */
30 public static int nesneSayısınıAl() {
31     return numberOfObjects;
32 }
33
34 /** Bu dairenin alanını döndür */
35 genel double getArea() {
36     return radius * radius * Math.PI;
37 }
38 }

```

erişimci metod

değiştirici metod

erişimci metod

getRadius() metodu (satırlar 20–22) yarıçapı döndürür ve **setRadius(newRadius)** metodu (satır 25–27) nesne için yeni bir yarıçap ayarlar. Yeni yarıçap negatifse, **0** nesne için yarıçap olarak ayarlanır. Bu yöntemler yarıçapı okumak ve değiştirmek için tek yollar olduğundan, yarıçap özelliğine erişim üzerinde tam kontrolünüz vardır. Bu yöntemlerin uygulamasını değiştirmeniz gerekirse, istemci programlarını değiştirmenize gerek yoktur. Bu, sınıfın bakımını kolaylaştırır.

Liste 8.10, **Circle** sınıfını kullanarak bir **Circle** nesnesi oluşturan ve **setRadius** metodunu kullanarak yarıçapı değiştiren bir istemci programı verir.

LİSTE 8.10 TestCircleWithPrivateDataFields.java

```

1 genel sınıf TestCircleWithPrivateDataFields {
2     /** Ana yöntem */
3     public static void main(String[] args) {
4         // 5.0 yarıçaplı bir daire oluştur
5         CircleWithPrivateDataFields myCircle =
6             new CircleWithPrivateDataFields(5.0);
7         System.out.println("Yarıçapı olan dairenin alanı "
8             + myCircle.getRadius() + " şudur: " + myCircle.getArea());
9
10        // myCircle'in yarıçapını %10 artır
11        myCircle.setRadius(myCircle.getRadius() * 1.1);
12        System.out.println("Yarıçapı olan dairenin alanı "
13            + myCircle.getRadius() + " şudur: " + myCircle.getArea());
14
15        System.out.println("Oluşturulan nesne sayısı "
16            + CircleWithPrivateDataFields.getNumberOfObjects());
17    }
18 }

```

public metodu çağır

public metodu çağır

public metodu çağır

Veri alanı **radius** özel olarak tanımlanmıştır. Özel veriler yalnızca tanımlandıkları sınıf içinde erişilebilir, bu nedenle istemci programında **myCircle.radius** kullanamazsınız. Özel verilere bir istemciden erişmeye çalışırsanız bir derleme hatası oluşur.

`numberOfObjects` özel olduğu için, değiştirilemez. Bu, müdahaleyi önler. Örneğin, kullanıcı `numberOfObjects` değerini 100 olarak ayarlayamaz. Bunu 100 yapmak için tek yol, `Circle` sınıfından 100 nesne oluşturmaktır.

Diyelim ki `TestCircleWithPrivateDataFields` ve `Circle` sınıfını birleştirip `TestCircleWithPrivateDataFields` içindeki `main` metodunu `Circle` içine taşıdınız. `main` metodunda `myCircle.radius` kullanabilir misiniz? Cevap için Kontrol Noktası Sorusu 8.22'ye bakın.



Design Guide

Verilerin değiştirilmesini önlemek ve sınıfın bakımını kolaylaştırmak için, data alanlarını özel olarak tanımlayın.

nokta ✓ Check

MyProgrammingLab™

8.20 Erişimci yöntemi nedir? Değiştirici yöntemi nedir? Erişimci yöntemler ve değiştirici yöntemler için adlandırma konvansiyonları nelerdir?

8.21 Data alanı kapsüllemenin faydaları nelerdir?

8.22 Aşağıdaki kodda, `radius Circle` sınıfında özel olup, `myCircle Circle` sınıfının bir nesnesidir. Vurgulanan kod herhangi bir probleme neden oluyor mu? Eğer öyleyse, nede nini açıklayın.

```
public class Circle {
    private double radius = 1;

    /** Bu dairenin alanını bulun */
    public double getArea() {
        return radius * yarıçap * Math.PI;
    }

    public static void main(String[] args) {
        Circle myCircle = new Circle();
        System.out.println("Yarıçap " + myCircle.yarıçap );
    }
}
```

8.10 Nesneleri Yöntemlere Geçirme



Bir nesneyi bir yönteme geçirmek, nesnenin referansını geçirmektir.

Objeleri yöntemlere geçirebilirsiniz. Bir diziyi geçmek gibi, bir nesneyi geçmek aslında nesnenin referansını geçmektir. Aşağıdaki kod, `myCircle` nesnesini `printCircle` yöntemine bir argüman olarak geçirir:

```
1 genel sınıfTest {
2     public static void main(String[] args) {
3         // CircleWithPrivateDataFields, Liste 8.9'da tanımlanmıştır
4         CircleWithPrivateDataFields myCircle = new CircleWithPrivateDataFields(5.0);
5         printCircle(myCircle);
6     }
7 }
8
9 genel statik void printCircle(CircleWithPrivateDataFields c) {
10     System.out.println("Yarıçapı olan dairenin alanı "
11         + c.getRadius() + " dir " + c.getArea());
12 }
13 }
```

bir nesne geç

değer ile geçiş

Java, argümanları geçmenin tam olarak bir modunu kullanır: değer ile geçiş. Önceki kodda, `myCircle` değerinin `printCircle` yöntemine geçildi. Bu değer, bir

`Circle` nesnesine bir referanstır.

Liste 8.11'deki program, bir ilkel tür

değerini geçmenin ve bir referans değerini geçmenin farkını göstermektedir.

LISTİNG 8.11 TestPassObject.java

```

1  genel classtestPassObject {
2      /** Ana yöntem */
3      public static void main(String[] args) {
4          // Yarıçapı 1 olan bir Çember nesnesi oluştur
5          CircleWithPrivateDataFields myCircle =
6              new CircleWithPrivateDataFields(1);
7
8          // Yarıçapı 1, 2, 3, 4 ve 5 için alanları yazdır.
9          int n = 1;
10         printAreas(myCircle, n);
11
12         // myCircle.radius ve times değerlerine bak
13         System.out.println("Yarıçap " + myCircle.getRadius());
14         System.out.pr
15         int n değeri " + n);
16     }
17
18     /** Yarıçap için alanların bir tablosunu yazdır */
19     genel statik void (CircleWithPrivateDataFields c, int times) printAreas {
20         System.out.println("Yarıçap Alan");
21         while(times >= 1) {
22             System.out.println(c.getRadius() + "t" + c.getArea());
23             c.setRadius(c.getRadius()
24                 + 1);
25         }
26     }
27
28     asla {revize e
29     dilmedi}

```

nesne geçir

nesne parametresi

Yarıçap Alan	
1.0	3.141592653589793
2.0	12.566370614359172
3.0	29.274333882308138
4.0	50.26548245743669
5.0	79.53981633974483
Yarıçap 6.0	
n 5'tir	



CircleWithPrivateDataFields sınıfı Liste 8.9'da tanımlanmıştır. Program, bir CircleWithPrivateDataFields nesnesi myCircle ve n'den bir tamsayı değeri geçirerek printAreas(myCircle, n) (satır 9) çağrısını yapar; bu, 1, 2, 3,

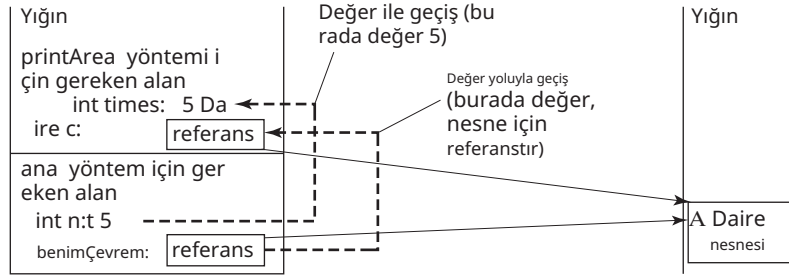
4, 5 için alanların bir tablosunu yazdırır; örnek çıktıda gösterildiği gibi.

Şekil 8.18, programdaki yöntemleri yürütmek için çağrı yığınının göstermektedir. Nesnelerin bir yığı n içinde saklandığını unutmayın (bkz. Bölüm 6.6).

Bir ilkel veri türünün argümanını geçirirken, argümanın değeri geçirilir. Bu durumda, n'nin (5) değeri times'a geçirilir. printAreas yönteminin içinde, times'ın içeriği değiştirilir; bu, n'nin içeriğini etkilemez.

Bir referans türünün argümanını geçirirken, nesnenin referansı geçirilir. Bu durumda, c, myCircle aracılığıyla da referans verilen nesne için bir referans içerir. Bu nedenle, printAreas yönteminin içinde c aracılığıyla nesnenin özelliklerini değiştirmek, myCircle değişkeni aracılığıyla yöntemin dışında bunu yapmakla aynı etkiye sahiptir. Referanslar üzerindeki değerle geçiş, en iyi şekilde paylaşarak geçiş olarak tanımlanabilir; yani, yöntemde referans verilen nesne, geçirilen nesne ile aynıdır.

paylaşım yoluyla



ŞEKİL 8.18 n değerinin times'a geçirildiği ve myCircle'a olan referansın printAreas metodundaki c'ye geçirildiği.

8.23 Bir ilke türü parametresini geçmek ile bir referans türü parametresini geçmek arasındaki farkı açıklayın. Aşağıdaki programların çıktısını gösterin:

nokta ✓ Check

MyProgrammingLab™

```
genel sınıfTest {
    public static voidtmain(String[] args) {nCount myC
        ount = tnewtCount();
        intttimes = t0t;

        fort(tintti = t0t; i < t100t; i++)increme
            nt(Count, times);System.out.print

        ln(t"count is "t+ myCount.count);nSystem.out.println(t"times
            is "t+ times);
    }

    public static voidtincrement(C sayısı, tintttimes) {nc.count++
        ;
        kere++;n}
}
```

```
public classtCount {npublic
    c intttcount;

    publictCount(tinttc) {ncou
        nt = c;
    }

    publictCount() {nco
        unt = t1t;n}
}
```

8.24 Aşağıdaki programın çıktısını göster:

```
genel sınıfTest {
    public static voidtmain(String[] args) {nCircle Circle
        1 = tnewtCircle(t1t);
        Daire daire2 = tnewtCircle(t2t);nswap1(

        daire1, daire2);
        System.out.println("swap1'den sonra: circle1 = "+
            circle1.radius + " circle2 = " + circle2.radius);

        swap2(circle1, circle2);
        System.out.println("swap2'den sonra: circle1 = "+
            circle1.radius + " circle2 = " + circle2.radius);
    }

    public static voidtswap1(Daire x, Daire y) {nÇember temp =
        x;
        x = y;ve
        ya = zaman;
    }
```

```

    }

    public static void swap2(Daire x, Daire y) {
        double temp = x.radius;
        x.yarıçap = y.yarıçap;
        y.yarıçap = temp;
    }

    class Circle {
        double radius;

        Circle(double newRadius) {
            radius = newRadius;
        }
    }

```

8.25 Aşağıdaki kodun çıktısını gösterin:

```

genel sınıf Test {
    public static void main(String[] args) {
        int[] a = {1, 2};
        swap(a, 1);
        System.out.println("a[0] = " + a[0] + " a[1] = " + a[1]);
    }

    public static void swap(int[] a, int i) {
        int temp = a[i];
        a[i] = a[0];
        a[0] = temp;
    }
}

```

(a)

```

genel sınıf Test {
    public static void main(String[] args) {
        int[] a = {1, 2};
        swap(a);
        System.out.println("a[0] = " + a[0] + " a[1] = " + a[1]);
    }

    public static void swap(int[] a) {
        int temp = a[0];
        a[0] = a[1];
        a[1] = temp;
    }
}

```

(b)

```

genel sınıf Test {
    public static void main(String[] args) {
        int t = new T();
        swap(t);
        System.out.println("e1 = " + t.e1 + " e2 = " + t.e2);
    }

    public static void swap(T t) {
        int temp = t.e1;
        t.e1 = t.e2;
        t.e2 = temp;
    }
}

class T {
    int e1;
    int e2;
}

```

(c)

```

genel sınıf Test {
    public static void main(String[] args) {
        int t1 = new T();
        T t2 = new T();
        System.out.println("t1'in i = " + t1.i + " ve j = " + t1.j);
        System.out.println("t2'nin i = " + t2.i + " ve j = " + t2.j);
    }
}

sınıf T {
    static int i = 0;
    int j = 0;

    T() {
        i++;
        j = i;
    }
}

```

(d)

8.26 Aşağıdaki programların çıktısı nedir?

```
import java.util.Date;
public

class Test {
    public static void main(String[] args) {
        Date date
        = null;
        m1(tarih);
        System.out.println(tarih);
    }

    public static void m1(Tarih tarih) {
        tarih = tne
        wtDate();
    }
}
```

(a)

```
import java.util.Date;
public

class Test {
    public static void main(String[] args) {
        Date date
        = tnewDate(t1234567t);
        m1(tarih);
        System.out.println(date.getTime());
    }

    public static void m1(Tarih tarih) {
        tarih = tne
        wtDate(t7654321t);
    }
}
```

(b)

```
import java.util.Date;
public

class Test {
    public static void main(String[] args) {
        Date date
        = tnewDate(t1234567t);
        m1(tarih);
        System.out.println(date.getTime());
    }

    public static void m1(Tarih tarih) {
        date.setTi
        me(t7654321t);
    }
}
```

(c)

```
import java.util.Date;
public

class Test {
    public static void main(String[] args) {
        Date date
        = tnewDate(t1234567t);
        m1(tarih);
        System.out.println(date.getTime());
    }

    public static void m1(Tarih tarih) {
        tarih = tnu
        ll;
    }
}
```

(td)

8.11 Nesne Dizisi



Bir dizi, nesneleri ve ilkel tür değerlerini tutabilir.

Bölüm 6, Tek Boyutlu Diziler, ilkel tür elemanlarının dizilerini nasıl oluşturacağınızı açıkladı. Ayrıca nesne dizileri de oluşturabilirsiniz. Örneğin, aşağıdaki ifade on **Circle** nesne sinin bir dizisini tanımlar ve oluşturur:

```
Circle[] circleArray = new Circle[10];
```

circleArray'yi başlatmak için, aşağıdaki gibi bir for döngüsü kullanabilirsiniz:

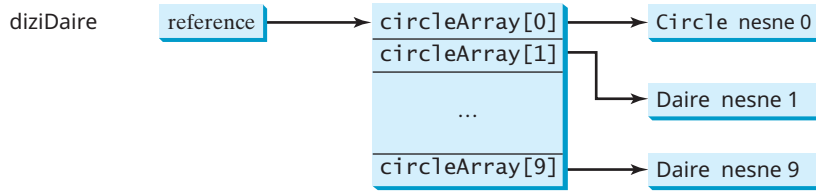
```
for(tinti = t0; i < circleArray.length; i++) {
    ncircleArray[i] = t
    newtCircle();
}
```

Nesne dizisi aslında bir *referans değişkenleri dizisi*'dir. Bu nedenle, çağırmak **circleArray[1].getArea()** iki seviyeli referanslama içerir, Şekil 8.19'da gösterildiği gibi. **circleArray** tüm diziyi referans alır; **circleArray[1]** bir **Circle** nesnesini referans alır.



Not

Bir nesne dizisi **new** operatörü kullanılarak oluşturulduğunda, dizideki her bir eleman default değeri **null** olan bir referans değişkenidir.



Fiçerik8.19 Nesneler dizisinde, dizinin bir elemanı bir

nesneye referans içerir.

Liste 8.12, nesne dizisinin nasıl kullanılacağını gösteren bir örnek vermektedir. Program, bir dizi çemberin alanlarını özetler. Program, beş **Circle** nesnesinden oluşan bir dizi olan **circleArray** oluşturur; ardından çember yarıçaplarını rastgele değerlerle başlatır ve dizideki çemberlerin toplam alanını görüntüler.

LİSTELER8.12ToplamAlan.java

```

1  public class ToplamAlan {
2      /** Ana yöntem */
3      public static void main(String[] args) {
4          // DaireDizisini Tanımla
5          DaireÖzelVeriAlanları[] daireDizisi;
6
7          // DaireDizisini Oluştur
8          circleArray = createCircleArray();
9
10         // DaireDizisini ve dairelerin toplam alanlarını Yazdır
11         printCircleArray(circleArray);
12     }
13
14     /** Daire nesnelerinin bir dizisini oluştur */
15     public static DaireÖzelVeriAlanları[] createCircleArray() {
16         DaireÖzelVeriAlanları[] daireDizisi =
17             yeniDaireÖzelVeriAlanları[5];
18
19         for(tintti = t0t; i < CircleArray.length; i++) {n20 CircleArray[i]
20             ] =
21             newtCircleWithPrivateDataFields(Math.random() * t100t);
22         }
23
24         // Daire dizisini döndür
25         returntCircleArray;
26     }
27
28     /** Dairelerin bir dizisini ve toplam alanlarını yazdır */
29     genel statik void Array( CircleWithPrivateDataFields[] CircleArray)nprintCircle {
30         System.out.printf("%-30s%-15s", "Yarıçap", "Alan");
31         for(tintti = t0t; i < circleArray.length; i++) {
32             System.out.printf(t"%-30f%-15fn", CircleArray[i].getRadius(),n34 CircleArray[i].getArea())
33             ;
34         }
35     }
36
37     System.out.println(t-----"t);
38
39     // Sonucu hesapla ve göster
40     System.out.printf("%-30s%-15f", "Dairelerin toplam alanı",sum(circleArray
41         );

```

nesneler dizisi

nesneler dizisini döndür

nesneler dizisini geçir

```

42     }
4344    /** Daire alanlarını ekle */
nesneler dizisini geçir 45    genel statik çift toplam(CircleWithDataFields[] CircleArray) {
46        // Toplamı başlat
47        çift toplam = t0t;
4849        // Alanları toplamaya ekle
50        fort(tintti = t0t; i < CircleArray.length; i++)
51 toplam += CircleArray[i].getArea();
5253        dönüş toplam
54 }
55 }

```



Yarıçap	Alan
70.577708	15648.941866
44.152266	6124.291736
24.867853	1942.792644
5.680718	101.380949
36.734246	4239.280350
Dairelerin toplam alanı 28056.687544	

Program, beş daire nesnesinden oluşan bir dizi oluşturmak için `createCircleArray()` (satır 8) yöntemini çağırır.

Bu bölümde birkaç daire sınıfı tanıtıldı. Bu örnek,

Section 8.9, Veri Alanı Kapsülleme'de tanıtılan `CircleWithDataFields` sınıfını kullanır.

Daire yarıçapları, `Math.random()` yöntemi kullanılarak rastgele üretilir (satır 21). `createCircleArray` yöntemi, `CircleWithDataFields` nesnelerinden oluşan bir dizi döndürür (satır 25). Dizi, her dairenin yarıçapını ve alanını ve dairelerin toplam alanını gösteren `printCircleArray` yöntemine iletilir.

Daire alanlarının toplamı, `sum` yöntemini çağırarak hesaplanır (satır 41). Bu yöntem, `CircleWithDataFields` nesnelerinin dizisini argüman olarak alır ve toplam alanı için bir `double` değeri döndürür.

8.27 Aşağıdaki kodda ne yanlış?

```

1    genel sınıfTest {
2        public static voidtmain(String[] args) {
3 java.util.Date[] tarihler = tnewtjava.util.Date[t10t];n4 System.out.prin
tln(dates[t0t]);
5 System.out.println(dates[t0t].toString());n6 }
7    }

```



MyProgrammingLab™

Anahtar Terimler

eylem 296
anonim nesne 305
özellik 296
davranış 296
sınıf 296
istemci 299

yapıcı 296
data alanı 296
data alanı kapsülleme 319
varsayılan yapıcı 303
nokta operatörü (.) 304
getter (veya erişimci) 320

örnek 296	özellik 296
örnek yöntemi 305	kamusal sınıf 299
örnek değişkeni 305	referans türü 304
örnekleme 296	referans değişkeni 304
argüman içermeyen yapıcı 303	setter (veya mutator) 320
boş değer 305	durum 296
nesne 296	statik yöntem 312
nesne yönelimli programlama (OOP)	statik değişken 312
paket özel (veya paket- erişimi) 317	Birleştirilmiş Modelleme Dili (UML) 297
özel yapıcı 319	

BÖLÜM ÖZETİ

1. Bir sınıf, nesneler için bir şablondur. Nesnelerin özelliklerini tanımlar ve nesne oluşturmak için yapıcılar ve bunları manipüle etmek için yöntemler sağlar.
2. Bir sınıf aynı zamanda bir veri türüdür. Nesne referans değişkenlerini tanımlamak için kullanabilirsiniz. Bir nesne referans değişkeni, bir nesne tutuyormuş gibi görünse de aslında o nesneye bir referans içerir. Katı bir şekilde konuşursak, bir nesne referans değişkeni ve bir nesne farklıdır, ancak çoğu zaman bu ayrım göz ardı edilebilir.
3. Bir nesne, bir sınıfın örneğidir. Bir nesne oluşturmak için **new** operatörünü kullanırsınız. Nesnenin referans değişkeni aracılığıyla o nesnenin üyelerine erişmek için nokta operatörünü (.) kullanırsınız.
4. Bir *örnek değişkeni* veya *yöntem*, bir sınıfın bir örneğine aittir. Kullanımı, bireysel örneklerle ilişkilidir. Bir *statik değişken*, aynı sınıfın tüm örnekleri tarafından paylaşılan bir değişkendir. Bir *statik yöntem*, örnekler kullanılmadan çağrılabilen bir yöntemdir.
5. Bir sınıfın her örneği, sınıfın statik değişkenlerine ve yöntemlerine erişebilir. Ancak, netlik açısından, statik değişkenleri ve yöntemleri çağırarak `SınıfAdı.değişken` ve `SınıfAdı.yöntem` kullanmak daha iyidir.
6. Modifikatörler, sınıfın, yöntemin ve verilerin nasıl erişileceğini belirtir. Bir **public** sınıf, yöntem veya veri, tüm istemcilere açıktır. Bir **private** yöntem veya veri yalnızca sınıf içinde erişilebilir.
7. İstemcilerin verileri görmesini veya değiştirmesini sağlamak için bir **get** yöntemi veya bir **set** yöntemi sağlayabilirsiniz. Günlük dilde, bir **get** yöntemi *getter* (veya *accessor*) olarak adlandırılır ve bir **set** yöntemi *setter* (veya *mutator*) olarak adlandırılır.
8. Bir **get** yönteminin imzası **public returnType getPropertyname()** şeklindedir. Eğer **returnType boolean** ise, **get** yöntemi **public boolean isPropertyname()** olarak tanımlanmalıdır. Bir **set** yönteminin imzası **public void setPropertyname(dataType propertyValue)** şeklindedir.
9. Tüm parametreler, değer ile geçiş yöntemi kullanılarak yöntemlere iletilir. Bir ilkel tür parametresi için, gerçek değer iletilir; bir referans tür parametresi için, nesnenin referansı iletilir.
10. Bir Java dizisi, ilkel tür değerlerini veya nesne tür değerlerini içerebilen bir nesnedir. Nesnelerden oluşan bir dizi oluşturulduğunda, elemanlarına varsayılan değer olan null atanır.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinde yapın.

MyProgrammingLab™

PROGRAMLAMA EGZERSİZLERİ

üç hedef

**Petdagtotgtitcal Ntote**

Bölüm 8–11, 15'teki alıştırmalar, üç hedefe ulaşmanıza yardımcı olur:

- Sınıfları tasarlayın ve UML sınıf diyagramları çizin
- UML'den sınıfları uygulayın
- Uygulama geliştirmek için sınıfları kullanın

Öğrenciler, çift numaralı alıştırmalar için UML diyagramlarının çözümlerini Companion Web Sitesi'nden indirebilir ve eğitmenler aynı siteden tüm çözümleri indirebilir.

Bölümler 8.2–8.5

8.1 (Dikdörtgen sınıfı) Bölüm 8.2'deki Çember sınıfı örneğini takip ederek, bir dikdörtgeni temsil etmek için Dikdörtgen adında bir sınıf tasarlayın. Sınıf şunları içerir:

- İki **double** veri alanı olan **width** ve **height**, dikdörtgenin genişliğini ve yüksekliğini belirtir. Varsayılan değerler her ikisi için de 1'dir.
- Varsayılan bir dikdörtgen oluşturan bir argüman almayan yapıcı.
- Belirtilen **width** ve **height** ile bir dikdörtgen oluşturan bir yapıcı.
- Bu dikdörtgenin alanını döndüren **getArea()** adlı bir yöntem.
- Çevreyi döndüren **getPerimeter()** adlı bir yöntem.

UML diyagramını çizin ve ardından sınıfı uygulayın. Genişliği 4 ve yüksekliği 40 olan bir **Rectangle** nesnesi ile genişliği 3.5 ve yüksekliği 35.9 olan diğer bir nesne oluşturacak bir testprogramı yazın. Her dikdörtgenin genişliğini, yüksekliğini, alanını ve çevresini bu sırayla görüntüleyin.

8.2 (**Stock** sınıfı) Bölüm 8.2'deki **Circle** sınıfının örneğini takip ederek, aşağıdaki içeriği içeren **Stock** adlı bir sınıf tasarlayın:

- Hisse senedinin sembolü için **symbol** adlı bir string veri alanı.
- Hisse senedinin adı için **name** adlı bir string veri alanı.
- Önceki gün için hisse senedi fiyatını saklayan **previousClosingPrice** adlı bir **double** veri alanı.
- Mevcut zaman için hisse senedi fiyatını saklayan **currentPrice** adlı bir **double** veri alanı.
- Belirtilen sembol ve ad ile bir hisse senedi oluşturan bir yapıcı.
- **previousClosingPrice** ile **currentPrice** arasındaki yüzde değişimi döndüren **getChangePercent()** adlı bir yöntem.

UML diyagramını çizin ve ardından sınıfı uygulayın. Bir test programı yazın, bu program **Stock** nesnesi oluşturur ve hisse senedi sembolü **ORCL**, isim **Oracle Corporation**, ve önceki kapanış fiyatı **34.5**. Yeni bir güncel fiyatı **34.35** olarak ayarlayın ve fiyat değişim yüzdesini gösterin.

Bölüm 8.6

***8.3** (**Tarih** sınıfını kullanın) 10000,100000,0000000,00000000,000000000,0000000000, için geçen süreyi ayarlayan bir **Tarih** nesnesi oluşturan bir program yazın. 100000000000, ve 1000000000000, ve tarih ve saati **toString()** yöntemi kullanarak görüntüler.

- *8.4** (Rastgele sınıfını kullanın) Bir Rastgele nesnesi oluşturacak bir program yazın ve tohum **1000** ve 0 ile 100 arasında rastgele 50 tam sayıyı **0** ve **100** kullanarak görüntüler **nextInt(100)** metodu.
- *8.5** (**GregorianCalendar** sınıfını kullanın) Java API'sinde **GregorianCalendar** sınıfı **java.util** paketinde bulunmaktadır; bu sınıfı bir tarihin yılını, ayını ve gününü elde etmek için kullanabilirsiniz. No-arg yapıcı, mevcut tarih için bir örnek oluşturur ve **get(GregorianCalendar.YEAR)**, **get(GregorianCalendar.MONTH)** ve **get(GregorianCalendar.DAY_OF_MONTH)** yöntemleri yıl, ay ve günü döndürür. İki görev gerçekleştiren bir program yazın:
- Mevcut yılı, ayı ve günü görüntüleyin.
 - GregorianCalendar sınıfında, 1 Ocak 1970'ten itibaren belirli bir geçen süreyi ayarlamak için kullanılacak **setTimeInMillis(long)** metodu bulunmaktadır. Değeri **1234567898765L** olarak ayarlayın ve yılı, ayı ve günü görüntüleyin.

Bölümler 8.7-8.9

- **8.6** (Takvimleri görüntüle) Liste 5.12'deki **PrintCalendar** sınıfını yeniden yazın ve takvimleri bir mesaj iletişim kutusunda görüntüleyin. Çıktı, sınıftaki birkaç statik yöntemden üretildiğinden, çıktıyı saklamak için statik bir **String** değişkeni **output** tanımlayabilir ve bunu bir mesaj iletişim kutusunda görüntüleyebilirsiniz.
- 8.7** (Hesap sınıfı) **Account** adında bir sınıf tasarlayın ve bu sınıf şunları içersin:
- Hesap için **id** adında özel bir **int** veri alanı (varsayılan **0**).
 - Hesap için **balance** adında özel bir **double** veri alanı (varsayılan **0**).
 - Mevcut faiz oranını saklayan **annualInterestRate** adında özel bir **double** veri alanı (varsayılan **0**). Tüm hesapların aynı faiz oranına sahip olduğunu varsayın.
 - Hesap oluşturulduğunda tarihi saklayan **dateCreated** adında özel bir **Date** veri alanı.
 - Varsayılan bir hesap oluşturan bir argüman almayan yapıcı.
 - Belirtilen id ve başlangıç bakiyesi ile bir hesap oluşturan bir yapıcı.
 - Erişim ve değiştirici yöntemler için **kimlik**, **bakiye**, ve **yıllıkFaizOranı**.
 - Oluşturulma tarihi için erişimci yöntemi **dateCreated**.
 - Aylık faiz oranını döndüren **getMonthlyInterestRate()** adlı bir yöntem.
- Aylık faizi döndüren **getMonthlyInterest()** adlı bir yöntem.
- theaccount**'tan belirli bir miktarı çeken, para çekme adlı bir yöntem.

Hesaba belirli bir tutarı yatıran **t mevduat** adı verilen bir yöntem.

UML diyagramını çizin ve ardından sınıfı uygulayın. (İpucu: Method **getMonthlyInterest()** aylık faizi döndürmek içindir, faiz oranını değil. Aylık faiz **bakiye * monthlyInterestRate.monthlyInterestRate** ise **annualInterestRate / 12**. Yıllık faiz oranının bir yüzde olduğunu unutmayın, e.g., %4.5 gibi. Bunu 100'e bölmeniz gerekiyor.)

1122 hesap kimliğine, 20.000 \$ bakiyeye ve %4.5 yıllık faiz oranına sahip bir **Account** nesnesi oluşturan bir test programı yazın. **withdrawMethod**unu kullanarak 2.500 \$ çekin, **deposit** methodunu kullanarak 3.000 \$ yatırım yapmak ve den bakiyeyi, aylık faizi ve bu hesabın oluşturulduğu tarihi yazdırın.

- 8.8** (**Fan** sınıfı) Bir fanı temsil etmek için **Fan** adında bir sınıf tasarlayın. Sınıf şunları içerir:
- Üç sabit **SLOW**, **MEDIUM** ve **FAST** adında sabitler, fan hızını belirtmek için **1, 2** ve **3** değerleri ile.



- Fanın hızını belirten özel bir **int** veri alanı **speed** (varsayılan **SLOW**'dır).
- Özel **boolean** veri alanı **on** adında bir alan, fanın açık olup olmadığını belirtir (varsayılan **false**).
- Özel **double** veri alanı **radius** adında bir alan, fanın yarıçapını belirtir (varsayılan **5**).
- Fanın rengini belirten **color** adında bir string veri alanı (varsayılan **blue**).
- Tüm dört veri alanı için erişimci ve değiştirici yöntemler.
- Varsayılan bir fan oluşturan bir argüman almayan yapıcı.
- Fan için bir string açıklaması döndüren **toString()** adında bir yöntem. Eğer fan açıksa, yöntem fan hızını, rengini ve yarıçapını birleştirilmiş bir string olarak döndürür. Eğer fan kapalıysa, yöntem fan rengini ve yarıçapını "fan kapalı" stringi ile birlikte birleştirilmiş bir string olarak döndürür.

UML diyagramını çizin ve ardından sınıfı uygulayın. İki **Fan** nesnesi oluşturan bir test programı yazın. İlk nesneye maksimum hız, yarıçap **10**, renk **yellow** atayın ve açın. İkinci nesneye orta hız, yarıçap **5**, renk **blue** atayın ve kapatın. Nesneleri **toString** yöntemini çağırarak görüntüleyin.

****8.9** (*Geometri: n kenarlı düzgün çokgen*) Bir n kenarlı düzgün çokgende, tüm kenarlar aynı uzunluktadır ve tüm açıların dereceleri aynıdır (yani, çokgen hem eşkenar hem de eş ağıdadır). **RegularPolygon** adında bir sınıf tasarlayın ve bu sınıf şunları içersin:

- Bir özel **int** veri alanı olan **n**, çokgenin kenar sayısını tanımlar ve varsayılan değeri **3**'dür.
- Bir özel **double** veri alanı olan **side**, kenarın uzunluğunu saklar ve varsayılan değeri **1**'dir.
- Bir özel **double** veri alanı olan **x**, çokgenin merkezinin x -koordinatını tanımlar ve varsayılan değeri **0**'dir.
- Bir özel **double** veri alanı olan **y**, çokgenin merkezinin y -koordinatını tanımlar ve varsayılan değeri **0**'dir.
- Varsayılan değerlerle bir düzgün çokgen oluşturan bir argüman almayan yapıcı.
- Belirtilen kenar sayısı ve kenar uzunluğu ile (**0,0**) merkezli bir düzgün çokgen oluşturan bir yapıcı.
- Belirtilen kenar sayısı, kenar uzunluğu ve x - ve y -koordinatları ile bir düzgün çokgen oluşturan bir yapıcı.
- Tüm veri alanları için erişimci ve değiştirici yöntemler.
- Çokgenin çevresini döndüren **getPerimeter()** yöntemi.
- Çokgenin alanını döndüren **getArea()** yöntemi. Düzgün bir çokgeni

$$n \text{ alanını hesaplamak için formül} \quad \text{Alan} = \frac{n \cdot \text{side}^2}{4 \cdot \tan(\pi/n)}$$

UML diyagramını çizin ve ardından sınıfı uygulayın. Üç **RegularPolygon** nesnesi oluşturan bir test programı yazın; bunlar, no-arg kurucusu kullanılarak, **RegularPolygon(6, 4)** kullanılarak ve **RegularPolygon(10,4, 5.6, 7.8)** kullanılarak oluşturulmuştur. Her nesne için, çevresini ve alanını gösterin.

***8.10** (*Cebir: ikinci dereceden denklemler*) **İkinciDerecedenDenklem** adında bir sınıf tasarlayın bir ikinci dereceden denklem için $ax^2 + bx + c = 0$. Sınıf şunları içerir:

- Özel veri alanları **a**, **b** ve **c** üç katsayıyı temsil eder.
- **a**, **b** ve **c** için argümanlar için bir yapıcı.

■ Üçget metodu **a,b** ve **c** için.

■ getDiscriminant() adında bir yöntem, ayırmıcıyı döndürür, bu da $4b^2 - 4ac$ dır.

■ getRoot1() ve getRoot2() adında iki kökü döndürmek için yöntemlerdir eşitlik

$$r_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad r_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Bu yöntemler yalnızca ayırmıcı negatif değilse kullanışlıdır. Bu yöntemlerin ayırmıcı negatifse 0 döndürmesini sağlayın.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. Kullanıcıdan **a,b** ve **c** değerlerini girmesini isteyen bir test programı yazın ve sonuca ayırmıcıya göre görüntüleyin. Ayırmıcı pozitifse, iki kökü gösterin. Eğer ayırmıcı 0 ise, bir kökü gösterin. Aksi takdirde, "Eşitliğin kökü yoktur." mesajını gösterin. Örnek çalışmaları için Programlama Egzersizi 3.1'e bakın.

***8.11** (Cebir: doğrusal denklemler) Doğrusal denklemler sistemi için LinearEquation 2×2 adında bir sınıf tasarlayın:

$$\begin{array}{l} ax + by = e \\ cx + dy = f \end{array} \quad x = \frac{ed - bf}{ad - bc} \quad y = \frac{af - ec}{ad - bc}$$

Sınıf şunları içerir:

■ Özel veri alanları **a,b,c,d,e**, ve **f**.

■ **a,b,c,d,e**, ve **f** için argümanlarla bir yapıcı.

■ **a, b, c, d, e**, ve **f** için altı get metodu.

■ isSolvable() adında bir metod, eğer 0 değilse true döner. $ad - bc \neq 0$ değildir.

■ getX() ve getY() metodları, denklemin çözümünü döner.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. Kullanıcıdan **a, b, c, d, e**, ve **f** girmesini isteyen bir test programı yazın ve sonucu gösterin.

Eğer $ad - bc = 0$ ise, "Denklemin çözümü yoktur." rapor edin. Örnek çalışmaları için Programlama Alıştırma 3.3'e bakın.

****8.12** (Geometri: kesişim) İki doğru parçasının kesiştiğini varsayalım. İlk doğru parçasının iki uç noktası (**x1,y1**) ve (**x2,y2**) ve ikinci doğru parçasının uç noktaları (**x3,y3**) ve (**x4,y4**) dir. Kullanıcının bu dört uç noktasını girmesini isteyen ve kesişim noktasını gösteren bir program yazın. (İpucu: 8.11. Alıştırmadaki LinearEquationsınıfını kullanın.)

İlk doğru parçasının uç noktalarını girin:

2.0 2.0 0 0

↵ Enter

İkinci doğru parçasının uç noktalarını girin:

0 2.0 2.0 0

↵ Enter

Kesim noktası: (1.0, 1.0)



****8.13** (Konum sınıfı) İki boyutlu bir dizide en büyük değeri ve konumunu bulmak için Konum adında bir sınıf tasarlayın. Sınıf, iki boyutlu bir dizide en büyük değeri ve indekslerini saklayan public veri alanları satır, sütun ve maxIndex içerir; bunlar satır ve sütun için int türünde ve maxIndex için double türündedir.

Aşağıdaki yöntemi yazın, bu yöntem iki boyutlu bir dizideki en büyük elemanın konumunu döndürsün:

```
public static Location bulunLargest(double[][] a)
```

Return değeri bir **Location** örneğidir. Kullanıcının iki boyutlu bir dizi girmesini isteyen ve dizideki en büyük elemanın konumunu gösteren bir test programı yazın. İşte bir örnek çalışma:



Array'deki satır ve sütun sayısını girin:

3 4 Enter

Diziyi girin:

23.5 35 2 10 Enter

4.5 3 45 3.5 Enter

35 44 5.5 9.6 Enter

En büyük elemanın konumu (1, 2) koordinatında 45'tir.

***8.14** (*Chronometre*) **StopWatch** adında bir sınıf tasarlayın. Sınıf şunları içerir:

- Özel veri alanları **startTime** ve **endTime** ile get yöntemleri.
- Mevcut zamanı **startTime** ile başlatan bir argüman almayan yapıcı.
- Mevcut zamanı **startTime** olarak sıfırlayan **start()** adında bir yöntem.
- Mevcut zamanı **endTime** olarak ayarlayan **stop()** adında bir yöntem.
- Kronometre için geçen süreyi milisaniye cinsinden döndüren **getElapsedTime()** adında bir yöntem.

UML diyagramını çizin ve ardından sınıfı uygulayın. 100.000 sayının sıralanma süresini ölçen bir testprogramı yazın ve seçim sıralaması kullanın.

BÖLÜM

9

STRINGLER

Hedefler

- Düzenli dizeleri işlemek için **String** sınıfını kullanmak (§9.2).
- Dizeleri oluşturmak (§9.2.1).
- Dizelerin değişmez olduğunu anlamak ve bir interned string oluşturmak (§9.2.2).
- Dizeleri karşılaştırmak (§9.2.3).
- Dize uzunluğunu ve karakterlerini almak ve dizeleri birleştirmek (§9.2.4).
- Alt dizeleri elde etmek (§9.2.5).
- Dizeleri dönüştürmek, değiştirmek ve ayırmak (§9.2.6).
- Dizeleri desenlere göre eşleştirmek, değiştirmek ve ayırmak (§9.2.7).
- Bir dizide bir karakter veya alt dize aramak (§9.2.8).
- Bir dize ile bir dizi arasında dönüştürmek (§9.2.9).
- Karakterleri ve sayıları bir dizeye dönüştürmek (§9.2.10).
- Biçimlendirilmiş bir dize elde etmek (§9.2.11).
- Bir dizenin palindrom olup olmadığını kontrol etmek (§9.3).
- Onaltılık sayıları ondalık sayılara dönüştürmek (§9.4).
- Tek bir karakteri işlemek için **Character** sınıfını kullanmak (§9.5).
- Esnek dizeleri işlemek için **StringBuilder** ve **StringBuffer** sınıflarını kullanmak (§9.6).
- **String**, **StringBuilder** ve **StringBuffer** sınıfları arasında ayırt etmek için (§9.2-9.6).
- Komut satırından **main** metoduna argüman nasıl geçileceğini öğrenmek için (§9.7).



9.1 Giriş



String, StringBuffer ve StringBuilder sınıfları, dizeleri işlemek için kullanılır.

Bir *string*, karakterlerin bir dizisidir. Dizeler programlamada sıkça kullanılır. Birçok dil, dizeleri karakter dizisi olarak ele alırken, Java'da bir dize bir nesne olarak ele alınır. Bu bölüm, dizeleri işlemek için sınıfları tanıtmaktadır.

9.2 String Sınıfı



*Bir **String** nesnesi değiştirilemez: Dize oluşturulduktan sonra içeriği değiştirilemez.*

String sınıfı, dizeleri manipüle etmek için 13 yapıcı ve 40'tan fazla metoda sahiptir. Programlamada çok faydalı olmasının yanı sıra, sınıfları ve nesneleri öğrenmek için de iyi bir örnektir.

9.2.1 Bir Dize Oluşturma

Bir dize nesnesi, bir dize sabitinden veya bir karakter dizisinden oluşturulabilir. Bir dizeyi bir dize sabitinden oluşturmak için, aşağıdaki sözdizimini kullanın:

```
String newString = tnewtString(stringLiteral);
```

Argüman **stringLiteral**, çift tırnaklar içinde yer alan bir karakter dizisidir. Aşağıdaki ifade, string literal "Welcome

to Java" için bir *String* nesne message oluşturur:

```
String message = newString("Java'ya Hoş Geldiniz");
```

string literal nesnesi

Java, bir dize literalini bir **String** nesnesi olarak ele alır. Bu nedenle, aşağıdaki ifade geçerlidir:

```
String message = "Java'ya hoş geldiniz";
```

Bir karakter dizisinden bir dize de oluşturabilirsiniz. Örneğin, aşağıdaki durum-
metinleri dizeyi oluşturur "İyi Gün":

```
char[] charArray = {'t','G','t','o','t','t','d','t','t','t','D','t','a','t','t','y','t'};
String message = tnewtString(charArray);
```



Not

A**String** değişkeni, bir **String** nesnesine referans tutar ve bir dize değerini saklar. Kesin olarak konuşursak, terimler String değişkeni, String nesnesi ve dize değeri farklıdır, ancak çoğu zaman aralarındaki ayrımlar göz ardı edilebilir. Basitlik açısından, terim *dize* genellikle **String** değişkeni, **String** nesnesi ve dize değeri için kullanılacaktır.

String değişkeni, String nesne, string değeri

9.2.2 Değiştirilemez Dizeler ve İçerilen Dizeler

değiştirmez

A**String** nesnesi *değiştirmez*; içeriği *değiştirilemez*. Aşağıdaki kod string'in içeriğini değiştiriyor mu?

```
String s = t"Java";
ns = t"HTML";
```

Cevap hayır. İlk ifade, içeriği "Java" olan bir **String** nesnesi oluşturur ve referansını s'ye atar. İkinci ifade, içeriği olan yeni bir **String** nesnesi oluşturur

"HTML" ve referansını `s`'ye atar. İlk `String` nesnesi atamadan sonra hala mevcuttur, ancak artık erişilemez, çünkü değişken `s` artık yeni nesneye işaret etmektedir, bu da Şekil 9.1'de gösterilmiştir.

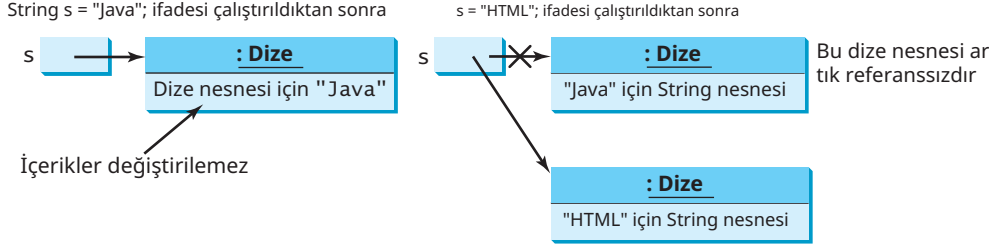
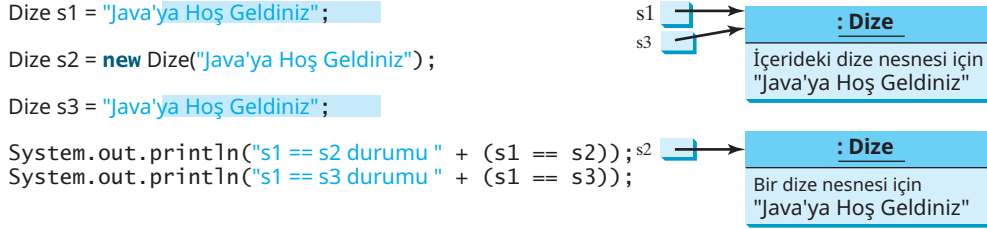


FIGURE 9.1 Dizeler değiştirilemez; bir kez oluşturulduktan sonra, içerikleri değiştirilemez.

`String`'ler değişmezdir ve programlamada yaygındır, bu nedenle JVM, verimliliği artırmak ve bellek tasarrufu sağlamak için aynı karakter dizisine sahip string literal'ler için benzersiz bir örnek kullanır. Böyle bir örneğe *interned string* denir. Örneğin, aşağıdaki iki ifadeler:



görüntüle

```
s1 == s2 is false
s1 == s3 is true
```

Önceki ifadelere göre, `s1` ve `s3` aynı interned string'e—"Java'ya Hoş Geldiniz"—işaret etmektedir, bu nedenle `s1 == s3` doğrudur. Ancak, `s1 == s2` yanlıştır, çünkü `s1` ve `s2` iki farklı string nesnesidir, aynı içeriğe sahip olsalar bile.

9.2.3 String Karşılaştırmaları

`String` sınıfı, Şekil 9.2'de gösterildiği gibi, dizeleri karşılaştırmak için yöntemler sağlar.

İki dizenin içeriğini nasıl karşılaştırırsınız? Aşağıdaki gibi `==` operatörünü kullanmaya çalışabilirsiniz:

```
if(dize1 == dize2)
    System.out.println("string1 ve string2 aynı nesnedir");
else
    System.out.println("string1 ve string2 farklı nesnelerdir");
```

Ancak, `==` operatörü yalnızca `string1` ve `string2` aynı nesneye referans verip vermediğini kontrol eder; aynı içeriğe sahip olup olmadıklarını söylemez. Bu nedenle, iki string değişkeninin aynı içeriğe sahip olup olmadığını öğrenmek için `==` operatörünü kullanamazsınız. Bunun yerine, `equals` metodunu kullanmalısınız. Örneğin, aşağıdaki kod iki stringi karşılaştırmak için kullanılabilir:

```
if(string1.equals(string2))
    System.out.println("string1 ve string2 aynı içeriğe sahiptir");
else
    System.out.println("string1 ve string2 eşit değildir");
```

string1.equals(string2)

java.lang.String	
+equals(s1: Object): boolean+equa	Returns true if this string is equal to string s1.
equalsIgnoreCase(s1: String): boolean	Returns true if this string is equal to string s1 case insensitive.
+karşılaştır(s1: String): int	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
+compareToIgnoreCase(s1: String): int	Same as compareTo except that the comparison is case insensitive.
+regionMatches(index: int, s1: String, ns1Index: int, len: int): boolean+regionMatches(ignoreCase: boolean, nindex: int, s1: String, s1Index: int, nlen: int): boolean	Returns true if the specified subregion of this string exactly matches the specified subregion in string s1.
+startsWith(prefix: String): boolean+endsWith(suffix: String): boolean	Same as the preceding method except that you can specify whether the match is case sensitive.
	Returns true if this string starts with the specified prefix.
	Returns true if this string ends with the specified suffix.

FIGURE 9.2 **String** sınıfı, dizeleri karşılaştırmak için yöntemleri içerir.

equals yönteminin parametre türünün Object olduğunu unutmayın. Objectsınıfını Bölüm 11'de tanıtacağız. Şimdilik, iki dizeyi karşılaştırmak için equals yöntemini kullanmak üzere Object yerine String ile değiştirebilirsiniz. Örneğin, aşağıdaki ifadeler true ve ardından false gösterir.

```
String s1 = newString("Java'ya Hoş Geldiniz");
String s2 = "Java'ya Hoş Geldiniz";
String s3 = "Java'ya Hoş Geldiniz C++";
System.out.println(s1.equals(s2)); // true
System.out.println(s1.equals(s3)); // false
```

compareTo yöntemi de iki dizeyi karşılaştırmak için kullanılabilir. Örneğin, aşağıdaki kod u düşünün:

```
s1.karşılaştır(s2)           s1.karşılaştır(s2)
```

Yöntem, eğer s1 s2'ye eşitse 0 değerini, eğer s1 s2'den leksikografik olarak (yani, Unicode sıralama sına göre) daha küçükse 0'dan küçük bir değer, eğer s1 s2'den leksikografik olarak daha büyükse 0'dan büyük bir değer döndürür.

compareTo yönteminden dönen gerçek değer, s1 ve s2 içindeki ilk iki farklı karakterin soldan sağa olan ofsetine bağlıdır. Örneğin, s1abc ve s2abg ise, s1.compareTo(s2) -4 döner. İlk iki karakter (a vs. a) s1 ve s2 içinden karşılaştırılır. Çünkü eşittirler, ikinci iki karakter (b vs. b) karşılaştırılır. Üçüncü iki karakter (c vs. g) de eşit olduğu için karşılaştırılır. Karakter c, g'den 4 daha az olduğu için karşılaştırma -4 döner.



Dikkat

Karşılaştırma operatörlerini kullanarak dizeleri karşılaştırırsanız sözdizimi hataları oluşacaktır >, >=, < veya <= kullanmak yerine, s1.compareTo(s2) kullanmalısınız.



Not

equals metodu, iki dize eşitse true, değilse false döner. compareTo metodu, bir dize diğer dize eşit, büyük veya küçük olup olmadığına bağlı olarak 0, pozitif bir tamsayı veya negatif bir tamsayı döner.

String sınıfı ayrıca dizeleri karşılaştırmak için **equalsIgnoreCase**, **compareToIgnoreCase** ve **regionMatches** metodlarını sağlar. **equalsIgnoreCase** ve

`compareToIgnoreCase` yöntemleri, iki dizeyi karşılaştırırken harflerin büyük/küçük harf durumunu göz ardı eder. `regionMatches` yöntemi, iki dize parçalarını eşitlik açısından karşılaştırır. Ayrıca, `str.startsWith(prefix)` ile dize `str`'ün belirli bir ön ek ile başlayıp başlamadığını kontrol edebilir ve `str.endsWith(suffix)` ile dize `str`'ün belirli bir son ek ile bitip bitmediğini kontrol edebilirsiniz.

9.2.4 Dize Uzunluğunu ve Karakterleri Alma, ve Dilleri Birleş tirme

String sınıfı, bir dizinin uzunluğunu elde etmek, bireysel

karakterleri almak ve dizeleri birleştirmek için yöntemler sağlar; bu, Şekil 9.3'te gösterilmiştir.

java.lang.String	
+uzunluk(): intn+c harAt(index: int): charn+concat(s1: Dize): Dize	Returns the number of characters in this string. Returns the character at the specified index from this string. Returns a new string that concatenates this string with string s1.

FIGURE 9.3 **String** sınıfı, string uzunluğunu, bireysel karakterleri ve stringleri birleştirmek için yöntemleri içerir.

Bir dizinin uzunluğunu `length()` yöntemini çağırarak alabilirsiniz. Örneğin, `message.length()`, `message` dizisinin uzunluğunu döndürür.

uzunluk()



Dikkat

`length`, `String` sınıfında bir yöntemdir ancak bir dizi nesnesinin bir özelliğidir. Bu nedenle, `s.length()` ile dize `s` içindeki karakter sayısını almalı ve `a.length` ile dizi `a` içindeki eleman sayısını almalısınız.

string uzunluğu vs. dizi uzunluğu

`s.charAt(index)` yöntemi, `s` dizisinde belirli bir karakteri almak için kullanılabilir; bu rada indeks 0 ile `s.length()-1` arasında olmalıdır. Örneğin, `message.charAt(0)`, Şekil 9.4'te gösterildiği gibi `W` karakterini döndürür.

charAt(dizin)



Not

Bir dize kullandığınızda, genellikle onun literal değerini bilirsiniz. Kolaylık olması açısından, Java, yeni değişkenler oluşturmadan dizelere doğrudan atıfta bulunmak için dize literalini kullanmanıza izin verir. Böylece, `"Welcome to Java".charAt(0)` doğrudur ve `W` döner.

dize değişmezi

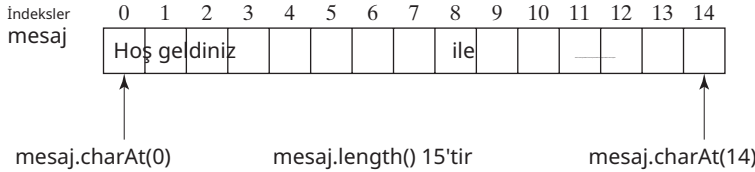


FIGURE 9.4 Bir **String** nesnesindeki karakterler, dahili olarak bir dizi kullanılarak saklanır.



Not

String sınıfı, karakterleri dahili olarak saklamak için bir dizi kullanır. Dizi özel `String` sınıfının dışından erişilemez. `String` sınıfı, string bilgilerini almak için birçok kamuya açık yöntem sağlar, örneğin `length()` ve `charAt(index)`. Bu, kapsülleme için iyi bir örnektir: sınıfın veri alanı, özel belirteç aracılığıyla kullanıcıdan gizlenmiştir ve bu nedenle kullanıcı doğrudan onu manipüle edemez. Dizi özel olmasaydı, kullanıcı diziyi değiştirerek string içeriğini değiştirebilirdi. Bu, `String` sınıfının değişmez olduğu ilkesini ihlal eder.

kapsayıcı dize

string indeks aralığı

**Dikkat**

Bir dizeye **s** erişmeye çalışmak, sınırların dışına çıkmak yaygın bir programlama hatasıdır. Bunu önlemek için, `s.length()-1`'den daha büyük bir indeks kullanmadığınızdan emin olun.

Örneğin, `s.charAt(s.length())` bir `StringIndexOutOfBoundsException`'ya neden olur.

İki dizeyi birleştirmek için `concat` yöntemini kullanabilirsiniz. Aşağıda gösterilen ifade, örneğin, `s1` ve `s2` dizelerini `s3` içine birleştirir:

```
s1.concat(s2)           String s3 = s1.concat(s2);
```

String birleştirmesi programlamada yoğun bir şekilde kullanıldığından, Java bunu başarmak için kullanışlı bir yol sağlar. İki string birleştirmek için artı (+) operatörünü kullanabilirsiniz, bu nedenle önceki ifade eşdeğerdir

```
S1 + kötü           String s3 = s1 + s2;
```

Aşağıdaki kod, `message`, " ve " ile `"HTML"`'yi birleştirerek tek bir dize oluşturur:

```
String myString = message + " and " + "HTML";
```

Unutmayın ki + operatörü bir sayıyı bir dize ile de birleştirebilir. Bu durumda, number bir dizeye dönüştürülür ve ardından birleştirilir. Birleştirmenin gerçekleşmesi için en az bir operandın bir dize olması gerektiğini unutmayın.

9.2.5 Alt Dize Elde Etme

Bir dizeden tek bir karakter elde etmek için `charAt` yöntemini kullanabilirsiniz; bu, Şekil 9.3'te gösterilmiştir. Bir dizeden bir alt dize elde etmek için `substring` yöntemini `String` sınıfında kullanabilirsiniz, Şekil 9.5'te gösterildiği gibi.

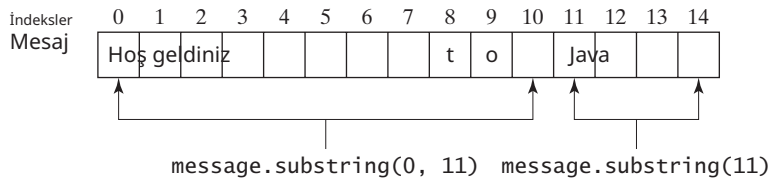
Örneğin,

```
String message = "Java'ya Hoş Geldiniz".substring(0,11) + "HTML";
```

string message artık HTML'ye Hoş Geldiniz olur.

java.lang.String	
+substring(beginIndex: int): Sicim	Returns this string's substring that begins with the character at the specified <code>beginIndex</code> and extends to the end of the string, as shown in Figure 9.6.
+substring(beginIndex: int, endIndex: int): Dize	Returns this string's substring that begins at the specified <code>beginIndex</code> and extends to the character at index <code>endIndex - 1</code> , as shown in Figure 9.6. Note that the character at <code>endIndex</code> is not part of the substring.

FIGURE 9.5 `String` sınıfı alt dizeleri elde etmek için yöntemleri içerir.



FÖRÜNTÜ 9.6 Alt dize yöntemi bir dizeden alt dize alır.



Not

Eğer `beginIndex` `endIndex` ise, `substring(beginIndex, endIndex)` uzunluğu 0 olan `beginIndex <= endIndex` boş bir dize döner. Eğer `beginIndex > endIndex` ise, bu bir çalışma zamanı hatası olur.

9.2.6 Dize Dönüştürme, Değiştirme ve Bölme

String sınıfı, dize dönüştürme, değiştirme ve bölme yöntemlerini sağlar,

Şekil 9.7'de gösterildiği gibi.

java.lang.String	
+ <code>toLowerCase(): String</code>	Returns a new string with all characters converted to lowercase.
+ <code>toUpperCase(): String</code>	Returns a new string with all characters converted to uppercase.
+ <code>trim(): String</code>	Returns a new string with whitespace characters trimmed on both sides.
+ <code>replace(oldChar: char, newChar: char): String</code>	Returns a new string that replaces all matching characters in this string with the new character.
+ <code>replaceFirst(oldString: String, newString: String): String</code>	Returns a new string that replaces the first matching substring in this string with the new substring.
+ <code>replaceAll(oldString: String, newString: String): String</code>	Returns a new string that replaces all matching substrings in this string with the new substring.
+ <code>split(delimiter: String): String[]</code>	Returns an array of strings consisting of the substrings split by the delimiter.

FIGURE 9.7 String sınıfı, dizeleri dönüştürme, değiştirme ve ayırma yöntemlerini içerir.

Bir dize oluşturulduğunda, içeriği değiştirilemez. `toLowerCase`, `toUpperCase`, `trim`, `replace`, `replaceFirst` ve `replaceAll` yöntemleri, orijinal dizeden türetilen yeni bir dize döndürür (orijinal dizeyi değiştirmeden!). `toLowerCase` ve `toUpperCase` yöntemleri, dizinin içindeki tüm karakterleri küçük veya büyük harfe dönüştürerek yeni bir dize döndürür. `trim` yöntemi, dizinin her iki ucundaki boşluk karakterlerini ortadan kaldırarak yeni bir dize döndürür. Dizedeki bir karakteri veya alt dizeyi yeni bir karakter veya yeni bir alt dize ile değiştirmek için birden fazla `replace` yöntemi sağlanmıştır.

Örneğin,

"Hoş geldin". `toLowerCase()` yeni bir dize döndürür, `hoş geldin`.

"Hoş geldin". `toUpperCase()` yeni bir dize döndürür, `HOŞ GELDİN`.

"İyi Geceler ". `trim()` yeni bir dize döndürür, `İyi Geceler`.

"Hoş geldin". `replace('e', 'A')` yeni bir dize döndürür, `HALcomA`.

"Hoş geldin". `replaceFirst("e", "AB")` yeni bir dize döndürür, `HABlcome`.

"Hoş geldin". `replaceAll("e", "AB")` yeni bir dize döndürür, `HABlcomAB`.

"Hoş geldin". `replace("el", "AB")` yeni bir dize döndürür, `HABcome`.

`toLowerCase()` t
`oUpperCase()` an
`trim()`
değiştiril
en değiştir
değiştir
yer değiştirmek

`split` metodu, belirtilen ayırıcılarla bir dizeden token'ları çıkarmak için kullanılabilir.

ayır

Örneğin, aşağıdaki kod

```
String[] belirteçler = t"Java#HTML#Perl".split(t"#");
nfort(i = t0; i < belirteçler.uzunluk; i++)
    System.out.print(belirteçler[i] + t" ");
```

gösterir

Java HTML Perl

9.2.7 Desenlere Göre Eşleştirme, Değiştirme ve Ayırma

Sıklıkla kullanıcı girişini doğrulayan kod yazmanız gerekecektir, örneğin girişin bir sayı, tamamen küçük harflerden oluşan bir dize veya bir Sosyal Güvenlik numarası olup olmadığını kontrol etmek için. Bu tür bir kodu nasıl yazarsınız? Bu görevi başarmanın basit ve etkili bir yolu, düzenli ifadeyi kullanmaktır.

A *düzenli ifade* (kısaca *regex*) bir dizi dizeyi eşleştirmek için bir deseni tanımlayan bir dize dir. Bir deseni belirterek bir dizeyi eşleştirebilir, değiştirebilir veya bölebilirsiniz. Bu, son derece yararlı ve güçlü bir özelliktir.

Öncelikle `String` sınıfındaki `matches` yöntemine bakalım. İlk bakışta, `matches` yöntemi `equals` yöntemine çok benziyor. Örneğin, aşağıdaki iki ifade her ikisi de `true` olarak değerlendirilir.

```
"Java".matches("Java");
"Java".equals("Java");
```

Ancak, `matches` yöntemi daha güçlüdür. Sadece sabit bir dizeyi değil, aynı zamanda bir deseni takip eden bir dizi dizeyi de eşleştirebilir. Örneğin, aşağıdaki ifadelerin hepsi `true` olarak değerlendirilir:

```
"Java eğlenceli".matches("ava.*");
"Java havalı".matches("Java.*n");
"Java güçlü".matches("Java.*n");
```

Önceki ifadelere göre `Java.*` bir düzenli ifadedir. `Java` ile başlayıp herhangi sıfır veya daha fazla karakterle devam eden bir dize desenini tanımlar. Burada, alt dize herhangi bir `.` herhangi sıfır veya daha fazla karakteri eşleştirir.

Aşağıdaki ifade `true` olarak değerlendirilir.

```
"440-02-4534".matches("d{3}-d{2}-d{4}");
```

Burada `\\d` tek bir rakamı temsil eder ve `\\d{3}` üç rakamı temsil eder.

`replaceAll`, `replaceFirst` ve `split` yöntemleri bir düzenli ifade ile kullanılabilir. Örneğin, aşağıdaki ifade, `$`, `+` veya `#` ile `NNN` dizisi ile değiştiren yeni bir dize döndürür.

```
String s = "a+b$#c".replaceAll("[$+#]", "t,NNN");
System.out.println(s);
```

Burada düzenli ifade `[$+#]`, `$`, `+` veya `#` ile eşleşen bir deseni belirtir. Bu nedenle, çıktı `aNNNbNNNNNNc`'dir.

Aşağıdaki ifade, stringi noktalama işaretleriyle sınırlı bir string dizisine böler.

```
String[] belirteçler = "Java,CC#,C++".split("[,;:]");
for (int i = 0; i < belirteçler.length; i++)
    System.out.println(belirteçler[i]);
```

Bu örnekte, düzenli ifade `[,;:;?]`, `...` veya `?` ile eşleşen bir deseni belirtir. Bu karakterlerin her biri, stringi bölmek için bir ayırıcıdır. Böylece, string `Java,CC#` ve `C++` olarak bölünür ve `tokens` dizisinde saklanır.

Düzenli ifade desenleri, başlangıç seviyesindeki öğrencilerin anlaması için karmaşıktır. Bu nedenle, bu bölümde basit desenler tanıtılmaktadır. Bu desenler hakkında daha fazla bilgi için Ek III.H, Düzenli İfadeler'e başvurun.

9.2.8 Bir String'de Bir Karakter veya Alt Dize Bulma

`String` sınıfı, bir string içinde bir karakter veya alt dize bulmak için birkaç aşırı yüklenmiş `indexOf` ve `lastIndexOf` yöntemi sağlar; bu, Şekil 9.8'de gösterilmiştir.

Neden düzenli ifade?
düzenli ifade

regex

eşleşmeler(regex)

Tümünü değiştir(regex)

bölme(normal ifade)

daha fazla çalışma

java.lang.String	
+indexOf(ch: char): intn+inde	Returns the index of the first occurrence of ch in the string. Returns -1 if not matched.
xOf(ch: char, fromIndex: sen): sen	Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matched.
+indexOf(s: String): intn+index	Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
Of(s: String, fromIndex: sen): sen	Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
+lastIndexOf(ch: int): intn+lastInd	Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
exOf(ch: int, fromIndex: int): intn+lastI	Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
ndexOf(s: String): intn+lastIndexOf	Returns the index of the last occurrence of string s. Returns -1 if not matched.
s: String, fromIndex: int): int	Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.

FIGURE 9.8 **String** sınıfı alt dizeleri eşleştirmek için yöntemleri içerir.

Örneğin,

"Java'ya Hoş Geldiniz".indexOf('W')**0** döner. indexOf
 "Java'ya Hoş Geldiniz".indexOf('o')**4** döner.
 "Java'ya Hoş Geldiniz".indexOf('o', 5) döner**9**.
 "Java'ya Hoş Geldiniz".indexOf("come") döner**3**.
 "Java'ya Hoş Geldiniz".indexOf("Java", 5) döner**11**.
 "Java'ya Hoş Geldiniz".indexOf("Java", 5) döner**-1**.

"Java'ya Hoş Geldiniz".lastIndexOf('W') döner**0**. lastIndexOf
 "Java'ya Hoş Geldiniz".lastIndexOf('o') döner**9**.
 "Java'ya Hoş Geldiniz".lastIndexOf('o', 5) döner**4**.
 "Java'ya Hoş Geldiniz".lastIndexOf("come") döner**3**.
 "Java'ya Hoş Geldiniz".lastIndexOf("Java", 5) döner**-1**.
 "Java'ya Hoş Geldiniz".lastIndexOf("Java") döner**11**.

9.2.9 Dize ve Diziler Arasında Dönüşüm

Dizeler diziler değildir, ancak bir dize bir diziye dönüştürülebilir ve tersine. Bir diziye karakterler dizisine dönüştürmek için **toCharArray** yöntemini kullanın. Örneğin, aşağıdaki **fade** diziyi **Java** bir diziye dönüştürür. toCharArray

```
char[] karakterler = t"Java".toCharArray();
```

Böylece, **chars[0]**J, **chars[1]**a, **chars[2]**v ve **chars[3]**a olur.

Ayrıca, **getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)** yöntemini kullanarak dizinin bir alt dizisini **srcBegin** indeksinden **srcEnd-1** indeksine kadar bir karakter dizisine **dst** kopyalayabilirsiniz. Örneğin, aşağıdaki kod, "3720" alt dizisini "CS3720" dizisinden **2** indeksinden **6-1** indeksine kadar **dst** karakter dizisine kopyalar ve **4** indeksinden başlar.

```
char[] dst = {'J', 'A', 'V', 'A', '3', '7', '2', '0'};  
"CS3720".getChars(2, 6, dst, 4); getChars
```

Böylece, **dst**{'J', 'A', 'V', 'A', '3', '7', '2', '0'} olur.

Bir karakter dizisini bir dizeye dönüştürmek için, `String(char[])` yapıcısını veya `valueOf(char[])` yöntemini kullanın. Örneğin, aşağıdaki ifade bir dizeyi `String` yapıcısını kullanarak bir diziden oluşturur.

```
String str = tnewString(tnew grafiği[]{'t','t','t','t','t','t','t','t'});
```

değer

Bir sonraki ifade, bir diziden bir dize oluşturmak için `valueOf` yöntemini kullanır.

```
String str = String.valueOf(tnew chart[]{'t','t','t','t','t','t','t','t'});
```

9.2.10 Karakterleri ve Sayısal Değerleri Dizeye Dönüştürme

Statik `valueOf` yöntemi, bir karakter dizisini bir dizeye dönüştürmek için kullanılabilir. Farklı parametre türleri ile karakter ve sayısal değerleri dizeye dönüştürmek için kullanılacak birkaç aşırı yüklenmiş `valueOf` yöntemi vardır, `char`, `double`, `long`, `int`, ve `float` gibi, Şekil 9.9'da gösterildiği gibi.

aşırı yüklenmiş `valueOf`

java.lang.String	
<code>+valueOf(c: char): String</code>	Returns a string consisting of the character c.
<code>Of(data: char[]): String</code>	Returns a string consisting of the characters in the array.
<code>+valueOf(f: float): String</code>	Returns a string representing the double value.
<code>ngn+valueOf(i: int): String</code>	Returns a string representing the float value.
<code>Of(l: long): String</code>	Returns a string representing the int value.
<code>boolean): String</code>	Returns a string representing the long value.
	Returns a string representing the boolean value.

ŞEKİL 9.9 Dize sınıfı, ilkel tür değerlerinden dizeler oluşturmak için statik yöntemleri içerir.

Örneğin, bir `double` değerini 5.44 bir dizeye dönüştürmek için `String.valueOf(5.44)` kullanın. Dönüş değeri, karakterlerden oluşan bir dizedir: `'5'`, `'.'`, `'4'` ve `'4'`.



Not

Bir dizeyi `double` değerine veya `int` değerine dönüştürmek için `Double.parseDouble(str)` veya `Integer.parseInt(str)` kullanabilirsiniz. `Double` ve `Integer`, `java.lang` paketi içinde bulunan iki sınıftır.

9.2.11 Dize Biçimlendirme

Dize sınıfı, biçimlendirilmiş bir dize oluşturmak için statik format yöntemini içerir. Bu yöntemi çağırmanın sözdizimi:

```
String.format(biçim, öge1, öge2, ..., ögek)
```

Bu yöntem, format yönteminin biçimlendirilmiş bir dize döndürmesi, printf yönteminin ise biçimlendirilmiş bir dize görüntülemesi dışında printf yöntemine benzer. Örneğin,

```
String s = String.format("t"%7.2f"%6d"%-4s"%t,t45.556,t14,t"AB"%t);nSystem.out.println(s);
```

gösterir

```
□□45.56□□□14AB□□
```

Dikkat edin ki

```
System.out.printf(format, öge1, öge2, ..., ögek);
```


eşdeğerdir

```
System.out.printf(
    String.format(biçim, öge1, öge2, ..., ögek));
```

burada kare kutu () boş bir alanı belirtir.

9.1 Diyelim ki **s1**, **s2**, **s3** ve **s4** aşağıdaki gibi dört dizedir:

```
String s1 = "Java'ya Hoş Geldiniz";
String s2 = s1;
String s3 = newString("Java'ya Hoş Geldiniz");
String s4 = "Java'ya Hoş Geldiniz";
```

Aşağıdaki ifadelerin sonuçları nelerdir?

- | | |
|--|--|
| a. <code>s1 == kötü</code> | m. <code>s1.uzunluk()</code> |
| b. <code>Kötü == kötü</code> | n. <code>s1.substring(t5t)</code> |
| c. <code>s1.eşittir(s2)</code> | . <code>s1.substring(t5t,t11t)</code> |
| d. <code>s2.eşittir(s3)</code> | p. <code>s1.startsWith(t"Wel"t)</code> |
| ve. <code>s1.karşılaştır(s2)</code> | q. <code>s1.endsWith(t"Java"t)</code> |
| f. <code>Kötü.Karşılaştır(Sa)</code> | r. <code>s1.toLowerCase()</code> |
| g. <code>s1 == s4</code> | s. <code>s1.toUpperCase()</code> |
| h. <code>s1.Charat(श0ढ)</code> | t. <code>"Hoş geldiniz ".trim()</code> |
| Ben. <code>s1.indexOf(t'j't)</code> | içinde. <code>s1.replace(t'o't,t'T't)</code> |
| j. <code>s1.indexOf(t"to"t)</code> | v. <code>s1.replaceAll(t"o"t,t"T"t)</code> |
| k. <code>s1.lastIndexOf(t'a't)</code> | içinde. <code>s1.replaceFirst(t"o"t,t "T"t)</code> |
| l. <code>s1.lastIndexOf(t"o"t,t15t)</code> | x. <code>s1.toCharArray()</code> |

9.2 String **Java'ya Hoş Geldiniz** oluşturmak için, şöyle bir ifade kullanabilirsiniz:

```
String s = "Java'ya Hoş Geldiniz";
```

veya:

```
String s = newString("Java'ya Hoş Geldiniz");
```

Hangisi daha iyi? Neden?

9.3 Varsayalım ki **s1** ve **s2** iki dizedir. Aşağıdaki ifadelerden hangileri yanlıştır?

```
String s = newString("yeni dize");
String s3 = s1 + s2;
String s3 = s1 - s2; ns1 ==
s2;
s1 >= s2; ns
1.compareTo(s2);
intti = s1.length(); nchartc
= s1(t0t);
karakterler = s1.charts(s1.length());
```

9.4 Aşağıdaki kodun çıktısı nedir?

```
String s1 = "Java'ya Hoş Geldiniz";
String s2 = s1.replace("o", "abc");
System.out.println(s1);
System.out.println(s2);
```

9.5 Let s1 " Hoşgeldin " ve s2 " hoşgeldin " olsun. Aşağıdaki

ifadeler için kodu yazın:

- `s1` ile `s2`'nin eşit olup olmadığını kontrol edin ve sonucu bir Boolean değişkenine atayın **Eşittir**.
- `s1` ile `s2`'nin eşit olup olmadığını kontrol edin, büyük/küçük harf duyarlı olarak, ve sonucu bir Boolean değişkenine **isEqual** atayın.
- `s1` ile `s2`'yi karşılaştırın ve sonucu bir **int** değişkenine **x** atayın.
- `s1` ile `s2`'yi karşılaştırın, büyük/küçük harf duyarlı olarak, ve sonucu bir **int** değişkenine **x** atayın.
- `s1`'nin **AAA** ön eki olup olmadığını kontrol edin ve sonucu bir Boolean değişkenine **b** atayın.
- `s1`'nin **AAA** son eki olup olmadığını kontrol edin ve sonucu bir Boolean değişkenine **b** atayın.
- `s1`'nin uzunluğunu bir **int** değişkenine **x** atayın.
- `s1`'nin ilk karakterini bir **char** değişkenine **x** atayın.
- `s1` ile `s2`'yi birleştiren yeni bir **s3** dizesi oluşturun.
- `1` indeksinden başlayarak `s1`'ün bir alt dizesini oluşturun.
- `1` indeksinden `4` indeksine kadar `s1`'ün bir alt dizesini oluşturun.
- `s1`'ü küçük harfe dönüştüren yeni bir **s3** dizesi oluşturun.
- `s1`'ü büyük harfe dönüştüren yeni bir **s3** dizesi oluşturun.
- `s1`'ün her iki ucundaki boşlukları kesen yeni bir **s3** dizesi oluşturun.
- `s1`'deki **e** karakterinin tüm örneklerini **E** ile değiştirin ve yeni dizeyi **s3**'ye atayın.
- Welcome to Java and HTML**'yi bir boşlukla ayrılmış **tokens** dizisine ayırın.
- `s1`'deki **e** karakterinin ilk örneğinin indeksini bir **int** değişkeni **x**'ye atayın.
- `s1`'deki **abc** dizesinin son örneğinin indeksini bir **int** değişkeni **x**'ye atayın.

9.6 String sınıfındaki herhangi bir yöntem dize içeriğini değiştirir mi?

9.7 Diyelim ki `s`, `new String()` kullanılarak oluşturuldu; `s.length()` nedir?

9.8 Bir **char**, bir karakter dizisi veya bir sayıyı nasıl bir dizeye dönüştürürsünüz?

9.9 Aşağıdaki kod neden bir **NullPointerException** oluşturur?

```

1 genel sınıfTest {
2     PrivateString metni;
3
4     public test(Dize s) {
5         Dize metni = s;
6     }
7
8     public static void main(String[] args) {
9         Test testi = new Test("ABC");
10        System.out.println(testi.text.toLowerCase());
11    }
12 }
```

9.10 Aşağıdaki programda ne yanlış?

```

1 genel sınıfTest {
2     Dize metni;
3 }
```

```

4     halk     geersiz Test(String s) {n5 te
xt = s;
6     }
7
8     public static voidtmain(String[] args) {n9 Test testi
= tnewtTest(t"ABC"t);n10 System.out.println(
test);n11 }

12 }

```

9.11 Aşağıdaki kodun çıktısını gösterin.

```

genel sınıfTest {
    public static voidmain(String[] args) {
        System.out.println("Merhaba, ABC, iyi".matches("ABC "));
        System.out.println("Merhaba, ABC, iyi".matches(".*ABC.*"));
        System.out.println("A,B;C".replaceAll(",;", "#"));
        System.out.println(t"A,B;C"t.replaceAll(t"[;]"t,t"#"t));nString[] token

        s = t"A,B;C"t.split(t"[;]"t);
        fort(tintti = t0t; i < tokens.length; i++)nSystem.out.
            print(tokens[i] +t" "t);n}

    }
}

```

9.3 Vaka Çalışması: Palindromları Kontrol Etme

Bu bölüm, bir dizenin palindrom olup olmadığını kontrol eden bir program sunmaktadır.

Bir dize, ileri ve geri okunduğunda aynıysa palindromdur. "mom", "dad" ve "noon" gibi kelimeler palindromlardır.

Problem, kullanıcıdan bir dize girmesini isteyen ve dizenin palindrom olup olmadığını rapor eden bir program yazmaktır. Bir çözüm, dizenin ilk karakterinin son karakterle aynı olup olmadığını kontrol etmektir. Eğer öyleyse, ikinci karakterin sondan ikinci karakterle aynı olup olmadığını kontrol edin. Bu işlem, bir uyumsuzluk bulunana kadar veya dizenin tüm karakterleri kontrol edilene kadar devam eder; eğer dize tek sayıda karaktere sahipse ortadaki karakter hariç tutulur.

Bu fikri uygulamak için, dizenin başındaki ve sonundaki iki karakterin konumunu belirtmek için iki değişken kullanın, diyelim ki **low** ve **high**. Listing 9.1'de (satırlar 22, 25) gösterildiği gibi. Başlangıçta, **low** 0'dır ve **high** s.length() - 1'dir. Eğer bu konumlardaki iki karakter eşleşirse, **low** değerini 1 artırın ve **high** değerini 1 azaltın (satırlar 31-32). Bu işlem, (**low** >= **high**) veya bir uyumsuzluk bulunana kadar devam eder.

LISTELER 9.1 CheckPalindrome.java

```

1  import java.util.Scanner;
2
3  public class CheckPalindrome {
4      /** Ana yöntem */
5      public static voidtmain(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişı = tnewtScanner(System.in);
8
9          // Kullanıcıdan bir dize girmesini iste
10         System.out.print("Bir dize girin: ");
11         ();
12
13         ift() isPalindrome(s)nString s = input.nextLine

```



VideoNotu

Palindrom kontrolü

girdi dizesi

```

14         System.out.println(s + " bir palindromdur");
15     aksi takdirde
16         System.out.println(s + " bir palindrom değildir");
17     }
18
19     /** Bir string'in palindrom olup olmadığını kontrol et */
20     public static boolean isPalindrome( Dizeler ) {
21         // String'deki ilk karakterin indeksi
22         int low = 0;
23
24         // String'deki son karakterin indeksi
25         int high = uzunluk.uzunluk() - 1;
26
27         while(düşük < yüksek) {
28             if( s.charAt(düşük) != s.charAt(yüksek)
29                 return false; // Palindrom değil
30
31         düşük++; // 32. yük
32         sek—; // 33 }

34
35     doğru döndür; // Dize bir palindromdur
36 {Hayı
37 r}

```

alt indeks

üst indeks

indeksleri güncelle



Bir dize girin: noon Enter
noon bir palindromdur



Bir dize girin: ay Enter
ay bir palindrom değildir

Scanner sınıfındaki `nextLine()` metodu (satır 11) bir satırı `s` içine okur ve ardından `isPalindrome(s)` `s`'nin bir palindrom olup olmadığını kontrol eder (satır 13).

9.4 Vaka Çalışması: Onaltılı Sayıları Onluk Sayılara Dönüştürme

Bu bölüm, bir onaltılı sayıyı onluk bir sayıya dönüştüren bir programı sunmaktadır.



Ana
Nokta

Bölüm 5.7, bir onluk sayıyı onaltılıya dönüştüren bir program vermektedir. Bir onaltılı sayıyı onluğa nasıl dönüştürürsünüz?

Verilen bir onaltılı sayı $h_{n-1}t_{n-1}t_{n-2}t_{n-3} \dots t_1t_0$, eşdeğer onluk değeri

$$h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \dots + h_1 \times 16^1 + h_0 \times 16^0$$

Örneğin, onaltılı sayı **AB8C** şudur:

$$10 \times 16^3 + 11 \times 16^2 + 8 \times 16^1 + 12 \times 16^0 = 43916$$

Programımız, kullanıcıdan bir onaltılı sayıyı bir dize olarak girmesini isteyecek ve bunu aşağıdaki yöntemle onluğa dönüştürecektir:

```
public static int hexToDecimal(String hex)
```

Bir zorlayıcı yaklaşım, her onaltılı karakteri bir onluk sayıya dönüştürmek, bunu `i` pozisyonundaki `16i` bir onaltılı basamak için çarpmak ve ardından tüm öğeleri bir araya ekleyerek onaltılı sayının eşdeğer onluk değerini elde etmektir.

Dikkat edin ki

$$h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \dots + h_1 \times 16^1 + h_0 \times 16^0$$

$$= (\dots ((h_n \times 16 + h_{n-1}) \times 16 + h_{n-2}) \times 16 + \dots + h_1) \times 16 + h_0$$

Bu gözlem, Horner algoritması olarak bilinir, bir onaltılı dizeyi ondalık bir sayıya dönüştürmek için aşağıdaki verimli koda yol açar:

```
inttdecimalValue = t0t;nfo
rt(tintti = t0t; i < hex.length(); i++) {ncharthexChar
= hex.charAt(i);
decimalValue = decimalValue * t16t+ hexCharToDecimal(hexChar);n}
```

İşte onaltılı sayı **AB8C** için algoritmanın bir izleme kaydı:

	i	hexChar	hexCharToDecimal(hexChar)	ondalıkDeğer
döngüden önce				0
1. yinelemeden sonra	0	A	10	10
2. yinelemeden sonra	1	B	11	10 * 16 + 11
3. yinelemeden sonra	2	8	8	(10 * 16 + 11) * 16 + 8
4. yinelemeden sonra	3	C	12	((10 * 16 + 11) * 16 + 8) * 16 + 12



Liste 9.2, tam programı vermektedir.

LİSTELEN9.2HexToDecimalConversion.java

```
1  import java.util.Scanner;
2
3  genel classtHexToDecimalConversion {
4      /** Ana yöntem */
5      public static voidtmain(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = tnewtScanner(System.in);
8
9          // Kullanıcıdan bir dize girmesini iste
10         System.out.print("Bir hex numarası girin: ");
11         String hex = input.nextLine();          girdi dizesi
12
13         System.out.println("Hex numarasının ondalık değeri "
14             + hex + " şudur " + hexToDecimal(hex.toUpperCase()));
15     }
16
17     public static intthexToDecimal(String hex) {
18         intdecimalValue = t0t;
19         fort(tint = t0t; i < hex.length(); i++) {
20             charthexChar = hex.charAt(i);
21             decimalValue = decimalValue * t16t+ hexCharToDecimal(hexChar);n22 }
23
24         returntdecimalValue;n25
25     }
26 }
```

onaltılıdan ondalığa

hex karakterini ondalık sayıya çevir
büyük harfe çevir

```
2728 public static int hexCharToDecimal(char ch) {
29     eğer(ch >= 'A' && ch <= 'F')
30         return 10 * ch - 'A';
31     aksi takdirde // ch '0', '1', ..., veya '9'
32     dönüş - '0';
33 }
```



Bir hex numarası girin: AB8C
Hex numarası AB8C için ondalık değer 43916



Bir hex numarası girin: af71
Hex numarası af71 için ondalık değer 44913

Program, konsoldan bir dize okur (satır 11) ve bir hex dizesini ondalık sayıya dönüştürmek için `hexToDecimal` yöntemini çağırır (satır 14). Karakterler küçük veya büyük harf olabilir. `hexToDecimal` yöntemini çağırılmadan önce büyük harfe dönüştürülürler.

`hexToDecimal` yöntemi, bir tamsayı döndürmek için satır 17–25'te tanımlanmıştır. Dizenin uzunluğu, satır 19'da `hex.length()` çağrılarak belirlenir.

`hexCharToDecimal` yöntemi, bir hex karakteri için ondalık bir değer döndürmek üzere satır 27–32'de tanımlanmıştır. Karakter küçük veya büyük harf olabilir. İki karakteri çıkarmanın, Unicode'larını çıkarmak olduğunu hatırlayın. Örneğin, '5' – '0', 5'dir.

9.5 Karakter Sınıfı



Bir karakter için bir nesne oluşturmak üzere `Karakter` sınıfını kullanabilirsiniz. Bir `Karakter` nesnesi bir karakter değeri içerir.

wrapper sınıfı

Java API'sindeki birçok yöntem bir nesne argümanı gerektirir. Temel veri değerlerinin nesne olarak işlenebilmesi için Java, her temel veri türü için bir sınıf sağlar. Bu sınıflar `Karakter`, `Boolean`, `Byte`, `Short`, `Integer`, `Long`, `Float` ve `Double` sırasıyla `char`, `boolean`, `byte`, `short`, `int`, `long`, `float` ve `double` için. Bu sınıflara *sarmalayıcı sınıflar* denir çünkü her biri bir nesne içinde bir temel tür değerini sarmalar veya kapsüller. Tüm bu sınıflar `java.lang` paketindedir ve temel değerleri işlemek için yararlı yöntemler içerir. Bu bölüm `Karakter` sınıfını tanıtmaktadır. Diğer sarmalayıcı sınıflar Bölüm 10, Nesnelerde Düşünme'de tanıtılacaktır.

`Karakter` sınıfı, bir karakterin kategorisini (büyük harf, küçük harf, rakam vb.) belirlemek ve karakterleri büyük harften küçük harfe ve tersine dönüştürmek için bir yapıcı ve birkaç yöntem içerir; bu, Şekil 9.10'da gösterilmiştir.

Bir `char` değerinden bir `Karakter` nesnesi oluşturabilirsiniz. Örneğin, aşağıdaki ifade `a` karakteri için bir `Karakter` nesnesi oluşturur.

```
Karakter karakter = new Karakter('a');
```

`charValue` metodu, `Karakter` nesnesi içinde sarılı karakter değerini döndürür. `compareTo` metodu, bu karakteri başka bir karakterle karşılaştırır ve bu karakterin Unicode'u ile diğer karakterin Unicode'u arasındaki farkı döndüren bir tamsayı verir. `equals` metodu, iki karakterin aynı olması durumunda yalnızca `true` döndürür. Örneğin, `charObject` yeni `Karakter('b')` olsun:

```
charObject.compareTo(new Karakter('a')) döner1
charObject.compareTo(new Karakter('b')) döner0
charObject.compareTo(new Karakter('c')) döner-1
```

`charObject.compareTo(new Character('d'))` döner -2
`charObject.equals(new Character('b'))` döner `true`
`charObject.equals(new Character('d'))` döner `false`

java.lang.Character	
+Karakter(değer: karakter)n+kar akterDeğeri(): karakter +compareTo(başka bir Karakter: Karakter): intn+equals(başka bir Karakter: Karakter): booleann+isDigit(ch: char): boolean +isLetter(ch: char): boolean +isLetterOrDigit(ch: char): boolean +isLowerCase(ch: char): boolean +isUpperCase(ch: char): boolean +toLowerCase(ch: char): char +toUpperCase(ch: karakter): karakter	Bir karakter nesnesi <code>char</code> değeri ile oluşturur. Bu nesneden <code>char</code> değerini döndürür. Bu karakteri başka bir karakterle karşılaştırır. Bu karakter başka bir karakterle eşitse <code>true</code> döndürür. Belirtilen karakter bir rakamsa <code>true</code> döndürür. Belirtilen karakter bir harfse <code>true</code> döndürür. Karakter bir harf veya rakamsa <code>true</code> döndürür. Karakter küçük harfse <code>true</code> döndürür. Karakter büyük harfse <code>true</code> döndürür. Belirtilen karakterin küçük harfini döndürür. Belirtilen karakterin büyük harfini döndürür.

FIGURE 9.10 Karakter sınıfı bir karakteri manipüle etmek için yöntemler sağlar.

Karakter sınıfındaki çoğu yöntem statik yöntemlerdir. `isDigit(char ch)` metodu, karakter bir rakam ise `true` döner, ve `isLetter(char ch)` metodu `true` döner, eğer karakter bir harf ise. `isLetterOrDigit(char ch)` metodu `true` döner, eğer karakter bir harf veya rakam ise. `isLowerCase(char ch)` metodu `true` döner, eğer karakter bir küçük harf ise, ve `isUpperCase(char ch)` metodu `true` döner, eğer karakter bir büyük harf ise. `toLowerCase(char ch)` metodu, karakter için küçük harfi döner, ve `toUpperCase(char ch)` metodu, karakter için büyük harfi döner.

Şimdi kullanıcıdan bir dize girmesini isteyen ve dizideki her harfin büyük/küçük harf farkı gözetme ksizin kaç kez tekrarlandığını sayan bir program yazalım.

Bu problemi çözmek için adımlar şunlardır:

1. Dizideki tüm büyük harfleri, `String` sınıfındaki `toLowerCase`

yöntemini kullanarak küçük harfe çevirin.

2. Her biri bir harfin tekrar sayısını sayan 26 int değerinden oluşan bir dizi, diyelim ki `counts` oluşturun. Yani, `counts[0]` 'a' harflerinin sayısını, `counts[1]` 'b' harflerinin sayısını ve bu şekilde devam eder.

3. Dizideki her karakter için, bunun (küçük harf) bir harf olup olmadığını kontrol edin. Eğer öyleyse, dizideki karşılık gelen sayıyı artırın.

Liste 9.3, tam programı vermektedir.

LISTE 9.3 CountEachLetter.java

```

1  import java.util.Scanner;
2
3  genel classtCountEachLetter {
4      /** Ana yöntem */
5      public static voidtmain(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = tnewtScanner(System.in);
8      }

```

352 Bölüm 9 Stringler

```
9      // Kullanıcıdan bir dize girmesini iste
10     System.out.print("Dizeyi girin: ");
11     String s = input.nextLine();
12
13     // Her harfi saymak için countLetters yöntemini çağır
14     int[] counts = countLetters(s.toLowerCase());
15
16     // Sonuçları göster
17     for(tintti = t0t; i < sayim.uzunluk; i++) {
18         ift(sayilar[i] != t0t)
19             System.out.println((char)('a' + i) + "şunları gösterir "+
20                 counts[i] + ((counts[i] == 1) ? " kez": " kez"));
21     }
22
23     yirmi iki }
24
25     /** Dizedeki her harfi say */
26     public static int[] countLetters( Dizeler ) {
27         int[] sayıları = tnew int[t26t];
28
29         kale(renk tonu = t0 ; i < ) ; i++) {
30             eğer( Character.isLetter(s.charAt(i))ns.length(
31                 sayım[s.charAt(i) - t'a't]++;n31 }
32
33     dönüş sayımları;
34 {Nah
35 }
```



```
Bir dize girin: bunlar
a 3 kez görünür
b 3 kez görünür
x 1 kez görünür
```

Ana yöntem, bir satırı (satır 11) okur ve her harfin sayısını sayar tahtada **countLetters** yöntemini çağırarak (satır 14). Harflerin durumu unutulduğundan, program **toLowerCase** yöntemini kullanarak dizeyi tamamen küçük harfe çevirir ve yeni diziyi **countLetters** yöntemine geçirir.

countLetters yöntemi (satırlar 25–34) 26 elemanlı bir dizi döndürür. Her eleman bir harfin dizideki s sayısını sayar. Yöntem, dizideki her karakteri işler.

Eğer karakter bir harfse, karşılık gelen sayısı 1 artırılır. Örneğin, eğer karakter (s.charAt(i)) a ise, karşılık gelen sayı counts['a' -

'a'] (yani, counts[0]) olur. Eğer karakter b ise,

karşılık gelen sayı counts['b' -

'a'] (yani, counts[1]) olur, çünkü b'nin Unicode'u a'ninkinden 1 fazladır. Eğer karakter z ise,

karşılık gelen sayı counts['z' - 'a'] (yani, counts[25]) olur, çünkü z'nin Unicode'u a'ninkinden 25 fazladır.

9.12 Bir karakterin küçük harf mi yoksa büyük harf mi olduğunu nasıl belirlersiniz?

9.13 Bir karakterin alfanümerik olup olmadığını nasıl belirlersiniz?

9.14 Aşağıdaki kodun çıktısını gösterin.

```
genel sınıfTest {
    public static void main(String[] args) {
        String s = "Merhaba, Günaydın";
        System.out.println(m(s));n}
}
```



```

public static int count(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        if (Character.isUpperCase(s.charAt(i))) {
            count++;
        }
    }
    return count;
}

```

9.6 StringBuilder ve StringBuffer Sınıfları

StringBuilder ve **StringBuffer** sınıfları, **String** sınıfına benzer ancak **String** sınıfı değiştirilemez.



Genel olarak, **StringBuilder** ve **StringBuffer** sınıfları, bir **String**'in kullanıldığı her yerde kullanılabilir. **StringBuilder** ve **StringBuffer**, **String**'ye göre daha esnektir. Yeni içerikler ekleyebilir, insert edebilir veya **StringBuilder** ve **StringBuffer** nesnelerine ekleyebilirsiniz, oysa **String** nesnesinin değeri, **String** oluşturulduğunda sabittir.

StringBuilder sınıfı, **StringBuffer**'a benzer, ancak **StringBuffer**'daki **buffer**'ı değiştiren yöntemler senkronize'dir, bu da yalnızca bir görevin bu yöntemleri çalıştırmasına izin verildiği anlamına gelir. Eğer sınıf birden fazla görev tarafından aynı anda erişilebilecekse **StringBuffer** kullanın. Eşzamanlı programlama, Bölüm 32'de tanıtılacaktır. Sadece tek bir görev tarafından erişiliyorsa **StringBuilder** kullanmak daha verimlidir. **StringBuffer** ve **StringBuilder**'daki yapılar ve yöntemler neredeyse aynıdır. Bu bölüm **StringBuilder**'ı kapsar. Bu bölümdeki tüm **StringBuilder** ifadelerini **StringBuffer** ile değiştirebilirsiniz. Program, başka herhangi bir değişiklik olmadan derlenebilir ve çalıştırılabilir.

StringBuilder

StringBuilder sınıfı, **builder**'ı yönetmek ve **builder**'daki dizeleri değiştirmek için üç yapı ve 30'dan fazla metoda sahiptir. Yapıları kullanarak boş bir dize oluşturucu veya bir dizeden bir dize oluşturucu oluşturabilirsiniz; bu, Şekil 9.11'de gösterilmiştir.

StringBuilder yapılar

java.lang.StringBuilder	
+StringBuilder()	Boş bir dize oluşturucu oluşturur ve kapasitesi 16'dır.
+StringBuilder(kapasite: int)	Belirtilen kapasiteye sahip bir dize oluşturucu oluşturur.
+StringBuilder(s: String)	Belirtilen dizeye sahip bir dize oluşturucu oluşturur.

ŞEKİL 9.11 **StringBuilder** sınıfı, **StringBuilder** örnekleri oluşturmak için yapıları içerir.

9.6.1 StringBuilder'daki Dizeleri Değiştirme

Bir dize oluşturucunun sonuna yeni içerikler ekleyebilir, bir dize oluşturucuda belirtilen bir konumda yeni içerikler ekleyebilir ve bir dize oluşturucudaki karakterleri silebilir veya değiştirebilirsiniz; bu, Şekil 9.12'de listelenen yöntemleri kullanarak yapılır.

StringBuilder sınıfı, bir dize oluşturucuya **boolean**, **char**, **char[]**, **double**, **float**, **int**, **long** ve **String** eklemek için birkaç aşırı yüklenmiş yöntem sağlar. Örneğin, aşağıdaki kod, yeni bir dize oluşturmak için **StringBuilder**'a dizeleri ve karakterleri ekler, Java'ya hoş geldiniz.

```

StringBuilder stringBuilder = new StringBuilder();
stringBuilder.append("Hoşgeldiniz");
stringBuilder.append(' ');
stringBuilder.append("Java'ya");

```

ekle

java.lang.StringBuilder	
<pre> +append(veri: char[]): StringBuilder n+append (veri: char[], offset: int, len: int): StringBuilder +append(v:t aPrimitiveType): StringBuildern+appen d(s: String): StringBuilder +delete(startIndex: int, endIndex: int): StringBuilder +deleteCharAt(index: int): StringBuildern+insert(ind ex: int, data: char[], offset: int,nlen: int): StringBuilder +insert(offset: int, veri: char[]): StringBuilder +insert(offset: int, b:t aPrimitiveType): StringBuilder +insert(offset: int, s: String): StringBuildern+replace(startIn dex: int, endIndex: int, s: String): StringBuildern+reve rse(): StringBuildern+setCharAt(index: int, ch: char): void </pre>	Bu dize oluşturucusuna bir char dizi ekler. Bu dize oluşturucusuna data içindeki bir alt dizi ekler. Bir ilkel tür değerini dize olarak bu oluşturucuya ekler. Bu dize oluşturucusuna bir dize ekler. startIndex ile endIndex-1 arasındaki karakterleri siler. Belirtilen indeksteki bir karakteri siler. Belirtilen indekste dizideki verilerin bir alt dizisini oluşturucuya ekler. Bu oluşturucuya konum ofsetinde veri ekler. Bu oluşturucuya bir dizeye dönüştürülmüş bir değeri ekler. Bu oluşturucuya konum ofsetinde bir dize ekler. Bu yapıcısındaki karakterleri startIndex ile endIndex -1 arasındaki belirtilen dize ile değiştirir. Yapıcısındaki karakterleri tersine çevirir. Bu yapıcıda belirtilen indekste yeni bir karakter ayarlar.

FIGURE 9.12 DizeYapıcı sınıfı dize yapıcılarını değiştirmek için yöntemleri içerir.

StringBuilder sınıfı ayrıca bir dize oluşturucuya **boolean**, **char**, char dizisi, **double**, **float**, **int**, **long** ve **String** eklemek için aşırı yüklenmiş yöntemler içerir. Aşağıdaki kodu göz önünde bulundurun:

```
ekle          stringBuilder.insert(11, "HTML ve ");
```

Diyelim ki **stringBuilder**, **insert** yönteminin uygulanmasından önce Hoşgeldiniz Java içeriyor. Bu kod, **stringBuilder** içinde 11. konuma "HTML ve " ekler (tam olarak J'den önce). Yeni **stringBuilder**, Hoşgeldiniz HTML ve Java olur.

Ayrıca, iki **delete** yöntemi kullanarak oluşturucudaki bir dizeden karakterleri silebilir, dizeyi **reverse** yöntemiyle tersine çevirebilir, karakterleri **replace** yöntemiyle değiştirebilir, yeni bir karakteri **setCharAt** yöntemiyle bir dizede ayarlayabilirsiniz.

Örneğin, diyelim ki **stringBuilder**, aşağıdaki her bir yöntemin uygulanmasından önce Hoşgeldiniz Java içeriyor:

```

sil          stringBuilder.delete(8, 11) oluşturucuyu Hoşgeldiniz Java olarak değiştirir.
deleteCharAt stringBuilder.deleteCharAt(8) builder'ı Welcome o Java. olarak değiştirir.
ters         stringBuilder.reverse() builder'ıJava ot emocleW olarak değiştirir.
değiştir    stringBuilder.replace(11, 15, "HTML") builder'ıWelcome to HTML olarak değiştirir.
setCharAt   stringBuilder.setCharAt(0, 'w') builder'ıwelcome to Java olarak ayarlar.

```

setCharAt hariç tüm bu değiştirme yöntemleri iki şey yapar:

- string builder'ın içeriğini değiştirir
- string builder'ın referansını döndürür

Örneğin, aşağıdaki ifade

```
StringBuilder stringBuilder1 = stringBuilder.reverse();
```

Yapıcıdaki dizeyi tersine çevirir ve yapıcının referansını **StringBuilder** olarak atar. Böylece, **StringBuilder** ve **String** her ikisi de aynı **DizeYapıcı** nesnesine işaret eder. Bir değer döndüren yöntemin, eğer yöntemin döndürdüğü değeri önemsemiyorsanız, bir ifade olarak çağırabileceğini unutmayın. Bu durumda, döndürme değeri basitçe göz ardı edilir. Örneğin, aşağıdaki ifadede

```
StringBuilder.reverse();
```

geri dönüş değeri göz ardı edilir.



Tip

Bir dize değişiklik gerektirmiyorsa, **String** yerine **StringBuilder** kullanın. Java **String** için bazı optimizasyonlar gerçekleştirebilir, örneğin, interned dizeleri paylaşmak.

Dize veya DizeYapıcı?

9.6.2 toString, capacity, length, setLength, ve charAt Yöntemleri

StringBuilder sınıfı, bir dize oluşturucuyu manipüle etmek ve özelliklerini elde etmek için ek yöntemler sağlar, Şekil 9.13'te gösterildiği gibi.

java.lang.StringBuilder	
+toString(): Stringn+cap acity(): intn+charAt(i ndex: int): charn+length(): int +setLength(newLength: int): voidn+subst ring(startIndex: int): Stringn+substring(startI ndex: int, endIndex: int): Sicim +trimToSize(): geçersiz	Returns a string object from the string builder. Returns the capacity of this string builder. Returns the character at the specified index. Returns the number of characters in this builder. Sets a new length in this builder. Returns a substring starting at startIndex . Returns a substring from startIndex to endIndex-1 . Reduces the storage size used for the string builder.

ŞEKİL 9.13 **StringBuilder** sınıfı, string builder'ları değiştirmek için yöntemleri içerir.

capacity() yöntemi, dize oluşturucunun mevcut kapasitesini döndürür. Kapasite, dize oluşturucunun boyutunu artırmadan saklayabileceği karakter sayısıdır.

kapasite()

length() yöntemi, dize oluşturucuda gerçekten saklanan karakter sayısını döndürür. **setLength(newLength)** yöntemi, dize oluşturucunun uzunluğunu ayarlar. Eğer **newLength** argümanı, dize oluşturucunun mevcut uzunluğundan daha azsa, dize oluşturucu, verilen **newLength** argümanının tam olarak belirttiği karakter sayısını içerecek şekilde kısaltılır. Eğer **newLength** argümanı mevcut uzunluğa eşit veya daha büyükse, yeterli sayıda null karakter (**\u0000**) dize oluşturucuya eklenir, böylece **length newLength** argümanına dönüşür. **newLength** argümanı 0'dan büyük veya eşit olmalıdır.

uzunluk()
setLength(int)

charAt(index) metodu, string builder'daki belirli bir index'teki karakteri döndürür. Index 0 tabanlıdır. Bir string builder'ın ilk karakteri index 0'da, sonraki karakter index 1'de ve devam eder. Index argümanı 0'dan büyük veya eşit olmalı ve string builder'ın uzunluğundan küçük olmalıdır.

davranış



Not

String'in uzunluğu her zaman builder'ın kapasitesine eşit veya ondan küçüktür. Uzunluk, builder'da saklanan string'in gerçek boyutudur ve kapasite, builder'ın mevcut boyutudur. Daha fazla karakter eklendiğinde builder'ın kapasitesi otomatik olarak artırılır. İçsel olarak, bir string builder karakterlerden oluşan bir dizidir, bu nedenle

uzunluk ve kapasite

yapıcının kapasitesi dizinin boyutudur. Eğer yapıcının kapasitesi aşılsa, dizi yeni bir dizi ile değiştirilir. Yeni dizi boyutu $2 * (\text{önceki dizi boyutu} + 1)$.



Ttipp

Belirlenen başlangıç kapasitesine sahip bir [StringBuilder oluşturmak için](#) new StringBuilder (initialCapacity) kullanabilirsiniz. Başlangıç kapasitesini dikkatlice seçerek, programınızı daha verimli hale getirebilirsiniz. Eğer kapasite her zaman builder'ın gerçek uzunluğundan büyükse, JVM builder için belleği yeniden tahsis etmek zorunda kalmayacaktır. Öte yandan, kapasite çok büyükse, bellek alanını boşa harcamış olursunuz. Kapasiteyi gerçek boyuta indirmek için trimToSize() metodunu kullanabilirsiniz.

ilk kapasite

boyutunuKüçült()

9.6.3 Vaka Çalışması: Palindromları Kontrol Ederken Alfaniümerik Olmayan Karakterleri Göz Ardı Etme

Liste 9.1, bir stringin palindrom olup olmadığını kontrol etmek için bir stringdeki tüm karakterleri dikkate aldı. Bir stringin palindrom olup olmadığını kontrol ederken alfaniümerik olmayan karakterleri göz ardı eden yeni bir program yazın.

Problemi çözmek için adımlar şunlardır:

1. Alfaniümerik olmayan karakterleri kaldırarak stringi filtreleyin. Bu, boş bir string builder oluşturup, stringdeki her alfaniümerik karakteri string builder'a ekleyerek ve stringi string builder'dan döndürerek yapılabilir.

[Karakterin](#) `ch` bir harf veya rakam olup olmadığını kontrol etmek için [Character](#) sınıfındaki `isLetterOrDigit(ch)` metodunu kullanabilirsiniz.

2. Filtrelenmiş string'in tersini elde edin. Ters string'i

filtrelenmiş string ile [equals](#) metodu kullanarak karşılaştırın.

Tam program Liste 9.4'te gösterilmiştir.

LISTE 9.4 PalindromeIgnoreNonAlphanumeric.java

```
1  import java.util.Scanner;
2
3  genel sınıftPalindromeIgnoreNonAlphanumeric {
4      /** Ana yöntem */
5      public static void main(String[] args) {
6          // Bir Scanner oluştur
7          Tarayıcı girişi = new Scanner(System.in);
8
9          // Kullanıcıdan bir dize girmesini iste
10         System.out.print("Dizeyi girin: ");
11         String s = input.nextLine();
12
13         // Sonucu göster
14         System.out.println("Alfaniümerik olmayan karakterleri yok sayarak,
15 + s + " bir palindrom mu? " + isPalindrome(s));
16     }
17
18     /** Bir string palindrom ise true döndür */
19     public static boolean palindromMuKontrolEt(String s) {
20         // Alfaniümerik olmayan karakterleri ortadan kaldırarak yeni bir string oluşturun
21         String s1 = filter(s);
22
23         // s1'in tersini alarak yeni bir string oluşturun
24         String s2 = reverse(s1);
25
26         // Tersinin orijinal string ile aynı olup olmadığını kontrol edin
```

palindrom kontrolü

```

27     return s2.equals(s1);
28 }
29
30 /** Alfasayısal olmayan karakterleri ortadan kaldırarak yeni bir dize oluştur */
31 public static String filtrele(String s) {
32     // Bir dize oluşturucu oluştur
33     StringBuilder stringBuilder = new StringBuilder();
34
35     // Dize içindeki her karakteri inceleyerek alfanümerik karakterleri atla
36     for (int i = 0; i < s.length(); i++) {
37         if (Character.isLetterOrDigit(s.charAt(i))) {
38             stringBuilder.append(s.charAt(i));
39         }
40     }
41
42     // Yeni filtrelenenmiş bir dize döndür
43     return stringBuilder.toString();
44 }
45
46 /** Belirtilen bir dizeyi ters çevirerek yeni bir dize oluştur */
47 public static String ters(String s) {
48     StringBuilder stringBuilder = new StringBuilder(s);
49     stringBuilder.reverse(); // StringBuilder'da reverse'i çağır
50     return stringBuilder.toString();
51 }
52 }

```

harf veya rakam ekle

Bir dize girin:

Alfasayısal karakterleri göz ardı ederek,
ab<?>cba bir palindrom mu? true



Bir dize girin:

Alfanümerik olmayan karakterleri göz ardı ederek,
abcc<?>kabin bir palindrom mu? false



filter(String s) metodu (satırlar 31–44) s dizesindeki her karakteri inceler ve karakter bir harf veya sayısal karakter ise onu bir dize oluşturucuya kopyalar. filter metodu, oluşturucudaki dizeyi döndürür. reverse(String s) metodu (satırlar 47–51) belirtilen dizeyi tersine çeviren yeni bir dize oluşturur. filter ve reverse metodları her ikisi de yeni bir dize döndürür. Orijinal dize değiştirilmez.

Liste 9.1'deki program, bir dizeyi palindrom olup olmadığını kontrol ederken, dizinin her iki ucundan karakter çiftlerini karşılaştırır. Liste 9.4, reverse metodunu kullanır.

StringBuilder sınıfı, string'i tersine çevirir, ardından iki string'in eşit olup olmadığını karşılaştırır yani orijinal string'in palindrom olup olmadığını belirler.

9.15 **StringBuilder** ile **StringBuffer** arasındaki fark nedir?

9.16 Bir string'den nasıl bir string builder oluşturursunuz? Bir string builder'dan nasıl bir string döndürürsünüz?

9.17 **StringBuilder** sınıfındaki reverse metodunu kullanarak bir string s tersine çevirmek için üç ifade yazın.

9.18 20 karakter uzunluğundaki bir string s 'den, 4 indeksinden başlayıp 10 indeksine biten bir alt string'i silmek için üç ifade yazın. delete metodunu kullanın. **StringBuilder** sınıfı.

9.19 Bir string ve bir string builder'daki karakterlerin iç depolaması nedir?

9.20 Diyelim ki **s1** ve **s2** aşağıdaki gibi verilmiştir:

```
StringBuilder s1 = tnewtStringBuilder(t"Java");nStringBuil
der s2 = tnewtStringBuilder(t"HTML");
```

Her bir aşağıdaki ifadenin ardından **s1** değerini gösterin. İfadelerin bağımsız olduğunu varsayın.

- | | |
|--|---|
| a. <code>s1.append("eğlencelidir");</code> | g. <code>s1.deleteCharAt(t3t);</code> |
| b. <code>s1.append(s2);</code> | h. <code>s1.delete(t1t,t3t);</code> |
| c. <code>s1.insert(2,"eğlencelidir");</code> | <code>Ben.s1.reverse();</code> |
| d. <code>s1.insert(t1t, s2);</code> | j. <code>s1.replace(1, 3, "Bilgisayar");</code> |
| ve. <code>S1.Charat(:2t);</code> | k. <code>s1.substring(t1t,t3t);</code> |
| f. <code>s1.uzunluk();</code> | l. <code>s1.substring(t2t);</code> |

9.21 Aşağıdaki programın çıktısını göster:

```
genel sınıfTest {
    public static voidtmain(String[] args) {nString s = t"J
        ava";t;
        StringBuilder oluşturuca = tnewtStringBuilder(s);nchange
            (s, builder);

        System.out.println(s);nSyste
            m.out.println(oluşturuca);n}

    private static voidchange(String s, StringBuilder builder) {
        s = s + " ve HTML";
        builder.append(" ve HTML");
    }
}
```

9.7 Komut Satırı Argümanları



main metodu komut satırından string argümanları alabilir.

Belki de main metodunun alışılmadık başlığını, `String[]` türünde `args` parametresine sahip olduğunu zaten fark ettiniz. `args`'ın bir string dizisi olduğu açıktır. main metodu, bir parametreye sahip normal bir metod gibidir. Normal bir metodu, gerçek parametreler geçirerek çağırabilirsiniz. main'e argüman geçirebilir misiniz? Evet, elbette geçirebilirsiniz. Aşağıdaki örneklerde, `T` `estMain` sınıfındaki main metodu A'daki bir metod tarafından çağrılmaktadır.

```
genel sınıf tA {
    public static void tmain(String[] args) {nString[] stri
        ngs = {t"New York";t,
            "Boston";t,t"Atlanta";t};n}
    TestMain.main(dizeler);
}
```

```
genel sınıf tTestMain {
    public static voidtmain(String[] args) {
        fort(tintti = t0t; i < args.length; i++)nSystem.out.
            println(args[i]);
    }
}
```

Amain metodu sadece normal bir metoddur. Ayrıca, komut satırından argümanlar g geçirebilirsiniz.

9.7.1 **main** Metoduna String Geçirme

Programı çalıştırdığınızda komut satırından **main** metoduna string geçirebilirsiniz. Aşağıdaki komut satırı, örneğin, üç string ile **TestMain** programını başlatır: **arg0, arg1**, ve **arg2**:

```
java TestAna arg0 arg1 arg2
```

arg0, arg1, ve **arg2** dizgeleridir, ancak komut satırında çift tırnak içinde görünmeleri gerekmez. Dizgeler bir boşlukla ayrılır. Boşluk içeren bir dize çift tırnak için de olmalıdır. Aşağıdaki komut satırını düşünün:

```
java TestMain "İlk num" alpha 53
```

Programı üç dize ile başlatır: **İlk num**, **alpha**, ve **53**. **İlk num** bir dize olduğu için çift tırnak içinde yer alır. **53**'ün aslında bir dize olarak değerlendirildiğini unutmayın. Komut satırında **53** yerine **"53"** kullanabilirsiniz.

main metodu çağırıldığında, Java yorumlayıcısı komut satırı argümanlarını tutmak için bir dizi oluşturur ve dizi referansını **args**'ye geçirir. Örneğin, bir programı **n** argümanıyl a çağırırsanız, Java yorumlayıcısı aşağıdaki gibi bir dizi oluşturur:

```
args = tnewtString[n];
```

Java yorumlayıcısı daha sonra **args**'i **main** metodunu çağırmak için geçirir.



Not

Programı hiçbir dize geçirilmeden çalıştırırsanız, dizi **new**

String[0] ile oluşturulur. Bu durumda, dizi **0** uzunluğunda boş bir dizi olarak oluşturulur. **args** bu boş diziye referans verir. Bu nedenle, **args null** değildir, ancak **args.length 0**'dir.

9.7.2 Vaka Çalışması: Hesap Makinesi

Diyelim ki tam sayılar üzerinde aritmetik işlemler gerçekleştiren bir program geliştireceksiniz. Program, bir dize argümanında bir ifade alır. İfade, bir tam sayı, ardından bir operatör ve başka bir tam sayıdan oluşur. Örneğin, iki tam sayıyı toplamak için bu komutu kullanın:

```
java Hesap Makinesi "2 + 3"
```

Program aşağıdaki çıktıyı gösterecektir:

```
2 + 3 = 5
```

Şekil 9.14, programın örnek çalıştırmalarını göstermektedir.

Ana programa geçirilen dizeler **args**'da saklanır, bu da bir dize dizisidir. Bu durumda, ifadeyi tek bir dize olarak geçiriyoruz. Bu nedenle, dizi yalnızca bir öğe içerir **args[0]** ve **args.length 1**'dir.

Programdaki adımlar şunlardır:

1. **args.length** kullanarak ifadenin komut satırında bir argüman olarak sağlanıp sağlanmadığını belirleyin. Değilse, programı **System.exit(1)** kullanarak sonlandırın.
2. İfadeyi **args[0]** içindeki üç token'a ayırın: **tokens[0]**, **tokens[1]** ve **tokens[2]**.
3. **tokens[0]** ve **tokens[2]**

üzerindeki operandlar üzerinde **tokens[1]** içindeki operatörü kullanarak ikili aritmetik işlemi gerçekleştirin.



VideoNotu

Komut satırı argümanı

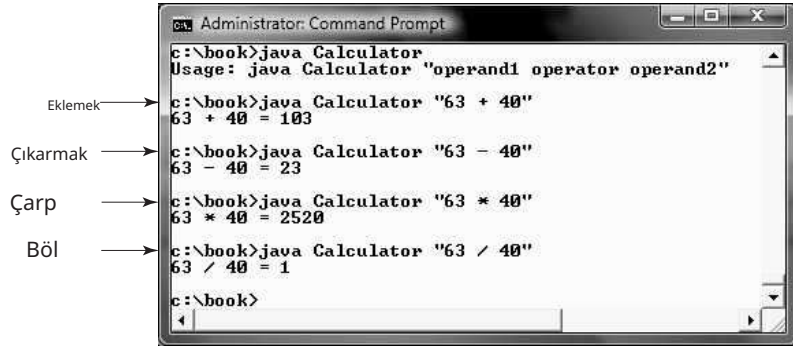


Fig9.14 Program, komut satırından bir argümanda (**operand1 operator operand2**) bir ifadeyi alır ve ifadeyi ve

aritmetik işlemin sonucunu görüntüler.

Program, Liste 9.5'te gösterilmiştir.

LISTING 9.5 Calculator.java

```

1  genel sınıf Hesap Makinesi {
2      /** Ana yöntem */
3      public static void main(String[] args) {
4          // Geçen string sayısını kontrol et
5          eğer (args.uzunluğu != 1) {
6              System.out.println(
7                  "Kullanım: java Calculator \"operand1 operatör operand2\"");
8              System.exit(1);
9          }

10
11         // İşlemin sonucu
12         int sonuç = 0;
13
14         // İşlemin sonucu
15         String[] tokens = args[0].split(" ");
16
17         // Operatörü belirle
18         anahtarlar jetonlar[1].charAt(0)
19         case '+': sonuç = Integer.parseInt(jetonlar[0]) + Integer.parseInt(jetonlar[2]);
20         Tamsayı.yüzde();
21         kırılma;
22         case '-': sonuç = Integer.parseInt(jetonlar[0]) - Integer.parseInt(jetonlar[2]);
23         Tamsayı.yüzde();
24         kırılma;
25         case '*': sonuç = Integer.parseInt(jetonlar[0]) * Integer.parseInt(jetonlar[2]);
26         Tamsayı.Yüzde();
27         kırılma;
28         case '/': sonuç = Integer.parseInt(jetonlar[0]) / Integer.parseInt(jetonlar[2]);
29         Tamsayı.Yüzde();
30     }

31
32     // Sonucu göster
33     System.out.println(jetonlar[0] + " " + jetonlar[1] + " " + jetonlar[2] + " = " + sonuç);
34
35 }
36 }

```


İfade, bir argümanda bir dize olarak geçirilir ve üç parçaya ayrılır—`tokens[0]`, `tokens[1]` ve `tokens[2]`—bir boşluk

ayırıcı olarak `split` yöntemini (satır 15) kullanarak.

`Integer.parseInt(tokens[0])` (satır 19) bir dijital dizeyi bir tam sayıya dönüştürür. Dize yalnızca rakamlardan oluşmalıdır. Eğer oluşmuyorsa, program anormal bir şekilde sonlanacaktır.

Bu programın çalışması için, ifade “operand1 opera-tor operand2” biçiminde girilmelidir. Operanlar ve operatör tam olarak bir boşlukla ayrılmıştır. Programı, ifadeleri farklı biçimlerde kabul edecek şekilde değiştirebilirsiniz (bkz. Programlama Egzersizi 9.28).

9.22 Bu kitap `main` yöntemini şöyle tanımlar

```
public static void main(String[] args)
```

Şunlardan biriyle değiştirilebilir mi?

```
public static void main(String args[])
public static void main(String[] x)
public static void main(String x[])
static void main(String x[])
```



9.23 Bu programın aşağıdaki gibi çağrıldığında çıktısını gösterin

1. `java Ttetttt Drtetam'ımı htavtet ediyorum`

2. `java Ttetsttt "1 2 3"`

3. `java Ttetstt`

```
genel sınıf Test {
    public static void main(String[] args) {
        System.out.println("String sayısı " + args.length);
        for(int i = 0; i < args.length; i++)
            System.out.println(args[i]);
    }
}
```

Anahtar Terimler

dahili dize 337

wrapper sınıfı 350

BÖLÜM ÖZETİ

1. String'ler `String` sınıfında kapsüllenmiş nesnelerdir. Bir string, 13 yapıcından biri kullanılarak veya basitçe bir string literal kullanılarak oluşturulabilir. Java, bir string literalini `String` nesnesi olarak ele alır.
2. A `String` nesnesi değişmezdir; içeriği değiştirilemez. Verimliliği artırmak ve bellek tasarrufu sağlamak için, JVM aynı karakter dizisine sahip iki literal string'i benzersiz bir nesnede saklar. Bu benzersiz nesneye *interned string* nesnesi denir.
3. Bir string'in uzunluğunu `length()` metodunu çağırarak alabilir, belirtilen `index` konumundaki karakteri `charAt(index)` metodu ile alabilir ve bir string'de bir karakter veya alt dize bulmak için `indexOf` ve `lastIndexOf` metodlarını kullanabilirsiniz.

4. İki dizeyi birleştirmek için **concat** yöntemini veya iki veya daha fazla dizeyi birleştirmek için **rtı (+)** operatörünü kullanabilirsiniz.
5. Dizeden bir alt dize elde etmek için **substring** yöntemini kullanabilirsiniz.
6. Dize karşılaştırmak için **equals** ve **compareTo** yöntemlerini kullanabilirsiniz. **equals** metodu, iki dize eşitse **true**, değilse **false** döner. **compareTo** metodu, bir dize diğer dize eşit, büyük veya küçük olduğuna bağlı olarak **0**, pozitif bir tam sayı veya negatif bir tam sayı döner.
7. *A düzenli ifade* (kısaca *regex*) bir dizi dize ile eşleşen bir deseni tanımlayan bir dizedir. Bir deseni belirterek bir dizeyi eşleştirebilir, değiştirebilir veya bölebilirsiniz.
8. Karakter sınıfı, tek bir karakter için bir sarmalayıcı sınıftır. Karakter sınıfı, bir karakterin bir harf (**isLetter(char)**), bir rakam (**isDigit(char)**), büyük harf (**isUpperCase(char)**) veya küçük harf (**isLowerCase(char)**) olup olmadığını belirlemek için yararlı statik yöntemler sağlar.
9. **StringBuilder** ve **StringBuffer** sınıfları, **String** sınıfının yerini almak için kullanılabilir. **String** nesnesi değişmezdir, ancak **StringBuilder** ve **StringBuffer** nesnelerine yeni içerikler ekleyebilir, yerleştirebilir veya ekleyebilirsiniz. Dize içeriği herhangi bir değişiklik gerektirmiyorsa **String** kullanın, değişebilirlerse **StringBuilder** veya **StringBuffer** kullanın.
10. Komut satırından **main** yöntemine dizeleri geçirebilirsiniz. **main** programına geçirilen dizeler **args** içinde saklanır, bu bir dizi dizisidir. İlk dize **args[0]** ile temsil edilir ve **args.length** geçirilen dize sayısını verir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak www.cs.armstrong.edu/liang/intro9e/test.html adresinde yapın.

MyProgrammingLab™

PROGRAMLAMA EGZERSİZLERİ

Bölümler 9.2-9.3

- *9.1** (*SSN Kontrolü*) Kullanıcıdan DDD-DD-DDDD formatında bir Sosyal Güvenlik numarası girmesini isteyen bir program yazın, burada D bir rakamdır. Program, Doğru Sosyal Güvenlik numarası için Geçerli SSN ve aksi takdirde Geçersiz SSN görüntüler.
- **9.2** (*Alt dizeleri kontrol et*) Bir dizinin başka bir dizinin alt dizesi olup olmadığını **indexOf** yöntemini kullanarak kontrol edebilirsiniz. Bu işlev için kendi yöntemini yazın. Kullanıcıdan iki dize girmesini isteyen bir program yazın ve ilk dizinin ikinci dizinin alt dizesi olup olmadığını kontrol edin.
- **9.3** (*Şifre kontrolü*) Bazı web siteleri şifreler için belirli kurallar koyar. Bir dizinin geçerli bir şifre olup olmadığını kontrol eden bir yöntem yazın. Şifre kurallarının şu şekilde olduğunu varsayın:
- Bir şifre en az sekiz karakter içermelidir.
 - Bir şifre yalnızca harfler ve rakamlardan oluşmalıdır.
 - Bir şifre en az iki rakam içermelidir.

Kullanıcıdan bir şifre girmesini isteyen ve Geçerli görüntüleyen bir program yazın. Şifre kuralları takip edilirse veya Geçersiz Şifre aksi takdirde.

- 9.4** (Belirtilen bir karakterin oluşumları) Aşağıdaki başlığı kullanarak bir dize içinde belirtilen bir karakterin oluşum sayısını bulan bir yöntem yazın:

```
public static inttcount(String str, tcharta)
```

Örneğin, `count("Welcome", 'e')` 2 döner. Kullanıcıdan bir dize ve ardından bir karakter girmesini isteyen ve karakterin dizideki oluşum sayısını gösteren bir test programı yazın.

- **9.5** (Bir dizideki her rakamın oluşumları) Aşağıdaki başlığı kullanarak bir dize içinde her rakamın oluşumlarını sayan bir yöntem yazın:

```
public static int[] count(String s)
```

Yöntem, bir rakamın dizide ne kadar sıklıkla görüldüğünü sayar. Dönüş değeri, bir rakam için sayıyı tutan on elemanlı bir dizidir. Örneğin, `int[] counts = count("12203AB3")` çalıştırdıktan sonra, `counts[0]` 1, `counts[1]` 1, `counts[2]` 2 ve `counts[3]` 2 olur.

Kullanıcıdan bir dize girmesini isteyen ve dizideki her rakamın oluşum sayısını gösteren bir test programı yazın.

- *9.6** (Bir dizideki harfleri say) Aşağıdaki başlığı kullanarak bir dizedeki harflerin sayısını sayacak bir yöntem yazın:

```
public static inttcountLetters(String s)
```

Bir test programı yazın, kullanıcıdan bir dize girmesini isteyin ve dizideki harf sayısını gösterin.

- *9.7** (Telefon tuş takımları) Telefonda bulunan uluslararası standart harf/numara eşlemesi:

1	2	3
	ABC	DEF
4	5	6
NOT ALIN	JKL	ÇOK FAZLA
7	8	9
PQRS	TUV	WXYZ
	0	

Büyük harfli bir harf verildiğinde bir sayı döndüren bir yöntem yazın:

```
public static inttgetNumber(tchartuppercaseLetter)
```

Kullanıcıdan bir telefon numarasını string olarak girmesini isteyen bir test programı yazın. Girdi numarası harfler içerebilir. Program, bir harfi (büyük veya küçük) bir rakam'a çevirir ve diğer tüm karakterleri olduğu gibi bırakır. İşte programın bir örnek çalışması:

Bir dize girin:
1-800-3569377

1-800-Çiçekler

→ Enter





Bir string girin: 1800flowers
18003569377

- *9.8** (*İkiliyi ondalığa*) Bir ikili sayıyı string olarak ondalık bir tamsayıya ayrıştırma bir yöntem yazın. Yöntem başlığı:

```
public static int binaryToDecimal(String ikiliString)
```

Örneğin, ikili string 10001, 17'dir. ($1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 = 17$). Bu nedenle, `binaryToDecimal("10001")` 17 döner. Unutmayın ki `Integer.parseInt("10001", 2)` bir ikili stringi ondalık bir değere ayrıştırır. Bu yöntemi bu alıştırma için kullanmayın.

Bir ikili dize girmesi için kullanıcıyı yönlendiren ve karşılık gelen ondalık tam sayı değeri gösteren bir test programı yazın.

Bölüm 9.4

- **9.9** (*İkiliyi onaltılıya*) İkili bir sayıyı onaltılı bir sayıya ayrıştıran bir yöntem yazın. Yöntem başlığı:

```
public static String ikiliToHex(String ikiliValue)
```

Kullanıcıdan bir ikili sayı girmesini isteyen ve karşılık gelen onaltılı değeri gösteren bir test programı yazın.

- **9.10** (*Onaltılıyı ikiliye*) Bir onaltılı sayıyı bir ikili

sayı olarak bir dizeye ayrıştıran bir yöntem yazın. Yöntem başlığı:

```
public static String decimalToBinary(int value)
```

Kullanıcıdan bir onaltılı tam sayı değeri girmesini isteyen ve karşılık gelen ikili değeri gösteren bir test programı yazın.

- **9.11** (*Bir dizideki karakterleri sırala*) Aşağıdaki başlığı kullanarak sıralı bir dize döndüren bir yöntem yazın:

```
public static String sort(String s)
```

Örneğin, `sort("acb")` `abc` döner.

Kullanıcıdan bir dize girmesini isteyen ve sıralanmış

dizeyi gösteren bir test programı yazın.

- **9.12** (*Anagramlar*) İki kelimenin anagram olup olmadığını kontrol eden bir yöntem yazın. İki kelime, herhangi bir sırada aynı harfleri içeriyorsa anagramdır. Örneğin, `silent` ve `listen` anagramlardır. Yöntemin başlığı:

```
public static boolean isAnagram(Dize s1, Dize s2)
```

Kullanıcıdan iki dize girmesini isteyen ve eğer bunlar anagramlarsa `iki dize anagramdır`, eğer anagram değillerse `iki dize anagram değildir` mesajını gösteren bir test programı yazın.

Bölüm 9.5

- *9.13** (*Bir dizeyi palindromları kontrol etmek için geçirin*) Dizeyi bir

kod satırı argümanı olarak geçirerek Liste 9.1'i yeniden yazın.

- *9.14** (*Tam sayıları topla*) İki program yazın. İlk program, belirtilmemiş sayıda tam sayıyı ayrı dizgeler olarak `main` yöntemine geçirir ve toplamalarını gösterir. Bu



VideoNotu

Numara dönüştürme

ikinci program, bir dize içinde bir boşlukla ayrılmış belirsiz sayıda tam sayıyı **main** yöntemine geçirir ve toplamalarını gösterir. İki programın adını verin **Egzersiz9_14a** ve **Egzersiz9_14b**, Şekil 9.15'te gösterildiği gibi.

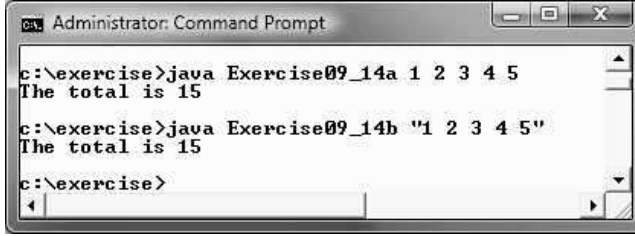


FIGURE 9.15 Program, komut satırından geçirilen tüm sayıları toplar.

***9.15** (Bir dizedeki büyük harf sayısını bul) Bir program yazın bir dizeyi **main** metoduna geçirip dizedeki büyük harf sayısını gösterir.

Kapsamlı

****9.16**(**String** sınıfını uygulayın) **String** sınıfı Java kütüphanesinde sağlanmaktadır. Kendi uygulamanızı aşağıdaki yöntemler için sağlayın (yeni sınıfın adını **MyString1**):

```
public MyString1(char[] chars);
public charAt(int index);
public int length();
public MyString1 substring(int begin, int end);
public MyString1 toLowerCase();
public boolean equals(MyString1 s);
public String valueOf(int i);
```

****9.17**(Başkentleri tahmin et) Kullanıcıdan bir eyaletin başkentini girmesini sürekli olarak isteyen bir program yazın. Kullanıcı girdisini aldıktan sonra, program cevabın doğru olup olmadığını bildirir. 50 eyalet ve başkentlerinin iki boyutlu bir dizide saklandığını varsayın, Şekil 9.16'da gösterildiği gibi. Program, kullanıcının tüm eyaletlerin başkentlerini yanıtlamasını ister ve toplam doğru sayısını gösterir. Kullanıcının cevabı büyük/küçük harf duyarlıdır.

Alabama	Montgomery
Alaska	Juneau
Arizona	Anka Kuşu
...	...
...	...

FIGURE 9.16 İki boyutlu bir dizi, eyaletleri ve başkentlerini saklar.

İşte bir örnek çalışma:

Alabama'nın başkenti nedir?	Montgomery	↵ Enter
Doğru cevap Montgomery olmalıdır		
Alaska'nın başkenti nedir?	Juneau	↵ Enter
Cevabınız doğru		
Arizona'nın başkenti nedir? ...		
...		
Doğru sayı 35'tir		



- **9.18** (*String sınıfını uygulayın*) *String* sınıfı Java kütüphanesinde sağlanmaktadır. Kendi uygulamanızı aşağıdaki yöntemler için sağlayın (yeni sınıfın adını *MyString2t*):

```
public MyString2(String s);
public int compareTo(MyString2 s);
public MyString2 substring(int begin, int end);
public MyString2 toUpperCase();
public MyString2 toLowerCase();
public char[] toChars();
public boolean contains(CharSequence s);
public static MyString2 valueOf(boolean b);
```

- *9.19** (*Ortak önek*) İki *string*'in en uzun ortak önekini döndüren bir yöntem yazın. Örneğin, *distance* ve *disinfection* için en uzun ortak önek *dis*'dir. Yöntemin başlığı şudur:

```
public static String önek(Dize s1, Dize s2)
```

İki dize ortak bir ön ek içermiyorsa, yöntem boş bir dize döner. Kullanıcıdan iki dize girmesini isteyen ve bunların

en uzun ortak ön ekini gösteren bir *main* yöntemi yazın.

- 9.20** (*Character sınıfını uygulayın*) *Character* sınıfı Java kütüphanesinde sağlanmıştır. Bu sınıf için kendi uygulamanızı sağlayın. Yeni sınıfı *MyCharacter* olarak adlandırın.

- **9.21** (*Yeni dize split yöntemi*) *String* sınıfındaki *split* yöntemi, ayrıcılarla bölünmüş alt dizelerden oluşan bir dizi dize döner. Ancak, ayrıcılar *split* yöntemiyle döndürülmez. Eşleşen ayrıcılarla bölünmüş alt dizelerden oluşan bir dizi dizeye eşleşen ayrıcıları içeren aşağıdaki yeni yöntemi uygulayın.

```
public static String[] split(String s, String regex)
```

Örneğin, *split("ab#12#453", "#")*, bir dizi *String* içinde *ab*, *#*, *12*, *#*, *453* döner ve *split("a?b?gf#e", "[?#]")*,

bir dizi *String* içinde *a*, *b*, *?*, *b*, *gf*, *#* ve *e* döner.

- **9.22** (*StringBuilder sınıfını uygulayın*) *StringBuilder* sınıfı Java kütüphanesinde sağlanmıştır. Aşağıdaki yöntemler için kendi uygulamanızı sağlayın (yeni sınıfı *MyStringBuilder1* olarak adlandırın):

```
public MyStringBuilder1(String s);
public MyStringBuilder1 append(MyStringBuilder1 s);
public MyStringBuilder1 append(CharSequence s);
public MyStringBuilder1 append(CharSequence s, int start, int end);
public MyStringBuilder1 append(char c);
public MyStringBuilder1 append(char[] s);
public MyStringBuilder1 append(char[] s, int start, int end);
public MyStringBuilder1 append(String s);
public MyStringBuilder1 append(String s, int start, int end);
public MyStringBuilder1 delete(int start, int end);
public MyStringBuilder1 deleteCharAt(int index);
public MyStringBuilder1 insert(int index, char c);
public MyStringBuilder1 insert(int index, CharSequence s);
public MyStringBuilder1 insert(int index, String s);
public MyStringBuilder1 insert(int index, String s, int start, int end);
public MyStringBuilder1 replace(int start, int end, CharSequence s);
public MyStringBuilder1 replace(int start, int end, String s);
public MyStringBuilder1 replace(int start, int end, String s, int start2, int end2);
public MyStringBuilder1 reverse();
public MyStringBuilder1 toString();
public MyStringBuilder1 toLowerCase();
public MyStringBuilder1 toUpperCase();
```

- **9.23** (*Finansal: kredi kartı numarası doğrulama*) Programlama Egzersizi 5.31'i yeniden yazın kredi kartı numarası için bir dize girişi kullanarak. Programı aşağıdaki yöntemleri kullanarak yeniden tasarlayın:

```
/** Kart numarası geçerliyse true döndür */
public static boolean isValid(String cardNumber)

/** Adım 2'den sonucu al */
public static int sumOfDoubleEvenPlace(String cardNumber)
```

```

/** Bu sayı tek haneliyse döndür; aksi takdirde,
 * iki hanenin toplamını döndür */
public static int getDigit(int number)

/** Sayıdaki tek haneli rakamların toplamını döndür */
public static int sumOfOddPlace(String cardNumber)

```

****9.24** (*StringBuilder sınıfını uygulayın*) **StringBuilder** sınıfı Java kütüphanesinde sağlanmıştır. Aşağıdaki yöntemler için kendi uygulamanızı sağlayın(yeni sınıfı **MyStringBuilder2** olarak adlandırın):

```

public MyStringBuilder2();
public MyStringBuilder2(char[] chars);
public MyStringBuilder2(String s);
public MyStringBuilder2 insert(int offset, MyStringBuilder2 s);
public MyStringBuilder2 reverse();
public MyStringBuilder2 substring(int begin);
public MyStringBuilder2 toUpperCase();

```

*****9.25** (*Oyun: adam asmaca*) Rastgele bir kelime üreten ve kullanıcıdan bir seferde bir harf tahmin etmesini isteyen bir adam asmaca oyunu yazın. Her harf, örnek çalışmada gösterildiği gibi, bir yıldız olarak görüntülenir. Kullanıcı doğru bir tahmin yaptığında, gerçek harf görüntülenir. Kullanıcı bir kelimeyi tamamladığında, kaç yanlış tahmin yaptığını gösterin ve kullanıcıya başka bir kelime ile oynamaya devam etmek isteyip istemediğini sorun. Aşağıdaki gibi kelimeleri saklamak için bir dizi tanımlayın:

```

// Bu diziye istediğiniz kelimeleri ekleyin
String[] words = {"write", "that", ...};

```

(Tahmin) ***** kelimesinde bir harf girin > p
 (Tahmin) p***** kelimesinde bir harf girin > r
 (Tahmin) pr*** kelimesinde bir harf girin > p
 p kelimedede zaten var
 (Tahmin) pr**r** kelimesinde bir harf girin > o
 (Tahmin) pro*r** kelimesinde bir harf girin > g
 (Tahmin) progr** kelimesinde bir harf girin > n
 n kelimedede yok
 (Tahmin) programda bir harf girin progr** > m
 (Tahmin) programda bir harf girin progr*m > a
 Kelime program. 1 kez kaçırdınız
 Başka bir kelime tahmin etmek ister misiniz? y veya n girin>

↵ Enter
 ↵ Enter
 ↵ Enter

↵ Enter
 ↵ Enter
 ↵ Enter

↵ Enter
 ↵ Enter



****9.26** (*ISBN-10'u Kontrol Et*) Programlama Egzersizi 3.9'u basitleştirmek için dize işlemlerini kullanın. Bir ISBN numarasının ilk 9 basamağını bir dize olarak girin.



VideoNotu

ISBN-10'u kontrol et

***9.27** (*Biyoinformatik: gen bulma*) Biyologlar, bir *genomu* modellemek için A, C, T ve G harflerinin bir dizisini kullanır. Bir *gen*, bir triplet ATG'den sonra başlayan ve bir triplet TAG, TAA veya TGA'dan önce biten bir genom alt dizisidir. Ayrıca, bir gen dizisinin uzunluğu 3'ün katı olmalıdır ve gen, ATG, TAG, TAA veya TGA tripletlerinden herhangi birini içermemelidir. Kullanıcıdan bir genom girmesini isteyen ve genomdaki tüm genleri gösteren bir program yazın. Girdi dizisinde hiçbir gen bulunamazsa, "hiçbir gen bulunamadı" mesajını gösterin. İşte örnek çalıştırmalar:

Bir genom dizisi girin: TTATGTTTTAAGGATGGGCGTTAGTT
 TTT
 GGGCGT

↵ Enter





Bir genom dizesi girin:
hiçbir gen bulunamadı

***9.28** (*Hesap Makinesi*) İşlemcilerin ve operatörlerin sıfır veya daha fazla boşlukla ayrıldığı bir ifadeyi kabul etmek için 9.5. Listeyi, Calculator.java dosyasını gözden geçirin. Örneğin, `3+4` `ve3 + 4` kabul edilebilir ifadelerdir.

***9.29** (*İş: ISBN-13 kontrol et*) **ISBN-13**, kitapları tanımlamak için yeni bir standarttır. 13 rakamı kullanır dt1tdt2tdt3tdt4tdt5tdt6tdt7tdt8tdt9tdt10tdt11tdt12tdt13t Son rakam, dt13t diğer rakamlardan aşağıdaki formülü kullanarak hesaplanan bir kontrol toplamıdır:

10 — $(tdt1t + t + 3tdt2t + t + dt3t + t + 3tdt4t + t + dt5t + t + 3tdt6t + t + dt7t + t + 3tdt8t + t + dt9t + t + 3tdt10t + t + dt11t + t + 3tdt12t) \% 10$

Kontrol toplamı **10** ise, bunu **0** ile değiştirin. Programınız girişi bir

string olarak okumalıdır. İşte örnek çalıştırmalar:



Bir ISBN-13'ün ilk 12 basamağını bir dize olarak girin:

ISBN-13 numarası 9780132130806



Bir ISBN-13'ün ilk 12 basamağını bir dize olarak girin:

ISBN-13 numarası 9780132130790

***9.30** (*Her Kelimenin İlk Harfini Büyük Yap*) Aşağıdaki yöntemi yazın, bu yöntem her kelimenin ilk harfini büyük yaptığı yeni bir dize döndürsün.

`public static voidtttitle(String s)`

Kullanıcıdan bir dize girmesini isteyen ve bu yöntemi çağıran bir test programı yazın, ve bu yöntemden dönen değeri gösterin. İşte bir örnek çalışma:



Bir dize girin:
Yeni dize: Londra İngiltere 2015

Kelimenin birden fazla boşlukla ayrılabilceğini unutmayın.

***9.31** (*Büyük/Küçük Harf Değiştirme*) Aşağıdaki yöntemi yazın, bu yöntem büyük harfleri küçük harfe ve küçük harfleri büyük harfe dönüştüren yeni bir dize döndürsün.

`public static String swapCase(String s)`

Kullanıcıdan bir dize girmesini isteyen ve bu yöntemi çağıran bir test programı yazın, ve bu yöntemden dönen değeri gösterin. İşte bir örnek çalışma:



Bir dize girin:
Yeni dize: buradayım

BÖLÜM

10

DÜŞÜNME NE SNELERİ

Hedefler

- Nesnelerin içeriğini korumak için değişmez sınıflardan değişmez nesneler oluşturmak (§10.2).
- Bir sınıf bağlamında değişkenlerin kapsamını belirlemek (§10.3).
- Çağırın nesneye atıfta bulunmak için **this** anahtar kelimesini kullanmak (§10.4).
- Yazılım geliştirmek için sınıf soyutlamasını uygulamak (§10.5).
- Prosedürel paradigma ile nesne yönelimli paradigma arasında ki farkları keşfetmek (§10.6).
- Bileşim ilişkilerini modellemek için sınıflar geliştirmek (§10.7).
- Nesne yönelimli paradigmayı kullanarak programlar tasarlamak (§§10.8–10.10).
- Sınıf tasarım yönergelerine uyan sınıflar tasarlamak (§10.11).
- Sarmalayıcı sınıflar kullanarak ilkel değerler için nesneler oluşturmak (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, ve **Boolean**) (§10.12).
- İlkel türler ile sarmalayıcı sınıf türleri arasında otomatik dönüşüm kullanarak programlamayı basitleştirmek (§10.13).
- Çok büyük sayıları hesaplamak için **BigInteger** ve **BigDecimal** sınıflarını kullanmak arbitrary precisions (§10.14).



10.1 Giriş



Bu bölümün odak noktası sınıf tasarımıdır ve prosedürel programlama ile nesne yönelimli programlama arasındaki farkları keşfeder.

Önceki iki bölüm nesneleri ve sınıfları tanıttı. Sınıfları nasıl tanımlayacağınızı, nesneleri nasıl oluşturacağınızı ve Java API'sindeki birkaç sınıftan nesneleri nasıl kullanacağınızı öğrendiniz (örneğin, [Date](#), [Random](#), [String](#), [StringBuilder](#), ve [Scanner](#)). Bu kitabın yaklaşımı, nesne yönelimli programlamadan önce problem çözmeyi ve temel programlama tekniklerini öğretmektir. Bu bölüm prosedürel ve nesne yönelimli programlamanın nasıl farklı olduğunu gösterir. Nesne yönelimli programlamanın faydalarını göreceksiniz ve bunu etkili bir şekilde kullanmayı öğreneceksiniz.

Nesne yönelimli yaklaşımın avantajlarını göstermek için birkaç örnek kullanacağız. Örnekler, yeni sınıflar tasarlamayı ve bunları uygulamalarda kullanmayı içermektedir. Öncelikle bu örnekleri destekleyen bazı dil özelliklerini tanıtırız.

10.2 Değiştirilemez Nesneler ve Sınıflar



Değiştirilemez nesneler oluşturmak için değiştirilemez sınıflar tanımlayabilirsiniz. Değiştirilemez nesnelerin içeriği değiştirilemez.



VideoNotu

Değiştirilemez nesneler ve this anahtar kelimesi

değiştirilemez nesne

değiştirilemez sınıf

Student sınıfı

Normalde, bir nesne oluşturur ve içeriğinin daha sonra değiştirilmesine izin verirsiniz. Ancak, bazen nesne oluşturulduktan sonra içeriğinin değiştirilemeyeceği bir nesne oluşturmak istenir. Bu tür bir nesneye *değiştirilemez nesne* ve sınıfına *değiştirilemez sınıf* denir. [String](#) sınıfı, örneğin, değiştirilemezdir. Eğer [set](#) yöntemini [CircleWithPrivateDataFields](#) sınıfında Listing 8.9'da silerseniz, sınıf değiştirilemez olur, bu çünkü yarıçap özel ve [set](#) yöntemi olmadan değiştirilemez.

Eğer bir sınıf değiştirilmezse, o zaman tüm veri alanları özel olmalı ve kamuya açık veri alanları için herhangi bir set metodu içeremez.

Tüm özel veri alanlarına ve mutatórlere sahip bir sınıf, mutlaka değiştirilemez değildir. Örneğin, aşağıdaki Öğrenci sınıfı tüm özel veri alanlarına sahip ve [set](#) metodları yoktur, ancak bu bir değiştirilemez sınıf değildir.

```

12 genel sınıf Öğrenci {
    özel int id;
    String name;
    Private java.util.Date dateCreated; private
    kamu Öğrenci(int ssn, String yeniİsim) {
        id = ssn;
        ad = yeniAd;
        tarihOluşturuldu = new java.util.Date();
    }
    kamu int getId() {
        geri dönüş;
    }
    kamu String getName() {
        return isim;
    }
    public java.util.Date getDateCreated() {
        return dateCreated;
    }
}

```

yirmiüç }

Aşağıdaki kodda gösterildiği gibi, veri alanı [dateCreated](#) [getDateCreated\(\)](#) yöntemi kullanılarak döndürülmektedir. Bu, bir [Date](#) nesnesine bir referanstır. Bu referans aracılığıyla, [dateCreated](#) içeriği değiştirilebilir.

```

genel sınıf Test {
    public static void main(String[] args) {
        Student student = new Student(111223333, "John");
        java.util.Date dateCreated = student.getDateCreated();
        dateCreated.setTime(200000); // Artık dateCreated alanı değiştirildi!
    }
}

```

Bir sınıfın değişmez olması için aşağıdaki gereksinimleri karşılaması gerekir:

- Tüm veri alanları özel olmalıdır.
- Veri alanları için herhangi bir değiştirici yöntemine sahip olamaz.
- Hiçbir erişimci yöntemi, değiştirilebilir bir veri alanına referans döndüremez.

İlgilenen okuyucular, değişmez nesnelerle ilgili geniş bir tartışma için Ek III.AB'ye başvurabilirler.

- 10.1** Bir sınıf yalnızca özel veri alanlarına ve hiç **set** yöntemlerine sahipse, sınıf değişmez midir?
- 10.2** Bir sınıftaki tüm veri alanları özel ve ilkel türlerse ve sınıf hiç **set** yöntemi ne sahip değilse, sınıf değişmez midir?
- 10.3** Aşağıdaki sınıf değişmez midir?

```

kamus sınıfı A {
    özel int[] değerler;

    kamus int[] getValues() {
        return değerler;
    }
}

```

Point Kontrol
MyProgrammingLab™

10.3 Değişkenlerin Kapsamı

Örnek ve statik değişkenlerin kapsamı, değişkenlerin nerede tanımlandığına bakılmaksızın tüm sınıfı kapsar.

Ana
Nokta

Bölüm 5, Yöntemler, yerel değişkenleri ve kapsam kurallarını tartıştı. Yerel değişkenler, bir yöntem içinde yerel olarak tanımlanır ve kullanılır. Bu bölüm, bir sınıf bağlamında tüm değişkenlerin kapsam kurallarını tartışmaktadır.

Sınıf içindeki örnek ve statik değişkenler, *sınıfın değişkenleri* veya *veri alanları* olarak adlandırılır. Bir yöntem içinde tanımlanan bir değişken, *yerel değişken* olarak adlandırılır. Bir sınıfın değişkenlerinin kapsamı, değişkenlerin nerede tanımlandığına bakılmaksızın tüm sınıfı kapsar. Bir sınıfın değişkenleri ve yöntemleri, Şekil 10.1a'da gösterildiği gibi sınıfta herhangi bir sırayla görünebilir. İstisna, bir veri alanının başka bir veri alanına referansa dayalı olarak başlatıldığı durumdur. Bu tür durumlarda,

sınıfın değişkenleri

```

public class Daire {
    public double alanBul() {
        return radyus * radyus * Math.PI;
    }

    private double radyus = 1;
}

```

(a) Değişken **radyus** ve yöntem **alanBul()** herhangi bir sırada tanımlanabilir.

```

genel sınıf tF { özel i
    ntti; özel intti = i +
    t1t;n}

```

(b) **i**, **j**'den önce tanımlanmalıdır çünkü **j**'nin başlangıç değeri **i**'ye bağlıdır.

FIGURE 10.1 Bir sınıfın üyeleri, bir istisna ile herhangi bir sırada tanımlanabilir.

diğer veri alanı önce tanımlanmalıdır, Şekil 10.1b'de gösterildiği gibi. Tutarlılık için, bu kitap veri alanlarını sınıfın başında tanımlar.

Bir sınıfın değişkenini yalnızca bir kez tanımlayabilirsiniz, ancak aynı değişken adını bir yöntem içinde birçok kez farklı iç içe olmayan bloklarda tanımlayabilirsiniz.

Eğer bir yerel değişken, bir sınıfın değişkeniyle aynı isme sahipse, yerel değişken öncelik alır ve aynı isme sahip sınıfın değişkeni *gizlenir*. Örneğin, aşağıdaki programda, **x** hem bir örnek değişkeni hem de bir yerel değişken olarak tanımlanmıştır.

```
genel classtF {nözel
    int x = t0t; // Örnek değişken
    özel intty = t0t;npublict

    F() {
    }

    genel voidtp() { nint
        x = t1t; // Yerel değişken
        System.out.println("x = "t+ x);nSystem.
        out.println("y = "t+ y);n}

    }
```

f.p() için çıktı nedir, burada f F sınıfının bir örneğidir? f.p() için çıktı 1 dir x ve 0 diry. İşte nedeni:

- **x** sınıfta başlangıç değeri 0 olan bir veri alanı olarak tanımlanmıştır, ancak aynı zamanda **p()** metodunda başlangıç değeri 1 olan bir değişken olarak da tanımlanmıştır. Sonuncu **x**, `System.out.println` ifadesinde referans alınmaktadır.
- **y**, **p()** metodunun dışında tanımlanmıştır, ancak **y** metodun içinde erişilebilir.



Tt tip

Karışıklığı ve hataları önlemek için, yerel değişken adları olarak örnek veya statik değişken adlarını kullanmayın, metod parametreleri hariç.

10.4 Aşağıdaki programın çıktısı nedir?

```
genel sınıfTest {
    özel static intti = t0t;nprivate static intttj = t0t;npublic static voidt

    main(String[] args) {nintti = t2t;

        inttk = t3t;n

        {
            inttj = t3t;
            System.out.println("i + j şudur "t+ i + j);
        }

        k = i + j;
        System.out.println("k şudur "t+ k);
        System.out.println("j şudur "t+ j);
    }
}
```

~

gizli değişkenler

Point Kontrol

MyProgrammingLab™

10.4 Bu Referans

Bu anahtar kelimesi **this**, nesnenin kendisine atıfta bulunur. Ayrıca, aynı sınıfın başka bir yapıcısını çağırmak için bir yapıcı içinde de kullanılabilir.



this anahtar kelimesi

Bu anahtar kelimesi, bir nesnenin kendisine atıfta bulunmak için kullanılabileceği bir referansın adıdır. Nesnenin örnek üyelerine atıfta bulunmak için **this** anahtar kelimesini kullanabilirsiniz. Örneğin, (a) bölümündeki aşağıdaki kod, nesnenin yarıçap'ına atıfta bulunmak için **this** kullanır ve açıkça **getArea()** yöntemini çağırır. **this** referansı genellikle atlanır, (b) bölümünde gösterildiği gibi. Ancak, gizlenmiş veri alanlarına atıfta bulunmak veya aşırı yüklenmiş bir yapıcıyı çağırmak için **this** referansı gereklidir.

```
public class Daire {
    private double yarıçap;

    ...

    public double getArea() { return
        Bu.yarıçap * Bu.yarıçap
        * Math.PI;
    }

    public String toString() {
        return "yarıçap: " + yarıçap
            + "alan: " + .getArea();
    }
}
```

(a)

Eşdeğer

```
public class Daire {
    private double yarıçap;

    ...

    public double getArea() {
        return yarıçap * yarıçap * Math.PI;
    }

    public String toString() {
        return "yarıçap: " + yarıçap
            + "alan: " + getArea();
    }
}
```

(b)

10.4.1 Gizli Veri Alanlarını Referans Vermek için **this** Kullanma

this anahtar kelimesi, bir sınıfın gizli veri alanlarını referans vermek için kullanılabilir. Örneğin, bir veri-alan adı genellikle veri alanı için bir set yönteminde parametre adı olarak kullanılır. Bu durumda, veri alanı set yönteminde gizlidir. Yeni bir değer atamak için yöntemde gizli veri alanı adını referans vermeniz gerekir. Gizli bir statik değişken, basitçe `ClassName.staticVariable` referansını kullanarak erişilebilir. Gizli bir örnek değişken ise, this anahtar kelimesi kullanılarak erişilebilir; bu, Şekil 10.2a'da gösterilmiştir.

gizlenmiş veri alanları

```
genel sınıf tF { özel i
    ntti = t5; özel Statik do
        ubletk = t0; public void setI(tintti) {

        n;
        this.tit = ti;
    }

    genel statik void setK(tdouletk) { n}
        F.k = k;

    // Diğer yöntemler atlandı
}
```

(a)

f1 ve f2'nin F sınıfının iki nesnesi olduğunu varsayalım.

f1.setI(10) çağırısı,
 this.i = 10 ifadesini çalıştırmak anlamına gelir; burada this f1'i referans alır.

f2.setI(45) çağırısı,
 this.i = 45 ifadesini çalıştırmak anlamına gelir; burada this f2'yi referans alır.

F.setK(33) çağırısı,
 F.k = 33 ifadesini çalıştırmak anlamına gelir. setK bir statik yöntemdir.

(b)

ŞEKİL 10.2 Anahtar kelime **this**, yöntemi çağıran nesneyi referans alır.

Bu anahtar kelimesi, bir örnek yöntemini çağıran nesneye atıfta bulunmamızı sağlar. Bir **f1.setI(10)** çağırması için, **this.i = i** ifadesi çalıştırılır, bu da parametre **i** değerini **f1** çağıran nesne **f1** içindeki veri alanı **i**'ye atar. **this** anahtar kelimesi, çağıran nesneyi ifade eder.

örnek yöntemi **setI**, Şekil 10.2b'de gösterildiği gibi. **F.k = k** satırı, parametre **k** içindeki değerin, sınıfın tüm nesneleri tarafından paylaşılan statik veri alanı **k**'ya atandığı anlamına gelir.

10.4.2 Bir Yapıcıyı Çağırarak İçin **this** Kullanma

this anahtar kelimesi, aynı sınıfın başka bir yapıcısını çağırarak kullanılabılır. Örneğin, Circle sınıfını aşağıdaki gibi yeniden yazabilirsiniz:

```
public class Circle {
    private double radius;
    public Circle() {
        this.radius = radius;
    }
    public Circle(double r) {
        this(1.0);
    }
    ...
}
```

this anahtar kelimesi, inşa edilen nesnenin gizli veri alanı **radius**'a referans vermek için kullanılır.

this anahtar kelimesi, başka bir yapıcıyı çağırarak kullanılır.

İkinci yapıcıdaki **this(1.0)** satırı, bir **double**

değer argümanı ile ilk yapıcıyı çağırır.



Not

Java, **this(arg-list)** ifadesinin yapıcıda herhangi bir başka çalıştırılabilir ifadeden önce ilk olarak görünmesini gerektirir.



Tipp

Bir sınıfın birden fazla yapıcısı varsa, bunları mümkün olduğunca **this(arg-list)** kullanarak uygulamak daha iyidir. Genel olarak, daha az veya hiç argümanı olmayan bir yapıcı, daha fazla argümanla bir yapıcıyı **this(arg-list)** kullanarak çağırabilir. Bu söz dizimini genellikle kodlamayı basitleştirir ve sınıfın okunmasını ve bakımını kolaylaştırır.

10.5 **this** anahtar kelimesinin rolünü tanımlayın.

10.6 Aşağıdaki kodda ne yanlış?

```
1 genel sınıfC {
2     özel inttp;
3
4     public C() {
5         System.out.println("C'nin argümanı olmayan yapıcısı çağırıldı");
6         this.t0t;
7     }
8
9     public C(inttp) {
10        p = p; n11 }
11
12
13        genel voidtsetP(inttp) {
14 p = p; n15}
15
16 }
```

Point Kontrol Et

MyProgrammingLab™

10.7 Aşağıdaki kodda ne yanlış?

```
public class Test {
    private int id;
    public void tm1() {
        this.id = 45;
    }

    public void tm2() {
        this.id = 45;
    }
}
```

10.5 Sınıf Soyutlama ve Kapsülleme

Sınıf soyutlaması, sınıf uygulamasının sınıf kullanımından ayrılmasıdır. Uygulama detayları kullanıcıdan kapsüllenmiş ve gizlenmiştir. Bu, sınıf kapsülleme olarak bilinir.



5. Bölümde, yöntem soyutlaması hakkında bilgi edindiniz ve bunu adım adım iyileştirebileceğinizi kullandınız. Java, birçok soyutlama seviyesi sağlar ve sınıf soyutlaması, sınıf uygulamasını sınıfın nasıl kullanıldığına ayırır. Bir sınıfın yaratıcısı, sınıfın işlevlerini tanımlar ve kullanıcıya sınıfın nasıl kullanılabileceğini bildirir. Sınıfın dışından erişilebilen yöntemler ve alanlar koleksiyonu, bu üyelerin nasıl davranması gerektiği açıklamasıyla birlikte, sınıfın sözleşmesi olarak hizmet eder. Şekil 10.3'te gösterildiği gibi, sınıfın kullanıcısının sınıfın nasıl uygulandığını bilmesine gerek yoktur. Uygulama detayları kapsüllenmiş ve kullanıcıdan gizlenmiştir. Bu, sınıf kapsülleme olarak adlandırılır. Örneğin, bir Daire nesnesi oluşturabilir ve alanını hesaplamadan dairenin alanını bulabilirsiniz.

Bu nedenle, bir sınıf aynı zamanda bir soyut veri tipi (ADT) olarak da bilinir.

sınıf soyutlaması

sınıfın sözleşmesi

sınıf kapsülleme

soyut veri tipi

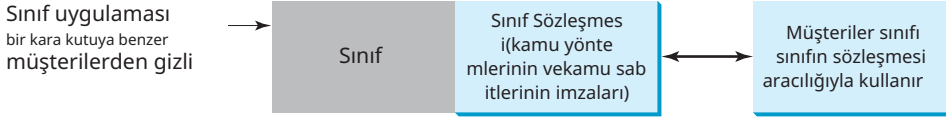


FIGURE 10.3 Sınıf soyutlaması, sınıf uygulamasını sınıfın kullanımından ayırır.

Sınıf soyutlaması ve kapsülleme, aynı paranın iki yüzüdür. Gerçek hayattan birçok örnek, sınıf soyutlama kavramını illüstre eder. Örneğin, bir bilgisayar sistemi inşa etmeyi düşünün. Kişisel bilgisayarınızın birçok bileşeni vardır - bir CPU, bellek, disk, anakart, fan vb. Her bir bileşen, özellikleri ve yöntemleri olan bir nesne olarak görülebilir. Bileşenlerin birlikte çalışmasını sağlamak için, yalnızca her bileşenin nasıl kullanıldığını ve diğerleriyle nasıl etkileşimde bulunduğunu bilmeniz gerekir. Bileşenlerin içsel olarak nasıl çalıştığını bilmenize gerek yoktur. İçsel uygulama kapsüllenmiş ve sizden gizlenmiştir. Bir bileşenin nasıl uygulandığını bilmeden bir bilgisayar inşa edebilirsiniz.

Bilgisayar sistemi analojisi, nesne yönelimli yaklaşımı tam olarak yansıtır. Her bir bileşen, bileşenin sınıfının bir nesnesi olarak görülebilir. Örneğin, bilgisayarda kullanılmak üzere her türlü fanı modelleyen bir sınıfınız olabilir; fan boyutu ve hızı gibi özellikler ve başlat ve durdur gibi yöntemler içerebilir. Belirli bir fan, bu sınıfın belirli özellik değerlerine sahip bir örneğidir.

Başka bir örnek olarak, bir kredi almayı düşünün. Belirli bir kredi, bir Kredi sınıfının bir nesnesi olarak görülebilir. Faiz oranı, kredi miktarı ve kredi süresi, onun veri özellikleridir ve



aylık ödemeyi ve toplam ödemeyi hesaplamak, onun yöntemleridir. Bir araba satın aldığınızda, bir kredi nesnesi, kredi faiz oranınız, kredi miktarınız ve kredi süreniz ile sınıfı örnekleyerek oluşturulur. Daha sonra, kredi

ödemelerinizin aylık ödemesini ve toplam ödemesini bulmak için yöntemleri kullanabilirsiniz. [Loan](#) sınıfının bir kullanıcısı olarak, bu yöntemlerin nasıl uygulandığını bilmeniz gerekir.

Liste 2.8, `ComputeLoan.java`, kredi ödemelerini hesaplamak için bir program sundu. O program, ödemeleri hesaplamak için kod `main` yönteminde olduğu için diğer programlarda yeniden kullanılamaz. Bu sorunu çözmenin bir yolu, aylık ödeme ve toplam ödeme hesaplamak için statik yöntemler tanımlamaktır. Ancak, bu çözümün sınırlamaları vardır. Diyelim ki, kredinizle bir tarihi ilişkilendirmek istiyorsunuz. Bir tarihi kredi ile ilişkilendirmenin iyi bir yolu yoktur, nesneler kullanmadan. Geleneksel prose dürel programlama paradigması eylem odaklıdır ve veriler eylemlerden ayrılır. Nesne yönelimli programlama paradigması nesnelere odaklanır ve eylemler, nesnelerdeki verilerle birlikte tanımlanır. Bir tarihi kredi ile ilişkilendirmek için, bir tarih ile birlikte kredinin diğer özelliklerini veri alanları olarak içeren bir kredi sınıfı tanımlayabilirsiniz. Artık bir kredi nesnesi, verileri ve verileri işlemek ve manipüle etmek için eylemleri içerir ve kredi verileri ile eylemleri tek bir nesnede entegre edilmiştir. Şekil 10.4, `Loan` sınıfı için UML sınıf diyagramını göstermektedir.

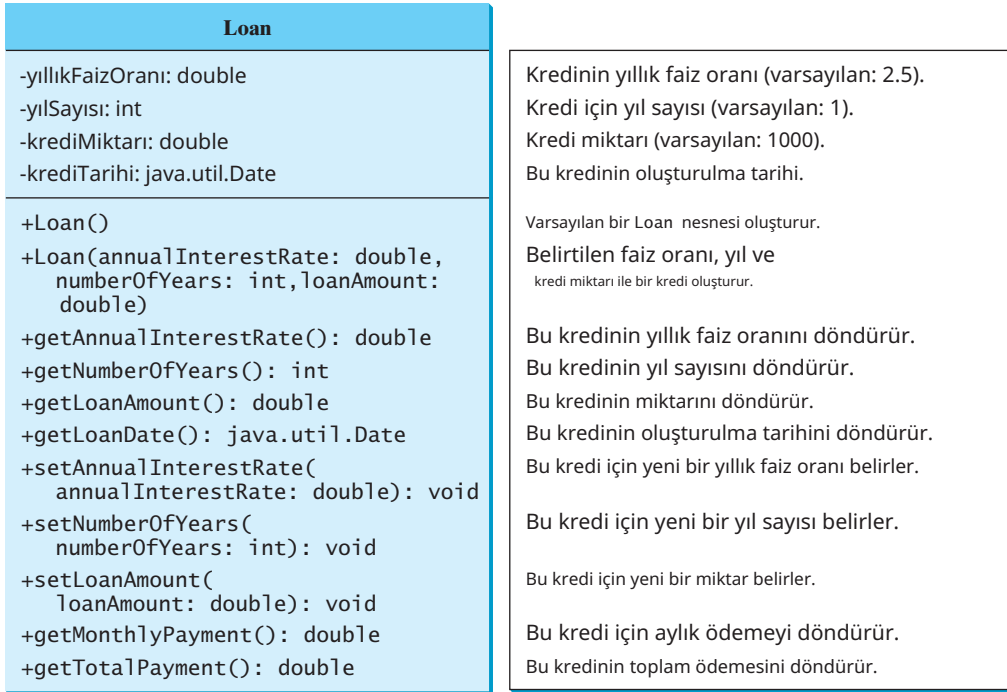


FIGURE 10.4 Kredi sınıfı, kredilerin özelliklerini ve davranışlarını modellemektedir.

Şekil 10.4'teki UML diyagramı, Kredi sınıfı için sözleşme olarak hizmet eder. Bu kitap boyunca, hem sınıf kullanıcısı hem de sınıf geliştiricisi rolleri üstleneceksiniz. Bir sınıf kullanıcısının, sınıfın nasıl uygulandığını bilmeden sınıfı kullanabileceğini unutmayın.

Kredi sınıfının mevcut olduğunu varsayın. Liste 10.1'deki program bu sınıfı kullanmaktadır.

LİSTE 10.1 TestLoanClass.java

```

1  import java.util.Scanner;
2
3  genel siniftTestLoanClass {
4      /** Ana yöntem */
5      public static void main(String[] args) {
```



```

6 // Bir Scanner oluştur
7 Tarayıcı girişi = tnewScanner(System.in);
89
9 // Yıllık faiz oranını girin
10 System.out.print(
11 "Yıllık faiz oranını girin, örneğin, 8.25: ");
12 doubletannualInterestRate = input.nextDouble();
13
14 // Yıl sayısını girin
15 System.out.print("Bir tam sayı olarak yıl sayısını girin: ");
16 inttnumberOfYears = input.nextInt();
17
18 // Kredi miktarını girin
19 System.out.print("Kredi miktarını girin, örneğin, 120000.95: ");
20 doubletloanTutar = input.nextDouble();
21
22 // Bir Loan nesnesi oluştur
23 Loan loan =
24 yeniKredi(yıllıkFaizOranı, yılSayısı, krediMiktarı);
25
26 // Kredi tarihini, aylık ödemeyi ve toplam ödemeyi göster
27 System.out.print("Kredi %s tarihinde oluşturuldu"+
28 "Aylık ödeme %.2fToplam ödeme %.2f",
29 kredi.getLoanDate().toString(), kredi.getMonthlyPayment(),n30 kredi.getTotalPay
ment());
31 {naak
}

```

oluştur Kredi nesnesi

örnek yöntemini çağır
örnek yöntemini çağır

```

Yıllık faiz oranını girin, örneğin, 8.25:      2.5  → Enter
Yıl sayısını tam sayı olarak girin:           5    → Enter
Kredi miktarını girin, örneğin, 120000.95:    1000 → Enter
Kredi, 16 Haziran 2012 Cumartesi 21:12:50 EDT tarihinde oluşturuldu.
Aylık ödeme 17.75'tir.
Toplam ödeme 1064.84'tür.

```



Ana yöntem, faiz oranını, ödeme süresini (yıllar cinsinden) ve kredi miktarını okur; bir Loan nesnesi oluşturur; ardından Loan sınıfındaki örnek yöntemleri kullanarak aylık ödemeyi (satır 29) ve toplam ödemeyi (satır 30) alır.

Loan sınıfı, Liste 10.2'deki gibi uygulanabilir.

LİSTE 10.2 Loan.java

```

1 halk classKredi {
2     private double yıllıkFaizOranı;
3     private int yılSayısı;
4     private double krediMiktarı;
5     özeljava.util.Date krediTarihi;
6
7     /** Varsayılan yapıcı */
8     publicKredi() {
9         thist(t2t.t5t,t1t,t1000t);n
10 }
11
12 /** Belirtilen yıllık faiz oranı ile bir kredi oluşturun,
13     yıl sayısı ve kredi miktarı
14     */

```

argüman almayan yapıcı

yapıcı

```

15 public Loan(double annualFaizOranı, int numberOfYears,
16             double krediMiktarı) {
17     this.yıllıkFaizOranı = yıllıkFaizOranı;
18     this.yılSayısı = yılSayısı;
19     this.krediMiktarı = krediMiktarı;
20     krediDate = new java.util.Date();
21 }

22
23 /** Yıllık Faiz Oranını Döndür */
24 public double getAnnualInterestRate() {
25     return yıllıkFaizOranı;
26 }
27
28 /** Yeni bir yıllık faiz oranı ayarla */
29 public void setAnnualInterestRate(double annualInterestRate) {
30     this.yıllıkFaizOranı = yıllıkFaizOranı;
31 }
32
33 /** Yıl Sayısını Döndür */
34 public int getNumberOfYears() {
35     return numberOfYears;
36 }
37
38 /** Yeni numberOfYears değerini ayarla */
39 public void setNumberOfYears(int numberOfYears) {
40     this.yılSayısı = yılSayısı;
41 }
42
43 /** loanAmount değerini döndür */
44 public double getLoanAmount() {
45     return krediMiktarı;
46 }
47
48 /** Yeni loanAmount değerini ayarla */
49 public void setLoanAmount(double loanAmount) {
50     this.krediMiktarı = krediMiktarı;
51 }
52
53 /** Aylık ödemeyi bul */
54 public double getMonthlyPayment() {
55     double aylıkFaizOranı = yıllıkFaizOranı / 1200;
56     double aylıkÖdeme = krediMiktarı * aylıkFaizOranı / (1 -
57         (1 / Math.pow(1 + aylıkFaizOranı, yılSayısı * 12)));
58     return aylıkÖdeme;
59 }
60
61 /** Toplam ödemeyi bul */
62 public double getTotalPayment() {
63     double toplamÖdeme = getMonthlyPayment() * numberOfYears * 12;
64     return toplamÖdeme;
65 }
66
67 /** Kredi tarihini döndür */
68 public java.util.Date getLoanDate() {
69     return krediDate;
70 }
71 }

```

Sınıf geliştiricisinin perspektifinden, bir sınıf birçok farklı müşteri tarafından kullanılmak üzere tasarlanmıştır. Geniş bir uygulama yelpazesinde faydalı olabilmesi için, bir sınıf yapıcılar, özellikler ve yöntemler aracılığıyla özelleştirme için çeşitli yollar sunmalıdır.

Loan sınıfı iki yapıcı, dört get yöntemi, üç set yöntemi ve aylık ödeme ile toplam ödeme bulma yöntemlerini içerir. Bir Loan nesnesini, argüman almayan yapıcıyı veya yıllık faiz oranı, yıl sayısı ve kredi miktarı olmak üzere üç parametrelili yapıcıyı kullanarak oluşturabilirsiniz. Bir kredi nesnesi oluşturulduğunda, tarihi loanDate alanında saklanır. getLoanDate yöntemi tarihi döndürür. Üç get yöntemi—getAnnualInterest, getNumberOfYears ve getLoanAmount—sırasıyla yıllık faiz oranını, ödeme yıllarını ve kredi miktarını döndürür. Bu sınıftaki tüm veri özellikleri ve yöntemler, Loan sınıfının belirli bir örneğine bağlıdır. Bu nedenle, bunlar örnek değişkenleri ve yöntemlerdir.



İmportant Pedagogical Tip

Şekil 10.4'te gösterilen **Loan** sınıfı için UML diyagramını kullanarak, **Loan** sınıfının nasıl uygulandığını bilmeseniz bile bir test programı yazın. Bu üç faydası vardır:

- Bir sınıf geliştirmek ve bir sınıf kullanmak iki ayrı görev olduğunu göstermektedir.
- Bu kitabın sırasını kesintiye uğratmadan belirli sınıfların karmaşık uygulamalarını atlamanızı sağlar.
- Bir sınıfı uygulamayı öğrenmek, o sınıfı kullanarak ona aşina iseniz daha kolaydır.

Sınıf örneklerinin tamamı için, sınıftan bir nesne oluşturun ve uygulamasına dikkat etmeden önce yöntemlerini kullanmayı deneyin.

10.8 Eğer 10.2 numaralı Listede **set** yöntemleri olmadan **Loan** sınıfını yeniden tanımlarsanız, sınıf değişmez mi?

10.6 Nesne Yönelimli Düşünme

Prosedürel paradigma, yöntemlerin tasarımına odaklanır. Nesne yönelimli paradigma, verileri ve yöntemleri nesneler halinde bir araya getirir. Nesne yönelimli paradigma kullanarak yazılım tasarımı, nesnelere ve nesneler üzerindeki işlemlere odaklanır.

Bölüm 1–7, döngüler, yöntemler ve diziler kullanarak problem çözme için temel programlama tekniklerini tanıttı. Bu teknikleri bilmek, nesne yönelimli programlama için sağlam bir temel oluşturur. Sınıflar, yeniden kullanılabilir yazılımlar oluşturmak için daha fazla esneklik ve modülerlik sağlar. Bu bölüm, Bölüm 3'te tanıtılan bir problemin nesne yönelimli yaklaşım kullanılarak çözümünü gösterir. Bu iyileştirmelerden, prosedürel ve nesne yönelimli programlama arasındaki farklar hakkında bilgi edinecek ve nesneler ve sınıflar kullanarak yeniden kullanılabilir kod geliştirmenin faydalarını göreceksiniz.

Liste 3.5, ComputeAndInterpretBMI.java, vücut kütle indeksini hesaplamak için bir program sundu. Kod, **ana**

metodda olduğu için diğer programlarda yeniden kullanılamaz. Yeniden kullanılabilir hale getirmek için, vücut kütle indeksini hesaplamak için aşağıdaki gibi bir statik yöntem tanımlayın:

```
public static doublegetBMI(doublekilo,doubleheight)
```

Bu yöntem, belirli bir ağırlık ve boy için vücut kütle indeksini hesaplamak için faydalıdır. Ancak, sınırlamaları vardır. Ağırlık ve boyu bir kişinin adı ve doğum tarihi ile ilişkilendirmeniz gerektiğini varsayalım. Bu değerleri saklamak için ayrı değişkenler tanımlayabilirsiniz, ancak bu değerler sıkı bir şekilde bağlı olmayacaktır. Onları birleştirmenin ideal yolu, hepsini içeren bir nesne oluşturmaktır. Bu değerler bireysel nesnelere bağlı olduğundan, örnek veri alanlarında saklanmalıdır. Şekil 10.5'te gösterildiği gibi **BMI** adında bir sınıf tanımlayabilirsiniz.

Point Kontrol Et

MyProgrammingLab™

Anahtar Nokta



VideoNotu
BMI sınıfı

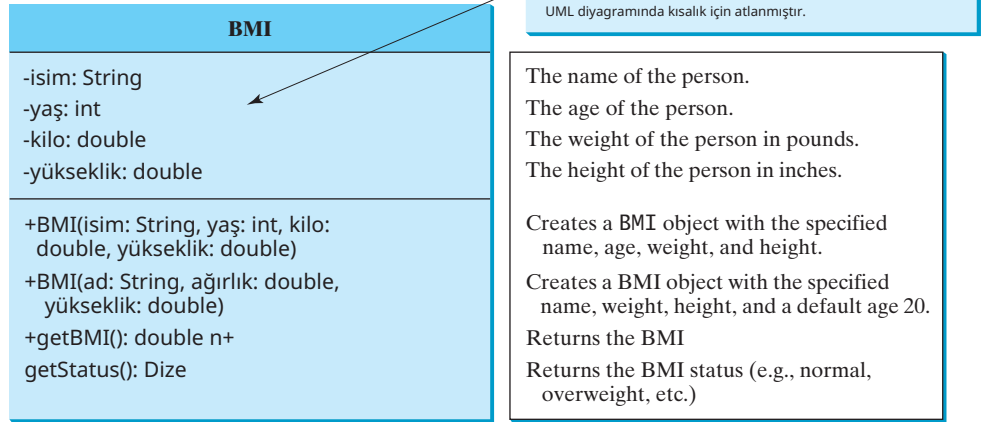


FIGURE 10.5 BMI sınıfı BMI bilgilerini kapsüller.

BMI sınıfının mevcut olduğunu varsayalım. Liste 10.3, bu sınıfı kullanan bir test programını vermektedir.

LISTE 10.3 UseBMIClass.java

bir nesne oluştur
örnek yöntemini çağır

bir nesne oluştur
örnek yöntemini çağır

```

1 public class KullanımBMIClass {
2     public static void main(String[] args) {
3         BMI bmi1 = newtBMI("Kim Yang",t18t,t145t,t70t);
4         System.out.println("Vücut Kitle İndeksi için " + bmi1.getName()n +
5             + bmi1.getbbhi() + " " + mi1.getStatus());
6
7         BMI bmi2 = newtBMI("Susan King",t,t215t,t70t);nb
8         System.out.println("BMI için " + bmi2.getName()n +
9             + bmi2.getBMI() + " " + bmi2.getStatus());
10    }n11
11 }

```



Kim Yang'ın BMI'si 20.81 Normal
 Susan King'in BMI'si 30.85 Obez

3. satır, Kim Yang için bmi1 nesnesini oluşturur ve 7. satır, Susan King için bmi2 nesnesini oluşturur. getName(), getBMI() ve getStatus() örnek yöntemlerini kullanarak BMI bilgilerini BMI nesnesinde döndürebilirsiniz.

BMI sınıfı, Liste 10.4'teki gibi uygulanabilir.

LISTE 10.4 BMI.java

```

1 genel sınıfBMI {
2     PrivateString adı;
3     özel intyaş;
4     özel doublekilo;// pound cinsinden
5     özel doubleyükseklik;// inç cinsinden
6     genel statik final doubletKILOGRAMS_PER_POUND = t0t.t45359237t;

```

```

7 public static final double METRE_PER_INÇ = 0.0254;
89
10 public BMI(Dize adı, renk tonu, double weight, double height) {          yapıcı
11     this.name = adı;
12     this.yaş = yaş;
13     this.ağırlık = ağırlık;
14     this.yükseklik = yükseklik;
15 }
16
17 public BMI(String isim, double ağırlık, double yükseklik) {          yapıcı
18     this(isim, t20t, ağırlık, boy);
19 }
20
21 genel double getBMI() {          BMI'ı al
22     double bmi = ağırlık * KILOGRAMS_PER_POUND /
23     ((yükseklik * METERS_PER_INCH) * (yükseklik * METERS_PER_INCH));
24     return Math.round(bmi * t100t) / t100t.t0t;
25 }
26
27 public String getStatus() {          getStatus
28     double bmi = getBMI();
29     if (bmi < t18,5t)
30         return "Aşırı Zayıf";
31     else if (bmi < t25t)
32         return "Normal'e dön;
33     aksi takdirde eğer (bmi < 30)
34         return "Aşırı Kilolu";
35     aksi takdirde
36         return "Obez";
37 }
38
39 public String getName() {
40     return isim;
41 }
42
43 genel int getAge() {
44     return yaş;
45 }
46
47 genel double getWeight() {
48     geri dönüş ağırlığı;
49 }
50
51 genel double getHeight() {
52     dönüş yüksekliği;
53 }
54 {nah
55 a}

```

BMI'yi ağırlık ve boy kullanarak hesaplamak için matematiksel formül Bölüm 3.9'da verilmiştir. Örnek yöntemi **getBMI()** BMI'yi döndürür. Ağırlık ve boy, nesnedeki örnekdata alanları olduğundan, **getBMI()** yöntemi bu özellikleri kullanarak nesne için BMI'yi hesaplayabilir.

Örnek yöntemi **getStatus()** BMI'yi yorumlayan bir dize döndürür. Yorumlama da Bölüm 3.9'da verilmiştir.

Bu örnek, nesne yönelimli paradigmanın prosedürel paradigmaya göre avantajlarını göstermektedir. Prosedürel paradigma, yöntemlerin tasarımına odaklanır. Nesne yönelimli paradigma, verileri ve yöntemleri nesneler içinde birleştirir. Nesne yönelimli paradigma kullanarak yazılım tasarımı, nesnelere ve nesneler üzerindeki işlemlere odaklanır. Nesne yönelimli yaklaşım

prosedürel vs. nesne yönelimli
paradigmalar

prosedürel paradigmanın gücünü, verileri

işlemlerle birleştiren ek bir boyutla birleştirir.

Prosedürel programlamada, veriler ve veriler üzerindeki işlemler ayrıdır ve bimetodoloji verileri yöntemlere göndermeyi gerektirir. Nesne yönelimli programlama, veriler ve onlarla ilgili işlemleri bir nesnede toplar. Bu yaklaşım, prosedürel programlamada var olan birçok sorunu çözer. Nesne yönelimli programlama yaklaşımı, programları gerçek dünyayı yansıtan bir şekilde organize eder; burada tüm nesneler hem nitelikler hem de aktivitelerle ilişkilidir. Nesnelerin kullanmak, yazılımın yeniden kullanılabilirliğini artırır ve programları geliştirmeyi ve bakımını kolaylaştırır. Java'da programlama, nesneler açısından düşünmeyi içerir; bir Java programı, birbirine işbirliği yapan nesnelerin bir koleksiyonu olarak görülebilir.

10.9 BMI sınıfı, Liste 10.4'te tanımlı mı değişmez?

MyProgrammingLab™

Point Kontrol Et



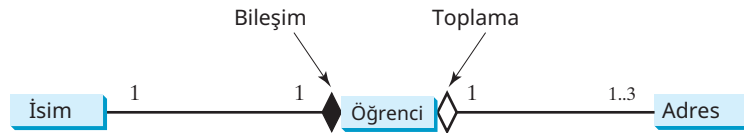
10.7 Nesne Bileşimi

Bir nesne başka bir nesneyi içerebilir. İki arasındaki ilişkiye bileşim denir.

Liste 10.2'de, **Loan** sınıfını bir **Date** veri alanı içerecek şekilde tanımladınız. **Loan** ve **Date** arasındaki ilişki kompozisyonudur. Liste 10.4'te, **BMI** sınıfını bir **String** veri alanı içerecek şekilde tanımladınız. **BMI** ve **String** arasındaki ilişki kompozisyonudur.

Kompozisyon aslında agregasyon ilişkisinin özel bir durumudur. Agregasyon modelleri *sahip-ilişkileri* ve iki nesne arasındaki sahiplik ilişkisini temsil eder. Sahip nesne *agregat nesne* olarak adlandırılır ve sınıfı *agregat sınıf* olarak adlandırılır. Konu nesne *agregat nesne* olarak adlandırılır ve sınıfı *agregat sınıf* olarak adlandırılır.

Bir nesne, birkaç diğer agregat nesne tarafından sahip olunabilir. Eğer bir nesne yalnızca bir agregat nesne tarafından sahipleniliyorsa, aralarındaki ilişkiye kompozisyon denir. Örneğin, "bir öğrencinin bir ismi vardır" **Student** sınıfı ile **Name** sınıfı arasında bir kompozisyon ilişkisidir, oysa "bir öğrencinin bir adresi vardır" **Student** sınıfı ile **Address** sınıfı arasında bir agregasyon ilişkisidir, çünkü bir adres birçok öğrenci tarafından paylaşılabilir. UML notasyonunda, dolu bir elmas bir agregat sınıfa (örneğin, **Student**) eklenir ve bu, bir agregat sınıfa (örneğin, **Name**) olan kompozisyon ilişkisini belirtir, ve boş bir elmas bir agregat sınıfa (örneğin, **Student**) eklenir ve bu, bir agregat sınıfa (örneğin, **Address**) olan agregasyon ilişkisini belirtir, bu şekil 10.6'da gösterildiği gibi.



FIGÜR 10.6 Bir öğrencinin bir ismi ve bir adresi vardır.

çokluk

İlişkiye dahil olan her sınıf bir *çokluk* belirtebilir. Bir çokluk, sınıfın nesnelerinin ilişkiye dahil olduğu sayıyı belirten bir sayı veya aralık olabilir. * karakteri sınırsız sayıda nesne anlamına gelir ve m..n aralığı nesne sayısının m ile n arasında, dahil olmak üzere olması gerektiğini belirtir. Şekil 10.6'da, her öğrenci sadece bir adrese sahiptir ve her adres en fazla 3 öğrenci tarafından paylaşılabilir. Her öğrencinin bir ismi vardır ve bir isim her öğrenci için benzersizdir.

Bir toplama ilişkisi genellikle toplama sınıfında bir veri alanı olarak temsil edilir. Örneğin, Şekil 10.6'daki ilişki aşağıdaki gibi temsil edilebilir:

```
genel sınıfAdı {n...
```

Toplanmış sınıf

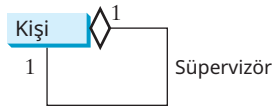
```
public classtStudent {npriva
tetName adı;nprivatet
Address adresi;n...
```

Toplayıcı sınıf

```
genel sınıf adresi {n...
```

Toplanmış sınıf

Toplama, aynı sınıfın nesneleri arasında var olabilir. Örneğin, bir kişinin bir üstü olabilir. Bu, Şekil 10.7'de gösterilmektedir.

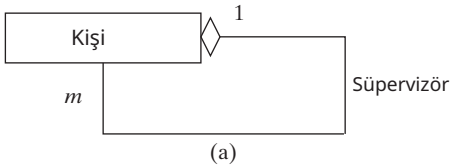


FIGON 10.7 Bir kişinin bir süpervizörü olabilir.

“Bir kişinin bir üstü vardır” ilişkisi, Şekil 10.7'de gösterildiği gibi, bir üst, **Person** sınıfında bir veri alanı olarak temsil edilebilir, aşağıdaki gibi:

```
// Verinin tipi sınıfın kendisidir
özel Kişi amiri;kamu sınıfı Kişi {
...
}
```

Eğer bir kişinin birden fazla süpervizörü olabiliyorsa, Şekil 10.8a'da gösterildiği gibi, süpervizörleri saklamak için bir dizi kullanabilirsiniz, Şekil 10.8b'de gösterildiği gibi.



(a)

```
genel sınıf Kişi { içinde ...
PrivatetPerson[] denetçileri;n}
```

(b)

FIGURE 10.8 Bir kişi birden fazla süpervizöre sahip olabilir.



Not

Toplama ve bileşim ilişkileri, aynı şekilde sınıflar kullanılarak temsil edildiğinden, bir çok metin bunları ayırt etmez ve her ikisine de bileşim der. Bu kitapta basitlik açısından aynı şekilde yapacağız.

toplama veya bileşim

10.10 İki nesne arasındaki toplama ilişkisi nedir?

10.11 İki nesne arasındaki bileşim ilişkisi nedir?

Print Kontrol Et

MyProgrammingLab™

10.8 Vaka Çalışması: Kurs Sınıfını Tasarlamak



Bu bölüm, dersleri modellemek için bir sınıf tasarlar.

Bu kitabın felsefesi, *örneklerle öğretmek ve uygulayarak öğrenmektir*. Kitap, nesne yönelimli programlamayı göstermek için geniş bir örnek yelpazesi sunmaktadır. Bu bölüm ve sonraki iki bölüm, sınıf tasarımıyla ilgili ek örnekler sunmaktadır.

Diyelim ki kurs bilgilerini işlemeniz gerekiyor. Her kursun bir adı vardır ve kayıtlı öğrencileri vardır. Bir öğrenciyi kursa ekleyip çıkarabilmelisiniz. Kursları modellemek için bir sınıf kullanabilirsiniz; bu, Şekil 10.9'da gösterilmiştir.

Ders	
-dersAdı: String -öğrenciler: String[] -öğrenciSayısı: int	Dersin adı. Ders için öğrencileri saklamak için bir dizi. Öğrenci sayısı (varsayılan: 0).
+Ders(dersAdı: String) +getDersAdı(): String +öğrenciEkle(öğrenci: String): void +öğrenciSil(öğrenci: String): void +getÖğrenciler(): String[] +getÖğrenciSayısı(): int	Belirtilen adla bir ders oluşturur. Kurs adını döndürür. Derse yeni bir öğrenci ekler. Bir öğrenciyi dersten çıkarır. Ders için öğrencileri döndürür. Ders için öğrenci sayısını döndürür.

Fiçin 10.9 Ders sınıfı dersleri modellemektedir.

A **Course** nesnesi, bir kurs adı geçirerek **Course(String name)** yapıcısı kullanılarak oluşturulabilir. Kursa öğrencileri **addStudent(String student)** yöntemi ile ekleyebilir, kursdan bir öğrenciyi **dropStudent(String student)** yöntemi ile çıkarabilir ve kurstaki tüm öğrencileri **getStudents()** yöntemi ile döndürebilirsiniz. Sınıf mevcutsa; Liste 10.5, iki kurs oluşturan ve onlara öğrenciler ekleyen bir test sınıfı vermektedir.

LISTE 10.5 TestCourse.java

```

1  genel sınıfTestDers {
2      public static void main(String[] args) {
3          Course course1 = newCourse("Veri Yapıları");
4          Course course2 = newCourse("Veritabanı Sistemleri");
5
6          course1.addStudent("Peter Jones");
7          ders1.addStudent(t"Kim Smith"t);
8          ders1.addStudent(t"Anne Kennedy"t);
9
10         course2.addStudent(t"Peter Jones"t);
11         ders2.addStudent(t"Steve Smith"t);
12
13         System.out.println("Kurs1'deki öğrenci sayısı: "
14             + course1.getNumberOfStudents());
15         String[] students = ders1.getStudents();
16         için(int i = 0 ; i < course1.getNumberOfStudents(); i++)
17             System.out.print(öğrenciler[i] + t", "t);
18
19         System.out.println();
20         System.out.print("Öğrenci sayısı kurs2: "
```



```
21 + ders2.getNumberOfStudents());n22 }
```

```
yirmiüç }
```

Ders1'deki öğrenci sayısı: 3
Peter Jones, Kim Smith, Anne Kennedy,
Ders2'deki öğrenci sayısı: 2



Ders sınıfı, Liste 10.6'da uygulanmıştır. Öğrencileri derste saklamak için bir dizi kullanır. Basitlik açısından, maksimum ders kaydı **100** olarak varsayılın. Dizi, 3. satırda **new String[100]** kullanılarak oluşturulmuştur. **addStudent** yöntemi (10. satır) bir öğrenciyi diziy e ekler. Yeni bir öğrenci derse eklendiğinde, **numberOfStudents** artırılır (12. satır). **getStudents** yöntemi diziy döndürür. **dropStudent** yöntemi (27. satır) bir alıştırma olarak bir akılmıştır.

LISTE 10.6 Course.java

```
1 public class Ders {
2     özelString kursAdı;
3     özelString[] öğrenciler = yeniDize[t100t] ;
4     özel int numaraÖğrenciler;
5
6     genelKurs(String kursAdı) {
7         bu.kursAdı = kursAdı;
8     }
9
10    public void addStudent(String öğrenci) {
11        öğrenciler[numaraÖğrenciler] = öğrenci;
12        numaraÖğrenciler++;
13    }
14
15    public String[] getStudents() {
16        return öğrenciler;
17    }
18
19    public int getNumberOfStudents() {
20        return numaraÖğrenciler;
21    }
22
23    public String getCourseName() {
24        return kursAdı;
25    }
26
27    public void dropStudent(String öğrenci) {
28        // Programlama Alıştırması 10.9'da bir egzersiz olarak bırakılmıştır
29    }
30 }
```

oluştur öğrenciler

bir kurs ekle

return öğrenciler

öğrenci sayısı

Dizi boyutu **100** (satır 3) olarak sabitlenmiştir, bu nedenle kursta 100'den fazla öğrenci olamaz. Sınıfı, Programlama Alıştırması 10.9'da dizinin boyutunu otomatik olarak artırarak geliştirebilirsiniz.

Bir **Kurs** nesnesi oluşturduğunuzda, bir dizi nesnesi oluşturulur. Bir **Kurs** nesnesi dizinin başvurusunu içerir. Basitlik açısından, **Kurs** nesnesinin diziyi içerdiğini söyleyebilirsiniz.

Kullanıcı bir **Kurs** nesnesi oluşturabilir ve bunu public yöntemler aracılığıyla **addStudent**, **dropStudent**, **getNumberOfStudents** ve **getStudents** ile manipüle edebilir. Ancak,

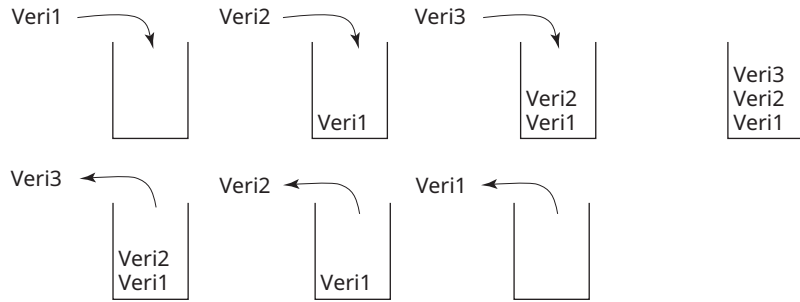
Kullanıcının bu yöntemlerin nasıl uygulandığını bilmesine gerek yoktur. **Course** sınıfı içsel uygulamayı kapsüller. Bu örnek, öğrencileri depolamak için bir dizi kullanır, ancak **öğrencile** **ri** depolamak için farklı bir veri yapısı da kullanabilirsiniz. **Course** kullanan program, public yöntemlerin sözleşmesi değişmediği sürece değişmek zorunda değildir.

10.9 Vaka Çalışması: Yığınlar için Bir Sınıf Tasarlamak



Bu bölüm, yığınları modellemek için bir sınıf tasarlar.

Bir yığın, verileri son giren ilk çıkar (LIFO) biçiminde tutan bir veri yapısıdır, bu şekilde gösterilmiştir Şekil 10.10.



FIGÜRÜNTÜ 10.10 Bir yığın, verileri son giren ilk çıkar (LIFO) şeklinde tutar.

Yığınların birçok uygulaması vardır. Örneğin, derleyici bir yöntemi işlemek için bir yığın kullanır. Bir yöntem çağrıldığında, parametreleri ve yerel değişkenleri bir yığına itilir. Bir yöntem başka bir yöntemi çağırıldığında, yeni yöntemin parametreleri ve yerel değişkenleri yığına itilir. Bir yöntem işini bitirdiğinde ve çağırana döndüğünde, ilişkili alan yığından serbest bırakılır.

Yığınları modellemek için bir sınıf tanımlayabilirsiniz. Basitlik açısından, yığının **int** değerlerini tuttuğu nu varsayın. Bu nedenle yığın sınıfına **StackOfIntegers** adını verin. Sınıfın UML diyagramı Şekil 10.11'de gösterilmiştir.



VideoNotu

StackOfIntegers sınıfı

StackOfIntegers	
-elements: int[] -size: int	An array to store integers in the stack. The number of integers in the stack.
+StackOfIntegers() +StackOfIntegers(capacity: int) +empty(): boolean +peek(): int +push(value: int): void +pop(): int +getSize(): int	Constructs an empty stack with a default capacity of 16. Constructs an empty stack with a specified capacity. Returns true if the stack is empty. Returns the integer at the top of the stack without removing it from the stack. Stores an integer into the top of the stack. Removes the integer at the top of the stack and returns it. Returns the number of elements in the stack.

FIGURE 10.11 StackOfIntegers sınıfı yığın depolamasını kapsar ve

yığını manipüle etmek için işlemleri sağlar.

Sınıfın mevcut olduğunu varsayalım. Liste 10.7'deki test programı, bir yığın oluşturmak için sınıfı kullanır (satır 3), on tam sayıyı **0,1,2, . . . , ve 9** (satır 6) depolar ve bunları ters sırayla gösterir (satır 9).

Listing 10.7 TestStackOfIntegers.java

```

1  genel classtTestStackOfIntegers {
2      public static voidtmain(String[] args) {
345         StackOfIntegers yığını = tnewtStackOfIntegers();           bir yığın oluştur

        fort(tint = t0t; i < t10t; i++)
678             stack.push(i);                                         yığına ekle

        while(!stack.empty())n9 Sys
tem.out.print(                yığın.pop() + " "t);n                 yığınınndan pop
10 }
11 }

```

9 8 7 6 5 4 3 2 1 0



StackOfIntegers sınıfını nasıl uyguluyorsunuz? Yığındaki elemanlar, `elements` adın da bir dizide saklanır. Bir yığın oluşturduğunuzda, dizi de oluşturulur. Parametre almayan yapıcı, varsayılan kapasitesi **16** olan bir dizi oluşturur. **size** değişkeni, yığındaki elemanların sayısını sayar ve **size - 1**, yığının en üstündeki elemanın indeksidir, Şekil 10.12'de gösterildiği gibi. Boş bir yığın için, **size 0**'dır.

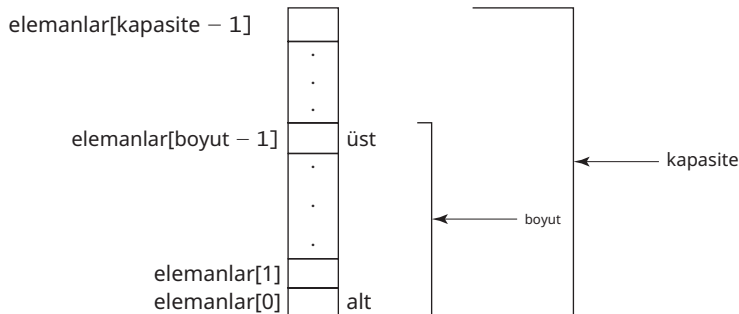


FIGURE 10.12 StackOfIntegers sınıfı yığın depolamasını kapsar ve yığını manipüle etmek için işlemleri sağlar.

StackOfIntegers sınıfı, Liste 10.8'de uygulanmıştır. **empty()**, **peek()**, **pop()** ve **getSize()** yöntemlerini uygulamak kolaydır. **push(int value)** uygulamak için, `value`'yi `elements[size]`'ye atayın eğer **size < capacity** ise (satır 24). Eğer yığın doluyorsa (yani, **size >= capacity**), mevcut kapasitenin iki katı kadar yeni bir dizi oluşturun (satır 19), mevcut dizinin içeriğini yeni diziyeye kopyalayın (satır 20) ve yığındaki mevcut dizinin referansını yeni diziyeye atayın (satır 21). Artık yeni değeri diziyeye ekleyebilirsiniz (satır 24).

Listing 10.8 StackOfIntegers.java

```

1  halk   classtStackOfIntegers {
2      private int[] elemanlar;
3      private intboyut;
4      genel statik final inttDEFAULT_CAPACITY = t16t;           maks kapasite 16
5
6      /** Varsayılan kapasite 16 olan bir yığın oluştur */
7      blictStackOfIntegers() {
8          thist(DEFAULT_CAPACITY);npu

```

```

9      }
1011
12     /** Belirtilen maksimum kapasite ile bir yığın oluştur */
13     public StackOfIntegers(int capacity) {
14         elemanlar = new int[kapasite];
15     }
16
17     /** Yığının üstüne yeni bir tam sayı ekle */
18     public void push(int değer) {
19         eğer(boyut >= elemanlar.uzunluğu) {
20             int[] temp = new int[elements.length * 2];
21             System.arraycopy(elements, 0, temp, 0, elements.length);
22             elements = temp;
23         }
24         öğe[size++] = değer;
25     }
26
27     /** Yığın üzerindeki en üst öğeyi döndür ve kaldır */
28     genel int pop() {
29         dönüş öğeleri[--boyut];
30     }
31
32     /** Yığın üzerindeki en üst öğeyi döndür */
33     genel int peek() {
34         dönüş öğeleri[boyut - 1];
35     }
36
37     /** Yığının boş olup olmadığını test et */
38     genel boolean empty() {
39         dönüş boyutu == 0;
40     }
41
42     /** Yığındaki eleman sayısını döndür */
43     genel int getSize() {
44         dönüş boyutu;
45     }
46 }

```

kapasiteyi iki katına çıkar

yığına ekle

10.10 Vaka Çalışması: GuessDate Sınıfını Tasarlamak



Statik yöntemler ve statik veriler içeren yardımcı sınıflar tanımlayabilirsiniz.

Liste 3.3, GuessBirthday.java ve Liste 7.6, GuessBirthdayUsingArray.java, doğum günlerini tahmin etmek için iki program sunmuştur. Her iki program da prosedürel paradigma ile geliştirilmiş aynı verileri kullanmaktadır. Bu iki programdaki kodun çoğunluğu beş veri kümesini tanımlamak içindir. Bu iki programdaki kodu yeniden kullanamazsınız, çünkü kod main yönteminde. Kodu yeni den kullanılabilir hale getirmek için, verileri kapsülleyen bir sınıf tasarlayın, Şekil 10.13'te tanımlanmış gibi.

getValue, GuessDate sınıfının belirli bir nesnesine bağımlı olmadığı için statik bir yöntem olarak tanımlandığını unutmayın. GuessDate sınıfı, dates'ı özel bir üye olarak kapsüller. Bu sınıfın kullanıcısının, dates'ın nasıl uygulandığını veya sınıfta dates alanının var olduğunu bilmesine gerek yoktur. Kullanıcının bilmesi gereken tek şey, bu yöntemi kullanarak dates'a nasıl erişileceğidir. Bu sınıfın mevcut olduğunu varsayalım. Bölüm 3.4'te gösterildiği gibi, beş tarih kümesi vardır. getValue(setNo, row, column) çağırısı, belirtilen satır ve sütundaki tarihi döndürür. Örneğin, getValue(1, 0, 0), 2 döndürür.

Tahmin Tarihi	
-tarihler: int[][]	The static array to hold dates.
+getValue(setNo: int, satır: int, n sütun: int): int	Returns a date at the specified row and column in a given set.

FIGURE 10.13 GuessDate sınıfı doğum günlerini tahmin etmek için verileri tanımlar.

GuessDate sınıfının mevcut olduğunu varsayalım. Liste 10.9, bu

sınıfı kullanan bir test programıdır.

LISTE 10.9 UseGuessDateClass.java

```

1  import java.util.Scanner;
2
3  genel siniftUseGuessDateClass {
4      public static voidtmain(String[] args) {
5          intdate = 0; // Tarih belirlenecek
6          int cevabi;
7
8          // Bir Scanner oluştur
9  Tarayıcı girişi = tnewtScanner(System.in);
10
11      fort(tint = t0t; i < t5t; i++) {
12          System.out.println("Doğum gününüz Set'te mi" + (i + 1) + "?");
13          güçlü(tinttj = t0t; j < t4t; j++) {
14              fort(tinttk = t0t; k < t4t; k++)
15                  System.out.print(GuessDate.getValue(i, j, k) + " ");
16          System.out.println();
17      }
18
19      System.out.print("Hayır için 0, Evet için 1 girin: ");
20      answer = input.nextInt();
21
22      eğer(cevap == 1)
23          date += GuessDate.getValue(i, t0t, t0t);
24      }
25
26      System.out.println("Doğum gününüz " + date);
27  }
28  }
```

statik yöntemi çağırarak

statik yöntemi çağırarak



Doğum gününüz Set1'de mi?

1 3 5 7
9 11 13 15
17 19 21 23
25 27 29 31

Hayır için 0 ve Evet için 1 girin:

0 Enter

Doğum gününüz Set2'de mi?

2 3 6 7
10 11 14 15
18 19 22 23
26 27 30 31

Hayır için 0 ve Evet için 1 girin: 1

Enter

Doğum gününüz Set3'de mi?
 4 5 6 7
 12 13 14 15
 20 21 22 23
 28 29 30 31
 Hayır için 0 ve Evet için 1 girin:

Doğum gününüz Set4'te mi?
 8 9 10 11
 12 13 14 15
 24 25 26 27
 28 29 30 31
 Hayır için 0 ve Evet için 1 girin:

Doğum gününüz Set5'te mi?
 16 17 18 19
 20 21 22 23
 24 25 26 27
 28 29 30 31
 Hayır için 0 ve Evet için 1 girin:

Doğum gününüz 26

getValue bir statik yöntem olduğu için, onu çağırmak için bir nesne oluşturmanıza gerek yoktur. GuessDate.getValue(i, j, k) (satır 15) Set i'deki j satırındaki ve k sütunundaki tarihi döndürür. GuessDate sınıfı, Liste 10.10'da olduğu gibi uygulanabilir.

LİSTE 10.10 GuessDate.java

statik alan

```

1 genel sınıftGuessTarih {
2     private final static int[][] tarihler = {
3         {{ t1t,
4             3, 5, 7,
5             { t9t,t11t,t13t,t15t},n5 {t17t,t1
6             9t,t21t,t23t},n6 {t25t,t27t,t29t,t3
7             1t}},n7 {{ t2t,t3t,t6t,t7t},n8 {t10t,t
8             11t,t14t,t15t},n9 {t18t,t19t,t22t,t
9             23t},n10 {t26t,t27t,t30t,t31t}},n1
10            1 {{t4t,t5t,t6t,t7t},n12 {t12t,t13t,
11            t14t,t15t},n13 {t20t,t21t,t22t,t23t},
12            n14 {t28t,t29t,t30t,t31t}},n15 {{t8
13            t,t9t,t10t,t11t},n16 {t12t,t13t,t14t,
14            t15t},n17 {t24t,t25t,t26t,t27t},n18
15            {t28t,t29t,t30t,t31t}},n19 {{t16t,t1
16            7t,t18t,t19t},n20 {t20t,t21t,t22t,t2
17            3t},n21 {t24t,t25t,t26t,t27t},n22 {
18            t28t,t29t,t30t,t31t}}};
19
20
21
22
23
24     /** Kullanıcının GuessDate'den nesne oluşturmasını engelle */
25     PrivatetGuessDate() {
26     }
27
28     /** Verilen bir kümede belirtilen satır ve sütundaki tarihi döndür */

```

özel yapıcı

```

2930 public Static intttgetValue(intttsetNo,inttt,intttj) {
31     return tarihler[setNo][i][j];
32 }

```

statik yöntem

Bu sınıf, tarihler için üç boyutlu bir dizi kullanır (satırlar 2–22). Beş sayı kümesin i temsil etmek için farklı bir veri yapısı (yani, beş iki boyutlu dizi) kullanabilirsiniz. `getValue` yönteminin uygulanması değişecektir, ancak `GuessDate` kullanan program, public yönteminin sözleşmesi `getValue` değişmediği sürece değişiklik yapmasına gerek kalmayacaktır. Bu, veri kapsüllemenin faydasını gösterir.

veri kapsüllemenin faydası
özel yapıcı

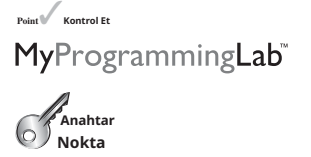
Sınıf, bu sınıf için nesne oluşturulmasını önlemek amacıyla özel bir parametresiz yapıcı t anımlar (satır 25). Bu sınıfta tüm yöntemler statik olduğundan, bu sınıftan nesne oluşturul masına gerek yoktur.

10.12 `Math` sınıfındaki parametresiz yapıcı neden özel olarak tanımlanmıştır?

10.11 Sınıf Tasarım Kılavuzları

Sınıf tasarım kılavuzları, sağlam sınıflar tasarlamak için faydalıdır.

Önceki iki örnekten ve önceki bölümlerdeki birçok diğer örnekten sınıf tasarla mayı öğrendiniz. Bu bölüm, bazı kılavuzları özetlemektedir.



10.11.1 Birliktelik

Bir sınıf, tek bir varlığı tanımlamalıdır ve tüm sınıf işlemleri mantıksal olarak bir araya gelerek tutarlı bir amacı desteklemelidir. Örneğin, öğrenciler için bir sınıf kullanabilirsiniz, ancak öğre ncileri ve personeli aynı sınıfta birleştirmemelisiniz, çünkü öğrenciler ve personel farklı varlıkla rdır.

tutarlı amaç

Birçok sorumluluğa sahip tek bir varlık, sorumlulukları ayırmak için birkaç sı nıfa bölünebilir. Örneğin, `String`, `StringBuilder` ve `StringBuffer` sınıfları hepsi dizelerle ilgilenir, ancak farklı sorumluluklara sahiptir. `String` sınıfı değişmez d izelerle ilgilenirken, `StringBuilder` sınıfı değiştirilebilir dizeler oluşturmak içind ir ve `StringBuffer` sınıfı, dizeleri güncellemek için senkronize yöntemler içerm esi dışında `StringBuilder` sınıfına benzer.

sorumlulukları ayırma

10.11.2 Tutarlılık

Standart Java programlama stilini ve adlandırma kurallarını takip edin. Sınıflar, veri alanları ve yön temler için bilgilendirici isimler seçin. Popüler bir stil, veri bildirimini yapıcıdan önce yerleştirmek ve yapıcıları yöntemlerden önce yerleştirmektir.

isimlendirme kuralları

İsimleri tutarlı hale getirin. Benzer işlemler için farklı isimler seçmek iyi bir uyg ulama değildir. Örneğin, `length()` yöntemi bir `String`' in boyutunu döndürür, bir `StringBuilder` ve `StringBuffer`. Bu sınıflarda bu yöntem için farklı adlar kullan ılırsa tutarsızlık olur.

isimlendirme tutarlılığı

Genel olarak, varsayılan bir örnek oluşturmak için tutarlı bir şekilde bir public no-arg yapıcı sa ğlamalısınız. Bir sınıf no-arg yapıcısını desteklemiyorsa, nedeni belgeleyin. Eğer hiçbir yapıcı açıkç a tanımlanmamışsa, boş bir gövdeye sahip bir public varsayılan no-arg yapıcısı varsayılır.

argüman almayan yapıcı

Kullanıcıların bir sınıf için bir nesne oluşturmasını önlemek istiyorsanız, sınıfta private bi r yapıcı tanımlayabilirsiniz; bu, `Math` sınıfı ve `GuessDate` sınıfı için geçerlidir.

10.11.3 Kapsülleme

Bir sınıf, verilerini istemcilerin doğrudan erişiminden gizlemek için `private` belirleyicisini kullanmalıdı r. Bu, sınıfın bakımını kolaylaştırır.

veri alanlarını kapsülleme

Bir **get** yöntemi sağlayın yalnızca alanın okunabilir olmasını istiyorsanız ve bir **set** yöntemi sağlayın yalnızca alanın güncellenebilir olmasını istiyorsanız. Örneğin, `Course` sınıfı `courseName` için bir `get` yöntemi sağlar, ancak kullanıcı oluşturulduktan sonra

kurs adını değiştirmesine izin verilmediği için bir `set` yöntemi yoktur.

10.11.4 Açıklık

açıklaması kolay

Bütünlük, tutarlılık ve kapsülleme, tasarım netliğine ulaşmak için iyi kılavuzlardır. Ayrıca, bir sınıfın açıklaması kolay ve anlaşılması kolay bir sözleşmeye sahip olması gerekir.

Kullanıcılar, sınıfları birçok farklı kombinasyon, sıra ve ortamda birleştirebilir.

bağımsız yöntemler

Bu nedenle, kullanıcıya nasıl veya ne zaman kullanacağı konusunda hiçbir kısıtlama getirmeyen bir sınıf tasarlamalısınız, özellikleri kullanıcıya herhangi bir sırayla ve herhangi bir değer kombinasyonu ile ayarlama imkanı verecek şekilde tasarlamalısınız ve yöntemleri gerçekleşme sıralarından bağımsız olarak çalışacak şekilde tasarlamalısınız. Örneğin, `Loan` sınıfı `loanAmount`, `numberOfYears` ve `annualInterestRate` özelliklerini içerir. Bu özelliklerin değerleri herhangi bir sırayla ayarlanabilir.

sezgisel anlam

Yöntemler, kafa karışıklığına neden olmadan sezgisel bir şekilde tanımlanmalıdır. Örneğin, `substring(int beginIndex, int endIndex)` yöntemi `String` sınıfında biraz- kaf a karıştırıcıdır. Yöntem, `beginIndex` ile `endIndex - 1` arasında bir alt dize döndürür, `endIndex`'e kadar değil. `beginIndex` ile `endIndex` arasında bir alt dize döndürmek daha sezgisel olurdu.

bağımsız özellikler

Diğer veri alanlarından türetilen bir veri alanı tanımlamamalısınız. Örneğin, aşağıdaki `Person` sınıfının iki veri alanı vardır: `birthDate` ve `age`. `age`, `birthDate`'den türetilen bir veriden, `age` bir veri alanı olarak tanımlanmamalıdır.

KÖTÜ KOD

```
public class Kişi {
    private java.util.Date doğumTarihi;
    özel int yaş;

    ...
}
```

10.11.5 Tamlık

Sınıflar, birçok farklı müşteri tarafından kullanılmak üzere tasarlanmıştır. Geniş bir uygulama yelpazesinde faydalı olabilmek için, bir sınıf, özellikler ve yöntemler aracılığıyla özelleştirme için çeşitli yollar sunmalıdır. Örneğin, `String` sınıfı, çeşitli uygulamalar için faydalı olan 40'tan fazla yöntemi içermektedir.

10.11.6 Örnek vs. Statik

Bir değişken veya yöntem, sınıfın belirli bir örneğine bağlıysa, bir örnek değişkeni veya yöntemi olmalıdır. Sınıfın tüm örnekleri tarafından paylaşılan bir değişken statik olarak tanımlanmalıdır. Örneğin, Listing 8.9'daki `CircleWithPrivateDataFields` içindeki `numberOfObjects` değişkeni, `CircleWithPrivateDataFields` sınıfının tüm nesneleri tarafından paylaşılmakta ve bu nedenle statik olarak tanımlanmaktadır. Belirli bir örneğe bağlı olmayan bir yöntem, statik bir yöntem olarak tanımlanmalıdır. Örneğin, `getNumberOfObjects` yöntemi, `CircleWithPrivateDataFields` içinde herhangi bir belirli örneğe bağlı değildir ve

bu nedenle statik bir yöntem olarak tanımlanmıştır.

Statik değişkenleri ve yöntemleri, okunabilirliği artırmak ve hataları önlemek için bir sınıf adından (bir referans değişkeni yerine) her zaman referans alın.

Bir yapıcıdan statik bir veri alanını başlatmak için bir parametre geçmeyin. Statik veri alanını değiştirmek için bir `set` yöntemi kullanmak daha iyidir. Böylece, (a) sınıfı (b) ile eğitilmelidir.


```
public class Something {
    private int t1;
    private static int t2;

    public Something(int t1, int t2) {
        // ...
    }
}
```

(a)

```
public class Something {
    private int t1;
    private static public int t2;

    Something(int t1) {
        // ...
    }

    public static void setT2(int t2) {
        Something.t2 = t2;
    }
}
```

(b)

Örnek ve statik, nesne yönelimli programlamanın ayrılmaz parçalarıdır. Bir veri alanı veya yöntem ya örnek ya da statiktir. Statik veri alanlarını veya yöntemleri yanlışlıkla gözden kaçırmayın. Statik olması gereken bir örnek yöntem tanımlamak yaygın bir tasarım hatasıdır. Örneğin, `factorial(int n)` yöntemi, `n` sayısının faktöriyelini hesaplamak için statik olarak tanımlanmalıdır, çünkü belirli bir örnekten bağımsızdır.

Bir yapıcı her zaman örnektir, çünkü belirli bir örneği oluşturmak için kullanılır. Bir statik değişken veya yöntem, bir örnek yönteminden çağrılabilir, ancak bir örnek değişkeni veya yöntemi bir statik yöntemden çağrılmaz.

yaygın tasarım hatası

10.13 Sınıf tasarım yönergelerini tanımlayın.

10.12 Temel Veri Tipi Değerlerini Nesneler Olarak İşleme

Bir temel tip değeri bir nesne değildir, ancak Java API'sindeki bir

sarmalayıcı sınıf kullanılarak bir nesne içinde sarılabilir.

Performans nedenleriyle, Java'da ilkel veri türü değerleri nesne değildir.

Nesnelerin işlenmesinin getirdiği ek yük nedeniyle, ilkel veri türü değerleri nesne olarak işlenirse dilin performansı olumsuz etkilenir. Ancak, birçok Java yöntemi nesneleri argüman olarak kullanmayı gerektirir. Java, bir ilkel veri türünü bir nesneye dahil etmenin veya sarmalının (örneğin, `int` değerini `Integer` sınıfına sarmak ve `double` değerini `Double` sınıfına sarmak) pratik bir yolunu sunar. Bir `char` değerinin nasıl sarmalanabileceğini 9.5. bölümde hatırlayın.

Bir sarmalayıcı sınıf kullanarak, ilkel veri türü değerlerini nesne olarak işleyebilirsiniz. Java, ilkel veri türleri için `java.lang` paketinde `Boolean`, `Character`, `Double`, `Float`, `Byte`, `Short`, `Integer` ve `Long` sarmalayıcı sınıflarını sağlar. `Boolean` sınıfı bir Boolean değerini `true` veya `false` olarak sarar. Bu bölüm, sayısal sarmalayıcı sınıfları tanıtmak için `Integer` ve `Double`'i örnek olarak kullanır.

Point Kontrol Et

MyProgrammingLab™



sarmalayıcı sınıf neden?



Not

Bir ilkel tür için çoğu sarmalayıcı sınıf adı, ilkel veri türü adının ilk harfi büyük olacak şekilde aynıdır. İstisnalar `Integer` ve `Character`'dir.

isimlendirme kuralı

Sayısal sarmalayıcı sınıflar birbirine çok benzer. Her biri aşağıdaki yöntemleri içerir: `doubleValue()`, `floatValue()`, `intValue()`, `longValue()`, `shortValue()` ve `byteValue()`. Bu yöntemler, nesneleri ilkel tür değerlerine “dönüştürür”. `Integer` ve `Double`'in ana özellikleri Şekil 10.14'te gösterilmiştir.

Bir sarmalayıcı nesnesi, ya bir ilkel veri türü değerinden ya da sayısal değeri temsil eden bir string'den oluşturulabilir—örneğin, `new Double(5.0)`, `new Double("5.0")`, `new Integer(5)` ve `new Integer("5")`.

yapıcılar

java.lang.Tamsayı	java.lang.Double
-değer: int <u>+MAX_DEĞER: int</u> <u>+MIN_DEĞER: int</u>	-değer: double <u>+MAX_DEĞER: double</u> <u>+MIN_DEĞER: double</u>
+Integer(değer: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longValue(): long +floatValue(): float +doubleValue(): double +compareTo(o: Integer): int +toString(): String <u>+valueOf(s: String): Tamsayı</u> <u>+valueOf(s: String, radix: int): Integer</u> <u>+parseInt(s: String, radix: int): int</u>	+Double(value: double)n+D ouble(s: String)n+byte Value(): byten+shortVa lue(): shortn+intValue(): i ntn+longVlaue(): lon gn+floatValue(): floatn +doubleValue(): doublen +compareTo(o: Double): int n+toString(): String <u>+valueOf(s: String): Çift</u> <u>+valueOf(s: String, radix: int): Double</u> <u>+parseDouble(s: String): double</u> <u>+parseDouble(s: String, radix: int): double</u>

FIGURE 10.14 Sarmalayıcı sınıflar, çeşitli veri türlerini işlemek için yapıcılar, sabitler ve dönüştürme yöntemleri sağlar.

argümansız kurucu
değişmez

Wrapper sınıflarının argüman almayan yapıcıları yoktur. Tüm wrapper sınıflarının örnekleri değiştirilemez; bu, nesneler oluşturulduktan sonra, iç değerlerinin değiştirilemeyeceği anlamı na gelir.

sabitler

Her sayısal wrapper sınıfı **MAX_VALUE** ve **MIN_VALUE** sabitlerine sahiptir. **MAX_VALUE** karşılık gelen ilkel veri tipinin maksimum değerini temsil eder. **Byte**, **Short**, **Integer** ve **Long** için, **MIN_VALUE** minimum **byte**, **short**, **int** ve **long** değerlerini temsil eder. **Float** ve **Double** için, **MIN_VALUE** minimum *pozitif Float* ve *double* değerlerini temsil eder. Aşağıdaki ifadeler maksimum tam sayıyı (2,147,483,647), minimum pozitif float'ı (1.4E-45) ve maksimum double kayan nokta sayısını gösterir. (1,79769313486231570t+t08d).

```
System.out.println("En büyük tam sayı "+ Integer.MAX_VALUE);
System.out.println("En küçük pozitif float "+
    Float.MIN_VALUE);
System.out.println(
    "En büyük çift hassasiyetli kayan nokta sayısı "+
    Çift.MAX_VALUE);
```

dönüşüm yöntemleri

Her sayısal sarmalayıcı sınıfı, sarmalayıcı nesne için bir **double**, **float**, **int**, **long** veya **short** değeri döndürmek için **doubleValue()**, **floatValue()**, **intValue()**, **longValue()** ve **shortValue()** yöntemlerini içerir. Örneğin,

```
new Double("12.4").intValue() 12 döner;
new Integer("12").doubleValue() 12.0 döner;
```

compareTo yöntemi

String sınıfının iki dizeyi karşılaştırmak için **compareTo** yöntemini içerdiğini unutmayın. Sayı sarmalayıcı sınıfları, iki sayıyı karşılaştırmak için **compareTo** yöntemini içerir ve bu sayı diğer sayıdan büyükse, eşitse veya küçükse **1**, **0** veya **-1** döner. Örneğin,

`new Double("12.4").compareTo(new Double("12.3"))` 1 döner; `new Double("12.3").compareTo(new Double("12.3"))` 0 döner; `new Double("12.3").compareTo(new Double("12.51"))` -1 döner; Sayısal sarmalayıcı sınıflar, **v**

`valueOf (String s)` adında yararlı bir statik yöntemdir. Bu yöntem, belirtilen diziyi `valueOf` yöntemleri tarafından temsil edilen değere ayarlanmış yeni bir nesne oluşturur. Örneğin,

```
Double doubleObject = Double.valueOf("12.4");
Integer integerObject = Integer.valueOf("12");
```

Sayısal bir diziyi int değerine dönüştürmek için **Integer** sınıfında `parseInt` yöntemini ve sayısal bir diziyi **double** değerine dönüştürmek için **Double** sınıfında `parseDouble` yöntemini kullanılır. Her sayısal sarmalayıcı sınıf, bir sayısal diziyi uygun bir sayısal değere dönüştürmek için iki aşırı yüklenmiş ayrıştırma yöntemine sahiptir; bu, **10** (ondalık) veya belirtilen herhangi bir tabana (örneğin, **2** ikili, **8** sekizli ve **16** onaltılı) dayanır. Aşağıdaki örnekler bu yöntemlerin nasıl kullanılacağını göstermektedir.

statik ayrıştırma yöntemleri

```
// Bu iki yöntem Byte sınıfındadır
public static byte parseByte(String s)
public static byte parseByte(String s, int radix)

// Bu iki yöntem Short sınıfındadır
public static short parseShort(String s)
public static short parseShort(String s, int radix)

// Bu iki yöntem Integer sınıfındadır
public static int parseInt(String s)
public static int parseInt(String s, int radix)

// Bu iki yöntem Long sınıfındadır
public static long parseLong(String s)
public static long parseLong(String s, int radix)

// Bu iki yöntem Float sınıfındadır
public static float parseFloat(String s)
public static float parseFloat(String s, int radix)

// Bu iki yöntem Double sınıfındadır
public static double parseDouble(String s)
public static double parseDouble(String s, int radix)
```

Örneğin,

```
Integer.parseInt("11", 2) 3 döner;
Integer.parseInt("12", 8) 10 döner;
Integer.parseInt("13", 10) 13 döner;
Integer.parseInt("1A", 16) 26 döner;
```

`Integer.parseInt("12", 2)` bir çalışma zamanı istisnası oluşturur çünkü **12** bir ikili sayı değildir.

Ondalık bir sayıyı onaltılık bir sayıya dönüştürmek için `format` yöntemini kullanabileceğinizi unutmayın.

ondalık sayıyı onaltılığa dönüştürme

Örneğin,

```
String.format("%x", 26) 1A döner;
```

10.14 Temel tür sarmalayıcı sınıflarını tanımlayın.

10.15 Aşağıdaki ifadelerin her biri derlenebilir mi?

- Tamsayı i = tnewtInteger(t"23"t);
- Tamsayı i = tnewtInteger(t23t);
- Tamsayı i = Integer.valueOf(t"23"t);
- Tamsayı i = Integer.parseInt(t"23", 8t);
- Double d = tnewtDouble();
- Double d = Double.valueOf(t"23.45"t);
- int = (Integer.valueOf(t"23"t)).intValue();
- doubletd = (Double.valueOf(t"23,4"t)).doubleValue();
- Ben. int = (Double.valueOf(t"23.4"t)).intValue();
- String s = (Double.valueOf(t"23.4"t)).toString();

10.16 Bir tam sayıyı nasıl bir dizeye dönüştürürsünüz? Sayısal bir dizeyi

bir tam sayıya nasıl dönüştürürsünüz? Bir double sayıyı nasıl bir dizeye dönüştürürsünüz? Sayısal bir dizeyi nasıl bir double değere dönüştürürsünüz?

10.17 Aşağıdaki kodun çıktısını gösterin.

```
genel sınıfTest {
    public static voidtmain(String[] args) {nInteger x = t
        newtInteger(t3t);
        System.out.println(x.intValue());nSystem.o
        ut.println(x.compareTo(tnewtInteger(t4t)));n}
}
```

10.18 Aşağıdaki kodun çıktısı nedir?

```
genel sınıfTest {
    public static voidtmain(String[] args) {nSystem.out.
        println(Integer.parseInt(t"10"t));nSystem.out.println(In
        teger.parseInt(t"10"t,t10t));nSystem.out.println(Integer.pars
        eInt(t"10"t,t16t)); nSystem.out.println(Integer.parseInt(t"11"
        t));nSystem.out.println(Integer.parseInt(t"11"t,t10t));nS
        ystem.out.println(Integer.parseInt(t"11"t,t16t));n}
}
```

10.13 Temel Türler ve Wrapper Sınıf Türleri A rasında Otomatik Dönüşüm



Bir temel tür değeri, bağlama bağlı olarak bir nesneye otomatik olarak dönüştürülebilir ve tersine.

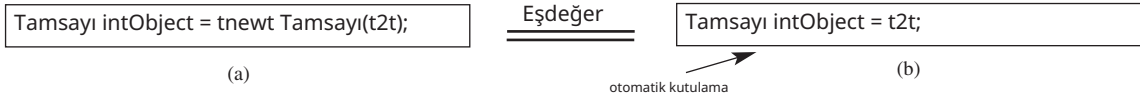
boxing
kutu açma

otomatik kutulama
otomatik kutudan çıkarma

Bir ilkel değeri bir sarmalayıcı nesneye dönüştürmeye *boxing* denir. Ters dönüşüm ise *unboxing* olarak adlandırılır. Java, ilkel türlerin ve sarmalayıcı sınıfların otomatik olarak dönüştürülmesini e izin verir. Derleyici, bir nesne gerektiren bir bağlamda görünen bir ilkel değeri otomatik olarak sarmalar ve bir ilkel değer gerektiren bir bağlamda görünen bir nesneyi açar. Bu *autoboxing* ve *autounboxing* olarak adlandırılır.

10.14 BigInteger ve BigDecimal Sınıfları 397

Örneğin, (a) içindeki aşağıdaki ifade, autoboxing nedeniyle (b) olarak basitleştirilebilir.



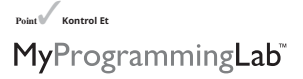
Aşağıdaki örneği düşünün:

```
1 Tamsayı[] intArray = {t1t,t2t,t3t};n2 Syste  
m.out.println(intArray[t0t] + intArray[t1t] + intArray[t2t]);
```

1. satırda, ilkel değerler **1,2** ve **3** otomatik olarak nesnelere **new Integer(1),new Integer(2)** ve **new Integer(3)** dönüştürülür. 2. satırda, nesneler **intArray[0],intArray[1]** ve **intArray[2]** otomatik olarak **int** değerlerine dönüştürülür ve toplanır.

10.19 Otomatik kutulama ve otomatik kutudan çıkarma nedir? Aşağıdaki ifadeler doğru mu?

- a. Tamsayı x = t3t+tnewtInteger(t5t);
- b. Tam sayı x = t3t;
- c. Çift x = t3t;
- d. Çift x = t3.0t;
- ve. **inttx =tnewtInteger(t3t);**
- f. **inttx = tnewtInteger(t3t) + tnewtInteger(t4t);**



10.20 Aşağıdaki kodun çıktısını gösterin?

```
genel sınıfTest {  
    public static voidtmain(String[] args) {nDouble x = t  
        newtDouble(t3.5t);  
        System.out.println(x.intValue());nSystem.o  
        ut.println(x.compareTo(t4.5t));n}  
}
```

10.14 BigInteger veBigDecimal Sınıfları

BigInteger ve BigDecimal sınıfları, herhangi bir boyut ve hassasiyette tam sayıları veya ondalık sayıları temsil etmek için kullanılabilir.

Eğer çok büyük tam sayılar veya yüksek hassasiyetli kayan nokta değerleri ile hesaplama yapmanız gerekiyorsa, java.math paketindeki BigInteger ve BigDecimal sınıflarını kullanabilirsiniz. Her ikisi de değişmezdir. long türünün en büyük tam sayısı Long.MAX_VALUE'dır (yani, 9223372036854775807). BigInteger'in bir örneği, herhangi bir boyutta bir tam sayıyı temsil edebilir. BigInteger ve BigDecimal örneği oluşturmak için new BigInteger(String) ve new BigDecimal(String) kullanılabilir, aritmetik işlemler yapmak için add, subtract, multiple, divide ve remainder yöntemlerini kullanabilir ve iki büyük sayıyı karşılaştırmak için compareTo yöntemini kullanabilirsiniz. Örneğin, aşağıdaki kod iki BigInteger nesnesi oluşturur ve bunları çarpar.

```
BigInteger a = tnewtBigInteger(t"9223372036854775807"t);nBigInte  
ger b = tnewtBigInteger(t"2"t);  
BüyükTamsayı c = a.çarpma(b); t// 9223372036854775807 * 2nSystem.o  
ut.println(c);
```

Çıktı **18446744073709551614.**



VideoNotu

Büyük sayıları işleyin

değişmez

Bir `BigDecimal` nesnesinin hassasiyetinde bir sınır yoktur. `divide` yöntemi, sonuç s onlandırılmıyorsa bir `ArithmeticException` fırlatabilir. Ancak, bu istisnayı önlemek iç in bir ölçek ve yuvarlama modu belirtmek üzere `divide(BigDecimal d , int scale, int roundingMode)` yöntemini kullanabilirsiniz; burada `scale`, ondal ık noktadan sonraki maksimum basamak sayısıdır. Örneğin, aşağıdaki kod iki `BigDecimal` nesnesi oluşturur ve `20` ölçeği ve yuvarlama modu ile bölme işlemi yapar `BigDecimal.ROUND_UP`.

```
BigDecimal a = new BigDecimal("1.0");
BigDecimal b = new BigDecimal("3");
BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);
System.out.println(c);
```

Çıktı `0.33333333333333333334`.

Bir tam sayının faktöriyelinin çok büyük olabileceğini unutmayın. Liste 10.11, herhangi bir tam sayının faktöriyel ini döndürebilen bir yöntemi verir.

LISTE 10.11 LargeFactorial.java

```
1  import java.math.*;
2
3  genel sınıf LargeFactorial {
4      public static void main(String[] args) {
5          System.out.println("50! şu " + factorial(50));
6      }
7
8      public static BigInteger faktöriyel(int n) {
9          BigInteger sonuç = BigInteger.BİR;
10         for (int i = 1; i <= n; i++)
11             sonuç = sonuç.multiply(new BigInteger(i + ""));
12
13         return sonuç;
14     }
15 }
```

sabit
çarp



```
50! şu
30414093201713378043612608166064768844377641568960512000000000000
```

`BigInteger.BİR` (satır 9) `BigInteger` sınıfında tanımlanmış bir sabittir. `BigInteger.BİR`, `new BigInteger("1")` ile aynıdır.

Yeni bir sonuç, `multiply` yöntemini çağırarak (satır 11) elde edilir.

10.21 Aşağıdaki kodun çıktısı nedir?

```
genel sınıf Test {
    public static void main(String[] args) {
        java.math.BigInteger x = new java.math.BigInteger("3");
        java.math.BigInteger y = new java.math.BigInteger("7");
        x.add(y);

        System.out.println(x);
    }
}
```

AÇIKLAMALAR

soyut veri tipi (ADT) 375	has-a ilişkisi 382
birleştirme 382	değişmez sınıf 370
boxing 396	değişmez nesne 370
sınıf soyutlaması 375	çokluk 382
sınıf kapsülleme 375	yığın 386
sınıfın sözleşmesi 375	this anahtar kelimesi 373
sınıfın değişkeni 371	unboxing 396
bileşim 382	

BÖLÜM ÖZETİ

1. Oluşturulduktan sonra, bir *değişmez nesne* değiştirilemez. Kullanıcıların bir nesneyi değiştirmesini önlemek için *değişmez sınıflar* tanımlayabilirsiniz.
2. Örnek ve statik değişkenlerin kapsamı, değişkenlerin nerede tanımlandığına bakıl maksızın, tüm sınıfı kapsar. Örnek ve statik değişkenler sınıfın herhangi bir yerinde tanımlanabilir. Tutarlılık açısından, bu kitapta sınıfın başında tanımlanırlar.
3. this anahtar kelimesi, çağıran nesneye atıfta bulunmak için kullanılabilir. Ayrıca, aynı sınıfın başka bir yapıcısını çağırmak için bir yapıcı içinde de kullanılabilir.
4. Prosedürel paradigma, yöntemlerin tasarımına odaklanır. Nesne yönelimli paradigma, verileri ve yöntemleri nesneler halinde bir araya getirir. Nesne yönelimli paradigma kullanarak yazılım tasarımı, nesnelere ve nesneler üzerindeki işlemlere odaklanır. Nesne yönelimli yaklaşım, prosedürel paradigmanın gücünü, verileri işlemlerle birleştiren ek bir boyutla birleştirir.
5. Pek çok Java yöntemi, argüman olarak nesnelerin kullanılmasını gerektirir. Java, bir ilk el veri türünü bir nesneye (örneğin, `int`'yi `Integer` sınıfına sarmak ve `double`'i `Double` sınıfına sarmak) dahil etmek veya sarmak için kullanışlı bir yol sunar.
6. Java, bir ilkel tür değerini bağlamda karşılık gelen sarmalayıcı nesneye otomatik olarak dönüştürebilir ve tersine.
7. `BigInteger` sınıfı, her boyuttaki tam sayıları hesaplamak ve işlemek için kullanışlıdır. `BigDecimal` sınıfı, herhangi bir rastgele hassasiyetle kayan nokta sayılarını hesaplamak ve işlemek için kullanılabilir.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: www.cs.armstrong.edu/liang/intro9e/test.html.

PROGRAMLAMA EGZERSİZLERİ

Bölümler 10.2–10.6

*10.1 (Time sınıfı) Time adında bir sınıf tasarlayın. Sınıf şunları içerir:

- saat, dakika ve saniye veri alanları, bir zamanı temsil eder.
- Geçerli zaman için bir Time nesnesi oluşturan bir argüman almayan yapıcı. (Veri alanlarının değerleri geçerli zamanı temsil edecektir.)

- Gece yarısından, 1 Ocak 1970'ten itibaren belirli bir geçen süre ile bir Time nesnesi oluşturan bir yapıcı. (Veri alanlarının değerleri bu zamanı temsil edecektir.)

- Belirtilen saat, dakika ve

saniye ile bir Time nesnesi oluşturan bir yapıcı.

- Sırasıyla `hour`, `minute` ve `second` veri alanları için üç `get` yöntemi.

- Geçen süreyi kullanarak nesne için yeni bir zaman ayarlayan `setTime(long elapsedTime)` adlı bir yöntem. Örneğin, geçen süre 55555000milisaniye ise, saat 10, dakika 19 ve saniye 10'dur.

UML diyagramını çizin ve ardından sınıfı uygulayın. İki `Time` nesnesi oluşturan bir test programı yazın (`new Time()` ve `new Time(55555000)` kullanarak) ve saat, dakika ve saniyeyi saat: dakika: saniye formatında görüntüleyin.

(İpucu: İlk iki yapıcı, geçen zamandan saat, dakika ve saniyeyi çıkaracaktır. Parametre almayan yapıcı için, mevcut zaman elde edilebilir) `System.currentTimeMillis()`, Şekil 2.6'da gösterildiği gibi, `ShowCurrentTime.java`.)

10.2 (BMI sınıfına aşağıdaki yeni yapıcıyı ekleyin:

```
/** Belirtilen isim, yaş, kilo,
 * ayak ve inç ile bir BMI oluşturun
 */
public BMI(String isim, int yaş, double kilo, double ayak,
            double inç)
```

10.3 (MyInteger sınıfını) MyInteger adında bir sınıf tasarlayın. Sınıf şunları içerir:

- Bir `int` veri alanı adı `value` olan ve bu

nesne tarafından temsil edilen `int` değerini saklayan.

- Belirtilen `int` değeri için bir `MyInteger` nesnesi oluşturan bir yapıcı.
 - Bir `get` yöntemi, `int` değerini döndürür.
- Bu nesnede bulunan değer sırasıyla çift, tek veya asal ise `true` döndüren `isEven()`, `isOdd()` ve `isPrime()` yöntemleri.
- Statik yöntemler `isEven(int)`, `isOdd(int)` ve `isPrime(int)` belirtilen `int` değerin sırasıyla çift, tek veya asal olması durumunda `true` döndürür.
- Statik yöntemler `isEven(MyInteger)`, `isOdd(MyInteger)` ve `isPrime(MyInteger)` belirtilen değerin sırasıyla çift, tek veya asal olması durumunda `true` döndürür.
- `equals(int)` ve `equals(MyInteger)` yöntemleri, bu nesnedeki değerin belirtilen değerle eşit olması durumunda `true` döndürür.
- `parseInt(char[])` yöntemi, sayısal karakterlerden oluşan bir diziyi `int` değerine dönüştürür.
- `parseInt(String)` yöntemi, bir diziyi `int` değerine dönüştürür.

UML diyagramını çizin ve ardından sınıfı uygulayın. Sınıftaki tüm yöntemleri test eden bir istemci programı yazın.

10.4 (MyPoint sınıfı) MyPoint adında bir sınıf tasarlayın, bu sınıf `x` ve `y` koordinatlarına sahip bir noktayı temsil etsin. Sınıf şunları içerir:

- `x` ve `y` koordinatlarını temsil eden veri alanları ve `get` yöntemleri.
- Bir argüman almayan yapıcı, bir nokta oluşturur (0,0).
- Belirtilen koordinatlarla bir nokta oluşturan bir yapıcı.
- İki `get` yöntemi veri alanları `x` ve `y` için, sırasıyla.



VideoNotu

MyPoint sınıfı

- Bu noktadan başka bir MyPoint türündeki noktaya olan mesafeyi döndüren distance adında bir yöntem.
- Bu noktadan belirtilen x ve y koordinatlarına sahip başka bir noktaya olan mesafeyi döndüren distance adında bir yöntem.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. İki nokta (0, 0) ve (10, 30.5) oluşturan ve aralarındaki mesafeyi gösteren bir test programı yazın.

Bölmeler 10.7-10.11

***10.5** (Asal çarpanları gösterme) Kullanıcıdan pozitif bir tam sayı girmesini isteyen ve tüm en küçük çarpanlarını azalan sırayla gösteren bir program yazın. Örneğin, tam sayı 120 ise, en küçük çarpanlar 5, 3, 2, 2, 2 olarak gösterilir. Çarpanları saklamak için StackOfIntegers sınıfını kullanın (örneğin, 2, 2, 2, 3, 5) ve bunları ters sırayla alıp gösterin.

***10.6** (Asal sayıları gösterme) 120'den küçük tüm asal sayıları azalan sırayla gösteren bir program yazın. Asal sayıları saklamak için StackOfIntegers sınıfını kullanın (örneğin, 2, 3, 5, . . .) ve bunları ters sırayla alıp gösterin.

**10.7(Oyun: ATM makinesi) Programlama Alıştırması

8.7'de oluşturulan Hesap sınıfını kullanarak bir ATM makinesi simüle edin. id'si 0, 1, . . . , 9 ve başlangıç bakiyesi \$100. Sistem, kullanıcının bir kimlik girmesini ister. Eğer kimlik yanlış girilirse, kullanıcıdan doğru bir kimlik girmesi istenir. Bir kimlik kabul edildiğinde, ana menü, örnek çalışmada gösterildiği gibi görüntülenir. Mevcut bakiyeyi görüntülemek için 1, para çekmek için 2, para yatırmak için 3 ve ana menüden çıkmak için 4 seçeneğini girebilirsiniz. Çıktıktan sonra, sistem tekrar bir kimlik istemektedir. Böylece, sistem başladığında durmayacaktır.

Bir kimlik girin: 4

Ana menü
1: bakiye kontrolü
2: para çekme
3: depo
4: çıkış
Bir seçim yapın: 1
Bakiyeniz 100.0

Ana menü
1: bakiye kontrol et
2: para çek
3: depo
4: çıkış
Bir seçim yapın: 2
Çekilecek miktarı girin: 3

Ana menü
1: bakiye kontrolü
2: para çekme
3: depo
4: çıkış
Bir seçim yapın: 1
Bakiyeniz 97.0



Ana menü
 1: bakiye kontrolü
 2: para çekme
 3: depo
 4: çıkış
 Bir seçim yapın:
 Yatırılacak bir miktar girin:

Ana menü
 1: bakiye kontrolü
 2: para çekme
 3: depo
 4: çıkış
 Bir seçim yapın:
 Bakiyeniz 107.0

Ana menü
 1: bakiye kontrol et
 2: para çek
 3: depo
 4: çıkış
 Bir seçim yapın:
 Bir kimlik girin:

***10.8 (Finansal: Vergi sınıfı) Programlama Egzersizi 7.12, dizileri kullanarak vergi hesaplamak için bir program yazar. Aşağıdaki örnek veri alanlarını içeren bir **Vergi** adlı sınıf tasarlayın:

- **int filingStatus**: Dört vergi beyannamesi durumundan biri: **0**—bireysel beyan, **1**—birlikte beyan eden evli veya nitelikli dul(erkek), **2**—ayrı beyan eden evli, ve **3**—hane reisi. Kamuya açık statik sabitleri **SINGLE_FILER(0)**, **MARRIED_JOINTLY_OR_QUALIFYING_WIDOW(ER)** kullanın. **(1)**, **EVLİ_AYRI(2)**, **HANEBAŞI(3)** durumları temsil etmek için.
- **int[] bracketler**: Her vergi durumu için vergi braketlerini saklar.
- **double[] oranlar**: Her braket için vergi oranlarını saklar.
- **double vergilendirilebilirGelir**: Vergilendirilebilir geliri saklar.

Her veri alanı için **get** ve **set** yöntemlerini ve vergi döndüren **getTax()** yöntemini sağlayın. Ayrıca, argüman almayan bir yapıcı ve **Tax(filingStatus, brackets, rates, taxableIncome)** yapıcısını sağlayın.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. **Tax** sınıfını kullanarak 2001 ve 2009 vergi tablolarını \$50,000 ile \$60,000 arasındaki vergilendirilebilir gelir için \$1,000 aralıklarla yazdıran bir test programı yazın. 2009 yılı için vergi oranları Tablo 3.2'de verilmiştir. 2001 yılı için vergi oranları

Tablo 10.1'de gösterilmektedir.

**10.9 (Kurs sınıfı) Kurs sınıfını aşağıdaki gibi gözden geçirin:

- Dizi boyutu Liste 10.6'da sabittir. Mevcut dizinin içeriğini kopyalayarak yeni daha büyük bir dizi oluşturarak otomatik olarak dizi boyutunu artıracak şekilde geliştirin.
 - **dropStudent** yöntemini uygulayın.
 - Yeni bir **clear()** yöntemi ekleyin, bu yöntem kursdan tüm öğrencileri kaldırır.
- Bir kurs oluşturan, üç öğrenci ekleyen, birini çıkaran ve

kurstaki öğrencileri gösteren bir test programı yazın.

TABLO 10.1 2001 Amerika Birleşik Devletleri Federal Kişisel Vergi Oranları

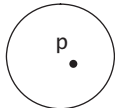
Vergi oranı	Tek dosya verenler	Evlilikte birlikte dosya verenler or nitelikli dul (dul erkek)	Evlilikte ayrı dosya verenler	Hane reisi
15%	\$27,050'a kadar	\$45,200'a kadar	22,600 \$'a kadar	36,250 \$'a kadar
%27,5	27,051 \$–65,550 \$	\$45,201–\$109,250	\$22,601–\$54,625	\$36,251–\$93,650
30.5%	\$65,551–\$136,750	\$109,251–\$166,500	\$54,626–\$83,250	\$93,651–\$151,650
35.5%	\$136,751–\$297,350	\$166,501–\$297,350	\$83,251–\$148,675	151,651 \$–297,350 \$
%39,1	297,351 \$ veya daha fazla	297,351 \$ veya daha fazla	148,676 \$ veya daha fazla	297,351 \$ veya daha fazla

* **10.10** (Oyun: **GuessDate** sınıfı) Listing 10.10'daki **GuessDate** sınıfını değiştirin. Tarihleri üç boyutlu bir dizide temsil etmek yerine, beş iki boyutlu dizi kullanarak beş sayı kümesini temsil edin. Böylece, şunları tanımlamanız gerekir:

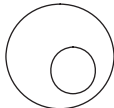
```
özel statik int[][] set1 = {{t1t, 3, 5, 7t}, ... };
özel statik int[][] set2 = {{t2t,t3t,t6t,t7t}, ... };
özel statik int[][] set3 = {{t4t,t5t,t6t,t7t}, ... };
özel statik int[][] set4 = {{t8t,t9t,t10t,t11t}, ... };
özel statik int[][] set5 = {{t16t,t17t,t18t,t19t}, ... };
```

* **10.11** (Geometri: **Circle2D** sınıfı) Aşağıdakileri içeren **Circle2D** sınıfını tanımlayın:

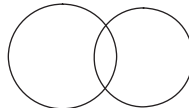
- İki **double** veri alanı **x** ve **y**, dairenin merkezini belirler ve **get** yöntemleri vardır.
- Bir veri alanı **radius** ile bir **get** yöntemi.
- **(0,0)** için **(x,y)** ve **1** için **radius** oluşturan bir argüman almayan yapıcı.
- Belirtilen **x,y** ve **radius** ile bir daire oluşturan bir yapıcı.
- Dairenin alanını döndüren bir **getArea()** yöntemi.
- Dairenin çevresini döndüren bir **getPerimeter()** yöntemi.
- Belirtilen nokta **(x, y)** bu dairenin içindeyse **true** döndüren bir **contains(double x, double y)** yöntemi (bkz. Şekil 10.15a).
- Belirtilen daire bu dairenin içindeyse **true** döndüren bir **contains(Circle2D circle)** yöntemi (bkz. Şekil 10.15b).
- Belirtilen daire bu daire ile örtüşüyorsa **true** döndüren bir **overlaps(Circle2D circle)** yöntemi (bkz. Şekil 10.15c).



(a)



(b)



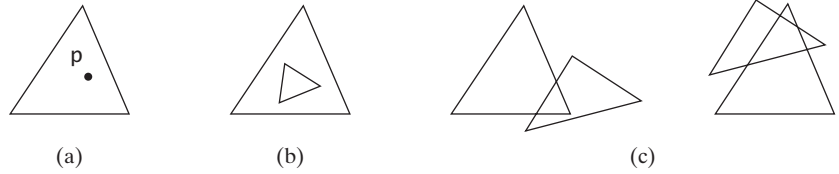
(c)

FIGÜRÜNTÜ 10.15 (a) Bir nokta dairenin içindedir. (b) Bir daire başka bir dairenin içindedir. (c) Bir daire başka bir daireyi örtmektedir.

UML diyagramını çizin ve ardından sınıfı uygulayın. Bir testprogramı yazın, bir **Circle2D** nesnesi **c1** oluşturun (**new Circle2D(2, 2, 5.5)**), alanını ve çevresini gösterin, ve **c1.contains(3, 3)**, **c1.contains(new Circle2D(4, 5, 10.5))** ve **c1.overlaps(new Circle2D(3, 5, 2.3))** sonuçlarını gösterin.

*** 10.12 (Geometri: **Triangle2D** sınıfı) Aşağıdakileri içeren **Triangle2D** sınıfını tanımlayın:

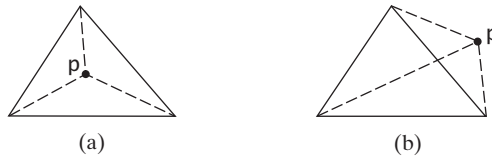
- MyPoint türünde p1, p2 ve p3 adında üç nokta, get ve set yöntemleri ile **MyPoint**, Alıştırma 10.4'te tanımlanmıştır.
- Varsayılan bir üçgen oluşturan, (0,0), (1,1) ve (2,5) noktaları ile bir argüman almayan yapıcı.
- Belirtilen noktalarla bir üçgen oluşturan bir yapıcı.
- Üçgenin alanını döndüren **getArea()** yöntemi.
- Üçgenin çevresini döndüren **getPerimeter()** yöntemi.
- Belirtilen nokta p'nin bu üçgenin içinde olup olmadığını döndüren **contains(MyPoint p)** yöntemi; eğer içindeyse true döner (bkz. Şekil 10.16a).
- Bir yöntem **contains(Triangle2D t)** belirtilen üçgenin bu üçgenin içinde olup olmadığını kontrol eder ve **true** döner (bkz. Şekil 10.16b).
- Bir yöntem **overlaps(Triangle2D t)** belirtilen üçgenin bu üçgenle örtüşüp örtüşmediğini kontrol eder ve **true** döner (bkz. Şekil 10.16c).



ŞEKİL 10.16(a) Bir nokta üçgenin içindedir. (b) Bir üçgen başka bir üçgenin içindedir. (c) Bir üçgen başka bir üçgenle örtüşmektedir.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. **Triangle2D** nesneleri **t1** oluşturmak için `new Triangle2D(new MyPoint(2.5, 2), new MyPoint(4.2, 3), new MyPoint(5.3, 5))` yapıcısını kullanarak bir test programı yazın, alanını ve çevresini gösterin ve **t1.contains(3, 3), t1.contains(new Triangle2D(new MyPoint(2.9, 2), new MyPoint(4, 1), MyPoint(1, 3.4)))**, ve **t1.overlaps(new Triangle2D(new MyPoint(2, 5.5), new MyPoint(4, -3), MyPoint(2, 6.5)))**.

(İpucu: Bir üçgenin alanını hesaplamak için formül için, Programlama Alıştırması 2.1 5'e bakın. **java.awt.geo.Line2D** sınıfını **contains** ve **overlaps** yöntemlerini uygulamak için Java API'sinde kullanın. **Line2D** sınıfı, iki doğru parçasının kesişip kesişmediğini ve bir doğrunun bir noktayı içerip içermediğini kontrol etmek için yöntemler içerir, vb. Daha fazla bilgi için lütfen Java API'sine bakın. Bir noktanın bir üçgenin içinde olup olmadığını tespit etmek için, Şekil 10.17'de gösterildiği gibi üç kesik çizgi çizin. Nokta bir üçgenin içindeyse, her kesik çizgi bir kenarla yalnızca bir kez kesişmelidir. Eğer bir kesik çizgi bir kenarı iki kez kesiyorsa, o zaman nokta üçgenin dışında olmalıdır.)



ŞEKİL 10.17(a) Bir nokta üçgenin içindedir. (b) Bir nokta üçgenin dışındadır.

*** 10.13** (Geometri: **MyRectangle2D** sınıfı) **MyRectangle2D** sınıfını tanımlayın ve bu sınıf şunları içersin:

- İki **double** veri alanı **x** ve **y** olan ve dikdörtgenin merkezini belirtene **ve** set **yöntemleri ile**. (Dikdörtgen kenarlarının **x-veya-y** eksenlerine paralel olduğunu varsayın.)
- Veri alanları **width** ve **height** ile **get** ve **set** yöntemleri.
- (0, 0) için (x, y) ve her ikisi için 1 olan varsayılan bir dikdörtgen oluşturan bir argüman almayan yapıcı.
- Belirtilen **x,y,width** ve **height** ile bir dikdörtgen oluşturan bir yapıcı.
- Dikdörtgenin alanını döndüren **getArea()** yöntemi.
- Dikdörtgenin çevresini döndüren **getPerimeter()** yöntemi.
- Belirtilen nokta (x, y) bu dikdörtgenin içindeyse **true** döndüren **contains(double x, double y)** yöntemi. (Bkz. Şekil 10.18a.)
- Belirtilen dikdörtgen bu dikdörtgenin içindeyse **true** döndüren **contains(MyRectangle2D r)** yöntemi. (Bkz. Şekil 10.18b.)
- Belirtilen dikdörtgen bu dikdörtgenle örtüşüyorsa **true** döndüren **overlaps(MyRectangle2D r)** yöntemi. (Bkz. Şekil 10.18c.)

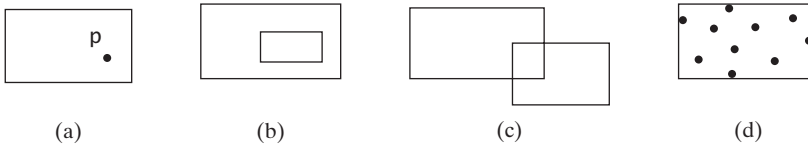


FIGURE 10.18 Bir nokta dikdörtgenin içindedir. (b) Bir dikdörtgen başka bir dikdörtgenin içindedir. (c) Bir dikdörtgen başka bir dikdörtgenle örtüşüyor. (d) Noktalar bir dikdörtgenin içinde yer alıyor.

UML diyagramını çizin ve ardından sınıfı uygulayın. Bir test programı yazın ve bir **MyRectangle2D** nesnesi **r1** oluşturun (**new MyRectangle2D(2, 2, 5.5, 4.9)**), alanını ve çevresini gösterin ve **r1.contains(3, 3)**, **r1.contains(new MyRectangle2D(4, 5, 10.5, 3.2))** ve **r1.overlaps(new MyRectangle2D(3, 5, 2.3, 5.4))** sonuçlarını gösterin.

*** 10.14** (**MyDate** sınıfı) **MyDate** adında bir sınıf tasarlayın. Sınıf şunları içerir:

- Bir tarihi temsil eden **yıl**, **ay** ve **gün** veri alanları. **ay** 0 tabanlıdır, yani, 0 Ocak içindir.
- Geçerli tarih için bir **MyDate** nesnesi oluşturan bir argüman almayan yapıcı.
- 1 Ocak 1970'ten itibaren geçen süreyi milisaniye cinsinden belirten bir **MyDate** nesnesi oluşturan bir yapıcı.
- Belirtilen yıl, ay ve gün ile bir **MyDate** nesnesi oluşturan bir yapıcı.
- Sırasıyla **yıl**, **ay** ve **gün** veri alanları için üç **get** yöntemi.
- Geçen süreyi kullanarak nesne için yeni bir tarih ayarlayan **setDate(long elapsedTime)** adında bir yöntem.

Sınıf için UML diyagramını çizin ve ardından sınıfı uygulayın. İki **MyDate** nesnesi oluşturan bir test programı yazın (**new MyDate()** ve **new MyDate(34355555133101L)** kullanarak) ve bunların yıl, ay ve gününü gösterin.

(İpucu: İlk iki yapıcı, geçen süreden yılı, ayı ve günü çıkaracaktır. Örneğin, geçen süre **561555550000** milisaniye ise, yıl

1987, ay 9, ve gün 18. Kodlamayı basitleştirmek için **GregorianCalendar** sınıfını Programlama Egzersizi 8.5'te tartışıldığı gibi kullanabilirsiniz.)

- *10.15** (*Geometri: sınırlayıcı dikdörtgeni bulma*) Sınırlayıcı bir dikdörtgen, iki boyutlu bir düzlemde bir dizi noktayı çevreleyen en küçük dikdörtgendir, Şekil 10.18'de gösterildiği gibi. Aşağıdaki gibi, dikdörtgeni döndüren bir yöntem yazın:

```
public static MyRectangle2D getRectangle(double[][] puan)
```

Rectangle2D sınıfı Egzersiz 10.13'te tanımlanmıştır. Kullanıcıdan beş nokta girmesini isteyen sınırlayıcı dikdörtgenin merkezini, genişliğini ve yüksekliğini gösteren bir test programı yazın. İşte bir örnek çalışma:



```
Beş nokta girin: 1.0 2.5 3 4 5 6 7 8 9 10 Enter
Sınırlayıcı dikdörtgenin merkezi (5.0, 6.25), genişlik 8.0, yükseklik 7.5
```

Bölümler 10.12–10.14

- *10.16** (2 veya 3 ile bölünebilir) 50 ondalık basamağa sahip olan ve 2 veya 3 ile bölünebilen ilk on sayıyı bulun.

- *10.17** (Kare sayılar) İlk on kare sayıyı bulun ki bunlar

Long.MAX_VALUE. Kare sayısı, n²

- *10.18** (Büyük asal sayılar) Beş asal sayı bulan bir program yazın, **Uzun.MAX_VALUE**.

- *10.19** (Mersenne asal sayısı) Bir asal sayıya *Mersenne asal sayısı* denir eğer

şu şekilde yazılabilir: $2^p - 1$ pozitif bir tam sayı p için. Tüm

Mersenne asal sayılarını bulup $p \leq 100$ ve çıktıyı aşağıda gösterildiği gibi görüntüleyen bir program yazın. (İpucu:

Sayının çok büyük olduğu için **long** içinde saklanamayacağından **BigInteger** kullanmalısınız. Programınızın çalışması birkaç saat sürebilir.)

$2^p - 1$

3	7
5	31

...

- *10.20** (Yaklaşık e) Programlama Egzersizi 4.26, e 'yi aşağıdaki

seriyi kullanarak yaklaşık olarak hesaplar:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots + \frac{1}{i!}$$

Daha iyi bir hassasiyet elde etmek için, hesaplamada 25 basamak hassasiyeti ile **BigDecimal** kullanın. $i = 100$ $= 100,200, \dots$, ve 1000.

- 10.21** (5 veya 6 ile bölünebilir) İlk on sayıyı bulun (**Long.MAX_VALUE**'dan büyük) 5 veya 6 ile bölünebilen.

BenKALITIM VE POLİMORFİZM

Hedefler

- Bir süper sınıftan alt sınıf tanımlamak (§11.2).
- Süper sınıfın yapıcılarını ve yöntemlerini **super** anahtar kelimesi ile çağırmak (§11.3).
- Alt sınıfta örnek yöntemlerini geçersiz kılmak (§11.4).
- Geçersiz kılma ile aşırı yükleme arasındaki farkları ayırt etmek (§11.5).
- **Object** sınıfındaki **toString()** yöntemini keşfetmek (§11.6).
- Polimorfizmi ve dinamik bağlamayı keşfetmek (§§11.7–11.8).
- Casting'i tanımlamak ve neden açık aşağı casting'in gerekli olduğunu açıklamak (§11.9).
- **Object** sınıfındaki **equals** yöntemini keşfetmek (§11.10).
- **ArrayList** içinde nesneleri depolamak, almak ve manipüle etmek (§11.11).
- **ArrayList** kullanarak bir **Stack** sınıfı uygulamak (§11.12).
- Üst sınıftaki verilerin ve yöntemlerin alt sınıflardan erişilebilir olmasını sağlamak **protected** görünürlük belirleyicisini kullanarak (§11.13).
- Sınıf uzantısını ve yöntem geçersiz kılmayı **final** belirleyicisini kullanarak önlemek (§11.14).



11.1 Giriş



Nesne yönelimli programlama, mevcut sınıflardan yeni sınıflar tanımlamanıza olanak tanır. Bu, kalıtım olarak adlandırılır.

miras alma

Kitabın önceki bölümlerinde tartışıldığı gibi, prosedürel paradigma yöntemleri tasarlamaya odaklanırken nesne yönelimli paradigma verileri ve yöntemleri nesneler içinde bir araya getirir. Nesne yönelimli paradigma kullanarak yazılım tasarımı nesnelere ve nesneler üzerindeki işlemlere odaklanır. Nesne yönelimli yaklaşım, prosedürel paradigmanın gücünü, verileri işlemlerle birleştiren ek bir boyutla birleştirir.

neden miras alma?

Kalıtım, yazılımı yeniden kullanmak için önemli ve güçlü bir özelliktir. Daireler, dikdörtgenler ve üçgenler modellemek için sınıflar tanımlamanız gerektiğini varsayalım. Bu sınıfların birçok ortak özelliği vardır. Bu sınıfları tasarlamamanın en iyi yolu, tekrarları önlemek ve sistemi anlaması kolay ve bakımının kolay olmasını sağlamak için nedir? Cevap kalıtım kullanmaktır.

11.2 Üst Sınıflar ve Alt Sınıflar



Kalıtım, genel bir sınıf (örneğin, bir üst sınıf) tanımlamanıza ve daha sonra bunu daha özel sınıflara (örneğin, alt sınıflara) genişletmenize olanak tanır.

Bir sınıfı aynı türdeki nesneleri modellemek için kullanırsınız. Farklı sınıflar bazı ortak özelliklere ve davranışlara sahip olabilir, bu da diğer sınıflar tarafından paylaşılabilecek bir sınıfta genelleştirilebilir. Genel sınıfı genişleten özel bir sınıf tanımlayabilirsiniz. Özel sınıflar, genel sınıftan özellikleri ve yöntemleri miras alır.

Geometrik nesneleri düşünün. Geometrik nesneleri modellemek için sınıfları tasarlamak istediğini varsayalım. Geometrik nesnelerin birçok ortak özelliği ve davranışı vardır. Belirli bir renkte çizilebilirler ve dolu veya boş olabilirler. Bu nedenle, tüm geometrik nesneleri modellemek için genel bir sınıf **GeometricObject**, tüm geometrik nesneleri modellemek için kullanılabilir. Bu sınıf, özellikle **color** ve **filled** ve bunların uygun **get** ve **set** yöntemlerini içerir. Bu sınıf ayrıca **dateCreated** özelliğini ve **getDateCreated()** ve **toString()** yöntemlerini içerdiğini varsayalım. **toString()** yöntemi nesnenin bir dize temsilini döndürür. Bir daire, özel bir geometrik nesne türü olduğundan, diğer geometrik nesnelerle ortak özellikler ve yöntemler paylaşır. Bu nedenle, **GeometricObject** sınıfını genişleten **Circle** sınıfını tanımlamak mantıklıdır. Benzer şekilde, **Rectangle** da **GeometricObject**'un bir alt sınıfı olarak tanımlanabilir. Şekil 11.1, bu sınıflar arasındaki ilişkiyi göstermektedir. Üst sınıfa işaret eden üçgen bir ok, iki sınıf arasındaki miras ilişkisini belirtmek için kullanılır.

Java terminolojisinde, başka bir sınıftan C1 olarak genişletilen bir sınıfa alt sınıf denir ve C2'ye üst sınıf denir. Üst sınıf, ayrıca ebeveyn sınıf veya temel sınıf olarak da adlandırılır ve alt sınıf, çocuk sınıf, genişletilmiş sınıf veya türetilmiş sınıf olarak adlandırılır. Bir alt sınıf, üst sınıftan erişilebilir veri alanlarını ve yöntemlerini miras alır ve ayrıca yeni veri alanları ve yöntemler ekleyebilir.

Circle sınıfı, **GeometricObject** sınıfından erişilebilir tüm veri alanlarını ve yöntemleri miras alır. Ek olarak, yeni bir veri alanı olan **radius**'a ve ona bağlı **get** ve **set** yöntemlerine sahiptir. **Circle** sınıfı ayrıca dairenin alanını, çevresini ve çapını döndüren **getArea()**, **getPerimeter()** ve **getDiameter()** yöntemlerini de içerir.

Dikdörtgen sınıfı, **GeometricNesne** sınıfından erişilebilir tüm veri alanlarını ve yöntemleri miras alır. Ayrıca, genişlik ve yükseklik veri alanlarına ve bunların ilişkili **get** ve **set** yöntemlerine sahiptir. **Dikdörtgen**'in alanını ve çevresini döndüren **getArea()** ve **getPerimeter()** yöntemlerini de içerir.

GeometricNesne, **Daire** ve **Dikdörtgen** sınıfları, Liste 11.1, 11.2 ve 11.3'te gösterilmiştir.

alt sınıf

üst sınıf



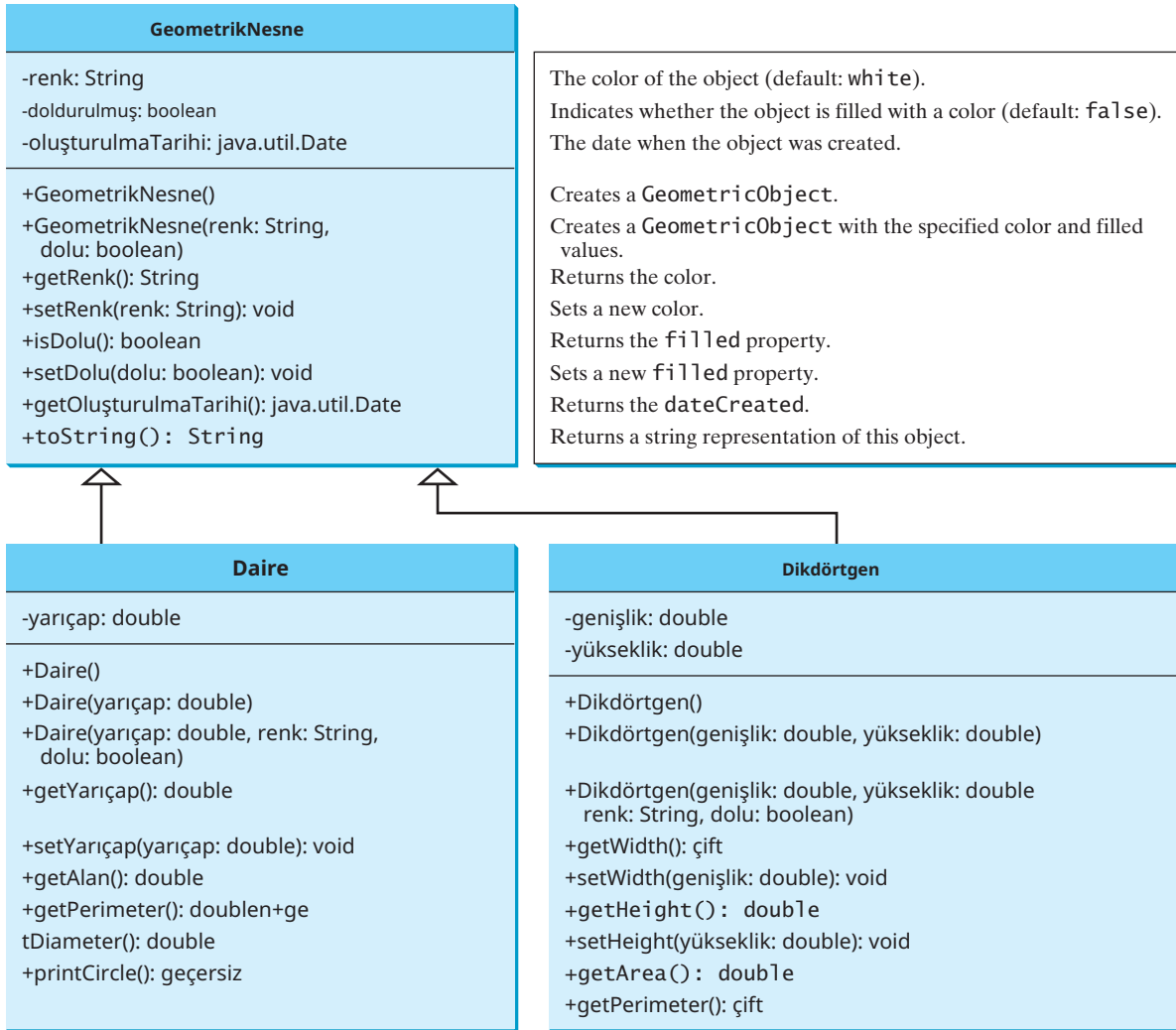
VideoNotu

Geometrik sınıf hiyerarşisi



Not

Geliştirilmiş **GeometricNesne**, **Daire** ve **Dikdörtgen** sınıfları ile isim çakışmasını önlemek için, bu sınıflara

FIGURE 11.1 `GeometricObject` sınıfı `Circle` ve `Rectangle` için süper sınıftır.

`BasitGeometrikNesne`, `BasitGeometrikNesne`den `Daire`, ve `BasitGeometrikNesne`den `Dikdörtgen` bu bölümde. Basitlik açısından, metinde hala `GeometrikNesne`, `Daire` ve `Dikdörtgen` sınıfları olarak adlandıracağız. İsim çakışmaları önlemenin en iyi yolu, bu sınıfları farklı paketlerde yerleştirmektir. Ancak, basitlik ve tutarlılık açısından, bu kitapta tüm sınıflar yerel paket içinde yer almaktadır.

LİSTELER 11.1 `BasitGeometrikNesne.java`

```

1 public class BasitGeometrikNesne {
2     private String renk = "beyaz";
3     private boolean doldurulmuş;
4     private java.util.Date oluşturulmaTarihi;
5
6     /** Varsayılan bir geometrik nesne oluştur */
7     public BasitGeometrikNesne() {
8         oluşturulmaTarihi = new java.util.Date();
9     }
10 }
  
```

veri alanları

yapıcı
inşa tarihi

410 Bölüm 11 Kalıtım ve Polimorfizm

```
9      }
1011     /** Belirtilen renkte bir geometrik nesne oluştur */
12     * ve doldurulmuş değer */
13     public SimpleGeometricObject(Dize rengi, tbooleanfilled) {
1415         oluşturulmaTarihi = new java.util.Date();
15         thist.renk = rengi;
16         thist.doldurulmuş = doldurulmuş;
17     }
1819     /** Rengi döndür */
20     public String getColor() {
21         dönüş rengi;
22 }
2324     /** Yeni bir renk ayarla */
25     public void setColor(Dize rengi) {
26         thist.renk = rengi;
27     }
2829     /** Doldurulmuş döndür. Doldurulmuş boolean olduğu için,
30     get yöntemi isFilled olarak adlandırılır */
31     genel boolean isFilled() {
32         dolu dolu;
33     }
3435     /** Yeni bir doldurulmuş ayarla */
36     genel void setFilled(tbooleanfilled) {
37         thist.doldurulmuş = doldurulmuş;
38     }
3940     /** Oluşturulma tarihini al */
41     public java.util.Date getDateCreated() {
42         return dateCreated;
43     }
4445     /** Bu nesnenin string temsilini döndür */
46     public String toString() {
47         return "oluşturulduğu tarih "+ dateCreated + "renk: " + color +
48             "ve dolu: " + filled;
49 }
50 }
```

LISTELERİ 11.2 CircleFromSimpleGeometricObject.java

üst sınıfı genişletir
data alanları

yapıcı

```
1     genel sınıf CircleFromSimpleGeometricObject
2     extends SimpleGeometricObject {
3     private double yarıçap;
4
5     public CircleFromSimpleGeometricObject() {n6 }
6
7
8     public CircleFromSimpleGeometricObject(double yarıçap) {
9         this.yarıçap = yarıçap;
10    }
11
12    public CircleFromSimpleGeometricObject(double yarıçap,
13        String renk, boolean doldurulmuş) {
14        this.yarıçap = yarıçap;
15        setColor(renk);
16    }
17 }
```

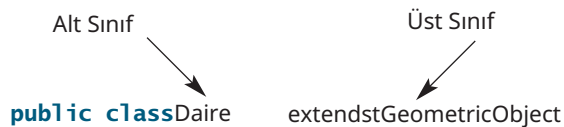
```

16 setDolu(dolu);n17 }

18
19 /** Yarıçapı döndür */
20 genel double getRadius() {
21     geri dönmek
yirmi iki }
22
23
24 /** Yeni bir yarıçap ayarla */
25 genel void setRadius(double radius) {
26     this.yarıçap = radius;
27 }
28
29 /** Alanı döndür */
30 genel double getArea() {
31     return yarıçap * yarıçap * Math.PI;
32 }
33
34 /** Çapı döndür */
35 genel double getDiameter() {
36     return 2 * yarıçap;
37 }
38
39 /** Çevreyi döndür */
40 genel double getPerimeter() {
41     return 2 * yarıçap * Math.PI;
42 }
43
44 /** Daire bilgilerini yazdır */
45 genel void printCircle() {
46     System.out.println("Çember oluşturuldu " + getDateCreated() +
47         " ve yarıçap " + radius);
48 }n49}

```

Circle sınıfı (Liste 11.2), **GeometricObject** sınıfını (Liste 11.1) aşağıdaki sözdizimini kullanarak genişletir:



Anahtar kelime **extends** (satırlar 1–2), derleyiciye **Circle** sınıfının **GeometricObject** sınıfını genişlettiğini ve böylece **getColor**, **setColor**, **isFilled**, **setFilled** ve **toString** yöntemlerini miras aldığını bildirir.

Aşırı yüklenmiş yapıcı **Circle(double radius, String color, boolean filled)**, **setColor** ve **setFilled** yöntemlerini çağırarak **color** ve **filled** özelliklerini ayarlamak için uygulanır (satırlar 12–17). Bu iki public yöntem, temel sınıf **GeometricObject** içinde tanımlanmıştır ve **Circle** içinde miras alınmıştır, bu nedenle türetilmiş

sınıfta kullanılabilirler.

Yapıcıda veri alanları **color** ve **filled** doğrudan aşağıdaki gibi kullanılmaya çalışılabilir: temel sınıftaki private üye

```

public CircleFromSimpleGeometricObject(double radius, String color, boolean filled) {

```

412 Bölüm 11 Kalıtım ve Polimorfizm

```
    this.radius = radius;
    this.color = color; // Yasa dışı
    this.filled = filled; // Yasa dışı
}
```

Bu yanlıştır, çünkü **color** ve **filled** özel veri alanları **GeometricObject** sınıfında, **GeometricObject** sınıfı dışında herhangi bir sınıfta erişilemez. **color** ve **filled** değerlerini okumak ve değiştirmek için tek yol, onların **get** ve **set** yöntemleridir.

Dikdörtgen sınıfı (Liste 11.3), GeometrikNesne sınıfını (Liste 11.1) aşağıdaki sözdizimi ni kullanarak genişletir:



extends anahtar kelimesi (satırlar 1-2), derleyiciye **Dikdörtgen** sınıfının **GeometrikNesne** sınıfını genişlettiğini, böylece **getColor**, **setColor**, **isFilled**, **setFilled** ve **toString** yöntemlerini miras aldığını söyler.

LİSTE 11.3 RectangleFromSimpleGeometricObject.java

üst sınıfı genişletir
data alanları

yapıcı

yöntemler

```
1  genel sınıfRectangleFromSimpleGeometricObject
2  extendstSimpleGeometricObject {
3      özel çift genişlik;
4      özel çift yükseklik;
5
6      publicRectangleFromSimpleGeometricObject() {n7 }
7
8
9      publicRectangleFromSimpleGeometricObject(
10         doublewidth,tdoubletheight) {
11         this.genislik = genişlik;
12         thist.yükseklik = yükseklik;
13     }
14
15     publicRectangleFromSimpleGeometricObject(
16         doublegenislik,doublehigh, String renk, booleanoldurulmuş) {
17         this.genislik = genişlik;
18         thist.yükseklik = yükseklik;
19         setColor(renk);
20         setFilled(doldurulmuş);
21     }
22
23     /** Genişliği döndür */
24     public doublegetGenislik() {
25         returngenislik;
26     }
27
28     /** Yeni bir genişlik ayarla */
29     public voidsetGenislik(doublegenislik) {
30         this.genislik = genişlik;
31     }
32 }
```

```

33  /** Yüksekliği döndür */
34  genel double getHeight() {
35      dönüş yüksekliği;
36  }
37
38  /** Yeni bir yükseklik ayarla */
39  genel void setHeight(double height) {
40      this.yükseklik = yükseklik;
41  }
42
43  /** Alanı döndür */
44  genel double getArea() {
45      dönüş genişliği * yükseklik;
46  }
47
48  /** Çevreyi döndür */
49  genel double getPerimeter() {
50      return 2 * (genişlik + yükseklik);
51 }

```

Listing 11.4'teki kod, **Circle** ve **Rectangle** nesneleri oluşturur ve bu nesneler üzerindeki yöntemleri çağırır. **toString()** yöntemi **GeometricObject** sınıfından miras alınmıştır ve bir **Circle** nesnesinden (satır 5) ve bir **Rectangle** nesnesinden (satır 13) çağrılır.

Listing 11.4: TestCircleRectangle.java

```

1  genel sınıf TestCircleRectangle {
2      public static void main(String[] args) {
3          GeometricObject çember =
4              new CircleFromSimpleGeometricObject(1.0);
5          System.out.println("Bir daire " + çember.toString());
6          System.out.println("Renk: " + çember.getColor());
7          System.out.println("Yarıçap: " + çember.getRadius());
8          System.out.println("Alan: " + çember.getArea());
9          System.out.println("Çap: " + çember.getDiameter());
10
11         GeometricObject dikdörtgen = new RectangleFromSimpleGeometricObject(2.0, 4.0);
12         System.out.println("Bir dikdörtgen " + dikdörtgen.toString());
13         System.out.println("Alanı " + dikdörtgen.getArea());
14         System.out.println("Çevresi " + dikdörtgen.getPerimeter());
15     }
16 }

```

Daire nesnesi
çağırarak toString
çağırarak getColor

Dikdörtgen nesnesi
çağırarak toString

10 Şubat 2011 Perşembe 19:54:25 EST'de oluşturulmuş bir daire
renk: beyaz ve dolu: false
Renk beyazdır
Yarıçap 1.0'dır
Alan 3.141592653589793'tür
Çap 2.0'dır
10 Şubat 2011 Perşembe 19:54:25 EST'de oluşturulmuş bir dikdörtgen
renk: beyaz ve dolu: false
Alan 8.0'dır
Çevre 12.0'dır



414 Bölüm 11 Kalıtım ve Polimorfizm

Şu noktaları miras alma ile ilgili not edin:

- alt sınıfta daha fazla
■ Geleneksel yorumun aksine, bir alt sınıf, üst sınıfının bir alt kümesi değildir. Aslında, bir alt sınıf genellikle üst sınıfından daha fazla bilgi ve yöntem içerir.
- özel veri alanları
■ Bir üst sınıftaki özel veri alanları, sınıf dışından erişilemez. Bu nedenle, bir alt sınıfta doğrudan kullanılamazlar. Ancak, üst sınıfta tanımlanmışsa, kamusal erişimciler/mutasyonlar aracılığıyla erişilebilir/değiştirilebilirler.
- genişletilemez is-a
■ Tüm is-a ilişkileri miras alma kullanılarak modellenmemelidir. Örneğin, bir kare bir diktörtgendir, ancak bir **Kare** sınıfını **Dikdörtgen** sınıfından genişletmemelisiniz, bu çünkü **genişlik** ve **yükseklik** özellikleri bir kare için uygun değildir. Bunun yerine, bir **Kare** sınıfı tanımlamalı ve **GeometrikNesne** sınıfını genişletmeli ve karenin kenarı için **kenar** özelliğini tanımlamalısınız.
- kör genişletme yok
■ Kalıtım, is-a ilişkisini modellemek için kullanılır. Yöntemleri yeniden kullanmak uğruna bir sınıfı körü körüne genişletmeyin. Örneğin, bir **Ağaç** sınıfının, boy ve kilo gibi ortak özelliklere sahip olsalar bile, bir **Kişi** sınıfını genişletmesi mantıklı değildir. Bir alt sınıf ve üst sınıf arasında is-a ilişkisi olmalıdır.
- çoklu miras
■ Bazı programlama dilleri, bir alt sınıfı birkaç sınıftan türetmenize izin verir. Bu yetenek *çoklu kalıtım* olarak bilinir. Ancak, Java çoklu kalıtıma izin vermez. Bir Java sınıfı yalnızca bir üst sınıftan doğrudan miras alabilir. Bu kısıtlama *tekil kalıtım* olarak bilinir. Bir alt sınıf tanımlamak için **extends** anahtar kelimesini kullanırsanız, yalnızca bir ebeveyn sınıfına izin verir. Bununla birlikte, çoklu kalıtım, Bölüm 15.4'te tanıtılacak olan arayüzler aracılığıyla elde edilebilir.
- tekil miras

11.1 Doğru mu yanlış mı? Bir alt sınıf, bir üst sınıfın alt kümesidir.

11.2 Bir alt sınıf tanımlamak için hangi anahtar kelimeyi kullanırsınız?

11.3 Tekil kalıtım nedir? Çoklu kalıtım nedir? Java çoklu kalıtımı destekliyor mu?

MyProgrammingLab™

Point Kontrol Et



11.3 super Anahtar Kelimesini Kullanma

super anahtar kelimesi, üst sınıfa atıfta bulunur ve üst sınıfın yöntemlerini ve yapıcılarını çağırmak için kullanılabilir.

Bir alt sınıf, üst sınıfından erişilebilir veri alanlarını ve yöntemleri miras alır. Yapıcıları miras alır mı? Üst sınıfın yapıcıları bir alt sınıftan çağrılabilir mi? Bu bölüm bu soruları ve sonuçlarını ele alır.

Bölüm 10.4, **this** Referansı, çağıran nesneyi referans almak için **this** anahtar kelimesinin kullanımını tanıttı. **super** anahtar kelimesi, **super** ifadesinin bulunduğu sınıfın üst sınıfını ifade eder. İki şekilde kullanılabilir:

- Bir üst sınıf yapıcısını çağırmak için.
- Bir üst sınıf yöntemini çağırmak için.

11.3.1 Üst Sınıf Yapıcılarını Çağıрма

Bir yapıcı, bir sınıfın bir örneğini oluşturmak için kullanılır. Özellikler ve yöntemlerin aksine, bir üst sınıfın yapıcıları bir alt sınıf tarafından miras alınmaz. Sadece **super** anahtar kelime si kullanılarak alt sınıfların yapıcılarından çağrılabilirler.

Bir üst sınıfın yapıcısını çağırmanın sözdizimi:

super(), veya **super(parametreler)**;

ifade **super()**, üst sınıfının argüman almayan yapıcısını çağırır ve ifadesuper(ar gümanlar), **argümanlar** ile eşleşen üst sınıf yapıcısını çağırır. İfade**super()** veya **super(argümanlar)**, alt sınıfın yapıcısının ilk satırında yer almalıdır; bu, bir üst sınıf yapıcısını açıkça çağırmanın tek yoludur. Örneğin, Liste 11.2'deki 12-17. satırlardaki yapıcı aşağıdaki kod ile değiştirilebilir:

```
public DaireBasitGeometrikNesne(
    double yarıçap, String renk, boolean doldurulmuş) {
    super(renk, doldurulmuş);
    this.yarıçap = yarıçap;
}
```

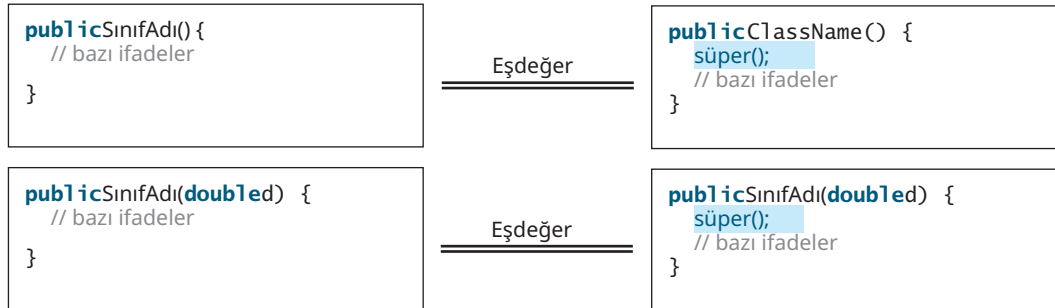


Dikkat

Üst sınıf yapıcısını çağırmak için **super** anahtar kelimesini kullanmalısınız ve çağrı, yapıcıdaki ilk ifade olmalıdır. Bir alt sınıfta üst sınıf yapıcısının adını çağırmak bir sözdizimi hatasına neden olur.

11.3.2 Yapıcı Zincirleme

Bir yapıcı, aşırı yüklenmiş bir yapıcıyı veya üst sınıf yapıcısını çağırabilir. Hiçbiri açıkça çağırılmazsa, derleyici otomatik olarak **super()** ifadesini yapıcıdaki ilk ifade olarak ekler. Örneğin:



Her durumda, bir sınıfın örneğini oluşturmak, tüm üst sınıfların yapıcılarını miras zinciri boyunca çağırır. Bir alt sınıfın nesnesini oluştururken, alt sınıf yapıcısı önce üst sınıf yapıcısını çağırır ve ardından kendi görevlerini yerine getirir. Eğer üst sınıf başka bir sınıftan türetilmişse, üst sınıf yapıcısı önce ebeveyn sınıf yapıcısını çağırır ve ardından kendi görevlerini yerine getirir. Bu süreç, miras hiyerarşisindeki son yapıcı çağrılana kadar devam eder. Buna *yapıcı zincirleme* denir.

yapıcı zincirleme

Aşağıdaki kodu dikkate alın:

```
1 public sınıf Fakülte extends Çalışan {
2     public static void main(String[] args) {
3         new Fakülte();
4     }
5
6     public Fakülte() {
```

416 Bölüm 11 Kalıtım ve Polimorfizm

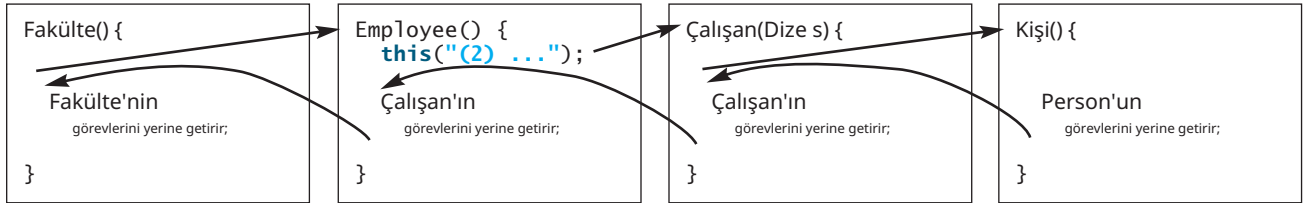
```
7      System.out.println("(4) Fakülte'nin görevlerini yerine getirir");
8  }
9  }
10
101 sınıf Çalışan extends Kişi {
12      public Çalışan() {
13          this("(2) Çalışan'ın aşırı yüklenmiş yapıcısını çağırmak");
14          System.out.println("(3) Çalışanın görevlerini yerine getirir ");
15      }
16
17      public Çalışan(String s) {
18          System.out.println(s);n19 }
19
20 }
21
22 class Kişi {
23     public Kişi() {
24         System.out.println("(1) Kişinin görevlerini yerine getirir");
25     }
26 }
```

aşırı yüklenmiş çağır
yapıcı



- (1) Kişinin görevlerini yerine getirir
- (2) Çalışanın aşırı yüklenmiş yapıcısını çağırır
- (3) Çalışanın görevlerini yerine getirir
- (4) Fakülte'nin görevlerini yerine getirir

Program yukarıdaki çıktıyı üretir. Neden? Sebebi tartışalım. 3. satırda, **new Faculty()**, **Faculty**'nin argüman almayan yapıcısını çağırır. **Faculty**, **Employee**'nin bir alt sınıfı olduğundan, **Faculty**'nin constructor'ında herhangi bir ifade çalıştırılmadan önce **Employee**'nin argüman almayan yapıcısı çağırılır. **Employee**'nin argüman almayan yapıcısı, **Employee**'nin ikinci yapıcısını (12. satır) çağırır. **Employee**, **Person**'nin bir alt sınıfı olduğundan, **Person**'nin argüman almayan yapıcısı, **Employee**'nin ikinci yapıcısında herhangi bir ifade çalıştırılmadan önce çağırılır. Bu süreç aşğıdaki şekilde gösterilmektedir.



argüman almayan yapıcı



Dikkat

Eğer bir sınıf genişletilmek üzere tasarlanmıssa, programlama hatalarını önlemek için bir argüman almayan yapıcı sağlanması daha iyidir. Aşağıdaki kodu göz önünde bulundurun:

```
1 public sınıf AppletextendstMeyve {
2     n2}
3
34 sınıf Meyve {
5     public Fruit(Dize adı) {
6         System.out.println("Fruit'in yapıcısı çağırıldı");
7     }
8 }
```


Apple'da açıkça tanımlanmış bir yapıcı olmadığı için, Apple'ın varsayılan argüman almayan yapıcısı örtük olarak tanımlanır. Apple, Fruit'ın bir alt sınıfı olduğundan, Apple'ın varsayılan yapıcısı otomatik olarak Fruit'ın argüman almayan yapıcısını çağırır. Ancak, Fruit'ın argüman almayan bir yapıcısı yoktur, çünkü Fruit'ın açık bir yapıcısı tanımlanmıştır. Bu nedenle, program derlenemez.



Tasarım Kılavuzu

Mümkünse, sınıfı genişletmeyi kolaylaştırmak ve hatalardan kaçınmak için her sınıf için bir no-arg yapıcı sağlamalısınız.

argüman almayan yapıcı

11.3.3 Üst Sınıf Yöntemlerini Çağırma

Üst sınıfta yapıcı dışında bir yöntemi referans almak için **super** anahtar kelimesi de kullanılabilir. Söz dizimi:

super.method(parametreler);

Circle sınıfındaki printCircle() yöntemini aşağıdaki gibi yeniden yazabilirsiniz:

```
genel void printCircle() {
    System.out.println("Daire oluşturuldu "+
        Harika.getDateCreated() + " ve yarıçap "+ radius);
}
```

Ancak, bu durumda **getDateCreated()** öncesinde **super** koymak gerekli değildir, çünkü **getDateCreated()** **GeometricObject** sınıfında bir yöntemdir ve **Circle** sınıfı tarafından miras alınmıştır. Yine de, bazı durumlarda, bir sonraki bölümde gösterildiği gibi, **super** anahtar kelimesine ihtiyaç vardır.

11.4 (a) sınıfını çalıştırmanın çıktısı nedir? (b) programı derlerken hangi sorun ortaya çıkıyor?

Point Kontrol Et

MyProgrammingLab™

```
sınıfA {
    genelA() {
        System.out.println(
            "A'nın argüman almayan yapıcısı çağırıldı");
    }
}

classB extends A {
}

public class C {
    public static void main(String[] args) {
        B b = new
            tB();
    }
}
```

(a)

```
sınıfA {
    public A(tinttx) {n}
}

classB extends A {
    public B() {n}
}

genel sınıfC {
    public static void main(String[] args) {
        B b = new
            tB();
    }
}
```

(b)

11.5 Bir alt sınıf, üst sınıfının yapıcısını nasıl çağırır?

11.6 Doğru mu yanlış mı? Bir alt sınıftan bir yapıcı çağırıldığında, üst sınıfın argüman almayan yapıcısı her zaman çağırılır.

11.4 Yöntemleri Geçersiz Kılma



Bir yöntemi geçersiz kılmak için, yöntem alt sınıfta üst sınıftaki ile aynı imzayı ve aynı dönüş türünü kullanarak tanımlanmalıdır.

yöntem geçersiz kılma

Bir alt sınıf, üst sınıftan yöntemleri miras alır. Bazen alt sınıfın, üst sınıfta tanımlanan bir yöntemin uygulamasını değiştirmesi gerekir. Bu duruma yöntem geçersiz kılma denir.

toString yöntemi GeometricObject sınıfında (Liste 11.1'deki 46–49. satırlar) geometrik bir nesnenin dize temsili döndürür. Bu yöntem, bir dairenin dize temsili döndürmek için geçersiz kılınabilir. Bunu geçersiz kılmak için, Liste 11.2'deki Circle sınıfına aşağıdaki yeni yöntemi ekleyin.

toString üst sınıfta

```
1 genel sınıf CircleFromSimpleGeometricObject
2     extend SimpleGeometricObject {
3         // Diğer yöntemler atlanmıştır
4
5         // Üst sınıfta tanımlanan toString yöntemini geçersiz kıl
6         public String toString() {
7             geri dönmek super.toString() + "yarıçap " + radius;8     }
9     }
```

toString() yöntemi GeometricObject sınıfında tanımlanmıştır ve Circle sınıfında değiştirilmiştir. Her iki yöntem de Circle sınıfında kullanılabilir. Circle sınıfından GeometricObject sınıfında tanımlanan toString yöntemini çağırmak için super.toString() (satır 7) kullanın.

Bir Circle alt sınıfı,

super.super.methodName() yok

GeometricObject sınıfında tanımlanan toString yöntemine super.super.toString() gibi bir sözdizimi kullanarak erişilebilir mi? Hayır. Bu bir sözdizimi hatasıdır.

Not edilmesi gereken birkaç nokta var:

erişilebilir örnek

yöntemini geçersiz kıl

■ Bir örnek yöntemi yalnızca erişilebilir olduğunda geçersiz kılınabilir. Bu nedenle, bir özel yöntem kendi sınıfının dışından erişilebilir olmadığından geçersiz kılanamaz. Bir alt sınıfta tanımlanan bir yöntem, üst sınıfta özel ise, iki yöntem tamamen ilgisizdir.

statik yöntemi geçersiz kılamaz

■ Bir örnek yöntemi gibi, bir statik yöntem de miras alınabilir. Ancak, bir statik yöntem geçersiz kılınamaz. Üst sınıfta tanımlanan bir statik yöntem, bir alt sınıfta yeniden tanımlanırsa, üst sınıfta tanımlanan yöntem gizlenir. Gizli statik yöntemler, **SuperClassName.staticMethodName** sözdizimi kullanılarak çağrılabilir.

11.7 Doğru mu yanlış mı? Üst sınıfta tanımlanan bir özel yöntemi geçersiz kılabilirsiniz.

11.8 Doğru mu yanlış mı? Üst sınıfta tanımlanan bir statik yöntemi geçersiz kılabilirsiniz.

11.9 Bir alt sınıftan üst sınıfın yapıcısını nasıl açıkça çağırırsınız?

11.10 Bir alt sınıftan geçersiz kılınmış bir üst sınıf yöntemini nasıl çağırırsınız?



MyProgrammingLab™



11.5 Geçersiz Kılma vs. Aşırı Yükleme

Aşırı yükleme, aynı ada sahip ancak farklı imzaları olan birden fazla yöntem tanımlamak anlamına gelir. Geçersiz kılma, alt sınıfta bir yöntem için yeni bir uygulama sağlamaktır.

5.8. Bölümde yöntem aşırı yüklemesini öğrendiniz. Bir yöntemi geçersiz kılmak için, yöntem alt sınıfta aynı imza ve aynı dönüş türü ile tanımlanmalıdır.

Geçersiz kılma ve aşırı yükleme arasındaki farkları göstermek için bir örnek kullanalım. Aşağıdaki (a) bölümünde, A sınıfındaki `p(double i)` yöntemi, B sınıfında tanımlanan aynı yöntemi geçersiz kılar. Ancak (b) bölümünde, A sınıfının iki aşırı yüklenmiş yöntemi vardır: `p(double i)` ve `p(int i)`. `p(double i)` yöntemi B sınıfından miras alınmıştır.

<pre> genel sınıfTest { public static void main(String[] args) {nA a = neww A(); a.p(t10t);n a.p(t10.0t);n} } sınıfB { public voidtp(tdoubleti) {nSyste m.out.println(i * t2t);n} } classAextendsB { // Bu yöntem rsiz kılmalar B'deki yöntem kamu boşluğu p(çift geçe) { System.out.println(i);n} } </pre> (a)	<pre> genel sınıfTest { public static void main(String[] args) {nA a = neww A(); a.p(t10t);n a.p(t10.0t);n} } sınıfB { public voidtp(tdoubleti) {nSyste m.out.println(i * t2t);n} } class A extends B { // Bu yöntem aşırı yükler B'deki yöntem kamu boşluğu p(int) { System.out.println(i);n} } </pre> (b)
--	---

Test sınıfını (a) içinde çalıştırdığınızda, hem `a.p(10)` hem de `a.p(10.0)` `p(double)` yöntemini çağırır. Ben) sınıfında tanımlanan yöntem `A10.0` görüntülemek için. Test sınıfını (b) çalıştırdığınızda, `a.p(10)` `p(int i)` yöntemini çağırır ve A sınıfında tanımlanan `10` görüntülemek için, `a.p(10.0)` ise `p(double i)` yöntemini çağırır ve B sınıfında tanımlanan `20.0` görüntüler.

Aşağıdakilere dikkat edin:

- Geçersiz kılınmış yöntemler, miras yoluyla ilişkili farklı sınıflardadır; aşırı yüklenmiş yöntemler ise ya aynı sınıfta ya da miras yoluyla ilişkili farklı sınıflarda olabilir.
- Geçersiz kılınmış yöntemlerin aynı imzası ve dönüş tipi vardır; aşırı yüklenmiş yöntemler ise aynı isimde sahip ancak farklı parametre listesine sahiptir.

Hatalardan kaçınmak için, *override annotation* olarak adlandırılan özel bir Java sözdizimini kullanarak, alt sınıftaki yöntemin önüne

@Override ekleyebilirsiniz. Örneğin:

override notasyonu

```

1 genel sınıfCircleFromSimpleGeometricObject
2     extendsSimpleGeometricObject {
3     // Diğer yöntemler atlanmıştır
4
5     @Geçersiz kıl
6     publicString toString() {
7         return super.toString() + "yarıçap "+ yarıçap;
8     }metin
9 }

```

toString üst sınıfta

Bu anotasyon, anotasyonlu yöntemin üst sınıftaki bir yöntemi geçersiz kılmasının gerektiği ni belirtir. Bu anotasyonla işaretlenmiş bir yöntem, üst sınıfının yöntemini geçersiz kılma ya da derleyici bir hata bildirir. Örneğin, `toString` yanlış yazılırsa `tostring` olarak, bir derleme hatası bildirilir. Geçersiz kılma anotasyonu kullanılmazsa, derleme hatası bildirmeyecektir. Notasyon kullanmak hataları önler.

11.11 Aşağıdaki kodda sorunları belirleyin:

```

1  genel sınıf Circle {
2      private double yarıçap;
3
4      public Circle(double yarıçap) {
5          yarıçap = yarıçap;
6      }
7
8      genel double getRadius() {
9          return yarıçap;
10     }
11
12     genel double getArea() {
13         return yarıçap * yarıçap * Math.PI;
14     }
15 }
16
17 class B extends Circle {
18     private double uzunluk;
19
20     B(double yarıçap, double uzunluk) {
21         super(yarıçap);
22         uzunluk = uzunluk;
23     }
24
25     @Geçersiz Kıl
26     genel double getArea() {
27         return getArea() * uzunluk;
28     }
29     { eksiklik }

```

11.12 Yöntem aşırı yüklemesi ile yöntem geçersiz kılma arasındaki farkı açıklayın.

11.13 Eğer bir alt sınıftaki bir yöntem, üst sınıfındaki bir yöntemle aynı imzaya sahipse ve aynı dönüş türüne sahipse, yöntem geçersiz kılınmış mı yoksa aşırı yüklenmiş mi olur?

11.14 Eğer bir alt sınıftaki bir yöntem, üst sınıfındaki bir yöntemle aynı imzaya sahipse ancak farklı dönüş türüne sahipse, bu bir sorun olur mu?

11.15 Eğer bir alt sınıftaki bir yöntem, üst sınıfındaki bir yöntemle farklı parametre türlerine sahipse aynı isme sahipse, yöntem geçersiz kılınmış mı yoksa aşırı yüklenmiş mi olur?

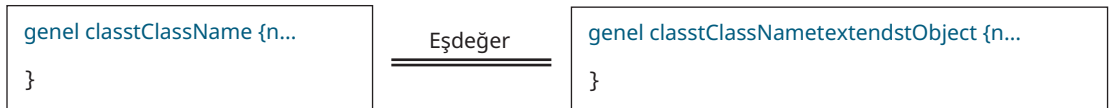
11.16 **@Override** anotasyonunu kullanmanın faydası nedir?

11.6 Nesne Sınıfı ve toString() Yöntemi



Java'daki her sınıf, **java.lang.Object** sınıfından türetilmiştir.

Bir sınıf tanımlandığında miras alınmazsa, sınıfın üst sınıfı varsayılan olarak **Object** sınıfıdır. Örneğin, aşağıdaki iki sınıf tanımı aynıdır:



String, StringBuilder, Loan ve GeometricObject gibi sınıflar, Object'nin alt sınıflarıdır (bu kitapta şimdiye kadar gördüğünüz tüm ana sınıflar gibi). Bu,

Object sınıfı tarafından sağlanan yöntemlerle tanışmak önemlidir, böylece bunları sınıflarınızda kullanabilirsiniz. Bu bölüm, **Object** sınıfındaki **toString** yöntemini tanıtmaktadır.

toString() yönteminin imzası:

toString()

public String toString()

Bir nesne üzerinde **toString()** çağırmak, nesneyi tanımlayan bir dize döndürür. Var sayılan olarak, nesnenin bir örneği olduğu sınıf adını, bir @ işaretini (@) ve nesnenin bellek adresini onaltılık olarak döndürür. Örneğin, aşağıdaki kodu Listing 10.2'de tanımlanan **Loan** sınıfı için düşünün:

dize temsili

```
Kredi kredi = new Kredi();
System.out.println(kredi.toString());
```

Bu kodun çıktısı **Loan@15037e5** gibi bir şey gösterir. Bu mesaj çok yararlı veya bilgilen dirici değildir. Genellikle, nesnenin tanımlayıcı bir dize temsili döndürmesi için **toString** yöntemini geçersiz kılmalısınız. Örneğin, **toString** yöntemi **Object** sınıfında, Listing 11.1'deki 46-49. satırlarda **GeometricObject** sınıfında aşağıdaki gibi geçersiz kılınmıştır:

```
public String toString() {
    return "oluşturulduğu tarih " + dateCreated + " renk: " + color + " ve dol
        duruldu: " + filled;
}
```



Not

Ayrıca bir nesneyi **System.out.println(object)** veya **System.out.print(object)** çağırmak için geçirebilirsiniz. Bu, **System.out.println(object.toString())** veya **System.out.print(object.toString())** çağırmakla eşdeğerdir. Böylece, **System.out.println(loan.toString())** yerine **System.out.println(kredi)t**.

nesneyi yazdır

11.7 Polimorfizm

Polimorfizm, bir süper tipin bir alt tip nesnesine referans verebileceği anlamına gelir.



Anahtar
Nokta

Nesne yönelimli programlamanın üç temel direği kapsülleme, kalıtım ve polimorfizmdir. İlk ikisini zaten öğrendiniz. Bu bölüm polimorfizmi tanıtmaktadır.

Öncelikle, iki yararlı terimi tanımlayalım: alt tip ve süper tip. Bir sınıf bir tür tanımlar. Bir alt sınıf tarafından tanımlanan bir tür alt tip olarak adlandırılır ve bir üst sınıfı tarafından tanımlanan bir tür süper tip olarak adlandırılır. Bu nedenle, **Circle**'nin **GeometricObject**'nin bir alt tipi ve **GeometricObject**'nin **Circle** için bir süper tip olduğunu söyleyebilirsiniz.

Kalıtım ilişkisi, bir alt sınıfın, üst sınıfından özellikleri miras almasını ve ek yeni özellikler kazanmasını sağlar. Bir alt sınıf, üst sınıfının bir uzmanlaşmasıdır; bir alt sınıfın her örneği, aynı zamanda üst sınıfının da bir örneğidir, ancak bunun tersi geçerli değildir. Örneğin, her daire bir geometrik nesnedir, ancak her geometrik nesne bir daire değildir. Bu nedenle, her zaman bir alt sınıfın örneğini, üst sınıf türündeki bir parametreye geçirebilirsiniz. Liste 11.5'teki kodu göz önünde bulundurun.

LİSTE 11.5 PolymorphismDemo.java

```
1 genel sınıfPolimorfizmDemo {
2     /** Ana yöntem */
3     public static voidtmain(String[] args) {
```

422 Bölüm 11 Kalıtım ve Polimorfizm

```
4 // Daire ve dikdörtgen özelliklerini göster
578 displayObject(tnewt CircleFromSimpleGeometricObject
    (1, "kırmızı", false));
polimorfik çağrı
polimorfik çağrı
displayObject(newRectangleFromSimpleGeometricObject
    (1, 1, "siyah", true));
9 }
1011 /** Geometrik nesne özelliklerini göster */
12 genel statik boşluk displayObject(SimpleGeometricObject nesnesi) {
13     System.out.println("Yaratıldı: " + object.getDateCreated() +
14         ". Renk: " + object.getColor());
15 }n16 }
```



09 Mar 2011 Pazartesi 19:25:20 EDT'de oluşturuldu. Renk beyaz
09 Mar 2011 Pazartesi 19:25:20 EDT'de oluşturuldu. Renk siyah

polimorfizm nedir?

displayObject yöntemi (satır 12), GeometricObject türünde bir parametre alır. Herhangi bir GeometricObject örneğini geçirerek displayObject çağırabilirsiniz (örneğin, new CircleFromSimpleGeometricObject(1, "red", false) ve new RectangleFromSimpleGeometricObject(1, 1, "black", false) 5-8. satırlarda). Bir alt sınıfın nesnesi, üst sınıf nesnesinin kullanıldığı her yerde kullanılabilir. Bu, genellikle polimorfizm olarak bilinir (Yunanca "birçok form" anlamına gelen bir kelimeden gelir). Basit terimlerle, polimorfizm, bir süper tipin değişkeninin bir alt tip nesnesine atıfta bulunabileceği anlamına gelir.

11.8 Dinamik Bağlama



Bir yöntem, miras zinciri boyunca birkaç sınıfta uygulanabilir. JVM, çalışma zamanında hangi yöntemin çağrılacağına karar verir.

Bir yöntem, bir üst sınıfta tanımlanabilir ve alt sınıfta geçersiz kılınabilir. Örneğin, toString() yöntemi Object sınıfında tanımlanmış ve GeometricObject sınıfında geçersiz kılınmıştır. Aşağıdaki kodu dikkate alın:

```
Object o = tnewtGeometricObject();nSystem.out.println(o.toString());
```

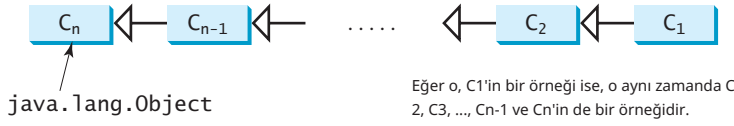
beyan edilen tür

gerçek tür

dinamik bağlama

Hangi toString() yöntemi o tarafından çağrılır? Bu soruyu yanıtlamak için önce iki terimi tanıtalım: beyan edilen tür ve gerçek tür. Bir değişken bir tür olarak beyan edilmelidir. Bir değişkeni beyan eden tür, değişkenin *beyan edilen türü* olarak adlandırılır. Burada o'nun beyan edilen türü Object'dır. Bir referans türündeki değişken, null değeri veya beyan edilen türün bir örneğine referans tutabilir. Örnek, beyan edilen türün veya alt türünün yapıcısı kullanılarak oluşturulabilir. Değişkenin *gerçek türü*, değişken tarafından referans verilen nesnenin gerçek sınıfıdır. Burada o'nun gerçek türü GeometricObject'dır, çünkü o, newGeometricObject() kullanılarak oluşturulmuş bir nesneye referans vermektedir. Hangi toString() yöntemi o tarafından çağrılır, o'nun gerçek türü tarafından belirlenir. Bu, *dinamik bağlama* olarak bilinir.

Dinamik bağlama şu şekilde çalışır: Bir nesne o, C₁, C₂, ..., C_{n-1} ve C_n sınıflarının bir örneği olsun; burada C₁, C₂'nin bir alt sınıfıdır, C₂, C₃'nün bir alt sınıfıdır, ..., ve C_{n-1}, C_n'nin bir alt sınıfıdır; bu, Şekil 11.2'de gösterilmiştir. Yani, C_n en genel sınıf, C₁ ise en özel sınıftır. Java'da, C_n Object sınıfıdır. Eğer o bir p yöntemini çağırırsa, JVM, bu sırayla C₁, C₂, ..., C_{n-1} ve C_n içinde p yönteminin uygulamasını arar, ta ki bulunana kadar. Bir uygulama bulunduğunda, arama durur ve ilk bulunan uygulama çağrılır.



ŞEKİL 11.2 Çağrılacak yöntem çalışma zamanında dinamik olarak bağlanır.

Liste 11.6, dinamik bağlamayı göstermek için bir örnek vermektedir.

LİSTE 11.6 DynamicBindingDemo.java

```

1  genel sınıf DynamicBindingDemo {
2      public static void main(String[] args) {
3          m(new MezunÖğrenci());
4          m(new Student());
5          m(new Person());
6          m(new Object());
7      }
8
9      genel statik boşluk m(Nesne x) {
10         System.out.println(x.toString());
11     }
12 }
13
14 sınıf MezunÖğrenci extends Öğrenci {
15 }
16
17 sınıf Öğrenci extends Kişi {
18     @Geçersiz Kıl
19     halk String toString() {
20         return "Öğrenci";
21     }
22 }
23
24 sınıf Kişi extends Nesne {
25     @Geçersiz Kıl
26     halk String toString() {
27         return "Kişi";
28     }
29 }

```



VideoNotu

Polimorfizm ve dinamik bağlama demosu

polimorfik çağrı

dinamik bağlama

toString() yöntemini geçersiz kılma

toString() yöntemini geçersiz kılma



Öğrenci
Öğrenci
Kişi

Java.wolf.object@130 19'dan itibaren değil

Yöntem m (satır 9) Object türünde bir parametre alır. m'yi herhangi bir nesne ile çağırabilirsiniz (örneğin, new GraduateStudent(), new Student(), new Person() ve new Object()) (satırlar 3-6).

m(Object x) yöntemi çalıştırıldığında, argüman x'in toString yöntemi çağrılır. x, GraduateStudent, Student, Person veya Object'in bir örneği olabilir. GraduateStudent, Student, Person ve Object sınıflarının kendi toString yöntemi uygulamaları vardır. Hangi uygulamanın kullanılacağı, x'in çalışma zamanındaki gerçek türü tarafından belirlenecektir. m(new GraduateStudent())'yi (satır 3) çağırarak, Student sınıfında tanımlanan toString yönteminin çağrılmasına neden olur.

m(new Student())'yi (satır 4) çağırarak, Student sınıfında tanımlanan toString yöntemini çağırır; m(new Person())'yi (satır 5) çağırarak toString yöntemini çağırır.

eşleşme vs. bağlama

tanımlanan yöntem **Person** sınıfında çağrılacak; ve **m(new Object())** (satır 6) çağrıldığında **Object** sınıfında tanımlanan **toString** yöntemi çağrılır.

Bir yöntem imzasını eşleştirmek ve bir yöntem uygulamasını bağlamak iki ayrı meseledir. Başvuru değişkeninin *bildirilen türü*, derleme zamanında hangi yöntemin eşleşeceğini belirler. Derleyici, derleme zamanında parametre türüne, parametre sayısına ve parametrelerin sırasına göre eşleşen bir yöntem bulur. Bir yöntem, miras alma zinciri boyunca birkaç sınıfta uygulanabilir. JVM, yöntemin uygulamasını çalışma zamanında dinamik olarak bağlar; bu, değişkenin gerçek türü tarafından belirlenir.

11.17 Polimorfizm nedir? Dinamik bağlama nedir?

11.18 Yöntem eşleştirme ile yöntem bağlama arasındaki farkı tanımlayın.

11.19 **new int[50]**, **new Integer[50]**, **new String[50]** veya **new**

Object[50]'yi **Object[]** türündeki bir değişkene atayabilir misiniz?

11.20 Aşağıdaki kodda ne yanlış?

```
1 genel sınıfTest {
2     public static voidtmain(String[] args) {
3 Tamsayı[] liste1 = {t12, 24, 55, 1t};n4 Çift[] liste2 = {
t12.4, 24.0, 55.2, 1.0t};
5     int[] liste3 = {t1, 2, 3t};
n Braintree(liste1);n Braintree(li
ste);n Braintree(liste);metin }
```

```
1011     public static voidtprintArray(Object[] list) {
12     fort(Nesne o: liste)
13 System.out.print(o + t" "t);n14 System.out.
println();
15 }n16
}
```

11.21 Aşağıdaki kodun çıktısını göster:

```
genel sınıfTest {
    public static voidtmain(String[] args) {nnewtPerso
n().printPerson();
    newtStudent().printPerson();n}

    classtStudenttextendstPerson {n@Ov
erride
    publicString getInfo() {
        return"Öğrenci";
    }n}

    classPerson {
        publicString getInfo() {
            return"İnsan";
        }

        public voidtprintPerson() {nSyste
m.out.println(getInfo());n}
    }
}
```

(a)

```
genel sınıfTest {
    public static voidtmain(String[] args) {nnewtPerso
n().printPerson();
    newtStudent().printPerson();n}

    }

    classÖğrenciextendsKişi {
        özel String getInfo() {
            return"Öğrenci";
        }
    }

    classKişi {
        özel String getInfo() {
            return"Kişi";
        }

        public voidtprintPerson() {nSyste
m.out.println(getInfo());n}
    }
}
```

(b)

11.22 Aşağıdaki programın çıktısını gösterin:

```

1  genel sınıfTest {
2      public static voidtmain(String[] args) {
3  A a = tnewtA(t3t);n4 }

5  }
67
   classtAtextendstB {
8      publictA(tinttt) {
9          System.out.println("A'nın yapıcısı çağrıldı");
10     }
11 }
1213
   sınıfB {
14     publictB() {
15         System.out.println("B'nin yapıcısı çağrıldı");
16     }
17 }

```

Yeni A(3) çağrıldığında **Object**'in argüman almayan yapıcısı çağrılıyor mu?

11.9 Nesneleri Dönüştürme ve instanceof Operatörü

Bir nesne referansı başka bir nesne referansına tür dönüşümü yapılabilir. Buna

nesne dönüştürme denir.



dönüştürme nesnesi

Önceki bölümde, ifade

```
m(newÖğrenci());
```

nesneyi **new Öğrenci()**, **Object** türündeki bir parametreye atar. Bu ifade şuna eşdeğerdir

```
Object o = newÖğrenci();// İkincil dönüşüm
m(o);
```

Object o = new Öğrenci() ifadesi, ikincil dönüşüm olarak bilinir ve yasaldir çünkü bir **Öğrenci** örneği bir **Object** örneğidir.

örtük dönüştürme

Bir nesne referansını **o Student** türündeki bir değişkene atamak istediğinizi varsayalım. Aşağıdaki ifadeyi kullanarak:

```
Öğrenci b = o;
```

Bu durumda bir derleme hatası meydana gelecektir. **Object o = new Student()** ifadesi neden çalışmıyor ama **Student b = o** çalışmıyor? Sebebi, bir **Student** nesnesinin her zaman bir **Object** örneği olmasıdır, ancak bir **Object** mutlaka bir **Student** örneği değildir. **o** gerçekten bir **Student** nesnesi olduğunu görebilirsiniz, ancak derleyici bunu bilmek için yeterince akıllı değildir. Derleyiciye **o** nun bir **Student** nesnesi olduğunu söylemek için *açık tür dönüşümü* kullanın. Söz dizimi, ilkel veri türleri arasında dönüşüm yapmak için kullanılanla benzerdir. Hedef nesne türünün parantez içine alın ve dönüşüm yapılacak nesnenin önüne yerleştirin, şu şekilde:

açıkça dönüştürme

```
Öğrenci b = (Öğrenci)o; // Açıkça dönüştürme
```

Bir alt sınıfın örneğini bir üst sınıfın değişkenine dönüştürmek (yukarı dönüştürme olarak bilinir) her zaman mümkündür, çünkü bir alt sınıfın örneği her zaman üst sınıfının bir örneğidir. Bir üst sınıfın örneğini alt sınıfının değişkenine dönüştürürken (aşağı dönüştürme olarak bilinir), derleyiciye niyetinizi onaylamak için açıkça dönüştürme kullanılmalıdır.

yukarı dönüştürme

aşağı dönüştürme

426 Bölüm 11 Kalıtım ve Polimorfizm

SınıfDönüşümHatası

örneği

(AltSınıfAdı) dönüştürme notasyonu. Dönüştürmenin başarılı olması için, dönüştürülecek nesnenin alt sınıfın bir örneği olduğundan emin olmalısınız. Eğer üst sınıf nesnesi alt sınıfın bir örneği değilse, çalışma zamanında **ClassCastException** oluşur. Örneğin, bir nesne **Öğrenci** değilse, **Öğrenci** değişkenine dönüştürülemez. Bu nedenle, bir dönüştürme işlemine girmeden önce nesnenin başka bir nesnenin örneği olduğundan emin olmak iyi bir uygulamadır. Bu, **instanceof** operatörünü kullanarak gerçekleştirilebilir. Aşağıdaki kodu düşünün:

```
Object myObject = new Circle();
... // Bazı kod satırları
/** myObject bir Daire örneği ise dönüştürme işlemi yap */
if(myObject instanceof Daire) {
    System.out.println("Dairenin çapı "+
        ((Daire içine alın) nesne).getDiameter());
    ...
}
```

Casting'in neden gerekli olduğunu merak ediyor olabilirsiniz. Değişken **myObject** olarak tanımlanmıştır **Object**. Tanımlanan tür, hangi yöntemin derleme zamanında eşleşeceğini belirler. **myObject.getDiameter()** kullanmak bir derleme hatasına neden olur, çünkü **Object** sınıfında **getDiameter** yöntemi yoktur. Derleyici, **myObject.getDiameter()** için bir eşleşme bulamaz. Bu nedenle, derleyiciye **myObject**'ün aynı zamanda **Circle** türünde bir örnek olduğunu belirtmek için **myObject**'ü **Circle** türüne dönüştürmek gereklidir.

Neden **myObject**'ü baştan bir **Circle** türü olarak tanımlamıyorsunuz? Genel programlamayı sağlamak için, herhangi bir alt türün değerini kabul edebilecek bir süper tür ile bir değişken tanımlamak iyi bir uygulamadır.

küçük harf anahtar kelimeleri



Not

instanceof, Java anahtar kelimesidir. Bir Java anahtar kelimesindeki her harf küçük harfle yazılır.

dönüştürme analojisi



İpucu

Casting'i anlamaya yardımcı olmak için, meyve, elma ve portakal benzetmesini de düşünün: **Fruit** sınıfı **Apple** ve **Orange** için süper sınıf olarak. Bir elma bir meyvedir, bu nedenle her zaman **Apple** örneğini **Fruit** değişkenine güvenle atayabilirsiniz. Ancak, bir meyve mutlaka bir elma değildir, bu nedenle **Fruit** örneğini **Apple** değişkenine atamak için açık casting kullanmanız gerekir.

Liste 11.7, çok biçimliliği ve tür dönüşümünü göstermektedir. Program, iki nesne oluşturur (5-6. satırlar), bir daire ve bir dikdörtgen, ve bunları görüntülemek için **displayObject** yöntemini çağırır (9-10. satırlar). **displayObject** yöntemi, nesne bir daire ise alanı ve çapı (15. satır), nesne bir dikdörtgen ise alanı (21-22. satırlar) görüntüler.

LISTE 11.7 CastingDemo.java

```
1 genel sınıf CastingDemo {
2     /** Ana yöntem */
3     public static void main(String[] args) {
4         // İki nesne oluştur ve başlat
5         Nesne nesne1 = new CircleFromSimpleGeometricObject(t1t);
6         Nesne nesne2 = new RectangleFromSimpleGeometricObject(t1t,t1t);
7
8         // Daire ve dikdörtgeni görüntüle
9         displayObject(object1);
10        displayObject(object2);
11    }
12 }
```

```

13  /** Bir nesneyi görüntülemek için bir yöntem */
14  genel statik boşluk görüntüNesnesi(Nesne nesne) {
15      eğer (nesneörnekDaireBasitGeometrikNesne) {
16          System.out.println("Dairenin alanı "+
17              ((CircleFromSimpleGeometricObject)object).getArea());
18          System.out.println("Dairenin çapı "+
19              ((CircleFromSimpleGeometricObject)object).getDiameter());
20      }
21      aksi takdirde eğer (nesneörnek
22          DikdörtgenBasitGeometrikNesne) {
23          System.out.println(" Dikdörtgenin alanı "+
24              ((RectangleFromSimpleGeometricObject)object).getArea());
25      }
26  }

```

asla (revize e
dilmedi)

polimorfik çağrı

polimorfik çağrı

```

Dairenin alanı 3.141592653589793
Dairenin çapı 2.0
Dikdörtgenin alanı 1.0

```



görüntüNesnesi(Nesne nesne) yöntemi genel programlamanın bir örneğidir. Bu herhangi bir Nesne örneğini geçirerek çağrılabilir.

Program, **Circle** nesnesini **object1**'e ve bir **Dikdörtgen** nesnesini **object2**'ye atamak için ör-
tük dönüştürmeyi kullanır (satırlar 5-6), ardından **görüntüNesnesi** yöntemini çağırarak bu ne-
snelerin bilgilerini görüntüler (satırlar 9-10).

görüntüNesnesi yönteminde (satırlar 14-26), nesne **Circle** örneği ise nesneyi **Circle**'y-
e dönüştürmek için açık dönüştürme kullanılır ve **getArea** ve **getDiameter** yöntemleri daire-
nin alanını ve çapını görüntülemek için kullanılır.

Dönüştürme, yalnızca kaynak nesne hedef sınıfın bir örneği olduğunda yapılabilir. Program, bir dö-
nüştürme gerçekleştirmeden önce kaynak nesnenin hedef sınıfın bir örneği olduğunu doğrulamak içi-
n **instanceof** operatörünü kullanır (satır 15).

Circle'ye (satırlar 17, 19) ve **Rectangle**'ye (satır 24) açık dönüştürme gereklidir çünkü
ü **getArea** ve **getDiameter** yöntemleri **Object** sınıfında mevcut değildir.



Dikkat

Nesne üye erişim operatörü (.) tür dönüştürme operatöründen önce gelir. Dönüşümün . operatöründe
n önce yapıldığından emin olmak için parantez kullanın, örneğin

dönüştürmeden önce gelir

```
((Çember)object).getArea());
```

Bir ilkel tür değerini dönüştürmek, bir nesne referansını dönüştürmekten farklıdır. Bir ilkel tür değeri
dönüştürmek yeni bir değer döndürür. Örneğin:

```

int age = 45;
byte newAge = (int)age; // newAge'ye yeni bir değer atanır

```

Ancak, bir nesne referansını dönüştürmek yeni bir nesne oluşturmaz. Örneğin:

```

Nesne o = new Circle();
Circle c = (Circle)o; // Yeni bir nesne oluşturulmaz

```

Artık referans değişkenleri **o** ve **c** aynı nesneye işaret ediyor.

11.23

Aşağıdaki ifadeler için doğru veya yanlış belirtin:

- Bir alt sınıfın örneğini her zaman başarıyla bir üst sınıfa dönüştürebilirsiniz.
- Bir üst sınıfın örneğini her zaman başarıyla bir alt sınıfa dönüştürebilirsiniz.

11.24 **GeometricObject** ve **Circle** sınıfları için 11.1 ve 11.2 numaralı listelerde,

aşağıdaki soruları yanıtlayın:

a. Aşağıdaki Boolean ifadeleri doğru mu yoksa yanlış mı?

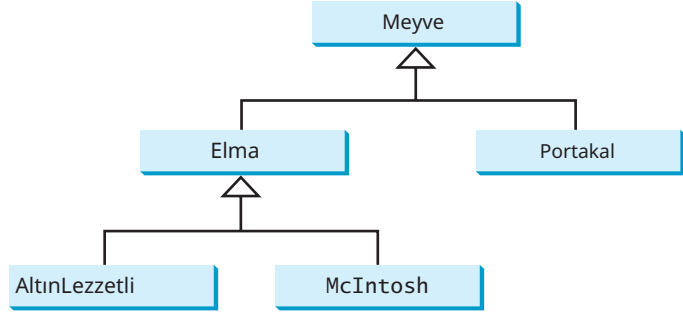
```
Daire daire = tnewtCircle(t1t);nGeome
tricObject nesnesi1 = tnewtGeometricObject();n(circletinstanc
eoftGeometricObject)
(object1tinstanceoftGeometricObject)n(circleti
nstanceoftCircle)
(nesne1örnekDaire)
```

b. Aşağıdaki ifadeler derlenebilir mi?

```
Daire daire = yeniDaire(5) ;
GeometrikNesne nesne = daire;
```

c. Aşağıdaki ifadeler derlenebilir mi?

```
GeometrikNesne nesne = yeniGeometrikNesne();
Daire daire = (Daire)nesne;
```

11.25 Varsayalım ki **Meyve**, **Elma**, **Portakal**, **GoldenDelicious** ve **McIntosh** aşağıdaki kalıtım hiyerarşisinde tanımlanmıştır:

Verilen kodun aşağıdaki gibi olduğunu varsayalım:

```
Meyve fruit = newGoldenDelicious();
Portakal orange = newOrange();
```

Aşağıdaki soruları yanıtlayın:

- a. `fruit instanceof Fruit` mı?
- b. `fruit instanceof Orange` mı?
- c. `fruit instanceof Apple` mı?
- d. `fruit instanceof GoldenDelicious` mı?
- e. `fruit instanceof McIntosh` mı?
- f. `orange instanceof Orange` mı?

- g. `orange Fruit` örneği mi?
- h. Portakal `Apple` örneği mi?
- i. `makeAppleCider` yönteminin `Apple` sınıfında tanımlandığını varsayalım. `meyve` bu yöntemi çağırabilir mi? `portakal` bu yöntemi çağırabilir mi?
- j. `makeOrangeJuice` yönteminin `Orange` sınıfında tanımlandığını varsayalım. `portakal` bu yöntemi çağırabilir mi? `meyve` bu yöntemi çağırabilir mi?
- k. `Orange p = new Apple()` ifadesi yasal mı?
- l. `McIntosh p = new Apple()` ifadesi yasal mı?
- m. `Apple p = new McIntosh()` ifadesi yasal mı?

11.26 Aşağıdaki kodda ne yanlış?

```

1  genel sınıfTest {
2      public static voidtmain(String[] args) {
3          Object meyve = newFruit();
4          Object elma = (Apple)meyve;
5      }
6  }
7
8      classtAppletextendstFruit {n9 }
9
10
11
12  classMeyve {

```

11.10 Nesne'nin eşitlik Metodu

`toString()` metoduna benzer şekilde, `equals(Object)` metodu da Nesne sınıfında tanımlanan başka bir metottur.

Nesne sınıfında sıkça kullanılan başka bir metot `equals` metodudur. İmzası



eşittir(Nesne)

```
public booleantequals(Nesne o)
```

Bu metot, iki nesnenin eşit olup olmadığını test eder. Çağırma sözdizimi:

```
nesne1.equals(nesne2);
```

Nesne sınıfındaki `equals` metodunun varsayılan uygulaması:

```
public booleantequals(Object nesnesi) {nret
    urnt(tthist== nesne);
}
```

Bu uygulama, iki referans değişkeninin aynı nesneye işaret edip etmediğini kontrol eder ve bunu `==` operatörü ile yapar. İki farklı nesnenin aynı içeriğe sahip olup olmadığını test etmek için bu yöntemi öz el sınıfınızda geçersiz kılmalısınız.

Java API'sindeki birçok sınıfta, örneğin `java.lang.String` ve `java.util.Date`, iki nesnenin içeriğinin eşit olup olmadığını karşılaştırmak için `equals` yöntemi geçersiz kılınmıştır. Bölüm 9.2'de, iki dizeyi karşılaştırmak için `equals` yöntemini zaten kullandınız, Dize Sınıfı. `String` sınıfındaki `equals` yöntemi, `Object` sınıfından miras alınmıştır ve iki dizinin içerik açısından özdeş olup olmadığını test etmek için `String` sınıfında geçersiz kılınmıştır.

430 Bölüm 11 Kalıtım ve Polimorfizm

İki dairenin yarıçapına göre eşit olup olmadığını karşılaştırmak için `Circle` sınıfında `equals` yöntemi aşağıdaki gibi geçersiz kılabilirsiniz:

```
public boolean equals(Object o) {
    if (o instanceof Circle) {
        return radius == ((Circle)o).radius;
    }
}
```

aksi takdirde

`false`'i döndür;

}

`==` televizyonlar, eşittir



Not

`==` karşılaştırma operatörü, iki ilkel veri türü değerini karşılaştırmak veya iki nesnenin aynı referanslara sahip olup olmadığını belirlemek için kullanılır. `equals` yöntemi, iki nesnenin aynı içeriğe sahip olup olmadığını test etmek için tasarlanmıştır, bu yöntemin nesnelerin tanımlayıcı sınıfında geçersiz kılındığı varsayılarak. `==` operatörü, iki referans değişkeninin aynı nesneye işaret edip etmediğini kontrol ettiğinden, `equals` yönteminden daha güçlüdür.



Dikkat

Bir alt sınıfta `equals(SomeClassName obj)` (örneğin, `equals(Circle c)`) yöntemini geçersiz kılmak yaygın bir hatadır. `equals(Object obj)` kullanmalısınız. z. Bakınız CheckPoint Soru 11.28.

eşittir (Nesne)

11.27 Her nesnenin bir `toString` yöntemi ve bir `equals` yöntemi var mı? Nereden geliyorlar? Nasıl kullanılıyorlar? Bu yöntemleri geçersiz kılmak uygun mu?

11.28 `equals` yöntemini geçersiz kılarken, yaygın bir hata alt sınıfta imzasını yanlış yazmaktır. Örneğin, `equals` yöntemi `equals(Circle circle)` olarak yanlış yazılmıştır; bunun yerine, `equals(Object circle)` olarak yazılmalıdır. SınıfTest ile (a) ve (b) deki `Circle` sınıfını çalıştırmanın çıktısını gösterin.

Point Kontrol Et

MyProgrammingLab™

```
genel sınıfTest {
    public static void main(String[] args) {
        Object circle1 = new Circle();
        Nesne çemberi = new Circle();
        System.out.println(circle1.equals(circle2));
    }
}
```

```
sınıfCircle {
    double radius;

    genel boolean equals(Daire daire) {
        return this.radius == daire.radius;
    }
}
```

(a)

```
sınıfCircle {
    double radius;

    genel boolean equals(Nesne çemberi) {
        return this.radius == ((Circle)çemberi).radius;
    }
}
```

(b)

11.11 ArrayList Sınıfı

Bir `ArrayList` nesnesi, nesnelerin bir listesini saklamak için kullanılabilir.

Artık nesneleri depolamak için çok yararlı bir sınıfı tanıtmaya hazırız. Nesneleri depolamak için bir dizi oluşturabilirsiniz. Ancak, dizi oluşturulduktan sonra boyutu sabittir. Java, `ArrayList`



VideoNotu

ArrayList sınıfı



Anahtar
Nokta

sınıf, sınırsız sayıda nesne depolamak için kullanılabilir. Şekil 11.3, ArrayList içindeki bazı yöntemleri göstermektedir.

<code>java.util.ArrayList<E></code>	
<code>+DiziList()</code> <code>+add(o: E): void</code> <code>d(index: int, o: E): void</code> <code>+clear(): void</code> <code>+içerir(o: Nesne): boolean</code> <code>+get(index: int): E</code> <code>+indexOf(o: Object): int</code> <code>+isEmpty(): boolean</code> <code>+lastIndexOf(o: Object): int</code> <code>+remove(o: Object): boolean</code> <code>+size(): int</code> <code>+remove(index: int): boolean</code> <code>+set(index: int, of: E): E</code>	Creates an empty list. Appends a new element <code>o</code> at the end of this list. Adds a new element <code>o</code> at the specified index in this list. Removes all the elements from this list. Returns true if this list contains the element <code>o</code>. Returns the element from this list at the specified index. Returns the index of the first matching element in this list. Returns true if this list contains no elements. Returns the index of the last matching element in this list. Removes the element <code>o</code> from this list. Returns the number of elements in this list. Removes the element at the specified index. Sets the element at the specified index.

FIGURE 11.3 Bir **ArrayList** sınırsız sayıda nesne depolar.

ArrayList, **E** ile genel bir tür olarak bilinen bir genel sınıftır. Bir **ArrayList** oluştururken **E**'yi değiştirmek için somut bir tür belirtebilirsiniz. Örneğin, aşağıdaki ifade bir **ArrayList** oluşturur ve referansını **cities** değişkenine atar. Bu **ArrayList** nesnesi, dizeleri depolamak için kullanılabilir.

```
ArrayList<String> şehirler = new ArrayList<String>();
```

Aşağıdaki ifade bir **ArrayList** oluşturur ve referansını **dates** değişkenine atar. Bu **ArrayList** nesnesi, tarihleri depolamak için kullanılabilir.

```
ArrayList<java.util.Date> tarihler = new ArrayList<java.util.Date> ();
```



Not

JDK 7'de, ifade

```
ArrayList<AConcreteType> list = yeni ArrayList<AConcreteType> ();
```

şu şekilde basitleştirilebilir

```
Dizi Listesi <AConcreteType> liste = new ArrayList ();
```

Somut tür, *tür çıkarsama* olarak adlandırılan bir özellik sayesinde yapıcıda artık gerekli değildir. Derleyici, türü değişken bildiriminden çıkarabilir. Özel genel sınıflar ve yöntemler tanımlama dahil olmak üzere generics hakkında daha fazla tartışma Bölüm 21, Generics'te sunulacaktır.

tip çıkarımı

Liste 11.8, nesneleri depolamak için **ArrayList** kullanma örneğini vermektedir.

LISTE 11.8 TestArrayList.java

```
1 import java.util.ArrayList;
2
```

ArrayList'i içe aktar

432 Bölüm 11 Kalıtım ve Polimorfizm

```
3 genel sınıfTestArrayList {
4     public static void main(String[] args) {
5         // Şehirleri depolamak için bir liste oluşturun
oluşturArrayList 6 ArrayList<String> şehirListesi = new ArrayList<String>();
7
8         // Listede bazı şehirler ekleyin
9         cityList.add("Londra");
eleman ekle 10         // cityList artık [Londra] içeriyor
11 cityList.add("Denver");
12         // cityList artık [Londra, Denver] içeriyor
13 cityList.add("Paris");
14         // cityList artık [Londra, Denver, Paris] içeriyor
15 cityList.add("Miami");
16         // cityList şimdi [Londra, Denver, Paris, Miami] içeriyor
17 cityList.add("Seul");
18         // İçerir [Londra, Denver, Paris, Miami, Seul]
19 cityList.add("Tokyo");
20         // İçerir [Londra, Denver, Paris, Miami, Seul, Tokyo]
21
liste boyutu 22         System.out.println("Liste boyutu? " + cityList.size());
23         System.out.println("Miami listede mi? "+
eleman içeriyor mu? 24             ntains("Miami")
25         );
26         System.out.println("Denver'in listede konumu? "
eleman indeksi 27             + st.indexOf("Denver")ncityList.co);
28         System.out.println("Liste boş mu? "+
boş mu? 29             cityList.isEmpty()ncityLi); // Yanlış yazdır
30
31         // 2. indekse yeni bir şehir ekle
32         cityList.add(2, "Xian");
33         // İçerir [Londra, Denver, Xian, Paris, Miami, Seul, Tokyo]
34
35         // Listedden bir şehri kaldır
eleman kaldır 36         cityList.remove("Miami");
37         // İçerir [Londra, Denver, Şian, Paris, Seul, Tokyo]
38
39         // 1. indeksteki bir şehri kaldır
eleman kaldır 40         cityList.remove(1);
41         // İçerir [Londra, Şian, Paris, Seul, Tokyo]
42
43         // Listede içerikleri görüntüle
toString() 44         System.out.println(cityList.toString());
45
46         // Listede içerikleri ters sırayla görüntüle
47         for(tinti = cityList.size() - t1t; i >= t0t; i )
elemanı al 48             Sistem.çıkışı.baskı( cityList.get(i) + " ");n
49             System.out.println();
50
51         // İki daire saklamak için bir liste oluşturun
oluşturArrayList 52         ArrayList<CircleFromSimpleGeometricObject> list
53             = new ArrayList<CircleFromSimpleGeometricObject>();
54
55         // İki daire ekle
56         list.add(tnewCircleFromSimpleGeometricObject(t2t));n56 list.add(tne
wtCircleFromSimpleGeometricObject(t3t));
57
58         // Listede ilk dairenin alanını göster
59         System.out.println("Dairenin alanı? "+
60             ((CircleFromSimpleGeometricObject)list.get(0)).getArea());
61     }
62 }
```




```
Liste boyutu? 6
Miami listede mi? Doğru
Denver'in listedeki yeri? 1
Liste boş mu? yanlış
[Londra, Şian, Paris, Seul, Tokyo]
Tokyo Seul Paris Şian Londra
Dairenin alanı? 12.566370614359172
```

ArrayList, java.util paketinde bulunduğundan, 1. satırda içe aktarılmıştır. Program, no-arg yapıcısını kullanarak bir dizi **ArrayList** oluşturur ve referansı cityList'e (6. satır) atar. **add** yöntemi (9-19. satırlar) dizinin sonuna dizeleri ekler. Bu nedenle, cityList.add("Londra") (9. satır) sonrasında liste şunları içerir:

ekle(Nesne)

```
[Londra]
```

cityList.add("Denver") (11. satır) sonrasında liste şunları içerir:

```
[Londra, Denver]
```

Paris, Miami, Seul ve Tokyo ekledikten sonra (13-19. satırlar), liste şunları içerir:

```
[Londra, Denver, Paris, Miami, Seul, Tokyo]
```

size() (22. satır) çağrısı, listenin mevcut boyutunu döndürür, bu da şu anda **6**'dır. **contains("Miami")** (24. satır) çağrısı, nesnenin listede olup olmadığını kontrol eder. Bu durumda, **Miami** listenin içinde olduğu için true döner. **indexOf("Denver")** (26. satır) çağrısı, Denver'in listede hangi indeksde olduğunu döndürür, bu da **1**'dir. Eğer **Denver** listede olmasaydı, **-1** dönerdi. **isEmpty()** yöntemi (28. satır) listenin boş olup olmadığını kontrol eder. Liste boş olmadığından **false** döner.

boyut()

cityList.add(2, "Şian") (31. satır) ifadesi, belirtilen indekste listeye bir nesne ekler. Bu ifadeden sonra liste şu hale gelir:

ekle(dizin, Nesne)

```
[Londra, Denver, Şian, Paris, Miami, Seul, Tokyo]
```

İfade **cityList.remove("Miami")** (satır 35) nesneyi listeden kaldırır. Bu ifadeden sonra, liste şu hale gelir

kaldır(Nesne)

```
[Londra, Denver, Şian, Paris, Seul, Tokyo]
```

İfade **cityList.remove(1)** (satır 39) belirtilen indeksteki nesneyi listeden kaldırır. Bu ifadeden sonra, liste şu hale gelir

kaldır(dizin)

```
[Londra, Şian, Paris, Seul, Tokyo]
```

Satır 43'teki ifade şuna eşittir

```
System.out.println(şehirListesi);
```

toString() yöntemi, listenin bir dize temsilini şu biçimde döndürür **e0.toString(), e1.toString(), ..., ek.toString()**, burada **e0, e1, ..., vek** listedeki öğelerdir.

get(index) yöntemi (satır 47) belirtilen indeksteki nesneyi döndürür.

getIndex()

ArrayList nesneleri diziler gibi kullanılabilir, ancak birçok fark vardır. Tablo 11.1, benzerlikleri ve farklılıklarını listelemektedir.

dizi vs. ArrayList

Bir dizi oluşturulduğunda, boyutu sabittir. Bir dizi elemanına, köşeli parantez notasyonu kullanılarak erişebilirsiniz (örneğin, **a[index]**). Bir **ArrayList** oluşturulduğunda, boyutu **0**'dır.

434 Bölüm 11 Kalıtım ve Polimorfizm

TABLO 11.1 Diziler ve ArrayList arasındaki Farklar ve Benzerlikler

İşlem	Dizi	Dizi Listesi
Bir dizi/DiziListesi oluşturma	<code>String[] a = tnewtString[t10t]</code>	<code>ArrayList<String> list = tnewtArrayList<>();</code>
Bir elemana erişim	<code>a[dizin]</code>	<code>list.get(indeks);</code>
Bir elemanı güncelleme	<code>a[indeks] = t"Londra"t;</code>	<code>list.set(index,t"Londra"t);</code>
Boyut döndürme	<code>a.uzunluk</code>	<code>liste.size();</code>
Yeni bir öğe ekleme		<code>list.add(t"Londra"t);</code>
Yeni bir öğe ekleme		<code>list.add(index,t"Londra"t);</code>
Bir öğe kaldırma		<code>list.remove(indeks);</code>
Bir öğe kaldırma		<code>list.remove(Nesne);</code>
Tüm öğeleri kaldırma		<code>liste.clear();</code>

Öge listede yoksa **get** ve **set** yöntemlerini kullanamazsınız. Bir listede öğe eklemek, insert etmek ve kaldırmak kolaydır, ancak bir dizide öğe eklemek, insert etmek ve kaldırmak oldukça karmaşıktır. Bu işlemleri gerçekleştirmek için diziyi manipüle etmek üzere kod yazmalısınız.

Diyelim ki tam sayıları saklamak için bir **ArrayList** oluşturmak istiyorsunuz. Aşağıdaki

kodla bir liste oluşturabilir misiniz?

```
ArrayList<int> list = new ArrayList<int>();
```

Hayır. Bu çalışmayacak çünkü bir **ArrayList** içinde saklanan öğeler bir nesne tipinde olmalıdır. Genel bir türü değiştirmek için **int** gibi bir ilkel veri tipi kullanamazsınız. Ancak, **Integer** nesnelerini saklamak için bir **ArrayList** oluşturabilirsiniz:

```
ArrayList<Integer> list = new ArrayList<Integer>();
```

Liste 11.9, kullanıcının bir dizi sayı girmesini isteyen ve dizideki farklı sayıları gösteren bir programı verir. Girişin **0** ile sona erdiğini varsayın ve **0** dizideki bir sayı olarak sayılmamaktadır.

LISTE 11.9 DistinctNumbers.java

```
1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 genel classtDistinctNumbers {
5     public static void tmain(String[] args) {
6 ArrayList<Tamsayı> liste = tnewtArrayList<Tamsayı>();
7
8         Scanner input = new Scanner(System.in);
9         System.out.print("Tam sayıları girin (giriş 0 ile sona erer): ");
10        int deęer;
11
12        yap{
13            deęer = input.nextInt(); // Girdiden bir deęer oku
14
15            eęer (! list.contains(deęer) && value != 0)
16                list.add(deęer); // Deęer listede yoksa ekle
17        } while(deęer != 0);
```

bir dizi listesi oluşturun

listede mi?

listeye ekle

```

1819 // Ayrı sayıları göster
20 için(int i = 0; i < size(); i++)
21 Sistem.çıkışı.baskı( list.get(i)nlist. + " "t);n
22 }
yirmüç }

```

Sayıları girin (giriş 0 ile sona erer):
Ayrı sayılar: 1 2 3 6 4 5

1 2 3 2 1 6 3 4 5 4 5 1 2 3 0

Enter



Program, Integer nesneleri için bir **ArrayList** oluşturur (satır 6) ve döngüde bir değeri tekrar tekrar o kur (satırlar 12–17). Her değer için, eğer listede yoksa (satır 15), listeye ekleyin (satır 16). Bu programı, bir **ArrayList** kullanmak yerine elemanları depolamak için bir dizi kullanarak yeniden yazabilirsiniz. Ancak, bu programı iki nedenle bir **ArrayList** kullanarak uygulamak daha basittir.

- İlk olarak, bir **ArrayList**'in boyutu esneklik, bu nedenle boyutunu önceden belirtmek zorunda değilsiniz. Bir dizi oluştururken, boyutu belirtilmelidir.
- İkincisi, **ArrayList** birçok yararlı yöntemi içerir. Örneğin, bir elemanın listede olup olmadığını **contains** yöntemi ile test edebilirsiniz. Bir dizi kullanıyorsanız, bu yöntemi uygulamak için ek kod yazmanız gerekir.

11.29 Aşağıdakileri nasıl yaparsınız?

- a. double değerleri saklamak için bir **ArrayList** oluşturmak?
- b. Bir nesneyi bir listeye eklemek?
- c. Bir nesneyi bir listenin başına eklemek?
- d. Bir listedeki nesne sayısını bulmak?
- e. Verilen bir nesneyi bir listeden çıkarmak?
- f. Listedenden son nesneyi çıkarmak?
- g. Verilen bir nesnenin bir listede olup olmadığını kontrol etmek?
- h. Belirtilen bir indeksten bir nesneyi bir listeden almak?

Point Kontrol Et
MyProgrammingLab™

11.30 Aşağıdaki koddaki hataları belirleyin.

```

ArrayList<String> list = new ArrayList<String> ();list.add(to"Den
ver"t);
list.add(t"Austin"t);nlist.
add(tnewtjava.util.Date());nString city =
list.get(t0t);
list.set(t3, "Dallas"t);nSyste
m.out.println(list.get(t3t));

```

11.31 Diyelim ki **ArrayList list** yinelenen öğeler içeriyor. Aşağıdaki kod

bu öğeyi dizi listesinden doğru bir şekilde kaldırıyor mu? Eğer değilse, kodu düzeltin.

```

fort(tintti = t0t; i < list.size(); i++)nlist.remove(ele
ment);

```

11.32 Aşağıdaki kodun neden **[1, 3]** yerine **[2, 3]** gösterdiğini açıklayın.

```

ArrayList<Integer> liste = new ArrayList<Integer>();list.add(t1t);

```

```
liste.add(2);
liste.add(3);
liste.remove(1);
System.out.println(liste);
```

11.12 Vaka Çalışması: Özel Bir Yığın Sınıfı



Bu bölüm, nesneleri tutmak için bir yığın sınıfı tasarlar.

Bölüm 10.9, [int](#) değerlerini depolamak için bir yığın sınıfı sundu. Bu bölüm, nesneleri depolamak için bir yığın sınıfı tanıtmaktadır. [Stack](#)'ı uygulamak için bir [ArrayList](#) kullanabilirsiniz; bu, Liste 11.10'da gösterilmiştir. Sınıfın UML diyagramı Şekil 11.4'te gösterilmektedir.



VideoNotu

MyStack sınıfı

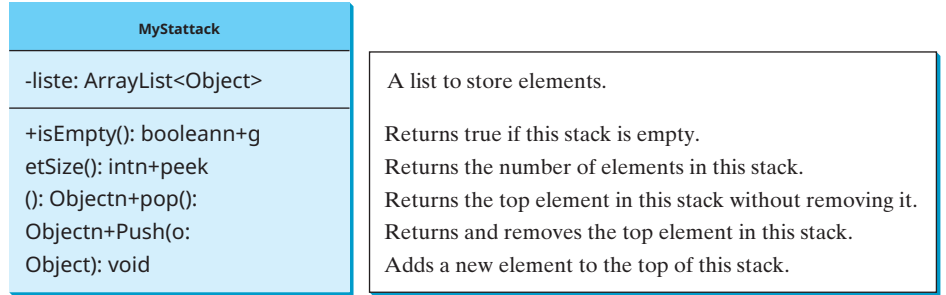


FIGURE 11.4 [MyStack](#) sınıfı yığın depolamasını kapsar ve yığınla manipüle etmek için işlemleri sağlar.

LISTING 11.10 [MyStack.java](#)

```
1  import java.util.ArrayList;
2
3  Stack {
4      özel ArrayList<Object> liste = new ArrayList<Object>();
5  }
6  genel boolean isEmpty() {
7      return liste.isEmpty();
8  }
9
10 genel int getSize() {
11     return liste.size();
12 }
13
14 public Object peek() {
15     return liste.get(liste.size() - 1);
16 }
17
18 public Object pop() {
19     Object o = liste.get(liste.size() - 1);
20     liste.remove(liste.size() - 1);
21     geri dön;
22 }
23
24 genel void push(Nesne o) {
25     liste.add(o);
26 }
27
```

dizi listesi

yığın boş mu?

yığın boyutunu al

yığın önizleme

kaldır

itme

```

28 @Geçersiz Kıl
29 public String toString() {
30     return "yığın: " + list.toString();
31 }

```

Bir dizi listesi, yığındaki elemanları depolamak için oluşturulur (satır 4). `isEmpty()` yöntemi (satırlar 6–8) `list.isEmpty()` döner. `getSize()` yöntemi (satırlar 10–12) döner `list.size()`. `peek()` yöntemi (satırlar 14–16) yığının en üstündeki elemanı kaldırmadan alır. Listenin sonu yığının tepe noktasıdır. `pop()` yöntemi (satırlar 18–22) yığından en üstteki elemanı kaldırır ve döner. `push(Object element)` yöntemi (satırlar 24–26) belirtilen elemanı yığına ekler. `toString()` yöntemi (satırlar 28–31) `Object` sınıfında tanımlanmış olup, yığının içeriğini göstermek için `list.toString()` çağrılarak geçersiz kılınmıştır. `toString()` yöntemi `ArrayList` içinde uygulanmış olup, tüm elemanların bir dizi listesindeki string temsilini döner.



Tasarım Kılavuzu

Liste 11.10'da, `MyStack` `ArrayList` içerir. `MyStack` ve `ArrayList` arasındaki ilişki *bileşim*dir. Kalıtım bir *is-a* ilişkisini modellerken, bileşim bir *has-a* ilişkisini modellemektedir. `MyStack`'i bir alt sınıf olarak da uygulayabilirsiniz `ArrayList` (bkz. Programlama Egzersizi 11.4). Ancak bileşimi kullanmak daha iyidir, çünkü bu, `ArrayList`'ten gereksiz ve uygunsuz yöntemleri miras almadan tamamen yeni bir yığın sınıfı tanımlamanıza olanak tanır.

bileşim

var

11.13 korumalı Veri ve Yöntemler

Bir sınıfın korumalı üyesi, bir alt sınıftan erişilebilir.

Şu ana kadar, veri alanlarının ve yöntemlerin sınıfın dışından erişilip erişilemeyeceğini belirtmek için `private` ve `public` anahtar kelimelerini kullandınız. Özel üyeler yalnızca sınıfın içinden erişilebilirken, genel üyeler diğer sınıflardan erişilebilir.

Sıklıkla, alt sınıfların üst sınıfta tanımlanan veri alanlarına veya yöntemlere erişmesine izin vermek istenir, ancak alt sınıf olmayanların bu veri alanlarına ve yöntemlere erişmesine izin verilmez. Bunu başarmak için `protected` anahtar kelimesini kullanabilirsiniz. Bu şekilde, bir üst sınıftaki korumalı veri alanlarına veya yöntemlere alt sınıflardan erişebilirsiniz.

`private`, `protected` ve `public` değiştiricileri, sınıfların ve sınıf üyelerinin nasıl erişildiğini belirttikleri için görünürlük veya erişilebilirlik değiştiricileri olarak bilinir. Bu değiştiricilerin görünürlüğü şu sırayla artar:

Görünürlük artar

 özel, varsayılan (değiştirici yok), korumalı, ge

Tablo 11.2, bir sınıftaki üyelerin erişilebilirliğini özetlemektedir. Şekil 11.5, sınıf `C1` içindeki bir genel, korumalı, varsayılan ve özel veri veya yöntemin, aynı paketdeki bir sınıf `C2`, aynı paketdeki bir alt sınıf `C3`, farklı bir paketdeki bir alt sınıf `C4` ve farklı bir paketdeki bir sınıf `C5` tarafından nasıl erişilebileceğini göstermektedir.

Üyeleri tamamen gizlemek için `private` belirtecini kullanın, böylece sınıf dışından doğrudan erişilemezler. Aynı paketdeki herhangi bir sınıftan doğrudan erişime izin vermek için hiçbir belirteç kullanmayın (varsayılan). Sınıf üyelerinin herhangi bir paketdeki alt sınıflar tarafından erişilmesine izin vermek için `protected` belirtecini kullanın veya aynı paketdeki sınıflar. Sınıf üyelerinin herhangi bir sınıf tarafından erişilmesine izin vermek için `public` belirtecini kullanın.

Sınıfınız iki şekilde kullanılabilir: (1) sınıfın örneklerini oluşturmak için ve (2) sınıfı genişletmek için alt sınıfları tanımlamak için. Üyeleri `private` yapın eğer kullanıcılar için tasarlanmamışlarsa.



neden `protected`?

Tablo 11.2 Veri ve Yöntemlerin Görünürlüğü

Bir sınıfta ki üyelere ait değiştirici	Aynı sınıftan erişildi	Aynı paketten erişildi	Farklı bir paketten bir alt sınıftan erişildi	Farklı bir paketten erişildi
halk	✓	✓	✓	✓
korumalı	✓	✓	✓	–
varsayılan (değiştirici yok)	✓	✓	–	–
özel	✓	–	–	–

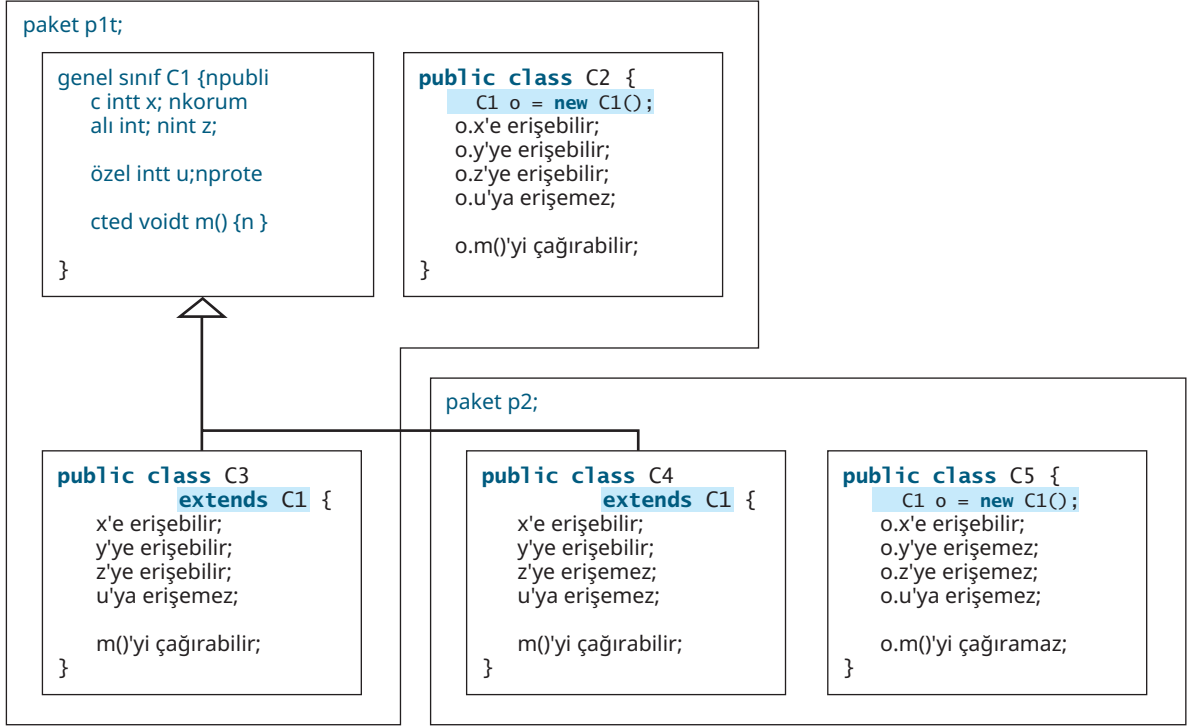


FIGURE 11.5 Görünürlük değiştiricileri, verilerin ve yöntemlerin nasıl erişileceğini kontrol etmek için kullanılır.

Sınıf dışından kullanım için. Üyeleri **public** yapın eğer sınıfın kullanıcıları için tasarlanmışlarsa. Alanları veya yöntemleri **protected** yapın eğer sınıfın genişleticileri için tasarlanmışlarsa ama sınıfın kullanıcıları için değil.

private ve **protected** belirteçleri yalnızca sınıf üyeleri için kullanılabilir. **public** belirteci ve varsayılan belirteç (yani, hiçbir belirteç) sınıf üyeleri üzerinde ve sınıf üzerinde de kullanılabilir. Hiçbir belirteç olmayan bir sınıf (yani, **public** sınıf değil) diğer paketlerden gelen sınıflar tarafından erişilemez.



Not

Bir alt sınıf, üst sınıfta tanımlanan korumalı bir yöntemi geçersiz kılabilir ve görünürlüğünü genel (**public**) hale getirebilir. Ancak, bir alt sınıf, üst sınıfta tanımlanan bir yöntemin erişilebilirliğini zayıflatamaz. Örneğin, bir yöntem üst sınıfta genel (**public**) olarak tanımlanmışsa, alt sınıfta da genel (**public**) olarak tanımlanmalıdır.

- 11.33** Aynı paketdeki bir sınıfın erişebilmesi için bir sınıfa hangi değiştiriciyi kullanmalısınız, bir farklı paketdeki bir sınıfın erişememesi için?
- 11.34** Farklı paketdeki bir sınıfın sınıfa erişememesi, ancak herhangi bir paketdeki alt sınıfların erişebilmesi için hangi değiştiriciyi kullanmalısınız?
- 11.35** Aşağıdaki kodda, sınıflar **A** ve **B** aynı pakettedir. (a) içindeki soru işaretleri boşluklarla değiştirilirse, sınıf **B** derlenebilir mi? Soru işaretleri **private** ile değiştirilirse, sınıf **B** derlenebilir mi? Soru işaretleri **protected** ile değiştirilirse, sınıf **B** derlenebilir mi?

Print Kontrol Et

MyProgrammingLab™

```

pakettp1;

genel sınıfA {nintti;
    ?
    ? voidtm() {n..
    }
}

```

(a)

```

pakettp1;

public classtBtextendstA {npublic
    voidtm1(String[] args) {nSystem.out.print
        ln(i);
        m();n
    }
}

```

(b)

- 11.36** Aşağıdaki kodda, sınıflar **A** ve **B** farklı paketlerde bulunmaktadır. (a) içindeki soru işaretleri boşluklarla değiştirilirse, sınıf **B** derlenebilir mi? Soru işaretleri **private** ile değiştirilirse, sınıf **B** derlenebilir mi? Soru işaretleri **protected** ile değiştirilirse, sınıf **B** derlenebilir mi?

```

pakettp1;

genel sınıfA {nintti;
    ?
    ? voidtm() {n..
    }
}

```

(a)

```

paketp2;

public classtBtextendstA {npublic
    voidtm1(String[] args) {nSystem.out.print
        ln(i);
        m();n
    }
}

```

(b)

11.14 Uzatma ve Geçersiz Kılmayı Önleme

Ne bir final sınıf ne de bir final yöntem genişletilebilir. Bir final veri alanı bir sabit'tir.



Sınıfların genişletilmesini zaman zaman önlemek isteyebilirsiniz. Bu gibi durumlarda, **final** değiştiricisi, bir sınıfın final olduğunu ve ebeveyn sınıf olamayacağını belirtmek için kullanılır. **Math** sınıfı bir final sınıftır. **String**, **StringBuilder** ve **StringBuffer** sınıfları da final sınıflardır. Örneğin, aşağıdaki **A** sınıfı finaldir ve genişletilemez:

```

halk    son    sınıfA {
    // Veri alanları, yapıcılar ve yöntemler atlandı
}

```

Ayrıca bir yöntemi final olarak tanımlayabilirsiniz; final bir yöntem, alt sınıfları tarafından geçersiz kılınamaz.

Örneğin, aşağıdaki yöntem `m` sonlandırılmıştır ve aşırı yüklenemez:

```
genel sınıf Test {
    // Veri alanları, yapıcılar ve yöntemler atlandı

    public son void m() {
        // Bir şey yap
    }
}
```



Not

Modifikatörler `public`, `protected`, `private`, `static`, `abstract` ve `final` sınıflar ve sınıf üyeleri (veri ve yöntemler) üzerinde kullanılır, ancak `final` modifikatörü bir yöntemde yerel değişkenler üzerinde de kullanılabilir. Bir `final` yerel değişken, bir yöntemi içinde bir sabittir.

Point Kontrol Et

MyProgrammingLab™

11.37 Bir sınıfın genişletilmesini nasıl engellersiniz? Bir yöntemin aşılmasını nasıl engellersiniz?

11.38 Aşağıdaki ifadeler için doğru veya yanlış belirtin:

- Korunan bir veri veya metoda aynı paketteki herhangi bir sınıf erişebilir.
- Korunan bir veri veya metoda farklı paketlerdeki herhangi bir sınıf erişebilir.
- Korunan bir veri veya metoda herhangi bir paketteki alt sınıfları erişebilir.
- Nihai bir sınıfın örnekleri olabilir.
- Nihai bir sınıf genişletilebilir.
- Nihai bir metod geçersiz kılınabilir.

AÇIKLAMALAR

gerçek tür 422

nesneleri dönüştürme 425

yapıcı zincirleme 415

beyan edilen tür 422

dinamik bağlama 422

miras alma 408

`instanceof` 426

is-a ilişkisi 437

metod geçersiz kılma 419

çoklu miras alma 414

geçersiz kıl 419

çok biçimlilik 422

korunmalı 437

tek miras alma 414

alt sınıf 408

alt tür 421

üst sınıf 408

üst tür 421

tip çıkarımı 431

BÖLÜM ÖZETİ

- Mevcut bir sınıftan yeni bir sınıf tanımlayabilirsiniz. Bu, sınıf *miras alma* olarak bilinir. Yeni sınıfa *alt sınıf*, *çocuk sınıf* veya *genişletilmiş sınıf* denir. Mevcut sınıfa ise *üst sınıf*, *ebeveyn sınıf* veya *temel sınıf* denir.

2. Bir yapıcı, bir sınıfın örneğini oluşturmak için kullanılır. Özellikler ve yöntemlerin aksine, bir üst sınıfın yapıcıları alt sınıfta miras alınmaz. Sadece alt sınıfların yapıcıları tarafından, **super** anahtar kelimesi kullanılarak çağrılabilir.
3. Bir yapıcı, aşırı yüklenmiş bir yapıcıyı veya üst sınıfının yapıcısını çağırabilir. Çağrı, yapıcının ilk ifadesi olmalıdır. Hiçbiri açıkça çağrılmazsa, derleyici **super()** ifadesini yapıcının ilk ifadesi olarak koyar, bu da üst sınıfın argüman almayan yapıcısını çağırır.
4. Bir yöntemi *geçersiz kılmak* için, yöntem alt sınıfta aynı imza ve üst sınıftaki ile aynı dönüş türü kullanılarak tanımlanmalıdır.
5. Bir örnek yöntemi yalnızca erişilebilir olduğunda geçersiz kılınabilir. Bu nedenle, bir özel yöntem kendi sınıfının dışından erişilemediği için geçersiz kılanamaz. Alt sınıfta tanımlanan bir yöntem, üst sınıfta özel ise, iki yöntem tamamen ilgisizdir.
6. Bir örnek yöntemi gibi, bir statik yöntem de miras alınabilir. Ancak, bir statik yöntem geçersiz kılınamaz. Üst sınıfta tanımlanan bir statik yöntem alt sınıfta yeniden tanımlanırsa, üst sınıfta tanımlanan yöntem gizlenir.
7. Java'daki her sınıf, **java.lang.Object** sınıfından türetilmiştir. Bir sınıf tanımlanırken hiçbir üst sınıf belirtilmezse, üst sınıfı **Object** olur.
8. Eğer bir yöntemin parametre türü bir üst sınıf ise (örneğin, **Object**), bu yöntem parametre alt sınıflarından herhangi bir nesne (örneğin, **Circle** veya **String**) geçirebilirsiniz. Bu, çok biçimlilik olarak bilinir.
9. Bir alt sınıfın örneğini bir üst sınıfın değişkenine dönüştürmek her zaman mümkündür, bir alt sınıfın örneği her zaman üst sınıfının bir örneğidir. Bir üst sınıfın örneğini alt sınıfının değişkenine dönüştürürken, açık bir dönüştürme kullanılmalıdır; bu, derleyiciye niyetinizi belirtmek için (**AltSınıfAdı**)
dönüştürme notasyonu ile yapılır.
10. Bir sınıf bir tür tanımlar. Bir alt sınıf tarafından tanımlanan bir tür *alt tür* olarak adlandırılır ve bir üst sınıf tarafından tanımlanan bir tür *üst tür* olarak adlandırılır.
11. Bir referans değişkeninden bir örnek yöntemini çağırırken, değişkenin *gerçek türü* hangi yöntem uygulamasının *çalışma zamanında* kullanılacağını belirler. Bu, dinamik bağlama olarak bilinir.
12. Bir nesnenin bir sınıfın örneği olup olmadığını test etmek için **obj instanceof AClass** kullanabilirsiniz.
13. Bir nesne listesi depolamak için bir nesne oluşturmak üzere **ArrayList** sınıfını kullanabilirsiniz.
14. Veri ve yöntemlerin farklı bir paketten alt sınıflar tarafından erişilmesini önlemek için **protected** belirleyicisini kullanabilirsiniz.
15. Bir sınıfın son olduğunu ve genişletilemeyeceğini belirtmek ve bir yöntemin son olduğunu ve geçersiz kılınmayacağını belirtmek için **final** belirleyicisini kullanabilirsiniz.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: www.cs.armstrong.edu/liang/intro9e/test.html.

MyProgrammingLab™

PROGRAMLAMA EGZERSİZLERİ

Bölümler 11.2–11.4

11.1 (Üçgen sınıfı) *GeometrikNesne sınıfını genişleten Üçgen adında bir sınıf tasarlayın. Sınıf şunları içerir:*

- Üç `double` veri alanı `side1`, `side2` ve `side3` adında varsayılan değerleri `1.0` olan üçgenin üç kenarını belirtmek için.
- Varsayılan bir üçgen oluşturan bir argüman almayan yapıcı.
- Belirtilen `side1`, `side2` ve `side3` ile bir üçgen oluşturan bir yapıcı.
- Üç veri alanı için erişim yöntemleri.
- Bu üçgenin alanını döndüren `getArea()` adında bir yöntem.
- Bu üçgenin çevresini döndüren `getPerimeter()` adında bir yöntem.
- Üçgen için bir dize tanımı döndüren `toString()` adında bir yöntem.

Üçgenin alanını hesaplamak için formül için, Programlama Egzersizi 2.15'e bakın. `toString()` yöntemi aşağıdaki gibi uygulanmıştır:

```
return "Triangle: side1 = " + side1 + " side2 = " + side2 +
    " side3 = " + side3;
```

Üçgen `Triangle` ve `GeometricObjects` sınıfları için UML diyagramlarını çizin ve sınıfları uygulayın. Kullanıcıdan üçgenin üç kenarını, bir rengi ve üçgenin dolu olup olmadığını belirtmek için bir Boolean değeri girmesini isteyen bir test programı yazın. Program, bu kenarlarla bir `Triangle` nesnesi oluşturmali ve girdi kullanarak `color` ve `filled` özelliklerini ayarlamalıdır. Program, alanı, çevreyi, rengi ve dolu olup olmadığını belirtmek için doğru veya yanlış değerini göstermelidir.

Bölümler 11.5–11.14

11.2 (Kişi, Öğrenci, Çalışan, Fakülte ve Personel sınıfları) *Kişi adında bir sınıf tasarlayın ve iki alt sınıfı olan Öğrenci ve Çalışan oluşturun.*

Fakülte ve Personel sınıflarını Çalışan sınıfının alt sınıfları yapın. Bir kişinin adı, adresi, telefon numarası ve e-posta adresi vardır. Bir öğrencinin sınıf durumu (birinci sınıf, ikinci sınıf, üçüncü sınıf veya son sınıf) vardır. Durumu bir sabit olarak tanımlayın. Bir çalışanın ofisi, maaşı ve işe alım tarihi vardır. İşe alım tarihi için Programlama Alıştırması 10.14'te tanımlanan `MyDate` sınıfını kullanarak bir nesne oluşturun. Bir fakülte üyesinin ofis saatleri ve bir rütbesi vardır. Bir personelin bir unvanı vardır. Her sınıfta `toString` yöntemini geçersiz kılın ve sınıf adını ve kişinin adını görüntüleyin.

Sınıflar için UML diyagramını çizin ve bunları uygulayın. Bir `Kişi`, `Öğrenci`, `Çalışan`, `Fakülte` ve `Personel` oluşturan bir test programı yazın ve onların `toString()` yöntemlerini çağırın.

11.3 (Hesap sınıflarının alt sınıfları) *Programlama Alıştırması 8.7'de Hesap sınıfı bir bank a hesabını modellemek için tanımlanmıştır. Bir hesabın hesap numarası, bakiye, yıllık faiz oranı ve oluşturulma tarihi gibi özellikleri vardır ve fonları yatırmak ve çekmek için yöntemleri vardır. Çekme ve birikim hesapları için iki alt sınıf oluşturun. Bir çek hesabının aşırı çekim limiti vardır, ancak bir tasarruf hesabı aşırı çekebilir.*

Sınıflar için UML diyagramını çizin ve ardından bunları uygulayın. **Account**, **SavingsAccount** ve **CheckingAccount** nesnelerini oluşturan bir test programı yazın ve bunların **toString()** yöntemlerini çağırın.

- 11.4** (**ArrayList** içindeki maksimum eleman) Bir **ArrayList** içinde maksimum değeri döndüren aşağıdaki yöntemi yazın. Yöntem, liste **null** ise veya liste boyutu **0** ise **null** döner.

```
public static Integer max(ArrayList<Integer> list)
```

Kullanıcıdan 0 ile biten bir sayı dizisi girmesini isteyen ve bu yöntemi çağırarak girdi içindeki en büyük sayıyı döndüren bir test programı yazın.

- 11.5** (**Course** sınıfı) Liste 10.6'daki **Course** sınıfını yeniden yazın. Öğrencileri depolamak için bir dizi yerine bir **ArrayList** kullanın. **Course** sınıfının orijinal sözleşmesini değiştirmemelisiniz (yani, yapıların ve yöntemlerin tanımı değiştirilmemelidir).
- 11.6** (**ArrayList** kullanın) Bir **ArrayList** oluşturan ve listeye bir **Loan** nesnesi, bir **Date** nesnesi, bir dize, bir **JFrame** nesnesi ve bir **Circle** nesnesi ekleyen bir program yazın ve bir döngü kullanarak listenin tüm elemanlarını nesnenin **toString()** yöntemini çağırarak görüntüleyin.
- 11.7** (**Karıştır ArrayList**) Aşağıdaki yöntemi yazın, bu yöntem bir

ArrayList tam sayılarının elemanlarını karıştırır.

```
public static void karıştır(ArrayList<Integer> liste)
```

**11.8 (Yeni Hesap sınıfı) Bir Hesap sınıfı Programlama Alıştırması

8.7'de belirtilmiştir. Aşağıdaki gibi yeni bir Hesap sınıfı tasarlayın:

- Müşterinin adını saklamak için **String** türünde yeni bir veri alanı **ad** ekleyin.
- Belirtilen ad, kimlik ve bakiye ile bir hesap oluşturan yeni bir yapıyı ekleyin.
- Hesaplar için işlemleri saklayan **ArrayList** türünde işlemler adında yeni bir veri alanı ekleyin. Her işlem, İşlem sınıfının bir örneğidir. İşlem sınıfı Şekil 11.6'da gösterildiği gibi tanımlanmıştır.



VideoNotu

Yeni Hesap sınıfı

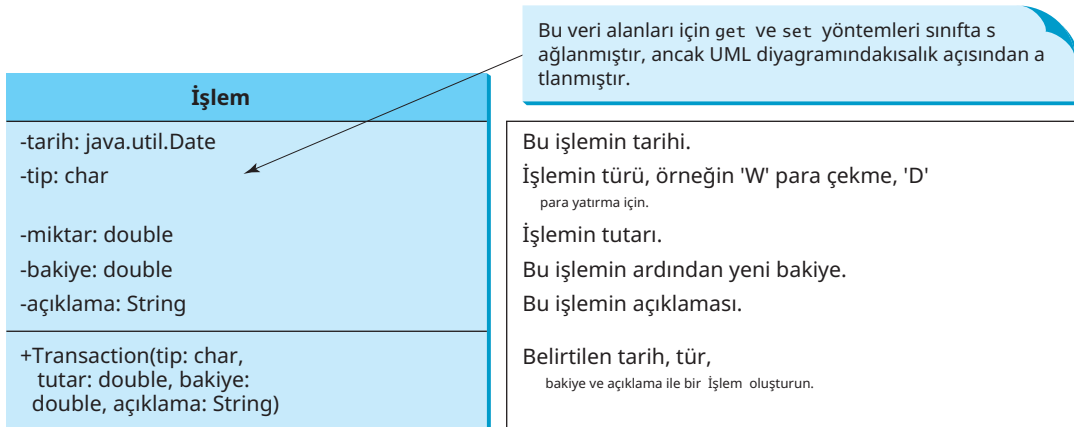


FIGURE 11.6 İşlem sınıfı, bir banka hesabı için bir işlemi tanımlar.

■ **withdraw** ve **deposit** yöntemlerini değiştirin, böylece bir işlem **transacti** **ons** dizi listesine eklenir.

■ Diğer tüm özellikler ve yöntemler, Programlama Egzersizi 8.7'deki gibi aynıdır.

Yıllık faiz oranı **1.5%**, bakiye **1000**, kimlik **1122** ve isim **George** olan bir **Hesap** oluşturan bir test programı yazın. Hesaba \$30, \$40 ve \$50 yatırın ve hesaptan \$5, \$4 ve \$2 çekin. Hesap sahibinin adını, faiz oranını, bakiyeyi ve tüm işlemleri gösteren bir hesap özeti yazdırın.

*11.9

(*En büyük satırlar ve sütunlar*) Rastgele olarak **0**s ve **1**s ile dolduran bir program yazın n-by-n matrisini yazdırın ve en çok olan satırları ve sütunları bulun **1**s. (*İpucu*: En çok satır ve sütun indekslerini depolamak için iki **ArrayList**s kullanın **1**s.) İşte programın bir örnek çalışması:



```
Dizi boyutunu n girin: 4 Enter
Rastgele dizi
001100111010101
En büyük satır indeksi: 2
En büyük sütun indeksi: 2, 3
```

11.10

(Kalıtım kullanarak **MyStack** uygulayın) Liste 11.10, **MyStack** bileşim kullanılarak uygulanmıştır. **ArrayList**'i genişleten yeni bir yığın sınıfı tanımlayın.

Sınıflar için UML diyagramını çizin ve ardından **MyStack**'i uygulayın. Kullanıcıdan beş dize girmesini isteyen ve bunları terssırada gösteren bir test programı yazın.

11.11

(*SıralaArrayList*) Aşağıdaki yöntemi yazın, bu bir **ArrayList** sayılarının sıralanmasını sağlar:

```
public static void sort(ArrayList<Integer> list)
```

Kullanıcıdan 5 sayı girmesini isteyen, bunları bir dizi listesinde saklayan ve artan sırada gösteren bir test programı yazın.

11.12

(*ToplaArrayList*) Aşağıdaki yöntemi yazın, bu bir **ArrayList** içindeki tüm sayıların toplamını döndürür:

```
genel statik double sum(ArrayList<Double> list)
```

Kullanıcıdan 5 sayı girmesini isteyen, bunları bir dizi listesinde saklayan ve toplamalarını gösteren bir test programı yazın.

GUI BtASICS

Hedefler

- Swing ve AWT'yi ayırt etmek için (§12.2).
- Java GUI API hiyerarşisini tanımlamak (§12.3).
- Kullanıcı arayüzlerini çerçeveler, paneller ve basit GUI bileşenleri kullanarak oluşturmak (§12.4).
- Yerleşim yöneticilerinin rolünü anlamak ve bileşenleri bir konteynerde düzenlemek için **FlowLayout**, **GridLayout** ve **BorderLayout** yöneticilerini kullanmak (§12.5).
- Bileşenleri bir alt konteynerde gruplamak için **JPanel** kullanmak (§12.6).
- Renkler için nesneler oluşturmak amacıyla **Color** sınıfını kullanmak (§12.7).
- Yazı tipleri için nesneler oluşturmak için **Font** sınıfını kullanmak (§12.8).
- Swing bileşenlerinde kenarlıklar, araç ipuçları, yazı tipleri ve renkler gibi ortak özellikleri uygulamak (§12.9).
- GUI bileşenlerinin kenarını süslemek (§12.9).
- **ImageIcon** sınıfını kullanarak resim simgeleri oluşturmak (§12.10).
- **JButton** sınıfını kullanarak düğmeler oluşturmak ve kullanmak (§12.11).
- **CheckBox** sınıfını kullanarak onay kutuları oluşturmak ve kullanmak (§12.12).
- **RadioButton** sınıfını kullanarak radyo düğmeleri oluşturmak ve kullanmak (§12.13).
- **Label** sınıfını kullanarak etiketler oluşturmak ve kullanmak (§12.14).
- **TextField** sınıfını kullanarak metin alanları oluşturmak ve kullanmak (§12.15).



12.1 Giriş



Java GUI, nesne yönelimli programlamayı öğrenmek için mükemmel bir pedagojik araçtır.

Java GUI programlama için API tasarımı, nesne yönelimli ilkenin nasıl uygulandığının mükemmel bir örneğidir. Bu bölüm iki amaca hizmet etmektedir. İlk olarak, Java GUI programlamanın temellerini sunmaktadır. İkinci olarak, OOP'yi göstermek için GUI'yi kullanmaktadır. Özellikle, bu bölüm Java GUI API'sinin çerçevesini tanıtmaktadır ve GUI bileşenlerini ve bunların ilişkilerini, konteynerleri ve düzen yöneticilerini, renkleri, yazı tiplerini, kenarlıkları, resim simgelerini ve ipuçlarını tartışmaktadır. Ayrıca, en sık kullanılan GUI bileşenlerinden bazılarını tanıtmaktadır.

12.2 Swing ve AWT



AWT GUI bileşenleri, daha çok yönlü ve kararlı Swing GUI bileşenleriyle değiştirilmiştir.

OOP'yi göstermek için basit GUI örnekleri kullandık. 8.6.3. Bölüm, GUI Bileşenlerini Gösterme. **JButton**, **JLabel**, **JTextField**, **JRadioButton** ve **JComboBox** gibi GUI bileşenlerini kullandık. GUI bileşen sınıflarının neden *J* ön eki var?

JButton yerine neden sadece Button olarak adlandırmayalım? Aslında, java.awt paketinde zaten Button adında bir sınıf var.

Java tanıtıldığında, GUI sınıfları Abstract Windows Toolkit (AWT) olarak bilinen bir kütüphanede toplandı.

AWT

AWT, basit grafik kullanıcı arayüzleri geliştirmek için iyidir, ancak kapsamlı GUI projeleri geliştirmek için uygun değildir. Ayrıca, AWT platforma özgü hatalara eğilimlidir. AWT kullanıcı arayüzü bileşenleri, daha sağlam, çok yönlü ve esnek bir kütüphane olan Swing bileşenleri ile değiştirilmiştir. Swing bileşenleri, Java kodu kullanılarak doğrudan tuvallere boyanır, java.awt.Window veya java.awt.Panel sınıflarının alt sınıfları olan bileşenler hariç, bu bileşenler belirli bir platformda yerel GUI kullanılarak çizilmelidir.

Swing bileşenleri

Swing bileşenleri, hedef platforma daha az bağımlıdır ve yerel GUI kaynaklarını daha az kullanır. Bu nedenle, yerel GUI'ye bağımlı olmayan Swing bileşenlerine hafif bileşenler, AWT bileşenlerine ise ağır bileşenler denir.

hafif bileşen
ağır bileşen

neden *J* ile başlıyor?

Yeni Swing bileşen sınıflarını AWT karşıtlarından ayırmak için, Swing GUI bileşen sınıfları *J* ön eki ile adlandırılmıştır. AWT bileşenleri Java'da hala destekleniyor olsa da, Swing bileşenleri kullanarak programlamayı öğrenmek daha iyidir, çünkü AWT kullanıcı arayüzü bileşenleri sonunda kaybolacaktır. Bu kitap yalnızca Swing GUI bileşenlerini kullanır.

12.1 Swing GUI sınıfları neden *J* ön eki ile adlandırılmıştır?

12.2 AWT GUI bileşenleri ile Swing GUI bileşenleri arasındaki farkı açıklayın.



MyProgrammingLab™

12.3 Java GUI API'si



GUI API'si, üç gruba ayrılabilen sınıfları içerir: bileşen sınıfları, konteyner sınıfları ve yardımcı sınıflar.

Java GUI API'sinin hiyerarşik ilişkileri Şekil 12.1'de gösterilmiştir. Üçgen okun masala ilişkisini, elmasın bileşim ilişkisini ve dolu elmasın özel bileşim ilişkisini gösterdiğini hatırlayın. Nesne bileşim ilişkisi Bölüm 10.7'de tanıtılmıştır.

bileşen sınıfı
kapsayıcı sınıf

yardımcı sınıf

Component sınıfının alt sınıflarına bileşen sınıfları denir ve kullanıcı arayüzü oluşturmak için kullanılır. JFrame, JPanel ve JApplet gibi sınıflara, diğer bileşenleri içermek için kullanılan konteyner sınıfları denir. Graphics, Color, Font, FontMetrics ve Dimension gibi sınıflara ise GUI bileşenlerini desteklemek için kullanılan yardımcı sınıflar denir.

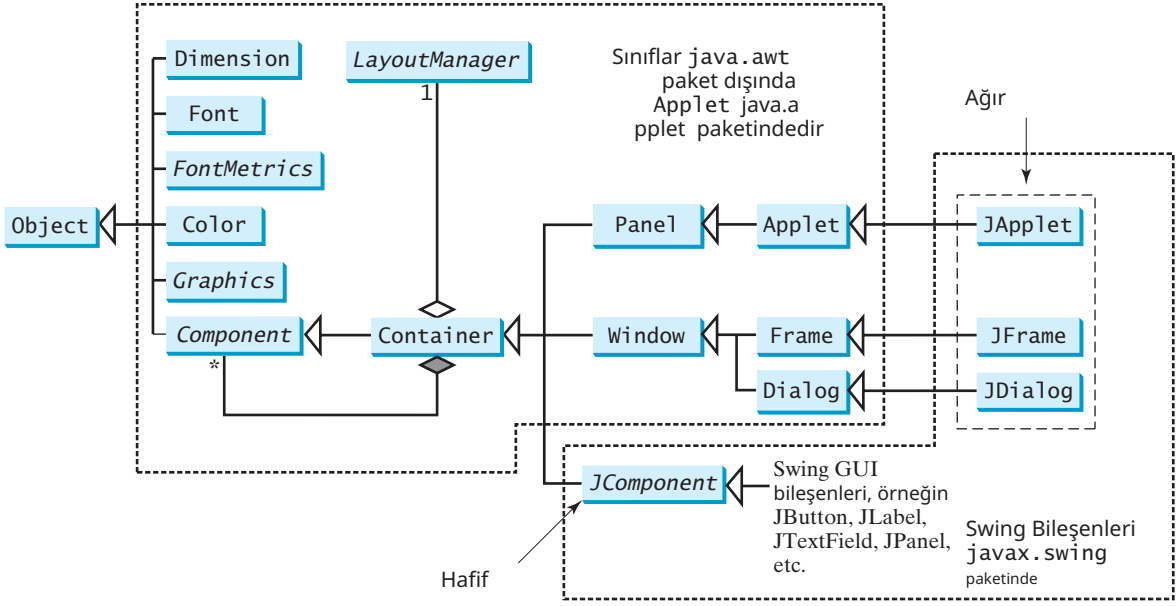


FIGURE 12.1 Java GUI programlama, bu hiyerarşik diyagramda gösterilen sınıfları kullanır.



Not

JFrame, JApplet, JDialog ve JComponent sınıfları ve onların alt sınıfları javax.swing paketinde gruplanmıştır. Applet, java.applet sınıfındadır. Şekil 12.1'deki diğer tüm sınıflar **java.awt** paketinde gruplanmıştır.

12.3.1 Bileşen Sınıfları

Bir **Component** örneği ekranda görüntülenebilir. **Component**, tüm kullanıcı arayüzü sınıflarının kök sınıfıdır ve konteyner sınıflarını içerir, **JComponent** ise tüm hafif Swing bileşenlerinin kök sınıfıdır. Hem **Component** hem de **JComponent** soyut sınıflardır (soyut sınıflar Bölüm 15'te tanıtılacaktır). Şu anda bilmeniz gereken tek şey, soyut sınıfların, new operatörünü kullanarak örnekler oluşturamayacağınız sınıflar olduğudur. Örneğin, **new JComponent()** kullanarak **JComponent** örneği oluşturamazsınız. Ancak, **JComponent**'ün somut alt sınıflarının yapıcılarını kullanarak **JComponent** örnekleri oluşturabilirsiniz. Sınıf miras alma hiyerarşisi ile tanışmak önemlidir. Örneğin, aşağıdaki ifadelerin hepsi doğru olarak görüntülenir:

```
JButton jbtOK = tnewtJButton(t"OK"t);nSystem.out.println(jbtOKinstanceoftJButton);nSystem.out.println(jbtOKinstanceoftJComponent);nSystem.out.println(jbtOKinstanceoftContainer);nSystem.out.println(jbtOKinstanceoftComponent);nSystem.out.println(jbtOKinstanceoftObject);
```

soyut sınıf

12.3.2 Konteyner Sınıfları

Container örneği, Component örneklerini tutabilir. Bir konteyner, başka bir konteyneri içine gömülmeden görüntülenebiliyorsa, üst düzey konteyner olarak adlandırılır. Window, Frame, Dialog, JFrame ve JDialog üst düzey konteynerlerdir. Window, Panel, Applet, Frame ve Dialog AWT bileşenleri için konteyner sınıflarıdır. Swing bileşenleri ile çalışmak için Container, JFrame, JDialog, JApplet ve JPanel kullanın, bu Tablo 12.1'de açıklanmıştır.

TABLE 12.1 GUI Konteyner Sınıfları

Konteyner Sınıfı	Açıklama
<code>java.awt.Container</code>	bileşenleri tutmak için kullanılır. Çerçeveler, paneller ve applet'ler onun alt sınıflarıdır.
<code>javax.swing.JFrame</code>	Java GUI uygulamalarında diğer Swing kullanıcı arayüzü bileşenlerini tutmak için üst düzey bir konteynerdir.
<code>javax.swing.JPanel</code>	kullanıcı arayüzü bileşenlerini gruplamak için görünmez bir konteynerdir. Paneller iç içe yerleştirilebilir. panelleri başka bir panelin içine yerleştirebilirsiniz. <code>JPanel</code> genellikle grafik çizmek için bir tuval olarak da kullanılır.
<code>javax.swing.JApplet</code>	Swing bileşenlerini kullanarak bir Java applet oluşturmak için temel bir sınıftır.
<code>javax.swing.JDialog</code>	kullanıcıdan ek bilgi almak veya kullanıcıya bildirim sağlamak için genellikle geçici bir pencere olarak kullanılan bir açılır penceredir.

12.3.3 GUI Yardımcı Sınıfları

Graphics, Color, Font, FontMetrics, Dimension ve LayoutManager gibi yardımcı sınıflar, Component'in alt sınıfları değildir. Grafik bağlamı, renkler, yazı tipleri ve boyut gibi GUI bileşenlerinin özelliklerini tanımlamak için kullanılırlar, bu da Tablo 12.2'de açıklanmıştır.

TABLO 12.2 GUI Yardımcı Sınıfları

Yardımcı Sınıf	Açıklama
<code>java.Ah sorunu.grafikler</code>	string, çizgiler ve basit şekiller çizmek için yöntemler sağlayan soyut bir sınıftır.
<code>java.awt.Color</code>	GUI bileşenlerinin renkleri ile ilgilenir. Örneğin, <code>JFrame</code> ve <code>JPanel</code> gibi bileşenlerde arka plan veya ön plan renklerini belirtebilir veya çizimlerdeki çizgilerin, şekillerin ve metinlerin renklerini belirtebilirsiniz.
<code>java.out.font</code>	GUI bileşenlerindeki metin ve çizimler için fontları belirtir. Örneğin, bir düğmedeki metin için font türünü (örneğin, SansSerif), stilini (örneğin, kalın) ve boyutunu (örneğin, 24 punto) belirtebilirsiniz.
<code>java.awt.FontMetrics</code>	fontların özelliklerini almak için kullanılan soyut bir sınıftır.
<code>java.awt.Dimension</code>	bir bileşenin genişliğini ve yüksekliğini (tam sayı hassasiyetinde) tek bir nesnede kapsar.
<code>java.awt.LayoutManager</code>	bileşenlerin bir konteyner içinde nasıl düzenleneceğini belirtir.



Not

Yardımcı sınıflar `java.awt` paketindedir. Swing bileşenleri, AWT'deki tüm sınıfları değiştirmez, yalnızca AWT GUI bileşen sınıflarını (örneğin, `Düğme`, `MetinAlanı`, `MetinBölgesi`) değiştirir. AWT yardımcı sınıfları, GUI programlamada hala faydalıdır.

12.3 Java GUI bileşen sınıflarının kök sınıfı hangisidir? Bir konteyner sınıfı `Component` sınıfının bir alt sınıfı mıdır? Swing GUI bileşen sınıflarının kök sınıfı hangisidir?

12.4 Aşağıdaki ifadelerden hangileri sözdizim hatası içermektedir?

```
Bileşen c1 = tnewtComponent();nJCom
ponent c2 = tnewtComponent();nCompo
nent c3 = tnewtButton();nJCompon
ent c4 = tnewtButton();
```


12.4 Çerçeveler

Bir çerçeve, diğer GUI bileşenlerini tutmak için bir penceredir.

Bir kullanıcı arayüzü oluşturmak için, kullanıcı arayüzü bileşenlerini tutmak üzere ya bir çerçeve ya da bir applet oluşturmanız gerekir. Bu bölüm çerçeveleri tanıtmaktadır. Java applet'leri oluşturma, Bölüm 18'de tanıtılacaktır.



12.4.1 Bir Çerçeve Oluşturma

Bir çerçeve oluşturmak için, Şekil 12.2'de gösterildiği gibi **JFrame** sınıfını kullanın.

javax.swing.JFrame	
+JFrame() +JFrame(başlık: String)n+setSize(genilik: int, yükseklik: int): voidn+setLocation(x: int, y: int): void +setVisible(visible: boolean): voidn+setDefaultCloseOperation(mode: int): voidn+setLocationRelativeTo(c: Bileşen): void +paket(): boşluk	Başlıksız varsayılan bir çerçeve oluşturur. Belirtilen başlığa sahip bir çerçeve oluşturur. Çerçevenin boyutunu ayarlar. Çerçevenin sol üst köşe konumunu ayarlar. Çerçeveyi görüntülemek için true ayarlar. Çerçeve kapatıldığında işlemi belirtir. Çerçevenin belirtilen bileşene göre konumunu ayarlar. Bileşen null ise, çerçeve ekranda ortalanır. Çerçevdeki bileşenleri tutacak şekilde çerçeve boyutunu otomatik olarak ayarlar.

ŞEKİL 12.2 JFrame sınıfı, GUI bileşenlerini görüntülemek için bir pencere oluşturmak için kullanılır.

Listing 12.1'deki program bir çerçeve oluşturur.

Listing 12.1 MyFrame.java

<pre> 1 import javax.swing.JFrame; 2 3 genel class MyFrame { 4 public static void main(String[] args) { 5 JFrame frame = new JFrame("MyFrame"); // Bir çerçeve oluştur 6 7 çerçeve.setLocationRelativeTo(null); // Bir çerçeveyi ortalara 8 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); 9 frame.setVisible(true); // Çerçeveyi göster 10 } 11 } </pre>	paket içe aktar çerçeve oluştur boyut ayarla çerçeveyi ortalara çıkışta kapat display the frame
---	---

Çerçeve, **frame.setVisible(true)** yöntemi çağrılana kadar gösterilmez. **frame.setSize(400, 300)** çerçevenin 400 piksel genişliğinde ve 300 piksel

yüksekliğinde olduğunu belirtir. Eğer **setSize** yöntemi kullanılmazsa, çerçeve yalnızca başlık çubuğunu gösterecek şekilde boyutlandırılır.

setVisible yöntemleri Component sınıfında tanımlandığı için, bunlar JFrame sınıfı tarafından miras alınır. Daha sonra bu yöntemlerin Component sınıfının birçok diğer alt sınıfında da faydalı olduğunu göreceksiniz.

MyFrame programını çalıştırdığınızda, ekranda bir pencere görüntülenecektir (bkz. Şekil 12.3a).

setLocationRelativeTo(null) (satır 7) çağrısı, çerçeveyi ekranda ortalar. **setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)** (satır 8) çağrısı, programın çerçeve kapatıldığında sonlanmasını sağlar. Bu ifade kullanılmazsa, çerçeve kapatıldığında program sonlanmaz. Bu durumda, programı durdurmak için Windows'taki DOS istemci penceresinde Ctrl+C tuşlarına basmanız veya kill komutunu kullanarak süreci durdurmanız gerekir.



FIGURE 12.3(a) Program, **MyFrame** başlığıyla bir çerçeve oluşturur ve görüntüler. (b) Bir OK butonu çerçeveye eklenir.

UNIX'te. Programı Eclipse veya NetBeans gibi bir IDE'den çalıştırıyorsanız, programı durdurmak için Konsol panelindeki kırmızı Terminate butonuna tıklamanız gerekir.

piksel ve çözünürlük



Not

Bir pikselin ekranda çizim yapmak için mevcut olan en küçük alan birimi olduğunu unutmayın. Bir pikseli küçük bir dikdörtgen olarak düşünebilir ve ekranı piksellerle kaplanmış olarak hayal edebilirsiniz.

Çözünürlük, ekranın yatay ve dikey boyutlarındaki piksel sayısını belirtir. Ekranın da fazla pikseli varsa, ekranın çözünürlüğü daha yüksektir. Çözünürlük ne kadar yüksekse, görebileceğiniz detay o kadar ince olur.

merkezlemeden önce boyut ayarla



Not

Çerçeveyi ortalamak için `setLocationRelativeTo(null)` yöntemini çağırmadan önce `setSize(w, h)` yöntemini çağırmanız gerekir.

12.4.2 Bir Çerçeveye Bileşenler Ekleme

Şekil 12.3'a da gösterilen çerçeve boştur. `add` yöntemini kullanarak, çerçeveye bileşenler ekleyebilirsiniz; bu, Liste 12.2'de gösterilmiştir.

LİSTE 12.2 MyFrameWithComponents.java

```
1 import javax.swing.*;
2
3 genel classt MyFrameWithComponents {
4     public static void main(String[] args) {
5         JFrame çerçevesi = new JFrame("MyFrameWithComponents");
6
7         // Çerçeveye bir buton ekle
8         JButton jbtOK = new JButton("OK");
9         çerçeve.add(jbtOK);
10
11         frame.setSize(400, 300);
12         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13         frame.setLocationRelativeTo(null); // Çerçeveyi ortala
14         frame.setVisible(true);
15     }
16 }
```

bir buton oluştur
çerçeveye ekle

boyut ayarla
pencere kapandığında çıkış yap
çerçeveyi ortala
görünür yap

Her **JFrame**, *java.awt.Container* sınıfının bir örneği olan bir **içerik paneli** içerir. GUI bileşenleri, çerçeve de içerik panelinde butonlar gibi yerleştirilir. Java'nın önceki sürümlerinde, çerçevenin içerik panelini döndürmek için **JFrame** sınıfındaki `getContentPane` yöntemini kullanmanız gerekiyordu, ardından bir bileşeni içerik paneline yerleştirmek için içerik panelinin `add` yöntemini çağırmanız gerekiyordu, aşağıdaki gibi:

```
java.awt.Container konteyneri = frame.getContentPane();
konteyner.add(jbtOK);
```

Bu zahmetliydi. Java 5'ten itibaren Java sürümleri, bileşenleri çerçevenin

içerik paneline `add` yöntemini çağırarak yerleştirmenize olanak tanır, aşağıdaki gibi:

```
çerçeve.add(jbtOK);
```

Bu özellik *içerik-panei delegasyonu* olarak adlandırılır. Teknik olarak, bir bileşen çerçevenin içerik paneline eklenir. Basitlik açısından, bir bileşenin bir çerçeveye eklendiğini söyleriz.

içerik-pane delegasyonu

Liste 12.2'de, `JButton` nesnesi `new JButton("OK")` kullanılarak oluşturuldu ve bu nesne çerçevenin içerik paneline eklendi (satır 9).

`add(Component comp)` yöntemi, `Container` sınıfında tanımlanmıştır ve bir `Component` örneğini konteynıra ekler. `JButton`, `Component` sınıfının bir alt sınıfı olduğunda `n`, `JButton` örneği de bir `Component` örneğidir. Bir bileşeni bir konteynırdan kaldırma için, `remove` yöntemini kullanın. Aşağıdaki ifade butonu konteynırdan kaldırır:

```
konteyner.remove(jbtOK);
```

Programı çalıştırdığınızda `MyFrameWithComponents`, pencere Şekil 12.3b'deki gibi görüntülenecektir. Buton her zaman çerçevenin ortasında yer alır ve çerçeveyi ne kadar yeniden boyutlandırırsanız boyutunu tamamen kaplar. Bunun nedeni, bileşenlerin çerçeveye içerik pane'inin düzen yöneticisi tarafından yerleştirilmesidir ve içerik pane için varsayılan düzen yöneticisi butonu ortada yerleştirir. Bir sonraki bölümde, bileşenleri istenen konumlara yerleştirmek için birkaç farklı düzen yöneticisi kullanacaksınız.

12.5 Bir çerçeve nasıl oluşturulur? Bir çerçevenin boyutu nasıl ayarlanır? Bir çerçeveye bileşenler nasıl eklenir? Eğer

`frame.setSize(400, 300)` ve `frame.setVisible(true)` Listing 12.2'de yer

değiştirse ne olurdu?

Point Kontrol Et
MyProgrammingLab™

12.5 Düzen Yöneticileri

Her konteyner, konteynerdeki GUI bileşenlerini düzenlemekten sorumlu bir nesne olan bir düzen yöneticisi içerir.



Birçok diğer pencere sisteminde, kullanıcı arayüzü bileşenleri, sabit kodlanmış piksel ölçümleri kullanılarak düzenlenir. Örneğin, bir pencerede bir butonu (10, 10) konumuna sabit kodlanmış piksel ölçümleri ile yerleştirirken, kullanıcı arayüzü bir sistemde iyi görünebilir ancak diğerinde kullanılamaz hale gelebilir. Java'nın *düzen yöneticileri*, kullanıcı arayüzünüzü tüm pencere sistemlerinde otomatik olarak eşleyen bir soyutlama seviyesi sağlar.

düzen yöneticisi

Java GUI bileşenleri, bileşenlerin doğru konumlarda yerleştirilmesi için arka planda çalışan düzen yöneticisi tarafından düzenlendiği konteynerlerde yerleştirilir. Önceki programda, çerçevede OK düğmesini nereye yerleştireceğinizi belirtmediniz, ancak Java, düzen yöneticisi bileşenleri doğru konumlara yerleştirdiği için nereye yerleştireceğini bilir. Bir düzen yöneticisi, bir düzen yöneticisi sınıfı kullanılarak oluşturulur.

Düzen yöneticileri, konteynerlerde `setLayout(aLayoutManager)` yöntemi kullanılarak ayarlanır. Örneğin, `XLayout` örneğini oluşturmak ve bunu bir konteynerde ayarlamak için aşağıdaki ifadeleri kullanabilirsiniz:

```
LayoutManager layoutManager = newXLayout();  
konteyner.setLayout(layoutManager);
```

Bu bölüm, üç temel düzen yöneticisini tanıtmaktadır: `FlowLayout`, `GridLayout` ve `BorderLayout`.

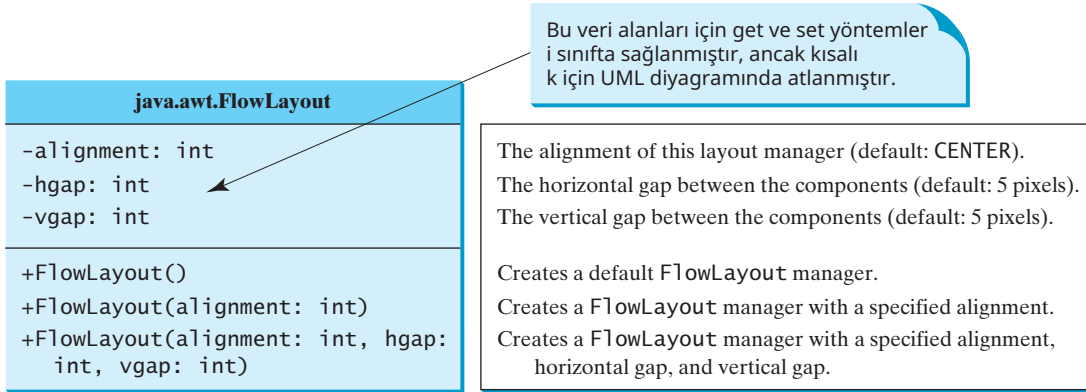


VideoNotu

FlowLayout kullanın

12.5.1 Akış Düzeni

FlowLayout en basit düzen yöneticisidir. Bileşenler, konteynerde eklenme sırasına göre soldan sağa doğru düzenlenir. Bir satır dolduğunda, yeni bir satır başlatılır. Bileşenlerin nasıl hizalanacağını üç sabitten birini kullanarak belirtebilirsiniz: **FlowLayout.RIGHT**, **FlowLayout.CENTER** veya **FlowLayout.LEFT**. Ayrıca, bileşenler arasındaki boşluğu piksel cinsinden de belirtebilirsiniz. **FlowLayout** için sınıf diyagramı Şekil 12.4'te gösterilmektedir.



ŞEKİL 12.4 FlowLayout bileşenleri satır satır düzenler.

Liste 12.3, akış düzenini gösteren bir programı vermektedir. Program, Şekil 12.5'te gösterildiği gibi, çerçeveye üç etiket ve metin alanı ekler.

LİSTE 12.3 ShowFlowLayout.java

JFrame'den türetilir

düzeni ayarla

etiket ekle

metin alanı ekle

```

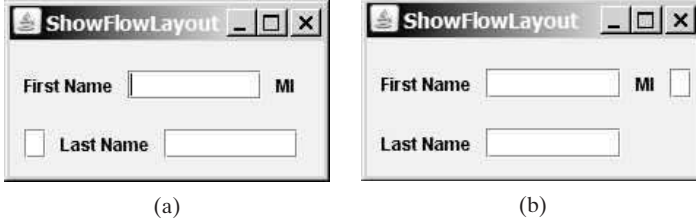
1  import javax.swing.JLabel;
2  import javax.swing.JTextField;
3  import javax.swing.JFrame;
4  import java.awt.FlowLayout;
5
6  public sınıf ShowFlowLayout extends JFrame {
7      public ShowFlowLayout() {
8          // Akış Düzeni ayarla, sola hizalanmış, yatay boşluk 10
9          // ve bileşenler arasında dikey boşluk 20
10         setLayout(yeni FlowLayout(FlowLayout.LEFT, 10, 20));
11
12         // Çerçeveye etiketler ve metin alanları ekle
13         add(new JLabel("İsim"));
14         add(new JTextField(8));
15         add(new JLabel("MI"));
16         add(new JTextField(1));
17         add(new JLabel("Soyisim"));
18         add(new JTextField(8));
19     }
20
21     /** Ana yöntem */
22     public static void main(String[] args) {
23         ShowFlowLayout çerçeve = new ShowFlowLayout();
24         çerçeve.setTitle("ShowFlowLayout");
25         çerçeve.setSize(200, 200);
  
```

```

26     frame.setLocationRelativeTo(null); // Çerçeveyi ortalara
27     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28     frame.setVisible(true);
kes}na0}

```

çerçeve oluştur
görünür yap



ŞEKİL 12.5 Bileşenler, **FlowLayout** yöneticisi tarafından konteynerdeki satırları birer birer doldurmak için eklenir.

Bu örnek, önceki bölümdeki programlardan farklı bir stil kullanarak bir program oluşturur; burada çerçeveler **JFrame** sınıfı kullanılarak oluşturulmuştur. Bu örnek, **JFrame** sınıfını genişleten bir sınıf olan **ShowFlowLayout** oluşturur (satır 6). Bu programdaki **main** metodu, **ShowFlowLayout**'un bir örneğini oluşturur (satır 23). **ShowFlowLayout**'un yapıcısı, bileşenleri çerçeveye inşa eder ve yerleştirir. Bu, GUI uygulamaları oluşturmanın tercih edilen stildir—üç neden için:

- Bir GUI uygulaması oluşturmak, bir çerçeve oluşturmak anlamına gelir, bu nedenle **JFrame**'yi genişletmek için bir çerçeve tanımlamak doğaldır.
- Çerçeve, yeni bileşenler veya işlevler eklemek için daha da genişletilebilir.
- Sınıf kolayca yeniden kullanılabilir. Örneğin, sınıfın birden fazla örneğini oluşturarak birden fazla çerçeve oluşturabilirsiniz.

Bir stili tutarlı bir şekilde kullanmak, programları okunması kolay hale getirir. Bundan sonra, çoğu GUI ana sınıfı **JFrame** sınıfını genişletecektir. Ana sınıfın yapıcısı, kullanıcı arayüzünü inşa eder. **main** metodu, ana sınıfın bir örneğini oluşturur ve ardından çerçeveyi görüntüler.

Program, 23. satır aşağıdaki kod ile değiştirilirse çalışır mı?

```
JFrame çerçevesi = tnewtShowFlowLayout();
```

Evet. Program yine de çalışacak çünkü **ShowFlowLayout**, **JFrame**'in bir alt sınıfıdır ve **setTitle**, **setSize**, **setLocationRelativeTo**, **setDefaultCloseOperation** ve **setVisible** (24-28. satırlar) yöntemleri

JFrame sınıfında mevcuttur.

Bu örnekte, **FlowLayout** yöneticisi bileşenleri bir çerçeveye yerleştirmek için kullanılır. Eğer çerçeveyi yeniden boyutlandırırsanız, bileşenler otomatik olarak uyacak şekilde yeniden düzenlenir. Şekil 12.5a'da, ilk satırda üç bileşen vardır, ancak Şekil 12.5b'de, ilk satırda dört bileşen vardır çünkü genişlik artırılmıştır.

Eğer **setLayout** ifadesini (10. satır) **setLayout(new FlowLayout(FlowLayout.RIGHT, 0, 0))** ile değiştirirseniz, tüm buton satırları boşluk olmadan sağa hizalanacaktır.

10. satırda anonim bir **FlowLayout** nesnesi oluşturulmuştur:

```
setLayout(yeni FlowLayout(FlowLayout.LEFT,t10t,t20t));
```

bu, şuna eşdeğerdir:

```
FlowLayout düzeni = new FlowLayout(FlowLayout.LEFT,t10t,t20t);setLayout(la  
yout);
```

Bu kod, `FlowLayout` sınıfının `layout` nesnesine açık bir başvuru oluşturur. Açık başvuru gerekli değildir, çünkü nesne `ShowFlowLayout` sınıfında doğrudan referans edilmemektedir.

Farz edelim ki aynı düğmeyi çerçeveye on kez ekliyorsunuz; çerçevede on düğme mi görünecek? Hayır, bir düğme gibi bir GUI bileşeni yalnızca bir konteynere ve yalnızca bir kez eklenebilir. Bir düğmeyi bir konteynere birden fazla kez eklemek, onu bir kez eklemekle aynıdır.



Not

GUI bileşenleri konteynerler tarafından paylaşılamaz, çünkü yalnızca bir GUI bileşeni aynı anda yalnızca bir konteynerde görünebilir. Bu nedenle, bir bileşen ile bir konteyner arasındaki ilişki, Şekil 12.1'de gösterildiği gibi dolu bir elmasla belirtilen bileşimdir.



Dikkat

Bir düzen yöneticisi sınıfından önce `new` operatörünü koymayı unutmayın; örneğin, `setLayout(new FlowLayout())`.



Not

Yapıcı `ShowFlowLayout()` açıkça `JFrame()` yapıcısını çağırır, ancak `JFrame()` yapıcısı dolaylı olarak çağırılır. Bakınız Bölüm 11.3.2, Yapıcı Zincirleme.

12.5.2 Izgara Düzeni

`GridLayout` yöneticisi bileşenleri bir ızgara (matris) düzeninde düzenler. Bileşenler, eklenme sırasına göre ilk satırdan başlayarak soldan sağa doğru ızgaraya yerleştirilir, ardından ikinci satır ve devam eder. `GridLayout` için sınıf diyagramı Şekil 12.6'da gösterilmiştir.

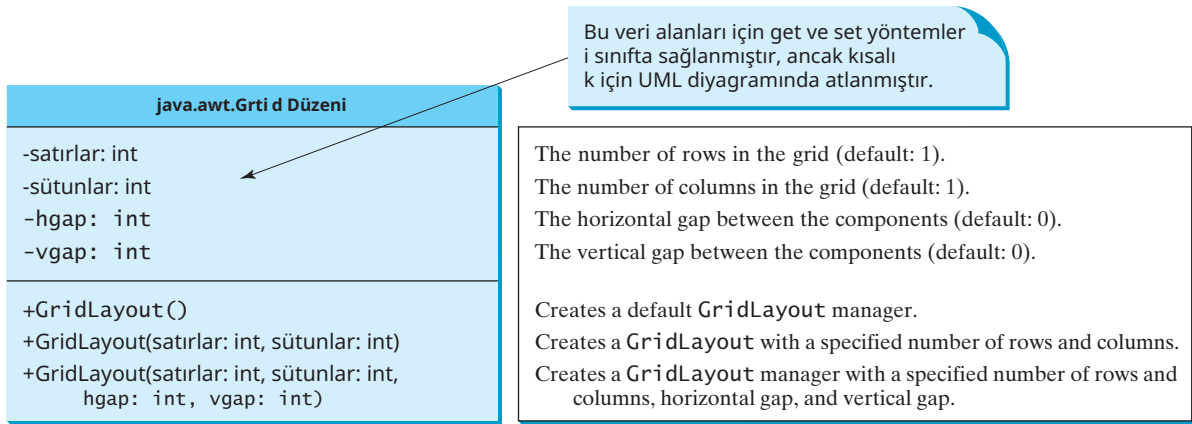


FIGURE 12.6 `GridLayout` bileşenleri bir ızgarada eşit boyutlu hücrelerde düzenler.

Izgaradaki satır ve sütun sayısını belirtebilirsiniz. Temel kurallar aşağıdaki gibidir:

- Satır sayısı veya sütun sayısı sıfır olabilir, ancak her ikisi birden olamaz. Birisıfır ve diğeri sıfırdan farklıysa, sıfır olmayan boyut sabitlenir, sıfır boyut iseyerleşim yöneticisi tarafından dinamik olarak belirlenir. Örneğin, on bileşeni olan bir ızgara içinsıfır satır ve üç sütun belirtirseniz, `GridLayout` dört satırlık üç sabit sütun oluşturur, son satırda bir bileşen bulunur. On bileşeni olan bir ızgara için üç satır ve sıfır sütun belirtirseniz, `GridLayout` dört sütundan oluşan üç sabit satır oluşturur, son satırda iki bileşen bulunur.

- Hem satır sayısı hem de sütun sayısı sıfırdan farklıysa, satır sayısı baskın parametredir; yani, satır sayısı sabittir ve yerleşim yöneticisi dinamik olarak sütun sayısını hesaplar. Örneğin, on bileşeni olan bir ızgara için üç satır ve üç sütun belirtirseniz, **GridLayout** dört sütundan oluşan üç satırlı ızgara oluşturur, son satırda iki bileşen bulunur.

Liste 12.4, ızgara düzenini gösteren bir programı vermektedir. Program, Liste 12.3'tekiyle benzerlik göstermektedir, ancak çerçeveye üç etiket ve üç metin alanı eklemektedir.

GridLayout yerine **FlowLayout** kullanılmıştır, Şekil 12.7'de gösterildiği gibi.

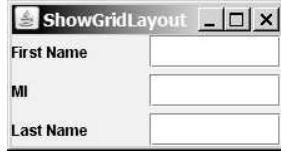


FIGURE 12.7 **GridLayout** yöneticisi konteyneri ızgaralara böler; ardından bileşenler hücreleri satır satır doldurmak için eklenir.

LİSTE 12.4 ShowGridLayout.java

```

1  import javax.swing.JLabel;
2  import javax.swing.JTextField;
3  import javax.swing.JFrame;
4  import java.awt.GridLayout;
5
6  genel classt ShowGridLayout extends JFrame {
7      public ShowGridLayout() {
8          // GridLayout'ı ayarla, 3 satır, 2 sütun ve bileşenler arasında yatay ve dikey olarak 5 boşluk
9          // bileşenler arasında
10         setLayout(new GridLayout(3, 2, 5, 5));
11
12         // Çerçeveye etiketler ve metin alanları ekle
13         add(new JLabel("İsim"));
14         add(new JTextField(8));
15         add(new JLabel("MI"));
16         add(new JTextField(1));
17         add(new JLabel("Soyisim"));
18         add(new JTextField(8));
19     }
20
21     /** Ana yöntem */
22     public static void main(String[] args) {
23         ShowGridLayout çerçeve = new ShowGridLayout();
24         çerçeve.setTitle("ShowGridLayout");
25         çerçeve.setSize(200, 125);
26         çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortala
27         çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28         çerçeve.setVisible(true);
29     }
30 }

```

düzeni ayarla

etiket ekle

metin alanı ekle

çerçeveyi oluştur

görünür yap

Çerçeveyi yeniden boyutlandırırsanız, bileşenlerin düzeni değişmez (yani, satır ve sütun sayısı değişmez ve boşluklar da değişmez).

Tüm bileşenler **GridLayout** konteynerinde eşit boyutta verilir.

setLayout ifadesini (satır 10) **setLayout(new GridLayout(3, 10))** ile değiştirmek yine de üç satır ve *iki* sütun elde eder. **sütunlar** parametresi göz ardı edilir.

çünkü **satırlar** parametresi sıfır değildir. Gerçek sütun sayısı

yerleşim yöneticisi tarafından hesaplanır.

setLayout ifadesi (satır 10) `setLayout(new GridLayout(4, 2))` veya `setLayout(new GridLayout(2, 2))` ile değiştirilse ne olurdu?

Lütfen kendiniz deneyin.



Not

FlowLayout ve **GridLayout** içinde, bileşenlerin konteynere eklenme sırası önemlidir. Sıra, bileşenlerin konteynerdeki konumunu belirler.

12.5.3 Sınır Düzeni

BorderLayout yöneticisi bir konteyneri beş alana böler: Doğu, Güney, Batı, Kuzey ve Merkez. Bileşenler, `add(Component, index)` kullanılarak **BorderLayout**'a eklenir; burada **index** bir sabittir **BorderLayout.EAST**, **BorderLayout.SOUTH**, **BorderLayout.WEST**, **BorderLayout.NORTH** veya **BorderLayout.CENTER**. **BorderLayout** için sınıf diyagramı Şekil 12.8'de gösterilmiştir.

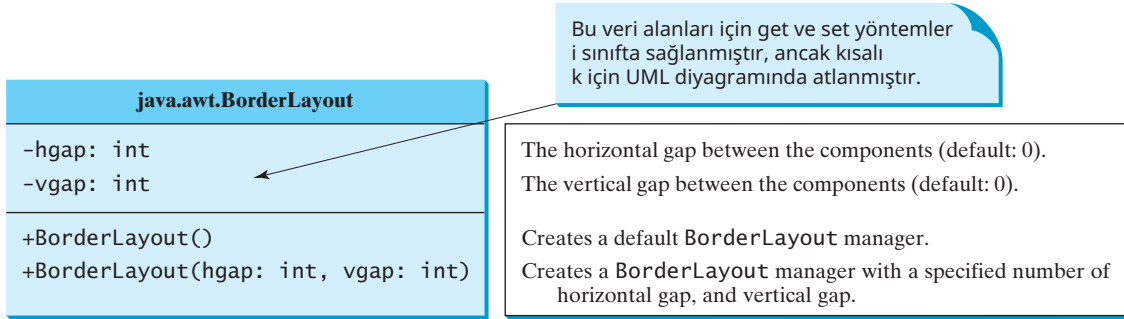


FIGURE 12.8 **BorderLayout** bileşenleri beş alanda düzenler.

Bileşenler, tercih edilen boyutlarına ve konteynerdeki yerleşimlerine göre düzenlenmiştir. Kuzey ve Güney bileşenleri yatay olarak uzanabilir; Doğu ve Batı bileşenleri dikey olarak uzanabilir; Merkez bileşeni, boş alanı doldurmak için hem yatay hem de dikey olarak uzanabilir.

Liste 12.5, sınır düzenini gösteren bir program vermektedir. Program, Şekil 12.9'da gösterildiği gibi, bir **BorderLayout** yöneticisi ile çerçeveye Doğu, Güney, Batı, Kuzey ve Merkez etiketli beş düğme ekler.



FIGURE 12.9 **BorderLayout** konteyneri beş alana böler; her biri bir bileşen tutabilir.

LİSTE 12.5 ShowBorderLayout.java

- 1 javax.swing.JButton'u içe aktarın;
- 2 import javax.swing.JFrame;
- 3 import java.awt.BorderLayout;


```

4
5 genel sınıfıShowBorderLayoutextendstJFrame {
6     publicvoidShowBorderLayout() {
7         // Yatay boşluk 5 ve dikey boşluk 10 ile BorderLayout ayarla
8         setLayout(yeniBorderLayout(t5t,t10t) );
9
10        // Düğmeleri çerçeveye ekle
11        add(new JButton("Doğu"), BorderLayout.EAST);
12        add(new JButton("Güney"), BorderLayout.SOUTH);
13        add(new JButton("Batı"), BorderLayout.WEST);
14        add(new JButton("Kuzey"), BorderLayout.NORTH);
15        add(new JButton("Merkez"), BorderLayout.CENTER);
16    }
17
18    /** Ana yöntem */
19    public static voidtmain(String[] args) {
20        ShowBorderLayout çerçevesi = newShowBorderLayout();
21        çerçeve.setTitle("ShowBorderLayout");
22        frame.setSize(300, 200);
23        frame.setLocationRelativeTo(null); // Çerçeveyi ortala
24        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25        frame.setVisible(true);

```

asla {revize e
dilmeli}

düzeni ayarla

butonlar ekleyin

çerçeveyi oluştur

görünür yap

Düğmeler çerçeveye eklenir (satırlar 11–15). **BorderLayout** için **add** yönteminin **FlowLayout** ve **GridLayout** için olanından farklı olduğunu unutmayın. **BorderLayout** ile bileşenlerin nereye yerleştirileceğini belirtirsiniz.

Bileşenlerin tüm alanları kaplaması gereksizdir. Programdan Doğu butonunu kaldırırmanız ve tekrar çalıştırırsanız, Orta butonun Doğu alanını kaplamak için sağa doğru uzandığını göreceksiniz.



Not

BorderLayout, bir indeks belirtiminin yokluğunu BorderLayout.CENTER olarak yorumlar. Örneğin, **add(component)**, **add(Component, BorderLayout.CENTER)** ile aynıdır. Eğer bir **BorderLayout** konteynerine iki bileşen eklerseniz, aşağıdaki gibi,

```
konteyner.add(bileşen1);ncontai
ner.add(bileşen2);
```

sadece son bileşen görüntülenir.

12.5.4 Yerleşim Yöneticilerinin Özellikleri

Yerleşim yöneticilerinin dinamik olarak değiştirilebilen özellikleri vardır.

- **FlowLayout**, **alignment**, **hgap** ve **vgap** özelliklerine sahiptir. Hizalamayı ve yatay ve dikey boşlukları belirtmek için **setAlignment**, **setHgap** ve **setVgap** yöntemlerini kullanabilirsiniz.
- **GridLayout**, **satırlar**, **sütunlar**, **hgap** ve **vgap** özelliklerine sahiptir. Satır sayısını, sütun sayısını ve yatay ve dikey boşlukları belirtmek için **setRows**, **setColumns**, **setHgap** ve **setVgap** yöntemlerini kullanabilirsiniz.
- **BorderLayout**, **hgap** ve **vgap** özelliklerine sahiptir. Yatay ve dikey boşlukları belirtmek için **setHgap** ve **setVgap** yöntemlerini kullanabilirsiniz.

Önceki bölümlerde, bir düzen yöneticisinin özellikleri oluşturulduktan sonra değişmediği için anonim bir düzen yöneticisi kullanılmıştır. Eğer bir düzen yöneticisinin özelliklerini dinamik olarak değiştirmek zorundaysanız, düzen yöneticisi bir değişkenle açıkça referans verilmelidir. Siz

değişken aracılığıyla düzen yöneticisinin özelliklerini değiştirebilirsiniz. Örneğin, aşağıdaki kod bir düzen yöneticisi oluşturur ve özelliklerini ayarlar:

```
// Bir düzen yöneticisi oluştur
FlowLayout flowLayout = newFlowLayout();

// Düzen özelliklerini ayarla
flowLayout.setAlignment(FlowLayout.RIGHT);nflowLa
yout.setHgap(t10t);
flowLayout.setVgap(t20t);
```

12.6 Program, Listing 12.3'teki 23. satırda ShowFlowLayout yerine

JFrame ile çalışır mı?

Program çalışır mı eğer ShowGridLayout Listing 12.4'te 23. satırda

JFrame ile değiştirilirse?

Program çalışır mı eğer ShowBorderLayout Listing 12.5'te 20. satırda

JFrame ile değiştirilirse?

12.7 Neden düzen yöneticileri kullanmanız gerekiyor? Bir

çerçeve için varsayılan düzen yöneticisi nedir? Bir bileşeni bir çerçeveye nasıl eklersiniz?

12.8 FlowLayout'u tanımlayın. Bir FlowLayout yöneticisi nasıl oluşturursunuz? Bir bileşeni bir FlowLayout konteynerine nasıl eklersiniz? Bir FlowLayout konteynerine eklenebilecek bileşen sayısında bir sınır var mı?Konteynerdeki bileşenler arasındaki yatay ve dikey boşlukları ayarlamak için hangi özellikler vardır? Hizalamayı belirtebilir misiniz?

12.9 GridLayout'u tanımlayın. Bir GridLayout yöneticisi nasıl oluşturursunuz? Bir bileşeni bir GridLayout konteynerine nasıl eklersiniz? Bir GridLayout konteynerine eklenebilecek bileşen sayısında bir sınır var mı?Konteynerdeki bileşenler arasındaki yatay ve dikey boşlukları ayarlamak için hangi özellikler vardır?

12.10 BorderLayout'u tanımlayın. Bir BorderLayout yöneticisi nasıl oluşturursunuz? Bir bileşeni bir BorderLayout konteynerine nasıl eklersiniz? Konteynerdeki bileşenler arasındaki yatay ve dikey boşlukları ayarlamak için hangi özellikler vardır?

12.11 Aşağıdaki program bir çerçevede bir düğme göstermesi bekleniyor, ancak hiçbir şey gösterilmiyor. Sorun nedir?

```
1 genel classtTesttextendstjavax.swing.JFrame {
2     publicTest() {
3         add(tnewtjavax.swing.JButton(t"OK"t));n4 }

4
5
6     public static voidtmain(String[] args) {
7         javax.swing.JFrame çerçevesi = tnewtjavax.swing.JFrame();
8         çerçeve.setSize(t100t,t200t);n9 çerçev
9         e.setVisible(tttrue);n10 }

11 }
```

12.6 Panelleri Alt Konteyner Olarak Kullanma



Bir konteyner başka bir konteynerin içine yerleştirilebilir. Paneller, istenen düzeni elde etmek için GUI bileşenlerini gruplamak amacıyla alt konteynerler olarak kullanılabilir.

On düğme ve bir metin alanını bir çerçeveye yerleştirmek istediğinizi varsayalım. Düğmeler ızgara düzeninde yerleştirilirken, metin alanı ayrı bir satıra yerleştirilir. Tüm bileşenleri tek bir konteynerde yerleştirerek istenen görünümü elde etmek zordur. Java GUI programlaması ile bir pencereyi panellere bölebilirsiniz. Paneller, kullanıcı arayüzü bileşenlerini gruplamak için alt konteynerler olarak işlev görür. Düğmeleri bir panelde ekler, ardından paneli çerçeveye eklersiniz.



Swing panelinin versiyonu **JPanel**'dir. Varsayılan bir **FlowLayout** yöneticisi ile bir panel oluşturma için **new JPanel()** kullanılabilir veya belirtilen düzen yöneticisi ile bir panel oluşturmak için **new JPanel(LayoutManager)** kullanabilirsiniz. Bir bileşeni panele eklemek için **add(Component)** yöntemini kullanın. Örneğin, aşağıdaki kod bir panel oluşturur ve ona bir düğme ekler:

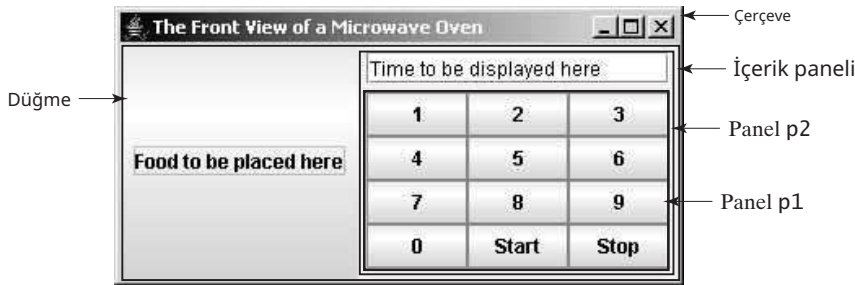
```
JPanel p = new JPanel(); np.add(
    new JButton("OK"));
```

Pencereler bir çerçeve içinde veya başka bir panelin içinde yerleştirilebilir. Aşağıdaki ifade

panel p'yi çerçeve f'ye yerleştirir:

```
f.add(p);
```

Liste 12.6, panellerin alt konteynerler olarak kullanılmasını gösteren bir örnek vermektedir. Program bir mikrodalga fırın için bir kullanıcı arayüzü oluşturur, Şekil 12.10'da gösterildiği gibi.



ŞEKİL 12.10 Program bileşenleri düzenlemek için panelleri kullanır.

LISTING 12.6 TestPanels.java

```
1 import java.awt.*;
2 import javax.swing.*;
3
4 genel sınıf TestPanel extends JFrame {
5     public TestPanel() {
6         // Düğmeler için p1 panelini oluştur ve GridLayout ayarla
7         JPanel p1 = new JPanel();
8         p1.setLayout(new GridLayout(4, 3));
9
10        // Panele düğmeleri ekle
11        for (int i = 1; i <= 9; i++) {
12            p1.add(new JButton("t" + i));
13        }
14
15        p1.add(new JButton("0"));
16        p1.add(new JButton("Başlat"));
17        p1.add(new JButton("Durdur"));
18
19        // Metin alanı ve p1'i tutmak için p2 panelini oluştur
20        JPanel p2 = new JPanel(new BorderLayout());
21        p2.add(new JTextField("Burada gösterilecek zaman"),
22            BorderLayout.KUZEY);
23        p2.add(p1, BorderLayout.CENTER);
24
25        // Çerçeveye içerik ekle
26        add(p2, BorderLayout.EAST);
27        add(new JButton("Buraya yerleştirilecek yiyecek"),
28            BorderLayout.CENTER);
29    }
30 }
```

paneli p1

paneli p2

p2'yi çerçeveye ekle

```

31  /** Ana yöntem */
32  public static voidtmain(String[] args) {
33      TestPanels frame = newTestPanels();
34      frame.setTitle("Bir Mikrodalga Fırının Ön Görünümü");
35      frame.setSize(400, 250);
36      frame.setLocationRelativeTo(null); // Çerçeveyi ortalayalım
37      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38      frame.setVisible(true);
39  }n40
}

```

setLayout yöntemi **java.awt.Container** içinde tanımlanmıştır. **JPanel** bir alt sınıf olduğundan **Container**, panelde yeni bir düzen yöneticisi ayarlamak için **setLayout** kullanabilirsiniz (satır 8). Satırlar 7–8, **JPanel p1 = new JPanel(new GridLayout(4, 3))** ile değiştirilebilir.

İstenilen düzeni elde etmek için program, sayı düğmelerini, **Stop** düğmesini ve **Start** düğmesini gruplamak için **GridLayout**'ün paneli **p1**'i kullanır ve bir metin alanını kuzeyde ve **p1**'i merkezde tutmak için **BorderLayout**'ün paneli **p2**'yi kullanır. Yiyeceği temsil eden düğme çerçevenin ortasına yerleştirilir ve **p2** çerçevenin doğusuna yerleştirilir.

Açıklama (satırlar 21–22)

```

p2.add(newJTextField("Burada gösterilecek zaman"),
BorderLayout.NORTH);

```

bir **JTextField** örneği oluşturur ve bunu **p2**'ye ekler. **JTextField**, kullanıcı girişi için kullanılabilen ve değerleri görüntülemek için de kullanılabilen bir GUI bileşenidir.

Üst sınıfContainer



Not

Container sınıfının, JButton gibi GUI bileşen sınıflarının üst sınıfı olduğunu belirtmekte fayda var. Her GUI bileşeni bir konteynirdir. Teorik olarak, bir düğümde düzeni ayarlamak için **setLayout** yöntemini kullanabilir ve bileşenleri bir düğümde ekleyebilirsiniz, çünkü Container sınıfındaki tüm public yöntemler JButton tarafından miras alınır, ancak pratik nedenlerden dolayı düğmeleri konteynir olarak kullanmamalısınız.

12.12 Belirtilen bir düzen yöneticisi ile bir panel nasıl oluşturulur?

12.13 Bir **JPanel** için varsayılan düzen yöneticisi nedir? Bir bileşeni

JPanel'ye nasıl eklersiniz?

12.14 Bir panelde **setTitle** yöntemini kullanabilir misiniz? Bir panel kullanmanın amacı nedir?

12.15 **JButton** gibi bir GUI bileşen sınıfı **Container**'ın bir alt sınıfı olduğuna göre, bir düğümde bileşen ekleyebilir misiniz?

12.7 Color Sınıfı



Anahtar
Nokta

*Her GUI bileşeninin arka plan ve ön plan renkleri vardır. Renkler, **Color** sınıfında oluşturulan nesnelerdir.*

GUI bileşenleri için renkleri **java.awt.Color** sınıfını kullanarak ayarlayabilirsiniz. Renkler, her biri yoğunluğunu tanımlayan bir **int** değeri ile temsil edilen kırmızı, yeşil ve mavi bileşenlerden oluşur; bu değerler **0** (en koyu ton) ile **255** (en açık ton) arasında değişir. Bu, *RGB modeli* olarak bilinir.

Aşağıdaki yapıcuyu kullanarak bir renk oluşturabilirsiniz:

```
publicColor(tinttr,tinttg,tinttb);
```

burada **r,g** ve **b** bir rengi kırmızı, yeşil ve mavi bileşenleri ile tanımlar. Örneğin,

```
Renk rengi = tnewtColor(t128t,t100t,t100t);
```



Not

argümanlar **r,g,b 0** ile **255** arasında olmalıdır. Eğer bu aralığın dışında bir değer argüman geçirilirse, bir **IllegalArgumentException** oluşacaktır.

java.awt.Component sınıfında tanımlanan setBackground(Color c) ve setForeground(Color c) yöntemlerini kullanarak bir bileşenin arka plan ve ön plan renklerini ayarlayabilirsiniz. İşte bir düğmenin arka plan ve ön planını ayarlama örneği:

```

JButton jbtOK = tnewtJButton(t"OK"t);njbto
K.setBackground(color);njbtoK.set
Foreground(tnewtColor(t100t,t1t,t1t));

```

Alternatif olarak, java.awt.Color içinde sabitler olarak tanımlanan 13 standart renkten birini (SİYAH, MAVİ, SİYAH, Koyu Gri, Gri, YEŞİL, AÇIK GRI, MACENTA, TURUNCU, PEMBE, KIRMIZI, BEYAZ, ve SARI) kullanabilirsiniz. Aşağıdaki kod, örneğin, bir butonun ön plan rengini kırmızıya ayarlar:

```
jbtOK.setForeground(Color.RED);
```

12.16 Renk nasıl oluşturulur? newColor(400, 200, 300) kullanarak bir Color oluşturma nın yanlış olan yanı nedir? new Color(10, 0, 0) ile new Color(200, 0, 0) Point Kontrol Et arasından hangisi daha doğru?

12.17 Rastgele bir renk ile bir Color nesnesi nasıl oluşturulur?

12.18 jbtOK nesnesine mavi arka plan nasıl ayarlanır?

MyProgrammingLab™

12.8 Font Sınıfı

Her GUI bileşenin font özelliği vardır. Fontlar, Font sınıfından oluşturulan nesnelerdir.

java.awt.Font sınıfını kullanarak bir font oluşturabilir ve bileşenler için fontları ayarlamak için thesetFont() yöntemini Component sınıfında kullanabilirsiniz.

Font için yapıcı:

```
publictFont(Dize adı, renk tonu stili, renk tonu);
```

SansSerif, Serif, Monospaced, Dialog ve DialogInput içinden bir font adı seçebilir, Font.PLAIN(0), Font.BOLD(1), Font.ITALIC(2) içinden bir stil seçebilir ve Font.BOLD, Font.ITALIC + Font.ITALIC(3), ve pozitif bir tam sayı font boyutu belirtin. İçin örneğin, aşağıdaki ifadeler iki font oluşturur ve bir fontu bir düğmeye ayarlar.

```

Yazı tipi font1 = tnewtFont(t"SansSerif"t, Font.BOLD, t16t);nFont
font2 = tnewtFont(t"Serif"t, Font.BOLD + Font.ITALIC, t12t);nJButton jbtOK = t

```

```

newtJButton(t"OK"t);
jbtOK.setFont(yazı tipi1);

```



Font

Sisteminiz "Times New Roman" gibi diğer yazı tiplerini destekliyorsa, bir Font nesnesi oluşturmak için bu yazı tipini kullanabilirsiniz. Sisteminizde mevcut olan yazı tiplerini bulmak için, java.awt.GraphicsEnvironment sınıfının statik yöntemi olan getLocalGraphicsEnvironment() kullanarak bir örneğini elde etmeniz gerekir. GraphicsEnvironment, belirli bir sistemdeki grafik ortamını tanımlayan soyut bir sınıftır. Sistemdeki mevcut tüm yazı tiplerini elde etmek için getAllFonts() yöntemini ve mevcut tüm yazı tiplerinin adlarını elde etmek için getAvailableFontFamilyNames() yöntemini kullanabilirsiniz. Örneğin, aşağıdaki ifadeler sistemdeki tüm mevcut yazı tipi adlarını yazdırır:

```

Grafik Ortamı e =
GraphicsEnvironment.getLocalGraphicsEnvironment();nString[]
fontnames = e.getAvailableFontFamilyNames();nfont(tintti = t0t; i < f
ontnames.length; i++)
System.out.println(yazı tipi adları[i]);

```



Anahtar
Nokta



12.19 Yazı tipi adı **Courier**, boyut **20** ve stil **bold** olan bir **Font** nesnesi nasıl oluşturursunuz?

12.20 Sisteminizdeki tüm mevcut yazı tiplerini nasıl bulursunuz?

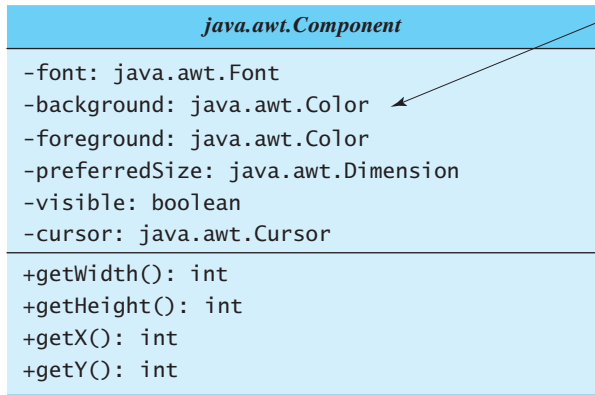
12.9 Swing GUI Bileşenlerinin Ortak Özellikleri

GUI bileşenlerinin ortak özellikleri vardır. Bunlar süper sınıflarda tanımlanmıştır.

Bileşen, Konteyner, ve JComponent.

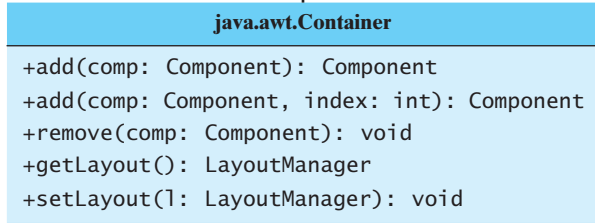
Bu bölümde şimdiye kadar birkaç GUI bileşeni (örneğin, **JFrame**, Konteyner, **JPanel**, **JButton**, **JLabel**, ve **TextField**) kullandınız. Bu kitapta daha birçok GUI bileşeni tanıtılacaktır. Swing GUI bileşenlerinin ortak özelliklerini anlamak önemlidir. **Component** sınıfı, tüm GUI bileşenleri ve konteynerler için kök sınıftır. Tüm Swing GUI bileşenleri (tüm **JFrame**, **JApplet**, ve **JDialog** hariç) **JComponent** sınıfının alt sınıflarıdır, Şekil 12.1'de gösterildiği gibi. Şekil 12.11, font, renk, fare imleci, boyut, araç ipucu metni ve kenarlık gibi özellikleri manipüle etmek için **Component**, Konteyner, ve **JComponent** içindeki bazı sık kullanılan yöntemleri listelemektedir.

Bir *araç ipucu*, fareyi bir bileşenin üzerine getirdiğinizde bileşende görüntülenen metindir. Genellikle bir bileşenin işlevini tanımlamak için kullanılır.



Bu veri alanları için get ve set yöntemleri sınıfta sağlanmıştır, ancak kısaltılmış için UML diyagramında atlanmıştır.

The font of this component.
The background color of this component.
The foreground color of this component.
The preferred size of this component.
Indicates whether this component is visible.
The mouse cursor shape.
Returns the width of this component.
Returns the height of this component.
getX() and **getY()** return the coordinate of the component's upper-left corner within its parent component.



Bu veri alanları için get ve set yöntemleri sınıfta sağlanmıştır, ancak kısaltılmış için UML diyagramında atlanmıştır.

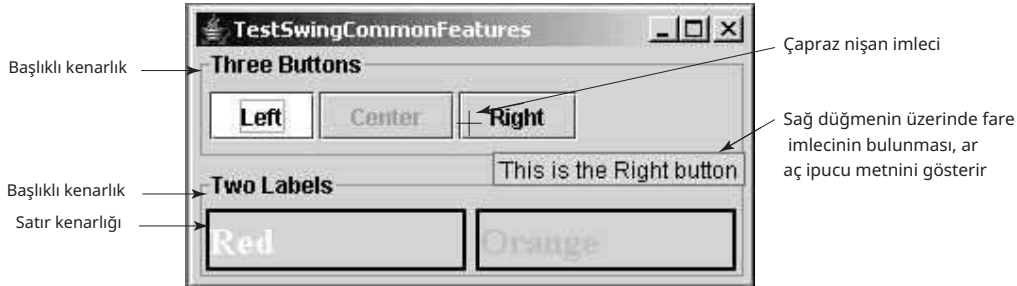
The tool tip text for this component. Tool tip text is displayed when the mouse points on the component without clicking.
The border for this component.

ŞEKİL 12.11 Tüm Swing GUI bileşenleri, **Component**, **Konteyner**, ve

JComponent 'den kamu yöntemlerini miras alır.

JComponent sınıfının herhangi bir nesnesi için bir kenarlık ayarlayabilirsiniz. Swing'in bir kenarlık kenarlığı vardır. Başlıklı bir kenarlık oluşturmak için `new TitledBorder(String title)` kullanın. Bir çizgi kenarlığı oluşturmak için `new LineBorder(Color color, int width)` kullanın; burada `width`, çizginin kalınlığını belirtir.

Liste 12.7, Swing'in ortak özelliklerini göstermek için bir örnektir. Örnek, üç düğmeyi tutmak için bir panel **p1** (satır 8) ve iki etiketi tutmak için bir panel **p2** (satır 26) oluşturur; bu, Şekil 12.12'de gösterilmiştir. **jbtLeft** düğmesinin arka planı beyaz (satır 12) ve **jbtCenter** düğmesinin ön planı yeşil (satır 13) olarak ayarlanmıştır. **jbtRight** düğmesinin araç ipucu satır 14'te ayarlanmıştır. Başlıklı kenarlıklar **p1** ve **p2** panellerinde (satırlar 18, 37) ve etiketlerde satır kenarlıkları (satırlar 33–34) ayarlanmıştır.



ŞEKİL 12.12 Yazı tipi, renk, kenarlık ve araç ipucu metni mesaj panelinde ayarlanmıştır.

Fare imleci, **p1** (satır 19) içinde çapraz nişan şekline ayarlanmıştır. **Cursor** sınıfı, fare imleci `ULT_CURSORCROSSHAIR_CURSOR()`, `HAND_CURSOR` gibi imleç şekillerini belirtir (`Cursor`), `MOVE_CURSOR()`, `TEXT_CURSOR()`, ve devamı. Çapraz nişan imleci için bir **Cursor** nesnesi, `newCursor(Cursor.CROSSHAIR_CURSOR)` (satır 19) kullanılarak oluşturulur ve bu imleç **p1** için ayarlanır. Varsayılan fare imlecini hala **p2** içinde kullanıldığını unutmayın, çünkü program **p2** için açıkça bir fare imleci ayarlamamıştır.

LİSTELER 12.7 TestSwingCommonFeatures.java

```

1  import java.awt.*;
2  import javax.swing.*;
3  import javax.swing.border.*;
4
5  genel sınıf TestSwingCommonFeatures extends JFrame {
6      public TestSwingCommonFeatures() {
7          // Üç düğmeyi gruplamak için bir panel oluşturun
8          JPanel p1 = new JPanel(new FlowLayout(FlowLayout.LEFT, 2, 2));
9          JButton jbtLeft = new JButton("Sol");
10         JButton jbtCenter = new JButton("Merkez");
11         JButton jbtRight = new JButton("Sağ");
12         jbtLeft.setBackground(Color.WHITE);
13         jbtCenter.setForeground(Color.GREEN);
14         jbtRight.setToolTipText("Bu Sağ buton");
15         p1.add(jbtLeft);
16         p1.add(jbtCenter);
17         p1.add(jbtRight);
18         p1.setBorder(new TitledBorder("Üç Buton"));
19         p1.setCursor(new Cursor(Cursor.CROSSHAIR_CURSOR));
20
21         // Bir yazı tipi ve bir kenar sınırı oluşturun
22         Font largeFont = new Font("TimesRoman", Font.BOLD, 20);
23         Sınır lineBorder = new LineBorder(Color.BLACK, 2);

```

arka plan ayarla
ön plan ayarla
bilgi ipucu metnini ayarla

başlıklı kenarlık ayarla
fare imlecini ayarla

bir font oluştur
bir kenarlık oluştur

ön plan ayarla

yazı tipini ayarla

çizgi kenarlığı ayarla

başlık kenarlığı ayarla

```

2425 // İki etiketi gruplamak için bir panel oluştur
26 JPanel p2 = new JPanel(new GridLayout(1, 2, 5, 5));
27 JLabel jlblRed = new JLabel("Kırmızı");
28 JLabel jlblOrange = new JLabel("Turuncu");
29 jlblRed.setForeground(Color.RED);
30 jlblRed.setForeground(Color.ORANGE);
31 jlblOrange.setFont(largeFont);jlblOrange.set
32 jlblOrange.setFont(büyükYazıTipi);
33 jlblRed.setBorder(lineBorder);jlblRed.
34 jlblOrange.setBorder(çizgiKenar);
35 p2.add(jlblRed);
36 p2.add(jlblTuruncu);
37 p2.setBorder(new TitledBorder("İki Etiket"));
38
39 // Çerçeveye iki panel ekle
40 setLayout(new GridLayout(2, 1, 5, 5));
41 add(p1);
42 ekle(p2);n43 }

44
45 public static void tmain(String[] args) {
46 // Bir çerçeve oluştur ve özelliklerini ayarla
47 JFrame çerçeve = tnewTestSwingCommonFeatures();n48 çerç
ve.setTitle(t"TestSwingCommonFeatures"t);n49 çerçeve.setSize(300t,150t);
50 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
51 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
52 frame.setVisible(true);
53 }n54
}

```

özellik varsayılan değerleri



Not

Aynı özellik, farklı bileşenlerde farklı varsayılan değerlere sahip olabilir. Örneğin, the **visible** özelliği **JFrame** içinde varsayılan olarak **false** iken, her **JComponent** örneğinde (örneğin, **JButton** ve **JLabel**) varsayılan olarak **true**'dur. Bir **JFrame** görüntülemek için, the **setVisible(true)** çağırarak **visible** özelliğini **true** olarak ayarlamanız gerekir, ancak bu özelliği bir **JButton** veya **JLabel** için ayarlamanıza gerek yoktur, çünkü zaten **true**'dur. Bir **JButton** veya **JLabel**'ı görünmez yapmak için, **setVisible(false)** çağırabilirsiniz. Lütfen programı çalıştırın ve 38. satıra aşağıdaki iki ifadeyi ekledikten sonra etkiyi görün:

```

jbtLeft.setVisible(tfalset);jlblRed.
setVisible(tfalset);

```

12.21 Swing GUI bileşeninde arka plan rengi, ön plan rengi, yazı tipi ve araç ipucu metnini nasıl ayarlarsınız?

12.22 Aşağıdaki kodda araç ipucu metni neden görüntülenmiyor?

```

1 import javax.swing.*;
2
3 genel class Test extends JFrame {
4     private JButton jbtOK = tnew JButton(t"OK"t);
5
6     public static void tmain(String[] args) {
7         // Bir çerçeve oluştur ve özelliklerini ayarla
8         JFrame çerçeve = tnewTest();n9 çerçeve.
setTitle(t"Mantık Hatası"t);n10 çerçeve.setSize(200t,100t);
11         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

```

Point Kontrol Et

MyProgrammingLab™


```

12 çerçeve.setVisible(true);n13 }

14
15 publicTest() {
16     jbtOK.setToolTipText("Bu bir düğmedir");
17     add(newJButton(t"Tamam"t) );
18 }
19 }

```

12.23 Aşağıdaki kodun çıktısını göster:

```

import javax.swing.*;

genel sınıfTest {
    public static void main(String[] args) {
        JButton jbtOK = new JButton(t"OK"t);
        System.out.println(jbtOK.isVisible());
        JFrame çerçeve = new JFrame(

        me();
        System.out.println(frame.isVisible());
    }
}

```

12.24 Aşağıda gösterildiği gibi bir düğmeyi bir konteynıra birkaç kez eklerseniz ne olur? Bu

sözdizimi hatalarına neden olur mu? Çalışma zamanı hatalarına neden olur mu?

```

JButton jbt = new JButton();
JPanel paneli = new JPanel();
paneli.add(jbt);
paneli.add(jbt);
paneli.add(jbt);

```

12.10 Görüntü Simgeleri

Görüntü simgeleri birçok GUI bileşeninde görüntülenebilir. Görüntü simgeleri, **ImageIcon** sınıfı kullanılarak oluşturulan nesnelerdir.



Bir simge, sabit boyutlu bir resimdir; genellikle küçük olup bileşenleri süslemek için kullanılır. Görüntüler genellikle resim dosyalarında saklanır. Java şu anda üç resim formatını desteklemektedir: GIF (Grafik Değişim Formatı), JPEG (Ortak Fotoğraf Uzmanları Grubu) ve PNG (Taşınabilir Ağ Grafikleri). Bu türlerin resim dosyası adları sırasıyla **.gif**, **.jpg** ve **.png** ile biter. Eğer bir bitmap dosyanız veya diğer formatlarda resim dosyalarınız varsa, bunları Java'da kullanılmak üzere GIF, JPEG veya PNG formatına dönüştürmek için görüntü işleme araçlarını kullanabilirsiniz.

Bir görüntü simgesini görüntülemek için önce **ImageIcon** nesnesi oluşturun ve **new javax.swing.ImageIcon(filename)** kullanın. Örneğin, aşağıdaki ifade, mevcut sınıf yolunda **image** dizininde bulunan bir resim dosyasından **us.gif** simgesi oluşturur:

```
ImageIcon icon = new ImageIcon(t"image/us.gif"t);
```

resim-dosya formatı

create ImageIcon

image/us.gif, **c:\ookimage\us.gif** konumundadır. Ters eğik çizgi (**/**) Windows dosya yolu notasyonudur. UNIX'te, eğik çizgi (**/**) kullanılmalıdır. Java'da, eğik çizgi (**/**) Java sınıf yolunda göreceli bir dosya yolunu belirtmek için kullanılır (örneğin, **image/us.gif**, bu örnekte olduğu gibi).

dosya yolu karakteri

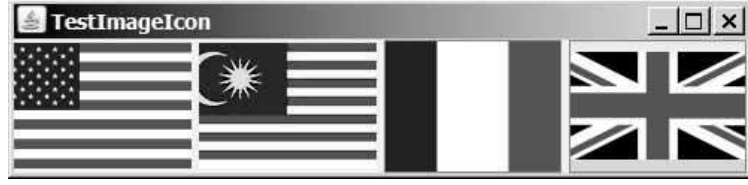


Tttip

Dosya adları Windows'ta büyük/küçük harf duyarlıdır ancak UNIX'te büyük/küçük harf duyarlıdır. Programlarınızın tüm platformlarda çalışabilmesi için, tüm resim dosyalarını tutarlı bir şekilde, küçük harf kullanarak adlandırın.

dosyaları tutarlı bir şekilde adlandırma

Bir resim simgesi, `new JLabel(imageIcon)` or `new JButton(imageIcon)` kullanılarak bir etiket veya düğme içinde görüntülenebilir. Liste 12.8, etiketlerde ve düğmelerde simgelerin görüntülenmesini göstermektedir. Örnek, Şekil 12.13'te gösterildiği gibi simgelerle iki etiket ve iki düğme oluşturur.



Fiçe 12.13 Görüntü simgeleri etiketlerde ve butonlarda görüntülenir.

Listing 12.8 TestImageIcon.java

```

1  import javax.swing.*;
2  import java.awt.*;
3
4  genel classt TestImageIcon extends JFrame {
5      Privatet ImageIcon usIcon = tnew ImageIcon("image/us.gif");
6      Privatet ImageIcon myIcon = tnew ImageIcon("image/my.jpg");
7      özel t ImageIcon frIcon = tnew ImageIcon("image/for.gif");
8      Privatet ImageIcon ukIcon = tnew ImageIcon("image/uk.gif");
9
10     public TestImageIcon() {
11         setLayout(tnew GridLayout(2, 2));
12         tnew JLabel(usIcon);
13         add(tnew JLabel(myIcon));
14         add(tnew JButton(frIcon)); nadd(
15         ekleme( tnew JButton(ukIcon));
16     }
17
18     /** Ana yöntem */
19     public static void tmain(String[] args) {
20         TestImageIcon çerçeve = tnew TestImageIcon(); n21 çerçeve.
21         setTitle("TestImageIcon");
22         çerçeve.setSize(200, 200);
23         çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortala
24         çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25         çerçeve.setVisible(true);
26     }
27
28     asla {revize e
29     dilmedi}

```

sınırları ve simgeleri paylaşma



Not

Sınırlar ve simgeler paylaşılabilir. Böylece, bir sınır veya simge oluşturabilir ve bunu herhangi bir GUI bileşeni için sınır veya simge özelliğini ayarlamak için kullanabilirsiniz. Örneğin, aşağıdaki ifadeler p1 ve p2 panelleri için bir sınır b ayarlar:

```
p1.setBorder(b); p2.
setBorder(b);
```

Aşağıdaki ifadeler jbt1 ve jbt2 butonlarında bir simge ayarlar:

```
jbt1.setIcon(icon); njbt2.
setIcon(icon);
```



tp

Bir splash screen, uygulama başlatılırken görüntülenen bir resimdir. Eğer programınız uzun bir süre yükleniyorsa, kullanıcıyı bilgilendirmek için bir splash screen gösterebilirsiniz. Örneğin, aşağıdaki komut:

```
java -sıra:image/us.gif TestImageIcon
```

program yüklenirken **TestImageIcon** bir resmi görüntüler.

karşılama ekranı

12.25 Sınıf dizindeki `image/us.gif` dosyasından bir `ImageIcon` nasıl oluşturursunuz?

12.26 Aşağıdaki kod üç buton gösterecek mi? Butonlar aynı simgeyi mi gösterecek?

```
1  import javax.swing.*;
2  import java.awt.*;
3
4  genel class Test extends JFrame {
5      public static void main(String[] args) {
6          // Bir çerçeve oluştur ve özelliklerini ayarla
7          JFrame çerçeve = new JFrame();
8          çerçeve.setTitle("Button Icons");
9          çerçeve.setSize(200, 100);
10         çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         çerçeve.setVisible(true);
12     }
13
14     public Test() {
15         ImageIcon usIcon = new ImageIcon("image/us.gif");
16         JButton jbt1 = new JButton(usIcon);
17         JButton jbt2 = new JButton(usIcon);
18
19         JPanel p1 = new JPanel();
20         p1.add(jbt1);
21
22         JPanel p2 = new JPanel();
23         p2.add(jbt2);
24
25         JPanel p3 = new JPanel();
26         p3.add(jbt1);
27
28         add(p1, BorderLayout.NORTH);
29         add(p2, BorderLayout.SOUTH);
30         add(p3, BorderLayout.CENTER);
31     }
32 }
```

Point Kontrol Et

MyProgrammingLab™

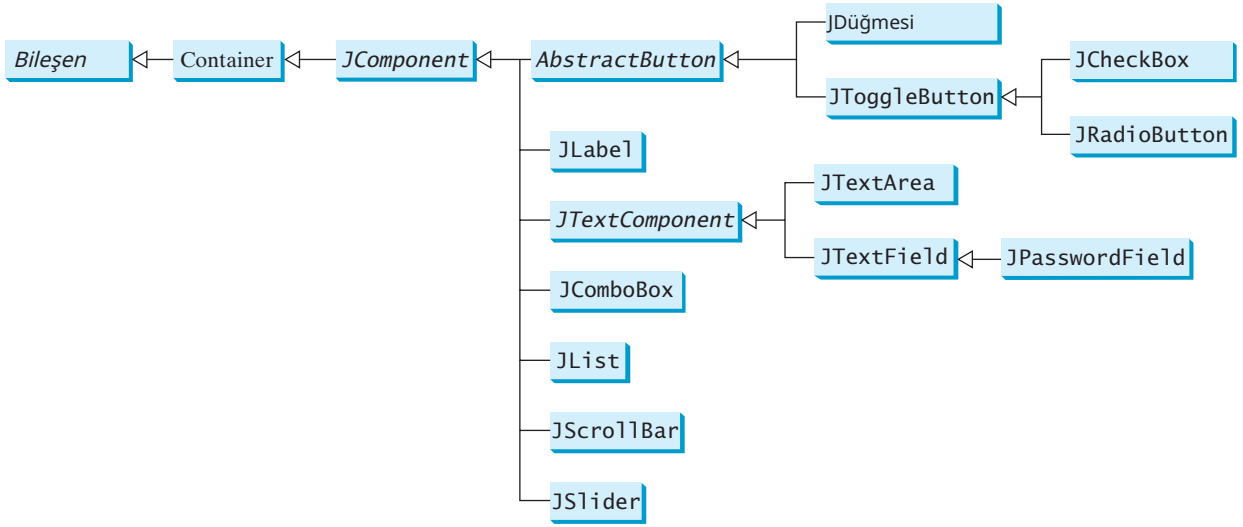
12.27 GUI bileşenleri tarafından bir kenarlık veya simge paylaşılabilir mi?

12.11t Düğmesi

Bir buton oluşturmak için **JButton** sınıfını kullanın.

Örneklerde GUI programlamanın temellerini göstermek için **JButton** kullandık. Bu bölüm **JButton**'ün daha fazla özelliğini tanıttı. Aşağıdaki bölümler GUI bileşenleri **JCheckBox**, **JRadioButton**, **JLabel**, **JTextField** ve **JPasswordField**'ü tanıttı. Daha fazla GUI bileşeni olan **JTextArea**, **JComboBox**, **JList**, **JScrollbar** ve **Slider**, Bölüm 17'de tanıtılacaktır. Bu sınıfların ilişkisi Şekil 12.14'te gösterilmektedir.





FIGÜR 12.14 Bu Swing GUI bileşenleri, kullanıcı arayüzleri oluşturmak için sıklıkla kullanılır.

bileşenler için adlandırma kuralı

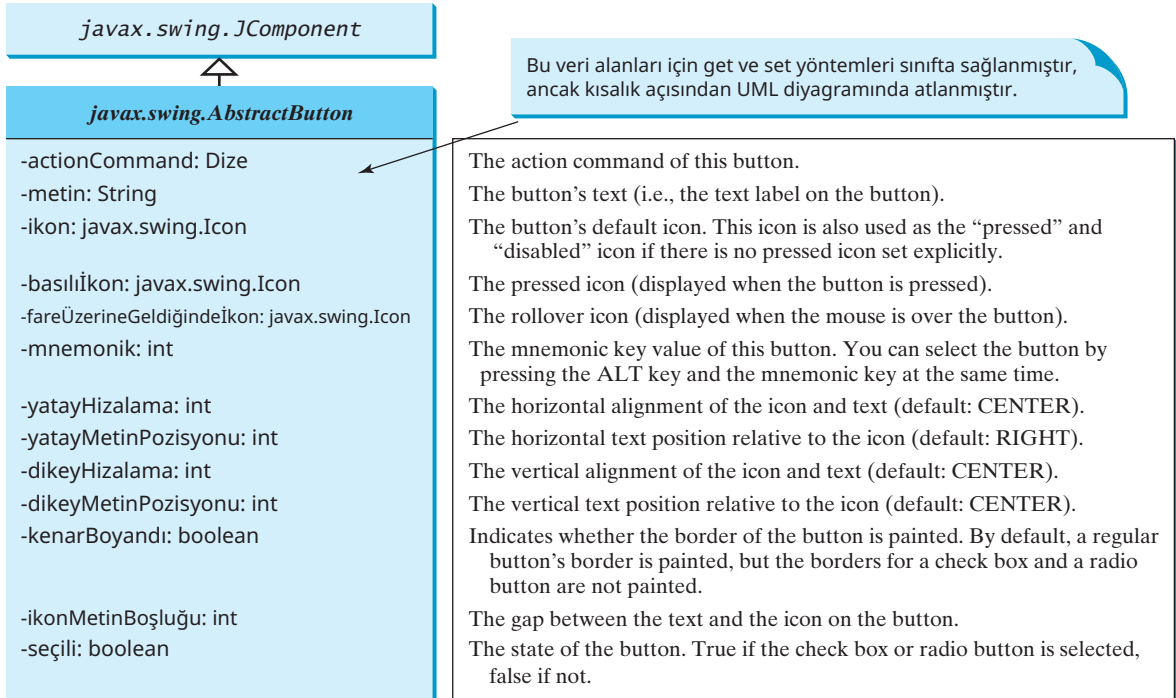


Not

Bu kitap boyunca, **jbt, jchk, jrb, jlb1, jtf, jpf, jta, jcbo, jlst, jsch** ve **jsld**, **JButton**, **JCheckBox**, **JRadioButton**, **JLabel**, **JTextField**, **JPasswordField**, **JTextArea**, **JComboBox**, **JList**, **JScrollbar** ve **JSlider** için referans değişkenlerini adlandırmak için kullanılır.

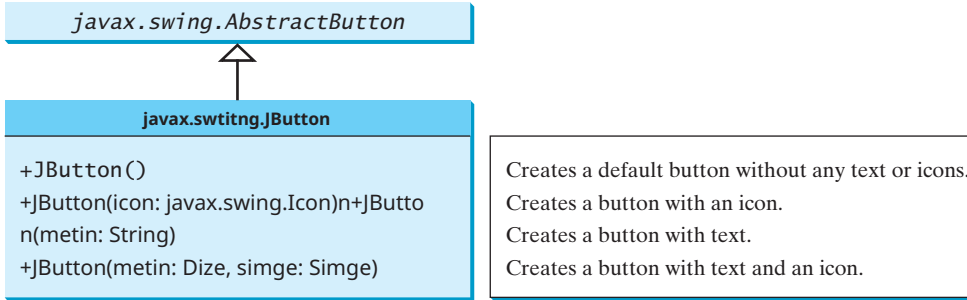
Bir *buton*, tıklandığında bir eylemi tetikleyen bir bileşendir. Swing, normal butonlar, toggle butonları, onay kutusu butonları ve radyo butonları sağlar. Bu butonların ortak özellikleri **javax.swing.AbstractButton** içinde tanımlanmıştır, Şekil 12.15'te gösterildiği gibi.

SoyutDüğme



ŞEKİL 12.15 AbstractButton, farklı türdeki düğmelerin ortak özelliklerini tanımlar.

JButton, AbstractButton'dan miras alır ve düğme oluşturmak için birkaç yapıcı sağlar, Şekil 12.16'da gösterildiği gibi.



FIGÜRİK12.16 JButton normal bir butonu tanımlar.

12.11.1 İkonlar, Basılı İkonlar ve Üzerine Geline İkonlar

Bir butonun varsayılan bir ikon, basılı bir ikon ve üzerine gelinen bir ikon vardır. Normalde varsayılan ikon kullanırsınız, çünkü diğer ikonlar özel efektler içindir. Bir buton basıldığında basılı ikon görüntülenir ve fare butonun üzerinde ancak basılmadığında üzerine gelinen ikon görüntülenir. Örneğin, Liste 12.9, ABD bayrağını normal bir ikon olarak, Kanada bayrağını basılı ikon olarak ve Britanya bayrağını üzerine gelinen ikon olarak gösterir; bu, Şekil 12.17'de gösterilmiştir.

LISTE12.9 TestButtonIcons.java

```

1  javax.swing.*'i içe aktarın;
2
3  genel sınıf TestButtonIcons textendst JFrame {
4      public static void main(String[] args) {
5          // Bir çerçeve oluştur ve özelliklerini ayarla
6          JFrame frame = new TestButtonIcons();
7          frame.setTitle("Buton Simgeleri");
8          çerçeve.setSize(200t,100t);
9          frame.setLocationRelativeTo(null); // Çerçeveyi ortalayın
10         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         frame.setVisible(true);
12     }
13
14     genel TestButtonIcons() {
15         ImageIcon usIcon = ImageIcon("image/usIcon.gif");
16         ImageIcon caIcon = ImageIcon("image/caIcon.gif");
17         ImageIcon ukIcon = ImageIcon("image/ukIcon.gif");
18
19         JButton jbt = new JButton("Tıklayın", usIcon);
20         jbt.setPressedIcon(caIcon);
21         jbt.setRolloverIcon(ukIcon);
22
23         ekle(ele geçir);
24     }
25 }

```

simge oluştur

normal simge
basılı simge
üzerine gelindiğinde simge

düğme ekle



(a) Varsayılan simge



(b) Basılı simge



(c) Üzerine gelindiğinde simge

FIGURE12.17 Bir düğmenin birden fazla simge türü olabilir.

12.11.2 Hizalamalar

yatay hizalama

Yatay hizalama simge ve metnin bir düğme üzerinde yatay olarak nasıl yerleştirileceğini belirtir. Yatay hizalamayı `setHorizontalAlignment(int)` ile beş sabitten biriyle ayarlayabilirsiniz: `LEADING`, `SOL`, `MERKEZ`, `SAĞ` veya `TRAILING`, Şekil 12.18'de gösterildiği gibi. Şu anda, `LEADING` ve `SOL` aynıdır, ve `TRAILING` ve `SAĞ` da aynıdır. Gelecekteki uygulama bunları ayırabilir. Varsayılan yatay hizalama `AbstractButton.CENTER`'dir.



FIGURE 12.18 Düğmenin üzerinde simge ve metnin nasıl yerleştirileceğini belirtebilirsiniz.

dikey hizalama

Dikey hizalama, simgenin ve metnin bir düğme üzerindeki dikey yerleşimini tanımlar. Dikey hizalamayı `setVerticalAlignment(int)` kullanarak, üç sabitten biri olan `ÜST`, `MERKEZ` veya `ALT` ile ayarlayabilirsiniz; bu, Şekil 12.19'da gösterilmiştir. Varsayılan dikey hizalama `AbstractButton.CENTER`'dir.

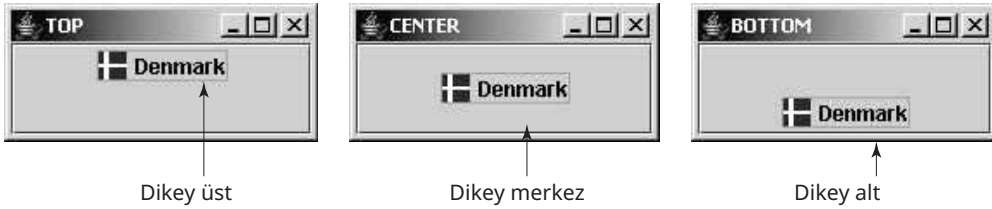


FIGURE 12.19 Simge ve metnin bir düğme üzerinde dikey olarak nasıl yerleştirileceğini belirtebilirsiniz.

12.11.3 Metin Pozisyonları

yatay metin pozisyonu

Yatay metin pozisyonu, metnin simgeye göre yatay konumunu tanımlar. Yatay metin pozisyonunu `setHorizontalTextPosition(int)` kullanarak, beş sabitten biri olan `ÖN`, `SOL`, `MERKEZ`, `SAĞ` veya `ARKA` ile ayarlayabilirsiniz; bu, Şekil 12.20'de gösterilmiştir. Şu anda, `ÖN` ve `SOL` aynıdır, ve `ARKA` ile `SAĞ` da aynıdır. Gelecekteki uygulamalar bunları ayırt edebilir. Varsayılan yatay metin pozisyonu `AbstractButton.RIGHT`'dir.

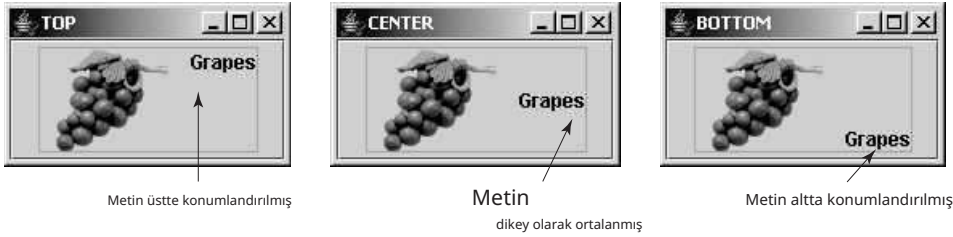


FIGURE 12.20 Metnin simgeye göre yatay konumunu belirtebilirsiniz.

dikey metin konumu

Dikey metin pozisyonu, metnin simgeye göre dikey konumunu tanımlar. Dikey metin pozisyonunu `setVerticalTextPosition(int)` kullanarak, üç sabitten biri ile ayarlayabilirsiniz.

sabitler **ÜST**, **MERKEZ**, veya **ALT**, Şekil 12.21'de gösterildiği gibi. Varsayılan dikey metin konumu **AbstractButton.CENTER**'dir.



ŞEKİL 12.21 Metnin simgeye göre dikey konumunu belirtebilirsiniz.

12.28 Bir butonun üzerine OK yazısı nasıl eklenir? Bir butondaki metin nasıl değiştirilir?

Bir butonda bir simge, basılı simge ve üzerine gelindiğinde simge nasıl ayarlanır?

12.29 Verilen bir **JButton** nesnesi **jbtOK** için, butonun ön plan rengini kırmızı, arka plan rengini sarı, mnemonik değerini **K**, ipucu metnini **Tıklayın OK ile devam etmek için**, yatay hizalamayı **SAĞ**, dikey hizalamayı **ALT**, yatay metin konumunu **SOL**, dikey metin konumunu **ÜST** ve simge metin boşluğunu **5** olarak ayarlamak için ifadeler yazın.

12.30 **AbstractButton** sınıfında tanımlanan en az beş özelliği listeyin.

Point Kontrol Et

MyProgrammingLab™

12.12t JCheckBox

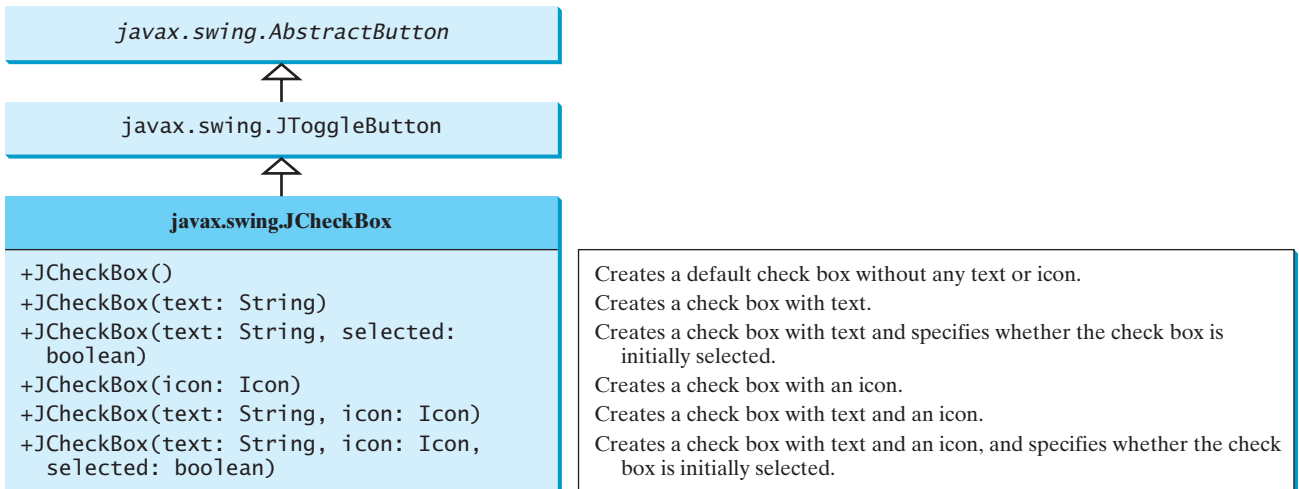
Bir onay kutusu butonu oluşturmak için **JCheckBox** sınıfını kullanın.

Bir *toggle butonu*, bir ışık anahtarı gibi iki durumlu bir butondur. **JToggleButton**, **AbstractButton** sınıfından türetilir ve bir *toggle butonu* uygular. Genellikle **JToggleButton**'in alt sınıfları **JCheckBox** ve **JRadioButton**, kullanıcının bir seçeneği açıp kapatmasını sağlamak için kullanılır. Bu bölüm **JCheckBox**'i tanıtmaktadır. **JRadioButton** bir sonraki bölümde tanıtılacaktır.

JCheckBox, **AbstractButton**'dan tüm özellikleri miras alır, bunlar arasında **metin**, **simge**, **mnemonic**, **dikey hizalama**, **yatay hizalama**, **yatay metin konumu**, **dikey metin konumu** ve **seçili** bulunur ve onay kutuları oluşturmak için birkaç yapıcı sağlar, Şekil 12.22'de gösterildiği gibi.



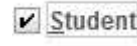
değiştirme düğmesi



ŞEKİL 12.22 **JCheckBox** bir onay kutusu düğmesi tanımlar.

İşte metni *Öğrenci* olan bir onay kutusu oluşturma örneği. Ön plan rengi **kırmızı**, arka plan rengi **beyaz**, mnemonik anahtarı **S** ve başlangıçta seçili.

```
JCheckBox jchk = new JCheckBox("Öğrenci", true);
jchk.setForeground(Color.RED);
jchk.setBackground(Color.WHITE);
jchk.setMnemonic('S');
```



mnemonikler

Butona ayrıca klavye mnemonikleri kullanılarak erişilebilir. **Alt+S** tuşuna basmak, onay kutusuna tıklamakla eşdeğerdir.

seçili

Bir onay kutusunun seçilip seçilmediğini görmek için `isSelected()` yöntemini kullanın.

12.31 Bir onay kutusu nasıl oluşturulur? Başlangıçta kutusu işaretli bir onay kutusu nasıl oluşturulur? Bir onay kutusunun seçilip seçilmediğini nasıl belirlersiniz?

MyProgrammingLab™

Point Kontrol Et



12.13JRadio Düğmesi

Bir radyo düğmesi oluşturmak için `JRadioButton` sınıfını kullanın.

Radyo düğmeleri, ayrıca seçenek düğmeleri olarak da bilinir, bir grup

seçenekten tek bir öge seçmenizi sağlar. Görünüm olarak radyo düğmeleri onay kutularına benzer, ancak onay kutuları ya işaretli ya da boş olan bir kare gösterirken, radyo düğmeleri ya

seçildiğinde dolu (seçili) ya da boş (seçili değilse) olan bir daire gösterir.

`JRadioButton`, `AbstractButton` sınıfından türetilir ve radyo düğmeleri oluşturmak için birkaç yapıcı sağlar, Şekil 12.23'te gösterildiği gibi. Bu yapıcılar,

`JCheckBox` için olan yapıcılara benzer.

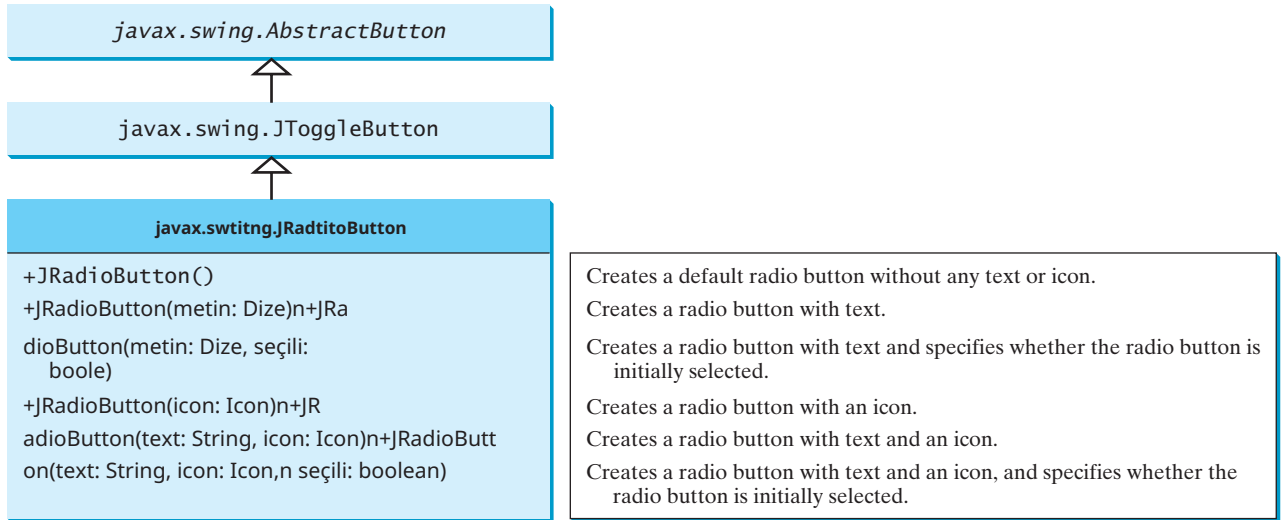
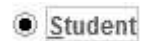


FIGURE 12.23 `JRadioButton` bir radyo düğmesini tanımlar.

İşte "Öğrenci" metni ile bir radyo düğmesi oluşturma örneği. Kod,

kırmızı ön plan, **beyaz** arka plan, mnemonik anahtar **S** ve başlangıçta seçili olarak belirtir.

```
JRadioButton jrb = new JRadioButton("Öğrenci", true);
jrb.setForeground(Color.RED);
jrb.setBackground(Color.WHITE);
jrb.setMnemonic('S');
```



Radyo düğmelerini gruplamak için, bir `java.swing.ButtonGroup`

örneği oluşturmanız ve `add` yöntemini kullanarak bunları eklemeniz gerekir, aşağıdaki gibi:

```
ButtonGroup gruba = tnewtButtonGroup();ngro
up.add(jrb1);
group.add(jrb2);
```

Bu kod, radyo düğmeleri `jrb1` ve `jrb2` için bir radyo düğmesi grubu oluşturur, böylece bunlar karşılıklı olarak seçilir. Gruplama olmadan, `jrb1` ve `jrb2` bağımsız olurdu.



Not

`ButtonGroup`, `java.awt.Component` sınıfının bir alt sınıfı değildir, bu nedenle bir `ButtonGroup` nesnesi bir konteynere eklenemez.

GUI yardımcı sınıfı

Bir radyo düğmesinin seçilip seçilmediğini görmek için `isSelected()` yöntemini kullanın.

12.32 Bir radyo düğmesi nasıl oluşturulur? Başlangıçta seçili olan bir radyo düğmesi nasıl oluşturulur? Radyo düğmelerini nasıl gruplarsınız? Bir radyo düğmesinin seçili olup olmadığını nasıl belirlersiniz?

Nokta Kontrol Et

MyProgrammingLab™

12.14 Etiketler

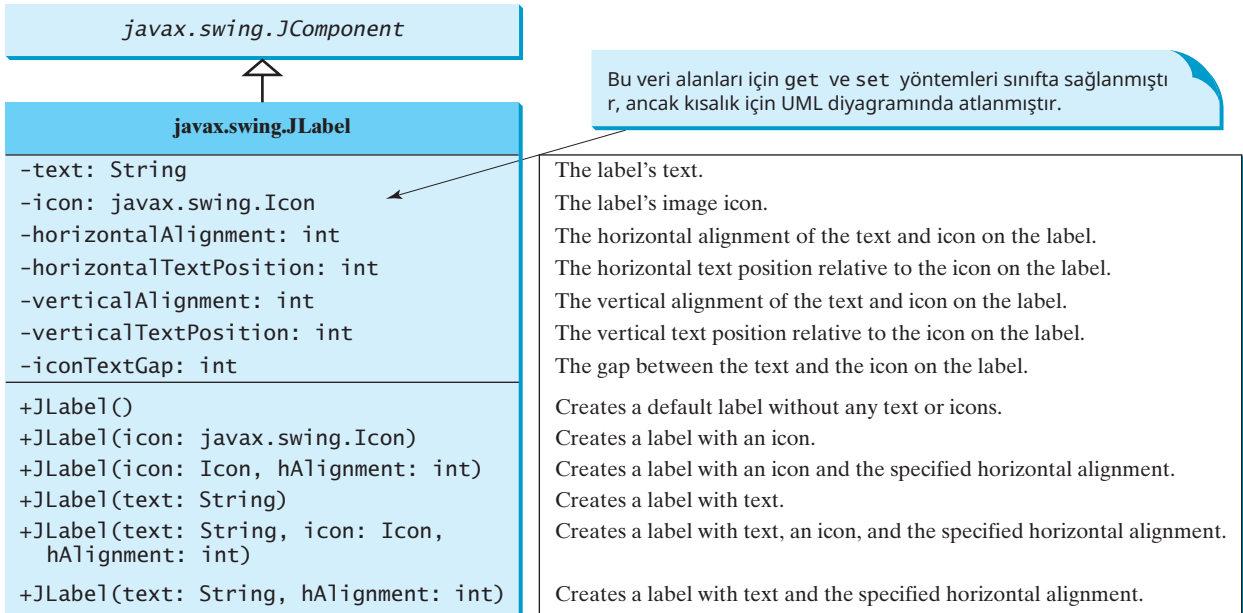
Bir etiket oluşturmak için `JLabel` sınıfını kullanın.

Bir *etiket* kısa bir metin, bir resim veya her ikisi için bir görüntüleme alanıdır. Genellikle diğer bileşenleri etiketlemek için kullanılır (genellikle metin alanları). Şekil 12.24,



Ana Nokta

JLabel1 sınıfındaki yapıları ve yöntemleri listeler.



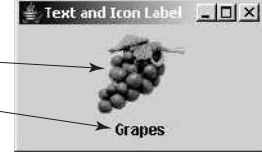
Fiçin 12.24 `JLabel` metin veya bir ikon, ya da her ikisini birden gösterir.

`JLabel`, `JComponent`'ten tüm özellikleri miras alır ve `JButton`'daki gibi birçok benzer özelliğe sahiptir; bunlar arasında metin, ikon, yatay hizalama, dikey hizalama, yatay metin konumu, dikey metin konumu ve ikon-metin boşluğu bulunur. Örneğin, aşağıdaki kod bir metin ve bir ikon ile etiket gösterir:

```
// Bir resim dosyasından bir resim ikonu oluştur
ImageIcon icon = new ImageIcon("image/grapes.gif");

// Metin, bir ikon ile bir etiket oluşturun,
// merkezlenmiş yatay hizalama ile
JLabel jlbl = new JLabel("Üzümler", icon, JLabel.CENTER);

// Etiketlin metin hizalamasını ve metin ile ikon arasındaki boşluğu ayarlayın
jlbl.setHorizontalTextPosition(JLabel.CENTER);
jlbl.setVerticalTextPosition(JLabel.BOTTOM);jlbl.setIcon
TextGap(5t);
```



12.33 Nasıl **Address** adında bir etiket oluşturunuz? Bir etiketteki ismi nasıl değiştirirsiniz? Bir etikette ikon nasıl ayarlarsınız?

MyProgrammingLab™

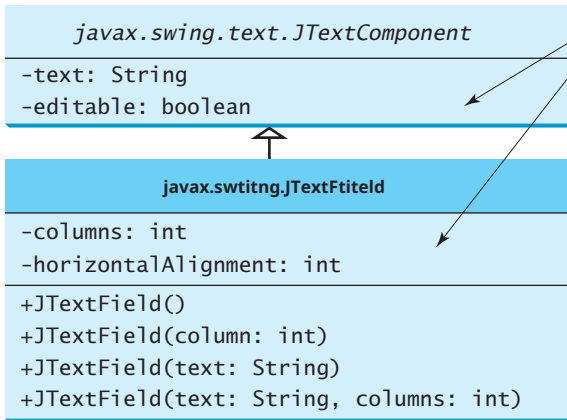
12.34 **JLabel** nesnesi **jlbl** map verildiğinde, etiketin ön plan rengini kırmızı, arka plan rengini sarı, kısayol tuşunu M, araç ipucu metnini Harita resmi, yatay hizalamayı SAĞ, dikey hizalamayı ALT, yatay metin konumunu SOL, dikey metin konumunu ÜST ve ikon-metin boşluğunu 5 olarak ayarlamak için ifadeler yazın.

12.15 Metin Alanları



Bir metin alanı oluşturmak için **JTextField** sınıfını kullanın.

Bir *metin alanı*, bir dize girmek veya göstermek için kullanılabilir. **JTextField**, **JTextComponent**'in bir alt sınıfıdır. Şekil 12.25, **JTextField**'daki yapıları ve yöntemleri listeler.



Bu veri alanları için get ve set yöntemleri sınıfta sağlanmıştı r, ancak kısalık için UML diyagramında atlanmıştır.

The text contained in this text component.
Indicates whether this text component is editable (default: true).

The number of columns in this text field.
The horizontal alignment of this text field (default: LEFT).
Creates a default empty text field with number of columns set to 0.
Creates an empty text field with a specified number of columns.
Creates a text field initialized with the specified text.
Creates a text field initialized with the specified text and columns.

FGÖRÜNTÜ 12.25 **JTextField**, bir dize girmenizi veya görüntülemenizi sağlar.

JTextField, **JTextComponent**'ten miras alır, bu da **JComponent**'ten miras alır. İşte kırmızı ön plan rengine ve sağ yatay hizalamaya sahip bir metin alanı oluşturmanın bir örneği:

```
JTextField jtfMessage = newJTextField("T-Firtına");
jtfMessage.setForeground(Color.RED);
jtfMessage.setHorizontalAlignment(JTextField.RIGHT);
```

Bir metin alanında yeni metin ayarlamak için **setText(newText)** yöntemini kullanın. Bir metin alanından metni almak için **getText()** yöntemini kullanın.



Not

Eğer bir metin alanı şifre girişi için kullanılıyorsa, **JPasswordField** ile değiştirin **JTextField**. **JPasswordField**, **JTextField**'yi genişletir ve giriş metnini yankı karakterleri (örneğin, *********) ile gizler. Varsayılan olarak, yankı karakteri *****. Yeni bir yankı karakteri belirtebilirsiniz **setEchoChar(char)** yöntemi ile.

12.35 10 sütun ve varsayılan metin Hoş geldiniz

Java olan bir metin alanı nasıl oluşturunuz? Bir metin alanının boş olup olmadığını kontrol etmek için kodu nasıl yazarsınız?

12.36 Şifre girmek için metin alanı nasıl oluşturunuz?

Nokta Kontrol Et

MyProgrammingLab™

Anahtar Terimler

AWT 446	düzen yöneticisi 451
bileşen sınıfı 446	hafif bileşen 446
kapsayıcı sınıf 446	Swing bileşenleri 446
ağır bileşen 446	başlangıç ekranı 467
yardımcı sınıf 446	üst düzey kapsayıcı 447

BÖLÜM ÖZETİ

- Her konteynerin, bileşenleri istenen konumlarda yerleştirmek ve konumlandırmak için kullanılan bir düzen yöneticisi vardır. Üç basit ve sık kullanılan düzen yöneticisi **FlowLayout**, **GridLayout** ve **BorderLayout**'dir.
- Bir **JPanel**'i, bileşenleri gruplamak ve istenen bir düzen elde etmek için bir alt konteyner olarak kullanabilirsiniz.
- Bileşenleri bir **JFrame** veya **JPanel** içinde yerleştirmek için **add** yöntemini kullanın. Varsayılan olarak, çerçevenin düzeni **BorderLayout** ve **JPanel**'in düzeni **FlowLayout**'dir.
- Renkler, her biri yoğunluğunu tanımlayan işaretli bir bayt değeri ile temsil edilen kırmızı, yeşil ve mavi bileşenlerden oluşur; bu değer **0** (en koyu ton) ile **255** (en açık ton) arasında değişir. Bu, *RGB modeli* olarak bilinir.
- Bir **Color** nesnesi oluşturmak için **new Color(r, g, b)** kullanın; burada **r**, **g** ve **b**, bir rengi kırmızı, yeşil ve mavi bileşenleri ile tanımlar. Alternatif olarak, **java.awt.Color** içinde sabitler olarak tanımlanan 13 standart renkten birini (**BLACK**, **BLUE**, **CYAN**, **DARK_GRAY**, **GRAY**, **GREEN**, **LIGHT_GRAY**, **MAGENTA**, **ORANGE**, **PINK**, **RED**, **WHITE**, **YELLOW**) kullanabilirsiniz.
- Her *Swing* GUI bileşeni, **javax.swing.JComponent**'in bir alt sınıfıdır ve **JComponent**, **java.awt.Component**'in bir alt sınıfıdır. **Component** içindeki **font**, **background**, **foreground**, **height**, **width** ve **preferredSize** özellikleri bu alt sınıflarda miras alınır; ayrıca **JComponent** içindeki **toolTipText** ve **border** da miras alınır.
- Herhangi bir *Swing* bileşeninde kenarlıklar kullanabilirsiniz. Bir resim simgesi oluşturmak için **ImageIcon** sınıfını oluşturun ve bir etiket ve düğmede görüntüleyin. İkonlar ve kenarlıklar paylaşılabilir.
- Düğmelerde (**JButton**, **JCheckBox**, **JRadioButton**) ve etiketlerde (**JLabel**) metin ve ikon görüntüleyebilirsiniz.
- JButton**, **JCheckBox**,

JRadioButton ve **JLabel** içindeki yatay ve dikey metin hizalamasını belirtebilirsiniz ve **JTextField** içindeki yatay metin hizalamasını belirtebilirsiniz.

10. İkon ile ilgili yatay ve dikey metin konumunu belirtebilirsiniz `JButton`, `JCheckBox`, `JRadioButton`, ve `JLabel`.

TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: www.cs.armstrong.edu/liang/intro9e/test.html.

MyProgrammingLab™

resim dosyalarını indir



Not

Egzersizlerde kullanılan resim simgeleri, www.cs.armstrong.edu/liang/intro9e/book.zip adresinden resim klasörü altında elde edilebilir.

Bölümler 12.2–12.6

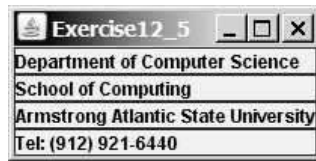
- 12.1 (FlowLayout yöneticisini kullanın) Aşağıdaki gereksinimleri karşılayan bir program yazın (bkz. Şekil 12.26):

- Bir çerçeve oluşturun ve düzenini FlowLayout olarak ayarlayın.
- İki panel oluşturun ve bunları çerçeveye ekleyin.
- Her panel üç düğme içerir. Panel FlowLayout kullanır.

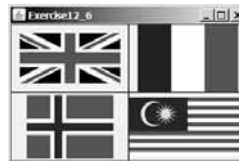


ŞEKİL 12.26 Egzersiz 12.1, ilk üç düğmeyi bir panelde ve diğer üç düğmeyi başka bir panelde yerleştirir.

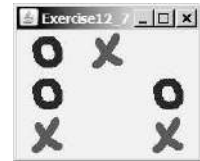
- 12.2 (BorderLayout yöneticisini kullanın) Önceki programı yeniden yazın t aynı kullanıcı arayüzünü oluşturmak için, ancak çerçeve için `FlowLayout` yerine `BorderLayout` kullanın. Bir paneli çerçevenin güneyine ve diğerini merkeze yerleştirin.
- 12.3 (`GridLayout` yöneticisini kullanın) Programlama Alıştırması 12.1'i yeniden yazın ve altı butonu bir çerçeveye ekleyin. Çerçeve için iki satır ve üç sütunluk bir `GridLayout` kullanın.
- 12.4 (Butonları gruplamak için `JPanel` kullanın) Programlama Alıştırması 12.1'i yeniden yazın ve aynı kullanıcı arayüzünü oluşturun. Düğmeleri ve panelleri ayrı ayrı oluşturmak yerine, `JPanel` sınıfını genişleten bir sınıf tanımlayın. Panel sınıfınıza üç düğme yerleştirin ve kullanıcı tanımlı panel sınıfından iki panel oluşturun.
- 12.5 (Etiketleri görüntüle) Dört etiket içinde dört satır metin görüntüleyen bir program yazın, Şekil 12.27'da gösterildiği gibi. Her etiketin etrafına bir kenar sınırı ekleyin.



(a)



(b)



(c)

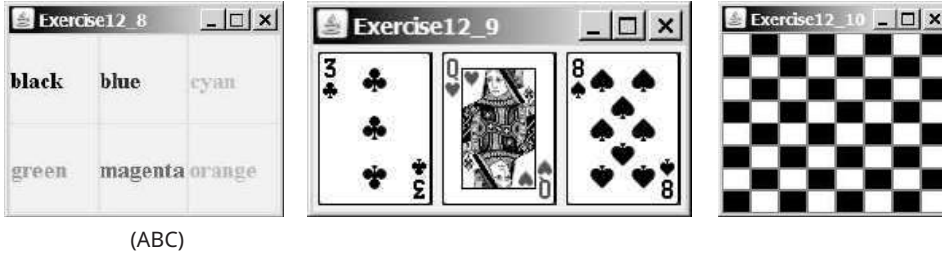
ŞEKİL 12.27 (a) Alıştırma 12.5 dört etiket gösterir. (b) Alıştırma 12.6 dört simge gösterir. (c) Alıştırma 12.7 etiketlerde resim simgeleri olan bir üçlü tahtası gösterir.

Bölümler 12.7–12.15

12.6 (*İkonları görüntüle*) Dört etiket içinde dört ikon görüntüleyen bir program yazın, Şekil 12.27b'de gösterildiği gibi. Her etiketin etrafına bir kenar sınırı ekleyin.

***12.7** (*Oyun: bir üçlü oyun tahtası görüntüle*) Dokuz etiket içeren bir çerçeve görüntüleyin. Bir etiket, X için bir resim ikonunu veya O için bir resim ikonunu görüntüleyebilir, Şekil 12.27c'de gösterildiği gibi. Ne görüntüleneceği rastgele belirlenir. X veya O resim ikonunu görüntülemek için `Math.random()` metodunu kullanarak bir tamsayı 0 veya 1 oluşturun. Bu resimler `x.gif` ve `o.gif` dosyalarında bulunmaktadır.

***12.8** (*Swing ortak özellikleri*) Altı etiket içeren bir çerçeve görüntüleyin. Etiketlerin arka planını beyaz olarak ayarlayın. Etiketlerin ön planını sırasıyla siyah, mavi, camgöbeği, yeşil, mor ve turuncu olarak ayarlayın, Şekil 12.28a'da gösterildiği gibi. Her etiketin kenarını sarı renkte bir kenar sınırı ile ayarlayın. Her etiketin yazı tipini Times Roman, kalın ve 20 piksel olarak ayarlayın. Her etiketin metnini ve araç ipucu metnini ön plan renginin adıyla ayarlayın.



ŞEKİL 12.28(a) Çerçeve altı etiket yerleştirilmiştir. (b) Üç kart rastgele seçilmiştir. (c) Düğmeler kullanarak bir dama tahtası gösterilmektedir.

***12.9** (*Oyun: üç kartı göster*) Üç etiket içeren bir çerçeve gösterin. Her etiket bir kartı gösterir, Şekil 12.28b'de gösterildiği gibi. Kart resim dosyaları `1.png`, `2.png`, ..., `54.png` (jokerler dahil) olarak adlandırılmıştır ve `image/card` dizininde saklanmaktadır. Üç kart da farklıdır ve rastgele seçilmiştir.

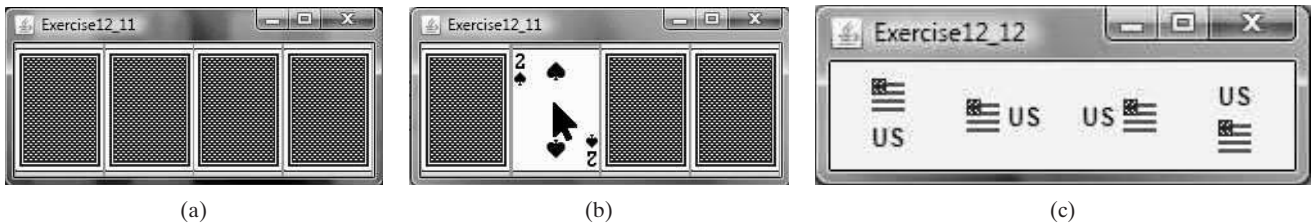
***12.10** (*Oyun: bir dama tahtası göster*) Her beyaz ve siyah hücrenin arka planı siyah veya beyaz olan bir `JButton` ile gösterildiği bir dama tahtası gösteren bir program yazın, Şekil 12.28c'de gösterildiği gibi.

***12.11** (*Oyun: dört kart göster*) Egzersiz 12.9'dan aynı kartları kullanarak dört düğme içeren bir çerçeve gösterin. Tüm düğmeler, Şekil 12.29a'da gösterildiği gibi, `ckCard.png` dosyasından aynı ikona sahiptir. Basılı ikonlar, bir deste karttan rastgele seçilen dört karttır, Şekil 12.29b'de gösterildiği gibi.



VideoNotu

Dama tahtası göster



ŞEKİL 12.29(a) Dört düğmenin aynı ikonu vardır. (b) Her düğmenin basılı ikonu desteden rastgele seçilmiştir. (c) Resim ikonları ve metinler dört etiket içinde gösterilmektedir.



VideoNotu

Rastgele bir matris görüntüle

12.12 (*Etiketleri kullan*) Şekil 12.29c'de gösterildiği gibi, resim simgesini ve metni dört etikette görüntüleyen bir program yazın.

12.13 (*54 kartı görüntüle*) Egzersiz 12.9'u genişleterek, her satırda dokuz tane olmak üzere, 54 etikette tüm 54 kartı görüntüleyin.

***12.14** (*Rastgele 0 veya 1 görüntüle*) Şekil 12.30'da gösterildiği gibi, 10'a 10 boyutunda bir kare matris görüntüleyen bir program yazın. Matristeki her bir eleman 0 veya 1, rastgele üretilmiştir. Her sayıyı bir etikette ortalananmış olarak görüntüleyin.

0	0	0	1	1	0	1	0	0	0
1	1	1	1	0	1	1	1	1	0
0	0	0	0	0	1	1	1	1	1
1	0	1	1	1	0	0	1	0	0
0	0	1	0	1	1	1	0	0	1
1	1	1	1	1	0	1	0	0	0
1	0	1	1	0	0	1	0	0	0
1	1	1	0	1	1	1	1	0	1
0	1	1	1	0	0	0	1	0	1
1	0	0	0	1	1	1	0	1	1

ŞEKİL 12.30 Program rastgele 0 ve 1 üretir.

GRAFİKLER

Hedefler

- **Graphics** sınıfındaki yöntemleri kullanarak grafik çizmek (§13.2).
- GUI
bileşeninde grafik çizmek için `paintComponent` yöntemini geçersiz kılmak (§13.2).
- Grafikleri çizmek için bir paneli tuval olarak kullanmak (§13.2).
- Dizgiler, çizgiler, dikdörtgenler, ovalar, yaylar ve çokgenler çizmek (§§13.3, 13.5–13.6).
- Font özelliklerini **FontMetrics** kullanarak elde etmek ve bir metni panelde ortalamak (§13.7).
- Bir GUI bileşeninde bir resmi görüntülemek (§13.10).
- Yeniden kullanılabilir GUI bileşenlerini **FigurePanel**, **MessagePanel**, **StillClock** ve **ImageViewer** geliştirmek (§§13.4, 13.8, 13.9, 13.11).



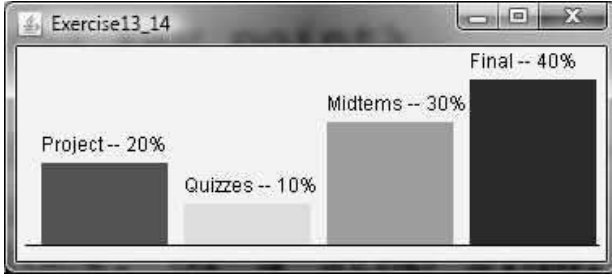
13.1 Giriş



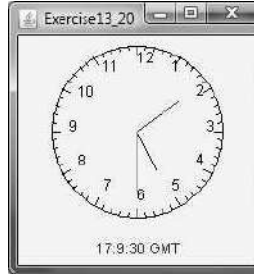
Bir GUI bileşeninde özel şekiller çizebilirsiniz.

Sorun

Bir çubuk grafik, bir saat veya bir dur işareti gibi şekiller çizmek istediğinizi varsayalım. Bunu nasıl yaparsınız?



(a)



(b)



(c)

ŞEKİL 13.1 Çizim yöntemlerini kullanarak şekiller çizebilirsiniz **Graphics** sınıfında.

Bu bölüm, **Graphics** sınıfındaki yöntemleri kullanarak dizeleri, çizgileri, dikdörtgenleri, ovaları, yayları, çokgenleri ve resimleri nasıl çizeceğinizi ve yeniden kullanılabilir GUI bileşenleri nasıl geliştireceğinizi açıklar.

13.2 Graphics Sınıfı



*Her GUI bileşeninin, **Graphics** sınıfının bir nesnesi olan bir grafik bağlamı vardır. **Graphics** sınıfı, çeşitli şekilleri çizmek için yöntemleri içerir.*

Graphics sınıfı, dizeleri, çizgileri, dikdörtgenleri, ovaları, yayları, çokgenleri ve poligonları çizmek için yöntemler sağlar; bu, Şekil 13.2'de gösterilmiştir.

Bir GUI bileşenini bir kağıt parçası ve **Graphics** nesnesini bir kalem veya fırça olarak düşünün. GUI bileşeninde grafik çizmek için **Graphics** sınıfındaki yöntemleri uygulayabilirsiniz.

Boyamak için, nerede boyayacağınızı belirtmeniz gerekir. Her bileşenin, sol üst köşede (0,0) orijini olan kendi koordinat sistemi vardır. *x* koordinatı sağa doğru artar ve *y* koordinatı aşağıya doğru artar. Java koordinat sisteminin, Şekil 13.3'te gösterildiği gibi, geleneksel koordinat sisteminden farklı olduğunu unutmayın.

Graphics sınıfı - soyut bir sınıf - farklı platformlarda ekran üzerinde figürler ve görüntüler göstermek için cihazdan bağımsız bir grafik arayüzü sağlar. Bir bileşen (örneğin, bir düğme, bir etiket veya bir panel) görüntülendiğinde, JVM otomatik olarak bir **Grafikler** bileşen için yerel platformda bir nesne ve bu nesneyi çizimleri görüntülemek için **paintComponent** yöntemini çağırarak üzere geçirir.

paintComponent yönteminin imzası aşağıdaki gibidir:

```
korumalı void çizimBileşeni(Graphics g)
```

Bu yöntem, **JComponent** sınıfında tanımlanmıştır ve bir bileşen ilk kez gösterildiğinde veya yeniden gösterildiğinde çağırılır.

Bir bileşen üzerinde çizim yapmak için, **JPanel** sınıfını genişleten ve ne çizeceğinizi belirtmek için **paintComponent** yöntemini geçersiz kılan bir sınıf tanımlamanız gerekir. Liste 13.1, bir panel üzerinde bir çizgi ve bir çizim bir örnek vermektedir; bu, Şekil 13.4'te gösterilmiştir.

boya Bileşeni

Java.Ah sorunu.grafikler

```

+setColor(color: Color): voidn+setFont
t(font: Font): voidn+drawString(s:
String, x: int, y: int): voidn+drawLine(x1: int, y1: int, x2: i
nt, y2:
int): geçersiz
+drawRect(x: int, y: int, w: int, h: int):
geçersiz
+fillRect(x: int, y: int, w: int, h: int): voidn+drawRoundRect(x: i
nt, y: int, w: int, h: int, aw:
int, ah: int): geçersiz
+fillRoundRect(x: int, y: int, w: int, h: int, n aw: int, ah: int): v
oid
+draw3DRect(x: int, y: int, w: int, h: int, n yükseltilmiş: b
oolean): void
+fill3DRect(x: int, y: int, w: int, h: int, n yükseltilmiş: boo
lean): void
+drawOval(x: int, y: int, w: int, h: int):
geçersiz
+fillOval(x: int, y: int, w: int, h: int): voidn+drawArc(x: int, y: in
t, w: int, h: int,
startAngle: int, arcAngle: int): voidn+fillArc(x: in
t, y: int, w: int, h: int, n startAngle: int, arcAngle: int
): voidn+drawPolygon(xPoints: int[], yPoints:
int[], nPoints: int): voidn+fillPolyg
on(xPoints: int[], yPoints: int[], n nPoints: int): void
+drawPolygon(g: Çokgen): voidn+fillP
olygon(g: Çokgen): voidn+drawPolylin
e(xPoints: int[], yPoints:
int[], nPoints: int): geçersiz

```

Sonraki çizimler için yeni bir renk ayarlar.

Sonraki çizimler için yeni bir yazı tipi ayarlar.

Bir dizeyi (x, y) noktasında çizer.

(x1, y1) noktasından (x2, y2) noktasına bir çizgi çizer.

Belirtilen sol üst köşe noktasında bir dikdörtgen çizer

(x, y) ve genişlik w ve yükseklik h ile.

Belirtilen sol üst köşe noktasında

(x, y) ve genişlik w ve yükseklik h ile doldurulmuş bir dikdörtgen çizer.

Belirtilen kavis genişliği

aw ve kavis yüksekliği ah ile yuvarlak köşeli bir dikdörtgen çizer.

Doldurulmuş yuvarlak köşeli bir dikdörtgen çizer, belirtilen yay

genişliği aw ve yay yüksekliği ah.

Yüzeyin üzerinde yükseltilmiş veya yüzeye gömülmüş bir 3-D dikdörtgen çizer.

Doldurulmuş bir 3-D dikdörtgen çizer, yüzeyin üzerinde yükseltilmiş veya yüzeye gömülmüş.

Belirtilen x,y,w ve h parametreleriyle tanımlanan dikdörtgenle sınırlı bir oval çizer.

Belirtilen x,y,w ve h parametreleriyle tanımlanan dikdörtgenle sınırlı doldurulmuş bir oval çizer.

Belirtilen x,y,w ve h parametreleriyle tanımlanan dikdörtgenle sınırlı bir ovalin parçası olarak tasarlanan bir yay çizer.

Belirtilen x,y,w ve h parametreleriyle tanımlanan dikdörtgenle sınırlı ovalin parçası olarak tasarlanan doldurulmuş bir yay çizer.

x ve

y koordinatlarının dizileriyle tanımlanan kapalı bir çokgen çizer. Her (x[i], y[i]) koordinat çifti bir noktadır.

x ve

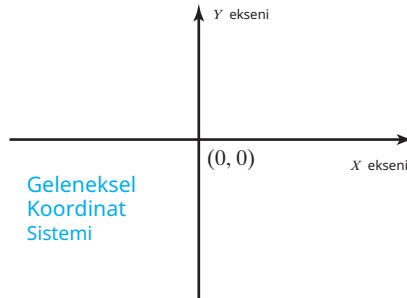
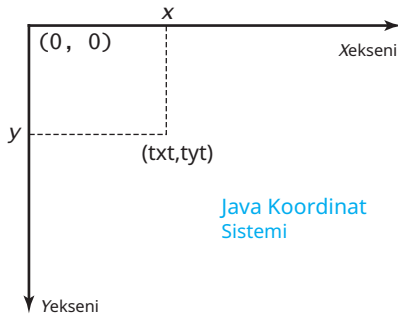
y koordinatlarının dizileriyle tanımlanan doldurulmuş bir çokgen çizer. Her (x[i], y[i]) koordinat çifti bir noktadır.

Bir Polygon nesnesiyle tanımlanan kapalı bir çokgen çizer.

Doldurulmuş bir çokgen çizer, bu bir Polygon nesnesi ile tanımlanır.

Bir poligon çizer, bu x- ve y-koordinatlarının dizileri ile tanımlanır. Her (x[i], y[i]) koordinat çifti bir noktadır.

FIGİN 13.2 Grafik sınıfı, dizeleri ve şekilleri çizmek için yöntemleri içerir.



FIGEKTİ 13.3 Java koordinat sistemi piksel cinsinden ölçülür, (0, 0) üst-

sol köşesinde yer alır.

LİSTE 13.1 TestPaintComponent.java

```

1 import javax.swing.*;
2 içe aktarma, Java.Ah soru.grafikleriyle aynıdır;
3

```

bir panel oluştur

```

4  genel siniftTestPaintComponenttextendstJFrame {
5      publicTestPaintComponent() {
6          add(newYeniPanel());
7      }
8
9      public static voidtmain(String[] args) {n10 TestPaint
Component çerçeve = tnewtTestPaintComponent();n11 çerçeve.setTitle(t"Tes
tPaintComponent"t);
12      frame.setSize(200, 100);
13      frame.setLocationRelativeTo(null); // Çerçeveyi ortala
14      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15      frame.setVisible(true);
16 }n17 }

```

yeni panel sınıfı

override paintComponent
üst sınıfta nesneleri çiz
çizgi çiz
metin çiz

```

18
1920212223242526n
classYeniPanelextendsJPanel {
    protected voidtpaintComponent(Graphics g) {n@Override
        pert.paintComponent(g);nprot
        g.drawLine(t0t,t0t,t50t,t50t);nsu
        "Afiş", 0, 40);
    }
}

```

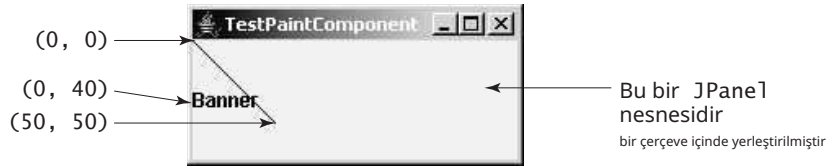


FIGURE 13.4 Bir panel üzerinde bir çizgi ve bir metin çizilir.

Grafikleri boyamak için paintComponent metodu, bileşen ilk kez görüntülendiğinde veya bileşenin yeniden görüntülenmesi gerektiğinde otomatik olarak çağrılır. super.paintComponent(g) çağırısı (satır 22), üst sınıfta tanımlanan paintComponent metodunu çağırır. Bu, yeni bir çizim görüntülenmeden önce görüntüleme alanının temizlenmesini sağlamak için gereklidir. Satır 23, (0,0) ile (50,50) arasında bir çizgi çizmek için drawLine metodunu çağırır.

Satır 24, (0,40) konumunda **Banner** dizesini çizmek için drawString metodunu çağırır.

Tüm çizim metodlarının, çizilecek nesnelerin konumlarını belirten parametreleri vardır. Java'da tüm ölçümler piksellerle yapılır.

JVM, bir bileşen üzerinde şeyleri çizmek için paintComponent'i çağırır. Kullanıcı asla paintComponent'i doğrudan çağırmamalıdır. Bu nedenle, paintComponent için korumalı görünürlük yeterlidir.

Pencereler görünmezdir ve bileşenleri gruplamak için küçük konteynerler olarak kullanılır, böylece istenen bir düzen elde edilir. **JPanel**'in bir diğer önemli kullanımı ise çizim yapmaktır. Herhangi bir Swing GUI bileşeninde şeyler çizebilirsiniz, ancak genellikle şeyleri çizmek için bir tuval olarak **JPanel** kullanmalısınız. Aşağıdaki gibi 19. satırda **JPanel**'i **JLabel** ile değiştirirseniz ne olur?

```

classNewPaneltextends JLabel {

```

Program çalışacaktır, ancak tercih edilmez, çünkü **JLabel** bir etiket oluşturmak için tasarlanmıştır, çizim yapmak için değil. Tutarlılık açısından, bu kitap bir tuval sınıfını alt sınıflandırarak tanımlayacaktır. **JPanel**.

JPanel'i genişletir?



Ttiti

Bazı ders kitapları bir tuval sınıfını **JComponent**'i alt sınıflandırarak tanımlar. Bunu yapmanın sorunu, tuvalde bir arka plan ayarlamak istiyorsanız, arka plan rengini boyamak için kod yazmanız gerektiğidir. Basit bir **setBackground(Color color)** metodu, **JComponent**'te bir arka plan rengi ayarlamayacaktır.

JComponent'i genişletir

Nokta Kontrol Et

MyProgrammingLab™

13.1 Mevcut bir mesajın altına yeni bir mesaj çizmek istediğinizi varsayalım. Olmalı mı

x-koordinatı,y-koordinatı, yoksa her ikisi de mi artar veya azalır?

13.2 Bir **Graphics** nesnesi nasıl oluşturulur?

13.3 **paintComponent** metodu nasıl çağrılır? Bir program bu

metodu nasıl çağırabilir?

13.4 Neden **paintComponent** metodu korumalı? Bir alt sınıfta bunu **public** veya **private** olarak değiştirirseniz ne olur? Neden **super.paintComponent(g)** 22. satırda **String** 13.1'de çağrılır?

13.5 Herhangi bir Swing GUI bileşeninde şeyler çizebilir misiniz? Neden bir paneli çizimler için bir tuval olarak kullanmalısınız, etiket veya düğme yerine?

13.3 Dize, Çizgi, Dikdörtgen ve Oval Çizme

Grafik bağlamında dizeleri, çizgileri, dikdörtgenleri ve ovaları çizebilirsiniz.

drawString(String s, int x, int y) metodu, bir diziyi

(x, y) noktasında başlatarak çizer; Şekil 13.5a'da gösterildiği gibi.

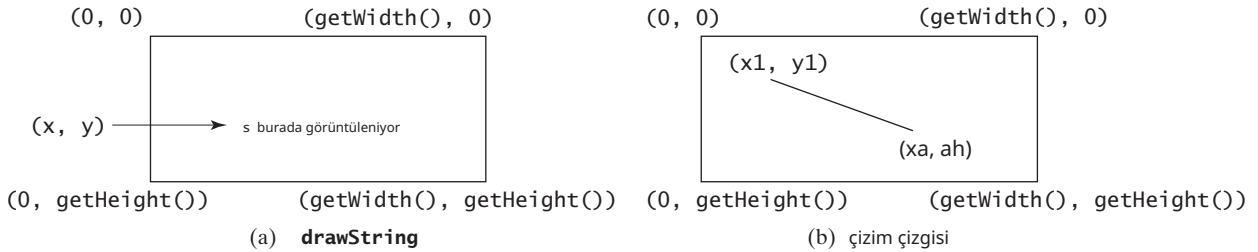
drawLine(int x1, int y1, int x2, int y2) metodu, (x1, y1) noktasında n (x2, y2) noktasına doğru düz bir çizgi çizer; Şekil 13.5b'de gösterildiği gibi.



Ana Nokta

çizmekString

ndrawLine



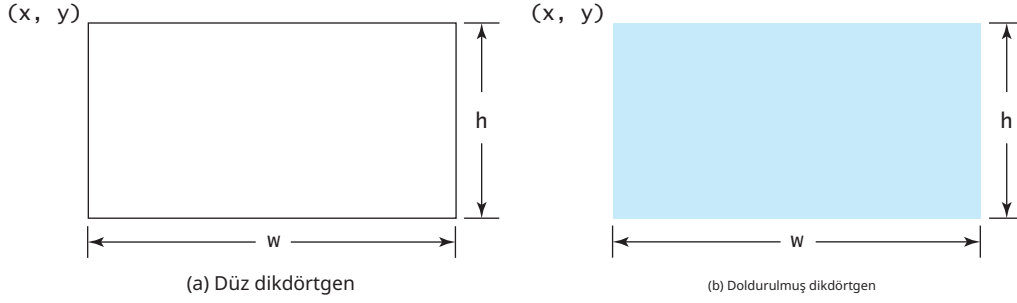
ŞEKİL 13.5 (a) The **drawString(s, x, y)** yöntemi bir diziyi (x, y) noktasında çizer. (b) The **drawLine(x1, y1, x2, y2)** yöntemi iki belirtilen nokta arasında bir çizgi çizer.

Java, dikdörtgenlerin veya renk ile doldurulmuş dikdörtgenlerin dış hatlarını çizmek için altı yöntem sağlar. Düz dikdörtgenler, yuvarlak köşeli dikdörtgenler veya üç boyutlu dikdörtgenler çizebilir veya doldurabilirsiniz.

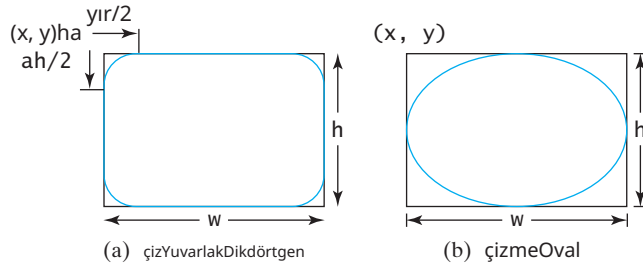
drawRect(int x, int y, int w, int h) metodu düz bir dikdörtgen çizer ve **fillRect(int x, int y, int w, int h)** metodu dolu bir dikdörtgen çizer. x ve y parametreleri dikdörtgenin sol üst köşesini temsil eder ve w ve h genişliği ve yüksekliği (bkz. Şekil 13.6).

drawRoundRect(int x, int y, int w, int h, int aw, int ah) metodu yuvarlak köşeli bir dikdörtgen çizer ve **fillRoundRect(int x, int y, int w, int h, int aw, int ah)** metodu dolu bir yuvarlak köşeli dikdörtgen çizer. x, y, w ve h parametreleri **drawRect** metodundaki ile aynıdır, aw parametresi köşedeki kavislerin yatay çapını ve ah parametresi köşedeki kavislerin dikey çapını temsil eder (bkz. Şekil 13.7a).

çizRectnfil
lRectçizYuvarlakDikdörtgend
oldurYuvarlakDikdörtgen



ŞEKİL 13.6 (a) The `drawRect(x, y, w, h)` metodu bir dikdörtgen çizer. (b) The `fillRect(x, y, w, h)` metodu doldurulmuş bir dikdörtgen çizer.



ŞEKİL 13.7 (a) The `drawRoundRect(x, y, w, h, aw, ah)` metodu yuvarlak-köşeli dikdörtgen. (b) The `drawOval(x, y, w, h)` metodu, onun

sınır dikdörtgenine dayanarak bir oval çizer.

Diğer bir deyişle, `aw` ve `ah` ovalin genişliği ve yüksekliğidir ve her köşede bir çeyrek daire oluşturur.

`Draw3DRect`
`nfill3DRect`

`draw3DRect(int x, int y, int w, int h, boolean raised)` metodu 3-D bir dikdörtgen çizer ve `fill3DRect(int x, int y, int w, int h, boolean raised)` metodu dolu bir 3-D dikdörtgen çizer. `x`, `y`, `w` ve `h` parametreleri `drawRect` metodundaki ile aynıdır. Son parametre, bir Boolean değer, dikdörtgenin yüzeyin üzerinde mi yoksa yüzeye gömülü mü olduğunu belirtir.

`çizmekOval`
`ndoldurOval`

Ovalı bir çerçeve veya dolu olarak çizmek isteyip istemediğinize bağlı olarak, ya `drawOval(int x, int y, int w, int h)` yöntemini ya da `fillOval(int x, int y, int w, int h)` yöntemini kullanabilirsiniz. Bir oval, sınırlayıcı dikdörtgenine dayanarak çizilir. Parametreler `x` ve `y` sınırlayıcı dikdörtgenin sol üst köşesini belirtir ve `w` ve `h` sırasıyla sınırlayıcı dikdörtgenin genişliğini ve yüksekliğini belirtir, Şekil 13.7b'de gösterildiği gibi.

Nokta Kontrol Et

MyProgrammingLab™

13.6 Metinleri, çizgileri çizme yöntemlerini ve dikdörtgenleri, yuvarlak köşeli dikdörtgenleri, 3-D dikdörtgenleri ve ovaları çizme/doldurma yöntemlerini tanımlayın.

13.7 (10,10) ile (70,30) arasında kalın bir çizgi çizin. Bir kalın çizgi etkisi yaratmak için yan yana birkaç çizgi çizebilirsiniz.

13.8 Genişliği 100 ve yüksekliği 50 olan bir dikdörtgeni, sol üst köşesi (10,10) noktasında olacak şekilde çizin/doldurun.

13.9 Genişliği 100, yüksekliği 200, köşe yatay çapı 40 ve köşe dikey çapı 20 olan yuvarlak köşeli bir dikdörtgen çizin/doldurun.

13.10 Yarıçapı 30 olan bir daire çizin/doldurun.

13.11 Genişliği 50 ve yüksekliği 100 olan bir oval çizin/doldurun.

13.4 Vaka Çalışması: FigurePanel Sınıfı

Bu vaka çalışması, çeşitli figürleri görüntülemek için **FigurePanel** sınıfını geliştirir.

Bu örnek, çeşitli figürleri görüntülemek için yararlı bir sınıf geliştirir. Sınıf, kullanıcının figür türünü ayarlamasına ve figürün dolu olup olmadığını belirtmesine olanak tanır ve figürü bir panelde görüntüler. Sınıfın UML diyagramı Şekil 13.8'de gösterilmiştir. Panel, çizgileri, dikdörtgenler i, yuvarlak köşeli dikdörtgenleri ve ovaları görüntüleyebilir. Hangi figürün görüntüleneceği, **type** özelliği ile belirlenir. Eğer **filled** özelliği **true** ise, dikdörtgen, yuvarlak köşeli dikdörtgen ve oval panelde doldurulur.



VideoNotu

FigurePanel sınıfı

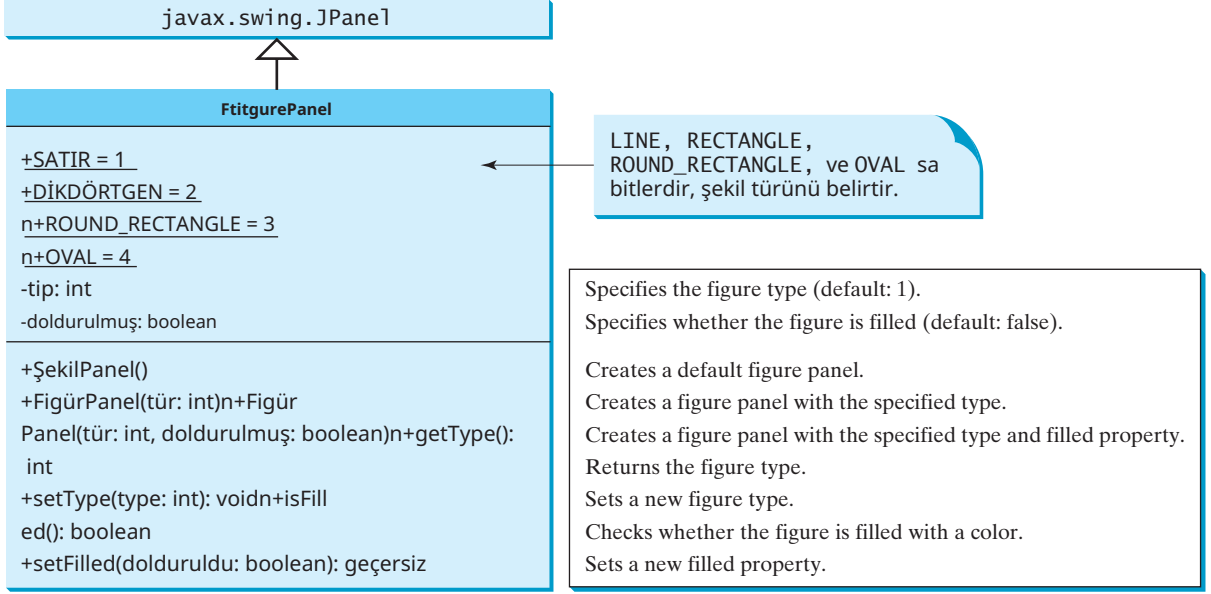


FIGURE 13.8 **FigurePanel** panelde çeşitli şekil türlerini görüntüler.

UML diyagramı, **FigurePanel** sınıfı için sözleşme olarak hizmet eder. Kullanıcı, sınıfın nasıl uygulandığını bilmeden sınıfı kullanabilir. Şimdi, Şekil 13.9'da gösterildiği gibi, altı şekil panelini görüntülemek için sınıfı kullanan bir program yazmaya başlayalım.

Listing 13.2 **TestFigürPanel.java**

```
1 import java.awt.*;
2 import javax.swing.*;
3
4 genel sınıf TestFigürPanel extends JFrame {
5     public TestFigürPanel() {
6         setLayout(new GridLayout(2, 2));
7         add(new FigürPanel(FigürPanel.LINE));
8         add(new FigürPanel(FigürPanel.RECTANGLE));
9         add(new FigürPanel(FigürPanel.ROUND_RECTANGLE));
10        add(new FigürPanel(FigürPanel.OVAL));
11        add(new FigürPanel(FigürPanel.RECTANGLE, true));
12        add(new FigürPanel(FigürPanel.ROUND_RECTANGLE, true));
13    }
14
15    public static void main(String[] args) {
16        TestFigürPanel çerçeve = new TestFigürPanel();
17        çerçeve.setSize(400, 200);
18        çerçeve.setTitle("TestFigürPanel");
19    }
20 }
```

şekil oluştur

```

19     frame.setLocationRelativeTo(null); // Çerçeveyi ortalayalım
20     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21     frame.setVisible(true);
22 }

```

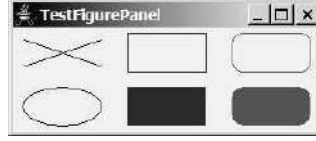


FIGURE 13.9 Altı `FigurePanel` nesnesi, altı şekli görüntülemek için oluşturulmuştur.

`FigurePanel` sınıfı, Liste 13.3'te uygulanmıştır. Dört sabit—`LINE`, `RECTANGLE`, `ROUND_RECTANGLE` ve `OVAL`—6-9. satırlarda tanımlanmıştır. Dört tür figür, `type` özelliğine göre (37. satır) çizilmektedir. `setColor` metodu (39, 44, 53, 62. satırlar) çizim için yeni bir renk ayarlamaktadır.

LİSTE 13.3 `FigurePanel.java`

sabitler

```

1  import java.awt.*;
2  import javax.swing.JPanel;
3
4  genel sınıf FigurePanel JPanel {
5      // Sabitleri tanımla
6      genel statik final int LINE = 1;
7      genel statik final int RECTANGLE = 2;
8      genel statik final int ROUND_RECTANGLE = 3;
9      genel statik final int OVAL = 4;
10
11     özel int type = 1;
12     özel boolean doldurulmuş = false;
13
14     /** Varsayılan FigurePanel oluştur */
15     public FigurePanel() {
16     }
17
18     /** Belirtilen türde bir FigurePanel oluştur */
19     public FigurePanel(int type) {
20         this.type = type;
21     }
22
23     /** Belirtilen türde ve doldurulmuş bir FigurePanel oluştur */
24     public FigurePanel(int type, boolean doldurulmuş) {
25         this.type = type;
26         this.doldurulmuş = doldurulmuş;
27     }
28
29     @Override // Panelde bir şekil çiz
30     korumalı void paintComponent(Graphics g) {
31         super.paintComponent(g);
32
33         // Şekil için uygun boyutu al
34         int width = getWidth();
35         int height = getHeight();
36
37         switch (type) {
38             case LINE: // İki çapraz çizgi göster

```

geçersiz kıl
boya bileşen(g)

tipi kontrol et

```

39     g.setColor(Color.BLACK);
40     g.drawLine (t0t,t10t, genişlik - t10t, yükseklik - t10t);
41     g.drawLine (genişlik - 10, 10, 10, yükseklik - 10);
42     kırılma;
43     case DİKDÖRTGEN:// Bir dikdörtgen göster
44 g.setColor(Renk.MAVİ);
45     eğer(dolu)
46         g.fillRect ((int) (0.1* genişlik), (int) (0.1* yükseklik),
47                     (int) (0.8* genişlik), (int) (0.8* yükseklik));
48     başka
49     g.drawRect ((int) (0.1* genişlik), (int) (0.1* yükseklik),
50                (int) (0.8* genişlik), (int) (0.8* yükseklik));
51     kırılma;
52     case YUVARLAK_DİKDÖRTGEN:// Yuvarlak köşeli bir dikdörtgen göster
53 g.setColor(Renk.KIRMIZI);
54     eğer(dolu)
55         g.fillRoundRect ((int) (0.1* genişlik), (int) (0.1* yükseklik),
56                         (int) (0.8* genişlik), (int) (0.8* yükseklik), 20, 20);
57     başka
58     g.drawRoundRect ((int) (0.1* genişlik), (int) (0.1* yükseklik),
59                    (int) (0.8* genişlik), (int) (0.8* yükseklik), 20, 20);
60     kırılma;
61     durum OVAL:// Bir oval göster
62 g.setColor(Renk.SİYAH);
63     eğer(dolu)
64         g.fillOval ((int) (0.1* genişlik), (int) (0.1* yükseklik),
65                   (int) (0.8* genişlik), (int) (0.8* yükseklik));
66     başka
67     g.drawOval ((int) (0.1* genişlik), (int) (0.1* yükseklik),
68               (int) (0.8* genişlik), (int) (0.8* yükseklik));
69 }n70 }

71
72 /** Yeni bir şekil türü ayarla */
73 public void setType(int tür) {
74     thist.tip = tür;
75     yenidenÇiz();
76 }
77
78 /** Şekil türünü döndür */
79 genel inttgetType() {
80     dönüş türü;
81 }
82
83 /** Yeni bir doldurulmuş özellik ayarla */
84 genel voidtsetFilled(tbooleanfilled) {
85     thist.doldurulmuş = doldurulmuş;
86     yenidenÇiz();
87 }
88
89 /** Şeklin doldurulup doldurulmadığını kontrol et */
90 genel booleanisFilled() {
91     return dolu;
92 }
93
94 @Override // Tercih edilen boyutu belirtin
95 public Dimension getPreferredSize() {
96     return new Dimension(t80t,t80t);np
97 }
98 }n}

```

çizgi çiz

bir dikdörtgeni doldur

bir dikdörtgen çiz

yuvarlak köşeli dikdörtgeni doldur

yuvarlak köşeli dikdörtgen çiz

bir oval doldur

oval çizim

paneli yeniden boyayın

paneli yeniden boyayın

geçersiz kıl
getPreferredSize()

Yeniden boyama metodu (satırlar 75, 86) Bileşen sınıfında tanımlanmıştır. Yeniden boyama çağrıldığında, paintComponent metodu çağrılır. Yeniden boyama metodu, görüntüleme alanını yenilemek için çağrılır. Genellikle, yeni şeyler gösterecek olduğunuzda çağırırsınız.

çağırma
boya Bileşeni



Uyarı

paintComponent metodu asla doğrudan çağrılmamalıdır. Bu metod, görüntüleme alanı değiştiğinde JVM tarafından veya yeniden boyama metodu tarafından çağrılır. Sisteme görüntüleme alanını nasıl boyayacağını söylemek için paintComponent metodunu geçersiz kılmalısınız, ancak yeniden boyama metodu asla geçersiz kilmamalısınız.

şu şekilde yeniden boyama isteği
yeniden boyama()



Not

Yeniden boyama metodu, görüntüleme alanını güncelleme talebinde bulunur ve hemen geri döner. Etkisi asenkron olup, paintComponent metodunun ayrı bir iş parçasığında JVM tarafından yürütülmesi gerekmektedir.

neden geçersiz kıl
getPreferredSize()

getPreferredSize() metodu (satırlar 95–97), Bileşen sınıfında tanımlanmış olup, FigurePanel'de geçersiz kılınmıştır. Bu, bir FigurePanel nesnesini düzenlerken düzen yöneticisinin dikkate alması için tercih edilen boyutu belirtir. Bu özellik, düzen yöneticisinin kurallarına bağlı olarak dikkate alınabilir veya alınmayabilir. Örneğin, bir bileşen, FlowLayout yöneticisine sahip bir konteynerde tercih edilen boyutunu kullanır, ancak GridLayout yöneticisine sahip bir konteynerde yerleştirildiğinde tercih edilen boyutu göz ardı edilir. JPanel'in bir alt sınıfında getPreferredSize() metodunu geçersiz kılmak iyi bir uygulamadır, çünkü JPanel için varsayılan genişlik ve yükseklik 0'dır. Varsayılan 0 genişlik ve yüksekliğe sahip bir JPanel bileşeni FlowLayout konteynerine yerleştirildiğinde hiçbir şey göremezsiniz.

13.12 Neden **JPanel** alt sınıfında **preferredSize** yöntemini geçersiz kılmalısınız?

13.13 Bir **Graphics** nesnesinde renkleri ve yazı tiplerini nasıl alırsınız ve belirlersiniz?

Nokta Kontrol Et
MyProgrammingLab™

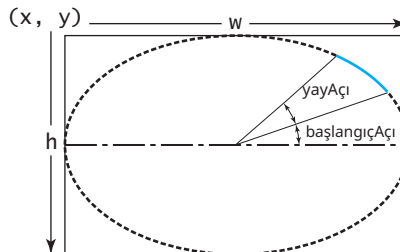
13.5 Yay Çizimi

Bir yay, bir dikdörtgenle sınırlı bir ovalin parçası olarak düşünülür.

Bir yay çizmek veya doldurmak için yöntemler şunlardır:

`DrawArc(tinttx,tintty,tinttw,tintth,tinttstartAngle,tinttarcAngle)nfillArc(tinttx,tintty,tinttw,tintth,tinttstartAngle,tinttarcAngle)`

Parametreler **x,y,w** ve **h**, **drawOval** yöntemindeki ile aynıdır; parametre **startAngle** başlangıç açısıdır; ve **arcAngle** yay tarafından kaplanan açıdır (yani, yay tarafından kaplanan açı). Açılar derece cinsinden ölçülür ve olağan matematiksel kurallara uyar (yani, 0'dan başlayarak doğu yönündedir ve pozitif açılar doğu yönünden saat yönünün tersine döngüyü gösterir); bkz. Şekil 13.10.



FIGÜR 13.10 Çizim **drawArc** yöntemi, belirtilen açılara göre bir oval temelinde bir yay çizer.



Liste 13.4, yayları nasıl çizeceğinize dair bir örnektir; çıktı Şekil 13.11'de gösterilmiştir.

LİSTE 13.4 DrawArcs.java

```

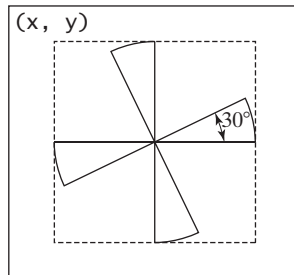
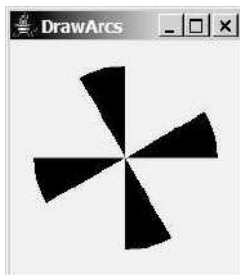
1  import javax.swing.JFrame;
2  import javax.swing.JPanel;
3  // içe aktarma, Java.Ah soru.grafikleriyle aynıdır;
4
5  genel classt DrawArcst extends JFrame {
6      public DrawArcs() {
7          setTitle("DrawArcs");
8          add(new DrawArcsPanel());
9      }
10
11     /** Ana yöntem */
12     public static void main(String[] args) {
13         DrawArcs çerçeve = new DrawArcs();
14         çerçeve.setSize(250, 300);
15         çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortala
16         çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17         çerçeve.setVisible(true);
18     }
19 }
20
21 // Bir JPanel üzerinde yay çizen sınıf
22 class ArcsPanel extends JPanel {
23     @Override // Bir fanın dört bıçağını çiz
24     protected void paintComponent(Graphics g) {
25         super.paintComponent(g);
26
27         int centerX = getWidth() / 2;
28         int centerY = getHeight() / 2;
29         int radius = (int) (Math.min(getWidth(), getHeight()) * 0.4);
30
31         int x = centerX - radius;
32         int y = centerY - radius;
33
34         g.fillArc(x, y, radius, radius, 0, 30);
35         g.fillArc(x, y, radius, radius, 90, 30);
36         g.fillArc(x, y, radius, radius, 180, 30);
37         g.fillArc(x, y, radius, radius, 270, 30);
38     }
39 }

```

bir panel ekle

t paintComponent'i geçersiz kıl

0°'den 30°'lik yay
90°'den 30°'lik yay
180°'den 30°'lik yay
270°'den 30°'lik yay



ŞEKİL 13.11 Program dört dolu yay çizer.

negatif dereceler

Açıların negatif olabileceği. Negatif bir başlangıç açısı, doğu yönünden saat yönünde döner, Şekil 13.12'de gösterildiği gibi. Negatif bir yay açısı, başlangıç açısından saat yönünde döner. Aşağıdaki iki ifade aynı yayı çizer:

```
g.fillArc(x, y, 2 * radius, 2 * radius, -30, -20);
g.fillArc(x, y, 2 * radius, 2 * radius, -50, 20);
```

İlk ifade negatif başlangıç açısını -30 ve negatif yay açısını -20 kullanır, Şekil 13.12a'da gösterildiği gibi. İkinci ifade negatif başlangıç açısını -50 ve pozitif yay açısını 20 kullanır, Şekil 13.12b'de gösterildiği gibi.

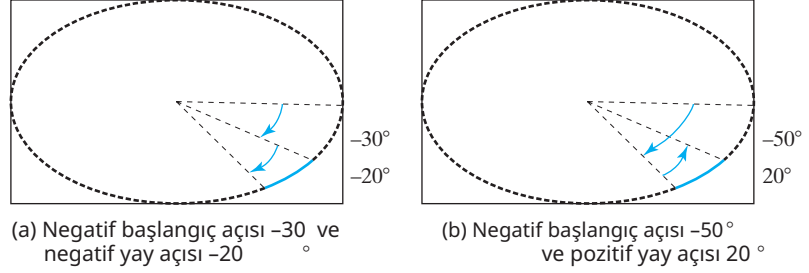


FIGURE 13.12 Açıların negatif olabileceği.

13.14 Yayları çizme/doldurma yöntemlerini tanımlayın.

13.15 Yarıçapı 50 olan bir dairenin üst yarısını çizin.

13.16 Yarıçapı 50 olan bir dairenin alt yarısını kırmızı renk kullanarak doldurun.

MyProgrammingLab™



13.6 Çokgenler ve Polilinler Çizme

Bir dizi noktayı birleştiren bir çokgen veya polilin çizebilirsiniz.

Bir çokgen çizmek için önce **PoLygon** sınıfını kullanarak bir **PoLygon** nesnesi oluşturun, bu

Şekil 13.13'te gösterilmiştir.

java.awt.Polygon	
+xpuan: int[]n+ypu an: int[]n+npuan: i nt	x-coordinates of all points in the polygon. y-coordinates of all points in the polygon. The number of points in the polygon.
+Çokgen() +Çokgen(xpoints: int[], ypoints: int[], n npoints: int)	Creates an empty polygon. Creates a polygon with the specified points.
+addPoint(x: int, y: int): voidn+contains(x: int, y: int): boolean	Appends a point to the polygon. Returns true if the specified point (x, y) is contained in the polygon.

FIGURE 13.13 **PoLygon** sınıfı bir çokgeni modellemektedir.

Bir çokgen, kapalı bir iki boyutlu alandır. Bu alan, her biri çokgenin bir kenarını (ve ya kenarını) oluşturan keyfî bir dizi doğru parçası ile sınırlıdır. Bir çokgen, her bir çifti n çokgenin bir köşesini tanımladığı bir (x, y) koordinat çiftleri listesi içerir ve iki ardışık çift, çokgenin bir kenarını oluşturan bir doğrunun uç noktalarıdır. İlk ve son noktalar, çokgeni kapatan bir doğru parçası ile birleştirilir.

İşte bir çokgen oluşturma ve içine noktalar ekleme örneği:

```
Çokgen poligon = tnewtPolygon();npolyg
on.addPoint(t40t,t20t);npolygon
.addPoint(t70t,t40t);npolygon.a
ddPoint(t60t,t80t);npolygon.ad
dPoint(t45t,t45t);npolygon.addP
oint(t20t,t60t);
```

Bu noktalar eklendikten sonra, **xpoints** {40,70,60,45,20}, **ypoints** {20,40,80,45,60} ve **npoints** 5. **xpoints**, **ypoints** ve **npoints**, çokgende kötü bir tasarım olan Polygon'da kamuya açık veri alanlarıdır. Kullanıcı, bir Polygon nesnesinin npoints veri alanını xpoints ve ypoints veri alanlarını düzgün bir şekilde değiştirmeden değiştirirse, bu Polygon nesnesinde tutarsız verilere yol açar.

Bir çokgeni çizmek veya doldurmak için, Graphics sınıfında aşağıdaki yöntemlerden birini kullanın:

```
drawPolygon(Polygon polygon);
fillPolygon(Polygon polygon);

drawPolygon(int[] xpoints, int[] ypoints, int npoints);
fillPolygon(int[] xpoints, int[] ypoints, int npoints);
```

Örneğin:

```
int
int y[] = {20, 40, 80, 45, 60};
g.drawPolygon(x, y, x.length);
```

Çizim yöntemi, (x[i], y[i]) noktasından point(x[i+1], y[i+1]) noktasına çizgiler çizerek çokgeni açar; çokgeni, ilk ve son noktalar arasında bir çizgi çizerek kapatır (bkz. Şekil 13.14a).

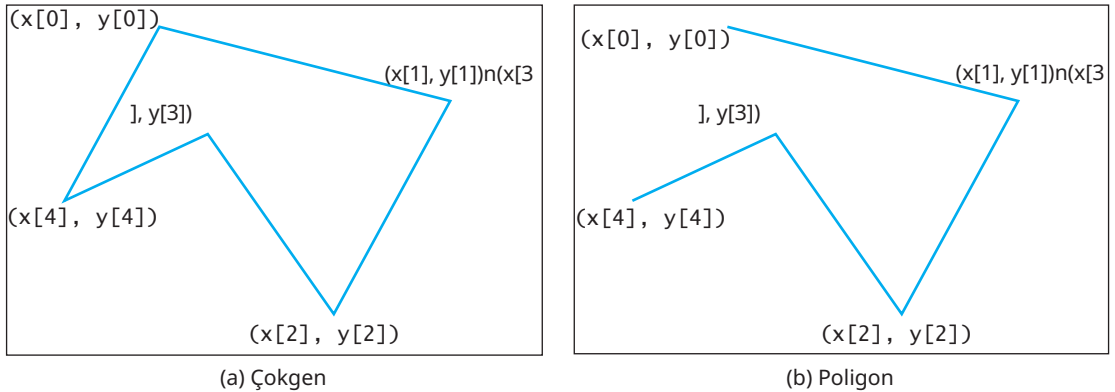


FIGURE 13.14 drawPolygon metodu bir çokgen çizer ve drawPolyLine metodu bir poligon çizer.

Bir poligon çizmek için, drawPolyline(int[] x, int[] y, int nPoints) metodunu kullanın; bu metod, x- ve y- koordinatlarının dizileriyle tanımlanan bağlı çizgiler dizisini çizer. Örneğin, aşağıdaki kod, Şekil 13.14b'de gösterilen poligonu çizer.

```
inttx[] = {t40t,t70t,t60t,t45t,t20t};nintty[
] = {t20t,t40t,t80t,t45t,t60t};ng.drawPol
yline(x, y, x.length);
```

Liste 13.5, bir altıgenin nasıl çizileceğine dair bir örnektir; çıktı Şekil 13.15'te gösterilmiştir.

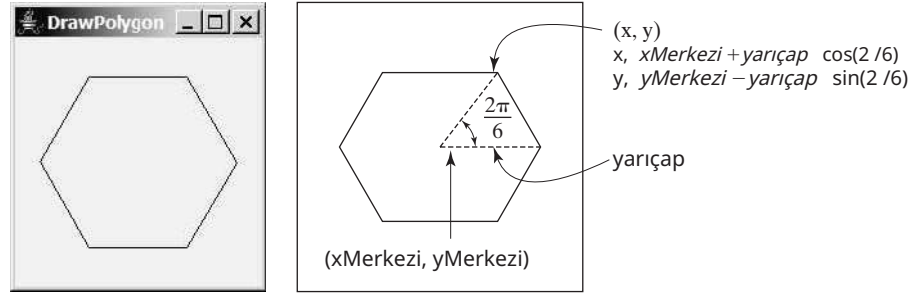


FIGURE 13.15 Program, bir altıgen çizmek için `drawPolygon` yöntemini kullanır.

Listing 13.5 `DrawPolygon.java`

```

1  import javax.swing.JFrame;
2  import javax.swing.JPanel;
3  // içe aktarma, Java.Ah soru.grafikleriyle aynıdır;
4  import java.awt.Polygon;
5
6  genel classt DrawPolygon extends JFrame {
7      public DrawPolygon() {
8          setTitle("DrawPolygon");
9          add(new
10             DrawPolygonsPanel());
11
12         /** Ana yöntem */
13         public static void main(String[] args) {
14             DrawPolygon çerçeve = new DrawPolygon();
15             çerçeve.setSize(200, 250);
16             çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortalayalım
17             çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18             çerçeve.setVisible(true);
19         }
20
21         // Panelde bir çokgen çiz
22         classt PolygonsPanel extends JPanel {
23             @Geçersiz Kıl
24             protected void paintComponent(Graphics g) {
25                 super.paintComponent(g);
26
27                 int centerX = getWidth() / 2;
28                 int centerY = getHeight() / 2;
29                 int radius = (int) (Math.min(getWidth(), getHeight()) * 0.4);
30
31                 // Bir Polygon nesnesi oluştur
32                 Polygon polygon = new Polygon();
33
34                 // Poligonun noktalarını bu sırayla ekle
35                 çokgen.noktaEkle (centerX + radius, centerY);
36                 çokgen.noktaEkle ((int) (centerX + radius *
37                     Math.cos(2 * Math.PI / 6)), (int) (centerY - radius *
38                     Math.sin(2 * Math.PI / 6)));
39                 çokgen.noktaEkle ((int) (centerX + radius *
40                     Math.cos(4 * Math.PI / 6)), (int) (centerY + radius *
41                     Math.sin(4 * Math.PI / 6)));
42                 çokgen.noktaEkle ((int) (centerX - radius *
43                     Math.cos(2 * Math.PI / 6)), (int) (centerY + radius *
44                     Math.sin(2 * Math.PI / 6)));
45                 çokgen.noktaEkle ((int) (centerX - radius *
46                     Math.cos(4 * Math.PI / 6)), (int) (centerY - radius *
47                     Math.sin(4 * Math.PI / 6)));
48                 çokgen.noktaEkle (centerX, centerY);
49             }
50         }
51     }
52 }

```

bir panel ekle

boya Bileşeni

bir nokta ekle

```

46  çokgen.noktaEkle((int)(xMerkezi + yarıçap *
47      Math.cos(4*2* Math.PI / 6)), (int)(yMerkezi - yarıçap *
48      Math.sin(4*2* Math.PI / 6)));
49  çokgen.noktaEkle((int)(xMerkezi + yarıçap *
50      Math.cos(5*2* Math.PI / 6)), (int)(yMerkezi - yarıçap *
51      Math.sin(5*2* Math.PI / 6)));
52
53      // Çokgeni çiz
54      g.drawPolygon(poligon);
55 }n56 }

```

çokgen çiz

13.17 Aşağıdaki noktaları birleştiren bir çokgen çizin: (20,40), (30,50), (40,90), (90,10), (10,30).

13.18 Bir Polygon nesnesi oluşturun ve noktaları ekleyin (20,40), (30,50), (40,90), (90,10), (10,30) bu sırayla. Çokgeni kırmızı renkle doldurun. Tüm bu noktaları birleştirmek için sarı renkte bir poligon çizin.

Nokta Kontrol Et

MyProgrammingLab™

13.7 Bir Dizeyi FontMetrics Sınıfını Kullanarak Ortalamak

FontMetrics sınıfını, bir dizeyi grafik bağlamında ölçmek için kullanabilirsiniz.



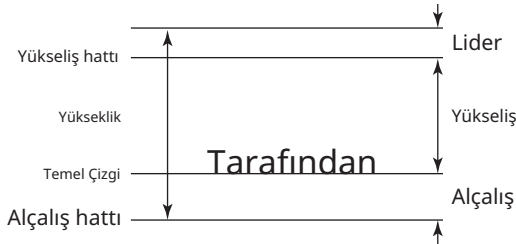
Bir dizeyi bir panelde herhangi bir konumda görüntüleyebilirsiniz. Merkezde görüntüleyebilir misiniz? Eve

t; bunu yapmak için, belirli bir yazı tipi için dizinin tam genişliğini ve yüksekliğini ölçmek üzere

FontMetrics sınıfını kullanmanız gerekir. FontMetrics, belirli bir yazı tipi için aşağıdaki özellikleri ölçebilir

(bkz.Şekil 13.16):

- **Leading**, *leading* olarak telaffuz edilir, metin satırları arasındaki boşluk miktarıdır.
- **Ascent**, temel çizgiden yükseliş çizgisine olan mesafedir. Yazı tipindeki çoğu karakterin üstü yükseliş çizgisinin altında olacaktır, ancak bazıları yükseliş çizgisinin üzerine çıkabilir.
- **Descent**, temel çizgiden alçalış çizgisine olan mesafedir. Çoğu alçalan karakteri n (örneğin, j, y ve g) altı alçalış çizgisinin üzerinde olacaktır, ancak bazıları alçalış çizgisinin altına uzanabilir.
- **Height**, leading, ascent ve descent'in toplamıdır.



ŞEKİL 13.16 FontMetrics sınıfı, belirli bir font için karakterlerin font özelliklerini belirlemek için kullanılabilir.

FontMetrics soyut bir sınıftır. Belirli bir font için bir FontMetrics nesnesi almak için, Graphics sınıfında tanımlanan aşağıdaki getFontMetrics yöntemlerini kullanın:

```
tpublic FontMetrics getFontMetrics(Yazı tipi yazı tipi)
```

Bu yöntem, belirtilen fontun font metriklerini döndürür.

```
tpublic FontMetrics getFontMetrics()
```

Bu yöntem, mevcut fontun font metriklerini döndürür.

FontMetrics sınıfında, bir fontun özelliklerini ve bir dize çizildiğinde genişliğini elde etmek için aşağıdaki örnek yöntemleri kullanabilirsiniz:

```
public int getAscent() // Yükselişi döndür
public int getDescent() // Alçalmayı döndür
public int getLeading() // Aralığı döndür
public int getHeight() // Yüksekliği döndür
public int stringWidth(String str) // Dizin genişliğini döndür
```

Liste 13.6, bir mesajı panelin ortasında gösteren bir örnek vermektedir, Şekil 13.17'de görüntü edildiği gibi.

Listing 13.6 TestCenterMessage.java

```
1  import javax.swing.*;
2  import java.awt.*;
3
4  genel sınıf TestCenterMessage extends JFrame {
5      public TestCenterMessage() {
6          CenterMessage messagePanel = new TestCenterMessage();
7          add(messagePanel);
8          messagePanel.setBackground(Color.WHITE);
9          messagePanel.setFont(new Font("Kaliforniya FB", Font.BOLD, 30));
10     }
11
12     /** Ana yöntem */
13     public static void main(String[] args) {
14         TestCenterMessage çerçevesi = new TestCenterMessage();
15         çerçevesi.setSize(300, 150);
16         çerçevesi.setTitle("CenterMessage");
17         çerçevesi.setLocationRelativeTo(null); // Çerçeveyi ortala
18         çerçevesi.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19         çerçevesi.setVisible(true);
20     }
21 }
22
23 class CenterMessage extends JPanel {
24     @Override /** Mesajı boyayın */
25     korumalı void paintComponent(Graphics g) {
26         super.paintComponent(g);
27
28         // Mevcut yazı tipi için yazı tipi metriklerini alın
29         FontMetrics fm = g.getFontMetrics();
30
31         // Gösterilecek merkezi konumu bulun
32         int stringWidth = fm.stringWidth("Java'ya Hoş Geldiniz");
33         int stringAscent = fm.getAscent();
34
35         // Temel çizgideki en soldaki karakterin konumunu alın
36         int xCoor = getWidth() / 2 - stringWidth / 2;
37         int yCoor = getHeight() / 2 + stringAscent / 2;
38
39         g.drawString("Java'ya Hoş Geldiniz", xCoor, yCoor);
40     }
41 }
```

bir mesaj paneli oluştur
bir mesaj paneli ekle
arka planı ayarla
fontu ayarla

t paintComponent'i geçersiz kıl

FontMetrics'i edin

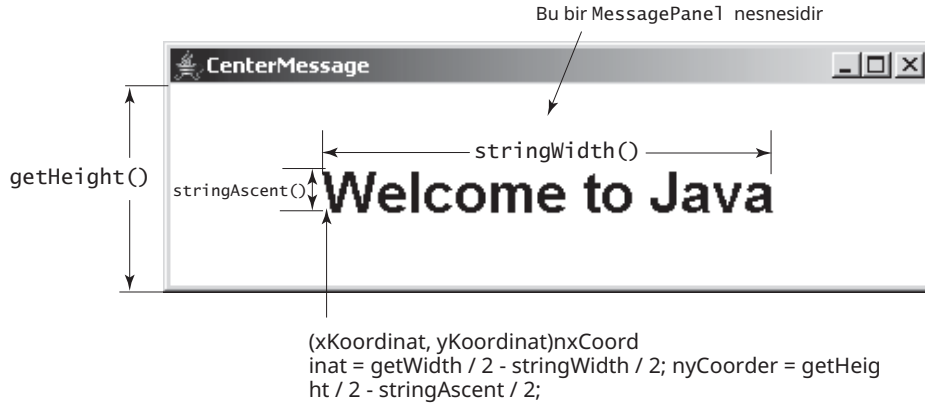


FIGURE 13.17 Program, string genişliğini ve

yüksekliğini ölçmek için FontMetrics sınıfını kullanır ve bunu panelin ortasında görüntüler.

getWidth() ve getHeight() (satırlar 36–37) Component sınıfında tanımlanmıştır ve sırasıyla bileşenin genişliğini ve yüksekliğini döndürür.

Mesaj **merkezlenmiş** olduğundan, stringin ilk karakteri

(xCoordinate, yCoordinate) konumuna yerleştirilmelidir; bu, Şekil 13.17'de gösterilmiştir.

13.19 Bir fontun leading, ascent, descent ve yüksekliğini nasıl bulursunuz?

13.20 Bir **Graphics** nesnesindeki bir stringin piksel cinsinden tam uzunluğunu nasıl bulursunuz?

13.8 Vaka Çalışması: MessagePane1 Sınıfı

Bu vaka çalışması, bir panelde bir mesaj görüntüleyen yararlı bir sınıf geliştirir. Sınıf, kullanıcının mesajın konumunu ayarlamasına, mesajı merkezlemesine ve mesajı belirli bir aralıkta hareket ettirmesine olanak tanır.

Sözleşme **MessagePanel** sınıfının görünümü Şekil 13.18'de gösterilmiştir.

Öncelikle, **MessagePanel** sınıfını kullanarak dört mesaj paneli görüntüleyen, Şekil 13.19'da gösterildiği gibi, Liste 13.7'de bir test programı yazalım.

LISTING 13.7 TestMessagePanel.java

```
1 import java.awt.*;
2 import javax.swing.*;
3
4 genel sınıf TestMessagePanel extends JFrame {
5     public TestMessagePanel() {
6         MesajPanel mesajPanel1 = new MessagePane1("Java'ya hoş geldiniz");
7         MessagePane1 mesajPanel2 = new MessagePane1("Java eğlencelidir");
8         MessagePane1 mesajPanel3 = new MessagePane1("Java harikadır");
9         MessagePane1 mesajPanel4 = new MessagePane1("Java'yı seviyorum");
10        mesajPanel1.setFont(new Font("SansSerif", Font.ITALIC, 20));
11        mesajPanel2.setFont(new Font("Courier", Font.BOLD, 20));
12        mesajPanel3.setFont(new Font("Times", Font.ITALIC, 20));
13        mesajPanel4.setFont(new Font("Californian FB", Font.PLAIN, 20));
14        mesajPanel1.setBackground(Color.RED);
15        mesajPanel2.setBackground(Color.CYAN);
16        mesajPanel3.setBackground(Color.GREEN);
17        mesajPanel4.setBackground(Color.WHITE);
```

Nokta Kontrol Et

MyProgrammingLab™



VideoNotu

MesajPaneli sınıfı

mesaj paneli oluştur

yazı tipini ayarla

arka planı ayarla

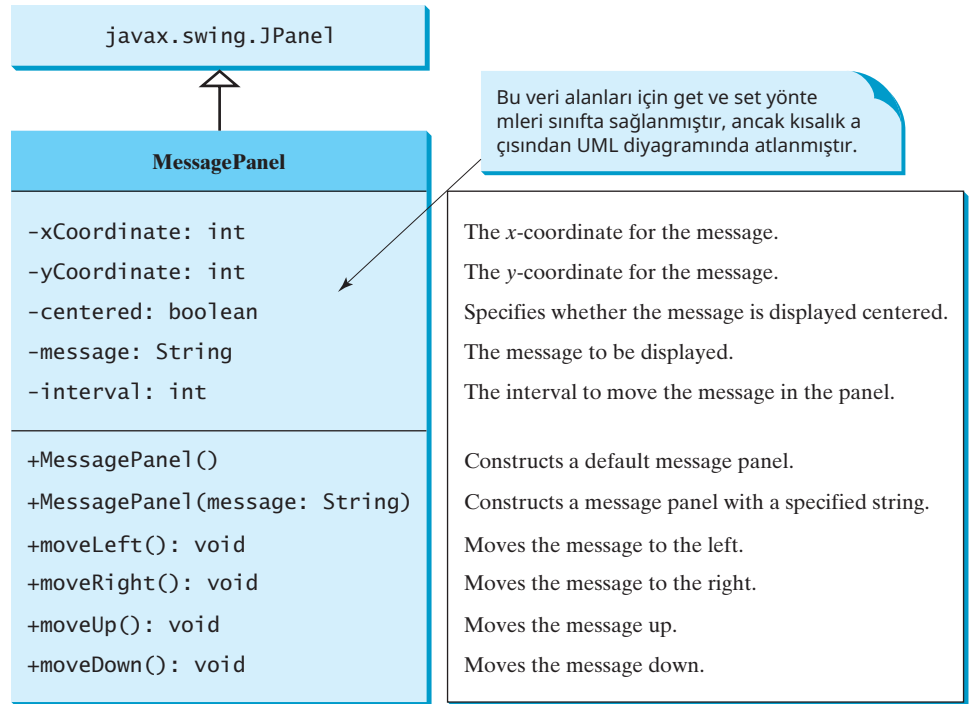
mesaj paneli ekle

```

18 messagePanel1.setCentered(true);
19
20 setLayout(new GridLayout(2,2));n21 add(me
ssagePanel1);
22 add(messagePanel2);n23 add(
messagePanel3);n24 add(messag
ePanel4);n25 }

26
27     public static void main(String[] args) {
28 TestMessagePanel çerçeve = new TestMessagePanel();n29 çerçeve.s
etSize(300,200);
30     frame.setTitle("TestMessagePanel");
31     frame.setLocationRelativeTo(null); // Çerçeveyi ortala
32     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
33     frame.setVisible(true);
34 {Nah}

```

FIGURE 13.18 **MessagePanel**, panelde bir mesaj görüntüler.FIGURE 13.19 **TestMessagePanel**, dört mesaj panelini görüntülemek için **MessagePanel** kullanır.

uygulamayı atla?

Bu bölümün geri kalanı, **MessagePanel** sınıfının nasıl uygulanacağını açıklar. Sınıfı nasıl uygulandığını bilmeden kullanabileceğiniz için, isterseniz uygulamayı atlayabilirsiniz.

`MessagePanel` sınıfı, Liste 13.8'de uygulanmıştır. Program uzun görünüyor ama aslında basit, çünkü çoğu yöntem `get` ve `set` yöntemleridir ve her bir yöntem görece kısaca ve okunması kolaydır.

LİSTE 13.8 `MessagePanel.java`

```

1  import java.awt.FontMetrics;
2  import java.awt.Dimension;
3  // içe aktarma, Java.Ah soru.grafikleriyle aynıdır;
4  import javax.swing.JPanel;
5
6  public class MesajPaneli extends JPanel {
7      /** Gösterilecek mesaj */
8      private String message = "Java'ya Hoş Geldiniz";
9
10     /** Mesajın gösterileceği x koordinatı */
11     özel int xKoordinat = 20;
12
13     /** Mesajın gösterileceği y koordinatı */
14     özel int yKoordinat = 20;
15
16     /** Mesajın ortada gösterilip gösterilmeyeceğini belirtir */
17     özel boolean merkezli;
18
19     /** Mesajı yatay olarak hareket ettirme aralığı
20         * ve dikey olarak */
21     özel int interval = 10;
22
23     /** Varsayılan özelliklerle oluştur */
24     public MesajPaneli() {
25     }
26
27     /** Belirtilen bir mesajla bir mesaj paneli oluştur */
28     public MesajPaneli(String mesaj) {
29         bu.mesaj = mesaj;
30     }
31
32     /** Mesajı döndür */
33     kamusal String getMessage() {
34         dönmesaj;
35     }
36
37     /** Yeni bir mesaj ayarla */
38     public void setMessage(String mesaj) {
39         bu.mesaj = mesaj;
40         yenidenÇiz();
41     }
42
43     /** xKoordinatı döndür */
44     genel int getXKoordinat() {
45         return xKoordinat;
46     }
47
48     /** Yeni bir xKoordinatı ayarla */
49     genel void setXCoorder(int x) {
50         bu.Koordinat = x;
51         yeniden boyama();
52     }
53
54     /** yKoordinatı döndür */

```

paneli yeniden boyayın

paneli yeniden boyayın

```

55     public int getYKoordinat() {
56         return yKoordinat;
57     }
58
59     /** Yeni bir yKoordinat ayarla */
60     public void setYKoordinat(int y) {
61         this.yKoordinat = y;
62         yenidenÇiz();
63     }
64
65     /** Ortalanmış olarak döndür */
66     public boolean merkezlenmişMi() {
67         return merkezlenmiş;
68     }
69
70     /** Mesajın merkezlenip merkezlenmediğini belirtmek için true veya false ayarlayın */
71     public void merkezle(boolean merkezlenmiş) {
72         this.merkezlenmiş = merkezlenmiş;
73         yenidenÇiz();
74     }
75
76     /** Aralık döndür */
77     public int getAralık() {
78         return aralık;
79     }
80
81     /** Yeni bir aralık ayarla */
82     public void setInterval(int interval) {
83         this.interval = aralık;
84         yenidenÇiz();
85     }
86
87     @Geçersiz kıl    /** Mesajı boyama */
88     korumalı void paintComponent(Grafik g) {
89         super.paintComponent(g);
90
91         if (merkezlenmiş) {
92             // Mevcut yazı tipi için yazı tipi metriklerini alın
93             FontMetrics fm = g.getFontMetrics();
94
95             // Gösterilecek merkezi konumu bulun
96             int stringWidth = fm.stringWidth(mesaj);
97             int stringAscent = fm.getAscent();
98             // Temel çizgideki en soldaki karakterin konumunu alın
99             xKoordinat = getWidth() / 2 - stringWidth / 2;
100             yKoordinat = getHeight() / 2 + stringAscent / 2;
101
102
103             g.drawString(mesaj, xKoordinat, yKoordinat);
104         }
105
106         /** Mesajı sola hareket ettir */
107         genel void moveLeft() {
108             xKoordinat -= aralık;
109             yenidenÇiz();
110         }
111
112         /** Mesajı sağa hareket ettir */
113         genel void moveRight() {
114             xKoordinat += aralık;

```

```

115 yenidenÇiz();
116 }
117
118 /** Mesajı yukarı taşı */
119 public void yukarıTaşı() {
120     yKoordinatı -= aralık;
121     yenidenÇiz();
122 }
123
124 /** Mesajı aşağı taşı */
125 public void aşağıTaşı() {
126     yKoordinatı += aralık;
127     yenidenÇiz();
128 }
129
130 @Override /** Tercih edilen boyut için get metodunu geçersiz kıl */
131 public Dimension getPreferredSize() {
132     return new Dimension(t200,t30t);
133 }
134 }

```

geçersiz kıl
getPreferredSize

paintComponent metodu, **centered** özelliği true ise mesajı ortalanmış olarak gösterir (satır 91). **message**, satır 8'de **Welcome to Java** olarak başlatılır. Eğer başlatılmamış olsaydı, **NullPointerException** çalışma zamanı hatası, argüman almayan yapıcı ile bir **MessagePanel** oluşturduğunuzda meydana gelecekti, çünkü **message**, satır 103'te **null** olacaktır.



Uyarı

MessagePanel sınıfı, panelde gösterilen mesajın konumunu belirtmek için xCordinate veya yCoordinate özelliklerini kullanır. x veya y özellik adlarını kullanmayın, çünkü bunlar zaten Component sınıfında, bileşenin ebeveyninin koordinat sistemindeki konumunu getX() ve getY() ile döndürmek için tanımlanmıştır.



Not

Component sınıfı, setBackground, setForeground ve setFont yöntemlerine sahiptir. Bu yöntemler, tüm bileşen için renkleri ve yazı tiplerini ayarlamak içindir. Eğer bir panelde farklı renkler ve yazı tipleriyle birkaç mesaj çizmek istiyorsanız, mevcut çizim için renk ve yazı tipini ayarlamak üzere Graphics sınıfındaki setColor ve setFont yöntemlerini kullanmanız gerekecektir.



Not

Java programlamanın temel özelliklerinden biri sınıfların yeniden kullanımudur. Bu kitap boyunca, yeniden kullanılabilir sınıflar geliştirilir ve daha sonra yeniden kullanılır. **MessagePanel** bir örnektir, ayrıca Listing 10.2'deki **Loan** ve Listing 13.3'teki **FigurePanel** de birer örnektir. **MessagePanel**, bir panelde bir mesaj görüntülenmesi gerektiğinde yeniden kullanılabilir. Sınıfınızı geniş bir uygulama yelpazesinde yeniden kullanılabilir hale getirmek için, onu kullanmanın çeşitli yollarını sağlamalısınız. **MessagePanel**, kitapta birçok örnekte kullanılacak birçok özellik ve yöntem sağlar.

yeniden kullanım için tasarım sınıfları

13.21 Eğer mesaj, Listing 13.8'deki 8. satırda başlatılmamışsa, MessagePanel.java, **MessagePanel**'i argüman almayan yapıcı ile oluşturduğunuzda ne olur?

13.22 Aşağıdaki program bir panelde bir mesaj görüntülenmesi bekleniyor, ancak hiçbir şey görüntülenmiyor. 2. ve 15. satırlarda sorunlar var. Düzeltin.

Nokta Kontrol Et

MyProgrammingLab™

```

1 genel classtTestDrawMessagetextendstjavax.swing.JFrame {
2     genel voidtTestDrawMessage() {
3         add(tnewtDrawMessage());n4 }
4
5
6     public static voidtmain(String[] args) {
7         javax.swing.JFrame çerçeve = tnewtTestDrawMessage();n8 çerçeve.s
9         etSize(t100t,t200t);
10        çerçeve.setVisible(ttrue);n10 }
11    }
12
13    classtDrawMessagetextendstjavax.swing.JPanel { n14 @O
15        verriide
16        korumalı voidPaintComponent(java.awt.Graphics g) {
17            supert.paintComponent(g);
18            g.drawString("Java'ya Hoş Geldiniz", 20, 20);
19        }
20    }

```

13.9 Vaka Çalışması: StillClock Sınıfı

Bu vaka çalışması, bir panelde saat gösteren bir sınıf geliştirir.

StillClock sınıfının sözleşmesi Şekil 13.20'de gösterilmiştir.



VideoNotu

StillClock sınıfı

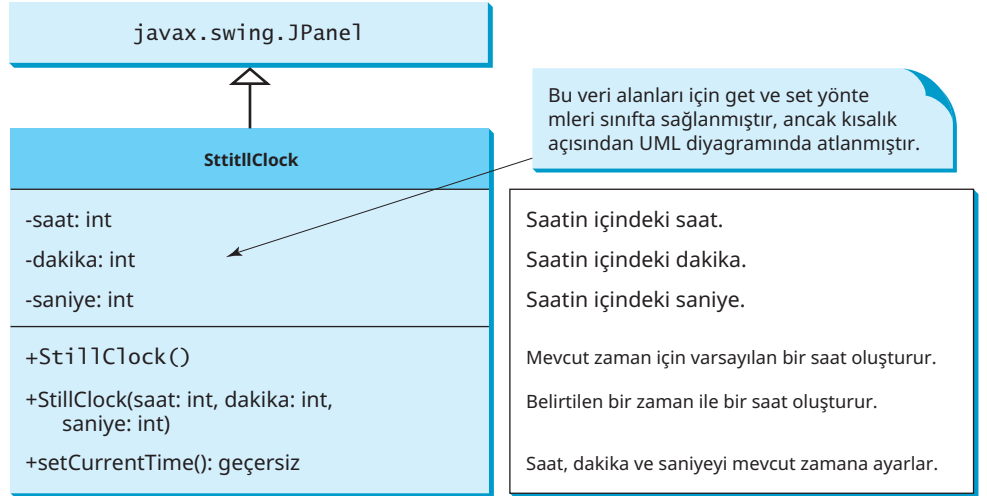


FIGURE 13.20 StillClock bir analog saat gösterir.

Öncelikle, Şekil 13.21'a'da gösterildiği gibi, **StillClock** sınıfını kullanarak bir analog saat görüntüleyen ve **MessagePanel** sınıfını kullanarak saat, dakika ve saniyeyi bir panelde görüntüleyen 13.9 numaralı listedeki bir test programı yazalım.

LISTE 13.9 DisplayClock.java

```

1 import java.awt.*;
2 import javax.swing.*;
3
4 genel classtDisplayClocktextendstJFrame {

```

```

5 public DisplayClock() {
6     // Mevcut zaman için bir analog saat oluştur
7     StillClock saat = new StillClock();
8
9     // Mesaj panelinde saat, dakika ve saniyeyi görüntüle
10    messagePanel = new JPanel(clock.getHour() +
11        ":" + saat.getMinute() + ":" + saat.getSecond());
12    messagePanel.setCentered(true);
13    messagePanel.setForeground(Color.blue);
14    messagePanel.setFont(new Font("Courier", Font.BOLD, 16));
15
16    // Saati ve mesaj panelini çerçeveye ekle
17    add(clock);
18    add(messagePanel, BorderLayout.SOUTH);
19 }
20
21 public static void main(String[] args) {
22    DisplayClock çerçeve = new DisplayClock();
23    çerçeve.setTitle("DisplayClock");
24    çerçeve.setSize(300, 350);
25    çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortala
26    çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27    çerçeve.setVisible(true);
28 }

```

saat oluştur

mesaj paneli oluştur

saat ekle

mesaj paneli ekle

28 {ekski
klik}

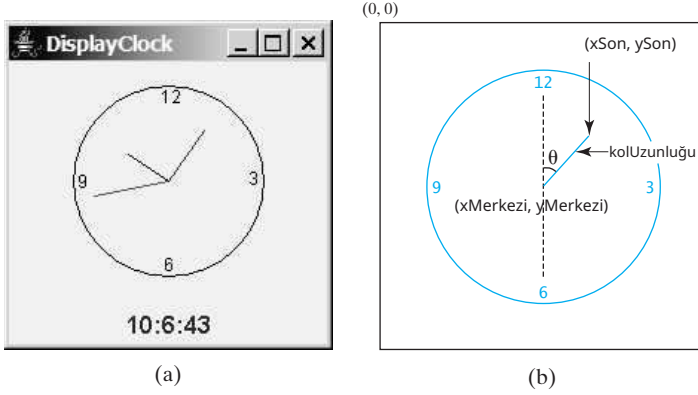


FIGURE 13.21 (a) DisplayClock programı, mevcut zamanı gösteren bir saat görüntüler. (b) Bir saat kolunun uç noktası, yayılan açı, kol uzunluğu ve merkez noktası verildiğinde belirlenebilir.

Bu bölümün geri kalanı, **StillClock** sınıfının nasıl uygulanacağını açıklar. Sınıfı nasıl uygulandığını bilmeden kullanabileceğiniz için, isterseniz uygulamayı atlayabilirsiniz.

uygulamayı atla?

Bir saat çizmek için, bir daire ve saniye, dakika ve saat için üç kol çizmeniz gerekir. Bir kol çizmek için, çizginin iki ucunu belirtmeniz gerekir. Şekil 13.21b'de gösterildiği gibi, bir uç, **(xMerkez, yMerkez)** saat merkezidir; diğer uç ise **(xSon, ySon)**, aşağıdaki formül ile belirlenir:

uygulama

$$\begin{aligned}
 x_{\text{Son}} &= x_{\text{Merkez}} + \text{kolUzunluğu} \times \sin(\theta) \\
 y_{\text{Son}} &= y_{\text{Merkez}} - \text{kolUzunluğu} \times \cos(\theta)
 \end{aligned}$$

Bir dakikada 60 saniye olduğundan, saniye kolu için açı

$$\text{saniye} \times (2\pi/60)$$

Dakika akısının konumu, dakika ve saniye ile belirlenir. Kesin dakika değeri saniyelerle birleştirildiğinde $\text{dakika} + \text{saniye}/60$ olur. Örneğin, eğer zaman 3 dakika ve 30 saniye ise, toplam dakikalar 3.5'tir. Bir saatte 60 dakika olduğundan, dakika akısı için aç

$$(\text{dakika} + \text{saniye}/60) \times (2\pi/60)$$

Bir dairenin 12 saate bölündüğünden, saat akrebinin açısı

$$(\text{saat} + \text{dakika}/60 + \text{saniye}/(60 \times 60)) \times (2\pi/12)$$

Akrelerin açılarını hesaplamada basitlik sağlamak için, saniyeleri atlayabilirsiniz, çünkü onlar ihmal edilebilir derecede küçüktür. Bu nedenle, saniye, dakika ve saat akrelerini uç noktaları şu şekilde hesaplanabilir:

```
xSecond = xCenter + secondHandLength * sin(saniye * (2π/60))
ySecond = yCenter - secondHandLength * cos(saniye * (2π/60))
xMinute = xCenter + minuteHandLength * sin(dakika * (2π/60))
yMinute = yCenter - minuteHandLength * cos(dakika * (2π/60))
xHour = xCenter + hourHandLength * sin((saat + dakika/60) * (2π/12))
yHour = yCenter - hourHandLength * cos((saat + dakika/60) * (2π/12))
```

StillClock sınıfı, Liste 13.10'da uygulanmıştır.

LISTE 13.10 StillClock.java

```
1  import java.awt.*;
2  import javax.swing.*;
3  import java.util.*;
4
5      genel sınıf StillClockText extends JPanel {
6      private int saat;
7      özel int dakika;
8      özel int ikinci;
9
10     /** Geçerli zamanla varsayılan bir saat oluştur */
11     public StillClock() {
12         setCurrentTime();
13     }
14
15     /** Belirtilen saat, dakika ve saniye ile bir saat oluştur */
16     public StillClock(int saat, int dakika, int saniye) {
17         this.saat = saat;
18         this.dakika = dakika;
19         this.saniye = saniye;
20     }
21
22     /** Saat döndür */
23     public int getSaat() {
24         return saat;
25     }
26
27     /** Yeni bir saat ayarla */
28     public void setSaat(int saat) {
29         this.saat = saat;
30         yenidenÇiz();
31     }
32
33     /** Dakikayı döndür */
```

```

34 genel inttgetMinute() {
35     return dakika;
36 }
37
38 /** Yeni bir dakika ayarla */
39 public void setMinute(int dakika) {
40     this.dakika = dakika;
41     yenidenÇiz();
42 }
43
44 /** İkinciye döndür */
45 genel inttgetSecond() {
46     dönüş saniyesi;
47 }
48
49 /** Yeni bir saniye ayarla */
50 public void setSecond(int second) {
51     this.saniye = saniye;
52     yeniden boyama();
53 }
54
55 @Override /** Saati çiz */
56 korumalı void paintComponent(Grafik g) {
57     super.paintComponent(g);
58
59     // Saat parametrelerini başlat
60     inttClockRadius =
61 (tintt)(Math.min(getWidth(), getHeight()) * t0.8t*t0.5t);
62     inttxCenter = getWidth() / t2t;
63     inttyCenter = getHeight() / t2t;
64
65     // Daire çiz
66 g.setColor(Color.BLACK);
67 g.drawO
68 val(xCenter - ClockRadius, yCenter - ClockRadius,
69     2 * saatRadius, t2t* saatRadius);
69 g.drawString(t"12"t, xCenter - t5t, yCenter - ClockRadius + t12t);
70 g.drawString(t"9"t, xCenter - ClockRadius + t3t, yCenter + t5t);
71 g.drawString(t"3"t, xCenter + Clock
72 Radius - t10t, yCenter + t3t);
73 g.drawString(t"6"t, xCenter - t3t, yCenter + watchRadi
74 us - t3t);
75
76 // Saniye akrebi çiz
77 inttsLength = (tintt)(clockRadius * t0.8t);
78 indeks İkinci = (renk tonu)(Merkez + Uzunluk *
79 Math.sin(saniye * (t2t* Math.PI / t60t)));
80 inttySecond = (tintt)(yCenter - sLength *
81     Math.cos(second * (2* Math.PI / 60)));
82 g.setColor(Color.red);
83 g.drawLine(xCenter, yCenter, xSecond, ySecond);
84
85 // Dakika ibresini çiz
86 inttmLength = (tintt)(clockRadius * t0.65t);
87 inttDakika = (renk tonu)(Merkez + Uzunluk *
88 Math.sin(dakika * (t2t* Math.PI / t60t)));
89 inttyMinute = (renk tonu)(Merkez - Uzunluk *
90     Math.cos(minute * (2* Math.PI / 60)));
91 g.setColor(Color.blue);
92 g.drawLine(xCenter, yCenter, xMinute, yMinute);
93
94 // Saat ibresini çiz
95 intthLength = (tintt)(clockRadius * t0.5t);

```

paneli yeniden boyayın

paneli yeniden boyayın

t paintComponent'i geçersiz kıl

```

94         intt Saat = (renk tonu)(Merkez + Uzunluk *
95 Math.sin((saat % t12t+ dakika / t60.0t) * (t2t* Math.PI / t12t)));
96         inttyHour = (tintt)(yCenter - hLength *
97         Math.cos((hour % 12+ minute / 60.0) * (2* Math.PI / 12)));
98         g.setColor(Color.green);
99 g.drawLine(xCenter, yCenter, xHour, yHour);n100 }

101102         genel voidtsetCurrentTime() {
103         // Mevcut tarih ve saat için bir takvim oluştur
geçerli zamanı al
104 Takvim takvimi = tnewtGregorianCalendar();

105106         // Mevcut saat, dakika ve saniyeyi ayarla
107         thist.hour = takvim.get(Calendar.HOUR_OF_DAY);
108         thist.dakika = takvim.get(Calendar.MINUTE);
109         this.ikinci = calendar.get(Calendar.SECOND);
110     }
111
112 @Geçersiz kıl
geçersiz kıl
113         publictDimension getPreferredSize() {
114         return newtDimension(t200t,t200t);
115 }n116
117     }

```

Program, çerçeve yeniden boyutlandırıldıkça saat boyutunun ayarlanmasına olanak tanır. Her seferinde çerçeveyi yeniden boyutlandırdığınızda, `paintComponent` metodu otomatik olarak yeni bir saati boyamak için çağrılır. `paintComponent` metodu, saati panel genişliğine (`getWidth()`) ve yüksekliğine (`getHeight()`) orantılı olarak gösterir (satırlar 60–63 `StillClock` içinde).

13.10 Görüntüleri Gösterme



Grafik bağlamında görüntüleri çizebilirsiniz.

Görüntü simgeleri oluşturmayı ve bunları etiketlerde ve düğmelerde göstermeyi 12.10. Bölüm, Görüntü Simgeleri'nde öğrendiniz. Örneğin, aşağıdaki ifadeler bir görüntü simgesi oluşturur ve bunu bir etikette gösterir:

```

ImageIcon imageIcon = tnewtImageIcon(t"image/us.gif"t);nJLabel
jlblImage = tnewtJLabel(imageIcon);

```

Bir görüntü simgesi sabit boyutlu bir görüntü gösterir. Esnek boyutta bir görüntü göstermek için `java.awt.Image` sınıfını kullanmanız gerekir. Bir görüntü, aşağıdaki gibi `getImage()` metodu kullanılarak bir görüntü simgesinden oluşturulabilir:

```
Resim resmi = imageIcon.getImage();
```

Bir etiketi görüntüleri görüntülemek için bir alan olarak kullanmak basit ve kullanışlıdır, ancak görüntünün nasıl görüntüleneceği üzerinde çok fazla kontrolünüz yoktur. Görüntüleri görüntülemenin daha esnek bir yolu, bir panelde `Graphics` sınıfının `drawImage` yöntemini kullanmaktır. `drawImage` yönteminin dört versiyonu Şekil 13.22'de gösterilmektedir.

ImageObserver, görüntü bilgilerini alacak bir GUI bileşenini belirtir-görüntü oluşturulurken. `JPanel` gibi bir Swing

bileşeninde `drawImage` yöntemini kullanarak görüntüleri çizmek için, `paintComponent` yöntemini geçersiz kılın ve bileşene panelde görüntüyü nasıl göstereceğini söyleyin.

Liste 13.11, `image/us.gif` dosyasından bir görüntü gösteren kodu verir. Dosya `image/us.gif` (satır 20) sınıf dizinindedir. Satır 21'de bir `Image` nesnesi elde edilir. `drawImage` yöntemi, görüntüyü tüm paneli dolduracak şekilde gösterir, Şekil 13.23'te gösterildiği gibi.

<i>java.awt.Graphics</i>	
+drawImage(image: Image, x: int, y: int, bgcolor: Color, observer: ImageObserver): void	Draws the image in a specified location. The image's top-left corner is at (x, y) in the graphics context's coordinate space. Transparent pixels in the image are drawn in the specified color <i>bgcolor</i> . The observer is the object on which the image is displayed. The image is cut off if it is larger than the area it is being drawn on.
+drawImage(image: Image, x: int, y: int, observer: ImageObserver): void	Same as the preceding method except that it does not specify a background color.
+drawImage(image: Image, x: int, y: int, width: int, height: int, observer: ImageObserver): void	Draws a scaled version of the image that can fill all of the available space in the specified rectangle.
+drawImage(image: Image, x: int, y: int, width: int, height: int, bgcolor: Color, observer: ImageObserver): void	Same as the preceding method except that it provides a solid background color behind the image being drawn.

FIGURE 13.22 *Graphics* nesnesi üzerindeki *drawImage* yöntemini uygulayarak bir resmi bir GUI bileşeninde görüntüleyebilirsiniz.

LİSTE 13.1 *DisplayImage.java*

```

1  import java.awt.*;
2  import javax.swing.*;
3
4  genel class DisplayImage extends JFrame {
5      public DisplayImage() {
6          add(new ImagePanel());
7      }
8
9      public static void main(String[] args) {
10         JFrame çerçeve = new DisplayImage();
11         çerçeve.setTitle("DisplayImage");
12         çerçeve.setSize(300, 300);
13         çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortalara
14         çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15         çerçeve.setVisible(true);
16     }
17 }
18
19 class ImagePanel extends JPanel {
20     özel ImageIcon imageIcon = new ImageIcon("image/us.gif");
21     özel Resim image = imageIcon.getImage();
22
23     @Override /** Panelde resmi çiz */
24     korumalı void paintComponent(Graphics g) {
25         super.paintComponent(g);
26
27         eğer (resim != null)
28             g.drawImage(image, 0, 0, getWidth(), getHeight(), this);
29     }
30 }

```

panel ekle

panel sınıf
resim simgesi oluştur
resmi al

t paintComponent'i geçersiz kıl

resmi çiz

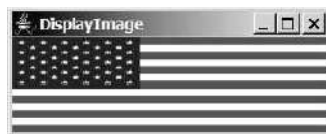


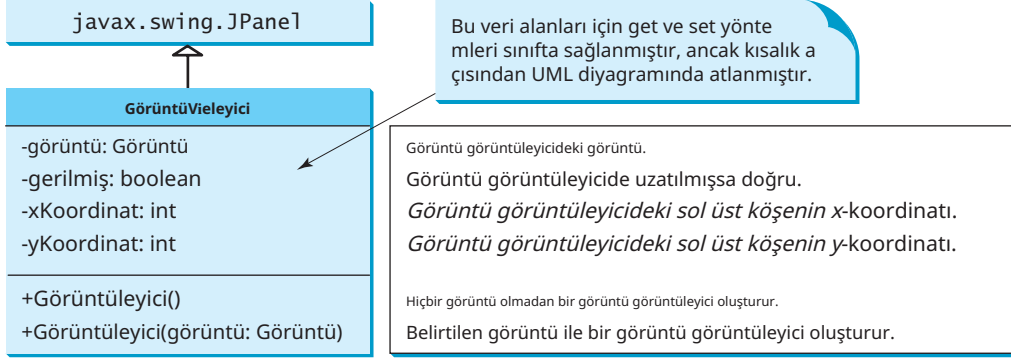
FIGURE 13.23 Bir resim bir panelde görüntülenir.

13.11 Vaka Çalışması: ImageViewer Sınıfı



*Bu vaka çalışması, bir resmi bir panelde görüntülemek için **ImageViewer** sınıfını geliştirir.*

Bir resmi görüntülemek, Java programlamasında yaygın bir görevdir. Bu vaka çalışması, bir panel de bir resmi görüntüleyen **ImageViewer** adlı yeniden kullanılabilir bir bileşen geliştirir. Sınıf, Şekil 13.24'te gösterildiği gibi, **image.stretched.xCoordinate** ve **yCoordinate** özelliklerini içerir ve bunlarla ilişkili erişim ve değişim yöntemlerine sahiptir.



Fiçe 13.24 GörüntüGörüntüleyici sınıfı bir panelde bir görüntü gösterir.

esnetilebilir resim

Swing bileşenlerinde **JLabel** ve **JButton** gibi resimler kullanabilirsiniz, ancak bu resimler esnetilebilir değildir. **ImageViewer** içindeki resim esnetilebilir.

Altı resmi görüntüleyen bir test programı yazalım, Liste 13.12'de gösterildiği gibi.

ImageViewer sınıfının bir örneği. Şekil 13.25, programın örnek bir çalışmasını göstermektedir.

LISTING 13.12tSixFlags.java

görüntü oluştur

görüntü görüntüleyici oluştur

```

1  import javax.swing.*;
2  import java.awt.*;
3
4  genel classtSixFlagstextendstJFrame {
5      publicSixFlags() {
6          Resim image1 = tnewtImageIcon(t"image/us.gif"t).getImage();n7 Resim image
7          2 = tnewtImageIcon(t"image/ca.gif"t).getImage();n8 Resim image3 = tnewtImage
8          3 = tnewtImageIcon(t"image/india.gif"t).getImage();n9 Resim image4 = tnewtImageIcon(t"image/u
9          4 = tnewtImageIcon(t"image/uk.gif"t).getImage();n10 Resim image5 = tnewtImageIcon(t"image/china.gif"t).getI
10         5 = tnewtImageIcon(t"image/china.gif"t).getImage();n11 Resim image6 = tnewtImageIcon(t"image/norway.gif"t).getImage();
11
12
13         setDüzen(          newtGridLayout(t2t,t0t,t5t,t5t));
14         add(tnewtImageViewer(image1));n15 add(tnewtImageViewer(image2));n16 add(tnewtImage
15         2));n16 add(tnewtImage
16         3));n17 add(tnewtImageViewer(i
17         4));n18 add(tnewtImageViewer(image5));
18         n19 add(tnewtImageViewer(image6));n20 }
19
20
21
22         public static voidtmain(String[] args) {
23             SixFlags çerçeve = tnewtSixFlags();n24 çerçeve.s
24             etTitle(t"SixFlags"t);
25             frame.setSize(400, 320);
26             frame.setLocationRelativeTo(nu11); // Çerçeveyi ortalayalım

```

```

27 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28 çerçeve.setVisible(true);
kes}na0}

```

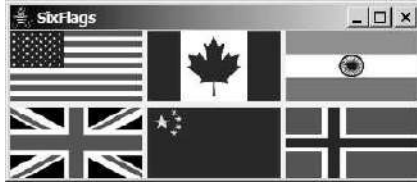


FIGURE 13.25 Altı görüntü, altı **ImageViewer** bileşeninde görüntülenmektedir.

ImageViewer sınıfı, Liste 13.13'te uygulanmıştır. (Not: Uygulamayı atlayabilirsiniz.) image, stretched, xCoordinate ve yCoordinate özellikleri için erişimci ve değiştirici yöntemler kolayca uygulanabilir. paintComponent yöntemi (satırlar 27–36) resmi panelde görüntüler. Satır 30, resmi görüntülemeye başlamadan önce null olmadığını garanti eder. Satır 31, resmin uzatılıp uzatılmadığını kontrol eder.

uygulama
uygulamayı atla?

LISTING 13.13 ImageViewer.java

```

1  import java.awt.*;
2  import javax.swing.*;
3
4  genel sınıf ImageViewer extends JPanel {
5      /** Özellik image'in değerini tutar */
6      private java.awt.Resim resmi;
7
8      /** Özellik stretched'in değerini tutar */
9      özel boolean stretched = true;
10
11     /** Özellik xCoordinate'in değerini tutar */
12     özel int xCoordinate;
13
14     /** yCoordinate özelliğinin değerini tut */
15     özel int yCoordinate;
16
17     /** Boş bir resim görüntüleyici oluştur */
18     kamusal ImageViewer() {
19     }
20
21     /** Belirtilen bir Image nesnesi için bir resim görüntüleyici oluştur */
22     kamusal ImageViewer(Image image) {
23         bu.image = image;
24     }
25
26     @Geçersiz Kıl
27     korumalı void çizimBileşeni(Graphics g) {
28         super.paintComponent(g);
29
30         eğer(resmi != null)
31             g.drawImage(image, xCoordinate, yCoordinate, n33.getWidth(), getHeight(), this);
32
33         başka
34         g.drawImage(resmi, xKoordinat, yKoordinat, tthist);
35     }
36 }

```

özellikler

yapıcı

yapıcı

görüntü null?

gerilmiş

uzatılmamış

```

3738      /** Özellik görüntüsünün dönüş değeri */
39      public java.awt.Image getImage() {
40          dönüş zamanı;
41      }
42
43      /** Özellik görüntüsü için yeni bir değer ayarla */
44      public void setImage(java.awt.Image resmi) {
45          bu.image = resmi;
46          yenidenÇiz();
47      }
48
49      /** Özellik uzatılmış değerini döndür */
50      public boolean isStretched() {
51          dönüş gerilmiş;
52      }
53
54      /** Özellik stretched için yeni bir değer ayarla */
55      public void setStretched(boolean stretched) {
56          this.stretched = stretched;
57          yeniden boyama();
58      }
59
60      /** Özellik xCoordinate'in değerini döndür */
61      public int getXCoordinate() {
62          return xKoordinat;
63      }
64
65      /** Özellik xCoordinate için yeni bir değer ayarla */
66      public void setXCoordinate(int xKoordinat) {
67          this.Koordinat = xKoordinat;
68          yenidenÇiz();
69      }
70
71      /** Özellik yCoordinate'in dönüş değeri */
72      public int getYCoordinate() {
73          return yKoordinat;
74      }
75
76      /** Özellik yCoordinate için yeni bir değer ayarla */
77      public void setYCoordinate(int yKoordinat) {
78          this.Koordinat = yKoordinat;
79          yenidenÇiz();
80      }
81      }

```

13.23 `Image` nesnesini `ImageIcon` nesnesinden nasıl oluşturursunuz?

13.24 `Image` nesnesinden `ImageIcon` nesnesini nasıl oluşturursunuz?

13.25 `Graphics` sınıfındaki `drawImage` yöntemini tanımlayın.

13.26 Bir `JLabel` içinde ve bir `JPanel` içinde görüntüleri görüntülemenin farklarını açıklayın.

13.27 Hangi paket `ImageIcon` içerir ve hangisi `Image` içerir?

BÖLÜM ÖZETİ

1. Her bileşenin, pencerenin sol üst köşesinde (0, 0) orijini olan kendi koordinat sistemi vardır. Java'da, x- koordinatı sağa doğru artar ve y- koordinatı aşağıya doğru artar.

2. Bir bileşen (örneğin, bir düğme, bir etiket veya bir panel) görüntülendiğinde, JVM otomatik olarak yerel platformda bileşen için bir **Graphics** nesnesi oluşturur ve bu nesneyi **paintComponent** yöntemini çağırmak için geçirir.
çizimleri görüntülemek için.
3. Normalde **JPanel**'i bir tuval olarak kullanırsınız. Bir **JPanel** üzerinde çizim yapmak için, **JPanel**'i genişleten yeni bir sınıf oluşturur ve panelin
grafikleri nasıl çizeceğini belirtmek için paintComponent yöntemini geçersiz kılarırsınız.
4. **super.paintComponent(g)** çağrısı, yeni bir çizim gösterilmeden önce görüntüleme alanının temizlenmesini sağlamak için gereklidir. Kullanıcı, **Component** sınıfında tanımlanan **repaint()** yöntemini çağırarak bileşenin yeniden görüntülenmesini isteyebilir. **repaint()** çağrısı, JVM tarafından **paintComponent**'in çağrılmasına neden olur. Kullanıcı asla **paintComponent**'i doğrudan çağırmamalıdır. Bu nedenle, **paintComponent** için korumalı görünürlük yeterlidir.
5. **Component** sınıfı **setBackground**, **setForeground** ve **setFont** yöntemlerine sahiptir. Bu yöntemler, tüm bileşen için renkleri ve yazı tiplerini ayarlamak için kullanılır. Farklı renkler ve yazı tipleriyle bir panelde birkaç mesaj çizmek istiyorsanız, **Graphics** sınıfındaki **setColor** ve **setFont** yöntemlerini kullanarak mevcut çizim için renk ve yazı tipi ayarlamanız gerekir.
6. **FontMetrics**, bir dizeyi tam olarak ölçmek için kullanılabilir; bu, bir dizeyi doğru konumda görüntülemek için boyutunu ölçmekte yardımcı olur.
7. Bir resmi görüntülemek için önce bir resim simgesi oluşturun. Daha sonra **ImageIcon**'in **getImage()** yöntemini kullanarak resim için bir **Image** nesnesi alabilir ve resmi **java.awt.Graphics** sınıfındaki **drawImage** yöntemi ile çizebilirsiniz.

TEST SORULARI

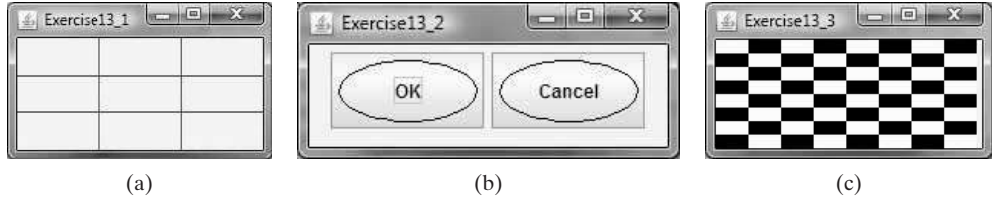
Bu bölüm için test sorularını çevrimiçi olarak yapın: www.cs.armstrong.edu/liang/intro9e/test.html.

PROGRAMLAMA EGZERSİZLERİ

MyProgrammingLab™

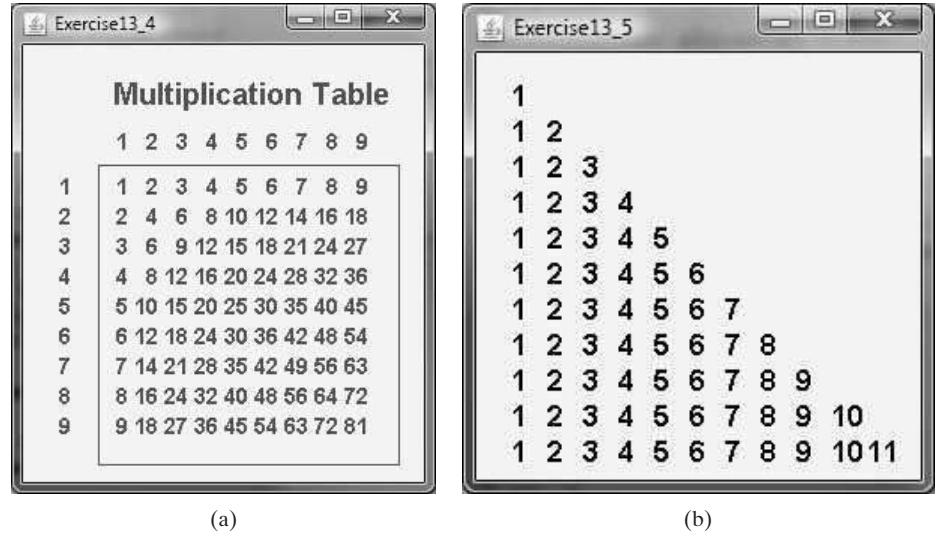
Bölümler 13.2–13.7

- * **13.1** (*Bir 3×3 ızgara*) Bir program yazın, bu program bir 3×3 ızgara, Şekil 13.26a'da gösterildiği gibi. Dikey çizgiler için kırmızı, yataylar için mavi renk kullanın.
- ** **13.2** (*Özel bir düğme sınıfı oluşturun*) Özel bir düğme sınıfı adıyla geliştirin **OvalButtonJButton** sınıfını genişleten ve düğme metnini bir oval içinde gösteren bir bileşendir. Şekil 13.26b, **OvalButton** sınıfını kullanarak oluşturulan iki düğmeyi göstermektedir.
- * **13.3** (*Bir satranç tahtası göster*) Programlama Egzersizi 12.10, her beyaz ve siyah hücrenin bir **JButton** olduğu bir satranç tahtası gösterir. **JPanel** üzerinde **Graphics** sınıfındaki çizim yöntemlerini kullanarak bir satranç tahtası çizen bir programı yeniden yazın, Şekil 13.26c'de gösterildiği gibi. Satranç tahtasındaki her hücreyi çizmek için **drawRect** yöntemini kullanın.
- * **13.4** (*Çarpanlar tablosunu görüntüle*) Çizim yöntemlerini kullanarak bir panelde çarpanlar tablosunu görüntüleyen bir program yazın, Şekil 13.27a'da gösterildiği gibi.



FIGİN 13.26(a) Alıştırma 13.1 bir ızgara gösterir. (b) Alıştırma 13.2 iki nesne gösterir **OvalButton**. (c) Egzersiz 13.3 bir dama tahtası gösterir.

**** 13.5**(Sayıları üçgen bir desende göster) Üçgen bir desende sayıları gösteren bir program yazın, Şekil 13.27b'de gösterildiği gibi. Gösterimdeki satır sayısı, pencere yeniden boyutlandırıldıkça pencereye uyacak şekilde değişir.



FIGİN 13.27(a) Alıştırma 13.4 bir çarpan tablosu gösterir. (b) Alıştırma 13.5 sayıları bir üçgen formasyonunda gösterir.

**** 13.6**(**FigurePanel** Sınıfını Geliştir) Listing 13.3'teki **FigurePanel** sınıfı çizgileri, dikdörtgenler, yuvarlak köşeli dikdörtgenleri ve ovaları görüntüleyebilir. Sınıfa uygun yeni kod ekleyerek yaylar ve çokgenleri görüntüleyin. Yeni **FigurePanel** sınıfını kullanarak Şekil 13.28a'da gösterildiği gibi şekilleri görüntülemek için bir test programı yazın.

**** 13.7**(Bir üç taş tahtası göster) **X,0** veya hiçbir şey göstermeyen özel bir panel oluşturun. Bir panel yeniden boyandığında ne gösterileceği rastgele belirlenir. Kullan **Math.random()** yöntemi, **0,1** veya **2** olan bir tamsayı üretmek için kullanılır; bu, **X,0** veya hiçbir şeyin görüntülenmesine karşılık gelir. Şekil 13.28b'de gösterildiği gibi, dokuz özel panel içer en bir çerçeve oluşturun.

**** 13.8**(Sekizgen çizim) Şekil 13.28c'de gösterildiği gibi bir sekizgen çizen bir program yazın.

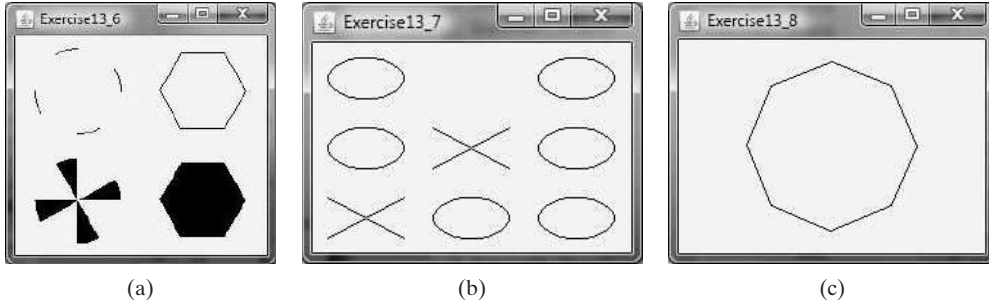


FIGURE 13.28(a) Dört panel geometrik şekillerin bir **GridLayout** çerçevesinde görüntülenmektedir. (b) **TicTacToe** hücreleri rastgele X, O veya hiçbir şey gösterir. (c) Egzersiz 13.8 bir sekizgen çizer.

*** 13.9** (*Dört fan oluştur*) Dört fanı bir çerçeveye yerleştiren bir program yazın **iki satır ve iki sütun ile GridLayout**, Şekil 13.29a'da gösterildiği gibi.

*** 13.10** (*Bir silindiri göster*) Şekil 13.29b'de gösterildiği gibi bir silindir çizen bir program yazın.

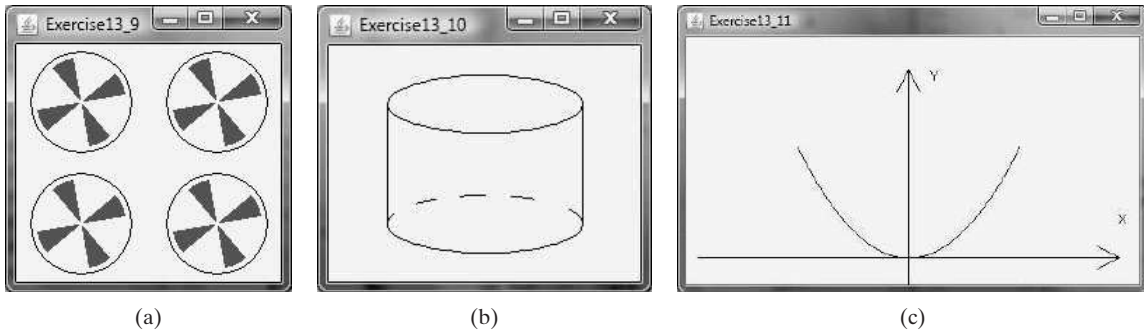


FIGURE 13.29(a) Egzersiz 13.9 dört fan çizer. (b) Egzersiz 13.10 bir silindir çizer. (c) Egzersiz 13.11 fonksiyon için bir diyagram çizer $f(x)=x^2$.

*** 13.11** (*Kare fonksiyonunu çiz*) Fonksiyon için bir diyagram çizen bir program yazın

$f_t(x)t = t \times t^2$ (Bkz. Şekil 13.29c).

İpucu: Aşağıdaki döngüyü kullanarak bir çokgen **p**'ye noktalar ekleyin:

```
double scaleFactor = t0.1t;
```

```
for(tinttx = t-100t; x <= t100t; x++) {np.addP  
oint(x + t200t,t200t- (tintt)(scaleFactor * x * x));n}
```

Bağlantı noktalarını kullanarak **g.drawPolyline(p.xpoints, p.ypoints, p.npoints)** bir **Graphics** nesnesi için **p.xpoints** bir dizi **x**-koordinatlarını, **p.ypoints** bir dizi **y**-koordinatlarını ve **p.npoints** **Polygon** nesnesindeki noktaların sayısını döndürür. **p**.

**** 13.12** (*Sine ve cosinus fonksiyonlarını çiz*) Kırmızıda sinüs fonksiyonunu ve mavi renkte cosinus fonksiyonunu çizen bir program yazın, Şekil 13.30a'da gösterildiği gibi.



VideoNotu

Bir fonksiyonu çiz

İpucu: π için Unicode

`\u03c0`. 2π görüntülemek için, kullanın

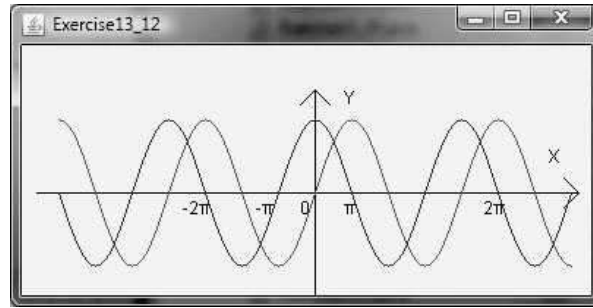
`g.drawString("-2 $\pi$ ", x, y)`. Bir trigonometrik fonksiyon için, örneğin

`sin(x)`, x radyan cinsindendir. Aşağıdaki döngüyü kullanarak noktaları bir çokgene `p` ekleyin:

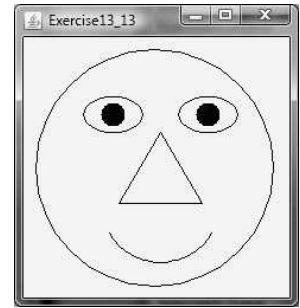
```
for(int x = -170; x <= 170; x++) {
    p.addPoint(x + 200,
        100 (tint)(t50t * Math.sin((x / t 100.0t) * t 2t * Math.PI)));n}
```

-2π noktası (100,100) konumundadır, eksenin merkezi (200,100) konumundadır ve 2π noktası (300,100) konumundadır. `drawPolyLine` yöntemini `Graphics` sınıfında kullanarak noktaları birleştirin.

****13.13** (Gülen yüz resmi yap) Şekil 13.30b'de gösterildiği gibi bir gülen yüz resmi çizen bir program yazın.



(a)



(b)

FIGURE 13.30(a) Alıştırma 13.12 sinüs/kosinüs fonksiyonlarını çizer. (b) Alıştırma 13.13 bir gülen yüz resmi çizer.

****13.14** (Bir çubuk grafik göster) Projeler, quizler, ara sınavlar ve final sınavı tarafından temsil edilen genel notun yüzdesini gösteren bir çubuk grafik kullanan bir program yazın. Şekil 13.1a'da gösterildiği gibi. Projelerin 20 yüzdesini aldığını ve kırmızı renkte gösterildiğini varsayın, quizlerin 10 yüzdesini aldığını ve mavi renkte gösterildiğini varsayın, ara sınavların 30 yüzdesini aldığını ve yeşil renkte gösterildiğini varsayın ve final sınavının 40 yüzdesini aldığını ve turuncu renkte gösterildiğini varsayın.

****13.15** (Bir pasta grafik göster) Projeler, quizler, ara sınavlar ve final sınavı tarafından temsil edilen genel notun yüzdesini gösteren bir pasta grafik kullanan bir program yazın. Şekil 13.31a'da gösterildiği gibi. Projelerin 20 yüzdesini aldığını ve kırmızı renkte gösterildiğini varsayın, quizlerin 10 yüzdesini aldığını ve mavi renkte gösterildiğini varsayın, ara sınavların 30 yüzdesini aldığını ve yeşil renkte gösterildiğini varsayın ve final sınavının 40 yüzdesini aldığını ve turuncu renkte gösterildiğini varsayın.

13.16 (Yazı tipi bilgilerini al) Java eğlencelidirmesajını bir panelde gösteren bir program yazın. Panelin yazı tipini `TimesRoman`, kalın ve 20 piksel olarak ayarlayın. Panel için yazı tipinin başlık, yükseklik, alçaklık ve dize genişliğini araç ipucu metni olarak gösterin, Şekil 13.31b'de gösterildiği gibi.

13.17 (Oyun: adam asmaca) Şekil 13.31c'de gösterildiği gibi, popüler adam asmaca oyunu için bir çizim gösteren bir program yazın.



VideoNotu

Bir çubuk grafik çizin

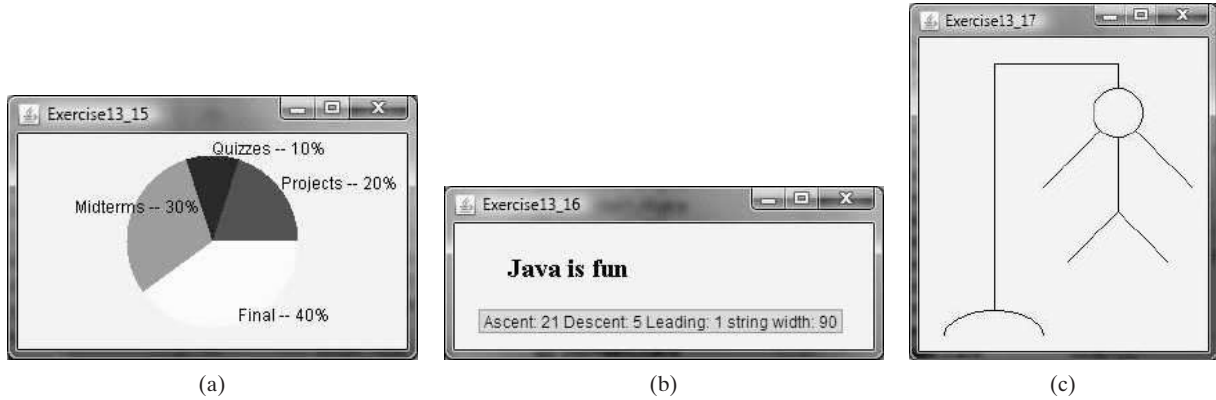


FIGURE 13.31 (a) Alıştırma 13.15, projelerin, quizlerin, ara sınavların ve final sınavının genel not içindeki yüzdelere göre bir pasta grafiği kullanır. (b) Alıştırma 13.16, bir araç ipucu metninde yazı tipi özelliklerini gösterir. (c) Alıştırma 13.17, adam asmaya oyu nu için bir taslak çizer.

13.18 (*StillClock* sınıfını kullanın) İki saat gösteren bir program yazın. İlk saatin saat, dakika ve saniye değerleri 4,20,45 ve ikinci saatin değerleri 22,46,15 olarak ayarlanmıştır, Şekil 13.32a'da gösterildiği gibi.

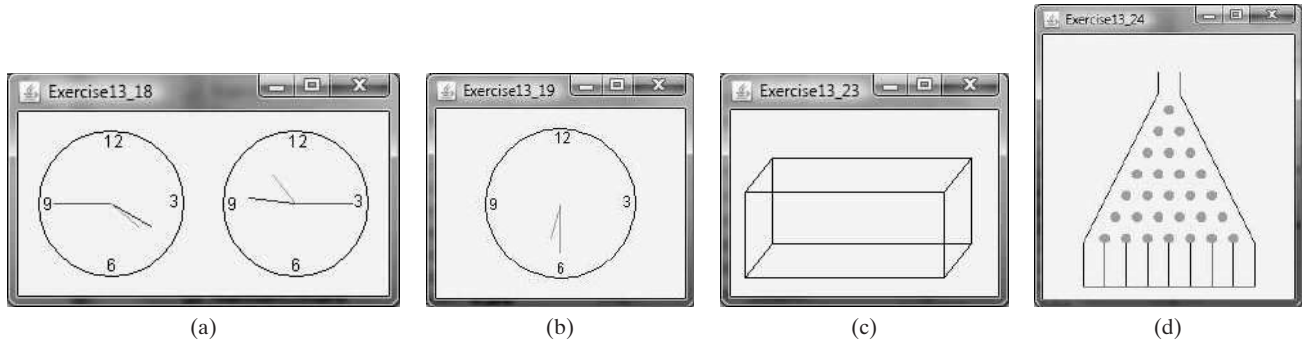


FIGURE 13.32 (a) Alıştırma 13.18, iki saat gösterir. (b) Alıştırma 13.19, rastgele saat ve dakika değerleriyle bir saat gösterir. (c) Alıştırma 13.23, bir dikdörtgen prizma gösterir. (d) Alıştırma 13.24, bir fasulye makinesini simüle eder.

***13.19** (*Rastgele zaman*) *StillClock* sınıfını üç yeni Boolean özelliği ile değiştirin—*saatİşaretiGörünür*, *dakikaİşaretiGörünür* ve *saniyeİşaretiGörünür*—ve bunların ilgili erişimci ve değiştirici yöntemlerini ekleyin. Bir işareti görünür veya görünmez yapmak için *set* yöntemlerini kullanabilirsiniz. Sadece saat ve dakika işaretlerini gösteren bir test programı yazın. Saat ve dakika değerleri rastgele üretilir. Saat 0 ile 11 arasında ve dakika ya 0 ya da 30'dur, Şekil 13.32b'de gösterildiği gibi.

****13.20** (*Detaylı bir saat çiz*) Bölüm 13.9'daki *StillClock* sınıfını, saat ve dakikalar üzerinde daha fazla detayla saat çizecek şekilde değiştirin, Şekil 13.1b'de gösterildiği gibi.

****13.21** (*Görüntülerle bir üç taş tahtası göster*) Programlama Alıştırması 12.7'yi, bir *JPanel* içinde bir görüntü göstermek şeklinde yeniden yazın, bunun yerine bir *JLabel* içinde bir görüntü simgesi göstermeyin.

***13.22** (*Bir DUR işareti göster*) Şekil 13.1c'de gösterildiği gibi bir DUR işareti gösteren bir program yazın. Altıgen kırmızı ve işaret beyazdır. (İpucu: Liste 13.5'e, *DrawPolygon.java* a'ya ve Liste 13.6'ya, *TestCenterMessage.java* ya bakın.)

- 13.23** (Bir dikdörtgen prizma göster) Şekil 13.32c'de gösterildiği gibi bir dikdörtgen prizma gösteren bir program yazın. Küp, çerçeve büyüdükçe veya küçüldükçe büyüyüp küçülmelidir.
- **13.24** (Oyun: fasulye makinesi) Programlama Egzersizi 6.21'de tanıtilan bir fasulye makinesi sergileyen bir program yazın. Fasulye makinesi, Şekil 13.32d'de gösterildiği gibi, büyütülebilir bir panelde ortalanmış olmalıdır.
- **13.25** (Geometri: n kenarlı düzgün çokgeni göster) **JPanel** sınıfının bir alt sınıfını tanımlayın, **RegularPolygonPanel** adında, n kenarlı düzgün bir çokgeni boyamak için. Sınıf, çokgenin kenar sayısını belirten **numberOfSides** adında bir özellik içerir. Çokgen panelde ortalanmıştır. Çokgenin boyutu panelin boyuna orantılıdır. **RegularPolygonPanel** 'den bir beşgen, altıgen, yedigen, sekizgen, dokuzgen ve ongen oluşturun ve bunları bir çerçevede gösterin, Şekil 13.33a'da gösterildiği gibi.

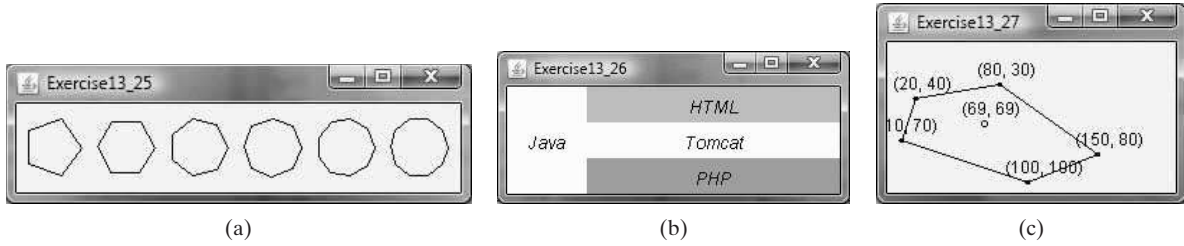


FIGURE 13.33(a) Alıştırma 13.25, birkaç n kenarlı çokgeni gösterir. (b) Alıştırma 13.26, dört dizeyi göstermek için **MessagePanel** kullanır. (c) Çokgen ve stratejik noktası gösterilir.

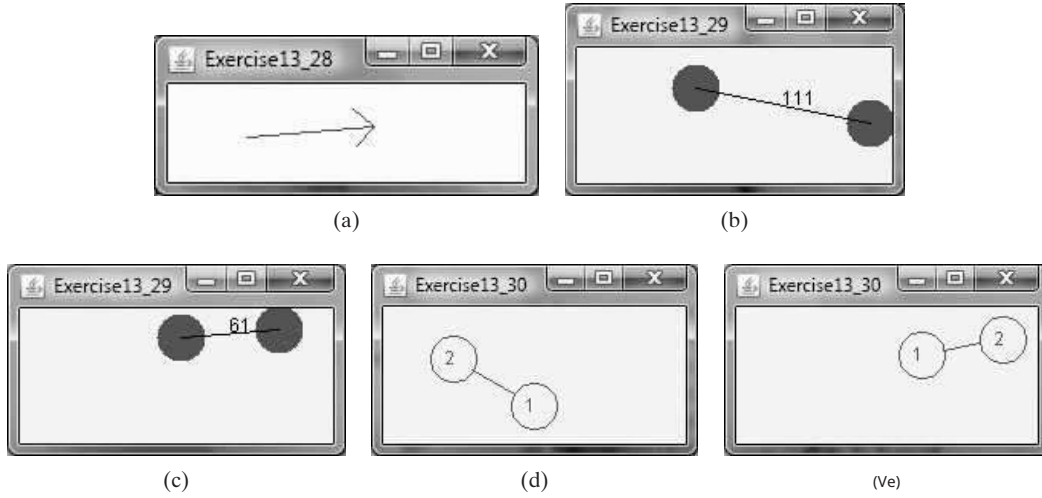
Bölümler 13.8–13.11

- 13.26** (MessagePanel sınıfını kullanın) Şekil 13.33b'de gösterildiği gibi, dört mesajı görüntüleyen bir program yazın.
- **13.27** (Geometri: bir çokgenin stratejik noktası) Bir çokgenin stratejik noktası, tüm köşelere en kısa toplam mesafeye sahip olan çokgenin içindeki bir noktadır. Şekil 13.33c'de gösterildiği gibi, stratejik noktayı bulan ve görüntüleyen bir program yazın. Programınız, çokgenin köşe koordinatlarını saat yönünde komut satırından aşağıdaki gibi geçmelidir:
- ```
java Alıştırması13_27 x1 y1 x2 y2 x3 y3 . . .
```
- Program, çokgeni ve stratejik noktasını çerçevede gösterir. (İpucu: Stratejik noktayı bulmak için n, çokgenin içindeki her piksel noktasını kontrol edin ve bunun bir stratejik nokta olup olmadığını görün. Bir noktanın çokgenin içinde olup olmadığını kontrol etmek için **contains** yöntemi ni kullanın.)
- \*\*13.28** (Ok çizgisi çiz) Aşağıdaki yöntem başlığını kullanarak bir başlangıç noktasından bir bitiş noktasına ok çizgisi çizen statik bir yöntem yazın:

```
public static void drawArrowLine(int tx1, int ty1, int tx2, int ty2, Graphics g)
```

Şekil 13.34a'da gösterildiği gibi rastgele bir ok çizgisi çizen bir test programı yazın. Çerçeveyi her yeniden boyutlandırdığınızda, yeni bir ok çizgisi çizilir.

- \*13.29** (İki daire ve aralarındaki mesafe) Yarıçapı 15 piksel olan iki dolu daire çizen bir program yazın, rastgele konumlarda merkezlenmiş ve iki daireyi birleştiren bir çizgi ile. İki merkez arasındaki mesafe, Şekil 13.34b-c'de gösterildiği gibi çizgi üzerinde görünülür. Çerçeveyi her yeniden boyutlandırdığınızda, daireler yeni rastgele konumlar da yeniden görüntülenir.



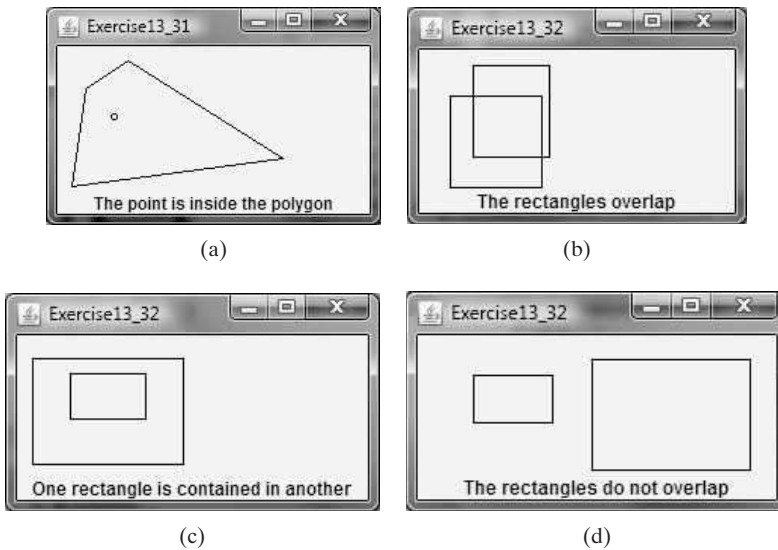
ŞEKİL 13.34(a) Program bir ok çizgisi gösterir. (b-c) Exercise13.29, iki dolu dairenin merkezlerini bağlar. (d-e) Exercise13.30, iki daireyi çevrelerinden bağlar.

**\*13.30**(İki daireyi bağla) Yarıçapı 15 piksel olan iki dolu daire çizen bir program yazın, rastgele konumlarda merkezlenmiş, iki daireyi bağlayan bir çizgi ile. Çizgi, dairelerin içine girmemelidir, Şekil 13.34d-e'de gösterildiği gibi. Çerçeveyi yeniden boyutlandırdığınızda, daireler yeni rastgele konumlarda yeniden görüntülenir.

**\*13.31**(Geometri: Bir çokgenin içinde mi?) Aşağıdaki gibi komut satırından beş noktanın koordinatlarını geçiren bir program yazın:

```
java Alıştırması13_31 x1 y1 x2 y2 x3 y3 x4 y4 x5 y5
```

İlk dört nokta bir çokgen oluşturur ve program, beşinci noktanın çokgenin içinde olup olmadığını gösteren bir etiket içinde bir mesaj ve çokgeni bir panelde görüntüler, Şekil 13.35a'da gösterildiği gibi.



ŞEKİL 13.35(a) Çokgen ve bir nokta görüntülenir. (b-d) İki dikdörtgen görüntülenir.

**\*13.32**(*Geometri: iki dikdörtgen*) Aşağıdaki gibi komut satırından iki dikdörtgenin merkez koordinatlarını, genişliğini ve yüksekliğini geçiren bir program yazın:

java Alıştırması13\_32 x1 y1 w1 h1 x2 y2 w2 h2

Program, dikdörtgenleri bir panelde gösterir ve iki dikdörtgenin örtüşüp örtüşmediğini, birinin diğerinin içinde olup olmadığını veya örtüşmediklerini belirten bir mesaj gösterir; bu, Şekil 13.35b-d'de gösterilmiştir. Mesajı bir etiket içinde gösterin. İki dikdörtgen arasındaki ilişkiyi kontrol etmek için Programlama Egzersizi 10.13'e bakın.

# EXCEPTION HANDLING VE TEXT I/O

## Hedefler

- İstisnalar ve istisna işleme hakkında genel bir bakış elde etmek için (§14.2).
- İstisna işlemenin avantajlarını keşfetmek için (§14.2).
- İstisna türlerini ayırt etmek için: **Hata** (fatal) ile **İstisna** (nonfatal) ve kontrol edilen ile kontrol edilmeyen (§14.3).
- Bir yöntem başlığında istisnaları bildirmek için (§14.4.1).
- Bir yöntemde istisnaları fırlatmak için (§14.4.2).
- İstisnaları işlemek için bir **try-catch** bloğu yazmak için (§14.4.3).
- Bir istisnanın nasıl yayıldığını açıklamak için (§14.4.3).
- Bir istisna nesnesinden bilgi almak için (§14.4.4).
- İstisna işleme ile uygulamalar geliştirmek (§14.4.5).
- Bir **try-catch** bloğunda **finally** ifadesini kullanmak (§14.5).
- İstisnaları yalnızca beklenmeyen hatalar için kullanmak (§14.6).
- Bir **catch** bloğunda istisnaları yeniden fırlatmak (§14.7).
- Zincirlenmiş istisnalar oluşturmak (§14.8).
- Özel istisna sınıfları tanımlamak (§14.9).
- Dosya/dizin özelliklerini keşfetmek, dosyaları/dizinleri silmek ve yeniden adlandırmak, ve **File** sınıfını kullanarak dizinler oluşturmak (§14.10).
- **PrintWriter** sınıfını kullanarak bir dosyaya veri yazmak (§14.11.1).
- **Scanner** sınıfını kullanarak bir dosyadan veri okumak (§14.11.2).
- Verilerin nasıl okunduğunu anlamak için bir **Scanner** kullanmak (§14.11.3).
- Bir dosyadaki metni değiştiren bir program geliştirmek (§14.11.4).
- Bir dosyayı dosya iletişim kutusu kullanarak açmak (§14.12).
- Web'den veri okumak (§14.13).



## 14.1 Giriş



İstisna yönetimi *bir programın olağanüstü durumlarla başa çıkmasını sağlar ve normal yürütmesine devam etmesine olanak tanır.*

*Çalışma zamanı hataları*, JVM bir işlemin gerçekleştirilmesininimkansız olduğunu tespit ettiğinde bir program çalışırken meydana gelir. Örneğin, bir diziyi sınırların dışında bir indeks kullanarak erişmeye çalışırsanız, `ArrayIndexOutOfBoundsException` ile bir çalışma zamanı hatası alırsınız. Eğer programınız bir tamsayı beklerken bir `double` değeri girerseniz, bir `InputMismatchException` ile bir çalışma zamanı hatası alırsınız.

istisna

Java'da, çalışma zamanı hataları istisna olarak fırlatılır. Bir *istisna*, normal bir şekilde yürütmenin devam etmesini engelleyen bir hata veya durumu temsil eden bir nesnedir. Eğer istisna işlenmezse, program anormal bir şekilde sonlanır. Programın çalışmaya devam etmesi veya düzgün bir şekilde sonlanması için istisnayı nasıl işleyebilirsiniz? Bu bölüm bu konuyu ve metin girişi ile ilişkisini tanıtmaktadır.

## 14.2 İstisna Yönetimi Genel Görünümü



*İstisnalar bir yöntemden fırlatılır. Yöntemin çağırıcısı istisnayı yakalayabilir ve işleyebilir.*



VideoNotu  
İstisna-yönetimi  
avantajları

İstisna nesnesinin nasıl oluşturulup fırlatıldığını göstermek için, Listing 14.1'deki örnekle başlayalım; bu örnek iki tamsayı okur ve bunların

bölümünü gösterir.

### LİSTELENME 14.1 Quotient.java

```
1 import java.util.Scanner;
2
3 genel classtQuotient {
4 public static void main(String[] args) {
5 Tarayıcı girişi = tnewScanner(System.in);
6
7 // Kullanıcıdan iki tamsayı girmesini iste
8 System.out.print("İki tamsayı girin: ");
9 int number1 = input.nextInt();
10 int number2 = input.nextInt();
11
12 System.out.println(number1 + " / " + number2 + " = " +
13 (sayı1 / sayı2));
14 }
15 }
```

iki tam sayı okur

tam sayı bölmesi



İki tam sayı girin: 5 2 Enter  
5 / 2 = 2



İki tam sayı girin: 3 0 Enter  
Thread "main" içinde istisna java.lang.ArithmeticException: / sıfıra bölme  
at Quotient.main(Quotient.java:11)

Eğer ikinci sayı için 0 girdiyseniz, bir çalışma zamanı hatası meydana gelecektir, çünkü bir tam sayıyı 0 ile bölemezsiniz. (Unutmayın ki, bir ondalık sayı 0 ile bölündüğünde bir *istisna oluşurmaz*.) Bu hatayı düzeltmenin basit bir yolu, ikinci sayıyı test etmek için bir `if` ifadesi eklemektir, Listing 14.2'de gösterildiği gibi.

## LISTING 14.2tQuotientWithIf.java

```

1 import java.util.Scanner;
23
 genel classtQuotientWithIf {
4 public static voidtmain(String[] args) {
5 Tarayıcı girişi = tnewtScanner(System.in);
67
 // Kullanıcıdan iki tamsayı girmesini iste
8 System.out.print("İki tamsayı girin: ");
9 inttnumber1 = input.nextInt(); iki tam sayı okur
10 inttnumber2 = input.nextInt();
1112
 ift(sayı2 != t0t) 2 numaralı test
13 System.out.println(sayı1 + t" / "+ sayı2 n14 + t" is "
 + (sayı1 / sayı2));
15 başka
16 System.out.println("Bölgenin sıfır olamaz ");
17 }
18 }

```

İki tam sayı girin: 5 0

Bölgenin sıfır olamaz



İstisna yönetimi kavramını göstermek için, Liste 14.2'yi bir yöntem kullanarak bir

bölüm hesaplayacak şekilde yeniden yazabiliriz; bu, Liste 14.3'te gösterilmiştir.

## LISTE 14.3QuotientWithMethod.java

```

1 import java.util.Scanner;
23
 genel classtQuotientWithMethod {
45 public static inttquotient(inttnumber1,inttnumber2) { quotient metodu
 ift(sayı2 == t0t) {
6 System.out.println("Bölgenin sıfır olamaz");
7 System.exit(t1t);
8 }
9
10 dönüş numarası1 / sayı2;
11 }
1213
 public static voidtmain(String[] args) {n14 Tarayıcı
 girişi = tnewtScanner(System.in);
1516
 // Kullanıcıdan iki tamsayı girmesini iste
17 System.out.print("İki tamsayı girin: ");
18 inttnumber1 = input.nextInt(); iki tam sayı okur
19 inttnumber2 = input.nextInt();
2021
 inttresult = bölüm(sayı1, sayı2); metodu çağır
22 System.out.println(number1 + " / "+ number2 + " şudur "
23 + sonuç;n24 }
25 }

```

## 520 Bölüm 14 İstisna Yönetimi ve Metin G/Ç



İki tam sayı girin: 5 3 Enter  
5 / 3 1'dir



İki tam sayı girin: 5 0 Enter  
Bölgenin sıfır olamaz

Metod `quotient` (satırlar 4–11) iki tam sayının bölümünü döndürür. Eğer `number2` ise, dönüş değeri veremez, bu yüzden program 7. satırda sonlandırılır. Bu açıkça bir problemdir. Metodun programı sonlandırmasına izin vermemelisiniz— çağırıcı programı sonlandırıp sonlandırmayacağına karar vermelidir.

Bir metod, çağırıcısına bir istisna meydana geldiğini nasıl bildirebilir? Java, bir metodun çağırıcı tarafından yakalanıp işlenebilecek bir istisna fırlatmasına olanak tanır. Liste 14.3, Liste 14.4'te gösterildiği gibi yeniden yazılabilir.

### Listing 14.4 `QuotientWithException.java`

oran metodu

istisna fırlat

iki tam sayı okur

deneme bloğu  
metodu çağır

yakala bloğu

```
1 import java.util.Scanner;
2
3 genel classt QuotientWithException {
4 public static int oran(int number1, int number2) {
5 if (sayi2 == 0)
6 throw new Aritmetikİstisna("Bölgenin sıfır olamaz");
7
8 dönüşnumarası / sayi2; n9 }
9
10
11 public static void main(String[] args) {
12 Tarayıcı girişi = new Scanner(System.in);
13
14 // Kullanıcıdan iki tam sayı girmesini iste
15 System.out.print("İki tam sayı girin: ");
16 int number1 = input.nextInt();
17 int number2 = input.nextInt();
18
19 denemek
20 int result = bölüm(sayı1, sayı2);
21 System.out.println(number1 + " / " + number2 + " dir "
22 + sonuç);
23 }
24 catch (Aritmetikİstisna ex) {
25 System.out.println("İstisna: bir tamsayı "+"
26 "sıfıra bölünemez ");
27 }
28
29 System.out.println("İşlem devam ediyor ...");
30 }
31 }
```



İki tam sayı girin: 5 3 Enter  
5 / 3 1'dir  
İşlem devam ediyor ...



İki tam sayı girin: 5 0

İstisna: bir tam sayı sıfıra bölünemez

İşlem devam ediyor ...



Eğer **number20** ise, metod bir istisna fırlatır (satır 6) çalıştırarak

```
throw new ArithmeticException("Payda sıfır olamaz");
```

atmak ifadesi

Atılan değer, bu durumda `new ArithmeticException("Bölgenin sıfır olamaz")`, *istisna* olarak adlandırılır. **throw** ifadesinin yürütülmesi *istisna atma* olarak adlandırılır. İstisna, bir istisna sınıfından oluşturulan bir nesnedir. Bu durumda, istisna sınıfı `java.lang.ArithmeticException`'dir. `ArithmeticException(str)` yapıcısı bir istisna nesnesi oluşturmak için çağrılır, burada **str** istisnayı tanımlayan bir mesajdır.

istisna  
istisna atma

Bir istisna atıldığında, normal yürütme akışı kesilir. İsimden de anlaşılacağı gibi, "bir istisna atmak", istisnayı bir yerden başka bir yere geçirmektir. Metodu çağırma ifadesi bir **try** bloğunda ve bir **catch** bloğunda yer alır. **try** bloğu (19-23. satırlar) normal koşullarda yürütülen kodu içerir. İstisna **catch** bloğu tarafında yakalanır. **catch** bloğundaki kod, *istisnayı ele almak* için yürütülür.

istisnayı ele al

Daha sonra, **catch** bloğundan sonraki ifade (29. satır) yürütülür.

**throw** ifadesi, bir metod çağrısına benzer, ancak bir metodu çağırmak yerine bir **catch** bloğunu çağırır. Bu anlamda, bir **catch** bloğu, atılan değerın türüyle eşleşen bir parametreye sahip bir metod tanımına benzer. Ancak bir metodun aksine, **catch** bloğu yürütüldükten sonra program kontrolü **throw** ifadesine geri dönmeyi; bunun yerine, **catch** bloğundan sonraki ifadeyi yürütür.

Tanımlayıcı **ex**, **catch**-blok başlığında

```
catch(ArithmeticException ex)
```

bir yöntemdeki bir parametre gibi davranır. Bu nedenle, bu parametre **catch**-blok parametresi olarak adlandırılır. **ex**'yi takip eden tür (örneğin, `ArithmeticException`), **catch** bloğunun hangi tür istisnayı yakalayabileceğini belirtir. İstisna yakalandıktan sonra, **catch** bloğunun gövdesinde bu parametreden atılan değere erişebilirsiniz.

yakala-blok parametresi

Özetle, bir **try-throw-catch** bloğu için bir şablon şu şekilde görünebilir:

denemek

```
Çalıştırılacak kod;
Bir istisna atabilecek bir ifade veya yöntem;
Daha fazla çalıştırılacak kod;
}
catch(tip ex) {
 İstisnayı işlemek için kod;
}
```

Bir istisna, bir **try** bloğunda bir **throw** ifadesi kullanılarak doğrudan atılabilir veya bir istisna atabilecek bir yöntemi çağırarak atılabilir.

Ana yöntem **quotient** (satır 20) çağrılır. Eğer **quotient** yöntemi normal bir şekilde çalışırsa, çağırıcıya bir değer döner. Eğer **quotient** yöntemi bir istisna ile karşılaşarsa, istisnayı çağırıcıya geri fırlatır. Çağırıcının **catch** bloğu istisnayı işler.

Artık istisna işleme kullanmanın *avantajını* görebilirsiniz: Bu, bir yöntemin bir istisna fırlatmasına olanak tanır, böylece çağırıcı istisnayı işleyebilir. Bu yetenek olmadan, çağrılan yöntem kendisi istisnayı ele almak zorundadır veya programı sonlandırmalıdır. Genellikle çağrılan yöntem hata durumunda ne yapacağını bilmez. Bu genellikle kütüphane yöntemleri için geçerlidir. Kütüphane yöntemi hatayı tespit edebilir, ancak yalnızca çağırıcı ne yapılması gerektiğini bilir.

avantaj

bir hata meydana geldiğinde yapılır. İstisna yönetiminin temel faydası, bir hatanın tespiti (çağrılan yöntemde yapılır) hatanın yönetiminden (çağırılan yöntemde yapılır) ayırmaktır.

Pek çok kütüphane yöntemi istisnalar atar. Liste 14.5, bir `InputMismatchException` okuma sırasında bir girdi.

### Liste 14.5 `InputMismatchExceptionDemo.java`

bir Scanner oluştur

try bloğu

yakala bloğu

```

1 import java.util.*;
23
 genel classtInputMismatchExceptionDemo {
4 public static voidtmain(String[] args) {
5 Tarayıcı girişi = tnewtScanner(System.in);
6 booleantcontinueInput = ttrue;
78
 nokta{
10 {
11 System.out.print("Bir tam sayı girin: ");
12 inttnumber = input.nextInt();ntry
13
14 // Sonucu göster
15 System.out.println(
16 "Girilen sayı "+ number);
17 devamGirişi = tfalset;n18 }
19
20 yakala(InputMismatchException ex) {
21 System.out.println("Tekrar deneyin. (" +
22 "Geçersiz giriş: bir tam sayı gereklidir");
23 input.nextLine(); // Girişi atla
24 }
25 } twhilet(continueInput;n25 }
26 }
```



```

Bir tamsayı girin: 3.5 [Enter]
Tekrar deneyin. (Yanlış giriş: bir tamsayı gereklidir)
Bir tamsayı girin: 4 [Enter]
Girilen sayı 4'tür
```

`input.nextInt()` (satır 11) çalıştırıldığında, `InputMismatchException`, girilen giriş bir tamsayı değilse oluşur. Diyelim ki 3.5 girildi. Bir `InputMismatchException` oluşur ve kontrol catch bloğuna aktarılır. catch bloğundaki ifadeler şimdi yürütülür. Satır 22'deki `input.nextLine()` ifadesi mevcut giriş satırını atar, böylece kullanıcı yeni bir giriş satırı girebilir. `continueInput` değişkeni döngüyü kontrol eder. Başlangıç değeri `true`'dur (satır 6) ve geçerli bir giriş alındığında `false`'ya (satır 17) değiştirilir. Geçerli bir giriş alındıktan sonra, girişi devam ettirmeye gerek yoktur.

#### 14.1 İstisna işleme kullanmanın avantajı nedir?

#### 14.2 Aşağıdaki ifadelerden hangisi bir istisna fırlatır?

```

System.out.println(t1t/t0t);Syste
m.out.println(t1.0t/t0t);
```

**14.3** Aşağıdaki kodda problemi belirtin. Kod herhangi bir istisna fırlatıyor mu?

```
uzundeğer = Long.MAX_VALUE + 1;
System.out.println(değer);
```

**14.4** JVM bir istisna oluştuğunda ne yapar? Bir istisnayı nasıl yakalarsınız?

**14.5** Aşağıdaki kodun çıktısı nedir?

```
genel sınıfTest {
 public static voidtmain(String[] args) {ntryt{

 intdeğer = 30;
 eğer(değer < 40)
 throw newException("değer çok küçük");
 }
 catch(Exception ex) {System
 .out.println(e.getMessage());n}

 System.out.println("catch bloğundan sonra devam et");
}
}
```

Satırın çıktısı ne olurdu eğer

```
intdeğer = 30;
```

şu şekilde değiştirilseydi

```
inttdeğeri = t50t;
```

**14.6** Aşağıdaki kodun çıktısını gösterin.

```
genel sınıfTest {
 public static voidtmain(String[] args) {nfort(tintti =
 t0t; i < t2t; i++) {
 System.out.print(i + t" "t);ntryt{

 System.out.println(t1t/t0t);n}

 catch(İstisna örneğin) {n}

 }
}
```

(a)

```
genel sınıfTest {
 public static voidtmain(String[] args) {ntryt{

 fort(tintti = t0t; i < t2t; i++) {nSystem.
 out.print(i + t" "t);nSystem.out.pr
 intln(t1t/t0t);n}

 }
 catch(İstisna örneğin) {n}

}
```

(b)

## 14.3 İstisna Türleri

*İstisnalar nesnelerdir ve nesneler sınıflar kullanılarak tanımlanır. İstisnaları n kök sınıfı **java.lang.Throwable**'dir.*



Önceki bölüm **ArithmeticException** ve **InputMismatch-Exception** sınıflarını kullandı. Kullanab ileceğiniz başka istisna türleri var mı? Kendi istisna sınıflarınızı tanımlayabilir misiniz? Evet. Java API'sinde birçok önceden tanımlanmış istisna sınıfı vardır. Şekil 14.1 bunlardan bazılarını göstermektedir ve Bölüm 14.9'da kendi istisna sınıflarınızı nasıl tanımlayacağınızı öğreneceksiniz

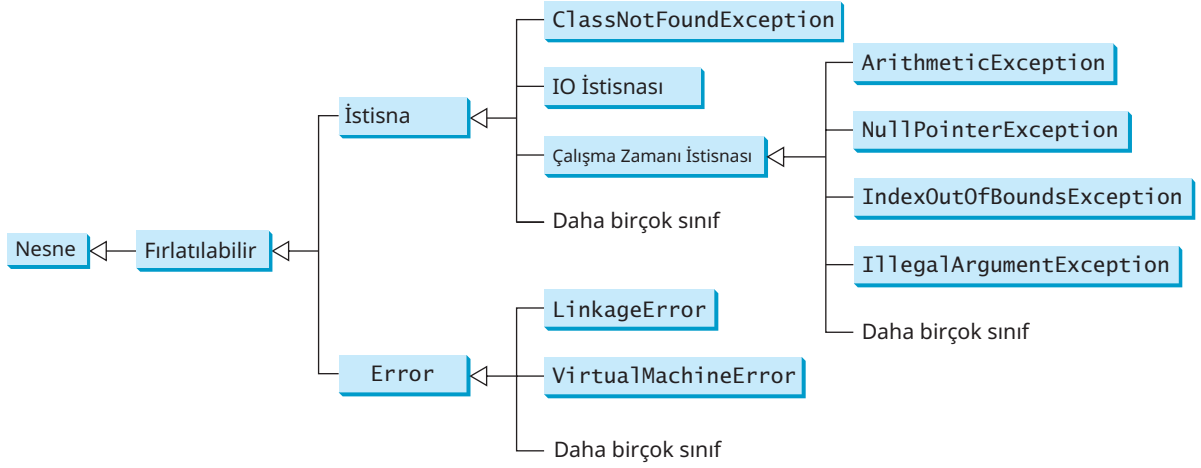


Fig 14.1 Atılan istisnalar, bu diyagramda gösterilen sınıfların örnekleridir veya bu sınıflardan birinin alt sınıflarının örnekleridir.



### Not

Error, Exception ve RuntimeException sınıf adları biraz kafa karıştırıcıdır. Bu üç sınıf da istisnadır ve hataların hepsi çalışma zamanında meydana gelir.

Throwable sınıfı istisna sınıflarının köküdür. Tüm Java istisna sınıfları doğrudan veya dolaylı olarak Throwable'dan türetilir. Kendi istisna sınıflarınızı Exception veya Exception'ın bir alt sınıfını genişleterek oluşturabilirsiniz.

İstisna sınıfları üç ana türe ayrılabilir: sistem hataları, istisnalar ve çalışma zamanı istisnaları.

sistem hatası

- Sistem hataları JVM tarafından fırlatılır ve **Error** sınıfında temsil edilir. **Error** sınıfı, iç sistem hatalarını tanımlar, ancak bu tür hatalar nadiren meydana gelir. Eğer bir hata meydana gelirse, kullanıcıyı bilgilendirmek ve programı düzgün bir şekilde sonlandırmaya çalışmaktan başka yapabileceğiniz pek bir şey yoktur. **Error** alt sınıflarının örnekleri Tablo 14.1'de listelenmiştir.

TABLO 14.1 Hata'nın Alt Sınıflarına Örnekler

| Sınıf               | İstisna Nedenleri                                                                                                                |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Bağlantı Hatası     | Bir sınıf, başka bir sınıfa bağımlıdır, ancak ikinci sınıf, birincisini derlenmesinden sonra uyumsuz bir şekilde değiştirmiştir. |
| Sanal Makine Hatası | JVM bozulmuş veya çalışmaya devam etmek için ihtiyaç duyduğu kaynaklar tükenmiştir.                                              |

istisna

- **İstisnalar** **Exception** sınıfında temsil edilir, bu sınıf programınız ve dış koşullar tarafından neden oluşan hataları tanımlar. Bu hatalar programınız tarafından yakalanabilir ve işlenebilir. **Exception** alt sınıflarına örnekler Tablo 14.2'de listelenmiştir.

TABLO 14.2 İstisna Alt Sınıflarına Örnekler

| Sınıf                         | İstisna Nedenleri                                                                                                                                                                                                                                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ClassNotFoundException</b> | Mevcut olmayan bir sınıfı kullanmaya çalışmak. Bu istisna, örneğin, <b>java</b> komutunu kullanarak mevcut olmayan bir sınıfı çalıştırmaya çalıştığınızda veya programınızın, yalnızca iki tanesinin bulunabildiği üç sınıf dosyasından oluştuğu durumlarda meydana gelecektir.                           |
| <b>IO İstisnası</b>           | Geçersiz girdi, bir dosyanın sonunu aşarak okuma ve mevcut olmayan bir dosyayı açma gibi girdi/çıkartı işlemleriyle ilgilidir. <b>IOException</b> alt sınıflarına örnekler <b>InterruptedException</b> , <b>EOFException</b> (EOF, Dosyanın Sonu'nun kısaltmasıdır) ve <b>FileNotFoundException</b> 'dir. |

- Çalışma zamanı istisnalar **RuntimeException** sınıfında temsil edilir ve yanlış tür dönüşümü, sınır dışı bir diziye erişim ve sayısal hatalar gibi programlama hatalarını tanımlar. Çalışma zamanı istisnaları genellikle JVM tarafından fırlatılır. Alt sınıfların örnekleri Tablo 14.3'te listelenmiştir.

çalışma zamanı istisnası

TABLO 14.3 RuntimeException Alt Sınıflarına Ait Örnekler

| Sınıf                            | İstisna Nedenleri                                                                                                    |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>AritmetikException</b>        | Bir tam sayıyı sıfıra bölmek. Not: kayan nokta aritmetiği istisna fırlatmaz (bkz. Ek E, Özel Kayan Nokta Değerleri). |
| <b>NullPointerException</b>      | Bir nesneye <b>null</b> referans değişkeni aracılığıyla erişim denemesi.                                             |
| <b>IndexOutOfBoundsException</b> | Diziye yapılan indeks aralığın dışındadır.                                                                           |
| <b>YasadışıTartışmaİstisna</b>   | Bir metoda yasadışı veya uygunsuz bir argüman geçirilmiştir.                                                         |

**RuntimeException**, **Hata**, ve alt sınıfları *kontROLSÜZ istisnalar* olarak bilinir. Tüm diğer istisnalar *kontrol edilen istisnalar* olarak bilinir, bu da derleyicinin programcıyı bunları **try-catch** bloğunda kontrol etmeye ve işlemeye zorladığı anlamına gelir veya yöntemin başlığında belirtmesi gerektiği anlamına gelir. Bir istisnanın yöntem başlığında belirtilmesi Bölüm 14.4'te ele alınacaktır.

denetimsiz istisna  
denetimli istisna

Çoğu durumda, kontROLSÜZ istisnalar, kurtarılamaz programlama mantığı hatalarını yansıtır. Örneğin, bir **NullPointerException**, bir nesneye bir referans değişkeni aracılığıyla erişmeye çalıştığınızda ve ona bir nesne atanmadıysa fırlatılır; bir **IndexOutOfBoundsException**, bir dizideki bir elemana dizinin sınırları dışında eriştiğinizde fırlatılır. Bunlar, programda düzeltilmesi gereken mantık hatalarıdır. KontROLSÜZ istisnalar, bir programda herhangi bir yerde meydana gelebilir. **try-catch** bloklarının aşırı kullanımını önlemek için, Java kontROLSÜZ istisnaları yakalamak veya bildirmek için kod yazmanızı zorunlu kılmaz.

## 14.7 Java **Throwable** sınıfını, alt sınıflarını ve istisna türlerini tanımlayın.

14.8 Aşağıdaki programlar hangi **RuntimeException** hatasını verecek, eğer varsa?

Nokta Kontrol Et

MyProgrammingLab™

```
genel sınıfTest {
 public static void main(String[] args) {System.out.
 println(t1t/t0t);
 }
```

(a)

```
genel sınıfTest {
 public static void main(String[] args) {int[] list = t
 new int[t5t];
 System.out.println(liste[t5t]);n}
 }
```

(b)

```
genel sınıfTest {
 public static voidtmain(String[] args) {nString s = t
 "abc"t;
 System.out.println(s.charAt(t3t));n}
 }
```

(c)

```
genel sınıfTest {
 public static voidtmain(String[] args) {nObject o =
 tnewtObject();
 Dize d = (Dize)o;n}
 }
```

(d)

```
genel sınıfTest {
 public static voidtmain(String[] args) {nObject o =
 tnnullt;
 System.out.println(o.toString());n}
 }
```

(Ve)

```
genel sınıfTest {
 public static voidtmain(String[] args) {nSystem.ou
 t.println(t1.0t/t0t);
 }
```

(f)

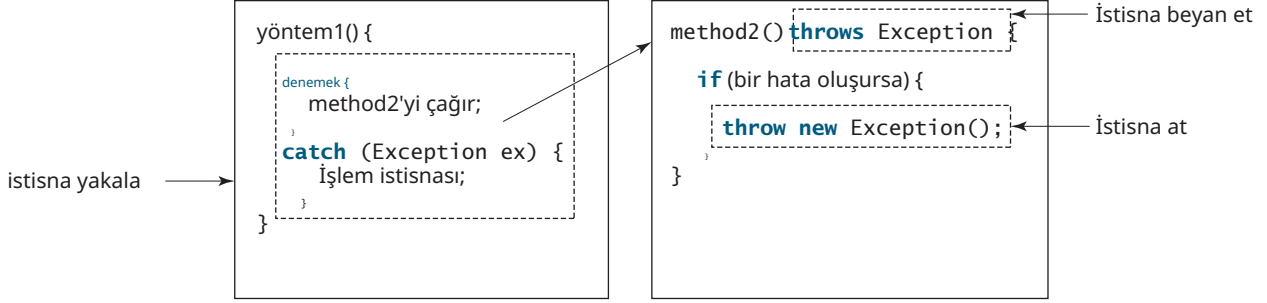


### 14.4 İstisna Yönetimi Hakkında Daha Fazla

*Bir istisna için bir işleyici, mevcut yöntemden başlayarak bir yöntem çağrılarının zinciri boyunca istisnayı geriye doğru yayarak bulunur.*

Önceki bölümler, istisna işleme hakkında bir genel bakış sağladı ve birkaç önceden tanımlanmış istisna türünü tanıttı. Bu bölüm, istisna işlemenin derinlemesine bir tartışmasını sunmaktadır.

Java'nın istisna işleme modeli, üç işlem üzerine kuruludur: *bir istisna beyan etme*, *bir istisna atma* ve *bir istisna yakalama*, Şekil 14.2'de gösterildiği gibi.



**FIGİN 14.2** Java'da istisna işleme, istisnaların bildirilmesi, istisnaların fırlatılması ve istisnaların yakalanması ve işlenmesini içerir.

#### 14.4.1 İstisnaların Bildirilmesi

Java'da, şu anda yürütülen ifade bir metoda aittir. Java yorumlayıcısı **main** metodunu çağırarak bir programı yürütmeye başlar. Her metod, atabileceği kontrol edilen istisnaların türlerini belirtmelidir. Bu, *istisnaları beyan etme* olarak bilinir. Sistem hataları ve çalışma zamanı hataları herhangi bir kodda meydana gelebileceğinden, Java'nın beyan etmenizi gerektirmediği **Hata** ve **RuntimeException** (kontROLSÜZ istisnalar) yöntemde açıkça belirtilmelidir. Ancak, yöntem tarafından fırlatılan diğer tüm istisnalar, yöntemin başlığında açıkça belirtilmelidir ki yöntemin çağırıcısı istisna hakkında bilgilendirilsin.

Bir yöntemde bir istisna bildirmek için, yöntem başlığında **throws** anahtar kelimesini kullanın, bu örnekte olduğu gibi:

```
public void myMethod() throws IOException
```

**throws** anahtar kelimesi, **myMethod**'in bir **IOException** fırlatabileceğini belirtir. Eğer yöntem birden fazla istisna fırlatabiliyorsa, fırlatılan istisnaların listesini, virgüllerle ayırarak, **throws**'tan sonra ekleyin:

```
genel void myMethod()
 İstisna, İstisna, ..., İstisna atar
```



#### Not

Eğer bir yöntem üst sınıfta istisnaları bildirmiyorsa, onu alt sınıfta istisnaları bildirmek için geçersiz kılamazsınız.

#### 14.4.2 İstisnaları Fırlatma

Bir hatayı tespit eden bir program, uygun bir istisna türünün bir örneğini oluşturabilir ve onu fırlatabilir. Bu, *istisna fırlatma* olarak bilinir. İşte bir örnek: Programın bir argümanın yöntemine geçildiğinde yöntem sözleşmesini ihlal ettiğini tespit ettiğini varsayalım (örneğin, argüman

istisna beyan et

istisna at

negatif olmamalıdır, ancak negatif bir argüman geçilmiştir); program bir örneğini oluşturabilir `IllegalArgumentException` ve bunu aşağıdaki gibi fırlatabilir:

```
IllegalArgumentException ex =
 new IllegalArgumentException("Hatalı Argüman");
throw ex;
```

Ya da isterseniz, aşağıdakileri kullanabilirsiniz:

```
throw new IllegalArgumentException("Hatalı Argüman");
```



### Not

`IllegalArgumentException`, Java API'sinde bir istisna sınıfıdır. Genel olarak, Java API'sindeki her istisna sınıfının en az iki yapıcı metodu vardır: bir argüman almayan yapıcı, ve istisnayı tanımlayan bir `String` argümanı olan bir yapıcı. Bu argümana *istisna mesajı* denir ve şu şekilde elde edilebilir: `getMessage()`.

istisna mesajı



### Ttıtıp

Bir istisna bildirmek için kullanılan anahtar kelime `throws`, bir istisna fırlatmak için kullanılan anahtar kelime ise `throw` fırlatmak vs t atmak `ow`'dur.

## 14.4.3 İstisnaları Yakalamak

Artık bir istisnayı nasıl bildireceğinizi ve nasıl fırlatacağınızı biliyorsunuz. Bir istisna fırlatıldığında, `try-catch` bloğunda yakalanabilir ve işlenebilir, şu şekilde:

istisna yakalama

```
denemek
statements; // İstisna fırlatabilecek ifadeler
}
catch (Exception1 exVar1) {
 istisna1 için işleyici;
}
catch (Exception2 exVar2) {
 istisna2 için işleyici;
}
...
catch (ExceptionN exVar3) {
 istisnaN için işleyici;
}
```

Eğer `try` bloğu sırasında herhangi bir istisna oluşmazsa, `catch` blokları atlanır.

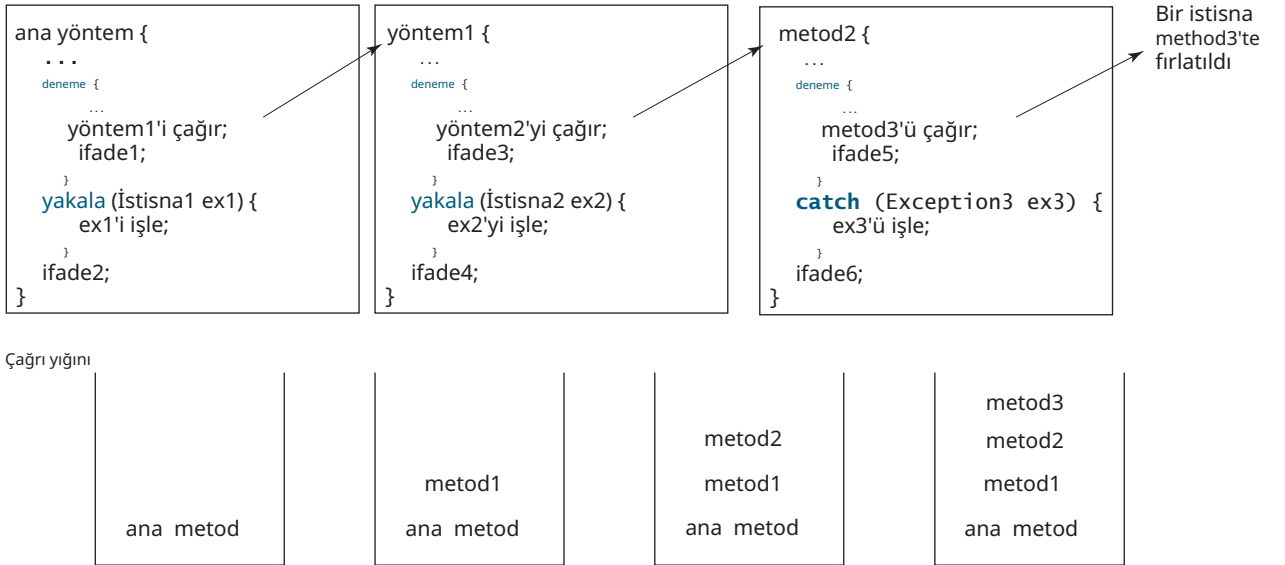
Eğer `try` bloğundaki ifadelerden biri bir istisna fırlatırsa, Java `try` bloğundaki kalan ifadeleri atlar ve istisnayı ele almak için kod bulma sürecine başlar. İstisnayı ele alan koda *exception handler* denir; bu, mevcut metottan başlayarak bir dizi metod çağrısı boyunca istisnayı yayma işlemiyle bulunur. Her `catch` bloğu sırayla, ilkinden sonuncusuna kadar, istisna nesnesinin türünün `catch` bloğunda ki istisna sınıfının bir örneği olup olmadığını görmek için incelenir. Eğer öyleyse, istisna nesnesi tanımlanan değişkene atanır ve `catch` bloğundaki kod çalıştırılır. Eğer bir işleyici bulunamazsa, Java bu metottan çıkar, istisnayı metodu çağırılan metoda iletir ve bir işleyici bulma sürecine devam eder. Eğer çağrılan metodlar zincirinde bir işleyici bulunamazsa, program sonlanır ve konsolda bir hata mesajı yazdırılır. Bir işleyici bulma sürecine istisna yakalama denir.

istisna işleyici  
istisna yayılımı

## 528 Bölüm 14 İstisna Yönetimi ve Metin G/Ç

Diyelim ki `main` metodu `method1`'i çağırıyor, `method1` `method2`'yi çağırıyor, `method2` `method3`'ü çağırıyor ve `method3` bir istisna fırlatıyor, Şekil 14.3'te gösterildiği gibi. Aşağıdaki durumu göz önünde bulundurun:

- Eğer istisna türü `Exception3` ise, `method2`'deki istisna `ex3`'ü işlemek için `catch` bloğu tarafından yakalanır. `statement5` atlanır ve `statement6` yürütülür.
- Eğer istisna türü `Exception2` ise, `method2` iptal edilir, kontrol `method1`'e geri döner ve istisna `method1`'deki istisna `ex2`'yi işlemek için `catch` bloğu tarafından yakalanır. `statement3` atlanır ve `statement4` yürütülür.
- Eğer istisna türü `Exception1` ise, `method1` iptal edilir, kontrol `main` metoduna geri döner ve istisna `main` metodundaki istisna `ex1`'yi işlemek için `catch` bloğu tarafından yakalanır. `statement1` atlanır ve `statement2` yürütülür.
- Eğer istisna türü `method2`, `method1` veya `main`'de yakalanmazsa, program sona erer ve `statement1` ve `statement2` yürütülmez.



**Fiçin 14.3** Eğer bir istisna mevcut yöntemde yakalanmazsa, çağırana iletilir. Süreç, istisna yakalanana veya `ana` yöntemle iletilene kadar tekrarlanır.

yakala bloğu



### Not

Çeşitli istisna sınıfları ortak bir üst sınıftan türetililebilir. Eğer bir `catch` bloğu üst sınıfın istisna nesnelerini yakalarsa, o üst sınıfın alt sınıflarının tüm istisna nesnelerini yakalayabilir.

istisna işleyicilerinin sırası



### Not

İstisnaların `catch` bloklarında belirtilme sırası önemlidir. Eğer bir üst sınıf türü için bir `catch` bloğu, bir alt sınıf türü için bir `catch` bloğundan önce gelirse, bir derleme hatası oluşur. Örneğin, sonraki sayfadaki (a) sıralaması hatalıdır, çünkü `RuntimeException`, `Exception`'in bir alt sınıfıdır. Doğru sıralama (b)'de gösterildiği gibi olmalıdır.



```

denemek{
 ...
}
yakala (İstisna eski) {
 ...
}
yakala() throws RuntimeException ex {
 ...
}

```

(a) Yanlış sıra

```

denemek{
 ...
}
yakala (RuntimeException ex) {
 ...
}
yakala (İstisna eski) {
 ...
}

```

(b) Doğru sıra



### Not

Java, kontrol edilen istisnalarla başa çıkmanızı zorunlu kılar. Eğer bir yöntem kontrol edilen bir istisna (bu, **Error** veya **RuntimeException** dışındaki bir istisna) bildiriyorsa, onu bir **try-catch** bloğunda çağırmalı veya çağıran yöntemde istisnayı fırlatmayı bildirmelisiniz. Örneğin, eğer **p1** yöntemi **p2** yöntemini çağırıyorsa ve **p2** bir kontrol edilen istisna (örneğin, **IOException**) fırlatabilir; aşağıdaki (a) veya (b) şeklinde kod yazmalısınız.

### kontrol edilen

istisnaları yakala veya bildir

```

void tp1() { np
 denemek {
 2();
 }
 yakala(IOException ex) {
 ...
 }
}

```

(a) Hata yakala

```

geçersiz p1() throws IOException {
 p2();
}

```

(b) İstisna fırlat



### Not

Yeni JDK 7 çoklu yakalama özelliğini, aynı işleme koduna sahip istisnalar için kodlamayı basitleştirmek amacıyla kullanabilirsiniz. Sözdizimi:

### JDK 7 çoklu yakalama

```

catch (Exception1 | Exception2 | ... | Exceptionk ex) {
 // İstisnaları işlemek için kod
}

```

Her istisna türü, bir dikey çubuk (|) ile bir sonraki istisnadan ayrılır. Eğer bir istisna yakalanırsa, işleme kodu çalıştırılır.

## 14.4.4 İstisnalardan Bilgi Alma

Bir istisna nesnesi, istisna hakkında değerli bilgiler içerir. İstisna ile ilgili bilgi almak için **java.lang.Throwable** sınıfındaki aşağıdaki örnek yöntemleri kullanabilirsiniz; Şekil 14.4'te gösterildiği gibi. **printStackTrace()** yöntemi yığın izini yazdırır

### metotlar Throwable

| java.lang.Throwable                                                                                               |
|-------------------------------------------------------------------------------------------------------------------|
| +getMessage(): String<br>+toString(): String<br>+printStackTrace(): void<br>+getStackTrace(): StackTraceElement[] |

Returns the message that describes this exception object.  
Returns the concatenation of three strings: (1) the full name of the exception class; (2) " " (a colon and a space); (3) the **getMessage()** method.  
Prints the **Throwable** object and its call stack trace information on the console.  
Returns an array of stack trace elements representing the stack trace pertaining to this exception object.

FIGURE 14.4 **Throwable** tüm istisna nesneleri için kök sınıftır.

konsolda bilgi. `getStackTrace()` metodu, `printStackTrace()` tarafından yazdırılan yığın izleme bilgilerine programatik erişim sağlar.

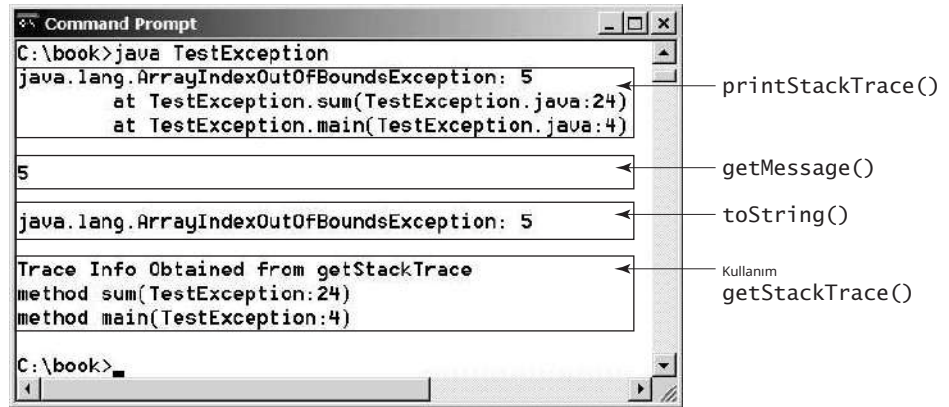
Liste 14.6, `Throwable` içindeki metotları kullanarak istisna bilgilerini görüntüleyen bir örnek vermektedir.

4. satır, dizideki tüm elemanların toplamını döndürmek için `sum` metodunu çağırır.

23. satırda, `ArrayIndexOutOfBoundsException`'i, `IndexOutOfBoundsException`'in bir alt sınıfını tetikleyen bir hata vardır.

Bu istisna, try-catch bloğunda yakalanır.

7, 8 ve 9. satırlar, yığın izini, istisna mesajını ve istisna nesnesini ve mesajını `printStackTrace()`, `getMessage()` ve `toString()` metotları kullanarak görüntüler, bu da Şekil 14.5'te gösterilmiştir. 12. satır, yığın izleme elemanlarını bir diziye alır. Her eleman bir metot çağırısını temsil eder. Her eleman için metodu (14. satır), sınıf adını (15. satır) ve istisna satır numarasını (16. satır) alabilirsiniz.



FİĞİN 14.5 `printStackTrace()`, `getMessage()`, `toString()` ve `getStackTrace()` yöntemlerini kullanarak istisna nesnelerinden bilgi alabilirsiniz.

#### LISTE 14.6 `TestException.java`

```

1 genel classtestException {
2 public static void main(String[] args) {
3 denemek
4 System.out.println(toplam(tnew int[] {t1t,t2t,t3t,t4t,t5t}));
5 }
6 catch(İstisna örneği) {
7 ex.printStackTrace();n8 System.out.p
rintln(t"n"t+ ex.getMessage());n9 System.out.println(t"n"t+ ex.toStr
ing());
10
11 System.out.println("Yığın İzleme Bilgisi getStackTrace'ten Alındı");12 Sta
ckTraceElement[] traceElements = ex.getStackTrace();
13 for(tintti = t0t; i < traceElements.length; i++) {
14 System.out.print("metod " + traceElements[i].getMethodName());
15 System.out.print("(" + traceElements[i].getClassName() + ":");
16 System.out.println(traceElements[i].getLineNumber() + ")");
17 }
18 }n19}

20
21 özel statik inttsum(tint[] listesi) {
22 sonuç = t0t;
23 kale(renk tonu = t0; i <= liste.uzunluk ; i++)

```

```

24 sonuç += liste[i];
25 returnsonuç;
asla {revize e
dilmeli}

```

### 14.4.5 Örnek: İstisnaları Bildirme, Fırlatma ve Yakalama

Bu örnek, 8.9 numaralı Listede, CircleWithPrivateDataFields.java dosyasındaki Circle sınıfındaki **setRadius** yöntemini değiştirerek istisnaları bildirme, fırlatma ve yakalamayı göstermektedir. Yeni **setRadius** yöntemi, yarıçap negatifse bir istisna fırlatır.

14.7 numaralı Liste, **CircleWithException** adında yeni bir daire sınıfı tanımlar; bu, **setRadius(double newRadius)** yönteminin **newRadius** argümanı negatifse bir **IllegalArgumentException** fırlatması dışında, **CircleWithPrivateDataFields** ile aynıdır.

#### LISTE 14.7 CircleWithException.java

```

1 genel classtCircleWithException {
2 /** Dairenin yarıçapı */
3 özel doubleradius;
4
5 /** Oluşturulan nesnelerin sayısı */
6 özel statik intnumberOfObjects = 0;
7
8 /** Yarıçapı 1 olan bir daire oluştur */
9 kamusalCircleWithException() {
10 bu(1.0);
11 }
12
13 /** Belirtilen yarıçapa sahip bir daire oluştur */
14 kamusalCircleWithException(doublenewRadius) {
15 setRadius(yeniRadyus);
16 numberOfObjects++;n17 }
17
18
19 /** Yarıçapı döndür */
20 genel doubletgetRadius() {
21 geri dönmek
22 }
23
24 /** Yeni bir yarıçap ayarla */
25 genel voidtsetRadius(tdoubletnewRadius)
26 throwsYasa DışıTartışmaİstisna {
27 ıft(newRadius >= t0t)
28 yarıçap = yeniYarıçap;
29 başka
30 throw new IllegalArgumentException(
31 "Yarıçap negatif olamaz");
32 }
33
34 /** numberOfObjects değerini döndür */
35 genel statik inttgetNumberOfObjects() {
36 returnnumberOfObjects;
37 }
38
39 /** Bu dairenin alanını döndür */
40 public doublealanBul() {
41 returnradyus * radyus * 3.14159;
42 }n43}

```

istisna beyan et

istisna at

Yeni **Daire** sınıfını kullanan bir test programı Liste 14.8'de verilmiştir.

### LISTE 14.8 TestCircleWithException.java

denemek

yakala

```

1 genel classtestCircleWithException {
2 public static voidtmain(String[] args) {
3 denemek{
4 CircleWithException c1 = tnewtCircleWithException(t5t);n5 CircleWithExcepti
on c2 = tnewtCircleWithException(t-5t);n6 CircleWithException c3 = tnewtCircle
WithException(t0t);
7
8 catch(IllegalArgumentException ex) {n}
9 System.out.println(ex);
10
11 }
12
13 System.out.println("Oluşturulan nesne sayısı: "+
14 CircleWithException.getNumberOfObjects());n14 }
15 }

```



```

java.lang.IllegalArgumentException: Yarıçap negatif olamaz
Oluşturulan nesne sayısı: 1

```

Orijinal **Daire** sınıfı, sınıf adının değişmesi dışında olduğu gibi kalır **CircleWithException**, yeni bir yapıcı **CircleWithException(newRadius)** is eklenmiştir ve **setRadius** metodu artık bir istisna bildiriyor ve eğer yarıçap

negatifse bunu fırlatıyor.

**setRadius** yöntemi, yöntem başlığında **IllegalArgumentException** fırlatacağını belirtir (**CircleWithException.java** dosyasındaki 25-32. satırlar). **throws IllegalArgumentException** ifadesi yöntem bildiriminden kaldırılırsa, **CircleWithException** sınıfı yine de derlenir, çünkü bu **RuntimeException**'in bir alt sınıfıdır ve her yöntem, yöntem başlığında belirtilip belirtilmediğine bakılmaksızın **RuntimeException** (kontROLSÜZ istisna) fırlatılabilir.

Test programı, istisnaları nasıl ele alacağını test etmek için üç **CircleWithException** nesnesi—**c1**, **c2** ve **c3** oluşturur. **new CircleWithException(-5)** (Listeleme 14.8'deki 5. satır) çağrıldığında, **setRadius** yöntemi çağrılır ve bu, negatif bir yarıçap olduğu için bir **IllegalArgumentException** fırlatır. **catch** bloğunda, nesne türü **ex**, **setRadius** yönteminin fırlattığı istisna nesnesi olan **IllegalArgumentException** ile eşleşir, bu nedenle bu istisna **catch** bloğu tarafından yakalanır.

İstisna işleyici, **ex.toString()** (Listeleme 14.8'deki 9. satır) kullanarak, istisna hakkında kısa bir mesaj yazdırır, **System.out.println(ex)**.

İstisna durumunda yürütmenin devam ettiğini unutmayın. Eğer işleyiciler istisnayı yakalamamış olsaydı, program aniden sona erecekti.

Test programı, **try** ifadesi kullanılsa bile derlenir, çünkü yöntem **IllegalArgumentException**, **RuntimeException** (bir kontROLSÜZ istisna) alt sınıfının bir örneğini fırlatır. Eğer bir yöntem **RuntimeException** veya **Error** dışındaki bir istisna fırlatıyorsa, yöntem bir **try-catch** bloğu içinde çağrılmalıdır.

**14.9** İstisnaları tanımlamanın amacı nedir? Bir istisnayı nasıl tanımlarsınız ve nerede? Bir yöntem başlığında birden fazla istisna tanımlayabilir misiniz?

**14.10** Kontrol edilen bir istisna nedir ve kontrol edilmeyen bir istisna nedir?

**14.11** Bir istisna nasıl fırlatılır? Tek bir **throw**

ifadesinde birden fazla istisna fırlatabilir misiniz?

**14.12** **throw** anahtar kelimesi ne için kullanılır? **throws** anahtar kelimesi ne için kullanılır?

Nokta Kontrol Et

**14.13** `statement2` aşağıdaki `try-catch` bloğunda bir istisna oluşturduğunu varsayalım:

```
denemek
 beyan1;nifad
 ifade2;
 e3;n}

catch(İstisna 1 ex1) {n}

catch(İstisna 2 ex2) {n}
```

```
ifade4;
```

Aşağıdaki soruları yanıtlayın:

- İfade `statement3` çalıştırılacak mı?
- İstisna yakalanmazsa, `statement4` çalıştırılacak mı?
- İstisna `catch` bloğunda yakalanırsa, `statement4` çalıştırılacak mı?

**14.14** Aşağıdaki program çalıştırıldığında ne gösterilir?

```
genel sınıfTest {
 public static voidtmain(String[] args) {ntryt{

 int[] list = new int[10];
 System.out.println("list[10] şu şekilde: "+ list[10]);
 }
 catch(ArithmeticException e) {System.ou
 t.println(t"ArithmeticException"t);n}

 catch(RuntimeException e) {System.o
 ut.println(t"RuntimeException"t);n}

 catch(Exception ex) {System
 .out.println(t"Exception"tn}

 }
}
```

**14.15** Aşağıdaki program çalıştırıldığında ne gösterilir?

```
genel sınıfTest {
 public static voidtmain(String[] args) { ntryt{

 metod();
 System.out.println("Metod çağırısından sonra");
 }
 catch(AritmetikHata ex) {
 System.out.println("AritmetikHata");
 }
 catch(ZamanAşımıHatası ex) {
 System.out.println("ZamanAşımıHatası");
 }
 catch(Hata e) {
 System.out.println("Hata");
 }
}

statik voidmetod()atarHata {
```

```

 System.out.println(t1t/t0t);n}

 }

```

**14.16** Aşağıdaki program çalıştırıldığında ne gösterilir?

```

genel sınıfTest {
 public static voidtmain(String[] args) {ntryt{

 metod();
 System.out.println("Metod çağırısından sonra");
 }
 catch(ZamanAşımıHatası ex) {
 System.out.println("main'de ZamanAşımıHatası");
 }
 catch(Exception ex) {
 System.out.println("main'de istisna");
 }
}

static voidtmetho()tthrowstException {ntryt{

 String s =t"abc"t;
 System.out.println(s.charAt(t3t));n}

 catch(RuntimeException ex) {
 System.out.println("metho()de RuntimeException");
 }
 catch(İstisna örneği) {
 System.out.println("metho()de istisna");
 }
}
}

```

**14.17** `getMessage()` metodu ne yapar?

**14.18** `printStackTrace` metodu ne yapar?

**14.19** `try-catch` bloğunun varlığı, istisna oluşmadığında ek yük getirir mi?

**14.20** Aşağıdaki kodda bir derleme hatasını düzeltin:

```

public voidtm(tinttvalue) {nift(val
 ue < t40t)
 throw newException("değer çok küçük");
}

```

## 14.5 finally Kısmı



*finally kısmı, bir istisna oluşup oluşmadığına bakılmaksızın her zaman çalıştırılır.*

Bazen, bir istisna oluşup oluşmadığına bakılmaksızın bazı kodların çalıştırılmasını isteyebilirsiniz. Java, bu amaca ulaşmak için kullanılabilecek bir **finally** kısmına sahiptir. finally kısmının sözdizimi şu şekilde görünebilir:

```

denemek
 ifadeler;n}

catch(İstisna ex) {handling ex;

}

```

```
nihayet{nfi
 nalStatements;n}
```

finally bloğundaki kod, try bloğunda bir istisna oluşup oluşmadığına veya yakalanıp yakalanmadığına bakılmaksızın her durumda çalıştırılır. Üç olası durumu düşünün:

- Eğer **try** bloğunda hiçbir istisna oluşmazsa, **finalStatements** çalıştırılır ve **try** ifadesinden sonraki ifade çalıştırılır.
- Eğer bir ifade, **try** bloğunda bir istisna oluşturursa ve bu **catch** bloğunda yakalanırsa, **try** bloğundaki diğer ifadeler atlanır, **catch** bloğu çalıştırılır ve **finally** ifadesi çalıştırılır. **try** ifadesinden sonraki ifade çalıştırılır.
- Eğer ifadelerden biri, herhangi bir **catch** bloğunda yakalanmayan bir istisna oluşturursa, **try** bloğundaki diğer ifadeler atlanır, **finally** ifadesi çalıştırılır ve istisna bu metodun çağrısına iletilir.

**finally** bloğu, finally bloğuna ulaşmadan önce bir **return** ifadesi olsa bile çalışır.



### Not

**catch** bloğu, finally ifadesi kullanıldığında atlanabilir.

**finally** ifadesinin yaygın bir kullanımı, G/Ç programlamasında görülür. Bir dosyanın tüm kodu altı altında kapatıldığından emin olmak için, bir dosya kapatma ifadesini finally bloğuna yerleştirebilirsiniz. Metin G/Ç daha sonra bu bölümde tanıtılacaktır.

yakalama bloğunu atlama

## 14.21 statement2 aşağıdaki ifadede bir istisna oluşturduğunu varsayalım:

denemek

```
beyan1;nifad
ifade2;
e3;n}
```

catch(İstisna 1 ex1) {n}

```
nihayet{nst
 atement4;n}
```

ifade5;

Aşağıdaki soruları yanıtlayın:

- Eğer hiçbir istisna oluşmazsa, statement4 çalışacak mı ve statement5 çalışacak mı?
- Eğer istisna **Exception1** türündeyse, statement4 çalışacak mı ve statement5 çalışacak mı?
- Eğer istisna **Exception1** türünde değilse, statement4 çalışacak mı ve statement5 çalışacak mı?

Nokta Kontrol Et

MyProgrammingLab™

## 14.6 İstisnaların Ne Zaman Kullanılacağı

*Bir yöntem, hatanın çağırını tarafından işlenmesi gerekiyorsa bir istisna fırlatmalıdır.*

try bloğu, normal koşullarda yürütülen kodu içerir. catch bloğu, olağanüstü koşullarda yürütülen kodu içerir. İstisna işleme, hata işleme kodunu normal programlama görevlerinden ayırarak programların okunmasını kolaylaştırır.



Ana  
Nokta

ve değiştirmek için. Ancak, istisna işleme genellikle daha fazla zaman ve kaynak gerektirdiğinden dikkatli olun, çünkü bu yeni bir istisna nesnesi oluşturmayı, çağrı yığınının geri sarmayı ve işleyiciyi aramak için çağrılan yöntemler zinciri boyunca istisnayı yaymayı gerektirir.

Bir yöntemde bir istisna meydana gelir. İstisnanın çağırılan tarafından işlenmesini istiyorsanız, bir istisna nesnesi oluşturmalı ve onu fırlatmalısınız. İstisnayı meydana geldiği yöntemde işleyebiliyorsanız, istisna fırlatmaya veya kullanmaya gerek yoktur.

Genel olarak, bir projede birden fazla sınıfta meydana gelebilecek yaygın istisnalar, istisna sınıfları için adaydır. Bireysel yöntemlerde meydana gelebilecek basit hatalar, istisna fırlatmadan en iyi şekilde işlenir. Bu, **if** ifadelerini kullanarak hataları kontrol ederek yapılabilir.

Kodda ne zaman **try-catch** bloğu kullanmalısınız? Beklenmeyen hata koşullarıyla başa çıkmanız gerektiğinde kullanın. Basit, beklenen durumlarla başa çıkmak için **try-catch** bloğu kullanmayın. Örneğin, aşağıdaki kod

```
denemek
System.out.println(refVar.toString());n}

catch (NullPointerException ex) {
 System.out.println("refVar null");
}
```

şu şekilde değiştirilmelidir

```
eğer (refVar != null)
 System.out.println(refVar.toString());
aksi takdirde
 System.out.println("refVar null");
```

Hangi durumların istisnai olduğu ve hangilerinin beklenen olduğu bazen karar vermek zordur. Önemli olan, basit bir mantık testini ele almak için istisna yönetimini kötüye kullanmamaktır.

**14.22** Aşağıdaki yöntem bir string'in sayısal bir string olup olmadığını kontrol eder:

```
public static boolean isNumeric(String token) {
 try {
 Double.parseDouble(token);
 return true;
 } catch (NumberFormatException e) {
 return false;
 }
}
```

Doğru mu? İstisnalar kullanmadan yeniden yazın.

## 14.7 İstisnaları Yeniden Fırlatma



*Java, bir istisna işleyicisinin istisnayı yeniden fırlatmasına izin verir; eğer işleyici istisnayı işleyemezse veya sadece çağırana istisna hakkında bilgi vermek isterse.*

Bir istisnayı yeniden fırlatma sözdizimi şu şekilde görünebilir:

```
denemek
ifadeler;n}

catch (İstisna örneği) {
```



```

 çıkışlardan önce işlemleri gerçekleştir;
 throwtex;
 }

```

ifade **throw ex**, istisnayı çağırana yeniden fırlatır, böylece çağırandaki diğer işleyicilerin istisna **ex**'yi işleme şansı olur.

**14.23** Farz edelim ki **statement2**, aşağıdaki ifadede bir istisna oluşturuyor:

```

denemek
 beyan1;nifad
 ifade2;
 e3;n}

catch(İstisna 1 ex1) {n}

catch(Exception2 ex2) {
 throwex2;
}
nihayet{nst
 atement4;n}

ifade5;

```

Aşağıdaki soruları yanıtlayın:

- Eğer hiçbir istisna oluşmazsa, **statement4** çalışacak mı ve **statement5** çalışacak mı?
- Eğer istisna **Exception1** türündeyse, **statement4** çalışacak mı ve **statement5** çalışacak mı?
- Eğer istisna **Exception2** türündeyse, **statement4** çalıştırılacak mı ve **statement5** çalıştırılacak mı?
- Eğer istisna **Exception1** veya **Exception2** değilse, **statement4** çalıştırılacak mı ve **statement5** çalıştırılacak mı?

## 14.8 Zincirleme İstisnalar

*Bir istisnayı başka bir istisna ile birlikte fırlatmak, bir zincirleme istisna oluşturur.*

Önceki bölümde, **catch** bloğu orijinal istisnayı yeniden fırlatır. Bazen, orijinal istisna ile birlikte yeni bir istisna (ek bilgilerle) fırlatmanız gerekebilir. Buna *zincirleme istisnalar* denir. Liste 14.9, zincirleme istisnaları nasıl oluşturup fırlatacağınızı göstermektedir.



zincirlenmiş istisna

### LISTE 14.9 ZincirlemeİstisnaDemo.java

```

1 public classZincirlemeİstisnaDemo {
2 public static voidtmain(String[] args) {
3 denemek
4 yöntem1();n5 }

6 catch(İstisna örneği) {
7 ex.printStackTrace();
8 }metin}

```

yığın izi

## 538 Bölüm 14 İstisna Yönetimi ve Metin G/Ç

```
11 public static void method1() throws Exception {
12 denemek
13 yöntem2();
14 }

15 catch (İstisna örneği) {
16 throw new Exception("method1'den yeni bilgi", ex);
17 }

18 }

19 }

20 public static void method2() throws Exception {
21 throw new Exception("method2'den yeni bilgi");
22 }

23 }

24 }
```

zincirlenmiş istisna

istisna at



```
java.lang.Exception: method1'den yeni bilgi
 at ChainedExceptionDemo.method1(ChainedExceptionDemo.java:16)
 at ChainedExceptionDemo.main(ChainedExceptionDemo.java:4)
Neden: java.lang.Exception: method2'den yeni bilgi
 ChainedExceptionDemo.method2(ChainedExceptionDemo.java:21) üzerinde
 ChainedExceptionDemo.method1(ChainedExceptionDemo.java:13) üzerinde
 ... 1 daha
```

Ana yöntem, method1'i (satır 4) çağırır, method1, method2'yi (satır 13) çağırır ve method2 bir istisna fırlatır (satır 21). Bu istisna, method1'deki catch bloğunda yakalanır ve satır 16'da yeni bir istisna içine sarılır. Yeni istisna fırlatılır ve ana yöntemdeki catch bloğunda satır 6'da yakalanır. Örnek çıktı, satır 7'deki printStackTrace() yönteminin çıktısını gösterir. Method1'den fırlatılan yeni istisna önce gösterilir, ardından method2'den fırlatılan orijinal istisna gelir.

## 14.9 Özel İstisna Sınıflarını Tanımlama

*java.lang.Exception sınıfını genişleterek özel bir istisna sınıfı tanımlayabilirsiniz.*



Ana  
Nokta



VideoNotu

Özel istisna sınıfları oluşturun

Java, oldukça fazla sayıda istisna sınıfı sağlar. Mümkün olduğunca bunları kullanın, kendi istisna sınıflarınızı tanımlamak yerine. Ancak, önceden tanımlanmış istisna sınıflarıyla yeterince tanımlanamayan bir sorunla karşılaşırsanız, Exception'dan veya IOException gibi Exception'ın bir alt sınıfından türetilmiş kendi istisna sınıfınızı oluşturabilirsiniz.

Listing 14.7'de, CircleWithException.java'da, setRadius yöntemi, radyus negatifse bir istisna fırlatır. Radyusu işleyiciye iletmek istiyorsanız, Listing 14.10'da gösterildiği gibi özel bir istisna sınıfı tanımlayabilirsiniz.

### LISTING 14.10 GeçersizYarıçapİstisnası.java

t İstisnayı genişletir

```
1 kamu sınıfı GeçersizYarıçapİstisnası extends Exception {
2 özel double radius;
3
4 /** Bir istisna oluştur */
5 public GeçersizYarıçapİstisnası(double radius) {
6 super("Geçersiz yarıçap "+ radius);
7 this.yarıçap = radius;
8 }
9
10 /** Yarıçapı döndür */
11 genel double getRadius() {
12 geri dönmek
13 }
14 }
```

Bu özel istisna sınıfı `java.lang.Exception` sınıfını genişletir (satır 1). `Exception` sınıfı `java.lang.Throwable` sınıfını genişletir. `Exception` içindeki tüm yöntemler (örneğin, `getMessage()`, `toString()` ve `printStackTrace()`) `Throwable` sınıfından miras alınmıştır. `Exception` sınıfı dört yapıcı içerir. Bunlar arasında, aşağıdaki iki yapıcı sıkça kullanılır:

| <code>java.lang.Exception</code>  |                                          |
|-----------------------------------|------------------------------------------|
| <code>+Hata()</code>              | Constructs an exception with no message. |
| <code>+Hata(mesaj: String)</code> | Belirtilen mesaj ile bir hata oluşturur. |

Satır 6, bir mesaj ile üst sınıfın yapıcısını çağırır. Bu mesaj, istisna nesnesinde ayarlanacak ve nesne üzerinde `getMessage()` çağrılarak elde edilebilir.



#### Ttip

Java API'sindeki çoğu istisna sınıfı, bir argüman almayan bir yapıcı ve bir mesaj parametresi olan bir yapıcı içerir.

Bir `InvalidRadiusException` oluşturmak için bir yarıçap geçirmeniz gerekir. Bu nedenle, Listing 14.7'deki `setRadius` yöntemi, Listing 14.11'de gösterildiği gibi değiştirilebilir.

## LISTELERİ 14.11 TestCircleWithCustomException.java

```

1 genel classtestCircleWithCustomException {
2 public static void main(String[] args) {
3 denemek
4 newtCircleWithCustomException(t5t);
5 newtCircleWithCustomException(t-5t);
6 newtCircleWithCustomException(t0t);
7 }
8 catch(Geçersiz Yarıçap İstisnası eski) {
9 System.out.println(ex);n10 }

11
12 System.out.println("Oluşturulan nesne sayısı: "+
13 CircleWithException.getNumberOfObjects());
14 }n15 }

16
17 classÖzelHataİleDaire {
18 /** Dairenin yarıçapı */
19 özel doubleradius;
20
21 /** Oluşturulan nesne sayısı */
22 özel statik intnumberOfObjects = 0;
23
24 /** Yarıçapı 1 olan bir daire oluştur */
25 halk CircleWithCustomException() throwsGeçersizYarıçapHatası { istisna beyan et
26 bu(1.0);
27 }
28
29 /** Belirtilen yarıçapa sahip bir daire oluştur */
30 publicÖzelHataİleDaire(doubleYeniYarıçap) istisna at
31 throwsGeçersizYarıçapHatası {
32 setRadius(YeniYarıçap);
33 nesneSayısı++;
34 }
35
36 /** Yarıçapı döndür */

```

```

37 genel double getRadius() {
38 geri dönmek
39 }
40
41 /** Yeni bir yarıçap ayarla */
42 public void setRadius(double yeniYarıçap)
43 atılır InvalidRadiusException {
44 if (newRadius >= t0t)
45 yarıçap = yeniYarıçap;
46 başka
47 yeniInvalidRadiusException(newYarıçap);
48 }
49
50 /** numberOfObjects değerini döndür */
51 genel statik int getNumberOfObjects() {
52 return numberOfObjects;
53 }
54
55 /** Bu dairenin alanını döndür */
56 public double alanBul() {
57 return radyus * radyus * 3.14159;
58 }
59 }

```



InvalidRadiusException: Geçersiz yarıçap -5.0  
Oluşturulan nesne sayısı: 0

setRadius metodu CircleWithCustomException içinde InvalidRadius-Exception atar, yarıçap negatif olduğunda (satır 47). InvalidRadiusException kontrol edilen bir istisna olduğundan, setRadius metodu bunu metod başlığında bildirmelidir (satır 42). CircleWithCustomException için oluşturucular, yeni bir yarıçap ayarlamak için setRadius metodunu çağırır ve bu bir InvalidRadiusException atabilir, bu nedenle oluşturucuların

InvalidRadiusException atması gerektiği bildirilmiştir (satırlar 25, 31).

Yeni CircleWithCustomException(-5) çağırısı InvalidRadius-Exception atar, bu da işleyici tarafından yakalanır. İşleyici, istisna nesnesindeki yarıçapı gösterir **ex**.

kontrol edilen özel istisna



**Tt**

**RuntimeException** genişleterek özel bir istisna sınıfı tanımlayabilir misiniz? Evet, ama bu iyi bir yol değildir, çünkü özel istisnayı kontrolsüz hale getirir. Özel bir istisna nın kontrol edilen olması daha iyidir, böylece derleyici bu istisnaların programınızda yakalanmasını zorlayabilir.

#### 14.24 Özel bir istisna sınıfını nasıl tanımlarsınız?

#### 14.25 Diyelim ki setRadius metodu, InvalidRadiusException'ı fırlatıyor

Listing 14.10'da tanımlandığı gibi. Aşağıdaki program çalıştırıldığında ne gösterilir?

```

genel sınıf Test {
 public static void main(String[] args) {
 try {
 metod();
 System.out.println("Metod çağırısından sonra");
 }
 catch (RuntimeException ex) {
 System.out.println("main'de RuntimeException");
 }
 }
}

```

Nokta Kontrol Et

MyProgrammingLab™

```

 }
 catch (Exception ex) {
 System.out.println("main'de istisna");
 }
}

static void tmethod() throws Exception {
 try {
 Çember c1 = new Circle(t1t);
 nc1.setRadius(t-1t);
 System.out.println(c1.getRadius());
 } catch (RuntimeException ex) {
 System.out.println("method()'de RuntimeException");
 }
 catch (İstisna örneği) {
 System.out.println("method()'de İstisna");
 throw ex;
 }
}
}

```

## 14.10 File Sınıfı

*File sınıfı, bir dosyanın/dizinlerin özelliklerini elde etmek için yöntemleri ve bir dosyanın/dizinlerin adını değiştirme ve silme işlemleri için yöntemleri içerir.*



İstisna yönetimini öğrendikten sonra, dosya işleme aşamasına geçmeye hazırsınız. Programda saklanan veriler geçicidir; program sona erdiğinde kaybolurlar. Bir programda oluşturulan verileri kalıcı olarak saklamak için, bunları bir diskte veya başka bir kalıcı depolama aygıtında bir dosyaya kaydetmeniz gerekir. Dosya daha sonra taşınabilir ve diğer programlar tarafından okunabilir. Veriler dosyalarda saklandığı için, bu bölüm File sınıfını kullanarak dosya/dizin özelliklerini elde etme, dosyaları/dizinleri silme ve yeniden adlandırma ile dizinler oluşturma yöntemlerini tanıtmaktadır. Bir sonraki bölüm, metin dosyalarından veri okuma/yazma yöntemlerini tanıtmaktadır.

neden dosya?

Her dosya, dosya sisteminde bir dizine yerleştirilir. Bir mutlak dosya adı (veya tam adı), bir dosya adını, tam yolu ve sürücü harfini içerir. Örneğin, **c:\ook\Welcome.java**, Windows işletim sistemindeki **Welcome.java** dosyası için mutlak dosya adıdır. Burada **c:\ook**, dosya için dizin yolu olarak adlandırılır. Mutlak dosya adları makineye bağımlıdır. UNIX platformunda, mutlak dosya adı şu şekilde olabilir: **/home/liang/book/Welcome.java**, burada **/home/liang/book**, **Welcome.java** dosyası için dizin yoludur.

mutlak dosya adı

bağımlı yolu

Bir *görelî dosya adı*, mevcut çalışma dizinine ilişkindir. Bir görelî dosya adı için tam dizin yolu atanır. Örneğin, **Welcome.java** bir görelî dosya adıdır. Eğer mevcut çalışma dizini **c:\ook** ise, mutlak dosya adı **c:\ook\Welcome.java** olur.

görelî dosya adı

File sınıfı, dosyalar ve yol adları ile ilgili makine bağımsız karmaşıklıkların çoğunu makine bağımsız bir şekilde ele alan bir soyutlama sağlamayı amaçlamaktadır. File sınıfı, dosya ve dizin özelliklerini elde etmek, dosyaları ve dizinleri yeniden adlandırmak ve silmek için yöntemleri içerir, bu da Şekil 14.6'da gösterilmiştir. Ancak, File sınıfı, dosya içeriklerini okuma ve yazma yöntemlerini içermez.

Dosya adı bir dizedir. **File** sınıfı, dosya adı ve dizin yolunun bir sarmalayıcı sınıfıdır. Örneğin, `new File("c:\ook")`, dizin için bir **File** nesnesi oluşturur: **ook** ve `new File("c:\ook\kest.dat")`, dosya için bir **File** nesnesi oluşturur: **ooktest.dat**, her ikisi de Windows'ta. File sınıfının `isDirectory()` yöntemini kullanarak nesnenin bir dizini temsil edip etmediğini kontrol edebilir ve `isFile()` yöntemini kullanarak nesnenin bir dosyayı temsil edip etmediğini kontrol edebilirsiniz.

| io.File                              |                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| +File(pathname: String)              | Belirtilen yol adı için bir File nesnesi oluşturur. Yol adı bir dizin veya dosya olabilir.                                                                                                                                                                                                                               |
| +File(parent: String, child: String) | Dizin ana altında bir çocuk için bir File nesnesi oluşturur. Çocuk bir dosya adı veya alt dizin olabilir.                                                                                                                                                                                                                |
| +File(parent: File, child: String)   | Dizin ana altında bir çocuk için bir File nesnesi oluşturur. Ana bir File nesnesidir. Önceki yapıcıda, ana bir dizedir.                                                                                                                                                                                                  |
| +exists(): boolean                   | File nesnesi tarafından temsil edilen dosya veya dizin mevcutsa true döner.                                                                                                                                                                                                                                              |
| +canRead(): boolean                  | File nesnesi tarafından temsil edilen dosya mevcutsa ve okunabiliyorsa true döner.                                                                                                                                                                                                                                       |
| +canWrite(): boolean                 | File nesnesi tarafından temsil edilen dosya mevcutsa ve yazılabiliriyorsa true döner.                                                                                                                                                                                                                                    |
| +isDirectory(): boolean              | File nesnesi bir dizini temsil ediyorsa true döner.                                                                                                                                                                                                                                                                      |
| +isFile(): boolean                   | File nesnesi bir dosyayı temsil ediyorsa true döner.                                                                                                                                                                                                                                                                     |
| +isAbsolute(): boolean               | Mutlak bir yol adı kullanılarak oluşturulmuşsa, File nesnesinin true döndürmesini sağlar.                                                                                                                                                                                                                                |
| +isHidden(): boolean                 | File nesnesinde temsil edilen dosya gizli ise true döndürür. gizli teriminin kesin tanımları sistem bağımlıdır. Windows'ta, bir dosyayı Dosya Özellikleri iletişim kutusunda gizli olarak işaretleyebilirsiniz. Unix sistemlerinde, bir dosya gizli olarak kabul edilir eğer adı bir nokta (.) karakteri ile başlıyorsa. |
| +getAbsolutePath(): String           | File nesnesi tarafından temsil edilen tam mutlak dosya veya dizin adını döndürür.                                                                                                                                                                                                                                        |
| +getCanonicalPath(): String          | Yol adından gereksiz adları, örneğin "." ve ".." kaldırarak, sembolik bağlantıları çözümler (Unix'te) ve sürücü harflerini standart büyük harfe dönüştürerek (Windows'ta) getAbsolutePath() ile aynı sonucu döndürür.                                                                                                    |
| +getName(): String                   | File nesnesi tarafından temsil edilen tam dizin ve dosya adının son adını döndürür. Örneğin, new File("c:\lookest.dat").getName() çağırısı test.dat döndürür.                                                                                                                                                            |
| +getPath(): String                   | File nesnesi tarafından temsil edilen tam dizin ve dosya adını döndürür. Örneğin, new File("c:\lookest.dat").getPath() çağırısı c:\lookest.dat döndürür.                                                                                                                                                                 |
| +getParent(): String                 | File nesnesi tarafından temsil edilen mevcut dizinin veya dosyanın tam üst dizinini döndürür. Örneğin, new File("c:\lookest.dat").getParent() çağırısı c:\look döndürür.                                                                                                                                                 |
| +lastModified(): long                | Dosyanın en son ne zaman değiştirildiğini döndürür.                                                                                                                                                                                                                                                                      |
| +length(): long                      | Dosyanın boyutunu döndürür; eğer dosya yoksa veya bir dizin ise 0 döner.                                                                                                                                                                                                                                                 |
| +listFile(): File[]                  | Bir dizin File nesnesi için dizin altındaki dosyaları döndürür.                                                                                                                                                                                                                                                          |
| +delete(): boolean                   | Bu File nesnesi tarafından temsil edilen dosyayı veya dizini siler. Yöntem, silme işlemi başarılı olursa true döner.                                                                                                                                                                                                     |
| +renameTo(dest: File): boolean       | Bu File nesnesi tarafından temsil edilen dosya veya dizini belirtilen isimle yeniden adlandırır dest'te temsil edilen. İşlem başarılı olursa yöntem true döner.                                                                                                                                                          |
| +mkdir(): boolean                    | Bu File nesnesinde temsil edilen bir dizin oluşturur. Dizini başarıyla oluşturulursa true döner.                                                                                                                                                                                                                         |
| +mkdirs(): boolean                   | mkdir() ile aynı, ancak ebeveyn dizinleri mevcut değilse ebeveyn dizinleri ile birlikte dizin oluşturur.                                                                                                                                                                                                                 |

**FIGURE 14.6** **File** sınıfı, dosya ve dizin özelliklerini elde etmek, dosyaları ve dizinleri silmek ve yeniden adlandırmak ve dizinler oluşturmak için kullanılabilir.

dosya adlarında



### Uyarı

Windows için dizin ayırıcı bir ters eğik çizgidir (**\**). Ters eğik çizgi, Java'da özel bir karakterdir ve bir dize literalinde **\** olarak yazılmalıdır (bkz. Tablo 2.6).

göreceli dosya adı



### Not

Bir File örneği oluşturmak, makinede bir dosya oluşturmaz. Bir Herhangi bir dosya adı için olup olmadığına bakılmaksızın bir File örneği. Dosyanın var olup olmadığını kontrol etmek için bir File örneği üzerinde exists() yöntemini çağırabilirsiniz.

Java dizin ayırıcı (/)

Programınızda mutlak dosya adları kullanmayın. Eğer aşağıdaki gibi bir dosya adı kullanırsanız **c:\look\Welcome.java**, Windows üzerinde çalışacak ancak diğer platformlarda çalışmayacaktır. Şu anki dizine göre bir dosya adı kullanmalısınız. Örneğin, mevcut dizindeki **Welcome.java** dosyası için **new File("Welcome.java")** kullanarak bir File nesnesi oluşturabilirsiniz. Mevcut dizindeki **image** dizininde bulunan **us.gif** dosyası için **new File("image/us.gif")** kullanarak bir File nesnesi oluşturabilirsiniz. İleri eğik çizgi (**/**) Java dizinidir.

ayırıcı, UNIX'teki ile aynıdır. `new File("image/us.gif")`

ifadesi Windows, UNIX ve diğer tüm platformlarda çalışır.

Liste 14.12, bir `File` nesnesi oluşturmayı ve `File` sınıfındaki yöntemleri kullanarak özelliklerini elde etmeyi gösterir. Program, `us.gif` dosyası için bir `File` nesnesi oluşturur. Bu dosya, mevcut dizindeki `image` dizininde saklanmaktadır.

## LISTING 14.12 `TestFileClass.java`

```

1 genel sınıf TestFileClass {
2 public static void main(String[] args) {
3 java.io.File dosyası = new java.io.File(t"image/us.gif"t);
4 System.out.println("Var mı? " + dosya.exists());
5 System.out.println("Dosya " + file.length() + " bayt");
6 System.out.println("Okunabilir mi? " + file.canRead());
7 System.out.println("Yazılabilir mi? " + file.canWrite());
8 System.out.println("Bir dizin mi? " + file.isDirectory());
9 System.out.println("Bir dosya mı? " + file.isFile());
10 System.out.println("Kesin mi? " + file.isAbsolute());
11 System.out.println("Gizli mi? " + file.isHidden());
12 System.out.println("Kesin yol " +
13 file.getAbsolutePath());
14 System.out.println("Son değişiklik tarihi " +
15 new java.util.Date(file.lastModified());
16 }

```

bir `File` nesnesi oluştur  
`exists()`  
`uzunluk()`  
`ncanRead()`  
`ncanWrite()`  
`nisDirectory()`  
`File()`  
`isAbsolute()`  
`Hidden()`  
`nge`  
`tAbsolutePath()`  
`nlast`  
`Modified()`

`lastModified()` metodu, dosyanın en son ne zaman değiştirildiğini gösteren tarih ve saati döndürür, UNIX zamanının başlangıcından (00:00:00 GMT, 1 Ocak 1970) itibaren milisaniye cinsinden ölçülür. `Date` sınıfı, bunu 14-15. satırlarda okunabilir bir formatta görüntülemek için kullanılır.

Şekil 14.7a, programın Windows'taki bir örnek çalışmasını gösterirken, Şekil 14.7b, UNIX'teki bir örnek çalışmasını göstermektedir. Şekillerde gösterildiği gibi, Windows'taki yol adlandırma kuralları UNIX'takilerden farklıdır.

```

C:\book>java TestFileClass
Does it exist? true
The file has 2998 bytes
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
Absolute path is C:\book\image\us.gif
Last modified on Tue Nov 02 08:20:45 EST 2004
C:\book>

```

(a) Windows'ta

```

[daniel@panda book]$ java TestFileClass
Does it exist? true
The file has 2998 bytes
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
Absolute path is /home/daniel/book/image/us.gif
Last modified on Tue Nov 02 08:20:45 EST 2004
[daniel@panda book]$

```

(b) UNIX'ta

**Fiç 14.7** Program bir `File` nesnesi oluşturur ve dosya özelliklerini görüntüler.

**14.26** Aşağıdaki ifadeyi kullanarak bir `File` nesnesi oluşturmanın yanlış olanı nedir?

`newFile(t"c:booktest.dat"t);`

**14.27** Bir dosyanın zaten var olup olmadığını nasıl kontrol edersiniz? Bir dosyayı nasıl silersiniz? Bir dosyayı nasıl yeniden adlandırırsınız? `File` sınıfını kullanarak dosya boyutunu (bayt sayısını) bulabilir misiniz? Bir dizini nasıl oluştursunuz?

Nokta Kontrol Et

MyProgrammingLab™

**14.28** I/O için **File** sınıfını kullanabilir misiniz? Bir **File** nesnesi oluşturmak diskte bir dosya mı oluşturur?

## 14.11 Dosya Girişi ve Çıkışı



*Bir dosyadan metin verisi okumak için **Scanner** sınıfını ve bir dosyaya metin verisi yazmak için **PrintWriter** sınıfını kullanın.*



VideoNotu

Veri yazma ve okuma

Bir **File** nesnesi, bir dosyanın veya bir yolun özelliklerini kapsüller, ancak bir dosya oluşturma veya bir dosyaya veri yazma/okuma yöntemlerini içermez (veri *girişi ve çıkışı* olarak adlandırılır veya kısaca *I/O*). I/O işlemleri gerçekleştirmek için uygun Java I/O sınıflarını kullanarak nesneler oluşturmanız gerekir. Bu nesneler, bir dosyadan veri okumak/yazmak için yöntemleri içerir. İki tür dosya vardır: metin ve ikili. Metin dosyaları esasen diskteki dizelerdir. Bu bölüm, **Scanner** ve **PrintWriter** sınıflarını kullanarak bir metin dosyasından/metin dosyasına dizeleri ve sayısal değerleri nasıl okuyup/yazacağınızı tanıtmaktadır. **İkili dosyalar 19. Bölümde** tanıtılacaktır.

### 14.11.1 **PrintWriter** Kullanarak Veri Yazma

`java.io.PrintWriter` sınıfı, bir dosya oluşturmak ve bir metin dosyasına veri yazmak için kullanılabilir. Öncelikle, bir metin dosyası için aşağıdaki gibi bir **PrintWriter** nesnesi oluşturmalısınız:

```
PrintWriter çıktısı = new PrintWriter(dosya adı);
```

Daha sonra, bir dosyaya veri yazmak için **PrintWriter** nesnesi üzerinde `print`, `println` ve `printf` yöntemlerini çağırabilirsiniz. **Şekil 14.8, **PrintWriter** içinde sık kullanılan yöntemleri** özetlemektedir.

| <code>java.io.PrintWriter</code>                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>+PrintWriter(dosya: File) +PrintWriter(dosyaAdı: String) +print(s: String): void +print(c: char): void +print(cDizisi: char[]): void +print(i: int): void +yazdırma(l: uzun): void +print(f: float): void +print(d: double): void +print(b: boolean): void</pre> <p>Ayrıca aşırı yüklenmiş<br/><code>println</code> yöntemlerini içerir.</p> <p>Ayrıca aşırı yüklenmiş<br/><code>printf</code> yöntemlerini içerir.</p> | <p>Belirtilen dosya nesnesi için bir <b>PrintWriter</b> nesnesi oluşturur.</p> <p>Belirtilen dosya adı dizesi için bir <b>PrintWriter</b> nesnesi oluşturur.</p> <p>Bir dizeyi dosyaya yazar.</p> <p>Bir karakteri dosyaya yazar.</p> <p>Bir dizi karakteri dosyaya yazar.</p> <p>Bir <code>int</code> değerini dosyaya yazar.</p> <p>Bir <code>long</code> değerini dosyaya yazar.</p> <p>Bir <code>float</code> değerini dosyaya yazar.</p> <p>Bir <code>double</code> değerini dosyaya yazar.</p> <p>Bir <code>boolean</code> değerini dosyaya yazar.</p> <p>Bir <code>println</code> metodu, bir <code>print</code> metodu gibi çalışır; ayrıca, bir satır ayırıcı yazdırır. Satır ayırıcı dizesi sistem tarafından tanımlanır. Windows'ta <code>\r</code> ve Unix'te <code>\n</code> olarak tanımlıdır.</p> <p><code>printf</code> yöntemi §3.16, "Konsol Çıktısını Biçimlendirme" bölümünde tanıtılmıştır.</p> |

**ŞEKİL 14.8** **PrintWriter** sınıfı, bir metin dosyasına veri yazma yöntemlerini içerir.

Liste 14.13, bir **PrintWriter** örneği oluşturan ve dosyaya **scores.txt**. İki satır yazan bir örnek vermektedir. Her satır bir ad (bir dize), bir orta ad kısaltması (bir karakter), bir soy ad (bir dize) ve bir puan (bir tamsayı) içermektedir.

### LISTE 14.13 **WriteData.java**

bir istisna fırlatır  
File nesnesi oluştur  
file mevcut mu?

```
1 genel sınıf WriteData {
2 public static void main(String[] args) IOException {
3 java.io.File dosyası = new java.io.File("scores.txt");
4 if (dosyası.exists()) {
```



```

5 System.out.println("Dosya zaten mevcut");
6 System.exit(1);
7 }
8
9 // Bir dosya oluştur
10 java.io.PrintWriter output = new java.io.PrintWriter(dosya);
11
12 // Dosyaya biçimlendirilmiş çıktı yaz
13 output.print("John T Smith ");
14 output.println(90);
15 çıktı.print(t"Eric K Jones "t);n16 çıktı.println(t
85t);
17
18 // Dosyayı kapat
19 output.close();
20 }

```

oluşturPrintWriter

veri yazdır

John T Smith 90 nE  
ric K Jones 85

puanlar.txt

dosyayı kapat

yirmi bir }

4-7. Satırlar, scores.txt dosyasının mevcut olup olmadığını kontrol eder. Eğer öyleyse, programdan çıkılır (6. satır).

**PrintWriter** yapıcısını çağırmak, dosya mevcut değilse yeni bir dosya oluşturur. Eğer dosya zaten mevcutsa, dosyadaki mevcut içerik kullanıcıyla doğrulamadan silinerek temiz edilir.

**PrintWriter** yapıcısını çağırmak bir I/O istisnası fırlatabilir. Java, bu tür bir istisna ile başa çıkmak için kod yazmanızı zorunlu kılar. Basitlik açısından, **throws IOException** ana yöntem başlığında (satır 2).

Kullanmışsınız **System.out.println** ve **System.out.printf** yöntemlerini konsola metin yazmak için. **System.out**, konsol için standart Java nesnesidir. Herhangi bir dosyaya metin yazmak için **PrintWriter** nesneleri oluşturabilirsiniz **print**, **println** ve **printf** kullanarak (satırlar 13-16).

Dosyayı kapatmak için **close()** yöntemi kullanılmalıdır. Bu yöntem çağrılmazsa, veriler dosyada düzgün bir şekilde kaydedilmeyebilir.

bir dosya oluştur

IOException fırlatır

yazdır yöntemi

dosyayı kapat

### 14.11.2 Veri Okuma **Scanner** Kullanarak

**java.util.Scanner** sınıfı, Bölüm 2.3'te, Konsoldan Girdi Okuma'da, dizeleri ve ilk değerleri okumak için kullanıldı. **Scanner**, girişini boşluk karakterleriyle ayrılmış token'lara böler. Klavyeden okumak için, aşağıdaki gibi **System.in** için bir **Scanner** oluşturursunuz:

```
Tarayıcı girişi = new Scanner(System.in);
```

Bir dosyadan okumak için, aşağıdaki gibi bir dosya için **Scanner** oluşturun:

```
Tarayıcı girişi = new Scanner(new File(dosyaadi));
```

Şekil 14.9, **Scanner**'da sıkça kullanılan yöntemleri özetlemektedir.

Liste 14.14, **Scanner** örneği oluşturan ve dosyadan veri okuyan bir örnek vermektedir. dosya **scores.txt**.

### LISTING 14.14 **ReadData.java**

```

1 java.util.Scanner'ı içe aktarın;
2
3 genel sınıf ReadData {
4 public static void main(String[] args) {
5 // Bir Dosya örneği oluştur
6 java.io.File dosya = new File("scores.txt");
7
8 // Dosya için bir Scanner oluştur

```

bir dosya oluştur

## 546 Bölüm 14 İstisna Yönetimi ve Metin G/Ç

bir Scanner oluştur

sonraki var mı?  
öğeleri oku

dosyayı kapat

```
9 Scanner girdi = new Scanner(dosya);
10
11 // Bir dosyadan veri oku
12 while (girdi.hasNext()) {
13 String ilkAd = girdi.next();
14 String mi = girdi.next();
15 String soyad = girdi.next();
16 int puan = girdi.nextInt();
17 System.out.println(
18 firstName + " " + mi + " " + lastName + " " + puan);
19 }
20
21 // Dosyayı kapat
22 input.close();
```

puanlar.txt

John J Smith 90  
Eric K Jones 85

### java.util.Scanner

+Scanner(source: File)  
+Scanner(source: String)  
+close()  
+hasNext(): boolean  
+next(): String  
+nextLine(): String  
+nextByte(): byte  
+nextShort(): short  
+nextInt(): int  
+nextLong(): long  
+nextFloat(): float  
+nextDouble(): double  
+useDelimiter(pattern: String): Scanner

Belirtilen dosyadan taranan değerleri üreten bir Scanner oluşturur.  
Belirtilen stringden taranan değerleri üreten bir Scanner oluşturur.  
Bu tarayıcıyı kapatır.  
Bu tarayıcının okunacak daha fazla verisi varsa true döner.  
Bu tarayıcıdan bir dize olarak bir sonraki tokeni döner.  
Bu tarayıcıdan satır sonu ile birlikte bir satır döner.  
Bu tarayıcıdan bir byte olarak bir sonraki tokeni döner.  
Bu tarayıcıdan bir short olarak bir sonraki tokeni döner.  
Bu tarayıcıdan bir int olarak bir sonraki belirteci döndürür.  
Bu tarayıcıdan bir long olarak bir sonraki belirteci döndürür.  
Bu tarayıcıdan bir float olarak bir sonraki belirteci döndürür.  
Bu tarayıcıdan bir double olarak bir sonraki belirteci döndürür.  
Bu tarayıcının ayırıcı desenini ayarlar ve bu tarayıcıyı döndürür.

FIGURE 14.9 Tarayıcı sınıfı verileri taramak için yöntemleri içerir.

Dosya sınıfı

istisna fırlatır

dosyayı kapat

Yeni bir `Scanner(String)` oluşturmanın, belirli bir dize için bir `Scanner` oluşturduğunu unutmayın. Bir dosyadan veri okumak için bir `Scanner` oluşturmak için, `java.io.File` sınıfını kullanarak `File`'i oluşturmak için `new File(filename)` yapıcısını (satır 6) kullanmalısınız ve dosya için bir `Scanner` oluşturmak için `new Scanner(File)` kullanmalısınız (satır 9).

Yapıcıyı çağırmak `new Scanner(File)` bir I/O istisnası atabilir, bu nedenle `main` metodu `throws Exception` ifadesini 4. satırda belirtir.

Her yinelemede `while` döngüsü, metin dosyasından (satırlar 12-19) ilk adı, orta kısaltmayı, soyadı ve puanı okur.

Dosya 22. satırda kapatılır.

Giriş dosyasını kapatmak gerekli değildir (22. satır), ancak dosya tarafından işgal edilen kaynakları serbest bırakmak için bunu yapmak iyi bir uygulamadır.

### 14.11.3 Scanner Nasıl Çalışır?

token-okuma yöntemi  
ayracı değiştir

Bu `nextByte()`, `nextShort()`, `nextInt()`, `sonrakiUzun()`, `nextFloat()`, `nextDouble()` ve `next()` yöntemleri *token-okuma yöntemleri* olarak bilinir, çünkü bunlar ayırıcılarla ayrılmış token'leri okur. Varsayılan olarak, ayırıcılar boşluk karakterleridir. Yeni bir desen ayarlamak için `useDelimiter(String regex)` yöntemini kullanabilirsiniz.

Bir giriş yöntemi nasıl çalışır? Bir token-okuma yöntemi öncelikle herhangi bir ayırıcıyı (var sayılan olarak boşluk) atlar, ardından bir ayırıcıda sona eren bir token okur. Token daha sonra otomatik olarak **byte, short, int, long, float** veya **double** türünde bir değere dönüştürülür; sırasıyla **nextByte()**, **nextShort()**, **nextInt()**, **nextLong()**, **nextFloat()** ve **nextDouble()** için. **next()** yöntemi için, herhangi bir dönüşüm yapılmaz. Token beklenen türle eşleşmezse, bir çalışma zamanı istisnası `java.util.InputMismatchException` fırlatılacaktır.

`inputMismatchExceptionnn`  
`ext()t vs.t nextLine()`

Her iki yöntem **next()** ve **nextLine()** bir dize okur. **next()** yöntemi ayırıcılarla sınırlı bir dize okur ve **nextLine()** bir satırı satır ayırıcı ile sona eren bir satırı okur.



### Not

Satır ayırıcı dizesi sistem tarafından tanımlanır. Windows'ta `\r` ve UNIX'te `\n` olarak tanımlanır. Belirli bir platformda satır ayırıcıyı almak için kullanın

satır ayırıcı

```
String lineSeparator = System.getProperty("line.separator");
```

Eğer bir klavyeden giriş yaparsanız, bir satır *Enter* tuşu ile biter, bu da the karakterine karşılık gelir.

Token okuma yöntemi, token'dan sonra gelen ayırıcıyı okumaz. Eğer bir token okuma yönteminde sonra **nextLine()** metodu çağrılırsa, bu metod ayırıcıdan başlayarak ve satır ayırıcı ile biten karakterleri okur. Satır ayırıcı okunur, ancak **nextLine()** tarafından döndürülen string'in bir parçası değildir.

**nextLine()**'in

Varsayalım ki `test.txt` adında bir metin dosyası bir satır içeriyor.

dosyadan giriş

34 567

Aşağıdaki kod çalıştırıldıktan sonra,

```
Tarayıcı girişi = new Scanner(new File("test.txt"));
int value = input.nextInt();
Dize satırı = input.nextLine();
```

**intValue34** içerir ve **line** ' ', 5, 6 ve 7 karakterlerini içerir.

Girdi klavyeden girilirse ne olur? Varsayalım ki 34 giriyorsunuz, Enter tuşuna basıyorsunuz, ardından 567 giriyorsunuz ve aşağıdaki kod için Enter tuşuna basıyorsunuz:

klavyeden giriş

```
Tarayıcı girişi = new Scanner(System.in);
int value = input.nextInt();
Dize satırı = input.nextLine();
```

**intValue34** alırsınız ve **line** boş bir string alır. Neden? İşte sebebi. Token okuma yöntemi **nextInt()** 34 okur ve ayırıcıda durur, bu durumda bir satır ayırıcıdır (Enter tuşu). **nextLine()** metodu satır ayırıcıyı okuduktan sonra sona erer ve satır ayırıcıdan önce okunan string'i döndürür. Satır ayırıcıdan önce hiçbir karakter olmadığından, **line** boş olur.

Bir dosyadan veya klavyeden veri okuyabilirsiniz **Scanner** sınıfını kullanarak. Ayrıca bir dizeden veri tarayabilirsiniz **Scanner** sınıfını kullanarak. Örneğin, aşağıdaki kod

bir dizeyi tara

```
Scanner input = new Scanner("13 14");
int sum = input.nextInt() + input.nextInt();
System.out.println("Toplam " + sum);
```

gösterir

Toplam 27'dir

## 14.11.4 Vaka Çalışması: Metni Değiştirme

Bir metin dosyasındaki bir dizeyi yeni bir dize ile değiştiren **ReplaceText** adlı bir program yazmanız gerektiğini varsayalım. Dosya adı ve dizeler, komut satırı argümanları olarak aşağıdaki gibi geçer:

```
java ChangeText sourceFile targetFile oldString newString
```

Örneğin,

```
java ChangeText FormatString.java t.txt StringBuffer StringBuffer
```

t.txt dosyasında **StringBuilde**r olan tüm örnekleri **StringBuffer** ile değiştirir ve yeni dosyayı **t.txt** olarak kaydeder.

Liste 14.15 programı vermektedir. Program, **main** metoduna geçirilen argüman sayısını kontrol eder (satırlar 7–11), kaynak ve hedef dosyaların var olup olmadığını kontrol eder (satırlar 14–25), kaynak dosya için bir **Scanner** oluşturur (satır 28), hedef dosya için bir **PrintWriter** oluşturur ve kaynak dosya adan bir satırı tekrar tekrar okur (satır 32), metni değiştirir (satır 33) ve hedef dosyaya yeni bir satır yazar (satır 34). Verilerin dosyaya düzgün bir şekilde kaydedilmesini sağlamak için çıktı dosyasını kapatmalısınız (satır 38).

## LISTING 14.15 ReplaceText.java

```

1 import java.io.*;
2 import java.util.*;
3
4 genel classt ReplaceText {
5 public static void main(String[] args) throws Exception {
6 // Komut satırı parametre kullanımını kontrol et
7 if (args.uzunluk != 4) {
8 System.out.println(
9 "Kullanım: java ReplaceText kaynakDosya hedefDosya eskiStr yeniStr");
10 System.exit(1);
11 }
12
13 // Kaynak dosyanın var olup olmadığını kontrol et
14 sourceFile = new File(args[0]);
15 if (!sourceFile.exists()) {
16 System.out.println("Kaynak dosya " + args[0] + " mevcut değil");
17 System.exit(1);
18 }
19
20 // Hedef dosyanın var olup olmadığını kontrol et
21 targetFile = new File(args[1]);
22 if (targetFile.exists()) {
23 System.out.println("Hedef dosya " + args[1] + " zaten mevcut");
24 System.exit(3);
25 }
26
27 // Girdi için bir Scanner ve çıktı için bir PrintWriter oluştur
28 Scanner input = new Scanner(sourceFile);
29 PrintWriter output = new PrintWriter(targetFile);
30
31 while (input.hasNext()) {
32 String s1 = input.nextLine();
33 String s2 = s1.replaceAll(args[2], args[3]);
34 output.println(s2);
35 }
36 }

```

komut kullanımını kontrol et

kaynak dosya mevcut mu?

hedef dosya mevcut mu?

bir Scanner oluştur

bir PrintWriter oluştur

sonraki var mı?

bir satır oku

```
37 input.close();n38 çıktı.clo
se();n39 }
```

dosyayı kapat

```
40 }
```

Normal bir durumda, bir dosya kopyalandıktan sonra program sonlandırılır. Komut satırı argümanları düzgün kullanılmadığında (satırlar 7-11), kaynak dosya mevcut değilse (satırlar 14-18) veya hedef dosya zaten mevcutsa (satırlar 22-25) program anormal bir şekilde sonlandırılır. Çıkış durumu kodları 1, 2 ve 3, bu anormal sonlandırmaları belirtmek için kullanılır (satırlar 10, 17, 24).

**14.29** Bir dosyaya veri yazmak için nasıl bir **PrintWriter** oluşturursunuz? Listing 14.13, WriteData.java'daki ana yöntemde **throws Exception** bildiriminin bulunmasının nedeni nedir? Listing 14.13'te **close()** yöntemi çağrılmazsa ne olurdu?

**14.30** Aşağıdaki program çalıştırıldıktan sonra **temp.txt** dosyasının içeriğini gösterin.

```
genel sınıfTest {
 public static voidtmain(String[] args) tthrowstException {njava.io.PrintWri
 ter çıktı = tnew
 java.io.PrintWriter("temp.txt");o
 ut.printf("miktar %f %e\r", 32.32, 32.32);out.printf("miktar %5.4
 f %5.4e\r", 32.32, 32.32);out.printf("%6b\r", (1>2));

 out.printf("%6s\r", "Java");out.close();

 }
```

Nokta Kontrol Et

MyProgrammingLab™

**14.31** Bir **Scanner**'i bir dosyadan veri okumak için nasıl oluşturursunuz? Listing 14.14, ReadData.java'dathrows Exception tanımlamanın nedeni nedir? Listing 14.14'te **close()** metodu çağrılmazsa ne olur?

**14.32** Var olmayan bir dosya için bir **Scanner** oluşturmaya çalışırsanız ne olur? Var olan bir dosya için bir **PrintWriter** oluşturmaya çalışırsanız ne olur?

**14.33** Satır ayırıcı tüm platformlarda aynı mı? Windows'taki satır ayırıcı nedir?

**14.34** 45 57.8 789 girdiğinizi varsayalım, ardından **Enter** tuşuna basın. Aşağıdaki kod çalıştırıldıktan sonra değişkenlerin içeriğini gösterin.

```
Tarayıcı girişi = tnewtScanner(System.in);ninttintV
alue = input.nextInt();
doubletdoubleValue = input.nextDouble();nString li
ne = input.nextLine();
```

**14.35** 45 girdiğinizi, **Enter** tuşuna bastığınızı, 57.8 girdiğinizi, **Enter** tuşuna bastığınızı, 789 girdiğinizi ve **Enter** tuşuna bastığınızı varsayalım. Aşağıdaki kod çalıştırıldıktan sonra değişkenlerin içeriğini gösterin.

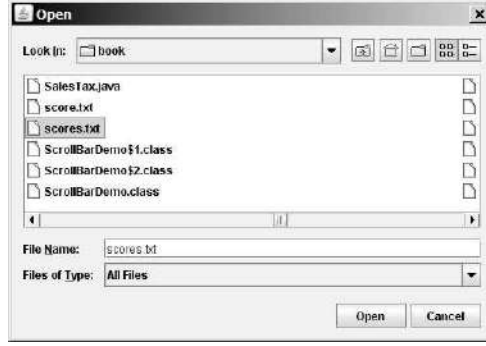
```
Tarayıcı girişi = tnewtScanner(System.in);ninttintV
alue = input.nextInt();
doubletdoubleValue = input.nextDouble();nString li
ne = input.nextLine();
```

## 14.12 Dosya Diyalogları

**JFileChooser**, bir dosya diyalogunu görüntülemek için bir GUI bileşenidir.

Java, bir dosya diyalogunu görüntülemek için **javax.swing.JFileChooser** sınıfını sağlar; bu, Şekil 14.10'da gösterilmiştir. Bu diyalog kutusundan kullanıcı bir dosya seçebilir.





FIGÜR 14.10 **JFileChooser** bir dosya açmak için bir dosya iletişim kutusu göstermek için kullanılabilir.

Liste 14.16, kullanıcıdan bir dosya seçmesini isteyen ve içeriğini konsolda görüntüleyen bir programı vermektedir.

#### LISTE 14.16 **ReadFileUsingJFileChooser.java**

```

1 import java.util.Scanner;
2 import javax.swing.JFileChooser;
3
4 public class ReadFileUsingJFileChooser {
5 public static void main(String[] args) throws Exception {
6 JFileChooser fileChooser = new JFileChooser();
7 if (fileChooser.showOpenDialog(null) == JFileChooser.APPROVE_OPTION) {
8 // Seçilen dosyayı al
9 java.io.File file = fileChooser.getSelectedFile();
10
11 // Dosya için bir Scanner oluştur
12 Scanner input = new Scanner(file);
13
14 // Dosyadan metin oku
15 while (input.hasNext()) {
16 System.out.println(input.nextLine());
17 }
18
19 // Dosyayı kapat
20 input.close();
21 }
22 başka{
23 System.out.println("Hiçbir dosya seçilmedi");
24 }
25 }

```

asla {revize e  
dilmeyi}

bir JFileChooser yaratd  
osya seçici gösterd  
urumu kontrol et  
getSelectedFile

showOpenDialog

APPROVE\_OPTION

getSelectedFile

Program, 6. satırda bir **JFileChooser** oluşturur. **showOpenDialog(null)** metodu, Şekil 14.10'da gösterildiği gibi bir diyalog kutusu açar. Metod, **Open** butonunun veya **Cancel** butonunun tıklanıp tıklanmadığını belirten bir **int** değeri döndürür, ya **APPROVE\_OPTION** ya da **CANCEL\_OPTION**.

**getSelectedFile()** metodu (10. satır) dosya diyalog kutusundan seçilen dosyayı döndürür. 13. satır dosya için bir tarayıcı oluşturur. Program, dosyadan satırları sürekli okur ve bunları konsola gösterir (16-18. satırlar).

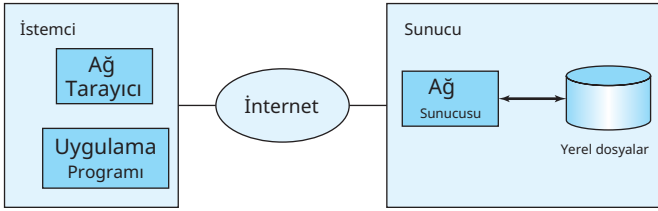
**14.36** Bir Dosya Açma iletişim kutusu nasıl oluşturulur? çağrıldığında ne döner **getSelectedFile()** bir **JFileChooser** nesnesi üzerinde?

## 14.13 Web'den Veri Okuma

Bilgisayarınızdaki bir dosyadan veri okuyabileceğiniz gibi, Web'deki bir dosyadan da veri okuyabilirsiniz.



Bir bilgisayar veya dosya sunucusundaki yerel bir dosyadan veri okumanın yanı sıra, dosyanın URL'sini (Uniform Resource Locator—Web'deki bir dosya için benzersiz adres) biliyorsanız, Web'deki bir dosyadan da verilere erişebilirsiniz. Örneğin, [www.google.com/index.html](http://www.google.com/index.html), Google Web sunucusunda bulunan **dex.html** dosyasının URL'sidir. URL'yi bir Web tarayıcısına girdiğinizde, Web sunucusu verileri tarayıcınıza gönderir ve tarayıcı verileri grafiksel olarak görüntüler. Şekil 14.11, bu sürecin nasıl çalıştığını göstermektedir.



**FIGİN 14.11** İstemci, bir Web sunucusundan dosyaları alır.

Bir uygulama programının bir URL'den veri okuyabilmesi için önce [java.net.URL](http://java.net/URL) sınıfını kullanarak bir [URL](http://java.net/URL) nesnesi oluşturmanız gerekir. Bu yapıyı ile:

```
public URL(Dize spesifikasyonu) throws MalformedURLException
```

Örneğin, aşağıdaki ifade <http://www.google.com/index.html> için bir URL nesnesi oluşturur.

`.com/index.html` için bir URL nesnesi oluşturur.

```
1 denemek
2 URL url = new URL("http://www.google.com/index.html");
3
4 try {
5 catch (MalformedURLException ex) {
6 ex.printStackTrace();
7 }
8 }
```

Bir **MalformedURLException**, URL dizisinde bir sözdizimi hatası varsa fırlatılır. Örneğin, `"http://www.google.com/index.html"` URL dizisi, iki eğik çizgi (`//`) gerektiğinden bir **MalformedURLException** çalışma zamanı hatasına neden olur. URL sınıfının geçerli bir URL'yi tanıyabilmesi için `http://` ön ekinin gerekli olduğunu unutmayın. Aşağıdaki kodla 2. satırı değiştirirseniz yanlış olur:

```
URL url = new URL("http://www.google.com/index.html");
```

Bir [URL](http://java.net/URL) nesnesi oluşturulduktan sonra, [URL](http://java.net/URL)

sınıfında tanımlanan `openStream()` yöntemini kullanarak bir giriş akışı açabilir ve bu akışı bir `Scanner` nesnesi oluşturmak için kullanabilirsiniz:

```
Scanner giriş = new Scanner(url.openStream());
```

Artık giriş akışından verileri, yerel bir dosyadan okur gibi okuyabilirsiniz. 14.17 numaralı listedeki örnek, kullanıcıdan bir URL girmesini ister ve dosyanın boyutunu gösterir.

### LİSTELERİ 14.17 ReadFileFromURL.java

```
1 import java.util.Scanner;
2
```

## 552 Bölüm 14 İstisna Yönetimi ve Metin G/Ç

```
3 genel classtReadFileFromURL {
4 public static voidtmain(String[] args) {
5 System.out.print("Bir URL girin: ");
6 String URLString = tnewtScanner(System.in).next();
7
8 denemek
9 newtjava.net.URL(URLString);
10 inttcount = t0t;
11 irişi = tnewtScanner(url.openStream());njava.net.URL url = t
12 while(girdi.hasNext()) {
13 Dize satırı = input.nextLine();nScanner g
14 sayım += satır.uzunluk();n15 }
16
17 System.out.println("Dosya boyutu "+ count + " bayt");
18 }
19 catcht(java.net.MalformedURLException ex) {
20 System.out.println("Geçersiz URL");
21 }
22 catcht(java.io.IOException ex) {
23 System.out.println("G/Ç Hataları: böyle bir dosya yok");
24 }
25 }n26
}
```

bir URL girin

bir URL nesnesi oluşturun

bir Scanner nesnesi oluşturun

okuyacak daha fazla şey var mı?

bir satır oku

Hatalı biçimlendirilmişURLException

IO İstisnası



Bir URL girin: HTTP://şu anda.Armstrong.amount/two/data/Lincoln.txt  
Dosya boyutu 1469 bayttır

Enter



Bir URL girin: HTTP://wuwuwu.Yahoo.com  
Dosya boyutu 190006 bayttır

Enter

Hatalı biçimlendirilmişURLException

Program, kullanıcının bir URL dizesi girmesini ister (satır 6) ve bir **URL** nesnesi oluşturur (satır 9).  
**Yapıcı, URL**

doğru bir şekilde oluşturulmadıysa bir `java.net.MalformedURLException` (satır 19) fırlatacaktır.

Program, URL için giriş akışından bir **Scanner** nesnesi oluşturur (satır 11). EğerURL doğru bir şekilde oluşturulmuş ancak mevcut değilse, bir **IOException** fırlatılacaktır (satır 22). Örneğin, <http://google.com/index1.html> uygun formu kullanır, ancak URL'nin kendisi mevcut değildir. Bu URL bu program için kullanılırsa bir **IOException** fırlatılacaktır.

**14.37** URL'den metin okumak için bir **Scanner** nesnesi nasıl oluşturursunuz?

Nokta Kontrol Et

MyProgrammingLab™

### Anahtar Terimler

mutlak dosya adı 541  
zincirlenmiş istisna 537  
kontrol edilen istisna 525  
istisna bildir 526  
dizin yolu 541

istisna 518  
istisna yayılması 527  
nispi dosya adı 541  
istisna fırlat 526  
kontrol edilmeyen istisna 525



## BÖLÜM ÖZETİ

1. İstisna yönetimi, bir yöntemin çağrısına bir istisna fırlatmasına olanak tanır.
2. Bir Java *istisnası*, `java.lang.Throwable` sınıfından türetilmiş bir sınıfın örneğidir. Java, `Error`, `Exception`, `RuntimeException`, `ClassNotFoundException`, `NullPointerException`, `ArithmeticException` gibi birçok önceden tanımlanmış istisna sınıfı sağlar. Ayrıca, `Exception` sınıfını genişleterek kendi istisna sınıfınız da tanımlayabilirsiniz.
3. İstisnalar, bir metodun yürütülmesi sırasında meydana gelir. `RuntimeException` ve `Error` *denetimsiz istisnalardır*, diğer tüm istisnalar ise *denetimli*.
4. Bir metodu *tanımlarken*, eğer metod bir istisna fırlatabilir ise, denetimli bir istisnayı tanımlamanız gerekir , böylece derleyiciye neyin yanlış gidebileceğini belirtmiş olursunuz.
5. Bir istisna tanımlamak için anahtar kelime `throws`, bir istisna fırlatmak için ise `throw` anahtar kelimesidir.
6. Denetimli istisnaları tanımlayan metodu çağırmak için, onu bir `try` ifadesi içinde yerleştirin. Bir metodun yürütülmesi sırasında bir istisna meydana geldiğinde, `catch` bloğu istisnayı yakalar ve işler.
7. Eğer bir istisna mevcut methodda yakalanmazsa, çağırana tarafa iletilir. Süreç, istisna yakalanana veya `main` metoduna iletilene kadar tekrarlanır.
8. Çeşitli istisna sınıfları, ortak bir üst sınıftan türetilir. Eğer bir `catch` bloğu, bir üst sınıfın istisna nesnelerini yakalarsa, o üst sınıfın alt sınıflarının tüm istisna nesnelerini de yakalayabilir.
9. Bir `catch` bloğunda istisnaların belirtilme sırası önemlidir. Eğer bir istisna nesnesini, o nesnenin üst sınıfının istisna nesnesinden sonra belirtirseniz, bir derlemehatası oluşur.
10. Bir yöntem içinde bir istisna meydana geldiğinde, eğer istisna yakalanmazsa yöntem hemen çıkar. Eğer yöntem çıkmadan önce bazı görevleri yerine getirmesi gerekiyorsa, istisnayı yöntemde yakalayabilir ve ardından çağırana yeniden fırlatabilirsiniz.
11. `finally` bloğundaki kod, `try` bloğunda bir istisna meydana gelip gelmediğinden veya bir istisna yakalanıp yakalanmadığına bakılmaksızın her durumda çalıştırılır.
12. İstisna yönetimi, hata işleme kodunu normal programlama görevlerinden ayırarak, programların okunmasını ve değiştirilmesini kolaylaştırır.
13. İstisna yönetimi basit testlerin yerini almak için kullanılmamalıdır. Mümkün olduğunda `if` ifadeleri kullanarak basit testler yapmalı ve istisna yönetimini `if` ifadeleriyle başa çıkılamayan durumlar için ayırmalısınız.
14. `File` sınıfı, dosya özelliklerini elde etmek ve dosyaları manipüle etmek için kullanılır. Bir dosya oluşturma veya bir dosyadan veri okuma/yazma yöntemlerini içermez.

- 15.** Bir metin dosyasından dize ve ilkel veri değerlerini okumak için **Scanner** kullanabilirsiniz ve veri yazmak için bir dosya oluşturmak ve metin dosyasına yazmak için **PrintWriter** kullanabilirsiniz.
- 16.** **JFileChooser** sınıfı, dosya seçimi için dosya diyalogları görüntülemek için kullanılabilir.
- 17.** Web'deki bir dosyadan okumak için **URL** sınıfını kullanabilirsiniz.

## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

MyProgrammingLab™

## PROGRAMLAMA EGZERSİZLERİ

### Bölümler 14.2-14.9

- \* 14.1** (**NumberFormatException**) Liste 9.5, **Calculator.java**, basit bir komut-satırı hesaplayıcısıdır. Herhangi bir operand sayısal değilse programın sona erdiğini unutmayın. Sayısal olmayanoperand'larla başa çıkan bir istisna işleyicisi olan bir program yazın; ardından aynı amacı başarmak için bir istisna işleyicisi kullanmadan başka bir program yazın. Programınız çıkmadan önce kullanıcıya yanlış operand türü hakkında bilgi veren bir mesaj göstermelidir (bkz. Şekil 14.12).



FIGURE 14.12 Program aritmetik işlemler yapar ve girdi hatalarını tespit eder.

- \* 14.2** (**InputMismatchException**) Kullanıcıdan iki tam sayı okumasını isteyen ve toplamını gösteren bir program yazın. Girdi yanlışsa programınız kullanıcıdan sayıyı tekrar okumasını istemelidir.
- \* 14.3** (**ArrayIndexOutOfBoundsException**) Aşağıdaki gereksinimleri karşılayan bir program yazın:
- 100 rastgele seçilmiş tam sayıdan oluşan bir dizi oluşturun.
  - Kullanıcıdan dizinin indeksini girmesini ister, ardından ilgili eleman değerini gösterir. Belirtilen indeks sınırların dışındaysa, mesajı gösterir **Sınırların Dışında**.
- \* 14.4** (**IllegalArgumentException**) 10.2 numaralı listedeki **Loan** sınıfını değiştirin vekredi miktarı, faiz oranı veya yıl sayısı sıfır veya daha az olduğunda **IllegalArgumentException** fırlatın.
- \* 14.5** (**IllegalTriangleException**) Programlama Egzersizi 11.1, üç kenarlı **Triangle** sınıfını tanımlar. Bir üçgende, herhangi iki kenarın toplamıöteki kenardan büyüktür. **Triangle** sınıfı bu kurala uymalıdır. **IllegalTriangleException** sınıfını oluşturun ve

Triangle sınıfının yapıcısını, kuralı ihlal eden kenarlarla bir üçgen oluşturulursa IllegalTriangleException nesnesi fırlatacak şekilde değiştirin, şöyle:

```
/** Belirtilen kenarlarla bir üçgen oluştur */
public Triangle(double taraf1, double taraf2, double taraf3) throws IllegalTriangleException {
 // Uygula
}
```

- \*14.6 (NumberFormatException) 9.2 numaralı liste, `hexToDecimal(String hexString)` yöntemini uygulayın; bu yöntem bir hex dizesini ondalık bir sayıya dönüştürür. `hexToDecimal` yöntemini, dize bir hex dizesi değilse

`NumberFormatException` fırlatacak şekilde uygulayın.

- \*14.7 (NumberFormatException) Programlama Egzersizi 9.8, `binaryToDecimal(String binaryString)` metodunu belirtir; bu metod, bir ikili dizeyi ondalık bir sayıya dönüştürür. `binaryToDecimal` metodunu uygulayın ve eğer dize bir ikili dize değilse bir `NumberFormatException` fırlatın.

- \*14.8 (HexFormatException) Alıştırma 14.6, eğer dize bir hex dizesi değilse `NumberFormatException` fırlatacak şekilde `hexToDecimal` yöntemini uygulayın. `HexFormatException` adında özel bir istisna tanımlayın. Eğer dize bir hex dizesi değilse `HexFormatException` fırlatacak şekilde `hexToDecimal` yöntemini uygulayın.



VideoNotu

HexFormatException

- \*14.9 (BinaryFormatException) Alıştırma 14.7, eğer dize bir ikili dize değilse `BinaryFormatException` fırlatacak şekilde `binaryToDecimal` yöntemini uygulayın. `BinaryFormatException` adında özel bir istisna tanımlayın. Eğer dize bir ikili dize değilse `BinaryFormatException` fırlatacak şekilde `binaryToDecimal` yöntemini uygulayın.

- \*14.10 (OutOfMemoryError) JVM'nin bir

`OutOfMemoryError` fırlatmasına neden olan bir program yazın ve bu hatayı yakalayıp işleyin.

## Bölümler 14.10–14.12

- \*\*14.11 (Remove text) Belirtilen bir dizeyi bir metin dosyasından tüm geçişlerini kaldıran bir program yazın. Örneğin,

```
java Alıştırma14_11 John dosya adı
```

belirtilen dosyadan `John` dizesini kaldırır. Programınız komut satırından argümanları almalıdır.

- \*\*14.12 (Reformat Java source code) Java kaynak kodunu bir sonraki satır süslemesi stilinde n satır sonu süslemesi stiline dönüştüren bir program yazın. Örneğin, (a) aşağıdaki Java kaynağı bir sonraki satır süslemesi stilini kullanır. Programınız bunu (b) satır sonu süslemesi stiline dönüştürür.

```
genel sınıfTestn{

 public static void tmain(String[] args)n{

 // Bazı ifadeler
 }
}
```

(a) Sonraki satır süslü parantez stili

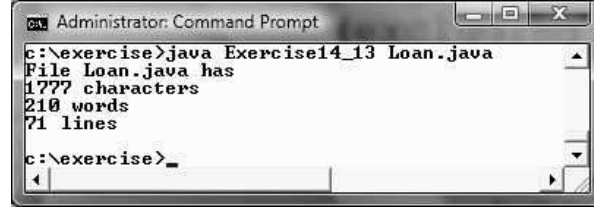
```
genel sınıfTest {
 public static void main(String[] args) {
 // Bazı ifadeler
 }
}
```

(b) Satır sonu süslü parantez stili

Programınızı, Java kaynak kodu dosyasını argüman olarak alarak komut satırında n çalıştırabilirsiniz. Bu, Java kaynak kodunu yeni bir formata dönüştürür. Örneğin , aşağıdaki komut, Java kaynak kodu dosyasını **Test.javadosyasını** satır sonu süslü parantez stiline dönüştürür.

```
java Alıştırması14_12 Test.java
```

- \*14.13** (Bir dosyadaki karakterleri, kelimeleri ve satırları sayma) Bir dosyadaki karakterlerin, kelimelerin ve satırların sayısını sayacak bir program yazın. Kelimeler, boşluk karakterleriyle ayrılır. Dosya adı, Şekil 14.13'te gösterildiği gibi bir komut satırı argümanı olarak geçilmelidir.



**FIGÜRİK14.13** Program, verilen dosyadaki karakter, kelime ve satır sayısını görüntüler.

- \*14.14** (Bir metin dosyasında puanları işleme) Bir metin dosyasının belirsiz sayıda boşlukla ayrılmış puan içerdiğini varsayalım. Kullanıcıdan dosyayı girmesini isteyen, dosyada n puanları okuyan ve toplamalarını ve ortalamasını görüntüleyen bir program yazın. Puanlar boşluklarla ayrılmıştır.
- \*14.15** (Veri yazma/okuma) Var olmayan bir dosya oluşturmak için **Exercise14\_15.txt** adında bir dosya oluşturacak bir program yazın. Dosyaya metin I/O kullanarak rastgele oluşturulmuş **100** tamsayı yazın. Tamsayılar dosyada boşluklarla ayrılmıştır. Verileri dosyadan geri okuyun ve sıralı verileri görüntüleyin.
- \*\*14.16** (Metni değiştir) Liste 14.15, **ReplaceText.java**, bir kaynak dosyadaki metni değiştiren ve değişikliği yeni bir dosyaya kaydeden bir programı verir. Programı orijinal dosyaya değişikliği kaydedecek şekilde gözden geçirin. Örneğin, çağırarak

```
java Exercise14_16 file eskiString yeniString
```

**eskiString** kaynak dosyadaki **yeniString** ile değiştirir.

- \*\*\*14.17** (Oyun: adam asmaca) Egzersiz 9.25'i yeniden yazın. Program, **hangman.txt** adlı bir metin dosyasında saklanan kelimeleri okur. Kelimeler boşluklarla ayrılmıştır.
- \*\*14.18** (Paket ifadesi ekle) Java kaynak dosyalarınızın **chapter1**, **chapter2**, ..., **chapter34** altındaki dizinlerde olduğunu varsayalım. Her Java kaynak dosyası için ilk satır olarak **package chapteri;** ifadesini ekleyen bir program yazın. **chapteri** dizini altında. **chapter1**, **chapter2**, ..., **chapter34** kök dizin **srcRootDirectory** altında olsun. Kök dizini ve **chapteri** dizini diğer klasörler ve dosyalar içerebilir. Programı çalıştırmak için şu komutu kullanın:

```
java Egzersiz14_18 srcRootDirectory
```

- \*14.19** (Kelimeleri say) Başkan Abraham Lincoln'ün Gettysburg konuşmasındaki kelime sayısını <http://cs.armstrong.edu/liang/>

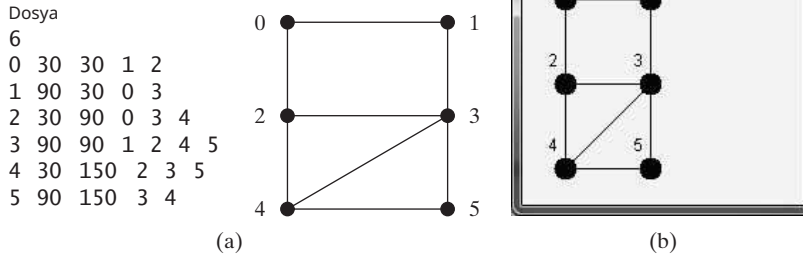
<data/Lincoln.txt> adresinden sayan bir program yazın.

**\*\*14.20** (Paket ifadesini kaldır) Java kaynak dosyalarınızın `chapter1,chapter2`, ..., `chapter34` dizinleri altında olduğunu varsayalım. Her Java kaynak dosyasının ilk satırında yer alan `package chapteri;` ifadesini kaldırmak için bir program yazın. `chapter1,chapter2`, ..., `chapter34` dizinlerinin kök dizin `srcRootDirectory` altında olduğunu varsayalım. Kök dizin ve `chapteri` dizini başka klasörler ve dosyalar içerebilir. Programı çalıştırmak için aşağıdaki komutu kullanın:

ğidaki komutu kullanın:

```
java Egzersiz14_20 srcRootDirectory
```

**\*\*14.21** (Bir grafik göster) Bir grafik, köşeler ve köşeleri bağlayan kenarlardan oluşur. Bir dosyadan bir grafik okuyan ve bunu bir panelde gösteren bir program yazın. Dosyadaki ilk satır, köşe sayısını belirten bir sayı (`n`) içerir. Köşeler `0,1,...,n-1` olarak etiketlenmiştir. Her bir sonraki satır, `u x y v1 v2 ...` formatında, köşe `u`'nın (`x,y`) konumunda olduğunu ve (`u,v1`), (`u,v2`) gibi kenarları tanımlar. Şekil 14.14a, bir grafik için dosyanın bir örneğini verir. Programınız, kullanıcıdan dosya adını girmesini ister, dosyadan verileri okur ve grafiği bir panelde gösterir; bu, Şekil 14.14b'de gösterilmiştir. Web URL'sinden veri okuyan başka bir program yazın, örneğin <http://cs.armstrong.edu/liang/data/graph.txt>. Bu program, kullanıcıdan dosyanın URL'sini girmesini istemelidir.



**FiÇERİK14.14** Alıştırma 14.21, graf hakkında bilgileri okur ve görsel olarak gösterir.

**\*\*14.22** (Metni değiştir) Alıştırma 14.16'yı, belirtilen dizindeki tüm dosyalar için bir dosyadaki bir dizeyi yeni bir dize ile değiştirecek şekilde revize edin:

```
java Alıştırma14_22 dir oldString newString
```

**\*\*14.23** (Web'deki bir metin dosyasında puanları işleyin) Web'deki metin dosyasının <http://cs.armstrong.edu/liang/data/Scores.txt> belirtilmemiş sayıda puan içerdiğini varsayın. Puanları dosyadan okuyan ve toplamlarını ve ortalamasını gösteren bir program yazın. Puanlar boşluklarla ayrılmıştır.

**\*14.24** (Büyük veri seti oluşturun) 1.000 satırlık bir veri dosyası oluşturun. Dosyadaki her satır, bir öğretim üyesinin adı, soyadı, unvanı ve maaşını içerir. Öğretim üyesinin adı ve soyadı, `i.` satır için `FirstNamei` ve `LastNamei` olarak belirlenmiştir. Unvan rastgele olarak yardımcı, doçent ve profesör olarak üretilecektir. Maaş, ondalık noktadan sonra iki basamaklı bir sayı olarak rastgele üretilecektir. Yardımcı doçent için maaş 50.000 ile

80,000, yardımcı doçent için 60,000 ile 110,000 arasında, ve tam doçent için 75,000 ile 130,000 arasında. Dosyayı **Salary.txt** olarak kaydedin. İşte bazı örnek veriler:

İlkAd1 Soyad1 asistan 60055.95

İlkAd2 Soyad2 ortak 81112.45

. . .

İlkAd1000 Soyad1000 tam 92255.21

**\* 14.25** (*Büyük veri kümesini işleme*) Bir üniversite çalışanlarının maaşlarını yayınlar <http://cs.armstrong.edu/liang/data/Salary.txt>. Dosyadaki her satır, bir öğretim üyesinin adını, soyadını, unvanını ve maaşını içermektedir (bkz. Egzersiz 14.24). Yardımcı profesörler, doçentler, profesörler ve tüm fakülte için toplam maaşı görüntüleyen bir program yazın ve ayrıca yardımcı profesörler, doçentler, profesörler ve tüm fakülte için ortalama maaşı görüntüleyin.

**\*\* 14.26** (*Bir dizin oluşturun*) Kullanıcıdan bir dizin adı girmesini isteyen ve **File**'in **mkdirs** yöntemini kullanarak bir dizin oluşturan bir program yazın. Program, bir dizin oluşturulursa "Dizin başarıyla oluşturuldu" mesajını, dizin zaten mevcutsa "Dizin zaten mevcut" mesajını görüntüler.

**\*\* 14.27** (*Kelime değiştir*) Bir dizinde kelimeler içeren birçok dosyanız olduğunu varsayalım **Exercise\_j**, burada *i* ve *j* rakamlardır. Eğer *i* tek haneli ise *i*'ye 0 ekleyen ve *j* tek haneli ise *j*'ye 0 ekleyen bir program yazın. Örneğin, bir dosyada bulunan **Exercise2\_1**, **Exercise02\_01** ile değiştirilir. Java'da, komut satırından **\*** sembolünü geçtiğinizde, bu dizindeki tüm dosyaları ifade eder (bkz. Ek III.AC). Programınızı çalıştırmak için aşağıdaki komutu kullanın.

```
java Egzersizi14_27 *
```

**\*\* 14.28** (*Dosyaları yeniden adlandırma*) **Exercise\_i** adında bir dizinde çok sayıda dosyanız olduğunu varsayalım, burada *i* ve *j* rakamlardır. Eğer *i* tek haneli ise, önüne 0 ekleyen bir program yazın. Örneğin, bir dizinde bulunan **Exercise2\_1** adlı bir dosya, **Exercise02\_1** olarak yeniden adlandırılacaktır. Java'da, komut satırından **\*** sembolünü geçtiğinizde, dizindeki tüm dosyaları ifade eder (bkz. Ek III.AC). Programınızı çalıştırmak için aşağıdaki komutu kullanın.

```
java Egzersizi14_28 *
```

**\*\* 14.29** (*Dosyaları yeniden adlandırma*) **Exercise\_i** adında bir dizinde çok sayıda dosyanız olduğunu varsayalım, burada *i* ve *j* rakamlardır. Eğer *j* tek haneli ise, önüne 0 ekleyen bir program yazın. Örneğin, bir dizinde bulunan **Exercise2\_1** adlı bir dosya, **Exercise2\_01** olarak yeniden adlandırılacaktır. Java'da, komut satırından **\*** sembolünü geçtiğinizde, dizindeki tüm dosyaları ifade eder (bkz. Ek III.AC). Programınızı çalıştırmak için aşağıdaki komutu kullanın.

```
java Egzersizi14_29 *
```

# AÖZ SINIFLAR VE INTERFACES

## Hedefler

- Soyut sınıfları tasarlamak ve kullanmak (§15.2).
- Soyut **Number** sınıfını kullanarak sayısal sarmalayıcı sınıfları, **BigInteger** ve **BigDecimal** genelleştirmek (§15.3).
- Takvim işlemleri için **Calendar** ve **GregorianCalendar** sınıflarını kullanmak (§15.4).
- Nesneler için ortak davranışları tanımlamak amacıyla arayüzleri kullanmak (§15.5).
- Arayüzleri tanımlamak ve arayüzleri uygulayan sınıfları tanımlamak (§15.5).
- **Comparable** arayüzünü kullanarak doğal bir sıralama tanımlamak (§15.6).
- **Cloneable** arayüzünü kullanarak nesneleri kopyalanabilir hale getirmek (§15.7).
- Somut sınıflar,
  - soyut sınıflar ve arayüzler arasındaki benzerlikleri ve farklılıkları keşfetmek (§15.8).
- Rasyonel sayıları işlemek için **Rational** sınıfını tasarlamak (§15.9).



## 15.1 Giriş



*Bir üst sınıf, ilgili alt sınıflar için ortak davranışları tanımlar. Bir arayüz, sınıflar (ilişkili sınıflar dahil) için ortak davranışları tanımlamak amacıyla kullanılabilir.*

problem  
arayüzü

Basit programlar yazmayı ve GUI bileşenlerini oluşturup görüntülemeyi öğrendiniz. Kullanıcı eylemlerine, örneğin bir düğmeye tıklayarak bir eylemi gerçekleştirmek için kod yazabilir misiniz?

Bu tür bir kod yazmak için arayüzler hakkında bilgi sahibi olmalısınız. Bir arayüz, sınıflar (ilişkili sınıflar dahil) için ortak davranış tanımlamak içindir. Arayüzleri tartışmadan önce, yakından ilgili bir konuyu tanıtıyoruz: soyut sınıflar.

## 15.2 Soyut Sınıflar



*Bir soyut sınıf, nesne oluşturmak için kullanılamaz. Bir soyut sınıf, somut alt sınıflarda uygulanan soyut yöntemler içerebilir.*



VideoNotu

Soyut GeometrikNesne  
sınıfı

soyut sınıf

Kalıtım hiyerarşisinde, sınıflar her yeni alt sınıf ile daha spesifik ve somut hale gelir. Eğer bir alt sınıftan üst sınıfa geri dönerseniz, sınıflar daha genel ve daha az spesifik hale gelir. Sınıf tasarımı, bir üst sınıfın alt sınıflarının ortak özelliklerini içermesini sağlamalıdır. Bazen bir üst sınıf o kadar soyuttur ki, herhangi bir spesifik örneği olamaz. Böyle bir sınıfa soyut sınıf denir.

Bölüm 11'de, GeometricObject, Circle ve Rectangle için üst sınıf olarak tanımlanmıştır. GeometricObject, geometrik nesnelerin ortak özelliklerini modellemektedir. Hem Circle hem de Rectangle, bir dairenin ve bir dikdörtgenin alanını ve çevresini hesaplamak için getArea() ve getPerimeter() yöntemlerini içerir. Tüm geometrik nesneler için alan ve çevre hesaplayabileceğinizden, getArea() ve getPerimeter() yöntemlerini GeometricObject sınıfında tanımlamak daha iyidir. Ancak, bu yöntemler GeometricObject sınıfında uygulanamaz, çünkü uygulamaları geometrik nesnenin spesifik türüne bağlıdır. Bu tür yöntemlere soyut yöntemler denir ve yöntem başlığında abstract belirteci ile gösterilir. GeometricObject'ta yöntemleri tanımladıktan sonra, soyut bir sınıf haline gelir. Soyut sınıflar, sınıf başlığında abstract belirteci ile gösterilir. UML grafik notasyonunda, soyut sınıfların ve soyut yöntemlerinin adları italik yazılır; bu, Şekil 15.1'de gösterilmiştir. Liste 15.1, yeni GeometricObject sınıfının kaynak kodunu vermektedir.

soyut yöntem

soyut değiştirici

### LISTING 15.1tGeometricObject.java

soyut sınıf

```

1 halk soyut classtGeometricObject {
2 privateString color = "beyaz";
3 özel boolean dolu;
4 Privatetjava.util.Tarih tarihiOluşturuldu;
5
6 /** Varsayılan bir geometrik nesne oluştur */
7 protectedtGeometricObject() {
8 dateCreated = tnewwtjava.util.Date();n9 }
9
10
11 /** Renk ve dolu değeri ile bir geometrik nesne oluştur */
12 protectedtGeometricObject(Dize rengi, tbooleanfilled) {
13 dateCreated = tnewwtjava.util.Date();
14 thist.renk = renk;
15 thist.doldurulmuş = doldurulmuş;
16 }
17
18 /** Renk döndür */
19 publictString getColor() {
20 dönüş rengi;

```



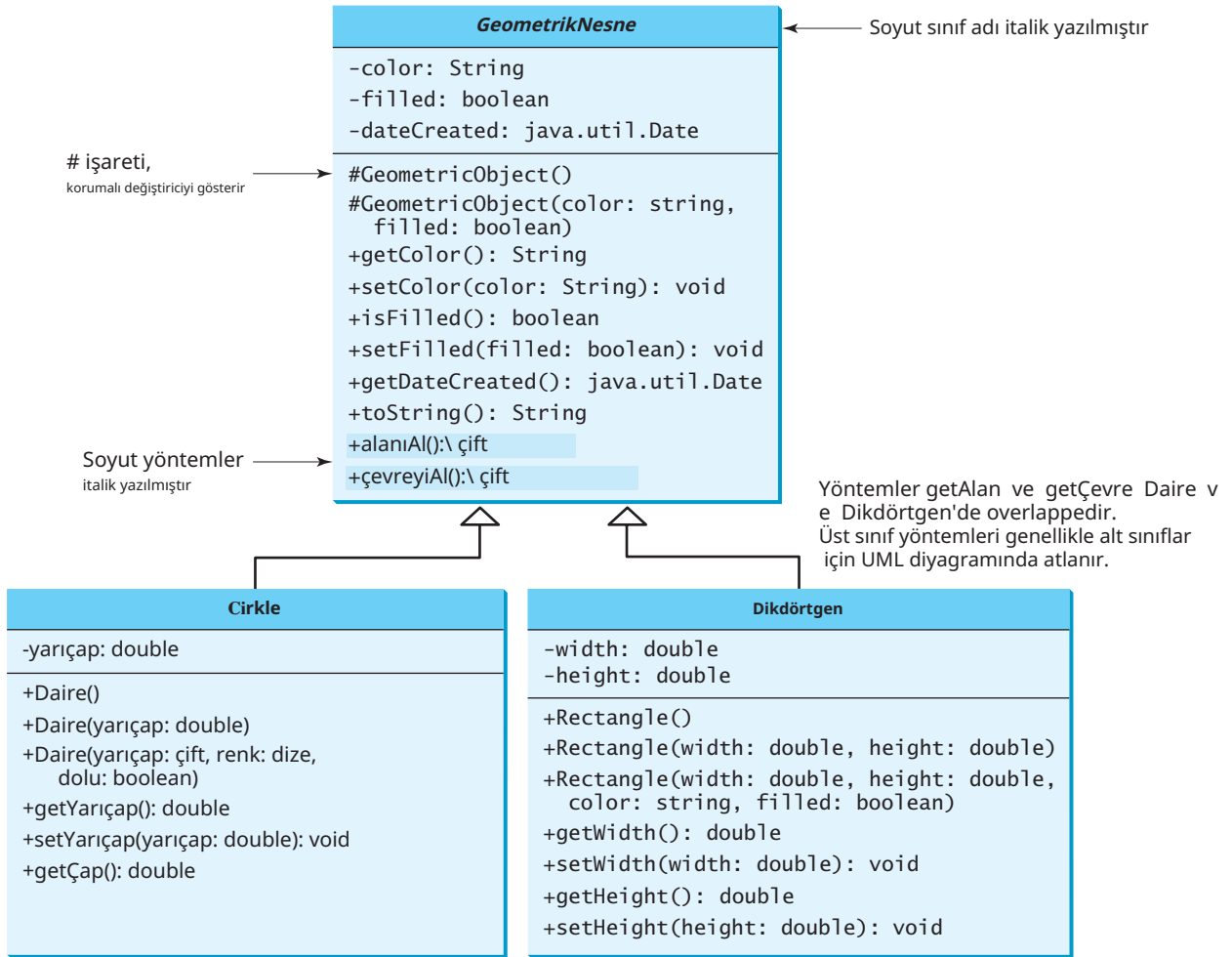


FIG RESİM 15.1 Yeni **GeometrikNesne** sınıfı soyut yöntemler içerir.

```

yirmi bir }
2223
 /** Yeni bir renk ayarla */
24 public voidtsetColor(Dize rengi) {
25 thist.renk = renk;
26 }
2728
 /** Doldurulmuşu döndür. Doldurulmuş boolean olduğundan,
29 * get metodu isFilled olarak adlandırılır */
30 genel booleantisFilled() {
31 return dolu;
32 }
3334
 /** Yeni bir doldurulmuş ayarla */
35 genel voidtsetFilled(tbooleanfilled) {
36 thist.doldurulmuş = doldurulmuş;
37 }
3839
 /** dateCreated'i al */
40 publictjava.util.Date getDateCreated() {

```

## 562 Bölüm 15 Soyut Sınıflar ve Arayüzler

```
41 return oluşturulmaTarihi;
42 }
43
44 @Geçersiz Kıl
45 public String toString() {
46 return "oluşturulma tarihi " + oluşturulmaTarihi + "renk: " + renk +
47 " ve dolduruldu: " + dolduruldu;
48 }
49
50 /** Soyut yöntem getArea */
51 public abstract double getAlan();
52
53 /** Soyut yöntem getPerimeter */
54 public abstract double getCevre();
55 }
```

soyut yöntem

soyut yöntem

Soyut sınıflar, normal sınıflara benzer, ancak soyut sınıfların örneklerini oluşturamazsınız **new** operatörünü kullanarak. Soyut bir yöntem, uygulama olmadan tanımlanır. Uygulaması, sınıflar tarafından sağlanır. Soyut yöntemler içeren bir sınıf, soyut olarak tanımlanmalıdır.

neden korumalı yapıcı?

Soyut sınıftaki yapıcı, yalnızca alt sınıflar tarafından kullanıldığı için korumalı (protected) olarak tanımlanır. Somut bir alt sınıfın örneğini oluşturduğunuzda, üst sınıfın yapıcısı, üst sınıfta tanımlanan veri alanlarını başlatmak için çağrılır.

Geometrik nesneler için ortak özellikleri (veri ve yöntemler) tanımlayan **GeometricObject** soyut sınıfı, yeterli yapıcılar sağlar. Geometrik nesnelerin alanlarını ve çevrelerini nasıl hesaplayacağınızı bilmediğiniz için, **getArea** ve **getPerimeter** soyut yöntemler olarak tanımlanır. Bu yöntemler alt sınıflarda uygulanır. **Circle** ve **Rectangle** uygulaması, bu bölümde tanımlanan **GeometricObject** sınıfını ge- nişletmek dışında, Listings 15.2 ve 15.3'tekiyle aynıdır.

uygulamaDaire

uygulamaDikdörtgen

### Listing 15.2 Circle.java

Özeti genişletir  
GeometrikNesne

```
1 kamu sınıfı Circle extends GeometrikNesne {
2 // Aynı Listing 11.2'deki 3-48. satırlar gibi, bu yüzden atlandı
3 }
```

### Listing 15.3 Rectangle.java

Özeti genişletir  
GeometrikNesne

```
1 kamu sınıfı Rectangle extends GeometrikNesne {
2 // Aynı Listing 11.3'teki 3-51. satırlar gibi, bu yüzden atlandı
3 }
```

## 15.2.1 Neden Soyut Metotlar?

Metotları **getArea** ve **getPerimeter** **GeometricObject** sınıfında soyut olarak tanımlamanın ne gibi bir avantaj sağladığını merak ediyor olabilirsiniz. Listing 15.4'teki örnek, bunları **GeometricObject** sınıfında tanımlamanın faydalarını göstermektedir. Program, bir daire ve bir dikdörtgen olmak üzere iki geometrik nesne oluşturur, eşit alanlara sahip olup olmadıklarını kontrol etmek için **equals** metodunu çağırır ve onları görüntülemek için **displayGeometricObject** metodunu çağırır.

### Listing 15.4 TestGeometricObject.java

bir daire oluştur

bir dikdörtgen oluştur

```
1 genel sınıf TestGeometricObject {
2 /** Ana yöntem */
3 public static void main(String[] args) {
4 // İki geometrik nesne oluştur
5 GeometrikNesne geoNesne1 = new Daire(5);
6 GeometrikNesne geoNesne2 = new Dikdörtgen(5, 3);
```

```

7
8 System.out.println("İki nesne aynı alana mı sahip? "+
9 equalArea(geoNesne1, geoNesne2));
10
11 // Daireyi göster
12 displayGeometricObject(geoObject1);
13
14 // Dikdörtgeni göster
15 displayGeometricObject(geoObject2);n16 }

17
18 /** İki geometrik nesnenin alanlarını karşılaştırmak için bir yöntem */
19 genel statik boolean equalArea(GeometricObject nesne1, eşitAlan
20 GeometricObject nesne2) {
21 return nesne1.getArea() == nesne2.getArea();
22 }
23
24 /** Bir geometrik nesneyi göstermek için bir yöntem */
25 genel statik boşluk geometrikNesneyiGoster(GeometricObject nesne) { geometrikNesneyiGoster
26 System.out.println();
27 System.out.println("Alan "+ nesne.getArea());
28 System.out.println("Çevre "+ nesne.getPerimeter());
29 }
30 }

```

İki nesne aynı alana mı sahip? false

Alan 78.53981633974483  
Çevre 31.41592653589793

Alan 15.0  
Çevre 16.0'dır



getArea() ve getPerimeter() yöntemleri GeometricObject sınıfında tanımlanmıştır Circle sınıfında ve Rectangle sınıfında geçersiz kılınmıştır. İfadeler (satırlar 5-6)

```

GeometricObject geoObject1 = new Circle(t5t);nGeom
etricObject geoObject2 = new Rectangle(t5t,t3t);

```

yeni bir daire ve dikdörtgen oluşturun ve bunları geoObject1 ve geoObject2 değişkenlerine atayın. Bu iki değişken GeometricObject türündedir.

equalArea(geoObject1, geoObject2) (satır 9) çağrıldığında, Circle sınıfında tanımlanan getArea() metodu object1.getArea() için kullanılır, çünkü geoObject1 bir dairedir, ve getArea() metodu Rectangle sınıfında tanımlanmıştır ve object2.getArea() için kullanılır, çünkü geoObject2 bir dikdörtgendir.

Benzer şekilde, displayGeometricObject(geoObject1) (satır 12) çağrıldığında, getArea() ve getPerimeter() yöntemleri Circle sınıfında kullanılır, ve displayGeometricObject(geoObject2) (satır 15) çağrıldığında, getArea ve getPerimeter yöntemleri Rectangle sınıfında kullanılır. JVM, bu yöntemlerden hangisinin çağrılacağını çalışma zamanında, yöntemi çağırana gerçek nesneye bağlı olarak dinamik olarak belirler.

Not edin ki, equalArea yöntemini, iki geo-metrik nesnenin aynı alana sahip olup olmadığını karşılaştırmak için tanımlayamazdınız eğer getArea yöntemi GeometricObject içinde tanımlanmamış olsaydı. Artık, GeometricObject içinde soyut yöntemlerin tanımlanmasının faydalarını gördünüz.

## 15.2.2 Soyut Sınıflar Hakkında İlginç Noktalar

Aşağıdaki soyut sınıflar hakkında dikkat edilmesi gereken noktalar:

soyut

sınıfta soyut yöntem

- Soyut bir yöntem, soyut olmayan bir sınıfta bulunamaz. Eğer bir soyut üst sınıfın alt sınıfı tüm soyut yöntemleri uygulamiyorsa, alt sınıf soyut olarak tanımlanmalıdır. Diğer bir deyişle, soyut bir sınıftan genişletilen soyut olmayan bir alt sınıf ta, tüm soyut yöntemler uygulanmalıdır. Ayrıca, soyut yöntemlerin statik olma dıđını unutmayın.

soyut sınıftan

örnek oluşturulamaz

- Soyut bir sınıf, **new** operatörü kullanılarak örneklendirilemez, ancak yine de alt sınıflarının yapıcılarında çağrılan yapıcılarını tanımlayabilirsiniz. Örneğın, **GeometricObject** yapıcıları **Circle** sınıfında ve **Rectangle** sınıfın da çağrılır.

soyut

yöntem olmadan soyut sınıf

- Soyut yöntemler içeren bir sınıf soyut olmalıdır. Ancak, herhangi bir soyut yöntem i çermeyen bir soyut sınıf tanımlamak mümkündür. Bu durumda, sınıfın örneklerin i **new** operatörü kullanarak oluşturamazsınız. Bu sınıf, alt sınıfları tanımlamak için bir temel sınıf olarak kullanılır.

soyut sınıfın üst sınıfı

somut olabilir

- Bir alt sınıf, üst sınıfı somut olsa bile soyut olabilir. Örneğın, **Object** sınıfı somuttur, ancak **GeometricObject** gibi alt sınıfları soyut olabilir.

somut yöntem geçersiz kılındı

soyut olmak için

- Bir alt sınıf, üst sınıfından bir yöntemi geçersiz kılabilir ve onu soyut olarak tanımlayabilir. Bu *çok alışılmadık*, ancak üst sınıftaki yöntemin alt sınıfta geçersiz hale geldiğı d urumlarda faydalıdır. Bu durumda, alt sınıf soyut olarak tanımlanmalıdır.

tip olarak soyut sınıf

- Soyut bir sınıftan **new** operatörünü kullanarak bir örnek oluşturamazsınız, ancak birsoyut sınıf veri türü olarak kullanılabilir. Bu nedenle, **GeometricObject** türünde elemanları olan bir dizi oluşturan aşağıdaki ifade doğrudur.

```
GeometricObject[] nesneleri = tnewtGeometricObject[t10t];
```

Daha sonra **GeometricObject** örneğı oluşturabilir ve referansını diziye şu şekilde atayabilirsiniz:

```
objects[0] = new Circle();
```

## 15.1 Aşağıdaki sınıflardan hangisi yasal bir soyut sınıf tanımlar?

MyProgrammingLab™

Nokta Kontrol Et

```
sınıfA {
 soyut voidbitmemiş() {
 }
}
```

(a)

```
genel sınıf soyutA {
 soyut voidbitmemiş();
}
```

(b)

```
sınıfA {
 soyut voidbitmemiş();
}
```

(c)

```
soyut sınıfA {
 korumalı voidbitmemiş();
}
```

(d)

```
soyut sınıfA {
 soyut voidbitmemiş();
}
```

(Ve)

```
soyut sınıfA {
 soyut intbitmemiş();
}
```

(f)

**15.2** `getArea` ve `getPerimeter` metodları `GeometricObject` sınıfından kaldırılabilir. `getArea` ve `getPerimeter`'i `GeometricObject` sınıfında soyut metodlar olarak tanımlamanın faydaları nelerdir?

**15.3** Doğru mu yanlış mı?

- Bir soyut sınıf, soyut sınıftan bir örnek oluşturmak için `new` operatörünü kullanamayacağınız dışında, bir soyut sınıf gibi kullanılabilir.
- Bir soyut sınıf genişletilebilir.
- Soyut olmayan bir üst sınıfın alt sınıfı soyut olamaz.
- Bir alt sınıf, bir üst sınıftaki somut bir yöntemi soyut olarak tanımlamak için geçersiz kılamaz.
- Soyut bir yöntem statik olmamalıdır.

## 15.3 Vaka Çalışması: Soyut Sayı Sınıfı

*Sayı, sayısal sarmalayıcı sınıflar için soyut bir üst sınıftır, `BigInteger` ve `BigDecimal`.*



Bölüm 10.12 sayısal sarmalayıcı sınıfları tanıttı ve Bölüm 10.14 `BigInteger` ve `BigDecimal` sınıflarını tanıttı. Bu sınıflar, bu sınıfların bir nesnesinden `byte`, `short`, `int`, `long`, `float` ve `double` değerini döndürmek için ortak yöntemlere sahiptir: `byteValue()`, `shortValue()`, `intValue()`, `longValue()`, `floatValue()` ve `doubleValue()`. Bu ortak yöntemler aslında `Sayı` sınıfında tanımlanmıştır, ki bu da sayısal sarmalayıcı sınıflar için bir üst sınıftır, `BigInteger` ve `BigDecimal`, Şekil 15.2'de gösterildiği gibi.

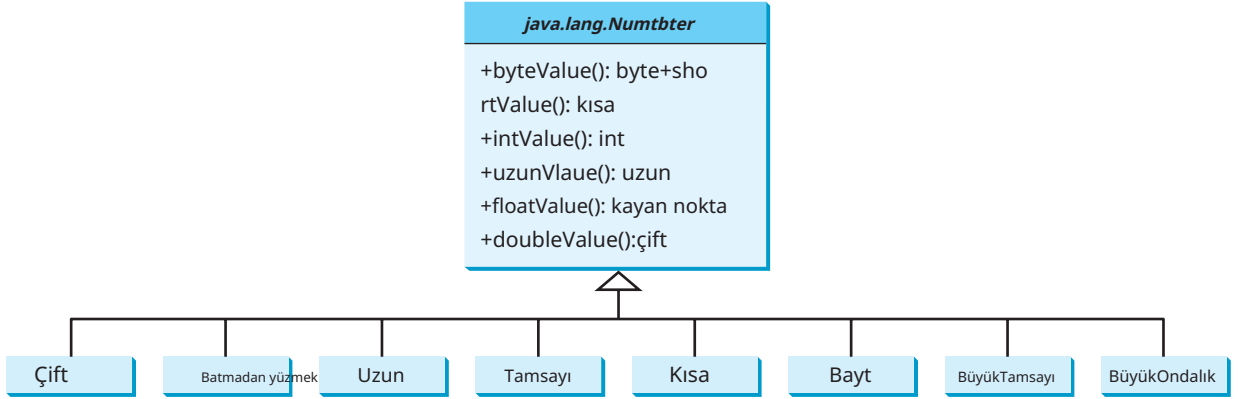


FIGURE 15.2 Sayı sınıfı, `Double`, `Float`, `Long`, `Integer`, `Short`, `Byte`, `BigInteger` ve `BigDecimal` için soyut bir üst sınıftır.

`intValue()`, `longValue()`, `floatValue()` ve `doubleValue()` yöntemleri `Number` sınıfında uygulanmadığı için,

`Number` sınıfında soyut yöntemler olarak tanımlanmıştır. `Number` sınıfı bu nedenle soyut bir sınıftır. `byteValue()` ve `shortValue()` yöntemi, `intValue()` yönteminden aşağıdaki gibi uygulanır:

```
public byte byteValue() {
 return (byte) intValue();
}
```

## 566 Bölüm 15 Soyut Sınıflar ve Arayüzler

```
public shorttshortValue() {nreturnt
(tshortt)intValue();n}
```

Number, sayısal sınıflar için üst sınıf olarak tanımlandığında, sayılar için ortak işlemleri gerçekleştir en yöntemler tanımlayabiliriz. Liste 15.5, Number nesnelerinin bir listesindeki en büyük sayıyı bula n bir programı vermektedir.

### LISTING 15.5 LargestNumbers.java

```
1 import java.util.ArrayList;
2 import java.math.*;
34
genel sınıf LargestNumbers {
5 public static void main(String[] args) {
6 ArrayList<Number> list = new ArrayList<Number>();
7 list.add(45); // Bir tamsayı ekle
8 list.add(3445.53); // Bir double ekle
9 // Bir BigInteger ekle
10 list.add(new BigInteger("3432323234344343101"));
11 // Bir BigDecimal ekle
12 list.add(new BigDecimal("2.0909090989091343433344343"));
13
14 System.out.println("En büyük sayı "+
15 getLargestNumber(list));
16 }
17
18 public static Number getLargestNumber(ArrayList<Number> list) {
19 eğer(list == null || list.size() == 0)
20 null değerini döndür;
21
22 Sayı numarası = list.get(0);
23 for(tinti = 1; i < liste.size(); i++)
24 eğer(number.doubleValue() < list.get(i).doubleValue())
25 sayı = list.get(i);
26
27 dönüş numarası;
28 }
29 }
```

bir dizi listesi oluştur  
listeye sayı ekle

çağır getLargestNumber

çift Değer



En büyük sayı 3432323234344343101

Program, **Number** nesnelerinin bir **ArrayList**'ini oluşturur (6. satır). Bir **Integer** nesnesi, bir **Double** nesnesi, bir **BigInteger** nesnesi ve bir **BigDecimal** nesnesini listeye ekler (7–12. satırlar). Dikkat edin ki **45** otomatik olarak bir **Integer** nesnesine dönüştürülür ve 7. satırda listeye eklenir ve ki **3445.53** otomatik olarak bir **Double** nesnesine dönüştürülür ve 8. satırda listeye eklenir autoboxing kullanılarak.

**getLargestNumber** metodunu çağırmak, listede en büyük sayıyı döndürür (15. satır). **getLargestNumber** metodu, liste null ise veya liste boyutu 0 ise null döndürür (19–20. satırlar). Listedeki en büyük sayıyı bulmak için, sayılar **doubleValue()** metodunu çağırarak karşılaştırılır (24. satır). **doubleValue()** metodu, **Number** sınıfında tanımlanmıştır ve **Number**'in somut alt sınıfında uygulanmıştır. Eğer bir sayı bir **Integer** nesnesi ise, **Integer**'in **doubleValue()** metodu çağırılır. Eğer bir sayı bir **BigDecimal** nesnesi ise, **BigDecimal**'in **doubleValue()** metodu çağırılır.

Eğer **doubleValue()** metodu **Number** sınıfında tanımlı değilse, **Number** sınıfını kullanarak farklı türdeki sayılar arasında en büyük sayıyı bulamazsınız.

## 15.4 Vaka Çalışması: Takvim veGregorianCalendar567

15.4 Aşağıdaki iki satır kod neden derleniyor ama çalışma zamanı hatası veriyor?

```
Sayı numberRef = tnewtInteger(t0t);nDou
ble doubleRef = (Double)numberRef;
```



15.5 Aşağıdaki iki satır kod neden derleniyor ama çalışma zamanı hatası veriyor?

```
Sayı[] sayıArray = tnewtInteger[t2t];numberArr
ay[t0t] = tnewtDouble(t1.5t);
```

15.6 Aşağıdaki kodun çıktısını gösterin.

```
genel sınıfTest {
 public static voidtmain(String[] args) {nNumber x
 = t3t;
 System.out.println(x.intValue());nSystem.
 out.println(x.doubleValue());n}
}
```

15.7 Aşağıdaki kodda ne yanlış? (Integer ve Double sınıfları için `compareTo` m  
etodunun 10.12. Bölümde tanıtıldığını unutmayın.)

```
genel sınıfTest {
 public static voidtmain(String[] args) {nNumber x
 = tnewtInteger(t3t);
 System.out.println(x.intValue());nSystem.
 out.println(x.compareTo(tnewtInteger(t4t)));n}
}
```

15.8 Aşağıdaki kodda ne yanlış?

```
genel sınıfTest {
 public static voidtmain(String[] args) {nNumber x
 = tnewtInteger(t3t);
 System.out.println(x.intValue());nSystem.
 out.println((Integer)x.compareTo(tnewtInteger(t4t)));n}
}
```

## 15.4 Vaka Çalışması: Takvim veGregorianCalendar



**GregorianCalendar** soyut sınıf **Calendar**'in somut bir alt sınıfıdır.

Bir **java.util.Date** örneği, milisaniyeyesinliği ile belirli bir anı temsil eder.**java.util.Calendar**, yıl, ay, gün, saat, dakika ve saniye gibi ayrıntılı takvimbilgilerin i çıkarmak için soyut bir temel sınıftır. **Calendar**'in alt sınıfları, Gregorian takvim i, ay takvimive Yahudi takvimi gibi belirli takvim sistemlerini uygulayabilir. Şu a nda, Java'da Gregoriantakvimi için **java.util.GregorianCalendar** desteklenme ktedir, Şekil 15.3'te gösterildiği gibi. **add** metodu, Calendar sınıfında soyut bir m etoddur, çünkü uygulaması somut bir takvim sistemine bağlıdır.

Varsayılan bir **GregorianCalendar** oluşturmak için **new GregorianCalendar()** kütakvim oluşturma llanabilirsinizve belirtilen yıl, ay ve tarih ile bir **GregorianCalendar** oluşturmak için **new GregorianCalendar(year, month, date)** kullanabilirsiniz. ay parametresi0 tab anlıdır—yani, 0 Ocak içindir.



VideoNotu

Takvim ve  
GregorianCalendar  
sınıfları

soyut ekle yöntemi

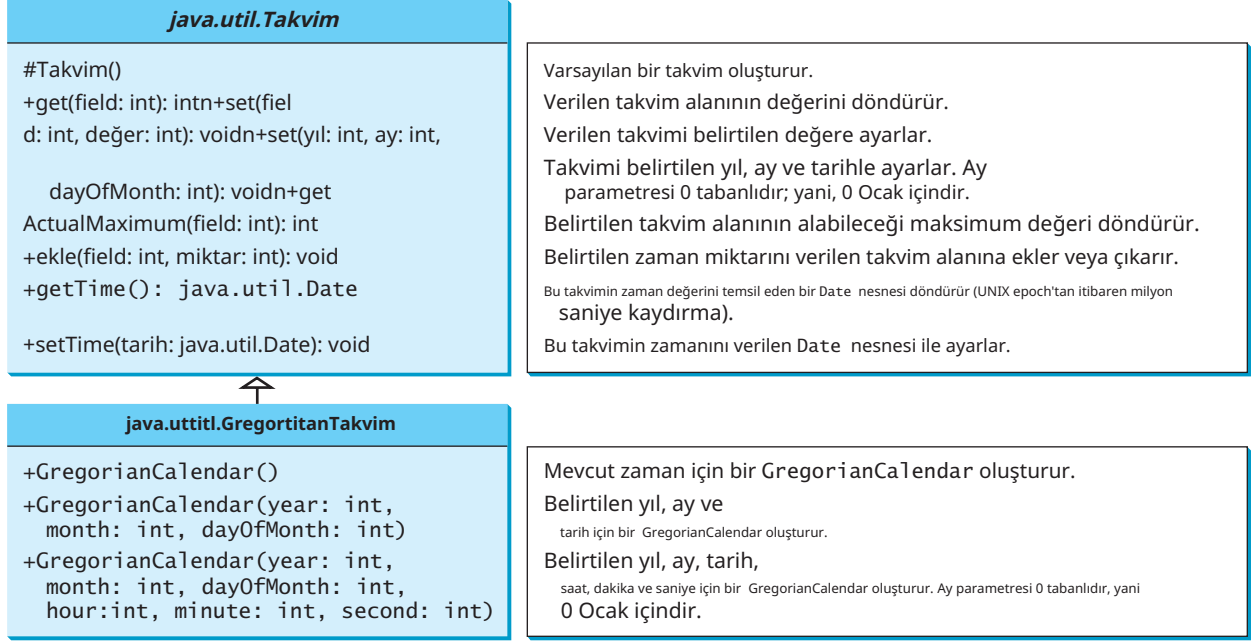


FIGURE 15.3 Soyut Calendar sınıfı çeşitli takvimlerin ortak özelliklerini tanımlar.

al(alan)

Calendar sınıfında tanımlanan `get(int field)` yöntemi, bir Calendar nesnesinden tarih ve saat bilgilerini çıkarmak için yararlıdır. Alanlar, Tablo 15.1'de gösterildiği gibi sabitler olarak tanımlanmıştır.

TABLE 15.1 Takvim Sınıfındaki Alan Sabitleri

| Sabit                      | Açıklama                                                                             |
|----------------------------|--------------------------------------------------------------------------------------|
| YIL                        | Takvimin yılı.                                                                       |
| AY                         | Takvimin ayı, Ocak için 0.                                                           |
| TARİH                      | Takvimin günü.                                                                       |
| SAAAT                      | Takvim saati (12 saat notasyonu).                                                    |
| GÜNÜN_SAAATI               | Takvim saati (24 saat notasyonu).                                                    |
| DAKİKA                     | Takvim dakikası.                                                                     |
| SANIYE                     | Takvim saniyesi.                                                                     |
| HAFTANIN_GÜNÜ              | Haftadaki gün numarası, Pazar için 1.                                                |
| AYIN_GÜNÜ                  | Aynı olarak TARİH.                                                                   |
| YILIN_GÜNÜ                 | Yıldaki gün numarası, yılın ilk günü için 1 ile.                                     |
| AYIN_HAFTASI               | Aydaki hafta numarası, ilk hafta için 1 ile.                                         |
| YILIN_HAFTASI              | Yıldaki hafta numarası, ilk hafta için 1 ile.                                        |
| ÖĞLEDEN_ÖNCE_ÖĞLEDEN_SONRA | Öğleden önce veya öğleden sonra göstergesi (0 öğleden önce ve 1 öğleden sonra için). |

Liste 15.6, mevcut zaman için tarih ve saat bilgilerini gösteren bir örnek vermektedir.

### LISTATİK 15.6 TestCalendar.java

```

1 import java.util.*;
2

```



## 15.4 Vaka Çalışması: Takvim veGregorianCalendar569

```
3 genel classtestCalendar {
4 public static void main(String[] args) {
5 // Mevcut tarih ve saat için bir Gregorian takvimi oluştur
6 Takvim takvim = newGregorianCalendar();
7 System.out.println("Mevcut zaman "+newDate());
8 System.out.println("YIL: "+ calendar.get(Calendar.YEAR));
9 System.out.println("AY: "+ calendar.get(Calendar.MONTH));
10 System.out.println("TARİH: "+ calendar.get(Calendar.DATE));
11 System.out.println("SAAAT: "+ calendar.get(Calendar.HOUR));
12 System.out.println("GÜN İÇİ SAAT: "+
13 calendar.get(Calendar.HOUR_OF_DAY));
14 System.out.println("DAKİKA: "+ calendar.get(Calendar.MINUTE));
15 System.out.println("SANİYE: "+ calendar.get(Calendar.SECOND));
16 System.out.println("HAFTANIN GÜNÜ: "+
17 calendar.get(Calendar.DAY_OF_WEEK));
18 System.out.println("AYIN GÜNÜ: "+
19 calendar.get(Calendar.DAY_OF_MONTH));
20 System.out.println("YILIN GÜNÜ: "+
21 calendar.get(Calendar.DAY_OF_YEAR));
22 System.out.println("AYIN HAFTASI: "+
23 calendar.get(Calendar.WEEK_OF_MONTH));
24 System.out.println("YILIN HAFTASI: "+
25 calendar.get(Calendar.WEEK_OF_YEAR));
26 System.out.println("AM_PM: "+ calendar.get(Calendar.AM_PM));
27
28 // 11 Eylül 2001 için bir takvim oluştur
29 Takvim takvim1 = newGregorianCalendar(2001, 8, 11);
30 String[] günİsimleri = {"Pazar", "Pazartesi", "Salı", "Çarşamba",
31 "Perşembe", "Cuma", "Cumartesi"};
32 System.out.println("11 Eylül 2001 bir "+
33 günİsimleri[takvim1.get(Calendar.DAY_OF_WEEK) - 1]);
34 }
35 }
```

mevcut zaman için takvim

takvimdeki alanları çıkar

takvim oluştur



```
Mevcut zaman Sun Nov 27 17:48:15 EST 2011
YIL: 2011
AY: 10
TARİH: 27
SAAT: 5
Saat_Günü: 17
Dakika: 48
Saniye: 15
Haftanın_Günü: 1
Aydın_Günü: 27
Yılın_Günü: 331
Aydın_Hafta: 5
Yılın_Haftası: 49
Öğleden_Önce_Öğleden_Sonra: 1
11 Eylül 2001 bir Salı'dır
```

Calendar sınıfında tanımlanan `set(int field, value)` metodu bir alanı ayarlamak için kullanılabilir. Örneğin, takvimi ayın ilk günü olarak ayarlamak için `calendar.set(Calendar.DAY_OF_MONTH, 1)` kullanabilirsiniz.

set(alan, değer)

`add(field, value)` metodu belirtilen miktarı verilen bir alana ekler. Örneğin, `add(Calendar.DAY_OF_MONTH, 5)` mevcut takvim zamanına beş gün ekler. `add(Calendar.DAY_OF_MONTH, -5)` mevcut takvim zamanından beş gün çıkarır.

ekle(alan, miktar)

**15.9** Takvim sınıfını kullanarak bir Takvim nesnesi oluşturabilir misiniz?

**15.10** Takvim sınıfında hangi yöntem soyut?

**15.11** Mevcut zaman için bir Takvim nesnesi nasıl oluşturursunuz?

**15.12** Bir Takvim nesnesi için, yılını, ayını, tarihini, saatini, dakikasını ve saniyesini nasıl alırsınız?

```

1 genel classtestEdible {
2 public static void main(String[] args) {
3 Object[] nesneleri = {tnewTiger(),tnewChicken(),tnewApple()};
4 for(tintti = t0t; i < nesneler.uzunluk; i++) {
5 if(nesneler[i].tinstanceoftEdible)
6 System.out.println(((Edible)objects[i]).howToEat());
7
8 if(objects[i].tinstanceoftAnimal) {n9 System.
9 out.println(((Animal)objects[i]).sound());n10 }
10 }
11 }

```

```
11 }n12 }n1
3 }
```

```
1415 soyut sınıfAnimal {
16 /** Hayvan sesini döndür */
17 public abstractString ses();
18 }
```

Hayvan sınıfı

```
1920 classTavukextendsHayvanuyguluyorYenilebilir {
21 @Geçersiz Kıl
22 publicString nasılYenir() {
23 return"Tavuk: Kızartın";
24 }
```

uyguluyor Yenilebilir

nasılYenir()

```
yirmidört }
25
26 @Geçersiz Kıl
27 publicString ses() {
28 return"Tavuk: kukuriku";
kes}na0}
```

```
3132 classKaplanextendsHayvan {
33 @Override
34 publicString ses() {
35 return"Kaplan: RROOAARR";
36 }Hayı
```

Kaplan sınıfı

```
37383940 soyut sınıfMeyveuyguluyorYenilebilir {
41 // Veri alanları, yapıcılar ve yöntemler burada atlanmıştır
42 }
```

Edible'ı uygular

```
4243 classElmaextendsMeyve {
44 @Override
45 publicString nasılYenir() {
46 return"Elma: Elma şarabı yap";
47 }n48 }
```

Elma sınıfı

```
4950 classPortakalextendsMeyve {
51 @Override
52 publicString nasılYenir() {
53 return"Portakal: Portakal suyu yap";
54 }n55 }
```

Portakal sınıfı

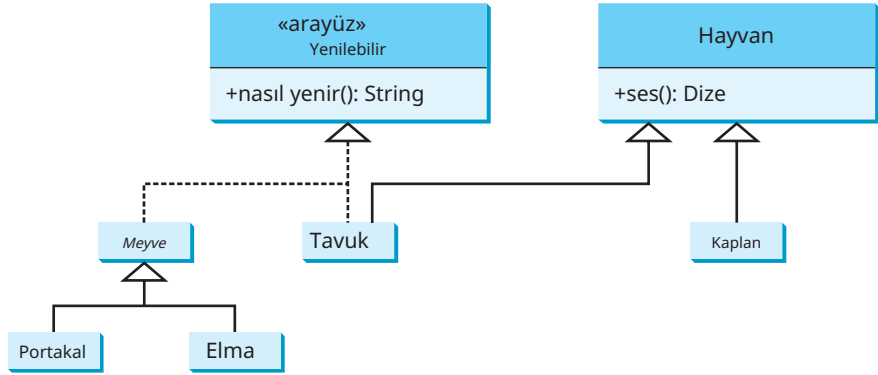
```
Kaplan: RROOAARR
Tavuk: Kızart
Tavuk: kukulu kukulu
Elma: Elma şırası yap
```



Bu örnek, birkaç sınıf ve arayüz kullanmaktadır. Onların miras ilişkisi Şekil 15.4'te gösterilmiştir.

Animal sınıfı, ses yöntemini (satır 17) tanımlar. Bu, soyut bir yöntemdir ve somut bir hayvan sınıfı tarafından uygulanacaktır.

Tavuk sınıfı, tavukların yenilebilir olduğunu belirtmek için Edible'ı uygular. Bir sınıf bir arayüzü uyguladığında, arayüzde tanımlanan tüm yöntemleri tam olarak uygulamaktadır.



**Fiçin 15.4** *Yenilebilir*, *Tavuk* ve *Meyve* için bir süper türdür. *Hayvan*, *Tavuk* ve *Kaplan* için bir süper türdür. *Meyve*, *Portakal* ve *Elma* için bir süper türdür.

imza ve dönüş türü. *Tavuk* sınıfı, *nasılYenir* yöntemini (satırlar 22–24) uygular. *Tavuk* ayrıca *ses* yöntemini uygulamak için *Animal*'dan genişler (satırlar 27–29).

*Meyve* sınıfı, *Edible*'ı uygular. *NasılYenir* yöntemini uygulamadığı için, *Meyve* soyut olarak belirtilmelidir (satır 39). *Meyve*'nin somut alt sınıfları, *nasılYenir* yöntemini uygulamak zorundadır. *Elma* ve *Portakal* sınıfları, *nasılYenir* yöntemini (satırlar 45, 52) uygular.

*main* metodu, *Tiger*, *Chicken* ve *Apple* için üç nesne içeren bir dizi oluşturur (3. satır) ve eğer eleman yenilebilir ise *howToEat* metodunu çağırır (6. satır) ve eğer eleman bir hayvan ise *sound* metodunu çağırır (9. satır).

Özünde, *Edible* arayüzü yenilebilir nesneler için ortak davranışı tanımlar. Tüm yenilebilir nesnelerin *howToEat* metodu vardır.



### Not

Tüm veri alanları *public static final* ve tüm metodlar *public abstract* bir arayüz de olduğundan, Java bu değiştiricilerin atlanmasına izin verir. Bu nedenle aşağıdaki arayüz tanımları eşdeğerdir:

```

genel arayüzT {final intt
 genel statik K = t1t;
 kamusal soyut voidtp();
}

```

Eşdeğer

```

genel arayüzT {ninttK = t
 1t;
 voidtp();n}

```



### Tt1tp

Bir arayüzde tanımlanan bir sabite, aşağıdaki sözdizimi kullanılarak erişilebilir **InterfaceName.CONSTANT\_NAME** (örneğin, **T.K**). Birçok sınıf tarafından paylaşılan ortak sabitleri bir arayüzde tanımlamak iyi bir uygulamadır. Örneğin, **LEFT**, **CENTER**, **RIGHT**, **LEADING**, **TRAILING**, **TOP** ve **BOTTOM** sabitleri *AbstractButton* içinde kullanıldığı gibi birçok diğer Swing bileşeninde de kullanılmaktadır. Bu sabitler, **javax.swing.SwingConstants** arayüzünde merkezi olarak tanımlanmıştır. Tüm Swing GUI bileşenleri *SwingConstants*'ı uygular. Sabitlere *SwingConstants* veya bir GUI bileşeni aracılığıyla başvuruabilirsiniz. Örneğin, **SwingConstants.CENTER**, **JBUTTON.CENTER** ile aynıdır.

15.14A bir arayüz olduğunu varsayalım. A türünde bir referans değişkeni `x` bu şekilde tanımlayabilir misiniz?

bir `x`;

15.15 Aşağıdakilerden hangisi doğru bir arayüzdür?

```
arayüzA {nvoidt
 print() { };n}
```

(a)

```
soyut arayüzA extends I1, I2 {abstract voidtp
 rint() { };
}
```

(b)

```
soyut arayüzA {nprint();
}
```

(c)

```
arayüzA {nvoidt
 print();n}
```

(d)

15.16 Açıklayın neden `SwingConstants.LEFT`, `AbstractButton.LEFT`, `JButton.LEFT`, `JCheckBox.LEFT`, `JRadioButton.LEFT` ve `JLabel.LEFT` hepsinin aynı değere sahip olduğunu.

## 15.6 Comparable Arayüzü

*Comparable arayüzü, nesneleri karşılaştırmak için compareTo metodunu tanımlar.*



Diyelim ki, iki nesnenin, örneğin iki öğrencinin, iki tarihin, iki dairenin, iki dikdörtgenin veya iki karenin daha büyük olanını bulmak için jenerik bir yöntem tasarlamak istiyorsunuz. Bunu başarmak için, iki nesne karşılaştırılabilir olmalıdır, bu nedenle nesnelerin ortak davranışı karşılaştırılabilir olmalıdır. Java, bu amaç için **Comparable** arayüzünü sağlar. Arayüz aşağıdaki gibi tanımlanmıştır:

```
// Nesneleri karşılaştırmak için arayüz, java.lang içinde tanımlanmıştır
paket java.lang;
```

java.lang.Karşılaştırılabilir

```
public interface Comparable<E>
{
 public int compareTo(genel aray
```

`compareTo` metodu, bu nesnenin belirtilen nesne `o` ile sırasını belirler ve bu nesne `o`'dan daha küçükse, eşitse veya daha büyükse sırasıyla negatif bir tamsayı, sıfır veya pozitif bir tamsayı döner.

`Comparable` arayüzü, jenerik bir arayüzdür. Jenerik tür `E`, bu arayüzü uygularken somut bir tür ile değiştirilir. Java kütüphanesindeki birçok sınıf, nesneler için doğal bir sıralama tanımlamak üzere `Comparable`'ı uygular. `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character`, `BigInteger`, `BigDecimal`, `Calendar`, `String` ve `Date` sınıfları, `Comparable` arayüzünü uygular. Örneğin, `Integer`, `BigInteger`, `String` ve `Date` sınıfları Java API'sinde şu şekilde tanımlanmıştır:

```
public class Integer extends Number
{
 @Override
 public int compareTo(Integer o) {
 // sınıf gövdesi atlandı
 }
}
```

```
public class BigInteger extends Number
{
 @Override
 public int compareTo(BigInteger o) {
 // sınıf gövdesi atlandı
 }
}
```

## 574 Bölüm 15 Soyut Sınıflar ve Arayüzler

```
public class String extends Object
 implements Comparable<String> {
 // sınıf gövdesi atlandı

 @Override
 public int compareTo(String o) {
 // Uygulama atlandı
 }
}
```

```
public class Date extends Object
 implements Comparable<Tarih> {
 // sınıf gövdesi atlandı

 @Override
 public int compareTo(Date o) {
 // Uygulama atlandı
 }
}
```

Bu nedenle, sayılar karşılaştırılabilir, dizeler karşılaştırılabilir ve tarihler de öyle. İki sayıyı, iki diziyi ve iki tarihi karşılaştırmak için **compareTo** metodunu kullanabilirsiniz. Örneğin, aşağıdaki kod

```
1 System.out.println(new Integer(t3t).compareTo(new Integer(t5t)));
2 System.out.println("ABC".compareTo("ABE"));
3 java.util.Date date1 = new java.util.Date(t2013t,t1t,t1t);
4 java.util.Date date2 = new java.util.Date(t2012t,t1t,t1t);
5 System.out.println(date1.compareTo(date2));
```

gösterir

```
-1
-2
1
```

1. Satır, 3'ün 5'ten küçük olduğu için negatif bir değer gösterir. 2. Satır, ABC'nin ABE'den küçük olduğu için negatif bir değer gösterir.

5. Satır, **date1**'nin **date2**'den büyük olduğu için pozitif bir değer gösterir.

Let **n** bir **Integer** nesnesi, **s** bir **String** nesnesi ve **d** bir **Date** nesnesi olsun. Aşağıdaki if adelerin hepsi **true**'dur.

```
n örnekOfTamsayı
n InstanceOfObject
n örnekOfKarşılaştırılabilir
```

```
s örnekOfString
s InstanceOfObject
s örnekOfKarşılaştırılabilir
```

```
d Java.UTIL.date örneği
d InstanceOfObject
d örnekOfKarşılaştırılabilir
```

Tüm **Karşılaştırılabilir** nesnelerin **karşılaştır** metodu olduğundan, Java API'sindeki **java.util.Arrays.sort(Object[])** metodu, nesnelerin karşılaştırılması ve sıralanması için **compareTo** metodunu kullanır; bu, nesnelerin **Comparable** arayüzünün örnekleri olması koşuluyla geçerlidir. Liste 15.8, bir dizi string ve bir dizi **BigInteger** nesnesini n sıralanmasına bir örnek vermektedir.

### LISTE 15.8 SortComparableObjects.java

bir dizi oluştur  
diziyi sırala

bir dizi oluştur

```
1 import java.math.*;
2
3 public class SortComparableObjects {
4 public static void main(String[] args) {
5 String[] şehirler = {"Savannah", "Boston", "Atlanta", "Tampa"};
6 java.util.Arrays.sort(şehirler);
7 için (String şehir: şehirler)
8 System.out.print(şehir + " ");
9 System.out.println();
10
11 BigInteger[] büyükSayılar = {new BigInteger("2323231092923992")},
```

```

12 new BigInteger("432232323239292"),
13 new BigInteger("54623239292"));
14 java.util.Arrays.sort(büyükSayılar);
15 için(BigInteger sayı: büyükSayılar)
16 System.out.print(sayı + " ");
17 }
18 }

```

diziyi sırala

Atlanta Boston Savannah Tampa n54  
623239292 432232323239292 2323231092923992



Program, bir dizi dize oluşturur (satır 5) ve dizeleri sıralamak için **sort** metodunu çağırır (satır 6). Program, **BigInteger** nesnelerinden oluşan bir dizi oluşturur (satırlar 11–13) ve **BigInteger** nesnelerini sıralamak için **sort** metodunu çağırır (satır 14).

Rectangle nesnelerinden oluşan bir diziyi sıralamak için **sort** metodunu kullanamazsınız, çünkü Rectangle Comparable'ı uygulamaz. Ancak, Comparable'ı uygulayan yeni bir dikdörtgen sınıfı tanımlayabilirsiniz. Bu yeni sınıfın örnekleri karşılaştırılabilir. Bu yeni sınıfa ComparableRectangle adı verelim, Liste 15.9'da gösterildiği gibi.

### LISTING 15.9 Karşılaştırılabilir Dikdörtgen.java

```

1 genel sınıf KarşılaştırılabilirDikdörtgenmetinendstDikdörtgen
2 applicationstComparable<ComparableRectangle> {
3 /** Belirtilen özelliklerle bir ComparableRectangle oluştur */
4 public ComparableRectangle(double width, double height) {
5 süper(genişlik, yükseklik);
6 }
7
8 @Override // Comparable'da tanımlanan compareTo metodunu uygula
9 public int compareTo(ComparableRectangle o) {
10 eğer(getArea() > o.getArea())
11 return 1;
12 aksi takdirde eğer(getArea() < o.getArea())
13 dönüş-1;
14 aksi takdirde
15 return 0;
16 }
17
18 @Override // GeometrikNesne'deki toString metodunu uygulayın
19 public String toString() {
20 return süper.toString() + " Alan: " + getArea();
21 }
22 }

```

Karşılaştırılabilir

uygulama compareTo

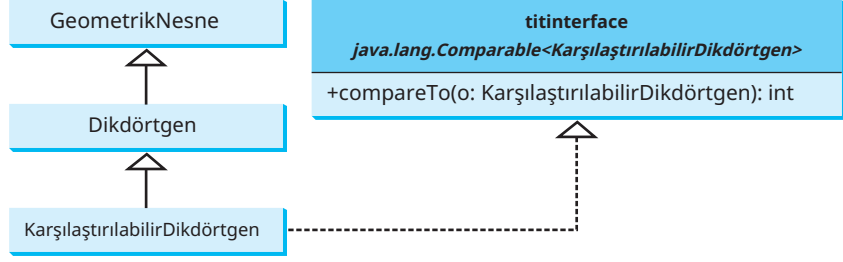
uygulama toString

**ComparableRectangle** Rectangle sınıfından türetilir ve **Comparable** arayüzünü uygular, bu Şekil 15.5'te gösterilmiştir. implements anahtar kelimesi, ComparableRectangle'nin Comparable arayüzünden tüm sabitleri miras aldığını ve arayüzdeki yöntemleri uyguladığını belirtir. **compareTo** metodu iki dikdörtgenin alanlarını karşılaştırır. Bir **ComparableRectangle** örneği aynı zamanda bir **Rectangle**, **GeometricObject**, **Object**, ve **Comparable** örneğidir.

Artık bir dizi **ComparableRectangle** nesnelerini sıralamak için **sort** metodunu kullanabilirsiniz, bu da Liste 15.10'da gösterilmiştir.

Notasyon:

Arayüz adı ve metod adı *arı* italik yazılmıştır. Kesik çizgiler ve boş üçgenler arayüze işaret etmek için kullanılır.



Fiçin 15.5 KarşılaştırılabilirDikdörtgen genişletir Dikdörtgen ve uygular Karşılaştırılabilir.

### LISTE 15.10 SıralamaDikdörtgenleri.java

bir dizi oluştur

diziye sırala

```

1 public class SıralamaDikdörtgenleri {
2 public static void main(String[] args) {
3 KarşılaştırılabilirDikdörtgen[] dikdörtgenler = {
4 new KarşılaştırılabilirDikdörtgen(3.4, 5.4),
5 yeniComparableRectangle(13.24, 55.4),
6 yeniComparableRectangle(7.4, 35.4),
7 yeniComparableRectangle(1.4, 25.4)};
8 java.util.Arrays.sort(dikdörtgenler);
9 için(Rectangle rectangle: rectangles) {
10 System.out.print(rectangle + " ");
11 System.out.println();
12 }
13 }
14 }

```



Genişlik: 3.4 Yükseklik: 5.4 Alan: 18.36  
 Genişlik: 1.4 Yükseklik: 25.4 Alan: 35.559999999999995  
 Genişlik: 7.4 Yükseklik: 35.4 Alan: 261.96  
 Genişlik: 13.24 Yükseklik: 55.4 Alan: 733.496

arayüzün faydaları

Bir arayüz, başka bir jenerik programlama biçimi sağlar. Bu örnekte, bir arayüz kullanımdan nesneleri sıralamak için bir jenerik *sıralama* yöntemini kullanmak zor olacaktır, çünkü *Comparable* ve başka bir sınıfı miras almak için çoklu miras almak gerekecektir, örneğin *Dikdörtgen*, aynı anda.

*Object* sınıfı, nesnelerin içeriklerinin aynı olup olmadığını karşılaştırmak için alt sınıfları tarafından geçersiz kılınması amaçlanan *equals* metodunu içerir. Diyelim ki *Object* sınıfı, *Comparable* arayüzünde tanımlandığı gibi *compareTo* metodunu içeriyor; *sort* metodu, herhangi bir nesne listesini karşılaştırmak için kullanılabilir. Bir *compareTo* metodunun *Object* sınıfında yer alıp almayacağı tartışmalıdır. *Object* sınıfında *compareTo* metodu tanımlanmadığı için, nesnelerin *Comparable* arayüzünün örnekleri olması durumunda karşılaştırılabilmesi için Java'da *Comparable* arayüzü tanımlanmıştır. *compareTo* metodunun *equals* ile tutarlı olması şiddetle tavsiye edilir (ancak zorunlu değildir). Yani, iki nesne *o1* ve *o2* için, *o1.compareTo(o2) == 0* yalnızca *o1.equals(o2)* doğrudysa geçerlidir.

Point Kontrol



**15.18** Aşağıdakilerden hangisi String sınıfındaki compareTo metodunun doğru yöntem başlığıdır?

```
public int compareTo(String o)
public int compareTo(Object o)
```

**15.19** Aşağıdaki kod derlenebilir mi? Neden?

```
Tamsayı n1 = new Tamsayı(t3t);
Object n2 = new Tamsayı(t4t);
System.out.println(n1.compareTo(n2));
```

**15.20** Bir sınıfta compareTo metodunu, Comparable arayüzünü uygulamadan tanımlayabilirsiniz. Comparable arayüzünü uygulamanın faydaları nelerdir?

**15.21** Doğru mu yanlış mı? Eğer bir sınıf Comparable'ı uygularsa, sınıfın nesnesi compareTo metodunu çağırabilir.

## 15.7 Cloneable Arayüzü

Cloneable arayüzü, nesneleri karşılaştırmak için compareTo metodunu tanımlar.



Sıklıkla bir nesnenin kopyasını oluşturmak istenir. Bunu yapmak için, Clone

metodunu kullanmanız ve Cloneable arayüzünü anlamanız gerekir.

Bir arayüz, sabitler ve soyut metodlar içerir, ancak Cloneable arayüzü özel bir durumdur. java.lang paketindeki Cloneable arayüzü aşağıdaki gibi tanımlanmıştır:

```
package java.lang;
public interface Cloneable { }
```

```
java.lang.Cloneable
```

```
interface Cloneable { }
```

Bu arayüz boştur. Boş bir gövdeye sahip bir arayüz *marker interface* denir.

işaretçi arayüzü

Bir marker arayüz, sabitler veya metodlar içermez. Bir sınıfın belirli istenen özelliklere sahip olduğunu belirtmek için kullanılır. Cloneable arayüzünü uygulayan bir sınıf, klonlanabilir olarak işaretlenir ve nesneleri, tanımlanan clone() metodu kullanılarak klonlanabilir.

Object sınıfı.

Java kütüphanesindeki birçok sınıf (örneğin, Date, Calendar ve ArrayList)

Cloneable arayüzünü uygular. Bu nedenle, bu sınıfların örnekleri kopyalanabilir. Örneğin, aşağıdaki kod

```
1 Takvim takvim1 = new GregorianCalendar(t2013t,t2t,t1t);
2 Takvim takvim2 = new GregorianCalendar(t2013t,t2t,t1t);
```

```
3 Takvim takvim1 = takvim1;
```

```
4 Takvim takvim2 = (Takvim)calendar.clone();
5 System.out.println("calendar == takvim1 is " + takvim1 == takvim2);
```

```
6 System.out.println("calendar == takvim2, " + takvim1 == takvim2);
```

```
7 System.out.println("calendar == takvim2, " + takvim1 == takvim2);
```

```
8 System.out.println("calendar.equals(calendar2) is " + takvim1.equals(calendar2));
```

gösterir

```
takvim == takvim1 true
calendar.equals(calendar2) is true
```

Yukarıdaki kodda, 2. satır calendar referansını calendar1'ye kopyalar, bu nedenle calendar1 aynı calendar nesnesine işaret eder. 3. satır yeni bir nesne oluşturur ki bu da

## 578 Bölüm 15 Soyut Sınıflar ve Arayüzler

bir kopyasını `takvim` oluşturur ve yeni nesnenin referansını `takvim2`'ye atar. `takvim2` ve `takvim` aynı içeriğe sahip farklı nesnelerdir.

Aşağıdaki kod

```
1 ArrayList<Double> list1 = new ArrayList<Double>();
2 list1.add(t1.5t);
3 list1.add(t2.5t);
4 list1.add(t3.5t);
5 ArrayList<Double> liste2 = (ArrayList<Double>)list1.clone();
6 ArrayList<Double> liste3 = liste1;
7 list2.add(t4.5t);
8 list3.remove(t1.5t);
9 System.out.println("list1 şudur "+ list1);
10 System.out.println("list2 şudur "+ list2);
11 System.out.println("list3 şudur "+ list3);
```

gösterir

```
list1 şudur [2.5, 3.5]
list2 şudur [1.5, 2.5, 3.5, 4.5]
list3 şudur [2.5, 3.5]
```

Yukarıdaki kodda, 5. satır `list1`'in klonu olan yeni bir nesne oluşturur ve bu yeni nesnenin referansını `list2`'ye atar. `list2` ve `list1`, aynı içeriğe sahip farklı nesnelerdir. 6. satır, `list1`'in referansını `list3`'e kopyalar, böylece `list1` ve `list3` aynı `ArrayList` nesnesine işaret eder. 7. satır, `4.5`'i `list2`'ye ekler. 8. satır, `1.5`'i `list3`'ten çıkarır. Çünkü `list1` ve `list3` aynı `ArrayList`'e işaret ediyor, 9. ve 11. satırlar aynı içeriği gösterir.

Bir diziyi `clone` metodu ile klonlayabilirsiniz. Örneğin, aşağıdaki kod

```
1 int[] liste1 = {t1t,t2t};
2 int[] liste2 = liste1.clone();
3 liste1[t0t] = altında;
4 liste2[t1t] = t8t;
5 System.out.println("list1 şu: "+ list1[0] + ", "+ list1[1]);
6 System.out.println("list2 şu: "+ list2[0] + ", "+ list2[1]);
```

gösterir

```
list1 is 7, 2
list2 is 1, 8
```

kopya dizileri

`Cloneable`'ı nasıl uygulayacağım

Özelleştirilmiş bir sınıf tanımlamak için `Cloneable` arayüzünü uygulayan, sınıf `Object` sınıfındaki `clone()` metodunu geçersiz kılmalıdır. Liste 15.11, `Cloneable` ve `Comparable` uygulayan `House` adında bir sınıf tanımlar.

### Liste 15.11 House.java

```
1 kamu sınıfı implements Cloneable, Comparable<House> {
2 özel uluslararası;
3 özel ikiye katlama;
4 Privatetjava.util.Date ne zaman inşa edildi;
5
6 publicHouse(tinttid,tdoubletarea) {
7 thist.id = kimlik;
8 thist.alan = alan;
9 WhenBuilt = newtjava.util.Date();
10 }
11
12 genel inttgetId() {
```

```

13 geri dönüş;
14 }
15
16 genel doubletgetArea() {
17 return alan;
18 }
19
20 public java.util.Date getWhenBuilt() {
21 inşa edildiğinde geri dön;
22 }
23
24 @Override /** Korunan clone metodunu geçersiz kılın
25 Object sınıfında tanımlanmıştır ve erişilebilirliğini artırın */
26 public Object clone() throws CloneNotSupportedException {
27 return super.clone();
28 }
29
30 @Override // Comparable'da tanımlanan compareTo metodunu uygulayın
31 public int compareTo(House o) {
32 if (alan > o.alan)
33 return 1;
34 else if (alan < o.alan)
35 return -1;
36 // aksi takdirde
37 return 0;
38 }
39 }

```

Bu istisna, eğer  
House,  
Cloneable'i uygulamıyorsa

House sınıfı, Object sınıfında tanımlanan clone metodunu (satırlar 26–28) uygular. Başlık şudur:

```
korunmalı native Object clone() throws CloneNotSupportedException;
```

native anahtar kelimesi, bu metodun Java'da yazılmadığını, ancak yerel platform için JVM'de uygulandığını gösterir. protected anahtar kelimesi, metodun aynı paket içinde ve ya bir alt sınıfta erişilmesini kısıtlar. Bu nedenle, House sınıfı metodun üzerine yazmalı ve görünürlük belirleyicisini public olarak değiştirmelidir, böylece metod herhangi bir pakette kullanılabilir. Object sınıfında yerel platform için uygulanan clone metodu, nesneleri klonlama görevini yerine getirdiğinden, House sınıfındaki clone metodu basitçe super.clone() çağrısını yapar. Object sınıfında tanımlanan clone metodu, CloneNotSupportedException fırlatabilir.

KlonDesteklenmeyen İstisna

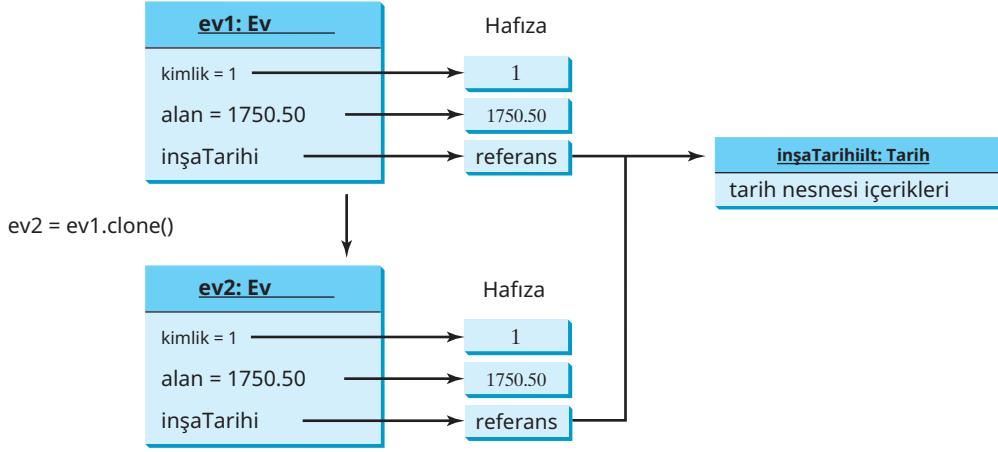
House sınıfı, compareTo metodunu (satırlar 31–38) tanımlayan Comparable arayüzünü uygular. Metod, iki evin alanlarını karşılaştırır.

Artık House sınıfının bir nesnesini oluşturabilir ve ondan bir özdeş kopya oluşturabilirsiniz, şu şekilde:

```
Ev ev 1 = newtHouse(t1,t1750.50t); Ev ev 2 = (
Ev)ev 1.clone();
```

house1 ve house2, özdeş içeriklere sahip iki farklı nesnedir. clone metodu, the Object sınıfında, orijinal nesneden hedef nesneye her alanı kopyalar. Eğer alan bir ilkel türde ise, değeri kopyalanır. Örneğin, area(double türü) değeri, house1'den house2'ye kopyalanır. Eğer alan bir nesne ise, alanın referansı kopyalanır. Örneğin, whenBuilt alanı Date sınıfındadır, bu nedenle referansı house2'ye kopyalanır, Şekil 15.6'da gösterildiği gibi. Bu nedenle, house1.whenBuilt == house2.whenBuilt doğrudur, ancak house1 == house2 yanlıştır. Bu, shallow copy olarak adlandırılır, deep copy yerine, yani eğer alan bir nesne türündeyse, nesnenin referansı kopyalanır, içeriği değil.

yüzeysel kopya  
derin kopya



**ŞEKİL 15.6**Varsayılan `kopyala` metodu yüzeysel bir kopya oluşturur.

derin kopya

Bir `House` nesnesi için derin kopya yapmak için, 26-27. satırlardaki `clone()` metodunu aşağıdaki kod ile değiştirin:

```
public Object clone() throws CloneNotSupportedException {
 // Yüzeysel kopya yap
 House houseClone = (House)super.clone();
 // whenBuilt üzerinde derin kopya yap
 houseClone.whenBuilt = (java.util.Date)(whenBuilt.clone());
 return houseClone;
}
```

veya

```
public Nesne clone() {
 try{
 // Yüzeysel bir kopya oluştur
 House houseClone = (House)super.clone();
 // whenBuilt üzerinde derin kopya
 houseClone.whenBuilt = (java.util.Date)(whenBuilt.clone());
 return houseClone;
 }
 catch(CloneNotSupportedException ex){
 return null;
 }
}
```

Şimdi aşağıdaki kodda bir `House` nesnesini kopyalarsanız:

```
Ev ev 1 = new House(t1,t1750.50t); Ev ev 2 = (
Ev)ev 1.clone();
```

`house1.whenBuilt == house2.whenBuilt` false olacaktır. `house1` ve `house2` iki farklı `Date` nesnesine referans-  
cektir.

**15.22** Eğer nesnenin sınıfı `java.lang.Cloneable`'ı uygulamıyorsa, `clone()` metodunu kullanarak bir nesneyi kopyalayabilir misiniz? `Date` sınıfı `Cloneable`'ı uyguluyor mu?

**15.23** Eğer `House` sınıfı (Listing 15.9'da tanımlanan) `clone()` metodunu geçersiz kılmamışsa veya `House` `java.lang.Cloneable`'ı uygulamıyorsa ne olur?

**15.24** Aşağıdaki kodun çıktısını gösterin:

```
java.util.Date date = new java.util.Date();
java.util.Date date1 = date;
java.util.Date date2 = (java.util.Date)(date.clone());
System.out.println(date == date1);
System.out.println(tarih == tarih2);
System.out.println(date.equals(tarih2));
```

**15.25** Aşağıdaki kodun çıktısını gösterin:

```
ArrayList<String> list = new ArrayList<String>();
list.add("New York");
ArrayList<String> list1 = list;
ArrayList<String> list2 = (ArrayList<String>)(list.clone());
list.add("Atlanta");

System.out.println(list == list1);
System.out.println(list == list2);
System.out.println("liste " + list);
System.out.println("liste1 " + list1);
System.out.println("liste2.get(0) " + list2.get(0));
System.out.println("liste2.size() " + list2.size());
```

**15.26** Aşağıdaki kodda ne yanlış?

```
genel sınıf Test {
 public static void main(String[] args) {
 GeometricObject x = new Circle(1.0);
 GeometricObject y = x.clone();
 System.out.println(x == y);
 }
}
```

## 15.8 Arayüzler vs. Soyut Sınıflar

Bir sınıf birden fazla arayüzü uygulayabilir, ancak yalnızca bir üst sınıfı genişletebilir.



Bir arayüz, bir soyut sınıf gibi daha az veya daha fazla aynı şekilde kullanılabilir, ancak bir arayüz tanımlamak, bir soyut sınıf tanımlamaktan farklıdır. Tablo 15.2 farklılıkları özetlemektedir.

**TABLE 15.2** Arayüzler vs. Soyut Sınıflar

| Değişkenler                                      | Konstrüktörler                                                                                                               | Metotlar                                                     |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Soyut sınıf Kısıtlama yok.                       | Yapıcılar, alt sınıflar tarafından yapıcı zincirleme ile çağrılır. Soyut bir sınıf, new operatörü kullanılarak örneklenemez. | Kısıtlama yok.                                               |
| Arayüz<br>Tüm değişkenler<br>genel statik final. | Yapıcı yok. Bir arayüz, new operatörü kullanılarak örneklenemez.                                                             | Tüm yöntemler public<br>abstract örnek yöntemleri olmalıdır. |

Java, sınıf genişletmesi için yalnızca *tekil miras* izin verir, ancak *birden fazla genişletmeye* arayüzler için izin verir. Örneğin,

tek miras  
çoklu miras

```
genel sınıf YeniSınıf TemelSınıf implements Arayüz1, ..., ArayüzN {
 // uygulama
}
```

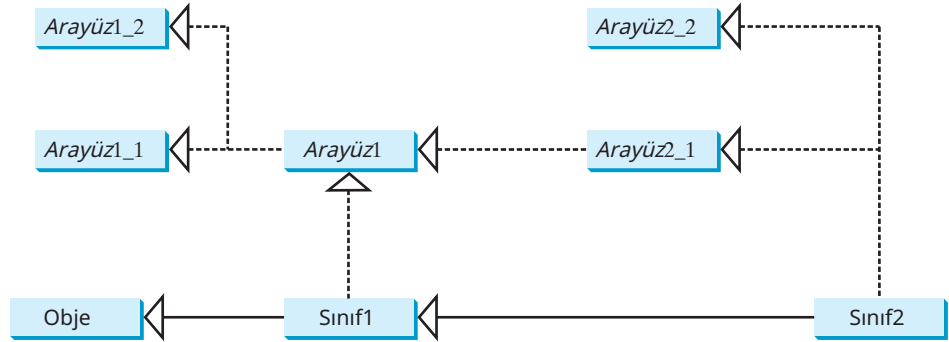
## 582 Bölüm 15 Soyut Sınıflar ve Arayüzler

Bir arayüz, **extends** anahtar kelimesini kullanarak diğer arayüzleri miras alabilir. Böyle bir arayüze alt arayüz denir. Örneğin, aşağıdaki kodda **NewInterface**, **Interface1**, . . . , ve **InterfaceN**'nin bir alt arayüzüdür.

```
genel arayüz YeniArayüz uzatır Interface1, . . . , InterfaceN {
 // sabitler ve soyut yöntemler
}
```

**NewInterface**'yi uygulayan bir sınıf, **NewInterface**, **Interface1**, . . . , ve **InterfaceN**'de tanımlanan soyut yöntemleri uygulamak zorundadır. Bir arayüz diğer arayüzleri uzatabilir ancak sınıfları uzatamaz. Bir sınıf, üst sınıfını uzatabilir ve birden fazla arayüzü uygulayabilir.

Tüm sınıflar tek bir kök, **Object** sınıfını paylaşır, ancak arayüzler için tek bir kök yoktur. Bir sınıf gibi, bir arayüz de bir tür tanımlar. Bir arayüz türündeki bir değişken, arayüzü uygulayan sınıfın herhangi bir örneğini referans alabilir. Bir sınıf bir arayüzü uyguluyorsa, arayüz sınıf için bir üst sınıf gibidir. Bir arayüzü veri türü olarak kullanabilir ve bir arayüz türündeki bir değişkeni alt sınıfına ve tersine dönüştürebilirsiniz. Örneğin, **c**'nin Şekil 15.7'de **Class2**'nin bir örneği olduğunu varsayalım. **c** aynı zamanda **Object**, **Class1**, **Interface1**'nin de bir örneğidir. **Arayüz1\_1**, **Arayüz1\_2**, **Arayüz2\_1**, ve **Arayüz2\_2**.



**Fiçin 15.7** Sınıf1 uygular Arayüz1; Arayüz1 genişletir Arayüz1\_1 ve Arayüz1\_2. Sınıf2 genişletir Sınıf1 ve uygular Arayüz2\_1 ve Arayüz2\_2.

isimlendirme kuralı



### Not

Sınıf adları isimdir. Arayüz adları sıfat veya isim olabilir.

is-a ilişkisi

is-kind-of ilişkisi



### Özet

Soyut sınıflar ve arayüzler, nesnelerin ortak davranışını belirtmek için kullanılabilir.

Bir arayüz mü yoksa bir sınıf mı kullanacağınıza nasıl karar verirsiniz? Genel olarak, güçlü bir is-a ilişkisi, açıkça bir ebeveyn-çocuk ilişkisini tanımlıyorsa, sınıflar kullanılarak modellenmelidir. Örneğin, Gregoryen takvimi bir takvimdir, bu nedenle **java.util.GregorianCalendar** ve **java.util.Calendar** sınıfı arasındaki ilişki sınıf mirası kullanılarak modellenir. Zayıf bir is-a ilişkisi, ayrıca bir türdür ilişkisi olarak bilinir, bir nesnenin belirli bir özelliğe sahip olduğunu gösterir. Zayıf bir is-a ilişkisi arayüzler kullanılarak modellenebilir. Örneğin, tüm dizeler karşılaştırılabilir, bu nedenle **String** sınıfı **Comparable** arayüzünü uygular.

tercih edilen arayüz

Genel olarak, arayüzler soyut sınıflara tercih edilir çünkü bir arayüz, alakasız sınıflar için ortak bir süper tür tanımlayabilir. Arayüzler, sınıflardan daha esnekler.

Sınıfları düşünün. **Animal** sınıfı. Varsayalım ki **howToEat** metodu

**Animal** sınıfında aşağıdaki gibi tanımlanmıştır:

```
soyut sınıf Animal {
 public abstract String nasılYenir();
}
```

Hayvan sınıf

Animal sınıfının iki alt sınıfı aşağıdaki gibi tanımlanmıştır:

```
sınıf Chicken extends Animal {
 @Override
 public String nasılYenir() {
 "Kızart"ı döndür;
 }
}
```

Tavuk sınıf

```
sınıf Duck extends Animal {
 @Override
 public String nasılYenir() {
 "Kızart"ı döndürün;
 }
}
```

Ördek sınıf

Bu miras hiyerarşisi göz önüne alındığında, çok biçimlilik, bir **Chicken**

nesnesine veya bir **Duck** nesnesine **Animal** türünde bir değışkende referans tutmanıza olanak tanır; aşağıdaki kodda olduğu gibi:

```
public static void main(String[] args) {neat(animal);
 Hayvan hayvan = new Tavuk();

 hayvan = new Duck();
 yemek(hayvan);n

 public static void neat(Hayvan hayvanı) {n}
 hayvan.howToEat();
 }
```

JVM, yöntemi çağıran gerçek nesneye dayalı olarak hangi **howToEat** yönteminin çağrılacağını d inamik olarak belirler.

Animal sınıfının bir alt sınıfını tanımlayabilirsiniz. Ancak, bir kısıtlama vardır: Alt sınıf başka bir hayv an için olmalıdır (örneğin, Turkey).

Arayüzlerin bu kısıtlaması yoktur. Arayüzler, her şeyi tek bir sınıf türüne uydurma k zorunda olmadığınız için sınıflardan daha fazla esneklik sağlar. Tanımlayabilirsiniz bir arayüzde **howToEat()** metodunu tanımlayın ve diğer sınıflar için ortak bir süper tür olarak hizmet etmesini sağlayın.

Örneğin,

```
public static void main(String[] args) {neat(stuff);
 Yenilebilir şeyler = new Chicken();

 şeyler = new Duck();
 yemek(şeyler);

 şeyler = new Brokoli();
 yemek(şeyler);n}
```

## 584 Bölüm 15 Soyut Sınıflar ve Arayüzler

Yenilebilir arayüzü

```
public static void eat(Yenilebilir şey) {
 stuff.howToEat();
}
```

Tavuk sınıf

```
arayüz Yenilebilir {
 public String nasılYenir();
}

sınıf Tavuk implements Yenilebilir {
 @Override
 public String nasılYenir() {
 "Kızart"ı döndür;
 }
}
```

Ördek sınıf

```
sınıf Ördek implements Yenilebilir {
 @Override
 public String nasılYenir() {
 "Kızart"ı döndürün;
 }
}
```

Brokkoli sınıfı

```
class Broccoli implements Yenilebilir {
 @Override
 public String nasılYenir() {
 "Karıştırarak kızartın" ifadesini döndürün;
 }
}
```

Yenilebilir nesneleri temsil eden bir sınıf tanımlamak için, sınıfın `Edible` arayüzünü uygulamasına izin verin. Sınıf artık **Edible** türünün bir alt türüdür ve herhangi bir **Edible** nesne **eat** metoduyla çağırılmak için geçirilebilir.

**15.27** Arayüzlerin neden soyut sınıflara tercih edildiğini göstermek için bir örnek verin.

**15.28** Soyut sınıflar ve arayüzler terimlerini tanımlayın. Soyut sınıflar ve arayüzler arasındaki benzerlikler ve farklılıklar nelerdir?

**15.28** Doğru mu yanlış mı?

- Bir arayüz ayrı bir bytecode dosyasına derlenir.
- Bir arayüz statik yöntemlere sahip olabilir.
- Bir arayüz bir veya daha fazla arayüzü genişletebilir.
- Bir arayüz bir soyut sınıfı genişletebilir.
- Bir soyut sınıf bir arayüzü genişletebilir.

## 15.9 Vaka Çalışması: Rational Sınıfı



Bu bölüm, rasyonel sayıları temsil etmek ve işlemek için **Rational** sınıfının nasıl tasarlanacağını göstermektedir.

Bir rasyonel sayı,  $a/b$  biçiminde bir pay ve bir paydadana oluşur; burada **a** pay ve **b** paydadır. Örneğin,  $1/3$ ,  $3/4$  ve  $10/4$  rasyonel sayılardır.

Bir rasyonel sayının paydası 0 olamaz, ancak payı 0 olabilir. Her tam sayı **i**, bir rasyonel sayı olan  $i/1$  ile eşdeğerdir. Rasyonel sayılar, kesirleri içeren tam hesaplamalarda kullanılır—örneğin,  $1/3 = 0.33333 \dots$ . Bu sayı, **double** veya **float** veri türlerinden biri kullanılarak kesirli formatta tam olarak temsil edilemez. Kesin sonucu elde etmek için rasyonel sayıları kullanmalıyız.



Java, tam sayılar ve kayan noktalı sayılar için veri türleri sağlar, ancak rasyonel sayılar için sağlamaz. Bu bölüm, rasyonel sayıları temsil etmek için bir sınıf tasarılmasının nasıl yapılacağını gösterir.

Rasyonel sayılar, tam sayılar ve kayan noktalı sayılarla birçok ortak özelliği paylaştiğundan ve **Number** sayısal sarmalayıcı sınıflar için kök sınıf olduğundan,

**Rasyonel** sınıfını **Number** sınıfının bir alt sınıfı olarak tanımlamak uygundur. Rasyonel sayılar karşılaştırılabilir olduğundan, **Rasyonel** sınıfı ayrıca **Comparable** arayüzünü de uygulamalıdır. Şekil 15.8, **Rasyonel** sınıfını ve **Number** sınıfı ile **Comparable** arayüzü ile olan ilişkisini göstermektedir.

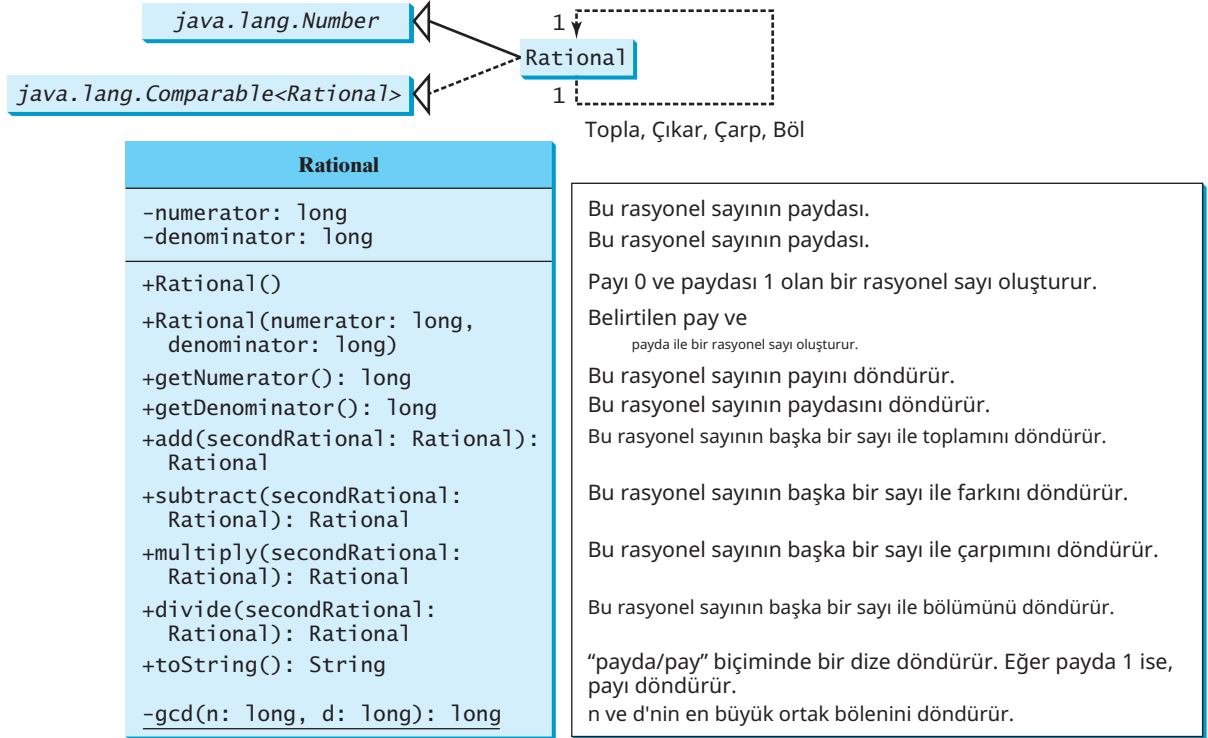


FIGURE 15.8 Rasyonel sınıfının özellikleri, yapıları ve yöntemleri UML'de gösterilmiştir.

Bir rasyonel sayı bir pay ve bir paydadan oluşur. Birçok eşdeğer rasyonel sayı vardır—örneğin,  $1/3 = 2/6 = 3/9 = 4/12$ .  $1/3$ 'ün payı ve paydası dışında ortak bir böleni yoktur, bu nedenle  $1/3$ 'ün en düşük terimlerde olduğu söylenir.

Bir rasyonel sayıyı en düşük terimlerine indirmek için, pay ve paydasının mutlak değerlerinin en büyük ortak bölenini (EBOB) bulmanız gerekir, ardından hem payı hem de paydayı bu değer ile bölmelisiniz. İki tam sayının **n** ve **d** EBOB'unu hesaplamak için kullanılan yöntemi, Liste 4.9, GreatestCommonDivisor.java'da önerildiği gibi kullanabilirsiniz. Bir **Rasyonel** nesnesindeki pay ve payda en düşük terimlerine indirilir.

Her zamanki gibi, iki **Rasyonel** nesnesi oluşturmak ve yöntemlerini test etmek için önce bir test programı yazalım. Liste 15.12 bir test programıdır.

## LİSTELER 15.12 TestRationalClass.java

```

1 genel sınıfıTestRationalClass {
2 /** Ana yöntem */
3 public static voidtmain(String[] args) {
4 // İki rasyonel sayı r1 ve r2 oluştur ve başlat

```

## 586 Bölüm 15 Soyut Sınıflar ve Arayüzler

bir Rational  
bir Rational oluştur

```
56
57
58 nal r1 = tnewtRational(t4t,t2t);
59 Rasyonel r2 = tnewtRational(t2t,t3t);nRatio
60
61 // Sonuçları göster
62 9 System.out.println(r1 + t" + "t+ r2 + t" = "n10 System.out.print + ma(r2));
63 ln(r1 + t" - "t+ r2 + t" = "n11 System.out.println(r1 + t" * "t+ r2 + t + karma(r2)nr1.topla);
64 " = "n12 System.out.println(r1 + t" / "t+ r2 + t" = "n13 System.out + r1.çarpma(r2)nr1.çı);
65 .println(r2 + t", "
66 + r2.doubleValue(nr1.);
67
68 14 }n15 }
```

ekle



```
2 + 2/3 = 8/3
2 - 2/3 = 4/3
2 * 2/3 = 4/3
2 / 2/3 = 3
2/3 0.6666666666666666'dır
```

Temel yöntem, iki rasyonel sayı, **r1** ve **r2** (satırlar 5–6) oluşturur ve **r1 + r2**, **r1 - r2**, **r1 x r2** ve **r1 / r2** (satırlar 9–12) sonuçlarını gösterir. **r1 + r2** işlemini gerçekleştirmek için, **r1.add(r2)** çağrısını yaparak yeni bir **Rational** nesnesi döndürün. Benzer şekilde, **r1.subtract(r2)** çağrısını **r1 - r2** için, **r1.multiply(r2)** çağrısını **r1 x r2** için ve **r1.divide(r2)** çağrısını **r1 / r2** için yapın.

**doubleValue()** yöntemi, **r2**'nin double değerini (satır 13) gösterir. **doubleValue()** yöntemi **java.lang.Number** içinde tanımlanmıştır ve **Rational** içinde geçersiz kılınmıştır.

Bir dize, artı işareti (+) kullanılarak bir nesne ile birleştirildiğinde, nesnenin **toString()** yönteminden elde edilen dize temsili dize ile birleştirilmek için kullanılır. Yanırla **r1 + r2 + " = " + r1.add(r2), r1.toString() + " + " + r2.toString() + " = " + r1.add(r2).toString()** ile eşdeğerdir.

Rasyonel sınıf, Liste 15.13'te uygulanmıştır.

### LİSTE 15.13 Rasyonel.java

```
1 genel sınıf Rational, Sayıyı genişletir Karşılaştırılabilir<Rational>'i uygular {
2 // Pay ve payda için veri alanları
3 özel uzunsayaç = t0t;
4 özel uzun payda = t1t;
5
6 /** Varsayılan özelliklerle bir rasyonel oluştur */
7 publicRational() {
8 thist(t0t,t1t);
9 }
10
11 /** Belirtilen pay ve payda ile bir rasyonel oluştur */
12 publicRational(tlongtpay,tlongtpayda) {
13 longtgcd = gcd(pay, payda);
14 susuzluk.pay = ((payda > t 0 t) t 1t :to -1t) * pay / gcd;
15 thist.payda = Math.abs(payda) / gcd;
16 }
17
18 /** İki sayının GCD'sini bul */
19 özel statik longtgcd(tlongtn,tlongtd) {
20 longtn1 = Math.abs(n);
21 longtn2 = Math.abs(d);
22 inttgcd = t1t;
```

```

2324 fort(tinttk = t1t; k <= n1 && k <= n2; k++) {
25 ift(n1 %k == t0t&& n2 %k == t0t)
kedi gövdesi = k;revize ed
ildi }
2829 returntgcd;n3
0 }
3132 /** Payı döndür */
33 genel longtgetNumerator() {
34 dönüş numarası;
35 }
3637 /** Paydayı döndür */
38 public longtgetDenominator() {
39 dönüş paydası;
40 }
4142 /** Bu rasyonel sayıya bir rasyonel sayı ekle */
43 public Rational add(Rational ikinciRasyonel) {
44 longtn = pay * SecondRational.getDenominator() +
45 payda * SecondRational.getNumerator();
46 longtd = payda * SecondRational.getDenominator();
47 return new Rational(n, d);
48 }
49 /** Bu rasyonel sayıdan bir rasyonel sayı çıkar */
50 public Rational çıkarma(Rasyonel saniyeRasyonel) {
51 longtn = pay * SecondRational.getDenominator()
52 - payda * ikinciRasyonel.getNumerator();
53 longtd = payda * SecondRational.getDenominator();
54 return new Rational(n, d);
55 }
56 /** Bu rasyoneli bir rasyonel sayı ile çarp */
57 public Rational çarpma(Rasyonel saniyeRasyonel) {
58 longn = pay * ikinciRasyonel.getNumerator();
59 longtd = payda * SecondRational.getDenominator();
60 return new Rational(n, d);
61 }
62 /** Bu rasyoneli bir rasyonel sayı ile böl */
63 public Rational bölme(Rasyonel saniyeRasyonel) {
64 longn = pay * ikinciRasyonel.getDenominator();
65 longd = payda * ikinciRasyonel.numerator();
66 return new Rational(n, d);
67 }
70 }
71
72 @Geçersiz Kıl
73 public String toString() {
74 eğer(kenar == 1)
75 dönüşsayısı + t""t;
76 aksi takdirde
77 dönüşnumeratörü + t""t+ paydası;
78 }
79
80 @Override // Object sınıfındaki equals metodunu geçersiz kıl
81 public boolean equals(Diğer nesne) {
82 eğer((this.subtract((Rational)(other))).getNumerator() == 0)

```

$$\frac{a}{b} + \frac{c}{d} = \frac{tadt+tbcbd}{tadt+tbcbd}$$

$$\frac{a}{b} - \frac{c}{d} = \frac{tadt-tbcbd}{tadt-tbcbd}$$

$$\frac{a}{b} * \frac{c}{d} = \frac{tcad}{tacbd}$$

$$\frac{a}{b} \div \frac{c}{d} = \frac{tadbc}{tacbd}$$

```

83 geri dönüş tehdidi;
84 aksi takdirde
85 false'i döndür;
86 }
87
88 @Override // Number sınıfında soyut intValue metodunu uygulayın
89 genel intValue() {
90 return(tint)doubleValue();
91 }
92
93 @Override // Number sınıfında soyut floatValue metodunu uygulayın
94 genel floatfloatValue() {
95 return(float)doubValue();
96 }
97
98 @Override // Number sınıfında doubleValue metodunu uygulayın
99 genel doubledoubleValue() {
100 dönüşnumeratörü * t1.0t/ payda;
101 }
102
103 @Override // Number sınıfındaki abstract longValue metodunu uygulayın
104 genel uzunuzunDeğer() {
105 return(linkt)doubleValue();
106 }
107
108 @Override // Comparable arayüzündeki compareTo metodunu uygulayın
109 public intcompareTo(Rational o) {
110 if(tthist.subtract(o).getNumerator() > t0t)
111 return1t;
112 else if(tthist.subtract(o).getNumerator() < t0t)
113 dönüş-1t;
114 aksi takdirde
115 return0t;
116 }n117
117 }

```

Rasyonel sayı, bir **Rational** nesnesinde kapsüllenmiştir. İçsel olarak, bir rasyonel sayı en düşük terimlerinde (satır 13) temsil edilir ve pay, işaretini belirler (satır 14). Payda her zaman pozitifdir (satır 15).

gcd metodu (Rasyonel sınıfındaki 19-30. satırlar) özel bir metottur; müşteriler tarafından kullanılmak üzere tasarlanmamıştır. gcd metodu yalnızca Rasyonel sınıfı tarafından dahili kullanım içindir. gcd metodu ayrıca statiktir, çünkü herhangi bir belirli Rasyonel nesnesine bağlı değildir.

abs(x) metodu (Rasyonel sınıfındaki 20-21. satırlar) Math sınıfında tanımlanmıştır ve x'in mutlak değerini döndürür.

İki Rasyonel nesnesi birbirleriyle etkileşime geçerek toplama, çıkarma, çarpma ve bölme işlemleri gerçekleştirebilir. Bu yöntemler yeni bir Rasyonel nesnesi döndürür (43-70. satırlar).

toString ve equals yöntemleri Object sınıfında Rasyonel sınıfında geçersiz kılınmıştır (72-86. satırlar). toString() metodu bir Rasyonel nesnesinin string temsilini payda/numaratör biçiminde döndürür veya payda 1 ise yalnızca numaratör döndürür. Bu **equals(Object other)** metodu, bu rasyonel sayının diğer rasyonel sayıya eşit olması durumunda true döner.

Number sınıfındaki soyut metotlar intValue, longValue, floatValue ve doubleValue, **Rational** sınıfında uygulanmıştır (satırlar 88-106). Bu metotlar, bu rasyonel sayı için t, long, float ve double değerini döner.

**Comparable** arayüzündeki **compareTo(Rational other)** metodu, bu rasyonel sayıyı diğer rasyonel sayıyla karşılaştırmak için **Rational** sınıfında uygulanmıştır (satırlar 108-116).

**Tiip**

Rational sınıfında numerator ve denominator özellikleri için get metotları sağlanmıştır, bununla birlikte set metotları sağlanmamıştır, bu nedenle bir Rational objesi oluşturulduğunda, içeriği değiştirilemez. **Rational** sınıfı değişmezdir. String sınıfı ve ilkel tür değerleri için sarmalayıcı sınıflar da değişmezdir.

değişmez

**Tiip**

Pay ve payda iki değişken kullanılarak temsil edilir. Pay ve paydayı temsil etmek için iki tam sayıdan oluşan bir dizi kullanmak mümkündür (bkz. Programlama Egzersizi 15.16). Rational sınıfındaki kamu metotlarının imzaları değiştirilmemiştir, ancak bir rasyonel sayının içsel temsili değiştirilmiştir. Bu, bir sınıfın veri alanlarının özel tutulması gerektiği fikrini açıklamak için iyi bir örnektir, böylece sınıfın uygulanması sınıfın kullanımından kapsüllenmiş olur.

kapsülleme

Rasyonel sınıfının ciddi sınırlamaları vardır ve kolayca taşabilir. Örneğin, aşağıdaki kod yanlış bir sonuç gösterecektir, çünkü payda çok büyüktür.

taşma

**genel sınıfTest {**

```
public static void main(String[] args) {Rational r1 =
 new Rational(1, 123456789);Rational r2 = new Rational(1, 123456789);Rational r3 = new Rational(1, 123456789);System.out.println("r1 * r2 * r3 is " + r1.multiply(r2.multiply(r3)));}
```

{}

```
r1 * r2 * r3 -1/2204193661661244627'dir
```



Bunu düzeltmek için, pay ve payda için BigInteger kullanarak Rasyonel sınıfını uygulayabilirsiniz (bkz. Programlama Egzersizi 15.21).

**15.30 Aşağıdaki kodun çıktısını gösterin?**

```
Rasyonel r1 = tnewtRational(t-2t,t6t);nSystem.out.println(r1.getNumerator());nSystem.out.println(r1.getDenominator());nSystem.out.println(r1.intValue());System.out.println(r1.doubleValue());
```

Point Kontrol  
MyProgrammingLab™

**15.31 Aşağıdaki kod neden yanlışdır?**

```
Rational r1 = tnewtRational(t-2t,t6t);nObject r2 = tnewtRational(t1t,t45t);nSystem.out.println(r2.compareTo(r1));
```

**15.32 Aşağıdaki kod neden yanlışdır?**

```
Nesne r1 = tnewtRational(t-2t,t6t);nRational r2 = tnewtRational(t1t,t45t);nSystem.out.println(r2.compareTo(r1));
```

### Anahtar Terimler

soyut sınıf 560  
soyut yöntem 560  
derin kopya 579  
arayüz 560

işaretçi arayüz 577  
sığ kopya 579  
alt arayüz 582

### BÖLÜM ÖZETİ

1. *Soyut sınıflar, veri ve yöntemlere sahip normal sınıflara benzer, ancak soyut sınıfların örneklerini `new` operatörü kullanarak oluşturamazsınız.*
2. Bir soyut yöntem, soyut olmayan bir sınıfta bulunamaz. Eğer bir soyut üst sınıfın alt sınıfı, üst sınıfın tüm miras alınan soyut yöntemlerini uygulamazsa, alt sınıf soyut olarak tanımlanmalıdır.
3. Soyut yöntemler içeren bir sınıf soyut olmalıdır. Ancak, herhangi bir soyut yöntem içermeyen bir soyut sınıf tanımlamak mümkündür.
4. Bir alt sınıf, üst sınıfı somut olsa bile soyut olabilir.
5. Bir arayüz, yalnızca sabitler ve soyut yöntemler içeren sınıf benzeri bir yapıdır. Birçok açıdan, bir arayüz soyut bir sınıfa benzer, ancak bir soyut sınıf sabitler ve soyut yöntemlerin yanı sıra değişkenler ve somut yöntemler de içerebilir.
6. Bir arayüz, Java'da özel bir sınıf gibi ele alınır. Her arayüz, normal bir sınıf gibi ayrı bir bytecode dosyasına derlenir.
7. `java.lang.Comparable` arayüzü, `compareTo` metodunu tanımlar. Java kütüphanesindeki birçok sınıf `Comparable`'ı uygular.
8. `java.lang.Cloneable` arayüzü bir işaretçi arayüzüdür. `Cloneable` arayüzün uygulayan sınıfın bir nesnesi kopyalanabilir.
9. Bir sınıf yalnızca bir üst sınıfı genişletebilir, ancak bir veya daha fazla arayüzü uygulayabilir.
10. Bir arayüz bir veya daha fazla arayüzü genişletebilir.

### TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

MyProgrammingLab™

### PROGRAMLAMA EGZERSİZLERİ

#### Bölümler 15.2–15.3

- \*\*15.1** (*Soyut yöntemler kullanarak fonksiyonları çizme*) Bir fonksiyonun diyagramını çizen soyut bir sınıf yazın. Sınıf aşağıdaki gibi tanımlanmıştır:

```
public abstract class AbstractDrawFunction extends JPanel {
 /** Noktaları tutmak için çokgen */
 PrivatetPolygon p = tnewtPolygon();
}
```

```

korumalı AbstractDrawFunction () {
 drawFunction();
}

/** y-koordinatını döndür */
soyut double f(double x);

/** x-koordinatları 100, 101, ..., 300 için noktaları elde et */
public void drawFunction() {
 for(tinttx = t-100t; x <= t100t; x++) {np.addP
 oint(x + t200t,t200t- (tintt)f(x));n}
}

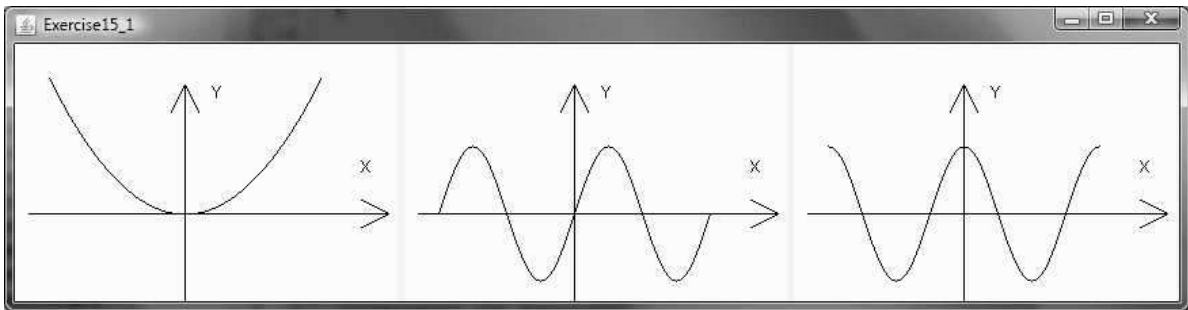
@Override/** Eksenleri, etiketleri çizin ve noktaları bağlayın */
korumalı void paintComponent(Graphics g) {
 // Tamamlanması gereken yer
}

```

Sınıfı aşağıdaki fonksiyonlarla test edin:

- a.  $f(x) = xt2t;$
- b.  $f(x) = \sin(x);$
- c.  $f(x) = \cos(x);$
- d.  $f(x) = \tan(x);$
- ve.  $f(x) = \cos(x) + 5\sin(x);$
- f.  $f(x) = 5\cos(x) + \sin(x);$
- g.  $f(x) = \log(x) + xt2t;$

Her bir fonksiyon için, AbstractDrawFunctionsınıfını genişleten ve **f** metodunu uygulayan bir sınıf oluşturun. Şekil 15.9, ilk üç fonksiyonun çizimlerini göstermektedir.



ŞEKİL 15.9 Alıştırma 15.1, kare, sinüs ve kosinüs fonksiyonlarını çizer.

**\*\*15.2** (Üçgen sınıfı) Soyut **GeometricObject** sınıfını genişleten yeni bir **Triangle** sınıfı tasarlayın. **Triangle** ve **GeometricObject** sınıfları için UML diyagramını çizin ve ardından **Triangle** sınıfını uygulayın. Kullanıcıdan üçgenin üç kenarını, bir *r* rengi ve üçgenin dolu olup olmadığını belirtmek için bir Boolean değeri girme sını isteyen bir test programı yazın. Program, bu kenarlarla bir **Triangle** nesne oluşturmalı ve girişi kullanarak renk ve dolu özelliklerini ayarlamalıdır. Program, alanı, çevreyi, rengi ve dolu olup olmadığını belirtmek için doğru veya yanlış değerini göstermelidir.

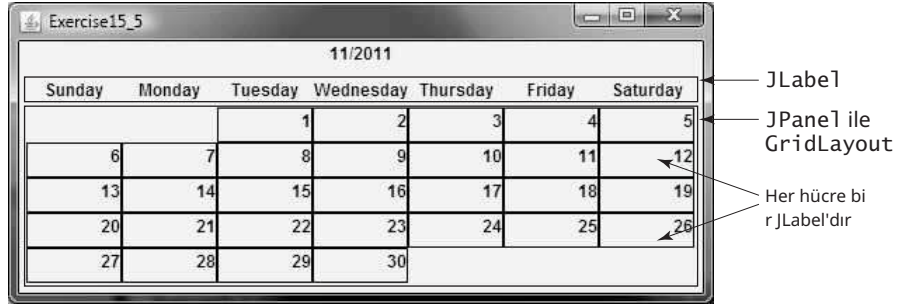
**\*15.3** (*Karıştırma ArrayList*) Aşağıdaki metodu yazın, bu bir **ArrayList** sayılarının karıştırılma sınıfı sağlar:

```
public static void shuffle(ArrayList<Number> list)
```

**\*15.4** (*Sıralama ArrayList*) Aşağıdaki metodu yazın, bu bir **ArrayList** sayılarının sıralanmasını sağlar.

```
public static void sort(ArrayList<Number> list)
```

**\*\*15.5** (*Bir takvim görüntüle*) Mevcut ayın takvimini görüntüleyen bir program yazın, Şekil 15.10'da gösterildiği gibi. Etiketleri kullanın ve etiketlerdeki metni ayı görüntülemek için ayarlayın. Bilgileri almak için **GregorianCalendar** sınıfını kullanın. ay, yıl, ayın ilk günü ve ay içindeki gün sayısı.

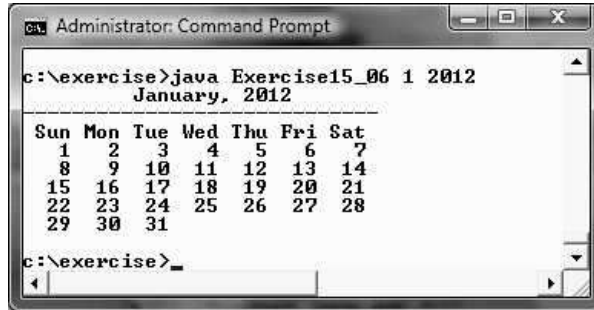


**FIGURE 15.10** Program mevcut ayın takvimini görüntüler.

**\*\*15.6** (*Takvimleri görüntüle*) Listing 5.12'deki **PrintCalendar** sınıfını yeniden yazın, belirtilen bir ay için takvim görüntülemek üzere **Calendar** ve **GregorianCalendar** sınıflarını kullanarak. Programınız komut satırından ay ve yılı alır. Örneğin:

```
java Egzersizi15_06 1 2012
```

Bu, Şekil 15.11'de gösterilen takvimi görüntüler.



**FIGURE 15.11** Program Ocak 2012 için bir takvim görüntüler.

Programı yıl olmadan da çalıştırabilirsiniz. Bu durumda, yıl mevcut yıldır. Eğer programı ay ve yıl belirtmeden çalıştırırsanız, ay mevcut aydır.



## Bölümler 15.4–15.8

- \*15.7** (*GeometricObject* karşılaştırılabilir) *GeometricObject* sınıfını *Comparable* arayüzünü uygulayacak şekilde değiştirin ve iki *GeometricObject* nesnesinin daha büyüğünü bulmak için *GeometricObject* sınıfında bir statik *max* metodu tanımlayın. UML diyagramını çizin ve yeni *GeometricObject* sınıfını uygulayın. İki dairenin daha büyüğünü ve iki dikdörtgenin daha büyüğünü bulmak için *max* metodunu kullanan bir test programı yazın.
- \*15.8** (*KarşılaştırılabilirDaire* sınıfı) *Daire* sınıfını genişleten ve *Karşılaştırılabilir* arayüzünü uygulayan bir sınıf tanımlayın. UML diyagramını çizin ve daireleri alanlarına göre karşılaştırmak için *compareTo* metodunu uygulayın. İki *KarşılaştırılabilirDaire* nesnesinin daha büyüğünü bulmak için bir test sınıfı yazın.
- \*15.9** (*Renklenebilir* arayüzü) *Renklenebilir* adında bir arayüz tasarlayın ve bir void metodu olan *howToColor()* ekleyin. *Renklenebilir* bir nesnenin her sınıfı *Renklenebilir* arayüzünü uygulamak zorundadır. *Renklenebilir* arayüzünü uygulayan ve *GeometrikNesne* sınıfını genişleten *Kare* adında bir sınıf tasarlayın. *howToColor* metodunu uygulayarak *Tüm dört kenarı renklendir* mesajını gösterin.
- İçeren bir UML diyagramı çizin *Renklenebilir*, *Kare* ve *GeometrikNesne*. Beş *GeometrikNesne* içeren bir dizi oluşturan bir test programı yazın. Dizideki her nesne için, eğer *renklenebilir* ise *howToColor* metodunu çağırın.
- \*15.10** (*MyStack* sınıfını gözden geçirin) *MyStack* sınıfını 11.9 numaralı Listede, *List* alanının derin bir kopyasını yapmak için yeniden yazın.
- \*15.11** (*Daire karşılaştırılabilir hale getirin*) 15.2 numaralı Listede *Daire* sınıfını *GeometrikNesne* sınıfını genişletecek ve *Karşılaştırılabilir* arayüzünü uygulayacak şekilde yeniden yazın. *Object* sınıfındaki *equals* metodunu geçersiz kılın. İki *Daire* nesnesi, yarıçapları aynıysa eşittir. *Daire*, *GeometrikNesne* ve *Karşılaştırılabilir* içeren UML diyagramını çizin.
- \*15.12** (*Enable Dikdörtgen karşılaştırılabilir*) 15.3 numaralı listedeki *Dikdörtgen* sınıfını *GeometricObject* sınıfından türetecek ve *Comparable* arayüzünü uygulayacak şekilde yeniden yazın. *Object* sınıfındaki *equals* metodunu geçersiz kılın. İki *Dikdörtgen* nesnesi, alanları aynıysa eşittir. *Dikdörtgen*, *GeometricObject* ve *Comparable* içeren UML diyagramını çizin.
- \*15.13** (*Sekizgen sınıfı*) *Sekizgen* adında, *GeometricObject* sınıfından türeyen ve *Comparable* ile *Cloneable* arayüzlerini uygulayan bir sınıf yazın. *Sekizgen* in tüm sekiz kenarının eşit boyutta olduğunu varsayın. Alan, aşağıdaki formül kullanılarak hesaplanabilir:

$$area = (2 + 4/\sqrt{2}) * side * side$$

*Sekizgen*, *GeometricObject*, *Comparable* ve *Cloneable* içeren UML diyagramını çizin. Kenar değeri 5 olan bir *Sekizgen* nesnesi oluşturan ve alanını ve çevresini gösteren bir test programı yazın. *clone* metodunu kullanarak yeni bir nesne oluşturun ve iki nesneyi *compareTo* metodu ile karşılaştırın.



VideoNot

Yeniden tasarlayın *Dikdörtgen* sınıfı

- \*15.14** (*Geometrik nesnelerin alanlarını toplama*) Bir dizideki tüm geometrik nesnelerin alanlarını toplayan bir metod yazın. Metod imzası şudur:

```
public static double sumArea(GeometricObject[] a)
```

İki daire ve iki dikdörtgenden oluşan dört nesnenin bir dizisini oluşturan ve *sumArea* metodunu kullanarak toplam alanlarını hesaplayan bir test programı yazın.

- \*15.15 (Kurs sınıfını klonlanabilir hale getirin) 10.6 numaralı listedeki `Kurs` sınıfını yeniden yazın `klon` metodunu eklemek için `öğrenciler` alanında derin bir kopya yapmak üzere.

### Bölüm 15.9

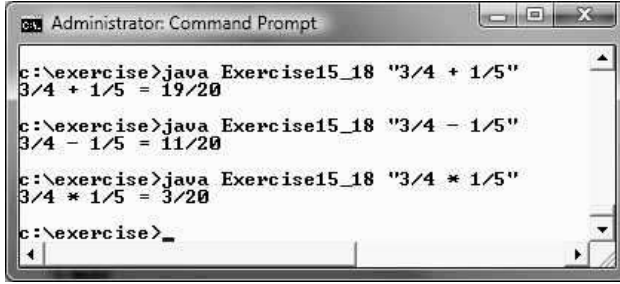
- \*15.16 (Kapsüllemenin faydalarını gösterin) 15.13 numaralı listedeki `Rasyonel` sınıfı pay ve payda için yeni bir iç temsil kullanarak yeniden yazın. Aşağıdaki gibi iki tam sayıdan oluşan bir dizi oluşturun:

özel longt[] r = tnew longt[t2t];

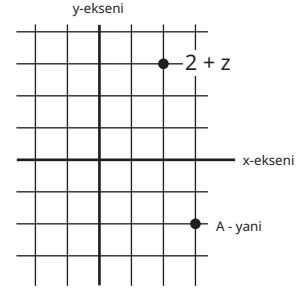
`r[0]` payı temsil etmek için ve `r[1]` paydayı temsil etmek için kullanın. `Rasyonel` sınıfındaki metodların imzaları değişmedi, bu nedenle önceki `Rasyonel` sınıfını kullanan biristemci uygulaması, bu yeni `Rasyonel` sınıfını yeniden derlenmeden kullanmaya devam edebilir.

- \*15.17 (Rasyonel sınıfı için `BigInteger` kullanın) Yeniden tasarlayın ve uygulayın 15.11 numaralı listedeki `Rasyonel` sınıfını pay ve payda için `BigInteger` kullanarak yeniden tasarlayın.

- \*15.18 (Rasyonel sayı hesaplayıcı oluşturun) Listing 9.5, `Calculator.java`'ya benzer bir program yazın. Tam sayılar yerine, Şekil 15.12a'da gösterildiği gibi rasyonel sayılar kullanın. Kesirlerin pay ve paydasını almak için, Bölüm 9.2.6, Dize Dönüştürme, Değiştirme ve Bölme'de tanıtilan `split` yöntemini kullanmanız gerekecek. Dizeyi tam sayılara dönüştürmek için `Integer.parseInt` metodu.



(a)



(b)

**Fiş 15.12(a)** Program, komut satırından üç argüman (operand1, operatör ve operand2) alır ve işlemin sonucunu gösterir. (b) Bir karmaşık sayı, bir düzlemde bir nokta olarak yorumlanabilir.

- \*15.19 (Math: Karmaşık sınıf) Bir karmaşık sayı, şeklinde bir sayıdır  $a + bi$ , burada  $a$  ve  $b$  reel sayılardır ve  $i$  sayıdır.  $a$  ve  $b$  reel sayıları, karmaşık sayının reel kısmı ve sanal kısmı olarak bilinir. Toplama, çıkarma, çarpma ve bölme işlemlerini karmaşık sayılar için aşağıdaki formülleri kullanarak gerçekleştirebilirsiniz:

$$a + bi + c + di = (a + c) + (b + d)i$$

$$a + bi - (c + di) = (a - c) + (b - d)i$$

$$(a + bi) * (c + di) = (ac - bd) + (bc + ad)i$$

$$(a + bi) / (c + di) = (ac + bd) / (c^2 + d^2) + (bc - ad)i / (c^2 + d^2)$$

Aşağıdaki formülü kullanarak karmaşık bir sayı için mutlak değeri de elde edebilirsiniz:

$$|a + bi| = \sqrt{a^2 + b^2}$$

(Karmaşık bir sayı,  $(a,b)$  değerlerini noktanın koordinatları olarak tanımlayarak bir düzlemde bir nokta olarak yorumlanabilir. Karmaşık sayının mutlak değeri, noktanın orijine olan mesafesine karşılık gelir; bu, Şekil 15.12b'de gösterilmiştir.)

Karmaşık sayıları temsil etmek için **Complex** adında bir sınıf tasarlayın ve karmaşık sayı işlemleri gerçekleştirmek için **add,subtract,multiply,divide** ve **abs** yöntemlerini oluşturun. Ayrıca, karmaşık bir sayı için bir dize temsili döndürmek üzere **toString** yöntemini geçersiz kılın. **toString** yöntemi, **(a + bi)** olarak bir dize döndürür. Eğer **b** 0 ise, sadece **a** döndürür.

Üç yapıcı sağlayın **Complex(a, b)**, **Complex(a)** ve **Complex()**. **Complex()**, 0 sayısı için bir **Complex** nesnesi oluşturur ve **Complex(a)**, **b** için 0 ile bir **Complex** nesnesi oluşturur. Ayrıca, **getRealPart()** ve **getImaginaryPart()** yöntemlerini sağlayın, böylece karmaşık sayının gerçek ve hayali kısmını sırasıyla döndürebiliriz.

Kullanıcıdan iki karmaşık sayı girmesini isteyen ve bunların toplama, çıkarma, çarpma ve bölme işlemlerinin sonuçlarını gösteren bir test programı yazın. İşte bir örnek çalışma:

```
İlk karmaşık sayıyı girin: 3.5 5.5 Enter
İkinci karmaşık sayıyı girin: -3.5 1 Enter
(3,5 + 5,5i) + (-3,5 + 1,0i) = 0,0 + 6,5 inç(3,5 + 5,5i) - (-3,
5 + 1,0i) = 7,0 + 4,5 inç(3,5 + 5,5i) * (-3,5 + 1,0i) = -17,7
5 + -15,75 inç(3,5 + 5,5i) / (-3,5 + 1,0i) = -0,5094 + -1,7 inç(3,5
+ 5,5i) = 6,519202405202649
```



## \*\*15.20

(Mandelbrot fraktalı) Mandelbrot fraktalı, bir Mandelbrot kümesinden oluşturulan iyi bilinen bir görüntüdür (bkz. Şekil 15.13a). Bir Mandelbrot kümesi, aşağıdaki yineleme kullanılarak tanımlanır:

$$z_{n+1} = z_n^2 + c$$

$c$ , bir karmaşık sayıdır ve yinelemenin başlangıç noktasıdır  $z_0 = 0$ . Verilen bir  $c$ , yineleme bir karmaşık sayı dizisi üretecektir:

$\{z_0, z_1, z_2, \dots, z_n, z_{n+1}, \dots\}$ . Dizinin ya sonsuza gittiği ya da  $c$  değerine bağlı olarak sınırlı kaldığı gösterilebilir. Örneğin, eğer  $c$  0 ise, dizi

$\{0, 0, 0, \dots\}$ , hangi sınırlıdır. Eğer  $c$  ise  $i$ , dizi

$\{0, i, -1, i, 0, i, -1, i, 0, i, -1, i, 0, i, \dots\}$  hangi sınırlıdır. Eğer  $c$  ise  $1 + i$ , dizi

$\{0, 1 + i, 2 + i, 3 + i, 4 + i, 5 + i, 6 + i, 7 + i, 8 + i, 9 + i, 10 + i, \dots\}$  hangi sınırsızdır. Bilindiği üzere eğer

bir karmaşık değerin mutlak değeri  $z_n$  dizide 2'den büyükse, o zaman dizi sınırsızdır. Mandelbrot kümesi,  $c$  değerinden oluşur

böylece dizi sınırlıdır. Örneğin, 0 ve  $i$  Mandelbrot kümesindedir. A

Mandelbrot resmi aşağıdaki kod kullanılarak oluşturulabilir:

```
1 class MandelbrotCanvas extends JPanel {
2 son statik int COUNT_LIMIT = 60;
```

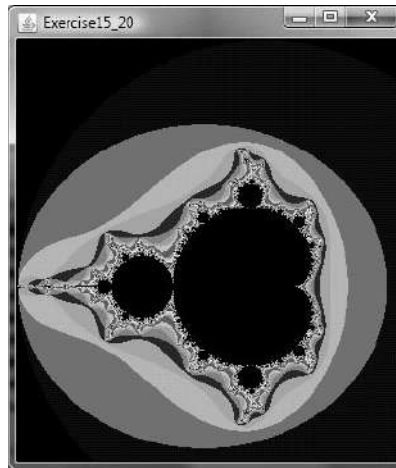
```

3
4 @Override /** Bir Mandelbrot görüntüsü çizin */
5 korumalı void paintComponent(Grafik g) {
6 super.paintComponent(g);
7
8 for(tdoubletx = t-2.0t; x < t2.0t; x += t0.01t)
9 for(tdoublety = t-2.0t; y < t2.0t; y += t0.01t) {
10 inttc = count(tnewtComplex(x, y));
11 ift(c == COUNT_LIMIT)
12 g.setColor(Color.BLACK); // c Mandelbrot kümesinde
13 aksi takdirde
14 g.setColor(tnewtColor(n15 c * t77t%t256
15 t, c * t58t%t256t, c * t159t%t256t));
16
17 g.drawRect((intt)(x * t100t) + t200t, (intt)(y * t100t) + t200t,
18 1, 1); // Belirtilen renkle küçük bir dikdörtgen doldur
19 }n20 }

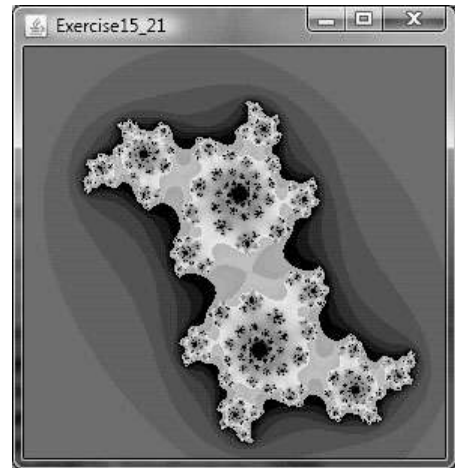
21
22 /** İterasyon sayısını döndür */
23 statik intcount(Complex c) {
24 Karmaşık z = tnewtKarmaşık(t0t,t0t);t// z0
25
26 for(inti = 0; i < COUNT_LIMIT; i++) {
27 z = z.multiply(z).add(c); // z1, z2, ... al
28 eğer(z.abs() > 2) return i; // Dizi sınırsızdır
29 }
30
31 returnCOUNT_LIMIT; // Sınırlı bir diziyi belirt
32 }
33 }

```

count(Complex c) metodu (satırlar 23–32) z1, z2, . . . , z60'ı hesaplar. Eğer hiçbir mutlak değerleri 2'yi aşmazsa, c'nin Mandelbrot kümesinde olduğunu varsayıyoruz. Elbette, bir hata her zaman olabilir, ancak 60 (COUNT\_LIMIT) yine de genellikle yeterlidir. Bir kez sıralamanın sınırsız olduğunu bulduğumuzda, metod



(a)



(b)

FIGURE 15.13A Mandelbrot görüntüsü (a)'da ve bir Julia seti görüntüsü (b)'de gösterilmektedir.

iterasyon sayısını (satır 28) döndürür. Metod, dizi

sınırlıysa COUNT\_LIMIT döndürür (satır 31).

8-9. satırlardaki döngü, her bir noktayı (x, y) inceler — t2t kahretsins kahretsini 0. — kahretsins kahretsins 2 01 aralığı ile, karşılık gelen karmaşık sayınınMandelbrot kümesinde olup c =t xt +t yti olmadığını görmek için (satır 10). Eğer öyleyse, noktayı siyah renkte boyayın (satır12). Değilse, yineleme sayısına bağlı bir renk ayarlayın (satır 15). Noktanın, genişliği u ve yüksekliği 1 olan bir kare içinde boyandığını unutmayın. Tüm noktalar400'e 400 piksel boyutunda bir ızgaraya ölçeklenir ve haritalanır (satırlar 14–15). 77,58 ve 159 değerlerinin ke yfi olarak ayarlandığını unutmayın. Yeni renkler elde etmek için farklı sayılar ayarlayabilirsiniz.

Bir Mandelbrot görüntüsü çizmek için programı tamamlayın, Şekil

15.13a'da gösterildiği gibi.

**\*\* 15.21 (Julia kümesi)** Önceki alıştırma Mandelbrot kümelerini tanımlar. Mandelbrot kümesi, dizinin sınırlı ile  $z_{t+1} = z_t^2 + c$  sabitleyip  $c$  değişken. Eğer  $c$  sabitlersek ve değiştirirsek  $z_t = 0$  ( $t = x + iy$ ), nokta (x, y), sabit bir karmaşık değer  $c$  için bir Julia kümesinde bulunuyor denir, eğer fonksiyon  $z_{t+1} = z_t^2 + c$  bağlı kalır. Julia kümesini çizmek için Alıştırma 15.20'yi gözden geçirin Şekil 15.13b'de gösterildiği gibi. [sayısını gözden geçirmeniz gerektiğini unutmayın](#) sabit bir  $c$  değeri kullanarak yöntem ( $t = 0, z_t = 0$  kalkan).

**15.22 (Rasyonel sınıfını kullanın)** Aşağıdaki toplam-

mantık serisini [Rasyonel](#) sınıfını kullanarak hesaplayan bir program yazın:

$$\frac{1}{2} + \frac{2}{3} + \frac{3}{4} + \dots + \frac{98}{99} + \frac{99}{100}$$

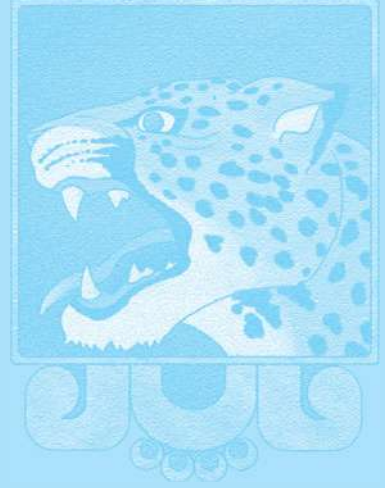
Çıktının, tam sayı taşması (çok büyük) nedeniyle yanlış olduğunu keşfedeceksiniz. Bu sorunu düzeltmek için, Programlama Egzersizi 15.17'ye bakın.

*Bu sayfa kasıtlı olarak boş bırakılmıştır*

# OLAY-TEŞE PROGRAMLAMA

## Hedefler

- Olay odaklı programlamanın tadını almak (§16.1).
- Olayları, olay kaynaklarını ve olay sınıflarını tanımlamak (§16.2).
- Dinleyici sınıflarını tanımlamak, dinleyici nesnelerini kaynak nesneye kaydetmek, ve olayları işlemek için kod yazmak (§16.3).
- İç içe sınıflar kullanarak dinleyici sınıflarını tanımlamak (§16.4).
- Anonim iç içe sınıflar kullanarak dinleyici sınıflarını tanımlamak (§16.5).
- Dinleyici sınıflarını oluşturma ve kaydetme için çeşitli kodlama stillerini keşfetmek (§16.6).
- Bir kredi hesaplayıcı için bir GUI uygulaması geliştirmek (§16.7).
- `MouseEvent`s ile başa çıkmak için programlar yazmak (§16.8).
- Dinleyici arayüz adaptörlerini kullanarak dinleyici sınıflarının kodlamasını basitleştirmek için (§16.9).
- `KeyEvent`s (§16.10) ile başa çıkmak için programlar yazmak.
- Animasyonları kontrol etmek için `javax.swing.Timer` sınıfını kullanmak (§16.11).



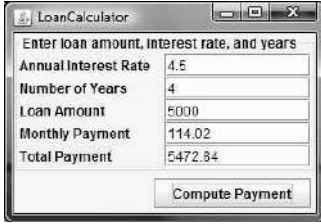
## 16.1 Giriş



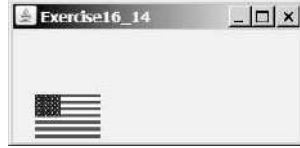
Bir düğme tıklaması veya bir zamanlayıcı gibi olayları işlemek için kod yazabilirsiniz.

problem

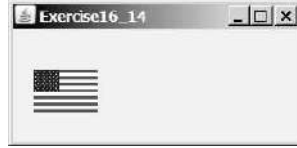
Kullanıcının bir kredi miktarı, yıllık faiz oranı ve yıl sayısını girmesine ve aylık ödeme ve toplam ödemenin elde etmek için *Compute Payment* düğmesine tıklamasına izin veren bir GUI programı yazmak istediğinizi varsayalım. Bunu nasıl başarabilirsiniz? Düğmeye tıklama olayın a yanıt vermek için kod yazmak üzere *olay tabanlı programlama* kullanmalısınız.



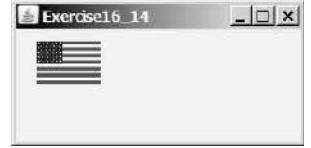
(a)



(b)



(c)



(d)

**FIGURE 16.1** (a) Program kredi ödemelerini hesaplar. (b)–(d) Bir bayrak yukarı doğru yükseliyor.

Bir bayrağın yükselmesini animasyon haline getiren bir program yazmak istediğinizi varsayalım. Bunu nasıl başarabilirsiniz? Bunu programlamanın birkaç yolu vardır. Etkili bir yol, bu bölümün nususunu oluşturan olay tabanlı programlamada bir zamanlayıcı kullanmaktır.

sorun

Olay odaklı programlamaya dalmadan önce, basit bir örneklerle bir tat almak faydalıdır. Örnek, Şekil 16.2'de gösterildiği gibi bir çerçevede iki düğme görüntüler.



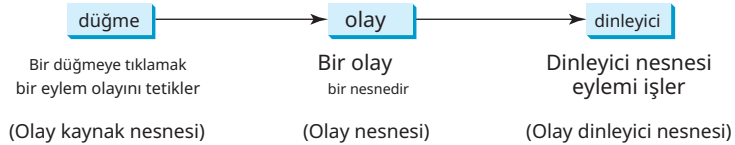
(a)



(b)

**FIGURE 16.2** (a) Program iki düğme gösterir. (b) Bir düğmeye tıklandığında konsolda b ir mesaj görüntülenir.

Bir düğme tıklamasına yanıt vermek için, düğme tıklama eylemini işlemek için kod yazmanız gerekir. Düğme, *olay kaynağı nesnesi*—eylemin kaynağıdır. Bir düğmedeki eylem olay ını işleyebilen bir nesne oluşturmanız gerekir. Bu nesne, Şekil 16.3'te gösterildiği gibi bir *olay dinleyicisi* olarak adlandırılır.



**FIGURE 16.3** Bir dinleyici nesnesi, kaynak nesneden tetiklenen olayı işler.

Tüm nesneler bir eylem olayı için dinleyici olamaz. Bir eylem olayının dinleyicisi olmak için iki gereksinim karşılanmalıdır:

ActionListener arayüzü

1. Nesne, **ActionListener** arayüzünün bir örneği olmalıdır. Bu arayüz, tüm eylem dinleyicileri için ortak davranış tanımlar.

addActionListener(dinleyici)

2. **ActionListener** nesne **dinleyici**, olay kaynağı nesnesi ile **source.addActionListener(listener)** yöntemi kullanılarak kaydedilmelidir.



**ActionListener** arayüzü, eylemGerçekleştir yöntemini içerir ve bu yöntemle işlemi için kullanılır. Dinleyici sınıfınız, olaya yanıt vermek için bu yöntemi geçersiz kılmalıdır. Liste 16.1, iki düğmedeki **ActionEvent**'i işleyen kodu verir. *OK* düğmesine tıkladığınızda, "OK düğmesine tıklandı" mesajı görüntülenir. *Cancel* düğmesine tıkladığınızda, "İptal düğmesine tıklandı" mesajı görüntülenir; bu, Şekil 16.2'de gösterilmiştir.

## Listing 16.1 tHandleEvent.java

```

1 import javax.swing.*;
2 import java.awt.event.*;
3
34 kamu sınıfı tHandleEvent extends JFrame {
5 public tHandleEvent() {
6 // İki buton oluştur
7 JButton jbtOK = new JButton("OK"); JButton jbtCancel = new JButton("Cancel");
8
9 // Butonları tutacak bir panel oluştur
10 JPanel paneli = new JPanel(); panel.add(jbtOK);
11 panel.add(jbtCancel);
12 add(panel); // Paneli çerçeveye ekle
13
14 // Dinleyicileri kaydet
15 tOKListenerClass dinleyici1 = new tOKListenerClass(); // dinleyici oluştur
16 tCancelListenerClass dinleyici2 = new tCancelListenerClass(); // dinleyici kaydet
17 jbtOK.addActionListener(dinleyici1);
18 jbtCancel.addActionListener(dinleyici2);
19 }
20
21 public static void tmain(String[] args) {
22 JFrame çerçevesi = new tHandleEvent();
23 çerçevesi.setTitle("Etiket Yönetim");
24 çerçevesi.setSize(200, 150);
25 çerçevesi.setLocation(200, 100);
26 çerçevesi.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27 çerçevesi.setVisible(true);
28 }
29
30 sınıf tOKListenerClass implements ActionListener { // dinleyici sınıf
31 @Geçersiz Kıl
32 public void tactionPerformed(ActionEvent e) { // olay işle
33 System.out.println("OK butonuna tıklandı");
34 }
35 }
36
37 sınıf tCancelListenerClass implements ActionListener { // dinleyici sınıf
38 @Geçersiz Kıl
39 public void tactionPerformed(ActionEvent e) { // olay işle
40 System.out.println("İptal butonuna tıklandı");
41 }
42 }
43 }

```

İkinci dinleyici sınıfı 34-46. satırlarda tanımlanmıştır. Her dinleyici sınıfı **ActionListener** olayını işlemek için **ActionEvent** nesnesi **tOKListenerClass** (18. satır) örneğidir ve **jbtOK** butonuna (20. satır) kaydedilmiştir. *OK* butonuna tıkladığında, **tactionPerformed(ActionEvent)** metodu (36. satır) içinde

OKDinleyiciSınıfı olay işlemek için çağrılır. `dinleyici2` nesnesi, `İptalDinleyiciSınıfı` (satır 19) örneğidir ve `jbtCancel` butonuna satır 21'de kaydedilmiştir. `İptal` butonuna tıklandığında, `actionPerformed(ActionEvent)` metodu (satır 43) `İptalDinleyiciSınıfı` içinde olay işlemek için çağrılır.

Artık Java'da olay odaklı programlamanın bir kesitini gördünüz. Muhtemelen birçok sorunuz var, örneğin neden bir dinleyici sınıfının `ActionListener`'i uygulamak için tanımlandığı. Aşağıdaki bölümler size tüm cevapları verecektir.

## 16.2 Olaylar ve Olay Kaynakları



*Bir olay, bir olay kaynağından oluşturulan bir nesnedir. Bir olayı tetiklemek, bir olay oluşturmak ve dinleyiciyi olayı işlemek için devretmek anlamına gelir.*

olay odaklı programlama  
event

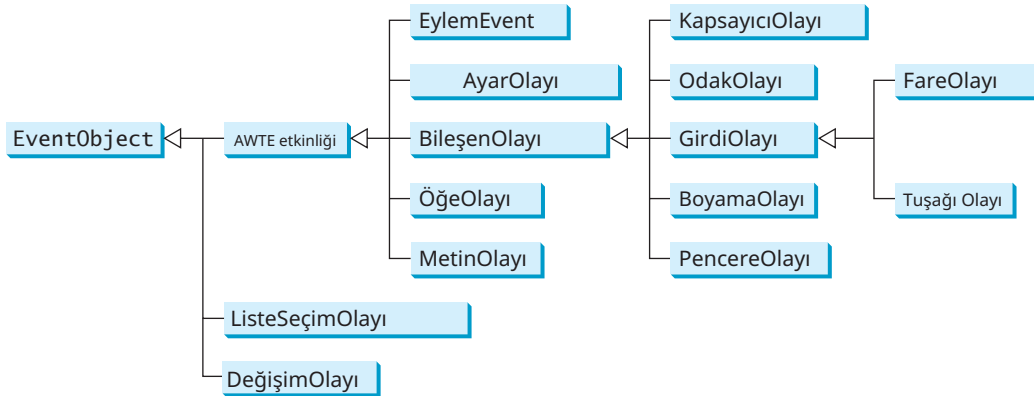
yangın olayı

olay kaynak nesnesi

kaynak nesne

Bir Java GUI programı çalıştırdığınızda, program kullanıcı ile etkileşime girer ve olaylar onun yürütülmesini yönlendirir. Bu, olay odaklı programlama olarak adlandırılır. Bir olay, programa bir şeyin olduğunu bildiren bir sinyal olarak tanımlanabilir. Olaylar, fare hareketleri, buton tıklamaları ve tuş vuruşları gibi dış kullanıcı eylemleri veya bir zamanlayıcı gibi iç program etkinlikleri tarafından tetiklenir. Program, bir olaya yanıt vermeyi veya onu görmezden gelmeyi seçebilir. Önceki bölümdeki örnek, size olay odaklı programlamanın bir tadını verdi.

Bir olayı oluşturan ve tetikleyen bileşene olay kaynağı nesnesi veya kısaca kaynak nesnesi veya kaynak bileşeni denir. Örneğin, bir buton, bir butona tıklama eylemi olayı için kaynak nesnedir. Bir olay, bir olay sınıfının bir örneğidir. Olay sınıflarının kök sınıfı `java.util.EventObject`'tır. Bazı olay sınıflarının hiyerarşik ilişkileri Şekil 16.4'te gösterilmiştir.



**Fiçin 16.4** Bir olay, `OlayNesnesi` sınıfının bir nesnesidir.

olay nesnesi  
`getSource()`

Bir *olay nesnesi* olayla ilgili olan tüm özellikleri içerir. Olayın kaynak nesnesini `getSource()` örnek yöntemi kullanarak `EventObject` sınıfında tanımlayabilirsiniz. `EventObject` sınıfının alt sınıfları, eylem olayları, pencere olayları, bileşen olayları, fare olayları ve tuş olayları gibi belirli olay türleriyle ilgilenir. Tablo 16.1'deki ilk üç sütun, bazı dış kullanıcı eylemlerini, kaynak nesneleri ve tetiklenen olay türlerini listelemektedir. Örneğin, bir düğmeye tıkladığınızda, düğme bir `ActionEvent` oluşturur ve tetikler; bu, bu tablonun ilk satırında belirtilmiştir. Burada düğme bir olay kaynak nesnesidir ve bir `ActionEvent`, kaynak nesne tarafından tetiklenen olay nesnesidir; bu, Şekil 16.2'de gösterilmiştir.



### Not

Bir bileşen bir olayı tetikleyebiliyorsa, bileşenin herhangi bir alt sınıfı aynı türde bir olayı tetikleyebilir. Örneğin, her GUI bileşeni `MouseEvent` ve `KeyEvent` tetikleyebilir, çünkü `Component` tüm GUI bileşenlerinin üst sınıfıdır.

**TABLO 16.1** Kullanıcı Eylemi, Kaynak Nesne, Olay Türü, Dinleyici Arayüzü ve İşleyici

| Kullanıcı Eylemi                 | Kaynak Nesne | Olay Türü             | Ateşlenen Dinleyici Arayüzü    | Dinleyici Arayüzü Yöntemleri                                         |
|----------------------------------|--------------|-----------------------|--------------------------------|----------------------------------------------------------------------|
| Bir düğmeye tıklayın             | JDüğmesi     | EylemEvent            | Eylem Dinleyici                | <b>actionPerformed(ActionEvent e)</b>                                |
| Bir metin alanında Enter'a basın | JTextField   | EylemEvent            | Eylem Dinleyici                | <b>actionPerformed(ActionEvent e)</b>                                |
| Yeni bir öğe seçin               | JComboBox    | EylemEventnItemEvent  | ActionListener<br>ItemListener | actionPerformed(ActionEvent e)<br>nitem<br>StateChanged(ItemEvent e) |
| Seçili veya seçili değil         | JRadioButton | ActionEventnItemEvent | ActionListener<br>ItemListener | actionPerformed(ActionEvent e)<br>nitem<br>StateChanged(ItemEvent e) |
| Seçili veya seçili değil         | JCheckBox    | EylemEventnItemEvent  | ActionListener<br>ItemListener | actionPerformed(ActionEvent e)<br>nitem<br>StateChanged(ItemEvent e) |
| Yeni bir öğe seçin               | JComboBox    | EylemEventnItemEvent  | ActionListener<br>ItemListener | actionPerformed(ActionEvent e)<br>nitem<br>StateChanged(ItemEvent e) |
| Fare basıldı                     | Bileşen      | FareOlayı             | FareDinleyici                  | fareBasıldı(FareOlayı e)                                             |
| Fare bırakıldı                   |              |                       |                                | fareBırakıldı(FareOlayı e)                                           |
| Fare tıklandı                    |              |                       |                                | fareTıklandı(FareOlayı e)                                            |
| Fare girdi                       |              |                       |                                | fareGirdi(FareOlayı e)                                               |
| Fare çıkış yaptı                 |              |                       |                                | fareCikti(MouseEvent e)                                              |
| Fare hareket etti                |              |                       | MouseMotionListener            | fareHareketEtti(MouseEvent e)                                        |
| Fare sürüklendi                  |              |                       |                                | fareSuruklendi(MouseEvent e)                                         |
| Tuşa basıldı                     | Bileşen      | Tuşağı Olayı          | Anahtar Dinleyici              | tuşaBasıldı(KeyEvent e)                                              |
| Tuş bırakıldı                    |              |                       |                                | <b>keyReleased(KeyEvent e)</b>                                       |
| Tuş basıldı                      |              |                       |                                | tuşaBasıldı(KeyEvent e)                                              |

**Not**

Şekil 16.4'teki tüm olay sınıfları, **java.awt.event** paketinde yer almaktadır. **ListSelectionEvent** ve **ChangeEvent** hariç, bunlar **javax.swing.event** paketindedir. AWT olayları başlangıçta AWT bileşenleri için tasarlanmıştı, birçok Swing bileşeni bunları tetikler.

**16.1** Bir olay kaynak nesnesi nedir? Bir olay nesnesi nedir? Bir olay kaynak nesnesi ile bir olay nesnesi arasındaki ilişkiyi tanımlayın.

**16.2** Bir düğme **MouseEvent** tetikleyebilir mi? Bir düğme **KeyEvent** tetikleyebilir mi? Bir düğme bir **ActionEvent** tetikleyebilir mi?

Point Kontrol

MyProgrammingLab™

## 16.3 Dinleyiciler, Kayıtlar ve Olayları Yönetme

*Bir dinleyici, bir olay kaynak nesnesi ile kaydedilmesi gereken bir nesnedir ve uygun bir olay işleme arayüzünün bir örneği olmalıdır.*

Java, olay işleme için bir delegasyon tabanlı model kullanır: bir kaynak nesne bir olay tetikler ve olaya ilgi duyan bir nesne bunu işler. Bu nesneye olay dinleyicisi veya kısaca dinleyici denir. Bir nesnenin bir kaynak nesnedeki bir olay için dinleyici olabilmesi için iki şeye ihtiyaç vardır, Şekil 16.5'te gösterildiği gibi.

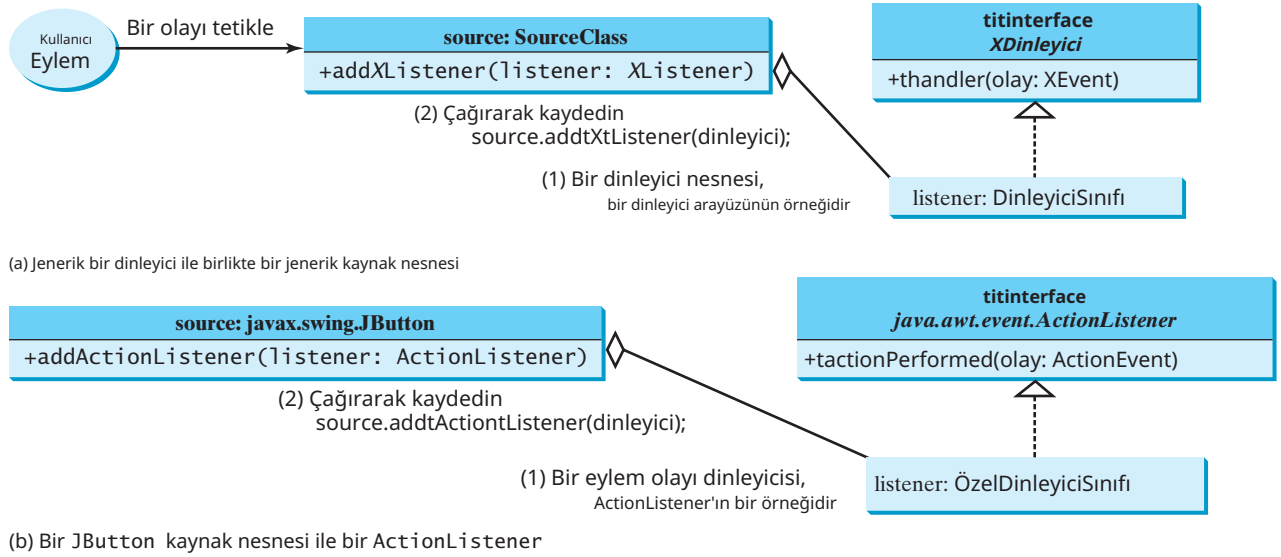
1. *Dinleyici nesnesi, olayın işlenmesi için doğru metoda sahip olduğundan emin olmak için ilgili olay-dinleyici arayüzünün bir örneği olmalıdır.* Java, her tür olay için bir dinleyici arayüzü sağlar. Dinleyici arayüzü genellikle **XListener** için **XEvent**, **MouseMotionListener** hariç. Son



event delegation  
olay dinleyici

olay-dinleyici arayüzü

XListenert/XEvent

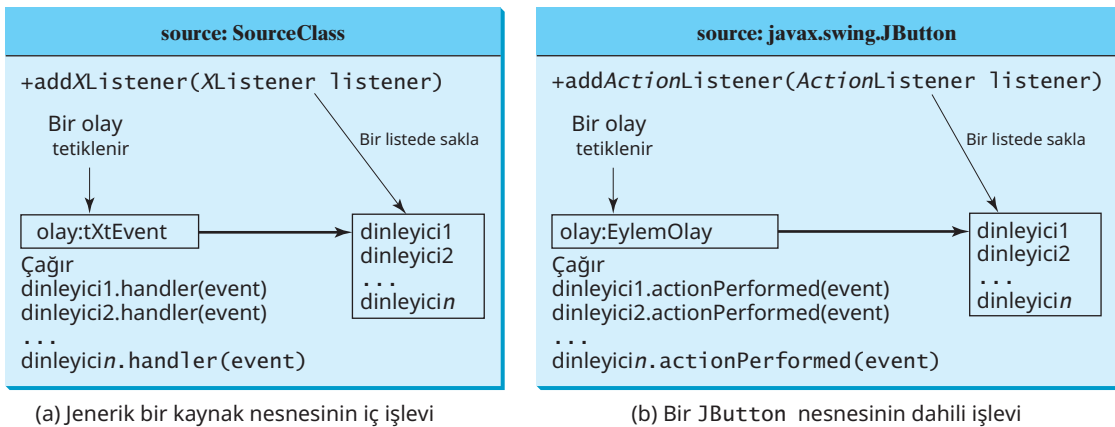


**FIGURE 16.5** Bir dinleyici, bir dinleyici arayüzünün bir örneği olmalı ve bir kaynak nesnesi ile kaydedilmelidir.

Tablo 16.1'deki üç sütun, olay türlerini, karşılık gelen dinleyici arayüzlerini ve dinleyici arayüzlerinde tanımlanan yöntemleri listelemektedir. Dinleyici arayüzü, olayı işlemek için *olay işleyici(leri)* olarak bilinen yöntem(ler)i içerir. Örneğin, bu tablonun ilk satırında gösterildiği gibi, karşılık gelen dinleyici arayüzü *ActionEvent*, *ActionListener*'dir; her *ActionEvent* dinleyicisi *ActionListener* arayüzünü implemente etmelidir; *ActionListener* arayüzü, bir *ActionEvent*'i işlemek için handler *actionPerformed(ActionEvent)*'i içerir.

dinleyici kaydet

2. *Dinleyici nesnesi, kaynak nesne tarafından kaydedilmelidir.* Kayıt yöntemleriyleşya türüne bağlıdır. *ActionEvent* için, yöntem *addActionListener* 'dır. Genel olarak, yöntem *addXListener* olarak adlandırılır *XEvent* için. Bir kaynak nesne, birden fazla türde olay tetikleyebilir ve her olay için kaynak nesne, kayıtlı dinleyicilerin bir listesini tutar ve bunları, dinleyici nesnesinin *handle*mı çağırarak olaya yanıt vermeleri için bilgilendirir; bu, Şekil 16.6'da gösterilmiştir. (Bu şeklin, bir kaynak sınıfının içsel uygulamasını gösterdiğini unutmayın. JButton gibi bir kaynak sınıfının nasıl uygulandığını bilmeniz gerekmez, ancak bu bilgi, Java olay odaklı programlama çerçevesini anlamınıza yardımcı olacaktır.)



**FIGURE 16.6** Kaynak nesnesi, olayın dinleyicilerini dinleyici nesnesinin işleyicisini çağırarak bilgilendirir.

Listing 16.1, `HandleEvent.java`'ya tekrar bakalım. Bir `JButton` nesnesi `ActionEvent` tetiklediğinden, `ActionEvent` için bir dinleyici nesnesi `ActionListener` sınıfının bir örneği olmalıdır, bu nedenle dinleyici sınıfı 34. satırda `ActionListener`'ı uygular. Kaynak nesne, dinleyiciyi kaydetmek için `addActionListener(listener)`'ı çağırır, şu şekilde:

```
JButton jbtOK = new JButton("OK");// Listing 16.1'deki 7. satır
```

kaynak nesne oluştur

```
OKListenerClass dinleyici1
=new OKListenerClass();// Listing 16.1'deki 18. satır
```

dinleyici nesnesi oluştur

```
jbtOK.addActionListener(listener1); // Line 20 in Listing 16.1
```

dinleyiciyi kaydet

Butona tıkladığınızda, `JButton` nesnesi bir `ActionEvent` tetikler ve dinleyicinin `actionPerformed` metodunu çağırarak olayı yönetmek için iletir.

Olay nesnesi, olaya ilişkin bilgileri içerir ve bu bilgiler, Şekil 16.7'de gösterildiği gibi yöntemler kullanılarak elde edilebilir. Örneğin, olayı tetikleyen kaynak nesneyi elde etmek için `e.getSource()` kullanabilirsiniz. Bir eylem olayı için, olayın meydana geldiği zamanı elde etmek için `e.getWhen()` kullanabilirsiniz.

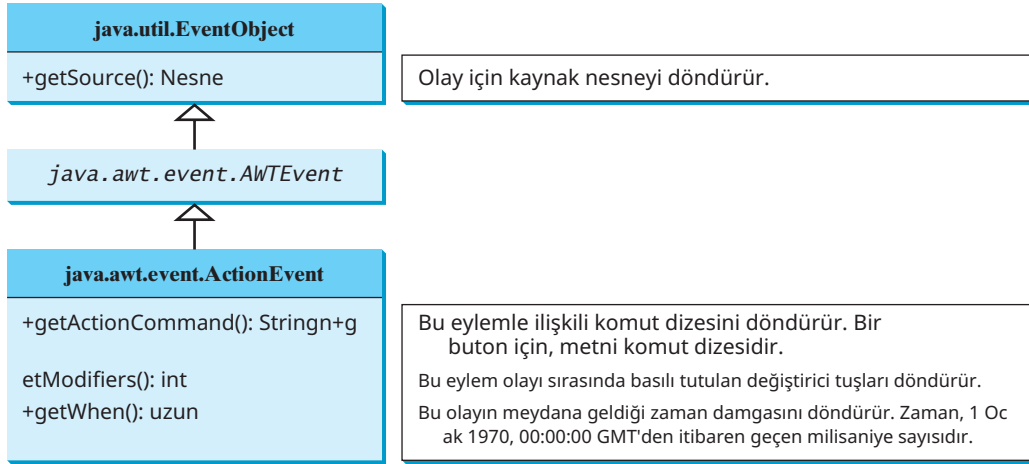


FIGURE 16.7 Bir olay nesnesinden yararlı bilgiler alabilirsiniz.

Artık bir dairenin boyutunu kontrol etmek için iki düğme kullanan bir program yazıyoruz, bu da

Şekil 16.8'de gösterilmiştir.



FIGURE 16.8 Kullanıcı, dairenin boyutunu büyütmek ve küçültmek için *Büyüt* ve *Küçült* düğmelerine tıklar.

Bu programı kademeli olarak geliştireceğiz. Öncelikle, kullanıcı arayüzünü ortada bir daire (satır 14) ve altta iki düğme (satır 15) ile gösteren Listing 16.2'deki programı yazacağız.

ilk versiyon

## LISTING 16.2tControlCircleWithoutEventHandling.java

```

1 import javax.swing.*;
2 import java.awt.*;
34
35 genel classtControlCircleWithoutEventHandling extends JFrame {
5 private JButton jbtEnlarge = new JButton("Büyüt");
9 özel JButton jbtShrink = yeniJButton("Küçült");
78 private CirclePanel canvas = new CirclePanel();
89
10 kamusalControlCircleWithoutEventHandling() {
11 JPanel panel = yeniJPanel(); // Düğmeleri gruplamak için paneli kullanın
12 panel.add(jbtEnlarge);
13 panel.add(jbtShrink);
14
15 bu.add(canvas, BorderLayout.CENTER); // Tuvali merkeze ekle
16 bu.add(panel, BorderLayout.SOUTH); // Düğmeleri çerçeveye ekle
17 }
18
19 /** Ana yöntem */
20 public static void main(String[] args) {
21 JFrame frame = yeniControlCircleWithoutEventHandling();
22 frame.setTitle("ControlCircleWithoutEventHandling");
23 frame.setLocationRelativeTo(null); // Çerçeveyi merkeze yerleştir
24 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25 frame.setSize(200, 200);
26 çerçeve.setVisible(true);
27 }
28
29 CirclePanel sınıfı
30 sınıf CirclePanel extends JPanel {
31 özel int radius = 5; // Varsayılan daire yarıçapı
32
33 @Override /** Daireyi yeniden boyayın */
34 korumalı void paintComponent(Grafik g) {
35 super.paintComponent(g);
36 g.drawOval(getWidth() / 2 - yarıçap, getHeight() / 2 - yarıçap,
37 2 * yarıçap, 2 * yarıçap);
38 }
39 }

```

Düğmeleri daireyi büyütme veya küçültme için nasıl kullanırsınız? *Büyüt* düğmesine tıkladığınızda, daireyi daha büyük bir yarıçapla yeniden boyamak istersiniz. Bunu nasıl başarabilirsiniz? Programı Liste 16.2'den Liste 16.3'e aşağıdaki özelliklerle genişletebilirsiniz:

1. **BüyütListener** adında bir dinleyici sınıfı tanımlayın ve **ActionListener**

(31–36. satırlar) arayüzünü uygulayın.

2. Bir dinleyici oluşturun ve bunu **jbtEnlarge** ile kaydedin (satır 18).

3. Yarıçapı artırmak için **CirclePanel** içinde **enlarge()** adında bir yöntem ekleyin, ardından paneli yeniden boyayın (satırlar 42–45).

4. **EnlargeListener** içinde **actionPerformed** yöntemini uygulayarak

**canvas.enlarge()** çağırısını yapın (satır 34).

5. Referans değişkeni **canvas**'ı **actionPerformed** yönteminden erişilebilir hale getirmek için, **EnlargeListener**'i **ControlCircle** sınıfının iç sınıfı olarak tanımlayın (satırlar 31–36). (İç sınıflar başka bir sınıfın içinde tanımlanır. İç sınıfları bir sonraki bölümde tanıttacağız.)

iç sınıf

6. Derleme hatalarını önlemek için, **CirclePanel** sınıfı (satırlar 38–53) artık **ControlCircle** i içinde bir iç sınıf olarak tanımlanmıştır, çünkü başka bir **CirclePanel** sınıfı zaten Liste 16.2'de tanımlanmıştır.

### LISTE 16.3 ControlCircle.java



VideoNot

Dinleyici ve kaydı

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.*;
4
5 genel classtControlCircle extends JFrame {
6 private JButton jbtEnlarge = new JButton("Büyüt");
7 özel JButton jbtShrink = yeni JButton("Küçült");
8 PrivatetCirclePanel canvas = tnewtCirclePanel();
9
10 public ControlCircle() {
11 JPanel panel = new JPanel(); // Düğmeleri gruplamak için paneli kullanın
12 panel.add(jbtEnlarge);
13 panel.add(jbtShrink);
14
15 bu.add(canvas, BorderLayout.CENTER); // Tuvali merkeze ekle
16 this.add(panel, BorderLayout.SOUTH); // Düğmeleri çerçeveye ekle
17
18 jbtEnlarge.addActionListener(tnewtEnlargeListener());
19 }
20
21 /** Ana yöntem */
22 public static void tmain(String[] args) {
23 JFrame frame = new ControlCircle();
24 frame.setTitle("ControlCircle");
25 frame.setLocationRelativeTo(null); // Çerçeveyi ortalama
26 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27 frame.setSize(200, 200);
28 çerçeve.setVisible(true);
29 }
30
31 rgeListener implements ActionListener { // İç sınıf
32 @Override
33 public void tactionPerformed(ActionEvent e) {nclasstEnla
34 canvas.enlarge();
35 }
36 }
37
38 classtCirclePanel extends JPanel {n} // İç sınıf
39 özel intradius = 5; // Varsayılan daire yarıçapı
40
41 /** Daireyi büyüt */
42 genel void tnenlarge() {
43 a(); nradius++;
44 yeniden boyam
45 }
46
47 @Geçersiz Kıl
48 korumalı void tpaintComponent(Grafik g) {
49 super.paintComponent(g);
50 g.drawOval(getWidth() / 2 - yarıçap, getHeight() / 2 - yarıçap,
51 2 * yarıçap, 2 * yarıçap);
52 }
53 }
54 }
```

dinleyici oluştur/kaydet

dinleyici sınıfı

CirclePanel sınıfı

büyüt metodu

theShrink düğmesi

Aynı şekilde, *Küçült* butonu için kodu ekleyebilirsiniz, böyleceKüçült butonuna tıkladığında daha küçük bir daire görüntülenir.

Point Kontrol

MyProgrammingLab™

- 16.3** Bir dinleyicinin uygun bir dinleyici arayüzünün örneği olması neden gereklidir? Bir dinleyici nesnesini nasıl kaydedeceğinizi ve bir dinleyici arayüzünü nasıl uygulayacağınızı açıklayın.
- 16.4** Bir kaynağın birden fazla dinleyicisi olabilir mi? Bir dinleyici birden fazla kaynağı dinleyebilir mi? Bir kaynak kendisi için bir dinleyici olabilir mi?
- 16.5** Dinleyici arayüzünde tanımlanan bir metodu nasıl uygularsınız? Dinleyici arayüzünde tanımlanan tüm metodları uygulamanız gerekiyor mu?
- 16.6** Bir eylem olayı için zaman damgasını almak için hangi metodu kullanırsınız?

## 16.4 İç Sınıflar



*Bir iç sınıf, başka bir sınıfın kapsamı içinde tanımlanan bir sınıftır. İç sınıflar, dinleyici sınıflarını tanımlamak için faydalıdır.*

Bu bölümde iç sınıfları tanıtıyoruz ve bir sonraki bölümde anonim iç sınıfları ele alıyoruz ve bunları dinleyici sınıflarını tanımlamak için kullanıyoruz. Öncelikle Şekil 16.9'daki kodu görelim. Şekil 16.9a'daki kod, Test ve A adında iki ayrı sınıf tanımlar. Şekil 16.9b'deki kod, A'yı Test içinde bir iç sınıf olarak tanımlar.

```
genel sınıfTest {n...
}

genel sınıf A {
 ...
}
```

(a)

```
genel sınıfTest {n...

 // İç sınıf
 genel sınıf A {
 ...
 }
}
```

(b)

```
// OuterClass.java: iç sınıf demo
public classtOuterClass {nprivat
 e inttdata;

 /** Dış sınıftaki bir yöntem */
 public voidm() {
 // Bir şey yap
 }

 // İç sınıf
 classİçSınıf {
 /** İç sınıftaki bir yöntem */
 genel voidtmi() {
 // Dış sınıfta tanımlanan veri ve yönteme doğrudan referans
 // verir
 ++ aldım;
 M();
 }
 }
}
```

(c)

**FIGURE 16.9** İç sınıflar, bağımlı sınıfları ana sınıf içinde birleştirir.

Şekil 16.9c'de OuterClass içinde tanımlanan InnerClass, bir iç sınıfın başka bir örneği dir. Bir iç sınıf, normal bir sınıf gibi kullanılabilir. Genellikle, bir sınıf yalnızca dış sınıfı tarafından kullanılıyorsa iç sınıf olarak tanımlanır. Bir iç sınıfın aşağıdaki özellikleri vardır:

- İç sınıf, iç sınıf adıyla adlandırılan bir sınıfa derlenir OuterClassName\$InnerClassName.class. Örneğin, iç sınıf A Test içinde Test\$A.class olarak derlenir, Şekil 16.9b.
- İç sınıf, bulunduğu dış sınıfta tanımlanan verilere ve yöntemlere referans verebilir, bu nedenle dış sınıfın bir nesnesinin referansını iç sınıfın yapıcısına geçirmenize gerek yoktur. Bu nedenle, iç sınıflar programları basit ve özlü hale getirebilir.



Örneğin, canvas, Liste 16.3'te (satır 8) ControlCircle içinde tanımlanmıştır. İç sınıf EnlargeListener içinde satır 34'te referans verilebilir.

- İç sınıf, sınıfın bir üyesine uygulanan aynı görünürlük kurallarına tabi olan bir görünürlük belirleyici ile tanımlanabilir.
- İç sınıf **static** olarak tanımlanabilir. Bir **static** iç sınıf, dış sınıf adını kullanarak erişilebilir. Bir **static** iç sınıf, dış sınıfın statik olmayan üyelerine erişemez.
- İç sınıfın nesneleri genellikle dış sınıfta oluşturulur. Ancak, başka bir sınıftan da bir iç sınıf nesnesi oluşturabilirsiniz. Eğer iç sınıf statik değilse, önce dış sınıfın bir örneğini oluşturmalısınız, ardından iç sınıf için bir nesne oluşturmak için aşağıdaki sözdizimini kullanmalısınız:

```
OuterClass.InnerClass innerObject = externalObject.newt InnerClass();
```

- Eğer iç sınıf statikse, onun için bir nesne oluşturmak için aşağıdaki sözdizimini kullanın:

```
OuterClass.InnerClass innerObject = t.newt OuterClass.InnerClass();
```

İç sınıfların basit bir kullanımı, bağımlı sınıfları bir ana sınıf içinde birleştirmektir. Bu, kaynak dosyalarının sayısını azaltır. Ayrıca, sınıf dosyalarını düzenlemeyi kolaylaştırır, çünkü hepsi ana sınıfın ek olarak alarak adlandırılmıştır. Örneğin, Şekil 16.9a'da iki kaynak dosyası **Test.java** ve **A.java** oluşturmak yerine, sınıf **A**'yı sınıf **Test** içine birleştirip sadece bir kaynak dosyası oluşturabilirsiniz, **Test.java** Şekil 16.9b'de. Ortaya çıkan sınıf dosyaları **Test.class** ve **Test\$A.class**'dir.

İç sınıfların bir başka pratik kullanımı, sınıf adlandırma çakışmalarını önlemektir. İki versiyonu **CirclePanel**, Liste 16.2 ve 16.3'te tanımlanmıştır. Çakışmayı önlemek için bunları iç sınıflar olarak tanımlayabilirsiniz.

Bir dinleyici sınıfı, bir GUI bileşeni için bir dinleyici nesnesi oluşturmak üzere özel olarak tasarlanmıştır (örneğin, bir düğme). Dinleyici sınıfı diğer uygulamalar tarafından paylaşılmayacak ve bu nedenle çerçeve sınıfının içinde bir iç sınıf olarak tanımlanması uygundur.

**16.7** Bir iç sınıf, içinde bulunduğu sınıftan farklı bir sınıfta kullanılabilir mi?

**16.8** İç sınıflar için **public**, **private** ve **static** değiştiricileri kullanılabilir mi?

Point Kontrol  
MyProgrammingLab™

## 16.5 Anonim Sınıf Dinleyicileri

*Anonim bir iç sınıf, isimsiz bir iç sınıftır. Bir iç sınıf tanımlamayı ve sınıfın bir örneğini oluşturmayı tek bir adımda birleştirir.*

Anahtar  
Nokta

İç sınıf dinleyicileri, *anonim iç sınıflar* kullanılarak kısaltılabilir. Liste 16.3'teki iç sınıf, aşağıda gösterildiği gibi bir anonim iç sınıf ile değiştirilebilir.

anonim iç sınıf

```
public ControlCircle() {
 // Atlandı

 jbtEnlarge.addActionListener(
 yeni BüyütmeDinleyici());
}

class BüyütmeDinleyici
 implements ActionListener {
 @Override
 public void actionPerformed(ActionEvent e) {
 canvas.büyüt();
 }
}
```

(a) İç sınıf EnlargeListener

```
public ControlCircle() {
 // Atlandı

 jbtEnlarge.addActionListener(
 yeni sınıf BüyütmeDinleyici
 applicationstActionListener () {
 n@Geçersiz kıl
 public void actionPerformed(ActionEvent e) {
 canvas.büyüt();
 }
 });
}
```

(b) Anonim iç sınıf

Anonim iç sınıfın sözdizimi:

```
new SuperClassName/InterfaceName() {
 // Üst sınıf veya arayüzdeki yöntemleri uygulayın veya geçersiz kılın

 // Gerekirse diğer yöntemler
}
```

Anonim iç sınıf, özel bir iç sınıf türü olduğundan, aşağıdaki özelliklere sahip bir iç sınıf gibi muamele görür:

- Anonim bir iç sınıf her zaman bir üst sınıfı genişletmeli veya bir arayüzü uygulamalıdır, bununla birlikte açık bir **extends** veya **implements** ifadesine sahip olamaz.
- Anonim bir iç sınıf, üst sınıftaki veya arayüzdeki tüm soyut yöntemleri uygulamalıdır.
- Anonim bir iç sınıf, bir örnek oluşturmak için her zaman üst sınıfından no-arg yapıcısını kullanır. Eğer anonim bir iç sınıf bir arayüzü uyguluyorsa, yapıcı **Object()** olur.
- Anonim bir iç sınıf, `OuterClassName$.class` adıyla bir sınıfa derlenir. Örneğin, dış sınıf **Test** iki anonim iç sınıfa sahipse, bunlar **Test\$1.class** ve **Test\$2.class** olarak derlenir.

Liste 16.4, Şekil 16.10'da gösterildiği gibi dört düğmeden gelen olayları işleyen bir örnek vermektedir.



FIGURE 16.10 Program, dört butondan gelen olayları işler.



VideoNot

Anonim dinleyici

## LISTING 16.4 AnonymousListenerDemo.java

```
1 import javax.swing.*;
2 import java.awt.event.*;
3
4 genel sınıf AnonymousListenerDemo extends JFrame {
5 public AnonymousListenerDemo() {
6 // Dört buton oluştur
7 JButton jbtNew = new JButton("Yeni");
8 JButton jbtOpen = new JButton("Aç");
9 JButton jbtSave = new JButton("Kaydet");
10 JButton jbtPrint = new JButton("Yazdır");
11
12 // Butonları tutacak bir panel oluştur
13 JPanel panel = new JPanel();
14 panel.add(jbtNew);
15 panel.add(jbtOpen);
16 panel.add(jbtSave);
17 panel.add(jbtPrint);
18
19 ekle(panel);
```

```

2021 // Anonim iç sınıf dinleyicisi oluştur ve kaydet
22 jbtNew.addActionListener(yeniActionListener() {
23 @Override
24 public void actionPerformed(ActionEvent e) {
25 System.out.println("Yeni İşlem");
26 }
27 }
28);
29
30 jbtOpen.addActionListener(tnewtActionListener() {n31 @Override
32 public void actionPerformed(ActionEvent e) {
33 System.out.println("İşlem Aç");
34 }
35 }n36);
37
38 jbtSave.addActionListener(n39 @Overri yeniActionListener() {
39 de
40 public void actionPerformed(ActionEvent e) {
41 System.out.println("İşlem Kaydet");
42 }
43 }n44);
45
46 jbtPrint.addActionListener(n47 @Overrid yeniActionListener() {
47 e
48 public void actionPerformed(ActionEvent e) {
49 System.out.println("İşlem Yazdır");
50 }
51 } {Nakha}

545 /** Ana yöntem */
56 public static void tmain(String[] args) {
57 JFrame frame = new AnonymousListenerDemo();
58 frame.setTitle("AnonymousListenerDemo");
59 frame.setLocationRelativeTo(null); // Çerçeveyi ortalara
60 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
61 çerçeve.pack();
62 çerçeve.setVisible(true);n63 }

64 }

```

anonim dinleyici olayı yönet

Program, anonim iç sınıflar kullanarak dört dinleyici oluşturur (satırlar 22–52). Anonim iç sınıflar kullanmadan, dört ayrı sınıf oluşturmanız gerekirdi. Anonim bir dinleyici, bir iç sınıf dinleyicisiyle aynı şekilde çalışır. Program, anonim bir iç sınıf kullanılarak yoğunlaştırılmıştır.

Anonim iç sınıflar, **OuterClassName\$.class** olarak derlenir, burada # 1'den başlar ve derleyicinin karşılaştığı her anonim sınıf için artırılır. Bu örnekte, anonim iç sınıflar **AnonymousListenerDemo\$1.class**, **AnonymousListenerDemo\$2.class** olarak derlenir, **AnonymousListenerDemo\$3.class** ve

**AnonymousListenerDemo\$4.class** olarak derlenir.

Çerçevenin boyutunu ayarlamak için **setSize** yöntemini kullanmak yerine, program **pack()** yöntemini (satır 61) kullanır; bu yöntem, içindeki bileşenlerin boyutuna göre çerçeveyi otomatik olarak boyutlandırır. paketle()



**16.9** Eğer A sınıfı B sınıfında bir iç sınıf ise, A için .class dosyası nedir? Eğer B sınıfı iki anonim iç sınıf içeriyorsa, bu iki sınıfın .class dosyası adları nelerdir?

MyProgrammingLab™

**16.10** Aşağıdaki kodda ne yanlış?

```
import java.swing.*;
import java.awt.*;

public class TestText extends JFrame {

 JButton jbtOK = yeniJButton("OK");
 add(jbtOK);

 özel sınıf Listener
 implements ActionListener {
 public void actionPerformed
 (ActionEvent e) {
 System.out.println
 (jbtOK.getActionCommand());
 }
 }

 /** Ana yöntem atlandı */
}
```

(a)

```
import java.awt.event.*;
import javax.swing.*;

public class TestText extends JFrame {

 JButton jbtOK = yeniJButton("OK");
 add(jbtOK);
 jbtOK.addActionListener(new
 ActionListener() {
 public void actionPerformed
 (ActionEvent e) {
 System.out.println(jb
 tOK.getActionCommand());
 }
 }); // Burada bir şey eksik

 /** Ana yöntem atlandı */
}
```

(b)

## 16.11 setSize(width, height) metodu ile

pack() metodu arasındaki fark nedir JFrame?

## 16.6 Dinleyici Sınıflarını Tanımlamanın Alternatif Yolları



*Dinleyici sınıflarını tanımlamak için iç sınıf veya anonim iç sınıf kullanmak tercih edilir.*

Dinleyici sınıflarını tanımlamanın birçok başka yolu vardır. Örneğin, Liste 16.4'ü sadece bir dinleyici oluşturarak, dinleyiciyi butonlarla kaydederek ve dinleyicinin olay kaynağını—yani hangi butonun olayı tetiklediğini—belirlemesine izin vererek yeniden yazabilirsiniz; bu, Liste 16.5'te gösterilmiştir.

### Listing 16.5 DetectSourceDemo.java

```
1 import javax.swing.*;
2 import java.awt.event.*;
3
34 genel class DetectSourceDemo extends JFrame {
5 // Dört buton oluştur
6 private JButton jbtNew = new JButton("Yeni");
7 private JButton jbtOpen = new JButton("Aç");
8 private JButton jbtSave = new JButton("Kaydet");
9 private JButton jbtPrint = new JButton("Yazdır");
10
101 public DetectSourceDemo() {
12 // Butonları tutacak bir panel oluştur
13 JPanel panel = new JPanel();
14 panel.add(jbtNew);
15 panel.add(jbtOpen);
16 panel.add(jbtSave);
17 panel.add(jbtPrint);
18 }
```

```

19 ekle(panel);
2021 // Bir dinleyici oluştur
22 ButtonListener dinleyici = new ButtonListener(); // dinleyici oluştur
2324 // Dinleyiciyi butonlarla kaydet
25 jbtNew.addActionListener(dinleyici); // dinleyici kaydet
26 jbtOpen.addActionListener(dinleyici);
27 jbtSave.addActionListener(dinleyici);
28 jbtPrint.addActionListener(dinleyici);
29 }
3031 classtButtonListenertimplementstActionListener {n32 @Over // dinleyici sınıfı
ride
33 public voidtactionPerformed(ActionEvent e) { // olayı yönet
34 //
35 System.out.println("Yeni İşlem");
36 == jbtOpen)nift(e.getSource() == jbtNew)
37 System.out.println("Aç İşlemi");
38 tSource() == jbtSave)nelse ift(e.getSource()
39 System.out.println("İşlem Kaydet");
40 else ift(e.getSource() == jbtPrint)nelse ift(e.ge
41 System.out.println("İşlem Yazdırma");n42 }
43 }
4445 /** Ana yöntem */
46 public static voidtmain(String[] args) {
47 JFrame frame = new DetectSourceDemo();
48 frame.setTitle("DetectSourceDemo");
49 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
50 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51 çerçeve.pack();
52 çerçeve.setVisible(true);n53 }
54 }

```

Bu program sadece bir iç dinleyici sınıfı tanımlar (satırlar 31–43), sınıftan bir dinleyici oluşturur (satır 22) ve bunu dört düğmeye kaydeder (satırlar 25–28). Bir düğmeye tıklandığında, düğme bir **ActionEvent** tetikler ve dinleyicinin **actionPerformed** metodunu çağırır. **actionPerformed** metodu, olayın kaynağını **getSource()** metodunu kullanarak kontrol eder (satırlar 34, 36, 38, 40) ve hangi düğmenin olayı tetiklediğini belirler.

Birçok olayı işlemek için bir dinleyici sınıfı tanımlamak verimlidir. Bu durumda, sadece bir dinleyici nesnesi oluşturursunuz. Anonim iç sınıflar kullanarak, dört dinleyici nesnesi oluşturursunuz.

Ayrıca, özel çerçeve sınıfını tanımlayarak Liste 16.4'ü yeniden yazabilirsiniz

**ActionListener** uygulayan, Liste 16.6'da gösterildiği gibi.

## Listing 16.6tFrameAsListenerDemo.java

```

1 import javax.swing.*;
2 import java.awt.event.*;
34 genel sınıf tFrameAsListenerDemo extends JFrame
5 applicationstActionListener { // ActionListener'ı uygulayın
6 // Dört buton oluştur
7 private JButton jbtNew = new JButton("Yeni");
8 private JButton jbtOpen = new JButton("Aç");
9 private JButton jbtSave = new JButton("Kaydet");

```

```

10 private JButton jbtPrint = new JButton("Yazdır");
11
12 public JFrameAsListenerDemo() {
13 // Butonları tutacak bir panel oluştur
14 JPanel paneli = new JPanel();
15 panel.add(jbtNew);
16 panel.add(jbtOpen);
17 panel.add(jbtSave);
18 panel.add(jbtPrint);
19 }
20 ekle(panel);
21
22 // Düğmelerle dinleyiciyi kaydet
23 jbtNew.addActionListener(tthist);
24 jbtOpen.addActionListener(tthist);
25 jbtSave.addActionListener(tthist);
26 jbtPrint.addActionListener(tthist);
27 }
28
29 @Geçersiz Kıl /** actionPerformed'i uygula */
30 public void actionPerformed(ActionEvent e) {
31 if (e.getSource() == jbtNew)
32 System.out.println("Yeni İşlem");
33 else if (e.getSource() == jbtOpen)
34 System.out.println("Aç İşlemi");
35 else if (e.getSource() == jbtSave)
36 System.out.println("Kaydet İşlemi");
37 else if (e.getSource() == jbtPrint)
38 System.out.println("Yazdır İşlemi");
39 }
40
41 /** Ana yöntem */
42 public static void main(String[] args) {
43 JFrame frame = new JFrameAsListenerDemo();
44 frame.setTitle("FrameAsListenerDemo");
45 frame.setLocationRelativeTo(null); // Çerçeveyi ortalara
46 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47 frame.pack();
48 çerçeve.setVisible(true);
49 }
50 }

```

dinleyicileri kaydet

olayı yönet

Frame sınıfı **JFrame** sınıfından türetilir ve **ActionListener** arayüzünü uygular (satır 5), bu nedenle sınıf bir lis-tener sınıfıdır ve eylem olayları için dinleyicidir. Dinleyici dört butona kaydedilmiştir (satırlar 23–26). Bir butona tıklandığında, buton bir **ActionEvent** tetikler ve dinleyicinin **actionPerformed** metodunu çağırır. **actionPerformed** metodu, olayın kaynağını **getSource()** metodu ile kontrol eder (satırlar 31, 33, 35, 37) ve hangi butonun olayı tetiklediğini belirler.

Bu tasarım istenmeyen bir durumdur, çünkü bir sınıfa çok fazla sorumluluk yükler. Olayları ele almakla tamamen sorumlu bir dinleyici sınıfı tasarlamak daha iyidir, bu da kodun okunmasını ve bakımını kolaylaştırır.

Hangi yol tercih edilir?

Dinleyici sınıflarını ya iç sınıflar ya da anonim iç sınıflar kullanarak tanımlamalısınız—hangisinin daha kısa, daha net ve daha temiz kod ürettiğini seçin. Genel olarak, dinleyicideki kod kısa ise ve dinleyici bir olay kaynağı için kaydedilmişse anonim iç sınıfları kullanın. Dinleyicideki kod uzun ise veya dinleyici birden fazla olay kaynağı için kaydedilmişse iç sınıfları kullanın.

## 16.12 Neden özel çerçeve sınıfı tanımlamaktan kaçınmalısınız

### Eylem Dinleyici

## 16.13 Olay nesnesinden kaynak nesnesini almak için hangi yöntemi kullanıyorsunuz e?

## 16.7 Vaka Çalışması: Kredi Hesaplayıcı

Bu vaka çalışması GUI bileşenlerini ve olayları kullanmaktadır.



Şimdi bu bölümün başında sunulan kredi hesaplayıcı problemi için programı yazacağız. İşte programdaki ana adımlar:

1. Kullanıcı arayüzünü oluşturun, Şekil 16.11'de gösterildiği gibi.
  - a. 5 satır ve 2 sütundan oluşan bir **GridLayout** paneli oluşturun. Panele etiketler ve metin alanları ekleyin. Panele "Kredi miktarını, faiz oranını ve yılı girin" başlığını ayarlayın.
  - b. **FlowLayout(FlowLayout.RIGHT)** ile başka bir panel oluşturun ve panele bir buton ekleyin.
  - c. İlk paneli çerçevenin ortasına ve ikinci paneli çerçevenin güney tarafına ekleyin.
2. Olayı işleyin.

Butona tıklama eylem olayını işlemek için dinleyiciyi oluşturun ve kaydedin. İşleyici, kredi miktarı, faiz oranı ve yıl sayısı üzerindeki kullanıcı girişini alır, aylık ve toplam ödemeleri hesaplar ve değerleri metin alanlarında görüntüler.

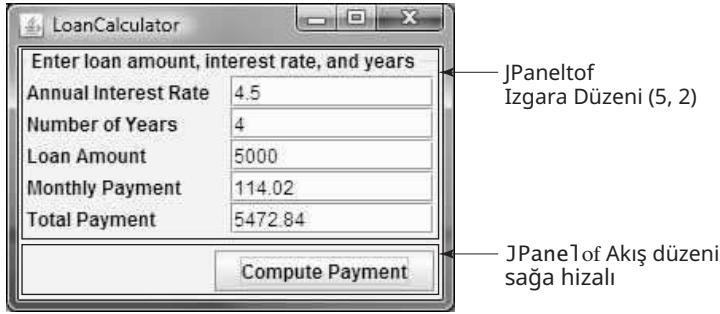


FIGURE 16.11 Program kredi ödemelerini hesaplar.

Tam program Liste 16.7'de verilmiştir.

### LISTING 16.7tLoanCalculator.java

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4 import javax.swing.border.TitledBorder;
5
6 genel sınıf KrediHesap MakinesiTextExtend JFrame {
7 // Faiz oranı, yıl,
8 // kredi miktarı, aylık ödeme ve toplam ödeme için metin alanları oluşturun
9 private JTextField jtfAnnualInterestRate = new JTextField();
10 private JTextField jtfYılSayısı = new JTextField();
11 private JTextField jtfKrediMiktarı = new JTextField();
12 private JTextField jtfAylıkÖdeme = new JTextField();
13 private JTextField jtfToplamÖdeme = new JTextField();
14
15 // Ödeme Hesapla butonunu oluştur
16 private JButton jbtKrediHesapla = new JButton("Ödeme Hesapla");

```

metin alanları

düğme

```

171819 LoanCalculator() {
20 // Etiketler ve metin alanlarını tutacak panel p1
UI oluştur JPanel p1 = new JPanel(new GridLayout(5,2));
21 p1.add(new JLabel("Yıllık Faiz Oranı"));
22 p1.add(jtfYıllıkFaizOranı);
23 p1.add(new JLabel("Yıl Sayısı"));
24 p1.add(jtfYılSayısı);
25 p1.add(new JLabel("Kredi Miktarı"));
26 p1.add(jtfLoanAmount);
27 p1.add(new JLabel("Aylık Ödeme"));
28 p1.add(jtfMonthlyPayment);
29 p1.add(new JLabel("Toplam Ödeme"));
30 p1.add(jtfTotalPayment);
31 p1.setBorder(yeni
32 TitledBorder("Kredi miktarını, faiz oranını ve yılı girin"));
33
34 // Butonu tutacak panel p2
çerçeveye ekle JPanel p2 = new JPanel(new FlowLayout(FlowLayout.RIGHT));
35 p2.add(jbtComputeLoan);
36
37 // Panelleri çerçeveye ekle
38 add(p1, BorderLayout.CENTER);
39 add(p2, BorderLayout.SOUTH);
40
41
42 // Dinleyiciyi kaydet
dinleyici kaydet jbtComputeLoan.addActionListener(new ActionListener() {
43 kanal }n
44
45 /** Ödeme Hesapla butonunu işle */
46 özel sınıf JButtonListener implements ActionListener {
47 @Geçersiz Kıl
48 public void actionPerformed(ActionEvent e) {
49 // Metin alanlarından değerleri al
girdi al double faiz =
50 Double.parseDouble(jtfAnnualInterestRate.getText());
51 int yıl = Integer.parseInt(jtfNumberOfYears.getText());
52 double krediMiktarı =
53 Double.parseDouble(jtfLoanAmount.getText());
54
55 // Bir kredi nesnesi oluştur. Kredi, Liste 10.2'de tanımlanmıştır
kredi oluştur Kredi kredi = new Kredi(faiz, yıl, krediTutarı);
56
57 // Aylık ödeme ve toplam ödemeyi göster
sonucu ayarla jtfMonthlyPayment.setText(String.format("%.2f",
58 kredi.getMonthlyPayment()));
59 }
60
61 jtfTotalPayment.setText(String.format("%.2f",
62 kredi.getTotalPayment()));
63
64 }
65
66 }
67
68 public static void main(String[] args) {
69 LoanCalculator calc = new LoanCalculator();
70 çerçeve = new JFrame();
71 çerçeve.setTitle("Loan Calculator");
72 çerçeve.setLocationRelativeTo(null);
73 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
74 çerçeve.setVisible(true);
75 }
76
77 }

```



Kullanıcı arayüzü yapıcıda oluşturulmuştur (satırlar 18–44). Düğme olayın kaynağıdır . Bir dinleyici oluşturulur ve düğmeye kaydedilir (satır 43).

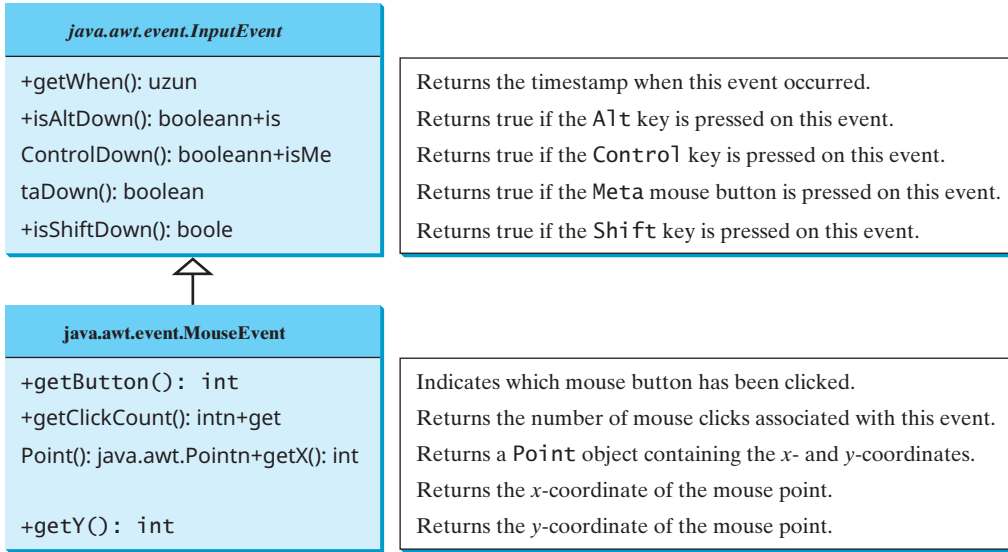
ButtonListener sınıfı (satırlar 47–66) actionPerformed metodunu uygular. Butona tıklandığında, **actionPerformed** metodu faiz oranını (51. satır), yıl sayısını (53. satır) ve kredi miktarını (54. satır) almak için çağrılır. Çağırma işlemi **jtfAnnualInterestRate.getText()** metin, **jtfYıllıkFaizOranı** metin alanında. **Kredi** sınıfı, kredi ödemelerini hesaplamak için kullanılır. Bu sınıf, Liste 10.2, Loan.java'da tanımlanmıştır. **Loan.getMonthlyPayment()** metodu çağrıldığında, kredinin aylık ödemesini döndürür (satır 62). **String.format** metodu, Bölüm 9.2.11'de tanımlanmıştır, bir sayıyı istenen bir biçime formatlamak için kullanılır ve onu bir dize olarak döndürür (satırlar 61, 63). Bir metin alanında **setText** metodunu çağırarak, metin alanında bir dize değeri ayarlar (satır 61).

## 16.8 Fare Olayları

*Bir fare olayı, bir fare düğmesine basıldığında, bırakıldığında veya tıklandığında, farenin hareket ettiğinde veya bir bileşenin üzerine sürüklendiğinde tetiklenir.*



MouseEvent nesnesi, olayı yakalar; buna bağlı tıklama sayısı, farenin konumu (x- ve y-koordinatları) veya hangi düğmenin basıldığı gibi bilgiler dahildir; bu, Şekil 16.12'de gösterilmiştir.



**FIGURE 16.12** Fare Olayı sınıfı fare olayları için bilgileri kapsar.

MouseEvent sınıfı InputEvent sınıfından türediği için, InputEvent sınıfında tanımlanan yöntemleri bir MouseEvent nesnesinde kullanabilirsiniz. Örneğin, isControlDown() yöntemi, bir MouseEvent tetiklendiğinde CTRL tuşunun basılı olup olmadığını tespit eder.

MouseEvent'te sol, orta ve sağ fare düğmelerini belirtmek için üç int sabiti—BUTTON1, BUTTON2 ve BUTTON3—tanımlanmıştır. Hangi düğmenin basıldığını tespit etmek için getButton() yöntemini kullanabilirsiniz. Örneğin, getButton() == MouseEvent.BUTTON3, sağ düğmenin basıldığını gösterir.

java.awt.Point sınıfı bir bileşen üzerindeki bir noktayı temsil eder. Sınıf, koordinatlar için iki public değişken, x ve y içerir. Bir Point oluşturmak için aşağıdaki yapıcıyı kullanın:

```
Nokta(tinttx,tintty)
```

fare düğmelerini tespit et

Point sınıfı

Bu, belirtilen x- ve y-koordinatları ile bir Point nesnesi oluşturur. Normalde, bir sınıftaki veri alanları özel olmalıdır, ancak bu sınıfın iki public veri alanı vardır.

Java, fare olaylarını işlemek için iki dinleyici arayüzü, `MouseListener` ve `MouseMotionListener` sağlar. Fareyi basma, bırakma, tıklama, girme, çıkma veya tıklama gibi eylemleri dinlemek için `MouseListener` arayüzünü, fareyi sürükleme veya hareket ettirme gibi eylemleri dinlemek için `MouseMotionListener` arayüzünü uygulayın.

|                                                                         |                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>titinterface</b><br><b><i>java.awt.event.MouseListener</i></b>       | <p><code>+mousePressed(e: MouseEvent): void</code> +mouse</p> <p><code>useReleased(e: MouseEvent): void</code> +mouse</p> <p><code>Clicked(e: MouseEvent): void</code> +mouseEnter</p> <p><code>ed(e: MouseEvent): void</code> +mouseExited(e: MouseEvent): void</p> | <p>Invoked after the mouse button has been pressed on the source component.</p> <p>Invoked after the mouse button has been released on the source component.</p> <p>Invoked after the mouse button has been clicked (pressed and released) on the source component.</p> <p>Invoked after the mouse enters the source component.</p> <p>Invoked after the mouse exits the source component.</p> |
| <b>titinterface</b><br><b><i>java.awt.event.MouseMotionListener</i></b> | <p><code>+mouseDragged(e: MouseEvent): void</code> +m</p> <p><code>ouseMoved(e: MouseEvent): void</code></p>                                                                                                                                                         | <p>Invoked after a mouse button is moved with a button pressed.</p> <p>Invoked after a mouse button is moved without a button pressed.</p>                                                                                                                                                                                                                                                     |

**FIGURE 16.13** `MouseListener` arayüzü, fare basma, bırakma, tıklama, girme ve çıkma olaylarını yönetir. `MouseMotionListener` arayüzü, fare sürükleme ve hareket etme olaylarını yönetir.

Fare olaylarını kullanmayı göstermek için, bir panelde bir mesajı görüntüleyen ve mesajın fare ile taşınmasına olanak tanıyan bir örnek veriyoruz. Mesaj, fare sürüklendiğinde hareket eder ve her zaman fare noktasında görüntülenir. Liste 16.8 programı vermektedir. Programın bir örnek çalışması Şekil 16.14'te gösterilmektedir.



**FIGURE 16.14** Mesajı fareyi sürükleyerek taşıyabilirsiniz.



VideoNot

Mesajı

fare ile taşı

### LISTING 16.8 `MoveMessageDemo.java`

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 genel class MoveMessageDemo extends JFrame {
6 public MoveMessageDemo() {

```

```

7 // Bir mesajı taşımak için MovableMessagePanel örneği oluştur
89 HareketliMesajPanel p = tnewtMovableMessagePanel
 ("Java'ya Hoş Geldiniz"); // bir panel oluştur

101
102 // Mesaj panelini çerçeveye yerleştir
12 add(p);
13 }

14
15 /** Ana yöntem */
16 public static voidtmain(String[] args) {
17 MoveMessageDemo çerçevesi = newMoveMessageDemo();
18 frame.setTitle("MoveMessageDemo");
19 frame.setSize(200, 100);
20 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
21 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22 frame.setVisible(true);
yirmiüç }

24
25 // İç sınıf: MovableMessagePanel bir mesaj çizer
2627 statik sınıftMovableMessagePanelextendstJPanel { // iç sınıf
28 privateString message = "Java'ya hoş geldiniz";
29 özel inttx = t20t;
30 özel intty = t20t;

31 /** dize s'yi çizmek için bir panel oluştur */
32 publictMovableMessagePanel(String s) {
33 mesaj = sn; // yeni bir mesaj ayarla
34 ouseMotionListener(tnewtMouseMotionListener() { // anonim dinleyici
35 @Override /** Fare sürüklenme olayını işleyin */
36 public voidtmouseDragged(MouseEvent e) {naddM // işleyiciyi geçersiz kıl
37 // Yeni konumu al ve ekranı yeniden boyala
38 x = e.getX(); // yeni konum
39 y = e.getY();
40 yeniden boyama(); // yeniden boyama
41 }
42
43 @Override /** Fare hareket ettirildiğinde olayı işleme */
44 public voidtmouseMoved(MouseEvent e) {
45 }n46);n47 }
 }

48
49 @Geçersiz Kıl
50 korumalı voidtpaintComponent(Grafik g) { // mesajı boyama
51 supert.paintComponent(g);
52 g.drawString(mesaj, x, y);n53 }

54 }n55 }

```

MovableMessagePanel sınıfı, bir mesaj çizmek için JPanel'i genişletir (satır 26). Ayrıca, fare sürüklendiğinde mesajın yeniden görüntülenmesini de yönetir. Bu sınıf, yalnızca bu sınıfta kullanıldığı için ana sınıfın içinde bir iç sınıf olarak tanımlanmıştır. Dahası, iç sınıf, ana sınıfın herhangi bir örnek üyesine referans vermediği için statik olarak tanımlanmıştır.

FareMouseMotionListener arayüzü, fare hareketi olaylarını işlemek için iki işleyici içerir: mouseMovedvemouseDragged. Bir düğmeye basılıyken fareyi hareket ettirdiğinizde, mouseDragged metodu görüntüleme alanını yeniden boyamak ve

fare noktasındaki mesaj. Düğmeye basmadan fareyi hareket ettirdiğinizde, **mouseMoved** metodu çağrılır. Çünkü dinleyici yalnızca mouse-dragged event ile ilgileniyor, **mouseDragged** metodu uygulanmıştır (satırlar 36–41).

**mouseDragged** metodu, bir düğmeye basılıyken fareyi hareket ettirdiğinizde çağrılır. Bu metod, **getX** ve **getY** metodlarını kullanarak fare konumunu alır (satırlar 38–39) ve bu, mesaj için yeni konum olur. **repaint()** metodunu çağırarak (satır 40) **paintComponent**'i çağırır (satır 50), bu da mesajı yeni bir konumda görüntüler.



MyProgrammingLab™

**16.14** Fare hareketi olayı için fare nokta konumunu almak için hangi yöntemi kullanıyorsunuz?

**16.15** Fare basıldığında, bırakıldığında, tıklanıldığında, içine girdiğinde ve çıktığında dinleyici arayüzü nedir? Fare hareket ettiğinde ve sürüklendiğinde dinleyici arayüzü nedir?

## 16.9 Dinleyici Arayüzü Adaptörleri



A dinleyici arayüzü adaptörü, *dinleyici arayüzündeki tüm yöntemler için varsayılan uygulamayı sağlayan bir sınıftır.*

dinleyici arayüzü adaptörü

Çünkü **MouseListener** arayüzündeki yöntemler soyut olduğundan, programınız bazı olaylarla ilgilenmese bile hepsini uygulamalısınız. Java, dinleyici arayüzündeki tüm yöntemler için varsayılan uygulamalar sağlayan dinleyici arayüzü adaptörleri adı verilen destek sınıfları sunar. Varsayılan uygulama basitçe boş bir gövdeyi içerir. Java, birden fazla işleyiciye sahip her AWT dinleyici arayüzü için dinleyici arayüzü adaptörleri sağlar. Bir dinleyici arayüzü adaptörü, **XDinleyici** için **XAda**ptörü olarak adlandırılır. Örneğin, **MouseEventAdapter**, **MouseListener** için bir dinleyici arayüzü adaptörüdür. Tablo 16.2, bu kitapta kullanılan bazı dinleyici arayüzü adaptörlerini listelemektedir.

**TABLO 16.2** Dinleyici Arayüzü Adaptörleri

| Adaptör           | Arayüz               |
|-------------------|----------------------|
| Fare Adaptörü     | FareDinleyici        |
| MouseEventAdapter | FareHareketDinleyici |
| TuşAdaptörü       | Anahtar Dinleyici    |
| PencereAdaptörü   | PencereDinleyici     |

**MouseEventAdapter** kullanarak, Liste 16.8'deki 34-46. satırlardaki kod (a'da gösterilmiştir) aşağıdaki kodla değiştirilebilir, (b)'de gösterildiği gibi.

```
addMouseListener(
 yeni FareHareketDinleyici() {
 @Override/** Fare sürükleme olayını işleyin */
 public void fareSürükleniyor(FareOlayı e){
 x = e.getX();
 y = e.getY();
 yenidenÇiz();
 }

 @Override/** Fare hareket ettiğinde olayı işleyin */
 public void fareHareketEtti(MouseEvent e) {
 }
 })
```

(a) Bir dinleyici arayüzü kullanma

```
addMouseListener(
 yeni MouseEventAdapter() {
 @Override/** Fare sürükleme olayını işleme */
 public void fareSürükleniyor(FareOlayı e){
 x = e.getX();
 y = e.getY();
 yenidenÇiz();
 }
 })
```

(b) Bir dinleyici arayüzü adaptörü kullanma

**16.16** Neden **ActionListener** arayüzünün dinleyici arayüzü adaptörü yok?

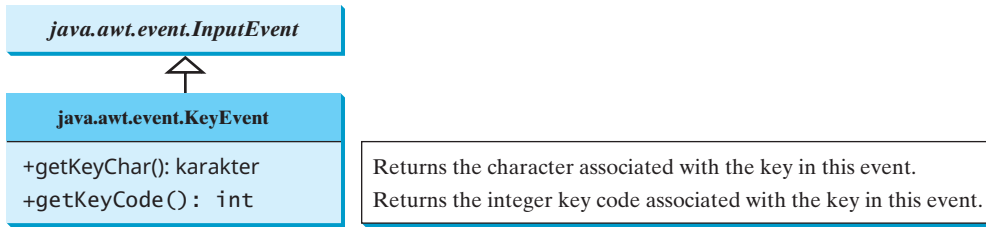
**16.17** Dinleyici arayüzü adaptörü kullanmanın avantajı nedir, dinleyici

arayüzü yerine?

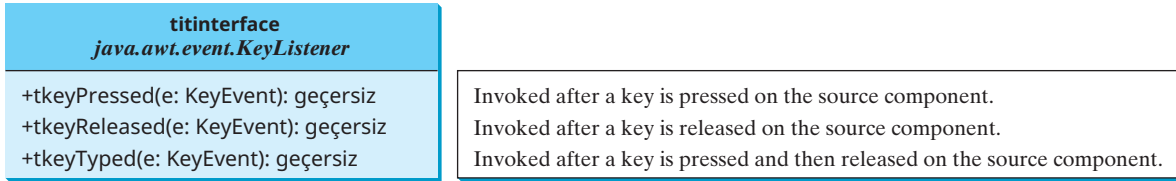
## 16.10 Anahtar Olaylar

*Bir tuş olayı, bir tuşa basıldığında, bırakıldığında veya bir bileşende yazıldığında tetiklenir.*

*Tuş olayları*, tuşların kontrol edilmesi ve eylemlerin gerçekleştirilmesi veya klavyeden girdi alınması için kullanılmalarını sağlar. **KeyEvent** nesnesi, olayın doğasını (yani, bir tuşa basıldığını, bırakıldığını veya yazıldığını) ve tuşun değerini tanımlar; bu, Şekil 16.15'te gösterilmiştir. Java, tuş olaylarını işlemek için **KeyListener** arayüzünü sağlar; bu, Şekil 16.16'da gösterilmiştir.



**FIGÜR 16.15** **KeyEvent** sınıfı, tuş olayları hakkında bilgi kapsüller.



**FIGURE 16.16** **KeyListener** arayüzü, basılan, bırakılan ve yazılan tuş olaylarını yönetir.

**keyPressed** işleyicisi bir tuşa basıldığında çağrılır, **keyReleased** işleyicisi bir tuş bırakıldığında çağrılır ve **keyTyped** işleyicisi bir tuş karakteri girildiğinde çağrılır. Eğer bir tuşun Unicode'u yoksa (örneğin, fonksiyon tuşları, değiştirici tuşlar, eylem tuşları ve kontrol tuşları), **keyTyped** işleyicisi çağrılmayacaktır.

Her tuş olayı ile ilişkili bir tuş karakteri veya tuş kodu vardır ve bu, **getKeyChar()** veya **getKeyCode()** metodu **KeyEvent** içinde. Tuş kodları, **KeyEvent** sınıfında tanımlanan sabitlerdir. Tablo 16.3 bazı sabitleri listelemektedir. Sabitlerin tam listesi için Java API'sine bakın. Unicode karakterinin tuşu için, tuş kodu Unicode değeri ile aynıdır. Tuş basıldığında ve tuş bırakıldığında olayları için, **getKeyCode()** tabloda ki tanıma göre değeri döndürür. Tuş yazıldığında olayı için, **getKeyCode()** **VK\_UNDEFINED(0)** döndürür ve **getKeyChar()** girilen karakteri döndürür.

Listing 16.9'daki program, kullanıcı tarafından girilen bir karakteri görüntüler. Kullanıcı, karakteri yukarı, aşağı, sola ve sağa, ok tuşları **VK\_UP**, **VK\_DOWN**, **VK\_LEFT** ve **VK\_RIGHT** kullanarak hareket ettirebilir. Şekil 16.17, programın bir örnek çalışmasını içermektedir.

TABLO 16.3 Tuş Sabitleri

| Sabit      | Açıklama         | Sabit           | Açıklama                                   |
|------------|------------------|-----------------|--------------------------------------------|
| VK_HOME    | Ev tuşu          | VK_SHIFT        | Shift tuşu                                 |
| VK_END     | End tuşu         | VK_BACK_SPACE   | Backspace tuşu                             |
| VK_PGUP    | Page Up tuşu     | VK_CAPS_LOCK    | Büyük Harf Tuşu                            |
| VK_PGDN    | Sayfa Aşağı tuşu | VK_NUM_LOCK     | Num Lock tuşu                              |
| VK_UP      | Yukarı ok tuşu   | VK_ENTER        | Enter tuşu                                 |
| VK_DOWN    | Aşağı ok tuşu    | VK_BELİRSİZ     | Tuş kodu bilinmiyor                        |
| VK_SOLDAN  | Sol ok tuşu      | VK_F1 ileVK_F12 | F1'den F12'ye kadar olan fonksiyon tuşları |
| VK_SAGDAN  | Sağ ok tuşu      |                 |                                            |
| VK_KAÇ     | Esc tuşu         | VK_0 ileVK_9    | 0'dan 9'a kadar olan sayı tuşları          |
| VK_TAB     | Tab tuşu         | VK_A ileVK_Z    | A'dan Z'ye kadar olan harf tuşları         |
| VK_CONTROL | Control tuşu     |                 |                                            |



**FIGİN 16.17** Program, bir karakteri göstererek ve onu yukarı, aşağı, sola veya sağa hareket ettirerek tuş olaylarına yanıt verir.

### LISTE 16.9 KeyEventDemo.java

bir panel oluştur

odaklanabilir

iç sınıf

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 genel classt KeyEventDemo extends JFrame {
6 private KlavyePanel klavyePanel = new KlavyePanel();
7
8 /** UI'yi Başlat */
9 public KeyEventDemo() {
10 // Kullanıcı girişini kabul etmek ve görüntülemek için klavye panelini ekle
11 ekle(klavyePanel);
12
13 // Odak ayarla
14 klavyePanel.setFocusable(true);
15 }
16
17 /** Ana yöntem */
18 public static void main(String[] args) {
19 KeyEventDemo çerçevesi = new KeyEventDemo();
20 çerçeve.setTitle("KeyEventDemo");
21 çerçeve.setSize(300, 300);
22 çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortalara
23 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24 çerçeve.setVisible(true);
25 }
26
27 // İç sınıf: KlavyePanel, tuş girişi almak için
28 static class KlavyePanel extends JPanel {

```

```

29 özel inttx = t100t;
30 özel inty = 100;
31 özel charkeyChar = 'A';// Varsayılan tuş
32
33 kamusalKeyboardPanel() {
34 addKeyListener(yAdapter()) {
35 @Override
36 public void keyPressed(KeyEvent e) {nnewtKe
37 anahtarlar(e.getKeyCode()) {
38 casetKeyEvent.VK_DOWN: y += t10t;tbreakt;
39 casetKeyEvent.VK_UP: y -= t10t;tbreakt;
40 casetKeyEvent.VK_LEFT: x -= t10t;tbreakt;
41 casetKeyEvent.VK_RIGHT: x += t10t;tbreakt;
42 varsayılan: keyChar = e.getKeyChar();
43 }
44 }
45 yenidençizim();
46 }
47);n48 } }

49
50 @Override /** Karakteri çiz */
51 korumalı voidtpaintComponent(Grafik g) {
52 supert.paintComponent(g);
53
54 g.setFont(tnewtFont(t"TimesRoman"t, Font.PLAIN, t24t));n55 g.drawString
(String.valueOf(keyChar), x, y);
satır {nakh
}nakh}

```

dinleyici kay

det

tuşun basıldığıı al

yeniden boyama

karakteri yeniden çiz

**KeyboardPanel** sınıfı, bir karakteri görüntülemek için **JPanel** sınıfından türetilmiştir (satır 28). Bu sınıf, yalnızca bu sınıfta kullanıldığı için ana sınıfın içinde iç sınıf olarak tanımlanmıştır. Ayrıca, iç sınıf statik olarak tanımlanmıştır çünkü ana sınıfın herhangi bir örnek üyesine referans vermez.

Program klavyeden giriş aldığı için, **KeyEvent** dinler ve tuş girişini işlemek için **KeyAdapter** sınıfını genişletir (satır 34).

Bir tuşa basıldığında, **keyPressed** handler çağrılır. Program, tuş kodunu elde etmek için **e.getKeyCode()** ve tuşun karakterini almak için **e.getKeyChar()** kullanır. Bir ok tuşu dışındaki bir tuşa basıldığında, karakter görüntülenir (satır 42). Bir ok tuşuna basıldığında, karakter ok tuşunun gösterdiği yönde hareket eder (satırlar 38–41).

Yalnızca odaklanmış bir bileşen **KeyEvent** alabilir. Bir bileşeni odaklanabilir hale getirmek için, onun **focusable** özelliğini **true** olarak ayarlayın (satır 14).

Bileşen her yeniden boyandığında, satır 54'teki **Graphics** nesnesi için yeni bir font oluşturulur. Bu verimli değildir; fontu bir kez veri alanı olarak oluşturmak daha iyi olurdu.

Artık Listing 16.3'teki **ControlCircle** örneğimiz için daha fazla kontrol ekleyebiliriz, böylece çemberin çapını sol/sağ fare düğmesine tıklayarak veya yukarı ve aşağı ok tuşlarına basarak arttırabilir/azaltabiliriz. Yeni program Listing 16.10'da verilmiştir.

## Listing 16.10 ControlCircleWithMouseAndKey.java

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.*;
4
5 genel class ControlCircleWithMouseAndKey extends JFrame {
6 private JButton jbtEnlarge = new JButton("Büyüt");
7 özel JButton jbtShrink = yeni JButton("Küçült");

```

## 624 Bölüm 16 Olay Tabanlı Programlama

```
8 PrivatetCirclePanel canvas = tnewtCirclePanel();
9
10 public tControlCircleWithMouseAndKey() {
11 JPanel panel = new JPanel(); // Düğmeleri gruplamak için paneli kullanın
12 panel.add(jbtEnlarge);
13 panel.add(jbtShrink);
14
15 bu.add(canvas, BorderLayout.CENTER); // Tuvali merkeze ekle
16 bu.add(panel, BorderLayout.SOUTH); // Düğmeleri çerçeveye ekle
17
18 jbtEnlarge.addActionListener(new ActionListener() { n19 @Override
19
20 public void actionPerformed(ActionEvent e) {
21 canvas.büyüt();
22 canvas.requestFocusInWindow();
23 }
24 });
25
26 jbtShrink.addActionListener(new ActionListener() {
27 @Override
28 public void actionPerformed(ActionEvent e) {
29 canvas.küçült();
30 canvas.requestFocusInWindow();
31 }
32 });
33
34 canvas.addMouseListener(new MouseAdapter() {
35 @Override
36 public void mouseClicked(MouseEvent e) {
37 Button() == MouseEvent.BUTTON1
38 canvas.büyüt();
39 else if (e.getButton() == MouseEvent.BUTTON3) nift(e.get
40 canvas.küçült();
41 }
42 });
43
44 canvas.setFocusable(true);
45 canvas.addKeyListener(new KeyAdapter() {
46 @Override
47 public void keyPressed(KeyEvent e) {
48 KeyCode() == KeyEvent.VK_UP
49 canvas.büyüt();
50 else if (e.getKeyCode() == KeyEvent.VK_DOWN) nift(e.get
51 canvas.küçült();
52 }
53 });
54 }
55
56 /** Ana yöntem */
57 public static void main(String[] args) {
58 JFrame çerçeve = new JFrame("ControlCircle"); // Çerçeveyi ortalama
59 çerçeve.setTitle("ControlCircle");
60 çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortalama
61 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
62 çerçeve.setSize(200, 200);
63 çerçeve.setVisible(true);
64 }
65
66 class CirclePanel extends JPanel { // İç sınıf
67 özel int radius = 5; // Varsayılan daire yarıçapı
```



```

6869 /** Daireyi büyüt */
70 genel voidtenlarge() {
71 yarıçap++;
72 yenidençiz();
73 }
7475 /** Daireyi küçült */
76 public voidküçült() {
77 eğer(yarıçap >= 1) yarıçap—;
78 yenidençiz();
79 }
80
81 @Geçersiz kıl
82 korumalı voidtpaintComponent(Grafik g) {
83 supert.paintComponent(g);
84 g.drawOval(getWidth() / 2- yarıçap, getHeight() / 2- yarıçap,
85 2 * yarıçap, 2* yarıçap);
86 }
87 }
88 }

```

Bir **MouseEvent** dinleyicisi, 34–42. satırlardaki fare tıklama olaylarını işlemek için oluşturulur. Eğer sol fare butonuna tıklanırsa, daire büyütülür (37–38. satırlar); eğer sağ fare butonuna tıklanırsa, daire küçültülür (39–40. satırlar).

Bir **KeyEvent** dinleyicisi, 45–53. satırlardaki tuşa basma olaylarını işlemek için oluşturulur. Eğer YUKARI ok tuşuna basılırsa, daire büyütülür (48–49. satırlar); eğer AŞAĞI ok tuşuna basılırsa, daire küçültülür (50–51. satırlar).

setFocusable çağırısı canvas'ı odaklanabilir hale getirir. Ancak, bir butona tıklandığında, canvas artık odaklanmaz. **canvas.requestFocusInWindow()**

(22, 30. satırlar) çağırısı, **canvas** üzerinde odaklamayı sıfırlar, böylece **canvas** tuş olaylarını dinleyebilir.

FareOlayı

Tuşağı Olayı

setFocusablenr  
equestFocusInWindow()

**16.18** Bir eylem olayı, bir fare olayı veya bir tuş olayı için zaman damgasını almak için hangi metodu kullanırsınız?

**16.19** Bir tuş olayı için anahtar karakterini almak için hangi yöntemi kullanıyorsunuz?

**16.20** Bir bileşene odaklanmak için nasıl ayarlama yaparsınız, böylece tuş olaylarını dinleyebilir?

**16.21** Klavyedeki her tuşun bir Unicode'u var mı? **KeyEvent** sınıfındaki bir tuş kodu, bir Unicode ile eşdeğer midir?

**16.22** **keyPressed** işleyicisi bir tuşa basıldıktan sonra mı çağrılır? **keyReleased** işleyicisi bir tuş bırakıldıktan sonra mı çağrılır? **keyTyped** işleyicisi herhangi

bir tuş yazıldığında mı çağrılır?

Point Kontrol

MyProgrammingLab™

## 16.11 Timer Sınıfını Kullanarak Animasyon

Bir**Timer**, sabit bir hızda **ActionEvent** tetikleyen bir kaynak nesnesidir.

Tüm kaynak nesneleri GUI bileşenleri değildir. **javax.swing.Timer** sınıfı, önceden tanımlanmış bir hızda **ActionEvent** tetikleyen bir kaynak bileşendir. Şekil 16.18, sınıftaki bazı yöntemleri listelemektedir.

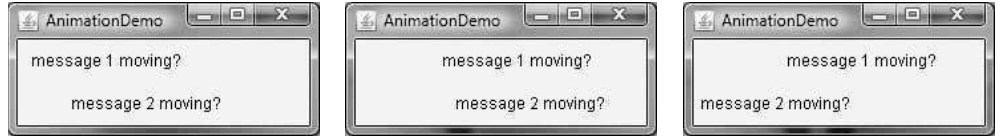
Bir**Timer** nesnesi, bir **ActionEvent** kaynağı olarak hizmet eder. Dinleyiciler, **ActionListener** örnekleri olmalı ve bir **Timer** nesnesi ile kaydedilmelidir. Bir **Timer** nesnesi, gecikme ve bir dinleyici ile tek yapıcı kullanılarak oluşturulur; burada gecikme, iki eylem olayı arasındaki milisaniye sayısını belirtir. **addActionListener** metodunu kullanarak ek dinleyiciler ekleyebilir ve **setDelay** metodunu kullanarak gecikme ayarını değiştirebilirsiniz. Zamanlayıcıyı başlatmak için **start()** metodunu çağırın; zamanlayıcıyı durdurmak için **stop()** metodunu çağırın.



| javax.swing.Timer                                       |                                                                                      |
|---------------------------------------------------------|--------------------------------------------------------------------------------------|
| +Zamanlayıcı(gecikme: int, dinleyici: ActionListener)n+ | Creates a Timer object with a specified delay in milliseconds and an ActionListener. |
| addActionListener(dinleyici: ActionListener): void+nst  | Adds an ActionListener to the timer.                                                 |
| start(): void                                           | Starts this timer.                                                                   |
| +dur(): geçersiz                                        | Stops this timer.                                                                    |
| +setDelay(gecikme: int): geçersiz                       | Sets a new delay value for this timer.                                               |

ŞEKİL 16.18 Bir Timer nesnesi sabit bir hızda bir ActionEvent tetikler.

Timer sınıfı animasyonları kontrol etmek için kullanılabilir. Liste 16.11, iki mesajı ayrı panellerde görüntüleyen bir programı vermektedir (bkz. Şekil 16.19). Her mesaj için animasyon hızını kontrol etmek için fare düğmesini kullanabilirsiniz. Sol fare düğmesine tıkladığında hız artar ve sağ düğmeye tıkladığında azalır.



ŞEKİL 16.19 İki mesaj panellerde hareket eder.

### LISTING 16.11 AnimationDemo.java

mesaj paneli oluştur

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 genel sınıf AnimasyonDemoText extends JFrame {
6 public AnimationDemo() {
7 // İki hareketli mesajı göstermek için iki MovingMessagePanel oluşturun
8 susama.setLayout(new GridLayout(2,1));
9 add(new MovingMessagePanel("mesaj 1 hareket ediyor?"));
10 add(new MovingMessagePanel("mesaj 2 hareket ediyor?"));
11 }
12
13 /** Ana yöntem */
14 public static void main(String[] args) {
15 AnimationDemo çerçeve = new AnimationDemo();
16 çerçeve.setTitle("AnimationDemo");
17 çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortala
18 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19 çerçeve.setSize(280, 100);
20 çerçeve.setVisible(true);
21 }
22
23 // İç sınıf: Hareket eden bir mesajı görüntüleme
24 static sınıf MovingMessagePanel extends JPanel {
25 private String message = "Java'ya hoş geldiniz";
26 özel int xKoordinat = 0;
27 özel int yKoordinat = 0;
28 private Timer zamanlayıcı = new Timer(1000, new TimerListener());

```

zamanlayıcı oluştur

```

2930 public MovingMessagePanel(Dize mesaj) {
31 this.mesaj = mesaj; mesaj ayarla
32
33 // Animasyon için zamanlayıcıyı başlat
34 timer.start(); zamanlayıcıyı başlat
35
36 // Fare butonlarını kullanarak animasyon hızını kontrol et
37 thist.addMouseListener(newtMouseAdapter() { fare dinleyici
38 @Geçersiz Kıl
39 public void mouseClicked(MouseEvent e) {
40 inttdelay = timer.getDelay();
41 if(e.getButton() == MouseEvent.BUTTON1)
42 setDelay(delay > t10t gecikme - t10t:t0t);
43 else if(e.getButton() == MouseEvent.BUTTON3)
44 timer.setDelay(delay < t50000t gecikme + t10t:t50000t);ntimer.
45 }n46 });n47 }

48
49 @Override /** Mesajı boyayın */
50 korumalı voidtpaintComponent(Grafik g) {
51 super.paintComponent(g);
52
53 eğer(xCoordinate > getWidth()) {
54 xCoordinate = -20; sıfırla-x-koordinatı
55 }
56 xKoordinat += t5t; mesajı taşı
57 g.drawString(mesaj, xKoordinat, yKoordinat);
58 }
59
60 classtTimerListenertimplementstActionListener {
61 @Geçersiz kıl
62 public voidtactionPerformed(ActionEvent e) {
63 yenidençizim();
64 }
65 }n66 }n
67 }

```

İki **MovingMessagePanel** örneği, iki mesajı görüntülemek için oluşturulmuştur (satırlar 9–10). **MovingMessagePanel** sınıfı, bir mesajı görüntülemek için **JPanel**'i genişletir (satır 24). Bu sınıf, yalnızca bu sınıfta kullanıldığı için ana sınıfın içinde bir iç sınıf olarak tanımlanmıştır. Ayrıca, iç sınıf statik olarak tanımlanmıştır çünkü ana sınıfın herhangi bir örnek üyesine referans vermez.

Bir iç sınıf dinleyicisi, bir zamanlayıcıdan **ActionEvent** dinlemek için 60. satırda tanımlanmıştır. 28. satırda dinleyici için bir **Timer** oluşturulur ve zamanlayıcı 34. satırda başlatılır. Zamanlayıcı bir **ActionEvent** her 1 saniyede bir başlangıçta, dinleyici 62. satırda paneli yeniden boyamak için yanıt verir. Panel boyandığında, *x*-koordinatı artırılır (56. satır), böylece mesaj sağa doğru gösterilir. *x*-koordinatı panelin sınırını aştığında, -20'ye sıfırlanır (54. satır), böylece mesaj soldan sağa doğru dairesel olarak hareket etmeye devam eder.

Panel ile fare tıklama olayını dinlemek için bir fare dinleyicisi kaydedilir (satırlar 37–46). Sol fare tuşuna tıklandığında, zamanlayıcı için yeni bir azaltılmış gecikme süresi ayarlanır (satırlar 41–42). Sağ fare tuşuna tıklandığında, zamanlayıcı için yeni bir artırılmış gecikme süresi ayarlanır (satırlar 43–44). Minimum gecikme süresi 0 ve maksimum ise **Integer.MAX\_VALUE**, ancak bu programda 50000 olarak ayarlanmıştır (satır 44).

13.9. Bölümde, Vaka Çalışması: **StillClock** Sınıfı, mevcut zamanı göstermek için bir **StillClock** çizdiniz. Saat gösterildikten sonra tık tık etmez. Ne yapabilirsiniz

saat her saniye yeni bir mevcut zamanı nasıl gösterir? Saatin çalışmasını sağlamak için, her saniye yeni bir mevcut zaman ile yeniden boyamak gerekir. Saatin yeniden boyanmasını kontrol etmek için, Liste 16.12'deki kodu kullanabilirsiniz.



#### VideoNot

Bir saati animasyonla göster

Bir saat oluştur

Zamanlayıcı oluştur

zamanlayıcıyı başlat

dinleyici sınıfı

işleyici uygulamak

yeni zamanı ayarla

yeniden boyama

### LISTE 16.12 SaatAnimasyonu.java

```

1 import java.awt.event.*;
2 import javax.swing.*;
3
4 public class SaatAnimasyonu extends JFrame {
5 private StillClock saat = new StillClock();
6
7 public SaatAnimasyonu() {
8 add(clock);
9
10 // 1000 ms gecikmeli bir zamanlayıcı oluştur
11 Zamanlayıcı zamanlayıcısı = new Timer(1000, new TimerListener());
12 timer.start();
13 }
14
15 özel sınıf TimerListener implements ActionListener {
16 @Override /** Eylem olayını işleyin */
17 public void actionPerformed(ActionEvent e) {
18 // Yeni zamanı ayarlayın ve mevcut zamanı göstermek için saati yeniden boyayın
19 clock.setCurrentTime();
20 watch.repaint();
21 }
22 }
23
24 /** Ana yöntem */
25 public static void main(String[] args) {
26 JFrame frame = new SaatAnimasyonu();
27 frame.setTitle("SaatAnimasyonu");
28 frame.setSize(200, 200);
29 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
30 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31 frame.setVisible(true);
32 }

```

Program, Şekil 16.20'de gösterildiği gibi çalışan bir saat görüntüler. **ClockAnimation** bir **StillClock** oluşturur (satır 5). Satır 11, bir **ClockAnimation** için bir **Timer** oluşturur. Zamanlayıcı satır 12'de başlatılır. Zamanlayıcı her saniye bir **ActionEvent** tetikler ve dinleyici yeni bir zamanı ayarlamak (satır 19) ve saati yeniden boyamak (satır 20) için yanıt verir. **setCurrentTime()** metodu tanımlı **StillClock** saatteki mevcut zamanı ayarlar.

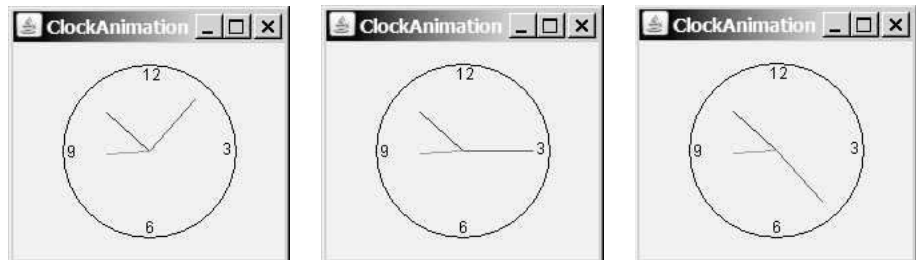


FIGURE 16.20 Panelde canlı bir saat görüntüleniyor.

16.23 Zamanlayıcıyı nasıl oluşturursunuz? Zamanlayıcıyı nasıl başlatırsınız? Zamanlayıcıyı nasıl durdurursunuz?

16.24 **Timer** sınıfının argüman almayan bir yapıcı metodu var mı? Bir

timer'a birden fazla dinleyici ekleyebilir misiniz?

Point Kontrol

MyProgrammingLab™

### Anahtar Terimler

|                             |                                |
|-----------------------------|--------------------------------|
| anonim iç sınıf 609         | olay kaynağı nesnesi 602       |
| event 602                   | event dinleyici nesnesi 603    |
| olay odaklı programlama 602 | iç sınıf 606                   |
| event işleyici 604          | dinleyici arayüzü adaptörü 620 |
| olay-dinleyici arayüzü 603  | kaynak nesnesi 602             |
| event nesnesi 602           |                                |

## BÖLÜM ÖZETİ

1. Olay sınıflarının kök sınıfı **java.util.EventObject**'tir. EventObject'in alt sınıfı arı, eylem olayları, pencere olayları, bileşen olayları, fare olayları ve tuş olayları gibi özel olay türleriyle ilgilenir. Bir olayın kaynak nesnesini, **EventObject** sınıfındaki **getSource()** örnek yöntemini kullanarak tanımlayabilirsiniz. Bir bileşen bir olay tetikleyebiliyorsa, bileşenin herhangi bir alt sınıfı aynı türde bir olayı tetikleyebilir.
2. Dinleyici nesnesinin sınıfı, karşılık gelen *olay-dinleyici arayüzü*'ni uygulamalıdır. Java, her olay sınıfı için bir dinleyici arayüzü sağlar. Dinleyici arayüzü genellikle **XListener** olarak adlandırılır ve **XEvent** için, **MouseEventListener** hariçtir. Örneğin, **ActionEvent** için karşılık gelen dinleyici arayüzü **ActionListener**'dir; **ActionEvent** için her dinleyici **ActionListener** arayüzünü uygulamalıdır. Dinleyici arayüzü, olayları işleyen *işleyici(ler)* olarak bilinen yöntem(ler)i içerir.
3. Dinleyici nesnesi, *kaynak nesne* tarafından kaydedilmelidir. Kayıt yöntemi eri olay türüne bağlıdır. **ActionEvent** için yöntem **addActionListener**'dir. Genel olarak, yöntem **addXListener** olarak adlandırılır **XEvent** için.
4. Bir *iç sınıf* veya *iç içe sınıf*, başka bir sınıfın kapsamı içinde tanımlanır. Bir iç sınıf, yerleştirildiği dış sınıfta tanımlanan verileri ve yöntemleri referans alabilir, bu nedenle dış sınıfın referansını iç sınıfın yapıcısına geçirmenize gerek yoktur.
5. *Dinleyici arayüzü adaptörleri*, dinleyici arayüzündeki tüm yöntemler için varsayılan uygul amalarsağlayan destek sınıflarıdır. Java, birden fazla işleyiciye sahip her AWT dinleyici ara yüzü için dinleyici arayüzü adaptörleri sağlar. Bir dinleyici arayüzü adaptörü **XAdapter** ol arak adlandırılır **XListener** için.
6. Bir kaynak nesnesi birkaç tür olay tetikleyebilir. Her olay için, kaynak nesne kayıtlı dinleyicilerin bir listesini tutar ve olayı işlemek için dinleyici nesnesindeki *handler*'ı çağırarak onları bilgilendirir.
7. Bir **MouseEvent**, bir fare bir bileşen üzerinde basıldığında, bırakıldığında, tıklanıldığında, girdiğinde, çıktığında, hareket ettiğinde veya sürüklendiğinde tetiklenir. Fare olayı nesnesi, olayı yakalar, örneğin, ona bağlı tıklama sayısı veya fare noktasının konumu (x- ve y- koordinatları) gibi.

8. Java, fare olaylarını işlemek için iki dinleyici arayüzü sağlar, **MouseListener** ve **MouseMotionListener**. Java, fare basıldığında, bırakıldığında, tıklandığında, girdiğinde ve ya çıktığında dinlemek için **MouseListener** arayüzünü uygular ve fare sürüklendiğinde de veyahareket ettiğinde dinlemek için **MouseMotionListener** arayüzünü uygular.
9. Bir **KeyEvent**, bir tuşa basıldığında, bırakıldığında veya yazıldığında tetiklenir. Tuş değeri ve tuş karakteri, tuş olayı nesnesinden elde edilebilir.
10. Bir dinleyicinin **keyPressed** handler'ı bir tuşa basıldığında çağrılır, **keyReleased** handler'ı bir tuş bırakıldığında çağrılır ve **keyTyped** handler'ı bir Unicode karakter tuşu girildiğinde çağrılır. Eğer bir tuşun Unicode'u yoksa (örneğin, fonksiyon tuşları, değiştirici tuşlar, eylem tuşları ve kontrol tuşları), bir dinleyicinin **keyTyped** handler'ı çağrılmayacaktır.
11. Java animasyonlarını kontrol etmek için **Timer** sınıfını kullanabilirsiniz. Bir zamanlayıcı, sabit bir hızda bir **ActionEvent** tetikler. Dinleyici, bir animasyonu simüle etmek için resmi günceller.

## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

MyProgrammingLab™

## PROGRAMLAMA EGZERSİZLERİ

### Bölümler 16.2–16.7

- \*16.1 (Dört kart seçin) Kullanıcının Yenile butonuna tıklayarak 52 kartlık bir desteden dört kart gösterdiği bir program yazın, Şekil 16.21a'da gösterildiği gibi. (İpucu: Dört kartı rastgele çizmek için Liste 6.2'ye bakın.)



(a)



(b)



(c)

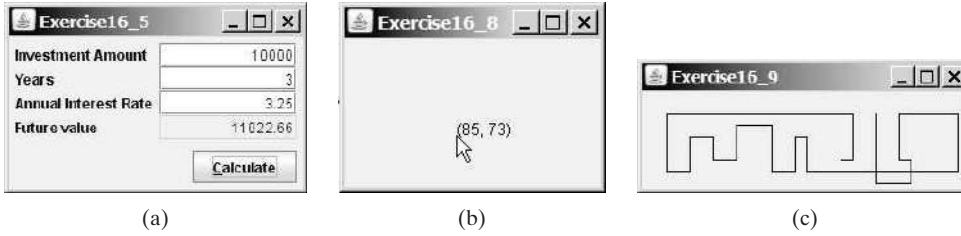
ŞEKİL 16.21 (a) Alıştırma 16.1 rastgele dört kart gösterir. (b) Alıştırma 16.3, topu hareket ettirmek için butonları kullanır. (c) Alıştırma 16.4, double sayılar üzerinde toplama, çıkarma, çarpma ve bölme işlemleri yapar.

- 16.2 (Konsolda hangi butona tıklandığını bulun) Konsolda hangi butona tıklandığını gösteren bir mesajgösterecek kodu Programlama Alıştırması 12.1'e ekleyin.
- \*16.3 (Topu hareket ettir) Bir panelde topu hareket ettiren bir program yazın. Topu sola, sağa, yukarı ve aşağı hareket ettirmek için bir panel sınıfı tanımlamalı ve metodları sağlamalısınız; bu, Şekil 16.21b'de gösterilmiştir. Topun tamamen görünüm dışına çıkmasını önlemek için sınırları kontrol edin.
- \*16.4 (Basit bir hesap makinesi oluşturun) Toplama, çıkarma, çarpma ve bölme işlemlerini gerçekleştiren bir program yazın; bu, Şekil 16.21c'de gösterilmiştir.

- \*16.5** (*Bir yatırım değeri hesaplayıcı oluşturun*) Belirli bir faiz oranında ve belirli bir yıl sayısı için bir yatırımın gelecekteki değerini hesaplayan bir program yazın. Hesaplama için formül:

$$\text{gelecekDeğer} = \text{yatırımMiktarı} * (1 + \text{aylıkFaizOranı})^{\text{yıl} * 12}$$

Yatırım miktarı, yıl sayısı ve yıllık faiz oranı için metin alanları kullanın. Kullanıcı Hesapla butonuna tıkladığında gelecekteki miktarı bir metin alanında gösterin; bu, Şekil 16.22a'da gösterilmiştir.



ŞEKİL 16.22(a) Kullanıcı yatırım miktarını, yılı ve faiz oranını girerek gelecek değeri hesaplar. (b) Alıştırma 16.8 fare pozisyonunu gösterir. (c) Alıştırma 16.9 ok tuşlarını kullanarak çizgileri çizer.

### Bölümler 16.8–16.9

- \*\*16.6** (*İki mesajı sırayla göster*) Bir panelde gösterilen iki mesajı [Java eğlencelidir](#) ve [Java güçlüdür](#) bir fare tıklamasıyla döndüren bir program yazın.
- \*16.7** (*Fare ile arka plan rengini ayarlama*) Fare düğmesine basıldığında bir panelin arka plan rengini siyah, fare düğmesi bırakıldığında ise beyaz olarak gösteren bir program yazın.
- \*16.8** (*Fare pozisyonunu gösterme*) Fare düğmesine tıkladığında fare pozisyonunu gösteren (bkz. Şekil 16.22b) ve fare düğmesine basıldığında fare pozisyonunu gösteren ve fare düğmesi bırakıldığında göstermeyi durduran iki program yazın.

### Bölüm 16.10

- \*16.9** (*Ok tuşları ile çizgi çizme*) Ok tuşlarını kullanarak çizgi segmentleri çizen bir program yazın. Çizgi, çerçevenin ortasından başlar ve sağ ok tuşu, yukarı ok tuşu, sol ok tuşu veya aşağı ok tuşu basıldığında doğruya, kuzeye, batıya veya güneye doğru çizer; bu, Şekil 16.22c'de gösterilmiştir.
- \*\*16.10** (*Bir dize girme ve gösterme*) Klavyeden bir dize alan ve bunu bir panelde gösteren bir program yazın. *Enter* tuşu bir dizinin sonunu belirtir. Yeni bir dize girildiğinde, panelde gösterilir.
- \*16.11** (*Bir karakter gösterme*) Klavyeden bir karakter girişi alıp, karakteri farenin gösterdiği yerde gösteren bir program yazın.

### Bölüm 16.11

- \*\*16.12** (*Çalışan bir fan gösterme*) Liste 13.4, `DrawArcs.java`, hareketsiz bir fanı gösterir. Bir fanın çalışmasını gösteren bir program yazın.
- \*\*16.13** (*Slayt gösterisi*) Yirmi beş slayt, kaynak koduyla birlikte kitapta indirilebilir olan `image` dizininde görüntü dosyaları olarak saklanmaktadır (`slide0.jpg`, `slide1.jpg`, ..., `slide24.jpg`). Her görüntünün boyutu  $800 \times 600$ . Bir Java uygulaması yazın



VideoNot

Yükselen bir bayrağı animasyonla göster

slaytları otomatik olarak tekrar tekrar gösterir. Her slayt bir saniye boyunca gösterilir. Slaytlar sırayla gösterilir. Son slayt bittiğinde, ilk slayt yeniden gösterilir ve bu şekilde devam eder. (*İpucu:* Çerçeveye bir etiket yerleştirin ve etikette bir slayt görüntü simgesi olarak ayarlayın.)

**\*\*16.14** (*Bayrak kaldır*) Bir bayrağı kaldıran bir Java programı yazın, Şekil 16.1'de gösterildiği gibi. (Görüntüleri nasıl göstereceğiniz için Bölüm 13.10'a bakın.)

**\*\*16.15** (*Yarış arabası*) Bir Java programı yazın, Şekil 16.23a'da gösterildiği gibi araba yarışını simüle eder. Araba soldan sağa hareket eder. Sağ uca çarptığında, soldan yeniden başlar ve aynı işlemi devam ettirir. Animasyonu kontrol etmek için bir zamanlayıcı kullanabilirsiniz. Arabanın yeni temel koordinatları  $(x, y)$  ile yeniden çizimini yapın, Şekil 16.23b'de gösterildiği gibi. Ayrıca, kullanıcıya bir düğmeye basma/bırakma ile animasyonu duraklatma/devam ettirme ve yukarı ve aşağı ok tuşlarına basarak araba hızını artırma/azaltma imkanı tanıyın.



FIGURE 16.23(a) Egzersiz 16.15 hareket eden bir aracı gösterir. (b) Yeni bir temel nokta ile bir aracı yeniden çizebilirsiniz.

**\*16.16** (*Yanıp sönen bir etiket göster*) Yanıp sönen bir etiket gösteren bir program yazın. (*İpucu:* Etiket yanıp sönmeye için, paneli etiketle ve onsuz (boş ekran) sabit bir hızda sırayla yeniden boyamanız gerekir. Sıralamayı kontrol etmek için bir **boolean** değeri kullanın.)

**\*16.17** (*Hareket eden bir etiketi kontrol et*) Liste 16.11, AnimationDemo.java dosyasını değiştirin, fare ile hareket eden bir etiketi kontrol etmek için. Fare basılıyken etiket donuyor ve buton bırakıldığında tekrar hareket ediyor.

## Kapsamlı

**\*16.18** (*Klavye ile bir daire hareket ettir*) Ok tuşlarını kullanarak yukarı, aşağı, sola veya sağa hareket eden bir daireyi hareket ettiren bir program yazın.

**\*\*16.19** (*Geometri: bir dairenin içinde mi?*) Merkezinde **(100,60)** olan ve yarıçapı **50** olan sabit bir daire çizen bir program yazın. Fare her hareket ettiğinde, fare noktasının dairenin içinde mi yoksa dışında mı olduğunu belirten bir mesaj gösterin, Şekil 16.24a'da gösterildiği gibi.



VideoNot

Fare noktasının konumunu kontrol et

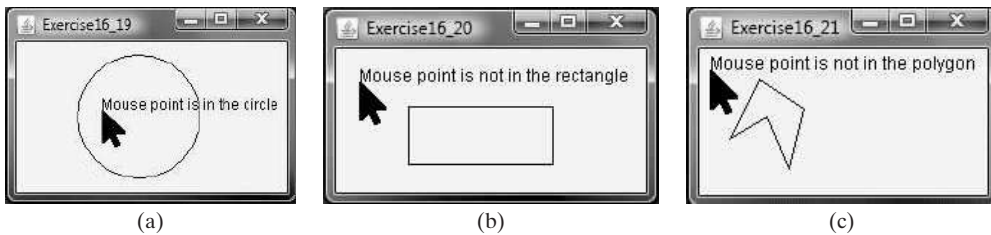


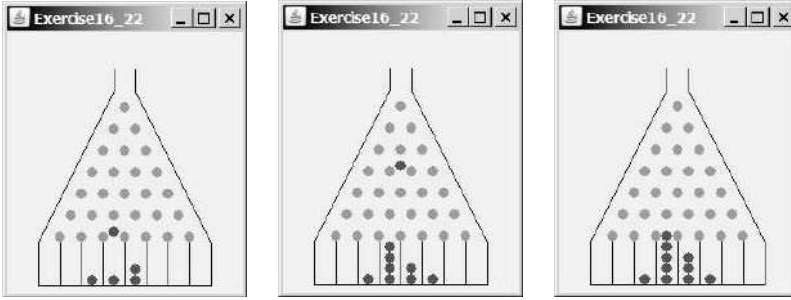
FIGURE 16.24 Bir noktanın bir dairenin, bir dikdörtgenin veya bir çokgenin içinde olup olmadığını tespit edin.



**\*\*16.20** (*Geometri: bir dikdörtgenin içinde mi?*) 100,60 koordinatlarında merkezlenmiş, genişliği 100 ve yüksekliği 40 olan sabit bir dikdörtgen çizen bir program yazın. Fare hareket ettikçe, fare noktasının dikdörtgenin içinde mi yoksa dışında mı olduğunu belirten bir mesaj gösterin; bu, Şekil 16.24b'de gösterilmiştir. Bir noktanın bir dikdörtgenin içinde olup olmadığını tespit etmek için, Programlama Egzersizi 10.13'te tanımlanan `MyRectangle2D` sınıfını kullanın.

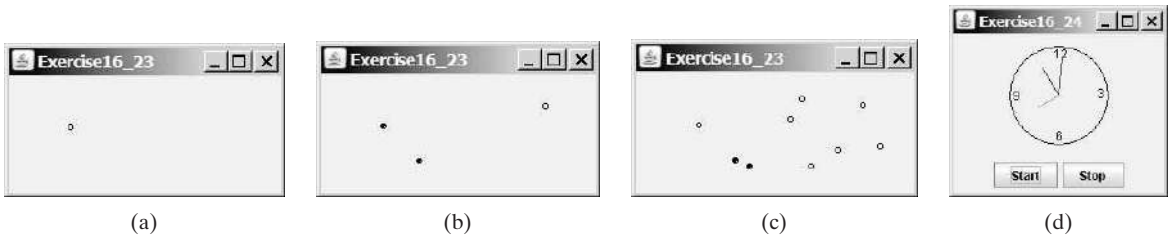
**\*\*16.21** (*Geometri: bir çokgenin içinde mi?*) 40,20, 70,40, 60,80, 45,45 ve 20,60 noktaları na sahip sabit bir çokgen çizen bir program yazın. Fare hareket ettikçe, fare noktasının çokgenin içinde mi yoksa dışında mı olduğunu belirten bir mesaj gösterin; bu, Şekil 16.24c'de gösterilmiştir. Bir noktanın bir çokgenin içinde olup olmadığını tespit etmek için, `Polygon` sınıfında tanımlanan `contains` metodunu kullanın (bkz. Şekil 13.13).

**\*\*\*16.22** (*Oyun: fasulye makinesi animasyonu*) Programlama Egzersizi 6.21'de tanıtılan fasulye makinesini animasyon haline getiren bir program yazın. Animasyon, on topun düşmesinden sonra sona erer; bu, Şekil 16.25'te gösterilmiştir.



**FIGURE 16.25** Toplar fasulye makinesine bırakılıyor.

**\*\*\*16.23** (*Geometri: en yakın nokta çiftleri*) Kullanıcının panelde tıklayarak dinamik olarak noktalar oluşturmaya izin veren bir program yazın. Başlangıçta panel boştur. Panelde iki veya daha fazla nokta olduğunda, en yakın nokta çiftini vurgulayın. Yeni bir nokta oluşturulduğunda, yeni bir en yakın nokta çifti vurgulanır. Noktaları küçük daireler ile gösterin ve noktaları dolu daireler ile vurgulayın; bu, Şekil 16.26a-c'de gösterilmiştir. (*İpucu:* noktaları bir `ArrayList` içinde saklayın.)

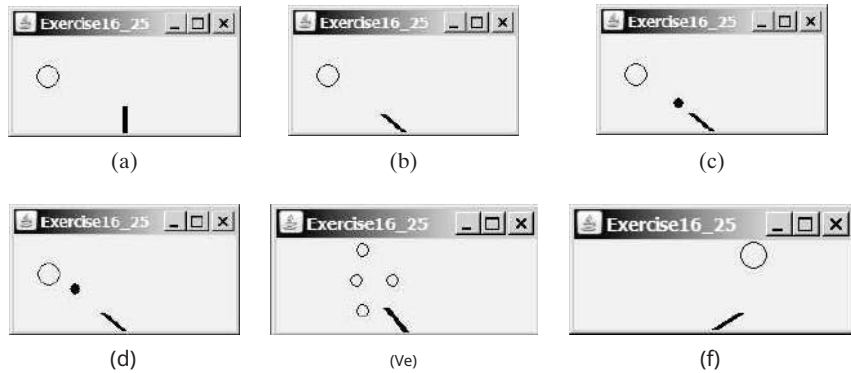


**FIGURE 16.26** Alıştırma 16.23, kullanıcının bir fare tıklamasıyla yeni noktalar oluşturmaya ve en yakın

noktaların çiftini vurgulamasına olanak tanır. Alıştırma 16.24, kullanıcının bir saati başlatmasına ve durdurmasına olanak tanır.

**\*16.24** (*Bir saati kontrol et*) Listing 16.12, `ClockAnimation.java` dosyasını değiştirin, saati başlatmak ve durdurmak için ikimetod `start()` ve `stop()` ekleyin. Kullanıcının saati `Start` ve `Stop` butonlarıyla kontrol etmesine olanak tanıyan bir program yazın, Şekil 16.26d'de gösterildiği gibi.

**\*\*\*16.25** (*Oyun: balonları vur*) Bir panelde rastgele bir konumda bir balon gösteren bir program yazın(Şekil 16.27a). Silahı balona nişan almak için sol ve sağ ok tuşlarını kullanarak güzel bir şekilde sola veya sağa yönlendirin (Şekil 16.27 b). Silahın içinden küçük bir top ateşlemek için yukarı ok tuşuna basın (Şekil 16.27c-d). Top balona çarptığında, enkaza (Şekil 16.27e) ve rastgele bir konumda yeni bir balon(Şekil 16.27f) gösterilir. Top balonu iskarlar ise, top panelin sınırına çarptığında kaybolur. Daha sonra başka bir top ateşlemek için yukarı ok tuşuna basabilirsiniz. Sol veya sağ ok tuşuna her bastığınızda, silah 5 derece sola veya sağa döner. (Eğitmenler oyunu şu şekilde değiştirebilir : 1. Yok edilen balonların sayısını göster; 2. bir geri sayım sayacı (örneğin, 60 saniye) göster ve süre dolduğunda oyunu sonlandır; ve/veya 3. balonun dinamik olarak yükselmesine izin ver.)



**FIGÜR 16.27**(a) Bir balon rastgele bir konumda gösterilir. (b) Balona nişan almak için sol/sağ ok tuşlarına basın. (c) Bir top ateşlemek için yukarı ok tuşuna basın. (d) Top, balona doğru düz bir şekilde hareket eder. (e) Top balona çarpar. (f) Yeni bir balon rastgele bir konumda gösterilir.

### **\*\*16.26** (*Fare ile bir daireyi hareket ettirin*) Yarıçapı 10

piksel olan bir daireyi gösteren bir program yazın. Fareyi dairenin içine koyabilir ve daireyi fare nereye giderse oraya sürükleyebilirsiniz (yani, fare basılıyken hareket ettirin), Şekil 16.28a–b'de gösterildiği gibi.



**FIGÜR 16.28**(a–b) Daireyi işaretleyebilir, sürükleyebilir ve hareket ettirebilirsiniz. (c) Bir daireye tıkladığınızda, rastgele bir konumda yeni bir daire gösterilir. (d) 20 daireye tıkladıktan sonra, harcanan süre panelde gösterilir.

**\*\*\*16.27** (*Oyun: göz-el koordinasyonu*) Yarıçapı olan bir daireyi gösteren bir program yazın 10 piksel, panelde rastgele bir konumda rastgele bir renkle doldurulmuş olarak gösterilir, Şekil 16.28c'de gösterildiği gibi. Daireye tıkladığınızda, kaybolur ve başka bir rastgele konumda yeni bir rastgele-renk daire gösterilir. Yirmi daireye tıkladıktan sonra, panelde harcanan süreyi gösterin, Şekil 16.28d'de gösterildiği gibi.

- \*\*\*16.28 (Simülasyon: kendini engelleyen rastgele yürüyüş) Bir ızgarada kendini engelleyen bir yürüyüş, aynı noktaya iki kez uğramadan bir noktadan diğerine giden bir yoldur. K kendini engelleyen yürüyüşlerin fizik, kimya ve matematikte uygulamaları vardır. Onlar, çözücüler ve polimerler gibi zincir benzeri varlıkları modellemek için kullanılabilir. Merkezden başlayıp bir sınır noktasında sona eren veya (yani, daha önce ziyaret edilmiş dört nokta ile çevrili) bir kör noktada sona eren rastgele bir yol gösteren bir program yazın. Lattice boyutunu  $16 \times 16$  olduğunu varsayın.



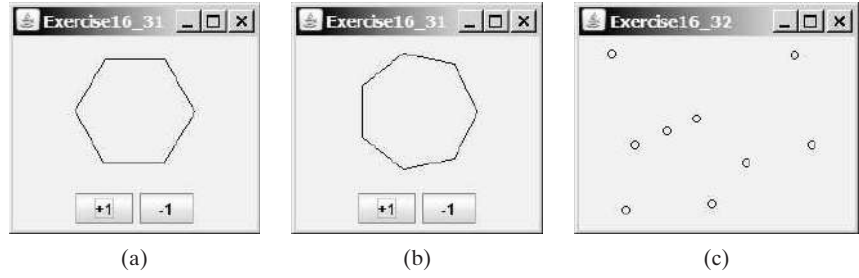
**FIGÜR 16.29** (a) Bir yol bir sınır noktasında sona erer. (b) Bir yol çıkmaz noktada sona erer. (c-d) Animasyon, bir yolun ilerlemesi ni adım adım gösterir.

- \*\*\*16.29 (Animasyon: kendini engelleyen rastgele yürüyüş) Önceki alıştırımayı, yürüyüşü adım adım bir animasyonda gösterecek şekilde revize edin.
- \*\*16.30 (Simülasyon: kendini engelleyen rastgele yürüyüş) Grid boyutu arttıkça kör yolların olasılığının arttığını göstermek için bir simülasyon programı yazın. Programınız, boyutu 10'dan 80'e kadar olan latticeleri simüle eder. Her lattice boyutu için, 10.000 kez kendini engelleyen rastgele bir yürüyüş simüle edin ve kör yolların olasılığını gösterin.

10 boyutlu bir ızgara için çıkmaz yolların olasılığı %10.6'dır  
 11 boyutlu bir ızgara için çıkmaz yolların olasılığı %14.0'dır  
 . . .  
 80 boyutlu bir ızgara için çıkmaz yolların olasılığı %99.5'tir



- \*16.31 (Geometri:  $n$  kenarlı düzgün çokgeni göster) Programlama Alıştırması 13.2 5, bir  $n$  kenarlı düzgün çokgeni göstermek için *RegularPolygonPanel* oluşturdu. Bir düzgün çokgeni gösteren ve boyutunu artırmak veya azaltmak için  $n + 1$  ve  $n - 1$  adlı iki düğme kullanan bir program yazın. Ayrıca, kullanıcıya sağ veya sol fare düğmesine tıklayarak ve yukarı ve aşağı ok tuşlarına basarak boyutu artırma veya azaltma imkanı tanıyın.
- \*\*16.32 (Geometri: nokta ekleme ve kaldırma) Kullanıcının bir panele tıklayarak dinamik olarak nokta oluşturmasına ve kaldırmasına izin veren bir program yazın (bkz. Şekil 16.30c). Kullanıcı farenin sağ tuşuna tıkladığında, bir nokta oluşturulur ve farenin noktasında görüntülenir. Kullanıcı, bir noktaya işaret ederek ve farenin sol tuşuna tıklayarak bir noktayı kaldırabilir.
- \*\*16.33 (Geometri: sarkaç) Şekil 16.31'de gösterildiği gibi, bir sarkacın sallanmasını animasyonlu hale getiren bir program yazın. Hızı artırmak için yukarı ok tuşuna ve azaltmak için aşağı ok tuşuna basın. Animasyonu durdurmak için *S* tuşuna ve devam ettirmek için *R* tuşuna basın.

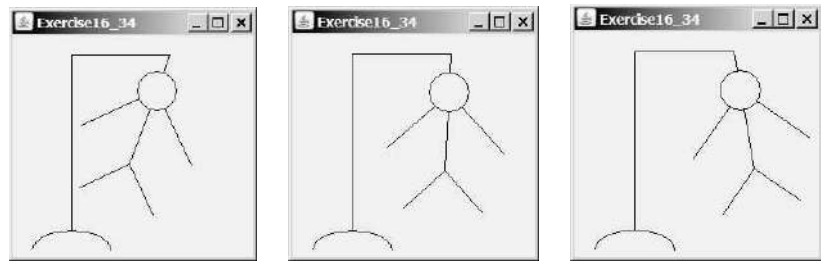


**FIGÜR16.30** Tıklamak  $+t_1$  veya  $-t_1$  düğmesi, Alıştırma 16.31'deki bir düzgün çokgenin kenar sayısını artırır veya azaltır. Alıştırma 16.32, kullanıcının noktaları dinamik olarak oluşturmasına/çıkarmasına olanak tanır.



**FIGÜR16.31** Alıştırma 16.33 bir sarkacın sallanmasını animasyon haline getirir.

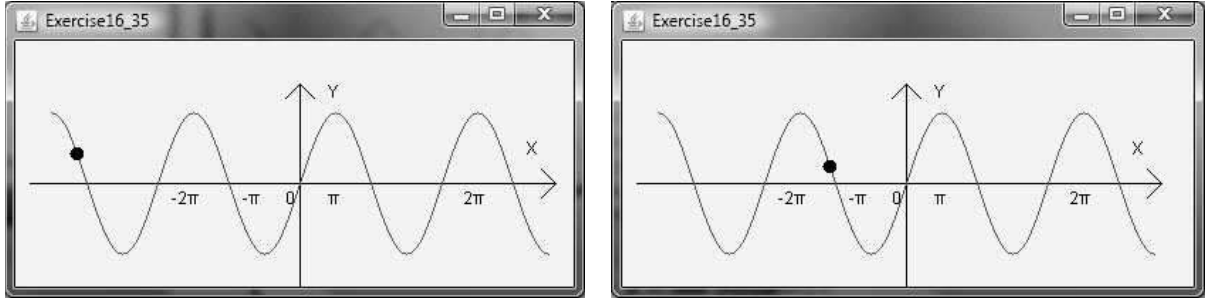
**\*\*16.34** (*Oyun: ipucu*) Şekil 16.32'de gösterildiği gibi, bir ipucu oyununun sallanmasını animasyonlu hale getiren bir program yazın. Hızı artırmak için yukarı ok tuşuna ve azaltmak için aşağı ok tuşuna basın. Animasyonu durdurmak için S tuşuna ve R tuşuna basarak devam ettirin.



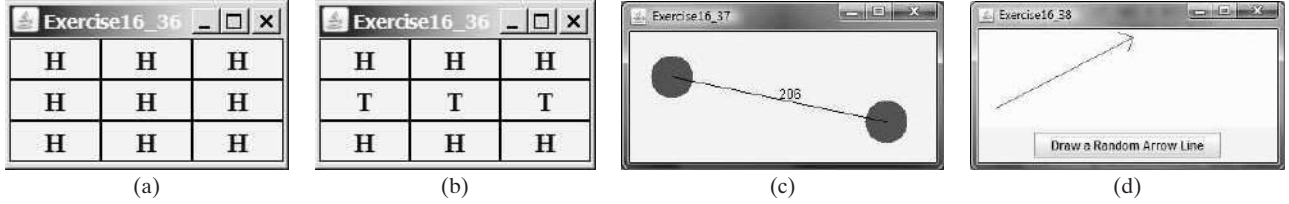
**FIGÜR16.32** Program bir ip cambazı oyununu animasyon haline getirir.

**\*\*\*16.35** (*Animasyon: eğri üzerindeki top*) Şekil 16.33'de gösterildiği gibi, bir topun bir sinüs eğrisi boyunca hareketini animasyonlu hale getiren bir program yazın. Top sağ kenara geldiğinde, soldan yeniden başlar. Kullanıcının animasyonu duraklatmasını/devam ettirmesini sağlamak için sol/sağ fare tuşuna tıklama ile etkinleştirin.

**\*16.36** (*Para at*) Her biri için baş (H) veya yazı (T) gösteren bir program yazın dokuz paranın, Şekil 16.34a–b'de gösterildiği gibi. Bir hücreye tıklandığında, para atılır. Bir hücre bir JLabel'dır. JLabel'ı genişleten özel bir hücre sınıfı yazın.



FIGÜR 16.33 Program, bir sinüs eğrisi boyunca hareket eden bir topu canlandırır.



FIGÜR 16.34 (a-b) Alıştırma 16.36, kullanıcının bir hücreye tıklayarak bir madeni para atmasını sağlar. (c) Kullanıcı daireleri sürükleyebilir. (d) Alıştırma 16.38 rastgele bir ok çizgi çizer.

tıklamaları işlemek için fare dinleyicisi ile. Program başladığında, tüm hücreler başlangıçta **H** gösterir.

**\*16.37** (İki hareketli köşe ve aralarındaki mesafe) 20 yarıçaplı iki çemberi (20, 20) ve (120, 50) konumunda gösteren bir program yazın ve iki çemberi bağlayan bir çizgi çizin, Şekil 16.34c'de gösterildiği gibi. Çemberler arasındaki mesafe çizgi boyunca gösterilir. Kullanıcı bir çemberi sürükleyebilir. Bu olduğunda, çember ve çizgisi hareket eder ve çemberler arasındaki mesafe güncellenir. Programınız, çemberlerin çok yakın olmasına izin vermemelidir. İki çemberin merkezleri arasında en az 70 piksel uzaklık bırakın.

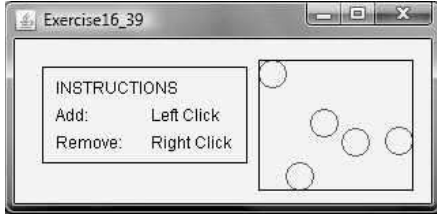
**\*\*16.38** (Ok işareti çizen bir çizgi çizin) Aşağıdaki yöntem başlığını kullanarak bir başlangıç noktasından bir bitiş noktasına ok işareti çizen statik bir yöntem yazın:

```
public static void drawArrowLine(int tx1, int ty1, int tx2, int ty2, Graphics g)
```

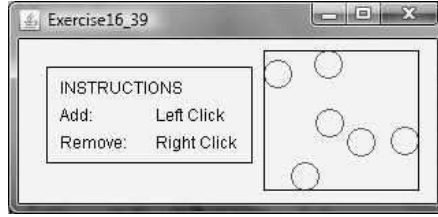
Bir test programı yazın, *Rastgele Ok Çiz* butonuna tıklandığında rastgele bir ok çizgi çizer, Şekil 16.34d'de gösterildiği gibi.

**\*\*16.39** (Geometri: sınırlayıcı dikdörtgeni bul) Kullanıcının iki boyutlu bir düzlemde dinamik olarak noktalar ekleyip kaldırmasına olanak tanıyan bir program yazın, Şekil 16.35a-b'de gösterildiği gibi. Noktalar eklendikçe ve kaldırıldıkça minimum sınırlayıcı dikdörtgen güncellenir. Her noktanın yarıçapının 10 piksel olduğunu varsayın.

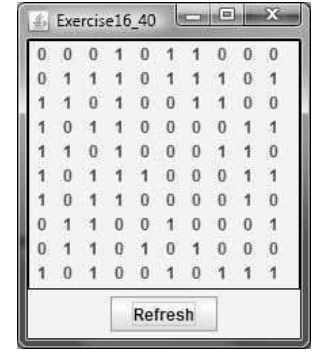
**\*16.40** (Rastgele 0 veya 1 göster) 10'a 10 boyutunda bir kare matris gösteren bir program yazın, Şekil 16.35c'de gösterildiği gibi. Matristeki her eleman 0 veya 1'dir, *Yenile* butonuna tıklandığında rastgele üretilir. Her sayıyı bir etiket içinde ortalanmış olarak gös-



(a)



(b)



(c)

**FIGÜR 16.35**(a–b) Alıştırma 16.39, kullanıcının noktaları dinamik olarak eklemesine/kaldırmasına olanak tanır ve sınırlayıcı dikdörtgeni gösterir. (c) Egzersiz 16.40, *Yenile* butonuna tıklanarak rastgele 0 ve 1'leri görüntüler.

# GUI BİLEŞENLERİ

## Hedefler

- Çeşitli kullanıcı arayüzü bileşenleri ile grafik kullanıcı arayüzleri oluşturmak (§§17.2–17.8).
- `JCheckBox`, `JRadioButton` ve `TextField` için dinleyiciler oluşturmak (§17.2).
- `JTextArea` kullanarak çok satırlı metinler girmek (§17.3).
- `JComboBox` kullanarak tek bir öğe seçmek (§17.4).
- `JList` kullanarak tek veya birden fazla öğe seçmek (§17.5).
- `JScrollBar` kullanarak bir değer aralığı seçmek (§17.6).
- `JSlider` kullanarak bir değer aralığı seçmek ve `JScrollBar` ile `JSlider` arasındaki farkları keşfetmek (§17.7).
- Bir uygulamada birden fazla pencere görüntülemek (§17.8).





## 17.1 Giriş



*Swing, kapsamlı bir kullanıcı arayüzü geliştirmek için birçok GUI bileşeni sağlar.*

Önceki bölümler kısaca **JButton**, **JCheckBox**, **JRadioButton**, **JLabel**, **TextField** ve **PasswordField**'i tanıttı. Bu bölüm, bu bileşenler için olayların nasıl işlendiğini ayrıntılı olarak tanıtmaktadır. Ayrıca **TextArea**, **ComboBox**, **List**, **Scrollbar** ve **Slider**'i de tanıttık. **Menu**, **ToolBar**, **TabbedPane**, **SplitPane**, **Spinner**, **Tree** ve **Table** gibi daha fazla GUI bileşeni bonusWeb Bölümleri 36–40'da tanıtılacaktır.

## 17.2 JCheckBox, JRadioButton VE

TextField için Olaylar



*Bir GUI bileşeni birçok türde olay tetikleyebilir. **ActionEvent** genellikle **JCheckBox**, **JRadioButton** ve **TextField** için işlenir ve **ItemEvent** **JCheckBox** ve **JRadioButton** için kullanılabilir.*

Önceki bölümde, **JButton** için bir eylem olayını nasıl yöneteceğinizi öğrendiniz. Bu bölüm, onay kutuları, radyo düğmeleri ve metin alanları için olayları yönetmeyi tanıtmaktadır.

Bir **JCheckBox** veya **JRadioButton** tıklandığında (yani, işaretlendiğinde veya işaret kaldırıldığında), bir **ItemEvent** ve ardından bir **ActionEvent** tetikler. Bir **TextField** üzerinde **Enter** tuşuna bastığınızda, bir **ActionEvent** tetikler.

Liste 17.1, onay kutularından, radyo düğmelerinden ve metin alanlarından olayları nasıl yöneteceğinizi gösteren bir programı vermektedir. Program, bir etiket görüntüler ve kullanıcının radyo düğmeleri ile etiketin metninin renklerini ayarlamasına, onay kutuları ile yazı tiplerini ayarlamasına ve bir metin alanından girilen yeni metni ayarlama adına tanır; bu, Şekil 17.1'de gösterilmiştir.



JPanelBorderLayout ile bir etiket ve bir metin alanı için

ŞEKİL 17.1 Program, onay kutuları, radyo düğmeleri ve metin alanlarını göstermektedir.

### LISTİNG 17.1tGUIEventDemo.java

etiket oluştur

onay kutuları oluştur

radyo düğmeleri oluştur

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4 import javax.swing.border.*;
5
6 genel class GUIEventDemo extends JFrame {
7 private JLabel jlblMessage = new JLabel("Merhaba", JLabel.CENTER);
8
9 // Mesajın fontunu ayarlamak için onay kutuları oluştur
10 private JCheckBox jchkBold = new JCheckBox("Kalin");
11 private JCheckBox jchkItalic = new JCheckBox("İtalik");
12
13 // Mesaj renklerini ayarlamak için üç radyo düğmesi oluştur
14 private JRadioButton jrbRed = new JRadioButton("Kırmızı");
15 private JRadioButton jrbGreen = new JRadioButton("Yeşil");
16 private JRadioButton jrbBlue = new JRadioButton("Mavi");
17
18 // Yeni bir mesaj ayarlamak için bir metin alanı oluştur

```



## 17.2 JCheckBox, JRadioButton ve JTextField için Olaylar 641

```
19 PrivatetJTextField jtfMessage = newtJTextField(t10t); metin alanı oluřtur
2021
 public static voidtmain(String[] args) {n22 GUIEvent
Demo çerçeve = newtGUIEventDemo();n23 çerçeve.pack(); çerçeve oluřtur
 çerçeveyi paketle
24 frame.setTitle("GUIEventDemo");
25 frame.setLocationRelativeTo(null);// Çerçeveyi ortal
26 frame.setDefau1tCloseOperation(JFrame.EXIT_ON_CLOSE);
27 frame.setVisible(true);
28 }
2930
 publictGUIEventDemo() { kullanıcı arayüzü oluřtur
31 jlblMessage.setBorder(newtLineBorder(Color.BLACK,t2t));n32 add(jlblMessa
ge, BorderLayout.CENTER); etiketi yerleřtir
333334
 // Onay kutularını tutmak için bir panel oluřtur
35 JPanel jpCheckBoxes = new JPanel(); onay kutuları için panel
36 jpCheckBoxes.setLayout(new GridLayout(2, 1));
37 jpCheckBoxes.add(jchkBo1d);
38 jpCheckBoxes.add(jchkItalic);n39 add(jpChe
ckBoxes, BorderLayout.EAST);
4041
 // Radyo düğmelerini tutacak bir panel oluřtur
42 JPanel jpRadioButtons = newJPanel(); radyo düğmeleri için panel
43 jpRadioButtons.setLayout(newGridLayout(3, 1));
44 jpRadioButtons.add(jrbRed);
45 jpRadioButtons.add(jrbGreen);n46 jpRadioB
uttons.add(jrbBlue);n47 add(jpRadioButtons,
BorderLayout.WEST);
4849
 // Üç düğmeyi gruplamak için bir radyo düğmesi grubu oluřtur
50 ButtonGroup group = newButtonGroup(); düğmeleri gruplandır
51 group.add(jrbRed);
52 group.add(jrbGreen);n53 group.
add(jrbBlue);
5455
 // Bařlangıç mesajı rengini mavi olarak ayarla
56 jrbBlue.setSelected(true);
57 jlblMessage.setForeground(Color.b1ue);
5859
 // Etiket ve metin alanını tutacak bir panel oluřtur
60 JPanel jpTextField = newJPanel(); metin alanı için panel
61 jpTextField.setLayout(newBorderLayout(5, 0));
62 jpTextField.add(
63 newJLabel("Yeni bir mesaj girin"), BorderLayout.WEST);
64 jpTextField.add(jtfMessage, BorderLayout.CENTER);n65 jtfMessage.se
tHorizontalAlignment(JTextField.RIGHT);n66 add(jpTextField, BorderLayout.N
ORTH);
6768
 // Onay kutuları ve radyo düğmeleri için mnemonik tuřları ayarla
69 jchkBo1d.setMnemonic('B'); mnemonikleri ayarla
70 jchkItalic.setMnemonic('I');
71 jrbRed.setMnemonic('E');
72 jrbGreen.setMnemonic('G');
73 jrbBlue.setMnemonic('U');
7475
 // Dinleyicileri onay kutuları ile kaydet
76 jchkBo1d.addActionListener(newActionListener() { dinleyici kaydet
77 @Override
78 public voidtactionPerformed(ActionEvent e) {
```

## 642 Bölüm 17 GUI Bileşenleri

```
79 setNewFont();n80 }

81 });
dinleyici kaydet 82 jchkItalic.addActionListener(tnewtActionListener() {n83 @Override

84 public voidtactionPerformed(ActionEvent e) {
85 setNewFont();n86 }

87 });
88
89 // Radyo düğmeleri için dinleyicileri kaydet
dinleyici kaydet 90 jrbRed.addActionListener(new ActionListener() {
91 @Override
92 public voidtactionPerformed(ActionEvent e) {
93 jlblMessage.setForeground(Color.red);
94 }
95 });
dinleyici kaydet 96 jrbGreen.addActionListener(tnewtActionListener() {n97 @Overrid
e
98 public voidtactionPerformed(ActionEvent e) {
99 jlblMessage.setForeground(Color.green);n100 }

101 });
dinleyici kaydet 102 jrbBlue.addActionListener(tnewtActionListener() {n103 @Overrid
e
104 public voidtactionPerformed(ActionEvent e) {
105 jlblMessage.setForeground(Color.blue);n106 }

107 });
108
109 // Metin alanı için dinleyici kaydet
dinleyici kaydet 110 jtfMessage.addActionListener(new ActionListener() {
111 @Override
112 public voidtactionPerformed(ActionEvent e) {
113 jlblMessage.setText(jtfMessage.getText());n114 jtfMessage.reques
tFocusInWindow();
115 }n116 });n1
17 }

118
119 özel voidtsetNewFont() {
120 // Bir yazı tipi stilini belirle
121 inttfontStyle = Font.PLAIN;
122 fontStyle += (jchkBold.isSelected() Font.BOLD : Font.PLAIN);
123 fontStyle += (jchkItalic.isSelected() Font.ITALIC : Font.PLAIN);
124
125 // Mesaj için font ayarla
126 Font font = jlblMessage.getFont();
127 jlblMessage.setFont(
128 newtFont(font.getName(), fontStyle, font.getSize()));
129 }n130
}
```

Program, bir etiket, onay kutuları, radyo düğmeleri ve bir metin alanı oluşturur (satırlar 7–19). Bir etiketi çerçevenin ortasına yerleştirir (satırlar 31–32), onay kutularını doğruya (satırlar 35–39), radyo düğmelerini batıya (satırlar 42–47) ve bir metin alanını kuzeye (satırlar 60–66) yerleştirir.

mnemonik tuşlar

Program ayrıca onay kutuları ve radyo düğmeleri için mnemonikler ayarlar (satırlar 69–73). Bir onay kutusunu veya radyo düğmesini seçmek için bir fare tıklaması veya bir kısayol tuşu kullanabilirsiniz.

## 17.2 JCheckBox, JRadioButton ve JTextField için Olaylar 643

Program, onay kutuları, radyo düğmeleri ve metin alanı için eylem dinleyicileri kaydeder (76–116. satırlar).

dinleyicileri kaydet

Bir onay kutusu işaretlendiğinde veya işaret kaldırıldığında, dinleyicinin actionPerformed metodu olayı işlemek için çağrılır (79, 85. satırlar). JLabel'de kullanılan mevcut yazı tipi adı ve boyutu lblMessage.getFont() kullanılarak getName() ve getSize() metodları ile elde edilir (128. satır). Yazı tipi stilleri (Font.BOLD ve Font.ITALIC) onay kutularında belirtilmiştir. Hiçbir yazı tipi stili seçilmezse, varsayılan yazı tipi stili Font.PLAIN'dır (121. satır). Yazı tipi stili, Font.PLAIN için bir tamsayı 0, Font.BOLD için 1 ve Font.ITALIC için 2'dir. Yazı tipi stili, yazı tiplerini temsil eden tamsayıları toplamak suretiyle birleştirilebilir (122–123. satırlar). Örneğin, **Font.BOLD + Font.ITALIC, kalın ve italik birleştirilmiş yazı tipini temsil eden 3'tür.**

onay kutuları

setFont metodu (127. satır) Component sınıfında tanımlanmış olup JLabel sınıfında miras alınır. Bu metod otomatik olarak repaint metodunu çağırır. lblMessage üzerinde setFont çağrıldığına, lblMessage otomatik olarak yeniden boyanır.

Bir onay kutusu tıklandığında bir **ItemEvent** ve ardından bir **ActionEvent** tetikler. Mesajı yeniden görüntülemek için **ItemEvent** veya **ActionEvent**'i işleyebilirsiniz. Bu örnekteki program **ActionEvent**'i işler. **ItemEvent**'i işlemek istiyorsanız, bir **ItemEvent** dinleyicisi oluşturun ve bunu bir onay kutusuna kaydedin. Dinleyici, bir **ItemEvent**'i işlemek için **itemStateChanged** işleyicisini uygulamalıdır. Örneğin, aşağıdaki kod bir **ItemListener**'i **jchkBold** ile kaydeder:

```
// ItemEvent'i dinlemek için
d.addItemListener(new ItemListener() {
 @Override/** ItemEvent'i işleme */
 public void itemStateChanged(ItemEvent e){jchkBold
 setNewFont();n}
});
```

Bir radyo düğmesine tıklandığında, eylem olay dinleyicisi **Jb1Message**'daki karşılık gelen ön plan rengini ayarlar (satırlar 93, 99, 105).

radyo düğmeleri

Program bir **ButtonGroup** oluşturur ve gruba üç **JRadioButton** örneği (**jrbRed**, **jrbGreen** ve **jrbBlue**) ekler (satırlar 50–53) böylece yalnızca birbirini dışlayarak seçilebilirler—metin ya kırmızı ya da yeşil ya da mavi olacaktır.

radyo düğmesi grubu

Bir radyo düğmesi seçildiğinde veya seçilmediğinde bir **ItemEvent** ve ardından bir **ActionEvent** tetikler. Bir rengi seçmek için **ItemEvent** veya **ActionEvent**'i işleyebilirsiniz. Bu program **ActionEvent**'i işler. Bir alıştırmaya olarak, kodu

**ItemEvent** kullanarak yeniden yazın.

Metin alanına yeni bir mesaj yazdıktan sonra *Enter* tuşuna bastığınızda, yeni bir mesaj görüntülenir. Metin alanında *Enter* tuşuna basmak bir eylem olayı tetikler. Dinleyici, **Jb1Message** içinde yeni bir mesaj ayarlar (satır 113).

JTextField için  
ActionEvent

**requestFocusInWindow()** metodu (satır 114) **Component** sınıfında tanımlanmıştır ve bileşenin giriş odaklanmasını talep eder. Böylece, **jtfMessage.requestFocusInWindow()** **jtfMessage** üzerinde giriş odaklanmasını talep eder.

requestFocusInWindow()

**actionPerformed** metodu çağrıldığında **jtfMessage** üzerinde imleci göreceksiniz.

**pack()** metodu (satır 23) çerçeveyi içindeki bileşenlerin boyutuna göre otomatik olarak boyutlandırır.

paketle()

### 17.1 Bir **JButton**, **JLabel**, **JCheckBox**, **JRadioButton** ve **JTextField** bir

**ActionEvent** tetikleyebilir mi?

### 17.2 Bir **JButton**, **JLabel**, **JCheckBox**, **JRadioButton** ve **JTextField** bir

**ItemEvent** tetikleyebilir mi?

### 17.3 **jtfMessage.requestFocusInWindow()** çağrıldıktan sonra ne olur?

Point Kontrol

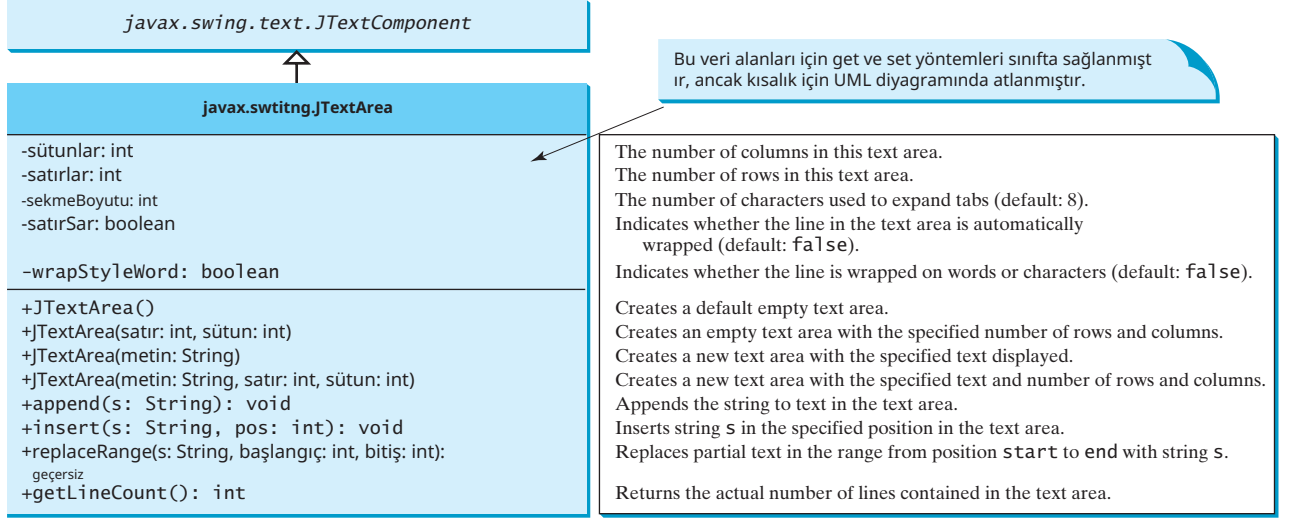
MyProgrammingLab™

## 17.3 Metin Alanları



**JTextArea kullanıcının birden fazla satır metin girmesine olanak tanır.**

Eğer kullanıcının birden fazla satır metin girmesine izin vermek istiyorsanız, birden fazla örnek oluşturabilirsiniz  **JTextField**. Daha iyi bir alternatif, kullanıcının birden fazla satır metin girmesine olanak tanıyan  **JTextArea** kullanmaktır. Şekil 17.2,  **JTextArea** içindeki yapıları ve yöntemleri listelemektedir.



ŞEKİL 17.2 JTextArea, birden fazla karakter satırı girmenizi veya görüntülemenizi sağlar.

**JTextField** gibi,  **JTextArea**,  **getText**,  **setText**,  **isEditable** ve  **setEditable** yöntemleri ni içeren  **JTextComponent**'den miras alır. Bir satırın  **lineWrap** özelliğinde sarılıp sarılı madığını belirtebilirsiniz. Eğer  **lineWrap**  **true** ise, satırın  **wrapStyleWord** özelliğinde n asıl sarılacağını belirtebilirsiniz. Eğer  **wrapStyleWord**  **true** ise, satır kelimeler üzerinde sarılır. Eğer  **false** ise, satır karakterler üzerinde sarılır. Aşağıdaki örnek, 5 satır ve 20 sütun, kelimeler üzerinde sarılmış, kırmızı ön plan rengi ve  **Courier** yazı tipi,  **kalın, 20** piksel ile bir metin alanı oluşturur.

satırı sar  
kelimeyi sar

```
JTextArea jtaNote = newJTextArea("Bu bir metin alanıdır",t5t,t20t);jtaNote.set
LineWrap(ttuet);
jtaNote.setWrapStyleWord(ttuet);jta
Note.setForeground(Color.red);jtaNote.s
etFont(tnewFont(t"Courier",t, Font.BOLD, t20t));
```

**JTextArea** kaydırmayı yönetmez, ancak bir  **JScrollPane** nesnesi oluşturarak bir  **JTextArea** örneğini t utabilir ve  **JScrollPane**'in  **JTextArea** için kaydırmayı yönetmesine izin verebilirsiniz, aşağıdaki gibi:

```
// Metin alanını tutmak için bir kaydırma paneli oluştur
JScrollPane scrollPane = newJScrollPane(jtaNote);
add(scrollPane, BorderLayout.CENTER);
```

JScrollPane



## TİP

Herhangi bir swing GUI bileşenini bir  **JScrollPane** içine yerleştirebilirsiniz.  **JScrollPane**, bi leşen görüntüleme alanına sığmayacak kadar büyükse otomatik olarak dikey ve yatay kaydırma sağlar.

Liste 17.3, bir resmi ve bir metni bir etiket içinde, ve bir metni bir metin alanında görüntüleyen bir programı göstermektedir; Şekil 17.3'te gösterildiği gibi.

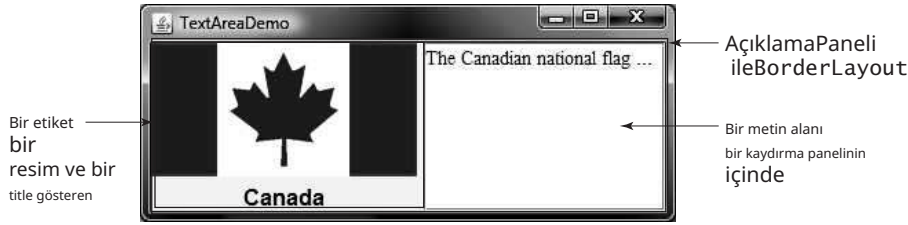


FIGURE 17.3 Program, bir etiket içinde bir resim, bir etiket içinde bir başlık ve metin alanında metin görüntüler.

Programdaki ana adımlar şunlardır:

1. Listing 17.2'de gösterildiği gibi, `JPanel` sınıfını genişleten `DescriptionPanel` adında bir sınıf tanımlayın. Bu sınıf, bir kaydırma paneli içinde bir metin alanı ve bir resim simgesi ile bir başlık görüntülemek için bir etiket içerir. Sınıf `DescriptionPanel` daha sonraki örneklerde yeniden kullanılacaktır.
2. Listing 17.3'de gösterildiği gibi, `JFrame` sınıfını genişleten `TextAreaDemo` adında bir sınıf tanımlayın. `DescriptionPanel` örneğini oluşturun ve çerçevenin ortasına ekleyin. `DescriptionPanel` ile `TextAreaDemo` arasındaki ilişki Şekil 17.4'te gösterilmektedir.

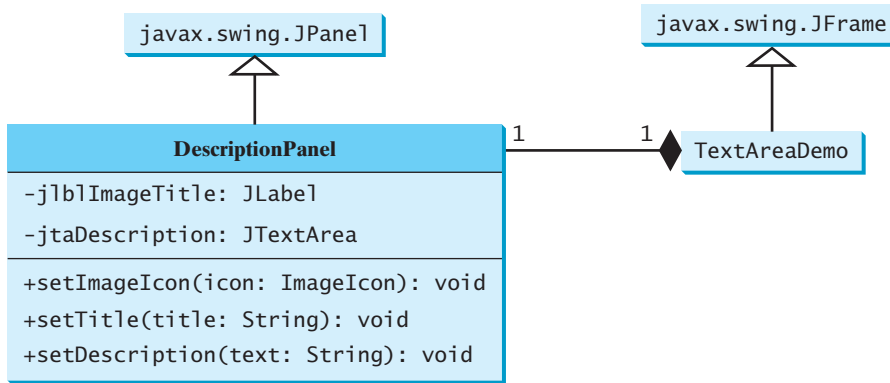


FIGURE 17.4 `TextAreaDemo`, bir resim, başlık ve bir ulusal bayrağın

deskripsiyonunu görüntülemek için `AçıklamaPaneli` kullanır.

## Listing 17.2 `DescriptionPanel.java`

```

1 import javax.swing.*;
2 import java.awt.*;
3
4 genel sınıf DescriptionPanel extends JPanel {
5 /** Bir resim simgesi ve bir başlık görüntülemek için etiket */
6 private JLabel lblImageTitle = new JLabel();
7
8 /** Metin görüntülemek için metin alanı */
9 özel JTextArea jtaDescription = new JTextArea();
10
11 kamusal DescriptionPanel() {
12 // İkonu ve metni ortalayın ve metni ikonun altına yerleştirin
13 lblImageTitle.setHorizontalAlignment(JLabel.CENTER);
14 lblImageTitle.setVerticalTextPosition(JLabel.BOTTOM);
15 jtaDescription.setVerticalTextPosition(JLabel.TOP);
16 }

```

etiket

metin alanı

etiket özellikleri

## 646 Bölüm 17 GUI Bileşenleri

```
17 // Etiket ve metin alanındaki yazı tipini ayarla
18 jlblImageTitle.setFont(tnewtFont(t"SansSerif"t, Font.BOLD, t16t));n19 jtaDescription.s
etFont(tnewtFont(t"Serif"t, Font.PLAIN, t14t));
20
21 // Metin alanı için lineWrap ve wrapStyleWord'u true olarak ayarla
22 setLineWrap(ttrue);
23 iption.setWrapStyleWord(ttrue);n24 jtaDescription.
25 jtaDescription.setEditable(tfalse);n26 jtaDescr
27
28 // Metin alanını tutmak için bir kaydırma paneli oluştur
29 JScrollPane scrollPane = newJScrollPane(jtaDescription);
30
31 // Panel için BorderLayout'u ayarla, etiket ve kaydırma panelini ekle
32 setLayout(newBorderLayout(5, 5));
33 add(scrollPane, BorderLayout.CENTER);
34 add(jlblImageTitle, BorderLayout.WEST);
35 }
36
37 /** Başlığı ayarla */
38 public voidsetTitle(Dize başlığı) {
39 jlblImageTitle.setText(title);n40 }
41
42 /** Resim simgesini ayarla */
43 public voidsetImageIcon(ImageIcon icon) {
44 jlblImageTitle.setIcon(icon);n45 }
46
47 /** Metin açıklamasını ayarla */
48 public voidsetDescription(Dize metni) {
49 jtaDescription.setText(text);n50 }
51 }
```

satırı sarmak  
kelimeyi sarmak  
sadece okunur

kaydırma paneli

Metin alanı, metin alanı için kaydırma işlevleri sağlayan bir **JScrollPane** (satır 27) içi ndedir. Fiziksel boyutundan daha fazla metin varsa kaydırma çubukları otomatik olarak görünür.

lineWrap özelliği, metin bir satıra sığmadığında satırın otomatik olarak sarılması için true (satır 22 ) olarak ayarlanmıştır. wrapStyleWord özelliği, satırın karakterler yerine kelimeler üzerinde sarılması iç in true (satır 23) olarak ayarlanmıştır. Metin alanı düzenlenemez olarak ayarlanmıştır (satır 24), bu ne denle metin alanındaki açıklamayı düzenleyemezsiniz.

Bu örnekte **DescriptionPanel** için ayrı bir sınıf oluşturmak gerekli değildir. Ancak, bu sınıf, çeşitli resimler için bir açıklama paneli görüntülemek üzere kullanacağınız bir sonraki bölü mde yeniden kullanım için oluşturulmuştur.

### Listing 17.3tTextAreaDemo.java

```
1 importjava.aut.*;
2 importjavax.swing.*;
3
34 genel classtTextAreaDemotextendstJFrame {
5 // Bir açıklama paneli tanımlayın ve oluşturun
6 özel Açıklama Panel açıklaması Panel = t newt Açıklama Panel();
7
8 public static voidtmain(String[] args) {n9 TextAreaD
emo çerçeve = tnewtTextAreaDemo();
10 çerçeve.pack();
11 frame.setLocationRelativeTo(nu11); // Çerçeveyi ortalayın }
```

oluştur açıklamaPaneli

oluştur çerçeve

```
12 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);n13 çerçeve.set
Title(t"TextAreaDemo"t);
14 çerçeve.setVisible(tttrue);n15 }
```

```
16
17 publicTextAreaDemo() {
18 // Açıklama panelinde başlık, metin ve resim ayarla
19 descriptionPanel.setTitle("Kanada");
20 String description = "Kanada ulusal bayrağı...";
21 descriptionPanel.setImageIcon(new ImageIcon("image/ca.gif"));
22 descriptionPanel.setDescription(description);
23
24 // Açıklama panelini çerçeveye ekle
25 setLayout(new BorderLayout());
26 add(descriptionPanel, BorderLayout.CENTER);
27 }
28 }
```

UI oluştur

ekle açıklamaPaneli

Liste 17.3'teki program, **DescriptionPanel** (satır 6) örneğini oluşturur ve title (satır 19), image (satır 21) ve açıklama panelindeki metni (satır 22) ayarlar. **DescriptionPanel JPanel** sınıfının bir alt sınıfıdır. **DescriptionPanel**, bir resim simgesini görüntülemek için bir etiket ve bir metin başlığı ile birlikte, resmin açıklamasını görüntülemek için bir metin alanı içerir.

**17.410** satır ve **20** sütun içeren bir metin alanı nasıl oluşturursunuz?

**17.5** Metin alanına üç satır nasıl ekler veya ekleyebilirsiniz?

**17.6** Kaydırılabilir bir metin alanı nasıl oluşturursunuz?

**17.7** Bir metin alanından metni almak için hangi yöntemi kullanıyorsunuz? Metin alanındaki satır sayısını nasıl alırsınız?

**17.8** Bir satır sarmasını nasıl belirtirsiniz? Karakterlerde sarmayı nasıl belirtirsiniz? Kelimelerde sarmayı nasıl belirtirsiniz?

Point Kontrol  
MyProgrammingLab™

## 17.4 Kombinasyon Kutuları

A kombinasyon kutusu, ayrıca seçim listesi veya açılır liste olarak bilinir, kullanıcının seçebileceği bir dizi öğe içerir.

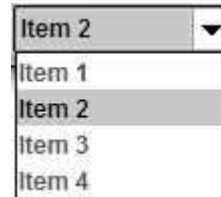


Bir açılır kutu, bir kullanıcının seçim aralığını sınırlamak için faydalıdır ve veri girişi için zahmeti i doğrulamayı önler. Şekil 17.5, sıklıkla kullanılan birkaç yapıcı ve yöntemi listelemektedir.

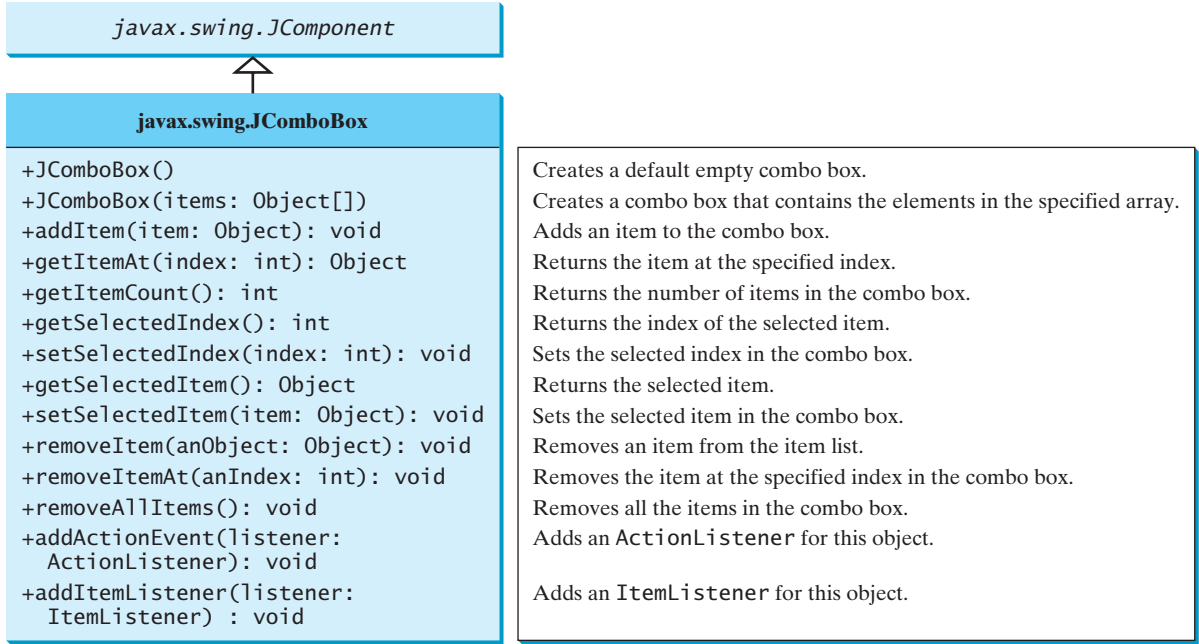
### JComboBox

Aşağıdaki ifadeler, dört öğeden oluşan, kırmızı ön plan, beyaz arka plan ve ikinci öğenin seçili olduğu bir açılır kutu oluşturur.

```
JComboBox jcb = new JComboBox(new Object[]
{ "Öğe 1", "Öğe 2", "Öğe 3", "Öğe 4" });
jcb.setForeground(Color.red);
jcb.setBackground(Color.white);
jcb.setSelectedItem("Öğe 2");
```

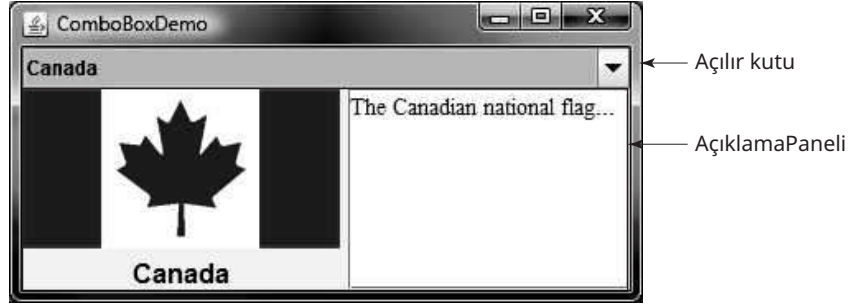


**JComboBox**, birçok diğer olay arasında **ItemEvent** ve **ActionEvent** tetikleyebilir. Bir öğe seçildiğinde, bir **ActionEvent** tetiklenir. Yeni bir öğe seçildiğinde, **JComboBox** iki kez **ItemEvent** etiketler: birincisi, daha önce seçilen öğeyi iptal etmek için, diğeri ise şu anda seçilen öğeyi seçmek içindir. Mevcut öğe yeniden seçilirse, hiçbir **ItemEvent** tetiklenmediğini unutmayın. Bir **ItemEvent**'ye yanıt vermek için, bir seçimi işlemek için **itemStateChanged(ItemEvent e)** i şleyicisini uygulamanız gerekir. Bir **JComboBox** menüsünden veri almak için, şu anda seçilen öğeyi döndürmek için **getSelectedItem()** veya **e.getItem()** yöntemini kullanarak **itemStateChanged(ItemEvent e)** işleyicisinden öğeyi alabilirsiniz.



ŞEKİL 17.5 JComboBox, bir dizi öğeden birini seçmenizi sağlar.

Liste 17.4, kullanıcıların bir ülkenin bayrağını görüntülemesine ve açıklamasını görmesine olanak tanıyan bir programı vermektedir. Kullanıcı, ülkeyi bir açılır kutudan seçerek bunu yapar; bu, Şekil 17.6'da gösterilmiştir.



ŞEKİL 17.6 Bir ülke seçildiğinde, ülkeye ait bir resim ve bayrağının açıklaması dahil olmak üzere, ülke hakkında bilgi açılır kutuda görüntülenir.

Programdaki ana adımlar şunlardır:

1. Kullanıcı arayüzünü oluşturun.

Seçim değerleri olarak ülke adları ile bir açılır kutu oluşturun. Bir AçıklamaPaneli nesnesi oluşturun (AçıklamaPaneli sınıfı, önceki bölümde tanıtılmıştır). Açılır kutuyu çerçevenin kuzeyine ve açıklama panelini çerçevenin ortasına yerleştirin.

2. Olayı işleyin.

Seçilen ülke adının bayrak başlığını, resmini ve açıklama panelindeki metni ayarlamak için `itemStateChanged` işleyicisini uygulamak üzere bir dinleyici oluşturun.



## 17.4tComboBoxDemo.java

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 kamu sınıfı ComboBoxDemo extends JFrame {
6 // Bayrak başlıkları için bir String dizisi oluşturun
7 özel String[] bayrakBaşlıkları = {"Kanada", "Çin", "Danimarka",
8 "Fransa", "Almanya", "Hindistan", "Norveç", "Birleşik Krallık",
9 "Amerika Birleşik Devletleri"};
10
11 // 9 ülkenin ulusal bayrakları için bir ImageIcon dizisi tanımlayın
12 özel ImageIcon[] bayrakGörüntüsü = {
13 yeniImageIcon("image/ca.gif"),
14 yeniImageIcon("image/china.gif"),
15 yeniImageIcon("image/denmark.gif"),
16 yeniImageIcon("image/fr.gif"),
17 yeniImageIcon("image/germany.gif"),
18 yeniImageIcon("image/india.gif"),
19 yeniImageIcon("image/norway.gif"),
20 yeniImageIcon("image/uk.gif"),
21 yeniImageIcon("image/us.gif")
22 };
23
24 // Bayrak açıklamaları için bir dizi dize tanımlayın
25 özel String[] bayrakAçıklaması = new String[9];
26
27 // Bir açıklama paneli tanımlayın ve oluşturun
28 özel Açıklama Panel açıklaması Panel = new Açıklama Panel();
29
30 // Ülkeleri seçmek için bir kombinasyon kutusu oluşturun
31 Privat JComboBox jcbo = new JComboBox(flagTitles);
32
33 public static void main(String[] args) {
34 ComboBoxDemo çerçeve = new ComboBoxDemo();
35 çerçeve.pack(
36 frame.setTitle("ComboBoxDemo");
37 frame.setLocationRelativeTo(null); // Çerçeveyi ortalara
38 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
39 frame.setVisible(true);
40 }
41
42 public ComboBoxDemo() {
43 // Metin açıklamasını ayarlayın
44 flagDescription[0] = "Kanada ulusal bayrağı...";
45 flagDescription[1] = "Çin için açıklama ... ";
46 flagDescription[2] = "Danimarka için açıklama ... ";
47 flagDescription[3] = "Fransa için açıklama ... ";
48 flagDescription[4] = "Almanya için açıklama ... ";
49 flagDescription[5] = "Hindistan için açıklama ... ";
50 flagDescription[6] = "Norveç için açıklama ... ";
51 flagDescription[7] = "İngiltere için açıklama ... ";
52 flagDescription[8] = "ABD için açıklama ... ";
53
54 // İlk ülkeyi (Kanada) görüntülemek için ayarla
55 setDisplay(0);
56
57 // Çerçeveye açılır kutu ve açıklama paneli ekle
58 add(jcbo, BorderLayout.NORTH);

```

ülke

görüntü simgesi

açıklama

açılır kutu

UI oluşturun

dinleyici

```

59 add(descriptionPanel, BorderLayout.CENTER);
60
61 // Dinleyiciyi kaydet
62 jcbo.addItemListener(new ItemListener() {
63 @Override/** Eleman seçimini işleme */
64 public void itemStateChanged(ItemEvent e) {
65 setDisplay(jcbo.getSelectedIndex());n66 }
67
68 }
69
70 /** Açıklama panelinde görüntü bilgilerini ayarla */
71 genel void setDisplay(tintindex) {
72 açıklamaPanel.setTitle(flagTitles[index]);n73 açıklamaPanel.setI
74 mageIcon(flagImage[index]);n74 açıklamaPanel.setDescription(flagDes
75 cription[index]);n75 }
76 }

```

Dinleyici, kombinasyon kutusundan **ItemEvent** dinler ve **ItemListener** (62–67. satırlar) uygular. **ItemEvent** yerine, kombinasyon kutusu eleman seçimini işlemek için **ActionEvent** kullanacak şekilde programı yeniden yazabilirsiniz.

Program, bayrak bilgilerini üç dizide saklar: **flagTitles**, **flagImage** ve **flagDescription** (satırlar 7–25). Dizi **flagTitles** dokuz ülkenin isimlerini içerir, dizi **flagImage** dokuz ülkenin bayraklarının resimlerini içerir ve dizi **flagDescription** bayrakların tanımlarını içerir.

Program, 28. satırda tanıtilan bir **DescriptionPanel** örneği oluşturur. Program, 31. satırda **flagTitles**'dan başlangıç değerleri ile bir açılır kutu oluşturur. Kullanıcı açılır kutudan bir öğe seçtiğinde, **itemStateChanged** işleyicisi çalıştırılır. İşleyici, seçilen indeksi bulur ve paneldeki karşılık gelen bayrak başlığını, bayrak resmini ve bayrak açıklamasını ayarlar.

**17.9** Bir açılır kutu nasıl oluşturulur ve üç öğe nasıl eklenir?

**17.10** Bir açılır kutudan bir öğe nasıl alınır? Bir açılır kutudan seçilen bir öğe nasıl alınır?

**17.11** Bir açılır kutudaki öğe sayısı nasıl alınır? Bir açılır kutudaki belirli bir indeksteki öğe nasıl alınır?

**17.12** Yeni bir öğe seçildiğinde **JComboBox** hangi olayları tetikler?

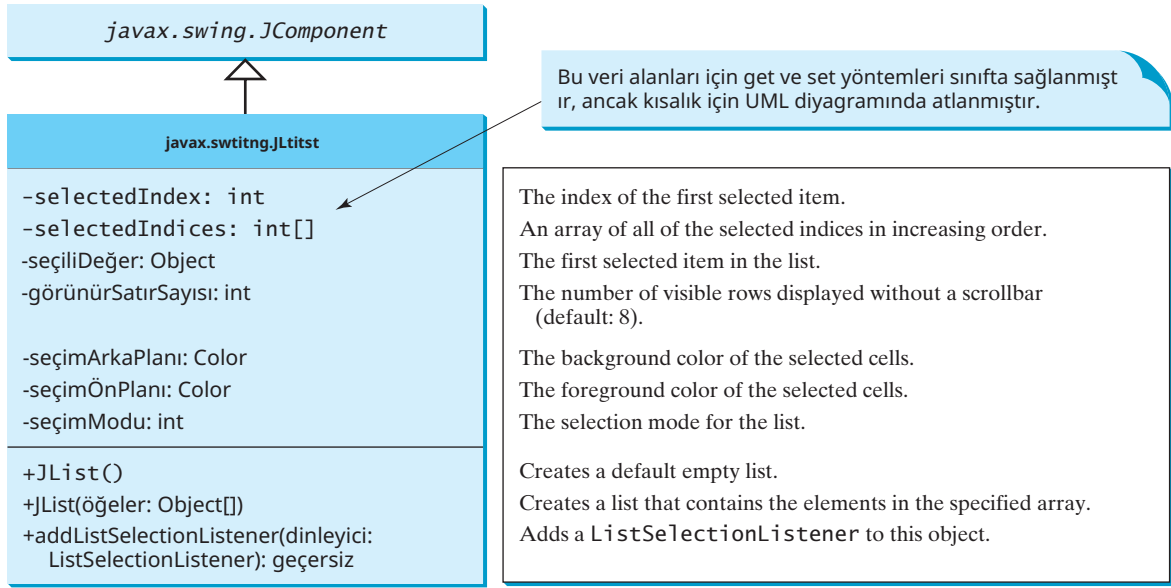
## 17.5 Listeler



*Bir liste, temelde bir açılır kutu ile aynı işlevi yerine getiren bir bileşendir, ancak kullanıcıya tek bir değer veya birden fazla değer seçmesine olanak tanır.*

Swing **JList** çok yönlüdür. Şekil 17.7, **JList**'de sıkça kullanılan birkaç yapıcı ve yöntemi listelemektedir.

**Seçim Modu** javax.swing.ListSelectionModel'da tanımlanan üç değerden biridir (**SINGLE\_SELECTION**, **SINGLE\_INTERVAL\_SELECTION** ve **MULTIPLE\_INTERVAL\_SELECTION**) ve tek bir öğe, tek aralıklı öğe veya çoklu aralıklı öğenin seçilip seçilemeyeceğini belirler. Tek seçim yalnızca bir öğenin seçilmesine izin verir. Tek aralıklı seçim birden fazla seçim izn verir, ancak seçilen öğeler bitişik olmalıdır. Çoklu aralıklı seçim, Şekil 17.8'de gösterildiği gibi, kısıtlama olmaksızın birden fazla bitişik öğenin seçilmesine izin verir. Varsayılan değer **MULTIPLE\_INTERVAL\_SELECTION**'dir.



ŞEKİL 17.7 JList, bir dizi öğeden birden fazla öğe seçmenizi sağlar.



FIGURE 17.8 JList üç seçim moduna sahiptir: tek seçim, tek aralık seçimi, ve çoklu aralık seçimi.

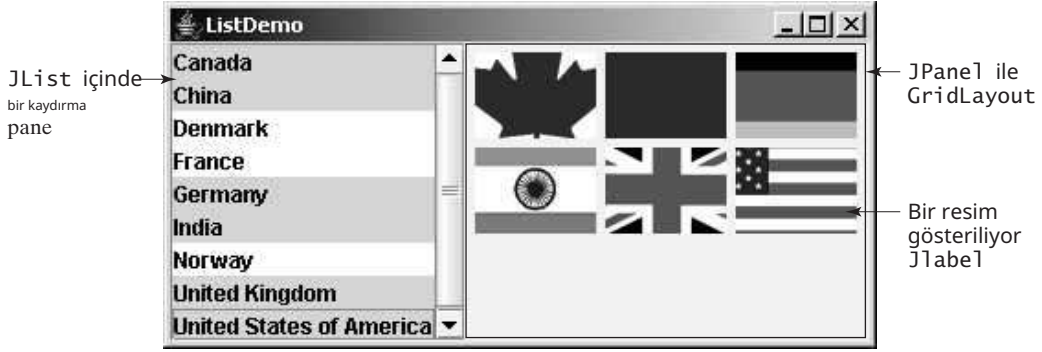
Aşağıdaki ifadeler, altı öğeden oluşan bir liste oluşturur: **kırmızı** ön plan, **beyaz** arka plan, **pembe** seçim ön planı, **siyah** seçim arka planı ve görünür satır sayısı **4**.

```
1 JList jlst = new JList(new String[]
2 {"Öğe 1", "Öğe 2", "Öğe 3", "Öğe 4", "Öğe 5", "Öğe 6"});
3 jlst.setForeground(Color.RED);
4 jlst.setBackground(Color.WHITE);
5 jlst.setSelectionForeground(Color.PINK);
6 jlst.setSelectionBackground(Color.BLACK);
7 jlst.setVisibleRowCount(4);
```

Listeler otomatik olarak kaydırılmaz. Bir listeyi kaydırılabilir hale getirmek için, bir kaydırma paneli oluşturun ve listeyi ona ekleyin.

**JList**, seçimleri dinleyicilere bildirmek için **javax.swing.event.ListSelectionEvent** ateşler. Dinleyici, olayı işlemek için **ValueChangedHandler**'ını **javax.swing.event.ListSelectionListener** arayüzünde uygulamalıdır.

Liste 17.5, kullanıcıların bir listede ülkeleri seçmelerine ve seçilen ülkelerin bayraklarını etiketlerde göstermelerine olanak tanıyan bir programı verir. Şekil 17.9, programın bir örnek çalışmasını göstermektedir.



**FIGURE 17.9** Listede seçilen ülkeler için, bayraklarının görüntüleri etiketlerde görüntülenir.

Programdaki ana adımlar şunlardır:

1. Kullanıcı arayüzünü oluşturun.

Dokuz ülke adını seçim değerleri olarak içeren bir liste oluşturun ve listeyi birkaydırma panelinin içine yerleştirin. Kaydırma panelini çerçevenin batısına yerleştirin. Ülkelerin bayrak görüntülerini göstermek için kullanılacak dokuz etiket oluşturun. Etiketleri panelin içine yerleştirin ve paneli çerçevenin ortasına yerleştirin.

2. Olayı işleyin.

Bir dinleyici oluşturun ve

`değerDeğiştirildi` metodunu

`ListSelectionListener` arayüzünde seçilen ülkelerin bayrak görüntülerini etiketlerde ayarlamak için uygulayın.

### Listing 17.5tListDemo.java

```

1 import java.awt.*;
2 import javax.swing.*;
3 import javax.swing.event.*;
4
5 genel classt ListDemo extends JFrame {
6 final int NUMBER_OF_FLAGS = 9;
7
8 // Bayrak başlıkları için bir String dizisi tanımlayın
9 özel String[] flagTitles = {"Kanada", "Çin", "Danimarka",
10 "Fransa", "Almanya", "Hindistan", "Norveç", "Birleşik Krallık",
11 "Amerika Birleşik Devletleri"};
12
13 // Ülkeleri seçmek için liste
14 BfiraFatJalasa sat = tanut jalasa (hadi gidelim);
15
16 // 9 ülkenin ulusal bayrakları için bir ImageIcon dizisi tanımlayın
17 özel ImageIcon[] imageIcons = {
18 yeniImageIcon("image/ca.gif"),
19 yeniImageIcon("image/china.gif"),
20 yeniImageIcon("image/denmark.gif"),
21 yeniImageIcon("image/fr.gif"),
22 yeniImageIcon("image/germany.gif"),
23 yeniImageIcon("image/india.gif"),
24 yeniImageIcon("image/norway.gif"),
25 yeniImageIcon("image/uk.gif"),
26 yeniImageIcon("image/us.gif")
27 };
28

```

```

29 // Görüntüleri görüntülemek için etiket dizileri
30 PrivatetJLabel[] jlblImageViewer = tnewtJLabel[NUMBER_OF_FLAGS];
31
32 public static voidtmain(String[] args) {n33 ListDe
mo çerçeve = tnewtListDemo();n34 çerçeve.setSiz
e(t650t,t500t);
35 frame.setTitle("ListDemo");
36 frame.setLocationRelativeTo(null); // Çerçeveyi ortal
37 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38 frame.setVisible(true);
39 }
40
41 public ListDemo() {
42 // Dokuz etiketi tutacak bir panel oluştur
43 JPanel p = tnewtJPanel(tnewtGridLayout(t3t,t3t,t5t,t5t));
44
45 fort(tintti = t0t; i < NUMBER_OF_FLAGS; i++) {n46 p.ad
d(jlblImageViewer[i] = tnewtJLabel());n47 jlblImageViewer[i].set
HorizontalAlignmenntn48 (SwingConstants.CENTER);
49 }
50
51 // p'yi ve listeyi çerçeveye ekle
52 add(p, BorderLayout.CENTER);
53 add(newJScrollPane(jlst), BorderLayout.WEST);
54
55 // Dinleyicileri kaydet
56 jlst.addListSelectionListener(newListSelectionListener() {
57 @Override /** Liste seçimlerini yönet */
58 public voidtvalueChanged(ListSelectionEvent e) {
59 // Seçilen indeksleri al
60 intt[] indeksler = jlst.getSelectedIndices();
61
62 int i;
63 // Etiketlerde simgeleri ayarla
64 fort(i = t0t; i < indeksler.uzunluk; i++) {
65 jlblImageViewer[i].setIcon(imageIcons[indices[i]]);n66 }
67
68 // Kalan etiketlerden simgeleri kaldır
69 fort(i < NUMBER_OF_FLAGS; i++) {
70 jlblImageViewer[i].setIcon(tnullt);n71 }
72 }nshz });ns
hch }n75 }

```

oluştur çerçeve

UI oluştur

olay işleyici

Anonim iç sınıf dinleyicisi, listede ülke adlarının seçimini işlemek için **ListSelectionEvent** dinler (satırlar 56–73). **ListSelectionEvent** ve **ListSelectionListener**, **javax.swing.event** paketinde tanımlanmıştır, bu nedenle bu paket programa dahil edilmiştir (satır 3).

Program, dokuz ülkenin bayrak görüntülerini göstermek için dokuz etiketlik bir dizi oluşturur. Program, dokuz ülkenin görüntülerini bir görüntü dizisine yükler (satırlar 17–27) ve başlık dizisindeki aynı sırayla dokuz ülkenin bir listesini oluşturur (satırlar 9–11). Böylece, görüntü dizisinin 0 indeksi, listedeki ilk ülkeye karşılık gelir.

Liste, görüntüleme alanını aşan öğe sayısı olduğunda kaydırılabilmesi için bir kaydırma paneline (satır 53) yerleştirilmiştir.

Varsayılan olarak, listenin seçim modu çoklu aralık olup, bu kullanıcıya listede farklı bloklar dan birden fazla öğe seçme imkanı tanır. Kullanıcı listede ülkeleri seçtiğinde, **valueChanged** işleyicisi (satırlar 58-72) çalıştırılır; bu işleyici, seçilen öğelerin indekslerini alır ve bunların karşılık gelen resim simgelerini etiketin içinde bayrakları gösterecek şekilde ayarlar.

**17.13** Bir dizi string ile nasıl bir liste oluşturursunuz?

**17.14** Bir listede görünür satır sayısını nasıl ayarlarsınız?

**17.15** Bir liste için hangi seçim modları mevcuttur? Seçim modunu nasıl ayarlarsınız?

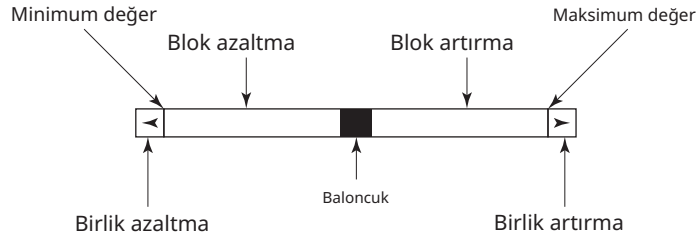
**17.16** Bir listenin ön plan ve arka plan rengini nasıl ayarlarsınız? Seçilen öğelerin ön plan ve arka plan rengini nasıl ayarlarsınız?



## 17.6 Kaydırma Çubukları

**JScrollBar**, kullanıcının bir değer aralığından seçim yapmasını sağlayan bir bileşendir.

Şekil 17.10 bir kaydırma çubuğunu göstermektedir. Normalde, kullanıcı kaydırma çubuğunun değerini fare ile bir hareket yaparak değiştirir. Örneğin, kullanıcı kaydırma çubuğunun balonunu yukarı ve aşağı sürükleyebilir veya kaydırma çubuğunun birim artış veya blok artış alanlarına tıklayabilir. Klavye hareketleri de kaydırma çubuğuna atanabilir. Geleneksel olarak, *Page Up* ve *Page Down* tuşları, kaydırma çubuğunun blok artış ve blok azalış alanlarına tıklamakla eşdeğerdir.



**FIGÜRİK 17.10** Bir kaydırma çubuğu, değer aralığını grafiksel olarak temsil eder.



### Not

Kaydırma çubuğunun yolu, **maksimum + görünürMiktar** ile ilişkilidir.

Bir kaydırma çubuğu maksimum değerine ayarlandığında, baloncukun sol tarafı **maksimum**'dadır, ve sağ tarafı **maksimum + görünürMiktar**'dadır.

**JScrollBar** aşağıdaki özelliklere sahiptir, Şekil 17.11'de gösterildiği gibi.

Kullanıcı kaydırıcı çubuğunun değerini değiştirdiğinde, kaydırıcı çubuğu bir **AdjustmentEvent** tetikler.

Bu olay için bir dinleyici sınıfı, **adjustmentValueChanged** işleyicisini

**java.awt.event.AdjustmentListener** arayüzünde uygulamalıdır.

Liste 17.6, bir panelde görüntülenen bir mesajı kontrol etmek için yatay ve dikey kaydırıcı çubukları kullanan bir programı vermektedir. Yatay kaydırıcı çubuğu mesajı sola ve sağa hareket ettirmek için, dikey kaydırıcı çubuğu ise yukarı ve aşağı hareket ettirmek için kullanılır. Programın bir örnek çalışması Şekil 17.12'de gösterilmektedir.

Programdaki ana adımlar şunlardır:

1. Kullanıcı arayüzünü oluşturun.

Bir **MessagePanel** nesnesi oluşturun ve bunu çerçevenin ortasına yerleştirin. Dikey bir kaydırıcı çubuğu oluşturun ve bunu çerçevenin doğusuna yerleştirin. Yatay bir kaydırıcı çubuğu oluşturun ve bunu çerçevenin güneyine yerleştirin.

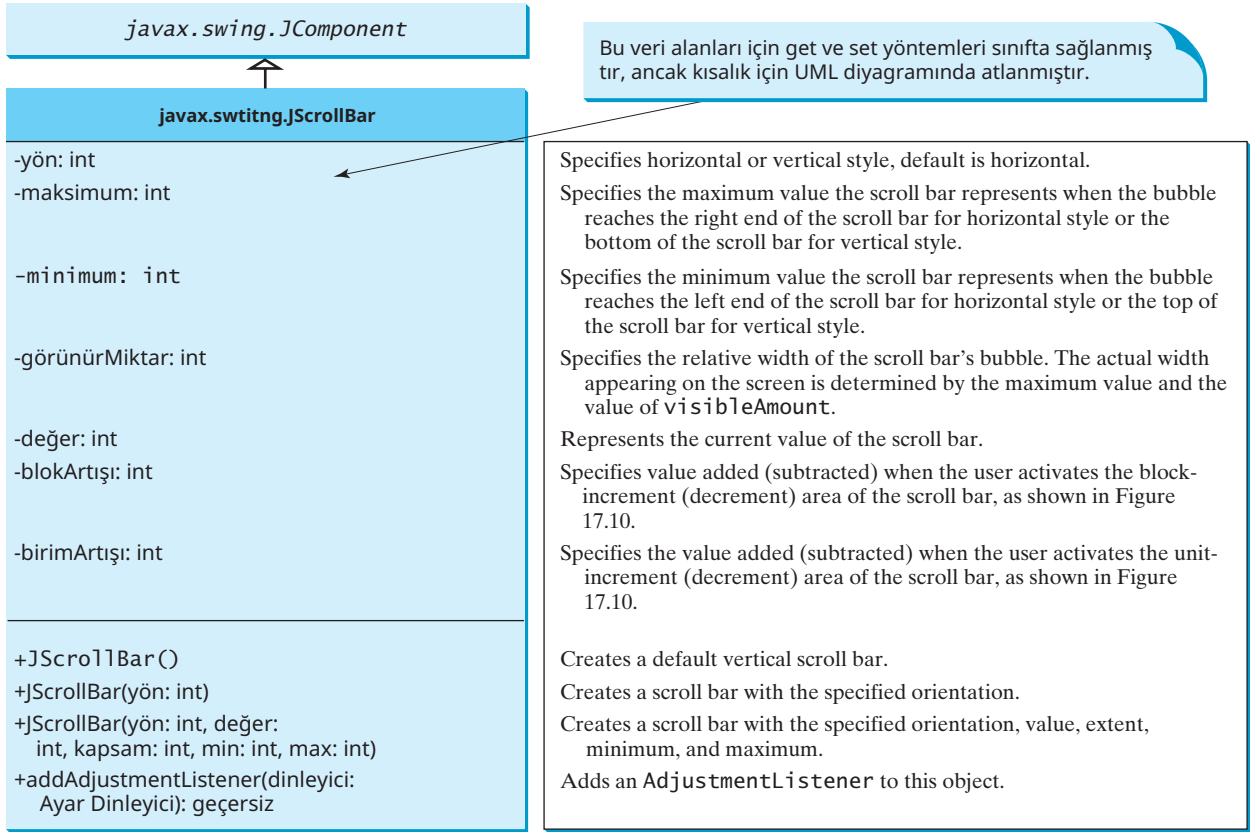


FIGURE 17.11 `JScrollBar`, bir dizi değer arasından seçim yapmanıza olanak tanır.



FIGİN 17.12 Kaydırma çubukları, mesajı bir panelde yatay ve dikey olarak hareket ettirir.

## 2. Olayı işleyin.

Kaydırma çubuklarındaki çubuk hareketine göre mesajı taşımak için `adjustmentValueChanged` işleyicisini uygulamak üzere dinleyiciler oluşturun.

## LİSTE 17.6 `ScrollBarDemo.java`

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 genel class ScrollBarDemo extends JFrame {
6 // Yatay ve dikey kaydırma çubukları oluşturun
7 Privatet JScrollBar jsbHort =

```

yatay kaydırma çubuğu

```

89 newJScrollBar(JScrollBar.HORIZONTAL);
dikey kaydırma çubuğu
101 özelJScrollBar jscbVert =
111 newJScrollBar(JScrollBar.VERTICAL);
12
13 // Bir MessagePanel oluştur
14 özelMessagePanel messagePanel =
15 yeniMessagePanel("Java'ya Hoş Geldiniz");
16
17 genel statik boşmain(String[] args) {
oluştur çerçeve
18 ScrollBarDemo çerçevesi = new ScrollBarDemo();
19 çerçeve.setTitle("ScrollBarDemo");
20 çerçeve.setLocationRelativeTo(null); // Çerçeveyi ortalara
21 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22 çerçeve.pack();
23 çerçeve.setVisible(true);
24 }
25 public ScrollBarDemo() {
26 // Çerçeveye kaydırma çubukları ve mesaj paneli ekle
UI oluştur
27 setLayout(new BorderLayout());
28 add(messagePanel, BorderLayout.CENTER);
29 add(jscbVert, BorderLayout.EAST);
30 add(jscbHort, BorderLayout.SOUTH);
31
32 // Kaydırma çubukları için dinleyici kaydet
ayar dinleyicisi
33 jscbHort.addAdjustmentListener(new AdjustmentListener() {
34 @Geçersiz Kıl
35 genel void adjustmentValueChanged(AdjustmentEvent e) {
36 // getValue() ve getMaximumValue() int döner, ancak daha iyi
37 // hassasiyet için double kullanın
38 double value = jscbHort.getValue();
39 double maximumValue = jscbHort.getMaximum();
40 double newX = (değer * messagePanel.getWidth() /
41 maksimumValue);
42 message
43 Panel.setXCoorder((tint)newX);
44 });
45 jscbVert.addAdjustmentListener(new AdjustmentListener() {
46 @Geçersiz Kıl
47 genel void adjustmentValueChanged(AdjustmentEvent e) {
48 // getValue() ve getMaximum() int döner, ancak daha iyi
49 // hassasiyet için double kullanın
50 double value = jscbVert.getValue();
51 double maximumValue = jscbVert.getMaximum();
52 double newY = (değer * messagePanel.getHeight() /
53 maksimumValue);
54 message
55 Panel.setYCoorder((tint)newY);
56 });
57 astar});

```

Program, iki kaydırma çubuğu (**jscbVert** ve **jscbHort**) (satırlar 7-10) ve bir örneğini oluşturur MessagePanel (messagePanel) (satırlar 13-14). messagePanel çerçevenin ortasına yerleştirilir (satır 28); **jscbVert** ve **jscbHort** çerçevenin doğu ve güney bölümlerine yerleştirilir (satırlar 29-30), sırasıyla.

Kaydırma çubuğunun yönünü yapıcıda belirtebilir veya **setOrientation** metodunu kullanabilirsiniz. Varsayılan olarak, özellik değeri **100** maksimum için, **0** minimum için, **10** blockIncrement için ve **10** visibleAmount için ayarlanmıştır.



Kullanıcı balonu sürüklediğinde veya artırma ya da azaltma birimine tıkladığında, kaydırma çubuğunun değeri değişir. Bir **AdjustmentEvent** örneği tetiklenir ve **adjustmentValueChanged** işleyicisini çağırarak dinleyiciye iletilir. Dikey kaydırma çubuğu için dinleyici mesajı yukarı ve aşağı hareket ettirir (satırlar 33-44), yatay çubuk için dinleyici ise mesajı sağa ve sola hareket ettirir (satırlar 45-56).

Dikey kaydırma çubuğunun maksimum değeri panelin yüksekliğine, yatay kaydırma çubuğunun maksimum değeri ise panelin genişliğine karşılık gelir. Yatay kaydırma çubuğunun mevcut ve maksimum değerleri arasındaki oran, **x** değeri ile mesaj panelinin genişliği arasındaki oranla aynıdır. Benzer şekilde, dikey kaydırma çubuğunun mevcut ve maksimum değerleri arasındaki oran, **y** değeri ile mesaj panelinin yüksekliği arasındaki oranla aynıdır. *x* koordinatı ve *y* koordinatı kaydırma çubuğu ayarlamalarına yanıt olarak ayarlanır (satırlar 39, 50).

- 17.17** Yatay bir kaydırma çubuğu nasıl oluşturulur? Dikey bir kaydırma çubuğu nasıl oluşturulur?
- 17.18** Kullanıcı bir kaydırıcıdaki değeri değiştirdiğinde hangi olayı bir kaydırıcı ateşleyebilir? Bu olay için karşılık gelen arayüz nedir? Arayüzde tanımlanan işleyici nedir?
- 17.19** Bir kaydırıcıdan değeri nasıl alırsınız? Bir kaydırıcıdan maksimum değeri nasıl alırsınız?

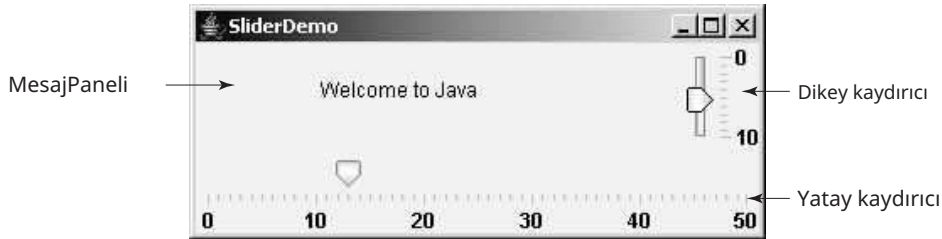
Point Kontrol  
MyProgrammingLab™

## 17.7 Kaydırıcılar

**JSlider**, **JScrollbar** ile benzerlik gösterir, ancak **JSlider** daha fazla özelliğe sahiptir ve birçok biçimde görünebilir.



Şekil 17.13, iki kaydırıcıyı göstermektedir. **JSlider**, kullanıcıya sınırlı bir aralık içinde bir düğmeyi kaydırarak grafiksel olarak bir değer seçme imkanı tanır. Kaydırıcı, hem ana işaretleme işaretlerini hem de bunlar arasındaki küçük işaretleme işaretlerini gösterebilir. İşaretleme işaretleri arasındaki piksel sayısı, **majorTickSpacing** ve **minorTickSpacing** özellikleriyle kontrol edilir. Kaydırıcılar yatay ve/veya dikey olarak, işaretlerle veya işaretsiz, etiketlerle veya etiket olmadan görüntülenebilir.



ŞEKİL 17.13 Kaydırıcılar, mesajı bir panelde yatay ve dikey olarak hareket ettirir.

Sıkça kullanılan yapıcılar ve **JSlider** içindeki özellikler Şekil 17.14'te gösterilmiştir.



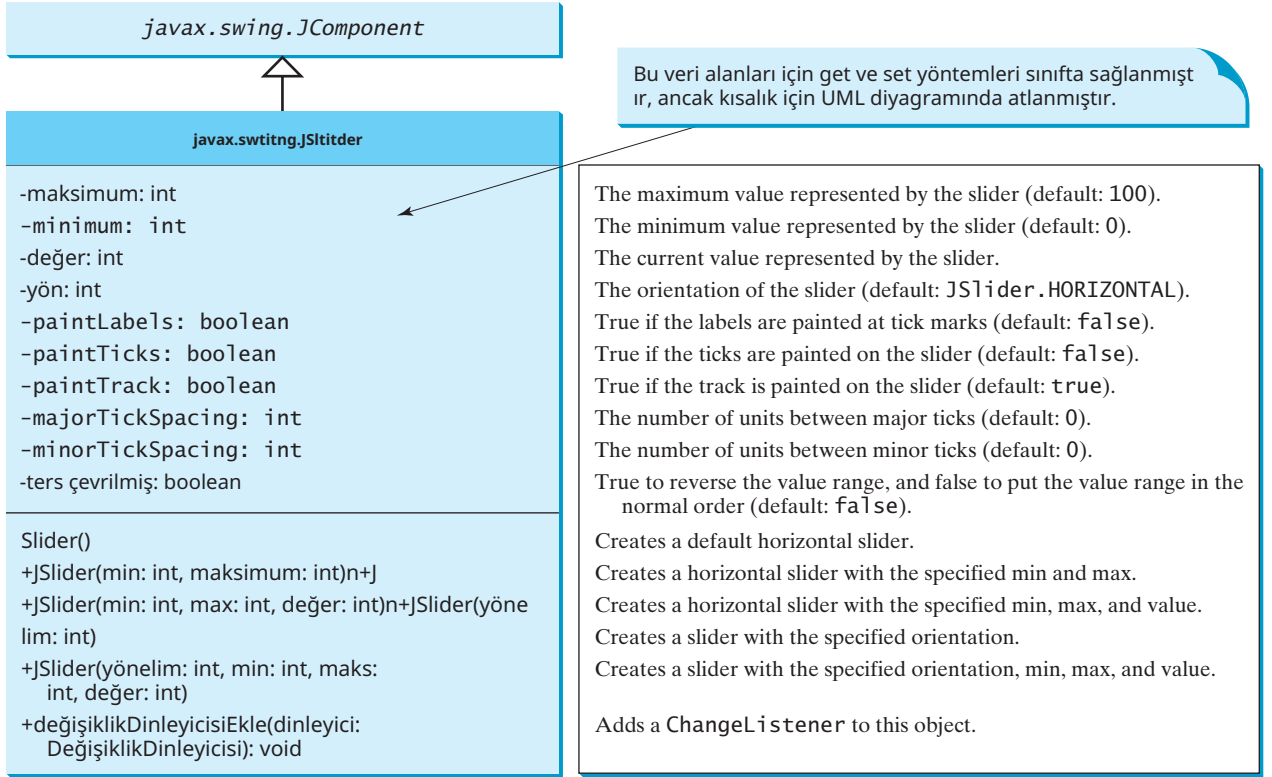
### Not

Dikey bir kaydırıcının değerleri varsayılan olarak üstten alta doğru azalırken, dikey bir kaydırma çubuğunun değerleri üstten alta doğru artar.



### Not

Tüm 17.14 Şekil'de listelenen özelliklerin ilişkili **get** ve **set** yöntemleri vardır, ancak kısalık açısından atlanmıştır. Geleneksel olarak, bir Boolean özelliği için **get** yöntemi **is<PropertyName>()** olarak adlandırılır. **JSlider** sınıfında, **get** yöntemleri için



**FİKİR 17.14 JSlider**, bir değer aralığından seçim yapmanıza olanak tanır.

etiketleriBoyama, işaretleriBoyama, izBoyama, ve ters olarak getPaintLabels(), getPaintTicks(), getPaintTrack() ve getInverted(), adlandırma kuralını ihlal eder.

Kullanıcı kaydırıcının değerini değiştirdiğinde, kaydırıcı bir örneğini tetikler **javax.swing.event.ChangeEvent**, bu da herhangi bir kayıtlı dinleyiciye iletilir. Kaydırıcının değerindeki değişikliklerden haberdar olması gereken herhangi bir nesne, **ChangeListener** arayüzünde tanımlanan stateChanged metodunu uygulamalıdır. **javax.swing.event** paketinde.

Liste 17.7'deki program, bir panelde gösterilen bir mesajı kontrol etmek için kaydırıcıları kullanır, bu da Şekil 17.14'te gösterilmiştir. Programdaki ana adımlar şunlardır:

1. Kullanıcı arayüzünü oluşturun.  
Bir **MessagePanel** nesnesi oluşturun ve bunu çerçevenin ortasına yerleştirin. Dikey bir kaydırıcı oluşturun ve bunu çerçevenin doğusuna yerleştirin. Yatay bir kaydırıcı oluşturun ve bunu çerçevenin güneyine yerleştirin.
2. Olayı işleyin.  
Mesajı kaydırıcıdaki düğüm hareketine göre taşımak için **ChangeListener** arayüzünde stateChanged işleyicisini uygulamak üzere dinleyiciler oluşturun.

### LISTE 17.7 SliderDemo.java

```
1 import java.awt.*;
2 import javax.swing.*;
3 import javax.swing.event.*;
```

```

45 genel classtSliderDemotextendstJFrame {
6 // Yatay ve dikey kaydırıcılar oluşturun
7 PrivatetJSlider jsldHort = tnewtJSlider(JSlider.HORIZONTAL);
8 PrivatetJSlider jsldVert = tnewtJSlider(JSlider.VERTICAL);
9
10 // Bir MessagePanel oluşturun
11 özelMessagePanel messagePanel =
12 yeniMessagePanel ("Java'ya Hoş Geldiniz");
13
14 public static voidtmain(String[] args) {n15 SliderD
emo çerçeve = tnewtSliderDemo();n16 çerçeve.setTitle(
t"SliderDemo"t);
17 frame.setLocationRelativeTo(nu11); // Çerçeveyi ortala
18 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19 frame.pack();
20 çerçeve.setVisible(ttrue);n21 }

22
23 publicSliderDemo() {
24 // Çerçeveye kaydırıcılar ve mesaj paneli ekle
25 setLayout(tnewtBorderLayout(t5t,t5t));n26 add(
messagePanel, BorderLayout.CENTER);n27 add(jsldVert, B
orderLayout.EAST);
28 add(jsldHort, BorderLayout.SOUTH);
29
30 // Kaydırıcılar için özellikleri ayarla
31 jsldHort.setMaximum(50);
32 jsldHort.setPaintLabels(true);
33 jsldHort.setPaintTicks(true);
34 jsldHort.setMajorTickSpacing(10);
35 jsldHort.setMinorTickSpacing(1);
36 jsldHort.setPaintTrack(false);
37 jsldVert.setInverted(true);
38 jsldVert.setMaximum(10);
39 jsldVert.setPaintLabels(true);
40 jsldVert.setPaintTicks(true);
41 jsldVert.setMajorTickSpacing(10);
42 jsldVert.setMinorTickSpacing(1);
43
44 // Kaydırıcılar için dinleyici kaydet
45 jsldHort.addChangeListener(newChangeListener() {
46 @Override /** Kaydırıcı ayar eylemlerini işleme */
47 public voidtstateChanged(ChangeEvent e) {
48 // getValue() ve getMaximumValue() int döner, ancak daha iyi
49 // hassasiyet için double kullanın
50 doubletvalue = jsldHort.getValue();
51 doubletmaximumValue = jsldHort.getMaximum();
52 doubletnewX = (değer * messagePanel.getWidth()) /
53 maksimumValue);n54 message
Panel.setXCoorder((tintt)newX);n55 }

56 });
57 jsldVert.addChangeListener(newChangeListener() {
58 @Override /** Kaydırıcı çubuğu ayarlama eylemlerini işleyin */
59 public voidtstateChanged(ChangeEvent e) {
60 // getValue() ve getMaximum() int döner, ancak daha iyi
61 // hassasiyet için double kullanın
62 doubletvalue = jsldVert.getValue();
63 doubletmaximumValue = jsldVert.getMaximum();

```

yatay kaydırıcı  
dikey kaydırıcı

oluştur çerçeve

UI oluştur

kaydırıcı özellikleri

dinleyici

dinleyici

```

64 double newY = (değer * messagePanel.getHeight() /
65 maksimumDeğer);
66 messagePanel.setYCoordinate((int) newY);
67 }
68 });n69 }n70
}

```

**JSlider**, **JScrollbar** ile benzerlik gösterir ancak daha fazla özelliğe sahiptir. Bu örnekte gösterildiği gibi, bir **JSlider** üzerinde maksimum, etiketler, büyük ve küçük işaretler belirtebilirsiniz (satırlar 31–35). Ayrıca, yolu gizlemeyi de seçebilirsiniz (satır 36). Dikey bir kaydırıcının varsayılan değerleri üstten alt a doğru azaldığı için, **setInverted** metodu sıralamayı tersine çevirir (satır 37).

**JSlider**, kaydırıcı değiştiğinde **ChangeEvent** tetikler. Dinleyici, **ChangeListener** içinde **stateChanged** işleyicisini uygulamalıdır (satırlar 45–68). **JScrollbar**'nın ayarlandığında **AdjustmentEvent** tetiklediğini unutmayın.

**17.20** Yatay bir kaydırıcı nasıl oluşturursunuz? Dikey bir kaydırıcı nasıl oluşturursunuz?

**17.21** Kullanıcı kaydırıcıda değeri değiştirdiğinde hangi olay tetiklenebilir? Olay için karşılık gelen arayüz nedir? Arayüzde tanımlanan işleyici nedir?

**17.22** Bir kaydırıcıdan değeri nasıl alırsınız? Bir kaydırıcıdan maksimum değeri nasıl alırsınız?

Point Kontrol

MyProgrammingLab™

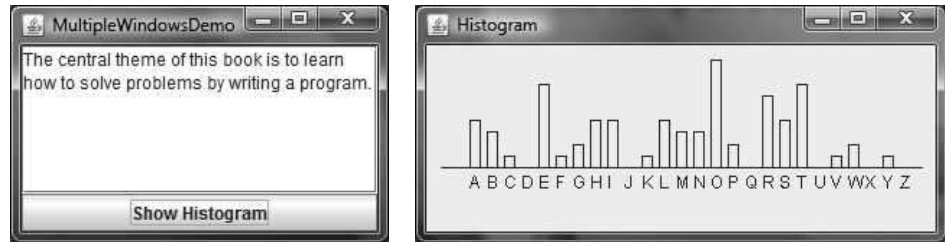
## 17.8 Birden Fazla Pencere Oluşturma



*Bir programda birden fazla pencere oluşturulabilir.*

Bazen, uygulamanın belirli bir görevi yerine getirmek için yeni bir pencere açabilmesi için bir uygulamada birden fazla pencere oluşturmak isteyebilirsiniz. Yeni pencerelere alt pencereler denir ve ana çerçeve ana pencere olarak adlandırılır.

Liste 17.8, kaydırma panelinde bir metin alanı ve *Histogramı Göster* adında bir düğme içeren bir ana pencere oluşturan bir programı vermektedir. Kullanıcı düğmeye tıkladığında, metin alanındaki harflerin oluşumlarını gösteren bir histogramı görüntüleyen yeni bir pencere açılır. Şekil 17.15, programın bir örnek çalışmasını içermektedir.



ŞEKİL 17.15 Histogram ayrı bir çerçevede görüntülenir.

Programdaki ana adımlar şunlardır:

1. Liste 17.8'deki çerçeve için **MultipleWindowsDemo** adında bir ana sınıf tanımlayın. Bir kaydırma panelinin içine bir metin alanı ekleyin ve kaydırma panelini çerçevenin ortasına yerleştirin. Bir buton oluşturun *Histogramı Göster* ve bunu çerçevenin güneyine yerleştirin.
2. Listing 17.9'da **JPanel** sınıfının bir alt sınıfını **Histogram** olarak tanımlayın. Sınıf, 26 harfin oluşumlarını sayan **int[]** türünde bir **count** veri alanı içerir. **count** içindeki değerler histogramda görüntülenir.

3. `MultipleWindowsDemo` içinde `actionPerformed` işleyicisini aşağıdaki gibi uygulayın:
  - a. **Histogram** nesnesinin bir örneğini oluşturun. Metin alanındaki harfleri sayın ve sayıyı **Histogram** nesnesinde ayarlayın.
  - b. Yeni bir çerçeve oluşturun ve **Histogram** nesnesini çerçevenin ortasına yerleştirin. Çerçeveyi gösterin.

## Listing 17.8 MultipleWindowsDemo.java

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 genel sınıf MultipleWindowsDemo extends JFrame {
6 özel JTextArea jta;
7 private JButton jbtShowHistogram = new JButton("Histogramı Göster");
8 Private Histogram histogram = new Histogram();
9
10 // Histogram panelini tutmak için yeni bir çerçeve oluştur
11 Private JFrame histogramFrame = new JFrame();
12
13 public MultipleWindowsDemo() {
14 // Metin alanını bir kaydırma panelinde sakla
15 JScrollPane ScrollPane = new JScrollPane(jta = new JTextArea());
16 ScrollPane.setPreferredSize(new Dimension(300, 200));
17 jta.setWrapStyleWord(true);
18 jta.setLineWrap(true);
19
20 // Kaydırma panelini ve butonu çerçeveye yerleştir
21 add(ScrollPane, BorderLayout.CENTER);
22 add(jbtShowHistogram, BorderLayout.SOUTH);
23
24 // Dinleyiciyi kaydet
25 jbtShowHistogram.addActionListener(new ActionListener() {
26 @Override /** Buton eylemini işleyin */
27 public void actionPerformed(ActionEvent e) {
28 // Metin alanındaki harfleri say
29 int[] count = countLetters();
30
31 // Harf sayısını histogramda görüntülemek için ayarla
32 histogram.showHistogram(count);
33
34 // Çerçeveyi göster
35 histogramFrame.setVisible(true);
36 }
37 });
38
39 // Histogram panelini çerçeveye ekle
40 histogramFrame.add(histogram);
41 histogramFrame.pack();
42 histogramFrame.setTitle("Histogram");
43 }
44
45 /** Metin alanındaki harfleri say */
46 private int[] harfleriSay() {
47 // 26 harf için sayım
48 int[] sayım = new int[26];
49
50 // Metin alanından içerikleri al

```

alt çerçeve oluştur

UI oluştur

görüntü alt çerçevesi

## 662 Bölüm 17 GUI Bileşenleri

```
51 Dize metni = jta.getText();
5253 // Her harfin (büyük/küçük harf duyarsız) kaç kez geçtiğini say
54 for(tint = t0t; i < text.length(); i++) {
55 chartcharacter = text.charAt(i);
5657 ift(character >= t'A't&& karakter <= t'Z't) {n58 count[
character - t'A't]++;n59 }

60 else ift(karakter >= t'a't&& karakter <= t'z't) {
61 sayı[karakter - t'a't]++;n62 }

63 }
6465 return count; // Sayı dizisini döndür
66 }

6768 public static void main(String[] args) {
69 MultipleWindowsDemo frame = new MultipleWindowsDemo();
70 frame.setLocationRelativeTo(null); // Çerçeveyi ortalara
71 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
72 frame.setTitle("MultipleWindowsDemo");
73 çerçeve.pack();n74 çerç
eve.setVisible(true);n75 }

76 }
```

ana çerçeveyi oluştur

### Listing 17.9tHistogram.java

```
1 import javax.swing.*;
2 import java.awt.*;
34
kamu sınıfı Histogram extends JPanel {
5 // 26 harfin oluşumlarını say
9 özel int[] sayı;
78
/** Sayıyı ayarla ve histogramı göster */
9 public void showHistogram(int[] sayı) {
10 thist.count = sayı;
11 yeniden boyama();
n12 }
13
14 @Geçersiz Kıl /** Histogramı boyama */
15 korumalı void paintComponent(Graphics g) {
16 eğer(count == null) return; // count null ise görüntüleme yok
1718
supert.paintComponent(g);

1920 // Panel boyutunu ve çubuk genişliğini ve aralığını dinamik olarak bulun
21 int genişlik = getWidth();
22 int yükseklik = getHeight();
23 int aralık = (genişlik - 40) / count.length;
24 int bireyselGenişlik = (int)((genişlik - 40) / 24) * 0.60;
2526
// Maksimum sayıyı bulun. Maksimum sayı en yüksek çubuğa sahiptir
27 int maksSayı = 0;
28 for(tintti = t0t; i < sayı.uzunluk; i++) {
29 if(maksSayı < count[i])
30 maksSayı = count[i];
31 }
```

histogramı boyama

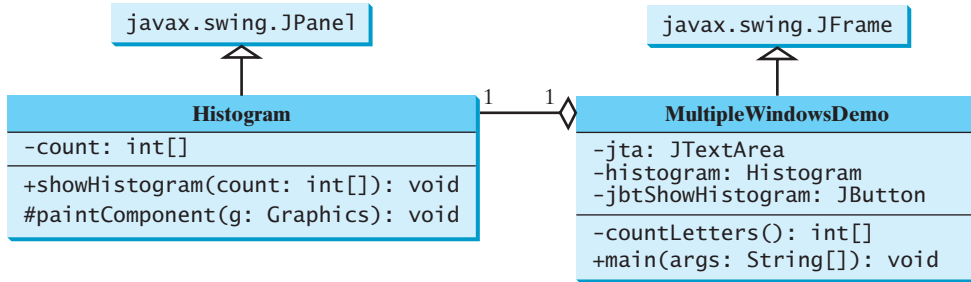
```

3233 // x, histogramdaki ilk çubuğun başlangıç pozisyonudur
34 inttx = t30t;
35
36 // Yatay bir taban çizgisi çizin
37 g.drawLine(t10t, yükseklik - t45t, genişlik - t10t, yükseklik - t45t);
38 fort(intti = t0t; i < sayım.uzunluk; i++) {
39 // Çubuk yüksekliğini bulun
40 intbar Yüksekliği =
 (tintt)(((tdoublet)count[i] / (tdoublet)maxCount) * (yükseklik - t55t));
42
43 // Bir çubuk (yani, dikdörtgen) göster
44 g.drawRect(x, height - 45- barHeight, individualWidth,
45 barHeight);
46
47 // Temel çizginin altında bir harf göster
48 g.drawString((char)(65+ i) + "", x, height - 30);
49
50 // Bir sonraki karakteri göstermek için x'i hareket ettir
51 x += interval;
52 }
53 }
54
55 @Geçersiz Kıl
56 publicDimension getPreferredSize() {
57 return newDimension(t300t,t300t);
58 }
59 {nass
60 }

```

tercihEdilenBoyut

Program iki sınıf içerir: **MultipleWindowsDemo** ve **Histogram**. Aralarındaki ilişkisi Şekil 17.16'da gösterilmektedir.



ŞEKİL 17.16 **MultipleWindowsDemo**, bir çerçevede bir metin alanındaki harflerin

oluşumlarının histogramını göstermek için **Histogram** kullanır.

**MultipleWindowsDemo**, bir metin alanını bir kaydırma panelinde ve bir düğmeyi tutan bir çerçevedir.

**Histogram**, metin alanındaki harflerin oluşumları için bir histogram gösteren **JPanel** sınıfının bir alt sınıfıdır.

Listenin 17.8, **MultipleWindowsDemo.java**'da, kullanıcı **Show Histogram** düğmesine tıkladığında, işleyici metin alanındaki harflerin oluşumlarını sayar (satır 29). Harfler, büyük/küçük harf durumuna bakılmaksızın sayılır. Harf olmayan karakterler sayılmaz. Sayım, 26 elemanlı bir `int` dizisinde saklanır (satır 48). Dizinin ilk elemanı `a` veya `A` harfi için sayımı saklarken, son eleman `z` veya `Z` harfi için sayımı saklar (satırlar 57–63). Sayım dizisi, görüntüleme için **histogram**'a iletilir (satır 32).

**MultipleWindowsDemo** sınıfı bir `main` metodunu içerir. `Main` metodu, **MultipleWindowsDemo**'nun bir örneğini oluşturur ve çerçeveyi görüntüler (satırlar 69–74).

**MultipleWindowsDemo** sınıfı ayrıca bir örneğini içerir **histogramFrame** (satır 8), bir **Histogram** örneğini tutar. Kullanıcı

**JFrame**, adlandırılmış

*Show Histogram* butonuna tıkladığında, **histogramFrame** histogramı görüntülemek için görünür olarak ayarlanır (satır 35).

Histogramdaki çubukların yüksekliği ve genişliği, histogramın pencere boyutuna göre dinamik olarak belirlenir.

Bir **JFrame** örneğini bir konteynıra ekleyemezsiniz. Örneğin, **histogramFrame**'i ana çerçeve ye eklemek bir çalışma zamanı istisnasına neden olur. Ancak, bir çerçeve örneği oluşturabilir ve yeni bir pencere açmak için görünür hale getirebilirsiniz.



MyProgrammingLab™

**17.23** Bir uygulamada birden fazla pencere nasıl oluşturulup gösterileceğini açıklayın.

## BÖLÜM ÖZETİ

1. JCheckBox, JRadioButton ve JTextField için olayları nasıl yöneteceğinizi öğrendiniz.
2. Swing GUI bileşenlerini kullanarak grafiksel kullanıcı arayüzleri oluşturmayı öğrendiniz. **JTextArea**, **JComboBox**, **JList**, **JScrollbar**, ve **JSlider**. Bu bileşenlerde olayları nasıl yöneteceğinizi de öğrendiniz.
3. Birden fazla pencereyi **JFrame** kullanarak nasıl başlatacağınızı öğrendiniz.

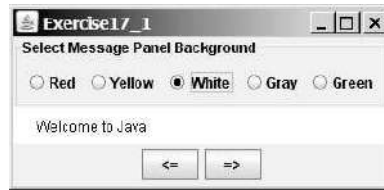
## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

## PROGRAMLAMA EGZERSİZLERİ

### Bölümler 17.2–17.5

- \*17.1** (*Radyo düğmeleri kullanın*) Şekil 17.17'de gösterildiği gibi bir GUI programı yazın. Mesajı sağa ve sola hareket ettirmek için düğmeleri kullanabilir ve mesaj panelinde görüntülenen mesajın arka plan rengini değiştirmek için radyo düğmelerini kullanabilirsiniz.



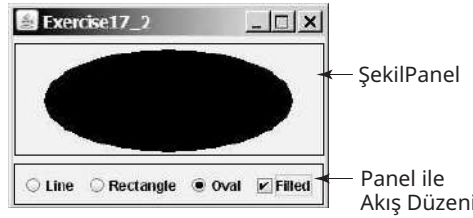
**ŞEKİL 17.17** <= ve => Düğmeler, paneldeki mesajı hareket ettirir ve radyo düğmeleri mesajın arka plan rengini değiştirir.

- \*17.2** (*Geometrik şekilleri seçin*) Şekil 17.18'de gösterildiği gibi çeşitli şekiller çizen bir program yazın. Kullanıcı, bir radyo düğmesinden bir şekil seçer ve bir onay kutusu kullanarak şeklin dolu olup olmadığını belirtir. (İpucu: Bir şekli görüntülemek için

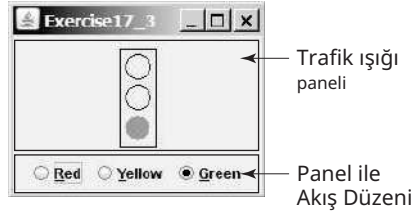
Liste 13.3'te tanımlanan **FigurePanel** sınıfını kullanın.)

- \*\*17.3** (*Trafik ışıkları*) Bir trafik ışığını simüle eden bir program yazın. Program, kullanıcının kırmızı, sarı veya yeşil olmak üzere üç ışıktan birini seçmesine izin verir. Bir radyo düğmesi seçildiğinde, ışık yanar ve aynı anda yalnızca bir ışık yanabilir (bkz. Şekil 17.19). Program başladığında hiçbir ışık yanmaz.





**FIGÜR 17.18** Program, bir şekil türü seçtiğinizde çizgiler, dikdörtgenler ve ovalar görüntüler.



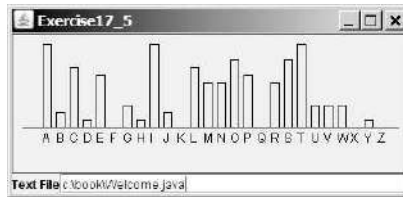
**FIGÜR 17.19** Radyo düğmeleri, bir trafik ışığını kontrol etmek için gruptaki yalnızca bir rengi seçmenize olanak tanımak için gruplanmıştır.

## Bölümler 17.6–17.8

**\*\*17.4** (Metin görüntüleyici) Şekil 17.20a' da gösterildiği gibi bir metin dosyasını bir metin alanında görüntüleyen bir program yazın. Kullanıcı, bir metin alanına bir dosya adı girer ve **Görüntüle** düğmesine tıklar; dosya daha sonra bir metin alanında görüntülenir.



(a)



(b)

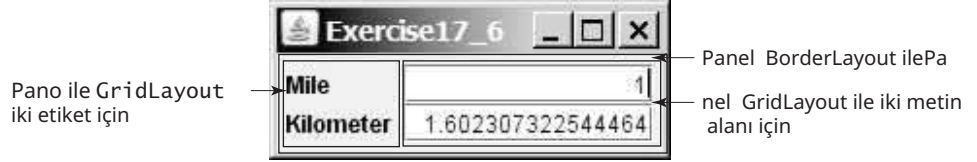
**FIGÜR 17.20** (a) Program, bir dosyadan metni bir metin alanında görüntüler. (b) Program, dosyadaki her harfin meydana gelişlerini gösteren bir histogram görüntüler.

**\*\*17.5** (Harflerin oluşumları için bir histogram oluşturun) Listing 17.8'deki program, MultipleWindow sDemo.java, bir metin alanındaki her harfin oluşumlarını göstermek için bir histogram görüntüler. Listing 17.9'da oluşturulan **Histogram** sınıfını yeniden kullanarak, bir panelde histogram görüntüleyecek bir program yazın. Histogram, bir metin dosyasında ki her harfin oluşumlarını göstermelidir; Şekil 17.20b'de gösterildiği gibi. Harflerin büyük/küçük harf duyarlı olmadığını varsayın.

- Histogramı gösterecek paneli çerçevenin ortasına yerleştirin.
- Bir etiket ve bir metin alanını bir panele yerleştirin ve paneli çerçevenin güney tarafına koyun. Metin dosyası bu metin alanından girilecektir.
- Metin alanında **Enter** tuşuna basmak, programın her harfin oluşumlarını saymasını ve sayıyı bir histogramda görüntülemesini sağlar.

**\*17.6** (Mil/kilometre dönüştürücü oluşturun) Mil ve

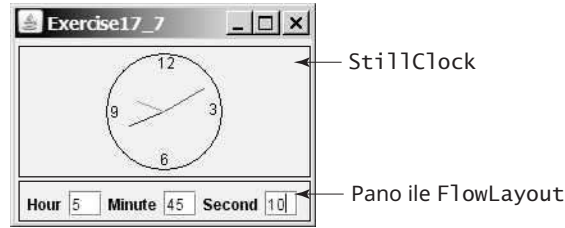
kilometreleri dönüştüren bir program yazın, Şekil 17.21'de gösterildiği gibi. Mile metin alanına bir değer girdiğinizde ve *Enter* tuşuna bastığınızda, karşılık gelen kilometre ölçümü Kilometre metin alanında görüntülenir. Benzer şekilde, Kilometre metin alanına bir değer girdiğinizde ve *Enter* tuşuna bastığınızda, karşılık gelen mil Mile metin alanında görüntülenir.



**FIGURE 17.21** Program, mil ile kilometre arasında dönüşüm yapar ve tersine.

**\*17.7** (Saat ayarını yap) Üç metin alanından gelen girdi ile saati ayarlayan ve bir saat görüntüleyen bir program yazın, Şekil 17.22'de gösterildiği gibi. **StillClock**'ı

Liste 13.10'da kullanın.

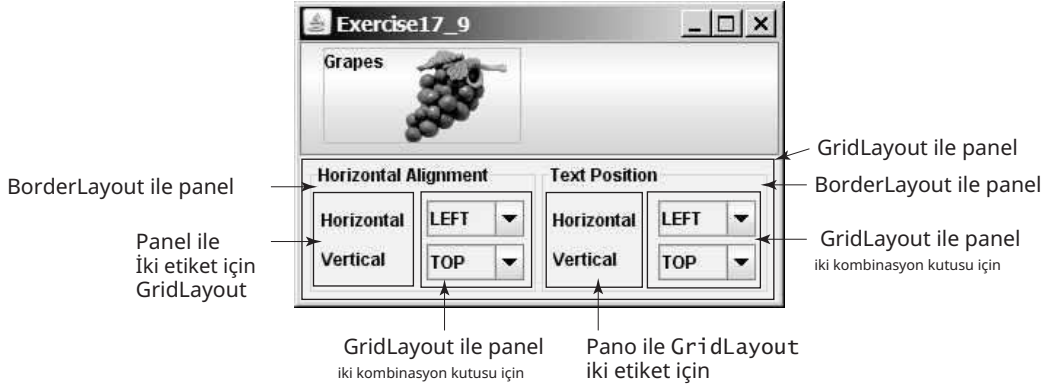


**FIGURE 17.22** Program, metin alanlarında belirtilen zamanı gösterir.

**\*\*17.8** (Bir yazı tipi seç) Bir panelde görüntülenecek bir mesajın yazı tipini dinamik olarak değiştirebilen bir program yazın. Mesaj aynı anda kalın ve italik olarak görüntülenebilir veya panelin ortasında görüntülenebilir. Yazı tipi adını veya yazı tipi boyutunu, Şekil 17.23'te gösterildiği gibi, açılır kutulardan seçebilirsiniz. Mevcut yazı tipi adları, `getAvailableFontFamilyNames()` kullanılarak elde edilebilir `GraphicsEnvironment`'da (bkz. Bölüm 12.8, **Font** Sınıfı). Yazı tipi boyutu için açılır kutu, 1 ile 100 arasındaki sayılarla başlatılır.

**FIGURE 17.23** Mesaj için yazı tipini dinamik olarak ayarlayabilirsiniz.

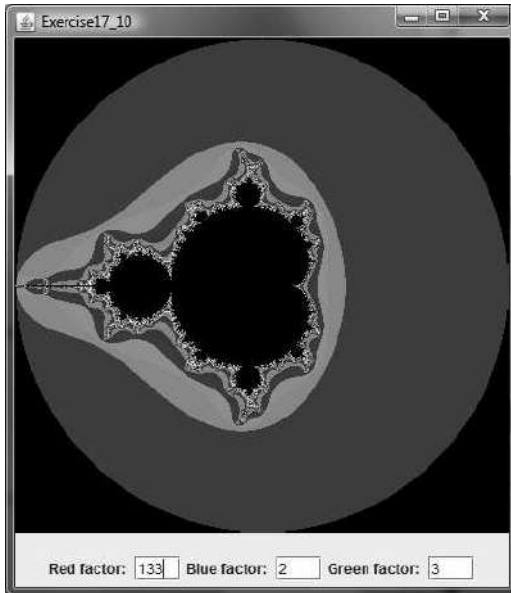
**\*\* 17.9** (*JLabel özelliklerini gösterin*) Kullanıcının dinamik olarak özellikleri ayarlamasını sağlayan bir program yazın `horizontalAlignment`, `verticalAlignment`, `horizontalTextAlignment` ve `verticalTextAlignment`, Şekil 17.24'te gösterildiği gibi.



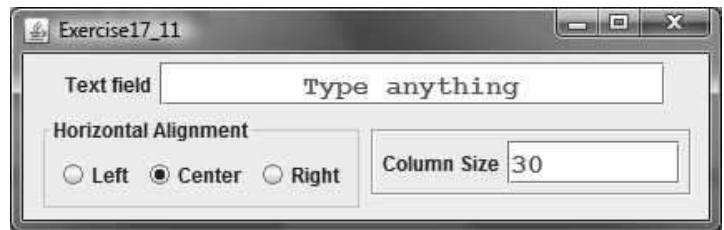
**FIGÜR 17.24** Bir etiketin hizalama ve metin konumlandırma özelliklerini dinamik olarak ayarlayabilirsiniz.

**\* 17.10** (*Mandelbrot fraktalı*) Programlama Egzersizi 15.20, Mandelbrot fraktalını görüntüler.

15. satırdaki 77,58 ve 159 değerlerinin `MandelbrotCanvas` sınıfında Programlama Egzersizi 15.20'de görüntünün rengini etkilediğini unutmayın. Kullanıcının bu değerleri metin alanlarından dinamik olarak girmesine izin verecek şekilde programı gözden geçirin, bu şekilde Şekil 17.25a'da görüldüğü gibi.



(a)



(b)

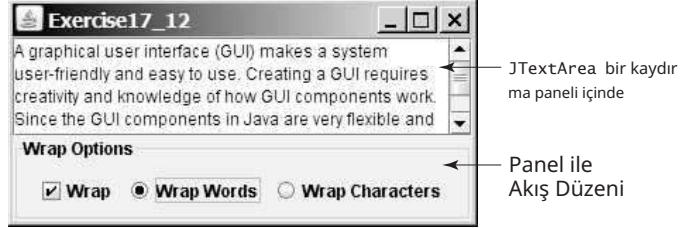
**FIGÜR 17.25** (a) Program, kullanıcının renkleri dinamik olarak ayarlamasına olanak tanır. (b) Bir metin alanının yatay hizalama ve sütun boyutu özelliklerini dinamik olarak ayarlayabilirsiniz.



VideoNot

Metin alanlarını kullanın

- \*17.11** (*JTextField özelliklerini göster*) Yatay hizalama ve sütun boyutu özelliklerini dinamik olarak ayarlayan bir program yazın, Şekil 17.25b'de gösterildiği gibi.
- \*17.12** (*JTextArea özelliklerini göster*) Metin alanının sarma stillerini gösteren bir program yazın. Program, metin alanının sarılı olup olmadığını belirtmek için bir onay kutusu kullanır. Metin alanı sarılıysa, karakterler veya kelimelerle sarılıp sarılma şeklini belirtebilirsiniz, Şekil 17.26'da gösterildiği gibi.



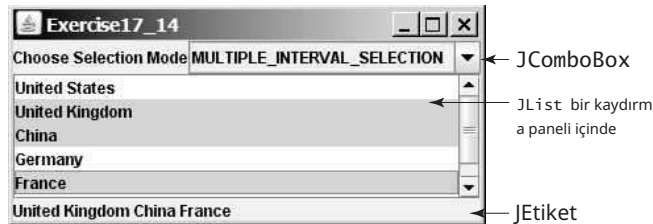
**FIGURE 17.26** Bir metin alanının karakterler veya kelimelerle dinamik olarak sarılacak şekilde ayarlarını yapabildiğini gösteren bir program.

- \*17.13** (*Çeşitli faiz oranlarıyla kredileri karşılaştır*) Programlama Alıştırması 4.21'i, Şekil 17.27'de gösterildiği gibi bir kullanıcı arayüzü oluşturacak şekilde yeniden yazın. Program, kullanıcının bir metin alanından kredi miktarını ve kredi süresini yıl cinsinden girmesine olanak tanımalı ve her faiz oranı için aylık ve toplam ödemeleri, %5'ten %8'e kadar, sekizde bir artışlarla bir metin alanında göstermelidir.



**FIGURE 17.27** Program, çeşitli faiz oranlarına dayalı olarak belirli bir kredi için aylık ödemeleri ve toplam ödemeleri gösteren bir tablo görüntüler.

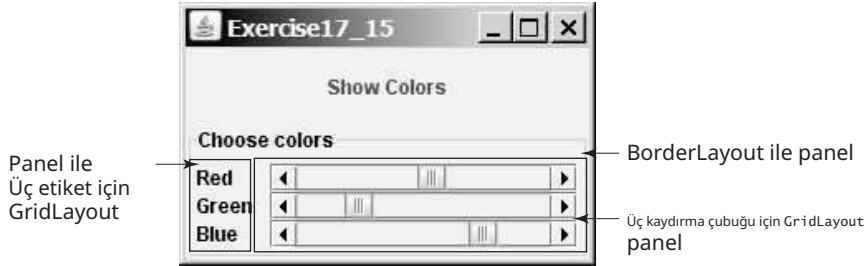
- \*17.14** (*JComboBox ve JList kullan*) Bir listede öğeleri seçmeyi gösteren bir program yazın. Program, seçim modunu belirtmek için bir açılır kutu kullanır, Şekil 17.28'de gösterildiği gibi. Öğeleri seçtiğinizde, bunlar listenin altında bir etiket içinde görüntülenir.



**FIGURE 17.28** Bir listede tek seçim, tek aralık seçimi veya çoklu aralık seçimi yapabilirsiniz.

## Bölümler 17.6–17.8

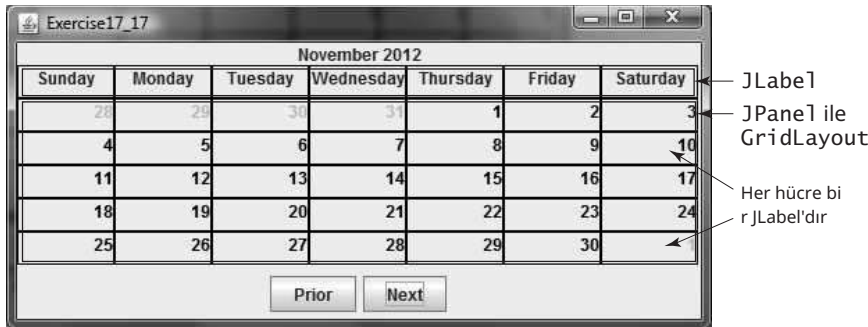
- \*\* 17.15** (Kaydırma Çubuğu Kullanın) Şekil 17.29'da gösterildiği gibi, bir etiket için ön plan rengini seçmek için kaydırma çubukları kullanan bir program yazın. Renklerin kırmızı, yeşil ve mavi bileşenlerini seçmek için üç yatay kaydırma çubuğu kullanılır. Kaydırma çubuklarını tutan panelde bir başlık kenarlığı kullanın.



**FIGURE 17.29** Kaydırma çubuklarını ayarladıkça etiketin ön plan rengi değişir.

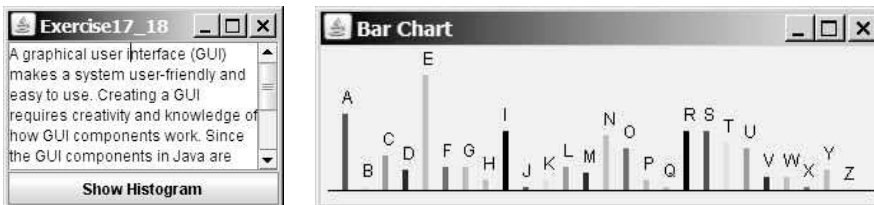
- \*\* 17.16** (JSlider Kullanın) Önceki alıştırmayı kaydırcılar kullanarak gözden geçirin.

- \*\*\* 17.17** (Bir takvim göster) Mevcut ayın takvimini gösteren bir program yazın. Önceki veya sonraki ayın takvimini göstermek için *Önceki* ve *Sonraki* düğmelerini kullanabilirsiniz. Mevcut ayın tarihlerini siyah, önceki ayın ve sonraki ayın tarihlerini ise gri olarak gösterin, Şekil 17.30'da gösterildiği gibi.



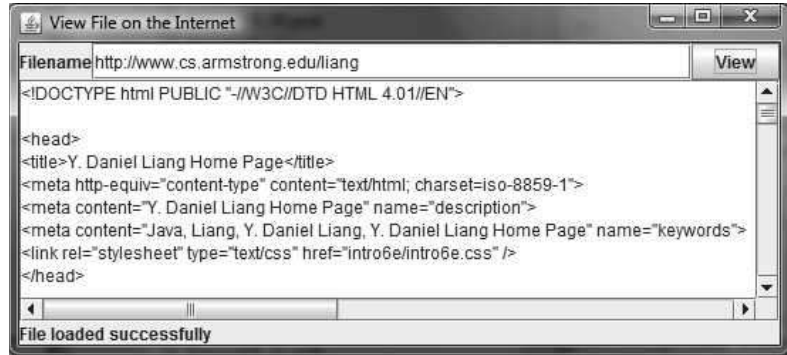
**FIGURE 17.30** Program mevcut ay için takvimi görüntüler.

- \* 17.18** (Liste 17.8'i gözden geçirin, *MultipleWindowsDemo.java*) Liste 17.8'deki **Histogram** bileşenini kullanarak harflerin oluşumlarını göstermek yerine, bir çubuk grafik kullanın, böylece görüntü Şekil 17.31'de gösterildiği gibi olsun.



**FIGURE 17.31** Her harfin kaç kez geçtiği bir çubuk grafikte gösterilir.

- \*\*17.19** (*Ülke bayrağını ve bayrak açıklamasını görüntüle*) Liste 17.4, ComboBoxDemo.java, kullanıcıların bir combo kutusundan ülkeyi seçerek bir ülkenin bayrak resmini ve açıklamasını görmesini sağlayan bir program verir. Açıklama, programda kodlanmış bir dizedir. Programı, metin açıklamasını bir dosyadan okumak için yeniden yazın. Varsayalım ki açıklamalar **description0.txt**, . . . , ve **description8.txt** dosyalarında text dizininde Kanada, Çin, Danimarka, Fransa, Almanya, Hindistan, Norveç, Birleşik Krallık ve Amerika Birleşik Devletleri olmak üzere bu sırayla saklanmaktadır.
- \*\*17.20** (*Slayt gösterisi*) Programlama Egzersizi 16.13, görüntüler kullanarak bir slayt gösterisi geliştirdi. O programı, metin dosyaları kullanarak bir slayt gösterisi geliştirmek için yeniden yazın. Varsayalım ki **slide0.txt**, **slide1.txt**, . . . , ve **slide9.txt** adında on metin dosyası **text** dizininde saklanmaktadır. Her slayt bir dosyadan metni görüntüler. Her slayt bir saniye boyunca gösterilir ve slaytlar sırayla görüntülenir. Son slayt bittiğinde, ilk slayt yeniden gösterilir ve bu şekilde devam eder. Slaytı görüntülemek için bir metin alanı kullanın.
- \*\*17.21** (*Web'den dosyaları al*) Bir Java programı yazın, bu program bir dosyayı bir Web sunucusundan alır, Şekil 17.32'de gösterildiği gibi. Kullanıcı arayüzü, dosya adının URL'sini girmek için bir metin alanı, dosyanın gösterileceği bir metin alanı ve bir eylemi göndermek için kullanılacak bir düğme içerir. Uygulamanın altına, *Dosya başarıyla yüklendi veya Ağ bağlantı problemi* gibi durumu belirtmek için bir etiket eklenir.



ŞEKİL 17.32 Program, İnternette belirtilen bir dosyanın içeriğini görüntüler.

## APLET'LER VE MULTİMEDYA

### Hedefler

- GUI uygulamalarını applet'lere dönüştürmek (§18.2).
- Applet'leri Web sayfalarına gömmek (§18.3).
- Applet'leri Web tarayıcılarından ve **appletviewer** komutundan çalıştırmak (§§18.3.1–18.3.2).
- Applet güvenlik kum havuzu modelini anlamak (§18.4).
- Hem uygulama hem de applet olarak çalışabilen bir Java programı yazmak (§18.5).
- Applet yaşam döngüsü yöntemlerini **init**, **start**, **stop** ve **destroy** geçersiz kılmak (§18.6).
- HTML'den applet'lere string değerleri geçirmek için (§18.7).
- Zıplayan bir top için bir animasyon geliştirmek (§18.8).
- Tic-tac-toe oyunu için bir applet geliştirmek (§18.9).
- URL sınıfını kullanarak kaynakları (görüntüler ve sesler) bulmak (§18.10).
- Herhangi bir Java programında ses çalmak için (§18.11).





## 18.1 Giriş



*Java appletleri, bir Web tarayıcısından çalışan Java programlarıdır.*

Web'de gezinirken, genellikle gördüğünüz grafik kullanıcı arayüzü ve animasyonlar Java kullanılarak geliştirilmiştir. Web tarayıcısından çalışan Java programlarına *Java appletleri* denir. Grafikler, resimler ve ses ile Java appletleri nasıl yazılır? Bu bölüm size bunu gösterecektir.

## 18.2 Applet Geliştirme



*Java appletleri, **Applet** sınıfının örnekleridir. **JApplet**, **Applet** sınıfının bir alt sınıfıdır ve **Swing** bileşenlerini kullanarak applet geliştirmek için uygundur.*



VideoNot  
ilk applet

Şimdiye kadar Java uygulamaları kullandınız ve yazdınız. Ancak, uygulama yazma ile ilgili öğrendiğiniz her şey, applet yazma için de geçerlidir. Uygulamalar ve appletler birçok ortak programlama özelliğini paylaşır, ancak bazı yönlerde biraz farklılık gösterirler. Örneğin, her uygulamanın Java yorumlayıcısı tarafından çağrılan bir **main** metoduna sahip olması gerekir. Öte yandan, Java appletlerinin bir **main** metoduna ihtiyacı yoktur. Web tarayıcısı ortamında çalışırlar. Appletler bir Web sayfasına gömülü olduğundan, Java, appletlerin bir Web tarayıcısından çalışmasını sağlayan özel özellikler sunar.

Applet sınıfı, applet'lerin bir Web tarayıcısından çalıştırılmasını sağlayan temel çerçeveyi sağlar. Her Java uygulaması, uygulama başladığında yürütülen bir **main** metoduna sahiptirken, applet'ler **main** metoduna sahip olmadıkları için tarayıcıya bağımlıdır. Her applet, **java.applet.Applet**'in bir örneğidir. Applet sınıfı bir AWT sınıfıdır ve **Swing** bileşeniyle çalışmak üzere tasarlanmamıştır. Java applet'lerinde **Swing** bileşenlerini kullanmak için, **java.applet.Applet**'in bir alt sınıfı olan **javax.swing.JApplet**'i genişleten bir Java applet'i tanımlamanız gerekir.

Geliştirdiğiniz her Java GUI programı, **JFrame**'i **JApplet** ile değiştirerek ve **main** metodunu silerek bir applet'e dönüştürülebilir.

Şekil 18.1a, bir Java GUI uygulama programını göstermektedir; bu program, Şekil 18.1b'de gösterildiği gibi bir Java applet'ine dönüştürülebilir.

```
import javax.swing.*;

genel class DisplayLabelText extends JFrame {
 public DisplayLabelText() {
 add(new JLabel("Harika!", JLabel.CENTER));
 }

 public static void main(String[] args) {
 JFrame çerçeve = new JFrame("DisplayLabelText");
 çerçeve.setTitle("DisplayLabelText");
 çerçeve.setSize(200, 100);
 çerçeve.setLocationRelativeTo(null);
 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 çerçeve.setVisible(true);
 }
}
```

(a) GUI uygulaması

```
import javax.swing.*;

genel class DisplayLabelText extends JApplet {
 public DisplayLabelText() {
 add(new JLabel("Harika!", JLabel.CENTER));
 }

 genel static void main(String[] args) {
 JFrame çerçeve = new JFrame("DisplayLabelText");
 çerçeve.setTitle("DisplayLabelText");
 çerçeve.setSize(200, 100);
 çerçeve.setLocationRelativeTo(null);
 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 çerçeve.setVisible(true);
 }
}
```

(b) Uygulama

**FIGURE 18.1** Bir GUI uygulamasını bir applete dönüştürebilirsiniz.

Liste 18.1, applet için tam kodu vermektedir.

### LISTE 18.1 DisplayLabel.java

```
1 import javax.swing.*;
2
```

JApplet'i genişlet

```
public class DisplayLabel {
```

```
 extends JApplet {
```



```

4 publicDisplayLabel() {
5 add(new JLabel("Harika!", JLabel.CENTER));
6 }
7 }

```

JFrame gibi, JApplet diğer grafiksel kullanıcı arayüzü bileşenlerini içerebilen bir konteynerdir (bkz. Şekil 12.1'deki GUI sınıfı diyagramları). JApplet için varsayılan düzen yöneticisi BorderLayout'dır. Bu nedenle, etiket applet'in ortasına yerleştirilmiştir (satır 5).

**18.1** Her applet, java.applet.Applet'in bir örneği midir?

**18.2** javax.swing.JApplet, java.applet.Applet'in bir alt sınıfı mıdır?

Point/Kontrol

MyProgrammingLab™

## 18.3 HTML Dosyası ve <applet> Etiketi

Bir applet çalıştırmak için, applet'i gömmek için bir <applet> etiketi içeren bir HTML dosyası oluşturmanız gerekir.



HTML, Web'de statik belgeleri sunan bir biçimlendirme dilidir. etiketleri, Web tarayıcısına bir Web sayfasını nasıl görüntüleyeceğini talimat vermek için kullanılır ve bir Web sayfasına applet'leri dahil eden <applet> adlı bir etiketi içerir.

HTML

Liste 18.2'deki HTML dosyası, applet DisplayLabel.class'ı gömülü olarak içerir.

### Liste 18.2 DisplayLabel.html

```

<html>
<kafa>
<title>Java Uygulaması Demosu</title>ve
</head>
<gövde>
<uygulama
 codet="DisplayLabel.class"
 genişlik=250 y
 yükseklik=150n >
</applet>ve
</body>
</html>

```

applet sınıfı

HTML etiketi, web tarayıcısına bir talimattır. Tarayıcı, etiketi yorumlar ve

etiket

daha sonra HTML belgesinin sonraki içeriklerini nasıl görüntüleyeceğine veya başka şekilde nasıl muamele edeceğine karar verir.

Etiketler köşeli parantezler içinde yer alır (< >). Bir etiketteki ilk kelime, etiket adı olarak adlandırılır ve etiket işlevlerini tanımlar. Bir etiketin, bazen eşittir işaretinden sonra değerlerle birlikte ek nitelikleri olabilir, bu da etiketin eylemini daha da tanımlar. Örneğin, önceki HTML dosyasında, <applet> etiket adı ve code, width ve height nitelikleridir. width ve height nitelikleri applet'in dikdörtgen görüntüleme alanını belirtir.

<applet> etiketi

Çoğu etiketin bir başlangıç etiketi ve karşılık gelen bir bitiş etiketi vardır. Etiket, başlangıç etiketi ile bitiş etiketi arasındaki bölge üzerinde belirli bir etkiye sahiptir. Örneğin, <applet...>...</applet> tarayıcıya bir applet görüntülemesini söyler. Bir bitiş etiketi her zaman başlangıç etiketinin adının önüne bir eğik çizgi eklenerek yazılır.

Bir HTML belgesi, belgenin HTML olarak yazıldığını beyan eden <html> etiketiyle başlar. Her belgenin iki bölümü vardır, bir başlık ve bir gövde, sırasıyla <head> ve <body> etiketleriyle tanımlanır. Başlık kısmı, belgenin başlığını içeren <title> etiketi ve tarayıcının belgeyi işlerken kullanabileceği diğer bilgileri içerir, gövde kısmı ise belgenin gerçek içeriğini barındırır. Başlık kısmı isteğe bağlıdır. Daha fazla bilgi için, Ek V.A, HTML ve XHTML Eğitimi'ne bakın.

Applet etiketinin tam sözdizimi aşağıdaki gibidir:

```

<uygulama
 [codebase=applet_url]

```

```

kod=classfilename.class
genişlik=applet_görüntüleme_genişliği_piksel_cinsinden
yükseklik=applet_görüntüleme_yüksekliği_piksel_cinsinden
[arşiv=archivefile]
[vspace =dikey_marginal]
[hspace = horizontal_margin]
[align = applet_alignment]
[alt =alternatif_metni]
>
<parametre adı=param_name1değer= param_value1>
<parametre adı=param_name2değer= param_value2>
...
<parametre adı=param_nameideğer= param_valuei>
</applet>

```

<param> etiketi

Gerekli olan **kod**, **genişlik** ve **yükseklik** nitelikleridir; diğerleri isteğe bağlıdır. <param> etiketi Bölüm 18.7'de, Applet'lere Dize Geçiş'i'nde tanıtılacaktır. Diğer nitelikler aşağıda açıklanmıştır.

codebase niteliği

■ **codebase**, sınıflarınızın yükleneceği temeli belirtir. Bu nitelik kullanılmazsa, Web tarayıcısı applet'i HTML sayfasının bulunduğu dizinden yükler. Eğer applet'iniz HTML sayfasından farklı bir dizinde bulunuyorsa, tarayıcının applet'i yüklemesi için **applet\_url** belirtmelisiniz. Bu nitelik, sınıfı İnternet'in herhangi bir yerinden yüklemenizi sağlar. Applet tarafından kullanılan sınıflar ihtiyaç duyulduğunda dinamik olarak yüklenir.

arşiv niteliği

■ **archive**, tarayıcıya applet'i çalıştırmak için gereken tüm sınıf dosyalarını içeren bir arşiv dosyasını yüklemesini söyler. Arşivleme, Web tarayıcısının tüm sınıfları tek bir sıkıştırılmış dosyadan bir kerede yüklemesine olanak tanır, böylece yükleme süresini azaltır ve performansını artırır. Arşiv oluşturmak için, Ek III.Q, Java Projelerini Paketleme ve Dağıtma'ya bakın.

■ **vspace** ve **hspace**, applet'in etrafındaki boş kenar boşluğunun dikey ve yatay olarak piksel cinsinden boyutunu belirtir.

■ **align**, applet'in tarayıcıda nasıl hizalanacağını belirtir. Dokuz değerden biri kullanılır: **left**, **right**, **top**, **texttop**, **middle**, **absmiddle**, **baseline**, **bottom**, veya **absbottom**.

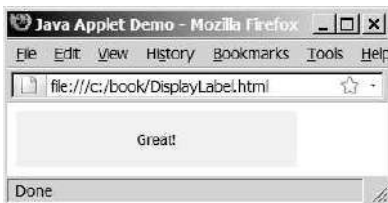
■ **alt**, tarayıcı Java'yı çalıştıramazsa görüntülenecek metni belirtir.

### 18.3.1 Web Tarayıcısından Applet Görüntüleme

Bir web tarayıcısından bir applet görüntülemek için, applet'in HTML dosyasını açın (örneğin, **DisplayLabel.html**). Çıktısı Şekil 18.2'a da gösterilmiştir.

Applet'inizi Web'de erişilebilir hale getirmek için, **DisplayLabel.class** ve

**DisplayLabel.html** dosyalarını bir Web sunucusunda saklamanız gerekir; bu, Şekil 18.3'te gösterilmiştir. Applet'i görüntüleyebilirsiniz



(a)



(b)

**FIGİN 18.2** DisplayLabel programı (a) yerel bir sunucudan ve (b) bir Web sunucusundan yüklenir.

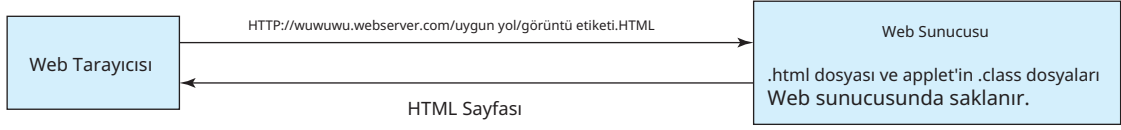


FIGURE 18.3 Bir web tarayıcısı bir Web sunucusundan bir HTML dosyası talep ediyor.

uygun bir URL'den. Örneğin, bu iki dosyayı Web sunucusuna yükledim [www.cs.armstrong.edu/](http://www.cs.armstrong.edu/). Şekil 18.2b'de gösterildiği gibi, applete erişebilirsiniz [www.cs.armstrong.edu/liang/intro9e/book/DisplayLabel.html](http://www.cs.armstrong.edu/liang/intro9e/book/DisplayLabel.html).

### 18.3.2 Applet Görüntüleme Applet Viewer Aracı Kullanarak

Applet'i, Şekil 18.4a'da gösterildiği gibi, DOS

istem isteminden **appletviewer** komutunu kullanarak başlatabileceğiniz applet görüntüleyici aracıyla test edebilirsiniz. Çıktısı

uygulama görüntüleyici

Şekil 18.4b'de gösterilmektedir.

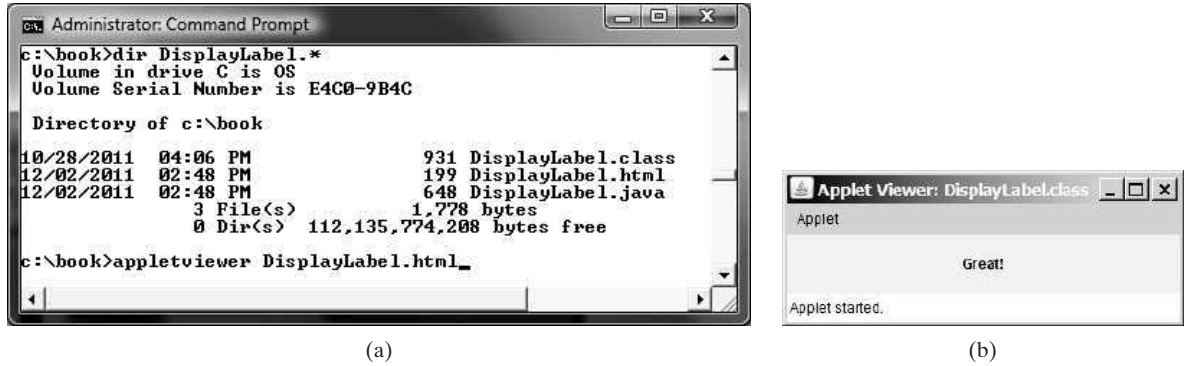


FIGURE 18.4 Appletviewer komutu, bir Java applet'ini applet viewer aracında çalıştırır.

Applet görüntüleyici bir tarayıcı gibi çalışır. Web tarayıcısını başlatmadan geliştirme sırasında applet'leri test etmek için kullanışlıdır.

**18.3** <applet> HTML etiketini tanımlayın. Bir appleti bir web sayfasına nasıl gömüyorsunuz?

**18.4** Applet'i **appletviewer** komutunu kullanarak nasıl test edersiniz?

Point/Kontrol

MyProgrammingLab™

## 18.4 Applet Güvenlik Kısıtlamaları

*Applet güvenlik kısıtlamaları, applet'ler çalıştırılırken güvenliğin sağlandığından emin olur.*

Java, tarayıcının çalıştığı sistemin zarar görmesini önlemek için applet'leri çalıştırmak üzere "sandbox güvenlik modeli" olarak adlandırılan bir modeli kullanır. Applet'lerin "sandbox" dışındaki kaynakları kullanmalarına izin verilmez. Özellikle, sandbox aşağıdaki faaliyetleri kısıtlar:

- Applet'lerin bilgisayarın dosya sisteminden okumasına veya yazmasına izin verilmez. Aksi takdirde, dosyalara zarar verebilir ve virüs yayabilirler.
- Applet'lerin tarayıcının bilgisayarında program çalıştırmasına izin verilmez. Aksi takdirde, yıkıcı yerel programları çağırabilir ve kullanıcının bilgisayarındaki yerel sisteme zarar verebilirler.
- Applet'lerin kullanıcının bilgisayarını ile başka bir bilgisayar arasında bağlantı kurmasına izin verilmez, applet'lerin depolandığı sunucu hariç. Bu kısıtlama, applet'in kullanıcının bilgisayarını kullanıcının bilgisi olmadan başka bir bilgisayara bağlamasını engeller.



imzalı applet

**Not**

Güvenlik kısıtlamalarını aşmak için *imzalı applet*'ler oluşturabilirsiniz. İmzalı applet'ler oluşturma hakkında ayrıntılı talimatlar için Ek III.S, İmzalı Applet'ler'e bakın.

**18.5**

Applet'ler üzerindeki bazı güvenlik kısıtlamalarını listeleyin.

Point/Kontrol

MyProgrammingLab™

Key  
Point

VideoNotu

Applet'leri bağımsız çalıştır

## 18.5 Applet'lerin Uygulama Olarak Çalıştırılmasını Sağlama

### Applet'in bağımsız bir

uygulama olarak çalışmasını sağlamak için applet'e bir ana yöntem ekleyebilirsiniz.

Bazı farklılıklara rağmen, **JFrame** sınıfı ve **JApplet** sınıfı birçok ortak özelliğe sahiptir. Her ikisi de **Container** sınıfının alt sınıfları olduğundan, tüm kullanıcı arayüzü bileşenleri, yerleşim yöneticileri ve olay işleme özellikleri aynıdır. Ancak, uygulamalar Java yorumlayıcısı tarafından statik **main** metodundan çağırılırken, appletler Web tarayıcısı tarafından çalıştırılır. Web tarayıcısı, appletin argüman almayan yapıcısını kullanarak bir applet örneği oluşturur ve appleti kontrol eder ve çalıştırır.

Genel olarak, bir applet işlevselliğini kaybetmeden bir uygulamaya dönüştürülebilir. Bir uygulama, appletler üzerinde uygulanan güvenlik kısıtlamalarını ihlal etmediği sürece bir applete dönüştürülebilir. Appletin bir uygulama olarak çalışabilmesi için applet içinde bir **main** metodu uygulayabilirsiniz. Bu özelliğin hem teorik hem de pratik sonuçları vardır. Teorik olarak, appletler ile uygulamalar arasındaki farkı bulanıklaştırır: Hem bir applet hem de bir uygulama olan bir sınıf yazabilirsiniz. Pratik açıdan, bir programı her iki şekilde çalıştırabilmek kullanışlıdır.

Böyle programlar nasıl yazılır? **MyApplet** adında bir appletiniz olduğunu varsayalım. Uygulama olarak çalışabilmesi için, appletin içine yalnızca aşağıdaki gibi bir **main** metodu eklemeniz gerekir:

```
public static void main(String[] args) {
```

```
// Bir çerçeve oluştur
```

```
JFrame = çerçeve
```

```
new JFrame("A
```

```
// Applet'in bir örneğini oluştur
```

```
MyApplet = MyApplet = MyApplet küçük uygulama
```

```
new MyApplet();
```

```
// Applet'i çerçeveye ekle
```

```
çerçeve.add(applet, BorderLayout.CENTER);
```

```
// Çerçeveyi göster
```

```
çerçeve.setSize(300,300);
```

```
frame.setLocationRelativeTo(null); // Çerçeveyi ortala
```

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
çerçeve.setVisible(true);
```

```
}
```

**DisplayLabel** sınıfını, Listing 18.1'de gösterildiği gibi, bir **main** metodu ekleyerek bağımsız bir uygulama olarak çalışacak şekilde revize edebilirsiniz (genellikle "bağımsız çalıştır" olarak kısaltılır).

### LISTING 18.3 Yeni DisplayLabel.java ile bir main Metodu

```
1 import javax.swing.*;
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```

8 public static void main(String[] args) {
9 // Bir çerçeve oluştur
10 JFrame frame = new JFrame("Applet çerçevesinde");
11
12 // Applet'in bir örneğini oluştur
13 DisplayLabel applet = new DisplayLabel();
14
15 // Applet'i çerçeveye ekle
16 frame.add(applet);
17
18 // Çerçeveyi göster
19 frame.setSize(300, 100);
20 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
21 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22 frame.setVisible(true);
23 }

```

yeni ana yöntem

Applet bir web tarayıcısından çalıştırıldığında, tarayıcı applet'in bir örneğini oluşturur ve gösterir. Applet bağımsız olarak çalıştırıldığında, ana yöntem bir çerçeve oluşturmak için çağrılır (10. satır) ve applet bu çerçeveye eklenir (13. satır) ve çerçeveye eklenir (16. satır). Çerçeve 22. satırda görüntülenir.

Bir applet'i bir konteynıra ekleyebileceğinizi, ancak bir çerçeveyi bir konteynıra ekleyemeyeceğinizi unutmayın. Bir çerçeve, bir başka konteynıra gömülemeyen üst düzey bir konteynırdır.

**18.6** Bir **JApplet**'e bileşenleri nasıl eklersiniz? JApplet'in varsayılan düzen yöneticisi nedir?

**18.7** Bir çerçeveyi bir applet'in içine yerleştirebilir misiniz?

**18.8** Bir applet'i bir çerçeveye yerleştirebilir misiniz?

**18.9** Uygulamalar ile applet'ler arasındaki farklar nelerdir? Bir uygulamayı nasıl çalıştırırsınız ve bir applet'i nasıl çalıştırırsınız? Uygulamalar ve applet'ler için derleme süreci farklı mı?

Point/Kontrol

MyProgrammingLab™

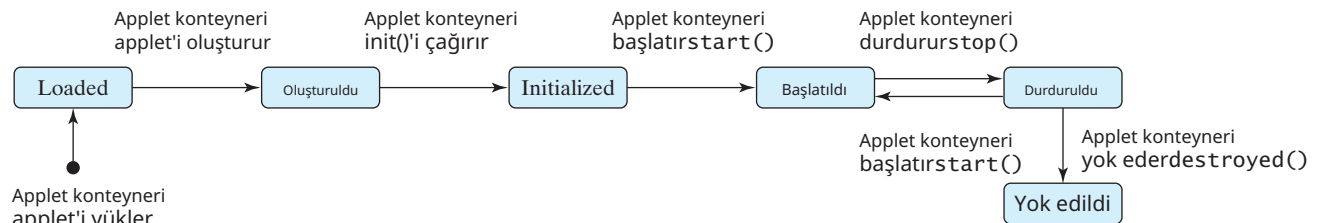
## 18.6 Applet Yaşam Döngüsü Yöntemleri

*Web tarayıcısı, applet yaşam döngüsü yöntemlerini kullanarak applet'leri kontrol eder ve çalıştırır.*

Applet'ler aslında bir Web tarayıcısının eklentisi olan *applet konteyneri*'nden çalıştırılır. Bireklenti, ek işlevler sağlamak için daha büyük bir yazılıma eklenebilen bir yazılım bileşenidir. **Applet** sınıfı, **init()**, **start()**, **stop()** ve **destroy()** yöntemlerini içerir; bunlar *yaşam döngüsü yöntemleri* olarak bilinir. Bu yöntemler, bir applet'in çalıştırılmasını kontrol etmek için applet konteyneri tarafından çağrılır. Varsayılan olarak hiçbir şey yapmadıkları için Applet sınıfında boş bir gövde ile uygulanmışlardır. İstenilen işlemleri gerçekleştirmek için **Applet**'in bir alt sınıfında bu yöntemleri geçersiz kılabilirsiniz. Şekil 18.5, applet konteynerinin bu yöntemleri nasıl çağırdığını göstermektedir.



applet konteyneri



**FIGÜRİ 18.5** Applet konteyneri, applet'i kontrol etmek için **init**, **start**, **stop** ve **destroy** yöntemlerini kullanır.

### 18.6.1 **init** Metodu

sıcaklık()

**init** metodu, applet oluşturulduktan sonra çağrılır. Eğer **Applet** sınıfının bir alt sınıfının başlatılması gereken bir işlemi varsa, bu metodu geçersiz kılmalıdır. Bu metotta genellikle HTML sayfasındaki **<applet>** etiketinden string parametre değerlerini alma işlemleri uygulanır.

### 18.6.2 **start** Metodu

başlangıç()

**start** metodu, **init** metodundan sonra çağrılır. Ayrıca, kullanıcı diğer sayfalarda gezinip applet içeren web sayfasına döndüğünde de çağrılır.

**Applet** sınıfının bir alt sınıfı, applet içeren web sayfası ziyaret edildiğinde herhangi bir işlem yapması gerekiyorsa bu metodu geçersiz kılar. Örneğin, animasyona sahip bir applet, animasyonu yeniden başlatmak için zamanlayıcıyı başlatabilir.

### 18.6.3 **stop** Metodu

durmak()

**stop** metodu, **start** metodunun tersidir. Kullanıcı, applet içeren sayfaya geri döndüğünde **start** metodu çağrılır. Kullanıcı sayfayı terk ettiğinde ise **stop** metodu çağrılır.

Bir **Applet** alt sınıfı, applet'in bulunduğu web sayfası artık görünür olmadığında her seferinde gerçekleştirilmesi gereken bir işlem varsa bu yöntemi geçersiz kılar. Örneğin, animasyona sahip bir applet, animasyonu duraklatmak için zamanlayıcıyı durdurabilir.

### 18.6.4 **destroy** Metodu

tahrip etmek()

**destroy** metodu, tarayıcı normal bir şekilde çıkarken çağrılır ve applete artık gerekli olmadığını ve tahsis ettiği kaynakları serbest bırakması gerektiğini bildirir. **stop** metodu her zaman **destroy** metodundan önce çağrılır.

Applet sınıfının bir alt sınıfı, yok edilmeden önce gerçekleştirilmesi gereken herhangi bir işlem varsa bu metodu geçersiz kılar. Genellikle, applet'in oluşturduğu belirli kaynakları serbest bırakmak istemediğiniz sürece bu metodu geçersiz kılmanıza gerek yoktur.

## 18.10 Applet

Point/Kontrol

sınıfındaki **init()**, **start()**, **stop()** ve **destroy()** metodlarını tanımlayın.

MyProgrammingLab™

**18.11** Aşağıdaki (a) applet neden hiçbir şey göstermiyor? (b) applet neden vurgulanan satırda bir çalışma zamanı **NullPointerException** alıyor?

```
import javax.swing.*;

public class WelcomeApplet extends JApplet {
 public WelcomeApplet() {
 JLabel lblMessage =
 new JLabel("Java'dır");
 }
}
```

(a)

```
import javax.swing.*;

genel sınıf WelcomeApplet extends JApplet {
 private JLabel lblMessage;

 public WelcomeApplet() {
 JLabel lblMessage =
 new JLabel("Java'dır");
 }

 @Geçersiz kıl
 genel void init() {
 add(lblMessage);
 }
}
```

(b)

## 18.7 Dize Geçişleri için Appletler

Bir HTML dosyasından bir applete dize parametreleri geçirebilirsiniz.



9.7. Bölüm, Komut Satırı Argümanları'nda, komut satırından Java uygulamalarına nasıl dize geçireceğinizi öğrendiniz. Dizeler, **main** yöntemine bir dize dizisi olarak geçer. Uygulama başladığında, **main** yöntemi bu dizeleri kullanabilir. Ancak, bir applet'te **main** yöntemi yoktur ve applet'ler Java yorumlayıcısı tarafından komut satırından çalıştırılmaz.

Peki, o zaman applet'ler argümanları nasıl kabul edebilir? Bu bölümde, Java applet'lerine dizeleri nasıl geçireceğinizi öğreneceksiniz. Bir applet'e geçmek için, bir parametre HTML dosyasında tanımlanmalı ve applet başlatıldığında okunmalıdır. Parametreler,

`<param>` etiketi kullanılarak tanımlanır. `<param>` etiketi, `<applet>` etiketinin içine yerleştirilmelidir. Söz dizimi şöyledir:

```
<tparam name= parametrename tvalue= stringvalue />
```

`<param>` etiketi bir parametreyi ve onun karşılık gelen dize değerini tanımlar.



### Not

HTML kodunda parametre adı ile parametre değeri arasında virgül yoktur.

HTML parametre adları büyük/küçük harf duyarlı değildir.

Diyelim ki bir mesaj görüntülemek için bir applet yazmak istiyorsunuz. Mesaj bir parametre olarak geçer. Ayrıca, mesajın belirli bir konumda *x*-koordinatı ve *y*-koordinatı ile görüntülenmesini istiyorsunuz; bu koordinatlar iki parametre olarak geçer. Parametreler ve değerleri Tablo 18.1'de listelenmiştir.

**TABLE 18.1** Görüntü Mesaj Applet için Parametre İsimleri ve Değerleri

Parametre Adı	Parametre Değeri
MESAJ	"Java'ya Hoş Geldiniz"
X	20
Y	30

HTML kaynak dosyası Liste 18.4'te verilmiştir.

### LISTING 18.4 Görüntü Mesaj.html

```
<html>
<kafa>
 <title>Java Applet'lerine Dize Geçirme</title>
</head>
<gövde>
 <p>Bu applet, HTML
 sayfasından bir mesaj alır ve görüntüler.</p>
 <applet
 kod = "DisplayMessage.class"
 genişlik = 200
 yükseklik = 50
 alt = "Applet'i görüntülemek için Java destekli bir tarayıcıya sahip olmalısınız"
 >
 <param name= MESSAGE value="Java'ya Hoş Geldiniz"/>
 <param name= X value=20/>
```

```

 <param adı= Y tdeğeri=t30t/>
 </applet>
</body> için
de</html>

```

Applet'ten parametre okumak için, **Applet** sınıfında tanımlanan aşağıdaki yöntemi kullanın:

```
public String getParameter(String parametreadı);
```

Bu, belirtilen parametrenin değerini döndürür.

Applet, Liste 18.5'te verilmiştir. Applet'in örnek çalışması Şekil 18.6'da gösterilmiştir.

## LISTELER 18.5 DisplayMessage.java

```

1 import javax.swing.*;
2
3 kamu sınıfı DisplayMessage extends JApplet {
4 @Override /** Applet'i başlat */
5 public void init() {
6 // HTML dosyasından parametre değerlerini al
7 String message = getParameter("MESSAGE");
8 int x = Integer.parseInt(getParameter("X"));
9 int y = Integer.parseInt(getParameter("Y"));
10
11 // Bir mesaj paneli oluştur
12 MessagePanel messagePanel = new MessagePanel(message);
13 messagePanel.setXCoordinate(x);
14 messagePanel.setYCoordinate(y);
15
16 // Mesaj panelini applet'e ekle
17 add(messagePanel);
18 }
19 }

```

applet'e ekle



## FİÇİN 18.6 Applet, HTML

sayfasından geçirilen Java'ya Hoş Geldiniz mesajını görüntüler.

Program, **init** yönteminde HTML dosyasından parametre değerlerini alır. Değerler, **getParameter** yöntemi kullanılarak elde edilen dizelerdir (satırlar 7-9). Çünkü **x** veya **ints** olduğundan, program bir dijital diziye **int** değerine dönüştürmek için **Integer.parseInt(string)** kullanır.

HTML dosyasında **Welcome to Java** ifadesini **Welcome to HTML** olarak değiştirirseniz ve HTML dosyasını Web tarayıcısında yeniden yüklerken, **Welcome to HTML** ifadesinin görüntülendiğini görmelisiniz. Benzer şekilde, **x** veya **y** değerleri, mesajı istenen bir konumda görüntülemek için değiştirilebilir.



### Dikkat

Applet'in **getParameter** metodu yalnızca bir applet örneği oluşturulduktan sonra çağrılabilir. Bu nedenle, bu metod applet sınıfının yapısında çağrılmaz. Onu **init** metodundan çağırmalısınız.



Bu applet'in bağımsız bir uygulama olarak çalışabilmesi için bir ana yöntem ekleyebilirsiniz. Applet, çalıştığında HTML dosyasından parametreleri alır ve bağımsız çalıştığında a komut satırından parametreleri alır. Program, Listeleme 18.6'da gösterildiği gibi, yeni bir `ana` yöntemi ve bir `isStandalone` adlı değişken eklenmesi dışında `DisplayMessage` ile aynıdır; bu değişken, applet olarak mı yoksa uygulama olarak mı çalıştığını belirtir.

### LISTE 18.6 DisplayMessageApp.java

```

1 import javax.swing.*;
2 import java.awt.Font;
3 import java.awt.BorderLayout;
4
5 kamu sınıfı DisplayMessageApp extends JApplet {
6 private String message = "Varsayılan bir mesaj"; // Görüntülenecek mesaj
7 özel int x = 20; // Varsayılan x-koordinatı
8 özel int y = 20; // Varsayılan y-koordinatı
9
10 /** Uygulama olup olmadığını belirleyin */
11 özel boolean isStandalone = false; // bağımsızdır
12
13 @Override /** Applet'i başlat */
14 genel void init() {
15 if (!isBağımsız) {
16 // HTML dosyasından parametre değerlerini al
17 mesaj = getParameter("MESSAGE"); n18 x = Integer.parseInt(getParameter("X")); n19 y = Integer.parseInt(getParameter("Y")); n20 } // applet parametreleri
18
19
20
21
22 // Bir mesaj paneli oluştur
23 MesajPanel messagePanel = newtMessagePanel(message); n24 messagePanel.setFont(newtFont("SansSerif", Font.BOLD, 20)); n25 messagePanel.setXCoord(x);
26 messagePanel.setYCoord(y);
27
28 // Mesaj panelini applete ekle
29 add(messagePanel);
30 }
31
32 /** Bir mesajı görüntülemek için ana yöntem
33 @param args[0] x-koordinatı
34 @param args[1] y-koordinatı
35 @param args[2] mesaj
36 */
37 public static voidtmain(String[] args) {
38 // Bir çerçeve oluştur
39 JÇerçeve çerçeve = newtJFrame("DisplayMessageApp");
40
41 // Applet'in bir örneğini oluştur
42 DisplayMessageApp küçük uygulama = newtDisplayMessageApp();
43
44 // Uygulama olarak çalışır
45 applet.isStandalone = true; // bağımsız
46
47 // Komut satırından parametreleri al
48 applet.getCommandLineParameters(args); // komut parametreleri
49
50 // applet örneğini çerçeveye ekle
51 çerçeve.add(applet, BorderLayout.CENTER);

```

```

52
53 // Applet'in init metodunu çağır
54 et.init();
55 applet.start();nappl
56
57 // Çerçeveyi göster
58 frame.setSize(300, 300);
59 frame.setLocationRelativeTo(null); // Çerçeveyi ortala
60 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
61 çerçeve.setVisible(true);
62 }
63
64 /** Komut satırı parametrelerini al */
65 özel void getCommandLineParameters(String[] args) {
66 // Kullanımı kontrol et ve x, y ve mesajı al
67 if (args.length != 3) {
68 System.out.println(
69 "Kullanım: java DisplayMessageApp x y mesaj");
70 System.exit(1);
71 }
72
73 // aksi takdirde {
74 x = Integer.parseInt(args[0]); y = Integer.p
75 arseInt(args[1]); message = args[2];
76 }
77 }
78 }

```

Programı bir applet olarak çalıştırdığınızda, **main** metodu göz ardı edilir. Uygulama olarak çalıştırdığınızda, **main** metodu çağrılır. Programın bir uygulama ve bir applet olarak örnek çalıştırmaları Şekil 18.7'de gösterilmiştir.



**FIGURE 18.7** **DisplayMessageApp** sınıfı (a) olarak bir applet ve (b) olarak bir uygulama olarak çalışabilir.

**main** metodu bir **JFrame** nesnesi **frame** oluşturur ve bir **JApplet** nesnesi applet oluşturur, sonra applet applet nesnesini **frame** **frame** içine yerleştirir ve **init** metodunu çağırır. Uygulama, bir applet gibi çalışır.

Ana yöntem, **isStandalone** değerini **true** olarak ayarlıyor (satır 45), böylece **init** yöntemi çağırıldığında HTML parametrelerini almaya çalışmıyor.

**setVisible(true)** yöntemi (satır 61), bileşenler applete eklendikten sonra çağırılır ve bileşenlerin görünür olmasını sağlamak için applet çerçeveye eklenir. Aksi takdirde, çerçeve başladığında bileşenler gösterilmez.



### Önemli Pedagojik Not

Artık tüm GUI örnekleri, bir **main** yöntemi ile applet olarak oluşturulacaktır. Böylece, programı ya bir applet ya da bir uygulama olarak çalıştırabileceksiniz. Kısalık açısından, **main** yöntemi metinde listelenmemiştir.

**main** metodunu atlama

Point/Kontrol

**18.12** Bir applete parametreleri nasıl geçirirsiniz?

**18.13** **getParameter** yöntemi nerede tanımlanmıştır?

## 18.14 DisplayMessage appleti aşağıdaki gibi revize edilirse ne yanlış olur?

```

public class DisplayMessage extends JApplet {
 /** Applet'i başlat */
 public DisplayMessage() {
 // HTML dosyasından parametre değerlerini al
 String message = getParameter("MESSAGE");
 int x =
 Integer.parseInt(getParameter("X"));
 int y =
 Integer.parseInt(getParameter("Y"));

 // Bir mesaj paneli oluştur
 MessagePanel messagePanel =
 new MessagePanel(message);
 messagePanel.setXCoordinate(x);
 messagePanel.setYCoordinate(y);

 // Mesaj panelini applete ekle
 add(messagePanel);
 }
}

```

(a) Revizyon 1

```

public class DisplayMessage extends JApplet {
 private String mesaj;
 private int x;
 private int y;

 @Override /** Applet'i başlat */
 public void init() {
 // HTML dosyasından parametre değerlerini al
 mesaj = getParameter("MESSAGE");
 x = Integer.parseInt(getParameter("X"));
 y = Integer.parseInt(getParameter("Y"));
 }

 public DisplayMessage() {
 // Bir mesaj paneli oluştur
 MessagePanel messagePanel =
 new MessagePanel(mesaj);
 messagePanel.setXCoordinate(x);
 messagePanel.setYCoordinate(y);

 // Mesaj panelini applete ekle
 add(messagePanel);
 }
}

```

(b) Revizyon 2

## 18.8 Vaka Çalışması: Zıplayan Top

Bu bölüm, bir panelde zıplayan bir topu gösteren bir applet sunmaktadır.

Applet, zıplama hareketini durdurmak ve yeniden başlatmak için iki buton kullanır ve zıplama hızını kontrol etmek için bir kaydırıcı çubuğu kullanır; bu, Şekil 18.8'de gösterilmiştir.



FIGURE 18.8 Topun hareketi *Suspend* ve *Resume* düğmeleri ile ve

kaydırıcı çubuk ile kontrol edilir.

Bu programı yazmak için ana adımlar şunlardır:

1. Bir topun zıplamasını göstermek için **JPanel** sınıfının bir alt sınıfını **Ball** olarak tanımlayın, Listing 18.7'de gösterildiği gibi.
2. Topun hızını bir kaydırıcı çubuk ile ayarlamak için **JPanel** sınıfının bir alt sınıfını **BallControl** olarak tanımlayın, ve iki kontrol düğmesi *Suspend* ve *Resume*, Listing 18.8'de gösterildiği gibi.
3. **BounceBallApp** adında bir applet tanımlayın ve **BallControl** örneğini içermesini sağlayın ve applet'i n bağımsız bir uygulama olarak çalışmasına izin verin, Şekil 18.9'da gösterildiği gibi.

Bu sınıflar arasındaki ilişki Şekil 18.9'da gösterilmektedir.

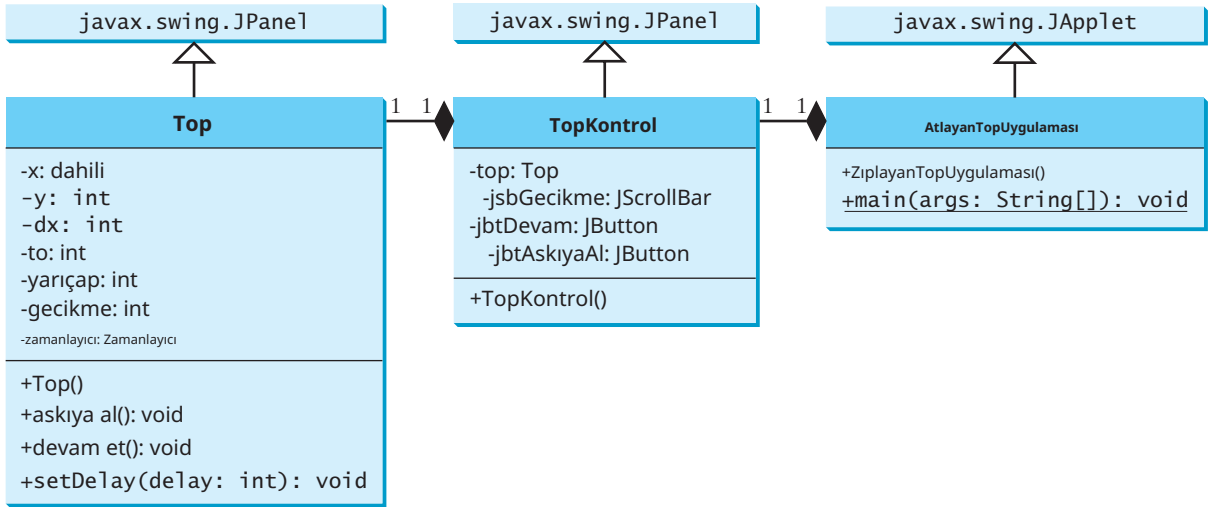


FIGURE 18.9 ZıplayanTopUygulaması TopKontrol içerir ve TopKontrol Top içerir.

### LISTING 18.7tBall.java

```

1 import javax.swing.Timer;
2 import java.awt.*;
3 import javax.swing.*;
4 import java.awt.event.*;
5
6 genel classtBalltextendstJPanel {
7
8 özel int delay = 10;
9
10 // Belirtilen milisaniye gecikmesi ile bir zamanlayıcı oluşturun
11 özel Timer timer = new Timer(delay, new TimerListener());
12
13 private int x = 0; private int y = 0; // Mevcut top pozisyonu
14 private int radius = 5; // Topun yarıçapı
15 özel int dx = 2; // Topun x-koordinatındaki artış
16 özel int dy = 2; // Topun y-koordinatındaki artış
17
18 kamusal Ball() {
19 timer.start();
20 }
21
22 özel sınıf TimerListener uyguluyor ActionListener {
23 @Override /** Eylem olayını işleyin */
24 kamusal void actionPerformed(ActionEvent e) {
25 yeniden çiz;
26 }
27 }
28
29 @Geçersiz Kıl
30 korumalı void paintComponent(Graphics g) {
31 super.paintComponent(g);
32 g.setColor(Renk.kırmızı);
33
34 // Sınırları kontrol et
35 eğer (x < 0 || x > getWidth())
36 dx *= -1;
37 }
38 }

```

zamanlayıcı gecikmesi

zamanlayıcı oluştur

zamanlayıcıyı başlat

zamanlayıcı dinleyici

topu yeniden boyama

topu boyama

```

37 eğer(y < 0 || y > getHeight())
38 bu *= t-1t;
39
40 // Top pozisyonunu ayarla
41 x += dx;
42 y += ila;
43 g.fillOval(x - yarıçap, y - yarıçap, yarıçap * t2t, yarıçap * t2t);n44 }

45
46 public voidsuspend() {
47 timer.stop(); // Zamanlayıcıyı duraklat
48 }
49
50 public voidresume() {
51 timer.start(); // Zamanlayıcıyı devam ettir
52 }
53
54 genel voidtsetDelay(tinttdelay) {
55 thist.delay = gecikme;
56 timer.setDelay(gecikme);n57 }

58 }

```

Animasyonu kontrol etmek için **Timer** kullanımı Bölüm 16.11'de, Timer Sınıfı ile Animasyon Kullanımı'nda tanıtılmıştır.

**Timer** Sınıfı, hareket eden bir topu göstermek için **JPanel**'i genişletir. Zamanlayıcı dinleyicisi, **ActionEvent**'i dinlemek için **ActionListener**'i uygular (satır 21). Satır 10, bir **Ball** için bir **Timer** oluşturur.

Zamanlayıcı, bir **Ball** oluşturulduğunda satır 18'de başlatılır. Zamanlayıcı, sabit bir hızda bir **ActionEvent** ateşler. Dinleyici, topun hareketini canlandırmak için satır 24'te topu yeniden boyamak için yanıt verir.

Topun merkezi (x,y) konumundadır, bu konum bir sonraki görüntülemeye (x + dx,y + dy) olarak değişir(satırlar 41–42). Top yatay sınırın dışına çıktığında, dx'nin işareti değiştirilir (pozitiften negatife veya tersine) (satırlar 35–36). Bu, topun yatay hareket yönünü değiştirmesine neden olur. Top dikey sınırın dışına çıktığında, dy'nin işareti değiştirilir(pozitiften negatife veya tersine) (satırlar 37–38). Bu, topun dikey hareket yönünü değiştirmesine neden olur. **suspend** ve **resume** yöntemleri (satırlar 46–52) zamanlayıcıyı durdurmak ve başlatmak için kullanılabilir. **setDelay(int)** yöntemi (satırlar 54–57) yeni bir gecikme ayarlar.

## LİSTELER18.8BallControl.java

```

1 import javax.swing.*;
2 import java.awt.event.*;
3 import java.awt.*;
4
5 genel classtBallControlextendsJPanel {
6
7 özeltBall topu = tnewtBall();
8
9 privateJButton jbtSuspend = newJButton("Beklet"); düğme
10 privateJButton jbtResume = newJButton("Devam Et");
11 privateJScrollbar jsbDelay = newJScrollbar(); kaydırma çubuğu
12
13 publicBallControl() {
14 // Düğmeleri bir panelde grupta
15 JPanel paneli = tnewtJPanel();n14 panel.ad
16 d(jbtSuspend);
17 panel.add(jbtResume);
18
19 // Panelde top ve düğmeleri ekle
20 ball.setBorder(new javax.swing.border.LineBorder(Color.red));
21 jsbDelay.setOrientation(JScrollbar.HORIZONTAL);
22 ball.setDelay(jsbDelay.getMaximum());n21 setLayout(t
23 newtBorderLayout());

```

```

22 add(jsbDelay, BorderLayout.NORTH);n23 add(ball,
BorderLayout.CENTER);n24 add(panel, BorderLay
out.SOUTH);
n
26 // Dinleyicileri kaydet
dinleyici kaydet 27 jbtSuspend.addActionListener(tnewtActionListener() {
28 @Geçersiz Kıl
askıya al 29 kamusal voidactionPerformed(ActionEvent e) {
30 ball.suspend();
31 {naaq});

dinleyici kaydet 33 jbt Resume.actionListener(yeni ActionListener()'a) {
34 @Geçersiz Kıl
devam et 35 kamusal voidactionPerformed(ActionEvent e) {
36 ball.resume();
37 {Isırırız});

dinleyici kaydet 39 jsbDelay.addAdjustmentListener(tnewtAdjustmentListener() {
40 @Geçersiz Kıl
yeni gecikme 41 genel voidtadadjustmentValueChanged(AdjustmentEvent e) {
42 ball.setDelay(jsbDelay.getMaximum() - e.getValue());
43 }n44 });n45
}n46 }

```

BallControl sınıfı, topu bir kaydırıcı ve iki kontrol butonu ile görüntülemek için JPanel'de n türetilmiştir. Suspend butonuna tıklandığında, topun suspend() metodu çağrılarak to pun hareketi askıya alınır (satır 30). Resume butonuna tıklandığında, topun resume() m etodu çağrılarak topun hareketi devam ettirilir (satır 36). Zıplama hızı kaydırıcı kullanılar ak değiştirilebilir.

### LISTELER 18.9 BounceBallApp.java

```

1 import java.awt.*;
2 import javax.swing.*;
3
4 kamu sınıfı BounceBallApp extends JApplet {
5 public BounceBallApp() {
ekleBallControl 6 add(tnewtBallControl());nBou
7 }n8 }

```

ana yöntem atlandı

**BounceBallApp** sınıfı, applet içinde bir **BallControl** örneği yerleştirir. Ana yöntem, appl et içinde sağlanmış (kısalık için listede gösterilmemiştir) böylece onu bağımsız olarak da çalıştırabilirsiniz.

**18.15** Program topu nasıl hareket ettiriyor?

**18.16** Listing 18.7 Ball.java'daki kod topun hareket yönünü nasıl değiştiriyor?

**18.17** Program, *Suspend* butonuna tıklandığında ne yapıyor? Program, *Resume* bu tonuna tıklandığında ne yapıyor?



## 18.9 Vaka Çalışması: Bir XOX Oyunu Geliştirme

*Bu bölüm, xox oynamak için bir applet geliştirir.*

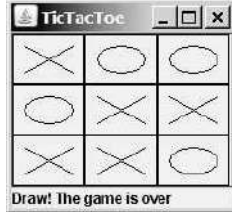
Bu ve önceki bölümlerdeki birçok örnekten nesneler, sınıflar, diziler, sınıf mirası, GUI, olay odaklı programlama ve appletler hakkında bilgi edindiniz. Şimdi bunu uygulama zamanı.

öğrendiklerinizle kapsamlı projeler geliştirmekte çalışmak. Bu bölümde, popüler tic-tac-toe oyununu oynamak için bir Java appleti geliştireceğiz.

İki oyuncu, mevcut bir hücreyi sırayla işaretler  $3 \times 3$  kenditokenleri (ya X ya da O) ile. Bir oyuncu, ızgarada yatay, dikey veya çapraz bir sıraya üç token yerleştirdiğinde, oyun sona erer ve o oyuncu kazanır. Tüm hücreler tokenlerle doldurulduğunda ve hiçbir oyuncu kazanmadığında berabere (kazanan yok) durumu oluşur. Şekil 18.10, örneğin temsilci örnek çalıştırmalarını göstermektedir.



(a) X oyuncusu oyunu kazandı



(b) Beraberlik—kazanan yok



(c) O oyuncusu oyunu kazandı

ŞEKİL 18.10 İki oyuncu bir tic-tac-toe oyunu oynuyor.

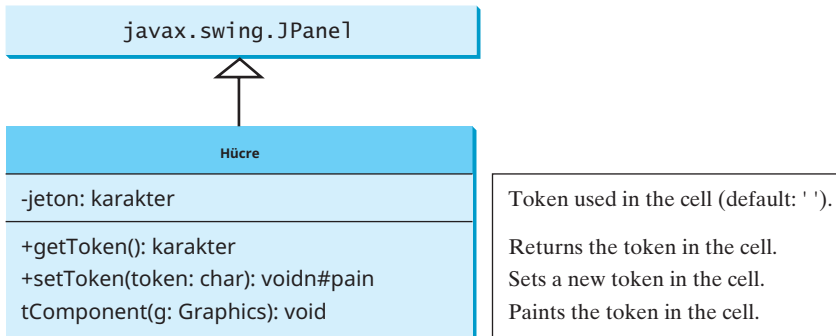
Şu ana kadar gördüğümüz tüm örnekler, sınıflarla modellemesi kolay basit davranışlar göstermektedir. Tic-tac-toe oyununun davranışı ise biraz daha karmaşıktır. Davranışı modelleyen sınıflar oluşturmak için, o oyunu incelemeniz ve anlamanız gerekmektedir.

Tüm hücrelerin başlangıçta boş olduğunu ve ilk oyuncunun X jetonunu, ikinci oyuncunun ise O jetonunu aldığını varsayın. Bir hücreyi işaretlemek için, oyuncu farenin imlecini hücreye getirir ve tıklar. Eğer hücre boşsa, jeton (X veya O) görüntülenir. Eğer hücre zaten doluysa, oyuncunun eylemi göz ardı edilir.

Önceki açıklamadan, bir hücrenin fare tıklama olayını işleyen ve token'ları gösteren bir GUI nesnesi olduğu açıktır. Böyle bir nesne ya bir düğme ya da bir panel olabilir. Panele çizim yapmak, düğmelere çizim yapmaktan daha esnektir, çünkü bir panelde token (X veya O) her boyutta çizilebilir, ancak bir düğmede yalnızca bir metin etiketi olarak gösterilebilir. Bu nedenle, bir hücreyi modellemek için bir panel kullanılmalıdır. Hücrenin durumunu (boş, X veya O) nasıl bilirsiniz? `Cell` sınıfında `char` türünde `token` adında bir özellik kullanırsınız.

`Cell` sınıfı, boş bir hücreye tıklandığında token'ı çizmekten sorumludur, bu nedenle `MouseEvent` dinlemek ve X ve O token'ları için şekilleri boyamak için kod yazmanız gerekir.

`Cell` sınıfı Şekil 18.11'de gösterildiği gibi tanımlanabilir.

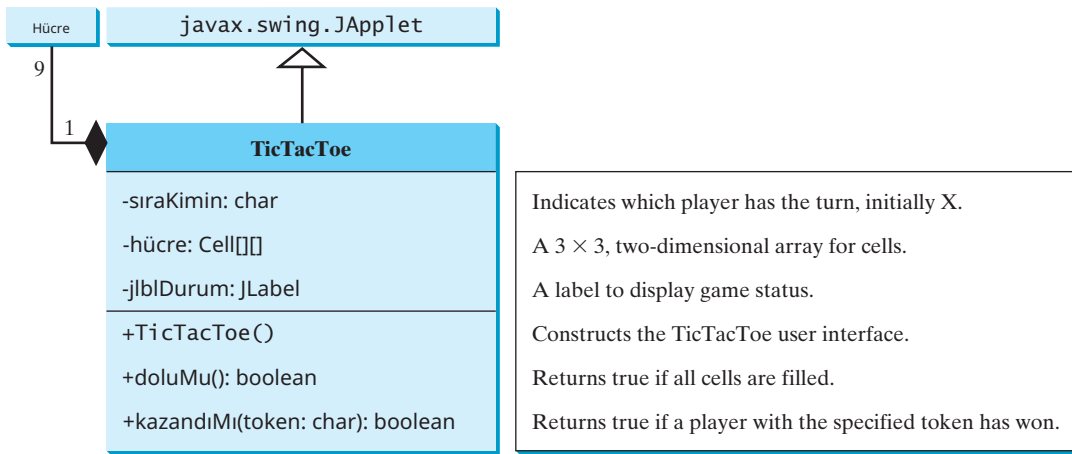


ŞEKİL 18.11 Cell sınıfı bir hücrede token'ı boyar.

Tic-tac-toe tahtası, `new Cell[3][3]` kullanılarak oluşturulan dokuz hücreden oluşur. Hangi oyuncunun sırası olduğunu belirlemek için, `char` türünde `whoseTurn` adında bir değişken tanımlayabilirsiniz. `whoseTurn` başlangıçta 'X' olup, daha sonra 'O'ya değişir ve ardından yeni bir hücre doldurulduğunda 'X' ve 'O' arasında değişir. Oyun bittiğinde, `whoseTurn`'ı ' ' olarak ayarlayın.

Oyunun ne zaman bittiğini, bir kazanan olup olmadığını ve varsa kazananın kim olduğunu nasıl bilirsiniz? Belirtilen bir token'ın kazanıp kazanmadığını kontrol etmek için `isWon(char token)` adında bir yöntem ve tüm hücrelerin dolu olup olmadığını kontrol etmek için `isFull()` adında bir yöntem tanımlayabilirsiniz.

Açıkça, önceki analizden iki sınıf ortaya çıkmaktadır. Biri, tek bir hücre için işlemleri yöneten `Cell` sınıfıdır; diğeri ise tüm oyunu oynayan ve tüm hücrelerle ilgilenen `TicTacToe` sınıfıdır. Bu iki sınıf arasındaki ilişki Şekil 18.12'de gösterilmektedir.



FIGİN 18.12 TicTacToe sınıfı dokuz hücre içerir.

Cell sınıfı yalnızca TicTacToe sınıfını desteklemek için olduğundan, TicTacToe içinde bir iç sınıf olarak tanımlanabilir. Tam program Liste 18.10'da verilmiştir.

### LISTELERİ 18.10 TicTacToe.java

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4 import javax.swing.border.LineBorder;
5
6 ana sınıf TicTacToe
7 kamu sınıfı TicTacToe textendstJApplet {
8 // Hangi oyuncunun sırası olduğunu belirtin; başlangıçta X oyuncusu sırasındadır
9 özel char kiminSırası = 'X';
10
11 // Hücreleri oluştur ve başlat
12 özel hücre[][] hücreler = yeniHücre[3][3];
13
14 // Bir durum etiketi oluştur ve başlat
15 private JLabel jlblStatus = new JLabel("X'in oynama sırası");
16
17 /** UI'yi başlat */
18 public TicTacToe() {
19 // Hücreleri tutacak panel p

```



```

19 JPanel p = new JPanel(new GridLayout(t3t,t3t,t0t,t0t));
20 için(int i = 0; i < 3; i++)
21 güçlü(tinttj = t0t; j < t3t; j++)
22 p.add(hücreler[i][j] = new Cell());
23
24 // Hücrelerin panelinde ve durum etiketinde kenarları ayarla
25 p.setBorder(new LineBorder(Color.red, 1));
26 jlb1Status.setBorder(new LineBorder(Color.yellow, 1));
27
28 // Paneli ve applet için etiketi yerleştir
29 add(p, BorderLayout.CENTER);
30 add(jlb1Status, BorderLayout.SOUTH);
31 }
32
33 /** Hücrelerin hepsinin dolu olup olmadığını belirle */
34 genel booleanisFull() {
35 için(int i = 0; i < 3; i++)
36 güçlü(tinttj = t0t; j < t3t; j++)
37 if(hücreler[i][j].getToken() == 't')
38 false'i döndür;
39
40 geri dönüş tehdidi;
41 }
42
43 /** Belirtilen token ile oyuncunun kazanıp kazanmadığını belirle */
44 herkese açık booleanisWon(tcharttoken) {
45 için(int i = 0; i < 3; i++)
46 eğer((cells[i][0].getToken() == token)
47 && (hücreler[i][1].getToken() == belirteç)n48 && (hücreler[i]
48 [2].getToken() == belirteç)) {
49 geri dönüş tehdidi;
50 }
51
52 güçlü(tinttj = t0t; j < t3t; j++)
53 if((cells[t0t][j].getToken() == belirteç)
54 && (hücreler[t1t][j].getToken() == belirteç)n55 && (hücreler[t
55 2t][j].getToken() == belirteç)) {
56 geri dönüş tehdidi;
57 }
58
59 if((cells[t0t][t0t].getToken() == jeton) n60 && (c
60 ells[t1t][t1t].getToken() == jeton) n61 && (cells[t2t][t2t].getTok
61 en() == jeton)) {
62 geri dönüş tehdidi;
63 }
64
65 if((cells[t0t][t2t].getToken() == jeton) n66 && (c
66 ells[t1t][t1t].getToken() == jeton) n67 && (cells[t2t][t0t].getTok
67 en() == jeton)) {
68 geri dönüş tehdidi;
69 }
70
71 false'i döndür;
72 }
73
74 // Bir hücre için iç sınıf
75 kamu sınıfı Hücre metni uzantısı JPanel {
76 // Bu hücre için kullanılan token
77 özel harita belirteci = 't';
78

```

kontrolDolu

satırları kontrol et

sütunları kontrol et

ana diyagonal kontrol et

alt diyagonal kontrol et

iç sınıf Hücre

```

79 kamusalHücre() {
80 setBorder(newLineBorder(Color.black, 1)); // Hücrenin kenarlığını ayarla
dinleyici kaydet 81 addMouseListener(tnewtMyMouseListener()); // Dinleyiciyi kaydet
82 }
83
84 /** Token'ı döndür */
85 kamusal chargetToken() {
86 jetonu iade et;
87 }
88
89 /** Yeni bir token ayarla */
90 genel voidtsetToken(tchartc) {
91 belirteç = c;n92 yenide
n boyama(); }

94
95 @Geçersiz Kıl /** Hücreyi boyayın */
hücreyi boyamak 96 korumalı voidtpaintComponent(Grafik g) {
97 supert.paintComponent(g);
98
99 ift(belirteç == t'X't) {
100 g.drawLine(t10t,t10t, getWidth() - t10t, getHeight() - t10t);n101 g.drawLine(getWid
th() - t10t,t10t,t10t, getHeight() - t10t);n102 }

103 else ift(token == t'O't) {
104 g.drawOval(t10t,t10t, getWidth() - t20t, getHeight() - t20t);n105 }

106 }n107

dinleyici sınıfı 108 e classtMyMouseListenerextendstMouseAdapter {
109 @Geçersiz kıl /** Bir hücre üzerindeki fare tıklamasını işleyin */
110 public voidtmouseClicked(MouseEvent e)nprivat {
111 // Eğer hücre boşsa ve oyun bitmemişse
112 ift(token == t' t&& kiminTurn != t' t) {
113 setToken(whoseTurn); // Hücredeki token'i ayarla
114
115 // Oyun durumunu kontrol et
116 ift(isWon(whoseTurn)) {
117 j1b1Status.setText(whoseTurn + " kazandı! Oyun bitti");
118 whoseTurn = ' '; // Oyun bitti
119 }
120 else ift(isFull()) {
121 j1b1Status.setText("Beraberlik! Oyun bitti");
122 whoseTurn = ' '; // Oyun bitti
123 }
124 aksi takdirde{
125 // Sıra değiştir
126 kiminDönüş = (kiminDönüş == t'X't) t'O't:t'X't;
127 // Kimin sırası olduğunu göster
128 j1b1Status.setText(whoseTurn + "ın sırası");
129 }
130 }n131 }n132 }
n133 }n134 }

```

ana yöntem atlandı

TicTacToe sınıfı, bir GridLayout panelinde yerleştirilmiş dokuz hücre ile kullanıcı arayüzünü başlatır (19–22. satırlar). Oyun durumunu göstermek için j1b1Status adında bir etiket kullanılır (14. satır). whoseTurn değişkeni (8. satır), bir sonraki token türünü takip etmek için kullanılır.

bir hücreye yerleştirildi. **isFull** (satırlar 34–41) ve **isWon** (satırlar 44–72) yöntemleri oyunun duru-  
munu kontrol etmek içindir.

Cell, TicTacToe içinde bir iç sınıf olduğundan, Cell sınıfından TicTacToe içindeki  
değişkenler (whoseTurn) ve yöntemler (isFull ve isWon) referans alınabilir. İç sınıf, pr-  
ogramları basit ve öz hale getirir. Eğer Cell, TicTacToe'nun iç sınıfı olarak tanımlan-  
masaydı, Cell içinde TicTacToe'daki değişkenler ve yöntemlerin kullanılabilmesi için  
bir TicTacToe nesnesi geçmeniz gerekirdi. Programı iç sınıf kullanmadan yeniden ya-  
zacaksınız, Programlama Egzersizi 18.6'da.

MouseEvent için dinleyici hücreye kaydedilir (satır 81). Eğer boş bir hücreye tıklanırsa ve oy-  
un bitmemişse, hücreye bir jeton yerleştirilir (satır 113). Eğer oyun bitmişse, whoseTurn ' ' (s-  
atırlar 118, 122) olarak ayarlanır. Aksi takdirde, whoseTurn yeni bir tura geçilir (satır 126).



#### İpucu

Bu tür bir Java projesi geliştirirken ve test ederken artan bir yaklaşım kullanın.  
Örneğin, bu program beş adıma ayrılabilir:

1. Kullanıcı arayüzünü düzenleyin ve bir hücrede sabit bir X simgesi gösterin.
2. Hücresin bir fare tıklamasıyla sabit bir X simgesi göstermesini sağlayın.
3. İki oyuncu arasında X ve O simgelerini sırayla gösterecek şekilde koordinasyon sağlayın.
4. Bir oyuncunun kazanıp kazanmadığını veya tüm hücrelerin dolup dolmadığını kontrol edin.
5. Her oyuncunun hamlesinde etiket üzerinde bir mesaj gösterme işlevini uygulayın.

artımlı geliştirme ve  
test etme

**18.18** Oyun başladığında, **whoseTurn** içinde ne değeri var? Oyun bittiğinde,

**whoseTurn** içinde ne değeri var?

**18.19** Kullanıcı oyunun bitmediği bir durumda boş bir hücreye tıkladığında ne olur? Oyun bittiğinde,  
kullanıcı boş bir hücreye tıkladığında ne olur?

**18.20** Program bir oyuncunun kazanıp kazanmadığını nasıl kontrol eder? Program tüm hücrelerin dolup  
dolmadığını nasıl kontrol eder?

**18.21** TicTacToe.java dosyasındaki 97. satırda **super.paintComponent(g)** ifadesini silin ve p-  
rogramı çalıştırarak ne olduğunu görün.

Point/Kontrol

MyProgrammingLab™

## 18.10 Kaynakları URL Sınıfını Kullanarak Bulma

Bir applet için bir kaynak dosyasını yüklemek üzere **URL** sınıfını kullanabilirsiniz, yeter ki kaynak  
dosyası applet'in sınıf dizininde bulunsun.



**ImageIcon** sınıfını bir resim dosyasından bir simge oluşturmak için kullanınız ve simgeyi bir GUI bileşenine, örneğin bir düğmeye veya

**bir etikete yerleştirmek için**

setIcon yöntemini veya yapıcuyu kullanınız. Örneğin, aşağıdaki ifadeler bir ImageIcon oluşturur ve bunu bir JLabel  
nesnesi **jlbl** üzerine ayarlar:

```
ImageIcon imageIcon = new ImageIcon("c:\\book\\imageus.gif"); jlbl.setIcon(imageIcon);
```

Bu yaklaşım bir sorun ortaya çıkarıyor. Dosya konumu sabittir, çünkü Windows platformunda  
mutlak dosyayolunu kullanır. Sonuç olarak, program diğer platformlarda çalışamaz ve applet o-  
larak çalışamaz. **image/us.gif** dosyasının sınıf dizini altında olduğunu varsayalım. Bu sorunu aş-  
mak için aşağıdaki gibi bir görelî yol kullanabilirsiniz:

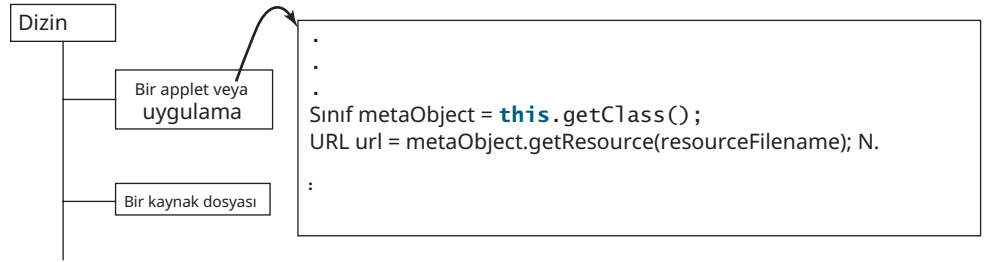
```
ImageIcon imageIcon = new ImageIcon("image/us.gif");
```

Bu, tüm platformlardaki Java uygulamalarıyla iyi çalışır ancak Java applet'leriyle çalışmaz, çünkü  
applet'ler yerel dosyaları yükleyemez. Hem uygulamalar hem de applet'lerle çalışabilmesi için,  
dosyanın **URL**'sini (Uniform Resource Locator) bulmanız gerekir.

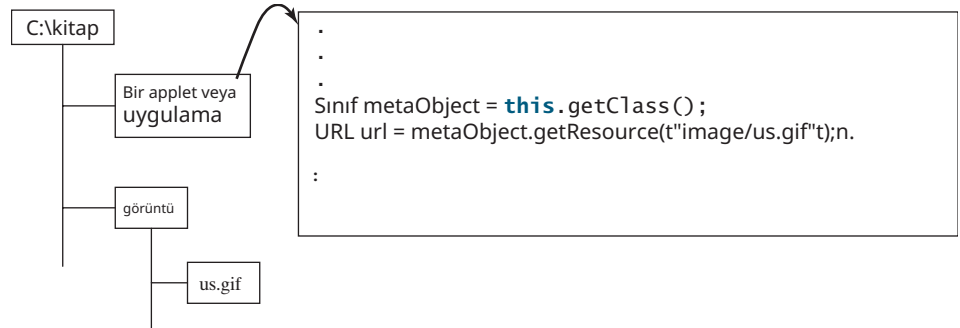
nedenURL sınıfı?

`java.net.URL` sınıfı, Bölüm 14.13'te İnternette bir metin dosyasını bulmak için kullanıldı. Aynı zamanda, İnternetteki resim dosyalarını ve ses dosyalarını bulmak için de kullanılabilir. Genel olarak, bir `URL` nesnesi, lokal bir makinedeki veya uzak bir sunucudaki bir "kaynağa" işaret eder. Bir kaynak bir dosya veya bir dizin olabilir.

`URL` sınıfı, bir kaynak dosyasını, dosyanın konumundan bağımsız olarak bir sınıftan bulmak için kullanılabilir, şartıyla ki kaynak dosyası sınıf dizininde bulunsun. Sınıf dizini, sınıfın saklandığı yerdir. Bir sınıftan bir dosya için `URL` nesnesini elde etmek için, applet veya uygulamada aşağıdaki ifadeyi kullanın:



`getClass()` metodu, mevcut sınıf için `java.lang.Class` sınıfının bir örneğini döndürür. Bu örnek, belleğe yüklenen her sınıf için JVM tarafından otomatik olarak oluşturulur. Bu örnek, meta nesne olarak da bilinir ve sınıf dosyası hakkında sınıf adı, yapılar ve yöntemler gibi bilgileri içerir. Sınıf yolundaki bir dosya için `URL` nesnesi elde etmek için meta nesne üzerinde `getResource(filename)` metodunu çağırabilirsiniz. Örneğin, sınıf dosyası `c:\ook` içindeyse, aşağıdaki ifadeler `c:\ook\image\us.gif` için bir `URL` nesnesi elde eder.



Artık bir `ImageIcon` oluşturabilirsiniz

```
ImageIcon imageIcon = new ImageIcon(url);
```

Liste 18.11, sınıf dizinindeki `image/us.gif` dosyasından bir resmi görüntüleyen kodu verir. `image/us.gif` dosyası sınıf dizinindedir ve `URL` nesnesi `getResource` metodu (satır 5) kullanılarak elde edilir. Satır 6'da bir resim simgesi ile bir etiket oluşturulur. Resim simgesi `URL` nesnesinden elde edilir.

### LİSTELER 18.11 `DisplayImageWithURL.java`

```
1 import javax.swing.*;
2
3 genel classt DisplayImageWithURL extends JApplet {
```

```

4 publicDisplayImageWithURL() {
5 java.net.URL url = bu.getClass().getResource("image/us.gif") ;
6 ekle(tnewn7} JLabel(newtImageIcon(url)));
8 }

```

görüntü URL'sini al  
bir etiket oluştur  
ana yöntem atlandı

Eğer 5-6. satırlardaki kodu aşağıdaki kodla değiştirirseniz,

```
add(tnewtJLabel(tnewtImageIcon(t"image/us.gif"t)));
```

programı bağımsız bir uygulama olarak çalıştırmaya devam edebilirsiniz, ancak bir tarayıcıdan applet olarak çalıştıramazsınız, Şekil 18.13'te gösterildiği gibi.



**FIGURE 18.13** Applet, applet ile aynı dizinde bulunan bir resim dosyasından bir görüntü yükler.

**18.22** Sınıf dizinindeki `image/us.gif` dosyası için bir URL nesnesi nasıl oluşturursunuz?

**18.23** Sınıf dizinindeki `image/us.gif` dosyasından bir ImageIcon nasıl oluşturursunuz?

Point/Kontrol

MyProgrammingLab™

## 18.11 Herhangi Bir Java Programında Ses Oynatma

*Applet sınıfı, bir ses dosyası için bir AudioClip nesnesi elde etme yöntemle rini içerir. AudioClip nesnesi, ses dosyalarını çalma yöntemlerini içerir.*



Ses dosyaları için birkaç format vardır. Java programları WAV,

AIFF, MIDI, AU ve RMF formatlarındaki ses dosyalarını çalıştırabilir.

Java'da (uygulama veya applet) bir ses dosyasını çalmak için önce dosya için bir *ses klipi nesnesi* oluşturun. Ses klipi bir kez oluşturulur ve dosyayı yeniden yüklemeyi tekrar tekrar çalıştırabilir. Bir ses klipi oluşturmak için `java.applet.Applet` sınıfındaki statik `newAudioClip()` yöntemini kullanın:

```
AudioClip audioClip = Applet.newAudioClip(url);
```

Ses başlangıçta yalnızca Java applet'lerinden oynatılabiliyordu. Bu nedenle, `AudioClip` arayüzü `java.applet` paketindedir. JDK 1.2'den itibaren, ses herhangi bir Java programında oynatılabilir.

Aşağıdaki ifadeler, örneğin, sınıf dizinindeki `beep.au` ses dosyası için bir `AudioClip` oluşturur:

```

Sınıf metaObject = tthist.getClass();nURL url
= metaObject.getResource(t"beep.au"t);nAudioClip audi
oClip = Applet.newAudioClip(url);

```

Bir ses parçasını bir ses klipi için manipüle etmek için, **play()**, **loop()** ve **stop()** yöntemlerini kullanın **java.applet.AudioClip** içinde, Şekil 18.14'te gösterildiği gibi.

titinterface <i>java.applet.AudioClip</i>	
<b>+play()</b>	Starts playing this audio clip. Each time this method is called, the clip is restarted from the beginning.
<b>+loop()</b>	Plays the clip repeatedly.
<b>+stop()</b>	Stops playing the clip.

**FIGURE 18.14** Ses Klipleri arayüzü ses çalma yöntemlerini sağlar.

Liste 18.12, Danimarka bayrağını gösteren ve Danimarka milli marşını tekrar tekrar çalan kodu verir. Resim dosyası **image/denmark.gif** ve ses dosyası **audio/denmark.mids**ni f dizininde saklanmaktadır. Satır 12, ses dosyası için **URL** nesnesini alır, satır 13 ses dosyası için bir ses klipi oluşturur ve satır 14 sesi tekrar tekrar çalar.

### LISTING 18.12 DisplayImagePlayAudio.java

görüntü URL'sini al  
bir etiket oluştur

ses URL'sini al  
bir ses klipi oluştur  
ses tekrar tekrar çal

ses başlat

ses durdur

ana yöntem atlandı

```

1 import javax.swing.*;
2 java.net.URL'yi içe aktarın;
3 import java.applet.*;
4
5
6 genel classtDisplayImagePlayAudioextendstJApplet {
7
8 publictDisplayImagePlayAudio() {
9 URL urlForImage = getClass().getResource("image/denmark.gif");n10 add((tnewtJLabel(
11 tnewtImageIcon(urlForImage)));
12
13 URL urlForAudio = getClass().getResource("audio/danmark.mid");
14 AudioClip = Applet.newAudioClip(urlForAudio);
15 AudioClip.loop();
16
17 @Geçersiz Kıl
18 genel voidstart() {
19 ıft(audioClip != tnultt) audioClip.loop();
20 }
21
22 @Geçersiz Kıl
23 genel voidtstop() {
24 ıft(audioClip != tnultt) audioClip.stop();
25 }n26
26 }

```

**stop** metodu (satırlar 23–25) applet görüntülenmediğinde sesi durdurur ve start metodu (satırlar 18–20) applet yeniden görüntülendiğinde sesi yeniden başlatır. Bu appleti **stop** ve **start** metodları olmadan bir tarayıcıdan çalıştırmayı deneyin ve etkisini gözlemleyin.

Bu programı ana metoddan ve bir Web tarayıcısından bağımsız bir uygulama olarak çalıştırarak test edin. Kısaklık açısından, tüm appletlerdeki **main** metodunun metinde basılmadığını hatırlayın.

**18.24** Java'da hangi tür ses dosyaları kullanılır?

**18.25** class dizinindeki **anthem/us.mid** dosyasından bir ses klipi nasıl oluşturursunuz?

**18.26** Bir ses klipini nasıl çalarsınız, tekrar tekrar çalarsınız ve durdurursunuz?

## 18.12 Vaka Çalışması: Ulusal Bayraklar ve Marşlar

*Bu vaka çalışması, bir ulusun bayrağını gösteren ve marşını çalan bir applet sunmaktadır.*



Applet'teki görüntüler, Danimarka, Almanya, Çin, Hindistan, Norveç, Birleşik Krallık ve ABD için adlandırılan yedi ulusal bayrağa aittir: **flag0.gif, flag1.gif, . . . , flag6.gif**. Bu görüntüler, sınıf yolundaki **image** dizininde saklanmaktadır. Ses, bu yedi ulusun ulusal marşlarından oluşmaktadır ve adlandırılmıştır: **anthem0.mid, anthem1.mid, . . . , ve anthem6.mid**. Bu sesler, sınıf yolundaki **audio** dizininde saklanmaktadır.

Program, kullanıcının bir açılır kutudan bir ulus seçmesini sağlar ve ardından bayrağını gösterir ve marşını çalar. Kullanıcı, sesi **Askıya Al**/butonuna tıklayarak durdurabilir ve

**Devam Et** butonuna tıklayarak devam ettirebilir; bu, Şekil 18.15'te gösterilmiştir.



ŞEKİL 18.15 Applet, bir dizi görüntü gösterir ve ses çalar.

Program, Liste 18.13'te verilmiştir.

### LİSTELER 18.13 FlagAnthem.java

```

1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4 import java.applet.*;
5
6 genel sınıf FlagAnthem extends JApplet {
7 özel final statik int NUMBER_OF_NATIONS = 7;
8 özel int current = 0;
9 Private ImageIcon[] simgeler = new ImageIcon[NUMBER_OF_NATIONS];
10 AudioClip[] audioClips = new AudioClip[NUMBER_OF_NATIONS];
11 Private AudioClip currentAudioClip;
12
13 Private JLabel lblImageLabel = new JLabel();
14 private JButton jbtResume = new JButton("Devam Et");
15 private JButton jbtSuspend = new JButton("Askıya Al");
16 Private JComboBox jcbNations = new JComboBox(new Object[]
17 { "Danimarka", "Almanya", "Çin", "Hindistan", "Norveç", "İngiltere", "ABD" });
18
19 public FlagAnthem() {
20 // Görüntü simgelerini ve ses kliplerini yükle
21 for (int i = 0; i < NUMBER_OF_NATIONS; i++) {
22 simge[i] = new ImageIcon(getClass().getResource(
23 "resim/bayrak" + i + ".gif"));
24 wAudioClip(
25 tResource("audio/anthem" + i + ".mid"));
26 naudioClips[i] = Applet.ne
27 }
28 }
29 JPanel paneli = new JPanel();

```



VideoNotu

Ses ve görüntü

görüntü simgeleri  
audio klipleri  
mevcut ses klibi

GUI bileşenleri

ikonlar oluştur

ses klipleri oluştur

UI oluştur

```

29 panel.add(jbtResume);
30 panel.add(jbtSuspend);
31 panel.add(new JLabel("Seçin"));
32 panel.add(jcboNations);
33 add(jlblImageLabel, BorderLayout.CENTER);n34 add(panel,
BorderLayout.SOUTH);
35
dinleyici kaydet 36 jbtResume.addActionListener(new ActionListener() {
37 @Geçersiz Kıl
38 kamusal void actionPerformed(ActionEvent e) {
ses başlat 39 başlangıç();
40 }
41 });
dinleyici kaydet 42 jbtSuspend.addActionListener(new ActionListener() {n43 @Override
e
44 kamusal void actionPerformed(ActionEvent e) {
ses durdur 45 durmak();
46 }
47 });
dinleyici kaydet 48 jcboNations.addActionListener(new ActionListener() {n49 @Override
e
50 kamusal void actionPerformed(ActionEvent e) {
51 dur();
bir ulus seçin 52 current = jcboNations.getSelectedIndex();
bir ulusu sunun 53 mevcutUlke(current);
54 }n55 });

56
57 lblImageLabel.setIcon(icons[0]);
58 lblImageLabel.setHorizontalAlignment(JLabel.CENTER);
59 ntAudioClip = audioClips[t0t];
60 currentAudioClip.play();ncurre
61 }
62
63 private void mevcutUlke(int index) {
64 lblImageLabel.setIcon(icons[index]);
65 jcboNations.setSelectedIndex(index);
66 currentAudioClip = audioClips[index];
67 geçerli ses klibi.play();
68 }
69
70 @Geçersiz Kıl
71 genel void start() {
bir klip oynat 72 geçerli ses klibi.play();
73 }
74
75 @Geçersiz Kıl
76 genel void stop() {
ses klibini durdur 77 currentAudioClip.stop();
doğru {n
ana yöntem atlandı 78 }

```

Bir etiket, 13. satırda bir bayrak görüntüsü göstermek için oluşturulmuştur. Yedi ülkenin bayrak görüntülerini n bir dizisi 22-23. satırlarda oluşturulmuştur. Ses kliplerinin bir dizisi 24-25. satırlarda oluşturulmuştur. Görüntü dosyaları ve audio dosyaları, bu dosyaların `getResource` yöntemi kullanılarak bulunabilmesi için applet sınıf do syasıyla aynı dizinde saklanmaktadır.

Ülke adları için açılır kutu 16-17. satırlarda oluşturulmuştur. Açılır kutuda yen i bir ülke adı seçildiğinde, mevcut sunum durdurulur ve yeni seçilen bir ülkesun ulur (51-53. satırlar).



presentNation(index) yöntemi (satırlar 63–68) belirtilen indeks ile bir ulusu sunar. Etiketin içinde yeni bir resim ayarlar (satır 64), seçilen indeksi ayarlayarak droplist ile senkronize eder (satır 65) ve yeni sesi oynatır (satır 67).

applet'in **start** ve **stop** yöntemleri sesin devam etmesi ve durdurulması için geçersiz kılınmıştır (satırlar 70–78).

**18.27** Başlangıç görüntü simgesini hangi kod ayarlıyor? Başlangıç ses klipini hangi kod oynatıyor?

**18.28** Program, *Suspend* butonuna tıklandığında ne yapar? Program, *Resume* butonuna tıklandığında ne yapar?

Point/Kontrol

MyProgrammingLab™

#### ANAHTAR TERİMLER

applet 673	HTML 673
applet konteyneri 677	imzalı applet 676
arşiv 674	etiket 673

## BÖLÜM ÖZETİ

1. **JApplet**, **Applet**'in bir alt sınıfıdır. Swing bileşenleri ile Java applet'leri geliştirmek için kullanılır.
2. Applet sınıf dosyası, applet'in nerede bulunacağını web tarayıcısına bildirmek için bir **<applet>** etiketi kullanılarak bir HTML dosyası içinde belirtilmelidir. Applet, **<param>** etiketi kullanarak HTML'den dize parametreleri alabilir.
3. Applet konteyneri, **Applet** sınıfındaki **init**, **start**, **stop** ve **destroy** yöntemleri aracılığıyla applet'leri kontrol eder ve çalıştırır.
4. Bir applet yüklendiğinde, applet konteyneri, argüman almayan yapıcısını çağırarak bir örneğini oluşturur. Applet oluşturulduktan sonra **init** yöntemi çağrılır. **start** yöntemi, **init** yönteminden sonra çağrılır. Ayrıca, applet'in bulunduğu sayfa tekrar ziyaret edildiğinde applet aktif hale geldiğinde de çağrılır. Applet, inaktif hale geldiğinde **stop** yöntemi çağrılır.
5. Tarayıcı normal şekilde çıktığında, applete artık gerekli olmadığını ve tahsis ettiği kaynakları serbest bırakması gerektiğini bildirmek için **destroy** metodu çağrılır. **stop** metodu her zaman **destroy** metodundan önce çağrılır.
6. Uygulamalar ve applet'ler çok benzerdir. Bir applet kolayca bir uygulamaya dönüştürülebilir ve tam tersi de geçerlidir. Ayrıca, bir applet bağımsız olarak çalışmak için **main** metoduyla yazılabilir.
7. HTML'deki applet etiketinde **param** niteliğini kullanarak bir applet'e argümanlar geçirebilirsiniz. Parametrenin değerini almak için **getParameter(paramName)** metodunu çağırın.
8. Applet'in **getParameter** metodu yalnızca applet'in bir örneği oluşturulduktan sonra çağrılabilir. Bu nedenle, bu metod applet sınıfının yapıcısında çağrılmaz. Bu metodu **init** metodundan çağırmalısınız.
9. Java uygulamaları ve applet'lerde resim ve sesleri nasıl dahil edeceğinizi öğrendiniz. Java uygulamaları ve applet'ler için ses ve resimleri yüklemek için, ses ve resim dosyası için bir **URL** nesnesi oluşturmalısınız. Kaynak dosyaları sınıf dizininde saklanmalıdır.

- 10.** Bir ses çalmak için, ses kaynağı için **URL** nesnesinden bir ses klibi oluşturun. Bunu bir kez çalmak için **AudioClip**'in **play()** metodunu, tekrar çalmak için **loop()** metodunu ve durdurmak için **stop()** metodunu kullanabilirsiniz.

## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

MyProgrammingLab™

## PROGRAMLAMA EGZERSİZLERİ

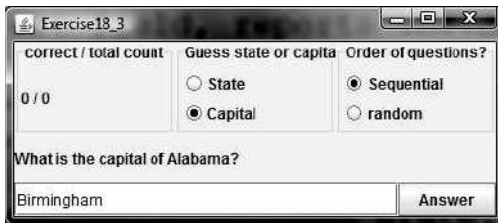


### Pedagogik Not

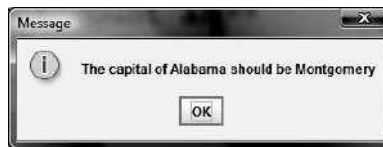
Her egzersiz için, bağımsız bir uygulama olarak çalışabilmesi için bir ana yöntem ekleyin.

### Bölümler 18.2–18.6

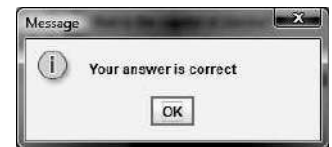
- 18.1** (*Kredi hesaplayıcı*) Kredi ödemesini hesaplamak için bir applet haline getirmek üzere Listeleme 16.7, **LoanCalculator.java** dosyasını gözden geçirin.
- 18.2** (*Uygulamaları applet'lere dönüştür*) Listeleme 16.12'deki **ClockAnimation**'i bir applet olarak yeniden yazın ve bağımsız çalışmasını sağlayın.
- \*18.3** (*Başkentleri ve eyaletleri tahmin et*) Programlama Egzersizi 9.17'yi gözden geçirerek, kullanıcıdan bir eyalet için bir başkent girmesini veya tersini sürekli olarak isteyen bir applet yazın; bu, Şekil 18.16a'da gösterilmiştir. *Cevap* butonuna tıkladığında, program kullanıcı girişini alanından alır, cevabın doğru olup olmadığını bir mesaj iletişim kutusunda (Şekil 18.16b–c) bildirir, doğru sayısını ve toplam sayıyı gösterir ve ardından bir sonraki soruyu görüntüler. Kullanıcı, programın soruları rastgele mi yoksa sıralı mı oluşturmasına ve bir başkent veya bir eyalet için sorular oluşturmasına izin verip vermeyeceğini belirtebilir.



(a)



(b)



(c)

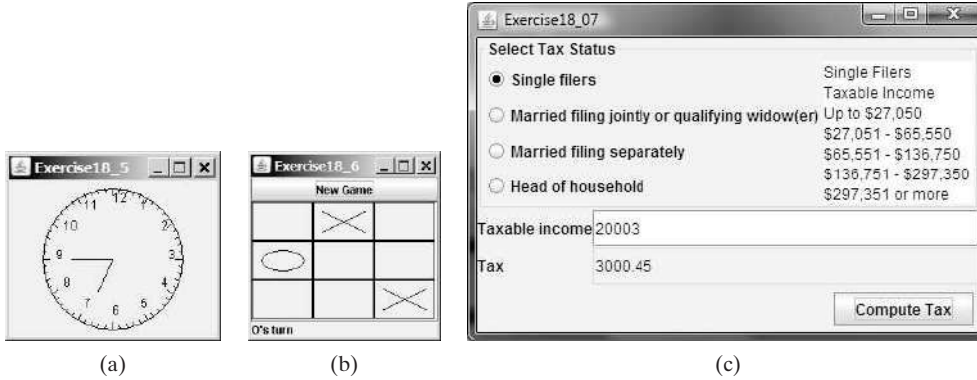
**ŞEKİL 18.16** Applet, kullanıcı bilgisini eyaletler ve başkentler hakkında test eder.

- \*18.4** (*String'leri applet'lere geçirme*) Listeleme 18.5, **DisplayMessage.java** dosyasını, standart bir renk, yazı tipi ve boyutla bir mesaj görüntüleyecek şekilde yeniden yazın. **mesaj.x.y**, **renk.yazı tipi adı** ve **yazı tipi boyutu**, aşağıda gösterildiği gibi **<applet>** etiketindeki parametrelerdir:

```
<uygulama
 codet=t"Exercise18_04.class"nwidth
 t=t200
 yükseklik=t50
 alt= "Applet'i görüntülemek için Java destekli bir tarayıcıya sahip olmalısınız"
>
 <param name=MESSAGEvalue="Java'ya Hoş Geldiniz"/>
 <param name = X value = 40 />
 <param name=tYtvalue=t50t/>n<pa
 ram name=tCOLORTvalue=t"kırmızı"t/>
```

```
<param name=tFONTNAMEvalue=t"Tek aralıklı"t/>n<param name=tFONTSIZEvalue=t20t/>
</applet>
```

- \*\*18.5** (Oyun: bir saat öğrenme aracı) Birinci sınıf öğrencisine saati nasıl okuyacağını göstermek için bir saat appleti geliştirin. Programlama Alıştırması 13.19'u, Şekil 18.17a'da gösterildiği gibi, bir saat ve bir dakika ibresi ile detaylı bir saat gösterecek şekilde değiştirin. Saat ve dakika değerleri rastgele üretilir. Saat 0 ile 11 arasında, dakika ise 0,15,30 veya 45'dir. Fare tıklamasıyla, saatte yeni bir rastgele zaman görüntülenir.



**ŞEKİL 18.17** (a) Saat üzerine tıklandığında, saat zamanı rastgele görüntülenir. (b) Yeni Oyun butonuna tıklamak yeni bir oyunu başlatır. (c) Vergi hesap makinesi, tanımlanan vergilendirilebilir gelir ve vergi durumu için vergiyi hesaplar.

- \*\*18.6** (Oyun: üçlü-dörtgen) Listing 18.10 TicTacToe.java programını aşağıdaki değişikliklerle yeniden yazın:

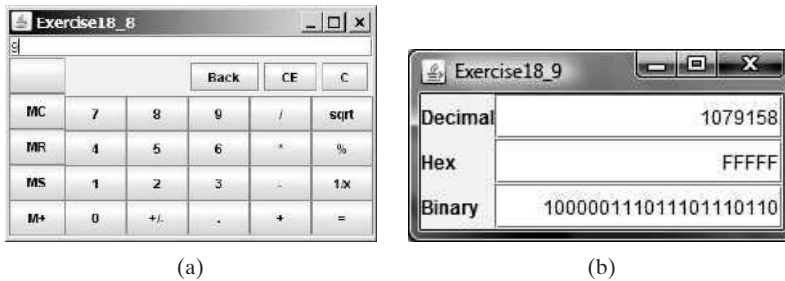
- Cell'i iç sınıf yerine ayrı bir sınıf olarak tanımlayın.
- Yeni Oyun adında bir buton ekleyin, Şekil 18.17b'de gösterildiği gibi.

Yeni Oyun butonuna tıklamak yeni bir oyunu başlatır.

- \*\*18.7** (Finansal uygulama: vergi hesaplayıcı) Vergiyi hesaplamak için bir applet oluşturun, Şekil 18.17c'de gösterildiği gibi. Applet, kullanıcının vergi durumunu seçmesine ve taxable income'u girmesine olanak tanır, böylece 2001 federal vergi oranlarına göre vergiyi hesaplayabilir, Programlama Egzersizi 10.8'de gösterildiği gibi.

- \*\*\*18.8** (Bir hesap makinesi oluşturun) Aşağıdaki hesap makinesini yerleştirmek ve toplama (+), çıkarma (-), bölme (/), karekök (sqrt) ve modül (%) işlevlerini uygulamak için FlowLayout, GridLayout ve

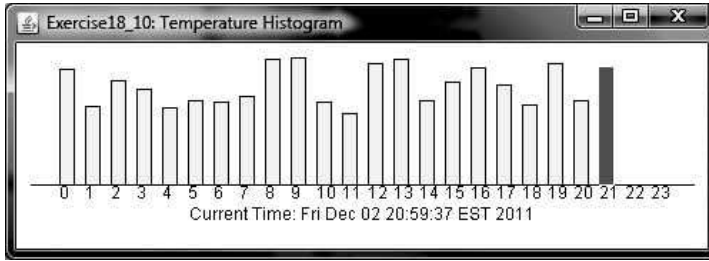
BorderLayout'un çeşitli panellerini kullanın (bkz. Şekil 18.18a).



**ŞEKİL 18.18** (a) Alıştırma 18.8, popüler bir hesap makinesinin Java uygulamasıdır. (b) Alıştırma 18.9, ondalık, onaltılık ve ikilik sayılar arasında dönüşüm yapar.

**\*18.9** (*Sayıları dönüştür*) Onaltılık, ikilik ve ondalık sayılar arasında dönüştüren bir applet yazın, Şekil 18.18b'de gösterildiği gibi. Ondalık değer metin alanına bir ondalık değer girdiğinizde ve *Enter* tuşuna bastığınızda, karşılık gelen onaltılık ve ikilik sayılar diğer iki metin alanında görüntülenir. Aynı şekilde, diğer alanlara da değerler girebilir ve bunları uygun şekilde dönüştürebilirsiniz.

**\*\*18.10** (*Bir alanı yeniden boyayın*) Bir panelin tüm görüntüleme alanını yeniden boyadığınızda, görüntüleme alanının sadece küçük bir kısmı değişir. Sadece etkilenen alanı yeniden boyayarak performansı artırabilirsiniz, ancak çağrıda bulunmayan `super.paintComponent(g)` paneli yeniden boyarken, çünkü bubütün görüntü alanının temizlenmesine neden olacaktır. Bu yaklaşımı kullanarak, son 24 saat boyunca her saatin sıcaklıklarını bir histogramda görüntülemek için bir applet yazın. 50 ile 90 derece Fahrenheit arasındaki sıcaklıkların rastgele elde edildiğini ve her saat güncellendiğini varsayalım. Mevcut saatin sıcaklığı yeniden gösterilmesi gerekirken, diğerleri değişmeden kalmalıdır. Mevcut saatin sıcaklığının vurgulamak için benzersiz bir renk kullanın (bkz. Şekil 18.19a).



(a)



(b)

**ŞEKİL 18.19** (a) Histogram, son 24 saatte her saatin ortalama sıcaklığını gösterir. (b) Program bir çalışan fanı simüle eder.

**\*\*18.11** (*Simülasyon: çalışan bir fan*) Şekil 18.19b'de gösterildiği gibi, çalışan bir fan simüle eden bir Java applet'i yazın. *Başlat*, *Durdur* ve *Ters* butonları fanı kontrol eder. Kaydırıcı, fanın hızını kontrol eder. `Fan` adında bir sınıf oluşturun, bir alt sınıf olarak `JPanel`, fanı göstermek için. Bu sınıf ayrıca fanı askıya almaya yeniden başlatma, hızını ayarlama ve yönünü tersine çevirme yöntemlerini içerir. `FanControl` adında bir sınıf oluşturun, bir fan ve fanı kontrol etmek için üç düğme ve bir kaydırıcı içersin. `FanControl` örneğini içeren bir Java applet'i oluşturun.

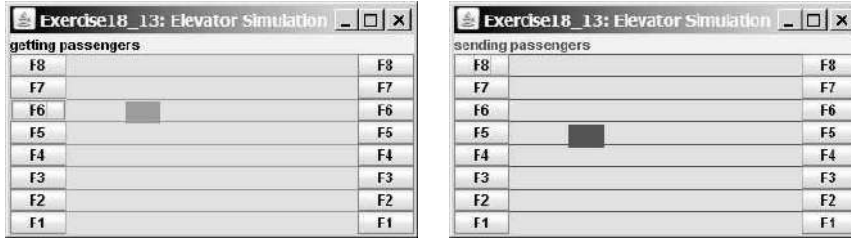
**\*\*18.12** (*Bir grup fanı kontrol et*) Üç fanı bir grup içinde gösteren bir Java applet'i yazın, hepsini başlatmak ve durdurmak için kontrol düğmeleri ile, Şekil 18.20'de gösterildiği gibi.



**FIGURE 18.20** Program, bir grup fanı çalıştırır ve kontrol eder.

**\*\*\*18.13** (*Bir asansör simülatörü oluşturun*) Bir asansörün yukarı ve aşağı gittiğini simüle eden bir applet yazın (bkz. Şekil 18.21). Soldaki düğmeler, yolcunun şu anda bulunduğu katı gösterir. Yolcu, asansörün kendi katına gelmesini istemek için soldaki bir düğmeye tıklamalıdır.

Asansöre girdikten sonra, yolcu sağdaki bir düğmeye tıklayarak belirtilen kata gitmesini talep eder.



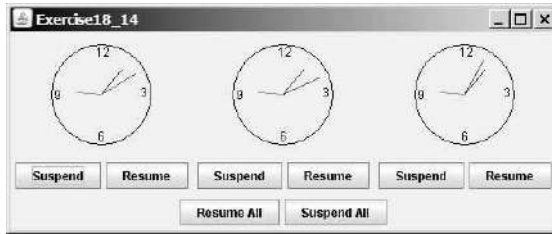
FIGÜR18.21 Program asansör işlemlerini simüle eder.



VideoNotu

Bir grup saati kontrol et

- \*18.14** (Bir grup saati kontrol et) Üç saati bir grup içinde gösteren bir Java applet'i yazın, hepsini başlatmak ve durdurmak için kontrol düğmeleri ile, Şekil 18.22'de gösterildiği gibi.



FIGÜR18.22 Üç saat bağımsız olarak bireysel kontrol ve grup kontrolü ile çalışır.

### Bölümler 18.10–18.12

- \*18.15** (Bir resmi büyüt ve küçült) Farklı boyutlarda tek bir resim dosyasından bir dizi resmi gösterecek bir applet yazın. Başlangıçta, bu resim için görüntüleme alanının genişliği 300 ve yüksekliği 300'dür. Programınız, görüntüleme alanını sürekli olarak 1 birim genişlikte ve 1 birim yükseklikte küçülmelidir, ta ki genişlik 50 ve yükseklik 50'ye ulaşana kadar. O noktada, görüntüleme alanı sürekli olarak 1 birim genişlikte ve 1 birim yükseklikte büyümelidir, ta ki genişlik 300 ve yükseklik 300'e ulaşana kadar. Görüntüleme alanı, tek bir resim için animasyon oluşturmak amacıyla (sırasıyla) küçülmeli ve büyütülmelidir.

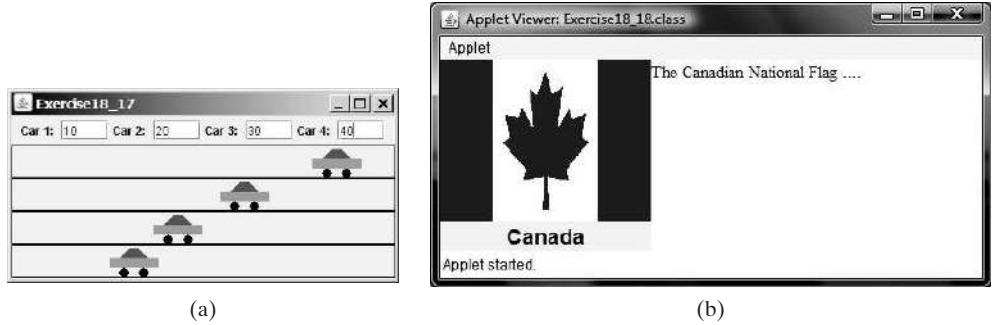
- \*\*\*18.16** (Bir borsa endeksi simüle et) Bir borsa endeksi ticker'ı görüntüleyen bir Java applet'i yazın (Şekil 18.23'e bakın). Borsa endeksi bilgisi, HTML dosyasındaki `<param>` tag'inden geçmektedir. Her endeksin dört parametresi vardır: Endeks Adı (örneğin, S&P



FIGÜR18.23 Program bir borsa endeksi ticker'ı görüntüler.

500), Mevcut Zaman (örneğin, 15:54), bir önceki günün endeksi (örneğin, 919.01) ve Değişim (örneğin, 4.54). Dow Jones, S&P 500, NASDAQ, NIKKEI ve Altın & Gümüş Endeksi gibi en az beş endeks kullanın. Pozitif değişimleri yeşil, negatif değişimleri kırmızı olarak görüntüleyin. Endeksler, applet'in görüntüleme alanında sağdansa hareket eder. Applet, fare düğmesine basıldığında ticker'ı dondurur; fare düğmesi bırakıldığında tekrar hareket eder.

**\*\*18.17** (*Yarış arabaları*) Şekil 18.24a'da gösterildiği gibi dört arabanın yarıştığını simüle eden bir applet yazın. Her aracın hızını ayarlayabilirsiniz, maksimum 100.



**FIGÜR 18.24**(a) Her aracın hızını ayarlayabilirsiniz. (b) Bu applet, her ülkenin bayrağını, adını ve açıklamasını birer birer gösterir ve şu anda

görüntülenen açıklamayı okur.

**\*\*18.18** (*Ulusal bayrakları göster*) Her birinin fotoğrafı, adı ve açıklamasıyla (bkz. Şekil 18.24b) birlikte açıklamayı okuyan sesle, ulusal bayrakları tanıtan bir applet yazın. Applet'in sekiz ülkenin bayraklarını gösterdiğini varsayın. Fotoğraf görüntü dosyalarının adlandırıldığını varsayın **flag0.gif, flag1.gif**, ve devamı, **flag7.gif**, applet'in dizininde **image** adlı bir alt dizinde saklanmaktadır. Her sesin uzunluğu 10 saniyeden azdır. Her ülkenin bayrağının adı ve açıklamasının HTML üzerinden **name0, name1, ..., name7, vedescription0, description1, ..., ve description7** parametreleri kullanılarak geçirildiğini varsayın. Ülke sayısını HTML parametresi olarak **numberOfCountries** kullanarak geçirin. İşte bir örnek:

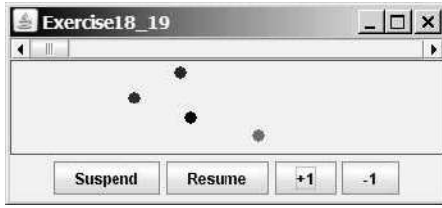
```
<param name = "numberOfCountries" value = 8>
<param name="name0" value="Kanada">
<param name="description0" value="Kanadalı ... ">
```



#### İpucu

Görüntüyü, adı ve metni görüntülemek için **DescriptionPanel** sınıfını kullanın. **DescriptionPanel** sınıfı, Liste 17.2'de tanıtılmıştır.

**\*\*\*18.19** (*Zıplayan toplar*) Bölüm 18.8'deki örnek, zıplayan bir topu simüle etmektedir. Örneği, Şekil 18.25a'da gösterildiği gibi birden fazla topu ekleyecek şekilde genişletin. Siz **+t1** veya **-t1** butonunu kullanarak topların sayısını artırabilir veya azaltabilir, ve topları dondurmak veya zıplamaya devam etmek için **Durdur** ve **Devam** butonlarını kullanabilirsiniz. Her topa rastgele bir renk atayın.



(a)



(b)

**ŞEKİL 18.25**(a) Applet, zıplayan topları eklemenize veya çıkarmanıza olanak tanır. (b) *Oynat* düğmesine tıklayarak bir ses klibini bir kez oynatın, *Döngü* düğmesine tıklayarak sesi tekrar tekrar oynatın ve *Durdur* düğmesine tıklayarak oynamayı sonlandırın.

**\*18.20** (Bir ses klibini oynat, döngüye al ve durdur) Aşağıdaki gereksinimleri karşılayan bir applet yazın:

- Sınıf dizininden bir ses dosyası alın.
- Şekil 18.25b'de gösterdiği gibi *Oynat*, *Döngü* ve *Durdur* etiketli üç düğme yerleştirin.
- Eğer *Oynat* butonuna tıklarsanız, ses dosyası bir kez çalınır. Eğer *Tekrar* butonuna tıklarsanız, ses dosyası sürekli olarak çalmaya devam eder. Eğer *Durdur* butonuna tıklarsanız, çalma durur.
- Applet bir uygulama olarak çalışabilir.

**\*\*18.21** (Bir alarm saati oluşturun) Saat, dakika ve saniyeyi gösteren büyük bir ekran paneline sahip dijital bir saat gösterecek bir applet yazın. Bu saat, kullanıcının bir alarm ayarlamasına izin vermelidir. Şekil 18.26a, böyle bir saatin örneğini göstermektedir. Alarmı açmak için, *Alarm* onay kutusunu işaretleyin. Alarm zamanını belirtmek için, *Set alarm* butonuna tıklayın ve yeni bir çerçeve görüntülenecektir; bu, Şekil 18.26b'de gösterilmiştir. Alarm zamanını çerçevede ayarlayabilirsiniz.



(a)



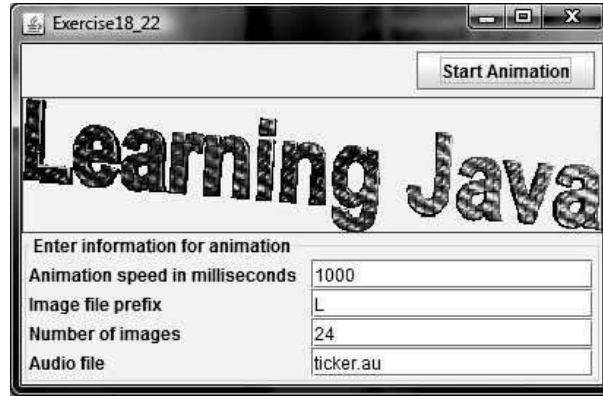
(b)

**ŞEKİL 18.26**Program, mevcut saat, dakika ve saniyeyi gösterir ve bir alarm ayarlamasına olanak tanır.

**\*\*18.22** (Sesli bir görüntü animatörü oluşturun) Aşağıdaki gereksinimleri karşılamak için Şekil 18.27'de gösterilen applet'i kullanarak animasyon oluşturun:

- Kullanıcının bir metin alanında animasyon hızını belirtmesine izin verin.
- Kullanıcıdan kare sayısını ve görüntünün dosya adı ön ekini alın. Örneğin, kullanıcı *n*'yi kare sayısı için ve *L*'yi görüntü ön eki için girerse, dosyalar *L1*, *L2* ve devamı *Ln* olacaktır. Görüntülerin *image* dizininde, applet'in dizininin bir alt dizininde saklandığını varsayın.
- Kullanıcının bir ses dosyası adı belirtmesine izin verin. Ses dosyası, applet ile aynı dizinde saklanır. Ses, animasyon çalışırken çalınır.





ŞEKİL 18.27 Bu applet, kullanıcının resim dosyalarını, bir ses dosyasını ve animasyon hızını seçmesine olanak tanır.

**\*\* 18.23** (Simülasyon: bayrağı kaldır ve marşı çal) Şekil 16.1b–d’de gösterildiği gibi, yukarı doğru yükselen bir bayrağı görüntüleyen bir applet oluşturun. Ulusal bayrak yükselirken, ulusal marşı çalın. (18.13 numaralı listedeki bir bayrak görüntüsü ve marş ses dosyasını kullanabilirsiniz.)

### Kapsamlı

**\*\*\* 18.24** (Oyun: fasulye makinesi animasyonu) Programlama Egzersizi 16.22’deki fasulye makinesi animasyonu geliştiren bir applet yazın. Applet, Şekil 18.28’de gösterildiği gibi slot sayısını ayarlamanıza olanak tanır. Animasyonu başlatmak veya yeniden başlatmak için *Başlat* butonuna tıklayın ve durdurmak için *Durdur* butonuna tıklayın.

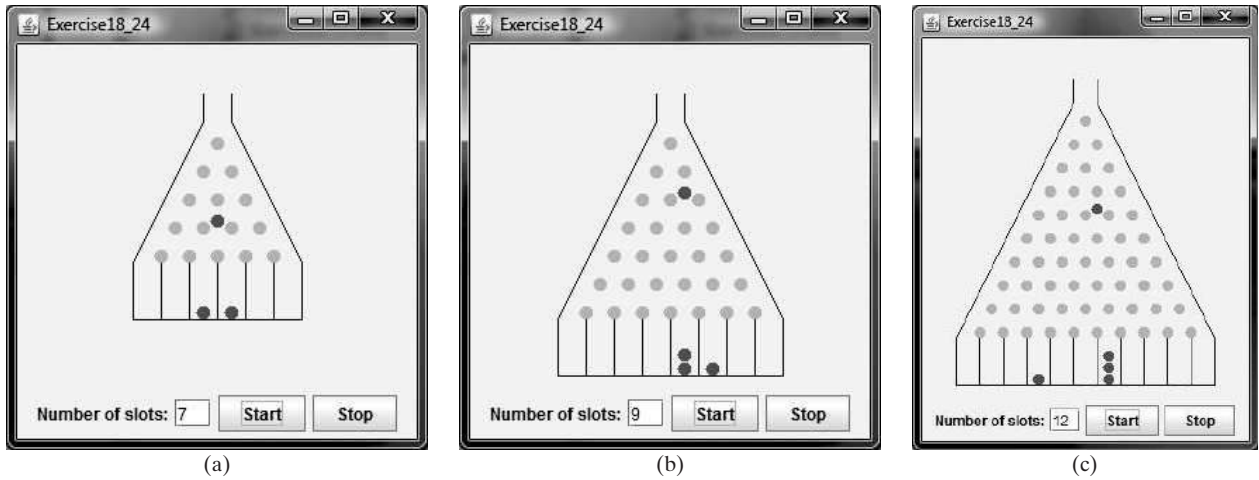


FIGURE 18.28 Applet, bir fasulye makinesi animasyonunu kontrol eder.

**\*\* 18.25** (Oyun: doğum günlerini tahmin et) Liste 3.3, *GuessBirthday.java*, bir doğum gününü tahmin etmek için bir program verir. Şekil 18.29’da gösterildiği gibi doğum günlerini tahmin etmek için bir applet oluşturun. Applet, kullanıcının tarihin beş setten herhangi birinde olup olmadığını kontrol etmesini ister. Tahmin et butonuna tıkladığında tarih metin alanında görüntülenir. Doğum Günü butonu.

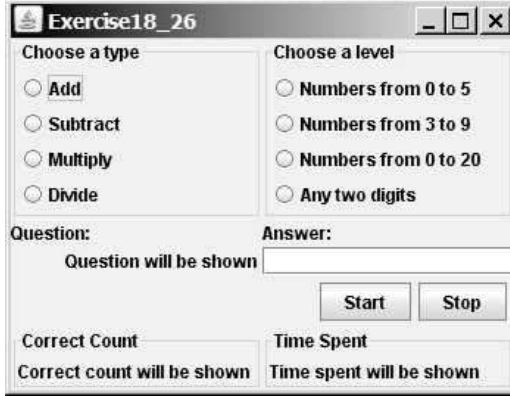
**\*\*\* 18.26** (Oyun: matematik sınavı) Liste 3.1, *AdditionQuiz.java*, ve Liste 3.4, *SubtractionQuiz.java*, matematik sınavları oluşturur ve puanlar. Kul



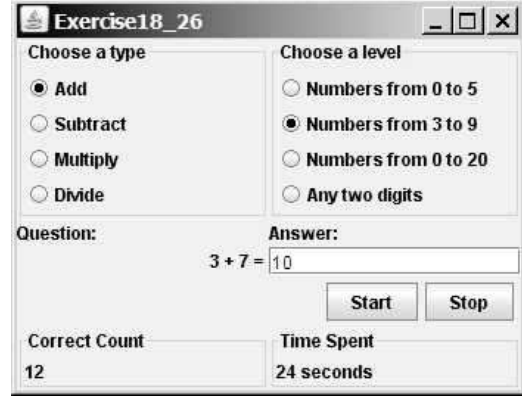


ŞEKİL 18.29 Bu applet doğum gününü tahmin eder.

kullanıcının bir soru türü ve zorluk seviyesi seçmesine olanak tanır, Şekil 18.30a'da gösterildiği gibi. Kullanıcı *Başlat* butonuna tıkladığında, program bir soru oluşturmaya başlar. Kullanıcı *Enter* tuşu ile bir cevap girdikten sonra yeni bir soru görüntülenir. Kullanıcı *Başlat* butonuna tıkladığında, geçen süre görüntülenir. Süre, *Durdur* butonuna tıklanana kadar her saniye güncellenir. Doğru cevap verildiğinde doğru sayısı güncellenir.



(a) Bir oturum başlamadan önce

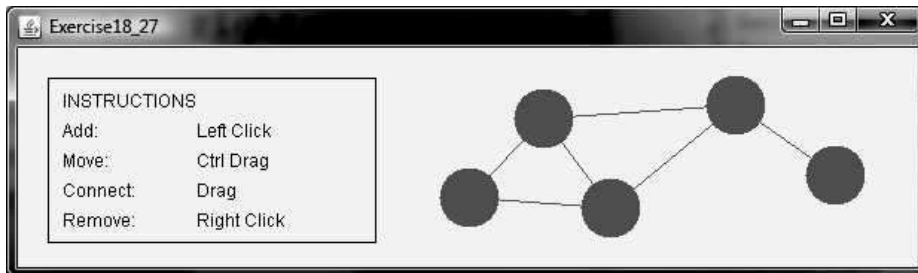


(b) Bir oturum başlatıldıktan sonra

ŞEKİL 18.30 Applet matematik becerilerini test eder.

### \*\*\* 18.27

(Grafikler) Bir grafik, köşeler ve köşeleri bağlayan kenarlardan oluşur. Kullanıcının köşeleri ve kenarları dinamik olarak çizibilmesini sağlayan bir program yazın, Şekil 18.31'de gösterildiği gibi. Her köşenin yarıçapı 20 pikseldir. Aşağıdaki işlevleri uygulayın: (1) Kullanıcı, fare noktasının içinde veya çok yakınında mevcut bir köşe olmadığından emin olarak, fare noktasında merkezlenmiş bir köşe yerleştirmek için sol fare düğmesine tıklar. (2) Kullanıcı, mevcut bir köşenin içinde sağ fare düğmesine tıklayarak köşeyi kaldırır. (3) Kullanıcı, bir köşenin içinde bir fare düğmesine basar ve başka bir köşeye sürükler ve ardından kenar oluşturmak için düğmeyi bırakır. (4) Kullanıcı, bir köşeyi taşımak için CTRL tuşuna basılı tutarak bir köşeyi sürükler.

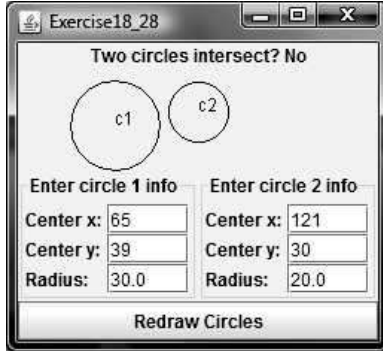


ŞEKİL 18.31 Applet, kullanıcıların dinamik olarak bir grafik çizmelerini sağlar.

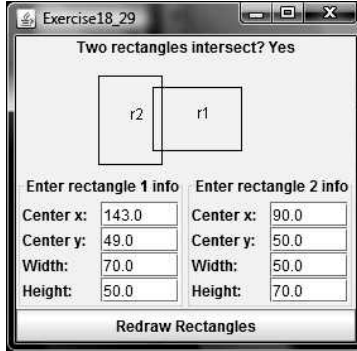
mevcut bir köşeye çok yakın. (2) Kullanıcı, mevcut bir köşenin içinde sağ fare düğmesine tıklayarak köşeyi kaldırır. (3) Kullanıcı, bir köşenin içinde bir fare düğmesine basar ve başka bir köşeye sürükler ve ardından kenar oluşturmak için düğmeyi bırakır. (4) Kullanıcı, bir köşeyi taşımak için **CTRL** tuşuna basılı tutarak bir köşeyi sürükler.

### \*\* 18.28

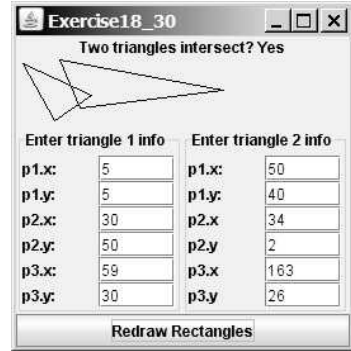
(Geometri: iki daire kesişiyor mu?) **Circle2D** sınıfı Programlama Egzersizi 10.11'de tanımlanmıştır. Kullanıcının dairelerin konumunu ve boyutunu belirtmesini sağlayan ve iki dairenin kesişip kesişmediğini gösteren bir applet yazın, Şekil 18.32a'da gösterildiği gibi. Kullanıcının bir dairenin içine fareyi işaretlemesine ve sürüklemesine izin verin. Daire sürüklendiğinde, dairenin merkez koordinatları metin alanlarında güncellenir.



(a)



(b)



(c)

ŞEKİL 18.32 İki dairenin, iki dikdörtgenin ve iki üçgenin örtüşüp örtüşmediğini kontrol edin.

### \*\* 18.29

(Geometri: iki dikdörtgen kesişiyor mu?) **MyRectangle2D** sınıfı Programlama Egzersizi 10.13'te tanımlandı. Kullanıcının dikdörtgenlerin konumunu ve boyutunu belirtmesine olanak tanıyan bir applet yazın ve iki dikdörtgenin kesişip kesişmediğini gösterin, Şekil 18.32b'de gösterildiği gibi. Kullanıcının bir dikdörtgenin içine fareyi işaretlemesine ve sürüklemesine izin verin. Dikdörtgen sürüklendiği sürede, dikdörtgenin merkez koordinatları metin alanlarında güncellenir.

### \*\* 18.30

(Geometri: iki üçgen kesişiyor mu?) **Triangle2D** sınıfı

Programlama Egzersizi 10.12'de tanımlandı. Kullanıcının iki üçgenin konumunu belirtmesini sağlayan ve iki üçgenin kesişip kesişmediğini gösteren bir applet yazın, Şekil 18.32c'de gösterildiği gibi.

### \* 18.31

(Sayıma başlayan kronometre) Şekil 18.33a'da gösterildiği gibi bir kronometre simüle eden bir applet yazın. Kullanıcı **Başlat** butonuna tıkladığında, butonun etiketi



(a)



(b)



(c)



(d)

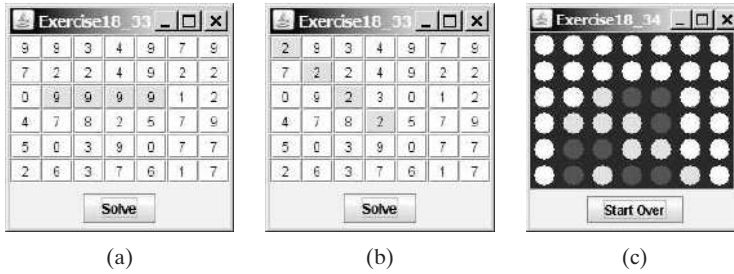
ŞEKİL 18.33 (a)–(c) Applet zamanı sayar. (d) Applet zamanı geri sayar.

şu şekilde değiştirildi: *Duraklat*, Şekil 18.33b'de gösterildiği gibi. Kullanıcı tıkladığında *Duraklatma* düğmesi, düğmenin etiketi *Devam* olarak değiştirilir, Şekil 18.33c'de gösterildiği gibi. *Temizle* düğmesi sayacı 0'a sıfırlar ve düğmenin etiketini *Başla* olarak sıfırlar.

### \*18.32 (Geri sayım kronometresi) Kullanıcının metin alanına

saniye cinsinden zaman girmesine ve dakikaları geri saymak için *Enter* tuşuna basmasına olanak tanıyan bir applet yazın, Şekil 18.33(d)'de gösterildiği gibi. Kalan saniyeler her bir saniyede bir yeniden görüntülenir. Dakikalar dolduğunda, program müziği kesintisiz çalmaya başlar.

\*\*18.33 (Desen tanıma: ardışık dört eşit sayı) Şekil 18.34a–b'de gösterildiği gibi, Programlama Egzersizi 7.19 için bir applet yazın. Kullanıcının 6 satır ve 7 sütundan oluşan bir ızgarada metin alanlarına sayıları girmesine izin verin. Kullanıcı, varsa dört eşit sayının bir dizisini vurgulamak için *Çöz* butonuna tıklayabilir.



**Fiçin 18.34**(a)–(b) Dört ardışık sayıyı vurgulamak için *Çöz* butonuna tıklama bir satırda, bir sütunda veya bir diyagonalda. (c) Applet, iki oyuncunun connect-four oyununu oynamasını sağlar.

\*\*\*18.34 (Oyun: dört bağla) Programlama Egzersizi 7.20, iki oyuncunun konsolda dört bağla oyun oynamasını sağlar. Programı, Şekil 18.34c'de gösterildiği gibi bir applet kullanarak yeniden yazın. Applet, iki oyuncunun sırayla kırmızı ve sarı diskler yerleştirmesine olanak tanır. Bir disk yerleştirmek için oyuncunun mevcut bir hücreye tıklaması gerekir. Bir *mevcut hücre* boş olup, aşağıdaki komşusu doludur. Applet, bir oyuncu kazandığında dört kazanan hücreyi yanıp söner ve tüm hücreler doluysa ve kazanan yoksa kazanan olmadığını bildirir.

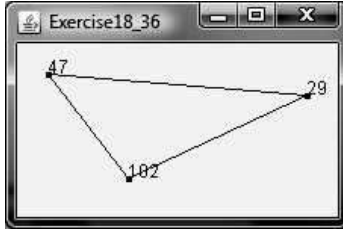
\*\*\*18.35 (Oyun: bilgisayarla dört bağla oyna) Bilgisayarla oynamak için Egzersiz 18.34'ü gözden geçirin. Program, kullanıcının önce bir hamle yapmasına, ardından bilgisayarın bir hamle yapmasına izin verir. Bilgisayarın yasal bir hamle yapması minimum gerekliliktir. Bilgisayarın akıllı hamleler yapması için iyi stratejiler tasarlamamız teşvik edilmektedir.

\*\*18.36 (Geometri: açıları göster) Kullanıcının bir üçgenin köşelerini sürüklemesine ve üçgen şekli değiştikçe açıları dinamik olarak göstermesine olanak tanıyan bir program yazın, Şekil 18.35a'da gösterildiği gibi. Fare, bir köşeye yakın hareket ettiğinde fare imlecini çapraz nişan şekline değiştirin. Açıları A, B ve C hesaplama için formüller aşağıdaki gibidir (bkz. Şekil 18.35b):

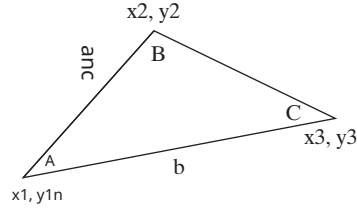
$$A = \text{Math.acos}((a^2 + b^2 - c^2) / (2 * a * b))$$

$$B = \text{Math.acos}((b^2 + c^2 - a^2) / (2 * b * c))$$

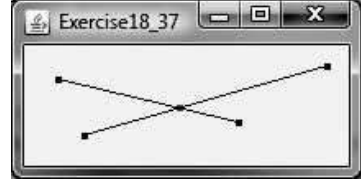
$$C = \text{Math.acos}((c^2 + a^2 - b^2) / (2 * c * a))$$



(a)



(b)



(c)

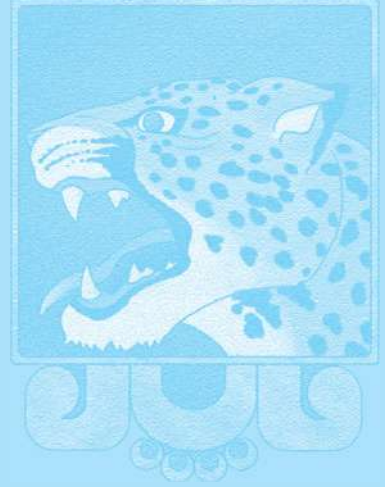
**FIGURE 18.35**(a–b) Exercise18.36, kullanıcının köşeleri sürüklemesine ve açıları dinamik olarak görüntülemesine olanak tanır. (c) Exercise18.37, kullanıcının köşeleri sürüklemesine ve çizgileri ile kesişim noktasını dinamik olarak görüntülemesine olanak tanır.

**\*\*18.37** (*Geometri: kesişim noktası*) İki doğru parçasını uç noktalarıyla birlikte gösteren bir program yazın, ve kesişim noktasını. Başlangıçta, uç noktalar (20,20) ve (56,130) birinci doğru için ve (100,20) ve (16,130) ikinci doğru için. Kullanıcı, bir noktayı sürüklemek için fareyi kullanabilir ve dinamik olarak gösterilen kesişim noktasını görüntüleyebilir, Şekil 18.35c'de gösterildiği gibi. *İpucu:* İki sınırsız doğrunun kesişim noktasını bulmak için ProgramlamaEgzersizi 3.25'e bakın.

## BtINARYt G/Ç

### Hedefler

- Java'da I/O'nun nasıl işlendiğini keşfetmek (§19.2).
- Metin I/O ve ikili I/O'yu ayırt etmek için (§19.3).
- `FileInputStream` ve `FileOutputStream` kullanarak bayt okumak ve yazmak için (§19.4.1).
- Verileri filtrelemek için temel sınıfları `FilterInputStream` ve `FilterOutputStream` kullanmak (§19.4.2).
- `DataInputStream` ve `DataOutputStream` kullanarak ilkel değerler ve dizgeler okumak ve yazmak için (§19.4.3).
- `BufferedInputStream` ve `BufferedOutputStream` kullanarak I/O performansını artırmak için (§19.4.4).
- Bir dosyayı kopyalayan bir program yazmak için (§19.5).
- `ObjectOutputStream` ve `ObjectInputStream` kullanarak nesneleri depolamak ve geri yüklemek için (§19.6).
- Nesneleri seri hale getirmek için `Serializable` arayüzünü uygulamak (§19.6.1).
- Dizileri seri hale getirmek için (§19.6.2).
- `RandomAccessFile` sınıfını kullanarak dosyaları okumak ve yazmak için (§19.7).



## 19.1 Giriş



*Java, metin G/Ç ve ikili G/Ç gerçekleştirmek için birçok sınıf sağlar.*

metin dosyası  
ikili dosya

Dosyalar metin veya ikili olarak sınıflandırılabilir. Notepad gibi bir metin düzenleyici kullanılarak işlenebilen (okunan, oluşturulan veya değiştirilen) bir dosyaya *metin dosyası* denir.

Diğer tüm dosyalara *ikili dosyalar* denir. İkili dosyaları bir metin düzenleyici kullanarak okuyamazsınız—bunlar programlar tarafından okunmak üzere tasarlanmıştır. Örneğin, Java kaynak programları metin dosyalarında saklanır ve bir metin düzenleyici tarafından okunabilir, ancak Java sınıf dosyaları ikili dosyalarda saklanır ve JVM tarafından okunur.

neden ikili G/Ç?

Teknik olarak kesin ve doğru olmasa da, bir metin dosyasını karakterlerin bir dizisi ve bir ikili dosyayı bitlerin bir dizisi olarak düşünebilirsiniz. Bir metin dosyasındaki karakterler, ASCII veya Unicode gibi bir karakter kodlama şeması kullanılarak kodlanır. Örneğin, ondalık tam sayı 199, bir metin dosyasında 1,9,9 karakterlerinin dizisi olarak saklanır ve aynı tam sayı, bir ikili dosyada C7 byte türünde bir değer olarak saklanır, çünkü ondalık 199, onaltılık C7'ye eşittir. (199=12×16<sup>1</sup>+7). İkili dosyaların avantajı, metin dosyalarından daha verimli işlenmeleridir.

metin G/Ç  
ikili G/Ç

Java, dosya girişi ve çıkışı gerçekleştirmek için birçok sınıf sunar. Bunlar, metin G/Ç sınıfları ve ikili G/Ç sınıfları olarak kategorize edilebilir. Bölüm 14.11, Dosya Girişi ve Çıkışı'nda, **Scanner** ve **PrintWriter** kullanarak bir metin dosyasından/ona dize ve sayısal değerler okuma ve yazma yöntemlerini öğrendiniz. Bu bölüm, ikili G/Ç gerçekleştiren sınıfları tanıtmaktadır.

## 19.2 Java'da Metin G/Ç Nasıl İşlenir?



*Metin verileri, **Scanner** sınıfı kullanılarak okunur ve **PrintWriter** sınıfı kullanılarak yazılır.*

Bir **File** nesnesinin bir dosyanın veya bir yolun özelliklerini kapsadığını ancak bir dosyadan/ona veri okuma/yazma yöntemlerini içermediğini hatırlayın. G/Ç gerçekleştirmek için, uygun Java G/Ç sınıflarını kullanarak nesneler oluşturmanız gerekir. Bu nesneler, bir dosyadan/ona veri okuma/yazma yöntemlerini içerir. Örneğin, **temp.txt** adlı bir dosyaya metin yazmak için, **PrintWriter** sınıfını kullanarak bir nesne oluşturabilirsiniz:

```
PrintWriter çıktı = new PrintWriter("temp.txt");
```

Artık nesne üzerinde **print** yöntemini çağırarak bir dizeyi dosyaya yazabilirsiniz. Örneğin, daha sonraki ifade **Java 101**'yi dosyaya yazar.

```
çıktı.print("Java 101");
```

Sonraki ifade dosyayı kapatır.

```
çıktı.close();
```

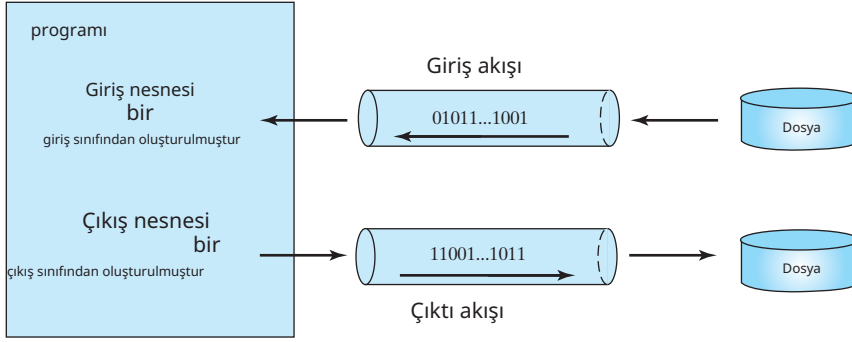
Çeşitli amaçlar için birçok G/Ç sınıfı vardır. Genel olarak, bunlar giriş sınıfları ve çıkış sınıfları olarak sınıflandırılabilir. Bir *giriş sınıfı* verileri okumak için yöntemleri içerir ve bir *çıkış sınıfı* verileri yazmak için yöntemleri içerir. **PrintWriter** bir çıkış sınıfı örneğidir ve **Scanner** bir giriş sınıfı örneğidir. Aşağıdaki kod, **temp.txt** dosyası için bir giriş nesnesi oluşturur ve dosyadan veri okur.

```
Tarayıcı girişi = tnewtScanner(tnewtFile("temp.txt"));nSystem.out.println(input.nextLine());
```

Eğer **temp.txt** dosyası **Java 101** metnini içeriyorsa, **input.nextLine()** dizesini **"Java 101"** olarak döndürür.

Şekil 19.1, Java G/Ç programlamasını göstermektedir. Bir giriş nesnesi, bir dosyadan bir akış veri okur ve bir çıkış nesnesi, bir dosyaya bir veri akışı yazar. Bir giriş nesnesine ayrıca **input stream** ve bir çıkış nesnesine *output stream* denir.

akış  
giriş akışı  
çıkış akışı



**FIGİN 19.1** Program, bir girdi nesnesi aracılığıyla veri alır ve bir

çıkış nesnesi aracılığıyla veri gönderir.

**19.1** Metin dosyası nedir ve ikili dosya nedir? Bir metin dosyasını veya bir ikili dosyayı

metin düzenleyici kullanarak görebilir misiniz?

**19.2** Java'da metin verilerini nasıl okur veya yazarız? Akış nedir?

Point/Kontrol

MyProgrammingLab™

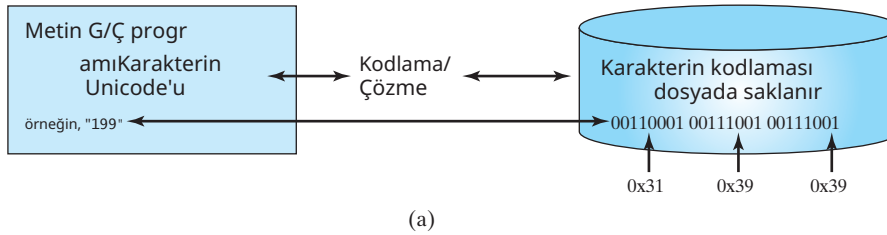
## 19.3 Metin G/Ç vs. İkili G/Ç

*İkili G/Ç, kodlama veya kod çözme işlemi içermez ve bu nedenle metin G/Ç'den daha etkilidir.*

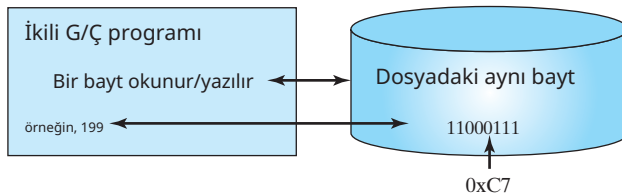
Bilgisayarlar ikili dosyalar ile metin dosyaları arasında ayrım yapmaz. Tüm dosyalar ikiliformatta saklanır ve bu nedenle tüm dosyalar esasen ikili dosyalardır. Metin G/Ç, karakter kodlama ve kod çözme için bir soyutlama düzeyi sağlamak amacıyla ikili G/Ç üzerine inşa edilmiştir; bu, Şekil 19.2a'da gösterilmiştir.

Kodlama ve kod çözme, metin G/Ç için otomatik olarak gerçekleştirilir. JVM, bir karakter yazarken Unicode'ı bir dosyaya özgü kodlamaya dönüştürür ve bir karakter okurken dosyaya özgü kodlamayı Unicode'a dönüştürür. Örneğin, metin G/Ç kullanarak bir dosyaya "199" dizesini yazdığınızı varsayalım. Her karakter dosyaya yazılır. Karakter 1 için Unicode 0x0031, Unicode 0x0031, dosyanın kodlama şemasına bağlı olarak bir koda dönüştürülür.

(not: 0x ön eki bir onaltılık sayıyı belirtir.) Amerika Birleşik Devletleri'nde, Windows'taki metin dosyaları için varsayılan kodlama ASCII'dir. Karakter 1 için ASCII kodu 49'dur (0x31 içinde)



(a)



(b)

**ŞEKİL 19.2** Metin G/Ç, kodlama ve kod çözme gerektirirken, ikili G/Ç gerektirmez.



hex) ve karakter için 9 57 olarak(0x39 hex cinsinden). Bu nedenle, karakterleri yazmak için 199, üç bayt—0x31,0x39 ve 0x39—çıktıya gönderilir, Şekil 19.2a'da gösterildiği gibi.

İkili G/Ç dönüşümler gerektirmez. Eğer bir sayısal değeri bir dosyaya ikili G/Ç kullanarak yazarsanız, bellek içindeki tam değer dosyaya kopyalanır. Örneğin, bir bayt türündeki değer 199 şu şekilde temsil edilir: 0xC7 ( $199 = 12 \times 16^1 + 7$ ) bellekte ve dosyada tam olarak 0xC7 olarak görünür, Şekil 19.2b'de gösterildiği gibi. İkili G/Ç kullanarak bir bayt okuduğunuzda, bir bayt değeri girişten okunur.

Genel olarak, bir metin düzenleyici veya bir metin çıktısı

programı tarafından oluşturulan bir dosyayı okumak için metin girişi kullanmalı ve bir Java ikili çıktı programı tarafından oluşturulan bir dosyayı okumak için ikili girişi kullanmalıdır.

İkili G/Ç, metin G/Ç'den daha verimlidir çünkü ikili G/Ç kodlama ve çözümleme gerektirmez. İkili dosyalar, ana makinedeki kodlama şemasından bağımsızdır ve bu nedenle taşınabilir. Herhangi bir makinedeki Java programları, bir Java programı tarafından oluşturulan bir ikili dosyayı okuyabilir. Bu nedenle Java sınıf dosyaları ikili dosyalardır. Java sınıf dosyaları, herhangi bir makinedeki bir JVM üzerinde çalışabilir.



### Not

Tutarlılık açısından, bu kitap metin dosyalarını adlandırmak için uzantı .txt ve ikili dosyaları adlandırmak için .dat kullanmaktadır.

.txt ve .dat

### 19.3 Metin G/Ç ile ikili G/Ç arasındaki farklar nelerdir?

**19.4** Bir Java karakteri bellekte nasıl temsil edilir ve bir karakter bir metin dosyasında nasıl temsil edilir?

**19.5** Eğer "ABC" dizesini bir ASCII metin dosyasına yazarsanız, dosyada hangi değerler saklanır?

**19.6** Eğer "100" dizesini bir ASCII metin dosyasına yazarsanız, dosyada hangi değerler saklanır? Bir sayısal byte türü değeri 100 ikili G/Ç kullanarak yazarsanız, dosyada hangi değerler saklanır?

**19.7** Bir Java programında bir karakteri temsil etmek için kullanılan kodlama şeması nedir? Varsayılan olarak, Windows'taki bir metin dosyası için kodlama şeması nedir?

Point/Kontrol

MyProgrammingLab™

## 19.4 İkili G/Ç Sınıfları

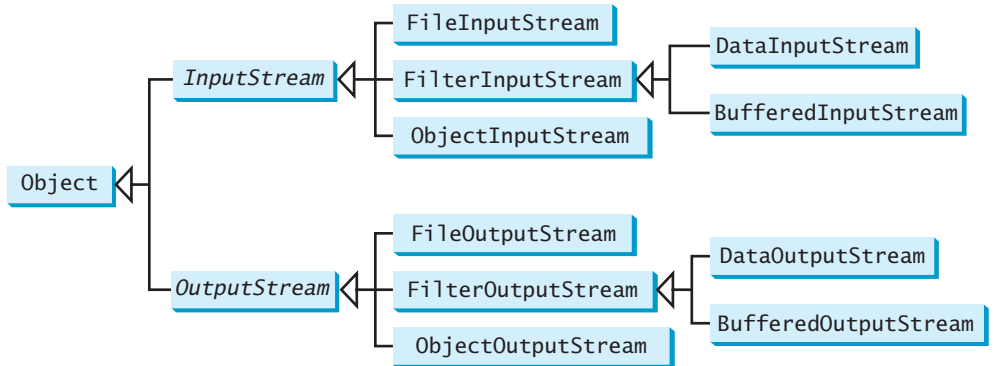


Keyn  
Point

*Soyut **InputStream**, ikili verileri okumak için kök sınıf ve soyut **OutputStream**, ikili verileri yazmak için kök sınıftır.*

Java I/O sınıflarının tasarımı, ortak işlemlerin süper sınıflarda genelleştirildiği ve alt sınıfların özel işlemler sağladığı kalıtımın uygulanmasına iyi bir örnektir.

Şekil 19.3, ikili I/O gerçekleştiren bazı sınıfları listelemektedir. **InputStream**,



ŞEKİL 19.3 **InputStream**, **OutputStream** ve alt sınıfları, ikili I/O gerçekleştirmek için

kullanılmaktadır.



<i>java.io.InputStream</i>	
<b>+read(): int</b>	Reads the next byte of data from the input stream. The value byte is returned as an <code>int</code> value in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned.
<b>+read(b: byte[]): int</b>	Reads up to <code>b.length</code> bytes into array <code>b</code> from the input stream and returns the actual number of bytes read. Returns -1 at the end of the stream.
<b>+read(b: byte[], off: int, len: int): int</b>	Reads bytes from the input stream and stores them in <code>b[off]</code> , <code>b[off+1]</code> , . . . , <code>b[off+len-1]</code> . The actual number of bytes read is returned. Returns -1 at the end of the stream.
<b>+available(): int</b>	Returns an estimate of the number of bytes that can be read from the input stream.
<b>+close(): void</b>	Closes this input stream and releases any system resources occupied by it.
<b>+skip(n: long): long</b>	Skips over and discards <code>n</code> bytes of data from this input stream. The actual number of bytes skipped is returned.
<b>+markSupported(): boolean</b>	Tests whether this input stream supports the <code>mark</code> and <code>reset</code> methods.
<b>+mark(readLimit: int): void</b>	Marks the current position in this input stream.
<b>+reset(): void</b>	Repositions this stream to the position at the time the <code>mark</code> method was last called on this input stream.

ŞEKİL 19.4 Soyut **InputStream** sınıfı, baytların giriş akışı için yöntemleri tanımlar.

İkili giriş sınıfları ve **OutputStream**, ikili çıkış sınıfları için kök sınıftır. Şekil 19.4 ve 19.5, **InputStream** ve **OutputStream** sınıflarındaki tüm yöntemleri listelemektedir.



### Not

İkili I/O sınıflarındaki tüm yöntemler, **java.io.IOException**

veya **java.io.IOException** alt sınıfını fırlatacak şekilde tanımlanmıştır.

IOException fırlatır

<i>java.io.OutputStream</i>	
<b>+write(int b): void</b>	Writes the specified byte to this output stream. The parameter <code>b</code> is an <code>int</code> value. (byte) <code>b</code> is written to the output stream.
<b>+write(b: byte[]): void</b>	Writes all the bytes in array <code>b</code> to the output stream.
<b>+write(b: byte[], off: int, len: int): void</b>	Writes <code>b[off]</code> , <code>b[off+1]</code> , . . . , <code>b[off+len-1]</code> into the output stream.
<b>+close(): void</b>	Closes this output stream and releases any system resources occupied by it.
<b>+flush(): void</b>	Flushes this output stream and forces any buffered output bytes to be written out.

ŞEKİL 19.5 Soyut **OutputStream** sınıfı, baytların çıkış akışı için yöntemleri tanımlar.

## 19.4.1 FileInputStream/tFileOutputStream

**FileInputStream/FileOutputStream**, dosyalardan/bir dosyaya bayt okumak/yazmak için kullanılır. Bu sınıflardaki tüm metotlar **InputStream** ve **OutputStream**'dan miras alınmıştır.

**FileInputStream/FileOutputStream** yeni metotlar tanıtmaz. Bir

**FileInputStream** oluşturmak için, Şekil 19.6'da gösterilen yapıcılarını kullanın.

A **java.io.FileNotFoundException**, mevcut olmayan bir dosya ile bir

**FileInputStream** oluşturmaya çalıştığınızda meydana gelecektir.

FileNotFoundException

Bir **FileOutputStream** oluşturmak için, Şekil 19.7'de gösterilen yapıcılarını kullanın.

Dosya mevcut değilse, yeni bir dosya oluşturulacaktır. Dosya zaten mevcutsa, ilk iki yapıcı dosyanın mevcut içeriğini silecektir. Mevcut içeriği korumak ve dosyaya yeni veriler eklemek için, son iki yapıcıcıyı kullanın ve **true** değerini **append** parametresine geçirin.

## 714 Bölüm 19 İkili G/Ç

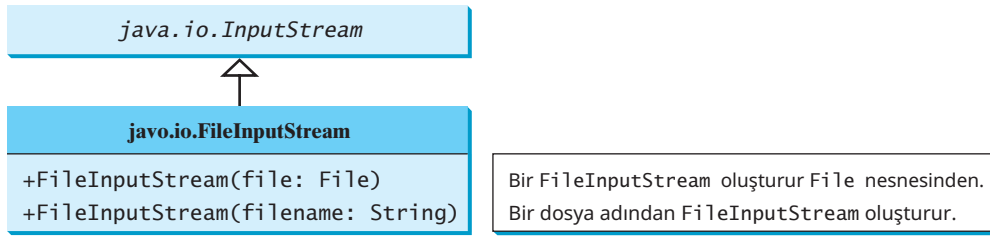


FIGURE 19.6 `FileInputStream` bir dosyadan byte akışı alır.

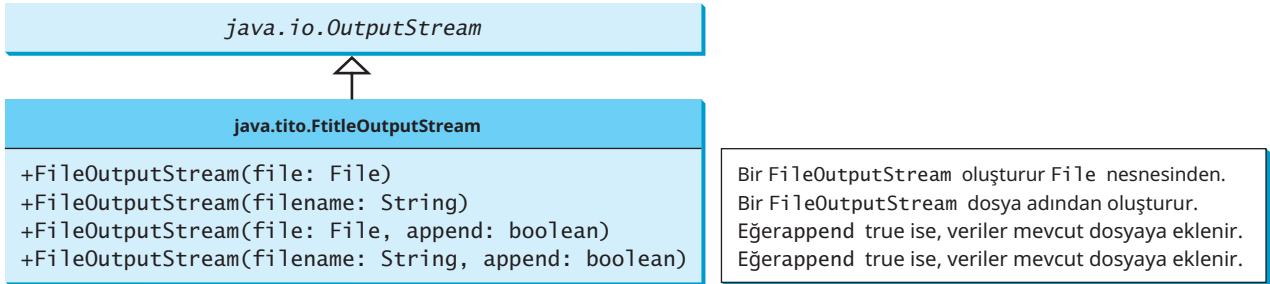


FIGURE 19.7 `FileOutputStream` bir dosyaya byte akışı çıktısı verir.

IO İstisnası

İ/O sınıflarındaki neredeyse tüm yöntemler `java.io.IOException` fırlatır. Bu nedenle, yöntemde `java.io.IOException` fırlatmak için beyan etmeniz veya kodu aşağıda gösterildiği gibi bir try-catch bloğuna yerleştirmeniz gerekir:

Metodda istisna bildirme

```

public static void main(String[] args)
 throws IOException {
 // G/Ç işlemlerini gerçekleştir
}

```

Try-catch bloğu kullanma

```

public static void main(String[] args) {
 denemek
 // G/Ç işlemlerini gerçekleştir
}
catch (IOException ex) {
 ex.printStackTrace();
}

```

Liste 19.1, 1 ile 10 arasındaki on bayt değerini `temp.dat` adlı bir dosyaya yazmak için ikili I/O kullanır ve bunları dosyadan geri okur.

### LISTING 19.1 `TestFileStream.java`

içe aktarmak

çıkış akışı

çıkış

```

1 import java.io.*;
2
3 genel sınıf TestFileStream {
4 public static void main(String[] args) throws Exception {
5 // Dosyaya bir çıktı akışı oluştur
6 FileOutputStream çıkış = new FileOutputStream("temp.dat");
7 // Değerleri dosyaya yaz
8 for (int i = 1; i <= 10; i++)
9 çıkış.yaz(i);
10 // Çıktı akışını kapat
11 çıkış.close();
12
13 }
14 }

```

```

15 // Dosya için bir girdi akışı oluştur
16 FileInputStream girdi = new FileInputStream("temp.dat");
17
18 // Dosyadan değerleri oku
19 int değer;
20 while((değer = girdi.read()) != -1)
21 System.out.print(değer + " ");
22
23 // Çıktı akışını kapat
24 girdi.close();
25 }

```

girdi akışı

girdi

1 2 3 4 5 6 7 8 9 10



A **FileOutputStream**, 6. satırda **temp.dat** dosyası için oluşturulur. **for** döngüsü, dosyaya onbayt değeri yazar (9-10. satırlar). **write(i)** çağrısı, **write((byte)i)** çağrısı yla aynıdır. 13. satır çıkış akışını kapatır. 16. satır **FileInputStream** oluşturur **temp.dat** dosyası için. Değerler dosyadan okunur ve konsolda 19-21. satırlarda görüntülenir. **((value = input.read()) != -1)** ifadesi (20. satır) bir baytı **input.read()**'de n okur, **value** ya atar ve bunun **-1** olup olmadığını kontrol eder. **-1** giriş değeri, bir dosyanın sonunu belirtir.

bir dosyanın sonu

Bu örnekte oluşturulan **temp.dat** dosyası bir ikili dosyadır. Bir Java programından okunabilir, ancak bir metin düzenleyicisinden okunamaz; Şekil 19.8'de gösterildiği gibi.



**FIGON 19.8** Bir ikili dosya metin modunda görüntülenemez.



#### İpucu

Bir akış artık gerekli olmadığında, her zaman **close()** yöntemini kullanarak kapatın. Akışları kapatmamak, çıkış dosyasında veri bozulmasına veya diğer programlama hatalarına neden olabilir.

akışı kapat



#### Not

Dosyanın kök dizini classpath dizinidir. Bu kitapta verilen örnek için kök dizin **c:\book\dur**, bu nedenle **temp.dat** dosyası **c:\book\dur**'da bulunmaktadır. **temp.dat**'ı belirli bir dizine yerleştirmek istiyorsanız, 6. satırı şu şekilde değiştirin:

dosya nerede?

```

FileOutputStream çıkış =
 new FileOutputStream ("dizin/temp.dat");

```



#### Not

Bir **FileInputStream** örneği, bir Scanner oluşturmak için argüman olarak kullanılabilir ve bir **FileOutputStream** örneği, bir **PrintWriter** oluşturmak için argüman olarak kullanılabilir. Bir dosyaya metin eklemek için bir **PrintWriter** oluşturabilirsiniz

metin dosyasına ekleme

```
yeniYazıcı(yeniDosyaÇıktıAkışı("temp.txt", true));
```

Eğer `temp.txt` mevcut değilse, oluşturulur. Eğer `temp.txt` zaten mevcutsa, yeni veriler dosyaya eklenir.

### 19.4.2 FiltreGirdiAkışı/FiltreÇıktıAkışı

*Filtre akışları*, bazı amaçlar için baytları filtreleyen akışlardır. Temel bayt girdi akışı, yalnızca bayt okumak için kullanılabilen bir `okuma` yöntemini sağlar. Eğer tamsayı, ondalıklı sayı veya dizeleri okumak istiyorsanız, bayt girdi akışını saran bir filtre sınıfına ihtiyacınız vardır. Bir filtre sınıfı kullanırsanız, baytlar ve karakterler yerine tamsayı, ondalıklı sayı ve dizeleri okumanızı sağlar.

`FilterInputStream` ve `FilterOutputStream` veri filtreleme için temel sınıflardır. Temel sayısal türleri işlemek gerektiğinde, `DataInputStream` ve

`DataOutputStream` kullanarak baytları filtreleyin.

### 19.4.3 DataInputStream/t DataOutputStream

`DataInputStream` akıştan baytları okur ve bunları uygun temel tür değerlerine veya dizelere dönüştürür. `DataOutputStream` temel tür değerlerini veya dizeleri baytlara dönüştürür ve baytları akışa çıkarır.

`DataInputStream` `FilterInputStream` sınıfından türetilir ve `DataInput` arayüzünü uygular, Şekil 19.9'da gösterildiği gibi. `DataOutputStream` `FilterOutputStream` sınıfından türetilir ve `DataOutput` arayüzünü uygular, Şekil 19.10'da gösterildiği gibi.

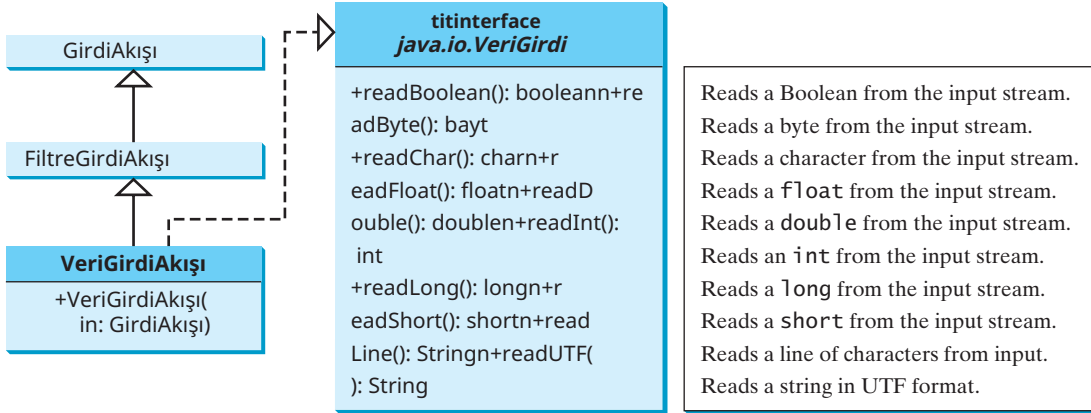
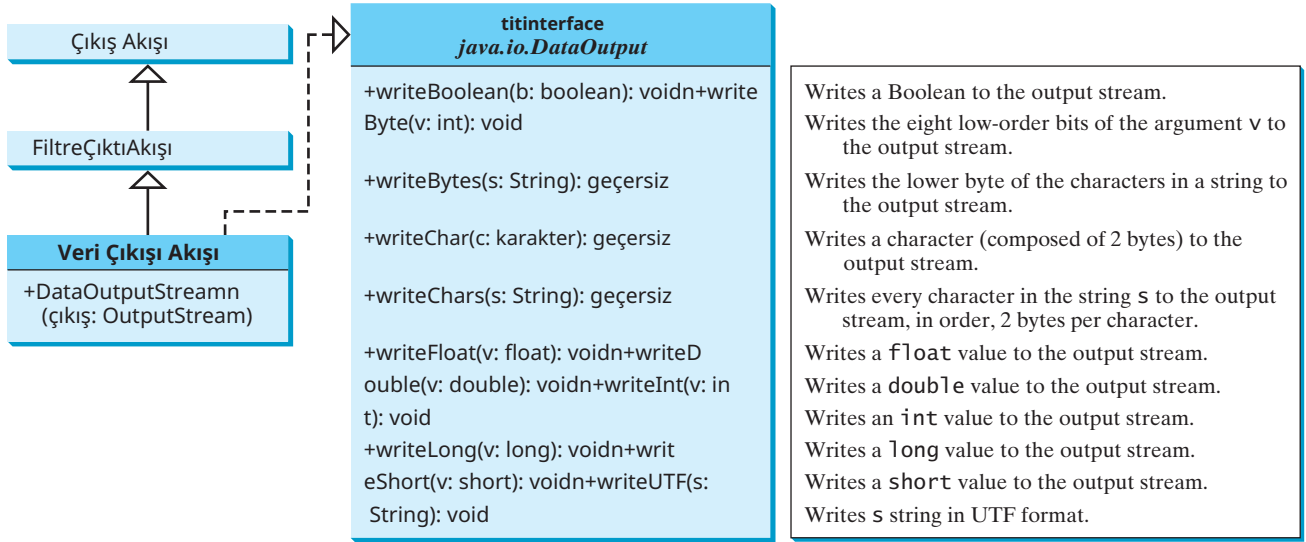


FIGURE 19.9 VeriGirdiAkışı bir byte girdi akışını ilkel veri türü değerlerine ve stringlere filtreler.

`DataInputStream`, temel veri türü değerlerini ve dizeleri okumak için `DataInput` arayüzünde tanımlanan yöntemleri uygular. `DataOutputStream`, temel veri türü değerlerini ve dizeleri yazmak için `DataOutput` arayüzünde tanımlanan yöntemleri uygular. Temel değerler bellekten çıktıya herhangi bir dönüşüm olmadan kopyalanır. Bir dizideki karakterler, birçok şekilde yazılabilir, bir sonraki bölümde tartışıldığı gibi.

### İkili G/Ç'de Karakterler ve Dizeler

Bir Unicode karakteri iki bayttan oluşur. `writeChar(char c)` yöntemi, karakteri `c` Unicode'unu çıktıya yazar. `writeChars(String s)` yöntemi, dizideki her karakterin Unicode'unu `s` çıktıya yazar. `writeBytes(String s)` yöntemi, dizideki her karakterin Unicode'unun alt baytını `s` çıktıya yazar. Unicode'un yüksek baytı atılır. `writeBytes` yöntemi, yalnızca



**Fiçin 19.10 DataOutputStream**, ilkel veri türü değerlerini ve dizeleri bir çıkış akışına yazmanıza olanak tanır.

ASCII karakterlerinden oluşur, çünkü bir ASCII kodu yalnızca bir Unicode'un alt baytında saklanır. Eğer bir string, ASCII olmayan karakterlerden oluşuyorsa, diziyi yazmak için `writeChars` yöntemini kullanmalısınız.

`writeUTF(String s)` yöntemi, çıkış akışına iki bayt uzunluk bilgisi yazar, daha sonra dizideki her karakterin değiştirilmiş UTF-8 temsilini takip eder. `string`. UTF-8, sistemlerin hem ASCII hem de

Unicode ile çalışmasına olanak tanıyan bir kodlama şemasıdır. Çoğu işletim sistemi ASCII kullanır. Java Unicode kullanır. ASCII karakter seti, Unicode karakter setinin bir alt kümesidir. Çoğu uygulama yalnızca ASCII karakter setine ihtiyaç duyduğundan, 8 bitlik bir ASCII karakterini 16 bitlik bir Unicode karakteri olarak temsil etmek israf olur. Değiştirilmiş UTF-8 şeması, bir karakteri bir, iki veya üç bayt kullanarak saklar. Karakterler, kodları `0x7F` veya daha düşüğe bir baytta, `0x7F`' den büyük ve `0x7FF`' den küçüğe iki baytta, veya `0x7FF`' den büyükse üç baytta kodlanır.

Bir UTF-8 karakterinin ilk bitleri, bir karakterin bir baytta, iki baytta veya üç baytta depolanıp depolanmadığını gösterir. İlk bit `0` ise, bu bir baytlık bir karakterdir. İlk bitler `110` ise, bu iki baytlık bir dizinin ilk baytıdır. İlk bitler `1110` ise, bu üç baytlık bir dizinin ilk baytıdır. Bir dizedeki karakter sayısını gösteren bilgi, UTF-8 karakterlerinden önceki ilk iki baytta depolanır. Örneğin, `writeUTF("ABCDEF")` aslında dosya sekiz bayt (yani, `00 06 41 42 43 44 45 46`) yazar, çünkü ilk iki bayt dizedeki karakter sayısını depolar.

UTF-8 şeması

`writeUTF(String s)` metodu, bir diziyi UTF-8 formatında bir dizi bayta dönüştürür ve bunları bir çıkış akışına yazar. `readUTF()` metodu, `writeUTF` metodu kullanılarak yazılmış bir diziyi okur.

UTF-8 formatı, her ASCII karakteri için bir bayt tasarrufu sağlama avantajına sahiptir, çünkü bir Unicode karakteri iki bayt yer kaplarken, UTF-8'de bir ASCII karakteri yalnızca bir bayt yer kaplar. Uzun bir dizedeki karakterlerin çoğu normal ASCII karakterleri ise, UTF-8 kullanmak daha verimlidir.

## Veri Girişi Akışı/Veri Çıkışı Akışı Oluşturma

Veri Girişi Akışı/Veri Çıkışı Akışı aşağıdaki yapıcılar kullanılarak oluşturulur

(Şekil 19.9 ve 19.10'a bakın):

```
publicDataInputStream(InputStream akış içi)
publicDataOutputStream(OutputStream çıkış akışı)
```

Aşağıdaki ifadeler veri akışları oluşturur. İlk ifade, **in.dat** dosyası için bir giriş akışı oluşturur; ikinci ifade, **out.dat** dosyası için bir çıkış akışı oluşturur.

```
DataInputStream girişi =
 newDataInputStream (tnewtFileInputStream(t"in.dat");nData
OutputStream çıkışı =
 yeniDataOutputStreamn (tnewtFileOutputStream(t"out.dat");
```

Liste 19.2, öğrenci isimlerini ve notlarını **temp.dat** adlı bir dosyaya yazar ve verileri dosyadan geri okur.

### LISTE 19.2 TestDataStream.java

```
1 import java.io.*;
2
3 genel sınıfTestDataStream {
4 public static voidtmain(String[] args) tthrowstIOException {
5 // temp.dat dosyası için bir çıktı akışı oluşturun
6 OutputStream çıkışı =
7 newDataOutputStream(tnewtFileOutputStream(t"temp.dat");nData
8
9
10 // Öğrenci test puanlarını dosyaya yazın
11 tput.writeUTF(t"John");
12 Output.writeDouble(t85.5t);nou
13 çıktı.writeUTF(t"Susan");
14 çıktı.writeDouble(t185.5t);n14 çıktı.write
15 UTF(t"Kim");n15 çıktı.writeDouble(t105.
16 25t);n16
17
18 // çıktı akışını kapatın
19 çıktı.close();
20
21 // temp.dat dosyası için bir girdi akışı oluşturun
22 putStream girdi =
23 newDataInputStream(tnewtFileInputStream(t"temp.dat");nDataIn
24
25 // Öğrenci test puanlarını dosyadan okuyun
26 System.out.println(input.readUTF() + " "t+ input.readDouble());n
27 System.out.println(input.readUTF() + t" "t+ input.readDouble());n27 System.out.println(in
28 put.readUTF() + t" "t+ input.readDouble());n28 }
29 }
```

çıkış akışı

çıkış

akışı kapat

girdi akışı

girdi



```
John 85.5n
Susan 185.5n
Kim 105.25
```

**DataOutputStream**, 6-7. satırlarda **temp.dat** dosyası için oluşturulur. Öğrenci isimleri ve notları 10-15. satırlarda dosyaya yazılır. 18. satır çıkış akışını kapatır. Aynı dosya için 21-22. satırlarda bir **DataInputStream** oluşturulur. Öğrenci isimleri ve notları dosyadan okunur ve 25-27. satırlarda konsolda görüntülenir.

**DataInputStream** ve **DataOutputStream**, Java ilkel tür değerlerini ve dizeleri makineden bağımsız bir şekilde okur ve yazar, böylece bir makinede bir veri dosyası yazmanıza ve farklı bir işletim sistemi veya dosya yapısına sahip başka bir makinede okumanıza olanak tanır. Bir uygulama, daha sonra bir veri girişi akışı kullanarak okunabilecek verileri yazmak için bir veri çıkışı akışı kullanır.



### Dikkat

Verileri, saklandıkları aynı sırada ve formatta okumalısınız. Örneğin, isimler **writeUTF** kullanılarak UTF-8 formatında yazıldığı için, isimleri **readUTF** kullanarak okum alısınız.

#### Bir Dosyanın Sonunu Tespit Etme

Bir **InputStream**'in sonunda verileri okumaya devam ederseniz, bir **EOFException** oluşacaktır. Bu istisna, bir dosyanın sonunu tespit etmek için kullanılabilir; bu, Liste 19.3'te gösterilmiştir. EOFException

### LISTE 19.3 DetectEndOfFile.java

```

1 import java.io.*;
2
3 genel classtDetectEndOfFile {
4 public static voidtmain(String[] args) {
5 denemek
6 DataOutputStream çıktı =
7 newDataOutputStream(tnewtFileOutputStream(t"test.dat"t));
8 çıktı.writeDouble(4.5);
9 çıktı.writeDouble(43.25);
10 çıktı.writeDouble(3.2);
11 çıktı.close();
12
13 DataInputStream girdi =
14 newtDataInputStream(tnewtFileInputStream(t"test.dat"t));
15 while(doğru) {
16 System.out.println();n17 }
17 input.readDouble()
18 }
19 catcht(EOFException ex) {
20 System.out.println("Tüm veriler okundu");
21 }
22 catch (IOException ex) {
23 ex.printStackTrace();n24 }
25 }n26 }

```

çıkış akışı  
çıkış  
akışı kapat  
giriş akışı  
girdi  
EOFException

```

4.5
43.25
3.2
Tüm veriler okundu

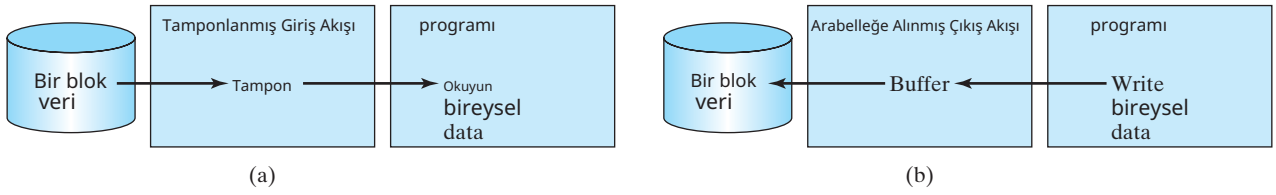
```



Program, dosyaya üç çift değer yazarak **DataOutputStream** (satırlar 6–10) kullanır ve verileri **DataInputStream** (satırlar 13–14) kullanarak okur. Dosyanın sonundan okuma yapıldığında, bir **EOFException** fırlatılır. İstisna, 19. satırda yakalanır.

#### 19.4.4 BufferedInputStreamt/BufferedOutputStream

**BufferedInputStream/BufferedOutputStream**, disk okuma ve yazma sayısını azaltarak girdi ve çıktıyı hızlandırmak için kullanılabilir. **BufferedInputStream** kullanarak, diskteki tüm veri bloğu bir kez bellekteki tampon içine okunur. Bireysel veriler daha sonra tampondan programınıza iletilir, Şekil 19.11a'da gösterildiği gibi. **BufferedOutputStream** kullanarak, bireysel veriler önce bellekteki tampona yazılır. Tampon dolduğunda, tampondaki tüm veriler bir kez diske yazılır, Şekil 19.11b'de gösterildiği gibi.



FIGİN19.11 Tampon G/Ç, verileri hızlı işleme için bir tamponda saklar.

**BufferedInputStream/BufferedOutputStream** yeni yöntemler içermez. Tüm yöntemler **BufferedInputStream/BufferedOutputStream** sınıflarından miras alınmıştır. **BufferedInputStream/BufferedOutputStream** bir tamponu sahne arkasında yönetir ve otomatik olarak verileri diskten okur/yazar.

Bir **BufferedInputStream/BufferedOutputStream** on herhangi bir **InputStream/OutputStream** kullanarak, Şekil 19.12 ve 19.13'te gösterilen yapıları kullanabilirsiniz.

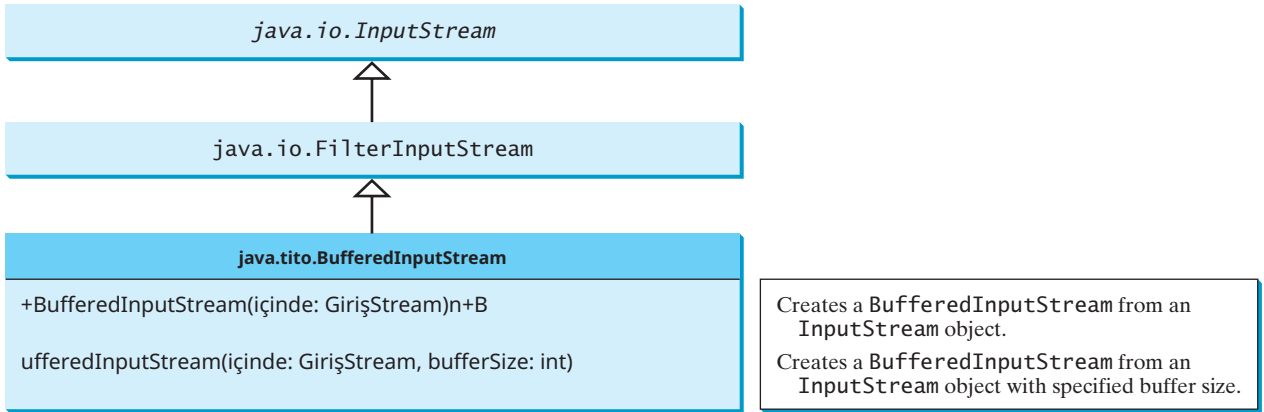


FIGURE 19.12 **BufferedInputStream** bir girdi akışını tamponlar.

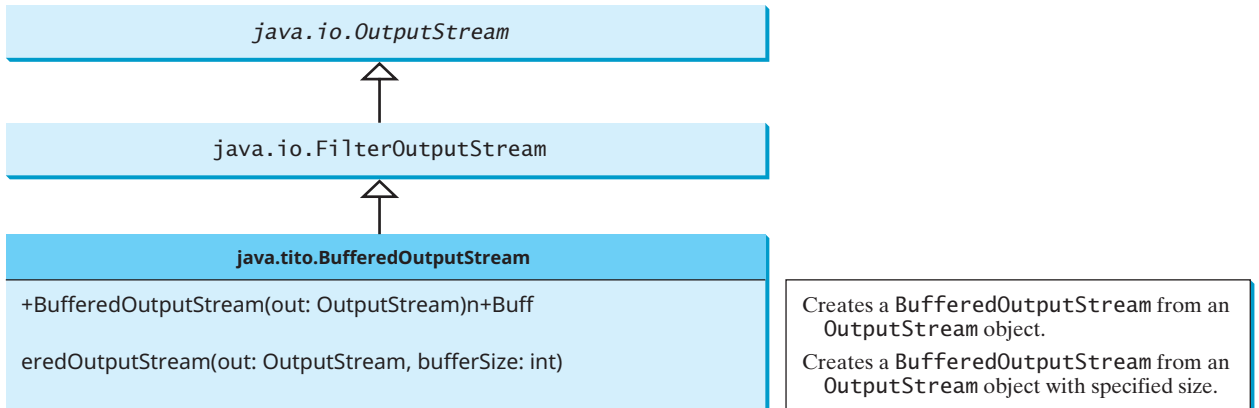


FIGURE 19.13 **BufferedOutputStream** bir çıktı akışını tamponlar.



Eğer bir tampon boyutu belirtilmemişse, varsayılan boyut **512** bayttır. **TestDataStream** programının performansını, akışa tamponlar ekleyerek 19.2 numaralı listede 6-7 ve 21-22. satırlarda aşağıdaki gibi artırabilirsiniz:

```
DataOutputStream çıkışı = newtDataOutputStream(n(tne
 yeniBufferedOutputStream wtFileOutputStream(t"temp.dat"t));nDataIn
putStream girişi = newtDataInputStream(n(tnewtFileI
 yeniArabelleğe AlınmışGiriş Akışı nputStream(t"temp.dat"t));
```



#### İpucu

Girdi ve çıktıyı hızlandırmak için her zaman tamponlu I/O kullanmalısınız. Küçük dosyalar için, performans iyileştirmelerini fark etmeyebilirsiniz. Ancak, 100 MB'den büyük dosyalar için, tamponlu I/O kullanarak önemli iyileştirmeler göreceksiniz.

Point/Kontrol

MyProgrammingLab™

- 19.8** Neden metotta **IOException** fırlatmayı beyan etmeniz veya Java I/O programları için **IOException**'i işlemek üzere bir try-catch bloğu kullanmanız gerekiyor?
- 19.9** Neden her zaman akışları kapatmalısınız?
- 19.10** **InputStream** içindeki **read()** metodu bir bayt okur. Neden bir **byte** yerine bir **int** döndürür? **InputStream** ve **OutputStream** içindeki soyut metotları bulun.
- 19.11** **FileInputStream/FileOutputStream**, **InputStream/OutputStream**'dan miras alınan metotların ötesinde herhangi bir yeni metot sunuyor mu? Nasıl bir **FileInputStream/FileOutputStream** oluşturursunuz?
- 19.12** Var olmayan bir dosya üzerinde bir girdi akışı oluşturmaya çalışırsanız ne olur? Var olan bir dosya üzerinde bir çıktı akışı oluşturmaya çalışırsanız ne olur? Var olan bir dosyaya veri ekleyebilir misiniz?
- 19.13** **java.io.PrintWriter** kullanarak var olan bir metin dosyasına veri nasıl eklersiniz?
- 19.14** Bir dosyanın belirsiz sayıda **double** değeri içerdiğini varsayalım. Bu değerler, bir **DataOutputStream** kullanarak **writeDouble** metodu ile dosyaya yazılmıştır. Tüm bu değerleri okumak için bir program nasıl yazarsınız? Bir dosyanın sonunu nasıl tespit edersiniz?
- 19.15** **FileOutputStream** kullanarak **writeByte(91)** ile bir dosyaya ne yazılır?
- 19.16** Bir girdi akışında (**FileInputStream**, **DataInputStream**) bir dosyanın sonunu nasıl kontrol edersiniz?
- 19.17** Aşağıdaki kodda ne yanlış?

```
import java.io.*;

genel sınıfTest {
 public static voidtmain(String[] args) {ntry{

 FileInputStream fis = newtFileInputStream(t"test.dat"t);n}

 catcht(IOException ex) {nex.p
 rintStackTrace();n}

 catcht(FileNotFoundException ex) {nex.pri
 ntStackTrace();

 }
 }
}
```

- 19.18** Varsayalım ki programı varsayılan ASCII kodlamasıyla Windows'ta çalıştırıyorsunuz. Program tamamlandıktan sonra, **t.txt** dosyasında kaç bayt var? Her bir baytın içeriğini gösterin.

```

genel sınıfTest {
 public static void main(String[] args) throws
 java.io.IOException {
 java.io.PrintWriter çıktı =
 yeni java.io.PrintWriter("t.txt");
 çıktı.printf("%s", "1234");
 çıktı.printf("%s", "5678");
 çıktı.close();
 }
}

```

**19.19** Program tamamlandığında, **t.dat** dosyasında kaç bayt var? Her bir baytın içeriğini gösterin.

```
import java.io.*;
```

```

genel sınıfTest {
 public static void main(String[] args) throws IOException {
 DataOutputStream çıktı = yeni DataOutputStream(
 yeni FileOutputStream("t.dat"));
 çıktı.writeInt(1234);
 çıktı.writeInt(5678);
 çıktı.close();
 }
}

```

**19.20** Aşağıdaki her bir ifade için **DataOutputStream output**, çıktıya kaç bayt gönderilir?

```

çıktı.writeChar('A');
ut.writeChars("BC");
.ut.writeUTF("DEF");

```

**19.21** Tamponlu akışların kullanılmasının avantajları nelerdir? Aşağıdaki ifadeler doğru mu?

```

TamponlanmışGirişAkışı girişi1 =
 newTBufferedInputStream(tnewTFileInputStream("t.dat"));

nputStream input2 = tnewTDataInputStream(
 newTBufferedInputStream(tnewTFileInputStream("t.dat")));

InputStream input3 = tnewTObjectInputStream(
 newTBufferedInputStream(tnewTFileInputStream("t.dat")));

```

## 19.5 Vaka Çalışması: Dosyaları Kopyalama



*Bu bölüm, dosyaları kopyalamak için yararlı bir yardımcı program geliştirir.*

Bu bölümde, kullanıcıların dosyaları kopyalamasına olanak tanıyan bir program yazmayı öğreneceksiniz. Kullanıcı, komut satırı argümanları olarak bir kaynak dosya ve bir hedef dosya sağlamalıdır. Komut:

```
java Kaynak hedefini kopyala
```

Program, kaynak dosyayı hedef dosyaya kopyalar ve dosyadaki byte sayısını gösterir. Program, kaynak dosya mevcut değilse veya hedef dosya zaten mevcutsa kullanıcıyı uyarmalıdır. Programın örnek çalışması Şekil 19.14'te gösterilmiştir.



**VideoNotu**

Dosyayı kopyala

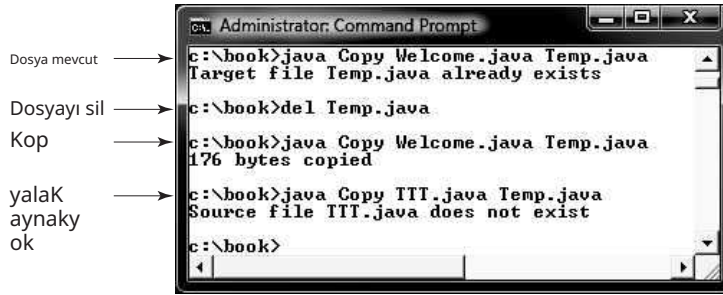


FIGURE 19.14 Program bir dosyayı kopyalar.

Bir kaynak dosyasının içeriğini bir hedef dosyaya kopyalamak için, kaynak dosyadan baytları okumak için bir girdi akışı ve hedef dosyaya baytları göndermek için bir çıktı akışı kullanmak uygundur, dosyanın içeriğinden bağımsız olarak. Kaynak dosya ve hedef dosya komut satırından belirtilir. Kaynak dosya için bir `InputStream` ve hedef dosya için bir `OutputStream` oluşturun. Girdi akışından bir bayt okumak için `read()` yöntemini kullanın, ardından baytı çıktı akışına yazmak için `write(b)` yöntemini kullanın. Performansı artırmak için `BufferedInputStream` ve `BufferedOutputStream` kullanın. Liste 19.4, probleme çözümü verir.

### LISTE 19.4 Copy.java

```

1 import java.io.*;
2
3 genel sınıf Kopyala {
4 /** Ana yöntem
5 * @param args[0] kaynak dosya için
6 * @param args[1] hedef dosya için
7 */
8 public static void tmain(String[] args) throws IOException {
9 // Komut satırı parametre kullanımını kontrol et
10 if(args.length != 2) {
11 System.out.println(
12 "Kullanım: java Copy kaynakDosya hedefDosya");
13 System.exit(1);
14 }
15
16 // Kaynak dosyanın var olup olmadığını kontrol et
17 Dosya kaynağıDosya = tnewFile(args[0]);
18 if (!sourceFile.exists()) {
19 System.out.println("Kaynak dosya " + args[0]
20 + " mevcut değil");
21 System.exit(2);
22 }
23
24 // Hedef dosyanın var olup olmadığını kontrol et
25 Dosya hedefFile = tnewFile(args[1]);
26 if(targetFile.exists()) {
27 System.out.println("Hedef dosya " + args[1]
28 + " zaten mevcut");
29 System.exit(3);
30 }
31
32 // Bir giriş akışı oluştur
33 BufferedInputStream girişi =

```

kullanımı kontrol et

kaynak dosyası

hedef dosya

girdi akışı

```

34 newtBufferedInputStream(tnewtFileInputStream(sourceFile));
35
36 // Bir çıktı akışı oluştur
çıkış akışı 37 OutputStream çıkış =
38 newtBufferedOutputStream(tnewtFileOutputStream(targetFile));nBuffered
39
40 // Girişten sürekli bir bayt oku ve çıktıya yaz
41 intr, kopyalananBaytSayısı = 0;
42 iken ((oku()) != -1t) {
43 çıktı.yaz(tbytet)r;nr = girdi.
44 kopyalananBaytSayısı++;
45 }
46
47 // Akışları kapat
akışı kapat 48 input.close();n49 çıktı.clo
se();
50
51 // Dosya boyutunu göster
52 System.out.println(numberOfBytesCopied + " bayt kopyalandı");
53 }
54 }

```

Program, kullanıcıdan iki gerekli argümanın komut satırından geçip geçmediğini ilk olarak 10-14. satırlarda kontrol eder.

Program, kaynak dosyanın ve hedef dosyanın var olup olmadığını kontrol etmek için **File** sınıfını kullanır. Eğer kaynak dosya mevcut değilse (18-22. satırlar) veya hedef dosya zaten mevcutsa (25-30. satırlar), program sona erer.

Bir giriş akışı, 33-34. satırlarda **BufferedInputStream** kullanılarak **FileInputStream** üzerine sarılarak oluşturulur ve bir çıkış akışı, 37-38. satırlarda **BufferedOutputStream** kullanılarak **FileOutputStream** üzerine sarılarak oluşturulur.

İfade **((r = input.read()) != -1)** (42. satır) bir baytı **input.read()**'den okur, onu **r**'ye atar ve **-1** olup olmadığını kontrol eder. **-1** giriş değeri, bir dosyanın sonunu belirtir. Program, giriş akışından sürekli olarak baytları okur ve bunları çıkış akışına gönderir, ta ki tüm baytlar okunana kadar.

**19.22** Program, bir dosyanın zaten mevcut olup olmadığını nasıl kontrol eder?

Point/Kontrol

**19.23** Program, verileri okurken dosyanın sonunu nasıl tespit eder?

MyProgrammingLab™

**19.24** Program, dosyadan okunan bayt sayısını nasıl sayar?

## 19.6 Nesne G/Ç



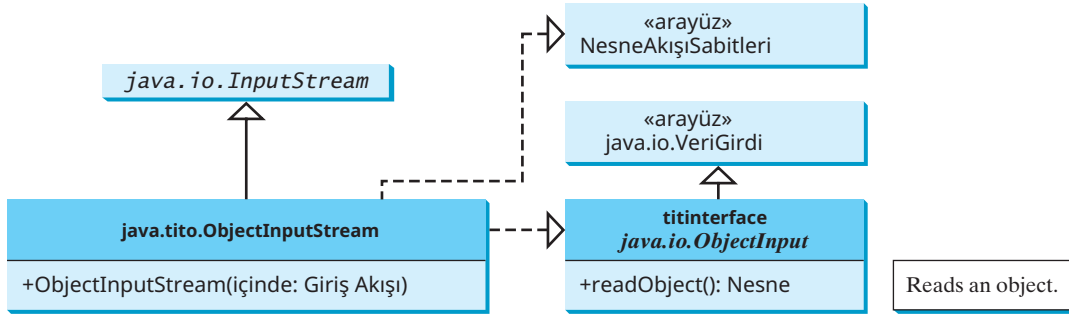
*NesneInputStream/NesneOutputStream sınıfları, serileştirilebilir nesneleri okumak/yazmak için kullanılabilir.*



VideoNotu  
Nesne G/Ç

**DataInputStream /DataOutputStream**, ilkel tür değerleri ve dizeler için G/Ç işlemleri yapmanıza olanak tanır. **NesneInputStream/NesneOutputStream**, ilkel tür değerleri ve dizelerin yanı sıra nesneler için G/Ç işlemleri yapmanıza olanak tanır. **NesneInputStream/NesneOutputStream**, **DataInputStream /DataOutputStream**'nin tüm işlevlerini içerdiği için, **DataInputStream /DataOutputStream**'yi tamamen **NesneInputStream/NesneOutputStream** ile değiştirebilirsiniz.

**NesneInputStream**, **InputStream**'i genişletir ve **ObjectInput** ve **ObjectStreamConstants**'i uygular; bu, Şekil 19.15'te gösterilmiştir. **ObjectInput**, **DataInput**'in bir alt arayüzüdür (**DataInput**, Şekil 19.9'da gösterilmiştir). **ObjectStreamConstants**, **NesneInputStream/NesneOutputStream**'i desteklemek için sabitleri içerir.



FiçİN 19.15 **ObjectInputStream** nesneleri, ilkel tür değerlerini ve dizeleri okuyabilir.

**ObjectOutputStream** extends **OutputStream** ve implements **ObjectOutput** ve **ObjectStreamConstants**, Şekil 19.16'da gösterildiği gibi. **ObjectOutput**, **DataOutput**'nın bir alt arayüzüdür (**DataOutput** Şekil 19.10'da gösterilmiştir).

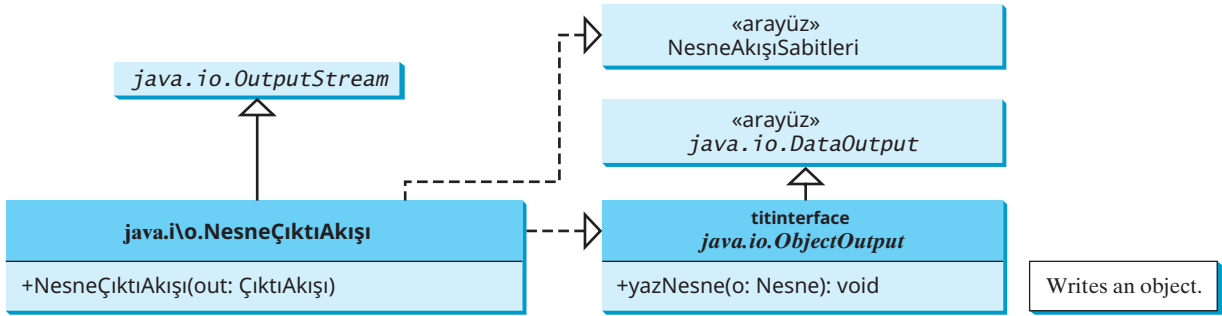


FIGURE 19.16 **ObjectOutputStream** nesneleri, ilkel tür değerlerini ve dizeleri yazabilir.

Herhangi bir **InputStream**/

**OutputStream** üzerinde bir **ObjectInputStream**/**ObjectOutputStream** sarmalayabilirsiniz, aşağıdaki yapıcılarını kullanarak:

```
// Bir ObjectInputStream oluşturun
public ObjectInputStream(InputStream in)
```

```
// Bir ObjectOutputStream oluşturun
public NesneÇıktıAkışı(ÇıktıAkışı out)
```

Liste 19.5, öğrenci isimlerini, notlarını ve mevcut tarihi **object.dat** adlı bir dosyaya yazar.

### LİSTE 19.5 TestNesneÇıktıAkışı.java

```
1 import java.io.*;
2
3 public class TestNesneÇıktıAkışı {
4 public static void tmain(String[] args) tthrowstIOException {
5 // object.dat dosyası için bir çıktı akışı oluşturun
6 NesneÇıktıAkışı çıktı =
7 new NesneÇıktıAkışı(new DosyaÇıktıAkışı("object.dat"));
8
9 // Dosyaya bir dize, çift değer ve nesne yaz
```

çıkış akışı

## 726 Bölüm 19 İkili G/Ç

çıkış

```
10 çıktı.writeUTF(t"John"t);n11 çıktı.write
Double(t85.5t);
12 Output.writeObject(tnewtjava.util.Date());
13
14 // Çıkış akışını kapat
15 output.close();
16 }
17 }
```

Bir **ObjectOutputStream**, 6–7. satırlarda **object.dat** dosyasına veri yazmak için oluşturulur. Birdize, bir double değeri ve bir nesne 10–12. satırlarda dosyaya yazılır. Performansı artırmak için, bu ifadeyi kullanarak akışa bir tampon ekleyebilirsiniz:

```
ObjectOutputStream çıktı = tnewtObjectOutputStream(nnewtBu
fferedOutputStream(tnewtFileOutputStream(t"object.dat"t)));
```

Birden fazla nesne veya ilkel değer akışa yazılabilir. Nesneler, yazıldıkları sırayla ve aynı türlerle karşılık gelen **ObjectInputStream**'den okunmalıdır. İstenilen türü elde etmek için Java'nın güvenlicasting'i kullanılmalıdır. Liste 19.6, **object.dat** dosyasından veri okur.

### LISTING 19.6 TestObjectInputStream.java

girdi akışı

```
1 import java.io.*;
2
3 genel sınıf TestObjectInputStream {
4 public static void tmain(String[] args)
5 throwstClassNotFoundException, IOException {
6 // Dosya nesnesi için bir girdi akışı oluştur
7 // Dosya nesnesi için bir girdi akışı oluştur
8 // Dosya nesnesi için bir girdi akışı oluştur
9 // Dosya nesnesi için bir girdi akışı oluştur
10 // Dosya nesnesi için bir girdi akışı oluştur
11 // Dosya nesnesi için bir girdi akışı oluştur
12 // Dosya nesnesi için bir girdi akışı oluştur
13 // Dosya nesnesi için bir girdi akışı oluştur
14 // Dosya nesnesi için bir girdi akışı oluştur
15 // Dosya nesnesi için bir girdi akışı oluştur
16 // Dosya nesnesi için bir girdi akışı oluştur
17 // Dosya nesnesi için bir girdi akışı oluştur
18 // Dosya nesnesi için bir girdi akışı oluştur
19 }
```

girdi

```
10 // Dosyaya bir dize, çift değer ve nesne yaz
11 Dize adı = input.readUTF();
12 doubletscore = input.readDouble();
13 java.util.Date tarih = (java.util.Date)();n14 System.out.println(tinput.readObject() + t
" "t+ tarih);
15
16 // Giriş akışını kapat
17 input.close();n18 }
```



John 85.5 Güneş Ara 04 10:35:31 EST 2011

ClassNotFoundException

**readObject()** metodu **java.lang.ClassNotFoundException** fırlatabilir, çünkü JVM bir nesneyi geri yüklediğinde, önce nesnenin sınıfını yükler eğer sınıf yüklenmemişse. **ClassNotFoundException** kontrol edilen bir istisna olduğundan, **main** metodu bunu 5. satırda fırlatacağını belirtir. Bir **ObjectInputStream**, **object.dat** dosyasından veri okumak için 7-8. satırlarda oluşturulur. Dosyadan verileri yazıldığı sırayla ve formatta okumanız gerekir. 11-13. satırlarda bir dize, bir double değeri ve bir nesne okunur. **readObject()** bir **Object** döndürdüğünden, **Date** türüne dönüştürülür ve 13. satırda bir **Date** değişkenine atanır.

## 19.6.1 Serializable Arayüzü

Her nesne bir çıktı akışına yazılamaz. Yazılabilen nesnelere *seri hale getirilebilir* denir. Seri hale getirilebilir bir nesne, **java.io.Serializable** arayüzünün bir örneğidir, bu nesne ile nesnenin sınıfı **Serializable** arayüzünü uygulamalıdır.

serileştirilebilir

Serializable arayüzü bir işaretçi arayüzüdür. Hiçbir yöntemi olmadığı için, Serializable'ı uygulayan sınıfta ek bir kod eklemenize gerek yoktur. Bu arayüzü uygulamak, Java serileştirme mekanizmasının nesneleri ve dizileri depolama sürecini otomatikleştirmesi ni sağlar.

Bu otomasyon özelliğini takdir etmek için, bir nesneyi depolamak için başka neler yapmanız gerektiğini düşünün. Diyelim ki bir JButton nesnesini depolamak istiyorsunuz. Bunu yapmak için, nesnedeki tüm özelliklerin (örneğin, renk, yazı tipi, metin, hizalama) mevcut değerlerini depolamanız gerekir. Çünkü JButton, AbstractButton'ın bir alt sınıfıdır, bu nedenle AbstractButton'ın özellik değerleri ile AbstractButton'ın tüm üst sınıflarının özellikleri de depolanmalıdır. Eğer bir özellik bir nesne türündeysse (örneğin, Color türündeki background), bunu depolamak, bu nesne içindeki tüm özellik değerlerini depolamayı gerektirir. Gördüğünüz gibi, bu çok zahmetli bir süreç olurdu. Neyse ki, bunu manuel olarak yapmak zorunda değilsiniz. Java, nesneleri yazma sürecini otomatikleştirmek için yerleşik bir mekanizma sağlar. Bu süreç, nesne serileştirme olarak adlandırılır ve ObjectOutputStream'de uygulanır. Buna karşılık, nesneleri okuma süreci nesne serileştirmesi olarak adlandırılır ve ObjectInputStream'de uygulanır.

serileştirme

serileştirmeden çıkarma

Java API'sindeki birçok sınıf **Serializable** arayüzünü uygular. java.util.Date gibi yardımcı sınıflar ve tüm Swing GUI bileşen sınıfları **Serializable** arayüzünü uygular. Desteklemeyen bir nesneyi **Serializable** arayüzü ile depolamaya çalışmak, bir **NotSerializableException** ile sonuçlanır.

Serileştirilebilir İstisna Değil

Serileştirilebilir bir nesne depolandığında, nesnenin sınıfı kodlanır; bu, sınıf adını ve sınıfın imzasını, nesnenin örnek değişkenlerinin değerlerini ve nesne tarafından referans verilen diğer nesnelerin kapanışını içerir. Nesnenin statik değişkenlerinin değerleri depolanmaz.



### Not

#### serileştirilemeyen alanlar

Eğer bir nesne **Serializable** arayüzünün bir örneği ise ancak serileştirilemeyen örnek veri alanları içeriyorsa, serileştirilebilir mi? Cevap hayır. Nesnenin serileştirilebilir olmasını sağlamak için, bu veri alanlarını **transient** anahtar kelimesi ile işaretleyin, böylece JVM nesneyi bir nesne akışına yazarken bunları göz ardı etsin. Aşağıdaki sınıfı düşünün:

geçici

```
public class C implements java.io.Serializable {
 private int v1;
 ; özel statik double v2;
 özel geçici A v3 = new A();
}
```

```
class A { } // A serileştirilemez
```

Bir **C** sınıfının nesnesi serileştirildiğinde, yalnızca **v1** değişkeni serileştirilir. **v2** değişkeni serileştirilemez çünkü bu bir statik değişkendir ve **v3** değişkeni serileştirilmez çünkü bu işaretlenmiştir. **geçici**. Eğer **v3** işaretlenmemişse **geçici**, bir **java.io.NotSerializableException** oluşacaktır.



### Not

#### kopya nesneler

Eğer bir nesne bir nesne akışına birden fazla kez yazılırsa, birden fazla kopya olarak mı saklanır? Hayır, saklanmaz. Bir nesne ilk kez yazıldığında, onun için bir seri numarası oluşturulur. JVM, nesnenin tamamının içeriğini seri numarası ile birlikte yazar.

nesne akışına numara. İlk seferden sonra, yalnızca seri numarası saklanır eğer aynı nesne tekrar yazılırsa. Nesneler geri okunduğunda, referansları aynıdır, bir nesne aslında bellekte oluşturulduğu için.

### 19.6.2 Dizileri Seri Hale Getirme

Bir dizi, tüm elemanları seri hale getirilebiliyorsa seri hale getirilebilir. Tüm bir dizi, bir dosyaya **writeObject** kullanılarak kaydedilebilir ve daha sonra **readObject** kullanılarak geri yüklenebilir. Liste 19.7, beş **int** değerinden oluşan bir diziyi ve üç string içeren bir diziyi saklar ve bunları geri okuyarak konsolda görüntüler.

#### LISTE 19.7 TestObjectStreamForArray.java

```

1 import java.io.*;
23
 genel classtestObjectStreamForArray {
4 public static voidtmain(String[] args)
5 throwstClassNotFoundException, IOException {
6 int[] sayılar = {t1t,t2t,t3t,t4t,t5t};
7 String[] strings = {t"John"t,t"Susan"t,t"Kim"t};
89
 // Dosya array.dat için bir çıktı akışı oluştur
çıkış akışı 101 OutputStream çıktı = tnewtObjectOutputStream(tnew
 FileOutputStream(t"array.dat"t,ttrue));nObject
11
12
13 // Dizileri nesne çıktı akışına yaz
diziyi sakla 14riteObject(sayılar);
 çıktı.writeObject(dizeler);noutput.w
15
16
17 // Akışı kapat
18 output.close();
19
20 // Dosya array.dat için bir girdi akışı oluştur
girdi akışı 21putStream girdi =
 newtObjectInputStream(tnewtFileInputStream(t"array.dat"t));nObjectIn
22
23
24 ers = (tintt[])(input.readObject());
diziyi geri yükle 25String[] newStrings = (String[])(input.readObject());nintt[] newNumb
26
27 // Dizileri görüntüle
28 fort(tint = t0t; i < newNumbers.length; i++)
29 System.out.print(newNumbers[i] + t" "t);n30 System.out.pr
31 intln();
32
33 fort(tint = t0t; i < newStrings.length; i++)
34 System.out.print(newStrings[i] + t" "t);
35
36 // Akışı kapat
37 input.close();
38 }

```



```

1 2 3 4 5 nj
ohn Susan Kim

```

Satırlar 14–15 iki diziyi **array.dat** dosyasına yazar. Satırlar 24–25 iki diziyi geri okur veya yazıldığı sırayla geri alır. **readObject()** metodu **Object** döndürdüğü için, nesneleri **int[]** ve **String[]** olarak dönüştürmek için casting kullanılır.



**19.25** Hangi tür nesneler **ObjectOutputStream** kullanılarak saklanabilir? Bir nesneyi yazma yöntemi nedir? Bir nesneyi okuma yöntemi nedir? **ObjectInputStream**'den bir nesneyi okuyan metodun dönüş tipi nedir?

**19.26** İki aynı tür nesneyi serileştirirseniz, aynı miktarda

alan alacaklar mı? Eğer hayırsa, bir örnek verin.

**19.27** **java.io.Serializable**'nin herhangi bir örneğinin başarıyla serileştirilebileceği doğru mu? Bir nesnedeki statik değişkenler serileştiriliyor mu? Bir örnek değişkenin serileştirilmemesi için nasıl işaretlenir?

**19.28** Bir diziyi **ObjectOutputStream**'e yazabilir misiniz?

**19.29** **DataInputStream/DataOutputStream** her zaman **ObjectInputStream/ObjectOutputStream** ile değiştirilebilir mi?

**19.30** Aşağıdaki kodu çalıştırmaya çalıştığınızda ne olacak?

```
import java.io.*;

genel sınıf Test {
 public static void main(String[] args) throws IOException {
 ObjectOutputStream oout =
 new ObjectOutputStream(new FileOutputStream("object.dat"));
 oout.writeObject(tnewA());
 }

 class A implements Serializable {
 byte b = tnewB();
 }

 sınıf B {
 n
 }
}
```

## 19.7 Rastgele Erişim Dosyaları

*Java, bir dosyanın rastgele konumlardan okunmasına ve yazılmasına olanak tanıyan **RandomAccessFile** sınıfını sağlar.*



Şu ana kadar kullandığınız tüm akışlar *yalnızca okuma* veya *yalnızca yazma* akışları olarak bilinir. Bu akışların dış dosyaları *sıralı* dosyalardır ve yeni bir dosya oluşturmadan güncellenemez.

Ancak, dosyaların değiştirilmesi genellikle gereklidir. Java, bir dosyanın rastgele konumlardan okunmasını ve yazılmasına izin vermek için **RandomAccessFile** sınıfını sağlar.

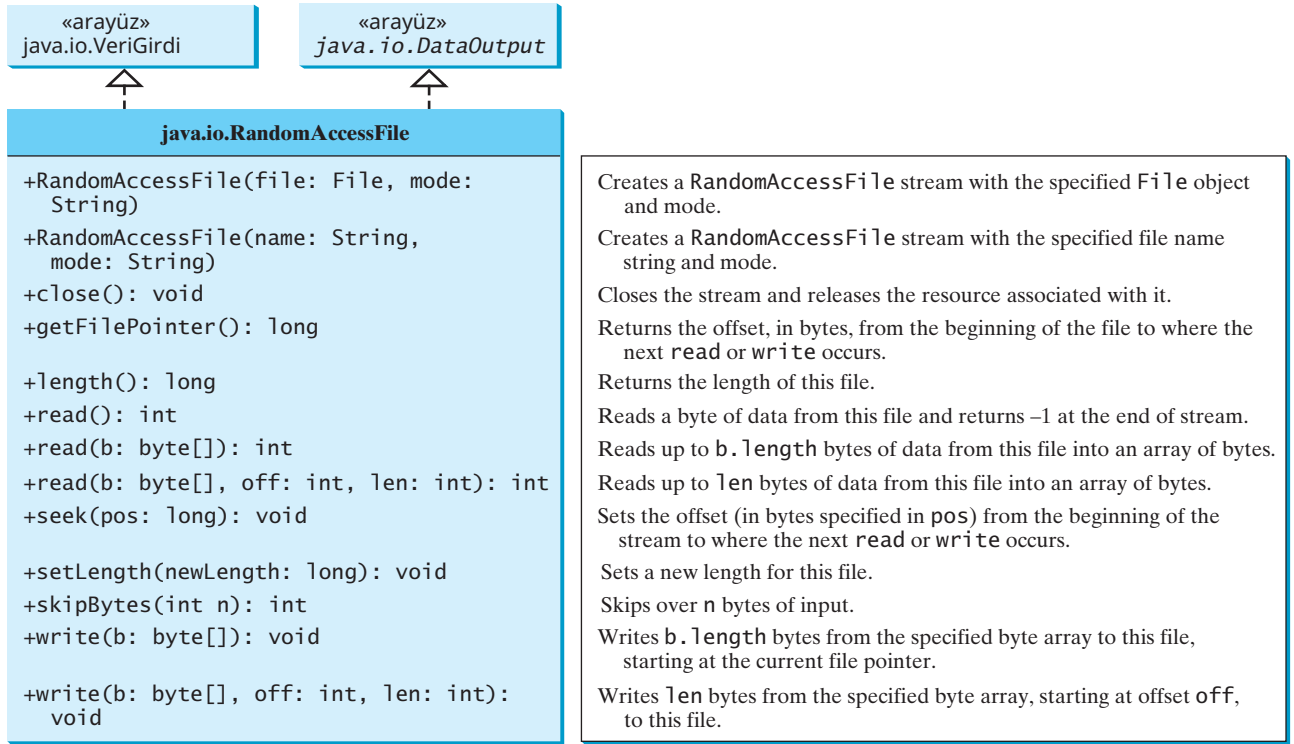
**RandomAccessFile** sınıfı, Şekil 19.17'de gösterildiği gibi **DataInput** ve **DataOutput** arayüzlerini uygular. **DataInput** arayüzü (bkz. Şekil 19.9), ilkel tür değerlerini ve dizeleri okumak için yöntemleri tanımlar (örneğin, **readInt**, **readDouble**, **readChar**, **readBoolean**, **readUTF**) ve **DataOutput** arayüzü (bkz. Şekil 19.10) ilkel tür değerlerini ve dizeleri yazmak için yöntemleri tanımlar (örneğin, **writeInt**, **writeDouble**, **writeChar**, **writeBoolean**, **writeUTF**).

Bir **RandomAccessFile** oluştururken, iki moddan birini belirtebilirsiniz: **r** veya **rw**. Mod **r**, akışın yalnızca okunabilir olduğunu belirtirken, mod **rw**, akışın hem okuma hem de yazma işlemlerine izin verdiğini gösterir. Örneğin, aşağıdaki ifade, programın **test.dat** dosyasından okumasına ve yazmasına izin veren yeni bir akış, **raf** oluşturur:

```
RandomAccessFile raf = new RandomAccessFile("test.dat", "rw");
```

Eğer **test.dat** zaten mevcutsa, **raf** ona erişmek için oluşturulur; eğer **test.dat** mevcut değilse, **test.dat** adında yeni bir dosya oluşturulur ve **raf** yeni dosyaya erişmek için oluşturulur. **raf.length()** yöntemi, **test.dat** içindeki byte sayısını herhangi bir zamanda döndürür. Dosyaya yeni veriler eklediğinizde, **raf.length()** artar.

salt-okunur  
salt-yazılabilir  
sıralı  
rastgele-erisim dosyası



**FIGURE 19.17** `RandomAccessFile`, `DataInput` ve `DataOutput` arayüzlerini ek yöntemlerle uygular rastgele erişimi desteklemek için.



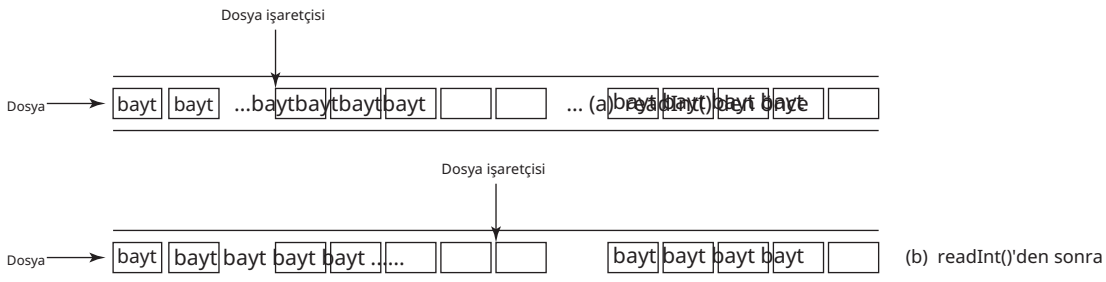
#### İpucu

Dosya değiştirilmek üzere tasarlanmamışsa, `r` modunda açın. Bu, dosyanın istenmeyen şekilde değiştirilmesini önler.

dosya işaretçisi

Bir rastgele erişim dosyası, bir dizi bayttan oluşur. *dosya işaretçisi* adı verilen özel bir işaretçi bu baytlardan birinin üzerinde konumlandırılır. Bir okuma veya yazma işlemi, dosya işaretçisinin konumunda gerçekleşir. Bir dosya açıldığında, dosya işaretçisi dosyanın başında ayarlanır. Veriyi okuduğunuzda veya dosyaya yazdığınızda, dosya işaretçisi bir sonraki veri ögesine doğru ilerler. Örneğin, eğer `readInt()` kullanarak bir `int` değeri okursanız, JVM dosya işaretçisinden 4 bayt okur, ve şimdi dosya işaretçisi önceki konumdan 4 bayt ileridedir, Şekil 19.18'de gösterildiği gibi.

Bir `RandomAccessFile raf` için, dosya işaretçisini belirli bir konuma taşımak için `raf.seek(position)` yöntemini kullanabilirsiniz. `raf.seek(0)` dosyanın başına taşır ver `af.seek(raf.length())` dosyanın sonuna taşır. Liste 19.8 bunu göstermektedir.



**FIGURE 19.18** Bir `int` değeri okunduktan sonra, dosya işaretçisi 4 bayt ileri taşınır.

**Rastgele Erişim Dosyası.** Bir adres defterini düzenlemek için **Rastgele Erişim Dosyası** kullanmanın bu yük bir vaka çalışması Ek VI.B'de verilmiştir.

## Listing 19.8 TestRandomAccessFile.java

```

1 import java.io.*;
23
4 genel sınıf TestRandomAccessFile {
5 public static void tmain(String[] args) throws IOException {
6 // Rastgele erişim dosyası oluştur
7 RandomAccessFile giriş = t new RandomAccessFile("inout.dat", "rw");
8
9 // Dosyayı temizle, eski içerikleri yok et, eğer varsa
10 inout.setLength(0);
11
12 // Dosyaya yeni tam sayılar yaz
13 for(tint = 0; i < 200; i++)
14 inout.writeInt(i);
15
16 // Dosyanın mevcut uzunluğunu göster
17 System.out.println("Mevcut dosya uzunluğu " + giriş.uzunluk());
18
19 // İlk sayıyı al
20 ut.seek(0); // Dosya işaretçisini başa taşı
21 System.out.println("İlk sayı " + inout.readInt());
22
23 // İkinci sayıyı al
24 ut.seek(1*4); // Dosya işaretçisini ikinci sayıya taşı
25 System.out.println("İkinci sayı " + inout.readInt());
26
27 // Onuncu sayıyı al
28 ut.seek(9*4); // Dosya işaretçisini onuncu sayıya taşı
29 System.out.println("Onuncu sayı " + inout.readInt());
30
31 // On birinci sayıyı değiştir
32 inout.writeInt(555);
33
34 // Yeni bir sayı ekle
35 ut.seek(inout.length()); // Dosya işaretçisini sona taşı
36 inout.writeInt(999);
37
38 // Yeni uzunluğu göster
39 System.out.println("Yeni uzunluk " + giriş.uzunluk());
40
41 // Yeni onuncu numarayı al
42 ut.seek(10*4); // Dosya işaretçisini bir sonraki numaraya taşı
43 System.out.println("Onuncu numara " + inout.readInt());
44 }
45 }
46 }
```

Rastgele Erişim Dosyası

boş dosya

yaz

işaretçi taşı

oku

dosyayı kapat

Mevcut dosya uzunluğu 800  
İlk sayı 0  
İkinci sayı 1  
Onuncu sayı 9  
Yeni uzunluk 804  
Onbirinci sayı 555



A **RandomAccessFile**, **inout.dat** adlı dosya için **rw** modunda oluşturulur, böylece hem okuma hem de yazma işlemlerine izin verilir (6. satır).

**inout.setLength(0)**, 9. satırda uzunluğu 0 olarak ayarlar. Bu, etkili bir şekilde, dosyanın eski içeriğini imha eder.

**for** döngüsü, 12-13. satırlarda 0 ile 199 arasında 200 **int** değerini dosyaya yazar. Her **int** değeri 4 bayt aldığından, **inout.length()** ile dönen dosyanın toplam uzunluğu şimdi 800 (16. satır), örnek çıktıda gösterildiği gibi.

19. satırda **inout.seek(0)** çağırısı, dosya işaretçisini dosyanın başına ayarlar. **inout.readInt()**, 20. satırda ilk değeri okur ve dosya işaretçisini bir sonraki numaraya hareket ettirir. İkinci sayı 24. satırda okunur.

**inout.seek(9 \* 4)** (27. satır), dosya işaretçisini onuncu sayıya taşır. **inout.readInt()** onuncu sayıyı okur ve dosya işaretçisini 28. satırda on birinci sayıya hareket ettirir. **inout.write(555)**, mevcut konumda (31. satır) yeni bir on birinci sayı yazar. Önceki on birinci sayı imha edilir.

**inout.seek(inout.length())** dosya işaretçisini dosyanın sonuna taşır (satır 34). **inout.writeInt(999)** dosyaya 999 yazar. Artık dosyanın uzunluğu 4 ile artar, bu nedenle **inout.length()** 804 döner (satır 38).

**inout.seek(10 \* 4)** dosya işaretçisini satır 41'deki onuncu sayıya taşır. Yeni onuncu sayı, 555, satır 42'de görüntülenir.

### 19.31 Can **RandomAccessFile** akışları,

**DataOutputStream** tarafından oluşturulan bir veri dosyasını okuyup yazabilir mi? Can **RandomAccessFile** akışları nesneleri okuyup yazabilir mi?

Point/Kontrol

MyProgrammingLab™

**19.32** Dosya için bir **RandomAccessFile** akışı oluşturun **address.dat**, böylece dosyadaki öğrenci bilgilerini güncelleyebilirsiniz. Dosya için bir **DataOutputStream** oluşturun **address.dat**. Bu iki ifade arasındaki farkları açıklayın.

**19.33** Aşağıdaki kodu derlemeye ve çalıştırmaya çalıştığınızda dosya **test.dat** mevcut değilse ne olur?

```
import java.io.*;

genel sınıfTest {
 public static void main(String[] args) {ntry{

 RandomAccessFile raf =new
 wtRandomAccessFile(t"test.dat"t,t"r"t");nintti = ra
 f.readInt();
 }
 catch (IOException ex) {
 System.out.println("IO istisnası");
 }
 }
}
```

#### ANAHTAR TERİMLER

ikili G/Ç 710  
serileştirme 727  
dosya işaretçisi 730  
rastgele erişim dosyası 729

ardışık erişim dosyası 729  
serileştirme 727  
akış 710  
metin G/Ç 710

## BÖLÜM ÖZETİ

1. *G/Ç*, *metin G/Ç* ve *ikili G/Ç* olarak sınıflandırılabilir. Metin *G/Ç*, verileri karakter dizileri olarak yorumlar. İkili *G/Ç*, verileri ham ikili değerler olarak yorumlar. Metnin bir dosyada nasıl saklandığı, dosyanın kodlama şemasına bağlıdır. Java, metin *G/Ç* için otomatik olarak kodlama ve kod çözme işlemleri gerçekleştirir.
2. *G GirdiAkışı* ve *ÇıktıAkışı* sınıfları, tüm ikili *G/Ç* sınıflarının kökleridir. *DosyaGirdiAkışı/DosyaÇıktıAkışı*, girdi/çıkı için bir dosya ile ilişkilendirir. *BufferedGirdiAkışı/BufferedÇıktıAkışı*, performansı artırmak için herhangi bir ikili *G/Ç* akışını sarmak için kullanılabilir. *VeriGirdiAkışı/VeriÇıktıAkışı*, ilkel değerleri ve dizeleri okumak/yazmak için kullanılabilir.
3. *NesneGirdiAkışı/NesneÇıktıAkışı*, ilkel değerler ve dizelerin yanı sıra nesneleri okumak/yazmak için kullanılabilir. Nesne *serileştirmesini* etkinleştirmek için, nesnenin tanımlayıcı sınıfı **java.io.Serializable** işaretçi arayüzünü uygulamalıdır.
4. *G RastgeleErişimDosyası* sınıfı, bir dosyaya veri okuma ve yazma işlemi yapmanızı sağlar. Bir dosyayı **r** modu ile açarak yalnızca okunabilir olduğunu belirtebilir veya **rw** modu ile güncellenebilir olduğunu belirtebilirsiniz. *RastgeleErişimDosyası* sınıfı, *DataInput* ve *DataOutput* arayüzlerini uyguladığından, *RastgeleErişimDosyası* içindeki birçok yöntem, *VeriGirdiAkışı* ve *VeriÇıktıAkışı* içindeki yöntemlerle aynıdır.

## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

## PROGRAMLAMA EGZERSİZLERİ

MyProgrammingLab™

### Bölüm 19.3

- \*19.1 (Bir metin dosyası oluşturun) Eğer yoksa **Exercise19\_01.txt** adında bir dosya oluşturmak için bir program yazın. Eğer zaten varsa, yeni verileri ona ekleyin. Metin *G/Ç* kullanarak dosyaya rastgele oluşturulmuş 100 tamsayı yazın. Tamsayılar bir boşlukla ayrılmıştır.

### Bölüm 19.4

- \*19.2 (Bir ikili veri dosyası oluşturun) Eğer yoksa **Exercise19\_02.dat** adında bir dosya oluşturmak için bir program yazın. Eğer zaten varsa, yeni verileri ona ekleyin. İkili *G/Ç* kullanarak dosyaya rastgele oluşturulmuş 100 tamsayı yazın.
- \*19.3 (Bir ikili veri dosyasındaki tüm tamsayıların toplamını alın) Diyelim ki **Exercise19\_03.dat** adında bir ikili veri dosyası oluşturulmuş ve verileri **writeInt(int)** içinde **DataOutputStream** kullanılarak oluşturulmuştur. Dosya, belirli olmayan sayıda tamsayı içermektedir. Tamsayıların toplamını bulmak için bir program yazın.
- \*19.4 (Bir metin dosyasını UTF'ye dönüştürün) Bir metin dosyasından karakter satırlarını okuyan ve her satırı bir UTF-8 dizesi olarak bir ikili dosyaya yazan bir program yazın. Metin dosyasının ve ikili dosyanın boyutlarını gösterin. Programı çalıştırmak için aşağıdaki komutu kullanın:

```
java Alıştırması19_04 Hoş Geldiniz.java Hoş Geldiniz.utf
```

## Bölüm 19.6

- \* **19.5** (Nesneleri ve dizileri bir dosyada sakla) Beş `int` değerini 1,2,3,4 ve 5, mevcut zaman için bir `Date` nesnesini ve double değerini 5.5 `Exercise19_05.dat` adlı dosyaya yazan bir program yazın.
- \* **19.6** (Kredi nesnelerini sakla) Liste 10.2'deki `Loan` sınıfı `Serializable`'i uygulamıyor. `Serializable`'i uygulayacak şekilde `Loan` sınıfını yeniden yazın. Beş `Loan` nesnesi oluşturan ve bunları `Exercise19_06.dat` adlı bir dosyaya saklayan bir program yazın.
- \* **19.7** (Bir dosyadan nesneleri geri yükle) `ObjectOutputStream` kullanılarak `Exercise19_07.dat` adlı bir dosya oluşturulduğunu varsayalım. Dosya `Loan` nesnelerini içeriyor. Liste 10.2'deki `Loan` sınıfı `Serializable`'i uygulamıyor. `Serializable`'i uygulayacak şekilde `Loan` sınıfını yeniden yazın. Dosyadan `Loan` nesnelerini okuyan ve toplam kredi miktarını hesaplayan bir program yazın. `Loan` nesnelerinin dosyada kaç tane olduğunu bilmiyorsanız varsayalım. Döngüyü sonlandırmak için `EOFException` kullanın.

## Bölüm 19.7

- \* **19.8** (Sayacı güncelle) Bir programın kaç kez çalıştırıldığını takip etmek istiyorsanız, dosyayı saymak için bir `int` saklayabilirsiniz. Bu program her çalıştırıldığında sayacı 1 artırın. Programı `Exercise19_08` olarak adlandırın ve sayacı `Exercise19_08.dat` dosyasında saklayın.
- \*\*\* **19.9** (Adres defteri) Ek VI.B, rastgele erişim dosyalarını kullanarak bir adres defteri oluşturma ve manipüle etme üzerine bir vaka çalışması içermektedir. Vaka çalışmasını, kullanıcının görüntülenen adresi değiştirmesine olanak tanımak için Şekil 19.19'da gösterildiği gibi bir `Güncelle` butonu ekleyerek değiştirin.



FIGURE 19.19 Uygulama, bir dosyadan adresleri saklayabilir, alabilir ve güncelleyebilir.

## Kapsamlı

- \* **19.10** (Bölünmüş dosyalar) Büyük bir dosyayı (örneğin, 10-GB'lık bir AVI dosyası) CD-R'ye yedeklemek istediğinizi varsayalım. Bunu, dosyayı daha küçük parçalara bölerek ve bu parçaları ayrı ayrı yedekleyerek başarabilirsiniz. Aşağıdaki komutu kullanarak büyük bir dosyayı daha küçük parçalara bölen bir yardımcı program yazın:

```
java Alistirması19_10 KaynakDosya numarasıOfPieces
```

Bu komut, `SourceFile.1.SourceFile.2, . . . ,SourceFile.n` dosyalarını oluşturur, burada `n` `numberOfPieces`'i temsil eder ve çıktı dosyaları yaklaşık olarak aynı boyuttadır.

- \*\* **19.11** (Bölünmüş dosyalar GUI) Şekil 19.20a'da gösterildiği gibi bir GUI ile `Exercise 19.10`'u yeniden yazın.

- \* **19.12** (Dosyaları birleştir) Aşağıdaki komutu kullanarak dosyaları yeni bir dosya içinde birleştiren bir yardımcı program yazın:

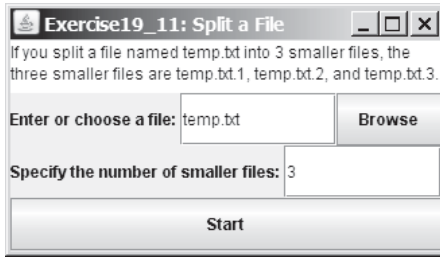
```
java Alistirma19_12 KaynakDosya1 . . . KaynakDosya HedefDosya
```

Bu komut, `SourceFile1, . . . ve SourceFilen`'i `TargetFile` içinde birleştirir.

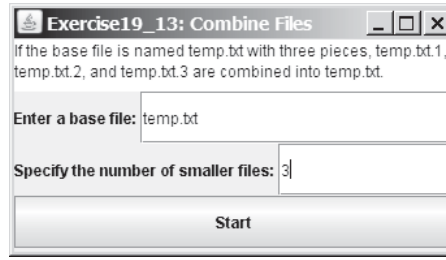


VideoNotu

Büyük bir dosyayı böl



(a)



(b)

FIGURE 19.20(a) Program bir dosyayı böler. (b) Program dosyaları yeni bir dosyada birleştirir.

### \*19.13 (Dosyaları birleştir GUI) Şekil

19.20b'de gösterildiği gibi, 19.12'yi bir GUI ile yeniden yazın.

**19.14 (Dosyaları şifrele)** Dosyayı, dosyadaki her bayta 5 ekleyerek kodlayın. Kullanıcının bir girdi dosyası adı ve bir çıktı dosyası adı girmesini isteyen bir program yazın ve gir di dosyasının şifrelenmiş versiyonunu çıktı dosyasına kaydedin.

**19.15 (Dosyaları şifre çözme)** Bir dosyanın Programlama Alıştırması 19.14'teki şemayı kullanarak şifrelendiğini varsayalım. Şifrelenmiş bir dosyayı çözmek için bir program yazın. Programınız, şifrelenmiş dosya için bir girdi dosyası adı ve girdi dosyasının şifresiz versiy onu için bir çıktı dosyası adı girmesini istemelidir.

**19.16 (Karakterlerin frekansı)** Kullanıcının bir ASCII metin dosyasının adını girmesini isteyen ve dosyadaki karakterlerin frekansını gösteren bir program yazın.

**\*\*19.17 (BitOutputStream)** Bir çıktı akışına bit yazmak için Şekil 19.21'de gösterildiği gibi **BitOutputStream** adında bir sınıf uygulayın. **writeBit(char bit)** metodu, biti bir bayt değişkeninde saklar. Bir **BitOutputStream** oluşturduğunuzda, bayt boştur. **writeBit('1')** çağrıldıktan sonra, bayt 00000000 olur. **writeBit("0101")** çağrıldıktan sonra, bayt 00010101 olur. İlk üç bit henüz doldurulmamıştır. Bir bayt dolduğunda, çıktı akışına gönderilir. Şimdi bayt boş olarak sıfırlanır. Akışı kapatmak için **close()** metodunu çağırmalısınız. Eğer bayt ne boş ne de doluysa, **close()** metodu önce baytta tam 8 bit yapmak için sıfırları doldurur ve ardından baytı çıkarır ve akışı kapatır. İpucu için, Programlama Alıştırması 4.46'ya bakın. Bitleri 010000100100001001101 adlı dosyaya gönderen bir test programı yazın.

BitTÇıkış Akışı	
+BitOutputStream(dosya: File)	Creates a BitOutputStream to writes bits to the file.
+writeBit(char bit): void	Writes a bit '0' or '1' to the output stream.
+writeBit(String bit): void	Writes a string of bits to the output stream.
+close(): void	This method must be invoked to close the stream.

FIGURE 19.21 **BitOutputStream** bir dosyaya bit akışı çıktısı verir.

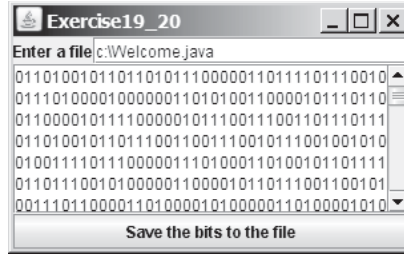
**\*19.18 (Bitleri görüntüle)** Bir tam sayının son baytının bit temsilini gösteren aşağıdaki metodu yazın:

```
public static String getBits(int tintValue)
```

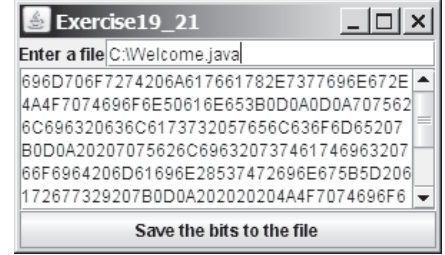
Bir ipucu için, Programlama Egzersizi 4.46'ya bakın. Kullanıcıdan bir dosya adı girmesini isteyen, dosyadan bayt okuyan ve her baytın ikili temsili gösteren bir program yazın.

**\*19.19** (*Onaltılık görüntüle*) Kullanıcıdan bir dosya adı girmesini isteyen, dosyadan bayt okuyan ve her baytın onaltılık temsili gösteren bir program yazın. (*İpucu:* Önce bayt değerini 8 bitlik bir dizeye dönüştürebilir, ardından bit dizisini iki haneli bir onaltılık dizeye dönüştürebilirsiniz.)

**\*\*19.20** (*İkili düzenleyici*) Kullanıcının metin alanına bir dosya adı girmesine ve *Enter* tuşuna basarak dosyanın ikili temsili bir metin alanında görüntülemesine olanak tanıyan bir GUI uygulaması yazın. Kullanıcı ayrıca ikili kodu değiştirebilir ve dosyaya geri kaydedebilir; bu, Şekil 19.22a'da gösterilmiştir.



(a)



(b)

**FIGURE 19.22** Programlar kullanıcının dosyanın içeriğini

(a) ikili ve (b) onaltılık olarak manipüle etmesine olanak tanır.

**\*\*19.21** (*Hex editörü*) Kullanıcının bir dosya adını metin alanına girmesine ve *Enter* tuşuna basarak dosyanın hex temsili bir metin

alanında görüntülemesine olanak tanıyan bir GUI uygulaması yazın. Kullanıcı ayrıca hex kodunu değiştirebilir ve dosyaya geri kaydedebilir, Şekil 19.22b'de gösterildiği gibi.



# BÖLÜM

# 20

## RÖZYİNELEME

### Hedefler

- Özyinelemeli bir yöntemin ne olduğunu ve özyineleme kullanmanın faydalarını tanımlamak (§20.1).
- Özyinelemeli matematiksel fonksiyonlar için özyinelemeli yöntemler geliştirmek (§§20.2–20.3).
- Özyinelemeli yöntem çağrılarının bir çağrı yığnında nasıl işlendiğini açıklamak (§§20.2–20.3).
- Özyineleme kullanarak problemleri çözmek (§20.4).
- Özyinelemeli bir yöntemi tasarlamak için aşırı yüklenmiş bir yardımcı yöntem kullanmak (§20.5).
- Özyineleme kullanarak bir seçim sıralaması uygulamak (§20.5.1).
- Özyineleme kullanarak bir ikili arama uygulamak (§20.5.2).
- Özyineleme kullanarak dizin boyutunu almak (§20.6).
- Hanoi Kuleleri problemini özyineleme kullanarak çözmek (§20.7).
- Özyineleme kullanarak fraktallar çizmek (§20.8).
- Özyineleme ve yineleme arasındaki ilişkiyi ve farkı keşfetmek (§20.9).
- Kuyruk özyinelemeli yöntemleri bilmek ve neden arzu edildiklerini anlamak (§20.10).



## 20.1 Giriş



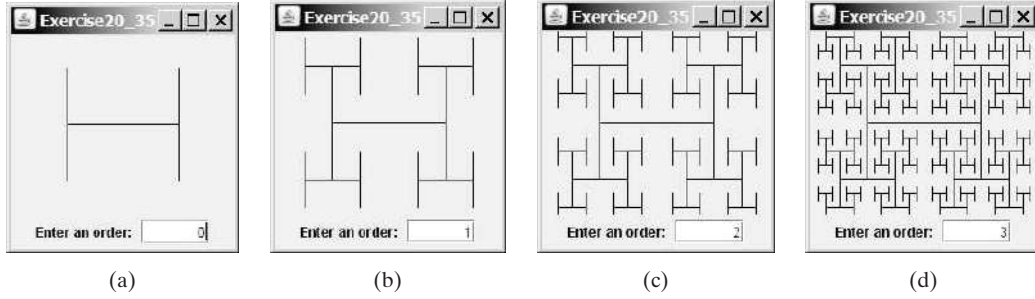
*Özyineleme, basit döngüler kullanarak programlaması zor olan problemlere zarif çözümler getiren bir tekniktir.*

arama kelimesi problemi

Bir dizin altında belirli bir kelimeyi içeren tüm dosyaları bulmak istediğinizi varsayalım. Bu problemi nasıl çözersiniz? Bunu yapmanın birkaç yolu vardır. Sezgisel ve etkili bir çözüm, alt dizinlerdeki dosyaları özyinelemeli olarak aramaktır.

H-ağaç problemi

Şekil 20.1'de gösterilen H-ağaçları, bir çipin tüm parçalarına eşit yayılma gecikmeleri ile zamanlama sinyallerini yönlendirmek için saat dağıtım ağı olarak çok büyük ölçekli entegrasyon (VLSI) tasarımında kullanılır. H-ağaçlarını görüntülemek için bir programı nasıl yazarsınız? İyi bir yaklaşım, özyineleme kullanmaktır.



**FIGÜ20.1** Bir H-ağacı özyineleme kullanılarak görüntülenebilir.

özyinelemeli yöntem

Özyineleme kullanmak, kendini çağıran yöntemler kullanarak programlama yapmaktır. Özyineleme, yararlı bir programlama tekniğidir. Bazı durumlarda, aksi takdirde zor bir problemi doğal, basit ve doğrudan bir çözümle geliştirmenizi sağlar. Bu bölüm, özyinelemeli programlama kavramlarını ve tekniklerini tanıtır ve "özyinelemeli düşünmeyi" nasıl yapacağınızı örneklerle gösterir.

## 20.2 Vaka Çalışması: Faktöriyel Hesaplama



*Özyinelemeli bir yöntem, kendini çağıran bir yöntemdir.*

Birçok matematiksel fonksiyon, özyineleme kullanılarak tanımlanır. Basit bir örnekle başlayalım. Bir sayının faktöriyeli  $n$  aşağıdaki gibi özyinelemeli olarak tanımlanabilir:

$$\begin{aligned} 0! &= 1; \\ n! &= n \times (n - 1)!; \quad n > 0 \end{aligned}$$

Verilen bir  $n$  için  $n!$  nasıl bulunur?  $1!$  bulmak kolaydır, çünkü  $0! = 1$  olduğunu biliyorsunuz ve  $1! = 1 \times 0!$ 'ye eşittir.  $(n - 1)!$  bildiğinizi varsayarsak,  $n!$ 'yi hemen  $n \times (n - 1)!$  kullanarak elde edebilirsiniz. Böylece,  $n!$  hesaplama problemi,  $(n - 1)!$  hesaplama problemine indirgenir.  $(n - 1)!$  hesaplanırken,  $n - 0$ 'a indirgenene kadar aynı fikri özyinelemeli olarak uygulayabilirsiniz.

$\text{factorial}(n)$ ,  $n!$  hesaplamak için bir yöntem olsun. Yöntemi  $n = 0$  ile çağırırsanız, hemen sonucu döndürür. Yöntem, en basit durumu çözmeyi bilir, bu duruma temel durum veya durdurma koşulu denir. Yöntemi  $n > 0$  ile çağırırsanız, problemi  $n - 1$  faktöriyelini hesaplamak için bir alt probleme indirger. Alt problem, özünde orijinal problemle aynı, ancak daha basit veya daha küçüktür. Alt problemin orijinal problemle aynı özelliğe sahip olduğu için, farklı bir argümanla yöntemi çağırabilirsiniz, bu duruma özyinelemeli çağrı denir.

Faktöriyel( $n$ ) hesaplamak için özyinelemeli algoritma basitçe şu şekilde tanımlanabilir:

```
if (n == 0)
 return 1;
```

temel durum veya durdurma koşulu

özyinelemeli çağrı

aksi takdirde

**return** n \* faktöriyel(n - 1);

Bir özyinelemeli çağrı, birçok başka özyinelemeli çağrıya yol açabilir, çünkü yöntem bir alt problemi sürekli olarak yeni alt problemlere ayırmaya devam eder. Bir özyinelemeli yöntemin sona ermesi için, problem nihayetinde bir durdurma durumuna indirgenmelidir; bu noktada yöntem, çağrısına bir sonuç döndürür. Çağrıci daha sonra bir hesaplama yapar ve sonucu kendi çağrısına döndürür. Bu süreç, sonuç orijinal çağrıya geri verilene kadar devam eder. Orijinal problem artık n ile faktöriyel(n - 1) sonucunu çarparak çözülebilir.

Liste 20.1, kullanıcıdan negatif olmayan bir tam sayı girmesini isteyen ve

sayının faktöriyelini gösteren tam bir programı vermektedir.

## LİSTE 20.1 ComputeFactorial.java

```

1 import java.util.Scanner;
2
3 genel classt ComputeFactorial {
4 /** Ana yöntem */
5 public static void tmain(String[] args) {
6 // Bir Scanner oluştur
7 Scanner input = new Scanner(System.in);
8 System.out.print("Pozitif bir tam sayı girin: ");
9 int n = input.nextInt();
10
11 // Faktöriyel göster
12 System.out.println("Faktöriyel " + n + " şudur " + faktöriyel(n));
13 }
14
15 /** Belirtilen sayı için faktöriyel döndür */
16 genel statik uzun faktöriyel(int n) {
17 if (n == 0) // Temel durum
18 return 1;
19 aksi takdirde
20 geri dönmek n * faktöriyel(n - 1); // Özyinelemeli çağrı
21 }
22 }

```

yirmi iki }

temel durum

özyineleme

Negatif olmayan bir tam sayı girin: 4  
4'ün faktöriyeli 24'tür

4

↵ Enter



Negatif olmayan bir tam sayı girin: 10  
10'un faktöriyeli 3628800'dür

10

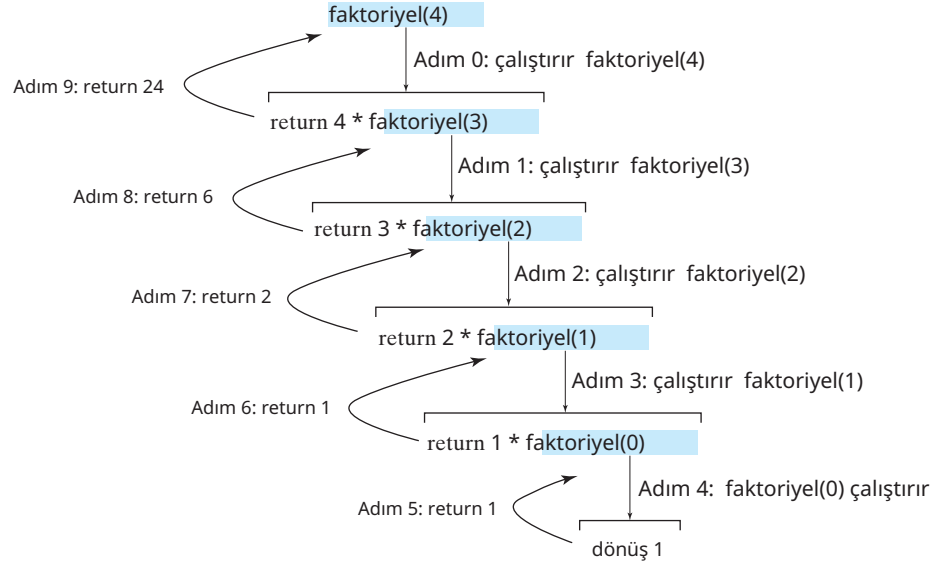
↵ Enter



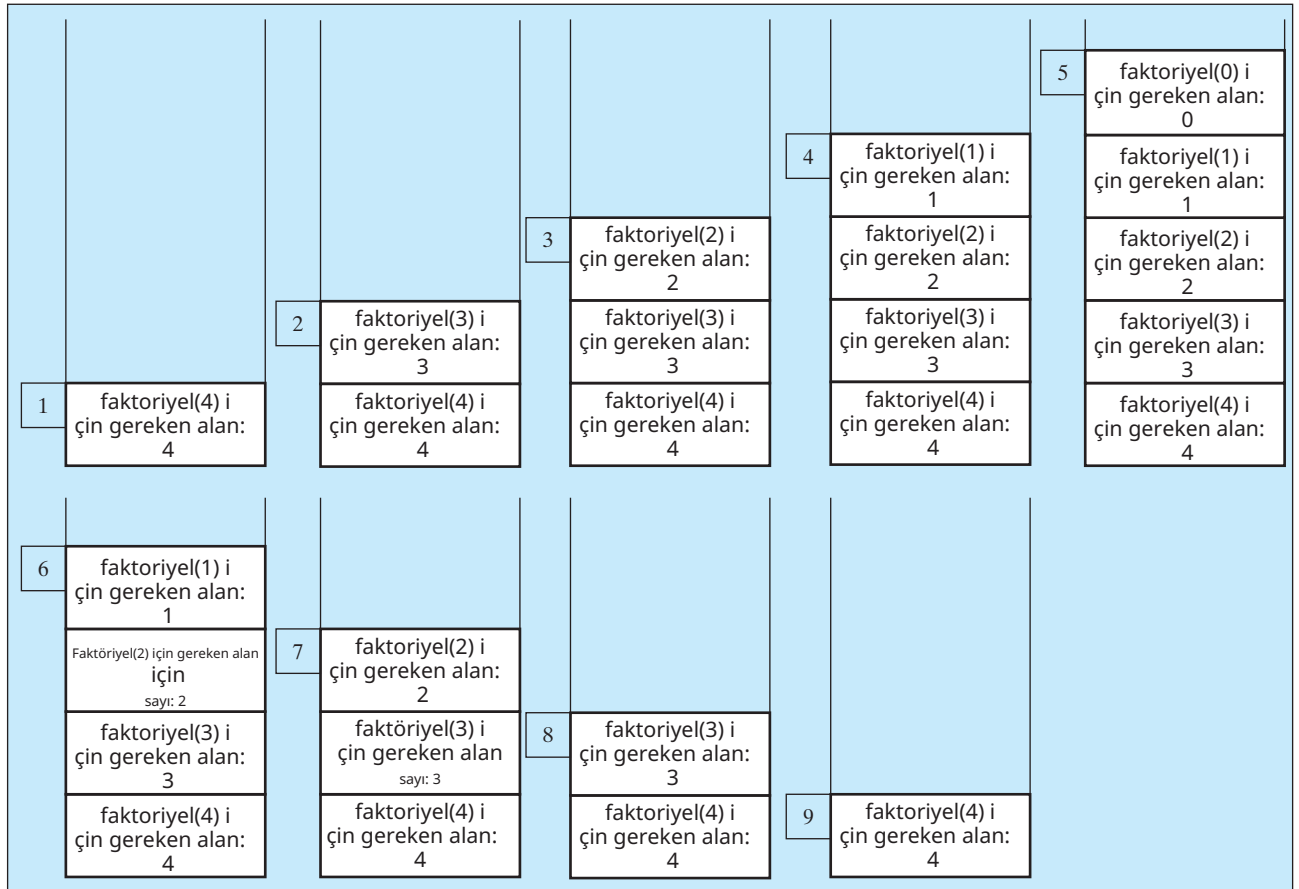
Faktöriyel metodu (satırlar 16–21), faktöriyel için özyinelemeli matematiksel tanımın Java koduna doğrudan bir çevirisidir. Faktöriyel çağrısı özyinelemelidir çünkü kendini çağırır. Faktöriyeye geçirilen parametre, temel durum olan 0'a ulaşana kadar azaltılır.

Özyinelemeli bir metodu nasıl yazacağınızı gördünüz. Özyineleme nasıl çalışır? Şekil 20.2, n = 4 ile başlayan özyinelemeli çağrıların yürütülmesini göstermektedir. Özyinelemeli çağrılar için yığın alanının kullanımı Şekil 20.3'te gösterilmektedir.

nasıl çalışır?



**FIGURE 20.2** faktoriyel(4) çağrıları faktoriyel oluşturur.



**FIGURE 20.3** faktöriyel(4) çalıştırıldığında, faktöriyel metodu özyinelemeli olarak çağrılır ve yığın alanı dinamik olarak değişir.

**Petdagtotgtitcal Ntote**

faktöriyel metodunu bir döngü kullanarak uygulamak daha basit ve daha etkilidir. Ancak, burada özyineleme kavramını göstermek için özyinelemeli **faktöriyel** metodunu kullanıyoruz. Bu bölümün ilerleyen kısımlarında, özyinelemeli olan ve özyineleme kullanmadan çözülmesi zor olan bazı problemleri sunacağız.

Eğer özyineleme, problemi nihayetinde temel duruma yakınsamasını sağlayacak şekilde a zaltmazsa, *sonsuz özyineleme* meydana gelebilir. Örneğin, **faktöriyel** metodunu aşağıdaki i gibi yanlış yazdığınızı varsayalım:

sonsuz özyineleme

```
genel statik uzun faktöriyel(int n) {
 geri dönmek n * faktöriyel(n - 1);
}
```

Metod sonsuz bir şekilde çalışır ve bir **StackOverflowError** oluşturur.

Bu bölümde tartışılan örnek, kendisini çağıran bir özyinelemeli yöntemi göstermektedir. Bu, *doğrudan özyineleme* olarak bilinir. Ayrıca *dolaylı özyineleme* oluşturmak da mümkündür. Bu, yöntem **A**'nı n, yöntem **B**'yi çağırdığı ve bunun da yöntem **A**'yı çağırdığı durumlarda meydana gelir. Özyinelemede daha fazla yöntem de yer alabilir. Örneğin, yöntem **A**, yöntem **B**'yi çağırır, bu da yöntem **C**'yi çağırır, bu da yöntem **A**'yı çağırır.

doğrudan özyineleme  
dolaylı özyineleme

**20.1** Özyinelemeli bir yöntem nedir? Sonsuz özyineleme nedir?

**20.2** Listing 20.1'deki **faktöriyel** yöntemi **faktöriyel(6)** için kaç kez çağrılır?

**20.3** Aşağıdaki programların çıktısını gösterin ve temel durumları ve özyinelemeli çağrılarını tanımlayın.

Nokta Check

MyProgrammingLab™

```
genel sınıfTest {
 public static void main(String[] args) {
 System.out.println(
 "Toplam " + xMethod(5));
 }

 public static int xMethod(int t) {
 geri dönme
 k
 return t + xMethod(n - t);
 }
}
```

```
genel sınıfTest {
 public static void tmain(String[] args) {
 nxMethod(1234567);
 }

 public static void txMethod(int t) {
 if (t > 0) {
 System.out.print(n % t);
 nxMethod(n / t);
 }
 }
}
```

**20.4** Pozitif bir tam sayı için hesaplama yapan bir özyinelemeli matematiksel tanım yazın

pamı **pozitif bir tam sayı n için.**

**20.5** Pozitif bir tam sayı için hesaplama yapan bir özyinelemeli matematiksel tanım yazın ve bir reel sayı  $x$  için.

$x$  **pozitif bir tam sayı n**

**20.6** Bir özyinelemeli matematiksel tanım yazın if bir tamsayı için.

$1 + 2 + 3 + \dots + t + n$  **pozitif**

## 20.3 Vaka Çalışması: Fibonacci Sayılarını Hesaplama

Bazı durumlarda, *özyineleme*, bir probleme sezgisel, basit ve doğrudan bir çözüm oluşturmanıza olanak tanır.



Önceki bölümdeki faktöriyel yöntemi, özyineleme kullanmadan kolayca yeniden yazılabilir. Bu bölümde, özyineleme kullanarak bir probleme sezgisel bir çözüm oluşturma örneğini gösteriyoruz. İyi bilinen Fibonacci serisi problemini düşünün:

The series: 0 1 1 2 3 5 8 13 21 34 55 89 ...  
 indices: 0 1 2 3 4 5 6 7 8 9 10 11

Fibonacci serisi, **0** ve **1** ile başlar ve her bir sonraki sayı, önceki iki sayının toplamıdır. Seri, özyinelemeli olarak şöyle tanımlanabilir:

```
fib(0) = 0;
fib(1) = 1;
fib(index) = fib(index - 2) + fib(index - 1); index >= 2
```

Fibonacci serisi, tavşan popülasyonunun büyümesini modellemek için ortaya atan ortaçağ matematikçisi Leonardo Fibonacci'nin adını almıştır. Sayısal optimizasyon ve çeşitli diğer alanlarda uygulanabilir.

Verilen bir **index** için **fib(index)** nasıl bulunur? **fib(2)**'yi bulmak kolaydır, çünkü **fib(0)** ve **fib(1)**'yi biliyorsunuz. **fib(index - 2)** ve **fib(index - 1)**'yi bildiğinizi varsayarsak, **fib(index)**'yi hemen elde edebilirsiniz. Böylece, **fib(index)**'yi hesaplama problemi, **fib(index - 2)** ve **fib(index - 1)**'yi hesaplamaya indirgenir. Bunu yaparke

n, **index0** veya **1**'e düşene kadar fikri özyinelemeli olarak uygularsınız.

Temel durum **index = 0** veya **index = 1**'dir. Eğer metodu **index = 0** veya **index = 1** ile çağırırsanız, hemen sonucu döner. Eğer metodu **index >= 2** ile çağırırsanız, problemi two alt probleme ayırır ve **fib(index - 1)** ve **fib(index - 2)** hesaplamak için özyinelemeli çağrılar kullanır. **fib(index)** hesaplamak için özyinelemeli algoritma basitçe şu şekilde tanımlanabilir:

```
eğer(index == 0)
 return 0;
aksi takdirde eğer(index == 1)
 return 1;
aksi takdirde
 return fib(index - 1) + fib(index - 2);
```

Liste 20.2, kullanıcıdan bir indeks girmesini isteyen ve o indeks için Fibonacci sayısını hesaplayan bir programı tam olarak vermektedir.

## LISTE 20.2 ComputeFibonacci.java

```
1 import java.util.Scanner;
2
3 genel sınıf ComputeFibonacci {
4 /** Ana yöntem */
5 public static void main(String[] args) {
6 // Bir Scanner oluştur
7 Scanner input = new Scanner(System.in);
8 System.out.print("Bir Fibonacci sayısı için bir indeks girin: ");
9 int index = input.nextInt();
10
11 // Fibonacci sayısını bul ve göster
12 System.out.println("İndeksdeki Fibonacci sayısı "
13 + index + " şudur: " + fib(index));
14 }
15
16 /** Fibonacci sayısını bulma yöntemi */
17 genel statik uzun fib(long index) {
18 if(index == 0) // Temel durum
19 return 0;
```

```

20 aksi takdirde eğer(index == 1)// Temel durum temel durum
21 return 1;
22 aksi takdirde// Azaltma ve özyinelemeli çağrılar
23 geri dönmek fib(indeks - t1t) + fib(indeks - t2t) ; özyineleme
n24 }
25 }

```

Bir Fibonacci sayısı için bir indeks girin:  
İndeks 1'deki Fibonacci sayısı 1'dir

1 Enter



Bir Fibonacci sayısı için bir indeks girin:  
İndeks 6'daki Fibonacci sayısı 8'dir

6 Enter



Bir Fibonacci sayısı için bir indeks girin:  
İndeks 7'deki Fibonacci sayısı 13'tür

7 Enter



Program, bilgisayar tarafından sahne arkasında yapılan önemli miktarda işi göstermez. Ancak Şekil 20.4, **fib(4)**'ü değerlendirmek için ardışık özyinelemeli çağrılarını gösterir. Orijinal yöntem, **fib(4)**, iki özyinelemeli çağrı yapar, **fib(3)** ve **fib(2)**, ve ardından **fib(3) + fib(2)** döner. Ancak bu yöntemler hangi sırayla çağrılır? Java'da, operatörler soldan sağa doğru değerlendirilir, bu nedenle **fib(2)**, **fib(3)** tamamen değerlendirildikten sonra çağrılır. Şekil 20.4'teki etiketler, yöntemlerin çağrıldığı sırayı gösterir.

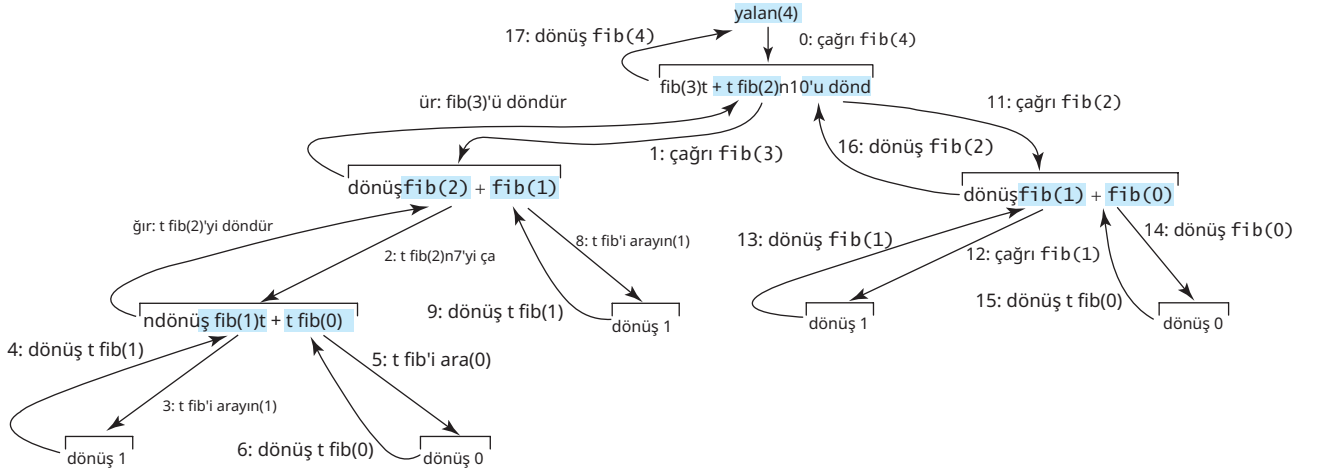


FIGURE 20.4 **fib(4)** çağrısı **fib** için özyinelemeli çağrılar oluşturur.

Şekil 20.4'te gösterildiği gibi, birçok tekrarlayan özyinelemeli çağrı vardır. Örneğin, **fib(2)** iki kez çağrılır, **fib(1)** üç kez ve **fib(0)** iki kez çağrılır. Genel olarak, **fib(index)** hesaplamak, **fib(index - 1)** hesaplamaktan yaklaşık iki kat daha fazla özyinelemeli çağrı gerektirir. Daha büyük indeks değerlerini denedikçe, çağrı sayısı önemli ölçüde artar; bu, Tablo 20.1'de gösterilmiştir.

Tablo 20.1 **fib(index)** için Özyinelemeli Çağrı Sayısı

indeks	2	3	4	10	20	30	40	50
arama sayısı	3	5	9	177	21891	2,692,537	331,160,281	2,075,316,483

**Petdagtotgtitcal Ntote**

fib yönteminin özyinelemeli uygulaması çok basit ve doğrudandır, ancak verimli değildir, çünkü özyinelemeli yöntemleri çalıştırmak için daha fazla zaman ve bellek gerektirir. Daha verimli bir çözüm için döngüler kullanarak Programlama Egzersizi 20.2'ye bakın. Pratik olmasa da, özyinelemeli fib yöntemi, özyinelemeli yöntemler yazmanın iyi bir örneğidir.

**20.7** Aşağıdaki iki programın çıktısını gösterin:

Nokta ✓ Check

MyProgrammingLab™

```
genel sınıfTest {
 public static voidtmain(String[] args) {nxMethod
 (t5t);
 }

 public static voidtxMethod(int n) {nift(n > t0t)
 {
 System.out.print(n + t" "t);n}
 xMethod(n - t1t);
 }
 }
}
```

```
genel sınıfTest {
 public static voidtmain(String[] args) {nxMethod
 (t5t);
 }

 public static voidtxMethod(tinttn) {nift(n > t0
 t) {
 xMethod(n - t1t);
 System.out.print(n + t" "t);n}
 }
 }
}
```

**20.8** Aşağıdaki yöntemde ne yanlış?

```
genel sınıfTest {
 public static voidtmain(String[] args) {nxMethod
 (t1234567t);
 }

 public static voidtxMethod(double n) {nif
 (n != t0t) {
 System.out.print(n);nxM
 ethod(n / t10t);
 }
 }
}
```

```
genel sınıfTest {
 public static voidtmain(String[] args) {nTest test
 = tnewtTest();
 System.out.println(test.toString());n}

 publictTest() {nTe
 st testi = tnewtTest();n}
 }
}
```

**20.9** Listing 20.2'deki **fib** yöntemi **fib(6)** için kaç kez çağrılıyor?**20.4 Problem Çözme Rekürsiyon Kullanarak**Keyn  
Point

*Eğer rekürsif düşünüyorsanız, birçok problemi rekürsiyon kullanarak çözebilirsiniz.*

Önceki bölümler iki klasik rekürsiyon örneği sundu. Tüm rekürsif yöntemlerin takip eden özellikleri vardır:

özyineleme özellikleri

eğer-değilse

temel durumlar

azaltma

- Yöntem, farklı durumlara yol açan bir **if-else** veya **switch** ifadesi kullanılarak uygulanır.
- Bir veya daha fazla temel durum (en basit durum) rekürsiyonu durdurmak için kullanılır.
- Her rekürsif çağrı, orijinal problemi azaltır ve onu giderek daha fazla bir temel duruma yaklaştırır, ta ki o durum haline gelene kadar.

Genel olarak, bir problemi rekürsiyon kullanarak çözmek için, onu alt problemlere ayırırsınız. Her alt-problem, orijinal problemle aynı ancak daha küçük boyuttadır. Her alt-problemi çözmek için aynı yaklaşımı rekürsif olarak uygulayabilirsiniz.



Rekursyon her yerde. Rekursif düşünmek eğlencelidir. Kahve içmeyi düşünün. Süreci rekursif olarak şu şekilde tanımlayabilirsiniz:

özyinelemeli düşün

```
public static void drinkCoffee(Cup cup) {nift(!cup.is
 Empty()) {
 cup.takeOneSip(); // Bir yudum al
 drinkCoffee(cup);
 }
}
```

Bir fincanın bir nesne olduğunu varsayalım ve bu nesnenin **isEmpty()** ve **takeOneSip()** gibi örnek yöntemleri var. Problemi iki alt probleme bölebilirsiniz: biri bir yudum kahve içmek ve diğeri fincandaki kahvenin geri kalanını içmektir. İkinci problem, orijinal problemle aynı ancak daha küçük bir boyuttadır. Problemin temel durumu, fincanın boş olduğu zamandır.

Bir mesajı **n** kez yazdırma problemini düşünün. Problemi iki alt probleme bölebilirsiniz: biri mesajı bir kez yazdırmak ve diğeri **n - 1** kez yazdırmaktır. İkinci problem, orijinal problemle aynı ama boyutu daha küçüktür. Problemin temel durumu **n == 0**'dır. Bu problemi aşağıdaki gibi özyineleme kullanarak çözebilirsiniz:

```
public static void printN(String message, int times) {nift(times >=
 1) {
 System.out.println(message);
 printN(message, times - 1);
 } // Temel durum times == 0'dır
}
```

özyinelemeli çağrı

Dikkat edin ki, önceki bölümdeki **fib** metodu çağrısına bir değer döndürür, ancak **drinkCoffee** ve **printN** metodları **void**'dir ve değer döndürmezler.

Eğer *özyinelemeli düşünüyorsanız*, bu kitapta daha önce sunulan birçok problemi çözmek için özyineleme kullanabilirsiniz. Liste 9.1'deki palindrom problemini düşünün. Bir dizinin soldan ve sağdan aynı şekilde okunması durumunda palindrom olduğunu hatırlayın. Örneğin, "mom" ve "dad" palindromdur, ancak "uncle" ve "aunt" değildir. Bir dizinin palindrom olup olmadığını kontrol etme problemi iki alt probleme ayrılabilir:

özyinelemeli düşün

- Dizin ilk karakterinin ve son karakterinin eşit olup olmadığını kontrol edin.
- İki uç karakteri göz ardı edin ve geri kalan alt dizeyi bir

palindrom olup olmadığını kontrol edin.

İkinci alt problem, orijinal problemin aynı ancak daha küçük boyutudur. İki temel durum vardır: (1) iki uç karakter aynı değildir ve (2) dize boyutu **0** veya **1**'dir. 1. durumda, dize bir palindrom değildir; 2. durumda, dize bir palindromdur. Bu problem için özyinelemeli yöntem, Liste 20.3'te gösterildiği gibi uygulanabilir.

## LİSTE 20.3 Substring Kullanarak Özyinelemeli Palindrom.java

```
1 public class SubstringKullanarakÖzyinelemeliPalindrom {
2 public static boolean isPalindrome(String s) {
3 if (s.length() <= 1) // Temel durum
4 geri dönüş tehdidi;
5 else if (s.charAt(0) != s.charAt(s.length() - 1)) // Temel durum
6 yanlış döndür;
7 aksi takdirde
8 geri dönmek isPalindrome(s.substring(1, s.length() - 1));
9 }
10 }
11 public static void main(String[] args) {
```

metot başlığı

temel durum

temel durum

özyinelemeli çağrı

```

12 System.out.println("Ay bir palindrom mu? "
13 + isPalindrome("moon"));
14 System.out.println("Öğle bir palindrom mu? "
15 + isPalindrome("noon"));
16 System.out.println("a bir palindrom mu? "+ isPalindrome("a"));
17 System.out.println("aba bir palindrom mu? "+
18 Palindrome("aba"));
19 System.out.println("ab bir palindrom mu? "+ isPalindrome("ab"));
20 }
 yirmi bir }

```



```

Ay bir palindrom mu? false
Öğle bir palindrom mu? true
A bir palindrom mu? true
Aba bir palindrom mu? true
Ab bir palindrom mu? false

```

8. satırdaki **substring** metodu, orijinal string ile aynı olan yeni bir string oluşturur ilk ve son karakterler hariç. Bir string'in palindrom olup olmadığını kontrol etmek, orijinal string'deki iki uç karakter aynıysa, substring'in palindrom olup olmadığını kontrol etmekle eşdeğerdir.

**20.10** Rekürsif metodların özelliklerini tanımlayın.

**20.11** Liste 20.3'teki **isPalindrome** metodu için, temel durumlar nelerdir? **isPalindrome("abdcxcdba")** çağrıldığında bu metod kaç kez çağrılır?

**20.12** Liste 20.3'te tanımlanan metodu kullanarak **isPalindrome("abcba")** için çağrı yığını nı gösterin.



MyProgrammingLab™

## 20.5 Rekürsif Yardımcı Metodlar



*Bazen orijinal problemi biraz değiştirerek rekürsif bir çözüm bulabilirsiniz. Bu yeni metoda rekürsif yardımcı metod denir.*

Listing 20.3'teki özyinelemeli **isPalindrome** metodu verimsizdir, çünkü her özyineleme çağrısı için yeni bir string oluşturur. Yeni stringler oluşturmaktan kaçınmak için, alt ve üst indeksleri alt dize aralığını belirtmek için kullanabilirsiniz. Bu iki indeks, özyinelemeli metoda geçirilmelidir. Orijinal metod **isPalindrome(String s)** olduğundan, string hakkında ek bilgi almak için **isPalindrome(String s, int low, int high)** adlı yeni metodu oluşturum Listing 20.4'te gösterildiği gibi.

### LİSTELER 20.4 RecursivePalindrome.java

```

1 public class ÖzyinelemeliPalindrom {
2 public static boolean isPalindrome(String s) {
3 return isPalindrome(s, 0, s.length() - 1);
4 }
5
6 özel statik boolean isPalindrome(String s, int low, int high) {
7 eğer sek <= düşük // Temel durum
8 doğruyu döndür;
9 aksi takdirde eğer (s.charAt(düşük) != s.charAt(yüksek)) // Temel durum
10 yanlış döndür;
11 aksi takdirde
12 return isPalindrome(s, düşük + 1, yüksek - 1);
13 }
14 }

```

yardımcı yöntem

temel durum

temel durum

```

15 public static void tmain(String[] args) {
16 System.out.println("Moon bir palindrom mu? "
17 + isPalindrome("moon"));
18 System.out.println("Noon bir palindrom mu? "
19 + isPalindrome("noon"));
20 System.out.println("A bir palindrom mu? "+ isPalindrome("a"));
21 System.out.println("ABA bir palindrom mu? "+ isPalindrome("aba"));
22 System.out.println("AB bir palindrom mu? "+ isPalindrome("ab"));
23 }

```

yirmidört }

İki aşırı yüklenmiş `isPalindrome` yöntemi tanımlanmıştır. İlk olarak, `isPalindrome(String s)`, bir dizinin palindrom olup olmadığını kontrol eder ve ikincisi, `isPalindrome(String s, int low, int high)`, bir alt dize `s(low..high)` palindrom mu kontrol eder. İlk yöntem, diziyi `s` ile `low = 0` ve `high = s.length() - 1` ikinci yönteme geçirir. İkinci yöntem, sürekli küçülen bir alt dizide palindrom kontrol etmek için yinelemeli olarak çağrılabilir. Yinelemeli programlamada, ilave parametreler alan ikinci bir yöntem tanımlamak yaygın bir tasarım tekniğidir. Böyle bir yöntem *yinelemeli yardımcı yöntem* olarak bilinir.

özyinelemeli yardımcı yöntem

Yardımcı yöntemler, dizeler ve dizilerle ilgili problemler için özyinelemeli çözümler tasarlarken çok faydalıdır. Aşağıdaki bölümler iki örnek daha vermektedir.

### 20.5.1 Özyinelemeli Seçim Sıralaması

Seçim sıralaması Bölüm 6.11.1'de tanıtılmıştır. En küçük öğeyi bulup listeyi ilk öğe ile değiştirdiğini hatırlayın. Daha sonra kalan en küçük öğeyi bulur ve kalan listedeki ilk öğe ile değiştirir ve bu şekilde devam eder, ta ki kalan liste yalnızca bir öğe içerecek şekilde. Problem iki alt probleme ayrılabilir:

- Listede en küçük öğeyi bul ve onu ilk öğe ile değiştir.
- İlk öğeyi göz ardı et ve kalan daha küçük listeyi özyinelemeli olarak sırala.

Temel durum, listenin yalnızca bir öğe içermesidir. Liste 20.5, özyinelemeli

sıralama yöntemini vermektedir.

### Listing 20.5 RecursiveSelectionSort.java

```

1 public class RecursiveSelectionSort {
2 public static void sort(double[] liste) {
3 sort(liste, 0, liste.uzunluk - 1); // Tüm listeyi sırala
4 }
5
6 // özel statik void sort(double[] liste, int low, int high) {
7 // eğer (low < high) {
8 // // Listede en küçük sayıyı ve indeksini bul
9 // int indexOfMin = düşük;
10 // double min = liste[düşük];
11 // için (int i = low + 1; i <= high; i++) {
12 // eğer (liste[i] < min) {
13 // min = liste[i];
14 // indexOfMin = i;
15 // }
16 }
17
18 // list[low .. high] içindeki en küçük elemanı list[low] ile değiştir
19 list[indexOfMin] = list[low];
20 list[low] = min;
21

```

yardımcı yöntem  
temel durum

özyinelemeli çağrı

```

22 // Kalan list[low+1 .. high]'ı sırala
23 sort(liste, düşük + t1t, yüksek);
24 }n25 }n2
6 }

```

İki aşırı yüklenmiş **sort** yöntemi tanımlanmıştır. İlk yöntem, **sort(double[] list), list[0..list.length - 1]** içindeki bir diziyi sıralar ve ikinci yöntem, **sort(double[] list, int low, int high), list[low..high]** içindeki bir diziyi sıralar. İkinci yöntem her zaman küçüklen bir alt diziyi sıralamak için özyinelemeli olarak çağrılabilir.



VideoNotu  
İkili arama

## 20.5.2 Özyinelemeli İkili Arama

İkili arama, Bölüm 6.10.2'de tanıtılmıştır. İkili aramanın çalışabilmesi için, dizideki elemanların artan bir sırada olması gerekir. İkili arama önce anahtarı dizinin ortasındaki elemanla karşılaştırır. Aşağıdaki üç durumu düşünelim:

- Durum 1: Eğer anahtar ortadaki elemandan küçükse, anahtarı dizinin ilk yarısında özyinelemeli olarak arayın.
- Durum 2: Eğer anahtar ortadaki elemanla eşitse, arama bir eşleşme ile sona erer.
- Durum 3: Eğer anahtar ortadaki elemandan büyükse, anahtarı dizinin ikinci yarısında özyinelemeli olarak arayın.

Durum 1 ve Durum 3 aramayı daha küçük bir listeye indirir. Durum 2, bir eşleşme olduğunda bir temel durumdur. Diğer bir temel durum, aramanın eşleşme olmadan tükenmesidir. Liste 20.6, özyinelemeli olarak ikili arama problemi için net, basit bir çözüm sunmaktadır.

### LİSTE 20.6 Özyinelemeli İkili Arama Yöntemi

yardımcı yöntem

temel durum

özyinelemeli çağrı

temel durum

özyinelemeli çağrı

```

1 public class ÖzyinelemeliİkiliArama {
2 public static int recursiveBinarySearch(tintt[] list, tinttkey) {
3 int düşük = 0;
4 int yüksek = liste.length - 1;
5 return özyinelemeliİkiliArama(liste, anahtar, düşük, yüksek);
6 }
7
8 özel static int recursiveBinarySearch(tintt[] list, tinttkey,
9 inttlow, inttthigh) {
10 eğer (düşük > yüksek) // Liste eşleşme olmadan tükenmiştir
11 return -düşük - 1;
12
13 int orta = (düşük + yüksek) / 2;
14 eğer (anahtar < liste[orta])
15 özyinelemeli İkili Aramayı döndür (liste, anahtar, düşük, orta - t1t);
16 aksi takdirde eğer (key == list[mid])
17 geri dönüş ortası;
18 aksi takdirde
19 özyinelemeli İkili Aramayı döndür (list, anahtar, orta + t1t, yüksek);
20 }n21
22 }

```

İlk yöntem, tüm listede bir anahtar bulur. İkinci yöntem,

**low** ile **high** arasındaki listede bir anahtar bulur.

İlk **binarySearch** yöntemi, **low = 0** ve **high =**

**list.length - 1** olan başlangıç dizisini ikinci **binarySearch** yöntemine geçirir. İkinci yöntem, anahtarı giderek küçüklen bir alt dizide bulmak için özyinelemeli olarak çağrılır.

**20.13** `isPalindrome("abcba")` için çağrı yığını göster Listing 20.4'te tanımlanan yöntemi kullanarak.

Nokta ✓ Check

**20.14** Çağrı yığını göster `selectionSort(new double[]{2, 3, 5, 1})` kullanarak Listing 20.5'te tanımlanan yöntemi.

MyProgrammingLab™

**20.15** Rekürsif yardımcı metod nedir?

## 20.6 Vaka Çalışması: Dizin Boyutunu Bulma

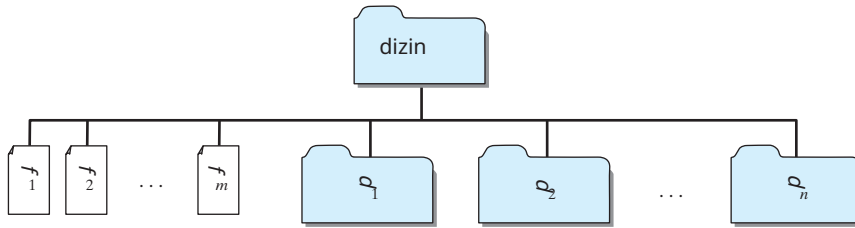
Özyinelemeli yöntemler, özyinelemeli yapılarla ilgili problemleri çözmek için etkilidir.

Önceki örnekler, özyineleme kullanmadan kolayca çözülebilir. Bu bölüm, özyineleme kullanmadan çözülmesi zor bir problemi sunmaktadır. Problem, bir dizinin boyutunu bulmaktır. Bir dizinin boyutu, dizindeki tüm dosyaların boyutlarının toplamıdır. Bir dizin, alt dizinler içerebilir. Varsayalım ki bir dizin dosyalar ve alt dizinler içeriyor.  $t_1, t_2, \dots, t_m$  ve alt dizinler  $d_1, d_2, \dots, d_n$  Şekil 20.5'te gösterildiği gibi.

Key nPoint

VideoNotu

Dizin boyutu



**FIGURE 20.5** Bir dizin dosyalar ve alt dizinler içerir.

Dizin boyutu aşağıdaki gibi özyinelemeli olarak tanımlanabilir:

$$size(d) = size(f_1) + size(f_2) + \dots + size(f_m) + size(d_1) + size(d_2) + \dots + size(d_n)$$

14.10. bölümde tanıtilan **File** sınıfı, bir dosyayı veya dizini temsil etmek için kullanılabilir ve dosyaların ve dizinlerin özelliklerini elde edebilir. Bu problem için **File** sınıfındaki iki yöntem faydalıdır:

■ **length()** yöntemi bir dosyanın boyutunu döndürür.

■ **listFiles()** yöntemi bir dizin altındaki **File** nesnelerinin bir dizisini döndürür.

Liste 20.7, kullanıcıdan bir dizin veya dosya girmesini isteyen ve

boyutunu gösteren bir programı vermektedir.

### Listing 20.7 DirectorySize.java

```

1 java.io.File'ı içe aktarın;
2 import java.util.Scanner;
3
4 genel class DirectorySize {
5 public static void main(String[] args) {
6 // Kullanıcıdan bir dizin veya dosya girmesini isteyin
7 System.out.print("Bir dizin veya dosya girin: ");
8 Scanner input = new Scanner(System.in);
9 String dizini = input.nextLine();
10
11 // Boyutu görüntüle
12 System.out.println(getSize(new File(directory)) + " bayt");
13 }
14 }

```

metodu çağır

## 750 Bölüm 20 Özyineleme

```
getSize metodu 15 public static long getSize(File dosya) {
 16 long size = 0; // Tüm dosyaların toplam boyutunu sakla
 17
 18 eğer (dosya.isDirectory()) {
dizin mi? 19 File[] dosyalar = dosya.listFiles(); // Tüm dosyalar ve alt dizinler
tüm alt öğeler 20 for(tintti = t0t; dosyalar != tnultt&& i < dosyalar.uzunluk; i++) {
özyinelemeli çağrı 21 boyut += getSize(dosyalar[i]); // Rekürsif çağrı
 22 {ıslatma}

 24 aksi takdirde{// Temel durum
temel durum 25 boyut += dosya.length();
 26 }
 27
 28 dönüş boyutu;
kes}na0}
```



Bir dizin veya dosya girin:  
48619631 bayt

c:\kitap



Bir dizin veya dosya girin:  
172 bayt

c:\kitap\Welcome.java



Bir dizin veya dosya girin:  
0 bayt

c:\kitap\MevcutOlmayanDosya



Eğer **dosya** nesnesi bir dizini temsil ediyorsa (satır 18), dizindeki her alt öğe (dosya veya alt dizin) özyinelemeli olarak çağrılır ve boyutu elde edilir (satır 21). Eğer **dosya** nesnesi bir dosyayı temsil ediyorsa (satır 24), dosya boyutu elde edilir (satır 25).

Yanlış veya mevcut olmayan bir dizin girildiğinde ne olur? Program bunun bir dizin olmadığını tespit edecek ve **dosya.length()** (satır 25) çağrılacak, bu da **0** döndürecektir. Böylece, bu durumda **getSize** metodu **0** döndürecektir.

tüm durumları test etme



**Ttip**

Hatalardan kaçınmak için, tüm durumları test etmek iyi bir uygulamadır. Örneğin, programı bir dosya girişi, boş bir dizin, var olmayan bir dizin ve var olmayan bir dosya için test etmelisiniz.

Nokta ✓ Check

MyProgrammingLab™

**20.16** **getSize** metodunun temel durumu nedir?

**20.17** Program, belirli bir dizin altındaki tüm dosya ve dizinleri nasıl alır?

**20.18** Eğer dizin üç alt dizine sahipse ve her alt dizinde dört dosya varsa, **getSize** metodu dizin için kaç kez çağrılacaktır?

## 20.7 Vaka Çalışması: Hanoi Kuleleri

*Hanoi Kuleleri problemi, kolayca çözülmesi mümkün olan klasik bir problem dir, ancak başka bir şekilde çözmek zordur.*

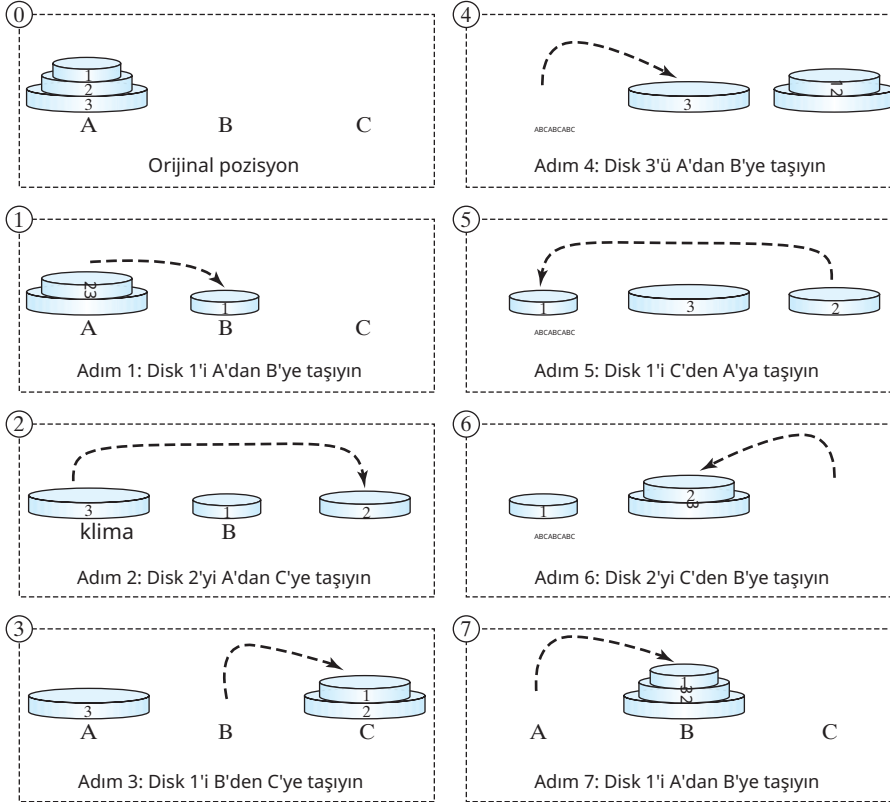
Problem, belirli sayıda farklı boyutlardaki diskin bir kuleden diğerine taşınmasını içerir ve aşağıdaki kurallara uyulmalıdır:

- 1, 2, 3, . . . ,  $n$  etiketli  $n$  disk vardır ve A, B ve C etiketli üç kule bulunmaktadır.
- Hiçbir disk, her zaman daha küçük bir diskin üstünde olamaz.



- Tüm diskler başlangıçta A kulesine yerleştirilmiştir.
- Aynı anda yalnızca bir disk taşınabilir ve bu, bir kuledeki en küçük disk olmalıdır.

Problemin amacı, tüm diskleri C'nin yardımıyla A'dan B'ye taşımaktır. Örneğin, eğer üç diskiniz varsa, tüm diskleri A'dan B'ye taşımak için gereken adımlar Şekil 20.6'da gösterilmiştir.



**FIGÜRİK 20.6** Hanoi Kuleleri probleminin amacı, diskleri A kulesinden B kulesine kuralları bozmadan taşımaktır.



### Not

Hanoi Kuleleri, birçok web sitesinin adandığı klasik bir bilgisayar bilimi problemidir. Göz atmaya değer olanlardan biri [www.cut-the-knot.com/recurrence/hanoi.html](http://www.cut-the-knot.com/recurrence/hanoi.html)'dir.

Üç disk durumunda, çözümü manuel olarak bulabilirsiniz. Ancak, daha fazla disk sayısı için —hatta dört disk için bile—problem oldukça karmaşıktır. Neyse ki, problemin doğası gereği öz yinelenmeli bir yapısı vardır, bu da basit bir özyinelemeli çözüme yol açar.

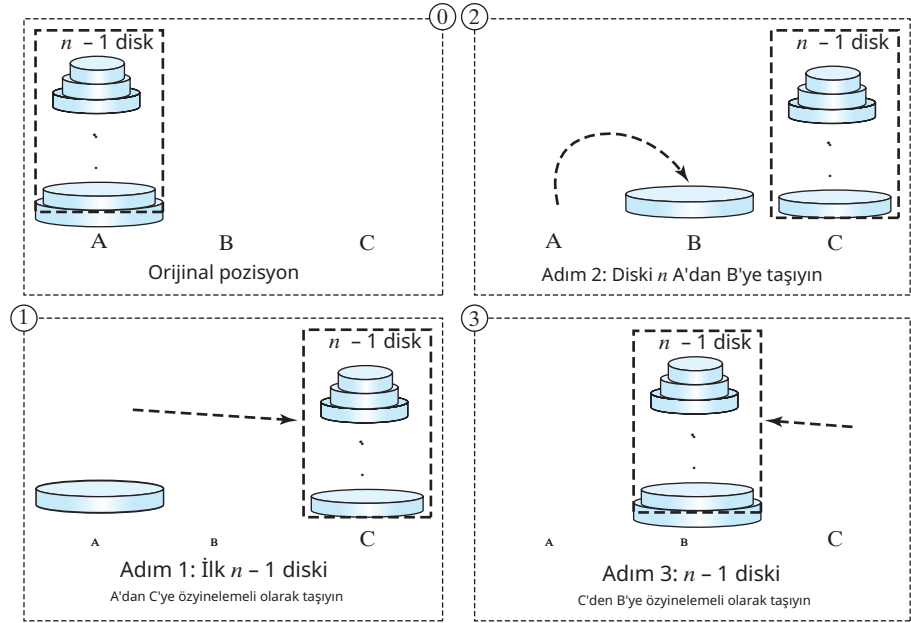
Problemin temel durumu  $n = 1$ 'dir. Eğer  $n = 1$  ise, diski A'dan B'ye basitçe taşıyabilirsiniz.  $n > 1$  olduğunda, orijinal problemi aşağıdaki üç alt probleme ayırabilir ve bunları sırasıyla çözebilirsiniz.

1. İlk  $n - 1$  diski A'dan C'ye, B kulesinin yardımıyla, Şekil 20.7'deki Adım

1'de gösterildiği gibi taşıyın.

2. Diski  $n$  A'dan B'ye taşıyın, Şekil 20.7'deki Adım 2'de gösterildiği gibi.

3.  $n - 1$  diski C'den B'ye, A kulesinin yardımıyla taşıyın, Şekil 20.7'deki Adım 3'te gösterildiği gibi.



**FIGURE 20.7** Hanoi kuleleri problemi üç alt probleme ayrılabilir.

Aşağıdaki yöntem,  $n$  diski **fromTower**'dan **toTower**'a **auxTower**'nın yardımıyla taşır:

```
void moveDisks(int n, char fromTower, char toTower, char auxTower)
```

Yöntemin algoritması şu şekilde tanımlanabilir:

```
eğer (n == 1) // Durdurma koşulu
 Disk 1'i fromTower'dan toTower'a taşı;
else {
 diskleriTaşı(n - 1, fromTower, auxTower, toTower);
 Disk n'i fromTower'dan toTower'a taşı;
 diskleriTaşı(n - 1, auxTower, toTower, fromTower);
}
```

Liste 20.8, kullanıcının disk sayısını girmesini isteyen ve diskleri taşımak için çözümü görün tüklemek üzere **moveDisks** yöntemini çağıran bir programı vermektedir.

### LISTE 20.8 TowersOfHanoi.java

```
1 import java.util.Scanner;
2
3 genel sınıf TowersOfHanoi {
4 /** Ana yöntem */
5 public static void main(String[] args) {
6 // Bir Scanner oluştur
7 Scanner input = new Scanner(System.in);
8 System.out.print("Disk sayısını girin: ");
9 int n = input.nextInt();
10
11 // Çözümü özyinelemeli olarak bul
12 System.out.println("Hareketler:");
13 diskleriTaşı(n, 'A', 'B', 'C');
14 }
15 }
```



```

16 /** n diski taşımak için çözümü bulma yöntemi
17 fromTower'dan toTower'a auxTower ile */
18 genel statik boşluk diskleriTaşı(int n, char fromTower,
19 char toTower, char auxTower) {
20 if (n == 1) // Durdurma koşulu
21 System.out.println("Diski taşı: " + n + " from " +
22 fromTower + " to " + toTower);
23 aksi takdirde {
24 diskleriTaşı(n - 1, fromTower, auxTower, toTower);
25 System.out.println("Diski taşı: " + n + " from " +
26 fromTower + " to " + toTower);
27 diskleriTaşı(n - 1, auxTower, toTower, fromTower);
28 }
29 }
30 }

```

temel durum

özyineleme

özyineleme

Disk sayısını girin:

4 Enter

Hareketler şunlardır:

Disk 1'i A'dan C'ye taşı  
 Disk 2'yi A'dan B'ye taşı  
 Disk 1'i C'den B'ye taşı  
 Disk 3'ü A'dan C'ye taşı  
 Disk 1'i B'den A'ya taşı  
 Disk 2'yi B'den C'ye taşı  
 Disk 1'i A'dan C'ye taşı  
 Disk 4'ü A'dan B'ye taşı  
 Disk 1'i C'den B'ye taşı  
 Disk 2'i C'den A'ya taşı  
 Disk 1'i B'den A'ya taşı  
 Disk 3'ü C'den B'ye taşı  
 Disk 1'i A'dan C'ye taşı  
 Disk 2'yi A'dan B'ye taşı  
 Disk 1'i C'den B'ye taşı



Bu problem doğası gereği özyinelemelidir. Özyineleme kullanmak, doğal ve basit bir çözüm bulmayı mümkün kılar. Özyineleme kullanmadan bu problemi çözmek zor olacaktır.

Programı  $n = 3$  için izlemeyi düşünün. Ardışık özyinelemeli çağrılar

Şekil 20.8'de gösterilmiştir. Gördüğünüz gibi, programı yazmak, özyinelemeli çağrılar izlemekten daha kolaydır. Bu

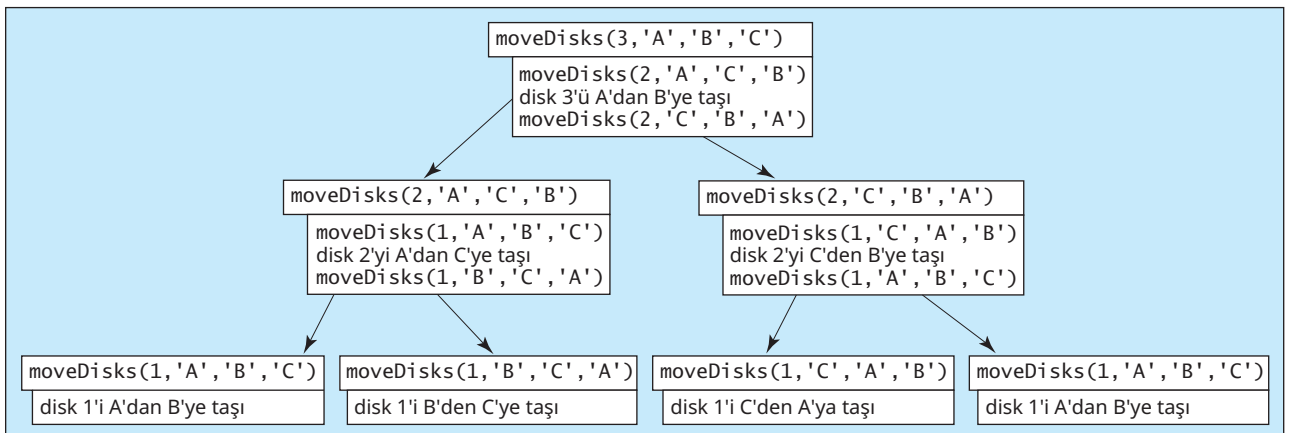


FIGURE 20.8 `moveDisks(3, 'A', 'B', 'C')` çağrılarını `moveDisks` özyinelemeli olarak başlatır.

sistem, arka planda çağrılarını yönetmek için yığınlar kullanır. Bir ölçüde, özyineleme, kullanıcıdan yinlemeleri ve diğer ayrıntıları gizleyen bir soyutlama düzeyi sağlar.

### 20.19 `moveDisks` yöntemi, Listing 20.8'de kaç kez çağrılır `moveDisks(5, 'A', 'B', 'C')`?

Nokta ✓ Check

MyProgrammingLab™

## 20.8 Vaka Çalışması: Fraktallar

Key Point

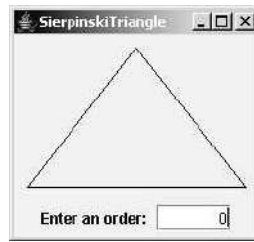
*Özyineleme kullanmak, fraktalları görüntülemek için idealdir, çünkü fraktallar doğası gereği özyinelemelidir.*

Bir fraktal, geometrik bir şekildir, ancak üçgenler, daireler ve dikdörtgenlerden farklı olarak, fraktallar parçalara ayrılabilir; her biri bütünün küçültülmüş boyutlu bir kopyasıdır. Fraktalların birçok iç içe geçmiş örneği vardır. Bu bölüm, ünlü Polonyalı matematikçi Sierpinski'nin adını taşıyan basit bir fraktal olan Sierpinski üçgenini tanıtmaktadır.

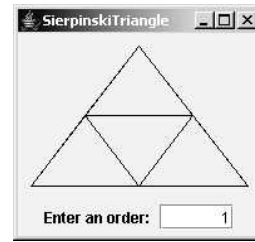
Bir Sierpinski üçgeni şu şekilde oluşturulur:

1. Şekil 20.9a'da gösterildiği gibi, sıfırıncı (veya seviye) Sierpinski fraktalı olarak kabul edilen bir eşkenar üçgenle başlayın.
2. Sıfırıncı seviye üçgenin kenarlarının orta noktalarını birleştirerek birinci seviye Sierpinski üçgeni oluşturun (Şekil 20.9b).
3. Ortadaki üçgeni olduğu gibi bırakın. Diğer üçgenlerin kenarlarının orta noktalarını birleştirerek ikinci seviye Sierpinski üçgeni oluşturun (Şekil 20.9c).
4. Aynı işlemi özyinelemeli olarak tekrarlayarak üçüncü, dördüncü, ... ve daha fazlası seviye Sierpinski üçgeni oluşturabilirsiniz (Şekil 20.9d).

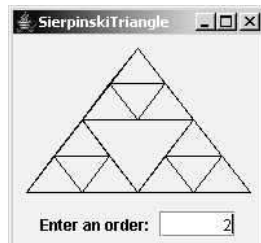
Problem doğası gereği özyinelemelidir. Bunun için özyinelemeli bir çözüm nasıl geliştirilir? Sıfırıncı seviye durumunu düşünün. Sıfırıncı seviye Sierpinski üçgenini çizmek kolaydır.



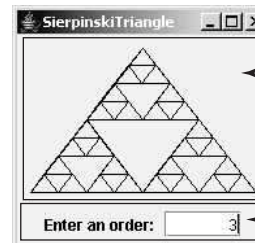
(a) Sipariş 0



(b) Sipariş 1



(c) Sipariş 2



(d) Sipariş 3

**FIGURE 20.9** Bir Sierpinski üçgeni, özyinelemeli üçgenlerin bir desendir.

Sierpinski üçgenini 1. derecede nasıl çizersiniz? Problem, üç tane 0. dereceli Sierpinski üçgeni çizmeye indirgenebilir. Sierpinski üçgenini 2. derecede nasıl çizersiniz? Problem, üç tane 1. dereceli Sierpinski üçgeni çizmeye indirgenebilir, bu nedenle n. dereceli Sierpinski üçgeni çizme problemi, üç tane n - 1. dereceli Sierpinski üçgeni çizmeye indirgenebilir.

Liste 20.9, herhangi bir derecede Sierpinski üçgenini görüntüleyen bir Java applet'ini vermektedir; bu, Şekil 20.9'da gösterilmiştir. Belirtilen derecede bir Sierpinski üçgeni görüntülemek için bir metin alanına bir derece girebilirsiniz.

## LISTE 20.9 SierpinskiTriangle.java

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.*;
4
5 genel sınıf SierpinskiTriangleText extends JApplet {
6 private JTextField jtfOrder = new JTextField("0", 5); // Derece
7 özel Sierpinski Üçgeni Panel Üçgen Paneli =
8 new SierpinskiTrianglePanel(); // Deseni görüntülemek için
9
10 public SierpinskiÜçgen() {
11 // Etiket, metin alanı ve bir düğme tutacak panel
12 JPanel panel = new JPanel();
13 panel.add(new JLabel("Bir sipariş girin: "));
14 panel.add(jtfOrder);
15 jtfOrder.setHorizontalAlignment(SwingConstants.RIGHT);
16
17 // Applete bir Sierpinski üçgeni paneli ekle
18 add(trianglePanel);
19 add(panel, BorderLayout.SOUTH);
20
21 // Bir dinleyici kaydet
22 jtfOrder.addActionListener(new ActionListener() {
23 @Geçersiz Kıl
24 kamusal void actionPerformed(ActionEvent e) {
25 üçgenPanel.setOrder(Integer.parseInt(jtfOrder.getText())); n26 }
26
27 });
28 }
29
30 statik sınıf SierpinskiTrianglePanel extends JPanel {
31 özel sipariş = t0t;
32
33 /** Yeni bir sıra ayarla */
34 public void setOrder(int tOrder) {
35 this.sipariş = sipariş;
36 yeniden boyama(); n37
37 }
38
39 @Geçersiz Kıl
40 korumalı void paintComponent(Graphics g) {
41 super.paintComponent(g);
42
43 // Panel boyutuna orantılı olarak üç nokta seçin
44 p1 noktası = new Point(getWidth() / t2t, t10t);
45 new Point(t10t, getHeight() - t10t);
46 p3 noktası = new Point(getWidth() - t10t, getHeight() - t10t); nPoint p2 =

```

dinleyici

yeni bir sipariş ayarla

üç başlangıç noktası

```

48 displayTriangles(g, sıra, p1, p2, p3);
49 }
50
51 static void tdisplayTriangles(Graphics g, tinttorder,
52 Nokta p1, Nokta p2, Nokta p3) {nprivat
53 if(sipariş == 0) {
54 // Üç noktayı bağlamak için bir üçgen çiz
55 g.drawLine(p1.x, p1.y, p2.x, p2.y); n56 g.drawLine(p1.x, p
56 1.y, p3.x, p3.y); n57 g.drawLine(p2.x, p2.y, p3.x, p3.y); n58 }
59
60 // Üçgenin her kenarındaki orta noktayı alın
61 Nokta p12 = ortaNokta(p1, p2);
62 Nokta p23 = ortaNokta(p2, p3);
63 Nokta p31 = ortaNokta(p3, p1);
64
65 // Üç üçgeni özyinelemeli olarak gösterin
66 r - t1t, p1, p12, p31);
67 gles(g, order - t1t, p12, p2, p23); ndisplayTriangles(g, orde
68 displayTriangles(g, order - t1t, p31, p23, p3); ndisplayTrian
69 }
70 }
71
72 özel statik Nokta ortaNokta(Nokta p1, Nokta p2) {
73 return new Point((p1.x + p2.x) / 2, (p1.y + p2.y) / 2);
74 } n75 } n7
6 }

```

bir üçgen çiz

üst alt üçgen  
sol alt üçgen  
sağ alt üçgen

ana yöntem atlandı

üçgeniGörüntüle metodu

Başlangıç üçgeni, panel boyutuna orantılı olarak ayarlanmış üç noktaya sahiptir (satırlar 44–46). Eğer **order == 0**, **displayTriangles(g, order, p1, p2, p3)** metodu, satırlarda 55–57 gösterildiği gibi, **p1**, **p2** ve **p3** noktalarını birleştiren bir üçgeni görüntüler. Aksi takdirde, aşağıdaki görevleri yerine getirir:

1. **p1** ve **p2** arasındaki orta noktayı (satır 61), **p2** ve **p3** arasındaki orta noktayı (satır 62) ve **p3** ve **p1** arasındaki orta noktayı (satır 63) elde edin, Şekil 20.10b'de gösterildiği gibi.
2. Üç daha küçük Sierpinski üçgenini görüntülemek için azalan bir sıralama ile **displayTriangles** metodunu özyinelemeli olarak çağırın (satırlar 66–68). Her küçük Sierpinski üçgeninin, orijinal büyük Sierpinski üçgeni ile yapısal olarak aynı olduğunu, tek farkın küçük üçgenin sıralamasını n bir eksik olmasıdır, Şekil 20.10b'de gösterildiği gibi.

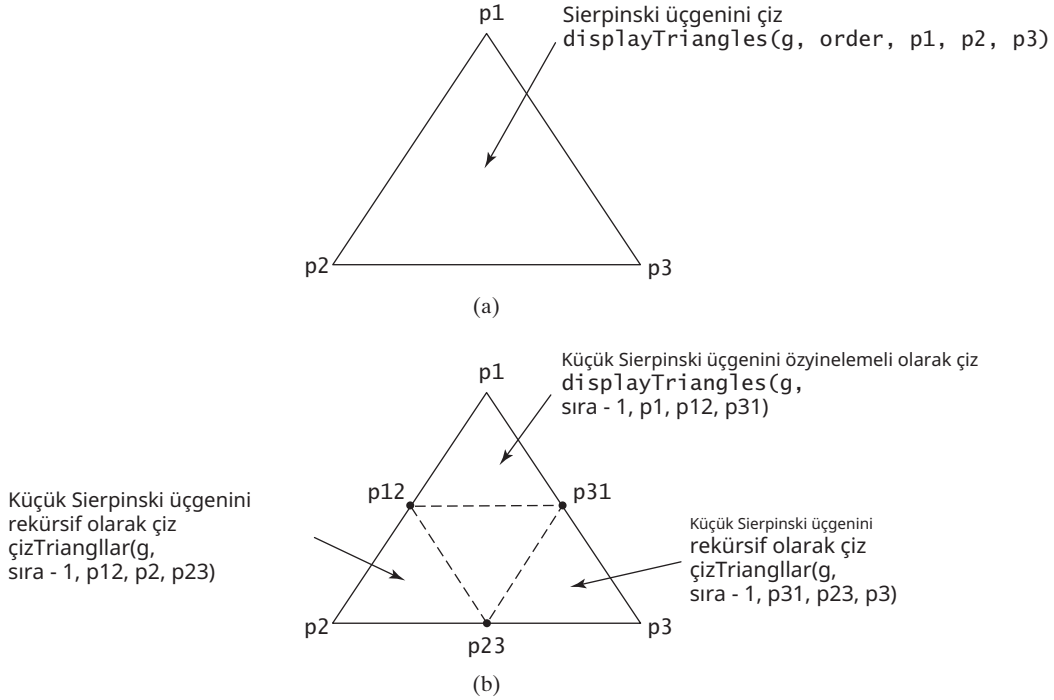
Bir Sierpinski üçgeni, bir **SierpinskiTrianglePanel** içinde görüntülenir. İç sınıf **SierpinskiTrianglePanel** içindeki **order** özelliği, Sierpinski üçgeninin sırasını belirtir. **Point** sınıfı, Bölüm 16.8, Fare Olayları'nda tanıtılmıştır ve bir bileşen üzerindeki bir noktayı temsil eder. **midpoint(Point p1, Point p2)** metodu, **p1** ve **p2** arasındaki orta noktayı döndürür (satırlar 72–74).

**20.20** İki nokta arasındaki orta noktayı nasıl elde edersiniz?

**20.21** **displayTriangles** metodunun temel durumu nedir?

**20.22** Sierpinski üçgeni için **displayTriangles** metodu kaç kez çağrılır? sırası 0, sırası 1, sırası 2 ve sırası n?





**FIGÜR20.10** Bir Sierpinski üçgeni çizmek, üç küçük Sierpinski üçgenini rekürsif olarak çizme çağrılarını oluşturur.

## 20.9 Rekürsiyon vs. İterasyon

*Rekürsiyon, program kontrolünün alternatif bir biçimidir. Temelde bir döngü olmadan tekrardır.*



Döngüleri kullandığınızda, bir döngü gövdesi belirtirsiniz. Döngü gövdesinin tekrarı, döngü kontrol yapısı tarafından kontrol edilir. Rekürsiyonda, yöntem kendisi tekrar tekrar çağrılır. Bir seçim ifadesi, yöntemi rekürsif olarak çağırıp çağırmamayı kontrol etmek için kullanılmalıdır.

Rekürsiyon önemli bir yük taşır. Program her seferinde bir yöntemi çağırıldığında, sistemin tüm yerel değişkenleri ve parametreleri için bellek ayırmak zorundadır. Bu, önemli ölçüde bellek tüketebilir ve belleği yönetmek için ekstra zaman gerektirir.

Rekürsif olarak çözülebilen herhangi bir problem, iterasyonlarla rekürsif olmayan bir şekilde de çözülebilir. Rekürsiyonun bazı olumsuz yönleri vardır: çok fazla zaman ve çok fazla bellek kullanır. O halde, neden kullanmalısınız? Bazı durumlarda, rekürsiyon kullanmak, aksi takdirde elde edilmesi zor olan, doğası gereği rekürsif bir problem için net, basit bir çözüm belirtmenizi sağlar.

Örnekler, dizin boyutu problemi, Hanoi Kuleleri problemi ve fraktal problemi gibi, özyineleme kullanmadan çözülmesi oldukça zor olan problemlerdir.

Özyineleme veya yineleme kullanma kararı, çözmeye çalıştığınız problemin doğasına ve anlayışınıza dayanmalıdır. Genel kural, problemi doğal olarak yansıtan sezgisel bir çözüm geliştirebilecek yaklaşımı kullanmaktır. Eğer yinelemeli bir çözüm belirginse, onu kullanın. Genellikle özyinelemeli seçeneğe göre daha verimli olacaktır.

özyineleme yükü

özyineleme avantajları

özyineleme mi yoksa yineleme mi?



### Not

Özyinelemeli programlar bellekten çıkabilir ve bu da bir [StackOverflowError](#) ile sonuçlanabilir.

Yığın Taşması Hatası

performans kaybı

**Ttítip**

Programınızın performansından endişe ediyorsanız, özyineleme kullanmaktan kaçının, çünkü yinelemeden daha fazla zaman alır ve daha fazla bellek tüketir. Genel olarak, özyineleme, Hanoi Kuleleri, özyinelemeli dizinler ve Sierpinski üçgenleri gibi doğası gereği özyinelemeli problemleri çözmek için kullanılabilir.

Nokta ✓ Check

MyProgrammingLab™

**20.23** Aşağıdaki ifadelerden hangileri doğrudur?

- Herhangi bir özyinelemeli yöntem, özyinelemsiz bir yönteme dönüştürülebilir.
- Rekürsif yöntemler, rekürsif olmayan yöntemlerden daha fazla zaman ve bellek harcar.
- Rekürsif yöntemler, rekürsif olmayan yöntemlerden *her zaman* daha basittir.
- Bir temel durumun ulaşıp ulaşılamadığını kontrol etmek için her zaman bir seçim ifadesi vardır.

**20.24** Bir yığın taşması istisnasının nedeni nedir?**20.10 Kuyruk Rekürsyonu**

*Bir kuyruk rekürsif yöntemi, yığın boyutunu azaltmak için etkilidir.*

Keyn  
Point

kuyruk rekürsyonu

Bir rekürsif yöntem, bir rekürsif çağrıdan dönerken yapılacak bekleyen işlemler yoksa *kuyruk rekürsif* olarak adlandırılır; bu, Şekil 20.11a'da gösterilmiştir. Ancak, Şekil 20.11b'deki yöntem **B**, bir yöntem çağrısı döndüğünde bekleyen işlemler olduğu için kuyruk rekürsif değildir.



**FIGİN 20.11** Kuyruk özyinelemeli bir yöntem, bir özyineleme çağrısından sonra bekleyen işlemlere sahip değildir.

Örneğin, Liste 20.4'teki **isPal indrome** yöntemi (satırlar 6–13) kuyruközyinelemelidir çünkü satır 12'de **isPalindrome**'i özyinelemeli olarak çağırıdıktan sonra bekleyen işlemler yoktur. Ancak, Liste 20.1'deki **factorial** yöntemi (satırlar 16–21) kuyruközyinelemeli değildir, çünkü her özyineleme çağrısından döndüğünde yapılacak bir bekleyen işlem, yani çarpma vardır.

Sonuç olarak, son çağrının sona ermesiyle yöntem sona erdiği için, yığın içinde aranan çağrılar saklamaya gerek yoktur. Bazı derleyiciler, yığın boyutunu azaltmak için son çağrıyı optimize edebilir.

Bir son çağrısız yöntem, genellikle yardımcı parametreler kullanılarak bir son çağrılı yönteme dönüştürülebilir. Bu parametreler sonucu içermek için kullanılır. Amaç, bekleyen işlemleri yardımcı parametreler içine öyle bir şekilde dahil etmektir ki, çağrı artık bekleyen bir işleme sahip olmasın. Yeni bir yardımcı özyinelemeli yöntem tanımlayabilirsiniz. Bu yöntem, aynı ada sahip ancak farklı bir imzaya sahip olan orijinal yöntemi aşabilir. Örneğin, Liste 20.1'deki faktöriyel yöntemi, Liste 20.10'da son çağrılı bir şekilde yazılmıştır.

## LİSTE20.10ComputeFactorialTailRecursion.java

```

1 public class ComputeFactorialTailRecursion {
2 /** Belirtilen bir sayı için faktöriyel döndür */
3 public static long factorial(int n) {
4 return faktöriyel(n, 1); // Yardımcı yöntemi çağır
5 }
6
7 /** Faktöriyel için yardımcı son çağrılı yöntem */
8 özel statik uzunfaktöriyel(int n, int sonuç) {
9 eğer(n == 0)
10 return sonuç;
11 aksi takdirde
12 return faktöriyel(n - 1, n * sonuç); // Rekürsif çağrı
13 }
14 }
```

orijinal metod  
yardımcı metodu çağır

yardımcı metod

özyinelemeli çağrı

İlk **faktöriyel** metodu (satır 3) basitçe ikinci yardımcı metodu (satır 4) çağırır. İkinci metod, **result** için bir yardımcı parametre içerir; bu, faktöriyel için sonucu saklar. Bu metod, satır 12'de rekürsif olarak çağrılır. Bir çağrı döndürüldükten sonra bekleyen bir işlem yoktur. Nihai sonuç, satır 10'da döndürülür; bu, aynı zamanda satır 4'te faktöriyel(n, 1) çağrısından dönen değerdir.

**20.25** Bu bölümde kuyruk-rekürsif yöntemleri tanımlayın.

**20.26** Listing 20.2'deki **fib** metodunu kuyruk rekürsyonu kullanarak yeniden yazın.

Nokta Check

MyProgrammingLab™

### ANAHTAR TERİMLER

temel durum 738	özyinelemeli yardımcı yöntem 747
doğrudan rekürsiyon 741	özyinelemeli yöntem 738
dolaylı rekürsiyon 741	durma koşulu 738
sonsuz rekürsiyon 741	kuyruk özyinelemesi 758

## BÖLÜM ÖZETİ

1. *A özyinelemeli yöntem*, kendisini doğrudan veya dolaylı olarak çağırarak bir yöntemdir. Bir özyinelemeli yöntemin sona ermesi için bir veya daha fazla *temel durum* olmalıdır.
2. *Özyineleme*, program kontrolünün alternatif bir biçimidir. Temelde döngü kontrolü olmadan tekrardır. Aksi takdirde zor çözülebilecek, doğası gereği özyinelemeli olan problemler için basit, net çözümler belirtmekte kullanılabilir.
3. Bazen, orijinal yöntemin, yinelemeli olarak çağrılabilmesi için ek parametreler alacak şekilde değiştirilmesi gerekir. Bu amaçla, bir *yinelemeli yardımcı yöntemi* tanımlanabilir.
4. Yineleme önemli bir yük taşır. Program her seferinde bir yöntemi çağırıldığında, sistem tüm yöntemin yerel değişkenleri ve parametreleri için bellek ayırmak zorundadır. Bu, önemli ölçüde bellek tüketebilir ve belleği yönetmek için ekstra zaman gerektirir.
5. Bir yöntem, geri dönüşte yapılacak bekleyen işlemler yoksa *kuyruk yinelemeli* olarak adlandırılır. Bazı derleyiciler, yığın boyutunu azaltmak için kuyruk yinelemesini optimize edebilir.

## TEST SORULARI

Bu bölüm için test sorularını çevrimiçi olarak yapın: [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

MyProgrammingLab™

## PROGRAMLAMA EGZERSİZLERİ

### Bölümler 20.2–20.3

**\*20.1** (Faktöriyel) Bölüm 10.14'te tanıtilan **BigInteger** sınıfını kullanarak, büyük bir sayıyı  $n$  faktöriyelini bulabilirsiniz (örneğin, **100!**). Faktöriyel yöntemini yineleme kullanarak uygulayın. Kullanıcıdan bir tamsayı girmesini isteyen ve faktöriyelini gösteren bir program yazın.

**\*20.2** (Fibonacci sayıları) Liste 20.2'deki **fib** yöntemini yinelemeler kullanarak yeniden yazın. *İpucu: **fib(n)**'yi yineleme olmadan hesaplamak için önce **fib(n - 2)** ve **fib(n - 1)**'yi elde etmeniz gerekir. **f0** ve **f1** iki önceki Fibonacci sayısını temsil etsin. Mevcut Fibonacci sayısı o zaman **f0 + f1** olacaktır. Algoritma şu şekilde tanımlanabilir:*

```
f0 = 0; // fib(0) için
f1 = 1; // fib(1) için
```

```
for(tint = t1t; i <= n; i++) { ncurrentFib
 = f0 + f1;
 f0 = f1;
 f1 = mevcutFib;n}
```

// Döngüden sonra, currentFib fib(n) dir

Kullanıcıdan bir indeks girmesini isteyen ve onun

Fibonacci sayısını gösteren bir test programı yazın.

**\*20.3** (Özyineleme kullanarak en büyük ortak böleni hesapla)  $\text{gcd}(m, n)$  da aşağıdaki gibi özyinelemeli olarak tanımlanabilir:

- Eğer  $m \% n0$  ise,  $\text{gcd}(m, n)n$  dir.
- Aksi takdirde,  $\text{gcd}(m, n)\text{gcd}(n, m \% n)$  dir.

GCD'yi bulmak için özyinelemeli bir yöntem yazın. Kullanıcıdan iki tam sayı girmesini isteyen ve GCD'lerini gösteren bir test programı yazın.

**20.4** (Toplam serisi) Aşağıdaki seriyi hesaplamak için bir özyinelemeli yöntem yazın:

$$m(i) = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{i}$$

Bir test programı yazın ve  $m(i)$  değerini  $i$  için görüntüleyin  $1, 2, \dots, 10$ .

**20.5** (Toplam serisi) Aşağıdaki seriyi hesaplamak için bir özyinelemeli yöntem yazın:

$$m(i) = \frac{1}{3} + \frac{2}{5} + \frac{3}{7} + \frac{4}{9} + \frac{5}{11} + \frac{6}{13} + \dots + \frac{i}{2i+1}$$

Bir test programı yazın ve  $m(i)$  değerini  $i$  için görüntüleyin  $1, 2, \dots, 10$ .

**\*20.6** (Toplam serisi) Aşağıdaki seriyi hesaplamak için bir özyinelemeli yöntem yazın:

$$m(i) = \frac{1}{2} + \frac{2}{3} + \dots + \frac{i}{i+1}$$

Bir test programı yazın ve  $m(i)$  değerini  $i$  için görüntüleyin  $1, 2, \dots, 10$ .



- \*20.7** (*Fibonacci serisi*) Liste 20.2, ComputeFibonacci.java dosyasını değiştirin, böylece program `fib` yönteminin kaç kez çağrıldığını bulsun. (*İpucu:* Her seferinde çağrıldığında bir statik değişken kullanın ve onu artırın.)

## Bölüm 20.4

- \*20.8** (*Bir tam sayının basamaklarını ters olarak yazdır*) Aşağıdaki başlığı kullanarak bir `int` değerini konsolda ters olarak görüntüleyen bir özyinelemeli yöntem yazın:

```
public static void reverseDisplay(int tintValue)
```

Örneğin, `reverseDisplay(12345)` `54321` olarak görüntüler. Kullanıcıdan bir tam sayı girmesini isteyen ve tersini görüntüleyen bir test programı yazın.

- \*20.9** (*Bir dizedeki karakterleri ters olarak yazdırın*) Aşağıdaki başlığı kullanarak bir dizeyi konsolda ters olarak görüntüleyen bir özyinelemeli yöntem yazın:

```
public static void reverseDisplay(Dize degeri)
```

Örneğin, `reverseDisplay("abcd")` `dcba` olarak görüntüler. Kullanıcıdan bir dize girmesini isteyen ve tersini görüntüleyen bir test programı yazın.

- \*20.10** (*Bir dizedeki belirli bir karakterin oluşumları*) Aşağıdaki yöntem başlığını kullanarak bir dizedeki belirli bir harfin kaç kez geçtiğini bulan bir özyinelemeli yöntem yazın:

```
public static int count(String str, tcharta)
```

Örneğin, `count("Welcome", 'e')` `2` döner. Kullanıcıdan bir dize ve bir karakter girmesini isteyen ve karakterin dizedeki oluşum sayısını görüntüleyen bir test programı yazın.

- \*20.11** (*Özyineleme kullanarak bir tamsayıdaki rakamların toplamını hesaplayın*) Bir tamsayıdaki rakamların toplamını hesaplayan bir özyinelemeli yöntem yazın. Aşağıdaki yöntem başlığını kullanın:

```
genel statik int sumDigits(tlongtn)
```

Örneğin, `sumDigits(234)` bir test programı yazdırır  $2 + 3 + 4 = 9$ . Bir test programı yazın kullanıcının bir tamsayı girmesini ister ve toplamını gösterir.

## Bölüm 20.5

- \*\*20.12** (*Bir dizedeki karakterleri ters sırayla yazdırın*) Yardımcı bir yöntem kullanarak Alıştırma 20.9'u yeniden yazın alt dize `yuksek` indeksini yönteme geçirin. Yardımcı yöntem başlığı şudur:

```
public static void reverseDisplay(Dize degeri, int tthigh)
```

- \*20.13** (*Bir dizideki en büyük sayıyı bulun*) Bir dizideki en büyük tamsayıyı döndüren özyinelemeli bir yöntem yazın. Kullanıcının sekiz tamsayıdan oluşan bir liste girmesini isteyen ve en büyük öğeyi gösteren bir test programı yazın.

- \*20.14** (*Bir dizedeki büyük harf sayısını bulun*) Bir dizedeki büyük harf sayısını döndüren özyinelemeli bir yöntem yazın. Kullanıcının bir dize girmesini isteyen ve dizideki büyük harf sayısını gösteren bir test programı yazın.

- \*20.15** (*Bir dizedeki belirli bir karakterin oluşumları*) Yardımcı bir yöntem kullanarak Alıştırma 20.10'u yeniden yazın alt dize `yuksek` indeksini yönteme geçirin. Yardımcı yöntem başlığı şudur:

```
public static int count(String str, tcharta, int tthigh)
```

- \*20.16** (*Bir dizideki büyük harflerin sayısını bulun*) Bir dizideki büyük harflerin sayısını döndüren bir özyinelemeli yöntem yazın. Aşağıdaki iki yöntemi tanımlamanız gerekiyor. İkincisi bir özyinelemeli yardımcı yöntemdir.

```
public static inttcount(tchart[] karakter) {
 public static inttcount(tchart[] karakter, tintthigh)
```

Kullanıcıdan bir satırda bir karakter listesi girmesini isteyen ve listedeki büyük harflerin sayısını gösteren bir test programı yazın.

- \*20.17** (*Bir dizide belirtilen bir karakterin geçişleri*) Belirtilen bir karakterin bir dizideki geçişlerini bulan bir özyinelemeli yöntem yazın. Aşağıdaki iki yöntemi tanımlamanız gerekiyor. İkincisi bir özyinelemeli yardımcı yöntemdir.

```
public static inttcount(tchart[] karakterler, tchart[]) {
 public static inttcount(tchart[] karakterler, tchart[], tintthigh)
```

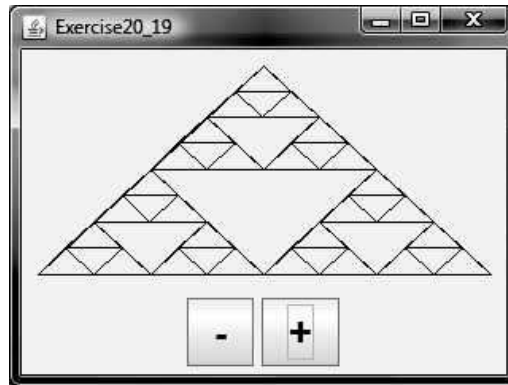
Kullanıcıdan bir satırda bir karakter listesi ve bir karakter girmesini isteyen ve listedeki karakterin geçiş sayısını gösteren bir test programı yazın.

### Bölmeler 20.6–20.10

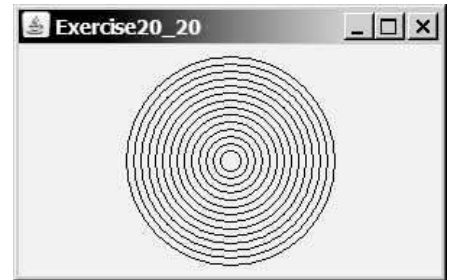
- \*20.18** (*Hanoi Kuleleri*) Listing 20.8, TowersOfHanoi.java dosyasını değiştirin, böylece program  $n$  diski A kulesinden B kulesine taşımak için gereken hareket sayısını bulur. (*İpucu:* Her seferinde yöntem çağırıldığında bir statik değişken kullanın ve artırın.)

- \*20.19** (*Sierpinski üçgeni*) Kullanıcının

+ ve - düğmelerini kullanarak mevcut sıralamayı 1 artırmaya veya azaltmaya olanak tanıyan bir applet geliştirmek için Liste 20.9'u gözden geçirin. Başlangıç sırası 0'dır. Mevcut sıra 0 ise, *Azalt* düğmesi göz ardı edilir.



(a)



(b)

**FİKÜR20.12**(a) Alıştırma 20.19 kullanır + ve - mevcut siparişi 1 artırmak veya azaltmak için düğmeler. (b) Egzersiz 20.20, bir özyinelemeli yöntem kullanarak oval çizer.

- \*20.20** (*Daireleri görüntüle*) Şekil 20.12b'de gösterildiği gibi ovalar görüntüleyen bir Java applet'i yazın. Ovalar panelin ortasında yer alır. İki bitişik oval arasındaki boşluk 10 pikseldir ve panelin kenarı ile en büyük oval arasındaki boşluk da 10'dur.

- \*20.21** (*Onaltılıya dönüştürme*) Bir ondalık sayıyı bir ikili sayıya dönüştüren bir özyinelemeli yöntem yazın. Yöntem başlığı:

```
public static String decimalToBinary(int value)
```

Kullanıcıdan bir ondalık sayı girmesini isteyen ve ikili karşılığını görüntüleyen bir test programı yazın.

- \*20.22** (*Onaltılıya dönüştürme*) Bir ondalık sayıyı bir onaltılı sayıya dönüştüren bir özyinelemeli yöntem yazın. Yöntem başlığı:

```
public static String decimalToHex(int value)
```

Kullanıcıdan bir ondalık sayı girmesini isteyen ve onun hex eşdeğerini gösteren bir test programı yazın.

- \*20.23** (*İkili sayıdan ondalık sayıya*) Bir ikili sayıyı dize olarak ayrıştıran ve bir ondalık tam sayıya dönüştüren özyinelemeli bir yöntem yazın. Yöntem başlığı:

```
public static int ikiliSayiToDecimal(String ikiliString)
```

Kullanıcıdan bir ikili string girmesini isteyen ve

onun ondalık karşılığını gösteren bir test programı yazın.

- \*20.24** (*Hex to decimal*) Bir hex sayısını string olarak

ondalık bir tamsayıya ayrıştıran bir özyinelemeli yöntem yazın. Yöntem başlığı:

```
public static int hexToDecimal(String hexString)
```

Kullanıcıdan bir hex string girmesini isteyen ve onun

ondalık karşılığını gösteren bir test programı yazın.

- \*\*20.25** (*String permutasyonu*) Bir stringin tüm permutasyonlarını yazdıran bir özyinelemeli yöntem yazın. Örneğin, **abc** stringi için yazdırma:

```
abcn
acbn
bacn
bcan
cabn
cba
```

(*İpucu:* Aşağıdaki iki yöntemi tanımlayın. İkincisi yardımcı bir yöntemdir.)

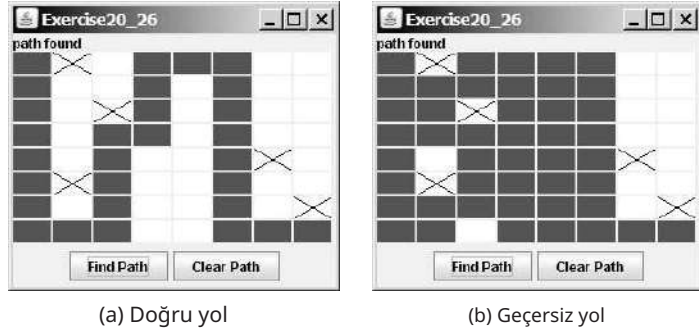
```
public static void displayPermutation(String s)
public static void displayPermutation(String s1, String s2)
```

İlk yöntem basitçe `displayPermutation(" ", s)` çağrısını yapar. İkinci yöntem, bir karakteri `s2`'den `s1`'e taşımak için bir döngü kullanır ve yeni bir `s1` ve `s2` ile özyinelemeli olarak çağırır. Temel durum, `s2`'nin boş olmasıdır ve `s1`'i konsola yazdırır.

Bir kullanıcıdan bir dize girmesini isteyen ve tüm permütasyonlarını gösteren bir test programı yazın.

- \*\*20.26** (*Bir labirent oluşturun*) Şekil 20.13a'da gösterildiği gibi bir labirentte bir yol bulacak bir applet yazın. Labirent, bir  $8 \times 8$  tahta. Yolun karşılaması gerekir aşağıdaki koşullar:

- Yol, labirentteki sol üst köşe hücresi ile sağ alt köşe hücresi arasındadır.
- Applet, kullanıcının bir hücreye işaret koymasını veya kaldırmasını sağlar. Bir yol, bitişik işaretli hücrelerden oluşur. İki hücre, yatay veya dikey komşularsa bitişik olarak kabul edilir, ancak çapraz komşularsa kabul edilmez.
- Yol, bir kare oluşturan hücreler içermemektedir. Örneğin, Şekil 20.13b'deki yol bu koşulu sağlamamaktadır. (Bu koşul, bir yolu tahtada kolayca tanımlamayı sağlar.)



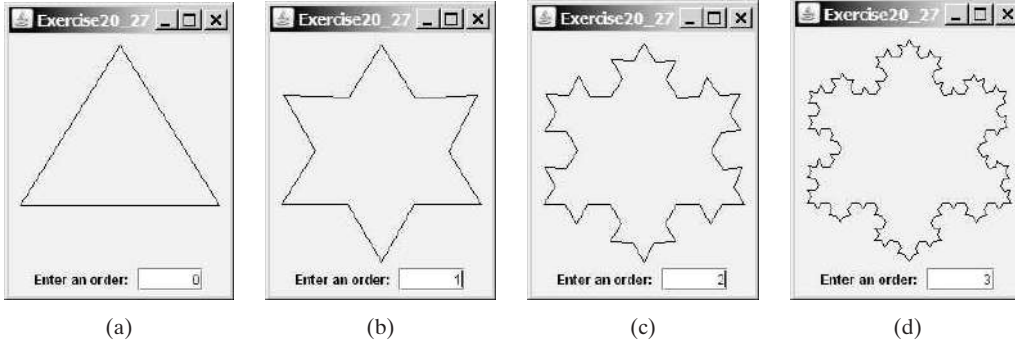
(a) Doğru yol

(b) Geçersiz yol

**FIGÜRİK20.13** Program, sol üst köşeden sağ alt köşeye bir yol bulur.

**\*\*20.27** (*Koch kar tanesi fraktalı*) Metin, Sierpinski üçgen fraktalını sundu. Bualıştırmada, ünlü İsveçli matematikçi adını taşıyan *Kochkar tanesi* olarak adlandırılan başka bir fraktalı görüntülemek için bir applet yazacaksınız. Bir Koch kar tanesi aşağıdaki gibi oluşturulur:

1. Eşkenar bir üçgenle başlayın, bu üçgen, Şekil 20.14a'da gösterildiği gibi, 0. dereceden (veya seviyeden) Koch fraktalı olarak kabul edilir.
2. Şekildeki her bir çizgiyi üç eşit çizgi parçasına bölün ve ortadaki çizgi parçasını taban olarak kullanarak dışa doğru bir eşkenar üçgen çizin, böylece Şekil 20.14b'de gösterildiği gibi 1. dereceden bir Koch fraktalı oluşturun.
3. Adım 2'yi tekrarlayarak, Şekil 20.14c-d'de gösterildiği gibi, 2, 3, ... gibi bir Koch fraktalı oluşturun.



(a)

(b)

(c)

(d)

**FIGURE20.14** Bir Koch kar tanesi, bir üçgenle başlayan bir fraktaldır.

**\*\*20.28** (*Özyinelemeli dizin boyutu*) Özyineleme kullanmadan, Liste 20.7, DirectorySize.java'yı yeniden yazın.

**\*20.29** (*Bir dizindeki dosya sayısı*) Kullanıcıdan bir dizin girmesini isteyen ve dizindeki dosya sayısını gösteren bir program yazın.

**\*\*20.30** (*Kelimeleri bul*) Bir dizin altındaki tüm dosyalarda bir kelimenin tüm geçişlerini bulan bir program yazın, özyinelemeli olarak. Parametreleri komut satırından aşağıdaki gibi geçirin:

```
java Exercise20_30 dirName kelime
```

**\*\*20.31** (*Kelimeleri değiştir*) Bir dizin altındaki tüm dosyalarda bir kelimenin tüm geçişlerini yeni bir kelime ile değiştiren bir program yazın, özyinelemeli olarak. Parametreleri komut satırından aşağıdaki gibi geçirin:

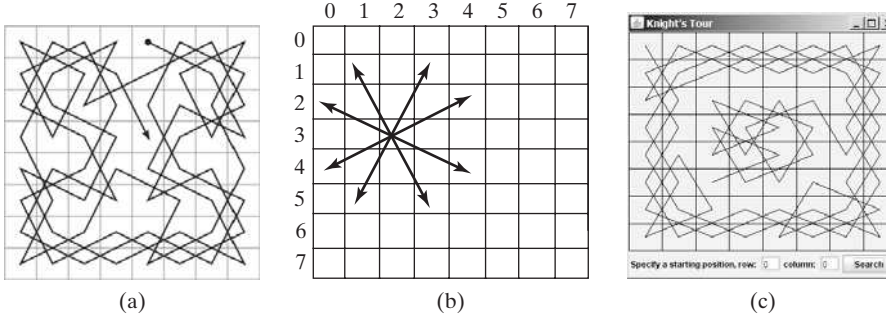
```
java Exercise20_31 dirName eskiKelime yeniKelime
```



VideoNotu

Bir dizinde bir dize ara

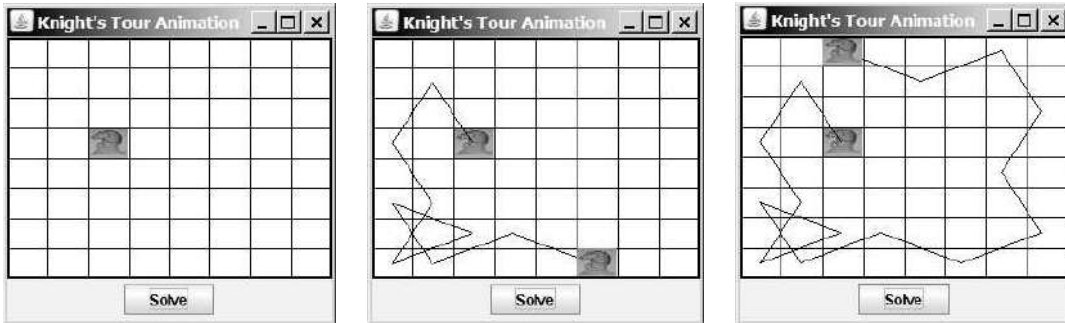
**\*\*\*20.32**(Oyun: Şövalye'nin Turu) Şövalye'nin Turu, eski bir bulmacadır. Amaç, bir satır aç tahtasında herhangi bir kareden başlayarak, her diğer kareye bir kez gitmektir, Şekil 20.15a'da gösterildiği gibi. Şövalyenin yalnızca L şeklinde hareket ettiğini unutmayın (bir yönde iki kare ve dik bir yönde bir kare). Şekil 20.15b'de gösterildiği gibi, şövalye sekiz kareye hareket edebilir. Bir applet içinde atın hamlelerini gösteren bir program yazın, Şekil 20.15c'de gösterildiği gibi.



**FIGÜR20.15**(a) Bir at tüm kareleri bir kez geçer. (b) Bir at L şeklinde bir hamle yapar. (c) Bir applet, Atın Turu yolunu gösterir.

(İpucu: Bu problem için bir kaba kuvvet yaklaşımı, atı bir kareden diğerine rastgele hareket ettirmektir. Böyle bir yaklaşım kullanarak, programınızın bitmesi uzun zaman alacaktır. Daha iyi bir yaklaşım bazı sezgisel yöntemler kullanmaktır. Bir atın, konumuna bağlı olarak iki, üç, dört, altı veya sekiz olası hamlesi vardır. Sezgisel olarak, önce atı en az erişilebilir karelere hareket ettirmeye çalışmalısınız ve daha erişilebilir kareleri açık bırakmalısınız, böylece aramanın sonunda başarı şansı daha iyi olacaktır.)

**\*\*\*20.33**(Oyun: Atın Turu animasyonu) Atın Turu problemini çözmek için bir applet yazın. Applet'iniz, kullancının bir atı herhangi bir başlangıç karesine hareket ettirmesine ve atın yol boyunca hareketini canlandırmak için Çöz butonuna tıklamasına izin vermelidir, Şekil 20.16'da gösterildiği gibi.



**FIGÜR20.16**Bir at yol boyunca geçer.

**\*\*20.34** (Oyun: Sekiz Kraliçe) Sekiz Kraliçe problemi, bir satırda bir kraliçe yerleştirerek, hiçbir iki kraliçenin birbirini tehdit edemeyeceği bir çözüm bulmaktır. Sekiz Kraliçe problemini çözmek için bir program yazın ve sonucu Şekil 20.17'de gösterildiği gibi görüntüleyin.



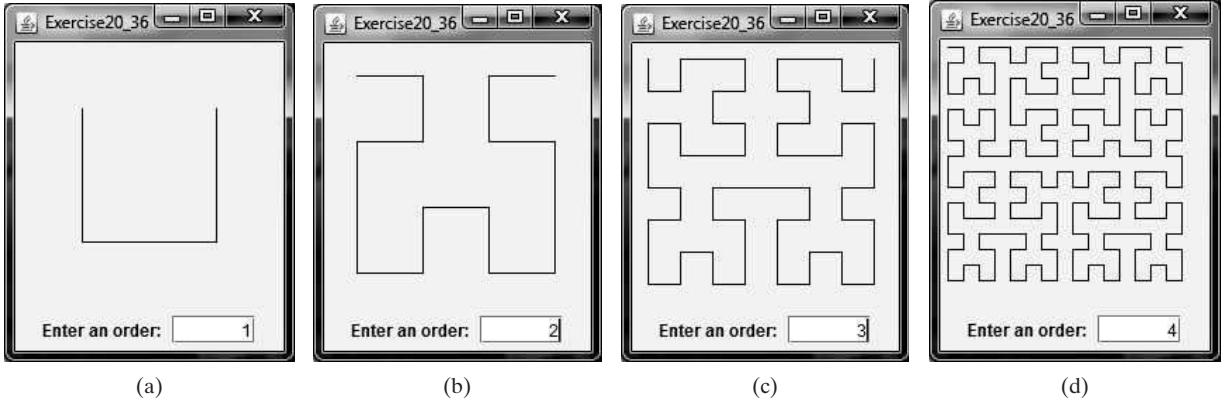
**FIGURE 20.17** Program, Sekiz Kraliçe probleminin bir çözümünü gösterir.

**\*\*20.35** (*H-ağaç fraktalı*) Bu bölümün başında tanıtılan bir H-ağaç, aşağıdaki gibi tanımlanan bir fraktaldır:

1. Bir H harfi ile başlayın. H'nin üç çizgisi aynı uzunluktadır, Şekil 20.1a'da gösterildiği gibi.
2. H harfi (sans-serif biçiminde, H) dört uç noktaya sahiptir. Dört uç noktanın her birinin merkezinde bir H çizin ve bu, Şekil 20.1b'de gösterildiği gibi, 1. sırasındaki bir H-ağaç oluşturur. Bu H'ler, dört uç noktanın bulunduğu H'nin yarısı boyutundadır.
3. Adım 2'yi tekrarlayarak 2,3, ..., vb. sırasındaki bir H-ağaç oluşturun, Şekil 20.1c-d'de gösterildiği gibi.

Şekil 20.1'de gösterildiği gibi bir H-ağaç çizen bir applet yazın.

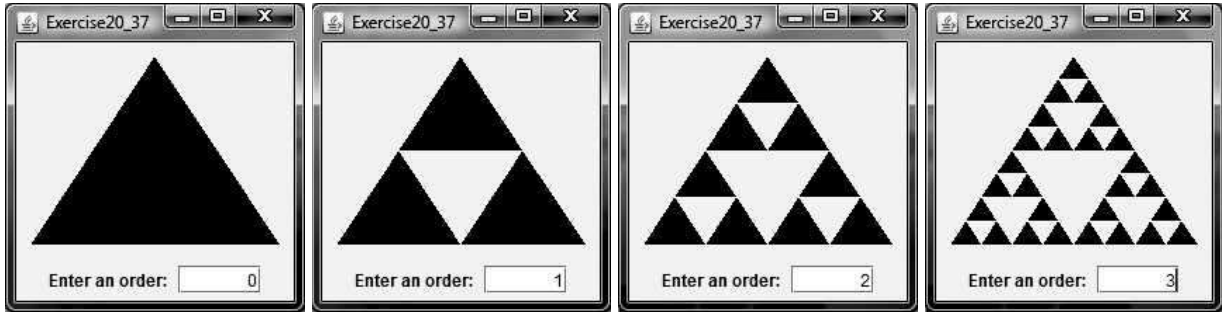
**\*\*20.36** (*Hilbert eğrisi*) İlk olarak 1891'de Alman matematikçi David Hilbert tarafından tanımlanan Hilbert eğrisi, bir kare ızgaradaki her noktayı ziyaret eden bir alan dolduran eğridir.  $2 \times 2$ ,  $4 \times 4$ ,  $8 \times 8$ ,  $16 \times 16$ , veya başka bir 2'nin kuvveti. Belirtilen sıradaki bir Hilbert eğrisini gösteren bir Java applet'i yazın, Şekil 20.18'de gösterildiği gibi.



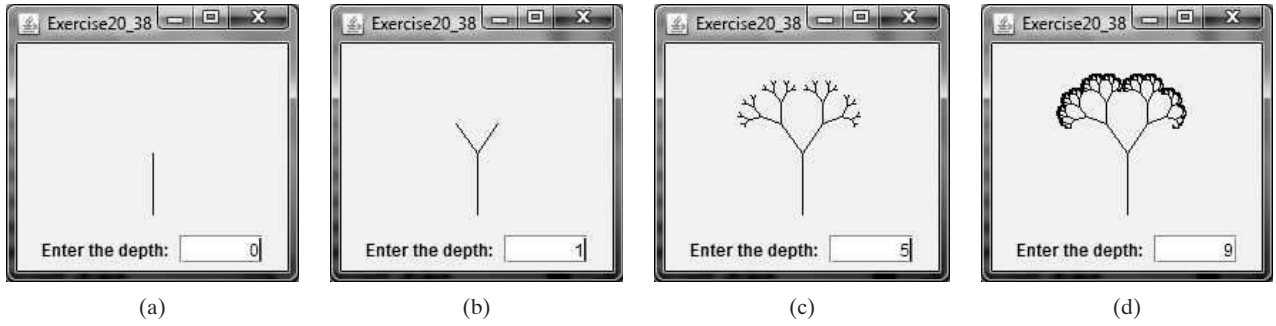
**FIGURE 20.18** Belirtilen sıradaki bir Hilbert eğrisi çizilir.

**20.37** (*Sierpinski üçgeni*) Kullanıcıdan sırayı girmesini isteyen ve doldurulmuş Sierpinski üçgenlerini Şekil 20.19'da gösterildiği gibi görüntüleyen bir program yazın.

**\*\*20.38** (*Özyinelemeli ağaç*) Şekil 20.20'de gösterildiği gibi bir özyinelemeli ağaç görüntülemek için bir applet yazın.



FIGÜR20.19Doldurulmuş bir Sierpinski üçgeni görüntüleniyor.



FIGÜR20.20Belirtilen derinlikte bir özyinelemeli ağaç çizilir.

**\*\*20.39**(*Ağacı sürükleme*) Ağacı, farenin sürüklendiği yere taşımak için Alıştırma 20.38'i gözden geçirin.

*Bu sayfa kasıtlı olarak boş bırakılmıştır*



# EKLER

---

## Ek A

Java Anahtar Kelimeleri

## Ek B

ASCII Karakter Seti

## Ek C

Operatör Öncelik Tablosu

## Ek D

Java Modifikatörleri

## Ek E

Özel Kaydırmalı Nokta Değerleri

## Ek F

Sayı Sistemleri

## Ek G

Bit Düzeyinde İşlemler

*Bu sayfa kasıtlı olarak boş bırakılmıştır*

## Java Anahtar Kelimeleri

Java dili tarafından kullanılmak üzere ayrılmış olan elli anahtar kelime şunlardır:

soyut	ikiye kat	<b>int</b>	üst
<b>assert</b>	lama	arayüz	değiştir
<b>boolean</b>	numaralandırma	uzun	senkronize
<b>break</b>	extends	yerel	bu
<b>byte</b>	final	yeni	at
durum	finally	paket	fırlat
yakalama	float	özel	geçici
karakteri	for	korumalı	denemek
classncon stncontin uedefault ndo	git	kamusal	boş
	eğer	geri dön	<b>volatile</b>
	uygulamalar	kısa	iken
	ithalat	statik	
	örnek	<b>strictfp*</b>	

Anahtar kelimeler **goto** ve **const**, Java'da şu anda kullanılmayan C++ anahtar kelimeleridir. Bu, Java derleyicilerinin bunları tanımlamasını ve Java programlarında göründüklerinde daha iyi hata mesajları üretmesini sağlar.

Literal değerler **true**, **false** ve **null** anahtar kelime değildir, tıpkı literal değer **100** gibi. Ancak, bunları tanımlayıcı olarak kullanamazsınız, tıpkı **100**'ü tanımlayıcı olarak kullanamayacağınız gibi. Kod listesindeki, Java IDE'lerindeki renkleriyle tutarlı olmak için **true**, **false** ve **null** için anahtar kelime rengi kullanıyoruz.

**assert**, JDK 1.4'te eklenen bir anahtar kelimedir ve **enum**, JDK 1.5'te eklenen bir anahtar kelimedir.

\*The **strictfp** anahtar kelimesi, bir yöntem veya sınıfın katı kayan nokta hesaplamalarını kullanmasına olanak tanıyan bir deyiştir. Kayan nokta aritmetiği iki modda gerçekleştirilebilir: *katı* veya *katı olmayan*. Katı mod, değerlendirme sonucunun tüm Java Sanal Makinesi uygulamalarında aynı olmasını garanti eder. Katı olmayan mod, hesaplamalardan ara sonuçların standart IEEE kayan nokta sayı formatından farklı bir genişletilmiş formatta saklanmasına izin verir. Genişletilmiş format makineye bağımlıdır ve kodun daha hızlı çalıştırılmasını sağlar. Ancak, katı olmayan modda farklı JVM'lerde kodu çalıştırdığınızda, her zaman tam olarak aynı sonuçları alamayabilirsiniz. Varsayılan olarak, kayan nokta hesaplamaları için katı olmayan mod kullanılır. Bir yöntem veya sınıfta katı modu kullanmak için, yöntem veya sınıf bildirimine **strictfp** anahtar kelimesini ekleyin. Katı kayan nokta, katı olmayan kayan noktadan biraz daha iyi hassasiyet sağlayabilir, ancak bu ayırım yalnızca bazı uygulamaları etkiler. Katılık miras alınmaz; yani, bir sınıf veya arayüz bildiriminde **strictfp** bulunması, genişletilmiş sınıfların veya arayüzlerin katı olmasına neden olmaz.

## ASCII Karakter Seti

Tablolar B.1 ve B.2, ASCII karakterlerini ve bunların sırasıyla ondalık ve onaltılık kodlarını göstermektedir. Bir karakterin ondalık veya onaltılık kodu, satır indeksinin ve sütun indeksinin bir kombinasyonudur. Örneğin, Tablo B.1'de, A harfi 6. satırda ve 5. sütundadır, bu nedenle ondalık karşılığı 65'tir; Tablo B.2'de, A harfi 4. satırda ve 1. sütundadır, bu nedenle onaltılık karşılığı 41'dir.

**TABLE B.1** Ondalık İndeks'teki ASCII Karakter Kümesi

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dcl	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	”	#	\$	%	&	'
4	(	)	*	+	,	—	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	—	‘	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{		}	~	sil		

**TÜNERLİ B.2** Onaltılık İndeks'teki ASCII Karakter Kümesi

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht	nl	vt	ff	cr	so	si
1	dle	dcl	dc2	dc3	dc4	nak	syn	etb	can	em	sub	esc	fs	gs	rs	us
2	sp	!	”	#	\$	%	&	'	(	)	*	+	,	—	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	—
6	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	sil

## Operatör Öncelik Tablosu

Operatörler, öncelik sırasına göre yukarıdan aşağıya doğru gösterilmiştir. Aynı gruptaki operatörlerin önceliği aynıdır ve bağlanırlıkları tabloda gösterilmiştir.

Operatör	İsim	Bağlanırlık
()	Parantezler	Soldan sağa
()	Fonksiyon çağırısı	Soldan sağa
[]	Dizi altindeksi	Soldan sağa
.	Nesne üye erişimi	Soldan sağa
++	Sondan artırım	Sağdan sola
--	Sondan azaltma	Sağdan sola
++	Önden artırım	Sağdan sola
--	Önden azaltma	Sağdan sola
+	Tekli artı	Sağdan sola
-	Tekli eksi	Sağdan sola
!	Tekli mantıksal olumsuzlama	Sağdan sola
(tip)	Tekli tür dönüşümü	Sağdan sola
yeni	nesne oluşturma	Sağdan sola
*	Çarpma	Soldan sağa
/	Bölme	Soldan sağa
%	Kalan (modül)	Soldan sağa
+	Toplama	Soldan sağa
-	Çıkarma	Soldan sağa
<<	Sola kaydırma	Soldan sağa
>>	İşaret genişletmeli sağa kaydırma	Soldan sağa
>>>	Sıfır genişletmeli sağa kaydırma	Soldan sağa
<	Biraz az	Soldan sağa
<=	Biraz az veya eşit	Soldan sağa
>	Biraz fazla	Soldan sağa
>=	Biraz fazla veya eşit	Soldan sağa
örneği	Nesne türünü kontrol etme	Soldan sağa

Operatör	İsim	Bağlanırlık
==	Eşitlik karşılaştırması	Soldan sağa
!=	Eşit değil	Soldan sağa
&	(Koşulsuz VE)	Soldan sağa
^	(Özel VEYA)	Soldan sağa
	(Koşulsuz VEYA)	Soldan sağa
&&	Koşullu VE	Soldan sağa
	Koşullu VEYA	Soldan sağa
?:	Üçlü koşul	Sağdan sola
=	Atama	Sağdan sola
+=	Toplama ataması	Sağdan sola
-=	Çıkarma ataması	Sağdan sola
*=	Çarpma ataması	Sağdan sola
/=	Bölme ataması	Sağdan sola
%=	Kalan ataması	Sağdan sola

## Java Değiştiricileri

Değiştiriciler, sınıflar ve sınıf üyeleri (yapıcılar, yöntemler, veriler ve sınıf düzeyiblokları) üzerinde kullanılır, ancak **final** değiştiricisi bir yöntemde yerel değişkenler üzerinde de kullanılabilir. Bir sınıfa uygulanabilen bir değiştiriciye sınıf değiştiricisi denir. Bir yöntemde uygulanabilen bir değiştiriciye yöntem değiştiricisi denir. Bir veri alanına uygulanabilen bir değiştiriciye veri değiştiricisi denir. Bir sınıf düzeyi bloğuna uygulanabilen bir değiştiriciye blok değiştiricisi denir. Aşağıdaki tablo, Java değiştiricilerinin bir özetini vermektedir.

Değiştirici	SınıfYapıcı	Yöntem	Veri	Blok	Açıklama
(varsayılan)*	√	√	√	√	Bir sınıf, yapıcı, yöntem, veya veri alanı bu pakette görünür.
genel	√	√	√	√	Bir sınıf, yapıcı, yöntem, veya veri alanı herhangi bir paketteki tüm programlara görünür.
özel		√	√	√	Bir yapıcı, yöntem veya veri alanı yalnızca bu sınıfta görünür.
korumalı		√	√	√	Bir yapıcı, yöntem veya veri alanı bu pakette ve bu sınıfın alt sınıflarında herhangi bir pakette görünür.
statik		√	√	√	Bir sınıf yöntemi, bir sınıf veri alanı veya bir statik başlatma bloğu tanımlayın.
son	√	√	√		Son bir sınıf genişletilemez. Son bir yöntem alt sınıfta değiştirilemez. Son bir veri alanı bir sabittir.
soyut	√	√			Bir soyut sınıf genişletilebilir. Bir soyut yöntem somut bir alt sınıfta uygulanmalıdır.
yerel		√			Yerel bir yöntem, yöntemin Java dışındaki bir dil kullanılarak uygulandığını gösterir.

\*Varsayılan erişim ile ilişkili bir değiştirici yoktur. Örneğin: `class Test {}`



Değiřtirici	SınıfYapıcı	Yöntem	Veri	Blok	Açıklama
senkronize		√		√	Bu yöntemi aynı anda yalnızca bir iş parçacığı çalıştırabilir.
katıfp	√	√			Değerlendirme sonucunun tüm JVM'lerde aynı olmasını garanti etmek için katı kayan nokta hesaplamları kullanın.
geçici			√		Serileştirilemeyen bir örnek veri alanını işaretleyin.

Değiřtiriciler varsayılan (değiřtirici yok), **public**, **private** ve **protected**, sınıfların ve sınıf üyelerinin nasıl erişileceğini belirttikleri için

görünürlük veya erişilebilirlik değiřtiricileri olarak bilinir.

Değiřtiriciler **public**, **private**, **protected**, **static**, **final** ve **abstract** iç sınıflara da uygulanabilir.

# APPENDİK E

## Özel Kayar Nokta Değerleri

Bir tam sayıyı sifıra bölmek geçersizdir ve `ArithmeticException` fırlatır, ancak birkay ar nokta değerini sifıra bölmek bir istisna oluşturmaz. Kayar nokta aritmetiği, işlemi n sonucu `double` veya `float` için çok büyükse sonsuzluğa taşabilir veya sonucu `double` veya `float` için çok küçükse sifıra taşabilir. Java, bu sonuçları belirtmek için `POSITIVE_INFINITY`, `NEGATIVE_INFINITY` ve `NaN` (Bir Numara Değil) gibi özel kayar nokta değerlerini sağlar. Bu değerler, `Float` sınıfında ve `Double` sınıfında özel sabitler olarak tanımlanmıştır.

Eğer pozitif bir ondalık sayı sifıra bölünürse, sonuç `POZİTİF_SONSUZ` olur. Eğer negatif bir ondalık sayı sifıra bölünürse, sonuç `NEGATİF_SONSUZ` olur. Eğer bir ondalık sifır sifıra bölünürse, sonuç `NaN` olur, bu da sonucun matematiksel olarak tanımsız olduğu anlamına gelir. Bu üç değer string temsilleri `Sonsuz`, `-Sonsuz` ve `NaN`'dir. Örneğin,

```
System.out.print(1.0/0); // Sonsuz'u yazdır
System.out.print(-1.0/0); // -Sonsuz'u yazdır
System.out.print(0.0/0); // NaN'ı yazdır
```

Bu özel değerler hesaplamalarda operatör olarak da kullanılabilir. Örneğin, bir sayı `POZİTİF_SONSUZ` ile bölündüğünde pozitif bir sifır verir. Tablo E.1, `/`, `*`, `%`, `+` ve `-` operatörlerinin çeşitli kombinasyonlarını özetlemektedir.

TABLO E.1 Özel Ondalık Değerler

$x$	$y$	$x/y$	$xt*sen$	$x\%y$	$x + y$	$x - y$
Sonsuz	$\pm 0.0$	;kalay sonsuzluğu	$\pm 0.0$	NaN	Sonsuz	Sonsuz
Sonsuz	;kalay sonsuzluğu	$\pm 0.0$	$\pm 0.0$	x	;kalay sonsuzluğu	sonsuz
$\pm 0.0$	$\pm 0.0$	NaN	$\pm 0.0$	NaN	$\pm 0.0$	;t0.0n;
kalaysonsuz	Sonsuz	;kalay sonsuzluğu	$\pm 0.0$	NaN	;kalay sonsuzluğu	;kalaysonsuzluk;
kalaysonsuzluk	;kalay sonsuzluğu	NaN	$\pm 0.0$	NaN	;kalay sonsuzluğu	SONSUZ
$\pm 0.0$	;kalay sonsuzluğu	$\pm 0.0$	NaN	$\pm 0.0$	;kalay sonsuzluğu	;he0.0n
NaN	Herhangi bir	NaN	NaN	NaN	NaN	NaN
Herhangi bir	NaN	NaN	NaN	NaN	NaN	NaN



### Not

Eğer operandlardan biri NaN ise, sonuç NaN'dir.

# Sayı Sistemleri

## F.1 Giriş

Bilgisayarlar, içsel olarak ikili sayıları kullanır, çünkü bilgisayarlar doğal olarak 0 ve 1'leri depolamak ve işlemek için yapılmıştır. İkili sayı sistemi iki basamağa sahiptir: 0 ve 1. Bir sayı veya karakter 0 ve 1'lerin bir dizisi olarak depolanır. Her 0 veya 1'e *bit* (ikili basamak) denir.

Günlük hayatımızda ondalık sayıları kullanırız. Bir programda 20 gibi bir sayı yazdığımızda, onun ondalık bir sayı olduğu varsayılır. İçsel olarak, bilgisayar yazılımı ondalık sayıları ikili sayılara ve tersine dönüştürmek için kullanılır.

Bilgisayar programlarını ondalık sayılar kullanarak yazarız. Ancak, bir işletim sistemi i le çalışmak için, ikili sayıları kullanarak "makine seviyesine" inmemiz gerekir. İkili sayılar genellikle çok uzun ve karmaşık olma eğilimindedir. Genellikle, her bir onaltılık basamağ ın dört ikili basamağı temsil ettiği onaltılık sayılar kullanılarak kısaltılır. Onaltılık sayı sist emi 16 basamağa sahiptir: 0-9 ve A-F. A, B, C, D, E ve F harfleri sırasıyla ondalık sayılar 1 0, 11, 12, 13, 14 ve 15'e karşılık gelir.

Ondalık sayı sistemindeki basamaklar 0, 1, 2, 3, 4, 5, 6, 7, 8 ve 9'dur. Bir ondalık sayı, bu bas amaklardan bir veya daha fazlasının dizisi ile temsil edilir. Her basamağın temsil ettiği değ er, b ulunduğu konuma bağlıdır; bu konum, 10'un tam bir kuvvetini belirtir. Örneğin, ondalık sayı 7 423'teki 7, 4, 2 ve 3 basamakları sırasıyla 7000, 400, 20 ve 3'ü temsil eder, aşağıda gösterildiği gibi:

$$\begin{array}{|c|c|c|c|} \hline 7 & 4 & 2 & 3 \\ \hline \end{array} = 7 \times 10^3 + 4 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$$

$$10^3 \ 10^2 \ 10^1 \ 10^0 = 7000 + 400 + 20 + 3 = 7423$$

Ondalık sayı sistemi on basamağa sahiptir ve pozisyon değerleri 10'un tam sayı kuvvetleridir. 10'un ondalık sayı sisteminin *tabanı* veya *radix* olduğunu söyleriz. Benzer şekilde, ikili sayı sisteminin ik i basamağı olduğundan, tabanı 2'dir ve onaltılık sayı sisteminin 16 basamağı olduğundan, tabanı 16'd ır.

Eğer 1101 bir ikili sayı ise, basamaklar 1, 1, 0 ve 1 şunları temsil eder  $1 \times 2^3$ ,  $1 \times 2^2$ ,  $0 \times 2^1$ , ve  $1 \times 2^0$ , **sırasıyla:**

$$\begin{array}{|c|c|c|c|} \hline 1 & 1 & 0 & 1 \\ \hline \end{array} = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$2^3 \ 2^2 \ 2^1 \ 2^0 = 8 + 4 + 0 + 1 = 13$$

Eğer 7423 bir onaltılık sayı ise, basamaklar 7, 4, 2 ve 3 şunları temsil eder  $7 \times 16^3$ ,  $4 \times 16^2$ ,  $2 \times 16^1$ , ve  $3 \times 16^0$ , **sırasıyla:**

$$\begin{array}{|c|c|c|c|} \hline 7 & 4 & 2 & 3 \\ \hline \end{array} = 7 \times 16^3 + 4 \times 16^2 + 2 \times 16^1 + 3 \times 16^0$$

$$16^3 \ 16^2 \ 16^1 \ 16^0 = 28672 + 1024 + 32 + 3 = 29731$$

ikili sayılar

ondalık sayılar

onaltılık sayı

taban

radix

## F.2 İkili ve Ondalık Sayılar Arasındaki Dönüşümler

ikili sistemden ondalık sisteme

Bir ikili sayı verildiğinde  $t_{n-1}t_{n-2} \dots t_2t_1t_0$ ,

eşdeğer ondalık değeri

$$b_n = t_{n-1} \times 2^{n-1} + t_{n-2} \times 2^{n-2} + \dots + t_2 \times 2^2 + t_1 \times 2^1 + t_0 \times 2^0$$

İkili sayıları ondalıklara dönüştürmenin bazı örnekleri:

İkili	Dönüşüm Formülü	Ondalık
10	$1 \times 2^1 + 0 \times 2^0$	2
1000	$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$	8
10101011	$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$	171

ondalık sistemden ikili sisteme

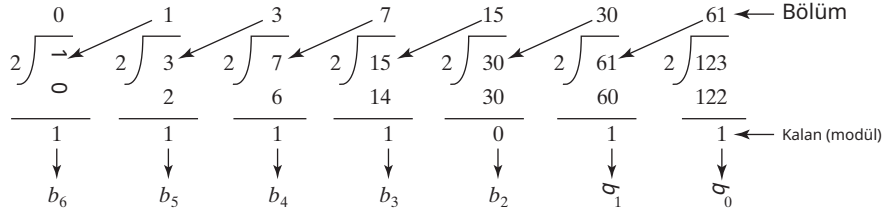
Bir ondalık sayıyı  $d$  ikili bir sayıya dönüştürmek, bitleri bulmak demektir ve böylece

$b_{n-1}b_{n-2} \dots b_2b_1b_0$

$$d = b_n \times 2^n + b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \dots + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0$$

Bu bitler,  $d$ 'yi 2'ye bölerek ve bölüm 0 olana kadar devam ederek bulunabilir. Kalanlar şunlardır:

Örneğin, ondalık sayı 123, ikili sistemde 1111011'dir. Dönüşüm şu şekilde yapılır:



### Uç

Windows Hesap Makinesi, Şekil F.1'de gösterildiği gibi, sayı dönüşümleri yapmak için yararlı bir araçtır. Çalıştırmak için, *Başlat* düğmesinden *Hesap Makinesi* araması yapın ve Hesap Makinesi'ni başlatın, sonra *Görünüm* altında *Bilimsel* seçeneğini seçin.

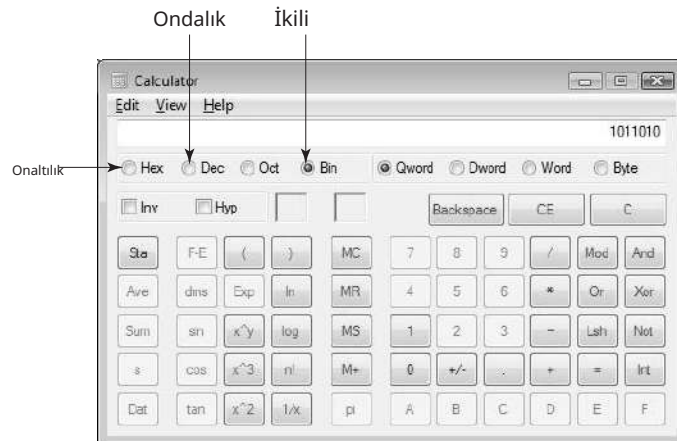


FIGURE F.1 Windows Hesap Makinesi'ni kullanarak sayı dönüşümleri gerçekleştirebilirsiniz.

## F.3 Onaltılık ve Ondalık

Sayılar Arasındaki Dönüşümler

Bir onaltılık sayı verildiğinde  $-t_1t_0t_{-1}t_{-2} \dots t_{-n}t_{-n-1}t_{-n-2} \dots$  eşdeğer ondalık değeri

onaltılıktan ondalık

$$h_{-n} \times 16^{-n} + t_{-n-1} \times 16^{-n-1} + t_{-n-2} \times 16^{-n-2} + \dots + t_{-1} \times 16^{-1} + t_0 \times 16^0 + h_1 \times 16^1 + h_2 \times 16^2 + \dots$$

Onaltılık sayıları ondalıklara dönüştürmenin bazı örnekleri:

Onaltılık	Dönüşüm Formülü	Ondalık
7F	$7 \times 16^1 + 15 \times 16^0$	127
nFFFF	$15 \times 16^3 + 15 \times 16^2 + 15 \times 16^1 + 15 \times 16^0$	65535
431	$4 \times 16^2 + 3 \times 16^1 + 1 \times 16^0$	1073

Ondalık bir sayıyı  $d$  onaltılık bir sayıya dönüştürmek, onaltılık basamakları bulmak demektir ve böylece

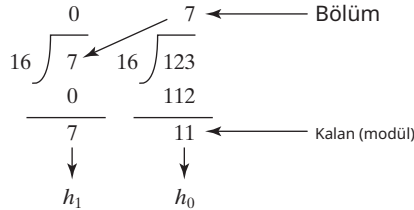
ondalıktan onaltılık

$$d = h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \dots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

Bu sayılar,  $d$ 'yi 16'ya ardışık olarak bölerek, bölüm 0 olana kadar bulunabilir. Kalanlar  $t_{-1}, t_{-2}, \dots, t_{-n}$  ve  $h_{-n}$ .

Örneğin, ondalık sayı 123, onaltılık sistemde 7B'dir. Dönüşüm şu şekilde yapılır:

şu şekildedir:



## F.4 İkili ve Onaltılık Sayılar Arasındaki Dönüşümler

Bir onaltılık sayıyı ikili bir sayıya dönüştürmek için, onaltılık sayıdaki her basamağı dört basamaklı bir ikili sayıya dönüştürün, F.1 Tablosunu kullanarak.

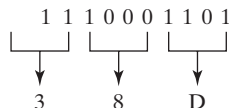
onaltılıktan ikili

Örneğin, onaltılık sayı 7B, 1111011'dir; burada 7, ikili olarak 111'dir ve B, ikili olarak 1011'dir.

Bir ikili sayıyı onaltılığa dönüştürmek için, ikili sayıdaki her dört ikili basamağı sağdan sola onaltılık bir sayıya dönüştürün.

ikiliyi onaltılığa

Örneğin, ikili sayı 1110001101, 38D'ye eşittir; çünkü 1101 D, 1000 8 ve 11 3'tür, aşağıda gösterildiği gibi.



**TABLE F.1** Onaltılık Sayıyı İkili Sayıya Dönüştürme

Onaltılık	İkili	Ondalık
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	
A	1010	10n
B	1011	11n
C	1100	12n
D	1101	13n
E	1110	14n
F	1111	15

**Not**

Sekizli sayılar da faydalıdır. Sekizli sayı sistemi 0'dan 7'ye kadar sekiz basamağa sahiptir. Onluk sayı 8, sekizli sistemde 10 olarak temsil edilir.

İşte sayı dönüşümleri pratiği yapmak için bazı iyi çevrimiçi kaynaklar:

[http://forums.cisco.com/CertCom/game/binary\\_game\\_page.htm](http://forums.cisco.com/CertCom/game/binary_game_page.htm)

<http://people.sinclair.edu/nickreeder/Flash/binDec.htm>

<http://people.sinclair.edu/nickreeder/Flash/binHex.htm>

**F.1** Aşağıdaki ondalık sayıları onaltılık ve ikilik sayılara dönüştürün:

100; 4340; 2000

**F.2** Aşağıdaki ikilik sayıları onaltılık ve ondalık sayılara dönüştürün:

1000011001; 100000000; 100111

**F.3** Aşağıdaki onaltılık sayıları ikilik ve ondalık sayılara dönüştürün:

FFAQ; 93; 2000

Point Kontrol

MyProgrammingLab™

## Bit Düzeyinde İşlemler

Makine düzeyinde program yazmak için genellikle ikili sayılarla doğrudan ilgilenmeniz ve bit düzeyinde işlemler gerçekleştirmeniz gerekir. Java, Tablo G.1'de tanımlanan bit düzeyindeki operatörleri ve kaydırma operatörlerini sağlar.

Bit operatörleri yalnızca tam sayı türlerine (byte, short, int ve long) uygulanır. Bir bit işlemine dahil olan bir karakter, bir tam sayıya dönüştürülür. Tüm bit düzeyindeki operatörler, =, |=, <<=, >>= ve >>>= gibi bit düzeyinde atama operatörleri oluşturabilir.

**Tablo G.1**

Operatör	İsim	Örnek (örnekteki baytları kullanarak)	Açıklama
&	Bit düzeyinde VE	10101110 & 10010010 10000010 verir	İki karşılık gelen baytın VE'si, her iki bayt da 1 ise 1 verir.
	Bit düzeyinde kapsayıcı VEYA	10101110   10010010 10111110 verir	İki karşılık gelen baytın VEYA'sı, herhangi bir bayt 1 ise 1 verir.
^	Bit düzeyinde hariç tutan VEYA	10101110 ^ 10010010 00111100 verir	İki karşılık gelen bitin XOR'u yalnızca iki bit farklıysa 1 verir.
~	Birinin tamamlayıcı	~10101110 verir 01010001	Operatör her biti 0'dan 1'e ve 1'den 0'a geçirir.
<<	Sola kaydırma	10101110 << 2 verir 10111000	Operatör, ilk operandın bitlerini ikinci operandda belirtilen bit sayısı kadar sola kaydırır ve sağda 0'larla doldurur.
>>	İşaret genişletmeli sağa kaydırma	10101110 >> 2 verir 11010110 >> 2 verir 00001011	Operatör, ilk operandın bitlerini, ikinci operandda belirtilen bit sayısı kadar sağa kaydırır ve sol tarafta en yüksek (işaret) bit ile doldurur.
>>>	İşaret genişletmeli sağa kaydırma	10101110 >> 2 verir 00101010 >> 2 verir 00001011	Operatör, ilk operandın bitini, ikinci operandda belirtilen bit sayısı kadar sağa kaydırır ve solda 0'lar ile doldurur.

*Bu sayfa kasıtlı olarak boş bırakılmıştır*



# İNDİS

## Semboller

-- (azaltma operatörü), 54–56  
- (çıkarma operatörü), 46, 50  
. (nokta operatörü), 23, 304  
. (nesne üye erişim operatörü), 304, 427  
/ (bölme operatörü), 46, 50  
//, satır içi yorum sözdiziminde, 25  
/\*, blok yorum sözdiziminde, 25  
/\*\*.\*/ (Javadoc yorum sözdizimi), 25  
/= (bölme atama operatörü), 53–54  
; (noktalı virgüller), yaygın hatalar, 93  
(ters eğik çizgi karakteri), dizin ayırıcı olarak, 542  
(kaçış karakterleri), 63–64  
|| (veya mantıksal operatör), 102–105  
+ (toplama operatörü), 46, 50  
+ (dize birleştirme operatörü), 38, 340  
++ (artırım operatörü), 54–56  
+= (toplama atama operatörü), artırılmış, 53–54  
= (atama operatörü), 42–43, 53–54  
= (eşitlik operatörü), 82  
-= (çıkarma atama operatörü), 53–54  
== (karşılaştırma operatörü), 82, 430  
== (eşit olan operatör), 82  
! (mantıksal değil operatörü), 102–105  
!= (eşit olmayan karşılaştırma operatörü), 82  
\$ (dolar işareti karakteri), kaynak kodunda kullanın, 40  
% (kalan veya modül operatörü), 46, 50  
%= (kalan atama operatörü), 53–54  
&& (ve mantıksal operatörü), 102–105  
( ) (parantezler), 18, 203  
\* (çarpma operatörü), 19, 46, 50  
\*= (çarpma atama operatörü), 53–54  
^ (özel veya mantıksal operatörü), 102–105  
{ } (küme parantezleri), 17–18, 85, 93  
< (küçüktür karşılaştırma operatörü), 82  
<= (küçük veya eşit karşılaştırma operatörü), 82  
(büyüktür karşılaştırma operatörü), 82  
(büyüktür veya eşittir karşılaştırma operatörü), 82

## A

**abs** metodu, **Math** sınıfı, 199–200, 588  
Kesin dosya adı, 541  
Soyut sınıflar  
örnek olay: soyut sayı sınıfı, 565–567  
örnek olay: **Takvim** ve **GregorianTakvim** sınıfları, 576–588  
özellikleri, 564–565  
**Circle.java** ve **Rectangle.java** örnekleri, 562  
arayüzlerle karşılaştırıldığında, 581–584

**Bileşen** ve **JComponent** olarak, 447  
**FontMetrics** olarak, 493  
**GeometricObject.java** örneği, 560–562  
**InputStream** ve **OutputStream** sınıfları, 712–713

arayüzlerle karşılaştırıldığında, 570

anahtar terimler, 590

genel bakış, 559–560

sorular ve alıştırmalar, 590–598

**Rational.java** örneği, 586–589

soyut yöntemlerin kullanıma nedenleri, 562

özet, 590

**TestCalendar.java** örneği, 568–570

**TestGeometricObject.java** örneği, 562–563

**TestRationalClass.java** örneği, 585–586

Soyut veri türü (ADT), 375

Soyut yöntemler

özellikleri, 564

**GeometricObject** sınıfı, 561–562

alt sınıflarda uygulama, 560

arayüzlerde, 570

içinde **Number** sınıfı, 588

kullanma nedenleri, 562–563

soyut değiştirici, soyut yöntemleri belirtmek için, 560

Soyut sayı sınıfı

**LargestNumbers.java**, 566–567

genel bakış, 565–567

Soyut Windows Araç Takımı. bkz AWT (Soyut Windows Araç Takımı)

**Soyut Buton** sınıfı

hızalama, 470

özet, 468–469

Soyutlama

sınıflar, 375–376

metotlar, 203–204

Erişimci metodlar. bkz Getter (erişimci) metodlar

**acos** metodu, trigonometrik, 197–198

**Eylem Olayları**

GUI bileşenleri tetikleme, 640

**JComboBox** sınıfı, 647, 650

olay işleyicileri ile işleme, 604–605

Zamanlayıcı sınıfı tetikleme, 625–626

**Eylem Dinleyici** arayüzü

animasyon kullanarak Zamanlayıcı sınıfı, 625

**ControlCircle.java**, 607–608

**DetectSourceDemo.java**, 613–614

event işleyicileri ve dinleyicileri kaydetme, 604–605

özet, 600–602

Eylemler (davranışlar), nesne, 296

Aktivasyon kayıtları, metodları çağırma ve, 182

Gerçek parametreler, metodları tanımlama ve, 179

Ada, yüksek seviyeli diller, 11

ekle yöntemi, bileşenleri çerçevelere eklemek için, 450–451

**addActionListener** yöntemi, 600

Toplama (+) operatörü, 46, 50

Toplama (+=) atama operatörü, artırılmış atama operatörleri, 53–54

Ayar dinleyicileri, 656

ADT (soyut veri türü), 375

Toplayıcı sınıflar, 382

Toplayıcı nesneler, 382

Toplama ilişkileri, nesneler, 382–383

AIFF ses dosyaları, 693

Algoritmalar, 34

Hizalama, **JBUTTON** sınıfı ve, 470

Belirsiz çağrı, yöntemlerin, 195

Amerikan Standart Bilgi Değişim Kodu (ASCII).  
gör ASCII (Amerikan Standart Bilgi Değişimi Kodu)

Anagramlar, 364

Ve (&&) mantıksal operatör, 102–105

Android telefonlar, Java kullanımı, 15

Animasyon

**AnimationDemo.java**, 626–628

**ClockAnimation.java**, 628–629

kullanarak **Timer** sınıfı için, 625–626

Anonim diziler, 238

Anonim sınıf dinleyicileri

**Anonimlistenderdemo.yavat**, 610612nlistd

**emo.yavat**, 653

**MoveMessageDemo.java**, 619

özeti, 609–610

Anonim nesneler, 305

API'ler (Uygulama Programı Arayüzleri), 16

**Applet** sınıfı

applet geliştirme, 672

üst düzey konteyner olarak, 447

Applet Viewer aracı, 675

**<applet>** etiketi, HTML, 673

Uygulamalar

vaka çalışması: zıplayan top, 683–686

vaka çalışması: ulusal bayraklar ve marşlar, 695–697

vaka çalışması: üçlü oyun, 686–688

konteynerler, 447, 677

geliştirme, 672

**displayimageplayaudio.yavat**, **shachn**

**displayimagevithurl.yavat**, 692693nddis

**playlabel.htmlt**, **shshzshshch**

**DisplayLabel.java**, 672–673, 676–677

**DisplayMessageApp.java**, 681–683

**DisplayMessage.html**, 679–680

**DisplayMessage.java**, 680–681

uygulama olarak çalıştırma yetkisi verme, 676–677

HTML **<applet>** etiketi, 673

Cava, 14

anahtar terimler, 697

yaşam döngüsü yöntemleri, 677–678

kaynakları **URL** sınıfı kullanarak bulma, 691–692

genel bakış, 671–672

string'leri geçirme, 679

audio dosyalarını oynatma, 693–694

sorular ve alıştırmalar, 698–708

güvenlik kısıtlamaları, 675–676

özet, 697–698

**TicTacToe.java**, 688–691

Web tarayıcısından görüntüleme, 674–675

Applet Viewer aracıyla görüntüleme, 675

**appletviewer** komutu, 675

Uygulama, applet'lerin çalışmasını sağlama, 676–677

Uygulama Programı Arayüzleri (API'ler), 16

Uygulamalar, Web sunucularında geliştirme, 14

**Arşiv** niteliği, applet'ler, 674

Yaylar, çizim, 488–490

Argümanlar

metotları tanımlama ve, 179

değerlerle geçiş, 186–189

komut satırından dize argümanları alma, 358–359

değişken uzunlukta argüman listeleri, 244–245

**AritmetikHatası** sınıfı, 521

Aritmetik/lojik birimler, CPU bileşenleri, 3

Dizi başlatıcıları, 227

Dizi değişkenleri, 226

**dizi kopyalama** metodu, **Sistem** sınıfı, 236

**DiziİndeksiHataları**, 230

**DiziListesi** sınıfı

vaka çalışması: özel yığın sınıfı, 436–437

dizileri klonlama, 578

dizi listelerine sayı ekleme ve oluşturma, 566–567

**DistinctNumbers.java** örneği, 434–436

nesne listesini depolama, 430–431

**TestArrayList.java** örneği, 431–434

Diziler, çok boyutlu

vaka çalışması: günlük sıcaklık ve nem, 278–279

vaka çalışması: doğum günlerini tahmin etme, 279

genel bakış, 277–278

sorular ve alıştırmalar, 281–294

özet, 281

Diziler, tek boyutlu

**ArrayList** sınıfı, 433–434

Diziler sınıfı, 252–253

vaka çalışması: harflerin sayısını sayma, 241–244

vaka çalışması: iskambil destesi, 234–236

örnek çalışma: loto numaraları, 231–234

string'leri oluşturma, 336

string'leri dönüştürme, 343–344

kopyalama, 236–237

oluşturma, 225–226, 574–576

bildirme, 225

**her biri için** döngüler, 229–231

indeksli değişkenler, 226–227

başlatıcılar, 227

anahtar terimler, 253

nesnelerin, 326–328

gözden geçirme, 223–225

metotlara geçirme, 237–240  
işleme, 227–229  
sorular ve alıştırmalar, 255–262  
metotlardan döndürme, 240–241  
arama, 245–248  
serileştirme, 728–729  
boyut ve varsayılan değerler, 226  
sıralama, 248–252, 574–576  
özet, 253–254  
Java'da nesne olarak ele alma, 304  
değişken uzunlukta argüman listeleri, 244–245

**Diziler, iki boyutlu**  
örnek çalışma: en yakın nokta çiftini bulma, 272–273  
örnek çalışma: çoktan seçmeli testi notlandırma, 270–272  
örnek çalışma: Sudoku, 274–277  
değişkenleri tanımlama ve iki boyutlu diziler oluşturma, 264–265  
iki boyutlu dizilerin uzunluğunu elde etme, 265–266  
özet, 263–264  
iki boyutlu dizilere yöntemlere geçirme, 269–270  
iki boyutlu dizileri işleme, 267–269  
sorular ve alıştırmalar, 281–294  
sarkık diziler, 266–267  
özet, 281

**Diziler sınıfı**, 252–253  
Ok tuşları, klavyelerde, 8

Yükseliş, metin yazı tiplerinde, 493

**ASCII (Amerikan Standart Bilgi Değişim Kodu)**  
karakter veri türü (**char**) ve, 63  
veri girişi ve çıkışı akışları, 717  
kodlama şeması, 4  
metin kodlaması, 710  
metin G/Ç ile ikili G/Ç, 711

**asin** metodu, trigonometrik, 197–198

Montajcılar, 10  
Montaj dili, 10

**Atama operatörü (=)**  
geliştirilmiş, 53–54  
genel bakış, 42–43

**Atama ifadeleri (atama ifadeleri)**  
değeri değişkenlere atama, 36  
genel bakış, 42–43

**Operatörlerin bağlanırlığı, 116**

**atan** metodu, trigonometrik, 197–198

Nitelikler, nesne, 296

AU dosyaları, ses formatları, 693

Ses dosyaları  
oynatma için appletler, 693–694  
vaka çalışması: ulusal bayraklar ve marşlar, 695–697  
**DisplayImagePlayAudio.java**, 694

**AudioClip** nesneleri, 693–694

Autoboxing/Autounboxing, 396–397

**AWT (Soyut Windows Araç Takımı)**  
**Applet** sınıfı, 447, 672  
**Color** sınıfı, 460–462  
**Component** sınıfı. *bkz.* **Component** sınıfı

**Container** sınıfı. *bkz.* **Containers/Container** sınıfı

**Date** sınıfı, 308–309, 567–568

**Dialog** sınıfı, 447

**Dimension** sınıfı, 448

**Error** sınıfı, 524, 526

event sınıfları, 603

**EventObject** sınıfı, 602–603, 605

istisnalar. *bkz.* **Exception** sınıfı

**FigurePanel** sınıfı, 485–488

**File** sınıfı, 541–543, 710

**FlowLayout** sınıfı. *bkz.* **FlowLayout** sınıfı

**Font** sınıfı, 461–462

**Frame** sınıfı, 447

**GeometricObject** sınıfı, 560–563

**Graphics** sınıfı. *bkz.* **Graphics** sınıfı

**GregorianCalendar** sınıfı. *bkz.* **GregorianCalendar** sınıfı

**GridLayout** sınıfı. *bkz.* **GridLayout** sınıfı

**GuessDate** sınıf, 388–391

**IllegalArgumentException** sınıf, 527

**Image** sınıf, 504

**ImageViewer** sınıf, 506–508

**InputMismatchException** sınıf, 522–523, 547

**KeyEvent** sınıf, 621

**LayoutManager** sınıf, 448

**MalformedURLException** sınıf, 551

**MessagePanel** sınıf, 495–497

**MouseEvent** sınıf, 617–619

**Point** sınıf, 617–618

**Polygon** sınıf, 490

**Scanner** sınıf. *bkz.* **Scanner** sınıf

**String** sınıf, 336

Salıncak vs., 446

## B

Babil yöntemi, 217

Arka plan rengi, ayar, 463

Ters eğik çizgi karakteri (|), dizin ayırıcı olarak, 542

Temel durumlar, özinelemede, 744

**BASIC**, yüksek seviyeli diller, 11

Bean makinesi oyunu, 258–259, 514, 633

**beginIndex** metodu, alt dizeleri elde etmek için dizilerden, 341

Davranışlar (hareketler), nesne, 296

Sahne arkasında değerlendirme, ifadeler, 116

**BigDecimal** sınıfı, 397–398, 565

**İkili**  
dosyalar, 710  
makine dili ikili kod olarak, 9–10  
operatörler, 47  
arama, 246–248, 748–749

ikili rakamlar (Bitler), 4

**İkili G/Ç**  
**BufferedInputStream** ve **BufferedOutputStream** sınıfları, 719–722  
karakterler ve dizeler, 716

İkili G/Ç (*devam*)

sınıfları, 712–713

**DataInputStream** ve **DataOutputStream** sınıfları, 716–718**DetectEndOfFile.java**, 719**FileInputStream** ve **FileOutputStream** sınıfları, 713–714**FilterInputStream** ve **FilterOutputStream** sınıfları, 716 özet, 710**TestDataStream.java**, 718–719**TestFileStream.java**, 714–716

vs. metin G/Ç, 711–712

## İkili sayılar

ondalığa dönüştürme, 763

birbirine dönüştürme, 364

**binarySearch** metodu, Arrays sınıfı, 252–253

Bitler (ikili rakamlar), 4

Blok yorumları, **Welcome.java** içinde, 17

Blok stili, programlama stili, 25–26

Blokler, **Welcome.java** içinde, 17

BMI (Vücut Kitle İndeksi), 97–99, 379–382

Boolean erişim metodu, 320

**boolean** veri tipi**java.util.Random**, 309–310

özet, 82–84

## Boolean ifadeleri

örnek olay: artık yıl belirleme, 105–106

koşullu ifadeler, 111–112

tanımlandı, 82

**eğer** ifadeleri ve, 84–85**eğer-değil** ifadeleri, 89–91

yazma, 95

Boolean sabitleri, 83

Boolean değerleri

tanımlandı, 82

format belirteci olarak, 113

cast etme yeteneği yok, 104

mantıksal operatörler ve, 101–102

testte gereksizlik, 94

Boolean değişkenleri

atama, 95

genel bakış, 83

testte gereksizlik, 94

**BorderLayout** sınıfı

genel bakış, 456

özellikleri, 457

**ShowBorderLayout.java** örneği, 456–457

## Sınırlar

**JComponent** sınıfı, 463

ayar, 463–464

paylaşım, 466

Aşağıdan yukarıya uygulama, 205–207

Kutuya alma, sarmalayıcı nesneyi ilkel değere dönüştürme, 396

Küme parantezleri. *bkz.* Küçük parantezler (**{}**)**break** ifadeleri

döngüleri kontrol etme, 159–162

**switch** ifadeleri ile kullanma, 109

Kesme noktaları, hata ayıklama için ayarlama, 120

Baloncuk sıralamaları, 258

**BufferedInputStream** ve **BufferedOutputStream** sınıfları, 719–722

Hatalar (mantık hataları), 27–28, 119–120

Otobüs, işlevi, 2–3

**Düğme Dinleyici** sınıfı, 617

Düğmeler

**BorderLayout** ekleme, 457

oluşturma, 311, 451

gruplama, 641

kullanılan simgeler, 469

görüntü simgeleri olarak görüntülenen, 466

**JButton** sınıfı. *bkz.* **JButton** sınıfı**JRadioButton** sınıfı. *bkz.* **JRadioButton** sınıfı

metin konumları, 470–471

türleri, 468

**byte** türü, sayısal türler, 45

Bytecode

Java kaynak dosyasını, 20–21

doğrulayıcı, 22'ye çevirme

Baytlar

tanımlandı, 4

depolama kapasitesini ölçme, 5

## C

C, yüksek seviyeli diller, 11

C++, yüksek seviyeli diller, 11

Kablo modemleri, 8

**Takvim** sınıfı, 567–568

Çağrı yığınları

hata ayıklamada görüntüleme, 120

metotları çağırma ve, 182

Çağırma

metotlar, 180–182

nesneler, 305

**canRead** metodu, **File** sınıfı, 542–543**canWrite** metodu, **File** sınıfı, 542–543**capacity** metodu, **StringBuilder** sınıfı, 355–356

Büyük/küçük harf duyarlılığı

tanımlayıcılar ve, 40

**Welcome.java** içinde, 18Casting. *bkz.* Tür dönüştürme

Nesneleri dönüştürme

**CastingDemo.java** örneği, 426–429

gözden geçirme, 425–426

## Hataları yakalama

**catch** bloğu, **finally** ifadesi kullanıldığında atlanır, 535**CircleWithException.java** örneği, 532**InputMismatchExceptionDemo.java** örneği, 522**QuotientWithException.java** örneği, 520–521**try-catch** blokları, 527–529

CD'ler (kompakt diskler), depolama cihazı olarak, 6

Hücreler

Sudoku ızgarasında, 274

tic-tac-toe vaka çalışmasında, 687–691

Celsius, Fahrenheit'a dönüştürme, 50–51, 213

Zincirlenmiş istisnalar, 537–538

**Karakter** sınıfı, 350–351

Karakterler/ karakter veri türü (**char**)

sayısal operatörlerin uygulanması, 201–202

ikili G/Ç'de, 716–717

vaka çalışması: para birimlerini sayma, 65–68

vaka çalışması: palindromları kontrol ederken

alfa sayısal olmayan karakterleri göz ardı etme, 356–358

sayısal türlere dönüştürme, 63–65

**Karakter** sınıfı, 350–351

karşılaştırma, 82

dizilerden dizgeler oluşturma, 336

dizgelere dönüştürme, 344

**CountEachLetter.java** örnek, 351–353

kaçış karakterleri, 63–64

bulma, 342–343

özet, 62

**RandomCharacter.java**, 202

stringlerde alma, 339–340

**TestRandomCharacter.java**, 202–203

Unicode ve ASCII ve, 62–63

**charAt** (indeks) metodu

stringlerde karakter alma, 339–340

**StringBuilder** sınıfı, 355–356

**charValue** metodu, **Character** sınıfı, 350

Onay kutuları

oluşturma, 311

eventler, 640–643

**JCheckBox** sınıfı, 471–472

buton türleri, 468

Kontrol edilen istisnalar, 525

Seçim listeleri. *bkz* Kombinasyon kutuları

**Daire** sınıfı, 296–297

Açıklık, sınıf tasarım yönergeleri, 392

Sınıf soyutlaması, 375

Sınıf diyagramları, UML, 297

Sınıf kapsülleme, 375–376

Sınıf yükleyicileri, 22

Sınıf değişkenleri, 312

**ClassCastException**, 426

Sınıflar

özet. *bak* Özet sınıflar

soyutlama ve kapsülleme, 375–376

vaka çalışması: yığınlar için sınıf tasarımı, 386–388

vaka çalışması: **Kurs** sınıfı tasarımı, 384–386

**CircleWithPrivateDataFields.java** örneğinde, 320–321

içinde **CircleWithStaticMembers.java** örneği, 313–314

müşterileri, 299

yorumlama, 25

içinde **ComputeExpression.java**

, 19data alanı kapsülleme, 319–320\Özel istisna

sınıflarını tanımlama, 538–541diziler için t

anımlama, 296–298

tasarım kılavuzları, 391–393

yeniden kullanım için tasarımı, 499

GUI bileşenlerini görüntülemek için, 310–312

event sınıfları, 603

olay dinleyicileri. *bkz* Dinleyici sınıfları

tanımlayıcılar, 40

iç (iç içe) sınıflar. *bkz* İç (iç içe) sınıflar

Java Kütüphanesinden, 308

isimler/adlandırma kuralları, 17, 44

uzantı önleme, 439–440

ilkel veri türleri için, 350

statik değişkenler, sabitler ve yöntemler, 312–313

**TestCircleWithPrivateDataFields.java** örneğinde, 321–322

in **TestCircleWithStaticMembers.java** örneğinde, 314–317

UML diyagramında, 298

değişken kapsamı ve, 371–372

görünürlük değiştiricileri, 317–319

**Welcome.java** içinde, 17

in **WelcomeWithThreeMessages.java** içinde, 18

Sınıflar, ikili G/Ç

**BufferedInputStream** ve **BufferedOutputStream** sınıfları, 719–722

**DataInputStream** ve **DataOutputStream** sınıfları, 716–718

**DetectEndOfFile.java**, 719

**FileInputStream** ve **FileOutputStream** sınıfları, 713–714

**FilterInputStream** ve **FilterOutputStream** sınıfları, 716

genel bakış, 712–713

**TestDataStream.java**, 718–719

**TestFileStream.java**, 714–716

Sınıfın sözleşmesi, 375

Saat hızı, CPU'lar, 3

**clone** metodu, yüzeysel ve derin kopyalar, 579–580

**Cloneable** arayüzü

**House.java** örneği, 578–581

genel bakış, 577–578

En yakın çift problemi, iki boyutlu diziye uygulanması, 272–273

COBOL, yüksek seviyeli diller, 11

Kod

basitleştirmek için diziler, 229

yorumlar ve, 109

programlama. *bkz.* Programlar/programlama

yeniden kullanım. *bkz.* Yeniden kullanılabilir kod

paylaşım. *gör* Paylaşım kodu

**Kod Tabanı** niteliği, applet'ler, 674

Kodlama (Uygulama)

incremental geliştirme, 137

yazılım geliştirme sürecinde, 60–61

Tutarlı amaç, sınıf tasarım kılavuzları, 391

Renk

**Bileşen** sınıfı ve, 499

arka plan ve ön plan rengini ayarlama, 463

**Renk** sınıfı

yardımcı sınıflar, 446, 448

Java GUI API'sinde, 460–462

## Combo kutuları

**ComboBoxDemo.java**, 648–649

oluşturma, 310–312

genel bakış, 647–648

Komut satırı argümanları, 358–361

## Yorumlar

kodun sürdürülebilirliği ve, 109

programlama stili ve, 25

içinde **Welcome.java**, 17

Ortak payda, en büyük ortak paydanayı bulma.

*bkz* EBO (en büyük ortak payda)

İletişim cihazları, bilgisayarlar ve, 8–9

Kompakt diskler (CD'ler), depolama cihazı olarak, 6

## Karşılaştırılabilir arayüz

**ComparableRectangle.java** örnek, 575–576

özet, 573–574

**Rasyonel** sınıf uygulaması, 585**SortComparableObjects.java** örnek, 574–575**SortRectangles.java** örnek, 576–577**compareTo** metodu**Karakter** sınıf, 350–351**Cloneable** arayüz ve, 577**Karşılaştırılabilir** arayüz tanımlama, 573–574**ComparableRectangle.java** örnek, 575–576

stringleri karşılaştırma, 338

**Rasyonel** sınıfında uygulama, 588

sarmalayıcı sınıflar ve, 394

**compareToIgnoreCase** metodu, stringler, 338–339

Stringleri karşılaştırma, 337–339

Karşılaştırma operatörü (**=**), 82, 430

Derleme hataları (Sözdizim hataları)

ortak hatalar, 18

debugging, 119–120

programlama hataları, 26–27

## Derleyiciler

belirsiz çağrı ve, 195

sözdizim hatalarını raporlama, 26

Java kaynak dosyasını bytecode dosyasına çevirme, 20–21

kaynak programı makine koduna çevirme, 10–11

Tamlik, sınıf tasarım kılavuzları, 392

Kompleks sayılar, **Math** sınıfı, 594**Bileşen** sınıfı

soyut sınıf olarak, 447

renk ve yazı tipi yöntemleri

, 499 **Bileşen**, **Kapsayıcı** ve **JComponent** orta  
k özellikleri, 462

alt sınıfları, 446, 451

## Bileşenler

**BorderLayout** ekleme, 456

çerçevelere ekleme, 450–451

kombinasyon kutuları, 647–648

**ComboBoxDemo.java**, 648–649

ortak özellikleri, 462

Swing ve AWT bileşenlerini karşılaştırma, 446

bileşen sınıfları, 446–447

**AçıklamaPanel.java**, 645–646

eventler, 640

**GUIEventDemo.java**, 640–643**Histogram.java**, 662–664**JComponent** sınıfı. *bkz* **JComponent** sınıfı**JFrame** görüntüleme, 449**JTextComponent** sınıfı, 474–475**ListDemo.java**, 652–654

listeler, 649–652

birden fazla pencere, 660–661

**MultipleWindowsDemo.java**, 661–662

isimlendirme kuralları, 468

genel bakış, 639–640

sorular ve alıştırmalar, 664–670

kayıtma çubukları, 654–655

**ScrollBarDemo.java**, 655–657**SliderDemo.java**, 658–660

kaydırıcılar, 657–658

özet, 664

metin alanları, 644–645

**TextAreaDemo.java**, 646–647

## Bileşim ilişkileri

**ArrayList** ve **MyStack** arasında, 436–437

nesneler ve, 382–383

## Hesapla ifadesi, 19

## Bilgisayarlar

iletişim cihazları, 8–9

CPU'lar, 3–4

giriş/çıkış cihazları, 7–8

hafıza, 4–5

İşletim sistemleri (OS), 12–13

genel bakış, 2–3

programlama dilleri, 9–12

depolama cihazları, 5–7

**concat** metodu, 340

Dizeleri birleştir, 36, 68, 340

Koşullu VE operatörü, 104

Koşullu ifadeler, 111–112

## Onay pencereleri

döngüleri kontrol etme, 164–165

seçimler yapma, 117–119

Dört bağla oyunu, 288

Tutarlılık, sınıf tasarım yönergeleri, 391

## Konsollar

tanımlandı, 16

formatlama çıktısı, 112–115

girdi, 16

çıktı, 16

girdi okuma, 37–40

## Sabitler

arayüzlerde erişim, 572

sınıf, 312–313

bildirim, 313

içinde **FigurePanel.java**, 486

tanımlayıcılar, 40

anahtar sabitler, 622

adlandırılmış sabitler, 43

adlandırma kuralları, 44

sarmalayıcı sınıflar ve, 394



Yapıcı zincirleme, 415–417

Yapıcılar

soyut sınıflarda, 562

için **BMI** sınıfı, 381

alt sınıf yapıcılarını çağırma, 414–415

için **Character** sınıfı, 350

nesneler oluşturma, 303

rastgele nesneler oluşturma, 310

**DataInputStream** ve **DataOutputStream** sınıfları için, 717

**Date** sınıfı için, 309

**GuessDate** sınıfı için, 390–391

**ImageViewer.java** örneğinde, 507

arayüzler ile soyut sınıflar, 581

bu referansı ile çağırma, 374–375

**Loan** sınıfı için, 377–379

nesne yöntemleri ve, 296–297

özel, 319

**SimpleCircle** örneğinde, 299–300

**String** sınıfı için, 336

**StringBuilder** sınıfı için, 353

**TV.java** örneğinde, 300

UML diyagramı, 298

wrapper sınıfları ve, 393

**Container/Container** sınıfıC

omponent, **Container** ve **JComponent** sınıfların

daki ortak özellikler, 462

**JPanel** alt sınıfı olarak, 459–460

özet, 446, 447–448

İçerik paneli, **JFrame** sınıfında, 450–451

İçerik-paneli delegasyonu, 451

**devam** ifadeleri, döngüleri kontrol etmek için, 159–162

Sözleşme, nesne sınıfı olarak, 296

Kontrol birimleri, **CPU**'lar, 3

Kontrol değişkenleri, **for** döngülerinde, 147–148

Sarmalayıcı sınıflar için dönüştürme yöntemleri, 394

Dizeleri dönüştürme

dizilerden/ dizilere, 343–344

sayısal değerlere/ sayısal değerlerden, 344

özet, 341

Koordinatlar

çokgen çizimi, 490–491

Java koordinat sistemi, 481

**MessagePanel** sınıfında, 499

Kopyalama

diziler, 236–237

dosyalar, 722–723

**CPU**'nun çekirdeği, 4

**cos** yöntemi, trigonometrik, 197–198

Kosinüs fonksiyonu, 511

Sayıci kontrollü döngüler, 135

Kupon toplayıcısı problemi, 260

**Kurs** sınıf, 384–386

**CPU**'lar (merkezi işlem birimleri), 3–4

Kıvrım parantezleri ({} )

blok söz diziminde, 17–18

atılmanın tehlikeleri, 85

kullanmayı unutmamak, 93

**currentTimeMillis** metodu, 51–52

İmleç, fare, 8

İmleç sınıf, 463

## D

.dat dosyaları (ikili), 712

Veri, referans almak için diziler, 224

Veri alanları

nesne verilerine erişim, 304–305

kapsülleme, 319–320, 391–392

arayüzlerde, 572

nesne durumu tarafından temsil edilen, 296–297

referans alma, 305, 373–374

içinde **SimpleCircle** örneği, 299–300

içinde **TV.java** örneği, 301

UML diyagramı, 298

Veri akışları. bak **DataInputStream/DataOutputStream** sınıfları

Veri türleri

ADT (soyut veri türü), 375

**boolean**, 8284, 309310

karakterler. bak Karakterler/ karakter veri türü (**char**)

**double**. bak **double** (çift hassasiyet) veri türü

**float**. bak Kesirli sayılar/float veri türü

temel. bak Temel türler

tam sayılar. bak Tam sayılar (**int** veri türü)

**long** veri türü. bak **long** veri türü

sayısal, 44–46, 56–58

referans türleri. bak Referans türleri

belirleme, 35

stringler, 68–69

tipleri, 41

soyut sınıf kullanarak, 564

**DataInputStream/DataOutputStream** sınıfları

**DetectEndOfFile.java**, 719

genel bakış, 716–718

**TestDataStream.java**, 718–719

Tarih sınıfı

vaka çalışması: **Takvim** ve **GregorianTakvim** sınıfları,

567–568

**java.util**, 308–309

De Morgan yasası, 104

Hata ayıklama

adım adım iyileştirmenin faydaları, 210

kod modülerleştirme ve, 189

seçimler, 119–120

Ondalık sayılar

**BigDecimal** sınıfı, 397–398

onaltılıya dönüştürme, 191–193, 348–350, 763

ikiliye dönüştürme/ikili den dönüştürme, 364, 763

bölme, 51

Sabitleri tanımlama, 43, 313

İstisnaları tanımlama

**CircleWithException.java** örneği, 531

**ReadData.java** örneği, 546

**TestCircleWithCustomException.java** örneği, 539

**throws** anahtar kelimesi için, 526

Statik yöntemlerin bildirilmesi, 313

Değişkenlerin bildirilmesi

dizi değişkenleri, 225

genel bakış, 41

veri türlerinin belirtilmesi ve, 35–36

iki boyutlu dizi değişkenleri, 264–265

Azaltma (`--`) operatörü, 54–56

Derin kopyalar, 579–580

Varsayılan alan değerleri, veri alanları için, 305–306

Silme tuşu, klavyelerde, 8

Sınırlayıcılar, token okuma yöntemleri ve, 546–547

Payda. *bkz* Gcd (en büyük ortak bölen)

Paydalar, rasyonel sayılarda, 584

Dağıtım, yazılım geliştirme sürecinde, 59

İniş, metin yazı tiplerinde, 493

**AçıklamaPaneli** sınıfı, 645–647

Açıklayıcı isimler

yararları, 40

değişkenler için, 35

Nesnelerin serileştirilmesi, 727

Tasarım kılavuzları, sınıflar, 391–393

**destroy** metodu, applet yaşam döngüsü, 677–678

Diyalog kutuları

onay diyalogu, 117–119, 164–165

**Dialog** sınıfı, 447

dosya diyalogu gösterimi, 549–550

girdi diyalogu, 70, 72

**JDialog** sınıfı, 447–448

mesaj diyalogu, 22–23

**Dialog** sınıfı, 447

Dial-up modemler, 8

Dijital abone hatları (DSL), 8

Dijital çok amaçlı disk (DVD), 6

Rakamlar, eşleştirme, 107

**Boyut** sınıfı, 448

Doğrudan özyineleme, 741

Dizinler

örnek çalışma: dizin boyutunu belirleme, 749

**DirectorySize.java**, 749–750

**File** sınıfı ve, 542

dosya yolları, 541

Diskler, depolama cihazı olarak, 6

Mesajı görüntüle

dialog kutusunda, 22–24

içinde **Welcome.java**, 17

içinde **WelcomeWithThreeMessages.java**, 18

Böl ve yönet stratejisi. *bkz* Aşamalı iyileştirme

Bölme (`/=`) atama operatörü, 53–54

Bölme operatörü (`/`), 46, 50

Dokümantasyon, programlama ve, 24

Nokta operatörü (`.`), 23, 304

Nokta aralığı, ekranların keskinliğini ölçme, 8

**double** (çift hassasiyet) veri türü

karakterleri ve sayısal değerleri dizelere dönüştürme, 344

dizeleri sayılara dönüştürme, 70

değişkenleri tanımlama ve, 41

**java.util.Random**, 309310

sayısal türlerin genel görünümü, 45

hassasiyeti, 154–155

**do-while** döngüleri

ne zaman kullanılacağına karar verme, 150–151

özet, 144–146

Objeleri aşağıya dönüştürme, 425

**drawArc** metodu, 488–490

**drawImage** metodu, 504–505

**drawLine** metodu, 483

**drawOval** metodu, 484, 488

**drawPolygon** metodu, 491

**drawPolyline** metodu, 491

**drawRect** metodu, 483–484

**drawString** metodu, 483

Sürücüler, 6

Açılır listeler. *bkz* Kombinasyon kutuları

DSL'ler (dijital abone hatları), 8

DVD'ler (Dijital çok amaçlı disk), 6

Dinamik bağlama, kalıtım ve, 422–425

## E

Tutulma

yerleşik hata ayıklama, 119

Java kaynak kodu oluşturma/düzenleme, 19

Programları Sonlandır düğmesi ile durdurma, 450

Sekiz Kraliçe bulmacası, 259, 262, 765–766

Eleman türü, diziler için belirtme, 225

Emirp, 218

Kapsülleme

in **CircleWithPrivateDataFields.java** örneği,

320–321

sınıf tasarımı yönergeleri, 391

sınıflar, 375–376

veri alanları, 319–320

in **GuessDate** sınıfında, 391

bilgi gizleme ile, 203

**Rational** sınıfının, 589

stringler ve, 339

Kodlama şemaları

tanımlandı, 4

karakterleri ikili eşdeğerlerine eşleme, 61

Dosya sonu istisnası (**EOFException**), 719

**endIndex** metodu, stringlerden alt stringler elde etmek için, 341

Satır sonu stili, blok stilleri, 25–26

Eşit (`=`) operatörü, atama için, 82

Eşit (`==`) operatörü, karşılaştırma için, 82

**equalArea** metodu, geometrik nesnelerin alanlarını karşılaştırmak için, 563

**equals** metodu

Diziler sınıf, 253

**Karakter** sınıf, 350–351

string karşılaştırma, 337

**Nesne** sınıf, 429–430

**equalsIgnoreCase** metodu, string karşılaştırma, 338–339

Hata sınıf, 524, 526

Hatalar, programlama. *bkz* Programlama hataları

Kaçış karakterleri (`\`), 63–64



Olay delegasyonu, 603

Olay işleyicileri/olay işleme, 601–602

anonim sınıf dinleyicileri, 611

Çocuk okulu, 613614, okulların listesini, 653

genel bakış, 604

Olay dinleyici nesnesi, 603

Olay dinleyicileri, 603–604. *bkz ayrıca* Dinleyici sınıfları

Olay kaynak nesnesi, 602–603

Olay odaklı programlama

dinleyici sınıflarını tanımlamak için alternatifler, 612

**Timer** sınıfı kullanarak animasyon, 625–626

**AnimationDemo.java**, 626–628

anonim sınıf dinleyicileri, 609–610

**AnonymousListenerDemo.java**, 610–612

vaka çalışması: kredi hesaplayıcı, 615

**ClockAnimation.java**, 628–629

**ControlCircle.java**, 607–608

**ControlCircleWithMouseAndKey.java**, 623–625

**ControlCircleWithoutEventHandling.java**, 606–607

tanımlandı, 602

**DetectSourceDemo.java**, 612–613

event handler'lar, 604

olay dinleyicileri, 603–604

eventler ve olay kaynakları, 602–603

**FrameAsListenerDemo.java**, 613614nH

**andleEvent.java**, 601602

iç sınıflar, 608–609

anahtar olayları, 621–622

anahtar terimler, 629

**KeyEventDemo.java**, 622–623

dinleyici arayüz adaptörleri, 620–621

**LoanCalculator.java**, 615–617

fare olayları, 617–618

**MoveMessageDemo.java**, 618–620

özet, 600–601

sorular ve alıştırmalar, 630–638

dinleyicileri kaydet, 604–605

özet, 629–630

Olay-dinleyici arayüzü, 603

**EventObject** sınıfı, **java.util**, 602–603, 605

Olaylar

**ControlCircleWithMouseAndKey.java**, 623–625

event kaynakları ve, 602–603

GUI bileşeni, 640

**GUIEventDemo.java**, 640–643

**JList** sınıfı, 651

tuş olayları, 621–622

**KeyEventDemo.java**, 622–623

fare olayları, 617–618

**MoveMessageDemo.java**, 618–620

**Timer** sınıfı tetikleniyor **ActionEvents**, 625–626

**Exception** sınıfı

istisnalar, 524

uzatma, 538

java.lang içinde, 539

alt sınıfları, 524–525

İstisna yönetimi. *ayrıca bakınız* Programlama hataları

istisnaları yakalama, 527–529, 531

zincirlenmiş istisnalar, 537–538

**CircleWithException.java** örnek, 531–532

istisnaları tanımlama (**throws**), 526, 531

özel istisna sınıflarını tanımlama, 538–541

**finally** ifadesi içinde, 534–535

istisnalar hakkında bilgi alma, 529–530

**InputMismatchExceptionDemo.java** örnek, 522–523

anahtar terimler, 552

genel bakış, 39, 517–518

sorular ve alıştırmalar, 554–558

**Quotient.java** örnek, 518

**QuotientWithException.java** örnek, 520–522

**QuotientWithIf.java** örnek, 519

**QuotientWithMethod.java** örnek, 519–520

istisnaları yeniden fırlatma, 536–537

özet, 553–554

**TestCircleWithException.java** örnek, 532–534

**TestException.java** örnek, 530

istisna fırlatma, 526–527, 531

istisna türleri, 523–525

istisnaları ne zaman kullanmalı, 535–536

İstisna yayılması, 527

İstisnalar. *ayrıca bakınız* Programlama hataları

yakalama (**try-catch**), 527–529

kontrol edilen ve kontrol edilmeyen, 525

**ClassCastException**, 426

değerlendirme, 526

tanımlı, 518

**EOFException**, 719

İçinde **İstisna** sınıfı, 524

**FileNotFoundException**, 713

işleme. *bakınız* İstisna işleme

İçinde **House.java** örnek, 579

**IOException**, 713–714

**NotSerializableException**, 727

fırlatma, 526–527

Özel veya (**^**) mantıksal operatör, 102–105

İşlem yığınları. *bakınız* Çağrı yığınları

**var** metodu, dosya örneklerini kontrol etmek için, 542–543

Açık tür dönüşümü, 56–57, 425

Üs metodu, **Math** sınıfı, 198–199

Üs işlemleri, 48

İfadeler

atama ifadeleri ve, 42

sahne arkasında değerlendirme, 116

Boolean. *bak* Boolean ifadeleri

değerlendirme, 50–51

**uzatır** anahtar kelime, arayüz mirası ve, 582

## F

Faktöriyel

vaka çalışması: faktöriyel hesaplama, 738–739

**ComputeFactorial.java**, 739–741

**ComputeFactorialTailRecursion.java**, 759

tail recursion ve, 758–759

Fahrenheit, Celsius'a dönüştürme, 50–51, 213

Fall-through davranışı, **switch** ifadeleri, 109

Feet, metreye dönüştürme, 213–214

**fib** metodu, 742–744

Fibonacci, Leonardo, 742

Fibonacci sayıları

vaka çalışması: hesaplama, 741–742

**ComputeFibonacci.java**, 742–744

özyinelemeli hesaplama, 761

**FigurePanel** sınıfı

**FigurePanel.java**, 486–488

özeti, 485

**TestFigurePanel.java**, 485–486

**File** sınıfı, 541–543, 710

Dosya G/Ç. *bkz* G/Ç (girdi/çıkı)

Dosya işaretçileri, rastgele erişim dosyaları ve, 730

**FileInputStream/FileOutputStream** sınıfları

özeti, 713–714

**TestFileStream.java**, 714–716

Dosyalar

vaka çalışması: dosyaları kopyalama, 722–723

vaka çalışması: metni değiştirme, 548–549

dosya diyaloglarını görüntüleme, 549–550

**File** sınıfı, 541–543, 710

girdi/çıkı, 544

**JFileChooser** sınıfı, 550

anahtar terimler, 552

sorular ve alıştırmalar, 554–558

veri okuma, 545–547

Web'den veri okuma, 551–552

özet, 553–554

**TestFileClass.java**, 543–544

veri yazma, 544–545

**FilterInputStream/FilterOutputStream** sınıfları, 716

**final** anahtar kelimesi, sabitleri tanımlamak için, 43

**final** değiştiricisi, sınıfların genişletilmesini önlemek için, 439–440

**finally** ifadesi, istisna işleme, 534–535

Kayan nokta literalleri, 49

Kayan nokta sayıları/**float** veri türü

yaklaşımı, 47

karakterleri ve sayısal değerleri dizelere dönüştürme, 344

tam sayılara dönüştürme, 56

**java.util.Random**, 309–310

döngülerle ilgili sayısal hataları en aza indirme, 154–155

sayısal türler için, 45

sayısal türlerin genel görünümü, 45

data türlerini belirtme, 35

kesinliği belirtme, 114

Akış Şemaları

**do-while** döngüleri, 145

**if** ifadeleri, 84–85

**if-else** ifadeleri, 90

**for** döngüleri, 147

**switch** ifadeleri, 108

**while** döngüleri, 135

**FlowLayout** sınıfı

genel görünümü, 452

özellikleri, 457

**ShowFlowLayout.java** örneği, 452–454

**Font** sınıfı

yardımcı sınıflar, 446, 448

Java GUI API'sinde, 461–462

**FontMetrics** sınıfı

bir dizeyi merkezleme, 493–494

yardımcı sınıflar, 446, 448

**TestCenterMessage.java** örneği, 494–495

Yazı Tipleri

**Bileşen** sınıfı, 499

oluşturma, 463

**GUIEventDemo.java**, 642

mesaj paneli için ayarlama, 495

**for** döngüleri

ne zaman kullanılacağına karar verme, 151

iç içe geçirme, 152, 267

genel bakış, 146–149

işlem dizileri ile, 227

değişken kapsamı ve, 196

**for-each** (geliştirilmiş) döngüler, 229–231

Ön plan rengi, 463–464

Resmi parametreler. *bkz* Parametreler

**format** metodu, dizeler, 344–347

Format belirteçleri, 113–115

FORTTRAN, yüksek seviyeli diller, 11

Fraktallar

vaka çalışması, 754–755

H-ağaç fraktalları, 766

Koch kar tanesi fraktalı, 764

**SierpinskiTriangle.java**, 755–757

Çerçeveler (pencereler)

ek bileşenler ekleme, 450–451

oluşturma, 310–312, 449, 457, 641, 676

**DisplayLabel.java**, 677

**Frame** sınıfı, 447

**JFrame** sınıfı. *bkz* **JFrame** sınıfı

**ListDemo.java**, 653

**MultipleWindowsDemo.java**, 661–662

**MyFrame.java** örneği, 449–450

**Scrolldemo.javat**, 656 **nsf**

**iderdemo.javat**, 659

Serbest hücreler, Sudoku izgarasında, 274

Fonksiyon tuşları, klavyelerde, 7

Fonksiyonlar, 179. *bkz* Yöntemler

Temel türler (Primitif türler). *bkz* Primitif türler

## G

Galton kutusu, 258–259

Çöp toplama, JVM ve, 236

GB'lar (gigabayt), depolama, 5

Gcd (en büyük ortak bölen)

örnek çalışma: en büyük ortak böleni bulma, 155–157

özyinelemeli olarak hesaplama, 760

**gcd yöntemi**, 189–190

**gcd** yöntemi, 588

**Rasyonel** sınıf ve, 585

Gen dizileri, 367

Genom, 367

**GeometricObject** sınıfı

**Circle.java** ve **Rectangle.java**, 562

özet, 560

**TestGeometricObject.java**, 562–563

**getAbsolutePath** metodu, **File** sınıfı, 542–543

**getArea** metodu, **SimpleCircle** örneği, 299

**getArray** metodu, 269–270

**getBMI** metodu, **BMI** sınıfı, 381

**getChars** metodu, dizeleri dizilere dönüştürme, 343

**getDateCreated** metodu, **Date** sınıfı, 370

**getFontMetrics** metodu, **Graphics** sınıfı, 493–494

**getHeight** metodu, **FontMetrics** sınıfı, 495

**getIndex** metodu, **ArrayList** sınıfı, 433

**getPerimeter** metodu, **SimpleCircle** örneği, 299

**getRadius** metodu, **CircleWithPrivateDataFields.java** örneği, 321

**rastgeleKüçükHarfAl** metodu, 241, 243

**boyutAl** metodu, dizin boyutunu bulma, 750

**kaynakAl** metodu, olaylar, 602

**yığıniziniAl** metodu,

istisnalar hakkında bilgi almak için, 529

**durumAl** metodu, **BMI** sınıfı, 381

Alıcı (getter) metodları

**DiziListesi** sınıfı ve, 434

data alanlarının kapsüllenmesi ve, 320–322

**günAl** metodu, **Tarih** sınıfı, 309

**genişlikAl** metodu, **FontMetrics** sınıfı, 495

GIF (Grafik Değişim Biçimi), 465

Gigabayt (GB), depolama, 5

Gigahertz (GHz), saat hızı, 4

GMT (Greenwich Ortalama Zamanı), 51

Gosling, James, 13

Grafikler

vaka çalışması: **ŞekilPaneli** sınıfı, 485–488

vaka çalışması: **GörüntüGörücü** sınıfı, 506–507

vaka çalışması: **MesajPaneli** sınıfı, 495–497

vaka çalışması: **DurakSaat** sınıfı, 500

bir dizeyi **FontMetrics** sınıfı kullanarak ortalama, 493–495

**GörüntüSaati.java** örneği, 500–502

**GörüntüGöster.java** örneği, 505

resimleri görüntüleme, 504–505

yaylar çizme, 488–490

çokgenler ve çoklu çizgiler çizme, 490–493

stringler, çizgiler, dikdörtgenler ve ovalar çizme, 483–484

**GörüntüGörücü.java** örneği, 507–508

**Mesaj.Panel.java** örneği, 497–500

özet, 479–480

sorular ve alıştırmalar, 509–516

**SixFlags.java** örneği, 506–507

**StillClock.java** örneği, 502–504

özet, 508–509

**TestPaintComponent.java** örnek, 481–483

**Graphics** sınıfı

**GetFontMetrics** metodu, 493

yardımcı sınıflar, 446, 448

özet, 480–481

Graphics Interchange Format (GIF), 465

Büyük (>) karşılaştırma operatörü, 82

Büyük veya eşit (>=) karşılaştırma operatörü, 82

En büyük ortak bölen. *bkz* Gcd (en büyük ortak bölen)

Greenwich Ortalama Zamanı (GMT), 51

**GregorianCalendar** sınıfı

**Cloneable** arayüzü ve, 577–578

java.util paketinde, 331

özet, 567–568

**TestCalendar.java**, 568–569

**GridLayout** sınıfı

özet, 454–455

özellikleri, 457

**ShowGridLayout.java** örneği, 455–456

Gridler

**GridLayout** yöneticisi, 454–456

iki boyutlu dizi kullanarak temsil etme, 274

**GuessDate** sınıfı, 388–391

GUI (grafik kullanıcı arayüzü)

alignment, 470

**BorderLayout** sınıfı, 456–457

GUI bileşenleri için sınıflar, 310–312

**Color** sınıfı, 460–462

Swing GUI bileşenlerinin ortak özellikleri, 462–465

**Bileşen** sınıfı, 447

bileşenler. *bkz* Bileşenler

**Kapsayıcı** sınıfı, 447–448

GUI uygulamalarını applet'lere dönüştürme, 672

**FlowLayout** sınıfı, 452–454

**Font** sınıfı, 461–462

çerçeveler, 449–451

**GridLayout** sınıfı, 454–456

sınıflardan oluşturulan GUI nesneleri, 296

yardımcı sınıflar, 448

simge, basılı simgeler ve üzerine gelindiğinde değişen simgeler, 469

resim simgeleri, 465–467

Java GUI API'si, 446–447

**JButton** sınıfı, 467–469

**JCheckBox** sınıfı, 471–472

**JLabel** sınıfı, 473–474

**JRadioButton** sınıfı, 472–473

**JTextField** sınıfı, 474–475

anahtar terimler, 475

yerleşim yöneticileri, 451

gözden geçirme, 445–446

alt konteyner olarak kullanılan paneller, 458–460

yerleşim yöneticilerinin özellikleri, 457–458

sorular ve alıştırmalar, 476–478

özet, 475–476

Swing vs. AWT, 446

metin konumları, 470–471

## H

Hata ayıklama için el izleri, 119–120

Adam asmaca oyunu, 367, 512, 556, 636

Sabit diskler, depolama cihazı olarak, 6

Donanım, 2

Has-a ilişkileri

birleştirme modellerinde, 382–383

bileşim ve, 437

Yığın, dinamik bellek tahsisi ve, 239

Yükseklik, metin yazı tiplerinde, 493

Yardımcı sınıflar, 448

Yardımcı yöntemler, özyinelemeli

gözen geçirme, 746

**RecursivePalindrome.java**, 746–747

Hertz (Hz), saat hızı, 3

Onaltılık tam sayılar, 49

Onaltılık sayılar

ikiliye dönüştürme, 364, 763

ondalığa dönüştürme, 191–193, 348–350, 763

Altıgenler, çizim, 491–492

Gizli veri alanları, referans verme, 373–374

Yüksek seviyeli diller, 10–12

Hilbert eğrisi, 766

Yatay kaydırma çubukları, 655

Yatay kaydırıcılar, 657, 659

Yatay metin konumlandırma, 470

HTML (Hiper Metin İşaretleme Dili)

**<applet>** etiketi, 673

applet parametrelerini tanımlama, 679

**DisplayLabel.html**, 673–674

**DisplayMessage.html**, 679–680

belge düzeni için betik dili, 14

Web tarayıcısından applet'leri görüntüleme, 674

H-ağaçları

fraktallar, 766

özyinelemeli yaklaşım, 738

Hz (Hertz), saat hızı, 3

## I

Simge. bkz Görüntü simgeleri

Tanımlayıcılar, 40

IDE'ler (entegre geliştirme ortamları)

Java kaynak kodu oluşturma/düzenleme için, 19–20

genel bakış, 16

IEEE (Elektrik ve Elektronik Mühendisleri Enstitüsü), kayan

nokta standardı (IEEE 754), 45

**eğer** ifadeleri

yaygın hatalar, 93–95

vücut kütle indeksini hesaplamada, 97–99

vergileri hesaplamada, 99–101

koşullu operatör ile kullanılır, 112

iç içe geçme, 91

genel bakış, 84–85

**SimpleIfDemo.java** örneği, 85–86

**if-else** ifadeleri

koşullu ifadeler ve, 112

belirsiz else durumu, 94–95

çok yönlü, 91–93

genel bakış, 89–91

özyineleme ve, 744

**IllegalArgumentException** sınıfı, 527

**Image** sınıfı, 504

Resim simgeleri

**ComboBoxDemo.java**, 649

oluşturma ve yerleştirme, 691–692

varsayılan simgeler, basılı simgeler ve üzerine gelindiğinde simgeler, 469

resimleri görüntüleme, 504

ImageIcon sınıfı hakkında genel bakış, 465–466

**TestImageIcon.java** örneği, 466–467

Resimler

vaka çalışması: **ImageViewer** sınıfı, 506–507

**DisplayImage.java** örneği, 505

görüntüleme, 504–505

**ImageViewer.java** örnek, 507–508

**SixFlags.java** örnek, 506–507

**ImageViewer** sınıfı

**ImageViewer.java** örnek, 507–508

genel bakış, 506–507

Değişmez

**BigInteger** ve **BigDecimal** sınıfları, 397–398

sınıf, 370

nesneler, 370–371

**Rasyonel** sınıf, 589

**String** nesnesi, 336–337

wrapper sınıfları, 394

Uygulama (kodlama), yazılım geliştirme sürecinde, 59–61

Uygulama yöntemleri, 207–210

İçsel tür dönüşümü, 64, 425

İçe aktarma

programa paket içe aktarma, 23–24

**JFrame** paketi, 449

**içe aktarma** ifadelerinin türleri, 24

Artırım (**++**) operatörü, 54–56

**increment** metodu, **Increment.java** örneğinde, 186–187

Artımlı geliştirme

adım adım iyileştirmenin faydaları, 210

kademeli kodlama, 137

test etme ve, 61

Girinti, programlama stili, 25

İndeksli değişkenler, 226–227

İndeksler

dizilerdeki elemanlara erişim, 224, 226

bir stringde karakterleri/alt stringleri bulma, 342–343

string indeks aralığı, 340

**indexOf** metodu, 342–343

Dolaylı özyineleme, 741

Sonsuz döngüler, 136

Sonsuz özyineleme, 741

Bilgi

istisnalar hakkında bilgi almak, 529–530

gizleme (kapsülleme), 203

Kalıtım

**ArrayList** nesnesi, 430–431

alt sınıf yapıcılarını çağırma, 414–415

üst sınıf yöntemlerini çağırma, 417

vaka çalışması: özel yığın sınıfı, 436–437

- nesneleri dönüştürme ve, 425–426
- CastingDemo.java** örneği, 426–429
- CircleFromGeometricObject.java** örneği, 410–412
- yapıcı zincirleme ve, 415–417
- DistinctNumbers.java** örneği, 434–436
- dinamik bağlama ve, 422–425
- equals** yöntemi **Object** sınıfı, 429–430
- arayüz kalıtımı, 570–571, 582
- is-a ilişkileri ve, 437
- anahtar terimler, 440
- Object** sınıfı ve, 420–421
- yöntemleri geçersiz kılma ve, 418–420
- özet, 407–408
- sınıfların genişletilmesini veya geçersiz kılınmasını önleme, 439–440
- korumalı** veriler ve yöntemler, 437–440
- sorular ve alıştırmalar, 442–444
- RectangleFromGeometricObject.java** örneği, 412–413
- SimpleGeometricObject.java** örnek, 409–410
- özet, 440–441
- üst sınıflar ve alt sınıflar ve, 408–409
- TestArrayList.java** örnek, 431–434
- TestCircleRectangle.java** örnek, 413–414
- super anahtar kelimesini kullanma, 414
- init** metodu, applet yaşam döngüsü yöntemleri, 677–678
- Değişkenleri başlatma
- diziler, 227–228
- değişkenleri tanımlama ve, 41
- LottoNumbers.java**, 232
- çok boyutlu diziler, 265
- iki boyutlu diziler, 267
- İç (iç içe) sınıflar
- AnimationDemo.java**, 627
- anonim, 609–610
- AnonymousListenerDemo.java**, 610–612
- dinleyici sınıflarını tanımlamak için, 608–609
- KeyEventDemo.java**, 622
- Movemesagedemo.javat**, 619
- ntitatioe.javat**, 689
- Girdi. *ayrıca bak* G/Ç (girdi/çıkıtı)
- konsoldan okuma, 37–40
- döngüler kullanarak yönlendirme, 143–144
- çalışma zamanı hataları, 27
- akışlar. *bak* **InputStream** sınıfları
- Girdi, işleme, çıkıtı (IPO), 39–40
- Girdi iletişim kutuları
- girdi almak için, 70–72
- ShowInputDialog** metodu, 70, 72
- InputMismatchException** sınıfı, 522–523, 547
- Girdi/çıkıtı cihazları, bilgisayarlar ve, 7–8
- InputStream** sınıfları
- BufferedInputStream**, 719–722
- vaka çalışması: dosyaları kopyalama, 723–724
- DataInputStream**, 716–718
- serileştirme ve, 727
- DetectEndOfFile.java**, 719
- FileInputStream**, 713–714
- FilterInputStream**, 716
- ObjectInputStream** sınıfı, 724–725
- genel bakış, 712–713
- TestDataStream.java**, 718–719
- TestFileStream.java**, 714–716
- TestObjectInputStream.java**, 726
- Klavye üzerindeki tuşu ekle, 8
- Yerleştirme sıralamaları, diziler, 250–252
- Örnek yöntemler
- nesne verilerine ve yöntemlerine erişim, 305
- CircleWithStaticMembers.java**, 314
- sınıf tasarım yönergeleri, 392–393
- çağırma, 377, 380
- örnek yöntemlerin ne zaman kullanılacağı vs. statik, 313–314
- Örnek değişkenler
- nesne verilerine ve yöntemlerine erişim, 305
- sınıf tasarım yönergeleri, 392–393
- statik değişkenlerle karşılaştırıldığında, 312–313
- TestCircleWithStaticMembers.java**, 314
- örnek değişkenleri ile statik arasında ne zaman kullanılacağı, 316
- Örnekler. *bkz.* Nesneler
- dosya örnekleme kontrol etme, 542
- nesne örnekleme kontrol etme, 296, 426
- Elektrik ve Elektronik Mühendisleri Enstitüsü (IEEE), kayan
- nokta standardı (IEEE 754), 45
- Tam sayı literalleri, 48–49
- Tam sayılar (**int** veri türü)
- ArrayList** için, 435
- BigInteger** sınıfı, 397–398
- char** türlerine tür dönüşümü, 63
- karakterleri ve sayısal değerleri dizelere dönüştürme, 344
- stringleri sayılara dönüştürme, 70
- değişkenleri tanımlama ve, 41
- bölme, 46, 51, 518–522
- iki arasında daha büyük olanı bulma, 179
- kayan nokta sayıları, 56'ya dönüştürüldü
- java.util.Random**, 309–310
- sayısal türler için, 44–45
- data türlerini belirtme,
- 35Java kaynak kodu oluşturma/düzenleme
- için Entegre Geliştirme Ortamları (IDE'ler), 19–20özet, 16
- Akıllı tahminler, 137
- Arayüz adaptörleri, dinleyici sınıfları, 620–621
- Arayüzler
- soyut sınıflarla karşılaştırıldığında, 581–584
- avantajları, 576
- vaka çalışması: **Rasyonel** sınıf, 584–585
- Klonlanabilir** arayüz, 577–578
- Karşılaştırılabilir** arayüz, 573–574
- ComparableRectangle.java** örneği, 575–576
- ortak sınıf davranışlarını tanımlamak için, 560
- House.java** örnek, 578–581
- anahtar terimler, 590
- genel bakış, 570
- sorular ve alıştırmalar, 590–598

Arayüzler (*devam*)**SortComparableObjects.java** örnek, 574–575**SortRectangles.java** örnek, 576–577

özet, 590

**TestEdible.java** örnek, 570–573

Interned strings, 336–337

Yorumlayıcılar, kaynak programı makine koduna çevirme, 10–11

Metotları çağırma, 180–182, 305

## G/Ç (girdi/çıkı)

binary G/Ç sınıfları, 712–713

**BufferedInputStream** ve **BufferedOutputStream** sınıfları, 719–722

vaka çalışması: dosyaları kopyalama, 722–723

vaka çalışması: metni değiştirme, 548–549

**Copy.java**, 723724**DataInputStream** ve **DataOutputStream** sınıfları, 716–718**DetectEndOfFile.java**, 719

dosya diyaloglarını görüntüleme, 549–550

**FileInputStream** ve **FileOutputStream** sınıfları, 713–714**FilterInputStream** ve **FilterOutputStream** sınıfları, 716

Java'da metin G/Ç işlemleri, 710–711

anahtar terimler, 732

nesne G/Ç, 724–725

genel bakış, 544, 709–710

sorular ve alıştırmalar, 733–736

rastgele erişim dosyaları, 729–731

Scanner sınıfını kullanarak dosyadan veri okuma, 545–547

Web'den veri okuma, 551–552

**serileştirilebilir** arayüzü, 727–728

dizileri serileştirme, 728–729

özet, 733

**TestDataStream.java**, 718–719**TestFileStream.java**, 714–716**TestObjectInputStream.java**, 726**TestObjectOutputStream.java**, 725–726**TestRandomAccessFile.java**, 731–732

metin G/Ç vs. ikili G/Ç, 711–712

G/Ç aygıtlarının türleri, 7–8

PrintWriter sınıfını kullanarak dosyaya veri yazma, 544–545

**IOException**, 713–714

IPO (girdi, işleme, çıkı), 39–40

## Is-a ilişkileri

Arayüzlerin ve sınıfların ne zaman kullanılacağına dair tasarım kılavuzu, 582

kalıtım ve, 437

**isAbsolute** metodu, **File** sınıfı, 542–543**isDigit** metodu, **Character** sınıfı, 351, 356–357**isDirectory** metodu, **File** sınıfı, 542–543**isFile** metodu, **File** sınıfı, 542–543**isHidden** metodu, **File** sınıfı, 542–543

Is-kind-of ilişkileri, 582

**isLetter** metodu, **Character** sınıfı, 351, 356–357**isLowerCase** metodu, **Character** sınıfı, 351**isPalindrome** metodu**RecursivePalindrome.java**, 746–747

kuyruk-özyinelemeli metod olarak, 758

**isPrime** metodu, asal sayılar, 191**isUpperCase** metodu, **Character** sınıfı, 351**isValid** metodu, ızgaraya uygulama, 276**ItemEvents**

GUI bileşenleri tetikleme, 640

**JComboBox** sınıfı, 647, 650

## Döngüler

döngüler ve, 134

özyineleme ile karşılaştırıldığında, 757–758

**J****JApplet** sınıfı

kapsayıcı sınıflar, 446, 448

applet geliştirme, 672

**JFrame** sınıfı ile karşılaştırıldığında, 676–677

top-level kapsayıcılar, 447

**java** komutu, Java programını çalıştırmak için, 21

Java Geliştirme Kiti (JDK)

jdb hata ayıklayıcı, 119

özet, 16

Java EE (Java Kurumsal Sürümü), 16

Java GUI API, 446–447

Java dil spesifikasyonu, 16

Java Kütüphanesi, 308

Java ME (Java Mikro Sürümü), 16

Java programlama

programları oluşturma, derleme ve çalıştırma, 19–22

mesaj iletişim kutusunda metin görüntüleme, 22–24

yüksek seviyeli diller, 11

giriş, 13–15

basit örnekler, 16–19

Java SE (Java Standart Sürümü), 16

Java Sanal Makinesi. *bkz* JVM (Java Sanal Makinesi)**java.awt** sınıfları. *bkz* AWT (Soyut Windows Araç Takımı)**javac** komutu, Java programını derlemek için, 21Javadoc yorumları (*/\*\*.\*/*), 25**java.io**

Dosya sınıfı, 541–543

**PrintWriter** sınıfı, 544–545**RandomAccessFile** sınıfı, 730**java.lang****Sınıf** sınıfı, 692**Karşılaştırılabilir** arayüz, 733**İstisna** sınıfı, 539**Sayı** sınıfı, 565

paketler, 61

**Atılabilir** sınıfı, 523–525, 529–530**java.net****YanlışFormatlıURLİstisnası** sınıfı, 551**URL** sınıfı, 551, 691–692**java.util****Diziler** sınıfı, 252–253**Takvim** sınıfı, 567–568**Tarih** sınıfı, 308–309, 567**OlayNesnesi** sınıfı, 602–603, 605**GregoryenTakvimi** sınıfı, 331, 567–568**Rastgele** sınıfı, 309–310**Tarayıcı** sınıfı, 38, 545–547



**javax.swing.** *bkz*Swing

### JButton sınıfı

- hizalama, 470
- butonlar oluşturma, 310–312, 451
- basılabilir buton oluşturma, 467–469
- varsayılan simgeler, basılı simgeler ve üzerine gelindiğinde simgeler, 469
- buton olarak görüntülenen resim simgeleri, 466
- Container sınıfından miras alarak, 460
- metin konumları, 470–471

### JCheckBox sınıfı

- onay kutuları oluşturma, 311
- eylemler, 640–643
- özeti, 471–472
- buton türleri, 468

### JComboBox sınıfı

- ComboBoxDemo.java**, 648–649
- kombinasyon kutuları oluşturma, 310–312
- GUI bileşenleri, 647–648

### JComponent sınıfı

- soyut sınıf olarak, 447
- Component, Container, v **JComponent** sınıflarındaki ortak özellikler, 462–463
- özeti, 446–447
- paintComponent** metodu, 480
- alt sınıflama, 483

jdb hata ayıklayıcı, 119

### JDialog sınıfı, 447, 448

### JDK (Java Geliştirme Kiti)

- jdb hata ayıklayıcı içinde, 119
- özeti, 16

### JFileChooser sınıfı, 550

### JFrame sınıfı

- çerçevelere bileşen ekleme, 450–451
- kapsayıcı sınıf olarak, 446, 447–448
- çerçeveler oluşturma, 310–312, 449, 641, 676
- JApplet** sınıfı ile karşılaştırıldığında, 676
- ShowFlowLayout** sınıfı genişletme, 452–454

### JLabel sınıfı

- çerçevelere etiket ekleme, 452
- grid'lere etiket ekleme, 455
- resim simgeleri oluşturma ve yerleştirme, 691
- etiket oluşturma, 311
- etiketler olarak görüntülenen resim simgeleri, 466
- özet, 473–474

### JList sınıfı

- listeler oluşturma, 310–312
- ListDemo.java**, 652–654
- listelerin özeti, 650–652

Ortak Fotoğraf Uzmanları Grubu (JPEG), 465

### JOptionPane sınıfı

- önceden tanımlanmış Java sınıfı olarak, 22
- showConfirmDialog** metodu, 117–119
- showInputDialog** metodu, 70, 72

### JPanel sınıfı

- Top** alt sınıf, 683–684
- Hücre** alt sınıf, 687–688
- kapsayıcı sınıf olarak, 446, 448

**AçıklamaPaneli** genişletme, 645–646

**ŞekilPaneli** sınıf, 485

**GörüntüGörücü** sınıf, 506–507

**boyaBileşeni** metot, 480

alt kapsayıcı olarak kullanılan paneller, 458–459

**DuranSaat** sınıf, 500

alt sınıflama, 482

**TestPanelleri.java** örnek, 459–460

JPEG (Ortak Fotoğraf Uzmanları Grubu), 465

### JRadyobutton sınıfı

- radio düğmeleri oluşturma, 310–312
- eşitlikler, 640
- genel bakış, 472–473
- düğme türleri, 468

### JSerialÇubuk sınıfı

- TopKontrol.java**, 685
- zıplayan top durumunda zıplama hızını kontrol etme stüdyo, 683
- genel bakış, 654–655
- ScrollBarDemo.java**, 655–657

### JScrollPane sınıfı

- AçıklamaPaneli.java**, 646
- genel bakış, 644
- kaydırma listeleri, 644

### JSlider sınıfı

- genel bakış, 657–658
- SliderDemo.java**, 658–660

### JTextArea sınıfı

- genel bakış, 644–645
- TextAreaDemo.java**, 646–647

### JTextComponent sınıfı, 474–475

### JTextField sınıfı

- çerçevelere metin alanları ekleme, 452
- ızgaralara metin alanları ekleme, 455
- metin alanları oluşturma, 311
- eventler, 640
- genel bakış, 474–475

### JVM (Java Sanal Makinesi)

- tanımlandı, 21
- çalışma zamanı hatalarını tespit etme, 518
- çöp toplama, 236
- heap'in depolama alanı olarak, 239
- interned string ve, 337

## K

KB (kilobayt), 5

Anahtar sabitler, 622

Klavye, 7

Anahtar Olayları

**ControlCircleWithMouseAndKey.javat**, 625nKey

**EventDemo.javat**, 622623

genel bakış, 621–622

Anahtar Dinleyici arayüzü, 621

Anahtar Kelimeler (rezervli kelimeler)

sözdizimi, 426

içinde**Welcome.java**, 17

Kilobayt (KB), 5  
Şövalye Turu, 765  
Koch kar tanesi fraktalı, 764

## L

### Etiketler

çerçevelere ekleme, 452  
ızgahlara ekleme, 455  
oluşturma, 311  
[AçıklamaPanel.java](#), 645–646  
resim simgeleri olarak görüntülenen, 466  
[JLabel](#) sınıfı, [bkz.JLabel](#) sınıfı  
yerleştirme, 641

LAN'lar (yerel alan ağları), 8–9, 645–646  
[lastIndexOf](#) metodu, dizgeler, 342–343  
[lastModified](#) metodu, [File](#) sınıfı, 542–543  
Latin kare, 293–294

### Yerleşim yöneticileri

[BorderLayout](#), 456457n  
[FlowLayout](#), 452454  
[Izgara Düzeni](#), 454456  
[LayoutManager](#) sınıfı, 448  
genel bakış, 451  
özellikleri, 457–458

Önde, metin yazı tiplerinde, 493

Uzunluk, dizgeler, 339–340, 355–356

[length](#) metodu, [File](#) sınıfı, 542–543

Küçüktür (<) karşılaştırma operatörü, 82  
Küçük veya eşit (<=) karşılaştırma operatörü, 82  
Harfler, sayma, 352–353

Kütüphaneler, API'ler olarak, 16

Yaşam döngüsü yöntemleri, applet'ler, 677–678

Satır yorumları, içinde [Welcome.java](#), 17

Satır numaraları, içinde [Welcome.java](#), 17

Doğrusal aramalar, diziler, 245–246

Çizim çizgileri, 483–484, 487

Linux OS, 12

### Dinleyici sınıfları

[ActionListener](#) arayüzü, 600–602  
ayarlama dinleyicileri, 656  
tanımlama alternatifleri, 612  
[AnimationDemo.java](#), 627  
anonim dinleyiciler için tanımlama, 609–610  
[AnonymousListenerDemo.java](#), 610–612  
[BallControl.java](#), 686  
[ButtonListener](#) sınıfı, 617  
vaka çalışması: ulusal bayraklar ve marşlar, 696  
[ClockAnimation.java](#), 628  
[Tsomboboxdemo.javat](#), 650  
[ControlCircle.javat](#), 607608n  
[ControlCircleWithoutEventHandling.javat](#), 606607n  
[DetectSourceDemo.javat](#), 612613  
olay dinleyicileri, 603–604  
[FrameAsListenerDemo.javat](#), 613614n  
[HandleEvent.javat](#), 601602  
tanımlama için iç sınıflar, 608–609

arayüz adaptörleri, 620–621, 651–652

dinleyicileri kaydet, 604–605

[SliderDemo.javat](#), 659

[TicTacToe.javat](#), 690

### Dinleyici arayüz adaptörleri

[Anahtar Olayları](#), 621

[ListeSeçimDinleyici](#) arayüzü, 651–652

[Fare Olayları](#), 620–621

### Listeler

oluşturma, 310–312

[ListDemo.java](#), 652–654

[JList](#) sınıfının genel görünümü, 650–652

[ListeSeçimDinleyici](#) arayüzü, 651–652

### Sabitler

Boolean sabitleri, 83

karakter sabitleri, 61

string sabitinden dizeleri oluşturma, 336

belirlenmiş, 48

kayan nokta literalleri, 69

tam sayı literalleri, 48–49

LiveLab notlandırma sistemi, 15

### Krediler

kredi hesaplama örneğinde faiz oranları, 71–72

Olay odaklı programlamada kredi hesaplayıcı vaka çalışması, 615–617

[Kredi.java](#) nesnesi, 377–379

Yerel alan ağları (LAN'ler), 8–9, 645–646

Yerel değişkenler, 196

Locker bulmacası, 260

Mantık hataları (bug'lar), 27–28, 119–120

Mantıksal operatörler (Boolean operatörleri)

gözden geçirme, 101–102

[TestBooleanOperators.java](#) örneği, 103–105

doğruluk tabloları, 102–103

### long veri tipi

karakterleri ve sayısal değerleri dizelere dönüştürme, 344

tam sayı literalleri ve, 49

[java.util.Random](#), 309–310

sayısal türlerin gözden geçirilmesi, 45

Döngü gövdesi, 134

### Döngü-devam-etme-koşulu

[do-while](#) döngüsü, 144–145

döngü tasarımı ve, 139

birden fazla çıkarma sınavında, 140

gözden geçirme, 134–135

### Döngüler

[break](#) ve [continue](#) anahtar kelimeleri kontrol olarak, 159–162

örnek çalışma: asal sayıları görüntüleme, 162–164

örnek çalışma: en büyük ortak böleni bulma, 155–157

örnek çalışma: sayıları tahmin etme, 137–139

örnek çalışma: Monte Carlo simülasyonu, 158–159

örnek çalışma: çoklu çıkarma testi, 139–141

örnek çalışma: gelecekteki öğrenim ücretini tahmin etme, 157

onay diyalogu ile kontrol etme, 164–165

diziler oluşturma, 237

hangisini kullanacağına karar verme, 150–151

tasarım stratejileri, 139



**do-while** döngüsü, 144–146  
 girdi ve çıktı yönlendirmeleri, 143–144  
 tekrarlama ile özyineleme karşılaştırması, 757–758  
 anahtar terimler, 165  
**için** döngü, 146–149  
 sayısal hataları en aza indirme ile ilgili, 154–155  
 iç içe geçirme, 152–153  
 genel bakış, 133–134  
 sorular ve alıştırmalar, 166–175  
 sentinel kontrollü, 141–143  
 özet, 166  
**iken** döngü, 133–137  
 Düşük seviyeli diller, 10

## M

Mac OS, 12  
 Makine dili  
   bayt kodu ile karşılaştırıldığında, 21  
   genel bakış, 9–10  
   kaynak programı, 10–11  
 Makine yığınları. *bkz* Çağrı yığınları  
 Ana sınıf  
   tanımlandı, 297  
   içinde **TestSimpleCircle.java** örneği, 298  
**ana** yöntem  
   içinde **ComputeExpression.java**, 19  
   çağırma, 181  
   ana sınıf vs., 297  
   komut satırından dize argümanları alma, 358–359  
   içinde **SimpleCircle.java** örneği, 300–301  
   içinde **TestSimpleCircle.java** örneği, 298  
   içinde **TestTV.java** örneği, 300–301  
   içinde **Welcome.java**, 17  
   içinde **WelcomeWithThreeMessages.java**, 18  
 Ana pencereler, 660  
 Bakım, yazılım geliştirme sürecinde, 59  
**MalformedURLException** sınıfı, 552  
 Mandelbrot fraktalı, 595–597  
 İşaretleyici arayüzler, 577  
 Markov matris, 290  
 Parantezleri eşleştir, içinde **Welcome.java**, 17  
**eşleşir** yöntemi, dizeler, 342  
**Math** sınıfı  
   **BigInteger** ve **BigDecimal** sınıfları, 397–398  
   karmaşık sayılar, 594–595  
   üs yöntemleri, 198–199  
   nesne yöntemlerini çağırma, 305  
   yöntemler genel olarak, 197  
   **pow(a, b)** yöntemi, 48  
   **rastgele** yöntemi, 96–97, 106–108, 200–203  
   yuvarlama yöntemleri, 199  
   hizmet yöntemleri, 199–200  
   trigonometri yöntemleri, 197–198  
 Matrisler  
   **GridLayout** yöneticisi için, 454  
   depolamak için iki boyutlu diziler, 264–265

**max** yöntemi, **Math** sınıfı  
   tanımlama ve çağırma, 180–182  
   aşırı yükleme, 194–195  
   genel bakış, 199–200  
**maxRow** değişkeni, en büyük toplamı bulmak için, 268  
 Mbps (saniyede milyon bit), 8  
 MB (megabayt), depolama için, 5  
 Megabayt (MB), depolama için, 5  
 Megahertz (MHz), saat hızı, 3  
 Bellek, bilgisayarlar, 4–5  
 Mersenne asal sayısı, 218  
 Mesaj diyalog kutuları, 22–23  
**MessagePanel** sınıfı  
   **DisplayClock.java**, 500–502  
   **MessagePanel.java**, 497–500  
   genel bakış, 495  
   **StillClock.java**, 502–503  
   **TestMessagePanel.java**, 495–496  
 Meta nesneleri, 692  
 Metreler, ayaklara dönüştürme, 213–214  
 Metot başlığı, 179  
 Metot imzası, 179  
 Metotlar  
   soyutlama, 203–204  
   nesne metotlarına erişim, 304–305  
   çağırma, 180–182  
   vaka çalışması: ondalıkları onaltılılara dönüştürme, 191–193  
   vaka çalışması: rastgele sayılar üretme, 201–203  
   sınıf, 312–313  
   yorumlama, 25  
   tanımlama, 178–180  
   tanımlayıcılar, 40  
   uygulama detayları, 207–210  
   nesne metotlarını çağırma, 305  
   anahtar terimler, 210  
   **Matematik** sınıfı, 197, 199–201  
   kodun modülerleştirilmesi, 189–191  
   isimlendirme kuralları, 44  
   nesne eylemleri tarafından tanımlanan, 296–297  
   aşırı yükleme, 193–196  
   genel bakış, 177–178  
   dizileri geçirme, 237–240  
   nesneleri geçirme, 322–326  
   parametreleri değerlerle geçirme, 186–189  
   iki boyutlu dizilere geçirme, 269–270  
   sorular ve alıştırmalar, 211–222  
   özyinelemeli yöntemler, 738  
   array'leri döndürme, 240–241  
   yuvarlama, 199  
   içinde **SimpleCircle.java** örneği, 300–301  
   statik, 313  
   adım adım iyileştirme, 203–204, 210  
   Özet, 211  
   üstten alta ve/veya alttan üste uygulama, 205–207  
   üstten alta tasarım, 204–205  
   izleme veya adım atma hata ayıklama tekniği olarak, 119  
   trigonometri, 197–198

Yöntemler (*devam*)

- değişken kapsamı ve, 196–197
- void** yöntem örneği, 183–185
- MHz (Megahertz), saat hızı, 3
- Microsoft Windows, 12
- MIDI dosyaları, 693
- Saniyede milyon bit (Mbps), 8
- min** yöntemi, **Math** sınıfı, 199–200
- Mnemonicler
  - montaj dilinde, 10
  - onay kutuları ve radyo düğmeleri için, 641–643
- Modemler (modülâtör/demodülâtör), 8
- Modifier tuşları, klavyelerde, 8
- Değiştiriciler, yöntem, 179
- Kodu modüler hale getirme
  - GreatestCommonDivisorMethod.java**, 189–190
  - özet, 189
  - PrimeNumberMethod.java**, 190–191
- Monitörler, 8
- Monte Carlo simülasyonu, 158–159
- Anakart, 3
- Fare
  - İmleç** sınıf, 463
  - G/Ç cihazı olarak, 8
- FareOlayları**
  - ControlCircleWithMouseAndKey.java**, 625 olay odaklı programlama, 617618
  - dinleyici arayüz adaptörleri, 620–621
  - MoveMessageDemo.java**, 618–620
- MouseListener** arayüzü, 618, 627
- MouseMotionListener** arayüzü, 619–620
- Çok boyutlu diziler. *bkz* Diziler, çok boyutlu
- Multimedya. *bkz* Appletler
- Çoktan seçmeli test, 270–272
- Çarpma (**\***=) atama operatörü, 53–54
- Çarpma operatörü (**\***), 19, 46, 50
- Çarpma tablosu, 152
- Nesne bileşiminde çokluklar, 382
- Çoklu işlem, 13
- Çoklu programlama, 13
- Çoklu iş parçacığı, 13
- Çok yönlü **if-else** ifadeleri
  - vergi hesaplamada, 99–101
  - gözden geçirme, 91–93
- Değiştirici yöntemler. *bkz* Setter (değiştirici) yöntemler

## N

- Adlandırılmış sabitler. *bak* Sabitler
- Adlandırma kuralları
  - sınıf tasarım yönergeleri, 391
  - bileşenler, 468
  - arayüzler, 582
  - programlama ve, 44
  - sarmalayıcı sınıflar, 393
- Adlandırma kuralları, tanımlayıcılar, 40
- N*-by-*n* matris, 215

Negatif açılar, yaylar çizme, 490

İç içe **if** ifadeleri

- vücut kitle indeksini hesaplamada, 97–99
- vergileri hesaplamada, 99–101
- genel bakış, 91
- İç içe döngü, 152–153, 267
- NetBeans
  - yerleşik hata ayıklama, 119
  - Java kaynak kodu oluşturma/düzenleme için, 19
  - Programları Durdur butonu ile durdurma, 450

## Ağ arayüz kartları (NIC'ler), 8

- yeni** operatörü
  - diziler oluşturma, 225
  - nesneler oluşturma, 303
- sonraki** yöntem, boşluk karakterleri ve, 69
- nextLine** yöntemi, boşluk karakterleri ve, 69
- Sonraki satır stili, blok stilleri, 25
- NIC'ler (ağ arayüz kartları), 8

## Argüman almayan yapılar

- sınıf tasarım yönergeleri, 391
- Kredi** sınıfı, 377
- sarıcı sınıfların olmaması, 394

Değil (!) mantıksal operatör, 102–105

Eşit değil (!=) karşılaştırma operatörü, 82

**NotSerializableException**, 727**null** değerler, nesneler, 305–306**NullPointerException**, çalışma zamanı hatası olarak, 306**Sayı** sınıfı

- vaka çalışması: soyut sayı sınıfı, 564
- sayısal sarıcı sınıflar için kök sınıf olarak, 585

## Sayılar/sayısal türler

- soyut sayı sınıfı, 565–567
- ikili. *bkz* İkili sayılar
- örnek olay: onaltılıları ondalıklara dönüştürme, 348–350
- örnek olay: asal sayıları görüntüleme, 162–164
- örnek olay: rastgele sayılar üretme, 201–203
- örnek olay: sayıları tahmin etme, 137–139
- char türlerine dönüştürme, 63
- sayısal türler arasında dönüşüm, 56–58, 364
- string'lere dönüştürme, 70, 344
- ondalık. *bkz* Ondalık sayılar
- double. *bkz* **double**
- kayan nokta. *bkz* Kayan nokta sayıları/**Float** veri türü
- rastgele sayılar üretme, 96–97
- GreatestCommonDivisorMethod.java**, 189–190
- onaltılı. *bkz* Onaltılı sayılar
- tam sayılar. *bak* Tam Sayılar (**tam** veri türü)
- EnBüyükSayılar.java**, 566–567
- genel bakış, 44–46
- AsalSayıYöntemi.java**, 190–191
- büyük sayıları işleme, 397–398
- Paydalar, rasyonel sayılarda, 584
- Sayısal tuş takımları, klavyelerde, 8
- Sayısal sabitler, 48–49
- Sayısal operatörler
  - karakterlere uygulanan, 64
  - genel bakış, 46–47

## O

Nesne sınıfı, 420–421, 429–430

Nesne G/Ç. *bak*

NesneGirdiAkışı/NesneÇıktıAkışı sınıfları

Nesne üye erişim operatörü (.), 304, 427

Nesne referans değişkenleri, 304

NesneGirdiAkışı/NesneÇıktıAkışı sınıfları

genel bakış, 724–725

serileştirilebilir arayüz, 727–728

Dizileri serileştirme, 728–729

TestNesneGirdiAkışı.java, 726

TestNesneÇıktıAkışı.java, 725–726

Nesne yönelimli programlama (OOP), 296, 304, 379–382

Nesneler

verilere ve yöntemlere erişim, 304–305

referans değişkenleri aracılığıyla erişim, 304

dizisi, 326–327

Dizi Listesi sınıfı, 430–431

diziler olarak, 239

Ses Klipleri sınıfı, 693–694

ilkel türler ile sarmalayıcı sınıf türleri arasında otomatik dönüşüm, 396–397

BigInteger ve BigDecimal sınıfları, 397–398

soyut sınıflardan oluşturulamaz, 564

vaka çalışması: yığınlar için sınıf tasarımı, 386–388

vaka çalışması: Kurs sınıfı tasarımı, 384–386

vaka çalışması: GuessDate sınıfı tasarımı, 388–391

tür dönüşümü, 425–426

DaireÖzelVeriAlanları.java örnek, 320–321

DaireStatikÜyeleri.java örnek, 313–314

sınıf soyutlama ve kapsülleme, 375–376

sınıf tasarım yönergeleri, 391–393

GUI bileşenlerini görüntülemek için sınıflar, 310–312

Java Kütüphanesinden sınıflar, 308

ilkel değişkenleri referans değişkenleri ile karşılaştırma, 306–308

oluşturma, 382–383

yapıcılar, 303

oluşturma, 298–299

sınıfları korumak için veri alanı kapsülleme, 319–320

Tarih sınıfı, 308–309

için sınıfları tanımlama, 296–298

equals metodu Object sınıfı, 429–430

event dinleyici nesnesi, 603

olay nesneleri, 602

değiştirilemez, 370–371

kalıtım. *bak* kalıtım

anahtar terimler, 328–329, 399

Kredi.javat, 377379

meta nesneleri, 692

null değerleri, 305–306

Object sınıfı, 420–421

nesne yönelimli düşünme, 379–382

özet, 295–296, 369–370

metotlara geçiş, 322–326

polimorfizm, 421–422

ilkel veri türü değerlerini işleme, 393–396

sorular ve alıştırmalar, 330–334, 399–406

Random sınıfı, 309–310

referans veri alanları ve, 305

SimpleCircle.java örneği, 300–301

statik değişkenler, sabitler ve metotlar ve, 312–313

özet, 329, 399

TestCircleWithPrivateDataFields.java örneği, 321–322

TestCircleWithStaticMembers.java örnek, 314–317

TestLoanClass.java, 376–377

TestSimpleCircle.java örnek, 298–300

TestTV.java örnek, 302–303

this referans ve, 373–375

TotalArea.java örnek, 327–328

TV.java örnek, 301–302

değişken kapsamı ve, 371–372

görünürlük değiştiricileri, 317–319

Sekizli tam sayılar, 49

Bir hata

diziler ve, 230

döngülerde, 136

OOP (nesne yönelimli programlama), 296, 304, 379–382

Operatörler

tanımlı, 46

uyumsuz, 104

Operatörler

atama operatörü (=), 42–43

artırılmış atama operatörleri, 53–54

karşılaştırma operatörleri, 82

artırma ve azaltma operatörleri, 54–56

sayısal operatörler, 46–47

öncelik ve bağlanırlık, 115–117

öncelik kuralları, 50–51

tekil ve ikili, 47

Seçenek pencereleri

önceden tanımlanmış Java sınıfları, 22

showConfirmDialog metodu, 117–119

showInputDialog metodu, 70, 72

Veya (||) mantıksal operatör, 102–105

İşletim sistemleri (OSs)

genel bakış, 12

görevleri, 12–13

Çıktı yönlendirmesi, 143–144

Çıktı akışları, 710–711. *bak*. G/Ç (girdi/çıkı)

OutputStream sınıfları

BufferedOutputStream sınıfları, 719–722

vaka çalışması: dosyaları kopyalama, 723–724

DataOutputStream sınıfları, 716–718

DetectEndOfFile.java, 719

FileOutputStream sınıfları, 713–714

FilterOutputStream sınıfları, 716

ObjectOutputStream sınıfı, 724–725

genel bakış, 712–713

serileştirme ve, 727

TestDataStream, 718719nTestFile

Stream, 715716nTestObjectOutp

utStream, 725726

Ovalar, çizim, 483–484, 487

Taşmalar

[Rasyonel](#) sınıf, 589

değişkenler, 45

Yükleme yöntemleri, 193–196

Geçersiz kılma yöntemleri, 418–420

## P

$\pi$  (pi), tahmin etme, 158–159, 215

Paket özel (paket erişimi) görünülük değiştiricileri, 317

Paketler

içe aktarma, 23–24

içe aktarma [JFrame](#) paketi, 449

sınıfları düzenleme, 318

programları düzenleme, 22

önceden tanımlı sınıflar, 23

*Aşağı Sayfa* tuşu, klavyelerde, 8

*Yukarı Sayfa* tuşu, klavyelerde, 8

[paintComponent](#) metodu

[DisplayImage.java](#) örneği, 505

daireler çizme ve, 488–489

[DrawPolygon.java](#) örneği, 492–493

[JComponent](#) sınıfı, 480, 482

[MessagePanel.java](#), 494

[StillClock.java](#), 503

[TestCenterMessage.java](#) örneği, 494

Palindromlar

durum çalışması: bir dizinin palindrom olup olmadığını kontrol etme, 347–348

durum çalışması: palindromları kontrol ederken

alfanümerik olmayan karakterleri göz ardı etme, 356–358

palindrom tam sayıları, 212

palindromik asal sayılar, 218

[RecursivePalindrome.java](#), 746–747

[RecursivePalindromeUsingSubstring.java](#), 745–746

Paneller

ekleme, 489

ekleme için [DisplayImage.java](#) örneği, 505

onay kutuları ve radyo düğmeleri için, 641

oluşturma, 311–312, 482

[AçıklamaPaneli](#) sınıfı, 645–647

[ŞekilPaneli](#) sınıfı. *bkz.* [ŞekilPaneli](#) sınıfı

[JPanel](#) sınıfı. *bkz.* [JPanel](#) sınıfı

[MesajPaneli](#) sınıfı. *bkz.* [MesajPaneli](#) sınıfı

alt konteynerler olarak, 458–460

<[param](#)> etiketi, applet'ler, 679

Parametreler

gerçek parametreler, 179

metotları tanımlama ve, 178–179

yerel değişken olarak, 196

sıra ilişkisi, 186

değerlerle geçiş, 186–189

değişken uzunlukta argüman listeleri, 244–245

Parantezler ( `()` )

metotları tanımlama ve çağırma ve, 203

[Welcome.java](#) içinde, 18

Ayrıştırma metodları, 395

Pascal, yüksek seviyeli diller, 11

Paylaşım yoluyla geçiş

metotlara diziler, 238

metotlara nesneler, 323–324

Değerle geçiş

metotlara diziler, 238

[Increment.java](#) örnek, 186–187

metotlara nesneler, 322–323

özet, 186

[TestPassByValue.java](#) örnek, 187–189

Şifreler, string'in geçerli şifre olup olmadığını kontrol etme, 362

Pentagonal sayılar, 212

Pikseller (resim elemanları)

koordinatlar, 481

tanımlı, 450

çözünürlüğü ölçme, 8

PNG (Taşınabilir Ağ Grafikleri), 465

[Point](#) sınıfı, 617–618

Çokgenler

çizim, 490–492

[DrawPolygon.java](#), 492–493

Polilinler, çizim, 491

Polimorfizm

[CastingDemo.java](#) örnek, 426–429

özet, 421

[PolymorphismDemo.java](#) örnek, 421–422

Taşınabilir Ağ Grafikleri (PNG), 465

Postfix azaltma operatörü, 54–55

Postfix artırma operatörü, 54–55

[pow](#) metodu, [Math](#) sınıfı, 48

Öncelik, operatör, 115–117

Prefix azaltma operatörü, 54–55

Prefix artırma operatörü, 54–55

Basılı simgeler, 469

Asal sayılar

örnek çalışma: asal sayıları görüntüleme, 162–164

[PrimeNumberMethod.java](#), 190–191

türleri, 218

Temel türler (fundamental types)

otomatik dönüşüm temel türler ile sarmalayıcı

sınıf türleri arasında, 396–397

dönüştürme, 427

için sınıflar, 350

temel tür parametrelerini referans tür parametreleri ile karşılaştırma, 324

ilkel değişkenleri referans değişkenleri ile karşılaştırma, 306–308

sarmalayıcı nesneyi dönüştürme (boxing/unboxing), 396

oluşturma dizileri, 326

ilkel veri türü değerlerini nesneler olarak işleme, 393–396

veri türlerini belirtme, 35

[print](#) metodu, [PrintWriter](#) sınıfı, 38, 544–545

[printf](#) metodu, [PrintWriter](#) sınıfı, 544

Dizileri yazdırma, 267

[println](#) metodu, [PrintWriter](#) sınıfı, 38, 544

[printStackTrace](#) metodu, 529

[PrintWriter](#) sınıfı

vaka çalışması: metni değiştirme, 548–549

verileri dosyaya yazma, 544–545

metin verilerini yazmak için, 710

## özel

veri alanlarının kapsülleme ve, 319–320

görünürlük değiştiricisi, 318–319, 437–440

## Problemler

alt problemlere ayırma, 164

ele almak için programlar oluşturma, 34

özyineleme ile çözme, 744–745

Prosedürel paradigma, nesne yönelimli paradigma ile karşılaştırıldığında, 381–382

Prosedürler, 179. bkz. Yöntemler

Dizileri işleme, 227–229

Programlama hataları. bkz. İstisnalar

[SınıfCastException](#), 426

yaygın sınıf tasarım hataları, 393

debugging, 119–120

mantık hataları, 27–28

döngülerle ilgili sayısal hataları en aza indirme, 154–155

çalışma zamanı hataları, 27

seçimler, 93–95

sözdizimi hataları, 18, 26–27

Programlama dilleri

montaj dili, 10

yüksek seviyeli diller, 10–12

Java. bkz. Java programlama

makine dili, 9–10

özeti, 2

Programlama stili

blok stilleri, 25–26

yorumlar ve, 25

girinti ve boşluk, 25

özeti, 24–25

Programlar/programlama

atama ifadeleri ve ifadeleri, 42–43

artırılmış atama operatörleri, 53–54

vaka çalışması: para birimlerini sayma, 65–68

vaka çalışması: mevcut zamanı gösterme, 51–53

karakter veri türü, 62–65

kademeli kodlama, 137

ifadeleri değerlendirme ve operatör öncelik kuralları, 50–51

üs işlemleri, 48

tanımlayıcılar, 40

artırım ve azaltım operatörleri, 54–56

girdi diyalogları, 70–72

giriş, 34

anahtar terimler, 72–73

modüler kodlama, 189–191

adlandırılmış sabitler, 43

adlandırma kuralları, 44

sayısal literaller, 48–49

sayısal operatörler, 46–47

sayısal tür dönüşümleri, 56–58

sayısal türler, 44–46

genel bakış, 2

sorular ve alıştırmalar, 74–80

konsoldan girdi okuma, 37–40

özyinelemeli yöntemler, 738

yazılım geliştirme süreci, 58–62

[string](#) veri türü, 68–69

özet, 73–74

değişkenler, 40–42

basit bir program yazma, 34–37

## Özellikler

yerleşim yöneticisi, 457–458

nesne, 296

[Korunan](#) veriler ve yöntemler, miras ve, 437–440

[korunan](#) görünürlük değiştiricisi, 318, 437–440

Pseudocode, 34

Halka açık sınıflar, 299

[halka açık](#) yöntem, 321

[halka açık](#) görünürlük değiştiricisi, 317–319, 437–440

Python, yüksek seviyeli diller, 11

## Soru

Quincunx, 258–259

## Bölümler

[Quotient.java](#) örnek, 518

[QuotientWithException.java](#) örnek, 520–522

[QuotientWithIf.java](#) örnek, 519

[QuotientWithMethod.java](#) örneği, 519–520

## R

Radyo düğmeleri

oluşturma, 310–312

eventler, 640

[GUIEventDemo.java](#), 640–643

[JRadioButton](#) sınıfı. *bak* [JRadioButton](#) sınıfı

gözden geçirme, 472–473

paneller için, 641

düğme türleri, 468

Düzensiz diziler, 266–267

RAM (rastgele erişim belleği), 5–6

[Random](#) sınıfı, [java.util](#), 309–310

[rastgele](#) metodu

örnek çalışma: rastgele sayılar üretme, 201–203

örnek çalışma: piyango, 106–108

[Matematik](#) sınıfı, 96–97, 200–201

Rastgele sayılar

örnek çalışma: rastgele sayılar üretme, 201–203

örnek çalışma: piyango, 106–108

vaka çalışması: Monte Carlo simülasyonu, 158–159

generating, 96–97

Rastgele erişim dosyaları

özeti, 729–731

[TestRandomAccessFile.java](#), 731–732

Rastgele erişim belleği (RAM), 5–6

[Rasyonel](#) sınıf

özeti, 584–585

[Rational.java](#) örneği, 586–589

[TestRationalClass.java](#) örneği, 585–586

Rasyonel sayılar, temsil etme ve işleme, 584–586

[readASolution](#) metodu, Sudoku ızgarasına uygulama, 276

Salt okunur akışlar, 729. *bkz. ayrıca* [InputStream](#) sınıf  
Dikdörtgenler, çizim, 483–484, 487

Özyineleme

ikili aramalar, 748–749

vaka çalışması: faktöriyel hesaplama, 738–739

vaka çalışması: Fibonacci sayıları hesaplama, 741–742

vaka çalışması: dizin boyutunu belirleme, 749

vaka çalışması: fraktallar, 754–755

vaka çalışması: Hanoi Kuleleri, 750–752

[Tsomputefactorial.yavat](#), 739741no

[mputefactoryaltairursion.yavat](#), 759nomputefi

[bonai.yavat](#), 742744

[DirectorySize.java](#), 749–750

yardımcı yöntemler, 746

tekrarlama ile karşılaştırıldığında, 757–758

anahtar terimler, 759

genel bakış, 737–738

problemi özyinelemeli düşünerek çözme, 744–745

sorular ve alıştırmalar, 760–767

Özyinelemeli palindrom sıralaması, özyi

nelemeli palindromeusingsubstring, 745746 özyinelemeli

seçim sıralaması, 747748

seçim sıralamaları, 747

[SierpinskiTriangle.java](#), 755–757

özet, 759

kuyruk özyineleme, 758

[TowersOfHanoi.java](#), 752–754

Özyinelemeli yöntemler, 738

Azaltma, özyinelemenin özellikleri, 744

Referans türleri

sınıflar olarak, 304

temel tür parametrelerini referans tür parametreleri ile  
karşılaştırma, 324

ilkel değişkenleri ile karşılaştırma, 306–308

referans veri alanları, 305

[string](#) veri türü olarak, 68

Referans değişkenleri

nesnelere erişim, 304

nesnelerin dizisi olarak dizi, 326

ilkel değişkenleri ile karşılaştırma, 306–308

[regionMatches](#) yöntemi, dizeler, 338–339

Dinleyicileri kaydet

[ControlCircle.javat](#), 607608nCon

[trolCircleWithMouseAndKey.javat](#), 624nDetect

[SourceDemo.javat](#), 613614

[GUIEventDemo.java](#), 641–643

[KeyEventDemo.java](#), 623

[LoanCalculator.java](#), 616

özet, 604–605

Regüler ifadeler, dizeleri eşleştirme, 342

Göreceli dosya adları, 541–542

Kalan (%) veya modül operatörü, 46, 50

Kalan (%) atama operatörü, 53–54

[repaint](#) metodu

mesaj paneline uygulama, 497–498

[StillClock.java](#), 502–503

Tekrar. *bkz.* Döngüler

[replace](#) metodu, dizeler, 341

[replaceAll](#) metodu, dizeler, 342

[replaceFirst](#) metodu, dizeler, 341

Gereksinimlerin tanımı, yazılım geliştirme sürecinde,  
58–59

Rezerve edilmiş kelimeler (anahtar kelimeler)

[syntax](#)'ı, 426

içinde [Welcome.java](#), 17

Çözünürlük, pikseller ve, 450

Kaynak dosyaları, applet'ler için [URL](#) sınıfını kullanarak bulma, 691–692

Kaynaklar, OS'lerin tahsisindeki rolü, 12

Sorumluluklar, sınıf tasarımı ilkesi olarak ayırım, 391

[return](#) ifadeleri, 181

Dönüş değeri türü

konstrüktörlerin olmaması, 303

metotları tanımlarken, 179

Yeniden kullanılabilir kod

adım adım iyileştirmenin faydaları, 210

kod modülerleştirme ve, 189

bunun için sınıfları tasarlama, 499

metot etkinleştirme, 182

bunun için metotlar, 178

ters metot, metotlardan diziler döndürme,  
240–241

RGB renk modeli, 460

RMF dosyaları, 693

Üzerine gelme simgeleri, 469

Yuvarlama yöntemleri, [Math](#) sınıfı, 199

Çalışma zamanı hataları

hata ayıklama, 119–120

değişken tanımlama, 525–526

istisna işleme ve, 39, 518

[NullPointerException](#) olarak, 306

programlama hataları, 27

Çalışma zamanı yığınları. *bkz.* Çağrı yığınları

## S

Sandbox güvenlik modeli, 675

[Scanner](#) sınıfı

girdi alma, 72

konsol girişi okumak için, 37–39

bir dosyadan veri okumak için, 545–547

metin verilerini okumak için, 710

Tarayıcılar

vaka çalışması: metin değiştirme, 548–549

oluşturma, 522

İşlemleri planlama, 13

Bilimsel gösterim, tam sayı sabitleri için, 49

Değişkenlerin kapsamı, 42, 196–197

Ekran çözünürlüğü, 8

Kaydırma çubukları

[TopKontrol.java](#), 685

zıplayan top durumunda zıplama hızını kontrol etme  
stüdyo, 683

genel bakış, 654–655

[Strolbardemo.javat](#), 655657



## Kaydırma panelleri

**AçıklamaPaneli.java**, 646  
genel bakış, 644  
kaydırma listeleri, 651

## Aramalar

diziler, 245  
ikili aramalar, 246–248, 748–749  
doğrusal aramalar, 245–246  
kelimeleri aramak için özyinelemeli yaklaşım, 738

Güvenlik kısıtlamaları, applet'ler, 675–676

## Seçim sıralamaları

diziler, 245, 249–250  
**RecursiveSelectionSort.java**, 747–748  
özyineleme kullanarak, 747

Seçim ifadeleri, 82, 84

## Seçimler

**Toplama.Quiz.java** örneği, 83–84  
**boolean** veri türü, 82–84  
durum çalışması: Vücut Kitle İndeksi hesaplama, 97–99  
durum çalışması: vergileri hesaplama, 99–101  
durum çalışması: artık yıl belirleme, 105–106  
durum çalışması: doğum günlerini tahmin etme, 86–89  
durum çalışması: piyango, 106–108  
yaygın hatalar, 93–95  
koşullu ifadeler, 111–112  
onay diyalogları, 117–119  
debugging, 119–120  
formatlama konsol çıktısı, 112–115  
erandom sayılar üretme, 96–97  
**eğer** ifadeleri, 84–86  
**eğer-değilse** ifadeleri, 89–91  
anahtar terimler, 120  
mantıksal operatörler, 101–105  
iç içe **eğer** ifadeleri ve çok yönlü **eğer-değilse** ifadeleri, 91–93  
operatör önceliği ve bağlanırlık, 115–117  
genel bakış, 81–82  
sorular ve alıştırmalar, 121–131  
özet ve alıştırma, 120–121  
**switch** ifadeleri, 108–111

Noktalı virgüller (;), yaygın hatalar, 93

Sentinel kontrollü döngüler, 141–143, 164–165

Sıralı dosyalar, girdi/çıktı akışları, 729

## Serileştirme

dizilerin, 728–729  
nesnelerin, 727

**setBackground** yöntemi, **Bileşen** sınıfı, 499

**setFont** yöntemi, **Bileşen** sınıfı, 499

**setForeground** yöntemi, **Bileşen** sınıfı, 499

**setLayout** yöntemi, **Bileşen** sınıfı, 451

**setLength** yöntemi, **StringBuilder** sınıfı, 355–356

**setRadius** yöntemi

**CircleWithPrivateDataFields.java** örneği, 321  
**SimpleCircle** örneği, 299

## Setter (değiştirici) yöntemleri

**ArrayList** sınıfı ve, 434  
veri alanlarının kapsüllenmesi ve, 320–322

Yüzeysel kopyalar, **clone** yöntemi ve, 579–580

Kod paylaşımı, 182

**short** sayısal tür, 45

Kısa devreli VEYA operatörü, 104

**showConfirmDialog** yöntemi, **JOptionPane** sınıfı, 117–119

**showInputDialog** yöntemi, **JOptionPane** sınıfı, 70, 72

**showMessageDialog** yöntemi, 22–23

Dizileri karıştırma, 228–229, 268

Sierpinski üçgeni

vaka çalışması, 754–755

özyinelemeli hesaplama, 762, 766–767

**SierpinskiTriangle.java**, 755–757

İmzalı applet'ler, 676

**sin** metodu, trigonometrik, 197–198

Sinüs fonksiyonu, 511

Tek hassasiyetli sayılar. *bkz* Kayar noktalı sayılar/**float**  
data türü

Tek boyutlu diziler. *bkz* Diziler, tek boyutlu

Batırma sıralamaları, 258

## Kaydırcılar

genel bakış, 657–658

**SliderDemo.java**, 658–660

## Yazılım

geliştirme süreci, 58–62

programlar olarak, 2

## sırala metodu

**Diziler** sınıfı, 252

**ComparableRectangle.java** örnek, 575–576

**SortRectangles.java** örnek, 576–577

özyineleme kullanarak, 747–748

## Dizileri sıralama

balon sıralamaları, 258

ekleme sıralamaları, 250–252

Özet, 248

seçim sıralamaları, 245, 249–250

Kaynak nesneler, olay kaynakları ve, 602–603

Kaynak program veya kaynak kodu, 10, 40

Boşluk, programlama stili ve, 25

Özel karakterler, 18

Özel ithalat, 24

Splash ekranları, 467

**split** metodu, dizeler, 341, 342

**StackOfIntegers** sınıfı, 386–387

**StackOverflowError**, özyineleme nedeniyle, 757

## Yığınlar

vaka çalışması: özel yığın sınıfı, 436–437

vaka çalışması: yığınlar için sınıf tasarımı, 386–388

**start** metodu, applet yaşam döngüsü yöntemleri, 677–678

Nesnelerin durumu, 296

## İfadeler

birer birer yürütme, 119

tekrarlayarak yürütme (döngüler), 134

yüksek seviyeli dillerde, 10

terminatörler, 17

Statik veriler, **GuessDate** sınıfında, 388, 390

## Statik yöntemler

**CircleWithStaticMembers.java** içinde, 313–314

sınıf tasarım yönergeleri, 392–393

bildirim, 313

tanımlandı, 312

**GuessDate** sınıfında, 388–391

çağırma, 23

örnek yöntemleri ile statik yöntemleri ne zaman kullanacağınız, 313–314

sarmalayıcı sınıflar ve, 395

## Statik değişkenler

in**CircleWithStaticMembers.java**, 313–314

sınıf, 312–313

sınıf tasarım yönergeleri, 392–393

değerlendirme, 313

örnek değişkenler ile karşılaştırıldığında, 312

**TestCircleWithStaticMembers.java**, 314

örnek değişkenler ile statik arasında ne zaman kullanılacağı, 316

## Aşamalı iyileştirme

faydaları, 210

uygulama detayları, 207–210

metot soyutlaması, 203–204

yukarıdan aşağıya ve/veya aşağıdan yukarıya uygulama, 205–207

yukarıdan aşağıya tasarım, 204–205

**durdurma** metodu, applet yaşam döngüsü yöntemleri, 677–678

## Depolama aygıtları

CD'ler ve DVD'ler, 6

diskler, 6

genel bakış, 5

USB bellekler, 7

Depolama birimleri, bellek ölçmek için, 4–5

**String** sınıfı, 336

String birleştirme operatörü (+), 36, 340

String sabitleri, 336

String değişkenleri, 336

**StringBuffer** sınıfı, 336, 353, 357

**StringBuilder** sınıfı

örnek çalışma: palindromları kontrol ederken alfanümerik olmayan karakterleri göz ardı etme, 356–358

string'leri değiştirme, 353–355

özet, 336, 353

**toString**, **kapasite**, **uzunluk**, **setLength**, ve **charAt**

metotları, 355–356

## String'ler

ikili G/Ç'de, 716–717

örnek çalışma: string'in palindrom olup olmadığını kontrol etme, 347–

348örnek çalışma: onaltılık sayıları ondalık sayılara çevirme, 348–350örnek

çalışma: palindromları kontrol ederken alfanümerik olmayan karakterler

eri göz ardı etme, 356–358

**Karakter** sınıfı, 350–351

komut satırı argümanları, 358–361

karşılaştırma, 337–339

birleştirme, 36, 68

oluşturma, 336

dönüştürme, değiştirme ve ayırma, 341

çift'e dönüştürme, 71–72

dizilere dönüştürme/almada, 343–344

tsayılara dönüştürme/almada, 70, 344

**CountEachLetter.java** örneği, 351–353

karakterleri veya alt dizeleri bulma, 342–343

biçimlendirme, 344–347

değiştirilemez ve interned, 336–337

anahtar terimler, 361

desenlere göre eşleştirme, değiştirme ve ayırma, 342

uzunluğu elde etme, bireysel karakterleri alma ve birleştirme, 339–340

genel bakış, 335–336

applet'lere geçirme, 679

sorular ve alıştırımlar, 362–368

**string** veri türü, 68–69

**StringBuilder** ve **StringBuffer** sınıfları, 353–356

alt dizeler, 37, 340–341

özet, 361–362

içinde**Welcome.java**, 17

## Strings (grafikler)

**FontMetrics** sınıfını kullanarak ortalama, 493–495

dişleme, 483–484

## Alt sınıflar

soyut yöntemler ve, 560

soyutlama, 564

yapıcılar, 414–415

grafik tuvali oluşturma, 482–483

Exception sınıfı, 524–525

kalıtım ve, 408–409

**RuntimeException** sınıfı, 525

Alt konteynerler, paneller olarak, 458–460

Alt izinler, 749

Alt arayüzler, 582

**substring** yöntemi, 340, 746

Alt dizeler, 340–343

Çıkarma (–) operatörü, 46, 50

Çıkarma (–=) atama operatörü, 53–54

Alt pencereler, 660

Sudoku bulmacası, 274–277

**toplam** yöntemi, 269–270

**super** anahtar kelimesi, 414

Üst sınıf yöntemleri, 417

## Üst sınıflar

soyut sınıfın somut olabileceği, 564

uzantı sınıfları, 581

**Component**, **Container**, **v** ve **JComponent** sınıflarındaki ortak özellikler, 462

**Container** sınıfı olarak, 460

miras alma ve, 408–409

ile ilgili alt sınıflar, 560

Tamamlayıcı karakterler, Unicode, 62

**swap** metodu

bir dizideki elemanları değiştirme, 239–240

**TestPassByValue.java** örneğinde, 187–189

## Swing

**AbstractButton** sınıfı, 468–469

appletler. *bkz.* **JApplet** sınıfı

AWT karşısında, 446

butonlar. *bkz.* **JButton** sınıfı

onay kutuları. *bkz.* **JCheckBox** sınıfı



kombinasyon kutuları. *bkz.* [JComboBox](#) sınıfı

Swing GUI bileşenlerinin ortak özellikleri, 462–465

bileşenler. *bkz.* [JComponent](#) sınıfı

sabitler, 572–573

kullanıcı arayüzleri oluşturma, 468

diyaloglar. *bkz.* [JDialog](#) sınıfı

event sınıfları, 603

dosya seçiciler ([JFileChooser](#) sınıfı), 550, 551–552

çerçeveler. *bkz.* [JFrame](#) sınıfı

etiketler. *bakınız* [JLabel](#) sınıfı

listeler. *bakınız* [JList](#) sınıfı

seçenek panelleri ([JOptionPane](#) sınıfı), 72

paneller. *bakınız* [JPanel](#) sınıfı

radio düğmeleri. *bakınız* [JRadioButton](#) sınıfı

kaydırma çubukları. *bakınız* [JScrollBar](#) sınıfı

kaydırma panelleri, 644, 646

kaydırıcılar ([JSlider](#) sınıfı), 657–660

metin alanları. *bakınız* [JTextArea](#) sınıfı

metin sınıfları, 644

metin bileşenleri ([JTextComponent](#) sınıfı), 474–475

metin alanları. *bakınız* [JTextField](#) sınıfı

## switch ifadeleri

[ChineseZodiac.java](#) örneği, 110–111

özet, 108–110

## Sözdizimi hataları (derleme hataları)

yaygın hatalar, 18

hata ayıklama, 119–120

programlama hataları, 26–27

## Sözdizimi kuralları, [Welcome.java](#), 18

## Sistem aktiviteleri, OS'lerin rolü, 12

## Sistem analizi, yazılım geliştirme sürecinde, 58–59

## Sistem tasarımı, yazılım geliştirme sürecinde, 58, 60

## Sistem hataları, 524

## Sistem kaynakları, tahsis etme, 12

## [Sistem.in](#), 37

## [Sistem.out](#), 37, 112–115

## T

### Tablolar, depolama, 264

### Etiketler, HTML, 673–674

### Kuyrukta geri çağırma

[ComputeFactorialTailRecursion.java](#), 759 genel bakış, 758

### tan [metodu](#), trigonometrik, 197–198

### TB (terabayt), depolama, 5

### Ekip çalışması, adım adım iyileştirme ile kolaylaştırılmıştır, 210

### Terabayt (TB), depolama, 5

### Test

adım adım iyileştirmenin faydaları, 210

yazılım geliştirme sürecinde, 59, 61–62

### Metin

örnek çalışma: metni değiştirme, 548–549

mesaj iletişim kutusunda görüntüleme, 22–24

dosyalar, 710

yazı tipi özellikleri, 493

### JButton sınıfı ile konumlandırma, 470–471

.[txt](#) dosyaları (metin), 712

### Metin alanları

[AçıklamaPanel.java](#), 645–646

overview of, 644–645

[TextAreaDemo.java](#), 646–647

### Metin alanları

çerçevelere ekleme, 452

ızzaraya ekleme, 455

oluşturma, 311

eylemler, 640

[JTextField](#) sınıfı, 474–475

panel için, 641

### Metin G/Ç

ikili G/Ç ile karşılaştırıldığında, 711–712

Java'da işleme, 710–711

özet, 710

### TextPad, Java kaynak kodu oluşturma/düzenleme için, 19

### bu referansı

yapıcıları çağırma ile, 374–375

özet, 373–375

gizli veri alanlarına referans verme ile, 373–374

### Üç boyutlu diziler. *bkz* Diziler, çok boyutlu

### [throw](#) anahtar kelimesi

zincirlenmiş istisnalar, 538

istisnaları yeniden fırlatmak için [throw ex](#), 537

istisnaları fırlatmak için, 527

### [Throwable](#) sınıfı

istisnalar hakkında bilgi almak, 529–530

[java.lang](#), 523–525

### İstisnaları fırlatma

[CircleWithException.java](#) örneği, 531

[QuotientWithException.java](#) örneği, 521

tekrar fırlatma, 536–537

[TestCircleWithCustomException.java](#) örneği, 539

[throw](#) anahtar kelimesi için, 526–527

### [throws](#) anahtar kelimesi

istisnaları tanımlamak için, 527

[IOException](#), 712–713

### Tic-tac-toe oyunu, 283

### [Timer](#) sınıfı

kullanarak animasyon, 625–626

[Animasyondemo.yavat](#), 626628n

[ball.yavat](#), 684685

[ClockAnimation.java](#), 628–629

### [toCharArray](#) metodu, dizileri dizilere dönüştürme, 343

### Toggle butonları, 468

### Token okuma yöntemleri, [Scanner](#) sınıfı, 546–547

### [toLowerCase](#) metodu, [Character](#) sınıfı, 341

### [toLowerCase](#) metodu, [Character](#) sınıfı, 351

### Araç ipuçları, bileşenler için, 462

### Yukarıdan aşağıya tasarım, 204–205

### Yukarıdan aşağıya uygulama, 205–207

### Üst düzey konteynerler, 447–448

### [toString](#) metodu

[ArrayList](#) sınıfı, 433

[Arrays](#) sınıfı, 253

**toString** metodu (*devam*)**Date** sınıfı, 309**Object** sınıfı, 429**StringBuilder** sınıfı, 355–356**total** değişkeni, toplamaları saklamak için, 268**toUpperCase** metodu, **Character** sınıfı, 341, 351

Hanoi Kuleleri problemi

vaka çalışması, 750–752

özyinelemeli hesaplama, 762

**TowersOfHanoi.java**, 752–754

Bir programı izleme, 36

**geçici** anahtar kelime, serileştirme ve, 727

Transistörler, CPU'lar, 3

Trigonometrik yöntemler, **Math** sınıfı, 197–198**trim** metodu, dizeler, 341

Doğru/yanlış (Boolean) değerler, 82

Doğruluk tabloları, 102–103

**try** bloğu**InputMismatchExceptionDemo.java** örnek, 522**QuotientWithException.java** örnek, 520

istisnaların ne zaman kullanılacağı, 535–536

**try-catch** blokları

istisnaları yakalamak, 525, 527–529

zincirlenmiş istisnalar, 537–538

**CircleWithException.java** örnek, 532–533

istisnaları yeniden fırlatmak, 536–537

istisnaların ne zaman kullanılacağı, 535–536

İkiz asal sayılar, 218

İki boyutlu diziler. *bak* Diziler, iki boyutlu**.txt** dosyaları (metin), 712

Tür dönüşümü

char ve sayısal türler arasındaki, 63

kesinlik kaybı, 67

sayısal tür dönüşümü için, 56–57

**U**

Kullanıcı Arayüzü (UI)

**BallControl.java**, 685**ComboBoxDemo.java**, 649

oluşturma, 641

**Sayfa demosu**, 653n**multipleindosdemo**, 661nscrollbard**emo**, 656**SliderDemo.javat**, 659**TextAreaDemo.java**, 647

UML (Birleşik Modelleme Dili)

birleşim gösterimi, 382

sınıf diyagramları ile, 297

**Kredi** sınıfı için diyagram, 376**StackOfIntegers** diyagramı, 386

statik değişkenler ve yöntemler diyagramı, 312–313

Tekil operatörler, 47

Kutunun açılması, 396

Kontrolsüz istisnalar, 525

Aşırı akma, kayan nokta sayıları, 45

Unicode

karakter veri türü (**char**) ve, 62–65

karakterleri karşılaştırma, 82

veri girişi ve çıkışı akışları, 717

rastgele sayılar oluşturma ve, 201

metin kodlaması, 710

metin G/Ç vs. ikili G/Ç, 711

Birleşik Modelleme Dili. *bak* UML (Birleşik Modelleme Dili)Birleşik Kaynak Bulucular. *bak* URL'ler (Birleşik Kaynak Bulucular)

Her bellek baytı için benzersiz adresler, 5

Evrensel seri veri yolu (USB) bellek sürücüleri, 7

UNIX epoch, 51

Yukarı dönüştürme nesneleri, 425

**URL** sınıfı**DisplayImagePlayAudio.yavat**, **shachn****displayImageVithurl.yavat**, 692693nyava  
**.net**, 551

kaynakları kullanarak bulma, 691–692

URL'ler (Birleşik Kaynak Bulucular)

**ReadFileFromURL.java** örneği, 551–552

Web'den veri okuma, 551

USB (evrensel seri veri yolu) bellek sürücüleri, 7

Kullanıcı arayüzü. *bak* UI (kullanıcı arayüzü)UTF-8, 717. *bak* ayrıcaUnicode**V****valueOf** yöntemleri

string'leri dizilere dönüştürme, 344

wrapper sınıfları ve, 395

Değer döndüren yöntemler

**return** ifadeleri gereklidir, 181**TestReturnGradeMethod.java**, 183–185**void** yöntemi ve, 179

Değişken uzunlukta argüman listeleri, 244–245

Değişkenler

Boolean değişkenleri. *bak* Boolean değişkenleriilkel değişkenleri referans değişkenleri ile karşılaştırma,  
306–308kontrol değişkenleri **for** döngülerinde, 147–148

değişkenleri tanımlama, 35–36, 41

dizi değişkenlerini tanımlama, 225

iki boyutlu diziler için tanımlama, 264–265

gösterme/değiştirme, 120

gizli, 372

tanımlayıcılar, 40

indeksli dizi değişkenleri, 226–227

isimlendirme kuralları, 44

taşma, 45

genel bakış, 40–41

referans değişkenleri, 304

kapsamı, 42, 196–197, 371–372

statik değişkenler, 312–313

Dikey hizalama, **AbstractButton** sınıfı, 470

Dikey kaydırma çubukları, 656  
 Dikey kaydırıcılar, 657, 659  
 Dikey metin konumu, 470–471  
 Sanal makineler (VM'ler), 21. *bkz. ayrıca* JVM (Java Sanal Makinesi)  
 Görünürlük (erişim) değiştiricileri  
   sınıflar ve, 317–319  
   **korumalı, genel, ve özel**, 437–440  
 Visual Basic, yüksek seviyeli diller, 11  
 VLSI (çok büyük ölçekli entegrasyon), 738  
 VM'ler (sanallaştırılmış makineler), 21. *bkz.* JVM (Java Sanal Makinesi)  
**void** yöntem  
   tanımlı, 179  
   tanımlama ve çağırma, 183  
   **Test yöntemleri**, 183

## W

WAV dosyası, 693  
 Ağ, dosya verilerini okuma, 551–552  
 Web tarayıcıları  
   applet yürütmesini ve yaşam döngüsünü kontrol etme, 677  
   applet'leri görüntüleme, 674–675  
 Ağ sunucuları, üzerinde uygulama geliştirme, 628  
**while** döngüleri  
   vaka çalışması: sayı tahmini, 137–139  
   vaka çalışması: çoklu çıkarma sınavı, 139–141  
   vaka çalışması: gelecekteki öğrenim ücretini tahmin etme, 157  
   ne zaman kullanılacağına karar verme, 150–151  
   tasarım stratejileri, 139

**do-while** döngüsü. *bkz.* **do-while** döngüsü  
 girdi ve çıktı yönlendirmeleri, 143–144  
 özet, 134–136  
 RepeatAdditionQuiz.java örneği, 136–137  
 sentinel kontrollü, 141–143  
 sentaksı, 134  
 Boşluk Karakterleri  
   karakterler, 69  
   token okuma yöntemlerinde ayırıcı olarak, 546–547  
 Wildcard import, 24  
 Windows. *bkz.* Çerçeveler (pencereler)  
 Windows, çoklu  
   **Histogram.java**, 662–664  
   **MultipleWindowsDemo.java**, 661–662  
   genel bakış, 660–661  
**Windows** sınıfı, 447  
 Windows OS'leri, 12  
 Kablosuz ağ, 8  
 Wrapper sınıfları  
   ilkel türler ile sarmalayıcı sınıf türleri arasında otomatik dönüşüm, 396–397  
   **File** sınıfı olarak, 541  
   sayısal, 585  
   ilkel türler ve, 350, 393–396  
 Metin veya kelimelerin satırlarını sarmak, 644, 646  
 Sadece yazma akışları, 729. *bkz.* **OutputStream** sınıfı

## X

**XListener/XEvent** dinleyici arayüzü, 603–604

*Bu sayfa kasıtlı olarak boş bırakılmıştır*

## Java Hızlı Referans

### Sık Kullanılan Statik Sabitler/Yöntemler

```
matematik.pi
Math.random()
Math.pow(a, b)
System.currentTimeMillis()
System.out.println(anyValue)
OptionPane.showMessageDialog(title, message)

OptionPane.showInputDialog(nisim mesaj)
Integer.parseInt(string)
Double.parseDouble(string)
Arrays.sort(type[] list)
Arrays.binarySearch(type[] list, type key)
```

### Dizi/Uzunluk/İlkleyici

```
int[] list = new int[10];
list.length;
int[] list = {1, 2, 3, 4};
```

### Çok Boyutlu Dizi/Uzunluk/İlkleyici

```
int[][] list = new int[10][10];
liste.uzunluk;
liste[0].uzunluk;
int[][] liste = {{1, 2}, {3, 4}};
```

### Düzensiz Dizi

```
int[][] m = {{1, 2, 3, 4}, n {1, 2, 3},
 {1, 2},
 {1}};
```

### Metin Dosyası Çıkışı

```
PrintWriter çıkışı =
 new PrintWriter(dosya adı);
çıkış.println(...);
çıkış.printn(...);
çıkış.printn(...);
```

### Metin Dosyası Girişi

```
Scanner input = new Scanner(
 new Dosya(filename));
```

### Dosya Sınıfı

```
Dosya file =
 new Dosya(filename);
file.exists()
file.renameTo(Dosya)
file.delete()
```

### Object Sınıfı

```
Object o = new Object();
no.to String();
no.equal s(o1);
```

### Comparable Arayüz

```
c.compareTo(Comparable)
c, Comparable bir nesnedir
```

### String Sınıfı

```
String s = "Hoş geldiniz";
String s = new String(char[]);
int length = s.length();
char ch = s.charAt(index);
int d = s.compareTo(s1);
boolean b = s.equals(s1);
boolean b = s.startsWith(s1);
boolean b = s.endsWith(s1);
String s1 = s.trim();
```

```
String s1 = s.toUpperCase();
String s1 = s.toLowerCase();
int index = s.indexOf(ch);
int index = s.lastIndexOf(ch);
String s1 = s.substring(ch);
String s1 = s.substring(i,j);
char[] c = s.toCharArray();
String s1 = s.replaceAll(regex, repl);
String[] belirteçler = s.split(regex);
```

### ArrayList Sınıfı

```
ArrayList<E> liste = new ArrayList<E>();
liste.add(object);
liste.add(index, nesne);
liste.clear();
Object o = liste.get(index);
boolean b = liste.isEmpty();
boolean b = liste.contains(object);
int i = liste.size();
liste.remove(index);
liste.set(index, object);
int i = liste.indexOf(object);
int i = liste.lastIndexOf(object);
```

### printf Metodu

```
System.out.printf("%b %c %d %f %e %s",
 true, 'A', 45, 45.5, 45.5, "Hoş Geldiniz");
System.out.printf("%-5d %10.2f %10.2e %8s", 45, 45.5, 45.5, "Hoş Geldiniz");
```

## Java Hızlı Referans

### Konsol Girişi

```
Tarayıcı girişi = tnewt Tarayıcı(System.in);nintt int
Value = input.nextInt();
longt longValue = input.nextLong();ndouble
t doubleValue = input.nextDouble();nfloatt floatVal
ue = input.nextFloat();nString string = input.nex
t();
```

### Konsol Çıkışı

```
System.out.println(anyValue);
```

### GUI Giriş Diyalogu

```
String string = JOptionPane.showInputDialog(
"Girişi girin");
intt intValue = Integer.parseInt(string);
double doubleValue =
Double.parseDouble(string);
```

### Mesaj Diyalogu

```
JOptionPane.showMessageDialog(null,
"Girişi girin");
```

### Temel Veri Türleri

byte	8 bit
kısa	16 bit
int	32 bit
uzun	64 bit
batmadan yüzmek	32 bit
çift	64 bit
karakter	16 bit
<b>boolean</b>	doğru/yanlış

### Aritmetik Operatörler

	toplama
	çıkarma
	çarpma
	bölme
	kalan
++değişken	önden artırım
--değişken	önden azaltma
değişken++	sondan artırım
değişken--	sondan azaltma

### Atama Operatörleri

=	atama
+=	toplama ataması
-=	çıkarma ataması
*=	çarpma ataması
/=	bölme ataması
%=	kalan ataması

### İlişkisel Operatörler

<	daha az
<=	daha az veya eşit
>	daha fazla
>=	daha fazla veya eşit
==	eşit
!=	eşit değil

### Mantıksal Operatörler

&&	kısa devre VE
	kısa devre VEYA
!	DEĞİL
^	özel VEYA

### if İfadeleri

```
if(koşul) {
 ifadeler;
}
```

```
if (koşul) {
 ifadeler;
}
```

```
başka{
 ifadeler;
}
```

```
if(koşul1) {
 ifadeler;
}
```

```
else if (koşul2) {
 ifadeler;
}
```

```
başka {
 ifadeler;
}
```

### switch İfadeleri

```
switch(intExpression) {
 case value1:
 ifadeler;
 kırıldım
 ...
 kaset değeri:
 ifadeler;
 mola ve
 varsayılan:
 ifadeler;
}
```

### döngü İfadeleri

```
while (koşul) {
 ifadeler;
}
```

```
nokta{
 ifadeler;
}while (koşul);
```

```
for(başlat; koşul;
 ayarlama) {
 ifadeler;
}
```

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 35

### Uluslararasılaştırma

#### Hedefler

- Java'nın uluslararasılaştırma özelliklerini tanımlamak (§35.1).
- Dil, ülke ve varyant ile bir yerel oluşturmak (§35.2).
- Yerel ayara göre tarih ve saat göstermek (§35.3).
- Yerel ayara göre sayıları, para birimlerini ve yüzdeleri göstermek (§35.4) .
- Kaynak paketleri kullanarak uluslararası izleyiciler için uygulamalar geliştirmek (§35.5).
- Metin G/Ç için kodlama şemalarını belirtmek (§35.6).

### 35.1 Giriş

Birçok web sitesi, okuyucuların anlayabildiği bir dilde yazılmış olanı seçebilmeleri için web sayfalarının birkaç versiyonunu sürdürmektedir. Dünyada bu kadar çok dil olduğu için, her yerdeki tüm müşterilerin ihtiyaçlarını karşılamak için yeterince farklı versiyon oluşturmak ve sürdürmek oldukça sorunlu olacaktır. Java bu konuda devreye giriyor. Java, uluslararasılaştırmayı desteklemek için sıfırdan tasarlanmış ilk dildir. Sonuç olarak, programlarınızın herhangi bir ülke veya dil için özelleştirilmesine olanak tanır, bu da gerektirmez

kodu zorlaştıran değişiklikler.

Uluslararasılaştırmayı destekleyen başlıca Java özellikleri şunlardır:

#### <kenar notu: Unicode>

- Java karakterleri *Unicode* kullanır, bu, dünyanın çeşitli dillerinde yazılı metinlerin değişimi, işlenmesi ve görüntülenmesini desteklemek için Unicode Konsorsiyumu tarafından oluşturulmuş 16 bitlik bir kodlama şemasını ifade eder. Unicode kodlamasının kullanımı, herhangi bir uluslararası dilde dizeleri işleyebilen Java programları yazmayı kolaylaştırır. (Tüm Unicode karakterlerini görmek için [mindprod.com/jgloss/reuters.html](http://mindprod.com/jgloss/reuters.html) adresini ziyaret edin.)

#### <margin note: Yerel sınıf>

- Java, belirli bir yerel hakkında bilgi kapsüllemek için *Locale* sınıfını sağlar. Bir *Locale* nesnesi, tarih, saat ve sayı gibi yerel duyarlı bilgilerin nasıl görüntüleneceğini ve dizeleri sıralama gibi yerel duyarlı işlemlerin nasıl gerçekleştirileceğini belirler. Tarih, saat ve sayıları biçimlendirme ile dizeleri sıralama için kullanılan sınıflar *java.text* paketinde gruplandırılmıştır.

#### <kenar notu: KaynakBundle>

- Java, *ResourceBundle* sınıfını, durum mesajları ve GUI bileşen etiketleri gibi yerel spesifik bilgileri programdan ayırmak için kullanır. Bilgiler kaynak kodunun dışında saklanır ve bir *ResourceBundle* üzerinden dinamik olarak çalışma zamanında erişilip yüklenebilir, programın içine sabit kodlanmak yerine.

Bu bölümde, farklı bölgeler, ülkeler ve diller için tarihleri, sayıları, para birimlerini ve yüzdeleri nasıl biçimlendireceğinizi öğreneceksiniz. Ayrıca, bir bileşenin kullanıcıların yerel ayarlarına ve tercihlerine bağlı olarak hangi resimlerin ve dizelerin kullanılacağını tanımlamak için kaynak paketlerini nasıl kullanacağınızı da öğreneceksiniz.

### 35.2 *Locale* Sınıfı

Bir *Locale* nesnesi, belirli bir dilin veya geleneğin kullanıldığı coğrafi, siyasi veya kültürel bir bölgeyi temsil eder. Örneğin, Amerikalılar İngilizce konuşur ve Çinliler Çince konuşur. Tarih, sayı, para birimi ve yüzdeleri biçimlendirme konvansiyonları bir ülkeden diğerine farklılık gösterebilir.

Örneğin, Çinliler tarihi temsil etmek için yıl/ay/gün kullanırken, Amerikalılar ay/gün/yıl kullanır. Yerel ayarların yalnızca ülke ile tanımlanmadığını anlamak önemlidir. Örneğin, Kanadalılar, Kanada'nın hangi bölgesinde yaşadıklarına bağlı olarak ya Kanadalı İngilizce ya da Kanadalı Fransızca konuşurlar.

#### NOT

##### <margin note: yerel ayar özelliği Bileşen>

Her Swing kullanıcı arayüzü sınıfı, *yerel ayar* özelliğini *Bileşen* sınıfından miras alır.

Bir *Yerel Ayar* nesnesi oluşturmak için, belirli bir dil ve isteğe bağlı ülke ve varyant ile üç yapıcından birini kullanın; Şekil 35.1'de gösterildiği gibi.



java.util.Yerel Ayar	
+Yerel Ayar(dil: String)	Bir dil kodundan bir yerel ayar oluşturur.
+Yerel Ayar(dil: String, ülke: String)	Dil ve ülke kodlarından bir yerel yapı oluşturur.
+Yerel Ayar(dil: String, ülke: String, varyant: String)	Dil, ülke ve varyant kodlarından bir yerel yapı oluşturur.
+getÜlke(): String	Bu yerel için ülke/bölge kodunu döndürür.
+getDil(): String	Bu yerel için dil kodunu döndürür.
+getVaryant(): String	Bu yerel için varyant kodunu döndürür.
+getVarsayılan(): Yerel Ayar	Makinedeki varsayılan yereli alır.
+getGörüntüÜlkesi(): String	Mevcut yerelde ifade edilen ülkenin adını döndürür.
+getGörüntüDili(): String	Mevcut yerelde ifade edilen dilin adını döndürür.
+getGörüntüAdı(): String	Yerelin adını döndürür. Örneğin, ad, Locale.CHINA yereli için <u>Çince</u> (Çin) dir.
+getGörüntüVaryantı(): String	Varlık varsa, yerelin varyantının adını döndürür.
+getMevcutYerelAyarları(): Yerel Ayar[]	Bir dizide mevcut yerelleri döndürür.

### Şekil 35.1

*Locale sınıfı bir yereli kapsüller.*

#### <kenar notu: dil>

Dil, geçerli bir dil kodu olmalıdır; yani, ISO-639 tarafından tanımlanan küçük harfle yazılmış iki harfli kodlardan biri olmalıdır. Örneğin, **zh** Çince'yi, **da** Danca'yı, **en** İngilizce'yi, **de** Almanca'yı ve **ko** Korece'yi temsil eder. Tablo 35.1 dil kodlarını listeler.

#### <kenar notu: ülke>

Ülke, geçerli bir ISO ülke kodu olmalıdır; yani, ISO-3166 tarafından tanımlanan büyük harfle yazılmış iki harfli kodlardan biri olmalıdır. Örneğin, **CA** Kanada'yı, **CN** Çin'i, **DK** Danimarka'yı, **DE** Almanya'yı ve **US** Amerika Birleşik Devletleri'ni temsil eder. Tablo 35.2 ülke kodlarını listeler.

Tablo 31.1 Yaygın Dil Kodları

Kod	Dil	Kod	Dil
da	Danca	ja	Japonca
de	Almanca	ko	Korece
el	Yunanca	nl	Flemenkçe
en	İngilizce	no	Norveççe
es	İspanyolca	pt	Portekizce
fi	Fince	sv	İsveççe
fr	Fransızca	tr	Türkçe
it	İtalyanca	zh	Çince

Tablo 31.2 Yaygın Ülke Kodları

Kod	Ülke	Kod	Ülke
AT	Avusturya	IE	İrlanda
BE	Belçika	HK	Hong Kong
CA	Kanada	IT	İtalya
CH	İsviçre	JP	Japonya
CN	Çin	KR	Kore
DE	Almanya	NL	Hollanda
DK	Danimarka	NO	Norveç
ES	İspanya	PT	Portekiz
FI	Finlandiya	SE	İsveç
FR	Fransa	TR	Türkiye
GB	Birleşik Krallık	TW	Tayvan
GR	Yunanistan	US	Amerika Birleşik Devletleri

#### <kenar notu: varyant>

Argüman varyantı nadiren kullanılır ve yalnızca bir tarayıcıya veya satıcıya özgü bilgileri belirtmek için olağanüstü veya sistem bağımlı durumlar için gereklidir. Örneğin, Norveççe'nin iki set yazım kuralı vardır, bir

geleneksel birine *bokmål* denir ve yeni birine *nynorsk* denir. geleneksel yazım için yerel ayar şu şekilde oluşturulacaktır:

```
newt Yerel Ayar(t"hayır",t "HAYIR",t "B");
```

Kolaylık olması açısından, *Locale* sınıfı birçok önceden tanımlanmış yerel sabit içerir. *Locale.CANADA* Kanada ülkesi ve İngilizce dili içindir; *Locale.CANADA\_FRENCH* Kanada ülkesi ve Fransızca dili içindir. Diğer birkaç yaygın sabit şunlardır:

```
Locale.US, Locale.UK, Locale.FRANCE, Locale.GERMANY,
Locale.ITALY, Locale.CHINA, Locale.KOREA, Locale.JAPAN, ve
```

*Locale* sınıfı ayrıca dil bazında aşağıdaki sabitleri sağlar:

```
Locale.CHINESE, Locale.ENGLISH, Locale.FRENCH,
Locale.GERMAN, Locale.ITALIAN, Locale.JAPANESE,
Locale.KOREAN, Locale.SIMPLIFIED_CHINESE, ve
Locale.TRADITIONAL_CHINESE
```

İPUCU:

Statik metodu çağırabilirsiniz *getAvailableLocales()* içinde *Locale* sınıfı, sistemde desteklenen tüm mevcut yerelleri elde etmek için. Örneğin,

```
Locale[] availableLocales = Calendar.getAvailableLocales();
```

bir dizideki tüm yerelleri döndürür.

İPUCU:

Makinenizde varsayılan bir yerel vardır. Programı çalıştırırken dil ve bölge parametrelerini sağlayarak bunu geçersiz kılabilirsiniz, şöyle:

### java Duser.language=zh Duser.region=CN Ana Sınıfı

#### <margin note: yerel duyarlı>

Bir işlemin görevini yerine getirmek için bir *Yerel* gerektirmesi *yerel-duyarlı* olarak adlandırılır. Örneğin, bir tarih veya saat gibi bir sayının görüntülenmesi, yerel-duyarlı bir işlemdir; sayı, kullanıcının yerelinin gelenek ve göreneklerine göre biçimlendirilmelidir. Aşağıdaki bölümler yerel-duyarlı işlemleri tanıtmaktadır.

### 35.3 Tarih ve Saat Gösterimi

#### <margin note: Tarih>

#### <margin note: Takvim>

Uygulamalar genellikle tarih ve saati elde etme ihtiyacı duyar. Java, tarih ve saati *java.util.Date* sınıfında sistem bağımsız bir şekilde kapsüller; ayrıca *java.util.TimeZone* ile zaman dilimleriyle başa çıkmayı ve *java.util.Calendar* ile *Date*'den ayrıntılı bilgi çıkarmayı sağlar. Farklı yerel ayarlar, tarih ve saati gösterme konusunda farklı geleneklere sahiptir. Yıl, ay veya gün önce mi gösterilmeli? Tarih alanlarını ayırmak için eğik çizgiler, noktalar veya iki nokta üst üste mi kullanılmalı? Ayların isimleri dilde nedir? *java.text.DateFormat* sınıfı, kullanıcıya gösterim için tarih ve saati yerel ayara duyarlı bir şekilde biçimlendirmek için kullanılabilir. *Date* sınıfı §8.6.1, "*Date* Sınıfı" bölümünde tanıtılmıştır ve *Calendar* sınıfı ile onun alt sınıfı *GregorianCalendar* §15.4, "Vaka Çalışması: *Calendar* ve *GregorianCalendar*" bölümünde tanımlanmıştır.

### 35.3.1 *TimeZone* Sınıfı

<kenar notu: *t* *TimeZonet*>

*TimeZone*, bir zaman dilimi ofsetini temsil eder ve ayrıca yaz saati uygulamasını hesaplar. Belirtilen bir zaman dilimi kimliği için bir *TimeZone* nesnesi almak için *TimeZone.getTimeZone(id)* kullanın. Bir *Calendar* nesnesinde bir zaman dilimi ayarlamak için, bir zaman dilimi kimliği ile *setTimeZone* yöntemini kullanın. Örneğin, *cal.setTimeZone(TimeZone.getTimeZone("CST"))*, zaman dilimini Merkezi Standart Zaman olarak ayarlar. Java'da desteklenen tüm mevcut zaman dilimlerini bulmak için, *TimeZone* sınıfında ki static method *getAvailableIDs()* yöntemini kullanın. Genel olarak, uluslararası zaman dilimi kimliği, Avrupa/Berlin, Asya/Taypey ve Amerika/Washington gibi kıta/şehir biçiminde bir dizedir. Ayrıca, ana makinedeki varsayılan zaman dilimini elde etmek için *TimeZone* sınıfındaki static method *getDefault()* yöntemini de kullanabilirsiniz.

### 35.3.2 *DateFormat* Sınıfı

<kenar notu: *t* *DateFormatt*>

*DateFormat* sınıfı, tarih ve saati bir dizi stil ile biçimlendirmek için kullanılabilir. *DateFormat* sınıfı, birkaç standart biçimlendirme stilini destekler. Tarih ve saati biçimlendirmek için, *getDateInstance*, *getTimeInstance* ve *getDateTimeInstance* yöntemlerinden birini kullanarak bir *DateFormat* örneği oluşturun ve örnek üzerinde *format(Date)* yöntemini uygulayın; bu, Şekil 35.2'de gösterilmiştir.

<i>java.text.DateFormat</i>	
<i>+format(date: Date): String n</i>	Bir tarihi tarih/saat dizesine biçimlendirir.
<i>+getDateInstance(): DateFormatn</i>	Varsayılan yerel ayar için varsayılan biçimlendirme stiline sahip tarih biçimlendiricisini alır.
<i>+getDateInstance(dateStyle: int): DateFormat n+</i>	Varsayılan yerel ayar için verilen biçimlendirme stiline sahip tarih biçimlendiricisini alır.
<i>getDateInstance(dateStyle: int, aLocale: _____)</i> <i>Yerel ayar: DateFormatn+getDateTimeInstance(): DateFormatn+getDateTimeInstance(dateStyle: int, ntimeStyle: int): DateFormat</i>	Verilen yerel için verilen biçimlendirme stiline sahip tarih biçimleyicisini alır.
<i>+getDateTimeInstance(dateStyle: int, ntimeStyle: int, aLocale: Locale): DateFormatn+getDateTimeInstance(): DateFormat</i>	Varsayılan yerel için varsayılan biçimlendirme stiline sahip tarih ve saat biçimleyicisini alır.
<i>+getDateTimeInstance(dateStyle: int, ntimeStyle: int, aLocale: Locale): DateFormatn+getDateTimeInstance(): DateFormat</i>	Verilen tarih ve saat biçimlendirme stilleri ile varsayılan yerel için tarih ve saat biçimlendiricisini alır.
<i>+getDateTimeInstance(dateStyle: int, ntimeStyle: int, aLocale: Locale): DateFormatn+getDateTimeInstance(): DateFormat</i>	Verilen yerel için verilen biçimlendirme stilleri ile tarih ve saat biçimlendiricisini alır.
<i>+getDateTimeInstance(dateStyle: int, ntimeStyle: int, aLocale: Locale): DateFormatn+getDateTimeInstance(): DateFormat</i>	Kısa stil kullanan varsayılan bir tarih ve saat biçimlendiricisi alır.

Şekil 35.2

*DateFormat* sınıfı tarih ve saati biçimlendirir.

*dateStyle* ve *timeStyle* aşağıdaki sabitlerden biridir:

***DateFormat.SHORT*, *DateFormat.MEDIUM*, *DateFormat.LONG*, *DateFormat.FULL*. Kesin sonuç yerel ayara bağlıdır, ancak genel olarak,**

- ***SHORT*** tamamen sayısaldir, örneğin 7/24/98 (tarih için) ve 4:49 PM (saat için).
- ***MEDIUM*** daha uzundur, örneğin 24-Jul-98 (tarih için) ve 4:52:09 PM (saat için).
- ***LONG*** daha da uzundur, örneğin 24 Temmuz 1998 (tarih için) ve 4:53:16 PM EST (saat için).
- ***TAM*** tamamen belirtilmiştir, örneğin Cuma, 24 Temmuz 1998 (tarih için) ve 4:54:13 PM EST (saat için).

Aşağıda verilen ifadeler, belirtilen bir zaman dilimi ile mevcut zamanı göstermektedir (CST), biçimlendirme stili (tam tarih ve tam saat) ve yerel ayar (ABD).

GregorianCalendar takvim = **new** GregorianCalendar();

```
DateFormat biçimlendirici = DateFormat.getDateInstance(
 DateFormat.FULL, DateFormat.FULL, Locale.US);
ZamanDilimi timeZone = TimeZone.getTimeZone("CST");
formatter.setTimeZone(timeZone);
System.out.println("Yerel saat " +
 formatter.format(calendar.getTime()));
```

### 35.3.3 BasitTarihBiçimi Sınıfı

**<margin note: BasitTarihBiçimi>**

Tarih ve saat biçimlendirme alt sınıfı, **BasitTarihBiçimi**, tarih ve saat biçimlendirme için herhangi bir kullanıcı tanımlı deseni seçmenizi sağlar. Aşağıda gösterilen yapıcı, bir **BasitTarihBiçimi** nesne oluşturmak için kullanılabilir ve bu nesne, bir **Tarih** nesnesini istenen biçimde bir dizeye dönüştürmek için kullanılabilir.

```
public BasitTarihBiçimi(String desen) Parametr
```

e **desen** özel anlamları olan karakterlerden oluşan bir dizedir. Örneğin, **y** yıl, **m** ay, **d** ayın günü, **G** dönem belirleyici, **h** saat, **m** saatin dakikası, **s** dakikanın saniyesi ve **z** zaman dilimidir. Bu nedenle, aşağıdaki kod "Mevcut zaman 1997.11.12 MS 04:10:18 PST'te" gibi bir dize göstermektedir çünkü desen "yyyy.MM.dd G 'at' hh:mm:ss z"dır.

```
BasitTarihBiçimi formatter
```

```
= new BasitTarihBiçimi("yyyy.MM.dd G 'at' hh:mm:ss z");
tarih mevcutZaman = new Tarih();
String dateString = formatter.format(currentTime);
System.out.println("Mevcut zaman " + dateString);
```

### 35.3.4 DateFormatSymbols Sınıfı

**<kenar notu: DateFormatSymbols>**

**DateFormatSymbols** sınıfı, ayların adları ve haftanın günlerinin adları gibi yerelleştirilebilir tarih-saat biçimlendirme verilerini kapsüller, Şekil 35.3'te gösterildiği gibi.

java.text.DateFormatSymbols	
+DateFormatSymbols()	Varsayılan yerel için bir DateFormatSymbols nesnesi oluşturur.
n+DateFormatSymbols(Yerel ayar) n	Verilen yerel için bir DateFormatSymbols nesnesi oluşturur.
+getAmPmStrings(): String[]	AM/PM dizelerini alır. Örneğin: "AM" ve "PM".
+getEras(): Dize[]	çağ dönemleri dizelerini alır. Örneğin: "M.S." ve "M.Ö.".
+getMonths(): String[] n	Ay dizelerini alır. Örneğin: "Ocak", "Şubat" vb.
+setMonths(newMonths: String[]): void n+	Bu yerel için ay dizelerini ayarlar.
getShortMonths(): String[]	Kısa ay dizelerini alır. Örneğin: "Oca", "Şub" vb.
+setShortMonths(newShortMonths: String[]): geçersiz	Bu yerel için kısa ay dizelerini ayarlar.
+getWeekdays(): String[] n	Hafta günü dizelerini alır. Örneğin: "Pazar", "Pazartesi" vb.
+setWeekdays(newWeekdays: String[]): void n+	Hafta günü dizelerini ayarlar.
etShotWeekdays(): String[]	Kısa hafta günü dizelerini alır. Örneğin: "Paz", "Pzt" vb.
+setShortWeekdays(newWeekdays: String[]): geçersiz	Kısa hafta günü dizelerini ayarlar. Örneğin: "Paz", "Pzt" vb.

### Şekil 35.3

The **DateFormatSymbols** sınıfı yerelleştirilebilir tarih-saat biçimlendirme verilerini kapsüller.

Örneğin, aşağıdaki ifade varsayılan yerel için ay adlarını ve hafta günü adlarını görüntüler.

```

DateFormatSymbols semboller = t.new DateFormatSymbols(); nS
tring[] ayAdları = semboller.getMonths();
for (int i = 0; i < monthNames.length; i++) {
 System.out.println(monthNames[i]); // Ocak'ı görüntüle, ...
}

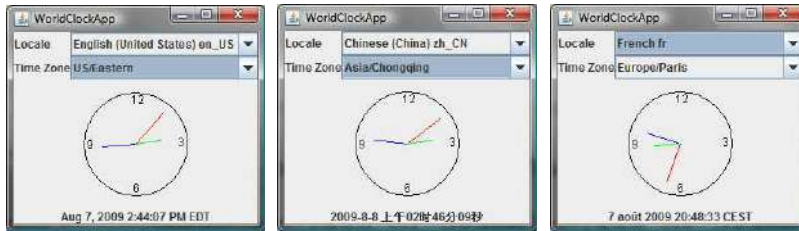
String[] weekdayNames = semboller.getWeekdays();
için (int i = 0; i < weekdayNames.length; i++) {
 System.out.println(weekdayNames[i]); // Pazar, Pazartesi, ... göster
}

```

Aşağıdaki iki örnek, yerel ayara dayalı olarak tarih, saat ve takvim nasıl görüntüleneceğini göstermektedir. İlk örnek, bir saat oluşturur ve tarihi ve saati yerel ayara duyarlı formatta görüntüler. İkinci örnek, günlerin adlarının uygun yerel dilde gösterildiği birkaç farklı takvimi görüntüler.

### 35.3.5 Örnek: Uluslararası Saat Gösterimi

Belirtilen yerel ayar ve saat dilimine dayalı olarak mevcut zamanı gösteren bir saat görüntüleyene bir program yazın. Yerel ayar ve saat dilimi, sistemde mevcut yerel ayarları ve saat dilimlerini içeren açılır kutulardan seçilir; Şekil 35.4'te gösterildiği gibi.

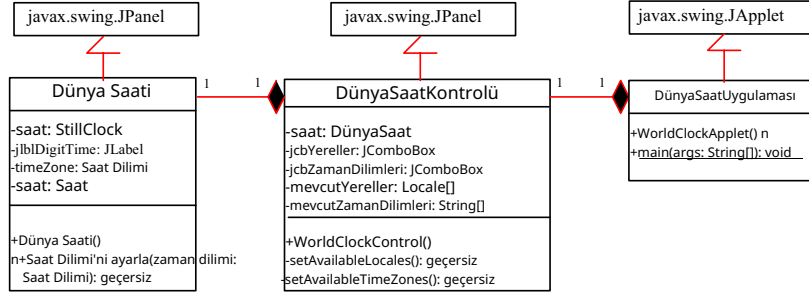


Şekil 35.4

Program, belirtilen yerel ayar ve saat dilimi ile mevcut zamanı gösteren bir saat görüntüler.

Programdaki ana adımlar şunlardır:

1. **JPanel** adında bir alt sınıf oluşturun **WorldClock** (Liste 35.1) ve bir **StillClock** sınıfının örneğini içersin (Liste 13.10'da geliştirilmiştir, StillClock.java), ve ortada yerleştirin. **JLabel** oluşturun ve dijital zamanı göstermek için güneyde yerleştirin. Belirli bir yerel ve zaman dilimi için mevcut zamanı elde etmek üzere **GregorianCalendar** sınıfını kullanın.
2. **JPanel** adında bir alt sınıf oluşturun **WorldClockControl** (Liste 35.2) **WorldClock** örneğini ve yerel ayarları ve saat dilimlerini seçmek için iki **JComboBox** örneğini içerecek şekilde.
3. **WorldClockApp** adında bir applet oluşturun (Liste 35.3) ve **WorldClockControl** örneğini içermesini sağlayın ve applet'in bağımsız olarak çalışmasını etkinleştirin. Bu sınıflar arasındaki ilişki Şekil 35.5'te gösterilmiştir.



Şekil 35.5

**DünyaSaatUygulaması** içerir **DünyaSaatKontrolü**, ve **DünyaSaatKontrolü** içerir **DünyaSaat**.

#### Listeleme 35.1 DünyaSaat.java

<kenar notu satır 11: zamanlayıcı oluştur>

<kenar notu satır 12: saat oluştur>

<kenar notu satır 26: zamanlayıcı dinleyici sınıfı>

```

1 javax.swing.*'i içe aktarın;
2 java.aut.*'u içe aktarın;
3 java.awt.event.*'i içe aktarın;
4 java.util.Calendar'ı içe aktarın;
5 java.util.TimeZone'u içe aktarın;
6 java.util.GregorianCalendar'ı içe aktarın;
7 java.text.*'i içe aktarın;
8
9 public class DünyaSaati extends JPanel {
10 private ZamanDilimi timeZone = TimeZone.getTimeZone("EST");
11 özel Zamanlayıcı timer = t newt Timer(t1000t,t newt TimerListener());
12 private DuranSaat clock = new DuranSaat();
13 özel JLabel jlbl DigitTime = t newt JLabel(t""t, JLabel.CENTER);
14
15 public DünyaSaati() {
16 setLayout(tnewt BorderLayout()); n 17 add(cl
17 ock, BorderLayout.CENTER); n 18 add(jlblDigitTim
18 e, BorderLayout.SOUTH); n 19 timer.start();
19
20 }
21
22 public void setTimeZone(ZamanDilimi timeZone) {
23 thist.timeZone = saat dilimi;
24 }
25
26 özel sınıf TimerListener Uygular ActionListener {
27 @Geçersiz Kıl
28 genel boşluk actionPerformed(ActionEvent e) {
29 Takvim takvim =
30 yeni GregorianCalendar(timeZone, getLocale());
31 saat.setSaat(takvim.get(Calendar.HOUR));
32 saat.setDakika(takvim.get(Calendar.MINUTE));
33 saat.setSaniye(takvim.get(Calendar.SECOND));
34
35 // Etiket üzerinde dijital zamanı göster
36 TarihFormatı biçimlendirici = TarihFormatı.getDateTimeInstance

```

```

37 (TarihFormatı.ORTA, TarihFormatı.UZUN, getLocale());
38 biçimlendirici.setTimeZone(timeZone);
39 jlblDigitTime.setText(formatter.format(calendar.getTime())); n 40 }

41 } n 42
}

```

### Liste 35.2 WorldClockControl.java

<margin note line 8: yerel diller>

<margin note line 9: saat dilimleri>

<margin note line 12: kombinasyon kutuları>

<margin note line 16: saat oluştur>

<margin note line 31: arayüz oluştur>

<margin note line 53: yeni yerel dil>

<margin note line 59: yeni saat dilimi>

```

1 javax.swing.*'i içe aktarın;
2 java.aut.*'u içe aktarın;
3 java.awt.event.*'i içe aktarın;
4 java.util.*'i içe aktarın;
5
6 genel sınıf WorldClockControl t extendst JPanel {
7 // Mevcut tüm yerel dilleri ve saat dilimi kimliklerini al
8 Privatet Locale[] kullanılabilirLocales = Locale.getAvailableLocales();
9 Private String[] kullanılabilir TimeZones = TimeZone.getAvailableIDs();
10
11 // Mevcut yerel dilleri ve saat dilimlerini görüntülemek için kombinasyon kutuları
12 özel JComboBox jcbLocales = t newt JComboBox();
13 özel JComboBox jcbTimeZones = yeni JComboBox();
14
15 // Bir saat oluştur
16 özel WorldClock clock = yeni WorldClock();
17
18 genel WorldClockControl() {
19 // jcbLocales'i mevcut tüm yerel dillerle başlat
20 setAvailableLocales();
21
22 // jcbTimeZones'i mevcut tüm saat dilimleriyle başlat
23 setAvailableTimeZones();
24
25 // yerel dil ve saat dilimini başlat
26 saat.setLocale(
27 kullanılabilirLocales[jcbLocales.getSelectedIndex()]); n 28 saat.setTimeZ
29 one(TimeZone.getTimeZone(
29 kullanılabilirTimeZones[jcbTimeZones.getSelectedIndex()]);
30
31 JPanel panel1 = new JPanel();
32 panel1.setLayout(new GridLayout(2, 1));
33 panel1.add(new JLabel("Yerel"));
34 panel1.add(new JLabel("Zaman Dilimi"));
35 JPanel panel2 = new JPanel();
36
37 panel2.setLayout(newt GridLayout(t2t,t 1t));
38 panel2.add(jcbLocales, BorderLayout.CENTER); n 39 panel2.ad
39 d(jcbTimeZones, BorderLayout.CENTER);
40
41 JPanel paneli3 = newt JPanel();
42 panel3.setLayout(tnewt BorderLayout());

```

```

43 panel3.add(panel1, BorderLayout.WEST); n 44 panel3.a
dd(panel2, BorderLayout.CENTER);
45
46 setLayout (new BorderLayout());
47 add(panel3, BorderLayout.NORTH); n 48 add(cl
ock, BorderLayout.CENTER);
49
50 jcbLocales.addActionListener (new ActionListener() {
51 @Geçersiz kıl
52 genel boşluk actionPerformed(ActionEvent e) {
53 saat.setLocale(
54 kullanılabilirLocales[jcbLocales.getSelectedIndex()]); n 55 }

56 });
57 jcbTimeZones.addActionListener(tnewt ActionListener() { n 58 @Override

59 genel boşluk actionPerformed(ActionEvent e) {
60 saat.setTimeZone(TimeZone.getTimeZone(n 61 kullanılabilirT
imeZones[jcbTimeZones.getSelectedIndex()]); n 62 }

63 }); n 64 }

65
66 özel boşluk setAvailableLocales() {
67 için (int i = 0; i < availableLocales.length; i++) {
68 jcbLocales.addItem(availableLocales[i].getDisplayName() + t " "n 69 + AvailableLocales[
i].toString());
70 } n 71 }

72
73 özel boşluk setAvailableTimeZones() {
74 // Zaman dilimlerini sırala
75 Arrays.sort(availableTimeZones);
76 için (int i = 0; i < availableTimeZones.length; i++) {
77 jcbTimeZones.addItem(availableTimeZones[i]); n 78 }

79 } n 80
}

```

### Liste 35.3 WorldClockApp.java

<kenar notu satır 8: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2
3 genel sınıf WorldClockApp t Applet'i genişletir {
4 /** Applet'i oluştur */
5 herkese açık Dünya Saati Uygulaması() {
6 add(tnewt WorldClockControl()); n 7 }

8 }

```

*WorldClock* sınıfı, belirtilen yerel ve saat dilimi için bir *Calendar* nesnesi elde etmek üzere *GregorianCalendar* kullanır (satır 28). *WorldClock* sınıfı *JPanel*'den türediği için, her GUI bileşeni nin *locale* özelliği vardır, takvim için yerel, *WorldClock* üzerinden *getLocale()* kullanılarak elde edilir (satır 28).



Bir *StillClock* örneği oluşturulur (satır 12) ve panelde yerleştirilir (satır 17). Saat, satır 28-35'teki mevcut *Calendar*

nesnesi kullanılarak her bir saniyede bir güncellenir.

Bir *DateFormat* örneği oluşturulur (satırlar 34-35) ve tarih, yerel ayara göre biçimlendirilir (satır 37).

WorldClockControl sınıfı, bir WorldClock örneği ve iki kombinasyon kutusu içerir. Kombinasyon kutuları, mevcut tüm yerel ayarları ve saat dilimlerini saklar (satırlar 64-77). Yeni seçilen yerel ayar ve saat dilimi, saatte ayarlanır (satırlar 50-61) ve mevcut yerel ayar ve saat dilimine dayalı olarak yeni bir zamanı görüntülemek için kullanılır.

### 35.3.6 Örnek: Bir Takvim Gösterimi

Belirtilen yerel ayara dayalı bir takvim görüntüleyen bir program yazın, Şekil 35.6'da gösterildiği gibi. Kullanıcı, sistem tarafından desteklenen tüm mevcut yerel ayarların listesini içeren bir kombinasyon kutusundan bir yerel ayar belirleyebilir. Program başladığında, yılın mevcut ayına ait takvim görüntülenir. Kullanıcı, takvimde gezinmek için *Önceki* ve *Sonraki* düğmeleri kullanabilir.



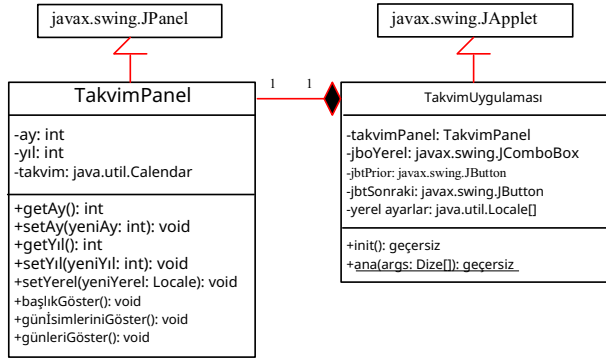
Şekil 35.6

*Takvim applet'i, belirtilen yerel ayara sahip bir takvim görüntüler.*

Programdaki ana adımlar şunlardır:

1. Belirtilen yerel ayar ve saat dilimine dayalı olarak verilen yıl ve ay için takvimi görüntülemek üzere *JPanel* sınıfının bir alt sınıfını *CalendarPanel* olarak oluşturun (Liste 35.4).

2. *CalendarApp* adında bir applet oluşturun (Liste 35.5). Bir *CalendarPanel* örneğini ve iki düğmeyi, *Önceki* ve *Sonraki* tutacak bir panel oluşturun. Paneli applet'in ortasına yerleştirin. Bir kombinasyon kutusu oluşturun ve bunu applet'in güneyine yerleştirin. Bu sınıflar arasındaki ilişkiler Şekil 35.7'de gösterilmiştir.



**Şekil 35.7**

*CalendarApp içerir CalendarPanel.*

#### Listeleme 35.4 CalendarPanel.java

<margin note line 9: başlık için etiket>  
 <margin note line 12: günler için etiketler>  
 <margin note line 14: takvim>  
 <margin note line 15: ay>  
 <margin note line 16: yıl>  
 <margin note line 19: günler için panel>  
 <margin note line 23: etiketler oluştur>  
 <margin note line 32: başlığı yerleştir>  
 <margin note line 33: günü yerleştir>  
 <margin note line 36: mevcut takvimi al>  
 <margin note line 39: takvimi güncelle>  
 <margin note line 42: başlığı göster>  
 <margin note line 43: günleri göster>  
 <margin note line 47: başlığı göster>  
 <margin note line 51: yeni başlık>  
 <margin note line 57: gün adlarını al>  
 <margin note line 69: boş jpDays paneli>  
 <margin note line 71: gün adlarını göster>  
 <margin note line 82: bu aydan önceki günler>  
 <margin note line 92: bu aydaki günler>  
 <margin note line 101: bu aydan sonraki günler>  
 <margin note line 108: jpDays'i yeniden boyama>  
 <kenar notu satır 113: takvimi güncelle>  
 <kenar notu satır 125: yeni ay ayarla>  
 <kenar notu satır 138: yeni yıl ayarla>  
 <kenar notu satır 146: yeni yerel ayarla>

```

1 java.aut.*'u içe aktarın;
2 javax.swing.*'i içe aktarın;
3 javax.swing.border.Line Kenarlığını içe aktar;
4 java.util.*'i içe aktarın;
5 java.text.*'i içe aktarın;
6
7 genel sınıf CalendarPanel t extendst JPanel {
8 // Başlık etiketi
9 özel JLabel jlb Başlığı = t newt JLabel(t" "t, JLabel.CENTER);
10
11 // Gün adlarını ve günleri görüntülemek için maksimum etiket sayısı
12 özel JLabel[] jlb Gün = t newt JLabel[t49t];

```

```

13
14 özel java.util.Takvim takvimi;
15 private int month; // Belirtilen ay
16 private int year; // Belirtilen yıl
17
18 // Gün adlarını ve günleri tutmak için jpDays paneli
19 özel JPanel Günleri = t newt JPanel(newt Grid Layout(t0t,t 7t));
20
21 genel Takvim Paneli() {
22 // Günleri görüntülemek için etiketler oluştur
23 kale (tintt i = t 0t; i < t 49t; i++) {
24 jlblDay[i] = t newt JLabel(); n 25 jlblDay[i].
setBorder(tnewt LineBorder(Color.black, t 1t));n 26 jlblDay[i].setHorizontalAl
ignment(JLabel.RIGHT); n 27 jlblDay[i].setVerticalAlignment(JLabel.TOP
); n 28 }
29
30 // Başlığı ve takvim gövdesini panelde yerleştir
31 thist.setLayout(tnewt BorderLayout());
32 thist.add(jlblHeader, BorderLayout.NORTH);
33 thist.add(jpDays, BorderLayout.CENTER);
34
35 // Mevcut ay ve yılı ayarla
36 takvim = new GregorianCalendar();
37 ay = takvim.get(Calendar.MONTH);
38 yıl = takvim.get(Calendar.YEAR);
39 takvimiGüncelle();
40
41 // Takvimi göster
42 başlığıGöster();
43 günleriGöster();
44 }
45
46 /** Yerel ayara göre başlığı güncelle */
47 private void başlığıGöster() {
48 SimpleDateFormat sdf =
49 new SimpleDateFormat("MMMM yyyy", getLocale());
50 String başlık = sdf.format(takvim.getTime());
51 jlblHeader.setText(başlık);
52 }
53
54 /** Yerel ayara göre gün adlarını güncelle */
55 özel voidt showDayNames() {
56 DateFormatSymbols dfs = t newt DateFormatSymbols(getLocale()); n 57 String
dayNames[] = dfs.getWeekdays();
58
59 // jlblDay[0], jlblDay[1], ..., jlblDay[6] gün adları için
60 kale (tintt i = t 0t; i < t 7t; i++) {
61 jlblDay[i].setText(dayNames[i + 1]);
62 jlblDay[i].setHorizontalAlignment(JLabel.CENTER);
63 jpDays.add(jlblDay[i]); // jpDays'e ekle
64 } n 65 }
66
67 /** Günleri göster */
68 herkese açık voidt showDays() {
69 jpDays.removeAll(); // jpDays'den tüm etiketleri kaldır
70

```

```

71 showDayNames () ; // Gün adlarını göster
72
73 // Bir ayın ilk gününün gününü al
74 intt başlangıçDayOfMonth = takvim.get(Calendar.DAY_OF_WEEK);
75
76 // Bu aydan önceki günlerle takvimi doldur
77 Calendar cloneCalendar = (Calendar)calendar.clone();
78 cloneCalendar.add(Calendar.DATE, -1); // Önceki ay olur
79 intt günlerInPrecedingMonth = cloneCalendar.getActualMaximum(
80 Takvim.DAY_OF_MONTH);
81
82 for (tintt i = t 0t; i < AyBaşlangıçGünü - t 1t; i++) {
83 jlblDay[i + t 7t].setForeground(Color.LIGHT_GRAY);n 84 jlblDay[i + t
7t].setText(daysInPrecedingMonth - n 85 startupDayOfMonth + t 2
t + i + t ""t);
86 jpDays.add(jlblDay[i + 7]); // jpDays'e ekle
87 }
88
89 // Bu ayın günlerini göster
90 int günlerBuAydaki = takvim.getActualMaximum(
91 Takvim.DAY_OF_MONTH);
92 for (int i = 1; i <= günlerBuAydaki; i++) {
93 jlblGun[i - 2 + ayınBaşlangıçGünü + 7] .
94 setForeground(Color.black);
95 jlblGun[i - 2 + ayınBaşlangıçGünü + 7] .setText(i + " ");
96 jpGunler.add(jlblGun[i - 2 + ayınBaşlangıçGünü + 7]);
97 }
98
99 // Bu aydan sonraki günlerle takvimi doldurun
100 intt j = t 1t;
101 for (tintt i = Şu Anki Aydaki günler - t 1t + Ayın Başlangıç Günü + t 7t;
102 i % t 7t != t 0t; i++) { n103 jlblDay[i
].setForeground(Color.LIGHT_GRAY); n104 jlblDay[i].setText(j++ + t
""t);
105 jpDays.add(jlblDay[i]); // jpDays'e ekle
106 }
107
108 jpDays.repaint () ; // jpDays'deki etiketleri yeniden boyayın
109 }
110
111 /** Takvimi belirtilen ay ve yılın ilk gününe ayarlayın
* */
112 özel geçersiz güncelleme Takvim() {
113 takvim.set(Calendar.YEAR, yıl);
114 takvim.set(Calendar.MONTH, ay);
115 takvim.set(Calendar.DATE, 1) ;
116 }
117
118 /** Ayı döndür */
119 public intt getMonth() {
120 return ay;
121 }
122
123 /** Yeni bir ay ayarla */
124 public void setMonth (int yeniAy) {
125 ay = yeniAy;
126 takvimiGüncelle();
127 başlığıGöster();
128 }

```

```

129 günleriGöster();
130 }
131
132 /** Yılı döndür */
133 public intt getYear() {
134 return yıl;
135 }
136
137 /** Yeni bir yıl ayarla */
138 public void setYear (int yeniYıl) {
139 yıl = yeniYıl;
140 takvimiGüncelle();
141 başlığıGöster();
142 günleriGöster();
143 }
144
145 /** Yeni bir yerel ayar belirle */
146 public void changeLocale(Locale yeniYerel) {
147 setLocale(yeniYerel);
148 başlığıGöster();
149 günleriGöster();
150 }
151 }

```

**TakvimPaneli** takvimi kontrol etmek ve görüntülemek için oluşturulmuştur. Başlıkta ay ve yılı, takvim gövdesinde ise gün adlarını ve günleri gösterir. Başlık ve gün adları yerel duyarlıdır.

#### <kenar notu: t showHeader>

showHeader metodu (satırlar 47–52) takvim başlığını "MMMM yyyy" biçiminde gösterir. SimpleDateFormat sınıfı, showHeader metodunda kullanılan bir DateFormat alt sınıfıdır. SimpleDateFormat tarih formatını özelleştirmenizi sağlar farklı standart dışı stillerde tarihi görüntülemek için.

#### <kenar notu: t showDayNamest>

showDayNames metodu (satırlar 55–65) takvimde gün adlarını gösterir. DateFormatSymbols sınıfı, showDayNames metodunda kullanılan, yerelleştirilebilir tarih-saat formatlama verilerini kapsülleyen bir sınıftır; bu veriler arasında ay adları, haftanın gün adları ve zaman dilimi verileri bulunur. getWeekdays metodu, gün adlarının bir dizisini almak için kullanılır.

#### <kenar notu: t showDayst>

showDays metodu (satırlar 68–109) belirtilen ayın günlerini yıl için gösterir. Şekil 35.6'da görebileceğiniz gibi, mevcut aydan önceki etiketler, önceki ayın son birkaç günü ile doldurulmuştur ve mevcut aydan sonraki etiketler, bir sonraki ayın ilk birkaç günü ile doldurulmuştur.

Mevcut aydan önceki günleri takvime doldurmak için, önceki ayın günlerini almak üzere *calendar* adlı bir klon oluşturulur, bu klona *cloneCalendar* denir (satır 77). *cloneCalendar* ayrı bellek alanına sahip *calendar* nesnesinin bir kopyasıdır. Böylece *cloneCalendar* nesnesinin özellikle rini değiştirerek *calendar* nesnesini bozmadan değiştirebilirsiniz. *clone()* metodu, §15.7'de tanımlanan *Object* sınıfında tanımlanmıştır, "*Cloneable* Arayüzü." Tanımlayıcı sınıfı *Cloneable* arayüzünü uyguladığı sürece herhangi bir nesneyi klonlayabilirsiniz. *Calendar* sınıfı *Cloneable* arayüzünü uygular.

*cloneCalendar.getActualMaximum(Calendar.DAY\_OF\_MONTH)* metodu (satırlar 90–91) belirtilen takvim için ay içindeki gün sayısını döndürür.

### Liste 35.5 CalendarApp.java

<margin note line 9: takvim paneli>  
 <margin note line 12: kombinasyon kutusu>  
 <margin note line 15: yerel ayarlar>  
 <margin note line 23: arayüz oluşturun>  
 <margin note line 50: yeni bir yerel ayar belirle>  
 <margin note line 60: önceki ay>  
 <margin note line 69: sonraki ay>  
 <margin note line 75: ana metod atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.border.*'i içe aktarın;
5 java.util.*'i içe aktarın;
6
7 genel sınıf TakvimApp t extendst JApplet {
8 // Takvimleri göstermek için bir CalendarPanel oluşturun
9 özel TakvimPanel takvimPanel = t newt TakvimPanel();
10
11 // Mevcut yerelleri seçmek için combo kutusu
12 özel JComboBox jcboLocale = t newt JComboBox();
13
14 // Mevcut yerelleri saklamak için yerel değişkenleri tanımlayın
15 özel Yerel Ayar yerel ayarları[] = Calendar.get Kullanılabilir Yerel Ayarlar();
16
17 // Önceki ve sonraki ayı görüntülemek için Önceki ve Sonraki düğmeleri
18 özel JButton jbl Önceki = t newt JButton(t"Önceki"t);
19 özel JButton jbtNext = yeni JButton("Sonraki");
20
21 /** Uygulamayı başlat */
22 genel void init() {
23 // Yerel seçimleri tutacak jpLocale paneli
24 JPanel jpLocale = yeni JPanel(yeni FlowLayout());
25 jpLocale.setBorder(yeni TitledBorder("Bir yerel seçin"));
26 jpLocale.add(jcboLocale);
27
28 // Yerel adları eklemek için combo kutusunu başlat
29 için (int i = 0; i < locales.length; i++)
30 jcboLocale.addItem(locales[i].getDisplayname());
31
32 // Düğmeleri tutacak jpButtons paneli
33 JPanel jpButtons = yeni JPanel(yeni FlowLayout());
34 jpButtons.add(jbtPrior);
35 Buttons.and(text);
36
37 // TakvimPaneli ve butonları tutmak için jpCalendar paneli
38 JPanel jpCalendar = t newt JPanel(tnewt BorderLayout()); n 39 jpCalendar.add(calendarPanel, BorderLayout.CENTER); n 40 jpCalendar.add(jpButtons, BorderLayout.SOUTH);
39
40 // jpCalendar ve jpLocale'yi applete yerleştir
41 add(jpCalendar, BorderLayout.CENTER); n 44 add(jpLocale, BorderLayout.SOUTH);
42
43 // Dinleyicileri kaydet
44 jcboLocale.addActionListener(tnewt ActionListener() { n 48 @Override
45 genel boşluk actionPerformed(ActionEvent e) {

```

```

50 if (e.getSource() == jcbLocale)
51 takvimPanel.changeLocale(n 52 yerel ayar[jcb
oLocale.getSelectedIndex()]); n 53 }

54 });
55
56 jbtPrior.addActionListener(new ActionListener() { n 57 @Override
e
58 genel boşluk actionPerformed(ActionEvent e) {
59 int mevcutAy = takvimPanel.getAy();
60 eğer (mevcutAy == 0) // Önceki ay Aralık için 11'dir
61 takvimPanel.setYıl(takvimPanel.getYıl() - 1) ;
62 takvimPanel.setAy((mevcutAy - 1) % 12) ;
63 });
64
65 jbtSonraki.addActionListener(new ActionListener() {
66 @Geçersiz Kıl
67 genel boşluk actionPerformed(ActionEvent e) {
68 int mevcutAy = takvimPanel.getAy();
69 eğer (mevcutAy == 11) // Sonraki ay Ocak için 0'dır
70 takvimPanel.setYıl(takvimPanel.getYıl() + 1) ;
71
72 takvimPanel.setMonth((currentMonth + t 1t) % t 12t);n 73 });
74
75 takvimPanel.changeLocale(n 76 yerel a
yar[jcbLocale.getSelectedIndex()]); n 77 }

78 }

```

`CalendarApp` kullanıcı arayüzünü oluşturur ve buton eylemlerini ve combo kutusu öge seçimleri yereller için yönetir. `Calendar.getAvailableLocales()` metodu (satır 15) takvimleri olan tüm mevcut yerelleri bulmak için kullanılır. `getDisplayNames()` metodu her bir yereli `n` adını döndürür ve adı combo kutusuna ekler (satır 30). Kullanıcı combo kutusunda bir yerel adı seçtiğinde, yeni bir yerel `calendarPanel`'ye iletilir ve yeni bir takvim yeni yerel temelinde görüntülenir (satırlar 72–73).

### 35.4 Sayıları Biçimlendirme

Sayıları biçimlendirme, yerel ayara oldukça bağlıdır. Örneğin, 5000.555 sayısı Amerika Birleşik Devletleri'nde 5,000.555 olarak, Fransa'da 5 000,555 olarak ve

Almanya'da 5.000,555 olarak görüntülenir.

Sayılar, biçimlendirme ve ayrıştırma yöntemlerini sağlayan soyut bir temel sınıf olan `java.text.NumberFormat` sınıfı kullanılarak biçimlendirilir; bu, Şekil 35.8'de gösterilmiştir.

<i>java.text.NumberFormat</i>	
+getInstance(): NumberFormat	Varsayılan yerel için varsayılan bir sayı formatı döndürür.
+getInstance(locale: Locale): NumberFormat	Belirtilen yerel için varsayılan bir sayı formatı döndürür.
+getIntegerInstance(): NumberFormat	Varsayılan yerel için bir tam sayı formatı döndürür.
+getIntegerInstance(locale: Locale): NumberFormat	Belirtilen yerel için bir tam sayı formatı döndürür.
+getParaÖrneği(): SayıFormatı	Mevcut varsayılan yerel için bir para birimi formatı döndürür.
+getSayıÖrneği(): SayıFormatı	getInstance() ile aynı.
+getSayıÖrneği(locale: Locale): SayıFormatı	getInstance(locale) ile aynı.
+getYüzdeÖrneği(): SayıFormatı	Varsayılan yerel için bir yüzde formatı döndürür.
+getYüzdeÖrneği(locale: Locale): SayıFormatı	Belirtilen yerel için bir yüzde formatı döndürür.
+format(sayı: double): String	Bir kayan noktalı sayıyı biçimlendirir.
+format(sayı: long): String	Bir tam sayıyı biçimlendirir.
+getMaksimumKesirHaneleri(): int	İzin verilen maksimum kesir basamağı sayısını döndürür.
+setMaksimumKesirHaneleri(yeniDeğer: int): geçersiz	İzin verilen maksimum kesir basamağı sayısını ayarlar.
+getMinimumKesirHaneleri(): int	İzin verilen minimum kesir basamağı sayısını döndürür.
+setMinimumKesirHaneleri(yeniDeğer: int): geçersiz	İzin verilen minimum kesir basamağı sayısını ayarlar.
+getMaksimumTamHaneleri(): int	Bir kesir sayısındaki izin verilen maksimum tam sayı basamağı sayısını döndürür.
+setMaksimumTamHaneleri(yeniDeğer: int): geçersiz	Bir kesir sayısındaki izin verilen maksimum tam sayı basamağı sayısını ayarlar.
+getMinimumTamHaneleri(): int	Bir kesir sayısındaki izin verilen minimum tam sayı basamağı sayısını döndürür.
+setMinimumTamHaneleri(yeniDeğer: int): geçersiz	Bir kesir sayısındaki izin verilen minimum tam sayı basamağı sayısını ayarlar.
+gruplamaKullanılıyorMu(): boolean	Bu biçimde gruplamanın kullanılıp kullanılmadığını belirtir. Örneğin, İngilizce yerel ayarında, gruplama açıkken, 1234567 sayısı "1,234,567" olarak biçimlendirilir.
+setGruplamaKullanılıyorMu(yeniDeğer: boolean): void	Bu formatta gruplamanın kullanılıp kullanılmayacağını ayarlar.
+parse(kaynak: String): Sayı	Dizgiyi bir sayıya ayrıştırır.
+getAvailableLocales(): Yerel ayar[]	Sayı Formatlarının yüklü olduğu yerel ayarların kümesini alır.

### Şekil 35.8

*NumberFormat sınıfı, sayıları biçimlendirme ve ayrıştırma yöntemlerini sağlar.*

NumberFormat ile, herhangi bir yerel ayar için sayıları biçimlendirebilir ve ayrıştırabilirsiniz. Kodunuz, ondalık noktalar, bin ayırıcılar, para birimi biçimi ve yüzde biçimleri için yerel ayar geleceklerinden tamamen bağımsız olacaktır.

#### 35.4.1 Düz Sayı Biçimi

Mevcut yerel ayar için NumberFormat örneğini NumberFormat.getInstance() veya NumberFormat.getNumberInstance() ile alabilir ve belirtilen yerel ayar için NumberFormat.getInstance(Locale) ile alabilirsiniz.

[SayıFormat.getNumberInstance\(Locale\)](#). Ardından *format(number)* metodunu SayıFormat örneği üzerinde çağırarak biçimlendirilmiş bir sayıyı dize olarak döndürebilirsiniz.

Örneğin, Fransa'da 5000.555 sayısını görüntülemek için aşağıdaki kodu kullanın:

```
SayıFormat sayıFormat = SayıFormat.getInstance(Locale.FRANCE);
System.out.println(sayıFormat.format(5000.555));
```



Sayıların görüntülenmesini `setMaximumFractionDigits` ve `setMinimumFractionDigits` gibi yöntemlerle kontrol edebilirsiniz. Örneğin, 5000.555 `numberFormat.setMaximumFractionDigits(1)` kullanırsanız 5000.6 olarak görüntülenecektir.

#### 35.4.2 Para Birimi Biçimi

Bir sayıyı para birimi değeri olarak biçimlendirmek için `SayıFormat.getCurrencyInstance()` kullanarak mevcut yerel için para birimi sayı formatını alabilirsiniz veya `SayıFormat.getCurrencyInstance(Locale)` belirtilen yerel için para birimi sayısını almak için kullanabilirsiniz.

Örneğin, 5000.555 sayısını Amerika Birleşik Devletleri'nde para birimi olarak görüntülemek için aşağıdaki kodu kullanın:

```
SayıFormat paraBirimiFormat =
 SayıFormat.getCurrencyInstance(Locale.US);
System.out.println(paraBirimiFormat.format(5000.555));
```

5000.555, \$5,000.56 olarak biçimlendirilir. Eğer yerel Fransa olarak ayarlanmışsa, sayı 5 000,56 € olarak biçimlendirilecektir.

#### 35.4.3 Yüzde Formatı

Bir sayıyı yüzde formatında biçimlendirmek için `NumberFormat.getPercentInstance()` veya `NumberFormat.getPercentInstance(Locale)` kullanarak mevcut yerel ayar veya belirtilen yerel ayar için yüzde sayı formatını alabilirsiniz.

Örneğin, 0.555367 sayısını Amerika Birleşik Devletleri'nde yüzde olarak göstermek için aşağıdaki kodu kullanın:

```
NumberFormat yüzdeFormat =
 NumberFormat.getPercentInstance(Locale.US);
System.out.println(yüzdeFormat.format(0.555367));
```

0.555367, %56 olarak biçimlendirilir. Varsayılan olarak, format, bir yüzde sayısındaki kesir kısmını keser. Ondalık

noktasından sonra üç basamağı korumak istiyorsanız, `percentFormat.setMinimumFractionDigits(3)` kullanın. Böylece 0.555367 %55.537 olarak gösterilecektir.

#### 35.4.4 Sayıları Ayırıştırma

Bir sayıyı `format(numericalValue)` yöntemiyle bir dizeye biçimlendirebilirsiniz. Ayrıca, belirli bir yerel ayarın kurallarıyla biçimlendirilmiş düz bir sayı, para birimi değeri veya yüzde sayısını `java.lang.Number` örneğine dönüştürmek için `parse(String)` yöntemini de kullanabilirsiniz. `parse` yöntemi, ayırıştırma başarısız olursa bir `java.text.ParseException` fırlatır. Örneğin, ABD \$5,000.56, aşağıdaki ifadelerle bir sayıya ayırıştırılabilir:

```
NumberFormat paraFormat =
 NumberFormat.getCurrencyInstance(Locale.US);
try {
 Number number = paraFormat.parse("$5,000.56");
 System.out.println(number.doubleValue());
}
catch (java.text.ParseException ex) {
 System.out.println("Parsel başarısız");
}
```

#### 35.4.5 DecimalFormat Sınıfı

Format veya ayrıştırma üzerinde daha fazla kontrol istiyorsanız, `NumberFormat` fabrika yöntemlerinden aldığınız değeri `java.text.DecimalFormat`'ye dönüştürün, bu da `NumberFormat`'nın bir alt sınıfıdır. Ardından, `DecimalFormat` sınıfının `applyPattern(String pattern)` yöntemini kullanarak

numarayı görüntülemek için desenleri belirtebilirsiniz.

Bir desen, ondalık noktadan önceki minimum rakam sayısını yerine getirebilir ve ondalık noktadan sonraki maksimum rakam sayısını belirtebilir. Karakterler `'0'` ve `'#'` zorunlu bir rakam ve isteğe bağlı bir rakamı belirtmek için kullanılır, sırasıyla. İsteğe bağlı rakam sıfırda görüntülenmez. Örneğin, desen `"00.0##"` ondalık noktadan önce minimum iki rakam ve ondalık noktadan sonra maksimum üç rakam belirtir. Eğer ondalık noktadan önce daha fazla gerçek rakam varsa, tüm bunlar görüntülenir. Eğer ondalık noktadan sonra üçten fazla rakam varsa, rakam sayısı yuvarlanır.

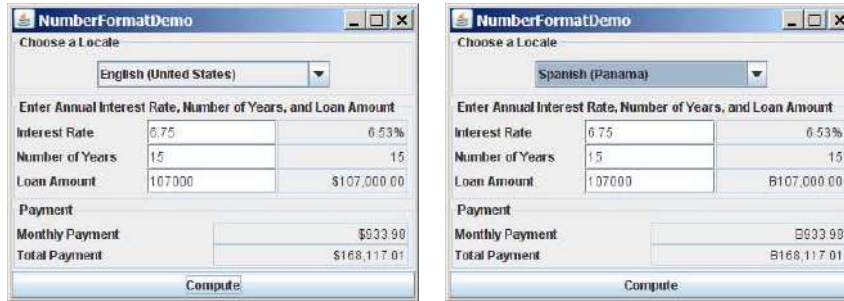
Deseni uygulama `"00.0##"`, sayı `111.2226` `111.223`, sayı `111.2226` `111.223`, sayı `1.22` `01.22`, ve sayı `1` `01.0`. İşte kod:

```
NumberFormat numberFormat = NumberFormat.getInstance(Locale.US);
DecimalFormat decimalFormat = (DecimalFormat)numberFormat;
decimalFormat.applyPattern("00.0##");
System.out.println(decimalFormat.format(111.2226));
System.out.println(decimalFormat.format(111.2226));
System.out.println(decimalFormat.format(1.22));
System.out.println(decimalFormat.format(1));
```

Karakter `'%'` bir deseni sonuna eklenerek, bir sayının yüzde olarak biçimlendirildiğini belirtmek için kullanılabilir. Bu, sayının `100` ile çarpılmasına ve bir yüzde işareti `%` eklenmesine neden olur.

#### 35.4.5 Örnek: Sayıları Biçimlendirme

Bir kredi hesaplayıcısı oluşturun. Hesaplayıcı, kullanıcının lokasyonları seçmesine olanak tanır ve sayıları lokasyona duyarlı bir biçimde görüntüler. Şekil 35.9'da gösterildiği gibi, kullanıcı faiz oranını, yıl sayısını ve kredi miktarını girer, ardından *Hesapla* butonuna tıklayarak faiz oranını yüzde biçiminde, yıl sayısını normal sayı biçiminde ve kredi miktarını, toplam ödemeyi ve aylık ödemeyi para birimi biçiminde görüntüler. Liste 35.6, sorunun çözümünü vermektedir.



Şekil 35.9

Yerel ayar, kredi

hesaplayıcısında görüntülenen sayıların formatını belirler.

#### Liste 35.6 NumberFormatDemo.java

<kenar notu satır 10: UI bileşenleri>

<kenar notu satır 42: UI oluştur>  
 <kenar notu satır 99: dinleyici kaydet>  
 <kenar notu satır 101: yeni yerel ayar>  
 <kenar notu satır 102: kredi hesapla>  
 <kenar notu satır 106: dinleyici kaydet>  
 <kenar notu satır 114: kredi hesapla>  
 <kenar notu satır 145: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.border.*'ı içe aktarın;
5 java.util.*'i içe aktarın;
6 java.text.NumberFormat'ı içe aktarın;
7
8 genel sınıf NumberFormatDemo t extendst JApplet {
9 // Mevcut yerelleri seçmek için combo kutusu
10 özel JComboBox jcboLocale = t newt JComboBox();
11
12 // Faiz oranı, yıl ve kredi miktarı için metin alanları
13 özelt JTextField jtfInterestRate = t newt JTextField(t"6.75"t);
14 özelt JTextField jtf Yıl Sayısı = t newt JTextField(t"15"t);
15 özelt JTextField jtfLoanAmount = t newt JTextField(t"107000"t);
16 özelt JTextField jtfFormattedInterestRate = t newt JTextField(t10t);
17 özelt JTextField jt FormattedYıl Sayısı = t newt JTextField(t10t);
18 özelt JTextField jtfFormattedLoanAmount = t newt JTextField(t10t);
19
20 // Aylık ödeme ve toplam ödeme için metin alanları
21 özelt JTextField jtf Toplam Ödeme = t newt JTextField();
22 özel JTextField jtfMonthlyPayment = yeni JTextField();
23
24 // Hesapla butonu
25 özel JButton jbtCompute = yeni JButton("Hesapla");
26
27 // Mevcut yerel ayar
28 özel Locale locale = Locale.getDefault();
29
30 // Mevcut yerelleri saklamak için yerel değişkenleri tanımlayın
31 özel Yerel Ayar yerel ayarları[] = Calendar.get Kullanılabilir Yerel Ayarlar();
32
33 /** Kombinasyon kutusunu başlat */
34 public void ComboBox()'ı başlat {
35 // Yerel adları kombinasyon kutusuna ekle
36 için (int i = 0; i < locales.length; i++)
37 jcboLocale.addItem(locales[i].getDisplayName()); n 38 }
38
39 /** Uygulamayı başlat */
40 genel void init() {
41 // Yerel ayarları seçmek için kombinasyon kutusunu tutacak panel p1
42 JPanel p1 = t newt JPanel(); n 44. p1.set
43 Layout(new FlowLayout()); n 45 p1.add(jcb Yerel A
44 yar);
45 initializeComboBox();
46 p1.setBorder(new TitledBorder("Bir Yerel Seçin"));
47
48 // Girdi tutmak için panel p2
49 JPanel p2 = t yeni JPanel();

```

```

51 p2.setLayout(new GridLayout(3, 3));
52 p2.add(new JLabel("Faiz Oranı"));
53 p2.add(jtfInterestRate);
54 p2.add(jtfFormattedInterestRate);
55 p2.add(new JLabel("Yıl Sayısı"));
56 p2.add(jtfNumberOfYears);
57 p2.add(jtfFormattedNumberOfYears);
58 p2.add(new JLabel("Kredi Miktarı"));
59 p2.add(jtfLoanAmount);
60 p2.add(jtfFormattedLoanAmount);
61 p2.setBorder(new TitledBorder("Yıllık Faiz Oranını Girin, " +
62 "Yıl Sayısı ve Kredi Miktarı"));
63
64 // Sonucu tutacak panel p3
65 JPanel p3 = new JPanel();
66 p3.setLayout(new GridLayout(2, 2));
67 p3.setBorder(new TitledBorder("Ödeme"));
68 p3.add(new JLabel("Aylık Ödeme"));
69 p3.add(jtfMonthlyPayment);
70 p3.add(new JLabel("Toplam Ödeme"));
71 p3.add(jtfTotalPayment);
72
73 // Metin alanı hizalamasını ayarla
74 jtfFormattedInterestRate.setHorizontalAlignment(JTextField.RIGHT); n 75 jtfFormattedNum
75 berOfYears.setHorizontalAlignment(JTextField.RIGHT); n 76 jtfFormattedLoanAmount.setHorizo
76 ntalAlignment(JTextField.RIGHT); n 77 jtfTotalPayment.setHorizontalAlignment(JTextField.RI
77 GHT);
78 jtfMonthlyPayment.setHorizontalAlignment(JTextField.RIGHT); n 79
79
80 // Düzenlenemez olarak ayarla
81 jtfFormattedInterestRate.setEditable(false); n 82 jtfFormatted
82 NumberOfYears.setEditable(false); n 83 jtfFormattedLoanAmount.
83 setEditable(false); n 84 jtfTotalPayment.setEditable(false);
84
85 jtfMonthlyPayment.setEditable(false);
86
87 // Sonuç ödemelerini tutacak panel p4 ve bir buton
88 JPanel p4 = new JPanel(); n 89 p4.setL
89 ayout(new BorderLayout()); n 90 p4.add(p3, Borde
90 rLayout.CENTER); n 91 p4.add(jbtCompute, Border
91 Layout.SOUTH);
92
93 // Panelleri applete yerleştir
94 add(p1, BorderLayout.NORTH); n 95 add(p
95 2, BorderLayout.CENTER); n 96 add(p4, Border
96 Layout.SOUTH);
97
98 // Dinleyicileri kaydet
99 jcboLocale.addActionListener(new ActionListener() { n100 @Override
100
101 genel boşluk actionPerformed(ActionEvent e) {
102 yerel ayar = yerel ayarlar[jcboLocale.getSelectedIndex()]; n103 hesaplam
103 aLoan();
104 } n105 });
105
106
107 jbtCompute.addActionListener(new ActionListener() {
108 @Geçersiz kıl

```

```

109 genel boşluk actionPerformed(ActionEvent e) {
110 hesaplamaLoan(); n111 }

112}); n113 }

114
115 /** Ödemeleri hesapla ve sonuçları yerel duyarlı formatta göster */
116 özel void computeLoan() {
117 // Kullanıcıdan girdi al
118 doublet kredi = t newt Double(jtfLoanAmount.getText()).doubleValue();
119 ikili faiz Oranı =
120 newt Double(jtfInterestRate.getText()).doubleValue() / t 1200t;
121 inttYıl Sayısı =
122 newt Integer(jtfNumberOfYears.getText()).intValue();
123
124 // Ödemeleri hesapla
125 aylık iki katı Ödeme = Kredi * Faiz Oranı/
126 (t1t - (Math.pow(t1t / (t1t + faizOranı), numberOfYears * t 12t)));
127 double toplamÖdeme = aylıkÖdeme * yılSayısı * 12;
128
129 // Formatlayıcıları al
130 NumberFormat yüzdeFormatter = n131 Numb
131 erFormat.getPercentInstance(locale); n132 NumberFormat para
132 birimiFormu =
133 NumberFormat.getCurrencyInstance(yerel); n134 NumberFor
134 mat numberForm = NumberFormat.getNumberInstance(locale); n135 yüzdeFormatter.setMin
135 imumFractionDigits(t2t);
136
137 // Biçimlendirilmiş girişi göster
138 jtfFormattedInterestRate.setText(n139 yüzdeFor
139 matter.format(interestRate * t 12t));n140 jtfFormattedNumberOfYear
140 s.setText
141 (numberForm.format(numberOfYears)); n142 jtfForm
142 attedLoanAmount.setText(currencyForm.format(kredi));
143
144 // Sonuçları para birimi formatında göster
145 jtfMonthlyPayment.setText(currencyForm.format(monthlyPayment)); n146 jtfTotalPay
146 ment.setText(currencyForm.format(totalPayment)); n147 }

148 }

```

computeLoan metodu (satırlar 114–145) kullanıcıdan faiz oranı, yıl sayısı ve kredi miktarını alır, aylık ödemeyi ve toplam ödemeyi hesaplar ve yıllık faiz oranını yüzde formatında, yıl sayısını normal sayı formatında ve kredi miktarını, aylık ödemeyi ve toplam ödemeyi yerel duyarlı formatta gösterir.

İfade `percentFormatter.setMinimumFractionDigits(2)` (satır 133) kesirli kısımların minimum sayısını 2 olarak ayarlar. Bu ifade olmadan, 0.075 % olarak değil, 7.5% olarak görüntülenir.

### 35.5 Kaynak Paketleri

Önceki örnekteki NumberFormatDemo, sayıları, para birimlerini ve yüzdeleri yerel geleneklere göre gösterirken, tüm mesaj dizelerini, başlıkları ve buton etiketlerini İngilizce olarak gösterir. Bu bölümde, mesaj dizelerini, başlıkları, butonları yerelleştirmek için kaynak paketlerini nasıl kullanacağınızı öğreneceksiniz. Etiketler, vb.

### <kenar notu: kaynak paketi>

A *kaynak paketi* bir Java sınıf dosyası veya metin dosyasıdır ve yerel-specifik bilgileri sağlar. Bu bilgilere Java programları tarafından dinamik olarak erişilebilir. Yerel-specifik bir kaynağa gerektiğinde—bir mesaj dizesi, örneğin—programınız, istenen yerel için uygun kaynak paketinden yükleyebilir. Bu şekilde, program kodunuzu kullanıcının yerelinden büyük ölçüde bağımsız olacak şekilde yazabilirsiniz, çoğu, eğer hepsi değilse, yerel-specifik bilgileri kaynak paketlerinde izole edebilirsiniz.

Kaynak paketleri ile, kodunuzun yerel duyarlı kısmını yerel bağımsız kısmından ayıran programlar yazabilirsiniz. Programlar, birden fazla yereli kolayca işleyebilir ve daha sonra daha fazla yereli desteklemek için kolayca değiştirilebilir.

### Kaynaklar, *ResourceBundle*

sınıfını veya *ResourceBundle* sınıfının bir alt sınıfını genişleten sınıfların içine yerleştirilmiştir. Kaynak paketleri, *anahtar/değer* çiftlerini içerir. Her anahtar, paketteki yerel belirli bir nesneyi benzersiz bir şekilde tanımlar. Objeyi almak için anahtarı kullanabilirsiniz. *ListResourceBundle* kaynak paketlerinin oluşturulmasını basitleştirmek için sıklıkla kullanılan bir alt sınıftır. İşte *ListResourceBundle* kullanan dört anahtar içeren bir kaynak paketi örneği:

```
// MyResource.java: kaynak dosyası
genel sınıf MyResource t java.util.List ResourceBundle'ı genişletir { nstatic final
 t Object[][] içerikler = {
 {t"nationalFlag",t "us.gif"},n {t"nationalAnthem",t "us.au"},n {t"nationalColor",t, Color.red}, n {t"annualGrowthRate",t, new Double(t7.8t)}n };

 genel Nesne[][] getContents() { nreturn içerikler;
 }
}
```

Anahtarlar büyük/küçük harf duyarlı dizelerdir. Bu örnekte, anahtarlar *nationalFlag*, *nationalAnthem*, *nationalColor* ve *annualGrowthRate*'dir. Değerler herhangi bir türde *Object* olabilir.

Tüm kaynaklar dizeler ise, bunlar .properties uzantılı uygun bir metin dosyasına yerleştirilebilir. Tipik bir özellik dosyası şöyle görünür:

```
#Çar Jul 01 07:23:24 EST 1998
nationalFlag=us.gif
ulusal Marş=us.au
```

Bir programda *ResourceBundle* içinden değerleri almak için, öncelikle aşağıdaki iki statik yöntemden birini kullanarak bir *ResourceBundle* örneği oluşturmanız gerekir:

```
public static final ResourceBundle getBundle(String baseName) throws MissingResourceException

public static final ResourceBundle getBundle(String baseName, Locale locale) throws MissingResourceException
```

İlk yöntem, varsayılan yerel için bir `ResourceBundle` döner ve ikinci yöntem, belirtilen yerel için bir `ResourceBundle` döner. `baseName` bir dizi sınıfın temel adıdır; her biri, belirli bir yerel için bilgileri tanımlar. Bu sınıflar Tablo 35.3'te adlandırılmıştır.

*Tablo 35.3:*

*Kaynak Paketi İsimlendirme Kuralları*

1. `BaseName_dil_ülke_variant.class`
2. `BaseName_dil_ülke.class`
3. `BaseName_dil.class`
4. `BaseName.class`
5. `BaseName_dil_ülke_variant.properties`
6. `BaseName_dil_ülke.properties`
7. `BaseName_dil.properties`
8. temel isim.özellikler

Örneğin, `MyResource_en_BR.class` Birleşik Krallığa özgü kaynakları, `MyResource_en_US.class` Amerika Birleşik Devletlerine özgü kaynakları ve `MyResource_en.class` tüm İngilizce konuşan ülkelere özgü kaynakları depolar.

`getBundle` metodu, belirtilen yerel ayara göre dil, ülke ve varyant ile eşleşen sınıfı yüklemeye çalışır ve dosya adlarını Tablo 35.3'te gösterilen sırayla arar. Bu sırada aranan dosyalar bir kaynak zinciri oluşturur. Eğer kaynak zincirinde hiçbir dosya bulunamazsa, `getBundle` metodu `MissingResourceException`, `RuntimeException`'in bir alt sınıfını yükseltir.

Bir kaynak paketi nesnesi oluşturulduktan sonra, anahtara göre değeri almak için `getObject` yöntemini kullanabilirsiniz. Örneğin,

```
ResourceBundle res = ResourceBundle.getBundle("MyResource");
String flagFile = (String)res.getObject("nationalFlag");
String anthemFile = (String)res.getObject("nationalAnthem");
Color color = (Color)res.getObject("nationalColor");
doublelet büyümeOranı = (Double)res.getObject("yillikBüyümeOranı");
int TIP
```

Eğer kaynak değeri bir dize ise, `getString` yöntemi `getObject` yönteminin yerini almak için kullanılabilir. `getString` yöntemi, `getObject` tarafından döndürülen değeri basitçe bir dizeye dönüştürür.

Aradığınız bir kaynak nesnesi kaynak paketinde tanımlı değilse ne olur? Java, nesneyi kaynak zincirindeki üst dosyada arayan akıllı bir arama şeması kullanır. Bu arama, nesne bulunana kadar veya kaynak zincirindeki tüm üst dosyalar aranana kadar tekrarlanır. Arama başarısız olursa bir `MissingResourceException` oluşturulur.

Önceki örnekteki `NumberFormatDemo` programını, çoklu dillerde mesajlar, başlık ve buton etiketleri gösterecek şekilde değiştirelim; Şekil 35.10'da gösterildiği gibi.



**Şekil 35.10**

*Program, birden fazla dildeki dizeleri görüntüler.*

Her dil için bir kaynak paketi sağlamanız gerekir. Programın üç dili desteklediğini varsayalım: İngilizce (varsayılan), Çince ve Fransızca. İngilizce dili için kaynak paketi, `MyResource.properties` adıyla aşağıdaki gibi verilmiştir:

```
#MyResource.properties İngilizce dili için
Yil_Sayısı=Yillar
Toplam_Ödeme=Fransızca Toplam\ Ödeme
Faiz_Oranı_Girin=Faiz\ Oranı,\ Yillar,\ ve\ Kredi\ Miktarı\ Girin
Ödeme=Ödeme
Hesapla=Hesapla
Yıllık_Faiz_Oranı=Faiz\ Oranı
Sayı_Biçimlendirme=Sayı\ Biçimlendirme\ Demo
Kredi_Miktarı=Kredi\ Miktarı
Bir_Bölge_Seçin=Bir\ Bölge\ Seçin
Aylık_Ödeme=Aylık\ Ödeme
```

Çince dili için kaynak paketi, `MyResource_zh.properties` adıyla aşağıdaki gibi verilmiştir:

```
#MyResource_zh.properties için Çince dil
Choose_a_Locale = Ülke Seçin
Enter_Interest_Rate =
 Faiz oranını, yıl sayısını, kredi toplamını
Annual_Interest_Rate = Faiz Oranı
Number_Of_Years = Yıl Sayısı
Loan_Amount = Kredi Miktarı
Payment = Ödeme
Monthly_Payment = Aylık Ödeme
Total_Payment = Toplam Ödeme
Hesapla = \u8a08\u7b97\u8cb8\u6b3e\u5229\u606f
```

Fransızca için kaynak paketi, `MyResource_fr.properties` olarak adlandırılmıştır, aşağıdaki gibidir:

```
#MyResource_fr.properties Fransızca için
Yil_Sayısı=annees
Yıllık_İlgi_Oranı=le taux d'interet
Kredi_Tutarı=Le montant du pret
İlgi_Oranı_Girin=inscrire le taux d'interet, les anneess, et le montant du
pret
Ödeme=paiement
Hesapla=Calculer l'hypothèque
```



Sayı\_Biçimlendirme=demonstration du formatting des chiffres  
Yerel\_Birimi\_Seç=Choisir la localite  
Aylık\_Ödeme=versement mensuel  
Toplam\_Ödeme=reglement total

Kaynak paket dosyası sınıf dizininde yer almalıdır (örneğin, c:ook bu kitapta verilen örnekler için). Program, Liste 35.7'de verilmiştir.

#### Liste 35.7 ResourceBundleDemo.java

<margin note line 11: kaynak al>  
<margin note line 61: arayüz oluşturun>  
<margin note line 120: dinleyici kaydet>  
<margin note line 123: kaynağı güncelle>  
<margin note line 128: dinleyici kaydet>  
<margin note line 169: yeni kaynak>  
<margin note line 192: uygulamadaki res>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.border.*'ı içe aktarın;
5 java.util.*'i içe aktarın;
6 Java.text.NumberFormat'ı içe aktarın;
7
8 genel sınıf ResourceBundle Demosu Applet'i genişletir {
9 // Mevcut yerelleri seçmek için combo kutusu
10 özel JComboBox jcboLocale = t newt JComboBox();
11 Privatet ResourceBundle res = ResourceBundle.getBundle(t"MyResource"t);
12
13 // Etiketleri oluşturun
14 özel Etiket jlblİlgi Oranı =
15 new JLabel(res.getString("Yıllık_İlgi_Oranı"));
16 özel JLabel jlblNumberOfYears =
17 new JLabel(res.getString("Yıl_Sayısı"));
18 özel JLabel jlblLoanAmount =
19 new JLabel(res.getString("Kredi_Tutarı"));
20 özel JLabel jlblMonthlyPayment =
21 new JLabel(res.getString("Aylık_Ödeme"));
22 özel JLabel jlblTotalPayment =
23 new JLabel(res.getString("Toplam_Ödeme"));
24
25 // Başlıklı kenarlıklar oluşturun
26 özel TitledBorder açılan kutu Başlığı =
27 new TitledBorder(res.getString("Bir_Dil_Seçin"));
28 özel TitledBorder girişi Title = t newt TitledBorder
29 (res.getString("Faiz_Oranını_Girin"));
30 özel Başlıklı Sınır ödeme Başlığı =
31 new TitledBorder(res.getString("Ödeme"));
32
33 // Faiz oranı, yıl, kredi miktarı için metin alanları,
34 özelt JTextField jtfInterestRate = t newt JTextField(t"6.75"t);
35 özel JTextField jtf Yıl Sayısı = t newt JTextField(t"15"t);
36 özelt JTextField jtfLoanAmount = t newt JTextField(t"107000"t);
37 özelt JTextField jtfFormattedInterestRate = t newt JTextField(t10t);
38 özel JTextField jt FormattedYıl Sayısı = t newt JTextField(t10t);
39 özelt JTextField jtfFormattedLoanAmount = t newt JTextField(t10t);
40
```

```

41 // Aylık ödeme ve toplam ödeme için metin alanları
42 özel JTextField jtf Toplam Ödeme = t newt JTextField();
43 özel JTextField jtfMonthlyPayment = yeni JTextField();
44
45 // Hesapla butonu
46 private JButton jbtCompute = new JButton(res.getString("Hesapla"));
47
48 // Mevcut yerel ayar
49 özel Locale locale = Locale.getDefault();
50
51 // Mevcut yerelleri saklamak için yerel değişkenleri tanımlayın
52 özel Yerel Ayar yerel ayarları[] = Calendar.get Kullanılabilir Yerel Ayarlar();
53
54 /** Kombinasyon kutusunu başlat */
55 public void ComboBox()'ı başlat {
56 // Yerel adları kombinasyon kutusuna ekle
57 için (int i = 0; i < locales.length; i++)
58 jcboLocale.addItem(locales[i].getDisplayName()); n 59 }

60
61 /** Uygulamayı başlat */
62 genel void init() {
63 // Yerel ayarları seçmek için kombinasyon kutusunu tutacak panel p1
64 JPanel p1 = t newt JPanel(); n 65 p1.set
Layout(tnewt FlowLayout()); n 66 p1.add(jcboLoca
le);
67 initializeComboBox(); n 68 p1.setB
order(comboBoxTitle); n 69

70 // Yıllık faiz oranı,
71 // yıl sayısı ve kredi miktarı için girdi tutacak panel p2
72 JPanel p2 = t yeni JPanel(); n 73 p2.setL
ayout(tnewt GridLayout(t3t,t 3t));n 74 p2.add(jlblIntere
stRate);
75 p2.add(jtfFaiz Oranı); n 76 p2.add(jtfF
ormattedInterestRate); n 77 p2.add(jlblNumberOfY
ears);
78 p2.add(jtfNumberOfYears); n 79 p2.ad
d(jtfFormattedNumberOfYears); n 80 p2.add(jlblLoan
Amount);
81 p2.add(jtfLoanAmount); n 82 p2.ad
d(jtfFormattedLoanAmount); n 83 p2.setBorder(i
nputTitle);
84
85 // Ödeme tutacak panel p3
86 JPanel p3 = t yeni JPanel(); n 87 p3.setL
ayout(tnewt GridLayout(t2t,t 2t));n 88 p3.setBorder(pa
ymentTitle);
89 p3.add(jlblAylıkÖdeme);
90 p3.add(jtfAylıkÖdeme);
91 p3.add(jlblToplamÖdeme);
92 p3.add(jtfToplamÖdeme);
93
94 // Metin alanı hizalamasını ayarla
95 jtfBiçimlendirilmişFaizOranı.setHorizontalAlignment
(JTextField.RIGHT);
96 jtfBiçimlendirilmişYılSayısı.setHorizontalAlignment
(JTextField.RIGHT);

```

```

99 jtfFormattedLoanAmount.setHorizontalAlignment(JTextField.RIGHT); n100 jtfTotalPayme
nt.setHorizontalAlignment(JTextField.RIGHT);
101 jtfMonthlyPayment.setHorizontalAlignment(JTextField.RIGHT);
102
103 // Düzenlenemez olarak ayarla
104 jtfFormattedInterestRate.setEditable(tfalset);n105 jtfFormatted
NumberOfYears.setEditable(tfalset);n106 jtfFormattedLoanAmount.
setEditable(tfalset);n107 jtfTotalPayment.setEditable(tfalset);

108 jtfMonthlyPayment.setEditable(tfalset);
109
110 // Sonuç ödemelerini ve bir düğmeyi tutacak panel p4
111 JPanel p4 = new JPanel();
112 p4.setLayout(new BorderLayout());
113 p4.add(p3, BorderLayout.CENTER);
114 p4.add(jbtCompute, BorderLayout.SOUTH);
115
116 // Panelleri applete yerleştir
117 add(p1, BorderLayout.NORTH); n118 add(
p2, BorderLayout.CENTER); n119 add(p4, Border
Layout.SOUTH);
120
121 // Dinleyicileri kaydet
122 jcbLocale.addActionListener(tnew ActionListener() { n123 @Override

124 genel boşluk actionPerformed(ActionEvent e) {
125 yerel ayar = yerel ayarlar[jcbLocale.getSelectedIndex()]; n126 updateStri
ngs();
127 hesaplamaLoan(); n128 }

129 });
130
131 jbtCompute.addActionListener(tnew ActionListener() { n132 @Override

133 genel boşluk actionPerformed(ActionEvent e) {
134 hesaplamaLoan(); n135 }

136}); n137}

138
139 /** Ödemeleri hesapla ve sonuçları yerel duyarlı formatta göster */
140 özel void computeLoan() {
141 // Kullanıcıdan girdi al
142 doublet kredi = t newt Double(jtfLoanAmount.getText()).doubleValue();
143 ikili faiz Oranı =
144 newt Double(jtfInterestRate.getText()).doubleValue() / t 1200t;
145 inttYıl Sayısı =
146 newt Integer(jtfNumberOfYears.getText()).intValue();
147
148 // Ödemeleri hesapla
149 aylık iki katı Ödeme = Kredi * Faiz Oranı/
150 (1 - (Math.pow(1 / (1 + faizOranı), yılSayısı * 12))) ;
151 double toplamÖdeme = aylıkÖdeme * yılSayısı * 12;
152
153 // Formatlayıcıları al
154 NumberFormat yüzdeBiçimlendirici =
155 NumberFormat.getPercentInstance(locale);
156 NumberFormat paraBiçimi =

```

```

157 NumberFormat.getCurrencyInstance(locale);
158 NumberFormat sayıBiçimi = NumberFormat.getNumberInstance(locale);
159 yüzdeBiçimlendirici.setMinimumFractionDigits(2);
160
161 // Biçimlendirilmiş girişi göster
162 jtfBiçimlendirilmişFaizOranı.setText(
163 yüzdeBiçimlendirici.format(faizOranı * 12));
164 jtfBiçimlendirilmişYılSayısı.setText
165 (sayıBiçimi.format(yılSayısı));
166 jtfBiçimlendirilmişKrediMiktarı.setText(paraBiçimi.format(kredi));
167
168 // Sonuçları para birimi formatında göster
169 jtfAylıkÖdeme.setText(paraBiçimi.format(aylıkÖdeme));
170 jtfToplamÖdeme.setText(paraBiçimi.format(toplamÖdeme));
171 }
172
173 /** Kaynak dizelerini güncelle */
174 özel voidt updateStrings() {
175 res = ResourceBundle.getBundle("MyResource", locale);
176 jlblFaizOranı.setText(res.getString("Yıllık_Faiz_Oranı"));
177 jlblYılSayısı.setText(res.getString("Yıl_Sayısı"));
178 jlblKrediMiktarı.setText(res.getString("Kredi_Miktarı"));
179 jlblTotalPayment.setText(res.getString("Toplam_Ödeme"));
180 jlblMonthlyPayment.setText(res.getString("Aylık_Ödeme"));
181 jbtCompute.setText(res.getString("Hesapla"));
182 comboBoxTitle.setTitle(res.getString("Bir_Locale_Seçin"));
183 inputTitle.setTitle(res.getString("Faiz_Oranını_Girin"));
184 paymentTitle.setTitle(res.getString("Ödeme"));
185
186 // Yeni etiketlerin görüntülendiğinden emin olun
187 yeniden boyama();
188 }
189
190 /** Ana yöntem */
191 public static void main(String[] args) {
192 // Applet'in bir örneğini oluşturun
193 ResourceBundleDemo uygulama = yeni ResourceBundleDemo();
194
195 // Bir kaynak dizesi ile bir çerçeve oluşturun
196 JFrame frame = new JFrame(
197 applet.res.getString("Sayı_Biçimlendirme"));
198
199 // Applet örneğini çerçeveye ekleyin
200 çerçeve.add(applet, BorderLayout.CENTER);
201
202 // init() ve start() çağırısı
203 applet.init();
204 applet.start();
205
206 // Çerçeveyi göster
207 çerçeve.setSize(400, 300);
208 çerçeve.setLocationRelativeTo(null);
209 çerçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
210 çerçeve.setVisible(true);
211 }
212
213 }

```

Özellik kaynak paketleri, .properties uzantısına sahip metin dosyaları olarak uygulanır ve uygulama veya applet için sınıf dosyalarıyla aynı konuma yerleştirilir. [ListResourceBundles](#) Java sınıf dosyaları olarak sağlanır.

Java kaynak kodu kullanılarak uygulandıkları için, yeni ve değiştirilmiş

[ListResourceBundles](#) dağıtım için yeniden derlenmelidir.

[PropertyResourceBundles](#) ile, çeviriler

uygulamaya eklendiğinde veya değiştirildiğinde yeniden derleme gerekmez. Yine de, [ListResourceBundles](#), [PropertyResourceBundles](#)'dan önemli ölçüde daha iyi performans sağlar.

Kaynak paketi bulunamazsa veya kaynak nesnesi kaynak paketinde bulunamazsa,

[MissingResourceException](#) oluşur. Çünkü

[MissingResourceException](#) [RuntimeException](#)'in bir alt sınıfıdır, bu nedenle istisna yakalamayı kodda açıkça yapmanıza gerek yoktur.

Bu örnek, Listing 35.6, NumberFormatDemo.java ile aynıdır, tek fark,

programın kaynak dizgileri işlemek için kod içermesidir. [updateString](#)

metodu (satırlar 172-186), yerel duyarlı

dizgileri görüntülemekten sorumludur. Bu metod, bir yeni yerel seçildiğinde

combo kutusunda çağrılır. ResourceBundle sınıfının res değişkeni, ResourceBundleDemo'da bir örnek

değişkenidir, bu nedenle main metodunda doğrudan kullanılamaz,

bu nedenle [main](#) metodu statiktir. Sorunu çözmek için, [ResourceBundleDemo](#)'nun bir

örneği olarak [applet](#) oluşturun ve ardından [res](#)

'yi [applet.res](#) kullanarak referans alabileceksiniz.

### 35.6 Karakter Kodlaması

Java programları Unicode kullanır. Bir karakteri metin G/Ç ile okuduğunuzda, karakterin

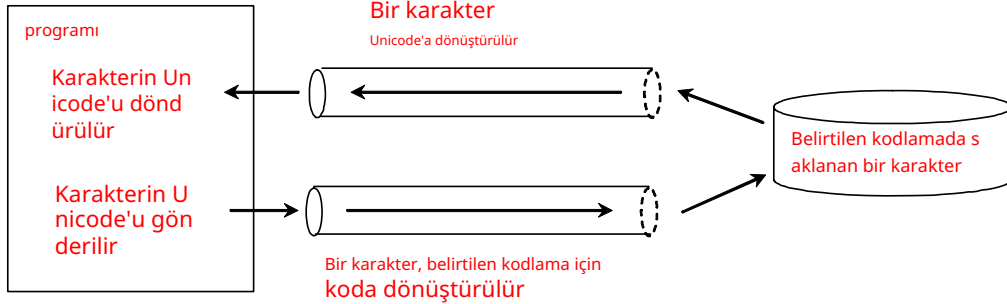
Unicode kodu döndürülür. Dosyadaki karakterin kodlaması

Unicode kodlamasından farklı olabilir. Java otomatik olarak

Unicode'a dönüştürür. Bir karakteri metin G/Ç ile yazdığınızda, Java

karakterin Unicode'unu dosya için belirtilen kodlamaya otomatik olarak

dönüştürür. Bu, Şekil 35.11'de gösterilmiştir.



#### Şekil 35.11

Dosyanın kodlaması, programda kullanılan kodlamadan farklı olabilir.

Bir kodlayıcı şeması belirtebilirsiniz [Scanner](#)/[PrintWriter](#)

metin I/O için, aşağıdaki gibi:

```
public Scanner(File file, String encodingName)
public PrintWriter(File file, String encodingName)
```

Java'da desteklenen kodlama şemalarının bir listesi için,

<http://download.oracle.com/javase/1.5.0/docs/guide/intl/encoding.doc.html> ve

[mindprod.com/jgloss/encoding.html](http://mindprod.com/jgloss/encoding.html) adresine bakın. Örneğin, kodlama adını kullanabilirsiniz

[GB18030](#) basit Çince karakterler için, [Big5](#) geleneksel Çince için

karakterler, [Cp939](#) Japonca karakterler için, [Cp933](#) Korece karakterler için ve [Cp838](#) Tayca karakterler için.

Aşağıdaki kod, GB18030 kodlamasını kullanarak bir dosya oluşturur (8. satır). Metni aynı kodlama ile okumanız gerekir (12. satır). Çıktı Şekil 35.12a'da gösterilmektedir.

#### Liste 35.8 EncodingDemo.java

<margin note line 8: kodlamayı belirt>

<margin note line 12: kodlamayı belirt>

```
1 java.util.*i içe aktarın;
2 java.io.*yu içe aktarın;
3 javax.swing.*i içe aktarın;
4
5 genel sınıf Kodlama Demosu {
6 public static void main(String[] args)
7 throwst IOException, FileNotFoundException {
8 PrintWriter çıktısı = t newt PrintWriter(t"temp.txt",t "GB18030"t);n 9 çıktı.print(t"u6
B22u8FCE Hoş Geldiniz u03b1u03b2u03b3"t);n 10 çıktı.close();
11
12 Tarayıcı girişi = t newt Tarayıcı(tnewt File(t"temp.txt",t "GB18030"t);n 13 JOptionPan
e.showMessageDialog(tnullt, input.nextLine());
14 } n 15
}
```



(a) GB18030 kodlaması kullanarak



(b) Varsayılan kodlama kullanarak

#### Şekil 35.12

Bir metin dosyası için bir kodlama şeması belirtebilirsiniz.

Eğer 8. ve 12. satırlarda bir kodlama belirtmezseniz, sistemin varsayılan kodlama şeması kullanılır. ABD'nin varsayılan kodlaması ASCII'dir. ASCII kodu 8 biti kullanır. Java 16 bitlik Unicode kullanır. Eğer bir Unicode ASCII kodu değilse, karakter '?' dosyaya yazılır. Böylece, bir ASCII dosyasına \u6B22 yazdığınızda, ? karakteri dosyaya yazılır. Geri okuduğunuzda, ? karakterini göreceksiniz, Şekil 35.12b'de gösterildiği gibi.

Sisteminizdeki varsayılan kodlamayı öğrenmek için

<margin note line 8: varsayılan kodlamayı al>

```
System.out.println(System.getProperty("file.encoding"));
```

Varsayılan kodlama adı Windows'ta [Cp1252](#)'dir, bu da ASCII'nin bir varyasyonudur.

#### Temel Terimler

- yerel ayar 7
- kaynak paketi
- dosya kodlama şeması

## Bölüm Özeti

1. Java, uluslararasılaştırmayı desteklemek için sıfırdan tasarlanan ilk dildir. Sonuç olarak, programlarınızın herhangi bir ülke veya dil için özelleştirilmesine olanak tanır ve kodda zahmetli değişiklikler gerektirmez.
2. Java karakterleri programda *Unicode* kullanır. Unicode kodlamasının kullanımı, herhangi bir uluslararası dilde dizeleri işleyebilen Java programları yazmayı kolaylaştırır.
3. Java, belirli bir yerel hakkında bilgi kapsüllemek için *Locale* sınıfını sağlar. Bir *Locale* nesnesi, tarih, saat ve sayı gibi yerel duyarlı bilgilerin nasıl görüntüleneceğini ve dizeleri sıralama gibi yerel duyarlı işlemlerin nasıl gerçekleştirileceğini belirler. Tarih, saat ve sayıları biçimlendirmek ve dizeleri sıralamak için kullanılan sınıflar, *java.text* paketinde gruplandırılmıştır.
4. Farklı yereller, tarih ve saati görüntüleme konusunda farklı geleneklere sahiptir. *java.text.DateFormat* sınıfı ve alt sınıfları, kullanıcıya gösterim için yerel duyarlı bir şekilde tarih ve saati biçimlendirmek için kullanılabilir.
5. Bir sayıyı varsayılan veya belirtilen bir yerel ayara formatlamak için, bir biçimlendirici almak için *NumberFormat* sınıfındaki *factory* sınıf yöntemlerinden birini kullanın. Kullanın *getInstance* veya *getNumberInstance* normal sayı formatını almak için kullanılır. *getCurrencyInstance* para birimi sayı formatını almak için kullanılır. Kullanın *yüzde göstermek için bir format almak üzere getPercentInstance*
6. Java, *ResourceBundle* sınıfını, durum mesajları ve GUI bileşen etiketleri gibi yerel spesifik bilgileri programdan ayırmak için kullanır. Bilgiler kaynak kodunun dışında saklanır ve çalışma zamanında dinamik olarak bir *ResourceBundle*'den erişilip yüklenebilir, programa sabit kodlanmak yerine.
7. Bir metin dosyası oluştururken bir kodlama belirtebilirsiniz. *PrintWriter* veya *Scanner*.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

## Gözden Geçirme Soruları

### Bölmeler 35.1-35.2

35.1

Java, Çince ve

Arapça gibi dillerde uluslararası karakterleri nasıl destekler?

35.2

Bir *Locale* nesnesi nasıl oluşturulur? Bir *Calendar* nesnesinden mevcut tüm yerelleri nasıl alırsınız?

35.3

Swing'de Kanada'nın Fransızca konuşan bölgesi için bir yerel nasıl ayarlanır?

*JButton*? Hollanda için bir Swing *JLabel*? yerel ayarını nasıl ayarlarsınız?

### Bölüm 35.3

35.4

"PST" saat dilimini bir *Calendar* nesnesi için nasıl ayarlarsınız?

35.5

Mevcut tarihi ve saati Almanca olarak nasıl görüntölürsünüz?

35.6

*SimpleDateFormat* sınıfını "yyyy.MM.dd hh:mm:ss" desenini kullanarak tarih ve saati görüntülemek için nasıl kullanırsınız?

35.7

WorldClockControl.java dosyasının 73. satırında, `Arrays.sort(availableTimeZones)` kullanılmaktadır mevcut saat dilimlerini sıralamak için. Eğer mevcut yerelleri `Arrays.sort(availableLocales)` kullanarak sıralamaya çalışırsanız ne olur?

#### Bölüm 35.4

35.8

Birleşik Krallık yerel ayarında 12345.678 sayısını biçimlendirmek için kodu yazın. Ondalık noktadan sonra iki basamak bırakın.

35.9

12345.678 sayısını ABD para biriminde biçimlendirmek için kodu yazın.

35.10

0.345678 sayısını en az üç ondalık basamakla yüzde olarak biçimlendirmek için kodu yazın.

35.11

Kodu yazın 3,456.78'i bir sayıya ayırtırmak için.

35.12

Kodu yazın `DecimalFormat` sınıfını kullanarak 12345.678 sayısını "0.0000#" deseni ile biçimlendirmek için.

#### Bölüm 35.5

35.13

`getBundle` metodu bir kaynak paketini nasıl bulur?

35.14

`getObject` metodu bir kaynağı nasıl bulur?

#### Bölüm 35.6

35.15

Bir metin dosyası için bir kodlama şeması nasıl belirtirsiniz?

35.16

Bir Unicode karakterini bir ASCII metin dosyasına yazarsanız ne olur?

35.17

Sisteminizde varsayılan kodlama adını nasıl bulabilirsiniz?

### Programlama Alıştırmaları

#### Bölümler 35.1-35.2

35.1\*

(Unicode görüntüleyici) Kullanıcının metin alanında bir Unicode belirtip, Enter tuşuna basarak belirtilen Unicode ile başlayan bir dizi Unicode karakterini gösteren bir applet geliştirin. Unicode karakterleri, 20 satırlık kaydırılabilir bir metin alanında görüntülenir. Her satır, satırdaki ilk karakterin kodu olan Unicode ile birlikte 16 karakter içerir.



Şekil 35.13

Applet, Unicode karakterlerini görüntüler.

35.2\*\*



(Tarih ve saati göster) Kullanıcının bir

locale, zaman dilimi, tarih stili ve saat stilini seçim kutularından seçmesine olanak tanıyan bir program yazın. Program, mevcut tarihi ve saati Şekil 35.14'te gösterildiği gibi görüntüler.

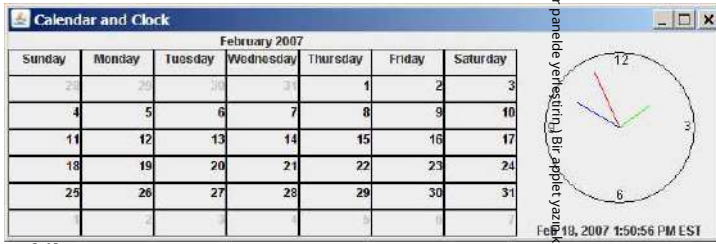


Şekil 35.14

Program mevcut tarih ve saati görüntüler.

### Bölüm 35.3

bir takvimdeki mevcut tarih ve bir saatteki mevcut zaman, Şekil 35.15'te gösterildiği gibi. Applet'in bağımsız olarak çalışmasına izin verin.



Şekil 35.15

Takvim ve saat mevcut tarih ve saati gösterir.

mevcut yerel ayarları ve saat dilimi kimliklerini butonlar kullanarak gösterir, Şekil 35.16'da gösterildiği gibi.



Şekil 35.16

Program, mevcut yerel ayarları ve saat dilimlerini butonlar kullanarak gösterir.

### Bölüm 35.4

35.5\*

(Kredi amortisman takvimini hesaplama) Şekil 35.17'de gösterildiği gibi bir applet kullanarak Alıştırma 4.22'yi yeniden yazın. Applet, kullanıcının kredi miktarını, kredi süresini ve faiz oranını ayarlamasını a olarak tanımlar ve ilgili faiz, anapara ve bakiyeyi para birimi formatında gösterir.

Payment#	Interest	Principal	Balance
1	\$68.33	\$806.93	\$9,193.07
2	\$63.63	\$811.64	\$8,381.42
3	\$48.69	\$816.38	\$7,565.05
4	\$44.13	\$821.14	\$6,743.91
5	\$39.34	\$825.93	\$5,917.98
6	\$34.52	\$830.75	\$5,087.24

Şekil 35.17

Program, kredi ödeme takvimini gösterir.

35.6 D

dolarlarını Kanada doları, Alman markı ve İngiliz sterlini cinsine çevirir, Şekil 35.18'de gösterildiği gibi. Kullanıcı, ABD doları miktarını ve dönüşüm oranını girer ve dönüştürülmüş miktarı göstermek için *Dönüştür* butonuna tıklar.

	Exchange Rate	Converted Amount
Canadian Dollars	1.5	\$3,000.00
German Marks	1.4	¥ 2,800.00
British Pounds	0.5	£1,000.00

Şekil 35.18

Program, ABD doları cinsinden Kanada doları, Alman markı ve İngiliz sterlini cinsine dönüştürür.

35.7 Kr

aylık ödemeyi ve toplam ödemeyi para biriminde gösterin.

35.8

*DecimalFormat*

ondalık basamak göstermek üzere *DecimalFormat*

sınıfını kullanarak.

## Bölüm 35.5

35.9 \*

(*Kaynak paketini kullan*) Şekil 35.3.6'da bir takvimi görüntülemek için örneği, "Bir Yerel Seçin" ve "Takvim Demo" etiketlerini Fransızca, Almanca, Çince veya seçtiğiniz bir dilde yerelleştirmek için değiştirin.

35.10 \*\*

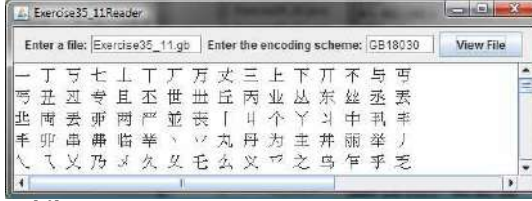
(*Bayrak ve marş*) Listing 18.13, *ImageAudioAnimation.java* dosyasını, resim ve ses dosyalarını almak için kaynak paketini kullanacak şekilde yeniden yazın.

(*İpucu:* Yeni bir ülke seçildiğinde, ona uygun bir yerel ayar belirleyin. Programınız, yerel ayar için kaynak dosyasından bayrak ve ses dosyasını aramalıdır.)

## Bölüm 35.6

35.11 \*\*

(Dosya kodlamalarını belirt) Exercise35\_11Writer adında bir program yazın, bu program 1307 × 16 Çince Unicode karakterini \u0E00 bir dosyaya yazsın Exercise35\_11.gb, GBK kodlama şemasını kullanarak. Her satıra 16 karakter yazın ve karakterleri boşluklarla ayırın. Exercise35\_11Reader adında bir program yazın, bu program belirli bir kodlama kullanarak bir dosyadan tüm karakterleri okur. Şekil 35.19, dosyayı GBK kodlama şemasını kullanarak görüntüler.



Şekil 35.19

Program, belirtilen kodlama şemasını kullanarak dosyayı görüntüler.

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 36

### JavaBeans

#### Hedefler

- Bir JavaBeans bileşenin ne olduğunu tanımlamak (§36.2).
- Beans ile  
normal nesneler arasındaki benzerlikleri ve farklılıkları açıklamak (§36.2).
- JavaBeans bileşenlerini geliştirmek için adlandırma kalıplarını takip edin  
(§36.3) .
- Java olay delegasyonu modelini gözden geçirmek için (§36.4).
- Özel olay sınıfları ve dinleyici arayüzleri oluşturmak için (§36.5).
- Java API'sinden veya özel olay setlerinden olay setleri kullanarak kaynak bileşenleri geliştirmek için (§36.6).

### 36.1 Giriş

Her Java kullanıcı arayüzü sınıfı bir JavaBeans bileşenidir. JavaBeans'i anlamak, GUI bileşenlerini öğrenmenize yardımcı olacaktır. Bölüm 16, "Olay Tabanlı Programlama"da, *JButton*, *JTextField*, *JRadioButton* ve *JComboBox* gibi kaynak bileşenlerinden ateşlenen olayları nasıl yöneteceğinizi öğrendiniz. Bu bölümde, özel olaylar oluşturmayı ve olayları ateşleyebilen kendi kaynak bileşenlerinizi geliştirmeyi öğreneceksiniz. Kendi olaylarınızı ve kaynak bileşenlerinizi geliştirerek, Java olay modelini ve GUI bileşenlerini daha iyi anlayacaksınız.

### 36.2 JavaBeans

#### <kenar notu: Cava dili>

JavaBeans, Java dilinin gücünü artıran bir yazılım bileşen mimarisidir; bu mimari, iyi biçimlendirilmiş nesnelerin, NetBeans ve Eclipse gibi saf Java oluşturucu araçlarında tasarım zamanında görsel olarak işlenmesine olanak tanır. Bu tür iyi biçimlendirilmiş nesnelere *JavaBeans* veya basitçe *beans* denir. Bileşenleri tanımlayan sınıflara ise

*JavaBeans bileşenleri* veya *bean bileşenleri*, aşağıdaki gereksinimlerle JavaBeans bileşen modeline uymalıdır:

- Bir bean, kamuya açık bir sınıf olmalıdır.
- Bir bean, kamuya açık bir argüman almayan yapıcıya sahip olmalıdır, ancak gerekirse başka yapıcılar da olabilir. Örneğin, *MyBean* adında bir bean, aşağıdaki imzaya sahip bir yapıcıya sahip olmalıdır:

**genel *MyBean()*:**

veya, eğer üst sınıfının bir argüman almayan yapıcısı varsa, yapıcıya sahip olmamalıdır.

#### <kenar notu: Serileştirilebilir>

- Bir bean, kalıcı bir durumu sağlamak için *java.io.Serializable* arayüzünü uygulamalıdır.

#### <kenar boşluğu notu: erişimci> n<kenar boşluğu notu: değiştirici>

- Bir bean genellikle, bir yapılandırma aracı tarafından görsel olarak görülebilen ve güncellenebilen özellikleri etkinleştiren, doğru bir şekilde oluşturulmuş kamuya açık erişim (get) yöntemleri ve değiştirici (set) yöntemleri ile özelliklere sahiptir.

#### <kenar notu: etkinlik kaydı>

- Bir bean, dinleyicileri ekleyip kaldırmasını sağlayan doğru şekilde yapılandırılmış kamu kayıt ve iptal yöntemlerine sahip olaylara sahip olabilir. Eğer bean, eventlerin kaynağı olarak bir rol oynuyorsa, dinleyicileri kaydetmek için kayıt yöntemleri sağlamalıdır. Örneğin, bir dinleyiciyi

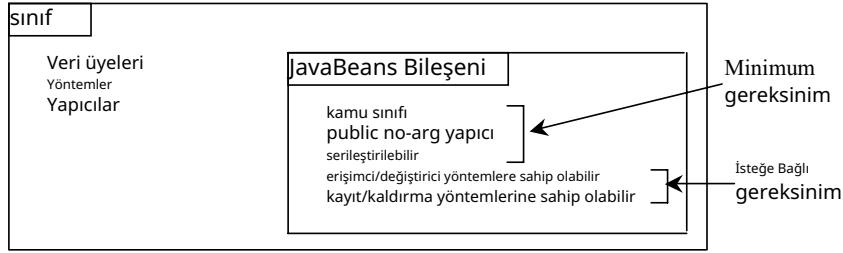
*ActionEvent* kullanarak *addActionListener* yöntemini *JButton* bean'inin.

İlk üç gereksinim göz önünde bulundurulmalı ve bu nedenle minimum JavaBeans bileşen gereksinimleri olarak adlandırılmaktadır. Son iki gereksinim uygulamalara bağlıdır. Bir bean bileşeni, get/set yöntemleri ve olay kaydı/kaldırma

yöntemleri olmadan yazılabilir.

JavaBeans bileşeni, özel bir Java sınıfı türüdür. JavaBeans bileşenleri ile Java sınıfları arasındaki ilişki Şekil'de gösterilmektedir.

36.1.



**Şekil 36.1**

*JavaBeans bileşeni, kamuya açık bir no-arg yapıcıya sahip, seri hale getirilebilir bir kamu sınıfıdır.*

Her GUI sınıfı bir JavaBeans bileşenidir, çünkü

- (1) kamuya açık bir sınıftır,
- (2) kamuya açık bir no-arg yapıcıya sahiptir ve
- (3) `java.awt.Component`'i genişleten bir sınıftır, bu da

`java.io.Serializable`'i uygular.

### 36.3 Bean Özellikleri

Özellikler, bir Java bean'in görünümünü veya davranışını etkileyebilen, ayrı, adlandırılmış niteliklerdir. Genellikle bir bean'in veri alanlarıdır. Örneğin, `JButton` bileşeni, düğmede görüntülenecek metni temsil eden `text` adında bir özelliğe sahiptir. Özel veri alanları, kullanıcıdan belirli uygulamaları gizlemek ve kullanıcıyı özellikleri yanlışlıkla bozmadan korumak için sıklıkla kullanılır. Bunun yerine, kullanıcıya özellikleri okuma ve yazma imkanı tanımak için erişimci ve değiştirici yöntemler sağlanır.

#### 36.3.1 Mülk İsimlendirme Desenleri

Bean özellik adlandırma deseni, JavaBeans komponent modelinin bir geleneğidir ve bean geliştiricisinin özellikleri sunma görevini basitleştirir. Bir özellik, bir ilkel veri türü veya bir nesne türü olabilir. Özellik türü, erişimci ve değiştirici

metotlarının imzasını belirler.

Genel olarak, erişimci yöntemi `get<PropertyName>()` olarak adlandırılır, bu parametre almaz ve bir ilkel tür değeri veya özellik türü ile aynı türde bir nesne döndürür. Örneğin,

<kenar notu: erişim yöntemi>

```
public String getMessage()
public int getXKoordinatı()
public int getYKoordinatı()
```

Boolean türündeki bir özellik için, erişimci metodun adı `is<PropertyName>()` olmalıdır ve bu bir `boolean` değer döndürür. Örneğin,

<margin note: boolean accessor method>

```
public boolean merkezlenmişMi()
```

Değiştirici yöntem `set<PropertyName>(dataType p)` olarak adlandırılmalıdır, bu özellik türü ile aynı olan tek bir parametre alır ve `void` döner. Örneğin,

<kenar notu: değiştirici yöntem>

```
public void setMessage(String s)
public void setXCoordinate(int x)
public void setYCoordinate(int y)
public void setCentered(boolean centered)
```

NOT

Birden fazla get ve set yöntemi olabilir, ancak adlandırma kalıplarına uyan bir get veya set yöntemi olmalıdır.

### 36.3.2 Özellikler ve Veri Alanları

Özellikler, bean'ın durumunu tanımlar. Doğal olarak, veri alanları özellikleri saklamak için kullanılır. Ancak, bir bean özelliği mutlaka bir veri alanı değildir. Örneğin, Listing 15.7'deki MessagePanel sınıfında, messageLength adında yeni bir özellik oluşturabilirsiniz; bu, message'daki karakter sayısını temsil eder. Özellik için get yöntemi aşağıdaki gibi tanımlanabilir:

```
public int getMessageLength() {return
 message.length();}
```

NOT

<margin note: salt okunur özellik>

<margin note: yalnızca yazma özelliği>

Bir özellik, bir get yöntemi ile ancak bir set yöntemi olmadan salt okunur olabilir veya bir set yöntemi ile ancak bir get yöntemi olmadan yalnızca yazma özelliğine sahip olabilir.

### 36.4 Java Olay Modeli İncelemesi

Bir bean, diğer bean'larla iletişim kurabilir. Java olay delegasyonu modeli, bean'ların olayları göndermesi, alması ve işlemesi için temel sağlar. 16. Bölüm, "Olay Tabanlı Programlama"da tanıtılan Java olay modelini gözden geçirelim. Java olay modeli, Şekil 16.3'te gösterildiği gibi aşağıdaki üç tür öğeden oluşur:

- Olay nesnesi
- Kaynak nesnesi
- Olay dinleyici nesnesi

<margin note: olay>

<margin note: kaynak nesnesi>

<margin note: dinleyici>

Bir olay, program için bir şeyin olduğunu bildiren bir sinyaldir. Dış kullanıcı eylemleri, fare hareketleri, fare butonu tıklamaları ve tuş vuruşları gibi veya bir zamanlayıcı gibi işletim sistemi tarafından tetiklenebilir. Bir olay nesnesi, olayı tanımlayan bilgileri içerir. Bir kaynak nesnesi, olayın kaynağıdır. Bir olay, bir kaynak nesnesinde meydana geldiğinde, bir olay nesnesi oluşturulur. Olayla ilgilenen bir nesne, olayı işler. Böyle bir nesneye dinleyici denir. Tüm nesneler olayları işleyemez. Bir dinleyici olmak için, bir nesne kaynak nesnesi tarafından dinleyici olarak kaydedilmelidir. Kaynak nesnesi, dinleyicilerin bir listesini tutar ve tüm kayıtlı dinleyicileri, dinleyici nesnesinde uygulanan olay işleme yöntemini çağırarak bilgilendirir. İşleyiciler, olay dinleyici arayüzü içinde tanımlanır. Her olay sınıfının karşılık gelen bir olay dinleyici arayüzü vardır. Java olay modeli, kaynak nesnesinin olayı işlenmesi için dinleyicilere devrettiği için delegasyon tabanlı model olarak adlandırılır.

### 36.4.1 Olay Sınıfları ve Olay Dinleyici Arayüzleri

Bir olay nesnesi, Şekil 16.2'de gösterildiği gibi, `ActionEvent`, `MouseEvent` ve `ItemEvent` gibi bir olay sınıfı kullanılarak oluşturulur. Tüm olay sınıfları `java.util.EventObject` sınıfından türetilir. Olay sınıfı, belirli olay türü ile ilgili olan veri değerlerini ve yöntemlerini içerir. Örneğin, `KeyEvent` sınıfı, bir tuş olayı ile ilgili veri değerlerini tanımlar ve olayla ilişkili tuşu almak için `getKeyChar()` gibi yöntemleri içerir.

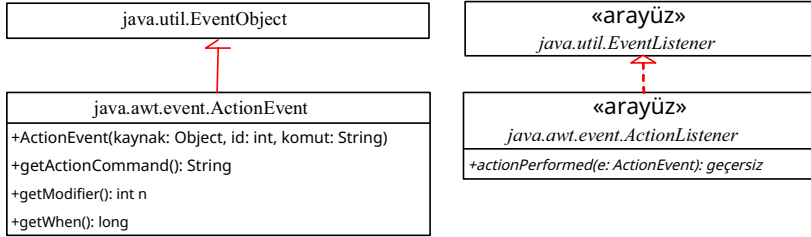
#### <kenar notu: işlemler>

Her olay sınıfı, *handler* olarak adlandırılan bir veya daha fazla yöntemi tanımlayan bir olay dinleyici arayüzü ile ilişkilidir. Bir olay dinleyici arayüzü, `java.util.EventListener` arayüzünün bir alt arayüzüdür. Dinleyici bileşenleri, olay algılandığında kaynak bileşeni tarafından dinleyicilerin yöntemleri çağırılır.

#### <kenar notu: olay seti>

Bir olay sınıfı ve dinleyici arayüzü birlikte var olduğundan, genellikle *event set* veya *event pair* olarak adlandırılırlar. Olay dinleyici arayüzü, `XEvent` için `XListener` olarak adlandırılmalıdır. Örneğin, `ActionEvent` için dinleyici arayüzü `ActionListener`'dir. Bir dinleyicinin parametre listesi her zaman olay sınıfı türünde bir argümandan oluşur. Tablo 16.2, bazı yaygın olarak kullanılan olayları ve dinleyici arayüzlerini listelemektedir. Şekil 36.2, `ActionEvent` ve

`ActionListener`.



Şekil 36.2

`ActionEvent` ve `ActionListener` bir olay çiftinin örnekleridir.

### 36.4.2 Kaynak Bileşenleri

Bir olayın üretildiği bileşene *olay*

*kaynağı* denir. Her Java GUI bileşeni bir veya daha fazla

event için olay kaynağıdır. Örneğin, `JButton` bir `ActionEvent` için bir olay kaynağıdır. Bir

`JButton` nesnesi tıklandığında bir `java.awt.event.ActionEvent` tetikler.

`JComboBox` bir `ActionEvent` ve `ItemEvent` için bir olay kaynağıdır. Bir `JComboBox`

nesnesi, combo kutusunda yeni bir öğe seçildiğinde bir `java.awt.event.ActionEvent` ve bir

`java.awt.event.ItemEvent` tetikler.

Kaynak bileşeni, olayı tetikleyen bir dış veya iç eylemi tespit eden kodu içerir. Eylemi tespit ettikten sonra, kaynak, dinleyiciler tarafından tanımlanan olay işleyicisini çağırarak dinleyicilere bir olay tetiklemelidir. Kaynak bileşeni ayrıca, Şekil 36.3'te gösterildiği gibi, dinleyicileri kaydetme ve kaydı kaldırma yöntemlerini de içermelidir.





**Şekil 36.3**

*Kaynak bileşeni olayları tespit eder ve bunları olay dinleyicilerinin işleyicilerini çağırarak işler.*

### 36.4.3 Dinleyici Bileşenleri

Bir olay için bir dinleyici bileşeni, olay dinleyici arayüzünü uygulamalıdır. Dinleyici bileşenin nesnesi, kaynak bileşenin bir dinleyicisi olarak kaydedilmedikçe olay bildirimlerini alamaz.

Bir dinleyici bileşeni, birden fazla türde olayı dinlemek için herhangi bir sayıda dinleyici arayüzünü uygulayabilir. Bir kaynak bileşeni birçok dinleyici kaydedebilir. Bir kaynak bileşeni kendisini bir dinleyici olarak kaydedebilir.

Liste 36.1, bir kaynak nesnesi (satır 8) ve bir dinleyici nesnesi (satır 14) oluşturan bir örnek vermektedir ve dinleyiciyi kaynak nesnesi ile kaydeder (satır 17). Şekil 36.4, kaynak ile dinleyici arasındaki ilişkiyi vurgulamaktadır. Dinleyici, `addActionListener` yöntemini çağırarak kaynak ile kaydedilir. Butona tıklandığında, kaynak tarafından bir `ActionEvent` üretilir. Kaynak nesnesi daha sonra dinleyiciyi, dinleyicinin `actionPerformed` yöntemini çağırarak bilgilendirir.

**Liste 36.1 TestSourceListener.java**

```

<kenar notu satır 8: kaynak nesne>
<kenar notu satır 14: dinleyici nesnesi>
<kenar notu satır 17: kayıt>
<kenar notu satır 22: dinleyici sınıfı>

1 javax.swing.*'i içe aktarın;
2 java.awt.event.*'i içe aktarın;
3
4 genel sınıf Test Kaynağı Dinleyicisi {
5 public static void main(String[] args) {
6 JFrame çerçevesi = t newt JFrame(t"TestSourceListener");
7 // Bir kaynak nesne oluşturun
8 JButton jbt = t newt JButton(t"OK"); n 9frame.
 add(jbt);
10 çerçeve.setSize(t200t,t 200t); n 11 çerçeve.setVisible(ttrue);
12
13 // Bir dinleyici oluşturun
14 MyListener dinleyicisi = yeni MyListener();
15
16 // Bir dinleyici kaydet
17 jbt.addActionListener(dinleyici); n 18 }

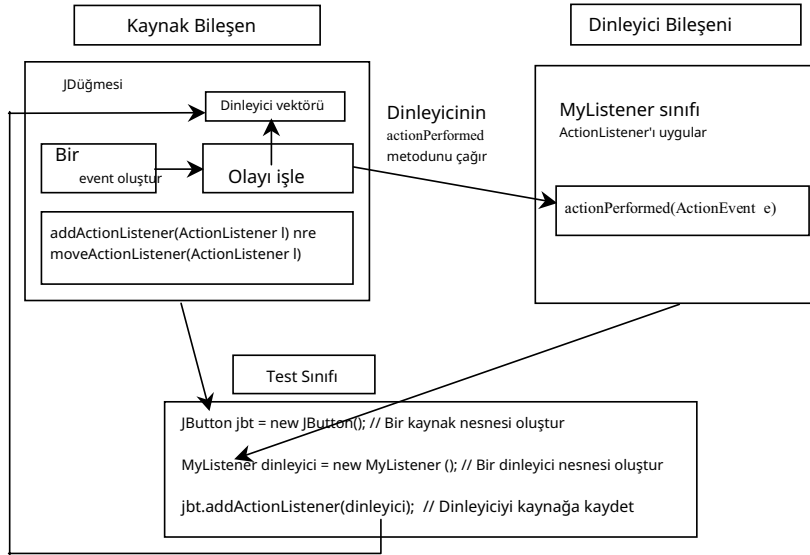
19 }
20

```

```

21 /** MyListener sınıfı */
22 klas Dinleyici t ActionListener'ı uygular {
23 @Geçersiz Kıl
24 genel boşluk actionPerformed(ActionEvent e) {
25 System.out.println("Ben bunu işleyeceğim!");
26 }
27 }

```



**Şekil 36.4**

*Dinleyici, kaynak ile kaydedilir ve kaynak, olayı işlemek için dinleyicinin işleyicisini çağırır.*

### 36.5 Özel Kaynak Bileşenleri Oluşturma

JButton gibi kaynak bileşenleri kullandınız. Bu bölüm, özel bir kaynak bileşeni nasıl oluşturacağınızı göstermektedir.

#### <margin note: kayıt yöntemi>

Bir kaynak bileşenin dinleyicileri eklemek ve kaldırmak için uygun kayıt ve kayıt iptali yöntemlerine sahip olması gerekir. Olaylar, unicast (sadece bir dinleyici nesnesi olaya bildirilir) veya multicast (bir dinleyici listesinde her nesne olaya bildirilir) olarak iletilebilir. Unicast dinleyici eklemek için adlandırma deseni

#### <kenar notu: tek noktaya yayın>

```

kamu boşluğu addt<Event>tListener(t<Event>tListener l) nt
hrows TooManyListenersException;

```

Multicast dinleyici eklemek için adlandırma deseni aynıdır, ancak TooManyListenersException'ı atmaz.

#### <margin note: çoklu yayın>

```

kamu boşluğu add<Event>Listener (<Event>Listener l)<

```

margin note: kayıttan çıkarma yöntemi> Bir dinleyiciyi (ister unicast ister çoklu yayın) kaldırmak için adlandırma deseni:

**kamu boşluğu** `remove<Event>Listener (<Event>Listener l)Bir`

kaynak bileşeni, bir olay nesnesi oluşturan ve bunu dinleyicilerin işleyicisini çağırmak için geçiren kodu içerir. Gerekirse bir standart Java olay sınıfı olan `ActionEvent` kullanarak olay nesneleri oluşturabilir veya kendi olay sınıflarınızı tanımlayabilirsiniz.

#### 10.8. Bölümdeki `Course` sınıfı, "Vaka Çalışması: `Course`

Sınıfını Tasarlama," dersleri modellemektir. Diyelim ki bir `Course` nesnesi, dersin öğrenci sayısı belirli bir kayıt sınırını aştığında bir `ActionEvent` tetikler. `CourseWithActionEvent` adlı yeni sınıf Şekil 36.5'te gösterilmektedir.

CourseWithActionEvent ile	
-kursAdi: Dize	Kursun adı.
-students: ArrayList<String>	Kursu alan öğrenciler.
-e kayıt kapasitesini: int	Maksimum kayıt (varsayılan: 10).
+CourseWithActionEvent() n+	Varsayılan bir kurs oluşturur.
CourseWithActionEvent(courseName: String)n+get	Belirtilen adla bir kurs oluşturur.
CourseName(): String	Kursun adını döndürür.
+addStudent(student : String): void n	Kurs listesine yeni bir öğrenci ekler.
+getStudents(): ArrayList<String> n+get	Kurs için öğrencileri bir dizi olarak döndürür.
NumberOfStudents(): int n+getEnroll	Kurs için öğrenci sayısını döndürür.
mentCap(): int n+setEnrollmentCap(e	Kayıt kapasitesini döndürür.
nrollmentCap: int): void n+addActionL	Yeni bir kayıt sınırı belirler.
istener(e: ActionEvent): void n+remove	Yeni bir ActionEvent dinleyicisi ekler.
ActionListener(e: ActionEvent): void n-p	Bir ActionEvent dinleyicisini siler.
rocessEvent(e: ActionEvent): void d	Bir ActionEvent'i işler.

**Şekil 36.5**

***Yeni `CourseWithActionEvent` sınıfı bir `ActionEvent` tetikleyebilir.***

Kaynak bileşeni dinleyicileri kaydetmek, eylemler oluşturmak ve dinleyicilerin arayüzlerinde tanımlanan yöntemleri çağırarak dinleyicileri bilgilendirmekten sorumludur. `CourseWithActionEvent` bileşeni birden fazla dinleyici kaydedebilir, kayıt sınırı aşıldığında `ActionEvent` nesneleri oluşturabilir ve dinleyicileri dinleyicilerin `actionPerformed` yöntemi ni çağırarak bilgilendirebilir. Liste 36.2 yeni sınıfı uygular.

#### Liste 36.2 `CourseWithActionEvent.java`

<margin note line 6: öğrencileri sakla>  
 <margin note line 7: kayıtSınırı>  
 <margin note line 8: dinleyicileri sakla>  
 <margin note line 10: argüman içermeyen yapıcı>  
 <margin note line 13: yapıcı>  
 <margin note line 17: return `courseName`>  
 <margin note line 23: etkinlik oluştur>  
 <margin note line 46: dinleyici kaydet>  
 <margin note line 58: dinleyiciyi kaldır>  
 <margin note line 67: olayı işle>

Satır Numaraları Olmadan Kodu Göster

```
1 java.util.*'i içe aktarın;
2 java.awt.event.*'i içe aktarın;
3
```

```

4 genel sınıf CourseWith ActionEvent {
5 private String courseName = "varsayılan isim";
6 özel ArrayList<String> öğrencinin = t newt ArrayList<String>();
7 özel intt kayıt sınırı = t 10t;
8 özel ArrayList<ActionListener> actionListener Listesi;
9
10 ActionEvent() ile herkese açık Kurs {
11 }
12
13 ActionEvent İçeren herkese açık Kurs(Dize kursu Adı) {
14 this.kursAdı = kursAdı;
15 }
16
17 public String getKursAdı() {
18 return kursAdı;
19 }
20
21 public void öğrenciEkle(String öğrenci) {
22 if (öğrenciler.boyut() >= kayıtLimiti) // Fire ActionEvent
23 prosesEvent(tnewt ActionEvent(tthist,n 24 ActionEve
nt.ACTION_PERFORMED, t nullt));
25 başka
26 öğrenciler.add(öğrenci);
27 }
28
29 public ArrayList<String> getStudents() {
30 return öğrenciler;
31 }
32
33 public intt getNumberOfStudents() {
34 return öğrenciler.size();
35 }
36
37 public intt getEnrollmentCap() {
38 return kayıtLimiti;
39 }
40
41 public void setEnrollmentCap(intt kayıtLimiti) {
42 this.kayıtLimiti = kayıtLimiti;
43 }
44
45 /** Bir eylem olayı dinleyicisi kaydet */
46 genel senkronize voidt addActionListener
47 (ActionListener dinleyici) {
48 ift (actionListenerList == t nullt) {
49 actionListenerList = t newt ArrayList<ActionListener>(t2t;n 50 }
51
52 if (!actionListenerList.contains(dinleyici)) {
53 actionListenerList.add(dinleyici);
54 }
55 }
56
57 /** Bir eylem olayı dinleyicisini kaldır */
58 genel senkronize voidt RemoveActionListener
59 (EylemDinleyici dinleyici) {
60 eğer (eylemDinleyiciListesi !=
61 null && eylemDinleyiciListesi.contains(dinleyici)) {
62 eylemDinleyiciListesi.remove(dinleyici);

```

63 } n 64 }

```
65
66 /** EylemOlayını Ateşle */
67 özel void olayİşle(ActionOlay e) {
68 DiziListesi<EylemDinleyici> liste;
69
70 synchronized (bu) {
71 eğer (eylemDinleyiciListesi == null) dön;
72 liste = (ArrayList<ActionListener>)actionListenerList.clone();
73 }
74
75 for (int i = 0; i < liste.size(); i++) {
76 ActionListener dinleyici = (ActionListener)liste.get(i);
77 dinleyici.actionPerformed(e);
78 }
79 }
80 }
```

Kaynak bileşeni birden fazla dinleyici için tasarlandığından, bir `java.util.ArrayList` örneği `actionListenerList` tüm dinleyicileri tutmak için kullanılır kaynak bileşeni (satır 8). Dizi listesindeki öğelerin veri tipi `ActionListener`. Bir dinleyici eklemek için, `dinleyici`, `actionListenerList`'e eklemek için

`actionListenerList.add(dinleyici);` (satır 53)

Bir dinleyiciyi, `dinleyici`, `actionListenerList`'ten kaldırmak için

`actionListenerList.remove(dinleyici);` (satır 62)

if ifadesi (satırlar 52-53), `addActionListener` metodunun dinleyiciyi zaten listede varsa iki kez eklemesini engeller. `removeActionListener` metodu, bir dinleyiciyi listede varsa kaldırır. `actionListenerList` bir `ArrayList` örneğidir ve dinamik olarak büyüyebleen veya küçülebilen esnek bir dizi olarak işlev görür. Başlangıçta, `actionListenerList` boyutu 2'dir, ancak kapasite dinamik olarak değiştirilebilir. Eğer `actionListenerList` içine iki dinleyiciden fazla eklenirse, liste boyutu otomatik olarak artırılacaktır.

NOT

<margin note: dinleyicileri saklama>

Dinleyicileri saklamak için `ArrayList` yerine `javax.swing.event.EventListenerList` de kullanabilirsiniz. `EventListenerList` kullanmak tercih edilir, çünkü senkronizasyon desteği sağlar ve dinleyici olmadığında etkilidir.

`addActionListener` ve `removeActionListener` metodları, birden fazla dinleyiciyi eşzamanlı olarak kaydetmeye çalışırken `actionListenerList` üzerinde veri bozulmasını önlemek için senkronizedir (satırlar 46, 58).

`addStudent` metodu (satırlar 21-27), öğrencilerin sayısının kayıt sınırından fazla olup olmadığını kontrol eder. Eğer öyleyse, bir `ActionEvent` oluşturur ve olayı işlemek için `processEvent` metodunu çağırır (satırlar 23-24). Aksi takdirde, derse yeni bir öğrenci ekleyin (satır 26).

UML diyagramı `ActionEvent` Şekil 36.2'de gösterilmiştir. Bir

`ActionEvent` oluşturmak için, aşağıdaki yapıyı kullanın:

```
ActionEvent(Object source, int id, String command)
```

burada **source** kaynak bileşeni belirtir, **id** olayı tanımlar, ve **command** olayla ilişkili bir komutu belirtir. **id** için **ActionEvent.ACTION\_PERFORMED** kullanın. Olayla bir komut ilişkilendirmek istemiyorsa **null** kullanın.

**processEvent** metodu (satırlar 67-79) bir **ActionEvent** oluşturulduğunda çağrılır. Bu, **actionListenerList** içindeki dinleyicileri, her dinleyicinin **actionPerformed** metodunu çağırarak olayı işlemek için bilgilendirir. **processEvent** çalışırken yeni bir dinleyici eklenebilir veya mevcut bir dinleyici kaldırılabilir. **actionListenerList** üzerinde bozulmayı önlemek için, dinleyicileri bilgilendirmek için **actionListenerList** için bir kopya **list** oluşturulur. Kopyayı oluştururken bozulmayı önlemek için, bunu senkronize bir blok içinde çağırın, satırlar 70-73'te olduğu gibi:

```
senkronize edilmiş (bu) {
 if (actionListenerList == null) return;
 List<ActionListener> list = (ArrayList) actionListenerList.clone();
}
```

Liste 36.3, yeni

**class** (satır 5) kullanarak bir kurs oluşturan, kayıt sınırını 2 olarak ayarlayan (satır 8), bir dinleyici kaydeden (satır 9) ve kursa üç öğrenci ekleyen (satırlar 11-13) bir test programı vermektedir. 13. satırı çalıştırıldığında, **addStudent** metodu öğrenci **Tim**'i kursta ekler ve kurs kayıt sınırını aştığı için bir **ActionEvent** tetikler. Kurs nesnesi, olayı işlemek için dinleyicinin **actionPerformed** metodunu çağırır ve bir mesaj gösterir **Kayıt sınırı aşıldı**.

### Listeleme 36.3 TestCourseWithActionEvent.java

<margin note line 5: kurs oluştur>

<margin note line 8: kayıtSınırı ayarla>

<margin note line 9: dinleyici oluştur>

<margin note line 10: dinleyiciyi kaydet>

<margin note line 11: öğrencileri ekle>

```
1 java.awt.event.*i içe aktarın;
2
3 genel sınıf TestCourseWith ActionEvent {
4 CourseWithActionEvent kurs =
5 new CourseWithActionEvent ("Java Programlama");
6
7 genel TestCourseWith ActionEvent() {
8 kurs.setEnrollmentCap(2);
9 ActionListener dinleyici = new Listener();
10 kurs.addActionListener(dinleyici);
11 kurs.addStudent("John");
12 kurs.addStudent("Jim");
13 kurs.addStudent("Tim");
14 }
15
16 public static void main(String[] args) {
17 new TestCourseWithActionEvent();
18 }
19
20 özel sınıf Dinleyici Uygular ActionListener {
21 @Geçersiz Kıl
```

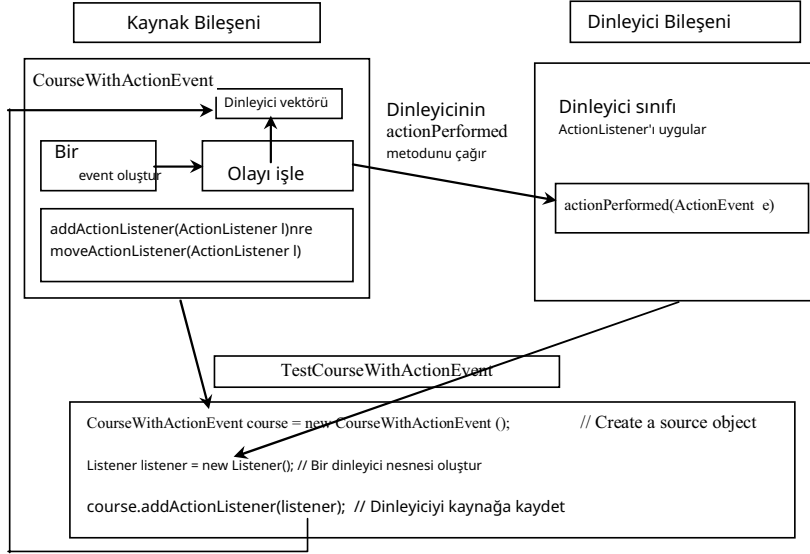
```

22 genel boşluk actionPerformed(ActionEvent e) {
23 System.out.println("Kaydolma sınırı aşıldı");
24 }
25 }
26 }

```

Olay işleme akışı kaynaktan dinleyiciye

Şekil 36.6'da gösterilmiştir.



### Şekil 36.6

Dinleyici, kaynak *course* ile kaydedilir ve kaynak dinleyicinin işleyicisini *actionPerformed* olayı işlemek için çağırır.

### 36.6 Özel Olay Setleri Oluşturma

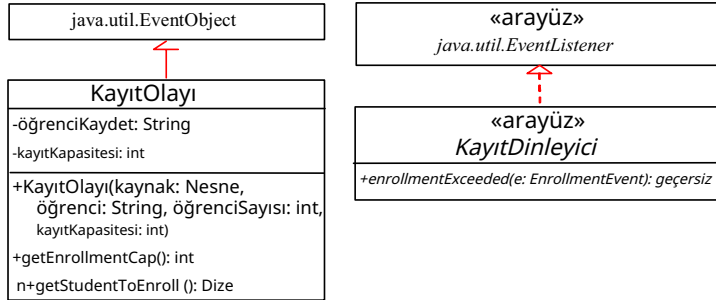
Java API birçok olay seti sağlar. Önceki bölümde olay seti *ActionEvent/ActionListener* kullandınız. Bir kurs nesnesi kayıt sınırı aşıldığında bir *ActionEvent* tetikler. Java API'deki mevcut olay setlerini kullanmak kullanışlıdır, ancak her zaman yeterli değildir. Bazen mevcut API olay sınıflarında bulunmayan bilgiler elde etmek için özel olay sınıfları tanımlamanız gerekir. Örneğin, kayıt sınırını ve kursun öğrenci sayısını bilmek istiyorsanız; bir *ActionEvent* nesnesi bu tür bilgileri sağlamaz. Kendi olay sınıfınızı ve olay dinleyici arayüzünüzü tanımlamanız gerekir.

Özel bir olay sınıfı, *java.util.EventObject* veya *java.util.EventObject*'un bir alt sınıfını genişletmelidir. Ayrıca, olayları oluşturmak için yapıcılar, veri üyeleri ve olayları tanımlamak için yöntemler sağlayabilir.

Özel bir olay dinleyici arayüzü, *java.util.EventListener* veya *java.util.EventListener* alt arayüzünü genişletmelidir ve olay için işleyicilerin imzasını tanımlamalıdır. Geleneksel olarak, dinleyici arayüzü, *XEvent* adlı ilgili olay sınıfı için *n* *XListener* olarak adlandırılmalıdır. Örneğin, *ActionListener* *ActionEvent* için dinleyici arayüzüdür.

*EnrollmentEvent* sınıfını, kayıt olayını ve buna karşılık gelen dinleyici arayüzünü tanımlayalım.

*EnrollmentListener* bir *enrollmentExceeded* işleyicisi tanımlamak için, Şekil 36.7'de gösterildiği gibi. *getStudentToEnroll()* metodu, kursa kaydolmaya çalışan öğrenciyi döndürür.



**Şekil 36.7**

*EnrollmentEvent* ve *EnrollmentListener* kayıt olma olayı için bir olay seti oluşturur.

Kayıt olma olayı setinin kaynak kodu Liste 36.4

ve 36.5'te verilmiştir.

#### Liste 36.4 EnrollmentEvent.java

<margin not satırı 1: extends t EventObject>n<margin note line 6: yapıcı>  
<kenar notu satırı 8: üst sınıf yapıcıyı çağırın>

```

1 public class EnrollmentEvent extends java.util.EventObject {
2 özel String studentToEnroll;
3 özel uluslararası kayıt sınırı;
4
5 /** Bir EnrollmentEvent oluştur */
6 genel Kayıt Etkinliği (Nesne kaynağı, Kayıt Olacak String öğrencisi,
7 intt enrollmentCap) {
8 süper(kaynak);
9 this.Kayıt Olacak Öğrenci = Kaydolacak Öğrenci;
10 this.kayıtLimiti = kayıtLimiti;
11 }
12
13 public String Öğrencilerin Kaydolmasını Sağla() {
14 Geri dönen öğrenci Kayıt olmak için;
15 }
16
17 genel longt getEnrollmentCap() {
18 return kayıtLimiti;
19 }
20 }

```

#### Listeleme 36.5 EnrollmentListener.java

<margin not satırı 1: extends t EventListener>n<margin in note line 3: handler>

```

1 genel arayüz EnrollmentListener extends java.util.EventListener {
2 /** Bir EnrollmentEvent'i işleyin, bir dinleyici tarafından uygulanacaktır */
3 genel geçersiz kayıt Aşıldı (Kayıt Olayı e);

```



4 }

Bir olay sınıfı, *EventObject*'un bir uzantısıdır. Bir olayı oluşturmak için, the constructor of *EventObject* bir kaynak nesnesini argüman olarak geçirerek çağrılmalıdır. *EnrollmentEvent* için yapıcıda, *super(source)* (satır 8) kaynak nesnesini argüman olarak alarak üst sınıfın yapıcısını çağırır. *EnrollmentEvent* olayla ilgili bilgileri içerir, örneğin öğrenci sayısı ve kayıt sınırı.

*EnrollmentListener* basitçe *EventListener* uzatır ve kayıt olaylarını işlemek için *enrollmentExceeded* yöntemini tanımlar.

NOT

<margin note: bir olay için bir kaynak belirtme>

Bir olay sınıfının argüman almayan bir yapıcısı yoktur, çünkü bir olayı oluştururken her zaman olay için bir kaynak belirtmelisiniz.

Şimdi, *CourseWithActionEvent* 36.2. Listesi'nde

*EnrollmentEvent/EnrollmentListener* kullanacak şekilde gözden geçirelim. *ActionEvent/ActionListener* yerine. 36.6. Listesi'ndeki yeni sınıf *CouseWithEnrollmentEvent* 36.2. Listesi'ndeki *CourseWithActionEvent* ile çok benzer.

### 36.6. Listesi CourseWithEnrollmentEvent.java

<margin note line 5: öğrencileri sakla>  
<margin note line 6: kayıtKap>  
<margin note line 8: dinleyicileri sakla>  
<margin note line 10: argüman içermeyen yapıcı>  
<margin note line 13: yapıcı>  
<margin note line 22: etkinlik oluştur>  
<margin note line 45: dinleyici kaydet>  
<margin note line 57: dinleyiciyi kaldır>  
<margin note line 66: olayı işle>

```
1 java.util.*'i içe aktarın;
2
3 genel sınıf CourseWithEnrollmentEvent {
4 private String courseName = "varsayılan isim";
5 özel ArrayList<String> öğrencinin = t newt ArrayList<String>();
6 özel intt kayıt sınırı = t 10;
7 özel ArrayList<Kayıt Dinleyicisi> kayıt Dinleyici Listesi;
8
9 genel CourseWithEnrollmentEvent() { n10 }

11
12 public CourseWithEnrollmentEvent(Dize ders Adı) {
13 this.kursAdı = kursAdı;
14 }
15
16 public String getKursAdı() {
17 return kursAdı;
18 }
19
20 public void öğrenciEkle(String öğrenci) {
21 if (students.size() == enrollmentCap) // KayıtOlayını Ateşle
22 prosesEvent(tnewt EnrollmentEvent(tthist,n23 öğrenci, enrollmentCap));
24 başka
25 öğrenciler.add(öğrenci);
```

```

26 }
27
28 public ArrayList<String> getStudents() {
29 return öğrenciler;
30 }
31
32 public int getNumberOfStudents() {
33 return öğrenciler.size();
34 }
35
36 public int getEnrollmentCap() {
37 return kayıtLimiti;
38 }
39
40 public void setEnrollmentCap(int kayıtLimiti) {
41 this.kayıtLimiti = kayıtLimiti;
42 }
43
44 /** Bir eylem olayı dinleyicisi kaydet */
45 genel senkronize geçersiz ekleme Kayıt Dinleyicisi
46 (EnrollmentListener dinleyici) {
47 if (enrollmentListenerList == null) {
48 enrollmentListenerList = new ArrayList<EnrollmentListener>(t2t);n49 }
49
50
51 eğer (!enrollmentListenerList.contains(listener)) {
52 enrollmentListenerList.add(dinleyici); n53 }
53
54 }
55
56 /** Bir eylem olayı dinleyicisini kaldır */
57 genel senkronize geçersiz Kayıt Dinleyiciyi kaldır
58 (EnrollmentListener dinleyici) {
59 if (enrollmentListenerList !=
60 null && enrollmentListenerList.contains(dinleyici)) {
61 enrollmentListenerList.remove(dinleyici); n62 }
62
63 }
64
65 /** Yangın Kayıt Etkinliği */
66 özel voidt prosesEvent(EnrollmentEvent e) {
67 ArrayList<EnrollmentListener> liste;
68
69 synchronized (bu) {
70 eğer (enrollmentListenerList == null) geri dön;
71 liste = (ArrayList<EnrollmentListener>)
72 enrollmentListenerList.clone();
73 }
74
75 for (int i = 0; i < liste.size(); i++) {
76 EnrollmentListener dinleyici = (EnrollmentListener)liste.get(i);
77 dinleyici.enrollmentExceeded(e);
78 }
79 }
80 }

```

Satır 8, `java.util.ArrayList` örneği `enrollmentListenerList` kaynağın tüm dinleyicil erini tutmak için oluşturur. Dizi listesindeki elemanların veri tipi `EnrollmentListener`.

EnrollmentListener için kayıt ve kayıttan çıkarma yöntemleri  
45, 57. satırlarda tanımlanmıştır.

`addStudent` yöntemi, öğrencilerin sayısının kayıt sınırından fazla olup olmadığını kontrol eder. Eğer öyleyse, bir `EnrollmentEvent` oluşturur ve olayı işlemek için `processEvent` yöntemini çağırır (satırlar 22–23). Aksi takdirde, derse yeni bir öğrenci ekler (satır 25).

Bir `EnrollmentEvent` oluşturmak için

```
EnrollmentEvent(Object source, String studentToEnroll,
int enrollmentCap) yapıcısını kullanın.
```

burada `source` kaynak bileşeni belirtir.

`processEvent` metodu (satırlar 66–78) bir `EnrollmentEvent` oluşturulduğunda çağrılır. Bu, `enrollmentListenerList` içindeki dinleyicileri her dinleyicinin `enrollmentExceeded` metodunu çağırarak olayı işlemek için bilgilendirir.

Şimdi, Liste 36.3'teki test programını gözden geçirelim.  
`EnrollmentEvent/EnrollmentListener` yerine `ActionEvent/ActionListener` kullanacağız. Liste 36.7'de verilen yeni program, `CourseWithEnrollmentEvent` (satır 3) kullanarak bir kurs oluşturur, `enrollmentCap`'i 2 (satır 6) olarak ayarlar, bir kayıt dinleyicisi oluşturur (satır 7), onu kaydeder (satır 8) ve kursa üç öğrenci ekler (satırlar 9–11). Satır 11 çalıştırıldığında, `addStudent` metodu öğrenci Tim'i kursa ekler ve kurs kayıt sınırını aştığı için bir `EnrollmentEvent` tetikler. Kurs nesnesi dinleyicinin `enrollmentExceeded` metodunu olayı işlemek için çağırır ve kurstaki öğrenci sayısını ve kayıt sınırını gösterir.

#### Liste 36.7 TestCourseWithEnrollmentEvent.java

```
<margin note line 3: kurs oluştur>
<margin note line 6: kayıtLimitini ayarla>
<margin note line 7: dinleyici oluştur>
<margin note line 8: dinleyiciyi kaydet>
<margin note line 9: öğrencileri ekle>

1 genel sınıf Test KursuWithEnrollmentEvent {
2 CourseWithEnrollmentEvent kursu =
3 new CourseWithEnrollmentEvent("Java Programlama");
4
5 genel TestCourseWithEnrollmentEvent() {
6 course.setEnrollmentCap(2);
7 EnrollmentListener dinleyici = t new NewListener();
8 course.addEnrollmentListener(listener);
9 kurs.addStudent(t"John Smith");
10 kurs.addStudent(t"Jim Peterson");
11 kurs.addStudent(t"Tim Johnson");
12 }

13
14 public static void main(String[] args) {
15 new TestCourseWithEnrollmentEvent();
16 }
17
18 özel sınıf Yeni Dinleyici t Kayıt Dinleyiciyi uygular {
19 genel geçersiz kayıt Aşıldı (Kayıt Etkinliği e) {
20 System.out.println(e.getStudentToEnroll() + " kaydolmaya çalıştı. "
21 + "Kayıt limiti " + e.getEnrollmentCap());
22 }
23 }
```

22 } n 23 }  
n 24 }

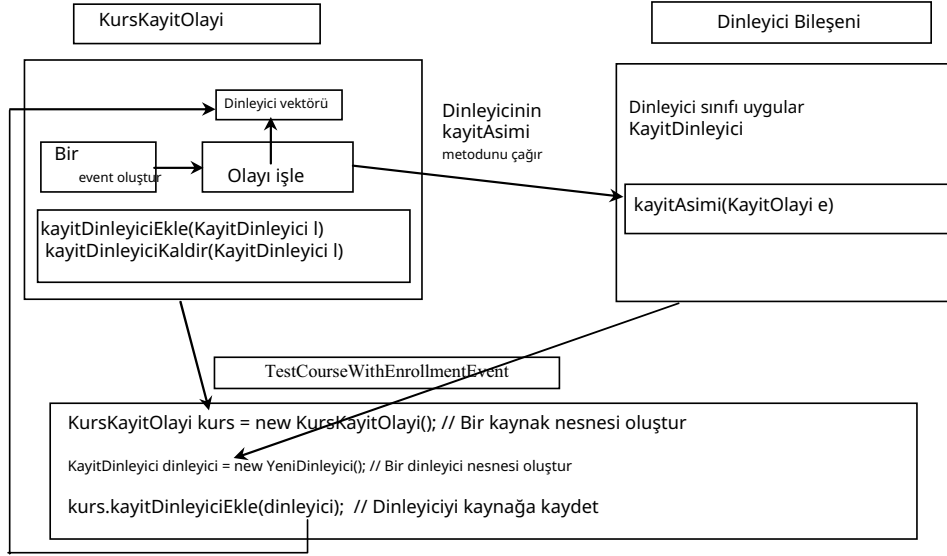
#### <Çıkış>

Tim Johnson kaydolmaya çalıştı  
Kayıt limiti 2

#### <Çıkış Sonu>

Olay işleme akışı kaynaktan dinleyiciye

Şekil 36.8'de gösterilmiştir.



#### Şekil 36.8

*Dinleyici, kaynak **kurs** ile kaydedilir ve kaynak dinleyicinin işleyicisini **kayitAsimi** olayı işlemek için çağırır.*

İPUCU

#### <kenar notu: **EylemOlayı**>

ActionEvent/ActionListener olay setini kullanmak çoğu durumda yeterlidir. Normalde, olay hakkında bilgi kaynaktan elde edilebilir. Örneğin, kurstaki öğrenci sayısı ve kayıt, bir kurs nesnesinden elde edilebilir. Kaynağı, herhangi bir olay e için e.getSource() çağırısını ya parak elde edebilirsiniz.

NOT

#### <kenar notu: **özelliklerin devralınması**>

EnrollmentEvent bileşeni sıfırdan oluşturulmuştur. Olay üretebilen bir bileşeni genişleten yeni bir bileşen oluşturursanız, yeni bileşen aynı türde olaylar üretme yeteneğini miras alır. Örneğin, JButton ja va.awt.Component sınıfının bir alt sınıfı olduğu için MouseEvent tetikleyebilir, JButton ayrıca fare olaylarını da al gılayabilir ve üretebilir.

bu olayları oluşturmak ve onlara dinleyiciler kaydetmek için kodu yazın, çünkü kod zaten üst sınıfta verilmiştir. Ancak, bileşeninizin olayları ateşleme yeteneğine sahip olması için kodu yazmanız gerekmektedir. üst sınıfta desteklenmeyen.

#### Temel Terimler

- **olay seti**
- **JavaBeans bileşeni**
- **JavaBeans olayları**
- **JavaBeans özellikleri**

#### Bölüm Özeti

1. JavaBeans, yeniden kullanılabilir yazılım bileşenleri oluşturmak için Java dilinin gücünü genişleten bir yazılım bileşen mimarisidir. JavaBeans özellikleri, fasulyenin durumunu tanımlar. Doğal olarak, veri alanları özellikleri depolamak için kullanılır. Ancak, bir fasulye özelliği mutlaka bir veri alanı değildir.
2. Bir kaynak bileşeni, dinleyicileri eklemek ve kaldırmak için uygun kayıt ve kayıttan çıkarma yöntemlerine sahip olmalıdır. Olaylar, tek yönlü (sadece bir dinleyici nesnesi olayı bildirir) veya çok yönlü (bir dinleyici listesinde her nesne olayı bildirir) olabilir.
3. Bir olay nesnesi, *ActionEvent*, *MouseEvent*, ve *ItemEvent* gibi bir olay sınıfı kullanılarak oluşturulur. Tüm olay sınıfları *java.util.EventObject* sınıfından türetilir. Her olay sınıfı, bir veya daha fazla metodu tanımlayan bir event listener arayüzü ile ilişkilidir ki bunlara *handler* denir. Bir olay dinleyici arayüzü, *java.util.EventListener* arayüzünün bir alt arayüzüdür. Bir olay sınıfı ve listener arayüzü birlikte var olduğundan, genellikle *event set* veya *event pair* olarak adlandırılır.

#### Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

#### Gözden Geçirme Soruları

##### Bölmeler 36.1-36.4

36.1

JavaBeans bileşeni nedir? Her GUI sınıfı bir JavaBeans bileşeni midir? Her GUI kullanıcı arayüzü bileşeni bir JavaBeans bileşeni midir? Bir JavaBeans bileşeninin bir GUI kullanıcı arayüzü

bileşeni olması gerektiği doğru mu?

36.2

JavaBeans bileşeninde erişimci ve değiştirici yöntemler için adlandırma kurallarını tanımlayın.

36.3

JavaBeans kayıt ve kayıttan çıkarma yöntemleri için adlandırma kurallarını tanımlayın.

36.4

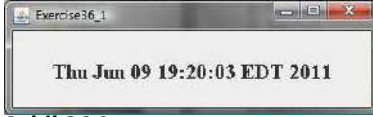
Bir olay çifti nedir? Bir olay sınıfını nasıl tanımlarsınız? Bir olay dinleyici arayüzünü nasıl tanımlarsınız?

#### Programlama Alıştırmaları

##### Bölmeler 36.1-36.6

36.1\*

(*EnableMessagePanel to fire(ActionEvent)*) MessagePanel sınıfı Listing 15.7'de JPanel'ın bir alt sınıfıdır; bir MouseEvent, KeyEvent ve ComponentEvent ateşleyebilir, ancak bir(ActionEvent) ateşleyemez. MessagePanel sınıfını, MessagePanel sınıfının bir örneği tıklandığında bir(ActionEvent) ateşleyebilecek şekilde değiştirin. Yeni sınıfın adını MessagePanelWithActionEvent olarak belirleyin. Mesaj paneline tıklandığında mevcut zamanı bir mesaj panelinde gösteren bir Java applet ile test edin; bu, Şekil 36.9'da gösterildiği gibi.



**Şekil 36.9**

Mesaj paneline tıkladığınızda mevcut zaman görüntülenir.

### 36.2\*

(Özel olay setleri ve kaynak bileşenleri oluşturun) Aşağıdaki gereksinimleri karşılayan bir proje geliştirin:

- Bir kaynak bileşeni oluşturun adlı Bellek İzleme bellek izleme için. Bileşen, boş bellek alanı belirli bir highLimit değerini aştığında veya belirli bir lowLimit değerinin altına düştüğünde bir MemoryEvent üretir. highLimit ve lowLimit, MemoryWatch içinde özelleştirilebilir özelliklerdir.
- MemoryEvent ve MemoryListener adında bir olay seti oluşturun. MemoryEvent basitçe java.util.EventObject'ı genişletir ve sistemin boş bellek ve toplam bellek değerlerini döndüren iki metod içerir: freeMemory ve totalMemory. MemoryListener arayüzü insufficientMemory ve sufficientMemory adında iki işleyici içerir.

sufficientMemory metodu, boş bellek alanı belirlenen yüksek sınırı aştığında çağrılır ve insufficientMemory, boş bellek alanı belirlenen düşük sınırın altına düştüğünde çağrılır. Sistemdeki boş bellek ve toplam bellek,

```
Çalışma zamanı çalışma zamanı = Runtime.getRuntime
e(). freeMemory();n
runtime.totalMemory();
```

- Bir MemoryEvent meydana geldiğinde boş bellek, toplam bellek ve belleğin yeterli ve yetersiz olup olmadığını gösteren bir dinleyici bileşeni geliştirin. Dinleyiciyi bağımsız çalışacak şekilde bir mainmetodu olan bir applet yapın.

### 36.3\*\*

(Hurricane kaynak bileşeni) Adı Hurricane olan bir sınıf oluşturun özellikleri adı ve kategori ve erişim yöntemleri ile. Hurricane bileşeni, kategori özelliği değiştiğinde bir(ActionEvent) üretir. Kasırga kategorisini gösteren bir dinleyici yazın. Eğer kategori 2 veya daha büyükse, bir mesaj "Kasırga Uyarısı!!!" görüntülenir, Şekil'de gösterildiği gibi.

36.10.

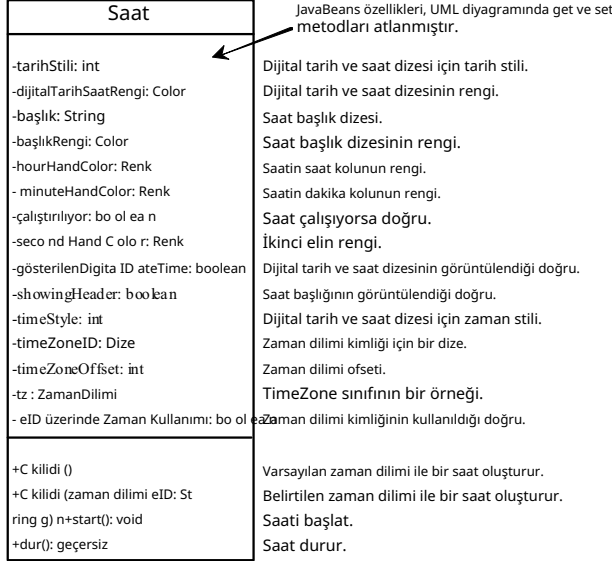


**Şekil 36.10**

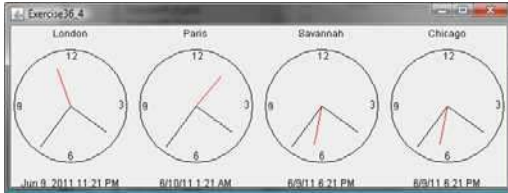
Kasırğa kategorisi değiştiğinde, mesaj panelinde uygun bir mesaj görüntülenir.

36.4 ★★

(Saat kaynak bileşeni) Analog bir saati görüntülemek için bir JavaBeans bileşeni oluşturun. Bu bean, kullanıcının özellikler aracılığıyla bir saati özelleştirmesine olanak tanır, Şekil'de gösterildiği gibi. 36.11. Şekil 36.12'de gösterildiği gibi dört saat görüntüleyen bir test programı yazın.



**Şekil 36.11**  
Saat bileşeni, analog bir saati görüntüler.



**Şekil 36.12**  
Program, Clock bileşenini kullanarak altı saat gösterir. \_\_\_\_\_

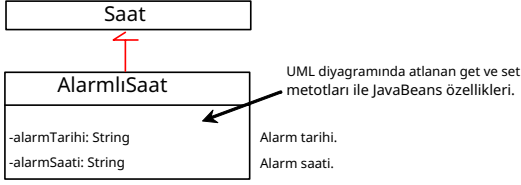
36.5 ★

(AlarmlıSaat oluşturun Saat den ) Alarm saati oluşturun, adı AlarmlıSaat, önceki alıştırmada oluşturulan Saat bileşenini genişleten, Şekil 36.13'te gösterildiği gibi. Bu bileşen iki yeni özellik içerir, alarmTarihi ve alarmSaati. alarmTarih i, yıl, ay ve günü virgüllerle ayrılmış bir dize olarak tanımlanır. Örneğin, 1998,5,13 yılı 1998, ay 5 ve gün 13'ü temsil eder. alarmSaati saatten oluşan bir dizedir,

10,45,2 dakika ve saniye, virgülle ayrılmıştır. Örneğin, 10,45,2 10 saat, 45 dakika ve 2 saniyeyi temsil eder.

Saat zamanı alarm zamanı ile eşleştğinde, ClockWithAlarm bir ActionEvent. Uyarıyı görüntüleyen bir test programı yazın

“Artık bir randevunuz var” mesajı, belirtilen bir zamanda (örneğin, tarih: 2004,1,1 ve saat: 10,30,0) bir iletişim kutusunda.

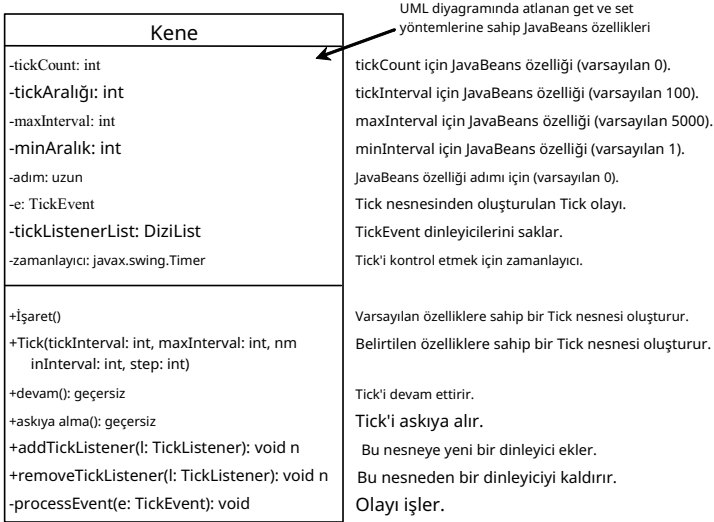


**Şekil 36.13**

Alarm fonksiyonları ile birlikte saat işlevselliğini genişleten ClockWithAlarm bileşeni.

36.6 \*\*\*

(Tick kaynak bileşeni) Farklı zaman aralıklarında tik olayları üretebilen özel bir kaynak bileşeni oluşturun, Şekil 36.14'te gösterildiği gibi. Tick bileşeni, javax.swing.Timer'a benzer. Timer sınıfı, sabit zaman aralıklarında bir zamanlayıcı oluşturur. Bu Tick bileşeni, hem değişken hem de sabit zaman aralıklarında tik olayları üretebilir.



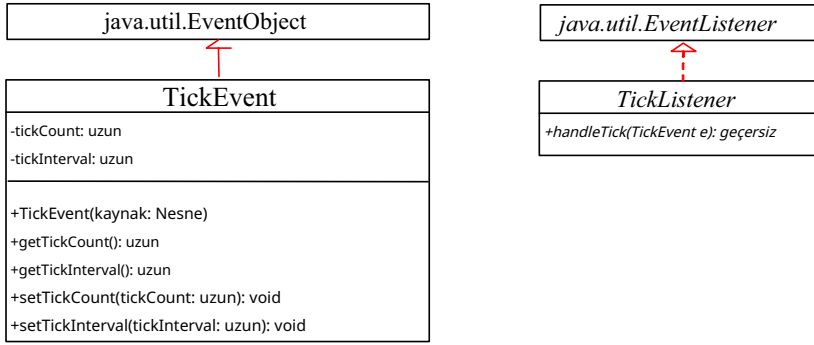
**Şekil 36.14**

Tick, TickEvent üreten bir bileşendir.

Bileşen, *tickCount*, *tickInterval*, *maxInterval*, *minInterval* ve *step* özelliklerini içerir. Bileşen, bir tick olayı meydana geldikten sonra *tickInterval*'ı *step* ekleyerek ayarlar. Eğer *step* 0 ise, *tickInterval* değişmez. Eğer *step* > 0 ise, *tickInterval* artırılır. Eğer *step* < 0 ise, *tickInterval* azaltılır. Eğer *tickInterval* > *maxInterval* veya *tickInterval* < *minInterval* ise, bileşen artık tick olayları üretmeyecektir.

Tick bileşeni, birden fazla dinleyici kaydetme, farklı zaman aralıklarında TickEvent nesneleri üretme ve dinleyicileri dinleyicilerinhandeTick yöntemini çağırarak bilgilendirme yeteneğine sahiptir. TickEvent ve TickListener için UML diyagramı Şekil 36.15'te gösterilmiştir.





**Şekil 36.15**

*TickEvent* ve *TickListener* bir tick olayı için bir olay kümesi oluşturur.

Hareketli mesajı göstermek için *DisplayMovingMessage* adında bir applet oluşturun ve named *MovingMessage* adında bir panel oluşturun. Panelin bir örneğini applet içinde yerleştirin. Mesajın sağa hareket etmesini sağlamak için, mesajı yeni bir artan *x*-koordinatı ile yeniden çizin. Bir *Tick* nesnesi kullanarak bir tick olayı oluşturun ve bir tick evi olayı gerçekleştiğinde mesajı yeniden çizmek için *repaint* yöntemini çağırın. Mesajı azalan bir hızla hareket ettirmek için, bir *Tick* nesnesi oluştururken pozitif bir adım (örneğin, 10) kullanın.

**\*\*\*Bu bir bonus Web bölümü**

## **BÖLÜM 37**

### **Kapsayıcılar, Yerleşim Yöneticileri ve Kenarlıklar**

#### **Hedefler**

- Swing kapsayıcısının iç yapısını keşfetmek (§37.2).
- Java'da bir yerleşim yöneticisinin nasıl çalıştığını açıklamak (§37.3).
- CardLayout ve BoxLayout kullanmak (§§37.3.1-37.3.2).
- Sabitleme konumunda bileşenleri yerleştirmek için mutlak yerleşim yöneticisini kullanmak (§37.3.3).
- Özel yerleşim yöneticileri oluşturmak için (§37.4).
- Kaydırılabilir paneller oluşturmak için [JScrollPane](#) kullanmak (§37.5).
- Sekme panelleri oluşturmak için [JTabbedPane](#) kullanmak (§37.6).
- Bölünmüş paneller oluşturmak için [JSplitPane](#) kullanmak (§37.7).
- Swing bileşenleri için çeşitli kenarlıklar kullanmak (§37.8).

## 37.1 Giriş

<kenar notu: konteyner> n<k  
enar notu: düzen yöneticisi>

Bölüm 12, "GUI Temelleri," kapsayıcılar ve yerleşim yöneticilerinin rolünü tanıttı. Bir kapsayıcıya bileşenleri nasıl ekleyeceğinizi ve *FlowLayout*, *BorderLayout* ve *GridLayout* kullanarak bileşenleri bir kapsayıcıda nasıl düzenleyeceğinizi öğrendiniz. Bir *kapsayıcı* bileşenleri tutan ve gruplandıran bir nesnedir. Bir *yerleşim yöneticisi* ise bileşenleri bir kapsayıcıda yerleştirmek için kullanılan özel bir nesnedir. Kapsayıcılar ve yerleşim yöneticileri, kullanıcı arayüzleri oluştururken kritik bir rol oynar. Bu bölüm, kapsayıcıları n kavramsal bir genel görünümünü sunar, Java'daki yerleşim yöneticilerini gözden geçirir ve birkaç yeni kapsayıcı ve yerleşim yöneticisini tanıtır. Ayrıca özel yerleşim yöneticileri oluşturmayı ve çeşitli kenarlıklar kullanmayı da öğreneceksiniz.

## 37.2 Swing Kapsayıcı Yapıları

JButton gibi kullanıcı arayüzü bileşenleri, bir kapsayıcıya yerleştirilmeden görüntülenemez. Bir kapsayıcı, diğer bileşenleri tutan bir bileşendir. Bir kullanıcı arayüzü bileşenini görüntüleyemezsiniz; onu bir kapsayıcıya yerleştirirsiniz ve kapsayıcı, içerdiği bileşenleri görüntüler.

Tüm kapsayıcılar için temel sınıf *java.awt.Container*'dir, bu da *java.awt.Component* sınıfının bir alt sınıfıdır. *Container* sınıfının aşağıdaki esas işlevleri vardır:

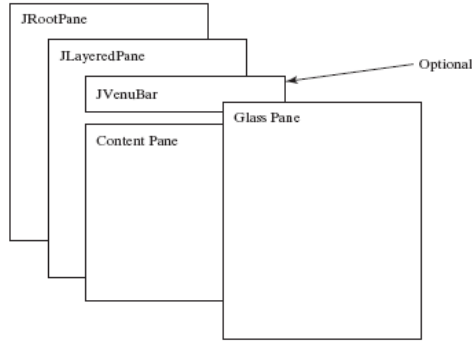
- Çeşitli *add* ve *remove* yöntemlerini kullanarak bileşenleri ekler ve kaldırır.
- Bileşenleri kapsayıcıda düzenlemek için kullanılan bir yerleşim yöneticisi belirtmek üzere bir *layout* özelliğini korur. Her kapsayıcının varsayılan bir yerleşim yöneticisi vardır.
- *java.awt.event.ContainerEvent* için kayıt yöntemleri sağlar.

AWT programlamasında, *java.awt.Frame* sınıfı Java uygulamaları için üst düzey kapsayıcı olarak kullanılır, *java.awt.Applet* sınıfı tüm Java applet'leri için kullanılır ve *java.awt.Dialog* diyalog pencereleri için kullanılır.

Bu sınıflar, Swing hafif bileşenleri ile düzgün çalışmaz.

Swing bileşenlerini barındırmak için *Frame*, *Applet* ve *Dialog* adında özel sürümler *JFrame*, *JApplet* ve *JDialog* geliştirilmiştir.

*JFrame*, *Frame*'in bir alt sınıfıdır, *JApplet*, *Applet*'in bir alt sınıfıdır ve *JDialog*, *Dialog*'ın bir alt sınıfıdır. *JFrame* ve *JApplet*, ağır karşıtlarının tüm fonksiyonlarını miras alır, ancak daha karmaşık bir iç yapıya sahiptirler ve birkaç katmanlı panel içerirler; bu, Şekil 37.1'de gösterilmiştir.



**Şekil 37.1**

### Swing üst düzey kapsayıcıları, hafif

bileşenleri gruplamak ve düzgün çalışmasını sağlamak için katmanlar kullanır.

`javax.swing.JRootPane`, Swing'in üst düzey kapsayıcıları tarafından sahne arkasında kullanılan hafif bir kapsayıcıdır, örneğin `JFrame`, `JApplet` ve `JDialog`. `javax.swing.JLayeredPane`, isteğe bağlı menü çubuğunu ve içerik panelini yöneten bir kapsayıcıdır. İçerik paneli, `Container` örneğidir. Varsayılan olarak, `JPanel` ile `BorderLayout`'dir.

Bu, kullanıcı arayüzü bileşenlerinin eklendiği kapsayıcıdır.

Bir `JFrame` veya `JApplet` içindeki içerik panelini almak için `getContentPane()` yöntemini kullanın. Eğer bir `Container`ı içerik paneli olarak ayarlamak istiyorsanız, `setContentPane()` yöntemini kullanın. Cam panel her şeyin üstünde yüzer. `javax.swing.JGlassPane` varsayılan olarak gizli bir paneldir. Eğer cam paneli görünür hale getirirseniz, o zaman kök panelin diğer tüm parçalarının üzerinde bir cam levha gibidir. Tamamen şeffaftır, eğer cam panelin `paint` yöntemini uygulamazsanız ve bir şey boyamazsanız, kök panel için giriş olaylarını yakalar. Genel olarak, `JRootPane`, `JLayeredPane` ve `JGlassPane` doğrudan kullanılmaz.

Şimdi en sık kullanılan üç Swing kapsayıcısını gözden geçirelim:

`JFrame`, `JApplet`, ve `JPanel`.

#### 37.2.1t JFrame

`JFrame`, `Frame`'in bir Swing versiyonudur, Java grafik uygulamaları için üst düzey bir kapsayıcıdır. `Frame` gibi, `JFrame` başlık çubuğu ve kenarlık ile bağımsız bir pencere olarak görüntülenir. Aşağıdaki özellikler `JFrame`'de sıklıkla faydalıdır:

- `contentPane` çerçevenin içerik panelidir.
- `iconImage` çerçeveyi temsil eden görüntüdür. Bu görüntü çerçevenin başlık çubuğundaki varsayılan Java görüntüsünün yerini alır ve çerçeve küçültüldüğünde de görüntülenir. Bu özellik türü `Image`. Aşağıdaki gibi `ImageIcon` sınıfını kullanarak bir görüntü alabilirsiniz:

```
Image image = (new ImageIcon(filename)).getImage();
```

- `jMenuBar` çerçeve için isteğe bağlı menü çubuğudur.

- `resizable` çerçevenin yeniden boyutlandırılıp boyutlandırılmayacağını belirtir bir `boolean` değerdir. Varsayılan değer `true`'dur.
- `title` çerçevenin başlığını belirtir.

### 37.2.2t JApplet

`JApplet` bir Applet'in `Swing` versiyonudur. Applet'in bir alt sınıfı olduğu için, Web tarayıcısı tarafından gereken tüm işlemlere sahiptir. İşte `Applet`'te tanımlanan dört temel yöntem:

```
// Bu applet'i içeren Web sayfası ilk yüklendiğinde tarayıcı tarafından çağrılır
genel voidt init()
```

```
// Tarayıcı tarafından init() yönteminden sonra ve
// Web sayfası her ziyaret edildiğinde çağrılır.
genel voidt start()
```

```
// Tarayıcı tarafından bu
// applet'in bulunduğu sayfa etkin olmadığında çağrılır.
genel voidt stop()
```

```
// Tarayıcı tarafından Web tarayıcısı kapandığında çağrılır.
genel voidt yok etme()
```

Ayrıca, `JApplet` `ContentPane` ve `JMenuBar` özelliklerine sahiptir, başka özelliklerin yanı sıra. `JFrame`'de olduğu gibi, bileşenleri doğrudan `JApplet`'e yerleştirmezsiniz; bunun yerine, bileşenleri applet'in içerik paneline yerleştirirsiniz. `Applet` sınıfının menü çubuğu olamaz, ancak `JApplet` sınıfı menü çubuğunu `setJMenuBar` yöntemiyle ayarlamanıza izin verir.

NOT: Bir applet yüklendiğinde, Web tarayıcısı applet'in no-arg constructor'ını çağırarak bir applet örneği oluşturur. Bu nedenle, constructor i nityönteminden önce çağrılır.

### zsht.2.zt Ypanel

Pano, kullanıcı arayüzü bileşenlerini gruplamak için alt kapsayıcılar olarak işlev görür. `javax.swing.JPanel` `JFrame` ve `JApplet`. ile farklıdır. İlk olarak, `JPanel` üst düzey bir kapsayıcı değildir; başka bir kapsayıcı içinde yer alması gerekir ve başka bir `JPanel` içinde de yer alabilir. İkinci olarak, `JPanel` `JComponent` sınıfının bir alt sınıfı olduğundan, hafif bir bileşendir, ancak `JFrame` ve `JApplet` ağır bileşenlerdir.

`JComponent`'un bir alt sınıfı olarak, `JPanel`, `JComponent`'in avantajlarından yararlanabilir, double buffering ve kenarlıklar gibi. `JFrame` veya `JApplet` yerine `JPanel` üzerinde şekiller çizmелisiniz, çünkü `JPanel` double buffering'i destekler, bu da titremeleri ortadan kaldırma tekniğidir.

### 37.3 Yerleşim Yöneticileri

Her kapsayıcının bileşenlerini düzenlemekten sorumlu bir yerleşim yöneticisi vardır. Kapsayıcının `setLayout` yöntemi bir yerleşim yöneticisi ayarlamak için kullanılabilir. Belirli türdeki kapsayıcılar varsayılan yerleşim yöneticilerine sahiptir. Örneğin, `JFrame` veya `JApplet`'in içerik paneli `BorderLayout` kullanır ve `JPanel` `FlowLayout` kullanır.

Yerleşim yöneticisi, bileşenleri kendi kurallarına ve özellik ayarlarına göre ve her bileşenle ilişkili kısıtlamalarla yerleştirir. Her yerleşim yöneticisinin kendine özgü bir kural seti vardır. Örneğin, *FlowLayout* yöneticisi bileşenleri soldan sağa doğru satırlara yerleştirir ve önceki satır dolduğunda yeni bir satır başlatır. *BorderLayout* yöneticisi bileşenleri kapsayıcının kuzeyine, güneyine, doğusuna, batısına veya ortasına yerleştirir. *GridLayout* yöneticisi bileşenleri soldan sağa doğru satır ve sütunlar halinde hücreler içinde yerleştirir.

Bazı yerleşim yöneticileri, kapsayıcıdaki bileşenlerin boyutunu ve konumunu etkileyebilecek özelliklere sahiptir. Örneğin, *BorderLayout*, bileşenler arasındaki yatay ve dikey mesafeyi belirleyen *hgap* (yatay boşluk) ve *vgap* (dikey boşluk) olarak adlandırılan özelliklere sahiptir. *FlowLayout* bileşenlerin hizalamasını (sol, merkez, sağ) belirtmek için kullanılabilecek özelliklere ve bileşenler arasındaki yatay veya dikey boşluğu belirtmek için özelliklere sahiptir.

*GridLayout* sütunlar ve satırlar arasındaki yatay veya dikey boşluğu belirtmek için kullanılabilecek özelliklere ve satır ve sütun sayısını belirtmek için özelliklere sahiptir. Bu özellikler, erişimci ve değiştirici yöntemleri kullanılarak alınabilir ve ayarlanabilir.

Bir bileşenin bir kapsayıcı içindeki boyutu birçok faktör tarafından belirlenir, şunlar gibi:

- Kapsayıcının kullandığı yerleşim yöneticisinin türü.
- Her bileşenle ilişkili yerleşim kısıtlamaları.
- Kapsayıcının boyutu.
- Tüm bileşenlere ortak olan belirli özellikler (örneğin, *preferredSize*, *minimumSize*, *maximumSize*, *alignmentX* ve *alignmentY*).

*preferredSize* özelliği, bileşenin en iyi görüldüğü ideal boyutu belirtir. Belirli bir yerleşim yöneticisinin kurallarına bağlı olarak, bu özellik dikkate alınabilir veya alınmayabilir. Örneğin, bir bileşenin tercih edilen boyutu, *FlowLayout* yöneticisine sahip bir kapsayıcıda kullanılır, ancak *GridLayout* yöneticisine sahip bir kapsayıcıda yerleştirildiğinde göz ardı edilir.

*minimumSize* özelliği, bileşenin faydalı olduğu minimum boyutu belirtir. Çoğu GUI bileşeni için, *minimumSize*, *preferredSize* ile aynıdır. Yerleşim yöneticileri genellikle *minimumSize*'ı *preferredSize*'dan daha fazla dikkate alır.

*MaximumSize* özelliği, bir bileşenin ihtiyaç duyduğu maksimum boyutu belirtir, böylece yerleşim yöneticisi, ihtiyaç duymayan bir bileşene gereksiz yere alan vermemiş olur. Örneğin, *BorderLayout*, merkez bileşenin boyutunu maksimum boyutuyla sınırlar ve alanı kenar bileşenlerine verir.

Sadece *alignmentX* (*alignmentY*) özelliği, bileşenin diğer bileşenlere göre nasıl hizalanmak istediğini belirtir *x* eksenini (*y*-ekseni) boyunca. Bu değer, 0 ile 1 arasında bir sayı olmalıdır; burada 0, orijinal hizalamayı, 1 ise en uzak hizalamayı temsil eder.

orijinden, *0.5* merkezlenmiştir ve benzeri. Bu iki özellik *BoxLayout* ve *OverlayLayout*'da kullanılır.

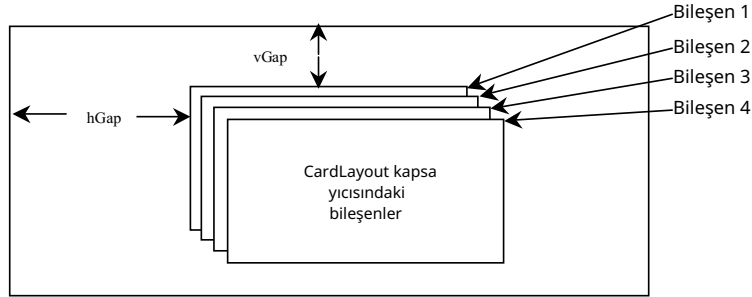
Java, çeşitli yerleşim yöneticileri sunar. *BorderLayout*, *FlowLayout* ve *GridLayout* kullanmayı öğrendiniz. Takip eden bölümler *CardLayout*, *Null* yerleşimi ve *BoxLayout*'ı tanıtmaktadır. *GridBagLayout*, *OverlayLayout* ve *SpringLayout* Ek III.S'de sunulmuştur.

İPUCU: Bir kapsayıcıda yeni bir yerleşim yöneticisi ayarlarsanız, kapsayıcının *validate()* yöntemini çağırarak kapsayıcının bileşenleri yeniden yerleşmesini zorlayın. Bir *JFrame* veya *JApplet*'deki bir yerleşim yöneticisinin özelliklerini değiştirerseniz, kapsayıcının bileşenleri yeni yerleşim özelliklerini kullanarak yeniden yerleşmesini zorlamak için *doLayout()* yöntemini çağırın. Bir *JPanel*'deki bir yerleşim yöneticisinin özelliklerini değiştirerseniz, bileşenleri yeni yerleşim özelliklerini kullanarak yeniden yerleşmesi için ya *doLayout()* ya da *revalidate()* yöntemini çağırın, ancak *revalidate()* kullanmak daha iyidir. *validate()* yönteminin *java.awt.Container*'da tanımlanmış bir genel yöntem olduğunu, *revalidate()* yönteminin *javax.swing.JComponent*'da tanımlanmış bir genel yöntem olduğunu ve *doLayout()* yönteminin *java.awt.Container*'da tanımlanmış bir genel yöntem olduğunu unutmayın.

### 37.3.1 KartYerleşimi

*KartYerleşimi* bileşenleri kapsayıcıda kartlar olarak yerleştirir. Aynı anda yalnızca bir kart görünür ve kapsayıcı bir kart yığını gibi davranır. Kartların sıralaması, kapsayıcının kendi bileşen nesnelerinin içsel sıralaması tarafından belirlenir. Bileşen yığınına çevreleyen yatay ve dikey boşlukların boyutunu belirtebilirsiniz.

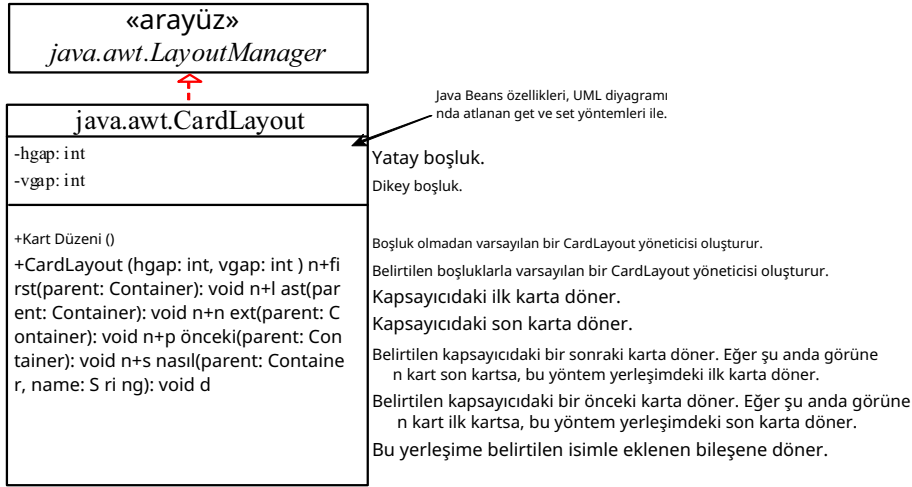
*CardLayout* yöneticisi, Şekil 37.2'de gösterildiği gibi.



Şekil 37.2

*CardLayout* bileşenleri kapsayıcıda kart yığını olarak yerleştirir.

*CardLayout* bir uygulamanın kartları sırasıyla çevirmesine veya belirli bir kartı doğrudan görüntülemesine olanak tanıyan bir dizi yöntem tanımlar; bu, Şekil 37.3'te gösterilmiştir.



**Şekil 37.3**

*CardLayout, kartı döndürmek için yöntemleri içerir.*

Bir bileşeni bir kapsayıcıya eklemek için, LayoutManager arayüzünde tanımlanan add(Component c, String name) yöntemini kullanın. String parametresi, name, bileşene kapsayıcıda açık bir kimlik verir.

Liste 37.1, bir çerçevede iki panel oluşturan bir programı vermektedir. İlk panel, görüntüleri göstermek için altı etiket tutmak üzere *CardLayout* kullanılmaktadır. İkinci panel, *FlowLayout* kullanarak *First*, *Next*, *Previous* ve *Last* adında dört düğmeyi ve *Image* etiketli bir açılır kutuyu gruplandırmaktadır; bu, Şekil 37.4'te gösterilmiştir.

Bu düğmeler, *CardLayout* panelinde hangi görüntünün gösterileceğini kontrol eder. Kullanıcı, *First* adındaki düğmeye tıkladığında, *CardLayout* panelindeki ilk görüntü görünür. Kombinasyon kutusu, kullanıcının doğrudan bir görüntü seçmesini sağlar.



**Şekil 37.4**

*Program, CardLayout olan bir panelde görüntüleri gösterir.*

#### **Listeleme 37.1 ShowCardLayout.java**

<margin note line 6: kart yerleşimi>  
<margin note line 13: UI oluştur>  
<margin note line 41: dinleyiciyi kaydet>  
<margin note line 45: ilk bileşen>



<margin note line 47: dinleyiciyi kaydet>  
 <margin note line 52: sonraki bileşen>  
 <margin note line 55: dinleyiciyi kaydet>  
 <margin note line 59: önceki bileşen>  
 <margin note line 62: dinleyiciyi kaydet>  
 <margin note line 66: son bileşen>  
 <margin note line 69: dinleyiciyi kaydet>  
 <margin note line 77: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 genel sınıf ShowCard Düzeni t Uygulamayı genişletir {
6 özel CardLayout kartYerleşimi = yeni CardLayout (20, 10) ;
7 özel JPanel kartPanel = yeni JPanel(kartYerleşimi);
8 özel JButton jbtİlk, jbtSonraki, jbtÖnceki, jbtSon;
9 özel JComboBox jcboResim;
10 özel son int BAYRAK_SAYISI = 6;
11
12 kamusal ShowCardLayout () {
13 kartPanel.setBorder(
14 yeni javax.swing.border.LineBorder(Color.kırmızı));
15
16 // kartPanel'e görüntüleri göstermek için 9 etiket ekleyin
17 için (int i = 1; i <= BAYRAK_SAYISI; i++) {
18 JLabel etiket =
19 yeni JLabel (new ImageIcon ("image/flag" + i + ".gif"));
20 kartPanel.add(etiket, String.valueOf(i));
21 }
22
23 // Düğmeleri ve bir kombinasyon kutusunu tutacak panel p
24 JPanel p = new JPanel ();
25 p.add(jbtFirst = new JButton ("İlk"));
26 p.add(jbtNext = new JButton ("Sonraki"));
27 p.add(jbtPrevious= new JButton ("Önceki"));
28 p.add(jbtLast = new JButton ("Son"));
29 p.add(new JLabel ("Resim"));
30 p.add(jcboImage = new JComboBox ());
31
32 // Kombinasyon kutusu öğelerini başlat
33 kale (tintt i = 1; i <= NUM_OF_FLAGS; i++)
34 jcboImage.addItem (String.valueOf (i));
35
36 // Panelleri çerçeveye yerleştir
37 add(cardPanel, BorderLayout.CENTER); n 38 add(p, B
orderLayout.SOUTH);
39
40 // Kaynak nesnelerle dinleyicileri kaydet
41 jbtFirst.addActionListener(tnewt ActionListener() { n 42 @Override
42
43 public void actionPerformed(ActionEvent e) {
44 // kartPanel'deki ilk bileşeni göster
45 cardLayout.first(cardPanel); n 46 }
46
47 });
48 jbtNext.addActionListener(tnewt ActionListener() { n 49 @Override
49
50 public void actionPerformed(ActionEvent e) {

```

```

51 // kartPanel'deki ilk bileşeni göster
52 kartLayout.next(cardPanel); n 53 }

54 });
55 jbtPrecious.addActionListener(tnew ActionListener() { n 56 @Override

57 public void actionPerformed(ActionEvent e) {
58 // kartPanel'deki ilk bileşeni göster
59 kartLayout.önceki(cardPanel); n 60 }

61 });
62 jbtLast.addActionListener(tnew ActionListener() { n 63 @Override

64 public void actionPerformed(ActionEvent e) {
65 // kartPanel'deki ilk bileşeni göster
66 kartLayout.last(cardPanel); n 67 }

68 });
69 jcbImage.addItemListener(tnew ItemListener() { n 70 @Override
e
71 public void itemStateChanged(ItemEvent e) {
72 // Belirtilen indeksteki bileşeni göster
73 kartLayout.show(cardPanel, (String)e.getItem()); n 74 }

75 });
76 }
77 }

```

6. satırda bir `CardLayout` örneği oluşturulmuş ve 7. satırda bir `CardLayout` paneli oluşturulmuştur. Bu tür ifadeleri `as setLayout(new FlowLayout())` kullanarak anonim bir yerleşim nesnesi oluşturmak ve bir kapsayıcı için yerleşimi ayarlamak için zaten kullanırsınız, bu programda olduğu gibi yerleşim yöneticisinin ayrı bir örneğini oluşturmak yerine. Ancak, `cardLayout` nesnesi, programın ilerleyen bölümlerinde `cardPanel` içindeki bileşenleri göstermek için faydalıdır. Örneğin, `cardLayout.first(cardPanel)` (45. satır) kullanarak `cardPanel` içindeki ilk bileşeni görüntülemeniz gerekir.

18-20. satırdaki ifade, kimlik ile birlikte resim etiketini ekler `String.valueOf(i)`. Daha sonra, kullanıcı `i` numaralı bir resmi seçtiğinde, belirtilen kimliği görüntülemek için `cardLayout.show()` metodu (73. satır) içinde `String.valueOf(i)` kullanılır.

### 37.3.2t Kutu Düzeni

`javax.swing.BoxLayout` bileşenleri bir satır veya sütun halinde düzenleyen bir Swing yerleşim yöneticisidir. Bir `BoxLayout` oluşturmak için, aşağıdaki yapıyı kullanın:

```
public BoxLayout(Container target, int axis)
```

Bu yapı, diğer yerleşim yapılarından farklıdır. Verilen hedef kapsayıcıya özel bir yerleşim yöneticisi oluşturur. `axis` parametresi `BoxLayout.X_AXIS` veya `BoxLayout.Y_AXIS`'dir, bileşenlerin yatay veya dikey olarak düzenlenip düzenlenmeyeceğini belirtir. Örneğin, aşağıdaki kod, `p1` için yatay bir `BoxLayout` oluşturur:

```
JPanel p1 = new JPanel();
```

```
BoxLayout boxLayout = yeni BoxLayout(p1, BoxLayout.X_AXIS); np1.setLa
yout(boxLayout);
```

Yerleşim yöneticisini ayarlamak için **p1** üzerinde **setLayout** metodunu çağırmanız gerekir.

**BoxLayout** herhangi bir kapsayıcıda kullanılabilir, ancak **Box** sınıfını kullanmak daha basittir; bu, **BoxLayout**'un bir kapsayıcısıdır. Bir **Box** kapsayıcısı oluşturmak için, aşağıdaki iki statik yöntemden birini kullanın:

```
Box box1 = Box.createHorizontalBox();
Box box2 = Box.createVerticalBox();
```

İlki, bileşenleri yatay olarak içeren bir kutu oluşturur, ikincisi ise bileşenleri dikey olarak içeren bir kutudur. Bir kutuya bileşen eklemek, **FlowLayout** veya **GridLayout** kapsayıcılarına eklemekle aynı şekilde, **add** yöntemini kullanarak yapılır, şu şekilde:

```
box1.add(new JButton("Bir Düğme"));
```

Bir kutudan bileşenleri, bir kapsayıcıya bileşen bırakmakla aynı şekilde kaldırabilirsiniz. Bileşenler, yatay bir kutuda soldan sağa, dikey bir kutuda ise yukarıdan aşağıya yerleştirilir.

**BoxLayoutGridLayout** ile benzerlik gösterir ancak birçok benzersiz özelliğe sahiptir. Öncelikle, **BoxLayout** bir bileşenin tercih edilen boyutunu, maksimum boyutunu ve minimum boyutunu dikkate alır. Kutudaki tüm bileşenlerin toplam tercih edilen boyutu kutu boyutundan küçükse, bileşenler maksimum boyutlarına kadar genişletilir. Kutudaki tüm bileşenlerin toplam tercih edilen boyutu kutu boyutundan büyükse, bileşenler minimum boyutlarına kadar küçültülür. Bileşenler minimum genişlikleri ne sığmıyorsa, bazıları gösterilmeyecektir.

**GridLayout**, kapsayıcıyı eşit boyutlu hücrelere böler ve bileşenler, tercih edilen maksimum veya minimum boyutlarına bakılmaksızın yerleştirilir.

İkincisi, diğer yerleşim yöneticilerinin aksine, **BoxLayout** bileşenin **alignmentX** veya **alignmentY** özelliğini dikkate alır. **alignmentX** özelliği, bileşeni dikey bir kutu yerleşiminde yerleştirmek için kullanılır ve **alignmentY** özelliği, onu yatay bir kutu yerleşiminde yerleştirmek için kullanılır.

Üçüncüsü, **BoxLayout** bileşenler arasında boşluklar bulundurmaz, ancak bileşenleri ayırmak için dolgu elemanları kullanabilirsiniz. Bir dolgu elemanı görünmez bir bileşendir. Üç tür dolgu elemanı vardır: **strut**, **sert alanlar** ve **yapıştırıcılar**.

#### <yan açıklama: dikme>

Bir **strut** basitçe bileşenler arasında biraz boşluk ekler. **Box** sınıfındaki **createHorizontalStrut(int)** statik yöntemi, yatay bir **strut** oluşturmak için kullanılır ve **createVerticalStrut(int)** statik yöntemi, dikey bir **strut** oluşturmak için kullanılır. Örneğin, aşağıda gösterilen kod, dikey bir kutudaki iki düğme arasında 8 piksel genişliğinde bir dikey **strut** ekler.

```
box2.add(tnewt JButton("Düğme 1"));nbo
x2.add(Box.createVerticalStrut(8));nbox2.add(
tnewt JButton("Düğme 2"));
```

#### <yan açıklama: sert alan>

Bir **rigid area**

**Box** sınıfındaki **createRigidArea(dimension)** statik yöntemi kullanılarak oluşturulabilen iki boyutlu bir alandır.

örnek, bir kutuya 10 piksel genişliğinde ve 20 piksel  
high bir sert alan ekleyen bir kodu gösterir.

```
box2.add(Box.createRigidArea(tnewt Dimension(t10t,t 20t));
```

**<yan not: yapıştırıcı>**

Bir *yapıştırıcı* bileşenleri mümkün olduğunca ayırır. Örneğin, yatay bir kutudaki iki bileşen arasında bir yapıştırıcı ekleyerek, bir bileşeni sol uçta ve diğerini sağ uçta yerleştirirsiniz. Bir yapıştırıcı, `Box.createGlue()` yöntemi kullanılarak oluşturulabilir.

Liste 37.2, bir yatay kutu ve bir dikey kutu oluşturan bir örneği gösterir. Yatay kutu, yazdır ve kaydet simgeleri olan iki düğme tutar. Dikey kutu, bayrakları seçmek için dört düğme tutar. Dikey kutudaki bir düğmeye tıklandığında, ilgili bayrak simgesi, Şekil 37.5'te gösterildiği gibi, applet'teki ortalanmış etiketin içinde görüntülenir.



Şekil 37.5

Bileşenler, `BoxLayout` kapsayıcılarının içine yerleştirilmiştir.

**Liste 37.2 ShowBoxLayout.java**

**<kenar notu satır 6: UI bileşenleri>**  
**<kenar notu satır 7: `BoxLayout` kapsayıcı>**  
**<kenar notu satır 8: `BoxLayout` kapsayıcı>**  
**<kenar notu satır 14: simgeleri oluştur>**  
**<kenar notu satır 28: düğmeler>**  
**<kenar notu satır 34: UI oluştur>**  
**<kenar notu satır 35: kutuya ekle>**  
**<kenar notu satır 79: ana yöntem atlandı>**

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 genel sınıf ShowBox Düzeni t Uygulamayı genişletir {
6 // İki kutu kapsayıcı oluştur
7 özel Kutu box1 = Box.createHorizontalBox();
8 özel Kutu box2 = Box.createVerticalBox();
9
10 // Bayrakları göstermek için bir etiket oluştur
11 özel JLabel jbl Flag = t newt JLabel();
12
13 // Bayraklar için resim simgeleri oluştur
14 özel Resim Simgesi ImageIconUS =
15 newt ImageIcon(getClass().getResource("image/us.gif"));
16 özel Resim Simgesi resim Simgesi Kanada =
17 yeni ImageIcon(getClass().getResource("/image/ca.gif"));
18 özel ImageIcon imageIconNorway =
19 yeni ImageIcon(getClass().getResource("/image/norway.gif"));
```

```

20 özel ImageIcon imageIconGermany =
21 yeni ImageIcon(getClass().getResource("/image/germany.gif"));
22 özel ImageIcon imageIconPrint =
23 yeni ImageIcon(getClass().getResource("/image/print.gif"));
24 özel ImageIcon imageIconSave =
25 yeni ImageIcon(getClass().getResource("/image/save.gif"));
26
27 // Görselleri seçmek için butonlar oluşturun
28 özel JButton jbtUS = t newt JButton(t"US"t);
29 private JButton jbtCanada = new JButton("Kanada");
30 private JButton jbtNorway = new JButton("Norveç");
31 private JButton jbtGermany = new JButton("Almanya");
32
33 genel ShowBox Düzeni() {
34 box1.add(tnewt JButton(imageIconPrint)); n 35 box1.ad
35 d(Box.createHorizontalStrut(t20t)); n 36 box1.add(tnewt JButt
36 on(imageIconSave));
37
38 box2.add(jbtUS); n 39 box2
39 .add(Box.createVerticalStrut(
40 8));
41 box2.add(jbtCanada); 41 box2.a
42 dd(Box.createGlue()); n 42 box2.add(jbtNorv
43 eç);
44 box2.add(Box.createRigidArea(tnewt Dimension(t10t,t 8t));n 44 box2.ad
45 d(jbtAlmanya);
46
47 box1.setBorder (yeni javax.swing.border.LineBorder(Color.kırmızı));
48 box2.setBorder(tnewt javax.swing.border.LineBorder(Color.black));
49
50 add(box1, BorderLayout.NORTH); n 50 add(b
51 ox2, BorderLayout.EAST); n 51 add(jlblFlag, Bor
52 derLayout.CENTER);
53
54 // Dinleyicileri kaydet
55 jbtUS.addActionListener(tnewt ActionListener() { n 55 @Override
56
57 public void actionPerformed(ActionEvent e) {
58 jlblFlag.setIcon(imageIconUS); n 58 }
59
60 });
61 jbtCanada.addActionListener(tnewt ActionListener() { n 61 @Override
62
63 public void actionPerformed(ActionEvent e) {
64 jlblFlag.setIcon(imageIconCanada); n 64 }
65
66 });
67 jbtNorway.addActionListener(tnewt ActionListener() { n 67 @Override
68
69 public void actionPerformed(ActionEvent e) {
70 jlblFlag.setIcon(imageIconNorveç); n 70 }
71
72 });
73 jbtAlmanya.addActionListener(tnewt ActionListener() { n 73 @Override
74
75 public void actionPerformed(ActionEvent e) {
76 jlblFlag.setIcon(imageIconAlmanya); n 76 }
77 }); n 78 }

```

İki `Box` sınıfı kapsayıcısı 7-8. satırlarda kolay statik yöntemler `createHorizontalBox()` ve `createVerticalBox()` kullanılarak oluşturulmuştur. Kutu kapsayıcıları her zaman `BoxLayout` yöneticisini kullanır. Kutu kapsayıcıları için yerleşim yöneticisini sıfırlayamazsınız.

Görüntü simgeleri, Bölüm 18.10'da tanıtilan resource URL üzerinden, görüntü dosyalarından (satırlar 14-25) oluşturulmuştur, "Kaynağı URL Sınıfı Kullanarak Bulma."

Yatay kutuya yazdırma ve kaydetme simgeleri olan iki düğme eklenmiştir (satırlar 34-36). Bu iki düğme arasında boyutu `20` olan yatay bir strut eklenmiştir (satır 35).

ABD, Kanada, Norveç ve Almanya metinlerine sahip dört düğme dikey kutuya eklenmiştir (satırlar 38-44). ABD düğmesi ile Kanada düğmesi arasında ayırıcı olarak boyutu `8` olan yatay bir strut eklenmiştir (satır 39). Norveç düğmesi ile Almanya düğmesi arasında bir sert alan eklenmiştir (satır 43). Kanada düğmesi ile Norveç düğmesi arasında mümkün olduğunca uzakta durması için bir yapıştırıcı eklenmiştir.

Strut, sert alan ve yapıştırıcı, `Component` örnekleridir, bu nedenle bokslara eklenebilirler. Teorik olarak, bunları boks kapsayıcısı dışındaki bir kapsayıcıya ekleyebilirsiniz. Ancak, diğer kapsayıcılarda göz ardı edilebilirler ve hiçbir etkisi olmayabilir.

### 37.3.3 Null Yerleşim Yöneticisi Kullanımı

Windows tabanlı bir görsel form tasarım aracı olan Visual Basic'i kullandıysanız, kullanıcı arayüzlerini Java'dan daha kolay oluşturduğunuzu bilirsiniz. Bunun başlıca nedeni, Visual Basic'te öğelerin mutlak konum ve boyutlarda yerleştirilmesidir, oysa Java'da öğeler çeşitli yerleşim yöneticileri kullanılarak kapsayıcılara yerleştirilir. Mutlak konumlar ve boyutlar, uygulama aynı platformda geliştirilip yayınlandığında iyidir, ancak bir geliştirme sisteminde iyi görünen bir şey, bir dağıtım sisteminde doğru görünmeyebilir. Bu sorunu çözmek için, Java, öğeleri yazı tiplerinden, ekran çözünürlüklerinden ve platform farklılıklarından bağımsız bir şekilde kapsayıcılara yerleştiren bir dizi yerleşim yöneticisi sağlar.

Kolaylık sağlamak için, Java ayrıca, bileşenleri sabit konumlarda yerleştirmenizi sağlayan null yerleşim yöneticisi olarak adlandırılan bir mutlak yerleşimi destekler. Bu durumda, bileşen,

bileşenin örnek yöntemi `setBounds()` (java.awt.Component içinde tanımlıdır) kullanılarak yerleştirilmelidir, şu şekilde:

**public void setBounds(int x, int y, int genişlik, int yükseklik);**

Bu, bileşenin konumunu ve boyutunu ayarlar, bir sonraki

örnekte olduğu gibi:

```
JButton jbt = new JButton("Yardım");
jbt.setBounds(10, 10, 40, 20);
```

Yardım butonunun sol üst köşesi (10, 10) konumuna yerleştirilmiştir; butonun genişliği `40`, yüksekliği ise `20`'dir.

İşte bir bileşeni null

yerleşim yöneticisi ile bir kapsayıcıya ekleme adımları:

1. Null bir yerleşim yöneticisi belirtmek için bu ifadeyi kullanın:

<kenar notu: boş düzeni ayarla>

```
konteyner.setLayout(null);
```

2. Bileşeni kapsayıcıya ekleyin:

```
JButton jbt = new JButton("Yardım");
container.add(jbt);
```

3. Bileşenin yerleştirileceği yeri belirtin,

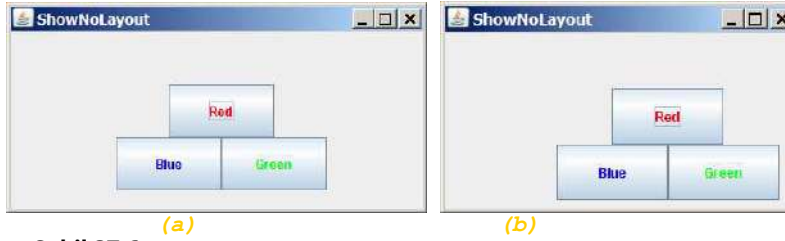
`setBounds` yöntemini kullanarak:

<kenar notu: `t.setBounds`>

```
jbt.setBounds(t10t,t 10t,t 40t,t 20t);
```

Liste 37.3, üç düğmeyi yerleştiren bir programı verir,

Şekil 37.6'a'da gösterildiği gibi.



Şekil 37.6

(a) Bileşenler, bir null yerleşim yöneticisi kullanılarak çerçeveye yerleştirilmiştir. (b) Null yerleşim yöneticisi ile, bileşenlerin boyutları ve konumları sabittir.

#### Liste 37.3 ShowNoLayout.java

<margin note line 5: UI bileşenleri>

<margin note line 11: UI oluştur>

<margin note line 28: ana yöntem atlandı>

```
1 java.awt.*'u içe aktarın;
2 javax.swing.*'i içe aktarın;
3
4 genel sınıf Gösteri Düzeni t Uygulamayı genişletir {
5 private JButton jbtRed = new JButton("Kırmızı");
6 private JButton jbtBlue = new JButton("Mavi");
7 private JButton jbtGreen = new JButton("Yeşil");
8
9 genel Gösteri Düzeni() {
10 // Düğmeler için ön plan rengini ayarla
11 jbtRed.setForeground(Color.RED); n 12 jbtBlue.
setForeground(Color.BLUE); n 13 jbtGreen.setForegr
ound(Color.GREEN);
14
15 // Yerleşim yöneticisi belirtme
16 setLayout(null);
17
18 // Bileşenleri kapsayıcıya ekle
19 ekle(jbtRed);
```

```

20 ekle(jbtBlue); n 21 ekle(
jbtGreen);
22
23 // Bileşenleri doğru yere yerleştir
24 jbtRed.setBounds(t150t,t 50t,t 100t,t 50t);n 25 jbtBl
ue.setBounds(t100t,t 100t,t 100t,t 50t);n 26 jbtGreen.set
Bounds(t200t,t 100t,t 100t,t 50t);n 27 }
28 }

```

Eğer bu programı 1024 × 768 çözünürlüğünde Windows'ta çalıştırırsanız, yerleşim boyutu tam yerindedir. Program daha yüksek bir çözünürlükte Windows'ta çalıştırıldığında, bileşenler çok küçük görünür ve kümelenir. birlikte. Daha düşük bir çözünürlükte Windows'ta çalıştırıldığında, tamamı gösterilmez.

Pencereyi yeniden boyutlandırırsanız, bileşenlerin konumunun ve boyutunun değişmediğini göreceksiniz; bu, Şekil 37.6b'de gösterilmiştir.

**İPUCU**

Platform bağımsız uygulamalar geliştirmek için null-layout-manager kullanmayın.

### 37.4 Özel Yerleşim Yöneticileri Oluşturma

Java'da sağlanan yerleşim yöneticilerine ek olarak, kendi yerleşim yöneticilerinizi oluşturabilirsiniz. Bunu yapmak için, bir yerleşim yöneticisinin bileşenleri nasıl yerleştirdiğini anlamanız gerekir. Bir kapsayıcının *setLayout* yöntemi, kapsayıcı için bir yerleşim yöneticisi belirtir. Yerleşim yöneticisi, bileşenleri yerleştirmekten ve bunları islenen bir konumda uygun bir boyutla görüntülemekten sorumludur. Her yerleşim yöneticisi, doğrudan veya dolaylı olarak *LayoutManager* arayüzünü uygulamalıdır. Örneğin,

*FlowLayout* doğrudan *LayoutManager*'ı uygular ve *BorderLayout*, *LayoutManager*'in bir alt sınıfı olan *LayoutManager2*'yi uygular. *LayoutManager* arayüzü, bir kapsayıcıda bileşenleri yerleştirmek için aşağıdaki yöntemleri sağlar:

- **public void** addLayoutComponent(String name, Component comp)  
Belirtilen adı taşıyan belirtilen bileşeni kapsayıcıya ekler.
- **public void** layoutContainer(Container parent)  
Belirtilen kapsayıcıdaki bileşenleri yerleştirir. Bu yöntemde, bileşenlerin nereye yerleştirileceğini belirten somut talimatlar sağlamalısınız.
- **public** Boyut minimumLayoutSize(Kapsayıcı parent)Belirtilen an  
a kapsayıcıdaki bileşenler göz önüne alındığında, belirtilen panel için minimum boyut boyutlarını hesaplar.
- **public** Boyut preferredLayoutSize(Kapsayıcı parent)Belirtilen ana k  
apsayıcıdaki bileşenler göz önüne alındığında, belirtilen panel için tercih edilen boyut boyutlarını hesaplar.

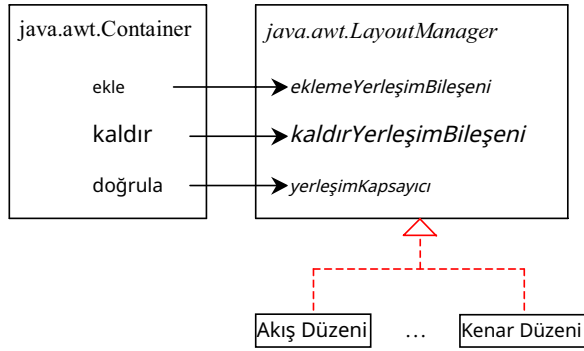


- **public void** removeLayoutComponent(Bileşen comp)

Belirtilen bileşeni yerleşimden kaldırır.

Bu yöntemler, *LayoutManager* içindeki yöntemler tarafından *java.awt.Container* sınıfındaki yöntemler aracılığıyla kapsayıcıdaki yerleşim yöneticisi tarafından çağrılır. Kapsayıcı, *layout* (bir *LayoutManager* örneği) adlı bir özelliğe sahiptir ve kapsayıcıdan bileşen ekleme ve kaldırma yöntemlerine sahiptir.

Kapsayıcı için çeşitli seçeneklerle bileşen eklemek üzere tanımlanmış beş aşırı yüklenmiş *add* yöntemi vardır. *remove* metodu, bir bileşeni kapsayıcıdan kaldırır. *add* metodu, *addImpl*'yi çağırır, bu da *addLayoutComponent* yöntemini çağırır. *LayoutManager* arayüzünde tanımlanmıştır. *layoutContainer* yöntemi *LayoutManager* arayüzündeki *layoutContainer* yöntemi, *validate()* yöntemi aracılığıyla dolaylı olarak çağrılır. *remove* metodu, *LayoutManager*'daki *removeLayoutComponent*'i çağırır. *validate* metodu, kapsayıcıya eklenen veya değiştirilen bileşenlerden sonra kapsayıcıyı yenilemek için çağrılır. *Container* ve *LayoutManager* arasındaki ilişki Şekil 37.7'de gösterilmektedir.



**Şekil 37.7**

*Container*'daki *add*, *remove* ve *validate* yöntemleri, *LayoutManager* arayüzünde tanımlanan yöntemleri çağırır.

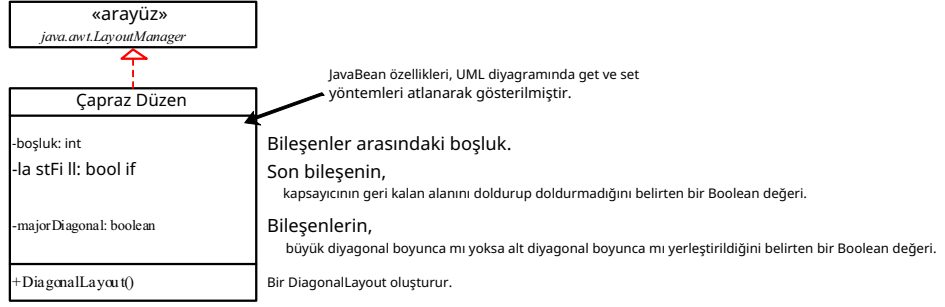
Kendi özel yerleşim yöneticimizi *DiagonalLayout* olarak tanımlayalım; bu, bileşenleri çapraz bir şekilde yerleştirir. *DiagonalLayout*'u test etmek için, örnek "FlowLayout," "GridLayout" ve "DiagonalLayout" adında radyo düğmeleri içeren bir applet oluşturur; bu, Şekil 37.8'de gösterilmiştir. Panelde bu üç yerleşimden birini dinamik olarak seçebilirsiniz.



### Şekil 37.8

*DiagonalLayout yöneticisi, bileşenleri kapsayıcıda çapraz bir şekilde yerleştirir.*

DiagonalLayout, FlowLayout'a benzer. DiagonalLayout, bileşenleri her bir bileşen in doğal preferredSize'ını kullanarak çapraz bir şekilde düzenler. Üç kısıtla ması vardır: gap, lastFill ve majorDiagonal; bu, Şekil 37.9'da gösterilmiştir. DiagonalLayout'un kaynak kodu Liste 37.4'te verilmiştir.



### Şekil 37.9

*DiagonalLayout yöneticisi, destekleyici*

*erişimci ve değiştirici yöntemlerle birlikte üç özelliğe sahiptir.*

### Liste 37.4 DiagonalLayout.java

<margin note line 6: özellikler>

<margin note line 34: yerleşim kapsayıcısı>

```
1 java.aut.*'u içe aktarın;
2
3 genel sınıf DiagonalLayout ve LayoutManager'ın uygulanması,
4 java.io.Serileştirilebilir {
5 /** Bileşenler arasındaki dikey boşluk */
6 özel iç boşluk = t 10t;
7
8 /** Bileşenlerin ana diyagonal boyunca yerleştirildiği doğruysa */
9 özel boolean majorDiagonal = t truet;
10
11 /*Son bileşenin alanı dolduracak şekilde uzatıldığı doğruysa */
12 özel boolean lastFill = t falset;
13
14 /** Yapıcı */
15 genel Çapraz Düzen() {
16 }
17
18 public voidt addLayoutComponent(Dize adı, Bileşen bileşimi) {
19 // DiagonalLayout için uygulamaya gerek yok
20 }
21
22 public voidt kaldırLayoutComponent(Bileşen kompozisyonu) {
23 // DiagonalLayout için uygulamaya gerek yok
24 }
25 }
```

```

25
26 genel Boyut tercih edilen Düzen Boyutu (Konteyner üst ögesi) {
27 minimum Düzen Boyutunu döndür (ebeveyn);
28 }
29
30 genel Boyut minimum Düzen Boyutu (Konteyner üst ögesi) {
31 return new Boyut(0, 0);
32 }
33
34 public void kapsayıcıyıYerleştir(Container parent) {
35 int bileşenSayısı = parent.getComponentCount();
36
37 Kenar insets = parent.getInsets();
38 intt w = parent.getSize().width - insets.left - insets.right;
39 intt h = parent.getSize().height - insets.bottom - insets.top;
40
41 if (büyükDiyagonal) {
42 intt x = t 10t, y = t 10t;
43
44 for (int j = 0; j < bileşenSayısı; j++) {
45 Bileşen c = parent.getComponent(j); n 46 Boyut d = c.getPr
46 eferredSize();
47
48 eğer (c.görünür())
49 eğer (sonDoldurma && (j == bileşenSayısı - 1))
50 c.setBounds(x, y, w - x, h - y);
51 aksi takdirde
52 c.setBounds(x, y, d.genislik, d.yükseklik);
53 x += d.yükseklik + boşluk;
54 y += d.yükseklik + boşluk;
55 }
56 }
57 aksi takdirde { // Alt diyagonal
58 intt x = w - t 10t, y = t 10t;
59
60 for (int j = 0; j < bileşenSayısı; j++) {
61 Bileşen c = parent.getComponent(j);
62 Boyut d = c.getPreferredSize();
63
64 eğer (c.görünür())
65 eğer (sonDoldurma & (j == bileşenSayısı - 1))
66 c.setBounds(t0t, y, x, h - y);
67 aksi takdirde
68 c.setBounds(x - d.genişlik, y, d.genişlik, d.yükseklik);
69
70 x -= (d.height + boşluk);
71 y += d.height + boşluk;
72 }
73 } n 74 }
75
76 public int getBoşluk() {
77 return boşluk;
78 }
79
80 public void setBoşluk(int boşluk) {
81 this.boşluk = boşluk;
82 }
83

```

```

84 public voidt setMajorDiagonal(tboolean newMajorDiagonal) {
85 majorDiagonal = yeniMajorDiagonal; n 86 }

87
88 genel boolean isMajorDiagonal() {
89 büyükDiagonal'ı döndür;
90 }
91
92 public voidt setLastFill(tboolean newLastFill) {
93 lastFill = newLastFill; n 94 }

95
96 genel boolean isLastFill() {
97 son Dolguyu döndür;
98 }
99 }

```

DiagonalLayout sınıfı, LayoutManager ve Serializable arayüzlerini (satırlar 3-4) uygular. Serializable'ı uygulamanın nedeni, onu bir JavaBeans bileşeni yapmaktır.

Insets sınıfı, bir kapsayıcının kenarlarının boyutunu tanımlar. Bu, sol, sağ, alt ve üst kenarların ölçümlerine karşılık gelen sol kenar, sağ kenar, üst kenar ve alt kenar değişkenlerini içerir (satırlar 37-39).

Dimension sınıfı, DiagonalLayout'ta bir bileşenin genişliğini ve yüksekliğini tek bir nesne kapsar. Bu sınıf, bileşenlerin belirli özellikleri ile ilişkilidir. Sınıf tarafından tanımlanan birkaç yöntem vardır.

**Bileşen** sınıfı ve **YerleşimYönetici** arayüzü bir **Boyut** nesnesi döndürür.

Liste 37.5, *DiagonalLayout*. kullanan bir test programı vermektedir.

#### Liste 37.5 ShowDiagonalLayout.java

*<kenar notu satır 9: diyagonal yerleşim>*  
*<kenar notu satır 27: UI oluştur>*  
*<kenar notu satır 48: dinleyiciyi kaydet>*  
*<kenar notu satır 55: dinleyiciyi kaydet>*  
*<kenar notu satır 62: dinleyiciyi kaydet>*  
*<kenar notu satır 70: ana yöntem atlandı>*

```

1 javax.swing.*'i içe aktarın;
2 javax.swing.border.*'ı içe aktarın;
3 java.aut.*'u içe aktarın;
4 java.awt.event.*'i içe aktarın;
5
6 public classt ShowDiagonalLayout t extendst JApplet {
7 özel FlowLayout flowLayout = t newt FlowLayout();
8 özel GridLayout gridLayout = t newt GridLayout(t2t,t2t);
9 özel Çapraz Düzen çapraz Düzen = t newt Çapraz Düzen();
10
11 özel JButton jbt1 = t newt JButton(t"Düğme 1"t);
12 özel JButton jbt2 = t newt JButton(t"Düğme 2"t);
13 özel JButton jbt3 = t newt JButton(t"Düğme 3"t);
14 özel JButton jbt4 = t newt JButton(t"Düğme 4"t);
15
16 özel JRadioButton jr FlowLayout =

```

```

17 newt JRadioButton(t"FlowLayout"t);
18 özel JRadioButton jr GridLayout =
19 newt JRadioButton(t"GridLayout"t);
20 özel JRadioButton jrbDiagonalLayout =
21 newt JRadioButton(t"DiagonalLayout"t,t truet);
22
23 özel JPanel jPanel 2t newt JPanel();
24
25 public Çapraz Düzeni Göster() {
26 // jPanel2'de varsayılan yerleşimi ayarla
27 jPanel2.setLayout(diagonalLayout); n 28 jPanel2.a
dd(jbt1);
29 jPanel2.add(jbt2) ;
30 jPanel2.add(jbt3) ;
31 jPanel2.add(jbt4) ;
32 jPanel2.setBorder(tnewt LineBorder(Color.black));
33
34 JPanel jPanel1 = newt JPanel();
35 jPanel1.setBorder(new TitledBorder ("Bir Yerleşim Yöneticisi Seçin")) ;
36 jPanel1.add(jrbFlowLayout);
37 jPanel1.add(jrbGridLayout); n 38 jPanel1.
add(jrbDiagonalLayout);
39
40 ButonGrubu butonGrubu1 = newt ButtonGroup();
41 butonGroup1.add(jrbFlowLayout); n 42 butonu
Group1.add(jrbGridLayout); n 43 butonuGroup1.a
dd(jrbDiagonalLayout);
44
45 add(jPanel1, BorderLayout.SOUTH); n 46 add(jPa
nel2, BorderLayout.CENTER); n 47

48 jrbFlowLayout.addActionListener(tnewt ActionListener() { n 49 @Override

50 public void actionPerformed(ActionEvent e) {
51 jPanel2.setLayout(flowLayout); n 52 jPanel2.valida
te();
53 }
54 });
55 jrbGridLayout.addActionListener(tnewt ActionListener() { n 56 @Override

57 public void actionPerformed(ActionEvent e) {
58 jPanel2.setLayout(gridLayout); n 59 jPanel2.validat
e());
60 }
61 });
62 jrbDiagonalLayout.addActionListener(tnewt ActionListener() { n 63 @Override

64 public void actionPerformed(ActionEvent e) {
65 jPanel2.setLayout(diagonalLayout); n 66 jPanel2.validat
e());
67 }
68 });
69 }
70 }

```

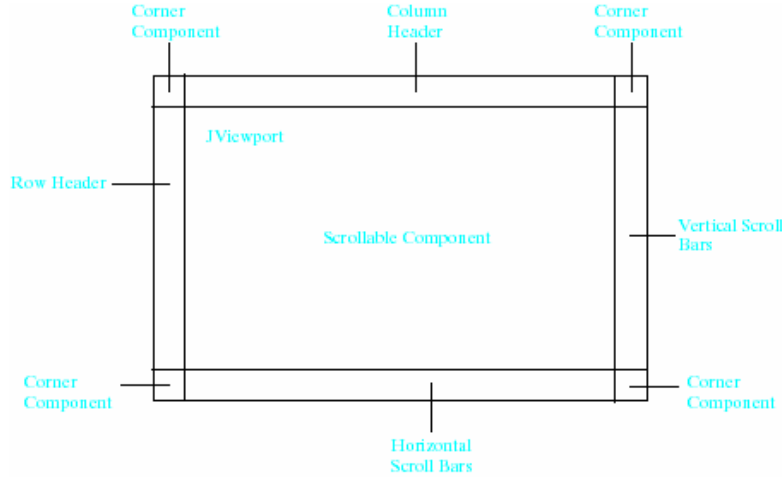
TestDiagonalLayout sınıfı, jPanel2'de düzeni dinamik olarak ayarlamanıza olanak tanır. Yeni bir düzen seçtiğinizde, düzen yöneticisi jPanel2'de ayarlanır ve revalidate() metodu çağrılır (satırlar 52, 59, 66),

bu da, bileşenleri kapsayıcıda görüntülemek için LayoutManager arayüzündeki layoutContainer metodunu çağırır.

### 37.5t JScrollPane

Sıklıkla, bir nesnenin içeriğini görüntüleme alanına tamamen sığmadığında kaydırmak için bir kaydırma çubuğu kullanmanız gerekir. Bu amaçla JScrollPane ve JSlider kullanılır, ancak bunlarla kaydırma uygulamak için kodu manuel olarak yazmanız gerekir. JScrollPane, kodlama olmadan otomatik kaydırmayı destekleyen bir bileşendir. Listing 17.3, TextAreaDemo.java'daki metin alanını kaydırmak ve Listing 17.5, ListDemo.java'daki bir listeyi kaydırmak için kullanıldı. Aslında, JComponent'in herhangi bir alt sınıfını kaydırmak için kullanılabilir.

Bir JScrollPane, içerilen bileşeni görüntülemek için bir görünüm alanına sahip özel bir kapsayıcı olarak görülebilir. Yatay ve dikey kaydırma çubuklarının yanı sıra, bir JScrollPane'in bir sütun başlığı, bir satır başlığı ve köşeleri olabilir, Şekil 37.10'da gösterildiği gibi.



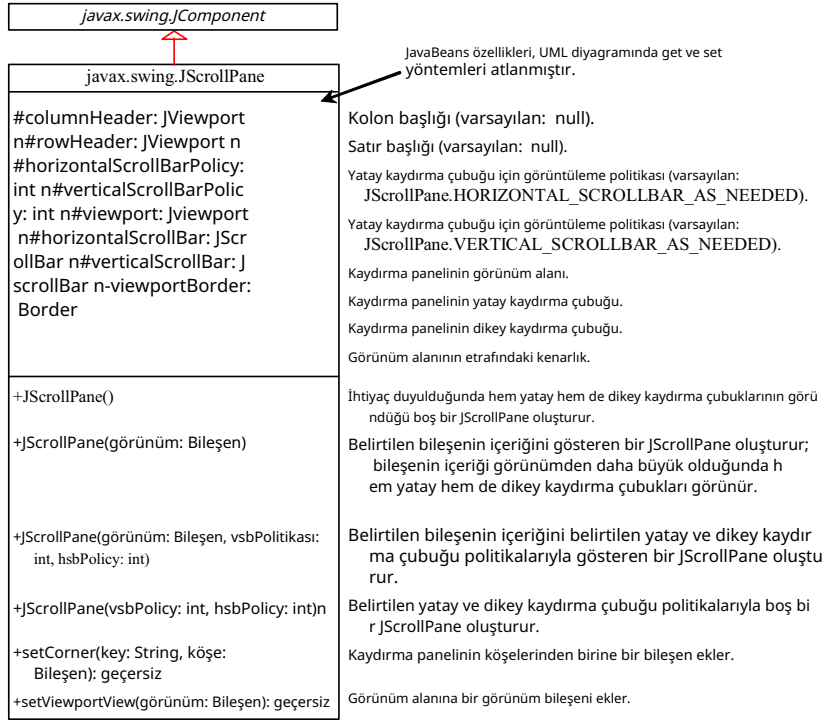
Şekil 37.10

*Bir JScrollPane, bir görünüm alanına, isteğe bağlı yatay ve dikey çubuklara, isteğe bağlı sütun ve satır başlıklarına ve isteğe bağlı köşelere sahiptir.*

#### <kenar notu: görünüm alanı>

Görünüm alanı, kaydırılabilir bir bileşenin görüntülediği JViewport örneğidir. Bir bileşeni bir kaydırma paneline eklediğinizde, bunu aslında kaydırma panelinin görünüm alanına yerleştiriyorsunuz. Şekil 37.11,

JScrollPane'de sıkça kullanılan özellikleri, yapıları ve yöntemleri göstermektedir.



**Şekil 37.11**

### *JScrollPane* kaydırılabilir paneldeki

*bileşenleri görüntülemek ve manipüle etmek için yöntemler sağlar.*

Yapıcı, görüntüleme bileşeni belirtilip belirtilmediğine bakılmaksızın her zaman bir görünüm alanı oluşturur. Normalde, bileşeniniz vardır ve onu bir kaydırılabilir panelde yerleştirmek istersiniz. Bir bileşen için kaydırılabilir bir panel oluşturmanın pratik bir yolu, *JScrollPane(bileşen)yapıcısını kullanmaktır*.

vsbPolicy parametresi aşağıdaki üç değerden biri olabilir:

`JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED`

`JScrollPane.VERTICAL_SCROLLBAR_NEVER`

`JScrollPane.VERTICAL_SCROLLBAR_ALWAYS`

hsbPolicy parametresi aşağıdaki üç değerden biri olabilir:

`JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED`

`JScrollPane.HORIZONTAL_SCROLLBAR_NEVER`

`JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS`

Bir köşe bileşeni ayarlamak için `setCorner(String key, Component corner)` yöntemini kullanabilirsiniz. Anahtar için geçerli değerler şunlardır:

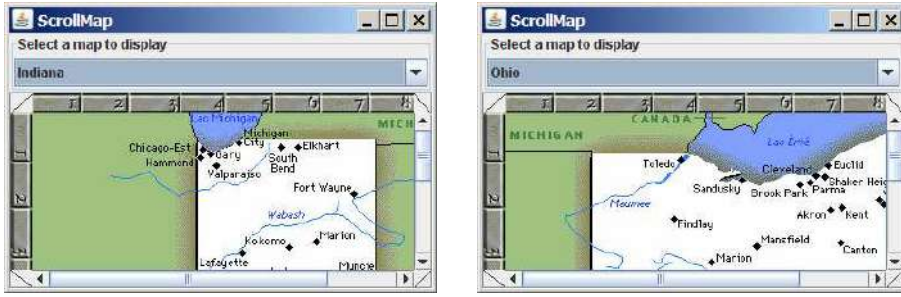
`JScrollPane.LOWER_LEFT_CORNER`

`JScrollPane.LOWER_RIGHT_CORNER`

`JScrollPane.UPPER_LEFT_CORNER`

`JScrollPane.UPPER_RIGHT_CORNER`

Liste 37.6, bir etikette bir harita gösteren ve büyük bir haritanın kaydırılabilmesi için etiketi bir kaydırma paneline yerleştiren bir örneği göstermektedir. Program, bir açılır kutudan bir harita seçmenize ve bunu kaydırma panelinde görüntülemeye nize olanak tanır; bu, Şekil 37.12'de gösterilmiştir.



Şekil 37.12

*Kaydırma paneli, içerikleri otomatik olarak kaydırmak için kullanılabilir.*

#### Listeleme 37.6 ScrollMap.java

<margin note line 8: etiketler>

<margin note line 17: UI oluştur>

<margin note line 28: kaydırma paneli>

<margin note line 46: dinleyiciyi kaydet>

<margin note line 88: ana yöntem atlandı >

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.border.*'ı içe aktarın;
5
6 genel sınıf ScrollMap t extendst JApplet {
7 // Etiketlerde resimler oluştur
8 özel JLabel lblIndianaMap = t newt JLabel(
9 newt ImageIcon(getClass().getResource(t"image/indianaMap.gif"t)));
10 özel JLabel lblOhioMap = t newt JLabel(
11 newt ImageIcon(getClass().getResource(t"image/ohioMap.gif"t)));
12
13 // Etiketlerde haritayı kaydırmak için bir kaydırma paneli oluştur
14 özel JScrollPane jspMap = t newt JScrollPane(lblIndianaMap);
15
16 genel Kaydırma Haritası() {
17 // Harita seçmek için bir kombinasyon kutusu oluştur
```



```

18 JComboBox jcboMap = t.newt JComboBox(t.newt String[] {t"Indiana",t,
19 "Ohio"t});
20
21 // Kombinasyon kutusunu tutacak panel p
22 JPanel p = t.newt JPanel(); n 23 p.setL
ayout(t.newt BorderLayout()); n 24 p.add(jcboMap);

25 p.setBorder (new TitledBorder ("Bir harita seçin"));
26
27 // Satır başlığı, sütun başlığı ve köşe başlığını ayarla
28 jspMap.setColumnHeaderView(t.newt JLabel(t.newt ImageIcon(getClass().)
29 getResource(t"/image/horizontalRuler.gif"t)));n 30 jspMap.setRow
HeaderView(t.newt JLabel(t.newt ImageIcon(getClass().)
31 getResource(t"/image/verticalRuler.gif"t))) ve 32 jspMap.setCor
ner(JScrollPane.UPPER_LEFT_CORNER,
33 new CornerPanel(JScrollPane.UPPER_LEFT_CORNER));
34 jspMap.setCorner (ScrollPaneConstants.UPPER_RIGHT_CORNER,
35 new CornerPanel(JScrollPane.UPPER_RIGHT_CORNER));
36 jspMap.setCorner (JScrollPane.LOWER_RIGHT_CORNER,
37 new CornerPanel(JScrollPane.LOWER_RIGHT_CORNER));
38 jspMap.setCorner (JScrollPane.LOWER_LEFT_CORNER,
39 new CornerPanel(JScrollPane.LOWER_LEFT_CORNER));
40
41 // Çerçeveye kaydırma paneli ve kombinasyon kutusu panelini ekle
42 add(jspMap, BorderLayout.CENTER); n 43 add(p,
BorderLayout.NORTH);
44
45 // Dinleyiciyi kaydet
46 jcboMap.addItemListener(t.newt ItemListener() {
47 /** Seçilen haritayı göster */
48 public voidt itemStateChanged(ItemEvent e) {
49 String seçiliItem = (String)e.getItem();
50 ift (selectedItem.equals(t"Indiana"t)) {
51 // Görünüm alanında yeni bir görünüm ayarla
52 jspMap.setViewportView(lblIndianaMap); n 53 }

54 else ift (selectedItem.equals(t"Ohio"t)) {
55 // Görünüm alanında yeni bir görünüm ayarla
56 jspMap.setViewportView(lblOhioMap); n 57 }

58
59 // Kaydırılabilir paneli yeniden doğrula
60 jspMap.revalidate(); n 61 }

62 }); n 63 }

64
65 // Kaydırılabilir panel köşesi için kullanılan bir çizgi gösteren panel
66 şık Köşe Paneli t JPanel'i genişletir {
67 // Çizgi konumu
68 özel Dize konumu;
69
70 genel Köşe Paneli(Dize konumu) {
71 thist.location = konum;
72 }
73
74 @Override /** Konuma bağlı olarak bir çizgi çizin */
75 korumalı voidt paintComponent(Graphics g) {
76 super.paintComponent(g);

```

```

77
78 ift (konum == "SOL ÜST KÖŞE"ye)
79 g.drawLine(t0t, getHeight(), getWidth(), t 0t);
80 else if (location == "ÜST SAĞ KÖŞE")
81 g.drawLine(t0t,t 0t, getWidth(), getHeight());
82 else if (location == "ALT SAĞ KÖŞE")
83 g.drawLine(t0t, getHeight(), getWidth(), t 0t);
84 else if (location == "ALT SOL KÖŞE")
85 g.drawLine(t0t,t 0t, getWidth(), getHeight()); n 86 }
87 } n 88
}

```

Program, resim haritalarını görüntülemek için bir kaydırılabilir panel oluşturur. Resimler, resim dosyalarından oluşturulur ve etiketlerde görüntülenir (satırlar 8-11). Bir resmi görüntülemek için, resmi içeren etiket kaydırılabilir panelin görünüm alanına yerleştirilir (satır 14).

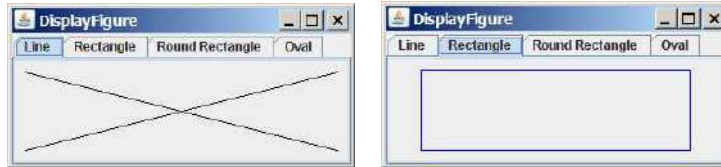
Kaydırılabilir panelin ana görünümü, bir başlık görünümü, bir sütun görünümü ve dört köşe görünümü vardır. Her görünüm, *Component* sınıfının bir alt sınıfıdır. Çünkü *ImageIcon* bir *Component* alt sınıfı değildir, bu nedenle doğrudan kaydırılabilir panelde bir görünüm olarak kullanılamaz. Bunun yerine program bir *ImageIcon* bir etikete ve etiketi bir görünüm olarak kullanır.

KöşePanel (satırlar 66-87) bir *JPanel* alt sınıfıdır ve bir çizgi göstermek için kullanılır. Çizginin nasıl çizileceği, konum köşenin konumuna bağlıdır. Konum köşe panelinin yapıcısında bir parametre olarak geçirilen bir dizedir.

Yeni bir harita seçildiğinde, harita resmini görüntülemek için etiket kaydırılabilir panelin görünüm alanına ayarlanır. *revalidate()* yöntemi (satır 60) yeni resmin görüntülenmesi için çağrılmalıdır. *revalidate()* yöntemi, bir kapsayıcının içindeki bileşenler eklendikten veya değiştirildikten sonra alt bileşenlerini yeniden düzenlemesini sağlar.

### 37,6 ton J Sekmeli Panel

*JTabbedPane* birden fazla bileşene erişmek için karşılıklı olarak dışlayıcı sekmeler sağlayan yararlı bir Swing kapsayıcısıdır; bu, Şekil 37.13'te gösterilmiştir.



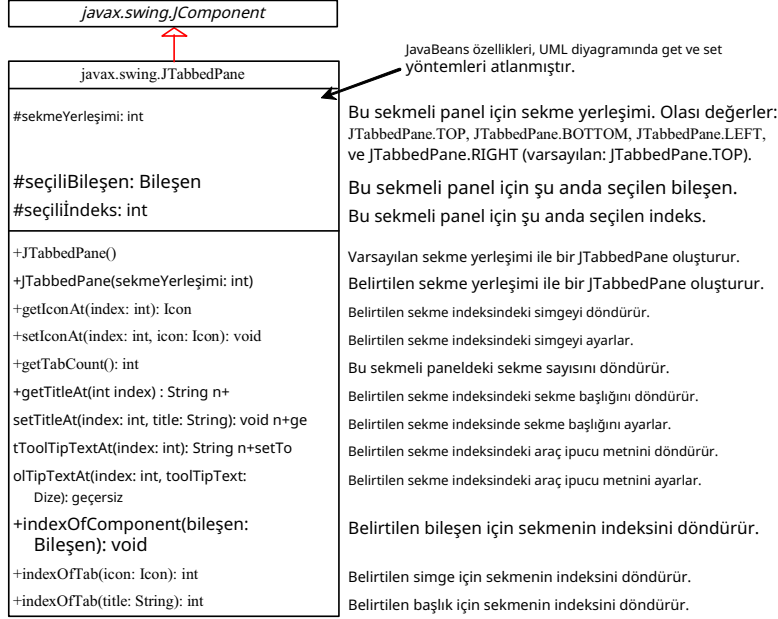
Şekil 37.13

*JTabbedPane* bileşenleri sekmeler aracılığıyla görüntüler.

Genellikle panelleri bir *JTabbedPane* içine yerleştirir ve her panel ile bir sekme ilişkilendirirsiniz. *JTabbedPane* kullanımı kolaydır, çünkü panelin seçimi otomatik olarak ilgili s ekmeye tıklanarak işlenir.

Bir grup panel arasında, bir sekmeye tıklayarak geçiş yapabilirsiniz.

verilen başlık ve/veya simge. Şekil 37.14, JTabbedPane'de sıkça kullanılan özellikleri, yapıcılar ve yöntemleri göstermektedir .



Şekil 37.14

#### JTabbedPane sekmeli paneldeki

bileşenleri görüntülemek ve manipüle etmek için yöntemler sağlar.

Liste 37.7, dört tür şekli görüntülemek için dört sekmeli bir panel kullanan bir örnek vermektedir: çizgi, dikdörtgen, yuvarlatılmış dikdörtgen ve oval. Şekli görüntülemek için ilgili sekmeye tıklayarak bir şekil seçebilirsiniz; bu, Şekil 37.13'te gösterilmiştir. Şekli görüntülemek için *FigurePanel* sınıfı, Liste 15.3'te, *FigurePanel.java*'da sunulmuştur.

Bir şekil türünü belirtmek için *type* özelliğini kullanabilirsiniz.

#### Liste 37.7 DisplayFigure.java

<margin note line 5: sekmeler paneli>

<margin note line 12: tür ayarla>

<margin note line 18: sekmeler ekle>

<margin note line 23: araç ipuçlarını ayarla>

<margin note line 28: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 javax.swing.*'i içe aktarın;
3
4 genel sınıf DisplayFigure t extendst JApplet {
5 özel JTabbedPane jtp Şekiller = t newt JTabbedPane();
6 özel Şekil Paneli kare Panel = t newt Şekil Paneli();
7 özel FigurePanel dikdörtgenPanel = yeni FigurePanel();
8 özel FigurePanel dairePanel = yeni FigurePanel();
9 özel FigurePanel ovalPanel = yeni FigurePanel();
10

```

```

11 genel DisplayFigure() {
12 karePanel.setType(FigurePanel.LINE);
13 dikdörtgenPanel.setType(FigurePanel.RECTANGLE);
14 dairePanel.setType(FigurePanel.ROUND_RECTANGLE);
15 ovalPanel.setType(FigurePanel.OVAL);
16
17 ekle(jtpFigures, BorderLayout.CENTER);
18 jtpFigures.add(karePanel, "Çizgi");
19 jtpFigures.add(dikdörtgenPanel, "Dikdörtgen");
20 jtpFigures.add(dairePanel, "Yuvarlak Dikdörtgen");
21 jtpFigures.add(ovalPanel, "Oval");
22
23 jtpFigürleri.setToolTipTextAt(0, "Kare");
24 jtpFigures.setToolTipTextAt(1, "Dikdörtgen");
25 jtpFigures.setToolTipTextAt(2, "Daire");
26 jtpFigures.setToolTipTextAt(3, "Oval");
27 }
28 }

```

Program, her biri bir şekil gösteren dört paneli tutmak için sekmeli bir panel oluşturur . Bir panel, bir sekme ile ilişkilidir. Sekmelerin başlıkları Çizgi, Dikdörtgen, Yuvarlak Dikdörtgen ve Oval'dır.

Varsayılan olarak, sekmeler sekmeli panelin üst kısmında yer alır. `tabPlacement` özelliği ni kullanarak farklı bir yerleşim seçebilirsiniz.

### 37,7 ton JBölünmüş Panel

`JSplitPane`, iki bileşeni ayrı bir çubuk olan *divider* ile içeren kullanışlı bir Swing kapsayıcısıdır, Şekil'de gösterildiği gibi. 37.15.



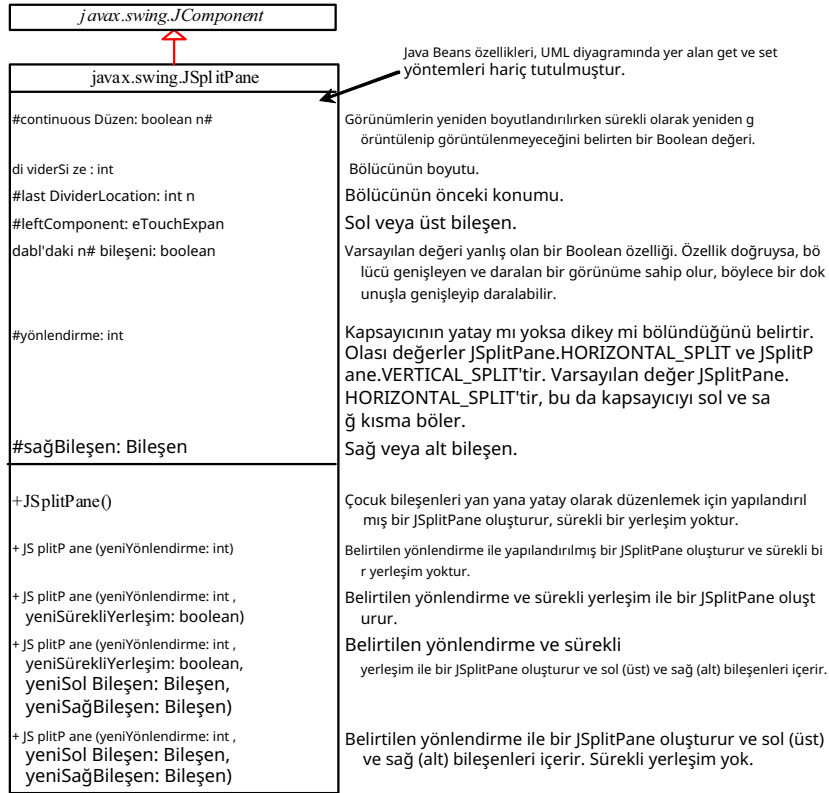
Şekil 37.15

`JSplitPane`, bir kapsayıcıyı iki parçaya böler.

Çubuk, kapsayıcıyı yatay veya dikey olarak bölebilir ve her bileşenin kapladığı alanı değiştirmek için sürüklenebilir.

Şekil 37.16, `JSplitPane`'de sıkça kullanılan özellikleri, yapıları ve yöntemleri göstermektedir.

.



Şekil 37.16

*JSplitPane bir bölünmüş panelin özelliklerini belirtmek ve bölünmüş paneldeki bileşenleri manipüle etmek için yöntemler sağlar.*

Liste 37.8, kullanıcının bir panel için dinamik olarak **FlowLayout**, **GridLayout** veya **BoxLayout** yöneticisini seçmesine olanak tanıyan radyo düğmeleri kullanan bir örnek vermektedir. Panel, Şekil 37.17'de gösterildiği gibi dört düğme içermektedir. Mevcut seçili yerleşim yöneticisinin tanımı bir metin alanında görüntülenir. Radyo düğmeleri, düğmeler ve metin alanı iki bölünmüş panelde yerleştirilmiştir.



Şekil 37.17

*Bölünmüş panellerde bileşen boyutunu ayarlayabilirsiniz.*

Liste 37.8 ShowLayout.java

<margin note line 7: açıklamalar>

<margin note line 13: radyo düğmeleri>  
 <margin note line 24: yerleşim yöneticileri>  
 <margin note line 54: bölünmüş panel>  
 <margin note line 57: bölünmüş panel>  
 <margin note line 76: dinleyici kaydet>  
 <margin note line 81: doğrula>  
 <margin note line 84: dinleyici kaydet>  
 <margin note line 89: doğrula>  
 <margin note line 93: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 genel sınıf Gösteri Düzeni t Uygulamayı genişletir {
6 // HTML dosyaları için URL'yi al
7 private String flowLayoutDesc = "FlowLayout bileşenleri " +
8 "tercih edilen boyutlarına göre " +
9 "soldan sağa doğru bir akış, bir paragraftaki metin satırları gibi.";
10 private String GridLayoutDesc = "GridLayout düzenler ...";
11 private String BorderLayoutDesc = "BoxLayout düzenler ...";
12
13 özel JRadioButton jr FlowLayout =
14 newt JRadioButton(t"FlowLayout"t);
15 özel JRadioButton jr GridLayout =
16 newt JRadioButton(t"GridLayout"t,t truet);
17 özel JRadioButton jr BorderLayout =
18 newt JRadioButton(t"BoxLayout"t);
19
20 özel JPanel Bileşenleri = t newt JPanel();
21 özel JTextArea jtfDescription = t newt JTextArea();
22
23 // Yerleşim yöneticilerini oluştur
24 özel FlowLayout flowLayout = t newt FlowLayout();
25 özel GridLayout GridLayout = t newt GridLayout(t2t,t2t,t 3t,t 3t);
26 özel BorderLayout boxLayout =
27 yeni BorderLayout(jpComponents, BorderLayout.X_AXIS);
28
29 genel ShowLayout() {
30 // Radyo düğmelerini tutacak bir kutu oluştur
31 Kutu jpChooseLayout = Box.createVerticalBox(); n 32 jpChooseL
32 ayout.add(jrbFlowLayout);
33 jpChooseLayout.add(jrbGridLayout); n 34 jpChoos
34 eLayout.add(jrbBoxLayout);
35
36 // Radyo düğmelerini gruplandır
37 ButtonGroup btg = yeni ButtonGroup();
38 btg.add(jrbFlowLayout);
39 btg.add(jrbGridLayout); n 40 btg.ad
40 d(jrbBoxLayout);
41
42 // Satırları ve kelimeleri sar
43 jtfDescription.setLineWrap(ttruet);n 44 jtfDescr
44 iption.setWrapStyleWord(ttruet);
45
46 // jpComponents'e dört buton ekle
47 jpComponents.add(new JButton("Buton 1"));
48 jpComponents.add(new JButton("Buton 2"));
49 jpComponents.add(new JButton("Buton 3"));

```

```

50 jpComponents.add(new JButton("Buton 4"));
51
52 // jpChooseLayout, jpComponents ve
53 // jtfDescription'ı tutmak için iki bölünmüş panel oluştur
54 JSplitPane jSplitPane2 = t newt JSplitPane(n 55 JSplitPane
55 .VERTICAL_SPLIT, jpComponents,
56 newt JScrollPane(jtfDescription));
57 JSplitPane jSplitPane1 = t newt JSplitPane(n 58 JSplitPane
58 .HORIZONTAL_SPLIT, jpChooseLayout, jSplitPane2);
59
60 // Varsayılan olarak FlowLayout'ı ayarla
61 jpComponents.setLayout(flowLayout); n 62 jpComp
62 onents.revalidate();
63 jtfDescription.setText(flowLayoutDesc);
64
65 add(jSplitPanel1, BorderLayout.CENTER);
66
67 // Dinleyicileri kaydet
68 jrbFlowLayout.addActionListener(tnewt ActionListener() { n 69 @Override
69
70 public void actionPerformed(ActionEvent e) {
71 jpComponents.setLayout(flowLayout); n 72 jtfDescriptio
72 n.setText(flowLayoutDesc); n 73 jpComponents.revalidate();
73
74 }
75 });
76 jrbGridLayout.addActionListener(tnewt ActionListener() { n 77 @Override
77
78 public void actionPerformed(ActionEvent e) {
79 jpComponents.setLayout(gridLayout); n 80 jtfDescriptio
80 n.setText(gridLayoutDesc); n 81 jpComponents.revalidate();
81
82 }
83 });
84 jrbBoxLayout.addActionListener(tnewt ActionListener() { n 85 @Override
85
86 public void actionPerformed(ActionEvent e) {
87 jpComponents.setLayout(boxLayout); n 88 jtfDescripti
88 on.setText(boxLayoutDesc); n 89 jpComponents.revalidate();
89
90 }
91 });
92 }
93 }

```

Bölünmüş paneller gömülebilir. Mevcut bir bölünmüş panele bir bölünmüş panel eklemek, üç bölünmüş panel ile sonuçlanır. Program, radyo düğmeleri için bir panel, düğmeler için bir panel ve bir kaydırılabilir panel tutmak için iki bölünmüş panel oluşturur (satırlar 54-58).

Radyo düğmeleri, yerleşim yöneticilerini seçmek için kullanılır. Seçilen bir yerleşim yöneticisi, düğmeleri yerleştirmek için panelde kullanılır (satırlar 66-88). Kaydırılabilir panel, seçilen yerleşim yöneticisini tanımlayan metni görüntülemek için bir `JTextArea` içerir (satır 56).

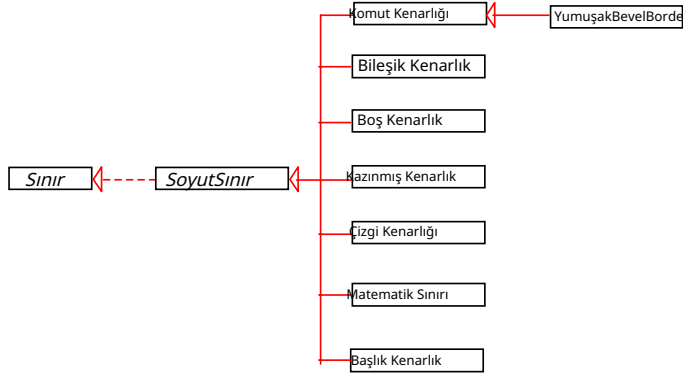
### 37.8 Swing Kenarlıkları

Swing, bileşenleri süslemek için kullanabileceğiniz çeşitli kenarlıklar sağlar. Başlıklıklı kenarlıklar ve çizgi kenarlıkları oluşturmayı öğrendiniz.

§12.9'da, "Swing GUI Bileşenlerinin Ortak Özellikleri." Bu bölüm bordürleri daha ayrıntılı bir şekilde tanıtmaktadır.

A Swing kenarlığı, **Border** arayüzünde tanımlanır. Her bir örneği **JComponent** bir kenarlığı **border** özelliği aracılığıyla ayarlayabilir JComponent. Eğer bir kenarlık varsa, iç kenar boşluğunu değiştirir. **AbstractBorder** sınıfı boyut suz boş bir kenarlık uygular. Bu, diğer kenarlık sınıflarının kolayca tanımlanabileceği uygun bir temel sınıf sağlar. Sekiz somut kenarlık sınıfı vardır,

**BevelBorder**, **SoftBevelBorder**, **CompoundBorder**, **EmptyBorder**, **EtchedBorder**, **LineBorder**, **MatteBorder**, ve **TitledBorder**, Şekil 37.18'de gösterildiği gibi.



Şekil 37.18

*Border arayüzü Swing kenarlıklarını tanımlar.*

[BL] **BevelBorder**, aşağı veya yukarı kaldırılabilen 3D görünümlü bir kenarlıktır. **BevelBorder**'ın aşağıdaki yapıları vardır; belirtilen **bevelType** ile bir **BevelBorder** oluşturur (**BevelBorder.LOWERED** veya **BevelBorder.RAISED**) ve renkler:

```
BevelBorder(tint bevelType)
BevelBorder(tint bevelType, Color highlight, Color shadow)
BevelBorder(tint bevelType, Color highlightOuterColor,
 Color highlightInnerColor,
 Color shadowOuterColor, Color shadowInnerColor)
```

[BL] **YumuşakBevelBorder**, yumuşatılmış köşelere sahip yukarı veya aşağı kaldırılmış bir bevel'dir. **YumuşakBevelBorder**'ın aşağıdaki yapıları vardır:

```
YumuşakBevelBorder(tint bevelType)
SoftBevelBorder (int bevelType, Renk vurgusu, Renk gölge)
SoftBevelBorder (int bevelType, Renk dış vurgusu,
 Renk iç vurgusu, Renk dış gölge,
 Renk iç gölge)
```

[BL] **BoşKenarlık** çizim olmayan ancak kenar boşluğu olan bir kenarlıktır. **BoşKenarlık** aşağıdaki yapıları sahiptir:

```
BoşKenarlık(İnset kenarInset)
BoşKenarlık(int üst, int sol, int alt, int sağ)
```



[BL] **YontulmuşKenarlık** yontulmuş bir kenarlık olup, yontulmuş veya dışarıda yontulmuş olabilir. **YontulmuşKenarlık** **yontuTürü** özelliğine sahiptir ve değeri **AŞAĞI** veya **YÜKSEK**'dir. **YontulmuşKenarlık** aşağıdaki yapıcılara sahiptir:

```
YontulmuşKenarlık() // Aşağıda bir kenarlık ile varsayılan yapıcı
YontulmuşKenarlık(Renk vurgulama, Renk gölge)
YontulmuşKenarlık(int yontuTürü)
YontulmuşKenarlık(int yontuTürü, Renk vurgulama, Renk gölge) [BL]Çi
```

**zgiKenarlık** kenar etrafında tek bir renkte ve keyfi kalınlıkta bir çizgi çizer. **ÇizgiKenarlık** aşağıdaki yapıcılara sahiptir:

```
ÇizgiKenarlık(Renk renk) // Kalınlık 1
ÇizgiKenarlık(Renk renk, int kalınlık)
ÇizgiKenarlık(Renk renk, int kalınlık, boolean yuvarlakKöşeler)
```

[BL] **MatteKenarlık** simgelerle yastıklanmış mat benzeri bir kenarlıktır. **MatteBorder** aşağıdaki yapıcılara sahiptir:

```
MatteBorder(Simgesimge döşeme simgesi)
MatteBorder(Insets borderInsets, Color matteColor)
MatteBorder(Insets borderInsets, Icon tileIcon)
MatteBorder(int üst, int sol, int alt,
 int sağ, Color matteColor)
MatteBorder(int üst, int sol, int alt, int sağ, Icon tileIcon)
```

[BL] **CompoundBorder** iki **Border** nesnesini tek bir border oluşturmak için, bir iç **Border** nesnesini dış **Border** nesnesinin iç kenar boşluklarına yerleştirerek birleş tirmek için aşağıdaki yapıcılığı kullanır:

```
CompoundBorder(Border outsideBorder, Border insideBorder)
```

[BL] **TitledBorder** belirli bir konumda bir dize başlığına sahip bir kenarlık. **TitledBorder** diğer kenarlıklarla birleştirilebilir. **TitledBorder** aşağıdaki yapıcılara sahiptir:

```
TitledBorder(Dize başlığı)
TitledBorder(Border border) // Diğer bir kenarlıkta boş başlık
TitledBorder(Border border, String title)
TitledBorder(Border border, String title, nintt titleJustification, int titlePosition)nTitledBorder(Border border, String title, nintt titleJustification, int titlePosition, n Font titleFont)
TitledBorder(Kenarlık kenarlığı, Dize başlığı, nintt titleJustification, int titlePosition, n Font titleFont, Color titleColor)
```

Kolaylık olması açısından, Java ayrıca javax.swing.BorderFactorysınıfını sağlar; bu sınıf, Şekil 37.19'da gösterilen kenarlıkları oluşturmak için statik yöntemleri içerir.

javax.swing.BorderFactory
+bevelBorderOluştur(type: int): Border
+bevelBorderOluştur(type: int, highlight: Color, shadow: Color): Border
+bevelBorderOluştur(type: int, highlightOuter: Color, highlightInner: Color, shadowOuter: Renk, gölgeİç: Renk): Kenarlık
+bileşikKenarlıkOluştur(): CompoundBorder
+bileşikKenarlıkOluştur(dışKenarlık: Border, içKenarlık: Border): Bileşik Kenarlık
+boşKenarlıkOluştur(): Border
+boşKenarlıkOluştur(üst: int, sol: int, alt: int, sağ: int): Border
+çizgiliKenarlıkOluştur(): Border
+çizgiliKenarlıkOluştur(highlight: Color, shadow: Color): Border
+çizgiliKenarlıkOluştur(type: int): Border
+çizgiliKenarlıkOluştur(type: int, highlight: Color, shadow: Color): Border
+çizgiKenarlığıOluştur(color: Color): Border
+çizgiKenarlığıOluştur(color: Color, kalınlık: int): Border
+alçaltılmışBevelBorderOluştur(): Border
+createMatteBorder(üst: int, sol: int, alt: int, sağ: int, renk: Color): MatteBorder
+createMatteBorder(üst: int, sol: int, alt: int, sağ: int, döşemeİkonu: Icon): MatteBorder
+createRaisedBevelBorder(): Border
+createTitledBorder(kenar: Border): TitledBorder
+createTitledBorder(kenar: Border, başlık: String): TitledBorder
+createTitledBorder(kenar: Border, başlık: String, başlıkHakkında: int, başlıkPozisyonu: int): BaşlıklıSınır
+createTitledBorder(sınır: Border, başlık: String, başlıkHakkında: int, başlıkPozisyonu: int, başlıkYazıTipi: Font): TitledBorder
+createTitledBorder(sınır: Border, başlık: String, başlıkHakkında: int, başlıkPozisyonu: int, başlıkYazıTipi: Font, başlıkRengi: Color): TitledBorder
+createTitledBorder(başlık: String): TitledBorder

### Şekil 37.19

*BorderFactory çeşitli türde sınırlar oluşturmak için statik yöntemleri içerir.*

Örneğin, bir oyma sınır oluşturmak için aşağıdaki ifadeyi kullanın:

```
Border sınır = BorderFactory.createEtchedBorder();
```

### NOT

Tüm sınır sınıfları ve arayüzleri, paket `javax.swing.border` dışında `javax.swing.BorderFactory` içinde gruplanmıştır.

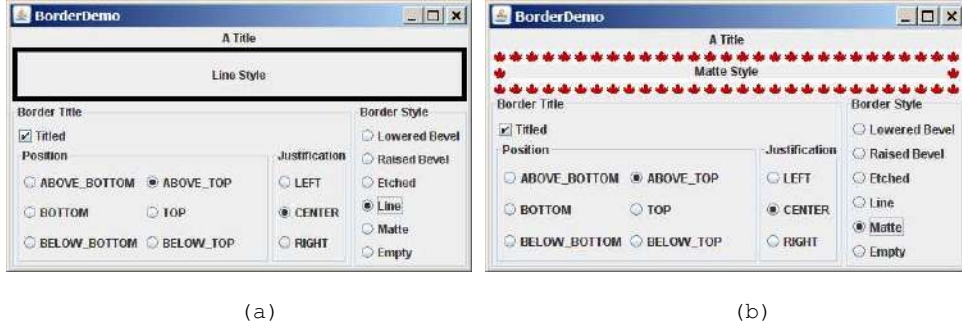
### NOT

Sınırlar ve simgeler paylaşılabilir. Böylece bir sınır or simge oluşturabilir ve bu nu herhangi bir GUI bileşeni için sınır veya simge özelliğini ayarlamak için kullanabilirsiniz. Örneğin, aşağıdaki ifadeler iki panel için border `b` ayarlar `p1` ve `p2`:

```
p1.setBorder(b);p2
.setBorder(b);
```

**\*\*\*NOTUN SONU**

Liste 37.9, çeşitli türde sınırlar oluşturan ve gösteren bir örnek vermektedir. Bir başlıkla veya başlıksız bir sınır seçebilirsiniz. Başlıksız bir sınır için, Aşağıya İndirilmiş Bevel, Yükseltilmiş Bevel, Aşındırılmış, Çizgi, Mat veya Boş gibi bir sınır stilini seçebilirsiniz. Başlıklı bir sınır için, başlık konumunu ve gerekçesini belirtebilirsiniz. Ayrıca, başka bir sınıra başlıklı bir sınırın içine yerleştirebilirsiniz. Şekil 37.20, programın örnek bir çalışmasını göstermektedir.



Şekil 37.20

Program, çeşitli kenarlık türlerini göstermektedir.

İşte programdaki ana adımlar:

1. Kullanıcı arayüzünü oluşturun.

- a. Bir `JLabel` nesnesi oluşturun ve çerçevenin ortasına yerleştirin.
- b. Kenarlık başlık konumunu seçmek için radyo düğmelerini gruplamak üzere `jpPosition` adında bir panel oluşturun. Bu panelin kenarlığını "Konum" başlığı ile başlıklı kenarlık olarak ayarlayın.
- c. Bir panel oluşturun adı `jpJustification` radyo düğmelerini sınır başlığı gerekçesini seçmek için gruplamak üzere. Bu panelin kenarını başlık "Gerekçe" ile başlıklı kenar ile ayarlayın.
- d. Bir panel oluşturun adı `jpTitleOptions` `jpPosition` panelini ve `jpJustification` panelini tutmak için.
- ve. Bir panel oluşturun adı `jpTitle` "Başlık" adında bir onay kutusu ve `jpTitleOptions` panelini tutmak için.
- f. Bir panel oluşturun adı `jpBorderStyle` kenar stillerini seçmek için radyo düğmelerini gruplamak üzere.
- a. Bir panel oluşturun adı `jpAllChoices` `jpTitle` ve `jpBorderStyle` panellerini tutmak için. `jpAllChoices` çerçevenin güneyine yerleştirin.

2. Olayı işleyin.

Dinleyicileri oluşturun ve kaydedin, `actionPerformed` işleyicisini uygulamak için etiket için kenarlığı, onay kutusundan gelen olaylara ve tüm radyo düğmelerinden gelen olaylara göre ayarlayın.

### Liste 37.9 BorderDemo.java

```
<!--margin note line 30: create UI-->
<!--margin note line 142: boş kenarlık-->
<!--margin note line 145: kabartma kenarlık-->
<!--margin note line 149: kabartma kenarlık-->
<!--margin note line 153: oyulmuş kenarlık-->
<!--margin note line 157: çizgi kenarlık-->
<!--margin note line 161: mat kenarlık-->
<!--margin note line 167: boş kenarlık-->
<!--margin note line 197: kenarlık üzerinde kenarlık-->
<!--margin note line 207: ana yöntem atlandı-->

1 java.awt.*u içe aktarın;
2 java.awt.event.ActionListener'ı içe aktarın;
3 java.awt.event.ActionEvent'i içe aktarın;
4 javax.swing.*i içe aktarın;
5 javax.swing.border.*ı içe aktarın;
6
7 genel sınıf BorderDemo t extendst JApplet {
8 // Mesajı görüntülemek için bir etiket tanımlayın
9 private JLabel jLabel1 = new JLabel("Kenarlık türünü göster",
10 JLabel.CENTER);
11
12 // Başlıkla veya başlıksız bir kenarlık seçmek için bir onay kutusu
13 özel JCheckBox jchk Başlıklı;
14
15 // Kenar stilleri için radyo düğmeleri
16 özel JRadioButton jrbLoweredBevel, jrbRaisedBevel,
17 jrbEtched, jrbLine, jrbMatte, jrbEmpty;
18
19 // Başlıklı kenar seçenekleri için radyo düğmeleri
20 özel JRadioButton jr AboveBottom, jr Bottom,
21 jrbAltAlt, jrbÜstÜst, jrbÜst, jrbAltÜst,
22 jrbSol, jrbMerkez, jrbSağ;
23
24 // Etiket için TitledBorder
25 özel TitledBorder jLabel1Border;
26
27 /** Yapıcı */
28 kamu düzeni Demo() {
29 // Bir JLabel örneği oluştur ve renkleri ayarla
30 jLabel1.setBackground(Renk.yellow); n 31 jLabel1.se
31 tBorder(jLabel1Border);
32
33 // Başlık konumunu yerleştir butonları
34 JPanel jpPosition = t.newt JPanel(); n 35 jpPositio
35 n.setLayout(t.newt GridLayout(t3t,t 2t));n 36 jpPosition.add(
37 jrbÜstAlt = new JRadioButton("ALTINDA_ÜST"));
38 jpPosition.add(jrbÜstÜst = new JRadioButton("ÜSTTE_ÜST"));
39 jpPosition.add(jrbAlt = new JRadioButton("ALT"));
40 jpPosition.add(jrbÜst = new JRadioButton("ÜST"));
41 jpPosition.add(
42 jrbBelowBottom = new JRadioButton("AŞAĞI_ALT"));
```

```

43 jpPosition.add(jrbBelowTop = new JRadioButton("AŞAĞI ÜST"));
44 jpPosition.setBorder(new TitledBorder("Pozisyon"));
45
46 // Başlık hizalama radyo düğmelerini yerleştir
47 JPanel jpJustification = new JPanel();
48 jpJustification.setLayout(new GridLayout(3,1));
49 jpJustification.add(jrbLeft = new JRadioButton("SOL"));
50 jpJustification.add(jrbCenter = new JRadioButton("ORTA"));
51 jpJustification.add(jrbRight = new JRadioButton("SAĞ"));
52 jpJustification.setBorder(new TitledBorder("Hizalama"));
53
54 // jpPosition ve
55 // jpJustification'ı tutmak için jpTitleOptions panelini oluştur
56 JPanel jpTitleOptions = t.newt JPanel(); n 57 jpTitleOptions.setLayout(t.newt BorderLayout()); n 58 jpTitleOptions.add(jpPosition, BorderLayout.CENTER); n 59 jpTitleOptions.add(jpJustification, BorderLayout.EAST);
57
58 // Bir onay kutusu ve başlık konumunu tutmak için jpTitle panelini oluştur
59 // radyo düğmeleri ve başlık gerekçesi radyo düğmeleri
60 JPanel jpTitle = t.newt JPanel(); n 64 jpTitle.setBorder(t.newt TitledBorder(t"Border Title"));n 65 jpTitle.setLayout(t.newt BorderLayout());
61 jpTitle.add(jchkTitled = t.newt JCheckBox(t"Titled"),n 67 BorderLayout.NORTH);
62
63 jpTitle.add(jpTitleOptions, BorderLayout.CENTER);
64
65 // Başlık konumu için radyo düğmelerini gruplandır
66 ButtonGroup btgTitlePosition = t.newt ButtonGroup(); n 72 btgTitlePosition.add(jrbAboveBottom);
67 btgTitlePosition.add(jrbBottom); n 74 btgTitlePosition.add(jrbBelowBottom); n 75 btgTitlePosition.add(jrbAboveTop); n 76 btgTitlePosition.add(jrbTop); n 77 btgTitlePosition.add(jrbBelowTop);
68
69
70 // Başlık hizalaması için radyo düğmelerini gruplandır
71 ButtonGroup btgTitleJustification = t.newt ButtonGroup(); n 81 btgTitleJustification.add(jrbLeft);
72 btgTitleJustification.add(jrbCenter); n 83 btgTitleJustification.add(jrbRight);
73
74
75 // Kenarlık stili radyo düğmelerini tutmak için jpBorderStyle Paneli oluştur
76 JPanel jpBorderStyle = new JPanel();
77 jpBorderStyle.setBorder(new TitledBorder("Kenarlık Stili"));
78 jpBorderStyle.setLayout(new GridLayout(6,1));
79 jpBorderStyle.add(jrbLoweredBevel = new JRadioButton("Alçaltılmış Bevel"));
80 jpBorderStyle.add(jrbRaisedBevel = new JRadioButton("Yükseltilmiş Bevel"));
81 jpBorderStyle.add(jrbEtched = new JRadioButton("Yontulmuş"));
82 jpBorderStyle.add(jrbLine = new JRadioButton("Çizgi"));
83 jpBorderStyle.add(jrbMatte = new JRadioButton("Matte"));
84 jpBorderStyle.add(jrbEmpty = new JRadioButton("Boş"));
85
86 // Kenarlık stilleri için radyo düğmelerini gruplandır
87 ButtonGroup btgBorderStyle = t.newt ButtonGroup(); n100 btgBorderStyle.add(jrbLoweredBevel);
88 btgBorderStyle.add(jrbRaisedBevel);
89

```

```

102 btgBorderStyle.add(jrbEtched); n103 btgBord
erStyle.add(jrbLine); n104 btgBorderStyle.add(j
rbMatte); n105 btgBorderStyle.add(jrbEmpty);

106
107 // jpTitle ve jpBorderStyle'ı yerleřtirmek için jpAllChoices Paneli oluřtur
108 JPanel jpAllChoices = t.newt JPanel(); n109 jpAllChoi
ces.setLayout(t.newt BorderLayout()); n110 jpAllChoices.add(jpTitl
e, BorderLayout.CENTER); n111 jpAllChoices.add(jpBorderStyle, Border
Layout.EAST);
112
113 // Panelleri çerçeveye yerleřtir
114 setLayout(t.newt BorderLayout()); n115 add(jL
abel1, BorderLayout.CENTER); n116 add(jpAllChoices,
BorderLayout.SOUTH);
117
118 // Dinleyicileri kaydet
119 ActionListener dinleyici = new ActionListener() ;
120 jchkTitled.addActionListener(dinleyici);
121 jrbAboveBottom.addActionListener(dinleyici);
122 jrbBottom.addActionListener(dinleyici);
123 jrbBelowBottom.addActionListener(dinleyici);
124 jrbAboveTop.addActionListener(dinleyici);
125 jrbTop.addActionListener(dinleyici);
126 jrbBelowTop.addActionListener(dinleyici);
127 jrbLeft.addActionListener(dinleyici);
128 jrbCenter.addActionListener(dinleyici);
129 jrbRight.addActionListener(dinleyici);
130 jrbLoweredBevel.addActionListener(dinleyici);
131 jrbRaisedBevel.addActionListener(dinleyici);
132 jrbLine.addActionListener(dinleyici);
133 jrbEtched.addActionListener(dinleyici);
134 jrbMatte.addActionListener(dinleyici);
135 jrbEmpty.addActionListener(dinleyici);
136 }
137
138 özel sınıf OlayDinleyici Uygular ActionListener {
139 @Override /** Onay kutusu ve radyo düğmeleri üzerindeki ActionEvents'i işleyin */
140 public void actionPerformed(ActionEvent e) {
141 // Kenarlık stilini al
142 Kenarlık kenarlık = yeni EmptyBorder (2, 2, 2, 2) ;
143
144 eğer (jrbLoweredBevel.seçili()) {
145 kenarlık = yeni BevelBorder(BevelBorder.DÜŞÜK);
146 jLabel1.setText ("Alçaltılmış Bevel Stili");
147 }
148 aksi takdirde eğer (jrbRaisedBevel.seçili()) {
149 kenarlık = yeni BevelBorder(BevelBorder.YÜKSEK);
150 jLabel1.setText ("Yükseltilmiş Bevel Stili");
151 }
152 aksi takdirde eğer (jrbEtched.seçili()) {
153 border = new EtchedBorder();
154 jLabel1.setText ("Yontulmuş Stil");
155 }
156 else ift (jrbLine.isSelected()) {
157 border = new LineBorder(Color.black, 5);
158 jLabel1.setText ("Çizgi Stili");
159 }
160 else ift (jrbMatte.isSelected()) {

```

```

161 kenarlık = t.newt MatteBorder(t15t,t 15t,t 15t,t 15t,
162 new ImageIcon(getClass().getResource)
163 ("/image/caIcon.gif"));
164 jLabel1.setText("Matte Stili");
165 }
166 else if (jrbEmpty.isSelected()) {
167 border = new EmptyBorder(2, 2, 2, 2);
168 jLabel1.setText("Boş Stil");
169 }
170
171 if (jchkTitled.isSelected()) {
172 // Başlık konumunu ve gerekçesini al
173 int titlePosition = TitledBorder.DEFAULT_POSITION;
174 int titleJustification = TitledBorder.DEFAULT_JUSTIFICATION;
175
176 if (jrbAboveBottom.isSelected()) n177 tit
177 lePosition = TitledBorder.ABOVE_BOTTOM;
178 else if (jrbBottom.isSelected())
179 titlePosition = TitledBorder.BOTTOM;
180 else if (jrbBelowBottom.isSelected())
181 titlePosition = TitledBorder.ALTINDA;
182 else if (jrbAboveTop.isSelected())
183 titlePosition = TitledBorder.YUKARIDA;
184 else if (jrbTop.isSelected())
185 titlePosition = TitledBorder.EN_ÜSTTE;
186 else if (jrbBelowTop.isSelected())
187 titlePosition = TitledBorder.YUKARIDAN_AŞAĞIDA;
188
189 if (jrbLeft.isSelected())
190 titleJustification = TitledBorder.SOLDAN;
191 else if (jrbCenter.isSelected())
192 titleJustification = TitledBorder.CENTER;
193 diğer eğer (jrbRight.seçildi())
194 titleJustification = TitledBorder.RIGHT;
195
196 jLabel1Border = yeni TitledBorder("A Başlık");
197 jLabel1Border.setBorder(border);
198 jLabel1Border.setTitlePosition(titlePosition); n199 jLabel1Border.setTitl
199 eJustification(titleJustification); n200 jLabel1.setBorder(jLabel1Border);
201 }
202 diğer {
203 jLabel1.setBorder(kenarlık); n204 }
205 }
206 }
207 }

```

Bu örnek, istenen görünümü elde etmek için UI bileşenlerini gruplamak üzere birçok panel kullanmaktadır. Şekil 37.20, panellerin ilişkisini göstermektedir. Sınır Başlık paneli, başlık özelliklerini ayarlamak için tüm seçenekleri gruplar. Konum seçenekleri, Konum panelinde gruplanmıştır. Gerekçe seçenekleri, Gerekçe panelinde gruplanmıştır. Sınır Stili paneli, Alçaltılmış Bevel, Yükseltilmiş Bevel, Aşındırılmış, Çizgi, Mat ve Boş kenarlıkları seçmek için radyo düğmelerini gruplar.

Etiket, başlık onay kutusunun seçimine bağlı olarak, başlıklı veya başlıksız seçilebilir. Etiket ayrıca hangi tür kenarlığın kullanıldığını belirten bir metin de gösterir,

kenar stili panelindeki radyo düğmesinin seçimine bağlı olarak.

TitledBorder, diğer kenarlıklarla karıştırılabilir. Bunu yapmak için, basitçe bir TitledBorder örneği oluşturun ve yeni bir kenarlığı TitledBorder içine yerleştirmek için setBorder yöntemini kullanın.

MatteBorder, kenarlık üzerinde simgeleri görüntülemek için kullanılabilir; bu, Şekil 37.20b'de gösterilmiştir.

## Bölüm Özeti

1. javax.swing.JRootPane, Swing'in üst düzey kapsayıcıları, örneğin JFrame, JApplet ve JDialog tarafından sahne arkasında kullanılan hafif bir kapsayıcıdır. javax.swing.JLayeredPane, isteyeğe bağlı menü çubuğunu ve içerik panelini yöneten bir kapsayıcıdır. İçerik paneli, Container örneğidir. Varsayılan olarak, BorderLayout ile bir JPanel'dir. Kullanıcı arayüzü bileşenlerinin eklendiği kapsayıcıdır. Bir JFrame veya JApplet içindeki içerik panelini elde etmek için getContentPane() yöntemi kullanılır. Herhangi bir Container örneğini yeni bir içerik paneli olarak ayarlamak için setContentPane yöntemini kullanabilirsiniz.
2. Her kapsayıcının bileşenlerini düzenlemekten sorumlu bir yerleşim yöneticisi vardır. Kapsayıcının setLayout yöntemi, bir yerleşim yöneticisi ayarlamak için kullanılabilir. Belirli türdeki kapsayıcıların varsayılan yerleşim yöneticileri vardır.
3. Yerleşim yöneticisi, bileşenleri kendi kurallarına ve özellik ayarlarına göre ve kısıtlı amalarla yerleştirir. her bileşenle ilişkilidir. Her yerleşim yöneticisinin kendi spesifik kural seti vardır. Bazı yerleşim yöneticileri, bileşenlerin kapsayıcı içindeki boyutunu ve konumunu etkileyebilecek özelliklere sahiptir.
4. Java ayrıca, bileşenleri sabit konumlarda yerleştirmenizi sağlayan mutlak yerleşimi destekler. Bu durumda, bileşen bileşenin örnek yöntemini kullanarak yerleştirilmelidir `setBounds()` (`java.awt.Component` içinde tanımlanmıştır). Mutlak konumlar ve boyutlar uygulanabilir, eğer uygulama aynı platformda geliştirilip dağıtılıyorsa, bir geliştirme sisteminde iyi görünen bir şey, farklı bir platformda bir dağıtım sisteminde doğru görünmeyebilir. Bu sorunu çözmek için, Java, bileşenleri kapsayıcılara farklı yazı tipleri, ekran çözünürlükleri ve işletim sistemlerinden ağımsız bir şekilde yerleştiren bir dizi yerleşim yöneticisi sağlar.
5. Java'da sağlanan yerleşim yöneticilerine ek olarak, `LayoutManager` arayüzünü `uygulayarak özel yerleşim yöneticileri oluşturabilirsiniz.`
6. Java, sabit yerleşim yöneticilerine sahip özel kapsayıcılar `Box`, `JScrollPane`, `JTabbedPane` ve `JSplitPane` sağlar.
7. Bir Swing kenarlığı, `Border` arayüzünde tanımlanır. Her JComponent örneği, JComponent içinde tanımlanan border özelliği aracılığıyla bir kenarlık ayarlayabilir. Eğer bir kenarlık varsa, inset'i değiştirir. Sekiz somut kenarlık sınıfı vardır: `BevelBorder`, `SoftBevelBorder`, `CompoundBorder`, `EmptyBorder`, `EtchedBorder`, `LineBorder`, `MatteBorder`, ve `TitledBorder`. Bu sınıfların ya `pıcılarını veya javax.swing.BorderFactory` içindeki statik yöntemleri kullanarak kenarlıklar oluşturabilirsiniz.

## Test Soruları



Bu bölüm için test sorularını çevrimiçi olarak  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

#### Gözden Geçirme Soruları

##### Bölüm 37.2

37.1 **JButton** **Container** sınıfının bir alt sınıfı olduğuna göre, bir butonu butonun içine ekleyebilir misiniz?

37.2 **JFrame**'nin başlık çubuğunda bir resim simgesi nasıl ayarlanır?  
**JApplet**'nin başlık çubuğunda bir resim simgesi ayarlayabilir misiniz?

37.3 Aşağıdakilerden hangileri **JFrame**, **JApplet**, ve **JPanel** içindeki özelliklerdir?

**contentPanet**, **t iconImaget**, **t jMenuBar**, **t yeniden boyutlandırılabilir**, **t başlık**

##### Bölüm 37.3

37.4 Java'daki yerleşim, Visual Basic'teki yerleşimlerden nasıl farklıdır?

37.5 Bir

kapsayıcıdaki bileşenlerin boyutunu belirleyen faktörleri tartışın.

**37.6 preferredSize**, **minimumSize** ve **maximumSize** özelliklerini tartışın.

37.7 **alignmentX** ve **alignmentY** özelliklerini tartışın.

37.8 **CardLayout** yöneticisi nedir? Bir **CardLayout** yöneticisi nasıl oluşturulur?

37.9 Java'da mutlak konumlandırma kullanabilir misiniz? Mutlak konumlandırmayı nasıl kullanırsınız? Kullanımından neden kaçınmalısınız?

37.10 **BoxLayout** nedir? **BoxLayout** nasıl kullanılır? Bileşenleri ayırmak için dolgu elemanlarını nasıl kullanırsınız?

##### Bölmeler 37.4-37.7

37.11 Özel bir yerleşim yöneticisi nasıl oluşturursunuz?

37.12 **JScrollPane** nedir? **JScrollPane** nasıl kullanılır?

37.13 **JTabbedPane** nedir? **JTabbedPane** nasıl kullanılır?

37.14 **JSplitPane** nedir? **JSplitPane** nasıl kullanılır?

37.15 **Box**, **JScrollPane**, **JTabbedPane** ve **JSplitPane**'de bir yerleşim yöneticisi belirtebilir misiniz?

##### Bölüm 37.8 Swing Kenarlıkları

37.16 Başlıklı bir kenarlık, bir çizgi kenarlığı, bir kabartma kenarlığı ve bir asitlenmiş kenarlık nasıl oluşturursunuz?

37.17 Her Swing GUI bileşeni için bir kenarlık ayarlayabilir misiniz? Bir kenarlık nesnesi farklı GUI bileşenleri tarafından paylaşılabilir mi?

**37.18 Hangi paket **Border**, **BevelBorder**, **CompoundBorder**, **EmptyBorder**, **EtchedBorder**, **LineBorder**, **MatteBorder**, **TitledBorder** ve **BorderFactory** içerir?**

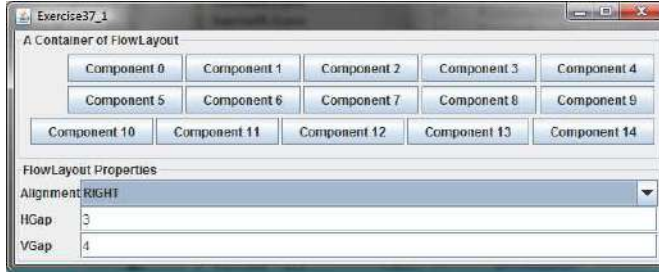
## Programlama Ağıştırmaları

### Bölüm 37.3

#### 37.1\* (FlowLayout özelliklerini gösterin) Kullanıcının

FlowLayout yöneticisinin özelliklerini dinamik olarak ayarlamasını sağlayan bir program oluşturun, Şekil 37.21'de gösterildiği gibi. **FlowLayout** yöneticisi, 15

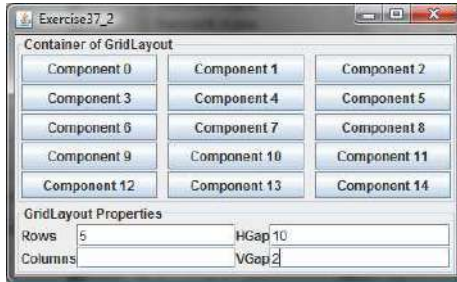
bileşeni bir panelde yerleştirmek için kullanılır. FlowLayout özelliklerini dinamik olarak ayarlayabilirsiniz: alignment, hgap ve vgap



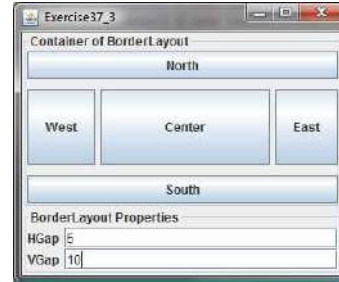
Şekil 37.21

Program, bir **FlowLayout** yöneticisinin özelliklerini dinamik olarak ayarlamanıza olanak tanır.

#### 37.2\* (GridLayout özelliklerini gösterin) Kullanıcının bir **GridLayout** yöneticisinin özelliklerini dinamik olarak ayarlamasını sağlayan bir program oluşturun, Şekil 37.22a'da gösterildiği gibi. **GridLayout** yöneticisi, bir panelde 15 bileşeni yerleştirmek için kullanılır. **GridLayout**'un satır, sütun, hgap ve vgap özelliklerini dinamik olarak ayarlayabilirsiniz.



(a)



(b)

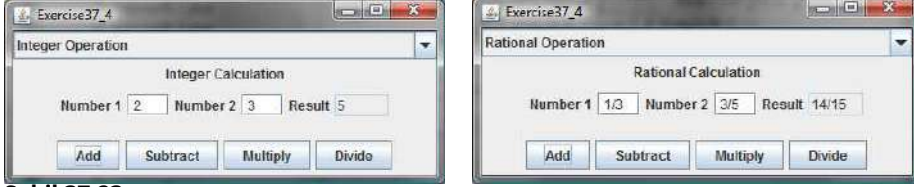
Şekil 37.22

(a) Program, bir **GridLayout** yöneticisinin özelliklerini dinamik olarak ayarlamanıza olanak tanır. (b) Program, bir

**BorderLayout** yöneticisinin özelliklerini dinamik olarak ayarlamanıza olanak tanır.

#### 37.3\* (BorderLayout özelliklerini gösterin) Kullanıcının bir **BorderLayout** yöneticisinin özelliklerini dinamik olarak ayarlamasını sağlayan bir program oluşturun Şekil 37.22b'de gösterildiği gibi. **BorderLayout** yöneticisi, bir panelde beş bileşeni yerleştirmek için kullanılır. **BorderLayout**'un hgap ve vgap özelliklerini dinamik olarak ayarlayabilirsiniz.

- 37.4\* (*CardLayout* kullanın) Tam sayılar ve rasyonel sayılar üzerinde aritmetik işlemler yapan bir applet yazın. Program, bir *CardLayout* yöneticisinde iki panel kullanır, biri tam sayı aritmetiği için diğeri rasyonel aritmetik içindir. Program, Tam Sayı ve Rasyonel olmak üzere iki öğe içeren bir açılır kutu sağlar. Kullanıcı Tam Sayı öğesini seçtiğinde, tam sayı paneli aktif hale gelir. Kullanıcı Rasyonel öğesini seçtiğinde, rasyonel paneli aktif hale gelir (bkz. Şekil 37.23).



Şekil 37.23

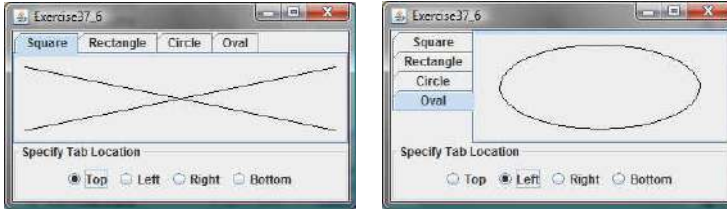
*CardLayout* tam sayı işlemleri ve

rasyonel sayı işlemleri gerçekleştiren panelleri seçmek için kullanılır.

- 37.5\* (Boş yerleşim kullanın) Şekil 18.18a'da gösterildiği gibi bir hesap makinesi yerleştirmek için mutlak yerleşim kullanın.

### Bölümler 37.4-37.8

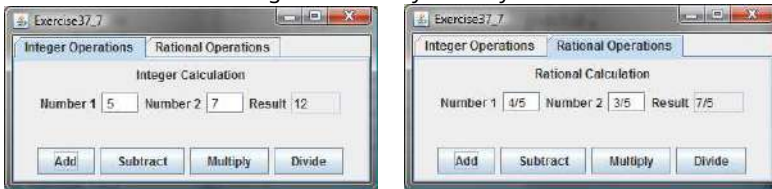
- 37.6\* (*Sekmeli paneller kullanın*) Sekmeli panelin sekme yerleşimini belirtmek için radyo düğmeleri içeren bir panel eklemek üzere Listeleme 37.7, DisplayFigure.java dosyasını değiştirin; bu, Şekil 37.24'te gösterilmiştir.



Şekil 37.24

Radyo düğmeleri, sekmeli panelin sekme yerleşimini seçmenizi sağlar.

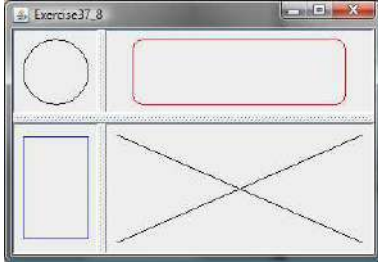
- 37.7\* (*Sekmeli panel kullanın*) Şekil 37.25'te görüldüğü gibi CardLayout yerine sekmeli paneller kullanılarak Egzersiz 37.4'ü yeniden yazın.



Şekil 37.25

Bir sekmeli panel, tam sayı işlemleri ve rasyonel sayı işlemleri gerçekleştiren panelleri seçmek için kullanılır.

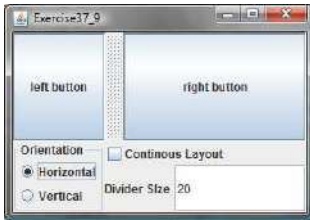
- 37.8\* (*JSplitPane kullanın*) Dört şekli bölünmüş panellerde gösteren bir program oluşturun, Şekil 37.26'da gösterildiği gibi. *FigurePanel* sınıfını Listing 15.3, FigurePanel.java'da tanımlandığı gibi kullanın.



**Şekil 37.26**

Dört şekil bölünmüş panellerde gösterilmektedir.

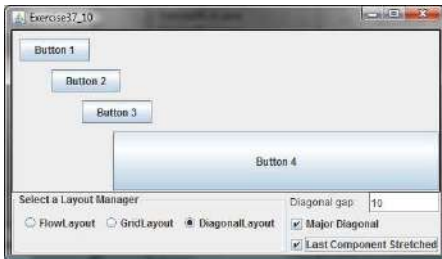
- 37.9\* (*JSplitPane özelliklerini Göstermek*) Kullanıcının bir bölünmüş panelin özelliklerini dinamik olarak ayarlamasını sağlayan bir program oluşturun, Şekil 37.27'de gösterildiği gibi.



**Şekil 37.27**

Program, bir bölünmüş panelin özelliklerini dinamik olarak ayarlamanıza olanak tanır.

- 37.10\* (*DiagonalLayout özelliklerini gösterin*) Liste 37.5'i yeniden yazın ShowDiagonalLayout.java, bir panel ekleyin ve bu panel **DiagonalLayout** özelliklerini gösterir. Panel, DiagonalLayout radyo butonu işaretlenmediğinde kaybolur ve DiagonalLayout radyo butonu işaretlendiğinde yeniden görünür; bu, Şekil 37.28'de gösterilmiştir.



**Şekil 37.28**

Program, DiagonalLayout'un özelliklerini dinamik olarak ayarlamanıza olanak tanır.

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 38

### Menüler, Araç Çubukları ve Diyaloglar

#### Hedefler

- JMenuBar, JMenu, JMenuItem, \_\_\_\_\_  
JCheckBoxMenuItem ve JRadioButtonMenuItem bileşenlerini kullanarak menüler oluşturmak (§38.2).
- JPopupMenu, JMenuItem, \_\_\_\_\_  
JCheckBoxMenuItem ve JRadioButtonMenuItem bileşenlerini kullanarak açılır menüler oluşturmak (§38.3).
- JToolBar kullanarak araç çubukları oluşturmak (§38.4).
- Action nesnelerini kullanarak işlemleri işlemek için kodu genelleştirmek (§38.5).
- JOptionPane sınıfını kullanarak standart diyaloglar oluşturmak (§38.6).
- Özel diyaloglar oluşturmak için JDialog sınıfını genişletmek (§38.7).
- JColorChooser kullanarak renkleri seçmek (§38.8).
- Open ve Save File diyaloglarını görüntülemek için JFileChooser kullanmak (§38.9).

### 38.1 Giriş

Java, grafik kullanıcı arayüzleri oluşturmak için kapsamlı bir çözüm sunar. Bu bölüm menüler, açılır menüler, araç çubukları ve diyalogları tanıtmaktadır. Ayrıca, işlemleri işlemek için kodu genelleştirmek amacıyla Action nesnelerini nasıl kullanacağınızı öğreneceksiniz.

### 38.2 Menüler

#### <kenar notu: menü>

*Menüler* seçim yapmayı kolaylaştırır ve pencere uygulamalarında yaygın olarak kullanılır. Java, menüleri uygulayan beş sınıf sağlar: JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem ve JRadioButtonMenuItem.

#### <kenar notu: menü öğesi>

JMenuBar, menüleri tutmak için kullanılan üst düzey bir menü bileşenidir. Bir menü, kullanıcının seçebileceği (veya açıp kapatabileceği) *menü öğelerinden* oluşur. Bir menü öğesi, JMenuItem, JCheckBoxMenuItem veya JRadioButtonMenuItem örneği olabilir. Menü öğeleri, ikonlar, klavye mnemonikleri ve klavye hızlandırıcıları ile ilişkilendirilebilir. Menü öğeleri, ayırıcılar kullanılarak ayrılabilir.

#### 38.2.1 Menüler Oluşturma

Java'da menüleri uygulamanın sırası şu şekildedir:

1. Bir menü çubuğu oluşturun ve bunu bir çerçeve veya bir applet ile setJMenuBar yöntemi kullanarak ilişkilendirin. Örneğin, aşağıdaki kod bir çerçeve ve bir menü çubuğu oluşturur ve menü çubuğunu çerçevede ayarlar:

```
JFrame çerçeve = new JFrame();
çerçeve.setSize(300, 200);
çerçeve.setVisible(true);
JMenuBar jmb = new JMenuBar();
çerçeve.setJMenuBar(jmb); // Bir çerçeveye menü çubuğu ekle
```

2. Menüler oluşturun ve bunları menü çubuğuna bağlayın. Aşağıdaki yapıyı kullanarak bir menü oluşturabilirsiniz:

```
public JMenu(String etiket)
```

İşte menü oluşturmanın bir örneği:

```
JMenu fileMenu = new JMenu("Dosya");
JMenu helpMenu = new JMenu("Yardım");
```

Bu, Şekil 38.1(a)'de gösterildiği gibi, Dosya ve Yardım etiketli iki menü oluşturur. Menüler, bir JMenuBar örneğine eklenene kadar görünmeyecektir, şu şekilde:

```
jmb.add(fileMenu);jmb.add(helpMenu);
```



(a) (b) (c)

**Şekil 38.1**

(a) Menü çubuğu, çerçevenin başlık çubuğunun altında görünür. (b) Menü çubuğundaki bir menüye tıklamak, menünün altındaki öğeleri açığa çıkarır. (c) Bir menü öğesine tıklamak, menü öğesinin altındaki alt menü öğelerini açığa çıkarır.

3. Menü öğeleri oluşturun ve bunları menülere ekleyin.

```
fileMenu.add(new JMenuItem("Yeni"));
fileMenu.add(new JMenuItem("Aç"));
fileMenu.addSeparator();
fileMenu.add(new JMenuItem("Yazdır"));
fileMenu.addSeparator();
fileMenu.add(new JMenuItem("Çıkış"));
```

Bu kod, Dosya menüsüne Yeni, Aç, bir ayırıcı çubuk, Yazdır, başka bir ayırıcı çubuk ve Çıkış menü öğelerini, bu sırayla, ekler; Şekil 38.1(b)'de gösterildiği gibi. addSeparator() metodu menüye bir ayırıcı çubuk ekler.

### 3.1. Alt menü öğeleri oluşturma.

Menüler içinde menüler yerleştirerek,

yerleştirilen menülerin alt menüler haline gelmesini sağlayabilirsiniz. İşte bir örnek:

```
JMenu yazılımYardımltMenüsü = new JMenu("Yazılım");
JMenu donanımYardımltMenüsü = new JMenu("Donanım");
helpMenu.add(yazılımYardımltMenüsü);
helpMenu.add(donanımYardımltMenüsü);
yazılımYardımltMenüsü.add(new JMenuItem("Unix"));
yazılımYardımltMenüsü.add(new JMenuItem("NT"));
yazılımYardımltMenüsü.add(new JMenuItem("Win95"));
```

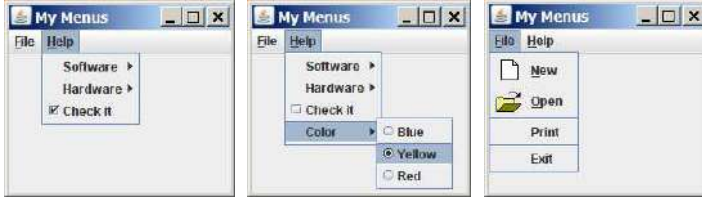
Bu kod, helpMenu içinde iki alt menü, yazılımYardımltMenüsü ve donanımYardımltMenüsü ekler. Menü öğeleri Unix, NT ve Win95 yazılımYardımltMenüsü'ne eklenir (bkz. Şekil 38.1(c)).

### 3.2. Onay kutusu menü öğeleri oluşturma.

Bir JMenu'ye bir JCheckBoxMenuItem de ekleyebilirsiniz.

JCheckBoxMenuItem, JMenuItem'in bir alt sınıfıdır ve JMenuItem'e bir Boolean durumu ekler ve durumu doğru olduğunda bir onay işareti gösterir. Bir menü öğesine tıklayarak onu açıp kapatabilirsiniz. Örneğin, aşağıdaki ifade onay kutusu menü öğesi "Onayla"yı ekler (bkz. Şekil 38.2(a)).

```
helpMenu.add(new JCheckBoxMenuItem("Onayla"));
```



(a)

(b)

(c)

**Şekil 38.2**

(a) Bir onay kutusu menü öğesi, bir menü öğesini onaylamak veya onayını kaldırmak için onay kutusu gibi kullanılabilir. (b) `JRadioButtonMenuItem` kullanarak karşılıklı olarak dışlayıcı menü seçimleri arasında seçim yapabilirsiniz. (c) Menüde resim simgeleri, klavye mnemonikleri ve klavye kısayolları ayarlayabilirsiniz.

### 3.3. Radyo düğmesi menü öğeleri oluşturma.

Ayrıca, `JRadioButtonMenuItem` sınıfını kullanarak bir menüye radyo düğmeleri ekleyebilirsiniz. Bu, genellikle menüde karşılıklı olarak dışlayıcı bir grup seçim olduğunda faydalıdır. Örneğin, aşağıdaki ifadeler, Renk adında bir alt menü ve bir renk seçimi için radyo düğmeleri seti ekler (bkz. Şekil 38.2(b)):

```
JMenu colorHelpSubMenu = new JMenu("Renk");
helpMenu.add(colorHelpSubMenu);

JRadioButtonMenuItem jrbmiMavi, jrbmiSarı, jrbmiKırmızı; ncolorHelp
pSubMenu.add(jrbmiBlue =
 new JRadioButtonMenuItem("Mavi"));
colorHelpSubMenu.add(jrbmiYellow =
 new JRadioButtonMenuItem("Sarı"));
colorHelpSubMenu.add(jrbmiRed =
 new JRadioButtonMenuItem("Kırmızı"));

ButtonGroup btg = new ButtonGroup();
btg.add(jrbmiBlue);
btg.add(jrbmiYellow);nbtg.
add(jrbmiRed);
```

### 4. Menü öğeleri `ActionEvent` oluşturur. Dinleyici sınıfınız, menü seçimlerine yanıt vermek için `ActionListener`'ı ve

`actionPerformed` işleyicisini uygulamalıdır.

### 38.2.2 Görüntü Simgeleri, Klavye Mnemonikleri ve Klavye Hızlandırıcıları

Menü bileşenleri `JMenu`, `JMenuItem`, `JCheckBoxMenuItem` ve `JRadioButtonMenuItem` simge ve mnemonik özelliklere sahiptir. Örneğin, aşağıdaki kodu kullanarak Yeni ve Aç menü öğeleri için simgeleri ayarlayabilir ve Dosya, Yardım, Yeni ve Aç için klavye mnemoniklerini ayarlayabilirsiniz:

```
JMenuItem jmiNew, jmiOpen;
fileMenu.add(jmiNew = new JMenuItem("Yeni"));
fileMenu.add(jmiOpen = new JMenuItem("Aç"));
jmiNew.setIcon(new ImageIcon("image/new.gif"));
jmiOpen.setIcon(new ImageIcon("image/open.gif"));
helpMenu.setMnemonic('H');
fileMenu.setMnemonic('F');n jmi
New.setMnemonic('N');n jmiO
pen.setMnemonic('O');
```



Yeni simgeler ve mnemonikler Şekil 38.2(c)'de gösterilmektedir. Ayrıca, bir simge veya mnemonik oluşturmak ve ayarlamak için aşağıdaki gibi JMenuItem yapıcılarını da kullanabilirsiniz.

```
public JMenuItem(Dize etiketi, Simge simgesi);
public JMenuItem(Dize etiketi, renk tonu animatıcı);
```

Varsayılan olarak, metin simgenin sağındadır. Metni simgenin solu na yerleştirmek için `setHorizontalTextPosition(SwingConstants.LEFT)` kullanın.

### <kenar notu: hızlandırıcı>

Bir menüyü seçmek için ALT tuşuna ve mnemonik tuşa basın. Örneğin, Dosya menüsünü seçmek için ALT+F'ye, ardından Aç menü öğesini seçmek için ALT+O'ya basın. Klavye mnemonikleri kullanışlıdır, ancak yalnızca mevcut açık menüden menü öğelerini seçmenizi sağlar. Ancak, tuş hızlandırıcıları, bir menü öğesini doğrudan CTRL ve hızlandırıcı tuşlarına basarak seçmenizi sağlar. Örneğin, aşağıdaki kodu kullanarak Aç menü öğesine CTRL+O hızlandırıcı tuşunu ekleyebilirsiniz:

```
jmiOpen.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_O, ActionEvent.CTRL_MASK));
```

`setAccelerator` yöntemi bir `KeyStroke` nesnesi alır. `KeyStroke` sınıfındaki `getKeyStroke` yöntemi, tuş vuruşunun bir örneğini oluşturur. `VK_O` O tuşunu temsil eden bir sabittir ve `CTRL_MASK` CTRL tuşunun tuş vuruşuyla ilişkili olduğu belirlenen bir sabittir.

NOT: Şekil 17.1'de gösterildiği gibi, `AbstractButton`, `JButton` ve `JMenuItem` için süper sınıftır ve `JMenuItem`, `JCheckBoxMenuItem`, `JMenuItem` ve `JRadioButtonMenuItem` için bir süper sınıftır. Menü bileşenleri butonlara çok benzer.

### 38.2.3 Örnek: Menüler Kullanma

Bu bölüm, aritmetik işlemler gerçekleştirmek için bir kullanıcı arayüzü oluşturmak için bir örnek verir. Arayüz, Sayı 1, Sayı 2 ve Sonuç için etiketler ve metin alanları içerir. Sonuç metin alanı, Sayı 1 ile Sayı 2 arasındaki aritmetik işlemin sonucunu gösterir. Şekil 38.3, programın bir örnek çalışmasını içerir.



Şekil 38.3

*Aritmetik işlemler, butonlara tıklayarak veya İşlem menüsünden menü öğelerini seçerek gerçekleştirilebilir.*

Programdaki ana adımlar şunlardır (Liste 38.1):

1. Bir menü çubuğu oluşturun ve bunu applet'e yerleştirin. İşlem ve Çıkış menülerini oluşturun ve menü çubuğuna ekleyin. İşlem menüsü altında Ekle, Çıkar, Çarp ve Böl menü öğelerini ekleyin ve Çıkış menüsü altında Kapat menü öğesini ekleyin.
2. Etiketleri ve metin alanlarını tutacak bir panel oluşturun ve paneli applet'in ortasına yerleştirin.
3. Ekle, Çıkar, Çarp ve Böl etiketli dört düğmeyi tutacak bir panel oluşturun. Paneli applet'in güneyine yerleştirin.
4. Menü öğelerinden ve düğmelerden gelen olayları işlemek için actionPerformed işleyicisini uygulayın.

Liste 38.1 MenuDemo.java

<margin note line 17: menü çubuğu>  
 <margin note line 20: menü çubuğunu ayarla>  
 <margin note line 28: menüleri kapat>  
 <margin note line 33: menü öğelerini ekle>  
 <margin note line 40: hızlandırıcı>  
 <margin note line 61: düğmeler>  
 <margin note line 72: dinleyiciyi kaydet>  
 <margin note line 78: dinleyiciyi kaydet>  
 <margin note line 84: dinleyiciyi kaydet>  
 <margin note line 90: dinleyiciyi kaydet>  
 <margin note line 96: dinleyiciyi kaydet>  
 <margin note line 102: dinleyiciyi kaydet>  
 <margin note line 108: dinleyiciyi kaydet>  
 <margin note line 114: dinleyiciyi kaydet>  
 <margin note line 120: dinleyiciyi kaydet>  
 <margin note line 129: hesap makinesi>  
 <margin note line 149: ana yöntem atlandı>

```

1 java.awt.*u içe aktarın;
2 java.awt.event.*i içe aktarın;
3 javax.swing.*i içe aktarın;
4
5 public class MenuDemo extends JApplet {
6 // Numara 1, Numara 2 ve Sonuç için metin alanları
7 özel JTextField jtfNum1, jtfNum2, jtfResult;
8
9 // Düğmeler "Ekle", "Çıkar", "Çarp" ve "Böl"
10 özel JButton jbtAdd, jbtSub, jbtMul, jbtDiv;
11
12 // Menü öğeleri "Ekle", "Çıkar", "Çarp", "Böl" ve "Kapat"
13 özel JMenuItem jmiAdd, jmiSub, jmiMul, jmiDiv, jmiClose;
14
15 genel MenuDemo() {
16 // Menü çubuğu oluştur
17 JMenuBar jmb = yeni JMenuBar();
18
19 // Menü çubuğunu applete ayarla
20 (ne zaman);
21
22 // Menü çubuğuna "İşlem" menüsünü ekle
23 JMenu operationMenu = new JMenu("İşlem");
24 operationMenu.setMnemonic('O');
25 jmb.add(operationMenu);
26
27 // Menü çubuğuna "Çıkış" menüsünü ekle
28 JMenu exitMenu = new JMenu("Çıkış");
29 exitMenu.setMnemonic('E');
30 jmb.add(exitMenu);
31

```

```

32 // "İşlem" menüsüne mnemonik ile menü öğeleri ekle
33 operationMenu.add(jmiAdd= new JMenuItem("Ekleyin", 'A'));
34 operationMenu.add(jmiSub = new JMenuItem("Çıkarın", 'S'));
35 operationMenu.add(jmiMul = new JMenuItem("Çarpın", 'M'));
36 operationMenu.add(jmiDiv = new JMenuItem("Bölün", 'D'));
37 exitMenu.add(jmiClose = new JMenuItem("Kapat", 'C'));

38
39 // Klavye kısayollarını ayarla
40 jmiAdd.setAccelerator(
41 KeyStroke.getKeyStroke(KeyEvent.VK_A, ActionEvent.CTRL_MASK)); n 42 jmiSub.setAccel
erator(
43 KeyStroke.getKeyStroke(KeyEvent.VK_S, ActionEvent.CTRL_MASK)); n 44 jmiMul.setAccel
erator(
45 KeyStroke.getKeyStroke(KeyEvent.VK_M, ActionEvent.CTRL_MASK)); n 46 jmiDiv.setAccel
erator(
47 KeyStroke.getKeyStroke(KeyEvent.VK_D, ActionEvent.CTRL_MASK));

48
49 // Metin alanları ve etiketleri tutacak panel p1
50 JPanel p1 = new JPanel(new FlowLayout());
51 p1.add(new JLabel("Sayı 1"));
52 p1.add(jtfNum1 = new JTextField(3));
53 p1.add(new JLabel("Sayı 2"));
54 p1.add(jtfNum2 = new JTextField(3));
55 p1.add(new JLabel("Sonuç"));
56 p1.add(jtfResult = t newt JTextField(4)); n 57 jtfResult.s
etEditable(false);

58
59 // Düğmeleri tutacak panel p2
60 JPanel p2 = newt JPanel(newt FlowLayout()); n 61 p2.add(j
btAdd = newt JButton("Add")); n 62 p2.add(jbtSub = t
newt JButton("Subtract")); n 63 p2.add(jbtMul = newt JButto
n("Çarp")); n 64 p2.add(jbtDiv = newt JButton("Divide")); n
65

66 // Çerçeveye paneller ekle
67 setLayout(newt BorderLayout()); n 68 add(p
1, BorderLayout.CENTER); n 69 add(p2, Border
Layout.SOUTH);

70
71 // Dinleyicileri kaydet
72 jbtAdd.addActionListener(newt ActionListener() { n 73 @Override

74 public void actionPerformed(ActionEvent e) {
75 hesapla('+'); n 76 }

77 });
78 jbtSub.addActionListener(newt ActionListener() { n 79 @Override

80 public void actionPerformed(ActionEvent e) {
81 hesapla('-'); n 82 }

83 });
84 jbtMul.addActionListener(newt ActionListener() { n 85 @Override

86 public void actionPerformed(ActionEvent e) {
87 hesapla('*'); n 88 }

89 });
90 jbtDiv.addActionListener(newt ActionListener() { n 91 @Override

92 public void actionPerformed(ActionEvent e) {
93 hesapla('/'); n 94 }

95 });
96 jmiAdd.addActionListener(newt ActionListener() { n 97 @Override

```

```

98 public void actionPerformed(ActionEvent e) {
99 hesapla('+'); n100 }

101 };
102 jmiSub.addActionListener(tnew ActionListener() { n103 @Override

104 public void actionPerformed(ActionEvent e) {
105 hesapla('-'); n106 }

107 };
108 jmiMul.addActionListener(tnew ActionListener() { n109 @Override

109 public void actionPerformed(ActionEvent e) {
110 hesapla('*');
111 }
112 };
113 };
114 jmiDiv.addActionListener(tnew ActionListener() { n115 @Override

115 public void actionPerformed(ActionEvent e) {
116 hesapla('/');
117 }
118 };
119 };
120 jmiClose.addActionListener(tnew ActionListener() { n121 @Override

121 public void actionPerformed(ActionEvent e) {
122 System.exit(t1t);n124 }

123 });n126 }

127
128 /** Sonucu hesapla ve jtfResult'ta göster */
129 private void hesapla(char operator) {
130 // Sayı 1 ve Sayı 2'yi al
131 intt sayı1 = (Integer.parseInt(jtfNum1.getText().trim()));
132 intt sayı2 = (Integer.parseInt(jtfNum2.getText().trim()));
133 intt sonuç = t0t;

134
135 // Seçilen işlemi gerçekleştir
136 anahtar (operatör) {
137 case '+': sonuç = sayı1 + sayı2;
138 kırılma;
139 case '-': sonuç = sayı1 - sayı2;
140 kırılma;
141 case '*': sonuç = sayı1 * sayı2;
142 kırılma;
143 case '/': sonuç = sayı1 / sayı2;
144 }

145
146 // Sonucu jtfResult'a ayarla
147 jtfResult.setText(String.valueOf(result)); n148 }

149 }

```

Program, iki menüyü tutan bir menü çubuğu, jmb oluşturur: operationMenu ve exitMenu (satırlar 17-30). operationMenu arithmetic işlemleri için dört menü öğesi içerir: Topla, Çıkar, Çarp ve Böl. exitMenu, programdan çıkmak için Close menü öğesini içerir. İşlem menüsündeki menü öğeleri klavye mnemonikleri ve hızlandırıcılarla oluşturulmuştur.

Kullanıcı, sayı alanlarına iki sayı girer. Bir operasyon menüden seçildiğinde, iki sayıyı içeren sonucu Sonuç alanında görüntülenir. Kullanıcı ayrıca aynı işlemi gerçekleştirmek için butonlara tıklayabilir.

private method calculate(char operator) (satırlar 129–148)  
Number 1 ve Number 2'deki metin alanlarından operandları alır,  
operandlar üzerinde ikili operatörü uygular ve sonucu  
Sonuç metin alanına ayarlar.

NOT:

#### <margin note: menüleri yerleştirme>

Menü çubuğu genellikle setJMenuBar yöntemi kullanılarak pencereye eklenir. Ancak, diğer bileşenler gibi, bir kapsayıcıya yerleştirilebilir. Örneğin, bir menü çubuğunu kapsayıcının güneyine yerleştirebilirsiniz

Kenar Düzeni.

### 38.3 Açılır Menüler

#### <kenar notu: açılır menü>

*Bir açılır menü, aynı zamanda bir bağlam menüsü olarak da bilinir, normal menüye benzer, ancak bir menü çubuğuna sahip değildir ve ekranda herhangi bir yerde yüzebilir. Bir açılır menü oluşturmak, normal bir menü oluşturmaya benzer. Öncelikle, JPopupMenu'nin bir örneğini oluşturursunuz, ardından açılır menüye JMenuItem, JCheckBoxMenuItem, JRadioButtonMenuItem ve ayrıcılar ekleyebilirsiniz. Örneğin, aşağıdaki kod bir JPopupMenu oluşturur ve içine JMenuItem'lar ekler:*

```
JPopupMenu jPopupMenu = new JPopupMenu();
jPopupMenu.add(new JMenuItem("Yeni"));
jPopupMenu.add(new JMenuItem("Aç"));
```

Bir normal menü her zaman

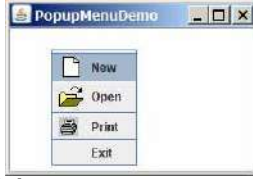
setJMenuBar yöntemi kullanılarak bir menü çubuğuna eklenir, ancak bir açılır menü bir üst bileşenle ilişkilidir ve show yöntemi kullanılarak görüntülenir. JPopupMenu sınıfı. Ebeveyn bileşeni ve açılır menünün konumunu belirtirsiniz, ebeveynin koordinat sistemini kullanarak şöyle:

```
jPopupMenu.show(bileşen, x, y);
```

#### <kenar notu: açılır tetikleyici>

Geleneksel olarak, bir GUI kapsayıcısına işaret ederek ve belirli bir fare düğmesine tıklayarak bir açılır menü gösterirsiniz, bu da açılır tetikleyici olarak adlandırılır. Açılır tetikleyiciler sistem bağımlıdır. Windows'ta, açılır menü sağ fare düğmesi irakıldığında gösterilir. Motif'te, açılır menü üçüncü fare düğmesine basıldığında ve basılı tutulduğunda gösterilir.

Liste 38.2, bir kaydırılabilir panelde bir metin alanı oluşturan bir örnek vermektedir. Fare metin alanına işaret ettiğinde, bir fare düğmesine tıklamak bir açılır menü gösterir; bu, Şekil 38.4'te gösterilmiştir.



**Şekil 38.4**

*Metin alanında açılır tetikleyici verildiğinde bir açılır menü görüntülenir.*

Programdaki ana adımlar (Liste 38.2):

1. JPopupMenu kullanarak bir açılır menü oluşturun. Yeni, Aç, Yazdır ve Çıkış için JMenuItem kullanarak menü öğeleri oluşturun. Hem etiket hem de simgeye sahip menü öğeleri için, JMenuItem(label, icon) yapısını kullanmak uygundur.
2. Menü öğelerini açılır menüye ekleyin.
3. Bir kaydırılabilir panel oluşturun ve içine bir metin alanı ekleyin. Kaydırılabilir paneli applet'in ortasına yerleştirin.
4. Menü öğelerinden gelen olayları işlemek için actionPerformed işleyicisini uygulayın.
5. Açılır tetikleyicileri işlemek için mousePressed ve mouseReleased yöntemlerini uygulayın.

**Listeleme 38.2 PopupMenuDemo.java**

<kenar notu satır 6: açılır menü>  
 <kenar notu satır 17: menü öğeleri ekle>  
 <kenar notu satır 27: dinleyiciyi kaydet>  
 <kenar notu satır 33: dinleyiciyi kaydet>  
 <kenar notu satır 39: dinleyiciyi kaydet>  
 <kenar notu satır 45: dinleyiciyi kaydet>  
 <kenar notu satır 54: açılır menüyü göster>  
 <kenar notu satır 59: açılır menüyü göster>  
 <kenar notu satır 69: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2 java.aut.*'u içe aktarın;
3 java.awt.event.*'i içe aktarın;
4
5 genel sınıf PopupMenuDemo t extendst JApplet {
6 özel JPopupMenu jPopupMenu1 = t newt JPopupMenu();
7 private JMenuItem jmiNew = new JMenuItem("Yeni",
8 new ImageIcon(getClass().getResource("image/new.gif"));
9 private JMenuItem jmiOpen = new JMenuItem("Aç",
10 new ImageIcon(getClass().getResource("image/open.gif"));
11 private JMenuItem jmiPrint = new JMenuItem("Yazdır",
12 new ImageIcon(getClass().getResource("image/print.gif"));
13 private JMenuItem jmiExit = new JMenuItem("Çıkış");
14 özel JTextArea jTextArea 1 = t newt JTextArea();
15
16 herkese açık PopupMenu Demo() {
17 jPopupMenu1.add(jmiNew); n 18 jPo
18 pupMenu1.add(jmiOpen); n 19 jPopupMe
19 nu1.addSeparator(); n 20 jPopupMenu1.add
20 (jmiPrint); n 21 jPopupMenu1.addSeparato
21 r(); n 22 jPopupMenu1.add(jmiExit); n 23 jPo
22 pupMenu1.add(jmiExit);
23
24

```

```

25 add(tnewt JScrollPane(jTextArea1), BorderLayout.CENTER);
26
27 jmiNew.addActionListener(tnewt ActionListener() { n 28 @Override
29 public void actionPerformed(ActionEvent e) {
30 System.out.println("Yeni İşlem");
31 }
32 });
33 jmiOpen.addActionListener(tnewt ActionListener() { n 34 @Override
35 public void actionPerformed(ActionEvent e) {
36 System.out.println("Open İşlemi");
37 }
38 });
39 jmiPrint.addActionListener(tnewt ActionListener() { n 40 @Override
41 public void actionPerformed(ActionEvent e) {
42 System.out.println("Yazdırma İşlemi");
43 }
44 });
45 jmiExit.addActionListener(tnewt ActionListener() { n 46 @Override
47 public void actionPerformed(ActionEvent e) {
48 System.exit(1);
49 }
50 });
51 jTextArea1.addMouseListener(tnewt MouseAdapter() { n 52 @Override
53 public void mousePressed(MouseEvent e) { // Motif için
54 showPopup(e); n 55 }
56
57 @Geçersiz Kıl
58 public void mouseReleased(MouseEvent e) { // Windows için
59 showPopup(e); n 60 }
61 }); n 62 }
63
64 /** Tetiklendiğinde açılır menüyü göster */
65 özel voidt showPopup(java.awt.event.MouseEvent evt) {
66 if (event.isPopupTrigger())
67 jPopupMenu1.show(event.getComponent(), event.getX(), event.getY()); n 68 }
69 }

```

Popup menülerin oluşturulma süreci, normal menülerin oluşturulma sürecine benzer. Bir popup menü oluşturmak için, bir JPopupMenu'yu temel alarak (satır 6) ona JMenuItems ekleyin (satırlar 17-23) .

Bir açılır menüyü göstermek için, ana bileşeni ve açılır menü için konumu belirterek show yöntemini kullanın (satır 61). show yöntemi, açılır menü metin alanında belirli bir fare tıklamasıyla tetiklendiğinde çağrılır. Açılır tetikleyiciler sistem bağımlıdır. Dinleyici, Windows'ta açılır menüyü göstermek için mouseReleased işleyicisini ve Motif'te açılır menüyü göstermek için mousePressed işleyicisini uygular (satırlar 52-54) (satırlar 48-50).

## &lt;margin note: açılır menüyü basitleştir&gt;

Java, JComponent sınıfında bir bileşene açılır menü eklemek için kullanılabilir yeni `setComponentPopupMenu(JPopupMenu)` yöntemini sağlar. Bu yöntem, fare dinleyici kaydını ve açılır menü görüntüleme sini otomatik olarak yönetir. Bu yöntemi kullanarak, satır 59-62'deki `showPopup` yöntemini silebilir ve satır 47-55'teki kodu aşağıdaki ifade ile değiştirebilirsiniz:

```
itetarea1.setComponentPopupMenu(jpopupmenu1);
```

## \*\*\*İPUCU SONU

## 38.4 JToolBar

## &lt;margin note: araç çubuğu&gt;

Kullanıcı arayüzlerinde, *araç çubuğu* genellikle menülerde de görünen komutları tutmak için kullanılır. Sık kullanılan komutlar, hızlı erişim için bir araç çubuğuna yerleştirilir. Araç çubuğundaki bir komuta tıklamak, menüden seçmekten daha hızlıdır.

Swing, araç çubuğu bileşenlerini tutmak için JToolBar sınıfını sağlar. JToolBar, bileşenleri yönetmek için varsayılan olarak `BoxLayout` kullanır. İstenirse farklı bir yerleşim yöneticisi ayarlayabilirsiniz.

Bileşenler genellikle simgeler olarak görünür. Simgeler bileşen olmadığından, doğrudan bir araç çubuğuna yerleştirilemezler.

Bunun yerine, araç çubuğuna düğmeler yerleştirir ve düğmelerin üzerine simgeleri ayarlırsınız. JToolBar'ın bir örneği, normal bir kapsayıcı gibidir. Genellikle, BorderLayout kapsayıcısının kuzey, batı veya doğusunda yer alır.

JToolBar sınıfındaki aşağıdaki özellikler genellikle faydalıdır:

- `orientation` araç çubuğundaki öğelerin yatay mı yoksa dikey mi görünüşünü belirtir. Olası değerler `JToolBar.HORIZONTAL` ve `JToolBar.VERTICAL`'dir. Varsayılan değer `JToolBar.HORIZONTAL`'dir.
- `yüzebilir` bir boolean değeri, araç çubuğunun yüzebilir olup olmadığını belirtir. Varsayılan olarak, bir araç çubuğu yüzebilir.

Liste 38.3, Şekil 38.5'te gösterildiği gibi, Yeni, Aç ve Yazdır komutlarını temsil eden simgelerle üç düğme tutan bir JToolBar oluşturmayı gösteren bir örnek vermektedir.





### Şekil 38.5

#### Araç Çubuğu Yeni,

Aç ve Yazdır komutlarını temsil eden simgeleri içerir.

Liste 38.3 ToolBarDemo.java

<kenar notu satır 6: düğmeler>  
<kenar notu satır 13: araç çubuğu>  
<kenar notu satır 27: araç çubuğu ekle>  
<kenar notu satır 29: ana yöntem atlandı>

```
1 javax.swing.*'i içe aktarın;
2 java.aut.*'u içe aktarın;
3
4 genel sınıf ToolBarDemo t extendst JApplet {
5 özel JButton jbtNewt newt JButton(
6 newt ImageIcon(getClass().getResource(t"/image/new.gif"t));
7 özel JButton jbtOpen = t newt JButton(
8 newt ImageIcon(getClass().getResource(t"/image/open.gif"t));
9 özel JButton jbPrint = t newt JButton(
10 newt ImageIcon(getClass().getResource(t"/image/print.gif"t));
11
12 genel Araç Çubuğu Demosu() {
13 JToolBar jToolBar1 = new JToolBar("Benim Araç Çubuğum");
14 jToolBar1.setFloatable(true);
15 jToolBar1.add(jbtNew); n 16 jToolB
ar1.add(jbtOpen); n 17 jToolBar1.add(j
bPrint);
18
19 jbtNew.setToolTipText("Yeni");
20 jbtOpen.setToolTipText("Aç");
21 jbPrint.setToolTipText("Yazdır");
22
23 jbtNew.setBorderPainted(false);
24 jbtOpen.setBorderPainted(false); n 25 jbPrint.s
etBorderPainted(false);
26
27 add(jToolBar1, BorderLayout.NORTH); n 28 }

29 }
```

Bir JToolBar 13. satırda oluşturulmuştur. Araç çubuğu varsayılan olarak BoxLayout ile bir kapsayıcıdır. Yönlendirme özelliğini kullanarak, araç çubuğundaki bileşenlerin nasıl düzenleneceğini belirtebilirsiniz. yatay veya dikey. Varsayılan olarak, yataydır.

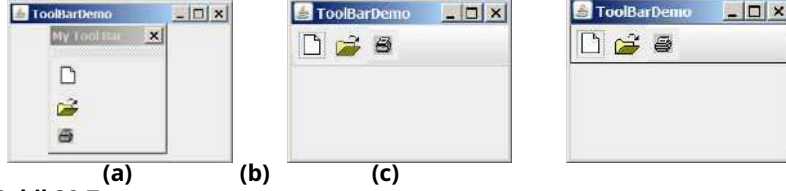
Varsayılan olarak, araç çubuğu taşınabilir ve taşınabilir bir denetleyici öğelerinin önünde görüntülenir. Araç çubuğunu pencerenin farklı konumlarına taşımak için taşınabilir denetleyiciyi sürükleyebilir veya araç çubuğunu ayrı bir pencerede gösterebilirsiniz, Şekil 38.6'da gösterildiği gibi.



Şekil 38.6

*Araç çubuğu düğmeleri taşınabilir.*

Ayrıca, taşınabilir araç çubuğu için bir başlık ayarlayabilirsiniz, Şekil 38.7(a)'de gösterildiği gibi. Bunu yapmak için, `JToolBar(String title)` yapısını kullanarak bir araç çubuğu oluşturun. Eğer taşınabilir `false` olarak ayarlanırsa, taşınabilir denetleyici görüntülenmez, Şekil 38.7(b)'de gösterildiği gibi. Eğer bir kenarlık ayarlarsanız (örneğin, bir çizgi kenarlığı), Şekil 38.7(c)'de gösterildiği gibi, çizgi kenarlığı görüntülenir ve taşınabilir denetleyici görüntülenmez.



**Şekil 38.7**

*Araç çubuğu düğmeleri birçok biçimde özelleştirilebilir.*

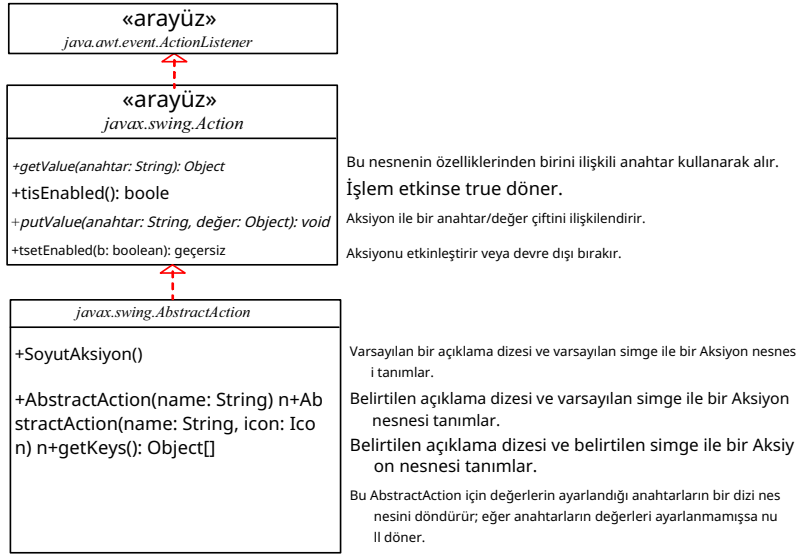
**İPUCU:** Taşınabilir özelliğin düzgün çalışması için aşağıdakileri yapın: (1) bir araç çubuğunu BorderLayout k apsayıcısının bir tarafına yerleştirin ve diğer taraflara e öge eklemeyin; (2) bir araç çubuğuna kenarlık ayarlamayı n. Bir kenarlık ayarlamak ona taşınamaz hale getirir.

### 38.5 Eylemleri İşleme: Action Arayüzü Kullanarak

Sıklıkla menüler ve araç çubukları bazı ortak eylemler içerir. Örneğin, bir dosyayı kaydetmek için Dosya menüsünden Kaydet seçeneğini seçebilir v eya araç çubuğundaki kaydet butonuna tıklayabilirsiniz. Swing, eylemleri iş lemek için eylem nesneleri oluşturmak üzere kullanılacak Action arayüzünü s ağlar. Action nesnelerini kullanarak, ortak eylem işleme merkezi hale getirilebilir ve diğer uygulama kodundan ayrılabilir.

Action arayüzü, Şekil 38.8'de gösterildiği gibi ActionListener'ın bir alt arayüz üdür. Ayrıca, eylemin etkin olup olmadığını kontrol etmek, eylemi etkinleştirm ek ve devre dışı bırakmak, ve ilişkili eylem değerini bir anahtar kullanarak al mak ve ayarlamak için birkaç yöntem tanımlar. Anahtar herhangi bir di ze olabilir, ancak dört anahtar önceden tanımlanmış anlamlara sahiptir:

Anahtar	Açıklama
<code>eylem.adı</code>	Eylem için bir ad
<code>Eylem.SMALL_ICON</code>	Eylem için küçük bir simge
<code>Eylem.SHORT_DESCRIPTION</code>	Eylem için bir ipucu
Uzun Açıklama	Çevrimiçi yardım için bir açıklama



### Şekil 38.8

Action arayüzü, aynı işlevselliğin birden fazla kontrol tarafından erişilmediği durumlarda ActionListener arayüzüne faydalı bir genişletme sağlar. AbstractAction sınıfı, Action için varsayılan bir uygulama sağlar.

AbstractAction, Şekil 38.8'de gösterildiği gibi, Action arayüzünün varsayılan bir uygulamasıdır. actionPerformed metodu hariç, Action arayüzündeki tüm yöntemleri uygular. Ayrıca, getKeys() metodunu tanımlar.

AbstractAction bir soyut sınıf olduğu için, yapıcısını kullanarak bir örneğini oluşturamazsınız. Ancak, AbstractAction'ın somut bir alt sınıfını oluşturabilirsiniz ve actionPerformed metodunu uygulayabilirsiniz. Bu alt sınıf, anonim bir iç sınıf olarak rahatlıkla tanımlanabilir. Örneğin, aşağıdaki kod bir programı sonlandırmak için bir Action nesnesi oluşturur.

```

Action exitAction = new AbstractAction("Çıkış") {
 public void actionPerformed(ActionEvent e) {
 System.exit(1);
 }
};

```

JMenu ve JToolBar gibi belirli kapsayıcılar, bir Action nesnesini nasıl ekleyeceklerini bilir. Bir Action nesnesi böyle bir kapsayıcıya eklendiğinde, kapsayıcı otomatik olarak Action nesnesi için uygun bir bileşen oluşturur ve Action nesnesi ile bir dinleyici kaydeder. İşte bir Action nesnesini bir menüye ve bir araç çubuğuna eklemenin bir örneği:

```

jMenu.add(exitAction);jToolBar.add(exitAction);

```

JButton, JRadioButton ve JCheckBox gibi birkaç Swing bileşeni, Action nesnelerinden örnekler oluşturmak için yapıcılar içerir. Örneğin, bir Action nesnesinde bir JButton oluşturabilirsiniz, şu şekilde:

```
JButton jbt = tnewt JButton(exitAction);
```

Action nesneleri ayrıca mnemonik ve hızlandırıcı tuşlarla ilişkilendirilebilir. Eylemleri bir mnemonik tuşla (örneğin, ALT+E) ilişkilendirmek için aşağıdaki ifadeyi kullanın:

```
çıkışAction.putValue(Action.MNEMONIC_KEY,tnewt Integer(KeyEvent.VK_E));
```

Eylemleri bir hızlandırıcı tuşla (örneğin, CTRL+E) ilişkilendirmek için aşağıdaki ifadeyi kullanın:

```
Tuş Vuruşu çıkışAnahtarı =
KeyStroke.getKeyStroke(KeyEvent.VK_E, KeyEvent.CTRL_MASK); nextitAction.
putValue(Action.ACCELERATOR_KEY, çıkışKey);
```

Liste 38.4, bir panelde üç menü öğesi, Sol, Merkez ve Sağ, üç araç çubuğu düğmesi, Sol, Merkez ve Sağ, ve üç normal düğme, Sol, Merkez ve Sağ oluşturulmasına bir örnek vermektedir. Düğmeleri tutan panel, FlowLayout kullanmaktadır. Sol, merkez ve sağ düğmelerinin eylemleri, FlowLayout'un hizalamasını sırasıyla sola, sağa ve merkeze ayarlamaktadır. Menü öğelerinin, araç çubuğu düğmelerinin ve paneldeki düğmelerin eylemleri, Action arayüzü kullanılarak ortak eylem işleyicileri aracılığıyla işlenebilir.



Şekil 38.9

Sol, Merkez ve Sağ menüde, araç çubuğunda ve normal düğmelerde görünmektedir.

Liste 38.4 ActionInterfaceDemo.java

<kenar notu satırı 11: resim simgesi> n<kenar notu satırı 19: eylem oluştur> n<kenar notu satırı 33: menü>  
<kenar boşluğu notu satırı 44: araç çubuğu> n<margin notu satırı 52: düğme> n<margin notu satırı 64: özel eylem> n<margin notu satırı 67: yapıcı> n<margin notu satırı 72: yapıcı> n<margin notu satırı 82: işleyici> n<margin notu satırı 93: ana yöntem atlandı>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 public class ActionInterfaceDemo t extends JApplet {
6 özel JPanel düğmesi Panel = t newt JPanel();
7 özel FlowLayout flowLayout = yeni FlowLayout();
8
9 genel ActionInterfaceDemo() {
10 // Görüntü simgeleri oluştur
11 ImageIcon leftImageIcon = tnewt ImageIcon(getClass().getResource(
```

```

12 "/image/sola hizalama.PNG"he));
13 ImageIcon centerImageIcon = tnewt ImageIcon(getClass().getResource(
14 "/image/orta hizalama.PNG"he));
15 ImageIcon rightImageIcon = tnewt ImageIcon(getClass().getResource(
16 "/image/sağa hizalama.PNG"he));
17
18 // Eylemler oluştur
19 Eylem leftAction = tnewt MyAction(t"Left"t, leftImageIcon,
20 "Paneldeki düğmeler için sola hizalama"
21 yeni Tamsayı(KeyEvent.VK_L),
22 KeyStroke.getKeyStroke(KeyEvent.VK_L, ActionEvent.CTRL_MASK));
23 Action centerAction = new MyAction("Merkez", centerImageIcon,
24 "Paneldeki düğmeler için merkez hizalama"
25 yeni Tamsayı(KeyEvent.VK_C),
26 KeyStroke.getKeyStroke(KeyEvent.VK_C, ActionEvent.CTRL_MASK));
27 Action rightAction = new MyAction("Sağ", rightImageIcon,
28 "Paneldeki düğmeler için sağa hizalama"
29 yeni Tamsayı(KeyEvent.VK_R),
30 KeyStroke.getKeyStroke(KeyEvent.VK_R, ActionEvent.CTRL_MASK));
31
32 // Menüleri oluştur
33 JMenuBar jMenuBar1 = new JMenuBar();
34 JMenu jmenuAlignment = new JMenu("Yerleşim");
35 setJMenuBar(jMenuBar1);
36 jMenuBar1.add(jmenuAlignment);
37
38 // Menüye eylemler ekle
39 jmenuAlignment.add(leftAction); n 40 jmenuA
40 lignment.add(centerAction); n 41 jmenuAlignment.
41 add(rightAction);
42
43 // Araç çubuğuna eylemler ekle
44 JToolBar jToolBar1 = t newt JToolBar(JToolBar.VERTICAL); n 45 jToolBar1.s
45 etBorder(BorderFactory.createLineBorder(Color.red)); n 46 jToolBar1.add(leftAction);
46
47 jToolBar1.add(centerAction); n 48 jToolBar
48 1.add(rightAction);
49
50 // Düğme paneline düğmeler ekle
51 ButtonPanel.setLayout(flowLayout); n 52 JButton j
52 btLeft = tnewt JButton(leftAction); n 53 JButton jbtCenter = tne
53 wt JButton(centerAction); n 54 JButton jbtRight = tnewt JButton(right
54 Action); n 55 butonuPanel.add(jbtLeft);
55
56 butonuPanel.add(jbtCenter); n 57 buton
57 uPanel.add(jbtRight);
58
59 // Araç çubuğunu doğuya ve paneli merkeze ekle
60 add(jToolBar1, BorderLayout.EAST); n 61 add(butt
61 onPanel, BorderLayout.CENTER); n 62 }
62
63
64 özel sınıf MyAction extends AbstractAction {
65 Dize adi;
66
67 MyAction(Dize adi, Simge simgesi) {
68 supert(isim, simge);
69 thist.name = adi;
70 }
71
72 MyAction(Dize adi, Simge simgesi, Dize açıklaması, Tamsayı animsaticısı, n 73 KeyStrok
73 e hızlandırıcı) {
74 supert(isim, simge);
75 putValue(Action.SHORT_DESCRIPTION, desc); n 76 putValue(
76 Action.MNEMONIC_KEY, animsaticı); n 77 putValue(Action.ACC
77 ELERATOR_KEY, hızlandırıcı);

```

```

78 thist.name = ad;
79 }
80
81 @Geçersiz kıl
82 public void actionPerformed(ActionEvent e) {
83 if (name.equals("Sol"))
84 flowLayout.setAlignment(FlowLayout.LEFT);
85 else if (name.equals("Merkez"))
86 flowLayout.setAlignment(FlowLayout.CENTER);
87 else if (name.equals("Sağ"))
88 flowLayout.setAlignment(FlowLayout.RIGHT);
89
90 butonuPanel.revalidate(); n 91 }
92 } n 93
93 }

```

İç sınıf MyAction, bir ad ve simge ile bir eylem oluşturmak için bir yapıcuya (satırlar 67-70) ve bir ad, simge, açıklama, mnemonik ve hızlandırıcı ile bir eylem oluşturmak için başka bir yapıcuya sahiptir (satırlar 72-79).

Yapıcılar, adı, simgeyi, açıklamayı, mnemonik ve hızlandırıcıyı ilişkilendirmek için putValue yöntemini çağırır. FlowLayout panelinde yeni bir hizalama ayarlamak için actionPerformed yöntemini uygular (satırlar 82-91). revalidate() yöntemi yeni hizalamayı doğrular (satır 90).

MyAction sınıfından üç eylem, leftAction, centerAction ve rightAction yaratıldı (satırlar 19-30). Her eylemin bir adı, simgesi, açıklaması, mnemonik ve hızlandırıcısı vardır. Eylemler, menü öğeleri ve araç çubuğundaki butonlar için ve paneldeki butonlar içindir. Menü ve araç çubuğu, bu nesneleri otomatik olarak eklemeyi bilir (satırlar 39-41, 46-48). Eylemlerden alınan özelliklerle üç normal buton yaratıldı (satırlar 51-54).

### 38.6 JOptionPane Diyaloglar

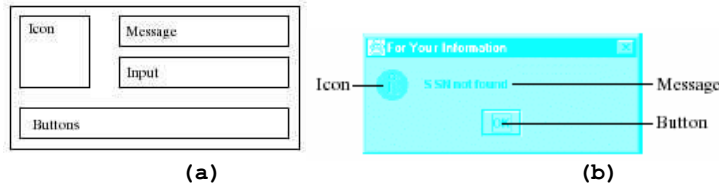
JOptionPane'i kullanarak giriş ve çıkış diyalog kutuları oluşturduunuz. Bu bölüm, JOptionPane ve diğer diyalog kutularına kapsamlı bir giriş sağlar. Bir *diyalog kutusu* genellikle kullanıcıdan ek bilgi almak veya bir olayın gerçekleştiğine dair bildirim sağlamak için geçici bir pencere olarak kullanılır.

Java, standart diyaloglar oluşturmak için kullanılacak JOptionPane sınıfını sağlar. Ayrıca, JDialog sınıfını genişleterek özel diyaloglar da oluşturabilirsiniz.

JOptionPane sınıfı, dört tür standart diyalog oluşturmak için kullanılabilir:

- Mesaj diyalogu bir mesaj gösterir ve kullanıcının Tamam'a tıklamasını bekler.
- Onay diyalogu bir soru gösterir ve Tamam veya İptal gibi onayı ister.
- Giriş diyalogu bir soru gösterir ve kullanıcının metin alanı, açılır kutu veya listeden girişini alır.
- Seçenek diyalogu bir soru gösterir ve kullanıcının bir dizi seçenekten yanıtını alır.

Bu diyaloglar, showXXXDialog statik yöntemleri kullanılarak oluşturulur ve genellikle Şekil 38.10(a)'da gösterildiği gibi görünür.



**Şekil 38.10**

(a) A *JOptionPane* iletişim kutusu bir simge, bir mesaj, bir girdi ve seçenek düğmeleri gösterebilir. (b) Mesaj iletişim kutusu bir mesaj gösterir ve kullanıcının Tamam'a tıklamasını bekler.

Örneğin, aşağıdaki yöntemi kullanarak bir mesaj diyalog kutusu oluşturabilirsiniz, Şekil 38.10(b)'de gösterildiği gibi:

```
JOptionPane.showMessageDialog (null, "SSN bulunamadı",
 "Bilginize", JOptionPane.INFORMATION_MESSAGE);
```

### 38.6.1 Mesaj Diyalogları

Bir *mesaj diyalogu* kutusu, kullanıcıyı uyaran bir mesajı gösterir ve kullanıcıdan diyalogu kapatmak için *OK* butonuna tıklamasını bekler. Mesaj diyalogları oluşturmak için kullanılan yöntemler şunlardır:

```
public static voidt showMessageDialog(Bileşen parentComponent,n Nesne mesaj
)
public static voidt showMessageDialog(Component parentComponent,n Object
 message,
 Dize başlığı,
 int mesaj türü)
public static voidt showMessageDialog(Component parentComponent,n Object
 message,
 Dize başlığı,
 intt messageType, n Si
 mge simgesi)
```

*parentComponent* herhangi bir bileşen veya null olabilir. Mesaj, bir nesnedir, ancak genellikle bir dize kullanılır. Bu iki parametre her zaman belirtilmelidir. Başlık, diyalogun başlık çubuğunda gösterilen bir dizedir ve varsayılan değeri "Mesaj"dır.

*messageType*, aşağıdaki sabitlerden biridir:

```
JOptionPane.ERROR_MESSAGEn
JOptionPane.INFORMATION_MESSAGEn
JOptionPane.PLAIN_MESSAGE
JOptionPane.UYARI_MESAJI
JOptionPane.SORU_MESAJI
```

Varsayılan olarak, *messageType* *JOptionPane.BILGI\_MESAJI*'dir. Her türün, *PLAIN\_MESSAGE* türü hariç, ilişkili bir simgesi vardır, Şekil 38.11'de gösterildiği gibi. Ayrıca, *icon* parametresinde kendi simgenizi de sağlayabilirsiniz.

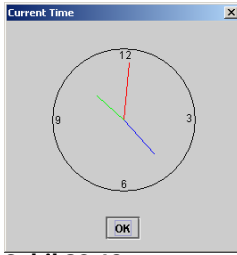


**Şekil 38.11**

*Beş tür mesaj iletişim kutusu vardır.*

Mesaj parametresi bir nesnedir. Eğer bir GUI bileşeni ise, bileşen görüntülenir. Eğer bir non-GUI bileşeni ise, nesnenin dize temsili görüntülenir. Örneğin, aşağıdaki ifade, Şekil 38.12'de gösterildiği gibi, bir mesaj iletişim kutusunda bir saat görüntüler. StillClock, Liste 15.10'da tanımlanmıştır.

```
JOptionPane.showMessageDialog(null, new StillClock(),
 "Mevcut Zaman", JOptionPane.PLAIN_MESSAGE);
```



**Şekil 38.12**

*Bir saat, bir mesaj iletişim kutusunda görüntülenir.*

### 38.6.2 Onay İletişim Kutuları

Bir mesaj iletişim kutusu bir mesaj görüntüler ve kullanıcının iletişim kutusunu kapatmak için *OK* butonuna tıklamasını bekler. Mesaj diyalogu herhangi bir değer döndürmez. Bir onay diyalogu bir soru sorar ve kullanıcının uygun bir butonla yanıt vermesini gerektirir. Onay diyalogu, seçilen butona karşılık gelen bir değer döndürür.

Onay diyalogu oluşturma yöntemleri şunlardır:

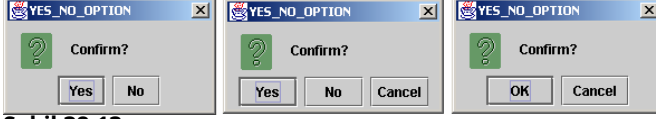
```
public static int showConfirmDialog(Bileşen parentComponent,
 Nesne message)
public static int showConfirmDialog(Bileşen parentComponent,
 Nesne message,
 Dize başlığı, n in
 t optionType)
public static int showConfirmDialog(Bileşen parentComponent,
 Nesne message,
 Dize başlığı,
 intt seçenek türü,
 int mesaj türü)
public static int showConfirmDialog(Bileşen parentComponent,
 Nesne message,
 Dize başlığı,
 int optionType,
 int messageType,
 İkon icon)
```



parentComponent, message, title, icon ve messageType parametreleri, showMessageDialog yöntemindeki ile aynıdır. title için varsayılan değer "Bir Seçenek Seçin" ve messageType için varsayılan değer QUESTION\_MESSAGE'dir. messageType, diyalogda hangi düğmelerin gösterileceğini belirler. Olası değerler şunlardır :

```
JOptionPane.YES_NO_OPTIONnJ
JOptionPane.YES_NO_CANCEL_OPTIONnJ
JOptionPane.OK_CANCEL_OPTION
```

Şekil 38.13, bu seçeneklerle onay diyalogunu göstermektedir.



**Şekil 38.13**

*Onay diyalogu bir soru ve üç tür seçenek düğmesi gösterir ve kullanıcıdan yanıtlar gerektirir.*

showConfirmDialog yöntemi, seçilen seçeneğe karşılık gelen aşağıdaki int\_\_\_ değerlerinden birini döndürür:

```
JOptionPane.EVET_SEÇENEĞİ
JOptionPane.HAYIR_SEÇENEĞİ
JOptionPane.IPTAL_SEÇENEĞİ
JOptionPane.TAMAM_SEÇENEĞİ
JOptionPane.KAPALI_SEÇENEĞİ
```

Bu seçenekler, etkinleştirilen düğmeye karşılık gelir, KAPALI\_SEÇENEĞİ hariç, bu da iletişim kutusunun düğmeler etkinleştirilmeden kapandığını belirtir.

### 38.6.3 Girdi Diyalogları

Bir girdi diyalogu kutusu, kullanıcıdan girdi almak için kullanılır. Girdi, bir metin alanından girilebilir veya bir açılır kutudan ya da bir listeden seçilebilir. Seçilebilir değerler bir dizi içinde belirtilebilir ve bunlardan biri başlangıçta seçili değer olarak atanabilir.

Bir girdi diyalogu oluşturulduğunda, seçilebilir bir değer belirtilmezse, girdi girme için bir metin alanı kullanılır. Yirmiden az seçim değeri belirtilirse, girdi diyalogunda bir açılır kutu görüntülenir. Yirmi veya daha fazla seçim değeri belirtilirse, girdi diyalogunda bir liste kullanılır.

Girdi diyalogları oluşturma yöntemleri aşağıda gösterilmiştir:

```
public static String showInputDialog(Object message) npublic static
ct String showInputDialog(Component parentComponent,n Object message)

public static String showInputDialog(Bileşen parentComponent,n Nesne mesajı,

 Dize başlık,
 int mesaj türü)
public static Object showInputDialog(Bileşen parentComponent,
 Object mesaj,
 int mesajTürü,
 İkon simgesi,
 Nesne[] seçimDeğerleri,
 Nesne başlangıçSeçimDeğeri)
```

Yukarıda listelenen ilk üç yöntem, giriş için bir metin alanı kullanır, Şekil 38.14(a)'da gösterildiği gibi. Yukarıda listelenen son yöntem, bir nesne olarak belirtilen bir başlangıç seçimine ek olarak, seçim değerleri olarak bir Object türünde dizi belirtir. İlk üçü

yöntemler, giriş iletişim kutusundaki metin alanından girilen bir String döndürür. Son yöntem, bir açılır kutudan veya bir listeden seçilen bir Object döndürür. Giriş iletişim kutusu, yirmiden az seçim değeri varsa bir açılır kutu gösterir; yirmi veya daha fazla seçim değeri varsa bir liste gösterir.

seçim değerleri, Şekil 38.14(c)'de gösterildiği gibi.



(a) metin alanı (b) kombinasyon kutusu (c) liste

#### Şekil 38.14

(a) Seçim değerleri belirtilmeden bir giriş iletişim kutusu oluşturulduğunda, giriş iletişim kutusu veri girişi için bir metin alanı gösterir. (b) Ne zaman seçim değerleri ile bir giriş iletişim kutusu oluşturma, giriş iletişim kutusu yirmiden az seçim değeri varsa bir açılır kutu gösterir. (c) Seçim değerleri ile bir giriş iletişim kutusu oluştururken, giriş iletişim kutusu yirmi veya daha fazla seçim değeri varsa bir liste gösterir.

#### NOT

showInputDialog yöntemi optionType parametresine sahip değildir. Giriş iletişim kutusu için butonlar yapılandırılmaz. OK ve İptal butonları her zaman kullanılır.

#### 38.6.4 Seçenek İletişim Kutuları

Bir seçenek iletişim kutusu özel butonlar oluşturmaya olanak tanır. Aşağıdaki yöntem kullanarak bir seçenek iletişim kutusu oluşturabilirsiniz:

```
public static intt showOptionDialog(Component parentComponent, n Nesne m
 esaji,
 Dize başlığı,
 intt optionType, nint
 t messageType, n Sim
 ge simgesi,
 Nesne[] seçenekleri, n
 Nesne başlangıç Değeri)
```

Butonlar options parametresi kullanılarak belirtilir. initialValue parametresi, başlangıç odaklanmasını alacak bir buton belirtmeye olanak tanır. showOptionDialog yöntemi, etkinleştirilen butonu belirten bir int değeri döndürür. Örneğin, Şekil 38.15'te gösterildiği gibi bir seçenek iletişim kutusu oluşturan kod burada:

```
tam değer =
 JOptionPane.showOptionDialog(null, "Bir buton seçin",
 "Seçenek Diyaloğu", JOptionPane.DEFAULT_OPTION,
 JOptionPane.PLAIN_MESSAGE, null,
 new Object[] {"Buton 0", "Buton 1", "Buton 2"}, "Buton 1");
```

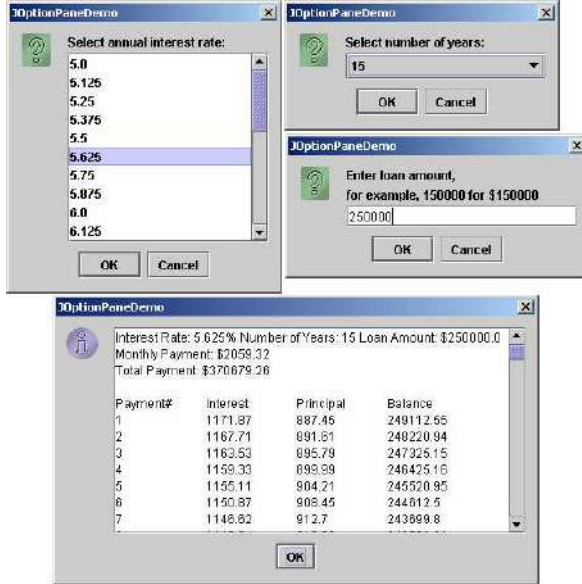


#### Şekil 38.15

Seçenek diyaloğu özel butonları gösterir.

### 38.6.5 Örnek: JOptionPane Diyalogları Oluşturma

Bu bölüm, JOptionPane diyaloglarının kullanımını gösteren bir örnek vermektedir. Program, kullanıcıdan bir girdi diyalogunda yıllık faiz oranını, bir girdi diyalogunda bir açılır kutudan yıl sayısını ve bir girdi diyalogunda kredi miktarını seçmesini ister ve kredi ödeme planını bir mesaj diyalogunda bir JTextArea içinde bir JScrollPane içinde gösterir; bu, Şekil 38.16'da gösterilmiştir.



Şekil 38.16

*Girdi diyalogları, girdi seçimi için bir liste veya bir açılır kutu içerebilir ve mesaj diyalogları,*

*JScrollPane gibi GUI nesneleri içerebilir.*

Programdaki ana adımlar şunlardır (Liste 38.5):

Kullanıcının bir listeden yıllık

faiz oranını seçmesine izin vermek için bir giriş iletişim kutusu gösterin.

Kullanıcının bir açılır kutudan yıl sayısını seçmesine izin vermek için bir giriş iletişim kutusu gösterin.

Kullanıcının kredi

miktarını girmesine izin vermek için bir giriş iletişim kutusu gösterin.

4. Aylık ödemeyi, toplam ödemeyi ve kredi ödeme planını hesaplayın ve sonucu bir metin alanında bir mesaj iletişim diyalog kutusunda görüntüleyin.

Liste 38.5 JOptionPaneDemo.java

<kenar notu satırı 12: giriş iletişim kutusu> n<kenar notu satırı 23: giriş iletişim kutusu> n<kenar notu satırı 29: giriş iletişim kutusu> n<kenar notu satırı 70: mesaj iletişim kutusu>

```
1 javax.swing.*'i içe aktarın;
2
3 genel sınıf JOptionPane Demosu {
```

```

4 public static void main(String args[]) {
5 // Yıllık faiz oranları için bir dizi oluşturun
6 Nesne[] oranListesi = newt Nesne[t25t];
7 int ben = t0t;
8 kale (tdoublet oranı = t 5t; oran <= t8t; oran += t1t.t0t / t8t)
9 oranListesi[i++] = newt Double(orant);
10
11 // Kullanıcıdan yıllık faiz oranını seçmesini isteyin
12 Nesne yıllıkInterstRateObject = JOptionPane.showInputDialog(
13 null, "Yıllık faiz oranını seçin:", "JOptionPaneDemo",
14 JOptionPane.QUESTION_MESSAGE, tnnull, RateList, t nullt);
15 iki kat yıllıkFaiz Oranı =
16 ((Double)yıllıkInterstRateObject).doubleValue();
17
18 // Yıl sayısı için bir dizi oluşturun
19 Nesne[] yılListesi = {newt Tamsayı(t7t),newt Tamsayı(t15t),
20 newt Tamsayı(t30t)};
21
22 // Kullanıcıdan yıl sayısını girmesini isteyin
23 Nesne numberOfYearsObject = JOptionPane.showInputDialog(tnnull,
24 "Yıl sayısını seçin:", "JOptionPaneDemo",
25 JOptionPane.QUESTION_MESSAGE, tnnull, YearList, t nullt);
26 intt numberOfYears = ((Integer)numberOfYearsObject).intValue();
27
28 // Kullanıcıdan kredi miktarını girmesini isteyin
29 String krediAmountString = JOptionPane.showInputDialog(tnnull,
30 "Kredi miktarını girin,örneğin, 150000 için $150000",
31 "JOptionPaneDemo"t,JOptionPane.QUESTION_MESSAGE);
32 doublet krediAmount = Double.parseDouble(loanAmountString);
33
34 // Aylık ödeme ve toplam ödemeyi elde edin
35 Kredi kredisi = newt Kredisi (n
36 yıllıkFaiz Oranı, Yıl Sayısı, Kredi Tutarı);
37 double aylıkÖdeme = loan.getMonthlyPayment();
38 double toplamÖdeme = loan.getTotalPayment();
39
40 // Çıktı dizesini hazırla
41 String çıktı = "Faiz Oranı: " + yıllıkFaizOranı + "%" +
42 " Yıl Sayısı: " + yılSayısı + " Kredi Miktarı: $"
43 + krediMiktarı;
44 çıktı += "Aylık Ödeme: " + "$" + 45 (int)(aylıkÖd
45 eme * 100) / 100.0; 46 çıktı += "Toplam Ödeme: $"
46 +
47 (int)(aylıkÖdeme * 12 * yılSayısı * 100) / 100.0 + "";
48
49 // Aylık faiz oranını al
50 double aylıkFaizOranı = yıllıkFaizOranı / 1200;
51
52 double bakiye = krediMiktarı;
53 double faiz;
54 double anapara;
55
56 // Başlığı göster
57 çıktı += "Ödeme#FaizAnaparaBakiye";
58
59 for (i = 1; i <= yılSayısı * 12; i++) {
60 faiz = (int)(aylıkFaizOranı * bakiye * 100) / 100.0;
61 anapara = (int)((aylıkÖdeme - faiz) * 100) / 100.0;
62 bakiye = (int)((bakiye - anapara) * 100) / 100.0;
63 çıktı += i + " " + faiz + " " + anapara + " " + 64 bakiye + " ";
64
65 }
66
67 // Aylık ödeme ve toplam ödemeyi göster
68 JScrollPane jsp = newt JScrollPane(newt JTextArea(çıkış)); n 69 jsp.setPrefer
redSize(newt java.awt.Dimension(t400t,t 200t));

```

```

70 JOptionPane.showMessageDialog(tnult, jsp,
71 "JOptionPaneDemo", JOptionPane.INFORMATION_MESSAGE, tnult);
72 }
73 }

```

JOptionPane diyalog kutuları *modal*'dir, bu da başka bir pencerenin, bir diyalog kutusu kapatılana kadar erişilemeyeceği anlamına gelir.

Girdi diyalog kutusunu bir metin alanından girdi almak için kullandınız. Bu örnek, girdi diyalog kutularının aynı zamanda bir liste (satırlar 12–14) veya bir açılır kutu (satırlar 23–25) içerebileceğini gösterir. Listenin elemanları nesnelerdir. Bu girdi diyalog kutularından dönen değer Object türündedir. Bir double değeri veya bir int değeri elde etmek için, dönen nesneyi Double veya Integer'a dönüştürmeli, ardından doubleValue veya intValue yöntemini kullanarak double veya int değerini almalısınız (satırlar 15–16 ve

26).

Mesaj iletişim kutusunu bir dizeyi görüntülemek için zaten kullandınız.

Bu örnek, mesaj iletişim kutusunun

GUI nesnelerini de içerebileceğini göstermektedir. Çıktı dizesi bir metin alanında yer almakta, metin alanı bir kaydırılabilir panelin içinde ve kaydırılabilir panel mesaj iletişim kutusuna yerleştirilmiştir (satırlar 68–71).

### 38.7 Özel İletişim Kutuları Oluşturma

Standart JOptionPane iletişim kutuları çoğu durumda yeterlidir.

Bazen özel iletişim kutuları oluşturmanız gerekir. Swing'de, JDialog sınıfı özel iletişim kutuları oluşturmak için genişletilebilir.

JFrame'de olduğu gibi, bileşenler JDialog'un contentPane'ine eklenir. Özel bir iletişim kutusu oluşturmak genellikle iletişim kutusundaki kullanıcı arayüzü bileşenlerini düzenlemeyi, iletişim kutusunu kapatmak için düğmeler eklemeyi ve

düğme eylemlerine yanıt veren dinleyicileri kurmayı içerir.

Standart iletişim kutusu *modal*dir, bu da iletişim kutusu kapatılmadan önce başka bir pencereye erişim sağlanamayacağı anlamına gelir. Ancak, JDialog'dan türetilen özel iletişim kutuları varsayılan olarak modal değildir. Bir iletişim kutusunu modal yapmak için, modal özelliğini true olarak ayarlayın. JDialog'un bir örneğini görüntülemek için, visible özelliğini true olarak ayarlayın.

Renk seçimi için özel bir iletişim kutusu oluşturalım, bu şekilde

Şekil 38.17(a)'de gösterilmiştir. Bu iletişim kutusunu, düğmenin ön plan rengi için renk seçmek amacıyla kullanın,

Şekil 38.17(b)'de gösterildiği gibi. Kullanıcı *Düğme Metni Rengini Değiştir* düğmesine tıkladığında, Renk Seçimi iletişim kutusu görüntülenir.



(a)



(b)

Şekil 38.17

*Özel iletişim kutusu, etiketlin*

*ön planı için bir renk seçmenizi sağlar.*

JDialog'ı genişleterek ColorDialog adında özel bir iletişim kutusu bileşeni oluşturun. Bir rengin kırmızı, yeşil ve mavi bileşenlerini belirtmek için üç kaydırıcı kullanın. Program, Liste 38.6'da verilmiştir.

Liste 38.6 ColorDialog.java

<margin note line 8: renk değeri>

<margin note line 12: kaydırıcılar>

<margin note line 17: düğmeler>

<margin note line 23: yapıcı>

<margin note line 27: yapıcı>

<margin note line 29: arayüz oluştur>

<margin note line 69: dinleyiciler>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.event.*'i içe aktarın;
5
6 genel sınıf ColorDialog t extendst JDialog {
7 // Renk bileşen değerlerini ve seçilen rengi tanımlayın
8 özel intt redValue, greenValue, blueValue;
9 özel Renk renk = t nullt;
10
11 // Kaydırıcıları oluştur
12 özel JSlider jaRed = t newt JSlider(t0t,t 128t);
13 özel JSlider jslGreen = t newt JSlider(t0t,t 128t);
14 özel JSlider jsl Mavi = t newt JSlider(t0t,t 128t);
15
16 // İki buton oluştur
17 özel JButton jbtOK = t newt JButton(t"AND"t);
18 private JButton jbtCancel = new JButton("İptal");
19
20 // Seçilen rengi göstermek için bir panel oluştur
21 özel JPanel jp Seçilen Renk = t newt JPanel();
22
23 publict ColorDialog() {
24 thist(tnull,t tehdit edildi);
25 }
26
27 genel Renk İletişim Kutusu(java.awt.Frame ebeveyni, t boolean modeli) {
28 supert(ebeveyn, modal);
29 setTitle("Renk Seçin");
30
31 // İki butonu OK ve İptal gruplandır
32 JPanel jpButtons = t newt JPanel(); n 33 jpButton
s.add(jbtOK);
34 jpButtons.add(jbtCancel);
35
36 // Etiketleri gruplandır
37 JPanel jpLabels = new JPanel();
38 jpLabels.setLayout(new GridLayout(3,0));
39 jpLabels.add(new JLabel("Kırmızı"));
40 jpLabels.add(new JLabel("Yeşil"));
41 jpLabels.add(new JLabel("Mavi"));
42
43 // Kırmızı, yeşil ve mavi renkleri seçmek için kaydırıcıları gruplandır
44 JPanel jpSliders = t newt JPanel(); n 45 jpSliders.
setLayout(tnewt GridLayout(t3t,t 0t));n 46 jpSliders.add(jslRed);
```

```

47 jpSliders.add(jslGreen); n 48 jpSliders.
add(jslBlue);
49
50 // jpLabels ve jpSliders'ı gruplandır
51 JPanel jpSelectColor = t newt JPanel(); n 52 jpSelectCo
lor.setLayout(tnewt BorderLayout()); n 53 jpSelectColor.setBorder(

54 BorderFactory.createTitledBorder("Renk Seç");
55 jpSelectColor.add(jpLabels, BorderLayout.WEST);
56 jpSelectColor.add(jpSliders, BorderLayout.CENTER);
57
58 // jpSelectColor ve jpSelectedColor grubu
59 JPanel jpColor = t newt JPanel(); n 60 jpColor.
setLayout(tnewt BorderLayout()); n 61 jpColor.add(jpSelect
Color, BorderLayout.SOUTH); n 62 jpColor.add(jpSelectedColor, Borde
rLayout.CENTER);
63
64 // jpButtons ve jpColor'u diyalog kutusuna yerleştir
65 add(jpButtons, BorderLayout.SOUTH); n 66 add(jpC
olor, BorderLayout.CENTER); n 67 paket();

68
69 jbtOK.addActionListener(new ActionListener() {
70 @Geçersiz Kil
71 public void actionPerformed(ActionEvent e) {
72 setVisible(tfalset);n 73 }

74 });
75
76 jbtCancel.addActionListener(tnewt ActionListener() { n 77 @Override

78 public void actionPerformed(ActionEvent e) {
79 renk = null;
80 görünür(false);
81 }
82 });
83
84 jslRed.addChangeListener(tnewt ChangeListener() { n 85 @Overrid
e
86 public void stateChanged(ChangeEvent e) {
87 kırmızıDeğer = jslRed.getValue();
88 renk = new Color(kırmızıDeğer, yeşilDeğer, maviDeğer);
89 jpSelectedColor.setBackground(renk);
90}n91});

92
93 jslGreen.addChangeListener(new ChangeListener() {
94 @Geçersiz Kil
95 public void stateChanged(ChangeEvent e) {
96 yeşilDeğer = jslGreen.getValue();
97 renk = new Color(kırmızıDeğer, yeşilDeğer, maviDeğer);
98 jpSelectedColor.setBackground(renk);
99} n100 });

101
102 jslBlue.addChangeListener(tnewt ChangeListener() { n103 @Overrid
e
104 public void stateChanged(ChangeEvent e) {
105 blueValue = jslBlue.getValue();

```

```

106 renk = t yeni Renk(kırmızıDeğer, yeşilDeğer, maviDeğer); n107 jpSelectedCol
or.setBackground(renk);
108}n109}); n1
10 }

111
112 @Geçersiz kıl
113 public Boyut getPreferredSize() {
114 newt java.awt.Dimension(t200t,t 200t);
115 }
116
117 /** Renk döndür */
118 genel Renk getColor() {
119 rengi döndür;
120 }
121 }

```

Renk diyalogunu kullanarak, Liste 38.7'deki düğmenin ön plan rengi için rengi seçmek üzere bir test sınıfı oluşturun.

Liste 38.7 TestColorDialog.java

<kenar notu satırı 12: dinleyici> n<kenar notu satırı 22: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2 java.awt.*'u içe aktarın;
3 java.awt.event.*'i içe aktarın;
4
5 genel sınıf Testi qColorDialog t Uygulamayı genişletir {
6 özel ColorDialog colorDialog1 = t newt ColorDialog();
7 private JButton jbtChangeColor = new JButton("Renk seç");
8
9 genel TestColor İletişim Kutusu() {
10 setLayout(new java.awt.FlowLayout());
11 jbtChangeColor.setText("Değiştir Buton Metni Rengi");
12 jbtChangeColor.addActionListener(new ActionListener() {
13 @Override
14 public void actionPerformed(ActionEvent e) {
15 colorDialog1.setVisible(ttrue);
16
17 if (colorDialog1.getColor() != t null)
18 jbtChangeColor.setForeground(colorDialog1.getColor()); n 19 }
19
20 });
21 ekle(jbtChangeColor); n 22 }

yirmiüç }

```

Özel iletişim kutusu, kullanıcının kaydırıcıları kullanarak rengini seçmesine olanak tanır. Seçilen renk, color değişkeninde saklanır. Kullanıcı *İptal* butonuna tıkladığında, renk null olur, bu da hiçbir seçim yapılmadığı anlamına gelir.

İletişim kutusu, kullanıcı *Değiştir Buton Metni Rengi* butonuna tıkladığında görüntülenir ve *Tamam* butonuna veya *İptal* butonuna tıkladığında kapatılır.

İPUCU: Gerekli olduğunda iletişim kutusunu modal olarak ayarlamamak yaygın bir hatadır. Bu örnekte,



dialog,

ColorDialog.java (Liste 38.6) dosyasının 24. satırında mod olarak ayarlanmıştır. Eğer dialog

mod değilse, *Change*

*Button Text Color* düğme işleyicisindeki tüm ifadeler, dialog kutusundan renk seçilmeden önce yürütülür.

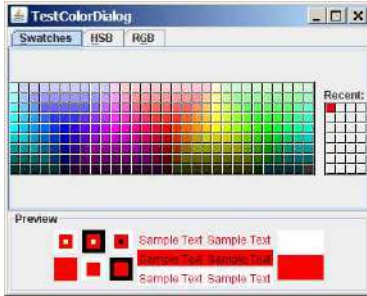
### 38.8t) Renk Seçici

Önceki örnekte, java.awt.Dialog'ın (üst düzey ağır bileşen) bir alt sınıfı olan JDialog'ın bir alt sınıfı olarak bir renk iletişim kutusu oluşturduunuz. Bu nedenle, bir bileşen olarak bir kapsayıcıya eklenemez. Renk iletişim kutuları, GUI programla mada yaygın olarak kullanılır. Swing, javax.swing.JColorChooser adında kullanışlı ve çok yönlü bir renk iletişim kutusu sağlar. JColorChooser, JComponent'ten miras alınan hafif bir bileşendir. Herhangi bir kapsayıcıya eklenebilir. Örn eğin, aşağıdaki kod, Şekil 38.18'de gösterildiği gibi bir JColorChooser'ı bir applet i çine yerleştirir.

JColorChooser'ı bir applet içinde yerleştirir, Şekil 38.18'de gösterildiği gibi.

### <kenar notu satırı 3: JColorChooser'ı oluştur>

```
kamu sınıfı JColorChooser Demosu uzanır javax.swing.JApplet {
 halk JColorChooser Demosu() {
 susuzluk.ads(yeni javax.swing.JColorChooser());
 }
}
```



Şekil 38.18

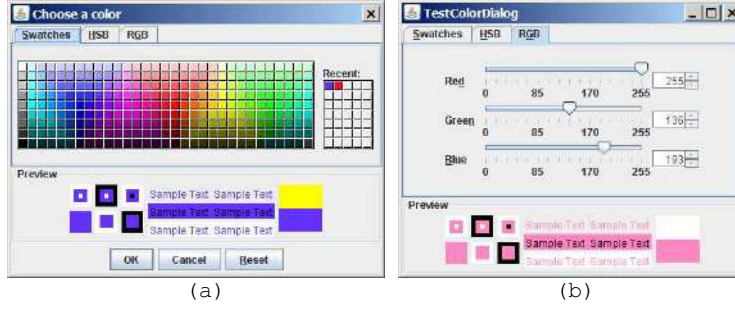
Bir JColorChooser örneği bir applet içinde görüntülenir; (b)

Sıklıkla bir JColorChooser örneği, JColorChooser'ın statik showDialog yöntemi ku llanılarak bir iletişim kutusunda görüntülenir:

```
public static Color showDialog(Component parentComponent, String ti
tle,
Başlangıç rengiRenk)
```

Örneğin, aşağıdaki kod bir JColorChooser'ı görüntüler, Şekil 38.18'de g österi ld iğ i gibi.

```
Color color = JColorChooser.showDialog(this, "Bir renk seçin",
Color.YELLOW);
```



**Şekil 38.19**

*JColorChooser* örneği, OK,

*İptal* ve Sıfırla düğmeleri ile bir iletişim kutusunda görüntülenir.

`showDialog` yöntemi, bir `JColorChooser` nesnesini tutmak için üç düğme, OK, *İptal* ve Sıfırla ile bir `JDialog` örneği oluşturur; bu, Şekil 38.19(a)'da gösterilmiştir. Yöntem, modal bir iletişim kutusu görüntüler.

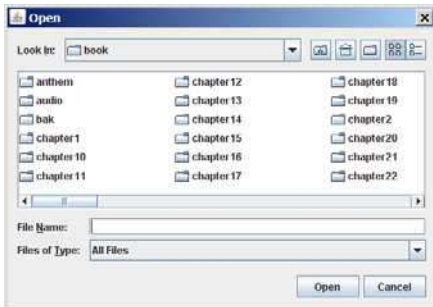
Kullanıcı OK düğmesine tıkladığında, yöntem iletişim kutusunu kapatır ve seçilen rengi döndürür. Kullanıcı *İptal* düğmesine tıklarsa veya iletişim kutusunu kapatırsa, yöntem iletişim kutusunu kapatır ve null döndürür.

`JColorChooser` bir sekmeli panel ve bir renk özikleme panelinden oluşur. Sekmeli panel, Şekil 38.19(b)'da gösterildiği gibi, renkleri Swatches, HSB ve RGB kullanarak seçmek için üç sekmeye sahiptir. Özikleme paneli, seçilen rengin etkisini gösterir.

NOT: `JColorChooser` çok esnektir. Sekmeli paneli veya renk özikleme panelini özel bileşenlerle değiştirmeye izin verir. Varsayılan sekmeli panel ve renk özikleme paneli yeterlidir. Özel bileşenleri nadiren kullanmanız gerekir.

### 38.9 JFileChooser

`javax.swing.JFileChooser` sınıfı, kullanıcının dosya sisteminde gezinip dosyaları yüklemek veya kaydetmek için seçebileceği bir iletişim kutusu görüntüler; bu, Şekil 38.20'de gösterilmiştir.



**Şekil 38.20**

*Swing JFileChooser*, dosyaları ve dizinleri gösterir ve kullanıcının dosya sisteminde görsel olarak gezinmesini sağlar.

`JColorChooser` gibi, `JFileChooser`, `JComponent`'tan miras alınan hafif bir bileşendir. Herhangi bir kapsayıcıya eklenebilir eğer

istenilen, ancak genellikle bir JFileChooser örneği oluşturursunuz ve \_\_\_\_\_  
dışarıda görüntülersiniz.

JFileChooser, JComponent'in bir alt sınıfıdır. Bir dosya diyalog kutusu oluşturmanın birkaç yolu vardır. En basiti, kullanmaktır  
JFileChooser'ın argüman almayan yapıcı metodu.

Dosya diyalog kutusu iki türde görünebilir: aç ve kaydet.  
*açık tür* bir dosyayı açmak için, *kaydet türü* ise  
bir dosyayı saklamak içindir. Açık dosya diyalogu oluşturmak için aşağıdaki yöntemi kullanın:

```
public int showOpenDialog(Bileşen parent)
```

Bu yöntem, bir dosya açmak için bir  
JFileChooser örneği içeren bir iletişim kutusu oluşturur. Yöntem, \_\_\_\_\_  
APPROVE\_OPTION veya CANCEL\_OPTION olan bir int değeri döndürür; bu,  
Aç butonunun mu yoksa İptal butonunun mu tıklandığını gösterir.  
Aynı şekilde, dosyaları kaydetmek için bir iletişim kutusu oluşturmak için aşağıdaki yöntemi kullanabilirsiniz:

```
public int showSaveDialog(Bileşen parent)
```

showOpenDialog veya showSaveDialog ile oluşturulan dosya diyalog kutusu  
modal'dır. JFileChooser sınıfı, JComponent'ten miras alınan özelliklere sahiptir.  
Ayrıca aşağıdaki yararlı özelliklere de sahiptir:

- **dialogType** bu iletişim kutusunun türünü belirtir. Kullanıcının bir dosya açmak için kullanabileceği bir dosya seçici açmak istediğinizde OPEN\_DIALOG kullanın. Benzer şekilde, kullanıcının bir dosya seçmesine izin vermek için SAVE\_DIALOG kullanın.
- **dialogTitle** iletişim kutusunun başlık çubuğunda görüntülenen dizedir.
- **currentDirectory** dosyanın mevcut dizinidir. Bu özelliğin türü java.io.File'dir. Mevcut kullanılacak dizin, setCurrentDirectory(new File(".")) kullanın.
- **seçilenDosya** seçtiğiniz dosyadır. getSelectedFile() yöntemini kullanarak diyalog kutusundan seçilen dosyayı döndürebilirsiniz. Bu özelliğin türü java.io.File'dir. Kullanmayı beklediğiniz bir görecel dosya adı varsa, kullanın setSelectedFile(new File(dosyaAdı)).
- **multiSelectionEnabled**, birden fazla dosyanın seçilip seçilemeyeceğini belirten bir boolean değeridir. Varsayılan olarak, false'dur.
- **selectedFiles** çoklu seçim iznine sahip dosya seçici ayarlandığında seçilen dosyaların listesidir. Bu özelliğin türü File[].

Swing menüleri, araç çubuğu, dosya seçici ve renk seçici kullanan basit bir metin düzenleyici örneği oluşturalım. Bu, kullanıcıya metin dosyalarını açma ve kaydetme, metni temizleme ve metnin rengini ve yazı tipini değiştirme imkanı sağlar. Şekil 38.21'de gösterildiği gibi. Liste 38.8 programı göstermektedir.



**Şekil 38.21**

*Editör, dosya menüsünden veya araç çubuğundan metin dosyalarını açmanıza ve kaydetmenize, ayrıca Düzenle menüsünden metnin rengini ve yazı tipini değiştirmenize olanak tanır.*

#### Listeleme 38.8 TextEditor.java

<margin note line 33: kullanıcı arayüzü oluşturun>

<margin note line 88: renk seçici>

<margin note line 100: renk seçici>

<margin note line 112: dosya seçici>

<margin note line 119: dosya seçici>

<margin note line 182: ana yöntem atlandı>

```

1 java.io.*'yu içe aktarın;
2 java.awt.*'u içe aktarın;
3 java.awt.event.*'i içe aktarın;
4 javax.swing.*'i içe aktarın;
5
6 genel sınıf Metin Düzenleyicisi t Applet'i genişletir {
7 // Resim simgelerini tanımla ve oluştur
8 özel Resim Simgesi openImageIcon =
9 new ImageIcon(getClass().getResource("image/open.gif"));
10 özel Resim Simgesi Resim Simgesini kaydet =
11 yeni ImageIcon(getClass().getResource("/image/save.gif"));
12
13 // Menü öğelerini oluştur
14 özel JMenuItem jmiOpen = new JMenuItem("Aç", openImageIcon);
15 özel JMenuItem jmiSave = new JMenuItem("Kaydet", saveImageIcon);
16 özel JMenuItem jmiClear = new JMenuItem("Temizle");
17 özel JMenuItem jmiExit = new JMenuItem("Çıkış");
18 özel JMenuItem jmiForeground = new JMenuItem("Ön Plan");
19 özel JMenuItem jmiBackground = new JMenuItem("Arka Plan");
20
21 // Bir araç çubuğuna yerleştirilecek düğmeleri oluştur
22 özel JButton jbtOpen = new JButton(openImageIcon);
23 özel JButton jbtSave = new JButton(saveImageIcon);
24 özel JLabel lblStatus = new JLabel();
25
26 // Mevcut dizin ile bir JFileChooser oluştur
27 özel JFileChooser jFileChooser1
28 = t new JFileChooser(tnew File(t".t));
29
30 // Bir metin alanı oluştur
31 özel JTextArea jta = new JTextArea();
32
33 genel TextEditor() {
34 // Menüye menü öğeleri ekle
35 JMenu jMenu1 = new JMenu("Dosya");
36 jMenu1.add(jmiOpen);
37 jMenu1.add(jmiSave); n 38 jMen
38 u1.add(jmiClear); n 39 jMenu1.addS
39 eparator(); n 40 jMenu1.add(jmiExit);
40
41

```

```

42 // Menüye menü öğeleri ekle
43 JMenu jMenu2 = new JMenu("Düzenle");
44 jMenu2.add(jmiForeground);
45 jMenu2.add(jmiBackground);
46
47 // Menüleri menü çubuğuna ekle
48 JMenuBar jMenuBar1 = yeni JMenuBar(); n 49 jMenu
Bar1.add(jMenu1);
50 jMenuBar1.add(jMenu2);
51
52 // Menü çubuğunu ayarla
53 setJMenuBar(jMenuBar1);
54
55 // Araç çubuğu oluştur
56 JToolBar jToolBar1 = yeni JToolBar();
57 jToolBar1.add(jbtOpen);
58 jToolBar1.add(jbtSave);
59
60 jmiOpen.addActionListener(tnew ActionListener() { n 61 @Override
62 public void actionPerformed(ActionEvent e) {
63 açık(); n 64 }
64
65 });
66
67 jmiSave.addActionListener(tnew ActionListener() { n 68 @Override
68
69 genel voidt actionPerformed(ActionEvent evt) {
70 kaydet(); n 71 }
71
72 });
73
74 jmiClear.addActionListener(tnew ActionListener() { n 75 @Override
75
76 genel voidt actionPerformed(ActionEvent evt) {
77 jta.setText(tnull); n 78 }
78
79 });
80
81 jmiExit.addActionListener(tnew ActionListener() { n 82 @Override
82
83 genel voidt actionPerformed(ActionEvent evt) {
84 System.exit(t0t);n 85 }
85
86 });
87
88 jmiForeground.addActionListener(tnew ActionListener() { n 89 @Override
89
90 genel voidt actionPerformed(ActionEvent evt) {
91 Seçilen renkRenk =
92 JColorChooser.showDialog(null, "Ön Plan Rengini Seçin",
93 jta.getForeground());
94
95 ift (selectedColor != t nullt)
96 jta.setForeground(selectedColor); n 97 }
97
98 });
99
100 jmiBackground.addActionListener(tnew ActionListener() { n101 @Override
101
102 genel voidt actionPerformed(ActionEvent evt) {
103 Seçilen renk =
104 JColorChooser.showDialog(null, "Arka Plan Rengini Seç",
105 jta.getForeground());
106
107 ift (selectedColor != t nullt)

```

```

108 jta.setBackground(selectedColor); n109 }

110 });
111
112 jbtOpen.addActionListener(tnewt ActionListener() { n113 @Override
113 genel voidt actionPerformed(ActionEvent evt) {
114 115 açık(); n116 }

117 });
118
119 jbtSave.addActionListener(tnewt ActionListener() { n120 @Override
120 genel voidt actionPerformed(ActionEvent evt) {
121 122 kaydet(); n123 }

124 });
125
126 add(jToolBar1, BorderLayout.NORTH); n127 add(jlb
lStatus, BorderLayout.SOUTH); n128 add(tnewt JScrollPa
ne(jta), BorderLayout.CENTER); n129 }

130
131 /** Dosyayı aç */
132 özel voidt açık() {
133 134 ift (jFileChooser1.showOpenDialog(tthist) ==
135 JFileChooser.APPROVE_OPTION) n135 open(j
FileChooser1.getSelectedFile()); n136 }

137
138 /** Belirtilen File örneği ile dosyayı aç */
139 özel voidt açık (Dosya dosyası) {
140 denemek {
141 // Belirtilen dosyadan oku ve jta'ya kaydet
142 BufferedInputStream in = t newt BufferedInputStream(
newt FileInputStream(dosya));
143 144 byte[] b = t yeni byte[] in.available();
145 inçokuma(b, t0t, b.uzunluk); n146 jta.ap
pend(tnewt String(b, t 0t, b.length)); n147 in.close();

148
149 // Açık dosya işleminin durumunu jlbStatus'ta göster
150 jlbStatus.setText (file.getName() + " Açıldı");
151 }
152 yakalamak (IOException örneği) {
153 jlbStatus.setText ("Hata açılıyor " + file.getName());
154 }
155 }

156
157 /** Dosyayı kaydet */
158 özel voidt save() {
159 160 ift (jFileChooser1.showSaveDialog(tthist) ==
161 JFileChooser.APPROVE_OPTION) { n161 save(jFil
eChooser1.getSelectedFile()); n162 }

163 }

164
165 /** Belirtilen File örneği ile dosyayı kaydet */
166 özel voidt save(Dosya dosyası) {
167 denemek {
168 // jta'daki metni belirtilen dosyaya yaz
169 BufferedOutputStream çıkışı = t newt BufferedOutputStream(
newt FileOutputStream(dosya));
170 171 byte[] b = (jta.getText()).getBytes();
172 çıkış.yaz(b, t0t, b.uzunluk); n173 out.close(
);

```

```

174 // jlblStatus'ta dosya kaydetme işleminin durumunu göster
175 jlblStatus.setText(file.getName() + " Kaydedildi");
176 }
177 }
178 yakalamak (IOException örneği) {
179 jlblStatus.setText("Hata kaydediliyor " + file.getName());
180 }
181 } n 18
2}

```

Program, Dosya ve Düzen menülerini oluşturur (satırlar 34-45). Dosya menüsü, bir dosya yüklemek için Aç komutunu, bir dosyayı kaydetmek için Kaydet komutunu, metin düzenleyiciyi temizlemek için Temizle komutunu ve programı so nlandırmak için Çıkış komutunu içerir. Düzen menüsü, metindeki ön plan rengi ve arka plan rengini ayarlamak için Ön Plan Rengi ve Arka Plan Rengi menü komutlarını içerir. Aç ve Kaydet menü komutlarına, satırlar 56-58'de oluşturulan araç çubuğundan da erişilebilir. Aç ve Kaydet işlemlerinin durumu, satır 24'te ol uşturulan durum etiketinde görüntülenir.

JFileChooser1, dosyaları açmak ve kaydetmek için dosya iletişim kutusunu görüntülemek üzere oluşturulan JFileChooser örneğidir (satırlar 27-28). new File( ".") kullanılarak mevcut dizin, sınıfın saklandığı dizin olarak ayarlanır.

Aç yöntemi, kullanıcı Aç menü komutuna veya Aç araç çubuğu düğmesine tıkl adığında çağrılır (satırlar 62, 108). showOpenDialog yöntemi (satır 12 5), Şekil 38.20'de gösterildiği gibi bir Aç iletişim kutusu görüntüler. Seçilen dosya alındığında, open(file) yöntemi (satır 127) dosyayı metin alanına yüklemek iç in bir BufferedInputStream ile çağrılır.

FileInputStream.

Kayıt yöntemi, kullanıcı Kayıt menü komutuna veya Kayıt araç çubuğu düğme sine tıkladığında çağrılır (satırlar 68, 114). showSaveDialog yöntemi (s atır 151) bir Kayıt iletişim kutusu görüntüler. Seçilen dosyayı aldıktan sonra, save(fi le) yöntemi (satır 153) metin alanındaki içerikleri dosyaya kaydetmek için çağrılır; bu işlem, bir FileOutputStream üzerine sarılmış bir BufferedOutputStream k ullanılarak gerçekleştirilir.

Renk iletişim kutusu, JColorChooser'ın statik showDialog yöntemini kullanarak gö rüntülenir (satırlar 87, 98). Bu nedenle, JFileChooser'ın bir örneğini oluşturmanıza gerek yoktur. showDialog yöntemi, bir renk seçildikten sonra OK düğmesi ne tıklanırsa seçilen rengi döndürür.

## Bölüm Özeti

1. Menüler, seçim yapmayı kolaylaştırır ve pencere uygulamalarında yaygı n olarak kullanılır. Java, menüleri uygulayan beş sınıf sağlar: JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem ve JRadioButtonMenuItem. B u sınıflar AbstractButton'ın alt sınıflarıdır. Düğmelere çok benzerler  
:\_\_\_\_\_
2. JMenuBar, menüleri tutmak için kullanılan üst düzey bir menü bileşenidir. Bir menü, kullanıcının seçebileceği (veya açıp kapatabileceği) menü öğelerinden oluşur. Bir menü ö ğesi, JMenuItem, JCheckBoxMenuItem veya JRadioButtonMenuItem'in bir örneği olabilir. Menü öğeleri simgeler, klavye mnemonikleri ve klavye hızlandırıcıları ile ili ş kilendirilebilir. Menü öğeleri, ayırıcılar kullanılarak ayrılabilir.

3. *Bir açılır menü, aynı zamanda bir bağlam menüsü* olarak da bilinir, normal bir menüye benzer, ancak bir menü çubuğuna sahip değildir ve ekranda herhangi bir yerde yüzebilir. Bir açılır menü oluşturmak, normal bir menü oluşturmakla benzerdir. Öncelikle, JPopupMenu'nin bir örneği ni oluşturursunuz, ardından açılır menüye JMenuItem, JCheckBoxMenuItem, JRadioButtonMenuItem ve ayırıcılar ekleyebilirsiniz.
4. Geleneksel olarak, bir açılır menüyü bir GUI bileşenine işaret ederek ve belirli bir fare düğmesine tıklayarak gösterirsiniz, bu "popup tetikleyici" olarak adlandırılır. Açılır menü tetikleyicileri sistem bağımlıdır. Windows'ta, açılır menü sağ fare düğmesi bırakıldığında görüntülenir. Motif'te, açılır menü üçüncü fare düğmesine basıldığında ve basılı tutulduğunda görüntülenir.
5. Swing, araç çubuğu bileşenlerini tutmak için JToolBar sınıfını sağlar. JToolBar, bileşenleri yönetmek için BorderLayout kullanır. Bileşenler genellikle simgeler olarak görünür. Simgeler bileşen olmadığından, doğrudan bir araç çubuğuna yerleştirilemezler. Bunun yerine, düğmeleri araç çubuğuna yerleştirir ve düğmelerin üzerine simgeleri ayarlırsınız. JToolBar'ın bir örneği, normal bir kapsayıcı gibidir. Genellikle, BorderLayout kapsayıcısının kuzey, batı veya doğusunda yer alır.
6. Swing, eylemleri işlemek için eylem nesneleri oluşturmak üzere kullanılabilecek Action arayüzünü sağlar. Action nesnelerini kullanarak, menü öğeleri için yaygın eylem işleme yapılabilir ve araç çubuğu düğmeleri merkezi hale getirilebilir ve diğer uygulama kodundan ayrılabilir.
7. JOptionPane sınıfı, mesaj diyalogları, onay diyalogları, giriş diyalogları ve seçenek diyalogları oluşturmak için statik yöntemler içerir. Ayrıca, JDialog sınıfını genişleterek özel diyaloglar da oluşturabilirsiniz.
8. Swing, javax.swing.JColorChooser adında kullanışlı ve çok yönlü bir renk diyalogu sağlar. JOptionPane gibi, JColorChooser JComponent'ten miras alınan hafif bir bileşendir. Herhangi bir kapsayıcıya eklenir.
9. Swing, kullanıcıların dosya sisteminde gezinip dosyaları yüklemek veya kaydetmek için seçebileceği bir diyalog kutusu görüntüleyen javax.swing.JFileChooser sınıfını sağlar.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

### Gözden Geçirme Soruları

#### Bölüm 38.2

38.1 Menü çubuğu nasıl oluşturulur?

38.2 Alt menü nasıl oluşturulur? Bir onay kutusu menü öğesi nasıl oluşturulur? Bir radyo düğmesi menü öğesi nasıl oluşturulur?

38.3 Bir menüde ayırıcı nasıl eklenir?



38.4 Bir menü öğesinde bir simge ve metin nasıl ayarlanır? Bir menü öğesinde klavye mnemonikleri ve hızlandırıcılar nasıl ilişkilendirilir?

### Bölüm 38.3

38.5 Bir açılır menü nasıl oluşturursunuz? Bir açılır menüyü nasıl gösterirsiniz?

38.6 Bir açılır tetikleyiciyi tanımlayın.

### Bölüm 38.4

38.7 JToolBar'da kullanılan yerleşim yöneticisi nedir? \_\_\_\_\_

yerleşim yöneticisini değiştirebilir misiniz?

38.8 JToolBar'a butonları nasıl eklersiniz? JToolBar'ı \_\_\_\_\_

bir çerçeveye veya applete nasıl eklersiniz?

### Bölüm 38.5

38.9 Action arayüzü ne için kullanılır?

38.10 JToolBar, JMenu, JButton, \_\_\_\_\_

JRadioButton ve JCheckBox'a bir Action nesnesi nasıl eklersiniz?

### Bölüm 38.6

38.11 JOptionPane sınıfı kullanılarak oluşturulan standart diyalog kutularını tanımlayın. \_\_\_\_\_

38.12 Bir mesaj diyalogu nasıl oluşturursunuz? Mesaj türleri nelerdir?

Mesaj diyalogundaki buton nedir?

38.13 Bir onay diyalogu nasıl oluşturursunuz? Buton seçenek türleri nelerdir?

38.14 Girdi almak için bir metin alanı ile bir girdi diyalogu nasıl oluşturursunuz? Değerler i girdi olarak seçmek için bir kombinasyon kutusu diyalogu nasıl oluşturursunuz? Değerleri girdi olarak seçmek için bir liste diyalogu nasıl oluşturursunuz?

### Bölümler 38.7-38.10

38.15 JDialog örneğini nasıl gösterirsiniz? JOptionPane'deki statik yöntemler kullanılarak oluşturulan standart bir diyalog kutusu modal mı? JDialog örneği modal mı?

38.16 JColorChooser örneğini nasıl görüntülersiniz? JColorChooser örneği modal mı? Seçilen rengi nasıl elde edersiniz?

38.17 JFileChooser örneğini nasıl görüntülersiniz? JFileChooser örneği modal mı? Seçilen dosyayı nasıl elde edersiniz? getSelectedFile() ve getSelectedDirectory() için dönüş tipi nedir? Mevcut dizini varsayılan dizin olarak nasıl ayarlarsınız? \_\_\_\_\_

JFileChooser diyalog penceresi?

## Programlama Alıştırıcıları

### Bölümler 38.2-38.3

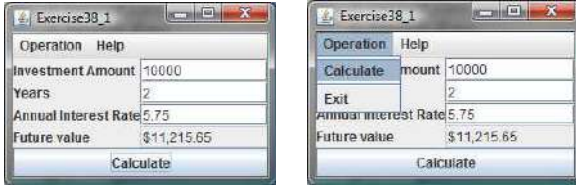
38.1 \*

(Bir yatırım değer hesaplayıcı oluşturun) Belirli bir faiz oranında ve belirli bir yıl sayısı için bir yatırımın gelecekteki değerini hesaplaya n bir program yazın. Hesaplama formülü şu şekildedir:

$$\text{gelecekDeğer} = \text{yatırımMiktarı} \times (1 + \text{aylıkFaizOranı})^{\text{years} \times 12}$$

Faiz oranı, yatırım miktarı ve yıl sayısı için metin alanları kullanın. Kullanıcı Hesapla butonuna tıkladığında veya İşlem menüsünden Hesapla seçeneğini seçtiğinde gelecekteki miktarı bir metin alanında gösterin.

menüsü (bkz. Şekil 38.22). Kullanıcı Yardım menüsünden H hakkında menü öğesine tıkladığında bir mesaj diyalog kutusu gösterilir.



**Şekil 38.22**

Kullanıcı gelecekteki değeri hesaplamak için yatırım miktarını, yıl sayısını ve faiz oranını girer.

38.2\*

(Açılır menüler kullanın) Liste 38.1, MenuDemo.java dosyasını değiştirin ve Şekil 38.23'te gösterildiği gibi İşlemler ve Çıkış menülerini içeren bir açılır menü oluşturun. Açılır menü, etiketlerin ve metin alanlarının bulunduğu panele sağ tıklayıldığında görüntülenir.



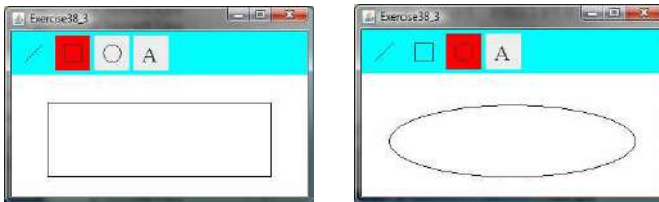
**Şekil 38.23**

Açılır menü, aritmetik işlemleri gerçekleştirmek için komutları içerir.

### Bölümler 38.4-38.5

38.3\*\*

(Bir boyama aracı) Bir boyama aracını taklit eden bir program yazın. Programınız, kullanıcının seçenekleri seçmesine ve seçilen seçeneklere göre şekiller çizmesine veya klavyeden karakterler almasına olanak tanımalıdır (bkz. Şekil 38.24). Seçenekler bir araç çubuğunda görüntülenir. Bir çizgi çizmek için kullanıcı önce araç çubuğundaki çizgi simgesine tıklar ve ardından Microsoft Paint t kullanarak çiziceği gibi fareyi kullanarak bir çizgi çizer.



**Şekil 38.24**

Bu alıştırma, çizgiler, dikdörtgenler, ovalar ve karakterler çizmenizi sağlayan bir prototip çizim aracı üretir.

38.4\*

(*Eylemleri kullanın*) Bir ImageViewer'da görüntülenecek bayrakları seçmek için kullanılacak menü öğeleri ve araç çubuğu düğmeleri içeren bir program yazın. Şekil 38.25'te gösterildiği gibi. Eylemler için işleme merkezi hale getirmek için Action arayüzünü kullanın.



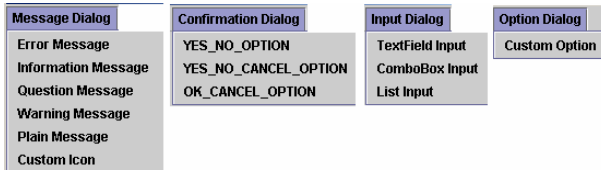
Şekil 38.25

Seçilen görüntüleri ImageViewer'da görüntülemek için menü öğeleri ve araç düğmeleri kullanılır.

### Bölümler 38.6-38.10

38.5\*

(*OptionPane'i gösterin*) Şekil 38.26'da gösterildiği gibi, tüm türlerde seçenek panelleri oluşturulan bir program yazın. Her menü öğesi, bir diyalog kutusu görüntüleme için statik showXXXDialog yöntemini çağırır.



Şekil 38.26

Bir menü öğesine tıklayarak bir diyalog kutusu görüntüleyebilirsiniz.

38.6\*

(*Özel diyalog oluşturun*) Şekil 38.27(a)'de gösterildiği gibi, kullanıcı bilgilerini toplamak için özel bir diyalog kutusu oluşturan bir program yazın.



(a)



(b)

Şekil 38.27

(a) Özel iletişim kutusu, kullanıcının kullanıcı adı ve şifre girmesini ister. (b) Program, kullanıcının bir dosyayı dosya açma iletişim kutusundan seçerek görüntülemesine olanak tanır.

38.7\*

(*JFileChooser kullanın*) Kullanıcının bir dosyayı

dosya açma iletişim kutusundan seçmesine olanak tanıyan bir program yazın.

Dosya açma iletişim kutusu, Gözet

butonuna tıklandığında görüntülenir, Şekil 38.27(b)'de gösterildiği gibi.

Dosya metin alanında görüntülenir ve

dosya adı, dosya açma iletişim kutusunda  
OK butonuna tıklandığında metin alanında görüntülenir.  
Dosya adını metin  
alanına da girebilir ve dosyayı

metin alanında görüntülemek için Enter tuşuna basabilirsiniz.

38.8\*

(Bir ses dosyası seçin) Dosya iletişim kutusunu kullanarak bir ses dosyası seçen bir program yazın ve üç buton kullanın: *Oynat*, *Döngü* ve *Durdur*, sesi kontrol etmek için, Şekil 38.28'de gösterildiği gibi. *Oynat* butonuna tıkladığınızda, ses dosyası bir kez çalınır. Eğer *Döngü* butonuna tıklarsanız, ses dosyası tekrar tekrar çalmaya devam eder. Durdur butonuna tıkladığınızda, çalma durur. Seçilen ses dosyaları, alıştırma dizininde **antheme** adlı klasörde saklanır. Alıştırma dizini, bu

alıştırma için sınıf dosyasını içerir.

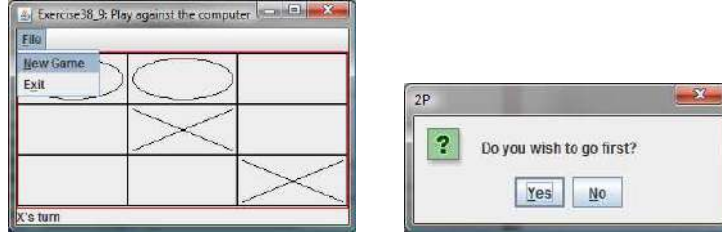


Şekil 38.28

Program, bir iletişim kutusundan bir ses dosyası seçmenize ve butonları kullanarak sesi oynatmanıza, tekrar çalmanıza veya durdurmanıza olanak tanır.

38.9\*\*

(Bilgisayarla TicTacToe oyna) Oyun §18.9, "Vaka Çalışması: TicTacToe," iki oyuncuyu kolaylaştırır. Bir oyuncunun bilgisayara karşı oynamasını sağlayan yeni bir oyun yazın. Şekil 38.29'da gösterildiği gibi, Yeni Oyun ve Çıkış adlı iki öğe ile bir Dosya menüsü ekleyin. Yeni Oyun'a tıkladığınızda, bir iletişim kutusu görüntülenir. Bu iletişim kutusundan, bilgisayarın önce gitmesine izin verip vermemeyeğinize karar verebilirsiniz.



**Şekil 38.29**

*Yeni TicTacToe oyunu, bilgisayara karşı oynamanızı sağlar.*

**\*\*\*Bu bir bonus Web bölümü**

## **BÖLÜM 39**

### **MVC ve Swing Modelleri**

#### **Hedefler**

- Veri ve mantığı, verilerin sunumundan ayırmak için model-görünüm-denetleyici yaklaşımını kullanmak (§39.2).
- JavaBeans olay modelini kullanarak model-görünüm-denetleyici bileşenlerini uygulamak (§39.2).
- Swing model-görünüm-denetleyici mimarisini açıklamak (§39.4).
- Sonraki ve önceki değerleri kaydırmak için JSpinner kullanmak (§39.5).
- Özel spinner modelleri ve editörleri oluşturmak için (§39.6).
- Bir listede tekil veya çoklu öğeleri seçmek için JList kullanmak (§39.7).
- ListModel ve DefaultListModel kullanarak öğeleri eklemek ve kaldırmak için (§39.8) .
- Varsayılan veya özel hücre render'ı kullanarak liste hücrelerini render etmek için (§39.9) .
- Özel combo kutusu modelleri ve render'ları oluşturmak için (§39.10).

## 39.1 Giriş

Swing kullanıcı arayüzü bileşenleri, MVC mimarisinin varyasyonları kullanılarak uygulanmıştır. Basit Swing bileşenlerini destekleyen modelleri düşünmeden kullandınız, ancak gelişmiş Swing bileşenlerini kullanmak için verileri depolamak, erişmek ve değiştirmek amacıyla modellerini kullanmalısınız.

Bu bölüm, MVC mimarisini ve Swing modellerini tanıtmaktadır. Özellikle, JSpinner, JList ve JComboBox'taki modelleri nasıl kullanacağınızı öğreneceksiniz.

Sonraki bölüm JTable ve JTree'yi tanıttacaktır.

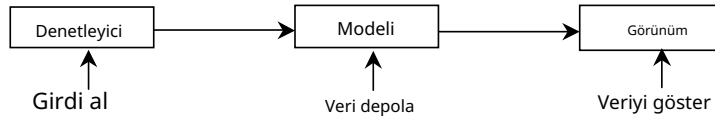
### 39.2 MVC

<kenar notu: model> n<

kenar notu: görünüm>

n<kenar notu: denetleyici>

Model-görünüm-denetleyici (MVC) yaklaşımı, verilerin görsel temsilinden veri depolama ve işleme işlemlerini ayırarak bileşenler geliştirme yöntemidir. Veri depolama ve işleme için kullanılan bileşen, model olarak bilinir ve bileşenin gerçek içeriğini içerir. Veriyi sunan bileşen, görünüm olarak bilinir ve tüm temel bileşen davranışlarını yönetir. Bileşeni düşündüğünüzde aklınıza gelen görünümüdür. Bileşenlerin tümünü görüntüler. Denetleyici, genellikle verileri elde etmekten sorumlu olan bir bileşendir; bu, Şekil 39.1'de gösterilmiştir.



### Şekil 39.1

Denetleyici verileri alır ve bir modelde saklar. Görünüm, modelde saklanan verileri görüntüler.

<kenar notu: MVC faydaları>

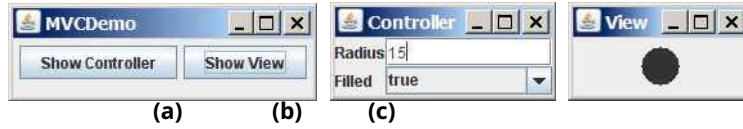
Bir bileşeni model ve görünüm olarak ayırmanın iki ana faydası vardır:

- Bu, verilerin aynı model aracılığıyla paylaşılmasını sağlamak için birden fazla görünümün mümkün olmasını sağlar. Örneğin, öğrenci adlarını saklayan bir model, aynı anda bir açılır kutuda ve bir liste kutusunda görüntülenebilir.
- Bu, karmaşık uygulamalar yazma görevini basitleştirir ve bileşenleri ölçeklenebilir ve bakımını kolay hale getirir. Görünümde değişiklikler yapılabilir, model etkilenmeden ve tersi de geçerlidir.

Bir model veri içerirken, bir görünüm veriyi görünür hale getirir. Bir görünüm bir model ile ilişkilendirildiğinde, model ile senkronize olur. Bu, modelin tüm görünümünün aynı veriyi tutarlı bir şekilde göstermesini sağlar. Tutarlılık ve senkronizasyon sağlamak için model, görünümde kullanılan özelliklerinden herhangi birinde bir değişiklik olduğunda görünümü bilgilendirmelidir. Bir değişiklik bildirimine yanıt olarak, görünüm, özellik değişikliğinden etkilenen görüş alanını yeniden görüntülemekten sorumludur.

Java olay delegasyonu modeli, MVC bileşen geliştirmeyi desteklemek için superior bir mimari sağlar. Model, uygun olay ve olay dinleyici kayıt yöntemleri ile bir kaynak olarak uygulanabilir. Görünüm, bir dinleyici olarak uygulanabilir. Böylece, modelde veriler değiştiğinde, görünüm bilgilendirilecektir. Görünümünden modeli seçmek için, görünümde bir set yöntemi ile modeli bir özellik olarak eklemek yeterlidir.

MVC yaklaşımını kullanarak bileşenlerin geliştirilmesini göstermek için bir örnek kullanalım. Örnek, CircleModel adında bir model, CircleView adında bir görünüm ve CircleControl adında bir kontrolör oluşturur. CircleModel, bir çemberi tanımlayan özellikleri (yarıçap, dolu ve renk) saklar. dolu, bir çemberin dolu olup olmadığını belirten bir boolean değerdir. CircleView, çemberin özelliklerine göre bir çember çizer. CircleControl, kullanıcının grafik kullanıcı arayüzünden çember özelliklerini girmesini sağlar. Show adında iki düğme ile bir applet oluşturun. Denetleyici ve Görünümü, Şekil 39.2(a)'de gösterildiği gibi. Show Controller düğmesine tıkladığınızda, denetleyici bir çerçevede görüntülenir, Şekil 39.2(b)'de gösterildiği gibi. Show View düğmesine tıkladığınızda, görünüm ayrı bir çerçevede görüntülenir, Şekil 39.2(c)'de gösterildiği gibi.



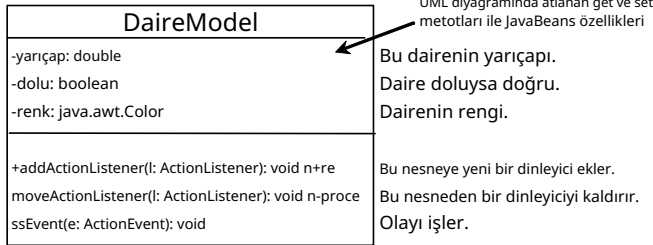
**Şekil 39.2**

Denetleyici, daire özelliklerini alır ve bunları bir daire modelinde saklar. Görünüm, daire modelinde belirtilen daireyi görüntüler.

**<margin note: tarih modeli>**

Daire modeli, yarıçap, dolu ve renk özelliklerini içerir ve ayrıca

eylem olayı için kayıt/kayıt iptali yöntemlerini içerir, Şekil 39.3'te gösterildiği gibi.



**Şekil 39.3**

Daire modeli verileri saklar ve veriler

değiştiğinde dinleyicileri bilgilendirir.

Bir özellik değeri değiştiğinde, dinleyiciler bilgilendirilir. CircleModel için tam kaynak kodu [Listeleme](#)

39.1'de verilmiştir.

**Listeleme 39.1 CircleModel.java**

**<kenar notu satırı 6: özellikler> n<kenar notu satırı 25: yangın olayı>**



*<marjin notu satırı 37: yangın olayı> n<m  
arjin notu satırı 49: yangın olayı> n<marji  
n notu satırı 54: standart kod> n<marjin notu  
satırı 62: standart kod> n<marjin notu satırı  
68: standart kod>*

```
1 java.awt.event.*i içe aktarın;
2 java.util.*i içe aktarın;
3
4 genel sınıf Çember Modeli {
5 /** Özellik yarıçap. */
6 özel ikili yarıçap = t 20t;
7
8 /** Özellik dolu. */
9 özel boolean doldurulmuş;
10
11 /** Özellik rengi. */
12 özel java.awt.Color renk;
13
14 /** Olay tetikleme mekanizması tarafından kullanılan yardımcı alan. */
15 özel ArrayList<ActionListener> eylemDinleyiciListesi;
16
17 genel double getYarıçap() {
18 return yarıçap;
19 }
20
21 genel void setYarıçap(double yarıçap) {
22 bu.yarıçap = yarıçap;
23
24 // Dinleyiciyi yarıçaptaki değişiklik için bilgilendir
25 süreçOlay(
26 new ActionEvent(this, ActionEvent.ACTION_PERFORMED, "yarıçap"));
27 }
28
29 genel boolean isFilled() {
30 return dolu;
31 }
32
33 public void setFilled(boolean dolu) {
34 this.dolu = dolu;
35
36 // Dolu üzerindeki değişiklik için dinleyiciyi bilgilendir
37 süreçOlay(
38 yeni ActionEvent(this, ActionEvent.ACTION_PERFORMED, "dolduruldu"));
39 }
40
41 genel java.awt.Color getColor() {
42 rengi döndür;
43 }
44
45 public voidt setColor(java.awt.Color color) {
46 thist.renk = renk;
47
48 // Renk değişikliği için dinleyiciyi bilgilendir
49 süreçOlay(
50 yeni ActionEvent(this, ActionEvent.ACTION_PERFORMED, "renk"));
51 }
52
53 /** Bir eylem olayı dinleyicisi kaydet */
54 genel senkronize voidt addActionListener(ActionListener l) {
55 if (actionListenerList == t nullt)
56 actionListenerList = t newt ArrayList<ActionListener>();
57
58 actionListenerList.add(l); n 59 }
```

```

60
61 /** Bir eylem olayı dinleyicisini kaldır */
62 genel senkronize voidt kaldırActionListener(ActionListener l) {
63 ift (actionListenerList != t nullt && actionListenerList.contains(l))
64 actionListenerList.remove(l); n 65 }

66
67 /** TickEvent'i ateşle */
68 özel void processEvent (ActionEvent e) {
69 ArrayList<ActionListener> liste;
70
71 senkronize edilmiş (bu) {
72 eğer (actionListenerList == null) return;
73 liste = (ArrayList<ActionListener>)(actionListenerList.clone());
74 }
75
76 için (int i = 0; i < liste.size(); i++) {
77 ActionListener dinleyici = liste.get(i);
78 dinleyici.actionPerformed(e);
79 } n 80 }
n 81 }

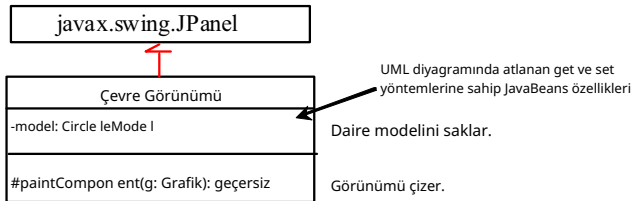
```

NOT

Kayıt/iptal/kayıt işlemi yöntemleri (54-80. satırlar) 27.2, CourseWithActionEvent.java'daki 49-82. satırlarla aynıdır. Eğer NetBeans ve Eclipse gibi bir GUI builder aracı kullanıyorsanız, kod otomatik olarak generat edilebilir.

#### <kenar notu: görünüm>

CircleView için UML diyagramı Şekil 39.4'te gösterilmiştir ve kaynak kodu Listede 39.2'de verilmiştir. Görünüm, modelden gelen bildirimleri dinler. Model, görünümün bir özelliği olarak içerir. Görünümde bir model ayarlandığında, bir dinleyici oluşturulur ve modelle kaydedilir (satırlar 13-17). Görünüm, JPanel'i genişletir ve daireyi modelde belirtilen özellik değerlerine göre çizmek için paintComponent yöntemini geçersiz kılar.



Şekil 39.4

Görünüm, daireyi modele göre görüntüler.

#### Liste 39.2 CircleView.java

##### <kenar notu satır 5: model>

##### <kenar notu satır 8: modeli ayarla>

##### <kenar notu satır 26: görünümü boyama>

```

1 java.awt.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3
4 genel sınıf CircleView t extendst javax.swing.JPanel {
5 özel Çember Modeli modeli;

```

```

6
7 /** Bir modeli ayarla */
8 genel geçersiz küme Modeli (Çember Modeli yeni Model) {
9 model = yeniModel;
10
11 if (model != t nullt)
12 // Model için dinleyici olarak görünümü kaydet
13 model.addActionListener(tnewt ActionListener() { n 14 @Override
15
16 public void actionPerformed(ActionEvent e) {
17 yenidençiz();
18 }
19 }); n 19 }
20
21 public DaireModel getModel() {
22 dönüş modeli;
23 }
24
25 @Geçersiz Kıl
26 korumalı voidt paintComponent(Graphics g) {
27 if (model != t nullt) {
28 supert.paintComponent(g);
29 g.setColor(model.getColor());
30
31 intt xCenter = getWidth() / t 2t;
32 intt yCenter = getHeight() / t 2t;
33 intt yarıçap = (intt)model.getRadius();
34
35 eğer (model.isFilled()) {
36 g.fillOval(xCenter - yarıçap, yCenter - yarıçap,
37 2 * yarıçap, 2 * yarıçap);
38 }
39 diğer {
40 g.drawOval(xCenter - yarıçap, yCenter - yarıçap,
41 2 * yarıçap, 2 * yarıçap);
42 }
43 }
44 }
45 }

```

Denetleyici, kullanıcının daire özelliklerini girmesine olanak tanıyan bir GUI arayüzü sunar. Daire modeli, özelliği olarak içerir. Denetleyici ile bir daire modeli ni ilişkilendirmek için setModel yöntemini kullanabilirsiniz. Yeni bir yarıçap elde etmek için bir metin alanı ve dairenin dolu olup olmadığını belirtmek için bir boolean değeri elde etmek için bir açılır kutu kullanır.

CircleController, Liste 39.3'te verilmiştir.

### Liste 39.3 CircleController.java

<margin note line 6: model>

<margin note line 14: kullanıcı arayüzü oluştur>

<margin note line 47: modeli ayarla>

```

1 java.awt.event.*'i içe aktarın;
2 java.aut.*'u içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 ortak sınıf Circle Controller, JPanel'i genişletecek {

```

```

6 özel Çember Modeli modeli;
7 özel JTextField jtf Yarıçap = r newt JTextField();
8 özel JComboBox jcb Doldurulmuş = newt JComboBox(newt Boolean[]{
9 newt Boolean(tfalse), t newt Boolean(ttrue));
10
11 /** Yeni form CircleController oluşturur */
12 genel Çevre Denetleyicisi() {
13 // Etiketleri gruplamak için panel
14 JPanel panel1 = newt JPanel(); n 15 panel1.
15 setLayout(newt GridLayout(t2t,t 1t)); n 16 panel1.add(newt
16 JLabel(t"Radius"t));
17 panel1.add(newt JLabel(t"Dolu"t));
18
19 // Metin alanı, kombinasyon kutusu ve başka bir paneli gruplamak için panel
20 JPanel panel2 = newt JPanel(); n 21 panel2.
21 setLayout(newt GridLayout(t2t,t 1t)); n 22 panel2.add(jtfRa
22 dius);
23 panel2.add(jcboFilled);
24
25 setLayout(newt BorderLayout()); n 26 add(p
26 anel1, BorderLayout.WEST); n 27 add(panel2, Bor
27 derLayout.CENTER);
28
29 // Dinleyicileri kaydet
30 jtfRadius.addActionListener(newt ActionListener() { n 31 @Override
31
32 public void actionPerformed(ActionEvent e) {
33 eğer (model != null) // Modelde yarıçapı ayarla
34 model.setRadius(Double.parseDouble(jtfRadius.getText())); n 35 }
35
36 });
37 jcboFilled.addActionListener(newt ActionListener() { n 38 @Override
38
39 public void actionPerformed(ActionEvent e) {
40 eğer (model != null) // Modelde dolu özellik değeri ayarla
41 model.setFilled(((Boolean)jcboFilled.getSelectedItem()).
42 booleanValue()); n 43 }
43
44 }); n 45 }
44
45
46
47 genel geçersiz küme Modeli (Çember Modeli yeni Model) {
48 model = yeniModel; n 49 }
49
50
51 genel CircleModel getModel() {
52 dönüş modeli;
53 } n 54
54 }

```

Son olarak, iki düğmeli MVCDemo adında bir applet oluşturalım, *Göster Kontrolcü* ve *Göster Görünüm*. *Göster Kontrolcü* düğmesi bir çerçevede bir kontrolcü gösterir ve *Göster Görünüm* düğmesi bir ayrı çerçevede bir görünüm gösterir. Program

Liste 39.4'te gösterilmektedir.

#### Liste 39.4 MVCDemo.java

<margin note line 8: model oluştur>  
 <margin note line 11: UI oluştur>  
 <margin note line 20: modeli ayarla>  
 <margin note line 33: modeli ayarla>  
 <margin note line 41: ana yöntem atlandı>

```
1 java.aut.*'u içe aktarın;
```

```

2 java.awt.event.*i içe aktarın;
3 javax.swing.*i içe aktarın;
4
5 genel sınıf MVCDemo t extendst JApplet {
6 private JButton jbtController = new JButton("Göster Kontrolcü");
7 private JButton jbtView = new JButton("Göster Görünüm");
8 özel CircleModel model = yeni CircleModel();
9
10 genel MVCDemo() {
11 setLayout(yeni FlowLayout());
12 add(jbtController);
13 ekle(jbtView);
14
15 jbtController.addActionListener(yeni ActionListener() {
16 @Override
17 public void actionPerformed(ActionEvent e) {
18 JFrame frame = yeni JFrame("Controller");
19 CircleController controller = yeni CircleController();
20 controller.setModel(model);
21 çerçeve.add(kontrolör); n 22 çerçeve.s
22 etSize(t200t,t 200t);n 23 çerçeve.setLocation(
23 t200t,t 200t);n 24 çerçeve.setVisible(ttrue);n 25 }
26 });
27
28 jbtView.addActionListener(yeni ActionListener() {
29 @Override
30 public void actionPerformed(ActionEvent e) {
31 JFrame frame = yeni JFrame("View");
32 CircleView view = yeni CircleView();
33 view.setModel(model);
34 çerçeve.add(görüntüleme); n 35
35 çerçeve.setSize(t500t,t 200t);n 36 çerçeve.se
36 tLocation(t200t,t 200t);n 37 çerçeve.setVisible(ttrue);
37 et);n 38 }
39 });
40 }
41 }

```

Model, verileri depolar ve işler, görünümler ise verileri sunmaktan sorumludur. Model-görünüm yaklaşımındaki temel sorun, görünümle model arasında tutarlılığı sağlamaktır. Modeldeki herhangi bir değişiklik, bağımlı görünümlere bildirilmelidir ve tüm görünümle aynı veriyi tutarlı bir şekilde göstermelidir. Modeldeki veriler, kontrolör aracılığıyla değiştirilir.

setRadius, setFilled ve setColor yöntemleri (satırlar 21, 33, 45) CircleModel'de processEvent yöntemini çağırarak özelliklerdeki herhangi bir değişikliği dinleyicilere bildirir.

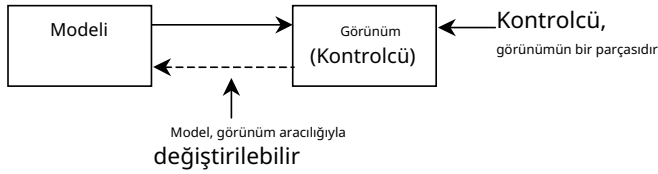
CircleView'deki setModel yöntemi yeni bir model ayarlar ve model için bir dinleyici kaydeder; bu, modelin addActionListener yöntemini çağırarak yapılır (satır 13). Modeldeki veriler değiştiğinde, dinleyicinin actionPerformed yöntemi çağırılır ve daire yeniden çizilir (satır 15).

CircleController denetleyicisi bir GUI sunar. Yarıçapı yarıçap metin alanından girebilirsiniz. Dairenin dolu olup olmadığını, iki Boolean nesnesi içeren açılır kutudan belirtebilirsiniz, `new Boolean(false)` ve `new Boolean(true)` (satırlar 8-9).

MVCDemo'da, her tıkladığınızda `Show Controller` düğmesine, yeni bir denetleyici oluşturulur (satır 18). Her tıkladığınızda `Show View` düğmesine, yeni bir görünüm oluşturulur (satır 30). Denetleyici ve görünüm aynı modeli paylaşır.

### 39.3 MVC Varyasyonları

Model-görünüm-denetleyici mimarisinin bir varyasyonu, denetleyiciyi görünümle birleştirir. Bu durumda, bir görünüm yalnızca dataları sunmakla kalmaz, aynı zamanda kullanıcı ile etkileşim kurmak ve kullanıcı girdisini kabul etmek için bir arayüz olarak da kullanılır; bu, Şekil 39.5'te gösterilmiştir.



### Şekil 39.5

Görünüm, kullanıcı ile etkileşimde bulunabilir ve verileri görüntüleyebilir.

Örneğin, kullanıcıya fare ile dairenin yarıçapını değiştirme imkanı vermek için, önceki örnekteki görünümü değiştirebilirsiniz.

Sol fare tuşuna tıkladığında, yarıçap

5 piksel artırılır. Sağ fare tuşuna tıkladığında, yarıçap

5 piksel azaltılır. ViewController adı verilen yeni görünüm, CircleView'i genişletek aşağıdaki gibi uygulanabilir:

<margin note line 6: fare dinleyicisi>

<margin note line 11: sol buton?>

<margin note line 13: sağ buton?>

```

1 java.awt.event.MouseEvent'i içe aktarın;
2
3 genel sınıf ViewController t extendst CircleView {
4 genel ViewController() {
5 // Fare dinleyicisini kaydet
6 addMouseListener(new java.awt.event.MouseAdapter() {
7 public void mousePressed(java.awt.event.MouseEvent e) {
8 CircleModel model = getModel(); // Modeli al
9
10 if (model != null)
11 if (e.getButton() == MouseEvent.BUTTON1)
12 model.setRadius(model.getRadius() + 5); // Sol buton
13 else if (e.getButton() == MouseEvent.BUTTON3)
14 model.setRadius(model.getRadius() - 5); // Sağ buton
15 }
16 });
17 }
18 }

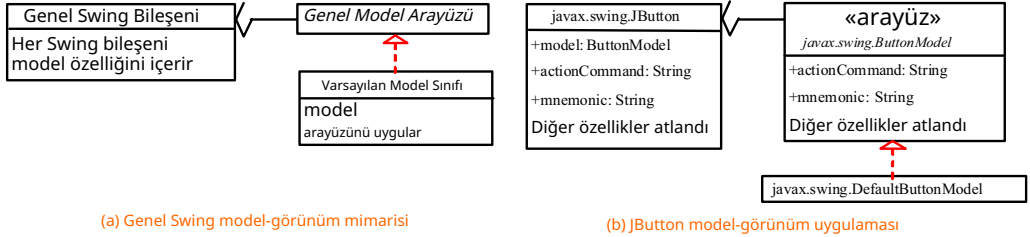
```

Model-görünüm-denetleyici mimarisinin bir başka varyasyonu, modelden bazı verileri görünümüne ekler, böylece sık kullanılan verilere doğrudan görünümde erişilebilir. Swing bileşenleri MVC mimarisi kullanılarak tasarlanmıştır. Her Swing GUI bileşeni, verileri depolamak için bir model kullanan bir görünümüdür. Bir Swing GUI bileşeni, modelde bazı veriler içerir, böylece bileşenden doğrudan erişilebilir.

### 39.4 Swing Model-Görünüm-Denetleyici Mimarisi

Her Swing kullanıcı arayüzü bileşeni (JPanel, JSplitPane, JFileChooser ve JColorChooser gibi bazı kapsayıcılar ve diyalog kutuları hariç) verilerini temsil eden bir model adında bir özelliğe sahiptir. Veri modeli, adı Model ile biten bir arayüzde tanımlanır. Örneğin, düğme bileşeninin modeli

ButtonModel'dir. Çoğu model arayüzünün varsayılan bir uygulama sınıfı vardır ve genellikle DefaultX olarak adlandırılır; burada X, model arayüzü adıdır. Örneğin, ButtonModel için varsayılan uygulama sınıfı DefaultButtonModel'dir. Bir Swing bileşeni, model arayüzü ve varsayılan model uygulama sınıfı arasındaki ilişki Şekil 39.6'da gösterilmektedir.



Şekil 39.6

Swing bileşenleri MVC mimarisi kullanılarak uygulanmaktadır.

Kolaylık olması açısından, çoğu Swing bileşeni, modellerinin bazı özelliklerini içerir ve bu özelliklere, modelin varlığının bilinmesine gerek kalmadan, doğrudan bileşenden erişilip değiştirilebilir. Örneğin, actionCommand ve mnemonic özellikleri hem ButtonModel hem de JButton'da tanımlanmıştır. Aslında, bu özellikler AbstractButton sınıfındadır. JButton, AbstractButton'ın bir alt sınıfı olduğundan, AbstractButton'dan tüm özellikleri miras alır.

Bir model belirtmeden bir Swing bileşeni oluşturduğunuzda, model özelliğine varsayılan bir veri modeli atanır. Örneğin, aşağıdaki kodda 9-10. satırlar, bir düğmenin model aracılığıyla actionCommand ve mnemonic özelliklerini ayarlar.

<margin note line 6: model al>  
<margin note line 9: model özelliklerini ayarla>

```
1 genel sınıf Testi SwingModel 1 {
2 public static void main(String[] args) {
3 javax.swing.JButton jbt = new javax.swing.JButton();
4
5 // Bileşenden varsayılan modeli al
6 javax.swing.ButtonModel modeli = jbt.getModel();
```

```

7
8 // Modelde özellikleri ayarla
9 model.setActionCommand("OK");
10 model.setMnemonic('O');
11
12 // Bileşenden özellik değerlerini göster
13 System.out.println("actionCommand şu: " + jbt.getActionCommand());
14 System.out.println("mnemonic şu: " + (char) (jbt.getMnemonic()));
15 }
16 }

```

#### <Çıktı>

actionCommand tamam  
mnemonic O'dur

#### <Çıkış Sonu>

Ayrıca yeni bir model oluşturabilir ve bunu bir Swing kapsayıcısına atayabilirsiniz. Örneğin, aşağıdaki kod bir ButtonModel örneği oluşturur (satır 7) ve bunu bir JButton örneğine atar (satır 14).

#### <kenar notu satır 7: model oluştur>

#### <kenar notu satır 10: model özelliklerini ayarla>

#### <kenar notu satır 14: yeni bir model ayarla>

```

1 genel sınıf SwingModel2 Testi {
2 public static void main(String[] args) {
3 javax.swing.JButton jbt = yeni javax.swing.JButton();
4
5 // Yeni bir buton modeli oluştur
6 javax.swing.ButtonModel modeli =
7 new javax.swing.DefaultButtonModel();
8
9 // Modelde özellikleri ayarla
10 model.setActionCommand("İptal");
11 model.setMnemonic('C');
12
13 // Modeli düğmeye ata
14 jbt.setModel(model);
15
16 // Bileşenden özellik değerlerini göster
17 System.out.println("actionCommand şu: " + jbt.getActionCommand());
18 System.out.println("mnemonic şu: " + (char) jbt.getMnemonic());
19 }
20 }

```

#### <Çıktı>

actionCommand şu: İptal  
mnemonic şu: C

#### <Çıkış Sonu>

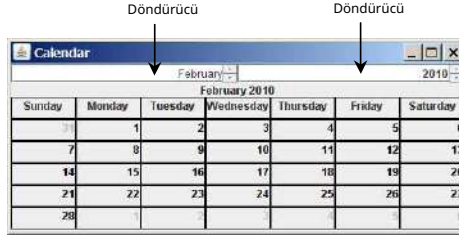
JButton, JToggleButton, JCheckBox, JRadioButton gibi basit Swing bileşenleri için modelleri kullanmak gereksizdir, JTextField ve JTextArea, çünkü bu Swing bileşenlerinin modellerinde sıkça kullanılan özellikler, bu bileşenlerde de bulunan özelliklerdir. Bu özelliklere doğrudan bileşenler aracılığıyla erişebilir ve bu özellikleri değiştirebilirsiniz. Gelişmiş bileşenler için,



JSpinner, JList, JComboBox, JTable ve JTree gibi, verileri depolamak, erişmek ve değiştirmek için modelleri ile çalışmanız gerekir.

### 39.5 JSpinner

Bir spinner, sağ tarafında kullanıcıya sayıları, tarihleri veya sıralı bir diziden değerleri seçme imkanı veren bir çift küçük ok düğmesi bulunan bir metin alanıdır. Klavye yukarı/aşağı ok tuşları da öğeler arasında geçiş yapar. Kullanıcının spinner'a doğrudan (geçerli) bir değer yazmasına da izin verilebilir. Bir spinner, bir combo kutusuna benzer ancak bazen önemli verileri gizleyebileceği bir açılır liste gerektirmediği için tercih edilir.



**Şekil 39.7**

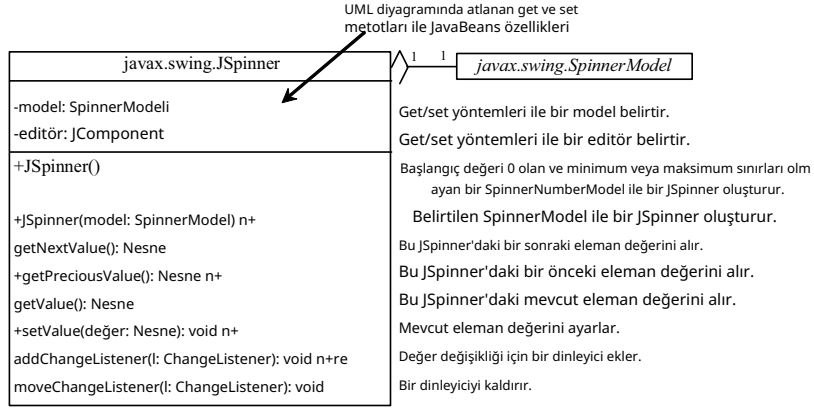
*İki JSpinner bileşeni, kullanıcının takvim için bir ay ve bir yıl seçmesine olanak tanır.*

Şekil 39.8, JSpinner'daki yapıları ve yaygın olarak kullanılan yöntemleri göstermektedir. Bir JSpinner'in sıra değeri,

SpinnerModel arayüzü, potansiyel olarak sınırsız bir eleman dizisini yönetir.

Model, dizinin elemanlarına indeksli rastgele erişimi desteklemez. Sadece üç dizi elemanına aynı anda erişilebilir—mevcut, sonraki ve önceki—sırasıyla getValue(), getNextValue() ve getPreviousValue() yöntemleri kullanılarak. Mevcut dizi elemanı, setValue yöntemi kullanılarak değiştirilebilir. Bir spinner'daki mevcut değer değiştirildiğinde, model

stateChanged(javax.swing.event.ChangeEvent e) yöntemi kayıtlı dinleyicilerin. Dinleyiciler javax.swing.event.ChangeListener'ı uygulamalıdır. SpinnerModel'deki tüm bu yöntemler, JSpinner'da da kolaylık olması açısından tanımlanmıştır, böylece modeldeki verilere doğrudan JSpinner üzerinden erişebilirsiniz.



**Şekil 39.8**

JSpinner, verileri depolamak için bir spinner modeli kullanır.

NOT: Bir model belirtmeden bir JSpinner nesnesi oluşturursanız, spinner bir dizi tam sayı gösterir.

Liste 39.5, bir dizi sayı için bir JSpinner nesnesi oluşturan ve spinner'dan önceki, mevcut ve sonraki sayıları bir etiket üzerinde gösteren bir örnek vermektir; bu, Şekil 39.9'da gösterilmiştir.



**Şekil 39.9**

Spinner'daki önceki, mevcut ve sonraki değerler etiket üzerinde gösterilmektedir.

#### Liste 39.5 SimpleSpinner.java

<margin note line 7: spinner>

<margin note line 18: spinner dinleyicisi>

<margin note line 27: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2 javax.swing.event.*'i içe aktarın;
3 Java.awt.BorderLayout'u içe aktarın;
4
5 public class BasitSpinner extends JApplet {
6 // Bir JSpinner oluşturun
7 özel JSpinner döndürücü = t newt JSpinner();
8
9 // Bir JLabel oluşturun
10 özel JLabel etiketi = t newt JLabel(t""t, JLabel.CENTER);
11
12 public BasitSpinner() {
13 // Spinner ve label'ı UI'ye ekle
14 add(döndürücü, BorderLayout.NORTH); n 15 ad
15 d(label, BorderLayout.CENTER);
16
17 // Bir dinleyici kaydet ve oluşturun

```

```

18 spinner.addChangeListener(tnewt ChangeListener() {
19 @Geçersiz kıl
20 public voidt stateChanged(javax.swing.event.ChangeEvent e) {
21 label.setText("Önceki değer: " + spinner.getPreviousValue()
22 + " Mevcut değer: " + spinner.getValue()
23 + " Sonraki değer: " + spinner.getNextValue());
24 }
25 }); n 26 } n
27 }

```

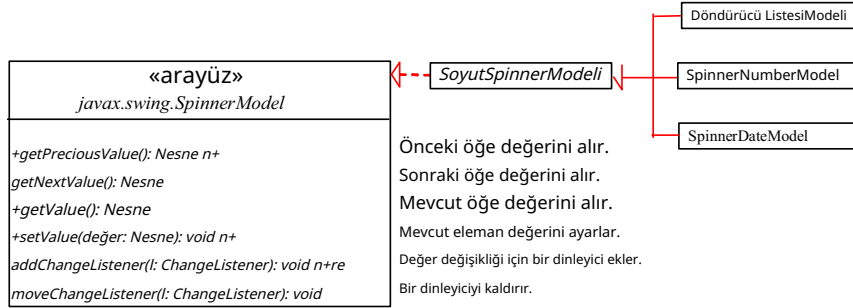
Bir `JSpinner` nesnesi, argüman almayan yapıcısı kullanılarak oluşturulur (satır 7). Varsayılan olarak, bir spinner bir dizi tam sayı gösterir.

Spinner üzerindeki değer değişikliği olayını işlemek için anonim bir iç sınıf olay adaptörü oluşturulur (satırlar 18-25). Bir spinnere ait önceki, mevcut ve sonraki değerler, `JSpinner`'in örnek yöntemleri `getPreviousValue()`, `getValue()` ve `getNextValue()` kullanılarak elde edilebilir.

Tam sayılardan farklı bir değer dizisi göstermek için spinner modellerini kullanmanız gerekir.

### 39.6 Spinner Modelleri ve Editörleri

`SpinnerModel`, tüm spinner modelleri için bir arayüzdür. `AbstractSpinnerModel`, `SpinnerModel`'i uygulayan ve kayıt/dış kayıt yöntemleri için uygulamayı sağlayan kullanışlı bir soyut sınıftır. `SpinnerListModel`, `SpinnerNumberModel` ve `SpinnerDateModel`, `SpinnerModel`'in somut uygulamalarıdır. Aralarındaki ilişki Şekil 39.10'da gösterilmektedir. Bu modellerin yanı sıra, `AbstractSpinnerModel`'i genişleten veya doğrudan `SpinnerModel`'i uygulayan özel bir spinner modeli oluşturabilirsiniz.

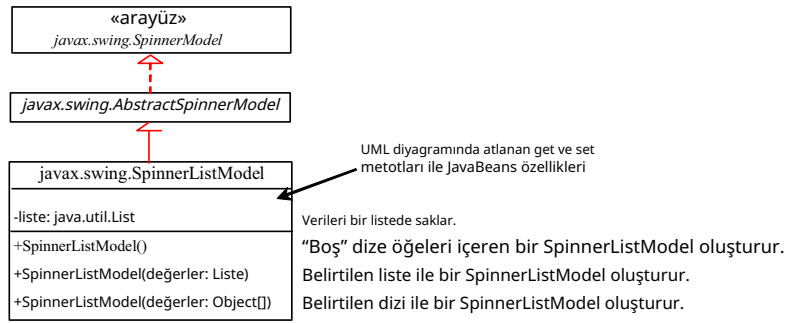


Şekil 39.10

`SpinnerListModel`, `SpinnerNumberModel` ve `SpinnerDateModel`, `SpinnerModel` için somut uygulamalardır.

#### 39.6.1 SpinnerListModeli

`SpinnerListModel` (bkz. Şekil 39.11), değerlerinin `java.util.List` içinde saklandığı `SpinnerModel`'in basit bir uygulamasıdır.



**Şekil 39.11**

SpinnerListModel, modelde bir veri dizisini saklamak için java.util.List kullanır.

Bir SpinnerListModel oluşturmak için bir dizi veya liste kullanabilirsiniz. Örneğin, aşağıdaki kod, bir dizide Freshman, Sophomore, Junior, Senior ve Graduate değerlerinden oluşan bir model oluşturur.

```

// Bir dizi oluştur
String[] notları = {"Birinci Sınıf", "İkinci Sınıf", "Orta", "Son Sınıf", "Mezun"};

// Bir diziden model oluştur
model = new SpinnerListModel(grades);

```

Alternatif olarak, aşağıdaki kod bir liste kullanarak bir model oluşturur:

```

// Bir dizi oluştur
String[] notları = {"Birinci Sınıf", "İkinci Sınıf", "Orta", "Son Sınıf", "Mezun"};

// Diziden bir ArrayList oluştur
list = new ArrayList(Arrays.asList(grades));

// Listeden bir model oluştur
model = new SpinnerListModel(list);

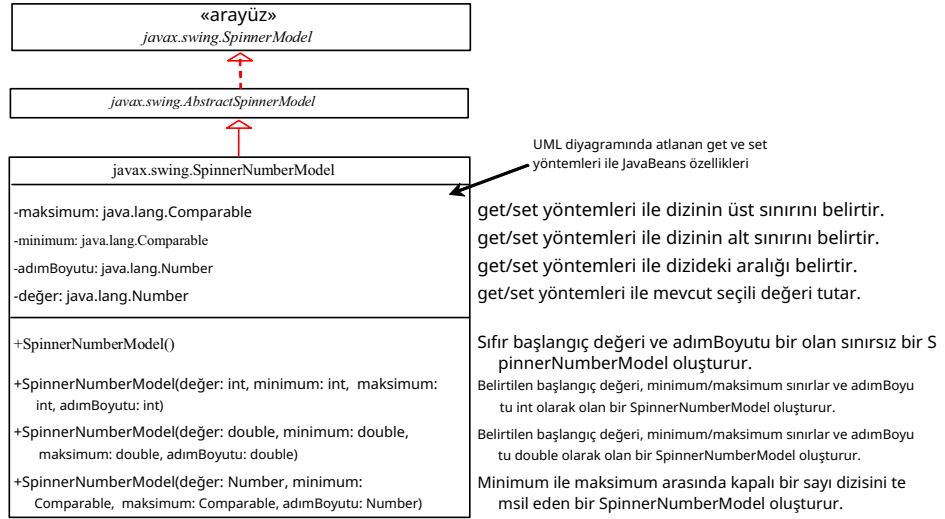
```

Alternatif kod gereksiz görünüyor. Ancak, modelden eleman eklemek veya çıkarmak gerektiğinde faydalıdır. Dizi oluşturulduğunda boyutu sabittir. Liste, elemanları dinamik olarak eklemenizi veya çıkarmanızı sağlayan esnek bir veri yapısıdır.

### 39.6.2 SpinnerNumberModeli

SpinnerNumberModel (Şekil 39.12'ye bakın), bir sayı dizisini temsil eden SpinnerModel'in somut bir uygulamasıdır. Maksimum, minimum ve adım boyutu özelliklerini içerir. Maksimum ve minimum özellikleri dizinin üst ve alt sınırlarını belirtir. Adım boyutu, SpinnerModel'de tanımlanan nextValue ve previousValue yöntemleri tarafından hesaplanan artış veya azalışın boyutunu belirtir. Minimum ve

maksimum özellikler, dizinin alt veya üst sınırının olmadığını belirtmek için null olabilir. Bu sınıftaki tüm özellikler Number veya Comparable olarak tanımlanmıştır, böylece tüm Java sayısal türleri karşılanabilir.



Şekil 39.12

*SpinnerNumberModel, bir sayı dizisini temsil eder.*

Bir SpinnerNumberModel'i tam sayılar veya double ile oluşturabilirsiniz. Örneğin, aşağıdaki kod, başlangıç değeri 2004 ve aralık 1 olan 0'dan 3000'e kadar bir sayı dizisini temsil eden bir model oluşturur.

```
// Bir spinner sayı modeli oluşturun
SpinnerNumberModel model = new SpinnerNumberModel(2004, 0, 3000, 1);
```

Aşağıdaki kod, bir sayı dizisini temsil eden bir model oluşturur.

0 ile 120 arasında, başlangıç değeri 50 ve aralık 0.1 olan sayılar.

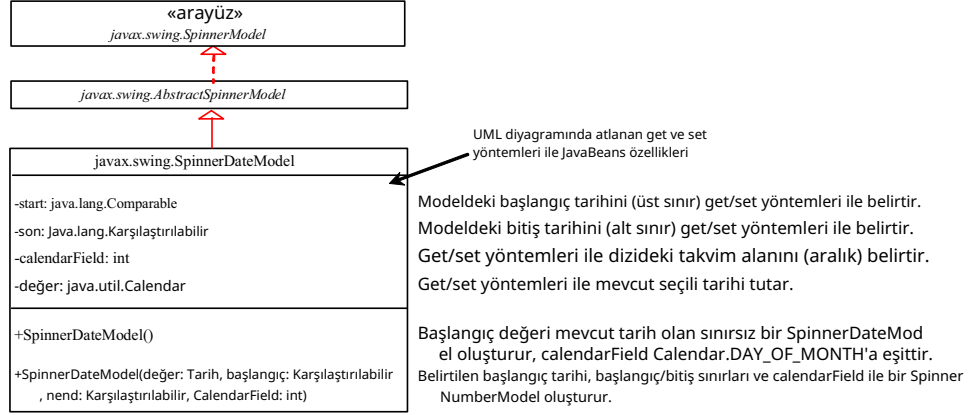
```
// Bir spinner sayı modeli oluşturun
SpinnerNumberModel modeli = new SpinnerNumberModel(50, 0, 120, 0.1);
```

### 39.6.3 SpinnerDateModeli

SpinnerDateModel (bkz. Şekil 39.13), bir tarih dizisini temsil eden SpinnerModel'in somut bir uygulamasıdır. Dizinin üst ve alt sınırları, start ve end adlı özellikler tarafından tanımlanır ve nextValue ve previousValue yöntemleriyle hesaplanan artış veya azalışın boyutu, calendarField adlı bir özellik tarafından tanımlanır. Start ve end özellikleri, dizinin alt veya üst sınırının olmadığını belirtmek için null olabilir. calendarField özelliğinin değeri, bir alanı belirten java.util.Calendar sabitlerinden biri olmalıdır.

Takvim. getNextValue ve getPreviousValue yöntemleri, tarihi bu miktar kadar ileri veya geri değiştirir. Örneğin, eğer

calendarField Calendar.DAY\_OF\_WEEK ise, nextValue mevcut değerdan 24 saat s onrasını üretecek ve previousValue 24 saat öncesini üretecektir. \_\_\_\_\_



Şekil 39.13

*SpinnerDateModel, bir tarih dizisini temsil eder.*

Örneğin, aşağıdaki kod, mevcut tarihten başlayarak alt/üst sınır olmadan ve takvim alanı ay olan bir tarih dizisini temsil eden bir spinner modeli oluşturur.

```
SpinnerDateModel model = new SpinnerDateModel(
 new Date(), null, null, Calendar.MONTH);
```

#### 39.6.4 Spinner Editörleri

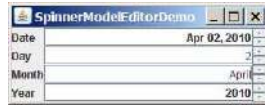
Bir JSpinner, *editor* olarak adlandırılan tek bir çocuk bileşene sahiptir, bu bileşen modelin mevcut ögesini veya değerini görüntülemekten sorumludur. Dört editör, JSpinner içinde statik iç sınıflar olarak tanımlanmıştır.

- **JSpinner.DefaultEditor** modelin mevcut değerinin yalnızca okunabilir bir görünümünü JFormattedTextField ile görüntülemek için diğer tüm özel editörler için basit bir temel sınıftır. JFormattedTextField, rastgele değerlerin biçimlendirilmesi için destek ekleyerek JTextField'i genişletir ve kullanıcı metni düzenledikten sonra belirli bir nesneyi almayı sağlar.
- **JSpinner.NumberEditor** bir JSpinner için özel bir editördür bu nun modeli bir SpinnerNumberModel'dir. Editörün değeri, formatı bir NumberFormatter örneği tarafından tanımlanan bir JFormattedTextField ile görüntülenir.
- **JSpinner.DateEditor** bir JSpinner için özel bir editördür bunun modeli bir SpinnerDateModel'dir. Editörün değeri, formatı bir DateFormatter örneği tarafından tanımlanan bir JFormattedTextField ile görüntülenir.
- **JSpinner.ListEditor** bir JSpinner için özel bir editördür bunun modeli bir SpinnerListModel'dir. Editörün değeri, JFormattedTextField ile görüntülenir.

JSpinner'in yapıcı metodu, SpinnerNumberModel için bir NumberEditor, SpinnerDateModel için bir DateEditor, SpinnerListModel için bir ListEditor ve diğer tüm modeller için bir DefaultEditor oluşturur. Editör, setEditor metodu kullanılarak da değiştirilebilir. JSpinner'in editörü, ChangeEvents için dinleyerek modelle senkronizedir. commitEdit() metodu, şu anda düzenlenen değeri modele kaydetmek için kullanılmalıdır.

### 39.6.5 Örnek: Spinner Modelleri ve Editörleri Kullanma

Bu örnek, tarihi görüntülemek için bir JSpinner bileşeni ve sayı dizisinde günü, dizi dizisinde ayı ve sayı dizisinde yılı görüntülemek için üç başka JSpinner bileşeni kullanır; bu, Şekil 39.14'te gösterilmiştir. Dört bileşen senkronizedir. Örneğin, yıl spinner'ında yılı değiştirirseniz, tarih spinner'ındaki tarih değeri gerektiği gibi güncellenir. Örneğin kaynak kodu, Liste 39.6'da verilmiştir.



Şekil 39.14

Dört spinner bileşeni, tarihi bir alanda ve günü, ayı ve yılı üç ayrı alanda görüntülemek için senkronizedir.

#### Liste 39.6 SpinnerModelEditorDemo.java

<margin note line 9: spinners>  
<margin note line 20: UI oluştur>  
<margin note line 41: tarih editörü>  
<margin note line 46: sayı editörü>  
<margin note line 54: spinner dinleyici>  
<margin note line 62: spinner dinleyici>  
<margin note line 70: spinner dinleyici>  
<margin note line 109: ana metod atlandı>

```
1 javax.swing.*'i içe aktarın;
2 javax.swing.event.*'i içe aktarın;
3 java.util.*'i içe aktarın;
4 java.text.*'i içe aktarın;
5 java.aut.*'u içe aktarın;
6
7 public classt SpinnerModelEditorDemo t extendst JApplet {
8 // Tarih, gün, ay ve yıl için dört spinner oluşturun
9 özel JSpinner jspDate =
10 newt JSpinner(tnewt SpinnerDateModel());
11 özel JSpinner jspGünü =
12 yeni JSpinner(yeni SpinnerNumberModel(1,1,31,1));
13 özel String[] ayİsimleri = yeni DateFormatSymbols().getMonths();
14 özel JSpinner jspAy = yeni JSpinner
15 (yeni SpinnerListModel(Arrays.asList(ayİsimleri).subList(0,12)));
16 özel JSpinner spinnerYıl =
17 yeni JSpinner(yeni SpinnerNumberModel(2004,1,3000,1));
18
19 genel SpinnerModelEditorDemo() {
20 // Grup etiketleri
21 JPanel panell = yeni JPanel();
```

```

22 panel1.setLayout (yeni GridLayout (4,1));
23 panel1.add(yeni JLabel ("Tarih"));
24 panel1.add(new JLabel ("Gün"));
25 panel1.add(new JLabel ("Ay"));
26 panel1.add(new JLabel ("Yıl"));

27
28 // Grup spinner'ları
29 JPanel panel2 = tnewt JPanel(); n 30 panel2.
setLayout(tnewt GridLayout(t4,t 1t)); n 31 panel2.add(jspD
ate);
32 panel2.add(jspDay); n 33 panel
2.add(jspMonth); n 34 panel2.add(sp
innerYear);

35
36 // Spinner ve etiketi UI'ye ekle
37 add(panel1, BorderLayout.WEST); n 38 add(pa
nel2, BorderLayout.CENTER);

39
40 // Tarih için editör ayarla
41 JSpinner.DateEditor dateEditor =
42 newt JSpinner.DateEditor(jspDate, t"MMM gg, yyyy"t);
43 jspDate.setEditor (dateEditor);

44
45 // Yıl için editör ayarla
46 JSpinner.NumberEditor yılEditör =
47 newt JSpinner.NumberEditor(spinnerYear, t"####"t);
48 spinnerYear.setEditor (yearEditor);

49
50 // Gün, ay ve yıl ile senkronize etmek için tarihi güncelle
51 güncellemeTarihi();

52
53 // jspDay için bir dinleyici kaydet ve oluştur
54 jspDay.addChangeListener(tnewt ChangeListener() { n 55 @Overri
de
56 public voidt stateChanged(javax.swing.event.ChangeEvent e) {
57 güncellemeTarihi(); n 58 }

59 });

60
61 // jspMonth için bir dinleyici kaydedin ve oluşturun
62 jspMonth.addChangeListener(tnewt ChangeListener() { n 63 @Overri
de
64 public voidt stateChanged(javax.swing.event.ChangeEvent e) {
65 güncellemeTarihi(); n 66 }

67 });

68
69 // spinnerYear için bir dinleyici kaydet ve oluşturun
70 spinnerYear.addChangeListener(tnewt ChangeListener() { n 71 @Overrid
e
72 public voidt stateChanged(javax.swing.event.ChangeEvent e) {
73 güncellemeTarihi(); n 74 }

75 }); n 76 }

77
78 /** Diğer spinner'larla senkronize etmek için tarih spinner'ını güncelle */
79 özel voidt güncellemeTarihi() {
80 // Mevcut ay ve yılı int olarak al
81 int ay = ((SpinnerListModel)jspMonth.getModel()).
82 getList().indexOf(jspMonth.getValue());
83 int yıl = ((Integer)spinnerYear.getValue()).intValue();

84
85 // Yeni ay ve yıl için maksimum gün sayısını ayarlayın
86 SpinnerNumberModel numberModel = n 87 (Sp
innerNumberModel)jspDay.getModel();

```



```

88 numberModel.setMaximum(tnewt Integer(maxDaysInMonth(yıl, ay)));
89
90 // Maksimumu aşarsa yeni bir gün ayarlayın
91 ift (((Tamsayı)(numberModel.getValue())).intValue() >
92 maxDaysInMonth(yıl, ay)) n 93 numberModel.s
94 etValue(tnewt Integer(maxDaysInMonth(yıl, ay)));
95
96 // Mevcut günü alın
97 int gün = ((Integer)jspDay.getValue()).intValue();
98
99 // Tarih spinner'ında yeni bir tarih ayarla
100 jspDate.setValue (
101 new GregorianCalendar(yıl, ay, gün).getTime());
102
103 /** Bir ay içindeki maksimum gün sayısını döndür. Örneğin,
104 Şub 2004'te 29 gün vardır. */
105 private int maxDaysInMonth (int yıl, int ay) {
106 Takvim takvim = new GregorianCalendar(yıl, ay, 1) ;
107 return takvim.getActualMaximum(Calendar.DAY_OF_MONTH);
108 } n10
9}

```

Bir tarih için JSpinner nesnesi, varsayılan SpinnerDateModel ile oluşturulur (satırlar 9-10). Spinner'da görüntülenen tarih formatı MMM dd, yyyy şeklindedir (örneğin, 01 Şub, 2006). Bu format, JSpinner'in iç sınıf yapıcısı kullanılarak oluşturulur.

DateEditor (satırlar 41-42) olarak ayarlanır ve jspDate'in editörü olarak atanır (satır 43) .

Bir gün için JSpinner nesnesi, 1 ile 31 arasında bir tamsayı dizisi ile SpinnerNumberModel ile oluşturulur; başlangıç değeri 1 ve aralık 1'dir (satırlar 11-12). Maksimum sayı, mevcut ay ve yıla göre updateDate() metodunda sıfırlanır (satırlar 91-93). Örneğin,

Şubat 2004'te 29 gün vardır, bu nedenle jspDay için maksimum değer Şubat 2004'te 29 olarak ayarlanır.

Bir ay için JSpinner nesnesi, bir ay isimleri listesi ile SpinnerListModel ile oluşturulur (satırlar 14-15). Ay isimleri yerel spesifiktir ve yeni DateFormatSymbols().getMonths() kullanılarak elde edilebilir (satır 13). Bazı takvimlerde 13 ay olabilir. Arrays.asList(monthNames) bir dizi stringden bir liste oluşturur ve su bList(0, 12) listede ilk 12 elemanı döndürür.

Bir JSpinner nesnesi olan spinnerYear, 1 ile 3000 arasındaki tam sayı dizisi ile bir SpinnerNumberModel ile oluşturulur. Başlangıç değeri 2004 ve aralık 1'dir (satırlar 16-17). Varsayılan olarak, yerel spesifik sayı ayırıcıları kullanılır.

Örneğin, 2004, spinner'da 2,004 olarak görüntülenecektir. Sayıyı ayırıcı olmadan göstermek için, yeni bir NumberEditor oluşturmak üzere sayı deseni ##### belirtilmiştir (satırlar 46-47). Editör, spinnerYear'ın editörü olarak ayarlanmıştır (satır 48).

updateDate() metodu, tarih spinner'ını gün, ay ve yıl spinner'ı ile senkronize eder. Gün, ay veya yıl spinner'ında yeni bir değer seçildiğinde, tarih spinner'ında yeni bir tarih ayarlanır. maxDaysInMonth metodu (satırlar 105-108), bir ay içindeki maksimum gün sayısını döndürür. Örneğin, Şubat 2004'te 29 gün vardır.

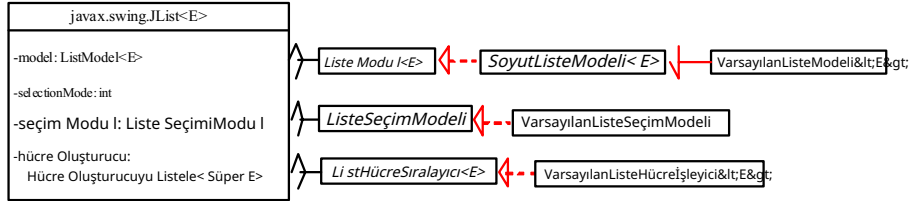
Bir `JSpinner` nesnesi, spinner'daki durum değişikliğini dinleyicilere bildirmek için `javax.swing.event.ChangeEvent` tetikleyebilir. Gün, ay ve yıl spinner'ları için spinner durumu değişikliklerini işlemek üzere anonim olay adaptörleri oluşturulmuştur (satırlar 54-75). Bu üç spinner'dan birinde yeni bir değer seçildiğinde, tarih spinner'ı değeri buna göre güncellenir. Egzersiz 39.3'te, gün, ay ve yıl spinner'larını tarih spinner'ı ile senkronize etmek için örneği geliştireceksiniz. Ardından, tarih spinner'ında yeni bir değer seçildiğinde, gün, ay ve yıl spinner'larındaki değerler buna göre güncellenecektir.

Bu örnek, `SpinnerNumberModel`, `SpinnerDateModel` ve `SpinnerListModel` kullanılmaktadır. Bunlar, API'de önceden tanımlanmış somut spinner modelleridir. Ayrıca özel spinner modelleri de oluşturabilirsiniz (bkz. Alıştırma 39.4).

### 39.7 JList ve Modelleri

`JList`'in temel özellikleri, §17.9, "Listeler" bölümünde, liste modelleri kullanmadan anlatılmıştır. Bir liste oluşturmayı ve liste seçimlerine nasıl yanıt verileceğini öğrendiniz. Ancak, liste modelleri kullanmadan bir listeden eleman ekleyemez veya çıkaramazsınız. Bu bölüm, liste modellerini tanıtır ve `JList`'in nasıl kullanılacağına dair ayrıntılı bir tartışma sunar.

`JList`'in iki destekleyici modeli vardır: bir liste modeli ve bir liste-seçim modelidir. *Liste modeli* verileri depolamak ve işlemek içindir. *Liste-seçim modeli* öğeleri seçmek içindir. Varsayılan olarak, öğeler dizeler veya simgeler olarak görüntülenir. Ayrıca, `ListCellRenderer` arayüzünü uygulayan özel bir render'ı tanımlayabilirsiniz. Bu arayüzlerin ve sınıfların ilişkisi Şekil 39.15'te gösterilmiştir.



Şekil 39.15

*JList*, birkaç destekleyici arayüz ve sınıf içerir.

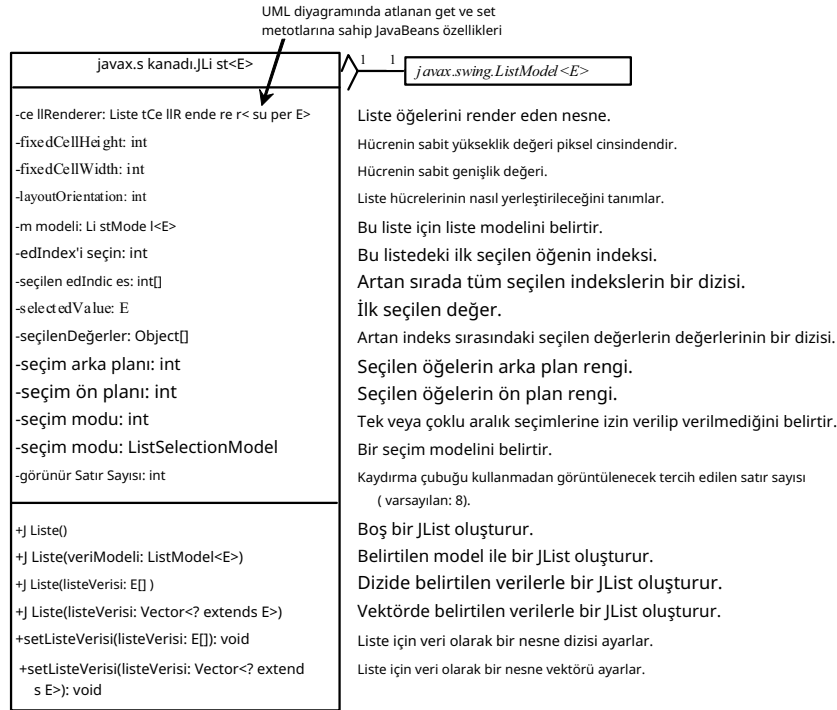
NOT

**&lt;marginal not: JDK 7 yeni özellikleri&gt;**

JDK 7'den itibaren, `JList`, `ListeModeli`, `SoyutListeModeli`, `VarsayilanListeModeli`, `ListeHücreİşleyici` ve `VarsayilanListeHücreİşleyici`, genel sınıflar ve arayüzler olarak yeniden tanımlanmıştır. Genel tür `E`, liste içinde saklanan eleman türünü temsil eder.

### 39.7.1 JList Yapıcıları, Özellikleri ve Yöntemleri

Şekil 39.16, JList'in özelliklerini ve yapıcılarını göstermektedir. Bir liste modelind en, bir nesne dizisinden veya bir vektörden bir liste oluşturabilirsiniz.



### Şekil 39.16

*JList, öğeleri bir listede görüntüler.*

### 39.7.2 Liste Düzeni Yönleri

layoutOrientation özelliği, öğelerin düzenini aşağıdaki üç değerden biriyle belirtir:

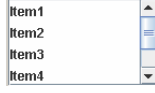
JList.VERTICAL, hücrelerin bir sütunda dikey olarak düzenlenmesi gerektiğini belirtir. Bu varsayılan değerdir.

JList.HORIZONTAL\_WRAP, hücrelerin yatay olarak düzenlenmesi gerektiğini ve gerektiğinde yeni bir satıra sarılacağını belirtir. Kullanılacak satır sayısı, `visibleRowCount` özelliği 0'dan büyükse belirlenir; aksi takdirde satır sayısı `JList`'in genişliği ile belirlenir.

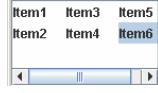
JList.VERTICAL\_WRAP hücrelerin gerektiğinde yeni bir sütuna sarılarak dikey olarak düzenlenmesi gerektiğini belirtir. Kullanılacak satır sayısı, `visibleRowCount` özelliği 0'dan büyükse belirlenir; aksi takdirde satır sayısı

JList'in yüksekliği ile belirlenir.

Örneğin, listede beş öğe (item1, item2, item3, item4 ve item5) olduğunu ve visibleRowCount'un 2 olduğunu varsayalım. Şekil 39.17 bu üç durumda düzeni göstermektedir.



(a) Dikey



(b) Dikey sarma



(c) Yatay sarma

Şekil 39.17

Düzen yönelimi, öğelerin bir listede nasıl yerleştirileceğini belirtir.

### 39.7.3 Liste-Seçim Modları ve Liste-Seçim Modelleri

selectionMode özelliği, tek bir öğe, tek aralıklı öğe veya çoklu aralıklı öğe seçilip seçilemeyeceğini gösterir. Seçim modları SINGLE\_SELECTION, SINGLE\_INTERVAL\_SELECTION, MULTIPLE\_INTERVAL\_SELECTION ve Şekil 39.18'de gösterildiği gibi. Tek seçim yalnızca bir öğenin seçilmesine izin verir. Tek aralıklı seçim birden fazla seçime izin verir, ancak seçilen öğeler bitişik olmalıdır. Bu öğeler, SHIFT tuşuna basılı tutarak birlikte seçilebilir.

Çoklu aralıklı seçim, kısıtlama olmaksızın birden fazla bitişik öğenin seçilmesine izin verir. Bu öğeler Ctrl tuşuna basılı tutarak seçilebilir. Varsayılan değer MULTIPLE\_INTERVAL\_SELECTION'dir.



(a) Tek seçim



(b) Tek aralıklı seçim



(c) Çoklu aralıklı seçim

Şekil 39.18

Bir liste üç seçim moduna sahiptir.

selectionModel özelliği, liste seçimlerini takip eden bir nesneyi belirtir. JList'in iki modeli vardır: bir liste modeli ve bir liste-seçim modeli. Liste modelleri veri yönetimi ile ilgilenirken, liste-seçim modelleri veri seçimleri ile ilgilenir. Bir liste-seçim modeli, üç seçim modunu (SINGLE\_SELECTION) tanımlayan ListSelectionModel arayüzünü uygulamalıdır.

SINGLE\_INTERVAL\_SELECTION ve MULTIPLE\_INTERVAL\_SELECTION), ve ListSectionListener için kayıt yöntemleri. Ayrıca, seçim aralıklarını ekleme ve kaldırma yöntemlerini tanımlar ve seçim modu,

anchorSelectionIndex, leadSelectionIndex ve valueIsAdjusting gibi özellikler için erişim yöntemlerini tanımlar.

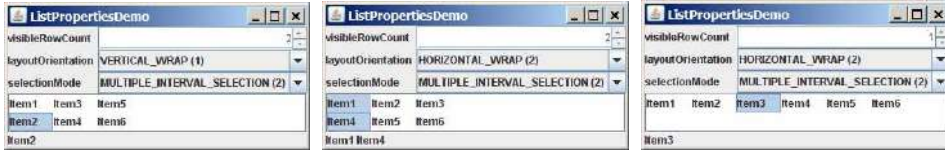
Varsayılan olarak, bir JList örneği DefaultListSelectionModel kullanır, bu da ListSelectionModel'in somut bir uygulamasıdır.

Genellikle, özel bir liste seçimi sağlamanız gerekmez

model, çünkü DefaultListSelectionModel sınıfı çoğu durumda yeterlidir. Liste seçim modelleri nadiren açıkça kullanılır, çünkü seçim modunu doğrudan JList içinde ayarlayabilirsiniz.

#### 39.7.4 Örnek: Liste Özellikleri Demo

Bu örnek, sabit sayıda öğeyi dize olarak görüntüleyen bir liste oluşturur. Örnek, görünürSıraSayısını bir spinner'dan, yerleşimYönünü bir combo kutusundan dinamik olarak ayarlamak için kullanılır; bu, Şekil 39.19'da gösterilmiştir. Bir veya daha fazla öğe seçtiğinizde, değerleri listenin altındaki bir durum etiketinde görüntülenir. Örneğin kaynak kodu, Liste 39.7'de verilmiştir.



Şekil 39.19

Bir listede visibleRowCount,

layoutOrientation ve selectionMode özelliklerini dinamik olarak ayarlayabilirsiniz.

#### Liste 39.7 ListPropertiesDemo.java

<margin note line 7: list>  
<margin note line 9: spinner>  
<margin note line 11: combo box>  
<margin note line 14: combo box>  
<margin note line 22: UI oluştur>  
<margin note line 53: spinner dinleyici>  
<margin note line 61: combo box dinleyici>  
<margin note line 69: combo box dinleyici>  
<margin note line 77: liste dinleyici>  
<margin note line 91: ana yöntem atlandı>

```
1 java.awt.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.event.*'i içe aktarın;
5
6 public class ListPropertiesDemo extends JApplet {
7 özel JList<String> jlist = new JList<String>(new String[] {
8 "Öğ1", "Öğ2", "Öğ3", "Öğ4", "Öğ5", "Öğ6"});
9 özel JSpinner jspVisibleRowCount =
10 yeni JSpinner(yeni SpinnerNumberModel(8, -1, 20, 1));
11 özel JComboBox jcbLayoutOrientation =
12 yeni JComboBox(yeni String[] {"DİKEY (0)",
13 "DİKEY_SARMA (1)", "YATAY_SARMA (2)"});
14 özel JComboBox jcbSelectionMode =
15 yeni JComboBox(yeni String[] {"TEK SEÇİM (0)",
16 "TEK ARALIK SEÇİMİ (1)",
17 "BİRDEN FAZLA ARALIK SEÇİMİ (2)"});
18 özel JLabel jlbDurum = new JLabel();
19
20 /** Applet'i oluştur */
```

```

21 genel Liste Özellikleri Demo() {
22 // Etiketleri bir panelde yerleştir
23 JPanel panel1 = new JPanel();
24 panel1.setLayout(new GridLayout(3, 1));
25 panel1.add(new JLabel("görünürSadırSayısı"));
26 panel1.add(new JLabel("yerleşimYönü"));
27 panel1.add(new JLabel("seçimModu"));
28
29 // Metin alanlarını bir panelde yerleştir
30 JPanel panel2 = new JPanel();
31 panel2.setLayout(new GridLayout(3, 1));
32 panel2.add(jspGörünürSadırSayısı);
33 panel2.add(jcboYerleşimYönü);
34 panel2.add(jcboSeçimModu);
35
36 // panel1 ve panel2'yi yerleştir
37 JPanel panel3 = new JPanel(); n 38 panel3
38 .setLayout(new BorderLayout(t5t,t 5t));n 39 panel3.add(pane
39 l1, BorderLayout.WEST); n 40 panel3.add(panel2, BorderL
40 ayout.CENTER);
41
42 // elemanları applet içinde yerleştir
43 add(panel3, BorderLayout.NORTH); n 44 add(tn
44 ewt JScrollPane(jlst, BorderLayout.CENTER); n 45 add(jlblStatus, Borde
45 rLayout.SOUTH);
46
47 // başlangıç özellik değerlerini ayarla
48 jlst.setFixedCellWidth(t50t);n 49 jlst.setFi
49 xedCellHeight(t20t);n 50 jlst.setSelectionMod
50 e(ListSelectionModel.SINGLE_SELECTION); n 51
52
53 // Dinleyicileri kaydet
53 jspVisibleRowCount.addChangeListener(tnewt ChangeListener() { n 54 @Override
55
56 public voidt stateChanged(ChangeEvent e) {
56 jlst.setVisibleRowCount(n 57 ((Integer)jsp
57 VisibleRowCount.getValue()).intValue()); n 58 }
58
59 });
60
61 jcboLayoutOrientation.addActionListener(tnewt ActionListener() { n 62 @Override
63
64 public void actionPerformed(ActionEvent e) {
64 jlst.setLayoutOrientation(n 65 jcboLayoutOri
65 entation.getSelectedIndex()); n 66 }
66
67 });
68
69 jcboSelectionMode.addActionListener(tnewt ActionListener() { n 70 @Override
71
72 public void actionPerformed(ActionEvent e) {
72 jlst.setSelectionMode(n 73 jcboSelectio
73 nMode.getSelectedIndex()); n 74 }
74
75 });
76
77 jlst.addListSelectionListener(tnewt ListSelectionListener() { n 78 @Override
79
80 public voidt valueChanged(ListSelectionEvent e) {

```

```

80 Object[] deęerleri = jlst.getSelectedValues(); n 81 Dizi gsterimi = t
""t;
82
83 fort (tintt i = t 0t; i < deęerler.uzunluk; i++) {
84 ekran += (Dize)deęerler[i] + t " "t;n 85 }

86
87 lblStatus.setText(grnt); n 88 }

89 });
90 }
91 }

```

Bir JList, altı string deęeri ile oluřturulur (satırlar 7-8). Bir JSpinner, başlangıç deęeri 8 olan bir SpinnerNumberModel kullanılarak oluřturulur, minimum deęer -1, maksimum deęer 20 ve adım 1 (satırlar 9-10). Bir JComboBox, dzen ynn seęmek iin VERTICAL (0), VERTICAL\_WRAP (1) ve HORIZONTAL\_WRAP (2) string deęerleri ile oluřturulur (satırlar 11-13). Bir JComboBox, seęim modunu seęmek iin SINGLE\_SELECTION (0), INTERVAL\_SELECTION (1) ve MULTIPLE\_INTERVAL\_SELECTION (2) string deęerleri ile oluřturulur (satırlar 14-17). Seęilen elemanları listede gstermek iin bir JLabel oluřturulur (satır 18).

Bir JList kaydırmayı desteklemez. Kaydırılabilir bir liste oluřturmak iin, bir JScrollPane oluřturun ve iine bir JList rneęi ekleyin (satır 44) .

Sabitleyici liste hcre geniřlięi ve ykseklięi satırlarda 48-49 belirtilmiřtir. Varsayılan seęim modu oklu aralık seęimidir. 50. satır seęim modunu tek seęim olarak ayarlar.

Spinner'dan yeni bir grnr satır sayısı seildięinde, setVisibleRowCount yntemi sayıyı ayarlamak iin kullanılır (53-58. satırlar). Yeni bir dzen ynelimi JComboBox aılır kutusundan seildięinde, setLayoutOrientation yntemi dzen ynelimini ayarlamak iin kullanılır (satırlar 60-65). Dikey, Dikey\_Sar ve

HORIZONTAL\_WRAP 0, 1 ve 2'dir, bu ęelerin combo kutusundaki indeks deęerlerine karřılık gelir. JComboBox combo kutusundan yeni bir seęim modu seildięinde, setSelectionMode yntemi seęim modunu ayarlamak iin kullanılır (satırlar 67-72). SINGLE\_SELECTION,

SINGLE\_INTERVAL\_SELECTION ve MULTIPLE\_INTERVAL\_SELECTION iin sabit deęerlerin 0 olduęunu unutmayın, 1 ve 2, bu ęelerin kombinasyon kutusundaki indeks deęerine karřılık gelir.

JList, seęimleri dinleyicilere bildirmek iin javax.swing.event.ListSelectionEvent tetikler. Dinleyici, olayı iřlemek iin valueChanged iřleyicisini uygulamalıdır. Kullanıcı listede bir ęe setięinde, valueChanged iřleyicisi alıřtırılır, bu da seęilen ęeleri alır ve tm ęeleri etiket zerinde grntler (satırlar 74-85).

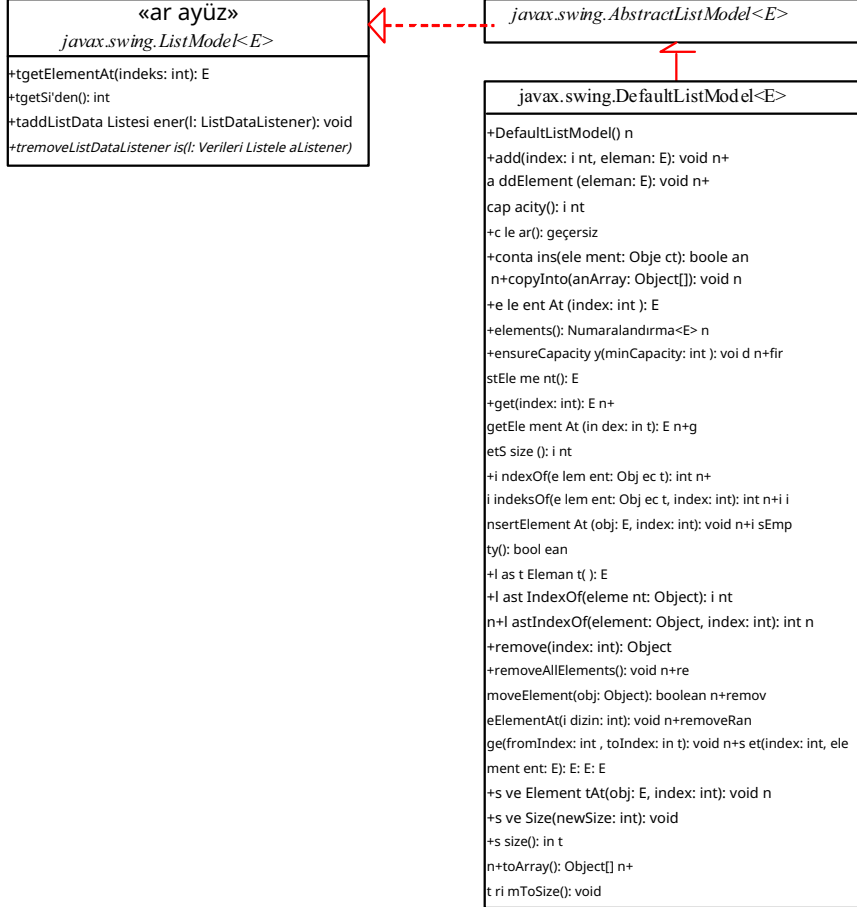
### 39.8 Liste Modelleri

nceki rnek, sabit bir dizi string ile bir liste oluřturur. Listeye yeni ęeler eklemek veya silmek istiyorsanız

mevcut öğeler, bir liste modeli kullanmalısınız. Bu bölüm liste modellerini tanıtmaktadır.

JList sınıfı, veri modeline veri depolama ve düzenleme sorumluluklarını devreder. JList sınıfının kendisi, listeye öğe ekleme veya kaldırma yöntemlerine sahip değildir. Bu yöntemler, Şekil

39.20'de gösterildiği gibi ListModel'de desteklenmektedir.



Şekil 39.20

ListModel, bir listede verileri depolar ve yönetir.

Tüm liste modelleri, ListDataEvent için kayıt yöntemlerini tanımlayan ListModel arayüzünü uygular. Liste öğeleri değiştirildiğinde ListDataListener örnekleri bilgilendirilir. ListModel ayrıca getSize ve getElementAt yöntemlerini tanımlar. getSize yöntemi listenin uzunluğunu döndürür ve getElementAt yöntemi belirtilen indeksteki öğeyi döndürür.



AbstractListModel, ListModel ve Serializable arayüzlerini uygular. AbstractListModel, ListModel'deki kayıt yöntemlerini uygular, ancak getSize ve getElementAt yöntemlerini uygulamaz.

DefaultListModel, AbstractListModel'den türetilir ve AbstractListModel tarafından uygulanmayan getSize ve getElementAt yöntemlerini uygular.

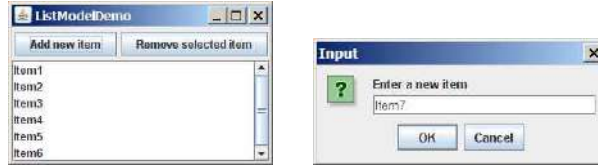
DefaultListModel'deki yöntemler, java.util.Vector sınıfındaki yöntemlere benzer. Bir öğeyi listeye eklemek için add yöntemini, bir öğeyi listeden kaldırmak için remove yöntemini, listeyi temizlemek için clear yöntemini, listedeki öğe sayısını döndürmek için getSize yöntemini ve bir öğeyi almak için getElementAt yöntemini kullanırsınız. Aslında, DefaultListModel verileri, esasen boyutu değiştirilebilen bir dizi olan Vector örneğinde depolar. Swing bileşenleri, Java Koleksiyonları Çerçevesi'nden önce geliştirilmiştir. Gelecek uygulamalarda, Vector java.util.ArrayList ile değiştirilebilir.

NOT

**<kenar notu: varsayılan liste modeli>**

Çoğu durumda, bir model belirtmeden bir Swing GUI nesnesi oluşturduğunuzda, varsayılan model sınıfının bir örneği oluşturulur. Ancak bu, JList için geçerli değildir. Varsayılan olarak, JList'teki model özelliği DefaultListModel'ın bir örneği değildir. Bir liste modeli kullanmak için, açıkça DefaultListModel kullanarak bir tane oluşturm alırsınız.

Liste 39.8, bir liste modeli kullanarak bir liste oluşturan ve kullanıcının listede öğe eklemesine ve silmesine izin veren bir örnek vermektedir; bu, Şekil 39.21'de gösterilmiştir. Kullanıcı yeni öğe ekle butonuna tıkladığında, yeni bir öğe almak için bir giriş iletişim kutusu görüntülenir.



Şekil 39.21

Liste modellerini kullanarak bir listede öğe ekleyebilir ve öğe çıkarabilirsiniz.

**Liste 39.8 ListModelDemo.java**

**<kenar notu satır 6: liste modeli>**

**<kenar notu satır 8: liste>**

**<kenar notu satır 15: öğe ekle>**

**<kenar notu satır 30: buton dinleyici>**

**<kenar notu satır 44: buton dinleyici>**

**<kenar notu satır 51: ana yöntem atlandı>**

- 1 java.aut.\*'u içe aktarın;
- 2 java.awt.event.\*'i içe aktarın;
- 3 javax.swing.\*'i içe aktarın;

```

4
5 public classt ListModelDemo t extendst JApplet {
6 Privatet DefaultListModel<String> listModel
7 = yeni DefaultListModel<String>();
8 private JList<String> jlst = yeni JList<String>(listModel);
9 private JButton jbtAdd = yeni JButton("Yeni öge ekle");
10 özel JButton jbtRemove = yeni JButton("Seçilen ögeyi kaldır");
11
12 /** Applet'i oluştur */
13 genel ListModelDemo() {
14 // Liste modeline öğeler ekle
15 listModel.addElement("Öge1");
16 listModel.addElement("Öge2");
17 listModel.addElement("Öge3");
18 listModel.addElement("Öge4");
19 listModel.addElement("Öge5");
20 listModel.addElement("Öge6");
21
22 JPanel paneli = yeni JPanel();
23 panel.add(jbtAdd); n 24 panel
 .add(jbtRemove);
25
26 add(panel, BorderLayout.NORTH); n 27 ekle(
 yeni JScrollPane(jlst), BorderLayout.CENTER);
28
29 // Dinleyicileri kaydet
30 jbtAdd.addActionListener(yeni ActionListener() {
31 @Override
32 public void actionPerformed(ActionEvent e) {
33 Dize yeniÖge =
34 JOptionPane.showInputDialog("Yeni bir öge girin");
35
36 if (yeniÖge != t nullt)
37 if (jlst.getSelectedIndex() == -1t)
38 listModel.addElement(newItem);
39 başka
40 listModel.add(jlst.getSelectedIndex(), newItem); n 41 }
42
43 });
44 jbtRemove.addActionListener(tnewt ActionListener() { n 45 @Override
46 public void actionPerformed(ActionEvent e) {
47 listModel.remove(jlst.getSelectedIndex()); n 48 }
49
50 }
51 }

```

Program, liste verilerini manipüle etmek için DefaultListModel'ın bir örneği olan listModel'i oluşturur (satır 6). Model, listedeki öğeleri eklemenizi ve çıkarmanızı sağlar.

Liste modeli kullanılarak bir liste oluşturulur (satır 7). Başlangıç elemanları, addElement yöntemi kullanılarak modele eklenir (satırlar 13-19).

Bir öğe eklemek için kullanıcı, *Yeni öge ekle* butonuna tıklayarak bir giriş iletişim kutusu görüntüler. İletişim kutusuna yeni bir öğe yazın.

Yeni öğe, listede şu anda seçili olan öğeden önce (38. satır) eklenir. Hiçbir öğe seçilmemişse, yeni öğe (36. satır) listeye eklenir.

Bir öğeyi kaldırmak için kullanıcı, öğeyi seçmeli ve ardından *Seçili öğeyi kaldır* butonuna tıklamalıdır. Sadece ilk seçili öğe kaldırılır. Tüm seçili öğeleri kaldıracak şekilde programı değiştirebilirsiniz (bkz. Alıştırma 39.6).

Seçili öğeyi kaldır butonuna tıklarsanız ancak şu anda hiçbir öğe seçilmemişse ne olur? Bu bir hata oluşturur.

Bunu düzeltmek için, Alıştırma 39.6'ya bakın.

### 39.9 Liste Hücresi Görüntüleyici

Önceki örnek, öğeleri bir listede dizgeler olarak gösterir. JList çok esnek ve çok yönlüdür ve basit metnin yanı sıra görüntüleri ve GUI bileşenlerini göstermek için de kullanılabilir. Bu bölüm, grafiklerin gösterimi için liste hücresi render'larını tanıtmaktadır.

Liste modeline veri depolama ve işleme görevini devretmenin yanı sıra, JList liste hücrelerinin render edilmesini liste hücre render edicilerine devreder. Tüm liste hücre render edicileri, aşağıdaki gibi tek bir yöntemi tanımlayan ListCellRender erer arayüzünü uygular:

```
public Bileşen getListCellRendererComponent(JList list, Object value, int index, boolean isSelected, boolean cellHasFocus)
```

Bu yönteme bir liste, hücre ile ilişkili değer, değer indexi ve değerin seçili olup olmadığı ile hücrenin odaklanıp odaklanmadığına dair bilgiler iletilir. Yöntemden dönen bileşen, listede hücre üzerinde boyanır. Varsayılan olarak, JList hücrelerini render etmek için DefaultListCellRenderer kullanır. DefaultListCellRenderer sınıfı

ListCellRenderer'ı uygular, JLabel'den türetilir ve ya bir dize ya da bir simge gösterebilir, ancak aynı hücrede her ikisini birden gösteremez.

Örneğin, JList'in varsayılan hücre render edicisini dizeleri göstermek için kullanabilirsini z; bu, Şekil 39.22(a)'de gösterildiği gibi, aşağıdaki kodu kullanarak yapılabilir:

```
JList list = new JList(new String[]{"Danmark", "Almanya", "Çin", "Hindistan", "Norveç", "İngiltere", "ABD"});
```



(a) Dize



(b) Simgeler



(c) Simgeler ve dizeler

Şekil 39.22

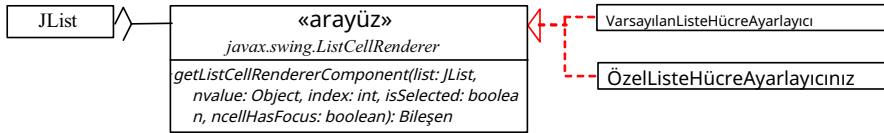
*Hücre renderleyici, bir listede liste öğelerini görüntüler.*

Şekil 39.22(b)'de gösterildiği gibi simgeleri görüntülemek için JList'in varsayılan hücre renderleyicisini kullanabilirsiniz, aşağıdaki kodu kullanarak:

```
ImageIcon denmarkIcon = yeni ImageIcon(getClass().getResource(
 "image/danmarkIcon.gif"));n...
```

```
JList listesi = t newt JList(tnewt ImageIcon[]{denmarkIcon, germanyIcon, n china
 Icon, indiaIcon, norwayIcon, ukIcon, usIcon});
```

Şekil 39.22(c)'de gösterildiği gibi bir hücrede bir simge ile birlikte bir dizeyi nasıl görüntülersiniz? ListCellRenderer'ı uygulayarak özel bir renderleyici oluşturmanız gerekir, Şekil 39.23'te gösterildiği gibi.



**Şekil 39.23**

*ListeHücreAyarlayıcı, hücrelerin bir listede nasıl görüntüleneceğini tanımlar.*

Bir listenin aşağıdaki gibi oluşturulduğunu varsayalım:

```
JList liste = yeni JList (yeni Object[]{{danimarkaSimgesi, "Danimarka"},
 {almanyaSimgesi, "Almanya"}, {cinaSimgesi, "Çin"},
 {hindistanSimgesi, "Hindistan"}, {norveçSimgesi, "Norveç"}, {ingiltereSimgesi, "İngiltere"},
 {abdSimgesi, "ABD"} });
```

Listedeki her öğe, bir simge ve bir string içeren bir dizi. Özel bir hücre renderleyici oluşturabilir ve bu renderleyici, liste veri modelinden bir simge ve bir string olarak bunları bir label'da görüntüleyebilir. Özel hücre renderleyici sınıfı, Liste 39.9'da verilmiştir.

### Liste 39.9 MyListCellRendererer.java

*<margin note line 6: hücre bileşeni>*  
*<margin note line 18: simge ayarla>*  
*<margin note line 19: metni ayarla>*  
*<margin note line 22: hücre seçildi>*  
*<margin note line 32: renderleme hücrelerini döndür>*

```
1 java.aut.*'u içe aktarın;
2 javax.swing.*'i içe aktarın;
3 javax.swing.border.*'ı içe aktarın;
4
5 genel sınıf MyList CellRenderer t, List CellRenderer'ı uygular {
6 özel JLabel jlb Hücre = t newt JLabel(t" "t, JLabel.LEFT);
7 özel Sınır çizgisi Sınır =
8 BorderFactory.createLineBorder(Color.black, t 1t);
9 Privatet Border emptyBorder = n 1
0 BorderFactory.createEmptyBorder(t2t,t 2t,t 2t,t 2t);
11
12 /** Bu yöntemi ListCellRenderer'da uygulayın */
13 public Bileşen getListCellRendererComponent
14 (JList liste, Object değer, int indeks, boolean seçiliMi,
```

```

15 boolean hücreOdaklı) {
16 Object[] çift = (Object[])değer; // Değeri bir diziye dönüştür
17 jlblCell.setOpaque(true);
18 jlblCell.setIcon((ImageIcon)çift[0]);
19 jlblCell.setText(çift[1].toString());
20
21 eğer (seçili) {
22 jlblCell.setForeground(list.getSelectionForeground()); n 23 jlblCell.setBackgro
und(list.getSelectionBackground()); n 24 }

25 aksi {
26 jlblCell.setForeground(list.getForeground()); n 27 jlblCell.setBackg
round(list.getBackground()); n 28 }

29
30 jlblCell.setBorder(cellHasFocus lineBorder : emptyBorder);
31
32 nlb Hücrelerini döndür;
33 }
34 }

```

MyListCellRenderer sınıfı, ListCellRenderer arayüzündeki `getListCellRendererComponent` yöntemini uygular. Bu yöntem, list, value, index, isSelected ve isFocused (satırlar 13-15) parametreleri ile çağrılır. Değer mevcut öğe değerini temsil eder. Bu durumda, iki öğeden oluşan bir dizi vardır. İlk öğe bir resim simgesidir (satır 18). İkinci öğe bir dizedir (satır 19). Hem resim simgesi hem de dize bir etiket üzerinde görüntülenir. `getListCellRendererComponent` metodu, listedeki hücrede boyanan etiketi döndürür (satır 32),

Hücre seçilmişse, hücrenin arka planı ve ön planı listenin seçim arka planı ve ön planı olarak ayarlanır (satır 22-23). Hücre odaklanmışsa, hücrenin kenarlığı çizgi kenarlığı olarak ayarlanır (satır 30); aksi takdirde, boş kenarlık olarak ayarlanır (satır 30). Boş kenarlık,

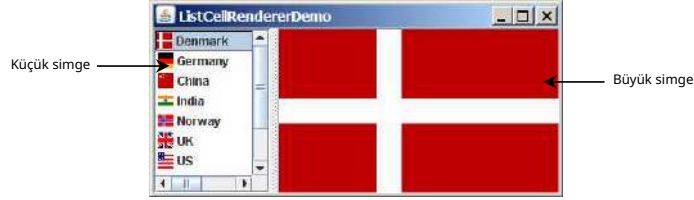
hücreler arasında bir ayırıcı görevi görür.

NOT

**<margin note: herhangi bir GUI renderleyici>**

Liste 39.9'daki örnek, bir JLabel'i renderleyici olarak kullanmaktadır. `getListCellRendererComponent` yönteminden döndürülen herhangi bir GUI bileşenini renderleyici olarak kullanabilirsiniz.

Bir ülke listesi oluşturan ve her ülkenin bayrak resmini ve adını listenin bir öğesi olarak gösteren bir örnek geliştirelim (Liste 39.10), Şekil 39.24'te gösterildiği gibi. Listede bir ülke seçildiğinde, bayrağı listenin yanındaki bir etiket içinde görüntülenir.



**Şekil 39.24**

*Resim ve metin liste hücresinde görüntülenir.*

**Liste 39.10 ListCellRendererDemo.java**

*<margin note line 7: ulus dizeleri>  
 <margin note line 9: küçük simgeler>  
 <margin note line 10: büyük simgeler>  
 <margin note line 13: liste modeli>  
 <margin note line 16: liste>  
 <margin note line 19: liste hücresi renderleyici>  
 <margin note line 22: bölünmüş panel>  
 <margin note line 25: resim etiketi>  
 <margin note line 31: resim simgelerini yükle>  
 <margin note line 33: öğeleri ekle>  
 <margin note line 35: resim simgelerini yükle>  
 <margin note line 40: renderleyiciyi ayarla>  
 <margin note line 49: liste dinleyicisi>  
 <margin note line 55: ana yöntem atlandı>*

```

1 javax.swing.*'i içe aktarın;
2 javax.swing.event.*'i içe aktarın;
3 java.aut.*'u içe aktarın;
4
5 public classt ListCellRendererDemo t extendst JApplet {
6 özel final statik intt NUMBER_OF_NATIONS = t7t;
7 özel String[] uluslar = yeni String[]
8 { "Danimarka", "Almanya", "Çin", "Hindistan", "Norveç", "İngiltere", "ABD" };
9 özel ImageIcon[] simgeler = yeni ImageIcon[NUMBER_OF_NATIONS];
10 özel ImageIcon[] büyükSimgeler = yeni ImageIcon[NUMBER_OF_NATIONS];
11
12 // Bir liste modeli oluştur
13 özel DefaultListModel listeModeli = yeni DefaultListModel();
14
15 // Liste modelini kullanarak bir liste oluştur
16 özel JList jlstUluslar = yeni JList(listeModeli);
17
18 // Bir liste hücresi oluşturucu oluştur
19 özel ListCellRenderer oluşturucu = yeni MyListCellRenderer();
20
21 // Bir bölünmüş panel oluştur
22 özel JSplitPane jSplitPane 1 = t newt JSplitPane();
23
24 // Görüntü göstermek için bir etiket oluştur
25 özel JLabel jlblImage = t newt JLabel(t""t, JLabel.CENTER);
26
27 /** ListCellRenderer'ı oluştur */
28 genel Liste Hücresi Oluşturucu Demosu() {
29 // Küçük ve büyük resim simgelerini yükle
30 kale (tintt i = t 0t; i < NUMBER_OF_NATIONS; i++) {
31 simge[i] = t newt ImageIcon(getClass().getResource(
32 "/image/flagIcon"t + i + t".gif"t));
33 listModel.addElement(tnewt Object[] {icons[i], uluslar[i]});
34 }

```

```

35 bigIcons[i] = new ImageIcon(getClass().getResource(
36 "/image/flag" + i + t".gif");
37 }
38
39 // Liste hücresi render'ını ayarla
40 jlstNations.setCellRenderer(oluşturucu); n 41 jlstNations.setPreferredSize(new Dimension(t200t,t 200t)); n 42 jSplitPane1.setLeftComponent(new JScrollPane(jlstNations)); n 43 jSplitPane1.setRightComponent(jlblImage);
44 jlstNations.setSelectedIndex(t0t); n 45 jlblImage.setIcon(bigIcons[t0t]); n 46 add(jSplitPane1, BorderLayout.CENTER);
47
48 // Dinleyiciyi kaydet
49 jlstNations.addListSelectionListener(new ListSelectionListener() {
50 public void valueChanged(ListSelectionEvent e) {
51 jlblImage.setIcon(bigIcons[jlstNations.getSelectedIndex()]); n 52 }
53 }); n 54 } n
55 }

```

Bu programda iki tür simge kullanılmaktadır. Küçük simgeler, flagIcon0.gif, ..., flagIcon6.gif dosyalarından oluşturulmuştur (satırlar 31-32). Bu resim dosyaları Da nimarka, Almanya, Çin, Hindistan, Norveç, Birleşik Krallık ve ABD'nin bayraklarıdır. Küçük simgeler liste içinde görüntülenir. Aynı ülkeler için büyük simgeler, flag0.gif, ..., flag6.gif dosyalarından oluşturulmuştur (satırlar 35-36). Büyük simgeler, bölünmüş panelin sağ tarafındaki bir etiket üzerinde görüntülenir.

ListCellRendererDemo sınıfı bir liste modeli oluşturur (satır 13) ve eşleri modele ekler (satır 33). Her bir öğe, bir dizi iki elemandan oluşur (görüntü simgesi ve dize). Liste, liste modelini kullanarak oluşturulur (satır 16). Liste hücresi oluşturucu (satır 19) oluşturulur ve listeye (satır 40) iliştilir.

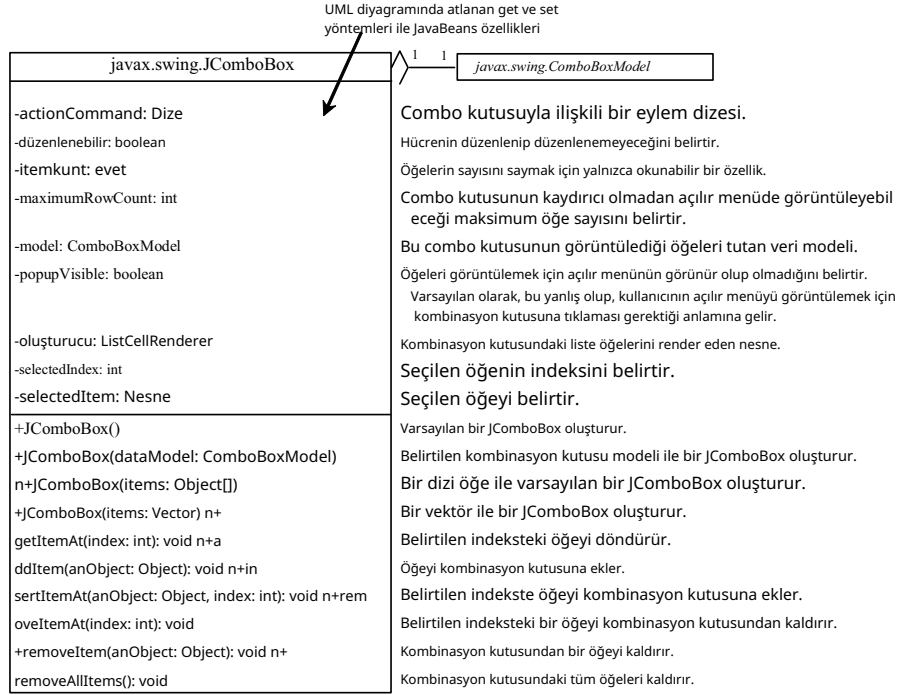
ListCellRendererDemo sınıfı bir bölünmüş panel oluşturur (satır 22) ve listeyi solda (satır 42) ve bir etiketi sağda (satır 43) yerleştirir.

Listeden bir ülke seçtiğinizde, liste seçim olayı handler'ı çağırılır (satırlar 49-53) ve bölünmüş panelin sağ tarafındaki etikete yeni bir görüntü ayarlamak için kullanılır (satır 51).

### 39.10 JComboBox ve Modelleri

JComboBox'ın temel özellikleri §17.8, "Kombinasyon Kutuları" bölümünde, kombinasyon kutusu modelleri kullanılmadan tanıtılmıştır. Bu bölüm, kombinasyon modellerini tanıtır ve JComboBox'ın kullanımını bazı detaylarla tartışır.

Bir kombinasyon kutusu bir listeye benzer. Kombinasyon kutuları ve listeler, bir listeden öğe seçmek için kullanılır. Bir kombinasyon kutusu, kullanıcının bir seferde bir öğe seçmesine izin verirken, bir liste birden fazla seçime izin verir. Bir kombinasyon kutusu seçildiğinde, bir açılır menüde bulunan bir açılır listeyi gösterir. Seçilen öğe, bir metin alanıymış gibi hücrede düzenlenebilir. Şekil 39.25, JComboBox'ın öğişkenlerini ve yapılarını göstermektedir. Bir kombinasyon kutusunun verileri ComboBoxModel'de saklanır. Bir kombinasyon kutusunu bir kombinasyon kutusu modelinden, nesne dizisinden veya bir vektörden oluşturabilirsiniz.

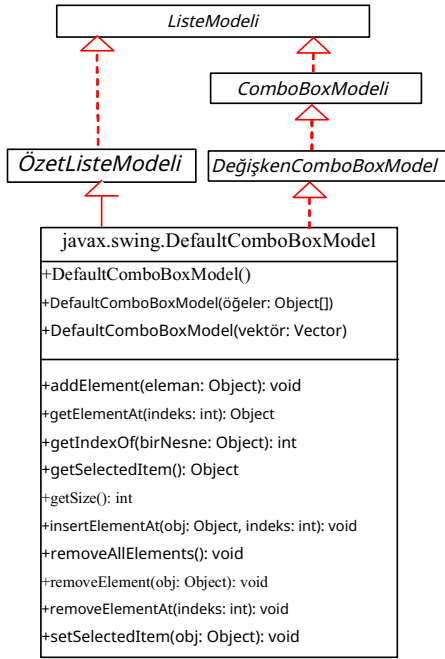


## Şekil 39.25

JComboBox, öğeleri bir listede görüntüler.

JComboBox, verileri saklama ve düzenleme sorumluluklarını veri modeline devreder. Tüm kombinasyon kutusu modelleri ComboBoxModel arayüzünü uygular, bu arayüz ListModel arayüzünü genişletir ve seçilen öğeyi almak ve ayarlamak için getSelectedItem ve setSelectedItem yöntemlerini tanımlar. Öğeleri ekleme ve kaldırma yöntemleri MutableComboBoxModel arayüzünde tanımlanmıştır, bu arayüz ComboBoxModel'i genişletir. JComboBox'ın bir örneği, açıkça bir model belirtilmeden oluşturulduğunda, DefaultComboBoxModel'in bir örneği kullanılır. DefaultComboBoxModel sınıfı AbstractListModel'i genişletir ve MutableComboBoxModel'i uygular, Şekil 39.26'da gösterildiği gibi.





**Şekil 39.26**

ComboBoxModel, bir açılır kutuda verileri depolar ve yönetir.

Genellikle, JComboBox'un (getItemAt, getSelectedItem ve getSelectedIndex) almak, (addItem

ve

insertItemAt) eklemek ve (removeItem, removeItemAt ve removeAllItems) listeden öğeleri kaldırmak için yöntemleri içerdiğinden, açılır kutu modellerini açıkça kullanmanıza gerek yoktur.

JComboBox, birçok diğer olayın yanı sıra ActionEvent ve ItemEvent tetikleyebilir. Yeni bir öğe seçildiğinde, JComboBox ItemEvent'i iki kez tetikler; bir kez önceki seçili öğeyi seçimi kaldırmak için ve diğerini de şu anda seçili olan öğeyi seçmek için.

JComboBox, bir ItemEvent oluşturduktan sonra bir ActionEvent tetikler.

Combo kutuları, combo

kutu öğeleri bir açılır menüde bulunan bir listede görüntülediği için hücreleri tam olarak listeler gibi render eder.

Bu nedenle, bir combo kutusu hücre render'ı, ListCellRenderers arayüzünü uygularak tam olarak bir liste hücre render'ı gibi oluşturulabilir. JList gibi, JComboBox'ın gösterdiği varsayılan bir hücre render'ı vardır; bu, bir dize veya bir simgeyi gösterir, ancak aynı anda her ikisini de göstermez. Bir dize ve bir simgenin kombinasyonunu göstermek için, özel bir render oluşturmanız gerekir. Özel liste hücre render'ı

Listing 39.9'daki MyListCellRenderer, herhangi bir değişiklik olmadan bir combo kutusu hücre render'ı olarak kullanılabilir.

Listing 39.11, her ülkenin bayrak resmini ve adını bir öğe olarak göstermek için bir combo kutusu oluşturan bir örnek verir; bu, Şekil 39.27'de gösterilmiştir. Listedeki bir ülke seçildiğinde, bayrağı combo kutusunun altında bir panelde görüntülenir.



**Şekil 39.27**

*Resim ve metin, bir combo kutusunun liste hücresinde görüntülenir.*

### **Listeleme 39.11 ComboBoxCellRendererDemo.java**

<margin note line 7: ulus dizeleri>  
 <margin note line 9: küçük simgeler>  
 <margin note line 10: büyük simgeler>  
 <margin note line 13: combo kutusu modeli>  
 <margin note line 16: combo kutusu>  
 <kenar notu satır 19: liste hücresi render'ı>  
 <kenar notu satır 22: resim etiketi>  
 <kenar notu satır 28: resim simgelerini yükle>  
 <kenar notu satır 30: öğeleri ekle>  
 <kenar notu satır 32: resim simgelerini yükle>  
 <kenar notu satır 37: render'ı ayarla>  
 <kenar notu satır 43: eylem dinleyici>  
 <kenar notu satır 50: ana yöntem atlandı>

```

1 java.awt.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4
5 genel sınıf ComboBoxCellRendererDemo t extendst JApplet {
6 özel final statik intt NUMBER_OF_NATIONS = t7t;
7 private String[] nations = new String[] {"Danimarka",
8 "Almanya", "Çin", "Hindistan", "Norveç", "İngiltere", "ABD"};
9 özel ImageIcon[] simgeler = yeni ImageIcon[NUMBER_OF_NATIONS];
10 özel ImageIcon[] büyükSimgeler = yeni ImageIcon[NUMBER_OF_NATIONS];
11
12 // Bir combo kutu modeli oluştur
13 özel Varsayılan ComboBox Modeli model = t newt Varsayılan ComboBoxModel();
14
15 // Belirtilen model ile bir combo kutu oluştur
16 özel JComboBox jcb Ülkeler = newt JComboBox(model);
17
18 // Bir liste hücresi oluşturucu oluştur
19 özel MyListCellRenderer renderer = new MyListCellRenderer();
20
21 // Görüntü göstermek için bir etiket oluştur
22 özel JLabel jlblImage = t newt JLabel(t""t, JLabel.CENTER);
23
24 /** Applet'i oluştur */
25 genel ComboBoxCellRendererDemo() {
26 // Küçük ve büyük resim simgelerini yükle
27 kale (tintt i = t 0t; i < NUMBER_OF_NATIONS; i++) {
28 simge[i] = t newt ImageIcon(getClass().getResource(
29 "/image/flagIcon"t + i + t".gif"t));
30 model.addElement(tnewt Object[] {icons[i], uluslar[i]});
31
32 bigIcons[i] = tnewt ImageIcon(getClass().getResource(
33 "/image/flag"t + i + t".gif"t));

```

```

34 }
35
36 // Kombinasyon kutusu için liste hücresi render'ını ayarla
37 jcbCountries.setRenderer(oluşturucu); n 38 jlblImage
39 ge.setIcon(bigIcons[t0t]); n 39 add(jcbCountries
, java.awt.BorderLayout.NORTH); n 40 add(jlblImage, java.awt.BorderL
ayout.CENTER);
41
42 // Dinleyiciyi kaydet
43 jcbCountries.addActionListener(tnewt ActionListener() { n 44 @Override
45
46 public void actionPerformed(java.awt.event.ActionEvent e) {
47 jlblImage.setIcon(bigIcons[jcbCountries.getSelectedIndex()]); n 47 }
48 }); n 49 } n
50 }

```

Program, Listeleme 39.10'daki önceki örneğe çok benzer. Her ülke için iki tür resim simgesi yüklenir ve icons ve bigIcons dizilerine kaydedilir (satırlar 27-34). Kombinasyon kutusundaki her öğe, bir simge ve bir string içeren bir dizidir (satır 30).

Listeleme 39.9'da tanımlanan MyListCellRenderer, satır 19'da bir hücre oluşturucu oluşturmak için kullanılır. Hücre oluşturucu, satır 37'de kombinasyon kutusuna takılır.

Kombinasyon kutusundan bir ülke seçtiğinizde, eylem olayı hizmetçisi çağrılır (satırlar 44-46). Bu hizmetçi, etikette yeni bir resim ayarlar (satır 45).

#### Anahtar Terimler

- **denetleyici**
- **modeli**
- **MVC mimarisi**
- **görünüm**

#### Bölüm Özeti

1. Model-görünüm yaklaşımındaki temel sorun, görünüm ile model arasında tutarlılığı sağlamaktır. Modeldeki herhangi bir değişiklik, ağırlıklı olarak görünümde yansıtılmalıdır ve tüm görünüm aynı veriyi tutarlı bir şekilde göstermelidir. Model, uygun olay ve olay dinleyici kayıt yöntemleri ile bir kaynak olarak uygulanabilir. Görünüm bir dinleyici olarak uygulanabilir.

Böylece, modelde veriler değiştiğinde, görünüm bildirilecektir.

2. Her Swing kullanıcı arayüzü bileşeni (örneğin, JButton, JTextField, JList ve JComboBox) verilerini modeline atıfta bulunan *model* adında bir özelliğe sahiptir. Veri modeli, adı Model ile biten bir arayüzde tanımlanır (örneğin, SpinnerModel, ListModel, ListSelectionModel ve ComboBoxModel).
3. Çoğu basit Swing bileşeni (örneğin, JButton, JTextField, JTextArea) modellerinin bazı özelliklerini içerir ve bu

özelliklere, modelin varlığı bilinmeden doğrudan bileşenden erişilip değıştirilebilir.

4. Bir `JSpinner`, kullanıcının sıralı bir diziden sayıları, arihleri veya değeri seçmesini sağlayan sağ tarafında bir çift küçük ok butonuyla birlikte bir metin alanı olarak görüntülenir. Bir `JSpinner`'ın dizisi değeri, `SpinnerModel` arayüzü tarafından tanımlanır.

`AbstractSpinnerModel`, `SpinnerModel`'i uygulayan ve kayıt/iptal yöntemleri için uygulama sağlayan kullanışlı bir soyut sınıftır. `SpinnerListModel`,

`SpinnerNumberModel` ve `SpinnerDateModel`, `SpinnerModel`'in somut uygulamalarıdır. `SpinnerNumberModel`, maksimum, minimum ve adımBoyutu özelliklerine sahip bir sayı dizisini temsil eder. `SpinnerDateModel`, tarih dizisini temsil eder.

`SpinnerListModel`, herhangi bir nesne değeri listesini saklayabilir.

5. A `JSpinner`, *editor* olarak adlandırılan tek bir çocuk bileşene sahiptir, bu modelin mevcut öğesini veya değeri görüntülemekten sorumludur. Dört editör, `JSpinner` içinde statik iç sınıflar olarak tanımlanmıştır: `JSpinner.DefaultEditor`, `JSpinner.`

`NumberEditor`,

`JSpinner.DateEditor`, ve `JSpinner.ListEditor`.

6. `JList`, iki destekleyici modele sahiptir: bir liste modeli ve bir liste-seçim modeli. *liste modeli*, verileri saklamak ve işlemek için kullanılır. *liste-seçim modeli*, öğeleri seçmek için kullanılır. Varsayılan olarak, öğeler dizeler veya simgeler olarak görüntülenir. Ayrıca, `ListCellRenderer` arayüzünü uygulayan özel bir render oluşturabilirsiniz.

7. `JComboBox`, verileri saklama ve sürdürme sorumluluklarını veri modeli ne devreder. Tüm combo kutusu modelleri, `ListModel` arayüzünü genişleten ve seçilen bir öğeyi almak ve ayarlamak için `getSelectedItem` ve `setSelectedItem` yöntemlerini tanımlayan `ComboBoxModel` arayüzünü uygular. Öğeleri ekleme ve kaldırma yöntemleri, `ComboBoxModel`'i genişleten `MutableComboBoxModel` arayüzünde tanımlanmıştır.

`JComboBox`'ın bir örneği, açıkça bir model belirtilmeden oluşturulduğunda, `DefaultComboBoxModel`'in bir örneği kullanılır.

`DefaultComboBoxModel` sınıfı, `AbstractListModel`'i genişletir ve `MutableComboBoxModel`'i uygular.

8. Kombinasyon kutuları hücreleri tam olarak listeler gibi render eder, çünkü kombinasyon kutusu eşya öğeleri bir açılır menüde bulunan bir listede görüntülenir. Bu nedenle, bir kombinasyon kutusu hücre render'ı, `ListCellRenderer` arayüzünü uygulayarak tam olarak bir liste hücre render'ı gibi oluşturulabilir.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

## Gözden Geçirme Soruları

### Bölümler 39.2-39.3

39.1

Model-görünüm-denetleyici mimarisi nedir?

39.2

Modelleri, görünümleri ve denetleyicileri nasıl uyguluyorsunuz?

39.3

MVC mimarisinin varyasyonları nelerdir?

#### Bölüm 39.4

39.4

Her Swing GUI bileşeninin (JPanel gibi konteynerler hariç) model adında bir özelliği var mı? Modelin türü tüm bileşenler için aynı mı? Bileşenler için?

39.5

Her model arayüzünün varsayılan bir uygulama sınıfı var mı? Eğer öyleyse, Swing bileşeni bir model belirtilmediğinde varsayılan model sınıfını mı kullanır?

#### Bölümler 39.5-39.6

39.6

Bir veri modeli belirtmeden bir JSpinner oluştursanız, varsayılan model nedir?

39.7

SpinnerListModel'da verileri depolamak için kullanılan iç veri yapısı nedir? Bir diziyi listeye nasıl dönüştürsünüz?

#### Bölümler 39.7-39.9

39.8

JList'in bir listeye öğe eklemek için addItem gibi bir yöntemi var mı? Bir listeye öğe nasıl eklenir? JList, bir listede simgeleri ve özel GUI nesnelerini görüntüleyebilir mi? Bir liste öğesi düzenlenebilir mi? Bir listede verileri nasıl başlatırsınız? Kaydırma olmadan bir listede görünür satırların maksimum sayısını nasıl belirlersiniz? Bir liste hücre sinin yüksekliğini nasıl belirlersiniz? Liste hücrelerinin yatay kenar boşluğunu nasıl belirlersiniz?

39.9 Bir liste modeli nasıl oluştursunuz? Bir liste modeline öğe nasıl eklersiniz? Bir liste modelinden öğe nasıl çıkarırsınız?

39.10 Üç liste seçim modu nedir? Seçim modlarını doğrudan bir JList örneğinde ayarlayabilir misiniz? Seçilen öğe(ler)i nasıl elde edersiniz?

39.11 Özel bir liste hücresi render'ı nasıl tanımlarsınız?

39.12 ListSelectionEvent'i işlemek için hangi işleyici kullanılır?

#### Bölüm 39.10

39.13 Bir açılır kutudan birden fazla öğe seçilebilir mi? Bir açılır kutu öğesi düzenlenebilir mi? Kaydırma olmadan bir açılır kutuda görünür maksimum satır sayısını nasıl belirlersiniz? JComboBox'ta bir açılır kutu hücresinin yüksekliğini bir yöntemle belirleyebilir misiniz? Bir açılır kutudaki seçilen öğeyi nasıl elde edersiniz?

39.14 Bir açılır kutuya öğe nasıl ekler veya çıkarırsınız?

39.15 Bir açılır kutunun hücre render'ı neden bir liste için olanla aynıdır?

## Programlama Alıştırmaları

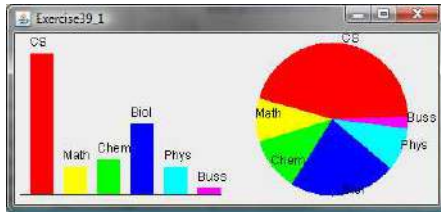
### Bölüm 39.2

39.1 \*\*\*

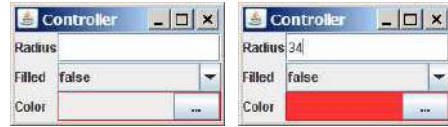
(MVC bileşenleri oluşturun) ChartModel adında bir model oluşturun, bu model double elemanlardan oluşan bir dizi olan data içinde veriler tutar ve stringlerden oluşan bir dizi olan dataName içinde verilerin isimlerini tutar. Örneğin, {200, 40, 50, 100, 40} olan kayıt verileri data dizisinde {"CS", "Math", "Chem", "Biol", "Phys"} ise dataName dizisindedir. Bu iki özellik, kendi

ilgili get yöntemleri, ancak bireysel set yöntemleri değil. Her iki özellik, setChart Data(String[] newDataName, double[] newData) yönteminde birlikte ayarlanır, böylece düzgün bir şekilde görüntülenebilirler. Verileri bir pasta grafiği olarak sunmak için PieChart adında bir görünüm oluşturun ve verileri bir çubuk grafiği olarak sunmak için BarChart adında bir görünüm oluşturun. Verileri bir çubuk grafiği olarak sunmak için, Şekil 39.28(a)'de gösterildiği gibi.

(İpucu: Her pasta toplam data'nın bir yüzdesini temsil eder. Pastayı, {Color.red, Color.yellow, Color.green, Color.blue, Color.cyan, Color.magenta, Color.orange, Color.pink, Color.darkGray} adlı bir diziden gelen renklerle renklendirin. i'inci pasta için colors[i % colors.length] kullanın. Veri adlarını görüntülemek için siyah renge kullanın.)



(a)



(b)

### Şekil 39.28

(a) İki görünüm, PieChart ve BarChart, ChartModel'den veri alır; (b) Eklips butonuna tıklamak, bir renk seçici diyalog kutusunu görüntüler.

39.2\*

(Revize Et Listing 39.3 CircleController.java) CircleController, yeni bir yarıçap almak için bir metin alanı ve dairenin dolu olup olmadığını belirtmek için bir Boolean değeri almak için bir açılır kutu kullanır.

Şekil 39.28(b)'de gösterildiği gibi, kullanıcıya renk seçme imkanı vermek için CircleController'a yeni bir satır ekleyin JColorChooser bileşenini kullanarak.

Yeni satır, Renk metni ile bir etiket, rengi göstermek için bir etiket ve bir eklips düğmesi içerir. Kullanıcı, JColorChooser iletişim kutusunu görüntülemek için eklips düğmesine tıklayabilir. Kullanıcı bir renk seçtiğinde, renk eklips düğmesinin solundaki etiketin arka planı olarak görüntülenir.

### Bölümler 39.5-39.6

39.3\*\*

(Spinner'ları senkronize et) Tarih spinner'ı, Liste 39.6'daki gün, ay ve yıl spinner'ları ile senkronizedir.

SpinnerModelEditorDemo.java. Gün, ay ve yıl spinner'larını tarih spinner'ı ile senkronize edecek şekilde geliştirin. Diğer bir deyişle, tarih spinner'ında yeni bir değer seçildiğinde, gün, ay ve yıl spinner'larındaki değerler buna göre güncellenir.

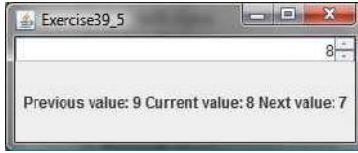
39.4★

(Özel spinner modeli) 2'nin kuvvetleri olan bir sayı dizisini temsil eden özel bir spinner modeli geliştirin; yani, 1, 2, 4, 8, 16, 32 ve devamı. Modeliniz AbstractSpinnerModel'i uygulamalıdır. ChangeListener için kayıt/kayıt iptali yöntemleri zaten AbstractSpinnerModel'de uygulanmıştır. getNextValue(), getPreviousValue(), getValue() ve setValue(Object) yöntemlerini uygulamanız gerekiyor.

39.5★

(Spinner'da görüntülenen sayıları ters çevir) Spinner'da görüntülenen sayılar, yukarı ok düğmesine tıklandığında artar ve aşağı ok düğmesine tıklandığında azalır. Yeni bir model oluşturarak diziyi tersine çevirebilirsiniz.

SpinnerNumberModel ve getNextValue ve getPreviousValue yöntemlerini geçersiz kılar. Şekil 39.29'da gösterildiği gibi yeni modeli kullanan bir test programı yazın.



**Şekil 39.29**

Spinner'daki sayılar azalan sıradadır.

### Bölümler 39.7-39.9

39.6★

(Listede seçilen öğeleri kaldır) Liste 39.8'i, ListModelDemo.java, aşağıdaki gereksinimleri karşılayacak şekilde değiştirin:

- Seçilen öğeleri kaldır butonuna tıklandığında, listeden tüm seçilen öğeleri kaldırın.
- Öğelerin DELETE tuşu kullanılarak silinmesine izin verin.

39.7★

(Özel liste hücresi işleyici) Liste 39.10, ListCellRendererDemo.java, her ülke için iki tür resim içerir. Küçük resimler listede görüntülemek için kullanılır ve büyük olanlar listeden dışarıda görüntülemek için kullanılır. Sadece büyük resimlerin mevcut olduğunu varsayın. Özel hücre işleyicisini, bir hücreyi işlemek için JLabel yerine bir JPanel kullanacak şekilde yeniden yazın. Her hücre bir resim ve bir dize içerir. Resmi bir ImageViewer'da ve dizeyi bir etiket içinde görüntüleyin. ImageViewer bileşeni, Liste 13.15'te tanıtılmıştır, ImageViewer.java. Resim bir ImageViewer'da uzatılabilir.

Bir ImageViewer'da. Bir görüntüleyicinin boyutunu 32 çarpı 32 olarak ayarlayın, Şekil 39.30'da gösterildiği gibi. Yeni özel hücre işleyicisini test etmek için Liste 39.10'u gözden geçirin.

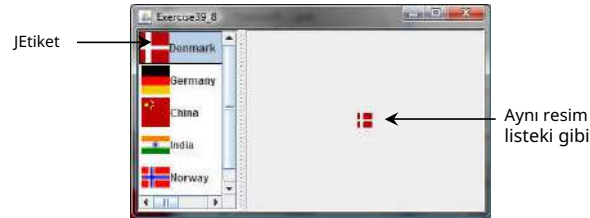


**Şekil 39.30**

*ImageViewer, listedeki resmi görüntülemek için kullanılır.*

39.8★

(Liste içindeki seçili öğeleri DELETE tuşunu kullanarak silin) Listeyi Değiştir 39.10, ListCe IIRendererDemo.java, DELETE tuşunu kullanarak listeden seçili öğeleri silmek için değiştirin. Listedeki bazı öğeler silindikten sonra, listede seçili bir öğenin indeksi, bigIcons dizisindeki öğenin indeksiyle eşleşmez. Sonuç olarak, bigIcons dizisindeki resim simgesini divideki sağ tarafta resmi görüntülemek için kullanamazsınız. Programı, listede seçili öğeden simgeyi alacak ve görüntülenecek şekilde revize edin, Şekil 39.31'de gösterildiği gibi.

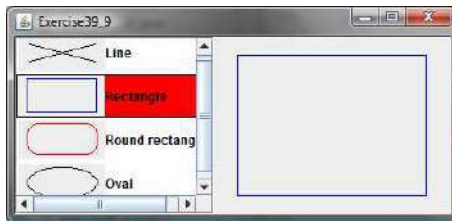


**Şekil 39.31**

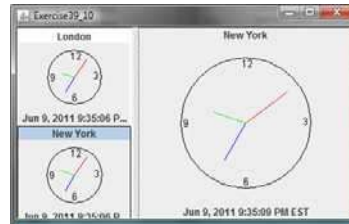
*Listede yer alan resimler, bir bölme panelinin sağ tarafında yer alan etikette görüntülemek için de kullanılır.*

39.9★★

(Şekilleri görüntüle) Şekil 39.32(a)'de gösterildiği gibi, JList örneğinde bir etiketle birlikte geometrik şekillerin bir listesini gösteren bir program oluşturun. Listedeki bir şekil seçildiğinde, seçilen şekli bir panelde görüntüleyin. Şekiller, Listing 14.5, FigurePanel.java'daki FigurePanel sınıfı kullanılarak çizilebilir.



(a)



(b)

**Şekil 39.32**



(a) Liste geometrik şekilleri ve isimlerini gösterir; (b) Liste şehirleri ve saatleri gösterir.

39.10 \*\*

(Saatlerin listesi) Bir program yazın, bu program bir şehirler listesi ve her birinin yer el saatlerini bir saatte gösterir, Şekil 39.32(b)'de gösterildiği gibi. Bir şehir listeden seçildiğinde, saati sağda büyük bir resimde gösterilir.

### Bölüm 39.10

39.11 \*\*

(Kombinasyon kutusunda özel hücre renderlayıcı oluşturun) Bir program oluşturun ve bu program geometrik şekillerin bir listesini ve bir etiketi bir kombinasyon kutusunda gösterir, Şekil 39.33(a)'de gösterildiği gibi. Bu alıştırmayı, Alıştırma 39.9 ile liste hücre renderlayıcısını paylaşabilir.



Şekil 39.33

(a) Kombinasyon kutusu, geometrik şekillerin ve şekil adlarının bir listesini içerir. (b) Kombinasyon kutusu, her biri

kendi rengiyle ön plan rengi olarak kullanılan bir renk adları listesi içerir.

39.12 \*\*

(Renkli metin renderlayın) Kullanıcının bir etiket için ön plan renklerini seçmesine olana k tanıyan bir program yazın, Şekil 39.33(b)'de gösterildiği gibi. Kombinasyon kutusu 13 standart rengi içerir (SİYAH, MAVİ, CAMGÖBEĞİ, KARANLIK GRİ, GRİ, YEŞİL, AÇIK GRİ, MACENTA, TURUNCU, PEMBE, KIRMIZI, BEYAZ, SARI). Kombinasyon kutusundaki her renk adı, kendi rengini ön plan rengi olarak kullanır.

39.13 \*

(Kombinasyon kutusunda seçilen bir öğeyi DELETE tuşu ile silin) Liste 39.11, ComboBoxCellRendererDemo.java dosyasını değiştirin, seçilen öğeyi kombinasyon kutusundan DELETE tuşunu kullanarak silin.

### Kapsamlı

39.14 \*

(Takvim) Bir spinner kullanarak bir takvimi kontrol eden bir program yazın, Şekil 39.7'de gösterildiği gibi. Takvimi görüntülemek için `CalendarPanel` sınıfını kullanın (bkz. Liste 31.4).

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 40

### JTable ve JTree

#### Hedefler

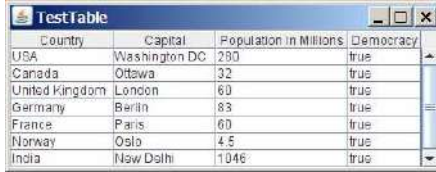
- JTable kullanarak tabloları görüntülemek (§40.2).
- TableModel, DefaultTableModel, TableColumnModel, DefaultTableColumnModel ve ListSelectionModel kullanarak satır ve sütunları işlemek (§§40.3–40.5) .
- Tablo modelinde otomatik sıralama ve filtreleme sağlamak (§40.4).
- Bir tabloda satır ve sütun eklemek, satır ve sütun silmek (§40.5) .
- Varsayılan render'lar ve editörler kullanarak tablo hücrelerini görüntülemek ve düzenlemek (§40.6).
- Özel renderleyiciler ve düzenleyiciler kullanarak tablo hücrelerini oluşturmak ve düzenlemek (§40.7).
- Tablo modeli olaylarını yönetmek (§40.8).
- JTree kullanarak verileri ağaç hiyerarşisinde görüntülemek (§40.9).
- Ağaç yapısını TreeModel ve DefaultTreeModel kullanarak modellemek (§40.10).
- Ağaç düğümlerini TreeNode, MutableTreeNode ve DefaultMutableTreeNode kullanarak eklemek, kaldırmak ve işlemek (§40.11).
- Ağaç düğümlerini ve yollarını TreeSelectionModel ve DefaultTreeSelectionModel kullanarak seçmek (§40.12).
- Varsayılan ve özel renderleyiciler ve düzenleyiciler kullanarak ağaç düğümlerini oluşturmak ve düzenlemek (§40.14).

## 40.1 Giriş

Önceki bölüm model-görünüm mimarisini, Swing MVC'yi ve JSpinner, JList ve JComboBox'taki modelleri tanıttı. Bu bölüm JTable ve JTree'yi tanıtmakta ve bu modellerin JTable ve JTree'de data işlemek için nasıl kullanılacağını açıklamaktadır.

## 40.2 JTable

JTable, verileri iki boyutlu bir ızgarada satırlar ve sütunlar halinde görüntüleyen bir Swing bileşenidir; Şekil 40.1'de gösterilmiştir.



Country	Capital	Population in Millions	Democracy
USA	Washington DC	280	true
Canada	Ottawa	32	true
United Kingdom	London	60	true
Germany	Berlin	83	true
France	Paris	60	true
Norway	Oslo	4.5	true
India	New Delhi	1046	true

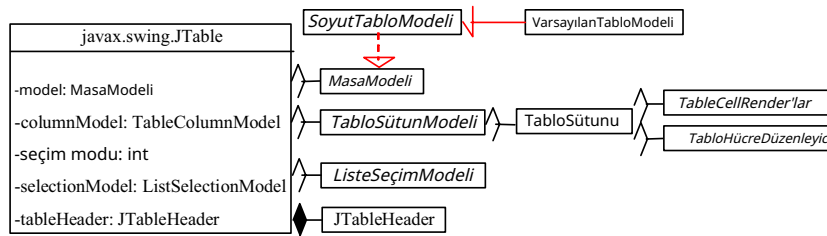
Şekil 40.1

JTable verileri bir tabloda görüntüler.

JTable doğrudan kaydırmayı desteklemez. Kaydırılabilir bir tablo oluşturmak için bir JScrollPane oluşturmanız ve JTable'in bir örneğini kaydırma paneline eklemeniz gerekir. Eğer bir tablo bir kaydırma paneline yerleştirilmezse, sütun başlığı görünmeyecektir, çünkü sütun başlığı kaydırma panelinin görünüm alanının başlığında yer alır.

JTable'in üç destekleyici modeli vardır: bir tablo modeli, bir sütun modeli ve bir liste-seçim modelidir. *tablo modeli* verileri saklamak yönetmek için kullanılır. *sütun modeli* tablodaki tüm sütunları temsil eder. *liste-seçim modeli* JList tarafından satırları, sütunları ve hücreleri seçmek için kullanılanla aynıdır.

JTable ayrıca TableColumn ve JTableHeader adında iki yararlı destekleyici sınıfa sahiptir. TableColumn, belirli bir sütun hakkında bilgi içerir. JTableHeader, tablo başlığını görüntülemek için kullanılabilir. Her sütunun varsayılan bir düzenleyici ve renderer'ı vardır. Ayrıca, TableCellEditor arayüzünü uygulayarak özel bir düzenleyici oluşturabilir ve TableCellRenderer arayüzünü uygulayarak özel bir render oluşturabilirsiniz. Bu arayüzlerin ve sınıfların ilişkisi Şekil 40.2'de gösterilmektedir.



Şekil 40.2

JTable birçok destekleyici arayüz ve sınıf içerir.

NOT: JTable için tüm destekleyici arayüzler ve sınıflar javax.swing.table paketinde gruplanmıştır.

Şekil 40.3, JTable'nin yapıcılarını, özelliklerini ve yöntemlerini göstermektedir.



### Şekil 40.3

*JTable sınıfı, tablolar oluşturmak, özelleştirmek ve manipüle etmek için kullanılır.*

JTable sınıfı, tablolar oluşturmak için yedi yapıcı içerir. Bir tabloyu, argüman almayan yapıcısı, modelleri, dikey bir dizi içindeki satır verileri ve

bir dizi içindeki kolon başlık adları veya vektörlerdeki satır verileri ve kolon başlık adları ile oluşturabilirsiniz. Liste 40.1, satır verileri ve kolon adları ile bir tablo oluşturur (satır 20) ve bunu bir kaydırma paneline yerleştirir (satır 23). Tablo, Şekil 40.1'de gösterilmiştir.

Listeleme 40.1 TestTable.java

```
<kenar notu satır 5: sütun adları>
<kenar notu satır 9: satır verileri>
<kenar notu satır 20: tablo oluştur>
<kenar notu satır 23: kaydırma paneli>
<kenar notu satır 25: ana yöntem atlandı>

1 javax.swing.*'i içe aktarın;
2
3 genel sınıf Test Tablosu t Uygulamayı genişletir {
4 // Tablo sütun adlarını oluştur
5 özel Dize[] sütun Adları =
6 { "Ülke", "Başkent", "Nüfus Milyonla", "Demokrasi" };
7
8 // Tablo verilerini oluştur
9 private Object[][] data = {
10 { "ABD", "Washington DC", 280, true },
11 { "Kanada", "Ottawa", 32, true },
12 { "Birleşik Krallık", "Londra", 60, true },
13 { "Almanya", "Berlin", 83, true },
14 { "Fransa", "Paris", 60, true },
15 { "Norveç", "Oslo", 4.5, true },
16 { "Hindistan", "Yeni Delhi", 1046, true }
17 };
18
19 // Bir tablo oluştur
20 özel JTable jTable1 = yeni JTable(veri, sütunİsimleri);
21
22 genel TestTable() {
23 ekle(yeni JScrollPane(jTable1));
24 }
25 }
```

NOT

<kenar notu: otomatik kutulama>

10. satırdaki 280 ve true gibi ilkel tür değerleri \_\_\_\_\_  
autoboxed olarak yeni Integer(280) ve yeni Boolean(true) olarak dönüştürülür.

JTable, tabloları özelleştirmenin birçok yolunu sağlayan çeşitli özelliklere sahip güçlü bir kontrolüdür. Sık kullanılan tüm özellikler Şekil 40.3'te belgelenmiştir. autoResizeMode özelliği, sütunların nasıl yeniden boyutlandırılacağını belirtir (tablo sütunlarını yeniden boyutlandırabilirsiniz ancak satırları boyutlandıramazsınız). Olası değerler şunlardır:

```
JTable.AUTO_RESIZE_OFFnJT
able.AUTO_RESIZE_LAST_COLUMNnJTa
ble.AUTO_RESIZE_SUBSEQUENT_COLUMNSnJTa
ble.AUTO_RESIZE_NEXT_COLUMN
JTable.AUTO_RESIZE_ALL_COLUMNS
```

Varsayılan mod JTable.AUTO\_RESIZE\_SUBSEQUENT\_COLUMNS'dır. Başlangıçta, tablodaki her sütun aynı genişliği (75 piksel) kaplar. AUTO\_RESIZE\_OFF ile, bir sütunu yeniden boyutlandırmak diğer sütunların genişliklerini etkilemez. AUTO\_RESIZE\_LAST\_COLUMN ile, \_\_\_\_\_

bir sütunu yeniden boyutlandırmak son sütunun genişliğini etkiler. AUTO\_RESIZE\_SUBSEQUENT\_COLUMNS ile, bir sütunu yeniden boyutlandırmak tüm sonraki sütunların genişliklerini etkiler. AUTO\_RESIZE\_NEXT\_COLUMN ile, bir sütunu yeniden boyutlandırmak sonraki sütunun genişliklerini etkiler. AUTO\_RESIZE\_ALL\_COLUMNS ile, bir sütunu yeniden boyutlandırmak tüm sütunların genişliklerini etkiler.

Liste 40.2, birkaç JTable özelliğinin kullanımını gösteren bir örnek vermektedir. Örnek, bir tablo oluşturur ve kullanıcının bir Otomatik Yeniden Boyutlandırma Modu seçmesine, satır yüksekliğini ve kenar boşluğunu belirtmesine ve ızgaranın gösterilip gösterilmeyeceğini belirtmesine olanak tanır. Programın bir örnek çalışması Şekil 40.4'te gösterilmiştir.



**Şekil 40.4**

*Otomatik boyutlandırma modunu, tablonun satır yüksekliğini ve satır kenar boşluğunu belirtebilir ve tabloda ızgarayı gösterip göstermeyeceğinizi ayarlayabilirsiniz.*

#### Listeleme 40.2 TablePropertiesDemo.java

<kenar notu satır 9: sütun adları>  
 <kenar notu satır 13: tablo verileri>  
 <kenar notu satır 23: tablo>  
 <kenar notu satır 27: spinnerlar>  
 <kenar notu satır 35: combo kutusu>  
 <kenar notu satır 40: UI oluştur>  
 <kenar notu satır 57: tablo özellikleri>  
 <kenar notu satır 64: spinner dinleyici>  
 <kenar notu satır 72: spinner dinleyici>  
 <kenar notu satır 80: onay kutusu dinleyici>  
 <kenar notu satır 88: combo kutusu dinleyici>  
 <kenar notu satır 109: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.event.*'i içe aktarın;
5
6 public class TablePropertiesDemo extends JApplet {
7 // Tablo sütun adlarını oluştur
8 özel Dize[] sütun Adları =
9 {"Ülke", "Başkent", "Milyonlarca Nüfus", "Demokrasi"};
10
11 // Tablo verilerini oluştur
12 özel Nesne[][] satırVerisi = {
13 {"ABD", "Washington DC", 280, true},
14 {"Kanada", "Ottawa", 32, true},
15 {"Birleşik Krallık", "Londra", 60, true},
16 {"Almanya", "Berlin", 83, true},
17 {"Fransa", "Paris", 60, true},
18 {"Norveç", "Oslo", 4.5, true},
19 {"Hindistan", "Yeni Delhi", 1046, true}

```

```

20 };
21
22 // Bir tablo oluřtur
23 özel jTable jTable1 = newt jTable(rowData, sütun Adları);
24
25 // İki spinner oluřtur
26 özel ISpinner spiRo Yükseklięi =
27 newt JSpinner(tnewt SpinnerNumberModel(t16t,t 1t,t 50t,t 1t));
28 özel JSpinner jspiRowMargin =
29 newt JSpinner(tnewt SpinnerNumberModel(t1t,t 1t,t 50t,t 1t));
30
31 // Bir onay kutusu oluřtur
32 özel JCheckBox jchkShowGrid = tnewt JCheckBox(t"showGrid"t,t true);
33
34 // Bir açılır kutu oluřtur
35 özel JComboBox jcboAutoResizeMode = new JComboBox(new String[]{
36 "AUTO_RESIZE_OFF"t,t"AUTO_RESIZE_LAST_COLUMN"t,
37 "AUTO_RESIZE_SUBSEQUENT_COLUMNS"t,t "AUTO_RESIZE_NEXT_COLUMN"t,
38 "AUTO_RESIZE_ALL_COLUMNS"t});
39
40 genel TablePropertiesDemo() {
41 JPanel panel1 = t newt JPanel(); n 42 panel1.
add(tnewt JLabel(t"rowHeight"t)); n 43 panel1.add(jspiR
owHeight);
44 panel1.add(tnewt JLabel(t"rowMargin"t)); n 45 panel
1.add(jspiRowMargin);
46 panel1.add(jchkShowGrid);
47
48 JPanel panel2 = t newt JPanel(); n 49 panel2.
add(tnewt JLabel(t"autoResizeMode"t)); n 50 panel2.add(jcbo
AutoResizeMode);
51
52 add(panel1, BorderLayout.SOUTH); n 53 add(p
anel2, BorderLayout.NORTH); n 54 ekle(
 yeni JScrollPane(jTable1));
55
56 // jTable1'i bařlat
57 jTable1.setAutoResizeMode(JTable.AUTO_RESIZE_OFF); n 58 jTable1.se
tGridColor(Renk.BLUE);
59 jTable1.setSelectionMode(ListSelectionModel.SINGLE_SELECTION); n 60 jTable1.setS
electionBackground(Color.RED);
61 jTable1.setSelectionForeground(Color.WHITE);
62
63 // jspiRowHeight için bir dinleyici kaydet ve oluřtur
64 jspiRowHeight.addChangeListener(tnewt ChangeListener() {
65 public voidt stateChanged(ChangeEvent e) {
66 jTable1.setRowHeight(n 67 ((Tamsayı)
(jspiRowHeight.getValue()).intValue()); n 68 }
69 });
70
71 // jspiRowMargin için bir dinleyici kaydet ve oluřtur
72 jspiRowMargin.addChangeListener(tnewt ChangeListener() {
73 public voidt stateChanged(ChangeEvent e) {
74 jTable1.setRowMargin(n 75 ((Integer)
jspiRowMargin.getValue()).intValue()); n 76 }
77 });
78
79 // jchkShowGrid için bir dinleyici kaydet ve oluřtur
80 jchkShowGrid.addActionListener(tnewt ActionListener() { n 81 @Override
82 public void actionPerformed(ActionEvent e) {
83 jTable1.setShowGrid(jchkShowGrid.isSelected()); n 84 }
85 });

```

```

86
87 // jchoAutoResizeMode için bir dinleyici kaydet ve oluştur
88 jchoAutoResizeMode.addActionListener(new ActionListener() { n 89 @Override
90
91 public void actionPerformed(ActionEvent e) {
92 String seçiliItem = n 92 (String)jchoAut
oResizeMode.getSelectedItemAt();
93
94 if (selectedItem.equals("AUTO_RESIZE_OFF"))
95 jTable1.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
96 else if (selectedItem.equals("AUTO_RESIZE_LAST_COLUMN"))
97 jTable1.setAutoResizeMode(JTable.AUTO_RESIZE_LAST_COLUMN);
98 else if (selectedItem.equals
99 ("AUTO_RESIZE_SUBSEQUENT_COLUMNS")) n100 jTable1.setAuto
ResizeMode(
101 jTable1.AUTO_RESIZE_SUBSEQUENT_COLUMNS);
102 else if (selectedItem.equals("AUTO_RESIZE_NEXT_COLUMN"))
103 jTable1.setAutoResizeMode(JTable.AUTO_RESIZE_NEXT_COLUMN);
104 aksi takdirde eğer (selectedItem.equals("AUTO_RESIZE_ALL_COLUMNS"))
105 jTable1.setAutoResizeMode(JTable.AUTO_RESIZE_ALL_COLUMNS); n106 }
107 }); n108 } n
109 }

```

Satır verilerini önceden biliyorsanız,  
JTable(Object[][] rowData, Object[] columnNames) yapıcısını kullanarak bir tablo oluşturmak  
 rahattır. 23. satırda gösterildiği gibi, bu \_\_\_\_\_  
 yapıcı kullanılarak bir JTable oluşturulmaktadır.

Satır yüksekliği ve satır marjını seçmek için iki JSpinner nesnesi (jspiRowHeight, jspiRowMargin) 26-29. satırlarda oluşturulmuştur. jspiRowHeight için başlangıç değeri 16 olarak ayarlanmıştır, bu da rowHeight için varsayılan özellik değeridir. jspiRowMargin için başlangıç değeri 1 olarak ayarlanmıştır, bu da rowMargin için varsayılan özellik değeridir. 32. satırda showGrid etiketi ile bir onay kutusu (jchkShowGrid) oluşturulmuş ve başlangıçta seçilmiştir. 35-38. satırlarda autoResizeMode seçmek için bir açılır kutu oluşturulmuştur.

JTable özelliklerinin (autoResizeMode, gridColor, \_\_\_\_\_  
 selectionMode, selectionBackground ve selectionForeground) değerleri  
 57-61. satırlarda ayarlanmıştır.

Spinner'lar, onay kutuları ve kombinasyon kutuları için işleme kodu  
 64-106. satırlarda verilmiştir.

### 40.3 Tablo Modelleri ve Tablo Sütun Modelleri

#### <kenar notu: TableModel>

JTable, veri depolama ve işleme işlemlerini tablo veri modeline devreder. Bir tablo veri modeli, tablo modeli \_\_\_\_\_  
 dinleyicilerini kaydetme, hücreleri manipüle etme ve satır sayısı, sütun sayısı, sütun sınıfı ve sütun adı alma yöntemlerini tanımlayan TableModel arayüzünü uygulamalıdır.

#### <kenar notu: AbstractTableModel>

AbstractTableModel sınıfı, TableModel'deki çoğu yöntemin kısmi uygulamalarını sağlar. Dinleyicilerin yönetimi ile ilgilenir ve TableModelEvents oluşturmak ve dinleyicilere iletmek için bazı kolaylıklar sunar. Somut bir TableModel oluşturmak için, yalnızca AbstractTableModel'i genişletip



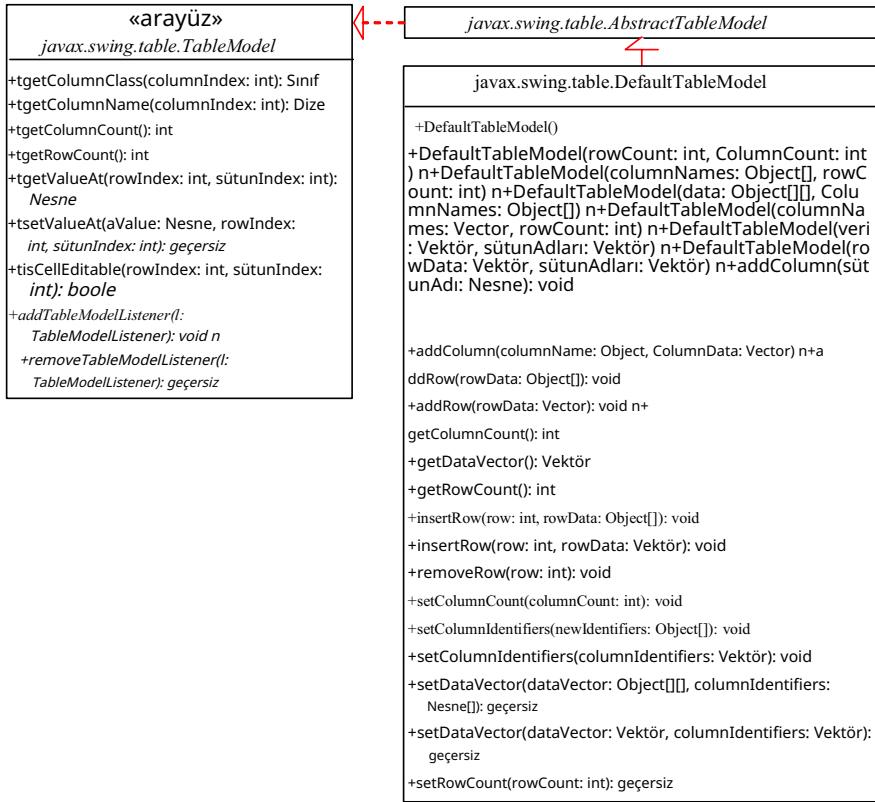
en az aşağıdaki üç yöntemi uygulamanız gerekir:

- `public int getRowCount()`
- `public int getColumnCount()`
- `public Object getValueAt(renk tonu kaş, renk tonu sütunu)`

**<kenar notu: DefaultTableModel>**

DefaultTableModel sınıfı AbstractTableModel'i genişletir ve bu üç yöntemi uygular. Ayrıca, DefaultTableModel veri için somut bir depolama sağlar. Veriler bir vektörde saklanır. Vektördeki elemanlar, her biri bireysel hücre değerini temsil eden nesne dizileridir. Veri erişimi ve değiştirilmesi için DefaultTableModel

Şekil 40.5'te gösterilmiştir.



**Şekil 40.5**

*TableModel, bir tabloda verileri depolar ve yönetir ve DefaultTableModel TableModel için varsayılan bir uygulama sağlar.*

Liste 40.3, tablo modellerini gösteren bir örnek vermektedir. Örnek, bir tablo modeli oluşturur (satır 16), modeli tabloya bağlar (satır 20), tabloya bir satır ekler (satır 25), ilk satırdan önce bir satır ekler (satır 26), 1 indeksine sahip bir satırı kaldırır (satır 28), yeni bir sütun ekler (satır 29) ve yeni değerler ayarlar

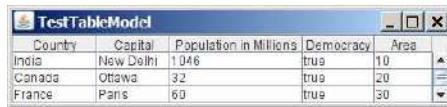
belirtilen hücreler (satırlar 30-32). Şekil 40.6 programın çıktısını göstermektedir.

#### Liste 40.3 TestTableModel.java

```
<kenar notu satır 6: sütun adları>
<kenar notu satır 10: satır verileri>
<kenar notu satır 16: tablo modeli oluştur>
<kenar notu satır 20: tablo oluştur>
<margin note line 23: kaydırma paneli>
<margin note line 25: satır ekle>
<margin note line 26: satır ekle>
<margin note line 28: satır kaldır>
<margin note line 29: sütun ekle>
<margin note line 30: değeri ayarla>
<margin note line 34: ana yöntem atlandı>

1 javax.swing.*'i içe aktarın;
2 javax.swing.table.*'ı içe aktarın;
3
4 genel sınıf TestTableModel tJApplet'i genişletir {
5 // Tablo sütun adlarını oluştur
6 özel Dize[] sütunAdları =
7 { "Ülke", "Başkent", "Nüfus Milyonla", "Demokrasi" };
8
9 // Tablo verilerini oluştur
10 özel Nesne[][] veri = {
11 { "ABD", "Washington DC", 280, true },
12 { "Kanada", "Ottawa", 32, true }
13 }; n 14

15 // Bir model oluştur
16 özel VarsayılanTableModel tableModel =
17 new DefaultTableModel(veri, sütunAdları);
18
19 // Bir tablo oluştur
20 özel JTable jTable1 = yeni JTable(tableModel);
21
22 genel TestTableModel() {
23 ekle(yeni JScrollPane(jTable1));
24
25 tableModel.addRow(yeni Object[] { "Fransa", "Paris", 60, true });
26 tableModel.insertRow(0, yeni Object[]
27 { "Hindistan", "Yeni Delhi", 1046, true });
28 tableModel.removeRow(1);
29 tableModel.addColumn("Alan");
30 tableModel.setValueAt(10, 0, 4);
31 tableModel.setValueAt(20, 1, 4);
32 tableModel.setValueAt(30, 2, 4);
33 }
34 }
```



Country	Capital	Population in Millions	Democracy	Area
India	New Delhi	1046	true	10
Canada	Ottawa	32	true	20
France	Paris	60	true	30

Şekil 40.6

TableModel ve DefaultTableModel, tablo verilerini ekleme, güncelleme ve kaldırma yöntemlerini içerir.

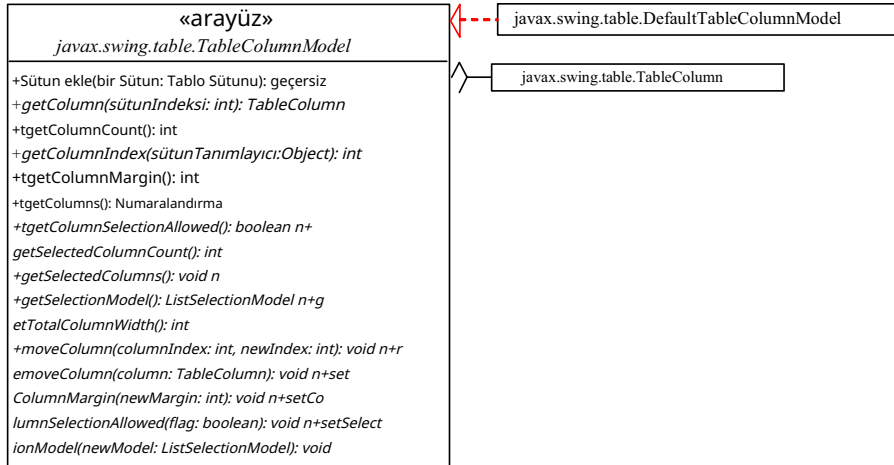
TableModel, tablo verilerini yönetir. Bir TableModel aracılığıyla satır ekleyebilir ve kaldırabilirsiniz. Ayrıca, bir TableModel aracılığıyla bir sütun da ekleyebilirsiniz. Ancak, bir TableModel aracılığıyla bir sütun kaldıramazsınız. JTable'dan bir sütun kaldırmak için, bir tablo sütun modelini kullanmalısınız.

**<kenar boşluğu notu: TableColumnModel>**

Tablo sütun modelleri, bir tablodaki sütunları yönetir. Tablo sütunlarını seçmek, eklemek, taşımak ve kaldırmak için kullanılabilirler. Bir tablo sütun modeli, tablo sütun modeli dinleyicilerini kaydetmek, sütunlara erişmek ve bunları manipüle etmek için yöntemleri tanımlayan TableColumnModel arayüzünü uygulamalıdır; bu, Şekil 40.7'de gösterilmiştir.

**<kenar notu: DefaultTableColumnModel>**

DefaultTableColumnModel, TableColumnModel ve PropertyChangeListener'ı uygulayan somut bir sınıftır. DefaultTableColumnModel sınıfı, sütunlarını bir vektörde saklar ve sütunları seçmek için bir ListSelectionModel örneği içerir.



**Şekil 40.7**

TableColumnModel, bir tabloda sütunları yönetir ve DefaultTableColumnModel bunun somut bir uygulamasıdır.

**<kenar boşluğu notu: TableColumn>**

Sütun modeli, bir tablodaki tüm sütunlarla ilgilenir. TableColumn sınıfı, tablodaki bireysel bir sütunu modellemek için kullanılır. Belirli bir sütun için bir TableColumn örneği, TableColumnModel'deki getColumn(index) yöntemi veya JTable'deki getColumn(columnIdentifier) yöntemi kullanılarak elde edilebilir.

Şekil 40.8, sütun genişliğini manipüle etmek ve hücre renderer'ını, hücre editörünü ve başlık renderer'ını belirtmek için TableColumn'daki özellikleri, yapıcıları ve yöntemleri göstermektedir.

javafx.swing.table.TableColumn	UML diyagramında atlanan get ve set yöntemleri ile JavaBeans özellikleri
#cellEditor: TableCellEditor n#	Bu sütundaki bir hücreyi düzenlemek için editör.
cellRenderer: TableCellRenderer n#	Bu sütundaki bir hücreyi görüntülemek için render.
headerRenderer: TableCellRenderer n#	Bu sütunun başlığını görüntülemek için render.
headerValue: Object	Bu sütunun başlık değeri.
#tanımlayıcı: Nesne	Bu sütunun tanımlayıcısı.
#maxWidth: int	Bu sütunun maksimum genişliği.
#minWidth: int	Bu sütunun minimum genişliği (varsayılan: 15 piksel).
#modelIndex: int	Tablo modelindeki sütunun indeksi (varsayılan: 0).
#tercihEdilenGenişlik: int	Bu sütunun tercih edilen genişliği (varsayılan: 75 piksel).
#ölçülebilir: boolean	Bu sütunun yeniden boyutlandırılıp boyutlandırılmayacağını belirtir (varsayılan: true).
#genişlik: int	Bu sütunun genişliğini belirtir (varsayılan: 75 piksel).
+TabloSütunu()	Varsayılan bir tablo sütunu oluşturur.
+TableColumn(modelIndex: int)	Belirtilen sütun için bir tablo sütunu oluşturur.
+TableColumn(modelIndex: int, genişlik: int)	Belirtilen sütun ve genişlik ile bir tablo sütunu oluşturur.
+TableColumn(modelIndex: int, genişlik: int, cellRenderer: TableCellRenderer)	Belirtilen sütun, genişlik ve hücre renderleyicisi ile bir tablo sütunu oluşturur.
+boyutuGenişliğeUydur(): void	Başlık hücresinin genişliğine uyacak şekilde sütunu yeniden boyutlandırır.

Şekil 40.8

*TableColumn sınıfı tek bir sütunu modellemektedir.*

Liste 40.4, tablo sütun modellerini gösteren bir örnek vermektedir. Örnek, tablo sütun modelini tablodan alır (satır 21), ilk sütunu ikinci sütuna taşır (satır 22) ve son sütunu kaldırır (satırlar 23). Şekil 40.9, programın çıktısını göstermektedir.

Liste 40.4 TestTableColumnModel.java

<margin note line 6: sütun adları>  
 <margin note line 10: satır verileri>  
 <margin note line 16: tablo oluştur>  
 <margin note line 19: kaydırma paneli>  
 <margin note line 21: sütun modeli>  
 <margin note line 22: bir sütunu taşı>  
 <margin note line 23: bir sütunu kaldır>  
 <margin note line 25: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2 javax.swing.table.*'ı içe aktarın;
3
4 public class TestTableColumnModel t JApplet'i genişletir {
5 // Tablo sütun adlarını oluştur
6 özel Dize[] sütun Adları =
7 { "Ülke", "Başkent", "Nüfus Milyonla", "Demokrasi" };
8
9 // Tablo verilerini oluştur
10 özel Nesne[][] veri = {
11 { "ABD", "Washington DC", 280, true },
12 { "Kanada", "Ottawa", 32, true }
13 };

```

```

14
15 // Bir tablo oluřtur
16 özel JTable jTable1 = yeni JTable(veri, sütunİsimleri);
17
18 public t Test Tablosu Sütun Modeli() {
19 add(tnewt JScrollPane(jTable1));
20
21 TableColumnModel sütunModel = jTable1.getColumnModel(); n 22 sütunMod
22 el.moveColumn(t0t,t 1t);
23 sütunModel.removeColumn(columnModel.getColumnModel().getColumn(t3t));n 24 }
25 }

```

Capital	Country	Population in Millions
Washington DC	USA	280
Ottawa	Canada	32

**Şekil 40.9**

*TableColumnModel, sütunları taşıma ve kaldırma yöntemlerini içerir.*

NOT: Tablo modeli ve tablo sütun modelinde tanımlanan bazı yöntemler, kolaylık sağlamak için JTable sınıfında da tanımlanmıştır. Örneği n, getColumnCount() yöntemi JTable, TableModel ve \_\_\_\_\_

TableColumnModel, column\_model'da tanımlanan addColumn metodu, aynı zamanda table model'da da tanımlanmıştır ve column model'da tanımlanan getColumn() metodu da JTable sınıfında tanımlanmıştır.

#### <kenar notu: TableHeader>

JTableHeader, JTable'in başlığını gösteren bir GUI bileşenidir (bkz. Şekil 40.10). Bir JTable oluşturduğunuzda, JTableHeader'in bir örneği otomatik olarak oluşturulur ve tableHeader özelliğinde saklanır. Varsayılan olarak, sütunları sütunun başlığını sürükleyerek yeniden sıralayabilirsiniz. Bunu devre dışı bırakmak için, reorderingAllowed özelliğini false olarak ayarlayın.

<i>javax.swing.JComponent</i>	
<i>↳</i>	
<b>javax.swing.table.JTableHeader</b>	
#columnModel: TableColumnModel n#	Tablo başlığının TableColumnModel'i.
draggedColumn: TableColumn n#	Sürüklenen sütun.
draggedDistance: TableCellRenderer n#r	Orijinal konumdan sürüklenen konuma olan mesafe.
eorderingAllowed: boolean	Sütunların yeniden sıralanmasına izin verilip verilmediği (varsayılan: doğru).
#resizingAllowed: boolean n	Sütunların yeniden boyutlandırılmasına izin verilip verilmediği (varsayılan: doğru).
#resizingColumn: TableColumn n	Yeniden boyutlandırılan sütun.
#table: JTable	Bu nesnenin başlık olduğu tablo.
+JTableHeader()	Varsayılan bir TableColumnModel ile bir JTableHeader oluşturur.
+JTableHeader(cm: TableColumnModel)	Bir TableColumnModel ile bir JTableHeader oluşturur.

**Şekil 40.10**

*JTableHeader sınıfı, JTable'nin başlığını görüntüler.*

## 40.4 Otomatik Sıralama ve Filtreleme

Otomatik sıralama ve filtreleme, iki kullanışlı özelliktir. JTable'deki herhangi bir sütunda otomatik sıralamayı etkinleştirmek için, bir örneğini oluşturun TableRowSorter ile bir tablo modeli ve JTable'nin rowSorter'ını aşağıdaki gibi ayarlayın:

**<kenar notu: bir TableRowSorter oluşturun>**

```
TableRowSorter<TableModel> sıralayıcı = new TableRowSorter<TableModel>(tableModel);
```

**<kenar notu: setRowSorter>**

```
jTable.setRowSorter(sort);
```



(a)

(b)

Şekil 40.11

(a) Tablo Ülkeye göre sıralanmıştır. (b) Tablo Başkente göre sıralanmıştır.

Tablo görüntülendiğinde, tabloyu Şekil 40.11'de gösterildiği gibi bir sütun başlığına tıklayarak sıralayabilirsiniz.

Tablodaki satırları seçmek için bir filtre belirtebilirsiniz. Filtre bir sütuna veya tüm sütunlara uygulanabilir. `javax.swing.RowFilter` sınıfı, filtreler oluşturmak için birkaç statik yöntem içerir. Belirtilen düzenli ifade ile bir `RowFilter` oluşturmak için `regexFilter` yöntemini kullanabilirsiniz. Örneğin, aşağıdaki ifade, ilk sütunu veya ikinci sütunu U harfi ile başlayan satırlar için bir filtre oluşturur.

**<kenar notu: bir filtre oluşturun>**

```
RowFilter rowFilter = RowFilter.regexFilter("U.*", tintt[0], tintt[1]);
```

`regexFilter` yöntemindeki ikinci argüman, bir dizi sütun indeksini belirtir. Hiçbir indeks belirtilmezse, filterde tüm sütunlar aranır.

**<kenar notu: JTable'da filtreyi ayarlayın>**

Filtrelemeyi etkinleştirmek için, bir filtreyi TableRowSorter ile ilişkilendirmeniz gerekir; bu, JTable'nin rowSorter özelliğine ayarlanır.

Liste 40.5, JTable'de otomatik sıralama ve

filtrelemeyi gösteren bir örnek vermektedir.

Liste 40.5 TestTableSortFilter.java

**<margin not satırı 7: sütun adları> n<margin not satırı 11: satır verileri> n<margin not satırı 22: tablo oluşturun> n<margin not satırı 25: TableRowSorter oluşturun>**

<kenar boşluğu notu satırı 33: sıralayıcıyı ayarla> n<kenar boşluğu notu satırı 49: filtreyi kaldır> n<kenar boşluğu notu satırı 51: bir filtre ayarlayın> n<kenar notu satırı 55: ana yöntem atlandı>

```

1 javax.swing.*'i içe aktarın;
2 javax.swing.table.*'ı içe aktarın;
3 java.awt.BorderLayout'u içe aktarın;
4
5 genel sınıf TestTableSortFilter t JApplet'i genişletir {
6 // Tablo sütun adlarını oluştur
7 özel Dize[] sütun Adları =
8 { "Ülke", "Başkent", "Nüfus Milyonla", "Demokrasi" };
9
10 // Tablo verilerini oluştur
11 özel Nesne[][] veri = {
12 { "ABD", "Washington DC", 280, true },
13 { "Kanada", "Ottawa", 32, true },
14 { "Birleşik Krallık", "Londra", 60, true },
15 { "Almanya", "Berlin", 83, true },
16 { "Fransa", "Paris", 60, true },
17 { "Norveç", "Oslo", 4.5, true },
18 { "Hindistan", "Yeni Delhi", 1046, true }
19 };
20
21 // Bir tablo oluştur
22 özel JTable jTable1 = yeni JTable(veri, sütunİsimleri);
23
24 // Bir TableRowSorter oluştur
25 özel TableRowSorter<TableModel> sıralama =
26 newt TableRowSorter<TableModel>(jTable1.getModel());
27
28 özel JTextField jtf Filtresi = t newt JTextField();
29 private JButton btFilter = new JButton("Filtrele");
30
31 public t Test Tablosu Sıralama Filtresi() {
32 // Otomatik sıralayıcıyı etkinleştir
33 jTable1.setRowSorter(sort);
34
35 JPanel panel = new JPanel(new java.awt.BorderLayout());
36 panel.add(new JLabel("Eşleşecek bir kelime belirtin:"),
37 BorderLayout.WEST); n 38 panel.
add(jtfFilter, BorderLayout.CENTER); n 39 panel.add(btFilter, Bo
rderLayout.EAST);
40
41 add(panel, BorderLayout.SOUTH); n 42 ekle(
 newt JScrollPane(jTable1), BorderLayout.CENTER);
43
44 btFilter.addActionListener(tnewt java.awt.event.ActionListener() n{
45 @Geçersiz Kıl
46 public voidt actionPerformed(java.awt.event.ActionEvent e) {
47 Dize metni = jtfFilter.getText();
48 if (text.trim().length() == t 0t)
49 sorts.setRowFilter(tnull);
50 başka
51 sorter.setRowFilter(RowFilter.regexFilter(text)); n 52 }
53 });

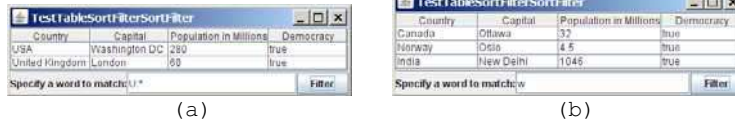
```

```

54 }
55 }

```

Örnek, bir TableRowSorter oluşturur (satır 25) ve jTable1'deki sorter'ı ayarlar (satır 33). Program, kullanıcının bir metin alanından filtre deseni girmesine izin verir; bu, Şekil 40.12'de gösterilmiştir. Eğer hiçbir şey girilmezse, filtre ayarlanmamıştır (satır 48). Eğer bir regex girilirse, Filtre butonuna tıklamak, filtreyi jTable1'e ayarlar (satır 50).

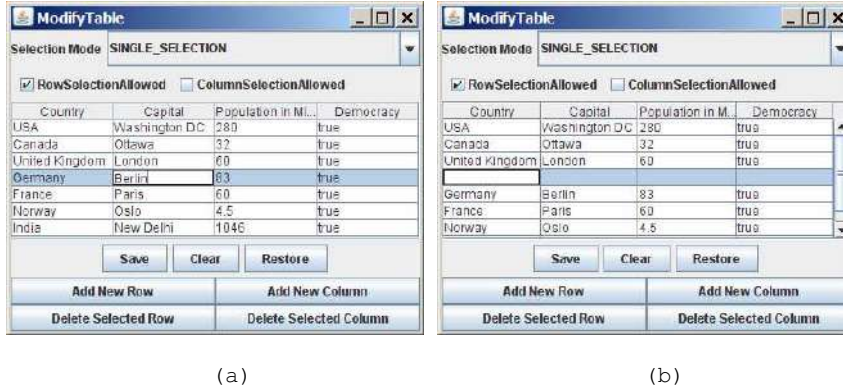


**Şekil 40.12**

(a) Tabloyu regex U.\* ile filtrele. (b) Tabloyu regex w ile filtrele.

## 40.5 Vaka Çalışması: Satır ve Sütunları Değiştirme

Bu vaka çalışması, tablo modelleri, tablo sütun modelleri, liste-seçim modelleri ve TableColumn sınıfının kullanımını gösterir. Program, kullanıcının seçim modunu ve seçim türünü seçmesine, satır ve sütun ekleyip çıkarmasına ve tabloyu kaydetmesine, temizlemesine veya geri yüklemesine izin verir; bu, Şekil 40.13a'da gösterilmiştir.



**Şekil 40.13**

*Bir tabloya satır ve sütunları etkileşimli olarak ekleyebilir, kaldırabilir ve değiştirebilirsiniz.*

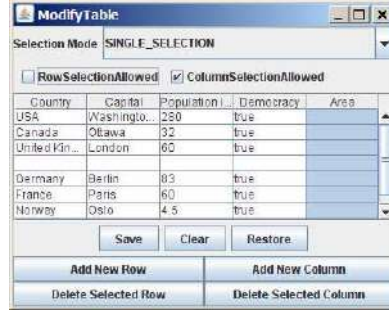
Yeni Satır Ekle butonu, şu anda seçili olan satırdan önce yeni boş bir satır ekler; bu, Şekil 40.13b'de gösterilmiştir. Eğer şu anda seçili bir satır yoksa, yeni boş bir satır tablonun sonuna eklenir.

Yeni Sütun Ekle butonuna tıkladığınızda, sütunun başlığını almak için bir giriş iletişim kutusu görüntülenir; bu, Şekil 40.14a'da gösterilmiştir. Yeni sütun tabloya eklenir; bu, Şekil 40.14b'de gösterilmiştir.





(a)



(b)

#### Şekil 40.14

*Bir tabloya yeni bir sütun ekleyebilirsiniz.*

Seçilen İlk Satırı Sil butonu, ilk seçilen satırı siler.  
Seçilen İlk Sütunu Sil butonu, ilk seçilen sütunu siler.

Kaydet butonu, mevcut tablo verilerini ve sütun adlarını kaydeder.  
Temizle butonu, tablodaki satır verilerini temizler. Geri Yükle butonu, kaydedilmiş tabloyu geri yükler.

Listeleme 40.6 programı verir.

#### Listeleme 40.6 ModifyTable.java

<kenar notu satır 10: sütun adları>  
<kenar notu satır 14: tablo verileri>  
<kenar notu satır 25: tablo modeli>  
<kenar notu satır 29: tablo>  
<kenar notu satır 32: düğmeler>  
<kenar notu satır 42: açılır kutu>  
<kenar notu satır 47: onay kutuları>  
<kenar notu satır 53: UI oluştur>  
<kenar notu satır 93: satır ekle>  
<kenar notu satır 104: sütun ekle>  
<kenar notu satır 112: satır sil>  
<kenar notu satır 120: sütun sil>  
<kenar notu satır 132: tabloyu kaydet>  
<kenar notu satır 148: tabloyu temizle>  
<kenar notu satır 155: tabloyu geri yükle>  
<kenar notu satır 173: satır seçimi izin verildi>  
<kenar notu satır 181: sütun seçimi izin verildi>  
<kenar notu satır 190: seçim modunu seç>  
<kenar notu satır 209: sütun adlarını al>  
<kenar notu satır 217: ana yöntem atlandı>

```

1 java.awt.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.table.*'ı içe aktarın;
5 java.io.*'yu içe aktarın;
6 java.util.Vector'ı içe aktarın;
7
8 public class ModifyTable extends JApplet {

```

```

9 // Tablo sütun adlarını oluştur
10 özel Dize[] sütunAdları =
11 {"Ülke", "Başkent", "Nüfus Milyonla", "Demokrasi"};
12
13 // Tablo verilerini oluştur
14 özel Nesne[][] satırVerisi = {
15 {"ABD", "Washington DC", 280, true},
16 {"Kanada", "Ottawa", 32, true},
17 {"Birleşik Krallık", "Londra", 60, true},
18 {"Almanya", "Berlin", 83, true},
19 {"Fransa", "Paris", 60, true},
20 {"Norveç", "Oslo", 4.5, true},
21 {"Hindistan", "Yeni Delhi", 1046, true}
22 };
23
24 // Bir tablo modeli oluştur
25 özel DefaultTableModel tabloModel = new DefaultTableModel(
26 satırVerisi, sütunAdları);
27
28 // Bir tablo oluştur
29 özel JTable jTable1 = new JTable(tabloModel);
30
31 // Düğmeler oluştur
32 private JButton jbtAddRow = new JButton("Yeni Satır Ekle");
33 private JButton jbtAddColumn = new JButton("Yeni Sütun Ekle");
34 private JButton jbtDeleteRow = new JButton("Seçili Satırı Sil");
35 özel JButton jbtDeleteColumn = new JButton(
36 "Seçili Sütunu Sil");
37 private JButton jbtSave = new JButton("Kaydet");
38 private JButton jbtClear = new JButton("Temizle");
39 özel JButton jbtRestore = new JButton("Restore");
40
41 // Seçim modları için bir kombinasyon kutusu oluştur
42 özel JComboBox jcbSelectionMode =
43 new JComboBox(new String[] {"SINGLE_SELECTION",
44 "SINGLE_INTERVAL_SELECTION", "MULTIPLE_INTERVAL_SELECTION"});
45
46 // Onay kutuları oluştur
47 özel JCheckBox jchkRowSelectionAllowed =
48 new JCheckBox("RowSelectionAllowed", true);
49 özel JCheckBox jchkColumnSelectionAllowed =
50 new JCheckBox("ColumnSelectionAllowed", false);
51
52 genel ModifyTable() {
53 JPanel panel1 = new JPanel();
54 panel1.setLayout(new GridLayout(2, 2));
55 panel1.add(jbtAddRow);
56 panel1.add(jbtAddColumn);
57 panel1.add(jbtDeleteRow);
58 panel1.add(jbtDeleteColumn);
59
60 JPanel panel2 = new JPanel();
61 panel2.add(jbtSave);
62 panel2.add(jbtClear);
63 panel2.add(jbtRestore);
64
65 JPanel panel3 = new JPanel();
66 panel3.setLayout(new BorderLayout(5, 0));
67 panel3.add(new JLabel("Seçim Modu"), BorderLayout.WEST);
68 panel3.add(jcbSelectionMode, BorderLayout.CENTER);
69
70 JPanel panel4 = new JPanel();
71 panel4.setLayout(new FlowLayout(FlowLayout.LEFT));
72 panel4.add(jchkRowSelectionAllowed);
73 panel4.add(jchkColumnSelectionAllowed);
74

```

```

75 JPanel panel5 = t.newt JPanel(); n 76 panel5
.setLayout(t.newt GridLayout(2,t,1)); n 77 panel5.add(pane
l3);
78 panel5.add(panel4);
79
80 JPanel panel6 = t.newt JPanel(); n 81 panel6
.setLayout(t.newt BorderLayout()); n 82 panel6.add(panel
1, BorderLayout.SOUTH); n 83 panel6.add(panel2, BorderLa
yout.CENTER);
84
85 add(panel5, BorderLayout.NORTH); n 86 ekle(
 yeni JScrollPane(jTable1),
87 BorderLayout.CENTER); n 88 add(pa
nel6, BorderLayout.SOUTH);
89
90 // Tablo seçim modunu başlat
91 jTable1.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
92
93 jbtAddRow.addActionListener(t.newt ActionListener() { n 94 @Override

95 public void actionPerformed(ActionEvent e) {
96 eğer (jTable1.getSelectedRow() >= 0)
97 tableModel.insertRow(jTable1.getSelectedRow(),
98 yeni java.util.Vector<String>());
99 başka
100 tableModel.addRow(yeni java.util.Vector<String>());
101 }
102 });
103
104 jbtAddColumn.addActionListener(yeni ActionListener() {
105 @Override
106 public void actionPerformed(ActionEvent e) {
107 String name = JOptionPane.showInputDialog("Yeni Sütun Adı");
108 tableModel.addColumn(name, yeni java.util.Vector());
109 } n110 });

111
112 jbtDeleteRow.addActionListener(newt ActionListener() {
113 @Geçersiz kıl
114 public void actionPerformed(ActionEvent e) {
115 eğer (jTable1.getSelectedRow() >= 0)
116 tableModel.removeRow(jTable1.getSelectedRow()); n117 }

118 });
119
120 jbtDeleteColumn.addActionListener(t.newt ActionListener() { n121 @Override

122 public void actionPerformed(ActionEvent e) {
123 if (jTable1.getSelectedColumn() >= t0t) {
124 TableColumnModel sütunModel = jTable1.getColumnModel(); n125 TableColumn tabl
eColumn =
126 sütunModel.getColumn(jTable1.getSelectedColumn()); n127 sütunModel.removeCol
umn(tableColumn);
128 } n129 } n130
});

131
132 jbtSave.addActionListener(newt ActionListener() {
133 @Geçersiz kıl
134 public void actionPerformed(ActionEvent e) {
135 denemek {
136 ObjectOutputStream çıkış = t.yeni ObjectOutputStream(
137 newt FileOutputStream(t"tablemodel.dat"));
138 out.writeObject(tableModel.getDataVector()); n139 out.writeObject(ge
tColumnNames());
140 çıkış.kapat();

```

```

141 }
142 catch (İstisna örneği) {
143 ex.printStackTrace(); n144 }

145} n146));

147
148 jbtClear.addActionListener (newt ActionListener() {
149 @Geçersiz kıl
150 public void actionPerformed(ActionEvent e) {
151 tableModel.setRowCount(t0t);n152 }

153 });
154
155 jbtRestore.addActionListener(tnewt ActionListener() { n156 @Override

157 public void actionPerformed(ActionEvent e) {
158 denemek {
159 ObjectInputStream in = t newt ObjectInputStream(
160 newt FileInputStream(t"tablemodel.dat"t));
161 Vector<String> rowData = (Vector<String>)in.readObject(); n162 Vector<String> sütunA
dları =
163 (Vector<String>)in.readObject(); n164 tableModel.setData
Vector(rowData, sütunAdları); n165 in.close();

166 }
167 catch (İstisna örneği) {
168 ex.printStackTrace(); n169 }

170} n171));

172
173 jchkRowSelectionAllowed.addActionListener (newt ActionListener() {
174 @Geçersiz kıl
175 public void actionPerformed(ActionEvent e) {
176 jTable1.setRowSelectionAllowed(n177 jchkRowSele
ctionAllowed.isSelected()); n178 }

179 });
180
181 jchkColumnSelectionAllowed.addActionListener (
182 newt ActionListener() {
183 @Geçersiz kıl
184 public void actionPerformed(ActionEvent e) {
185 jTable1.setColumnSelectionAllowed(n186 jchkColumnS
electionAllowed.isSelected()); n187 }

188 });
189
190 jcbSelectionMode.addActionListener(tnewt ActionListener() { n191 @Override

192 public void actionPerformed(ActionEvent e) {
193 String seçiliItem = n194 (String)jcbSel
ectionMode.getSelectedItemAt();
194
195 if (selectedItem.equals(t"SINGLE_SELECTION"t))
196 jTable1.setSelectionMode(n198 ListSelection
Model.SINGLE_SELECTION);
199 else if (selectedItem.equals(t"SINGLE_INTERVAL_SELECTION"t))
200 jTable1.setSelectionMode(n201 ListSelection
Model.SINGLE_INTERVAL_SELECTION);
202 else if (selectedItem.equals(t"MULTIPLE_INTERVAL_SELECTION"t))
203 jTable1.setSelectionMode(n204 ListSelection
Model.MULTIPLE_INTERVAL_SELECTION); n205 }

206 });

```

```

207 }
208
209 Privatet Vector<String> getColumnNames() { n210
Vector<String> sütunAdları = t newt Vector<String>();
211
212 fort (tintt i = t0t; i < jTable1.getColumnCount(); i++) n213 Colum
nNames.add(jTable1.getColumnNane(i));
214
215 sütun Adlarını döndür;
n216 }
217 }

```

Bir tablo modeli, satır verileri ve sütun adları ile DefaultTableModel kullanılarak oluşturulur (satırlar 25-26). Bu model, bir JTable oluşturmak için kullanılır (satır 29).

GUI nesneleri (butonlar, açılır kutu, onay kutuları) satır 32-50'de oluşturulur ve UI 'da satır 53-88'de yerleştirilir.

Tablo seçim modu, liste seçim moduyla aynıdır. Varsayılan olarak, seçim modu MULTIPLE\_INTERVAL\_SELECTION'dır. Seçim açılır kutusundaki başlangıç değerini eşleştirmek için (jcbSelectionMode), tablonun seçim modu SINGLE\_SELECTION olarak ayarlanmıştır.

Yeni Satır Ekle butonunun eylemi 93-102. satırlarda işlenir. insertRow metodu, seçilen satırdan önce yeni bir satır ekler (97-98. satırlar). Eğer şu anda hiçbir satır seçilmemişse, addRow metodu tablo modeline yeni bir satır ekler (100. satır).

Yeni Sütun Ekle butonunun eylemi 104-110. satırlarda işlenir. addColumn metodu, tablo modeline yeni bir sütun ekler (108. satır).

Seçilen Satırı Sil butonunun eylemi 112-118. satırlarda işlenir. removeRow(rowIndex) metodu, seçilen satırı tablo modelinden kaldırır (116. satır).

Seçilen Sütunu Sil butonunun eylemi 120-130. satırlarda işlenir. Bir sütunu kaldırmak için, TableColumnModel'deki removeColumn metodunu kullanmalısınız (127. satır).

Kaydet butonunun eylemi 132-146. satırlarda işlenir. Satır verilerini ve sütun adlarını bir çıktı dosyasına nesne akışı kullanarak yazar (136-140. satırlar). Sütun adları, getColumnNames() metodu kullanılarak elde edilir (209-216. satırlar). tableModel'i kaydetmeyi deneyebilirsiniz, çünkü tableModel bir örnektir.

DefaultTableModel (satırlar 25-26) ve DefaultTableModel sıralanabilir. Ancak, tableModel TableModel olayı için sıralanamayan dinleyiciler içerebilir.

Clear butonunun eylemi satırlar 148-153'te işlenir. Satır 0'a ayarlayarak tabloyu temizler (satır 151).

Restore butonunun eylemi satırlar 155-171'de işlenir. Dosyadan row verilerini ve sütun adlarını nesne akışı kullanarak okur. (satırlar 159-165) ve yeni verileri ve sütun adlarını tablo modeline ayarlar (satır 164).

## 40.6 Tablo Render'ları ve Editörler

Tablo hücreleri hücre render'ları tarafından boyanır. Varsayılan olarak, bir hücre nesnesinin dize temsili (toString()) görüntülenir ve dize, bir metin alanında olduğu gibi düzenlenebilir. JTable, varsayılan dize render'larını ve editörlerini değiştirmek için belirtilebilecek, Tablo 40.1'de listelenen önceden tanımlanmış render'lar ve editörler kümesini korur.

**Tablo 40.1**

*Tablolar için önceden tanımlanmış render'lar ve editörler*

Sınıf	Render'lar	Yazar
<u>Nesne</u>	JLabel (sola hizalı) JTextField	
<u>Tarih</u>	JLabel (sağa hizalı) JTextField	
<u>Numara</u>	JLabel (sağa hizalı) JTextField	
<u>GörüntüSimgesi</u>	JLabel (orta hizalı)	
<u>Boolean</u> <u>aligned)</u>	JCheckBox (orta hizalı) JCheckBox	JCheckBox (merkez

Önceden tanımlanmış render'lar ve editörler, tablo modelindeki getColumnClass() metodundan dönen sınıfa uymak için otomatik olarak bulunur ve yüklenir. String dışındaki bir sınıf için önceden tanımlanmış bir render veya editör kullanmak istiyorsanız, TableModel'in bir alt sınıfını genişleterek kendi tablo modelinizi oluşturmanız gerekir. Tablonuzun model sınıfında, aşağıdaki gibi sütunun sınıfını döndürmek için getColumnClass() metodunu geçersiz kılmalısınız:

```

halk Sınıf getColumnClass(int kolon) {
 return getValueAt(t0t, sütun).getClass();n }

```

Varsayılan olarak, tüm hücreler düzenlenebilir. Bir hücrenin düzenlenmesini engellemek için, TableModel'deki isCellEditable(int rowIndex, int columnIndex) metodunu false döndürecek şekilde geçersiz kılmalısınız. Varsayılan olarak, bu metod AbstractTableModel'de true döner.

Önceden tanımlanmış tablo render'larını ve editörlerini göstermek için, kitaplar için bir tablo görüntüleyen bir program yazalım. Tablo, Şu şekilde gösterilen Başlık, Gerekli Kopyalar, Yayıncı, Yayın Tarihi, Stokta, ve Kitap Fotoğrafı sütun adlarına sahip üç satırdan oluşmaktadır. Tarihlerin ve simgelerin düzenlenebilir olmadığını varsayın; kullanıcıların bu iki sütunu düzenlemesini engelleyin.

Title	Copies Needed	Publisher	Date Published	In-stock	Book Photo
Introduction to Java Programming	120	Que Education & Training	Jan 6, 1999	<input type="checkbox"/>	
Introduction to Java Programming, 2E	220	Que Education & Training	Jan 6, 1999	<input type="checkbox"/>	
Introduction to Java Programming, 3E	220	Pentice Hall	Nov 30, 2000	<input checked="" type="checkbox"/>	

**Şekil 40.15**

*Boolean ve resim hücreleri için önceden tanımlanmış render'ları etkinleştirmek için özel bir tablo modeli kullanmalısınız.*

Liste 40.7, Boolean ve resim hücreleri için önceden tanımlanmış render'ları etki-  
nleştirmek üzere getColumnClass yöntemini (satırlar 15-17) geçersiz kılan  
MyTableModel adlı özel bir tablo modelini vermektedir. MyTableModel ayrıca i-  
sCellEditable() yöntemini (satırlar 20-24) geçersiz kılar. Varsayılan olarak, isC-  
ellEditable() true döner. Örnek, kullanıcının resim simgelerini ve tarihleri dü-  
zenlemesine izin vermediğinden, bu yöntem tarih ve resim sütunlarının düzenle-  
nmesini devre dışı bırakmak için false dönecek şekilde geçersiz kılınmıştır. Bir h-  
ücrenin düzenlenebilir olması için, tablo modelindeki isCellEditable() true olmalıdı-  
r.

**Liste 40.7 MyTableModel.java**

<margin note line 15: column class>  
<margin note line 20: hücre düzenlenebilir mi?>

```

javax.swing.*i içe aktar;n
import javax.swing.table.*;himp
ort java.util.*;

genel sınıf MyTableModel, DefaultTableModel'i genişletir {n publi
c MyTableModel() {

}

/** Belirtilen veri ve sütun adları ile bir tablo modeli oluşturun */
public MyTableModel(Object[] data, Object[] columnNames) {
 süper(veri, sütunAdları);n }

/** Sütun için bir sınıf döndürmek üzere bu yöntemi geçersiz kılın */
public Class getColumnClass(int column) {
 return getValueAt(0, sütun).getClass();n }

/** Bu yöntemi geçersiz kılın, eğer hücre düzenlenebilir ise true döndürün */
public boolean isCellEditable(int row, int column) {
 Sınıf columnClass = getColumnClass(column);
 return columnClass != ImageIcon.class &&
 columnClass != Date.class;
}
N}

```

MyTableModel'dan oluşturulan bir tablo modeli kullanarak bir JTable  
oluşturursanız, sayılar, Boolean değerler, tarihler ve simgeler için varsayılan  
render'lar ve editörler bu sütunları görüntülemek ve düzenlemek için kullanılı-  
r. Liste 40.8 bir test programı verir. Program, MyTableModel kullanarak bir t-  
ablo modeli oluşturur (satır 36). JTable, MyTableModel'deki getColumnCla-  
ss() yönteminde belirtilen sınıf için önceden tanımlanmış bir hücre render'ı ve önc-  
eden tanımlanmış bir editör atar.

**Liste 40.8 TableCellRendererEditorDemo.java**

<margin note line 7: sütun adları>  
<margin note line 12: resim simgeleri>  
<margin note line 20: satır verileri>  
<margin note line 36: tablo modeli>  
<margin note line 40: tablo>  
<margin note line 47: ana yöntem atlandı>

```

java.auto.*yu içe aktarın;
javax.swing.*i içe aktarın;
java.util.*i içe aktarın;

public class TableCellRendererEditorDemo extends JApplet {
 // Tablo sütun adlarını oluştur
 private String[] columnNames =
 {"Başlık", "Gerekli Kopyalar", "Yayınevi", "Yayın Tarihi",
 "Stokta", "Kitap Fotoğrafı"};
}

```

```

// Görüntü simgeleri oluştur
özel ImageIcon intro1eImageIcon = new ImageIcon(
getClass().getResource("image/intro1e.gif"));n özel Ima
geIcon intro2eImageIcon = new ImageIcon(
getClass().getResource("image/intro2e.gif"));n özel Ima
geIcon intro3eImageIcon = new ImageIcon(
getClass().getResource("image/intro3e.jpg"));

// Tablo verilerini oluştur
private Object[][] rowData = {
{"Java Programlamaya Giriş", 120,
"Que Eğitim & Eğitim",
new GregorianCalendar(1998, 1-1, 6).getTime(),n false, in
tro1eImageIcon},
{"Java Programlamaya Giriş, 2E", 220,
"Que Eğitim & Eğitim",
new GregorianCalendar(1999, 1-1, 6).getTime(),n false, in
tro2eImageIcon},
{"Java Programlamaya Giriş, 3E", 220,
"Prentice Hall",
new GregorianCalendar(2000, 12-1, 0).getTime(),n true, in
tro3eImageIcon},
};

// Bir tablo modeli oluştur
özel MyTableModel tableModel = new MyTableModel(n rowDa
ta, sütunAdları);

// Bir tablo oluştur
özel JTable jTable1 = new JTable(tableModel);

public TableCellRendererEditorDemo() {n JTabl
e1.setRowHeight(60);
add(new JScrollPane(jTable1), BorderLayout.CENTER);n }
}

```

Örnek, iki sınıf tanımlar: MyTableModel ve TableCellRendererEditorDemo. MyTableModel, DefaultTableModel'in bir uzantısıdır. MyTableModel'in amacı, getColumnClass() yönteminin varsayılan uygulamasını geçersiz kılmak ve kolonun sınıfını döndürmektir, böylece kolon için uygun bir önceden tanımlanmış JTable kullanılabilir. Varsayılan olarak, getColumnClass() Object.class döndürür.

## 40.7 Özel Tablo Render'ları ve Editörleri

Önceden tanımlanmış render'lar ve editörler kullanışlı ve kolaydır, bununla birlikte işlevleri sınırlıdır. Önceden tanımlanmış resim simgesi render'ı, resim simgesini bir etiket içinde görüntüler. Resim simgesi ölçeklenemez. Tüm resmin bir hücreye sığmasını istiyorsanız, bir özel render oluşturmanız gerekir.

Özel bir render, DefaultTableCellRenderer'ı genişleterek oluşturulabilir; bu, TableCellRenderer arayüzü için varsayılan bir uygulamadır. Özel render, tablo hücrelerini render etmek için bir bileşen döndürmek üzere getTableCellRendererComponent yöntemini geçersiz kılmalıdır. getTableCellRendererComponent yöntemi aşağıdaki gibi tanımlanmıştır:

```

halk Component getTableCellRendererComponent
(JTable tablo, Object değer, boolean seçildi mi,
boolean odaklandı mı, int satır, int sütun)

```



Bu yöntem imzası, özel liste hücre render'ları oluşturma için kullanılan `getListCellRendererComponent()` yöntemine çok benzer.

Bu yönteme bir `JTable`, hücre ile ilişkili değer, değerın seçilip seçilmediğı ve hücrenin odaklanıp odaklanmadığı bilgisi ile değerlerin satır ve sütun indeksleri geçilir. Yöntemden döndürülen bileşen, tablodaki hücreye boyanır. 40.9 numaralı listede yer alan sınıf,

`MyImageCellRenderer`, bir panelde resim simgelerini görüntülemek için bir render tanımlar.

#### Listeleme 40.9 `MyImageCellRenderer.java`

<margin note line 7: *override method*>

<margin note line 10: *getImage()*>

<margin note line 11: *resim görüntüleyici oluştur*>

<margin note line 13: *resim görüntüleyiciyi döndür*>

```
import javax.swing.*;
import javax.swing.table.*;
import java.awt.*;

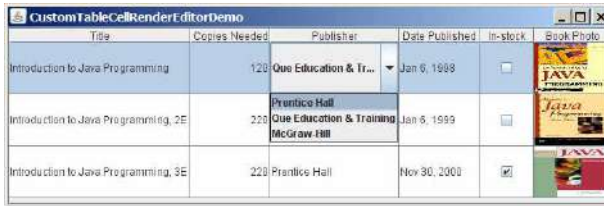
public class MyImageCellRenderer extends DefaultTableCellRenderer {
 /** DefaultTableCellRenderer'da bu metodu geçersiz kılın */
 public Bileşen getTableCellRendererComponentn (JTable
 e tablosu, Nesne değeri, boolean isSelected,n boolean isFo
 cused, int satır, int sütun) {
 Resim resmi = ((ImageIcon)value).getImage();n Image
 Viewer imageView = new ImageViewer(image);

 imageView'ı döndür;n }
}
```

Özel bir editör de oluşturabilirsiniz. `JTable`, bir hücreyi bir metin alanında, bir onay kutusunda veya bir açılır kutuda düzenlemek için kullanılabilecek `DefaultCellEditor` sınıfını sağlar. Kullanmak için, basitçe bir metin alanı, bir onay kutusu veya bir açılır kutu oluşturun ve bunu `DefaultCellEditor`'ın yapıcısına geçirerek bir editör oluşturun.

Özel bir renderleyici ve editör kullanarak, önceki örnek

ölçeklenmiş görüntüleri gösterecek şekilde ve Yayıncı sütunundaki hücreleri düzenlemek için özel bir kombinasyon editörü kullanacak şekilde gözden geçirilebilir; bu, Şekil 40.16'da gösterilmiştir. Program, Liste 40.10'da verilmiştir.



Title	Copies Needed	Publisher	Date Published	In-stock	Book Photo
Introduction to Java Programming	120	Que Education & Training	Jan 6, 1998	<input type="checkbox"/>	
Introduction to Java Programming, 2E	220	Prentice Hall	Jan 6, 1999	<input type="checkbox"/>	
Introduction to Java Programming, 3E	220	Prentice Hall	Nov 30, 2000	<input checked="" type="checkbox"/>	

Şekil 40.16

Özel bir render, ölçeklenmiş bir görüntü gösterir ve özel bir editör, Yayıncı sütununu bir açılır kutu kullanarak düzenler.

#### Liste 40.10 `CustomTableCellRenderEditorDemo.java`

<kenar notu satır 8: *sütun adları*>

<kenar notu satır 13: resim simgeleri>  
 <kenar notu satır 21: satır verileri>  
 <kenar notu satır 37: tablo modeli>  
 <kenar notu satır 41: tablo>  
 <kenar notu satır 46: render ayarla>  
 <kenar notu satır 49: açılır kutu>  
 <kenar notu satır 56: editör ayarla>  
 <kenar notu satır 63: ana yöntem atlandı>

```
import java.awt.*;
import javax.swing.*;
import javax.swing.table.*;
import java.util.*;

public class CustomTableCellRendererEditorDemo extends JApplet {
 // Tablo sütun adlarını oluştur
 private String[] columnNames =
 {"Başlık", "Gerekli Kopyalar", "Yayinevi", "Yayın Tarihi",
 "Stokta", "Kitap Fotoğrafı"};

 // Görüntü simgeleri oluştur
 özel ImageIcon intro1eImageIcon = new ImageIcon(
 ImageIcon(getClass().getResource("image/intro1e.gif")));
 özel ImageIcon intro2eImageIcon = new ImageIcon(
 ImageIcon(getClass().getResource("image/intro2e.gif")));
 özel ImageIcon intro3eImageIcon =
 yeni ImageIcon(getClass().getResource("image/intro3e.jpg"));

 // Tablo verilerini oluştur
 private Object[][] rowData = {
 {"Java Programlamaya Giriş", 120,
 "Que Eğitim & Eğitim",
 new GregorianCalendar(1998, 1-1, 6).getTime(), n false, in
 tro1eImageIcon},
 {"Java Programlamaya Giriş, 2E", 220,
 "Que Eğitim & Eğitim",
 new GregorianCalendar(1999, 1-1, 6).getTime(), n false, in
 tro2eImageIcon},
 {"Java Programlamaya Giriş, 3E", 220,
 "Prentice Hall",
 new GregorianCalendar(2000, 12-1, 0).getTime(), n true, in
 tro3eImageIcon},
 };

 // Bir tablo modeli oluştur
 özel MyTableModel tableModel = new MyTableModel(n rowDa
 ta, sütunAdları);

 // Bir tablo oluştur
 özel JTable jTable1 = new JTable(tableModel);

 public CustomTableCellRendererEditorDemo() {
 // Görüntüleri görüntülemek için özel renderleyici ayarla
 TableColumn bookCover = jTable1.getColumn("Kitap Fotoğrafı");
 bookCover.setCellRenderer(new MyImageCellRenderer());

 // Yayıncılar için bir kombinasyon kutusu oluşturun
 JComboBox jcbPublishers = new JComboBox();
 jcbPublishers.addItem("Prentice Hall");
 jcbPublishers.addItem("Que Education & Training");
 jcbPublishers.addItem("McGraw-Hill");

 // Kombinasyon kutusunu yayıncı sütunu için düzenleyici olarak ayarlayın
 TableColumn publisherColumn = jTable1.getColumn("Publisher");
 publisherColumn.setCellEditor(
 new DefaultCellEditor(jcbPublishers));

 jTable1.setRowHeight(60); n ad
 d(new JScrollPane(jTable1), n Border
 Layout.CENTER);
 }
}
```

}  
N)

Bu örnek, önceki örnekte (satırlar 37-38) oluşturulan aynı tablo modelini (MyTableModel) kullanmaktadır. Varsayılan olarak, resim simgeleri önceden tanımlanmış resim simgesi işleyicisi kullanılarak görüntülenir. Resmi görüntülemek için MyImageCellRenderer kullanmak istiyorsanız, Kitap Fotoğrafı sütunu için MyImageCellRenderer işleyicisini açıkça belirtmeniz gerekir (satır 46). Benzer şekilde, Yayıncı sütunu için kombinasyon kutusu düzenleyicisini açıkça belirtmeniz gerekir (satırlar 56-57); aksi takdirde varsayılan düzenleyici kullanılacaktır.

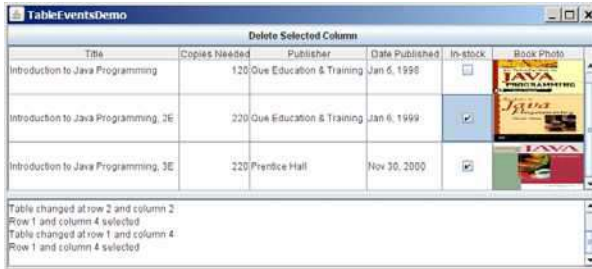
Yayıncı sütunundaki bir hücreyi düzenlediğinizde, üç öğeden oluşan bir kombinasyon kutusu görüntülenir. Kutudan bir öğe seçtiğinizde, hücrede görüntülenir. Seçimleri işlemek için kod yazmadınız. Seçimler DefaultCellEditor sınıfı tarafından işlenir.

Kitap Fotoğrafı sütununu yeniden boyutlandırıldığında, resim tüm hücreye sığacak şekilde yeniden boyutlandırılır. Önceden tanımlanmış resim render'ı ile, hücre resimden daha küçükse yalnızca resmin bir kısmını görebilirsiniz.

## 40.8 Tablo Model Olayları

JTable tablo olaylarını tetiklemez. MouseEvent, KeyEvent ve ComponentEvent gibi olayları, üst sınıfı JComponent'ten miras alarak tetikler. Tablo olayları, bu modellere değişiklik yapıldığında tablo modelleri, tablo sütun modelleri ve tablo seçim modelleri tarafından tetiklenir. Tablo modelleri, tablo verileri değiştiğinde TableModelEvent tetikler. Tablo sütun modelleri, sütunlar eklendiğinde, kaldırıldığında veya taşındığında ya da bir sütun seçildiğinde TableColumnModelEvent tetikler. Tablo seçim modelleri, bir seçim yapıldığında ListSelectionEvent tetikler.

Bu olayları dinlemek için, uygun bir model ile bir dinleyici kayıt ettirilmelidir ve doğru dinleyici arayüzü uygulanmalıdır. Liste 40.11, bu olayların nasıl kullanılacağını gösteren bir örnek gösterir. Program, bir satır veya sütun seçildiğinde, bir hücre düzenlendiğinde veya bir sütun kaldırıldığında bir metin alanında mesajlar görüntüler. Şekil 40.17, programın bir örnek çalışmasıdır.



Şekil 40.17

Tablo olay işleyicileri, tablo olaylarını bir metin alanında görüntüler.

Listeleme 40.11 TableEventsDemo.java

<kenar notu satır 10: sütun adları>  
 <kenar notu satır 15: resim simgeleri>  
 <kenar notu satır 23: tablo verileri>  
 <kenar notu satır 39: tablo modeli>  
 <kenar notu satır 43: tablo>  
 <kenar notu satır 46: sütun modeli>  
 <kenar notu satır 50: seçim modeli>  
 <kenar notu satır 86: tablo modeli dinleyicisi>  
 <kenar notu satır 94: sütun modeli dinleyicisi>  
 <kenar notu satır 131: seçim modeli dinleyicisi>  
 <kenar notu satır 141: ana yöntem atlandı>

```

1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.event.*'i içe aktarın;
5 javax.swing.table.*'ı içe aktarın;
6 java.util.*'i içe aktarın;
7
8 genel sınıf TableEvents Demo textendst Applet {
9 // Tablo sütun adlarını oluştur
10 özel Dize[] sütun Adları =
11 {"Başlık", "Gerekli Kopyalar", "Yayınevi", "Yayın Tarihi",
12 "Stokta", "Kitap Fotoğrafı"};
13
14 // Resim simgeleri oluştur
15 özel Resim Simgesi intro1 Resim Simgesi =
16 newt ImageIcon(getClass().getResource("t/image/intro1e.gif"));
17 özel ImageIcon intro2eImageIcon =
18 yeni ImageIcon(getClass().getResource("/image/intro2e.gif"));
19 özel ImageIcon intro3eImageIcon =
20 yeni ImageIcon(getClass().getResource("/image/intro3e.jpg"));
21
22 // Tablo verilerini oluştur
23 özel Nesne[][] satırVerisi = {
24 {"Java Programlamaya Giriş", 120,
25 "Que Eğitim & Eğitim",
26 yeni GregorianCalendar(1998, 1-1, 6).getTime(),
27 false, intro1eImageIcon},
28 {"Java Programlamaya Giriş, 2E", 220,
29 "Que Eğitim & Eğitim",
30 yeni GregorianCalendar(1999, 1-1, 6).getTime(),
31 yanlış, intro2eImageIcon},
32 {"Java Programlamaya Giriş, 3E", 220,
33 "Prentice Hall",
34 yeni GregorianCalendar(2000, 12-1, 0).getTime(),
35 doğru, intro3eImageIcon},
36 };
37
38 // Bir tablo modeli oluştur
39 özel MyTableModel tableModel = yeni MyTableModel(
40 satırVeri, sütunAdları);
41
42 // Bir tablo oluştur
43 özel JTable jTable1 = yeni JTable(tableModel);
44
45 // Tablo sütun modelini al
46 özel TableColumnModel tableColumnModel =
47 jTable1.getColumnModel();
48
49 // Tablo seçim modelini al
50 özel ListSelectionModel selectionModel =
51 jTable1.getSelectionModel();
52
53 // Bir metin alanı oluştur

```

```

54 özel JTextArea jtaMessage = new JTextArea();
55
56 // Bir buton oluştur
57 özel JButton jbtDeleteColumn =
58 new JButton("Seçilen Sütunu Sil");
59
60 genel TableEvents Demo() {
61 // Görüntüleri görüntülemek için özel bir render ayarlayın
62 TableColumn bookCover = jTable1.getColumn("Kitap Fotoğrafı");
63 bookCover.setCellRenderer(new MyImageCellRenderer());
64
65 // Yayıncılar için bir kombinasyon kutusu oluşturun
66 JComboBox jcboPublishers = yeni JComboBox(); n 67 jcboPub
lishers.addItem(t"Prentice Hall"); n 68 jcboPublishers.addIt
em(t"Que Education & Training"); n 69 jcboPublishers.addItem(t"McGraw
-Hill");
70
71 // Yayıncı sütunu için kombinasyon kutusunu düzenleyici olarak ayarlayın
72 TableColumn publisherColumn = jTable1.getColumn("Yayıncı");
73 publisherColumn.setCellEditor(
74 newt DefaultCellEditor(jcboPublishers));
75
76 jTable1.setRowHeight(t60t); n 77 jTabl
e1.setColumnSelectionAllowed(ttrue);
78
79 JSplitPane jSplitPanel = newt JSplitPane(
80 JSplitPane.VERTICAL_SPLIT); n 81 jSplitPane
1.add(tnewt JScrollPane(jTable1), JSplitPane.LEFT); n 82 jSplitPane1.add(tnewt JScroll
Pane(jtaMessage), JSplitPane.RIGHT); n 83 add(jbtDeleteColumn, BorderLayout.NORTH);
84
85 add(jSplitPanel, BorderLayout.CENTER);
86
87 tableModel.addTableModelListener(tnewt TableModelListener()) { n 87 @Override
88
89 public voidt tableChanged(TableModelEvent e) {
90 jtaMessage.append("Tablo, " + 90 e.getFirst
tRow() + " satırında ve " + e.getColumn() + ""); 91 }
92
93 });
94
95 tableColumnModel.addColumnModelListener(
96 newt TableColumnModelListener()) {
97
98 @Geçersiz Kıl
99 public voidt sütunRemoved(TableColumnModelEvent e) {
100 jtaMessage.append("İndeksli sütun " + e.getFromIndex() +
" silindi ");100 }
101
102 @Geçersiz kıl
103 public void sütunEklendi(TableColumnModelEvent e) {
104 }
105
106 @Geçersiz kıl
107 public void sütunTaşındı(TableColumnModelEvent e) {
108 }
109
110 @Geçersiz kıl
111 public void sütunAralığıDeğiştirdi(ChangeEvent e) {
112 }
113
114 @Geçersiz kıl
115 public void sütunSeçimiDeğiştirdi(ListSelectionEvent e) {
116 } n117});
118
119 jbtDeleteColumn.addActionListener (newt ActionListener() {

```

```

120 @Geçersiz Kıl
121 public void actionPerformed(ActionEvent e) {
122 if (jTable1.getSelectedColumn() >= t0t) {
123 TableColumnModel sütunModel = jTable1.getColumnModel(); n124 TabloSütun tabl
oSütun =
125 sütunModel.getColumnModel(jTable1.getSelectedColumn()); n126 sütunModel.removeCol
umn(tableColumn);
127 } n128 } n129
 };

130
131 seçimModel.addListSelectionListener(
132 new ListSelectionListener() {
133 @Geçersiz kıl
134 public void valueChanged(ListSelectionEvent e) {
135 jTableMessage.append("Satır " + jTable1.getSelectedRow() +
136 " ve sütun " + jTable1.getSelectedColumn() +
137 " seçildi");
138 } n139)); n1
40 } n141 }

```

Satır ve sütun seçim olaylarına yanıt vermek için, ListSelectionListener'da value Changed yöntemini uygulamanız gerekir. Hücre düzenleme olayına yanıt verme k için, TableModelListener'da tableChanged yöntemini uygulamanız gerekir. Sütun silme olayına yanıt vermek için, TableColumnModelListener'da columnRemoved yöntemini uygulamanız gerekir. Önceki örnekteki aynı tabloyu kullanalım, ancak seçilen sütunu silmek için bir düğme ve mesajları görüntülemek için bir metin alanı ekleyelim.

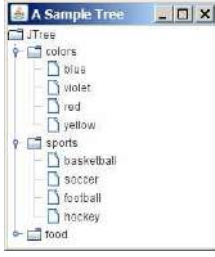
Bir tablo modeli, Listing 40.7'de verilen MyTableModel kullanılarak oluşturulur (satırlar 39-40). Bir tablo oluşturulduğunda (satır 43), varsayılan sütun modeli ve seçim modeli de oluşturulur. Bu nedenle, tablo sütun modelini ve seçim modelini tablodan elde edebilirsiniz (satırlar 46-51).

Bir satır veya sütun seçildiğinde, selectionModel tarafından bir ListSelectionEvent tetiklenir, bu da seçilen satır ve sütunu metin alanında görüntülemek için işleyiciyi çağırır (satırlar 134-138). Tablo içeriği veya yapısı değiştiğinde,

tableModel tarafından bir TableModelEvent tetiklenir, bu da değiştirilen verilerin son satırını ve son sütununu metin alanında görüntülemek için işleyiciyi çağırır (satırlar 88-91). Bir sütun, Seçilen Sütunu Sil düğmesine tıklanarak silindiğinde, tableColumnModel tarafından bir ColumnModelEvent tetiklenir, bu da silinen sütunun indeksini görüntülemek için işleyiciyi çağırır (satırlar 97-100).

## 40.9tJAğaç

JTree, verileri ağaç benzeri bir hiyerarşide görüntüleyen bir Swing bileşenidir, Şekil 40.18'de gösterildiği gibi.



**Şekil 40.18**

*JTree, verileri ağaç benzeri bir hiyerarşide görüntüler.*

Ağaçta görüntülenen tüm düğümler, hiyerarşik bir dizin listesi şeklinde dir. Ağaç, hiyerarşik ilişkileri olan yapılandırılmış verileri gezmek için kullanılabılır. Bir düğümün çocuk düğümleri olabilir. Bir düğüm, çocukları yoksa 'yaprak' olarak adlandırılır; ebeveyni olmayan bir düğüm ise ağacının 'kökü' olarak adlandırılır. Bir ağaç, her düğümün kendi alt ağacı için kök olarak işlev gördüğü bir çok alt ağaçtan oluşabilir.

Bir yaprak olmayan düğüm, düğüme veya düğümün önündeki tutamağa çift tıklanarak genişletilebilir veya daraltılabilir. Tutamak genellikle düğümün genişletilmiş veya daraltılmış olduğunu göstermek için görünür bir işarete sahiptir. Örneğin, Windows'ta + sembolü düğümün genişletilebileceğini, - sembolü ise daraltılabileceğini gösterir.

JTable gibi, JTree de birçok destekleyici arayüz ve sınıfa sahip çok karmaşık bir bileşendir. JTree, javax.swing paketindedir, ancak destekleyici arayüz ve sınıfların hepsi javax.swing.tree paketinde yer almaktadır. Destekleyici arayüzler TreeModel, TreeSelectionModel, TreeNode ve MutableTreeNode'dur; destekleyici sınıflar ise DefaultTreeModel, DefaultMutableTreeNode, DefaultTreeCellEditor'dır,

DefaultTreeCellRenderer ve TreePath.

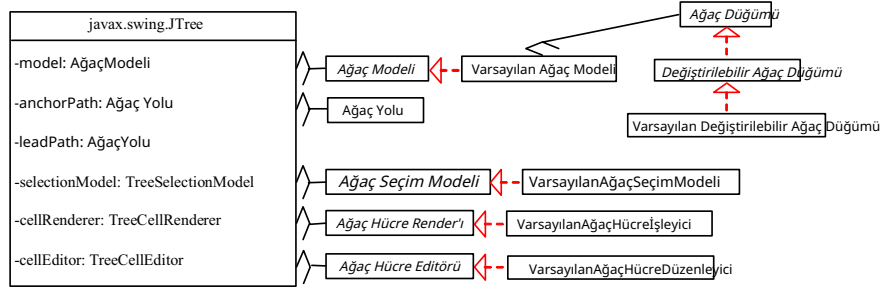
JTree ağacı görüntülerken, ağacın veri temsili TreeModel, TreeNode ve TreePath tarafından yönetilir. TreeModel tüm ağacı temsil eder, TreeNode bir düğümü temsil eder ve TreePath bir düğüme giden yolu temsil eder. ListModel veya TableModel'in aksine, TreeModel doğrudan ağaç verilerini saklamaz veya yönetmez. Ağaç verileri TreeNode ve TreePath içinde saklanır ve yönetilir.

DefaultTreeModel, TreeModel'in somut bir uygulamasıdır. MutableTreeNode, çocuk düğümleri ekleyerek veya kaldırarak veya düğümde saklanan bir kullanıcı nesnesinin içeriğini değiştirerek değiştirilebilen bir ağaç düğümünü temsil eden TreeNode'un bir alt arayüzüdür.

TreeSelectionModel arayüzü, ağaç düğümü seçimlerini yönetir. DefaultTreeCellRenderer sınıfı, bir düğümde bir etiket ve/veya bir simge görüntüleyebilen v arsayılan bir ağaç düğümü render'ı sağlar. DefaultTreeCellEditor, hücreleri bir metin alanında düzenlemek için kullanılabilir.

TreePath, TreeModel'den sağlanan bir Nesne dizisidir. Dizinin elemanları, kök her zaman dizinin ilk elemanı (indeks 0) olacak şekilde sıralanmıştır.

Şekil 40.19, bu arayüzlerin ve sınıfların nasıl ilişkili olduğunu göstermektedir.



**Şekil 40.19**

*JTree*, birçok destekleyici arayüz ve sınıf içerir.

Şekil 40.20, JTree'nin yapıcılarını, sık kullanılan özelliklerini ve yöntemlerini göstermektedir.



javax.swing.JTree	Jav aBe ans özellikleri, UML diyagramında atlanan get ve set metotları ile birlikte
#hücreDüzenleyici: AğaçHücreDüzenleyici	Ağaçta girişleri düzenlemek için bir hücre düzenlemesini belirtir.
#hücreGörüntüleyici: AğaçHücreGörüntüleyici	Bir hücre render'ını belirtir.
#düzenlenebilir: boolean	Hücrelerin düzenlenebilir olup olmadığını belirtir (varsayılan: yanlış).
#model: AğaçModeli	Ağaç modelini korur.
#kökGörünür: boolean	Kökün görüntülenip görüntülenmeyeceğini belirtir (yapıcıya bağlı olarak).
#satırYüksekliği: int	Ağaçta görüntülenen düğüm için satır yüksekliğini belirtir (varsayılan: 16 piksel).
#genişletildiğindeKaydırır: boolean	Eğer doğruysa, bir düğüm genişletildiğinde, mümkün olan en fazla t orun görünür hale getirilir (varsayılan: 16 piksel).
#seçimModeli: AğaçSeçimModeli	Bu ağaçtaki seçilen düğümlerin kümesini modellemektedir.
#kökTutacaklarıGöster: boolean	Kök tutucularının görüntülenip görüntülenmeyeceğini belirtir (varsayılan: doğru).
#değiştirmeTıklamaSayısı: int	Bir düğüm genişletilmeden önceki fare tıklama sayısı (varsayılan: 2).
-anchorSelectionPat h: TreePat h	Bağlantı noktası olarak tanımlanan yol.
-expandsEdPath'leri seçin: boolean	Seçim içindeki yolların genişletilmesi gerekiyorsa doğru (varsayılan: doğru).
-lead ionPath'leri seçin: TreePath	Öncü olarak tanımlanan yol.
+JTree()	Şekil 36.18'de gösterildiği gibi, örnek bir ağaç modeli ile bir JTree oluşturur.
+JTree(değer: java.util.Hashtable) n+J	Bir Hashtable'daki anahtar/değer çiftlerini çocukları olarak içeren görünmez bir r kök ile bir JTree oluşturur.
Tree(değer: Object[])	Bir dizi içindeki öğeleri çocukları olarak içeren görünmez bir kök ile bir JTree oluşturur.
+JTree(newModel: TreeModel) n+J	Belirtilen ağaç modeli ile bir JTree oluşturur.
Tree(root: TreeNode)	Belirtilen ağaç düğümünü kök olarak kullanan bir JTree oluşturur.
+JTree(root: TreeNode, askAllowsChildren: boolean)	Belirtilen ağaç düğümünü kök olarak kullanan bir JTree oluşturur ve bir düğümün belirtilen şekilde yaprak düğüm olup olmadığını belirler.
+JTree(değer: Vector) n+addSelect ionPath(yol h: TreePath): void n+add S ele ionPaths(yollar: TreePath[]): vo id n+addSelect ionRow(row: int): voi d	Görünmez bir kök ve vektördeki elemanları çocukları olarak kullanan bir JTree oluşturur.
+addSelect ionRows(rows: int[]): voi d n+clearSelection(): v oid	Belirtilen TreePath'i mevcut seçime ekler.
+çökmek Pa th(path: AğaçYolu): void	Belirtilen TreePaths'i mevcut seçime ekler.
+seçimYolunuAl(): AğaçYolu	Belirtilen satırdaki yolu mevcut seçime ekler.
+seçimYollarınıAl(): AğaçYolu[]	Seçimi temizler.
+sonSeçilenYolBileşeniAl()	Belirtilen yol ile tanımlanan düğümün kapatıldığından ve görünür olduğundan e min olur.
+satırSayısınıAl(): int	Kökten ilk seçilen düğüme kadar olan yolu döndürür.
+removeSelectionPat h(yol: TreePath): void n+r emoveSelectionPat hs(yollar: TreePath[]): void	Kökten seçilen tüm düğümlere giden yolları döndürür.
	İlk seçilen TreePath'teki son düğümü döndürür.
	Şu anda görüntülenen satır sayısını döndürür.
	Belirtilen yoldaki düğümü kaldırır.
	Belirtilen yollardaki düğümleri kaldırır.

**Şekil 40.20**

*JTree sınıfı, ağaçlar oluşturmak, özelleştirmek ve manipüle etmek için kullanılır.*

JTree sınıfı, ağaçlar oluşturmak için yedi yapıcı içerir. Bir ağaç, argüman almayan yapıcısı, bir ağaç modeli, bir ağaç düğümü, bir Hashtable, bir dizi veya bir vektör kullanılarak oluşturulabilir. Argüman almayan yapıcı kullanılarak, Şekil 40.18'de gösterildiği gibi bir örnek ağaç oluşturulur. Bir Hashtable, bir dizi veya bir vektör kullanarak, bir kök oluşturulur ancak gösterilmez. Bir Hashtable'daki tüm anahtarlar, bir dizideki tüm nesneler ve bir vektördeki tüm elemanlar, kökün çocukları olarak ağaç içine eklenir. Kökün gösterilmesini istiyorsanız, kökGörünür özelliğini true olarak ayarlayın.

Yol seçimi ile ilgili tüm yöntemler, §40.12'de ele alınacak olan `TreeSelectionModel` arayüzünde de tanımlanmıştır, "TreePath ve TreeSelectionModel."

Liste 40.12, varsayılan bir ağaç oluşturan, nesnelerden oluşan bir diziden oluşturulan, bir vektörden oluşturulan ve bir hash tablosundan oluşturulan dört ağacı örnek olarak vermektedir; bu, Şekil 40.21'de gösterilmiştir. Kullanıcının kökGörünür, satırYüksekliği ve kökTutacaklarını dinamik olarak ayarlamasına imkan tanın.



Şekil 40.21

Bir ağaçta kökGörünür, satırYüksekliği ve kökTutacaklarını dinamik olarak ayarlayabilirsiniz.

Liste 40.12 SimpleTreeDemo.java

<margin note line 9: ağaç 1>  
<margin note line 12: ağaç 2>  
<margin note line 16: ağaç 3>  
<margin note line 22: ağaç 4>  
<margin note line 43: ağaç>  
<margin note line 63: combo kutusu dinleyicisi>  
<margin note line 86: spinner dinleyicisi>  
<margin note line 97: ana yöntem atlandı>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.event.*'i içe aktarın;
5 java.util.*'i içe aktarın;
6
7 genel sınıf SimpleTreeDemo textendst JApplet {
8 // Varsayılan bir ağaç oluşturun
9 özel JTree jTree1 = tnew JTree();
10
11 // Bir Dizi Nesne ile bir ağaç oluşturun.
12 özel JTree jTree 2 = yeni JTree(yeni String[]
13 {t"köpek",t "inek",t "kedi",t "domuz",t "tavşan"});
14
15 // Bir Hashtable ile bir ağaç oluşturun
16 özel Vector<Object> vektör = t new Vector<Object>(Arrays.asList(
17 newt Nesne[] {t "kırmızı",t "yeşil",t "siyah",t "beyaz",t "mor"}));
18 özel JTree jTree3 = newt Tree(vektör);
19
20 özel Hashtable<Integer, String> hashtable =
21 yeni Hashtable<Integer, String>();
22 özel JTree jTree4;
23
24 // Kök görünürlük seçimi için bir kombinasyon kutusu oluşturun
25 özel JComboBox jcbRootVisible = yeni JComboBox(
26 yeni String[] {"false", "true"});
27
28 // showRootHandles seçimi için bir kombinasyon kutusu oluşturun
```

```

29 özel JComboBox jcboShowsRootHandles = yeni JComboBox (
30 yeni String[] {"false", "true"});
31
32 // Satır yüksekliği seçimi için bir spinner oluşturun
33 özel JSpinner jSpinnerRowHeight = new JSpinner (
34 yeni SpinnerNumberModel (16, 1, 50, 1));
35
36 genel SimpleTreeDemo() {
37 jTree1.setRootVisible(tfalset);
38
39 hashtable.put (1, "kırmızı");
40 hashtable.put (2, "yeşil");
41 hashtable.put (3, "mavi");
42 hashtable.put (4, "sarı");
43 jTree4 = yeni JTree (hashtable);
44
45 JPanel panel1 = yeni JPanel (new GridLayout (1, 4));
46 panel1.add(tnewt JScrollPane(jTree1)); n 47 panel1.a
47 dd(tnewt JScrollPane(jTree2)); n 48 panel1.add(tnewt JS
48 crollPane(jTree3)); n 49 panel1.add(tnewt JScrollPane(jT
49 ree4));
50
51 JPanel panel2 = yeni JPanel ();
52 panel2.add(tnewt JLabel(t"rootVisible")); n 53 panel2.a
53 dd(jcboRootVisible);
54 panel2.add(tnewt JLabel(t"rowHeight")); n 55 panel
55 2.add(jSpinnerRowHeight);
56 panel2.add(tnewt JLabel(t"showsRootHandles")); n 57 panel
57 2.add(jcboShowsRootHandles);
58
59 add(panel1, BorderLayout.CENTER); n 60 add(pa
60 nel2, BorderLayout.SOUTH);
61
62 // Dinleyicileri kaydet
63 jcboRootVisible.addActionListener(tnewt ActionListener) {
64 public void actionPerformed(ActionEvent e) {
65 boolean rootVisible =
66 jcboRootVisible.getSelectedItem().equals(t"true"); n 67 jTree1.setRootVisibl
67 e(rootVisible);
68 jTree2.setRootVisible(rootVisible); n 69 jTree3.setRootVis
68 ible(rootVisible); n 70 jTree4.setRootVisible(rootVisible); n 7
69 1 }
70
71
72 });
73
74 jcboShowsRootHandles.addActionListener(tnewt ActionListener) { n 75 @Override
75
76 public void actionPerformed(ActionEvent e) {
77 boolean showRootHandles =
78 jcboShowsRootHandles.getSelectedItem().equals(t"true"); n 79 jTree1.setShowsR
79 ootHandles(showsRootHandles);
80 jTree2.setShowsRootHandles(showsRootHandles); n 81 jTree3.setSh
80 owsRootHandles(showsRootHandles); n 82 jTree4.setShowsRootHandle
81 s(showsRootHandles); n 83 }
82
83
84 });
85
86 jSpinnerRowHeight.addChangeListener(tnewt ChangeListener() {
87 public void stateChanged(ChangeEvent e) {
88 int yükseklik =
89 ((Tamsayı)(jSpinnerRowHeight.getValue())).intValue(); n 90 jTree1.setRowHeight(h
89 eight);
90 jTree2.setRowHeight(yükseklik); n 92 jTree3.set
91 RowHeight(height); n 93 jTree4.setRowHeight(hei
92 ght); n 94 }
93
94 }

```

```
95 }); n 96 } n
97 }
```

Bu örnekte dört ağaç oluşturulmuştur. İlk ağaç, varsayılan örnek ağaç ile (satır 9) argüman almayan yapıcı kullanılarak oluşturulmuştur. İkincisi, bir nesne dizisi kullanılarak (satır 12-13) oluşturulmuştur. Tüm dizideki nesneler kökün çocukları haline gelir. Üçüncüsü bir vektör kullanılarak oluşturulur (satırlar 16-18). Vektördeki tüm elemanlar kökün çocukları haline gelir. Dördüncüsü bir hash tablosu kullanılarak oluşturulur (satırlar 39-43). Bir Hashtable, bir Map gibidir. Hashtable, Java 2'den daha önce tanıtılmıştır ve o zamandan beri Map ile değiştirilmiştir. Java API'sinde (örneğin, JTree) kullanılır, Java 2'den önce geliştirilmiştir. Hash tablosunun anahtarları kökün çocukları haline gelir.

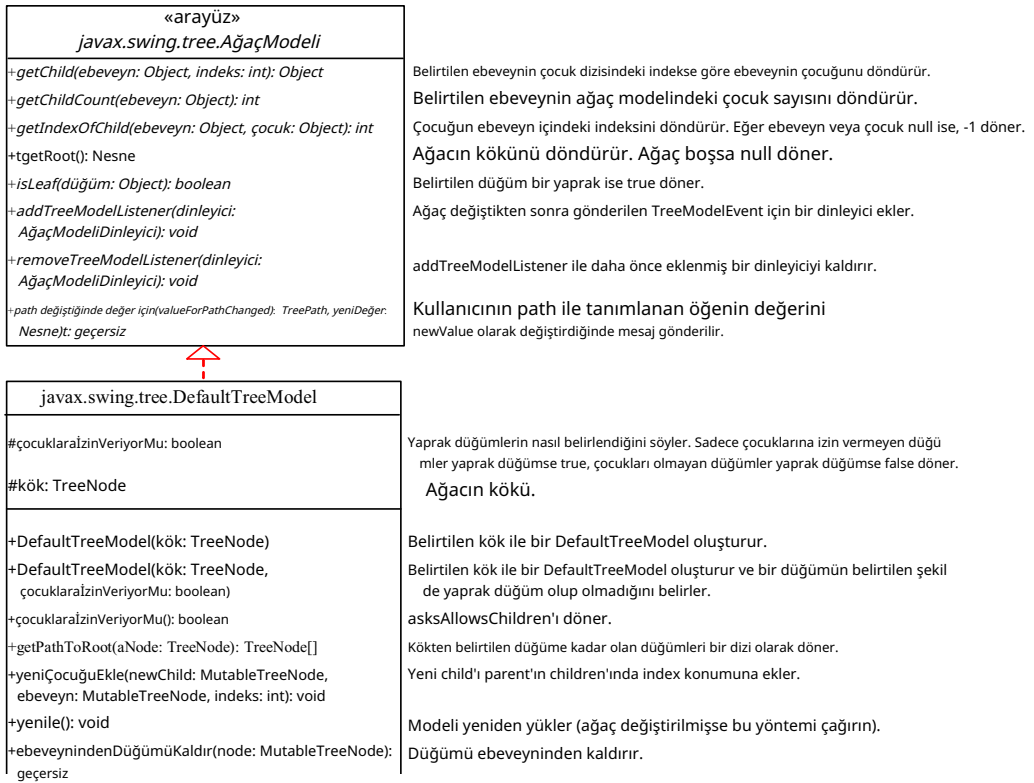
JTree doğrudan kaydırmayı desteklemez. Kaydırılabilir bir ağaç oluşturmak için bir JScrollPane oluşturun ve JTree'nin bir örneğini kaydırma paneline ekleyin (satırlar 46-49).

Örnek, kökün görünür olup olmadığını ve kök kollarının iki açılır kutudan görünür olup olmadığını belirtmenizi sağlar (satırlar 63-83). Ayrıca, bir spinner'da düğümün satır yüksekliğini belirtmenize de olanak tanır (satırlar 85-94).

#### 40.10 TreeModel ve DefaultTreeModel

TreeModel arayüzü tüm ağacı temsil eder. ListModel veya TableModel'den farklı olarak, TreeModel doğrudan ağaç verilerini saklamaz veya yönetmez. TreeModel, ağaç hakkında yapısal bilgileri içerir ve ağaç verileri TreeNode tarafından saklanır ve yönetilir.

DefaultTreeModel, TreeNodes kullanan TreeModel için somut bir uygulamadır. Şekil 40.22, TreeModel'i ve VarsayılanAğaçModeli.



**Şekil 40.22**

*TreeModel*, tüm bir ağacı temsil eder ve *DefaultTreeModel* bunun somut uygulamasıdır.

Bir ağaç oluşturulduktan sonra, getModel yöntemi kullanılarak ağaç modelini elde edebilirsiniz. Liste 40.13, ağaç modelini kullanarak bir ağaçtaki tüm düğümleri dolaşan bir örnek verir. Satır 1, JTree'nin varsayılan örnek düğümleri ile argüman almayan yapıcısını kullanarak bir ağaç oluşturur, Şekil 40.18'de gösterildiği gibi. Ağacın modeline satır 4'te erişilir. Satır 5, ağaçtaki düğümleri dolaşmak için dolaşım yöntemini çağırır.

**Listeleme 40.13 TestTreeModel.java**

<kenar notu satır 3: varsayılan ağaç>  
 <kenar notu satır 4: ağaç modeli>  
 <kenar notu satır 5: getRoot>  
 <kenar notu satır 11: yaprak mı?>  
 <kenar notu satır 12: getChildCount>  
 <kenar notu satır 13: getChild>

```
1 genel sınıf TestTree Modeli {
2 public static void main(String[] args) {
3 javax.swing.JTree jTree1 = t newt javax.swing.JTree();
```

```

4 javax.swing.tree.TreeModel modeli = jTree1.getModel(); n 5 geçiş(model,
model.getRoot());
6 }
7
8 özel statik voidt geçişi
9 (javax.swing.tree.TreeModel modeli, Nesne kökü) {n 10 System.out.pri
nt(root + t " "t);
11 ift (model.isLeaf(root)) t returnt;
12 için (int i = 0; i < model.getChildCount(root); i++) {
13 geçiş(model, model.getChild(kök, i)); n 14 }

15 } n 16
}

```

<Çıktı>

JTree renkleri mavi mor kırmızı sarı spor basketbol futbol  
amerikan futbolu hokey yiyecek soslisli sandviç pizza ravioli muzlar

<Son Çıktı>

Traversalsal metodu ağacın kökünden başlar. Kök, getRoot metodunu çağırarak elde edilir (satır 5). Eğer kök bir yapraksa, metod döner (satır 11). Aksi takdirde, kökün çocuklarından başlamak için transversal metodunu özyinelem eli olarak çağırır (satır 13).

#### 40.11 TreeNode, MutableTreeNode ve DefaultMutableTreeNode

TreeModel tüm ağacı temsil ederken, TreeNode ağacın bir tek düğümünü saklar. MutableTreeNode, düğümün içeriğini değiştirmek, bir çocuk düğüm eklemek ve kaldırmak, yeni bir ebeveyn ayarlamak ve düğümü kendisini kaldırmak için ek yöntemler ile TreeNode'un bir alt arayüzünü tanımlar.

DefaultMutableTreeNode, çocukların bir vektörde saklandığı bir MutableTreeNode'un somut bir uygulamasıdır ve düğüm oluşturma, bir düğümün ebeveynini ve çocuklarını inceleme ve değiştirme, ayrıca düğümün ait olduğu ağacı inceleme işlemlerini sunar. Normalde, bir ağaç düğümü oluşturmak için DefaultMutableTreeNode kullanmalısınız. Şekil 40.23, TreeNode, MutableTreeNode ve DefaultMutableTreeNode'u göstermektedir.

«arayüz» <i>javax.swing.tree.TreeNode</i>
<i>+tchildren(): java.util Enumeration</i> <i>+tgetAllowsChildren(): boole</i> <i>+tgetChildAt(childIndex: int): TreeNode</i> <i>+tgetChildCount(): int</i> <i>+tgetIndex(düğüm: TreeNode): int</i> <i>+tgetParent(): TreeNode</i> <i>+tisLeaf(): boole</i>

Bu düğümün çocuklarını döndürür.  
Eğer bu düğüm çocuk alabiliyorsa true döndürür.  
childIndex indeksindeki çocuk TreeNode'u döndürür.  
Bu düğümün altındaki çocuk sayısını döndürür.  
Belirtilen düğümün mevcut düğümün çocukları içindeki indeksini döndürür.  
Bu düğümün ebeveynini döndürür.  
Eğer bu düğüm bir yapraks true döndürür.

«arayüz» <i>javax.swing.tree.MutableTreeNode</i>
<i>+insert(child: MutableTreeNode, index: int): void n+re</i> <i>move(index: int): void</i> <i>+remove(düğüm: MutableTreeNode): void</i> <i>n+removeFromParent(): void</i> <i>+setParent(newParent: MutableTreeNode): void n+s</i> <i>etUserObject(object: Object): void</i>

Belirtilen çocuğu bu düğüm altında belirtilen indekse ekler.  
Belirtilen indeksteki çocuğu bu düğümün çocuk listesinden kaldırır.  
Bu düğümü, bu düğümün çocuk listesinden kaldırır.  
Bu düğümü, üst düğümünden kaldırır.  
Bu düğümün üst düğümünü belirtilen newParent olarak ayarlar.  
Bu düğümün kullanıcı nesnesini belirtilen nesneye sıfırlar.

<i>javax.swing.tree.DefaultMutableTreeNode</i>
<i>#allowsChildren: Boolean n</i> <i>#parent: MutableTreeNode</i> <i>n#userObject: Nesne</i>
<i>+DefaultMutableTreeNode() n</i> <i>+DefaultMutableTreeNode(userObject: Object) n+</i> <i>DefaultMutableTreeNode(userObject: Object, nall</i> <i>owsChildren: boolean)</i> <i>+add(newChild: MutableTreeNode) n+</i> <i>getChildAfter(aChild: TreeNode): TreeNode n+get</i> <i>ChildBefore(aChild: TreeNode): TreeNode n+getFir</i> <i>stChild(): TreeNode</i> <i>+getLastChild(): TreeNode n</i> <i>+getFirstLeaf(): DefaultMutableTreeNode n+</i> <i>getLastLeaf(): DefaultMutableTreeNode n+g</i> <i>etNextLeaf(): DefaultMutableTreeNode n+get</i> <i>PreciousLeaf(): DefaultMutableTreeNode n+getLe</i> <i>afCount(): int</i> <i>+getDepth(): int</i> <i>+getLevel(): int</i> <i>+getNextNode(): DefaultMutableTreeNode n+</i> <i>getPreciousNode(): DefaultMutableTreeNode n+g</i> <i>etSiblingCount(): int</i> <i>+getNextSibling(): DefaultMutableTreeNode n+g</i> <i>etPath(): TreeNode[]</i> <i>+getRoot(): TreeNode n</i> <i>+isRoot(): boolean</i> <i>+breadthFirstEnumeration(): Sayım n+</i> <i>derinlikFirstEnumeration(): Sayım n+</i> <i>postorderEnumeration(): Sayım n+p</i> <i>reorderEnumeration(): Sayım</i>

Düğümün çocukları olmasına izin veriliyorsa doğru.  
Bu düğümün ebeveynini saklar.  
Bu düğümün içeriğini saklar.  
Kullanıcı nesnesi olmadan bir ağaç düğümü oluşturur ve çocuklara izin verir.  
Belirtilen kullanıcı nesnesi ile bir ağaç düğümü oluşturur ve çocuklara izin verir.  
Belirtilen kullanıcı nesnesi ve çocukların izinli olup olmadığını belirtmek için belirtilen mod ile bir ağaç düğümü oluşturur.  
Belirtilen düğümü bu düğümün çocuk vektörünün sonuna ekler.  
Bu iki yöntem, belirtilen çocuğun bu düğümün çocuk vektöründeki bir sonraki (önceki) kardeşini döndürür.  
Bu iki yöntem, bu düğümün çocuğunun vektöründeki ilk (son) çocuğunu döndürür.  
Bu dört yöntem, bu düğümün bir alt nesli olan ilk (son, sonraki ve önceki) yaprağı döndürür. İlk (son, sonraki ve önceki) yaprak, özinelemeli olarak ilk (son, sonraki ve önceki) çocuğun ilk (son, sonraki ve önceki) yaprağı olarak tanımlanır.  
Bu düğümün altındaki toplam yaprak sayısını döndürür.  
Bu düğümün kök olduğu ağacın derinliğini döndürür.  
Kökten bu düğümüne olan mesafeyi döndürür.  
Bu düğümün öncelikli dolaşımında bu düğümün sonra (önce) gelen düğümü döndürür.  
Bu düğümün kardeş sayısını döndürür.  
Bu düğümün ebeveyninin çocuk vektöründeki bir sonraki kardeşini döndürür.  
Kökten bu düğümüne giden yolu döndürür.  
Bu düğümü içeren ağacın kökünü döndürür.  
Bu düğüm ağacın kökü ise true döndürür.  
Bu dört yöntem, bu düğümün kök olduğu alt ağacı genişlik öncelikli sırayla (derinlik öncelikli sıra, sonradan, öncelikli) dolaşan bir enumerasyon oluşturur ve döndürür. Bu dolaşım §25.2.4, "Ağaç Dolaşımı" bölümünde tartışılmıştır.

## Şekil 40.23

Ağaç Düğümü bir düğümü temsil eder.

NOT

**<kenar notu: derinlik öncelikli gezinti>**

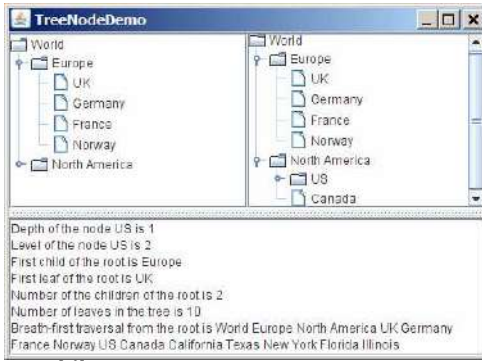
Graf teorisinde, derinlik öncelikli gezinti, öncelikli gezinti ile aynı şekilde tanımlanır, ancak DefaultMutableTreeNode'daki depthFirstEnumeration() yönteminde, bu, sonradan gezinti ile aynıdır.

NOT

**<kenar notu: ağaç oluşturma>**

Bir kökten yeni JTree(TreeNode) kullanarak veya bir modelden yeni JTree(TreeModel) oluşturabilirsiniz. Bir ağaç modeli oluşturmak için, önce ağacın kökünü temsil etmek için bir TreeNode örneği yaratır ve ardından kökle uyumlu bir DefaultTreeModel örneği oluşturursunuz.

Liste 40.14, dünya, kıtalar, ülkeler ve eyaletleri görüntülemek için iki ağaç oluşturan bir örnek vermektedir. İki ağaç aynı düğümleri paylaştığı için aynı içeriği görüntüler. Program ayrıca ağacın özelliklerini bir metin alanında gösterir, bu da Şekil 40.24'te gösterilmiştir.



**Şekil 40.24**

İki ağacın verileri aynıdır çünkü kökleri aynıdır.

**Liste 40.14 TreeNodeDemo.java**

**<margin note line 9: ağaç düğümleri>**

**<margin note line 12: çocuk ekle>**

**<margin note line 19: çocuk ekle>**

**<margin note line 62: ana yöntem atlandı>**

```
1 java.awt.*'u içe aktarın;
2 javax.swing.*'i içe aktarın;
3 javax.swing.tree.*'yi içe aktarın;
4 java.util.*'i içe aktarın;
5
6 genel sınıf TreeNodeDemo textendst JApplet {
7 genel TreeNode Demosu() {
8 // İlk ağacı oluştur
9 DefaultMutableTreeNode kökü, Avrupa, Kuzey Amerika, ABD;
10
11 europe = new DefaultMutableTreeNode("Avrupa");
12 europe.add(new DefaultMutableTreeNode("İngiltere"));
13 europe.add(new DefaultMutableTreeNode("Almanya"));
```



```

14 europe.add(new DefaultMutableTreeNode("Fransa"));
15 europe.add(new DefaultMutableTreeNode("Norveç"));
16
17 Kuzey Amerika = new DefaultMutableTreeNode("Kuzey Amerika");
18 us = new DefaultMutableTreeNode("ABD");
19 us.add(new DefaultMutableTreeNode("Kaliforniya"));
20 us.add(new DefaultMutableTreeNode("Texas"));
21 us.add(new DefaultMutableTreeNode("New York"));
22 us.add(new DefaultMutableTreeNode("Florida"));
23 us.add(new DefaultMutableTreeNode("Illinois"));
24 northAmerica.add(us);
25 northAmerica.add(new DefaultMutableTreeNode("Kanada"));
26
27 root = new DefaultMutableTreeNode("Dünya");
28 root.add(europe);
29 root.add(kuzeyAmerika);
30
31 JPanel paneli = t yeni JPanel();
32 panel.setLayout(new GridLayout(1,t,2t)); n 33 panel.
add(new JScrollPane(new JTree(root))); n 34 panel.add(new JS
crollPane(new JTree(new DefaultTreeModel(root))));
35
36 JTextArea jtaMessage = new JTextArea();
37 jtaMessage.setWrapStyleWord(true); n 38 jtaMes
sage.setLineWrap(true); n 39 add(new JSplit
Pane(JSplitPane.VERTICAL_SPLIT, n 40 panel, t new JScrollPane(jta
Message)), BorderLayout.CENTER);
41
42 // Ağaç bilgilerini al
43 jtaMessage.append("ABD düğümünün derinliği " + us.getDepth()); 44 j
taMessage.append("ABD düğümünün seviyesi " + us.getLevel()); 45 jtaMessage.append
("Kökün ilk çocuğu " +
46 root.getFirstChild()); 47 jtaMes
sage.append("Kökün ilk yaprağı " + 48 root.getFirstLeaf());
49
50 jtaMessage.append("Kökün çocuk sayısı " + 50 root.getChildCount());
51
52 jtaMessage.append("Ağaçtaki yaprak sayısı " + 52 root.getLea
fCount());
53 Dize genişliği ilkAramaSonucu = t""t;
54
55 // Genişlik öncelikli gezinti
56 Numaralandırma bf = root.breadthFirstEnumeration();
57 while (bf.hasMoreElements())
58 breadthFirstSearchResult += bf.nextElement().toString() +
" "; 59 jtaMessage.append("Kökten genişlik öncelikli gezinti " 60 + breadthFirstSe
archResult);
61 } n 62
}

```

Bir JTree, bir TreeNode root (satır 33) veya bir TreeModel (satır 34) kullanılarak oluşturulabilir, hangisi uygunsa. Bir TreeModel aslında bir TreeNode root (satır 34) kullanılarak oluşturulur. İki ağaç aynı içeriğe sahiptir çünkü kök aynıdır. Ancak, iki JTree nesnesinin farklı olduğunu ve dolayısıyla TreeModel nesnelerinin de farklı olduğunu belirtmek önemlidir, her iki ağaç da ayna köke sahiptir.

Bir ağaç, düğümleri ağaca ekleyerek oluşturulur (satırlar 9-29). Her düğüm, DefaultMutableTreeNode sınıfı kullanılarak oluşturulur. Bu sınıf, ağacı manipüle etmek için birçok yöntem sağlar (örneğin, bir çocuk eklemek, bir çocuğu kaldırmak) ve ağaç hakkında bilgi almayı sağlar (örneğin, seviye, derinlik, çocuk sayısı, yaprak sayısı,

geçişler). Bu yöntemlerin kullanımına dair bazı örnekler  
43-60. satırlarda verilmiştir.

Bu örnekte gösterildiği gibi, genellikle doğrudan `TreeModel` kullanmanıza gerek yoktur. `DefaultMutableTreeNode` kullanmak yeterlidir, çünkü ağaç verileri `DefaultMutableTreeNode` içinde saklanmaktadır, ve `DefaultMutableTreeNode`, ağacı değiştirmek ve ağaç bilgilerini elde etmek için gerekli tüm yöntemleri içerir.

## 40.12 `TreePath` ve `TreeSelectionModel`

`JTree` sınıfı, ağaç yollarını seçmek için yöntemler içerir. `TreePath` sınıfı, bir ata ile bir soyundan birine giden bir yolu temsil eder. Şekil 40.25, `TreePath`'ı göstermektedir.

javax.swing.tree.TreePath	
+TreePath(singlePath: Object) n	Tek bir öge içeren bir TreePath oluşturur.
+TreePath(path: Object[]) n	Bir dizi nesneden bir yol oluşturur.
+getLastPathComponent(): Object n	Bu yolun son bileşenini döndürür.
+getParentPath(): TreePath n	Son yol bileşeni hariç tüm bileşenleri içeren bir yol döndürür.
+getPath(): Object[]	Bu TreePath'in bileşenlerini içeren sıralı bir nesne dizisi döndürür.
+getPathComponent(element: int): Nesne n+	Belirtilen indeksteki yol bileşenini döndürür.
getPathCount(): int	Yolda bulunan elemanların sayısını döndürür.
+isDescendant(aTreePath: TreePath): boolean n+	aTreePath bu TreePath'teki tüm bileşenleri içeriyorsa true döndürür.
pathByAddingChild(child: Object): TreePath	Bu TreePath'in tüm elemanlarını ve child'ı içeren yeni bir yol döndürür.

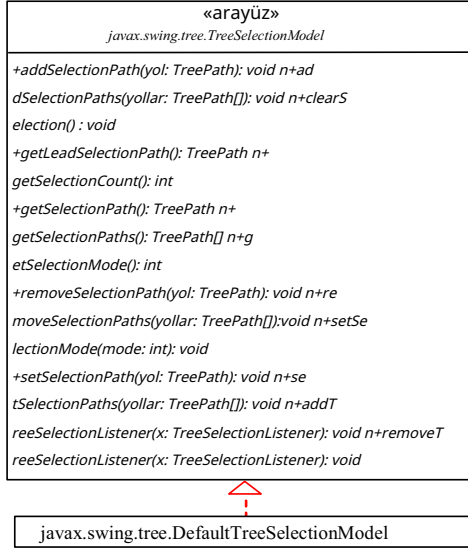
Şekil 40.25

*TreePath*, bir ağaçtaki bir ata ile bir torun arasındaki yolu temsil eder.

**<margin note: ağaç yollarını elde et>**

Bir `TreePath`'i tek bir nesneden veya bir dizi nesneden oluşturabilirsiniz, ancak genellikle `TreePath` örnekleri `JTree` ve `TreeSelectionModel`'deki yöntemlerden döndürülür. Örneğin, `JTree`'deki `getLeadSelectionPath()` yöntemi, kökten seçilen düğüme kadar olan yolu döndürür. Bir ağaç yolundan düğümleri çıkarmanın birçok yolu vardır. Genellikle, yolun son düğümünü elde etmek için `getLastPathComponent()` yöntemini kullanırsınız ve ardından yukarıdaki bağlantıdan tüm düğümleri almak için `getParent()` yöntemini kullanırsınız.

Ağaç düğümlerinin seçimi, Şekil 40.26'da gösterildiği gibi `TreeSelectionModel` arayüzünde tanımlanmıştır. `DefaultTreeSelectionModel` sınıfı, mevcut seçimi temsil eden `TreePath` nesnelerinin bir dizisini koruyan `TreeSelectionModel`'in somut bir uygulamasıdır. Seçilen son `TreePath`, *lead path* olarak adlandırılır ve `getLeadSelectionPath()` yöntemi kullanılarak elde edilebilir. Tüm seçim yollarını elde etmek için, bir dizi ağaç yolu döndüren `getSelectionPaths()` yöntemini kullanın.



Belirtilen TreePath'i mevcut seçime ekler.  
Belirtilen TreePath'leri mevcut seçime ekler.  
Seçimi temizler.  
Seçimdeki son yolu döndürür.  
Seçimdeki yol sayısını döndürür.  
Seçimdeki ilk yolu döndürür.  
Seçimdeki tüm yolları döndürür.  
Mevcut seçim modunu döndürür,  
Yolu seçimden kaldırır.  
Seçimden yolları kaldırır.  
Seçim modunu ayarlar.  
Seçimi yola ayarlar.  
Seçimi yollara ayarlar.  
Bir TreeSelectionListener kaydeder.  
Bir TreeSelectionListener'ı kaldırır.

**Şekil 40.26**

*Ağaç Seçim Modeli, bir ağaçtaki seçimi yönetir ve Varsayılan Ağaç Seçim Modeli bunun somut bir uygulamasıdır.*

**<kenar notu: ağaç seçim modları>**

Ağaç Seçim Modeli üç seçim modunu destekler: ardışık seçim, ardışık olmayan seçim ve tek seçim. Tek seçim yalnızca bir ögenin seçilmesine izin verir. Ardışık seçim birden fazla seçime izin verir, ancak seçilen öğeler ardışık olmalıdır. Ardışık olmayan seçim en esnek olanıdır; belirli bir anda herhangi bir ögenin seçilmesine izin verir. Varsayılan ağaç seçim modu ardışık olmayan seçimdir. Bir seçim modunu ayarlamak için TreeSelectionModel'deki setSelectionMode(int mode) yöntemini kullanın. Üç modun sabitleri şunlardır:

- BİTİŞİK\_AĞAÇ\_SEÇİMİ
- KESİK\_AĞAÇ\_SEÇİMİ
- TEK\_AĞAÇ\_SEÇİMİ

NOT

**<kenar notu: TreeSelectionModel'i atla>**

Bir JTree oluşturduğunuzda, otomatik olarak bir DefaultTreeSelectionModel oluşturulur ve bu nedenle nadiren TreeSelectionModel'in bir örneğini açıkça oluşturmanız gerekir. TreeSelectionModel'deki yöntemlerin çoğu JTree'de de bulunduğundan, seçim yollarını alabilir ve TreeSelectionModel ile doğrudan ilgilenmeden seçimi işleyebilirsiniz.

Liste 40.15, bir seçilen yolu veya seçilen yolları ağaçta gösteren bir örnek vermektedir. Kullanıcı bir düğümü veya birden fazla düğümü seçebilir ve *Yol Göster* butonuna tıklayarak ilk seçilen yolun özelliklerini veya *Yolları Göster* butonuna tıklayarak tüm yolları görüntüleyebilir.

metin alanındaki seçilen yollar, Şekil 40.27'de gösterildiği gibi.  
*Yolu Göster* butonu, son düğümden köke kadar bir yolu görüntüler.



**Şekil 40.27**  
Seçilen yol(lar) işleniyor.

#### Listeleme 40.15 TestTreePath.java

<kenar notu satır 7: varsayılan ağaç>  
<kenar notu satır 8: metin alanı>  
<kenar notu satır 9: Yolu Göster butonu>  
<kenar notu satır 10: Yolları Göster butonu>  
<kenar notu satır 13: bölünmüş panel>  
<kenar notu satır 23: Yolu Göster butonu>  
<kenar notu satır 26: seçilen yol>  
<kenar notu satır 28: yol sayısı>  
<kenar notu satır 32: son düğüm>  
<kenar notu satır 35: üst öğeyi al>  
<kenar notu satır 39: Yolları Göster butonu>  
<kenar notu satır 43: seçilen yollar>  
<kenar notu satır 44: bir yolu görüntüle>  
<kenar notu satır 48: ana yöntem atlandı>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.tree.*'yi içe aktarın;
5
6 genel sınıf TestTreePath textendst JApplet {
7 özel JTree jTree = t newt JTree();
8 özel JTextArea jtag Çıktı = t newt JTextArea();
9 private JButton jbtShowPath = new JButton("Yolu Göster");
10 private JButton jbtShowPaths = new JButton("Yolları Göster");
11
12 genel TestTreePath() {
13 JSplitPane splitPane = t newt JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
14 newt JScrollPane(jTree), t newt JScrollPane(jtaOutput));
15
16 JPanel paneli = t newt JPanel(); n 17 panel.
add(jbtShowPath); n 18 panel.add(jbtS
howPaths);
19
20 add(splitPane, BorderLayout.CENTER); n 21 add(pan
el, BorderLayout.NORTH);
22
23 jbtShowPath.addActionListener(newt ActionListener() {
24 @Geçersiz Kil
25 public void actionPerformed(ActionEvent e) {
26 TreePath yolu = jTree.getSelectionPath();
```

```

27 jtaOutput.append("Tek bir yolu işliyor"); 28 jtaOutput
t.append("# eleman sayısı: " + path.getPathCount()); 29 jtaOutput.append("son ele
man: "
30 + path.getLastPathComponent()); 31 jtaO
utput.append("Yoldaki son düğümden köke: "); 32 TreeNode node = (TreeNode)path.getL
astPathComponent();
33 while (düğüm != null) {
34 jtaOutput.append(node.toString() + t "t); n 35 düğüm = node.ge
tParent();
36 } n 37 }}};

38
39 jbtShowPaths.addActionListener (new ActionListener() {
40 @Geçersiz Kil
41 public void actionPerformed(ActionEvent e) {
42 jtaOutput.append("Birden fazla yol işleniyor"); 43 jav
ax.swing.tree.TreePath[] paths = jTree.getSelectionPaths();
44 while (tintt i = t0t; i < yollar.uzunluk; i++)
45 jtaOutput.append(paths[i].toString() + t "n"t); n46)}};

47 } n 48
}

```

getSelectionPath() metodu JTree'den çağrıldığında, TreePath döndürür (satır 25). Yoldaki ilk düğüm her zaman ağacın köküdür. TreePath'den çağrılan getPathCount() yoldaki düğümlerin sayısını döndürür (satır 27). TreePath'den çağrılan getLastPathComponent() yoldaki son düğümü döndürür (satır 29). Dönen düğüm tipi Object'tir. getParent() metodunu TreeNode'dan çağırmak için yi bir TreeNode'a (satır 31) dönüştürmeniz gerekir (satır 34).

getSelectionPath() metodu (satır 25) ilk seçilen yolu döndürürken, getSelectionPaths() metodu (satır 41) tüm seçilen yolları bir yol dizisi içinde döndürür.

### 40.13 Vaka Çalışması: Ağaçları Değiştirme

İçeriği aynı olan iki ağaç oluşturmak için bir program yazın: dünya, kıtalar, ülkeler ve eyaletler, Şekil 40.28'de gösterildiği gibi. Soldaki ağaç için, kullanıcının bir seçim modu seçmesine, düzenlenip düzenlenemeyeceğini belirtmesine, ilk seçilen düğümün altına yeni bir çocuk eklemesine ve tüm seçilen düğümleri kaldırmasına izin verin.



Şekil 40.28

Bir düğümü yeniden adlandırabilir, bir çocuk ekleyebilir ve bir ağaçta düğümleri dinamik olarak kaldırabilirsiniz.

Seçim modunu selectionMode açılır kutusundan seçebilirsiniz. Sol ağaç düğümlerinin düzenlenip düzenlenemeyeceğini editable onay kutusundan belirtebilirsiniz.

Bir düğmeye tıkladığınızda, eğer sol ağaçta şu anda seçili düğüm yoksa, bir mesaj iletişim kutusu görüntülenir, Şekil 40.29a'da gösterildiği gibi. Seçili Düğüm için Çocuk Ekle düğmesine tıkladığınızda, kullanıcıdan seçili düğüm için bir çocuk adı girmesini istemek üzere bir giriş iletişim kutusu görüntülenir, Şekil 40.29b'de gösterildiği gibi. Yeni düğüm, ilk seçili düğümün çocuğu olur.

Seçili Düğümleri Kaldır düğmesine tıkladığınızda, sol ağaçtaki tüm seçili düğümler kaldırılır.



Şekil 40.29

Ağaç yapısına yeni bir düğüm ekleyebilirsiniz.

Liste 40.16 programı vermektedir.

Liste 40.16 ModifyTree.java

<margin note line 8: açılır kutu>  
<margin note line 13: onay kutusu>  
<margin note line 16: düğmeler>  
<margin note line 21: ağaçlar>  
<margin note line 25: ağaç düğümleri>  
<margin note line 27: düğümleri doldur>  
<margin note line 56: jTree1 oluştur>  
<margin note line 57: jTree2 oluştur>  
<margin note line 69: seçim modunu seç>  
<margin note line 74: seçim modunu ayarla>  
<margin note line 86: düzenlenebilir seç>  
<margin note line 89: düzenlenebilir ayarla>  
<margin note line 93: çocuk ekle>  
<margin note line 96: seçili düğümü al>  
<margin note line 111: yeni düğüm ekle>  
<margin note line 114: ağaç modelini yeniden yükle>  
<margin note line 119: düğümü kaldır>  
<margin note line 122: seçilen yolları al>  
<margin note line 140: düğümü kaldır>  
<margin note line 144: ağaç modelini yeniden yükle>  
<margin note line 149: ana yöntem atlandı>

```
1 java.aut.*'u içe aktarın;
2 java.awt.event.*'i içe aktarın;
3 javax.swing.*'i içe aktarın;
4 javax.swing.tree.*'yi içe aktarın;
5
6 genel sınıf Ağacı Değiştir textendst Applet {
7 // Seçim modlarını seçmek için bir kombinasyon kutusu oluştur
8 özel JComboBox jcb SelectionMode = t newt JComboBox(newt String[]{
9 "BİTİŞİK AĞAÇ SEÇİMİ", "BİTİŞİK OLMAYAN AĞAÇ SEÇİMİ",
10 "TEK AĞAÇ SEÇİMİ"});
11
12 // Düzenlenebilirliği belirtmek için bir onay kutusu oluştur
```

```

13 özel JCheckBox jchk Düzenlenebilir = new JCheckBox();
14
15 // İki buton oluştur
16 özel JButton jbtAdd =
17 yeni JButton("Seçilen Düğüm için Bir Çocuk Ekle");
18 özel JButton jbtRemove = yeni JButton("Seçilen Dğümleri Kaldır");
19
20 // İki ağaç tanımla
21 özel JTree jTree1, jTree2;
22
23 genel ModifyTree() {
24 // İlk ağacı oluştur
25 DefaultMutableTreeNode kök, avrupa, kuzeyAmerika, abd;
26
27 avrupa = yeni DefaultMutableTreeNode("Avrupa");
28 avrupa.add(yeni DefaultMutableTreeNode("İngiltere"));
29 avrupa.add(yeni DefaultMutableTreeNode("Almanya"));
30 avrupa.add(yeni DefaultMutableTreeNode("Fransa"));
31 avrupa.add(yeni DefaultMutableTreeNode("Norveç"));
32
33 kuzeyAmerika = new DefaultMutableTreeNode("Kuzey Amerika");
34 abd = yeni DefaultMutableTreeNode("ABD");
35 abd.add(new DefaultMutableTreeNode("Kaliforniya"));
36 abd.add(new DefaultMutableTreeNode("Teksas"));
37 abd.add(new DefaultMutableTreeNode("New York"));
38 abd.add(new DefaultMutableTreeNode("Florida"));
39 abd.add(new DefaultMutableTreeNode("Illinois"));
40 kuzeyAmerika.add(abd);
41 kuzeyAmerika.add(new DefaultMutableTreeNode("Kanada"));
42
43 kök = new DefaultMutableTreeNode("Dünya");
44 kök.add(avrupa);
45 kök.add(kuzeyAmerika);
46
47 jcbSelectionMode.setSelectedIndex(t1t);
48
49 JPanel p1 = new JPanel();
50 p1.add(new JLabel("seçimModu"));
51 p1.add(jcbSelectionMode);
52 p1.add(new JLabel("düzenlenebilir"));
53 p1.add(jchkEditable);
54
55 JPanel p2 = yeni JPanel(new GridLayout(1, 2));
56 p2.add(tnewt JScrollPane(jTree1 = tnewt JTree(root))); n 57 p2.add(tne
wt JScrollPane(jTree2 =
58 new JTree(new DefaultTreeModel(root))); // jTree1 ile aynı kök
59
60 JPanel p3 = t yeni JPanel(); n 61 p3.add(
jbtAdd);
62 p3.add(jbtRemove);
63
64 add(p1, BorderLayout.NORTH); n 65 add(
p2, BorderLayout.CENTER); n 66 add(p3, Bord
erLayout.SOUTH);
67
68 // Dinleyicileri kaydet
69 jcbSelectionMode.addActionListener(tnewt ActionListener() { n 70 @Override
71 public void actionPerformed(ActionEvent e) {
72 if (jcbSelectionMode.getSelectedItem().
73 equals(t"CONTIGUOUS_TREE_SELECTION"t)) n 74 jTree1.getSe
lectionModel().setSelectionMode(n 75 TreeSelectionMode.CONTIGUOU
S_TREE_SELECTION);
76 else if (jcbSelectionMode.getSelectedItem().
77 eşittir(t"DISCONTIGUOUS_TREE_SELECTION"t)) n 78 jTree1.getSel
ectionModel().setSelectionMode(

```

```

79 TreeSelectionMode.DISCONTIGUOUS_TREE_SELECTION);
80 başka
81 jTree1.getSelectionModel().setSelectionMode(n 82 TreeSelectionMode.SINGLE_TREE_SELECTION); n 83 }

84 });
85
86 jchkEditable.addActionListener(tnew ActionListener() { n 87 @Override

88 public void actionPerformed(ActionEvent e) {
89 jTree1.setEditable(jchkEditable.isSelected()); n 90 }

91 });
92
93 jbtAdd.addActionListener(tnew ActionListener() { n 94 @Override

95 public void actionPerformed(ActionEvent e) {
96 DefaultMutableTreeNode parent = (DefaultMutableTreeNode) n 97 jTree1.getLastSelectedPathComponent();

98 eğer (parent == null) {
99
100 JOptionPane.showMessageDialog(tnult,
101 "Soldaki ağaçta hiçbir düğüm seçilmedi");
102 geri dönüş;
103 }

104 // Yeni bir düğüm ekle
105
106 Dize nodeName = JOptionPane.showInputDialog(
107 null, "Ebeveyn için bir çocuk düğüm girin " + parent, "Bir Çocuk Ekle",
108 JOptionPane.QUESTION_MESSAGE);

109 // Yeni düğümü treeNode'un çocuğu olarak ekle
110
111 parent.add(new DefaultMutableTreeNode (nodeName));
112
113 // Yeni bir ağaç düğümü eklendiği için modeli yeniden yükle
114 ((DefaultTreeModel)(jTree1.getModel())).reload(); n115 ((DefaultTreeModel)(jTree2.getModel())).reload(); n116 }

117 });
118
119 jbtRemove.addActionListener(tnew ActionListener() {

120 public void actionPerformed(ActionEvent e) {
121 // Tüm seçilen yolları al
122 TreePath[] yollar = jTree1.getSelectionPaths();

123 if (yollar == null) {
124
125 JOptionPane.showMessageDialog(tnult,
126 "Soldaki ağaçta hiçbir düğüm seçilmedi");
127 geri dönüş;
128 }

129 // Tüm seçilen düğümleri kaldır
130
131 for (tintt i = t0t; i < yollar.uzunluk; i++) {
132 DefaultMutableTreeNode düğüm = (DefaultMutableTreeNode) n133 (paths[i].getLastPathComponent());

134 eğer (node.isRoot()) {
135 JOptionPane.showMessageDialog(tnult,
136 "Kökü kaldıramıyorum");
137 }
138 başka
139 node.removeFromParent(); n141 }

142
143 // Yeni bir ağaç düğümü eklendiği için modeli yeniden yükle
144 ((DefaultTreeModel)(jTree1.getModel())).reload();

```



```

145 ((DefaultTreeModel)(jTree2.getModel())).reload(); n146 }
147}); n148 } n
149 }

```

İki JTree nesnesi (jTree1 ve jTree2) aynı kök ile oluşturulmuştur (satırlar 56-58), ancak her birinin kendi TreeSelectionModel'i vardır. Kombinasyon kutusunda bir seçim modu seçtiğinizde, yeni seçim modu jTree1'in seçim modelinde ayarlanır (satırlar 69-83). jTree2 için seçim modu etkilenmez.

Editable onay kutusu işaretlendiğinde veya işaret kaldırıldığında, jTree1'deki editable özelliği buna göre ayarlanır. Eğer editable true ise, sol ağaçta bir düğümü düzenleyebilirsiniz.

Seçilen Düğüm için Çocuk Ekle butonuna tıkladığınızda, ilk seçilen düğüm ebeveyn olarak döndürülür (satırlar 93-94). Avrupa, Birleşik Krallık ve ABD'yi bu sırayla seçtiğinizi varsayalım; ebeveyn Avrupa'dır. Eğer parent null ise, sol ağaçta hiçbir düğüm seçilmemiştir (satırlar 96-100). Aksi takdirde, kullanıcıdan bir input iletişim kutusundan yeni bir düğüm girmesini isteyin (satırlar 103-105) ve bu düğümü parent'ın bir çocuğu olarak ekleyin (satır 108). Ağaç değiştirildiği için, her iki ağacın modellerinin değiştiğini bildirmek için reload() yöntemini çağırmanız gerekir (satırlar 111-112). Aksi takdirde, yeni düğüm jTree1 ve jTree2'de görüntülenmeyebilir.

Seçilen Düğümleri Kaldır butonuna tıkladığınızda, her seçilen düğüm için tüm ağaç yolları paths (satır 119) olarak elde edilir. Avrupa, Birleşik Krallık ve ABD'yi bu sırayla seçtiğinizi varsayalım; üç ağaç yolu elde edilir. Her yol kökten seçilen düğüme kadar uzanır. Hiçbir düğüm seçilmezse, paths null olur. Seçilen bir düğümü silmek, her seçilen ağaç yolundaki son düğümü silmek anlamına gelir (128-138). Bir yoldaki son düğüm, getLastPathComponent() kullanılarak elde edilir. Düğüm kökse, kaldırılması mümkün değildir (satırlar 132-135). removeFromParent() metodu bir düğümü kaldırır (satır 137).

#### 40.14 Ağaç Düğümü Görselleştirme ve Düzenleme

JTree, düğüm görselleştirmeyi bir renderere devreder. Tüm renderlar, aşağıdaki gibi tek bir metodu tanımlayan TreeCellRenderer arayüzünün örnekleridir: getTreeCellRendererComponent

```

halk Component getTreeCellRendererComponent
(JTree ağaç, Object değer, boolean seçili, boolean genişletilmiş,
boolean yaprak, int satır, boolean odak var);

```

TreeCellRenderer arayüzünü uygulayarak özel bir ağaç hücresi rendereri oluşturabilir veya varsayılan bir uygulama sağlayan DefaultTreeCellRenderer sınıfını kullanabilirsiniz.

Ağaç Hücresi Görüntüleyici. Yeni bir JTree oluşturulduğunda, DefaultTreeCellRenderer sınıfının bir örneği ağaç görüntüleyicisine atanır. DefaultTreeCellRenderer sınıfı, yaprak düğümleri, genişletilmiş düğümler ve daraltılmış düğümler için leafIcon, openIcon ve closedIcon adında üç simge özelliğini korur. Ayrıca metin ve arka plan için renkler sağlar. Aşağıdaki kod, ağaçta yeni yaprak, açık ve kapalı simgeleri ve yeni arka plan seçim rengini ayarlar:

```
DefaultTreeCellRenderer oluşturuç =
```

```
(DefaultTreeCellRenderer))Tree1.getCellRenderer(); nrenderere
r.setLeafIcon(yourCustomLeafImageIcon);
renderer.setOpenIcon(yourCustomOpenImageIcon);nrender
derer.setClosedIcon(yourCustomClosedImageIcon);nrender
er.setBackgroundSelectionColor(Color.red);
```

NOT: Varsayılan yaprak, açık simge ve kapalı simge görünüm ve hisse bağlıdır. Örneğin, Windows görünüm ve hissinde, açık simge - ve kapalı simge + dır.

JTree varsayılan bir hücre düzenleyici ile birlikte gelir. Eğer JTree'nin düzenlenebilir özelliği true ise, varsayılan düzenleyici, düğmeye üç kez tıklandığında düzenleme için bir metin alanı etkinleştirir. Varsayılan olarak, bu özellik false olarak ayarlanmıştır. Özel bir düzenleyici oluşturmak için, tablo hücresi düzenlemede kullandığınız aynı sınıf olan DefaultCellEditor sınıfını genişletmeniz gerekir. Bir metin alanı, bir onay kutusu veya bir açılır kutu kullanabilir ve bunu DefaultCellEditor'ın yapısına geçirerek bir düzenleyici oluşturabilirsiniz. Aşağıdaki kod, renkleri düzenlemek için bir açılır kutu kullanır. Açılır kutu düzenleyicisi Şekil 40.30a'da gösterilmektedir.

```
// Düzenleyiciyi özelleştir
JComboBox jcboColor = new JComboBox();
jcboColor.addItem("red");
jcboColor.addItem("yeşil");
jcboColor.addItem("mavi");
jcboColor.addItem("sarı");
jcboColor.addItem("turuncu");
```

```
JTree1.setCellEditor(new javax.swing.DefaultCellEditor(jcboColor)); nJTree1.setEdi
table(true);
```



(a)



(b)

**Şekil 40.30**

*Ağaç düğümlerini düzenlemek için özel bir editör sağlayabilirsiniz.*

Önceki kodda oluşturulan editörle ilgili iki sinir bozucu sorun var. İlk olarak, sa dece bir fare tıklamasıyla etkinleştiriliyor. İkincisi, düğümün simgesinin üzerine geliyor, Şekil 40.30a'da gösterildiği gibi. Bu iki sorun, aşağıdaki kodda gösterildiği gibi DefaultTreeCellEditor kullanılarak düzeltilir:

```
jTree1.setCellEditor
(new javax.swing.tree.DefaultTreeCellEditor(jTree1,
new javax.swing.tree.DefaultTreeCellRenderer(),
new javax.swing.DefaultCellEditor(jcboColor)));
```

Yeni editör Şekil 40.30b'de gösterilmektedir.

DefaultTreeCellEditor kullanarak düzenleme, üçü fare tıklamasıyla başlar. Kombinasyon kutusu düğümün simgesiyle örtüşmez.

## 40.15 Ağaç Olayları

JTree, birçok diğer olayın yanı sıra TreeSelect  
ionEvent ve TreeExpansionEvent tetikleyebilir. Yeni bir düğüm s  
eçildiğinde, JTree bir  
TreeSelectionEvent tetikler. Bir düğüm genişletildiğinde veya  
a küçültüldüğünde, JTree bir TreeExpansionEvent tetikler. Ağaç s  
eçim olayını işlemek için, bir dinleyici TreeSelectionListener a  
arayüzünü uygulamalıdır; bu arayüz, valueChanged adında tek bi  
r işleyici içerir.

AğaçGenişletmeDinleyicisi, düğüm genişletme veya düğüm k  
apama işlemlerini yönetmek için treeCollapsed ve treeExpa  
nded adında iki işleyici içerir.

Aşağıdaki kod, TreeSelectionEvent için bir dinleyici sınıfında seçilen bir düğümü  
görüntüler:

```
public void valueChanged(TreeSelectionEvent e) {
 TreePath path = e.getNewLeadSelectionPath();
 TreeNode treeNode = (TreeNode)path.getLastPathComponent();
 System.out.println("Seçilen düğüm: " + treeNode.toString());
}
```

## Bölüm Özeti

1. JTable, bir tablo modeli, bir sütun modeli ve bir liste-seçim modeli olmak üze  
re üç destekleyici modele sahiptir. *Tablo modeli*, verileri depolamak ve işlemek içi  
n kullanılır. *Sütun modeli*, tablodaki tüm sütunları temsil eder. *Liste-seçim mode*  
*li*, JList tarafından satırları, sütunları ve hücreleri seçmek için kullanılanla aynı  
dır.  
JTable ayrıca, TableColumn ve JTableHeader adında iki yararlı destekleyici sınıf  
a sahiptir. TableColumn, belirli bir sütun hakkında bilgileri içerir. JTabl  
eHeader, bir JTable'nin başlığını görüntülemek için kullanılabilir. Her sütunun vars  
ayılan bir düzenleyici ve render'ı vardır. Ayrıca,

TableCellEditor arayüzünü uygulayarak özel bir düzenleyici oluşturabilir ve TableCellRend  
erer arayüzünü uygulayarak özel bir render oluşturabilirsiniz.

2. JTable gibi, JTree de birçok destekleyici arayüz ve sınıfa sahip çok karmaş  
ık bir bileşendir. JTree ağacı görüntülerken, ağacın veri temsili TreeModel, TreeNo  
de ve TreePath tarafından yönetilir. TreeModel tüm ağacı temsil eder, TreeN  
ode bir düğümü temsil eder ve TreePath bir düğüme giden yolu temsil eder.  
ListModel veya TableModel'in aksine, ağaç modeli doğrudan ağaç verilerini sakl  
amaz veya yönetmez. Ağaç verileri TreeNode ve TreePath içinde saklanır ve yönetil  
ir. Bir TreePath, TreeModel'den elde edilen nesnelerin bir dizisidir. Dizinin el  
emanları, kökün her zaman dizinin ilk elemanı (indeks 0) olacak şekilde sır  
alanmıştır. TreeSelectionMode arayüzü ağaç düğümü seçimlerini yönetir. D  
efaultTreeCellRenderer sınıfı, bir düğümde bir etiket ve/veya bir simge görüntül  
eyebilen bir varsayılan ağaç düğümü oluşturucusu sağlar. DefaultTreeCellEditor ,  
hücreleri bir metin alanında düzenlemek için kullanılabilir. TreePath sınıfı, bir yold  
aki düğümlerin bir kümesini temsil eden bir destek sınıfıdır.

3. JTable ve JTree, javax.swing paketinde yer alırken, destekleyici arayüz ve sınıflarının hepsi sırasıyla javax.swing.table ve javax.swing.tree paketlerinde bulunmaktadır.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

## Gözden Geçirme Soruları

### Bölümler 40.2-40.7

40.1 Bir tabloyu nasıl başlatırsınız? Kaydırmadan görünür satırların maksimum sayısını belirtebilir misiniz? Bir tablo hücresinin yüksekliğini nasıl belirlersiniz? Tablo hücrelerini yatay kenar boşluğunu nasıl belirlersiniz?

Bir satırı nasıl ekler veya kaldırırınız?  
Bir sütunu nasıl ekler veya kaldırırınız?

40.3 Bir tablo sütununun otomatik boyutlandırması nedir? Kaç tür otomatik boyutlandırma mevcuttur?

40.4 Izgara, yatay ızgaralar ve dikey ızgaraları gösteren özellikler nelerdir? Tablo satır yüksekliğini, dikey kenar boşluğunu ve yatay kenar boşluğunu belirten özellikler nelerdir?

40.5 Varsayılan tablo render'ları ve editörleri nelerdir? Özel bir tablo hücresi render'ı ve editörü nasıl oluşturursunuz?

40.6 Varsayılan ağaç render'ları ve editörleri nelerdir? Özel bir ağaç hücresi render'ı ve editörü nasıl oluşturursunuz?

40.7 Tablo hücresi düzenlemeyi nasıl devre dışı bırakırsınız?

### Bölümler 40.8-40.14

40.8 Bir ağaç nasıl oluşturulur? Bir ağaç düğümünün satır yüksekliğini nasıl belirtirsiniz? JTree örneğinden varsayılan ağaç modeli ve ağaç seçim modelini nasıl elde edersiniz?

40.9 TreeModel kullanarak bir ağaçta verileri nasıl başlatırsınız? DefaultMutableTreeNode örneğine bir çocuk nasıl eklenir?

40.10 Ağaç düğümü düzenlemesini nasıl etkinleştirirsiniz?

40.11 Bir ağaçtan bir düğüm nasıl eklenir veya kaldırılır?

40.12 Seçilen bir ağaç düğümünü nasıl elde edersiniz?

## Programlama Alıştırmaları

### Bölümler 40.2-40.7

40.1 \*

(Bir kredi takvimi için bir tablo oluşturun) Egzersiz 31.5, bir amortisman takvimini bir metin alanında görüntüler. Bir program yazın ki

kullanıcının kredi miktarını, yıl sayısını ve faiz oranını spinner'lardan girmesine veya seçmesine olanak tanır ve bir tabloda program, Şekil 40.31'de gösterildiği gibi. Kredi miktarı için adım \$500, yıl sayısı için 1 ve yıllık faiz oranı için %0.125'tir.

Payment#	Interest	Principal	Balance
1	\$416.67	\$374.13	\$99,625.87
2	\$415.11	\$375.09	\$99,250.19
3	\$413.54	\$377.25	\$98,872.94
4	\$411.97	\$378.82	\$98,494.11
5	\$410.39	\$380.40	\$98,113.71
6	\$408.81	\$381.99	\$97,731.72

**Şekil 40.31**

*Tablo, kredi takvimini göstermektedir.*

40.2 \*

(*Satırları ve sütunları sil*) Liste 40.6, ModifyTable.java, yalnızca ilk seçilen satırı veya sütunu silmenize olanak tanır. Programın, tüm seçilen satırları veya sütunları silmesine izin verin. Ayrıca programın, DELETE tuşuna basarak bir satırı veya sütunu silmesine izin verin.

40.3 \*

(*Bir öğrenci tablosu oluşturun*) Öğrenci kayıtları için bir tablo oluşturun. Her kayıt, ad, doğum tarihi, sınıf durumu, eyalet içi ve bir fotoğraf içerir; bu, Şekil 40.32a'da gösterilmiştir. Ad, String tipindedir; doğum tarihi, Date tipindedir; sınıf durumu, aşağıdaki beş değerden biri olmalıdır: Birinci sınıf, İkinci sınıf, Üçüncü sınıf, Dördüncü sınıf veya Lisansüstü; eyalet içi, öğrencinin eyalet sakini olup olmadığını belirten bir boolean değerdir; ve fotoğraf bir resim simgesidir. Varsayılan editörleri isim, doğum günü ve eyalet içindeki için kullanın. Sınıf durumu için özel editör olarak bir kombinasyon kutusu sağlayın.

Name	Birthday	Class Status	In-State	Photo
Jeff F. Smith	Sep 29, 1998	Freshman	<input checked="" type="checkbox"/>	
John F. Kay	Sep 29, 1997	Freshman Sophomore Junior Senior Graduate	<input type="checkbox"/>	

(a)

(b)

Country	Capital	Population	Democracy
USA	Washington DC	280	true
Canada	Ottawa	32	true
United Kingdom	London	60	true
Germany	Berlin	83	true
France	Paris	60	true

**Şekil 40.32**

(a) Tablo öğrenci kayıtlarını gösterir. (b) Dosyadaki veriler JTable'da görüntülenir.

40.4 \*

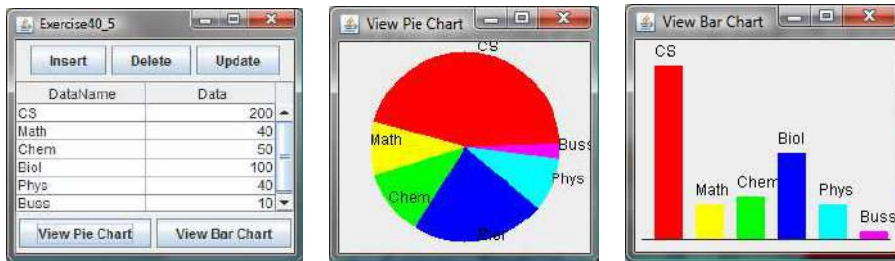
(Bir metin dosyasından veri için bir tablo görüntüleyin) Exercise36\_4Table.txt adında bir tablonun bir metin dosyasında saklandığını varsayalım. Dosyadaki ilk satır başlıktır ve kalan satırlar tablodaki satırlara karşılık gelir. Elemanlar virgülle ayrılmıştır. JTable bileşeni kullanarak tabloyu görüntülemek için bir program yazın. Örneğin, aşağıdaki metin dosyası, Şekil 40.32b'de gösterildiği gibi bir tabloda görüntülenir.

Ülke, Başkent, Nüfus, Demokrasi
ABD, Washington DC, 280, doğru
Kanada, Ottawa, 32, doğru
Birleşik Krallık, Londra, 60, doğru
Almanya, Berlin, 83, doğru
Fransa, Paris, 60, doğru
Norveç, Oslo, 4.5, doğru
Hindistan, Yeni Delhi, 1046, doğru

40.5 \*\*\*

(JTable kullanarak bir denetleyici oluşturun) Exercise 35.1'de, bir grafik modeli (ChartModel) ve iki görünüm (PastaGrafi ve ÇubukGrafi) oluşturmştunuz. Kullanıcının verileri değiştirmesine olanak tanıyan bir denetleyici oluşturun; bu, Şekil 40.33'te gösterilmiştir. Değişikliklerin pasta grafik görünümünde ve çubuk grafik görünümünde etkili olduğunu göreceksiniz. Alıştırmanız aşağıdaki sınıflardan oluşmaktadır:

- ChartController adında bir kontrolcü. Bu sınıf, verileri görüntülemek için bir tablo kullanır. Tablo içindeki verileri değiştirebilirsiniz. Tablo içindeki seçili satırın üzerine yeni bir satır eklemek için *Insert* butonuna, seçili satırı silmek için *Delete* butonuna ve tablodaki yaptığınız değişiklikleri güncellemek için *Update* butonuna tıklayın.
- MyTableModel sınıfı. Bu sınıf, sayısal değerler için JTable'in varsayılan editörü kullanabilmeniz için getColumnClass yöntemini geçersiz kılmak üzere DefaultTableModel'den türetilmiştir. Bu sınıf, Liste 40.7'deki ile aynıdır.
- Alıştırma 35.1'den ChartModel, PieChart ve BarChart sınıfları.
- Ana sınıf Exercise36\_5. Bu sınıf, bir kontrolcü ve iki buton ile bir kullanıcı arayüzü oluşturur, View Pie Chart ve View Bar Chart. View Pie Chart butonuna tıklayarak bir pasta grafiği görüntülemek için bir çerçeve açılır ve View Bar Chart butonuna tıklayarak bir çubuk grafiği görüntülemek için bir çerçeve açılır.



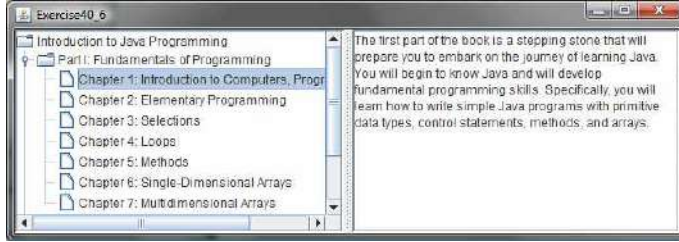
Şekil 40.33

Kontrolcüdeki verileri değiştirebilirsiniz. Görünümler kontrolcü ile senkronizedir.

## Bölümler 40.8-40.14

40.6\*

(*Kitap bölümleri için bir ağaç oluşturun*) Bir kitabın içindekiler tablosunu görüntülemek için bir ağaç oluşturun. Ağaçta bir düğüm seçildiğinde, seçilen düğümü tanımlayan bir paragraf görüntülenir; Şekil 40.34'te gösterildiği gibi.



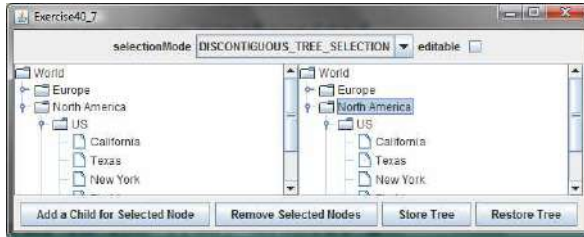
Şekil 40.34

*Düğümün içeriği, düğümüne*

*tıklandığında bir metin alanında görüntülenir.*

40.7\*

(*Ağaçları sakla ve geri yükle*) Şekil 40.35'te gösterildiği gibi, ağaçları saklamak ve geri yükleme için iki düğme eklemek üzere Listing 40.16, ModifyTree.java dosyasını değiştirin. Ağaç mod elini saklamak için nesne G/Ç kullanın.



Şekil 40.35

*Ağaç verilerini bir dosyaya saklayabilir ve daha sonra geri yükleyebilirsiniz.*

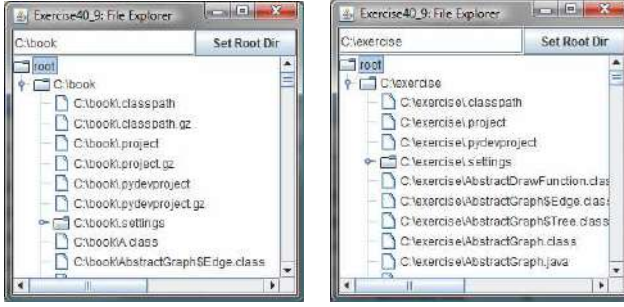
40.8\*

(*Ağaçları gez*) Varsayılan JTree yapıcısını kullanarak bir ağaç oluşturun ve düğümleri gen işlik öncelikli, derinlik öncelikli, ön sıralama ve

son sıralama ile gezin.

40.9\*\*\*

(*Dosya gezgini*) JTree kullanarak bir dosya gezgini geliştirin. Program, kullanıcının bir diz in girmesine izin verir ve dizin altındaki tüm dosyaları görüntüler, Şekil 40.36'da gösterild iği gibi.



**Şekil 40.36**

Dosya gezgini, bir dizindeki dosyaları keşfeder.

40.10 \*\*

(AĞAÇ düğümlerini EKLE ve SİL tuşları ile ekleyin ve silin)

Listing 40.16, ModifyTree.java dosyasını,

INSERT tuşuna basarak yeni bir çocuk düğümü eklemek ve DELETE tuşuna basarak bir düğümü silmek için değiştirin.

40.11 \*

(Tabloları bulun ve içeriklerini gösterin) Programlama Alıştırması 33.6'yı  
JTable içinde tablo içeriklerini gösterecek şekilde gözden geçirin, metin alanında değil.



\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 41

### Gelişmiş Java Veritabanı Programlama

#### Hedefler

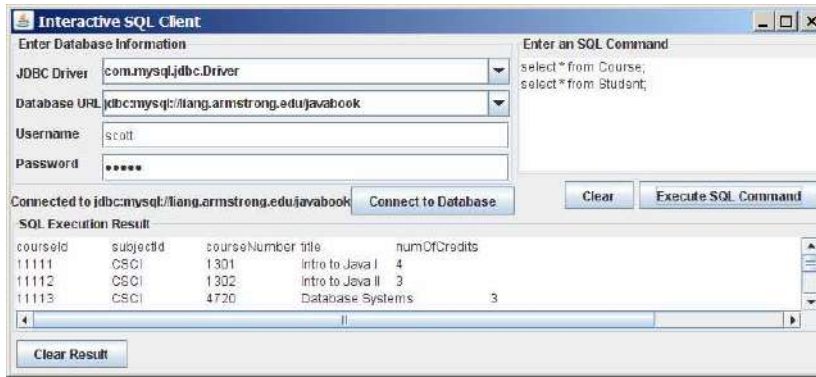
- Yerel veya uzak veritabanına erişmek için evrensel bir SQL istemcisi oluşturmak (§41.2).
- SQL ifadelerini toplu modda çalıştırmak için (§41.3).
- Güncellenebilir ve kaydırılabilir sonuç kümesini işlemek için (§41.4).
- RowSet kullanarak Java veritabanı programlamasını basitleştirmek için (§41.5).
- RowSet için özel bir tablo modeli oluşturmak için (§41.5).
- JDBC'de resimleri depolamak ve almak için (§41.7).

## 41.1 Giriş

Önceki bölüm, JDBC'nin temel özelliklerini tanıttı. Bu bölüm, gelişmiş özelliklerini kapsar. Herhangi bir yerel veya uzaktan ilişkisel veritabanına erişmek için evrensel SQL istemcisi geliştirmeyi, performansı artırmak için toplu modda ifadeleri çalıştırmayı, güncelenebilir sonuç kümelerini ve sonuç kümeleri aracılığıyla bir veritabanını güncellemeyi, database erişimini basitleştirmek için RowSet kullanmayı ve resimleri depolayıp almayı öğreneceksiniz.

## 41.2 Evrensel SQL İstemcisi

Önceki bölümde, veritabanına bağlanmak için çeşitli sürücüler kullandınız, SQL ifadelerini çalıştırmak için ifadeler oluşturdunuz ve SQL sorgularından gelen sonuçları işlediniz. Bu bölüm, hangi ilişkisel veritabanına bağlanmanızı ve SQL komutlarını etkileşimli olarak çalıştırmanızı sağlayan evrensel bir SQL istemcisi sunmaktadır; bu, Şekil 41.1'de gösterilmiştir. İstemci, herhangi bir JDBC veri kaynağına bağlanabilir ve SQL SELECT komutlarını ve SELECT olmayan komutları çalıştırmak için gönderebilir. Çalıştırma sonucu, SELECT sorguları için görüntülenir ve çalıştırma durumu, SELECT olmayan komutlar için görüntülenir. Liste 41.1 programı vermektedir.



Şekil 41.1

Herhangi bir JDBC veri kaynağına bağlanabilir ve SQL komutlarını etkileşimli olarak çalıştırabilirsiniz.

### Liste 41.1 SQLClient.java

<margin note satırı 10: bağlantı> n<margin note satırı 13: ifade> n<margin note satırı 24: URL'ler> n<margin note satırı 28: sürücüler> n<margin note satırı 47: kullanıcı arayüzü oluştur> n<margin note satırı 109: SQL'i çalıştır> n<margin note satırı 114: connect veritabanı> n<margin note satırı 119: clear command> n<margin note satırı 124: sonucu temizle> n<margin note satırı 139: sürücüyü yükle> n<margin note satırı 140: veritabanına bağlan> n<margin note satırı 161: SQL seçimini işle>

<kenar notu satırı 164: işlem seçilmemiş> n<kenar n  
otu satırı 222: ana yöntem atlandı>

```
import java.awt.*;nim
port java.awt.event.*;nimport
javax.swing.*;nimport jav
ax.swing.border.*;nimport java.sql
.*;
java.util.*'i içe aktarın;

public class SQLClient extends JApplet {
 // Veritabanına bağlantı
 private Connection connection;

 // SQL komutlarını çalıştırmak için ifade
 private Statement statement;

 // SQL komutlarını girmek için metin alanı
 private JTextArea jtsqlCommand = new JTextArea();

 // SQL komutlarından sonuçları görüntülemek için metin alanı
 private JTextArea jtaSQLResult = new JTextArea();

 // Bir veritabanı bağlantısı için JDBC bilgisi
 JTextField jtfUsername = new JTextField();
 JPasswordField jpfPassword = new JPasswordField();
 JComboBox jchoURL = new JComboBox(new String[] {
 "jdbc:mysql://localhost/javabook",
 "jdbc:odbc:exampleMDSDataSource",n "j
 dbc:oracle:thin:@liang.armstrong.edu:1521:orcl"});n JComboBox j
 cboDriver = new JComboBox(new String[] {n "com.mysql.jdbc.Dr
 iver", "sun.jdbc.odbc.JdbcOdbcDriver",n "Oracle.jdbc.driver.OracleDriv
 er"});

 JButton jbtExecuteSQL = new JButton("SQL Komutunu Çalıştır");
 JButton jbtClearSQLCommand = new JButton("Temizle");
 JButton jbtConnectDB1 = new JButton("Veritabanına Bağlan");
 JButton jbtClearSQLResult = new JButton("Sonucu Temizle");

 // Başlıklı kenarlıklar oluştur
 Border titledBorder1 = new TitledBorder("Bir SQL Komutu Girin");
 Border titledBorder2 = new TitledBorder("SQL Çalıştırma Sonucu");
 Border titledBorder3 = new TitledBorder(
 "Veritabanı Bilgilerini Girin");

 JLabel lblConnectionStatus = new JLabel("Şu anda bağlantı yok");

 /** Applet'i başlat */
 public void init() {
 JScrollPane jScrollPane1 = new JScrollPane(jtsqlCommand);n jScrollPane1
 e1.setBorder(titledBorder1);
 JScrollPane jScrollPane2 = new JScrollPane(jtaSQLResult);n jScrollPane2
 .setBorder(titledBorder2);

 JPanel jPanel1 = new JPanel(new FlowLayout(FlowLayout.RIGHT));n jPanel1.ad
 d(jbtClearSQLCommand);
 jPanel1.add(jbtExecuteSQL);n JPa

 nel jPanel2 = new JPanel();
```

```

jPanel2.setLayout(new BorderLayout());n JPanel
2.add(jScrollPane1, BorderLayout.CENTER);n JPanel2.add(jP
anel1, BorderLayout.SOUTH);
jPanel2.setPreferredSize(new Dimension(100, 100));

JPanel jPanel3 = new JPanel();n JPanel
3.setLayout(new BorderLayout());n JPanel3.add(
jlblConnectionStatus, BorderLayout.CENTER);n JPanel3.add(jbtConne
ctDB1, BorderLayout.EAST);

JPanel jPanel4 = new JPanel();n JPanel
4.setLayout(new GridLayout(4, 1, 10, 5));n JPanel4.add(jcbo
Driver);
jPanel4.add(jcboURL);
jPanel4.add(jtfxUsername);
jPanel4.add(jpfxPassword);

JPanel jPanel5 = new JPanel();
jPanel5.setLayout(new GridLayout(4, 1));
jPanel5.add(new JLabel("JDBC Sürücüsü"));
jPanel5.add(new JLabel("Veritabanı URL'si"));
jPanel5.add(new JLabel("Kullanıcı Adı"));
jPanel5.add(new JLabel("Şifre"));

JPanel jPanel6 = new JPanel();n JPanel
6.setLayout(new BorderLayout());n JPanel6.setB
order(titledBorder3);n JPanel6.add(jPanel
4, BorderLayout.CENTER);n JPanel6.add(jPanel5, Bor
derLayout.WEST);

JPanel jPanel7 = new JPanel();n JPanel
7.setLayout(new BorderLayout());n JPanel7.add(
jPanel3, BorderLayout.SOUTH);n JPanel7.add(jPane
l6, BorderLayout.CENTER);

JPanel jPanel8 = yeni JPanel();
jPanel8.setLayout(new BorderLayout());n JPanel
8.add(jPanel2, BorderLayout.CENTER);n JPanel8.add(
jPanel7, BorderLayout.WEST);n JPanel jPanel9 = n
ew JPanel(new FlowLayout(FlowLayout.LEFT));n JPanel9.add(jbtClearSQLResu
lt);

jcboURL.setEditable(true);n jcbo
Driver.setEditable(true);n add(jPane
l8, BorderLayout.NORTH);n add(jScrollPa
ne2, BorderLayout.CENTER);n add(jPanel9, Bord
erLayout.SOUTH);

jbtExecuteSQL.addActionListener(new ActionListener() {n public voi
d actionPerformed(ActionEvent e) {
SQL'i yürüt();n }

}};

jbtConnectDB1.addActionListener(new ActionListener() {n public vo
id actionPerformed(ActionEvent e) {
connectToDB();n
}

}};

```

```

jbtClearSqlCommand.addActionListener(new ActionListener() {n public v
oid actionPerformed(ActionEvent e) {
jtaSqlCommand.setText(null);n }

}

jbtClearSQLResult.addActionListener(new ActionListener() {n public void
actionPerformed(ActionEvent e) {
jtaSQLResult.setText(null);n }

}

}

/** DB'ye bağlan */
private void connectToDB() {
// Kullanıcı girişinden veritabanı bilgilerini al
String driver = (String)jcboDriver.getSelectedItemAt();
String url = (String)jcboURL.getSelectedItemAt();
String kullanıcı adı = jtfUsername.getText().trim();n String ş
ifre = new String(jpfPassword.getPassword());

// Veritabanına bağlantı
try {
bağlantı = DriverManager.getConnection(n url, kull
anıcı adı, şifre);
jlblConnectionStatus.setText("Bağlantı kuruldu: " + url);
}
catch (java.lang.Exception ex) {n ex.prin
tStackTrace();
}

}

/** SQL komutlarını çalıştır */
özel void çalıştırSQL() {
if (bağlantı == boş) {
jtaSQLResult.setText("Lütfen önce bir veritabanına bağlanın");
return;
}
başka {
String sqlCommand = jtaSqlCommand.getText().trim();n String[]
komutları = sqlCommand.replace('n', ' ').split(";");

for (String aCommand: komutlar) {n if (a
Command.trim().toUpperCase().startsWith("SELECT")) {n prosesSQLSele
ct(aCommand);
}
başka {
prosesSQLNonSelect(aCommand);n }
}
}

}

/** SQL SELECT komutlarını çalıştır */
private void processSQLSelect(String sqlCommand) {
try {
// Mevcut bağlantı için yeni bir ifade al
statement = connection.createStatement();

// Bir SELECT SQL komutunu çalıştır
ResultSet resultSet = statement.executeQuery(sqlCommand);

```

```

// Sonuç kümesindeki sütun sayısını bul
int columnCount = resultSet.getMetaData().getColumnCount();
String row = "";

// Sütun adlarını görüntüle
for (int i = 1; i <= ColumnCount; i++) {n row += res
ultSet.getMetaData().getColumnName(i) + "t";n }

jtaSQLResult.append(satır + 'n');

while (resultSet.next()) {
// Satırı boş olarak sıfırla
row = "";

for (int i = 1; i <= columnCount; i++) {
// Bir String olmayan sütun bir dizeye dönüştürülür
row += resultSet.getString(i) + "";
}

jtaSQLResult.append(satır + 'n');n }
}
catch (SQLException ex) {n jtaS
QLResult.setText(ex.toString());n }
}

/** SQL DDL ve değişiklik komutlarını çalıştır */
private void processSQLNonSelect(String sqlCommand) {
try {
// Mevcut bağlantı için yeni bir ifade al
statement = connection.createStatement();

// Bir SELECT olmayan SQL komutunu çalıştır
statement.executeUpdate(sqlCommand);

jtaSQLResult.setText("SQL komutu çalıştırıldı");
}
catch (SQLException ex) {n jtaS
QLResult.setText(ex.toString());n }
}
}

```

Kullanıcı JDBC sürücüsünü, veritabanı URL'sini, kullanıcı adını ve şifreyi seçer veya girer ve belirtilen veritabanına bağlanmak için *Veritabanına Bağlan* düğmesine tıklar. Bu işlem connectToDB() yöntemi ile gerçekleştirilir (satırlar 130-147).

Kullanıcı *SQL Komutunu Çalıştır* düğmesine tıkladığında, executeSQL() yöntemi çağrılır (satırlar 150-168) ve SQL komutları metin alanından (jtaSQLCommand) alınarak her bir komut noktalı virgülle (;) ayrılır. Daha sonra komutun bir SELECT sorgusu mu yoksa DDL yada veri değiştirme ifadesi mi olduğunu belirler (satırlar 160-165). Eğer komut bir SELECT sorgusu ise, processSQLSelect yöntemi çağrılır (satırlar 171-205). Bu yöntem, sorgu

sonucunu elde etmek için executeQuery yöntemini (satır 177) kullanır. Sonuç, metin alanı jtaSQLResult'ta (satır 188) görüntülenir. Komut bir SELECT dışı sorguysa, processSQLNonSelect() süreci

metod çağrılır (satırlar 208-221). Bu metod, SQL komutunu yürütmek için `executeUpdate` metodunu (satır 214) kullanır.

`ResultSet` arayüzündeki `getMetaData` metodu (satırlar 180, 185) `ResultSetMetaData` örneğini elde etmek için kullanılır. `getColumnCount` metodu (satır 180) sonuç kümesinde ki sütun sayısını döndürür ve `columnName(i)` metodu (satır 185) *i*'inci sütunun adını döndürür.

### 41.3 Parti İşleme

Tüm önceki örneklerde, SQL komutları veritabanına birer birer yürütülmek üzere gönderilmektedir. Bu, büyük sayıda güncelleme işlemi için verimsizdir. Örneğin, bir tabloya bin satır eklemek istediğinizi varsayalım. Her seferinde bir `INSERT` komutu göndermek, tüm `INSERT` komutlarını bir seferde toplu olarak göndermekten neredeyse bin kat daha uzun sürer. Performansı artırmak için, JDBC, seçim dışı SQL komutlarını işlemek için toplu güncellemeyi tanıttı. Bir toplu güncelleme, bir dizi seçim dışı SQL komutundan oluşur. Bu komutlar bir araya toplanır ve hepsi birlikte veritabanına gönderilir.

Toplu güncellemeyi kullanmak için, `Statement` arayüzündeki `addBatch` yöntemini kullanarak bir toplu işte seçim dışı komutlar ekleyebilirsiniz. Tüm SQL komutları toplu işe ekledikten sonra, toplu işi yürütmek için `executeBatch` yöntemini kullanın.

Örneğin, aşağıdaki kod bir tablo oluşturma komutunu ekler, iki `insert` ifadesini bir toplu işte ekler ve toplu işi yürütür.

```
Statement statement = connection.createStatement();

// SQL komutlarını toplu işe ekle
statement.addBatch("create table T (C1 integer, C2 varchar(15))");
statement.addBatch("insert into T values (100, 'Smith')");
statement.addBatch("insert into T values (200, 'Jones')");

// Toplu işi yürüt
int count[] = statement.executeBatch();
```

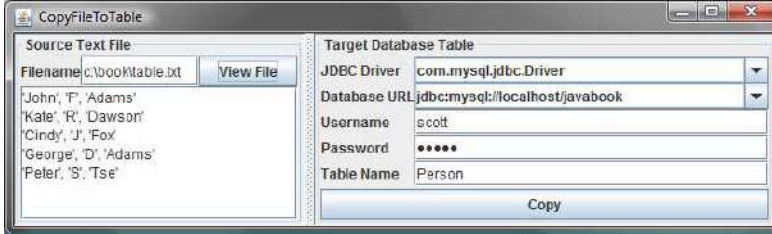
`executeBatch()` yöntemi, her biri SQL komutuyla etkilenen satır sayısını sayan bir dizi döndürür. İlk sayı 0 döndürür çünkü bu bir DDL komutudur. Diğer sayılar 1 döndürür çünkü sadece bir satır etkilenmiştir.

**NOT:** Bir sürücünün toplu güncellemeleri destekleyip desteklemediğini öğrenmek için,

`DatabaseMetaData` örneği üzerinde `supportsBatchUpdates()` yöntemini çağırın. Eğer sürücü toplu güncellemeleri destekliyorsa, `true` döndürecektir. MySQL, Access ve Oracle için JDBC sürücülere toplu güncellemeleri destekler.

Toplu işleme göstermek için, bir metin dosyasından veri alıp bu verileri bir tabloya kopyalayan bir program yazmayı düşünün. Metin dosyası, her biri tabloda bir satıra karşılık gelen satırlardan oluşur. Bir satırdaki alanlar virgüllerle ayrılır. Bir satırdaki dize değerleri tek tırnak içinde yer alır. Metin dosyasını görüntülemek için *Dosyayı Görüntüle* butonuna tıklayabilir ve metni tabloya kopyalamak için *Kopyala* butonuna tıklayabilirsiniz. Tablo, veritabanında önceden tanımlanmış olmalıdır. Şekil 41.2, text file table.txt dosyasının Person tablosuna kopyalandığını göstermektedir. Person, aşağıdaki ifade kullanılarak oluşturulmuştur:

tableti Kişisi oluştur (adını  
a barchart(t20t),n mi t grafi  
ği(t1t),  
soyadı r barchart(t20t)n)



**Şekil 41.2**

*CopyFileToTable aracı, metin dosyalarını veritabanı tablolarına kopyalar.*

Liste 41.2, sorunun çözümünü vermektedir.

#### **Liste 41.2 CopyFileToTable.java**

<margin not satırı 15: sürücüler> n<m  
argin note satırı 18: URL'ler> n<m  
argin note satırı 31: kullanıcı arayüzü olu  
ştur> n<margin note satırı 74: dosyayı g  
örüntüle> n<margin note satırı 81: tabl  
oya> n<margin note satırı 114: sürücüyü yü  
kle> n<margin note satırı 118: veritabanına bağlan  
> n<margin not satırı 125: satır ekle>  
<margin note line 144: ifade>  
<margin note line 154: grup>  
<margin note line 180: grubu çalıştır>  
<margin note line 199: ana yöntem atlandı>

```

import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.sql.*;
import java.util.*;

public class CopyFileToTable extends JFrame {
 // Metin dosyası bilgileri
 özel JTextField jtfFilename = new JTextField();
 özel JTextArea jtaFile = new JTextArea();

 // JDBC ve tablo bilgileri
 özel JComboBox jcbDriver = new JComboBox(new String[] { "com.mysql.
 jdbc.Driver", "sun.jdbc.odbc.JdbcOdbcDriver", "oracle.jdbc.driver.Ora
 cleDriver" });
 özel JComboBox jcbURL = new JComboBox(yeni Dize[] { "jdbc:mysql:
 //localhost/javabook",
 "jdbc:odbc:exampleMDSDataSource",

```



```

"jdbc:oracle:thin:@liang.armstrong.edu:1521:orcl"));
private JTextField jtfKullaniciAdi = new JTextField();
private JPasswordField jtfSifre = new JPasswordField();
private JTextField jtfTabloAdi = new JTextField();

private JButton jbtDosyayiGoruntule = new JButton("Dosyayı Görüntüle");
private JButton jbtKopyala = new JButton("Kopyala");
private JLabel jlblDurum = new JLabel();

genel CopyFileToTable() {
 JPanel jPanel1 = new JPanel();
 jPanel1.setLayout(new BorderLayout());
 jPanel1.add(new JLabel("Dosya Adı"), BorderLayout.WEST);
 jPanel1.add(jbtDosyayiGoruntule, BorderLayout.EAST);
 jPanel1.add(jtfDosyaadi, BorderLayout.CENTER);

 JPanel jPanel2 = new JPanel();
 jPanel2.setLayout(new BorderLayout());
 jPanel2.setBorder(new TitledBorder("Kaynak Metin Dosyası"));
 jPanel2.add(jPanel1, BorderLayout.NORTH);
 jPanel2.add(new JScrollPane(jtaFile), BorderLayout.CENTER);

 JPanel jPanel3 = new JPanel();
 jPanel3.setLayout(new GridLayout(5, 0));
 jPanel3.add(new JLabel("JDBC Sürücüsü"));
 jPanel3.add(new JLabel("Veritabanı URL'si"));
 jPanel3.add(new JLabel("Kullanıcı Adı"));
 jPanel3.add(new JLabel("Şifre"));
 jPanel3.add(new JLabel("Tablo Adı"));

 JPanel jPanel4 = new JPanel();
 jPanel4.setLayout(new GridLayout(5, 0));
 jPanel4.setJComboBox(jcboDriver);
 jPanel4.setEditable(true);
 jPanel4.add(jcboURL);
 jPanel4.add(jcboURL);
 jPanel4.add(jtfKullaniciAdi);
 jPanel4.add(jtfPassword);
 jPanel4.add(jtfTableName);

 JPanel jPanel5 = yeni JPanel();
 jPanel5.setLayout(new BorderLayout());
 jPanel5.setBorder(new TitledBorder("Hedef Veritabanı Tablosu"));
 jPanel5.add(jbtCopy, BorderLayout.SOUTH);
 jPanel5.add(jPanel3, BorderLayout.WEST);
 jPanel5.add(jPanel4, BorderLayout.CENTER);

 add(jlblStatus, BorderLayout.SOUTH);
 add(new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, jPanel2, jPanel5), BorderLayout.CENTER);
 jbtViewFile.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent evt) {
 showFile();
 }
 });

 jbtCopy.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent evt) {
 try {

```

```

 kopyaDosya();n
 }
 catch (Exception ex) {n jlblS
 tatus.setText(ex.toString());n }

}
});
}

/** Dosyayı metin alanında göster */
private void showFile() {
 Tarayıcı girişi = null;n dene
 yin {
 // Dosyadan metin okumak için bir Scanner kullan
 input = new Scanner(new File(jtfFilename.getText().trim()));

 // Bir satır oku ve satırı metin alanına ekle
 while (input.hasNext())
 jtaFile.append(input.nextLine() + 'n');n }

 catch (FileNotFoundException ex) {
 System.out.println("Dosya bulunamadı: " + jtfFilename.getText());
 }
 catch (IOException ex) {n ex.p
 rintStackTrace();n }

 nihayet {
 if (input != null) input.close();n }

}

özel void copyFile() throws Exception {
 // JDBC sürücüsünü yükle
 Class.forName(((String)jcbDriver.getSelectedItemAt()).trim());
 System.out.println("Sürücü yüklendi");

 // Bir bağlantı kur
 Connection conn = DriverManager.getConnection
 (((String)jcbURL.getSelectedItemAt()).trim(),
 jtfUsername.getText().trim(),
 String.valueOf(jtfPassword.getPassword()).trim());
 System.out.println("Veritabanına bağlandı");

 // Metin dosyasındaki her satırı oku ve tabloya ekle
 insertRows(conn);
}

private void insertRows(Connection connection) {
 // SQL INSERT ifadesini oluştur
 String sqlInsert = "" + jtfTableName.getText()n + " değerlerine ekleyin (";

 // Dosyadan metin okumak için bir Scanner kullan
 Scanner input = null;

 // Metin alanından dosya adını al
 String filename = jtfFilename.getText().trim();

 denemek {
 // Bir tarayıcı oluştur

```

```

giriş = yeni Tarayıcı(yeni Dosya(dosya adı));

// Bir ifade oluştur
Açıklama bildirimi = bağlantı.createStatement();

System.out.println("Sürücü ana sürüm numarası? " +
 connection.getMetaData().getDriverMajorVersion());

// batchUpdatesSupported'in desteklenip desteklenmediğini belirle
boolean batchUpdatesSupported = false;

denemek {
 if (connection.getMetaData().supportsBatchUpdates()) {n BatchUpda
 tesSupported = true;
 System.out.println("batch güncellemeleri destekleniyor");
 }
 başka {
 System.out.println("Sürücü " +
 "batch güncellemelerini desteklemiyor");
 }
}
catch (UnsupportedOperationException ex) {
 System.out.println("İşlem desteklenmiyor");
}

// Sürücünün toplu güncellemeleri destekleyip desteklemediğini belirle
if (batchUpdatesSupported) {
 // Bir satır oku ve ekleme tablosu komutunu toplu işleme ekle
 while (input.hasNext()) {
 statement.addBatch(sqlInsert + input.nextLine() + ");n }

 statement.executeBatch();

 lblStatus.setText("Toplu güncellemeler tamamlandı");
 }
 başka {
 // Bir satır oku ve ekleme tablosu komutunu çalıştır
 while (input.hasNext()) {
 deyim.executeUpdate(sqlInsert + input.nextLine() + ");n }

 lblStatus.setText("Tek satır güncellemesi tamamlandı");
 }
 }
 catch (SQLException ex) {n Syst
 em.out.println(ex);n }
 catch (FileNotFoundException ex) {
 System.out.println("Dosya bulunamadı: " + filename);
 }
 nihayet {
 if (input != null) input.close();n }
}
}

```

insertRows yöntemi (satırlar 128-195), sürücü batch güncellemeleri destekliyorsa, SQL INSERT komutlarını veritabanına göndermek için batch güncellemelerini kullanır.

Satırlar 152-164, sürücünün batch güncellemeleri destekleyip desteklemediğini kontrol eder.

Eğer sürücü bu işlemi desteklemiyorsa, supportsBatchUpdates() yöntemi çağrıldığında UnsupportedOperationException istisnası fırlatılacaktır (satır 162).

Tabloların veritabanında zaten oluşturulmuş olması gerekir. Dosya formatı ve içerikler ve ritabanı tablo spesifikasyonu ile eşleşmelidir. Aksi takdirde, SQL INSERT komutu başarısız olacaktır.

Egzersiz 41.1'de, bir veritabanına bin kayıt eklemek için bir program yazacak ve toplu güncellemelerle ve toplu güncellemeler olmadan performansı karşılaştıracaksınız.

#### 41.4 Kaydırılabilir ve Güncellenebilir Sonuç Kümesi

##### <margin note: ardışık ileri okuma>

Önceki örneklerde kullanılan sonuç kümeleri ardışık olarak okunur. Bir sonuç kümesi, mevcut veri satırına işaret eden bir imleç (cursor) tutar.

Başlangıçta imleç ilk satırın önünde konumlandırılmıştır. next() metodu imleci bir sonraki satıra ileri taşır. Bu, ardışık ileri okuma olarak bilinir. Bu, JDBC 1 tarafından desteklenen bir sonuç kümesindeki satırları işleme yöntemidir.

JDBC'nin yeni sürümleri ile, satırları hem ileri hem de geri kaydırabilir ve imleci istenen bir konuma taşımak için first, last, next, previous, absolute veya relative metodunu kullanabilirsiniz. Ayrıca, sonuç kümesindeki bir satırı ekleyebilir, silebilir veya güncelleyebilir ve değişikliklerin veritabanında otomatik olarak yansıtılmasını sağlayabilirsiniz.

Kaydırılabilir veya güncellenebilir bir sonuç kümesi elde etmek için, önce uygun türde ve eşzamanlılık modunda bir ifade (statement) oluşturmalısınız. Statik bir ifade için,

```
Statement statement = connection.createStatement
(int resultSetType, int resultSetConcurrency);
```

Hazırlanmış bir ifade için,

```
HazırlıklıStatement bildirimi = bağlantı.prepareStatement n (String sql, t
intt resultSetType,t intt resultSetConcurrency);
```

##### <kenar boşluğu notu: kaydırılabilir>

resultSetType'in olası değerleri,

ResultSet'te tanımlanan sabitlerdir:

- TYPE\_FORWARD\_ONLY: Sonuç kümesine yalnızca ileri şekilde erişilir.
- TYPE\_SCROLL\_INSENSITIVE: Sonuç kümesi kaydırılabilir, ancak database'deki değişikliklere duyarlı değildir.
- TYPE\_SCROLL\_SENSITIVE: Sonuç kümesi kaydırılabilir ve diğerleri tarafından yapılan değişikliklere duyarlıdır. Sonuç kümesinin kaydırılabilir ve güncellenebilir olmasını istiyorsanız bu türü kullanın.

##### <kenar notu: güncellenebilir>

resultSetConcurrency'in olası değerleri,

ResultSet'te tanımlanan sabitlerdir:

- CONCUR\_READ\_ONLY: Sonuç kümesi, database'i güncellemek için kullanılamaz.

- **CONCUR\_UPDATABLE**: Sonuç kümesi, database'i güncellemek için kullanılabilir.

Örneğin, sonuç kümesinin kaydırılabilir ve güncellenebilir olmasını istiyorsanız, bir ifade oluşturabilirsiniz, şöyle:

```
Statement statement = connection.createStatement
(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE)
```

Bir Statement nesnesinde executeQuery yöntemini kullanarak, aşağıdaki gibi bir sonuç kümesi döndüren bir SQL sorgusu çalıştırabilirsiniz:

```
ResultSet resultSet = bildirim.executeQuery(sorgu);
```

#### <margin note: **ResultSet yöntemleri**>

Artık ilk satıra, sonraki satıra, önceki satıra ve son satıra geçmek için first(), next(), previous() ve last() yöntemlerini kullanabilirsiniz.

absolute(int row) yöntemi imleci belirtilen satıra taşır; ve

getXxx(int columnIndex) veya getXxx(String columnName) yöntemi,

mevcut satırdaki belirli bir alanın değerini almak için kullanılır.

insertRow(), deleteRow() ve updateRow() yöntemleri de

mevcut satırı eklemek, silmek ve güncellemek için kullanılabilir. insertRow veya

güncellemeden önce, mevcut satırdaki alana yeni bir değer yazmak için

updateXxx(int columnIndex, Xxx

value) veya update(String columnName, Xxx value) yöntemini kullanmanız gerekir. cancelRowUpdates() yöntemi bir satıra yapılan

güncellemeleri iptal eder. close() yöntemi sonuç kümesini kapatır ve

kaynaklarını serbest bırakır. wasNull() yöntemi, son okunan

sütunun SQL NULL değerine sahip olup olmadığını kontrol eder ve true döner.

Liste 41.3, kaydırılabilir ve güncellenebilir bir sonuç kümesi oluşturmanın nasıl yapılacağını gösteren bir örnek vermektedir. Program, StateCapital tablosu için bir sonuç kümesi oluşturur. StateCapital tablosu aşağıdaki gibi tanımlanmıştır:

```
table StateCapital oluştur (n stat
e t varchar(t40t),
başkent varchar (40)
);
```

#### Liste 41.3 ScrollUpdateResultSet.java

<margin note line 7: sürücüyü yükle>

<margin note line 11: DB'ye bağlan>

<margin note line 14: otomatik taahhüt ayarla>

<margin note line 18: kaydırılabilir güncellenebilir>

<margin note line 22: sonuç kümesini al>

<margin note line 29: imleci hareket ettir>

<margin note line 32: satırı güncelle>

<margin note line 35: imleci hareket ettir>

<margin note line 39: satır ekle>

<margin note line 43: imleci hareket ettir>

<margin note line 44: satırı sil>

<margin note line 52: sonuç kümesini kapat>

<margin note line 55: sonuç kümesini göster>

```
java.sql.*'ı içe aktarın;
```

```

public class ScrollUpdateResultSet {
 public static void main(String[] args)
 throws SQLException, ClassNotFoundException {
 // JDBC sürücüsünü yükle
 Class.forName("oracle.jdbc.driver.OracleDriver");
 System.out.println("Sürücü yüklendi");

 // Bir veritabanına bağlan
 Bağlantı bağlantısı = DriverManager.getConnectionn ("jdbc:orac
 e:thin:@liang.armstrong.edu:1521:orcl", n "scott", "tiger");

 bağlantı.setAutoCommit(true);
 System.out.println("Veritabanına bağlandı");

 // Mevcut bağlantı için yeni bir ifade al
 Statement statement = connection.createStatement(
 ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);

 // Sonuç Kümesi Al
 ResultSet resultSet = statement.executeQueryn ("durum
 seçin, StateCapital'den sermaye");

 System.out.println("Güncellemeden önce ");
 displayResultSet(resultSet);

 // İkinci satırı güncelle
 resultSet.absolute(2); // İmleci ikinci satıra taşı
 resultSet.updateString("state", "New S"); // Sütunu güncelle
 resultSet.updateString("capital", "New C"); // Sütunu güncelle
 resultSet.updateRow(); // Veritabanındaki satırı güncelle

 // Son satırdan sonra ekle
 resultSet.last();
 resultSet.moveToInsertRow(); // İmleci ekleme satırına taşı
 resultSet.updateString("state", "Florida");
 resultSet.updateString("capital", "Tallahassee");
 resultSet.insertRow(); // Satırı ekle
 resultSet.moveToCurrentRow(); // İmleci mevcut satıra taşı

 // Dördüncü satırı sil
 resultSet.absolute(4); // İmleci 5. satıra taşı
 resultSet.deleteRow(); // İkinci satırı sil

 System.out.println("Güncellemeden sonra ");
 resultSet = statement.executeQuery
 ("select state, capital from StateCapital");
 displayResultSet(resultSet);

 // Bağlantıyı kapat
 resultSet.close();
 }

 özel statik void displayResultSet(ResultSet resultSet) n, SQLException'i at
 ar {
 ResultSetMetaData rsMetaData = resultSet.getMetaData();n resultSet.
 beforeFirst();
 while (resultSet.next()) {
 for (int i = 1; i <= rsMetaData.getColumnCount(); i++)n System.out.pri
 ntf("%-12st", resultSet.getObject(i));n System.out.println();
 }
 }
}

```

```

 }
 N }
}

```

### <Çıktı>

Şoför yüklendi  
Veritabanına bağlandı

Güncellemelerden önce	
Indiana	Indianapolis
Illinois Springfield	Kaliforniya
a	Sacramento
Georgia	Atlanta'nın
Teksas	Austin

Güncellemeden sonra	
Indiana	Indianapolis
Yeni S	Yeni C
Kaliforniya	Sacramento
Teksas	Austin
Florida	Tallahassee

### <Son Çıktı>

#### <kenar notu: kaydırılabilir ve güncellenebilir>

18-19. satırlardaki kod, kaydırılabilir ve \_\_\_\_\_  
güncellenebilir sonuç kümesi üretmek için bir Statement oluşturur.

#### <kenar notu: satırı güncelle>

Program, sonuç kümesindeki ikinci satıra (satır 29) imleci taşır, bu satırdaki iki sütunu günceller (satırlar 30-31) ve altındaki veritabanındaki satırı güncellemek için updateRow() yöntemini çağırır (satır 32) .

#### <margin note: satır ekle>

Güncellenebilir bir ResultSet nesnesinin, eklenmek üzere bir satır oluşturmak için bir sahne alanı olarak hizmet eden özel bir satırı vardır. Bu özel satıra *ekleme satırı* denir. Bir satırı eklemek için önce imleci ekleme satırına taşımak için moveToInsertRow() yöntemini çağırın (satır 36), ardından sütunları updateXxx yöntemiyle güncelleyin (satırlar 37-38) ve son olarak insertRow() yöntemini kullanarak satırı ekleyin (satır 39). moveToCurrentRow() çağırısı, imleci mevcut eklenmiş satıra taşır (satırlar

40) .

#### <margin note: satır ekle>

Program dördüncü satıra geçer ve deleteRow() yöntemini çağırarak satırı veritabanından siler (satırlar 43-44).

#### <margin note: sürücü desteği>

NOT: Tüm mevcut sürücüler kaydırılabilir ve güncellenebilir sonuç kümesini desteklememektedir. Örnek, Oracle ojdbc6 sürücüsü kullanılarak test edilmiştir. JDBC sürücüsünün hangi sonuç türü ve parametresi modlarının desteklendiğini öğrenmek için DatabaseMetaData arayüzünde supportsResultSetType(int type) ve supportsResultSetConcurrency(int type, int concurrency) yöntemlerini kullanabilirsiniz. Ancak bir sürücü kaydırılabilir ve güncellenebilir sonuç kümesini desteklese bile, bir sonuç kümesi için bir

karmaşık sorgu bir güncelleme gerçekleştiremeyebilir. Örneğin, birkaç tabloyu içeren bir sorgunun sonuç kümesi güncelleme işlemlerini desteklemiyor olabilir.

NOT: Program çalışmayabilir, eğer 22-23. satırlar ile değiştirilirse

```
ResultSet resultSet =statement.executeQuery ("State
Capital'den * seçin");
```

#### **41.5 RowSet, JdbcRowSet ve CachedRowSet**

##### **<kenar notu: ResultSet'i genişletir>**

JDBC, veritabanı programlamasını basitleştirmek için kullanılabilecek yeni bir RowSet arayüzü tanıttı. RowSet arayüzü, java.sql.ResultSet'i ek yeteneklerle genişleterek bir RowSet örneğinin bir JDBC url'sine, kullanıcı adına ve şifreye bağlanmasını, bir SQL komutu ayarlamasını, komutu çalıştırmasını ve yürütme sonucunu almasını sağlar. Özünde, Connection, Statement ve ResultSet'i tek bir arayüzde birleştirir.

NOT:

##### **<kenar notu: desteklenir>**

Tüm JDBC sürücüleri RowSet'i desteklememektedir. Şu anda, JDBC-ODBC sürücüsü RowSet'in tüm özelliklerini desteklememektedir.

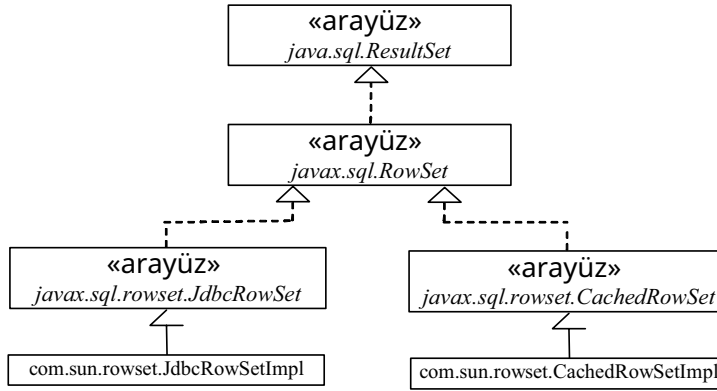
##### **41.5.1 RowSet Temelleri**

##### **<margin note: bağlı vs. bağlı olmayan>**

İki tür RowSet nesnesi vardır: **bağlı** ve **bağlı olmayan**. Bir **bağlı** RowSet nesnesi bir veri kaynağı ile bağlantı kurar ve bu bağlantıyı yaşam döngüsü boyunca sürdürür. **Bağlı olmayan** RowSet nesnesi bir veri kaynağı ile bağlantı kurar, veri kaynağından data almak için bir sorgu çalıştırır ve ardından bağlantıyı kapatır. Bir **bağlı olmayan** rowset, **bağlı olmadığı** sürede verilerine değişiklikler yapabilir ve ardından bu değişiklikleri verinin orijinal kaynağına geri gönderebilir, ancak bunu yapmak için bağlantıyı yeniden kurması gerekir.

RowSet'in birkaç versiyonu vardır. İki sık kullanılan versiyonu JdbcRowSet ve CachedRowSet'tir. Her ikisi de RowSet'in alt arayüzleridir. JdbcRowSet **bağlı**dır, ancak CachedRowSet bağlantısı kesilmiş durumdadır. Ayrıca, JdbcRowSet hem serileştirilebilir hem de kopyalanabilir değildir, oysa CachedRowSet her ikisidir. database satıcıları bu arayüzler için somut uygulamalar sağlamakta serbesttir. Sun, JdbcRowSet için referans uygulaması JdbcRowSetImpl ve CachedRowSet için CachedRowSetImpl sağlamıştır. Şekil 41.3 bu bileşenlerin ilişkisini göstermektedir.





**Şekil 41.3**

*JdbcRowSetImpl ve CachedRowSetImpl,*

*RowSet'in somut uygulamalarıdır.*

RowSet arayüzü, alma (get) ve ayarlama (set) yöntemleri ile JavaBeans özelliklerini içerir. Yeni bir url, kullanıcı adı, parola ve SQL deyimi için bir komut ayarlamak üzere set yöntemlerini kullanabilirsiniz. RowSet kullanarak, Liste 37.1 sadeleştirilebilir, Liste 41.4'te gösterildiği gibi.

**Liste 41.4 SimpleRowSet.java**

<kenar notu satır 9: sürücüyü yükle>  
 <kenar notu satır 13: RowSet oluştur>  
 <kenar notu satır 16: url ayarla>  
 <kenar notu satır 17: kullanıcı adını ayarla>  
 <kenar notu satır 18: parolayı ayarla>  
 <kenar notu satır 19: komutu ayarla>  
 <kenar notu satır 21: komutu çalıştır>  
 <kenar notu satır 25: sonucu al>  
 <kenar notu satır 29: bağlantıyı kapat>

```

java.sql.SQLException'ı içe aktar;ni
import javax.sql.RowSet;
com.sun.rowset.*'i içe aktarın;

public class SimpleRowSet {
 public static void main(String[] args)
 throws SQLException, ClassNotFoundException {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir satır seti oluştur
 RowSet rowSet = new JdbcRowSetImpl();

 // RowSet özelliklerini ayarla
 rowSet.setUrl("jdbc:mysql://localhost/javabook");
 rowSet.setUsername("scott");
 rowSet.setPassword("kaplan");

```

```

rowSet.setCommand("öğrenciden ilkAd, mi, soyadını seçin " + n "b
urada lastName = 'Smith'");
rowSet.execute();

// Sonuçları döngü ile gez ve öğrenci isimlerini yazdır
while (rowSet.next())
 System.out.println(rowSet.getString(1) + "t" + n rowSet.get
String(2) + "t" + rowSet.getString(3));

// Bağlantıyı kapat
rowSet.close();
}
}

```

Satır 13, `JdbcRowSetImpl` kullanarak bir `RowSet` nesnesi oluşturur. Program, `RowSet`'in `set` yöntemini kullanarak bir URL, kullanıcı adı ve parola ayarlar (satırlar 16-18) ve bir sorgu ifadesi için bir komut belirler (satır 19). Satır 24, `RowSet`'teki komutu yürütür. `next()` ve `getString(int)` yöntemleri için `ResultSet`'ten miras alınan sorgu sonucunu işleme (satırlar 25-26).

#### <kenar notu: CachedRowSet kullanma>

`JdbcRowSet`'i 13. satırda `CachedRowSet` ile değiştirirseniz, program gayet iyi çalışacaktır. JDBC-ODBC sürücüsünün `JdbcRowSetImpl`'i desteklediğini, bununla birlikte `CachedRowSetImpl`'i desteklemediğini unutmayın.

İPUCU

#### <kenar notu: meta verileri elde et>

`RowSet`, `ResultSet`'in bir alt arayüzü olduğundan, `ResultSet`'teki tüm yöntemler `RowSet`'te kullanılabilir. Örneğin, `getMetaData()` yöntemini kullanarak bir `RowSet`'ten `ResultSetMetaData` elde edebilirsiniz.

#### 41.5.2 RowSet için PreparedStatement

§37.5, "PreparedStatement" bölümündeki tartışma, `PreparedStatement` arayüzünü kullanarak parametrelili SQL deyimlerinin işlenmesini tanıttı.

`RowSet`, parametrelili SQL deyimlerini destekleme yeteneğine sahiptir. `PreparedStatement`'de parametre değerlerini ayarlamak için kullanılan `set` yöntemleri `RowSet`'te uygulanmıştır. Bu yöntemleri, parametrelili bir SQL komutu için parametre değerlerini ayarlamak üzere kullanabilirsiniz. Liste 41.5, `RowSet`'te parametrelili bir deyim nasıl kullanacağını göstermektedir. Satır 19, `RowSet`'te `lastName` ve `mi` için iki parametre ile bir SQL sorgu deyimini ayarlamaktadır. Bu iki parametre string olduğundan, gerçek değerler i satır 21-22'de ayarlamak için `setString` yöntemi kullanılmaktadır.

#### Liste 41.5 `RowSetPreparedStatement.java`

<margin note line 9: sürücüyü yükle>  
 <margin note line 13: RowSet oluştur>  
 <margin note line 16: url ayarla>  
 <margin note line 19: parametrelili SQL>  
 <margin note line 21: parametre ayarla>  
 <margin note line 22: parametre ayarla>  
 <margin note line 23: çalıştır>  
 <margin note line 25: meta veriler>  
 <margin note line 38: bağlantıyı kapat>

```

java.sql.*'ı içe aktarın;
javax.sql.RowSet'i içe aktarın;
com.sun.rowset.*'i içe aktarın;

public class RowSetPreparedStatement {
 public static void main(String[] args)
 throws SQLException, ClassNotFoundException {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir satır seti oluştur
 RowSet rowSet = new JdbcRowSetImpl();

 // RowSet özelliklerini ayarla
 rowSet.setUrl("jdbc:mysql://localhost/javabook");n rowSet.se
 tUsername("scott");
 rowSet.setPassword("kaplan");
 rowSet.setCommand("Öğrenciden * seçin; burada lastName = " +n "ve mi = ");

 rowSet.setString(1, "Smith");n rowSe
 t.setString(2, "R");n rowSet.exe
 cute();

 ResultSetMetaData rsMetaData = rowSet.getMetaData();n for (int
 i = 1; i <= rsMetaData.getColumnCount(); i++)n System.out.printf("
 %-12st", rsMetaData.getColumnName(i));n System.out.println();

 // Sonuçları döngüye al ve öğrenci isimlerini yazdır
 while (rowSet.next()) {
 for (int i = 1; i <= rsMetaData.getColumnCount(); i++)n System.out.p
 rintf("%-12st", rowSet.getObject(i));n System.out.println();

 }

 // Bağlantıyı kapat
 rowSet.close();
 }
}

```

#### 41.5.3 Kaydırma ve Güncelleme RowSet

Varsayılan olarak, bir ResultSet nesnesi ne kaydırılabilir ne de güncellenebilir. Ancak, bir RowSet nesnesi her ikisidir. Bir RowSet aracılığıyla bir veritabanını kaydırmak ve güncellemek, bir ResultSet aracılığıyla yapmaktan daha kolaydır. Liste 41.6, Liste 41.3'ü bir RowSet kullanarak yeniden yazmaktadır. İmleci hareket ettirmek için `absolute(int)` gibi yöntemleri ve veritabanını güncellemek için `delete()`, `updateRow()` ve `insertRow()` gibi yöntemleri kullanabilirsiniz.

#### Liste 41.6 ScrollUpdateRowSet.java

<margin note line 9: sürücüyü yükle>

<margin note line 13: bir RowSet oluştur>

<margin note line 16: url ayarla>  
<margin note line 17: kullanıcı adını ayarla>  
<margin note line 18: parolayı ayarla>  
<margin note line 19: SQL komutunu ayarla>  
<margin note line 20: çalıştır>  
<kenar notu satır 23: rowSet'i görüntüle>  
<kenar notu satır 26: imleci hareket ettir>  
<kenar notu satır 29: satırı güncelle>  
<kenar notu satır 34: ekleme hazırla>  
<kenar notu satır 36: satırı ekle>  
<kenar notu satır 41: satırı sil>  
<kenar notu satır 47: rowSet'i kapat>

```
java.sql.*'ı içe aktarın;
javax.sql.RowSet'i içe aktarın;
import com.sun.rowset.JdbcRowSetImpl;

public class ScrollUpdateRowSet {
 public static void main(String[] args)
 throws SQLException, ClassNotFoundException {
 // JDBC sürücüsünü yükle
 Class.forName("oracle.jdbc.driver.OracleDriver");
 System.out.println("Sürücü yüklendi");

 // Bir satır seti oluştur
 RowSet rowSet = new JdbcRowSetImpl();

 // RowSet özelliklerini ayarla
 rowSet.setUrl("jdbc:oracle:thin:@liang.armstrong.edu:1521:orcl");
 rowSet.setUsername("scott");
 rowSet.setPassword("kaplan");
 rowSet.setCommand("StateCapital'den durumu, başkenti seçin");
 rowSet.execute();

 System.out.println("Güncellemeden önce ");
 displayRowSet(rowSet);

 // İkinci satırı güncelle
 rowSet.absolute(2); // İmleci 2. satıra hareket ettir
 rowSet.updateString("state", "New S"); // Sütunu güncelle
 rowSet.updateString("capital", "New C"); // Sütunu güncelle
 rowSet.updateRow(); // Veritabanındaki satırı güncelle

 // İkinci satırdan sonra ekle
 rowSet.last();
 rowSet.moveToInsertRow(); // İmleci ekleme satırına hareket ettir
 rowSet.updateString("state", "Florida");
 rowSet.updateString("capital", "Tallahassee");
 rowSet.insertRow(); // Satırı ekle
 rowSet.moveToCurrentRow(); // İmleci mevcut satıra taşı

 // Dördüncü satırı sil
 rowSet.absolute(4); // İmleci beşinci satıra taşı
 rowSet.deleteRow(); // İkinci satırı sil
```

```

 System.out.println("Güncellemeden sonra ");
 displayRowSet (rowSet);

 // Bağlantıyı kapat
 rowSet.close();

}

özel statik void displayRowSet(RowSet rowSet)n, SQLException
on'ı atar {
 ResultSetMetaData rsMetaData = rowSet.getMetaData();n rowSet
 .beforeFirst();
 while (rowSet.next()) {
 for (int i = 1; i <= rsMetaData.getColumnCount(); i++)n System.out.p
 rintf("%-12st", rowSet.getObject(i));n System.out.println();

 }
}
}
}
}

```

**<kenar notu: CachedRowSet'in güncellenmesi>**

JdbcRowSet'i 13. satırda CachedRowSet ile değiştirirseniz, veritabanı değişmez. CachedRowSet'teki değişikliklerin veritabanında etkili olması için, tüm değişiklikleri yaptıktan sonra `acceptChanges()` yöntemini çağırmanız, aşağıdaki gibi:

```

// Değişiklikleri veritabanına yaz
((com.sun.rowset.CachedRowSetImpl) rowSet).acceptChanges();

```

Bu yöntem otomatik olarak veritabanına yeniden bağlanır ve tüm değişiklikleri veritabanına yazar.

#### 41.5.4 RowSetEvent

Bir RowSet nesnesi, nesnenin imleci hareket ettiğinde, bir satır değiştiğinde veya tüm satır kümesi değiştiğinde bir RowSetEvent tetikler. Bu olay, bir RowSet'i RowSet'e bağmlı bileşenlerle senkronize etmek için kullanılabilir. Örneğin, RowSet'in içeriğini görüntüleyen bir görsel bileşen, RowSet ile senkronize olmalıdır. RowSetEvent, senkronizasyonu sağlamak için kullanılabilir. RowSetListener'daki yardımcılar `cursorMoved(RowSetEvent)`, `rowChanged(RowSetEvent)` ve `cursorSetChanged(RowSetEvent)` dir.

Liste 41.7, RowSetEvent'i gösteren bir örnek vermektedir. Bir dinleyici RowSetEvent için 14-26. satırlarda kaydedilmiştir. `rowSet.execute()`(33. satır) çalıştırıldığında, tüm satır seti değişir, bu nedenle dinleyicinin `rowSetChanged` işleyicisi çağrılır. `rowSet.last()` (35. satır) çalıştırıldığında, imleç hareket eder, bu nedenle dinleyicinin `cursorMoved` işleyicisi çağrılır. `rowSet.updateRow()` (37. satır) çalıştırıldığında, satır güncellenir, bu nedenle dinleyicinin `rowChanged` işleyicisi çağrılır.

#### Liste 41.7 TestRowSetEvent.java

**<margin note line 9: sürücüyü yükle>**

**<margin note line 13: RowSet oluştur>**

**<margin note line 14: dinleyiciyi kaydet>**

<margin note line 33: satır seti değişti>  
<margin note line 35: imleç hareket etti>  
<margin note line 37: satır güncellendi>

```
java.sql.*'ı içe aktarın;
javax.sql.*'ı içe aktarın;
com.sun.rowset.*'i içe aktarın;

public class TestRowSetEvent {
 public static void main(String[] args)
 throws SQLException, ClassNotFoundException {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir satır seti oluştur
 RowSet rowSet = new JdbcRowSetImpl();
 rowSet.addRowSetListener(new RowSetListener() {
 public void cursorMoved(RowSetEvent e) {
 System.out.println("İmleç hareket etti");
 }

 public void rowChanged(RowSetEvent e) {
 System.out.println("Satır değişti");
 }

 public void rowSetChanged(RowSetEvent e) {
 System.out.println("satır seti değişti");
 }
 });

 // RowSet özelliklerini ayarla
 rowSet.setUrl("jdbc:mysql://localhost/javabook");
 rowSet.setUsername("scott");
 rowSet.setPassword("kaplan");
 rowSet.setCommand("Öğrenciden * seç");
 rowSet.executeUpdate();

 rowSet.last(); // İmleç taşındı
 rowSet.updateString("lastName", "Yao"); // Sütunu güncelle
 rowSet.updateRow(); // Satır güncellendi

 // Bağlantıyı kapat
 rowSet.close();
 }
}
```

#### 41.6 Özel RowSetTableModel

Sıklıkla bir sorgu sonuç kümesini bir JTable'de görüntülemeniz gerekir. Bir satır seti için bir tablo modeli tanımlayabilir ve bu modeli bir JTable'ye bağlayabilirsiniz. Bir tablo modeli tanımlamak için, AbstractTableModel sınıfını genişletin ve en az üç yöntem uygulayın: getRowCount(), getColumnCount() ve getValueAt(int row,

int column). AbstractTableModel sınıfı §40.3'te tanıtılmıştır, "Tablo Modelleri ve Tablo Sütun Modelleri."

Liste 41.8 RowSetTableModel sınıfını göstermektedir.

#### Liste 41.8 RowSetTableModel.java

<margin note line 8: rowSet>  
<margin note line 11: getRowSet>  
<margin note line 16: setRowSet>  
<margin note line 19: dinleyici ekle>  
<margin note line 25: getRowCount()>  
<margin note line 38: getColumnCount()>  
<margin note line 52: getValueAt>  
<margin not satırı 65: getColumnName()>n<margin  
rgin not satırı 77: rowSetChanged>n<margin  
not satırı 83: rowChanged>n<margin not  
satırı 89: CursorMoved>

```
java.sql.*'ı içe aktarın;
javax.sql.*'ı içe aktarın;
javax.swing.table.AbstractTableModel'i içe aktar;

public class RowSetTableModel, AbstractTableModel'ı genişletir
 RowSetListener'ı uygular {
 // Sonuç kümesi için RowSet
 private RowSet rowSet;

 /** Rowset'i döndür */
 public RowSet getRowSet() {
 return rowSet;
 }

 /** Yeni bir rowset ayarla */
 public void setRowSet(RowSet rowSet) {
 if (rowSet != null) {
 this.rowSet = rowSet;n ro
wSet.addRowSetListener(this);n fireTa
bleStructureChanged();
 }
 }

 /** Row setindeki satır sayısını döndür */
 public int getRowCount() {
 denemek {
 rowSet.last();
 return rowSet.getRow(); // Mevcut satır numarasını al
 }
 catch (Exception ex) {n ex.p
rintStackTrace();n }
 }

 0;n'yi döndür
}
```

```

/** Satır kümesindeki sütun sayısını döndür */
public int getColumnCount() {
 denemek {
 if (rowSet != null) {
 return rowSet.getMetaData().getColumnCount();n }
 }
 } catch (SQLException ex) {n ex.p
 rintStackTrace();n }

 0;n'yi döndür
}

/** Belirtilen satır ve sütundaki değeri döndür */
public Object getValueAt(int row, int column) {
 try {
 rowSet.absolute(row + 1);n ret
 urn rowSet.getObject(column + 1);n }

 catch (SQLException sqlex) {n sqle
 x.printStackTrace();n }

 null döndür;

/** Belirtilen sütundaki sütun adını döndür */
public String getColumnName(int column) {
 denemek {
 return rowSet.getMetaData().getColumnLabel(column + 1);n }

 catch (SQLException ex) {n ex.p
 rintStackTrace();n }

 dönüş "";n }

/** rowSetChanged'i uygulayın */
public void rowSetChanged(RowSetEvent e) {
 System.out.println("RowSet değişti");
 fireTableStructureChanged();
}

/** rowChanged'i uygulayın */
public void rowChanged(RowSetEvent e) {
 System.out.println("Satır değişti");
 fireTableDataChanged();
}

/** cursorMoved'i uygulayın */
public void cursorMoved(RowSetEvent e) {

```



```

 System.out.println("İmleç taşındı");
 }
}

```

RowSetTableModel sınıfı, rowSet özelliğini alma (get) ve ayarlama (set) yöntemleri ile tanımlar (satırlar 11-22). setRowSet yöntemi, modelde yeni bir rowSet ayarlar. Model, rowSet'teki değişikliklere yanıt olarak rowSet için bir dinleyici haline gelir (satır 19). AbstractTableModel'de tanımlanan fireTableStructureChanged() yöntemi, modelin rowSet'te ki verilerle yeniden doldurulması için çağrılır (satır 20).

getRowCount() yöntemi, rowSet'teki satır sayısını döndürür.  
rowSet.last() çağırısı, imleci son satıra (satır 27) taşır ve  
rowSet.getRow() satır numarasını (satır 28) döndürür.

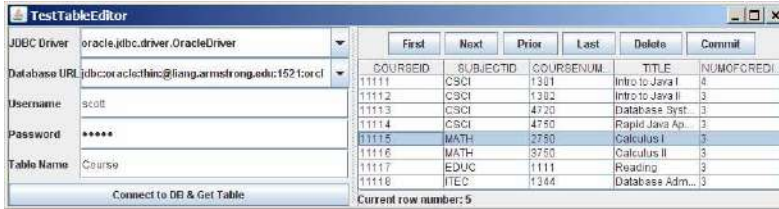
getColumnCount() metodu, rowSet'teki sütun sayısını döndürür.  
rowSet'teki sütun sayısı, meta  
data'dan elde edilebilir (satır 41).

getValueAt(row, column) metodu, belirtilen satır ve sütundaki hücre değerini döndürür (satırlar 52-62). getColumnName(column) metodu, belirtilen sütun için sütun adını döndürür (satırlar 65-74).

NOT  
<kenar notu: dizin tutarsızlığı>  
JTable'deki satır ve sütun indeksi 0 tabanlıdır.  
Ancak, RowSet'teki satır ve sütun indeksi 1-abanlıdır.

RowSetTableModel, RowSetListener'ı uygular (satırlar 77-91). Bu nedenle, bir RowSetTableModel, RowSet olayları için bir dinleyici olabilir.

Şimdi, bir row set'i JTable'de görüntüleyen yararlı bir yardımcı program geliştirmeye odaklanalım. Şekil 41.4'te gösterildiği gibi, bir JDBC sürücüsü ve veritabanı girer veya seçer, bir kullanıcı adı ve bir parola girer ve veritabanına bağlanmak ve tablo içeriğini JTable'de görüntülemek için bir tablo adı belirtirsiniz. Ardından, imleci tablodaki ilk satıra, sonraki satıra, önceki satıra ve son satıra taşımak için İlk, Sonraki, Önceki, ve Son düğmelerini kullanabilir, seçili bir satırı silmek için Sil düğmesini kullanabilir ve veritabanındaki değişikliği kaydetmek için Onayla düğmesini kullanabilirsiniz.



Şekil 41.4

Program, tabloyu gezmenizi ve satırları silmenizi sağlar.

Pencerenin altındaki durum çubuğu, row set içindeki mevcut satırı gösterir. Row set içindeki imleç ve JTable'deki satır senkronizedir. İmleci, gezinme düğmelerini kullanarak veya JTable'deki bir satırı seçerek hareket ettirebilirsiniz.

İki sınıf tanımlayın: TestTableEditor (Liste 41.9) ve TableEditor(Liste 41.10). TestTableEditor, kullanıcının veritabanı bağlantı bilgilerini ve bir tablo adını girmesini sağlayan ana sınıftır. Veritabanı bağlandığında, tablo içeriği TableEditor örneğinde görüntülenir. TableEditor sınıfı, bir tabloyu gezmek ve tabloyu değiştirmek için kullanılabilir.

#### Liste 41.9 TestTableEditor.java

<margin note line 8: sürücüler>  
 <margin note line 13: url'ler>  
 <margin note line 19: UI bileşenleri>  
 <margin note line 29: UI oluştur>  
 <margin note line 60: sürücüyü yükle>  
 <margin note line 61: rowSet oluştur>  
 <margin note line 62: url ayarla>  
 <margin note line 63: kullanıcı adını ayarla>  
 <margin note line 64: parolayı ayarla>  
 <margin note line 65: komutu ayarla>  
 <margin note line 67: komutu çalıştır>  
 <margin note line 69: rowSet'i tableEditor1'e>  
 <margin note line 77: ana yöntem atlandı>

```

import javax.swing.*; içe aktar
import java.awt.*; içe aktar
import java.awt.event.*; içe aktar
import javax.sql.RowSet'i içe aktarın;
import com.sun.rowset.CachedRowSetImpl;

public class TestTableEditor extends JApplet {
 private JComboBox jcboDriver = new JComboBox(new String[] {
 "sun.jdbc.odbc.JdbcOdbcDriver",
 "com.mysql.jdbc.Driver",n "Oracle.jdbc.driver.OracleDriver"n });

 özel JComboBox jcboURL = new JComboBox(new String[] {n "jdbc:odbc:
 exampleMDBDataSource",
 "jdbc:mysql://localhost/javabook",n "jdbc:
 oracle:thin:@liang.armstrong.edu:1521:orcl"n });

 özel JButton jbtConnect =
 new JButton("Veritabanına Bağlan & Tablo Al");
 private JTextField jtfUserName = new JTextField();
 private JPasswordField jpfPassword = new JPasswordField();
 private JTextField jtfTableName = new JTextField();
 özel TableEditor tableEditor1 = new TableEditor();n özel JLabel lbl
 Status = new JLabel();

 /** Yeni form TestTableEditor oluşturur */
 public TestTableEditor() {
 JPanel jPanel1 = new JPanel(new GridLayout(5, 0));n jPanel1.ad
 d(jcboDriver);

```

```

jPanel1.add(jcboURL);
jPanel1.add(jtfUserName);
jPanel1.add(jpfPassword);
jPanel1.add(jtfTableName);

JPanel jPanel2 = new JPanel(new GridLayout(5, 0));
jPanel2.add(new JLabel("JDBC Sürücüsü"));
jPanel2.add(new JLabel("Veritabanı URL'si"));
jPanel2.add(new JLabel("Kullanıcı Adı"));
jPanel2.add(new JLabel("Parola"));
jPanel2.add(new JLabel("Tablo Adı"));

JPanel jPanel3 = new JPanel(new BorderLayout());
jPanel3.add(jbtConnect, BorderLayout.SOUTH);
jPanel3.add(jPanel2, BorderLayout.WEST);
jPanel3.add(jPanel1, BorderLayout.CENTER);
tableEditor1.setPreferredSize(new Dimension(400, 200));

jcboURL.setEditable(true);n jcbo
Driver.setEditable(true);n add(new J

SplitPane(JSplitPane.HORIZONTAL_SPLIT,n jPanel3, tableE
ditor1), BorderLayout.CENTER);n add(jlblStatus, BorderL
ayout.SOUTH);

jbtConnect.addActionListener(new ActionListener() {n public voi
d actionPerformed(ActionEvent evt) {n try {

// Veritabanına bağlan ve bir rowset oluştur
Class.forName(((String)jcboDriver.getSelectedItem()).trim());
RowSet rowSet = new CachedRowSetImpl();
rowSet.setUrl(((String)jcboURL.getSelectedItem()).trim());n rowSet.setUs
ername(jtfUserName.getText().trim());
rowSet.setPassword(new String(jpfPassword.getPassword()));n rowSet.se
tCommand("select * from " +
jtfTableName.getText().trim());n rowSe
t.execute();
rowSet.beforeFirst();n tabl
eEditor1.setRowSet(rowSet);n }

catch (Exception ex) {n lblS
tatus.setText(ex.toString());n }

}
n});n
}
}

```

Kullanıcı *DB'ye Bağlan & Tablo Al* butonuna tıkladığında, bir *CachedRowSet* oluşturulur (satır 61). URL, kullanıcı adı, parola ve bir komut satır setinde ayarlanır (satırlar 62-66). Satır seti çalıştırılır (satır 67) ve *TableEditor*'a bağlanır (satır 69).

#### Listeleme 41.10 TableEditor.java



```

 /** Bir TableEditor oluřtur */
 public TableEditor() {
 JPanel jPanel1 = new JPanel();
 jPanel1.add(jbtFirst);
 jPanel1.add(jbtSonraki);
 jPanel1.add(jbtPrior);
 jPanel1.add(jbtLast);
 jPanel1.add(jbtDelete);
 jPanel1.add(jbtCommit);

 setLayout(new BorderLayout());n add(
 jPanel1, BorderLayout.NORTH);n add(ne
 w JScrollPane(jTable1), BorderLayout.CENTER);n add(jlblStatus,
 BorderLayout.SOUTH);

 // Tablo için seçim modelini ayarla
 jTable1.setSelectionModel(listSelectionModel);

 // Dinleyicileri kaydet
 jbtFirst.addActionListener(new ActionListener() {n public void
 actionPerformed(ActionEvent evt) {n moveCursor("first");

 }
 })
 jbtNext.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent evt) {
 moveCursor("sonraki");
 }
 })
 jbtPrior.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent evt) {
 moveCursor("önceki");
 }
 })
 jbtLast.addActionListener(new ActionListener() {
 public void actionPerformed(ActionEvent evt) {
 moveCursor("son");
 }
 })
 jbtDelete.addActionListener(new ActionListener() {n public voi
 d actionPerformed(ActionEvent evt) {n delete();

 }
 })
 jbtCommit.addActionListener(new ActionListener() {n public v
 oid actionPerformed(ActionEvent evt) {n try {

 ((CachedRowSetImpl)rowSet).acceptChanges();n }

 catch (java.sql.SQLException ex) {n ex.prin
 tStackTrace();

 }
 }

```

```

 }
}

listSelectionModel.addListSelectionListener(n new List
SelectionListener() {
 public void valueChanged(ListSelectionEvent e) {n handSele
ctionValueChanged(e);
 }
}

}

/* Bir satırı sil */
private void delete() {
 try {
 // Veritabanından kaydı sil
 int currentRow = rowSet.getRow();
 rowSet.deleteRow();
 if (rowSet.isAfterLast())n rowSe
t.last();
 else if (rowSet.getRow() >= currentRow)n rowSet.
absolute(currentRow);
 setTableCursor();n }

 catch (java.sql.SQLException ex) {n jlblStat
us.setText(ex.toString());n }

}

/** Tablo içinde imleci ayarla ve durumdaki satır numarasını ayarla */
private void setTableCursor() throws java.sql.SQLException {
 int satır = rowSet.getRow();
 listSelectionModel.setSelectionInterval(row - 1, row - 1);
 jlblStatus.setText("Mevcut satır numarası: " + row);
}

/** İmleci belirtilen konuma taşı */
private void moveCursor(String whereToMove) {
 try {
 if (whereToMove.equals("ilk"))n rowSet.
first();
 else if (whereToMove.equals("sonraki") && !rowSet.isLast())n rowSet.n
ext();
 else if (whereToMove.equals("önceki") && !rowSet.isFirst())n rowSet.önceki()
;
 else if (whereToMove.equals("last"))n rowSet
.last();
 setTableCursor();n }

 catch (java.sql.SQLException ex) {n jlblStat
us.setText(ex.toString());n }

}

/** Tablo içindeki seçimi işleyin */
private void handleSelectionValueChanged(ListSelectionEvent e) {

```

```

 int seçilmişRow = jTable1.getSelectedRow();

 denemek {
 if (selectedRow != -1) {n rowSe
 t.absolute(selectedRow + 1);n setTableCu
 rsor();
 }
 }
 catch (java.sql.SQLException ex) {n jlblStat
 us.setText(ex.toString());n }
 }
}
N}

```

setRowSet yöntemi (satırlar 26-35) TableEditor'da yeni bir satır seti ayarlar. rowSet tablo modeline (satır 29) bağlanır ve tablo modeli tabloya (satır 32) eklenir. Satır 32-34'teki kod, sütun adlarının sıralanmasını sağlar.

Gezinme düğmelerinin işlenmesi *İlk*, *Sonraki*, *Önceki* ve *Son* sadece first(), next(), previous() ve last() yöntemlerini çağırarak rowSet içinde imleci hareket ettirmek ve (satırlar 126-133) aynı zamanda jTable'da seçili satırı setTableCursor() (satır 134) çağırarak ayarlamaktır.

Sil düğmesini uygulamak için, rowSet'ten satırı kaldırmak için deleteRow() yöntemini (satır 104) çağırın. Satır kaldırıldıktan sonra, imleci rowSet içindeki bir sonraki satıra ayarlayın (satırlar 105-108) ve tablodaki imleci senkronize edin (satır 109).

deleteRow() yöntemi, CachedRowSet'ten satırı kaldırır. Commit butonu, değişiklikleri veritabanına kaydeder (satır 84) .

jTable1'de liste seçim olayları için işleyiciyi uygulamak için, satır kümesindeki cursor'ı jTable1'de seçilen satırla eşleştirecek şekilde ayarlayın (satırlar 142-154) .

#### 41.7 JDBC'de Görüntüleri Saklama ve Alma

Bir veritabanı yalnızca sayıları ve dizeleri değil, aynı zamanda görüntüleri de saklayabilir. SQL3, ikili verileri saklamak için BLOB (Binary Large Object) adlı yeni bir veri türü tanıttı; bu, görüntüleri saklamak için kullanılabilir. Diğer bir yeni SQL3 türü, karakter formatında büyük metin saklamak için CLOB (Character Large Object) türüdür. JDBC, bu yeni SQL türleri için eşleme desteği sağlamak amacıyla java.sql.Blob ve java.sql.Clob arayüzlerini tanıttı. SQL BLOB ve CLOB değerlerine ResultSet ve PreparedStatement arayüzlerinde erişmek için getBlob, setBinaryStream, getClob, setBlob ve setClob yöntemlerini kullanabilirsiniz.

Bir görüntüyü bir tablodaki bir hücreye saklamak için, hücre için ilgili sütun BLOB türünde olmalıdır. Örneğin, aşağıdaki SQL deyimi, bayrak sütununun türü BLOB olan bir tablo oluşturur.

```

tablo oluşturun Ülke(isim varchar(30), bayrak blob, açıklama ve
rchar(255));

```

Önceki ifadede, açıklama sütunu 255 karakterle sınırlıdır, bu da MySQL için üst sınırdır. Oracle için üst sınır 32,762 bayttır. Büyük bir karakter alanı için, Oracle'da iki GB karakter depolayabilen CLOB türünü kullanabilirsiniz. MySQL CLOB'u desteklemez. Ancak, uzun bir dize depolamak için BLOB kullanabilir ve binary verileri karakterlere dönüştürebilirsiniz.

NOT

**<margin note: destekleniyor?>**

Access BLOB ve CLOB türlerini desteklemez.

Bir tabloya resimlerle bir kayıt eklemek için, aşağıdaki gibi bir hazırlanan ifade tanımlayın:

```
PreparedStatement pstmt = connection.prepareStatement(
 "insert into Country values(?, ?, ?)");
```

**<margin note: resmi depola>**

Resimler genellikle dosyalarda depolanır. Öncelikle bir resim dosyası için InputStream örneğini alabilir ve ardından setBinaryStream yöntemini tablo hücrelerine bir giriş akışı ile ilişkilendirmek için kullanabilirsiniz, aşağıdaki gibi:

```
// Resmi tablo hücrelerine depola
File file = new File(imageFilename);
InputStream inputImage = new FileInputStream(file);
pstmt.setBinaryStream(2, inputImage, (int) (file.length()));
```

**<margin note: resmi al>**

Bir tablodan resmi almak için, aşağıda gösterildiği gibi getBlob yöntemini kullanın:

```
// Resmi tablo hücrelerine depola
Blob blob = rs.getBlob(1);
ImageIcon imageIcon = new ImageIcon(
 blob.getBytes(1, (int) blob.length()));
```

Listing 41.11, JDBC'de resimleri nasıl saklayıp alacağınızı gösteren bir programı vermektedir. Program önce Ülke tablosunu oluşturur ve dataları buraya kaydeder. Ardından program, tablodan ülke isimlerini alır ve bunları bir kombinasyon kutusuna ekler. Kullanıcı kombinasyon kutusundan bir isim seçtiğinde, ülkenin bayrağı ve açıklaması görüntülenir, bu da Şekil 41.5'te gösterilmiştir.



**Şekil 41.5**

Program, bir tablodan veri, resimler de dahil olmak üzere, almanıza ve bunları görüntülemenize olanak tanır.

#### **Listeleme 41.11 StoreAndRetrieveImage.java**



<margin note line 45: sürücüyü yükle>  
 <margin note line 49: veritabanına bağlan>  
 <margin note line 54: ifade oluştur>  
 <margin note line 57: ifadeyi hazırla>  
 <margin note line 62: verileri veritabanına yükle>  
 <margin note line 75: ekle>  
 <margin note line 83: resim URL'sini al>  
 <margin note line 86: ikili akış>  
 <margin note line 103: kombinasyon kutusunu doldur>  
 <margin note line 109: ismi ayarla>  
 <margin note line 113: resim simgesini al>  
 <margin note line 118: açıklamayı ayarla>  
 <margin note line 125: ana yöntem atlandı>

```

java.sql.*'ı içe aktarın;
java.io.*'yu içe aktarın;
javax.swing.*; içe aktar
java.awt.*; içe aktar
java.awt.event.*; içe aktar

public class StoreAndRetrieveImage extends JApplet {
 // Veritabanına bağlantı
 private Connection connection;

 // Statik SQL ifadeleri için ifade
 private Statement stmt;

 // Hazırlanmış ifade
 özel Hazırlanan Bildirim pstmt = null; özel Açıkla
 maPanel açıklamasıPanel1 n = yeni AçıklamaPanel();

 özel JComboBox jcboCountry = new JComboBox();

 /** Yeni form StoreAndRetrieveImage oluşturur */
 public StoreAndRetrieveImage() {
 denemek {
 connectDB(); // Veritabanına bağlan
 storeDataToTable(); // Verileri tabloya kaydet (resim dahil)
 fillDataInComboBox(); // Kombinasyon kutusunu doldur
 getFlagInfo((String)(jcboCountry.getSelectedItem()));n }

 catch (Exception ex) {n ex.p
 rintStackTrace();n }

 jcboCountry.addItemListener(new ItemListener() {n public v
 oid itemStateChanged(ItemEvent evt) {n getFlagInfo((Stri
 ng)(evt.getItem()));n }

 }

 add(jcboCountry, BorderLayout.NORTH);n add
 (descriptionPanel1, BorderLayout.CENTER);

 private void connectDB() throws Exception {
 // Sürücüyü yükle

```

```

Class.forName("com.mysql.jdbc.Driver");
System.out.println("Sürücü yüklendi");

// Bağlantıyı kur
connection = DriverManager.getConnection
("jdbc:mysql://localhost/javabook", "scott", "tiger");
System.out.println("Veritabanı bağlandı");

// Statik SQL için bir ifade oluştur
stmt = connection.createStatement();

// Bayrak ve açıklamayı almak için bir hazırlıklı ifade oluştur
pstmt = connection.prepareStatement("select flag, description " +
"from Country where name = ?");
}

private void storeDataToTable() {
String[] countries = {"Kanada", "İngiltere", "ABD", "Almanya",
"Hindistan", "Çin"};

String[] image Dosya Adları = {"image/a.gif", "image/uk.gif",ve "image/us.
gif", "image/germany.gif", "image/india.gif",n "image/china.gif"};

String[] descriptions = {"Kanada bayrağını tanımlayan bir metin " +
"hariç tutulmuştur", "Britanya bayrağı ...", "Amerikan bayrağı ...",
"Alman bayrağı ...", "Hindistan bayrağı ...", "Çin bayrağı ..."};

denemek {
// Kayıtları eklemek için bir hazırlık ifadesi oluştur
PreparedStatement pstmt = connection.prepareStatement(
"insert into Country values(?, ?, ?)");

// Tüm önceden tanımlanmış kayıtları sakla
for (int i = 0; i < countries.length; i++) {
pstmt.setString(1, countries[i]);

// Resmi tablo hücrelerine kaydet
java.net.URL url =
this.getClass().getResource(imageFileNames[i]);n OutputStr
eam inputImage = url.openStream();
pstmt.setBinaryStream(2, inputImage,n (int)(i
nputImage.available()));n pstmt.setStri

ng(3, açıklamalar[i]);n pstmt.executeUpdate();
}

System.out.println("Country Tablosu dolduruldu");
}
catch (Exception ex) {n ex.p
rintStackTrace();n }
}

özel void fillDataInComboBox(), İstisna atar {n ResultSet rs = stmt.
executeQuery("Ülkeden ad seçin");n while (rs.next()) {

jcboCountry.addItem(rs.getString(1));n }
}

```

```

 özel void getFlagInfo(Dize adı) {n deneyin {
 pstmt.setString(1, name);n Res
 ultSet rs = pstmt.executeQuery();n if (rs.next()
) {
 Blob blob = rs.getBlob(1);n Imag
 eIcon imageIcon = new ImageIcon(n blob.get
 Bytes(1, (int)blob.length()));n açıklamaPanel1.set
 ImageIcon(imageIcon);n açıklamaPanel1.setName(n
 ame);
 Dize açıklaması = rs.getString(2);n açıklamaPa
 nel1.setDescription(description);n }
 }
 catch (Exception ex) {n Syst
 em.err.println(ex);n }
}
}

```

DescriptionPanel (satır 14) bir ülkeyi gösteren bir bileşendir (isim, bayrak ve açıklama). Bu bileşen, Listing 17.2, DescriptionPanel.java'da sunulmuştur.

storeDataToTable yöntemi (satırlar 58-95) tabloyu verilerle doldurur. fillDataInComboBox yöntemi (satırlar 97-102) ülke isimlerini alır ve bunları açılır kutuya ekler. retrieveFlagInfo(name) yöntemi (satırlar 104-121) belirtilen ülke ismi için bayrağı ve açıklamayı alır.

#### Anahtar Terimler

- **BLOB türü**
- **CLOB türü**
- **toplu mod**
- **önbellek satırı kümesi**
- **satır kümesi**
- **kaydırılabilir sonuç kümesi**
- **güncellenebilir sonuç kümesi**

#### Bölüm Özeti

1. Bu bölüm, herhangi bir yerel veya uzak ilişkisel veritabanına erişmek için kullanılabilir ecek evrensel bir SQL istemcisi geliştirdi.
2. Toplu işleme için bir ifadeye SQL deyimlerini eklemek üzere addBatch(SQLString) yöntemini kullanabilirsiniz.
3. Sonuç kümesinin kaydırılabilir ve güncellenebilir olmasını belirtmek için bir ifade oluşturabilirsiniz. Varsayılan olarak, sonuç kümesi bunlardan hiçbiri değildir.
4. RowSet, Java veritabanı programlamasını basitleştirmek için kullanılabilir. Bir RowSet nesnesi kaydırılabilir ve güncellenebilir. Bir RowSet, bir RowSetEvent tetikleyebilir.
5. JDBC kullanarak SQL BLOB

tipini kullanarak resim verilerini depolayabilir ve alabilirsiniz.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

## Gözden Geçirme Soruları

### Bölüm 41.3

41.1 JDBC'de toplu işleme nedir? Toplu işlemenin faydaları nelerdir?

41.2 Bir SQL deyimini bir toplu işleme nasıl ekleyebilirsiniz? Bir toplu işlemi nasıl çalıştırırsınız?

41.3 Bir SELECT deyimini bir toplu işleme çalıştırabilir misiniz?

41.4 JDBC sürücüsünün toplu güncellemeleri destekleyip desteklemediğini nasıl anlarsınız?

### Bölüm 41.4

41.5 Kaydırılabilir bir sonuç kümesi nedir? Güncellenebilir bir sonuç kümesi nedir?

41.6 Kaydırılabilir ve güncellenebilir bir ResultSet nasıl oluşturulur?

41.7 Bir JDBC sürücüsünün kaydırılabilir ve güncellenebilir bir ResultSet'i destekleyip desteklemediğini nasıl anlarsınız?

### Bölümler 41.5-41.6

41.8 RowSet'in avantajları nelerdir?

41.9 jdbcRowSet ve CachedRowSet nedir? Aralarındaki farklar nelerdir?

41.10 Bir jdbcRowSet ve CachedRowSet nasıl oluşturulur?

41.11 Bir RowSet'i kaydırabilir ve güncelleyebilir misiniz? CachedRowSet'teki değişiklikleri veritabanına yazmak için hangi yöntem çağrılmalıdır?

41.12 RowSetListener'daki işleyicileri tanımlayın.

### Bölüm 41.7

41.13 Görselleri bir veritabanına nasıl kaydedersiniz?

41.14 Veritabanından resimleri nasıl alırsınız?

41.15 Oracle, SQL3 BLOB türünü ve CLOB türünü destekliyor mu? Peki ya MySQL ve Access?

## Alıştırmalar

### 41.1\*

(Toplu güncelleme) Veritabanına bin kayıt ekleyen bir program yazın ve toplu güncellemelerle ve toplu güncellemeler olmadan performansını karşılaştırın, Şekil 41.6a'da gösterildiği gibi. Tablo aşağıdaki gibi tanımlanmış olsun:

**tablet Temp oluştur(num1 t doublet, num2 t doublet, num3 t doublet)**

Her kayıt için rastgele sayılar oluşturmak üzere Math.random() yöntemini kullanın.

Alıştırma 37.3'te tartışılan DBConnectionPanel içeren bir iletişim kutusu oluşturun. Bu iletişim kutusunu veritabanına bağlanmak için kullanın. Şekil 41.6a'daki *Veritabanına Bağlan* butonuna tıkladığınızda, Şekil 41.6b'deki iletişim kutusu görüntülenir.



(a)

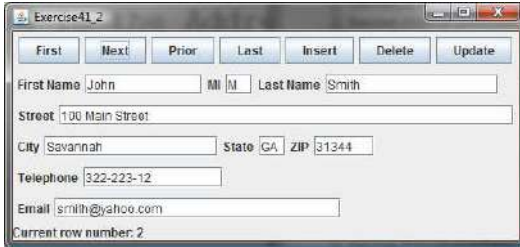
(b)

Şekil 41.6

Program, toplu güncellemeler kullanmanın sonucunda elde edilen performans iyileştirmelerini göstermektedir.

41.2\*\*

(Kaydırılabilir sonuç kümesi) İlk, Sonraki, Önceki, Son, Ekle, Sil, ve Güncelle düğmelerini kullanarak bir program yazın ve Adres tablosundaki tek bir kaydı, Şekil 41.7'de gösterildiği gibi değiştirin.



Şekil 41.7

Düğmeleri kullanarak

Adres tablosundaki tek bir kaydı görüntüleyebilir ve değiştirebilirsiniz.

41.3\*\*

( *ResultSetTableModel* ) Liste 41.8, RowSetTableModel.java, RowSet için bir tablo modeli tanımlar. ResultSet için ResultSetTableModel adında yeni bir sınıf geliştirin. ResultSetTableModel, AbstractTableModel'den türetilir. Şekil 41.8'de gösterildiği gibi, Course tablosunu bir JTable'de görüntüleyen bir test programı yazın. Sütunlarda otomatik sıralamayı etkinleştirin.

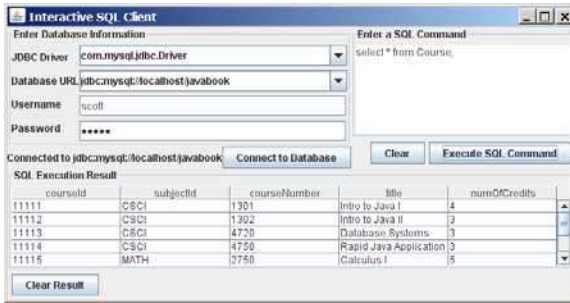
courseId	subjectId	courseNumber	title	numOfCredits
11111	CSCI	1301	Introduction to Java I	4
11112	CSCI	1302	Intro to Java II	3
11113	CSCI	4720	Database Systems	3
11114	CSCI	4750	Rapid Java Application	3
11115	MATH	2750	Calculus I	5
11116	MATH	3750	Calculus II	5
11117	EDUC	1111	Reading	3
11118	ITEC	1344	Database Administration	3

**Şekil 41.8**

*Course tablosu, ResultSetTableModel kullanılarak bir JTable'de görüntülenir.*

41.4\*\*

(Revise `SQLClient.java`) Liste 41.1, `SQLClient.java`'yı  
Şekil 41.9'da gösterildiği gibi, sorgu sonucunu bir `JTable`'de görüntüleyecek şekilde yeniden yazın.



**Şekil 41.9**

*Sorgu sonucu bir JTable 'de görüntülenir.*

41.5\*\*\*

(Edit table using RowSet) Liste 41.10'u, yeni bir satır eklemek için bir `Insert` butonu ve satırı güncellemek için bir `Update` butonu ekleyecek şekilde yeniden yazın.

41.6\*

(Display images from database) Liste 41.11'de oluşturulan `Country` tablosunu görüntülemek için `JTable` kullanan bir program yazın, `StoreAndRetrieveImage.java`, Şekil 41.10'da gösterildiği gibi.



**Şekil 41.10**

*Ülke tablosu bir JTable örneğinde görüntülenir.*\_\_\_\_\_

41.7\*\*

(RowSet kullanarak resimleri saklayın ve alın) RowSet kullanarak Listing 41.11, StoreAndRetrieveImage.java'daki örneği yeniden yazın.

41.8\*

(*Maaş tablosunu doldurun*) Performansı artırmak için Egzersiz 33.8'i bir toplu mod kullanarak yeniden yazın.

**\*\*\*Bu bir bonus Web bölümü**

## BÖLÜM 42

### Servletler

#### Amaçlar

- Bir servletin nasıl çalıştığını açıklamak (§42.2).
- Servletleri oluşturmak/geliştirmek/çalıştırmak (§42.3).
- Servletleri Tomcat ve  
GlassFish gibi uygulama sunucularında dağıtmak (§42.3).
- Servletler API'sini tanımlamak (§42.4).
- Basit servletler oluşturmak (§42.5).
- HTML formlarını oluşturmak ve işlemek (§42.6).
- Veritabanlarına erişmek için servletler geliştirmek (§42.7).
- Oturumları izlemek için gizli alanlar, çerezler ve HttpSession kullanmak (§42.8).
- Servletlerden resim göndermek (§42.9).



## 42.1 Giriş

### <Yan Not: servlet>

**Servletler** Web sunucusunda çalışan Java programlarıdır. İstemci isteklerini işlemek veya dinamik Web sayfaları üretmek için kullanılabilirler. Örneğin, hisse senedi alıntılarını görün tüleyen dinamik Web sayfaları oluşturmak veya istemci kayıt formlarını işlemek ve kayıtlarını bir veritabanında depolamak için servletler yazabilirsiniz. Bu bölüm, Java servletlerinin kavramını tanıtmaktadır. Java servletlerini NetBeans kullanarak nasıl geliştireceğinizi öğreneceksiniz.

NOT:

### <Yan Not: neden NetBeans?>

Bir IDE kullanmadan servletler geliştirebilirsiniz. Ancak, NetBeans gibi bir IDE kullanmak geliştirme görevini büyük ölçüde basitleştirebilir. Araç, destekleyici dizinleri ve dosyaları otomatik olarak oluşturabilir. Java Web geliştirme için en iyi desteğe sahip olduğu için NetBeans'i seçiyoruz. Bu bölüm için hala favori IDE'nizi veya herhangi bir IDE kullanmadan devam edebilirsiniz.

NOT:

### <Yan Not: neden servletler?>

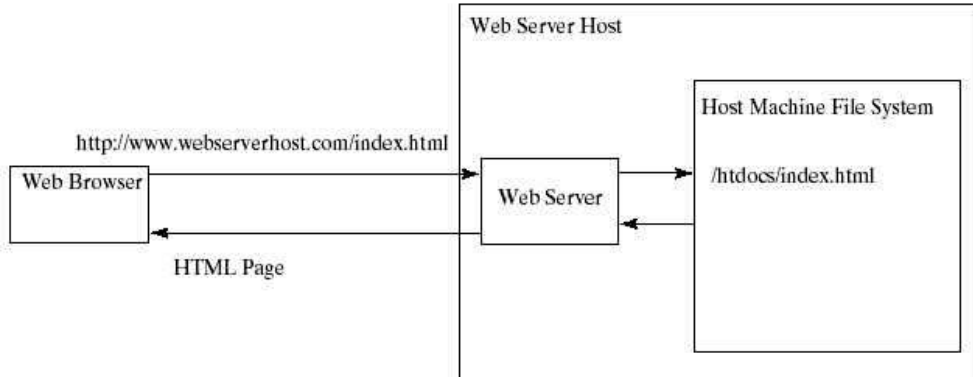
Servletler, Java Web teknolojilerinin temelini oluşturur. JSP, JSF ve Java Web hizmetleri servletlere dayanmaktadır. Servletler hakkında iyi bir anlayış, yerine getirdiğiniz Java Web teknolojisinin büyük resmini görmenize ve JSP, JSF ve Web hizmetlerini öğrenmenize yardımcı olur.

## 42.2 HTML ve Ortak Geçit Arayüzü

Java servletleri Web ortamında çalışır. Java servletlerini anlamak için, HTML ve Ortak Geçit Arayüzü'nü (CGI) gözden geçirelim.

### 42.2.1 Statik Web İçerikleri

Web sayfalarını HTML kullanarak oluşturursunuz. Web sayfalarınız, Web sunucusunda dosya olarak saklanır. Dosyalar genellikle Unix'te /htdocs dizininde saklanır, Şekil 42.1'de gösterildiği gibi. Bir kullanıcı, bir Web tarayıcısından dosyanın URL'sini yazar. Tarayıcı, Web sunucusuyla iletişime geçer ve dosyayı talep eder. Sunucu dosyayı bulur ve tarayıcıya geri gönderir. Tarayıcı, daha sonra dosyayı kullanıcıya gösterir. Bu, talep edenin kim olduğuna veya dosyanın ne zaman talep edildiğine bakılmaksızın değişmeyen statik bilgiler için iyi çalışır. Statik bilgiler dosyalarda saklanır. Dosyalardaki bilgiler güncellenebilir, ancak her bir talep için aynı belgeye yapılan her istek, kesinlikle aynı sonucu döndürür.



Şekil 42.1

*Bir Web tarayıcısı, bir Web sunucusundan statik bir HTML sayfası talep eder.*

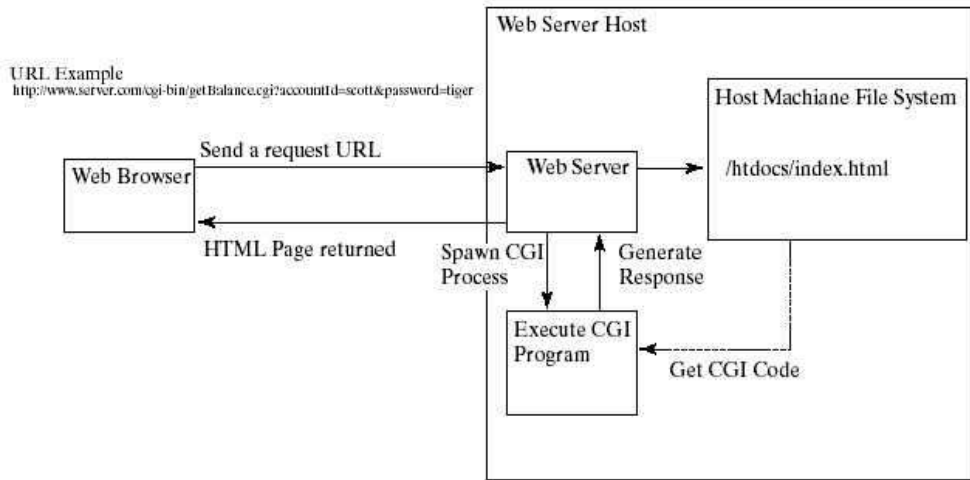
#### 42.2.2 Dinamik Web İçerikleri ve Ortak Geçit Arayüzü

Ancak, tüm bilgiler statik bir doğaya sahip değildir. Hisse senedi fiyatları, bir işlem gerçekleştirildiğinde güncellenir. Seçim günü oy sayımları sürekli olarak güncellenir. Hava durumu raporları sık sık güncellenir. Bir müşterinin banka hesabındaki bakiye, bir işlem gerçekleştirildiğinde güncellenir. Web'de güncel bilgileri görüntülemek için, bu bilgileri görüntülemek üzere HTML sayfaları dinamik olarak üretilmelidir. Dinamik Web sayfaları, Web sunucuları tarafından üretilir. Web sunucusu, kullanıcı isteklerini işlemek için belirli programları çalıştırmalıdır, böylece özelleştirilmiş bir yanıt üretebilir.

##### <Yan Not: CGI>

Ortak Geçit Arayüzü, veya CGI, dinamik Web içeriği oluşturmak için önerilmiştir. Arayüz, Web sunucularının dış programlarla etkileşimde bulunması için standart bir çerçeve sağlar, bu programlara CGI programları denir. Şekil 42.2'de gösterildiği gibi, Web sunucusu bir Web tarayıcısından bir istek alır ve bunu CGI programına iletir. CGI programı isteği işler ve çalışma zamanında bir yanıt üretir. CGI programları herhangi bir dilde yazılabilir, ancak Perl dili en popüler seçimdir. CGI programları genellikle /cgi-bin dizininde saklanır. İşte bir müşterinin banka hesabı bakiyesini görüntülemek için bir CGI programının sahte kod örneği:

1. Hesap kimliği ve parolasını alın.
2. Hesap kimliğini ve parolasını doğrulayın. Eğer başarısız olursa, yanlış hesap kimliği ve parolasını bildiren bir HTML sayfası oluşturun ve çıkın.
3. Veritabanından hesap bakiyesini alın; hesap kimliğini ve bakiyeyi görüntülemek için bir HTML sayfası oluşturun.



**Şekil 42.2**

*Bir Web tarayıcısı, bir Web sunucusundan dinamik bir HTML sayfası talep eder.*

#### 42.2.3 GET ve POST Yöntemleri

#### <Yan Not: sorgu dizesi>

En yaygın iki HTTP isteği, ayrıca *yöntemler* olarak da bilinir, GET ve POST'tur. Web tarayıcısı, bir URL veya bir HTML formu kullanarak Web sunucusunun bir CGI programını çalıştırmasını tetiklemek için bir istek gönderir. HTML formları §42.6, "HTML Formları" bölümünde tanıtılacaktır. Bir URL'den doğrudan bir CGI isteği gönderirken, GET yöntemi kullanılır. Bu URL, *sorgu dizesi* olarak bilinir. URL sorgu dizesi, CGI programının konumunu, parametreleri ve değerlerini içerir. Örneğin, aşağıdaki URL, sunucu tarafında getBalance CGI programının çağrılmasına neden olur:

HTTP://www.wu.web sunucusu ana bilgisayar.com/CGI-bin/

**getBalance.cgiaccountId=scott+smith&password=kaplan**

Program ile parametreleri ayıran sembol ? sembolüdür. Parametre adı ve değeri = sembolü kullanılarak ilişkilendirilir. Parametre çiftleri & sembolü ile ayrılır. + sembolü bir boşluk karakterini belirtir. Yani, burada accountId scott smith.

Bir HTML formundan istek gönderirken, ya GET yöntemi ya da POST yöntemi kullanılabilir. Form bunlardan birini açıkça belirtir. Eğer GET yöntemi kullanılıyorsa, formdaki veriler istek dizisine, sanki bir URL kullanılarak gönderilmiş gibi eklenir. Eğer POST yöntemi kullanılıyorsa, formdaki veriler istek dosyasının bir parçası olarak paketlenir. Sunucu programı verileri dosyayı okuyarak elde eder. POST yöntemi GET yönteminden daha güvenlidir.

NOT

#### <Yan Not: GET vs. POST>

GET ve POST yöntemleri her ikisi de Web sunucusuna istek gönderir. POST yöntemi her zaman gerekli CGI programının çalıştırılmasını tetikler. GET yöntemi, eğer önceki aynı istek Web tarayıcısında önbelleğe alınmışsa, CGI programının çalıştırılmasını sağlamayabilir. Web tarayıcıları sıklıkla Web sayfalarını önbelleğe alır, böylece aynı isteğe hızla yanıt verilebilir ve Web sunucusuna başvurulmaz. Tarayıcı, GET yöntemiyle gönderilen isteği bir URL sorgu dizesi olarak kontrol eder. Eğer kesin aynı URL için sonuçlar diskte önbelleğe alınmışsa, o zaman URL için önceki Web sayfaları görüntülenebilir. Her zaman yeni bir Web sayfasının görüntülenmesini sağlamak için POST yöntemini kullanın. Örneğin, isteğiniz veritabanını güncelleyecekse POST yöntemi kullanın. İsteğiniz zaman \duyarlı değilse, örneğin veritabanındaki bir öğrencinin adresini bulmak gibi, performansı artırmak için GET yöntemi kullanın.

#### 42.2.4 CGI'den Java Servletlerine

##### <Yan Not: CGI ve servletler>

CGI, bir kullanıcı isteğini kabul eden, sunucu tarafında işleyen ve Web tarayıcısına yanıt döndüren dinamik Web uygulamaları oluşturmak için nispeten basit bir yaklaşım sunar. Ancak CGI, aynı anda çok sayıda isteği işlerken çok yavaştır, çünkü Web sunucusu her CGI programını çalıştırmak için bir işlem başlatır. Her işlem, CGI programını içeren ve çalıştıran kendi çalışma ortamına sahiptir. Birçok CGI programının aynı anda çalıştırılmasının neye yol açacağını hayal etmek zor değildir. Sistem kaynakları hızla tükenir ve bu da sunucunun çökmesine neden olabilir.

##### <Yan Not: servlet motoru>

CGI programlarının performans sorununu gidermek için birkaç yeni yaklaşım geliştirilmiştir. Java servletleri, bu amaç için başarılı bir teknolojidir.

Java servletleri, CGI programları gibi işlev gören Java programlarıdır. Bir Web tarayıcısından gelen isteğe göre çalıştırılırlar. Tüm servletler, *servlet konteyneri* içinde çalışır, ayrıca *servlet sunucusu* veya *servlet motoru* olarak da adlandırılır. Bir servlet konteyneri, bir Java Sanal Makinesi içinde çalışan tek bir işlemdir. JVM, her servleti işlemek için bir iş parçacığı oluşturur. Java iş parçacıkları, tam teşekküllü işlemlerden çok daha az yük taşır.

Tüm iş parçacıkları, JVM'ye tahsis edilen aynı belleği paylaşır. JVM, tek bir servlet yürütme döngüsünün ötesinde kalıcı olduğundan, servletler JVM'de zaten oluşturulmuş nesneleri paylaşabilir. Örneğin, eğer birden fazla servlet aynı veritabanına erişirse, bağlan tı nesnesini paylaşabilirler. Servletler, CGI'den çok daha verimlidir.

Servletlerin, Java'da yerleşik olan başka faydaları da vardır. Java programları olarak, nesne yönelimli, taşınabilir ve platformdan bağımsızdırlar. Java bildiğiniz için, veritabanlarına ve ağ kaynaklarına erişim için Java API'sinin desteğiyle servletleri hemen geliştirebilirsiniz.

### 42.3 Servlet Oluşturma ve Çalıştırma

<sayfa açıklaması: Tomcat>

<sayfa açıklaması: GlassFish>

Java servletlerini çalıştırmak için bir servlet konteynerine ihtiyacınız vardır. Birçok servlet konteyneri ücretsiz olarak mevcuttur. İki popüler olanı Tomcat(Apache tarafından geliştirilmiştir, [www.apache.org](http://www.apache.org)) ve GlassFish (Sun tarafından geliştirilmiştir, [glassfish.dev.java.net](http://glassfish.dev.java.net)). Hem Tomcat hem de GlassFish, NetBeans 7 (Java EE sürümü) ile birlikte paketlenmiş ve entegre edilmiştir. NetBeans'ten bir servlet çalıştırdığınızda, Tomcat veya GlassFish otomatik olarak başlatılacaktır. İkisini de kullanmayı seçebilirsiniz veya başka bir uygulama sunucusu kullanabilirsiniz. GlassFish, Tomcat'tan daha fazla özelliğe sahiptir ve daha fazla sistem kaynağı alır.

#### 42.3.1 Bir Servlet Oluşturma

Servlet API'sine girişimizden önce, servletlerin nasıl çalıştığını görmek için basit bir örneğe bakalım. Bir servlet, bir ölçüde bir applet'e benzer. Her Java applet'i Applet sınıfının bir alt sınıfıdır. Applet'i uygulamak için Applet sınıfındaki uygun yöntemleri geçersiz kılmanız gerekir. Her servlet, HttpServlet sınıfının bir alt sınıfıdır. Servlet'i uygulamak için HttpServlet sınıfındaki uygun yöntemleri geçersiz kılmanız gerekir. Liste 42.1, doGet yöntemi kullanarak HTML'de bir yanıt üreten bir servlet'tir.

Liste 42.1 FirstServlet.java

<Side Remark line 11: GET işlemini işleme>

<Side Remark line 14: içerik türü>

<Side Remark line 15: tarayıcıya çıktı>

<Side Remark line 25: akışı kapat>

```
paket bölümü42;

javax.servlet.*'i içe aktarın;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet {
 /** HTTP GET yöntemini işleyin.
 * @param request servlet isteği
 * @param response servlet yanıtı
 */
 korumalı void doGet(HttpServletRequest request isteği,
```

```

 HttpServletResponse response) throws ServletException, IOException {
 response.setContentType("text/html");
 java.io.PrintWriter out = response.getWriter();
 // sayfanızı burada ıktılayın
 out.println("<html>");
 out.println("<head>");
 out.println("<title>Servlet</title>");
 out.println("</head>");
 out.println("<body>");
 out.println("Merhaba, Java Servletleri");
 out.println("</body>");
 out.println("</html>");
 out.close();
 }
 }
}

```

**<Yan Açıklama: istek>**  
**Açıklama: yanıt>**  
**ama: Yazıcı Yazarı>**

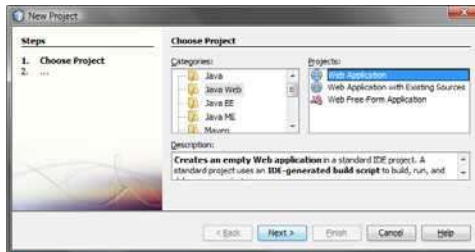
doGet metodu (satır 11), Web tarayıcısı GET metodunu kullanarak bir istek gönderdiğinde çağrılır. doGet metodunun iki parametresi vardır, request ve response. request, Web tarayıcısından veri almak için ve response, verileri tarayıcıya geri göndermek için kullanılır. Satır 14, dataların tarayıcıya text/html olarak gönderildiğini belirtir. Satır 15, verileri tarayıcıya ıkarmak için PrintWriter örneğini alır.

#### 42.3.2 NetBeans'te Servlet Oluşturma

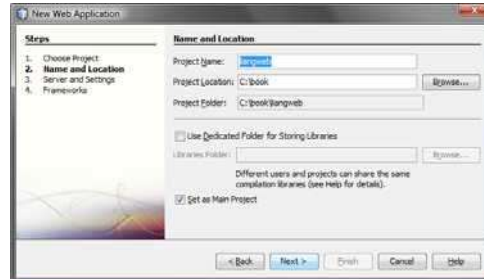
**<Side Remark: bir Web projesi oluşturun>**

NetBeans sık sık güncellenir. Bu yazının yazıldığı sırada mevcut sürüm 7.0'dır. NetBeans 7'de bir servlet oluşturmak için önce bir Web projesi oluşturmalısınız, şu şekilde:

1. Dosya > Yeni Proje'yi seçin. Yeni Proje iletişim kutusunu görüntülemek için. Kategoriler bölümünde Java Web'i ve Projeler bölümünde Web Uygulaması'nı seçin, Şekil 42.3a'da gösterildiği gibi. Yeni Web Uygulaması iletişim kutusunu görüntülemek için İleri'ye tıklayın, Şekil 42.3b'de gösterildiği gibi.
2. Proje Adı alanına liangweb yazın ve Proje Konumu alanına c:\ook yazın. Ana Proje Olarak Ayarla seçeneğini işaretleyin. Sunucu ve ayarları belirtmek için iletişim kutusunu görüntülemek üzere İleri butonuna tıklayın, bu Şekil 42.4'te gösterildiği gibi.
3. Sunucu için Apache Tomcat 7.0.11 ve J2EE Sürümü için Java EE 5'i seçin. Web projesini oluşturmak için Tamamla butonuna tıklayın, bu Şekil 42.5'te gösterildiği gibi.



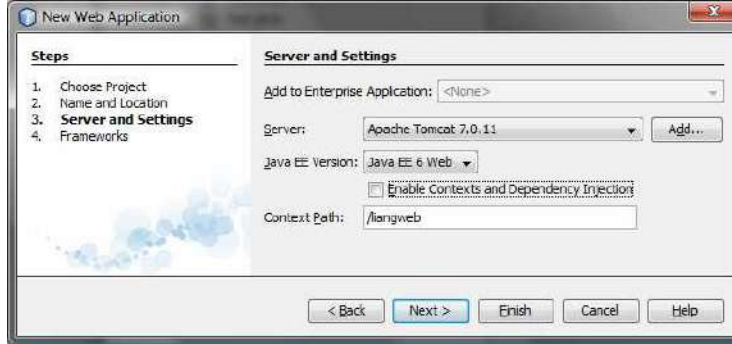
(a)



(b)

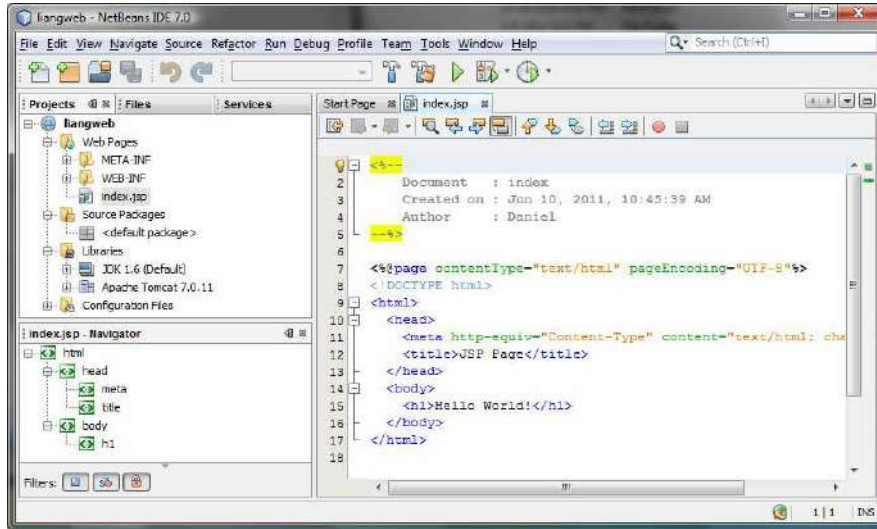
Şekil 42.3

(a) Bir Web projesi oluşturmak için Web Uygulaması'nı seçin. (b) Proje adı ve konumunu belirtin.



Şekil 42.4

Sunucuları ve ayarları seçin.



Şekil 42.5

Yeni bir Web projesi oluşturuldu.

Artık projede bir servlet oluşturabilirsiniz, aşağıdaki gibi:

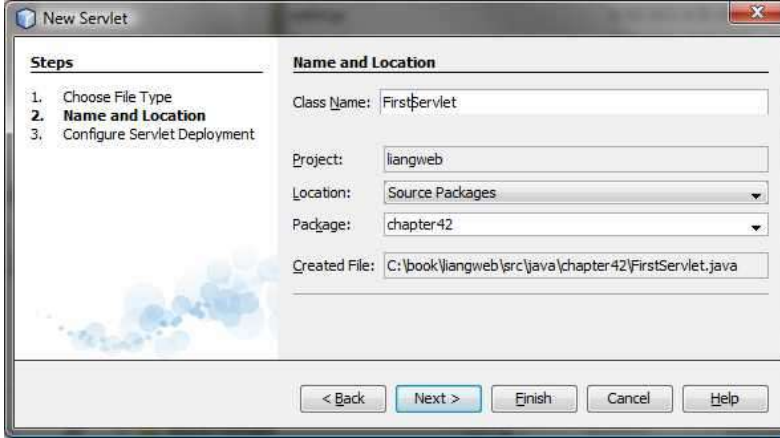
**<Yan Not: bir servlet oluşturun>**

1. Proje panelinde liangweb düğmesine sağ tıklayın ve bir bağlam menüsü görüntüleyin. Yeni > Servlet seçeneğini seçin ve Yeni Servlet iletişim kutusunu görüntüleyin, Şekil 42.6'da gösterildiği gibi.
2. Sınıf Adı alanına FirstServlet yazın ve Paket alanına chapter42 yazın ve İleri düğmesine tıklayın ve Servlet Dağıtımını Yapılandır iletişim kutusunu görüntüleyin, Şekil 42.7'de gösterildiği gibi.

3. servlet bilgilerini web.xml'e eklemek için onay kutusunu seçin ve tamamla düğmesine tıklayın, böylece servlet oluşturulmuş olur. Artık projede bir servlet şablonu oluşturulmuştur, Şekil 42.8'de gösterildiği gibi.
4. İçerik panelindeki kodu, Liste 42.1'deki kod ile servlet için değiştirin.

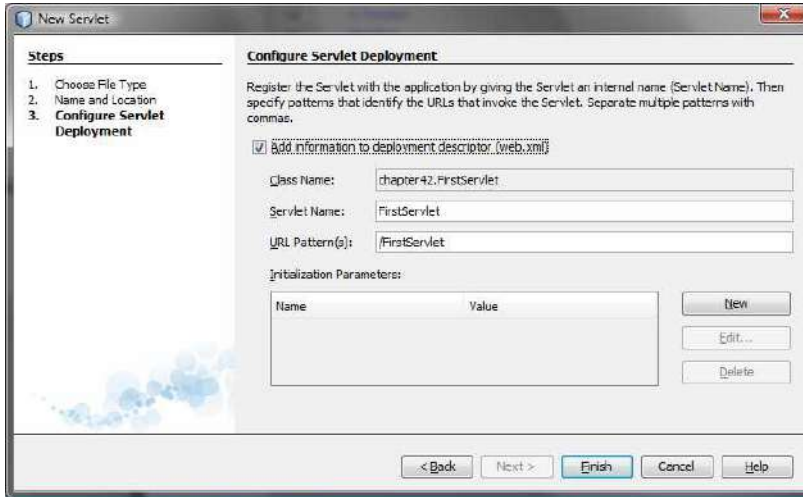
**<Yan Not: bir servlet çalıştırın>**

5. Proje panelinde liangweb düğmesine sağ tıklayın, bir bağlam menüsü görüntülemek için ve Web sunucusunu başlatmak için Çalıştır seçeneğini seçin. Web browserinde, <http://localhost:8084/liangweb/FirstServlet> URL'sini girin. Artık servlet sonucunun görüntülendiğini göreceksiniz, Şekil 42.9'da gösterildiği gibi.



Şekil 42.6

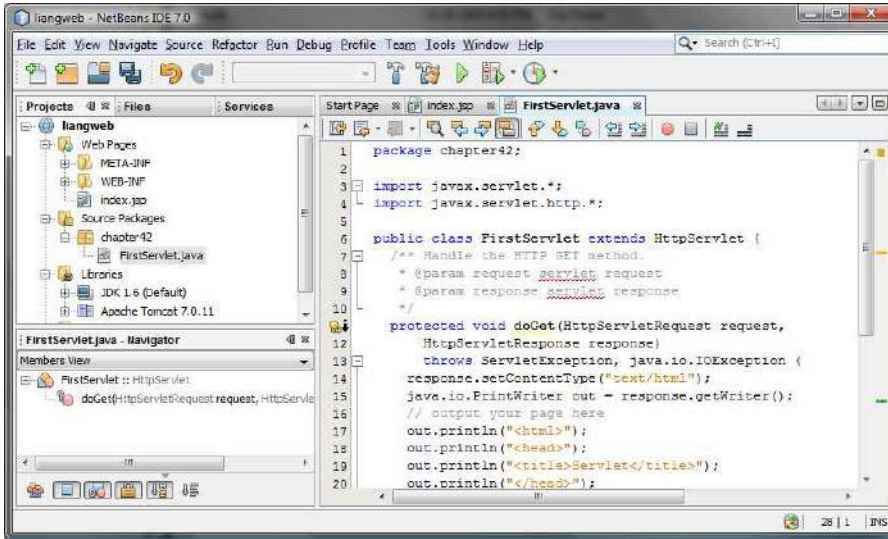
*Yeni Servlet iletişim kutusunda bir servlet oluşturabilirsiniz.*



Şekil 42.7

*web.xml'e servlet bilgilerini eklemek için onay kutusunu tıklamanız gerekir.*





Şekil 42.8

Projede yeni bir servlet sınıfı oluşturulmuştur.



Şekil 42.9

Servlet sonucu bir Web tarayıcısında görüntülenir.

NOT

**<Yan Not: IDE sorunları>**

Servlet tarayıcıda görüntülenmiyorsa, şunları yapın: 1. xml.web dosyasına servlet'i eklediğinizden emin olun. 2. Sağ tıklayın **liangweb** ve Temizle ve Oluştur'u seçin. 3. Sağ tıklayın **liangweb** ve Çalıştırı seçin. Yeniden girin <http://localhost:8084/liangweb/FirstServlet> URL'sinde. Hala çalışmıyorsa, NetBeans'ten çıkın ve yeniden başlatın.

**\*\*\*NOTUN SONU**

NOT

**<Yan Not: port numarası>**

Sunucu ayarlarına bağlı olarak, 8084 dışında bir port numaranız olabilir.

**\*\*\*NOTUN SONU**

İPUCU

**<Yan Not: Web projesini dağıt>**

**<Yan Not: WAR dosyası>**



Bir Web uygulamasını, bir Web arşiv dosyası (WAR) kullanarak bir Web uygulama sunucusuna (örneğin, Tomcat) dağıtabilirsiniz. liangweb projesi için bir WAR dosyası oluşturmak üzere, liangweb'e sağ tıklayın ve Projeyi Oluştur seçeneğini seçin. Artık liangweb.war dosyasını c:\ook\liangweb\dist klasöründe bulabilirsiniz. Tomcat üzerinde dağıtmak için, liangweb.war dosyasını webapps dizinine yerleştirmeniz yeterlidir. Tomcat başladığında, .war dosyası otomatik olarak yüklenecektir.

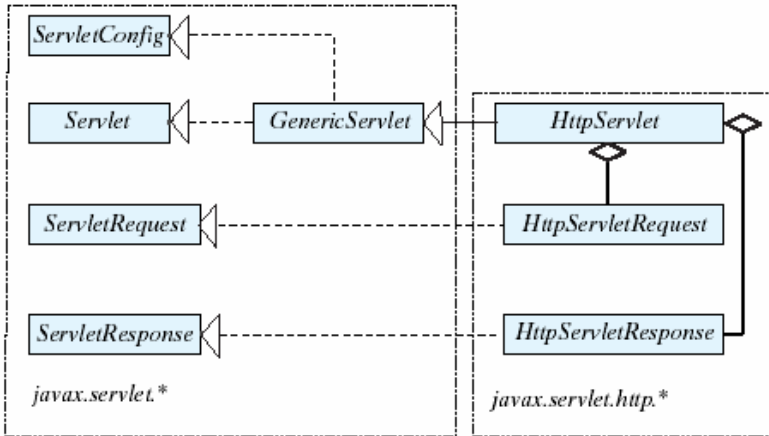
NOT:

<yan not: Tomcat Eğitimi>

NetBeans'i geliştirme aracı ve Tomcat'i dağıtım sunucusu olarak kullanmak istiyorsanız, lütfen Ek V.E, "Tomcat Eğitimi"ne bakın.

#### 42.4 Servlet API

FirstServlet.java içindeki kaynak kodunu anlamak için servlet API'sini bilmeniz gerekir. Servlet API'si, servlet'leri destekleyen arayüzleri ve sınıfları sağlar. Bu arayüzler ve sınıflar, Şekil 42.10'da gösterildiği gibi, javax.servlet ve javax.servlet.http adlı iki pakette gruplandırılmıştır. javax.servlet paketi temel arayüzleri sağlarken, javax.servlet.http paketi bunlardan türetilmiş sınıflar ve arayüzler sağlar ve HTTP isteklerini hizmet verme konusunda özel yollar sunar.



Şekil 42.10

Servlet API'si, servlet'leri geliştirmek ve çalıştırmak için kullandığınız arayüzler ve sınıflar içerir.

##### 42.4.1 Servlet Arayüzü

javax.servlet.Servlet arayüzü, tüm servlet'lerin uygulaması gereken yöntemleri tanımlar. Yöntemler aşağıda listelenmiştir:

```
/** Her oluşturulan servlet için çağrılır */
public void init() throws ServletException;

/** Gelen isteklere yanıt vermek için çağrılır */
```

**kamu boşluğu** hizmet(ServletRequest isteği, ServletResponse nresponse)

**atar** ServletException, IOException;

*/\*\* Servlet tarafından kaynakları serbest bırakmak için çağrılır \*/*  
**genel void** yoket();

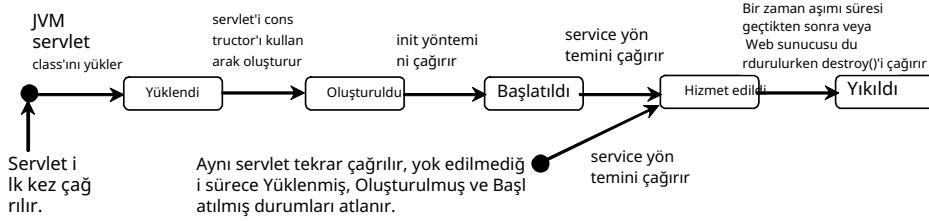
#### <Yan Not: servlet yaşam döngüsü>

init, service ve destroy yöntemleri, *yaşam döngüsü yöntemleri* olarak bilinir ve aşağıdaki sırayla çağrılır (bkz. Şekil 42.11):

1. **init** yöntemi, servlet ilk oluşturulduğunda çağrılır ve servlet yok edilmediği sürece bir daha çağrılmaz. Bu, applet oluşturulduktan sonra çağrılan ve applet yok edilmediği sürece bir daha çağrılmayan applet'in init yöntemine benzer.

2. **service** yöntemi, sunucu her seferinde servlet için bir istek aldığı anda çağrılır. Sunucu yeni bir iş parçacığı oluşturur ve **service**'i çağırır.

3. **destroy** yöntemi, bir zaman aşımı süresi geçtikten sonra veya Web sunucusu sonlandırıldığında çağrılır. Bu yöntem, servlet için kaynakları serbest bırakır.



Şekil 42.11

JVM, servlet'i kontrol etmek için init, service ve destroy yöntemlerini kullanır.

#### 42.4.2 GenericServlet Sınıfı, ServletConfig Arayüzü ve HttpServlet Sınıfı

javax.servlet.GenericServlet sınıfı, genel, protokolden bağımsız bir servlet tanımlar. javax.servlet.Servlet ve javax.servlet.ServletConfig'i uygular. ServletConfig, bir Web sunucusundan başlatma sırasında bilgi almak için dört yöntem (getInitParameter, getInitParameterNames, getServletContext ve getServletName) tanımlayan bir arayüzdür. Servlet ve

ServletConfig'teki tüm yöntemler GenericServlet'te uygulanmıştır, service hariç. Bu nedenle, GenericServlet soyut bir sınıftır.

javax.servlet.http.HttpServlet sınıfı, HTTP protokolü için bir servlet tanımlar. GenericServlet sınıfını genişletir ve service yöntemini uygular. Hizmet yöntemi, HTTP isteklerinin bir dağıtıcısı olarak uygulanmıştır. HTTP istekleri aşağıdaki yöntemlerde işlenir:

- **doGet** GET isteğine yanıt vermek için çağrılır.
- **doPost** POST isteğine yanıt vermek için çağrılır.
- **doDelete** DELETE isteğine yanıt vermek için çağrılır. Böyle bir istek genellikle sunucudaki bir dosyayı silmek için kullanılır.

- **doPut** PUT isteğine yanıt vermek için çağrılır. Böyle bir istek genellikle sunucuya bir dosya göndermek için kullanılır.
- **doOptions** OPTIONS isteğine yanıt vermek için çağrılır. Bu, sunucu hakkın da, desteklediği HTTP yöntemleri gibi bilgiler döndürür.
- **doTrace** TRACE isteğine yanıt vermek için çağrılır. Böyle bir istek genellikle hata ayıklama için kullanılır. Bu yöntem, uygun izleme bilgilerini içeren bir HTML sayfa sı döndürür.

Tüm bu yöntemler aşağıdaki imzayı kullanır:

**korunmalı geçersiz** dotXxt(HttpServletRequest isteği, HttpServletResponse nres p) **atar** ServletException, java.io.IOException

HttpServlet sınıfı bu yöntemler için varsayılan uygulamayı sağlar. GET, POST, DELETE veya PUT isteğini işlemek istiyorsanız doGet, doPost, doDelete ve doPut yöntemlerini geçersiz kılmanız gerekir. Varsayılan olarak, nothing yapılmayacaktır. Normalde, servlet HTTP 1.1 tarafından uygulananlardan daha fazla yeni HTTP yöntemleri uygulamıyorsa doOptions yöntemini geçersiz kilmamalısınız. doTrace yöntemi ni geçersiz kılmaya da gerek yoktur.

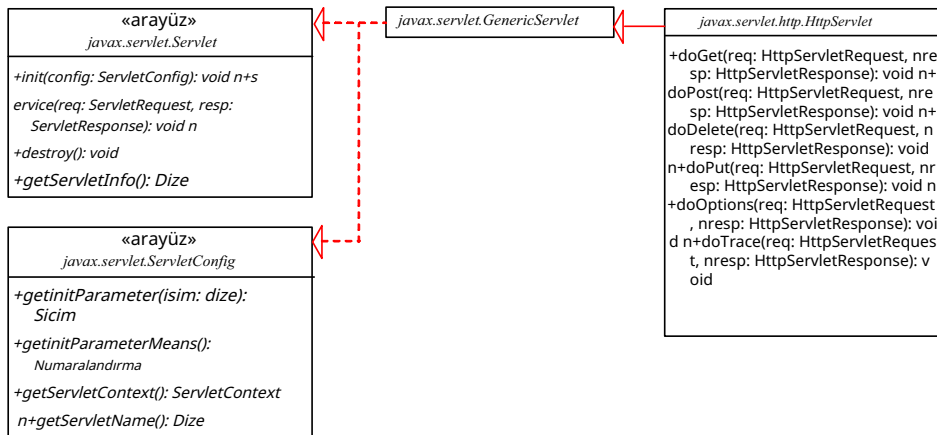
NOT: GET ve POST istekleri genellikle kullanılırken, DELETE, PUT, OPTIONS ve TRACE kullanılmamaktadır. Bu istekler hakkında daha fazla bilgi için lütfen HTTP 1.1 spesifikasyonuna bakın: [www.cis.ohio-state.edu/htbin/rfc/rfc2068.html](http://www.cis.ohio-state.edu/htbin/rfc/rfc2068.html).

NOT: HttpServlet içindeki yöntemler tümüyle soyut olmayan yöntemlerdir, ancak HttpServlet soyut bir sınıf olarak tanımlanmıştır. Böylece HttpServlet'den doğrudan bir servlet oluşturma zsiniz. Bunun yerine, HttpServlet'i genişleterek servlet'inizi tanımlama nız gerekir.

Bu arayüzlerin ve sınıfların ilişkisi Şekil

42.12'de gösterilmektedir.

<PD: UML Sınıf Diyagramı>



Şekil 42.12

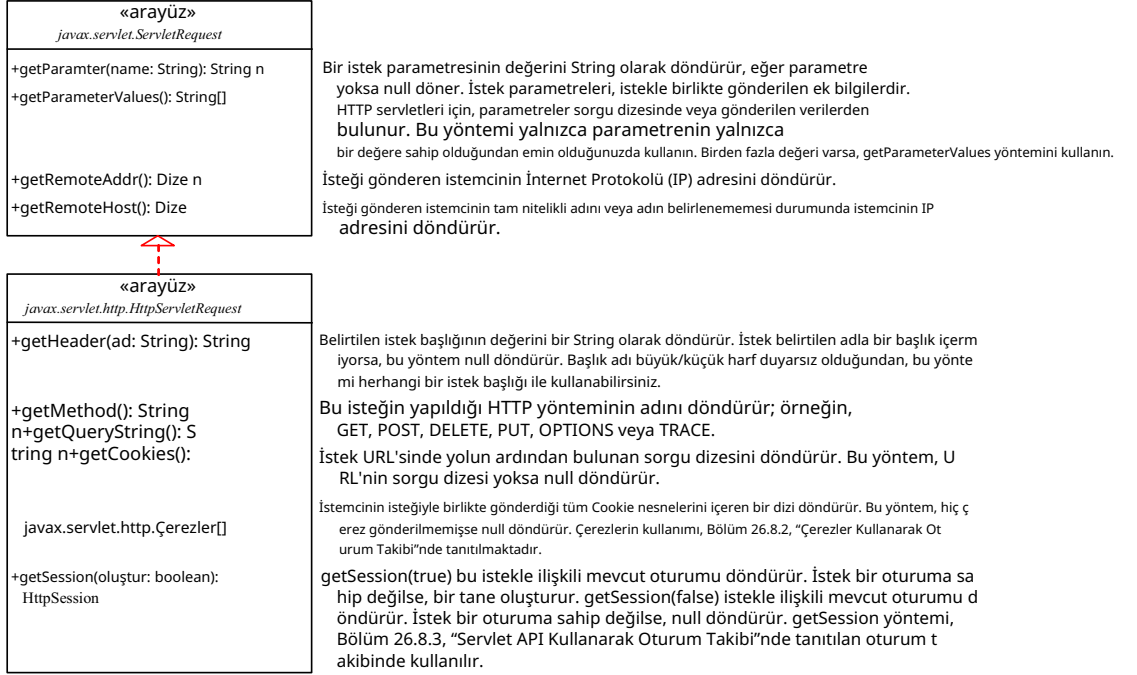
HttpServlet, Servlet ve ServletConfig arayüzlerini uygulayan soyut sınıf GenericServlet'ten miras alır.

#### 42.4.3 ServletRequest Arayüzü ve HttpServletRequest Arayüzü

her doXxx yöntemi, HTTP istek bilgilerini içeren bir nesne olan HttpServletRequest türünde bir parametreye sahiptir. Bu bilgiler, parametre adı ve değerleri, öznitelikler ve bir giriş akışını içerir. HttpServletRequest, ServletRequest'in bir alt arayüzüdür.

ServletRequest, her türlü istemci için bilgi sağlamak üzere daha genel bir arayüz tanımlar. Bu iki arayüzde sıkça kullanılan yöntemler Şekil 42.13'te gösterilmiştir.

##### <PD: UML Sınıf Diyagramı>



Şekil 42.13

HttpServletRequest, ServletRequest'in bir alt arayüzüdür.

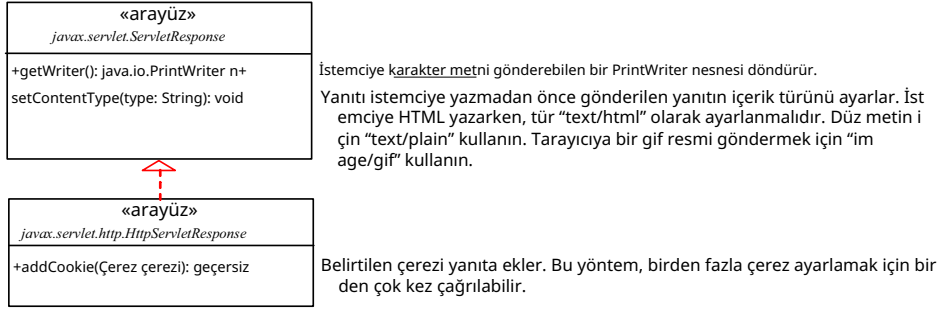
#### 42.4.4 ServletResponse Arayüzü ve HttpServletResponse Arayüzü

ndaki her doXxx yöntemi, istemciye yanıt göndermeye yardımcı olan HttpServletResponse türünde bir parametreye sahiptir. HttpServletResponse, ServletResponse'in bir alt arayüzüdür. ServletResponse, istemciye çıktı göndermek için daha genel bir arayüz tanımlar.

Bu iki arayüzde sıkça kullanılan yöntemler Şekil

42.14'te gösterilmiştir.

##### <PD: UML Sınıf Diyagramı>



Şekil 42.14

*HttpServletResponse, ServletResponse'un bir alt arayüzüdür.*

## 42.5 Servlet Oluşturma

Servletler, Java appletlerinin zıttıdır. Java appletleri, istemci tarafında bir Web tarayıcısından çalışır. Java programları yazmak için sınıflar tanımlarsınız. Bir Java applet yazmak için, Applet sınıfını genişleten bir sınıf tanımlarsınız. Web tarayıcısı, Applet sınıfında tanımlanan yöntemler aracılığıyla appletin çalıştırılmasını kontrol eder. Benzer şekilde, bir Java servleti yazmak için, HttpServlet sınıfını genişleten bir sınıf tanımlarsınız. Servlet konteyneri, HttpServlet sınıfında tanımlanan yöntemler aracılığıyla servletin çalıştırılmasını kontrol eder. Bir Java applet gibi, bir servletin de main yöntemi yoktur. Bir servlet, yöntemleri çağırarak için servlet motoruna bağımlıdır. Her servlet aşağıda gösterilen yapıya sahiptir:

### paket bölüm42;

```
import javax.servlet.*;
içerik aktarmak javax.servlet.http.*;
içerik aktarmak java.io.*;

genel sınıf MyServlet'im uzantıları HttpServlet {
 /** Servlet'i başlatmak için servlet motoru tarafından çağırılır */
 genel boş init() atar ServletException {
 ...
 }

 /** HTTP Get isteğini işleme */
 genel boş doGet(HttpServletRequest istek,
HttpServletResponse
yanıt) atar ServletException, IOException {
 ...
 }

 /** HTTP Post isteğini işleme */
 public void doPost(HttpServletRequest istek, HttpServletResponse
yanıt) atar ServletException, IOException {
 ...
 }

 /** Kaynağı serbest bırakmak için servlet motoru tarafından çağırılır */
```

```
kamu boşluğu yok et() {n ...
```

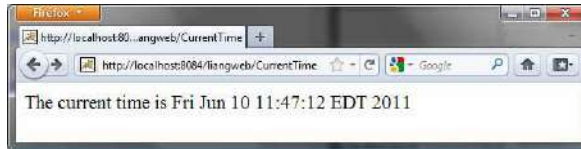
```
}
```

```
// Gerekirse diğer yöntemler
```

```
}
```

Servlet motoru, servletleri init, doGet, doPost, destroy ve diğer yöntemler kullanarak kontrol eder. Varsayılan olarak, doGet ve doPost yöntemleri bir şey yapmaz. GET isteğini işlemek için doGet yöntemini geçersiz kılmanız gerekir; POST isteğini işlemek için doPost yöntemini geçersiz kılmanız gerekir.

Liste 42.2, mevcut zamanı görüntülemek için dinamik bir Web sayfası oluşturan basit bir Java servletini vermektedir; Şekil 42.15'te gösterildiği gibi.



**Şekil 42.15**

*Servlet CurrentTime mevcut zamanı görüntüler.*

#### **Liste 42.2 CurrentTime.java**

*<Yan Notu satır 9: GET işlemi>*

*<Yan Notu satır 11: içerik türü>*

*<Yan Notu satır 12: tarayıcıya çıktı>*

*<Yan Notu satır 14: akışı kapat>*

```
paket chapter42;
```

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class CurrentTime extends HttpServlet {
 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
 // oluşturur
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();
 out.println("<p>Mevcut zaman " + new java.util.Date());
 out.close(); // Akışı kapat
 }
}
```

HttpServlet sınıfının bir doGet metodu vardır. doGet metodu, tarayıcı bir GET metodu kullılarak servlet'e bir istek gönderdiğinde çağrılır.

Servlet sınıfınız, GET isteğine yanıt vermek için doGet metodunu geçersiz kılmalıdır. Bu durumda, mevcut zamanı görüntülemek için kodu yazarsınız.

Servletler, HttpServletResponse nesnesi aracılığıyla tarayıcıya yanıt döner. setContentType("text/html") metodu MIME türünü "text/html" olarak ayarladığından, tarayıcı yanıtı HTML olarak görüntüler. getWriter metodu, HTML'yi istemciye geri göndermek için bir PrintWriter nesnesi döner.

NOT: URL sorgu dizesi, servlet'e bir istek göndermek için GET metodunu kullanır. Mevcut zaman, mevcut zamanı görüntüleyen Web sayfası önbelleğe alındıysa güncel olmayabilir. Yeni bir mevcut zamanı görüntülenmesini sağlamak için, tarayıcıda sayfayı yenileyin. Bir sonraki örnekte, mevcut zamanı almak için POST metodunu kullanan yeni bir servlet yazacaksınız.

## 42.6 HTML Formları

HTML formları, verileri Web sunucusuna uygun bir biçimde göndermenizi sağlar. Şekil 42.16'da gösterildiği gibi, form metin alanları, metin alanı, onay kutuları, açılır kutular, listeler, radyo düğmeleri ve düğmeler içerebilir.

Şekil 42.16

*Bir HTML formu metin alanları, radyo düğmeleri, açılır kutular, liste, onay kutuları, metin alanları ve düğmeler içerebilir.*

### <Yan Not: HTML/XHTML Eğitimi>

Şekil 42.16'da form oluşturmak için kullanılan HTML kodu Listeleme 42.3'te verilmiştir. (HTML ile tanışık değilseniz, lütfen Ek V.A, "HTML ve XHTML Eğitimi"ne bakın.)

### Listeleme 42.3 StudentRegistrationForm.html

<Yan Not satır 9: form etiketi>

<Yan Not satır 12: etiket>

<Yan Not satır 13: metin alanı>

<Yan Not satır 21: radyo düğmesi>

<Yan Not satır 26: açılır kutu>

<Yan Not satır 35: liste>

<Yan Not satır 44: onay kutusu>

<Yan Not satır 51: metin alanı>

<Yan Not satır 54: gönder düğmesi>

<Yan Not satır 55: sıfırla düğmesi>

<!--Bir HTML Formu Demo-->

```

<html>
<kafa>
<title>Öğrenci Kayıt Formu</title>
</head>
<gövde>
<h3>Öğrenci Kayıt Formu</h3>

<form action = t "GetParameterst"ny
öntemi = t "al"t>
<!-- İsim metin alanları -->
<p><label>Soyadı</label>
<input type = "text" name = t "soyadi"t boyut = t "20"t />
<label>Ad</label>
<input type = "text" name = t "ad"t boyut = "20"t />n
<label>tMIt</label>
<input type = "metin" name = "mi" boyut = "1" /></p>

<!-- Cinsiyet radyo düğmeleri -->
<p><label>Cinsiyet:</label>
<giriş türü = r "radyo"t adı = t "cinsiyet"t değeri = "M"checked />
Erkek
<giriş türü = t "radyo"nun adı = t "cinsiyet"t değeri = "F" /> Kadın</p>

<!-- Ana dal açılır kutusu -->
<p><label>Ana dal</label>
<ad seç = "bölüm" boyut = "1"t>
n<seçenek değeri = "CS">Bilgisayar Bilimleri</option>
<option value = "Math">Matematik</option>
<option>İngilizce</option>
<option>Çince</option>
</select>

<!-- Küçük bölüm listesi -->
<label>Küçük bölüm</label>
<select name = "minor" boyut = "2" çoklu>
<option>Bilgisayar Bilimleri</option>
<option>Matematik</option>
<option>İngilizce</option>
<option>Çince</option>
</select></p>

<!-- Hobiler kontrol kutuları -->
<p><label>Hobi:</label>
<input type = "checkbox" name = "tenis"/> Tenis
<giriş türü = "onay kutusu" adı = t "golf"t />t Golfn<
giriş türü = "onay kutusu" adı = "masa tenisi" checked/>Masa Tenisi
</p>

<!-- Notlar metin alanı -->
<p>Notlar:</p>
<p><textarea adı = "notlar" satırlar = "3" sütunlar = "56"t>
n</textarea></p>

<!-- Gönder ve Sıfırla butonları -->
<p><input type = "submit" value = "Gönder"/>

```



```
<giriş türü = "sıfırla" value = "Sıfırla"/></p>
</form>
</body>
</html>
```

Aşağıdaki HTML etiketleri HTML formlarını oluşturmak için kullanılır:

<Yan Not: <form>>

<Yan Not: action>

<Yan Not: method>

- <form> ... </form> bir form gövdesini tanımlar. <form> etiketinin özellikleri action ve method. Action özelliği form gönderildiğinde Web sunucusunda çalıştırılacak sunucu programını belirtir. method niteliği ya get ya da post'dur.

<Yan Açıklama: <etiket>>

- <label> ... </label> basitçe bir etiket tanımlar.

<Yan Açıklama: <giriş>>

- <input> bir giriş alanını tanımlar. Bu etiketin nitelikleri tip, ad, değer, kontrol edildi, boyut ve maksimum uzunluk'dur. Tip nitelik, giriş türünü belirtir. Olası türler metin için bir satırlık metin alanı, radio için bir radio düğmesi ve onay kutusu bir onay kutusu içindir. Ad niteliği, özelliği için resmi bir ad verir. Bu ad niteliği, servlet programı tarafından ilişkili değerini almak için kullanılır. Bir gruptaki radio düğmelerinin adları aynı olmalıdır. Değer niteliği bir metin alanı ve metin alanı için varsayılan bir değeri belirtir. Kontrol edildi niteliği, bir radio düğmesinin veya bir onay kutusunun başlangıçta kontrol edilip edilmediğini gösterir. Boyut niteliği, bir metin alanının boyutunu belirtir ve maksimum uzunluk niteliği, bir metin alanının maksimum uzunluğunu belirtir.

<Yan Açıklama: <seç>>

- <select> ... </select> bir kombinasyon kutusu veya bir liste tanımlar. Bu etiketin nitelikleri ad, boyut ve çoklu'dur. Boyut niteliği, listede görünen satır sayısını belirtir. Çoklu niteliği, birden fazla değer için bir listeden seçildi. Boyutunu 1 olarak ayarlayın ve bir kombinasyon kutusu için çoklu kullanmayın.

<Yan Not: <option>>

- <option>... </option> bir <select> ... </select> etiketi içinde bir seçim listesini tanımlar. Bu etiket, seçilen seçenek için bir değer belirtmek üzere value öz niteliği ile kullanılabilir (örneğin, <option value = "CS">Bilgisayar Bilimleri). Hiçbir değer belirtilmezse, seçilen seçenek değeridir.

<Yan Not: <textarea>>

- <textarea> ... </textarea> bir metin alanını tanımlar. Öz nitelikler şunlardır: ad, satırlar ve sütunlar. Satır ve sütun öz nitelikleri, bir metin alanındaki satır ve sütun sayısını belirtir.

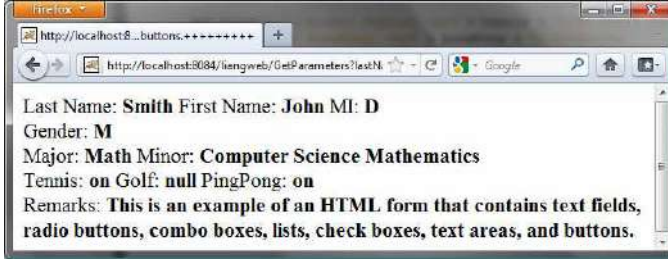
NOT:

<Yan Not: bir HTML dosyası oluşturun>

HTML dosyasını NetBeans'ten oluşturabilirsiniz. Liangweb'e sağ tıklayın ve Yeni, *HTML* seçeneğini seçin, Yeni HTML Dosyası iletişim kutusunu görüntülemek için. Dosya adı olarak *StudentRegistrationForm* yazın ve dosyayı oluşturmak için *Bitir* 'e tıklayın.

#### 42.6.1 HTML Formlarından Parametre Değerlerini Alma

HTML formundan parametre değerlerini nasıl alacağınızı göstermek için, Listeleme 42.4, Şekil 42.16'daki önceki öğrenci kayıt formundan tüm parametre değerlerini almak için bir servlet oluşturun ve bunların değerlerini gösterir, Şekil 42.17'de gösterildiği gibi.



Şekil 42.17

Servlet, Şekil 42.16'da girilen parametre değerlerini görüntüler.

#### Listeleme 42.4 GetParameters.java

<Yan Not satır 9: GET işlemi>  
<Yan Not satır 11: içerik türü>  
<Yan Not satır 12: tarayıcıya çıktı>  
<Yan Not satır 15: parametreleri al>  
<Yan Not satır 20: birden fazla değer>  
<Yan Not satır 38: akışı kapat>

paket bölüm42;

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class GetParameters extends HttpServlet {
 /** HTTP Post isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
 // İstemciden parametreleri alın
 String lastName = request.getParameter("lastName");
 String firstName = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 String cinsiyet = request.getParameter("cinsiyet");
 String major = request.getParameter("major");
 String[] minors = request.getParameterValues("minor");
 String tennis = request.getParameter("tennis");
 String golf = request.getParameter("golf");
 String pingPong = request.getParameter("pingPong");
```

```

String remarks = request.getParameter("remarks");

out.println("Soyadı: " + lastName + " Ad: "
+ firstName + " MI: " + mi + "
");
out.println("Cinsiyet: " + gender + "
");
out.println("Bölüm: " + major + " Yan Bölüm: ");

if (küçükler != null)
for (int i = 0; i < küçükler.uzunluk; i++)n out.printl
n(küçükler[i] + " ");

out.println("
 Tenis: " + tennis + " Golf: " +
golf + " Masa Tenisi: " + pingPong + "
");
out.println("Notlar: " + remarks + "");
out.close(); // Akışı kapat
}
}

```

HTML formu zaten StudentRegistrationForm.html dosyasında oluşturulmuş ve Şekil 42.16'da gösterilmiştir. Formun eylemi

GetParameters , Gönder butonuna tıkladığınızda GetParameters servlet'ini çağırır.

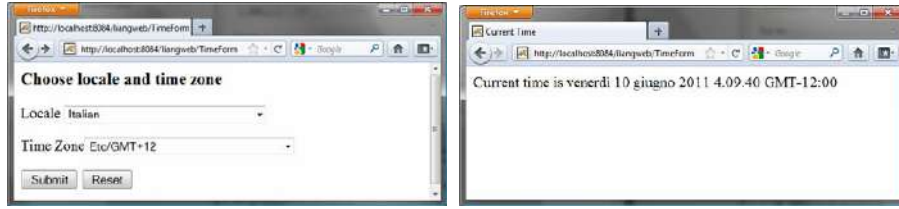
Formdaki her GUI bileşenin bir ad niteliği vardır. Servlet, getParameter(attributeName) yönteminde ad niteliğini kullanarak parametre değerini bir dize olarak alır. Birden fazla değere sahip bir liste durumunda, getParameterValues(attributeName) yöntemini kullanarak parametre değerlerini bir dize dizisi olarak döndürür (satır 20).

HTML formunda bir metin alanı, metin alanı, kombinasyon kutusu, liste, onay kutusu v eya radyo düğmesi için isteğe bağlı olarak değer niteliğini belirtebilirsiniz. Metin alanı ve metin alanı için değer niteliği, metin alanında ve metin alanında görüntülenecek varsayıla n bir değeri belirtir. Kullanıcı, bunu değiştirmek için yeni değerler yazabilir. Kombinasyo n kutusu, liste, onay kutusu ve radyo düğmesi için değer niteliği, getParameter ve getParameterValues yöntemlerinden döndürülecek parametre değerini belirtir. Eğer bir ko mbinasyon kutusu veya liste için değer niteliği belirtilmemişse, seçilen dizeyi kombi nasyon kutusundan veya listeden döndürür. Eğer bir radyo düğmesi veya onay kutusu için değer niteliği belirtilmemişse, işaretli bir radyo düğmesi veya işaretli bir onay kutus u için dize on döndürür ve işaretlenmemiş bir onay kutusu için null döndürür.

NOT: Bir özellik mevcut değilse, getParameter(attributeName) yöntemi null döner. Eğer parametrenin boş bir değeri servlete geçirilirse, getParameter(attributeName) yöntemi boş bir değerle dönen bir dize döner. Bu durumda, dizenin uzunluğu 0'dır.

#### 42.6.2 Yerel ve Zaman Dilimine Dayalı Olarak Mevcut Zamanı Alma

Bu örnek, GET ve POST isteklerini işleyen bir servlet oluşturur. GET isteği, yerel ayar için bir açılır kutu ve zaman dilimi için bir açılır kutu içeren bir for m oluşturur; bu, Şekil 42.18a'da gösterilmiştir. Kullanıcı, bu formdan bir yerel ayar ve bir zaman dilimi seçerek, yerel ayar ve zaman dilimine dayalı olarak mevcut zamanı elde etm ek için bir POST isteği gönderebilir; bu, Şekil 42.18b'de gösterilmiştir.



(a)

(b)

**Şekil 42.18**

*TimeForm servletindeki GET yöntemi (a) içinde bir form görüntüler ve TimeForm servletindeki POST yöntemi (b) içinde yerel ayar ve saat dilimine göre zamanı görüntüler.*

Liste 42.5 servlet'i vermektedir.

**Liste 42.5 TimeForm.java**

<Yan Notu 15. satır: GET işlemi>  
 <Yan Notu 17. satır: içerik türü>  
 <Yan Notu 18. satır: tarayıcıya çıktı>  
 <Yan Notu 20. satır: form oluştur>  
 <Yan Notu satır 42: akışı kapat>  
 <Yan Notu satır 46: POST işle>  
 <Yan Notu satır 48: içerik türü>  
 <Yan Notu satır 49: tarayıcıya çıktı>  
 <Yan Notu satır 51: yerel ayar al>  
 <Yan Notu satır 53: zaman dilimini al>  
 <Yan Notu satır 56: takvim oluştur>  
 <Yan Notu satır 65: akışı kapat>

paket bölüm42;

import javax.servlet.\*;  
import javax.servlet.http.\*;  
import java.io.\*;  
java.util.\*'i içe aktarın;  
java.text.\*'i içe aktarın;

genel sınıf TimeForm, HttpServlet'i genişletir {n özel st  
atik final String CONTENT\_TYPE = "text/html";n özel Locale[] allLocale  
= Locale.getAvailableLocales();n özel String[] allTimeZone = TimeZone.get  
AvailableIDs();

/\*\* HTTP Get isteğini işleyin \*/

public void doGet(HttpServletRequest request, HttpServletResponse  
response) throws ServletException, IOException {  
response.setContentType(CONTENT\_TYPE);  
PrintWriter out = response.getWriter();  
out.println("<h3>Yerel ayar ve zaman dilimi seçin</h3>");  
out.println("<form method=\"post\" action=\" +  
"ZamanFormu">");  
out.println("Yerel ayar <select size=\"1\" name=\"locale\">");

// Tüm yerel ayarları doldurun  
for (int i = 0; i < allLocale.length; i++) {n out.println("<op  
tion value=\"\" + i + \"\">\" +

```

 allLocale[i].getDisplayName() + "</option>");n }

 out.println("</select>");

 // Tüm saat dilimlerini doldurun
 out.println("<p>Saat Dilimi<select size='1' name='timezone'>");
 for (int i = 0; i < allTimeZone.length; i++) {
 out.println("<option value='\" + allTimeZone[i] + \"'>\" + allTimeZone[i] +
 "</option>");
 }
 out.println("</select>");

 out.println("<p><input type='submit' value='Gönder'>");
 out.println("<input type='reset' value='Sıfırla'></p>");
 out.println("</form>");
 out.close(); // Akışı kapat

 /** HTTP Post isteğini işleme */
 public void doPost(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType(CONTENT_TYPE);n Pri
 ntWriter out = Response.getWriter();n out.println(
 "<html>");
 int localeIndex = Integer.parseInt(
 request.getParameter("locale"));
 String timeZoneID = request.getParameter("timezone");
 out.println("<head><title>Geçerli Zaman</title></head>");
 out.println("<body>");
 takvim takvim =
 new GregorianCalendar(allLocale[localeIndex]);n TimeZon
 e timeZone = TimeZone.getTimeZone(timeZoneID);n DateFormat d
 ateFormat = DateFormat.getDateInstance(n DateFormat.FULL, D
 ateFormat.FULL, allLocale[localeIndex]);n dateFormat.setTimeZone(timeZ
 one);
 out.println("Geçerli zaman " +
 dateFormat.format(calendar.getTime()) + "</p>");
 out.println("</body></html>");
 out.close(); // Akışı kapat
 }
}

```

Bu servleti çalıştırdığınızda, servlet TimeForm'un doGet metodu dinamik olarak zaman formunu oluşturmak için çağrılır. Formun metodu POST'tur ve eylem aynı servleti, TimeForm'u çağırır. Form sunucuya gönderildiğinde, doPost metodu isteği işlemek için çağrılır.

allLocale ve allTimeZone değişkenleri (satırlar 11-12), sırasıyla, tüm mevcut yerel ayarları ve saat dilimi kimliklerini tutar. Yerel ayarların adları yerel ayar listesinde görüntülenir. Yerel ayarların değerleri, allLocale dizisindeki yerel ayarların indeksleridir. Saat dilimi kimlikleri dizelerdir. Saat dilimi listesinde görüntülenirler. Ayrıca, listenin değerleridir. Yerel ayar ve saat diliminin indeksleri servlete parametreler olarak iletilir. doPost metodu, parametrelerin değerlerini alır (satırlar 51-53) ve yerel ayar ve saat dilimine göre mevcut zamanı bulur.

NOT

**<Yan Not: karakter kodlamasını ayarlayın>**

Bir Asya yerel ayarı (örneğin, Çince, Korece veya Japonca) seçerseniz, zaman doğru bir şekilde görüntülenmeyecektir, çünkü varsayılan karakter kodlaması UTF-8'dir. Bu sorunu gidermek için, uluslararası bir karakter kodlaması ayarlamak için 48. satıra aşağıdaki ifadeyi ekleyin:

```
response.setCharacterEncoding("GB18030");
```

Kodlama hakkında bilgi için bkz. §31.6, "Karakter Kodlaması."

### \*\*\*NOTUN SONU

## 42.7 Servletlerde Veritabanı Programlama

Pek çok dinamik Web uygulaması verileri depolamak ve yönetmek için veritabanları kullanır. Servletler, herhangi bir ilişkisel veritabanına JDBC aracılığıyla bağlanabilir. 37. Bölümde, "Java Veritabanı Programlama," JDBC aracılığıyla ilişkisel verilere erişmek ve bunları düzenlemek için Java programları oluşturmayı öğrendiniz. Bir servleti bir veritabanına bağlamak, bir Java uygulamasını veya appleti bir veritabanına bağlamaktan farklıdır. Java servletleri ve JDBC'yi biliyorsanız, bunları birleştirerek ilginç ve pratik Web tabanlı etkileşimli projeler geliştirebilirsiniz.

Bir servletten bir veritabanına bağlanmayı göstermek için, bir kayıt formunu işleyen bir servlet oluşturalım. Müşteri, bir HTML formunda veri girer ve formu sunucuya gönderir; bu, Şekil 42.19'da gösterilmiştir. Gönderimin sonucu Şekil 42.20'de gösterilmektedir. Sunucu, formdan verileri toplar ve bunları bir veritabanında saklar.

Firefox - Simple Registration without Confirmation

http://localhost:8084/longweb/SimpleRegistration.html

Please register to your instructor's student address book.

Last Name \* Smith First Name \* John MI M

Telephone 3222231212 Email smith@acm.org

Street 100 Main Street

City Savannah State Georgia-GA Zip 31419

Submit Reset

\* required fields

Şekil 42.19

HTML formu, kullanıcının öğrenci bilgilerini girmesine olanak tanır.

Firefox - Simple Registration

http://localhost:8084/longweb/SimpleRegistration

John Smith is now registered in the database

Şekil 42.20

Servlet, formu işler ve verileri bir veritabanında saklar.

Kayıt verileri, aşağıdaki alanlardan oluşan bir Adres tablosunda saklanır: firstName, mi, lastName, street, city, state, zip, telephone ve email, aşağıdaki ifadede tanımlanmıştır:

```
tablet Adresi oluşturun (n ad t
varchar(t25t),n mi t grafiği(
t1t),
soyadı t varchar(t25t),n sok
ak t varchar(t40t),n şehir
t varchar(t20t),n eyal
et t varchar(t2t),n zip
t varchar(t5t),n tele
fon t varchar(t10t),n e-posta
t varchar(t30t)
)
```

<Yan Açıklama: mysqljdbc.jar> n<Y  
an Açıklama: ojdbc6.jar>

MySQL, Oracle ve Access Bölüm 37'de kullanıldı. Herhangi bir ilişkisel veritabanını kullanabilirsiniz. Servlet, JDBC-ODBC sürücüsünden başka bir database sürücüsü kullanıyorsa (örneğin, MySQL JDBC sürücüsü ve Oracle JDBC sürücüsü), JDBC sürücüsünü (örneğin, MySQL için mysqljdbc.jar ve Oracle için ojdbc6.jar) projedeki Kütüphaneler düğümüne eklemeniz gerekir.

<Yan Not: .html dosyasını yerleştir>

Verileri toplamak ve bunları post yöntemiyle veritabanına göndermek için 42.6 numaralı Listede SimpleRegistration.html adlı bir HTML dosyası oluşturun.

#### Liste 42.6 SimpleRegistration.html

<Yan Not satır 9: eylem>

<Yan Not satır 31: formu gönder>

```
<!-- SimpleRegistration.html -->
<html>
<kafa>
<başlık>Basit Kayıt Onaysız</başlık>
</head>
<gövde>
Lütfen eğitmeninizin öğrenci adres defterine kaydolun.

<form yöntemi = "post" eylem = "BasitKayıt"
<p>Soyadıt*t
<input type = "text" name = "soyadi">
Ad*
<input type = "text" isim = "ad">
MI<input type = "text" adı = "mi" boyut = "3">
</p>
Telefon
<giriş türü = "text" adı = "telefon" boyut = "20">
E-posta
<giriş türü = "text" adı = "e-posta" boyut = "28">
</p>
<p>Sokak<input type = "text" adı = "sokak" boyut = "50">
</p>
<p>Şehir<input type = "text" adı = "şehir" boyut = "23">
```

```

Eyalet
<select boyut = "1" ad = "eyalet">
 <option değer = "GA">Georgia-GA</option>
 <option değer = "OK">Oklahoma-OK</option>
 <option değer = "IN">Indiana-IN</option>
</select>
Posta Kodu <input type="text" value="9" />
<p>
 <input type="submit" value="Gönder" />
 <input type="reset" value="Sıfırla" />
</p>
</form>
<p>* zorunlu alanlar</p>
</body>
</html>

```

<Yan Not: .class dosyasını yerleştir>

Listing 42.7'de SimpleRegistration adlı servlet'i oluşturun.

#### Listeleme 42.7 SimpleRegistration.java

<Yan Not satır 14: db'yi başlat>  
 <Yan Not satır 18: POST'u işleme>  
 <Yan Not satır 20: içerik türü>  
 <Yan Not satır 21: tarayıcıya çıktı>  
 <Yan Not satır 24: parametreleri al>  
 <Yan Not satır 39: kaydı sakla>  
 <Yan Not satır 50: akışı kapat>  
 <Yan Not satır 58: db'ye bağlan>  
 <Yan Not satır 63: ifadeyi hazırla>  
 <Yan Not satır 76: değerleri ayarla>  
 <Yan Not satır 85: SQL'i çalıştır>

```

paket bölüm42;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*;

public class SimpleRegistration extends HttpServlet {
 // Veritabanına bir öğrenciyi saklamak için hazırlanan bir ifade kullanın
 private PreparedStatement pstmt;

 /** Değişkenleri başlat */
 public void init() throws ServletException {
 initializeJdbc();
 }

 /** HTTP Post isteğini işleme */
 public void doPost(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType("text/html");
 Print
 Writer out = Response.getWriter();

 // İstemciden parametreleri alın
 String lastName = request.getParameter("lastName");
 String firstName = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 }
}

```



```

String phone = request.getParameter("telefon");
String email = request.getParameter("email");
String address = request.getParameter("sokak");
String city = request.getParameter("şehir");
String state = request.getParameter("eyalet");
String zip = request.getParameter("posta kodu");

denemek {
 if (lastName.length() == 0 || firstName.length() == 0) {
 out.println("Soyadı ve Ad gereklidir");
 }
 başka {
 StoreStudent(soyadı, adı, mi, telefon, e-posta, n adresi, şehir, eyalet, zip);

 out.println(firstName + " " + lastName +
 " artık veritabanında kayıtlıdır");
 }
}
catch (İstisna örneği) {
 out.println("Hata: " + ex.getMessage());
}
Sonunda {
 out.close(); // Akışı kapat
}
}

/** Veritabanı bağlantısını başlat */
private void initializeJdbc() {
 denemek {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir bağlantı kurun
 Connection conn = DriverManager.getConnection(
 "jdbc:mysql://localhost/javabook", "scott", "tiger");
 System.out.println("Veritabanına bağlandı");

 // Bir Statement oluşturun
 pstmt = conn.prepareStatement("Address tablosuna ekle " +
 "(soyadı, adı, mi, telefon, e-posta, sokak, şehir, " +
 " + "eyalet, posta kodu) değerleri (?, ?, ?, ?, ?, ?, ?, ?)");
 }
 catch (Exception ex) {n ex.printStackTrace();n }
}

/** Bir öğrenci kaydını veritabanına kaydet */
private void storeStudent(String lastName, String firstName,
 String mi, String phone, String email, String address,
 String city, String state, String zip) throws SQLException {
 pstmt.setString(1, lastName);
 pstmt.setString(2, firstName);n pstmt
 .setString(3, mi);
 pstmt.setString(4, telefon);n pstmt
 .setString(5, e-posta);n pstmt
 .setString(6, adres);n pstmt.setStrin
 g(7, şehir);n pstmt.setString(8,
 durum);
}

```

```

 pstmt.setString(9, zip);n pstmt
 .executeUpdate();n }
 }

```

init metodu (satır 13) servlet başladığında bir kez çalıştırılır. Servlet başladıktan sonra, servlet, servlet konteynerinde canlı olduğu sürece birçok kez çağrılabilir. Sürücüyü yükleyin ve servlet'in init metodundan (satır 14) veritabanına bağlanın. Eğer bir hazırlıklı ifade veya çağrılabilir ifade kullanılıyorsa, bu da init metodunda oluşturulmalıdır. Bu örnekte, bir hazırlıklı ifade tercih edilir, çünkü servlet her zaman farklı değerlerle aynı ekleme ifadesini kullanır.

Bir servlet, JDBC aracılığıyla herhangi bir ilişkisel veritabanına bağlanabilir. Bu örnekteki initializeJdbc metodu, bir MySQL veritabanına bağlanır (satır 58). Bağlandıktan sonra, veritabanına bir öğrenci kaydı eklemek için bir hazırlıklı ifade oluşturur. Bu örnekte MySQL kullanılmıştır; bunu herhangi bir ilişkisel veritabanıyla değiştirebilirsiniz.

Soyadı ve adı zorunlu alanlardır. Bunlardan biri boşsa, servlet istemciye bir hata mesajı gönderir (satırlar 35–36). Aksi takdirde, servlet verileri

hazırlanmış ifade kullanarak veritabanında saklar.

## 42.8 Oturum Takibi

Web sunucuları, Hiper Metin Taşıma Protokolü (HTTP) kullanır. HTTP, stateless bir protokoldür. HTTP Web sunucusu, bir istemciden gelen istekleri ilişkilendiremez ve bu nedenle her isteği bağımsız olarak ele alır. Bu protokol, her isteğin tipik olarak bir HTML dosyası veya bir metin dosyasının istemciye geri gönderilmesiyle sonuçlandığı basit Web taraması için iyi çalışır. Bu tür basit istekler izole edilmiştir. Ancak, etkileşimli Web uygulamalarındaki istekler genellikle ilişkilidir. Aşağıdaki senaryodaki iki isteği düşünün:

İstek 1: Bir istemci, sunucuya kayıt verilerini gönderir; sunucu daha sonra verileri kullanıcıya onay için geri gönderir.

İstek 2: İstemci, İstek 1'de gönderilen verileri onaylar.

İstek 2'de, İstek 1'de gönderilen veriler onaylanır. Bu iki istek bir oturumda ilişkilidir. *oturum*, belirli bir zaman diliminde tek bir istemci ile Web sunucusu arasında ilişkili etkileşimlerin bir serisi olarak tanımlanabilir. Bir oturumda istekler arasındaki verileri takip etmek, oturum takibi olarak bilinir.

Bu bölüm, oturum takibi için üç tekniği tanıtmaktadır: *gizli değerler kullanma*, *çerezler kullanma* ve *servlet API'sinden oturum takibi araçlarını kullanma*.

### 42.8.1 Gizli Değerler Kullanarak Oturum Takibi

Bir oturumu, verileri dinamik olarak oluşturulan bir HTML formunda gizli değerler olarak istemciye geçirerek takip edebilirsiniz. Bunu yapmak için şöyle bir alan ekleyebilirsiniz:

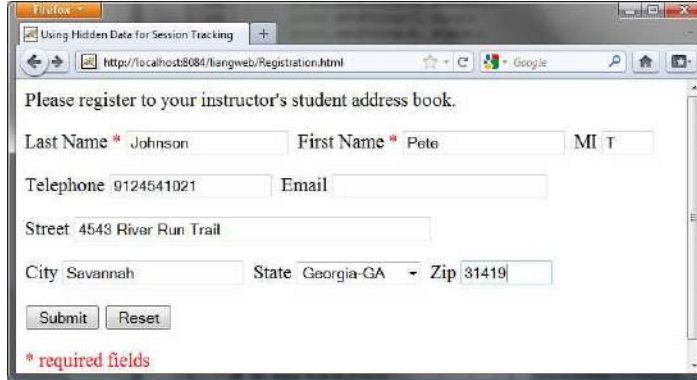
```

<giriş türü = "gizli" isim = "soyadı"t değeri = "Smith"

```

Sonraki istek, verileri servlet'e geri gönderecektir. Servlet, bu gizli değeri, getParameter yöntemi kullanarak diğer parametre değerleri gibi alır.

Gizli değerlerin bir formda nasıl kullanılacağını göstermek için bir örnek kullanalım. Örnek, bir kayıt formunu işleyen bir servlet oluşturur. İstemci, formu GET yöntemi ile gönderir; bu, Şekil 42.21'de gösterilmiştir. Sunucu, formdaki verileri toplar, bunları istemciye gösterir ve istemciden onay ister; bu, Şekil 42.22'de gösterilmiştir. İstemci, gizli değerlerle birlikte POST yöntemi kullanarak isteği göndererek verileri onaylar. Son olarak, servlet verileri bir veritabanına yazar.



Firefox - Using Hidden Date for Session Tracking +

http://localhost:8084/lanqweb/Registration.html

Please register to your instructor's student address book.

Last Name \* Johnson First Name \* Pete MI T

Telephone 9124541021 Email

Street 4543 River Run Trail

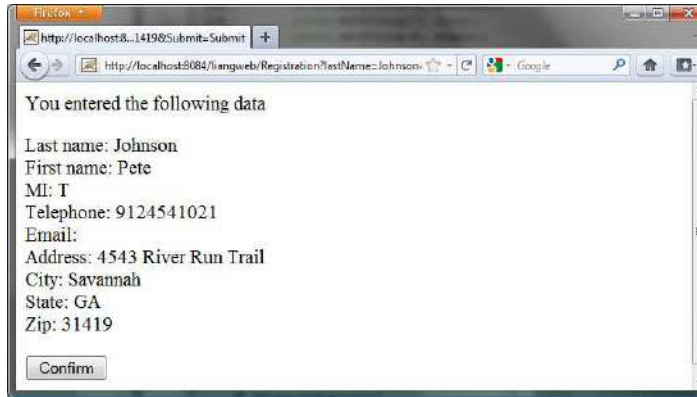
City Savannah State Georgia-GA Zip 31419

Submit Reset

\* required fields

Şekil 42.21

Kayıt formu kullanıcı bilgilerini toplar.



Firefox - http://localhost:14198/Submit:Submit +

http://localhost:8084/lanqweb/Registration.html?testName=Johnson

You entered the following data

Last name: Johnson

First name: Pete

MI: T

Telephone: 9124541021

Email:

Address: 4543 River Run Trail

City: Savannah

State: GA

Zip: 31419

Confirm

Şekil 42.22

Servlet, istemciden girişi onaylamasını ister.

Veri toplamak ve bunu onay için GET yöntemi kullanarak veritabanına göndermek üzere Listing 42.8'de Registration.html adında bir HTML formu oluşturun. Bu dosya, Listing 42.6 ile neredeyse aynıdır, SimpleRegistration.html ancak eylem Kayıt (satır 9) ile değiştirilmiştir.

#### Listeleme 42.8 Kayıt.html

<Yan Not satır 9: eylem>

<Yan Not satır 32: formu gönder>

<!-- Kayıt.html -->

<html>

<kafa>

<title>Oturum Takibi için Gizli Verilerin Kullanımı</title>

</head>

<gövde>

Lütfen eğitmeninizin öğrenci adres defterine kaydolun.

<form yöntemi = "get" eylem = "Kayıt">

<p>Soyadı <font color = "#FF0000">\*</font>

<input type = "text" name = "lastName">&nbsp;

Ad <font color = "#FF0000">\*</font>

<input type = "text" name = "ad">&nbsp;

MI <input type = "text" name = "mi" boyut = "3">

</p>

Telefon

<input type = "text" name = "telefon" boyut = "20">&nbsp;

E-posta

<input type = "text" name = "e-posta" boyut = "28">&nbsp;

</p>

<p>Sokak <input type = "text" name = "sokak" boyut = "50">

</p>

<p>Şehir <input type = "text" name = "şehir" boyut = "23">&nbsp;

Eyalet

<select boyut = "1" ad = "eyalet">

<option değer = "GA">Georgia-GA</option>

<option değer = "OK">Oklahoma-OK</option>

<option değer = "IN">Indiana-IN</option>

</select>&nbsp;

Posta Kodu <input type = "text" name = "zip" boyut = "9">

</p>

<p><input type = "submit" value = "Gönder" >

<input type = "reset" value = "Sıfırla" >

</p>

</form>

<p><font color = "#FF0000">\*</font> zorunlu alanlar</p>

</body>

</html>

Listeleme 42.9'da Kayıt adında servlet oluşturun.

#### Liste 42.9 Registration.java

<Yan Not 14. satır: db'yi başlat>

<Yan Not 18. satır: GET işle>

<Yan Not 20. satır: içerik türü>

<Yan Not 21. satır: tarayıcıya çıktı>

<Yan Not 24. satır: parametreleri al>

<Yan Not 51. satır: istemci doğrulama>

<Yan Not 80. satır: POST işle>

<Yan Not 82. satır: içerik türü>

<Yan Not 83. satır: tarayıcıya çıktı>

<Yan Not 86. satır: parametreleri al>

<Yan Not 126. satır: kaydı sakla>

paket chapter42;

```

javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*;

public class Registration extends HttpServlet {
 // Veritabanına bir öğrenciyi kaydetmek için hazırlanan bir ifade kullanın
 private PreparedStatement pstmt;

 /** Değişkenleri başlat */
 public void init() throws ServletException {
 initializeJdbc();
 }

 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType("text/html");
 Print
 Writer out = Response.getWriter();

 // Formdan verileri alın
 String lastName = request.getParameter("lastName");
 String firstName = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 String telefon = request.getParameter("telefon");
 String email = request.getParameter("email");
 String street = request.getParameter("street");
 String city = request.getParameter("city");
 String state = request.getParameter("state");
 String zip = request.getParameter("zip");

 if (lastName.length() == 0 || firstName.length() == 0) {
 out.println("Soyadı ve Ad gereklidir");
 }
 başka {
 // Onay iste
 out.println("Aşağıdaki verileri girdiniz");
 out.println("<p>Soyadı: " + lastName);
 out.println("
Ad: " + firstName);
 out.println("
MI: " + mi);
 out.println("
Telefon: " + telephone);
 out.println("
Email: " + email);
 out.println("
Adres: " + street);
 out.println("
Şehir: " + city);
 out.println("
Eyalet: " + state);
 out.println("
Posta Kodu: " + zip);

 // Cevapları işlemek için eylemi ayarlayın
 out.println("<p><form method=\"post\" action=\" " +
 "Kayıt>");
 // Gizli değerleri ayarla
 out.println("<p><input type=\"hidden\" " +
 "value=\" " + soyad + " name=\"lastName\">");
 out.println("<p><input type=\"hidden\" " +
 "value=\" " + ad + " name=\"firstName\">");
 out.println("<p><input type=\"hidden\" " +
 "value=\" " + mi + " name=\"mi\">");
 out.println("<p><input type=\"hidden\" " +
 "value=\" " + telefon + " name=\"telephone\">");
 out.println("<p><input type=\"hidden\" " +

```

```

 "value=" + email + " name=\"email\">");
 out.println("<p><input type=\"hidden\" " +
 "value=" + sokak + " name=\"street\">");
 out.println("<p><input type=\"hidden\" " +
 "value=" + şehir + " name=\"city\">");
 out.println("<p><input type=\"hidden\" " +
 "value=" + eyalet + " name=\"state\">");
 out.println("<p><input type=\"hidden\" " +
 "value=" + postaKodu + " name=\"zip\">");
 out.println("<p><input type=\"submit\" value=\"Onayla\" >");
 out.println("</form>");
 }

 out.close(); // Akışı kapat

 /** HTTP Post isteğini işleme */
 public void doPost(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType("text/html");
 Print
 Writer out = Response.getWriter();

 denemek {
 String soyad = request.getParameter("lastName");
 String ad = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 String telefon = request.getParameter("telephone");
 String email = request.getParameter("email");
 String sokak = request.getParameter("street");
 String şehir = request.getParameter("city");
 String eyalet = request.getParameter("state");
 String postaKodu = request.getParameter("zip");

 storeStudent(soyad, ad, mi, telefon, email,
 sokak, şehir, eyalet, postaKodu);

 out.println(ad + " " + soyad +
 " artık veritabanında kayıtlı");
 }
 catch (İstisna örneği) {
 out.println("Hata: " + ex.getMessage());
 }
 }

 /** Veritabanı bağlantısını başlat */
 private void initializeJdbc() {
 denemek {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir bağlantı kurun
 Connection conn = DriverManager.getConnection
 ("jdbc:mysql://localhost/javabook", "scott", "tiger");
 System.out.println("Veritabanına bağlandı");

 // Bir Statement oluşturun
 pstmt = conn.prepareStatement("insert into Address " +
 "(soyad, ad, mi, telefon, email, sokak, şehir, " +
 "eyalet, postaKodu) values (?, ?, ?, ?, ?, ?, ?, ?)");
 }
 }

```

```

 catch (Exception ex) {n Syst
 em.out.println(ex);n }
 }

 /** Bir öğrenci kaydını veritabanına kaydet */
 private void storeStudent(String lastName, String firstName,
 String mi, String phone, String email, String address,
 String city, String state, String zip) throws SQLException {
 pstmt.setString(1, lastName);
 pstmt.setString(2, firstName);n pstmt
 .setString(3, mi);
 pstmt.setString(4, telefon);
 pstmt.setString(5, email);
 pstmt.setString(6, adres);
 pstmt.setString(7, şehir);
 pstmt.setString(8, eyalet);
 pstmt.setString(9, postaKodu);
 pstmt.executeUpdate();
 }
}

```

Servlet, istemcinin girdiğini görüntüleyen ve istemcinin onayını isteyen bir HTML sayfası oluşturarak GET isteğini işler. Girdi verileri, yeni oluşturulan formlardaki gizli değerlerden oluşur, bu nedenle onay isteğinde geri gönderilecektir. Onay isteği POST yöntemini kullanır. Servlet, gizli değerleri alır ve veritabanına kaydeder.

İlk isteğin veritabanına hiçbir şey yazmadığı için GET yöntemini kullanmak uygun değildir. İkinci isteğin veritabanında bir güncelleme ile sonuçlanması nedeniyle POST yöntemi kullanılmalıdır.

NOT: Gizli değerler, istek GET yöntemini kullanıyorsa URL sorgu dizesinden de gönderilebilir.

#### 42.8.2 Çerezler Kullanarak Oturum Takibi

Oturumları, istemcinin bilgisayarında disk üzerinde ad/değer çiftlerini depolayan küçük metin dosyaları olan çerezler kullanarak takip edebilirsiniz. Çerezler, HTTP yanıtının başlığındaki talimatlar aracılığıyla sunucudan gönderilir. Talimatlar, tarayıcıya belirli bir ad ve ilişkili değeri olan bir çerez oluşturmasını söyler. Tarayıcı da zaten anahtar adıyla bir çerez varsa, değer güncellenecektir. Tarayıcı, aynı sunucuya gönderilen her istekte çerezi gönderecektir. Çerezlerin, gönderilmeyeceği bir son kullanma tarihi ayarlanabilir. Çerezleri oluşturmak ve manipüle etmek için javax.servlet.http.Cookie kullanılır; bu, Şekil 42.23'te gösterilmiştir.

<PD: UML Sınıf Diyagramı>

javax.servlet.http.Çerez	
+Cookie(ad: String, değer: String)	Belirtilen ad-değer çifti ile bir çerez oluşturur.
+getName(): String	Çerezin adını döndürür.
+getValue(): String	Çerezin değerini döndürür.
+setValue(yeniDeğer: String): void	Çerez oluşturulduktan sonra çereze yeni bir değer atar.
+getMaxAge(): int	Çerezin maksimum yaşını, saniye cinsinden döndürür.
+setMaxAge(süresi: int): void	Çerezin maksimum yaşını belirtir. Varsayılan olarak, bu değer -1'dir, bu da çerezin tarayıcı kapatılana kadar kalıcı olduğu anlamına gelir. Bu değeri 0 olarak ayarlarsanız, çerez silini
+getSecure(): boolean	r. Tarayıcının yalnızca güvenli bir protokol üzerinden çerez gönderip göndermediğini kontrol eder ve true döndürür.
+setSecure(durum: boolean): void	Tarayıcıya çerezin yalnızca güvenli
+getComment(): String	protokol, örneğin HTTPS veya SSL kullanılarak gönderilip gönderilmeyeceğini belirtir.
+setComment(amaç: String): void	Bu çerezin amacını tanımlayan yorumu döndürür veya çerezde yorum yoksa null döner.
	Bu çerez için yorumu ayarlar.

**Şekil 42.23**

*Çerez, bir ad/değer çifti ve çerez hakkında diğer bilgileri yerine getirir.*

Bir çerezi tarayıcıya göndermek için, aşağıda gösterildiği gibi HttpServletResponse sınıfındaki addCookie yöntemini kullanın:

```
response.addCookie(cookie);
```

burada response, HttpServletResponse'nin bir örneğidir.

Bir tarayıcıdan çerezleri almak için request.getCookies(); kullanın.

istek, HttpServletRequest örneğidir.

Çerezlerin kullanımını göstermek için, Listing 42.9, Registration.java ile aynı görevi yerine getiren bir örnek oluşturalım. Oturum takibi için gizli değerler kullanmak yerine, çerezler kullanır.

Listing 42.10'da RegistrationWithHttpCookie adında bir servlet oluşturun. Registration.html ile aynı olan RegistrationWithCookie.html adında bir HTML dosyası oluşturun, tek farkı eylemin KayıtWithCookieHtt.t.

#### **Listeleme 42.10 RegisterWithCookie.java**

<Yan Not satır 19: GET işlemini işleme>  
 <Yan Not satır 25: parametreleri al>  
 <Yan Not satır 40: çerezleri oluştur>  
 <Yan Not satır 42: çerezleri ekle>  
 <Yan Not satır 62: istemci doğrulama>  
 <Yan Not satır 84: POST işlemini işleme>  
 <Yan Not satır 100: çerezleri al>  
 <Yan Not satır 125: kaydı sakla>  
 paket bölüm42;

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
java.sql.*'ı içe aktarın;
```



```

public class RegistrationWithCookie extends HttpServlet {
 private static final String CONTENT_TYPE = "text/html";
 // Veritabanına bir öğrenciyi kaydetmek için bir hazırlıklı ifade kullanın
 private PreparedStatement pstmt;

 /** Değişkenleri başlat */
 public void init() throws ServletException {
 initializeJdbc();
 }

 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType("text/html");n Print
 Writer out = Response.getWriter();

 // Formdan verileri alın
 String lastName = request.getParameter("lastName");
 String firstName = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 String telefon = request.getParameter("telefon");n String email = r
 equest.getParameter("email");
 String street = request.getParameter("street");
 String city = request.getParameter("city");
 String state = request.getParameter("state");
 String zip = request.getParameter("zip");

 if (lastName.length() == 0 || firstName.length() == 0) {
 out.println("Soyadı ve Ad gereklidir");
 }
 başka {
 // Çerezleri oluşturun ve çerezleri tarayıcılara gönderin
 Cookie cookieLastName = new Cookie("lastName", lastName);
 // cookieLastName.setMaxAge(1000);
 Response.addCookie(cookieLastName);n Co
 okie cookieFirstName = new Cookie("firstName", FirstName);n Response.ad
 dCookie(cookieFirstName);
 // cookieFirstName.setMaxAge(0);n Co
 kie cookieMi = new Cookie("mi", mi);n Response.
 addCookie(cookieMi);
 Çerez çerezTelefon = new Çerez("telefon", telefon);n yanıt.addCookie(cookie
 Telephone);
 Çerez çerezEmail = new Çerez("e-posta", e-posta);n yanıt.add
 Cookie(cookieEmail);
 Cookie cookieStreet = new Cookie("sokak", sokak);n yanıt.addCo
 okie(cookieStreet);
 Cookie cookieCity = new Cookie("şehir", şehir);n yanıt.ad
 dCookie(cookieCity);
 Çerez cookieState = new Cookie("durum", durum);n yanıt.ad
 dCookie(cookieState);
 Cookie cookieZip = new Cookie("zip", zip);n Response
 .addCookie(cookieZip);

 // Onay iste
 out.println("Aşağıdaki verileri girdiniz");
 out.println("<p>Soyadı: " + lastName);
 out.println("
Ad: " + firstName);
 out.println("
MI: " + mi);
 out.println("
Telefon: " + telephone);
 out.println("
Email: " + email);

```

```

 out.println("
Sokak: " + street);
 out.println("
Şehir: " + city);
 out.println("
Eyalet: " + state);
 out.println("
Posta Kodu: " + zip);

 // Cevapları işlemek için eylemi ayarlayın
 out.println("<p><form method=\"post\" action=\" " +
 "RegistrationWithCookie>");
 out.println("<p><input type=\"submit\" value=\"Onayla\" >");
 out.println("</form>");
 }

 out.close(); // Akışı kapat

 /** HTTP Post isteğini işleme */
 public void doPost(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType(CONTENT_TYPE);
 PrintWriter çıkışı = Response.getWriter();

 String lastName = "";
 String firstName = "";
 String mi = "";
 String telefon = "";
 String e-posta = "";
 String sokak = "";
 String şehir = "";
 String durumu = "";
 String zip = "";

 // Çerezleri oku
 Çerez[] çerezler = request.getCookies();

 // Çerez değerlerini al
 for (int i = 0; i < cookies.length; i++) {
 if (cookies[i].getName().equals("lastName"))
 lastName = cookies[i].getValue();
 else if (cookies[i].getName().equals("firstName"))
 firstName = çerezler[i].getValue();
 else if (cookies[i].getName().equals("mi"))
 mi = çerezler[i].getValue();
 else if (cookies[i].getName().equals("telefon"))
 telefon = çerezler[i].getValue();
 else if (cookies[i].getName().equals("email"))
 email = çerezler[i].getValue();
 else if (cookies[i].getName().equals("sokak"))
 sokak = çerezler[i].getValue();
 else if (cookies[i].getName().equals("şehir"))
 şehir = çerezler[i].getValue();
 else if (cookies[i].getName().equals("durum"))
 state = çerezler[i].getValue();
 else if (cookies[i].getName().equals("zip"))
 zip = çerezler[i].getValue();
 }

 denemek {
 storeStudent(soyadı, adı, mi, telefon, e-posta, sokak,
 şehir, eyalet, zip);

 out.println(adı + " " + soyadı +

```

```

 " artık veritabanında kayıtlıdır");

 out.close(); // Akışı kapat
}
catch(İstisna örneği) {
 out.println("Hata: " + ex.getMessage());
}
}

/** Veritabanı bağlantısını başlat */
private void initializeJdbc() {
 denemek {
 // JDBC sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir bağlantı kurun
 Connection conn = DriverManager.getConnection
 ("jdbc:mysql://localhost/javabook", "scott", "tiger");
 System.out.println("Veritabanına bağlandı");

 // Bir Statement oluşturun
 pstmt = conn.prepareStatement("insert into Address " +
 "(soyad, ad, mi, telefon, email, sokak, şehir, "
 + "eyalet, postaKodu) values (?, ?, ?, ?, ?, ?, ?, ?)");
 }
 catch (Exception ex) {n Syst
 em.out.println(ex);n }
 }

 /** Bir öğrenci kaydını veritabanına kaydet */
 private void storeStudent(String soyadı, String adı,
 String mi, String telefon, String e-posta, String sokak,
 String şehir, String eyalet, String zip) throws SQLException {
 pstmt.setString(1, soyadı);
 pstmt.setString(2, FirstName);n pstmt
 .setString(3, mi);
 pstmt.setString(4, telefon);
 pstmt.setString(5, e-posta);
 pstmt.setString(6, sokak);
 pstmt.setString(7, şehir);
 pstmt.setString(8, eyalet);
 pstmt.setString(9, zip);
 pstmt.executeUpdate();
 }
}

```

Takip etmek istediğiniz her değer için bir çerez oluşturmalısınız, Çerez sınıfının tek yapıcısını kullanarak, bu çerezin adını ve değerini aşağıda gösterildiği gibi tanımlarsınız (satır 40):

```

Çerez çereziLastName = t.yeni Cookie ("soyadi", soyadı);

```

Çerezi tarayıcıya göndermek için, aşağıdaki gibi bir ifade kullanın (satır 42) :

```

response.addCookie(cookieLastName);

```

Eğer tarayıcıda aynı isimde bir çerez zaten varsa, değeri güncellenir; aksi takdirde, yeni bir çerez oluşturulur.

Çerezler, istemciden gelen her istekle birlikte otomatik olarak Web sunucusuna gönderilir. Servlet, get\_cookies yöntemi (satır 100) kullanarak tüm çerezleri bir diziye alır:

```
Cookie[] cookies = request.getCookies();
```

Çerezin adını almak için getName yöntemini (satır 104) kullanın:

```
String name = cookies[i].getName();
```

Çerezin değeri, getValue yöntemi kullanılarak elde edilebilir: \_\_\_\_\_

```
String value = cookies[i].getValue();
```

Çerezler, form parametreleri ve gizli değerler gibi dizeler olarak saklanır. Eğer bir çerez sayısal bir değeri temsil ediyorsa, bunu Integer sınıfındaki parseInt yöntemi veya Double sınıfındaki parseDouble yöntemi kullanarak bir tamsayıya veya double'a dönüştürmeniz gerekir. \_\_\_\_\_

Varsayılan olarak, yeni oluşturulan bir çerez, tarayıcı kapatılana kadar kalır. Ancak, setMaxAge yöntemini kullanarak bir son kullanma tarihi ayarlayabilir ve bir çerezin tarayıcıda 2,147,483,647 saniyeye (yaklaşık 24,855 gün) kadar kalmasına izin verebilirsiniz.

#### 42.8.3 Servlet API Kullanarak Oturum Takibi

Artık gizli değerler kullanarak oturum takibi ve çerezler kullanarak oturum takibi öğrendiniz. Bu iki oturum takibi yöntemi sorunlara sahiptir. Verileri ya gizli değerler ya da çerezler olarak tarayıcıya gönderirler. Veriler güvenli değildir ve bilgisayar bilgisi olan herkes bunları elde edebilir. Gizli veriler HTML biçimindedir ve tarayıcıdan görüntülenebilir. Çerezler, tarayıcının Önbellek dizininde saklanır. Güvenlik endişeleri nedeniyle bazı tarayıcılar çerezleri kabul etmez. İstemci çerezleri kapatabilir ve sayısını sınırlayabilir.

Başka bir sorun, gizli verilerin ve çerezlerin verileri dize olarak iletmesidir. Bu iki yöntemle nesneleri iletemezsiniz.

Bu sorunları çözmek için, Java servlet API'si, bir kullanıcıyı birden fazla sayfa isteği veya bir web sitesine ziyaret sırasında tanımlamak ve o kullanıcı hakkındaki bilgi depolamak için bir yol sağlayan javax.servlet.http.HttpSession arayüzünü sunar. Servlet konteyneri, bir HTTP istemcisi ile bir HTTP sunucusu arasında bir oturum oluşturmak için bu arayüzü kullanır. Oturum, belirli bir süre boyunca, kullanıcının birden fazla bağlantısı veya sayfa isteği arasında devam eder. Bir oturum genellikle bir kullanıcıya karşılık gelir; bu kullanıcı bir siteyi birçok kez ziyaret edebilir. Oturum, büyük bir veri kümesinin izlenmesini sağlar. Veriler nesne olarak depolanabilir ve sunucu tarafında tutuldukları için güvenlidir.

Java servlet API'sini oturum izleme için kullanmak üzere, öncelikle HttpServletRequest arayüzündeki getSession() yöntemi kullanılarak bir oturum nesnesi oluşturun: \_\_\_\_\_

```
HttpSession session = request.getSession();
```

Bu, oturumu alır veya istemcinin sunucuda bir oturumu yoksa yeni bir oturum oluşturur.

HttpSession arayüzü, oturuma veri okumak ve depolamak için yöntemler ve oturumu manipüle etmek için yöntemler sağlar; bu, Şekil 42.24'te gösterilmiştir.

### <PD: UML Sınıf Diyagramı>

«arayüz» javax.servlet.http.HttpSession	
+getAttribute(ad: String): Object	Bu oturumda belirtilen ad ile bağlı olan nesneyi döndürür, eğer ad altında hiçbir nesne bağlı değilse null döner.
+setAttribute(ad: String, değer: Object): geçersiz	Bu oturuma belirtilen adı kullanarak bir nesne bağlar. Eğer aynı adı taşıyan bir nesne zaten oturuma bağlıysa, nesne değiştirilir.
+getId(): Dize	Bu oturuma atanan benzersiz tanımlayıcıyı içeren bir dize döndürür. Tanımlayıcı, servlet konteyneri tarafından atanır ve uygulama bağımlıdır.
+getLastAccessedTime(): uzun	Müşterinin bu oturumla ilişkili bir isteği en son gönderdiği zamanı, 1 Ocak 1970 GMT'den itibaren geçen milisaniye cinsinden döndürür ve konteynerin isteği aldığı zamanla işaretlenir.
+geçersiz kıl(): geçersiz	Bu oturumu geçersiz kılar, ardından ona bağlı olan herhangi bir nesneyi çözümlemeden çıkarır.
+isNew(): boole	Oturumun mevcut istekte yeni oluşturulmuşsa true döndürür.
+removeAttribute(ad: String): void	Belirtilen adla bağlı olan nesneyi bu oturumdan kaldırır. Eğer oturumda belirtilen adla bağlı bir nesne yoksa, bu yöntem hiçbir şey yapmaz.
+getMaxInactiveInterval(): int n+ setMaxInactiveInterval(interval: int): void	İstemci istekleri arasında, servlet konteynerinin bu oturumu geçersiz kılmadan önce geçen süreyi, saniye cinsinden döndürür. Negatif bir süre, oturumun asla zaman aşımına uğramayacağını gösterir. Bu değeri belirtmek için setMaxInactiveInterval kullanın.

Şekil 42.24

*HttpSession, birden fazla isteğe sahip bir istemci ile sunucu arasında kalıcı bir oturum oluşturur.*

NOT: HTTP durumsuzdur. Peki sunucu, aynı istemciden gelen birden fazla isteği nasıl bir oturumla ilişkilendirir?  
Bu, arka planda servlet konteyneri tarafından yönetilir ve servlet programcısına şeffaftır.

HttpSession kullanarak gösterim yapmak için, Liste 42.9, Registration.java ve Liste 42.10, RegistrationWithCookie.java dosyalarını yeniden yazalım.  
Oturum takibi için gizli değerler veya çerezler kullanmak yerine, servlet HttpSession kullanır.

Liste 42.11'de RegistrationWithHttpSession adında bir servlet oluşturun. Registration.html ile aynı olan RegistrationWithHttpSession.html adında bir HTML dosyası oluşturun, tek farkı eylemin yerine KayıtHttpSessionHt.

#### Liste 42.11 RegistrationWithHttpSession.java

<Yan Not 18. satır: GET işle</Yan Not>  
<Yan Not 25. satır: parametreleri al>  
<Yan Not 40. satır: adres oluştur>  
<Yan Not 52. satır: oturum oluştur>  
<Yan Not 55. satır: nitelik ayarla>  
<Yan Not 80. satır: POST işle>  
<Yan Not 87. satır: oturumu al>  
<Yan Not 90. satır: adres al>  
<Yan Not 93. satır: adresi sakla>

paket bölüm42;

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*; içe aktarın;
```

```

public class RegistrationWithHttpSession extends HttpServlet {
 // Bir öğrenciyi veritabanına kaydetmek için hazırlanmış bir ifade kullanın
 private PreparedStatement pstmt;

 /** Değişkenleri başlat */
 public void init() throws ServletException {
 initializeJdbc();
 }

 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 // Yanıt türünü ayarlayın ve çıktı akışını tarayıcıya yönlendirin
 response.setContentType("text/html");
 PrintWriter çıkışı = yanıt.getWriter();

 // Formdan verileri alın
 String lastName = request.getParameter("lastName");
 String firstName = request.getParameter("firstName");
 String mi = request.getParameter("mi");
 String telefon = request.getParameter("telefon");n String email = r
 equest.getParameter("email");
 String street = request.getParameter("street");
 String city = request.getParameter("city");
 String state = request.getParameter("state");
 String zip = request.getParameter("zip");

 if (lastName.length() == 0 || firstName.length() == 0) {
 out.println("Soyadı ve Ad gereklidir");
 }
 başka {
 // Bir Adres nesnesi oluşturun
 Address address = new Address();
 address.setLastName(lastName);
 address.setFirstName(firstName);
 address.setMi(mi);
 adres.setTelefon(telefon);n adres.setEm
 ail(e-posta);
 adres.setStreet(sokak);
 adres.setŞehir(şehir);
 adres.setState(durum);
 adres.setZip(zip);

 // Bir HttpSession alın veya yoksa bir tane oluşturun
 HttpSession httpSession = request.getSession();

 // Öğrenci nesnesini oturuma kaydet
 httpSession.setAttribute("address", address);

 // Onay iste
 out.println("Aşağıdaki verileri girdiniz");
 out.println("<p>Soyadı: " + lastName);
 out.println("<p>Adı: " + firstName);
 out.println("<p>MI: " + mi);
 out.println("<p>Telefon: " + telephone);
 out.println("<p>Email: " + email);
 out.println("<p>Adres: " + street);
 out.println("<p>Şehir: " + city);
 out.println("<p>Eyalet: " + state);
 out.println("<p>Posta Kodu: " + zip);

```

```

// Cevapları işlemek için eylemi ayarlayın
out.println("<p><form method=\"post\" action=\" " +
"RegistrationWithHttpSession>");
out.println("<p><input type=\"submit\" value=\"Onayla\" >");
out.println("</form>");
}

out.close(); // Akışı kapat

/** HTTP Post isteğini işleme */
public void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
// Yanıt türünü ayarlayın ve çıktı akışını tarayıcıya yönlendirin
response.setContentType("text/html");
PrintWriter çıkışı = yanıt.getWriter();

// HttpSession'ı alın
HttpSession httpSession = request.getSession();

// HttpSession'daki Address nesnesini alın
Address address = (Address) (httpSession.getAttribute("address"));

denemek {
mağazaÖğrenci(adres);

out.println(address.getFirstName() + " " + address.getLastName()
+ " artık veritabanında kayıtlı");
out.close(); // Akışı kapat
}
catch (İstisna örneği) {
out.println("Hata: " + ex.getMessage());
}
}

/** Veritabanı bağlantısını başlat */
private void initializeJdbc() {
denemek {
// JDBC sürücüsünü yükleyin
Class.forName("com.mysql.jdbc.Driver");
System.out.println("Sürücü yüklendi");

// Bağlantıyı kurun
Connection conn = DriverManager.getConnection
("jdbc:mysql://localhost/javabook" , "scott", "tiger");
System.out.println("Veritabanına bağlandı");

// Bir Statement oluşturun
pstmt = conn.prepareStatement("insert into Address " +
"(soyad, ad, mi, telefon, email, sokak, şehir, "
+ "eyalet, postaKodu) values (?, ?, ?, ?, ?, ?, ?, ?)");

}
catch (Exception ex) {n Syst
em.out.println(ex);n }
}

/** Veritabanına bir adres kaydedin */
private void storeStudent(Address address) throws SQLException {
pstmt.setString(1, address.getLastName());

```

```

 pstmt.setString(2, adres.getFirstName());n pstmt.setSt
ring(3, adres.getMi());
 pstmt.setString(4, adres.getTelephone());n pstmt.setSt
ring(5, adres.getEmail());n pstmt.setString(6, adre
s.getStreet());n pstmt.setString(7, adres.getCity());n
 pstmt.setString(8, adres.getState());n pstmt.set
String(9, adres.getZip());n pstmt.executeUpdate();
 }
}

```

İfade (satır 52)

```

HttpSession httpSession = request.getSession();

```

sessiyonu alır veya eğer oturum yoksa yeni bir oturum oluşturur.

Nesneler HttpSession içinde saklanabileceğinden, bu program bir Address sınıfı tanımlar. Bir Address nesnesi oluşturulur ve setAttribute yöntemi kullanılarak oturumda saklanır; bu yöntem nesneyi aşağıda gösterilen adla bağlar (satır 55):

```

httpSession.setAttribute("address", address);

```

Nesneyi almak için aşağıdaki ifadeyi kullanın (satır 90):

```

Address address = (Address)(httpSession.getAttribute("address"));

```

Bir istemci ile bir servlet arasında yalnızca bir oturum vardır. Bir oturumda herhangi bir sayıda nesne saklayabilirsiniz. Varsayılan olarak, Tomcat ve GlassFish gibi birçok Web sunucusunda maksimum inaktif aralık 1800 saniyedir (yani, yarım saat), bu da oturumun 30 dakika boyunca aktivite yoksa sona ereceği anlamına gelir. Varsayılanı değiştirmek için setMaxInactiveInterval yöntemini kullanabilirsiniz. Örneğin, maksimum inaktif aralığı bir saate ayarlamak için

```

httpSession.setMaxInactiveInterval(3600);

```

Negatif bir değer ayarlarsanız, oturum asla sona ermeyecektir.

#### <Yan Not: Address sınıfını oluştur>

Bu servlet programının çalışması için, NetBeans'te Address sınıfını aşağıdaki gibi oluşturmalısınız:

1. Proje panelindeki liangweb düğümünün bağlam menüsünden Yeni ve Java Sınıfını seçin ve Yeni Java Sınıfı iletişim kutusunu görüntüleyin.
2. Sınıf Adı olarak Address ve paket adı olarak chapter42 girin. Sınıfı oluşturmak için Tamam'a tıklayın.
3. Kodunuzu, Liste 42.12'de gösterildiği gibi girin.

#### Liste 42.12 Address.java

<Yan Açıklama satırı 1: paket bölüm42> n<Yan Açıklama satırı 3: sınıf Adresi>

```

paket bölümü42;

genel sınıf Adres {n özel Diz
 e ilkAd;
 özel Dize mi;n özel Diz
 e lastName;n özel Dize telefon
;
}

```



```

 özel String caddesi;
 özel Dize şehri;n özel Diz
 e durumu;
 özel Dize e-postası;
 özel String zip;

 public String getFirstName() {
 this.firstName'i döndürün;
 }

 public void setFirstName(String name) {
 this.firstName = ilkAd;
 }

 genel Dize getMi() {
 this.mi'yi döndür;
 }

 genel void setMi(String mi) {
 this.mi = mi;
 }

 public String getLastName() {
 this.lastName'i döndür;
 }

 public void setLastName(String soyad) {
 this.lastName = lastName;
 }

 public String getTelephone() {
 bunu geri ver.telefon;
 }

 genel geçersiz setTelephone(String telefon) {
 this.telefon = telefon;
 }

 genel Dize getEmail() {
 bu.e-postayı iade et;
 }

 genel geçersiz setEmail(Dize e-postası) {
 this.email = e-posta;
 }

 genel Dize getStreet() {
 return this.sokak;
 }

 public void setSokak(String sokak) {
 this.sokak = sokak;
 }

 public String getSehir() {

```

```

 return this.sehir;
 }

 public void setSehir(String sehir) {
 this.sehir = sehir;
 }

 public String getEyalet() {
 return this.eyalet;
 }

 public void setEyalet(String eyalet) {
 this.durum = durum;
 }

 genel Dize getZip() {
 this.zip'i döndür;
 }

 public void setZip(String zip) {
 this.zip = zip;
 }
}

```

Bu destek sınıfı, gelecek bölümlerde de yeniden kullanılacaktır.

## 42.9 Servletlerden Görüntü Gönderme

Şu ana kadar dinamik

HTML metni üreten Java servletleri yazmayı öğrendiniz. Java servletleri, bir tarayıcıya metin göndermekle sınırlı değildir. İhtiyaç duyulduğunda görüntü döndürebilirler. Görüntüler dosyalarda saklanabilir veya programlardan oluşturulabilir.

### 42.9.1 Dosyalardan Görüntü Gönderme

Görüntüleri dosyalardan göndermek için HTML <img> etiketini kullanabilirsiniz. Etiketin söz dizimi:

---

**<img src t = URL t alt = metin hizalama = [üst | orta | alt | metinüst]>**

---

Öznitelik src \_\_\_\_\_ resmin kaynağını belirtir. Öznitelik alt \_\_\_\_\_ resmin tarayıcıda görüntülenememesi durumunda gösterilecek alternatif metni belirtir. Öznitelik hizalama, tarayıcıya resmi nereye yerleştireceğini söyler.

Bir servlette dosyadan resim almayı göstermek için, bir ülkenin bayrağını dinamik olarak üreten ve bayrağı tanımlayan bir metin oluşturan bir servlet oluşturalım. Bayrak, bir resim dosyasında saklanır ve bayrağı tanımlayan metin bir metin dosyasında saklanır.



**Şekil 42.25**

*Servlet, metinle birlikte bir resim döndürür.*

Listing 42.13'te ImageContent adında servlet oluşturun.

#### **Listeleme 42.13 ImageContent.java**

**<Yan Not satır 16: resim etiketi>**

**<Yan Not satır 20: dosyayı oku>**

```

paket bölüm42;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class ImageContent extends HttpServlet {
 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();
 String country = request.getParameter("ülke");
 out.println("");

 // Bir dosyadan açıklamayı okuyun ve tarayıcıya gönderin
 java.util.Scanner input = new java.util.Scanner(
 new File("c:\\look\\" + country + ".txt"));

 // Metin dosyasından bir satır okuyun ve tarayıcıya gönderin
 while (input.hasNext()) {
 out.println(input.nextLine());
 }

 out.close();
 }
}

```

**<Yan Not: resim dosyası konumu>**

Bir dizin oluşturmalsınız C:\ook\liangweb\web\resources\image ve bu dizinde resim dosyalarını saklayın.

Ülke parametresi hangi resim dosyasının ve metin dosyasının gösterileceğini belirler. Servlet, HTML içeriğini tarayıcıya gönderir. İçerikler, resim dosyasına referans veren bir <img> etiketi (satırlar 16–17) içerir.

Servlet, metin dosyasındaki karakterleri okur ve bunları tarayıcıya gönderir (satırlar 20-26).

#### 42.9.2 Görüntü Nesnesinden Görüntü Gönderme

Önceki örnek, bir görüntü dosyasında saklanan bir görüntüyü gösterir. Ayrıca programda dinamik olarak oluşturulan bir görüntüyü de gösterebilirsiniz.

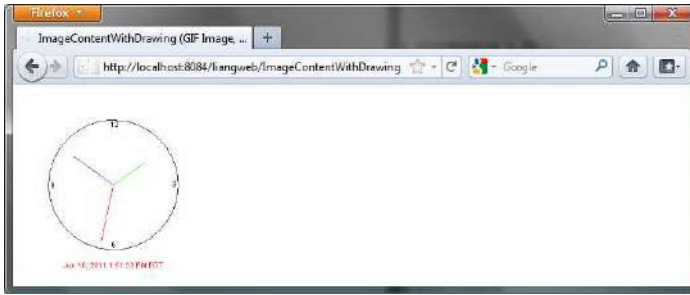
Görüntü bir tarayıcıya gönderilmeden önce, tarayıcı tarafından kabul edilebilir bir formata kodlanması gerekir. Görüntü kodlayıcıları Java API'sinin bir parçası değildir, ancak birkaç ücretsiz kodlayıcı mevcuttur. Bunlardan biri, <http://www.acme.com/java/software/Acme.JPM.Encoders.GifEncoder.html> adresinde bulunan GifEncoder sınıfıdır ve ook\lib\acme.jar içinde yer almaktadır. Görüntüyü kodlamak ve tarayıcıya göndermek için aşağıdaki ifadeyi kullanın:

```
 yeni GifEncoder(görüntü, çıkış, tehdit edildi).encode();
```

burada out, servlet'ten tarayıcıya giden bir ikili çıktı akışıdır, bu aşağıdaki ifade kullanılarak elde edilebilir:

```
 OutputStream out = response.getOutputStream();
```

Dinamik olarak görüntü oluşturmayı göstermek için, mevcut zamanı gösteren bir saat görüntüleyen bir servlet oluşturalım, Şekil 42.26'da gösterildiği gibi.



Şekil 42.26

*Servlet, mevcut zamanı gösteren bir saat döndürür.*

#### <Yan Not: acme.jar>

Listing 42.14'te ImageContentWithDrawing adında bir servlet oluşturun. NetBeans'teki liangweb projesinde Kütüphaneler düğümüne acme.jar ekleyin.  
(acme.jar c:\ook\lib.'de bulunuyor.)

#### Liste 42.14 ImageContentWithDrawing.java

<Yan Not satır 10: import GifEncoder>

<Yan Not satır 18: GET işlemi>

<Yan Not satır 20: gif türü>

<Yan Not satır 24: resim>

<Yan Not satır 28: grafikler>

<Yan Not satır 30: grafik çiz>

<Yan Not satır 35: akışı kapat>

<Yan Not satır 38: saat çiz>

paket bölüm42;

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
java.util.*'i içe aktarın;
java.text.*'i içe aktarın;
java.awt.*; içe aktar
import java.awt.image.BufferedImage;
import Acme.JPM.Encoders.GifEncoder;

public class ImageContentWithDrawing extends HttpServlet {
 /** Değişkenleri başlat */
 özel final statik int genişlik = 300;n özel final st
 atik int yükseklik = 300;

 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse
 response) throws ServletException, IOException {
 Response.setContentType("image/gif");n Outp
 utStream out = Response.getOutputStream();

 // Resmi oluştur
 Resim resmi = new BufferedImage(genişlik, yükseklik,n Bu
 fferedImage.TYPE_INT_ARGB);

 // Resmin Grafik bağlamını al
 Graphics g = image.getGraphics();

 drawClock(g); // Grafiğe bir saat çiz

 // Resmi kodla ve çıkış akışına gönder
 new GifEncoder(image, out, true).encode();

 out.close(); // Akışı kapat

 özel void drawClock(Graphics g) {
 // Saat parametrelerini başlat
 int saatYarıçapı =
 (int)(Math.min(genişlik, yükseklik) * 0.7 * 0.5);
 int xMerkez = (genişlik) / 2;
 int yCenter = (yükseklik) / 2;

 // Daire çiz
 g.setColor(Renk.siyah);
 g.drawOval(xCenter - saatYarıçapı,yCenter - saatYarıçapı,
 2 * saatYarıçapı, 2 * saatYarıçapı);
 g.drawString("12", xCenter - 5, yCenter - saatYarıçapı + 12);
 g.drawString("9", xCenter - saatYarıçapı + 3, yCenter + 5);
 g.drawString("3", xCenter + saatYarıçapı - 10, yCenter + 3);
 g.drawString("6", xCenter - 3, yCenter + saatYarıçapı - 3);

 // GregorianCalendar kullanarak mevcut zamanı al
 TimeZone timeZone = TimeZone.getDefault();
 GregorianCalendar cal = new GregorianCalendar(timeZone);

 // Saniye akrebi çiz
 int second = (int)cal.get(GregorianCalendar.SECOND);
 int sLength = (int)(saatYarıçapı * 0.9);
 int xSecond = (int)(xCenter + sLength * Math.sin(saniye *
```

```

 (2 * Math.PI / 60)));
 int ySecond = (int)(yCenter sLength * Math.cos(second *n (2 * Math.PI /
 60)));
 g.setColor(Renk.kirmizi);
 g.drawLine(xCenter, yCenter, xSecond, ySecond);

 // Dakika akrebi çiz
 int dakika = (int)cal.get(GregorianCalendar.MINUTE);n int mLength
 h = (int)(clockRadius * 0,75);
 int xMinute = (int)(xCenter + mLength * Math.sin(dakika *n (2 * Math.PI
 / 60)));
 int yMinute = (int)(yCenter mLength * Math.cos(dakika *n (2 * Math.PI /
 60)));
 g.setColor(Color.mavi);
 g.drawLine(xCenter, yCenter, xMinute, yMinute);

 // Saat ibresini çiz
 int hour = (int)cal.get(GregorianCalendar.HOUR_OF_DAY);n int hLength
 h = (int)(clockRadius * 0.6);
 int xHour = (int)(xCenter + hLength * Math.sin((saat + dakika / 60.0) * (2 * Mat
 h.PI / 12)));
 int yHour = (int)(yCenter hLength * Math.cos((saat + dakika n / 60.0) * (2 * Ma
 th.PI / 12)));
 g.setColor(Color.yesil);
 g.drawLine(xCenter, yCenter, xHour, yHour);

 // Belirtilen stil, yerel ayar ve saat diliminde görüntüleme formatını ayarla
 DateFormat formatter = DateFormat.getDateTimeInstance
 (DateFormat.MEDIUM, DateFormat.LONG);

 // Mevcut tarihi görüntüle
 g.setColor(Renk.kirmizi);
 Bugünün dizesi = formatter.format(cal.getTime());n FontMe
 trics fm = g.getFontMetrics();
 g.drawString(bugün, (genişlik -
 fm.stringWidth(today)) / 2, yCenter + watchRadius + 30);n }
 }

```

Görüntü tarayıcıya ikili veri olarak gönderildiğinden, yanıtın içerik türü image/gif olarak ayarlanmıştır (satır 20). GifEncoder sınıfı, görüntüyü tarayıcı tarafından anlaşılan içeriğe kodlamak için kullanılır (satır 33). İçerik, OutputStream nesnesi out'a gönderilir.

Program, BufferedImage sınıfını kullanarak belirtilen genişlik, yükseklik ve görüntü tipi ile bir görüntü oluşturur (satırlar 24-25):

```

Image image = new BufferedImage(width, height,
 BufferedImage.TYPE_INT_ARGB);

```

Resimleri görüntü üzerinde çizmek için, getGraphics yöntemini kullanarak grafik bağlamını almanız gerekir (satır 28):

```

Graphics g = image.getGraphics();

```

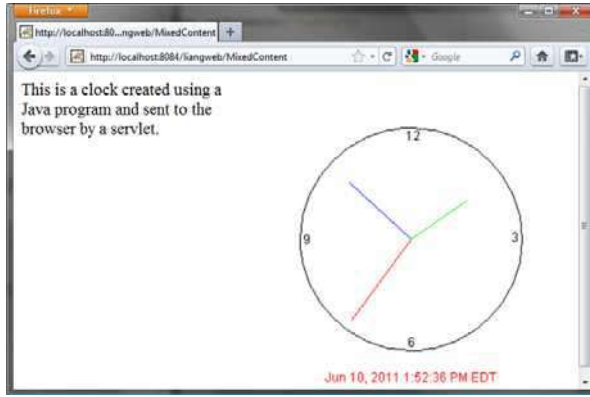
Grafikler sınıfında basit şekiller çizmek için çeşitli çizim yöntemlerini kullanabilir veya daha karmaşık grafikler çizmek için Java 2D'yi kullanabilirsiniz. Bu örnek, mevcut zamanı göstere n bir saat çizmek için basit çizim yöntemlerini kullanmaktadır.

#### 42.9.3 Resimleri ve Metinleri Birlikte Göndermek

Önceki örnekteki servlet, resimler döndürmektedir. Genellikle resimler diğer içeriklerle karıştırılır. Bu durumda, resimleri göndermeden önce içerik türünü "image/gif" olarak ayarlamamız ve metni göndermeden önce içerik türünü "text/html" olarak ayarlamamız gerekir. Ancak, içerik türü bir istekte değiştirilemez. Bu kısıtlamayı aşmak için, HTML içeriğinin resmi görüntülemek için bir <img> etiketi içinde bir GET isteği gömebilirsiniz.

HTML içeriği görüntülendiğinde, resmi almak için ayrı bir GET isteği sunucuya gönderilir. Böylece metin ve resim iki ayrı GET isteği ile elde edilir.

Resimleri ve metinleri karıştırmayı göstermek için, önceki örnekte oluşturulan saat resmini bazı metinlerle karıştıran 42.15 numaralı listede bir servlet oluşturalım; bu, Şekil 42.27'de gösterilmiştir.



Şekil 42.27

*Servlet, metinle birlikte bir resim döndürür.*

#### Liste 42.15 MixedContent.java

<Yan Notu satır 9: GET işlemi>  
<Yan Notu satır 11: içerik türü>  
<Yan Notu satır 14: al parametresi>  
<Yan Notu satır 16: resim etiketi>  
<Yan Notu satır 22: akışı kapat>

```
paket bölüm42;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class MixedContent extends HttpServlet {
 /** HTTP Get isteğini işleyin */
 public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();

 String country = request.getParameter("ülke");

 out.println("<img kaynak = "" +
```

```

 "ImageContentWithDrawing\" sağa hizala>"); ou
t.println("Bu, bir Java programı kullanılarak oluşturulmuş ve bir servlet tarafın
dan tarayıcıya gönderilmiş bir saattir.");
out.close();n }
}

```

Servlet, resim etiketi ile bir HTML dosyası oluşturur.

```

```

HTML dosyası tarayıcı tarafından işlenir. Tarayıcı resim etiketini gördüğünde, sunucuya istek gönderir. Listing 42.14'te oluşturulan ImageContentWithDrawing, resmi tarayıcıya göndermek için çağrılır.

#### Anahtar Terimler

- Ortak Geçit Arayüzü
- CGI programları
- çerez
- GET ve POST yöntemleri
- CamBalık
- HTML formu
- URL sorgu dizisi
- sunucu uygulaması
- servlet konteyneri (servlet motoru)
- servlet yaşam döngüsü yöntemleri
- Erkek kedi

## Bölüm Özeti

1. Bir servlet, bir Web sunucusundan çalışan özel bir program türüdür. Tomcat ve GlassFish, servletleri çalıştırabilen Web sunucularıdır.
2. Bir servlet URL'si, ana bilgisayar adı, port ve istek string'i (örneğin, `http://localhost:8084/liangweb/ServletClass.`) ile belirtilir. Bir servlet'i çağırmanın bir kaç yolu vardır: (1) bir Web tarayıcısından bir servlet URL'si yazarak, (2) bir HTML sayfasında bir hiper bağlantı yerleştirerek ve (3) bir HTML formuna bir servlet URL'si gömerek. Tüm istekler GET yöntemini tetikler, ancak HTML formunda POST yöntemini açıkça belirtebilirsiniz.
3. Bir servlet'i, HttpServlet sınıfını genişleten bir sınıf tanımlayarak, GET yöntemine yanıt vermek için `doGet(HttpServletRequest, HttpServletResponse)` yöntemini uygulayarak ve POST yöntemine yanıt vermek için `doPost(HttpServletRequest, HttpServletResponse)` yöntemini uygulayarak geliştirirsiniz.
4. Bir istemciden servlet'e iletilen istek bilgileri, HttpServletRequest nesnesinde bulunur. İstekten bilgi almak için `getParameter`, `getParameterValues`, `getRemoteAddr`, `getRemoteHost`, `getHeader`, `getQueryString`, `getCookies` ve `getSession` yöntemlerini kullanabilirsiniz.
5. İstemciye geri gönderilen içerik, HttpServletResponse nesnesinde bulunur. İstemciye içerik göndermek için önce içeriğin türünü (örneğin, html/plain) ayarlayın. `setContentType(contentType)` yöntemi, ardından içeriği çıktı olarak verin



HttpServletResponse nesnesi üzerinde bir I/O akışı aracılığıyla. getWriter() yöntemi kullanarak bir karakter PrintWriter akışı elde edebilir ve getOutputStream() kullanarak bir ikili OutputStream elde edebilirsiniz.

6. Bir servlet birçok istemci tarafından paylaşılabilir. Servlet ilk oluşturulduğunda, init yöntemi çağrılır. Servlet yok edilmediği sürece tekrar çağrılmaz. Sunucu, servlet için bir istek aldıkça service yöntemi çağrılır. Sunucu yeni bir iş parçacığı oluşturur ve service yöntemini çağırır. Zaman aşımı süresi geçtikten sonra veya Web sunucusu durdurulduğunda destroy yöntemi çağrılır.
7. Bir oturumu takip etmenin üç yolu vardır. Bir oturumu, servletten istemciye dinamik olarak oluşturulan bir HTML formunda gizli bir değer olarak veri geçirerek takip edebilirsiniz. Örneğin, <input type="hidden" name="lastName" value="Smith"> gibi bir alan ekleyerek. Bir sonraki istek, verileri servlet'e geri gönderecektir. Servlet, getParameter yöntemi kullanarak bu gizli değeri diğer parametre değerleri gibi alır.
8. Oturumları çerezler kullanarak takip edebilirsiniz. Bir çerez, new Cookie(String name, String value) yapıcısı kullanılarak oluşturulur. Çerezler, HttpServletResponse nesnesi aracılığıyla sunucudan gönderilir ve tarayıcıya belirli bir isim ve ilişkili değeri olan bir çerez eklemesi için addCookie(aCookie) yöntemi kullanılır. Tarayıcıda zaten anahtar adıyla bir çerez varsa, değer güncellenecektir.  
  
Tarayıcı, aynı sunucuya gönderilen herhangi bir istekle birlikte çerezi gönderecektir. Çerezlerin belirli bir son kullanma tarihi olabilir, bu tarihten sonra sunucuya gönderilmeyeceklerdir.
9. Java servlet API, büyük bir veri kümesinin izlenmesini sağlayan bir oturum izleme aracı sağlar. Bir oturum, bir HttpServletRequest nesnesi aracılığıyla getSession() yöntemi kullanılarak elde edilebilir. Veriler nesne olarak saklanabilir ve sunucu tarafında setAttribute(String name, Object value) yöntemi kullanılarak tutulduğu için güvenlidir.
10. Java servletleri, bir tarayıcıya metin göndermekle sınırlı değildir. GIF, JPEG veya PNG formatında resimler döndürebilirler.

### Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

### Gözden Geçirme Soruları

#### Bölümler 42.1-42.2

42.1 Ortak geçit arayüzü nedir?

42.2 HTML formundaki GET ve POST yöntemleri arasındaki farklar nelerdir?

42.3 Bir GET isteğini doğrudan bir URL'den gönderebilir misiniz? Bir POST isteğini doğrudan bir URL'den gönderebilir misiniz?

42.4 Aşağıdaki URL'de, parametre adı ile birlikte liang ana bilgisayarında 8084 numaralı portta FindScore servlet'ine GET isteği göndermenin yanlış olan kısmı nedir?

[http://liang:8084/findScorename="P Yates"](http://liang:8084/findScorename=)

42.5 CGI ve servletler arasındaki farklar nelerdir?

### Bölüm 42.3

42.6 Internet Explorer'ın Adres alanına tam dosya adını yazarak bir HTML dosyasını (örneğin c:est.html) görüntüleyebilir misiniz? Sadece servlet sınıf dosya adını yazarak bir servlet çalıştırabilir misiniz?

42.7 NetBeans'te bir Web projesi nasıl oluşturursunuz?

42.8 NetBeans'te bir servlet nasıl oluşturursunuz?

42.9 NetBeans'te bir servlet nasıl çalıştırılır?

42.10 NetBeans'ten bir servlet çalıştırdığınızda, varsayılan olarak port numarası nedir? Port numarası zaten kullanılıyorsa ne olur?

42.11 .war dosyası nedir? NetBeans'te bir Web projesi için .war dosyasını nasıl elde edersiniz?

### Bölüm 42.4

42.12 Bir servletin yaşam döngüsünü tanımlayın.

42.13 Web sunucusunu başlattığınızı, aşağıdaki servleti Web tarayıcısından uygun bir URL vererek iki kez çalıştırdığınızı ve sonunda Tomcat'i durdurduğunuzu varsayalım. Servlet ilk kez çağrıldığında konsolda ne görüntülendi? Servlet ikinci kez çağrıldığında konsolda ne görüntülendi? Tomcat kapatıldığında konsolda ne görüntülendi?

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

genel sınıf Test uzantıları HttpServlet {
 halk Test() {
 System.out.println("Yapıcı çağrıldı");
 }

 /** Değişkenleri başlat */
 public voidt init() t fırlatır ServletException {
 System.out.println("init çağrıldı");
 }

 /** HTTP Get isteğini işleme */
 public voidt doGet(HttpServletRequest isteği, nHttpServletResponse yanıtı)
 atar ServletException, IOException {
 System.out.println("doGet çağrıldı");
 }

 /** Kaynakları temizle */
 public void destroy() {
 System.out.println("destroy çağrıldı");
 }
}
```

### Bölümler 42.5-42.7

42.14 Listing 42.2, CurrentTime.java'da içerik türünü "html/plain" olarak değiştirirseniz ne gösterilecektir?

42.15 out.close() ifadesi, yanıtın çıktı akışını kapatmak için kullanılır. Neden bu ifade bir try-catch bloğu içinde yer almaz?

42.16 paramName mevcut değilse request.getParameter(paramName) çağrıldığında ne olur?

42.17 HTML formunda bir metin alanı, kombinasyon kutusu, onay kutusu ve metin alanı nasıl yazılır?

42.18 HTML formundan bir metin alanı, kombinasyon kutusu, liste, onay kutusu, radyo düğmesi ve metin alanı için parametre değerini nasıl alırsınız?

42.19 Servlet, JDBC-ODBC sürücüsünden başka bir veritabanı sürücüsü kullanıyorsa, sürücü NetBeans'te nereye yerleştirilmelidir?

#### *Bölüm 42.8*

42.20 Oturum takibi nedir? Oturum takibi için üç teknik nelerdir?

42.21 Bir çerez nasıl oluşturulur, bir çerez bir tarayıcıya nasıl gönderilir, bir tarayıcıdan çerezler nasıl alınır, bir çerezin adı nasıl alınır, çerezde yeni bir değer nasıl ayarlanır ve çerezin son kullanma tarihi nasıl ayarlanır?

42.22 Beş çerez göndermek için servlet içinde beş Cookie nesnesi oluşturmak zorunda mısınız?

42.23 Bir oturum nasıl alırsınız, oturum için nesne değerini nasıl ayarlarsınız ve oturumdan nesne değerini nasıl alırsınız?

42.24 Varsayalım ki, Liste'deki 53. satıra aşağıdaki kodu eklediniz.

42.11.

```
httpSession.setMaxInactiveInterval(1);
```

Kullanıcı tarayıcıdan Onayla butonuna tıkladıktan sonra ne olur? Cevabınızı programı çalıştırarak test edin.

42.25 Varsayalım ki, Liste'deki 53. satıra aşağıdaki kodu eklediniz.

42.11.

```
httpSession.setMaxInactiveInterval(-1);
```

Kullanıcı tarayıcıdan Onayla butonuna tıkladıktan sonra ne olur?

#### *Bölüm 42.9*

42.26 Görüntüleri tarayıcıya göndermek için hangi çıktı akışını kullanmalısınız? Yanıt için hangi içerik türünü ayarlamalısınız?

42.27 Görüntüler ve metinle dinamik içeriklerle nasıl başa çıkarsınız?

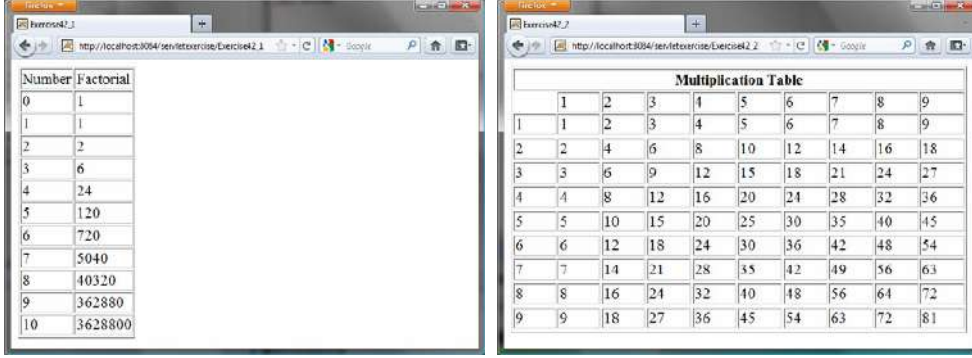
### **Programlama Alıştırmaları**

NOT: Bu bölümdeki çift numaralı alıştırmaların çözümleri, even numberedexercise.zip dosyasından indirilebilen exercis e\servletexercise içindedir.

#### *Bölüm 42.5*

42.1\*

(*Faktöriyel tablosu*) 0'dan 10'a kadar olan sayıların faktöriyelini içeren bir tabloyu görüntülemek için bir servlet yazın, Şekil 42.28a'da gösterildiği gibi.



(a)

Number	Factorial
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800

(b)

	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

Şekil 42.28

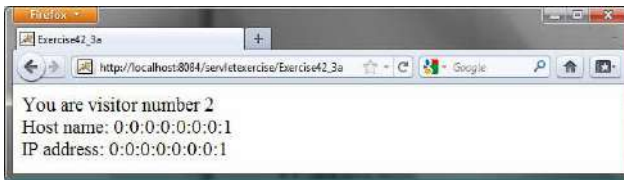
(a) Servlet, 0'dan 10'a kadar olan sayıların faktöriyelini bir tabloda görüntüler. (b) Servlet çarpanlar tablosunu görüntüler.

42.2\*

(*Çarpanlar tablosu*) Şekil 42.28(b)'de gösterildiği gibi bir çarpanlar tablosunu görüntülemek için bir servlet yazın.

42.3\*

(*Ziyaret sayısı*) Servlet'in üzerinde ziyaret sayısını gösteren bir servlet geliştirin. Ayrıca, Şekil 42.29'da gösterildiği gibi, istemcinin ana bilgisayar adını ve IP adresini de gösterin.



Şekil 42.29

Servlet, ziyaret sayısını ve istemcinin ana bilgisayar adını, IP adresini ve istek URL'sini gösterir.

Bu programı üç farklı şekilde uygulayın:

1. Sayıyı saklamak için bir örnek değişken kullanın. Servlet ilk kez oluşturulduğunda, sayı 0'dır. Servlet'in doGet yöntemi her çağrıldığında sayı artırılır. Web sunucusu durduğunda, sayı kaybolur.

2. Sayıyı Exercise39\_3.dat adlı bir dosyada saklayın ve servlet'in init yöntemi nde sayıyı okumak için RandomAccessFile kullanın. Sayı, servlet'in doGet yöntemi her çağrıldığında artırılır. Web sunucusu durduğunda, sayıyı dosyaya geri kaydedin.
3. Tüm istemcilerden toplam ziyaretleri saymak yerine, istemcinin IP adresi ile tanımlanan her istemci için ziyaretleri sayın. IP adresleri ve ziyaret sayıları çiftini saklamak için Map kullanın. İlk ziyaret için haritada bir giriş oluşturulur. Sonraki ziyaretler için, ziyaret sayısı güncellenir.

## Bölüm 42.6

42.4\*

(*Vergi hesaplama*) Kullanıcının tahsil edilebilir gelir ve dosyalama durumu girmesi için bir HTML formu yazın, Şekil 42.30a'da gösterildiği gibi. *Vergi Hesaplama* butonuna tıklamak, vergiyi hesaplamak ve görüntülemek için bir servleti çağırır, Şekil 42.30b'de gösterildiği gibi. Vergiyi hesaplamak için Listing 3.7'de tanıtilan computeTax yönteminin kullanın, ComputingTax.java.



(a)

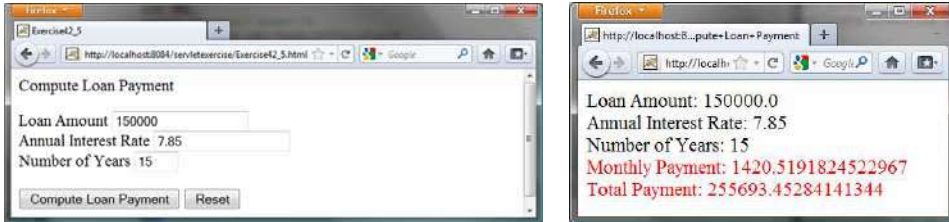
(b)

Şekil 42.30

*Servlet vergi hesaplar.*

42.5\*

(*Kredi hesaplama*) Kullanıcının kredi miktarını, faiz oranını ve yıl sayısını girmesini isteyen bir HTML formu yazın, Şekil'de gösterildiği gibi 42.31a. *Kredi Ödeme Hesaplama* butonuna tıkladığınızda, aylık ve toplam kredi ödemelerini hesaplamak ve görüntülemek için bir servlet çağırılır, Şekil 42.31b'de gösterildiği gibi. Aylık ve toplam ödemeleri hesaplamak için Liste 10.2'de verilen Kredi sınıfını kullanın, Loan.java.



(a)

(b)

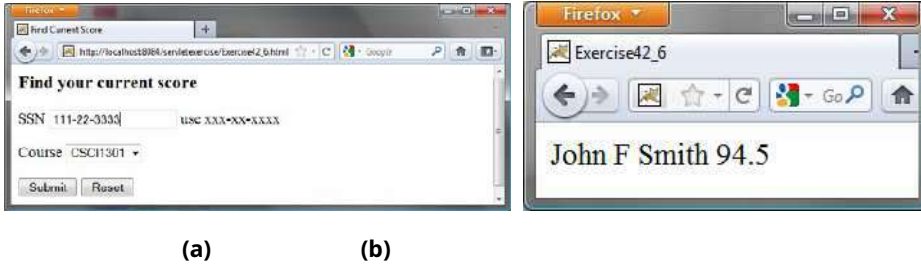
Şekil 42.31

*Servlet kredi ödemesini hesaplar.*

42.6\*\*

(Metin dosyalarından notları bul) SSN ve sınıf kimliği verildiğinde, öğrencinin adını ve mevcut notunu görüntüleyen bir servlet yazın. Her sınıf için, öğrenci adı, SSN ve mevcut notu saklamak için bir metin dosyası kullanılır. Dosya, sınıf kimliği ile .txt uzantısı ile adlandırılır. Örneğin, sınıf kimliği csci1301 ise, dosya adı csci1301.txt olur. Her satırın öğrenci adı, SSN ve not içerdiğini varsayalım. Bu üç öge # işareti ile ayrılmıştır. Kullanıcının SSN ve sınıf kimliğini girmesine olanak tanıyan bir HTML formu oluşturun, Şekil 42.32a'da gösterildiği gibi. *Gönder* butonuna tıkladığınızda, sonuç görüntülenir, Şekil 42.32b'de gösterildiği gibi. Eğer SSN veya sınıf kimliği eşleşmiyorsa, bir hata rapor edin. Üç dersin mevcut olduğunu varsayın:

CSCI1301, CSCI1302 ve CSCI3720.



**Şekil 42.32**

*HTML formu, kullanıcıdan SSN ve sınıf kimliğini alır ve bunları puanı elde etmek için servlete gönderir.*

#### Bölüm 42.7

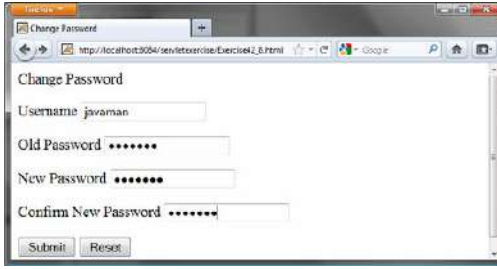
42.7\*\*

(Veritabanı tablolarından puanları bul) Önceki servisi yeniden yazın. Her sınıf için, öğrenci adını, ssn'yi ve puanı saklamak için bir tablo kullanıldığını varsayın. Tablo adı, sınıf kimliği ile aynıdır. Örneğin, sınıf kimliği csci1301 ise, tablo adı csci1301 olacaktır.

42.8\*

(Parolayı değiştir) Kullanıcının bir HTML formundan parolayı değiştirmesine olanak tanıyan bir servlet yazın, Şekil 42.33a'da gösterildiği gibi. Kullanıcı bilgileri, kullanıcı adı, parola ve ad olmak üzere üç sütun içeren Account adlı bir veritabanı tablosunda saklandığını varsayın; burada ad, kullanıcının gerçek adıdır. Servlet aşağıdaki görevleri yerine getirir:

- Kullanıcı adı ve eski parolanın tabloda olduğunu doğrulayın. Eğer değilse, hatayı bildirin ve HTML formunu yeniden gösterin.
- Yeni parola ve onaylanan parolanın aynı olduğunu doğrulayın. Eğer değilse, bu hatayı bildirin ve HTML formunu yeniden gösterin.
- Kullanıcı bilgileri doğru bir şekilde girilmişse, parolayı güncelleyin ve güncellenmenin durumunu kullanıcıya bildirin, Şekil 42.33b'de gösterildiği gibi.



(a)



(b)

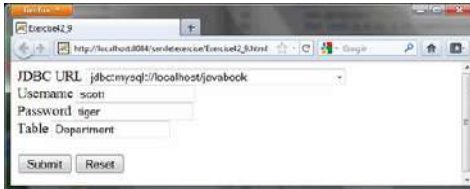
Şekil 42.33

*Kullanıcı, kullanıcı adını ve eski parolayı girer ve yeni bir parola belirler. Servlet, güncellemenin durumunu*

*kullanıcıya bildirir.*

42.9\*\*

(Veritabanı tablolarını görüntüle) Kullanıcıdan bir JDBC sürücüsü, veritabanı URL'si, kullanıcı adı, parola ve tablo adı girmesini veya seçmesini isteyen bir HTML formu yazın; Şekil 42.34a'da gösterildiği gibi. Gönder butonuna tıklamak, tablo içeriğini görüntüler; Şekil 42.34b'de gösterildiği gibi.



(a)

deptId	name	chairId	collegeld
ACCT	Accounting	333115555	BUSS
BIOL	Biology	111225555	SC
CHEM	Chemistry	111225555	SC
CS	Computer Science	111221115	SC
EDUC	Education	333114444	EDUC
MATH	Mathematics	111221116	SC

(b)

Şekil 42.34

*Kullanıcı veritabanı bilgilerini girer ve içeriğini görüntülemek için bir tablo belirtir.*

## Bölüm 42.8

42.10\*

(Çerezleri sakla) Bir tarayıcıda aşağıdaki çerezleri saklayan bir servlet yazın ve maksimum yaşlarını iki gün olarak ayarlayın.

Çerez 1: adı "color" ve değeri kırmızı.

Çerez 2: adı "radius" ve değeri 5.5.

Çerez 3: adı "count" ve değeri 2.

42.11\*

(Çerezleri al) Tüm çerezleri istemcide görüntüleyen bir servlet yazın. İstemci, tarayıcı dan servletin URL'sini yazarak tarayıcıda saklanan tüm çerezleri görüntüler. Şekil 42.35'e bakın.



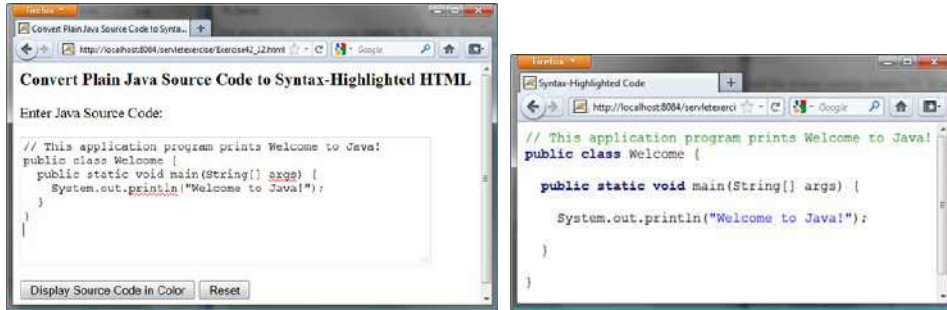
Şekil 42.35

İstemcideki tüm çerezler tarayıcıda görüntülenir.

## Kapsamlı

42.12\*\*\*

(Sözdizimi vurgulama) Kullanıcının bir metin alanına bir Java programı girmesini isteyen bir HTML formu oluşturun; bu, Şekil 42.36a'da gösterilmiştir. Form, Java kaynak kodunu sözdizimi vurgulanmış HTML formatında görüntüleyen bir servleti çağırır; bu, Şekil 42.36b'de gösterilmiştir. Anahtar kelimeler, yorumlar ve literaller sırasıyla kalın lacivert, yeşil ve mavi renkte görüntülenir.



(a)

(b)

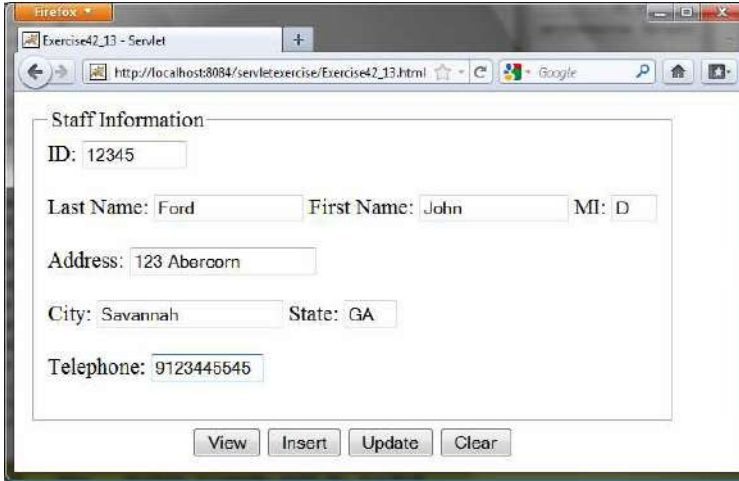
Şekil 42.36

(a) içindeki düz metin Java kodu, (b) içinde sözdizimi vurgulanmış olarak HTML'de görüntülenir.

42.13\*\*

(Bir Personel tablosuna erişim ve güncelleme) Şekil 42.37'de gösterildiği gibi, Egzersiz 33.1 için bir Java servleti yazın.





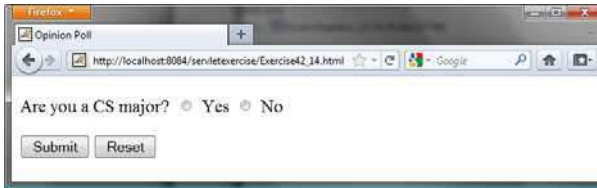
**Şekil 42.37**

*Web sayfası, personel bilgilerini görüntülemenizi, eklemenizi ve güncellenizi sağlar.*

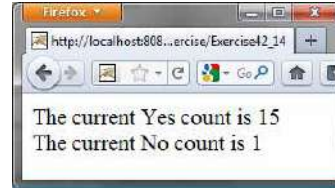
42.14\*\*\*

(Anket) Kullanıcıdan "Bir CS ana dalı mısınız?" gibi bir soruya yanıt vermesini isteyen bir HTML formu oluşturun, Şekil 42.38a'da gösterildiği gibi.

Gönder butonuna tıklandığında, servlet veritabanındaki Evet veya Hayır sayısını artırır ve mevcut Evet ve Hayır sayılarını gösterir, Şekil 42.38b'de gösterildiği gibi.



(a)



(b)

**Şekil 42.38**

*HTML formu, (a) içindeki bir soru için kullanıcıdan Evet veya Hayır girmesini ister ve servlet Evet veya Hayır sayılarını günceller (b).*

Poll adında bir tablo oluşturun, aşağıdaki gibi:

```
tablo oluştur Poll (
 soru varchar(40) birincil anahtar,
 evetSayısı int,
 hayırSayısı int);
```

Tabloya bir satır ekleyin, aşağıdaki gibi:

```
Poll değerlerine ekle ('Sen bir CS ana dalı mısınız?', 0, 0);
```

**\*\*\*Bu bir bonus Web bölümü**

## **BÖLÜM 43**

### **JavaServer Sayfaları**

#### Amaçlar

- Basit bir JSP sayfası oluşturmak için (§43.2).
- Bir JSP sayfasının nasıl işlendiğini açıklamak için (§43.3).
- JSP betiklerini kodlamak için JSP yapıları kullanmak (§43.4).
- JSP'de önceden tanımlanmış değişkenler ve direktifler kullanmak (§§43.5–43.6).
- JSP'de JavaBeans bileşenlerini kullanmak (§43.7).
- JSP'de JavaBeans özelliklerini almak ve ayarlamak (§43.8).
- JavaBeans özelliklerini giriş parametreleri ile ilişkilendirmek (§43.9).
- Bir JSP sayfasından diğerine istekleri yönlendirmek (§43.10).
- JSP kullanarak veritabanı tablolarını taramak için bir uygulama geliştirmek (§43.11) .

### 43.1 Giriş

Servlet'ler dinamik Web içeriği oluşturmak için kullanılabilir. Ancak, bir dezavantaj ı, HTML etiketlerini ve metinlerini Java kaynak kodunun içine yerleştirmeniz gerektiğidir. Servlet'leri kullanarak, HTML metninde değişiklik yapıldığında Java kaynak kodunu değiştirmeniz ve yeniden derlemeniz gerekir. Eğer bir servlette çok fazla HTML betiği varsa, program okunması ve bakımı zor hale gelir, çünkü HTML metni Java programının bir parçasıdır. Bu dezavantajı gidermek için JavaServer Pages (JSP) tanıtılmıştır. JSP, normal şekilde düzenli HTML betiği yazmanıza ve dinamik içerik üretmek için Java kodu yerleştirmenize olanak tanır.

### 43.2 Basit Bir JSP Sayfası Oluşturma

#### <Yan Not: JSP etiketi>

JSP, dinamik Web sayfaları oluşturmanın ve Web uygulamaları inşa etme görevini basitleştirmenin kolay bir yolunu sağlar. Bir JavaServer sayfası, dinamik içerik oluşturmayı sağlayan özel etiketler, yani *JSP etiketleri* ile birlikte normal bir HTML sayfası gibidir. HTML script ile bir Web sayfası oluşturabilir ve dinamik içerik oluşturmak için Java kodunu JSP etiketleri içinde kapsayabilirsiniz. İşte basit bir JSP sayfasının örneği.

#### <Yan Not: JSP etiketi>

```
<!-- CurrentTime.jsp -->
```

```
<html><h
```

```
ead>ve<t
```

```
itle> içinde
```

```
Geçerli Zaman
```

```
</title>ve <
```

```
/head> ve
```

```
<body>
```

```
Geçerli zaman <%= new java.util.Date () %>
```

```
</body>
```

```
</html>
```

Dinamik içerik, <%= ile başlayan ve %> ile biten etiketin içinde yer alır. Geçerli zaman, java.util.Date sınıfının bir nesnesinin toString yöntemini çağırarak bir dize olarak döndürülür.

#### <Yan Not: NetBeans'te JSP>

NetBeans gibi bir IDE, JSP geliştirme görevini büyük ölçüde basitleştirebilir. NetBeans'te JSP oluşturmak için önce bir Web projesi oluşturmanız gerekir. Önceki bölümde liangweb adında bir Web projesi oluşturulmuştur. Kolaylık olması açısından, bu bölümde JSP'yi liangweb projesinde oluşturacağız.

CurrentTime.jsp'yi oluşturma ve çalıştırma adımları şunlardır:

1. Proje panelinde liangweb düğmesine sağ tıklayın ve Yeni'yi seçin  
> **JSP** Yeni JSP iletişim kutusunu görüntülemek için, Şekil 43.1'de gösterildiği gibi.

#### <Yan Not: CurrentTime.jsp oluşturun>

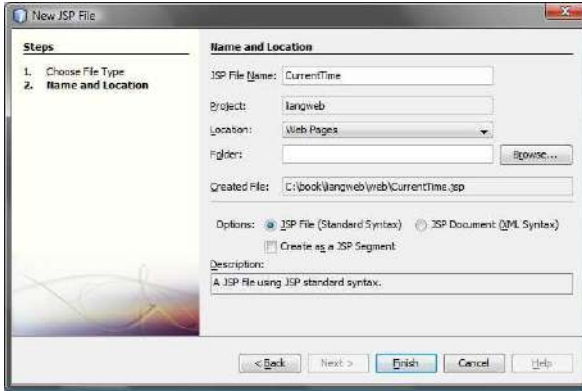
2. JSP Dosya Adı alanına CurrentTime yazın ve *Tamam'a* tıklayın.

CurrentTime.jsp'nin liangweb'deki Web Sayfaları düğümü altında görüldüğünü göreceksiniz.

3. Şekil 43.2'de gösterildiği gibi CurrentTime.jsp için kodu tamamlayın.

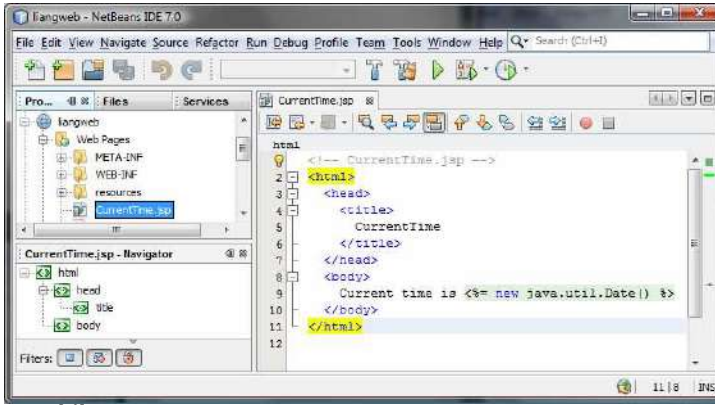
#### <Yan Not: CurrentTime.jsp'yi çalıştırın>

4. Proje panelinde CurrentTime.jsp'ye sağ tıklayın ve Çalıştır'ı seçin  
Dosya. Web tarayıcısında, Şekil 43.3'te gösterildiği gibi JSP sayfasının görüntülendiğini göreceksiniz.



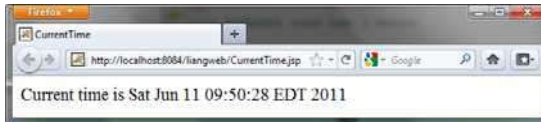
**Şekil 43.1**

NetBeans kullanarak bir JSP sayfası oluşturabilirsiniz.



**Şekil 43.2**

Bir JSP sayfası için bir şablon oluşturulur.



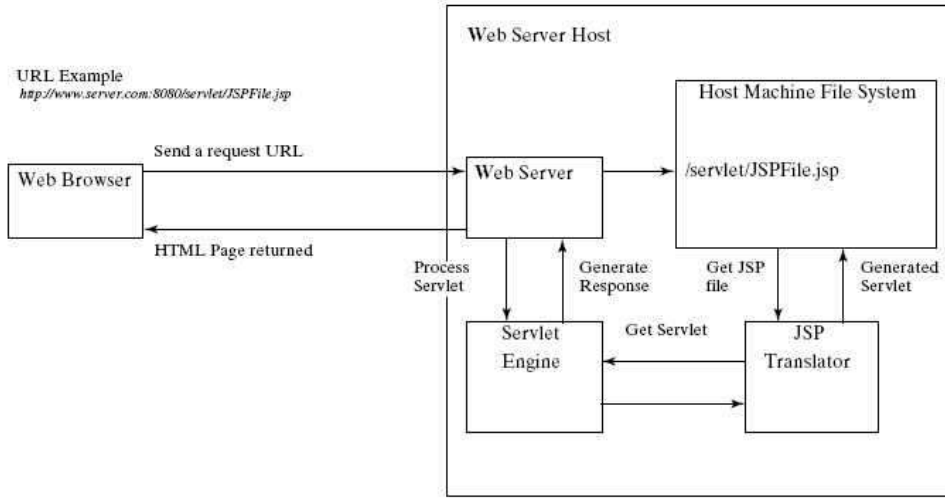
**Şekil 43.3**

Bir JSP sayfasının sonucu bir Web tarayıcısında görüntülenir.

NOT: Servletler gibi, NetBeans'te JSP geliştirebilir, bir .war dosyası oluşturabilir ve ardından .war dosyasını Tomcat ve GlassFish gibi bir Java Web sunucusunda dağıtabilirsiniz.

### 43.3 JSP Sayfası Nasıl İşlenir?

Bir JSP sayfası, bir Web sunucusu tarafından işlenmeden önce görüntülenemez. Web sunucusu JSP'yi desteklemeli ve JSP sayfası .jsp uzantılı bir dosyada saklanmalıdır. Web sunucusu, .jsp dosyasını bir Java servlet'ine çevirir, servlet'i derler ve çalıştırır. Çalıştırmanın sonucu, görüntüleme için tarayıcıya gönderilir. Şekil 43.4, bir JSP sayfasının bir Web sunucusu tarafından nasıl işlendiğini göstermektedir.



**Şekil 43.4**

*Bir JSP sayfası bir servlet'e çevrilir.*

NOT: Bir JSP sayfası, sayfa ilk kez istendiğinde bir servlet'e çevrilir. Sayfa değiştirilmediyse yeniden çevrilmez. İlk kez gerçek bir kullanıcının gecikme ile karşılaşmamasını sağlamak için, JSP geliştiricileri sayfa kurulduktan sonra test etmelidir.

#### 43.4 JSP Betik Yapıları

**<Yan Not: betik ögesi>**

**<Yan Not: yönerge>**

**<Yan Not: eylem>**

JSP yapılarının üç ana türü vardır: betik yapıları, yönergeler ve eylemler. *Betik* ögeleri, Java kodunu sonuçta oluşacak servletin bir parçası haline getirmek için belirtmenizi sağlar. *Yönergeler* sonuçta oluşacak servletin genel yapısını kontrol etmenizi sağlar. *Eylemler* JSP motorunun davranışını kontrol etmenizi sağlar. Bu bölüm betik yapıları tanıtmaktadır.

Java kodunu sonuçta oluşacak bir servlet içine yerleştirmek için kullanılacak üç tür JSP betik yapısı vardır: ifadeler, scriptlet'ler ve bildirimler.

**<Yan Not: JSP ifadesi>**

Bir JSP *ifadesi* doğrudan çıktıya bir Java ifadesi eklemek için kullanılır. Aşağıdaki biçime sahiptir:

**<%= Java ifadesi %>**

İfade değerlendirilir, bir dizeye dönüştürülür ve servletin çıkış akışına gönderilir.

**<Yan Not: JSP scriptlet'i>**

Bir JSP *scriptlet* size bir Java ifadesini servlet'in `jspService` yöntemine eklemenizi sağlar, bu yöntem service yöntemi tarafından çağrılır. Bir JSP scriptlet aşağıdaki biçime sahiptir:

**<% Java ifadesi %>**

**<Yan Not: JSP beyanı>**

Bir JSP *bildirimi* servlet'e yöntemler veya alanlar tanımlamak için kullanılır. Aşağıdaki biçime sahiptir:

**<%! Java bildirimi %>**

HTML yorumları aşağıdaki biçime sahiptir:

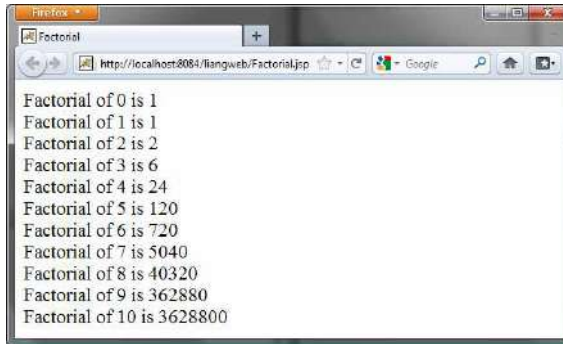
**<!-- HTML Yorum -->**

**<Yan Not: JSP yorumu>**

Eğer yorumun sonuç HTML dosyasında görünmesini istemiyorsanız, JSP'de aşağıdaki yorumu kullanın:

**<%-- JSP Yorumu --%>**

Liste 43.1, 0'dan 10'a kadar olan sayılar için faktöriyel hesaplayan bir JavaServer sayfası oluşturur, Şekil 43.5'te gösterildiği gibi.



**Şekil 43.5**

*JSP sayfası faktöriyel hesaplarını gösterir.*

**Liste 43.1 Factorial.jsp**

**<Yan Not 9. satır: JSP scriptlet>**

**<Yan Not 10. satır: JSP ifadesi>**

**<Yan Not 14. satır: JSP beyanı>**

```
<html><head> ve <title> içinde Faktöriyel</title></head><gövde>

<% for (int i = 0; i <= 10; i++) { %>
 Faktöriyel <%= i %> şudur
 <%=t computeFactorial(i) t %>
n <%t } t %>

<%! özel longt computeFactory(tintt n) {
```

```

 eğer (n == 0)
 döndür 1;
 aksi takdirde
 return n * computeFactorial(n - 1);
 }
}

</body>
</html>

```

JSP scriptlet'leri <% ve %> arasında yer alır. Bu nedenle \_\_

```
for (int i = 0; i <= 10; i++) {, (line 9)
```

bir scriptlet'tir ve bu nedenle servlet'in

jspService yöntemine doğrudan eklenir.

JSP ifadeleri <%= ve %> arasında yer alır. Bu nedenle \_\_

```
<%= i %>, (line 10)
```

bir ifadedir ve servlet'in çıktı akışına eklenir.

JSP bildirimleri <%! ve %> arasında yer alır. Bu nedenle \_\_

```
<%! özel longt computeFactory(tintt n) { n ...
}
%>
```

bir bildirimdir ve servlet'te yöntemleri veya alanları tanımlar.

9. satır aşağıda gösterilen iki alternatifle değiştirilirse ne farklı olacak? Her ikisi de i yi çalışıyor, ancak önemli bir fark var. (a)'da, i servlet içinde yerel bir değişkenken, (b) 'de i servlet'e çevrildiğinde bir örnek değişkenidir.

```
<% intt i = t 0;t %>n
<% fort (; i <= t10;t i++) {t %>
```

(a)

```
<%!t int i; %>
<% fort (i = t 0;t i <= t10;t i++) t{ %>
```

(b)

DİKKAT: JSP için döngü gövdesi, tek bir ifade içerse bile, süslü parantezler içine yerleştirilmelidir. 9. satırdaki açılış süslü parantezi ({) ve 12. satırdaki kapanış süslü parantezi (<% } %>) silmek yanlış olur.

DİKKAT: JSP ifadesinin sonunda noktalı virgül yoktur. Örneğin, <%= i; %> yanlıştır. Ancak, bir JSP scriptlet'inde her Java ifadesi iç in bir noktalı virgül olmalıdır. Örneğin, <% int i = 0 %> yanlıştır.

UYARI: JSP ve Java öğeleri büyük/küçük harf duyarlıdır, ancak HTML değildir.

### 43.5 Ön Tanımlı Değişkenler

<Yan Not: JSP örtük nesnesi>

JSP'de değişkenler kullanabilirsiniz. Kolaylık olması açısından, JSP, JSP ifadeleri ve scriptlet'leri ile kullanılabilecek sekiz ön tanımlı değişken sağlar. Bu değişkenler ayrıca *JSP örtük nesneleri* olarak da bilinir.

<Yan Not: request>

request istemcinin isteğini temsil eder, bu bir HttpServletRequest örneğidir. İstek parametrelerine ve

HTTP başlıklarına, örneğin çerezler ve ana bilgisayar adı gibi, erişmek için bunu kullanabilirsiniz.

**<Yan Not: response>**

response servletin yanıtını temsil eder, bu bir HttpServletResponse örneğidir. Yanıt türünü ayarlamak ve

çıktıyı istemciye göndermek için bunu kullanabilirsiniz.

**<Yan Not: out>**

out karakter çıktı akışını temsil eder, bu response.getWriter() ile elde edilen bir Print Writer örneğidir. Karakter içeriğini istemciye göndermek için bunu kullanabilirsiniz

**<Yan Açıklama: oturum>**

session istekle ilişkili HttpSession nesnesini temsil eder, request.getSession() ile elde edilir.

**<Yan Açıklama: uygulama>**

application tüm istemciler için kalıcı verileri depolamak üzere ServletContext nesnesini temsil eder. session ile application arasındaki fark, session'ın bir istemciye bağlı olması, ancak application'ın tüm istemcilerin kalıcı verileri paylaşmasıdır.

**<Yan Açıklama: yapılandırma>**

config sayfanın ServletConfig nesnesini temsil eder.

**<Yan Açıklama: pageContext>**

pageContext PageContext nesnesini temsil eder. PageContext, birçok sayfa niteliğine merkezi bir erişim noktası sağlamak için JSP'de tanıtilen yeni bir sınıftır.

**<Yan Açıklama: sayfa>**

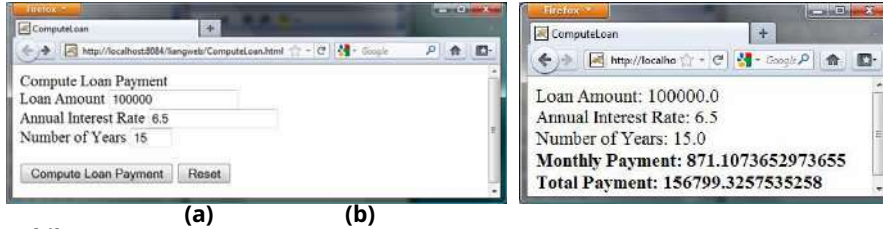
page bunun bir alternatiftir.

Örnek olarak, kullanıcıdan

kredi miktarını, yıllık faiz oranını ve yıl sayısını girmesini isteyen bir HTML sayfası yazalım.

Şekil 43.6a'da gösterildiği gibi. *Kredi Ödemesini Hesapla* butonuna tıklamak, aylık ve toplam kredi ödemelerini hesaplamak ve görüntülemek için bir JSP'yi çağırır,

Şekil 43.6b'de gösterildiği gibi.



Şekil 43.6

JSP, kredi ödemelerini hesaplar.

HTML dosyasının adı ComputeLoan.html (Liste 43.2) olarak adlandırılmıştır. JSP dosyasının adı ComputeLoan.jsp (Liste 43.3) olarak adlandırılmıştır.

### Liste 43.2 ComputeLoan.html

**<Yan Not 7. satır: form eylemi>**

**<Yan Not 10. satır: metin alanı>**

**<Yan Not 15. satır: gönder>**

```
<!-- ComputeLoan.html -->
<html>
<kafa>
<title>KrediHesapla</title>
```



```

</head>
<gövde>
 <form method = "get" action = "ComputeLoan.jsp">
 Kredi Ödeme Hesapla

 Kredi Miktarı
 <input type = "text" name = "loanAmount"/>

 Yıllık Faiz Oranı
 <input type = "text" name = "yillikFaizOranı"/>

 Yıl Sayısı
 <input type = "text" name = "yilSayısı" size = "3"/>

 <p><input type = "submit" name = "Gönder"
 value = "Kredi Ödemesini Hesapla"/>
 <input type = "reset" value = "Sıfırla"/></p>
 </form>
</body> için
de</html>

```

Liste 43.3 ComputeLoan.jsp

<Side Remark line 7: JSP scriptlet>

<Side Remark line 8: parametreleri al>

<Side Remark line 17: JSP ifadesi>

```

<!-- ComputeLoan.jsp -->
<html>
<kafa>
 <title>KrediHesapla</title>
</head>
<gövde>
 <% double krediMiktarı = Double.parseDouble(
 request.getParameter("krediMiktarı"));
 double yıllıkFaizOranı = Double.parseDouble(
 request.getParameter("yillikFaizOranı"));
 double yılSayısı = Integer.parseInt(
 request.getParameter("yilSayısı"));
 double aylıkFaizOranı = yıllıkFaizOranı / 1200;
 double aylıkÖdeme = krediMiktarı * aylıkFaizOranı /
 (1 - 1 / Math.pow(1 + aylıkFaizOranı, yılSayısı * 12));
 double toplamÖdeme = aylıkÖdeme * yılSayısı * 12; %>
 Kredi Miktarı: <%= krediMiktarı %>

 Yıllık Faiz Oranı: <%= annualInterestRate %>

 Yıl Sayısı: <%= numberOfYears %>

 Aylık Ödeme: <%= monthlyPayment %>

 Toplam Ödeme: <%= totalPayment %>

</body>
</html>

```

ComputeLoan.html, kullanıcının kredi miktarını, yıllık faiz oranını ve yıl sayısını girmesi için ilk olarak görüntülenir. Bu dosya herhangi bir JSP ögesi içermediğinden, normal bir HTML dosyası olarak .html uzantısıyla adlandırılmıştır.

ComputeLoan.jsp, HTML formundaki *Kredi Ödeme Hesapla* butonuna tıklandığında çağrılır. JSP sayfası, 7-12. satırlarda tanımlı değişken request kullanarak parametre değerlerini alır ve 13-16. satırlarda aylık ödeme ve toplam ödemeyi hesaplar. Aylık ödemenin hesaplanması için formül \$2.9, "Problem: Kredi Ödemelerini Hesaplama."

JSP scriptlet <% 7. satırda JSP declaration <%! ile değiştirilirse ne yanlış olur? Önceden tanımlı değişkenler (örneğin, request, response, out) servlet yöntemleri doGet ve doPost'ta tanımlanan yerel değişkenlere karşılık gelir. Bunlar JSP scriptlet'lerinde görünmelidir, JSP deklarasyonlarında değil.

İPUCU: ComputeLoan.jsp, aşağıdaki

sorgu dizesi kullanılarak da çağrılabilir:

<http://localhost:8084/liangweb/ComputeLoan.jsp?loanAmount=10000&annualInterestRate=6&numberOfYears=15>.

### 43.6 JSP Yönergeleri

Bir JSP yönergesi, JSP motoruna JSP sayfası hakkında bilgi veren bir ifadedir. Örneğin, JSP sayfanız java.lang paketinden başka bir paketten bir Java sınıfı kullanıyorsa, bu paketi içe aktarmak için bir yönerge kullanmalısınız. Bir JSP yönergesinin genel sözdizimi aşağıda gösterilmiştir:

```
<%@ yönerge nitelik = "değer"%>, veya
<%@ yönerge nitelik1= "değer1"
 nitelik2= "değer2"
 ...
 nitelikn= "değern"%>
```

Olası yönergeler şunlardır:

- **sayfa** sayfa için sınıfları içe aktarma ve içerik türünü ayarlama gibi bilgiler sağlamaya olanak tanır. Sayfa yönergesi JSP dosyasında herhangi bir yerde görünebilir.
- **include** sayfa servlet'e çevrildiğinde bir dosyayı servlete eklemenizi sağlar. Include yönergesi, dosyanın eklenmesini istediğiniz yere yerleştirilmelidir.
- **taglib** özel etiketler tanımlamanıza olanak tanır.

Aşağıdakiler, sayfa yönergesi için yararlı niteliklerdir:

- **import** bu sayfa için içe aktarılacak bir veya daha fazla paketi belirtir. Örneğin, <%@ page import="java.util.\*, java.text.\*" %> java.util.\* ve java.text.\*'yi içe aktarır.
- **contentType** sonuç JSP sayfası için içerik türünü belirtir. Varsayılan olarak, içerik türü JSP için text/html'dir. Servisler için varsayılan içerik türü text/plain'dir.
- **oturum** sayfanın oturumun bir parçası olup olmadığını belirtmek için bir boolean değeri tanımlar. Varsayılan olarak, oturum doğrudur.
- **buffer** çıkış akışı tampon boyutunu belirtir. Varsayılan olarak, bu 8KB'dir. Örneğin, <%@ page buffer="10KB" %> direktifi çıkış tampon boyutunun 10KB olduğunu belirtir. <%@ page buffer="none" %> yönergesi bir tampon kullanılmadığını belirtir.
- **autoFlush** bir boolean değerini belirtir, bu değer çıktının buffer'ının dolduğunda otomatik olarak boşaltılıp boşaltılmayacağını veya buffer taşarsa bir istisna mı oluşturulacağını gösterir. By varsayılan, bu özellik doğrudur. Bu durumda, tampon özelliği boş olamaz.
- **isThreadSafe** bir boolean değerini belirtir, bu değer sayfanın aynı anda erişilebilirliğini ve veri bozulması olmadan erişilip erişilemeyeceğini gösterir. Varsayılan olarak, bu doğrudur. Eğer false olarak ayarlanırsa, JSP sayfası Single ThreadModel arayüzünü uygulayan bir servlet'e dönüştürülecektir.
- **errorPage** mevcut sayfada bir istisna meydana geldiğinde işlenen bir JSP sayfasını belirtir. Örneğin, <%@ page direktifi

errorPage="HandleError.jsp" %> bir istisna meydana geldiğinde HandleError.jsp'nin işlendiğini belirtir.

- **isErrorPage** sayfanın bir hata sayfası olarak kullanılıp kullanılmayacağını belirtmek için bir boolean değerini tanımlar. Varsayılan olarak, bu özellik false'dur.

Liste 43.4, bir sınıfı içe aktarmak için sayfa direktifinin nasıl kullanılacağını gösteren bir örnek vermektedir. Örnek, Liste 10.2'de oluşturulan `Loan` sınıfını kullanarak Liste 43.3, `ComputeLoan.jsp`'yi basitleştirmektedir. `Loan` sınıfının bir nesnesini oluşturabilir ve aylık ödeme ve toplam ödemeyi hesaplamak için `monthlyPayment()` ve `totalPayment()` yöntemlerini kullanabilirsiniz.

#### Liste 43.4 ComputeLoan1.jsp

<Yan Not satır 7: JSP direktifi>

<Yan Not satır 14: nesne oluştur>

```
<!-- ComputeLoan1.jsp -->
<html>
 <kafa>
 <title>Loan Sınıfını Kullanarak Kredi Hesaplama</title>
 </head>
 <tbody>
 <%@ page importt = t "chapter43.Loan" t %>n<
 % doublet CreditAmount = Double.parseDouble(n re
 quest.getParameter(t"loanAmount"t));ndoubl
 et AnnualInterestRate = Double.parseDouble(n request.get
 Parameter(t"annualInterestRate"t));nintt numberOfYea
 rs = Integer.parseInt(
 request.getParameter(t"numberOfYears"t));n Kre
 di kredisi =
 yeni Loan(yıllıkFaizOranı, yılSayısı, krediMiktarı);
 %>
 Kredi Miktarı: <%= loanAmount %>

 Yıllık Faiz Oranı: <%= annualInterestRate %>

 Yıl Sayısı: <%=numberOfYears%>

 Aylık Ödeme: <%= loan.getMonthlyPayment() %>

 Toplam Ödeme: <%= loan.getTotalPayment() %>

 </body>
 </html>
```

<Yan Not: Kredi sınıfı oluştur>

Bu JSP, Kredi sınıfını kullanır. Sınıfı liangweb projesinde chapter43 paketinde aşağıdaki gibi oluşturmalısınız:

```
paket chapter43;

public class Kredi {
 // Listing 10.2, Loan.java'daki 2-69. satırlarla aynı, bu yüzden atlandı
}
```

Yönerge `<%@ page import = "chapter43.Kredi" %>` Kredi sınıfını 7. satırda içe aktarır. 14. satır, verilen kredi miktarı, yıllık faiz oranı ve yıl sayısı için bir Kredi nesnesi oluşturur. 20-21. satırlar, Kredi nesnesinin `monthlyPayment()` ve `totalPayment()` yöntemlerini çağırarak aylık ödeme ve toplam ödemeyi görüntüler.

#### 43.7 JSP'de JavaBeans Kullanımı

Normalde bir programda bir sınıfın bir örneğini oluşturur ve onu o programda kullanırsınız. Bu yöntem, nesneyi değil, sınıfı paylaşmak içindir. JSP, bir sınıfın nesnesini farklı sayfalar arasında paylaşmanıza olanak tanır. Bir nesnenin paylaşılabilmesi için, sınıfının bir JavaBeans bileşeni olması gerekir.

Şunu hatırlayın ki bu, aşağıdaki üç özelliği içerir:

- Sınıf kamuya açıktır.
- Sınıfın argüman içermeyen bir kamuya açık yapıcı (constructor) vardır.
- Sınıf serileştirilebilir. (Bu gereklilik

JSP'de gerekli değildir.)

JavaBeans bileşeni için bir örnek oluşturmak için aşağıdaki

sözdizimini kullanın:

```
<jsp:useBean id = t "objectName" tscope = t "scopeAttribute" nclass = t "SınıfAdı" t />
```

Bu sözdizimi kabaca şuna eşdeğerdir

```
<% ClassName objectName = new ClassName() %>
```

ancak kapsam (scope) niteliği eksiktir. Kapsam niteliği nesnenin kapsamını belirtir ve nesne zaten kapsam içindeyse yeniden oluşturulmaz. Aşağıda kapsam niteliği için dört olası değer listelenmiştir:

- uygulama nesnenin uygulamaya bağlı olduğunu belirtir.  
Nesne, uygulamanın tüm oturumları tarafından paylaşılabilir.
- oturum nesnenin istemcinin oturumuna bağlı olduğunu belirtir. Bir istemcinin oturumu, bir Web tarayıcısı ile bir Web sunucusu arasında otomatik olarak oluşturulur. Aynı tarayıcıdan bir istemci, aynı sunucudaki iki servlet veya iki JSP sayfasına eriştiğinde, oturum aynıdır.
- sayfa varsayılan kapsamdır, bu nesnenin sayfaya bağlı olduğunu belirtir.
- istek nesnenin istemciye bağlı olduğunu belirtir.

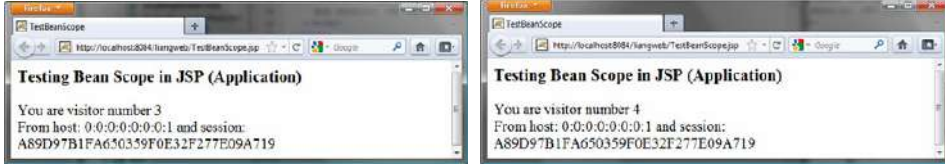
JSP motoru `<jsp:useBean id="objectName" scope="scopeAttribute" class="ClassName" />` işlendiğinde, önce aynı id ve kapsamda bir nesne arar. Bulunursa, mevcut bean kullanılır; aksi takdirde, yeni bir bean oluşturulur.

Bir bean oluşturmanın başka bir sözdizimi:

```
<jsp:useBean id = t "objectName" tscope = t "scopeAttribute" nclass = t "ClassName" t >
 ifadeler
</jsp:useBean>
```

İfadeler bean oluşturulduğunda yürütülür. Eğer aynı ID ve sınıf adına sahip bir bean zaten kapsamda varsa, ifadeler yürütülmez.

Liste 43.5, Count adında bir JavaBeans bileşeni oluşturur ve bunu bir JSP sayfasına yapılan ziyaretlerin sayısını hesaplamak için kullanır; Şekil 43.7'de gösterildiği gibi.



**Şekil 43.7**  
Sayfa ziyaret edildiğinde, sayfanın ziyaret sayısı artar.

#### Liste 43.5 Count.java

<Yan Not 1. satır: paket beyanı>

**paket** bölüm43;

**genel sınıf** Sayim{  
**özel int** sayim = 0;

**/\*\* Sayım özelliğini döndür \*/**

**kam int** getCount() {  
**return** sayim;

**/\*\* Sayıyı artır \*/**

**kamu boşluğu** artıřSayısı() {n sayısı+  
+;

}

TestBeanScope.jsp adlı JSP sayfası, Listeleme  
43.6'da oluşturulmuştur.

#### Listeleme 43.6 TestBeanScope.jsp

<Yan Not satır 2: içe aktarma direktifi>

<Yan Not satır 3: bean oluştur>

<Yan Not satır 12: bean kullan>

<Yan Not satır 14: istek>

<Yan Not satır 15: oturum>

```
<!-- TestBeanScope.jsp -->
<%@ sayfa içe aktar = "chapter43.Count"%>
<jsp:useBean id = "count" scope = "application"
 class = "chapter43.Count">
</jsp:useBean>
<html>
<kafa>
<başlık>TestBeanScope</başlık>
</head>
<gövde>
<h3>JSP'de Bean Kapsamını Test Etme (Uygulama)</h3>
<% count.increaseCount(); %>
Ziyaretçi numaranız <%= count.getCount() %>

Host'tan: <%= request.getRemoteHost() %>
ve oturum: <%= session.getId() %>
</body>
</html>
```

scope niteliği, bean'in kapsamını belirtir. scope="application"(3. satır), bean'in JSP motorunda canlı olduğunu ve tüm istemcilerin erişimi için mevcut olduğunu belirtir. Bean, <jsp:useBean id="count" scope="application"class="Count" > (3-4. satırlar) direkt ifi ile herhangi bir istemci tarafından paylaşılabilir. TestBeanScope.jsp'ye erişen her istemci, count'un 1 artmasına neden olur. İlk istemci, count nesnesinin yaratılmasına neden olur ve TestBeanScope'a sonraki erişimler aynı esneyi kullanır.

Eğer scope="application" scope="session" olarak değiştirilirse, bean'in kapsamı aynı tarayıcıdan gelen oturumla sınırlıdır. Sayfa yalnızca aynı tarayıcıdan talep edilirse sayım artacaktır. Eğer scope="application" scope="page" olarak değiştirilirse, bean'in kapsamı sayfayla sınırlıdır ve başka bir sayfa bu bean'e erişemez. Sayfa her zaman 1 sayımını gösterecektir. Eğer scope="application" scope="request" olarak değiştirilirse, bean'in kapsamı istemcinin isteğiyle sınırlıdır ve sayfadaki diğer istekler her zaman 1 sayımını gösterecektir.

Sayfa yok edilirse, sayım 0'dan yeniden başlar. Sorunu, sayımı rastgele erişim dosyasında veya bir veritabanı tablosunda saklayarak çözebilirsiniz. Sayımı bir veritabanındaki Count tablosunda sakladığınızı varsayın.

Count sınıfı, Liste 43.7'de değiştirilebilir.

#### Liste 43.7 Count.java (Gözden Geçirilmiş Versiyon)

<Side Remark line 1: paket ifadesi>

<Side Remark line 16: SQL'i çalıştır>

<Side Remark line 43: sürücüyü yükle>

<Side Remark line 46: bağlantı>

<Side Remark line 49: ifade>

**paket bölüm43;**

java.sql.\*'ı içe aktarın;

**genel sınıf Count {n özel**

int sayısı = 0;n özel Açıklam

a bildirimi = null;

public Count() {ve Jdbc();n'yi başlatın }

/\*\* Sayı özelliğini döndür \*/

public int getCount() {

try {

**ResultSet rset =statement.executeQueryn ("Count'tan countValue'yu seçin");n rset.next();**

sayım = rset.getInt(1);n }

catch (Exception ex) {n ex.printStackTrace();n }

dönüş sayısı;n }

/\*\* Sayıyı artır \*/

```

 genel geçersiz artışSayısı() {n sayısı+
 +;
 denemek {
 statement.executeUpdate(n "
 güncelleme Sayımı set countValue = " + count);n }

 catch (İstisna örneği) {
 ex.printStackTrace();
 }
 }

 /** Veritabanı bağlantısını başlat */
 public void initializeJdbc() {
 denemek {
 Class.forName("com.mysql.jdbc.Driver");

 // Örnek veritabanına bağlan
 Connection connection = DriverManager.getConnection
 ("jdbc:mysql://localhost/javabook" , "scott", "tiger");

 bildirim = bağlantı.createStatement();n }

 catch (Exception ex) {n ex.p
 rintStackTrace();n }

 }
}
N}

```

### 43.8 Özellikleri Alma ve Ayarlama

Geleneksel olarak, bir JavaBeans bileşeni özel özelliklerini okumak ve değiştirmek için get ve set yöntemlerini sağlar. JSP'de aşağıda gösterilen sözdizimini kullanarak özelliği alabilirsiniz:

```
<jsp:getProperty name = "beanId" property = "sample"/>
```

Bu yaklaşık olarak şuna eşdeğerdir

```
<%= beanId.getSample() %>
```

Özelliği JSP'de aşağıdaki sözdizimini kullanarak ayarlayabilirsiniz:

```
<jsp:setProperty name = "beanId"
property = "sample" value = "test1" />
```

Bu şuna eşdeğerdir

```
<% beanId.setSample("test1"); %>
```

### 43.9 Özellikleri Girdi Parametreleri ile İlişkilendirme

Sıklıkla özellikler, giriş parametreleri ile ilişkilendirilir. Diyelim ki score adında bir giriş parametresinin değerini almak ve bunu score adındaki JavaBeans özelliğine ayarlamak istiyorsunuz. Aşağıdaki kodu yazabilirsiniz:

```
<% double score = Double.parseDouble(
 request.getParameter("score")); %>
<jsp:setProperty name = "beanId" property = "score"
value = "<%= score %>" />
```

Bu zahmetlidir. JSP, bunu basitleştirmek için kullanılabilecek uygun bir sözdizimi sağlar:

```
<jsp:setProperty name t = t "beanId"t propertyt = t "puan"nparam = t "puan"t />
```

Değer niteliğini kullanmak yerine, bir giriş parametresini adlandırmak için param niteliğini kullanırsınız. Bu parametrenin değeri, özelliğe ayarlanır.

NOT: Bir bean özelliği bir giriş parametresi ile ilişkilendirildiğinde basit tür dönüşümü otomatik olarak gerçekleştirilir. Bir dize giriş parametresi, uygun bir ilkel veri türüne veya bir ilkel tür için bir sarmalayıcı sınıfa dönüştürülür. Örneğin, eğer bean özelliği int türündeyseniz, parametrenin değeri int türüne dönüştürülecektir. Eğer bean özelliği Integer türündeyseniz, parametrenin değeri Integer türüne dönüştürülecektir.

Sıklıkla bean özelliği ve parametre aynı adı taşır. Aşağıdaki uygun ifadeyi kullanarak beanId içindeki tüm bean özelliklerini, özellik adlarıyla eşleşen parametrelerle ilişkilendirebilirsiniz:

```
<jsp:setProperty name = "beanId"property = "*" />
```

#### 43.9.1 Örnek: JavaBeans Kullanarak Kredi Ödemelerini Hesaplama B

u örnek, bean özelliklerini girdi parametreleriyle ilişkilendirerek Listing 43.4, ComputeLoan1.jsp'yi basitleştirmek için JavaBeans kullanır. Yeni ComputeLoan2.jsp, Listing 43.8'de verilmiştir.

#### Listeleme 43.8 ComputeLoan2.jsp

<Yan Not satır 7: içe aktar>

<Yan Not satır 8: bean oluştur>

<Yan Not satır 11: bean kullan>

```
<!-- ComputeLoan2.jsp -->
<html>
 <kafa>
 <title>Kredi Sınıfını Kullanarak Kredi Hesaplama</title>
 </head>
 <gövde>
 <%@ page importt = t "chapter43.Loan"t %>n
 <jsp:useBean idt = t "kredi"t classt = t "chapter43.Loan"nscope
 et = t "page"t >t</jsp:useBean>
 <jsp:setProperty name = "loan"property = "*" />
 Kredi Miktarı: <%= loan.getLoanAmount() %>

 Yıllık Faiz Oranı: <%= loan.getAnnualInterestRate() %>

 Yıl Sayısı: <%= loan.getNumberOfYears() %>

 Aylık Ödeme: <%= loan.monthlyPayment() %>

 Toplam Ödeme: <%=loan.totalPayment() %>

 </body>
</html>
```

Satırlar 8-9

```
<jsp:useBean idt = t "kredi"t classt = t "chapter43.Loan"nscope
 et = t "sayfa"t >t</jsp:useBean>
```



Loan sınıfı için loan adında bir bean oluşturur. Satır 10

```
<jsp:setProperty name = "loan" property = "*" />
```

bean özelliklerini loanAmount, annualInterestRate ve numberOfYears ile giriş parametre değerleriyle ilişkilendirir ve tür çevrimini otomatik olarak gerçekleştirir.

Satırlar 11-13, loan bean'in erişim yöntemlerini kullanarak loan amount, annual interest rate ve number of years değerlerini alır.

Bu program, Listings 43.3 ve 43.4'teki ComputeLoan.jsp ve ComputeLoan1.jsp ile aynı şekilde çalışır, ancak kodlama çok daha basitleştirilmiştir.

#### 43.9.2 Örnek: JavaBeans Kullanarak Faktöriyel Hesaplama

Bu örnek, FactorialBean adında bir JavaBeans bileşeni oluşturur ve bunu, FactorialBean.jsp adındaki bir JSP sayfasında bir giriş sayısının faktöriyelini hesaplamak için kullanır; Şekil 43.8'de gösterildiği gibi.



Şekil 43.8

*Bir giriş tam sayısının faktöriyeli, FactorialBean'deki bir yöntem kullanılarak hesaplanır.*

FactorialBean.java adında bir JavaBeans bileşeni oluşturun (Listing 43.9). FactorialBean.jsp oluşturun (Listing 43.10).

#### Liste 43.9 FactorialBean.java

<Yan Not: paket ifadesi>

<Yan Not: alma (get)>

<Yan Not: ayarlama (set)>

```
paket chapter43;
```

```
public class FactorialBean {
 private int number;
```

```
 /** Sayı özelliğini döndür */
```

```
 public int getNumber() {
 return number;
 }
```

```
 /** Sayı özelliğini ayarla */
```

```
 public void setNumber(int newValue) {
 number = newValue;
 }
```

```

 /** Faktöriyel elde et */
 public long getFactorial() {
 long factorial = 1;
 for (int i = 1; i <= sayi; i++)n faktöriyel *= i
 ;
 faktöriyeli döndür;n
 }
}

```

#### Listeleme 43.10 FactorialBean.jsp

<Yan Not satır 2: içe aktar>

<Yan Not satır 3: bean oluştur>

<Yan Not satır 15: form>

<Yan Not satır 21: özellik al>

```

<!-- FactorialBean.jsp -->
<%@ sayfa içe aktar = "chapter43.FactorialBean"%>
<jsp:useBean id="factorialBeanId"
 sınıf = "chapter43.FactorialBean" kapsam = "sayfa">
</jsp:useBean>
<jsp:setProperty name=t "factorialBeanId"t property=t /> n<html>

<head>
 ve <title>
 FaktöriyelBean
 </title>
</head>
<gövde>
 <h3>Bir Bean Kullanarak Faktöriyel Hesapla</h3>
 <form method = "post">
 Yeni değer girin: <input name = "number"/>

 <input type = "submit"name = "Gönder"
 value = "Faktöriyel Hesapla"/>
 <input type = "reset" value = "Sıfırla"/>

 Faktöriyel
 <jsp:getProperty name = "factorialBeanId"
 property = "number"/> dir
 <%@ page import t = "java.text.*"t %>n<
 %t NumberFormat format = NumberFormat.getNumberInstance(); t %>n <
 % = t format.format(factorialBeanId.getFactorial())t %>
 </form> n <
 /body> için
de</html>

```

jsp:useBean etiketi (satırlar 3-4), FactorialBean sınıfının bir bean'i olan factorialBeanId 'yi oluşturur. Satır 5 <jsp:setProperty name="factorialBeanId" property="\*" /> tüm bean özelliklerini aynı adı taşıyan giriş parametreleriyle ilişkilendirir. Bu durumda, bean özelliği number, giriş parametresi number ile ilişkilendirilmiştir. Compute Factorial butonuna tıkladığınızda, JSP otomatik olarak number için giriş değerini string'den int'e dönüştürür ve diğer ifadeler çalıştırılmadan önce factorialBean'e ayarlar.

Satırlar 21-22 <jsp:getProperty name="factorialBeanId" property="number" /> etiketi (satır 21) <%= factorialBeanId.getNumber() %> ile eşdeğerdir.

method `factorialBeanId.getFactorial()` (satır 25), `factorialBeanId`'deki sayı için faktöriyel döndürür.

#### TASARIM REHBERİ

##### <Yan Not: Java kodunu HTML'den ayırma>

Bir JSP sayfasında çok fazla Java kodunu HTML ile karıştırmak, kodu okunmasını ve bakımını zorlaştırır. Java kodunu mümkün olduğunca .java dosyasına taşımalsınız.

##### \*\*\*TASARIM REHBERİ SONU

Önceki tasarım kılavuzunu takip ederek, Java kodunu satır 23-25'te `FactorialBean` sınıfına taşıyarak önceki örneği geliştirebilirsiniz. Yeni `FactorialBean.java` ve `FactorialBean.jsp`, Listeler 43.11 ve 43.12'de verilmiştir.

#### Liste 43.11 NewFactorialBean.java

##### <Yan Not 1: paket ifadesi>

##### <Yan Not 9: alma>

##### <Yan Not 14: ayarlama>

```
paket chapter43;

import java.text.*;

public class NewFactorialBean {
 private int number;

 /** Sayı özelliğini döndür */
 public int getNumber() {
 return number;
 }

 /** Sayı özelliğini ayarla */
 public void setNumber(int newValue) {
 number = newValue;
 }

 /** Faktöriyel elde et */
 public long getFactorial() {
 long factorial = 1;
 for (int i = 1; i <= sayı; i++) factorial *= i;
 faktöriyeli döndür;
 }

 /** Sayıyı biçimlendir */
 public static String format(long number) {
 NumberFormat format = NumberFormat.getNumberInstance();
 return format.format(number);
 }
}
```

#### Liste 43.12 NewFactorialBean.jsp

##### <Yan Not satır 2: içe aktar>

##### <Yan Not satır 3: bean oluştur>

##### <Yan Not satır 15: form>

##### <Yan Not satır 21: özellik al>

```

<!-- NewFactorialBean.jsp -->
<%@ sayfa import = "chapter43.NewFactorialBean"%>
<jsp:useBean id = "factorialBeanId"
 sınıf = "chapter43.NewFactorialBean" kapsam = "sayfa">
</jsp:useBean>
<jsp:setProperty name=t "factorialBeanId" t property=t "t" /> n<html>

 <head>
 ve <title>
 FaktöriyelBean
 </title>
 </head>
 <gövde>
 <h3>Bir Bean Kullanarak Faktöriyel Hesapla</h3>
 <form method = "post">
 Yeni değer girin: <input name = "number"/>

 <input type = "submit" name = "Gönder"
 value = "Faktöriyel Hesapla"/>
 <input type = "reset" value = "Sıfırla"/>

 Faktöriyel
 <jsp:getProperty name = "factorialBeanId"
 property = "number"/> dir
 <%= t NewFactorialBean.format(factorialBeanId.getFactorial())t %>n </form>

 </body> içi
 </html>

```

Bu sayfada bir sorun var. Program büyük faktöriyel değerlerini gösteremiyor. Örneğin, 21 değerini girdiyseniz, program yanlış bir faktöriyel gösterecektir. Bu sorunu düzeltmek için yapmanız gereken tek şey, NewFactorialBean sınıfını BigInteger kullanarak faktöriyel hesaplamak için gözden geçirmektir. Bakınız Egzersiz 43.18.

#### 43.9.3 Örnek: Uluslararası Zamanı Gösterme

Liste 42.5, TimeForm.java, kullanıcının bir yerel ayar ve zaman dilimi belirtmesi için HTML formu oluşturmak üzere doGet yöntemini kullanan bir Java servleti verir (Şekil 42.18a) ve belirtilen zaman diliminde belirtilen yerel ayar için mevcut zamanı göstermek üzere doPost yöntemini kullanır (Şekil 42.18b). Bu bölüm, servleti JSP kullanarak yazmaktadır. Formu göstermek için bir JSP sayfası ve mevcut zamanı göstermek için başka bir JSP sayfası oluşturmalısınız.

ZamanForm.java servlet'inde, allLocale ve allTimeZone dizileri veri alanlarıdır. doGet ve doPost yöntemleri her ikisi de dizileri kullanır. Mevcut yerel ayarlar ve saat dilimleri her iki sayfada da kullanıldığından, tüm mevcut yerel ayarları ve saat dilimlerini içeren bir nesne oluşturmak daha iyidir.

Bu nesne her iki sayfa tarafından paylaşılabilir.

TimeBean.java adında bir JavaBeans bileşeni oluşturalım (Liste 43.13). Bu sınıf, 7. satırda bir dizide tüm mevcut yerel ayarları ve 8. satırda bir dizide tüm saat dilimlerini alır. localeIndex ve timeZoneIndex bean özellikleri (9-10. satırlar) dizideki bir elemana atıfta bulunmak için tanımlanmıştır. currentTimeString() yöntemi (42-52. satırlar) belirtilen yerel ayar ve saat dilimi ile mevcut zaman için bir dize döndürür.

#### Liste 43.13 TimeBean.java

<Yan Not 1. satır: paket beyanı>  
 <Yan Not 7: tüm yerel ayarlar>  
 <Yan Not 8: tüm zaman dilimleri>  
 <Yan Not 9: yerel ayar indeksi>  
 <Yan Not 10: zaman dilimi indeksi>  
 <Yan Not 13: zaman dilimini sırala>  
 <Yan Not 17: tüm yerel ayarları döndür>  
 <Yan Not 21: tüm zaman dilimlerini döndür>  
 <Yan Not 42: mevcut zamanı döndür>

**paket chapter43;**

java.util.\*'i içe aktarın;  
java.text.\*'i içe aktarın;

```
genel sınıf TimeBean {
 özel Locale[] allLocale = Locale.getAvailableLocales();n özel String[] all
 TimeZone = TimeZone.getAvailableIDs();n özel int localeIndex;
 özel int timeZoneIndex;

 public TimeBean() {n A
 rrays.sort(allTimeZone);n }

 public Yerel Ayar[] getAllLocale() {n ret
 urn allLocale;

 }

 public String[] getAllTimeZone() {n retur
 n allTimeZone;

 }

 public int getLocaleIndex() {n retur
 n localeIndex;

 }

 public intl get TimeZone Index() {ve r
 eturn timeZone Index;

 }

 public void setLocaleIndex(int index) {n locale
 Index = index;

 }

 public void setTimeZoneIndex(int index) {n time
 ZoneIndex = index;

 }

 /** Geçerli zaman için bir dize döndür
 * belirtilen yerel ayar ve zaman dilimi ile */
 public String currentTimeString(
 int localeIndex, int timeZoneIndex) {n Takvi
 m takvimi =
 yeni GregorianCalendar(allLocale[localeIndex]);
 Zaman Dilimi timeZone =
 TimeZone.getTimeZone(allTimeZone[timeZoneIndex]);
 Tarih/Saat Biçimleyici dateFormat = DateFormat.getDateInstance(
 DateFormat.FULL, DateFormat.FULL, allLocale[localeIndex]);
 dateFormat.setTimeZone(timeZone);
```

```

 return dateFormat.format(calendar.getTime());n }
 }
}

```

DisplayTimeForm.jsp'yi oluşturun (Liste 43.14). Bu sayfa, Şekil 42.18'a'da gösterilen forma benzer bir formu görüntüler. 2. satır TimeBean sınıfını içe aktarır. 3-5. satırlar da bir bean oluşturulur ve 17, 19, 24 ve 26. satırlarda tüm yerelleri ve zaman dilimlerini döndürmek için kullanılır. Bean'in kapsamı uygulama (satır 4) olduğundan, bean uygulamanın tüm oturumları tarafından paylaşılabilir.

#### Liste 43.14 DisplayTimeForm.jsp

*<Yan Not satır 2: import sınıf>*  
*<Yan Not satır 3: timeBeanId>*  
*<Yan Not satır 15: eylem>*  
*<Yan Not satır 19: tüm yereller>*  
*<Yan Not satır 26: tüm zaman dilimleri>*

```

<!-- DisplayTimeForm.jsp -->
<%@ sayfa import = "chapter43.TimeBean" %>
<jsp:useBean id = "timeBeanId"
 sınıf = "chapter43.TimeBean" kapsam = "application" >
</jsp:useBean>

<html><head>
 <title> içinde
 Zaman Gösterim Formu
 </title>
</head>
<gövde>
<h3>Yerel ayar ve zaman dilimi seçin</h3>
<form method = "post" action = "DisplayTime.jsp">
 Yerel Ayar <select size = "1" name = "localeIndex">
 <% for (int i = 0; i < timeBeanId.getAllLocale().length; i++) {%>
 <option value = "<%= i %>">
 <%=timeBeanId.getAllLocale()[i] t %>n</
 option>
 <%}%>
 </select>

 Zaman Dilimi <select size = "1" name = "timeZoneIndex">
 <% for (int i = 0; i < timeBeanId.getAllTimeZone().length; i++) {%>
 <option value = "<%= i %>">
 <%=timeBeanId.getAllTimeZone()[i] t %>n</
 option>
 <%}%>
 </select>

 <input type = "submit" name = "Gönder"
 value = "Zamanı Al"/>
 <input type = "reset" value = "Sıfırla"/>
 </form>
</body> için
de</html>

```

DisplayTime.jsp'yi oluşturun (Listeleme 43.15). Bu sayfa, belirtilen yerel ayar ve zaman dilimi ile zamanı görüntülemek için DisplayTimeForm.jsp'den çağrılır, tıpkı Şekil 42.18'de olduğu gibi.

### Listeleme 43.15 DisplayTime.jsp

<Yan Not 2. satır: sayfa kodlaması>

<Yan Not 3. satır: import>

<Yan Not 4. satır: timeBeanId>

<Yan Not 7. satır: parametre al>

<Yan Not 18. satır: nesne kullan>

```
<!-- DisplayTime.jsp -->
<%@page page Kodlama = "GB18030"%>
n<%@ page importt = t "chapter43.TimeBean"%
>n<jsp:useBean idt = t "timeBeanId"
 classy = t "bölüm 43.TimeBean"tscopet = t "uygulama"in</jsp:u
seBean>
<jsp:setProperty namet = t "timeBeanId"t propertyt = t "*"t />n<

html>
<kafa>
<başlık>
 Zamanı Göster
</title>
</head>
<gövde>
<h3>Yerel ayar ve zaman dilimi seçin</h3>
 Mevcut zaman
 <%=t timeBeanId.currentTimeString(timeBeanId.getLocaleIndex(),n time
 BeanId.getTimeZoneIndex()) t %>
</body><ht
ml> içinde
```

Sayfa için karakter kodlamasını GB18030 olarak ayarlar uluslararası karakterleri görüntülemek için. Varsayılan olarak, UTF-8.

5. satır chapter43.TimeBean'i içe aktarır ve önceki sayfadaki ile aynı id'yi kullanarak bir bean oluşturur. Nesne zaten önceki sayfada oluşturulduğundan, bu sayfadaki (4-6. satırlar) ve önceki sayfadaki timeBeanId aynı nesneye işaret eder.

#### 43.9.4 Örnek: Öğrenci Kaydı

Liste 42.11, RegistrationWithHttpSession.java, bir HTML formundan öğrenci bilgilerini eide eden bir Java servlet'i verir (bkz. Şekil 42.21) ve bilgileri kullanıcı onayı için görüntüler (bkz. Şekil 42.22).

Kullanıcı onayladıktan sonra, servlet verileri veritabanına kaydeder. Bu bölüm, servlet'i JSP kullanarak yeniden yazar. Kullanıcı onayı için verileri görüntülemek üzere GetRegistrationData.jsp adında bir JSP sayfası ve verileri veritabanına kaydetmek için StoreData.jsp adında bir başka JSP sayfası oluşturacaksınız.

Her oturumun aynı veritabanına bağlanması gerektiğinden, veritabanına bağlanmak ve bir öğrenciyi veritabanına kaydetmek için bir sınıf tanımlamalısınız. Bu sınıf StoreData olarak adlandırılmıştır ve Liste 43.16'da verilmiştir. initializeJdbc metodu (satırlar 15–31) veritabanına bağlanır ve

Adres tablosuna bir kayıt kaydetmek için bir hazırlıklı ifade oluşturur. storeStudent metodu (satırlar 34–45) bir öğrenci adresini kaydetmek için hazırlıklı ifadeyi çalıştırır. Adres sınıfı Liste 42.12'de oluşturulmuştur.

#### Liste 43.16 StoreData.java

<Yan Not satır 15: veritabanını başlat>

<Yan Not satır 32: öğrenci kaydet>

```
_____ paket chapter43;

java.sql.*'ı içe aktarın;
_____ bölüm42.Adresi içe aktar;

genel sınıf StoreData {
_____ // Veritabanına bir öğrenciyi kaydetmek için bir hazırlıklı ifade kullanın
_____ özel PreparedStatement pstmt;

_____ kamu StoreData() {
_____ başlatmaJdbc();

_____ /** Veritabanı bağlantısını başlat */
_____ özel void initializeJdbc() {
_____ dene {
_____ Class.forName("com.mysql.jdbc.Driver");

_____ // Örnek veritabanına bağlan
_____ Bağlantı connection = DriverManager.getConnection
_____ ("jdbc:mysql://localhost/javabook" , "scott", "tiger");

_____ // Bir Statement oluştur
_____ pstmt = Connection.prepareStatement("Adrese Ekle " +n "(soyadı, adı, m
_____ i, telefon, e-posta, sokak, şehir, "
_____ + "eyalet, posta kodu) değerleri (?, ?, ?, ?, ?, ?, ?, ?)");

_____ }
_____ catch (Exception ex) {n Sys
_____ tem.out.println(ex);

_____ }

_____ /** Bir öğrenci kaydını veritabanına kaydet */
_____ public void storeStudent(Address address) throws SQLException {
_____ pstmt.setString(1, adres.getLastName());n pstmt.s
_____ etString(2, adres.getFirstName());
_____ pstmt.setString(3, adres.getMi());n pstmt.s
_____ etString(4, adres.getTelephone());
_____ pstmt.setString(5, adres.getEmail());n pstmt.se
_____ tString(6, adres.getStreet());
_____ pstmt.setString(7, adres.getCity());n pstmt.se
_____ tString(8, adres.getState());
_____ pstmt.setString(9, adres.getZip());
_____ pstmt.executeUpdate();

_____ }
_____ }
```



Formu görüntüleyen HTML dosyası, Listing 42.8'deki Registration.html ile aynıdır ancak eylem, **HtGetRegistrationData.jsp**.

Formdan verileri alan GetRegistrationData.jsp, Listing 43.17'de gösterilmiştir. 3-4. satırlarda bir bean oluşturulur. 5. satır, formdan özellik değerlerini alır. Bu bir kısayol notasyonudur. Bu notasyonu kullanmak için parametre adlarının ve özellik adlarının aynı olması gerektiğini unutmayın.

#### Listeleme 43.17 GetRegistrationData.jsp

<Yan Not satır 2: import>

<Yan Not satır 3: addressId>

<Yan Not satır 5: özellik değerlerini al>

```
<!-- GetRegistrationData.jsp -->
<%@ sayfa import = "chapter42.Address"%>
<jsp:useBean id = "addressId"
 class = x "bölüm 42.Adres"t kapsam = t "session"t></jsp:useBean>n<jsp
:setProperty name = t "addressId"t property = t "*"t />

<html>
<gövde>
 <h1>JSP Kullanarak Kayıt</h1>

 <%
 eğer (addressId.getLastName() == null ||
 addressId.getFirstName() == null) {
 out.println("Soyadı ve Ad gereklidir");
 return; // Metodu sonlandır
 }
 %>

 <p>Aşağıdaki verileri girdiniz</p>
 <p>Soyadı: <%= addressId.getLastName() %></p>
 <p>Ad: <%= addressId.getFirstName() %></p>
 <p>MI: <%= addressId.getMi() %></p>
 <p>Telefon: <%= addressId.getTelephone() %></p>
 <p>E-posta: <%= addressId.getEmail() %></p>
 <p>Adres: <%= addressId.getStreet() %></p>
 <p>Şehir: <%= addressId.getCity() %></p>
 <p>Eyalet: <%= addressId.getState() %></p>
 <p>Posta Kodu: <%= addressId.getZip() %></p>

 <!-- Cevapları işlemek için eylemi ayarlayın -->
 <form method = "post" action = "StoreStudent.jsp">
 <input type = "submit" value = "Onayla">
 </form>n <
/body> için
de</html>
```

GetRegistrationData.jsp, kullanıcı

Onay butonuna tıkladığında StoreStudent.jsp'yi (satır 31) çağırır. Liste 43.18'de, aynı addressId önceki sayfa ile aynı oturum kapsamı içinde satır 3-4'te paylaşılmaktadır. StoreData için bir bean,

satır 5-6'da uygulama kapsamı ile oluşturulmaktadır.

### Liste 43.18 StoreStudent.jsp

<Yan Not satır 2: import>

<Yan Not satır 3: addressId>

<Yan Not satır 5: storeDataId>

```
<!-- StoreStudent.jsp -->
<%@ sayfa import = "chapter42.Address" %>
<jsp:useBean id = "addressId" class = "chapter42.Address"
scope = "session"></jsp:useBean>
<jsp:useBean id = t "storeDataId" class = t "chapter43.StoreData" nscope = t "application">t</jsp:useBean>

<html>n <
 gövde>n <
 %
 StoreDataId.storeStudent(adresId); n out.println(
 addressId.getFirstName() + t "t +n adresId.getLastName() +
 " artık veritabanında kayıtlıdır");
 out.close(); // Akışı kapat
 %>
</body> için
de</html>
```

NOT

<Yan Not: uygun kapsamlar>

addressId için kapsam oturum, ancak storeDataId için kapsam uygulama. Neden? GetRegistrationData.jsp öğrenci bilgilerini alır ve StoreData.jsp bilgileri aynı oturumda saklar. Bu nedenle oturum kapsamı addressId için uygundur. Tüm oturumlar aynı veritabanına erişir ve verileri saklamak için aynı hazırlanmış ifadeyi kullanır. storeDataId için uygulama kapsamıyla, StoreData için bean yalnızca bir kez oluşturulmalıdır.

NOT

<Yan Not: istisnalar>

storeStudent metodu 11. satırda bir java.sql.SQLException fırlatabilir. JSP'de, kontrol edilen istisnalar için try-blokunu atlayabilirsiniz. Bir istisna durumunda, JSP bir hata sayfası gösterir.

İPUCU

<Yan Not: bean kullanımı>

Bean'leri kullanmak, JSP geliştirmek için etkili bir yoldur. Java kodunu mümkün olduğunca bir bean içine koymalısınız. Bean, yalnızca JSP programlamasını basitleştirmekle kalmaz, aynı zamanda kodun yeniden kullanılabilir olmasını da sağlar. Bean ayrıca kalıcı oturumları uygulamak için de kullanılabilir.

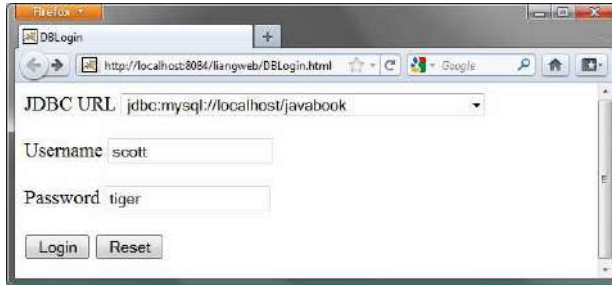
### 43.10 JavaServer Sayfalarından İstekleri İletme

JSP kullanılarak geliştirilen web uygulamaları genellikle birbirine bağlı birçok sayfadan oluşur. JSP, bir sayfayı başka bir sayfaya iletmek için kullanılacak aşağıdaki sözdiziminde bir iletmek etiketi sağlar:

```
<jsp:forward page = "destination" />
```

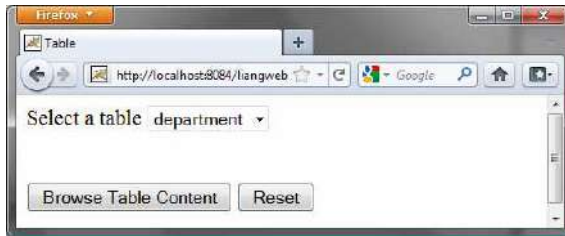
#### 43.11 Vaka Çalışması: Veritabanı Tablolarını Gözden Geçirme

Bu bölüm, tabloları gözden geçirmek için çok yararlı bir JSP uygulamasını sunmaktadır. Uygulamayı başlattığınızda, ilk sayfa kullanıcıdan bir veritabanı için JDBC sürücüsünü, URL'yi, kullanıcı adını ve şifreyi girmesini ister; bu, Şekil 43.9'da gösterilmiştir. Veritabanına giriş yaptıktan sonra, gözden geçirmek için bir tablo seçebilirsiniz; bu, Şekil 43.10'da gösterilmiştir. Tablo İçeriğini Gözden Geçir butonuna tıkladığınızda, tablo içeriği gösterilir; bu, Şekil 43.11'de gösterilmiştir.



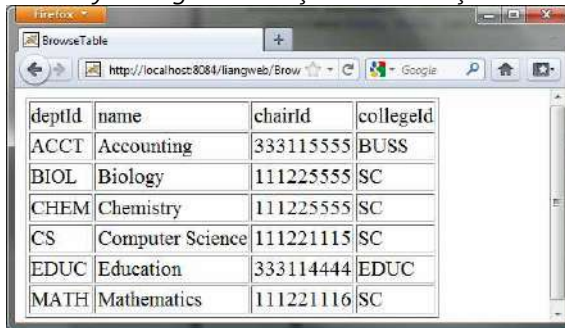
Şekil 43.9

*Bir veritabanına erişmek için JDBC sürücüsünü, URL'yi, kullanıcı adını ve şifreyi sağlamanız gerekir.*



Şekil 43.10

*Bu sayfadan göz atmak için bir tablo seçebilirsiniz.*



deptId	name	chairId	collegeId
ACCT	Accounting	333115555	BUSS
BIOL	Biology	111225555	SC
CHEM	Chemistry	111225555	SC
CS	Computer Science	111221115	SC
EDUC	Education	333114444	EDUC
MATH	Mathematics	111221116	SC

Şekil 43.11

*Seçilen tablonun içeriği görüntülenir.*

DBBean.java adında bir JavaBeans bileşeni oluşturun (Liste 43.19).

### Liste 43.19 DBBean.java

<Yan Not satır 16: sürücüyü yükle>  
<Yan Not satır 19: veritabanına bağlan>  
<Yan Not satır 33: tabloları al>  
<Yan Not satır 51: tablo adlarını döndür>  
<Yan Not satır 55: getConnection()>  
<Yan Not satır 59: kullanıcıAdı>  
<Yan Not satır 67: şifre>  
<Yan Not satır 79: sürücü>  
<Yan Not satır 83: url>

```
paket chapter43;

java.sql.*'ı içe aktarın;

genel sınıf DBBean {n öz
 el Bağlantı bağlantısı = null;n özel String kul
 lanıcı adı;
 private String password;
 private String driver;
 private String url;

 /** Veritabanı bağlantısını başlat */
 public void initializeJdbc() {
 dene {
 System.out.println("Sürücü " + driver + " olarak ayarlandı.");
 Class.forName(driver);

 // Örnek veritabanına bağlan
 connection = DriverManager.getConnection(url, username,
 password);
 }
 catch (Exception ex) {n ex.
 printStackTrace();n }
 }

 /** Veritabanındaki tabloları al */
 public String[] getTables() {
 String[] tables = null;

 dene {
 DatabaseMetaData dbMetaData = bağlantı.getMetaData();n Resul
 tSet rsTables = dbMetaData.getTables(null, null, null,n new String[] {"T
 ABLE"});

 int boyut = 0;
 while (rsTables.next()) size++;n rsTabl

 es = dbMetaData.getTables(null, null, null,n new String[] {"T
 ABLE"});

 tablolar = new String[size];n int
 i = 0;
 while (rsTables.next())
 tablolar[i++] = rsTables.getString("TABLE_NAME");n }

 catch (İstisna örneği) {
```

```

 ex.printStackTrace();n }

 dönüş tabloları;n }

 /** Bağlantı özelliğini döndür */
 public Connection getConnection() {
 return connection;
 }

 public void setUsername(String newUsername) {n kull
 anıcı adı = newUsername;
 }

 public String getUsername() {n ret
 urn kullanıcı adı;
 }

 public void setPassword(String newPassword) {n şifre
 = newPassword;
 }

 public String getPassword() {n dön
 üş parolası;
 }

 public void setDriver(String newDriver) {n sürücü
 = newDriver;
 }

 public String getDriver() {n dönü
 ş sürücüsü;
 }

 public void setUrl(String newUrl) {n url = n
 ewUrl;
 }

 public String get Url() {ve ret
 urn url;
 }
 }
}

```

DBLogin.html adında bir HTML dosyası oluşturun (Liste 43.20) ve kullanıcıdan database bilgilerini girmesini isteyin ve üç JSP dosyası adlandırın DBLoginInitialization.jsp (Liste 43.21), Table.jsp (Liste 43.22), ve BrowseTable.jsp (Liste 43.23) veritabanı

informasyonunu işlemek ve elde etmek için.

### Liste 43.20 DBLogin.html

<Yan Not satır 9: form eylemi>  
 <Yan Not satır 12: combo kutusu>  
 <Yan Not satır 18: gönder>

```

<!-- DBLogin.html -->
<html><h
 ead> ve <t
 itle> içinde

```

```

 DBGiriş
 </title>
</head>
<gövde>
 <form method = "post" action = "DBLoginInitialization.jsp">n JDBC
 C URL'si
 <select name = "url" to size = "1">
 <option>tjdbc:odbc:ExampleMDBDataSource</option>n
 <option>tjdbc:mysql://localhost/javabook</option>n<op
 tion>tjdbc:oracle:thin:@liang.armstrong.edu:1521:orclt</option>n</select>

 Kullanıcı Adı <input name = "username" />

 Şifre <input name = "password" />

 <input type = "submit" name = "Submit" value = "Giriş" />
 <input type = "reset" value = "Sıfırla" />
 </form> n <
</body> için
</html>

```

#### Listeleme 43.21 DBLoginInitialization.jsp

<Yan Not satır 2: import>  
 <Yan Not satır 3: bean oluştur>  
 <Yan Not satır 14: db'ye bağlan>  
 <Yan Not satır 17: hata bildir>  
 <Yan Not satır 20: tabloları al>

```

<!-- DBLoginInitialization.jsp -->
<%@ sayfa import = "chapter43.DBBean"%>
<jsp:useBean id = "dBBBeanId" scope = "oturum"
 class = "chapter43.DBBean">
</jsp:useBean>
<jsp:setProperty name = "dBBBeanId" value = "*" />n<ht
ml>
<kafa>
 <title>tDBLoginInitialization</title>n </he
ad>
<gövde>
 <%-- Veritabanına bağlan --%>
 <% dBBBeanId.initializeJdbc(); %>

 <% if (dBBBeanId.getConnection() == null) { %>
 Hata: Giriş başarısız. Tekrar deneyin.
 <%t }
 başka {t%>
 <jsp:ileri sayfa = "Table.jsp" />n<%t } t
 %>
</body>
</html>

```

#### Listeleme 43.22 Table.jsp

<Yan Not satır 2: import>  
 <Yan Not satır 3: bean al>  
 <Yan Not satır 11: tabloları al>  
 <Yan Not satır 16: form oluştur>

```

<!-- Tablo.jsp -->
<%@ sayfa import = "chapter43.DBBean" %>
<jsp:useBean id = "dBBeanId" scope = "oturum"
 class = "chapter43.DBBean">
</jsp:useBean>
<html>
 <kafa>
 <başlık>Tablo</başlık>
 </head>
 <gövde>
 <% String[] tablolar = dBBeanId.getTables();
 if (tablolar == null) { %>
 Hiç tablo yok
 <%t %>
 başka { t %>
 <form method = "post" action = "BrowseTable.jsp">
 Bir tablo seçin
 <select name = "tablename" size = "1">
 <% for (int i = 0; i < tablolar.length; i++) { %>
 <option><%= tablolar[i] %></option>
 <%t %>
 } t %>
 </select>

 <input type = "submit" name = "Submit"
 value = "Tablo İçeriğini Gözet">
 <input type = "reset" value = "Sıfırla">
 </form>
 </body> için
 de</html>

```

### Listeleme 43.23 BrowseTable.jsp

<Yan Not satır 2: import>  
 <Yan Not satır 3: bean al>  
 <Yan Not satır 13: tablo adını al>  
 <Yan Not satır 15: tablo sütunu>  
 <Yan Not satır 21: sütun adları>  
 <Yan Not satır 28: tablo içeriği>  
 <Yan Not satır 35: içeriği göster>

```

<!-- BrowseTable.jsp -->
<%@ sayfa import = "chapter43.DBBean"%>
<jsp:useBean id = "dBBeanId" scope = "oturum"
 class = "chapter43.DBBean">
</jsp:useBean>
<%@ sayfa import = "java.sql.*"%>
<html>
 <kafa>
 <title>Tabloyu Gözet</title>
 </head>
 <gövde>

 <% t String tableName = request.getParameter(t"tablename");n R

 esultSet rsColumns = dBBeanId.getConnection().getMetaData().
 getColumns(renault,t nuuly, tablo Adı, null'a);

```

```

%>
<tablo kenarlığı = t "1"t>n
<tr>
 <% // Tabloya sütun adlarını ekle
 while (rsColumns.next()) { %>
 <td><%=t rsColumns.getString(t"COLUMN_NAME"t)t %></td>n <%
 t)t%>
 </tr>

<%t Açıklama bildirimi =
 dBBeanId.getConnection().createStatement(); n Resul
 tSet rs =statement.executeQuery(
 "select * from " + tableName);

 // Sütun sayısını al
 int columnCount = rs.getMetaData().getColumnCount();

 // Satırları rowData'ya kaydet
 while(rs.sonraki()) {
 out.println(t"<tr>"t);
 for (tintt i = 0; i < ColumnCount; i++) { %> n<td>t<%
 = rs.getObject(i + t 1t) %>t</td>
 <%t }
 out.println(t"</tr>"t);n } t
 %>

</table> içinde
</body> için
de</html>

```

Uygulamayı DBLogin.html'den başlatıyorsunuz. Bu sayfa kullanıcıdan bir JDBC sürücüsü, URL, kullanıcı adı ve şifre girmesini ister bir veritabanına giriş yapmak için. Erişilebilir sürücülerin ve URL'lerin seçim listesinde sağlandığı bir liste vardır. Bu veritabanı sürücülerinin projede Kütüphaneler düğümüne eklenmiş olduğundan emin olmalısınız.

Giriş düğmesine tıkladığınızda, DBLoginInitialization.jsp çağrılır. Bu sayfa ilk kez işlendiğinde, dBBeanId adında bir DBBean örneği yaratılır. Girdi parametreleri sürücü, url, kullanıcı adı ve şifre bean özelliklerine aktarılır. initializeJdbc yöntemi sürücüyü yükler ve veritabanına bir bağlantı kurar. Eğer giriş başarısız olursa, bağlantı özelliği null olur. Bu durumda, bir hata mesajı görüntülenir. Eğer giriş başarılı olursa, kontrol

Table.jsp'ye yönlendirilir.

Table.jsp, aynı oturumda DBLoginInitialization.jsp ile dBBeanId paylaşır, bu nedenle dBBeanId aracılığıyla bağlantıya erişebilir ve veritabanındaki tabloları veritabanı meta verilerini kullanarak elde edebilir. Tablo adları bir formda seçim kutusunda görüntülenir. Kullanıcı bir tablo adını seçip *Browse Table Content* butonuna tıkladığında, BrowseTable.jsp işlenir.

BrowseTable.jsp, aynı oturumda Table.jsp ve DBLoginInitialization.jsp ile dBBeanId paylaşır. Seçilen tablonun içeriklerini Table.jsp'den alır.

### JSP Betik Yapıları Söz Dizimi

- <%= Java ifadesi %> İfade değerlendirilir ve sayfaya eklenir.



- <% Java ifadesi %> Java ifadeleri jspService metoduna eklenir.
- <%= Java bildirimi %> Veri alanlarını ve yöntemleri tanımlar.
- <%-- JSP yorumu %> JSP yorumları, sonuçta oluşan HTML dosyasında görünmez.
- <%@ yönerge attribute="değer" %> JSP yönergeleri, JSP motoruna JSP sayfası hakkında bilgi verir. Örneğin, <%@ page import="java.util.\*, java.text.\*" %> java.util.\* ve java.text.\*'yi içe aktarır.
- <jsp:useBean id="objectName" scope="scopeAttribute" class="ClassName" /> Yeni ise bir bean oluşturur. Eğer bir bean zaten oluşturulmuşsa, id'yi aynı kapsamda bean ile ilişkilendirir.
- <jsp:useBean id="objectName" scope="scopeAttribute" class="ClassName" %> ifadeleri </jsp:useBean> ifadeler bean oluşturulduğunda çalıştırılır. Eğer aynı id ve sınıf adıyla bir bean zaten varsa, ifadeler çalıştırılmaz.
- <jsp:getProperty name="beanId" property="sample" /> Bean'den özellik değerini alır, bu da <%= beanId.getSample() %>.
- <jsp:setProperty name="beanId" property="örnek" değer="test1" /> Bean için özellik değerini ayarlar, bu da <%= beanId.setSample("test1"); %>.
- <jsp:setProperty name="beanId" property="score" param="score" /> Bir girdi parametresi ile özelliği ayarlar.
- <jsp:setProperty name="beanId" property="\*" /> Bean'deki tüm özellikler i beanId ile ilişkilendirir ve ayarlar özellik adlarıyla eşleşen girdi parametreleri ile.
- <jsp:forward page="destination" /> Bu sayfayı yeni bir sayfaya yönlendirir.

### JSP Ön Tanımlı Değişkenler

xuygulama, tüm istemciler için kalıcı verileri saklamak üzere ServletContext nesnesini temsil eder.

- config , sayfa için ServletConfig nesnesini temsil eder.
- out , response.getWriter()'dan elde edilen PrintWriter'in bir örneği olan karakter çikti akışını temsil eder.
- sayfa bunun alternatifi.
- request istemcinin isteğini temsil eder, bu da servlet'in service metodundaki HttpServletRequest örneğidir.
- response , servlet'in service metodundaki HttpServletResponse örneği olan istemcinin yanıtını temsil eder.
- oturum , request.getSession() ile elde edilen istek ile ilişkili HttpSession nesnesini temsil eder.

### Bölüm Özeti

1. Bir JavaServer sayfası, dinamik içerik oluşturulmasını sağlayan özel etiketler olarak bilinen *JSP etiketleri* ile birlikte normal bir HTML sayfası gibidir. Statik HTML ile bir Web sayfası oluşturabilir ve dinamik içerik oluşturma kodunu JSP etiketleri içinde yerleştirebilirsiniz.
2. Bir JSP sayfası, .jsp uzantısına sahip bir dosyada saklanmalıdır. Web sunucusu .jsp dosyasını bir java servlet'ine çevirir, servlet'i derler ve çalıştırır. Çalıştırmanın sonucu brauzere görüntülenmek üzere gönderilir.
3. Bir JSP sayfası, sayfa ilk kez istendiğinde bir servlet'e dönüştürülür. Sayfa değiştirilmediyse yeniden dönüştürülmez. İlk kez gerçek bir kullanıcının

bir gecikme ile karşılaşan JSP geliştiricileri, sayfa

kurulduktan sonra testi yapmalıdır.

4. JSP yapılarının üç ana türü vardır: betik yapıları, direktifler ve eylemler.

*Betik* öğeleri, Java kodunu belirtmenizi sağlar ve bu kod, sonuçta oluşan

*servlet*. *Yönergeler* sonuçta oluşan servlet'in genel yapısını kontrol etmenizi sağlar. *Eylemler* JSP motorunun davranışlarını kontrol etmenizi sağlar.

5. Sonuçta oluşan servlet'e Java kodu eklemek için üç tür betik yapısı kullanılabilir: ifadeler, scriptlet'ler ve bildirimler.

6. Kapsam niteliği (uygulama, oturum, sayfa ve istek) bir JavaBeans nesnesinin kapsamını belirtir. Uygulama, nesnenin uygulamaya bağlanmasını belirtir. Oturum, nesnenin istemcinin oturumuna bağlanmasını belirtir. Sayfa, nesnenin sayfaya bağlanmasını belirten varsayılan kapsamdır. İstek, nesnenin istemcinin isteğine bağlanmasını belirtir.

7. JSP kullanılarak geliştirilen web uygulamaları genellikle birbirine bağlı birçok sayfadan oluşur. JSP, bir sayfayı başka bir sayfaya yönlendirmek için kullanılabilecek aşağıdaki sözdiziminde bir yönlendirme etiketi sağlar: `<jsp:forward page="destination" />`.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın:

[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

### Gözden Geçirme Soruları

#### Bölmeler 43.1-43.3

- 43.1 Bir JavaServer sayfasının dosya adı uzantısı nedir? Bir JSP sayfası nasıl işlenir?

- 43.2 NetBeans'te JSP içeren bir .war oluşturabilir misiniz? .war, bir Java uygulama sunucusunda nereye yerleştirilmelidir?

- 43.3 Bir HTML dosyasını (örneğin, c:est.html) Internet Explorer'ın Adres alanına dosyanın tam adını yazarak görüntüleyebilirsiniz. Neden bir JSP dosyasını sadece dosya adını yazarak görüntüleyemezsiniz?

#### Bölüm 43.4

- 43.4 JSP ifadesi, JSP scriptlet'i ve JSP bildirimi nedir?

Bu yapıları JSP'de nasıl yazarsınız?

- 43.5 Aşağıdaki JSP kodunda üç sözdizimi hatası bulun:

```
<%! int k %>
<% for (int j = 1; j <= 9; j++) %>
<%=
```

- 43.6 Aşağıdaki JSP'de, servlet'e çevrildiğinde hangi değişkenler örnek değişkenlerdir ve hangileri yerel değişkenlerdir?

```
<%! int k; %>
<%! int ben; %>
<% fort (tintt j = t 1t; j <= t 9t; j++) k += t 1t;t%>n<%
=t k>t
 <%=t it>
 <%=t getTime()t>
n<% ö
zel longt getTime() {
 uzun süre = System.currentTimeMillis();
 return zaman;
}
```

#### Bölüm 43.5

- 43.7 JSP'deki önceden tanımlanmış değişkenleri tanımlayın.

43.8 ComputeLoan.jsp'deki 7. satırdaki JSP scriptlet `<%` JSP deklarasyonu `<%!` ile değiştirilir se ne yanlış olur?

43.9 JSP deklarasyonlarında önceden tanımlanmış değişkenleri (örneğin, `request`, `response`, `out`) kullanabilir misiniz?

#### Bölüm 43.6

43.10 Sayfa direktifi için JSP direktiflerini ve niteliklerini tanımlayın.

43.11 Bir sınıfın paket ifadesi yoksa, onu içe aktarabilir misiniz?

43.12 Eğer bir JSP'den özel bir sınıf kullanıyorsanız, sınıf nerede bulunmalıdır?

#### Bölüm 43.7

43.13 Bir JSP scriptlet'inde bir nesne oluşturabilirsiniz. Yeni operatorü kullanarak oluşturulan bir nesne ile `<jsp:useBean ... >` etiketi kullanarak oluşturulan bir bean arasındaki fark nedir?

43.14 Scope niteliği ne için kullanılır? Dört scope niteliğini tanımlayın.

43.15 `<jsp:useBean ... >` ifadesinin JSP motoru tarafından nasıl işlendiğini tanımlayın.

#### Bölümler 43.8-43.10

43.16 Bean özelliklerini girdi parametreleriyle nasıl ilişkilendirirsiniz?

43.17 Başka bir JSP sayfasına istekleri yönlendirmek için bir ifade nasıl yazarsınız?

### Programlama Alıştırmaları

NOT: Bu bölümdeki çift numaralı alıştırmaların çözümleri, `alıştırma\jspexercise` içinde bulunmaktadır ve `evennumberedexercise.zip` dosyasından indirilebilir, bu dosya

Eşlik Eden Web Sitesi'nden temin edilebilir.

#### Bölüm 43.4

43.1 (JS 43.2/JS)

#### Bölüm 43.5

43.3\*

JSP'de parametreleri elde etmek için Liste 42.4'teki `Servlet`'i, `GetParameters.java`, JSP kullanarak yeniden yazın. Liste 42.3'teki `Student_Registration_Form.html` ile aynı olan bir HTML formu oluşturun, ancak eylem parametre değerlerini elde etmek için `Exercise40_3.jsp` ile değiştirilmiştir.

#### Bölüm 43.6

43.4 (JS)

ComputeTax'ı JSP'de içe aktarın.

43.5\*

(Metin dosyalarından puanları bul) Servisler kullanarak Alıştırma 42.6'yı yeniden yazın.

43.6\*\*

(Veritabanı tablolarından puanları bul) Servisler kullanarak Alıştırma 42.7'yi yeniden yazın.

#### Bölüm 43.7

43.7\*\*

(Şifreyi değiştir) Servisler kullanarak Alıştırma 42.8'i yeniden yazın.

#### Kapsamlı

43.8\*

(JSP'de çerezleri sakla) JSP kullanarak Alıştırma 42.10'u yeniden yazın. response.addCookie(Cookie) kullanarak bir çerez ekleyin.

43.9\*

(JSP'de çerezleri al) JSP kullanarak Alıştırma 42.11'i yeniden yazın. `Cookie[] cookies = request.getCookies()` kullanarak tüm çerezleri alın.

43.10\*

43.11\*\*\*

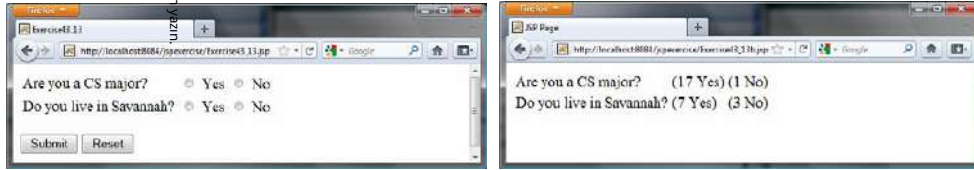
(Sözdizimi vurgulama) JSP kullanarak Alıştırma 42.12'yi yeniden yazın.

43.12\*\*

(Anket) JSP kullanarak Alıştırma 42.13'ü yeniden yazın.

43.13\*\*\*

(Çoktan seçmeli anket) Alıştırma 42.13'teki Anket tablosu yalnızca bir soru içermektedir. Farz edelim ki birden fazla soru içeren bir Anket tablosuna sahipsiniz. Tablodaki tüm soruları okuyan ve bunları Şekil 43.12a'da gösterildiği gibi bir formda görüntüleyen bir JSP yazın. Kullanıcı Gönder butonuna tıkladığında, başka bir JSP sayfası çağırılır. Bu sayfa, her soru için Evet veya Hayır sayımlarını günceller ve Anket tablosundaki her soru için mevcut Evet ve Hayır sayımlarını gösterir, Şekil 43.12b'de gösterildiği gibi. Tablo bir çok soru içerebilir. Şekildeki sorular sadece örneklerdir. Soruları alfabetik sıraya göre sıralayın.

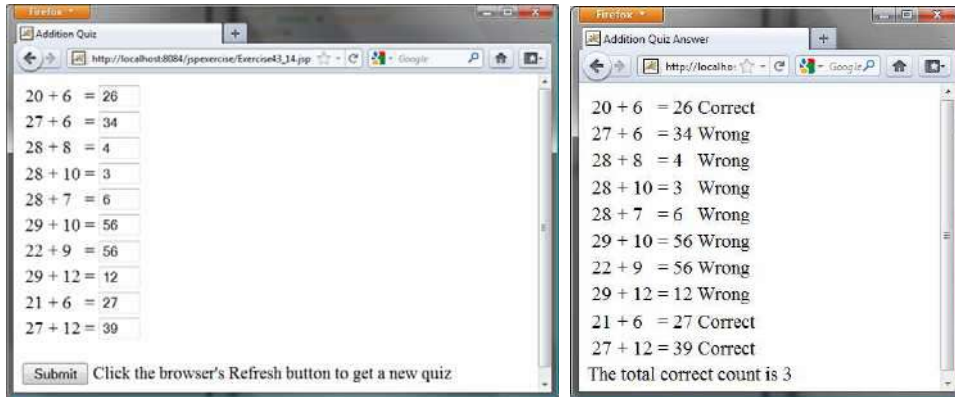


Şekil 43.12

Form, kullanıcından (a) bölümündeki her soru için Evet veya Hayır girmesini ister ve güncellenmiş Evet veya Hayır sayıları (b) bölümünde görüntülenir.

43.14\*\*

(Toplama sınavı) Şekil 43.13a'da gösterildiği gibi rastgele toplama sınavları üreten bir JSP programı yazın. Kullanıcı tüm soruları yanıtladıktan sonra, JSP sonucu gösterir; bu da Şekil 43.13b'de gösterilmiştir.



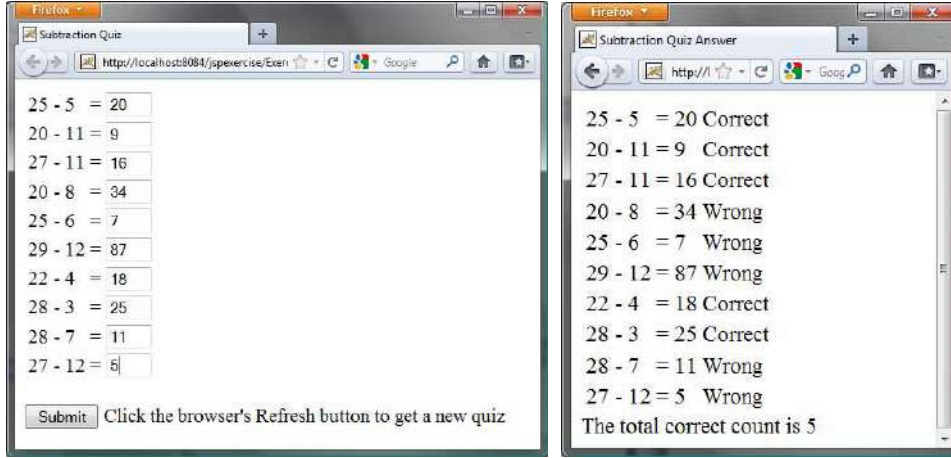
(a) (b)

Şekil 43.13

Program, (a) bölümünde toplama sorularını ve (b) bölümünde cevapları görüntüler.

43.15\*\*

(Çıkarma sınavı) Şekil 43.14a'da gösterildiği gibi rastgele çıkarma sınavları üreten bir JSP programı yazın. İlk sayı her zaman ikinci sayıdan büyük veya ona eşit olmalıdır. Kullanıcı tüm soruları yanıtladıktan sonra, JSP sonucu gösterir; bu da Şekil 43.14b'de gösterilmiştir.



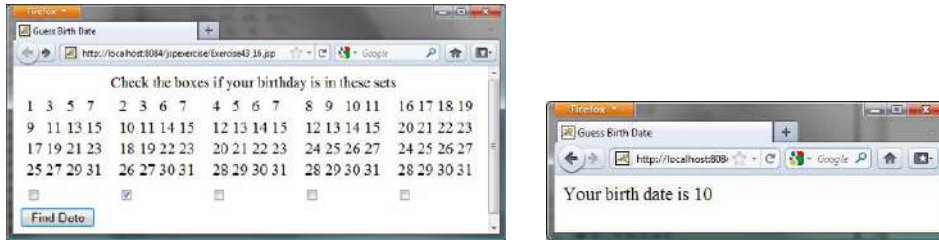
(a) (b)

Şekil 43.14

Program, (a) bölümünde çıkarma sorularını ve (b) bölümünde cevapları görüntüler.

43.16\*\*

(Doğum günü tahmini) Liste 3.3, GuessBirthDay.java, bir doğum gününü tahmin etmek için bir program verir. Şekil 43.15a'da gösterildiği gibi beş set sayı görüntüleyen bir JSP programı yazın. Kullanıcı uygun kutucukları işaretleyip Tarih Bul butonuna tıkladığında, program tarihi gösterir; bu da Şekil 43.15b'de gösterilmiştir. uygun kutulara tıklayın ve Tarih Bul düğmesine tıkladığınızda program, Şekil 43.15b'de gösterildiği gibi tarihi görüntüler.



(a)

(b)

Şekil 43.15

(a) Program, kullanıcının kutuları işaretlemesi için beş set sayı gösterir. (b) Program tarihi gösterir.

43.17\*\*

(*Başkentleri tahmin et*) Kullanıcının bir eyalet için bir başkent girmesini isteyen bir JSP yazın, Şekil 43.16a'da gösterildiği gibi. Kullanıcı girdisini aldıktan sonra, program cevabın doğru olup olmadığını bildirir, Şekil 43.16b'de gösterildiği gibi. Başka bir soruyu görüntülemek için *İleri* butonuna tıklayabilirsiniz. Eyaletleri ve başkentleri depolamak için iki boyutlu bir dizi kullanabilirsiniz, Alıştırma 9.22'de önerildiği gibi. Diziden bir liste oluşturun ve soruların rastgele sırayla görünmesi için listeyi yeniden sıralamak üzere shuffle yöntemini uygulayın.



**Şekil 43.16**

(a) Program bir soru gösterir. (b) Program sorunun cevabını gösterir.

43.18\*

(*Büyük faktöriyel*) Büyük faktöriyel işlemleri yönetmek için Listeleme 43.11'i yeniden yazın. §14.12'de tanıtilen `BigInteger` sınıfını kullanın.

43.19\*\*

(*Bir Personel tablosuna erişim ve güncelleme*) Şekil 43.17'de gösterildiği gibi, Alıştırma 33.1 için bir JSP yazın.

**Şekil 43.17**

JSP sayfası, personel bilgilerini görüntülemenizi, eklemenizi ve güncellenizi sağlar.

43.20\*

(*Tahmin numarası*) 1 ile 1000 arasında rastgele bir sayı üreten bir JSP sayfası yazın ve kullanıcının bir tahmin girmesine izin verin. Kullanıcı bir tahmin girdiğinde, program kullanıcının tahmininin doğru olup olmadığını, çok yüksek mi yoksa çok düşük mü olduğunu söylemelidir.

**\*\*\*Bu bir bonus Web bölümü**

## **BÖLÜM 44**

### **Java Sunucusu Yüzleri**

#### Amaçlar

- JSF'nin ne olduğunu açıklamak (§44.1).
- NetBeans kullanarak bir JSF sayfası oluşturmak (§44.2).
- Bir JSF yönetilen bean oluşturmak (§44.2).
- Bir facelet içinde JSF ifadelerini kullanmak (§44.2).
- JSF GUI bileşenlerini kullanmak (§44.3).
- Bir formdan girdi almak ve işlemek (§44.4).
- Uygulamada oturumları izlemek için, oturum, görünüm ve istek kapsamı (§44.5).
- Girişi JSF doğrulayıcıları kullanarak doğrulamak için (§44.6).
- Veritabanını facelet'lerle bağlamak için (§44.7).

#### 44.1 Giriş

<kenar boşluğu notu: sunucu uygulamalar

i> n<kenar boşluğu notu: JSP> n<

kenar boşluğu notu: JSF>

42. Bölümde tanıtılan servlet'lerin kullanımı, Java Web teknolojisinin bir temelidir. Sunucu tarafı uygulamaları yazmanın ilkel bir yoludur. 43. Bölümde tanıtılan JSP, bir beti kleme yeteneği sağlar ve Java kodunu XHTML içine yerleştirmenize olanak tanır. JS P kullanarak Web programları geliştirmek, servlet'lerden daha kolaydır. Ancak, JSP'nin ba zı problemleri vardır. Öncelikle, Java kodunu HTML ile karıştırdığı için çok karmaşık ol abilir. İkincisi, kullanıcı arayüzü geliştirmek için JSP kullanmak bir hayli zahmetlidir. JavaS erver Faces (JSF) imdada yetişir. JSF, Java kodunu HTML'den tamamen ayırmanıza olanak ta nır. Yeniden kullanılabilir UI bileşenlerini bir sayfada bir araya getirerek, bu bileşenleri Java programlarına bağlayarak ve istemci tarafından üretilen olayları sunucu tarafı olay işl eyicilerine bağlayarak web uygulamaları hızlı bir şekilde geliştirebilirsiniz. JSF kullanılara k geliştirilen uygulama, hata ayıklaması v ve bakımı kolaydır.

<kenar notu: JSF 2> n<ke

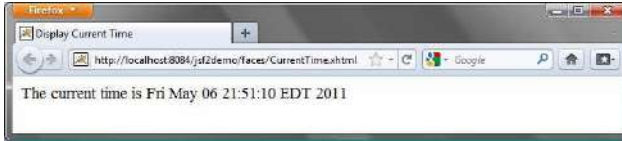
nar notu: XHTML> n<ken

ar notu: CSS>

NOT: Bu bölüm, JavaServer Faces için en son standart olan JSF 2'yi tanı tmaktadır. Bu bölüme başlamak için XHTML (Genişletilebilir Hiper Meti n İşaretleme Dili) ve CSS (Kaskad Stil Sayfası) bilmeniz gerekmektedir. XHTML ve CSS hakkında bilgi için, lütfen Eşlik Eden Web Sitesi'n deki Ekler Bölüm V.A ve Bölüm V.B'ye bakın.

#### 44.2 JSF ile Başlarken

Basit bir örnek, NetBeans kullanarak JSF projeleri geliştirmenin temellerini gösterecektir. Örnek, sunucudaki tarih ve saati görüntülemektir; Şekil 44.1'de gösterildiği gibi.



Şekil 44.1

Uygulama, sunucudaki tarih ve saati görüntüler.

##### 44.2.1 JSF Projesi Oluşturma

Uygulamayı oluşturmak için adımlar şunlardır.

<kenar notu: bir proje oluşturun>

Yeni Proje iletişim kutusunu görüntülemek için Dosya > Yeni Proje'yi seçin. Bu kutuda, Kate goriler panelinde Java Web'i ve Projeler panelinde Web Uygulaması'nı seçin. Yeni Web U ygulaması iletişim kutusunu görüntülemek için İleri'ye tıklayın.

Yeni Web Uygulaması iletişim kutusunda, aşağıdaki

alanları girin ve seçin, Şekil 44.2'a da gösterildiği gibi:

Proje Adı: jsf2demo

Proje Konumu: c:\oök

Ana Proje Olarak Ayarla'yı işaretleyin

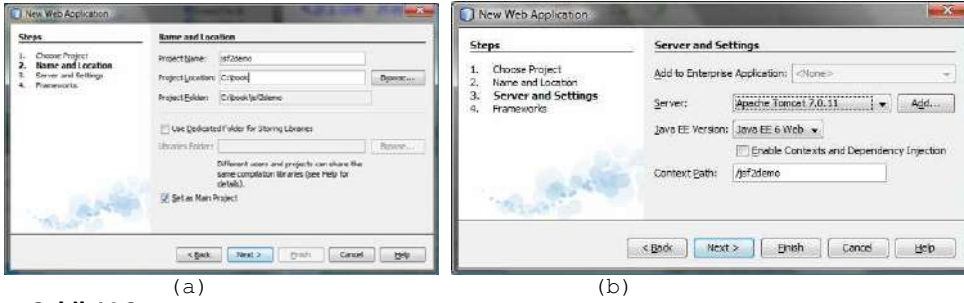


Sunucuları ve ayarları seçmek için iletişim kutusunu görüntülemek üzere *İleri* düğmesine tıklayın. Şekil 44.2b'de gösterildiği gibi aşağıdaki alanları seçin. (Not: Java EE 6'yı destekleyen herhangi bir sunucu kullanabilirsiniz, örneğin GlassFish 3.x.)  
Sunucu: Apache Tomcat 7.0.11  
Java EE Sürümü: Java EE 6 Web

Çerçeveleri seçmek için iletişim kutusunu görüntülemek üzere

İleri'ye tıklayın, Şekil 44.3'te gösterildiği gibi. *JavaServer Faces* ve JSF 2.0'ı Kayıtlı Kütüphaneler olarak işaretleyin.

Proje oluşturmak için *Tamamla*'ya tıklayın, Şekil 44.4'te gösterildiği gibi.



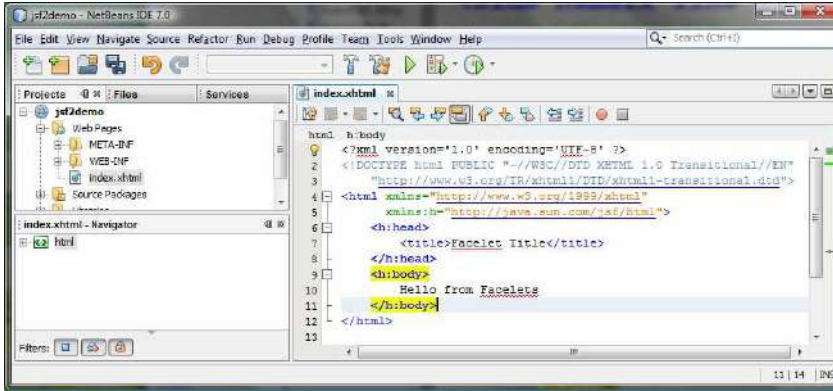
Şekil 44.2

*Yeni Web Uygulaması iletişim kutusu, yeni bir Web projesi oluşturmanıza olanak tanır.*



Şekil 44.3

*Web projesi oluşturmak için JavaServer Faces ve JSF 2.0'ı işaretleyin.*



**Şekil 44.4**

Yeni bir Web projesinde varsayılan bir JSF sayfası oluşturulmuştur.

#### 44.2.2 Temel JSF Sayfası

##### **<kenar notu: görünüm>**

Yeni bir proje, Şekil 44.4'te gösterildiği gibi index.xhtml adında varsayılan bir sayfa ile oluşturulmuştur. Bu sayfa, JSF etiketlerini XHTML etiketleriyle birleştiren bir facelet olarak bilinir. Liste 44.1, index.xhtml'in içeriğini listeler.

Liste 44.1 index.xhtml

**\*\*\*PD: Lütfen aşağıdaki kodda satır numaraları ekleyin\*\*\* \*\*\*Düzen:**  
Lütfen tam olarak düzenleyin. Boşluğu atlamayın. Bu, kitapta yer alan tüm kaynak kodları için geçerlidir. Teşekkürler, AU.

**<margin note line 1: xml version>** n<mar  
gin note line 2: comment> n<margin  
note line 3: DOCTYPE> n<margin no  
te line 5: default namespace> n<margin note line  
6: JSF namespace> n<margin note line 7: h:  
head>

**<kenar notu satırı 10: h:body>**

```
<xml version='1.0' encoding='UTF-8' > n<!-- index.
xhtml -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://w
ww.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http
://www.w3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"> n <h:hea
d>
<title>Facelet Başlığı</title>
</h:head>
<h:gövde>
Facelets'ten Merhaba
</h:body>
</html>
```

##### **<kenar notu: XML bildirimi>**

1. Satır, belgenin XML sürüm 1.0'a uyduğunu ve UTF-8 kodlamasını kullandığını belirtmek için bir XML beyanıdır. Beyan isteğe bağlıdır, ancak kullanmak iyi bir uygulamadır. Aksi takdirde, beyan olmadan bir belge farklı bir sürümde varsayılabilir, bu da hatalar açabilir. Bir XML beyanı mevcutsa, belgedeki ilk öge olmalıdır. Bunun nedeni, bir XML işleyicisinin belgenin doğru bir şekilde işlenebilmesi için bilgi almak üzere ilk satıra bakmasıdır.

belge hakkında bilgi almak için ilk satırı arar, böylece doğru bir şekilde işlenebilir.

#### <kenar notu: XML yorumu>

Satır 2, dosyadaki içerikleri belgelemek için bir yorumdur. XML yorumları her zaman <!-- ile başlar ve --> ile biter.

#### <kenar boşluğu notu: DOCTYPE>

Satır 3-4, belgede kullanılan XHTML sürümünü belirtir. Bu, dökümanın sözdizimini doğrulamak için Web tarayıcısı tarafından kullanılabilir.

#### <kenar boşluğu notu: öge> n<kenar boşluğu notu: etiket>

Bir XML belgesi, etiketlerle tanımlanan öğelerden oluşur. Bir öge, bir başlangıç etiketi ve bir bitiş etiketi arasında yer alır. XML öğeleri, ağaç benzeri bir hiyerarşi içinde düzenlenmiştir. Öğeler alt öğeler içerebilir, ancak XML belgesinde yalnızca bir kök öge vardır. Tüm öğeler kök etiketinin içinde yer almalıdır. XHTML'deki kök öge, the html etiketi kullanılarak tanımlanır (satır 5).

XML'deki her etiket, başlangıç etiketi ve bitiş etiketi çiftinde kullanılmalıdır. Bir başlangıç etiketi < ile başlar, ardından etiket adı gelir ve > ile biter. Bir bitiş etiketi, başlangıç etiketiyle aynı olup, </ ile başlar. html için başlangıç ve bitiş etiketleri <html> ve </html>'dir.

#### <kenar boşluğu notu: html etiketi>

html ögesi, bir XHTML sayfasındaki tüm diğer öğeleri içeren kök öğedir. Başlangıç <html> etiketi (satır 5-6), belgede kullanılan öğeler için ad alanını belirtmek üzere bir veya daha fazla xmlns (XML ad alanı) niteliği içerebilir. Ad alanları, Java paketlerine benzer. Java paketleri, sınıfları düzenlemek ve isim çakışmalarını önlemek için kullanılır.

XHTML ad alanları, etiketleri düzenlemek ve isim çakışmalarını çözmek için kullanılır. Aynı isimde bir öge iki ad alanında tanımlanmışsa, onları ayırt etmek için tam nitelikli etiket adları kullanılabilir.

#### <kenar notu: xmlns>

Her xmlns niteliği, eşit işareti

(=) ile ayrılmış bir isim ve bir değere sahiptir. Aşağıdaki bildirim (satır 5)

```
xmlns="http://www.w3.org/1999/xhtml"
```

herhangi bir niteliksiz etiket adının varsayılan

standart xhtml ad alanında tanımlandığını belirtir.

Aşağıdaki bildirim (satır 6)

```
xmlns:h="http://java.sun.com/jsf/html"
```

JSF etiket kütüphanesinde tanımlanan etiketlerin

dokümanda kullanılmasına izin verir. Bu etiketlerin bir h ön eki olmalıdır.

#### <kenar notu: h:head>

Bir html elementi bir başlık ve gövde içerir. h:head elementi (satırlar 7-9) bir HTML başlık elementi tanımlar. Başlık genellikle

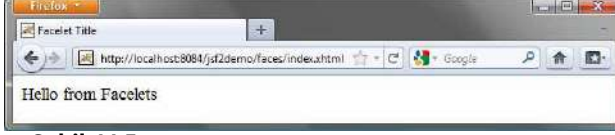
brauzer penceresinin başlık çubuğunda görüntülenir.

#### <kenar notu: h:gövde>

Bir h:body elementi sayfanın içeriğini tanımlar. Bu basit örnekte, Web tarayıcısında görüntülenecek bir dize içerir.

NOT: XML etiket adları büyük/küçük harf duyarlıdır, oysa HTML etiketleri değildir. Yani, `<html>` XML'de `<HTML>`'den farklıdır. XML'deki her başlangıç etiketi, eşleşen bir bitiş etiketine sahip olmalıdır; bazı HTML etiketlerinin bitiş etiketine ihtiyacı yoktur.

Artık index.xhtml dosyasına sağ tıklayarak JSP sayfasını görüntüleyebilirsiniz. Sayfa, Şekil 44.5'te gösterildiği gibi bir tarayıcıda görüntülenir.



**Şekil 44.5**  
*index.xhtml tarayıcıda görüntülenir.*

#### 44.2.3 JSF için Yönetilen JavaBeans

JSF uygulamaları, uygulamanın verilerini (modelde bulunan) grafik sunumdan (görünüm) ayıran Model-Görünüm-Denetleyici (MVC) mimarisi kullanılarak geliştirilir. Denetleyici, görünüm ile model arasındaki etkileşimleri koordine etmekten sorumlu olan JSF çerçevesidir.

JSF'de, facelet'ler verileri sunmak için görünüm olarak kullanılır. Veriler Java nesnelerinden elde edilir. Nesneler Java sınıfları kullanılarak tanımlanır. JSF'de, bir facelet'ten erişilen nesneler JavaBeans nesneleridir. JavaBeans ile tanışık değilseniz, lütfen Bölüm 36, "JavaBeans"i okuyun. Bu bölümdeki örneğimiz, mevcut zamanı görüntülemek için bir JSF facelet

geliştirmektedir. Mevcut zamanı bir dize olarak döndüren bir `getTime()` yöntemine sahip bir JavaBean oluşturacağız. Facelet, mevcut zamanı elde etmek için bu yöntemi çağıracaktır.

TimeBean adında bir JavaBean oluşturmak için adımlar şunlardır.

Adım 1. Proje düğümü jsf2demo'ya sağ tıklayın ve Şekil 44.6'da gösterildiği gibi bir bağlam menüsü görüntüleyin. Yeni JSF Yönetilen Bean'i seçmek için *Yeni, JSF Yönetilen Bean* seçeneğini tıklayın ve Şekil 44.7'de gösterilen Yeni JSF Yönetilen Bean iletişim kutusunu görüntüleyin. (Not: Eğer menüde JSF Yönetilen Bean'i görmüyorsanız, JSF Yönetilen Bean'i bulmak için *Diğer* seçeneğini seçin.)

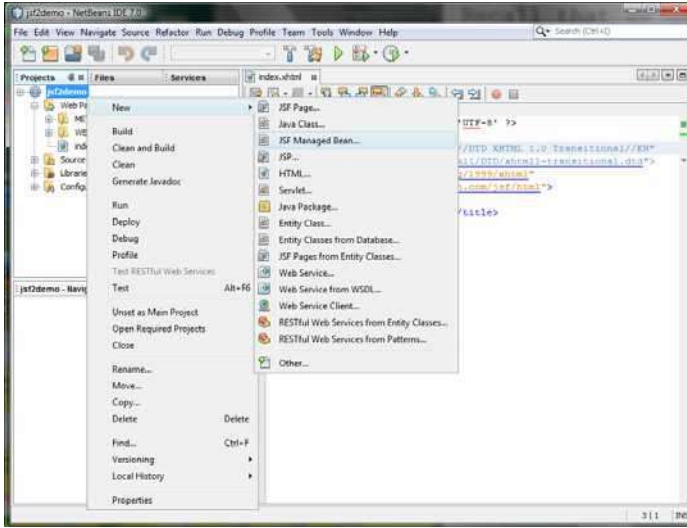
Adım 2. Aşağıdaki alanları girin ve seçin, Şekil 44.8'de gösterildiği gibi:

Sınıf Adı: TimeBean  
Paket: jsf2demo  
Ad: timeBean  
Kapsam: request

Tamamla *butonuna tıklayın* TimeBean.java dosyasını oluşturmak için, Şekil 44.9'da gösterildiği gibi.

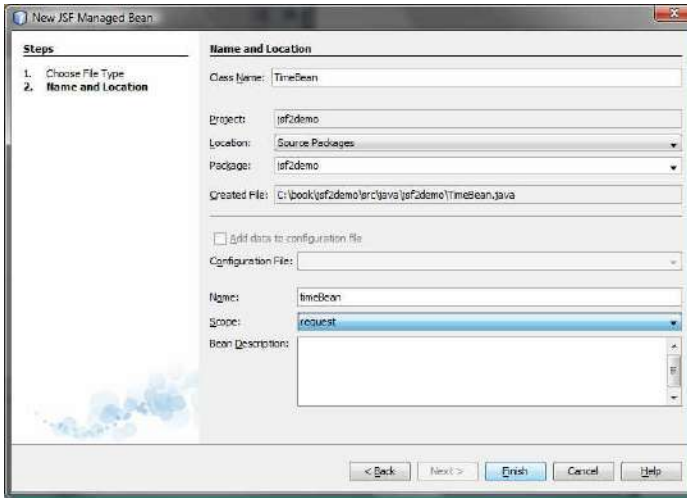
Adım 3. Mevcut zamanı döndüren `getTime()` metodunu ekleyin,

Liste 44.2'de gösterildiği gibi.



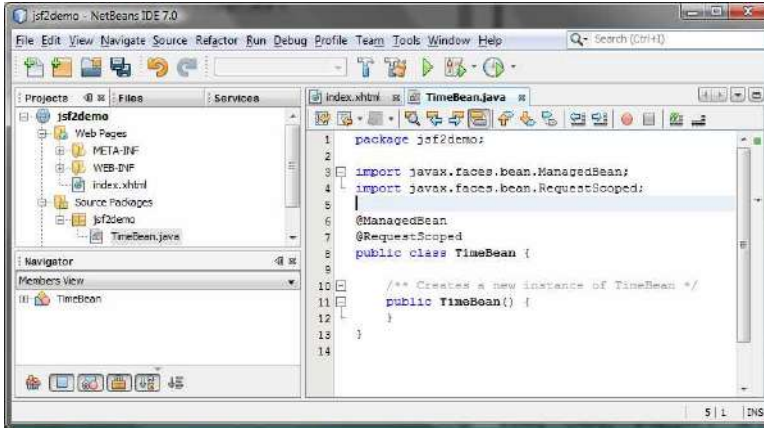
**Şekil 44.6**

*JSF için bir JavaBean oluşturmak üzere JSF Yönetilen Bean'i seçin.*



**Şekil 44.7**

*Bean için ad, konum, kapsam belirtin.*



Şekil 44.8

JSF için bir JavaBean oluşturuldu.

#### Listeleme 44.2 TimeBean.java

**<margin not satırı 6: @ManagedBean>n<margin not satırı 7: @RequestScoped>n<margin not satırı 9: time özelliği>**

paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar; ja  
vax.faces.bean.RequestScoped'i içe aktar;

@ManagedBean  
@RequestScoped  
genel sınıf TimeBean {n publi  
c String getTime() {n return ne  
w java.util.Date().toString();n }

}

**<kenar notu: @ManagedBean>**

TimeBean, uygulamada kullanılan TimeBean nesnelerini oluşturacak ve yönetec ek olan JSF çerçevesini belirten @ManagedBean anotasyonuna sahip bir JavaBeans'dir . 15. Bölümde @Override anotasyonunu kullanmayı öğrendiniz. @Override anotasyonu, anotasyonlu metodun bir üst sınıftaki bir metodu geçersiz kılmasının gerektiğini derleyiciye bildirir. @ManagedBean anotasyonu, bean'ın JSF facade'leri tarafından kullanılabilmesi için kodun üretilmesini derleyiciye bildirir.

**<kenar notu: @RequestScope>**

@RequestScope anotasyonu, JavaBeans nesnesinin kapsamının bir istek içinde olduğunu belirtir. JSP kapsamları, önceki bölümde tanıtılmıştır. Ayrıca, bir oturum veya tüm uygulama için kapsamı belirtmek üzere @SessionScope veya @ApplicationScope kullanabilirsiniz.

#### 44.2.4 JSF İfadeleri

Önceki bölümde, mevcut zamanı döndürmek için bir HTML dosyasında Java kodu girmek için JSP betimlemesi kullandığımızı hatırlayın. Ancak JSP betimlemesi JSF ile çalışmaz. JSF sayfasından mevcut zamanı nasıl görüntüleyebilirsiniz?

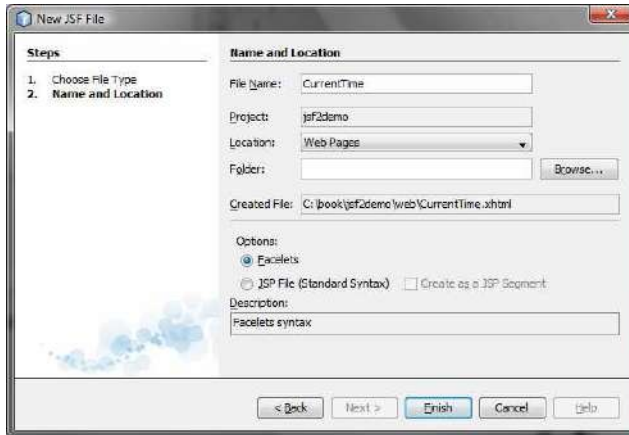
Mevcut zamanı, bir TimeBean nesnesinde getTime() metodunu çağırarak bir JSF ifadesi kullanarak görüntüleyebilirsiniz.

index.xhtml dosyasını bozmamak için, aşağıdaki gibi CurrentTime.xhtml adını da yeni bir JSF sayfası oluşturuyoruz:

Adım 1. Proje panelinde jsf2demo düğmesine sağ tıklayın, bir bağlam menüsü görüntülemek için *Yeni, JSF Sayfası* seçeneğini seçin ve Yeni JSF Dosyası iletişim kutusunu görüntüleyin, Şekil 44.9'da gösterildiği gibi.

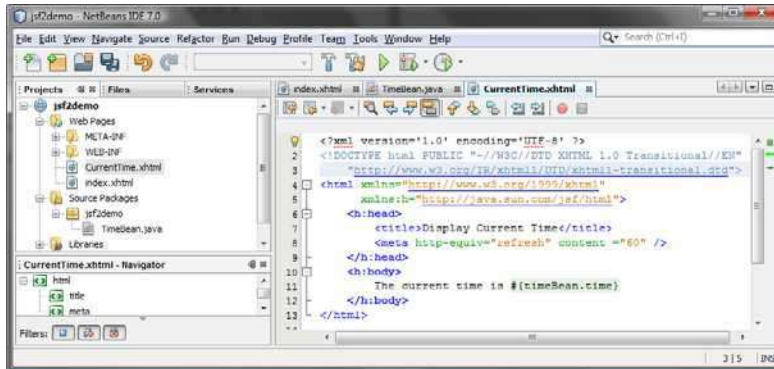
Adım 2. Dosya Adı alanına CurrentTime yazın, Facelets'i seçin ve Bitir seçeneğine tıklayarak CurrentTime.xhtml dosyasını oluşturun, Şekil 44.10'da gösterildiği gibi.

Adım 3. Mevcut zamanı almak için bir JSF ifadesi ekleyin, Şekil 44.3'te gösterildiği gibi.



Şekil 44.9

*Yeni JSF Dosyası iletişim kutusu, bir JSF sayfası oluşturmak için kullanılır.*



Şekil 44.10

*Yeni bir JSF sayfası olan CurrentTime oluşturuldu.*

Liste 44.3 CurrentTime.xhtml

*< margin note line 8: sayfayı yenile >*  
*< margin note line 11: JSF ifadesi >*

<xml version='1.0' encoding='UTF-8' >

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
 xmlns:h="http://java.sun.com/jsf/html">
 <h:head>
 <title>Mevcut Zamanı Göster</title>
 <meta http-equiv="refresh" content="60" />
 </h:head>
 <h:gövde>
 Mevcut zaman #{timeBean.time}
 </h:gövde>
</html>

```

8. satır, tarayıcıya her 60 saniyede bir yenilemesini söylemek için h:head etiketinin içinde bir meta etiket tanımlar. Bu satır ayrıca şu şekilde yazılabilir

```

<meta http-equiv="refresh" content="60"></ meta>

```

#### <kenar notu: boş öge>

Bir öge, başlangıç etiketi ile bitiş etiketi arasında içerik yoksa *boş öge* olarak adlandırılır. Boş bir öğede, veriler genellikle başlangıç etiketinde nitelikler olarak belirtilir. Kısalık için, bir boş ögeyi, başlangıç etiketinin sağ açılı parantezinin önüne bir eğik çizgi koyarak kapatabilirsiniz; bu, 8. satırda gösterildiği gibidir.

8. satır, mevcut zamanı elde etmek için #{timeBean.time} JSF ifadesini kullanır. time Bean, TimeBean sınıfının bir nesnesidir. Nesne adı, @ManagedBean anotasyonunda (Liste 44.2'deki 6. satır) aşağıdaki sözdizimi kullanılarak değiştirilebilir:

```
@ManagedBean(name="anyObjectName")
```

Varsayılan olarak nesne adı, ilk harfi küçük olacak şekilde sınıf adıdır.

time bir JavaBeans özelliği olduğunu unutmayın, çünkü getTime() yöntemi TimeBeans'de tanımlıdır. JSF ifadesi, mevcut zamanı elde etmek için ya özellik adını kullanabilir ya da yöntemi çağırabilir. Bu nedenle, aşağıdaki iki ifade de geçerlidir.

```

#{timeBean.time}
#{timeBean.getTime()}

```

JSF ifadesinin sözdizimi

```
#{expression}
```

JSF ifadeleri, JavaBeans nesnelerini facelet'lerle bağlar. Bu bölümdeki gelecek örneklerde JSF ifadelerinin daha fazla kullanımını göreceksiniz.

### 44.3 JSF GUI Bileşenleri

JSF, GUI bileşenlerini görüntülemek için birçok öge sağlar. Tablo 44.1, yaygın olarak kullanılan bazı öğeleri listelemektedir. h öneki ile başlayan etiketler, JSF HTML Etiket kütüphanesinde, f öneki ile başlayan etiketler, JSF Core Etiket kütüphanesinde.

#### Tablo 44.1

JSF GUI Form Elemanları



JSF Etiketi	Açıklama
h:form	bir sayfaya XHTML formu ekler.
h:panelGroup	Java akış düzeni konteynerine benzer.
h:panelGrid	Java ızgara düzeni konteynerine benzer.
h:inputText	girdi için bir metin kutusu görüntüler.
h:outputText	çıktı için bir metin kutusu görüntüler.
h:inputTextArea	girdi için bir metin alanı görüntüler.
h:inputSecret	şifre girişi için bir metin kutusu görüntüler.
h:outputLabel	bir etiket görüntüler.
h:outputLink	bir hiper metin bağlantısı görüntüler.
h:selectOneMenu	bir öğe seçmek için bir açılır kutu görüntüler.
h:selectOneRadio	bir dizi radyo düğmesi görüntüler.
h:selectBooleanCheckbox	bir onay kutusu görüntüler.
h:selectOneListbox	bir öğe seçmek için bir liste görüntüler.
h:selectManyListbox	birden fazla öğe seçmek için bir liste görüntüler.
	f:selectItem bir h:selectOneMenu, h:selectOneRadio veya h:selectManyListbox içindeki bir öğeyi belirtir.
h:message	girişi doğrulamak için bir mesaj görüntüler.
h:veriTablosu	bir veri tablosu görüntüler.
h:column	bir veri tablosundaki bir sütunu belirtir.
h:graphicImage	bir resim görüntüler.

Liste 44.4, bu öğelerin bazılarını kullanarak bir öğrenci kayıt formunu görüntüleyen bir örnektir; Şekil 44.11'de gösterilmiştir.

**Şekil 44.11**

*Bir öğrenci kayıt formu JSF öğeleri kullanılarak görüntülenmektedir.*

Liste 44.4 StudentRegistrationForm.xhtml

**<kenar notu satır 6: jsf çekirdek ad alanı>**

*<margin not satırı 14: graphImage>n<margin not satırı 18: h:panelGrid>n<margin not satırı 19: h:outputLabel>n<margin not satırı 20: h:inputText>n<margin not satırı 30: h:selectOneRadio>n<margin not satırı 31: f:selectItem>n<margin not satırı 41: h:selectOneMenu>n<margin not satırı 46: h:selectManyListBox>n<margin not satırı 56: h:selectManyCheckbox>n<margin not satırı 66: h:inputTextarea>n<margin not satırı 71: h:commandButton>*

```
<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
PE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.w3.org/1999/xhtml"
```

```
xmlns:h = "http://java.sun.com/jsf/html" n xmlns:
sf = "http://java.sun.com/jsf/core"> n <h:head>
```

```
<title>Öğrenci Kayıt Formu</title>
</h:head>
<h:gövde>
```

```
<h:form>
<!-- h:graphicImage kullanın -->
<h3>Öğrenci Kayıt Formu
```

```
<h:graphicImage name="usIcon.gif" library="image"/>
```

```
</h3>
```

```
<!-- h:panelGrid kullanın -->
```

```
<h:panelGrid columns="6" style="color:green">
```

```
<h:outputLabel value="Soyadı"/>
```

```
<h:inputText id="lastNameInputText" />
```

```
<h:outputLabel value="Ad" />
```

```
<h:inputText id="firstNameInputText" />
```

```
<h:outputLabel value="MI" />
```

```
<h:inputText id = "miInputText" size = "1" /> n </h:panelGrid>
```

```
<!-- Radyo düğmeleri kullanın -->
```

```
<h:panelGrid columns="2">
```

```
<h:outputLabel>Cinsiyet </h:outputLabel>
```

```
<h:selectOneRadio id="genderSelectOneRadio">
```

```
<f:selectItem itemValue="Erkek"
```

```
itemLabel="Erkek"/>
```

```
<f:selectItem itemValue="Kadın"
```

```
itemLabel="Kadın"/>
```

```
</h:selectOneRadio>
```

```
</h:panelGrid>
```

```
<!-- Kombinasyon kutusu ve liste kullanın -->
```

```
<h:panelGrid columns="4">
```

```
<h:outputLabel value="Ana Dal" />
```

```
<h:selectOneMenu id="majorSelectOneMenu">
```

```
<f:selectItem itemValue="Bilgisayar Bilimleri"/>
```

```
<f:selectItem itemValue="Matematik"/>
```

```
</h:selectOneMenu>
```

```
<h:outputLabel value="Yan Dal" />
```

```
<h:selectManyListBox id="minorSelectManyListBox">
```

```
<f:selectItem itemValue="Bilgisayar Bilimleri"/>
```

```
<f:selectItem itemValue="Matematik"/>
```

```

 <f:selectItem itemValue="İngilizce"/>
 </h:selectManyListbox>
 </h:panelGrid>

 <!-- Onay kutuları kullanın -->
 <h:panelGrid sütunları = "4">
 <h:outputLabel value="Hobi: "/>
 <h:selectManyCheckbox id="hobbySelectManyCheckbox">
 <f:selectItem itemValue="Tenis"/>
 <f:selectItem itemValue="Golf"/>
 <f:selectItem itemValue="Masa Tenisi"/>
 </h:selectManyCheckbox>
 </h:panelGrid>

 <!-- Metin alanı kullanın -->
 <h:panelGrid sütunları = "1">
 <h:outputLabel>Açıklamalar:</h:outputLabel>
 <h:inputTextarea id="remarksInputTextarea"
 style="width:400px; height:50px;" />
 </h:panelGrid>

 <!-- Komut düğmesi kullanın -->
 <h:commandButton value="Kaydet" />
 </h:form>
</h:body> n
</html>

```

#### <kenar notu: jsf çekirdek xmlns>

f öneki ile işaretlenmiş etiketler JSF çekirdek etiket kütüphanesindedir. Satır 6

```
xmlns:f="http://java.sun.com/jsf/core">
```

bu etiketler için kütüphaneyi bulur.

#### <kenar notu: h:graphicImage>

h:graphicImage etiketi, usIcon.gif dosyasındaki bir resmi görüntüler (satır 14). Dosya /resources/image klasöründe bulunmaktadır. JSF 2.0'da, tüm kaynaklar (resim dosyaları, ses dosyaları, CCS dosyaları)

Web Sayfaları düğümü altındaki kaynaklar klasörüne yerleştirilmelidir. Bu

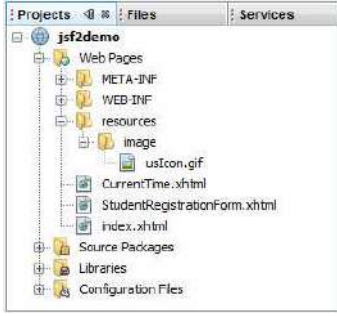
klasörleri aşağıdaki gibi oluşturabilirsiniz:

Adım 1: Proje panelinde Web Sayfaları düğmesine sağ tıklayın, bir bağlam menüsü görüntüleyin ve *Yeni, Klasör* seçeneğini seçerek Yeni Klasör iletişim kutusunu görüntüleyin. (Klasör bağlam menüsünde yoksa, *Diğer* seçeneğini seçerek bulabilirsiniz.)

Adım 2: Klasör Adı olarak kaynakları girin ve *Tamam* düğmesine tıklayarak kaynaklar klasörünü oluşturun, Şekil 44.12'de gösterildiği gibi.

Adım 3: Proje panelinde kaynaklar düğmesine sağ tıklayarak kaynaklar altında resim klasörünü oluşturun. Artık usImage.gif dosyasını

resim klasörüne yerleştirebilirsiniz.



**Şekil 44.12**

*Kaynaklar klasörü oluşturuldu.*

**<kenar boşluğu notu: h:panelGrid>**

JSF, alt öğeleri içermek ve düzenlemek için h:panelGrid ve h:panelGroup öğelerini sağlar. h:panelGrid öğeleri Java'nın ızgara düzen yöneticisi gibi bir ızgaraya yerleştirir. h:panelGroup öğeleri Java GUI akış düzen yöneticisi gibi yerleştirir. Satırlar 18-25, altı öğeyi (etiketler ve giriş metinleri) h:panelGrid içinde yerleştirir. Columns niteliği, ızgaradaki her satırın 6 sütun içerdiğini belirtir. Öğeler, yüzeyde görüldükleri sırayla soldan sağa doğru bir satıra yerleştirilir. Bir satır dolduğunda, öğeleri tutmak için yeni bir satır oluşturulur. Bu örnekte h:panelGrid kullandık. Öğelerin nasıl düzenleneceğini görmek için h:panelGroup ile değiştirebilirsiniz.

**<margin note: stil niteliği>**

JSF html etiketi ile stil niteliğini kullanarak öğenin ve alt öğelerinin CSS stilini belirtebilirsiniz. 8. satırdaki stil niteliği, bu h:panelGrid elementindeki tüm öğeler için yeşil rengi belirtir.

**<kenar boşluğu notu: h:outputLabel>**

h:outputLabel elementi, bir etiketi görüntülemek için kullanılır (19. satır). Etiketin metnini belirtmek için value niteliğini kullanabilirsiniz.

**<kenar notu: h:inputText>**

h:inputText elementi, kullanıcının bir metin girmesi için bir metin giriş kutusu görüntülemek için kullanılır (20. satır). id niteliği, diğer öğeler veya sonucu programı bu öğeyi referans almak için faydalıdır.

**<kenar notu: h:selectOneRadio>**

h:selectOneRadio elementi, bir grup radyo düğmesi görüntülemek için (kısım 30). Her radyo düğmesi, f:selectItem elementi kullanılarak tanımlanır (31-34. satırlar).

**<kenar notu: h:selectOneMenu>**

h:selectOneMenu elementi, bir açılır kutu görüntülemek için kullanılır (41. satır). Combo kutusundaki her bir öğe, f:selectItem öğesi kullanılarak tanımlanmıştır (42-43. satırlar).

**<kenar notu: h:selectManyListbox>**

h:selectManyListbox öğesi, kullanıcının bir listeden birden fazla öğe seçmesi için bir liste görüntülemek amacıyla kullanılır (46. satır). Listede her bir öğe, f:selectItem öğesi kullanılarak tanımlanmıştır (47-49. satırlar).

**<kenar boşluğu notu: h:selectManyCheckbox>**

h:selectManyCheckbox elementi, bir grup onay kutusunu görüntülemek için kullanılır (satır 56). Onay kutusundaki her bir öge, bir f:selectItem elementi kullanılarak tanımlanır (satırlar 57-59).

**<kenar notu: h:Metin alanını seçin>**

h:selectTextarea elementi, birden fazla satır girişi için bir metin alanı görüntülemek için kullanılır (satır 66). Metin alanının genişliğini ve yüksekliğini belirtmek için style özniteliğini kullanabilirsiniz (satır 67).

**<kenar notu: h:Metin alanını seçin>**

h:selectTextarea elementi, birden fazla satır girişi için bir metin alanı görüntülemek için kullanılır (satır 66). Style özniteliği, metin alanının genişliğini ve yüksekliğini belirtmek için kullanılır (satır 67).

**<kenar boşluğu notu: h:commandButton>**

h:commandButton elementi, bir düğme görüntülemek için kullanılır. Düğmeye tıklandığında, bir işlem gerçekleştirilir. Varsayılan işlem, sunucudan aynı sayfayı talep etmektir. Sonra ki bölüm, formun nasıl işleneceğini gösterir.

#### 44.4 Formun İşlenmesi

Önceki bölüm, yaygın JSF

elementlerini kullanarak bir formun nasıl görüntüleneceğini tanıttı. Bu bölüm, girişi nasıl elde edeceğinizi ve işleyeceğinizi gösterir.

Formdan girişi elde etmek için, her bir giriş ögesini bir yönetilen bean'deki bir özellik ile bağlayın. Şimdi, Liste 44.5'te gösterildiği gibi kayıt.

Liste 44.5 Registration.java

**<kenar notu satır 6: yönetilen bean>**

**<kenar notu satır 7: istek kapsamı>**

**<kenar notu satır 9: özellik soyadı>**

**<kenar notu satır 82: getResponse()>**

paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar; ja  
vax.faces.bean.RequestScoped'i içe aktar;

@ManagedBean

@RequestScoped

public class Kayıt {özel String las

tName'de;n Private String Firs

tName'de;ve Private String mi;

özel Dize cinsiyet;n özel Diz

e büyük;n özel Dize[] küçü

k;n özel Dize[] hobi;n özel Diz

e açıklamalar;n public String

getLastName() {n return last

Name;

;

public void setLastName(String lastName) {n this.last

Name = lastName;

;

```

 public String getFirstName() {n return
 FirstName;
 }

 public void setFirstName(String FirstName) {n this.first
 Name = FirstName;
 }

 public String getMi() {n retur
 n mi;
 }

 public void setMi(String mi) {n this.mi
 = mi;
 }

 public String getGender() {n retur
 n cinsiyet;
 }

 public void setGender(String cinsiyet) {n this.ge
 nder = cinsiyet;
 }

 public String getMajor() {n retur
 n major;
 }

 public void setMajor(String major) {n this.maj
 or = major;
 }

 public String[] getMinor() {n return
 minör;
 }

 public void setMinor(String[] minör) {n this.min
 or = minör;
 }

 public String[] getHobby() {n dönüş
 hobisi;
 }

 public void setHobby(String[] hobi) {n this.hobb
 y = hobi;
 }

 public String getRemarks() {n dönü
 ş açıklamaları;
 }

 public void setRemarks(String comments) {n this.r
 emarks = açıklamalar;
 }

 public String getResponse() {n if (last
 Name == null)
 return ""; // Talep yapılmamış
 else {
 String allMinor = "";
 }
}

```

```

 for (String s: minör) {n allMin
 or += s + " ";n }

String allHobby = "";n for (
String s: hobi) {n allHobby +=
 s + " ";n }

return "<p style=\"color:red\">Girdiğiniz
" +
 "Soyadı: " + lastName + "
" +
 "Ad: " + firstName + "
" +
 "MI: " + mi + "
" +
 "Cinsiyet: " + gender + "
" +
 "Bölüm: " + major + "
" +
 "Yan Dal: " + allMinor + "
" +
 "Hobi: " + allHobby + "
" +
 "Notlar: " + remarks + "</p>";
 }
 N }
}

```

#### <kenar notu: fasulye özellikleri>

Registration sınıfı, JSF kayıt formundaki öğelere bağlanacak olan lastName, firstName, mi, gender, major, minor ve remarks özelliklerini tanımlayan yönetilen bir bean'dir.

Kayıt formu artık Liste 44.6'da gösterildiği gibi yeniden tanımlanabilir. Şekil 44.13, yeni JSF sayfasının kullanıcı girişi gösterdiğini, Register butonuna tıkladıktan sonra gösterir.

Liste 44.6 ProcessStudentRegistrationForm.xhtml

```

<!-- margin note line 6: jsf core namespace -->
<!-- margin note line 21: soyadı bağla -->
<!-- margin note line 24: adı bağla -->
<!-- margin note line 27: mi bağla -->
<!-- margin note line 34: cinsiyeti bağla -->
<!-- margin note line 46: ana dalı bağla -->
<!-- margin note line 52: yan dalı bağla -->
<!-- margin note line 63: hobi bağla -->
<!-- margin note line 75: notlar bağla -->
<!-- margin note line 82: yanıt bağla -->

<?xml version='1.0' encoding='UTF-8'> <?xml version='1.0' encoding='UTF-8'>
<!DOCTYPE
PE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"
xmlns:f="http://java.sun.com/jsf/core">
<h:head>
<title>Öğrenci Kayıt Formu</title>
</h:head>
<h:gövde>
<h:form>
<!-- h:graphicImage kullanın -->
<h3>Öğrenci Kayıt Formu
<h:graphicImage name="usIcon.gif" library="image"/>
</h3>

```

```

<!-- h:panelGrid kullanın -->
<h:panelGrid columns="6" style="color:green">
 <h:outputLabel value="Soyadı"/>
 <h:inputText id="lastNameInputText"
 value="#{registration.lastName}"/>
 <h:outputLabel value="Ad" />
 <h:inputText id="firstNameInputText"
 value="#{registration.firstName}"/>
 <h:outputLabel value="MI" />
 <h:inputText id="miInputText" size="1"
 value="#{registration.mi}"/>
</h:panelGrid>

<!-- Radyo düğmeleri kullanın -->
<h:panelGrid columns="2">
 <h:outputLabel>Cinsiyet </h:outputLabel>
 <h:selectOneRadio id="genderSelectOneRadio"
 value="#{registration.gender}">
 <f:selectItem itemValue="Erkek"
 itemLabel="Erkek"/>
 <f:selectItem itemValue="Kadın"
 itemLabel="Kadın"/>
 </h:selectOneRadio>
</h:panelGrid>

<!-- Kombinasyon kutusu ve liste kullanın -->
<h:panelGrid columns="4">
 <h:outputLabel value="Bölüm" />
 <h:selectOneMenu id="majorSelectOneMenu"
 value="#{registration.major}">
 <f:selectItem itemValue="Bilgisayar Bilimleri"/>
 <f:selectItem itemValue="Matematik"/>
 </h:selectOneMenu>
 <h:outputLabel value="Yan Dal" />
 <h:selectManyListbox id="minorSelectManyListbox"
 value="#{registration.minor}">
 <f:selectItem itemValue="Bilgisayar Bilimleri"/>
 <f:selectItem itemValue="Matematik"/>
 <f:selectItem itemValue="İngilizce"/>
 </h:selectManyListbox>
</h:panelGrid>

<!-- Onay kutuları kullanın -->
<h:panelGrid sütunları = "4">
 <h:outputLabel value="Hobi: "/>
 <h:selectManyCheckbox id="hobbySelectManyCheckbox"
 value="#{registration.hobby}">
 <f:selectItem itemValue="Tenis"/>
 <f:selectItem itemValue="Golf"/>
 <f:selectItem itemValue="Masa Tenisi"/>
 </h:selectManyCheckbox>
</h:panelGrid>

<!-- Metin alanı kullanın -->
<h:panelGrid sütunları = "1">
 <h:outputLabel>Açıklamalar:</h:outputLabel>
 <h:inputTextarea id="remarksInputTextarea"
 style="width:400px; height:50px;"
 value="#{registration.remarks}"/>
</h:panelGrid>

```



```

<!-- Komut düğmesini kullan -->
<h:commandButton value="Kaydol" />

<h:outputText kaçış = "false" style = "renk: kırmızı" n değe
r = "#{registration.response}" /> n </h:form
>
</h:gövde>
</html>

```

**Student Registration Form**

Last Name: Yao First Name: John MI: P

Gender: ☒ Male ☐ Female

Major: Mathematics Minor: Computer Science

Hobby: ☒ Tennis ☐ Golf ☒ Ping Pong

Remarks: Done

Register

You entered  
 Last Name: Yao  
 First Name: John  
 MI: P  
 Gender: Male  
 Major: Mathematics  
 Minor: Computer Science English  
 Hobby: Tennis Ping Pong  
 Remarks: Done

**Şekil 44.13**

*Kullanıcı girişi,*

*Kaydol düğmesine tıkladıktan sonra toplanır ve görüntülenir.*

#### **<kenar notu: giriş metinlerini bağlama>**

Bu listedeki yeni JSF formu, `h:inputText` öğesini soyadı, adı ve mi ile yönetilen bean'deki `lastName`, `firstName` ve `mi` özelliklerine bağlar (22, 24, 27. satırlar). Kaydol düğmesine tıklan diğında, sayfa sunucuya gönderilir ve yönetilen bean'deki özellikleri ayarlamak için `set` yöntemleri çağrılır.

#### **<kenar notu: radyo düğmelerini bağlama>**

`h:selectOneRadio` öğesi cinsiyet özelliğine bağlanmıştır (34. satır). Her radyo düğmesinin bir `itemValue`'si vardır. Seçilen radyo düğmesinin `itemValue`'si, sayfa sunucuya gönderildiğinde bean'deki cinsiyet özelliğine ayarlanır.

#### **<kenar notu: bağlama kombinasyon kutusu>**

`h:selectOneMenu` öğesi, major özelliğine bağlanmıştır (satır 46). Sayfa sunucuya gönderildiğinde, seçilen öğe bir string olarak döndürülür ve major özelliğine ayarlanır.

**<kenar notu: bağlama liste kutusu>**

h:selectManyListbox elementi minor özelliğine bağlanmıştır (satır 52). Sayfa sunucuya gönderildiğinde, seçilen öğeler bir dizi string olarak döndürülür ve minor özelliği ne ayarlanır.

**<margin note: onay kutularını bağlama>**

h:selectManyCheckbox elementi hobby özelliğine bağlanmıştır (satır 63). Sayfa sunucuya gönderildiğinde, işaretli kutular itemValues dizisi olarak döndürülür ve hobby özelliğine ayarlanır.

**<margin note: metin alanını bağlama>**

h:selectTextarea elementi remarks özelliğine bağlanmıştır (satır 75). Sayfa sunucuya gönderildiğinde, metin alanındaki içerik bir string olarak döndürülür ve remarks özelliğine ayarlanır.

**<margin note: yanıtı bağlama>**

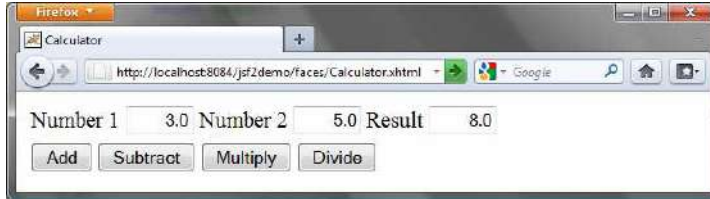
h:outputText elementi response özelliğine bağlanmıştır (satır 82). Bu, bean içinde yalnızca okunabilir bir özelliktir. lastName null ise ""'dır (satır 83-84, Listing 44.5). Sayfa istemciye döndüğünde, yanıt özelliği değeri çıktı metin elementinde görüntülenir (satır 82).

**<margin note: kaçış niteliği>**

h:outputText öğesinin escape niteliği false olarak ayarlanmıştır (satır 81) ve içeriğin HTML'de görüntülenmesine olanak tanır. Varsayılan olarak, escape niteliği true'dur, bu da içeriğin normal metin olarak kabul edildiğini gösterir.

**44.5 Vaka Çalışması: Hesap Makinesi**

Bu bölüm, toplama, çıkartma, çarpma ve bölme işlemlerini gerçekleştirmek için JSF kullanarak bir hesap makinesi geliştirmektedir; Şekil 44.21'de gösterilmiştir.



**Şekil 44.21**

*Bu JSF uygulaması, toplama, çıkartma, çarpma ve bölme işlemlerini gerçekleştirmenizi sağlar.*

Bu projeyi geliştirmek için adımlar şunlardır:

**<margin note: yönetilen bean oluşturma>**

Adım 1. Listing 44.7'de gösterildiği gibi, istek kapsamında Calculator adında yeni bir yönetilen bean oluşturun, Calculator.java.

**<margin note: JSF facelet oluşturma>**

Adım 2. Listing 44.8'de gösterildiği gibi, Calculator adında bir JSP facelet oluşturun, Calculator.xhtml.

Listeleme 44.7 Calculator.java

**<margin note line 9: mülk numarası1>**

<margin note line 10: mülk numarası2> \_\_\_\_\_

<margin note line 11: mülk sonucu> \_\_\_\_\_

<margin note line 40: ekle> \_\_\_\_\_

<margin note line 44: çıkar> \_\_\_\_\_

<margin note line 48: böl> \_\_\_\_\_

<margin note line 52: çarp> \_\_\_\_\_

paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar; ja  
vax.faces.bean.RequestScoped'i içe aktar;

@ManagedBean

@RequestScoped

public class HesapMakinesi {

private Double number1;

private Double number2;

private Double result;

public HesapMakinesi() {

}

public Double getNumber1() {n dö  
nüş numarası1;

}

public Çift kapı Numarası 2() {ve dö  
nüş numarası 2;

}

public Double getResult() {n dönü  
ş sonucu;

}

public void setNumber1(Double number1) {n this.  
number1 = number1;

}

public void setNumber2(Double number 2) {on thi  
s.number 2 = number 2;

}

public void setResult(Double result) {n this.resul  
t = result;

}

public void add() {n son  
uç = sayı1 + sayı2;n }

}

public void subtract() {n sonuç  
= sayı1 - sayı2;n }

}

genel geçersiz bölme () {  
sonuç = sayı1 / sayı2;n }

}

public void çarpı() {n sonuç = s  
ayı1 \* sayı2;

}

}  
}

Yönetilen bean'ın number1, number2 ve result adında üç özelliği vardır (9-38. satırlar). add(), subtract(), divide() ve multiply() yöntemleri number1 ile number2'yi toplar, çıkarır, çarpır ve böler ve sonucu result'a atar (40-54. satırlar).

Liste 44.8 Calculator.xhtml

*<kenar notu satırı 14: sağa hizala> n<kenar notu satırı 15: ciltleme metni girişi> n<kenar notu satırı 28: eylem>*

```
<xml version='1.0' encoding='UTF-8'>n<!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" n "http://www.w3.or
g/TR/xhtml1/DTD/xhtml1-transitional.dtd">n<html xmlns="http://www.w
3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html">n <h:hea
d>
<title>Hesap Makinesi</title>
</h:head>
<h:gövde>
<h:form>
<h:panelGrid sütunları = "6">
<h:outputLabel value="Numara 1"/>
<h:inputText id="number1InputText" size = "4"
style="text-align: right"
value="#{calculator.number1}"/>
<h:outputLabel value="Sayı 2" />
<h:inputText id="number2InputText" size = "4"
style="text-align: right"
value="#{calculator.number2}"/>
<h:outputLabel value="Sonuç" />
<h:inputText id="resultInputText" size = "4"
style="text-align: right"
value="#{calculator.result}"/>n </pa
nelGrid>
<h:panelGrid sütunları = "4">
<h:commandButton value="Ekle"
action ="#{calculator.add}"/>
<h:commandButton value="Çıkar"
action ="#{calculator.subtract}"/>
<h:commandButton value="Çarp"
action ="#{calculator.multiply}"/>
<h:commandButton value="Böl"
action ="#{calculator.divide}"/>
</h:panelGrid>
</h:form>
</h:gövde>
</html>
```

Üç metin girişi bileşeni ve etiketleri grid panelinde yerleştirilmiştir (satırlar 11-24). Dört buton bileşeni grid panelinde yerleştirilmiştir (satırlar 26-35).

Bean özelliği number1, Sayı 1 için metin girişine bağlanmıştır (satır 15). CSS stili text-align: right (satır 14), metnin giriş kutusunda sağa yaslanmış olduğunu belirtir.

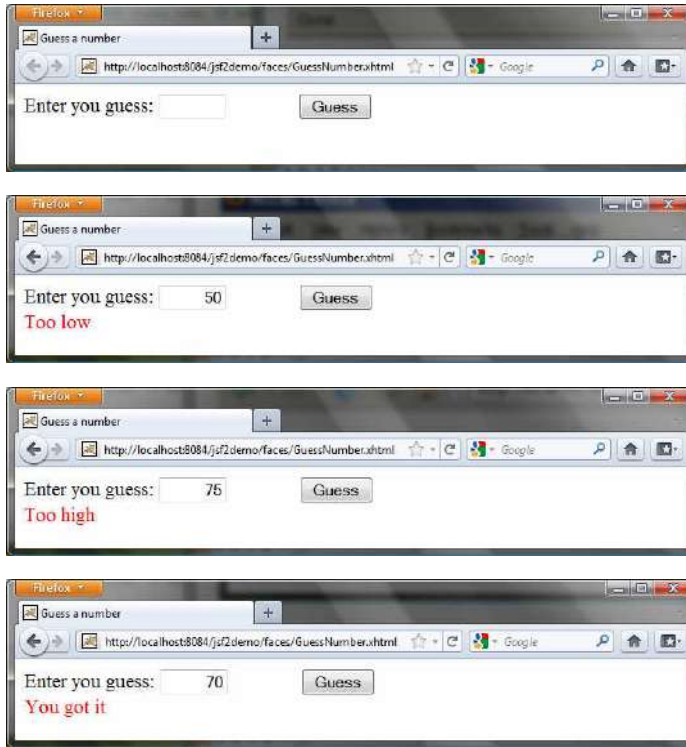
Butonun *Ekle* eylem niteliği,

hesap makinesi bean'inde (satır 28) *ekle* yöntemine ayarlanmıştır. *Ekle* butonuna tıklandığında, bean'deki *ekle* yöntemi *number1* ile *number2* toplamak için çağrılır ve sonuç *result*'a atanır. Sonuç özelliği *Result* girdi metnine bağlı olduğundan, yeni sonuç artık *metin* girdi alanında görüntülenir.

#### 44.6 Oturum Takibi

Bölüm 43, "JSP," JavaBeans nesnelerini farklı sayfalar arasında paylaşarak JavaBeans kullanarak oturum takibini tanıtmıştır. JavaBeans nesnelerini uygulama kapsamı, oturum kapsamı, sayfa kapsamı veya istek kapsamı olarak belirtebilirsiniz. JSF, JavaBeans ile oturum takibini uygulama kapsamı, oturum kapsamı ve istek kapsamı düzeyinde destekler. Ayrıca, JSF 2.0, bean'in görünümde kaldığı sürece canlı kalmasını sağlayan görünüm kapsamını destekler. Görünüm kapsamı, oturum ve istek kapsamı arasında.

Aşağıdaki örneği düşünün; bu örnek kullanıcıdan bir sayı tahmin etmesini ister. Sayfa başladığında, program rastgele 0 ile 99 arasında bir sayı üretir. Bu sayı oturumda saklanır. Kullanıcı bir tahmin girdiğinde, program tahmini oturumdaki rastgele sayı ile kontrol eder ve kullanıcıya tahminin çok yüksek, çok düşük veya tam doğru olup olmadığını söyler; bu durum Şekil 44.14'te gösterilmiştir.



**Şekil 44.14**

*Kullanıcı bir tahminde bulunur ve program sonucu gösterir.*

Bu projeyi geliştirmek için adımlar şunlardır:

#### **<margin note: yönetilen bean oluştur>**

Adım 1. Görünüm kapsamına sahip *GuessNumber* adında yeni bir yönetilen bean oluşturun. Şekil 44.9'da gösterildiği gibi, *GuessNumber.java*.

*<margin note: JSF facelet oluştur>*

Adım 2. GuessNumber adında bir JSP facelet oluşturun

Şekil 44.10'da gösterildiği gibi, GuessNumber.xhtml.

Liste 44.9 GuessNumber.java

*<margin note line 7: görünüm kapsamı>*

*<margin note line 9: rastgele sayı>*

*<margin note line 10: kullanıcı tarafından tahmin>*

*<margin note line 13: rastgele sayı oluştur>*

*<margin note line 16: al metodu>*

*<margin note line 20: ayar metodu>*

*<margin note line 24: yanıt al>*

*<margin note line 28: tahmini kontrol et>*

paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar; ja  
vax.faces.bean.ViewScoped'i içe aktar;

@ManagedBean

@ViewScoped

genel sınıf Tahmin Numarası {ve  
özel int numarası; özel  
String'de tahmin String;

public GuessNumber() {n sa  
yı = (int)(Math.random() \* 100);n }

public String getGuessString() {n return  
tahminString;

public void setGuessString(String tahminString) {n this.guess  
String = tahminString;

public String getResponse() {  
if (guessString == null)  
return ""; // Kullanıcı girişi henüz yok

int guess = Integer.parseInt(guessString);  
if (guess < number)  
return "Çok düşük";  
else if (guess == number)  
return "Buldun";  
else  
return "Çok yüksek";  
}

Yönetilen bean, bean için view scope'u ayarlamak üzere @ViewScope notasyonunu (satır 5) kullanır. Görünüm kapsamı bu proje için en uygun olanıdır. Bean, görünüm değiştiği sürece hayattadır. Bean, sayfa ilk kez görüntülendiğinde oluşturulur. Bean oluşturulduğunda 0 ile 99 arasında rastgele bir sayı number'a (satır 13) atanır. Bu sayı, bean hayatta olduğu sürece değişmeyecektir.

getResponse metodu, kullanıcı girişinden guessString'i bir tam sayıya (satır 28) dönüştürür ve tahminin çok düşük (satır 30), çok yüksek (satır 34) veya tam doğru (satır 32) olup olmadığını belirler.

Liste 44.10 GuessNumber.xhtml

*<margin note line 14: metin girişini bağla>*

*<margin note line 18: metin çıkışını bağla>*

```
<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
PE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.o
rg/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.
w3.org/1999/xhtml"
 xmlns:h="http://java.sun.com/jsf/html"> n <h:hea
d>
 <title>Bir sayı tahmin et</title>
 </h:head>
 <h:gövde>
 <h:form>
 <h:outputLabel value="Tahmininizi girin: "/>
 <h:inputText style="text-align: right; width: 50px"
 id="guessInputText"
 value="#{guessNumber.guessString}"/>
 <h:commandButton style="margin-left: 60px" value="Tahmin Et" />

 <h:outputText style="renk: kırmızı"
 value="#{guessNumber.response}" />
 </h:form>
 </h:gövde>
</html>
```

Bean özelliği guessString, metin girişine bağlıdır (satır 14). CSS stili text-align: right (satır 13), metnin giriş kutusunda sağa yaslandığını belirtir.

CSS stili margin-left: 60px (satır 15), komut butonunun sol kenar boşluğunun 60 piksel olduğunu belirtir.

Bean özelliği response, metin çıkışına bağlıdır (satır 18). CSS stili color: red (satır 17), metnin çıkış kutusunda kırmızı olarak görüntüleneceğini belirtir.

#### *<kenar notu: kapsam>*

Proje **view** kapsamını kullanıyor. Kapsam request kapsamına değiştirilirse ne olur? Sayfa her yenilendiğinde, JSF yeni bir bean oluşturur ve yeni bir rastgele sayı atar. Kapsam session kapsamına değiştirilirse ne olur? Bean, tarayıcı açık olduğu sürece yasak kalır. Kapsam **application** kapsamına değiştirilirse ne olur? Bean, uygulama sunucudan başlatıldığında bir kez oluşturulacaktır.

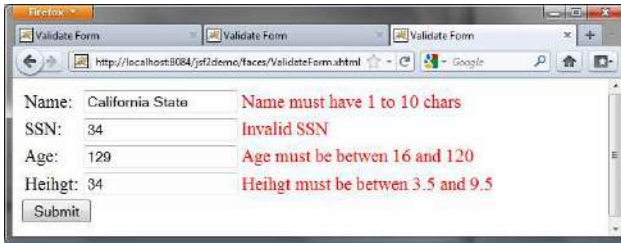
#### **44.7 Giriş Doğrulama**

Önceki GuessNumber sayfasında, *Guess* butonuna tıklamadan önce giriş kutusuna bir tam sayı olmayan bir değer girerseniz bir hata oluşur. Sorunu düzeltmenin basit bir yolu, herhangi bir event'i işlemden önce metin alanını kontrol etmektir. JSF, giriş doğrulama için birkaç kullanışlı ve güçlü yol sağlar. JSF Core Tag Kütüphanesinde standart doğrulayıcı etiketlerini kullanabilir veya özel doğrulayıcılar oluşturabilirsiniz. Tablo 44.2, bazı JSF giriş doğrulayıcı etiketlerini listelemektedir.

**Tablo 44.2***JSF Girdi Doğrulayıcı Etiketleri*

JSF Etiketi	Açıklama
f:validateLength	girdi uzunluğunu doğrular.
f:validateDoubleRange	sayısal girdinin kabul edilebilir çift değer aralığında olup olmadığını doğrular.
f:validateLongRange	sayısal girdinin kabul edilebilir uzun değer aralığında olup olmadığını doğrular.
f:validateRequired	bir alanın boş olmadığını doğrular.
f:validateRegex	girdinin bir düzenli ifadeye uyup uymadığını doğrular.
f:validateBean	özel doğrulama yapmak için bir bean içinde özel bir yöntemi çağırır.

Aşağıdaki örneği düşünün; bu örnek, kullanıcı girdisi toplamak için bir formu gösterir, Şekil 44.15'te gösterildiği gibi. Formdaki tüm metin alanları doldurulmalıdır. Aksi takdirde, hata mesajları görüntülenir. SSN doğru formatta olmalıdır. Aksi takdirde, bir hata görüntülenir. Tüm girdiler doğruysa, Gönder butonuna tıklamak, sonucu bir çıktı metninde gösterir, Şekil 44.16'da gösterildiği gibi.

*(a) Girdi gerekli olduğunda ancak boşsa, gerekli mesajlar görüntülenir.**(b) Girdi yanlışsa hata mesajları görüntülenir.***Şekil 44.15***Girdi alanları doğrulanır.*





**Şekil 44.16**

*Doğru girdi değerleri görüntülenir.*

Bu projeyi oluşturmak için adımlar şunlardır.

Adım 1. Listing 44.11'de gösterildiği gibi `ValidateForm` adında yeni bir sayfa oluşturun.

Adım 2.

Liste 44.12'de gösterildiği gibi `ValidateForm` adında yeni bir yönetilen bean oluşturun.

Liste 44.11 `ValidateForm.xhtml`

*<margin note line 14: zorunlu girdi>*  
*<margin note line 15: zorunlu mesaj>*  
*<margin note line 16: doğrulayıcı mesaj>*  
*<margin note line 18: uzunluğu doğrula>*  
*<margin note line 20: mesaj ögesi>*  
*<margin note line 27: regex'i doğrula>*  
*<margin note line 36: tam sayı aralığını doğrula>*  
*<margin note line 45: ondalık aralığını doğrula>*

```
<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.or
g/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.w
3.org/1999/xhtml"
xmlns:h = "http://java.sun.com/jsf/html" n xmlns
s:f = "http://java.sun.com/jsf/core"> n <h:head>

<title>Formu Doğrula</title>
</h:head>
<h:gövde>
<h:form>
<h:panelGrid sütunları = "3">
<h:outputLabel value="İsim:"/>
<h:inputText id="nameInputText" required="true"
requiredMessage="İsim zorunludur"
validatorMessage="İsim 1 ile 10 karakter arasında olmalıdır"
value="#{validateForm.name}">
<f:validateLength minimum = "1" maksimum = "10" /> n
</h:inputText>
<h:message for = "nameInputText" style = "renk: kırmızı"/> n <
h:outputLabel value = "SSN:" />
<h:inputText id="ssnInputText" required="true"
requiredMessage="SSN zorunludur"
validatorMessage="Geçersiz SSN"
value="#{validateForm.ssn}">
<f:validateRegex pattern="[\d]{3}-[\d]{2}-[\d]{4}" />
</h:inputText>
<h:message for = "ssnInputText" style = "renk: kırmızı"/>
```

```

<h:outputLabel value="Yaş:" />
<h:inputText id="ageInputText" required="true"
 requiredMessage="Yaş gereklidir"
 validatorMessage="Yaş 16 ile 120 arasında olmalıdır"
 value="#{validateForm.ageString}">
 <f:validateLongRange minimum = "16" maksimum = "120"/> n
</h:inputText>
<h:message for="ageInputText" style="color:red"/>

 <h:outputLabel value="Boy:" />
 <h:inputText id="heightInputText" required="true"
 requiredMessage="Boy gereklidir"
 validatorMessage="Boy 3.5 ile 9.5 arasında olmalıdır"
 value="#{validateForm.heightString}">
 <f:validateDoubleRange minimum = "3,5" maksimum = "9,5"/> n <
 /h:inputText>
 <h:message for="heightInputText" style="color:red"/> n </h:panel
Grid>

<h:commandButton value="Gönder" />

 <h:outputText style="color:red"
 value="#{validateForm.response}" />
</h:form>
</h:gövde>
</html>

```

**<kenar notu: gerekli öznelik> n<kenar n  
otu: gerekliMesaaj>**

Her bir giriş metin alanı için, bir giriş değerinin alan için gerekli olduğunu belirtmek üzere required niteliğini true (satır 14) olarak ayarlayın. Gerekli bir giriş alanı boş olduğunda, requiredMessage görüntülenir (satır 15).

**<margin note: validatorMessage>n<m  
argin note: f:validateLength>**

validatorMessage niteliği, girdi alanı geçersiz olduğunda gösterilecek bir mesajı belirtir (satır 16). f:validateLength etiketi, girdinin minimum veya maksimum uzunluğunu belirtir (satır 18). JSF, girdi uzunluğunun geçerli olup olmadığını belirleyecektir.

**<kenar notu: h:mesaj>**

h:message ögesi, girdi geçersiz olduğunda validatorMessage gösterir. Ögenin for n iteliği, mesajın gösterileceği ögenin kimliğini belirtir (satır 20).

**<kenar notu: f:validateRegex>**

f:validateRegex etiketi, girdi doğrulamak için bir düzenli ifade belirtir (satır 27). Düzenli ifadeler hakkında bilgi için Ek III.H.

**<kenar notu: f:validateLongRange>**

f:validateLongRange etiketi, minimum ve maksimum niteliğini kullanarak bir tam sayı girişi için bir aralık belirtir (satır 36). Bu projede, geçerli bir yaş değeri 16 ile 120 arasındadır.

**<kenar notu: f:validateDoubleRange>**

f:validateDoubleRange etiketi, minimum ve maksimum niteliğini kullanarak bir double girişi için bir aralık belirtir (satır 36). Bu projede, geçerli bir yükseklik değeri 3.5 ile 9.5 arasındadır.

Listeleme 44.12 ValidateForm.java

<margin note line 49: bazı girişler ayarlanmamış>

```
paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar; ja
vax.faces.bean.RequestScoped'i içe aktar;

@ManagedBean
@RequestScoped
genel sınıf ValidateForm {n özel Di
 ze adı;n özel Dize ssn;n ö
 zel Dize ageString;n öze
 l Dize heightString;

 public String getName() {n dön
 ũş adı;

 }

 public void setName(Dize adı) {n this.name
 = name;

 }

 public String getSsn() {n retur
 n ssn;

 }

 public void setSsn(String ssn) {n this.ssn
 = ssn;

 }

 public String getAgeString() {n return
 ageString;

 }

 public void setAgeString(String ageString) {n this.ageSt
 ring = ageString;

 }

 public String getHeightString() {n return
 heightString;

 }

 public void setHeightString(String heightString) {n this.heightSt
 ring = heightString;

 }

 public String getResponse() {n if (na
 me == null || ssn == null || ageString == null n || heightString ==
 null) {
 dönüş "" ;n }

 başka {
 return "Girdiğiniz " +
 " İsim: " + name +
 " SSN: " + ssn +
 " Yaş: " + ageString +
 " Boy: " + heightString;

 }
}
```

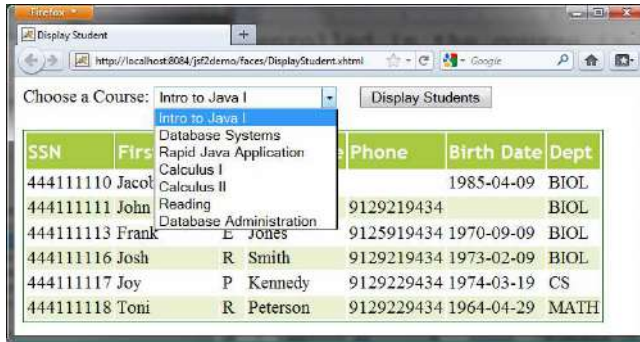
```
}
}
```

Geçersiz bir girdi varsa, değeri bean'e ayarlanmamaktadır. Bu nedenle yalnızca tüm girdiler doğru olduğunda, `getResponse()` metodu tüm girdi değerlerini döndürecektir (satırlar 46-58)

#### 41.8 Veritabanını Facets ile Bağlama

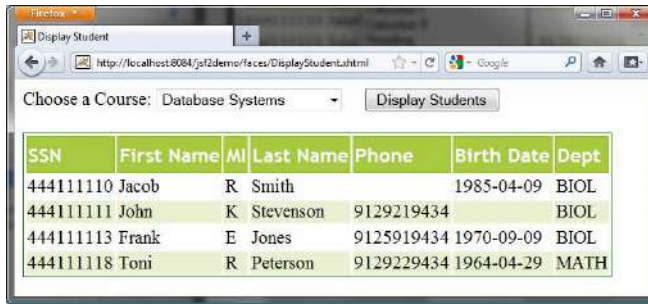
Sıklıkla bir Web sayfasından veritabanına erişmeniz gerekir. Bu bölüm, veritabanlarını kullanarak Web uygulamaları oluşturma örnekleri vermektedir.

Aşağıdaki örneği düşünün; bu örnek, kullanıcının bir dersi seçmesine olanak tanır, Şekil 44.17'de gösterildiği gibi. Bir ders, açılır listeden seçildikten sonra, derse kayıtlı öğrenciler tabloda gösterilir, Şekil 41.18'de gösterildiği gibi. Bu örnekte, Kurs tablosundaki tüm ders b aşlıkları, kombinasyon kutusuna bağlanmıştır ve derse kayıtlı öğrenciler için sorgu sonu cu tabloya bağlanmıştır.



Şekil 44.17

*Bir dersi seçmeniz ve derse kayıtlı öğrencileri görüntülemeniz gerekiyor.*



Şekil 44.18

*Tablo, derse kayıtlı öğrencileri gösterir.*

Bu projeyi oluşturmak için adımlar şunlardır:

#### <margin note: Yönetilen bean>

Adım 1. Uygulama kapsamına sahip CourseName adında bir yönetilen bean oluşturun, Liste 44.13'te gösterildiği gibi.

#### <margin note: JSF sayfası>

Adım 2. Liste 44.14'te gösterildiği gibi DisplayStudent adında bir JSP sayfası oluşturun.

<margin note: stil sayfası>

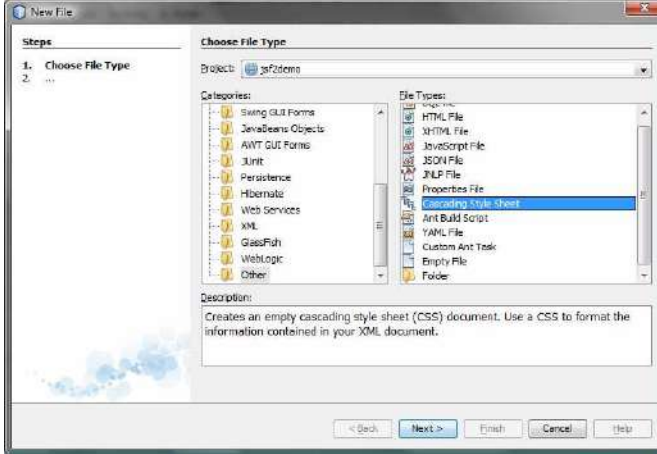
Adım 3. Tabloyu biçimlendirmek için aşağıdaki gibi bir akışkan stil sayfası oluşturun :

Adım 3.1. Yeni Dosya iletişim kutusunu görüntülemek için kaynaklar düğmesine sağ tıklayın ve *Yeni, Diğerleri* seçeneğini seçin; Şekil 44.19'da gösterildiği gibi.

Adım 3.2. Kategoriler bölümünde Diğer seçeneğini ve Dosya Türleri bölümünde Akışkan Stil Sayfası'nı seçerek Yeni Akışkan Stil Sayfası iletişim kutusunu görüntüleyin; Şekil 44.20'de gösterildiği gibi.

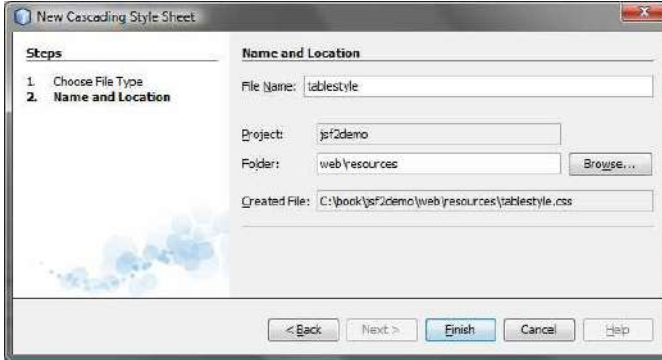
Adım 3.3. Dosya Adı olarak tablestyle girin ve *Bitir* butonuna tıklayarak tablestyle.css dosyasını kaynaklar düğümü altında oluşturun.

Adım 3.4. CSS stilini Liste 44.15'te gösterildiği gibi tanımlayın.



Şekil 44.19

NetBeans'te Web projesi için CSS dosyaları oluşturabilirsiniz.



Şekil 44.20

Yeni Kaskad Stil Sayfası iletişim kutusu yeni bir stil sayfası dosyası oluşturur.

Liste 44.13 CourseName.java

<margin note line 9: uygulama kapsamı>

<margin note line 18: JDBC'yi başlat>

<margin note line 27: veritabanına bağlan>

<margin note line 32: ders başlıklarını al>

<margin note line 34: SQL'i çalıştır>  
<margin note line 39: başlıklar dizisi>  
<margin note line 72: öğrenciyi al>  
<margin note line 77: bir dersi ayarla>  
<margin note line 83: satır kümesini al>

paket jsf2demo;

javax.faces.bean.ManagedBean'i içe aktar;ni  
import javax.faces.bean.ApplicationScoped;niimport j  
ava.sql.\*;  
javax.sql.rowset.CachedRowSet'yi içe aktar;

@ManagedBean

@UygulamaKapsamlı

genel sınıf DersAdı {

private PreparedStatement studentStatement = null;

private CachedRowSet rowSet; // Kurs başlıkları için

private String choice; // Seçilen kurs

private String[] titles; // Kurs başlıkları

/\*\* CourseName sınıfının yeni bir örneğini oluşturur \*/

public CourseName() {

initializeJdbc();n }

/\*\* Veritabanı bağlantısını başlatır \*/

private void initializeJdbc() {

dene {

Class.forName("com.mysql.jdbc.Driver");

// Örnek veritabanına bağlan

Connection connection = DriverManager.getConnection(

"jdbc:mysql://localhost/javabook", "scott", "tiger");

// Ders başlıklarını al

HazırlıklıStatement bildirimi = bağlantı.prepareStatement(n "dersten baş

ık seçin");

rowSet = new com.sun.rowset.CachedRowSetImpl();n rowS

et.populate(statement.executeQuery());n titles = new

String[rowSet.size()];

int ben = 0;

denemek {

while (rowSet.next()) {n başlı

k[i++] = rowSet.getString(1);n }

} catch (Exception ex) {n ex.pri

ntStackTrace();n }

// Öğrencileri almak için bir SQL ifadesi tanımla

studentStatement = connection.prepareStatement(

"select Student.ssn, "

+ "Öğrenci.ad, Öğrenci.mi, Öğrenci.soyadı, "n + "Öğrenci.telefon, Ö  
ğrenci.birthDate, Öğrenci.street, "n + "Öğrenci.zipCode, Öğrenci.dep  
tId "

+ "Öğrenci, Kayıt, Kurs "

+ "nerede Kurs.başlık = ? "

+ "ve Öğrenci.ssn = Kayıt.ssn "

+ "ve Kayıt.kursId = Kurs.kursId;");

} catch (Exception ex) {

```

 ex.printStackTrace();n }
 }

 public String[] getTitles() {
 return başlıklar;
 }

 public String getChoice() {
 return seçim;
 }

 public void setChoice(Dize seçimi) {n this.choice
 = seçim;
 }

 public ResultSet getStudents() SQLException'ı atar {n if (choice ==
 null) {
 if (titles == null) ve retur
 n null;in else

 studentStatement.setString(1, başlıklar[0]);n } else {

 studentStatement.setString(1, choice); // Kurs başlığını ayarla
 }

 // Belirtilen kurs için öğrencileri al
 CachedRowSet rowSet = new com.sun.rowset.CachedRowSetImpl ();
 rowSet.populate (studentStatement.executeQuery ());
 satırSetini döndür;n
 }
 }
}

```

### Biz, Bölüm 33, "Java

Veritabanı Programlama"da oluşturulan aynı MySQL veritabanını javabook kullanıyoruz. Bu yönetilen bean'in kapsamı application.

Bean, proje sunucudan başlatıldığında oluşturulur. Veritabanı bağlantısı, bean'in yapıcı metodunda bir kez oluşturulur (satırlar 17-19). initializeJdbc metodu, MySQL için JDBC sürücüsünü yükler (satır 24), MySQL veritabanına bağlanır (satırlar 25-26), kurs başlıklarını elde etmek için bir ifade oluşturur (satırlar 29-30) ve belirtilen kurs için öğrenci bilgilerini elde etmek için bir ifade oluşturur (satırlar 44-52). Satırlar 31-41, kurs başlıklarını elde etmek için ifadeyi çalıştırır ve bunları titles dizisine kaydeder.

getStudents() metodu, belirtilen derse kayıtlı tüm öğrencileri içeren bir ResultSet döndürür (satırlar 70-84). Başlık için seçim, belirtilen başlık için öğrenciyi almak üzere ifadeye ayarlanmıştır (satır 77). Seçim null ise, başlıklar dizisindeki ilk başlık ifadeye ayarlanır (satır 77). Başlıklar null ise, getStudents() null döner (satır 73).

Liste 44.14 DisplayStudent.xhtml

```

<margin note line 9: stil sayfası>
<margin note line 14: seçim başla>
<margin note line 15: başlıklar>
<margin note line 18: görüntüleme butonu>
<margin note line 22: sonuç kümesini başla>
<margin note line 23: satırSınıfları>
<margin note line 24: başlıkSınıfı>

```

<margin note line 25: stilSınıfı>  
 <margin note line 28: ssn sütunu>  
 <margin note line 33: ad sütunu>  
 <margin note line 38: mi sütunu>  
 <margin note line 43: soyad sütunu>  
 <margin note line 48: telefon sütunu>  
 <margin note line 53: doğumTarihi sütunu>  
 <margin note line 58: deptId sütunu>

```

<xml version='1.0' encoding='UTF-8'><!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" n "http://www.w3.or
g/TR/xhtml1/DTD/xhtml1-transitional.dtd"><html xmlns="http://www.w
3.org/1999/xhtml"
 xmlns:h="http://java.sun.com/jsf/html"n xmlns:f
="http://java.sun.com/jsf/core">n <h:head>

 <title>Öğrenci Göster</title>
 <h:outputStylesheet name="tablestyle.css"/>
</h:head>
<h:gövde>
 <h:form>
 <h:outputLabel value="Bir Kurs Seçin: " />
 <h:selectOneMenu value="#{courseName.choice}">
 <f:selectItems value="#{courseName.titles}" />
 </h:selectOneMenu>

 <h:commandButton style="margin-left: 20px"
 value="Öğrencileri Göster" />

 <h:dataTable value="#{courseName.students}" var="student"
 rowClasses="tekTabloSatırı, çiftTabloSatırı"
 headerClass="tabloBaşlık"
 styleClass="tablo">
 <h:sütun>
 <f:facet name="header">SSN</f:facet>n #{stu
dent.ssn}
 </h:sütun>

 <h:sütun>
 <f:facet name="header">İsim</f:facet>
 #{student.firstName}
 </h:sütun>

 <h:sütun>
 <f:facet name="header">MI</f:facet>n #{st
udent.mi}
 </h:sütun>

 <h:sütun>
 <f:facet name="header">Soyadı</f:facet>
 #{student.lastName}
 </h:sütun>

 <h:sütun>
 <f:facet name="header">Telefon</f:facet>
 #{student.phone}
 </h:sütun>

 <h:sütun>

```



```

 <f:facet name="header">Doğum Tarihi</f:facet>
 #{student.birthDate}
 </h:sütun>

 <h:sütun>
 <f:facet name="header">Bölüm</f:facet>
 #{student.deptId}
 </h:column>n
</h:dataTable>n
</h:form>
</h:body>n
</html>

```

9. satır, Adım 3'te oluşturulan tablestyle.css stil sayfasının bu XMTHL dosyasında kullanıldığını belirtir. rowClasses = "oddTableRow, evenTableRow"özniteliği, satırlara sırayla oddTableRow ve evenTableRow kullanarak uygulanan stili belirtir (satır 23). headerClasses = "tableHeader" niteliği, header stilinin kullanıldığını belirtir (header stili için) (satır 24). styleClasses = "table" niteliği, tablodaki diğer tüm öğelerin stilinin kullanıldığını belirtir (satır 25).

Satır 14, courseName bean'inde seçim özelliğini combo kutusu ile bağlar. Combo kutusundaki seçim değerleri, başlıklar dizisi özelliği ile bağlanır (satır 15).

Satır 22, tablo değerini bir veritabanı sonuç kümesi ile bağlar attribut değeri="#{courseName.students}". var="student" niteliği, sonuç kümesindeki bir satırı öğrenci ile ilişkilendirir. Satırlar 26-59, öğrenci.ssn (satır 28), öğrenci.firstName (satır 33), öğrenci.mi (satır 38), öğrenci.lastName (satır 33), öğrenci.phone (satır 48), öğrenci.birthDate (satır 53) ve öğrenci.deptId (satır 58) kullanarak sütun değerlerini belirtir.

Liste 44.15 tablestyle.css

*<margin not satırı 2: tableHeader> n<margin not satırı 14: oddTableRow> n<margin not satırı 18: evenTableRow> n<margin not satırı 28: table>*

```

/* Tablo için stil */
tableHeader {
 yazı tipi ailesi: "Trebuchet MS", Arial, Helvetica, sans-serif; n kenarlık dar
 altma: daraltma;
 yazı tipi boyutu: 1,1
 em; n metin hizala
 ma:sol; n dolgu üst:
 5px; n dolgu-alt:4px; n a
 rka plan rengi:#A7C942; n renk:
 beyaz;
 sınır:1px katı #98bf21;
}

.tekTabloSatırı {
 sınır:1px katı #98bf21;
}

.çiftTabloSatırı {
 arka plan rengi: #eeeeee;
}

```

```

yazı boyutu:1em;

iç boşluk:3px 7px 2px 7px;

renk:#000000;
arka plan rengi:#EAF2D3;
}

.tablo {
sınır:1px katı yeşil;
}

```

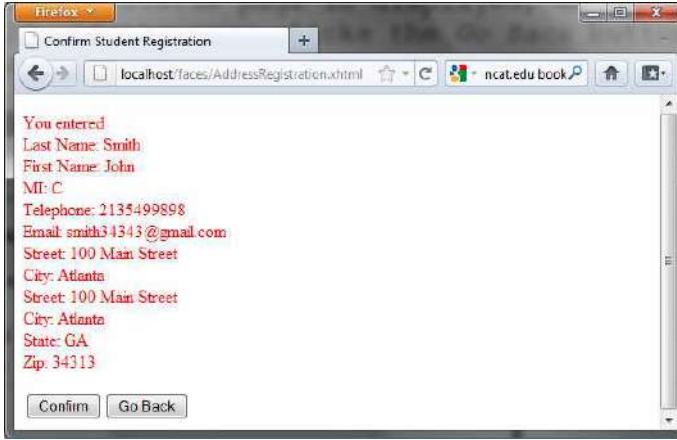
Stil sayfası dosyası, tablo başlığı stili için tableHeader (satır 2), tek tablo satırları için oddTableRow (satır 14), çift tablo satırları için evenTableRow (satır 18) ve diğer tüm tablo öğeleri için table (satır 28) stil sınıflarını tanımlar.

#### 41.9 Yeni JSF Sayfaları Açma

Şimdiye kadar gördüğünüz tüm örnekler, bir projede yalnızca bir JSF sayfası kullanmaktadır. Öğrenci bilgilerini database kaydetmek istediğinizi varsayalım. Uygulama önce, öğrenci bilgilerini toplamak için Şekil 44.21'de gösterildiği gibi sayfayı görüntüler. Kullanıcı bilgileri girdikten sonra *Gönder* butonuna tıkladığında, girdiği bilgileri onaylaması için yeni bir sayfa görüntülenir; bu da Şekil 44.22'de gösterilmektedir. Kullanıcı *Onayla* butonuna tıklarsa, veriler veritabanına kaydedilir ve Şekil 44.23'te gösterildiği gibi durum sayfası görüntülenir. Kullanıcı *Geri Dön* butonuna tıklarsa, ilk sayfaya geri döner.

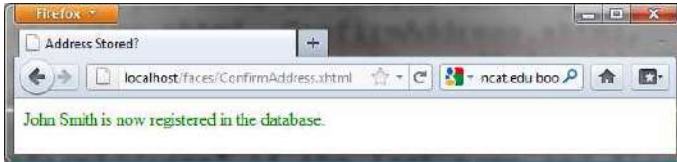
**Şekil 44.21**

*Bu sayfa, kullanıcının girdi yapmasına olanak tanır.*



**Şekil 44.22**

*Bu sayfa, kullanıcının girişi onaylamasına olanak tanır.*



**Şekil 44.23**

*Bu sayfa kullanıcı girişinin durumunu gösterir.*

Bu proje için, AddressRegistration.xhtml, ConfirmAddress.xhtml, AddressStoredStatus.xhtml adında üç JSP sayfası oluşturmanız gerekiyor. Proje AddressRegistration.xhtml ile başlar. Gönder butonuna tıkladığınızda, eğer son isim ve ilk isim boş değilse, butonun eylemi "ConfirmAddress" döner ve bu da ConfirmAddress.xhtml' in gösterilmesine neden olur. Onayla butonuna tıkladığınızda, durum sayfası

AddressStoredStatus görüntülenir. Geri Dön butonuna tıkladığınızda, ilk sayfa olan AddressRegistration şimdi görüntülenir.

Liste 44.16 AddressRegistration.xhtml

*<margin note satır 6: jsf çekirdek ad alanı> n<margin note satır 22: bağlama lastName>n<margin note satır 25: bağlama ilkAd>n<margin note satır 28: bağlama mi>  
<margin not satırı 34: telefonu bağla>n<margin not satırı 37: e-postayı bağla>n<margin not satırı 43: bağlama caddesi>n<margin not satırı 49: şehri bağla>n<margin not satırı 52: bağlama durumu>n<margin not satırı 59: bağlama zip>n<margin not satırı 64: işlem kaydı>*

```
<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"
```

```

xmlns:f="http://java.sun.com/jsf/core"> n <h:hea
d>
<title>Öğrenci Kayıt Formu</title>
</h:head>
<h:gövde>
<h:form>
<!-- h:graphicImage kullanın -->
<h3>Öğrenci Kayıt Formu
<h:graphicImage name="usIcon.gif" library="image"/>

</h3>

Lütfen eğitmeninizin öğrenci adres defterine kaydolun.
<!-- h:panelGrid kullanın -->
<h:panelGrid sütunları = "6">
<h:outputLabel value="Soyadı" style="color:red"/>
<h:inputText id="lastNameInputText"
value="#{addressRegistration.lastName}"/>
<h:outputLabel value="Ad" style="color:red"/>
<h:inputText id="firstNameInputText"
value="#{addressRegistration.firstName}"/> n <h:out
putLabel value="MI" />
<h:inputText id="miInputText" size="1" n value=
"#{addressRegistration.mi}"/> n </h:panelGr
id>

<h:panelGrid sütunları = "4">
<h:outputLabel value="Telefon"/>
<h:inputText id="telephoneInputText"
value="#{addressRegistration.telephone}"/>
<h:outputLabel value="E-posta"/>
<h:inputText id="emailInputText"
value="#{addressRegistration.email}"/>
</h:panelGrid>

<h:panelGrid sütunları = "4">
<h:outputLabel value="Sokak"/>
<h:inputText id="streetInputText"
value="#{addressRegistration.street}"/>
</h:panelGrid>

<h:panelGrid sütunları = "6">
<h:outputLabel value="Şehir"/>
<h:inputText id="cityInputText"
value="#{addressRegistration.city}"/>
<h:outputLabel value="Eyalet"/>
<h:selectOneMenu id="stateSelectOneMenu"
value="#{addressRegistration.state}">
<f:selectItem itemLabel="Georgia-GA" itemValue="GA" />
<f:selectItem itemLabel="Oklahoma-OK" itemValue="OK" />
<f:selectItem itemLabel="Indiana-IN" itemValue="IN"/>
</h:selectOneMenu>
<h:outputLabel value="Posta Kodu"/>
<h:inputText id="zipInputText"
value="#{addressRegistration.zip}"/> n </h:p
anelGrid>

<!-- Komut düğmesi kullanın -->
<h:commandButton value="Kaydet"
action="#{addressRegistration.processSubmit()}" /> n

<h:outputText kaçış = "yanlış" stil = "renk: kırmızı"

```

```

value="#{addressRegistration.requiredFields}" /> n </h:form>
>
</h:gövde>
</html>

```

Listeleme 44.17 ConfirmAddress.xhtml

*<margin note line 15: onay işlemi>*

*<margin note line 17: Adres Kayıt sayfasına git>*

```

<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.or
g/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.w
3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"> n <h:hea
d>
<title>Öğrenci Kaydını Onayla</title>
</h:head>
<h:gövde>
<h:form>
<h:outputText kaçış = "false" style = "renk: kırmızı" n değ
r = "# {registration1.input}" /> n <h:panel
Grid sütunlar = "2">
<h:commandButton value="Onayla"
action = "#{registration1.storeStudent()}" />
<h:commandButton value="Geri Dön"
action = "ÖğrenciKaydı"/>
</h:panelGrid>
</h:form> <
/h:body> n<
/html>

```

Listeleme 44.18 AdresStoredStatus.xhtml

**\*\*\*PD: Lütfen aşağıdaki kodda satır numaraları ekleyin\*\*\* \*\*\*Düzen:**  
**Lütfen tam olarak düzenleyin. Boşluğu atlamayın. Bu, kitapta yer alan tüm kaynak kodları i**  
**çin geçerlidir. Teşekkürler, AU.**

*<margin note line 12: görüntü durumu>*

```

<xml version='1.0' encoding='UTF-8' > n<!DOCTYPE
E html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"n "http://www.w3.or
g/TR/xhtml1/DTD/xhtml1-transitional.dtd"> n<html xmlns="http://www.w
3.org/1999/xhtml"
xmlns:h="http://java.sun.com/jsf/html"> n <h:hea
d>
<title>Adres Kaydedildi mi?</title>
</h:head>
<h:gövde>
<h:form>
<h:outputText kaçış = "false" style = "renk: yeşil" n değer = "#
{registration1.status}" /> n </h:form>

</h:gövde>
</html>

```

Liste 44.19 AddressRegistration.java

<kenar notu satır 7: yönetilen bean>  
<kenar notu satır 8: oturum kapsamı>  
<kenar notu satır 10: lastName özelliği>  
<kenar notu satır 24: veritabanını başlat>  
<kenar notu satır 107: yeni bir sayfaya git>  
<kenar notu satır 113: gerekli alanları kontrol et>  
<kenar notu satır 121: girişi al>  
<kenar notu satır 157: adresi sakla>  
<kenar notu satır 169: durumu güncelle>  
<kenar notu satır 175: yeni bir sayfaya git>

```
paket jsf2demo;
```

```
javax.faces.bean.ManagedBean'i içe aktarın;
import javax.faces.bean.SessionScoped; import
java.sql.*;
```

```
@ManagedBean
```

```
@SessionScoped
```

```
genel sınıf Adres Kaydı { in özel String last
```

```
 Name;
```

```
 özel Dize ad; n özel Dize mi;
```

```
 özel String telefon;
```

```
 özel Dize e-postası;
```

```
 özel String sokak;
```

```
 özel String şehir;
```

```
 özel String eyalet;
```

```
 özel String postaKodu;
```

```
 özel String durum = "Hiçbir şey saklanmadı";
```

```
 // Veritabanına bir öğrenciyi saklamak için hazırlanmış bir ifade kullanın
```

```
 özel PreparedStatement pstmt;
```

```
 kamu AdresKaydı() {
```

```
 initializeJdbc();
```

```
 }
```

```
 kamu String getSoyad() {
```

```
 return soyad;
```

```
 }
```

```
 kamu void setSoyad(String soyad) {
```

```
 this.soyad = soyad;
```

```
 }
```

```
 kamu String getAd() {
```

```
 return ad;
```

```
 }
```

```
 public void setFirstName(String FirstName) { n this.first
```

```
 Name = FirstName;
```

```
 }
```

```
 public String getMi() { n retur
```

```
 n mi;
```

```
 }
```

```
 public void setMi(String mi) { n this.mi
```

```
 = mi;
```

```
 }
```

```

public String getTelephone() { n dönü
ş telefonu;
}

public void setTelephone(String telefon) { n this.telepho
ne = telefon;
}

public String getEmail() { n dönü
ş e-postası;
}

public void setEmail(String email) { n this.ema
il = email;
}

public String getStreet() { n return
street;
}

public void setStreet(String cadde) { n this.street
= sokak;
}

public String getCity() { n dönüş
şehri;
}

public void setCity(String city) { n this.city =
city;
}

public String getState() { n dönüş
durumu;
}

public void setState(Dize durumu) { n this.stat
e = durumu;
}

public String getZip() { n retur
n zip;
}

public void setZip(String zip) { n this.zip
= zip;
}

private boolean isRrequiredFieldsFilled() {
return !(lastName == null || firstName == null
|| lastName.trim().length() == 0
|| FirstName.trim().length() == 0); N }

public String processSubmit() {
if (isRrequiredFieldsFilled()) {
return "ConfirmAddress";
} else {
geri dönmek ""; N
}
}

```

```

 }

 public String getRequiredFields() {
 if (isRequiredFieldsFilled()) {
 return "";
 }
 return "Soyadı ve Adı gereklidir";
 }

 public String getInput() {
 return "<p style='color:red'>Girdiğiniz
"
 + "Soyadı: " + lastName + "
"
 + "Ad: " + firstName + "
"
 + "MI: " + mi + "
"
 + "Telefon: " + telephone + "
"
 + "E-posta: " + email + "
"
 + "Sokak: " + street + "
"
 + "Şehir: " + city + "
"
 + "Sokak: " + street + "
"
 + "Şehir: " + city + "
"
 + "Eyalet: " + state + "
"
 + "Posta Kodu: " + zip + "</p>";
 }

 /** Veritabanı bağlantısını başlat */
 private void initializeJdbc() {
 denemek {
 // Açıkça bir MySQL sürücüsünü yükle
 Class.forName("com.mysql.jdbc.Driver");
 System.out.println("Sürücü yüklendi");

 // Bir bağlantı kur
 Connection conn = DriverManager.getConnection(
 "jdbc:mysql://localhost/javabook", "scott", "tiger");

 // Bir Statement oluştur
 pstmt = conn.prepareStatement("Adres (soyadı,"
 + " adı, mi, telefon, e-posta, sokak, şehir,"
 + " eyalet, posta kodu) değerleri (?, ?, ?, ?, ?, ?, ?) ekle");
 } catch (Exception ex) {
 System.out.println(ex);
 }
 }

 /** Bir adresi veritabanına kaydet */
 public String storeStudent() {
 denemek {
 pstmt.setString(1, soyad);
 pstmt.setString(2, ad);
 pstmt.setString(3, mi);
 pstmt.setString(4, telefon);
 pstmt.setString(5, email);
 pstmt.setString(6, sokak);
 pstmt.setString(7, şehir);
 pstmt.setString(8, eyalet);
 pstmt.setString(9, postaKodu);
 pstmt.executeUpdate();
 durum = ad + " " + soyad
 + " artık veritabanında kayıtlı.";
 } catch (Exception ex) {

```



```

 durum = ex.getMessage();
 }

 return "AdresKaydedildiDurumu";
}

public String getStatus() {
 return durum;
}

```

AdresKayıt JSF sayfasındaki *Kayıt* butonunun eylemi `processSubmit()` (`AddressRegistration.xhtml`'da 64. satır). Bu yöntem soyadının ve adının boş olmadığını kontrol eder (`AddressRegistration.java`'da 105-111. satırlar). Eğer öyleyse, "ConfirmAddress" adlı bir diz e döndürür, bu da `ConfirmAddress` JSF sayfasının görüntülenmesine neden olur.

`ConfirmAddress` JSF sayfası, kullanıcıdan girilen verileri görüntüler (`ConfirmAddress.xhtml`'deki 12. satır). `getInput()` metodu (`AddressRegistration.java`'daki 121-134. satırlar) gir işi toplar.

`ConfirmAddress` JSF sayfasındaki *Onayla* butonunun eylemi `storeStudent()` (`ConfirmAddress.xhtml`'de 15. satır) dir. Bu yöntem adresi veritabanına kaydeder (`AddressRegistration.java`'da 157-176. satırlar) ve "AddressStoredStatus" adlı bir dize döndürür, bu da `AddressStoredStatus` sayfasının görüntülenmesine neden olur. Durum mesajı bu sayfada görüntülenir (`AddressStoredStatus.xhtml`'de 12. satır).

`ConfirmAddress` sayfasındaki *Geri Dön* butonunun eylemi "AddressRegistration" (`ConfirmAddress.xhtml`'de 17. satır) dir. Bu, kullanıcının girişi yeniden vermesi için `AddressRegistration` sayfasının görüntülenmesine neden olur.

Yönetilen bean'in kapsamı oturumdur (`AddressRegistration.java`'da 8. satır) bu nedenle birden fazla sayfa aynı bean'i paylaşabilir.

Bu programın veritabanı sürücüsünü açıkça yüklediğini unutmayın (`AddressRegistration.java`'da 140. satır). Bazen, NetBeans gibi bir IDE uygun bir sürücüyü bulamaz. Bir sürücüyü açıkça yüklemek bu sorunu önleyebilir.

## Bölüm Özeti

1. JSF, Java kodunu HTML'den tamamen ayırmanıza olanak tanır.
2. Bir facelet, JSF etiketlerini XHTML etiketleriyle karıştıran bir XHTML sayfasıdır.
3. JSF uygulamaları, uygulamanın verilerini (modelde bulunan) grafiksel sunumdan (görünüm) ayıran Model-Görünüm-Kontrolcü (MVC) mimarisi kullanılarak geliştirilir.
4. Kontrolcü, görünüm ile model arasındaki etkileşimleri koordine etmekten sorumlu olan JSF çerçevesidir. modeli.
5. JSF'de, facelet'ler verileri sunmak için görünüm olarak kullanılır. Veriler Java nesnelerinden elde edilir. Nesneler, Java sınıfları kullanılarak tanımlanır.

6. JSF'de, bir facelet'ten erişilen nesneler JavaBeans nesneleridir.
7. JSF ifadesi, ya özellik adını kullanabilir ya da mevcut zamanı elde etmek için metodu çağırabilir.
8. JSF, GUI bileşenlerini görüntülemek için birçok öge sağlar. h ön eki ile başlayan etiketler JSF HTML Etiket kütüphanesindedir. f ön eki ile başlayan etiketler ise JSF Core Etiket kütüphanesindedir.
9. JavaBeans nesnelerini uygulama kapsamı, oturum kapsamı, sayfa kapsamı, görünüm kapsamı veya istek kapsamı olarak belirtebilirsiniz.
10. Görünüm kapsamı, görünümde kaldığınız sürece bean'i canlı tutar. Görünüm kapsamı, oturum ve istek kapsamları arasındadır.
11. JSF, girdi doğrulama için birkaç kullanışlı ve güçlü yol sağlar. JSF Core Etiket Kütüphanesindeki standart doğrulayıcı etiketleri kullanabilir veya özel doğrulayıcılar oluşturabilirsiniz.

### Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın:  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

#### Gözden Geçirme Soruları

##### Bölüm 44.2

44.1 JSF nedir?

44.2 NetBeans'te bir JSF projesi nasıl oluşturulur?

44.3 Bir JSF projesinde bir JSF sayfası nasıl oluşturulur?

44.4 Facelet nedir?

44.5 Bir facelet için dosya uzantısı adı nedir?

44.6 Yönetilen bean nedir?

44.7 @ManagedBean anotasyonu ne için kullanılır?

44.8 @RequestScope anotasyonu ne için kullanılır?

44.9 h öneki ve f öneki ile JSF etiketleri için ad alanı nedir?

##### Bölümler 44.3-44.5

44.10 Aşağıdaki etiketlerin kullanımını tanımlayın?

h:form, h:panelGroup, h:panelGrid, h:inputText, h:outputText, nh:inputTextArea, h:inputSecret, h:outputLabel, h:outputLink, nh:selectOneMenu, h:selectOneRadio, h:selectBooleanCheckbox, nh:selectOneListbox, h:selectManyListbox, h:selectItem, h:mesaj, nh:dataTable, h:sütun, h:grafikResim

44.11 h:outputText etiketinde escape niteliği ne için kullanılır?

44.12 JSF'deki her GUI bileşen etiketinin stil niteliği var mı?

## Bölüm 44.6

44.13 JSF oturum kapsamı nedir?

44.14 Yönetilen bir bean'de oturum kapsamını nasıl ayarlarsınız?

44.15 Dört oturum kapsamını tanımlayın.

44.16 Varsayılan oturum kapsamı nedir?

## Bölüm 44.7

44.17 Minimum 2 ve maksimum 12 karakter uzunluğunda bir girişi doğrulayan bir etike t yazın. \_\_\_\_\_

44.18 Bir düzenli ifade kullanarak SSN için bir girişi doğrulayan bir etiket yazın.

44.19 Minimum 4.5 ve maksimum 19.9 olan bir double değeri için bir girişi doğrulayan bir etiket yazın. \_\_\_\_\_

44.20 Bir tamsayı değeri için girişi doğrulayan bir etiket yazın, minimum 4 ve maksimum 20 ile.

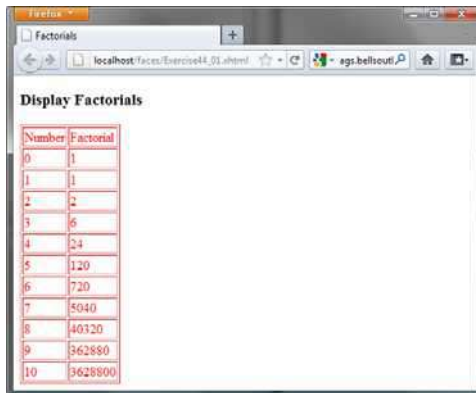
44.21 Girişi zorunlu kılan bir etiket yazın.

## Programlama Alıştırmaları

44.1 \*

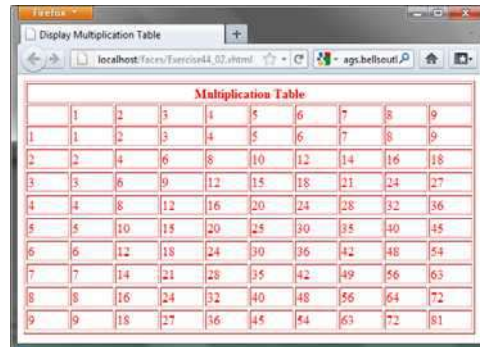
(JSF'de Faktöriyel tablosu) Faktöriyel sayfasını gösteren bir JSF sayfası yazın Şekil 44.24a'da gösterildiği gibi. Tabloyu h:outputText içinde gösterin bileşen. HTML \_\_\_\_\_

içeriklerini görüntülemek için escape özelliğini false olarak ayarlayın.



Number	Factorial
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800

(a)



	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

(b)

Şekil 44.24

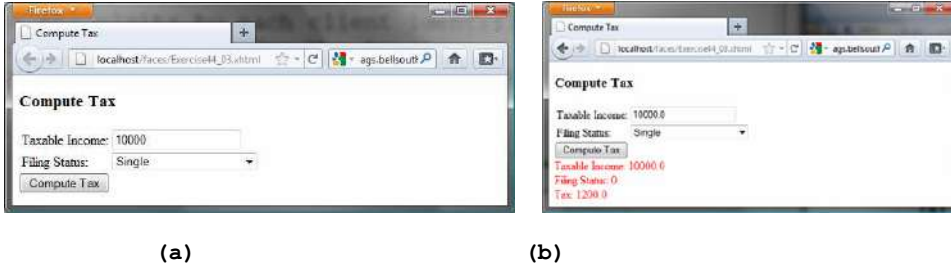
- (a) JSF sayfası, 0'dan 10'a kadar olan sayıların faktöriyelini bir tabloda görüntüler. —  
(b) JSF sayfası çarpanlar tablosunu görüntüler.

44.2\*

(Çarpan tablosu) Şekil 44.24b'de gösterildiği gibi bir çarpan tablosunu görüntüleyen bir JSF sayfası yazın.

44.3\*

(Vergi hesaplama) Kullanıcının vergilendirilebilir gelir ve dosyalama durumunu girmesine izin veren bir JSF sayfası yazın, Şekil 44.25a'da gösterildiği gibi. Tıklamak *Vergi Hesaplama* butonu, vergiyi hesaplamak ve görüntülemek için bir servleti çağırır, Şekil 44.25b'de gösterildiği gibi. Vergiyi hesaplamak için, Listeleme 3.7'de tanımlanmış computeTax yöntemini kullanın, ComputeTax.java.

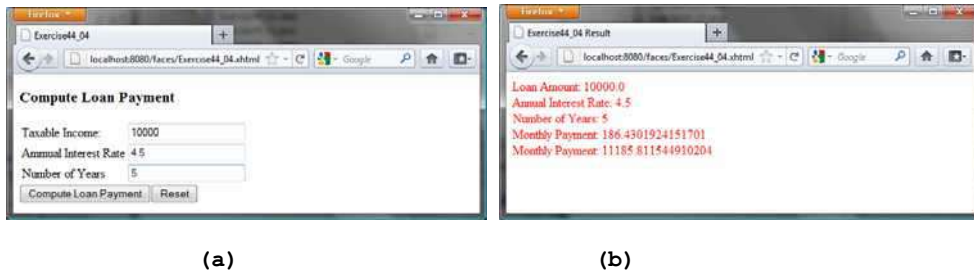


Şekil 44.25

JSF sayfası vergiyi hesaplar.

44.4\*

(Kredi hesaplama) Kullanıcının kredi miktarını, faiz oranını ve yıl sayısını girmesine olanak tanıyan bir JSF sayfası yazın, Şekil 44.26a'da gösterildiği gibi. *Kredi Ödeme Hesaplama* butonuna tıklayarak aylık ve toplam kredi ödemelerini hesaplayın ve gösterin, Şekil 44.26b'de gösterildiği gibi. Aylık ve toplam ödemeleri hesaplamak için Listing 10.2, *Loan.java*'da verilen *Loan* sınıfını kullanın.

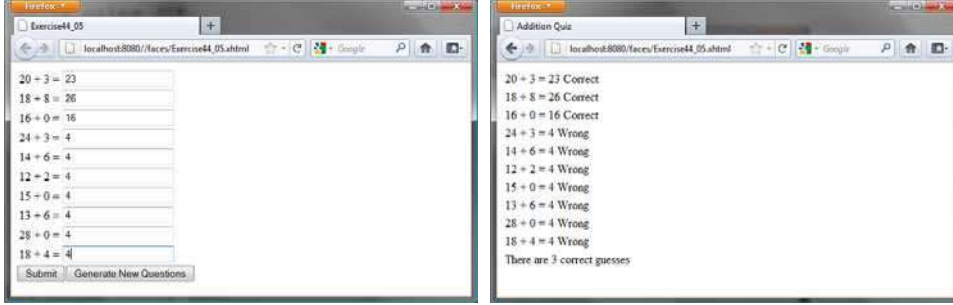


Şekil 44.26

JSF sayfası kredi ödemesini hesaplar.

44.5\*

(*Toplama sınavı*) Kullanıcının tüm soruları yanıtladıktan sonra, JSP'nin sonucu gösterdiği şekilde, Şekil 44.27a'da gösterildiği gibi rastgele toplama sınavları üreten bir JSF programı yazın.



Şekil 44.27

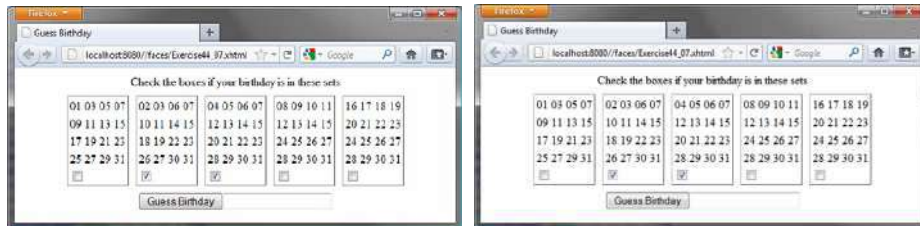
Program, (a) bölümünde toplama sorularını ve (b) bölümünde cevapları görüntüler.

43.18\*

(*Büyük faktöriyel*) Egzersiz 44.1'i büyük faktöriyel ile başa çıkacak şekilde yeniden yazın. §15.11'de tanıtilan `BigInteger` sınıfını kullanın.

44.7\*

(*Doğum günü tahmini*) Liste 3.3, `GuessBirthDay.java`, bir doğum gününü tahmin eden bir programı verir. Beş set numarayı gösteren bir JSP programı yazın, Şekil 44.28a'da gösterildiği gibi. Kullanıcı kontrol ettikten sonra uygun kutuları işaretler ve *Tarih Bul* butonuna tıklar, program d tarihi gösterir, Şekil 44.28b'de gösterildiği gibi.



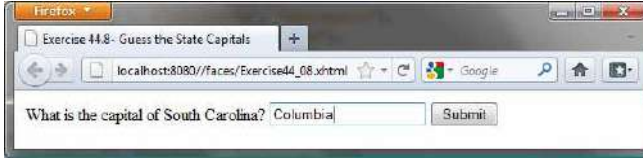
Şekil 44.28

(a) Program, kullanıcının kutuları işaretlemesi için beş set sayı gösterir. (b) Program tarihi gösterir.

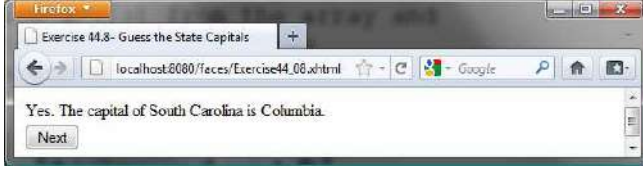
44.8\*

(Başkentleri tahmin et) Kullanıcının bir eyaletin başkentini girmesini isteyen bir JSF yazı n, Şekil 44.29a'da gösterildiği gibi. Kullanıcı girdisini aldıktan sonra, program cevabın do ğru olup olmadığını bildirir, Şekil'de gösterildiği gibi.

44.29b. Başka bir soruyu görüntülemek için *İleri* butonuna tıklayabilirsiniz. Eyaletleri ve başkentleri depolamak için, Alıştırma 9.22'de önerildiği gibi, iki boyutlu bir dizi kullana bilirsiniz. Diziden bir liste oluşturun ve soruların rastgele sırada görünmesi için karıştır ma yöntemini uygulayın.



(a)



(b)

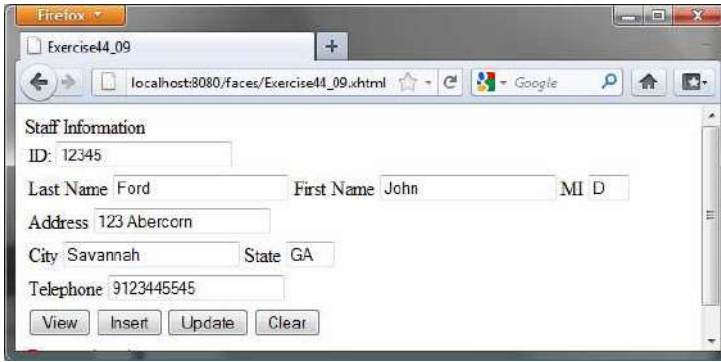
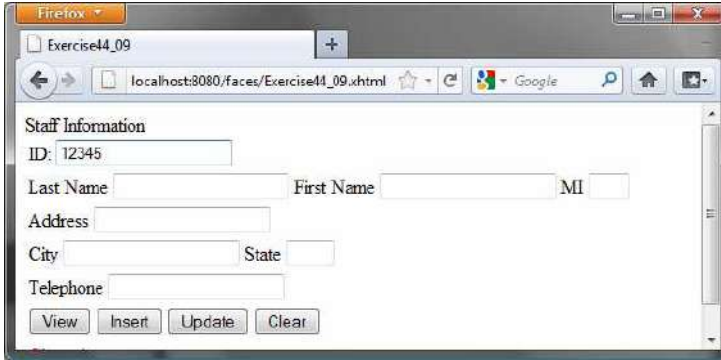
Şekil 44.29

(a) Program bir soru gösterir. (b) Program sorunun cevabını gösterir.

44.9\*

(Bir Personel tablosuna erişim ve güncelleme) Veritabanında saklanan personel bi lgilerini görüntüleyen, giren ve güncelleyen bir JSF programı yazın, Şekil 44.30a'da gös terildiği gibi. Görüntüle butonu, belirtilen ID ile bir kaydı görüntüler. Personel tablosu a şağıdaki gibi oluşturulur:

```
create table Personel (
 id char(9) not null,
 soyad varchar(15),
 ad varchar(15),
 mi char(1),
 adres varchar(20),
 şehir varchar(20),
 eyalet char(2),
 telefon char(10),
 e-posta varchar(40),
 birincil anahtar (id)
);
```



**Şekil 44.30**

*Web sayfası, personel bilgilerini görüntülemenizi, girmenizi ve güncellemenizi sağlar.*

44.10 \*

(*Rastgele kartlar*) Şekil 44.22'de gösterildiği gibi, 52 kartlık bir desteden dört rastgele kart görüntüleyen bir JSF yazın. Kullanıcı *Yenile* butonuna tıkladığında, dört yeni rastgele kart görüntülenir.



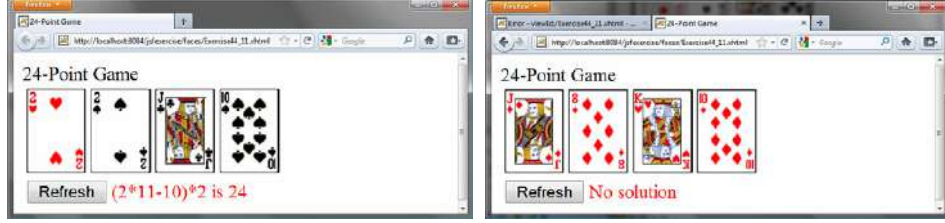
**Şekil 44.22**

*Bu JSF uygulaması dört rastgele kart gösterir.*

44.11 \*\*\*

(*Oyun: 24 puanlık kart oyunu*) JSF kullanarak 25.11 numaralı alıştırmayı yeniden yazın, Şekil 44.23'te gösterildiği gibi. *Yenile* butonuna tıkladığınızda, program

dört rastgele kart gösterir ve 24 puanlık bir çözüm varsa bir ifade gösterir. Aksi takdirde, Çözüm yok.

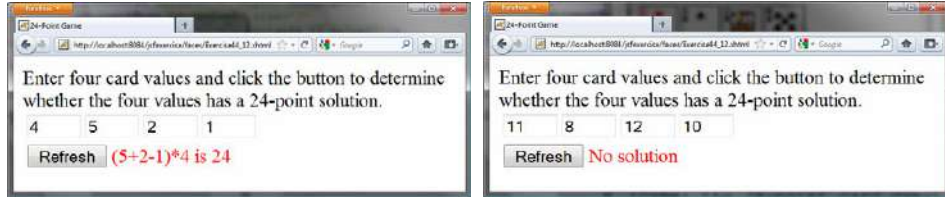


#### Şekil 44.23

JSF uygulaması 24-Point kart oyununu çözer.

44.12 \*\*\*

(Oyun: 24 puan kart oyunu) Şekil 44.24'te gösterildiği gibi JSF kullanarak Alıştırma 25.10'u yeniden yazın. Program, kullanıcının dört kart değerini girmesine izin verir ve Çözüm Bul butonuna tıkladığında bir çözüm bulur.



#### Şekil 44.24

Kullanıcı dört sayı girer ve program bir çözüm bulur.



*\*\*\*Bu bir bonus Web bölümü*

## **BÖLÜM 45**

### **Web Hizmetleri**

#### Amaçlar

- Bir Web hizmetinin ne olduğunu tanımlamak (§45.1).
- Bir Web hizmeti sınıfı oluşturmak (§45.2).
- Bir Web hizmetini yayınlamak ve test etmek (§45.3).
- Bir Web hizmeti istemci referansı oluşturmak (§45.4).
- WSDL'nin rolünü açıklamak için (§45.4).
- Bir Web hizmetinde nesne türünde argümanlar geçirmek için (§45.5).
- Bir istemcinin bir Web hizmeti ile nasıl iletişim kurduğunu keşfetmek için (§45.5) .
- SOAP isteklerinin ve SOAP yanıtlarının ne olduğunu tanımlamak için (§45.5).
- Web hizmetlerinde bir oturumu takip etmek için (§45.6).

## 45.1 Giriş

<Yan Not: platformdan bağımsız>

<Yan Not: dilden bağımsız>

Web hizmeti, programların İnternet'te HTTP üzerinden iletişim kurmasını sağlayan bir teknolojidir. Web hizmetleri, bir sistemdeki bir programın başka bir sistemdeki bir nesneyle bir yöntemi çağırmasını sağlar. Herhangi bir platformda herhangi bir dil kullanarak Web hizmetleri geliştirebilir ve kullanabilirsiniz. Web hizmetleri basit ve geliştirmesi kolaydır.

<Yan Not: SOAP>

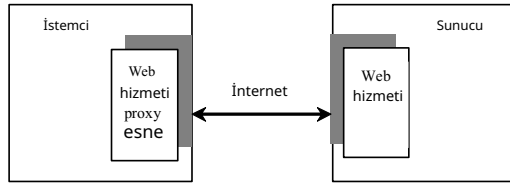
<Yan Not: Web hizmetleri yayınlama>

<Yan Not: Web hizmetlerini tüketme>

Web hizmetleri, HTTP kullanarak Web üzerinde çalışır. Web hizmetleri için birkaç API vardır. Popüler bir standart, Basit Nesne Erişimi Protokol (SOAP), XML'e dayanmaktadır. Bir Web hizmetinin bulunduğu bilgisayar sunucu denir. Sunucu, hizmeti istemciye sunmak için Web hizmetini yayınlama gerektirir. Bir istemcinin Web hizmetini kullanması, Web hizmetini tüketme olarak bilinir.

<Yan Not: proxy nesne>

Bir istemci, bir proxy nesne aracılığıyla bir Web hizmeti ile etkileşimde bulunur. Proxy nesne, istemci ile Web hizmeti arasındaki iletişimi kolaylaştırır. İstemci, proxy nesne üzerinde yöntemleri çağırarak argümanlar gönderir. Proxy nesne, isteği sunucuya gönderir ve sunucudan sonucu geri alır; bu, Şekil 45.1'de gösterilmiştir.



Şekil 45.1

*Bir proxy nesnesi, bir istemci ile bir Web*

*hizmeti arasında bir kolaylaştırıcı olarak hizmet eder.*

## 45.2 Web Hizmetleri Oluşturma

<Yan Not: Web hizmeti aracı>

Web hizmetleri oluşturmak için birçok araç vardır. Bu kitap, Web hizmetleri oluşturmayı NetBeans kullanarak göstermektedir.

<Yan Not: GlashFish 3'ü kurun>

NOT:

Apache Tomcat Sunucusu Web hizmetleriyle iyi çalışmaz. Web hizmetlerini NetBeans kullanarak geliştirmek ve dağıtmak için GlassFish 3'ü kurmanız gerekir. NetBeans 7'de GlassFish 3'ü nasıl kuracağınız hakkında bilgi için Ek II.I'ye bakın.

\*\*\*NOT BİTİŞ

Artık öğrenci notlarını almak için bir Web hizmeti oluşturuyoruz. Bir Web hizmeti, istemcinin çağırması için yöntemleri içeren bir sınıftır. Sınıfı `ScoreService` olarak adlandırın ve bir öğrencinin notunu döndüren `findScore(String name)` adlı bir yöntem oluşturun.

<Yan Not: NetBeans Web projesi>

Öncelikle aşağıdaki adımları izleyerek bir Web projesi oluşturmanız gerekiyor:

Yeni Proje iletişim kutusunu görüntülemek için Dosya > Yeni Proje'yi seçin.

Yeni Proje iletişim kutusunda, Kategoriler panelinde Java Web'i seçin ve Projeler panelinde Web Uygulaması'nı seçin. Yeni Web Uygulaması iletişim kutusunu görüntülemek için İleri'ye tıklayın. Proje adı olarak WebServiceProject girin, projenin saklanacağı yeri belirtin ve Sunucu ve Ayar iletişim kutusunu görüntülemek için İleri'ye tıklayın.

Sunucu olarak GlassFish 3'ü ve Java EE sürümü olarak Java EE 6 Web'i seçin. Projeyi oluşturmak için Bitir'e tıklayın.

**<Yan Not: Web hizmeti sınıfı oluşturun>**

Artık projede ScoreService sınıfını aşağıdaki gibi oluşturabilirsiniz:

Proje panelinde WebServiceProject'e sağ tıklayın ve bir bağlam menüsü görüntüleyin. Yeni > Web Hizmeti seçeneğini seçin ve Yeni Web Hizmeti iletişim kutusunu görüntüleyin. Web Hizmeti Adı alanına ScoreService yazın ve Paket alanına chapter45 yazın. ScoreService'i oluşturmak için Bitir'e tıklayın.

Kaynak kodunu Liste 45.1'de gösterildiği gibi tamamlayın.

Liste 45.1 ScoreService.java

**<Yan Not satır 4: @WebService için import>** **<Yan Not satır 5: @WebMethod için import>** **<Yan Not satır 7: WebService tanımlayın>** **<Yan Not satır 19: WebMethod tanımlayın>**

paket bölümü45;

Java.util.HashMap'i içe aktarın;

import javax.jws.WebService; // @WebService notasyonu için

import javax.jws.WebMethod; // @WebMethod notasyonu için

@WebService(name = "ScoreService", serviceName = "ScoreWebService")

public class ScoreService {

// İsimle indekslenmiş bir haritada puanları saklar

private HashMap<String, Double> scores =

new HashMap<String, Double>();

public ScoreService() {n puanlar

lar.put("John", 90,5);n puanlar

.put("Michael", 100,0);n puanlar.put(

"Michelle", 98,5);n }

@WebMethod(operationName = "findScore")n p

ublic double findScore(Dize adı) {n Double d = S

cores.get(name);

eğer (d == null) {

System.out.println("Öğrenci " + name + " bulunamadı ");

return -1;

}

başka {

System.out.println("Öğrenci " + name + "\'nin notu "

+ d.doubleValue());

return d.doubleValue();n }

}

}

<Yan Not: anotasyon nedir?>

<Yan Not: şablon kodu>

4-5. satırlar, programda 7. ve 19. satırlarda kullanılan anotasyonları içe aktarır. Anotasyon, Java'da kodlamayı basitleştirmenizi sağlayan yeni bir özelliktir. Derleyici i, anotasyonlu direktifler için otomatik olarak kod üretecektir. Böylece, programcuyu mekanik olarak üretilebilecek ayrıntılı şablon kodu yazmaktan kurtarır. Anotasyon (7. satır)

```
@WebService(name = "ScoreService", serviceName = "ScoreWebService")
```

derleyiciye ScoreService sınıfının ScoreWebService adlı Web hizmeti ile ilişkili olduğunu söyler.

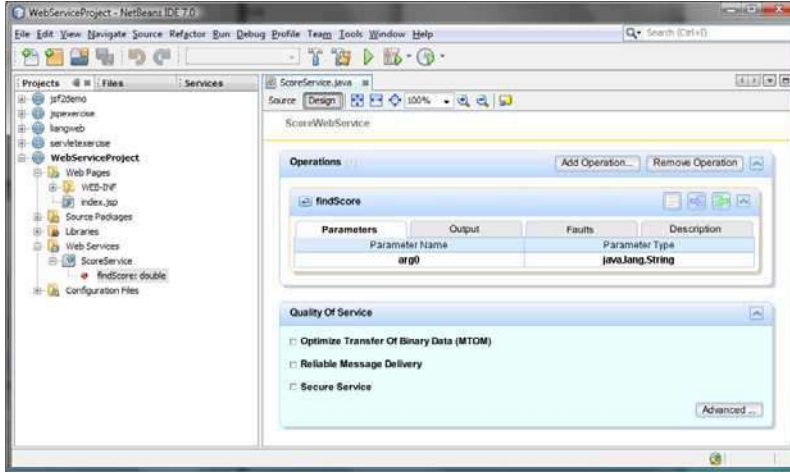
Anotasyon (19. satır)

```
@WebMethod(operationName = "findScore")
```

findScore'ın bir istemciden çağrılabilen bir yöntem olduğunu belirtir.

findScore yöntemi, isim hash haritasında varsa bir not döndürür. Aksi takdirde , -1.0 döndürür.

Hizmet için kodu manuel olarak yazabilir veya Şekil 45.2'de gösterildiği gibi Tasarım sekmesinden oluşturabilirsiniz.



Şekil 45.2

Hizmetler Tasarım panelinden de oluşturulabilir.

### 45.3 Web Hizmetlerini Dağıtma ve Test Etme

<Yan Not: web hizmetlerini yayınlama>

Bir Web hizmeti oluşturulduktan sonra, onu istemcilerin kullanabilmesi için dağıtmanız gerekir. Web hizmetlerini dağıtma, web hizmetlerini yayınlama olarak da bilinir. Dağıtmak için, Proje'de WebServiceProject'e sağ tıklayın ve bir bağlam menüsü görüntülemek için Dağıt'ı seçin. Bu komut, eğer hizmet dağıtılmışsa önce hizmeti geri alacak ve ardından yeniden dağıtacaktır.

Artık Web hizmetini test edebilirsiniz; bunu bir tarayıcıya aşağıdaki URL'yi girerek yapabilirsiniz, Şekil 45.3'te gösterildiği gibi.

http://localhost:8080/WebServiceProject/ScoreWebServiceTester

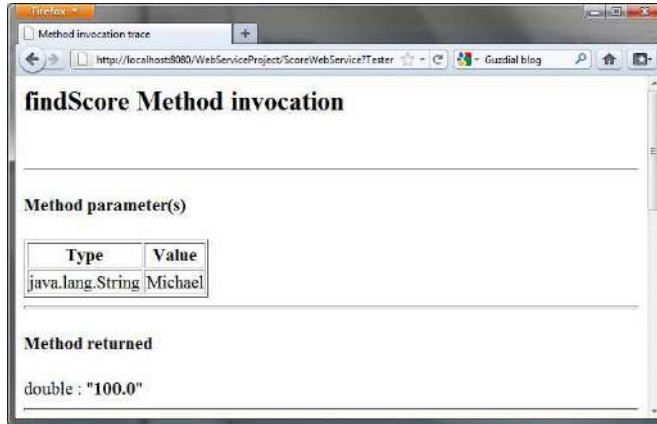


**Şekil 45.3**

*Test sayfası, Web hizmetlerini test etmenizi sağlar.*

ScoreWebService'in, Liste 45.1'de 7. satırda belirttiğiniz ad olduğunu unutmayın. Bu Web hizmetinin yalnızca findScore adında bir uzaktan yöntemi vardır. Bir Web hizmeti sınıfında sınırsız sayıda uzaktan yöntem tanımlayabilirsiniz. Eğer öyleyse, bu yöntemlerin hepsi test sayfasında görüntülenecektir.

findScore yöntemini test etmek için Michael girin ve findScore butonuna tıklayın. Yöntemin 100.0 döndürdüğünü göreceksiniz, Şekil 45.4'te gösterildiği gibi.



**Şekil 45.4**

*Yöntem bir test değeri döndürür.*

**<Yan Not: başka bir makineden test etme>**

NOT: Bilgisayarınız İnternete bağlıysa, başka bir bilgisayardan Web hizmetlerini test etmek için aşağıdaki URL'yi girin:

http://host:8080/WebServiceProject/ScoreWebServiceTester

**<Yan Not: ipconfig>**

Burada *host* Web hizmetinin çalıştığı sunucunun host adı veya IP adresidir. Windows'ta, IP adresinizi bulmak için *ipconfig*. komutunu yazabilirsiniz.

\*\*\*NOT BİTİŞ

<Yan Not: Windows güvenlik duvarı>

NOT: Sunucuyu Windows'ta çalıştırıyorsanız, güvenlik duvarı uzak istemcilerin hizmete erişimini engelleyebilir. Bunu etkinleştirmek için şunları yapın:

1. Windows kontrol panelinde, Windows Güvenlik Duvarı'na tıklayın ve Windows Güvenlik Duvarı iletişim kutusunu görüntüleyin.
2. Gelişmiş sekmesinde, Yerel Alan Bağlantısına çift tıklayın ve Gelişmiş Ayarlar iletişim kutusunu görüntüleyin. HTTP erişimini sunucuya etkinleştirmek için Web Sunucusu(HTTP) seçeneğini işaretleyin.
3. İletişim kutusunu kapatmak için *Tamam* butonuna tıklayın.

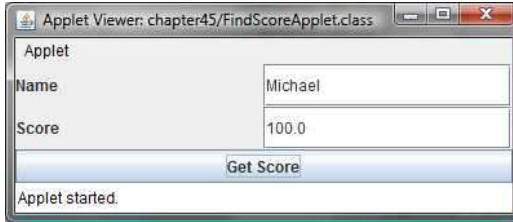
\*\*\*NOT BİTİŞ

#### 45.4 Web Hizmetlerini Tüketme

<Yan Not: Web hizmetlerini tüketme>

Bir Web hizmeti yayımlandıktan sonra, onu kullanmak için bir istemci programı yazabilirsiniz. Bir istemci, herhangi bir program (bağımsız uygulama, applet, servlet/JSP/JSF uygulaması veya başka bir Web hizmeti) olabilir ve herhangi bir dide yazılabilir.

Bir Web hizmeti istemcisi oluşturmak için NetBeans kullanacağız. İstemcimiz, ana yönteme sahip bir Java appletidir, bu nedenle bağımsız olarak da çalıştırabilirsiniz. Applet, kullanıcının bir isim girmesine izin verir ve puanı gösterir; Şekil 45.5'te gösterildiği gibi.



Şekil 45.5

*Applet istemcisi, puanları bulmak için Web hizmetini kullanır.*

Müşteri için bir proje oluşturalım.

ScoreWebServiceClientProject adında bir proje aşağıdaki gibi oluşturulabilir:

Yeni Proje iletişim kutusunu görüntülemek için Dosya > Yeni Proje'yi seçin. Yeni Proje iletişim kutusunda, Kategoriler panelinde **Java** seçin ve Projeler panelinde Java Uygulaması seçin. Yeni Java Uygulaması iletişim kutusunu görüntülemek için İleri butonuna tıklayın. Proje adı olarak ScoreWebServiceClientProject girin, projenin saklanacağı yeri belirtin ve Ana Sınıf Oluştur onay kutusunun işaretini kaldırın. Projeyi oluşturmak için Tamam butonuna tıklayın.

<Yan Not: Web hizmeti referansı>

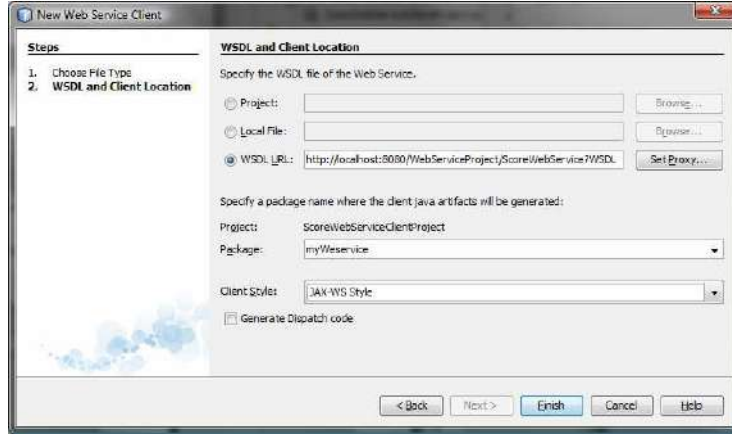
Bu projeye bir Web hizmeti referansı oluşturmanız gerekiyor. Referans, Web hizmeti ile etkileşimde bulunmak için bir proxy nesnesi oluşturmanıza olanak tanıyacaktır. Bir Web hizmeti referansı oluşturmak için adımlar şunlardır:

Proje panelinde **ScoreWebServiceClientProject** üzerine sağ tıklayarak bir bağlam menüsü görün tüleyin. Yeni Web Hizmeti İstemcisi'ni seçmek için Yeni > Web Hizmeti İstemcisi seçeneğine tıklayın, Şekil'de gösterildiği gibi Yeni Web Hizmeti İstemcisi iletişim kutusu görüntülenecektir. 45.6.

WSDL URL'si radyo düğmesini işaretleyin ve WSDL URL alanına

<http://localhost:8080/WebServiceProject/ScoreWebService?WSDL> girin.

Paket adı alanına **myWebservice** girin ve JAX sürümü olarak **JAX-WS** seçin. Web hizmeti referansını oluşturmak için Tamam butonuna tıklayın.

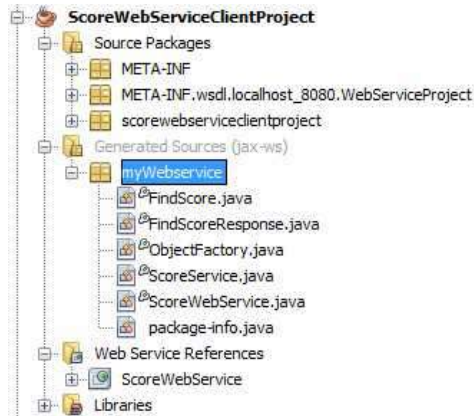


**Şekil 45.6**

*Yeni Web Servisi İstemcisi iletişim kutusu bir Web servisi referansı oluşturur.*

Artık Projeler sekmesindeki **Web Servisi**

Referansları klasöründe oluşturulan **ScoreWebService**'i göreceksiniz. IDE, referans için birçok destekleyici dosya oluşturmuştur. Tüm oluşturulan java dosyalarını proje panelindeki Dosyalar sekmesinden görebilirsiniz, bu da Şekil 45.7'de gösterilmiştir. Bu dosyalar, Web servisi ile iletişim kurmak için proxy nesnesi tarafından kullanılacaktır.

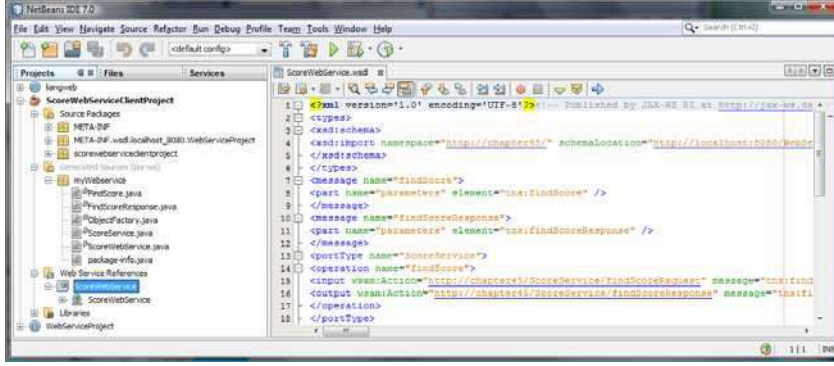


**Şekil 45.7**

*İstemcinin projesindeki Oluşturulan Kaynaklar klasöründe Web hizmetleri için otomatik olarak oluşturulan şablon kodunu görebilirsiniz.*

**<Yan Not: WSDL nedir?>**

NOT: Bir Web servisi referansı oluşturduğunuzda, Şekil 45.6'da gösterildiği gibi bir WSDL URL'si girdiniz. Bu, bir .wsdl dosyası oluşturur. Bu durumda, Web Servisi Referansları klasörü altında ScoreWebService.wsdl olarak adlandırılmıştır, bu da Şekil 45.8'de gösterilmiştir. Peki WSDL nedir? WSDL Web Servisi Tanım Dili anlamına gelir. Bir .wsdl dosyası, istemciye mevcut Web servisini tanımlayan bir XML dosyasıdır—yani, uzaktan erişilebilen yöntemler, parametreleri ve dönüş değerleri türleri, vb.



**Şekil 45.8**

.wsdl dosyası, Web hizmetlerini istemcilere tanımlar.

**<Yan Not: referansı yenile>**

NOT: Web hizmeti değiştirilirse, istemci için referansı yenilemeniz gerekir. Bu nu yapmak için, Web Hizmeti Referansları altındaki Web hizmeti düğmesine sağ tıklayın ve bir bağlam menüsü görüntüleyin ve Yenile İstemci'yi seçin.

Artık Web hizmeti için bir applet istemcisi oluşturmak üzere hazırsınız. Proje panesindeki ScoreWebServiceClientProject düğmesine sağ tıklayın, bir bağlam menüsü görüntülenecek ve Yeni > JApplet seçeneğini seçerek chapter45 paketinde FindScoreApplet adında bir Java applet oluşturun, bu Listing 45.2'de gösterilmiştir.

**Listeleme 45.2 FindScoreApplet.java**

**<Yan Not satır 11: bir hizmet nesnesi oluşturun>**

**<Yan Not satır 12: bir proxy nesnesi oluşturun>**

**<Yan Not satır 39: uzak yöntemi çağırın>**

**<Yan Not satır 51: ana yöntem atlandı>**

paket bölümü45; nimp

ort myWebservice.ScoreWebService;nimport  
myWebservice.ScoreService;nimport java  
x.swing.\*;

java.auto.\*'yu içe aktarın;  
java.awt.event.\*'i içe aktarın;

public class FindScoreApplet extends JApplet {  
// Bir hizmet nesnesi ve bir proxy nesnesi tanımlayın  
private ScoreWebService scoreWebService = new ScoreWebService();  
private ScoreService proxy = scoreWebService.getScoreServicePort();  
  
private JButton jbtGetScore = new JButton("Skoru Al");



```

 özel JTextField jtf Name = new JTextField();ve özel JTextField jtf Score = new JTextField();

 genel geçersiz başlangıç () {
 JPanel jPanel1 = new JPanel();
 jPanel1.setLayout(new GridLayout(2, 2));
 jPanel1.add(new JLabel("İsim"));
 jPanel1.add(jtfName);
 jPanel1.add(new JLabel("Skor"));
 jPanel1.add(jtfScore);

 add(jbtGetScore, BorderLayout.SOUTH);n add(jPanel1, BorderLayout.CENTER);n jbtGetScore.addActionListener(new ActionListener() {n public void actionPerformed(ActionEvent evt) {n getScore();

 }
 });
 }

 özel void getScore() {n deneyin
 {
 // Öğrenci skorunu al
 çift puan = proxy.findScore(jtfName.getText().trim());

 // Sonucu göster
 if (score < 0)
 jtfScore.setText("Bulunamadı");
 else
 jtfScore.setText(new Double(score).toString());n
 }

 catch(İstisna örneği) {
 ex.printStackTrace();
 }
}
N}

```

Program, bir Web hizmeti nesnesi oluşturur (satır 11) ve Web hizmeti ile etkileşimde bulunmak için bir proxy nesnesi oluşturur (satır 12).

Bir öğrenci için bir not bulmak üzere, program proxy nesnesi üzerinde uzaktaki findScore yöntemini çağırır (satır 39).

## 45.5 Argümanları Geçme ve Döndürme

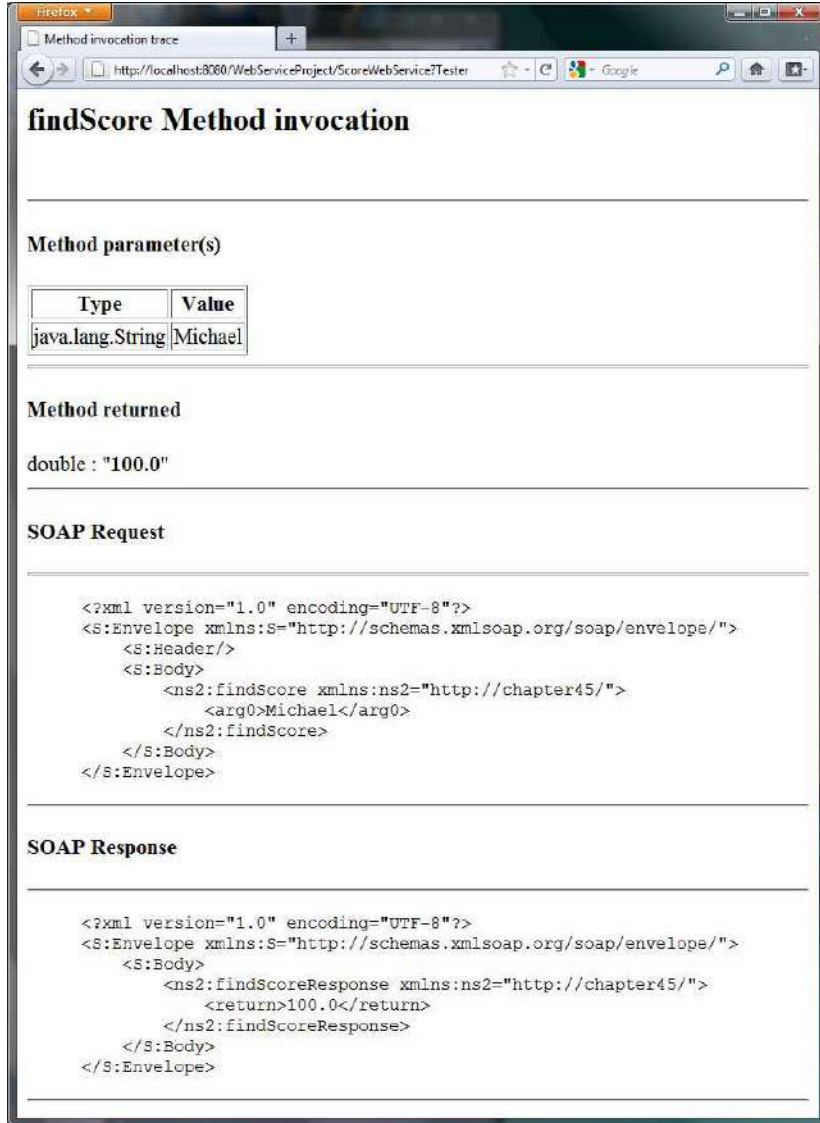
### <Yan Açıklama: SABUN>

Önceki örnekte, oluşturduğunuz bir Web hizmeti istemcisi, bir dize argümanı ile findScore yöntemini çağırır ve Web hizmeti yöntemi çalıştırır ve bir notu double değer olarak döndürür. Bu nasıl çalışır? İstemci ve sunucu arasındaki iletişimi kolaylaştıran Basit Nesne Erişim Protokolü (SOAP) 'dır.

### <Yan Not: SOAP isteği>

### <Yan Not: SOAP yanıtı>

SOAP, XML'e dayanmaktadır. İstemci ile sunucu arasındaki mesaj XML'de tanımlanmıştır. Şekil 45.9, findScore yöntemine ait SOAP isteği ve SOAP yanıtını göstermektedir.



#### Şekil 45.9

*Müşteri isteği ve sunucu yanıtı XML formatında tanımlanmıştır.*

findScore yöntemini çağırırken, SOAP isteği sunucuya gönderilir. İstek, yöntem ve argüman hakkında bilgileri içerir. Şekil 45.9'da gösterildiği gibi, XML metni

```
<ns1:findScore>n
 <arg0>Michael</arg0> v
e</ns1:findScore>
```

findScore yönteminin

Michael argümanı ile çağıldığını belirtir.

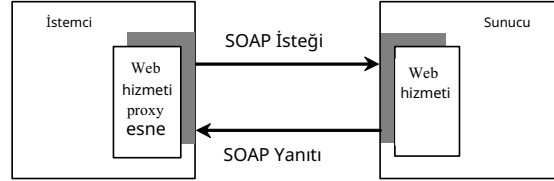
SOAP isteğini aldıktan sonra, Web servisi bunu ayrıştırır. Ayrıştırdıktan sonra, Web servisi uygun bir yöntemi çağırır.

belirtilen argümanlar (varsa) ve yanıtı SOAP formatında geri gönderir  
yanıt. Şekil 45.9'da gösterildiği gibi, XML metni

```
<ns1:findScoreResponse>
 <return>100.0</return>
</ns1:findScoreResponse>
```

metodun 100.0 döndürdüğünü belirtir. \_\_\_\_\_

Proxy nesnesi, Web hizmetinden SOAP yanıtını alır  
ve ayırıştırır. Bu süreç Şekil 45.10'da gösterilmektedir.



**Şekil 45.10**

*Bir proxy nesnesi SOAP istekleri gönderir ve SOAP yanıtları alır.*

**<Yan Not: XML serileştirmesi>**

**<Yan Not: XML serileştirmeden çıkarma>**

Bir istemci ile bir Web servisi arasında herhangi bir türde argüman geçirebilir m isiniz? Hayır. SOAP yalnızca ilkel türleri, sarmalayıcı türleri, dizileri, String, Date, Time, List ve birkaç diğer türü destekler. Ayrıca belirli özel sınıfları da destekler. Bir sunucuya gönderilen veya sunucudan alınan bir nesne XML formatına serileştirilir. Nesnelerin serileştirilmesi/serileştirmeden çıkarılması süreç, otomatik olarak gerçekleştirilen XML serileştirmesi/serileştirmeden çıkarma olarak adlandırılır. Özel bir sınıfın Web yöntemleri ile kullanılabilmesi için, sınıf aşağıdaki gereksinimleri karşılamalıdır:

**<Yan Not: argüman almayan yapıcı>**

Sınıfın argüman almayan bir yapıcıya sahip olması gerekir.

**<Yan Not: get ve set yöntemleri>**

Serileştirilmesi gereken örnek değişkenlerin, kamuya açık get ve set yöntemlerine sahip olması gerekir. Bu değişkenlerin sınıfları SOAP tarafından desteklenmelidir.

Özel bir sınıfın nesne argümanını nasıl geçireceğini göstermek için, Liste 45.3, iki uzak yöntemle sahip AddressService adında bir Web hizmeti sınıfı tanımlar:

getAddress(String firstName, String lastName) belirtilen firstName ve lastName için bir Address nesnesi döndürür.

storeAddress(Address address) bir Student nesnesini veritabanına kaydeder.

Adres bilgileri, database'de Address adında bir tabloda saklanmaktadır. Address sınıfı, Listing 42.12'de tanımlanmıştır, Address.java. Bir Address nesnesi, Address sınıfının tüm özellikleri için get ve set yöntemleri ile birlikte bir argüman almayan bir yapıcıya sahip olduğu için, uzaktan bir metoda geçirilebilir veya döndürülebilir.

İşte AddressService adında bir Web hizmeti ve projedeki Address sınıfını oluşturmak için adımlar.

Proje panelinde WebServiceProject düğmesine sağ tıklayın ve bir bağlam menüsü görüntülemek için. Yeni > Web Servisi'ni seçin. Yeni Web Servisi iletişim kutusunu görüntülemek için.

Web Servisi Adı alanına AddressService yazın.

Paket alanı, chapter45 girin. Tıklayın. Bitir servisi oluşturmak için sınıf.

Proje panelinde WebServiceProject düğmesine sağ tıklayın ve bir bağlam menüsü görüntüleyin. Yeni > Java Sınıfı' nı seçin ve Yeni Java Sınıfı iletişim kutusunu görüntüleyin. Sınıf Adı alanına Address yazın. Paket alanına, chapter42 yazın. Sınıfı oluşturmak için Bitir butonuna tıklayın.

Adres sınıfı, Liste 42.12'de gösterildiği gibidir. AdresServisi sınıfını, Liste 45.3'te gösterildiği gibi tamamlayın.

### Liste 45.3 AddressService.java

<Yan Not 8. satır: hizmet adını tanımla>  
<Yan Not 12. satır: hazırlanan ifade>  
<Yan Not 15. satır: hazırlanan ifade>  
<Yan Not 18. satır: veritabanını başlat>  
<Yan Not 21. satır: uzak yöntemi tanımla>  
<Yan Not 22. satır: getAddress>  
<Yan Notu satır 49: uzak yöntemi tanımla>  
<Yan Notu satır 50: storeAddress>  
<Yan Notu satır 68: veritabanını başlat>

paket chapter45;

```
import chapter42.Adres;
import java.sql.*;
import javax.jws.WebMethod;
import javax.jws.WebService;
```

```
@WebService(name = "AdresServisi",
 serviceName = "AdresWebServisi")
```

```
public class AdresServisi {
 // bir adresi almak için statement1 ve saklamak için statement2
 private PreparedStatement statement1;
```

```
 // bir adresi saklamak için statement2
 private PreparedStatement statement2;
```

```
 public AdresServisi() {
 initializeJdbc();
 }
```

```
@WebMethod(operationName = "getAddress")
 public Adres getAddress(String firstName, String lastName) {
```

```
 try {
 deyim1.setString(1, firstName);n deyim1.set
 String(2, lastName);n ResultSet resultSet =
 deyim1.executeQuery();
```

```
 if (resultSet.next()) {
 Adres adresi = new Adres();n adres.setFi
 rstName(resultSet.getString("firstName"));n adres.setLastName(resul
 tSet.getString("lastName"));n adres.setMi(resultSet.getString("mi"))
 ;
 adres.setTelephone(resultSet.getString("telefon"));n adres.setFirstNa
 me(resultSet.getString("email"));n adres.setCity(resultSet.getStri
 ng("telefon"));n adres.setState(resultSet.getString("durum"));n
 adres.setZip(resultSet.getString("zip"));
```

```

 return adres;
 }
 aksi takdirde
 boş değer döndür;
 } catch (SQLException ex) {n ex.pri
 ntStackTrace();n }

 null değerini döndür;n }

@WebMethod(operationName = "storeAddress")n pu
blic void storeAddress(Adres adresi) {n deneyin {

 deyim2.setString(1, adres.getLastName());n deyim2.setStrin
 g(2, adres.getFirstName());n deyim2.setString(3, adres.getMi
 ());
 deyim2.setString(4, adres.getTelephone());n deyim2.setStrin
 g(5, adres.getEmail());n deyim2.setString(6, adres.getStr
 eet());n deyim2.setString(7, adres.getCity());n deyim2.set
 String(8, adres.getState());n deyim2.setString(9, adres.
 getZip());n deyim2.executeUpdate();

 } catch (SQLException ex) {n ex.pri
 ntStackTrace();n }

}

/** Veritabanı bağlantısını başlat */
public void initializeJdbc() {
 dene {
 Class.forName ("com.mysql.jdbc.Driver") ;

 // Örnek veritabanına bağlan
 Connection connection = DriverManager.getConnection(
 "jdbc:mysql://localhost/javabook", "scott", "tiger");

 bildiri1 = bağlantı.prepareStatement(n "adresten * s
 eçin; burada ilkAd = ve soyadı = ");n bildirim2 = bağlantı.prepareStatement(
 "Adrese Ekle" +
 "(soyadı, adı, mi, telefon, e-posta, sokak, şehir, "n + "eyalet, zip) değerleri (, ,
 , , , , ,)");
 } catch (Exception ex) {n ex.pri
 ntStackTrace();n }

}
N}

```

Yeni Web servisi, AddressService sınıfı için AddressWebService (satır 9) olarak a  
dlandırılmıştır.

Servis dağıtıldığında, AddressWebService'in yapıcı metodu (satırlar 17-19) bir v  
eritabanı bağlantısını başlatmak ve prepared statement1 ile statement2'yi (satırla  
r 68-85) oluşturmak için çağrılır.

findAddress metodu, belirtilen firstName ve lastName için Address tablosunda a dresi arar. Bulduğunda, adres bilgileri bir Address nesnesi (satırlar 29-38) ola rak döndürülür. Aksi takdirde, metod null döndürür (satır 41).

storeAddress metodu, Address nesnesinden gelen adres bilgilerini veritabanına (satırlar 52-61) kaydeder.

NOT:

<Yan Not: veritabanı sürücüsü>

Bu örneğin çalışması için MySQL kütüphanesini WebServiceProject'e eklemeyi unutmayın.

Servisi kullanmadan önce, onu dağıtmalısınız. Proje'deki WebServiceProje ct düğmesine sağ tıklayın, bir bağlam menüsü görüntülemek için ve Dağıt seçe neğini seçin.

Artık AddressWebService'i kullanan bir Web istemcisi geliştirmeye haz ırsınız. İstemci, Şekil 45.11'de gösterildiği gibi bir JSP programıdır. Programın iki işle vi vardır. İlk olarak, kullanıcı soyadını ve adını girebilir ve bir kaydı aramak için A ra butonuna tıklayabilir, bu Şekil 45.12'de gösterilmiştir. İkinci olarak, kullanıcı tam adres bilgilerini girebilir ve bilgileri veritabanına kaydetmek için Kaydet butonuna t ıklayabilir, bu Şekil 45.13'te gösterilmiştir.

Firefox - Address Information

http://localhost:8080/AddressWebServiceClientProject/TestAddressWebService.jsp

Last Name \* First Name \* MI

Telephone Email

Street

City State Georgia-GA Zip

Search Store Reset

\* required fields

Şekil 45.11

*TestAddressWebService sayfası kullanıcının adresleri aramasına ve adresleri saklamasına olanak tanır.*

Firefox - Address Information

http://localhost:8080/AddressWebServiceClientProject/TestAddressWebService.jsp

Last Name \* Pam First Name \* Peter MI

Telephone Email

Street

City State Georgia-GA Zip

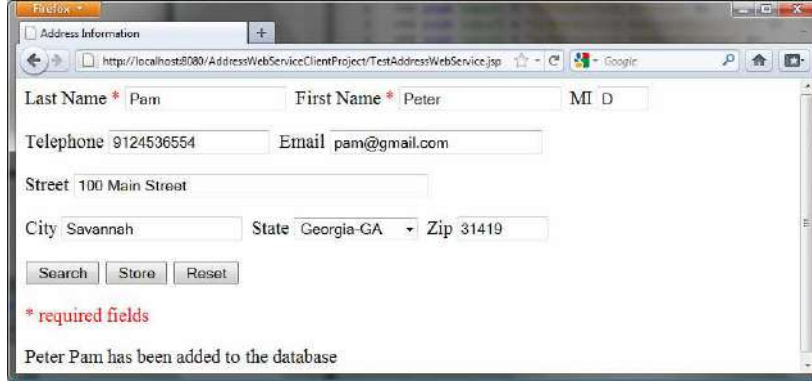
Search Store Reset

\* required fields

Peter Pam is not in the database

Şekil 45.12

*Arama düğmesi bir adres bulur ve görüntüler.*



**Şekil 45.13**

*Depola düğmesi adresi veritabanına kaydeder.*

Müşteri için bir proje oluşturalım. AddressWebServiceClientProject adında bir proje aşağıdaki gibi oluşturulabilir:

Yeni Web Uygulaması iletişim kutusunu görüntülemek için Dosya > Yeni Proje'yi seçin.

Yeni Web Uygulaması iletişim kutusunda, Kategoriler panelinde Java Web'i seçin ve Projeler panelinde Web Uygulaması'nı seçin.

İsim ve Konum iletişim kutusunu görüntülemek için İleri'ye tıklayın.

AdresWebServiceClientProjesi olarak proje adını girin, projenin saklanacağı yeri belirtin ve Ana Proje olarak ayarla onay kutusunu işaretlemeyin. Sunucu ve Ayarlar iletişim kutusunu görüntülemek için İleri'ye tıklayın.

Sunucu alanında GlassFish Server 3'ü seçin ve Java EE Sürüm alanında Java EE 6 Web olarak belirtin, ardından projeyi oluşturmak için Bitir'e tıklayın.

**<Yan Not: Web hizmeti referansı>**

Bu projeye bir Web hizmeti referansı oluşturmanız gerekiyor. Referans, Web hizmeti ile etkileşimde bulunmak için bir proxy nesnesi oluşturmanıza olanak tanıyacaktır. Bir Web hizmeti referansı oluşturmak için adımlar şunlardır:

Proje panelinde AdresWebServiceClientProjesi düğmesine sağ tıklayın ve bir bağlam menüsü görüntüleyin. Yeni > Web Servis İstemcisi'ni seçerek Yeni Web Servis İstemcisi iletişim kutusunu görüntüleyin.

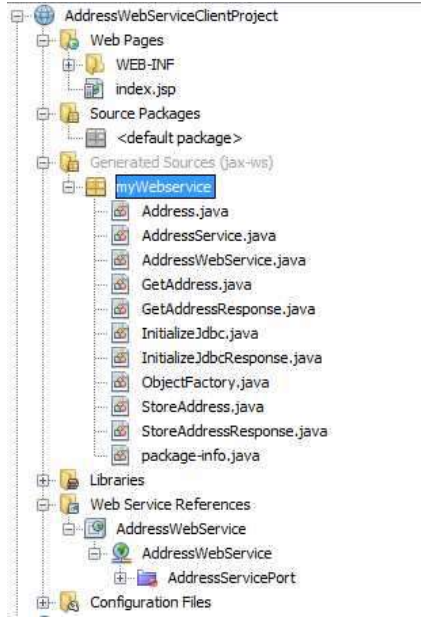
WSDL URL radyo düğmesini kontrol edin ve WSDL URL alanına

<http://localhost:8080/WebServiceProject/AddressWebService?WSDL>

adresini girin.

3. Paket adı alanına myWebservice yazın ve JAX sürümü olarak JAX-WS seçin. Web hizmeti referansını oluşturmak için Finish butonuna tıklayın.

Artık AddressWebService'e bir referans oluşturuldu. Bu işlemin ayrıca Address.java dosyasını istemci projesine kopyaladığını unutmayın; bu, Şekil 45.14'te gösterilmiştir.



**Şekil 45.14**

*Address.java otomatik olarak Web hizmeti istemci referans paketine kop yalanır.*

Listing 45.4'te gösterildiği gibi, AddressWebServiceClientP roject projesinde TestAddressWebService adında bir JSP oluşturun.

#### Listeleme 45.4 TestAddressWebService.jsp

*<Side Remark line 2: Address'i içe aktar>*

*<Side Remark line 3: AddressWebServices'i içe aktar>*

*<Side Remark line 4: AddressServices'i içe aktar>*

*<Side Remark line 14: aynı sayfayı çağır>*

*<Side Remark line 82: Web hizmeti oluştur>*

*<Side Remark line 83: proxy nesnesini al>*

*<Side Remark line 85: Store butonunu işle>*

*<Side Remark line 86: uzak yöntemi çağır>*

*<Side Remark line 90: Search butonunu işle>*

*<Side Remark line 91: uzak yöntemi çağır>*

```
<!-- TestAddressWebService.jsp -->
<%@ sayfa içe aktarma = t "myWebservice.Address"t %>n
<%@ sayfa içe aktarma = t "myWebservice.AddressWebService"t %>n<
%@ sayfa içe aktarma = t "myWebservice.AddressService"t %>n<js
p:useBean idt = t "addressId"
sınıf="myWebservice.Address" kapsam = "oturum"></jsp:useBean>
<jsp:setProperty name = "addressId" property = "*" />

<html>
<kafa>
<title>Adres Bilgisi</title>
</head>
<gövde>
<form method = to "post"t actiont = t "TestAddressWebService.jsp"t>
```



```

Soyadı*
<input type = "text" name = "lastName"
 <%if (addressId.getLastName() != null) {
 out.print("value = \"\" + addressId.getLastName() + "\"");}%>
 size = "20" />

Ad*
<input type = "text" name = "firstName"
 <%if (addressId.getFirstName() != null) {
 out.print("value = \"\" + addressId.getFirstName() + "\"");}%>
 size = "20"/>

BEN
<input type = "text" name = "mi"
 <%if (addressId.getMi() != null) {
 out.print("value = \"\" + addressId.getMi() + "\" "); } %>
 size = "3"/>

<p>Telefon
<input type = "text" name = "telefon"
 <%if (addressId.getTelephone() != null) {
 out.print("value = \"\" + addressId.getTelephone() + "\" ");}%>
 size = "20"/>

E-posta
<input type = "text" name = "email"
 <%if (addressId.getEmail() != null) {
 out.print("value = \"\" + addressId.getEmail() + "\" ");}%>
 size = "28"/>
</p>

<p>Sokak
<input type = "text" name = "street"
 <%if (addressId.getStreet() != null) {
 out.print("value = \"\" + addressId.getStreet() + "\" ");}%>
 size = "50"/>
</p>

<p>Şehir
<input type = "text" name = "city"
 <%if (addressId.getCity() != null) {
 out.print("value = \"\" + addressId.getCity() + "\" ");}%>
 size = "23"/>

Eyalet
<select size = "1" name = "state">
 <option value = "GA">Georgia-GA</option>
 <option value = "OK">Oklahoma-OK</option>
 <option value = "IN">Indiana-IN</option>
</select>

Posta Kodu
<input type = "text" name = "zip"
 <%if (addressId.getZip() != null) {
 out.print("value = \"\" + addressId.getZip() + "\" "); } %>
 size = "9"/>

```

```

</p>

<p><input type = "submit" name = "Submit" value = "Ara">
 <input type = "submit" name = "Submit" value = "Depola">
 <input type = "reset" value = "Sıfırla">
</p>
</form>
<p>* zorunlu alanlar</p>

<%
if t (request.getParameter("Submit") != t nullt) {n Adres
WebService adresiWebService = new AdresWebService(); n AdresServis proxy'
si = adresWebService.getAddressServicePort();

ift (request.getParameter("Submit"t).equals(t"Store"t)) {n proxy.st
oreAddress(addressId);
 out.println(addressId.getFirstName() + " " +
 addressId.getLastName() + " veritabanına eklendi");
}
else ift (request.getParameter("Submit"t).equals(t"Search"t)) { n Adres ad
resi = proxy.getAddress(addressId.getFirstName(), n adresId.getLastName(
));
 ift (adres == t nullt)
 out.print (addressId.getFirstName() + " " +
 addressId.getLastName() + " veritabanında yoktur");

 başka
 adresId = adres; N }

%>
</body>için
de</html>

```

2-4. satırlar JSP sayfası için sınıfları içe aktarır. Address sınıfı (2. satır) WebServiceProject'te oluşturuldu ve AddressWebService için bir Web hizmeti referansı oluşturulduğunda otomatik olarak AddressWebServiceClientProjesine kopyalandı. Address için bir JavaBeans nesnesi oluşturuldu ve 5-7. satırlardaki girdi parametreleri ile ilişkilendirildi.

Kullanıcı arayüzü, (satırlar 14-77) biçiminde düzenlenmiştir. İki butonun eylemi Ara ve Kaydet aynı sayfayı TestAddressWebService.jsp (satır 14) çağırır.

Bir butona tıklandığında, AddressWebService için bir proxy nesnesi elde edilir (satırlar 82-83). Kaydet butonu için, proxy nesnesi storeAddress yöntemini çağırarak veritabanına bir adres ekler (satır 86). Ara butonu için, proxy nesnesi getAddressServicePort yöntemini çağırarak bir adres döndürür (satırlar 91-92). Belirtilen ad ve soyad için hiçbir adres bulunamazsa, döndürülen adres null'dır (satır 93).

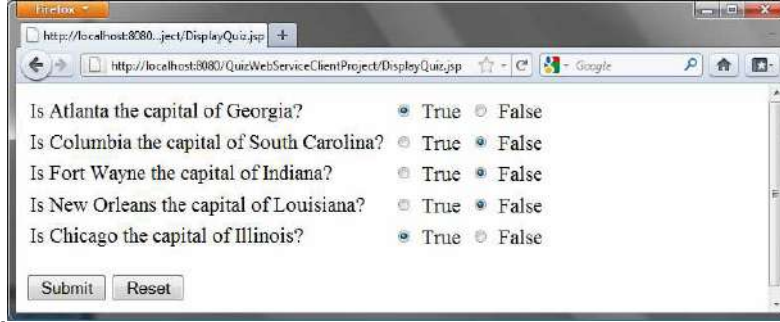
## 45.6 Web Servisi Oturum Takibi

### <Yan Not: HttpSession>

§42.8.3, "Servlet API Kullanarak Oturum Takibi," servletler için oturum takibini tanıtmıştır. javax.servlet.http.HttpSession arayüzünü kullanarak. HttpSession'ı Web hizmetleri için oturum takibi uygulamak üzere kullanabilirsiniz. Bunu göstermek için, istemci için rastgele Doğru/Yanlış

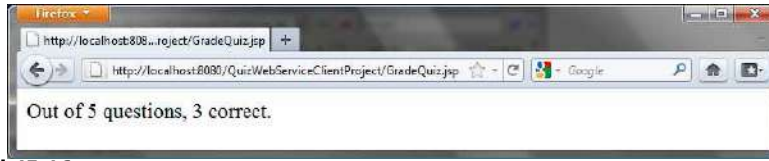
soruları üreten ve bu sorulara verilen cevapları puanlayan bir örneği düşünün.

Web istemcisi, iki JSP sayfasından oluşur: DisplayQuiz.jsp ve GradeQuiz.jsp. DisplayQuiz sayfası, soruları görüntülemek için `getQuestion()` hizmet yöntemini çağırır; bu, Şekil 45.15'te gösterilmiştir. Gönder butonuna tıkladığınızda, program `gradeQuiz` hizmet yöntemini çağırarak cevapları değerlendirir. Sonuç, Şekil 45.16'da gösterildiği gibi GradeQuiz sayfası nda görüntülenir.



Şekil 45.15

Gönder butonu, cevapları notlandırma için gönderir.



Şekil 45.16

Cevaplar puanlanır ve görüntülenir.

Neden bu proje için oturum takibi gereklidir? Her seferinde bir istemci bir quiz görüntülediğinde, istemci için quizin rastgele yeniden sıralanmasını sağlar. Her istemci her seferinde farklı bir quiz alır. DisplayQuiz sayfası yenilendi. Müşteri cevabı gönderdiğinde, Web servisi cevabı daha önce oluşturulan quiz ile kontrol eder. Bu nedenle quiz oturumda saklanmalıdır.

Kolaylık olması açısından, QuizService adında bir Web hizmeti sınıfı oluşturalım. WebServiceProject içinde chapter45 paketinde. Listeleme 45.5 programı vermektedir.

#### Liste 45.5 QuizService.java

<Yan Notu 9. satır: oturum takibini etkinleştir>  
<Yan Notu 10. satır: hizmet adını tanımla>  
<Yan Notu 12. satır: sınav>  
<Yan Notu 16. satır: quiz'i başlat>  
<Yan Notu 28. satır: karıştır>  
<Yan Notu 31. satır: hizmet yöntemini tanımla>  
<Yan Notu 32. satır: getQuestions>  
<Yan Notu 42. satır: hizmet yöntemini tanımla>  
<Yan Notu 43. satır: gradeQuiz>  
<Yan Notu satır 46: cevapları kontrol et>

paket chapter45;

```
import javax.jws.WebMethod;
import javax.jws.WebService;
```

Java.util.List'i içe aktarın;  
Java.util.ArrayList'i içe aktarın;  
com.sun.xml.ws.developer.servlet.HttpSessionScope'u içe aktarın; n@

HttpSessionScope

@WebService(name = "QuizService", serviceName = "QuizWebService")

publik sınıf QuizService {

özel ArrayList<Object[]> quiz = yeni ArrayList<Object[]>();

public QuizService() {

// Soruları ve cevapları başlat

quiz.add(new Object[]{

"Atlanta, Georgia'nın başkenti mi?", true});

quiz.add(new Object[]{

"Columbia, Güney Carolina'nın başkenti mi?", true});

quiz.add(new Object[]{

"Fort Wayne, Indiana'nın başkenti mi?", false});

quiz.add(new Object[]{

"New Orleans, Louisiana'nın başkenti mi?", false});

quiz.add(new Object[]{

"Chicago, Illinois'un başkenti mi?", false}); // Müşteri için

rastgele bir quiz oluşturmak için karıştır

@WebMethod(operationName = "getQuestions")

public java.util.List<String> getQuestions() {

// Quizden soruları çıkar

List<String> soruları = new ArrayList<String>(); n for (int i = 0; i

< quiz.size(); i++) {

Questions.add((String)(quiz.get(i)[0])); N }

return questions; // Quizdeki soruları döndür

@WebMethod(operationName = "gradeQuiz") n

public List<Boolean> gradeQuiz(List<Boolean> answer) {n List<Boole

an> result = new ArrayList<Boolean>(); n for (int i = 0; i < qui

z.size(); i++)

result.add(quiz.get(i)[1] == cevaplar.get(i)); n sonuç dön

dürün;

}

N}

QuizService adında bir Web hizmeti sınıfı,

getQuestions ve gradeQuiz adında iki metoda sahiptir. Yeni Web hizmeti

QuizWebService olarak adlandırılmıştır (satır 10).

@HttpSessionScope notasyonu (satır 9) JAX-WS 2.2'de yenidir,

bu, Web hizmetinin her istemci oturumu için ayrı bir

örneği otomatik olarak sürdürmesini sağlar. Bu notasyonu kullanmak için,

projenizin kütüphanesine JAX-WS 2.2 eklemeniz gerekir. Bu,

projedeki *Library* düğmesine tıklayıp *Add Library* seçerek yapılabilir.

<Side Remark: bir quiz oluşturma>

Beş Doğru/Yanlış sorusunun hizmetten mevcut olduğunu varsayalım. Quiz, bir ArrayList içinde saklanmaktadır (satırlar 16-25). Listede her bir eleman, iki değerden oluşan bir dizedir. İlk değer, soruyu tanımlayan bir dizedir ve ikinci değer, cevabın doğru mu yoksa yanlış mı olması gerektiğini belirten bir Boolean değeridir.

#### <Yan Not: rastgele karıştırma>

Bir yeni quiz, yapıcıda oluşturulur ve quiz, Collections sınıfındaki karıştırma yöntemi kullanılarak karıştırılır (satır 28) .

#### <Yan Not: getQuestions>

getQuestions yöntemi (satırlar 31-40) soruları bir listede döndürür. Sorular quizden (satırlar 34-37) çıkarılır ve döndürülür (satır 39).

#### <Yan Not: gradeQuiz>

gradeQuiz yöntemi (satırlar 42-49) istemciden gelen cevapları quizdeki cevaplarla kontrol eder. İstemcinin cevapları anahtar ile kıyaslanır ve notlandırma sonucu bir listede saklanır. Listede her bir eleman, cetvelin doğru veya yanlış olduğunu gösteren bir boolean değeridir (satırlar 44-46).

#### <Yan Not: Web hizmeti istemcisi oluşturma>

Web hizmetini oluşturduktan ve yayımladıktan sonra, istemci için bir proje oluşturulur.  
QuizWebServiceClientProject adlı proje aşağıdaki gibi oluşturulabilir:

Yeni Web Uygulaması iletişim kutusunu görüntülemek için Dosya > Yeni Proje'yi seçin.  
Yeni Web Uygulaması iletişim kutusunda, Kategoriler panelinde Java Web'i seçin ve Projeler panelinde Web Uygulaması'nı seçin.  
İsim ve Konum iletişim kutusunu görüntülemek için İleri'ye tıklayın.  
QuizWebServiceClientProject'u proje adı olarak girin, projenin saklanacağı konumu belirtin ve  
Ana Proje olarak ayarla onay kutusunu işaretlemeyin. İleri butonuna tıklayın ve Sunucu ve Ayarlar iletişim kutusunu görüntüleyin.  
Sunucu alanında GlassFish Server 3'ü seçin ve Java EE Sürüm alanında Java EE 6 Web olarak belirtin, ardından projeyi oluşturmak için Bitir'e tıklayın.

#### <Yan Not: Web hizmeti referansı>

QuizWebService'i kullanmak için, aşağıdaki gibi bir Web hizmeti istemcisi oluşturmanız gerekir:

##### Proje

panesinde QuizWebServiceClientProject projesine sağ tıklayın ve bir bağlam menüsü görüntüleyin. Yeni > Web Servis İstemcisi seçeneğini seçin  
Yeni Web Servis İstemcisi iletişim kutusunu görüntülemek için.  
WSDL URL radyo düğmesini işaretleyin ve

<http://localhost:8080/WebServiceProject/QuizWebService?WSDL>

WSDL URL alanına girin.

Paket alanına myWebservice yazın.

Bitir düğmesine tıklayarak QuizWebService için referansı oluşturun.

Artık QuizWebService'e bir referans oluşturuldu. QuizService'deki uzak yöntemlere erişmek için bir proxy nesnesi oluşturabilirsiniz. Liste 45.6 ve 45.7, DisplayQuiz.jsp ve GradeQuiz.jsp'yi göstermektedir.

#### Liste 45.6 DisplayQuiz.jsp

##### <Yan Açıklama satırı 2: QuizWebService'i içe aktar>n<

##### Yan Açıklama satırı 3: QuizServices'i içe aktar>

**<Yan Açıklama satırı 4: QuizWebServices oluşturun>n<**  
**Yan Açıklama satırı 11: proxy nesnesini alın> n<Y**  
**an Açıklama satırı 12: soruları alın> n<Yan Açı**  
**kılama satırı 20: soruları görüntüleyin>**

```
<!-- DisplayQuiz.jsp -->
<%@ page importt = t "myWebservice.QuizWebService"t %>n<
%@ page importt = t "myWebservice.QuizService"t %>n<js
p:useBean idt = t "quizWebService"tscopet =t "session"nclasst = t
"myWebservice.QuizWebService"t>
</jsp:useBean>n

<html>
<gövde>n
<%
QuizService proxy = quizWebService.getQuizServicePort();
java.util.List<String> sorular =
 (java.util.ArrayList<String>) (proxy.getQuestions());
%>
<form method = to "post"t actiont = t "GradeQuiz.jsp"t>n<t
able>
<% fort (tintt i = t 0t; i < Questions.size(); i++) {t%>n <tr>

<td>
<label><%=t Questions.get(i) t %></label>n </t
d>
<td>
<giriş türü = r "radyo"t adı = <%= t "soru"t + it%>nvaluet = t "D
oğru"t /> Doğru
</td>
<td>
<giriş türü = r "radyo"t namet = t <%=t "soru"t + it%>nvaluet =
to "Yanlış"t />t Yanlış
</td>
</tr>
<js>
</table>
<p><input type = "submit"name = "Submit"value = "Gönder">
<input type = "reset"value = "Sıfırla">
</p>
</form> ve<
/body>içind
e</html>
```

Bu sayfa, 12-13. satırlardaki getQuestions() çağrısını yaparak bir quiz oluşturur. Sorular, radyo düğmeleri ile birlikte bir tabloda gösterilir (16-32. satırlar). Gönder'e tıklamak, GradeQuiz.jsp'yi çağırır.

#### Liste 45.7 GradeQuiz.jsp

**<Yan Not 2. satır: QuizWebService'i içe aktar>**  
**<Yan Not 3. satır: QuizServices'i içe aktar>**  
**<Yan Not 4. satır: QuizWebServices oluştur>**  
**<Yan Not 11. satır: proxy nesnesini al>**  
**<Yan Not 12. satır: soruları al>**  
**<Yan Not 15. satır: müşterinin cevaplarını al>**  
**<Yan Not 25. satır: cevapları notlandır>**

### <Yan Notu satır 28: analiz sonucu>

```
<!-- GradeQuiz.jsp -->
<%@ sayfa import = "myWebservice.QuizWebService"%>
<%@ sayfa import = "myWebservice.QuizService"%>
<jsp:useBean id = "quizWebService" kapsam = "oturum"
 class = "myWebservice.QuizWebService">
</jsp:useBean>n

<html>
<gövde>
<%
QuizService proxy'si = quizWebService.getQuizServicePort(); njava.util
.List<String> quiz = proxy.getQuestions();

// Cevabı DisplayQuiz sayfasından al
java.util.List<Boolean> cevaplar = yeni java.util.ArrayList<Boolean>();
for (tintt i = t 0t; i < quiz.size(); i++) { n String tru
 eOrFalse = request.getParameter(t"soru"t + i); nift (trueOrFalse.equals(t"
 Doğru"t))
 answers.add(true); // Doğru yanıtlandı
 else if (trueOrFalse.equals("False"))
 answers.add(false); // Yanlış yanıtlandı
}

// Cevapları değerlendir
java.util.List<Boolean> sonuç = proxy.gradeQuiz(answers);

// Doğru sayısını bul
intt CorrectCount = t 0t;nf
ort (tintt i = t 0t; i < result.size(); i++) { nift (result.ge
 t(i))
 doğruSayı ++; N}

%>

 Toplam <%= result.size() %> sorudan, <%= correctCount %> doğru.
</body>için
de</html>
```

Bu sayfa, DisplayQuiz sayfasından (satırlar 15-21) HTML formundan geçen cevapl arı toplar, gradeQuiz metodunu çağırarak quiz'i değerlendirir (satır 25), doğru sayısını bulur (satırlar 28-31) ve sonucu gösterir (satır 35).

NOT:

Tüm beş soruyu Gönder butonuna tıklamadan önce yanıtlamanız gerekiyor. Bir radyo düğmesi seçilmezse bir çalışma zamanı hatası oluşacaktır. Bu sorunu Alıştırma 45.5'te düzeltebilirsiniz.

#### Anahtar Terimler

- @WebService
- @WebMethod
- Bir Web hizmeti tüketmek
- proxy nesnesi
- Bir Web hizmeti yayınlamak

- Web hizmeti
- Web hizmeti istemci referansı
- WSDL

## Bölüm Özeti

1. Web hizmetleri, bir sistemdeki bir Java programının başka bir sistemdeki bir nesnede bir yöntemi çağırmasını sağlar.
2. Web hizmetleri platformdan ve dilden bağımsızdır. Herhangi bir dil kullanarak Web hizmetleri geliştirebilir ve kullanabilirsiniz.
3. Web hizmetleri, HTTP kullanarak Web üzerinde çalışır. SOAP, Web hizmetlerini uygulamak için popüler bir protokoldür.
4. Sunucu, hizmeti istemciye sunmak zorundadır, bu "Web hizmeti yayınlama" olarak bilinir. Bir istemcinin Web hizmetini kullanması "Web hizmetini tüketme" olarak bilinir.
5. Bir istemci, bir proxy nesnesi aracılığıyla bir Web hizmeti ile etkileşimde bulunur. Proxy nesnesi, istemci ile Web hizmeti arasındaki iletişimi kolaylaştırır.
6. Bir Web hizmetini işaretlemek için Java anotasyonu @WebService'i kullanmanız ve uzaktan bir yöntemi işaretlemek için @WebMethod anotasyonunu kullanmanız gerekir.
7. Bir Web hizmeti sınıfı, sınırsız sayıda uzaktan yöntemle sahip olabilir.
8. Bir Web hizmeti yayımlandıktan sonra, onu kullanmak için bir istemci programı yazabilirsiniz. Öncelikle bir Web istemci referansı oluşturmalsınız. Referanstan, kolaylaştırmak için bir proxy nesnesi oluşturursunuz. bir sunucu ile bir istemci arasındaki iletişim.
9. WSDL, Web Servisi Tanım Dili anlamına gelir. Bir .wsdl dosyası, mevcut Web hizmetini tanımlayan bir XML dosyasıdır. istemci—yani, uzaktaki yöntemler, parametreleri ve dönüş değerleri tipleri vb.
10. İstemci ile sunucu arasındaki mesaj XML'de tanımlanır. Bir SOAP isteği, Web hizmetine gönderilen bilgileri tanımlar ve bir SOAP yanıtı, Web hizmetinde alınan bilgileri tanımlar.
11. İstemci ile Web hizmeti arasında iletilen nesneler XML'de serileştirilir. Tüm nesne türleri SOAP tarafından desteklenmez.
12. HttpSession kullanarak Web hizmetlerinde oturumları, servletlerdeki gibi takip edebilirsiniz.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın:  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

### Gözden Geçirme Soruları

#### Bölüm 45.1

45.1 Web hizmeti nedir?

45.2 Java dışındaki bir dilden bir Web hizmeti çağırabilir misiniz?

45.3 Web hizmetleri geri çağırıyor destekliyor mu? Yani, bir Web hizmeti, istemcinin programından bir yöntemi çağırabilir mi?

45.4 SOAP nedir? Bir Web hizmetini yayınlamak ne demektir? Bir Web hizmetini kullanmak ne demektir? Bir proxy nesnesinin rolü nedir?

#### Bölümler 45.2-45.6

45.5 Bir Web hizmetini belirtmek için hangi anotasyon kullanılır? Bir Web yöntemini belirtmek için hangi anotasyon kullanılır?

45.6 NetBeans'te bir Web hizmetini nasıl dağıtırsınız?



45.7 Bir istemciden bir Web hizmetini test edebilir misiniz?

45.8 Bir istemci için nasıl bir Web hizmeti referansı oluşturursunuz?

45.9 WSDL nedir? SOAP nedir? SOAP isteği nedir? SOAP yanıtı nedir?

45.10 Bir uzak metoda ilkel tür argümanları geçirebilir misiniz? Bir uzak metoda herhangi bir nesne türü geçirebilir misiniz? Bir uzak metoda özel bir tür argüman geçirebilir misiniz?

45.11 Bir Web oturumunu izlemek için nasıl bir HttpSession nesnesi elde edersiniz?

45.12 NetBeans'teki aynı projede bir pakette iki Web hizmeti referansı oluşturabilir misiniz?

45.13 Listing 45.5, QuizService.java'daki 40-41. satırlarda sınavı klonlamazsanız ne olur?

### Programlama Alıştırmaları

45.1\*

(Bir veritabanı tablosundan bir puan alın) Puanların Scores tablosunda saklandığını varsayalım. Tablo şu şekilde oluşturuldu:

```
table Puanları oluşturun (isim t varchar(t20),
n puan t numarası, izin t boolean);
```

```
insert into Scores values ('John', 90.5, 1);
insert into Scores values ('Michael', 100, 1);
insert into Scores values ('Michelle', 100, 0);
```

Listing 45.1, ScoreService.java'daki findScore metodunu, belirtilen isim için bir puan almak üzere gözden geçirin. Programınızın permission sütununa ihtiyacı olmadığını unutmayın; bunu göz ardı edin. Bir sonraki alıştırmada permission sütununa ihtiyaç duyacaktır.

45.2\*

(Skorları bulma izni) Önceki alıştırmayı gözden geçirin, böylece findScore metodu izin false olduğunda -1 döndürsün. 1, 0 veya -1 döndüren getPermission(String name) adında başka bir metod ekleyin. Bu metod, öğrenci Skorlar tablosunda ise ve izin true ise 1, öğrenci Skorlar tablosunda ise ve izin false ise 0, ve öğrenci Skorlar tablosunda değilse -1 döndürür.

45.3\*

(Kredi hesaplama) Belirtilen miktar, yıl sayısı ve yıllık faiz oranı ile bir kredi hesaplayabilirsiniz. Aylık ödeme ve toplam ödeme hesaplamak için iki uzak metod içeren bir Web servisi yazın. Kullanıcıdan kredi miktarını, yıl sayısını ve yıllık faiz oranını girmesini isteyen bir istemci programı yazın.

45.4\*

(Web servisi ziyaret sayısı) getCount() adında bir metod içeren bir Web servisi yazın, bu metod istemciden kaç kez çağrıldığını döndürsün. Sayım değişkenini saklamak için bir oturum kullanın.

45.5★

(Quiz) Kullanıcı, Quiz uygulamasında §45.6, Web Servisi Oturum Takibi'nde gönder butonuna tıklamadan önce beş sorunun hepsine yanıt vermelidir.

**Gönder butonu, bir radyo butonu işaretlenmemişse bir çalışma zamanı hatası oluşacaktır.**

**Bu sorunu düzeltin.**

**\*\*\*Bu bir bonus Web bölümü**

## **BÖLÜM 46**

### **Uzak Yöntem Çağrısı**

Amaçlar

- RMI'nin nasıl çalıştığını açıklamak için (§46.2).
- RMI uygulamaları geliştirme sürecini tanımlamak için (§46.3).
- RMI ile soket düzeyindeki programlama arasındaki farkı ayırt etmek için (§46.4).
- RMI kullanarak üç katmanlı uygulamalar geliştirmek için (§46.5).
- Etkin uygulamalar geliştirmek için geri çağırımları kullanmak (§46.6).

## 46.1 Giriş

Uzak Yöntem Çağırısı (RMI), dağıtılmış Java sistemleri oluşturmak için bir çerçeve sağlar. RMI kullanarak, bir sistemdeki bir Java nesnesi, ağdaki başka bir sistemde ki bir nesnede bir yöntemi çağırabilir. Dağıtılmış bir Java sistemi, ağdaki işbirlikçi dağıtılmış nesnelerin bir koleksiyonu olarak tanımlanabilir. Bu bölümde, RMI'yi kullanarak yararlı dağıtılmış uygulamalar oluşturmayı öğreneceksiniz.

## 46.2 RMI Temelleri

RMI, dağıtılmış nesneler arasında iletişimi kolaylaştırmak için Java Dağıtılmış Nesne Modelidir. RMI, soketlerin üzerine inşa edilmiş daha yüksek seviyeli bir API'dir. Soket düzeyinde programlama, verileri bilgisayarlar arasında soketler aracılığıyla iletmenizi sağlar. RMI, ayrıca uzak bir nesnede yöntemleri çağırmanıza olanak tanır. Uzak nesneler, sanki yerel ana bilgisayarda bulunuyorlarmış gibi işlenebilir. Farklı makineler arasında veri iletimi, JVM tarafından şeffaf bir şekilde yönetilir.

<Yan Not: istemci>

<Yan Not: sunucu>

Birçok açıdan, RMI istemci/sunucu mimarisinin bir evrimidir. Bir *istemci* hizmetler için isteklerde bulunan bir bileşendir ve bir *sunucu* istenen hizmetleri sağlayan bir bileşendir. İstemci/sunucu mimarisi gibi, RMI de istemciler ve sunucular kavramını korur, ancak RMI yaklaşımı daha esneklerdir.

- Bir RMI bileşeni, söz konusu senaryoya bağlı olarak hem istemci hem de sunucu olarak hareket edebilir.
- Bir RMI sistemi, işlevselliği bir sunucudan bir istemciye ve tersine geçirebilir. Tipik olarak, bir istemci/sunucu sistemi yalnızca sunucu ve istemci arasında veri iletir.

### 46.2.1 RMI Nasıl Çalışır?

<Yan Not: yerel nesne>

<Yan Not: uzak nesne>

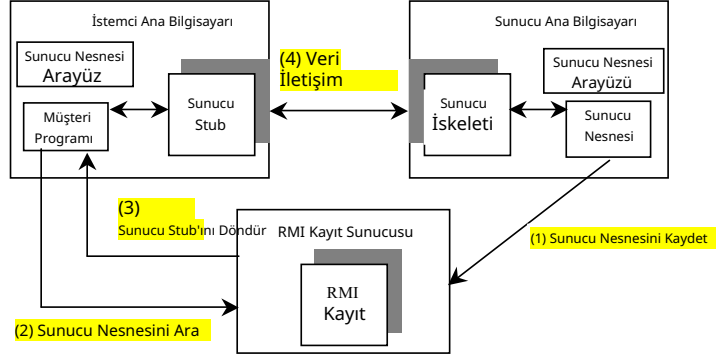
Bu bölümden önce kullandığınız tüm nesnelere yerel nesneler denir. *Yerel nesneler* yalnızca yerel host içinde erişilebilir. Uzak bir host'tan erişilebilen nesneler uzak nesneler denir. Bir nesnenin uzaktan çağırılabilmesi için, hem sunucuya hem de istemciye erişilebilen bir Java arayüzünde tanımlanması gerekir. Ayrıca, arayüz `java.rmi.Remote` arayüzünü genişletmelidir. `java.io.Serializable` arayüzü gibi, `java.rmi.Remote` bir işaretçi arayüzüdür ve sabitler veya yöntemler içermez. Sadece uzak nesneleri tanımlamak için kullanılır.

RMI mimarisinin ana bileşenleri aşağıda listelenmiştir (bkz.

Şekil 46.1):

- Sunucu nesne arayüzü: sunucu nesnesi için yöntemleri tanımlayan `java.rmi.Remote` alt arayüzü.
- Sunucu sınıfı: uzak nesne arayüzünü uygulayan bir sınıf.
- Sunucu nesnesi: sunucu sınıfının bir örneği.
- RMI kaydı: uzak nesneleri kaydeden ve nesneleri bulmak için adlandırma hizmetleri sağlayan bir yardımcı program.
- İstemci programı: uzak sunucu nesnesindeki yöntemleri çağırarak bir program.

- Sunucu stub'ı: istemci ana bilgisayarında bulunan ve uzak sunucu nesnesinin yerini alan bir nesne.
- Sunucu iskeleti: sunucu ana bilgisayarında bulunan ve stub ile gerçek sunucu nesnesiyle iletişim kuran bir nesne.



**Şekil 46.1**

Java RMI, uzak nesneler için adlandırma hizmetleri sağlamak üzere bir kayıt defteri kullanır ve istemci ile sunucu arasındaki iletişimi kolaylaştırmak için stub ve skeleton kullanır.

RMI şu şekilde çalışır:

(1) Bir sunucu nesnesi RMI kayıt defterine kaydedilir.

(2) Bir istemci, uzak

nesneyi bulmak için RMI kayıt defterine bakar.

(3) Uzak nesne bulunduğunda, stub istemciye döndürülür.

(4) Uzak nesne, yerel

nesne gibi kullanılabilir. İstemci ile sunucu arasındaki iletişim, stub ve skeleton aracılığıyla sağlanır.

<Yan Not: stub>

<Yan Not: skeleton>

RMI mimarisinin uygulanması karmaşıktır, ancak iyi haber, RMI'nin parametre geçişini ve uzaktan yöntemleri çağırma işlemleri için sıkıcı kod yazmaktan özgürleştiren bir mekanizma sağlamasıdır. Temel fikir, istemci ve sunucu arasındaki iletişimi yönetmek için *stub* ve *skeleton* olarak bilinen iki yardımcı sınıf kullanmaktır.

Stub ve skeleton otomatik olarak üretilir. *stub*

istemci makinesinde bulunur. İstemcinin sunucu nesnesi hakkında bilmesi gereken tüm referans bilgilerini içerir.

Müşteri bir sunucu nesnesinde bir yöntemi çağırdığında, aslında stub içinde kapsüllenmiş bir yöntemi çağırır. Stub, parametreleri sunucuya göndermekten ve sunucudan sonucu alıp müşteriye geri göndermekten sorumludur.

Skeleton, sunucu tarafındaki stub ile iletişim kuran.

Skeleton, müşteriden parametreleri alır, bunları sunucuya yürütme için iletir ve sonucu stub'a geri döner.

#### 46.2.2 Parametre Geçişi

Müşteri, parametrelerle bir uzak yöntemi çağırdığında, parametrelerin geçişi stub ve skeleton tarafından yapılır.

Açıkça, bir sunucudaki uzak bir nesnede yöntemleri çağırmak, bir istemcideki yerel bir nesnede yöntemleri çağırmaktan farklıdır, çünkü uzak nesne ayrı bir makinede farklı bir adres alanındadır. Üç tür parametreyi ele alalım:

**<Yan Not: ilkel tür>**

- char, int, double veya boolean gibi ilkel veri türleri, \_\_\_\_\_ yerel bir çağrı gibi değerle geçilir.

**<Yan Not: yerel nesne>**

- java.lang.String gibi yerel nesne türleri de değerle geçilir, ancak bu, yerel bir çağrıda bir nesne parametresinin geçişinden tamamen farklıdır. Yerel bir çağrıda, bir nesne parametresinin referansı geçilir, bu da nesnenin bellek adresine karşılık gelir. Uzak bir çağrıda, nesne referansını geçmenin bir yolu yoktur, çünkü bir makinedeki adres, farklı bir JVM için anlamsızdır. Herhangi bir nesne, seri hale getirilebilir olduğu sürece uzak bir çağrıda parametre olarak kullanılabilir. Stub, nesne parametresini seri hale getirir ve bunu ağ üzerinden bir akışta gönderir. Skeleton, akışı bir nesneye geri deserializes eder.

**<Yan Not: uzak nesne>**

- Uzak nesne türleri, yerel nesnelerden farklı bir şekilde iletilir. Bir istemci, bir uzak nesne türünde bir parametre ile uzak bir yöntemi çağırdığında, uzak nesnenin stub'ı iletilir. Sunucu stub'ı alır ve parametreyi onun aracılığıyla işler. Uzak nesnelerin iletilmesi § 46.6, "RMI Geri Çağrılar" bölümünde tartışılacaktır.

**46.2.3 RMI Kaydı**

Bir istemci uzak nesneyi nasıl bulur? RMI kaydı, sunucunun nesneyi kaydetmesi ve istemcinin nesneyi bulması için kayıt hizmetleri sağlar.

LocateRegistry sınıfında, bir Registry'ye referans döndürmek için birkaç aşırı yüklenmiş statik `getRegistry()` yöntemi kullanabilirsiniz; bu, Şekil 46.2'de gösterilmiştir. Bir Registry elde edildikten sonra, `bind` veya `rebind` yöntemi ile kayıt defterinde benzersiz bir ad ile bir nesne bağlayabilir veya `lookup` yöntemi ile bir nesneyi bulabilirsiniz; bu, Şekil 46.3'te gösterilmiştir.

java.rmi.registry.LocateRegistry	y
+getRegistry(): Registryn+getRegistry(port: int): Registryn+getRegistry(ana bilgisayar: String): Registryn+getRegistry(ana bilgisayar:String, port: int): Kayıt defteri	Yerel ana bilgisayar için varsayılan kayıt portu 1099'da uzak nesne Kaydı'na bir referans döndürür. Belirtilen portta yerel ana bilgisayar için uzak nesne Kaydı'na bir referans döndürür. Belirtilen ana bilgisayardaki uzaktan nesne Kaydı'na referans döner, varsayılan kayıt portu 1099'dur. Belirtilen ana bilgisayar ve porttaki uzaktan nesne Kaydı'na referans döner.

**Şekil 46.2**

LocateRegistry sınıfı, bir ana bilgisayarda bir kayıt elde etmek için yöntemler sağlar.

java.rmi.registry.Registry
+bind(name: String, nesne: Remote): void +rebind(name: String, obj: Remote): void +unbind(name: String): void
+list(name: String): String[]
+lookup(name: String): Uzak

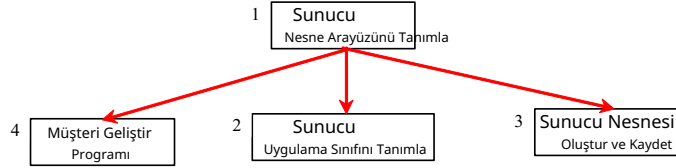
Belirtilen adı uzaktan nesne ile bağlar.  
Belirtilen adı uzaktan nesne ile bağlar. Ad için mevcut olan herhangi bir bağlama değiştirilir.  
Belirtilen adı uzaktan nesne ile ilişkili olan bağlamayı yok eder.  
Kayıtlı isimlerin bir dizisini döndürür.  
Belirtilen isimle ilişkili uzak nesne için bir referans, bir stub döndürür.

### Şekil 46.3

Registry sınıfı, uzak nesne kaydında uzak nesnelere referansları bağlama ve elde etme yöntemlerini sağlar.

### 46.3 RMI Uygulamaları Geliştirme

Artık RMI hakkında temel bir anlayışa sahip olduğunuza göre, basit RMI uygulamaları yazmaya hazırsınız. Bir RMI uygulaması geliştirmenin adımları Şekil 46.4'te gösterilmiştir ve aşağıda listelenmiştir.



### Şekil 46.4

Bir RMI uygulaması geliştirmenin adımları.

1. Sunucu ve istemcileri arasında sözleşme olarak hizmet eden bir sunucu nesne arayüzü tanımlayın, aşağıdaki taslakta gösterildiği gibi:

```
public interface ServerInterface extends Remote {
 public void servise1(...) throws RemoteException;
 // Diğer yöntemler
}
```

Bir sunucu nesne arayüzü java.rmi.Remote arayüzünü genişletmelidir.

2. Sunucu nesne arayüzünü uygulayan bir sınıf tanımlayın, aşağıdaki taslakta gösterilmiştir:

```
public class ServerInterfaceImpl extends UnicastRemoteObject implements ServerInterface {
 public void servise1(...) throws RemoteException {
 // Uygula
 }
 // Diğer yöntemleri uygula
}
```

Sunucu uygulama sınıfı, java.rmi.server.UnicastRemoteObject sınıfından türemelidir. UnicastRemoteObject sınıfı, TCP akışları kullanarak nokta-nokta aktif nesne referansları için destek sağlar.

3. Sunucu uygulama sınıfından bir sunucu nesnesi oluşturun ve bunu bir RMI kaydına kaydedin:

```
ServerInterface sunucusu = t.newt ServerInterfaceImpl(...); nRegistry
y.kayıtları = LocateRegistry.getRegistry(); nregistry.rebind(
t."UzakNesneAdı", sunucu);
```

4. Uzak bir nesneyi bulup yöntemlerini çağıran bir istemci geliştirin, aşağıdaki t aslında gösterildiği gibi:

```
Kayıtları defteri = LocateRegistry.getRegistry(ana bilgisayar);
nServerInterface sunucusu = (ServerInterfaceImpl)
 kayıtları.defteri.lookup(t."UzakNesneAdı");
nserver.service1(...);
```

Aşağıdaki örnek, bu adımlar aracılığıyla bir RMI uygulamasının geliştirilmesini göstermektedir.

#### 46.3.1 Örnek: Bir RMI Sunucusundan Öğrenci Notlarını Alma

Bu örnek, bir RMI sunucusundan öğrenci notlarını alan bir istemci oluşturur. İstemci, Şekil 46.5'te gösterildiği gibi, belirtilen isim için notu görüntüler.



(a) Applet olarak çalıştırma. (b) Uygulama olarak çalıştırma.

#### Şekil 46.5

Notu almak için bir öğrenci adı girip Notu Al butonuna tıklayabilirsiniz.

1. Listing 46.1'de StudentServerInterface adında bir sunucu arayüzü oluşturun. Arayüz, istemciye sunucunun findScore yöntemini çağırarak bir öğrenci notunu nasıl alacağını söyler.

#### Liste 46.1 StudentServerInterface.java

<Yan Not 3. satır: alt arayüz>

<Yan Not 9. satır: sunucu yöntemi>

```
Java.rmi.*'yi içe aktarın;
```

```
genel arayüz ÖğrenciServerInterface Remote'ı genişletir {
```

```
/**
```

```
* Belirtilen isim için puanı döndür
```

```
* @param name öğrenci ismi
```

```
* @return bir double puan veya öğrenci bulunamazsa -1
```

```
*/
```

```
public double findScore(Dize adı), RemoteException'ı atar;n}
```

Uzaktan kullanılabilecek herhangi bir nesne, java.rmi.Remote arayüzünü genişleten bir arayüzde tanımlanmalıdır (satır 3).

StudentServerInterface, Remote'u genişleterek, bir öğrencinin puanını bulmak için bir istemci tarafından uzaktan çağrılabilen findScore metodunu tanımlar. Bu arayüzdeki her metod, bir java.rmi.RemoteException fırlatabileceğini beyan etmelidir (satır 9). Bu nedenle, bu metodu çağıran istemci kodunuz, bu istisnayı bir try-catch bloğunda yakalamaya hazır olmalıdır.



2. StudentServerInterfaceImpl adında bir sunucu uygulaması oluşturun (Liste 46.2) ve StudentServerInterface'i uygulayın. findScore metodu, belirtilen bir öğrenci için notu döndürür. Not bulunamazsa -1 döner.

#### Liste 46.2 StudentServerInterfaceImpl.java

<Yan Not satır 8: hash map>

<Yan Not satır 17: notu sakla>

<Yan Not satır 24: notu al>

```
import java.rmi.*;
import java.rmi.server.*;
import java.util.*;

genel sınıf ÖğrenciServerInterfaceImpl n Unicast
RemoteObject'i genişletiyor
implements StudentServerInterface {
 // İsimle indekslenmiş bir haritada notları saklar
 private HashMap<String, Double> scores =
 new HashMap<String, Double>();

 public ÖğrenciServerInterfaceImpl(), RemoteException'ı oluşturur {
 initializeStudent();
 }

 /** Öğrenci bilgilerini başlat */
 protected void initializeStudent() {
 scores.put("John", new Double(90.5));
 scores.put("Michael", new Double(100));
 scores.put("Michelle", new Double(98.5));
 }

 /** Student arayüzünden findScore metodunu uygula
 */
 public double findScore(Dize adı) RemoteException'ı atar {
 Double d = (Double)scores.get(name);

 eğer (d == boş) {
 System.out.println("Öğrenci " + name + " bulunamadı ");
 return -1;
 }
 başka {
 System.out.println("Öğrenci " + name + "\n\n notu "
 + d.doubleValue());
 return d.doubleValue();
 }
 }
}
```

StudentServerInterfaceImpl sınıfı StudentServerInterface'i uygular. Bu sınıf ayrıca java.rmi.server.RemoteServer sınıfını veya onun alt sınıfını genişletmelidir. RemoteServer, uzak nesneleri oluşturmak ve dışa aktarmak için gereken yöntemleri tanımlayan soyut bir sınıftır. Genellikle onun alt sınıfı java.rmi.server.UnicastRemoteObject kullanılmaktadır (satır 6). Bu alt sınıf, RemoteServer'da tanımlanan tüm soyut yöntemleri uygular.

StudentServerInterfaceImpl, StudentServerInterface'de tanımlanan findScore yöntemi (satırlar 25-37) uygular. Basitlik açısından, üç öğrenci, John, Michael ve Michelle, ve bunların karşılık gelen notları, scores adında bir java.util.HashMap örneğinde saklanmaktadır. HashMap, Java Koleksiyonlar Çerçevesi'ndeki Map arayüzünün somut bir sınıfıdır ve bir anahtar kullanarak bir değeri arayıp almayı mümkün kılar. Hem değerler hem de anahtarlar Objecttipindedir. findScore yöntemi, isim hash haritasında varsa notu döner ve isim bulunamazsa -1 döner.

3. Sunucu uygulamasından bir sunucu nesnesi oluşturun ve RMI sunucusuna kaydedin (Liste 46.3).

#### Liste 46.3 RegisterWithRMIServer.java

<Yan Not 7. satır: sunucu nesnesi>

<Yan Not 8. satır: kayıt referansı>

<Yan Not 9. satır: kayıt>

```
java.rmi.registry.* içe aktar;

public class RegisterWithRMIServer {
 /** Ana yöntem */
 public static void main(String[] args) {n try {

 ÖğrenciSunucuArayüzü nesnesi =
 new StudentServerInterfaceImpl();
 Registry registry = LocateRegistry.getRegistry();
 registry.rebind("StudentServerInterfaceImpl", obj);
 System.out.println("Öğrenci sunucusu " + obj + " kaydedildi");

 }
 catch (Exception ex) {n ex.p
 rintStackTrace();n }

}
```

RegisterWithRMIServer, sunucuyu başlatmaktan sorumlu olan bir ana metoda sahiptir. Aşağıdaki görevleri yerine getirir: (1) bir sunucu nesnesi oluşturmak (satır 8); (2) bir RMI kayıt sunucusuna referans (satır 9) ve (3) nesneyi kayıt sunucusunda kaydetme (satır 10).

- 4.

Listing 46.4'te StudentServerInterfaceClient adında bir applet olarak bir istemci oluşturun. İstemci, RMI'den sunucu nesnesini bulur. kayıt sunucusunu kullanarak puanları bulur.

#### Listeleme 46.4 StudentServerInterfaceClient.java

<Yan Not satır 9: uzak nesne>

<Yan Not satır 11: bağımsız?>

<Yan Not satır 19: RMI'yi başlat>

<Yan Not satır 31: dinleyiciyi kaydet>

<Yan Not satır 33: puanı al>

<Yan Notu satır 61: öğrenciyi bul>

<Yan Notu satır 71: ana yöntem>

<Yan Notu satır 74: bağımsız>

```
javax.swing.*; içe aktar
java.awt.*; içe aktar
import java.awt.event.*; nimp
ort java.rmi.registry.LocateRegistry;
```

```

import java.rmi.registry.Registry;

public class StudentServerInterfaceClient extends JApplet {
 // Bir Öğrenci örneği tanımlayın
 özel StudentServerInterface öğrenci;

 özel boolean bağımsız; // Applet mi yoksa uygulama mı

 private JButton jbtGetScore = new JButton("Skoru Al");
 private JTextField jtfName = new JTextField();
 özel JTextField jtfScore = yeni JTextField();

 genel geçersiz başlangıç () {
 // RMI'yi Başlat
 RMI'yi başlat();

 JPanel jPanel1 = new JPanel();
 jPanel1.setLayout(new GridLayout(2, 2));
 jPanel1.add(new JLabel("İsim"));
 jPanel1.add(jtfName);
 jPanel1.add(new JLabel("Puan"));
 jPanel1.add(jtfScore);

 add(jbtGetScore, BorderLayout.SOUTH);n add(
 jPanel1, BorderLayout.CENTER);n jbtGetSc

 ore.addActionListener(new ActionListener() {n public void actionP
 erformed(ActionEvent evt) {n getScore();

 }

 });
 }

 özel void getScore() {n deneyin
 {
 // Öğrenci puanını al
 çift puan = student.findScore(jtfName.getText().trim());

 // Sonucu göster
 if (score < 0)
 jtfScore.setText("Bulunamadı");
 else
 jtfScore.setText(new Double(score).toString());n }

 catch(İstisna örneği) {
 ex.printStackTrace();
 }
 }

 /** RMI'yi başlat */
 protected void initializeRMI() {
 String host = "";
 if (!isStandalone) ana bilgisayar = getCodeBase().getHost();

 denemek {
 Kayıt defteri kayıt defteri = LocateRegistry.getRegistry(ana bilgisayar);

```

```

 student = (StudentServerInterface)
 registry.lookup("StudentServerInterfaceImpl");
 System.out.println("Sunucu nesnesi " + student + " bulundu");
 }
 catch(Exception ex) {n System
 em.out.println(ex);n }
}

/** Ana yöntem */
public static void main(String[] args) {n StudentsSe
 rverInterfaceClient applet =n new StudentsSer
 verInterfaceClient();n applet.isStandalone =
 true;
 JFrame çerçeve = new JFrame();n çe
 rçeve.setTitle("StudentServerInterfaceClient");n çerçeve.ad
 d(applet, BorderLayout.CENTER);
 çerçeve.setSize(250, 150);n ap
 plet.init();
 çerçeve.setLocationRelativeTo(null);n çerçe
 ve.setVisible(true);
 çerçeve.setDefaultCloseOperation(3);n }
}

```

StudentServerInterfaceClient, belirli bir öğrenci için notu bulmak üzere sunucudaki findScore yöntemini çağırır. StudentServerInterfaceClient içindeki ana yöntem, sunucu stub'unu bulmaktan sorumlu olan initializeRMI yöntemidir (satırlar 55-68).

initializeRMI() metodu bağımsız uygulamaları appletlerden farklı şekilde ele alır. Ana bilgisayar adı, appletin indirildiği yerin adı olmalıdır. Bu, Applet'in getCodeBase().getHost() (satır 52) kullanılarak elde edilebilir. Bağımsız uygulamalar için, anabilgisayar adı açıkça belirtilmelidir.

lookup(String name) metodu (satır 62) belirtilen isimle uzak nesneyi döndürür. Bir uzak nesne bulunduğunda, bu nesne yerel bir nesne gibi kullanılabilir. Stub ve skeleton uzak yöntem çağrısını çalıştırmak için arka planda kullanılır.

5. Bu örneği çalıştırmak için aşağıdaki adımları izleyin.

5.1. Kitap dizininden bir DOS istemcisinde "**start rmiregistry**" yazarak RMI Kayıt Sunucusunu başlatın. Varsayılan olarak, rmiregistry tarafından 1099 port numarası kullanılır. Farklı bir port numarası kullanmak için, bir DOS istemcisinde "**start rmiregistry portnumber**" komutunu yazmanız yeterlidir.

5.2. Aşağıdaki komutu kullanarak C:\book dizininde RegisterWithRMIServer sunucusunu başlatın:

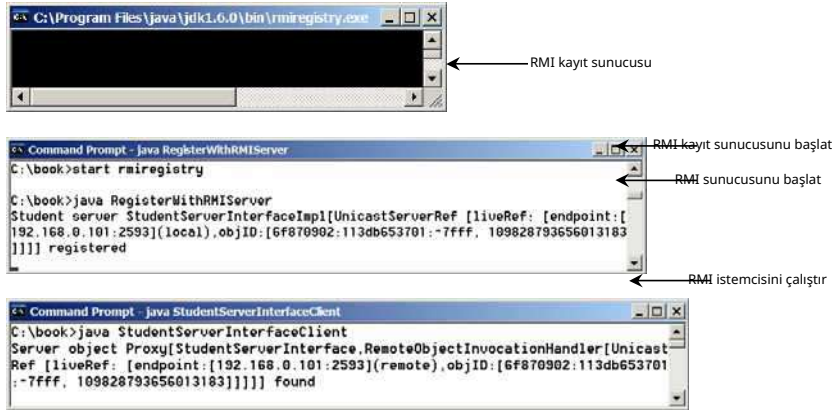
**C:\book>java RegisterWithRMISunucusu**

5.3. StudentServerInterfaceClient istemcisini bir uygulama olarak çalıştırın. Uygulamanın örnek çalışması Şekil 46.5(b)'de gösterilmiştir.

5.4. appletviewer'dan StudentServerInterfaceClient.html istemcisini çalıştırın. Bir örnek çalışma Şekil 46.5(a)'de gösterilmiştir.

NOT: RMI sunucusunu çalıştıracağınız dizinden rmiregistry'yi başlatmalı sınız, Şekil 46.6'da gösterildiği gibi. Aksi takdirde,

StudentServerInterfaceImpl\_Stub üzerinde ClassNotFoundException hatası alırsınız.



Şekil 46.6

*Bir RMI programı çalıştırmak için önce RMIRegistry'yi başlatın, ardından sunucu nesnesini kayıt defteri ile kaydedin. İstemci, kayıt defterinden onu bulur.*

NOT: Sunucu, kayıt defteri ve istemci üç farklı makinede olabilir. İstemciyi ve sunucuyu ayrı makinelerde çalıştırıyorsanız,

StudentServerInterface'i her iki makinede de bulundurmalısınız. İstemciyi bir applet olarak dağıtıyorsanız, tüm istemci dosyalarını kayıt defteri ana bilgisayarında bulundurmalısınız.

DİKKAT: Uzak nesne uygulama sınıfını değiştirirseniz, nesneyi RMI kayıt defterine yeniden yüklemek için sunucu sınıfını yeniden başlatmanız gerekir. Eski rmiregistry sürümlerinde, rmiregistry'yi yeniden başlatmanız gerekebilir.

#### 46.4 RMI ile Soket Düzeyinde Programlama

RMI, daha yüksek bir soyutlama seviyesinde program yapmanıza olanak tanır. Soket sunucusu, soket, bağlantı ve veri gönderme veya alma detaylarını gizler. Hatta arka planda çoklu iş parçacığı sunucusu uygular, oysa soket düzeyinde programlamada birden fazla istemciyi yönetmek için iş parçacıklarını açıkça uygulamanız gerekir.

RMI uygulamaları ölçeklenebilir ve bakımı kolaydır. RMI sunucusunu değiştirebilir veya başka bir makineye taşıyabilirsiniz, istemci programını yalnızca sunucuyu bulmak için URL'yi sıfırlamak dışında değiştirmeden. (URL'yi sıfırlamaktan kaçınmak için, istemciyi URL'yi bir komut satırı parametresi olarak geçecek şekilde değiştirebilirsiniz.) Soket düzeyinde programlamada, veri göndermek için bir istemci işlemi, veriyi okumak için bir sunucu işlemi gerektirir. İstemci ve sunucunun soket düzeyindeki uygulaması sıkı bir şekilde senkronizedir.

RMI istemcileri doğrudan sunucu yöntemini çağırabilirken, soket düzeyindeki programlama yalnızca değerleri geçmekle sınırlıdır. Soket düzeyindeki programlama çok ilkel bir yöntemdir. İstemci/sunucu uygulamaları geliştirmek için kullanılmaktan kaçınılmalıdır. Bir benzetme olarak, soket düzeyindeki programlama, montaj dilinde programlamaya benzerken, RMI programlama yüksek seviyeli bir dilde programlamaya benzer.

#### 46.5 RMI Kullanarak Üç Katmanlı Uygulamalar Geliştirme

Üç katmanlı uygulamalar son yıllarda önemli bir ilgi görmüştür, bu da büyük ölçüde geleneksel iki katmanlı istemci/sunucu veritabanı sistemlerini değiştirmek için daha ölçeklenebilir ve yük dengeleyici sistemlere olan talep nedeniyle olmuştur. Merkezi bir veritabanı sistemi yalnızca veri erişimini sağlamla kalmaz, aynı zamanda veriler üzerindeki iş kurallarını da işler. Bu nedenle, merkezi bir veritabanı genellikle ağır bir yük altındadır, çünkü kapsamlı veri manipülasyonu ve işleme gerektirir. Bazı durumlarda, veri işleme istemci tarafından gerçekleştirilir ve iş kuralları istemci tarafında saklanır. İstemci ve veritabanı arasında bir tampon olarak bir orta katman kullanmak tercih edilir. Orta katman, iş mantığını ve kurallarını uygulamak ve verileri işleyerek veritabanındaki yükü azaltmak için kullanılabilir.

Üç katmanlı bir mimari, sunucudaki iş yükünü azaltmaktan daha fazla sını yapar. Ayrıca birden fazla ağ sitesine erişim sağlar. Bu, farklı sunuculardaki birden fazla veritabanına erişmesi gereken Java applet'leri için özellikle yararlıdır, çünkü bir applet yalnızca indirildiği sunucu ile bağlantı kurabilir.

Gösterim için, §46.3.1'deki "Örnek: RMI Sunucusundan Öğrenci Notlarını Alma" örneğini, bir hash haritası yerine bir veritabanında saklanan notları bulmak için yeniden yazalım.

Ayrıca, sistem, bir öğrencinin notunu yayınlaması için üniversiteye izin vermediği bir öğrenciyi erişimden engelleyebilir. İstemci ile veritabanı arasında orta katman olarak hizmet vermek üzere bir RMI bileşeni geliştirilmiştir; veritabanına bir arama isteği gönderir, sonucu işler ve istemciye uygun bir değer döndürür.

Basitlik açısından, bu örnek, §46.3.1'den `StudentServerInterface` arayüzünü ve `StudentServerInterfaceClient` sınıfını herhangi bir değişiklik yapmadan yeniden kullanır. Tek yapmanız gereken, sunucu arayüzü için yeni bir uygulama sağlamak ve sunucuyu RMI ile kaydetmek için bir program oluşturmaktır. Programı tamamlamak için adımlar şunlardır:

1. Notları, ad, not ve izin içeren üç sütun bulunan `Score` adlı bir veritabanı tablosunda saklayın. İzin değeri 1 veya 0'dır, bu da öğrencinin üniversiteye notunu açıklama izni verip vermediğini gösterir. Aşağıda tabloyu oluşturmak ve üç kayıt eklemek için kullanılan ifade bulunmaktadır:

```
table Puanlar (isim varchar(20), not int, izin int);
insert Puanlar (isim, not, izin) values ('John', 90, 1);
```

```
insert Puanlar (isim, not, izin) values ('Michael', 100, 1);
insert Puanlar (isim, not, izin) values ('Michelle', 100, 0);
```

2. Listing 46.5'te Student3TierImpl adında yeni bir sunucu uygulaması oluşturun. Sunucu, Scores tablosundan bir kaydı alır, alınan bilgiyi işler ve sunucu istemciye geri gönderir.

#### Listeleme 46.5 Öğrenci3TierImpl.java

<Side Remark line 14: db'yi başlat>

<Side Remark line 31: sürücüyü yükle>

<Side Remark line 39: db'ye bağlan>

<Side Remark line 43: ifadeyi hazırla>

<Side Remark line 58: adı ayarla>

<Side Remark line 61: SQL'i çalıştır>

<Side Remark line 66: notu al>

```
import java.rmi.*;nim
port java.rmi.server.*;nimport j
ava.sql.*;

genel sınıf Öğrenci3TierImpl UnicastRemoteObjectn'i genişletir Öğrenci
ServerInterface'i uygular {
 // DB sorgulamak için hazırlanan ifadeyi kullanın
 private PreparedStatement pstmt;

 /** Student3TierImpl nesnesini oluşturur ve varsayılan
 * portta dışa aktarır.
 */
 public Student3TierImpl(), RemoteException {initializeDB();'yi
 oluşturur;

 /** Student3TierImpl nesnesini oluşturur ve belirtilen
 * portta dışa aktarır.
 * @param port Dışa aktarma için port
 public Student3TierImpl(int port) RemoteException'ı atar {n super(port);

 başlatmaDB();n }

 /** JDBC sürücüsünü yükleyin, bağlantıyı kurun ve
 * ifade oluşturun */
 korumalı geçersiz başlatmaDB() {n den
 eyin {
 // JDBC sürücüsünü yükle
 // Class.forName("Oracle.jdbc.driver.OracleDriver");n Class.forNa
 me("com.mysql.jdbc.Driver ");

 System.out.println("Sürücü kaydedildi");

 // Bağlantıyı kur
 /*Bağlantı bağlantısı = DriverManager.getConnectionn ("jd
 bc:oracle:thin:@drake.armstrong.edu:1521:orcl",n "scott", "tiger
 "); */
 Connection conn = DriverManager.getConnection
 "jdbc:mysql://localhost/javabook" , "scott", "tiger");
 System.out.println("Veritabanı bağlandı");

 // DB sorgulamak için bir hazırlıklı ifade oluşturun
 pstmt = conn.prepareStatement(
 "Scores tablosundan adı = ? olan tüm kayıtları seçin.");
```

```

 }
 catch (Exception ex) {n Syst
 em.out.println(ex);n }
 }

 /** Belirtilen ad için puanı döndür
 * Puan bulunamazsa -1 döndür.
 */
 public double findScore(Dize adı) RemoteException'ı atar {n double puan = -1;

 denemek {
 // Hazırlanmış ifadede belirtilen adı ayarla
 pstmt.setString(1, name);

 // Hazırlanmış ifadeyi çalıştır
 ResultSet rs = pstmt.executeQuery();

 // Puanı al
 if (rs.next()) {
 if (rs.getBoolean(3))
 score = rs.getDouble(2);
 }
 }
 catch (SQLException ex) {n Syst
 em.out.println(ex);n }

 System.out.println(name + "'ın puanı " + score + "'");
 return score;
}
}

```

Student3TierImpl, §46.3.1'deki StudentServerInterfaceImpl ile benzerdir, ancak Student3TierImpl sınıfı puanı bir JDBC veri kaynağından alır, bir hash map'ten değil.

Scores adındaki tablo, isim, puan ve izin olmak üzere üç sütundan oluşmaktadır. İzin, öğrencinin puanını göstermeye izin verip vermediğini belirtir. SQL, boolean türünü desteklemediğinden, izin 1 değerinin doğru ve 0 değerinin yanlış olduğu bir sayı olarak tanımlanmıştır.

initializeDB() metodu (satırlar 28-42) veritabanı ile bağlantılar kurar ve sorguyu işlemek için bir hazırlıklı ifade oluşturur.

findScore metodu (satırlar 47-68) hazırlıklı ifadede ismi ayarlar, ifadeyi çalıştırır, sonucu işler ve izni doğru olan bir öğrencinin notunu döndürür.

3. RegisterStudent3TierServer sınıfında (Liste 46.6) sunucu nesnesini kaydeden bir ana metod yazın. StudentServerInterfaceImpl, Liste 46.2'deki ile aynı isimdir, böylece §46.3.1'de oluşturulan StudentServerInterfaceClient'i kullanarak sunucu test edebilirsiniz.

#### Liste 46.6 RegisterStudent3TierServer.java

<Yan Not satır 7: localhost üzerindeki kayıt



<Yan Not satır 8: sunucu nesnesini kaydet

```
import java.rmi.registry.*;

class RegisterStudent3TierServer {
 public static void main(String[] args) {
 try {
 StudentServerInterface obj = new Student3TierImpl();
 Registry registry = LocateRegistry.getRegistry();
 registry.rebind("StudentServerInterfaceImpl", obj);
 System.out.println("Öğrenci sunucusu " + obj + " kaydedildi");
 } catch (Exception ex) {
 ex.printStackTrace();
 }
 }
}
```

4. Bu örneği çalıştırmak için aşağıdaki adımları izleyin.

4.1. RMI Kayıt Sunucusunu başlatmak için DOS istemcisinde "**start rmiregistry**" yazın kitap dizininden.

4.2. Aşağıdaki komutla C:\ook dizininde RegisterStudent3TierServer sunucusunu başlatın:

```
C:\ook>java RegisterStudent3TierServer
```

4.3. Client StudentServerInterfaceClient'i bir uygulama veya applet olarak çalıştırın. Örnek bir çalışma Şekil 46.6'da gösterilmiştir.

## 46.6 RMI Geri Çağrılar

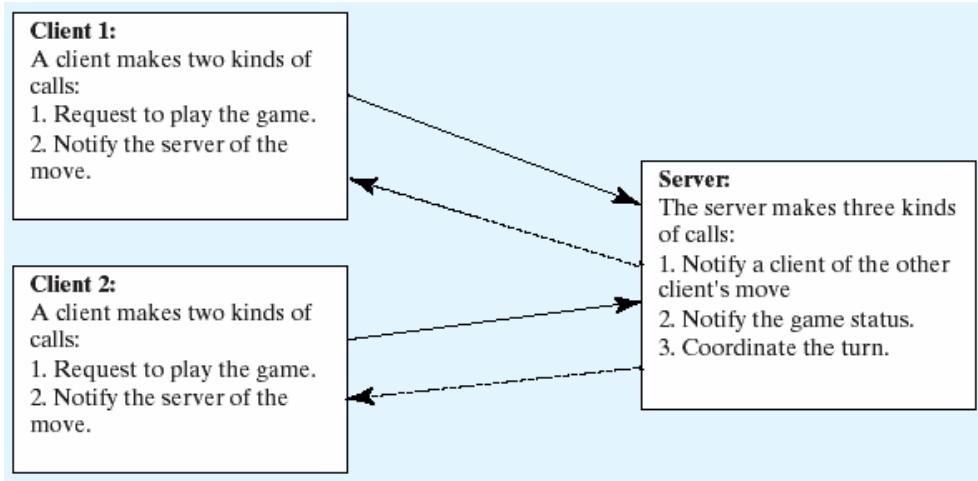
Geleneksel bir istemci/sunucu sisteminde, bir istemci bir isteği bir sunucuya gönderir ve sunucu isteği işler ve sonucu istemciye döner. Sunucu, istemci üzerindeki yöntemleri çağırır. RMI'nin önemli bir avantajı, geri çağrılarını desteklemesidir, bu da sunucunun istemci üzerindeki yöntemleri çağırmasını sağlar. RMI geri çağrı özelliği ile interaktif dağıtık uygulamalar geliştirebilirsiniz.

§30.9, "Vaka Çalışmaları: Dağıtık TicTacToe Oyunları" bölümünde, distribütor soket programlaması kullanarak dağıtık bir TicTacToe oyunu geliştirdiniz. Aşağıdaki örnek, RMI geri çağrı özelliğini kullanarak interaktif bir TicTacToe oyunu geliştirmeyi göstermektedir.

Bu bölümde şimdiye kadar gördüğünüz tüm örnekler, sınıflarla modellemesi kolay ve sade davranışlara sahiptir. TicTacToe oyununun davranışı biraz karmaşıktır. Oyunu modellemek için sınıfları oluşturmak üzere onu incelemeniz ve anlamanız ve süreci istemci ve sunucu arasında uygun şekilde dağıtmanız gerekir.

Açıkça, istemcinin kullanıcı etkileşimlerini yönetmekten sorumlu olması ve sunucunun istemci ile koordine olması gerekir. Özellikle, istemci sunucuya kaydolmalı ve sunucu yalnızca iki oyuncu alabilir. Bir istemci hamle yaptığında, sunucuya bildirmelidir; sunucu daha sonra diğer oyuncuya hamleyi bildirir. Sunucu oyunun durumunu belirlemelidir; yani, oyunun kazanılıp kazanılmadığını veya berabere kalıp kalmadığını ve oyuncuları bilgilendirmelidir. Sunucu ayrıca sırayı koordine etmelidir; yani, belirli bir zamanda hangi istemcinin sırada olduğunu.

Bir oyuncuyu bilgilendirmenin ideal yaklaşımı, istemcide uygun özellikleri ayarlay an veya bir oyuncuya mesaj gönderen bir yöntemi çağırmasıdır. Şekil 46.7, istemciler ve sunucu arasındaki ilişkiyi göstermektedir.



Şekil 46.7

Sunucu, istemcilerle olan etkinlikleri koordine eder.

Bir istemcinin yaptığı tüm çağrılar, TicTacToe adlı bir uzak arayüzde kapsüllen ebilir (Liste 46.7) ve sunucunun çağırdığı tüm çağrılar, Callback adlı başka bir arayüzde tanımlanabilir (Liste 46.8). Bu iki arayüz aşağıdaki gibi tanımlanmıştır:

#### Liste 46.7 TicTacToeInterface.java

<Yan Not satır 3: alt arayüz>  
<Yan Not satır 9: sunucu yöntemi>  
<Yan Not satır 12: sunucu yöntemi>

Java.rmi.\*'yi içe aktarın;

genel arayüz TicTacToeInterface Remote {n /\*\*'i genişletiyor

\* TicTacToe sunucusuna bağlanın ve token'ı döndürün.

\* Döndürülen token '' ise, istemci sunucuya bağlı değildir.

\*

\*/

public char connect(Callback client) throws RemoteException;

/\*\* Bir istemci, hamlesini sunucuya bildirmek için bu yöntemi çağırır \*/

public void myMove(int row, int column, char token)

RemoteException'ı atar;n}

#### Liste 46.8 Callback.java

<Side Remark line 3: alt arayüz>  
<Side Remark line 5: sunucu yöntemi>  
<Side Remark line 8: sunucu yöntemi>  
<Side Remark line 12: sunucu yöntemi>

Java.rmi.\*'yi içe aktarın;

```

public interface CallBack extends Remote {
 /** Sunucu, istemciye sıranın geldiğini bildirir */
 public void takeTurn(boolean turn) throws RemoteException;

 /** Sunucu, istemci tarafından görüntülenecek bir mesaj gönderir */
 public void notify(java.lang.String message)
 RemoteException'ı atar;

 /** Sunucu, bir istemciyi diğer oyuncunun hamlesi hakkında bilgilendirir */
 public void mark(int row, int column, char token)
 RemoteException'ı atar;
}

```

Bir istemcinin ne yapması gerekiyor? İstemci, oyuncuyla etkileşimde bulunur. Tüm hücrelerin başlangıçta boş olduğunu ve ilk oyuncunun X token'ını, ikinci oyuncunun ise O token'ını aldığını varsayın. Bir hücreyi işaretlemek için, oyuncu farenin imlecini hücreye yönlendirir ve tıklar. Eğer hücre boşsa, token (X veya O) gösterilir. Eğer hücre zaten doluysa, oyuncunun eylemi göz ardı edilir.

Önceki açıklamadan, bir hücrenin fare tıklama olaylarını yöneten ve token'ları gösteren bir GUI nesnesi olduğu açıktır. Böyle bir nesne için aday bir buton veya panel olabilir. Pencere butonlardan daha esnektir. Token (X veya O) bir panelde herhangi bir boyutta dişe çizilebilir, ancak yalnızca bir buton üzerinde etiket olarak gösterilebilir.

Cell sınıfını JPanel'in bir alt sınıfı olarak tanımlayın. Oyunu modellemek için  $3 \times 3$  zgarayı bir dizi olarak tanımlayabilirsiniz: `Cell[][] cell = new Cell[3][3]`. Bir hücrenin durumunu (işaretli olup olmadığını) nasıl bilirsiniz? Cell sınıfında boolean türünde `marked` adında bir özellik kullanabilirsiniz. Oyuncunun sırası olup olmadığını nasıl bilirsiniz? boolean türünde `myTurn` adında bir özellik kullanabilirsiniz. Bu özellik (başlangıçta false) sunucu tarafından bir geri çağırma ile ayarlanabilir.

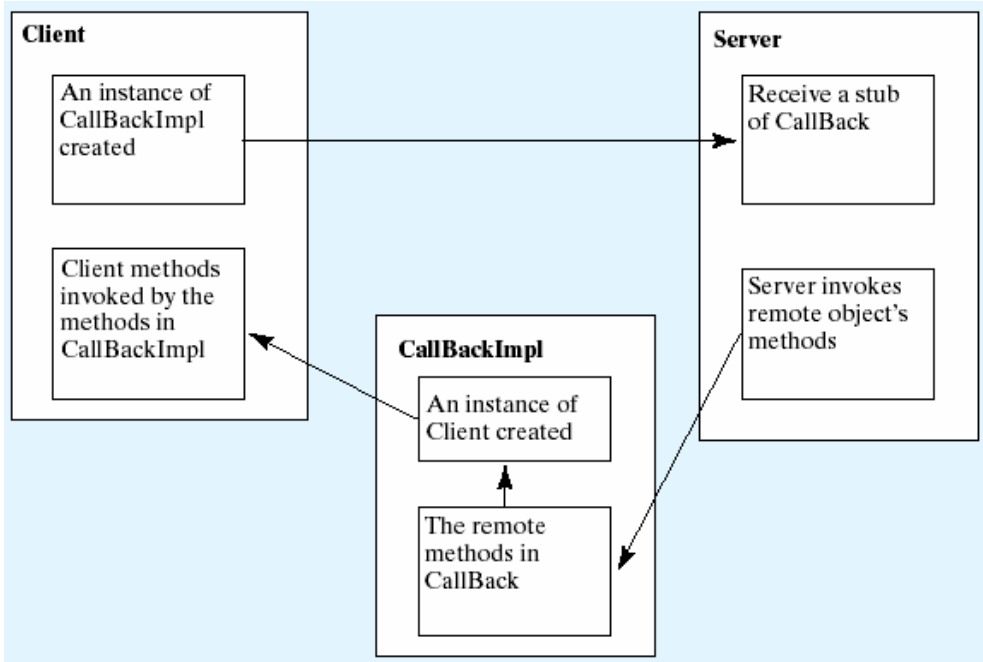
Cell sınıfı, boş bir hücreye tıklandığında token'ı çizmekten sorumludur, bu nedenle `MouseEvent`'i dinlemek ve X ve O token'ları için şekil boyamak için kod yazmanız gerekir. Hangi şeklin çizileceğini belirlemek için char türünde `marker` adında bir değişken tanıtır. Bu değişken, bir istemciye tüm hücreler tarafından paylaşıldığından, istemciye tanımlanması ve Cell sınıfının istemcinin iç sınıfı olarak tanımlanması tercih edilir, böylece bu değişken tüm hücreler tarafından erişilebilir olacaktır.

Şimdi dikkatimizi sunucu tarafına çevirelim. Sunucu ne yapmalı? Sunucu,

`TicTacToeInterface`'i uygulamalı ve istemcilere oyun durumunu bildirmelidir. Sunucu, hücrelerdeki hamleleri kaydetmeli ve her oyuncu hamle yaptığında durumu kontrol etmelidir. Durum bilgisi,  $3 \times 3$  karakterlik bir dizi içinde tutulabilir. Tahtanın dolu olup olmadığını kontrol etmek için `isFull()` adında bir yöntem ve belirli bir oyuncunun kazanıp kazanmadığını kontrol etmek için `isWon(token)` adında bir yöntem uygulayabilirsiniz.

Bir istemci sunucuya bağlandığında, sunucu istemciye hangi token'ı kullanacağını bildirir; yani, birinci istemci için X ve ikinci istemci için O. Bir istemci hamlesini sunucuya bildirdiğinde, sunucu oyun durumunu kontrol eder ve istemcilere bildirir.

Şimdi en kritik soru, sunucunun bir istemciyi nasıl bilgilendirdiğidir. Bir istemcinin, istemci tarafında bir sunucu stub'ı oluşturarak sunucu yöntemin i çağırdığını biliyorsunuz. Sunucu, istemciyi doğrudan çağırılmaz çünkü istemci uzak bir nesne olarak tanımlanmamıştır. Callback arayüzü, sunucunun istemciye geri çağırmasını kolaylaştırmak için oluşturulmuştur. Callback'in uygulanmasında, bir istemci örneği Callback'in yapısında bir parametre olarak geçirilir. İstemci, Callback'in bir örneğini oluşturur ve stub'ını sunucuya iletir, sunucuda tanımlanan connect() adlı bir uzak yöntem kullanarak. Sunucu daha sonra Callback örneği aracılığıyla istemcinin yöntemini çağırır. İstemci, Callback uygulaması ve sunucu arasındaki üçgen ilişki Şekil 46.8'de gösterilmektedir.



**Şekil 46.8**

*Sunucu, istemciden bir Callback stub'ı alır ve Callback arayüzünde tanımlanan uzak yöntemleri çağırır; bu yöntemler istemcide tanımlanan yöntemleri çağırabilir.*

İşte örneği tamamlamak için adımlar.

1. TicTacToeImpl.java (Liste 46.9) oluşturun ve TicTacToeInterface'i uygulayın. Sunucu RMI ile kaydetmek için programda bir ana yöntem ekleyin.

#### **Liste 46.9 TicTacToeImpl.java**

**<Yan Not satır 9: geri çağırma nesneleri>**

**<Yan Not satır 34: bağlanmayı uygula>**

**<Yan Not satır 58: myMove'u uygula>**

**<Yan Not satır 101: kazanıldı mı>**

**<Yan Not satır 124: dolu mu>**

**<Yan Not satır 137: nesneyi kaydet>**

Java.rmi.\*'yi içe aktarın;

```
java.rmi.server.*'ı içe aktarın;
java.rmi.registry.*'yi içe aktarın;
java.rmi.registry.*'yi içe aktarın;
```

**genel sınıf TicTacToeImpl UnicastRemoteObject'n'i genişletir TicTacToeInterface'i uygular {**

```
// İki oyuncu tanımlayın, oyuncuların geri çağırması için kullanılır
private Callback player1 = null;
özel Geri Arama oynatıcı2 = null;

// tahta oyuncuların hamlelerini kaydeder
private char[][] board = new char[3][3];

/** TicTacToeImpl nesnesini oluşturur ve varsayılan portta dışa aktarır.
*/
public TicTacToeImpl(), RemoteException'ı oluşturur {n super()
er();
}

/** TicTacToeImpl nesnesini oluşturur ve belirtilen
* portta dışa aktarır.
* @param port Dışa aktarma için port

public TicTacToeImpl(int port), RemoteException {n super(port);'u ata
r;
}

/**
* TicTacToe sunucusuna bağlanın ve token'ı döndürün.
* Döndürülen token '' ise, istemci sunucuya bağlı değildir.
*
*/
public char connect(Callback istemcisi) RemoteException'ı atar {n if (player1 =
= null) {
 // player1 (ilk oyuncu) kaydedildi
 player1 = client;
 player1.notify("İkinci oyuncunun katılmasını bekleyin");
 return 'X';
}
else if (player2 == null) {
 // player2 (ikinci oyuncu) kaydedildi
 player2 = client;
 player2.notify("İlk oyuncunun hamlesini bekleyin");
 player2.takeTurn(false);
 player1.notify("Sıra bende (X jetonu)");
 player1.takeTurn(true);
 dönüş 'O';n }

başka {
 // Zaten iki oyuncu var
 client.notify("Oyunda zaten iki oyuncu var");
 return ' ';
}
}

/** Bir istemci, hamlesini sunucuya bildirmek için bu yöntemi çağırır*/
public void myMove(int row, int column, char token)
RemoteException fırlatır {
 // Belirtilen hücreye token ayarla
```

```

board[row][column] = token;

// Diğer oyuncuya hamleyi bildir
if (token == 'X')
 player2.mark(satır, sütun, 'X');n else

player1.mark(row, column, 'O');

// Bu token ile oyuncunun kazanıp kazanmadığını kontrol et
if (isWon(token)) {
 if (token == 'X') {
 player1.notify("Kazandım!");
 player2.notify("Kaybettim!");
 player1.takeTurn(false);
 }
 başka {
 player2.notify("Kazandım!");
 player1.notify("Kaybettim!");
 player2.takeTurn(false);
 }
}
else if (isFull()) {
 player1.notify("Beraberlik!");
 player2.notify("Beraberlik!");
}
else if (belirteç == 'X') {
 player1.notify("İkinci oyuncunun hamlesini bekle");
 player1.takeTurn(false);
 player2.notify("Sıra bende, (O token)");
 player2.takeTurn(true);
}
else if (belirteç == 'O') {
 player2.notify("İlk oyuncunun hamlesini bekleyin");
 player2.takeTurn(false);
 player1.notify("Sıra bende, (X token)");
 player1.takeTurn(true);
}
}

/** Belirtilen token ile bir oyuncunun kazanıp kazanmadığını kontrol et */
public boolean isWon(char token) {
 for (int i = 0; i < 3; i++)n if ((pano[i][
0] == jeton) && (pano[i][1] == jeton)n && (pano[i][2] == jeton))

 true döndür;

 for (int j = 0; j < 3; j++)n if ((tahta[0]
[j] == jeton) && (tahta[1][j] == jeton)n && (tahta[2][j] == jeton))

 true döndür;

 if ((tahta[0][0] == jeton) && (tahta[1][1] == jeton)n && (tahta[2][2]
== jeton))
 true döndür;

 if ((tahta[0][2] == jeton) && (tahta[1][1] == jeton)n && (tahta[2][0]
== jeton))
 true döndür;
}

```

```

 false döndür;
 }

 /** Tahtanın dolu olup olmadığını kontrol et */
 public boolean isFull() {
 for (int i = 0; i < 3; i++)
 for (int j = 0; j < 3; j++)
 if (board[i][j] == '\u0000')
 false döndür;

 doğruyu döndür;
 }

 public static void main(String[] args) {
 try {
 TicTacToeInterface obj = new TicTacToeImpl();
 Kayıt defteri kayıt defteri = LocateRegistry.getRegistry();
 ebind("TicTacToeImpl", obj);
 System.out.println("Sunucu " + obj + " kaydedildi");
 } catch (Exception ex) {
 ex.printStackTrace();
 }
 }
}

```

2. CallbackImpl.java (Liste 46.10) dosyasını oluşturun ve Callback arayüzünü uygulayın.

#### Liste 46.10 CallbackImpl.java

*<Yan Açıklama satırı 15: uygulama> n<Yan Açıklama satırı 20: uygulama> n<Yan Açıklama satırı 25: uygulama>*

```

Java.rmi.*'yi içe aktarın;
java.rmi.server.*'ı içe aktarın;

genel sınıf CallbackImpl, UnicastRemoteObject'ni genişletir, Callback'ı uygular {
 // İstemci, sunucu tarafından geri çağırılacak
 private TicTacToeClientRMI thisClient;

 /** Yapıcı */
 public CallbackImpl(Object client) RemoteException'ı atar {
 thisClient = (TicTacToeClientRMI)client;
 }

 /** Sunucu, istemciyi sıranın geldiğini bildirir */
 public void takeTurn(boolean turn) throws RemoteException {
 thisClient.setMyTurn(turn);
 }

 /** Sunucu, istemci tarafından görüntülenecek bir mesaj gönderir */
 public void notify(String message) throws RemoteException {
 thisClient.setMessage(message);
 }

 /** Sunucu, bir istemciyi diğer oyuncunun hamlesi hakkında bilgilendirir */
}

```

```

public void mark(int satır, int sütun, karakter belirtici)n Remot
eException'ı atar {
thisClient.mark(satır, sütun, belirteç);n }
}

```

3. Bir applet TicTacToeClientRMI oluşturun (Liste 46.11) bir oyuncu ile etkileşimde bulunmak ve sunucu ile iletişim kurmak için. Bağımsız çalışmasını sağlayın.

#### Liste 46.11 TicTacToeClientRMI.java

*<Yan Not satır 20: sunucu nesnesi>*  
*<Yan Not satır 33: UI oluştur>*  
*<Yan Not satır 58: kayıt sunucusu>*  
*<Yan Not satır 62: sunucu nesnesi>*  
*<Yan Not satır 70: geri çağır>*  
*<Yan Not satır 115: dinleyici kaydet>*  
*<Yan Not satır 165: bağımsız>*

```

Java.rmi.*'yi içe aktarın;
java.auto.*'yu içe aktarın;
java.awt.event.*'i içe aktarın;
javax.swing.*'i içe aktarın;
import javax.swing.border.*;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;

public class TicTacToeClientRMI extends JApplet {
 // işaretçi, token türünü belirtmek için kullanılır
 private char marker;

 // myTurn, oyuncunun şimdi hareket edip edemeyeceğini gösterir
 private boolean myTurn = false;

 // Her hücre boş olabilir veya 'O' ya da 'X' ile işaretlenebilir
 private Cell[][] cell;

 // ticTacToe, oyuncularla koordinasyon sağlamak için oyun sunucusudur
 private TicTacToeInterface ticTacToe;

 // Hücreler ve panel için kenarlık
 private Border lineBorder =
 BorderFactory.createLineBorder(Color.yellow, 1);

 özel JLabel jlblStatus = new JLabel("jLabel1");n özel JLabel jlblIde
ntification = new JLabel();

 boolean isBağımsız = false;

 /** Uygulamayı başlat */
 public void init() {
 JPanel jPanel1 = new JPanel();n jPanel
1.setBorder(lineBorder);n jPanel1.setL
ayout(new GridLayout(3, 3, 1, 1));

 add(jlblStatus, BorderLayout.SOUTH);n add(jP
anel1, BorderLayout.CENTER);n add(jlblIde
ntification, BorderLayout.NORTH);
 }
}

```



```

// Hücreleri oluştur ve hücreleri panelde yerleştir
cell = new Cell[3][3];
for (int i = 0; i < 3; i++)n for (int j =
0; j < 3; j++)n jPanel1.add(cell[i][j])
= new Cell(i, j));

denemek {
RMI'yi başlat();n }

catch (Exception ex) {n ex.p
rintStackTrace();n }

}

/** RMI'yi başlat */
korumalı boolean startupizeRMI() İstisna oluşturur {n String host
= "";
if (!isStandalone) ana bilgisayar = getCodeBase().getHost();

denemek {
Registry registry = LocateRegistry.getRegistry(host);
ticTacToe = (TicTacToeInterface) registry.lookup("TicTacToeImpl");
System.out.println("Sunucu nesnesi " + ticTacToe + " bulundu");
}
catch (Exception ex) {n Syst
em.out.println(ex);n }

// Sunucu tarafından istemciyi kontrol etmek için kullanılacak geri çağırma oluştur
CallBackImpl callBackControl = new CallBackImpl(this);

eğer (
(marker = ticTacToe.connect((CallBack)callBackControl)) != ' '){

System.out.println("bağlandı " + marker + " oyuncusu olarak.");
jlblIdentification.setText("Siz " + marker + " oyuncususunuz.");
return true;
}
başka {
System.out.println("zaten iki oyuncu bağlandı olarak");
return false;
}
}

/** myTurn değişkenini true veya false olarak ayarla */
public void setMyTurn(boolean myTurn) {
this.myTurn = myTurn;
}

/** durum etiketinde mesaj ayarla */
public void setMessage(String message) {
jlblStatus.setText(message);
}

/** belirtilen hücreyi token ile işaretle */
public void mark(int row, int column, char token) {
cell[row][column].setToken(token);
}

```

```

 }

 /** TicTacToe tahtasında bir hücre modellemek için iç sınıf Cell */
 private class Cell extends JPanel {
 // işaretli, hücrenin kullanılıp kullanılmadığını gösterir
 private boolean marked = false;

 // satır ve sütun, hücrenin tahtada nerede görüldüğünü gösterir
 int row, column;

 // Hücre için token
 private char token;

 /** bir hücre oluştur */
 public Cell(final int row, final int column) {
 this.row = row;
 this.column = sütun; n add
 MouseListener(new MouseAdapter() { n public
 void mouseClicked(MouseEvent e) { n if (myTurn &
 & !marked) {
 // Hücreyi işaretle
 setToken(marker);

 // Hareketi sunucuya bildir
 try {
 ticTacToe.myMove(satır, sütun, işaretleyici); n }

 catch (RemoteException ex) { n Syst
 em.out.println(ex); n }

 }
 }
 }
 };

 setBorder(lineBorder);
}

/** Bir hücreye token ayarla (bir hücreyi işaretle) */
public void setToken(char c) {
 belirteç = c; n iş
 aretli = doğru; n yeni
 den boyama();
}

/** Token için bir şekil çizmek üzere hücreyi boyama */
protected void paintComponent(Graphics g) {
 super.paintComponent(g);

 // Kenarı çiz
 g.drawRect(0, 0, getSize().width, getSize().height);

 if (belirteç == 'X') {
 g.drawLine(10, 10, getSize().width - 10, n getSize().h
 eight - 10);
 g.drawLine(getSize().width - 10, 10, 10, n getSize().h
 eight - 10);
 }
 else if (token == 'O') { n g.draw
 Oval(10, 10, getSize().width - 20,

```

```

 getSize().height - 20);n }
 }
}

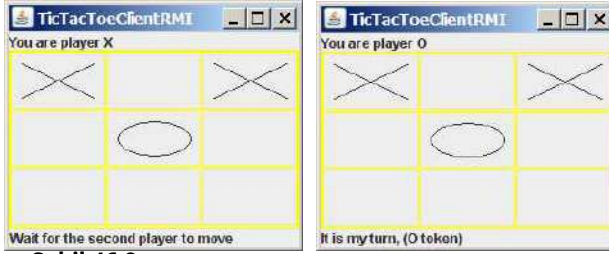
/** Ana yöntem */
public static void main(String[] args) {n TicTacToeC
lientRMI applet = new TicTacToeClientRMI();n applet.isStandalone =
true;
 applet.init();
 applet.start();
 JFrame çerçevesi = new JFrame();n ç
erçeve.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);n çerçev
e.setTitle("TicTacToeClientRMI");
 frame.add(applet, BorderLayout.CENTER);nframe
.setSize(400, 320);
 çerçeve.setVisible(true);n }
}

```

4. Bu örneği çalıştırmak için aşağıdaki adımları izleyin.
- 4.1. RMI Kayıt Sunucusunu başlatmak için DOS istemcisinde "**start rmiregistry**" yazın kitap dizininden.
- 4.2. Sunucu TicTacToeImpl'i başlatmak için aşağıdaki komutu C:\ook dizininde çalıştırın:

**C:\kitap>java TicTacToeImpl**

TicTacToeClientRMI istemcisini bir uygulama veya applet olarak çalıştırın. Örnek bir çalışma Şekil 46.9'da gösterilmiştir.



**Şekil 46.9**

*İki oyuncu RMI sunucusu üzerinden birbirleriyle oynar.*

TicTacToeInterface, iki uzak yöntem tanımlar: connect(CallBack client) ve myMove(int row, int column, char token). connectmetodu iki rol oynar: biri, sunucuya bir CallBack stub'u iletmek, diğeri ise sunucunun oyuncuya bir token atamasını sağlamaktır. myMove metodu, oyuncunun belirli bir hamle yaptığını sunucuya bildirir.

CallBack arayüzü, üç uzak yöntem tanımlar: takeTurn(boolean turn), notify(String message) ve mark(int row, int column, char token). takeTurn metodu, istemcinin myTurn özelliğini true veya false olarak ayarlar. notify metodu, istemcinin durum etiketinde bir mesaj görüntüler. mark metodu, belirtilen konumdaki token ile istemcinin hücrelerini işaretler.

TicTacToeImpl, istemcilerle koordinasyon sağlamak ve oyunu yönetmek için bir sunucu uygulamasıdır. player1 ve player2 değişkenleri, her biri bir istemciye karşılık gelen CallBack örnekleridir; bu örnekler, istemci connect metodunu çağırıldığında istemciden geçerli board değişkeni, iki oyuncunun hamlelerini kaydeder.

Bu bilgi, oyunun durumunu belirlemek için gereklidir. Bir istemci connect yöntemini çağırıldığında, sunucu birinci oyuncuya X ve ikinci oyuncuya O atar ve yalnızca iki oyuncuya izin verir. Programı, izleyici olarak ek istemcileri kabul edecek şekilde değiştirebilirsiniz. Daha fazla ayrıntı için Egzersiz 46.7'ye bakın.

İki oyuncu oyunda olduğunda, sunucu onların sıralarını koordine eder. Bir istemci myMove yöntemini çağırıldığında, sunucu hamleyi kaydeder ve diğer oyuncuyu diğer oyuncunun hücrelerini işaretleyerek bilgilendirir. Ardından, oyuncunun kazanıp kazanmadığını veya tahtanın dolup dolmadığını kontrol eder. Eğer her iki koşul da geçerli değilse ve bu nedenle oyun devam ediyorsa, sunucu diğer oyuncuya sırayı verir.

CallBackImpl, CallBack arayüzünü uygular. Yapıcısı aracılığıyla

TicTacToeClientRMI'nin bir örneğini oluşturur.

CallBackImpl, sunucu isteğini istemciye ileterek istemcinin yöntemlerini çağırır. Sunucu takeTurn yöntemini çağırıldığında, CallBackImpl istemcinin setMyTurn() yöntemini çağırarak istemcideki myTurn özelliğini ayarlar. Sunucu notify() yöntemini çağırıldığında, CallBackImpl istemcinin setMessage() yöntemini çağırarak istemcinin durum etiketinde mesajı ayarlar. Sunucu mark yöntemini çağırıldığında, CallBackImpl istemcinin mark yöntemini çağırarak belirtile n hücreyi işaretler.

TicTacToeClientRMI, bağımsız bir uygulama veya bir applet olarak çalışabilir. init ializeRMI metodu, bağımsız bir uygulama veya applet olarak çalışmak için URL oluşturmak, TicTacToeImpl sunucu stub'ını bulmak, CallBack sunucu nesnesini oluşturmak ve istemciyi sunucuya bağlamakla sorumludur.

İlginç bir şekilde, istemci için TicTacToeImpl stub'ını bulmak, sunucu için CallBack stub'ını elde etmekten farklıdır. TicTacToeImpl stub'ı, RMI kayıt defteri üzerinden lookup() metodunu çağırarak elde edilir ve CallBack stub'ı, TicTacToeImpl stub'ındaki connect metodu aracılığıyla sunucuya iletilir. İlk stub'ı lookup metodu ile elde etmek yaygın bir uygulamadır, ancak sonraki stub'ları uzaktan çağırarak aracılığıyla parametre olarak iletmek gelenektir.

myTurn ve marker değişkenleri TicTacToeClientRMI'de tanımlandığı için, Cell sınıfı, istemcideki tüm hücrelerin bunlara erişebilmesi için TicTacToeClientRMI içinde bir iç sınıf olarak tanımlanmıştır. Egzersiz 46.8, Cell sınıfını iç sınıf olarak uygulayan alternatif yaklaşımlar önermektedir.

#### Anahtar Terimler

- geri çağırma
- RMI kayıt sunucusu
- iskelet
- saplama

## Bölüm Özeti

1. RMI, dağıtılmış nesneleri kullanarak dağıtılmış uygulamalar oluşturmak için yüksek seviyeli bir Java API'sidir.
2. RMI'nin ana fikri, nesneler arasındaki iletişimi kolaylaştırmak için stub'lar ve iskeletler kullanmasıdır. Stub ve iskelet otomatik olarak üretilir, bu da programcıları zorlayıcı soket düzeyinde ağ programlamasından kurtarır.
3. Bir nesnenin uzaktan kullanılabilmesi için, java.rmi.Remote arayüzünü geniş leten bir arayüzde tanımlanması gerekir.
4. Bir RMI uygulamasında, başlangıçtaki uzak nesne, sunucu tarafındaki RMI kaydına kaydedilmeli ve istemci tarafındaki kayıt üzerinden lookup yöntemi ile elde edilmelidir. Diğer uzak nesnelerin stub'larının sonraki kullanımları, uzak yöntem çağrılarını aracılığıyla parametre olarak geçirilebilir.
5. RMI, ölçeklenebilir ve yük dengelemesi yapılmış çok katmanlı dağıtık uygulamalar geliştirmek için özellikle yararlıdır.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

## Gözden Geçirme Soruları

### Bölmeler 46.2-46.3

- 46.1 Uzak bir nesne için bir arayüzü nasıl tanımlarsınız?
- 46.2 Stub ve skeleton'ın rollerini tanımlayın.
- 46.3 java.rmi.Remote nedir? Bir sunucu sınıfını nasıl tanımlarsınız?
- 46.4 RMI kaydı ne için kullanılır? Bir RMI kaydı nasıl oluşturulur?
- 46.5 Bir RMI Kaydı başlatmak için komut nedir?
- 46.6 Uzak bir nesneyi RMI kayıt sunucusuna nasıl kaydedersiniz?
- 46.7 Özel bir RMI sunucusunu başlatmak için komut nedir?
- 46.8 Bir istemci, RMI kayıt sunucusu aracılığıyla uzak nesne stub'unu nasıl bulur?

46.9 Bir kayıt sunucusunu nasıl elde edersiniz? Uzak bir nesneyi nasıl kaydedersiniz? Uzak nesneyi nasıl bulursunuz?

### Bölmeler 46.4-46.6

- 46.10 RMI'nin soket düzeyindeki programlamaya göre avantajları nelerdir?
- 46.11 RMI'de parametrelerin nasıl geçirildiğini tanımlayın.
- 46.12 TicTacToeInterface içindeki connect metodu şu şekilde tanımlanmışsa sorun nedir?

genel boolean bağlantısı (Geri Arama istemcisi, t grafiği belirteci) nt  
throwst RemoteException;

veya olarak

genel boolean bağlantısı (Geri Arama istemcisi, Karakter belirteci) nth  
throwst RemoteException;

46.13 Geri çağırma nedir? Geri çağırma RMI'de nasıl çalışır?

## Programlama Alıştırmaları

### Bölüm 46.3

46.1 \*

(*Müşteri sayısını sınırlayın*) §46.3.1'deki örneği "Örnek: RMI Sunucusundan Öğrenci Notlarını Alma," eşzamanlı müşteri sayısını on ile sınırlamak için değiştirin.

46.2\*

(*Kredi hesapla*) RMI kullanarak Alıştırma 30.1'i yeniden yazın. Aylık ödeme ve toplam ödeme hesaplamak için bir uzak arayüz tanımlamanız gerekiyor.

46.3\*\*

(*Web ziyaret sayısı*) RMI kullanarak Alıştırma 30.4'ü yeniden yazın. Sayıyı elde etmek ve artırmak için bir uzak arayüz tanımlamanız gerekiyor.

46.4\*\*

(*Adresleri görüntüle ve ekle*) RMI kullanarak Alıştırma 34.6'yı yeniden yazın. Adres eklemek ve adres bilgilerini almak için uzak bir arayüz tanımlamanız gerekiyor.

### Bölüm 46.5

46.5\*\*

(*Veritabanı tablosundaki adres*) Alıştırma 46.4'ü yeniden yazın. Adresin bir tabloda saklandığını varsayın.

46.6\*\*

(*Üç katmanlı uygulama*) Egzersizi değiştirmek için üç katmanlı yaklaşımı kullanın 46.4, aşağıdaki gibi:

- Öğrenci bilgilerini manipüle etmek için bir applet istemcisi oluşturun, Şekil 30.23(a)'de gösterildiği gibi.
- Öğrenci bilgilerini almak, eklemek ve güncellemek için yöntemler içeren bir uzak nesne arayüzü oluşturun ve arayüz için bir nesne uygulaması geliştirin.

### Bölüm 46.6

46.7\*\*

(*Sohbet*) RMI kullanarak 30.13 numaralı alıştırma'yı yeniden yazın. Bir mesaj göndermek ve almak için uzak arayüzü tanımlamanız gerekiyor.

46.8\*\*

(*TicTacToe'yu Geliştir*) TicTacToe örneğini §46.6, "RMI

Geri Çağrılar," aşağıdaki gibi değiştirin:

- Bir istemcinin oyunu izlemek için sunucuya gözlemci olarak bağlanmasına izin verin.
- Cell sınıfını iç sınıf olmayan bir sınıf olarak yeniden yazın.

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 47

### 2-4 Ağaçları ve B-Ağaçları

#### Hedefler

- 2-4 ağacının ne olduğunu bilmek (§47.1).
- Tree arayüzünü uygulayan Tree24 sınıfını tasarlamak (§47.2).
- 2-4 ağacında bir öge aramak (§47.3).
- Bir 2-4 ağacına bir eleman eklemek ve bir düğümü nasıl böleceğini bilmek (§47.4) .
- Bir 2-4 ağacından bir elemanı silmek ve aktarma ve birleşim işlemlerini nasıl gerçekleştireceğini bilmek (§47.5).
- Bir 2-4 ağacındaki elemanları gezmek (§47.6).
- Tree24 sınıfını uygulamak ve test etmek (§§47.7-47.8).
- 2-4 ağacının karmaşıklığını analiz etmek (§47.9).
- B-ağaçlarını büyük miktarda veriyi dizinlemek için kullanmak (§47.10) .

#### 47.1 Giriş

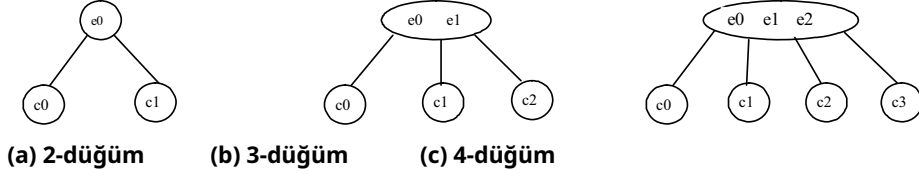
<margin note: tamamen dengeli ağaç>

<margin note: 2-düğüm>

<margin note: 3-düğüm>

<margin note: 4-düğüm>

Bir 2-4 ağacı, ayrıca 2-3-4 ağacı olarak da bilinir, tamamen dengeli bir arama ağacıdır ve tüm yaprak düğümleri aynı seviyede görünür. Bir 2-4 ağacında, bir düğüm bir, iki veya üç eleman içerebilir. Bir iç 2-düğüm bir eleman ve iki çocuk içerir. Bir iç 3-düğüm iki eleman ve üç çocuk içerir. Bir iç 4-düğüm üç eleman ve dört çocuk içerir, Şekil 47.1'de gösterildiği gibi.



Şekil 47.1

Bir 2-4 ağacının iç düğümü iki, üç veya dört çocuğa sahiptir.

<kenar notu: sıralandı>

Her çocuk, muhtemelen boş olan bir alt 2-4 ağacıdır. Kök düğümün üstü yoktur ve yaprak düğümlerinin çocukları yoktur. Ağaçtaki elemanlar farklıdır. Bir düğümdeki elemanlar, şu şekilde sıralanmıştır

$$E(c_0) < e_0 < E(c_1) < e_1 < E(c_2) < e_2 < E(c_3)$$

<margin note:  $E(c_k)$ >

<margin note: sol alt ağaç>

<margin note: sağ alt ağaç>

burada  $E(c_k)$ ,

$c_k$ , Şekil 47.2,

bir 2-4 ağacının örneğini göstermektedir. Sol alt ağaç olarak adlandırılır

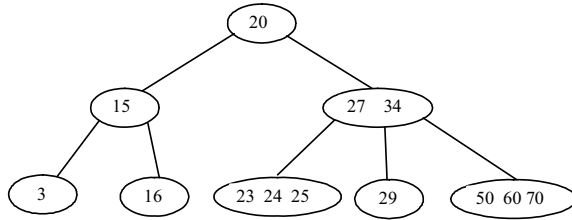
$e_k$  ve

tkctt1

olarak adlandırılır

sağ alt ağaç

$e_k$ .



Şekil 47.2

Bir 2-4 ağacı tam bir tamamlanmış arama ağacıdır.

<kenar notu: ikili vs. 2-4>

Bir ikili ağaçta, her düğüm bir öge içerir. Bir 2-4 ağacı, bir 2-4 ağacı düğümünün iki veya üç öge içerebileceğinden, karşılık gelen bir ikili arama ağacından daha kısa olma eğili mindedir.

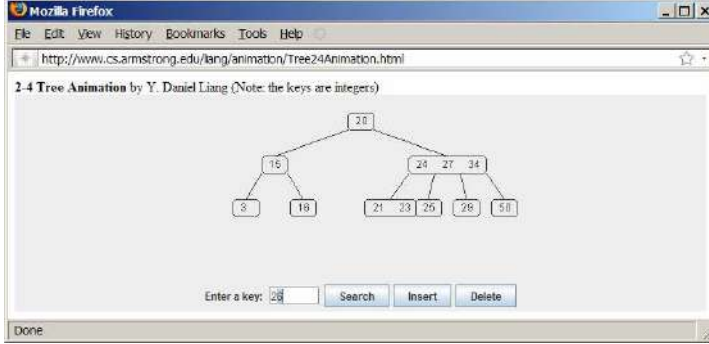
Eğitim NOTU



<yan not: 2-4 ağacı animasyonu>

Çalıştırın

[www.cs.armstrong.edu/liang/animation/Tree24Animation.html](http://www.cs.armstrong.edu/liang/animation/Tree24Animation.html)  
dosyanın nasıl çalıştığını görmek için, Şekil 47.3'te gösterildiği gibi.



Şekil 47.3

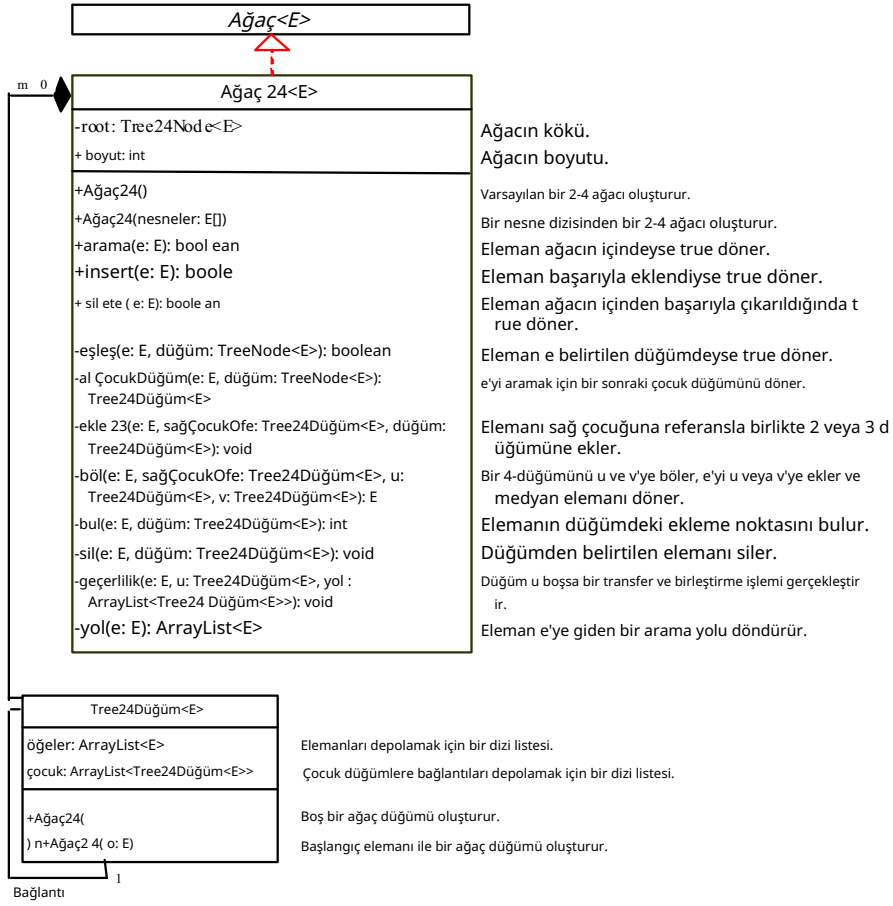
*Animasyon aracı, 2-4 ağacında*

*elemanları görsel olarak eklemenizi, silmenizi ve aramanızı sağlar.*

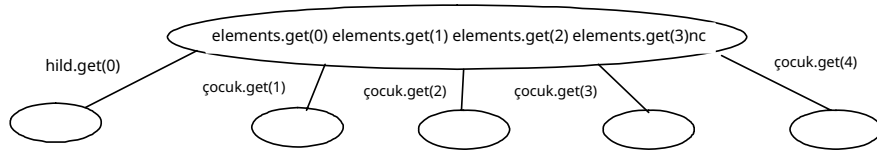
\*\*\*Son NOT

#### 47.2 2-4 Ağaçları için Sınıfların Tasarımı

Tree24 sınıfı, Şekil 47.4'te gösterildiği gibi Tree arayüzünü uygulayarak tasarlanabilir. Tree arayüzü, Listeleme 27.3'te tanımlanmıştır Tree.java. Tree24Node sınıfı ağaç düğümlerini tanımlar. Düğümdeki elemanlar, elements adında bir listede saklanır ve çocuk düğümlere olan bağlantılar, child adında bir listede saklanır, Şekil 47.5'te gösterildiği gibi.



**Şekil 47.4**  
Tree24 sınıfı, Tree'yi uygular.



**Şekil 47.5**  
Bir 2-4 ağaç düğümü, elemanları ve çocuk düğümlerine bağlantıları  
arayüz listelerinde saklar.

### 47.3 Bir Elemanı Aramak

Bir 2-4 ağaçta bir elemanı aramak, bir ikili ağaçta bir elemanı ararkenki gibidir. Fark, bir düğüm içinde bir elemanı aramanız gerektiğidir; ayrıca yol boyunca elemanları aramanız gerekir. Bir 2-4 ağaçta bir elemanı aramak için, kökten başlayarak aşağıya scan edersiniz.

Eğer bir eleman düğümde yoksa, uygun bir alt ağaçta ilerleyin.

Bir eşleşme bulunana kadar veya boş bir

alt ağa ulaşana kadar işlemi tekrarlayın. Algoritma, Liste 47.1'de açıklanmıştır.

#### Liste 47.1 Bir 2-4 Ağaçta Bir Elemanı Arama

<margin note line 2: kökten başla>

<margin note line 6: bulundu>

<margin note line 9: bir alt ağaçta ara>

<margin note line 13: bulunamadı>

```
boolean search(E e) {
 current = root; // Kökten başla

 while (current != null) {
 if (match(e, current)) { // Eleman düğümde mevcut
 return true; // Eleman bulundu
 }
 başka {
 current = getChildNode(e, current); // Bir alt ağaçta ara
 }
 }

 false döndür; // Eleman ağaçta yok
}
```

`match(e, current)` metodu, `e` elemanının mevcut nodda olup olmadığını kontrol eder. `getChildNode(e, current)` metodu, elemanın daha fazla arama için alt ağacın kökünü döndürür. Başlangıçta, `current` köke (2. satır) işaret etsin. Mevcut nodda elemanı aramaya devam edin `current` `null` (4. satır) olana kadar veya eleman mevcut nodda bir element ile eşleşene kadar.

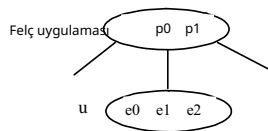
#### 47.4 2-4 Ağacına Bir Eleman Ekleme

<kenar boşluğu notu: taşma>  $n < ken$

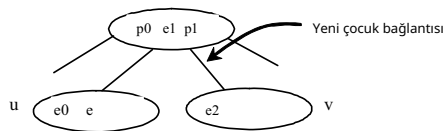
ar boşluğu notu: bölünmüş>

Bir elemanı  $e$  2-4 ağacına eklemek için, elemanın ekleneceği bir yaprak nodu bulun. Eğer yaprak nodu 2-nod veya 3-nod ise, elemanı nodun içine basitçe ekleyin. Eğer nod 4-nod ise, yeni bir eleman eklemek *overflow* yaratır. Taşmayı çözmek için, şu şekilde bir *split* işlemi gerçekleştirin:

- Let  $u$ , elemanın ekleneceği yaprak 4-nodu olsun ve  $parentOf(u)$ ,  $u$ 'nun ebeveyni olsun, Şekil 47.6(a)'de gösterildiği gibi.
- Yeni bir  $n$  adlı nod oluşturun;  $e_{n-1}$  ve  $v$ 'ye taşı.
- Eğer  $e < e_{n-1}$  ise  $u$ 'ya ekleyin; aksi takdirde  $e$ 'yi  $v$ 'ye ekleyin. Varsayalım ki  $e_0 < e_{n-1}$ ,  $e$   $u$ 'ya eklenir, Şekil 47.6(b)'de gösterildiği gibi.
- Ekleyin  $e_{n-1}$  ve sağ çocuğu (yani,  $v$ ) üst düğüme ekleyin, Şekil 47.6(b)'de gösterildiği gibi.



(a) Eklemeyen önce  $e$

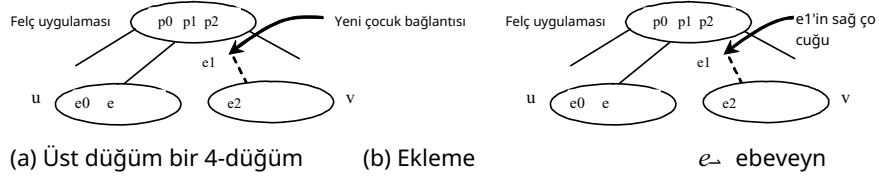


(b)  $e$  ekledikten sonra

### Şekil 47.6

Bölme işlemi yeni bir düğüm oluşturur ve medyan elementini üst düğümüne ekler.

Üst düğüm Şekil 47.6'da bir 3-düğümüdür. Yani,  $e$  üst düğümüne eklemek için yer vardır. Eğer 4-düğüm olursa, Şekil 47.7'de görüldüğü gibi ne olur? Bu, üst düğümün bölünmesini gerektirir. Süreç, bir yaprak 4-düğümün bölmeyle aynıdır, tek fark, elementi sağ çocuğuyla birlikte eklemeniz gerektirir.

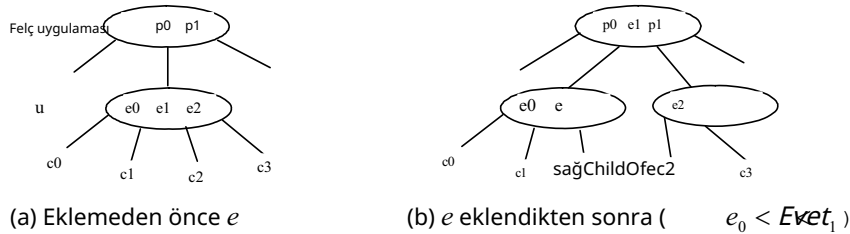


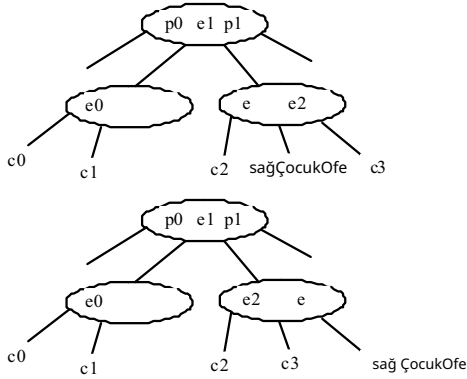
### Şekil 47.7

Ekleme işlemi, ana düğüm bir 4-düğümse devam eder.

Algoritma şu şekilde değiştirilebilir:

- Let  $u$  4 düğümlü (*leaf veya nonleaf*) olsun ve elemanın eklenmesi için  $u$ 'nun ebeveyni *parentOfu* olsun, Şekil 47.8(a)'de gösterildiği gibi.
- Yeni bir  $v$  adlı düğüm oluşturun,  $e_{\infty}$  ve çocukları  $c_{\infty}$  ve  $c_{\omega}$  den vs.
- Eğer  $e < e_{\infty}$  ile birlikte sağ çocuk bağlantısını ekleyin; aksi takdirde  $v$  ile birlikte sağ çocuk bağlantısını ekleyin, Şekil 47.6(b), (c), (d)'deki durumlar için gösterildiği gibi.  
 $e_0 < e_{\infty_1}$ ,  $e_1 < e_{\infty_2}$ ,  
 $v \wedge e_2 < e$ , sırasıyla.
- Ekleyin  $e_{\infty}$  sağ çocuğu ile birlikte (yani,  $v$ ) üst düğüme, özinelemeli olarak.





(c)  $e$  ekledikten sonra (  $e_1 < E_{vet_2}$  ) (d)  $e$  ekledikten sonra (  $e_2 < e$  )

#### Şekil 47.8

Bir iç düğüm taşıma sorununu çözmek için bölünebilir.

Liste 47.2, bir öge eklemek için bir algoritma verir.

#### Liste 47.2 Bir 2-4 Ağacına Bir Ögenin Eklenmesi

<margin note line 3: yeni bir düğüm oluştur>

<margin note line 5: e'yi ara>

<margin note line 6: e ekle>

<margin note line 9: bir öge eklendi>

<margin note line 10: öge eklendi>

<margin note line 13: bir düğüme ekle>

<margin note line 15: bir 2- veya 3-düğüm>

<margin note line 20: 4-düğümü ayır>

<margin note line 23: yeni kök>

<margin note line 29: medyanı ebeveyne ekle>

```
public boolean insert(E e) { nift (ro
 ot == t nullt)
 root = new Tree24Node<E>(e); // Eleman için yeni bir kök oluştur
 else {
 e'yi eklemek için yaprakDüğüm bul
 insert(e, null, leafNode); // e'nin sağ çocuğu null
 }

 size++; // Boyutu artır
 return true; // Eleman eklendi
}

özel void insert(E e, Tree24node<E> rightchildofe, n Tree24node<
 E> y) {
 if (u bir 2- veya 3-düğümse) { // u bir 2- veya 3-düğüm
 insert23(e, rightChildOfe, u); // e'yi u düğümüne ekle
 }
 else { // 4-düğüm u'yu ayır
 Tree24Node<E> v = new Tree24Node<E>(); // Yeni bir düğüm oluştur
 E median = split(e, rightChildOfe, u, v); // u'yu böl

 if (u == root) { // u kök
 root = new Tree24Node<E>(median); // Yeni kök
```

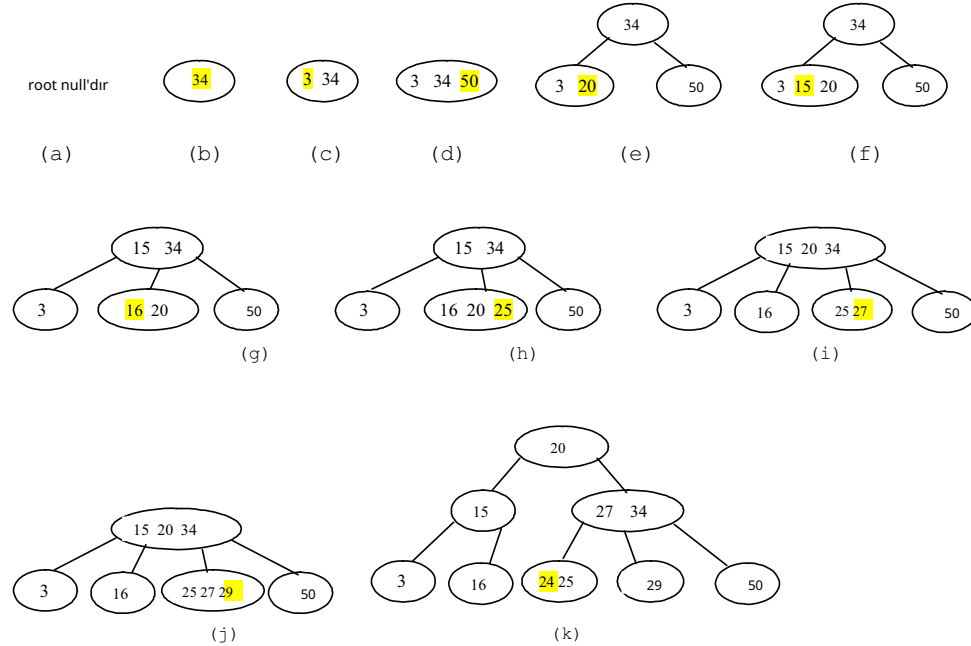
```

 root.child.add(u); // u, median'ın sol çocuğu
 root.child.add(v); // v, median'ın sağ çocuğu
 }
 başka {
 u'nun ebeveynini al, parentOfu;
 insert(median, v, parentOfu); // Median'ı ebeveyne ekle
 }
}
N}

```

insert(E e, Tree24Node<E> rightChildOfe, Tree24Node<E> u) metodu, e ögesini ve sağ çocuğunu u düğümüne ekler. e'yi bir yaprak düğümüne eklerken, e'nin sağ çocuğu null'dır (satır 6). Düğüm 2- veya 3-düğümse, ögeyi düğümüne basitçe ekleyin (satırlar 15-17). Düğüm 4-düğümse, düğümü bölmek için split metodunu çağırın (satır 20). Split metodu, median ögesini döndürür. Median ögesini ebeveyn düğümüne eklemek için insert metodunu yineleyerek çağırın (satır 29). Şekil 47.9, 34, 3, 50, 20,

15, 16, 25, 27, 29 ve 24'ü 2-4 ağacına ekleme adımlarını göstermektedir.



**Şekil 47.9**

*Boş bir ağaca 34, 3, 50, 20, 15, 16, 25, 27, 29 ve 24 eklendikten sonra ağaç değişir.*

#### 47.5 2-4 Ağaçtan Bir Elemanı Silme

Bir 2-4 ağaçtan bir elemanı silmek için, önce elemanı içeren düğümü bulmak için ağaçta arama yapın. Eğer eleman ağaçta yoksa, yöntem false döner. *u* elemanı içeren düğüm ve *parentOfu* *u*'nün ebeveyni olsun. Üç durumu göz önünde bulundurun:

Durum 1:  $u$  bir yaprak 3-düğümü veya 4-düğümü.  $e$  'yi  $u$  'dan silin.

**<kenar notu: taşıma>**

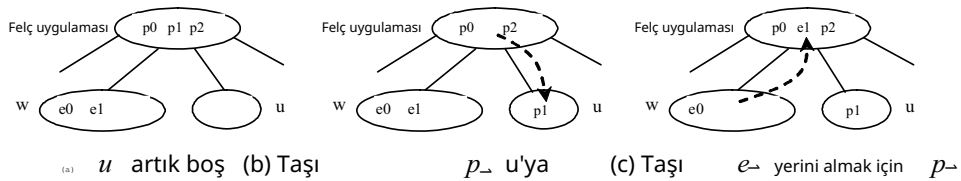
Durum 2:  $u$  bir yaprak 2-düğümü.  $e$  'yi  $u$  'dan silin. Şimdi  $u$  boştur. Bu durum *underflow* olarak bilinir. Bir underflow'u düzeltmek için, iki alt durumu göz önünde bulundurun:

**<kenar notu: aktarım>**

Durum 2.1:  $u$  'nun hemen sol veya sağ kardeşi bir 3- veya 4-düğümü. Düğüm  $w$  olsun, Şekil 47.10(a)'da gösterildiği gibi (varsayalım ki  $w$   $u$  'nun sol kardeşidir).

*parentOfu* için  $u$  , Şekil 47.10(b)'de gösterildiği gibi,

$w$ 'den bir ögeyi alıp, *parentOfu*'deki taşınan öğenin yerine yerleştirin, Şekil'de gösterildiği gibi 47.10 (c) .

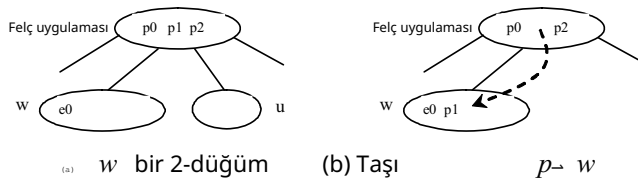


**Şekil 47.10**

*Transfer işlemi boş düğüm  $u$ 'yu doldurur.*

**<kenar notu: füzyon>**

Durum 2.2: Hem  $u$  'nun hemen solundaki hem de sağındaki kardeşi 2-düğüm ise varsa ( $u$  yalnızca bir kardeşe sahip olabilir). Düğüm  $w$  olsun, Şekil 47.11(a)'da gösterildiği gibi (varsayalım ki  $w$ ,  $u$  'nun sol kardeşidir). Fusion işlemi gerçekleştirin,  $u$  'yu atın ve *parentOfu* 'dan  $w$  'ya bir eleman taşıyın, Şekil 47.11(b)'de gösterildiği gibi. Eğer *parentOfu* boş hale gelirse, Durum 2'yi yineleyerek *parentOfu* üzerinde bir transfer veya fusion gerçekleştirin.

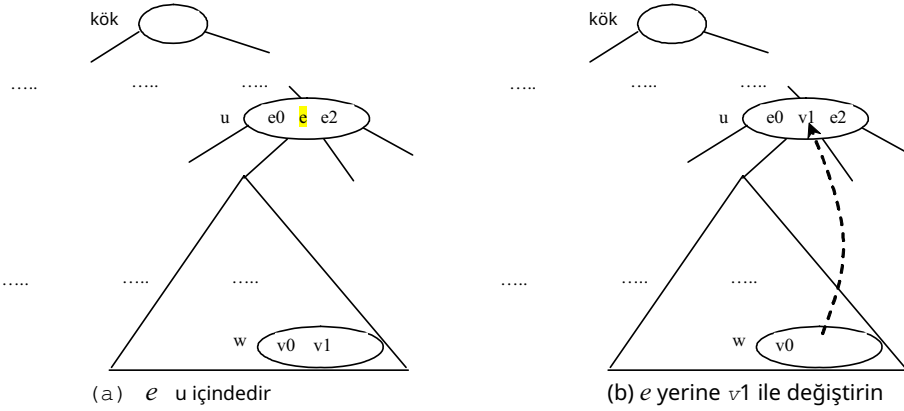


**Şekil 47.11 'ye**

*Fusion işlemi boş düğüm  $u$ 'yu atar.*

**<kenar notu: dahili düğüm>**

Durum 3:  $u$  bir yaprak olmayan düğümdür.  $e$  'nin sol alt ağacındaki en sağdaki yaprak düğümü bulun. Bu düğüm  $w$  olsun, Şekil 47.12(a)'da gösterildiği gibi.  $w$  'deki son elemanı alarak  $e$  'yi  $u$  'da değiştirin, Şekil 47.12(b)'de gösterildiği gibi. Eğer  $w$  boş hale gelirse,  $w$  üzerinde bir transfer veya fusion işlemi uygulayın.



**Şekil 47.12**

*İç düğümdeki bir eleman, bir yaprak*

*düğümdeki bir elemanla değiştirilir.*

Liste 47.3, bir elemanı silme algoritmasını tanımlar.

**Liste 47.3 Bir 2-4 Ağaçtan Bir Elemanı Silme**

*<kenar notu satır 3: düğümü bul>*  
*<kenar notu satır 5: e'yi sil>*  
*<kenar notu satır 10: eleman bulunamadı>*  
*<kenar notu satır 14: e'yi sil>*  
*<kenar notu satır 19: e'yi sil>*  
*<kenar notu satır 22: alt akışı kontrol et ve düzelt>*  
*<kenar notu satır 25: en sağdaki elemanı bul>*  
*<kenar notu satır 34: alt akışı kontrol et ve düzelt>*  
*<kenar notu satır 39: alt akışı kontrol et ve düzelt>*

```

/** Ağaçtan belirtilen elemanı sil */
public boolean delete(E e) {
 Elemanı içeren düğümü bul
 if (düğüm bulunduysa) {
 delete(e, node); // Düğümde e elemanını sil
 size--; // Bir eleman silindikten sonra
 return true; // Eleman başarıyla silindi
 }

 return false; // Eleman ağaçta yok
}

/** Belirtilen elemanı düğümde sil */
private void delete(E e, Tree24Node<E> node) {
 if (e bir yaprak düğümdeyse) {
 // Kökten e'ye giden yolu al
 ArrayList<Tree24Node<E>> path = path(e);

 e'yi düğümde çıkar;

 // Yolda alt akış için düğümü kontrol et ve düzelt
 validate(e, node, path); // Alt akış düğümünü kontrol et
 }
 else { // e bir iç düğümde

```



Düğüm u'nun sol alt ağacındaki en sağdaki düğümü bul;  
En sağdaki düğümünden en sağdaki ögeyi al;

// Kökten e'ye giden yolu al

```
ArrayList<Tree24Node<E>> path = path(rightmostElement);
```

Düğümdeki ögeyi en sağdaki öge ile değiştir

// Yolda alt akış için düğümü kontrol et ve düzelt  
validate(rightmostElement, rightmostNode, path);

}

/\*\* Gerekirse bir transfer veya füzyon işlemi gerçekleştir \*/

private void validate(E e, Tree24Node<E> u,

ArrayList<Tree24Node<E>> path) {

for (int i = path.size() - 1; i >= 0; i--) {

if (u boş değilse)

return; // Tamam, transfer veya füzyon gerçekleştirmeye gerek yok

Tree24Node<E> parentOfu = path.get(i - 1); // u'nun ebeveynini al

// İki kardeşi kontrol et

if (u'nun sol kardeşi birden fazla öğeye sahipse) {

u ile sol kardeşi arasında bir transfer gerçekleştir

}

else if (u'nun sağ kardeşi birden fazla öğeye sahipse) {

u ile sağ kardeşi arasında bir transfer gerçekleştir

}

else if (u'nun sol kardeşi varsa) { // Sol kardeş ile füzyon

u ile sol kardeşi arasında bir füzyon gerçekleştir

u = parentOfu; // Ebeveyn düğümünü kontrol etmek için döngüye geri dön

}

else { // Sağ kardeş ile birleşme (sağ kardeşin var olması gerekir)

u ile sağ kardeşi arasında bir birleşme gerçekleştir

u = parentOfu; // Ebeveyn düğümünü kontrol etmek için döngüye geri dön

}

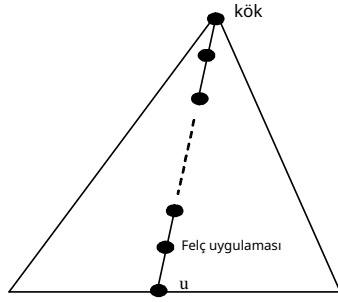
}

N}

delete(E e) metodu, e ögesini içeren düğümü bulur ve  
delete(E e, Tree24Node<E> node) metodunu (satır 5) çağırarak  
düğümünden ögeyi siler.

Düğüm bir yaprak düğümse, kökten e ögesine giden yolu al (satır 17), düğümünden e ögesini sil (satır 19) ve boş düğümü kontrol etmek ve düzeltmek için validate metodu nu çağır. validate(E e, Tree24Node<E> u, ArrayList<Tree24Node<E>> path) metodu, düğüm boşsa bir transfer veya birleşme operasyonu gerçekleştirir. Bu işlemler, u düğümünün ebeveyninin boş hale gelmesine neden olabileceğinden, kökten u düğümüne kadar olan ebeveynleri elde etmek için bir yol alınır, bu şekilde Şekil'de gösterildiği gibi.

47.13.



**Şekil 47.13**

*Yolda bulunan düğümler, bir transfer ve*

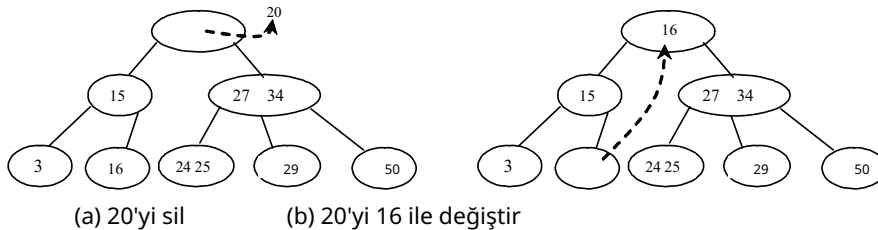
*birleşme işlemi sonucunda boş hale gelebilir.*

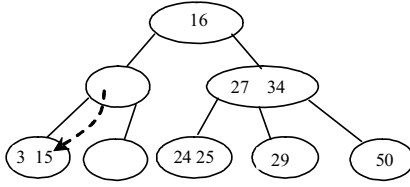
Eğer düğüm bir yaprak olmayan düğümse, düğümün sol alt ağaçtaki en sağdaki ögeyi bul (satırlar 25–26), kökten en sağdaki eleme giden yolu al (satır 29), düğümde e ögesini en sağdaki öge ile değiştir (satır 31) ve eğer boşsa en sağdaki düğümü düzeltmek için validate metodunu çağır (satır 34).

validate(E e, Tree24Node<E> u, ArrayList<Tree24Node<E>> path)u, boş olup olmadığını kontrol eder ve boş düğümü düzeltmek için bir transfer veya birleşim işlemi gerçekleştirir. validate metodu, düğüm boş olmadığından çıkar (satır 43). Aksi takdirde, aşağıdaki durumlardan birini dikkate alın:

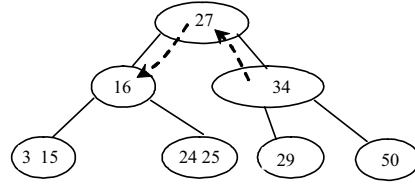
1. Eğer u'nun birden fazla elemanı olan bir sol kardeşi varsa, u ile sol kardeşi arasında bir transfer gerçekleştirin (satır 49).
2. Aksi takdirde, eğer u'nun birden fazla elemanı olan bir sağ kardeşi varsa, u ile sağ kardeşi arasında bir transfer gerçekleştirin (satır 52).
3. Aksi takdirde, eğer u'nun bir sol kardeşi varsa, u ile sol kardeşi arasında bir birleşim gerçekleştirin (satır 55) ve u'yu parentOfu'ya sıfırlayın (satır 56).
4. Aksi takdirde, u'nun bir sağ kardeşi olmalıdır. u ile sağ kardeşi arasında bir birleşim gerçekleştirin (satır 59) ve u'yu parentOfu'ya sıfırlayın (satır 60).

Önceki durumlardan yalnızca biri gerçekleştirilir. Daha sonra, gerekirse yeni bir düğüm u üzerinde bir transfer veya birleşim işlemi gerçekleştirmek için yeni bir yineleme başlar. Şekil 47.14, 2-4 ağacından 20, 15, 3, 6 ve 34 elemanlarının silinme adımlarını göstermektedir (Şekil 47.9(k)).

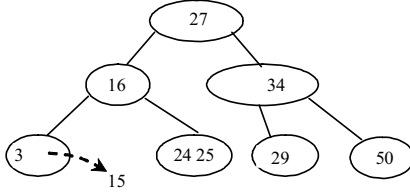




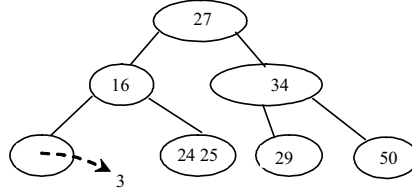
(c) Birleşim gerçekleştir



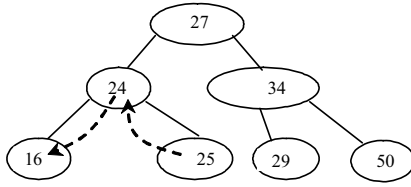
(d) Transfer gerçekleştir



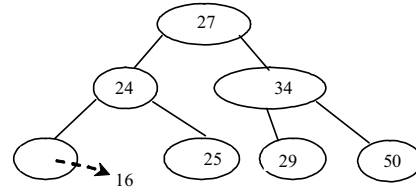
(e) 15'i sil



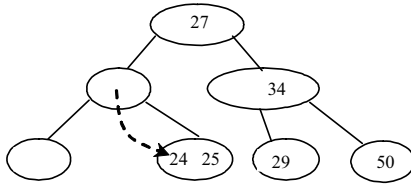
(f) 3'ü sil



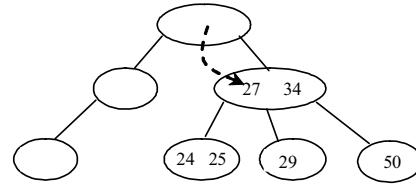
(g) Bir transfer gerçekleştir



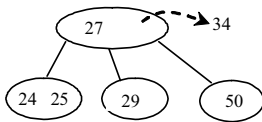
(h) 16'yı sil



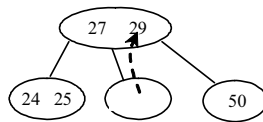
(i) Bir füzyon gerçekleştir



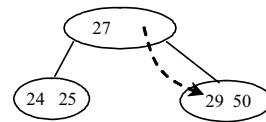
(j) Bir füzyon gerçekleştir



(k) 34'ü sil



(l) 34'ü 16 ile değiştir



(m) Bir füzyon gerçekleştir

#### Şekil 47.14

20, 15, 3, 6 ve 34'ün bir 2-4

ağacından silinmesinden sonra ağaç değişir.

#### 47.6 2-4 Ağaçta Elemanları Gezinme

Inorder, preorder ve postorder gezintileri 2-4 ağaçları için faydalıdır.

Inorder gezintisi elemanları artan sırayla ziyaret eder. Preorder gezintisi elemanları kök içinde ziyaret eder, ardından alt ağaçları soldan sağa doğru özyinelemeli olarak ziyaret eder. Postorder gezintisi

alt ağaçları soldan sağa doğru özyinelemeli olarak ziyaret eder ve ardından kök içindeki elemanları ziyaret eder.

Örneğin, Şekil 47.9(k) içindeki 2-4 ağacında, inorder gezintisi

3 15 16 20 24 25 27 29 34 50

Ön sipariş gezintisi

20 15 3 16 27 34 24 25 29 50

Son sipariş gezintisi

3 16 1 24 25 29 50 27 34 20

#### 47.7 Tree24 Sınıfını Uygulama

Liste 47.4, Tree24 sınıfının tam kaynak kodunu vermektedir.

##### Liste 47.4 Tree24.java

```
<margin not satırı 4: kök> n<margin not satırı 5: boyut> n<margin not satırı 8: no-arg yapıcı> n<margin not satırı 12: yapıcı>
<kenar notu satırı 18: arama> n<kenar notu satırı 22: bulundu> n<kenar notu satırı 26: sonraki alt ağaç> n<kenar notu satırı 34: bir eşleşme bul> n<kenar notu satırı 36: eşleşti> n<kenar notu satırı 43: sonraki alt ağaç> n<kenar notu satırı 44: yaprak düğümü> n<kenar notu satırı 47: ekleme noktası> n<kenar notu satırı 54: ağaca ekle> n<kenar notu satırı 55: boş ağaç> n<kenar notu satırı 59: yaprak düğümünü bul> n<kenar notu satırı 71: düğüme ekle> n<kenar notu satırı 79: düğüme ekle> n<kenar notu satırı 85: taşma yok> n<kenar notu satırı 90: taşma> n<kenar notu satırı 91: split> n<kenar notu satırı 93: u köktür> n<kenar notu satırı 101: parentOfu'ya ekle> n<kenar notu satırı 110: düğüme ekle> n<kenar notu satırı 112: ekleme noktası> n<kenar notu satırı 119: böl>
```

```
<margin note line 123: medyanı al>
<margin note line 127: e ekle>
<margin note line 133: rightChildOf'e ekle>
<margin note line 138: medyanı döndür>
<margin note line 142: yolu al>
<margin note line 147: aranan düğümü ekle>
<margin note line 156: yolu döndür>
<margin note line 160: ağaçtan sil>
<margin note line 162: düğümü bul>
<margin note line 164: bulundu mu?>
<margin note line 165: düğümden sil>
<margin note line 177: düğümden sil>
<margin note line 178: yaprak düğümü mü?>
```

<margin note line 182: e sil>  
 <margin note line 184: düğüm kök mü?>  
 <margin note line 190: ağacı doğrula>  
 <margin note line 192: yapraksız düğüm>  
 <margin note line 194: en sağdaki eleman>  
 <margin note line 206: elemanı değiştir>  
 <margin note line 209: ağacı doğrula>  
 <margin note line 214: ağacı doğrula>  
 <margin note line 222: sol kardeşle aktar>  
 <margin note line 226: sağ kardeşle aktar>  
 <margin note line 233: sol kardeşle birleş>  
 <margin note line 248: sağ kardeşle birleş>  
 <margin note line 262: ekleme noktasını bul>  
 <margin note line 273: sol kardeşle aktar>  
 <margin note line 290: sağ kardeşle aktar>  
 <margin note line 305: sol kardeşle birleş>  
 <margin note line 319: sağ kardeşle birleş>  
 <margin note line 338: öncelikli>

<margin note line 343: recursive preorder>

<margin note line 374: iç Tree24Node sınıfı>

<margin note line 376: eleman listesi>

<margin note line 378: çocuk listesi>

Java.util.ArrayList'i içe aktarın;

genel sınıf Tree24<E, Karşılaştırılabilir<E>>'yi genişletir, Tree<E>'yi uygular { n özel

Tree24Node<E> root;

private int size;

/\*\* Varsayılan bir 2-4 ağacı oluştur \*/

public Tree24() {

}

/\*\* Bir nesne dizisinden 2-4 ağacı oluştur \*/

public Tree24(E[] elements) {

for (int i = 0; i < elementler.uzunluk; i++) n insert(elementler[i]);

}

/\*\* Ağaçta bir elemanı ara \*/

public boolean search(E e) {

Tree24Node<E> current = root; // Kökten başla

while (current != null) {

if (matched(e, current)) { // Eleman düğümde var  
return true; // Eleman bulundu

}

aksi takdirde {

current = getChildNode(e, current); // Bir alt ağaçta ara

}

}

return false; // Eleman ağaçta yok

}

```

/** Eleman bu düğümde bulunursa true döndür */
private boolean matched(E e, Tree24Node<E> node) {
 for (int i = 0; i < node.elements.size(); i++)
 if (node.elements.get(i).equals(e))
 return true; // Eleman bulundu

 return false; // Bu düğümde eşleşme yok
}

/** Bir çocuk düğümünü bulmak için e ögesini arayın */
private Tree24Node<E> getChildNode(E e, Tree24Node<E> node) {
 if (node.child.size() == 0)
 return null; // düğüm bir yaprak

 int i = locate(e, node); // e için ekleme noktasını bul
 return node.child.get(i); // Çocuk düğümü döndür
}

/** e ögesini ağaca ekle
 * Ögenin başarıyla eklenmesi durumunda true döndür
 */
public boolean insert(E e) {
 if (root == null)
 root = new Tree24Node<E>(e); // öge için yeni bir kök oluştur
 else {
 // e'yi eklemek için yaprak düğümünü bul
 Tree24Node<E> leafNode = null;
 Tree24Node<E> geçerli = kök;
 while (geçerli != null)
 if (eşleşen(e, geçerli)) {
 return false; // Aynı öge bulundu, hiçbir şey eklenmedi
 }
 başka {
 yaprakDüğüm = mevcut;
 mevcut = getChildNode(e, mevcut);
 }

 // Elemanı e yaprak düğümüne ekle
 insert(e, null, yaprakDüğüm); // e'nin sağ çocuğu null
 }

 boyut++; // Boyutu artır
 return true; // Eleman eklendi
}

/** Eleman e'yi düğüm u'ya ekle */
private void insert(E e, Tree24Node<E> e'ninSağÇocuğu,
 Tree24Node<E> u) {
 // Eleman e'ye giden arama yolunu al
 ArrayList<Tree24Node<E>> path = path(e);

 for (int i = path.size() - 1; i >= 0; i--) {
 if (u.elements.size() < 3) { // u bir 2-düğüm veya 3-düğüm
 insert23(e, rightChildOf(u), u); // e'yi u düğümüne ekle
 break; // u'nun ebeveynine daha fazla ekleme gerek yok
 }
 }
}

```

```

aksi takdirde {
 Tree24Node<E> v = new Tree24Node<E>(); // Yeni bir düğüm oluştur
 E median = split(e, rightChildOfe, u, v); // u'yu böl

 if (u == kök) {
 root = new Tree24Node<E>(median); // Yeni kök
 root.child.add(u); // u, medyanın sol çocuğu
 root.child.add(v); // v, medyanın sağ çocuğu
 break; // u'nun ebeveynine daha fazla ekleme gerek yok
 }
 başka {
 // For döngüsündeki bir sonraki yineleme için yeni değerler kullan
 e = median; // Ebeveyne eklenecek eleman
 rightChildOfe = v; // Elemanın sağ çocuğu
 u = path.get(i - 1); // Elemanı eklemek için yeni düğüm
 }
}

}

}

/** Elemanı 2 veya 3'e ekle ve ekleme noktasını döndür */
private void insert23(E e, Tree24Node<E> rightChildOfe,
 Tree24Node<E> düğümü) {
 int i = this.locate(e, node); // Nereye ekleyeceğini bul
 node.elements.add(i, e); // Elemanı düğüme ekle
 if (rightChildOfe != null)
 node.child.add(i + 1, rightChildOfe); // Çocuk bağlantısını ekle
}

/** 4-düğüm u'yu u ve v'ye ayır ve e'yi u veya v'ye ekle */
private E split(E e, Tree24Node<E> rightChildOfe,
 Tree24Node<E> u, Tree24Node<E> v) {
 // Düğüm u'daki son öğeyi düğüm v'ye taşı
 v.elements.add(u.elements.remove(2));
 E median = u.elements.remove(1);

 // Yaprak olmayan bir düğüm için çocukları ayır
 // Düğüm u'daki son iki çocuğu düğüm v'ye taşı
 if (u.child.size() > 0) {
 v.child.add(a.child.remove(2)); n v.child
 .add(a.child.remove(2)); N }

 // e'yi 2- veya 3-düğüm u veya v'ye ekle.
 if (e.compareTo(median) < 0) n ins
 ert23(e, rightChildOfe, u); başka türlü

 insert23(e, rightChildOfe, v);

 return median; // Medyan öğeyi döndür
}

/** Eleman e'ye giden bir arama yolu döndür */
private ArrayList<Tree24Node<E>> path(E e) {
 ArrayList<Tree24Node<E>> list = new ArrayList<Tree24Node<E>>();
 Tree24Node<E> current = root; // Kökten başla

```

```

while (current != null) {
 list.add(current); // D ğ m  listeye ekle
 if (matched(e, current)) {
 break; // Eleman bulundu
 }
 aksi takdirde {
 akim = getChildNode(e, ge erli); N }

}

listeyi d nd r; // D ğ mlerin bir dizisini d nd r
}

/** A a tan belirtilen  geyi sil */
public boolean delete(E e) {
 // e  gesini i eren d ğ m  bul
 Tree24Node<E> node = root;
 while (d ğ m != null)
 if (e le en(e, d ğ m)) {
 delete(e, node); // D ğ mden e  gesini sil
 size--; // Bir  ge silindikten sonra
 return true; //  gesi ba arıyla silindi
 }
 aksi takdirde {
 d ğ m = getChildNode(e, d ğ m);n }

return false; //  gesi a a ta yok
}

/** D ğ mden belirtilen  geyi sil */
private void delete(E e, Tree24Node<E> node) {
 if (node.child.size() == 0) { // e bir yaprak d ğ mde
 // K kten e'ye giden yolu al
 ArrayList<Tree24Node<E>> path = path(e);

 node.elements.remove(e); // e  gesini kaldır

 if (node == root) { //  zel durum
 if (node.elements.size() == 0)
 root = null; // Bo  a a 
 return; // Tamam
 }

 validate(e, node, path); // Alt akı  d ğ m n  kontrol et
 }
 else { // e bir i  d ğ mde
 // D ğ m n sol alt a acındaki en sağdaki d ğ m  bul
 int index = locate(e, node); // node'daki e'nin indeksi
 Tree24Node<E> ge erli = node.child.get(index); n while (c
 urrent.child.size() > 0) {
 akim = current.child.get(current.child.size() - 1); N }

 E en sağdakiElement =
 current.elements.get(current.elements.size() - 1);

```



```

// Kökten e'ye giden yolu al
ArrayList<Tree24Node<E>> path = path(rightmostElement);

// Silinen ögeyi en sağdaki öge ile değiştir
node.elements.set(index, current.elements.remove(
 current.elements.size() - 1));

validate(rightmostElement, current, path); // Underflow'u kontrol et
}

/** Gerekirse transfer ve karıştırma işlemlerini gerçekleştir */
private void validate(E e, Tree24Node<E> u,
 ArrayList<Tree24Node<E>> path) {
 for (int i = path.size() - 1; u.elements.size() == 0; i--) {
 Tree24Node<E> parentOfu = path.get(i - 1); // u'nun ebeveynini al
 int k = locate(e, parentOfu); // Ebeveyn düğümde e'nin indeksi

 // İki kardeşi kontrol et
 if (k > 0 && parentOfu.child.get(k - 1).elements.size() > 1) { n leftSiblingTransfer(k, u, parentOfu);
 }
 aksi takdirde (k + 1 < parentOfu.child.size() &&
 parentOfu.child.get(k + 1).elements.size() > 1) {
 rightSiblingTransfer(k, u, parentOfu);
 }
 else if (k - 1 >= 0) { // Sol kardeşle birleşme
 // u düğümünün sol kardeşini al
 Tree24Node<E> leftNode = parentOfu.child.get(k - 1);

 // u düğümünde sol kardeşle birleşme gerçekleştir
 leftSiblingFusion(k, leftNode, u, parentOfu);

 // Kök boş olduğunda tamamlandı
 if (parentOfu == root && parentOfu.elements.size() == 0) { n root = leftNode;
 }
 u = parentOfu; // Ebeveyn düğümünü kontrol etmek için döngüye geri dön
 }
 else { // Sağ kardeşle birleşme (sağ kardeşin var olması gerekir)
 // u düğümünün sol kardeşini al
 Tree24Node<E> rightNode = parentOfu.child.get(k + 1);

 // u düğümünde sağ kardeşle birleşme gerçekleştir
 rightSiblingFusion(k, rightNode, u, parentOfu);

 // Kök boş olduğunda tamamlandı
 if (parentOfu == root && parentOfu.elements.size() == 0) { n root = rightNode;
 }
 u = parentOfu; // Ebeveyn düğümünü kontrol etmek için döngüye geri dön
 }
 }
}

```

```

 }
 }
}

/** D ğ mdeki elemanın eklemeye noktasını bul */
private int locate(E o, Tree24Node<E> node) {
 for (int i = 0; i < node.elements.size(); i++) { n if (o.compareTo
 (node.elements.get(i)) <= 0) { n return i;

 }
 }

 node.elements.size();

}

/** Sol kardeş ile bir transfer ger ekleřtirin */
private void leftSiblingTransfer(int k,
 Tree24Node<E> u, Tree24Node<E> parentOfu) {
 // Bir  geyi ebeveynden u'ya tařı
 u.elements.add(0, parentOfu.elements.get(k - 1));

 // Bir  geyi sol d ğ mden ebeveyne tařı
 Tree24Node<E> leftNode = parentOfu.child.get(k - 1);
 parentOfu.elements.set(k - 1,
 leftNode.elements.remove(leftNode.elements.size() - 1));

 // Sol kardeřten d ğ me  ocuk baėlantısını tařı
 if (leftNode.child.size() > 0) n u.child.a
 dd(0, leftNode.child.remove(n leftNode.child.
 size() - 1));

}

/** Saė kardeş ile bir transfer ger ekleřtirin */
private void rightSiblingTransfer(int k,
 Tree24Node<E> u, Tree24Node<E> parentOfu) {
 // Bir  geyi ebeveynden u'ya aktar
 u.elements.add(parentOfu.elements.get(k));

 // Bir  geyi saė d ğ mden ebeveyne aktar
 Tree24Node<E> rightNode = parentOfu.child.get(k + 1);
 parentOfu.elements.set(k, rightNode.elements.remove(0));

 // Saė kardeřten d ğ me  ocuk baėlantısını tařı
 if (rightNode.child.size() > 0)
 u.child.add(rightNode.child.remove(0)); N }

}

/** Sol kardeş ile bir birleřim ger ekleřtirin */
private void leftSiblingFusion(int k, Tree24Node<E> leftNode,
 Tree24Node<E> u, Tree24Node<E> parentOfu) {
 // Ebeveynden sol kardeře bir  ge aktarın
 leftNode.elements.add(parentOfu.elements.remove(k - 1));

 // Boř d ğ me olan baėlantıyı kaldırın
 parentOfu.child.remove(k);
}

```

```

// Yaprak olmayan düğüm için çocuk bağlantılarını ayarlayın
if (u.child.size() > 0)
 left Node.child.add(a.child.remove(0)); N }

/** Sağ kardeş ile bir birleşim gerçekleştirin */
private void rightSiblingFusion(int k, Tree24Node<E> rightNode,
 Tree24Node<E> u, Tree24Node<E> parentOfu) {
 // Ebeveynden sağ kardeşe bir öge aktarın
 rightNode.elements.add(0, parentOfu.elements.remove(k));

 // Boş düğüme olan bağlantıyı kaldırın
 parentOfu.child.remove(k);

 // Yaprak olmayan düğüm için çocuk bağlantılarını ayarlayın
 if (u.child.size() > 0)
 rightNode.child.add(0, u.child.remove(0)); N }

/** Ağaçtaki düğüm sayısını alın */
public int getSize() {
 boyut döndür;
}

/** Kökten ön sipariş gezintisi */
public void preorder() {
 preorder(kök);
}

/** Bir alt ağaçtan ön sipariş gezintisi */
private void preorder(Tree24Node<E> kök) {
 eğer (kök == null) döndür;
 for (int i = 0; i < root.elements.size(); i++) n System.out.prin
 t(root.elements.get(i) + " "); n for (int i = 0; i < root.child.si
 ze(); i++) n preorder(root.child.get(i));
}

/** Kökten inorder geçiş */
public void inorder() {
 // Egzersiz olarak bırakıldı
}

/** Kökten postorder geçiş */
public void postorder() {
 // Egzersiz olarak bırakıldı
}

/** Ağaç boşsa true döndür */
public boolean isEmpty() {
 root'a dönüş == null; N }

/** Ağaçtaki elemanları gezmek için bir iterator döndür */
public java.util.Iterator iterator() {

```

```

// Egzersiz olarak bırakıldı
return null;

}

/** 2-4 ağaç düğümünü tanımla */
protected static class Tree24Node<E extends Comparable<E>> {
 // elemanlar maksimum üç değere sahiptir
 ArrayList<E> elemanlar = new ArrayList<E>(3);
 // Her birinin maksimum dört çocuğu vardır
 ArrayList<Tree24Node<E>> çocuk
 = new ArrayList<Tree24Node<E>>(4);

 /** Boş bir Tree24 düğümü oluştur */
 Tree24Node() {
 }

 /** Başlangıç elemanı ile bir Tree24 düğümü oluştur */
 Tree24Node(E o) {
 elemanlar.add(o);
 }
}
}
N)

```

**<kenar boşluğu notu: kök>n<**

**kenar boşluğu notu: boyut>**

Tree24 sınıfı, root ve size veri alanlarını içerir (satırlar 4–5).  
kök, kök düğümünü referans alır ve boyut, ağaçtaki eleman sayısını saklar.

**<margin note: yapıcılar>**

Tree24 sınıfının iki yapıcısı vardır: boş bir ağaç oluşturan bir argüman almayan yapıcı (satırlar 8–9) ve bir dizi elemandan başlangıçta bir Tree24 oluşturan bir yapıcı (satırlar 12–15).

**<margin note: arama>**

arama metodu (satırlar 18–31) ağaçta bir elemanı arar. Eleman ağaçta varsa true (satır 23) döner ve arama boş bir alt ağaçta (satır 30) sona ererse false döner.

**<margin note: eşleştiri>**

eşleştiri(e, düğüm) metodu (satırlar 34–40) eleman e'nin düğümde nerede olduğunu kontrol eder.

**<kenar notu: getChildNode>**

getChildNode(e, düğüm) metodu (satırlar 43–49) e'nin aranması gereken bir alt ağacın kökünü döner.

**<kenar notu: ekle(e)>**

insert(E e) metodu bir ağaca bir eleman ekler (satırlar 54–78). Eğer ağaç boşsa, yeni bir kök oluşturulur (satır 56). Metod, elemanın ekleneceği bir yaprak düğüm bulur ve insert(e, null, leafNode) çağrısını yaparak elemanı ekler (satır 71).

**<kenar notu: insert(e, rightChildOf, u)>**

insert(e, rightChildOf, u) metodu bir elemanı düğüm u'ya ekler (satırlar 79–107). Metoda önce path(e) (satır 82) çağrısını yaparak kökten düğüm u'ya kadar bir arama yolu elde eder. for döngüsünün her yinelenmesi u'yu ve onun ebeveyni parentOf'u dikkate alır (satırlar 84–106). Eğer u bir 2-düğüm ise

veya 3-düğüm, e'yi ve onun çocuk bağlantısını rightChildOfe'yi u'ya eklemek için insert23(e, rightChildOfe, u) çağrısını yapın (satır 86). Bölmeye gerek yoktur (satır 87). Aksi takdirde, yeni bir düğüm v oluşturun (satır 90) ve u'yu v'ye bölmek için split(e, rightChildOfe, u, v) çağrısını yapın (satır 91). Splitmetodu e'yi ya u'ya ya da v'ye ekler ve orijinal u'daki medyanı döndürür. Eğer u kökse, medyanı tutacak yeni bir kök oluşturur ve u ve v'yi medyan için sol ve sağ çocuklar olarak ayarlayın (satırlar 95-96). Eğer kök değilse, medyanı parentOfu'ya bir sonraki yinelemede ekleyin (satırlar 101-103).

#### <kenar notu: insert23>

insert23(e, rightChildOfe, node) metodu e'yi ve sağ çocuğuna olan referansı düğüme ekler (satırlar 110-116). Metod öncelikle locate(e, node) (satır 112) çağrısını yaparak bir ekleme noktası bulur, sonra e'yi düğüme ekler (satır 113). Eğer rightChildOfe null değilse, düğümün çocuk listesine eklenir (satır 115).

#### <kenar notu: split>

split(e, rightChildOfe, u, v) metodu 4-düğüm u'yu (satırlar 119-139) böler. Bu şu şekilde gerçekleştirilir: (1) u'dan v'ye son elemanı taşıyın ve u'dan medyan elemanını çıkarın (satırlar 122-123); (2) eğer u bir yaprak olmayan düğümse, u'dan v'ye son iki çocuk bağlantısını taşıyın (satırlar 127-130); (3) e < medyan ise, e'yi u'ya ekleyin; aksi takdirde, e'yi v'ye ekleyin (satırlar 133-136); (4) medyanı döndürün (satır 138).

#### <margin note: path></margin note: path>

path(e) metodu, e'yi bulmak için kökten aranan düğümlerin bir ArrayList'ini döndürür (satırlar 142-157). Eğer e ağaçta varsa, yolun sonundaki düğüm e'yi içerir. Aksi takdirde, son düğüm e'nin eklenmesi gereken yerdir.

#### <margin note: delete(e)></margin note: delete(e)>

delete(E e) metodu, ağaçtan bir ögeyi siler (satırlar 160-174). Metod önce e'yi içeren düğümü bulur ve delete(e, node) metodunu çağırarak e'yi düğümden siler (satır 165). Eğer öge ağaçta yoksa, false döner (satır 173).

#### <margin note: delete(e, node)></margin note: delete(e, node)>

delete(e, node) metodu, u düğümünden bir ögeyi siler (satırlar 177-211). Eğer düğüm bir yaprak düğümse, e'ye giden yolu elde et (satır 180), e'yi sil (satır 182), ağaç boşsa root' u null olarak ayarla (satırlar 184-188) ve boş düğümler üzerinde transfer ve birleşim operasyonunu uygulamak için validate metodunu çağır (satır 190). Eğer düğüm bir yaprak düğüm değilse, en sağdaki ögeyi bul (satırlar 194-200), e'ye giden yolu elde et (satır 203), e'yi en sağdaki öge ile değiştir (satırlar 206-207) ve boş düğümler üzerinde transfer ve birleşim işlemlerini uygulamak için validate metodunu çağır (satır 209).

#### <margin note: validate></margin note: validate>

validate(e, u, path) metodu, ağacın geçerli bir 2-4 ağaç olduğunu garanti eder (satırlar 214-259). for döngüsü, u boş olmadığında sona erer (satır 216). Döngü gövdesi, boş düğüm u'yu düzeltmek için bir transfer veya birleşim işlemi gerçekleştirerek çalıştırılır. Eğer bir düğüm fazla eleman içeren bir sol kardeş varsa, u ile sol kardeş arasında bir transfer gerçekleştir (satır 222). Aksi takdirde, eğer birden fazla eleman içeren bir sağ kardeş varsa, u ile sol kardeş arasında bir transfer gerçekleştir (satır 226). Aksi takdirde, eğer bir sol kardeş varsa, u ile sol kardeş arasında bir birleşim gerçekleştir (satırlar 230-239) ve bir sonraki döngü iterasyonunda parentOfu'yu doğruyla (satır 241). Aksi takdirde, u ile sağ kardeş arasında bir birleşim gerçekleştir.

**<kenar notu: bulun>**

locate(e, node) metodu, e'nin node içindeki indeksini bulur (satırlar 262-270) .

**<kenar notu: aktarım> n<ken**

**ar notu: füzyon>**

leftSiblingTransfer(k, u, parentOfu) metodu, u'yu sol kardeşi ile transfer eder (satırlar 273-287). rightSiblingTransfer(k, u, parentOfu) metodu, u'yu sağ kardeşi ile transfer eder

(satırlar 290-302). leftSiblingFusion(k, leftNode, u, parentOfu)

metodu, u'yu sol kardeşi leftNode ile birleştirir (satırlar

305-316). rightSiblingFusion(k, rightNode, u, parentOfu) metodu

u'yu sağ kardeşi rightNode ile birleştirir (satırlar 319-330) .

**<kenar notu: ön sipariş>**

preorder() metodu, ağaçtaki tüm elemanları ön sıralama ile gösterir (satırlar 338-350).

**<kenar notu: Tree24Node>**

iç sınıf Tree24Node, ağaçtaki bir düğüm için bir sınıf tanımlar (satırlar 374-389).

## 47.8 Tree24 Sınıfını Test Etme

Liste 47.5 bir test programı vermektedir. Program, bir 2-4 ağacı oluşturur ve elemanları 6-20. satırlarda ekler, 22-56. satırlarda ise elemanları siler.

### Liste 47.5 TestTree24.java

**<kenar notu satır 4: bir Tree24 oluştur>**

**<kenar notu satır 6: 34 ekle>**

**<kenar notu satır 7: 3 ekle>**

**<kenar notu satır 8: 50 ekle>**

**<kenar notu satır 15: 24 ekle>**

**<kenar notu satır 21: 70 ekle>**

**<kenar notu satır 25: 34 sil>**

```
1 genel sınıf TestTree24 {
2 public static void main(String[] args) {
3 // 2-4 ağacı oluştur
4 Tree24<Tamsayı> ağaç = t new Tree24<Tamsayı>();
5
6 ağaç.insert(t34t);
7 ağaç.insert(t3t);
8 ağaç.insert(t50t);
9 ağaç.insert(t20t);
10 ağaç.insert(t15t);
11 ağaç.insert(t16t);
12 ağaç.insert(t25t);
13 ağaç.insert(t27t);
14 ağaç.insert(t29t);
15 ağaç.insert(t24t);
16 System.out.print ("24 ekledikten sonra:");
17 printTree(tree);
18 Tree.insert(t23t);n 19 Tree.i
nsert(t22t);n 20 Tree.insert(t6
0t);n 21 Tree.insert(t70t);
```

```

22 System.out.print("70 ekledikten sonra:"); 23
printTree(tree);
24
25 ağaç.sil(34);
26 System.out.print("34 silindikten sonra:"); 27 p
rintTree(tree);
28
29 ağaç.delete(25);
30 System.out.print("25 silindikten sonra:"); 31
printTree(tree);
32
33 ağaç.delete(50);
34 System.out.print("50 silindikten sonra:"); 35
printTree(ağaç);
36
37 ağaç.delete(16);
38 System.out.print("16 silindikten sonra:"); 39
printTree(ağaç);
40
41 ağaç.delete(3);
42 System.out.print("3 silindikten sonra:"); 43
printTree(ağaç);
44
45 ağaç.delete(15);
46 System.out.print("15 silindikten sonra:"); 47
printTree(ağaç);
48 }
49
50 public static void printTree(Ağaç ağaç) {
51 // Ağaçta gez
52 System.out.print("Öncelikli: "); 53
tree.preorder();
54 System.out.print("Düğüm sayısı " + tree.getSize()); 55 System.out.printl
n();
56 } n 57
}

```

#### <Çıktı>

24 ekledikten sonra:  
Öncelikli: 20 15 3 16 27 34 24 25 29 50  
Düğüm sayısı 10

70 ekledikten sonra:  
Öncelikli: 20 15 3 16 24 27 34 22 23 25 29 50 60 70  
Düğüm sayısı 14

34 silindikten sonra:  
Ön Sipariş: 20 15 3 16 24 27 50 22 23 25 29 60 70  
Düğüm sayısı 13

25 silindikten sonra:  
Ön sipariş: 20 15 3 16 23 27 50 22 24 29 60 70  
Düğüm sayısı 12

50 silindikten sonra:  
Ön sipariş: 20 15 3 16 23 27 60 22 24 29 70  
Düğüm sayısı 11

16 silindikten sonra:

Ön sipariş: 23 20 3 15 22 27 60 24 29 70  
Düğüm sayısı 10

3 silindikten sonra:

Ön sipariş: 23 20 15 22 27 60 24 29 70  
Düğüm sayısı 9

15 silindikten sonra:

Ön sipariş: 27 23 20 22 24 60 29 70  
Düğüm sayısı 8

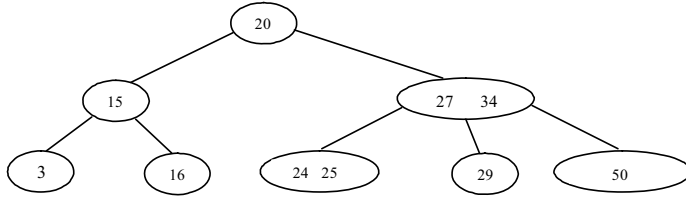
<Çıkış Sonu>

Şekil 47.15, ağaç elemanlar eklendikçe nasıl evrildiğini göstermektedir.

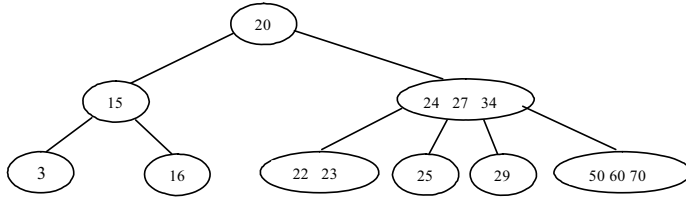
34, 3, 50, 20, 15, 16, 25, 27, 29 ve 24 ağaçta eklendikten sonra, ağaç Şekil 47.15(a)'da gösterildiği gibidir. 23, 22, 60 ve 70 eklendikten sonra, ağaç Şekil 47.15(b)'de gösterildiği gibidir. 23, 22, 60 ve

70 eklendikten sonra, ağaç Şekil 47.15(b)'de gösterildiği gibidir. 34 silindikten sonra, ağaç Şekil 47.15(c)'de gösterildiği gibidir. 25 silindikten sonra, ağaç Şekil 47.15(d)'de gösterildiği gibidir. 50 silindikten sonra, ağaç Şekil 47.15(e)'de gösterildiği gibidir. 16 silindikten sonra, ağaç Şekil 47.15(f)'de gösterildiği gibidir. 3 silindikten sonra, ağaç Şekil 47.15(g)'de gösterildiği gibidir.

15 silindikten sonra, ağaç Şekil 47.15(h)'de gösterildiği gibidir.

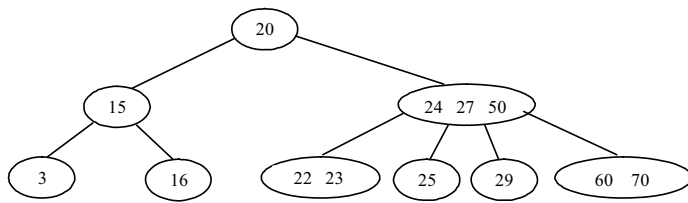


(a) 34, 3, 50, 20, 15, 16, 25, 27, 29 ve 24'ü bu sırayla ekledikten sonra

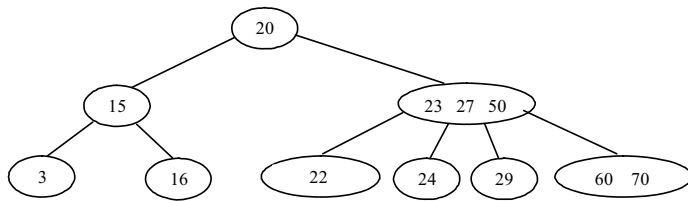


(b) 23, 22, 60 ve 70 eklendikten sonra

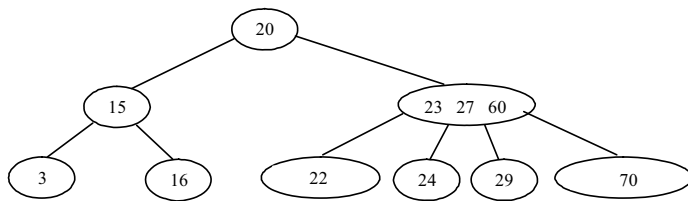




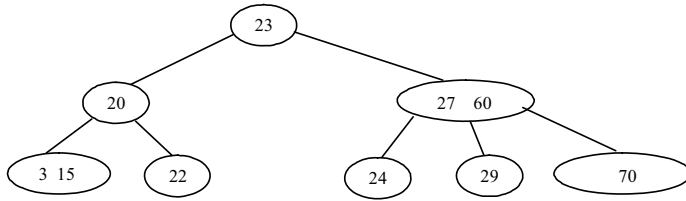
(c) 34 silindikten sonra



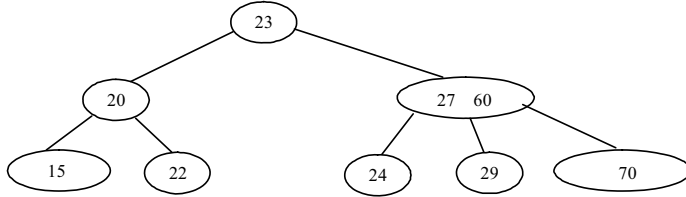
(d) 25 silindikten sonra



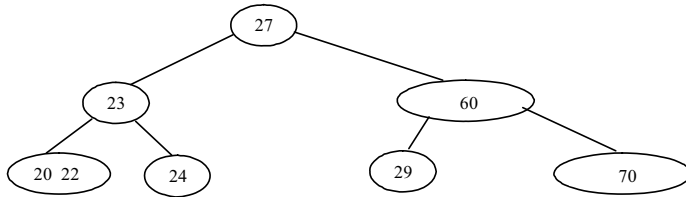
(e) 50 silindikten sonra



(f) 16 silindikten sonra



(g) 3 silindikten sonra



(h) 15 silindikten sonra

#### Şekil 47.15

Ağaç, elemanlar eklendikçe ve silindikçe evrim geçirir.

#### 47.9 Zaman Karmaşıklığı Analizi

Bir 2-4 ağacı tamamen dengeli bir ikili ağaç olduğundan, yüksekliği en fazla

$O(\lg n)$ . Arama, ekleme ve silme yöntemleri, ağaçtaki bir yol boyunca nodlar üzerinde çalışır. Bir düğüm içindeki

elementi aramak sabit bir zaman alır. Bu nedenle, arama yöntemi  $O(\lg n)$  zaman. İçin ekleme yöntemi için, bir düğümü bölme süresi sabit bir zaman alır.

Yani, insert metodu alır  $O(\lg n)$  zaman. Silme yöntemi için, bu bir transfer ve füzyon işlemi gerçekleştirmek için sabit bir zaman alır. Bu nedenle, delete metodu alır  $O(\lg n)$  zaman.

#### 47.10 B-Ağaç

Şu ana kadar tüm veri kümesinin ana bellekte saklandığını varsayıyoruz.

Veri seti çok büyükse ve ana belleğe sığmıyorsa, çoğu veritabanında verilerin disklerde saklandığı durumlarda olduğu gibi ne olur? Bir veritabanı tablosunda bir milyon kaydı düzenlemek için bir AVL ağacı kullandığınızı varsayalım.

Bir kaydı bulmak için, geçilen düğümlerin ortalama sayısı

$\log_{20} 1,000,000 \approx 20$ . Tüm düğümlerin ana bellekte saklanması durumunda bu uygundur.

Ancak, diskte saklanan düğümler için bu, 20 disk okuması anlamına gelir. Disk I/O maliyetlidir ve bellek erişiminden binlerce kat daha yavaştır.

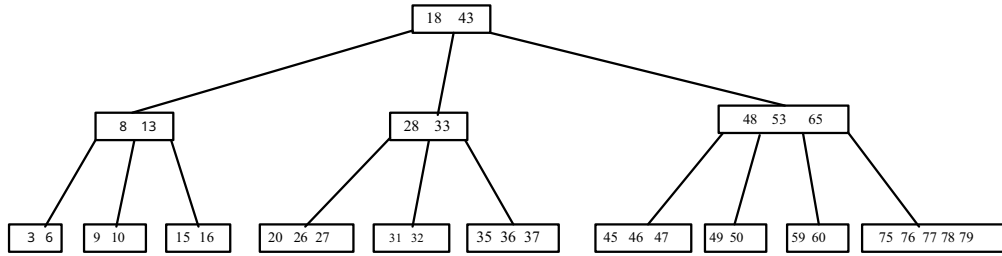
Performansı artırmak için, disk I/O sayısını azaltmamız gerekiyor. Arama, ekleme ve silme işlemleri için verimli bir veri yapısı

ikincil depolama alanında, örneğin sabit disklerde saklanan veriler için B-ağacı, 2-4 ağacının genelleştirilmesidir.

Bir  $d$  dereceli B-ağacı aşağıdaki gibi tanımlanır:

1. Kök düğüm hariç her düğüm  $aoao$   $12/d$  içerir ve  $tdt1$  anahtarlar.
2. Kök, en fazla  $tdt1$  anahtarlar.
3.  $k$  anahtarına sahip bir yaprak olmayan düğüm,  $tk1$  çocuklara sahiptir.
4. Tüm yaprak düğümleri aynı derinliğe sahiptir.

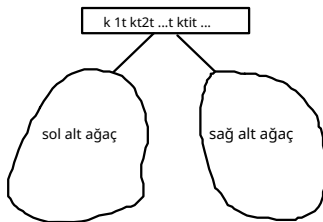
Şekil 47.16, 6. dereceden bir B-ağacını göstermektedir. Basitlik açısından, anahtarları temsil etmek için tam sayılar kullanıyoruz. Her anahtar, veritabanındaki gerçek kayda işaret eden bir işaretçi ile ilişkilendirilmiştir. Basitlik açısından, veritabanındaki kayıtlara işaret eden işaretçiler şekilden çıkarılmıştır.



Şekil 47.16

6. dereceden bir B-ağacında, kök hariç her düğüm 2 ile 5 anahtar arasında bir değer içerebilir.

Bir B-ağacının bir arama ağacı olduğunu unutmayın. Her düğümdeki anahtarlar artan sırayla yerleştirilmiştir. Bir iç düğümdeki her anahtarın bir sol alt ağacı ve bir sağ alt ağacı vardır; bu, Şekil 47.17'de gösterilmiştir. Sol alt ağaçtaki tüm anahtarlar, ebeveyn düğümdeki anahtardan daha küçüktür ve sağ alt ağaçtaki tüm anahtarlar, ebeveyn düğümdeki anahtardan daha büyüktür.



Şekil 47.17

$k_1$  anahtarının sol (sağ) alt ağacındaki anahtarlar,  $k_1$ 'den daha küçüktür (büyüktür).

**<margin note: bir düğüm başına bir blok>**

Bir diskteki IO işlemlerinin temel birimi bir bloktur. Bir diskten veri okuduğunuzda, veriyi içeren tüm blok okunur. Siz

bir düğümün tek bir

disk bloğuna sığabilmesi için uygun bir sıralama d seçilmelidir. Bu, disk IO'larının sayısını en aza indirecektir.

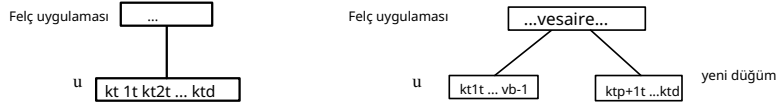
2-4 ağacı aslında 4. dereceden bir B-ağacıdır. Bir 2-4 ağacında insertion ve silme teknikleri, bir B-ağacı için kolayca genelleştirilebilir.

**<kenar boşluğu notu: ekleme>**

Bir B-ağacına anahtar eklemek, bir 2-4 ağacında yapılanlara benzer. Öncelikle anahtarın ekleneceği yaprak düğümü bulun. Anahtarı düğüme ekleyin. Eklemeden sonra, eğer yaprak düğüm  $d$  anahtara sahipse, bir taşma meydana gelir. Taşmayı çözmek için, 2-4 ağacında kullanılan *bölme* işlemini aşağıdaki gibi gerçekleştirin:

Let  $u$  bölünmesi gereken düğümü belirtir ve  $m$ , düğümdeki medyan anahtarıdır. Yeni bir düğüm oluşturun ve  $m$ 'den büyük tüm anahtarları bu yeni düğüme taşıyın.  $m$ 'yi  $u$ 'nın ebeveyn düğümüne ekleyin. Artık  $u$ ,  $m$ 'nin sol çocuğu ve  $v$ ,  $m$ 'nin sağ çocuğu olur, Şekil'de gösterildiği gibi.

47.18. Eğer  $m$ 'yi  $u$ 'nın ebeveyn düğümüne eklemek bir taşma yaratıyorsa, ebeveyn  $d$  düğümünde aynı bölme işlemini tekrarlayın.



**Şekil 47.18**

(a)  $u$  düğümüne yeni bir anahtar eklendikten sonra. (b) Medyan anahtar  $k_p$ , parent Ofu'ya eklenir. \_\_\_\_\_

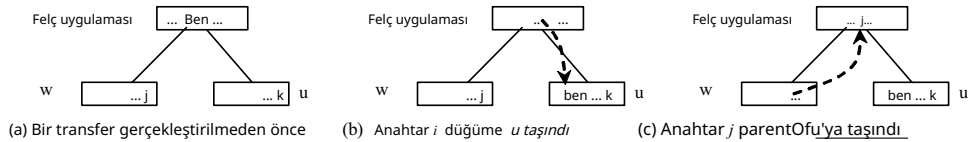
**<kenar notu: silme>**

Bir B-ağacından bir anahtar  $k$ , 2-4 ağacındaki gibi silinebilir. Öncelikle anahtarı içeren  $u$  düğümünü bulun. İki durumu göz önünde bulundurun:

Durum 1: Eğer  $u$  bir yaprak düğümse,  $u$ 'den anahtarı kaldırın. Kaldırdıktan sonra, eğer  $u$ 'den anahtar  $k$  silinirse, alt akış meydana gelir. Bir alt akış düzeltmek için, daha fazla  $w$  olan bir kardeş  $u$  ile bir transfer gerçekleştirin. Eğer böyle bir kardeş varsa, Şekil 47.19'da gösterildiği gibi.

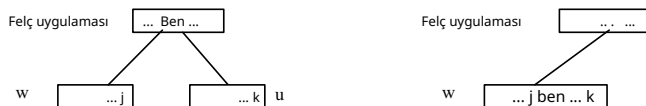
Aksi takdirde, Şekil

47.20'de gösterildiği gibi,  $u$ 'nin bir kardeşi  $w$  ile birleştirilmesi gerçekleştirilir.



**Şekil 47.19**

Transfer işlemi, bir anahtarı parentOfu'dan  $u$ 'ya ve bir anahtarı  $u$ 'nın kardeşi parentOfu'ndan aktarır.



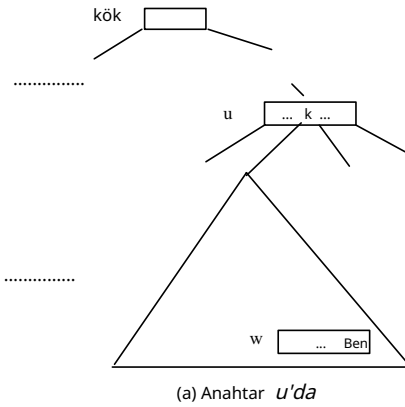
(a) Bir füzyon gerçekleştirilmeden önce

(b) Bir füzyon gerçekleştirildikten sonra

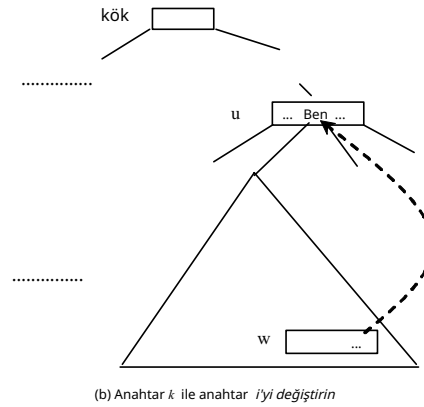
### Şekil 47.20

Füzyon işlemi, anahtar  $i$ 'yi  $parentOfu$ 'dan  $w$ 'ye taşıyor ve  $u$ 'daki tüm anahtarları  $w$ 'ye taşıyor.

Durum 2:  $u$  bir yaprak olmayan düğümdür.  $k$ 'nın sol alt ağacındaki en sağ yaprak düğümü bulun. Bu düğüm  $w$  olsun, Şekil 47.21(a)'de gösterildiği gibi.  $w$ 'deki son anahtarı  $k$ 'nın yerine  $u$ 'da taşıyın, Şekil 47.21(b)'de gösterildiği gibi. Eğer  $w$  alt akış (underflow) durumu olursa,  $w$  üzerinde bir transfer veya füzyon işlemi uygulayın.



(a) Anahtar  $u$ 'da



(b) Anahtar  $k$  ile anahtar  $i$ 'yi değiştirin

### Şekil 47.21

Bir iç düğümdeki anahtar, bir yaprak düğümdeki bir öge ile değiştirilir.

#### <kenar notu: B-ağaç performansı>

B-ağacının performansı, disk IO'larının sayısına (yani, erişilen düğüm sayısına) bağlıdır. Arama, ekleme ve silme işlemleri için erişilen düğüm sayısı, ağacın yüksekliğine bağlıdır.

En kötü durumda, her düğüm  $aoao$   $12/d$  anahtar içerir. Bu nedenle, ağacın yüksekliği  $ao^{tntdtlog2}$  burada  $n$  anahtar sayısını temsil eder. En iyi durumda, her düğüm  $tdt1$  anahtar içerir. Bu nedenle, ağacın yüksekliği  $dtlognt.dlognt$ . On milyon anahtar için 12. dereceden bir B-ağacını düşünün. Ağacın yüksekliği  $log000,000,10t6 \approx 7$  ve  $log000,000,10t12 \approx 9$ . Bu nedenle, arama, ekleme ve silme işlemleri için ziyaret edilen maksimum düğüm sayısı 47'dir. Eğer bir AVL ağacı kullanırsanız, ziyaret edilen maksimum düğüm sayısı  $log000,000,10t2 \approx 24$ .

#### Anahtar Terimler

- 2-3-4 ağacı
- 2-4 ağacı
- 2-düğüm
- 3-düğüm
- 4-düğüm
- B-ağaç
- birleştirme işlemi
- bölme işlemi

- aktarma işlemi

## Bölüm Özeti

1. Bir 2-4 ağacı tamamen dengeli bir arama ağacıdır. Bir 2-4 ağacında, bir düğüm bir, iki veya üç öge içerebilir.
2. Bir 2-4 ağacında bir öge aramak, bir binary ağaçta bir öge aramaya benzer. Fark, bir düğüm içinde bir öge aramış olmanızdır.
3. Bir ögeyi 2-4 ağacına eklemek için, ögenin ekleneceği bir yaprak düğüm bulmanızdır. Eğer yaprak düğüm bir 2- veya 3-düğümse, ögeyi düğüme basitçe ekleyin. Eğer düğüm bir 4-düğümse, düğümü bölün.
4. Bir 2-4 ağacından bir ögeyi silme işlemi, bir binary ağaçtan bir öge silme işlemiyle benzerdir. Fark, boş düğümler için aktarma veya birleştirme işlemleri yapmanız gerektiğidir.
5. Bir 2-4 ağacının yüksekliği  $O(\log n)$  şeklindedir. Bu nedenle, arama, ekleme ve silme yöntemlerinin zaman karmaşıklıkları  $O(\log n)$  şeklindedir.
6. Bir B-ağaç, 2-4 ağacının genelleştirilmiş halidir. Bir B-ağacının  $d$  derecesindeki her düğüm  $aoao$   $12/-$   $d$  ve  $tdt1$  anahtarlar hariç kök. 2-4 ağaçları AVL ağaçlarından daha düz ve B-ağaçları 2-4 ağaçlarından daha düzdür. B-ağaçları, büyük miktarda verinin disklerde saklandığı veritabanı sistemlerinde veri için indeksler oluşturmak için etkilidir.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

### Gözden Geçirme Soruları

#### Bölümler 47.1-47.2

47.1

2-4 ağacı nedir? 2-düğüm, 3-düğüm ve 4-düğüm nedir?

47.2

Tree24 sınıfındaki veri alanlarını ve \_\_\_\_\_

Tree24Node sınıfındaki veri alanlarını tanımlayın.

47.3

Yüksekliği 5 olan bir 2-4 ağacındaki minimum eleman sayısı nedir? Yüksekliği 5 olan bir 2-4 ağacındaki maksimum eleman sayısı nedir?

#### Bölümler 47.3-47.5

47.4

2-4 ağacında bir elemanı nasıl ararsınız?

47.5

2-4 ağacına bir elemanı nasıl eklersiniz?

47.6

2-4 ağacından bir elemanı nasıl silersiniz?

47.7

1, 2, 3, 4, 10, 9, 7, 5,

8, 6'yı bu sırayla eklerken bir 2-4 ağacının değişimini gösterin.

47.8

Aşağıdaki soruda oluşturulan ağaç için, 1, 2, 3, 4, 10, 9, 7, 5, 8, 6'nın bu sırayla silinmesinden sonra ağacın değişimini gösterin.

47.9

1, 2, 3, 4, 10 eklenirken 6. dereceden bir B-ağacının değişimini gösterin, 9, 7, 5, 8, 6, 17, 25, 18, 26, 14, 52, 63, 74, 80, 19, 27 bunu, bu sırayla.

47.10

Aşağıdaki soruda oluşturulan ağaç için, 1, 2, 3, 4, 10, 9, 7, 5, 8, 6'nın bu sırayla silinmesinden sonra ağacın değişimini gösterin.

## Programlama Alıştırmaları

47.1\*

(Sıralı yürütme) Tree24'teki sıralı yöntem bir

çalışma olarak bırakılmıştır. Bunu uygulayın.

47.2P

egzersiz. Uygula.

47.3It

alıştırma olarak bırakılmıştır. Bunu, inorder kullanarak elemanları yinelemek için uygulayın.

47.4\*

(2-4 ağacını grafiksel olarak göster) 2-4

ağacını gösteren bir applet yazın.

47.5\*\*\*

(2-4 ağaç animasyonu) Şekil 47.4'te gösterildiği gibi, 2-4 ağaç ekleme, silme ve arama yöntemlerini animasyonlu olarak gösteren bir Java applet'i yazın.

47.6\*\*

(Tree24 için ebeveyn referansı) Aşağıda gösterildiği gibi, bir düğümün ebeveynine bir referans eklemek için Tree24Node'u yeniden tanımlayın:

Tree24Node<E>
elemanlar: ArrayList<E>
çocuk: ArrayList<Tree24Node<E>>
ebeveyn: Tree24Node<E>
+Tree24()
n+Tree24(o: E)

Elemanları depolamak için bir dizi listesi.

Çocuk düğümlere bağlantıları depolamak için bir dizi listesi.

Bu düğümün ebeveynine atıfta bulunur.

Boş bir ağaç düğümü oluşturur.

Başlangıç elemanı ile bir ağaç düğümü oluşturur.

Aşağıdaki iki yeni yöntemi Tree24'e ekleyin:

```
public Tree24Node<E> getParent(Tree24Node<E> node)
```

Belirtilen düğüm için ebeveyni döndürür.

```
public ArrayList<Tree24Node<E>> getPath(Tree24Node<E> node)
```

Belirtilen düğümden köke kadar olan yolu bir dizi

listesinde döndürür.

1, 2, ..., 100 sayısını ağaca ekleyen ve

tüm yaprak düğümleri için yolları gösteren bir test programı yazın.

47.7\*\*\*

(BTree sınıfı) B-ağaçları için bir sınıf tasarlayın ve uygulayın.



\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 48

### Kırmızı-Siyah Ağaçlar

#### Hedefler

- Kırmızı-siyah ağacın ne olduğunu bilmek (§48.1).
- Kırmızı-siyah ağacı 2-4 ağacına ve tersine dönüştürmek (§48.2) .
- RBTre sınıfını BinaryTree sınıfını genişletecek şekilde tasarlamak için (§48.3) .
- Kırmızı-siyah ağaçta bir eleman eklemek ve gerekli ise double-red ihlali için (§48.4).
- Kırmızı-siyah ağaçtan bir eleman silmek ve gerekli ise double-black sorununu çözmek için (§48.5).
- RBTre sınıfını uygulamak ve test etmek için (§§48.6-48.7).
- AVL ağaçları, 2-4 ağaçları ve RBTre'nin performansını karşılaştırmak için (§48.8) .

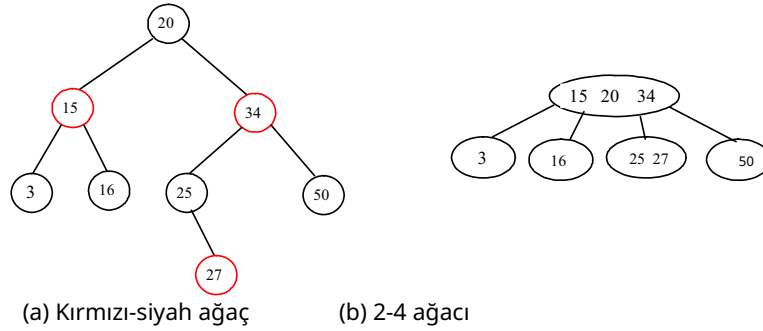
#### 48.1 Giriş

<kenar notu: 2-4'ten türetilmiştir>  $n < k$

enar notu: renk özelliği>  $n < \text{kenar no}$   
tu: harici>

<kenar notu: siyah derinliği>

Kırmızı-siyah ağaç, bir 2-4 ağaç'tan türetilmiş bir ikili arama ağacıdır. Kırmızı-siyah ağaç, bir 2-4 ağacına karşılık gelir. Kırmızı-siyah ağaçtaki her düğümün bir renk niteliği vardır: kırmızı veya siyah, Şekil 48.1(a)'de gösterildiği gibi. Bir düğüm dışsal olarak kabul edilir, ancak bir dışsal düğüm mutlaka bir yaprak düğüm değildir. Örneğin, düğüm 25 dışsaldır, ancak bir yaprak değildir. Bir siyah derinlik, bir düğümden k öke giden yoldaki siyah düğümlerin sayısı olarak tanımlanır. Örneğin, düğüm 25'in siyah derinliği 2'dir ve düğüm 27'nin siyah derinliği de 2'dir.



Şekil 48.1

Kırmızı-siyah ağaç, 2-4 ağacı kullanılarak temsil edilebilir ve tersine.

NOT: Kırmızı düğümler metinde mavi olarak görünmektedir.

Kırmızı-siyah ağacın aşağıdaki özellikleri vardır:

1. Kök siyah renktedir.
2. İki bitişik düğüm aynı anda kırmızı olamaz.
3. Tüm dış düğümlerin aynı siyah derinliği vardır.

Şekil 48.1(a)'deki kırmızı-siyah ağaç, bu üç özelliği de karşılamaktadır. Bir kırmızı-siyah ağaç, 2-4 ağacına dönüştürülebilir ve tersine. Şekil 48.1(b), Şekil 48.1(a)'deki kırmızı-siyah ağaç için eşdeğer bir 2-4 ağacını göstermektedir.

#### 48.2 Kırmızı-Siyah Ağaçlar ve 2-4 Ağaçları Arasındaki Dönüşüm

Kırmızı-siyah ağaçlar için ekleme ve silme algoritmaları tasarlayabilirsiniz, ancak 2-4 ağaçları hakkında bilgi sahibi olmanıza gerek yoktur. Ancak, kırmızı-siyah ağaçlar ile 2-4 ağaçları arasındaki ilişki, kırmızı-siyah ağaçların yapısı ve işlemleri hakkında faydalı bir sezgi sağlar. Bu nedenle, bu bölüm bu iki ağaç türü arasındaki ilişkiyi tartışmaktadır.

<margin note: kırmızı-siyah 2-4'e

Kırmızı-siyah bir ağacı 2-4 ağacına dönüştürmek için, her kırmızı düğümü annesiyle birleştirerek 3-düğüm veya 4-düğüm oluşturmak yeterlidir. Örneğin, kırmızı düğümler 15 ve 34, 4-düğüm oluşturmak için anneleriyle birleştirilir ve

kırmızı düğüm 27, 3-düğüm oluşturmak için annesiyle birleştirilir; bu, Şekil 48.1(b)'de gösterilmiştir.

**<kenar notu: 2-4 kırmızı-siyah>**

2-4 ağacını kırmızı-siyah ağacına dönüştürmek için, her düğüm için aşağıdaki dönüşümleri gerçekleştirin  $u$  :

**<kenar notu: 2 düğüm dönüştürme>**

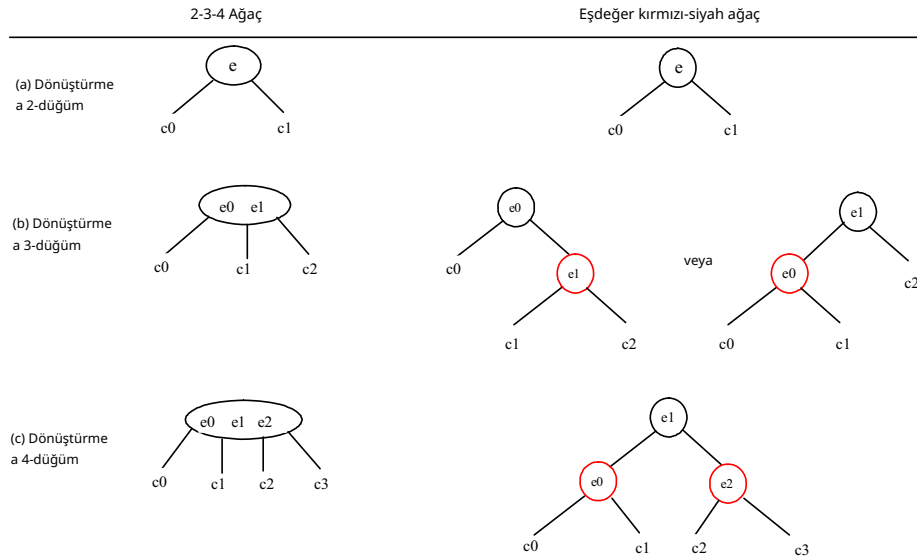
1. Eğer  $u$  2 düğümse, siyah renge boyayın, Şekil 48.2(a)'de gösterildiği gibi.

**<kenar notu: 3 düğümü dönüştürme>**

2. Eğer  $u$  3 düğümlü bir yapı ise, eleman değerleri  $e_0$  ve  $e_1$ , bunu dönüştürmenin iki yolu vardır. Ya  $e_0$  ebeveyn  $e_1$  veya yap  $e_1$  ebeveyn  $e_0$ . Her durumda, ebeveynin rengini siyah ve çocuğun rengini kırmızı yapın, Şekil 48.2(b)'de gösterildiği gibi.

**<kenar notu: 4 düğüm dönüştürme>**

3. Eğer  $u$  4 düğümlü bir yapı ise ve eleman değerleri  $e_0, e_1$ , ve  $e_2$  yapmak  $e_1$  ebeveyn  $e_0$  ve  $e_2$ . Renk  $e_1$  siyah ve  $e_0$  ve  $e_2$  kırmızı, Şekil 48.2(c)'de gösterildiği gibi.



**Şekil 48.2**

*Bir 2-4 ağacındaki bir düğüm, bir kırmızı-siyah ağacındaki düğümlere dönüştürülebilir.*

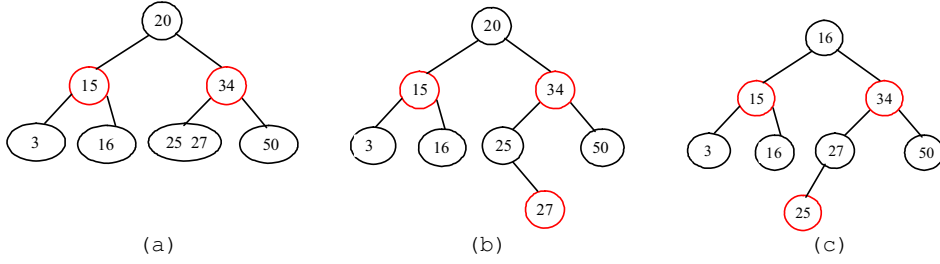
**<kenar notu: benzersiz değil>**

Şekil 48.1(b) için 2-4 ağacındaki dönüşümü uygulayalım.

4-düğüm dönüştürüldükten sonra, ağaç Şekil 48.3(a) olarak gösterilmiştir.

3-düğüm dönüştürüldükten sonra, ağaç Şekil 48.3(b)'de gösterildiği gibidir.

3-düğüm için dönüşümün benzersiz olmadığını unutmayın. Bu nedenle, 2-4 ağacından kırmızı-siyah ağacına dönüşüm *benzersiz değildir*. 3-düğüm dönüştürüldükten sonra, ağaç ayrıca Şekil 48.3(c)'de gösterildiği gibi olabilir.



**Şekil 48.3**

*2-4 ağacından kırmızı-siyah ağacına dönüşüm benzersiz değildir.*

Dönüşümün, üç özelliği de karşılayan bir kırmızı-siyah ağacı ürettiğini kanıtlayab ilirsiniz.

**<kenar notu: Özellik 1 kanıtı>**

Özellik 1. Kök siyahtır.

Kanıt: Eğer bir 2-4 ağacının kökü bir 2-düğümse, kırmızı-siyah ağacının kökü siyahtır. Eğer bir 2-4 ağacının kökü bir 3-düğüm veya 4-düğümse, dönüşüm kökünde siyah bir ebeveyn üretir.

**<kenar notu: Özellik 2 kanıtı>**

Özellik 2. İki bitişik düğüm aynı anda kırmızı olamaz.

Kanıt: Kırmızı bir düğümün ebeveyni her zaman siyah olduğundan, yan yana olan iki düğüm de kırmızı olamaz.

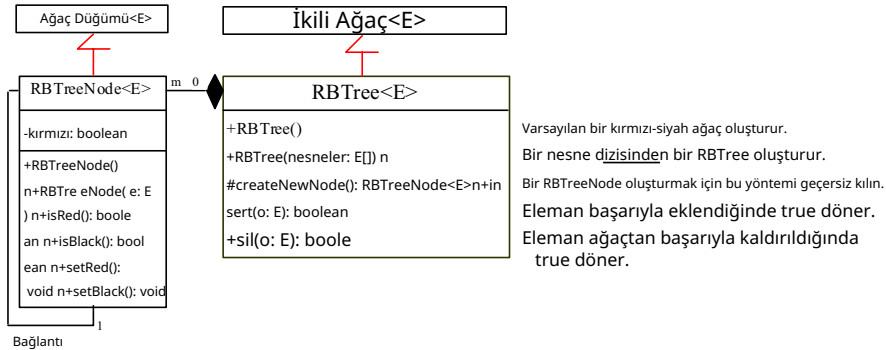
**<kenar notu: Özellik 3 kanıtı>**

Özellik 3. Tüm dış düğümler aynı siyah derinliğe sahiptir.

Kanıt: Bir 2-4 ağacındaki bir düğümü kırmızı-siyah ağaç düğümlerine dönüştürdüğünüzde, bir siyah düğüm ve sıfır, bir veya iki kırmızı düğüm elde edersiniz; bu, orijinal düğümün 2-, 3- veya 4-düğüm olup olmamasına bağlıdır. Sadece bir yaprak 2-4 düğümü dış kırmızı-siyah düğümler üretebilir. Bir 2-4 ağacı mükemmel bir şekilde dengelidir, bu nedenle kökten bir dış düğüme kadar olan herhangi bir yoldaki siyah düğüm sayısı aynıdır.

### 48.3 Kırmızı-Siyah Ağaçlar için Sınıfların Tasarımı

Bir kırmızı-siyah ağaç, ikili arama ağacıdır. Bu nedenle, `RBTree` sınıfını `BinaryTree` sınıfını genişletecek şekilde tanımlayabilirsiniz; bu, Şekil 48.4'te gösterilmiştir. `BinaryTree` ve `TreeNode` sınıfları §26.2.5'te tanımlanmıştır.

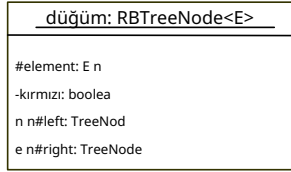


#### Şekil 48.4

*RBTree sınıfı, insert ve delete yöntemleri için yeni uygulamalar ile BinaryTree'yi genişletir.*

##### <kenar notu: RBTreeNode>

Kırmızı-siyah ağaçtaki her düğümün bir renk özelliği vardır. Renk ya kırmızı ya da siyah olmadığından, bunu belirtmek için boolean türünü kullanmak etkilidir. RBTreeNode sınıfı, renk özelliği ile BinaryTree.TreeNode'dan genişletilecek şekilde tanımlanabilir. Kolaylık olması açısından, renk kontrolü ve yeni bir renk ayarlama yöntemlerini de sağlıyoruz. TreeNode'nun BinaryTree içinde statik bir iç sınıf olarak tanımlandığını unutmayın. RBTreeNode, RBTree içinde statik bir iç sınıf olarak tanımlanacaktır. BinaryTreeNode'un, RBTreeNode'da miras alınan element, left ve right veri alanlarını içerdiğini unutmayın. Bu nedenle, RBTreeNode dört veri alanı içerir, Şekil 48.5'te gösterildiği gibi.



#### Şekil 48.5

*Bir RBTreeNode, element, kırmızı, sol ve sağ veri alanlarını içerir.*

##### <kenar notu: createNewNode()>

BinaryTree sınıfında, createNewNode() metodu bir TreeNode nesnesi oluşturur. Bu metod, RBTree sınıfında bir RBTreeNode oluşturmak için geçersiz kılınmıştır. createNewNode() metodunun dönüş türünün BinaryTree sınıfında TreeNode olduğunu, ancak RBTree sınıfındaki createNewNode() metodunun dönüş türünün RBTreeNode olduğunu unutmayın. Bu sorun değil, çünkü RBTreeNode, TreeNode'un bir alt türüdür.

Bir kırmızı-siyah ağaçta bir öge aramak, bir normal ikili arama ağacında aramakla aynıdır. Bu nedenle, BinaryTree sınıfında tanımlanan arama yöntemi RBTree için de çalışır.

Ögeyi eklemek ve silmek için insert ve delete yöntemleri geçersiz kılınmıştır ve eğer gerekirse, kırmızı-siyah ağacın üç özelliğinin gözetilmesini sağlamak için renklendirme ve yeniden yapılandırma işlemleri gerçekleştirilir.

Eğitim NOTU

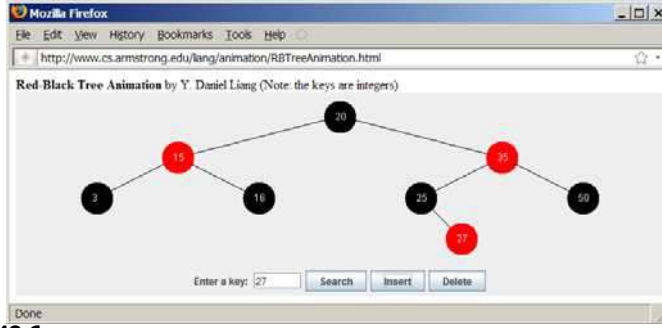
##### <side remark: Kırmızı-Siyah ağaç animasyonu>

Çalıştırın

[www.cs.armstrong.edu/liang/animation/RBTreeAnimation.html](http://www.cs.armstrong.edu/liang/animation/RBTreeAnimation.html)

kırmızı-siyah ağacın nasıl çalıştığını görmek için, Şekil'de gösterildiği gibi

48.6.



**Şekil 48.6**

Animasyon aracı, bir kırmızı-siyah ağaçta öğeleri görsel olarak eklemeni zi, silmenizi ve aramanızı sağlar.

\*\*\*Son NOT

#### 48.4 insert Yöntemini Geçersiz Kılma

<margin note: çift kırmızı>

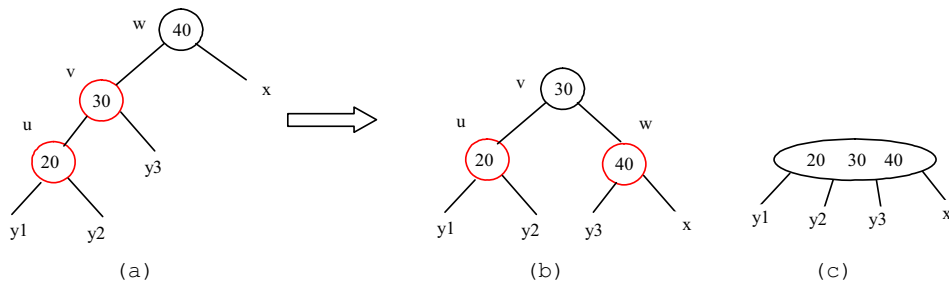
Yeni bir öge her zaman bir yaprak düğüm olarak eklenir. Eğer yeni düğüm kökse, siyah renkte boyayın. Aksi takdirde, kırmızı renkte boyayın. Eğer yeni düğümün ebeveyni kırmızıysa, bu kırmızı-siyah ağacın 2. Özelliğini ihlal eder. Buna çift-kırmızı ihlali diyoruz.

Yeni düğümü  $u$ ,  $v$  ebeveyni,  $w$   $v$ 'nin ebeveyni ve  $x$   $v$ 'nin kardeşi olarak tanımlayalım. Çift-kırmızı ihlalini düzeltmek için, iki durumu göz önünde bulundurun:

Durum 1:  $x$  siyah veya  $x$  null'dır.  $u$ ,  $v$ ,  $w$  ve  $x$  için dört olası konfigürasyon vardır; bu, Şekil 48.7(a), 48.8(a), 48.9(a) ve 48.10(a)'da gösterilmiştir. Bu durumda,  $u$ ,  $v$  ve  $w$ , ilgili 2-4 ağacında bir 4-düğüm oluşturur; bu, Şekil 48.7(c)'de gösterilmiştir.

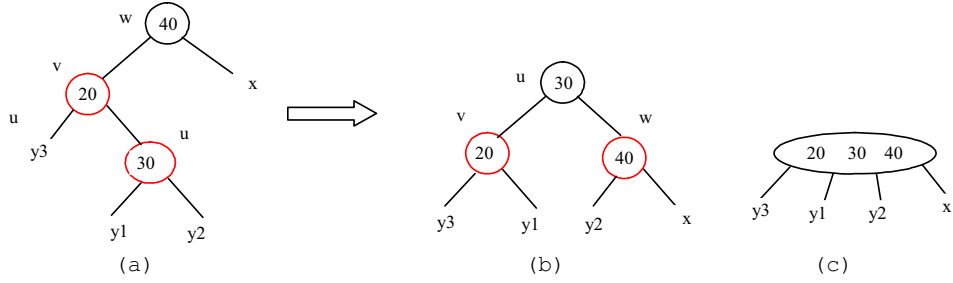
48.8(c), 48.9(c) ve 48.10(c), ancak kırmızı-siyah ağaçta yanlış temsil edilmektedir. Bu hatayı düzeltmek için, üç nodetu,  $v$  ve  $w$ 'yi, Şekil 48.7(b), 48.8(b), 48.9(b) ve

48.10(b)'de gösterildiği gibi yeniden yapılandırın ve renklendirin.  $x$ 'in,  $y_1$ ,  $y_2$  ve  $y_3$  null olabilir.



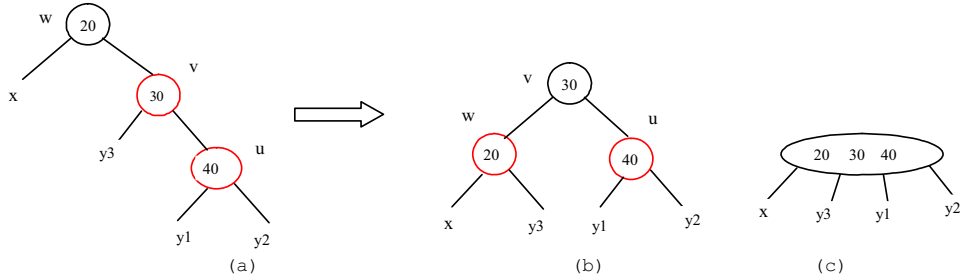
**Şekil 48.7**

Durum 1.1:  $u < v < w$ .



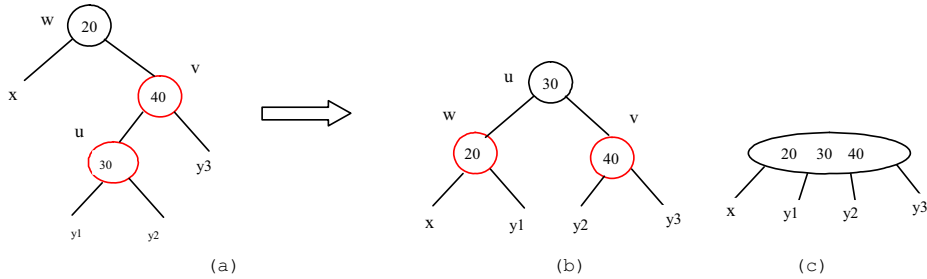
**Şekil 48.8**

*Durum 1.2:  $v < u < w$*



**Şekil 48.9**

*Durum 1.3:  $w < v < u$*

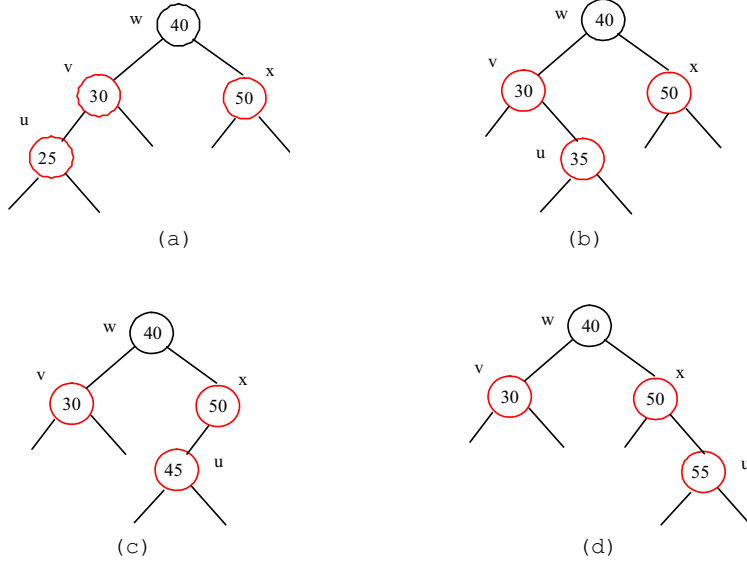


**Şekil 48.10**

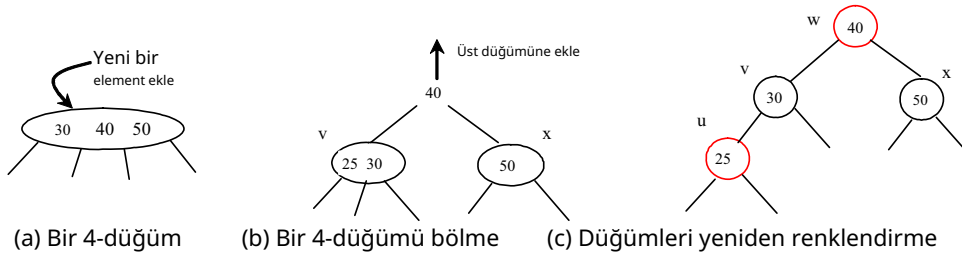
*Durum 1.4:  $w < u < v$*

Durum 2:  $x$  kırmızı.  $u$ ,  $v$ ,  $w$  ve  $x$  için dört olası yapılandırma vardır; bunlar Şekil 48.11(a), 48.11(b), 48.11(c) ve 48.11(d)'de gösterilmiştir.

Bu yapılandırmaların tümü, Şekil 48.12(a)'de gösterildiği gibi, 2-4 ağacındaki karşılık gelen 4-düğümde bir taşma durumuna karşılık gelir. Taşma sorununu düzeltmek için, Şekil 48.12(b)'de gösterildiği gibi, bir bölme işlemi gerçekleştirilir. Kırmızı-siyah ağaçtaki sorununu düzeltmek için eşdeğer bir yeniden renklendirme işlemi gerçekleştiririz.  $w$  ve  $u$ 'yu kırmızı,  $w$ 'nin iki çocuğunu siyah renklendir.  $u$ 'nin  $v$ 'nin sol çocuğu olduğunu varsayalım, Şekil 48.11(a)'de gösterildiği gibi. Yeniden renklendirmeden sonra, düğümler Şekil 48.12(c)'de gösterilmektedir. Artık  $w$  kırmızı, eğer  $w$ 'nin ebeveyni siyah ise, çift-kırmızı ihlali düzeltilmiştir. Aksi takdirde,  $w$  düğümünde yeni bir çift-kırmızı ihlali meydana gelir. Çift-kırmızı ihlali ortadan kaldırmak için aynı işlemi  $w$  üzerinde, özyinelemeli olarak devam ettirmemiz gerekiyor.



**Şekil 48.11**  
Durum 2'nin dört olası yapılandırması vardır.



**Şekil 48.12**  
Bir 4-düğümün bölünmesi, kırmızı-siyah ağaçtaki düğümlerin yeniden renklendirilmesi ile karşılık gelir.

Bir öge eklemek için daha ayrıntılı bir algoritma  
Listing 48.1'de açıklanmıştır.

#### Liste 48.1 Kırmızı-Siyah Ağaç'a Bir Eleman Ekleme

<kenar notu satır 1: ağaca ekle>  
<kenar notu satır 2: super.insert'i çağır>  
<kenar notu satır 4: yinelenen eleman>  
<kenar notu satır 6: renk ve derinliği sağla>  
  
<kenar notu satır 13: renk ve derinliği sağla>  
<kenar notu satır 14: yolu al>  
<kenar notu satır 15: düğüm indeksi>  
<kenar notu satır 16: u, v'yi al>  
<kenar notu satır 20: u kök mü?> -  
<kenar notu satır 22: çift-kırmızı ihlali>  
  
<kenar notu satır 27: çift kırmızıyı düzelt>



<kenar notu satır 29: w'yi al>  
 <kenar notu satır 32: x'i al>  
 <kenar notu satır 36: Durum 1>  
 <kenar notu satır 38: Durum 1.1>  
 <kenar notu satır 41: Durum 1.2>  
 <kenar notu satır 44: Durum 1.3>  
 <kenar notu satır 47: Durum 1.4>  
 <kenar notu satır 50: Durum 2>  
 <kenar notu satır 51: yeniden renklendirme>  
 <kenar notu satır 54: w kök mü?>  
 <kenar notu satır 57: yukarı doğru yayıl>  
 <kenar notu satır 61: yeni çift kırmızıyı düzelt>

```

public boolean insert(E e) {
 boolean başarılı = super.insert(e);
 if (!başarılı)
 return false; // e zaten ağaçta
 else {
 sureRBTree(e);
 }

 return true; // e eklendi
}

/** Ağacın bir kırmızı-siyah ağaç olduğundan emin olun */
private void ensureRBTree(E e) {
 Kökten eleman e'ye giden yolu al.
 int i = path.size() - 1; // Yolda mevcut düğümün indeksi
 Yoldan u, v'yi al. u, e'yi içeren düğümdür ve v
 u'nun ebeveynidir.
 u'yu kırmızı yap;

 if (u == root) // Eğer e kök olarak eklenirse, kökü siyah yap
 u.setBlack();
 else if (v.isRed())
 fixDoubleRed(u, v, path, i); // u'daki çift-kırmızı ihlalini düzelt
}

/** u düğümündeki çift-kırmızı ihlalini düzelt */
özel void fixDoubleRed(RBTreeNode<E> u, RBTreeNode<E> v, ArrayList<
 RBTreeNode<E>> path, int i) {
 Yoldan w'yi al. w, u'nun büyük ebeveynidir.

 // v'nin kardeşi x'i al
 RBTreeNode<E> x = (w.left == v) ?
 (RBTreeNode<E>) (w.right) : (RBTreeNode<E>) (w.left);

 if (x == null || x.isBlack()) {
 // Durum 1: v'nin kardeşi x siyah
 if (w.left == v && v.left == u) {
 // Durum 1.1: u < v < w, Düğümleri yeniden yapılandır ve renklendir
 }
 else if (w.left == v && v.right == u) {
 // Durum 1.2: v < u < w, Düğümleri yeniden yapılandır ve renklendir
 }
 else if (w.right == v && v.right == u) {

```

```

 // Durum 1.3: $w < v < u$, D    mleri yeniden yapilandir ve renklendir
 }
 ba  ka {
 // Durum 1.4: $w < u < v$, D    mleri yeniden yapilandir ve renklendir
 }

 else { // Durum 2: v'nin karde  i x kırmızı
 w ve u'yu kırmızı yap
 w'nin iki   ocu  unu siyah yap.

 e  er (w k  kse) {
 w'yi siyah yap;
 }
 aksi takdirde e  er (w'nin ebeveyni kırmızıysa) {
 // Yeni   ift-kırmızı ihlalini d  zeltmek i  in yol boyunca yayıl
 u = w;
 v = w'nin ebeveyni;
 fixDoubleRed(u, v, path, i - 2); // i - 2 yukarıya yayılır
 }
 }
}
N)

```

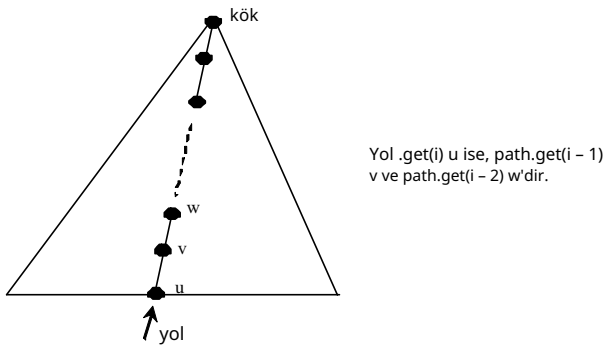
#### <kenar notu: ekle(E, e)>

insert(E e) metodu (satırlar 1-10),

BinaryTree sınıfındaki insert metodunu   a  ırarak eleman i  in yeni bir yaprak d    m   oluşturur (satır 2). E  er element zaten a  a  ta varsa, false d  nd  r (satır 4). Aksi takdirde, ensureRBTree(e) (satır 6) metodunu   a  ırarak a  acın kırmızı-siyah a  acın renk ve siyah derinlik   zelli  ini sa  ladı  ından emin ol.

#### <kenar notu: sureRBTree(E, e)>

ensureRBTree(E e) metodu (satırlar 13-24), k  kten e'ye giden yolu elde eder (satır 14),    ekil 48.13'te g  sterildi  i gibi. Bu yol, algoritmayı uygulamak i  in   nemli bir rol oynar. Bu yoldan, u ve v d    mlerini elde edersiniz (satırlar 16-17). E  er u k  kse, u'yu siyah yap (satırlar 20-21). E  er v kırmızıysa, u d    m  nde   ift-kırmızı ihlali meydana gelir. Sorun u d  zeltmek i  in fixDoubleRed metodunu   a  ır.



**  ekil 48.13**

*Yol, u'dan k  ke kadar olan d    mlerden olu  ur.*

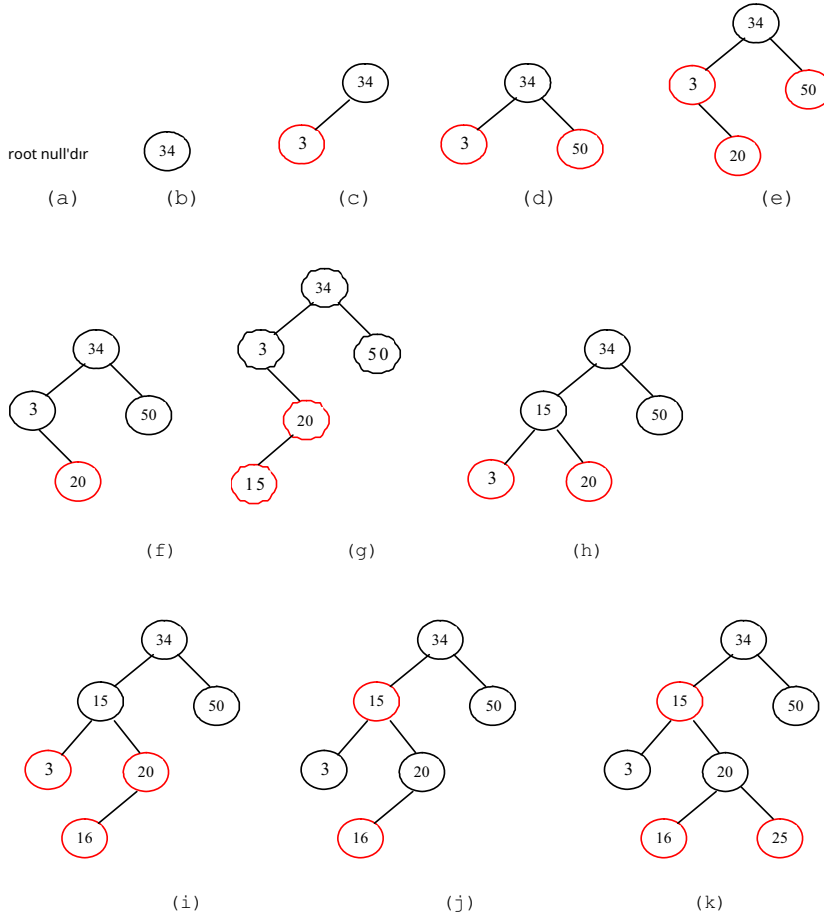
#### <kenar notu: fixDoubleRed>

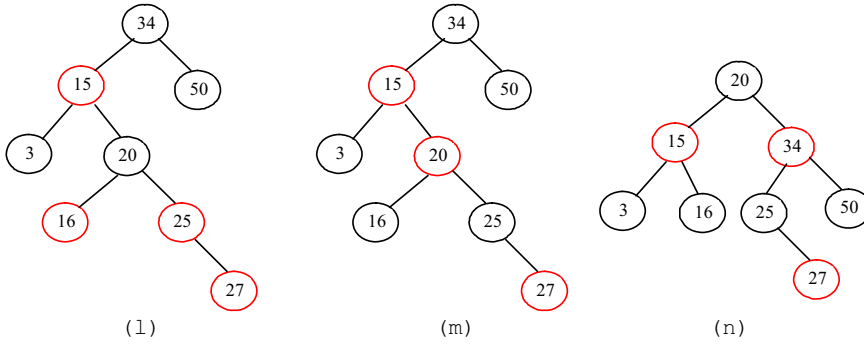
fixDoubleRed metodu (satırlar 27-63)   ift-kırmızı ihlalini d  zeltir. İlk olarak, yolu kullanarak w'yi (v'nin ebeveyni) alır (satır 29) ve x'yi (

v'nin kardeşi) (satırlar 32-33). Eğer x boşsa veya siyah bir düğümse, ve üç düğüm u, v ve w'yi yeniden yapılandır ve renklendirerek sorunu ortadan kaldır (satırlar 35-49). Eğer x kırmızı bir düğümse, düğümleri u, v, w ve x'i renklendir (satırlar 51-52). eğer w kökse, w'yi siyah yapın (satırlar 54-56). Eğer w'nin ebeveyni kırmızıysa, çift-kırmızı ihlali w'de tekrar ortaya çıkar. Sorunu çözmek için `fixDoubleRed`'i yeni u ve v ile çağırın (satır 61). Şimdi i - 2'nin yolda yeni u'ya işaret ettiğini unutmayın. Bu ayarlama, yolda yeni nodları w ve w'nin ebeveynini bulmak için gereklidir.

### <kenar notu: ekleme örneği>

Şekil 48.14, 34, 3, 50, 20, 15, 16, 25 ve 27 boş bir kırmızı-siyah ağaç içine. (d) içinde 20'yi ağaca eklerken, Durum 2, 3 ve 50'yi siyaha yeniden renklendirmek için geçerlidir. (g) içindeki ağaca 15 eklenirken, Durum 1.4, 15, 20 ve 3 düğümlerini yeniden yapılandırmak ve yeniden renklendirmek için geçerlidir. (i) içindeki ağaca 16 eklenirken, Durum 2, 3 ve 20 düğümlerini siyaha ve 15 ve 16 düğümlerini kırmızıya yeniden renklendirmek için geçerlidir. (l) içindeki ağaca 27 eklenirken, Durum 2, 16 ve 25 düğümlerini siyaha ve 20 ve 27 düğümlerini kırmızıya yeniden renklendirmek için geçerlidir. Şimdi, düğümde yeni bir çift-kırmızı problemi ortaya çıkmaktadır. 20. Düğüm yapılarını yeniden yapılandırmak ve renklendirmek için Durum 1.2'yi uygulayın. Yeni ağaç (n) için de gösterilmektedir.





**Şekil 48.14**

*Kırmızı-siyah ağaç içine ekleme: (a) başlangıçta boş ağaç; (b) ekleme*

*34; (c) 3 ekleme; (d) 50 ekleme; (e) 20 ekleme, çift*

*kırmızıya neden olur; (f) yeniden renklendirmeden sonra (Durum 2); (g) 15 ekleme, çift*

*kırmızıya neden olur; (h) yeniden yapılandırma ve yeniden renklendirmeden sonra (Durum 1.4); (i) 16 ekleme*

*çift kırmızıya neden olur; (j) yeniden renklendirmeden sonra (Durum 2); (k) 25 ekleme;*

*(l) 27 ekleme, 27'de çift kırmızıya neden olur; (m) 20'de çift kırmızı*

*yeniden renklendirmeden sonra tekrar ortaya çıkar (Durum 2); (n) yeniden yapılandırma ve*

*yeniden renklendirmeden sonra (Durum 1.2).*

#### 48.5 Silme Yöntemini Geçersiz Kılma

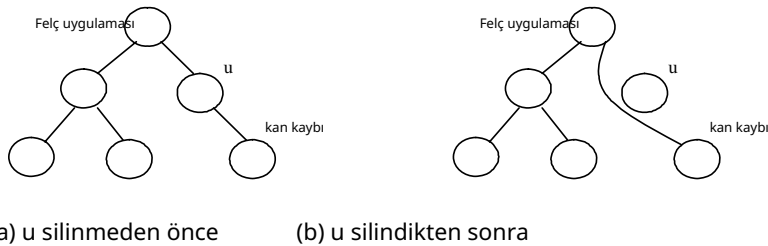
Bir kırmızı-siyah ağaçtan bir ögeyi silmek için, önce ögeyi ağaçta arayın ve ögeyi içeren düğümü bulun. Eğer öge ağaçta yoksa, yöntem false döner.  $u$ , ögeyi içeren düğüm olsun. Eğer  $u$ , hem sol hem de sağ çocukları olan bir iç düğümse,  $u$ 'nun sol alt ağacındaki en sağdaki düğümü bulun.  $u$ 'deki ögeyi en sağdaki düğümdeki öge ile değiştirin. Şimdi yalnızca dış düğümlerin silinmesini dikkate alacağız.

Let  $u$  dışarıdan silinecek bir düğüm olsun.  $u$  dış düğüm olduğundan,

it en fazla bir çocuğa sahiptir, bu da *childOfu* ile gösterilir. *childOfu* null olabilir.

*parentOfu*,  $u$ 'nun ebeveynini belirtir, Şekil 48.15(a)'de gösterildiği gibi.

$u$ 'yu silmek için *childOfu*'yu *parentOfu* ile bağlayın, Şekil 48.15(b)'de gösterildiği gibi.



(a)  $u$  silinmeden önce

(b)  $u$  silindikten sonra

**Şekil 48.15**

*$u$  dış bir düğümdür ve *childOfu* null olabilir.*

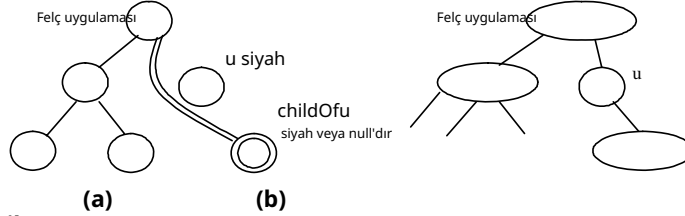
Aşağıdaki durumu düşünün:

- If  $u$  is red, we are done.

- Eğer  $u$  siyah ve  $childOfu$  kırmızı ise,  $childOfu$ 'yu siyah renge boyayın,  $childOfu$  için siyah yüksekliği korumak amacıyla.

**<kenar notu: çift siyah>**

- Aksi takdirde,  $childOfu$ 'ya hayali bir *double black* atayın, Şekil 48.16(a)'de gösterildiği gibi. Buna *double-black problem* diyoruz, bu da siyah derinliğin 1 eksik olduğunu gösterir, bu da bir siyah düğüm  $u$ 'yu silmekten kaynaklanmaktadır.



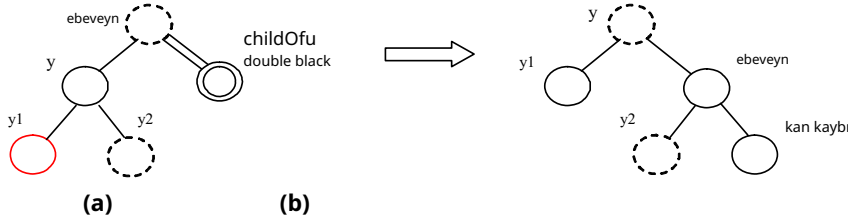
**Şekil 48.16**

(a)  $childOfu$  çift siyah olarak gösterilir. (b)  $u$ , bir 2-4 ağacında boş bir düğümü temsil eder.

Bir kırmızı-siyah ağaçtaki çift siyah, karşılık gelen 2-4 ağacındaki  $u$  için boş bir düğüm karşılık gelir (yani, alt akış durumu), Şekil 48.16(b)'de gösterildiği gibi. Çift siyah sorunun  $u$  çözmek için, eşdeğer transfer ve birleşim işlemleri gerçekleştireceğiz. Üç durumu göz önünde bulundurun:

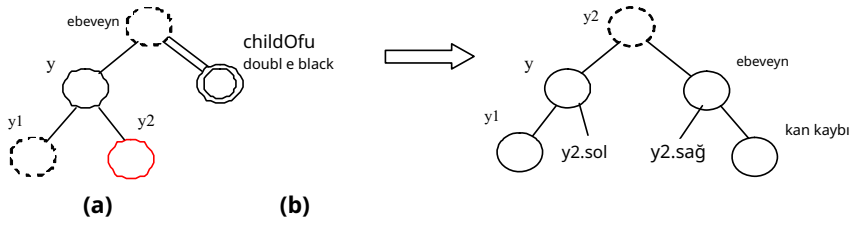
**<margin note: Durum 1>**

Durum 1:  $childOfu$ 'nun kardeşi  $y$  siyahtır ve kırmızı bir çocuğa sahiptir. Bu durumun, Şekil 48.17(a), 48.18(a), 48.19(a) ve 48.20(a)'de gösterildiği gibi dört olası konfigürasyonu vardır. Kesik daire, düğümün ya kırmızı ya da siyah olduğunu belirtir. Çift siyah sorunun ortadan kaldırmak için, düğümleri yeniden yapılandırın ve renklendirin, Şekil 48.17(b), 48.18(b), 48.19(b) ve 48.20(b)'de gösterildiği gibi.



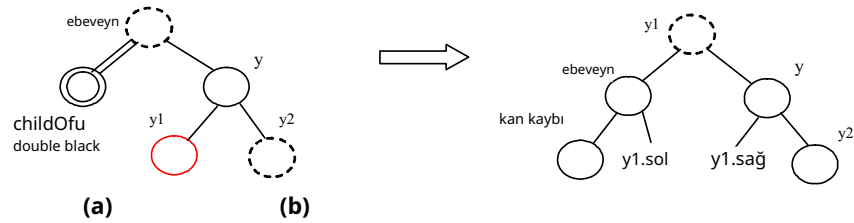
**Şekil 48.17**

Durum 1.1:  $childOfu$ 'nun kardeşi  $y$ , siyahtır ve  $y1$  kırmızıdır.



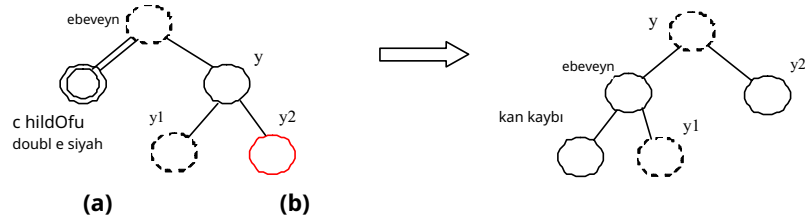
**Şekil 48.18**

Durum 1.2: childOfu'nun kardeşi y siyah ve y2 kırmızı.



**Şekil 48.19**

Durum 1.3: childOfu'nun kardeşi y siyah ve y1 kırmızı.



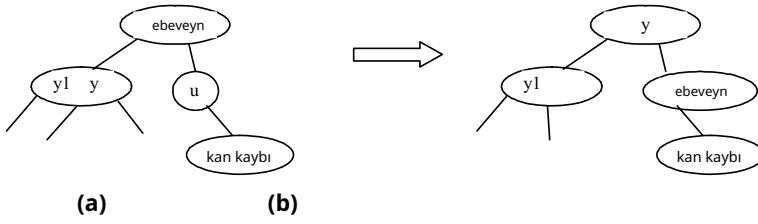
**Şekil 48.20**

Durum 1.4: childOfu'nun kardeşi y siyah ve y2 kırmızı.

Not

<kenar notu: transfer işlemi>

Durum 1, 2-4 ağaçlarındaki bir *transfer* işlemi ile ilişkilidir. Örneği n, Şekil 48.17(a) için karşılık gelen 2-4 ağacı Şekil 48.21(a)'de gösterilmiştir ve bir transfer işlemi ile 48.21(b)'ye dönüştürülmüştür.



**Şekil 48.21**

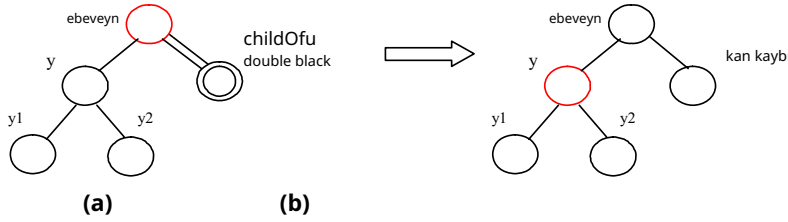
Durum 1, karşılık gelen 2-4 ağaçlarındaki bir transfer işlemi ile ilişkilidir.

### \*\*\*NOTUN SONU

<marginal not: Durum 2>

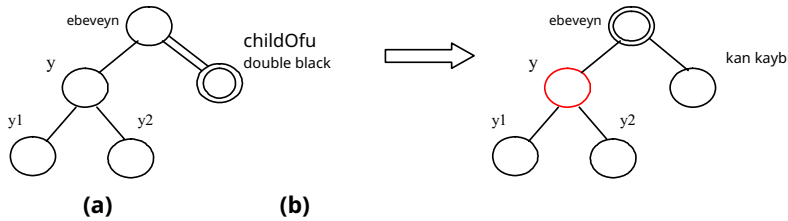
<marginal not: yayılma>

Durum 2: *childOfu*un kardeşi *y* siyah ve çocukları siyah veya null'dır. Bu durumda, *y*'nin rengini kırmızıya değiştirin. Eğer *ebeveyn* kırmızı ise, onu siyah yapın ve işimiz bitti, Şekil 48.22'de gösterildiği gibi. Eğer *ebeveyn* siyah ise, *ebeveyn*i çift siyah olarak adlandırıyoruz, Şekil 48.23'te gösterildiği gibi. Çift-siyah problemi *yayılır* *ebeveyn* düğümüne.



**Şekil 48.22**

Durum 2: Renk değiştirme, *ebeveyn* kırmızı ise çift-siyah problemini ortadan kaldırır.



**Şekil 48.23**

Durum 2: Yeniden renklendirme, *ebeveyn* siyah ise çift siyah sorununu yayar.\_\_\_\_\_

Not

<kenar notu: sol childOfu>

Şekil 48.22 ve 48.22, childOfu'nun ebeveynin sağ çocuğu olduğun u göstermektedir. Eğer childOfu ebeveynin sol çocuğu ise, yeniden r enklendirme aynı şekilde gerçekleştirilir.

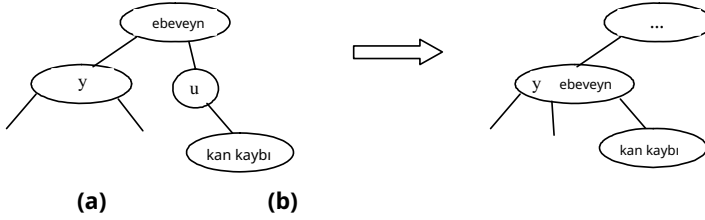
Not

<margin note: füzyon işlemi>

Durum 2, 2-4 ağacında bir füzyon işlemi ile ilişkilidir.

Örneğin, Şekil

48.22(a) için karşılık gelen 2-4 ağacı Şekil 48.24(a)'de gösterilmiştir ve bir füzyon işlemi ile 48.24(b)'ye dönüştürülmüştür.



Şekil 48.24

Durum 2, karşılık gelen 2-4 ağacında bir füzyon işlemi ile ilişkilidir.

\*\*\*NOTUN SONU

<margin note: Durum 3>

<margin note: ayarlama>

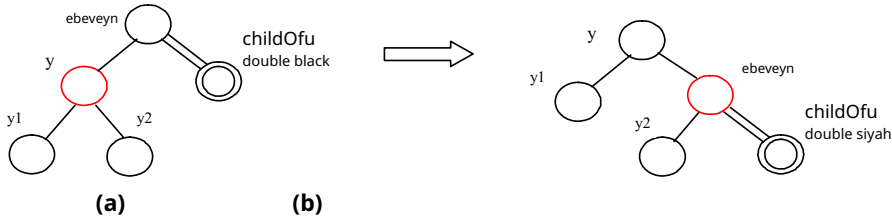
Durum 3: *childOfu*nun kardeşi y kırmızıdır. Bu durumda, bir ayarlama işlemi gerç ekleştirin. Eğer y ebeveynin sol çocuğu ise, y1 ve y2'yi y'nin sol ve sağ çocuğu olarak belir leyin, Şekil 48.25'te gösterildiği gibi. Eğer y ebeveynin sağ çocuğu ise, y1 ve y2'yi y'nin sol ve sağ çocuğu olarak belirleyin, Şekil 48.26'da gösterildiği gibi. Her iki durumda da, y'yi siyah ve ebeveyni kırmızı yapın.

*childOfu* hala kurgusal bir çift-siyah düğümdür. Ayarlamaadan sonra, *childOfu*'nun karde ş i artık siyahtır ve ya Durum 1 ya da Durum 2

geçerlidir. Eğer Durum 1 geçerliyse, bir kez yapılan yeniden yapılandırma ve yenide n renklendirme işlemi çift-siyah sorununu ortadan kaldırır. Eğer Durum 2 geçerliyse, ç ift-siyah sorunu yeniden ortaya çıkamaz, çünkü ebeveyn artık kırmızıdır.

Bu nedenle, Durum 1 veya Durum 2'nin bir kez uygulanması Durumu tamamlayacaktır.

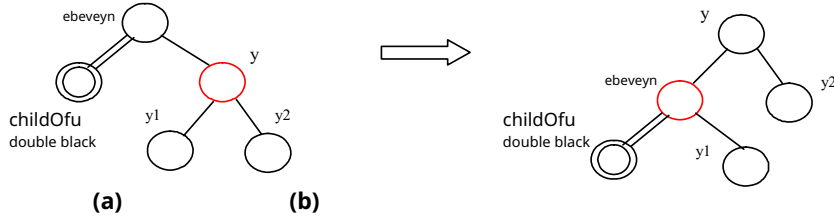
3.



Şekil 48.25

Durum 3.1:  $y_e$  ebeveynin sol kırmızı çocuğudur.





**Şekil 48.26**

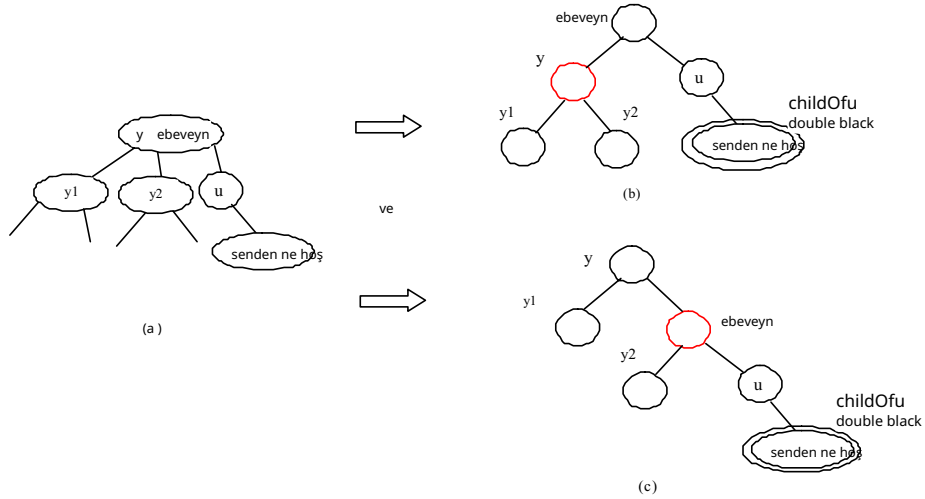
Durum 3.2:  $y$ , ebeveynin sağ kırmızı çocuğudur.

Not

<kenar notu: 3 düğümün benzersiz olmayan dönüşümü>

Durum 3, bir 3-düğümün

iki şekilde kırmızı-siyah ağaca dönüştürülebileceği gerçeğinden kaynaklanmaktadır; bu, Şekil 48.27'de gösterilmiştir.



**Şekil 48.27**

Bir 3-düğüm, kırmızı-siyah ağaç düğümlerine iki şekilde dönüştürülebilir.

Yukarıdaki tartışmaya dayanarak, Liste 48.2 bir ögeyi silmek için daha ayrıntılı bir algoritma sunmaktadır.

#### Liste 48.2 Kırmızı-Siyah Ağaçtan Bir Ögeyi Silme

- <margin note line 1: ağaçtan e'yi sil>
- <margin note line 2: düğümü bul>
- <margin note line 4: öge bulunamadı>
- <margin note line 6: içsel öge mi?>
- <margin note line 7: en sağdaki düğüm>
- <margin note line 12: dışsal düğüme giden yol>
- <margin note line 15: düğümü sil>
- <margin note line 17: bir öge silindi>

*<margin note line 18: silme başarılı>*

*<margin note line 22: bir düğümü sil>*

*<margin not satırı 23: u>n<ma*

*rgin not satırı 24: parentOfu, grandparentOfu>n<margin not satırı 23: childOfu>*

*<margin note line 26: sil u>*

*<margin note line 30: tamam>*

*<margin note line 32: childOfu'yu siyah olarak ayarla>*

*<margin note line 35: çift siyahı düzelt>*

*<margin note line 39: çift siyahı düzelt>*

*<margin note line 42: y, y1, y2>*

*<margin note line 47: Durum 1.1'i işle>*

*<margin note line 51: Durum 1.3'ü işle>*

*<margin note line 57: Durum 1.2'yi işle>*

*<margin note line 61: Durum 1.4'ü işle>*

*<margin note line 66: Durum 2'yi işle>*

*<margin note line 77: çift siyahı yay>*

*<margin note line 83: Durum 3.1'i işle>*

*<margin note line 88: Durum 3.2'yi işle>*

*<margin note line 95: çift siyahı düzelt>*

```
public boolean delete(E e) {
 Silinecek düğümü bul
 eğer (düğüm bulunamazsa)
 return false;

 eğer (düğüm bir iç düğümse) {
 Düğümün alt ağacındaki en sağdaki düğümü bul;
 Düğümdeki ögeyi en sağdaki ile değiştir;
 En sağdaki düğüm artık silinecek düğümdür;
 }

 Kökten silinecek düğüme giden yolu al;

 // Yoldaki son düğümü sil ve gerekirse yay
 deleteLastNodeInPath(path);

 size--; // Bir öge silindikten sonra
 return true; // Öge silindi
}

/** Yoldaki son düğümü sil. */
public void deleteLastNodeInPath(ArrayList<TreeNode<E>> path) {
 Yoldaki son düğüm u'yu al;
 parentOfu ve grandparentOfu'yu yoldan al;
 u'dan childOfu'yu al;
 Düğüm u'yu sil. childOfu'yu parentOfu ile bağla

 // Düğümleri yeniden renklendir ve gerekirse çift siyahı düzelt
 if (childOfu == root || u.isRed())
 return; // childOfu kökse veya u kırmızıysa tamam
 else if (childOfu != null && childOfu.isRed())
 childOfu.setBlack(); // Siyah yap, tamam
 else // u siyah, childOfu null veya siyah
 // parentOfu'da çift siyahı düzelt
```

```
fixDoubleBlack(grandparentOfu, parentOfu, childOfu, yol, i); N}
```

```
/** parent düğümündeki çift siyah sorununu düzelt */
```

```
private void fixDoubleBlack(
```

```
 RBTTreeNode<E> grandparent, RBTTreeNode<E> parent,
```

```
 RBTTreeNode<E> db, ArrayList<TreeNode<E>> path, int i) {
```

```
 y, y1 ve y2'yi elde et
```

```
 if (y.isBlack() && y1 != null && y1.isRed()) { n if (parent.rig
```

```
 ht == db) {
```

```
 // Durum 1.1: y sol siyah kardeş ve y1 kırmızı
```

```
 Problemi düzeltmek için parent, y ve y1'i yeniden yapılandır ve renklendir;
```

```
 }
```

```
 aksi takdirde {
```

```
 // Durum 1.3: y sağ siyah kardeş ve y1 kırmızı
```

```
 Sorunu düzeltmek için ebeveyni, y1'i ve y'yi yeniden yapılandır ve renklendir;
```

```
 }
```

```
 else if (y.siyah()) && y2 != null && y2.kırmızı()) {
```

```
 if (ebeveyn.sağ == db) {
```

```
 // Durum 1.2: y sol siyah kardeş ve y2 kırmızı
```

```
 Sorunu düzeltmek için ebeveyni, y2'yi ve y'yi yeniden yapılandır ve renklendir;
```

```
 }
```

```
 aksi takdirde {
```

```
 // Durum 1.4: y sağ siyah kardeş ve y2 kırmızı
```

```
 Sorunu düzeltmek için ebeveyni, y'yi ve y2'yi yeniden yapılandır ve renklendir;
```

```
 }
```

```
 else if (y.siyah()) {
```

```
 // Durum 2: y siyah ve y'nin çocukları siyah veya null
```

```
 y'yi kırmızıya renklendir;
```

```
 if (ebeveyn.kırmızı())
```

```
 ebeveyn.siyahYap(); // Tamam
```

```
 else if (ebeveyn != kök) {
```

```
 // İki siyahı ebeveyn düğümüne yay
```

```
 // İki siyahın yeni görünümünü özyinelemeli olarak düzelt
```

```
 db = ebeveyn;
```

```
 ebeveyn = büyük ebeveyn;
```

```
 büyük ebeveyn =
```

```
 (i >= 3) ? (RBTTreeNode<E>)(path.get(i - 3)) : null;
```

```
 fixDoubleBlack(büyük ebeveyn, ebeveyn, db, path, i - 1);
```

```
 }
```

```
 else if (y.kırmızı()) {
```

```
 eğer (parent.right == db) {
```

```
 // Durum 3.1: y, parent'ın sol kırmızı çocuğu
```

```
 parent.left = y2;
```

```
 y.right = üst öge; N
```

```
 }
```

```
 aksi takdirde {
```

```
 // Durum 3.2: y, parent'ın sağ kırmızı çocuğu
```

```
 parent.right = y.left;
```

```
 y.left = üst; N }
```

```

parent.setRed(); // Parent'ı kırmızı yap
y.setBlack(); // y'yi siyah yap
connectNewParent(büyükanne, ebeveyn, y); // y yeni ebeveyn
fixDoubleBlack(y, ebeveyn, db, yol, i - 1);
}
N}

```

#### <kenar notu: sil(E, e)>

delete(E e) metodu (satırlar 1–19) e'yi içeren düğümü bulur (satır 2). Düğüm yoksa, false döner (satırlar 3–4). Eğer düğüm iç düğümse, sol alt ağaçtaki en sağdaki düğümü bul ve düğümdeki elementi en sağdaki düğümdeki element ile değiştir (satırlar 6–9). Şimdi silinecek düğüm dış düğüm. Kökten düğümüne giden yolu al (satır 12). deleteLastNodeInPath(yol) metodunu çağırarak yoldaki son düğümü sil ve ağacın hala bir kırmızı-siyah ağaç olduğundan emin ol (satır 15).

#### <margin note: deleteLastNodeInPath(yol)>

deleteLastNodeInPath metodu (satırlar 22–36) son düğüm u, parentOfu, grandparentOfu ve childOfu'yu alır (satırlar 23–26). Eğer childOfu kökse veya u kırmızıysa, ağaç iyidir (satırlar 29–30). Eğer childOfu kırmızıysa, siyah yap (satırlar 31–32). İşimiz bitti. Aksi takdirde, u siyah ve childOfu null veya siyah. double-black sorununu ortadan kaldırmak için fixDoubleBlack metodunu çağır (satır 35).

#### <kenar notu: fixDoubleBlack>

fixDoubleBlack metodu (satırlar 39–97) double-black sorununu ortadan kaldırır. y, y1 ve y2'yi al (satır 42). y, double-bk düğümünün kardeşidir. y1 ve y2, y'nin sol ve sağ çocuklarıdır. Uç durumu göz önünde bulundur:

1. Eğer y siyah ve çocuklarından biri kırmızıysa, çift-siyah problemi, 1. Durumda (satırlar 44–63) bir kez yeniden yapılandırma ve yeniden renklendirme ile düzeltiler.
2. Eğer y siyah ve çocukları null veya siyah ise, y'yi kırmızıya çevirin. Eğer y'nin ebeveyni siyah ise, ebeveyni yeni çift-siyah düğüm olarak belirtin ve fixDoubleBlack'ı rekürsif olarak çağırın (satır 77).
3. Eğer y kırmızı ise, düğümleri ayarlayarak ebeveyni y'nin çocuğu yapın (satırlar 84, 89) ve ebeveyni kırmızı, y'yi siyah renge boyayın (satırlar 92–93). y'yi yeni ebeveyn yapın (satır 94). Aynı çift-siyah düğüm üzerinde ebeveyn için farklı bir renk ile fixDoubleBlack'ı rekürsif olarak çağırın (satır 95).

#### <margin note: silme örneği>

Şekil 48.28, elemanları silme adımlarını göstermektedir. Şekil 48.28(a)'dan 50'yi silmek için, Şekil 48.28(b)'de gösterildiği gibi 1.2 Durumunu uygulayın. Yeniden yapılandırma ve renklendirmeden sonra, yeni ağaç Şekil

48.28(c)'de gösterildiği gibidir.

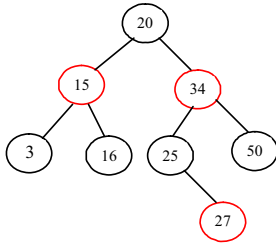
Şekil 48.28(c)'de 20'yi silerken, 20 bir iç düğümdür ve 16 ile değiştirilir, bu Şekil 48.28(d)'de gösterilmiştir. Artık en sağdaki düğümü silmek için 2. Durum uygulanır, bu da Şekil 48.28(e)'de gösterilmiştir. Düğümleri yeniden renklendirmek, Şekil 48.28(f)'de gösterildiği gibi yeni bir ağaç sonucunu verir.

15'i silerken, düğüm 3'ü düğüm 20 ile bağlayın ve düğüm 3'ü siyah renge boyayın, bu da Şekil 48.28(g)'de gösterilmiştir. İşimiz bitti.

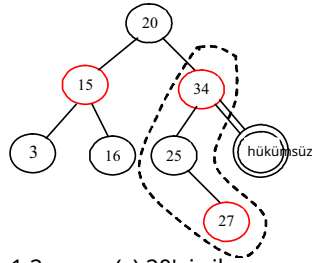
25 silindikten sonra, yeni ağaç Şekil 48.28(j)'de gösterildiği gibidir. Şimdi 16'yı sil. Şekil 48.28(k)'de gösterildiği gibi Durum 2'yi uygula. Yeni ağaç Şekil 48.28(l)'de gösterilmektedir.

34 silindikten sonra, yeni ağaç Şekil 48.28(m)'de gösterildiği gibidir.

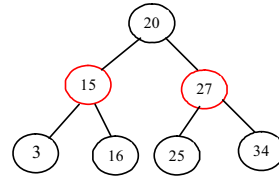
27 silindikten sonra, yeni ağaç Şekil 48.28(n)'de gösterildiği gibidir.



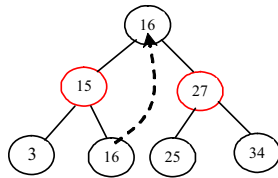
(a) 50'yi sil



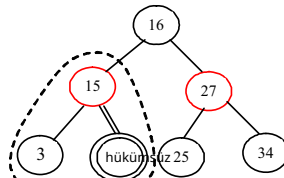
(b) Durum 1.2



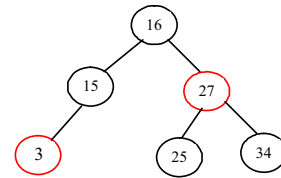
(c) 20'yi sil



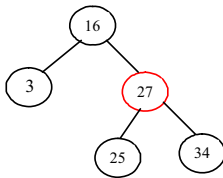
(d) 20'yi değiştirmek için 16'yı kopyala



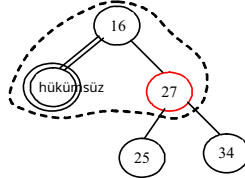
(e) Durum 2



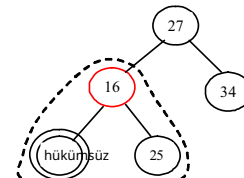
(f) 15'i sil



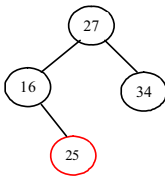
(g) 3'ü sil



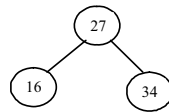
(h) Durum 3



(i) Durum 2

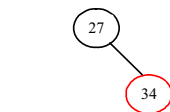
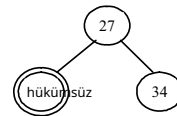


(j) 25'i sil



(j) 16'yı sil

(k) Durum 2



(l) 34'ü sil



(m) 27'yi sil

(n) Boş ağaç

kök: hükümsüz

## Şekil 48.28

Kırmızı-siyah ağaçtan elemanları sil.

### 48.6 RBTree Sınıfını Uygulama

Liste 48.3, RBTree sınıfı için tam bir uygulama sunmaktadır.

#### Liste 48.3 RBTree.java

```
<margin note line 5: argüman almayan yapıcı>
<margin note line 9: yapıcı>
<margin note line 14: yeni bir düğüm oluştur>
<margin note line 19: ağaca ekle>
<margin note line 20: super.insert'i çağır>
<margin note line 22: kopya eleman>
<margin note line 24: renk ve derinliği sağla>

<margin note line 31: renk ve derinliği sağla>
<margin note line 33: yolu al>
<margin note line 35: düğüm indeksi>
<margin note line 38: u'yu al>
<margin note line 41: v'yi al>

<margin note line 46: u kök mü?>
<margin note line 49: çift-kırmızı ihlali>

<margin note line 53: çift kırmızıyı düzelt>
<margin note line 56: get w>
<margin note line 61: get x>
<margin note line 65: Durum 1>
<margin note line 66: Durum 1.1>
<margin note line 73: Durum 1.2>
<margin note line 81: Durum 1.3>
<margin note line 87: Durum 1.4>
<margin note line 96: Durum 2>
<margin note line 98: yeniden renklendirme>
<margin note line 103: w kök mü?>
<margin note line 108: yukarı doğru yayıl>
<margin note line 110: yeni çift kırmızıyı düzelt>

<margin note line 116: yeniden yapılandır/renklendir>
<margin note line 133: ağaçtan e'yi sil>
<margin note line 135: düğümü bul>
<margin note line 147: eleman bulunamadı>
<margin note line 153: iç eleman mı?>
<margin note line 155: en sağdaki düğüm>
<margin note line 160: dış düğüme giden yol>
<margin note line 169: düğümü sil>
<margin note line 171: bir eleman silindi>
<margin note line 172: silme başarılı>

<margin note line 176: bir düğümü sil>
<margin note line 179: u>
<margin note line 180: u'nun ebeveyni>
<margin note line 182: u'nun büyükanı>
<margin note line 185: u'nun çocuğu>
<margin note line 189: u'yu sil>
<margin note line 193: tamam>
```

<margin note line 195: u'nun çocuğunu siyah yap>  
<margin note line 198: çift siyahı düzelt>

<margin note line 202: çift siyahı düzelt>  
<margin note line 206: y, y1, y2>  
<margin note line 212: işlem Durum 1.1>  
<margin note line 221: işlem Durum 1.3>  
<margin note line 234: işlem Durum 1.2>  
<margin note line 245: işlem Durum 1.4>  
<margin note line 255: işlem Durum 2>  
<margin note line 263: çift siyahı yay>  
<margin note line 271: işlem Durum 3.1>  
<margin note line 276: işlem Durum 3.2>  
<margin note line 285: çift siyahı düzelt>

<margin note line 304: büyük ebeveynle bağlan>  
<margin note line 318: önceliği geçersiz kıl>  
<margin note line 327: RBTreeNode>

Java.util.ArrayList'i içe aktarın;

genel sınıf RBTree<E Comparable'ı genişletir<E>> BinaryTree'yi genişletir<E> {

/\*\* Varsayılan bir RB ağacı oluştur \*/

genel RBTree() {

/\*\* Bir dizi elemandan bir RB ağacı oluştur \*/

genel RBTree(E[] öğeleri) {

süper(elemanlar);

/\*\* RBTreeNode oluşturmak için createNewNode'u geçersiz kıl \*/

korumalı RBTreeNode<E> yeniDüğümOluştur(E e) {

yeni RBTreeNode<E>(e) döndür;

/\*\* Gerekirse ağacı dengelemek için ekleme yöntemini geçersiz kıl \*/

public boolean ekle(E e) {

boolean başarılı = süper.ekle(e);

if (!başarılı)

return false; // e zaten ağaçta var

başka {

sureRBTree(e); N }

return true; // e eklendi

/\*\* Ağacın bir kırmızı-siyah ağaç olduğundan emin olun \*/

özel void ensureRBTree(E e) {

// Kökten e elemanına giden yolu al

ArrayList<TreeNode<E>> path = path(e);

int i = path.size() - 1; // Yoldaki mevcut düğüm için indeks

// u, yoldaki son düğümdür. u, e elemanını içerir

RBTreeNode<E> u = (RBTreeNode<E>) (path.get(i));

```

// v, varsa u'nun ebeveynidir
RBTreeNode<E> v = (u == root) ? null :
(RBTreeNode<E>)(path.get(i - 1));

u.setRed(); // u'yu kırmızı yapmakta bir sakınca yoktur

if (u == root) // Eğer e kök olarak eklenirse, kökü siyah yap
 u.setBlack();
else if (v.isRed())
 fixDoubleRed(u, v, path, i); // u'daki çift-kırmızı ihlalinin düzelt
}

/** u düğümündeki çift-kırmızı ihlalinin düzelt */
özel void fixDoubleRed(RBTreeNode<E> u, RBTreeNode<E> v,
 ArrayList<TreeNode<E>> yol, int i) {
 // w, u'nun büyükbabasıdır
 RBTreeNode<E> w = (RBTreeNode<E>)(yol.get(i - 2));
 RBTreeNode<E> parentOfw = (w == kök) ? null :
 (RBTreeNode<E>)yol.get(i - 3);

 // v'nin x adındaki kardeşini al
 RBTreeNode<E> x = (w.sol == v) ?
 (RBTreeNode<E>)(w.sağ) : (RBTreeNode<E>)(w.sol);

 if (x == null || x.siyah()) {
 // Durum 1: v'nin kardeşi x siyah
 if (w.sol == v && v.sol == u) {
 // Durum 1.1: $u < v < w$, Düğümleri yeniden yapılandır ve renklendir
 yenidenYapılandırVeRenkDeğiştir(u, v, w, w, parentOfw);

 w.sol = v.sağ; // v.sağ, Şekil 48.6'da y3'tür
 v.sağ = w;
 }
 yoksa eğer (w.sol == v && v.sağ == u) {
 // Durum 1.2: $v < u < w$, Düğümleri yeniden yapılandır ve renklendir
 yenidenYapılandırVeRenkDeğiştir(v, u, w, w, parentOfw);
 v.sağ = u.sol;
 w.sol = u.sağ;
 u.sol = v;
 u.sağ = w;
 }
 yoksa eğer (w.sağ == v && v.sağ == u) {
 // Durum 1.3: $w < v < u$, Düğümleri yeniden yapılandır ve renklendir
 yenidenYapılandırVeRenkDeğiştir(w, v, u, w, parentOfw);
 w.sağ = v.sol;
 v.sol = w;
 }
 başka {
 // Durum 1.4: $w < u < v$, Düğümleri yeniden yapılandır ve renklendir
 yenidenYapılandırVeRenklendir(w, u, v, w, w'ninAnaDüğümü);
 w.sağ = u.sol;
 v.sol = u.sağ;
 u.sol = w;
 u.sağ = v;
 }
 }
}

```



```

 }
 else { // Durum 2: v'nin kardeşi x kırmızı
 // Döğümleri renklendir
 w.kırmızıYap();
 u.kırmızıYap();
 ((RBTreeNode<E>)(w.sol)).siyahYap();
 ((RBTreeNode<E>)(w.sağ)).siyahYap();

 eğer (w == root) {
 w.setBlack();n }

 else if (((RBTreeNode<E>)parentOfw).isRed()) {
 // Yeni çift-kırmızı ihlalini düzeltmek için yol boyunca yayıl
 sen = w;
 v = (RBTreeNode<E>)parentOfw;
 fixDoubleRed(u, v, path, i - 2); // i - 2 yukarıya yayılır
 }
 }
}

/** b'yi parentOfw ile bağla ve a, b, c'yi a < b < c için yeniden renklendir */
özel void restructureRecolor(RBTreeNode<E> a, RBTreeNode<E> b,
 RBTreeNode<E> c, RBTreeNode<E> w, RBTreeNode<E> parentOfw) {
 if (parentOfw == null)
 kök = b;
 else if (parentOfw.left == w)
 parentOfw.left = b;
 başka
 ebeveynOfw.right = b;

 b.setBlack(); // b alt ağaçta kök olur
 a.setRed(); // a b'nin sol çocuğu olur
 c.setRed(); // c b'nin sağ çocuğu olur
}

/** RBTree'den bir öğeyi sil.
 * Öğenin başarıyla silinmesi durumunda true döner
 * Öğenin ağaçta olmaması durumunda false döner */
public boolean delete(E e) {
 // Silinecek düğümü bul
 TreeNode<E> geçerli = kök;
 while (geçerli != null) {n if (e.co
 mpareTo(current.element) < 0) {
 akım = akım.sol;
 }
 else if (e.compareTo(geçerli.element) > 0) {
 akım = akım.sağ;
 }
 başka
 break; // Eleman, current tarafından işaret edilen ağaçta
 }

 if (geçerli == null)
 return false; // Eleman ağaçta yok

 java.util.ArrayList<TreeNode<E>> yol;

```

```

// current düğümü bir iç düğümdür
if (current.left != null && current.right != null) {
 // current'ın sol alt ağacındaki en sağdaki düğümü bul
 TreeNode<E> sağEn = current.left;
 while (sağEn.right != null) {
 sağEn = sağEn.right; // Sağa devam et
 }

 yol = path(sağEn.element); // Değiştirmeden önce yolu al

 // current içindeki elementi sağEn'deki element ile değiştir
 current.element = sağEn.element;
}
başka
yol = path(e); // current düğümüne giden yolu al

// Yoldaki son düğümü sil ve gerekirse yay
deleteLastNodeInPath(yol);

boyut--; // Bir element silindikten sonra
return true; // Element silindi
}

/** Yoldaki son düğümü sil. */
public void deleteLastNodeInPath(ArrayList<TreeNode<E>> yol) {
 int i = yol.size() - 1; // Yoldaki düğüm için indeks
 // u, yolun son düğümüdür
 RBTreeNode<E> u = (RBTreeNode<E>) (path.get(i));
 RBTreeNode<E> parentOfu = (u == root) null :
 (RBTreeNode<E>)(path.get(i - 1)); n RBTre
 eNode<E> grandparentOfu = (parentOfu == null ||
 parentOfu == root) null:
 (RBTreeNode<E>)(path.get(i - 2)); n RBTre
 eNode<E> childOfu = (u.left == null)
 (RBTreeNode<E>) (u.right) : (RBTreeNode<E>) (u.left);

 // Düğüm u'yu sil. childOfu'yu parentOfu ile bağla
 connectNewParent(parentOfu, u, childOfu);

 // Düğümleri yeniden renklendir ve gerekirse çift siyahı düzelt
 if (childOfu == root || u.isRed())
 return; // childOfu kökse veya u kırmızıysa tamam
 else if (childOfu != null && childOfu.isRed())
 childOfu.setBlack(); // Siyah yap, tamam
 else // u siyah, childOfu null veya siyah
 // Ebeveynin çift siyah sorununu düzelt
 fixDoubleBlack(grandparentOfu, parentOfu, childOfu, path, i);
}

/** Ebeveyn düğümündeki çift-siyah sorununu düzelt */
özel geçersiz düzeltmeDoubleBlack(
 RBTreeNode<E> büyük ebeveyn, RBTreeNode<E> ebeveyn,
 n RBTreeNode<E> db, ArrayList<TreeNode<E>> yolu, int i) {
 // y, y1 ve y2'yi elde et
 RBTreeNode<E> y = (parent.right == db) ?

```

```

 (TreeNode<E>)(parent.left) : (TreeNode<A>)(parent.right);
 RBTTreeNode<E> y1 = (RBTTreeNode<E>)(y.left);n RBTre
 eNode<E> y2 = (RBTTreeNode<E>)(y.right);n if (y.isBlack

 () && y1 != null && y1.isRed()) {
 if (ebeveyn.sağ == db) {
 // Durum 1.1: y sol siyah kardeş ve y1 kırmızı
 connectNewParent(büyük ebeveyn, ebeveyn, y);
 recolor(parent, y, y1); // Renkleri ayarla

 // Çocuk bağlantılarını ayarla
 parent.left = y.right;
 y.right = üst öge;
 }
 başka {
 // Durum 1.3: y sağ siyah kardeş ve y1 kırmızı
 connectNewParent(grandparent, parent, y1);
 recolor(parent, y1, y); // Renkleri ayarla

 // Çocuk bağlantılarını ayarla
 parent.right = y1.left;
 y.sol = y1.sağ;
 y1.left = üst;
 y1.sağ = y;
 }
 }
 else if (y.isBlack() && y2 != null && y2.isRed()) {
 if (ebeveyn.sağ == db) {
 // Durum 1.2: y sol siyah kardeş ve y2 kırmızı
 connectNewParent(büyük ebeveyn, ebeveyn, y2);
 recolor(parent, y2, y); // Renkleri ayarla

 // Çocuk bağlantılarını ayarla
 y.right = y2.left;n pare
 nt.left = y2.right;
 y2.left = y;n y2.
 right = üst öge;
 }
 başka {
 // Durum 1.4: y sağ siyah kardeş ve y2 kırmızı
 connectNewParent(büyükanne, ebeveyn, y);
 recolor(parent, y, y2); // Renkleri ayarla

 // Çocuk bağlantılarını ayarla
 y.sol = ebeveyn;n e
 beveyn.sağ = y1;
 }
 }
 else if (y.Siyahtır()) {
 // Durum 2: y siyah ve y'nin çocukları siyah veya null
 y.setRed(); // y'yi kırmızıya çevir
 if (parent.isRed())
 parent.setBlack(); // Tamam
 else if (parent != root) {
 // İki siyahı ebeveyn düğümüne yay
 // İki siyahın yeni görünümünü özyinelemeli olarak düzelt

```

```

 db = ebeveyn.n
 ebeveyn = büyük ebeveyn;
 büyükbaba veya büyükanne =
 (i >= 3) (RBTreeNode<E>)(path.get(i - 3)) : null;
 fixDoubleBlack(büyükanne, ebeveyn, db, yol, i - 1);
 }
}
else { // y.kırmızı mı?
 if (ebeveyn.sağ == db) {
 // Durum 3.1: y, ebeveynin sol kırmızı çocuğu
 ebeveyn.sol = y2;
 y.right = üst öge;
 }
 başka {
 // Durum 3.2: y, ebeveynin sağ kırmızı çocuğu
 ebeveyn.sağ = y.sol;
 y.sol = ebeveyn;
 }

 ebeveyn.kırmızıYap(); // Ebeveyni kırmızı yap
 y.siyahYap(); // y'yi siyah yap
 yeniEbeveyniBağla(büyükanne, ebeveyn, y); // y yeni ebeveyn
 fixDoubleBlack(y, ebeveyn, db, yol, i - 1);
}
}

/** Ebeveyn, yeniEbeveyn ve c'yi yeniden renklendir. Durum 1 kaldırma */
özel void recolor(RBTreeNode<E> parent,
 RBTreeNode<E> newParent, RBTreeNode<E> c) {
 // newParent için ebeveynin rengini koru
 if (parent.isRed()) n new
 Parent.setRed(); başka
 türlü
 newParent.setBlack();

 // c ve parent newParent'ın çocukları olur; onları siyah yap
 parent.setBlack(); n c.s
 etBlack();
}

/** newParent'ı grandParent ile bağla */
özel void connectNewParent(RBTreeNode<E> grandparent,
 RBTreeNode<E> ebeveyn, RBTreeNode<E> newParent) {
 if (ebeveyn == kök) {
 kök = yeniEbeveyn;
 eğer (kök != null)
 yeniEbeveyn.siyahOl();
 }
 aksi takdirde eğer (büyükanne.sol == ebeveyn)
 büyükanne.sol = yeniEbeveyn;
 başka
 büyükanne.sağ = yeniEbeveyn;
}

/** Bir alt ağaçtan ön sipariş gezintisi */
korunmalı void önSipariş(TreeNode<E> kök) {

```

---

---

u) yoldan. Eğer u kökse, u'yu siyah yap (satırlar 46–47). Eğer v kırmızıysa, u ve v'deki çift kırmızıyı düzeltmek için `fixDoubleRed`'i çağır (satırlar 48–49).

#### <kenar notu: `fixDoubleRed`>

`fixDoubleRed(u, v, path, i)` metodu, `node` u'daki double-red ihlalini düzeltir. Metod önce w'yu (u'nun büyükbabası, path'ten) alır (satır 56), `parentOfw`'yu eğer varsa (satırlar 57–58) ve x'i (v'nin kardeşi) alır (satırlar 61–62). Eğer x null veya siyah ise, double-red ihlalini düzeltmek için dört alt durumu consider eder (satırlar 66–95). Eğer x kırmızı ise, w'yu ve u'yu kırmızıya boyar ve w'nin iki çocuğunu siyah yapar (satırlar 100–103). Eğer w kök ise, w'yu siyah yapar (satırlar 103–105). Aksi takdirde, yeni double-red ihlalini düzeltmek için path boyunca yayılır (satırlar 108–110).

#### <kenar notu: `sil`>

`delete(E e)` metodu `RBTree`'de satırlar 133–173'te geçersiz kılınmıştır. Metod, e'yi içeren `node`'u bulur (satırlar 135–145). Eğer `node` null ise, hiçbir eleman bulunamaz (satırlar 147–148). Metod iki durumu

dikkate alır:

- Eğer `node` içsel ise, sol alt ağaçtaki en sağdaki `node`'u bul (satırlar 155–158). Kökten en sağdaki `node`'a bir yol al (satır 160) ve `node`'daki elemanı en sağdaki `node`'daki elemanla değiştir (satır 163).
- Eğer `node` dışsal ise, `node`'a giden yolu kökten al (satır 166).

Yoldaki son `node`, silinecek `node`'dur. `deleteLastNodeInPath(path)` çağrısını yaparak silinmesini sağla ve `node` silindikten sonra ağacın kırmızı-siyah olduğunu garanti et (satır 169).

#### <kenar notu: `deleteLastNodeInPath`>

`deleteLastNodeInPath(path)` metodu önce u, `parentOfu`, `grandparentOfu` ve `childOfu`'yu elde eder (satırlar 179–186). u, path'teki son düğümdür. `childOfu`'yu `parentOfu`'nun çocuğu olarak bağlayın (satır 189). Bu, u'yu ağaçtan siler. Üç durumu göz önünde bulundurun:

- `childOfu` kökse veya `childOfu` kırmızıysa, işimiz bitti (satırlar 192–193).
- Aksi takdirde, eğer `childOfu` kırmızıysa, onu siyah yapın (satırlar 194–195).
- Aksi takdirde, `childOfu` üzerindeki çift-siyah sorununu düzeltmek için `fixDoubleBlack`'i çağırın (satır 198).

#### <kenar notu: `fixDoubleBlack`>

`fixDoubleBlack` metodu önce y, y1 ve y2'yi elde eder (satırlar 206–209). y, ilk çift-siyah düğümün kardeşidir ve y1 ile y2, y'nin sol ve sağ çocuklarıdır. Üç durumu göz önünde bulundurun:

- eğer y siyah ve y1 veya y2 kırmızıysa, Durum 1 için çift-siyah sorununu düzeltin (satırlar 212–254).
- Aksi takdirde, eğer y siyahsa, düğümleri yeniden renklendirerek Durum 2 için çift-siyah sorununu düzeltin. Eğer `parent` siyah ve kök değilse, double black'i `parent`'a yayarak `fixDoubleBlack`'i yinelemeli olarak çağırın (satırlar 263–267).

- Aksi takdirde,  $y$  kırmızıdır. Bu durumda, düğümleri ayarlayarak parent'ı  $y$ 'nin çocuğu yapın (satırlar 271–280). Ayarlanmış düğümlerle `fixDoubleBlack`'ı çağırın (satır 285) ve çift-siyah sorununu düzeltin.

*<kenar notu: ön sipariş>*

`preorder(TreeNode<E> root)` yöntemi, düğüm

renklerini görüntülemek için geçersiz kılınmıştır (satırlar 318-324).

#### 48.7 RBTre Sınıfını Test Etme

Liste 48.4, bir test programı vermektedir. Program, 34, 3 ve 50 tam sayılarından oluşan bir dizi ile başlatılan bir RBTre oluşturur (satırlar 4–5), satırlar 10–22'de elemanlar ekle ve satırlar 25–46'da elemanları siler.

##### Liste 48.4 TestRBTre.java

*<margin note line 5: bir RBTre oluştur>*

*<margin note line 8: 20 ekle>*

*<margin note line 11: 15 ekle>*

*<margin note line 14: 16 ekle>*

*<margin note line 17: 25 ekle>*

*<margin note line 20: 27 ekle>*

*<margin note line 23: 50 sil>*

*<margin note line 29: 15 sil>*

*<margin note line 32: 3 sil>*

*<margin note line 35: 25 sil>*

*<margin note line 38: 16 sil>*

*<margin note line 41: 34 sil>*

*<margin note line 44: 27 sil>*

```
public class TestRBTre {
 public static void main(String[] args) {
 // Bir RB ağacı oluştur
 RBTre<Tamsayı> ağacı =
 new RBTre<Integer>(new Integer[]{34, 3, 50});n printTre
e(tree);

 ağaç.insert(20); n pr
intTree(ağaç);

 ağaç.insert(15);
 printTree(ağaç);

 ağaç.insert(16);
 printTree(ağaç);

 ağaç.insert(25);
 printTree(ağaç);

 ağaç.insert(27);
 printTree(ağaç);

 ağaç.sil(50);
 ağaç.yazdır(ağaç);

 ağaç.sil(20);
 ağaç.yazdır(ağaç);
 }
}
```

```

 ağaç.sil(15);
 ağaç.yazdır(ağaç);

 ağaç.sil(3);
 ağaç.yazdır(ağaç);

 ağaç.sil(25);
 ağaç.yazdır(ağaç);

 ağaç.sil(16);
 ağaç.yazdır(ağaç);

 ağaç.sil(34);
 ağaç.yazdır(ağaç);

 ağaç.sil(27);
 ağaç.yazdır(ağaç);
 }

 public static void ağaçYazdır(BinaryTree ağaç) {
 // Ağaçta gez
 System.out.print("Sıralı (sıralı): "); ağaç.sıralı();

 System.out.print("\nPostorder: ");ntree.postorder();
 System.out.print("\nPreorder: ");ntree.preorder();
 System.out.print("Düğüm sayısı " + tree.getSize()); System.out.println();
 System.out.println();
 }
}
N}

```

### <Çıktı>

Inorder (sıralı): 3 34 50

Postorder: 3 50 34

Preorder: 34 (siyah) 3 (kırmızı) 50 (kırmızı)

Düğüm sayısı 3

Inorder (sıralı): 3 20 34 50

Postorder: 20 3 50 34

Preorder: 34 (siyah) 3 (siyah) 20 (kırmızı) 50 (siyah)

Düğüm sayısı 4

Inorder (sıralı): 3 15 20 34 50

Postorder: 3 20 15 50 34

Preorder: 34 (siyah) 15 (siyah) 3 (kırmızı) 20 (kırmızı) 50 (siyah)

Düğüm sayısı 5

Artan sıralama (sıralı): 3 15 16 20 34 50

Sonradan sıralama: 3 16 20 15 50 34

Ön Sipariş: 34 (siyah) 15 (kırmızı) 3 (siyah) 20 (siyah) 16 (kırmızı) 50 (siyah)

Düğüm sayısı 6

İç Sıralama (sıralı): 3 15 16 20 25 34 50

Son Sipariş: 3 16 25 20 15 50 34

Ön Sipariş: 34 (siyah) 15 (kırmızı) 3 (siyah) 20 (siyah) 16 (kırmızı) 25 (kırmızı)



50 (siyah)  
Düğüm sayısı 7

İç Sıralama (sıralı): 3 15 16 20 25 27 34 50  
Son Sipariş: 3 16 15 27 25 50 34 20  
Ön Sipariş: 20 (siyah) 15 (kırmızı) 3 (siyah) 16 (siyah) 34 (kırmızı) 25 (siyah)  
27 (kırmızı) 50 (siyah)  
Düğüm sayısı 8

İç Sıralama (sıralı): 3 15 16 20 25 27 34  
Son Sipariş: 3 16 15 25 34 27 20  
Ön Sipariş: 20 (siyah) 15 (kırmızı) 3 (siyah) 16 (siyah) 27 (kırmızı)  
25 (siyah) 34 (siyah)  
Düğüm sayısı 7

İç Sıralama (sıralı): 3 15 16 25 27 34  
Son Sipariş: 3 15 25 34 27 16  
Ön Sipariş: 16 (siyah) 15 (siyah) 3 (kırmızı) 27 (kırmızı) 25 (siyah) 34 (siyah)  
Düğüm sayısı 6

Artan (sıralı): 3 16 25 27 34  
Postorder: 3 25 34 27 16  
Öncelik sırası: 16 (siyah) 3 (siyah) 27 (kırmızı) 25 (siyah) 34 (siyah)  
Düğüm sayısı 5

Artan (sıralı): 16 25 27 34  
Postorder: 25 16 34 27  
Öncelik sırası: 27 (siyah) 16 (siyah) 25 (kırmızı) 34 (siyah)  
Düğüm sayısı 4

Artan (sıralı): 16 27 34  
Postorder: 16 34 27  
Öncelik sırası: 27 (siyah) 16 (siyah) 34 (siyah)  
Düğüm sayısı 3

Sıralı (sıralanmış): 27 34  
Sonradan: 34 27  
Öncelikli: 27 (siyah) 34 (kırmızı)  
Düğüm sayısı 2

Sıralı (sıralanmış): 27  
Sonradan: 27  
Öncelikli: 27 (siyah)  
Düğüm sayısı 1

Sıralı (sıralanmış):  
Sonradan:  
Öncelikli:  
Düğüm sayısı 0

<Çıkış Sonu>

Şekil 48.14, ağaç elemanlar eklendikçe nasıl evrildiğini gösterir ve  
Şekil 48.28, ağaç elemanlar silindikçe nasıl evrildiğini gösterir.

#### 48.8 RBTTree Sınıfının Performansı

**<kenar notu:** İki dil *yükseklik*>

Bir kırmızı-siyah ağacındaki arama, ekleme ve silme süreleri, ağacın yüksekliğine bağlıdır. Bir kırmızı-siyah ağaç, bir 2-4 ağacına karşılık gelir. Bir 2-4 ağacındaki bir düğümü kırmızı-siyah ağaç düğümlerine dönüştürdüğünüzde, orijinal düğümün 2-düğüm, 3-düğüm veya 4-düğüm olup olmamasına bağlı olarak bir siyah düğüm ve sıfır, bir veya iki kırmızı düğüm elde edersiniz. Bu nedenle, bir kırmızı-siyah ağacın yüksekliği, karşılık gelen 2-4 ağacının yüksekliğinin en fazla iki katıdır. Bir 2-4 ağacının yüksekliği

$\log_2 n$ , bir kırmızı-siyah ağacın yüksekliği

dir İki dil .

**<margin note: kırmızı-siyah vs. AVL>**

Bir kırmızı-siyah ağacın zaman karmaşıklığı, Tablo 48.1'de gösterildiği gibi bir AVL ağacıyla aynıdır. Genel olarak, bir kırmızı-siyah ağaç, bir AVL ağacından daha verimlidir, çünkü bir kırmızı-siyah ağaç, ekleme ve silme işlemleri için düğümlerin yalnızca bir kez yeniden yapılandırılmasını gerektirir.

**<kenar notu: kırmızı-siyah vs. 2-4>**

Kırmızı-siyah ağaç, Tablo 48.1'de gösterildiği gibi, 2-4 ağacı ile aynı zaman karmaşıklığına sahiptir. Genel olarak, kırmızı-siyah ağaç, iki nedenle 2-4 ağacından daha etkilidir:

1. Kırmızı-siyah ağaç, ekleme ve silme işlemleri için düğümlerin yalnızca bir kez yeniden yapılandırılmasını gerektirir. Ancak, bir 2-4 ağacı, bir ekleme işlemi için birçok bölme ve bir silme işlemi için birleşme gerektirebilir.
2. Kırmızı-siyah ağaç, ikili arama ağacıdır. İkili bir ağaç, bir 2-4 ağacından daha az alan verimliliği ile uygulanabilir, çünkü bir 2-4 ağacındaki bir düğüm en fazla üç öğe ve dört çocuk içerir. 2-düğüm ve 3-düğüm için alan, 2-4 ağacında israf edilmektedir.

Tablo 48.1

RBTTree, AVLTree ve Tree234'teki Yöntemler için Zaman Karmaşıklıkları

Metotlar	Kırmızı-Siyah Ağaç	AVL Ağaç	2-4 Ağaç
arama(e: E)	$O(\text{günlük})n$	$O(\text{günlük})n$	$O(\text{günlük})n$
ekle(e: E)	$O(\text{günlük})n$	$O(\text{günlük})n$	$O(\text{günlük})n$
sil (e: E)	$O(\text{günlük})n$	$O(\text{günlük})n$	$O(\text{günlük})n$
getSize()	1'den itibaren	1'den itibaren	$O(1)$
isEmpty()	1'den itibaren	1'den itibaren	$O(1)$

Liste 48.5, AVL ağaçlarının, 2-4 ağaçlarının ve kırmızı-siyah ağaçlarının performansına dair ampirik bir test sunmaktadır.

#### Liste 48.5 TreePerformanceTest.java

**<margin note line 6: bir AVL ağacı>**

**<margin note line 11: bir 2-4 ağacı>**

**<margin note line 16: bir kırmızı-siyah ağaç>**

**<margin note line 21: başlangıç zamanı>**

**<margin note line 29: karıştır>**

**<margin note line 33: ağaca ekle>**

<margin note line 35: karıştır>

<margin note line 39: konteynerden çıkar>

<margin note line 41: bitiş zamanı>

<margin note line 42: geçen zamanı döndür>

```
public class TreePerformanceTest {
 public static void main(String[] args) {
 final int TEST_SIZE = 500000; // Testte kullanılan ağaç boyutu

 // Bir AVL ağacı oluştur
 Tree<Integer> tree1 = new AVLTree<Integer>();
 System.out.println("AVL ağacı süresi: " +
 getTime(tree1, TEST_SIZE) + " milisaniye");

 // Bir 2-4 ağacı oluştur
 Tree<Integer> tree2 = new Tree24<Integer>();
 System.out.println("2-4 ağacı süresi: " +
 + getTime(tree2, TEST_SIZE) + " milisaniye");

 // Kırmızı-siyah ağaç oluştur
 Tree<Integer> tree3 = new RBTree<Integer>();
 System.out.println("RB ağacı süresi: " +
 + getTime(tree3, TEST_SIZE) + " milisaniye");

 }

 public static long getTime(Tree<Integer> tree, int testSize) {
 long startTime = System.currentTimeMillis(); // Başlangıç zamanı

 // Ayrı tamsayıları saklamak için bir liste oluştur
 java.util.List<Integer> list = new java.util.ArrayList<Integer>();
 for (int i = 0; i < testSize; i++)
 list.add(i);

 java.util.Collections.shuffle(list); // Listeyi karıştır

 // Listeye elemanları ağaca ekle
 for (int i = 0; i < testSize; i++)
 ağaç.insert(liste.get(i));

 java.util.Collections.shuffle(list); // Listeyi karıştır

 // Listedeki ağaçtan öğeleri sil
 for (int i = 0; i < testSize; i++)
 ağaç.sil(list.get(i));

 // Geçen süreyi döndür
 return System.currentTimeMillis() - startTime;
 }
}
```

<Çıktı>

AVL ağacı süresi: 7609 milisaniye

2-4 ağacı süresi: 8594 milisaniye

RB ağacı süresi: 5515 milisaniye

<Çıkış Sonu>

getTestTime metodu, 0'dan testSize - 1'e kadar olan farklı tam sayıların bir listesini oluşturur (satırlar 25-27), listeyi karıştırır (satır 29) ve ekler

liste öğelerini bir ağaç yapısına (satırlar 32–33) dönüştürür, listeyi tekrar karıştırır (satır 35), ağaçtan öğeleri kaldırır (satırlar 38–39) ve nihayet çalışma süresini döndürür (satır 42).

Program, bir AVL (6. satır), bir 2-4 ağacı (11. satır) ve bir kırmızı-siyah ağacı (16. satır) oluşturur. Program, üç ağaçta 500000 eleman ekleme ve kaldırma için yürütme süresini alır.

**<margin note: kırmızı-siyah ağaç en iyisi>**

Kırmızı-siyah ağaç en iyi performansı gösteriyor, ardından AVL

ağacı geliyor.

NOT:

**<kenar notu: java.util.TreeSet>**

Java API'sindeki `java.util.TreeSet` sınıfı kırmızı-siyah ağaç kullanılarak uygulanmıştır. Kümedeki her giriş ağaçta saklanır. Kırmızı-siyah ağaçtaki arama, ekleme ve silme yöntemleri  $O(\text{günlük})_{\text{tn}}$  zaman, `get`, `add`, `remove` ve `contains` yöntemleri `java.util.TreeSet` alır  $O(\text{günlük})_{\text{tn}}$  zaman.

NOT:

**<kenar notu: java.util.TreeMap>**

Java API'deki `java.util.TreeMap` sınıfı, bir kırmızı-siyah ağaç kullanılarak uygulanmıştır. Haritadaki her giriş ağaçta saklanır. Girişlerin sırası, anahtarları tarafından belirlenir. Kırmızı-siyah ağaçta ki arama, ekleme ve silme yöntemleri alır  $O(\text{günlük})_{\text{tn}}$  zaman, `get`, `put`, `remove` ve `containsKey` yöntemleri `java.util.TreeMap` alır  $O(\text{günlük})_{\text{tn}}$  zaman.

**Anahtar Terimler**

- siyah derinlik
- çift-siyah ihlali
- çift-kırmızı ihlali
- dış düğüm
- kırmızı-siyah ağaç

**Bölüm Özeti**

1. Kırmızı-siyah ağaç, bir ikili arama ağacıdır, 2-4 ağacından türetilmiştir. Kırmızı-siyah ağaç, 2-4 ağacına karşılık gelir. Bir kırmızı-siyah ağacı 2-4 ağacına veya tam tersine dönüştürebilirsiniz.
2. Kırmızı-siyah ağaçta, her düğüm kırmızı veya siyah renktedir. Kök her zaman siyah'tir. İki bitişik düğüm aynı anda kırmızı olamaz. Tüm dış düğümlerin aynı siyah derinliği vardır.
3. Kırmızı-siyah ağaç bir ikili arama ağacı olduğundan, `RBTree` sınıfı `BinaryTree` sınıfını genişletir.
4. Kırmızı-siyah ağaçta bir öğe aramak, ikili arama ağacında aramakla aynıdır, çünkü bir kırmızı-siyah ağaç bir ikili arama ağacıdır.
5. Yeni bir öğe her zaman bir yaprak düğüm olarak eklenir. Yeni düğüm kökse, siyah renkte boyayın. Aksi takdirde, kırmızı renkte boyayın. Yeni düğümün ebeveyni kırmızıysa, çift-kırmızı durumunu düzeltmemiz gerekir. *renk atayarak ve/veya ağacı yeniden yapılandırarak ihlal.*
6. Silinecek bir düğüm içsel ise, sol alt ağacındaki en sağdaki düğümü bulun. Düğümdeki öğeyi en sağdaki düğümdeki öğe ile değiştirin. En sağdaki düğümü silin.

7. Silinecek dış düğüm kırmızı ise, dış düğümün ebeveyn düğümünü dış düğümün çocuk düğümü ile basitçe yeniden bağlayın.
8. Silinecek dış düğüm siyah ise, ağacın dış düğümlerindeki siyah yüksekliğin doğru bir şekilde korunmasını sağlamak için birkaç durumu göz önünde bulundurunuz gerekir.
9. Kırmızı-siyah ağacın yüksekliği  $O(\log n)$ 'dir. Bu nedenle, arama, eklem ve silme yöntemlerinin zaman karmaşıklıkları  $O(\log n)$ 'dir.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak [www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html) adresinde yapın.

### Gözden Geçirme Soruları

#### Bölümler 48.1-48.2

48.1

Kırmızı-siyah ağaç nedir? Dış düğüm nedir? Siyah derinlik nedir?

48.2

Kırmızı-siyah ağacın özelliklerini tanımlayın.

48.3

Kırmızı-siyah ağacı nasıl 2-4 ağacına dönüştürürsünüz? Dönüşüm benzersiz mi?

48.4

2-4 ağacını kırmızı-siyah ağaca nasıl dönüştürürsünüz? Dönüşüm benzersiz mi?

#### Bölümler 48.3-48.5

48.5

RBTreeNode'daki veri alanları nelerdir? \_\_\_\_\_

48.6

Kırmızı-siyah bir ağaçta bir öge nasıl eklenir ve çift-kırmızı ihlali nasıl düzeltilir?

48.7

Kırmızı-siyah bir ağaçtan bir öge nasıl silinir ve çift-siyah problemi nasıl düzeltilir?

48.8

Ağaçta 1, 2, 3, 4, 10, 9, 7, 5, 8, 6'yı bu sırayla eklediğinizde ağacın değişimini gösterin. \_ \_ \_

48.9

Önceki soruda oluşturulan ağaç için, 1, 2, 3, 4, 10, 9, 7, 5, 8, 6'yı bu sırayla sildikten sonraki ağacın değişimini gösterin. \_ \_ \_ \_ \_ \_ \_

### Programlama Alıştırmaları

48.1\*

(Kırmızı-siyah ağaçtan 2-4 ağacına) Kırmızı-siyah bir ağacı 2-4 ağacına dönüştüren bir program yazın.

48.2\*

(2-4 ağaçtan kırmızı-siyah ağaca) Kırmızı-siyah bir ağacı 2-4 ağacına dönüştüren bir program yazın.

48.3\*\*\*

(Kırmızı-siyah ağaç animasyonu) Kırmızı-siyah ağaç ekleme, silme ve arama yöntemlerini animasyonlu olarak gösteren bir Java applet'i yazın, Şekil 48.6'da gösterildiği gibi.

48.4\*\*

(RBTre için Ebeveyn referansı) BinaryTree'de tanımlanan TreeNode sınıfının, düğümün ebeveynine bir referans içerdiğini varsayın, Egzersiz 26.17'de gösterildiği gibi. Bu değişikliği desteklemek için RBTre sınıfını uygulayın. Sayıları ekleyen bir test programı yazın.  
1, 2, ..., 100'ü ağaca ekleyin ve tüm yaprak düğümlerinin yollarını gösterin.

\*\*\*Bu bir bonus Web bölümü

## BÖLÜM 49

Java'nın 2D'si

Hedefler

- Java 2D şekillerini render etmek için bir Graphics2D nesnesi elde etmek (§49.2) .
- Şekillerin modellemesini render etmekten ayırmak için geometrik modeller kullanmak (§49.3).
- Şekillerin hiyerarşisini bilmek (§49.3).
- Line2D, Rectangle2D, RoundRectangle2D, Ellipse2D ve Arc2D kullanarak çizgileri, dikdörtgenleri, elipsleri, yayları modellemek (§49.4).
- Koordinat dönüşümünü translate, rotate, \_\_\_\_\_  
ve scale yöntemlerini kullanarak gerçekleştirmek için (§49.5).
- BasicStroke sınıfını kullanarak çizgilerin özelliklerini belirtmek için \_\_\_\_\_  
(§49.6) .
- GradientPaint kullanarak değişken bir renk tanımlamak ve bir resmi TexturePaint kullanarak boyamak için (§49.7).
- QuadCurve2D ve CubicCurve2D sınıflarını kullanarak ikinci dereceden eğrileri ve üçüncü dereceden eğrileri modellemek için (§49.8).
- Path2D kullanarak keyfi bir geometrik yolu modellemek ve WIND\_EVEN\_ODD ve WIND\_NON\_ZERO kurallarını kullanarak iç noktaları tanımlamak için (§49.9).
- Area sınıfını kullanarak yapısal alan geometrisi gerçekleştirmek için (§49.10).

## 49.1 Giriş

Graphics sınıfındaki yöntemleri kullanarak, çizgiler, dikdörtgenler, ovalar, yaylar ve çokgenler çizmeyi öğrendiniz. Bu bölüm, gelişmiş ve karmaşık iki boyutlu grafikler çizmenizi sağlayan Java 2D'yi tanıtmaktadır.

NOT:

Bu bölüm, Java 2D'deki temel ve yaygın olarak kullanılan özellikleri tanıtmaktadır. Java 2D hakkında kapsamlı bilgi için, *Computer Graphics Using Java 2D and 3D* kitabına bakınız, yazarları Hong Zhang ve Y. Daniel Liang, Prentice Hall tarafından yayımlanmıştır.

## 49.2 Graphics2D Nesnesi Elde Etme

Metin içinde Graphics sınıfındaki çizim yöntemlerini kullandınız. Graphics sınıfı ilkel bir sınıftır. Java 2D API'si, grafiklerin işlenmesi için `java.awt.Graphics` sınıfını genişleten `java.awt.Graphics2D` sınıfını sağlar. Normalde, bir GUI bileşenin de grafik çizmek için kodu `paintComponent` yönteminde yazarsınız.

Yöntemin kodlama şablonu aşağıdaki gibidir:

```
korumalı voidt paintComponent(Graphics g) { nsupert
 .paintComponent(g);

 // Grafik çizmek için Graphics içindeki yöntemi kullanın
 ...
}
```

`paintComponent` yöntemine geçirilen parametre aslında bir `Graphics2D` örneğidir. Bu nedenle, bir `Graphics2D` referansı elde etmek için, parametre g'yi `Graphics2D`'ye aşağıdaki gibi basitçe dönüştürebilirsiniz:

```
korumalı voidt paintComponent(Graphics g) { nsupert
 .paintComponent(g);

 Graphics2D g2d = (Graphics2D)g; // Bir Graphics2D nesnesi alın

 // Grafik çizmek için Graphics2D içindeki yöntemi kullanın
 ...
}
```

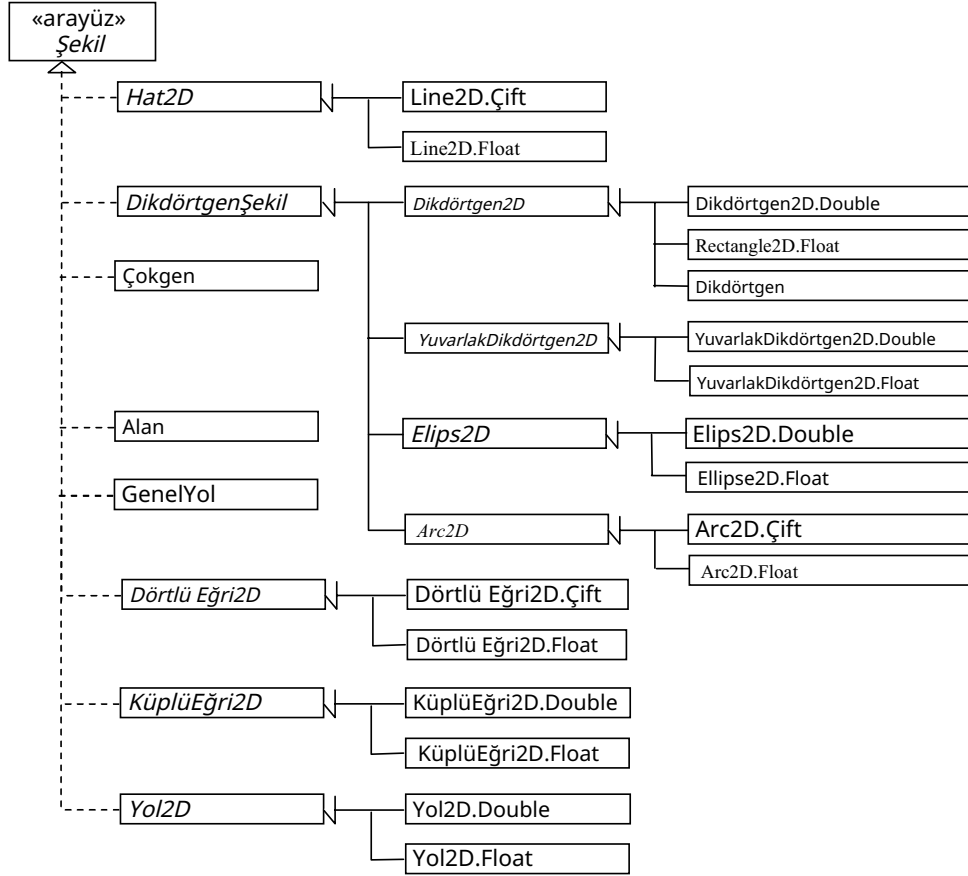
`Graphics2D`, `Graphics`'in bir alt sınıfı olduğundan, `Graphics`'teki tüm yöntemler `Graphics2D`'de kullanılabilir. Ayrıca, `Graphics2D` içindeki yöntemleri de kullanabilirsiniz.

## 49.3 Geometrik Modeller

Çizgiler, dikdörtgenler, yaylar, elipsler ve çokgenler çizmek için Graphics sınıfındaki yöntemleri kullandınız. Java 2D API'si, render etmeyi modellemeyi ayırmak için model-görünüm denetleyici mimarisini kullanır. Bu yaklaşım, şekiller oluşturmaya ve şekilleri birleştirmek için modeller kullanarak dönüştürme ve döndürme gibi manipülasyonlar gerçekleştirmenize olanak tanır ve şekilleri render etmek için `Graphics2D`'yi kullanmanızı sağlar.



Java 2D, temel şekilleri oluşturmak ve bunları daha karmaşık şekiller oluşturmak için birleştirmek için olanaklar sağlar. Şekil 49.1, Java 2D'yi destekleyen çeşitli şekilleri göstermektedir.



Şekil 49.1

Java 2D çeşitli şekilleri tanımlar.

**<Yan not: Shape içindeki yöntemler>**

Shape arayüzü, şekiller için ortak özellikleri tanımlar ve bir noktanın veya bir dikdörtgenin bir şeklin içinde olup olmadığını test etmek için contains yöntemini sağlar ve şeklin bir dikdörtgenle örtüşüp örtüşmediğini test etmek için intersects yöntemini sağlar; bu, Şekil 49.2'de gösterilmiştir. Bu yöntemler genellikle geometrik programlamada faydalıdır.

«arayüz» java.awt.Shape
+ıçerir(x: çift, y: çift): bool eann+ıçerir(x: çift, y: çift, w: double, h: double): boolean +contains(p: Point2D): boolean +contains(r: Rectangle2D): boolean +intersects(x: double, y: double, w: double, h: double): boolean n+int ersects(r: Dik açı2D): boolean n +getBounds2D(): Rectangle2D

Belirtilen koordinatların şeklin içinde olup olmadığını test eder.  
Belirtilen üst-sol köşesi (x, y), genişliği w ve yüksekliği h olan dik dörtgenin şeklin içinde olup olmadığını test eder.  
Belirtilen Point2D'nin şeklin içinde olup olmadığını test et.  
Belirtilen Rectangle2D'nin şeklin içinde olup olmadığını test eder.  
Belirtilen üst sol köşesi (x, y), genişliği w ve yüksekliği h olan dikdörtgenin bu şekil ile kesişip kesişmediğini test eder.  
Belirtilen Rectangle2D'nin bu şekil ile kesişip kesişmediğini test eder.  
Şekli kapsayan bir sınırlayıcı dikdörtgen döndürür.

Şekil 49.2

*Shape, tüm Java 2D şekilleri için kök arayüzdür.*

#### <Yan not: somut şekil sınıfları>

Line2D, Rectangle2D, RoundRectangle2D, Arc2D, Ellipse2D, QuadCurve2D, Cubic Curve2D ve Path2D sınıfları soyut sınıflardır. Her biri double ve float koordinatları için sırasıyla Double ve Float adında iki somut statik iç sınıf içerir. Örneğin, Line2D.DoubleLine2D sınıfında tanımlanan Double statik iç sınıfına atıfta bulunur.

Bir nesne oluşturmak için Line2D.Double veya Line2D.Float kullanabilirsiniz, double veya float koordinatları kullanmak isteğinize bağlı olarak. Bu iç sınıflar ayrıca kendi d iş sınıflarının alt sınıflarıdır. Yani Line2D.Double, Line2D'nin bir alt sınıfıdır.

#### <Yan not: Point2D>

Bir nokta, soyut Point2D sınıfı kullanılarak modellenenebilir. Bu, sırasıyla double ve float koordinatları için iki somut statik iç sınıf olan Point2D.Double ve Point2D.Float içerir. Point2D.Double ve Point2D.Float, Point2D'nin alt sınıflarıdır. Point sınıfı, JDK 1.1'de tanıtılmıştır ve şimdi geriye dönük uyumluluk için Java 2D'ye dahil edilmiştir. Point artık Point2D'nin bir alt sınıfı olarak tanımlanmaktadır. Point2D, iki nokta arasındaki mesafeyi bulmak için yöntemleri içerir.

#### <Yan not: bir şekil oluşturun>

Bir şekil oluşturmak için, somut bir şekil sınıfının yapıcısını kullanın. Örneğin, (x1, y1) ile (x2, y2) arasında bir çizgi modellemek için, aşağıdaki yapıcıyı kullanarak double veri türünde bir Line2D nesnesi oluşturabilirsiniz:

#### <Yan not: bir çizgi oluşturun>

Doğru2D doğru = t newt Doğru2D.Double(x1, y1, x2, y2);

Graphics2D sınıfı, şeklin sınırını çizmek için draw(Shape s) yöntemini ve şeklin içini doldurmak için fill(Shape s) yöntemini içerir. Bir GUI bileşeninde çizgiyi render etmek için,

#### <Yan not: bir çizgiyi render edin>

g2d.draw(satır);

burada g2d, GUI bileşeni için bir Graphics2D nesnesidir.

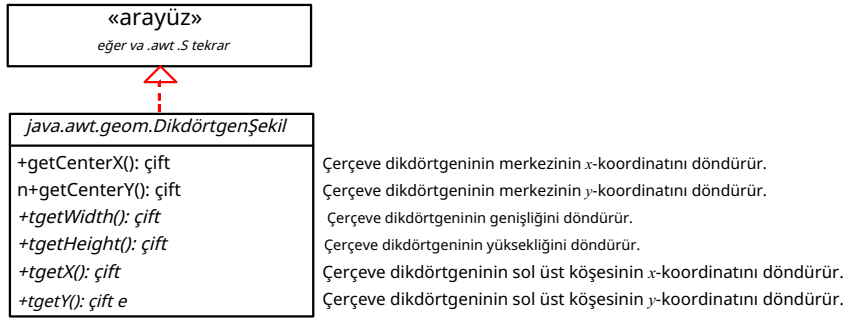
### 49.4 Rectangle2D, RoundRectangle2D, Arc2D ve Ellipse2D

#### <Yan notu: DikdörtgenŞekil>

DikdörtgenŞekil, Rectangle2D, RoundRectangle2D, Arc2D ve Ellipse2D için soyut bir temel sınıftır, geometrisi bir

dikdörtgen çerçeve ile tanımlanır. Şekil 49.3,

DikdörtgenŞekil için UML diyagramını göstermektedir.



Şekil 49.3

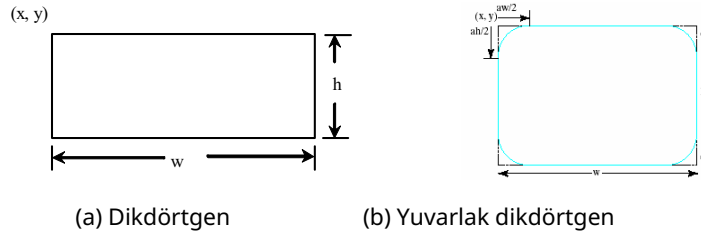
*DikdörtgenŞekil, bir sınırlayıcı dikdörtgen ile bir şekil tanımlar.*

#### <Yan not: Rectangle2D>

Rectangle2D, yatay ve dikey kenarları olan bir dikdörtgeni modellemektedir. Rectangle sınıfı JDK 1.1'de tanıtılmıştır ve şimdi geri uyumluluk için Java 2D'ye dahil edilmiştir. Rectangle artık Rectangle2D'nin bir alt sınıfı olarak tanımlanmaktadır. Tam sayılı koordinatlara sahip bir dikdörtgeni modellemektedir, Rectangle2D.Double ve Rectangle2D.Float ise sırasıyla double ve float koordinatlara sahip bir dikdörtgeni modellemektedir. Bir Rectangle oluşturabilirsiniz

```
new Rectangle(x, y, w, h)
```

x ve y parametreleri dikdörtgenin sol üst köşesini temsil eder, w ve h ise genişliği ve yüksekliğidir (bkz. Şekil 49.4a).



Şekil 49.4

- (a) Bir dikdörtgen dört parametre ile tanımlanır. (b) Bir yuvarlak dikdörtgen altı parametre ile tanımlanır.

Aşağıdaki kod, sırasıyla tam sayı, double ve float koordinatları ile üç Rectangle2D nesnesi oluşturur. Dikdörtgenin sol üst köşesi (20, 40) noktasında olup, genişliği 100 ve yüksekliği 200'dir.

```
Rectangle2D ri = new Rectangle(20, 40, 100, 200);
Rectangle2D rd = new Rectangle.Double(20D, 40D, 100D, 200D);
Rectangle2D rf = new Rectangle.Double(20F, 40F, 100F, 200F);
```

#### <Yan not: RoundRectangle2D>

RoundRectangle2D, yuvarlak köşeleri olan bir dikdörtgeni modellemektedir. Bir RoundRectangle oluşturmak için

```
new RoundRectangle2D.Double(x, y, w, h, aw, ah)
```

Parametreler  $x$ ,  $y$ ,  $w$  ve  $h$  bir dikdörtgeni belirtir, parametre  $aw$  köşelerdeki yayların yatay çapı ve  $ah$  köşelerdeki yayların dikey çapıdır (bkz. Şekil 49.4(b)). Diğer bir deyişle,  $aw$  ve  $ah$ , her köşede bir çeyrek daire üreten ovalin genişliği ve yüksekliğidir.

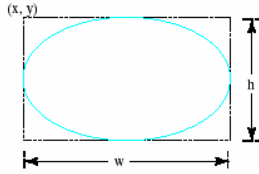
#### <Yan not: Ellipse2D>

Ellipse2D bir elipsi modellemektedir. Bir Ellipse2D'yi aşağıdaki gibi oluşturabilirsiniz

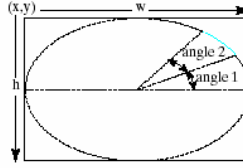
```
new Ellipse2D.Double(x, y, w, h)
```

Parametreler  $x$ ,  $y$ ,  $w$  ve  $h$ ,

elipsin sınırlayıcı dikdörtgenini belirtir; Şekil 49.5a'da gösterildiği gibi.



(a) Elips



(b) Yay

Şekil 49.5

Bir elips veya oval, sınırlayıcı dikdörtgeni ile tanımlanır.

#### <Yan not: Arc2D>

Arc2D, eliptik bir yayı modellemektedir. Bir Arc2D oluşturabilirsiniz

```
new Arc2D.Double(x, y, w, h, startAngle, arcAngle, type)
```

Parametreler  $x$ ,  $y$ ,  $w$  ve  $h$ , yay için sınırlayıcı dikdörtgeni belirtir; parametre  $startAngle$  başlangıç açısıdır;  $arcAngle$ , yayı kapsayan açı (yani, yay tarafından kaplanan açı) olarak tanımlanır. Açılar ölçümü dereceler cinsindendir ve genellikle matematiksel kurallara uyar (yani, 0 derece doğu yönündedir ve pozitif açılar doğu yönünden saat yönünün tersine dönüşü gösterir); Şekil

49.5(b)'ye bakınız.

Parametre türü Arc2D.OPEN, Arc2D.CHORD veya Arc2D.PIE'dir. Arc2D.OPEN yayın açık olduğunu belirtir. Arc2D.CHORD, yayının başlangıcından yayının sonuna bir doğru parçası çizerek bağlandığını belirtir. Arc2D.PIE, yayın başlangıç noktasından tam elipsin merkezine ve o noktadan yay segmentinin sonuna doğru düz doğru parçaları çizerek bağlandığını belirtir.

Liste 49.1, Graphics2D kullanarak çeşitli şekillerin nasıl çizileceğini gösteren bir programı vermektedir. Şekil 49.6, programın örnek bir çalışmasını göstermektedir.

#### Liste 49.1 Graphics2DDemo.java

#### <Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>  
 <Yan not: Satır 14: Graphics2D referansı>  
 <Yan not: Satır 16: bir çizgi çiz>  
 <Yan not: Satır 17: bir dikdörtgen çiz>  
 <Yan not: Satır 18: bir dikdörtgen doldur>  
 <Yan not: Satır 19: yuvarlak dikdörtgen>  
 <Yan not: Satır 20: bir elips çiz>  
 <Yan not: Satır 21: bir yay çiz>  
 <Yan not: Satır 28: ana metot atlandı>

```
java.auto.*'yu içe aktarın;
```

```
java.awt.geom.*'u içe aktar
```

```
;import javax.swing.*;
```

```
genel sınıf Graphics2DDemo, JApplet'i genişletir {n pu
```

```
blic Graphics2DDemo() {
```

```
add(new ShapePanel());n }
```

```
statik sınıf ShapePanel, JPanel'i genişletir {n pro
```

```
tected void paintComponent(Graphics g) {n super.p
```

```
aintComponent(g);
```

```
Grafik2D g2d = (Grafik2D)g;
```

```
g2d.draw(new Line2D.Double(10, 10, 40, 80));n g2d.dr
```

```
aw(new Rectangle2D.Double(50, 10, 30, 70));n g2d.fill(new R
```

```
ectangle2D.Double(90, 10, 30, 70));n g2d.fill(new RoundRect
```

```
angle2D.Double(130, 10, 30, 70, 20, 30));n g2d.draw(new Ellipse2D.Double(17
```

```
0, 10, 30, 70));
```

```
g2d.draw(
```

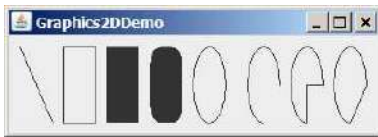
```
new Arc2D.Double(220, 10, 30, 70, 0, 270, Arc2D.OPEN));n g2d.draw(
```

```
new Arc2D.Double(260, 10, 30, 70, 0, 270, Arc2D.PIE));n g2d.draw(
```

```
 yeni Arc2D.Double(300, 10, 30, 70, 0, 270, Arc2D.CHORD));n }
```

```
}
```

```
}
```



Şekil 49.6

Java 2D kullanarak çeşitli şekiller çizebilirsiniz.

Şekil sınıfları Line2D, Rectangle2D, RoundRectangle2D, Arc2D ve Ellipse2D java.awt.geom paketindedir. Bu nedenle, 2. satırda içe aktarılmışlardır.

<Yan not: Line2D>

Graphics2D yöntemlerini çağırarak için 14. satırda bir Graphics2D referansı elde edilir. new Line2D.Double(10, 10, 40, 80) ifadesi (16. satır) Line2D.Double'in bir örneğin i oluşturur, bu da aynı zamanda bir

Line2D ve Shape örneğidir. Bu örnek, (10, 10) noktasından (40, 80) noktasına bir çizgi modellemektedir. \_\_\_\_

**<Yan not: Rectangle2D>**

new Rectangle2D.Double(50, 10, 30, 70) ifadesi (17. satır) Rectangle2D.Double'ın bir örneğini oluşturur, bu da aynı zamanda bir örnektir.

Rectangle2D ve Shape. Örnek, sol üst köşe noktası (50, 10) olan ve genişliği 30, yüksekliği 70 olan bir dikdörtgeni modellemektedir. \_\_\_\_

**<Yan not: doldurma>**

doldurma(Shape) metodu (satır 18) dolu bir dikdörtgen çizer.

**<Yan not: RoundRectangle2D>**

new RoundRectangle2D.Double(130, 10, 30, 70, 20, 30) ifadesi (satır 19) RoundRectangle2D.Double'ın bir örneğini oluşturur, bu da RoundRectangle2D ve Shape'ın bir örneğidir. Örnek, parametreleri Graphics sınıfındaki drawRoundRect(int x, int y, int w, int h, int aw, int ah) metodundaki ile aynı olan yuvarlak köşeli bir dikdörtgeni modellemektedir. \_\_\_\_\_

**<Yan notu: Ellipse2D>**

Yeni Ellipse2D.Double(300, 10, 30, 70) ifadesi (satır 20)

Ellipse2D.Double'ın bir örneğini oluşturur, bu da Ellipse2D ve Shape'ın bir örneğidir. Örnek bir elipsi modellemektedir. Bu yapıcısındaki parametreler, Graphics sınıfındaki drawOval(int x, int y, int w, int h) metodundaki parametrelerle aynıdır. \_\_\_\_\_

**<Yan notu: Arc2D>**

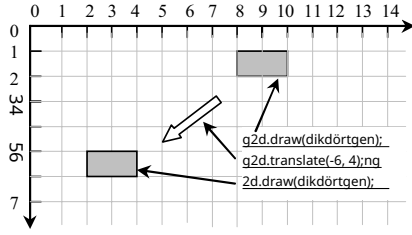
Yeni Arc2D.Double(170, 10, 30, 70, 0, 270, Arc2D.OPEN) (ifade 21) Arc2D.Double'ın bir örneğini oluşturur, bu da Arc2D ve Shape'ın bir örneğidir. Örnek açık bir yay modellemektedir. Bu yapıcısındaki parametreler, Graphics sınıfındaki drawArc(int x, int y, int w, int h, int startAngle, int arcAngle) metodundaki parametrelerle benzerdir, tek fark son parametrenin yayın açık mı kapalı mı olduğunu belirtmesidir. Arc2D.OPEN değeri yayın açık olduğunu belirtir. Arc2D.PIE değeri (satır 23) yayın, yay segmentinin başlangıcından tam elipsin merkezine ve o noktadan yay segmentinin sonuna doğru düz çizgi segmentleri çizerek kapandığını belirtir. Arc2D.CHORD değeri (satır 25) yayın, yay segmentinin başlangıcından yay segmentinin sonuna doğru düz bir çizgi segmenti çizerek kapandığını belirtir. \_\_\_\_\_

## 49.5 Koordinat Dönüşümleri

Java 2D, geometrik nesneleri modellemek için sınıflar sağlar. Ayrıca, çeviri, döndürme ve ölçeklendirme kullanarak koordinat dönüşümlerini destekler.

### 49.5.1 Çeviriler

Graphics sınıfındaki translate(double x, double y) metodunu kullanarak, belirtilen mesafe kadar sonraki render'ı önceki konuma göre hareket ettirebilirsiniz. Örneğin, translate(5, -10) sonraki render'ı 5 piksel sağa ve 10 piksel yukarı hareket ettirirken, translate(-5, 10) tüm şekilleri 5 piksel sola ve 10 piksel aşağı hareket ettirir. Şekil 49.7, çeviri uygulandıktan önce ve sonra gösterilen bir dikdörtgeni göstermektedir. g2d.translate(-6, 4) çağırıldıktan sonra, dikdörtgen önceki konumdan 6 piksel sola ve 4 piksel aşağıda görüntülenir. \_\_\_\_\_



Şekil 49.7

(a) `g2d.translate(-6, 4)` uygulandıktan sonra, `dikdörtgen`in sonraki render'ı belirtilen mesafe kadar önceki konuma göre hareket ettirilmiştir.

Liste 49.2, koordinatların çevirisinin etkisini gösteren bir programı gösterir. Şekil 49.8, programın bir örnek çalışmasını göstermektedir.

#### Liste 49.2 TranslationDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>

<Yan not: Satır 14: Graphics2D referansı>

<Yan not: Satır 15: bir `dikdörtgen`>

<Yan not: Satır 17: rastgele sayı>

<Yan not: Satır 19: yeni bir renk ayarla>

<Yan not: Satır 21: `dikdörtgeni` göster>

<Yan not: Satır 22: çevir>

<Yan not: Satır 26: ana metod atlandı>

```
java.auto.*'yu içe aktarın;
java.awt.geom.*'u içe aktar
;import javax.swing.*;
```

```
genel sınıf TranslateDemo, JApplet'i genişletir {n pub
lic TranslateDemo() {
 add(new ShapePanel());n }
```

```
class ShapePanel, JPanel'i genişletir {n
protected void paintComponent(Graphics g) {n sup
er.paintComponent(g);
```

```
Grafik2D g2d = (Grafik2D)g;
Rectangle2D dikdörtgen = new Rectangle2D.Double(10, 10, 50, 60);
```

```
java.util.Random rastgele = new java.util.Random();n for (i
nt i = 0; i < 10; i++) {
 g2d.setColor(new Color(random.nextInt(256), n ran
dom.nextInt(256), random.nextInt(256)));n g2d.draw
(dikdörtgen);
 g2d.translate(20, 5);n }
```

```
}
N);
```



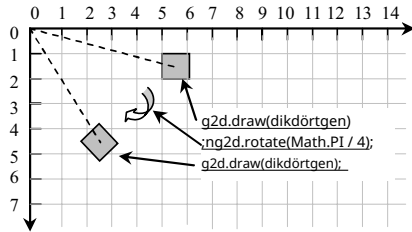
Şekil 49.8

*Dikdörtgenler ardışık olarak yeni konumlarda görüntülenir.*

17. satır bir Random nesnesi oluşturur. Random sınıfı 88.6.2, "Random Sınıfı" bölümünde tanıtılmıştır. `random.nextInt(256)` (19. satır) 0 ile 255 arasında rastgele bir int değeri döndürür. `setColor` metodu (19. satır) sonraki render işlemleri için yeni bir renk ayarlar. 21. satır bir dikdörtgen çizer. 22. satırdaki `translate(20, 5)` metodu, sonraki render işlemini 20 piksel sağa ve 5 piksel aşağı taşır.

#### 49.5.2 Dönmeler

Graphics2D sınıfındaki `rotate(double theta)` metodunu kullanarak sonraki renderlamayı orijinden saat yönünde theta derece döndürebilirsiniz, burada theta, radyan cinsinden bir double değerdir. Varsayılan olarak orijin (0, 0) noktasıdır. Orijini belirli bir konuma taşımak için `translate(x, y)` metodunu kullanabilirsiniz. Örneğin, `rotate(Math.PI / 4)` sonraki renderlamayı orijinden kuzey yönünde saat yönünün tersine 45 derece döndürür; bu, Şekil 49.9'da gösterilmiştir.



Şekil 49.9

*`g2d.rotate(Math.PI / 4)` işlemi yapıldıktan sonra, dikdörtgen orijinden 45 derece döndürülmüştür.*

Liste 49.3, koordinatların döndürülmesinin etkisini gösteren bir programı vermektir. Şekil 49.10, programın örnek bir çalışmasını göstermektedir.

#### Liste 49.3 RotationDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>  
 <Yan not: Satır 5: applet>  
 <Yan not: Satır 14: Graphics2D referansı>  
 <Yan not: Satır 15: bir dikdörtgen>  
 <Yan not: Satır 17: yeni orijin>  
 <Yan not: Satır 18: merkez noktasını çiz>  
 <Yan not: Satır 19: rastgele sayı>



<Yan notu: Satır 21: yeni bir renk ayarla>

<Yan notu: Satır 23: dikdörtgeni göster>

<Yan notu: Satır 24: döndür>

<Yan notu: Satır 28: ana metod atlandı>

```
java.auto.*'yu içe aktarın;
java.awt.geom.*'u içe aktar
;import javax.swing.*;
```

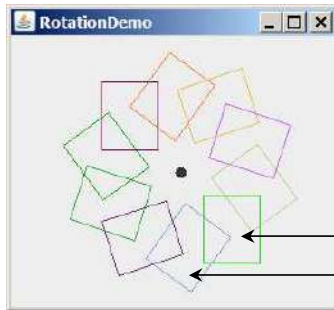
```
genel sınıf RotationDemo, JApplet'i genişletir {n pu
blic RotationDemo() {
add(new ShapePanel());n }
```

```
class ShapePanel, JPanel'i genişletir {n
protected void paintComponent(Graphics g) {n sup
er.paintComponent(g);
```

```
Grafik2D g2d = (Grafik2D)g;
Rectangle2D dikdörtgen = new Rectangle2D.Double(20, 20, 50, 60);
```

```
g2d.translate(150, 120); // Orijini merkeze taşı
g2d.fill(new Ellipse2D.Double(-5, -5, 10, 10));
java.util.Random rastgele = new java.util.Random();
for (int i = 0; i < 10; i++) {
g2d.setColor(new Color(rastgele.nextInt(256),
rastgele.nextInt(256), rastgele.nextInt(256)));
g2d.draw(dikdörtgen);
g2d.rotate(Math.PI / 5);n }
```

```
}
N }
}
```



Bu dikdörtgen önce çizilir

Bu dikdörtgen sonraki olarak çizilir

Şekil 49.10

*rotate metodu çağırıldıktan sonra, dikdörtgenler yeni konumlarda art arda görüntülenir.*

translate(150, 120) metodu orijini (0, 0) konumundan (150, 120) konumuna taşır (satır 17). Döngü on kez tekrarlanır. Her yineleme rastgele yeni bir renk ayarlar (satır 21), dikdörtgeni çizer (satır 23) ve yeni orijinden 36 derece döner (satır 24).

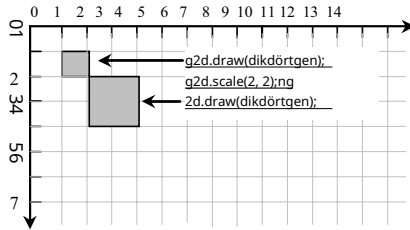
### 49.5.3 Ölçekleme

## Graphics2D

sınıfındaki `scale(double sx, double sy)` metodunu, belirtilen ölçekleme faktörleriyle sonraki render'ları yeniden boyutlandırmak için kullanabilirsiniz.

Örneğin, `scale(2, 2)` nesneyi *x*- ve *y*-

koordinatlarını iki katına çıkararak yeniden boyutlandırır, Şekil 49.11'de gösterildiği gibi.



Şekil 49.11

*g2d.scale(2, 2) uygulandıktan sonra, orijinal dikdörtgendeki x- ve y-koordinatları iki katına çıkar.*

Liste 49.4, ölçeklendirme kullanmanın etkisini gösteren bir programı vermektir. Şekil 49.12, programın bir örnek çalışmasını göstermektedir.

### Liste 49.4 ScalingDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>

<Yan not: Satır 14: Graphics2D referansı>

<Yan not: Satır 15: bir dikdörtgen>

<Yan not: Satır 18: dikdörtgeni göster>

<Yan not: Satır 19: ölçek>

<Yan not: Satır 23: ana metod atlandı>

```
java.auto.*'yu içe aktarın;
import java.awt.geom.*;ni
import javax.swing.*;npu
```

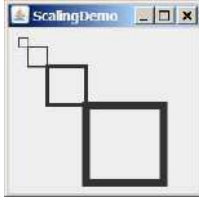
```
blin sınıfı ScalingDemo, JApplet'i genişletir {n publi
c ScalingDemo() {
 add(new ShapePanel());n }
```

```
class ShapePanel, JPanel'i genişletir {n
protected void paintComponent(Graphics g) {n sup
er.paintComponent(g);
```

```
Grafik2D g2d = (Grafik2D)g;
Rectangle2D dikdörtgen = new Rectangle2D.Double(10, 10, 10, 10);
```

```
for (int i = 0; i < 4; i++) {n g2d.draw(r
ectangle);
g2d.scale(2, 2);n }
```

```
}
N }
}
```



Şekil 49.12

*Ölçeklendirme uygulandıktan sonra, dikdörtgenler ardışık olarak görüntülenir.*

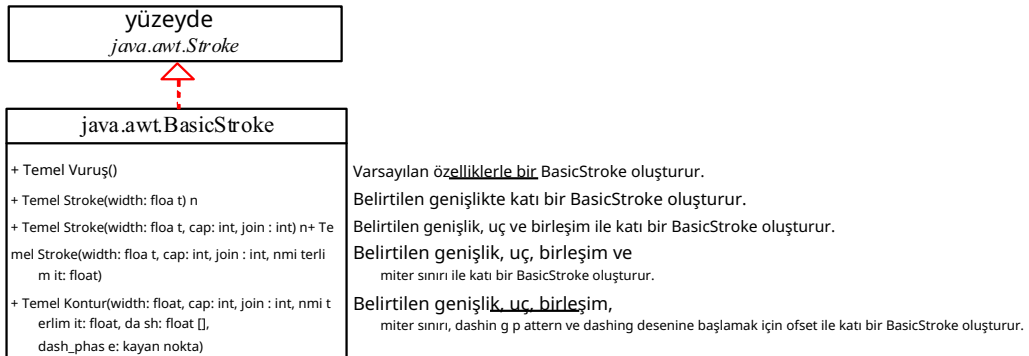
Program dört dikdörtgen çizer. İlk dikdörtgenin sol üst köşesi (10, 10) konumunda dır. Döngünün ilk yinelemesinde Graphics2D nesnesi g2d üzerinde scale(2, 2) (satır 19) çağırıldıktan sonra, ikinci dikdörtgenin sol üst köşesi (20, 20) konumundadır, çünkü bu ölçekleme metodu mevcut nesnedeki koordinatları iki katına çıkarır.

Döngünün ikinci yinelemesinde Graphics2D nesnesi g2d üzerinde scale(2, 2) (satır 19) çağırıldıktan sonra, üçüncü dikdörtgenin sol üst köşesi (40, 40) konumundadır. Döngünün üçüncü yinelemesinde Graphics2D nesnesi g2d üzerinde scale(2, 2) (satır 19) çağırıldıktan sonra, dördüncü dikdörtgenin sol üst köşesi (80, 80) konumunda dır.

Çizgi segmentlerinin kalınlığının, her seferinde scale(2, 2) çağrıldığında iki katı na çıktığını belirtmek ilginçtir. Çizgilerin kalınlığını bir sonraki bölümde tartışacağız.

## 49.6 Vuruş

Java 2D, çizgilerin özelliklerini belirtmenize olanak tanır, bunlara *vuruşlar* denir. Çizginin genişliğini, çizgi uçlarının nasıl olduğunu (uç kapakları olarak adlandırılır), çizgilerin nasıl birleştiğini (çizgi birleşimleri olarak adlandırılır) ve çizginin kesiği olup olmadığını belirtebilirsiniz. Bu özellikler bir Stroke nesnesinde tanımlanır. Aşağıda gösterildiği gibi, BasicStroke sınıfını kullanarak bir Stroke nesnesi oluşturabilirsiniz.



Şekil 49.13

*Bir Stroke, BasicStroke sınıfını kullanarak oluşturabilirsiniz.*

width parametresi, stroke'un kalınlığını belirtir ve varsayılan değeri 1.0'dır.

cap parametresi üç değerden biridir:

- BasicStroke.CAP\_ROUND yuvarlak uç için.
- BasicStroke.CAP\_SQUARE kare uç için.
- BasicStroke.CAP\_BUTT ek dekorasyon olmadan.

Parametre join, üç değerden biridir:

- BasicStroke.JOIN\_BEVEL, geniş kenarlarının dış köşelerini düz bir segmentle birleştirmek için.
- BasicStroke.JOIN\_MITER, yol segmentlerini dış kenarlarını uzatarak birleştirmek için.
- BasicStroke.JOIN\_ROUND, yol segmentlerini köşeyi çizgi genişliğinin yarıçapında yuvarlayarak birleştirmek için.

Parametre miterlimit, iki çizgi arasındaki açı küçük olduğunda çok uzun bir birleştirme önlemek için JOIN\_MITER için bir sınır belirler.

Parametre dash array, opak ve şeffaf bölümler arasında geçiş yaparak bir dash desenini tanımlar. dash\_phase parametresi, dashing desenine başlamak için ofseti belirtir.

Graphics2D' de bir stroke ayarlamak için, voi

d setStroke(Stroke stroke) kullanın.

Liste 49.5, temel stroke'ların etkisini gösteren bir programı vermektedir. Şekil 49.14, programın bir örnek çalışmasını göstermektedir.

#### Liste 49.5 BasicStrokeDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>

<Yan not: Satır 14: Graphics2D referansı>

<Yan not: Satır 16: bir stroke ayarla>

<Yan not: Satır 18: bir çizgi çiz>

<Yan not: Satır 20: çevir>

<Yan not: Satır 31: bir dikdörtgen çiz>

<Yan not: Satır 49: ana metod atlandı>

```
java.auto.*'yu içe aktarın;
```

```
java.awt.geom.*'u içe aktar
```

```
;import javax.swing.*;
```

```
genel sınıf BasicStrokeDemo, JApplet'i genişletir {n pub
```

```
lic BasicStrokeDemo() {
```

```
add(new ShapePanel());
```

```
}
```

```
class ShapePanel, JPanel'i genişletir {n
```

```
protected void paintComponent(Graphics g) {
```

```
super.paintComponent(g);
```

```
Graphics2D g2d = (Graphics2D)g; n g2
```

```
d.setStroke(new BasicStroke(15.0f, BasicStroke.CAP_ROUND,
```

```

 BasicStroke.JOIN_BEVEL));n g2
d.draw(new Line2D.Double(10, 10, 40, 80));

g2d.translate(100, 0);
g2d.setStroke(new BasicStroke(15.0f, BasicStroke.CAP_SQUARE, n BasicS
troke.JOIN_BEVEL));
g2d.draw(new Line2D.Double(10, 10, 40, 80));

g2d.translate(100, 0);
g2d.setStroke(new BasicStroke(15.0f, BasicStroke.CAP_BUTT, n BasicSt
roke.JOIN_BEVEL));
g2d.draw(new Line2D.Double(10, 10, 40, 80));

g2d.translate(100, 0);
g2d.draw(new Rectangle2D.Double(10, 10, 30, 70));

g2d.translate(100, 0);
g2d.setStroke(new BasicStroke(15.0f, BasicStroke.CAP_ROUND, n Basic
Stroke.JOIN_MITRE));
g2d.draw(new Rectangle2D.Double(10, 10, 30, 70));

g2d.translate(100, 0);
g2d.setStroke(new BasicStroke(15.0f, BasicStroke.CAP_SQUARE, n BasicS
troke.JOIN_ROUND));
g2d.draw(new Rectangle2D.Double(10, 10, 30, 70));

g2d.translate(100, 0);
g2d.setStroke(new BasicStroke(4.0f, BasicStroke.CAP_SQUARE,
BasicStroke.JOIN_ROUND, 1.0f, yeni kayan nokta[{8}, 0]);
g2d.draw(new Line2D.Double(10, 10, 40, 80)); N }

}
}

```



Şekil 49.14

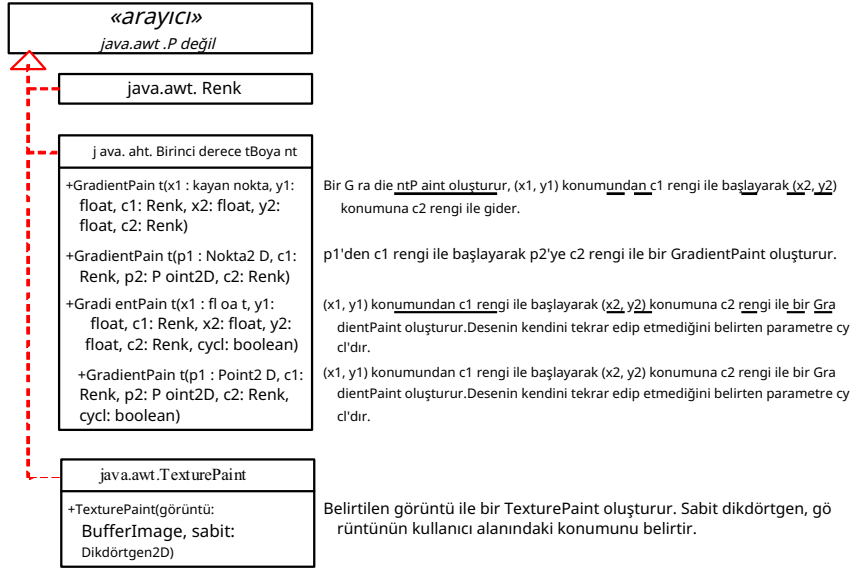
*Çizgiler için özellikleri belirtebilirsiniz.*

İfade `new BasicStroke(15.0f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_BEVEL)` (satır 16) bir BasicStroke örneği oluşturur, aynı zamanda Stroke arayüzünün de bir örneğidir. `setStroke(Stroke)` metodu, Graphics2D bağlamı için bir Stroke nesnesi ayarlar. Program, satır 21, 26, 34, 39, 49'da yeni Stroke nesneleri ayarlar. Satır 44, genişliği 4.0f, yuvarlak kare kap, yuvarlak birleşim, miter limiti 1.0, dashing deseni {8} ve dash fazı 0 olan yeni bir Stroke nesnesi ayarlar.

## 49.7 Boya

Renk ayarlamak için Graphics sınıfındaki setColor(Color c) metodunu kullanabilirsiniz. Bu yalnızca düz bir rengi ayarlar. Graphics2D, bir boyayı ayarlamak için setPaint(Paint p) metodunu sağlar. Boya, rengin genelleştirilmesidir. Basit düz renklerden daha fazla özelliği temsil edebilir.

Boya, Şekil 49.15'te gösterildiği gibi Color dahil üç somut sınıf için bir arayüzdür.

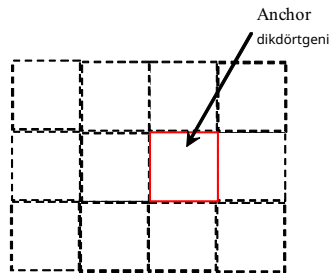


Şekil 49.15

Bir Paint nesnesi renkleri belirtir.

GradientPaint, iki nokta ve iki renk ile belirtilen değişken bir rengi tanımlar. Konum, ilk noktadan ikinci noktaya hareket ettikçe, paylaşım ilk renkten ikinci renge yavaşça değişir. A GradientPaint döngüsel veya döngüsel olmayan olabilir. Döngüsel bir boya, aynı deseni periyodik olarak tekrarlar.

TexturePaint bir şekli veya karakterleri doldurmak için bir görüntü tanımlar. parametre görüntü BufferedImage olarak belirtilir. Ankrāj parametresi, görüntünün sabitlendiği bir dikdörtgeni belirtir. Görüntü, ankrāj dikdörtgeninin etrafında tekrar edilir, Şekil 49.16'da gösterildiği gibi.



Şekil 49.16

Bir TexturePaint, bir ankray dikdörtgenindeki bir görüntü ile belirtilir.

Liste 49.6, GradientPaint ve TexturePaint kullanmanın etkisini gösteren bir programı verir. Şekil 49.17, programın bir örnek çalışmasını gösterir.

#### Liste 49.6 PaintDemo.java

<Yan not: Satır 17: GradientPaint>

<Yan not: Satır 29: düz renk>

<Yan not: Satır 35: URL al>

<Yan not: Satır 36: TexturePaint>

<Yan not: Satır 39: boya ayarla>

<Yan not: Satır 60: ana metot>

```
import java.awt.*;ni
import java.awt.geom.*;nim
import javax.imageio.ImageIO;nimpo
import javax.swing.*;

genel sınıf PaintDemo, JApplet'i genişletir {n pu
 public PaintDemo() {
 add(new ShapePanel());n }
}

class ShapePanel, JPanel'i genişletir {n
 protected void paintComponent(Graphics g) {n sup
 super.paintComponent(g);

 Graphics2D g2d = (Graphics2D)g;n g2

 d.setPaint(new GradientPaint(10, 10, Color.RED, 40, 40,n Color.BLUE, t
 rue));
 g2d.fill(new Rectangle2D.Double(10, 10, 90, 70));n g2d.setFo
 nt(new Font("Serif", Font.BOLD, 50));n g2d.drawString("
 GradientPaint", 10, 120);

 g2d.translate(100, 0);
 g2d.setPaint(new GradientPaint(10, 10, Renkli.SARI, 40, 40,n Renkli.SİYAH
));
 g2d.fill(new Rectangle2D.Double(10, 10, 90, 70));

 g2d.translate(100, 0);
 g2d.setPaint(Color.YELLOW);n g2
 d.fill(new Rectangle2D.Double(10, 10, 90, 70));

 denemek {
 java.net.URL URL'si =
 getClass().getClassLoader().getResource("image/ca.gif");n java.awt.i
 mage.BufferedImage image = ImageIO.read(url);n TexturePaint T
 exturePaint = new TexturePaint(image,n new Rectangle2D.D
 ouble(10, 10, 100, 70));
 g2d.translate(130, 0);
 g2d.setPaint(texturePaint);n g2
 d.fill(new Ellipse2D.Double(10, 10, 100, 70));
```

```

 doku Boyası = yeni Doku Boyası(görüntü, yeni
 Rectangle2D.Double(10, 10, 50, 70));n g2d.transl
 ate(110, 0);
 g2d.setPaint(texturePaint);n g2
 d.fill(new Ellipse2D.Double(10, 10, 100, 70));n TexturePaint
 = new TexturePaint(image,
 new Rectangle2D.Double(10, 10, 50, 35));n g2d.tr
 anslate(110, 0);
 g2d.setPaint(texturePaint);n g2
 d.fill(new Ellipse2D.Double(10, 10, 100, 70));n g2d.drawStri
 ng("TexturePaint", -190, 120);
 }
 catch (java.io.IOException ex) {n ex.pri
 ntStackTrace();
 }
 N }
}
}
}

```



Şekil 49.17

*Şekiller ve karakterler, gradyan boya, düz renk ve doku boyası ile çizilir.*

17-18. satırlardaki ifade

```

g2d.setPaint(tnewt GradientPaint(t10t,t 10t, Color.RED, t 40t,t 40t,n
Color.BLUE, t true));

```

GradientPaint'in bir örneğini oluşturur ve g2d'deki boyayı ayarlar.

Program, dolgu dikdörtgeni çizmeye başlamadan önce yeni bir Paint nesnesi (17, 24, 29. satırlar) ayarlar. setPaint metodunu bir Color nesnesi (29. satır) ayarlamak için veya Graphics sınıfındaki setColor metodunu kullanarak bir renk ayarlamak için kullanabileceğinizi unutmayın.

Şekil 49.17'de gördüğünüz gibi, gradyan renkleri ilk dikdörtgende tekrarlanır, çünkü GradientPaint döngüseldir (17-18. satırlar). Gradyan renkleri ikinci dikdörtgende tekrarlanmaz, çünkü GradientPaint döngüsel değildir (24-25. satırlar).

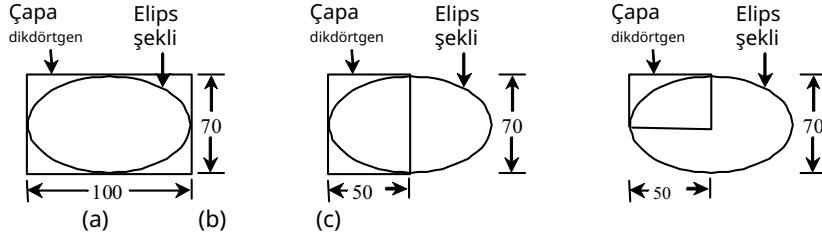
Bir TexturePaint oluşturmak için, bir resim dosyasından bir BufferedImage oluşturmanız gerekir. Resim dosyasının URL'si 33-49. satırlarda oluşturulur. Bu konu, §18.10, "URL Sınıfını Kullanarak Kaynakları Bulma"da tanıtılmıştır. Resim URL'sinden bir BufferedImage elde etmek için ImageIO sınıfındaki static read metodunu kullanabilirsiniz (35. satır).



36-37. satırlardaki ifade

```
TexturePaint texturePaint = new TexturePaint(image,
new Rectangle2D.Double(10, 10, 100, 70));
```

bir TexturePaint oluşturur ve resmi, üst sol köşesi (10, 10) olan ve genişliği ile yüksekliği 100 ve 70 olan dikdörtgende sabitler. Bu TexturePaint nesnesi, 39. satırda g2d'ye ayarlanır. 40. satır, bu TexturePaint ile bir elipsi doldurur, Şekil 49.18a'da gösterildiği gibi.



Şekil 49.18

Çapa dikdörtgeni, başlangıç resminin boyutunu ve konumunu tanımlar.

42-43. satırlardaki ifade

```
texturePaint = new TexturePaint(image,
new Rectangle2D.Double(10, 10, 50, 70));
```

(10, 10) olan üst sol köşesi ve genişliği ile yüksekliği 50 ve 70 olan dikdörtgende resmi sabitleyen bir TexturePaint oluşturur. Bu TexturePaint nesnesi, 45. satırda g2d'ye ayarlanır. 46. satır, bu TexturePaint ile bir elipsi doldurur, Şekil 49.18(b)'de gösterildiği gibi. Şekil 49.17'deki örnek çıktıda gördüğünü z gibi, doku boyası çapa dikdörtgeninden tekrar edilir.

53. satır bir dizeyi görüntüler. Karakterler, 51. satırda ayarlanan boyayla doldurulur.

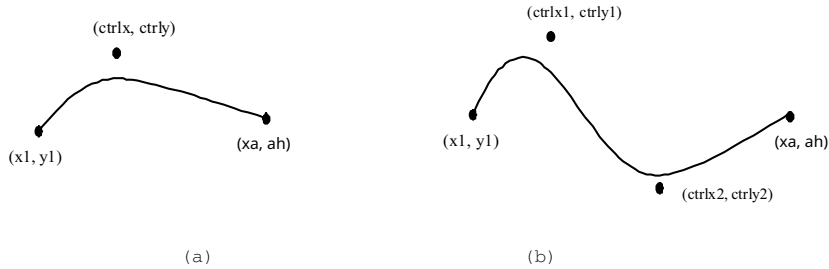
#### 49.8 QuadCurve2D ve CubicCurve2D

Java 2D, ikinci dereceden eğrileri ve üçüncü dereceden eğrileri modellemek için QuadCurve2D ve CubicCurve2D sınıflarını sağlar. QuadCurve2D.Double ve QuadCurve2D.Float, QuadCurve2D'nin iki somut alt sınıfıdır. CubicCurve2D.Double ve CubicCurve2D.Float, CubicCurve2D'nin iki somut alt sınıfıdır.

Bir ikinci dereceden eğri matematiksel olarak bir ikinci dereceden polinom olarak tanımlanır. Bir QuadCurve2D.Double oluşturmak için aşağıdaki yapıcıcıyı kullanın:

```
QuadCurve2D.Double(double x1, double y1,
double ctrlx, double ctrly, double x2, double y2)
```

$(x_1, y_1)$  ve  $(x_2, y_2)$  iki uç noktayı belirtirken  $(ctrlx, ctrly)$  bir kontrol noktasını belirtir. Kontrol noktası genellikle eğrinin üzerinde değildir, bunun yerine eğrinin eğilimini tanımlar, Şekil 49.19a'da gösterildiği gibi.



Şekil 49.19

(a) Bir ikinci dereceden eğri üç nokta kullanılarak tanımlanır. (b) Bir üçüncü dereceden eğri dört nokta kullanılarak tanımlanır.

Bir üçüncü dereceden eğri matematiksel olarak bir üçüncü dereceden polinom olarak tanımlanır. `CubicCurve2D.Double` oluşturmak için aşağıdaki yapıcuyu kullanın:

```
CubicCurve2D.Double(double x1, double y1, double ctrlx1,
 double ctrly1, double ctrlx2, double ctrly2, double x2, double y2)
```

burada  $(x_1, y_1)$  ve  $(x_2, y_2)$  iki uç noktayı belirtir ve  $(ctrlx1, ctrly1)$  ve  $(ctrlx2, ctrly2)$  iki kontrol noktasıdır. Kontrol noktaları genellikle eğrinin üzerinde değildir, eğrinin eğilimini tanımlamak için kullanılır, Şekil 49.19(b)'de gösterildiği gibi.

Liste 49.7, nasıl ikinci dereceden ve üçüncü dereceden eğriler çizeceğinizi gösteren bir programı vermektedir. Şekil 49.20, programın bir örnek çalışmasını göstermektedir.

#### Liste 49.7 CurveDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>

<Yan not: Satır 14: Graphics2D referansı>

<Yan not: Satır 17: ikinci dereceden eğri>

<Yan not: Satır 22: üçüncü dereceden eğri>

<Yan not: Satır 32: ana metod atlandı>

```
java.auto.*yu içe aktarın;
import java.awt.geom.*;
import javax.swing.*;

public class CurveDemo, JApplet'i genişletir {
 public CurveDemo() {
 add(new CurvePanel());
 }

 // statik sınıf CurvePanel JPanel'i genişletir
 protected void paintComponent(Graphics g) {
 super.paintComponent(g);

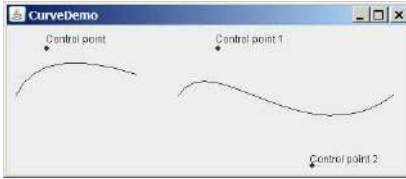
 Grafik2D g2d = (Grafik2D)g;
```

```

// İkinci dereceden bir eğri çiz
g2d.draw(new QuadCurve2D.Double(10, 80, 40, 20, 150, 56));n g2d.fillO
val(40 + 3, 20 + 3, 6, 6);
g2d.drawString("Kontrol noktası", 40 + 5, 20);

// Üçüncü dereceden bir eğri çiz
g2d.draw(new CubicCurve2D.Double
(200, 80, 240, 20, 350, 156, 450, 80));
g2d.fillOval(240 + 3, 20 + 3, 6, 6);
g2d.drawString("Kontrol noktası 1", 240 + 3, 20);
g2d.fillOval(350 + 3, 156 + 3, 6, 6);
g2d.drawString("Kontrol noktası 2", 350 + 3, 156 + 3);
}
}
N)

```



Şekil 49.20

Java 2D kullanarak ikinci dereceden ve üçüncü dereceden eğriler çizebilirsiniz.

#### <Yan not: QuadCurve2D>

Graphics2D referansı, Graphics2D'deki metotları çağırmak için 14. satırda elde edilir. new QuadCurve2D.Double(10, 80, 40, 20, 150, 56) ifadesi (17. satır) QuadCurve2D.Double'in bir örneğini oluşturur, bu da QuadCurve2D ve Shape'ın bir örneğidir. Örnek, iki uç nokta (10, 80), (150, 56) ve bir kontrol noktası (40, 20) ile bir kuadratik eğriyi modellemektedir.

fillOval (18. satır) ve drawString (19. satır) metotları Graphics sınıfında tanımlanmıştır ve bu nedenle Graphics2D sınıfında kullanılabilir.

#### <Yan açıklama: CubicCurve2D>

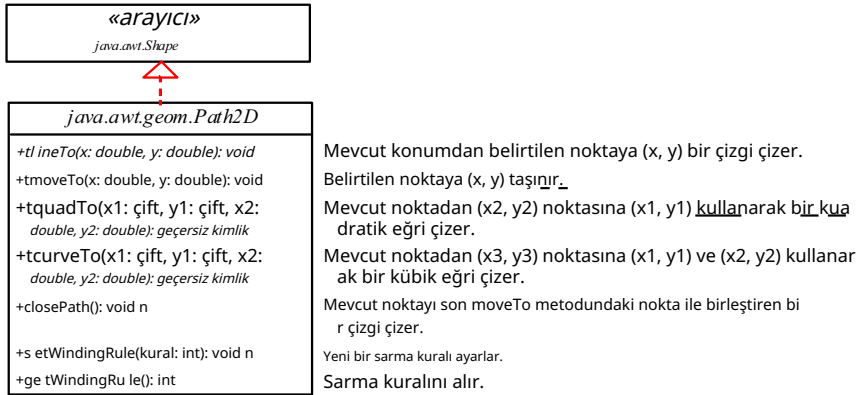
new CubicCurve2D.Double(200, 80, 240, 20, 350, 156, 450, 80) (22-23. satırlar) ifadesi, QuadCurve2D ve Shape'ın bir örneği olan CubicCurve2D.Double'in bir örneğini oluşturur. Örnek, iki uç nokta (200, 80), (450, 80) ve iki kontrol noktası (240, 20), (350, 156) ile bir kuadratik eğrileri modellemektedir.

### 49.9 Yol2D

Path2D sınıfı, keyfi bir geometrik yolu modellemektedir. Path2D.Double ve Path2D.Float, Path2D'nin iki somut alt sınıfıdır. Java 2D ayrıca artık Path 2D.Float ile yerini alan GeneralPath sınıfını da içermektedir.

Yol segmentlerini, Şekil 49.21'de gösterildiği gibi metotları kullanarak oluşturabilirsiniz.

#### <PD: UML Sınıf Diyagramı>



Şekil 49.21

*Path2D sınıfı, yol segmentleri oluşturmak için yöntemleri içerir.*

Path2D.Double ve Path2D.Float'tan bir yapıcı kullanarak bir Path2D oluşturabilirsiniz. Yol inşası süreci, bir kalemle çizim yapmak olarak görülebilir. Her an, kalemin bir mevcut pozisyonu vardır. Kalem (x, y) noktasına taşımak için moveTo(x, y) metodunu kullanabilir, mevcut noktadan bu yeni noktaya doğru bir düz çizgi çizerek (x, y) noktasını yola eklemek için lineTo(x, y) metodunu kullanabilir, mevcut konumdan (x, y) noktasına (ctrlx, ctrlly) kontrol noktası kullanarak bir kuadratik eğri çizmek için quadTo(ctrlx, ctrlly, x, y) metodunu kullanabilir, mevcut konumdan (x, y) noktasına (ctrlx1, ctrlly1) ve (ctrlx2, ctrlly2) kontrol noktaları kullanarak bir kübik eğri çizmek için curveTo(ctrlx1, ctrlly1, ctrlx2, ctrlly2, x, y) metodunu kullanabilir ve mevcut noktayı son moveTo metodundaki nokta ile birleştirmek için closePath() metodunu kullanabilirsiniz.

Liste 49.8, Path2D kullanarak bir şeklin nasıl çizileceğini gösteren bir programı vermektedir. Şekil 49.22, programın bir örnek çalışmasını göstermektedir.

#### Liste 49.8 Path2DDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>

<Yan not: Satır 5: applet>

<Yan not: Satır 14: Graphics2D referansı>

<Yan not: Satır 16: yeni konum>

<Yan not: Satır 17: bir kübik eğri çiz>

<Yan not: Satır 18: yeni konum>

<Yan not: Satır 19: bir kübik eğri çiz>

<Yan not: Satır 20: bir çizgi çiz>

<Yan not: Satır 21: yolu kapat>

<Yan not: Satır 23: yolu göster>

<Yan not: Satır 26: ana metod atlandı>

```
java.auto.*'yu içe aktarın;
import java.awt.geom.*;
import javax.swing.*;
import java.awt.*;
```

```
public class Path2DDemo, JApplet'i genişletir {
 public Path2DDemo() {
 add(new ShapePanel());
 }
}
```

```

class ShapePanel, JPanel'i genişletir {n pr
 oTECTED void paintComponent(Graphics g) {n super.pai
 ntComponent(g);

 Graphics2D g2d = (Graphics2D)g;n Pat
 h2D yolu = new Path2D.Double();n path.m
 oveTo(100, 100);
 path.curveTo(150, 50, 250, 150, 300, 100);n path.mov
 eTo(100, 100);
 path.curve(150, 150, 250, 50, 300, 100); path.line(200,
 20);
 yol.closePath();

 g2d.draw(yol);
 }
}

```



Şekil 49.22

*Path2D sınıfını kullanarak rastgele bir şekil çizebilirsiniz.*

`new Path2D.Double()` ifadesi (satır 15) boş bir yol oluşturur. `moveTo(100, 100)` metodu (s. 16) mevcut kalem pozisyonunu (100, 100) noktasına ayarlar. `path.curveTo(150, 50, 250, 150, 300, 100)` ifadesini çağırmak (satır 17) (100, 100) noktasından (300, 100) noktasına kontrol noktaları (150, 50) ve (250, 150) ile bir kübik eğri oluşturur. `path.moveTo(100, 100)` ifadesini çağırmak (satır 18) kalem pozisyonunu tekrar (100, 100) noktasına taşır. `path.curveTo(150, 150, 250, 50, 300, 100)` ifadesini çağırmak (s. 19) (100, 100) noktasından (300, 100) noktasına kontrol noktaları (150, 150) ve (250, 50) ile bir kübik eğri oluşturur. Şu anda mevcut pozisyon (300, 100) noktasındadır. `path.lineTo(200, 20)` ifadesini çağırmak (satır 20) (300, 100) noktasından (200,

20). `path.closePath()` (satır 21) çağrıldığında, mevcut konumu (yani, (200, 20)) son `moveTo` konumuyla (yani, (100, 100)) birleştiren bir çizgi çizer. Son olarak, `g2d.draw(path)` (satır 23) çağrıldığında, yol çizilir.

Basit bir şekil için, hangi noktanın bir şeklin içinde olduğunu belirlemek kolaydır. Bir yol birçok şekil oluşturabilir. Hangi noktanın kapalı bir yolun içinde olduğunu belirlemek kolay değildir. Java 2D, iç noktaları tanımlamak için winding kurallarını kullanır. İki winding kuralı vardır: `WIND_EVEN_ODD` ve `WIND_NON_ZERO`.

#### <Yan açıklama: **WIND\_EVEN\_ODD**>

`WIND_EVEN_ODD` kuralı, bir noktadan sonsuzluğa doğru rastgele bir yönde bir ışın gönderildiğinde, yolun bu ışınla tek sayıda kesişmesi durumunda o noktayı yolun içinde tanımlar. Şekil 49.23a'daki yolu düşünün. A ve C noktaları yolun dışındadır, çünkü ışın yolu iki kez keser. B noktası yolun içindedir, çünkü ışın yolu bir kez keser.



```

Graphics2D g2d = (Graphics2D)g; // Graphics2D'yi al

g2d.translate(10, 10); // Yeni bir orijine çevir
g2d.draw(createAPath()); // Bir yol oluştur ve çiz

g2d.translate(160, 0); // Yeni bir orijine çevir
Path2D path2 = createAPath(); // Bir yol oluştur
path2.setWindingRule(Path2D.WIND_EVEN_ODD); // Yeni bir kural belirle
g2d.fill(path2); // Bir yol oluştur ve doldur

g2d.translate(160, 0); // Yeni bir orijine çevir
Path2D path3 = createAPath(); // Bir yol oluştur
path3.setWindingRule(Path2D.WIND_NON_ZERO); // Yeni bir kural belirle
g2d.fill(path3); // Bir yol oluştur ve doldur
}

private Path2D createAPath() {
 // Dış dikdörtgeni tanımla
 Path2D path = new Path2D.Double();
 path.moveTo(0, 0);
 path.lineTo(0, 100);
 path.lineTo(100, 100);
 path.lineTo(100, 0);
 path.lineTo(0, 0);

 // İç dikdörtgeni tanımla
 path.moveTo(30, 30);
 path.lineTo(30, 70);
 path.lineTo(70, 70);
 path.lineTo(70, 30);
 path.lineTo(30, 30);

 dönüş yolu;
}
}

```



Şekil 49.24

*Dolgu kuralı iç noktaları tanımlar.*

#### **<Yan açıklama: createAPath>**

createAPath() metodu iki dikdörtgen için bir yol oluşturur. Dış dikdörtgen 33-37. satırlar da, iç dikdörtgen ise 40-49. satırlarda oluşturulur.

Program, 16. satırda koordinatların orijini (10, 10) noktasına taşır, \_\_\_\_\_  
yolu oluşturmak için createAPath'i çağırır ve 17. satırda görüntüler.

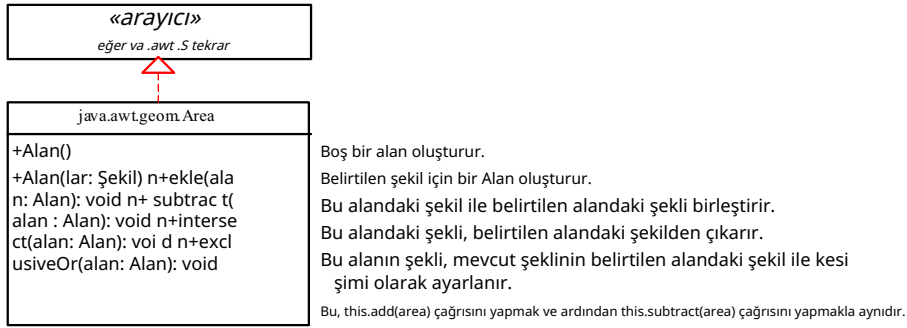
Program, koordinatın orijini 19. satırda (160, 0) konumuna taşır, \_\_\_\_ \_  
yeni bir yol oluşturur (20. satır), yolun sarma kuralını  
WIND\_EVEN\_ODD (21. satır) olarak ayarlar ve 22. satırda görüntüler.

Program, koordinatın orijini 24. satırda (160, 0) konumuna taşır, \_\_\_\_ \_  
yeni bir yol oluşturur (25. satır), yolun sarma kuralını  
WIND\_NON\_ZERO (26. satır) olarak ayarlar ve 27. satırda görüntüler.

Dikkat edin ki, eğer bir yol kapatılmamışsa, fill metodu onu kapatır ve dolu bir yol çizer.

#### 49.10 Yapısal Alan Geometrisi

Şekiller, yeni şekiller oluşturmak için birleştirilebilir. Buna yapısal alan geometrisi denir. Java 2D, şekil oluşturmak için Area sınıfını sağlar, bu Şekil 49.2 5'te gösterilmiştir.



Şekil 49.25

Area sınıfı, yeni alanlar oluşturmak için yöntemleri içerir.

Area, Shape'i uygular ve küme teorik işlemleri gerçekleştirmek için add, subtr act, intersect ve exclusiveOr yöntemlerini sağlar. Bu işlemler, alanlarda saklan an şekiller üzerinde gerçekleştirilir. İki alanın birleşimi, her iki alanda da bulunan tüm noktaları içerir. İki alanın farkı, ilk alanda bulunan ancak ikinci alanda bulun mayan noktaları içerir. İki alanın kesişimi, her iki alanda da bulunan tüm noktalar ı içerir. Simetrik fark, tam olarak iki alandan birinde bulunan noktaları içerir.

Liste 49.10, Alan sınıfını kullanarak yapısal geometriyi gösteren bir programı vermektedir. Şekil 49.26, programın bir örnek çalışmasını göstermektedir.

#### Liste 49.10 AreaDemo.java

<Yan not: Satır 2: şekil sınıfları için içe aktarma>  
<Yan not: Satır 5: applet>  
<Yan not: Satır 14: Graphics2D referansı>  
<Yan not: Satır 17: iki şekil>  
<Yan not: Satır 19: yeni orijin>  
<Yan not: Satır 23: şekilleri çiz>  
<Yan not: Satır 25: ekle>  
<Yan not: Satır 30: alanı doldur>



<Yan not: Satır 33: çıkar>  
 <Yan not: Satır 35: alanı doldur>  
 <Yan not: Satır 38: kesiş>  
 <Yan not: Satır 40: alanı doldur>  
 <Yan not: Satır 42: exclusiveOr>  
 <Yan not: Satır 43: alanı doldur>  
 <Yan not: Satır 48: ana metot atlandı>

```
java.auto.*'yu içe aktarın;
import java.awt.geom.*;
import javax.swing.*;

sınıfı AreaDemo, JApplet'i genişletir {
 public AreaDemo() {
 add(new ShapePanel());
 }
}

class ShapePanel, JPanel'i genişletir {
 protected void paintComponent(Graphics g) {
 super.paintComponent(g);

 Graphics2D g2d = (Graphics2D)g; // Graphics2D'yi al

 // İki şekil oluştur
 Shape shape1 = new Ellipse2D.Double(0, 0, 50, 50);
 Shape shape2 = new Ellipse2D.Double(25, 0, 50, 50);
 g2d.translate(10, 10); // Yeni bir orijine çevir
 g2d.draw(shape1); // Şekli çiz
 g2d.draw(shape2); // Şekli çiz

 Area area1 = new Area(shape1); // Bir alan oluştur
 Area area2 = new Area(shape2);
 area1.add(area2); // area2'yi area1'e ekle
 g2d.translate(100, 0); // Yeni bir orijine çevir
 g2d.draw(area1); // Alan içindeki şeklin dış hatlarını çiz

 g2d.translate(100, 0); // Yeni bir orijine çevir
 g2d.fill(area1); // Alan içindeki şekli doldur

 alan1 = yeni Alan(şekil1);
 area1.subtract(area2); // area2'yi area1'den çıkar
 g2d.translate(100, 0); // Yeni bir orijine çevir
 g2d.fill(area1); // Alan içindeki şekli doldur

 alan1 = yeni Alan(şekil1);
 area1.intersect(area2); // area2'nin area1 ile kesişimi
 g2d.translate(100, 0); // Yeni bir orijine çevir
 g2d.fill(area1); // Alan içindeki şekli doldur

 alan1 = yeni Alan(şekil1);
 area1.exclusiveOr(area2); // area2'nin area1 ile özel veya işlemi
 g2d.translate(100, 0); // Yeni bir orijine çevir
 g2d.fill(area1); // Alan içindeki şekli doldur
 }
}
```



Şekil 49.26

*Area sınıfı, yapısal geometri gerçekleştirmek için kullanılabilir.*

Program iki elips oluşturur (satırlar 17-18) ve bunları gösterir (satırlar 20-21). Program iki alan oluşturur ve add (satır 25), subtract (satır 33), intersect (satır 38) ve exclusiveOr (satır 43) çağırarak yapısal alan geometrisi gerçekleştirir.

#### Anahtar Terimler

- yapıcı alan geometrisi
- kübik eğriler
- gradyan boya
- kuadratik eğriler
- dönme
- ölçekleme
- çizgi
- doku boya
- çeviri
- RÜZGAR\_EŞİT\_TEK
- RÜZGAR\_SIFIR\_DIŞI

#### Bölüm Özeti

1. Java 2D API, grafiklerin işlenmesi için gelişmiş yeteneklerle java.awt.Graphics2D sınıfını sağlar.
2. Java 2D API, modellemeyi render etmeyi ayıran nesne yönelimli bir yaklaşıma sunar. Tüm şekiller Shape arayüzü altında tanımlanır.
3. Line2D, Rectangle2D, RoundRectangle2D, Arc2D, Ellipse2D, QuadCurve2D, CubicCurve2D ve Path2D sınıfları soyut sınıflardır. Her biri, sırasıyla double ve float koordinatları için Double ve Float adında iki somut statik iç sınıf içerir. İç sınıflar, ilgili soyut sınıflarının alt sınıflarıdır.
4. Bir nokta, soyut Point2D sınıfı kullanılarak modellenir. Point2D.Double ve Point2D.Float adında iki somut statik iç sınıf içerir; bunlar Point2D'nin alt sınıflarıdır.
5. Graphics2D sınıfı, şekilleri render etmek için kullanılır. Şeklin sınırını render etmek için draw(Shape) metodunu ve şeklin içini doldurmak için fill(Shape) metodunu çağırabilirsiniz.
6. Graphics sınıfında, bir sonraki render işlemini önceki konuma göre belirtilen mesafe kadar taşımak için translate(double x, double y) metodunu kullanabilirsiniz.
7. Graphics2D sınıfında, orijinden theta derece döndürmek için rotate(double theta) metodunu kullanabilirsiniz. Burada theta, radyan cinsinden bir double değerdir.
8. Graphics2D sınıfındaki scale(double sx, double sy) metodunu kullanarak, belirtilen ölçekleme faktörleriyle sonraki renderlamayı yeniden boyutlandırabilirsiniz.
9. Java 2D, çizgilere stroke denilen özellikleri belirtmenize olanak tanır. vuruş.

10. Çizginin genişliğini, çizginin nasıl bittiğini (end caps denir), çizgilerin nasıl birleştiğini (line joins denir) ve çizginin kesikli olup olmadığını belirtebilirsiniz. Bu özellikler, bir Stroke nesnesinde tanımlanır.
11. Bir Stroke nesnesi oluşturmak için BasicStroke sınıfını kullanabilirsiniz.
12. Bir stroke ayarlamak için, Graphics2D sınıfındaki setStroke(Stroke) metodunu kullanın.
13. Graphics2D setPaint(Paint) metodunu bir paint ayarlamak için sağlar. Paint, rengin genelleştirilmesidir. Basit düz renklerden daha fazla özelliğe sahiptir.
14. GradientPaint, iki nokta ve iki renk ile belirtilen değişken bir rengi tanımlar. Konum, ilk noktadan ikinci noktaya hareket ettikçe, boya yavaş yavaş ilk renkten ikinci renge değişir.
15. Bir GradientPaint döngüsel veya döngüsel olmayan olabilir. Döngüsel bir boya, aynı deseni periyodik olarak tekrarlar.
16. TexturePaint, bir şekli veya karakterleri doldurmak için bir resmi tanımlar. Bir doku boyası, bir dikdörtgende sabitlenmiş bir resimle tanımlanır.
17. Java 2D, ikinci dereceden eğrileri ve üçüncü dereceden eğrileri modellemek için QuadCurve2D ve CubicCurve2D sınıflarını sağlar.
18. Bir ikinci dereceden eğri matematiksel olarak bir ikinci dereceden polinom olarak tanımlanır.
19. Bir üçüncü dereceden eğri matematiksel olarak bir üçüncü dereceden polinom olarak tanımlanır.
20. Path2D sınıfı, keyfi bir geometrik yolu modellemektedir. Path2D.Double ve Path2D.Float, Path2D'nin iki somut alt sınıfıdır.
21. Çevirme kuralı, bir yol içindeki iç noktaları tanımlar.
22. WIND\_EVEN\_ODD kuralı, bir noktayı bir yolun içinde olarak tanımlar eğer bir ışın, noktadan sonsuzluğa doğru keyfi bir yönde yol ile tek sayıda kesişiyorsa.
23. WIND\_NON\_ZERO kuralı ile yolun yönü dikkate alınır. Bir nokta, bir ışın, noktadan sonsuzluğa doğru keyfi bir yönde yol ile ters yönde eşit olmayan sayıda kesişiyorsa yolun içindedir.
24. Java 2D, yapısal alan geometrisi gerçekleştirmek için Area sınıfını sağlar.
25. Area, Shape'i uygular ve birleştirme, fark, kesişim ve simetrik fark gibi küme teorisi işlemlerini gerçekleştirmek için add, subtract, intersect ve exclusiveOr yöntemlerini sağlar.

## Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

### Gözden Geçirme Soruları

#### Bölmeler 49.2-49.3

49.1 Graphics2D nesnesine nasıl bir referans alırsınız?

49.2 Shape arayüzünde tanımlanan bazı yöntemleri listeleyn.

49.3 Line2D nesnesi nasıl oluşturulur?

49.4 Line2D.Double ve Line2D.Float, Line2D'nin iç sınıfları mıdır? Ayrıca Line2D'nin alt sınıfları mıdır?

49.5 Bir Shape nesnesi nasıl render edilir?

49.6 Point2D, Point2D.Double, Point2D.Float ve Point arasındaki ilişkiler nelerdir? Point2D'de hangi yöntemlerin tanımlandığını görmek için Java API'sini kontrol edin.

#### Bölüm 49.4

49.7 Rectangle2D, Rectangle2D.Double, Rectangle2D.Float ve Rectangle arasındaki ilişkiler nelerdir?

49.8 Çizim/doldurma yöntemlerini kullanarak Graphics sınıfında çizgiler, dikdörtgenler, elipsler ve yaylar gibi temel şekilleri çizebilir veya bir Shape nesnesi oluşturup bunları draw (Shape) veya fill(Shape) ile render edebilirsiniz. Bunları kullanmanın avantajları nelerdir?

#### Bölüm 49.5

49.9 Bir dikdörtgen new Rectangle2D.Double(2, 3, 4, 5) kullanılarak oluşturulmuş olsun. g2d.translate(10, 10) ve g2d.draw(rectangle) uygulandıktan sonra nerede görüntülenir?

49.10 Bir dikdörtgen new Rectangle2D.Double(2, 3, 4, 5) kullanılarak oluşturulmuş olsun. g2d.rotate(Math.PI / 5) ve g2d.draw(rectangle) uygulandıktan sonra nerede görüntülenir?

49.11 Bir dikdörtgen new Rectangle2D.Double(2, 3, 4, 5) kullanılarak oluşturulmuş olsun. g2d.scale(10, 10) uygulandıktan sonra nerede görüntülenir? g2d.draw(dikdörtgen)?

#### Bölümler 49.6–49.7

49.12 Graphics2D'de bir Stroke nasıl oluşturulur ve ayarlanır?

49.13 Graphics2D'de bir Paint nasıl oluşturulur ve ayarlanır?

49.14 Gradyan boyası nedir? GradientPaint nasıl oluşturulur?

49.15 Doku boyası nedir? TexturePaint nasıl oluşturulur?

#### Bölümler 49.8–49.10

49.16 QuadCurve2D nasıl oluşturulur? CubicCurve2D nasıl oluşturulur?

49.17 Path2D'deki yöntemleri tanımlayın?

49.18 Dönme kuralı nedir? WIND\_EVEN\_ODD nedir? WIND\_NON\_ZERO nedir?

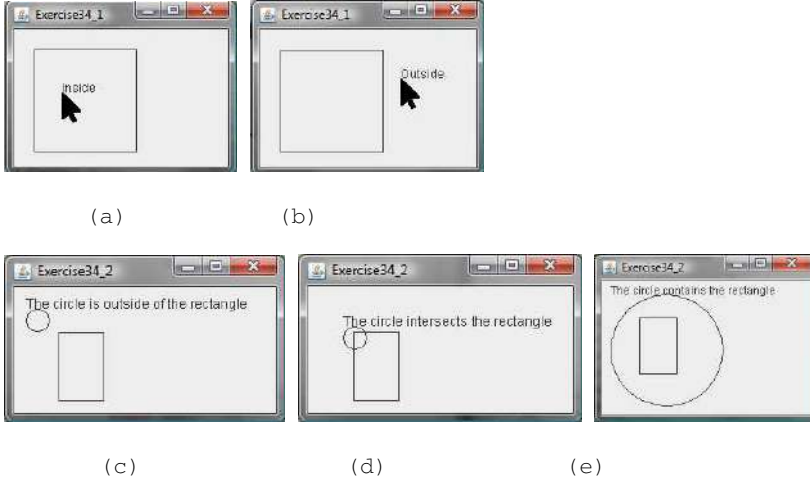
49.19 Bir şekilden bir Area nasıl oluşturursunuz? Area sınıfındaki add, subtract, intersect ve exclusiveOr yöntemlerini tanımlayın.

#### Programlama Alistirmaları

##### Bölüm 49.4

49.1 \*

( Bir dikdörtgenin içinde mi? ) (20, 20) noktasında, 100 genişlikte ve 100 yükseklikte bir dikdörtgen gösteren bir program yazın. Fareyi her hareket ettirdiğinizde, fare noktasının dikdörtgenin içinde olup olmadığını gösteren bir mesaj görüntüleyin; Şekil 49.27a–b'de gösterildiği gibi.



Şekil 49.27

(a-b) Alıştırma 49.1, bir noktanın bir dikdörtgenin içinde olup olmadığını tespit eder. (c-e) Alıştırma 49.2, bir dairenin bir dikdörtgeni içerip içermediğini, kesip kesmediğini veya dışında olup olmadığını tespit eder.

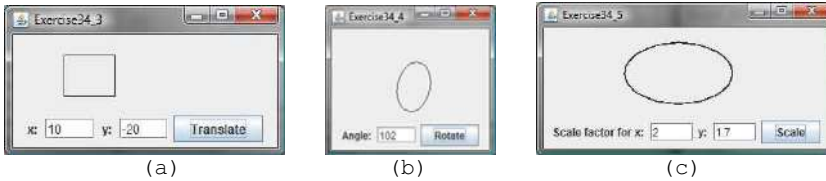
49.2 \*

(İçeriyor, kesiyor veya dışında mı?) (40, 40) noktasında, 40 genişlikte ve 60 yükseklikte bir dikdörtgen gösteren bir program yazın. Bir daire gösterin. Dairenin sınırlayıcı dikdörtgeninin sol üst köşesi fare noktasıdır. Yukarı/aşağı ok tuşuna basmak da dairenin yarıçapını 5 piksel artırır/azaltır. Dairenin dikdörtgeni içerip içermediğini, kesip kesmediğini veya dışında olup olmadığını belirtmek için fare noktasında bir mesaj gösterin; Şekil 49.27c'de gösterildiği gibi.

Ve.

49.3 \*

(Çevir) Üst sol köşe noktası (40, 40) olan, genişliği 50 ve yüksekliği 40 olan bir dikdörtgeni görüntüleyen bir program yazın. x ve y metin alanlarına değerleri girin ve dikdörtgeni yeni bir konuma taşımak için Çevir butonuna basın, Şekil 49.28a'da gösterildiği gibi.



Şekil 49.28

(a) Egzersiz 49.3 koordinatları çevirir. (b) Egzersiz 49.4 koordinatları döndürür. (c) Egzersiz 49.5 koordinatları ölçeklendirir.

49.4 \*

(Dönüş) Bir elipsi görüntüleyen bir program yazın. Elipsin merkezi (0, 0) noktasındadır ve genişliği 60, yüksekliği 40'dır. Orijini (100, 70) noktasına taşımak için translate metodu kullanın. Değeri metin alanına girin. —

## Alan açısını girin ve elipsi yeni

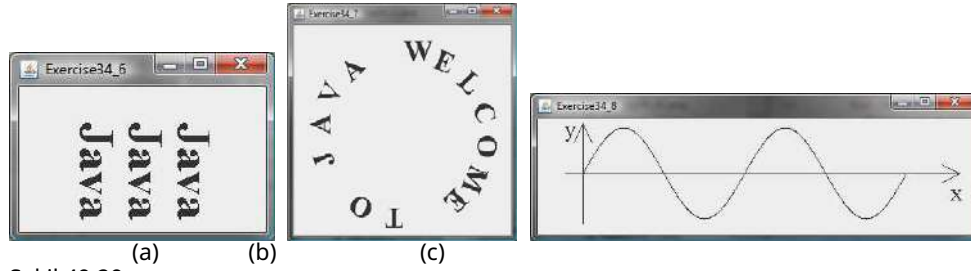
konuma döndürmek için Döndür butonuna basın, Şekil 49.28b'de gösterildiği gibi.

49.5\*

(*Grafikleri ölçeklendir*) Bir elipsi görüntüleyen bir program yazın. Elipsin merkezi (0, 0) noktasındadır ve genişliği 60, yüksekliği 40'dır. Orijini (150, 50) noktasına taşımak için translate metodunu kullanın. Ölçekleme faktörlerini metin alanlarına girin ve elipsi ölçeklendirmek için *Ölçek* butonuna basın, Şekil 49.28c'de gösterildiği gibi.

49.6\*

(*Dikey dizeler*) Şekil 49.29a'da gösterildiği gibi, üç dizeyi dikey olarak görüntüleyen bir program yazın.



Şekil 49.29

(a) Alıştırma 49.6, dizeleri dikey olarak görüntüler. (b) Alıştırma 49.7 daire etrafında karakterleri görüntüler. (c) Alıştırma 49.8, bir sinüs fonksiyonunu görüntüler.

49.7\*

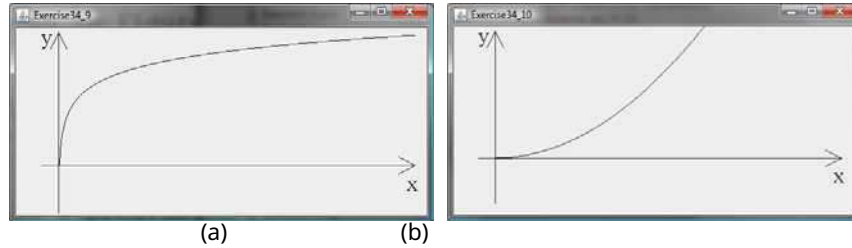
(*Daire etrafında karakterler*) Şekil 49.29b'de gösterildiği gibi, bir dizeyi daire etrafında görüntüleyen bir program yazın.

49.8\*

(*Sinüs fonksiyonunu çiz*) Şekil 49.29c'de gösterildiği gibi, sinüs fonksiyonunu çizen bir program yazın.

49.9\*

(*Logaritma fonksiyonunu çiz*) Şekil 49.30a'da gösterildiği gibi, logaritma fonksiyonunu çizen bir program yazın.



Şekil 49.30

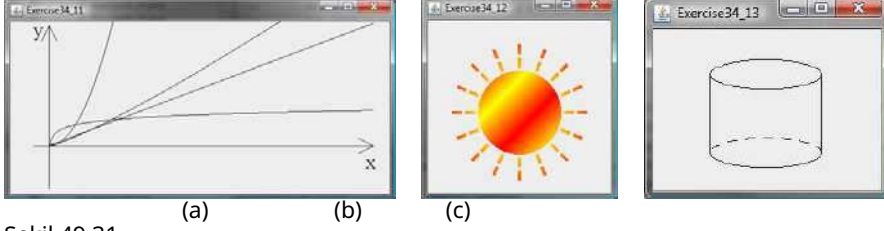
(a) Alıştırma 49.9, logaritma fonksiyonunu görüntüler. (b) Alıştırma 49.10  $n^2$  fonksiyonunu görüntüler.

49.10\*

( *$n^2$  fonksiyonunu çiz*) Şekil 49.30b'de gösterildiği gibi,  $n^2$  fonksiyonunu çizen bir program yazın.

49.11\*

(Logaritma,  $n$ ,  $n \log n$  ve  $n^2$  fonksiyonlarını çiz) Şekil 49.31a'da gösterildiği gibi, logaritma,  $n$ ,  $n \log n$  ve  $n^2$  fonksiyonlarını çizen bir program yazın.



Şekil 49.31

(a) Egzersiz 49.11 birkaç fonksiyonu gösterir. (b) Egzersiz 49.12 güneş ışığını gösterir. (c) Egzersiz 49.13 bir silindiri gösterir.

49.12\*

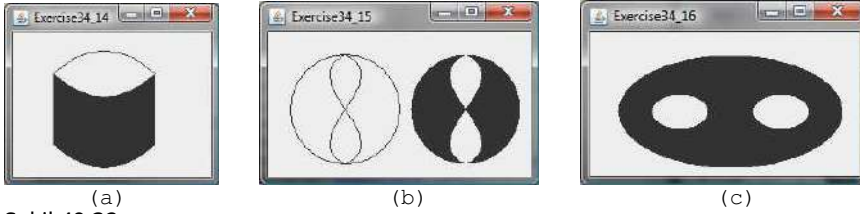
(Güneş Işığı) Bir daireyi, güneşi canlandırmak için bir gradyan renkle dolduracak ve güneşten çıkan ışınları kesik çizgilerle gösterecek bir program yazın, Şekil 49.31b'de gösterildiği gibi.

49.13\*

(Bir silindiri görüntüle) Şekil 49.31c'de gösterildiği gibi bir silindiri görüntüleyen bir program yazın. Kesik yay çizmek için kesik çizgiler kullanın.

49.14\*

(Doldurulmuş silindir) Şekil 49.32a'da gösterildiği gibi, doldurulmuş bir silindiri görüntüleyen bir program yazın.



Şekil 49.32

(a) Egzersiz 49.14 dolu bir silindiri gösterir. (b) Egzersiz 49.15'de iki alanın simetrik farkını gösterir. (c) Egzersiz 49.16

bir çift göz gösterir.

49.15\*

(Alan geometrisi) İki alan oluşturan bir program yazın: bir daire ve iki kübik eğriden oluşan bir yol. Alanları çizin ve doldurun. Alanların simetrik farkı, Şekil 49.32(b)'de gösterildiği gibi.

49.16\*

(Gözler) Oval içinde iki göz gösteren bir program yazın, Şekil 49.32c'de gösterildiği gibi.

49.17\*\*

(Geometri: bir çokgenin stratejik noktası) Alıştırma 14.33'ü gözden geçirin ve kullancının köşeleri sürükleyip hareket ettirmesine ve programın çokgeni ve onun stratejik noktasını dinamik olarak yeniden göstermesine olanak tanıyın. Bir program yazın ve çokgenin beş noktasının başlangıçta (25, 20), (170, 25), (200, 100), (100, 110) ve (50, 80) konumunda olduğunu varsayın.

— — — — —

49.18\*

(*Grafikleri ölçeklendir ve döndür*) Kullanıcının DURA işaretini ölçeklendirmesine ve döndürmesine olanak tanıyan bir applet yazın. Kullanıcı, boyutu artırmak/azaltmak için CT, RL ve +/- tuşlarına basabilir ve sola veya sağa döndürmek için SAĞ/SOL ok tuşuna basabilir.



Şekil 49.33

Applet, resmi ölçeklendirebilir ve döndürebilir.



*\*\*\*Bu bir bonus Web bölümü*

## **BÖLÜM 50**

### **JUnit Kullanarak Test Etme**

#### Hedefler

- JUnit'in ne olduğunu ve JUnit'in nasıl çalıştığını bilmek (§50.2).
- Komut penceresinden bir JUnit test sınıfı oluşturmak ve çalıştırmak (§50.2).
- NetBeans'ten bir JUnit test sınıfı oluşturmak ve çalıştırmak (§50.3).
- Eclipse'ten bir JUnit test sınıfı oluşturmak ve çalıştırmak (§50.4).

## 50.1 Giriş

Bu kitabın başında, 2.16. Bölümde, gereksinimlerin belirlenmesi, analiz, tasarım, uygulama, test, dağıtım ve bakım süreçlerini içeren yazılım geliştirme sürecini tanıttık.

Test, bu sürecin önemli bir parçasıdır. Bu bölüm, Java sınıflarını JUnit kullanarak nasıl test edeceğinizi tanıtmaktadır.

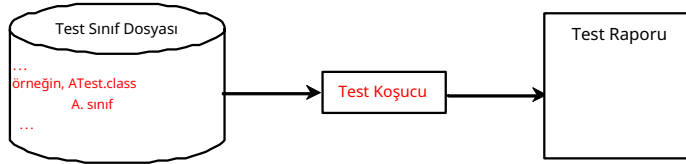
## 50.2 JUnit Temelleri

**<key term>JUnit**

**<key term>test koşucusu**

**<key term>test sınıfı**

*JUnit* Java programlarını test etmek için de facto çerçevedir. JUnit, bir jar dosyası içinde paketlenmiş üçüncü taraf açık kaynaklı bir kütüphanedir. Jar dosyası, test programlarını çalıştırmak için kullanılan *test koşucusu* adında bir araç içerir. Diyelim ki A adında bir sınıfınız var. Bu sınıfı test etmek için ATest adında bir test sınıfı yazarsınız. Bu test sınıfı, *test sınıfı* olarak adlandırılır ve A sınıfını test etmek için yazdığınız yöntemleri içerir. Test koşucusu, ATest'i çalıştırarak bir test raporu oluşturur; bu, Şekil 50.1'de gösterilmiştir.



Şekil 50.1

JUnit test koşucusu, bir test raporu oluşturmak için test sınıfını çalıştırır.

JUnit'in nasıl çalıştığını bir örnekle göreceksiniz. Örneği oluşturmak için, öncelikle JUnit'i

<http://sourceforge.net/projects/junit/files/> adresinden indirmeniz gerekiyor. Şu anda, en son sürüm junit-4.10.jar'dır. Bu dosyayı c:\book\lib dizinine indirin ve classpath ortam değişkenine aşağıdaki gibi ekleyin:

**set classpath=;%classpath%;c:\book\lib\junit-4.10.jar**

Bu ortam değişkeninin doğru bir şekilde ayarlandığını test etmek için, yeni bir komut penceresi açın ve aşağıdaki komutu yazın:

**java org.junit.runner.JUnitCore**

Mesajın Şekil 50.2'de gösterildiği gibi görüntülenmesi gerekir.



Şekil 50.2

JUnit test çalıştırıcısı JUnit sürümünü görüntüler.

JUnit'i kullanmak için bir test sınıfı oluşturun. Geleneksel olarak, test edilecek sınıf A olarak adlandırılıyorsa, test sınıfı ATest olarak adlandırılmalıdır. Bir test sınıfının basit bir şablonu şöyle görünebilir:

```
mytest paketi;

org.junit.*'i içe aktar; ni
import static org.junit.Assert.*; npublic sınıf

if ATest {
 @Test
 public void m1() {
 // Bir test metodu yazın
 }

 @Test
 public void m2() {
 // Başka bir test metodu yazın
 }

 @Önce
 public void setUp() throws Exception {
 // Test yöntemleri tarafından kullanılan ortak nesneler burada ayarlanabilir
 }
}
```

Testi konsoldan çalıştırmak için aşağıdaki komutu kullanın:

```
java org.junit.runner.JUnitCore mytest.ATest
```

#### **<anahtar terim>JUnitCore**

Bu komut çalıştırıldığında, JUnitCore ATest'in yürütülmesini kontrol eder. Öncelikle test için kullanılan ortak nesneleri ayarlamak üzere setUp() metodunu çalıştırır ve ardından bu sırayla m1 ve m2 test yöntemlerini yürütür. İstenirse birden fazla test yöntemi tanımlayabilirsiniz.

Aşağıdaki yöntemler bir test yöntemini uygulamak için kullanılabilir:

assertTrue(booleanExpression)

Yöntem, booleanExpression true olarak değerlendirildiğinde başarıyı bildirir.

assertEquals(Nesne, Nesne)

Yöntem, iki nesne

equals yöntemini kullanarak aynıysa başarıyı bildirir.

assertNull(Nesne)

Yöntem, geçirilen nesne referansı null ise başarıyı bildirir.

assertFailure(Dize)

Yöntem, testi başarısız kılar ve dizeyi yazdırır.

Liste 50.1,

java.util.ArrayList'i test etmek için bir test sınıfı örneğidir.

#### **Liste 50.1 ArrayListTest.java**

**<Yan Not satır 7: test sınıfı>**

<Yan Not satır 15: test metodu>  
<Yan Not satır 17: doğrulama>  
<Yan Not satır 20: doğrulama>  
<Yan Not satır 24: test metodu>  
<Yan Not satır 26: doğrulama>  
<Yan Not satır 32: doğrulama>

```
paket mytest;

içe aktarma org.junit.*;
içe aktarma static org.junit.Assert.*;
içe aktarma java.util.*;

genel sınıf ArrayListTest {
 private ArrayList<String> list = new ArrayList<String>();

 @Önce
 public void setUp() İstisnayı oluşturur {n }

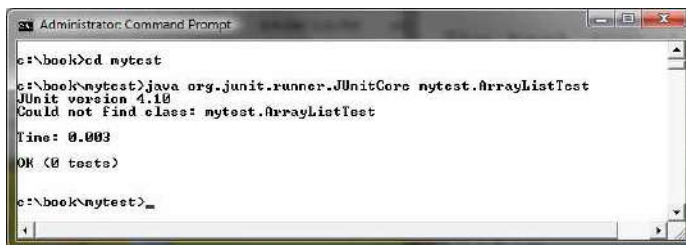
 @Test
 public void testInsertion() {n list.add(
 "Pekin");
 İddiaEquals("Pekin", list.get(0));n list.add("Şang
 ay");
 list.add("Hongkong");
 İddiaEquals("Hongkong", list.get(list.size() - 1));n }

 @Test
 public void testDeletion() {n list.cle
 ar();
 İddiaTrue(list.isEmpty());

 list.add("A");
 list.add("B");
 list.add("C");
 list.remove("B");nass
 ertEquals(2, list.size());

 }
}
```

Programın bir test çalışması Şekil 50.3'te gösterilmiştir. Öncelikle ArrayListTest.java dosyasını derlemeniz gerektiğini unutmayın. ArrayListTest sınıfı mytest paketinde yer almaktadır. Bu nedenle ArrayListTest.java dosyasını mytest adındaki dizine yerleştirmelisiniz.



Şekil 50.3

ArrayListTest çalıştırılarak test raporu görüntülenir.

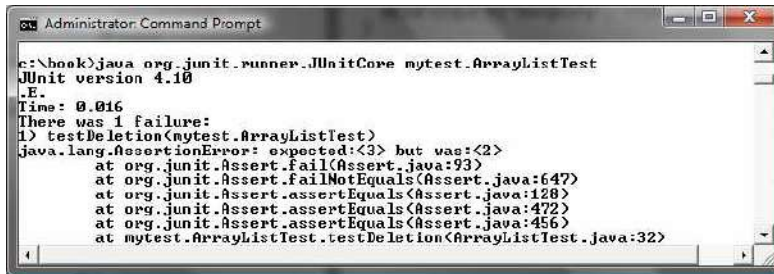
Bu JUnit çalışmasında hata bildirilmemektedir. Yanlışlıkla değiştirirseniz

```
assertEquals(2, list.size());
```

32. satırı

```
assertEquals(3, list.size());
```

ArrayListTest'i şimdi çalıştırın. Şekil 50.4'te gösterildiği gibi bir hata bildirilecektir.



Şekil 50.4

Test raporu bir hata bildiriyor.

Herhangi bir sayıda test metodu tanımlayabilirsiniz. Bu örnekte, testInsertion ve testDeletion adında iki test metodu tanımlanmıştır. JUnit, testInsertion ve testDeletion'ı bu sırayla çalıştırır.

NOT: Test sınıfı, bu örnekteki gibi mytest gibi adlandırılmış bir paket te yer almalıdır. Test sınıfı varsayılan bir pakette yer alırsa JUnit çalışm ayacaktır.

Liste 50.2, Liste 10.2'deki Loan sınıfını test etmek için bir test sınıfı vermektedir. Kolaylık olması açısından, LoanTest.java ile aynı dizinde Loan.java oluşturuyoruz. Loan sınıfı Liste 50.3'te gösterilmektedir.

#### Liste 50.2 LoanTest.java

<Yan Not 6. satır: test sınıfı>  
<Yan Not 12. satır: test metodu>  
<Yan Not 19. satır: doğrulama>  
<Yan Not 22. satır: doğrulama>  
<Yan Not 28. satır: aylık ödemeyi hesapla>  
<Yan Not 37. satır: toplam ödemeyi hesapla>

paket mytest;

import org.junit.\*;nimp  
ort static org.junit.Assert.\*;npublic class L

oanTest {

```

@Önce
public void setUp() İstisnayı oluřturur {n }

@Test
public void testPaymentMethods() {n doub
le AnnualInterestRate = 2,5;n int numbe
rOfYears = 5;
double krediMiktari = 1000;
Kredi kredi = new Kredi(yıllıkFaizOranı, yılSayısı,
krediMiktari);

assertTrue(kredi.getAylıkÖdeme() ==
getAylıkÖdeme(yıllıkFaizOranı, yılSayısı,
krediMiktari));
assertTrue(kredi.getToplamÖdeme() ==
getToplamÖdeme(yıllıkFaizOranı, yılSayısı,
krediMiktari));
}

/** Aylık ödemeyi bul */
private double getAylıkÖdeme(double yıllıkFaizOranı,
int yılSayısı, double krediMiktari) {
double aylıkFaizOranı = yıllıkFaizOranı / 1200;
double aylıkÖdeme = krediMiktari * aylıkFaizOranı / (1 -
(1 / Math.pow(1 + aylıkFaizOranı, yılSayısı * 12)));
return aylıkÖdeme;
}

/** Toplam ödemeyi bul */
public double getToplamÖdeme(double yıllıkFaizOranı,
int yılSayısı, double krediMiktari) {
return getAylıkÖdeme(yıllıkFaizOranı, yılSayısı,
krediMiktari) * yılSayısı * 12;
}
N}

```

### Liste 50.3 Loan.java

<Yan Not 1: mytest paketi>

<Yan Not 3: Loan sınıfı>

<Yan Not 56: getMonthlyPayment>

<Yan Not 64: getTotalPayment>

```

paket mytest;

kamu sınıfı Kredisı {
private double yıllıkFaizOranı;
private int yılSayısı;
özel double krediMiktari;
özel java.util.Date krediTarihi;

/** Varsayılan yapıcı */
public Loan() {
this(2,5; 1, 1000);n }

```

```

 /** Belirtilen yıllık faiz oranı,
 yıl sayısı ve kredi miktarı ile bir kredi oluştur
 */
 public Loan(double yıllıkFaizOranı, int yılSayısı,
 double krediMiktarı) {
 this.yıllıkFaizOranı = yıllıkFaizOranı;
 this.yılSayısı = yılSayısı;
 this.krediMiktarı = krediMiktarı;
 krediTarihi = new java.util.Date();
 }

 /** Yıllık faiz oranını döndür */
 public double getYıllıkFaizOranı() {
 return yıllıkFaizOranı;
 }

 /** Yeni bir yıllık faiz oranı ayarla */
 public void setYıllıkFaizOranı(double yıllıkFaizOranı) {
 this.yıllıkFaizOranı = yıllıkFaizOranı;
 }

 /** Yıl sayısını döndür */
 public int getYılSayısı() {
 return yılSayısı;
 }

 /** Yeni numberOfYears değerini ayarla */
 public void setNumberOfYears(int numberOfYears) {
 this.numberOfYears = numberOfYears;
 }

 /** loanAmount değerini döndür */
 public double getLoanAmount() {
 return loanAmount;
 }

 /** Yeni loanAmount değerini ayarla */
 public void setLoanAmount(double loanAmount) {
 this.loanAmount = loanAmount;
 }

 /** Aylık ödemeyi bul */
 public double getMonthlyPayment() {n doub
 lemonthInterestRate = yıllıkFaizOranı / 1200;n çift aylıkPayment = kre
 diAmount * aylıkFaizRatı / (1 -n (1 / Math.pow(1 +monthInterestRate, numberOf
 Years * 12)));n return aylıkPayment;

 }

 /** Toplam ödemeyi bul */
 public double getTotalPayment() {n doubl
 e totalPayment = getMonthlyPayment() * numberOfYears * 12;n return totalPay
 ment;

 }

```

```

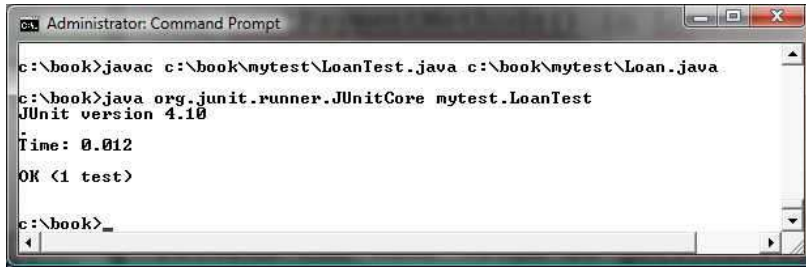
 /** Kredi tarihini döndür */
 public java.util.Date getLoanDate() {
 return loanDate;
 }
}

```

testPaymentMethods() LoanTest içinde bir Loan örneği oluşturur (satır 16-17) ve loan.getMonthlyPayment()'in getMonthlyPayment(annualInterestRate, numberOfYears, loanAmount) ile aynı değeri döndürüp döndürmediğini test eder. İkinci yöntem LoanTest sınıfında tanımlanmıştır (satırlar 28-34) .

testPaymentMethods() ayrıca loan.getTotalPayment()'in getTotalPayment(annualInterestRate, numberOfYears, loanAmount) ile aynı değeri döndürüp döndürmediğini test eder. İkinci yöntem LoanTest sınıfında tanımlanmıştır (satırlar 37-41).

Programın bir örnek çalışması Şekil 50.5'te gösterilmiştir.



Şekil 50.5

JUnit test çalıştırıcısı LoanTest'i çalıştırır ve hata bildirmez.

### 50.3 NetBeans'ten JUnit Kullanımı

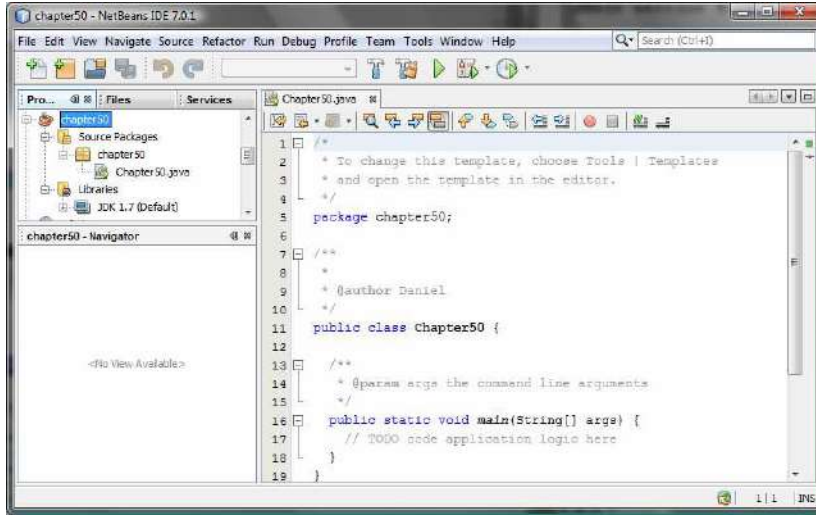
NetBeans ve Eclipse gibi bir IDE, test sınıfları oluşturma ve çalıştırma sürecini büyük ölçüde basitleştirebilir. Bu bölüm, NetBeans'ten JUnit kullanmayı tanıtır ve bir sonraki bölüm Eclipse'ten JUnit kullanmayı tanıtır.

Eğer NetBeans ile tanışık değilseniz, Ek II.B'ye bakın. NetBeans 7.0'ı kurduğunuzu varsayın. Aşağıdaki gibi chapter50 adında bir proje oluşturun:

Adım 1: Yeni Proje iletişim kutusunu görüntülemek için *Dosya*, *Yeni Proje* seçin.  
Adım 2: Kategoriler bölümünde Java'yı ve Projeler bölümünde Java Uygulaması'nı seçin. Yeni Java Uygulaması iletişim kutusunu görüntülemek için *İleri* tıklayın.

Adım 3: Proje Adı olarak chapter50 ve Proje Konumu olarak c:\book girin. Projeyi oluşturmak için *Tamamla* tıklayın, Şekil 50.6'da gösterildiği gibi.





Şekil 50.6

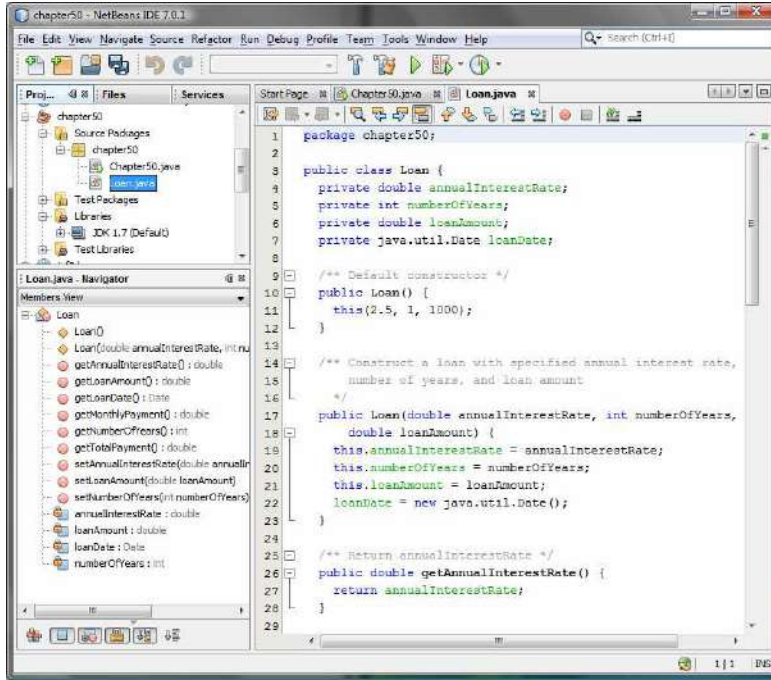
chapter50 adında yeni bir proje oluşturuldu.

Bir test sınıfı oluşturmayı göstermek için önce test edilecek sınıfı oluşturuyoruz. Sınıfı, Liste 10.2'den Loan olarak adlandırılabilir. chapter50 altında Loan sınıfını oluşturmak için adımlar şunlardır. \_\_\_\_\_

Adım 1: chapter50 proje düğmesine sağ tıklayın ve *Yeni, Java Sınıfı* seçin, Yeni Java Sınıfı iletişim kutusunu görüntülemek için.

Adım 2: Sınıf Adı olarak Loan ve Paket alanında chapter50 girin ve sınıfı oluşturmak için *Tamamla* tıklayın.

Adım 3: Liste 10.2'deki kodu Loan sınıfına kopyalayın ve ilk satırın package chapter50 olduğunu kontrol edin, Şekil 50.7'de gösterildiği gibi.



Şekil 50.7

Kredi sınıfı oluşturuldu.

Artık Kredi sınıfını test etmek için aşağıdaki gibi bir test sınıfı oluşturabilirsiniz:

Adım 1: Projedeki Loan.java dosyasına sağ tıklayın ve bir bağlam menüsü görüntülemek için Araçlar'ı, JUnit Testi Oluştur'u seçin ve JUnit sürümünü seçin

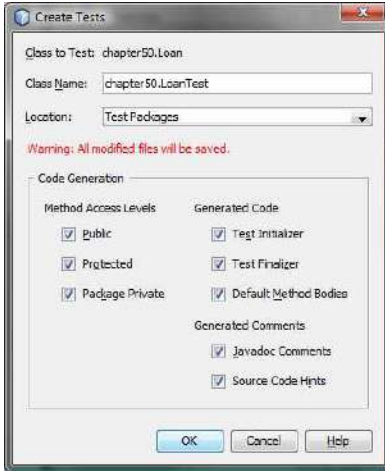
diyalog kutusunu görüntülemek için, Şekil 50.8'de gösterildiği gibi.

Adım 2: JUnit 4.x'i seçin. Şekil 50.9'da gösterildiği gibi Test Oluştur penceresinin açıldığını göreceksiniz. Test sınıfı oluşturmak için *Tamam* butonuna tıklayın. Şekil 50.10'da gösterildiği gibi LoanTest adında bir Test sınıfı oluşturulacaktır. LoanTest.java'nın projedeki Test Paketleri düğümünün altında yer aldığını unutmayın.



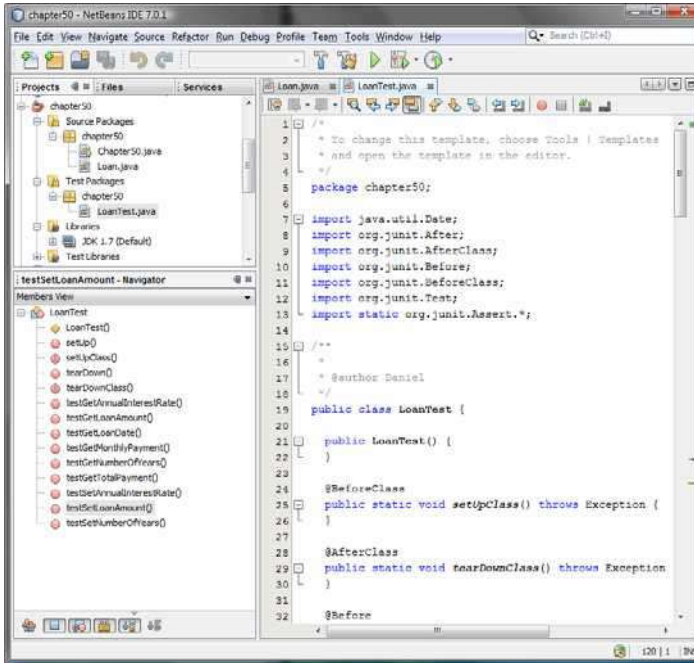
Şekil 50.8

Test sınıfları oluşturmak için JUnit 4.x çerçevesini seçmelisiniz.



Şekil 50.9

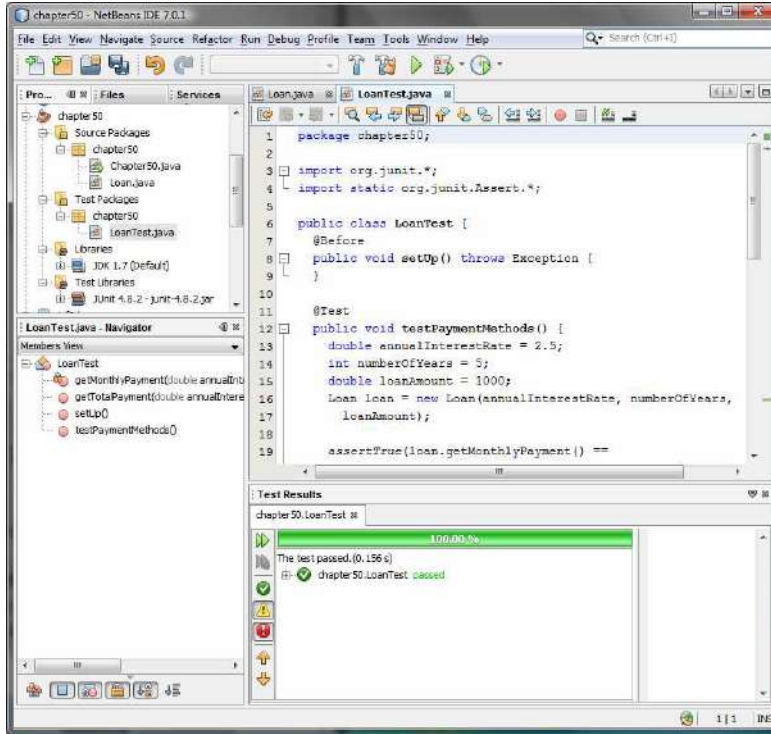
Test Oluştur penceresi bir Test sınıfı oluşturur.



Şekil 50.10

LoanTest sınıfı otomatik olarak oluşturulur.

Artık Listing 50.2'den kodu kopyalayarak LoanTest'i değiştirebilirsiniz. LoanTest.java'yı çalıştırın. Şekil 50.11'de gösterildiği gibi test raporunu göreceksiniz.



Şekil 50.11

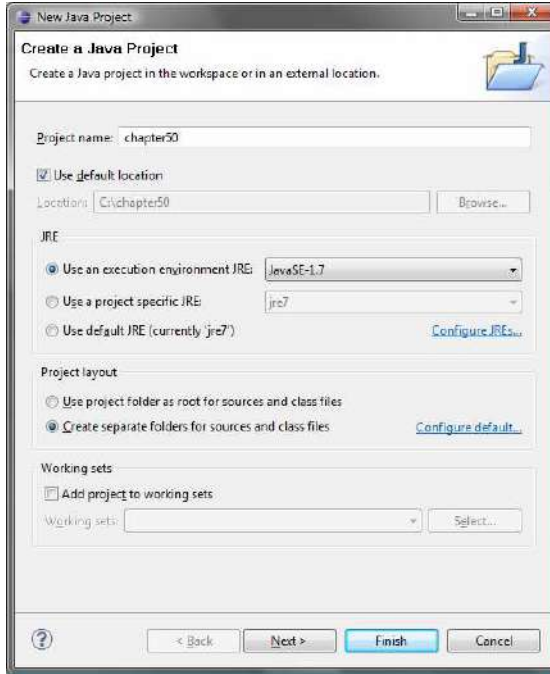
LoanTest sınıfı çalıştırıldıktan sonra test raporu görüntülenir.

#### 50.4 Eclipse'den JUnit Kullanımı

Bu bölüm, Eclipse'den JUnit kullanımını tanıtmaktadır. Eğer Eclipse ile yeniyseniz, Ek II.D'ye bakın. Eclipse 3.7'yi kurduğunuzu varsayın. Aşağıdaki gibi chapter50 adında bir proje oluşturun:

Adım 1: Dosya menüsünden Yeni Java Projesi'ni seçin ve Yeni Java Projesi diyalog kutusunu görüntüleyin, Şekil 50.12'de gösterildiği gibi.

Adım 2: Proje adı alanına chapter50 yazın ve projeyi oluşturmak için Tamam'a tıklayın.



Şekil 50.12

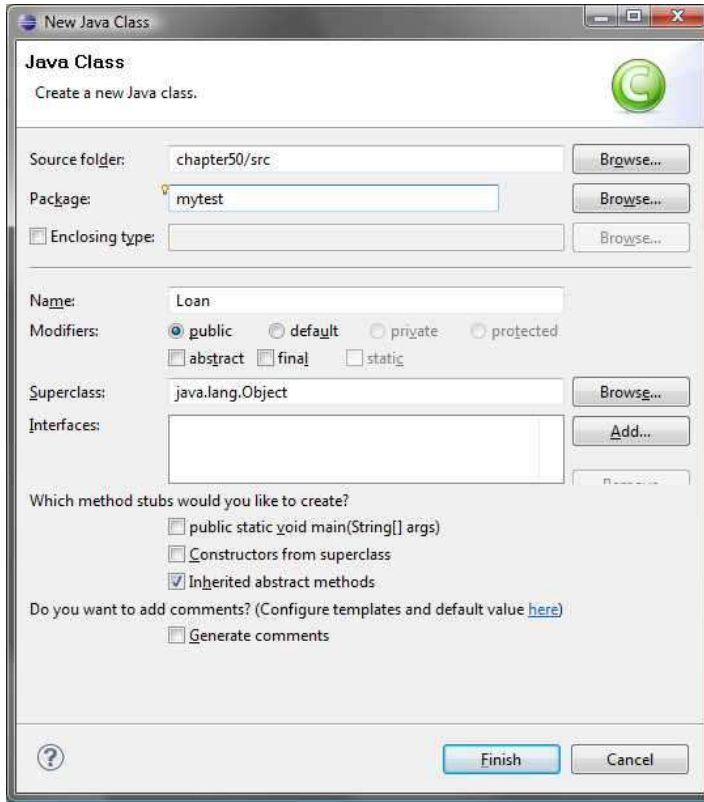
Yeni Java Projesi diyalogu yeni bir proje oluřturur.

Bir test sınıfı oluřturmayı g stermek i in  nce test edilecek sınıfı oluřturuyoruz. Sınıfı, Liste 10.2'den Loan olarak adlandırıl m. chapter50 altında Loan sınıfını oluřturmak i in adlar řunlardır. \_\_\_\_\_

Adım 1: chapter50 proje d ğ m ne saė t klayın ve Yeni men s nden Sınıfı se in, Şekil 50.13'te g sterildiėi gibi Yeni Java Sınıfı diyalogunu g r nt leyin.

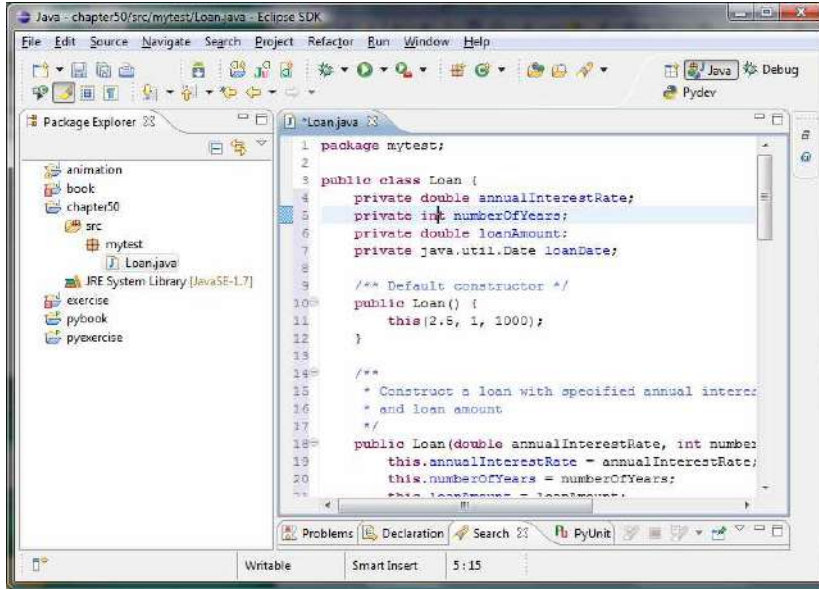
Adım 2: Paket alanına mytest yazın ve sınıfı oluřturmak i in *Bitir* butonuna t klayın.

Adım 3: Liste 10.2'deki kodu Loan sınıfına kopyalayın ve ilk satırın package mytest olduėundan emin olun, Şekil 50.14'te g sterildiėi gibi.



Şekil 50.13

Yeni Java Sınıfı iletişim kutusu yeni bir Java sınıfı oluşturur.



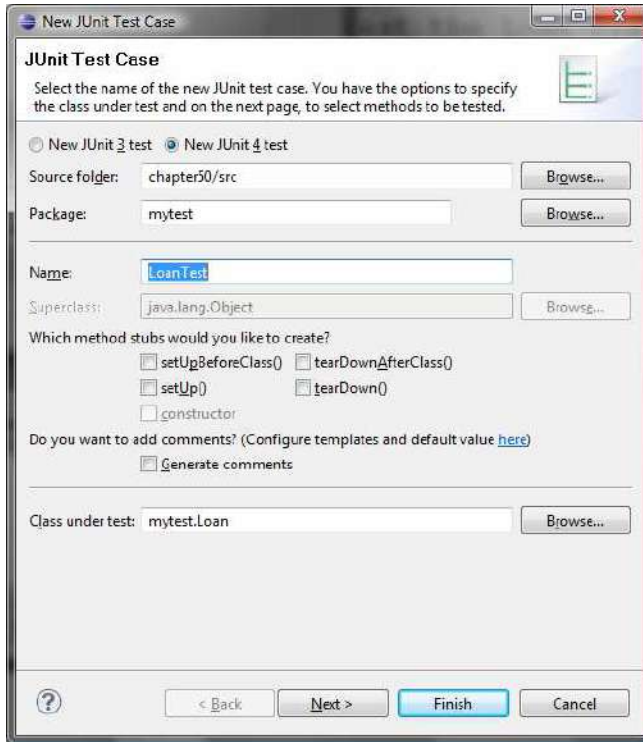
Şekil 50.14

Kredi sınıfı oluşturuldu.

Artık Kredi sınıfını test etmek için aşağıdaki gibi bir test sınıfı oluşturabilirsiniz:

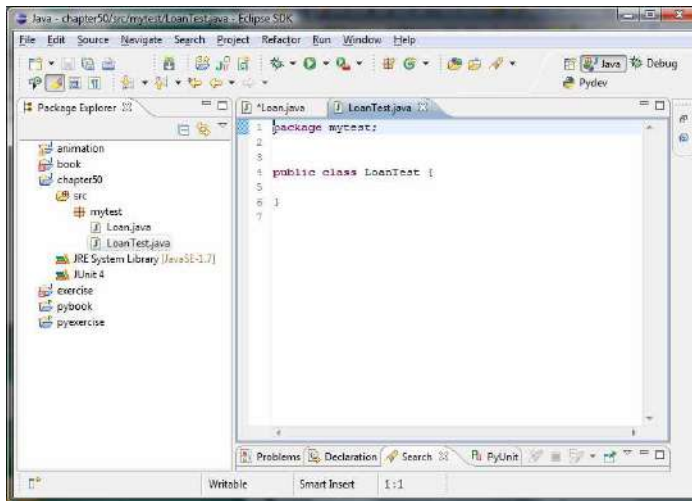
Adım 1: Projede Loan.java üzerine sağ tıklayın, bir bağlam menüsü görüntülenecek ve *Yeni, JUnit Test Durumu* seçeneğini seçin, böylece Yeni JUnit Test Durumu iletişim kutusu açılacak, Şekil 50.15'te gösterildiği gibi.

Adım 2: *Bitir* butonuna tıklayın. Proje derleme yoluna JUnit 4 eklemeniz için bir iletişim kutusu göreceksiniz. Eklemek için *Tamam* butonuna tıklayın. Artık Şekil 50.16'da gösterildiği gibi LoanTest adında bir test sınıfı oluşturulmuştur.



Şekil 50.15

Yeni JUnit Test Durumu iletişim kutusu bir Test sınıfı oluşturur.

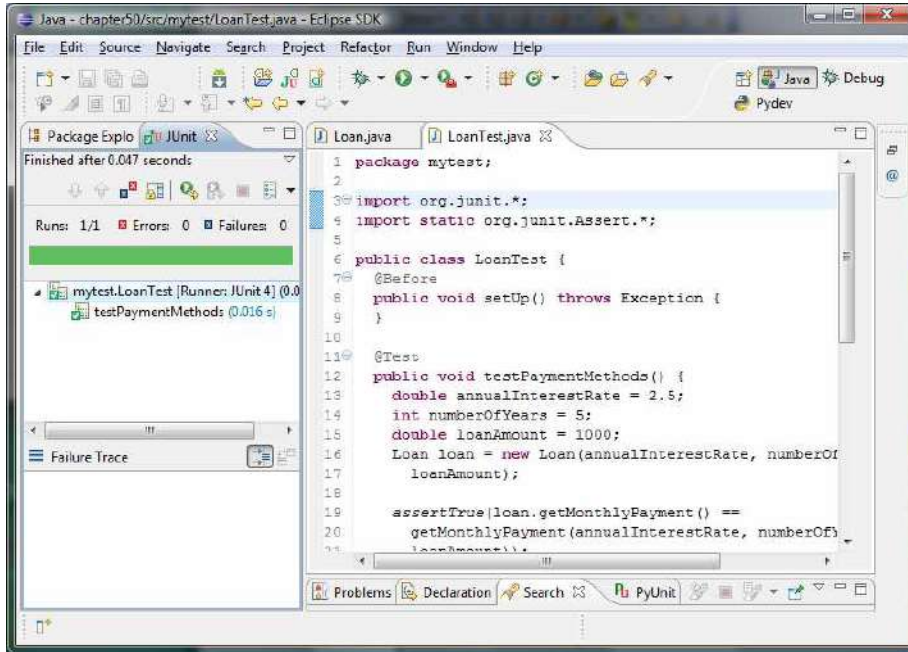


Şekil 50.16

LoanTest sınıfı otomatik olarak oluşturulur.

Artık LoanTest'i Liste 50.2'den kodu kopyalayarak değiştirebilirsiniz. LoanTest.java'yı çalıştırın. Şekil 50.17'de gösterildiği gibi test raporunu göreceksiniz.





Şekil 50.17

LoanTest sınıfı çalıştırıldıktan sonra test raporu görüntülenir.

#### Anahtar Terimler

- **JBirim**
- **JUnitCore**
- **test sınıfı**
- **test çalıştırıcısı**

#### Bölüm Özeti

1. JUnit, Java programlarını test etmek için açık kaynaklı bir çerçevedir.
2. Bir Java sınıfını test etmek için, test edilecek sınıf için bir test sınıfı oluşturur ve JUnit' in test çalıştırıcısını kullanarak test sınıfını çalıştırarak bir test raporu oluşturursunuz.
3. Komut penceresinden bir test sınıfı oluşturup çalıştırabilir veya NetBeans ve Eclipse gibi bir araç kullanabilirsiniz.

#### Test Soruları

Bu bölüm için test sorularını çevrimiçi olarak yapın  
[www.cs.armstrong.edu/liang/intro9e/test.html](http://www.cs.armstrong.edu/liang/intro9e/test.html).

#### Gözden Geçirme Soruları

50.1 JUnit nedir?

50.2 JUnit test çalıştırıcısı nedir?

50.3 Test sınıfı nedir? Bir test sınıfı nasıl oluşturulur?

50.4 assertTrue metodunu nasıl kullanırsınız?

50.5 assertEquals metodunu nasıl kullanırsınız?

### **Programlama Alıştırmaları**

50.1

java.lang.String sınıfındaki length, charAt, substring, \_\_\_\_\_  
ve indexOf metodlarını test etmek için bir test sınıfı yazın.

50.2

java.util.HashSet sınıfındaki add, remove, addAll, removeAll, size, isEmpty ve contains metodlarını test etmek için bir test sınıfı yazın. \_\_\_\_\_

50.3

Listing 5.7 PrimeNumberMethod.java'daki isPrime metodunu test etmek için bir test sınıfı yazın.

50.4

Listing 10.4'teki BMI sınıfındaki getBMI ve getStatus metodlarını test etmek için bir test sınıfı yazın.